
Algoritmalar Giriş

Üçüncü Baskıdan Çeviri

Thomas H. Cormen
Charles E. Leiserson
Ronald L. Rivest
Clifford Stein

Algoritmalar Giriş

Üçüncü Baskıdan Çeviri

Çeviri Editörleri

Prof. Dr. Urfat Nuriyev

Prof. Dr. Efendi Nasiboğlu

Doç. Dr. Tahsin Öner



PALME YAYINCILIK
Ankara, 2017

PALME YAYINLARI : 1237

ALGORİTMALARA GİRİŞ / Üçüncü Baskıdan Çeviri

Yazarları : Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest ve Clifford Stein
Çeviri Editörleri : Prof. Dr. Urfat Nuriyev, Prof. Dr. Efendi Nasiboğlu, Doç. Dr. Tahsin Öner
Yayın Koordinatörü : H. İbrahim Somyürek
Yayına Hazırlama : PALME Dizgi-Grafik Tasarım Birimi
Yayıncı Sertifika No : 14142
Palme Yayıncılık © 2017
ISBN : 978-605-355-649-7
Baskı : Vadi Grup Ciltevi
Basımevi Sertifika No : 26687

Kitabın Özgün Adı : INTRODUCTION TO ALGORITHMS, 3rd. Edition
Yazarları : Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest ve Clifford Stein
Yayıncı Firma : The MIT Press, 55 Hayward Street, Cambridge, Massachusetts 02142, U. S. A.
Telifi : Copyright © 2009 Massachusetts Institute of Technology. The MIT Press.
This translation of INTRODUCTION TO ALGORITHMS, 3rd. Edition is published by arrangement with The MIT Press. INTRODUCTION TO ALGORITHMS, 3rd. edition'ın çevirisi, The MIT Press ile yapılan sözleşme gereğince yayımlanmıştır.
Orijinal ISBN : 978-0-262-03384-8
Türkçe Basım : TURKISH language edition published by PALME YAYINCILIK, Copyright © 2016. All rights reserved.
TÜRKÇE baskısı, PALME YAYINCILIK tarafından yayımlanmıştır.
Telif hakkı © 2016. Telif hakları saklıdır.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, (including photocopying, recording or information storage retrieval) without permission in writing from the The MIT Press.

Bu kitabın Türkiye'deki her türlü yayın hakkı PALME Yayıncılık Ltd.Şti'ne aittir, tüm hakları saklıdır. Kitabın tamamı ya da bir kısmı 5846 sayılı yasanın hükümlerine göre, kitabı yayınlayan firmanın önceden izni olmadan elektronik, mekanik, fotokopi ya da herhangi bir kayıt sistemiyle çoğaltılamaz. Yayımlanamaz, depolanamaz.



PALME

YAYIN, DAĞITIM, PAZARLAMA, İÇ VE DIŞ TİCARET LTD. ŞTİ.

Merkez: A. Adnan Saygun Caddesi, No.: 10/1, Sıhhiye/ANKARA

Tel.: (312) 433 37 57

Faks: (312) 433 52 72

www.palmeyayinevi.com

e-mail: bilgi@palmeyayinevi.com

Ankara Şubesi : Olgunlar Sokak, No.: 4/5, Bakanlıklar/ANKARA

İzmir Şubesi : Kazım Dirlik Mahallesi, Ankara Caddesi, No.: 259/C, Bornova/İZMİR

Tel.: (312) 417 95 28

Faks: (312) 419 69 64

Tel.: (232) 343 10 77

Faks: (232) 343 10 78

İçindekiler

Çeviri Editörlerinin Önsözü	xiii
Çeviri Kurulu	xv
Önsöz	xvii

I Temel Bilgiler

Giriş	3
1 Hesaplama Algoritmalarının Rolü	5
1.1 Algoritmalar	5
1.2 Bir teknoloji olarak algoritmalar	11
2 Başlarken	16
2.1 Araya sokma sıralaması	16
2.2 Algoritmaları analiz etme	23
2.3 Algoritmaların tasarımı	29
3 Fonksiyonların Büyümesi	43
3.1 Asimptotik gösterim	43
3.2 Standart gösterimler ve ortak fonksiyonlar	53
4 Böl-ve-Yönet	65
4.1 Maksimum-altdizi problemi	68
4.2 Matris çarpımı için Strassen algoritması	75
4.3 Özyinelemeleri çözmek için yerine koyma yöntemi	83
4.4 Özyinelemeleri çözmek için özyineleme-ağacı yöntemi	88
4.5 Özyinelemeleri çözmek için ana yöntem	93
* 4.6 Ana teoremin kanıtı	97
5 Olasılıksal Analiz ve Rassal Algoritmalar	114
5.1 Kiralama problemi	114
5.2 Rassal gösterge değişkenleri	118
5.3 Rassal algoritmalar	122
* 5.4 Olasılıksal analiz ve rassal gösterge değişkenlerinin ek kullanımları	130

II Sıralama ve Sıra İstatistikleri

Giriş	147
6 Bellek Yığını Sıralaması	151
6.1 Bellek yığınları	151
6.2 Bellek yığını özelliğini sağlama	154
6.3 Bir bellek yığını oluşturma	156
6.4 Bellek yığını sıralama algoritması	159
6.5 Öncelikli kuyruklar	162
7 Hızlı Sıralama	170
7.1 Hızlı sıralamanın tanımı	170
7.2 Hızlı sıralamanın performansı	174
7.3 Hızlı sıralamanın rassal versiyonu	179
7.4 Hızlı sıralamanın analizi	180
8 Doğrusal Zamanda Sıralama	191
8.1 Sıralama için alt sınırlar	191
8.2 Sayma sıralaması	194
8.3 Taban sıralaması	197
8.4 Kova sıralaması	200
9 Medyanlar ve Sıra İstatistikleri	213
9.1 Minimum ve maksimum	214
9.2 Beklenen doğrusal zamanda seçim	215
9.3 En kötü-durumlu doğrusal zamanda seçim	220

III Veri Yapıları

Giriş	229
10 Temel Veri Yapıları	232
10.1 Yığınlar ve kuyruklar	232
10.2 Bağlı listeler	236
10.3 İşaretçileri ve nesneleri uygulama	241
10.4 Köklü ağaçların gösterimi	246
11 Çırpı Tabloları	253
11.1 Doğrudan-adres tabloları	254
11.2 Çırpı tabloları	256
11.3 Çırpı fonksiyonları	262
11.4 Açık adresleme	269
* 11.5 Kusursuz çırpılama	277

12	İkili Arama Ağaçları	286
12.1	İkili arama ağacı nedir?	286
12.2	Bir ikili arama ağacını sorgulama	289
12.3	Araya sokma ve silme	294
★ 12.4	Rassal oluşturulmuş ikili arama ağacı	299
13	Kırmızı-Siyah Ağaçlar	308
13.1	Kırmızı-siyah ağaçların özellikleri	308
13.2	Döndürmeler	312
13.3	Araya sokma	315
13.4	Silme	323
14	Veri Yapılarını Genişletme	339
14.1	Dinamik sıra istatistikleri	339
14.2	Bir veri yapısı nasıl genişletilir	345
14.3	Aralık ağaçları	348

IV İleri Tasarım ve Analiz Teknikleri

	Giriş	357
15	Dinamik Programlama	359
15.1	Çubuk kesme	360
15.2	Matris-zinciri çarpımı	370
15.3	Dinamik programlamanın elemanları	378
15.4	En uzun ortak alt dizi	390
15.5	Optimal ikili arama ağaçları	397
16	Açgözlü Algoritmalar	414
16.1	Etkinlik-seçim problemi	415
16.2	Açgözlü stratejinin elemanları	423
16.3	Huffman kodları	428
★ 16.4	Matroidler ve açgözlü yöntemler	437
★ 16.5	Bir matroid olarak görev-çizelgeleme problemi	443
17	Amortize Analiz	451
17.1	Toparlama analizi	452
17.2	Hesaplama yöntemi	456
17.3	Potansiyel yöntem	459
17.4	Dinamik tablolar	463

V İleri Veri Yapıları

Giriş	481
18 B-Ağaçları	484
18.1 B-ağaçlarının tanımı	488
18.2 B-ağaçları üzerinde temel işlemler	491
18.3 B-ağacından bir anahtar silme	499
19 Fibonacci Bellek Yığınları	505
19.1 Fibonacci bellek yığınlarının yapısı	507
19.2 Birleştirilebilir bellek yığını işlemleri	510
19.3 Anahtar azaltma ve düğüm silme	518
19.4 Maksimum dereceyi sınırlama	523
20 van Emde Boas Ağaçları	531
20.1 Ön yaklaşımlar	532
20.2 Özyinelemeli bir yapı	536
20.3 van Emde Boas ağacı	545
21 Ayırık Kümeler için Veri Yapıları	561
21.1 Ayırık-küme işlemleri	561
21.2 Ayırık-kümelerin bağlı-liste gösterimi	564
21.3 Ayırık-küme ormanları	568
* 21.4 Yol sıkıştırma ile bir rank tarafından birleşimin analizi	573

VI Çizge Algoritmaları

Giriş	587
22 Temel Çizge Algoritmaları	589
22.1 Çizgelerin gösterimi	589
22.2 Enine arama	594
22.3 Derinliğine arama	603
22.4 Topolojik sıralama	612
22.5 Güçlü bağlantılı bileşenler	615
23 Minimum Kapsayan Ağaçlar	624
23.1 Minimum kapsayan ağacı büyütme	625
23.2 Kruskal ve Prim algoritmaları	631

24	Tek-Kaynaklı En Kısa Yollar	643
24.1	Bellman-Ford algoritması	651
24.2	Yönlü çevrimsiz çizgelerde tek-kaynaklı en kısa yollar	655
24.3	Dijkstra algoritması	658
24.4	Fark kısıtları ve en kısa yollar	664
24.5	En kısa yolların özelliklerinin kanıtları	671
25	Tüm-İkililerin En Kısa Yolları	684
25.1	En kısa yollar ve matris çarpımı	686
25.2	Floyd-Warshall algoritması	693
25.3	Seyrek çizgeler için Johnson algoritması	700
26	Maksimum Akış	708
26.1	Akış ağları	709
26.2	Ford-Fulkerson yöntemi	714
26.3	Maksimum iki parçalı eşleme	732
★ 26.4	İtme-yeniden etiketleme algoritmaları	736
★ 26.5	Öne-yeniden-etiketleme algoritması	748

VII Seçme Konular

	Giriş	769
27	Çoklu İş Parçacıklı Algoritmalar	772
27.1	Dinamik çoklu iş parçalamanın temelleri	774
27.2	Çoklu iş parçacıklı matris çarpımı	792
27.3	Çoklu iş parçacıklı birleştirerek sıralama	797
28	Matris İşlemleri	813
28.1	Doğrusal denklem sistemlerini çözme	813
28.2	Matrislerin tersini alma	827
28.3	Simetrik pozitif-tanımlı matrisler ve en küçük-kareler yaklaşımı	832
29	Doğrusal Programlama	843
29.1	Standart ve gevşek biçimler	850
29.2	Problemleri doğrusal programlar olarak formüle etme	859
29.3	Simpleks algoritma	864
29.4	Dualite	879
29.5	Başlangıç temel elverişli çözüm	886

30	Polinomlar ve FFT	898
30.1	Polinomların gösterimi	900
30.2	DFT ve FFT	906
30.3	Verimli FFT uygulamaları	915
31	Sayı-Teorik Algoritmalar	926
31.1	Temel sayı-teorik kavramlar	927
31.2	En büyük ortak bölen	933
31.3	Modüler aritmetik	939
31.4	Modüler doğrusal denklemleri çözme	946
31.5	Çin kalan teoremi	950
31.6	Bir elemanın kuvvetleri	954
31.7	RSA ortak-anahtarlı şifreleme sistemi	958
★	31.8 Asallık testi	965
★	31.9 Tam sayıları çarpanlara ayırma	975
32	Katar Eşleme	985
32.1	Sade katar-eşleme algoritması	988
32.2	Rabin-Karp algoritması	990
32.3	Sonlu otomata ile katar eşleme	995
★	32.4 Knuth-Morris-Pratt algoritması	1002
33	Hesaplanabilir Geometri	1014
33.1	Doğru parçasının özellikleri	1015
33.2	Herhangi bir doğru parçası çiftinin kesişip kesişmediğinin belirlenmesi	1021
33.3	Dışbükey örtü bulma	1029
33.4	En yakın nokta çiftini bulma	1039
34	NP-Tamlık	1048
34.1	Polinom zaman	1053
34.2	Polinom-zamanlı doğrulama	1061
34.3	NP-tamlık ve indirgenebilirlik	1067
34.4	NP-tamlık kanıtları	1078
34.5	NP-tam problemler	1086
35	Yaklaşım Algoritmaları	1106
35.1	Tepe-örtüsü problemi	1108
35.2	Gezgin-satıcı problemi	1111
35.3	Küme-örtüsü problemi	1117
35.4	Rassal ve doğrusal programlama	1123
35.5	Altküme-toplam problemi	1128

VIII. Ekler: Matematiksel Altyapı

Giriş	1143
A Toplamlar	1145
A.1 Toplam formülleri ve özellikleri	1145
A.2 Toplamların sınırlanışı	1149
B Kümeler, vs.	1158
B.1 Kümeler	1158
B.2 Bağıntılar	1163
B.3 Fonksiyonlar	1166
B.4 Çizgeler	1168
B.5 Ağaçlar	1173
C Sayma ve Olasılık	1183
C.1 Sayma	1183
C.2 Olasılık	1189
C.3 Ayrık rassal değişkenler	1196
C.4 Geometrik ve binom dağılımlar	1201
* C.5 Binom dağılımının kuyrukları	1208
D Matrisler	1217
D.1 Matrisler ve matris işlemleri	1217
D.2 Temel matris özellikleri	1222

Bibliyografi 1231
İndeks 1251

Çeviri Editörlerinin Önsözü

“Bir algoritma inandırıcı olmalıdır.”

Donald Knuth, Bilgisayar Programlama Sanatı, Cilt 1, 1999

Günümüzde üniversitelerin Bilgisayar Mühendisliği, Bilgisayar Bilimleri, Matematik-Bilgisayar bölümlerinin yanı sıra benzeri programları uygulayan diğer bölümlerinde, Veri Yapıları ve Algoritmalar, Algoritma Analizi ve Tasarımı gibi dersler temel dersler olarak sayılmaktadır. Bu bölümlerin başarısı, öğrencilerinin bu derslerdeki başarısına bağlıdır. Öğrencilerin gerek lisans gerekse lisansüstü seviyedeki diğer ders ve tez çalışmalarında başarı sağlamaları, ancak algoritma analizi ile ilgili temel süreçlere hâkim olmalarıyla mümkündür.

Algoritmalar, uygulama alanı bakımından, yukarıda saydığımız gibi sadece bilişim alanıyla ilgili bölümlerle değil; elektronik, mekanik, ulaşım, üretim, planlama, sağlık vb. birçok sektörle iç içedir, yakın ilişki halindedir. Bu bakımdan, hem bu alanlarda akademik çalışmaları yürüten akademisyenler hem de kamu veya özel sektörde çalışan profesyoneller zaman zaman ihtiyaçlarını karşılamak için alanlarıyla ilgili algoritmaları öğrenme gereksinimi duymaktadırlar.

Bu noktadan baktığımızda, hemen her alandan seçilmiş algoritmalara hak ettiği derinlikte değinen Türkçe yazılmış kitapların sayısı maalesef çok azdır. Bu eksiklik, bizi, elinizdeki bu çalışmayı yapmaya yönelten en önemli sebep olmuştur. Bilgisayar Bilimleri alanında tanınmış bilim adamlarının hazırladıkları bu temel eser, bilgi teknolojileri ve algoritmalarla ilgili herkesin kütüphanesinde yer alması gereken çok önemli bir eserdir. Bu kitap, profesyoneller için her zaman yararlanacakları bir el kitabı, öğretmenler için gerek konu anlatımında gerekse soru seçimlerinde değerlendirebilecekleri bir kılavuz kitap, lisans ve lisansüstü öğrenciler için ise hem dersleri için hem de derslerinin de ötesinde yararlanabilecekleri bir ansiklopedi olarak değerlendirilebilir. Bu değerlendirme, söz konusu eksikliği gidermek için bu kitabı tercih etmemizin başlıca sebebidir.

Bu kitapta, algoritmalar, yaygın olarak kullanılan bir dilde ve başlangıç seviyesinde programlama bilgisine sahip birinin anlayabileceği bir kaba kod sistemi ile (herhangi bir programlama dilinin yapısal özelliklerinden ve gramerinden bağımsız olarak) açıklanmıştır. Teoremler ve analizlerin anlaşılmasını kolaylaştırmak için tablo, resim ve grafikler kullanılmış; yeri geldiğinde gereken matematiksel alt yapı açıklanmış; akıl yürütmeye sevk etmek için basamaklandırma ve yönlendirme yöntemleri uygulanmıştır.

Sekiz ana başlık, otuz beş esas ve dört ek bölümden oluşan bu kitapta, üniversitelerin bilgisayar bilimleri ile ilgili bölüm ve programların temel içerikleri de dahil olmak üzere, çok çeşitli alanlardan algoritmalar mevcuttur. Bunun için biyografiler dizini incelenebilir.

Verilen konuların çokluğu, bu konuların derinliği ve anlatımdaki titizlik, bu kitabın algoritma analizi ve tasarımı konusunda dünyada önde gelen kaynaklardan biri olarak tanınmasını sağlamıştır. Böylesine iddialı ve türünde tek olduğunu düşündüğümüz bu kitabı çevirerek dilimize kazandırmış olmaktan büyük bir mutluluk duyuyoruz.

Farklı üniversitelerden alanlarında uzman meslektaşlarımızdan oluşan güzel bir ekip ile yabancı dilde yazılmış bir kitabın Türkçe çevirisinin ne kadar zor olduğunu bilmenin verdiği titizlikle çalışılmıştır. Kitabın çevirisinde, olabildiğince yaygın olarak benimsenen matematik ve bilgisayar terimlerinin kullanılmasına gayret edilmiştir. Bununla birlikte, Türkçe ortak bilim dilinin gelişmesi için katkıda bulunmaya olabildiğince özen gösterilmiştir.

Hiçbir çeviri orijinalinin her nüansını yeniden üretemez. Bu bakımdan “çeviride kayıp” kavramı doğmuştur. Burada, çeviride emek harcayan meslektaşlarımızın dil ve anlatım birliğini sağlamanın ne kadar zor olduğunu değerli okurlarımızın takdirlerine bırakıyoruz.

Kitabın her bir okuyucuya yararlı olacağını umarak, yapıcı eleştirileri açık olduğumuzu saygıyla bildiriyoruz. Varlığını kaçınılmaz bulduğumuz, dizgi, bas-kı ve çeviri hatalarını bize (urfat.nuriyev@ege.edu.tr veya efendi.nasibov@deu.edu.tr veya. tahsin.oner@ege.edu.tr) bildiren herkese minnettarlığımızı sunarız.

Ayrıca, belirtmek isteriz ki, kitaptaki birçok konu yazarlar ve arkadaşları tarafından ders videoları olarak hazırlanmıştır. Bu videolardan birçoğu ise Prof. Dr. Ali Yazıcı ve Haluk Ar tarafından Türkçeye çevrilmiştir ve www.acikders.org.tr/course/view.php?id=133 adresinden izlenebilir.

Son olarak kitabın basım ve yayına hazır olmasına gösterdiği titizlik ve işbirliği için böyle bir kitabı Türkçeye kazandıran PALME YAYINCILIK yetkililerine, dizgiyi özenle yapan H. Turgut Yozgatlı’ya teşekkür ederiz.

Bu kitabı Bilgisayar Bilimleri ile uğraşan her seviyeden üniversite öğrencisine ve genç araştırmacılara mutlak suretle öneriyoruz. Kitabı bütünüyle okuduktan sonra çok şeyler kazandıracağına kesinlikle inanıyoruz.

Prof. Dr. Urfat NURİYEV (urfat.nuriyev@ege.edu.tr)

Ege Üniversitesi, Matematik Bölümü, Bilgisayar Bilimleri Anabilim Dalı, 2016

Prof. Dr. Efendi NASİBOĞLU (efendi.nasibov@deu.edu.tr)

Dokuz Eylül Üniversitesi, Bilgisayar Bilimleri Bölümü, 2016

Doç. Dr. Tahsin ÖNER (tahsin.oner@ege.edu.tr)

Ege Üniversitesi, Matematik Bölümü, 2016

ÇEVİRİ EDİTÖRLERİ		
Prof. Dr. Urfat NURİYEY,	Prof. Dr. Efendi NASİBOĞLU,	Doç. Dr. Tahsin ÖNER
REDAKSİYON KURULU		
Prof. Dr. Urfat NURİYEY	Prof. Dr. Efendi NASİBOĞLU	Doç. Dr. Tahsin ÖNER
Doç. Dr. Sedat AKLEYLEK	Yrd. Doç. Dr. Hakan KUTUCU	Yrd. Doç. Dr. Mehmet KURT
Prof. Dr. Efendi NASİBOĞLU <i>Dokuz Eyl�l �niversitesi, Bilgisayar Bilimleri B�l�m�</i>	��r. G�r. Dr. Necla KIRCALI G�RSOY <i>Ege �niversitesi</i>	
Prof. Dr. Urfat NURİYEY <i>Ege �niversitesi, Matematik B�l�m�,</i> <i>Bilgisayar Bilimleri Anabilim Dalı</i>	��r. G�r. Dr. Aşlı G�LER SERİNKEN <i>Yaşar �niversitesi</i>	
Doç. Dr. Sedat AKLEYLEK <i>Ondokuz Mayıs �niversitesi,</i> <i>Bilgisayar M�hendisliĐi B�l�m�</i>	��r. G�r. Dr. G�lşah �NER <i>Dokuz Eyl�l �niversitesi</i>	
Doç. Dr. Burak ORDİN <i>Ege �niversitesi, Matematik B�l�m�,</i> <i>Bilgisayar Bilimleri Anabilim Dalı</i>	��r. G�r. Dr. Alper VAHAPLAR <i>Dokuz Eyl�l �niversitesi, Bilgisayar Bilimleri B�l�m�</i>	
Doç. Dr. Tahsin �NER <i>Ege �niversitesi, Matematik B�l�m�</i>	��r. G�r. Dr. Zeynep �RS YORGANCI�LU <i>Yaşar �niversitesi</i>	
Doç. Dr. Burhan PEKTAŞ	Araş. G�r. Mehmet Berkehan AK�AY <i>�zmir Ekonomi �niversitesi, Yazılım M�hendisliĐi B�l�m�</i>	
Yrd. Doç. Dr. Murat Erşen BERBERLER <i>Dokuz Eyl�l �niversitesi, Bilgisayar Bilimleri B�l�m�</i>	Araş. G�r. Can ATILGAN <i>Dokuz Eyl�l �niversitesi, Bilgisayar Bilimleri B�l�m�</i>	
Yrd. Doç. Dr. Korhan G�NEL <i>Adnan Menderes �niversitesi, Matematik B�l�m�</i>	Araş. G�r. Alican DO�AN <i>Dokuz Eyl�l �niversitesi, Bilgisayar Bilimleri B�l�m�</i>	
Yrd. Doç. Dr. Arif G�RSOY <i>Ege �niversitesi, Matematik B�l�m�,</i> <i>Bilgisayar Bilimleri Anabilim Dalı</i>	Araş. G�r. G�lce HANER <i>�zmir Ekonomi �niversitesi, End�stri M�hendisliĐi B�l�m�</i>	
Yrd. Doç. Dr. Mehmet KURT	Araş. G�r. Dr. Tina BE�ERİ SEVİM <i>�zmir Y�ksek TeknoloĐi Enstit�s�, Matematik B�l�m�</i>	
Yrd. Doç. Dr. Hakan KUTUCU <i>Karab�k �niversitesi, Bilgisayar M�hendisliĐi B�l�m�</i>	Araş. G�r. İbrahim ŞENT�RK <i>Ege �niversitesi, Matematik B�l�m�</i>	
Yrd. Doç. Dr. Fidan NURİYEVA <i>Dokuz Eyl�l �niversitesi, Bilgisayar Bilimleri B�l�m�</i>	Araş. G�r. Dr. Alper �LKER <i>AĐrı İbrahim �e�en �niversitesi, Matematik B�l�m�</i>	
Yrd. Doç. Dr. Refet POLAT <i>Yaşar �niversitesi, Matematik B�l�m�</i>	Dr. Mehmet �mit G�RSOY	
Yrd. Doç. Dr. Nur UYLAŞ SATI <i>MuĐla Sıtkı Ko�man �niversitesi</i>	Duygu Selin BALLI	
Yrd. Doç. Sel�uk TOPAL <i>Bitlis Eren �niversitesi, Matematik B�l�m�</i>	TuĐ�e KATICAN	
	Elvin NASİBOV “Logo Yazılım” řirketi	

Önsöz

Algoritmalar, bilgisayarlardan daha önce de vardı. Fakat şimdi, hem bilgisayarlar var hem de daha çok algoritma var ve bu algoritmalar hesaplamanın kalbinde yer alıyor.

Bu kitap, bilgisayar algoritmalarıyla ilgili modern çalışmalar için kapsamlı bir giriş niteliğindedir. Kitapta, birçok algoritmaya, bunların tasarım ve analiz süreçlerine, yeterli derinlikte ve her seviyeden okuyucuya hitap edecek şekilde yer verilmiştir. Açıklamalar olabildiğince temel ve basit seviyede yapılmaya çalışılmıştır. Buunla birlikte, anlatımlarda, konu derinliğinden veya matematiksel titizliklerden ödün verilmemiştir.

Her bölümde bir algoritma, bu algoritmaya ait bir tasarım tekniği ile bir uygulama alanı veya ilgili bir konu yer almaktadır. Algoritmalar alışılmış bir dilde ve başlangıç düzeyinde programlama bilgisi olan birinin anlayabileceği şekilde tasarlanan bir kaba kod ile açıklanmıştır. Kitapta, bazıları birkaç parçadan oluşan ve algoritmaların nasıl çalıştığını gösteren 244 şekil yer alır. Bir tasarım kriteri olarak vurguladığımız *Verimlilik* kavramı sebebiyle, tüm algoritmaların çalışma zamanlarının titiz bir analizi de dahil edilmiştir.

İçerik öncelikle algoritma ve veri yapıları konularını öğrenmek isteyen lisans ve lisansüstü öğrencilerin kullanımına yönelik olarak hazırlanmıştır. Algoritma tasarımı kavramı, matematiksel yönüyle olduğu kadar mühendislik yönüyle de tartışıldığından, içerik, sektör çalışanlarının kişisel çalışmalarına da uygundur.

Kitap, bu üçüncü baskısında bir kez daha güncellenmiştir. Bu güncellemeler yeni bölümler eklemek, kaba kodları düzeltmek ve daha aktif bir yazma üslubu kullanmak gibi geniş bir yelpazede yapılmıştır.

Öğretmene

Çok yönlü ve eksiksiz bir kitap tasarlamaya çalıştık. Kitaptan, lisans seviyesindeki veri yapıları derslerinden lisansüstü seviyesindeki algoritma derslerine kadar bir çok ders için yararlanabilirsiniz. Bu kitabı hazırlarken, tipik tek dönemlik bir derse yetecek kadardan çok daha fazla sayıda kaynak inceledik. Bu yüzden bu kitabı, öğretmek istediğiniz konuyla ilgili en iyi dokümanları seçip alabileceğiniz bir “açık büfe” olarak düşünebilirsiniz,

Herhangi bir konuyu işlerken, o konuyla ilgili ihtiyacınız olan diğer konuları, o an kullandığınız bölümden hemen önce veya sonraki bölümlerde kolayca bulabilirsiniz. Bölümler nispeten bağımsız tasarlandığı için bir bölümün çok uzak bir konuya gereksiz ve beklenmeyen bir bağlantısı var mı diye endişe etmenize gerek yoktur. Anlatımlar, konuların doğal sınırları içinde kolaydan zora doğru sıralanmıştır. Bu bakımdan bir bölümün ilk konularını lisans dersleri için, sonraki konularını veya bölümün tamamını lisansüstü dersleri için kullanabilirsiniz.

Bu kitapta 957 alıştırmaya ve 158 problem yer alır. Her konunun sonunda alıştırmalar ve her bölümün sonunda problemler bulunmaktadır. Alıştırmalar genellikle temel bilgiyi ölçen kısa sorulardan oluşur. Bazıları basit, kendini test etme alıştırmalarıdır. Bazıları ise daha kapsamlıdır ve daha ziyade ödev için uygundur. Problemler genel olarak daha gelişmiş çalışmalar içindir ve öğrenciyi yeni konular öğrenmeye yönlendirir. Bu problemler genellikle öğrencinin çözüme ulaşmasını sağlayacak şekilde düzenlenmiş basamaklar halinde sorulardan oluşur.

Kitabın bu baskısıyla beraber, önceki baskılardaki tecrübelerimizden yararlanarak, hepsi için olmasa da bazı alıştırmaya ve problemlerin çözümlerini genel paylaşıma açtık. Web sitemizden bu çözümlere ulaşabilirsiniz: <http://mitpress.mit.edu/algorithms/>. Bu linkten ödev olarak vermeyi planladığınız alıştırmaya ve problemlerin çözümünün olup olmadığını kontrol edebileceksiniz. Bu sitedeki çözümlerin zaman içerisinde artmasını planlıyoruz. Bu yüzden bu kontrolü zaman zaman yapmanız gerekir.

Lisans öğrencilerinden daha çok lisansüstü öğrencileri için daha uygun olan konuları yıldızla(*) işaretledik. Bu yıldızlı bölümlerin daha zor olduğu anlamına gelmez ancak daha ileri matematik altyapısı gerektirebilir. Aynı şekilde, yıldızlı alıştırmalar ileri düzeyde altyapı ve ortalamadan daha yüksek bir yaratıcılık gerektirebilir.

Öğrenciye

Bu kitap ile, algoritma alanında zevk alacağınız bir başlangıç yapmanızı umuyoruz. Bu amaç doğrultusunda, kitaptaki her bir algoritmayı ilgi çekici ve anlaşılabilir bir dille ifade etmeye çalıştık. Pek bilinmeyen veya zor olarak tanınan algoritmaları kavrayabilmeniz için her bir algoritmayı basamak basamak tanımladık. Ayrıca algoritmaların analizinin daha anlaşılabilir olması için önceden bilmeniz gereken bazı matematiksel açıklamalara da yer verdik. Eğer bir konuyla ilgili önceden biraz bilgi sahibiyse o zaman giriş bölümlerini atlayarak daha gelişmiş içerikleri inceleyebilirsiniz.

Bu kitap çok geniş bir içeriğe sahip olduğu için, muhtemelen, dersleriniz bu kitabın ancak birkaç konusuyla ilgili olacaktır. Aslında bu kitabı sadece derslerinizde kullanacağınız bir ders kitabı olarak değil, aynı zamanda, matematik veya mühendislik alanındaki kariyeriniz boyunca faydalı olacak bir el kitabı olarak tasarlamaya çalıştık.

Bu kitaptan yararlanmak için ön koşullar nelerdir?

- Programlama tecrübesine sahip olmalısınız. Özellikle, özyinelemeli süreçler ile, dizi ve bağlı listeler gibi basit veri yapılarını anlamış olmalısınız.
- Matematiksel ispatlarla ve özellikle tümevarıma dayanan ispatlarla ilgili alt-yapınız olmalıdır. Kitabın çok küçük bir kısmı ise temel analiz bilgilerine dayanır. Bunun yanında, ihtiyacınız olan matematiksel teknikler I.ve VIII. Bölümlerde verilmiştir.

Problem ve alıştırmaların çözümlerini yayınlamamıza dair taleplerinizi değerlendirdik. Web sitemizde bazı problem ve alıştırmaların çözümleri yer almaktadır: <http://mitpress.mit.edu/algorithms/>. Kendi çözümlerinizi bizimkilerle karşılaştırabilirsiniz fakat bunları bize göndermemenizi rica ediyoruz.

Profesyonele

Bu kitap geniş konu yelpazesi sayesinde mükemmel bir algoritma el kitabı niteliğindedir. Her bölüm kendi içinde bağımsız olarak düzenlendiğinden özel olarak ilgilendiğiniz konuya da odaklanabilirsiniz.

Ele aldığımız algoritmaların çoğunun harika pratik uygulamaları vardır. Bu yüzden bu kitap ile çeşitli uygulama alanlarına ve bazı mühendislik sorunlarına da hitap ediyoruz. Teorik açıdan ele aldığımız bir çok algoritma için sık sık pratik uygulamalara da değindik.

Eğer herhangi bir algoritmayı uygulamak isterseniz, size düşen yalnızca verdiğimiz kaba kodun favori programlama dilinize dönüştürülmüş bir versiyonunu bulmaktır. Algoritmaları açıklamak için kullandığımız her bir kaba kodu kısa ve öz bir şekilde tasarladık. Haliyle, derleyicilerinizle ilgili karşılaştığımız hatalar, veya yazılım ve mühendislik alanındaki özel bilgiler gerektiren konular bizim ilgi alanımız değildir. Zaten algoritmaları herhangi bir programlama dilinin yapısal özelliklerini – kolaylıklarını kullanmadan, oldukça basit bir dil ile açıkladık.

Bu kitabı bir ders kapsamı dışında kullanmanız durumunda, problem ve alıştırmaların bir dersin sorumlusu tarafından kontrol edilemeyeceğini biliyoruz ve bunu anlayışla karşılıyoruz. Bu durumda en azından bazı alıştırmalar ve problemlerin çözümlerini sitemizden kontrol edebilirsiniz: <http://mitpress.mit.edu/algorithms/> Lütfen çözümlerinizi bize göndermeyiniz.

Meslektaşlarımıza

Kitapta güncel literatüre de atıf yapan geniş bir kaynakçaya yer verdik. Her bölüm o konuyla ilgili tarihi detayları ve kaynakları içeren bölüm sonu notlarıyla bitiyor. Fakat tabi ki bu bölüm sonu notları o alanla ilgili eksiksiz bir kaynak

listesi içermiyor. Bu boyuttaki bir kitap için inanılması zor olsa da, maalesef bazı sınırlamaları nedeniyle birçok ilginç algoritmaya yer veremedik.

Öğrencilerden problem ve alıştırma­ların çözümünü paylaşmamız için sayısız istek gelmesine rağmen, çözümü kolayca bulmalarındansa kendi çabalarıyla çözüme ulaşmalarını tercih ettiğimiz için, ilgili çözümleri vermemeyi bir gizlilik politikası olarak belirledik.

Üçüncü Baskıdaki Değişiklikler

Kitabın üçüncü baskısıyla ikinci baskısı arasında ne değişti? Bu değişikliğin büyüklüğü birinci ve ikinci baskısı arasındaki değişiklikler kadardır. İkinci baskıdaki değişikliklerde söylediğimiz gibi, kitaba nasıl baktığınızla ilişkili olarak ya az ya da biraz daha fazla değişiklik vardır.

İçindekiler bölümüne göz gezdirdiğimizde ikinci baskıdaki bir çok bölümün üçüncü baskıda da yer aldığını görüyoruz. İki bölüm ve bir konu çıkarmakla birlikte üç yeni bölüm ve iki yeni konu ekledik.

İlk iki baskıdaki hibrid yapıyı koruduk. Yani, sadece problem tanımını vermedik veya sadece tekniksel detayları açıklamadık, her ikisini birlikte ele aldık. Bu kitapta, teknik tabanlı olan böl-ve-yönet, dinamik programlama, aç gözlü algoritmalar, amorti edilmiş algoritmalar, NP-tamlık ve yaklaşım algoritmaları gibi konulara da yer verdik. Bununla birlikte sıralamayla ilgili kısımlar, dinamik kümeler için veri yapıları ve çizge problemleri için algoritmalar konularını ekledik. Algoritmaları tasarlamak ve analiz etmek için gereken teknikleri nasıl uygulayacağınızı bilmeniz gerektiği halde, problemler bazen hangi tekniğin çözüm için en uygun olduğunu söyler. Bu bakımdan hibrid yaklaşım önemlidir.

Üçüncü baskıdaki en belirgin değişiklikleri şöyle özetleyebiliriz:

- Van Emde Boas ağaçları ve çoklu görevli algoritmalar hakkında yeni bölümler ekledik ve matrislerle ilgili temel bilgileri kendi ek bölümleri içine yerleştirdik.
- Yineleme hakkındaki bölümü, böl-ve-yönet tekniğini daha ayrıntılı olarak kapsaması için yeniden düzenledik ve bunun ilk iki konusunda iki problem üzerinde böl-ve-yönet tekniğini uyguladık. Matris çarpımıyla ilgili Strassen Algoritmasını, matris işlemleriyle ilgili bölümden bu bölüme taşıdık.
- Nadiren öğretilen iki terimli yığınlar ve ağ sıralama ismindeki iki bölümü kaldırdık. Ağ sıralama bölümündeki anahtar fikir, yani 0-1 prensibi, bu baskıda karşılıklı değişim algoritmaları için 0-1 sıralama lemması olarak 8 ve 7. Problemlerde karşımıza çıkıyor. Fibonnaci dizisinin davranışı ise iki terimli yığınlar için bir temel teşkil etmez.

- Dinamik programlama ve aç gözlü algoritmaları revize ettik. Bu baskıdaki dinamik programlama, ikinci baskıdaki assembly-line takvimleme problemi-ne nazaran daha ilginç olan çubuk kesme problemiyle başlıyor. Ayrıca memoizasyonları ikinci baskıdakinden daha fazla vurguladık ve dinamik programlama algoritmalarının çalışma zamanını anlamak için alt problem grafinin bir gösterimini tanıttık. Aç gözlü algoritmaların girişinde, yani etkinlik seçme probleminde, ikinci baskıdakinden daha keskin bir başlangıç yaptık.
- Kırmızı-siyah ağaçları da içeren İkili arama ağaçlarından tepe silme metodu artık silinmesi istenen tepenin kesinlikle silindiğini garanti eder. İlk iki baskıda, belirli durumlarda, silme prosedürü sırasında bazı başka tepeler de silinebiliyordu. Yeni tepe silme yöntemimizde, programın diğer bileşenleri ağaçtaki tepeleri işaret etmeyi sürdürürse, yanlışlıkla da olsa silinen tepeyi tekrar göstermek gibi hatalı bir gösterim yapmayacaktır.
- Ağ akışı ile ilgili bilgilerde ayırıt üzerinde akış hakkındaki bilgileri temel alır. Bu yaklaşım ilk iki baskıdaki ağ akışına göre daha hassastır.
- Matris temelleri ve Strassen algoritması hakkındaki bilgiler diğer bölümlere taşındı. Böylece matris işlemlerini içeren bölüm ise ikinci baskıya göre daraltıldı.
- Knuth-Morris-Pratt katar eşleme algoritmasının davranışlarını yeniledik.
- Bazı hataları düzelttik. Bu hataların çoğu web sayfamızda ikinci baskı hata listesinde yayınlandı.
- Birçok istek doğrultusunda kaba kodun gösterimlerinde değişiklik yaptık. Artık C, C++, Java ve Python’da olduğu gibi atamalar için “=” ve eşitliği test etmek için “==” kullanıyoruz. Benzer şekilde, **do** ve **then**’i kaldırdık ve satır sonu yorumu olarak “//” işaretini uyarladık. Ayrıca artık obje özelliklerine erişmek için nokta gösterimini kullanıyoruz. Yani bir nesnenin x özelliğine erişmek için nesne. x ifadesini kullanacağız. Kaba kodlarımız nesneye dayalı olmaktan ziyade prosedür biçiminde tasarlandı. Başka bir deyişle, objeler üzerinde çalışan metodlar tasarlamak yerine nesneleri parametre olarak kabul eden prosedürleri çağırmayı tercih ettik.
- 100 yeni alıştırma ve 28 yeni problem ekledik. Bunun yanında kaynakçayı güncelledik ve birkaç yeni referans ekledik.
- Son olarak tüm kitabı gözden geçirerek yazımı güçlendirmek ve basitleştirmek adına cümle ve paragrafları yeniden yazdık.

Web Sitesi

Bizimle iletişime geçmek ve ek bilgi almak için web sitemiz olan <http://mitpress.mit.edu/algorithms/> adresini kullanabilirsiniz. Web sitesinde bilinen hatalara, seçilmiş alıştırma ve problemlerin çözümlerine ve profesör şakalarını açıklayan listelere, ayrıca ekleyebileceğimiz yeni içeriklere yer verilmektedir. Bunun yanında yine web sitesi hataları ve tavsiyelerinizi bize nasıl ileteceğinize dair bilgiler de içerir.

Bu kitabı nasıl yazdık

İkinci baskıda olduğu gibi, üçüncü baskı da Latex2 ϵ ile yazıldı. Times yazıtipi ile birlikte matematiksel karakterler için MathTime Pro 2 yazı tipini kullandık. Teknik desteklerinden dolayı Publish or Perish Inc.’den Micheal Spivak’a, Personal Inc.’den Lance Carnes’a ve Dartmouth College’den Tim Tregubou’a teşekkür ederiz. Önceki iki baskıda olduğu gibi dizin bölümünü C dilinde kendi yazdığımız Windex’i kullanarak derledik. Kaynakça ise BIBTEX’le meydana getirildi. Bu kitap için olan PDF dosyaları MacBook OS 10.5 te tasarlandı.

Üçüncü baskıdaki resimler MacDraw Pro ile çizildi. Resimlerdeki bazı matematiksel anlatımlar ise Latex2 ϵ için psfrog paketi ile yer aldı. Maalesef MacDraw Pro on yıldır piyasaya sürülmemiş eski bir yazılım. Fakat mutluluk verici olan OS 10.4 altında Classic ortamda çalışabilen bir çift Macintosh’a sahip olmamız. Bu yüzden MacDraw Pro genellikle çalıştırılabilir. Classic ortam altında MacDraw Pro’nun diğer bilgisayar bilimi yazımı için olan ve güzel çıktı veren (1) çizim yazılımlarına göre kullanımını daha kolay buluyoruz. Pre-Intel Macs’ın ne kadar çalışmaya devam edeceğini kim bilebilir? Eğer Apple’dan bizi dinleyen varsa: *Lütfen Mac Draw Pro’nun uyumlu OS X versiyonunu tasarlayın!*

Üçüncü baskı için teşekkürler

Yirmi yılı aşkın süredir MIT Press ile çalışıyoruz ve bu nasıl olağanüstü bir ilişkidir! Ellen Faran, Bob Prior, Ada Brunstein ve Mary Reilly’e destek ve yardımları için teşekkür ediyoruz.

Üçüncü baskı oluşturulurken Dartmouth College Bilgisayar Bilimleri Departmanı, MIT Bilgisayar Bilimi ve Yapay Zeka Laboratuvarı ve Columbia Üniversitesi

¹Mac OS X altında çalışan birçok çizim programı araştırdık fakat MacDraw Pro ile karşılaştırıldığında önemli ihmallere gördük. Resimleri farklı ve iyi bilinen bir programla oluşturmak için çalışmalarda bulunduk. Her resmi üretmenin en az beş deneme sonucunda bulduk ve sonuçlar iyi görünmedi. Bu yüzden karar MacDraw Pro’yu eski Macintoshlar altında çalıştırmak oldu.

tesi Endüstri Mühendisliği ve Araştırma Departmanında çalışılıyordu ve coğrafi olarak dağılmıştık. Saygın üniversitelerimize ve meslektaşlarımıza sağladıkları destek ve teşvikten dolayı teşekkür ederiz.

Julie Sussman, P.P.A bir kez daha teknik düzeltmen olarak bizleri kurtardı. Yine ve bir kez daha yapmaktan kaçındığımız hataları Julie yakaladı. Kendisi ayrıca bazı yerlerde, yaptığımız anlatımları geliştirmemize yardım etti. Eğer teknik düzeltmen için Hall of Fame varsa Julie kesinlikle birincidir. Kendisi olağanüstü başarılıdır. Teşekkürler, teşekkürler, teşekkürler Julie! Priya Natorajan da baskıdan önce düzeltebileceğimiz bazı hatalar buldu. Eğer bazı hatalar varsa (şüphesiz olur) yazarın sorumluluğudur (Muhtemelen Julie okuduktan sonra meydana gelmiştir.).

Van Emde Boas ağaçlarını ele alış tarzı Micheal Beudeki'den etkilenen Eric Demaine'in notlarından türetildi. Ayrıca Javed Aslam, Bradley Kuszmaul ve Hui Zha'dan bu baskı için bazı düşünceler edindik.

Çoklu görev bölümü Harald Prokop'un orijinal notları baz alınarak yazıldı. İçerik MIT'deki Cilk Projesinde çalışan Bradley Kuszmaul ve Mattes Frigo'yu içeren birkaç kişiden etkilenecek yazıldı. Çoklugu görev kaba kodunun tasarımının ilhamı C için MIT'deki Cilk uzantısında ve C++ için CilkArt's Cilk++ uzantısından alındı.

Bunların yanı sıra hataları rapor eden ve kitabın gelişimi için önerilerde bulunan birçok okuyucuya da teşekkür ediyoruz. Rapor edilen tüm hataları düzelttik ve bir çok tavsiyeyi yapabildiğimiz kadarıyla değerlendirdik. Katkıda bulunanların sayısının oldukça artması sevindirici fakat isimlerini listelememiz pek mümkün değil.

Son olarak eşlerimiz—Nicole Cormen, Wendy Leiserson, Gail Rivest ve Rebecca Ivry—ve çocuklarımız—Ricky, Will, Debley ve Katie Leiserson; Alex ve Christopher Rivest; ve Molly, Nooh ve Benjamin Stein'a kitabı hazırlarken ki destek ve sevgileri için teşekkür ediyoruz. Ailelerimizin sabır ve teşviği bu projeyi olası kıldı. Bu kitabı şefkatle onlara adıyoruz.

THOMAS H. CORMEN
CHARLES E. LEISERSON
RONALD L. RIVEST
CLIFFORD STEIN

*Lebanon, New Hampshire
Cambridge, Massachusetts
Cambridge, Massachusetts
New York, New York*

Şubat 2009

Algoritmalar Giriş

Üçüncü Baskı



I Temel Bilgiler

Giriş

Bu bölüm, sizi, algoritmaların analizi ve tasarımı hakkında düşünmeye sevk edecektir. Bu bölümde, algoritmaları, bu kitap boyunca kullanacağımız tasarım stratejilerinin bazılarını ve algoritma analizinde kullanılan temel fikirlerin birkaçını ifade etmek için yumuşak bir giriş hedeflenmektedir. Kitabın sonraki bölümleri bu taban üzerine inşa edilecektir.

1. Bölüm algoritmalara ve modern hesaplama sistemlerindeki konumlarına genel bir bakış sunmaktadır. Bu bölüm algoritmanın ne olduğunu tanımlar ve bazı örnekler listeler. Bu bölüm aynı zamanda algoritmaları hızlı donanım, grafiksel kullanıcı arayüzü, nesne yönelimli sistemler ve ağlar gibi teknolojilerin yanında bir teknoloji olarak da düşünmemiz gerektiğini ortaya koymaktadır.

2. Bölümde ilk algoritmalarımızla, n sayıdan oluşan bir diziyi sıralama problemini çözen algoritmalarla tanışacağız. Bu algoritmalar, seçeceğimiz bir dilde uygulayabilmeniz için, doğrudan herhangi bir programlama diline çevrilemesine rağmen yöntemin yapısını açık ve yeterli bir biçimde gösteren kabakod (pseudocode) formatında yazılmıştır. Burada artan bir yaklaşım kullanan Araya sokma Sıralaması (insertion sort) ve “böl-ve-yönet” (divide-and-conquer) olarak bilinen özyinelemeli bir teknik kullanan Birleştirecek Sıralama (merge sort) tanıtılmıştır. Her biri için gereken zaman n değeri ile artsa da bu artış oranı iki algoritma arasında farklılık gösterir. Bu çalışma zamanlarını 2. Bölümde belirleyeceğiz ve bu zamanı açıklamak için yararlı bir notasyon geliştireceğiz.

3. Bölüm, “asimptotik notasyon” olarak isimlendirdiğimiz bu notasyonu ayrıntılı bir biçimde tanımlıyor. Bu bölüm algoritma çalışma zamanlarını alttan ve/veya üstten sınırlama için kullandığımız çeşitli asimptotik notasyonları tanımlayarak başlıyor. 3. Bölümün tamamı esas itibariyle yeni matematiksel bilgiler öğretmekten ziyade, bu kitaptaki notasyon eşleşmelerini kullanmanızı sağlama amacı taşıyan bir matematiksel notasyon sunumudur.

4. Bölüm, 2. Bölüm de tanıtılan böl-ve-yönet yöntemini daha ayrıntılı araştırır. Bu bölüm iki kare matrisin çarpımı ile ilgili Strassen'in sürpriz yöntemini de içeren böl-ve-yönet algoritmalarının ek örneklerini sunar. 4. Bölüm özyinelemeli de algotimaların çalışma zamanını tanımlamada yardımcı olan özyinelemeli çözüm yöntemlerini içerir. “Ana yöntem” böl-ve-yönet algoritmalarından doğan özyinelemeli problemleri çözmek için çok sık kullandığımız güçlü bir tekniktir. 4.Bölümün geniş bir kısmı “ana yöntem”in doğruluğunu kanıtlamak için ayrıldıysa da bu kanıtı ihmal edip yine de yöntemi kullanabilirsiniz.

5. Bölüm olasılıksal analiz ve rassal algoritmaları içerir. Olasılıksal analizi, genellikle, algoritmaların çalışma zamanının karakteristik bir olasılık dağılımına bağlı olduğu durumlarda, aynı boyutlu farklı girdiler için farklı olabilen çalışma zamanlarını belirlemek için kullanırız. Bazı durumlarda, girdilerin bilinen bir olasılık dağılımına uyduğunu kabul ederiz ve böylece muhtemel tüm girdiler için ortalama çalışma zamanını buluruz. Başka bir durum da, olasılık dağılımının girdilerden değil algoritmanın çalışması esnasında üretilen rastgele seçimlerden gelmesidir. Davranış biçimleri sadece girdileri tarafından değil, aynı zamanda, rassal-sayı üreteçleri tarafından üretilen değerler ile de belirlenen algoritmalara “rassal algoritmalar” (randomized algorithm) denir. Rassal algoritmaları, kötü performansla neden olan belirsiz girdilerin olasılık dağılımını teyit etmek için ya da kısıtlı bir alanda yanlış sonuçlar üretmesine izin verilen algoritmaların hata oranını sınırlamak için kullanabiliriz.

Ek A-D bu kitabı okurken faydalanacağınız diğer matematiksel bilgileri içerir. Muhtemelen ek bölümlerdeki bilgileri bu kitabı okumadan önce de (özel tanımlama ve gösterim biçimlerini bazı durumlarda geçmişte gördüğünüz teamüllerden farklı olarak kullanmamıza rağmen) gördünüz, bu yüzden ekleri bir referans kaynağı olarak düşünebilirsiniz. Diğer taraftan, birinci bölümdeki bir çok bilgiyi muhtemelen daha önceden görmediniz. Birinci bölümdeki tüm bölümler ve ekler ders dokümanı üslubuyla yazılmıştır.

Algoritma nedir? Algoritma öğrenmek neden değerlidir? Bilgisayarlarda kullanılan diğer teknolojilerle ilgili algoritmaların rolü nedir? Bu bölümde bu soruları cevaplayacağız.

1.1 Algoritmalar

Kabaca, **algoritma** bazı değerleri ya da değer kümelerini **girdi** olarak alan ve bazı değerleri ya da değer kümelerini **çıkı** olarak üreten iyi tanımlanmış bir hesaplama sürecidir. Bu sebeple algoritma, girdileri çıktılarına dönüştüren bir hesaplama adımları sıralamasıdır.

Algoritmayı, aynı zamanda iyi belirlenmiş **hesaplama problemi**ni çözmek için bir araç olarak da görebiliriz. Problemin açıklaması genel anlamda istenen girdi/çıkı ilişkisini belirtir. Algoritma girdi/çıkı ilişkisini sağlamak için özel bir hesaplama süreci tanımlar.

Örneğin, bir sayı dizisinin azalmayan bir sırasına gereksinim duyabiliriz. Bu problem uygulamada sık sık ortaya çıkar ve pek çok standart tasarım tekniklerine ve analiz araçlarına giriş yapmak için verimli bir zemin sağlar. **Sıralama problemi**ni kuramsal olarak şöyle tanımlarız:

Girdi: n adet sayıdan oluşan bir dizi: $\langle a_1, a_2, \dots, a_n \rangle$

Çıkı: Girdi dizisinin $a'_1 \leq a'_2 \leq \dots \leq a'_n$ olacak biçimde (yeniden sıralanmış) bir $\langle a'_1, a'_2, \dots, a'_n \rangle$ permütasyonu.

Örneğin, verilen bir $\langle 31, 41, 59, 26, 41, 58 \rangle$ girdi dizisi için sıralama algoritması $\langle 26, 31, 41, 41, 58, 59 \rangle$ çıktısı döndürür. Bu tür bir girdi dizisine sıralama probleminin bir **örneği** denir. Genel olarak, **problemin bir örneği** problemin çözümünün hesaplanması için ihtiyaç duyulan bir (problemin açıklamasında ifade edilen kısıtlar her neyse, onu sağlayan) girdi içerir.

Bilgisayar bilimlerinde bir çok programın ara adım olarak kullanması nedeniyle sıralama işlemi temel bir işlemdir. Sonuç olarak, hali hazırda kullanılan oldukça fazla sayıda iyi sıralama algoritması vardır. Sıralanan elemanların—diğer faktörlerin yanı sıra—sayısına, daha önceden sıralanmış elemanların kapsamına, elemanların değerleri ile ilgili muhtemel kısıtlamalara, bilgisayarın mimarisine ve kullanılacak ana hafıza, harddisk ve hatta teypler gibi depolama araçlarına bağlı olarak, verilen bir uygulama için hangi algoritma daha iyidir?

Bir algoritma eğer her girdi örneği için doğru çıktı ile sonlanıyorsa **doğru** olarak isimlendirilir. Doğru algoritma verilen bir hesaplama problemini **çözer** deriz. Yanlış bir algoritma bazı girdi örneklerinde her zaman sonlanmayabilir yada yanlış bir cevap ile sonlanabilir. Beklenenin aksine, eğer hata oranını kontrol edebilirsek yanlış algoritmalar bazen yararlı olabilirler. Kontrol edilebilir hata oranına sahip algoritmaların bir örneğini 31. Bölümde büyük asal sayıların bulma algoritmalarını incelerken göreceğiz. Tabi ki bununla birlikte, genellikle, biz doğru algoritmalarla ilgileniyor olacağız.

İngilizcede algoritma kelimesi bir bilgisayar programı hatta bir donanım tasarımı olarak bile tarif edilebilir. Gereken tek şey, bu tarifi izlenecek hesaplama işleminin kesin bir tanımını sunmak zorunda olduğudur.

Ne tür problemler algoritmalarla çözülür?

Sıralama, algoritmaların geliştirildiği tek hesaplama problemi değildir. (Muhtemelen bu kitabın boyutunu gördüğünüz zaman böyle sandınız). Algoritmaların pek çok pratik uygulaması vardır ve aşağıdaki gibi örnekleri içerir:

- İnsan Gen Projesi (The Human Genome Project) ile insan DNA'sındaki 100.000 genin hepsinin belirlenmesi, insan DNA'sının yapıldığı 3 milyar kimyasal baz çiftinin sırasının tanımlanması, bu bilgilerin veritabanında depolanması ve veri analizi için araçların geliştirilmesi amaçlarına yönelik önemli gelişmeler sağlamıştır. Bu adımların her biri özel algoritmalar gerektirir. Bahsedilen bu tür problemlerin çözümlerinin bu kitabın kapsamı dışında olmasına rağmen, bu biyolojik problemleri çözen bazı yöntemler bu kitabın çeşitli bölümlerinde yer alan prensipleri kullanır. Neticede bilim adamları kaynakları verimli kullanarak görevlerini (iş parçalarını) gerçekleştirmek ister. Laboratuvar tekniklerinden ortaya çıkarılabilecek daha fazla bilgi, hem insan hem de makine için zaman ve para tasarrufu demektir.
- İnternet dünyanın her yerindeki insanların büyük miktarlardaki bilgiye hızlı bir biçimde erişmesine ve onları kullanmasına olanak sağlıyor. İnternetteki siteler ancak hızlı algoritmaların yardımıyla bu büyük hacimdeki verileri yönetebilir ve yönlendirebilirler. Verinin seyahat edeceği iyi güzergahları bulmak (bu tür problemleri çözme teknikleri 24. Bölüm de yer almaktadır) ve dağılmış bilgiler arasında sayfayı hızlıca bulmak için bir arama motoru kullanmak

(İlgili teknikler 11. ve 12. Bölümdedir) algoritmaların kullanımını önemli kılan problem örneklerindendir.

- Elektronik ticaret, ürün ve hizmetlerin elektronik olarak taşınmasına ve el değiştirmesine olanak sağlar, ve bu işlemler kredi kartı numarası, parolası ve banka bilgileri gibi kişisel bilgilerin güvenliğine bağlıdır. Elektronik ticarete sayısal algoritma ve sayılar teorisi tabanlı açık-anahtar kriptografisi ve dijital imza (31. Bölümde) gibi temel teknikler kullanılır.
- Fabrikalar ve diğer ticari şirketler sık sık küçük ölçekli kaynaklarını en iyi biçimde dağıtmaya ihtiyaç duyarlar. Bir petrol şirketi beklenen kârını maksimize etmek için istasyonlarının nerede yer alması gerektiğini bilmek isteyebilir. Politikada bir aday seçimi kazanma şansını maksimize etmek için nerede reklam kampanyası için para harcayacağını belirlemek isteyebilir. Bir havayolu şirketi mümkün olan en az maliyetle, her uçuşu kapsayacak ve ekibin çalışma planlamasıyla ilgili yasal düzenlemeleri yerine getirecek bir biçimde çalışanlarını uçuşlara atamak isteyebilir. İnternet servis sağlayıcısı müşterilerine daha iyi hizmet vermek için ek kaynaklarını nerede konumlandıracağını bilmek isteyebilir. Tüm bu problem örnekleri 29. Bölümde üzerinde durduğumuz Lineer Programlama ile çözülebilir.

Bu örneklerin bazı detayları bu kitabın konsepti dışında olmasına rağmen, bu problemlere ve problem alanlarına uygulamak için gereken teknikleri verdik. Aynı zamanda aşağıda özetlenen bazı özel problemlerin nasıl çözüleceğini de ifade ettik.

- Her bir bitişik kavşak çifti arasındaki uzaklıkları işaretlenmiş olarak verilen bir yol haritası olsun ve bir kavşaktan diğerine en kısa güzergahı belirlemek isteyelim. Eğer güzergahların birbirlerini çaprazlamasına (kesmesine) engel olsa bile muhtemel güzergahların sayısı çok fazla olabilir. Muhtemel tüm güzergahlardan hangisinin en kısa yol olduğunu nasıl seçeriz? Burada, yol haritasını bir çizge (Altıncı kısım ve Ek B de değinilmiştir) olarak modelleriz (gerçek yolların bir modeli) ve çizgedeki bir tepeden diğerine en kısa yolu bulmaya çalışırız. Bu problemi verimli bir şekilde nasıl çözeceğimizi 24. Bölümde göreceğiz.
- $X = \langle x_1, x_2, \dots, x_m \rangle$ ve $Y = \langle y_1, y_2, \dots, y_n \rangle$ sıralı semboller dizisi verilsin ve X ve Y nin en uzun ortak bir altdizisini bulmak isteyelim. X in bir altdizisi bazı elemanları (veya muhtemelen hepsi ya da hiçbiri) silinmiş bir X dizisidir. Örneğin, $\langle A, B, C, D, E, F, G \rangle$ nin bir altdizisi $\langle B, C, E, G \rangle$ olabilir. X ve Y nin en uzun ortak alt dizisinin uzunluğu bu iki dizinin benzerliği hakkında bir ölçüm verir. Örneğin, bu iki dizi DNA sarmalındaki baz çiftleriye o zaman bunların uzun bir ortak altdiziye sahip olduklarında benzer olduklarını düşünebiliriz. Eğer X m adet sembole ve Y de n adet sembole sahipse X ve Y sırasıyla 2^m ve 2^n altkümeye sahiptirler. X ve Y nin muhtemel tüm alt kümelerini seçmek

ve onları eşlemek m ve n çok küçük değillerse ulaşamaz derecede uzun bir zaman alır. 15. Bölümde bu problemi daha verimli bir biçimde çözmek için kullanılan ve dinamik programlama olarak bilinen genel bir tekniği nasıl kullanacağımızı göreceğiz.

- Bize mekanik bir tasarımın her bir parçasının diğer parçaların bir kısmını içerebildiği bir parçalar kütüphanesi verilsin, ve bu parçaları, her parça kullanıldığı herhangi bir parçadan önce görünecek bir sırada listelemeye ihtiyacımız olsun. Eğer tasarım n parçadan oluşursa o zaman $n!$ (n faktöriyel) tane muhtemel sıralamamız vardır. Faktöriyel fonksiyonu üslü fonksiyonlardan bile daha hızlı büyüdüğünden uygulanabilir (gerçekçi) bir biçimde, muhtemel tüm sıralamaları üretemeyiz ve bu sıralamalar içinde, her parçanın kullanıldığı parçalardan önce geldiği sıralamaları seçemeyiz. (Tabi az sayıda parça olmadığı durumda) Bu problem topolojik sıralamanın bir örneğidir ve bu tür problemleri verimli bir şekilde nasıl çözeceğimizi 22. Bölümde göreceğiz.
- Düzlemde n tane nokta verilsin ve bu noktaların dışbükey bir örtüsünü (convex hull) bulmak isteyelim. Dışbükey örtü, bu noktaları içeren en küçük çokgendir. Sezgisel olarak, her bir noktanın bir tahtaya tam çakılmamış bir çiviyle temsil edildiğini düşünebiliriz. Dışbükey örtü, tüm çivileri çevreleyen sıkı bir lastik bant olarak temsil edilebilir. Lastik bandın dönüş yaptığı bölgedeki her çivi dışbükey örtünün köşeleridir. (Örnek olarak sayfa 1029 da Resim 33.6'yı inceleyiniz.) Noktaların 2" tane alt kümesinden herhangi biri dışbükey kabuk olabilir. Dışbükey örtünün köşelerinin bilinmesi yeterli değildir, ayrıca köşelerin hangi sırada olduklarını da bilmeye ihtiyaç duyarız. Bu nedenle dışbükey örtünün köşeleri için bir kaç seçeneğimiz vardır. 33. Bölümde dışbükey örtü bulmak için iki iyi yöntem verilmiştir.

Bu listeler ayrıntıdan çok uzaktır (Bu kitabın ağırlığından tahmin edebildiğiniz gibi) fakat bir çok ilginç algoritma için ortak iki özellik sergilerler:

1. Ezici çoğunluğu problemi manuel olarak çözmeyen bir çok alternatif çözüme sahiptirler. Her hangi bir çözümü veya en iyisini bulmak oldukça zor sunulabilir.
2. Güncel uygulamalara sahiptirler. Aşağıda listelenen problemler en kısa yol bulmanın kullanıldığı en basit örneklerdir: Karayolu veya demiryolu taşımacılığı yapan bir nakliye firması ekonomik açıdan en kısa yol bulma problemi ile ilgilidir çünkü daha kısa yol almak daha düşük iş gücü ve yakıt maliyeti ile sonuçlanır. Yada internette bir rotalama merkezi ağ üzerinde mesajları daha hızlı yönlendirmek için en kısa yolu bulmaya ihtiyaç duyabilir. Yada bir kişi New York'tan Boston'a bir web sitesinden uygun yol tariflerini ararak gitmek isteyebilir, yada bir bayan sürüş esnasında GPS kullanmak isteyebilir.

Algoritmalarla çözülen her problem kolaylıkla tanımlanmış alternatif çözümlere sahip değildir. Örneğin, varsayalım ki bir sinyal örneğini temsil eden sayısal değerler kümesi verilsin ve bu örneklerin Ayırık Fourier dönüşümlerini hesaplamak isteyelim. Ayırık Fourier dönüşümü bir sayısal katsayılar kümesi üreterek zaman kümesini frekans kümesine çevirir, böylece örneklendirilmiş sinyal üzerinde çeşitli frekansların gücünü belirleyebiliriz. Ayırık Fourier dönüşümü sinyal işleminin yanında, veri sıkıştırma, büyük polinomları ve tam sayıları çarpma konularında da uygulama alanına sahiptir. 30. Bölümde bu problem için verimli bir Hızlı Fourier Dönüşümü (Fast Fourier Transformation – FFT) algoritması verilmiştir. Bu bölümde ayrıca FFT hesaplamak için bir donanım devresi de çizilmiştir.

Veri Yapıları

Bu kitap aynı zamanda çeşitli veri yapılarını içerir. *Veri yapısı*, veriye erişimi ve üzerinde yapılacak değişiklikleri kolaylaştırmayı amaçlayan veri depolama ve düzenleme yöntemidir. Tüm amaçlar için geçerli tek bir veri yapısı yoktur, bu yüzden veri yapılarının bir çoğunun kabiliyet ve sınırlarını bilmek önemlidir.

Teknik

Bu kitabı algoritmalar için bir “başvuru kitabı” olarak kullanabilirsiniz, ancak, yayımlanmış bir algoritmayı kolayca bulamayacağınız bir problemle karşılaştığınız anlar olabilir (Örneğin, bu kitaptaki bazı alıştırmalar ve problemler). Bu kitap size, kendi yöntemlerinizi geliştirebilmeniz için, yönteminizin doğru cevabı verdiğini göstereceğiniz ve verimliliğini açıklayacağınız algoritma tasarımı ve analizi tekniklerini öğretecektir. Farklı bölümlerde algoritmik problem çözümünün farklı yönlerine yer verilmektedir. Bazı bölümlerde, örneğin 9. Bölümdeki medyan bulma ve diğer istatistik değerler bulma; 23. Bölümdeki en küçük dallanmış ağaç hesaplama; 26. Bölümdeki ağdaki en büyük akışı hesaplama gibi özel problemlere yer verilmiştir. Diğer bölümlerde de örneğin 4. Bölümdeki böl-ve-yönet, 15. Bölümdeki dinamik programlama ve 17. Bölümdeki amortize analiz gibi, genel tekniklere yer verilmiştir.

Zor Problemler

Bu kitabın büyük bir bölümü verimli algoritmalarla ilgilidir. Verimliliğin genel ölçüsü hızdır, yani sonucu üretmek için gereken süredir. Bununla birlikte, verimli çözümü bilinmeyen problemler de vardır. 34. bölümde NP-tam olarak bilinen bu ilginç problemlerin bir bölümü çalışılmıştır.

NP-tam problemler neden ilginçtir? Birincisi, NP-tam problem için verimli bir algoritma hiç bulunmamasına rağmen böyle bir algoritma bulunamayacağı da kanıtlanamamıştır. Başka bir deyişle, NP-tam problemler için verimli bir algorit-

ma olup olmayacağını kimse bilmiyor. İkincisi, NP-tam problemler kümesi, “onlardan herhangi birisi için verimli bir algoritma varsa o zaman hepsi için verimli bir algoritma vardır” gibi dikkat çekici bir özelliğe sahiptir. NP-tam problemler arasındaki bu ilişki boşuna umutlandıran verimli çözüm eksikliğine neden olur. Üçüncüsü, bazı NP-tam problemler verimli algoritmalarını bildiğimiz problemlerle benzerdir fakat aynı değildir. Bilgisayar bilimciler, problemin ifadesinde, çözüm için en iyileyen algoritmanın verimliliğine dönüşecek kadar büyük bir değişikliğe neden olabilecek küçük değişikliklerin nasıl olması gerektiği ile ilgilenirler.

Sık sık sürpriz bir biçimde ortaya çıkan gerçek yaşam uygulamaları nedeniyle, NP-tam problemler hakkında bilgi sahibi olmanız gerekir. Eğer NP-tam bir problem için verimli bir algoritma üretmek üzere size başvurulursa, muhtemelen sonuçsuz bir arayış içinde çok fazla zaman harcarsınız. Eğer problemin NP-tam olduğunu gösterirseniz, verimli bir algoritma geliştirmek için zaman harcamak yerine iyi ama muhtemelen en iyi olmayan çözümler önerirsiniz.

Somut bir örnek olark, merkez bir deposu olan bir dağıtım şirketi düşünelim. Her gün, ürünler depoda dağıtım araçlarına yüklensin ve farklı adreslere dağıtım yapılsın. Gün sonunda, her araç sonraki gün için yüklemeye hazır biçimde geri gelmek zorunda olsun. Şirket, maliyetleri azaltmak amacıyla her bir araç için, dağıtım duraklarının sıralamasını en düşük ortalama seyahat mesafesini sağlayacak biçimde seçmek istesin. Bu problem iyi bilinen “Gezgin Satıcı Problemi” (Traveling Salesman Problem) dir ve NP-tam dır. Verimli bir algoritmaya sahip değildir. Bazı kabuller altında ortalama uzaklığı, optimal olanın çok az üzerinde veren verimli algoritmalar biliyoruz. 35. Bölüm bu tür “Yaklaşım algoritmaları”-nı inceliyor.

Paralellik

Uzun yıllar sabit bir oranda artan işlemci saat hızlarını itimat edebildik. Bununla birlikte fiziksel sınırlar, giderek artan saat hızları için köklü bir engel teşkil eder, çipler güç yoğunluğunun saat hızıyla süper doğrusal artması nedeniyle saat hızları yeterli yüksekliğe erişir erişmez hafif bir risk ile çalışır. Saniye başına daha fazla hesaplama gerçekleştirmek için, çipler sadece 1 değil bir çok “çekirdek” içerir. Bu çok çekirdekli bilgisayarları bir kaç tanesi sıralı tek çipli bilgisayarlara, diğer bir deyişle “paralel bilgisayar”ların bir tipine benzetebiliriz. Çok çekirdekli bilgisayarlardan en iyi performansı elde etmek amacıyla, algoritmaları, paralellik anlayışı içinde tasarlamak gerekir. 27. Bölüm çok çekirdekliyi avantajlı kılan “çoklu iş parçacığı” algoritmaları için bir model tanıtmaktadır. Bu model, teorik bakış açısına göre bazı üstünlüklere sahiptir ve satranç oyunu programı gibi bir çok bilgisayar programının temellerini düzenler.

Alıřtırmalar**1.1-1**

Sıralama gerektiren veya dıřbükey örtü hesaplaması gerektiren gerek-dünya örneęi veriniz.

1.1-2

Hızdan bařka, gerek dünyada bařka ne tür verimlilik ölçümleri kullanılabilir?

1.1-3

Daha önce gördüğünüz bir veri yapısı seçiniz ve kabiliyet ve sınırlarını tartışınız.

1.1-4

En kısa yol ile gezgin satıcı problemlerinin benzer ve farklı yönleri nelerdir?

1.1-5

Sadece en iyi çözümün yapılacağı bir gerek dünya örneęi söyleyiniz. Daha sonra “yaklařık” en iyi çözümün yeterli olduęu bir gerek dünya örneęi söyleyiniz.

1.2 Bir teknoloji olarak algoritmalar

Varsayalım ki bilgisayarlar sonsuz hızlı ve hafıza sorunları yok. Algoritma alışmak için herhangi bir sebebiniz var mı? Çözüm yönteminizin sonlandığını ve doğru cevabı verdiğini göstermeyi istemekten bařka bir sebebiniz yoksa bile cevap evettir.

Eęer bilgisayarlar sonsuz hızlıysa bir problemi özmek için herhangi bir doğru yöntem üretilebilir. Muhtemelen uygulamanızın iyi yazılım mühendisliği pratięinin sınırları içinde olmasını isteyebilirdiniz (Örneęin, uygulamanız iyi tasarlanmış ve belgelendirilmiş olmalı) fakat çoęu zaman uygulama için en kolay yöntemi kullanırdınız.

Tabi ki bilgisayarlar hızlı olabilir ama sonsuz hızda olmayacak. Ve hafıza ucuz olabilir ama bedava olmayacak. Hesaplama zamanı ve hafıza boyutu bu yüzden sınırlandırılmış bir kaynak ile gerekleştirilecek. Bu kaynakları akıllıca kullanmalısınız. Bu konuda zaman ve hafıza boyutu açısından verimli olan algoritmalar size yardımcı olacak.

Verimlilik

Aynı problemi çözmek için verimlilikleri bakımından önemli ölçüde farklı olan çeşitli algoritmalar tasarlanmıştır. Bu farklılıklar donanım ve yazılımla ilgili farklılıklardan daha önemli olabilir.

Örnek olarak, 2. Bölümde, sıralama için iki algoritma göreceğiz. İlkinde, **araya sokma sıralaması** olarak bilinen yöntem ile n elemanlı bir sıralama işlemi, n ye bağlı olmayan c_1 sabiti için kabaca $c_1 \cdot n^2$ kadar bir süre alır. Burada süre kabaca n^2 ile orantılıdır. İkincisinde, **birleştirerek sıralama**, lg_2 ve c_2 , n ye bağlı olmayan bir sabiti göstermek üzere, kabaca $c_2 \cdot n \cdot lg n$ kadar bir süre alır. Araya sokma sıralaması karakteristik olarak birleştirerek sıralamaya göre daha küçük bir sabit çarpana sahiptir, yani $c_1 < c_2$ dir. Sabit çarpanların çalışma zamanına etkisinin girdinin n boyutuna göre çok daha az olduğunu daha sonra göreceğiz. Araya sokma sıralamasının çalışma zamanını $c_1 \cdot n \cdot n$ olarak ve birleştirerek sıralamanın çalışma zamanını $c_2 \cdot n \cdot lg n$ olarak yazalım. Bu durumda araya sokma sıralamasının n çarpanına, birleştirerek sıralamanın ise daha küçük olan $lg n$ çarpanına sahip olduğunu görürüz. (Örneğin, $n=1000$ iken, $lg n$ yaklaşık olarak 10 dur, ve n 1 milyona eşitken $log n$ yaklaşık olarak 20 dir.) Küçük boyutlu girdiler için araya sokma sıralaması birleştirerek sıralamaya göre genellikle daha hızlı çalışmasına rağmen, n girdi boyutu yeterince büyük olduğu zaman, birleştirerek sıralamanın n e karşı $lg n$ avantajı farklı sabit çarpanlar için tercih edilecektir. c_1 in c_2 den ne kadar küçük olduğunun bir önemi yoktur, belli bir girdi sayısının ötesinde, birleştirerek sıralama daima daha hızlı olacaktır.

Somut bir örnek vermek gerekirse, araya sokma sıralaması yapan daha hızlı bir bilgisayara (bilgisayar A) karşı birleştirerek sıralama yapan daha yavaş bir bilgisayarı (bilgisayar B) düşünelim. Her biri, 10 milyon sayı barındıran bir diziyi sıralamak zorunda olsun. (10 milyon sayı çok fazla gibi görünse de, sayılar 8byte tam sayı olursa girdiler yaklaşık 80 megabyte yer kaplar ki pahalı olmayan bir bir dizüstü bilgisayarın hafızası bile bundan bir çok kat fazladır.) Varsayalım, A bilgisayarı saniyede 10 milyar komut(kitabın yazımı zamanındaki herhangi tek sıralı bir bilgisayardan daha hızlı bir değer) ve B bilgisayarı saniyede sadece 10 milyon komut gerçekleştirsin, o halde ham işlem gücünde A bilgisayarı B bilgisayarındaki 1000 kat hızlıdır. Farkı daha dramatik yapmak için, dünyanın en kurnaz programcısının A bilgisayarı için makina dilinde araya sokma sıralamasını kodladığını ve sonuçta n tane sayı sıralaması için $2n^2$ komut gerektiğini düşünelim. Ayrıca ortalama bir programcının birleştirerek sıralamayı uyguladığını, verimsiz bir derleyicide yüksek seviyeli bir dil kullandığını, sonuçlanan kodun $50n \lg n$ komut sürdüğünü varsayalım. 10 milyon sayısı sıralamak için B bilgisayarında

$$\frac{50 \cdot 10^7 \log 10^7 \text{ komut}}{10^7 \text{ saniye başına komut}} \approx 1163 \text{ saniye} \quad (20 \text{ dakikadan daha az) bir zaman gerekirken A bilgisayarı}$$

$$\frac{2 \cdot (10^7)^2 \text{ komut}}{10^{10} \text{ saniye başına komut}} = 20.000 \text{ saniye (5.5 saatten daha fazla)}$$

bir zaman gerekir.

Çalışma zamanı daha yavaş gelişen bir algoritma kullanarak, hatta daha zayıf bir derleyiciyle, B bilgisayarı A bilgisayarından 17 kez daha hızlı çalışır! Birleştirerek sıralamanın avantajı 100 milyon sayıyı sıraladığımız zaman daha da belirgindir: Araya sokma sıralaması 23 günden daha fazla zaman alırken, birleştirerek sıralama 4 saatten daha az sürer. Genelde, problem boyutu artarken birleştirerek sıralamanın avantajı göreceli olarak artar.

Algoritmalar ve diğer teknolojiler

Yukarıdaki örnek, algoritmaları, bir bilgisayar donanımı gibi, bir **teknoloji** olarak düşünmemiz gerektiğini gösteriyor. Toplam sistem performansı, hızlı donanım seçimi kadar verimli algoritma seçimine de bağlıdır. Algoritmaların geliştirilmesinde olduğu gibi, diğer bilgisayar teknolojilerinde de hızlı gelişmeler yaşanıyor.

Algoritmaların gerçekten;

- gelişmiş bilgisayar mimarileri ve fabrikasyon teknolojileri,
- kullanımı kolay, sezgileri güçlü, grafiksel kullanıcı arayüzleri
- nesne yönetimli sistemler,
- bütünleşmiş web teknolojileri ve
- kablolu ve kablosuz hızlı ağlar

gibi diğer gelişmiş teknolojilerin ışığında, çağdaş bilgisayarlar için önemli olup olmadığını merak edebilirsiniz.

Cevap evet. Bazı uygulamalar (örneğin bazı basit web tabanlı uygulamalar) uygulama seviyesinde açıkça algoritmik içerik gerektirmezken, bazıları gerektirir. Örneğin, bir noktadan başka bir noktaya nasıl seyahat edileceğini belirleyen bir web tabanlı servis düşünelim. Bu uygulama hızlı bir donanıma, grafiksel kullanıcı arayüze, geniş alanlı bir ağa ve ayrıca muhtemelen nesne yönelimine dayanabilir. Bununla birlikte, bazı işlemler için rota bulma (muhtemelen en kısa yol bulan algoritmalar), harita oluşturma ve adres işaretleme gibi algoritmalar gerekebilir.

Ayrıca, algoritmik içerik gerektirmeyen bir uygulama bile ağır algoritmalara dayanır. Bu uygulama hızlı bir donanıma dayanır mı? Donanım tasarımında algoritmalar kullanılır. Bu uygulama grafiksel kullanıcı arayüzlerine dayanır mı? Herhangi bir grafiksel kullanıcı arayüzün tasarımı algoritmalara dayanır. Bu uygulama ağ sistemlerine dayanır mı? Ağdaki rotalama ağır algoritmalara dayanır. Bu uygulama makine dilinden başka bir dilde yazıldı mı? O zaman bu, algorit-

maların kullanımını yaygınlaştıran derleyici, yorumlayıcı yada çevirici gibi tüm öğeler tarafından işlenirdi. Algoritmalar çağdaş bilgisayarlarda kullanılan bir çok teknolojinin özündedir.

Üstelik, kapasitelerinin giderek artmasıyla bilgisayarları her zamankinden daha büyük problemleri çözmek için kullanıyoruz. Araya sokma sıralamasının ve birleştirerek sıralamanın yukarıda yapılan karşılaştırmasında gördüğümüz gibi, ikisi arasındaki farklılıklar özellikle daha büyük boyutlu problemlerde göze çarpıyor.

Algoritmik bilgi ve teknikle ilgili sağlam bir temele sahip olmak, uzman bir programcılığı acemi bir programcıdan ayıran özelliklerden biridir. Modern programlama teknolojisi ile, algoritma hakkında fazla bilgi sahibi olmadan bazı görevleri başarabilirsiniz, fakat iyi bir algoritma birikimi ile çok çok daha fazlasını yapabilirsiniz.

Alıştırmalar

1.2-1

Uygulama düzeyinde algoritmik içerik gerektiren bir uygulama örneği veriniz ve içeren algoritmanın işlevini tartışınız.

1.2-2

Aynı makinede araya sokma sıralaması ve birleştirerek sıralama uygulamalarını karşılaştırdığımızı varsayalım. n boyutlu bir girdi için, birleştirerek sıralama $64n \log n$ adımda çalışırken, araya sokma sıralaması $8n^2$ adımda çalışıyor. n nin hangi değerleri için araya sokma sıralaması birleştirerek sıralamadan iyidir

1.2-3

Aynı makinede, çalışma zamanı $100n^2$ olan algoritmanın çalışma zamanı 2^n olan algoritmadan daha hızlı olmasını sağlayan en küçük n değeri kaçır?

Problemler

1-1 Çalışma zamanlarının karşılaştırılması

Aşağıdaki tabloda yer alan her $f(n)$ fonksiyonu ve t zamanı için, t zamanda çözülebilen bir problemin en büyük n boyutunu algoritmanın problemi $f(n)$ mikrosaniyede çözdüğünü varsayarak belirleyiniz.

	1 saniye	1 dakika	1 saat	1 gün	1 ay	1 yıl	1 yüzyıl
$\lg n$							
$\sqrt{n}$							
n							
$n \lg n$							
n_2							
n^3							
2^n							
$n!$							

Bölüm notları

Algoritmaların genel konularıyla ilgili harika bir çok metin şu kaynaklarda bulunuyor: Aho, Hopcroft, and Ullman [5, 6]; Baase and Van Gelder [28]; Brassard and Bratley [54]; Dasgupta, Papadimitriou, and Vazirani [82]; Goodrich and Tamassia [148]; Hofri [175]; Horowitz, Sahni, and Rajasekaran [181]; Johnsonbaugh and Schaefer [193]; Kingston [205]; Kleinberg and Tardos [208]; Knuth [209, 210, 211]; Kozen [220]; Levitin [235]; Manber [242]; Mehlhorn [249, 250, 251]; Purdom and Brown [287]; Reingold, Nievergelt, and Deo [293]; Sedgwick [306]; Sedgwick and Flajolet [307]; Skiena [318]; and Wilf [356]. Bentley [42, 43] and Gonnet [145] de algoritma tasarımının bazı pratik yönleri tartışılıyor. Algoritma alanları incelemeleri *Handbook of Theoretical Computer Science, Volume A* [342] ve the *CRC Algorithms and Theory of Computation Handbook* [25] de bulunabilir. Hesaplamalı Biyoloji’de kullanılan algoritmaların bir özeti Gusfield [156], Pevzner [275], Setubal and Meidanis [310] ve Waterman [350] kitaplarında bulunabilir.

Bu bölüm size algoritmaların analizi ve tasarımı ile ilgili olarak, kitap boyunca kullanacağımız genel bir çerçeve sunacak. Bu bölüm, bağımsız fakat 3. ve 4. Bölümlerde göreceğimiz konular için bazı referanslar içeren bir bölümdür (Ek A da da nasıl çözüleceği gösterilen bazı özetler de içerir).

1. Bölümde tanıtılan ve sıralama problemini çözen araya sokma sıralama algoritmasını inceleyerek başlayacağız. Bilgisayarda programlama yaptıysanız, algoritmalarımızı nasıl detaylandırdığımızı göstermek için kullandığımız “pseudocode (kabakod)” kavramı, size tanıdık gelecektir. Araya sokma sıralama algoritmasını açıkladıktan sonra, sıralamanın doğruluğunu tartışacağız ve çalışma zamanını analiz edeceğiz. Bu analiz bizi sıralanacak elemanların sayısı ile zamanın nasıl artacağına odaklanan bir gösterim ile tanıştıracak. Araya sokma sıralaması incelemesinden sonra, algoritmaları tasarlamak ve birleştirerek sıralama algoritmasını geliştirirken kullanmak üzere divide-and-conquer (böl-ve-yönet) yaklaşımıyla tanışacağız. Bu bölümü birleştirerek sıralamanın çalışma zamanının analiziyle bitireceğiz.

2.1 Araya sokma sıralaması

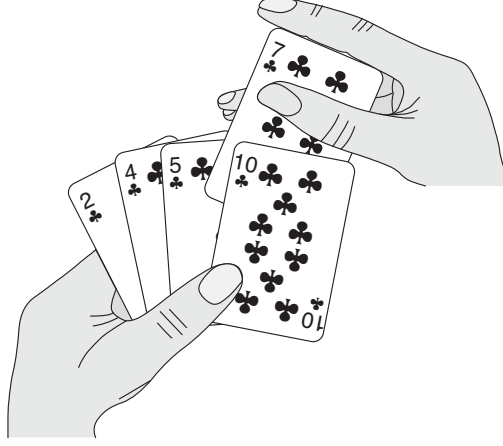
İlk algoritmamız, araya sokma sıralaması, 1. Bölümde tanıtılan *sıralama problemini* çözer.

Girdi: n elemanlı bir $\langle a_1, a_2, \dots, a_n \rangle$ dizisi

Çıktı: $\langle a_1, a_2, \dots, a_n \rangle$ girdi dizisinin $a'_2 \leq a'_2 \leq \dots \leq a'_2$ şartını sağlayan (yeniden sıralanmış) bir $\langle a'_1, a'_2, \dots, a'_n \rangle$ permütasyonu.

Sıralamak istediğimiz sayılara *anahtar* da denir. Kavramsal olarak bir dizi sıralamamıza rağmen, girdi bize n elemanlı bir dizi formunda gelir.

Bu kitapta, genel olarak algoritmaları bir çok bakımdan C, C++, Java, Python ya da Pascal’a benzeyen *kabakod* ile yazılmış programlarla açıklayacağız.

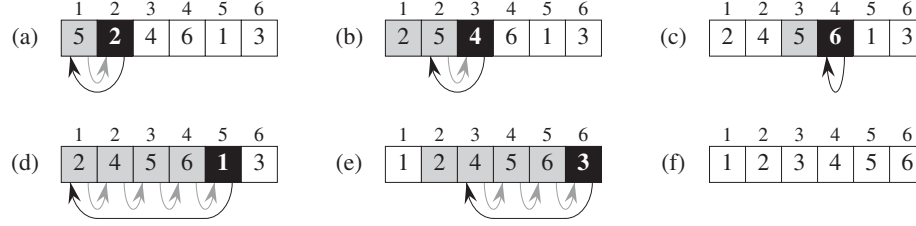


Şekil 2.1 Kartların araya sokma sıralaması kullanılarak el ile sıralanması.

Bu dillerden herhangi biriyle ilgilendiyseniz algoritmalarımızı okurken küçük bir probleminiz var demektir. Kabakodu gerçek koddan ayıran şey kabakodda verilen algoritmayı açıklamak için en etkin yöntem neyse, onu uygularız. Bazen, en net yöntem metindir, bu yüzden “gerçek” kodun içinde metinlere rastlarsanız şaşırmayın. Gerçek kod ve kabakod arasındaki başka bir fark, kabakodun yazılım mühendisliğinin konularıyla genel olarak ilgili olmamasıdır. Veri soyutlama, modülerlik ve hata ayıklama ile ilgili hususlar algoritmanın özünü daha net bir biçimde iletebilmek için genellikle gözardı edilir.

Küçük boyutlu elemanların sıralanması için verimli bir algoritma olan **araya sokma sıralaması** ile başlıyoruz. Araya sokma sıralaması bazı insanların oyun kartlarını elle sıraladığı gibi çalışır. Boş bir el ile ve kartlar masaya kapatılmış biçimde başlarız. Sonra, masadan bir kartı alırız ve sol elimizde doğru pozisyona ekleriz. Bir kartın doğru pozisyonunu bulmak için, Şekil 2.1 de gösterildiği gibi, o kartı daha önceden elimizde yer alan kartların her biriyle sağdan sola doğru kontrol ederiz. Her zaman için sol elimizdeki kartlar sıralıdır ve bu kartlar orijinalinde masadaki destenin en üstünde olanlardır.

Araya sokma sıralaması için, parametre olarak, sıralanacak n uzunluklu bir veri yığını içeren $A[1..n]$ dizisini alan INSERTION-SORT isimli prosedürün kabakodunu yazalım. (A daki elemanların n uzunluğu kabakod içinde $A.length$ olarak tanımlanıyor.) Algoritma, girdi sayılarını **yerinde** sıralar: A dizisindeki sayıları, herhangi bir anda dışarıda depolanmış sayılarla birlikte, yeniden düzenler. INSERTION-SORT prosedürü tamamlandığında A girdi dizisi sıralanmış bir veri dizisini içerir.



Şekil 2.2 $A = \langle 5, 2, 4, 6, 1, 5 \rangle$ dizisi için INSERTION-SORT işlemi. Dizi indisleri dikdörtgenlerin üzerinde, indise ait değerler dikdörtgenlerin içindedir. (a)–(e): 1-8 inci satırlardaki **for** döngüsündeki iterasyonlar. Her bir döngü iterasyonunda, siyah dikdörtgenler 5. satırda $A[j]$ 'den alınan ve şekilde solundaki gölgeli dikdörtgenlerle karşılaştırılan anahtardır. Gölgeli oklar 6. satırda sağa tek bir pozisyona taşınan dizi değerlerini, siyah oklar 8. Satırda anahtarın taşınacağı yeri gösterir. (f) Sıralanmış son dizi.

INSERTION-SORT (A)

```

1  for  $j = 2$  to  $A.length$ 
2       $key = A[j]$ 
3      //  $A[j]$  yi sıralanmış  $A[1..j-1]$  ye ekle
4       $i = j - 1$ 
5      while  $i > 0$  ve  $A[i] > key$ 
6           $A[i+1] = A[i]$ 
7           $i = i - 1$ 
8       $A[i+1] = key$ 

```

Döngü değişmezleri ve araya sokma sıralamasının doğruluğu

Şekil 2.2, $A = \langle 5, 2, 4, 6, 1, 3 \rangle$ için bu algoritmanın nasıl çalıştığını gösteriyor. j indisi ele alınmış olan “o anki kartı” gösterir. **For** döngüsünün (j ile indislenmiş olan) her bir iterasyonunun başlangıcında, $A[1..j-1]$ elemanlarını içeren altdizi anlık sıralı eli oluşturur, geri kalan $A[j+1..n]$ altdizisi halen masadaki kart yığınına karşılık gelir. Aslında, $A[1..j-1]$ elemanları 1 den $j-1$ e kadar pozisyonlarda olan evvelki elemanlardır, fakat şimdi sıralı şekildedir. $A[1..j-1]$ in bu özelliklerini **döngü değişmezi** olarak şöyle konumlandırırız:

1-8 satırları arasındaki **for** döngüsünün her bir iterasyonunun başlangıcında, $A[1..j-1]$ altdizisi, $A[1..j-1]$ ndeki orijinal elemanları içerir fakat sıralı değildir.

Döngü değişmezlerini algoritmanın neden doğru çalıştığını anlamamıza yardımcı olması için kullanırız:

Başlangıç: Döngünün ilk iterasyonu öncesi bu doğrudur.

Yineleme: Eğer döngünün herhangi iterasyon öncesi bu doğru ise sonraki iterasyon öncesi de doğru olur.

Sonlandırma: Döngü sonlandığı zaman, değişmezler bize algoritmanın doğru olduğunu göstermeye yardım eden faydalı bir özellik verir.

İlk iki özellik sağlandığı zaman, döngü değişmezi döngünün her iterasyonu için doğrudur. (Tabii ki, her bir iterasyondan önce döngü değişmezinin doğru olduğunu kanıtlamak için döngü değişmezinin kendisinden başka, belirtilen diğer unsurları kullanmakta serbestiz). Matematiksel tümevarıma benzerliğine dikkat ederek, temel durumu ve tümevarımsal adımları kanıtlarsınız. Burada, değişmezin önce temel duruma karşılık gelen ilk iterasyonu sağladığını ve iterasyondan-iterasyona tümevarımsal adıma uygun olarak sağlandığını gösteririz.

Doğruluğu gösterdiğimiz üçüncü özellik ise bu yüzden belki de en önemli özelliktir. Genel olarak, döngü değişmezini döngüyü sona erdirmeye koşuluyla birlikte kullanırız. Bitiş özelliği matematiksel tümevarımda kullanışımızdan farklıdır. Matematiksel tümevarımda tümevarımsal adımlar sonsuzdur, burada döngü sona erdiğinde tümevarımı durdururuz.

Bu özelliğin araya sokma sıralaması için nasıl çalıştığını görelim.

Başlangıç: $j=2^1$ için döngü değişmezinin ilk iterasyonu sağladığını göstererek başlıyoruz. $A=[1..j-1]$ altdizisi bu durumda henüz tek elemanlıdır, yani $A[1]$ dir. Üstelik bu altdizi sıralıdır, ki bu da bizi döngü değişmezinin ilk döngü öncesi sağlandığını gösterir.

Yineleme: Şimdi ikinci özelliği ele alalım: Her bir iterasyonun döngü değişmezi ni sağladığını gösterelim. Kabaca, **for** döngüsünün ana yapısı $A[j]$ için uygun pozisyon bulunana kadar $A[j-1]$, $A[j-2]$, $A[j-3]$ ü sağa doğru birer pozisyon taşıyarak çalışır (4-7. satırlar) ve bulunduğu (8. satır) bu pozisyona $A[j]$ değerini ekler. Bu durumda $A[1..j]$ altdizisinin elemanlarını sıralı bir dizi içerir. **For** döngüsünün sonraki iterasyonu için artan j , döngü değişmezi ni korur.

İkinci özelliğin daha biçimsel bir işlevi, 5-7. satırlardaki **while** döngüsü için bir döngü değişmezi bildirmemizi ve göstermemizi gerektirecekti. Bu noktada, bununla birlikte, bu tür çıkmaza girmiş bir biçimselleştirmeyi tercih etmi-

¹Döngü **for** döngüsü olduğu zaman, döngü değişmezi ni ilk iterasyon için kontrol ettiğimiz an döngü sayaç değişkenine başlangıç değişkenini atandıktan hemen sonra ve döngü başlığındaki ilk testten hemen öncesidir. INSERTION-SORT un bu durumunda, 2 den j değişkenine atama olur ama daha önce $j \leq A.length$ olup olmadığı test edilir.

yoruz ve biçimsel olmayan ve dış döngü için ikinci özelliği sağladığını gösteren analizimizi yeterli buluyoruz.

Sonlandırma: Nihayet, döngü sonlandığında ne olduğunu belirleriz. **for** döngüsünü sonlandıran şart $j > A.length = n$ olmasıdır. Çünkü, her bir döngü iterasyonu için j , 1 er artar, o halde $j = n + 1$ olmak zorundadır. Döngü değişiminin ifadesinde j yerine $n + 1$ i değiştirirsek, $A[1..n]$ veri dizisinin elemanlarını içeren ve sıralı $A[1..n]$ alt dizisini elde ederiz. $A[1..n]$ dizisini incelediğimizde sonuçta sıralanmış giriş dizisi olduğunu gözlemliyoruz. Bu nedenle, algoritma doğru çalışır.

Döngü değişimlerinin bu yöntemini bu ve sonraki bölümlerde de doğruluğu göstermek için kullanacağız.

Kabakod prensipleri

Kabakod aşağıdaki prensipleri kullanıyoruz.

- Satırbaşları blok yapıları gösterir. Örneğin, **for** döngüsünün ana yapısı 1. satır ile başlar ve 2-8. satırları içerir. **While** döngüsünün ana yapısı 5. satırda başlar ve 6 ve 7. satırda devam eder ama 8. satıra geçmez. Satır başı stilimiz **if-else** yapısına da uygulanır.² Blok yapılarında genel olarak kullanılan **begin** ve **end** gibi gösterimlerinin yerine satırbaşı stiline kullanılması dağınıklığı azaltır, netliği artırır.³
- **while**, **for** ve **repeat-until** döngü yapıları ve **if-else** koşul yapısı C, C++, Java, Python ve Pascal'da benzer yorumlara sahiptir.⁴ Bu kitapta, C++, Java ve Pascal'da olduğunun aksine, döngü sayacı döngüden çıktıktan sonra değerini korur. Bu yüzden, bir **for** döngüsünden hemen sonra döngü sayacının değeri **for** döngüsü aşıldıktan sonraki ilk değer olur. Bu özelliği araya sokma sıralamasının doğruluğu bahsinde kullandık. 1.satırdaki **for** $j = 2$ **to** $A.length$ ifadesi **for** döngüsü başlığıdır. Bu durumda döngü sonlandığında $j = A.length + 1$ (ya da $n = A.length$ için $j = n + 1$) olur. **For** döngüsünde her bir iterasyonda döngü sayacı artarsa **to**, azalırsa **downto** kelimelerini kullanırız. Döngü sayacı 1 den

²**If-else** koşulunda, else ile eşleştiği **if** i aynı girinti seviyesinde kullanıyoruz. **Then** kelimesini ihmal etmemize rağmen, bazen, if i takip eden test doğru olduğunda **then clause** (then parçası) olarak parçalamayı tercih ederiz. Çok koşullu testler için birinci testten sonraki testlerde **elseif** kullanırız.

³Bu kitaptaki her kabakod prosedürü tek bir sayfa içindedir. Böylece bölünmüş sayfalarındaki kodların paragraflarını birleştirerek düzenlemek zorunda kalmayacaksınız.

⁴Blok yapıları çoğu dil gramer farklılıklarına rağmen aynı formata sahiptir. Python dilinde **repeat-until** döngüsü eksiktir. Ayrıca **for** döngüsünde de bu kitaptaki **for** döngüsünden küçük farklılıklar vardır.

daha büyük bir miktarda değişirse, değişim miktarı opsiyonel olarak yazılan **by** ile belirtilir.

- “//” sembolü satırın geri kalanının bir yorum olduğunu gösterir.
- $i = j = e$ biçimindeki çoklu atama, e ifadesinin değerini i ve j değişkenine atama anlamındadır. Bu $j = e$ atamasını takiben $i = j$ atamasına eşdeğer olarak görülmelidir.
- i , j veya key gibi değişkenler verilen prosedür için yereldir. Net bir biçimde ifade etmedikçe evrensel değişkenler kullanmayacağız.
- Dizi elemanlarının girişini dizi ismini takip eden köşeli parantezler içinde indisini belirterek yaparız. Örneğin, $A[i]$ ifadesi A dizisinin i inci elemanını gösterir. “.” gösterimi dizinin değer aralığını belirtmek için kullanılır. Yani, $A[1..j]$ ifadesi A nın $A[1]$, $A[2]$, $A[3]$, ..., $A[j]$ elemanlarını içeren alt dizisini gösterir.
- Genel olarak verileri **özelliklerin** bileşimi olan **nesneler** içinde düzenleriz. Belirli bir özelliğin girişini bazı nesneye yönelik programlama dillerinde kullanılan gramer gibi yaparız. Bu dillerdeki giriş “nesne” ismi, sonra nokta, sonra da belirli özelliğin ismi biçimindedir. Örneğin, bir diziyi, kaç eleman içerdiğini *length* özelliğinde tutan bir nesne olarak gösterdik. Dizin eleman sayısını belirlemek için $A.length$ ifadesini yazarız.

Diziyi veya nesneyi temsil eden bir değişkeni, dizi yada nesneyi temsil eden veriler için bir işaretçi gibi düşünürüz. Bir x nesnesinin tüm f özelliği için, $y = x$ ataması $y.f$ ile $x.f$ in aynı olacağı anlamına gelir. Ayrıca, eğer $x.f = 3$ ataması yaparsak, bu sonrası için $y.f$ de 3 e eşittir anlamına gelir. Başka bir değişle, $y = x$ atamasından sonra x ve y aynı nesnedir.

Özellik gösterimi “basamaklayabilir”. Örneğin, varsayalım ki f özelliğinin kendisi g özelliğine sahip bir nesne tipinin işaretçisi olsun. O zaman $x.f.g$ gösterimi $(x.f).g$ olarak gösterilir. Başka bir değişle eğer $y = x.f$ atamışsak o zaman $x.f.g$ ifadesi $y.g$ ile aynı olur.

Bazen, işaretçi bir nesneyi göstermez. Bu durumda bu işaretçiye NIL değeri veririz.

- Parametreleri bir prosedüre **değer** olarak göndeririz: çağrılan prosedür parametrelerin bir kopyasını alır ve bir parametre için bir değer atar ise bu değişiklik çağırılan prosedür tarafından görülmez. Nesneler gönderildiğinde, nesneyi temsil eden verinin işaretçisi kopyalanır fakat nesnenin özellikleri kopyalanmaz. Örneğin, x çağrılan bir prosedürün parametresi ise çağrılan prosedür içindeki $x = y$ ataması çağırılan prosedür için görünmez. Ancak $x.f = 3$ ataması görünür. Benzer biçimde diziler işaretçiyle gönderilir, yani tüm diziden ziya-

de, dizi için bir işaretçi gönderilir. Tek tek dizi elemanlarındaki değişiklikler çağıran prosedürde görünür.

- **Return** ifadesi kontrolü çağıran prosedürdeki çağırma noktasına geri taşır. Bir çok return ifadesi aynı zamanda çağırıcıya geri göndermek için bir değer alır. Bizim kabakodumuz tek bir **return** ifadesinde geri gönderilecek çoklu değerlere izin verdiğinden bazı programlama dillerinden farklıdır.
- “and” ve “or” operatörleri *kısa devre* (short circuiting) dir. Bu “ x and y ” deyimini hesaplamak için ilk önce x değerini hesaplar demektir. Eğer x FALSE olarak hesaplanırsa o zaman tüm deyim TRUE olamaz ve neticede y de hesaplanmaz. Başka bir deyişle, eğer x TRUE olursa tüm deyim hesaplanabilmesi için y yi de hesaplamak zorunda kalırız. Benzer şekilde “ x or y ” deyiminde eğer ve yalnız eğer x FALSE hesaplanıyorsa y yi de hesaplarız. Kısa-devre operatörleri x NIL olduğunda $x.f$ yi hesaplamayı denerken ne olur kaygısı taşımadan “ $x \neq NIL$ ” ya da “ $x.f = y$ ” gibi deyimleri yazmaya olanak sağlar.
- **Error** kelimesi çağrılacak prosedür için koşulların yanlış olması nedeniyle hata oluştuğunu gösterir. Çağrılan prosedür hatanın ayıklanmasından sorumludur ve bu durumda yapacağımız reaksiyonu açıkça belirtmeyiz.

Alıştırmalar

2.1-1

Şekil 2.2 yi model olarak kullanın ve $A = \langle 31, 41, 59, 26, 41, 58 \rangle$ dizisinde INSERTION-SORT işlemini şekillendirin.

2.1-2

INSERTION-SORT prosedürünü azalmayan yerine artmayan sırada tekrar yazın.

2.1-3

Arama problemini düşünelim.

Girdi: n elemanlı $A = \langle a_1, a_2, \dots, a_n \rangle$ dizisi ve v değeri

Çıktı: $v = A[i]$ yi sağlayan i indisi veya A dizisinde v değeri yoksa NIL değeri.

v değerini araştıran *doğrusal arama* (ki bu arama yöntemi A dizisi boyunca tarاما yapar) için bir kabakod yazın. Döngü değişmezi kullanarak algoritmanın doğru olduğunu kanıtlayın. Döngü değişmezinin gereken 3 özelliği sağladığından emin olun.

2.1-4

İki n elemanlı A ve B dizisinde kayıtlanmış n -bit ikilik sistemdeki tam sayının toplamı problemini düşünün. Toplam tam sayı $(n + 1)$ bitlik C dizisinde kayıtlan-

lanmalıdır. Problemi kuramsal olarak belirtin ve iki tam sayının toplanması için bir kabakod yazın.

2.2 Algoritmaları analiz etme

Algoritmayı *analiz etmek* algoritmanın gerektirdiği kaynakları öngörmek anlamına gelir. Bazen hafıza, iletişim bant genişliği ya da bilgisayar donanımı gibi kaynaklar birincil öneme sahip olsa da genellikle ölçmek istediğimiz şey hesaplama zamanıdır. Genelde, bir problem için bir kaç aday algoritmayı analiz ederek en verimlisini belirleyebiliriz. Bu tür analizler birden fazla uygulanabilir aday çıkarabilir fakat çoğu kez bu süreçte bir kaç verimsiz algoritma elenebilir.

Bir algoritmayı analiz etmeden önce kullanacağımız teknolojik bir uygulamanın teknolojik kaynaklarına ve bunların maliyetlerini içeren bir modeline sahip olmalıyız. Bu kitabın büyük bir bölümü için, yaygın tek işlemcili *rassal-erişimli makine (RAM)* hesaplama modelini uygulama teknolojimiz olarak kabul ettik ve algoritmalarımızın bir bilgisayar programı olarak uygulanacağını düşündük. RAM modelinde, komutlardan biri diğerinden sonra yerine getirilir, eş zamanlı işlem yoktur.

Açıkçası, RAM modelinin komutlarını ve maliyetlerini kesin bir biçimde tanımlamalıyız. Bunu yapmak sıkıcı olabilir ancak algoritma tasarımı ve analizinde az da olsa bakış açısı kazandırabilir. Biz yine de RAM modelini hileli kullanmak için dikkatli olmalıyız. Örneğin, bir RAM sıralama yapan bir komuta sahip olsaydı? O zaman sadece tek bir komutla sıralama yapabilirdik. Böyle bir RAM gerçekçi olmayabilir çünkü gerçek bilgisayarlar böyle bir komuta sahip olmazlar. Bu nedenle bizim perspektifimiz gerçek bilgisayarların nasıl tasarlandığı ile ilgilidir. RAM modeli gerçek bilgisayarlarda bulunan yaygın komutları içerir: Aritmetik (toplama, çıkarma, çarpma, bölme, kalanı bulma, alt ve üst yuvarlama), veri hareketi (yükleme, depolama, kopyalama) ve kontrol (koşullu ve koşulsuz dallanma, altprogram çağırma ve döndürme) Her komut sabit bir çalışma zamanı alır.

RAM modelindeki veri tipleri tam sayılar ve (reel sayıların depolanması için) kayan noktalardır. Genel anlamda bu kitapta hassasiyetle ilgilenmiyoruz, fakat bazı uygulamalarda hassasiyet çok önemlidir. Ayrıca, verinin her kelimesinin boyutuyla ilgili bir sınır kabul ettik. Örneğin, n boyutlu girdi ile çalışırken genel olarak tam sayıların bazı $c \geq 1$ sabiti için $c \lg n$ bitle temsil edildiğini kabul ederiz. $c \geq 1$ gereklidir çünkü her kelime bize ayrı girdi elemanlarını indisleme olanağı sağlayan n nin değerini sağlayabilir. c yi bir sabitle kısıtlamamız nedeniyle kelimenin boyutu keyfi olarak artmamış olur (Eğer kelime boyutu keyfi büyürse, bir kelimede çok büyük boyutlu veri depolayabilirdik ve onu sabit zamanlar için çalıştırabilirdik. Gerçekçi olmayan bir senaryo).

Gerçek bilgisayarlar yukarıda listelenmeyen ve RAM modelinde belirsiz bir alanı temsil eden komutlar da içerir. Örneğin, üs alma sabit zamanlı bir komut mudur? Genel bir durumda: hayır; x ve y reel sayı olduğunda x^y yi hesaplamak bir kaç komut ile olur. Kısıtlanmış durumlarda ise üs alma sabit zamanlı bir komuttur. Bir çok bilgisayar sabit bir zamanda bir tam sayının bitlerini k pozisyon sola kaydıran “sola kaydırma” komutuna sahiptir. Bir çok bilgisayarda, bir tam sayının bitlerini bir pozisyondan sola kaydırmak 2 ile çarpmaya eşittir. O halde bitleri k pozisyon sola kaydırmak 2^k çarpımına eşittir. Bu nedenle, böyle bilgisayarlar 2^k yı, k , bir bilgisayar kelimesindeki bitlerin sayısından uzun olmamak üzere, tam sayıyı 1 den k ya kadar pozisyon sola kaydırarak, bir tane sabit-zamanlı komut ile hesaplayabilir. Biz RAM modelindeki belirsiz alanları önlemek için çaba harcayacağız fakat k yeteri kadar küçük pozitif bir tam sayı olduğunda 2^k nın sabit zamanlı işlem olarak hesaplanmasını gerçekleştireceğiz.

RAM modelinde, modern bilgisayarlarda yaygın olan hafıza hiyerarşisini modellemek için çaba harcamayız. Yani ön bellek ve sanal hafızayı modellemeyeceğiz. Bazı hesaplama modelleri, gerçek makinelerdeki gerçek programlarda bazen önemli olan hafıza hiyerarşisi etkilerini hesaplamak için çaba harcar. Bu kitaptaki az sayıda problem hafıza hiyerarşisi etkilerini belirler fakat bu kitaptaki çoğu analizde bu etkileri düşünmeyeceğiz. Hafıza hiyerarşisini içeren modeller RAM modele göre biraz daha karmaşıktır. Bu yüzden bunlarla çalışmak biraz daha zordur. Üstelik, RAM modeli analizi genellikle güncel makinelerdeki performansların öngörülmesinde mükemmeldir.

RAM modelinde basit bir algoritma analiz etmek bile zor olabilir. Gereken matematiksel araçlar; kombinatorik, olasılık teorisi, cebirsel yatınlık ve çok önemli terim ve formülleri tespit etme kabiliyetini kapsayabilir. Basit, kolay anlaşılır formüllerle bu davranışı özetleyen bir araca ihtiyaç duyarız, çünkü bir algoritmanın davranışı her girdi için farklı olabilir.

Verilen bir algoritmayı analiz etmek için genelde sadece bir tane makine modelini seçsek de halen, “analizimizi nasıl açıklarız” noktasında bir çok seçenekle karşı karşıyayız. Yazımı ve idaresi kolay, algoritmanın gerektirdiği kaynakların karakteristik önemini gösteren ve sıkıcı ayrıntıları önleyen bir yol isteriz.

Araya sokma sıralamasının analizi

INSERTION-SORT prosedürünün çalışma zamanı girdiye bağlıdır: Binlerce sayının sıralanması 3 tane sayının sıralanmasından daha fazla zaman alır. Üstelik, INSERTION-SORT önceki sıralanmışlıklarına bağlı olarak aynı boyutlu iki veri dizisinin sıralanmasında da farklı zamanlar alır. Genelde, algoritmanın aldığı zaman girdi boyutuyla artar, bu yüzden bir programın çalışma zamanını girdilerin boyutlarına bağlı bir fonksiyon olarak ifade etmek gelenekseldir. Bunu yapmak için “çalışma zamanı” ve “girdi boyutu” kavramlarını dikkatli tanımlamalıyız.

Girdi boyutu için en iyi gösterim çalışılan probleme bağlıdır. Sıralama ya da ayrık Fourier dönüşümlerini hesaplama gibi bir çok problem için, en doğal ölçüm, sıralama problemindeki “ n boyutlu dizi” örneği gibi, *girdideki elemanların sayısıdır*. İki sayının çarpılması gibi diğer bir kaç problem için, girdi boyutunun en iyi ölçümü, sıradan bir ikili gösterimdeki girdilerin temsil edilmesi için ihtiyaç duyulan *bitlerin toplam sayısıdır*. Bazen, girdi boyutunu bir yerine iki sayı ile tanımlamak daha uygundur. Mesela, algoritmanın girdisi bir çizge ise girdi boyutu çizgedeki tepe ve ayrıtların sayısı olarak tanımlanabilir. Biz çalıştığımız her problemde kullandığımız girdi boyutu ölçümünü belirteceğiz.

Belirli bir girdide bir algoritmanın **çalışma zamanı** ilkel işlemlerin ya da hesaplanan “adımların” sayısıdır. Mümkün olduğu kadar makine-bağımsız olduğu için adım kavramını tanımlamak daha uygundur. Şu an için şöyle düşünelim: Prosedürümüzün her bir satırını hesaplamak için sabit bir zaman gereksin. Bir satır bir diğerinden farklı bir zaman sürebilir. Farz edelim ki i inci satırın tüm hesaplaması c_i kadar zaman alıyor. Burada c_i sabit bir değerdir. Bu bakış açısı RAM modeline uygundur ve aynı zamanda kabakodun güncel bilgisayarlarda nasıl hesaplanabileceğini yansıtır.⁵

Sonraki tartışma, INSERTION-SORT un çalışma zamanı ile ilgili açıklamamızın, c_i maliyetine sahip tüm durumları kullanan dağınık bir formülden daha özet ve kolay düzenlenen daha basit gösterime doğru gelişmesi gerektiğidir. Bu daha basit gösterim aynı zamanda bir algoritmanın diğerinden daha verimli olup olmadığını belirlemeyi kolaylaştıracaktır.

Biz analize, INSERTION-SORT un kabakodundaki her bir ifadenin çalışma sayısını ve birim maliyetini düşünerek başlarız. Her bir $j = 2, 3, \dots, n$ ve $n = A.length$ olmak üzere, t_j, j değeri için 5. satırda hesaplanan **while** döngüsü iterasyon sayısını gösterebilir. **For** ya da **while** döngüsü olağan yollarla sonlandığında (mesela, döngü başlığındaki şart nedeniyle) döngü bir kez daha tekrarlanır. Hesaplanabilir durum olmadıkları için, yorumlar bir zaman almaz.

⁵Burada bazı detaylar vardır. Metin olarak ifade ettiğimiz hesaplama adımları genellikle sabit bir miktardan daha fazla zaman gerektiren bir prosedür çeşitidir. Örneğin, bu kitabın daha sonraki bölümlerinde “noktaları x koordinatlarına göre sıralayın” diyebiliriz ki bu, sabit bir miktardan daha fazla zaman alır. Ayrıca, altprogram çağıran bir ifade, altprogram bir kez çağrıldığında daha fazla sürse de, sabit bir zaman alır. O halde altprogram **çağırma** işlemini parametreleri göz ardı ederek altprogramı **çalıştırma** sürecinden ayırıyoruz.

INSERTION-SORT(A)	<i>maliyet</i>	<i>adet</i>
1 for $j = 2$ to $A.length$	c_1	n
2 $key = A[j]$	c_2	$n - 1$
3 $\nparallel A[1, \dots, j-1]$ sıralı dizisine $A[j]$ yi ekle.	0	$n - 1$
4 $i = j - 1$		c_4 $n - 1$
5 while $i > 0$ ve $A[i] > key$	c_5	$\sum_{j=2}^n t_j$
6 $A[i+1] = A[i]$	c_6	$\sum_{j=2}^n (t_j - 1)$
7 $j = j - 1$	c_7	$\sum_{j=2}^n (t_j - 1)$
8 $A[i+1] = key$	c_8	$n - 1$

Algoritmanın çalışma zamanı, çalıştırılan her bir ifadenin çalışma zamanlarının toplamıdır. Bir ifadenin çalışması c_i adım alırsa ve n defa çalıştırılırsa toplam çalışma süresi $c_i n$ olacaktır.⁶ n girdiye sahip INSERTION-SORT un çalışma zamanı $T(n)$ ı hesaplamak için *maliyetlerin* ve *adet sütununun* çarpımlarını toplarız:

$$\begin{aligned}
 T(n) &= c_1 n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\
 &\quad + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n - 1) .
 \end{aligned}$$

Daha belirli bir girdi boyutu için, algoritmanın çalışma zamanı verilen *bu girdiye* bağlı olabilir. Örneğin, INSERTION-SORT da, dizinin önceden sıralı olduğu durum en iyi durumdur. Bu durumda, her bir $j = 2, 3, \dots, n$ için $i, j - 1$ başlangıç değerindeyken beşinci satırda $A[i] \leq key$ olur. Böylece $j = 2, 3, \dots, n$ için $t_j = 1$ dir ve en iyi-durum çalışma zamanı şöyle hesaplanır:

$$\begin{aligned}
 T(n) &= c_1 n + c_2(n - 1) + c_4(n - 1) + c_5(n - 1) + c_8(n - 1) \\
 &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) .
 \end{aligned}$$

Bu çalışma zamanını a ve b sabitler (c_i maliyetlerine bağlı) olmak üzere, $an+b$ olarak ifade edebiliriz. Bu ise n ye göre **doğrusal bir fonksiyondur**.

Eğer dizi ters sıradaysa, yani azalan sıradaysa, bu en kötü-durum. Bu durumda her bir $A[j]$ elemanını sıralı $A[1, \dots, j - 1]$ dizisinin her elemanı ile karşılaştırırız. Böylece $j = 2, 3, \dots, n$ için $t_j = j$ dir.

⁶ Bu özellik “bellek” gibi kaynaklar için illaki gerekli değildir. Belleğin m kelimesini kullanan ve n defa çalıştırılan bir ifade belleğin mn ayrık kelimesini illaki kullanmaya gerek duymaz.

$$\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1$$

ve

$$\sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$$

(Bu toplamaları nasıl hesaplayacağınızı incelemek için Ek A ya bakın), INSERTION-SORT en kötü-durum çalışma zamanını şöyle buluruz:

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) \\ &\quad + c_6 \left(\frac{n(n-1)}{2} \right) + c_7 \left(\frac{n(n-1)}{2} \right) + c_8(n-1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n \\ &\quad - (c_2 + c_4 + c_5 + c_8) . \end{aligned}$$

Bu en kötü-durum çalışma zamanını yine c_i maliyetine bağlı olan a, b, c sabitleri için $an^2 + bn + c$ olarak ifade edebiliriz. Bu n e bağlı **kuadratik fonksiyon** (ikinci dereceden) dur.

Genel anlamda, araya sokma sıralamasındaki gibi, verilen bir girdi değeri için algoritmanın çalışma zamanı belirlenir. Sonraki bölümlerde göreceğimiz ve davranışları sabit bir girdi için bile değişen bazı ilginç “rassal” algoritmalara rağmen, bir algoritmanın çalışma zamanı belirlenir.

En kötü-durum ve ortalama durum analizi

Araya sokma sıralaması analizimizde, dizinin zaten sıralı olduğu en iyi durum ile dizinin ters sıralı olduğu en kötü-durumun ikisini de inceledik. Kitabın geri kalanı için, yine de, sık sık sadece **en kötü-durumun çalışma zamanını** bulmak üzerine yoğunlaşacağız, çünkü n boyutlu herhangi bir girdi için en uzun çalışma zamanı odur. Bu bakış açısı için 3 nedenimiz var.

- Bir algoritmanın en kötü-durum çalışma zamanı bize herhangi bir girdinin çalışma zamanı hakkında bir üst sınır verir. Bu, algoritmanın asla sürmeyeceği zamanı bilmeyi garanti eder. Böylelikle algoritmanın çalışma zamanı hakkında bazı denemelere ihtiyaç duymayız ve asla daha kötü olmayacağını umarız.
- Bazı algoritmalar için genelde en kötü-durum meydana gelir. Örneğin, özel bir veri parçası için veritabanı aramasında, verinin veri tabanında bulunmaması sık sık meydana gelir ve bu arama algoritmasının en kötü-durumdur. Bazı uygulamalarda olmayan bilginin aranması çok sık olur.

- “Ortalama Durum” aşağı yukarı en kötü-durum kadar kötüdür. Varsayalım ki rastgele n tane sayı seçeceğiz ve araya sokma sıralamasını uygulayacağız. $A[j]$ elemanını $A[1..j-1]$ alt dizisine eklemek ne kadar süre alır? Ortalama olarak, $A[1..j-1]$ deki elemanların yarısı $A[j]$ den küçüktür, yarısı da büyüktür. Bu yüzden ortalama olarak, $A[1..j-1]$ alt dizisinin yarısını kontrol ederiz ve böylece t_j yaklaşık olarak $j/2$ olur. Oluşan ortalama-durum çalışma zamanı tıpkı en kötü-durum çalışma zamanındaki gibi, girdi boyutunun bir kuadratik fonksiyonu olarak sonuçlanır.

Bu kitap boyunca bazı özel durumlarda, algoritmaların *ortalama-durum* çalışma zamanıyla ilgileneceğiz; çeşitli algoritmalara uygulanan *olasılıksal analiz*-nin tekniklerini göreceğiz. Ortalama-durum analizinin kapsamı sınırlıdır. Çünkü özel bir problem için “ortalama” girdinin ne olduğu anlaşılmayabilir. Sık sık, verilen bir boyutta verilen tüm girdilerin eşit olasılıkta olduğunu kabul ederiz. Pratik olarak, bu kabul ihlal edilebilse de bazen olasılık analizini mümkün kılmak ve *beklenen* çalışma zamanını sağlamak için rasgele seçimler yapan *rassal algoritmalar* kullanırız. Rassal algoritmaları daha detaylı olarak 5. Bölümde ve sonraki bir kaç bölümde inceleyeceğiz.

Büyüme mertebesi

INSERTION-SORT süreci analizimizi kolaylaştırmak için bazı soyut basitleştirmeler kullandık. İlk olarak, her bir ifadenin gerçek maliyetini, maliyetleri temsil eden c_i sabitlerini kullanmak suretiyle göz ardı ettik. Daha sonra, bu sabitlerin gerçek ihtiyacımızdan daha fazla detay verdiğini gözlemledik. Buna göre c_i maliyetlerine bağlı olan bazı a , b ve c sabitleri için en kötü-durum çalışma zamanını $an^2 + bn + c$ olarak açıkladık. Bu yüzden, sadece gerçek maliyetleri değil aynı zamanda soyut c_i maliyetlerini de göz ardı ettik.

Şimdi bir tane daha basitleştirilmiş soyutlama yapacağız. O da *büyüme oranı* ya da *büyüme mertebesi*dir. Alt mertebeli terimlerin göreceli olarak n nin büyük değerleri için önemsiz olması nedeniyle bir formülün baş terimini (örneğin an^2) düşüneceğiz. Ayrıca, sabit çarpanlar büyük girdiler için hesaplama verimliliğini belirlemede büyüme oranından daha az önemli olduğundan, baş terimin katsayısını da görmezden geleceğiz. Araya sokma sıralaması için, alt mertebeli terimleri ve baş terimin katsayısını görmezden geldiğimizde geriye n^2 kalır. Bu durumda Araya sokma sıralamasının en kötü-durum çalışma zamanı $\Theta(n^2)$ dir deriz. Bu bölümde Θ notasyonunu kullanacağız fakar 3. Bölümde daha detaylı inceleyeceğiz.

Bir algoritmanın en kötü-durum çalışma zamanı daha az büyüme mertebesine sahip ise diğerine göre daha verimlidir diye düşünürüz. Sabit çarpanlar ve alt mertebeli terimler nedeniyle, küçük girdiler için, yüksek mertebeli çalışma zamanına sahip olan algoritmalar düşük mertebeli çalışma zamanına sahip olan

algoritmalara göre daha az zaman alabilirler. Fakat, yeterince büyük girdiler için örneğin en kötü-durum çalışma zamanı $\Theta(n^2)$ olan algoritma $\Theta(n^3)$ ten daha hızlıdır.

Alıştırmalar

2.2-1

$n^3/1000 - 100n^2 - 100n + 3$ fonksiyonunun Θ -notasyonunu açıklayın.

2.2-2

Bir A dizisinde kayıtlı n sayının sıralamasını düşünün. İlk önce A nın en küçük elemanı bulunup $A[1]$ ile yer değiştirsin. Daha sonra ikinci en büyük bulunsun ve $A[2]$ ile yer değiştirsin. Bu şekilde A nın ilk $n - 1$ elemanı bulunsun. **Seçme sıralaması** olarak bilinen bu algoritmanın kabakodunu yazın. Hangi döngü sabiti bu algoritmayı yineliyor? Bu niye sadece $n - 1$ elemanın çalışmasına ihtiyaç duyar? En iyi ve en kötü-durumdaki çalışma zamanlarının Θ -notasyonunu bulunuz.

2.2-3

Alıştırma 2.1.3 teki doğrusal aramayı tekrar düşünün. Aranacak elemanın dizideki herhangi bir elemanla eşdeğer olduğu düşünüldüğünde, girdi dizisinde ortalama kaç elemanın kontrol edilmesine ihtiyaç vardır? En kötü-durum için ne düşünürsünüz? Doğrusal aramanın Θ -notasyonu için ortalama durum ve en kötü-durum çalışma zamanları nedir?

2.2-4

Hemen hemen her algoritmayı en iyi-durum çalışma zamanına sahip olması için nasıl değiştirebiliriz?

2.3 Algoritmaların tasarımı

Algoritma tasarım tekniklerinin geniş bir yelpazesinden seçim yapabiliriz. Araya sokma sıralaması için, **artan** bir yaklaşım kullandık. Buna göre sıralanmış $A[1..j - 1]$ alt dizisine $A[j]$ tekil elemanını uygun yere ekledik ve $A[1..j]$ sıralı alt dizisini elde ettik.

Bu bölümde, daha detaylı olarak 4. Bölümde inceleyeceğimiz “böl-ve-yönet” olarak bilinen alternatif bir tasarım yaklaşımından bahsedeceğiz. Böl-ve-yönet yaklaşımını en kötü-durum çalışma zamanı, araya sokma sıralamasından daha az olan bir sıralama algoritmasını tasarlamak için kullanırız. Böl-ve-yönet algoritmalarının bir faydası da çalışma zamanlarının genellikle 4. Bölümde göreceğimiz teknikler kullanılarak kolayca belirlenmesidir.

2.3.1 Böl-ve-yönet yaklaşımı

Bazı kullanışlı algoritmalar yapısal olarak **özyinelemelidir**. Yani verilen bir problemi çözerken, yakın ilişkili altproblemlerin üstesinden gelmek için, özyinelemeli olarak bir ya da daha çok kez kendilerini çağırırlar. Bu algoritmalar tipik olarak **böl-ve-yönet** yaklaşımını takip eder: Bu yaklaşım, problemi, orijinal probleme benzer fakat daha az boyutlu bir kaç altprobleme ayırır, altproblemleri özyinelemeli olarak çözer ve sonra orijinal problemin çözümünü oluşturmak için bu çözümleri birleştirir.

Böl-ve-yönet anlayışının her bir özyineleme seviyesi 3 adımdan oluşur:

Böl, aynı problemin daha küçük örneklerle ayrılır.

Yönet Altproblemler özyinelemeyle çözülür. Eğer altproblem boyutları yeterli kadar küçükse daha kolay bir şekilde çözülürler.

Birleştir Altproblemlerin çözümleri orijinal problemin çözümü için birleştirilir.

Birleştirerek sıralama algoritması böl-ve-yönet anlayışına yakındır. Bu algoritma sezgisel olarak aşağıdaki adımlara sahiptir:

Böl: Sıralanacak n elemanlı diziyi her birinde $n/2$ eleman olan iki altdiziyeye böl.

Yönet: İki altdiziyi özyinelemeli olarak birleştirerek sıralama kullanarak sırala.

Birleştir: Sıralama cevabını bulmak için iki altdiziyi birleştir.

Sıralanacak altdizi uzunluğu 1 olan “dip nokta”da yapılacak bir iş kalmamıştır. Çünkü 1 uzunluklu her dizi zaten sıralıdır.

Birleştirerek sıralamanın anahtar işlemi, birleştirme adımıdır sıralı iki diziyi birleştirmektir. Biz yardımcı MERGE (A, p, q, r) prosedürünü kullanarak birleştirme yapacağız. Burada A diziyi, p, q ve r de ($p \leq q < r$) dizideki bir indisi temsil eder. Bu prosedür $A[p..q]$ ve $A[q + 1..r]$ sıralanmış altdizilerini birleştirmeyi üstlenir. Bu prosedür, bu iki prosedürü $A[p..r]$ formunda **birleştirir**.

MERGE prosedürümüz $\Theta(n)$ zaman alır. Burada $n = r - p + 1$ birleştirilecek elemanların toplamıdır ve aşağıdaki gibi çalışır. Kart oynama düzenimize geri dönersek, varsayalım ki masada yüz yukarı iki deste kart olsun. Her deste en küçükü üstte olacak şekilde sıralanmış olsun. Şimdi iki deste kartı masada yüz aşağı olacak biçimde tek bir destede sıralamak isteyelim. Temel adımımız, yüz yukarı olan kart destelerinde üstte olan kartlardan en küçüğünü seçmek ve onu alıp yüz aşağı olacak şekilde birleştireceğimiz destenin ilk kartı olarak yerleştirmektir. Bu

adımı destelerden biri bitene kadar tekrarlarız. Daha sonra geriye kalan desteyi de olduğu gibi alır ve birleştirme destesine yüz aşağı kapatırız. Kaba bir hesaplama, her basit adım sabit bir zaman alır çünkü sadece en üstteki iki kartı karşılaştırıyoruz. O halde en çok n temel adım vardır ve birleştirme işlemi $\Theta(n)$ zaman alır.

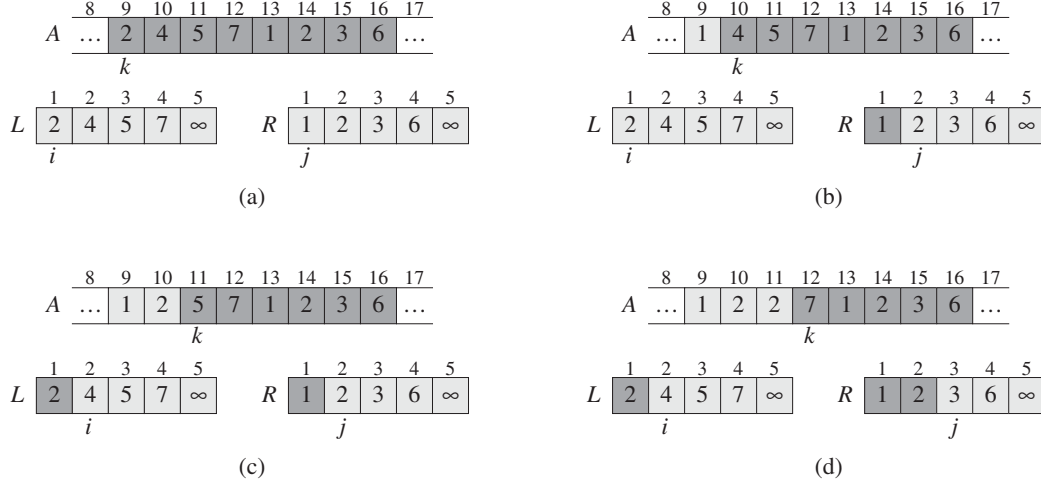
Aşağıdaki kabakod yukarıdaki yöntemi uyguluyor fakat ek olarak her temel adımda destelerin bitip bitmediğini de kontrol ediyor. Her bir destenin zeminine kodu basitleştirmek için özel bir değeri olan *işaretçi* bir kart yerleştiriyoruz. Burada, işaretçi kartın özel değerini ∞ olarak alıyoruz. Böylece her iki deste için ∞ değerine sahip bir kart ortaya çıktığında, daha küçük bir kart olamaz. Fakat bu bir kere olduğunda, tüm diğer kartlar tek bir destede yerleştirilmiş demektir. O halde $r - p + 1$ kart kesinlikle birleştirilmiştir ve birleştirmeyi durdurabiliriz.

```

MERGE ( $A, p, q, r$ )
1   $n_1 = q - p + 1$ 
2   $n_2 = r - q$ 
3   $L[1 \dots n_1 + 1]$  ve  $R[1 \dots n_2 + 1]$  yeni diziler olsun.
4  for  $i = 1$  to  $n_1$ 
5       $L[i] = A[p + i - 1]$ 
6  for  $j = 1$  to  $n_2$ 
7       $R[j] = A[q + j]$ 
8   $L[n_1 + 1] = \infty$ 
9   $R[n_2 + 1] = \infty$ 
10  $i = 1$ 
11  $j = 1$ 
12 for  $k = p$  to  $r$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 

```

Daha detaylı olarak MERGE işlemi şöyle çalışır: 1. satır $A[p \dots q]$ alt dizisinin n_1 uzunluğunu, 2. satır $A[q + 1 \dots r]$ alt dizisinin n_2 uzunluğunu hesaplar. 3. satırda her bir dizideki ekstra pozisyonun işaretçi tutacağı, sırasıyla $n_1 + 1$ ve $n_2 + 1$ uzunluklu L ve R dizilerini (sol ve sağ) oluşturur. 4 ve 5. satırdaki **for** döngüsünün $A[p \dots q]$ alt dizisini $L[1 \dots n_1]$ içine ve 6 ve 7. satırdaki **for** döngüsünün $A[q + 1 \dots r]$ alt dizisini $R[1 \dots n_2]$ içine kopyalar. 8 ve 9. satır L ve R dizilerinin sonuna işaretçileri



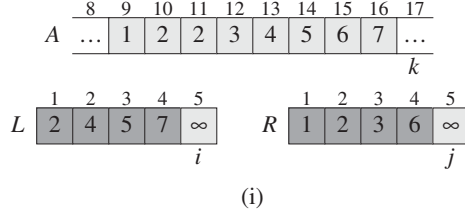
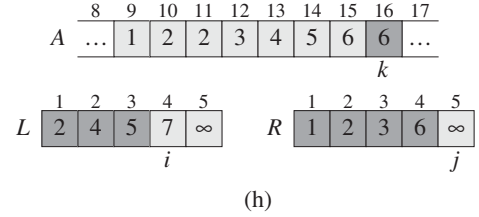
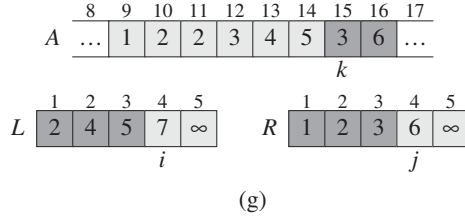
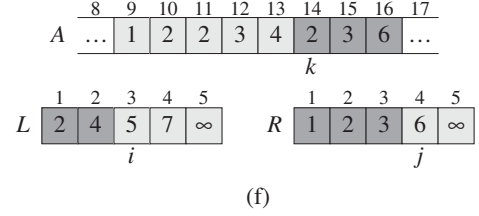
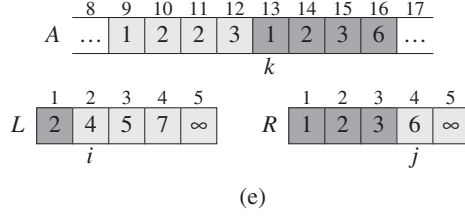
Şekil 2.3 $A[9..16]$ alt dizisi $\langle 2,4,5,7,1,2,3,6 \rangle$ dizisi için MERGE $(A, 9, 12, 16)$ olarak çağrılan 10-17. Satırlardaki işlemlerdir. Kopyalamadan ve işaretçiler eklendikten sonra L dizisi $\langle 2,4,5,7,\infty \rangle$ ve R dizisi $\langle 1,2,3,6,\infty \rangle$ elemanlarını içerir. A içindeki hafif gölgeli yerler son değerleri, L ve R içindeki hafif gölgeli yerler ise A dizisine henüz kopyalanmamış verileri içerir. Beraber düşündüğümüzde, hafif gölgeli yerler daima iki işaretçi ile birlikte, $A[9..16]$ dizisinin orijinal değerlerini içerir. A 'daki koyu gölgeli yerler kopyalanacak değerleri, L ve R içindeki koyu gölgeli yerler önceden A içine kopyalanmış değerleri içerir. **(a)–(h)** A , L ve R dizileri ve sırasıyla onların k , i ve j indisleri 12-17. satırlardaki döngüsündeki her bir iterasyonun önceki değerleridir.

ekler. Aşağıdaki döngü değişkenlerini sürdürerek $r - p + 1$ temel adım uygulayan 10-17. satırlar Şekil 2.3 te şekillendirilmiştir:

12-17. satırların **for** döngülerinin her birinin başında $A[p..k-1]$ sıralanmış alt dizisi, $L[1..n_1+1]$ ve $R[1..n_2+1]$ in $k-p$ en küçük elemanını içerir. Ayrıca, $L[i]$ ve $R[j]$ bu dizilerin henüz A ya kopyalanmamış en küçük elemanlarıdır.

Bu döngü değişmezini, 12-17. satırlardaki **for** döngüsünün önce ilk iterasyonunu sağladığını göstermeliyiz. Zira döngünün her bir iterasyonu bu değişmezi korur ve bu değişmez bize döngü sonlandığı zaman doğruluğunu göstermek için kullanışlı bir özellik sunar.

Başlangıç: Döngünün ilk iterasyonu öncesinde $A[p..k-1]$ alt dizisi boşdur ve $k = p$ dir. Bu boş alt dizi L ve R nin $k-p = 0$ en küçük elemanını içerir çünkü $i = j = 1$ için $L[i]$ ve $R[j]$ kendilerinin en küçük elemanlarıdır ve henüz A ya kopyalanmamıştır.



Şekil 2.3 devamı (i) Sonuçtaki diziler ve indisler. Bu noktada, $A[9..16]$ daki alt dizisi sıralıdır ve L ve R deki A ya kopyalanmayan tek elemanlar işaretçileridir.

Yineleme: Her iterasyonun döngü değişmezini sağladığını görmek için ilk önce $L[i] \leq R[j]$ olduğunu kabul edelim. O zaman $L[i]$ henüz A ya kopyalanmamış en küçük elemandır. Çünkü 14. satır $L[i]$ yi $A[k]$ içine kopyaladıktan sonra $A[p..k-1]$, $k-p$ tane en küçük eleman içerir. $A[p..k]$, $k-p+1$ tane en küçük eleman içerecektir. Artan k (**for** döngüsü güncellemesinde) ve i (15. satırda) sonraki iterasyon için döngü değişmezini yeniden belirler. Eğer $L[i] > R[j]$ olursa, o zaman 16-17. satır döngü değişmezlerini yinelemek için uygun bir etki yapar.

Sonlandırma: Sonlandırmada, $k = r + 1$ dir. Döngü değişmezleri tarafından, $A[p..k-1]$ alt dizisi, ki artık $A[p..r]$ dir, sıralı $L[1..n_1+1]$ ve $R[1..n_2+1]$ nin $k-p = r-p+1$ tane en küçük elemanını içerir. L ve R dizileri birlikte $n_1 + n_2 + 2 = r-p+3$ eleman içerir. İki en büyük haricinde A ya geri kopyalanmıştır, bu iki büyük eleman işaretçilerdir.

$n = r - p + 1$ iken MERGE prosedürünün $\Theta(n)$ zamanda çalıştığını görmek için, 1-2. ve 8-11. satırlarının her birinin sabit zaman aldığını, 4-7. satırların **for** döngüsünün $\Theta(n_1 + n_2) = \Theta(n)$ çalışma zamanı⁷ aldığını ve 12-17. satırlardaki **for** döngüsünün her birinin sabit zaman aldığı n iterasyonu olduğunu gözlemlemeliyiz.

MERGE prosedürünü birleştirerek sıralama algoritmasında bir altprogram olarak kullanabiliriz. MERGE-SORT(A, p, r) prosedürü $A[p..r]$ alt dizisindeki elemanları sıralar. Eğer $p \geq r$ ise, alt dizi en çok bir elemana sahiptir ve daha önceden sıralanmıştır. Aksi halde, bölme adımı $A[p..r]$ parçasını $n/2$ eleman içeren $A[p..q]$ ve $n/2$ eleman içeren $A[q+1..r]$ gibi iki alt diziye bölen q indisini kolaylıkla hesaplar.⁸

MERGE-SORT (A, p, r)

```

1  if  $p < r$ 
2       $q = \lfloor (p + r)/2 \rfloor$ 
3      MERGE-SORT ( $A, p, q$ )
4      MERGE-SORT ( $A, q + 1, r$ )
5      MERGE ( $A, p, q, r$ )

```

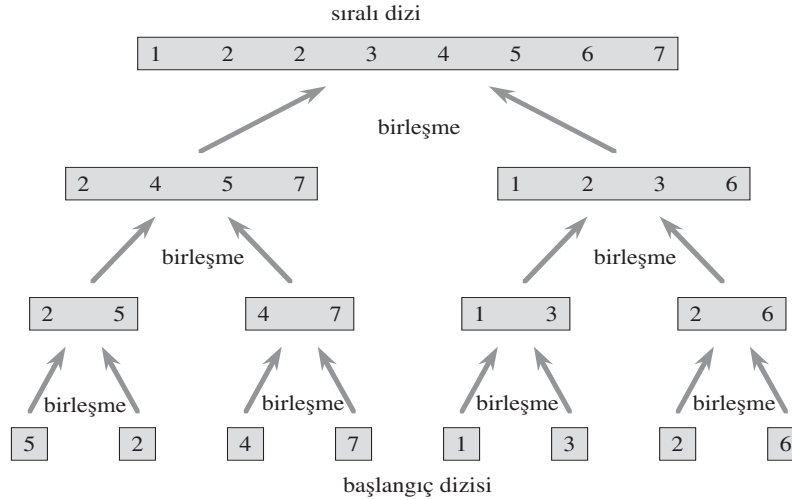
Girilen $A = \langle A[1], A[2], \dots, A[n] \rangle$ dizisini sıralamak için $A.length = n$ için MERGE-SORT ($A, 1, A.length$) başlangıç komutunu çağıracağız. Şekil 2.4, $n, 2$ nin bir kuvveti olduğunda prosedürün aşağıdan yukarıya işlemlerini gösterir. Algoritma, 1-elemanlı dizilerin 2 uzunluklu sıralı formda birleşen çiftlerini, 2 uzunluklu dizilerin 4 uzunluklu sıralı formda birleşen çiftlerini içerir. Böylece iki dizinin uzunluğu $n/2$ olana kadar devam eder ve sonuçta n uzunluklu sıralı birleşmiş form elde edilir.

2.3.2 Böl-ve-yönet algoritmaların analizi

Algoritma kendisini çağıran bir özyineleme içerdiğinde çalışma zamanını genellikle küçük girdilerin çalışma zamanı açısından, n boyutlu bir problemde tüm çalışma zamanını tanımlayan *özyineleme denklemi* ya da *özyineleme* olarak tanımlarız. Daha sonra özyinelemeyi çözmek için matematiksel araçlar kullanırız ve algoritmanın performansı hakkında üst sınırlar belirleriz.

⁷ Θ -notasyonunu içeren denklemlerin nasıl yorumlanacağını 3. Bölümde göreceğiz.

⁸ $\lceil x \rceil$ ifadesi x ten küçük olmayan en küçük tam sayıyı, $\lfloor x \rfloor$ ifadesi x ten büyük olmayan en büyük tam sayıyı gösterir. Bu notasyonlar 3. Bölümde tanımlanmıştır. $\lfloor (p + 2)/2 \rfloor$ ye atanan q nun ardışık olarak $\lceil n/2 \rceil$ ve $\lfloor n/2 \rfloor$ boyutlu $A[p..q]$ ve $A[q + 1..r]$ dizilerini sağladığını doğrulamak için en kolay yol her p ve r nin tek ya da çift olup olmadığına bağlı olarak ortaya çıkan dört durumu da incelemektir.



Şekil 2.4 $A = \langle 5, 2, 4, 7, 1, 3, 2, 6 \rangle$ dizisinin birleştirerek sıralama işlemi. Algoritma aşağıdan yukarı sürdükçe sıralanan dizilerin uzunluğu artmaktadır.

Böl-ve-yönet algoritmasının çalışma zamanı için bir özyineleme, temel paradigmanın üç adımından ortaya çıkar. Daha önce olduğu gibi, n boyutlu problemin çalışma zamanı $T(n)$ olsun. Eğer problem boyutu yeterince küçükse, bazı sabit c ler için $n \leq c$ demektir. Basitçe, çözüm sabit bir zaman alır ve $\Theta(1)$ yazabiliriz. Varsayalım ki problemimizin bölümleri, orijinal boyutu $1/b$ olan her bir a altproblemini sağlasın. (Birleştirerek Sıralama için, hem a hem de b 2 dir fakat bazı böl-ve-yönet algoritmalarında a, b ye eşit değildir.) Bir tane n/b boyutlu altproblemi çözmek $T(n/b)$ dir ve böylece a yı çözmek $a.T(n/b)$ zaman alır. Eğer problemi altprobleme bölmek için gereken zaman $D(n)$ ve orijinal problem için altproblemlerin çözümlerini birleştirmek için gereken zaman $C(n)$ olursa, o zaman şu özyinelemeyi elde ederiz:

$$T(n) = \begin{cases} \Theta(1) & n \leq c \text{ ise,} \\ aT(n/b) + D(n) + C(n) & \text{aksi halde.} \end{cases}$$

4. Bölümde, bu formun yaygın özyinelemelerinin nasıl çözüleceğini göreceğiz.

Birleştirerek sıralamanın analizi

MERGE-SORT için, kabakod, elemanların sayısı verilmediğinde bile doğru çalışmasına rağmen eğer problemin boyutu 2 nin kuvveti kabul edersek özyineleme-tabanlı analizimiz basitleşmiş olur. Her bir bölme adımı kesinlikle $n/2$

boyutlu 2 altdizi verir. 4. Bölümde, bu varsayımın, özyineleme için çözümün büyüme mertebesini etkilemediğini göreceğiz.

$T(n)$ için n sayının birleştirilerek sıralamasında, en kötü-durum çalışma zamanı yinelemesini kurarken aşağıdaki sonucu çıkarırız. Sadece 1 elemanın birleştirilerek sıralaması sabit zaman alır. $n > 1$ sayıda elemana sahip olduğumuzda çalışma zamanını aşağıdaki gibi böleriz.

Böl: Sabit zaman alan bölme adımı sadece altdizinin ortasını hesaplar. Böylece, ifade $(D(n) = \Theta(1))$ dir.

Yönet: Özyinelemeli olarak her birinin boyutu $n/2$ olan iki altproblem çözüyoruz. Çalışma zamanı $2.T(n/2)$ dir.

Birleştir: Daha öncede hatırlattığımız gibi n elemanlı altdizinin MERGE prosedürü $\Theta(n)$ zaman alır. Yani $C(n) = \Theta(n)$ dir.

Birleştirilerek sıralama analizi için $D(n)$ ve $C(n)$ fonksiyonlarını eklediğimiz zaman $\Theta(n)$ ve $\Theta(1)$ fonksiyonlarını ekliyoruz demektir. Bu toplam n ye bağlı doğrusal bir fonksiyondur ve neticede $\Theta(n)$ olur. Bunu “yönet” adımından $2.T(n/2)$ terimine eklediğimizde birleştirilerek sıralamanın en kötü-durum çalışma zamanı için özyineleme şöyle oluşur:

$$T(n) = \begin{cases} \Theta(1) & n = 1 \text{ ise,} \\ 2T(n/2) + \Theta(n) & n > 1 \text{ ise.} \end{cases} \quad (2.1)$$

4. Bölümde, $T(n)$ in $\Theta(n \lg n)$ olarak göstermeye yarayan “ana teorem” i göreceğiz. Burada $\lg n$ gösterimi $\lg_2 n$ yerine kullanılır. Logaritma fonksiyonu doğrusal fonksiyona göre daha yavaş büyüdüğü için yeterince büyük girdiler için en kötü-durumda çalışma zamanı $\Theta(n \lg n)$ olan birleştirilerek sıralama, çalışma zamanı $\Theta(n^2)$ olan araya sokma sıralamasına göre daha üstündür.

(2.1) özyineleme denkleminin çözümünün neden $T(n) = \Theta(n \lg n)$ olduğunu sezgisel olarak anlamak için ana teoreme ihtiyaç duymayız. (2.1) denklemini aşağıdaki gibi tekrar yazalım:

$$T(n) = \begin{cases} c & n = 1 \text{ ise,} \\ 2T(n/2) + cn & n > 1 \text{ ise,} \end{cases} \quad (2.2)$$

burada c sabiti 1 boyutlu problemi çözmek için gereken zamanı temsil ettiği gibi bölme ve birleştirme adımlarının elemanları başına gereken zamanı da temsil eder.⁹

⁹ 1 boyutlu problemi çözmeye zamanının ve bölme ve birleştirme adımlarının, dizi elemanları başına gereken zamanının her ikisinin kesinlikle aynı sabitle temsil etmesi muhtemel değildir. Bu sorunu bahsi geçen zamanlardan daha büyük bir zamanı c harfi ile göstererek çözebiliriz. Bu durumda özyinelemenin, çalışma zamanı hakkında bir üst sınır verdiğini anlarız. Ya da bu zamanlardan daha az bir zamanı c harfi ile göstererek çözebiliriz. Bu durumda da özyinelemenin, çalışma zamanı hakkında bir alt sınır verdiğini anlarız. Her iki sınır da $n \lg n$ mertebesindedir ve birlikte düşündüğümüzde çalışma zamanı $\Theta(n \lg n)$ olur.

Şekil 2.5, (2.2) özyinelemesini nasıl çözebileceğimizi gösteriyor. Kolay olması açısından, n nin 2 nin tam katı olduğunu kabul edelim. Şeklin (a) kısmı, (b) kısmında sunulan özyinelemenin eşit iki ağaca genişletilen $T(n)$ i gösteriyor. cn terimi köktür (özyinelemenin en üst seviyesinde katlanan maliyet). Kökün iki alt ağacı daha küçük olan $T(n/2)$ özyinelemeleridir. (c) kısmı $T(n/2)$ yi genişleterek bir adım ileri taşıma işlemini gösterir. Özyinelemenin ikinci seviyesinde iki tepenin her birinde katlanan maliyet $cn/2$ dir. Problem, boyutları 1 e düşene kadar ağaçtaki her bir tepeyi özyinelemede tanımlandığı gibi onu meydana getiren her biri c maliyetli parçalarına bölerek genişletmeye devam ederiz (d) kısmı **özyineleme ağacının** sonucunu gösterir.

Sonraki adımda ağacın her bir seviyesine karşılık gelen maliyetleri toplarız. En üst seviye toplam $c.n$ maliyetine sahiptir, sonraki seviye $c(n/2) + c(n/2) = c.n$ toplam maliyetine sahiptir, sonraki seviye $c(n/4) + c(n/4) + c(n/4) + c(n/4) = c.n$ maliyetine sahiptir ve bu şekilde devam eder. Genel olarak, i inci seviye her biri $c(n/2^i)$ maliyete sahip 2^i tepeye sahiptir. Bu durumda i inci seviyenin toplam maliyeti $2^i c.(n/2^i) = c.n$ olur. En alt seviye n tepeye sahiptir ve her biri c sabit maliyetine sahiptir. Bu durumda da toplam maliyet $c.n$ olur.

Şekil 22.5 teki özyineleme ağacının toplam seviye sayısı, girdi boyutuna karşılık gelen yaprakların sayısı n olmak üzere, $\lg n + 1$ dir. Kuramsal olmayan tümevarım ilkesi de bu iddiayı doğrulamaktadır. Temel durum $n = 1$ iken, yani ağacın sadece 1 seviyeye sahip olması durumunda oluşur. $\lg 1 = 0$ için, $\lg n + 1$ doğru seviye sayısını verir. Tümevarım ilkesine göre, farz edelim ki 2^i yapraklı özyineleme ağacının seviyelerinin sayısı $\lg 2^i + 1 = i + 1$ olsun (Çünkü, i nin herhangi bir değeri için $\lg 2^i = i$ dir). Girdi boyutunu 2 nin bir kuvveti olarak kabul ettiğimizden, sonraki girdi boyutun 2^{i+1} dir. $n = 2^{i+1}$ yapraklı bir ağaç, 2^i yapraklı bir ağaçtan bir seviye fazlalığa sahiptir. Böylece toplam seviye sayısı $(i + 1) + 1 = \lg 2^{i+1} + 1$ dir.

(2.2) özyinelemesiyle sunulan toplam maliyeti hesaplamak için tüm seviyelerin maliyetleini toplarız. Özyineleme ağacı her biri $c.n$ maliyetli $\lg n + 1$ seviyeye sahiptir. Dolayısıyla maliyetler toplamı $c.n.(\lg n + 1) = c.n.\lg n + c.n$ olur. Düşük mertebeli terimleri ve c sabitlerini görmezden gelirsek $\Theta(n \lg n)$ sonucuna ulaşırız.

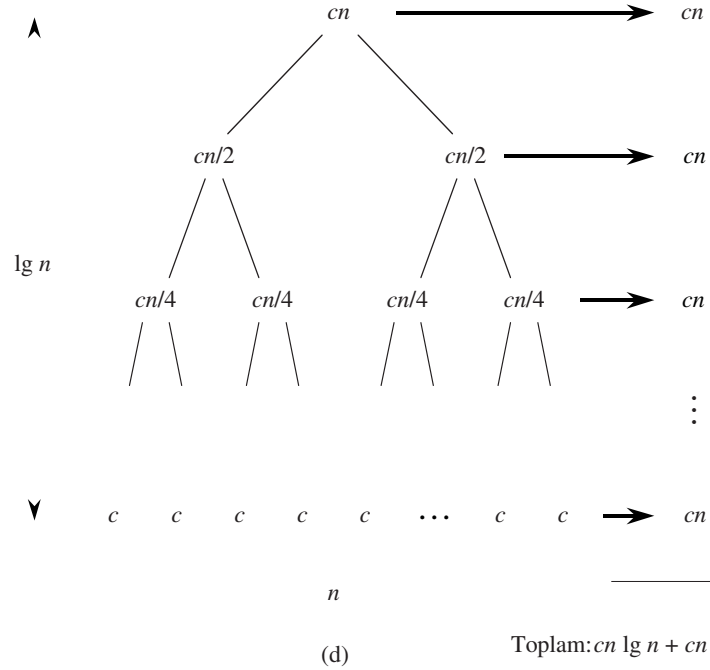
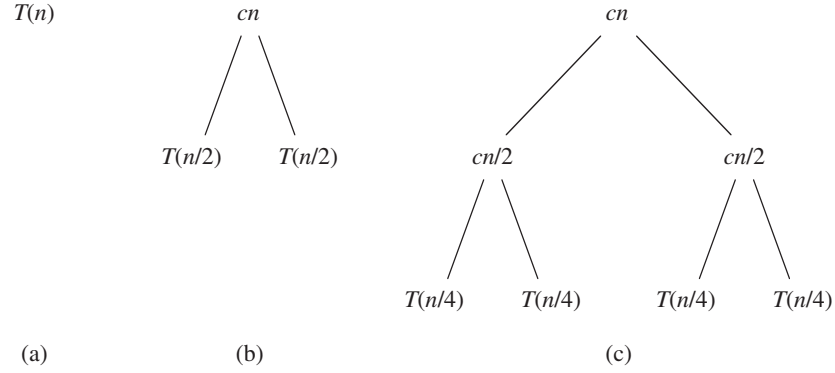
Alıştırmalar

2.3-1

Şekil 2.4 ü model olarak kullanınız ve $A = \langle 3, 41, 52, 26, 38, 57, 9, 49 \rangle$ dizisinde birleştirmeli sıralama işlemini örnekleyiniz.

2.3-2

MERGE prosedürünü işaretçileri kullanmadan, bunun yerine tüm elemanları A ya geri kopyalanmış L ya da R dizisini bir kez durdurarak ve sonra diğer dizide kalanları A ya geri kopyalayarak tekrar yazınız.



Şekil 2.5 $T(n) = 2T(n/2) + cn$ özyinelemesi için özyineleme ağacının nasıl inşa edileceği. **(a)** kısmı $T(n)$ i, **(b)-(d)** kısımları özyineleme ağacına ilerledikçe $T(n)$ in genişlemesini içerir. **(d)** kısmındaki tam genişletilmiş ağaç $\lg n + 1$ seviyesine sahiptir (örneğin, gösterildiği gibi $\lg n$ yükseklikindedir) ve her seviye cn toplam maliyetini sağlar. Bu yüzden toplam maliyet $cn \lg n + cn$, yani $\Theta(n \lg n)$ dir.

2.3-3

n , 2 nin tam kuvveti olduğunda aşağıdaki $T(n)$ özyinelemesinin $T(n) = n \lg n$ olduğunu matematiksel tümerarım kullanarak gösterin.

$$T(n) = \begin{cases} 2, & n = 2 \text{ ise,} \\ 2T(n/2) + n, & k > 1 \text{ için } n = 2^k \text{ ise.} \end{cases}$$

2.3-4

Araya sokma sıralamasını özyinelemeli bir prosedürle şöyle açıklayabiliriz: $A[1..n]$ dizisini sıralamak amacıyla, özyinelemeli olarak $A[1..n-1]$ i sıralarız ve daha sonra $A[n]$ i sıralı olan $A[1..n-1]$ dizisine ekleriz. Araya sokma sıralamasının bu özyinelemeli versiyonunun en kötü-durum çalışma zamanı için bir özyineleme yazın.

2.3-5

Arama problemini düşünelim (Alıştırma 2.1-3 e bakınız). Eğer A sıralı olursa dizinin v ye karşılık gelen orta noktasını kontrol edebileceğimizi ve dizinin yarısını artık dikkate almayacağımızı gözlemleyin. **İkili arama** algoritması her seferinde dizinin geri kalan parçasının boyutunu yarıya bölerek tekrar eder. İkili arama için iteratif veya özyinelemeli bir kabakod yazın. İkili aramanın en kötü-durum çalışma zamanının $\Theta(\lg n)$ olduğunu kanıtlayın.

2.3-6

Kısım 2.1 de, INSERTION-SORT prosedürünün 5-7. satırlarındaki **while** döngüsünün sıralı $A[1..n-1]$ altdizisi boyunca (geri) tarama yapmak için doğrusal arama kullandığını gözlemleyin. Bunun yerine, araya sokma sıralamasının en kötü-durum çalışma zamanı ortalamasını $\Theta(n \lg n)$ e geliştirmek için, ikili aramayı (Alıştırma 2.3-5'e bakınız) kullanabilir miyiz?

2.3-7 *

n tane tam sayı içeren bir S kümesi ve başka bir x tam sayısı verilsin. w deki iki elemanın toplamının x e eşit olup olmadığını kesin olarak bulan $O(n \lg n)$ -zamanlı bir algoritma tanımlayınız.

Problemler
2-1 Birleştirerek sıralamadaki küçük dizilerde araya sokma sıralaması

En kötü-durumda birleştirerek sıralamanın $\Theta(n \lg n)$ ve araya sokma sıralamasının $\Theta(n^2)$ zamanda çalışmasına rağmen, araya sokma sıralaması barındırdığı sabit faktörler nedeniyle pratikte bir çok makinede bazı küçük problem örnekleri için

daha hızlıdır. Bu yüzden, altproblemler yeterince küçük olduğu zaman birleştirerek sıralamada araya sokma sıralaması kullanarak özyineleme basamaklarını **basitleştirmek** mantıklıdır. Birleştirerek sıralama için, k uzunluklu n/k tane alt listeyi araya sokma sıralaması ile sıralayan ve daha sonra bunları standart birleştirerek sıralama sistemiyle birleştiren bir değişiklik düşünün. (k , belirlenecek bir değerdir.)

- Araya sokma sıralamasının her biri k uzunluklu n/k tane alt listeyi en kötü-durumda $\Theta(nk)$ zamanda sıralayabileceğini gösteriniz.
- Alt listelerin en kötü-durumda $\Theta(n \lg(n/k))$ zamanda nasıl birleştirildiğini gösteriniz.
- En kötü-durumda $\Theta(nk + n \lg(n/k))$ zamanda çalışan değiştirilmiş algoritma verilsin. Standart birleştirerek sıralamayla Θ -notasyonuna göre aynı çalışma zamanına sahip değiştirilmiş algoritmanın n ye bağlı bir fonksiyonu için k nın en büyük değeri kaçtır?
- Pratikte k yı nasıl seçmeliyiz?

2-2 Kabarcık Sıralamanın Doğruluğu

Kabarcık sıralama (Bubble Sort) popüler fakat verimsiz bir algoritmadır. Sıralanmamış bitişik elemanları defalarca takas ederek çalışır.

BUBBLESORT (A)

```

1  for  $i = 1$  to  $A.length - 1$ 
2    for  $j = A.length$  downto  $i + 1$ 
3      if  $A[j] < A[j - 1]$ 
4         $A[j]$  ile  $A[j - 1]$  i değiştir

```

- A' , A nın BUBBLESORT çıktısı olsun. Kabarcık sıralamanın doğru olduğunu kanıtlamak için sonlandığını ve n dizi uzunluğu olmak üzere

$$A'[1] \leq A'[2] \leq \dots \leq A'[n] \quad (2.3)$$

olduğunu göstermeliyiz. Kabarcık sıralamanın doğru olduğunu göstermek amacıyla aslında başka ne yapmaya ihtiyaç duyarız?

Sonraki iki bölüm (2.3) eşitsizliğini kanıtlayacak.

- 2-4. satırlarındaki **for** döngüsü için kendi döngü değişmezini belirleyin ve bunun sonlandığını kanıtlayın. Kanıtınız bu bölümde sunulan döngü değişmezi kanıtı yapısını kullanmalıdır.
- (b) de kanıtlanan döngü değişmezinin sonlandırma koşulunu kullanarak 1-4. satırlarındaki **for** döngüsü için (2.3) eşitsizliğini kanıtlamada yardımcı olacak olan kendi döngü değişmezini belirleyin. Bu kanıtınız da bu bölümde sunulan döngü değişmezi kanıtı yapısını kullanmalıdır.

- d. Kabarcık sıralamanın en kötü-durum çalışma zamanı nedir? Araya sokma sıralamasının çalışma zamanıyla nasıl kıyaslarız?

2-3 Horner Kuralı'nın Doğruluğu

Aşağıdaki kod parçası, verilen $a_0, a_1, \dots, a_n$ katsayılı bir polinom ve x değeri için Horner Kuralı'nı gerçekleştirir.

$$\begin{aligned} P(x) &= \sum_{k=0}^n a_k x^k \\ &= a_0 + x(a_1 + x(a_2 + \dots + x(a_{n-1} + xa_n) \dots)) \end{aligned}$$

```

1  y = 0
2  for i = n downto 0
3      y = ai + x • y

```

- a. Θ -notasyonuna göre, Horner Kuralı'nın bu kod parçasığı için çalışma zamanı nedir?
- b. Polinomum her bir terimini sıfırdan hesaplayan düz polinom hesaplama algoritması için bir kabakod yazın. Bu algoritmanın çalışma zamanı nedir? Horner Kuralı ile bunu nasıl karşılaştırırsınız?
- c. Aşağıdaki döngü değişmezini düşünün:

2-3 satırlarındaki **for** döngüsünün her bir iterasyonunun başlangıcında,

$$y = \sum_{k=0}^{n-(i+1)} a_{k+i+1} x^k$$

dir. Hiç bir terimin sıfır olmadığı bir toplamı yorumlayın. Bu bölümde sunulan döngü değişmezi kanıtı yapısıyla döngü değişmezini kullanın ve son bulduğunu gösterin. $y = \sum_{k=0}^n a_k x^k$.

- d. Verilen kod parçasının $a_0, a_1, \dots, a_n$ ile belirtilmiş polinomu doğru hesapladığını iddia ederek kanıtı tamamlayın.

2-4 Ters Çevirmeler

$A[1..n]$ birbirinden ayrık sayılar içeren bir dizi olsun. Eğer $i < j$ ve $A[i] > A[j]$ ise o zaman (i, j) çiftine A nın bir *tersi* denir.

- a. $\langle 2, 3, 8, 6, 1 \rangle$ dizisinin beş tane tersini listeleyin.

- b. $\{1, 2, \dots, n\}$ kümesinden alınacak hangi dizi en çok terse sahiptir? Kaç tanedir?
- c. Girdi dizisindeki terslerin sayısı ile araya sokma sıralamasının çalışma zamanı arasındaki ilişki nedir? Açıklayınız.
- d. n elemanlı herhangi bir sıralamada terslerin sayısını belirleyen ve en kötü-durumda çalışma zamanı $\Theta(n \lg n)$ olan bir algoritma bulunuz (*İpucu*: Kabarcık sıralamayı değiştirin).

Bölüm notları

Knuth, *The Art of Computer Programming* başlıklı üç ciltlik kitabının ilk cildini 1968 de yayımladı [209, 210, 211]. İlk cilt *bilgisayar algoritmaları* için algoritmaların çalışma zamanlarına odaklanarak modern bir çalışma başlatmıştır. Kitabın tüm ciltleri, burada sunulan bir çok konu için ilgi çekici ve değerli bir referans olmaya devam etmektedir. Knuth’a göre “algoritma” kelimesi dokuzuncu yüzyılda yaşamış Fars matematikçi “al-Khowârizmî” nin isminden türetilmiştir.

Aho, Hopcroft ve Ullman [5] algoritmaların görece performanslarını karşılaştırmak için (Θ -notasyonunu da içeren notasyon, 3. Bölümde incelenmiştir) asimptotik analizi öne sürmüşlerdir. Ayrıca, özyinelemeli algoritmaların çalışma zamanını belirlemek için özyineleme kullanımını tanıtmışlardır.

Knuth [211] bir çok sıralama algoritmasının ansiklopedik bir işleyişini sunar. Yaptığı sıralama algoritmaları karşılaştırması (sayfa 381) araya sokma sıralaması için yaptığımız gibi kesin adım sayma analizini içerir. Knuth’un araya sokma sıralaması incelemesi algoritmanın çeşitli versiyonlarını kapsar. Bunların en önemlisi olan ve D. L. Shell tarafından tanıtılan Shell Sıralaması, daha hızlı bir algoritma üretmek amacıyla, girdinin periyodik altdizileri için araya sokma sıralamasını kullanır.

Birleştirerek sıralama da Knuth tarafından tanımlanmıştır. Knuth, 1938 de, tek bir geçişte delikli kartların iki destesini birleştiren yetenekli bir makineden söz etmiştir. Bilgisayar biliminin öncülerinden biri olan J. Von Neumann 1945 te birleştirerek sıralama için EDVAC bilgisayarıda çalışan bir program yazmıştır.

Programların doğruluğunu kanıtlama erken tarihi, P. Naur ile bu alandaki ilk makaleyi paylaşan Gries [153] tarafından başlatılmıştır. Gries döngü değişkenlerini W. Floyd’a atfeder. Mitchell [256] tarafından yazılan ders kitabı programların doğruluğunun kanıtlanmasıyla ilgili daha güncel işlemleri tanımlar.

2. Bölümde tanımlanan “bir algoritmanın çalışma zamanının büyümesinin mertebesi”, algoritmanın verimliliğinin basit bir eğilimini verir ve aynı zamanda bize alternatif algoritmaların görelî performanslarını karşılaştırma imkanı tanır. n girdi boyutu yeterince büyükken, en kötü-durum çalışma zamanı $\Theta(n \lg n)$ olan birleştirerek sıralama en kötü-durum çalışma zamanı $\Theta(n^2)$ olan araya sokma sıralamasından daha üstündür. 2. Bölümde yaptığımız gibi, bazen algoritmanın kesin çalışma zamanını belirleyebilmemize rağmen bunun hesaplanma süreci genellikle hassas değildir. Yeterince büyük girdiler için, girdi boyutunun kendisi, kesin çalışma zamanının sabit çarpanlarına ve düşük dereceli terimlerine baskın gelir.

Yalnızca çalışma zamanı büyümesinin mertebesini hesaplamayla ilgili olan yeterince büyük girdi boyutlarına odaklandığımızda algoritmaların **asimptotik** verimliliğini inceleriz. Yani, biz girdi boyutları sınırlar içinde sıçrama sınırı olmaksızın artarken algoritmanın çalışma zamanının nasıl arttığıyla ilgileniriz. Genellikle, asimptotik olarak daha verimli olan bir algoritma çok küçük girdiler dışında her zaman en iyi seçenek olacaktır.

Bu bölümde algoritmaların asimptotik analizini sadeleştiren bir kaç standart yöntemden bahsedilecek. Bir sonraki kısım, daha önce Θ -notasyonu örneğinde gördüğümüz gibi bir kaç “asimptotik gösterim” türünün tanımıyla başlayacak. Daha sonra kitap boyunca kullanılan bir kaç gösterimsel eğilim sunacağız. Son olarak algoritmaların analizinde yaygın olarak ortaya çıkan fonksiyonların alışkanlıklarını inceleyeceğiz.

3.1 Asimptotik gösterim

Algoritmaların asimptotik çalışma zamanını açıklamak için kullandığımız gösterimler tanım kümesi $\mathbb{N} = \{0, 1, 2, \dots\}$ doğal sayılar olan fonksiyonlarla tanımlanır. Böyle gösterimler genellikle yalnız tam sayı girdi boyutuyla tanımlanan en kötü-durum çalışma zamanı $T(n)$ ı açıklamak için uygundur. Biz genellikle bunu

uygun bulmakla beraber, çeşitli yollarla asimptotik gösterimle oynarız. Örneğin, gösterimin tanım kümesini reel sayılara genişletebiliriz yada doğal sayıların bir altkümesine kısıtlayabiliriz. Gösterimin kesin manasını anlamak için yaptığımız bu oynamayı yaparken yanlış yapmadığımızdan emin olmalıyız. Bu bölümde temel asimptotik gösterimleri tanımlayacağız ve ayrıca bazı yaygın oynamalarla karşılaşacağız.

Asimptotik gösterim, fonksiyonlar ve çalışma zamanları

Asimptotik gösterimleri öncelikle, daha önce araya sokma sıralamasının en kötü-durum çalışma zamanı için $\Theta(n^2)$ diye yazdığımız gibi, algoritmaların çalışma zamanını açıklamak için kullanacağız. Asimptotik gösterim aslında fonksiyonlara uygulanır. Araya sokma sıralamasının en kötü-durum çalışma zamanını bazı a , b ve c sabitleri için $an^2 + bn + c$ olarak genelleştirelim. Araya sokma sıralamasının çalışma zamanını $\Theta(n^2)$ diye yazarak, bu fonksiyonun bazı detaylarını soyutlamıştık. Asimptotik gösterim fonksiyona uygulandığından, $\Theta(n^2)$ olarak yazdığımız şey, araya sokma sıralamasının en kötü-durum analizi için genelleştirdiğimiz $an^2 + bn + c$ fonksiyonuydu.

Bu kitapta, asimptotik gösterim uygulayacağımız fonksiyonlar genellikle algoritmaların genelleştirilmiş çalışma zamanları olacaktır. Fakat asimptotik gösterimler algoritmaların diğer bazı (örneğin, kullandıkları alan miktarı) genel etkilerini gösteren fonksiyonlara veya algoritmalarla hiç bir ilgisi olmayan fonksiyonlara bile uygulanabilir.

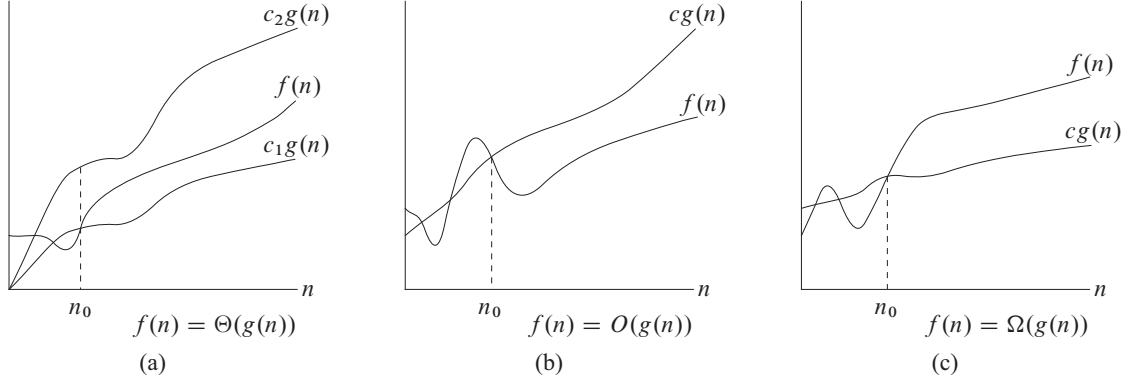
Bir algoritmanın çalışma zamanına uygulamak için asimptotik gösterim kullandığımızda, hangi çalışma zamanını kasıt ettiğimizi anlamaya ihtiyaç duyarız. Bazen en kötü-durum çalışma zamanıyla ilgileniriz. Fakat bizim istediğimiz çalışma zamanını girdiden bağımsız olarak genelleştirmektir. Başka bir ifadeyle, genellikle, sadece en kötü-durum değil tüm girdileri kapsayan geniş bir durumu oluşturmak isteriz. Girdinin önemli olmadığı genelleştirilmiş çalışma zamanına iyi uygulanmış asimptotik gösterimleri göreceğiz.

Θ notasyonu

2. Bölümde araya sokma sıralamasının en kötü-durum çalışma zamanının $T(n) = \Theta(n^2)$ olduğunu bulmuştuk. Bu gösterimin ne anlama geldiğini tanımlayalım. Verilen bir $g(n)$ fonksiyonu için $\Theta(g(n))$ i *fonksiyonların bir kümesi* olarak gösteririz

$$\Theta(g(n)) = \{f(n): c_1, c_2 \text{ ve } n_0 \text{ pozitif sabitleri vardır öyle ki} \\ 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n), \text{ tüm } n \geq n_0 \text{ için}\}.$$

¹Küme gösteriminde iki nokta üst üste işareti “öyle ki” anlamına gelir.



Şekil 3.1 Θ , O ve Ω notasyonlarının grafik örnekleri. Her bir parçada, muhtemel en küçük değer n_0 ile gösterilir; herhangi bir daha büyük değer için fonksiyon çalışır. **(a)** Θ notasyonu sabit bir çarpanı olan fonksiyonu sınırlar. Eğer n_0 da ve sağında, $f(n)$ in değeri daima $c_1g(n)$ ve $c_2g(n)$ (bunlar dahil) arasında olacak biçimde pozitif bir n_0 , c_1 ve c_2 var ise $f(n) = \Theta(g(n))$ yazarız. **(b)** O notasyonu sabit bir çarpanı olan fonksiyon için bir üst sınır verir. Eğer n_0 da ve sağında $f(n)$ in değeri daima $c(g(n))$ e eşit veya ondan daha küçük olacak biçimde n_0 ve c pozitif sabitleri varsa $f(n) = O(g(n))$ yazarız. **(c)** Ω notasyonu sabit bir çarpanı olan fonksiyon için bir alt sınır verir. Eğer n_0 da ve sağında $f(n)$ in değeri daima $c(g(n))$ e eşit veya ondan daha fazla olacak biçimde n_0 ve c pozitif sabitleri varsa $f(n) = \Omega(g(n))$ yazarız.

Eğer yeterince büyük n için, $f(n)$ i, $c_1g(n)$ ve $c_2g(n)$ arasında “sandviçleyebilir” pozitif c_1 ve c_2 sabitleri varsa $f(n)$, $\Theta(g(n))$ kümesine aittir. $\Theta(g(n))$ bir küme olduğu için, $f(n)$ in $\Theta(g(n))$ in bir elamanı olduğunu belirtmek için $f(n) \in \Theta(g(n))$ yazabiliriz. Bunun yerine aynı gösterimi açıklamak için genellikle $f(n) = \Theta(g(n))$ diye yazacağız. Eşitliği bu yolla yazdığımız için şaşırma olabilirsiniz fakat bu bölümde daha sonra böyle yazmanın avantajlı olduğunu göreceğiz.

Şekil 3.1(a) $f(n)$ ve $g(n)$ fonksiyonlarının $f(n) = \Theta(g(n))$ için sezgisel bir grafiğini gösteriyor. n_0 da ve sağındaki tüm n değerleri için $f(n)$ in değeri $c_1g(n)$ e eşit veya üzerinde ve $c_2g(n)$ e eşit veya altındadır. Başka bir deyişle, tüm $n \geq n_0$ için $f(n)$ fonksiyonu, sabit bir çarpanı olan $g(n)$ e eşittir. Buna $g(n)$, $f(n)$ için **asimptotik sıkı sınırdır** deriz.

$\Theta(g(n))$ in tanımı, her $f(n) \in \Theta(g(n))$ üyesinin **asimptotik negatif olmayan (non-negatif)** olmasını gerektirir ki bu n yeterince büyük olduğunda $f(n)$ non-negatif olur anlamına gelir. (Bir **asimptotik pozitif** fonksiyon yeterince büyük tüm n ler için pozitifdir.) Sonuç olarak, $g(n)$ fonksiyonunun kendisi asimptotik non-negatif olmak zorundadır, aksi halde $\Theta(g(n))$ boş kümedir. Biz ayrıca bu nedenle, Θ -notasyonu ile kullanılan her fonksiyonunun asimptotik non-negatif olacağını varsayacağız. Bu varsayım bu bölümde iyi incelenmiş olan diğer asimptotik gösterimler için de uygundur.

2. Bölümde, Θ -notasyonu için düşük mertebeli terimleri atan ve en büyük dereceli terimin katsayısını görmezden gelen kuralsız (informal) bir gösterim ile tanışmıştık. Şimdi sezgisel olarak elde ettiğimiz son ifadeyi kurallı (formal) bir şekilde tanımlayalım ve $\frac{1}{2}n^2 - 3n = \Theta(n^2)$ yi gösterelim. Bunu yapmak için c_1 , c_2 ve n_0 pozitif sabitlerini belirleyelim:

$$c_1 n^2 \leq \frac{1}{2} n^2 - 3n \leq c_2 n^2$$

tüm $n \geq n_0$ için. Tüm tarafları n^2 ile bölersek

$$c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$$

elde ederiz.

Sağ taraftaki eşitsizliği $n \geq 1$ için herhangi $c_2 \geq 1/2$ sabiti seçerek sağlayabiliriz. Benzer şekilde, sol taraftaki eşitsizliği $n \geq 7$ için herhangi bir $c_1 \leq 1/14$ sabiti seçerek sağlayabiliriz. Böylece, $c_1 = 1/14$, $c_2 = 1/2$ ve $n_0 = 7$ seçerek $\frac{1}{2}n^2 - 3n = \Theta(n^2)$ yi sağlayabiliriz. Tabi ki sabitler için başka seçimler de vardır fakat önemli olan şey *bazı* seçimlerin varlığıdır. Not edelim ki bu sabitler $\frac{1}{2}n^2 - 3n$ fonksiyonuna bağlıdır; $\Theta(n^2)$ ye ait başka bir fonksiyon farklı sabitler gerektirebilir.

Kurallı tanımı aynı zamanda $6n^3 \neq \Theta(n^2)$ olduğunu sağlamak için de kullanabiliriz. Tüm $n \geq n_0$ için $6n^3 \leq c_2 n^2$ olmasını sağlayan c_2 ve n_0 in var olması durumunu düşünün. Her taraf n^2 ye bölünürse $n \leq c_2/6$ elde edilir fakat bu yeterince büyük bir n değeri için sağlanamaz, çünkü c_2 sabittir.

Sezgisel olarak, bir asimptotik pozitif fonksiyonun düşük dereceli terimleri, yeterince büyük bir n değeri için önemsiz olduklarından asimptotik sıkı sınır belirlerken göz ardı edilebilir. n büyük olduğunda en yüksek-dereceli terimin küçük bir kısmı bile düşük-dereceli terimleri önemsizleştirir. O halde, en yüksek dereceli terimin katsayısından çok az küçük olan bir değere c_1 ve çok az büyük olan değere c_2 dersek Θ -notasyonunun tanımındaki eşitsizlikler sağlanır. En yüksek dereceli terimin katsayısı da benzer şekilde ihmal edilebilir, çünkü bu sadece katsayıya eşit olan sabit çarpanlar c_1 ve c_2 yi değiştirir.

Örnek olarak, a , b ve c sabitler ve $a > 0$ için herhangi bir $f(n) = an^2 + bn + c$ kuadratik fonksiyonunu düşünelim. Daha düşük dereceli terimleri atarsak ve sabitleri gözardı edersek $f(n) = \Theta(n^2)$ olur. Formal olarak aynı şeyi göstermek için $c_1 = a/4$, $c_2 = 7a/4$ ve $n_0 = 2 \cdot \max(|b|/a, \sqrt{|c|/a})$ alırsak. Tüm $n \geq n_0$ için $0 \leq c_1 n^2 \leq an^2 + bn + c \leq c_2 n^2$ eşitsizliğini sağlayabilirsiniz. Genelde, a_i sabit ve $a_d > 0$ olmak üzere, herhangi bir $p(n) = \sum_{i=0}^d a_i n^i$ polinomu için $p(n) = \Theta(n^d)$ eşitliğine sahibiz (Bknz. Problem 3-1).

Herhangi bir sabit, sıfır dereceli bir polinom olduğundan sabit fonksiyonu, $\Theta(n^0)$, ya da $\Theta(1)$ ile gösterebiliriz. Bu son gösterim küçük bir hiledir, çünkü bu

ifade değişkenin sonsuza gittiğini göstermez.² Biz genellikle $\Theta(1)$ notasyonunu bir sabiti ya da bazı değişkenlerle ilgili sabit bir fonksiyonu açıklamak için kullanacağız.

O-notasyonu

Θ gösterimi bir fonksiyon alttan ve üstten asimptotik olarak sınırlar. Sadece **asimptotik üst sınıra** sahipsek o zaman O -notasyonunu kullanırız. Verilen bir $g(n)$ fonksiyonu için $O(g(n))$ (büyük-o diye okunur) kümesi şöyle tanımlanır:

$$O(g(n)) = \{f(n) : \text{her } n \geq n_0 \text{ için } 0 \leq f(n) \leq c g(n) \text{ şartını sağlayan } c \text{ ve } n_0 \text{ pozitif sabitleri vardır}\}.$$

O -notasyonunu fonksiyon üzerinde sabit bir çarpanı olan üst sınır vermek için kullanırız. Şekil 3.1(b) O -notasyonu hakkında sezgisel bir fikir veriyor. n_0 a eşit veya sağındaki tüm n değerleri için $f(n)$ fonksiyonunun değeri $c.g(n)$ e eşit veya ondan küçüktür.

Bir $f(n)$ fonksiyonunun $O(g(n))$ kümesinin bir üyesi olduğunu belirtmek için $f(n) = O(g(n))$ diye yazarız. Hemen belirtelim ki, $f(n) = \Theta(g(n))$ ifadesi $f(n) = O(g(n))$ olmasını gerektirir çünkü Θ -notasyonu O -notasyonundan daha güçlüdür. Teorik olarak yazmak gerekirse, $\Theta(g(n)) \subseteq O(g(n))$ dir. O halde, herhangi bir $f(n) = an^2 + bn + c$ kuadratik fonksiyonunun $a > 0$ için $\Theta(n^2)$ olduğunu kanıtı aynı zamanda herhangi bir kuadratik fonksiyonun $O(n^2)$ olduğunu da gösterir. Şaşırtıcı olan şeyse $a > 0$ iken herhangi bir $an + b$ doğrusal fonksiyonunun $c = a + |b|$ ve $n_0 = \max(1, -b/a)$ alarak kolayca $O(n^2)$ ye ait olduğunu gösterilebilmesidir.

Eğer daha önce O -notasyonunu gördüyseniz örneğin $n = O(n^2)$ yazmamızı garip karşılayabilirsiniz. Literatürde bazen, O -notasyonunun asimptotik sıkı sınır olarak informal biçimde tanımlandığını görürüz ki biz bunu Θ -gösterimi kullanarak tanımlamıştık. Bu kitapta, bununla birlikte, $f(n) = O(g(n))$ yazdığımızda sadece $g(n)$ in bazı sabit katlarının $f(n)$ in asimptotik üst sınırı olduğunu ifade etmiş olacağız, bu üst sınırın nasıl bir sıklıkta olduğundan bahsetmeyeceğiz. Algoritmalar literatüründe, asimptotik sıkı sınırlardan asimptotik üst sınırı ayırmak standart bir yaklaşımdır.

O -notasyonu kullanarak, genelde bir algoritmanın çalışma zamanını daha genel yapısını incelemek suretiyle tanımlayabiliriz. Örneğin, 2. Bölümde üzerinde durduğumuz araya sokma sıralamasının en kötü-durum çalışma zamanındaki iç içe iki döngü yapısı $O(n^2)$ üst sınırını sağlar. Açıklamak gerekirse: içteki döngü-

²Ana problem, fonksiyonlarla ilgili normal gösterimimizin fonksiyonları değerlerden ayırt etmesidir. λ -calculus da, fonksiyonun parametreleri açık bir şekilde belirtilir: n^2 fonksiyonu $\lambda n.n^2$ ya da $\lambda r.r^2$ olabilir. Daha kesin bir gösterim benimsemek cebirsel yönlendirmeleri zorlaştırabilir, bu yüzden hileleri hoş görüyoruz.

nün her bir iterasyonun maliyeti yukarıdan $O(I)$ (sabit) ile sınırlıdır, i ve j indisleri en çok n olur, içteki döngü i ve j nin her bir n^2 çifti için en çok bir kere çalışır.

O -notasyonu bir üst sınır belirttiğinden dolayı, bunu algoritmanın en kötü-durum çalışma zamanını sınırlamak için kullandığımız zaman her girdi için algoritmanın çalışma zamanı üzerinde bir sınırimız olur-ki bunu daha önce “örtü durumu” ile izah etmiştik. O halde, araya sokma sıralamasının en kötü-durum çalışma zamanı üzerinde tanımlı bir $\Theta(n^2)$ sınırı, aynı zamanda onun her girdi için çalışma zamanıdır. Araya sokma sıralamasının en kötü-durum çalışma zamanı için tanımlı bir $\Theta(n^2)$ sınırı araya sokma sıralamasının her girdisi için çalışma zamanının bir $\Theta(n^2)$ sınırını sağlamaz. Örneğin, 2. Bölümde girdilerin önceden sıralı verildiğini ve çalışma zamanının $\Theta(n)$ olduğunu görmüştük.

Teknik olarak, araya sokma sıralamasının çalışma zamanının $O(n^2)$ olduğunu söylemek gerçeği yansıtmaz, çünkü verilen bir n değeri için gerçek çalışma zamanı n boyutlu girdinin özelliklerine bağlı olarak değişir. Eğer “çalışma zamanı $O(n^2)$ dir” dersek o zaman herhangi bir n değeri için seçilen n boyutlu girdinin özelliklerine bağlı olmaksızın, “ $O(n^2)$ nin eşit olduğu $f(n)$ vardır ve bu girdi için gereken çalışma zamanı üstten $f(n)$ değeri ile sınırlıdır” demek isteriz. Aynı şekilde, en kötü-durum çalışma zamanının $O(n^2)$ olduğunu ifade etmiş oluruz.

Ω -notasyonu

O -notasyonunun bir fonksiyon üzerinde asimptotik bir üst sınır belirlediği gibi Ω -notasyonu **asimptotik bir alt sınır** belirler. Verilen bir $g(n)$ fonksiyonu için $\Omega(g(n))$ (n de g nin büyük omegası ya da sadece n de g nin omegası diye söylenir) aşağıdaki gibi fonksiyonların bir kümesi olarak tanımlanır:

$$\Omega(g(n)) = \{f(n): \text{öyle pozitif } c \text{ ve } n_0 \text{ sabitleri vardır ki} \\ \text{tüm } n \geq n_0 \text{ için } 0 \leq cg(n) \leq f(n) \text{ dir}\}.$$

Şekil 3.1(c) Ω -notasyonu hakkında sezgisel bir fikir veriyor. n_0 a eşit veya sağındaki tüm n değerleri için $f(n)$ fonksiyonunun değeri $cg(n)$ e eşit veya ondan büyüktür.

Şimdiye kadar gördüğümüz asimptotik gösterim tanımlamalarından hareketle aşağıdaki teoremi kanıtlamak kolaydır (Bknz: Alıştırma 3.1-5)

Teorem 3.1

Herhangi iki $f(n)$ ve $g(n)$ fonksiyonu için $f(n) = \Theta(g(n))$ dir eğer ve yalnız eğer $f(n) = O(g(n))$ ve $f(n) = \Omega(g(n))$ dir. ■

Bu teoremin uygulamasının bir örneği olarak, daha önceden kanıtladığımız $a > 0$ olmak üzere herhangi a , b ve c sabitleri için $an^2 + bn + c = \Theta(g(n))$ eşitliğimiz $an^2 + bn + c = O(g(n))$ ve $an^2 + bn + c = \Omega(g(n))$ eşitliklerini gerektirir. Pratikte, Teorem 3.1 i, asimptotik sıkı sınırlardan asimptotik üst ve alt sınırları elde ederken değil, bu örnekte yaptığımız gibi, genellikle, asimptotik üst ve alt sınırlardan asimptotik sıkı sınırları kanıtlamak için kullanırız.

Bir algoritmanın *çalışma zamanı* (niteleyici yok) $\Omega(g(n))$ dir dediğimizde, yeterince büyük ve özelliği önemli olmaksızın seçilen herhangi bir n boyutlu girdi için, çalışma zamanı en az $g(n)$ in sabit bir katıdır demek isteriz. Yani, algoritmanın en iyi durum çalışma zamanı için bir alt sınır veriyoruz. Örneğin, araya sokma sıralamasının en iyi-durum çalışma zamanı $\Omega(n)$ dir demek eklemeli sıralamanın çalışma zamanı $\Omega(n)$ dir anlamına gelir.

Bu yüzden araya sokma sıralamasının çalışma zamanı hem $\Omega(n)$ hem de $O(n^2)$ dir, çünkü çalışma zamanı n nin doğrusal bir fonksiyonu ile kuadratik bir fonksiyonu arasına denk gelir. Üstelik, bu sınırlar asimptotik olarak mümkün olduğu kadar sıkıdır: Örneğin, araya sokma sıralamasının çalışma zamanı $\Omega(n^2)$ değildir, çünkü araya sokma sıralamasını $\Theta(n)$ zamanda çalıştıran bir girdi vardır (örneğin zaten sıralı olan bir girdi). Araya sokma sıralamasının en kötü-durum çalışma zamanının $\Omega(n^2)$ olduğunu söylemek de çelişki değildir, çünkü algoritmayı $\Omega(n^2)$ zamanda çalıştıran bir girdi durumu vardır.

Denklem ve eşitsizliklerde asimptotik gösterim

Şimdiye kadar asimptotik gösterimin matematiksel formüllerde nasıl kullanıldığını gördük. Örneğin, O -notasyonu ile tanıştığımızda “ $n = O(n^2)$ ” yazdık. $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$ de yazabilirdik. Peki böyle formülleri nasıl yorumlarız?

Asimptotik gösterim eşitliğin (ya da eşitsizliğin) sağ tarafında yalnız (yani büyük bir formül içinde değilse) kaldığı zaman, mesela $n = O(n^2)$ iken, eşitlik işaretinin küme üyeliği anlamına geldiğini yani $n \in O(n^2)$ olduğunu tanımlamıştık. Genelde, asimptotik gösterim bir formül içindeyse, denklemde var olan bazı bilinmeyen fonksiyonları isimlendirmeyi önemsemiyoruz. Örneğin $2n^2 + 3n + 1 = 2n^2 + \Theta(n)$ demek $2n^2 + 3n + 1 = 2n^2 + f(n)$ demektir. Burada, $f(n)$, $\Theta(n)$ kümesine ait bir fonksiyondur. Bu durumda $f(n) = 3n + 1$ dir ve aslında $\Theta(n)$ içindedir.

Asimptotik gösterimi bu şekilde kullanmak denklemdeki gereksiz ve karışık detayları elememize yardım eder. Örneğin, 2. Bölümde birleştirerek sıralamanın en kötü-durum çalışma zamanını özyinelemeli olarak şöyle yapmıştık:

$$T(n) = 2T(n/2) + \Theta(n).$$

dir. Eğer sadece $T(n)$ in dir. Eğer davranışı ile ilgilenirsek, tüm alt dereceli terimleri kesin olarak belirtmeye gerek yoktur; bunların hepsinin $\Theta(n)$ ile gösterilen bilinmeyen bir fonksiyon içinde olduğunu anlamak yeterlidir.

Bir ifade içindeki belirsiz fonksiyonların sayısının asimptotik gösterim belirme sayısına eşit olduğu anlaşılır. Örneğin aşağıdaki ifadede

$$\sum_{i=1}^n O(i)$$

sadece bir tane bilinmeyen (i ye bağlı) fonksiyon vardır. Bu durumda bu ifade $O(1) + O(2) + \dots + O(n)$ ile aynı değildir ki zaten bu pek anlamlı da değildir.

Bazı durumlarda, asimptotik gösterim, aşağıdaki gibi, bir denklemin sol tarafında da görünür:

$$2n^2 + \Theta(n) = \Theta(n^2).$$

Bu tür denklemleri şu kurala göre yorumlarız: *Eşitlik işaretinin sol tarafındaki bilinmeyen fonksiyonun nasıl seçildiği önemli değildir; eşitliği sağlamak amacıyla, eşitlik işaretinin sağ tarafındaki bilinmeyen fonksiyonu seçmek için bir yol vardır.* O halde, örneğimizde herhangi bir f için $f(n) \in \Theta(n)$ dir ve tüm n değerleri için $g(n) = 2n^2 + f(n) = g(n)$ yi sağlayan bazı $g(n) \in \Theta(n^2)$ fonksiyonları vardır. Başka bir deyişle, denklemin sağ tarafı sol tarafından daha kaba bir detay seviyesine sahiptir.

Bu tür bağlantıları aşağıdaki gibi bir zincir haline getirebiliriz:

$$\begin{aligned} 2n^2 + 3n + 1 &= 2n^2 + \Theta(n) \\ &= \Theta(n^2). \end{aligned}$$

Her bir denklemi yukarıdaki kurallarla ayrı ayrı yorumlayabiliriz. İlk denklem, tüm n değerleri için $2n^2 + 3n + 1 = 2n^2 + f(n)$ olacak şekilde bazı $f(n) \in \Theta(n)$ fonksiyonları olduğunu ifade eder. İkinci denklem, herhangi $g(n) \in \Theta(n)$ fonksiyonu (daha önce belirtilen $f(n)$ gibi) ve tüm n değerleri için $2n^2 + g(n) = h(n)$ olacak biçimde bazı $h(n) \in \Theta(n^2)$ fonksiyonları olduğunu ifade eder. Bu yorumun $2n^2 + 3n + 1 = \Theta(n^2)$ gerektirdiğini, bunu da denklemleri zincir haline getirirken öğrendüğümüze dikkat edin.

***o*-notasyonu**

O-notasyonu ile sağlanan asimptotik üst sınır asimptotik olarak sıkı olabilir veya olmayabilir. $2n^2 = O(n^2)$ sınırı asimptotik olarak sıkıdır fakat $2n = O(n^2)$ değildir. *o*-notasyonunu asimptotik olarak sıkı olmayan bir üst sınırı belirlemek için kullanırız. Kuramsal olarak $o(g(n))$ (küçük oh) ı şöyle tanımlarız:

$$o(g(n)) = \{f(n): \text{her } c > 0, \text{ her } n \geq n_0 \text{ için } 0 \leq f(n) \leq c g(n) \text{ olacak şekilde } n_0 > 0 \text{ sabiti vardır}\}.$$

Örneğin $2n = o(n^2)$ iken $2n^2 \neq o(n^2)$ dir.

O-notasyonu ile *o*-notasyonunun tanımları birbirine benzerdir. Temel fark şudur: $f(n) = O(g(n))$ de $0 \leq f(n) \leq c \cdot g(n)$ in bazı $c > 0$ sabitleri için sağlanırken, $f(n) = o(g(n))$ de $0 \leq f(n) \leq c g(n)$ ise her $c > 0$ sabitleri için sağlanır. Sezgisel olarak, *o*-notasyonundan sonsuza giderken $f(n)$ in $g(n)$ e göre önemsiz olduğunu söyleyebiliriz. Yani,

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \quad (3.1)$$

dır.

Bazı yazarlar bu limiti o -notasyonunun tanımını diye kullanırlar; bu kitaptaki tanım ise asimptotik olarak negatif olmayan bilinmeyen fonksiyonları sınırlar.

ω -notasyonu

O -notasyonu için o -notasyonu neyse Ω -notasyonu için de ω -notasyonu odur. Sıkı asimptotik olmayan alt sınırı göstermek için ω -notasyonunu kullanırız. Tanım şu şekilde ifade de edilebilir.

$f(n) \in \omega(g(n))$ eğer ve yalnız eğer $g(n) \in o(f(n))$ dır.

Kuramsal olarak ise $\omega(g(n))$ (" n nin g sinin küçük omegası diye okunur) kümesini şöyle tanımlayalım.

$\omega(g(n)) = \{f(n): \text{her } c > 0 \text{ pozitif sabiti ve her } n \geq n_0 \text{ için } 0 \leq c g(n) \leq f(n) \text{ şartını sağlayan bir } n_0 > 0 \text{ sabiti vardır}\}.$

Örneğin; $n^2/2 = \omega(n)$ iken $n^2/2 \neq \omega(n^2)$ dir. $f(n) = \omega(g(n))$ bağıntısı eğer limit mevcut ise

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

olmasını gerektirir. Böylelikle n sonsuza giderken $f(n)$ in $g(n)$ e göre çok büyük olduğunu görürüz.

Fonksiyonları karşılaştırma

Reel sayıların bağıntı özelliklerinin çoğu asimptotik karşılaştırmalara da uygulanabilir. Aşağıdaki özelliklerde $f(n)$ ve $g(n)$ asimptotik olarak pozitifdir.

Geçişkenlik:

$$\begin{aligned} f(n) = \Theta(g(n)) \text{ ve } g(n) = \Theta(h(n)) & \text{ gerektirir } f(n) = \Theta(h(n)) , \\ f(n) = O(g(n)) \text{ ve } g(n) = O(h(n)) & \text{ gerektirir } f(n) = O(h(n)) , \\ f(n) = \Omega(g(n)) \text{ ve } g(n) = \Omega(h(n)) & \text{ gerektirir } f(n) = \Omega(h(n)) , \\ f(n) = o(g(n)) \text{ ve } g(n) = o(h(n)) & \text{ gerektirir } f(n) = o(h(n)) , \\ f(n) = \omega(g(n)) \text{ ve } g(n) = \omega(h(n)) & \text{ gerektirir } f(n) = \omega(h(n)) . \end{aligned}$$

Yansıma:

$$\begin{aligned} f(n) &= \Theta(f(n)) , \\ f(n) &= O(f(n)) , \\ f(n) &= \Omega(f(n)) . \end{aligned}$$

Simetri:

$$f(n) = \Theta(g(n)) \text{ eğer ve yalnız eğer } g(n) = \Theta(f(n))$$

Ters simetri:

$$f(n) = O(g(n)) \text{ eğer ve yalnız eğer } g(n) = \Omega(f(n))$$

$$f(n) = o(g(n)) \text{ eğer ve yalnız eğer } g(n) = \omega(f(n))$$

Bu özellikler asimptotik gösterimler için geçerli olduğundan, f ve g fonksiyonlarının asimptotik karşılaştırması ile a ve b gibi iki reel sayının karşılaştırması arasında bir bağlantı kurabiliriz.

$$f(n) = \Theta(g(n)), a = b \text{ ye benzer.}$$

$$f(n) = O(g(n)), a \leq b \text{ ye benzer.}$$

$$f(n) = \Omega(g(n)), a \geq b \text{ ye benzer.}$$

$$f(n) = o(g(n)), a < b \text{ ye benzer.}$$

$$f(n) = \omega(g(n)), a > b \text{ ye benzer.}$$

Eğer $f(n) = o(g(n))$ ise $f(n)$ in $g(n)$ den **asimptotik küçük**, eğer $f(n) = \omega(g(n))$ ise $f(n)$ in $g(n)$ den **asimptotik büyük** olduğunu söyleyebiliriz.

Bu arada, reel sayıların bir özelliği asimptotik gösterimler için sağlanmaz.

Üçlük Yasası: Herhangi iki reel a ve b sayısı için aşağıdaki durumlardan sadece biri geçerlidir: $a < b$, $a = b$ ya da $a > b$.

Herhangi iki reel sayının karşılaştırılabilmesine rağmen, tüm fonksiyonlar asimptotik olarak karşılaştırılmaz. Yani, $f(n)$ ve $g(n)$ fonksiyonları için ne $f(n) = O(g(n))$ ne de $f(n) = \Omega(g(n))$ durumu olabilir. Örneğin asimptotik gösterim kullanarak n ve $n^{1+\sin n}$ fonksiyonlarını karşılaştıramayız çünkü $n^{1+\sin n}$ ifadesinin üstü 0 ve 2 arasında tüm değerleri alır.

Alıştırmalar**3.1-1**

$f(n)$ ve $g(n)$ asimptotik negatif olmayan fonksiyonlar olsun. Θ -notasyonunun temel tanımını kullanarak $\max(f(n), g(n)) = \Theta(f(n) + g(n))$ olduğunu gösterin.

3.1-2

$b > 0$ olmak üzere herhangi a ve b reel sabitleri için aşağıdaki eşitliğin sağlandığını gösterin:

$$(n + a)^b = \Theta(n^b) \tag{3.2}$$

3.1-3

“ A algoritmasının çalışma zamanı en az $O(n^2)$ dir” ifadesinin neden anlamsız olduğunu açıklayın.

3.1-4

$2^{n+1} = O(2^n)$ midir? $2^{2n} = O(2^n)$ midir?

3.1-5

Teorem 3.1 i kanıtlayın.

3.1-6

“Bir algoritmanın çalışma zamanı $\Theta(g(n))$ dir eğer ve yalnız eğer algoritmanın en kötü-durum çalışma zamanı $O(g(n))$ ve en iyi-durum çalışma zamanı $\Omega(g(n))$ dir” olduğunu kanıtlayın.

3.1-7

$o(g(n)) \cap \omega(g(n))$ in boş küme olduğunu kanıtlayın.

3.1-8

Gösterimimizi farklı oranlarda bağımsız olarak sonsuza gidebilen iki n ve m parametrelili duruma genişletebiliriz. Verilen $g(n, m)$ fonksiyonu için $O(g(n, m))$ fonksiyonlar kümesini şöyle gösteririz:

$O(g(n, m)) = \{f(n, m): \text{ her } n \geq n_0 \text{ ya da } m \geq m_0$
 için $0 \leq f(n, m) \leq c g(n, m)$
 şartını sağlayan c, n_0 ve m_0 pozitif sabitleri vardır}.

Aynı şekilde $\Theta(g(n, m))$ ve $\Omega(g(n, m))$ için de benzer tanımlar yapınız.

3.2 Standart gösterimler ve ortak fonksiyonlar

Bu bölümde bazı standart matematiksel fonksiyonlar ve gösterimler ile bunlar arasındaki ilişkileri inceleyeceğiz. Ayrıca asimptotik gösterimlerin kullanımını örnekleyeceğiz.

Monotonluk

Bir $f(n)$ fonksiyonu $m \leq n$ iken $f(m) \leq f(n)$ ise **monoton artan**, benzer şekilde $m \leq n$ iken $f(m) \geq f(n)$ ise **monoton azalandır**. Bir $f(n)$ fonksiyonu $m < n$ iken $f(m) < f(n)$ ise **kesin artan**, $m < n$ iken $f(m) > f(n)$ ise **kesin azalandır**.

Zeminler ve tavanlar

Herhangi bir x reel sayısı için x ten küçük eşit olan en büyük tam sayıyı $\lfloor x \rfloor$ ile (“ x in zemini” diye okunur) ve x ten büyük eşit olan en küçük sayıyı $\lceil x \rceil$ (“ x in tavanı” diye okunur) ile göstereceğiz. Her bir x reel sayısı için

$$x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1 \quad (3.3)$$

dir. Herhangi bir n tam sayısı için

$$\lceil n/2 \rceil + \lfloor n/2 \rfloor = n$$

dir ve her $x \geq 0$ reel sayısı ve $a, b > 0$ tam sayıları için

$$\left\lceil \frac{\lfloor x/a \rfloor}{b} \right\rceil = \left\lceil \frac{x}{ab} \right\rceil, \quad (3.4)$$

$$\left\lfloor \frac{\lceil x/a \rceil}{b} \right\rfloor = \left\lfloor \frac{x}{ab} \right\rfloor, \quad (3.5)$$

$$\left\lceil \frac{a}{b} \right\rceil \leq \frac{a + (b - 1)}{b}, \quad (3.6)$$

$$\left\lfloor \frac{a}{b} \right\rfloor \geq \frac{a - (b - 1)}{b} \quad (3.7)$$

dir. Tıpkı $f(x) = \lfloor x \rfloor$ zemine yuvarlama fonksiyonu gibi, tavana yuvarlama fonksiyonu $f(x) = \lceil x \rceil$ da monoton artandır

Modüler aritmetik

Her bir a tam sayısı ve n pozitif tam sayısı için $a \bmod n$ değeri a/n işleminin **kalanı** (ya da **artığı**) dir.

$$a \bmod n = a - n \lfloor a/n \rfloor \quad (3.8)$$

Bunu

$$0 \leq a \bmod n < n \text{ izler.} \quad (3.9)$$

Bir tam sayının diğer bir tam sayı tarafından bölünmesiyle elde edilen kalanın iyi tanımlı bir gösterimi verilsin, bu, kalanların denkliliğini ifade eden özel bir gösterim sağlamak için uygundur. Eğer $(a \bmod n) = (b \bmod n)$ ise $a \equiv b \pmod{n}$ diye yazabiliriz ve $a \bmod n$ ye göre b ye **denktir** deriz. Başka bir deyişle, $a \equiv b \pmod{n}$ eğer ve yalnız eğer a ve b , n ye bölündüklerinde aynı kalana sahiptirler. Benzer şekilde, $a \equiv b \pmod{n}$ eğer ve yalnız eğer n , $b - a$ nın bir bölenidir. Eğer $a, \bmod n$ ye göre b ye denk değilse $a \not\equiv b \pmod{n}$ diye yazabiliriz.

Polinomlar

Negatif olmayan d tam sayısı verilsin, **d inci dereceden n ye bağlı bir polinom** aşağıda verilen formdaki bir $p(n)$ fonksiyonudur:

$$p(n) = \sum_{i=0}^d a_i n^i$$

burada $a_0, a_1, \dots, a_d$ polinomun **katsayılarıdır** ve $a_d \neq 0$ dır. Bir polinom asimptotik olarak pozitifdir eğer ve yalnız eğer $a_d > 0$ dır. d inci dereceden asimptotik pozitif bir $p(n)$ polinomu için $p(n) = \Theta(n^d)$ dir. Herhangi bir $a \geq 0$ reel sayı sabiti için n^a fonksiyonu monoton artan ve $a \leq 0$ reel sayı sabiti için n^a monoton azalandır. Eğer bazı k sabitleri için $f(n) = O(n^k)$ ise $f(n)$ fonksiyonu **polinom sınırlıdır**, deriz.

Üslü ifadeler

Herhangi $a > 0$, m ve n reel sayıları için aşağıdaki eşitlikler vardır:

$$\begin{aligned} a^0 &= 1, \\ a^1 &= a, \\ a^{-1} &= 1/a, \\ (a^m)^n &= a^{mn}, \\ (a^m)^n &= (a^n)^m, \\ a^m a^n &= a^{m+n}. \end{aligned}$$

Tüm n ve $a \geq 1$ için, a^n n e göre monoton artandır. Pratikte, $0^0 = 1$ kabul edeceğiz.

Polinomların ve üslü ifadelerin büyüme oranlarını şu kurala göre ilişkilendirebiliriz: Tüm $a > 1$ ve b reel sayı sabitleri için

$$\lim_{n \rightarrow \infty} \frac{n^b}{a^n} = 0 \quad (3.10)$$

dır ki buradan da $n^b = o(a^n)$ dir.

Sonuç olarak tabanı 1 den kesin büyük olan bir üslü bir fonksiyon herhangi bir polinom fonksiyonundan daha hızlı büyür.

Doğal logaritmik fonksiyonun tabanı olan 2,71828... yi e ile gösterirsek, her x reel sayısı için

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots = \sum_{i=0}^{\infty} \frac{x^i}{i!} \quad (3.11)$$

dir. Burada “!”, daha sonra tanımlanacak olan faktöryel fonksiyonudur. Her bir x reel sayısı için şu eşitsizlik geçerlidir:

$$e^x \geq 1 + x \quad (3.12)$$

dir. Burada eşitlik sadece $x = 0$ için sağlanır. $|x| \leq 1$ olduğunda, şu yaklaşıma sahip oluruz:

$$1 + x \leq e^x \leq 1 + x + x^2. \quad (3.13)$$

$x \rightarrow 0$ iken e^x in $1 + x$ e yakınsaması uygundur:

$$e^x = 1 + x + \Theta(x^2).$$

(Bu eşitlikte, asimptotik gösterimi $x \rightarrow \infty$ yerine $x \rightarrow 0$ limit davranışını göstermektedir). Tüm x ler için

$$\lim_{n \rightarrow \infty} \left(1 + \frac{x}{n}\right)^n = e^x \quad (3.14)$$

dir.

Logaritmalar

Aşağıdaki gösterimleri kullanacağız:

$$\begin{aligned} \lg n &= \log_2 n && (\text{ikilik logaritma}), \\ \ln n &= \log_e n && (\text{doğal logaritma}), \\ \lg^k n &= (\lg n)^k && (\text{üstel logaritma}), \\ \lg \lg n &= \lg(\lg n) && (\text{bileşke logaritma}). \end{aligned}$$

Burada şu önemli hatırlatmayı yapalım: Logaritmik fonksiyonlar sadece bir sonrakı terime uygulanır yani $\lg n + k$ demek $(\lg n) + k$ demektir yoksa $\lg(n + k)$ demek değildir. Eğer $b > 1$ bir sabit ise, o zaman $n > 0$ için $\log_b n$ kesinlikle artandır.

Tüm $a > 0$, $b > 0$, $b \neq 1$, $c > 0$ ve n için

$$\begin{aligned} a &= b^{\log_b a}, \\ \log_c(ab) &= \log_c a + \log_c b, \\ \log_b a^n &= n \log_b a, \\ \log_b a &= \frac{\log_c a}{\log_c b}, \\ \log_b(1/a) &= -\log_b a, \\ \log_b a &= \frac{1}{\log_a b}, \\ a^{\log_b c} &= c^{\log_b a}, \end{aligned} \quad (3.15)$$

burada yukarıdaki her denklemde, logaritma tabanları 1 değildir.

Denklem (3.15) e göre, logaritmanın tabanını bir sabitten başka bir sabite değiştirmek logaritmanın sadece sabit çarpanını değiştirir, bu nedenle biz genellikle O -notasyonu gibi sabit çarpanın önemsenmediği yerlerde “lg n ” gösterimini kullanırız. Bilgisayar bilimciler için en doğal logaritma tabanı 2dir çünkü bir çok algoritma ve veri yapısı bir problem ikiye ayırma işlemini içerir.

$|x| < 1$ iken $\ln(1 + x)$ için basit bir seri açılımı vardır.

$$\ln(1 + x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \frac{x^5}{5} - \dots$$

Ayrıca $x > -1$ için eşitsizlikler mevcuttur:

$$\frac{x}{1+x} \leq \ln(1+x) \leq x \quad (3.17)$$

Bu eşitlik yalnız $x = 0$ için sağlanır.

Eğer bazı k sabitleri için $f(n) = O(\log^k n)$ ise $f(n)$ fonksiyonu **polilogaritmik sınırlıdır** denir. Polinomların ve polilogaritmaların büyümesini denklem (3.10) da n yerine $\lg n$ ve a yerine 2^a yazarak ilişkilendirebiliriz.

$$\lim_{n \rightarrow \infty} \frac{\lg^b n}{(2^a)^{\lg n}} = \lim_{n \rightarrow \infty} \frac{\lg^b n}{n^a} = 0.$$

Bu limitten şu sonucu çıkarırız: Herhangi bir $a > 0$ sabiti için

$$\lg^b n = o(n^a)$$

dir. Buradan, herhangi pozitif polinom fonksiyonun herhangi bir polilogaritmik fonksiyondan daha hızlı büyüdüğü anlaşılır.

Faktöriyeler

$n!$ (n faktöriyel diye okunur) $n \geq 0$ tam sayıları için şöyle tanımlıdır.:

$$n! = \begin{cases} 1 & n = 0 \text{ ise,} \\ n \cdot (n-1)! & n > 0 \text{ ise.} \end{cases}$$

Yani, $n! = 1 \cdot 2 \cdot 3 \cdots n$ dir.

Faktöriyel çarpımında n terimli bir faktöriyelde en büyük çarpan n olduğu için faktöriyel fonksiyonunun zayıf bir üst sınırı $n^n \geq n!$ dir. e doğal logaritma tabanı olmak üzere **Stirling'in Yaklaşımı** bize daha sıkı bir alt ve üst sınır verir.

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right) \quad (3.18)$$

burada e , bize aynı zamanda bir sıkı üst sınır ve bir alt sınır veren doğal logaritmanın tabanıdır. Alistırma 3.2-3 te

$$\begin{aligned} n! &= o(n^n) , \\ n! &= \omega(2^n) , \\ \lg(n!) &= \Theta(n \lg n) , \end{aligned} \quad (3.19)$$

ifadelerinin kanıtları soruluyor. Bunun için Stirling's Yaklaşımı yardımcı olacaktır. Aşağıdaki denklem tüm $n \geq 1$ değerleri için sağlanır:

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n e^{\alpha_n} \quad (3.20)$$

burada

$$\frac{1}{12n+1} < \alpha_n < \frac{1}{12n} . \quad (3.21)$$

Fonksiyonel iterasyon

$f^{(i)}(n)$ gösterimini $f(n)$ fonksiyonunun n nin başlangıç değerine i kere tekrarla uygulandığını ifade etmek için kullanırız. Kuramsal olarak tanımlamak gerekirse; $f(n)$ reel sayılarda tanımlı bir fonksiyon olsun. Negatif olmayan tam sayısı için, özyinelemeli olarak şöyle yazabiliriz:

$$f^{(i)}(n) = \begin{cases} n & i = 0 \text{ ise,} \\ f(f^{(i-1)}(n)) & i > 0 \text{ ise.} \end{cases}$$

Örneğin, $f(n) = 2n$ ise $f^{(i)}(n) = 2^i n$ dir.

İteratif logaritmik fonksiyon

$\lg^* n$ ("log yıldız n " diye okunur) gösterimini iteratif logaritmaların göstermek için kullanırız. $\lg^{(i)} n, f(n) = \lg n$ için yukarıdaki gibi tanımlansın. Pozitif olmayan bir logaritmanın tanımsız olması nedeniyle eğer sadece $\lg^{(i-1)} n > 0$ ise $\lg^{(i)} n$ tanımlanır. Burada $\lg^{(i)} n$, i (n nin başlangıç değerine i kez logaritmik fonksiyon uygulanması), $\lg^i n$ (n nin logaritmasının i kuvveti) ile karıştırmamayın. O halde, iteratif logaritma fonksiyonu aşağıdaki gibi tanımlanabilir:

$$\lg^* n = \min \{i \geq 0: \lg^{(i)} n \leq 1\}.$$

İteratif logaritma çok yavaş büyür:

$$\begin{aligned} \lg^* 2 &= 1, \\ \lg^* 4 &= 2, \\ \lg^* 16 &= 3, \\ \lg^* 65536 &= 4, \\ \lg^* (2^{65536}) &= 5. \end{aligned}$$

Keşfedilebilir kainatta atomların sayısı yaklaşık 10^{80} olduğundan ki bu sayı 2^{65536} sayısından oldukça düşüktür, $\lg^* n > 5$ olacak şekilde n büyüklüğündeki bir girişle zor karşılaşıyoruz.

Fibonacci sayıları

Fibonacci sayılarını aşağıdaki özyineleme ile tanımlarız:

$$\begin{aligned} F_0 &= 0, \\ F_1 &= 1, \\ F_i &= F_{i-1} + F_{i-2} \end{aligned} \quad (3.22)$$

Yani, her bir Fibonacci sayısı önceki iki tanesinin toplamıdır. O halde dizi şöyle olur:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

Fibonacci sayıları aşağıdaki denklemin kökleri olan ϕ *altın oran* ve onun $\hat{\phi}$, eşleniği ile ilişkilidir.

$$x^2 = x + 1 \quad (3.23)$$

ve verilen formüle göre şu değerleri buluruz: (Alıştırma 3.2-6 ya bakınız)

$$\begin{aligned} \phi &= \frac{1 + \sqrt{5}}{2} \\ &= 1.61803 \dots, \\ \hat{\phi} &= \frac{1 - \sqrt{5}}{2} \\ &= -.61803 \dots \end{aligned} \quad (3.24)$$

Özel olarak ;

$$F_i = \frac{\phi^i - \hat{\phi}^i}{\sqrt{5}}$$

Bu bağıntıyı tümevarımla kanıtlayabiliriz. (Alıştırma 3.2-7). $|\hat{\phi}| < 1$ olduğundan bu eşitsizlik

$$\begin{aligned} \frac{|\hat{\phi}^i|}{\sqrt{5}} &< \frac{1}{\sqrt{5}} \\ &< \frac{1}{2}, \end{aligned}$$

$$F_i = \left\lfloor \frac{\phi^i}{\sqrt{5}} + \frac{1}{2} \right\rfloor$$

gerektirir, bu bize, $\phi^i / \sqrt{5}$ değerine en yakın yuvarlak tam sayının i . Fibonacci sayısı F_i ye eşit olduğunu gösterir.. Sonuç olarak Fibonacci sayılarının üstel büyüdüklərini anlamış oluruz.

Alıřtırmalar

3.2-1

Eğer $f(n)$ ve $g(n)$ monoton artan fonksiyonlar ise $f(n) + g(n)$ ve $f(g(n))$ in de monoton artan olduğunu, ayrıca, $f(n)$ ve $g(n)$ negatif değilse $f(n) \cdot g(n)$ nin de monoton artan olduğunu kanıtlayın.

3.2-2

(3.16) denklemini kanıtlayın.

3.2.-3

(3.19) denklemini kanıtlayın. Ayrıca $n! = \omega(2^n)$ ve $n! = o(n^n)$ olduğunu ispat edin.

3.2-4 *

$\lceil \lg n \rceil!$ fonksiyonu polinom sınırlı mıdır? $\lceil \lg \lg n \rceil!$ fonksiyonu polinom sınırlı mıdır?

3.2-5 *

$\text{Lg}(\lg^* n)$ mi yoksa $\lg^*(\lg n)$ mi asimptotik olarak daha büyüktür?

3.2-6

Altın oran ϕ ve eşleniğı $\hat{\phi}$ nin $x^2 = x + 1$ i sağladığını kanıtlayın.

3.2-7

ϕ = altın oran, $\hat{\phi}$ = altın oranın eşleniğı i inci Fibonacci sayısının

$$F_i = \frac{\phi^i - \hat{\phi}^i}{\sqrt{5}}$$

eşitliğini sağladığını tümevarımla kanıtlayın.

3.2-8

$k \ln k = \Theta(n)$ ın $k = \Theta(n / \ln n)$ yi gerektirdiğini kanıtlayın.

Problemler

3-1 Polinomların asimptotik davranışı

$a_d > 0$ olmak üzere

$$p(n) = \sum_{i=0}^d a_i n^i$$

ve k da bir sabit olsun. Asimptotik gösterimlerin tanımlarını kullanarak aşağıdaki özellikleri kanıtlayın.

- a. $k \geq d$ ise, o zaman $p(n) = O(n^k)$.
- b. $k \leq d$ ise, o zaman $p(n) = \Omega(n^k)$.
- c. $k = d$ ise, o zaman $p(n) = \Theta(n^k)$.
- d. $k > d$ ise, o zaman $p(n) = o(n^k)$.
- e. $k < d$ ise, o zaman $p(n) = \omega(n^k)$.

3-2 Asimptotik büyümelerin karşılaştırılması

Aşağıdaki tabloda (A, B) deki ifadeleri, A nın mı yoksa B nin mi O , o , Ω , ω veya Θ olduğunu belirtin. $k \geq 1$, $\epsilon > 0$ ve $c > 1$ sabitler olduğun varsayın. Cevabınız her kutucuğa “evet” ya da “hayır” şeklinde yazarak olmalıdır.

	A	B	O	o	Ω	ω	Θ
a.	$\lg^k n$	n^ϵ					
b.	n^k	c^n					
c.	$\sqrt{n}$	$n^{\sin n}$					
d.	2^n	$2^{n/2}$					
e.	$n^{\lg c}$	$c^{\lg n}$					
f.	$\lg(n!)$	$\lg(n^n)$					

3-3 Asimptotik büyüme oranlarının sıralanması

- a. Aşağıdaki fonksiyonları büyüme sırasına göre dizin yani, $g_1 = \Omega(g_2)$, $g_2 = \Omega(g_3)$, $\dots$, $g_{29} = \Omega(g_{30})$ nı sağlayan $g_1, g_2, \dots, g_{30}$ fonksiyonlarını düzenleyin. Listenizi, $f(n)$ ve $g(n)$ fonksiyonları aynı sınıftadır eğer ve yalnız eğer $f(n) = \Theta(g(n))$ şartını sağlayacak şekilde denk sınıflara ayırın.

$\lg(\lg^* n)$	$2^{\lg^* n}$	$(\sqrt{2})^{\lg n}$	n^2	$n!$	$(\lg n)!$
$(\frac{3}{2})^n$	n^3	$\lg^2 n$	$\lg(n!)$	2^{2^n}	$n^{1/\lg n}$
$\ln \ln n$	$\lg^* n$	$n \cdot 2^n$	$n^{\lg \lg n}$	$\ln n$	1
$2^{\lg n}$	$(\lg n)^{\lg n}$	e^n	$4^{\lg n}$	$(n+1)!$	$\sqrt{\lg n}$
$\lg^*(\lg n)$	$2^{\sqrt{2^{\lg n}}}$	n	2^n	$n \lg n$	$2^{2^{n+1}}$

- b.** (a) kısmındaki her $g_i(n)$ için $f(n)$ in ne $O(g_i(n))$ ne de $\Omega(g_i(n))$ olduğu negatif olmayan bir $f(n)$ örneği verin.

3-4 Asimptotik gösterim özellikleri

$f(n)$ ve $g(n)$ asimptotik pozitif fonksiyonlar olsun. Aşağıdaki ifadeleri kanıtlayın ya da geçersiz olduğunu gösterin.

- a.** $f(n) = O(g(n))$ gerektirir $g(n) = O(f(n))$
- b.** $f(n) + g(n) = \Theta(\min(f(n), g(n)))$.
- c.** Yeterince büyük tüm n ler için $f(n) = O(g(n))$ gerektirir $\lg(f(n)) = O(\lg(g(n)))$, $\lg(g(n)) \geq 1$ ve $f(n) \geq 1$
- d.** $f(n) = O(g(n))$ gerektirir $2^{f(n)} = O(2^{g(n)})$.
- e.** $f(n) = O((f(n))^2)$.
- f.** $f(n) = O(g(n))$ gerektirir $g(n) = \Omega(f(n))$.
- g.** $f(n) = \Theta(f(n/2))$.
- h.** $f(n) + o(f(n)) = \Theta(f(n))$.

3-5 O ve Ω ile ilgili çeşitlilikler

Bazı yazarlar Ω ın tanımını bizim yaptığımızdan biraz farklı yaparlar; bu alternatif tanımı $\tilde{\Omega}$ (sonsuz omega) ile göstereyim. Sonsuz sayıdaki n tam sayıları için $f(n) \geq c g(n) \geq 0$ şartını sağlayan bir c pozitif sabiti var ise $f(n) = \tilde{\Omega}(g(n))$ diyoruz.

- a.** Her asimptotik negatif olmayan iki $f(n)$ ve $g(n)$ fonksiyonunu ya $f(n) = O(g(n))$ ya $f(n) = \tilde{\Omega}(g(n))$ ya da her ikisi birden olduğunu, eğer $\tilde{\Omega}$ yerine Ω kullanırsak bunun doğru olmadığını gösterin.

- b.** Programların işlem sürelerini ifade ederken Ω yerine $\tilde{\Omega}$ kullanmanın potansiyel dezavantajlarını söyleyin.

Bazı yazarlar da O u biraz farklı biçimde tanımlarlar, bu farklı tanım için O' ı kullanalım. $f(n) = O'(g(n))$ eğer ve yalnız eğer $|f(n)| = O(g(n))$ dır deriz.

- c.** Teorem 3.1 de O yerine O' koyar ve Ω yı değiştirmez isek “eğer ve yalnız eğer” ın her iki yönü ne olur?

Bazı yazarlar da logaritmik çarpanların ihmal ederek O u $\tilde{O}$ (yumuşak-oh) diye tanımlarlar.

$\tilde{O}(g(n)) = \{f(n) : \text{her } n \geq n_0 \text{ için } 0 \leq f(n) \leq c g(n) \lg^k(n) \text{ şartını sağlayan } c, k \text{ ve } n_0 \text{ pozitif sabitleri vardır.}\}$

- d.** $\tilde{\Omega}$ ve $\tilde{\Theta}$ yı benzer şekilde tanımlayın. Teorem 3.1 i benzer şekilde yorumlayın.

3-6 İteratif fonksiyonlar

$\lg^*$ içinde kullanılan $*$ tekrar operatörünü reel sayılar üzerinde her $f(n)$ monoton artan fonksiyonuna uygulayabiliriz. Her $c \in \mathbb{R}$ sabiti için, f_c^* yı şu şekilde tanımlarız:

$$f_c^*(n) = \min \{i \geq 0 : f^{(i)}(n) \leq c\}$$

her durum için iyi tanımlanmış olması gerekmez. Başka bir deyişle $f_c^*(n)$ niceliği, kendi argümentini c ye ya da daha aza düşürmeyi gerektiren f fonksiyonunun uygulama tekrarı sayısıdır.

Aşağıdaki her $f(n)$ fonksiyonu ve c sabiti için $f_c^*(n)$ de mümkün olabildiğince sıkı sınır bulun.

	$f(n)$	c	$f_c^*(n)$
a.	$n - 1$	0	
b.	$\lg n$	1	
c.	$n/2$	1	
d.	$n/2$	2	
e.	$\sqrt{n}$	2	
f.	$\sqrt{n}$	1	
g.	$n^{1/3}$	2	
h.	$n / \lg n$	2	

Bölüm notları

Knuth [209] O -notasyonunun kökenini P. Bachmann'ın 1982'deki sayılar teorisi maklesine dayandırıyor. o -notasyonu E. Landau tarafından asal sayıların yayılmasını tartışmak için 1909 yılında keşfedildi. Üst ve alt sınırların her ikisi için literatürdeki uygulamasının popüler ama teknik olarak yarım yamalak olan O -notasyonunun kullanımını düzeltmek için Knuth [213], Θ ve Ω notasyonlarını kullanmayı desteklemiştir. Θ -notasyonu teknik olarak daha iyi olmasına rağmen çoğu insan O -notasyonunu kullanmaya devam etmiştir. Asimptotik gösterimlerin tarihçesi ve gelişimi üzerindeki tartışmalar Knuth [209,213] ve Brassard ve Bratley [54] in çalışmalarında mevcuttur.

Farklı tanımlar ortak bir dilde uzlaşsalar dahi tüm yazarlar asimptotik gösterimleri aynı şekilde tanımlamamıştır. Bazı farklı tanımlar mutlak değerleri uygun bir şekilde sınırlı olması halinde *asimptotik negatif olmayan* şartını sağlamayan fonksiyonları da kapsar.

(3.20) denklemi Robbins [297] tendir. Elementer matematiksel fonksiyonların diğer özellikleri iyi matematik referansları olan Abramowitz ve Stegun ya da Zwillinger, ya da Apostol ya da Thomas gibi kalkülüs (calculus) kitaplarında bulunabilir. Knuth [209] ve Graham, Knuth ve Patashanik bilgisayar bilminde kullanılan soyut matematik üzerinde zengin materyallere sahiptir.

Kısım 2.3.1 de birleştirerek sıralamanın, böl-ve-yönet paradigmasına nasıl bir örnek oluşturduğunu gördük. Hatırlarsak böl-ve-yönet de, özyinelemenin her bir kademesine üç adımı da uygulayarak, bir problemi özyinelemeli olarak çözüyoruz:

Problemi, aynı problemin daha küçük örnekleri olan, birkaç küçük altprobleme **böl**.

Altproblemleri özyinelemeli olarak çözüp, onları **yönet**. Eğer altproblemlerin boyutları yeterince küçükse, o halde altproblemleri direkt olarak çöz.

Altproblemlerin çözümlerini ana problemin çözümü için **birleştir**.

Altproblemlerin özyinelemeli olarak çözümü için yeterince büyük olması halinde, buna **özyinelemeli durum** diyoruz. Altproblemler özyineleme yapılmayacak kadar yeterince küçük olduğunda, özyinelemeye ‘dibe vurmuş’ deriz ve bu sayede **temel duruma** ulaşmış oluruz. Bazen, problemin daha küçük parçası olan altproblemlere ek olarak, ana probleme çok fazlada benzemeyen altproblemleri çözmemiz gerebilir. Bu problemleri çözmeyi birleştirme adımı olarak ele alabiliriz.

Bu bölümde, böl-ve-yönet temelli bir çok algoritma göreceğiz. Bu algoritmalardan birincisi maksimum-altdizi problemini; sayıların bir dizisini girdi olarak alarak ve değerlerinin toplamı maksimum olan ardışık altdiziyi belirleyerek çözmekte. Ayrıca, $n \times n$ tipindeki matrislerin çarpımı için iki tane böl-ve-yönet algoritması göreceğiz. Bunlardan biri, yani kare matrisleri doğrudan yöntemle çarpmaktan daha iyi olmayan $\Theta(n^3)$ zamanda çalışmaktadır. Diğeri ise, Strassen’in algoritmasını kullanarak, doğrudan yöntemi asimptotik olarak geçebilmekte ve $O(n^{2.81})$ zamanda çalışmaktadır.

Özyinelemeler

Özyinelemeler, böl-ve-yönet değerler dizisi ile el ele çalışır, çünkü özyinelemeler bize böl-ve-yönet algoritmasının çalışma süresini karakterize edebilmemiz için

doğal bir yol sunar. Bir **özyineleme**, küçük girdileri üzerindeki değerleri cinsinden bir fonksiyon tanımlayan bir denklem veya eşitsizliktir. Örneğin, bölüm 2.3.2 de çözümünün $T(n) = \Theta(n \lg n)$ olarak iddia ettiğimiz,

$$T(n) = \begin{cases} \Theta(1) & n = 1 \text{ ise,} \\ 2T(n/2) + \Theta(n) & n > 1 \text{ ise,} \end{cases} \quad (4.1)$$

özyinelemesi ile MERGE-SORT yönteminin $T(n)$ ile gösterilen en kötü-durum çalışma süresinden bahsetmiştik.

Özyinelemeler bir çok biçimde olabilir. Örneğin, bir özyineleme algoritması altproblemleri, $2/3$ den $1/3$ e farklı parçalara ayırabilir. Eğer, böl ve birleştir adımları doğrusal zaman tutarsa, böyle bir algoritma $T(n) = T(2n/3) + T(n/3) + \Theta(n)$ özyinelemesini oluşturur.

Altproblemler, ana problemin boyutunun sabit kesiri olmak zorunda değildir. Örnek olarak, doğrusal aramanın özyinelemeli modeli (bkz. Alıştırma 2.1-3), ana probleminden bir eleman daha az içeren altproblemi oluşturur. Her özyineleme çağırımı, sabit süre artı onun oluşturduğu özyineleme süresi kadar zaman alır, yani $T(n) = T(n-1) + \Theta(1)$ dir.

Bu bölüm, özyinelemeleri çözmek—yani çözümde asimptotik “ Θ ” ya da “ O ” sınırlar elde etmek için üç yöntem sunar.

- **Yerine koyma yöntemi**, bir sınır tahmini yapıp bu tahminin doğruluğunu göstermek için matematiksel tümevarım kullanacağız.
- **Özyineleme-ağaç yöntemi**, özyinelemeyi ağaca dönüştürür ve bu ağaçtaki her bir dal, özyinelemenin çeşitli seviyelerinde ortaya çıkan maliyeti gösterir.
- **Ana yöntem**, $a \geq 1$ $b > 1$ ve $f(n)$ verilen bir fonksiyon olmak üzere,

$$T(n) = aT(n/b) + f(n) \quad (4.2)$$

biçimindeki özyinelemeler için sınırlar elde eder. Bu tür özyinelemeler sıklıkla ortaya çıkar. (4.2) denklemindeki gibi bir özyinelemeler bir altproblem oluşturan böl-ve-yönet algoritmalarını karakterize eder. Bu altproblemlerin her biri ana problemin $1/b$ si kadar boyuta sahiptir ve böl ve birleştir adımları ile birlikte $f(n)$ zaman alır.

Ana yöntemi kullanmak için, üç durumu hatırlamanız gerekmektedir, fakat bunu bir kere yaptıktan sonra kolayca bir çok basit özyinelemeler için asimptotik sınırlar belirleyebilecek hale geleceksiniz. Ana yöntemini, maksimum-alt dizisi probleminin ve matris çarpımı probleminin böl-ve-yönet algoritması ve bunun yanı sıra bu kitapta bahsedilen ve gene böl-ve-yönet algoritmasına dayanan diğer algoritmalar için çalışma sürelerini belirlemek için kullanacağız.

Bazen, $T(n) \leq 2T(n/2) + \Theta(n)$ gibi eşitsizliklerden ziyade eşitlik olmayan özyinelemeleri görebiliriz. Çünkü böyle bir özyineleme $T(n)$ de sadece bir üst sınırı belirttiğinden, Θ -notasyonundan ziyade O -notasyonunu kullanan çözümünü ifade edecektir. Benzer şekilde, eğer eşitsizlik $T(n) \geq 2T(n/2) + \Theta(n)$ ye dönüştürülseydi, o zaman özyineleme $T(n)$ de sadece bir alt sınırı verdiğinden, çözümünde Ω -notasyonunu kullanacaktık.

Özyinelemelerde teknikler

Pratikte, özyinelemeleri belirlerken ve çözerken bazı teknik detayları ihmal ederiz. Örneğin, n tek sayı olmak üzere eğer n eleman üzerinde MERGE-SORT çağırdığımızda, bunu $\lfloor n/2 \rfloor$ ve $\lceil n/2 \rceil$ ölçülü altproblemlerle sonuçlandırırız. Ölçülerin hiçbiri $n/2$ olamaz, çünkü n tek sayı olduğundan $n/2$ bir tam sayı olamaz. Teknik olarak, özyineleme MERGE-SORT un en kötü-durum çalışma zamanının gerçekte

$$T(n) = \begin{cases} \Theta(1) & n = 1 \text{ ise,} \\ T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + \Theta(n) & n > 1 \text{ ise,} \end{cases} \quad (4.3)$$

olduğunu tanımlar.

Sınır koşulları, genelde bizim gözardı ettiğimiz bir başka ayrıntı sınıfını temsil eder. Bir algoritmanın sabit-ölçülü girdi üzerindeki çalışma zamanının sabit olmasından dolayı, algoritmaların çalışma zamanlarından ortaya çıkan özyinelemeler yeterince küçük n ler için genellikle $T(n) = \Theta(1)$ dir. Sonuç olarak, uygunluk için, özyinelemelerin sınır koşullarının ifadelerini genellikle ihmal edeceğiz ve $T(n)$ nin küçük n için sabit olduğunu kabul edeceğiz. Örneğin, çoğunlukla (4.1) özyinelemesini, küçük n ler için açıkça değerler vermeden

$$T(n) = 2T(n/2) + \Theta(n) \quad (4.4)$$

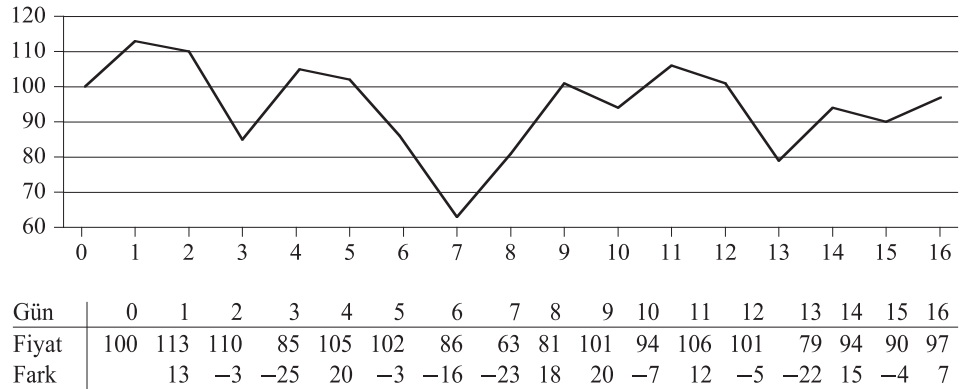
olarak ifade edeceğiz. Bunun sebebi ise, $T(1)$ değerini değiştirmenin özyinelemenin tam çözümünün değiştirmesine rağmen, çözümün genel olarak bir sabit çarpandan daha fazla değişmemesi olup, böylece büyüme derecesi değişmemektedir.

Özyinelemeleri belirlerken ve çözerken, çoğunlukla zeminleri, tavanları ve sınır değerleri ihmal ederiz. Bu ayrıntılara takılmadan ilerleyip, daha sonra bunların bir mesele teşkil edip etmeyeceğini belirleriz. Genelde olmasa da, mesele teşkil ettiğinde bunu bilmeniz gerekmektedir. Tecrübe bu konuda yardımcı olur, bu yüzden böl-ve-yönet algoritmasını karakterize eden birçok özyinelemenin asimptotik sınırlarına etki etmeyen bu şekildeki ayrıntıları belirleyen bazı teoremler yapınız. Bu bölümde, her ne kadar bu ayrıntıların bazılarını değineceğiz ve özyineleme çözüm yöntemlerinin ince detaylarını açıklayacağız.

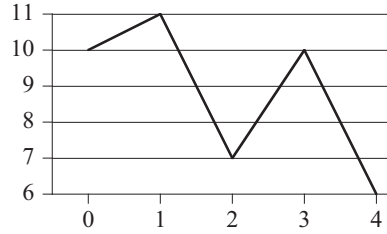
4.1 Maksimum-alttdizi problemi

Bir an için farzet ki, sana Değişken Kimyasal Şirketine yatırım yapma imkanı sunuldu. Şirketin ürettiği kimyasallar gibi, şirketin hisse senedi de oldukça değişken olsun. Sana stoktan bir ünite sadece bir kez satın almana ve daha sonraki bir zamanda satabilmene izin verildi. Satın alma ve satış yapmalar, ticaret gün sonunda sona eriyor. Bu kısıtlamayı telafi etmek için, ilerde hisse senet fiyatlarının ne olacağını öğrenmene izin verildi. Senin amacın kazancını en fazla yapabilmek. Şekil 4.1 de 17-günlük periyoda ait hisse bedelleri gösterilmektedir. 0. günden itibaren, hisse bedeli \$100 olduğunda, herhangi bir zamanda hisse alabilirsin. Elbette, sen kazancının en fazla yapmak için, “ucuz al, pahalı sat” yani en düşük değerde alıp, en yüksek fiyattan satmak isteyeceksin. Fakat malesef, verilen bir periyotta en düşük fiyattan alıp en yüksek fiyattan satman mümkün olmayabilir. Şekil 4.1 de, en düşük fiyata 7. günün sonunda, 1. günün sonundaki en yüksek fiyattan sonra ulaşmaktadır.

En düşük fiyattan satın alıp, en yüksek fiyattan satmanın kazancını en fazla yapabileceğini her zaman düşünmelisin. Örneğin, Şekil 4.1 de, 7. gün sonunda en düşük fiyattan satın alarak kazancımızı en yüksek hale getirebilirdik. Eğer bu strateji her zaman çalışırsa, kazancımızı nasıl en yüksek hale getirmeyi belirlemek kolay olacaktır: en yüksek ve en düşük değerleri bul, en yüksek fiyattan sola doğru çalışarak daha önlardaki en düşük fiyatı bul, en düşük fiyattan sağa doğru çalışarak en daha sonralardaki yüksek fiyatı bul, farkı en yüksek olan ikiliyi al. Şekil 4.2 basit bir karşıt örnek göstermektedir,



Şekil 4.1 Değişken Kimyasal Şirketi’nde hisse senetlerinin alışverişe kapanışından sonraki 17-günlük periyodu hakkında bilgi. Tablonun yatay eksenı günü göstermekte ve düşey eksen fiyatı göstermekte. Tablonun en alt satır ise bir önceki gün ile fiyat farkını göstermekte.



Gün	0	1	2	3	4
Fiyat	10	11	7	10	6
Fark		1	-4	3	-4

Şekil 4.2 En büyük kazancın her zaman en düşük fiyatta başlamadığını veya en yüksek fiyatta bitmediğini gösteren bir örnek. Yine, yatay eksen günü, düşey eksen fiyatı göstermektedir. Burada, hissenin \$3 lık en büyük kazanç 2. gün sonunda satın alınıp 3. gün sonunda satılması ile elde ediliyor. 2. günün sonundaki \$7 lık fiyat tüm dönemin en düşük fiyatı olmayıp, 3. gün sonundaki \$10 lık fiyat ise tüm dönemin en yüksek fiyatı değildir.

böylece görülüyor ki en yüksek kazanç bazen ne en düşük fiyattan ne de en yüksek fiyattan satma ile gelmeyebiliyor.

Bir kaba-kuvvet çözümü

Bu probleme rahatlıkla bir kaba-kuvvet çözüm oluşturabiliriz: sadece satın alma gününün satış gününden önce geldiği tüm mümkün alış ve satış gün ikililerini dene. Bu şekilde n günlük bir periyot $\binom{n}{2}$ sayıda gün ikililerine sahiptir. $\binom{n}{2}$ nin $\Theta(n^2)$ olmasından dolayı, ümit edebileceğimiz en iyi durum, her bir gün ikilisini sabit zamanda hesaplamaktır, bu yaklaşım $\Omega(n^2)$ zaman alıcaktır. Daha iyisini yapabilir miyiz?

Bir dönüşüm

$o(n^2)$ çalışma zamanına sahip bir algoritma tasarlamak için, girdiye biraz farklı bir yoldan bakacağız. İlk günden son güne kadar olan net değişimin en fazla olduğu günler dizisi bulmak istiyoruz. Günlük fiyatlara bakmak yerine, $i - 1$. gün sonundaki ile i . gün sonundaki fiyatlar arasındaki farkın i . gün fiyat değişimi olarak kabul edelim. Şekil 4.1 alt satırdaki günlük değişimleri göstermektedir. Eğer bu satırı bir A dizisi olarak kabul edelim, bu şekil 4.3 de gösterilmektedir, bu durumda değerleri toplamı en büyük olacak şekilde A nın boş olmayan, ardışık alt dizisini bulmamız gerekir. Bu alt diziyi **maksimum alt dizi** olarak adlandıracacağız. Örneğin, şekil 4.3 deki dizide, toplamın 43 olmasından dolayı $A[1..16]$ nın maksimum alt dizisi $A[8..11]$ dir. Böylece hisseyi 8. günden önce (yani 7. gün sonu) almak isteyeceksin ve 11. gün sonunda satıp hisse başı \$7 lık bir kazanç elde edeceksin.

İlk bakışta bu dönüşümün bir yardımı yok gibi gözükmektedir. Bizim hala n günlük bir periyot için $\binom{n-1}{2} = \Theta(n^2)$ alt dizileri gözden geçirmemiz gerekmektedir. Alıştırma 4.1-2 de sana,

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
A	13	-3	-25	20	-3	-16	-23	18	20	-7	12	-5	-22	15	-4	7

maksimum-alt dizisi

Şekil 4.3 Maksimum-alt dizisi problemi olarak hisse fiyatları değişimi. Burada, $A[8..11]$ alt dizisi, 43 toplamı ile birlikte, A dizisinin bütün ardışık alt dizilerinin en büyük toplamına sahiptir.

bir alt dizinin fiyatının hesaplanma süresinin alt dizinin uzunluğuyla orantılı olmasına rağmen, tüm $\Theta(n^2)$ alt dizisi toplamaları hesaplanırken, bir önceki alt dizide hesaplanan toplam değerinin eklenmesi ile her bir alt dizisi toplamı $O(1)$ zamanda hesaplanacak şekilde düzenlenerek, kaba-kuvvet çözümünün $\Theta(n^2)$ zaman aldığı soruluyor.

O halde, maksimum-alt dizisi problemi için daha verimli bir çözüm arayalım. Bunu yaparken, “maksimum-alt dizisi” den ziyade “bir maksimum-alt dizisi” den bahsedelim. Çünkü maksimum toplama sahip birden fazla alt dizisi olabilir.

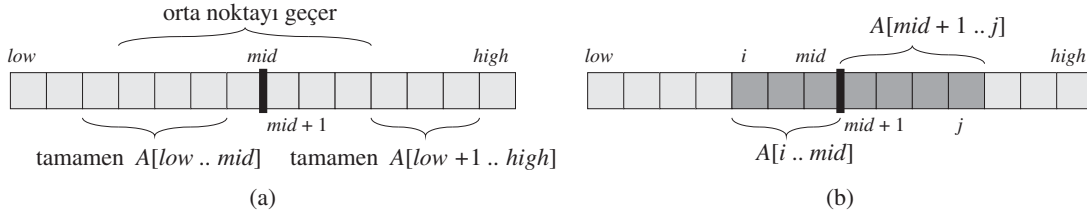
Maksimum-alt diziler eğer dizi bazı negatif sayıları içerdiğinde ilgi çekici olmaktadır. Eğer dizi tamamen negatif olmayan sayı içeriklerine sahip ise, tüm dizi en büyük toplamı vereceğinden dolayı maksimum-alt dizisi problemi zorlu bir durumu temsil etmez.

Böl-ve-yönet yöntemi ile çözüm

Şimdi maksimum-alt dizisi problemini çözmek için böl-ve-yönet tekniğini nasıl kullanabileceğimizi düşünelim. $A[mid..high]$ alt dizisinin maksimum-alt dizisini bulmak istediğimizi farzedelim. Böl-ve-yönet bize alt diziyi mümkün olduğunca eşit iki alt diziyeye bölmemizi önerir. Yani, *orta* diye adı verdiğimiz $A[low..mid]$ ve $A[mid + 1..high]$ alt dizilerini oluşturmak için orta nokta bulalım. Şekil 4.4(a) da gösterildiği üzere, $A[low..mid]$ nın herhangi $A[i..j]$ ardışık dizisi aşağıdaki yerlerden sadece birinde bulunmalıdır:

- tamamen $A[low..mid]$ alt dizisinde, böylece $low \leq i \leq j \leq mid$,
- tamamen $A[mid + 1..yüksek]$ alt dizisinde, böylece $mid < i \leq j \leq high$, ya da
- orta noktayı geçer, böylece $low \leq i \leq mid < j \leq high$.

Buna göre, $A[low..high]$ nın maksimum alt dizisi tam olarak bu yerlerden birinde bulunmalıdır. Aslında, $A[düşük..yüksek]$ nın maksimum alt dizisi, tamamen $A[low..mid]$ nın içinde, tamamen $A[mid + 1..high]$ nın içinde yada orta noktayı geçen tüm alt diziler üzerinde en büyük toplama sahip olmalıdır. $A[low..mid]$ nın ve $A[mid + 1..high]$ nın maksimum alt dizilerini yinelemeli olarak özyinelemeli.



Şekil 4.4 (a) $A[low..high]$ nin mümkün olan altdizi konumu: tamamen $A[low..mid]$ nin içinde, tamamen $A[mid + 1..high]$ nin içinde ya da mid , orta nokta noktasını geçen. (b) Orta noktayı geçen $A[low..high]$ nin herhangi altdizisi $A[i..mid]$ ve $A[mid + 1..j]$ yi içerir, burada, $low \leq i \leq mid$ ve $mid < j \leq high$ dir.

Böylece, tamamen orta noktayı geçen maksimum altdiziyi bulmak kalır ve bu üçlünden en büyük toplama sahip altdiziyi alırız.

Orta noktayı geçen maksimum altdiziyi $A[low..high]$ altdizi ölçüsüne doğrusal zamanda kolaylıkla bulabiliriz. Bu problem bizim asıl problemimizin küçük örneği değildir, çünkü ona, seçtiği altdizinin orta noktayı geçmiş olması gibi bir kısıtlama eklenmiştir. Şekil 4.4 (b) de görüldüğü gibi, orta noktayı geçen herhangi bir altdizi $low \leq i \leq mid$ ve $mid < j \leq high$ olmak üzere, $A[i..mid]$ ve $A[mid + 1..j]$ iki altdizisinden oluşmuştur. Böylece, sadece $A[i..mid]$ ve $A[mid + 1..j]$ formunda maksimum altdiziler bulup onları birleştirmemiz gerekmektedir. FIND-MAX-CROSSING-SUBARRAY yöntemi bir maksimum altdizideki değer toplamı ile birlikte, A dizisini ve low , mid ve $high$ indislerini girdi olarak alır ve orta noktayı geçen bir maksimum altdiziyi ayıran indisleri içeren sıralı çokluya dönüştür.

FIND-MAX-CROSSING-SUBARRAY (A , low , mid , $high$)

```

1  left-sum =  $-\infty$ 
2  sum = 0
3  for i = mid downto low
4      sum = sum + A[i]
5      if sum > left-sum
6          left-sum = sum
7          max-left = i
8  right-sum =  $-\infty$ 
9  sum = 0
10 for j = mid + 1 to high
11     sum = sum + A[j]
12     if sum > right-sum
13         right-sum = sum
14         max-right = j
15 return (max-left, max-right, left-sum + right-sum)
```

Bu yöntem şu şekilde işler, 1-7 satırları için $A[low..mid]$ sol yarısının maksimum altdizisini bul. Bu altdizinin $A[mid]$ ni içermesi gerektiğinden, 3-7 satırlarının **for** döngüsü mid de i indisi ile başlayıp low a doğru çalışır, böylece hesapladığı tüm altdiziler $A[mid]$ formundadır. 1-2 satırları, o ana kadar ki bulunan en büyük toplamı tutan $left-sum$ değişkenlerini başlatır ve sum , $A[i..mid]$ deki içeriklerin toplamını tutar. Satır 5 de, $left-sum$ dan daha büyük değere sahip bir $A[i..mid]$ altdizisi bulduğumuz zaman, $left-sum$ ı bu altdizinin 6.satırdaki toplamını yenileriz, ve 7. satırda $max-left$ değişkenini bu i indisini kaydetmek için yenileriz. 8-14 satırları, $A[mid + 1..high]$ sağ yarısı için aynı şekilde çalışır. Burada, 10-14 **for** döngüsü satırları j indisini $mid + 1$ de başlatıp, $high$ a doğru çalışır, böylece hesapladığı tüm altdiziler $A[mid + 1..j]$ şeklindedir. Son olarak, satır 15 te, orta noktayı içeren $max-left$ ve $max-right$ indisleriyle birlikte $A[max-left..max-right]$ altdizisindeki değerlerin toplamı olan $left-sum + right-sum$ değeri de geri döndürülür.

Eğer $A[low..high]$ altdizisi n tane içeriğe sahipse (yani $n = high - low + 1$), iddiamız FIND-MAX-CROSSING-SUBARRAY ($A, low, mid, high$) çağırımı $\Theta(n)$ zaman alacaktır. İki **for** döngüsünün her bir iterasyonu $\Theta(1)$ zaman alacağından, toplamda kaç iterasyonun olacağını saymamız gerekmektedir. 3-7 satırlarının **for** döngüsü $mid - low + 1$ iterasyon yapar ve 10-14 satırlarının **for** döngüsü $high - mid$ iterasyon yapar. Böylece toplam iterasyon sayısı

$$\begin{aligned} (mid - low + 1) + (high - mid) &= high - low + 1 \\ &= n . \end{aligned}$$

Elimizdeki FIND-MAX-CROSSING-SUBARRAY doğrusal-zaman prosedürü ile, böl-ve-yönet algoritması için maksimum-alt dizisi problemini çözmek için kabakod yazabiliriz:

FIND-MAXIMUM-SUBARRAY ($A, low, high$)

```

1  if high == low
2      return (low, high, A[low])           // temel durum: sadece bir eleman
3  else mid =  $\lfloor (low + high)/2 \rfloor$ 
4      (left-low, left-high, left-sum) =
        FIND-MAXIMUM-SUBARRAY( $A, mid + 1, high$ )
6      (cross-low, cross-high, cross-sum) =
        FIND-MAX-CROSSING-SUBARRAY( $A, low, mid, high$ )
7      if left-sum  $\geq$  right-sum and left-sum  $\geq$  cross-sum
8          return (left-low, left-high, left-sum)
9      elseif right-sum  $\geq$  left-sum and right-sum  $\geq$  cross-sum
10         return (right-low, right-high, right-sum)
11     else return (cross-low, cross-high, cross-sum)
```


FIND-MAXIMUM-SUBARRAY ($A, 1, A.length$) başlangıç çağırımı $A[1..n]$ nin maksimum altdizisini bulacaktır.

FIND-MAX-CROSSING-SUBARRAY a benzer olarak, bir maksimum altdizinin içindeki değerler toplamı ile birlikte, FIND-MAXIMUM-SUBARRAY özyinelemeli prosedürü maksimum altdizi prosedürünü sınırlandıran indisleri içeren bir çokdeğerliye dönecektir. Satır 1, altdizinin sadece bir elemana sahip olduğu durumda, temel durumu test eder. Bir elemana sahip bir altdizinin sadece—kendisi—olan bir tek altdizisi vardır ve böylece satır 2, o tek elemanın değeri ile birlikte başlangıç ve bitiş indisli değişken grubuna döner. 3-11 satırlar ise özyinelemeli durumu işler. Satır 3 orta noktanın *mid* indisini hesaplayarak bölme kısmını halleder. $A[low..mid]$ altdizisini **sol altdizi**, $A[mid + 1..high]$ altdizini de **sağ altdizi** olarak alalım. Çünkü, $A[low..high]$ altdizinin en az iki eleman içerdiğini biliyoruz, her bir sağ ve sol altdizi en az bir eleman içerir. 4 ve 5 satırları sırasıyla sol ve sağ altdiziler içinde maksimum altdiziyi özyinelemeli olarak bularak yönetir. 6-11 satırlar ise birleştirme kısmını oluşturur. Satır 6 ise orta noktayı geçen maksimum altdiziyi bulur. (Satır 6'nın orijinal problemin küçük olmayan örneği olan altproblemi çözdüğünü hatırlayalım, onun birleştirme kısmında olduğunu göz önüne alacağız). Satır 7 sol altdizinin maksimum toplamı bir altdizi içerip içermediğini test etmektedir ve satır 8'de o maksimum altdiziye döner. Aksi takdirde, satır 9 sağ altdizinin maksimum toplamı bir altdizi içerip içermediğini kontrol etmektedir, satır 10'da o maksimum altdiziye dönmektedir. Eğer, ne sol ne de sağ altdizi maksimum toplamı elde eden bir altdiziyi içermiyorsa, o halde bir maksimum altdizi ortanoktayı geçmek zorundadır ve satır 11 onu döndürür.

Böl-ve-yönet algoritmasını çözümleme

Şimdi, özyinelemeli FIND-MAXIMUM-SUBARRAY prosedürünü çalışma süresini tanımlayan bir özyineleme kuralım. Kısım 2.3.2'de birleştirerek sıralamayı analiz ederken yaptığımız gibi, orijinal problemin boyutu 2'nin kuvvetiymiş gibi basitleştirici bir kabul yapacağız ve böylece tüm altproblemlerin boyutları tam sayılar olacak. n elemanlı altdizi üzerinde FIND-MAXIMUM-SUBARRAY'ın çalışma süresini $T(n)$ ile göstereceğiz. Başlayanlar için, satır 1 sabit zaman alır. Temel durumda, $n = 1$ durumu, kolaydır: satır 2 sabit zaman tutar ve böylece

$$T(1) = \Theta(1) \quad (4.5)$$

dir.

$n > 1$ iken özyinelemeli durum meydana gelir. Satır 1 ve 3 sabit zaman tutar. Satır 4 ve 5'te çözülmüş her altproblem $n/2$ elemanlı altdizi üzerindedir (orijinal problemin 2'nin kuvveti şeklinde olduğu kabulümüz bize $n/2$ nin tam sayı olduğunu garanti eder) ve böylece herbirini çözmek için $T(n/2)$ zaman harcarız. Çünkü biz iki altproblemi çözmemiz gerekir—sol altdizi için ve sağ altdizi için—satır 4 ve 5'ten gelen çalışma süresine olan katkı $2T(n/2)$ ye gelir. Gördüğümüz gibi,

satır 6 da FIND-MAX-CROSSING-SUBARRAY a çağrı $\Theta(n)$ zaman almaktadır. Satırlar 7-11 sadece $\Theta(1)$ zaman almaktadır. Özyinelemeli durum için, bu yüzden,

$$\begin{aligned} T(n) &= \Theta(1) + 2T(n/2) + \Theta(n) + \Theta(1) \\ &= 2T(n/2) + \Theta(n) . \end{aligned}$$

dir. (4.6)

(4.5) ve (4.6) denklemlerinin birleştirilmesi bize FIND-MAXIMUM-SUBARRAY ın çalışma süresi $T(n)$ için bir özyineleme verir:

$$T(n) = \begin{cases} \Theta(1) & n = 1 \text{ ise,} \\ 2T(n/2) + \Theta(n) & n > 1 \text{ ise} \end{cases} \quad (4.7)$$

çalışma süresi $T(n)$ için bir özyineleme verir.

Bu özyineleme, birleştirerek sıralama için olan (4.1) daki özyineleme ile aynıdır. Bölüm 4.5 de ana yöntemden göreceğimiz gibi, bu özyineleme $T(n) = \Theta(n \lg n)$ çözüme sahiptir. Ayrıca çözümün neden $T(n) = \Theta(n \lg n)$ olması gerektiğini anlamak için Şekil 2.5 te özyineleme ağacına tekrar göz atabilirsiniz.

Böylece, böl-ve-yönet yöntemi kaba kuvvet yönteminden daha asimptotik olarak daha hızlı bir algoritmaya yol açar. Birleştirerek sıralama ile birlikte ve şimdi de maksimum-altldizi problemi, böl-ve-yönet yönteminin ne kadar güçlü olabileceği fikrini oluşturuyor. Bu yöntem, bazen bir problem için asimptotik olarak en hızlı algoritmaya yol açacaktır ve ve bazen de biz daha iyisini yapabiliriz. Alıştırma 4.1-5 te maksimum-altldizi problemi için aslında doğrusal-zaman algoritmasının olduğu gösterilmektedir ve böl-ve-yöneti kullanmamaktadır.

Alıştırmalar

4.1-1

A nın tüm elemanları negatif olduğunda FIND-MAXIMUM-SUBARRAY dönümü ne yapar?

4.1-2

Maksimum altldizi problemini çözmek için kaba-kuvvet yöntemi kullanan bir kaba kod yazınız. İşleminiz $\Theta(n^2)$ sürede çalışmalıdır.

4.1-3

Kendi bilgisayarınızda maksimum-altldizi problemi için kaba-kuvvet ve özyineleme algoritması oluşturunuz. Problem boyutunun n_0 olması, kaba-kuvvet algoritmasının özyineleme algoritmasını geçtiği noktada bize ne verir? Daha sonra, problemin boyutunun n_0 dan daha küçük olduğunda, kaba-kuvvet algoritmasını kullanmak için özyineleme algoritmasının temel durumunu değiştir. Bu geçiş noktasını değiştirir mi?

4.1-4

Boş bir altldizinin değerleri toplamı 0 olsun. Sonucun boş bir altldizi olmasına izin vermek için, maksimum-altldizi problem tanımını değiştirdiğimizi farzedelim.

Boş altdizilerin bir sonuç olmasına izin vermeyen herhangi bir algoritmayı nasıl değiştirdiniz?

4.1-5

Aşağıdaki fikirleri, maksimum-altdizi problemi için özyinelemeli olmayan, doğrusal-zaman algoritması geliştirmek için kullanınız. Dizinin sol bitiminden başlayınız, sağa doğru ileriye, o ana kadar görülen maksimum-altdiziyi takip et. Aşağıdaki gözlemi yaparak, $A[1..j]$ nin bir maksimum altdizinin bilinmesi sonucunu, $j+1$ indisinde biten bir maksimum altdizi bulmak için genişletiniz. $1 \leq i \leq j+1$ olmak üzere, $A[1..j+1]$ nin bir maksimum altdizisi, ya $A[1..j]$ nin ya da $A[1..j+1]$ maksimum altdizisidir. j indisinde biten bir maksimum altdiziyi bilmeye dayalı, $A[1..j+1]$ formunda bir maksimum altdizi belirleyiniz.

4.2 Matris çarpımı için Strassen algoritması

Matrisleri daha önceden gördüyseniz, muhtemelen onların nasıl çarpıldığını biliyorsunuz (Aksi halde, Ek bölümler D deki Kısım D.1 i okumanız gerekmektedir). $A = (a_{ij})$ ve $B = (b_{ij})$ ler $n \times n$ tipinde karesel matrisler olsun, bu matrislerin çarpımı $C = A \cdot B$ de $i, j = 1, 2, \dots, n$ için c_{ij} elemanlarını

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj} \quad (4.8)$$

ile tanımlayalım.

Matristeki n^2 adet elemanı hesaplamak istiyoruz ve her biri n değer toplamıdır. Aşağıdaki prosedür $n \times n$ tipindeki A ve B matrislerini alır ve çarpar, onların $n \times n$ çarpımı C matrisine dönüştürür.

SQUARE-MATRIX-MULTIPLY(A, B)

```

1   $n = A.rows$ 
2   $C$   $n \times n$  tipinde yeni bir matris olsun
3  for  $i = 1$  to  $n$ 
4      for  $j = 1$  to  $n$ 
5           $c_{ij} = 0$ 
6          for  $k = 1$  to  $n$ 
7               $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$ 
8  return  $C$ 
```

SQUARE-MATRIX-MULTIPLY prosedürü şöyle çalışır. Satırlar 3-7 için **for** döngüsü her i satırı elemanları hesaplar ve verilen i satırı içinde satır 4-7 de geçen **for**

döngüsü her j sütunu için c_{ij} leri hesaplar. Satır 5, biz (4.8) denkleminde verilen toplamı hesaplarken, c_{ij} yi sıfırlar ve satırlar 6-7 nin **for** döngüsünün her bir iterasyonu (4.8) denklemine bir terim daha ekler.

Çünkü ardı ardına üç tane **for** döngüsü tam olarak n adet iterasyonu çalışmaktadır, ve satır 7 nin her yürütümü sabit zaman tutmaktadır, SQUARE-MATRIX-MULTIPLY prosedürü $\Theta(n^3)$ zaman alır.

Matris çarpımının tanımı çok fazla çarpma işlemi gerektirdiğinden, ilk başta matris çarpımı algoritmasının $\Omega(n^3)$ zamanda yapılacağını düşünebilirsiniz. Fakat böyle değildir, matris çarpımını $o(n^3)$ zamanda yapabilen algoritma mevcuttur. Bu bölümde, Strassen'nin $n \times n$ tipindeki matrislerin çarpımı için dikkate değer bir özyineleme algoritmasını göreceksiniz. Bu algoritma, $\Theta(n^{\lg 7})$ zamanda çalışmakta olup Kısım 4.5 te gösterilecektir. $\lg 7$ sayısı 2.80 ile 2.81 arasında bir sayı olduğundan, Strassen algoritması $O(n^{2.81})$ zamanda çalışmaktadır ve bu basit SQUARE-MATRIX-MULTIPLY prosedüründen asimptotik olarak daha iyidir.

Basit bir böl-ve-yönet algoritması

$C = A \cdot B$ matris çarpımını hesaplayan böl-ve-yönet algoritmasını kullanırken, işleri kolaylaştırmak için, $n \times n$ tipindeki matrislerde n yi 2 nin tam kuvveti şeklinde kabul edelim. Çünkü her bölme adımında $n \times n$ tipindeki matrisleri dört adet $n/2 \times n/2$ matrislerine parçalayacağız ve $n \geq 2$ olduğu sürece n yi 2 nin kuvveti olarak aldığımızdan, $n/2$ boyutunun tam sayı olduğunu garanti edebileceğiz.

Kabul edelim ki her bir A , B ve C matrislerini dört adet $n/2 \times n/2$ tipindeki matrislere parçalansın

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} \quad (4.9)$$

bu durumda $C = A \cdot B$ denklemini

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \cdot \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \quad (4.10)$$

olarak tekrar yazalım. Denklem (4.10)

$$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21}, \quad (4.11)$$

$$C_{12} = A_{11} \cdot B_{12} + A_{12} \cdot B_{22}, \quad (4.12)$$

$$C_{21} = A_{21} \cdot B_{11} + A_{22} \cdot B_{21}, \quad (4.13)$$

$$C_{22} = A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \quad (4.14)$$

şeklinde dört tane denkleme karşılık gelir.

Bu denklemlerin her biri dört iki tane $n/2 \times n/2$ tipindeki matrislerin çarpımını ve onların $n/2 \times n/2$ tipindeki çarpımlarının toplamalarını belirtir. Biz doğrudan bu denklemleri kullanarak, özyinelemeli böl-ve-yönet algoritması oluşturacağız:

SQUARE-MATRIX-MULTIPLY-RECURSIVE(A, B)

```

1   $n = A.rows$ 
2   $C$   $n \times n$  tipinde yeni bir matris olsun
3  if  $n == 1$ 
4       $c_{11} = a_{11} \cdot b_{11}$ 
5  else  $A, B$  ve  $C$  yi (4.9) denklemlerindeki gibi parçala
6       $C_{11} = \text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{11}, B_{11})$ 
           $+ \text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{12}, B_{21})$ 
7       $C_{12} = \text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{11}, B_{12})$ 
           $+ \text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{12}, B_{22})$ 
8       $C_{21} = \text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{21}, B_{11})$ 
           $+ \text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{22}, B_{21})$ 
9       $C_{22} = \text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{21}, B_{12})$ 
           $+ \text{SQUARE-MATRIX-MULTIPLY-RECURSIVE}(A_{22}, B_{22})$ 
10 return  $C$ 
```

Bu prosedür, başta göze çarpmayan fakat çok önemli bir uygulama detayını atlamaktadır. Satır 5 te matrisleri parçalama işlemini nasıl yapıyoruz? Eğer 12 tane $n/2 \times n/2$ tipindeki matrisleri oluşturuyor olsaydık, elemanları kopyalamaya $\Theta(n^2)$ zaman harcardık. Aslında, matrisleri içeriklerini kopyalamadan da parçalayabiliriz. Burada indis hesabı önemlidir. Bir altmatrisi orijinal matrisin satır indisleri ve sütun indisleri aralığı ile tanımlarız. Altmatrisi, biraz göz ardı ederek orijinal matrisi nasıl temsil ettiğimizden biraz daha farklı şekilde temsil edeceğiz. Bunun bize kazancı, altmatrisleri indis hesabı ile belirttiğimizden dolayı, satır 5 in yürütümü sadece $\Theta(1)$ zaman alır (Yeri geldiğinde parçalama ya da kopyalama yapıp yapmadığının tüm çalışma süresinden asimptotik olarak çok farklı olmasına rağmen).

Şimdi SQUARE-MATRIX-MULTIPLY ın çalışma süresini karakterize eden bir özyineleme oluşturalım. $T(n)$ bu prosedür ile iki tane $n \times n$ tipinde matrisin çarpma süresini gösterebilir. Temel durumda, $n = 1$ de, satır 4 de sadece bir skaler çarpımı yapacağız ve böylece

$$T(1) = \Theta(1) \quad (4.15)$$

olur.

Özyineleme durumu $n > 1$ iken olur. Daha önce bahsedildiği üzere, satır 5 te indisleri kullanarak, matrisleri parçalama $\Theta(1)$ zaman alır. Satırlar 6-9 da SQUARE-MATRIX-MULTIPLY-RECURSIVE i özyinelemeli olarak toplam sekiz kere çağırıyoruz. Çünkü her özyineleme çağırısı iki tane $n/2 \times n/2$ matrisi çarpar, dolayısıyla ile $T(n/2)$ nin tüm çalışma süresine eklenmesiyle, tüm sekiz özyineleme çağırısının süresi $8T(n/2)$ dir. Ayrıca satırlar 6-9 da dört adet matris toplama işleminden de bahsetmeliyiz. Bu matrislerin her biri $n^2/4$ tane içeriğe sahipti ve bu

dört toplamın her biri $\Theta(n^2)$ zaman almaktadır. Matris toplamaları sayısı bir sabit olduğundan, satırlar 6-9 daki matrisleri toplama için geçen toplam süre $\Theta(n^2)$ dir. (Matris toplama sonuçlarını C matrisi içinde doğru yerlerine yerleştirmek için tekrardan tekrardan indis hesabı yapacağız, her içerik $\Theta(1)$ kadar zaman tutacaktır.) Bu nedenle, özyineleme durumunun toplam zamanı parçalama zamanlarının toplamıdır, tüm özyineleme çağrılarının süresi ve özyineleme çağrılarında kaynaklanan matris toplamaları için gereken zaman:

$$\begin{aligned} T(n) &= \Theta(1) + 8T(n/2) + \Theta(n^2) \\ &= 8T(n/2) + \Theta(n^2) \end{aligned} \quad (4.16)$$

dir. Dikkat edecek olursak, parçalamayı matrisleri kopyalayarak yaptığımızda, bu $\Theta(n^2)$ zaman alacaktır, özyineleme değişmeyecektir ve böylece tüm çalışma süresi bir sabit kat kadar artacaktır.

Denklemler (4.15) ve (4.16) birleştirirsek bu bize SQUARE-MATRIX-MULTIPLY-RECURSIVE in çalışma süresi için özyineleme verecektir:

$$T(n) = \begin{cases} \Theta(1) & n = 1 \text{ ise,} \\ 8T(n/2) + \Theta(n^2) & n > 1 \text{ ise.} \end{cases} \quad (4.17)$$

Kısım 4.5 te ana yöntemde göreceğimiz gibi, (4.17) özyinelemesi $T(n) = \Theta(n^3)$ çözümüne sahiptir. Böylece, bu basit böl-ve-yönet yaklaşımı SQUARE-MATRIX-MULTIPLY prosedüründen hızlı değildir.

Strassen'in algoritmasını detaylı incelemeden önce, denklem (4.16) elemanlarının nereden geldiklerine bakalım. Her bir $n \times n$ matrisini indis hesabı ile parçalamak $\Theta(1)$ zaman almaktadır fakat bizim parçalamamız gereken iki tane matrisimiz vardır. Her ne kadar iki matrisi parçalamak $\Theta(2)$ zaman alacağını düşünseniz de, 2 sabiti Θ -notasyonu tarafından içerilir. k içeriğe sahip iki matrisin toplanması $\Theta(k)$ zaman alır. Topladığımız her bir matris $n^2/4$ içeriğe sahip olduğundan dolayı, her matris çiftinin toplanması $\Theta(n^2/4)$ zaman alacağı söyleyebiliriz. Yine, her ne kadar Θ -notasyonu $1/4$ ün sabitinin çarpanını içermesiyle birlikte, iki tane $n^2/4 \times n^2/4$ matrisin toplanması $\Theta(n^2)$ zaman alır. Bu şekilde dört adet matris toplamamız var ve yeniden, bu işlemlerin $\Theta(4n^2)$ alacağını söylemek yerine, $\Theta(n^2)$ zaman alacağını söyleyebiliriz. (Elbette, dört matris toplamının $\Theta(4n^2/4)$ zaman alacağını söyleyebilirdiniz ve $4n^2/4 = n^2$ dir, fakat burada ki esas nokta Θ -notasyonu katsayı sabitlerini içermesidir). Böylece, $\Theta(n^2)$ nin iki terimini bir taneye birleştirerek tamamlarız.

Sekiz özyineleme çağrısını araştırırken, sadece 8 in sabit çarpanını içiremeyiz. Diğer bir ifadeyle, onların hepsi birlikte $T(n/2)$ zamandan ziyade $8T(n/2)$ zaman alacağını söylemeliyiz. Geriye dönüp Şekil 2.5 te ki özyineleme ağacına bakarak, özyineleme 2.1 için (Bu özyineleme 4.7 ile aynıdır), $T(n) = 2T(n/2) + \Theta(n)$ özyineleme durumu ile birlikte bir fikir elde edebilirsiniz. 2 nin çarpanı ağacın her bir dalının kaç çocuğa sahip olduğunu belirler, bu da sırasıyla ağacın her seviyesinde, toplama kaç tane terim kattığını belirler. Eğer, denklem 4.6 da 8 in

çarpanını ya da özyineleme (4.1) de 2 nin çarpanını göz ardı etseydik, özyineleme ağacı daha çok dallanmak yerine daha doğrusal olurdu ve her seviye sadece toplama sadece bir terim katkı sağlardı.

O halde, asimptotik notasyonun sabit çarpanları önemsememesine rağmen, $T(n/2)$ gibi özyineleme notasyonlarının bunları önemsedğini göz önünde bulundurunuz.

Strassen yöntemi

Strassen yönteminin anahtarı, özyineleme ağacını biraz daha az dallı yapmaktır. Bu da, sekiz tane $n/2 \times n/2$ matrislerinin özyineleme çarpımını yapmak yerine, sadece yedi tane yapmaktır. Bir matris çarpmasını elemanın bize maliyeti, bir kaç tane $n/2 \times n/2$ matrisinin yeni toplamı olacaktır, fakat hala sabit miktarda toplamadır. Daha önceden de olduğu gibi, matris toplamalarının sabit sayısı, çalışma zamanını karakterize etmek için özyineleme denklemini oluşturduğumuzda, Θ -notasyonu tarafından içermektedir.

Aslında Strassen'nin yöntemi çok açık değildir (Bu kitapta ki bu duruma çok dikkat edilmemiştir). Dört adımı vardır:

1. A ve B giriş matrisleri ile C çıkış matrisini, denklem (4.9) daki gibi $n/2 \times n/2$ altmatrislerine böl. Bu adım indis hesabı ile SQUARE-MATRIX-MULTIPLY-RECURSIVE deki gibi $\Theta(1)$ süre alır.
2. Her biri $n/2 \times n/2$ tipinde ve adım 1 de oluşturulan iki matrisin toplamı veya farkı olan $S_1, S_2, \dots, S_{10}$ gibi 10 adet matris oluşturun. Bu 10 matrisi $\Theta(n^2)$ zamanda oluşturabiliriz.
3. Adım 1 de oluşturulan altmatrisleri ve adım 2de oluşturulan 10 matrisi kullanarak, özyinelemeli olarak yedi $P_1, P_2, \dots, P_7$ matrislerinin çarpımını hesapla. Her bir P_i matrisi $n/2 \times n/2$ tipindedir.
4. P_i matrislerinin çeşitli kombinasyonlarını toplayarak ve çıkartarak C sonuç matrisinin istenilen $C_{11}, C_{12}, C_{21}, C_{22}$, altmatrislerinin hesapla. Dört matrisin tümünü $\Theta(n^2)$ zamanda hesaplayabiliriz.

Adımlar 2-4 ün detaylarını göreceğiz, fakat Strassen'nin yönteminin çalışma süresi için bir özyineleme oluşturmak için yeterince bilgiye sahibiz. Matrisin n olan boyutunun 1 e düştüğünü kabul edelim, SQUARE-MATRIX-MULTIPLY-RECURSIVE nın 4. satırındaki gibi basit bir skaler çarpma yapalım. $n > 1$ olduğunda, adımlar 1, 2 ve 4 ün toplam $\Theta(n^2)$ zaman alır ve adım 3 bize yedi tane $n/2 \times n/2$ matris çarpımı yapmamızı gerektirir. Böylece, Strassen algoritmasının $T(n)$ çalışma süresi için aşağıdaki özyinelemeyi elde ederiz:

$$T(n) = \begin{cases} \Theta(1) & n = 1 \text{ ise,} \\ 7T(n/2) + \Theta(n^2) & n > 1 \text{ ise.} \end{cases} \quad (4.18)$$

Burada bir matris çarpımını, sabit sayıda matrislerin toplamıyla değiş tokuş ettik. Özyinelemeleri ve onların çözümlerini anladığımız zaman, göreceğiz ki bu daha düşük asimptotik çalışma süresine öncülük edecektir. Kısım 4.5 te ana yöntem ile (4.18) özyinelemesi $T(n) = \Theta(n^{\lg 7})$ çözümüne sahiptir.

Şimdi detayları incelemeye geçeceğiz. Adım 2 de aşağıdaki 10 matrisi oluşturacağız:

$$\begin{aligned}
 S_1 &= B_{12} - B_{22} , \\
 S_2 &= A_{11} + A_{12} , \\
 S_3 &= A_{21} + A_{22} , \\
 S_4 &= B_{21} - B_{11} , \\
 S_5 &= A_{11} + A_{22} , \\
 S_6 &= B_{11} + B_{22} , \\
 S_7 &= A_{12} - A_{22} , \\
 S_8 &= B_{21} + B_{22} , \\
 S_9 &= A_{11} - A_{21} , \\
 S_{10} &= B_{11} + B_{12} .
 \end{aligned}$$

$n/2 \times n/2$ matrislerini 10 kere ekleyip ya da çıkarmamız gerektiğinden dolayı, bu adım aslında $\Theta(n^2)$ zaman alır.

Adım 3 te aşağıdaki $n/2 \times n/2$ matrisini hesaplamak için, yedi adet $n/2 \times n/2$ matrisini özyinelemeli olarak çarpabiliriz. Her biri, A ve B altmatrislerinin çarpımlarının toplamı veya farkıdır:

$$\begin{aligned}
 P_1 &= A_{11} \cdot S_1 = A_{11} \cdot B_{12} - A_{11} \cdot B_{22} , \\
 P_2 &= S_2 \cdot B_{22} = A_{11} \cdot B_{22} + A_{12} \cdot B_{22} , \\
 P_3 &= S_3 \cdot B_{11} = A_{21} \cdot B_{11} + A_{22} \cdot B_{11} , \\
 P_4 &= A_{22} \cdot S_4 = A_{22} \cdot B_{21} - A_{22} \cdot B_{11} , \\
 P_5 &= S_5 \cdot S_6 = A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22} , \\
 P_6 &= S_7 \cdot S_8 = A_{12} \cdot B_{21} + A_{12} \cdot B_{22} - A_{22} \cdot B_{21} - A_{22} \cdot B_{22} , \\
 P_7 &= S_9 \cdot S_{10} = A_{11} \cdot B_{11} + A_{11} \cdot B_{12} - A_{21} \cdot B_{11} - A_{21} \cdot B_{12} .
 \end{aligned}$$

Dikkat edilirse, hesaplamamız gereken, yukarıdaki denklemlerde orta sütundaki çarpımlardır. Sağ taraftaki sütun, bu çarpımların adım 1 de oluşturulan orijinal altmatrisler anlamında neye eşit olduğunu göstermektedir.

Adım 4, C çarpım matrisinin dört adet $n/2 \times n/2$ altmatrislerini oluşturmak için adım 3 te oluşturulan P_i matrislerini toplayıp çıkartır.

$$C_{11} = P_5 + P_4 - P_2 + P_6$$

ile başlayalım. Her bir P_i nin kendi içinde açılması ile eşitliğin sağ tarafını açalım ve bir birini götürülen terimleri dikey olarak hizalayarak, C_{11} in

$$\begin{array}{r}
 A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22} \\
 \quad \quad \quad - A_{22} \cdot B_{11} \quad \quad \quad + A_{22} \cdot B_{21} \\
 \quad \quad \quad - A_{11} \cdot B_{22} \quad \quad \quad - A_{12} \cdot B_{22} \\
 \quad \quad \quad \quad \quad \quad - A_{22} \cdot B_{22} - A_{22} \cdot B_{21} + A_{12} \cdot B_{22} + A_{12} \cdot B_{21} \\
 \hline
 A_{11} \cdot B_{11} \quad \quad \quad + A_{12} \cdot B_{21}
 \end{array}$$

sonucuna sonucuna eşit olduğunu buluruz ve bu sonuç denklem (4.11) e denktir.

Benzer şekilde,

$$C_{12} = P_1 + P_2$$

olarak alırsak bu duruma C_{12} ,

$$\begin{array}{r}
 A_{11} \cdot B_{12} - A_{11} \cdot B_{22} \\
 \quad \quad \quad + A_{11} \cdot B_{22} + A_{12} \cdot B_{22} \\
 \hline
 A_{11} \cdot B_{12} \quad \quad \quad + A_{12} \cdot B_{22}
 \end{array}$$

ifadesine eşittir ve denklem (4.12) ye denktir.

$$C_{21} = P_3 + P_4 \text{ eşitliği, } C_{21} \text{ in}$$

$$\begin{array}{r}
 A_{21} \cdot B_{11} + A_{22} \cdot B_{11} \\
 \quad \quad \quad - A_{22} \cdot B_{11} + A_{22} \cdot B_{21} \\
 \hline
 A_{21} \cdot B_{11} \quad \quad \quad + A_{22} \cdot B_{21}
 \end{array}$$

e denk olmasını sağlar ve denklem (4.13) e denktir.

Son olarak, $C_{22} = P_5 + P_1 - P_3 - P_7$ eşitliği ile C_{21} nin

$$\begin{array}{r}
 A_{11} \cdot B_{11} + A_{11} \cdot B_{22} + A_{22} \cdot B_{11} + A_{22} \cdot B_{22} \\
 \quad \quad \quad - A_{11} \cdot B_{22} \quad \quad \quad + A_{11} \cdot B_{12} \\
 \quad \quad \quad \quad \quad \quad - A_{22} \cdot B_{11} \quad \quad \quad - A_{21} \cdot B_{11} \\
 - A_{11} \cdot B_{11} \quad \quad \quad - A_{11} \cdot B_{12} + A_{21} \cdot B_{11} + A_{21} \cdot B_{12} \\
 \hline
 A_{22} \cdot B_{22} \quad \quad \quad + A_{21} \cdot B_{12}
 \end{array}$$

eşit olduğunu ve bunun da denklem (4.14) e denk olduğunu görürüz. Tüm bunların hepsi ile birlikte adım 4 te sekiz adet $n/2 \times n/2$ matrisi toplarız ya da çıkarırız ve bu adım $\Theta(n^2)$ zaman alır.

Böylece, adımlar 1-4 ü oluşturan ve doğru matris çarpımına yol açan Strassen Algoritmasını gördük ve bu özyineleme (4.18) onun çalışma süresini karakterize eder. Kısım 4.5 te göreceğimiz bu özyineleme, $T(n) = (n^{\lg 7})$ çözümüne sahip olduğundan, Strassen Yöntemi, doğrudan SQUARE-MATRIX-MULTIPLY yönteminden asimptotik olarak, daha hızlıdır. Bu bölümün sonundaki notlar Strassen Algoritmasının bazı pratik yönlerini açıklamaktadır.

Alıştırmalar

Not: Alıştırmalar 4.2-3, 4.2-4 ve 4.2-5 her ne kadar Strassen Algoritmasının türleri üzerine olsa da, bunları çözmeden Kısım 4.5 i okumanız tavsiye edilmektedir.

4.2-1

Strassen Algoritmasını kullanarak,

$$\begin{pmatrix} 1 & 3 \\ 7 & 5 \end{pmatrix} \begin{pmatrix} 6 & 8 \\ 4 & 2 \end{pmatrix}$$

matris çarpımını hesaplayınız. Çalışmanızı yazınız.

4.2-2

Strassen Algoritması için kabakod yazınız.

4.2-3

n sayısı 2 nin tam bir kuvveti olmadığında, $n \times n$ tipindeki matrislerin çarpımını hesaplamak için Strassen Algoritmasına nasıl bir değişiklik yapardınız. Algoritmanızın $\Theta(n^{\lg 7})$ zamanda çalıştığını gösteriniz.

4.2-4

k adet çarpma işlemi ile 3×3 tipindeki matrislerin çarpılabileceği (çarpmada değişmeyi varsaymadan) ve $n \times n$ tipindeki matrislerin $o(n^{\lg 7})$ zamanda çarpılabileceği en büyük k sayısı nedir? Bu algoritmanın çalışma süresi ne olurdu?

4.2-5

V. Pan, 132,464 adet çarpma işlemi ile 68×68 tipinde matrisleri ve 143,640 adet çarpma işlemi ile de 70×70 tipinde matrisleri ve 155,424 adet çarpma işlemi ile 72×72 tipinde matrisleri çarpmanın yollarını buldu. Böl-ve-yönet matris çarpımı algoritmasında kullanıldığından hangi yöntem en iyi asimptotik çalışma süresine yol açar. Bunlar Strassen Algoritmasıyla karşılaştırıldığında sonuç ne olur?

4.2-6

Strassen Algoritmasını altprogram olarak kullanıldığından, $k \times n$ matrisi ile $n \times nk$ matrisini ne kadar hızlı çarpabilirsiniz? Aynı soruyu girdi matrislerinin tersine çevirerek sırasıyla yanıtlayınız.

4.2-7

Sadece üç adet çarpma işlemi yaparak $a + bi$ ve $c + di$ karmaşık sayılarının nasıl çarpıldığını gösteriniz. Algoritmanız, a , b , c ve d sayılarını girdi olarak alıp, ayrı ayrı $ac - bd$ gerçel bileşeni ile $ad + bc$ sanal bileşeni üretmelidir.

4.3 Özyinelemeleri çözmek için yerine koyma yöntemi

Artık, özyinelemelerin böl-ve-yönet algoritmasının çalışma süresini nasıl karakterize ettiğini biliyoruz, özyinelemeleri nasıl çözeceğimizi öğreneceğiz. Bu bölümde “yerine koyma” yöntemi ile başlayacağız.

Özyinelemeleri çözmek için **yerine koyma yöntemi** iki adımdan oluşur:

1. Çözümün biçiminin tahmin edilmesi.
2. Sabitleri bulmak için matematiksel tümevarımın kullanımı ve çözümün çalıştığının gösterilmesi.

Tümevarım hipotezi daha küçük değerlere uygulanırken, fonksiyonun tahmini çözüm değerini yerine koyarız, bu yüzden “yerine koyma” adımı alır. Bu yöntem oldukça güçlüdür, fakat bizim onu uygulayabilmemiz için cevabın biçimini tahmin edebiliyor olmamız gerekmektedir.

Yerine koyma yöntemini, özyinelemenin üst ya da alt sınırları belirlemek için kullanabiliriz. Örnek olması adına, (4.3) ve (4.4) özyinelemelerine benzer

$$T(n) = 2T(\lfloor n/2 \rfloor) + n \quad (4.19)$$

özyinelemesi için bir üst sınır belirleyelim. Tahminimizce sonuç $T(n) = O(n \lg n)$ dir. Özyineleme yöntemi bize $c > 0$ sabitinin uygun bir seçimi için $T(n) \leq cn \lg n$ eşitsizliğini kanıtlamamızı gerektirir. Bu sınırın tüm $m > n$ pozitif tam sayıları için gerçekleştiğini kabul ederek başlayalım, özel olarak $m = \lfloor n/2 \rfloor$ olması, $T(\lfloor n/2 \rfloor) \leq c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)$ sonucunu getirir. Özyinelemenin içinde yerine koyarak,

$$\begin{aligned} T(n) &\leq 2(c \lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \\ &\leq cn \lg(n/2) + n \\ &= cn \lg n - cn \lg 2 + n \\ &= cn \lg n - cn + n \\ &\leq cn \lg n \end{aligned}$$

dir ve son adım $c \geq 1$ olduğu sürece sağlanır.

Matematiksel tümevarım, çözümümüzün sınır koşullarında sağlandığını göstermemizi gerektirir. Genel olarak bunu, sınır koşullarının tümevarımsal kanıtlar için temel durumlar kadar uygun olduğunu göstererek yaparız. (4.19) özyinelemesi için, c sabitini yeterince büyük seçerek, $T(n) \leq cn \lg n$ eşitsizliğinin sınır koşullarda da sağlandığını göstermeliyiz. Bu gereksinim, bazen problemlere yön verir. Bulduğumuz durum adına $T(1) = 1$ in özyinelemenin yegane sınır koşulu olduğunu kabul edelim. Böylece $n = 1$ olması, $T(n) \leq cn \lg n$ sınırının tek sayılarda $T(1) = 1$ ile birlikte $T(1) \leq c1 \lg 1 = 0$ olmasına yol açar. Sonuç olarak, özyinelememiz için temel durum çalışmayacaktır.

Bu engeli, biraz fazla emek harcayarak özel bir sınırı koşulu altında tümevarım hipotezi kanıtlayarak aşabiliriz. (4.19) özyinelemesinde, örneğin, $n \geq n_0$ için sadece $T(n) \leq cn \lg n$ olduğunu kanıtlamamızı gerektiren asimptotik notasyondan faydalanırız, burada n_0 *bizim seçmemiz gereken* bir sabittir. Biraz zahmetli olan $T(1) = 1$ sınır koşulunu elimizde tutalım, fakat onu tümevarımsal kanıttan çıkarmalım. Bunu öncelikle, $c > 3$ için özyinelemenin doğrudan $T(1)$ koşuluna bağlı olmadığını gözlemleyerek yapacağız. Böylece, $n_0 = 2$ alarak, tümevarım kanıtının temel durumu olarak $T(1)$ i $T(2)$ ve $T(3)$ ile değiştirebileceğiz. Dikkat edilirse, özyinelemenin $n = 2$ temel durumu ve tümevarım kanıtının $n = 2$ ve $n = 3$ temel durumları arasında fark oluşturduk $T(1) = 1$ ile birlikte, $T(2) = 4$ ve $T(3) = 5$ den elde ettik. Şimdi c yi yeterince büyük seçerek, bazı $c \geq 1$ sabitleri için, $T(n) \leq cn \lg n$ ı sağlayan tümevarım kanıtını tamamlayabiliriz böylece $T(2) \leq c2 \lg 2$ ve $T(3) \leq c3 \lg 3$ olur. Nihayetinde, $c \geq 2$ nin herhangi bir seçimi $n = 3$ ve $n = 3$ temel durumlarının gerçekleşmesi için yeterlidir. İnceleyeceğimiz çoğu özyinelemeler için, özyineleme kabulünün küçük n ler için çalışacak hale getirmek açıktır.

İyi bir tahmin yapma

Ne yazık ki, özyinelemelerin çözümü için doğru bir tahmin yapmanın genel bir yolu yoktur. Bir çözümünün tahmini, deneyime bazende yaratıcılığa ihtiyaç duyar. Yine de buna rağmen sezgilerinizi iyi bir tahminci olabilmek için kullanabilirsiniz. Ayrıca daha sonra Kısım 4.4 te göreceğimiz özyineleme ağacını daha iyi tahminler yapmak için kullanabilirsiniz.

Eğer özyinelemeniz daha önceden gördüğünüz özyineleme ile benzer ise, benzer çözüm tahmini yapmak anlamlıdır. Örneker olarak, “17” sayısının sağ taraf-taki ifade eklenmesiyle oldukça zor görünen

$$T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$$

özyinelemesini göz önüne alalım. Sezgisel olarak, yinede eklenen bu terim özyinelemenin çözümünü etkilemez. n büyük olduğunda, $\lfloor n/2 \rfloor$ ile $\lfloor n/2 \rfloor + 17$ ara-

sındaki fark çok büyük olmaz: Her iki n de neredeyse eşit şekilde yarıya bölünür. Sonuç olarak, yerine koyma yöntemini kullanarak doğrulayabileceğimiz, $T(n) = O(n \lg n)$ tahminini yapabilirsiniz (bkz. alıştırma 4.3-6).

İyi bir tahmin yapmanın başka bir yolu da, özyineleme üzerine kabaca üst ve alt sınır tahmini yapıp daha sonra belirsizlikleri indirgemektir. Örneğin, özyinelemede n terimine sahip olduğumuzdan (4.19) özyinelemesi için $T(n) = \Omega(n)$ nin bir alt sınırı ile başlayabiliriz ve $T(n) = \Theta(n^2)$ nin başlangıç üst sınırını kanıtlayabiliriz. Daha sonra doğru çözüme ulaşmaya kadar, azar azar üst sınırı düşürerek ve alt sınırı yükselterek, $T(n) = \Theta(n \lg n)$ nin asimptotik olarak en dar çözümüne ulaşabiliriz.

İnce farklar

Bazen bir özyinelemenin çözümünde asimptotik sınırı doğru şekilde tahmin edebilirsiniz, fakat bir şekilde işin matematik kısmı tümevarımın çözümünde hata yapabilir. Problem çoğunlukla dönüp dolaşıp tümevarımsal kabulün detaylı çözüm için yeterince güçlü olmadığına gelmektedir. Eğer tahmininizi düşük-mertebe terimi çıkartarak tekrar gözden geçirirseniz, bir engele çarptığınız zaman matematik sizi bu engele takılmadan yolunuza devam etmenizi sağlayacaktır.

$T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$ özyinelemesini alalım. Çözümün $T(n) = O(n)$ olduğunu tahmin ederiz ve c nin uygun bir seçimi için $T(n) \leq cn$ olduğunu göstermeye çalışırız. Tahminimizi özyinelemede yerine koyarsak, c 'nin herhangi bir seçimi için $T(n) \leq cn$ ni sağlamayan

$$\begin{aligned} T(n) &\leq c \lfloor n/2 \rfloor + c \lceil n/2 \rceil + 1 \\ &= cn + 1 \end{aligned}$$

ifadesini elde ederiz. Fakat $T(n) = O(n^2)$ gibi daha daha büyük bir yaparak, çalıştığını görebiliriz. Her ne kadar bu büyük tahmin değerini çalışır hale getirebilsek, bizim orijinal tahminimiz $T(n) = O(n)$ doğru bir tahmindir. Bunun doğruluğunu göstermek için, daha güçlü özyinelemeli hipotez yapmamız gerekmektedir.

Sezgisel olarak, tahminimiz neredeyse doğru: tek hata 1 sabitidir, en düşük-mertebe terim. Yinede, biz tümevarım hipotezinin tam formunu kanıtlamadıkça matematiksel tümevarım işe yaramayacaktır. Bir önceki tahminimizden en düşük-mertebe terimi çıkararak bu zorluğun üstesinden gelebiliriz. Yeni tahminimiz $d \geq 0$ bir sabit olmak üzere, $T(n) \leq cn - d$ dir. Şimdi elimizde $d \geq 1$ olduğunda

$$\begin{aligned} T(n) &\leq (c \lfloor n/2 \rfloor - d) + (c \lceil n/2 \rceil - d) + 1 \\ &= cn - 2d + 1 \\ &\leq cn - d \end{aligned}$$

eşitsizliği vardır. Daha öncede de olduğu gibi, sınır koşullarını kontrol etmek için c sabitini yeterince büyük seçmeliyiz.

Bir alt-sıralı terimi çıkarma fikrini çok mantıklı bulmayabilirsiniz. Hepsinden öte eğer matematik bunu çözemezse, tahminimizi arttırmamız gerekir, doğru mu? Elbette gerek yok! Özyineleme ile üst sınır kanıtlarken, aslında daha zayıf bir üst sınırın çalıştığını kanıtlamak daha zordur, çünkü daha zayıf bir sınırı kanıtlamak için, aynı daha zayıf sınırı özyinelemeli olarak kanıtta kullanmalıyız. Son örneğimizde, özyinelememiz birden özyinelemeli terime sahip ise, her özyineleme meydana gelen en düşük-mertebeli terimi çıkarmamız gerekir. Aşağıdaki örnekte, d sabitini iki kere çıkardık, bir kere $T(\lfloor n/2 \rfloor)$ terimi için ve bir kere de $T(\lceil n/2 \rceil)$ terimi için. $T(n) \leq cn - 2d + 1$ eşitsizliği ile bitirdik ve $cn - 2d + 1$ yi $cn - d$ den daha küçük yapacak d nin değerlerini bulmak kolaydı.

Tuzaklardan kaçınma

Asimptotik gösterimde hata yapmak kolaydır. Örneğin, (4.19) özyinelemesinde $T(n) = O(n)$ olduğunu tahmin ederek hatalı olarak $T(n) \leq cn$ olduğunu kanıtlayabiliriz ve daha sonra

$$\begin{aligned} T(n) &\leq 2(c \lfloor n/2 \rfloor) + n \\ &\leq cn + n \\ &= O(n) \quad \Leftarrow \text{hatalı!!} \end{aligned}$$

iddiasında bulunuruz, çünkü burada c sabittir. Hatamız özyinelemeli hipotezin *tam biçimini* ispatlamamış oluşumuz, bu da $T(n) \leq cn$ dir. O halde, $T(n) = O(n)$ olduğunu göstermek istiyorsak, $T(n) \leq cn$ olduğunu açıkça ispatlamamız gerekmektedir.

Değişkenleri değiştirme

Bazen, küçük bir cebirsel iş bile daha önce gördüğünüz, bilinmeyen özyineleme yapabilir. Örnek olarak, zor görünen

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n$$

özyinelemesini ele alalım. Bu özyinelemeyi bir değişken değişimi ile basit hale getirebiliriz. Uygun olması açısından, $\sqrt{n}$ gibi sayıları tam sayılara yuvarlama konusunda endişe etmemize gerek yok. $m = \lg n$ olarak alırsak,

$$T(2^m) = 2T(2^{m/2}) + m$$

elde ederiz.

Şimdi $S(m) = T(2^m)$ yeni özyinelemesi elde etmek için

$$S(m) = 2S(m/2) + m$$

olarak alabiliriz. Bu özyineleme (4.19) ile oldukça benzerdir. Aslında, bu yeni özyineleme $S(m) = O(m \lg m)$ çözümüne sahiptir. $S(m)$ i yerine tekrar $T(n)$ ile yer değiştirsek,

$$T(n) = T(2^m) = S(m) = O(m \lg m) = O(\lg n \lg \lg n) \text{ ifadesini elde ederiz.}$$

Alıştırmalar

4.3-1

$T(n) = T(n-1) + n$ nin çözümünün $O(n^2)$ olduğunu gösteriniz.

4.3-2

$T(n) = T(\lceil n/2 \rceil) + 1$ in çözümünün $O(\lg n)$ olduğunu gösteriniz.

4.3-3

$T(n) = 2T(\lfloor n/2 \rfloor) + n$ nin çözümünün $O(n \lg n)$ olduğunu gördük. Ayrıca bu özyinelemenin çözümünün $\Omega(n \lg n)$ olduğunu da gösteriniz. Çözümün $\Theta(n \lg n)$ olduğu sonucuna varınız.

4.3-4

Farklı tümevarım hipotezleri yaparak, tümevarımsal kanıtın sınır değerlerini uygulamadan özyineleme (4.19) için $T(1) = 1$ sınır koşulunun zorluğunun üstesinden gelebileceğimizi gösteriniz.

4.3-5

Birleştirerek sıralama için (4.3) “tam” özyinelemesinin çözümünün $\Theta(n \lg n)$ olduğunu gösteriniz.

4.3-6

$T(n) = 2T(\lfloor n/2 \rfloor + 17) + n$ in çözümünün $O(n \lg n)$ olduğunu gösteriniz.

4.3-7

Kısım 4.5 teki ana yöntemi kullanarak, $T(n) = 4T(n/3) + n$ özyinelemesinin çözümünün $T(n) = \Theta(n^{\log_3 4})$ olduğunu gösteriniz. Ayrıca, $T(n) \leq cn^{\log_3 4}$ kabulü ile yerine koyma yönteminin çalışmadığını gösteriniz. Daha sonra alt-sıralı terimi çıkarmanın özyineleme yönteminin çalışmasını nasıl sağladığını gösteriniz.

4.3-8

Kısım 4.5 teki ana yöntemi kullanarak, $T(n) = 4T(n/2) + n$ özyinelemesinin çözümünün $T(n) = \Theta(n^2)$ olduğunu gösteriniz. Ayrıca, $T(n) \leq cn^2$ kabulü ile yerine koyma yönteminin çalışmadığını gösteriniz. Daha sonra alt-sıralı terimi çıkarmanın özyineleme yönteminin çalışmasını nasıl sağladığını gösteriniz.

4.3-9

Değişken değişimi yaparak $T(n) = 3T(\sqrt{n}) + \log n$ özyinelemesini çözünüz. Çözümünüz asimptotik olarak dar olmalıdır. Değerlerin bütünüleyici olup olmadığı hakkında endişelenmeyiniz.

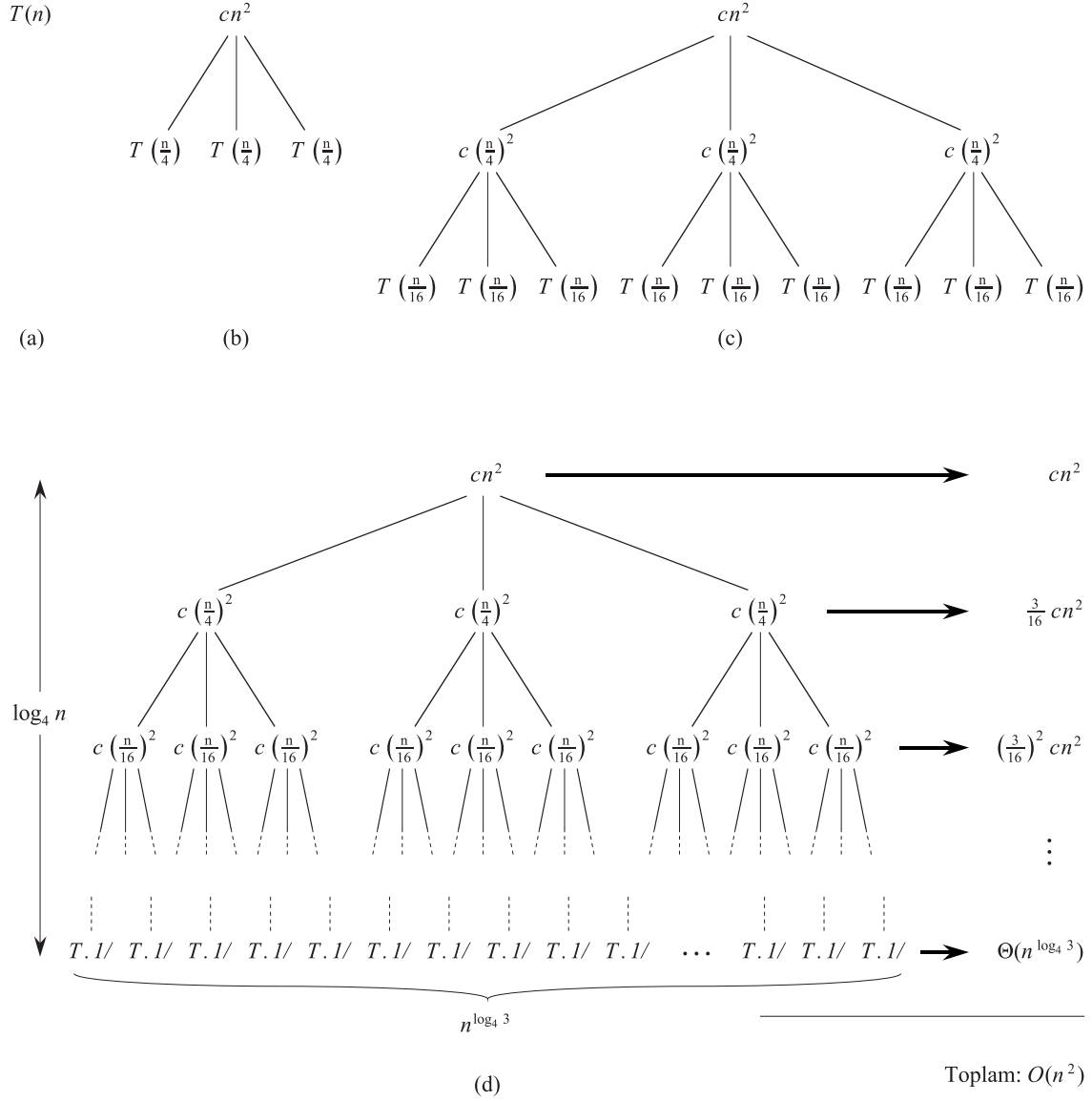
4.4 Özyinelemeleri çözmek için özyineleme-ağacı yöntemi

Bir özyinelemenin çözümünün doğrulayan kısa bir kanıt elde etmek için yerine koyma yöntemi kullanabiliyor olmanıza rağmen, iyi bir tahmin ile gelen sorun yaşayabilirsiniz. Daha önce Kısım 2.3.2 de birleştirerek sıralamada analiz ettiğimiz gibi, yineleme ağacı oluşturmak, bize iyi bir tahmin planlamak için açıkça yardımcı olacaktır. Bir **özyineleme ağacı**nda, her bir dal özyineleme fonksiyonu yürütümleri kümesinde bir yerlerde bir basit altproblemin maliyetini temsil etmektedir. Seviye-başına maliyetlerin kümesini elde etmek için ağacın her bir seviyesindeki maliyetleri toplarız ve sonra özyinelemenin her seviyesinin toplam maliyetini belirlemek için seviye-başına maliyetleri toplarız.

Bir özyineleme ağacı daha sonra yerine koyma yöntemi ile doğrulayabileceğiniz gibi, en çok iyi bir tahmin oluşturmaya yarar. Daha sonra tahmininizi doğrulayacağınızdan dolayı, iyi bir tahmin oluşturmak için özyineleme ağacını kullanıldığında, çoğu zaman küçük miktarda “hatayı” tolere edebilirsiniz. Eğer özyineleme ağacı oluştururken ve maliyetleri toplarken çok dikkatliyseniz, özyineleme ağacını bir özyinelemenin doğrudan kanıtı olarak kullanabilirsiniz. Bu bölümde, özyineleme ağaçlarını iyi tahminler oluşturmak için kullanacağız ve Kısım 4.6 da özyineleme ağaçlarını ana teoremin temelini oluşturan teoremleri doğrudan kanıtlamak için kullanacağız.

Örnek olarak, bir özyineleme ağacının $T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$ özyinelemesi için nasıl iyi bir tahmin temin ettiğini görelim. Çözüm için üst sınır bulmaya yoğunlaşarak işe başlayalım. Çünkü özyinelemeleri çözerken, zeminler ve tavanlar genellikle önemli değildir (İşte bu göz ardı edebileceğimiz kusura bir örnektir), $c > 0$ sabit katsayısı için $T(n) = 3T(n/4) + cn^2$ özyinelemesi için bir özyineleme ağacı oluştururuz.

Şekil 4.5 bize $T(n) = 3T(n/4) + cn^2$ için nasıl özyineleme ağacı oluşturduğumuzu göstermektedir. Uygun olması açısından, n sayısı 4 ün tam kuvveti olup (göz ardı edilebilir kusura başka bir örnek), tüm altproblemler tam sayı boyutludur. Şeklin (a) şikkı (b) şikkındaki özyineleme ağacına denk olarak genişletilebilen $T(n)$ i göstermektedir. Kökteki cn^2 terimi özyinelemenin en üst seviyesindeki maliyeti göstermektedir ve kökün üç altağacı ise $n/4$ boyutlu alt problemlerin neden olduğu maliyeti göstermektedir. Şık (c) ise şık (b) nin $T(n/4)$ maliyetli ağaç dalların genişletilerek bir adım ileriye taşınma işlemini göstermektedir. Kökün üç çocuğunun her birinin maliyeti $c(n/4)^2$ dir. Özyinelemede tanımlandığı şekilde ağaçtaki her bir dalı bileşenlerine ayırarak genişletmeye devam ederiz.



Şekil 4.5 $T(n) = 3T(n/4) + cn^2$ özyinelemesi için özyineleme ağacının inşası. (a) kısmı, $T(n)$ yi göstermektedir, (b)–(d) arasında devamlı genişleyerek özyineleme ağacını oluşturmaktadır. (d) kısmında tamamen açılmış ağaç $\log_4 n$ yüksekliğine sahiptir ($\log_4 n + 1$ seviye sahiptir).

Çünkü altproblem boyutları her seferinde 4 ün katı şekilde azalarak bir seviye aşağıya indiğimizden, nihayetinde sınır koşuluna ulaşırız. Kökten ne kadar uzakta bir tanesine ulaşırız? i derinliğinde, bir dal için altproblem boyutu $n/4^i$ 'tür. Böylece, $n/4^i = 1$ olduğu zaman altproblem boyutu $n = 1$ e gelir ya da benzer şekilde, $i = \log_4 n$ olduğunda. Böylece, ağaç $\log_4 n + 1$ adet seviyeye sahiptir (derinlik seviyeleri $0, 1, 2, \dots, \log_4 n$)

Sırada, ağacın her seviyesindeki maliyeti belirleyelim. Her seviye bir üsttekenden üç tane daha fazla dala sahiptir ve böylece derinlik i deki dal sayısı 3^i dir. Kökten her bir seviye aşağıya indiğimizde altproblemlerin boyutu 4 ün katı olarak düşmektedir, $i = 0, 1, 2, \dots, \log_4 n - 1$ olmak üzere derinlik i deki her dalın maliyeti $c(n/4^i)^2$ dir. Çarpma yaparak, derinlik i deki tüm dalların maliyeti $i = 0, 1, 2, \dots, \log_4 n - 1$ olmak üzere $3^i c(n/4^i)^2 = (3/16)^i cn^2$ olarak bulunur. En alt seviye, yani derinlik $\log_4 n$ de $3^{\log_4 n} = n^{\log_4 3}$ tane dal vardır, $T(1)$ i sabit olarak aldığımızda toplam maliyet $n^{\log_4 3}$ olur ki bu $\Theta(n^{\log_4 3})$ dır ve her bir dal $T(1)$ maliyet oluşturur.

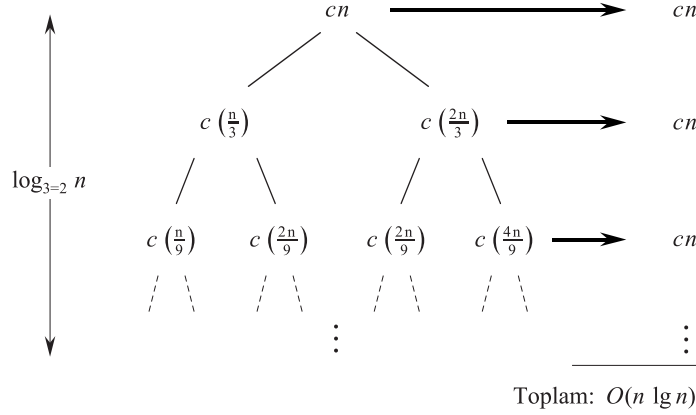
Şimdi tüm seviyedeki dalların maliyetlerini toplayarak, ağacın toplam maliyetini belirlersek:

$$\begin{aligned} T(n) &= cn^2 + \frac{3}{16} cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \dots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3}) \\ &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\ &= \frac{(3/16)^{\log_4 n} - 1}{(3/16) - 1} cn^2 + \Theta(n^{\log_4 3}) \quad (\text{denklem (A.5) ten}). \end{aligned}$$

Kusurların göz ardı edecek kadar küçük oluşu ve azalan geometrik serileri üst sınır olarak kullanıya kadar, bu son formül biraz karışık görünmektedir. Bir adım geri giderek ve denklem (A.6) yı uygulayarak, aşağıdaki eşitliği elde ederiz:

$$\begin{aligned} T(n) &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\ &< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \\ &= \frac{1}{1 - (3/16)} cn^2 + \Theta(n^{\log_4 3}) \\ &= \frac{16}{13} cn^2 + \Theta(n^{\log_4 3}) \\ &= O(n^2). \end{aligned}$$

Böylece orijinal özyinelememiz $T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$ için $T(n) = O(n^2)$ tahminini türetiriz. Bu örnekte, cn^2 nin katsayıları azalan bir



Şekil 4.6 $T(n) = T(n/3) + T(2n/3) + cn$ özyinelemesi için bir özyineleme ağacı.

geometrik seri oluşturur ve denklem (A.6) ile bu katsayılar $16/13$ sabiti ile üstten sınırlıdır. Kökün toplam maliyete katkısı cn^2 olduğundan, kök toplam maliyetin sabit kısmını verir. Diğer bir ifadeyle, kökün maliyeti ağacın toplam maliyetine hakim olur.

Aslında, $O(n^2)$ özyineleme için bir üst sınır ise (gerçeklenecektir), o halde dar bir sınırdır. Neden? İlk özyineleme çağrısı $\Theta(n^2)$ maliyetine sahiptir ve bu yüzden $\Omega(n^2)$ özyineleme için alt sınır olmalıdır.

Şimdi yerine koyma yöntemini, $T(n) = O(n^2)$ nun $3T(\lfloor n/4 \rfloor) + \Theta(n^2)$ özyinelemesi için üst sınır olduğu üzerine yaptığımız tahminimizin doğru olduğunu göstermek için kullanılabiliriz. $d > 0$ sabiti için $T(n) \leq dn^2$ olduğunu göstermek istiyoruz. $c > 0$ sabitini daha önceden kullandığımız gibi kullanarak,

$$\begin{aligned}
 T(n) &\leq 3T(\lfloor n/4 \rfloor) + cn^2 \\
 &\leq 3d \lfloor n/4 \rfloor^2 + cn^2 \\
 &\leq 3d(n/4)^2 + cn^2 \\
 &= \frac{3}{16} dn^2 + cn^2 \\
 &\leq dn^2
 \end{aligned}$$

elde ederiz, burada son adım $d \geq (16/13)c$ olduğu sürece sağlanır.

Diğer taraftan, daha karmaşık bir örnek verirse, Şekil 4.6

$$T(n) = T(n/3) + T(2n/3) + O(n)$$

için özyineleme ağacını göstermektedir. (Tekrar, basitlik, basitlik olması açısından zemin ve tavan fonksiyonlarını tekrardan dahil etmeyeceğiz). Daha önceden olduğu gibi, c bize $O(n)$ teriminde sabit çarpanı gösterebilir. Şekilde gösterilen özyineleme ağacının tüm için özyineleme ağacını göstermektedir. (Tekrar, basitlik boyunca değerleri toplarsak, tüm seviyeler için cn değerini elde ederiz.

Kökten bir yaprağa en uzun basit yol $n \rightarrow (2/3)n \rightarrow (2/3)^2n \rightarrow \dots \rightarrow 1$ dir. $k = \log_{3/2} n$ iken $(2/3)^k n = 1$ olduğundan, ağacın yüksekliği $\log_{3/2} n$ dir.

Sezgisel olarak, özyinelemenin çözümünün maliyetinin en fazla seviye sayısı kere her bir seviyenin maliyeti kadar olmasını bekleriz yada $O(cn \log_{3/2} n) = O(n \lg n)$ olmasını. Şekil 4.6 bize özyineleme ağacının sadece en üst seviyelerini göstermektedir. Ağaçtaki her seviyenin maliyeti cn değildir. Mesela yaprakların maliyeti. Eğer bu özyineleme ağacı $\log_{3/2} n$ yükseklikte tam ikili ağaç olsaydı, $2^{\log_{3/2} n} = n^{\log_{3/2} 2}$ yaprak olurdu. Her yaprağın maliyeti sabit olduğundan, tüm yaprakların toplam maliyeti $\Theta(n^{\log_{3/2} 2})$ olurdu yani $\omega(n \lg n)$, çünkü $\log_{3/2} 2$ sayısı 1 den büyük bir sabittir. Bu özyineleme ağacı tam ikili ağaç değildir, sonuç olarak yaprak sayısı $n^{\log_{3/2} 2}$ yapraktan daha azdır. Dahası, kökten aşağıya indikçe daha fazla iç dal eksiktir. Sonuç olarak, özyineleme ağacının alt seviye dallarına doğru indikçe maliyet toplam maliyete cn 'den daha az katkı sağlayacaktır. Tüm maliyeti doğru hesaplayabiliriz, fakat unutmamak gerekir ki, biz yerine koyma yönteminde kullanmak üzere tahmin oluşturmayı deniyoruz. Kusura göz ardı edelim ve üst sınır için $O(n \lg n)$ tahminini gösterme girişimimizin doğru olduğunu kabul edelim.

Aslında, yerine koyma yönteminin özyineleme çözümü için $\log_{3/2} n$ nin üst olduğunu göstermek için kullanabiliriz. d uygun pozitif bir sabit olmak üzere, $T(n) \leq dn \lg n$ olduğunu gösteririz. $d \geq c/(\lg 3 - (2/3))$ olmak üzere,

$$\begin{aligned}
 T(n) &\leq T(n/3) + T(2n/3) + cn \\
 &\leq d(n/3) \lg(n/3) + d(2n/3) \lg(2n/3) + cn \\
 &= (d(n/3) \lg n - d(n/3) \lg 3) \\
 &\quad + (d(2n/3) \lg n - d(2n/3) \lg(3/2)) + cn \\
 &= dn \lg n - d((n/3) \lg 3 + (2n/3) \lg(3/2)) + cn \\
 &= dn \lg n - d((n/3) \lg 3 + (2n/3) \lg 3 - (2n/3) \lg 2) + cn \\
 &= dn \lg n - dn(\lg 3 - 2/3) + cn \\
 &\leq dn \lg n
 \end{aligned}$$

dir. Böylece, özyineleme ağacında maliyetin daha doğru bir hesabını yapmamıza gerek duymadık.

Alıştırmalar

4.4-1

Bir özyineleme ağacı kullanarak, $T(n) = 3T(\lfloor n/2 \rfloor) + n$ özyinelemesinde iyi bir asimptotik üst sınır belirleyiniz. Bu tahmini, yerine koyma yöntemi ile gerçekleyiniz.

4.4-2

Bir özyineleme ağacı kullanarak, $T(n) = T(n/2) + n^2$ özyinelemesinde iyi bir asimptotik üst sınır belirleyiniz. Bu tahmini, yerine koyma yöntemi ile gerçekleyiniz.

4.4-3

Bir özyineleme ağacı kullanarak, $T(n) = 4T(n/2 + 2) + n$ özyinelemesinde iyi bir asimptotik üst sınır belirleyiniz. Bu tahmini, yerine koyma yöntemi ile gerçekleyiniz.

4.4-4

Bir özyineleme ağacı kullanarak, $T(n) = 2T(n - 1) + 1$ özyinelemesinde iyi bir asimptotik üst sınır belirleyiniz. Bu tahmini, yerine koyma yöntemi ile gerçekleyiniz.

4.4-5

Bir özyineleme ağacı kullanarak, $T(n) = T(n - 1) + T(n/2) + n$ özyinelemesinde iyi bir asimptotik üst sınır belirleyiniz. Bu tahmini, yerine koyma yöntemi ile gerçekleyiniz.

4.4-6

c bir sabit olmak üzere, bir özyineleme ağacını kullanarak $T(n) = T(n/3) + T(2n/3) + cn$ özyinelemesinin çözümünün $\Omega(n \lg n)$ olduğunu gösteriniz.

4.4-7

c bir sabit olmak üzere, $T(n) = 4T(\lfloor n/2 \rfloor) + cn$ için bir özyineleme ağacı çizin, çözümü için dar bir asimptotik sınır elde ediniz. Sınırlarınızı yerine koyma yöntemi ile gerçekleyiniz.

4.4-8

$a \geq 1$ ve $c > 0$ ler sabitler olsun. $T(n) = T(n - a) + T(a) + cn$ özyinelemesi için özyineleme ağacını kullanarak asimptotik olarak kesin bir çözüm veriniz.

4.4-9

$0 < \alpha < 1$ ve $c > 0$ ler sabitler olsun. $T(n) = T(\alpha n) + T((1 - \alpha)n) + cn$ özyinelemesi için özyineleme ağacını kullanarak asimptotik olarak kesin bir çözüm veriniz.

4.5 Özyinelemeleri çözmek için ana yöntem

$a \geq 1$ ve $b \geq 1$ sabitler ve $f(n)$ asimptotik olarak pozitif fonksiyon olmak üzere,

$$T(n) = aT(n/b) + f(n) \quad (4.20)$$

biçimindeki özyinelemelerin çözülmesi için ana yöntem bir “başvuru kitabı” yöntemi sağlar. Ana yöntemi kullanmak için, üç durumu ezberlemek gereklidir, zaten sonrasında kâğıt kalem kullanmaksızın, bir çok özyinelemeyi kolayca çözebilirsiniz.

Burada a ve b pozitif sabitler olmak üzere, (4.20) deki özyineleme, boyutu n olan bir problemin, her bir parçası n/b boyutlu, a altprobleme bölünmesinde geçen sürenin algoritmasını tanımlar. $T(n/b)$ zamanlı a altproblemlerin her biri özyinelemeyle çözülür. $f(n)$ fonksiyonu, problem bölme maliyetini ve altproblemlerin sonuçlarının birleştirilmesini kapsar. Örneğin özyineleme, $a = 7$, $b = 2$ ve $\Theta(n^2)$ ye sahip Strassen'in algoritmasından ileri gelmiştir.

n/b bir tam sayı olamayacağından teknik doğruluk gereği özyineleme aslında iyi tanımlı değildir. Fakat, a tane $T(n/b)$ teriminin her birinin $T(\lfloor n/b \rfloor)$ ya da $T(\lceil n/b \rceil)$ ile yer değiştirmesi özyinelemenin asimptotik davranışını etkilemeyecektir. (Bir sonraki kısımda bu iddiayı kanıtlayacağız.) Dolayısıyla, bu şekildeki böl-ve-yönet özyinelemelerini yazarken zemin ve tavan fonksiyonlarını çıkarmak için çoğunlukla onu kullanışlı buluruz.

Ana teorem

Ana yöntem aşağıdaki teoreme bağlıdır.

Teorem 4.1 (Ana teorem)

$a \geq 1$ ve $b > 1$ sabitler, $f(n)$ bir fonksiyon olsun ve $T(n)$ negatif olmayan tam sayılar üzerinde

$$T(n) = aT(n/b) + f(n)$$

özyinelemesiyle tanımlansın, burada n/b yi ya $\lfloor n/b \rfloor$ ya da $\lceil n/b \rceil$ anlamında yorumlarız. O zaman $T(n)$ aşağıdaki asimptotik sınırlara sahiptir:

1. Eğer bazı $\epsilon > 0$ sabiti için $f(n) = O(n^{\log_b a - \epsilon})$ ise o zaman $T(n) = \Theta(n^{\log_b a})$ dir.
2. Eğer $f(n) = O(n^{\log_b a})$ ise o zaman $T(n) = \Theta(n^{\log_b a} \lg n)$ dir.
3. Eğer bazı $\epsilon > 0$ sabiti için $f(n) = \Omega(n^{\log_b a + \epsilon})$ ve bazı $c < 1$ sabiti ve de yeterince büyük n için $af(n/b) \leq cf(n)$ ise o zaman $T(n) = \Theta(f(n))$ dir. ■

Bazı örnekler için ana teoremi uygulamadan önce, onun ne söylediğini anlamaya zaman harcayalım. Üç durumun her birinde, $f(n)$ fonksiyonunu $n^{\log_b a}$ ile karşılaştırıyoruz. Sezgisel olarak, iki fonksiyondan büyük olan özyineleme için çözümü belirleyecektir. Eğer durum 1 de olduğu gibi $n^{\log_b a}$ fonksiyonu daha büyük ise, çözüm $T(n) = \Theta(n^{\log_b a})$ dir. Eğer durum 3 de olduğu gibi $f(n)$ fonksiyonu daha büyük ise, çözüm $T(n) = \Theta(f(n))$ dir. Eğer durum 2 deki gibi iki fonksiyonun büyüklüğü aynı ise, logaritmik bir faktör ile çarpabiliriz ve çözüm $T(n) = \Theta(n^{\log_b a} \lg n) = \Theta(f(n) \lg n)$ olur.

Bu sezginin ötesinde, bazı teknik ayrıntıların bilincinde olmaya gerek vardır. İlk durumda, sadece $f(n)$ nin $n^{\log_b a}$ dan küçük olması değil, polinomiyal küçük

olması zorunludur. Yani bazı $\epsilon > 0$ sabiti için $f(n)$, $n^{\log_b a}$ dan bir n^ϵ vasıtasıyla asimptotik olarak daha küçük olmak zorundadır. Üçüncü durumda sadece $f(n)$ nin $n^{\log_b a}$ den büyük olması yeterli değil aynı zamanda polinomiyal büyük olmalı ve $af(n/b) \leq cf(n)$ “düzenlilik” koşulunu sağlamak zorundadır. Bu durum karşılık getirebileceğimiz polinomiyal sınırlı fonksiyonların çoğuyla gerçekleşir.

Bu üç durumun $f(n)$ için tüm olası durumları karşılamadığına dikkat edelim. $f(n)$, $n^{\log_b a}$ dan küçük ama polinomiyal küçük olmadığı zaman, durum 1 ve durum 2 arasında bir boşluk vardır. Benzer şekilde $f(n)$, $n^{\log_b a}$ dan büyük fakat polinomiyal büyük olmadığı zaman, durum 2 ve durum 3 arasında da bir boşluk vardır. Eğer $f(n)$ fonksiyonu bu boşluklardan birine düşer veya üçüncü durumdaki düzenlilik koşulunu sağlamazsa, özyinelemenin çözümü için ana yöntemi kullanamazsınız.

Ana yöntemin kullanımı

Ana yöntemi kullanmak için, hangi durumun uygulanacağını basitçe belirleyeceğiz ve cevabımızı kağıda dökeceğiz.

İlk örnek olarak,

$$T(n) = 9T(n/3) + n$$

i düşünelim.

Bu özyineleme için, $a = 9$, $b = 3$, $f(n) = n$ ve dolayısıyla da $n^{\log_b a} = n^{\log_3 9} = \Theta(n^2)$ ye sahibiz. $\epsilon = 1$ için, $f(n) = O(n^{\log_3 9 - \epsilon})$ olduğundan, ana teoremin 1. Durumunu uygulayabiliriz ve $T(n) = \Theta(n^2)$ sonucunu elde ederiz.

Şimdi $a = 1$ $b = 3/2$, $f(n) = 1$ ve $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$ için

$$T(n) = (2n/3) + 1$$

i düşünelim. $f(n) = \Theta = (n^{\log_b a}) = \Theta(1)$ olduğundan durum 2 uygulanır ve böylece özyinelemenin çözümü $f(n) = \Theta(\lg n)$ olur.

$$T(n) = 3T(n/4) + n \lg n$$

özyinelemesi için, $a = 3$, $b = 4$, $f(n) = n \lg n$ ve $n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$ e $\epsilon \approx 0.2$ sahibiz. $f(n) = \Omega(n^{\log_4 3 + \epsilon})$ için, olduğundan dolayı eğer $f(n)$ nin düzenlilik koşulunu sağladığını gösterirsek durum 3 uygulanır. Yeterince büyük n ve $c = 3/4$ için $af(n/b) = 3(n/4) \lg(n/4) \leq (3/4)n \lg n = cf(n)$ for $c = 3/4$ e sahibiz. Sonuç olarak, durum 3 den bu özyinelemeye sonucu $T(n) = \Theta(n \lg n)$ dir.

Ana yöntem

$$T(n) = 2T(n/2) + n \lg n$$

$a = 2$, $b = 2$, $f(n) = n \lg n$ ve $n^{\log_b a} = n$ için uygun bir forma sahipmiş gibi görünse de bu özyinelemeye uygulanamaz. $f(n) = n \lg n$, $n^{\log_b a} = n$ den asimp-

totik olarak daha büyük olduğundan durum 3 ün uygulanabileceğini yanlışlıkla düşünebilirsiniz. Buradaki problem polinomiyal olarak büyük olmamasıdır. Herhangi pozitif bir ϵ sabiti için, $f(n)/n^{\log_b a} = (n \lg n)/n = \lg n$ oranı asimptotik olarak n^ϵ dan daha azdır. Sonuç olarak özyineleme durum 2 ve durum 3 arasında bir boşluğa düşer. (Bir çözüm için Alıştırma 4.6-2 ye bakınız.)

Kısım 4.1 ve 4.2 de gördüğümüz özyinelemeleri çözmek için ana yöntemi kullanalım. Özyineleme 4.7,

$$T(n) = 2T(n/2) + \Theta(n),$$

maksimum altdizi problemi ve birleştirerek sıralamanın ikisi için de böl-ve-yönet algoritmasının geçen süresini karakterize eder. (Bizim uygulamamız için, özyinelemede belirtilen temel durumu ihmal ediyoruz.) Burada, $a = 2, b = 2, f(n) = (n)$ ve dolayısıyla $n^{\log_b a} = n^{\log_2 2} = n$ e sahibiz. $f(n) = \Theta(n)$ olduğu için durum $\Theta 2$ uygulanır ve böylece $T(n) = \Theta(n \lg n)$ çözümünü elde ederiz.

Özyineleme (4.17)

$$T(n) = 8T(n/2) + \Theta(n^2),$$

matris çarpımı için gördüğümüz ilk böl-ve-yönet algoritmasında geçen süreyi tanımlar. Şimdi $a = 8, b = 2$ ve $f(n) = \Theta(n^2)$ sahibiz ve bundan dolayı $n^{\log_b a} = n^{\log_2 8} = n^3$ dür. $n^3, f(n)$ den polinomiyal olarak daha büyük olduğundan (yani, $\epsilon = 1$ için $f(n) = O(n^{3-\epsilon})$) durum 1 uygulanır ve $T(n) = \Theta(n^3)$ olur.

Son olarak,

$$T(n) = 7T(n/2) + \Theta(n^2)$$

Strassen'in algoritmasının çalışma süresini tanımlayan özyineleme (4.18) i düşünelim. Burada $a = 7, b = 2$ ve $f(n) = \Theta(n^2)$ ye ve böylece $n^{\log_b a} = n^{\log_2 7}$ ye sahibiz. $\log_2 7$ yi $\lg 7$ olarak yazalım ve $2.80 < \lg 7 < 2.81$ olduğunu anımsayıp, $\epsilon = 0.8$ için $f(n) = O(n \lg^{7-\epsilon})$ a bakalım. Yeniden durum 1 uygulanır ve $T(n) = \Theta(n^{\lg 7})$ çözümü elde edilir.

Alıştırmalar

4.5-1

Aşağıdaki özyinelemelerde sıkı asimptotik sınırlar vermek için ana yöntemi kullanınız.

a. $T(n) = 2T(n/4) + 1.$

b. $T(n) = 2T(n/4) + \sqrt{n}.$

c. $T(n) = 2T(n/4) + n.$

d. $T(n) = 2T(n/4) + n^2.$

4.5-2

Profesör Caesar, Strassen'in algoritmasından asimptotik olarak daha hızlı bir matris-çarpım algoritması geliştirmek istiyor. Onun algoritması, her bir matrisi $n/4 \times n/4$ boyutundaki parçalara bölen, bu bölme ve birleştirme adımları birlikte $\Theta(n^2)$ zaman alacak olan, böl-ve-yönet yöntemini kullanacaktır. Strassen'in algoritmasını yenmek için, algoritmasının kaç tane altproblem yaratmak zorunda olacağını belirlemesi gerekmektedir. Eğer algoritması a tane altproblem yaratırsa, o halde özyineleme için geçen $T(n)$ zamanı $T(n) = aT(n/4) + \Theta(n^2)$ olur. Profesör Ceaser'in algoritmasının, Strassen'in algoritmasından asimptotik olarak daha hızlı olması için a nın en büyük tam sayı değeri ne olmalıdır?

4.5-3

$T(n) = T(n/2) + \Theta(1)$ ikili-arama özyinelemesinin çözümünün $T(n) = \Theta(\lg n)$ olduğunu göstermek için ana yöntemi kullanınız (İkili-araştırmanın bir tanımlaması için alıştırma 2.3-5 e bakınız).

4.5-4

$T(n) = 4T(n/2) + n^2 \lg n$ özyinelemesi için ana yöntem uygulanır mı? Neden uygulanır veya uygulanmaz? Bu özyineleme için bir asimptotik üst sınır veriniz.

4.5-5 ★

Ana teoremin 3. durumunun parçası olan, bazı $c < 1$ sabiti için $af(n/b) \leq cf(n)$ düzenlilik koşulunu düşünelim. Sabitleri $a \geq 1$ ve $b > 1$ ve de düzenlilik koşulu hariç ana teoremin 3. durumundaki bütün koşulları sağlayan bir $f(n)$ fonksiyonu örneği veriniz.

★ 4.6 Ana teoremin kanıtı

Bu bölüm ana teoremin (Teorem 4.1) bir kanıtını içeriyor. Ana teoremi uygulamak için kanıtını anlamaya gerek yoktur.

Kanıt iki bölümden oluşuyor. İlk kısım $T(n)$ nin sadece $b > 1$ in tam kuvvetlerinin, yani $n = 1, b, b^2, \dots$ için varsayımın basitleştirilmesi altında ana özyinelemeyi analiz eder. Bu kısım ana teoremin neden doğru olduğunu anlamaya gerekli tüm sezgileri veriyor. İkinci kısım analizin bütün n pozitif tam sayıları için nasıl genişletileceğini gösteriyor; zemin ve tavan problemlerinin kullanımı için matematiksel teknik olarak uygulanıyor.

Bu bölümde sadece b nin tam kuvvetleri üzerinde tanımlı fonksiyonların davranışını tanımlamak için, az da olsa bazen asimptotik gösterimlerimizi suistimal edeceğiz. Asimptotik gösterimlerin tanımlarının, sınırları yeterince büyük sayılar

için, bunlar sadece b nin kuvvetleri değil, kanıtlanması gerektiğini hatırlatalım. Negatif olmayan tam sayılar yerine, sadece $\{b^i : i = 0, 1, 2, \dots\}$ kümesine uygulanabilecek yeni asimptotik gösterimler yapabileceğimiz için bu suistimaller minördür.

Yine de kısıtlı bir tanım kümesi üzerinde asimptotik gösterim kullanırken yanlış sonuçlar çıkartmamak için her zaman uyanık olmak zorundayız. Örneğin, n nin 2 nin bir tam kuvveti olduğunda $T(n) = O(n)$ kanıtlamak $T(n) = O(n)$ olduğunu garanti etmez. n nin bütün değerleri için uygulanan en iyi üst sınır $T(n) = O(n^2)$ olması durumunda $T(n)$ fonksiyonu aşağıdaki gibi tanımlanabilir

$$T(n) = \begin{cases} n & n = 1, 2, 4, 8, \dots \text{ ise,} \\ n^2 & \text{aksi halde,} \end{cases}$$

Kesin sonucun bu durumundan dolayı, kullandığımız bağlamdan tamamıyla kesin yargıya varmadan sonlu bir küme üzerinde asimptotik gösterimleri kesinlikle kullanmamalıyız.

4.6.1 Tam kuvvetler için kanıt

Ana teoremin kanıtının ilk kısmı, $n, b > 1$ in tam kuvveti varsayımı altında, burada b tam sayı olmak zorunda değil, ana yöntem için

$$T(n) = aT(n/b) + f(n),$$

(4.20) özyinelemesini çözümler. Bu analizi üç lemmaya ayırıyoruz. İlki, ana özyinelemenin çözüm problemini, bir toplam içeren ifadenin değerlendirme problemine indiriyor. İkincisi, bu toplamın sınırlarını belirliyor. Üçüncü lemma ise; n, b nin bir tam kuvveti olması durumunda, ana teoremin bir versiyonunu kanıtlamak için, ilk ikisini birlikte ifade ediyor.

Lemma 4.2

$a \geq 1$ ve $b > 1$ olacak şekilde sabit ve $f(n)$, b nin tam kuvvetleri üzerinde tanımlı negatif olmayan bir fonksiyon olsun. i pozitif bir tam sayı olmak üzere; $T(n)$, b nin kuvvetleri üzerinde aşağıdaki özyinelemeyle tanımlansın:

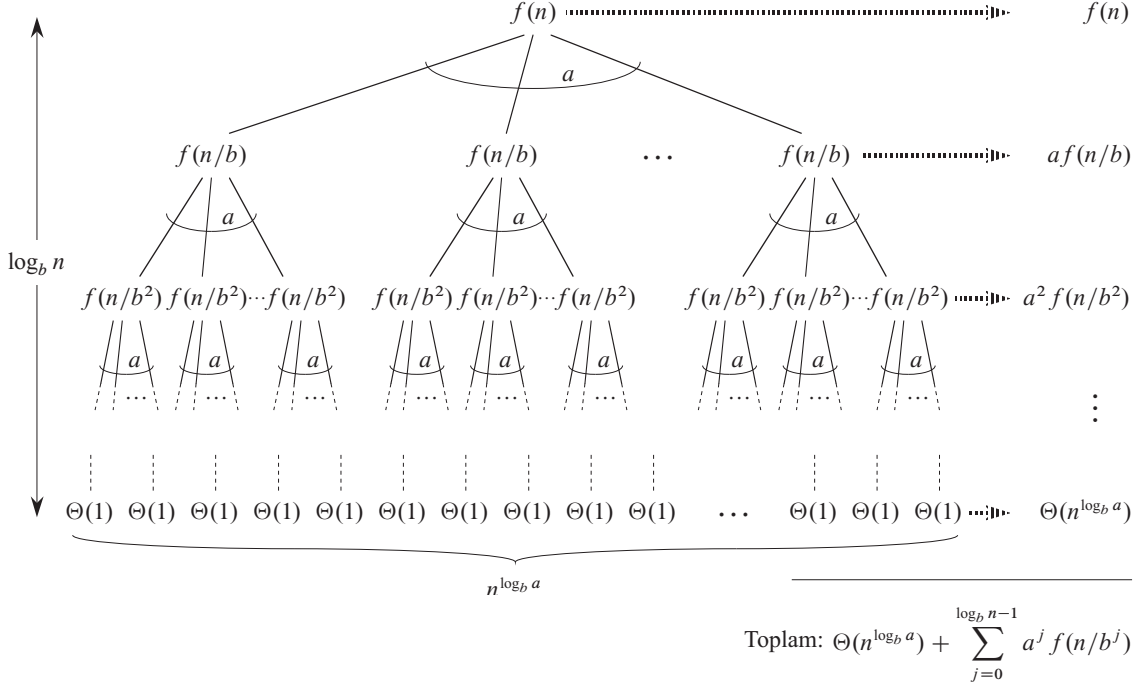
$$T(n) = \begin{cases} \Theta(1) & n = 1 \text{ ise,} \\ aT(n/b) + f(n) & n = b^i \text{ ise,} \end{cases}$$

burada i bir pozitif tam sayıdır. O halde;;

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j) \quad (4.21)$$

dir.

Kanıt Şekil 4.7 deki özyineleme ağacını kullanalım. Bu ağacın kökü $f(n)$ maliyetine ve her birinin maliyeti $f(n/b)$ olan a çocuğa sahiptir. (Özellikle özyineleme



Şekil 4.7 Özyineleme ağacı, $T(n) = aT(n/b) + f(n)$ tarafından doğurulur. Bu ağaç, $n^{\log_b a}$ yaprağa ve $\log_b n$ yüksekliğe sahip a -lı bir tam ağaçtır. Her bir derinlikteki düğümlerin maliyeti sağ tarafta gösterildi ve toplamaları (4.21) eşitliğinde verildi.

ağacını görselleştirdiğimizde a yı bir tam sayıymış gibi düşünmek uygun olur fakat matematiğin buna ihtiyacı yoktur. Bu çocukların her biri, derinliği 2 olan a^2 düğüm yapan, a tane çocuğa ve bu a çocuğun her biri de $f(n/b^2)$ maliyetine sahiptir. Genelde, j derinlikte a^j vardır ve her biri $f(n/b^j)$ maliyetine sahiptir. Her bir yaprağın maliyeti $T(1) = \Theta(1)$ dir ve $n/b^{\log_b n} = 1$ olduğundan her yaprak $\log_b n$ derinliktedir. Ağaçta $a^{\log_b n} = n^{\log_b a}$ yaprak vardır.

Şekilde de gösterildiği gibi, ağaçtaki her bir derinlikteki düğümlerin maliyetini toplayarak denklemini elde edebiliriz. j derinlikli bütün iç düğümlerin maliyeti, $a^j f(n/b^j)$ dir ve dolayısıyla bütün iç düğümlerin toplam maliyeti

$$\sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$$

dir. Böl-ve-yönet algoritmasında temel olarak bu toplam, problemlerin altproblemlere bölünüşü ve sonra altproblemlerin yeniden birleştirilmesinin maliyetini

sunmaktadır. Bütün yaprakların maliyeti, boyutu 1 olan tüm $n^{\log_b a}$ altproblemlerini meydana getirmenin maliyeti $\Theta(n^{\log_b a})$ dır. ■

Özyineleme ağacı cinsinden, ana teoremin üç durumu, ağacın toplam maliyetinin (1) yapraklardaki maliyetler tarafından hakim olunmuş, (2) ağacın seviyeleri arasında eşit olarak dağıtılmış ya da (3) kökün maliyeti tarafından hakim olunmuş durumlarına karşılık gelir.

(4.21) eşitliğindeki toplam, böl-ve-yönet algoritmasının temelindeki bölmenin ve adımları birleştirmenin maliyetini tanımlar. Bir sonraki lemma, toplamsal büyüme üzerindeki asimptotik sınırları sağlar.

Lemma 4.3

$a \geq 1$ ve $b > 1$ olacak şekilde sabit ve $f(n)$, b nin tam kuvvetleri üzerinde tanımlı negatif olmayan bir fonksiyon olsun. b nin tam kuvvetleri üzerinde,

$$g(n) = \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j) \quad (4.22)$$

ile tanımlı bir $g(n)$ fonksiyonu, b nin tam kuvvetleri için aşağıdaki asimptotik sınırlara sahiptir:

1. Bazı $\epsilon > 0$ sabiti için $f(n) = O(n^{\log_b a - \epsilon})$ ise o zaman $g(n) = O(n^{\log_b a})$ dir.
2. $f(n) = \Theta(n^{\log_b a})$ ise o zaman $g(n) = \Theta(n^{\log_b a} \lg n)$ dir.
3. Bütün yeterince büyük n ve bazı $c < 1$ sabiti için eğer $af(n/b) \leq cf(n)$ ise o zaman $g(n) = \Theta(f(n))$ dir.

Kanıt Durum 1 için, $f(n/b^j) = O((n/b^j)^{\log_b a - \epsilon})$ i gerektiren $f(n) = O(n^{\log_b a - \epsilon})$ eşitliğine sahibiz. Denklem (4.22) de yerine koyduğumuzda

$$g(n) = O\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right) \quad (4.23)$$

ü elde ederiz. Yaprakları artan geometrik seri olan dış faktörleri etkinleştirerek ve basitleştirerek O -notasyonu dahilinde toplamı sınırlandırabiliriz:

$$\begin{aligned} \sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \epsilon} &= n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} \left(\frac{ab^\epsilon}{b^{\log_b a}}\right)^j \\ &= n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} (b^\epsilon)^j \\ &= n^{\log_b a - \epsilon} \left(\frac{b^{\epsilon \log_b n} - 1}{b^\epsilon - 1}\right) \end{aligned}$$

$$= n^{\log_b a - \epsilon} \left(\frac{n^\epsilon - 1}{b^\epsilon - 1} \right).$$

b ve ϵ sabit oldukları için, en son ifademiz $n^{\log_b a - \epsilon} O(n^\epsilon) = O(n^{\log_b a})$ olarak yeniden yazılabilir. Denklem (4.23) deki toplamda bu ifade yerine konulursa

$$g(n) = O(n^{\log_b a})$$

elde edilir, böylece durum 1 kanıtlanmış olur.

Durum 2 $f(n) = \Theta(n^{\log_b a})$, varsaydığı için, $f(n/b^j) = \Theta((n/b^j)^{\log_b a})$ eşitliğine sahibiz. Denklem (4.22) de yerine koyduğumuzda

$$g(n) = \Theta \left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j} \right)^{\log_b a} \right) \quad (4.24)$$

yı elde ederiz.

Durum 1 deki gibi toplamı Θ -notasyonu içinde sınırlandırırız, ama bu sefer de geometrik bir seri elde edemeyiz. Buna karşılık, toplamdaki her bir terimin aynı olduğunu bulacağız:

$$\begin{aligned} \sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j} \right)^{\log_b a} &= n^{\log_b a} \sum_{j=0}^{\log_b n - 1} \left(\frac{a}{b^{\log_b a}} \right)^j \\ &= n^{\log_b a} \sum_{j=0}^{\log_b n - 1} 1 \\ &= n^{\log_b a} \log_b n. \end{aligned}$$

Bu ifadeyi denklem (4.24) deki toplamda yerine koyduğumuzda,

$$\begin{aligned} g(n) &= \Theta(n^{\log_b a} \log_b n) \\ &= \Theta(n^{\log_b a} \lg n), \end{aligned}$$

elde edilir, durum 2 kanıtlanır.

Durum 3 ü benzer şekilde kanıtlarız. $f(n)$, $g(n)$ nin (4.22) tanımında olduğundan ve $g(n)$ nin bütün terimleri negatif olmadığından, b nin tam kuvvetleri için $g(n) = \Omega(f(n))$ i sonuçlandırabiliriz. Yeterince büyük bütün n ve bazı $c < 1$ sabiti için lemmadaki durumdan $af(n/b) \leq cf(n)$ varsayabiliriz. Bu varsayımı $f(n/b) \leq (c/a)f(n)$ olacak şekilde yeniden yazar ve j defa yinelersek, burada değerlerin özyineleme için yeterince büyük olduğunu varsayıyoruz, $f(n/b^j) \leq (c/a)^j f(n)$ veya denk olarak $a^j f(n/b^j) \leq c^j f(n)$ i elde ederiz. En sonuncu ve en küçük olduğu için bu değer n/b^j dir, n/b^j in yeterince büyük olduğunu varsaymak yeterlidir.

Denklem (4.22) de yerine koyduğumuzda ve basitleştirdiğimizde bi geometrik seri elde edilir, fakat durum 1 deki serilerin aksine, bu azalan terimlere sahip-

tir. c bir sabit olduğundan, varsayımımızdaki yeterince büyük n leri kapsamayan terimleri elde etmek için bir $O(1)$ terimi kullanacağız:

$$\begin{aligned}
 g(n) &= \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j) \\
 &\leq \sum_{j=0}^{\log_b n - 1} c^j f(n) + O(1) \\
 &\leq f(n) \sum_{j=0}^{\infty} c^j + O(1) \\
 &= f(n) \left(\frac{1}{1-c} \right) + O(1) \\
 &= O(f(n)).
 \end{aligned}$$

Böylece b nin tam kuvvetleri için $g(n) = \Theta(f(n))$ i sonuçlandırabiliriz. Durum 3 ün de kanıtlanmasıyla, lemmanın kanıtı sonlandırılır. ■

Şimdi n , b nin bir tam kuvveti olması durumu için ana teoremin bir versiyonunu kanıtlayabiliriz.

Lemma 4.4

$a \geq 1$ ve $b > 1$ olacak şekilde sabit ve $f(n)$, b nin tam kuvvetleri üzerinde tanımlı negatif olmayan bir fonksiyon olsun. i bir pozitif tam sayı olmak üzere, b nin tam kuvvetleri üzerinde $T(n)$:

$$T(n) = \begin{cases} \Theta(1) & n = 1 \text{ ise,} \\ aT(n/b) + f(n) & n = b^i \text{ ise} \end{cases}$$

özyinelemesiyle tanımlansın. O zaman b nin tam kuvvetleri için $T(n)$ aşağıdaki asimptotik sınırlara sahiptir:

1. Bazı $\epsilon > 0$ sabiti için $f(n) = O(n^{\log_b a - \epsilon})$ ise $T(n) = \Theta(n^{\log_b a})$ dır.
2. Eğer $f(n) = \Theta(n^{\log_b a})$ ise o zaman $T(n) = \Theta(n^{\log_b a} \lg n)$ dir.
3. Eğer bazı $\epsilon > 0$ sabiti için, $f(n) = \Omega(n^{\log_b a + \epsilon})$ ve bütün yeterince büyük n ve bazı $c < 1$ sabiti için $af(n/b) \leq cf(n)$ ise $T(n) = \Theta(f(n))$ dir.

Kanıt Lemma 4.2 deki (4.21) toplamını hesaplamak için, Lemma 4.3 deki sınırları kullanacağız. Durum 1 için

$$\begin{aligned}
 T(n) &= \Theta(n^{\log_b a}) + O(n^{\log_b a}) \\
 &= \Theta(n^{\log_b a})
 \end{aligned}$$

ve durum 2 için

$$\begin{aligned} T(n) &= \Theta(n^{\log_b a}) + \Theta(n^{\log_b a} \lg n) \\ &= \Theta(n^{\log_b a} \lg n) . \end{aligned}$$

Durum 3 için

$$\begin{aligned} T(n) &= \Theta(n^{\log_b a}) + \Theta(f(n)) \\ &= \Theta(f(n)) , \end{aligned}$$

e sahibiz çünkü $f(n) = \Omega(n^{\log_b a + \epsilon})$ ■
dir.

4.6.2 Zeminler ve tavanlar

Ana teoremin kanıtını tamamlamak için, özyinelemenin sadece b nin tam kuvvetlerinde değil, bütün tam sayılarda tanımlanabilmesi için, analizimizi ana özyinelemede geçen zeminler ve tavanlar durumuna genişletmeliyiz.

$$T(n) = aT(\lceil n/b \rceil) + f(n) \quad (4.25)$$

üzerinde alt sınır ve

$$T(n) = aT(\lfloor n/b \rfloor) + f(n) \quad (4.26)$$

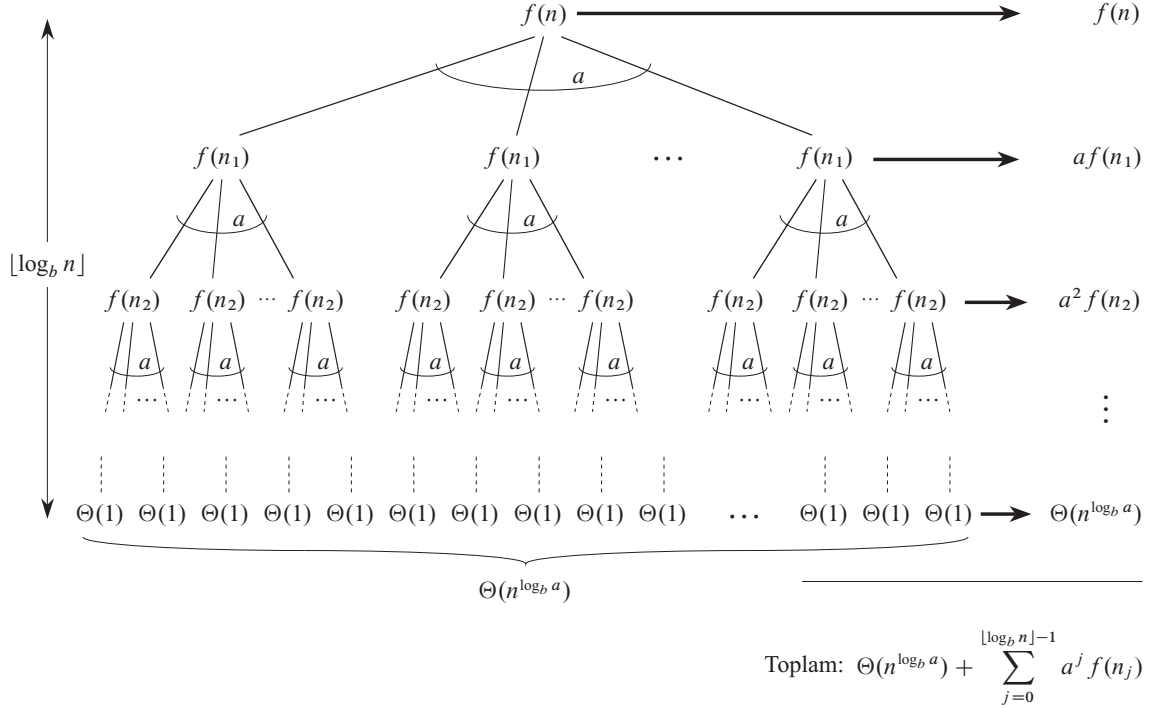
üzerinde üst sınır elde etmek alışlagelmiştir çünkü birinci durumdaki istenen sonuca ulaşmak için $\lceil n/b \rceil \geq n/b$ sınırını kabul ettirebilir ve ikinci durumda $\lfloor n/b \rfloor \leq n/b$ sınırını kabul ettirebiliriz. Aynı tekniği, (4.25) deki üst sınır özyinelemesindeki tekniği, (4.26) daki alt sınır özyinelemesi için kullanırız ve sadece bu sonuncu sınırı vereceğiz.

Şekil 4.8 deki özyineleme ağacını elde etmek için Şekil 4.7 deki özyineleme ağacı üzerinde değişiklik yapacağız. Özyineleme ağacında aşağılara indikçe, argümanlar üzerinde bir dizinin özyinelenebilir yürütmelerini elde ederiz.

$$\begin{aligned} n , \\ \lceil n/b \rceil , \\ \lceil \lceil n/b \rceil / b \rceil , \\ \lceil \lceil \lceil n/b \rceil / b \rceil / b \rceil , \\ \vdots \end{aligned}$$

Aşağıda, dizideki j elemanı n_j ile gösterelim

$$n_j = \begin{cases} n & j = 0 \text{ ise,} \\ \lceil n_{j-1}/b \rceil & j > 0 \text{ ise.} \end{cases} \quad (4.27)$$



Şekil 4.8 Özyineleme ağacı $T(n) = aT(\lceil n/b \rceil) + f(n)$ tarafından doğuruldu. n_j özyineleme argümanını denklem (4.27) verir.

İlk amacımız k nın derinliğini belirlemek, öyle ki n_k bir sabit. $\lceil x \rceil \leq x + 1$ eşitsizliğini kullanarak,

$$\begin{aligned}
 n_0 &\leq n, \\
 n_1 &\leq \frac{n}{b} + 1, \\
 n_2 &\leq \frac{n}{b^2} + \frac{1}{b} + 1, \\
 n_3 &\leq \frac{n}{b^3} + \frac{1}{b^2} + \frac{1}{b} + 1 \\
 &\vdots
 \end{aligned}$$

elde ederiz. Genel olarak,

$$\begin{aligned}
n_j &\leq \frac{n}{b^j} + \sum_{i=0}^{j-1} \frac{1}{b^i} \\
&< \frac{n}{b^j} + \sum_{i=0}^{\infty} \frac{1}{b^i} \\
&= \frac{n}{b^j} + \frac{b}{b-1}
\end{aligned}$$

sahibiz.

$j = \lfloor \log_b n \rfloor$ olsun,

$$\begin{aligned}
n_{\lfloor \log_b n \rfloor} &< \frac{n}{b^{\lfloor \log_b n \rfloor}} + \frac{b}{b-1} \\
&< \frac{n}{b^{\log_b n - 1}} + \frac{b}{b-1} \\
&= \frac{n}{n/b} + \frac{b}{b-1} \\
&= b + \frac{b}{b-1} \\
&= O(1),
\end{aligned}$$

elde ederiz ve $\lfloor \log_b n \rfloor$ derinliğinde, problemin boyutunun en fazla bir sabit olduğunu görürüz.

Şekil 4.8 den

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\lfloor \log_b n \rfloor - 1} a^j f(n_j), \quad (4.28)$$

n nin keyfi bir tam sayı ve b nin bir tam kuvveti olmasına kısıtlama getirilmesi hariç denklem (4.21) ile hemen hemen aynıdır.

Şimdi Lemma 4.3 ün kanıtı için sanki bir benzeriymiş gibi denklem (4.28) den

$$g(n) = \sum_{j=0}^{\lfloor \log_b n \rfloor - 1} a^j f(n_j) \quad (4.29)$$

toplamını hesaplayabiliriz. Durum 3 ile birlikte, eğer $n > b + b/(b-1)$ için, $c < 1$ sabit olmak üzere, $af(\lceil n/b \rceil) \leq cf(n)$ ise o zaman bundan $a^j f(n_j) \leq c^j f(n)$ sonucu çıkar. Bu nedenle, denklem (4.29) daki toplamı tıpkı Lemma 4.3 deki gibi hesaplayabiliriz. Durum 2 için, $f(n) = \Theta(n^{\log_b a})$ ya sahibiz. Eğer $f(n_j) = O(n^{\log_b a} / a^j) = O((n/b^j)^{\log_b a})$ eşitliğini gösterirsek, Lemma 4.3 ün ikinci durumu için kanıtı tamamlamış olacağız. $j \leq \lfloor \log_b n \rfloor$ nin $b^j/n \leq 1$ i gerektirdiğine dikkat edelim. $c(1 + b/(b-1))^{\log_b a}$ bir sabit olduğundan, $f(n) = O(n^{\log_b a})$ sınırı yeterince büyük tüm n_j ler için;

$$\begin{aligned}
f(n_j) &\leq c \left(\frac{n}{b^j} + \frac{b}{b-1} \right)^{\log_b a} \\
&= c \left(\frac{n}{b^j} \left(1 + \frac{b^j}{n} \cdot \frac{b}{b-1} \right) \right)^{\log_b a} \\
&= c \left(\frac{n^{\log_b a}}{a^j} \right) \left(1 + \left(\frac{b^j}{n} \cdot \frac{b}{b-1} \right) \right)^{\log_b a} \\
&\leq c \left(\frac{n^{\log_b a}}{a^j} \right) \left(1 + \frac{b}{b-1} \right)^{\log_b a} \\
&= O \left(\frac{n^{\log_b a}}{a^j} \right)
\end{aligned}$$

olacak şekilde bir $c > 0$ sabitinin var olmasını gerektirir. Böylece durum 2 yi kanıtlamış oluruz. Durum 1 in kanıtı da neredeyse benzerdir. Cebir anlaşılması daha güç olmasına rağmen, $f(n_j) = O((n/b^j)^{\log_b a - \epsilon})$ sınırını kanıtlamak için kullanılacak yöntem, durum 2 ye karşılık gelen kanıta benzerdir.

Şimdi bütün n tam sayıları için ana teoremdaki, üst sınırları kanıtlamış olduk. Alt sınırlar için olan kanıt da buna benzerdir.

Alıştırmalar

4.6-1 ★

Denklem (4.27) de b nin keyfi bir reel sayı olması yerine, pozitif bir tam sayı olması durumunda n_j için basit ve kesin bir ifade veriniz.

4.6-2 ★

$k \geq 0$ olması durumunda eğer $f(n) = \Theta(n^{\log_b a} \lg^k n)$ ise ana özyinelemenin $T(n) = \Theta(n^{\log_b a} \lg^{k+1} n)$ çözümüne sahip olduğunu gösteriniz. Yalın olması için, analizinizi b nin tam kuvvetleriyle sınırlandırınız.

4.6-3 ★

Bazı $c < 1$ için $af(n/b) \leq cf(n)$ düzenlilik koşulunun $f(n) = \Omega(n^{\log_b a + \epsilon})$ olacak şekilde bir $\epsilon > 0$ gerektirmesi bakımından ana teoremin 3.durumunun üzerinde fazla durulmuş olduğunu gösteriniz.

Problemler

4-1 Özyineleme alıştırmaları

Aşağıdaki her bir özyinelemede $T(n)$ için asimptotik üst ve alt sınırları veriniz. $n \leq 2$ için $T(n)$ i sabit varsayın. Sınırlarınızı olabildiğince sıkılaştırınız ve cevaplarınızı gerçekleyiniz.

- a. $T(n) = 2T(n/2) + n^4$.
- b. $T(n) = T(7n/10) + n$.
- c. $T(n) = 16T(n/4) + n^2$.
- d. $T(n) = 7T(n/3) + n^2$.
- e. $T(n) = 7T(n/2) + n^2$.
- f. $T(n) = 2T(n/4) + \sqrt{n}$.
- g. $T(n) = T(n-2) + n^2$.

4-2 Parametrik-geçiş maliyetleri

Bu kitap genelinde, N -elemanlı bir dizi geçirilse bile, yordam çağrıları boyunca parametre geçişlerinin sabit zaman aldığını varsayalım. Bu varsayım bir çok sistemde geçerlidir çünkü dizinin kendisi değil, dizinin işaretleyicisi geçer. Bu problem 3 parametre-geçiş stratejisinin çıkarımını inceler:

1. Bir dizi işaretleyici ile geçirilir. Zaman = $\Theta(1)$
 2. Bir dizi kopyalayarak geçirilir. N dizinin boyutu olmak üzere, Zaman = $\Theta(N)$
 3. Bir dizi sadece çağrı prosedürü tarafından ulaşılabilen alt görüntü kümesi kopyalanarak geçirilir. Eğer $A[p..q]$ geçirilirse Zaman = $\Theta(q - p + 1)$ dir.
- a. N orijinal problemin boyutu ve n bir alt problemin boyutu olsun. Sıralı bir dizide bir sayıyı bulmak için ikili arama algoritmasını göz önüne alınız. (Bakınız Alıştırma 2.3-5) Diziler yukardaki üç yöntemin her birini kullanarak geçildiğinde, ikili aramanın en kötü çalışma zamanları için özyinelemeler ve özyinelemelerin çözümlerindeki iyi üst sınırları veriniz.
 - b. Kısım 2.3.1 deki MERGE-SORT algoritması için (a) kısmını tekrar yapılandırınız.

4-3 Daha fazla özyineleme örnekleri

Aşağıdaki her bir özyinelemede $T(n)$ için asimptotik üst ve alt sınırları veriniz. Yeterince küçük n için $T(n)$ nin sabit olduğunu varsayalım. Sınırlarınızı olabildiğince sıkılaştırınız ve cevaplarınızı gerçekleyiniz.

- a. $T(n) = 4T(n/3) + n \lg n.$
- b. $T(n) = 3T(n/3) + n / \lg n.$
- c. $T(n) = 4T(n/2) + n^2 \sqrt{n}.$
- d. $T(n) = 3T(n/3 - 2) + n/2.$
- e. $T(n) = 2T(n/2) + n / \lg n.$
- f. $T(n) = T(n/2) + T(n/4) + T(n/8) + n.$
- g. $T(n) = T(n - 1) + 1/n.$
- h. $T(n) = T(n - 1) + \lg n.$
- i. $T(n) = T(n - 2) + 1 / \lg n.$
- j. $T(n) = \sqrt{n}T(\sqrt{n}) + n.$

4-4 Fibonacci sayıları

Bu problem (3.22) deki özyinelemeyle tanımlanan Fibonacci sayılarının özelliklerini geliştiriyor. Fibonacci özyinelemesini çözmek için doğuran fonksiyon tekniğini kullanacağız.

F_i i Fibonacci sayısı olmak üzere

$$\begin{aligned} \mathcal{F}(z) &= \sum_{i=0}^{\infty} F_i z^i \\ &= 0 + z + z^2 + 2z^3 + 3z^4 + 5z^5 + 8z^6 + 13z^7 + 21z^8 + \cdots, \end{aligned}$$

olarak $\mathcal{F}$ **doğuran fonksiyonu** (veya **resmi kuvvet serileri**) tanımlansın.

- a. $\mathcal{F}(z) = z + z\mathcal{F}(z) + z^2\mathcal{F}(z)$ olduğunu gösteriniz.

$$b. \phi = \frac{1 + \sqrt{5}}{2} = 1.61803 \dots$$

ve

$$\hat{\phi} = \frac{1 - \sqrt{5}}{2} = -0.61803 \dots$$

olduğunda

$$\begin{aligned} \mathcal{F}(z) &= \frac{z}{1 - z - z^2} \\ &= \frac{z}{(1 - \phi z)(1 - \hat{\phi} z)} \\ &= \frac{1}{\sqrt{5}} \left(\frac{1}{1 - \phi z} - \frac{1}{1 - \hat{\phi} z} \right), \end{aligned}$$

olduğunu gösteriniz.

c. Aşağıdaki eşitliği gösteriniz

$$\mathcal{F}(z) = \sum_{i=0}^{\infty} \frac{1}{\sqrt{5}} (\phi^i - \hat{\phi}^i) z^i.$$

d. En yakın seviyeye yuvarlayarak $i > 0$ olduğunda $F_i = \phi^i / \sqrt{5}$ for $i > 0$ olduğunu kanıtlamak için (c) yi kullanınız. (*İpucu:* $|\hat{\phi}| < 1$ olduğuna dikkat ediniz.)

4-5 Yonga testi

Profesör Diogenes prensipte her birini test etmenin mümkün olduğu n özdeş entegre-devre çok işlemciye sahip olduğu varsayılıyor. Profesörün jig testi, iki çok işlemciyle aynı anda uyum sağlıyor. Jig yüklendiği zaman, her bir çok işlemci diğerini test eder ve iyi veya kötü olduğu hakkında rapor verir. İyi bir çok işlemci kesin olarak diğer çok işlemcinin iyi olup olmadığını her zaman rapor edebilir fakat profesör kötü bir çok işlemcinin cevabına güvenmez. Dolayısıyla testin olası dört çıktısı aşağıdaki gibidir:

A çok işlemcisinin ifadesi	B çok işlemcisinin ifadesi	Sonuç
B iyi	A iyi	ikisi de iyi ya da ikisi de kötü
B iyi	A kötü	en az biri kötü
B kötü	A iyi	en az biri kötü
B kötü	A kötü	en az biri kötü

a. Kötü çok işlemcilerin profesörü kandırmak için anlayabileceğini varsayalım. Eğer $n/2$ çok işlemciden daha fazlası kötüyse, profesör bu şekilde ikili teste dayanan herhangi bir stratejide uygulanan hangi çok işlemcilerin tam olarak belirleyemediğini gösteriniz.

- b. $n/2$ çok işlemciden daha fazlasının iyi olduğunu varsayarak verilen n çok işlemcinin arasından bir tek iyi çok işlemci bulunması problemini ele alalım. $\lfloor n/2 \rfloor$ ikili test problemi boyutun neredeyse yarısından birine indirgemeye yeterli olduğunu gösteriniz.
- c. $n/2$ çok işlemciden daha fazlasının iyi varsayarak, iyi işlemcilerin $\Theta(n)$ ikili testiyle tanımlanabileceğini gösteriniz. Testlerin sayısını ifade eden özyinelemeleri veriniz ve bu özyinelemeleri çözünüz.

4-6 Monge dizileri

Eğer $1 \leq i < k \leq m$ ve $1 \leq j < l \leq n$ olacak şekilde her i, j, k ve l için

$$A[i, j] + A[k, l] \leq A[i, l] + A[k, j] .$$

ye sahipsek, reel sayıların bir $m \times n$ A dizisi **Monge dizisidir**.

Başka bir deyişle, Monge dizisinden iki satır ve iki sütun aldığımızda ve bu satır ve sütunların kesişimindeki dört elemanı ele aldığımızda sol üst ve sağ alttaki elemanların toplamı, sol alt ve sağ üstteki elemanların toplamından küçük veya eşittir. Örneğin aşağıdaki dizi bir Monge dizisidir:

10	17	13	28	23
17	22	16	29	23
24	28	22	34	24
11	13	6	17	7
45	44	32	37	23
36	33	19	21	6
75	66	51	53	34

- a. Bir Dizi Monge'dur eğer ve yalnız eğer her $i = 1, 2, \dots, m - 1$ ve $j = 1, 2, \dots, n - 1$ için

$$A[i, j] + A[i + 1, j + 1] \leq A[i, j + 1] + A[i + 1, j] .$$

dir, gösteriniz (*İpucu*: “Gerek” şart için, satır ve sütunlar üzerinde ayrı ayrı tümevarım uygulayın).

- b. Aşağıdaki dizi Monge dizisi değildir. Monge dizisi olması için bir elemanı değiştiriniz (*İpucu*: (a) yı kullanın).

37	23	22	32
21	6	7	10
53	34	30	31
32	13	9	6
43	21	15	8

- c. $f(i)$, i satırının en soldaki minimum elemanını içeren sütunun indisi olsun. Herhangi bir $m \times n$ Monge dizisi için $f(1) \leq f(2) \leq \dots \leq f(m)$ olduğunu kanıtlayınız.
- d. Bir $m \times n$ A Monge dizisinin her bir satırında en soldaki elemanı hesaplayan böl-ve-yönet algoritmasının bir çıkarımı aşağıda verilmiştir:
- A nın çift-numaralı satırlarını içeren A nın A' altmatrisini oluşturun. A' nün her bir satırı için en soldaki minimumu özyinelemeli olarak belirleyin. Daha sonra A nın tek numaralı satırlarında en soldaki minimumu hesaplayın.
- A nın tek numaralı satırlarında en soldaki minimumu (çift-numaralı satırların en solundaki minimum) $O(m + n)$ zamanda nasıl hesapladığını açıklayın.
- e. (d) de tanımlanan algoritmanın çalışma süresini tanımlayan özyinelemeyi yazınız. Çözümünün $O(m + n \log m)$ olduğunu gösteriniz.

Bölüm notları

Algoritmaların dizaynı için bir teknik olarak kullanılan böl-ve-yönet yöntemi, Karatsuba ve Ofman'ın [194] 1962 deki makalesine kadar dayanır. Bu yöntem önceki zamanlarda da kullanılmış olabilir fakat Heideman, Johnson ve Berrus'a [163] göre C. F. Gauss ilk hızlı Fourier dönüşüm algoritmasını 1805 de tasarladı ve Gauss'un bu formülasyonu; problemi, çözümleri birleştirilen daha küçük alt-problemlere parçalar.

Kısım 4.1 deki maksimum-alt dizisi problemi Bentley tarafından çalışılan problemin ufak bir varyasyonudur. [43, Bölüm 7].

Strassen'in algoritması [325] yayınlandığı 1969 yılında oldukça heyecana neden oldu. Bu tarihten önce, birkaçı basit SQUARE-MATRIX-MULTIPLY prosedüründen asimptotik olarak daha hızlı bir algoritmanın olabirliğini hayal ettiler. O zamandan bu yana matris çarpımı için asimptotik üst sınır geliştirilmektedir. $m \times n$ matrislerinin çarpımı için günümüze kadar gelen asimptotiksel olarak en etkin algoritma, Coppersmith ve Winograd'a [78] göre, geçen $O(n^{2.376})$ süresine sahiptir. Bilinen en iyi alt sınır açıkça $\Omega(n^2)$ sınırır (Açıktır çünkü, matris çarpımın n^2 elemanını doldurmalıyız).

Pratik bir bakış açısından, Strassen'in algoritması aşağıdaki dört nedenden dolayı matris çarpımı için sıklıkla bir seçim yöntemi olarak kullanılmaz.

1. Strassen'in algoritmasının $\Theta(n^{\lg 7})$ çalışma süresindeki gizli sabit çarpan, $\Theta(n^3)$ zaman SQUARE-MATRIX-MULTIPLY prosedüründeki sabit çarpandan daha büyüktür.
2. Matrisler seyrek olduğunda, seyrek matrisler için uyarlanan yöntemler daha hızlıdır.

3. Strassen'in algoritması, SQUARE-MATRIX-MULTIPLY kadar nümerik olarak kararlı değildir. Başka bir deyişle, tam sayı olmayan değerler üzerindeki bilgi-sayar aritmetiğinin kısıtlı tamlığından dolayı büyük hatalar Strassen'in algoritmasında SQUARE-MATRIX-MULTIPLY den daha fazla birikir.
4. Özyinelemenin basamaklarında oluşan altmatrisler çok bellek gerektiriyor.

Son iki neden 1990 lı yıllarda hafifletilmiştir. Higham[167] nümeriksel kararlılıktaki farkın üzerinde fazla durulmuş olduğunu göstermiştir. Strassen'in algoritması bazı uygulamalar için çok fazla nümeriksel kararsız olmasına rağmen diğerlerine göre kabul edilebilir sınırların içindedir. Bailey, Lee ve Simon; Strassen algoritması için gereken hafızayı azaltacak teknikler üzerinde durmuştur. Pratikte, yoğun matrisler için hızlı matris çarpım uygulamaları Strassen algoritmasını "crossover noktasının" üstünde olan matris boyutları için kullanılmıştır ve alt-problemin boyutu crossover noktasının altına düşürüldüğü zaman daha basit bir yönteme geçiş yaparlar. Crossover noktasının kesin değeri yüksek ölçüde sisteme bağlıdır. Komut işlemeki ve gizlemeki etkileri göz ardı edip işlemleri sayan analizler $n = 8$ (Higham[167]) veya $n = 12$ (Huss-Lederman ve diğerleri[186]) kadar düşük crossover noktaları üretilmiştir. D'Alberto ve Nicolau[81], yazılım paketleri yüklendiğinde, karşılaştırma ile crossover noktalarını belirleyen uyumlu bir tasarımı geliştirmişlerdir. $n = 400$ den $n = 2150$ ye kadar değer alan çeşitli sistemler üzerinde crossover noktalarını bulmuşlar fakat birkaç sistem üzerinde crossover noktalarını bulamamışlardır.

Özyinelemeler 1202 lerin öncesine kadar, Fibonacci sayılarına adı verilen L. Fibonacci tarafından çalışılmıştır. A. De Moivre özyinelemeleri çözmek için doğuran fonksiyonlar yöntemini tanımlamıştır. Alıştırma 4-6.2 de doğrulanan genişletilmiş yöntemi sağlayan ana yöntem Bentley, Haken ve Saxe[44] tarafından uyarlanmıştır. Knuth[209] ve Liu[237] doğuran fonksiyonlar yöntemini kullanarak doğrusal özyinelemelerin nasıl çözüleceğini göstermişlerdir. Purdom ve Brown[287] ve Graham, Knuth ve Patashnik[152] özyineleme çözümlerinin genişletilmiş irdemesini içerirler.

Birkaç araştırmacı, Akra ve Bazzi[13], Roura[299], Verma[346] ve Yap[360], ana yöntem tarafından çözümlenen daha genel böl-ve-yönet yöntemlerinin çözümü için yöntemler vermişlerdir. Burada Akra ve Bazzi'nin, Leighton[228] tarafından uyarlanmış sonucunu tanıtacağız. Akra-Bazzi yöntemi

$$T(x) = \begin{cases} \Theta(1) & 1 \leq x \leq x_0 \text{ ise,} \\ \sum_{i=1}^k a_i T(b_i x) + f(x) & x > x_0 \text{ ise} \end{cases} \quad (4.30)$$

formundaki özyinelemeler için çalışır, burada

- $x \geq 1$ bir reel sayıdır,
- $i = 1, 2, \dots, k$ için $x_0 \geq 1/b_i$ ve $x_0 \geq 1/(1 - b_i)$ olacak şekilde x_0 bir sabittir,
- $i = 1, 2, \dots, k$ için a_i bir pozitif sabittir,

- $i = 1, 2, \dots, k$ için b_i $0 < b_i < 1$ aralığındadır,
- $k \geq 1$ tam sayıdır ve
- **polinomial-büyüme koşulunu** sağlayan negatif olmayan bir fonksiyondur: her $x \geq 1$, $i = 1, 2, \dots, k$ ve $b_i \leq u \leq x$ koşuluna uyan her u için $c_1 f(x) \leq f(u) \leq c_2 f(x)$ olacak şekilde pozitif c_1 ve c_2 sabitleri vardır. (Eğer x deki bazı polinomlar tarafından $|f'(x)|$ üstten sınırlıysa, o zaman $f(x)$ polinomial-büyüme şartını sağlar. Örneğin herhangi α ve β reel sabitleri için, $f(x) = x^\alpha \lg^\beta x$ bu koşulu sağlar.)

Ana yöntem $T(\lfloor n/3 \rfloor) + T(\lfloor 2n/3 \rfloor) + O(n)$ gibi bir özyineleme için uygulanmamasına rağmen, Akra-Bazzi yöntemi uygulanır. (4.30) daki özyinelemeyi çözmek için, ilk olarak $\sum_{i=1}^k a_i b_i^p = 1$ olacak şekilde tek p reel sayısını buluruz (Böyle bir p her zaman vardır). O halde özyinelemenin çözümü,

$$T(n) = \Theta \left(x^p \left(1 + \int_1^x \frac{f(u)}{u^{p+1}} du \right) \right)$$

dir.

Akra-Bazzi yöntemi kullanım için kısmen zor olabilir ancak problemin bölümünü büyük oranda eşit olmayan altproblemlere modelleyen özyinelemeleri çözmeyi sağlar. Ana yöntem kullanım için daha basittir fakat sadece altproblemlerin boyutlarının eşit olduğu durumlarda uygulanır.

Bu bölüm olasılıksal analiz ve rassal algoritmaları tanıtır. Eğer olasılık teorisinin temellerine aşına değilseniz konuyu özetleyen Ek C yi okumalısınız. Bu kitap boyunca olasılıksal analiz ve rassal algoritmaları birkaç kez gözden geçireceğiz.

5.1 Kiralama problemi

Varsayalım ki, yeni bir ofis asistanı işe almaya ihtiyaç duyuyorsunuz. Daha önceki işe alım girişimlerinüz başarısız oldu ve iş ve işçi bulma kurumunu kullanmaya karar verdiniz. Kurum size her gün bir aday gönderiyor. O kişiyle görüşüp işe alıp almamaya karar veriyorsunuz. İş ve işçi bulma kurumuna başvuranlarla görüşmek için küçük bir ücret ödemelisiniz. Aslında başvuru sahibini işe almak daha masraflı çünkü şu andaki ofis asistanını işten çıkarmalı ve iş ve işçi bulma kurumuna önemli bir miktar ödemelisiniz. İş için en iyi kişiyi bulmaya önem veriyorsunuz. Bu yüzden her bir başvuru sahibi ile görüştükten sonra eğer başvuru sahibi şu anda çalışan ofis asistanından daha nitelikli ise şu anda çalışanı işten çıkarıp yerine başvurayı almaya karar verdiniz. Bu stratejiden doğan masrafları ödemeye razı olabilirsiniz ancak bu fiyatı da tahmin etmek istersiniz.

Aşağıda verilen HIRE-ASSISTANT prosedürü kabakod içinde bu işe alım stratejisini ifade eder. Ofis asistanlığı için adayların 1 den n e kadar numaralandırıldığı kabul edilir. Prosedür, i . adayla görüştükten sonra onun şimdiye kadar görüştüğünüz en iyi aday olup olmadığına karar verebildiğinizi kabul eder. Başlangıç duruma getirmek için prosedür 0 la numaralandırılan yapay bir aday tasarlar ve bu aday tüm adaylardan daha az niteliklidir.

HIRE-ASSISTANT (n)

1. $eniye = 0$ // 0 nolu aday en düşük yetenekli kukla adaydır.
2. **for** $i = 1$ **to** n
3. i adayını mülakat et
4. **if** i nolu aday $eniye$ adaydan daha iyidir
5. $eniye = i$
6. i nolu aday işe alınacak

Bu problemin fiyatlandırma modeli Bölüm 2 de anlatılan modelden farklıdır. HIRE-ASSISTANT'ın çalışma süresine değil mülakatlar ve işe alım masraflarına odaklanılır. Görünüşte bu algoritmaların masrafı çalışma sürelerini analiz etmekten farklıdır. Masrafları veya çalışma sürelerini analiz etmek kullanılan analitik teknikler aynıdır. Her durumda, temel işlemlerin kaç kez çalıştırıldığını sayıyoruz.

Görüşmenin düşük bir maliyeti vardır buna c_i diyelim. Oysa işe alım pahalıdır bunu da c_h olarak maliyetlendirelim. m işe alınan kişi sayısı olursa tüm maliyet $O(c_i n + c_h m)$ algoritmasıyla ilişkilendirilecektir. İşe aldığımız kişi kaç olursa olsun, her zaman n adayla görüşme yapılır ve böylece $c_i n$ görüşmeyle ilişkilendirilen maliyet olur. Bu yüzden işe alım maliyeti olan $c_h m$ i analiz etmeye odaklanıyoruz. Bu nicelik algoritmanın her çalışmasında değişir.

Bu senaryo, genel bir hesaplama örneğine model sunar. Sıklıkla dizideki her elemanı inceleyerek ve mevcut bir “galip” bularak dizideki maksimum ve minimum değeri bulmaya ihtiyaç duyuyoruz. Kiralama problemi kazanan elemanı ne kadar sıklıkta güncellediğimizi modeller.

En kötü-durum analizi

En kötü-durumda, görüşülen her adayı işe alırız. Bu durum, eğer adaylar niteliksel sıralama olarak kesinlikle artarak geliyorsa gerçekleşir. Bu durumda n kez işe alırız ve toplam işe alım maliyeti $O(c_h n)$ olur.

Tabi ki adaylar her zaman niteliksel sıralama olarak artarak gelmez. Aslında geldiklerindeki sıralamaları konusunda hiçbir fikrimiz olmaz ve bu sıralamayı kontrol edemeyiz. Bu yüzden tipik ve ortalama bir durumda ne olmasını beklediğimizi sormak doğaldır.

Olasılıksal analiz

Olasılıksal analiz, olasılığın problem analizlerinde kullanılmasıdır. Genelde olasılık analizlerini algoritmaların çalışma sürelerini analiz etmek için kullanırız. Bazen ise işe alım sürecindeki maliyet gibi diğer nicelikleri analiz etmek için

kullanırız. Olasılıksal analizi gerçekleştirmek için girdilerin dağılımı bilgilerini kullanmalı veya varsayımlar yapmalıyız. Sonrasında algoritmayı analiz edip olası girdi dağılımının ortalamasını alarak ortalama durum çalışma süresini hesaplarız. Böylece tüm olası girdilerin çalışma sürelerinin ortalamasını alırız. Bu gibi çalışma sürelerini raporlarken, *ortalama-durum çalışma süresi* olarak yönlendirilir.

Girdilerin dağılımını tespit ederken çok dikkatli olmalıyız. Bazı problemler için olası girdilerin düzenlenmesi ile ilgili kabuller yapabilir daha sonra olasılıksal analizi etkili bir algoritma tasarlamak ve problemin içeriğini elde etmek için kullanabiliriz. Diğer problemler için uygun bir girdi dağılımı tanımlayamayız ve bu durumda olasılıksal analiz kullanamayız.

Kiralama problemi için, adayların rassal bir sıralamada geldiğini kabul edebiliriz. Bu problem için bunun anlamı nedir? Varsayalım ki herhangi iki aday karşılaştırıp hangisinin daha nitelikli olduğuna karar veriyoruz. Adaylar için tam bir sıralama vardır. (Tam sıralamanın tanımı için Ek B ye bakınız) Böylece aday için sırasını $rank(i)$ gösterip daha yüksek sıralamalar daha nitelikli adayları temsil edecek şekilde her aday 1 den n e kadar tek bir sayıyla numaralandırılır. Sıralama listesi $\langle rank(1), rank(2) \dots rank(n) \rangle$, $\langle 1, 2, \dots n \rangle$ listesinin permütasyonudur. Başvuranların rassal bir sıralamada gelmesi demek sıralama listesi $n!$ 'den herhangi birine eşit demektir. Bunun yerine *düzgün rassal permütasyon* aralık düzeni için; herbir mümkün permütasyon eşit olasılıkla görülür diyebiliriz.

Kısım 5.2 kiralama probleminin olasılıksal analizini içerir.

Rassal algoritmalar

Olasılıksal analizi kullanmak için, girdilerin dağılımı ile ilgili bilmemiz gereken bir şeyler var. Bir çok durumda, girdi dağılımı ile ilgili çok az şey biliriz. Eğer dağılım ile ilgili bir şeyler biliyorsak, bu bilgiyi algoritmanın bir kısmını rastgele yaparak modelleyebiliriz fakat olasılık ve rastgeleliği algoritma tasarımı ve analizi için bir araç olarak kullanabiliriz.

Kiralama probleminde adaylar bize rastgele bir sırada sunulmuş gibi görünebilir ancak adayların ne olduğu ya da olmadığı konusunda bilgili olmanın yolu yoktur. Bu yüzden işe alım problemi için bir algoritma geliştirmek için adaylarla görüşme sırası kontrol altında olmalıdır. O halde modeli biraz değiştireceğiz. Diyelim ki iş bulma ajansı n adaya sahip olsun ve görüşme için bir aday seçelim. Adaylar hakkında hiçbir şey bilmememize rağmen (isimleri dışında) önemli bir

değişiklik yaptık. Görüşmeye gelen adayların rassal sıralamasına güvenmek yerine rassal sıralamayı ve süreci kontrol altına aldık.

Genel olarak bir algoritmanın davranışı sadece girdilerle değil aynı zamanda **rassal** sayı üretici ile üretilen değerlerle karar veriliyorsa bu algoritmaya rassal denir. Şu anda elimizde **rassal bir sayı üreten** RANDOM fonksiyonumuzun olduğunu varsayalım. $\text{RANDOM}(a, b)$ fonksiyonu bir kez çağırıldığında a ve b arasında, a ve dahil olmak üzere bir tam sayı döndürür ve bu aralıktaki tüm sayıların çıkma olasılığı eşittir. Örneğin; $\text{RANDOM}(0,1)$ 1/2 olasılıkla 0 üretirken 1/2 olasılıkla 1 üretir. $\text{RANDOM}(3,7)$ 1/5 olasılıkla 3,4,5,6,7 sayılarını verir. RANDOM tarafından verilen her sayı daha önceki çağrılardan bağımsızdır. Çıktısını elde etmek için yuvarlanan $(b - a + 1)$ kenarlı bir zar olarak düşünebilirsiniz. (Pratikte bir çok programlama ortamının önerdiği **sözde rassal-sayı üretici**: istatistiksel olarak rastgele ‘görülen’ numaraları geri döndüren belirleyici algoritma)

Rassal bir algoritmanın çalışma sürelerini analiz ederken Rassal numara üretici tarafından döndürülen değerlerin çalışma sürelerinin beklentilerini oluştururuz. Bir Rassal algoritmanın çalışma süresine bakılarak **beklenen çalışma süresi** olarak girdinin rastgele olduğu algoritmalarla diğerlerinin farkına varılır.Genel olarak olasılık dağılımının algoritmanın girdisi üzerinde olduğu zaman için ortalama durum çalışma süresini ele aldık ve algoritmanın kendi Rassal seçimlerini yaptığı zaman için beklenen çalışma süresini ele aldık.

Alıştırmalar

5.1-1

Hangi adayın en iyi olduğunu belirleyebildiğimiz HIRE-ASSISTANT sürecinin 4.satırında adayların tam sıralı olması gerektiğini gösteriniz.

5.1-2 ★

$\text{RANDOM}(a,b)$ prosedürünün $\text{RANDOM}(0,1)$ e çağrı yapan bir uygulamasını tanımlayınız. Prosedürünüzün beklenen çalışma süresi, a ve b nin fonksiyonu olarak nedir?

5.1-3 ★

Çıktımızın 1/2 olasılıkla 0 ve 1/2 olasılıkla 1 olduğunu varsayalım. Çıktısı 0 ya da 1 olan BIASED-RANDOM fonksiyonu kullanılabilir. $0 < p < 1$ aralığında $1 - p$ olasılıkla 0 ve bazı p olasılıkla 1 çıkartır ancak p nin ne olduğunu bilmiyorsunuz. BIASED RANDOM u alt yöntem gibi kullanan bir algoritma yazınız ve tarafsız bir

yanıtlarla dönsün, $1/2$ olasılıkla 0 ve $1/2$ olasılıkla 1 e dönsün. Algoritmanızdaki p fonksiyonunun tahmini çalışma süresi nedir?

5.2 Rassal gösterge değişkenleri

Kiralama problemlerini de içeren bir çok algoritmayı analiz etmek için rassal gösterge değişkenleri kullanırız. Rassal gösterge değişkenleri olasılık ve beklentileri birbirine çeviren bir yöntemdir. Varsayalım ki S bir örnek uzayı ve A olayı verildi. Öyleyse **rassal gösterge değişkeni** $I\{A\}$

$$I\{A\} = \begin{cases} 1 & A \text{ gerçekleşirse,} \\ 0 & A \text{ gerçekleşmezse.} \end{cases} \quad (5.1)$$

A olayı ile tanımlanan şekilde ilişkilendirilir. Basit bir örnek olarak, bir metal paranın atımıyla elde edilen turanın beklenen değerini belirleyelim. Örnek uzayımız $S = \{H, T\}$ olsun. $\Pr\{H\} = \Pr\{T\} = 1/2$ olasılıkla rassal gösterge değişkeni X_H , H olay olacak şekilde paranın tura gelmesiyle ilişkilendirilerek tanımlanabilir. Bu değişken, tura gelirse 1, aksi halde 0 olacak şekilde kaç defa tura geldiğini sayar, şöyle yazılır:

$$\begin{aligned} X_H &= I\{H\} \\ &= \begin{cases} 1 & H \text{ gerçekleşirse,} \\ 0 & T \text{ gerçekleşmezse.} \end{cases} \end{aligned}$$

Paranın bir kez atılmasıyla elde edilen turanın beklenen sayısı gösterge değişkenimiz olan X_H nin beklenen değeridir

$$\begin{aligned} E[X_H] &= E[I\{H\}] \\ &= 1 \cdot \Pr\{H\} + 0 \cdot \Pr\{T\} \\ &= 1 \cdot (1/2) + 0 \cdot (1/2) \\ &= 1/2. \end{aligned}$$

Böylece bir atışta tura gelmesinin beklenen değeri $1/2$ olur. Takip eden lemmamızın gösterdiği gibi, A olayı ile ilişkilendirilen rassal gösterge değişkeninin beklenen değeri A nın gerçekleşme olasılığına eşittir.

Lemma 5.1

Verilen örnek S uzayını da ve S uzayındaki A olayında $X_A = I\{A\}$ olsun. Öyleyse $E[X_A] = \Pr\{A\}$ olur.

Kanıt (5.1) deki rassal gösterge değişkeninin tanımından ve beklenen değer tanımında

$$\begin{aligned} E[X_A] &= E[I\{A\}] \\ &= 1 \cdot \Pr\{A\} + 0 \cdot \Pr\{\bar{A}\} \\ &= \Pr\{A\} , \end{aligned}$$

$\bar{A}$, $S-A$ yı yani A nın tümleyenini gösterir. ■

Rassal gösterge değişkenleri bazı uygulamalar için örneğin; bir metal paranın tura gelme sayısını gösterirken kullanışsız gibi görünse de rastgele denemelerin yapıldığı durumları analiz etmek için faydalıdır. Örneğin, rassal gösterge değişkenleri (C.37) denklemi için basit bir yol sunar. Bu denklemde n kez atılan paranın tura gelme sayısını ayrı ayrı olasılıklarını dikkate alarak hesaplarız. C38 denkleminde sunulan daha basit yöntem ise rassal gösterge değişkenleri kapalı şekilde kullanır. Bu argümanı daha açık hala getirmek için X_i . Kez tura gelmesi olayıyla ilişkilendirilerek rassal gösterge değişkeni alınır. $X_i = I\{H \text{ olayında } i \text{ atış sonucu}\}$ X n kez para atılışı sonucunda oluşan toplam tura sayısını gösterebilir, öyleyse;

$$X = \sum_{i=1}^n X_i$$

dir. Turanın beklenen sayısını hesaplamak istiyoruz, öyleyse bunu elde etmek için aşağıdaki denklemin her iki tarafının beklentisini de ele almalıyız.

$$E[X] = E\left[\sum_{i=1}^n X_i\right] .$$

Yukarıda ki denklem n tane rassal gösterge değişkeninin beklenen toplam değerini verir. Lemma 5.1 ile her rassal değişkenin beklentisinin doğrusallığı ile toplamın beklentisini hesaplamak kolaydır: bu toplam n tane rassal değişkenin beklentilerinin toplamına eşittir. Beklentinin doğrusallığı rassal gösterge değişkenlerinin kullanımını güçlü ve analitik teknik yapar; bu teknik rassal değişkenler arasında bağımlılık olsa bile uygulanır. O halde beklenen değerini hesaplayabiliriz:

$$\begin{aligned}
E[X] &= E\left[\sum_{i=1}^n X_i\right] \\
&= \sum_{i=1}^n E[X_i] \\
&= \sum_{i=1}^n 1/2 \\
&= n/2.
\end{aligned}$$

Böylece (C.37) denkleminde uygulanan yöntemle karşılaştırıldığında, rassal gösterge değişkenleri hesaplamayı hayli basitleştirdi. Rassal gösterge değişkenlerini bu kitap boyunca kullanacağız.

Rassal gösterge değişkenleri kullanılarak kiralama problemlerinin çözülmesi

Kiralama problemine geri dönersek, yeni bir ofis asistanı alacağımız zaman ki beklenen değeri hesaplamak istiyoruz. Olasılıksal analizi kullanmak için, önceki bölümde olduğu gibi adayların rastgele geldiğini kabul ediyoruz. (Kısım 5.3’de bu varsayımın nasıl kaldırılacağını göreceğiz.) X değeri işe alım sayısına eşit olan rassal bir değişken olsun. Böylece

$$E[X] = \sum_{x=1}^n x \Pr\{X = x\}$$

yı elde etmek için C.20 de beklenen değer tanımı uygulanabilir fakat bu hesaplama kullanışsız olabilir. Bunun yerine hesaplamayı basitleştirmek için rassal gösterge değişkenlerini kullanacağız.

Rassal gösterge değişkenlerini kullanmak için işe alım sayısı ile ilişkilendirilmiş tek değişkenle tanımlanmış $E[X]$ i hesaplamak yerine her adayın işe alınıp alınmadığıyla ilgili olan n tane değişken tanımlıyoruz. Özellikle, i . adayın alınma olasılığıyla ilişkilendirilen X_i bir rassal gösterge değişkeni olsun.

Böylelikle

$$\begin{aligned}
X_i &= I \{ \text{aday } i \text{ işe alınır} \} \\
&= \begin{cases} 1 & \text{aday } i \text{ işe alınırsa,} \\ 0 & \text{aday } i \text{ işe alınmazsa,} \end{cases}
\end{aligned}$$

ve

$$X = X_1 + X_2 + \cdots + X_n \tag{5.2}$$

dir. Lemma 5.1 den

$$E[X_i] = \Pr\{\text{aday } i \text{ işe alındı}\}$$

olduğunu biliyoruz ve bu yüzden HİRE-ASSISTANT ın 5 ve 6. satırlarının çalıştırılma olasılığını hesaplamalıyız.

6. satırda aday i nin aday 1 den $i - 1$ e kadar olan adaylardan daha iyi olduğu durumu için aday i işe alındı. Çünkü adayların rassal bir sıralamayla geldiğini kabul ettik, ilk i aday rastgele bir sıra ile görüldü. Bu i adaydan herhangi biri şimdiye kadar ki en iyisidir. Aday i nin aday 1 den $i - 2$ ye kadar olan adaylardan daha donanımlı olma olasılığı $1/i$ dir. Böylelikle işe alınma olasılığı da $1/i$ dir. Lemma 5.1 ile

$$E[X_i] = 1/i . \quad (5.3)$$

sonucuna ulaşabiliriz Şimdi $E[X]$ i hesaplayabiliriz.

$$E[X] = E\left[\sum_{i=1}^n X_i\right] \quad ((5.2) \text{ denkleminde}) \quad (5.4)$$

$$= \sum_{i=1}^n E[X_i] \quad (\text{beklentinin doğrusallığından})$$

$$= \sum_{i=1}^n 1/i \quad ((5.3) \text{ denkleminde})$$

$$= \ln n + O(1) \quad ((A.7) \text{ denkleminde}) \quad (5.5)$$

n kişiyle görüşmemize rağmen aslında yaklaşık olarak onların $\ln n$ tanesini işe alırız. Bu sonucu bir sonraki lemmada özetliyoruz.

Lemma 5.2

Adayların rastgele bir sırada oluşturulduğunu kabul ederek, HİRE-ASSISTANT algoritmasının $O(c_h \ln n)$ ortalama durum toplam işe alım maliyeti vardır.

Kanıt Tahmini işe alınan sayısını gösteren (5.5) denklemi ve işe alım ücreti tanımımızdan yaklaşık $\ln n$ dir. ■

Ortalama-durum işe alım maliyeti en kötü-durum işe alım maliyeti olan $O(c_h n)$ üzerine önemli bir gelişmedir.

Alıştırmalar

5.2-1

HIRE ASSISTANT da, adayların rassal sırada oluşturulduğu kabul edilirse 1 kerede işe alma olasılığı nedir? n defa işe alma olasılığı nedir?

5.2-2

HIRE-ASSISTANT da adayların rastgele sırada oluşturulduğu kabul edilirse 2 kere işe alma olasılığı nedir?

5.2-3

n kez zar atımının beklenen değerini hesaplamak için rassal gösterge değişkenlerini kullanırız.

5.2-4

Şapka-kontrol problemi olarak bilinen sıradaki problemi çözmek için rassal gösterge değişkenlerini kullanırız. n müşteriden her biri restorandaki vestiyere bir şapka verir. Vestiyer ise şapkaları müşterilere rassal bir sıralamada verdiği göre kendi şapkasını geri alan müşteri sayısının beklenen değeri nedir?

5.2-5

$A[1, \dots, n]$ n farklı sayının bir sayı dizisi olsun. Eğer $i < j$ ve $A[i] > A[j]$ ise (i, j) çifti A nın tersi olarak adlandırılır. (Tersinir olması üzerine daha fazlası için Problem 2-4 ü inceleyiniz.) Varsayalım ki A nın elemanları $\langle 1, 2, \dots, n \rangle$ ın düzgün rassal permütasyonu olsun. Tersin beklenti sayısını hesaplamak için rassal gösterge değişkenlerini kullanınız.

5.3 Rassal algoritmalar

Önceki bölümlerde girdilerin dağılımını bilmenin algoritmanın ortalama-durum analizini yapmak için bize nasıl yardım ettiğini gösterdik. Bir çok durumda böyle bir bilginiz yoktur, böylelikle ortalama durum analizi önlenir. 5.1 de bahsedildiği gibi rassal algoritmaları kullanabiliriz.

Kiralama problemi gibi bir problem için girdinin tüm permütasyonlarının eşit olduğunu kabul etmek faydalıdır. Olasılıksal analiz rassal bir algoritmanın gelişimine rehberlik edebilir. Girdilerin dağılımını varsaymak yerine bir dağılım sunarız. Özellikle algoritmayı çalıştırmadan önce her permütasyonunun eşit olacağı özelliği göstermek için adayların rastgele sırasını değiştiririz. Algoritmada değişiklik yapmamıza rağmen yaklaşık $\ln n$ kez işe alım yapmayı bekliyoruz.

Fakat şimdi bu durumun belirli dağılımdaki girdiler için değil *herhangi* bir girdi için olmasını bekliyoruz.

Şimdi olasılıksal analiz ile rassal algoritmalar arasındaki farklılıkları keşfedelim. 5.2 Kısımında, adayların rassal bir sıralamada geldiğini varsayarak, yeni bir ofis asistanı almanın beklenti sayısının yaklaşık $\ln n$ olduğunu söyledik. Buradaki algoritmanın deterministik olduğuna dikkat edelim; bunun herhangi belirli bir girdi için yeni bir ofis asistanının alınma sayısı her zaman aynıdır. Ayrıca yeni bir ofis asistanı işe alınma sayısı farklı girdiler için değişkenlik gösterir ve bu değişik adayların sırasına bağlıdır. Bu sayı sadece değişik adayların sıralamasına bağlı olduğundan belirli bir girdi listesi yapılabilir. Adayların sırası $\langle rank(1), rank(2), \dots, rank(n) \rangle$ olacak şekilde $A_1 = \langle 1, 2, \dots, 10 \rangle$ sıralama listesi verilirse 10 kez işe alım yapılır. Her başarılı aday öncekinden daha iyi olduğu için 5-6 satırları her ötelemede çalıştırılır. $A_2 = \langle 10, 9, 8, 7, 6, 5, 4, 3, 2, 1 \rangle$ sıralama listesi verilirse yeni bir ofis asistanı birinci ötelemede ve bir kerede işe alınır. $A_3 = \langle 5, 2, 1, 8, 4, 7, 10, 9, 3, 6 \rangle$ sıralama listesi verildiğinde sıra 5, 8 ve 10 daki adaylarla görüşülerek 3 kez alım yapılır. Algoritmanın maliyeti kaç kez alım yapıldığına bağlı olduğu hatırlatılarak, A_1 gibi pahalı girdilerin A_2 gibi ucuz girdilerin ve A_3 gibi makul girdilerin var olduğunu görüyoruz.

Öte yandan önce adayların sıralamasını yapan ardından en iyi adaya karar veren rassal algoritmayı düşünelim. Bu durumda, girdi dağılımında değil algoritmanın içinde rastgeleleştirmeyi yapıyoruz. Verilen bir A_3 üzerinde algoritmayı ilk kez çalıştırdığımızda 10 güncelleme performansı yaparak A_1 permütasyonunu üretir. Fakat ikinci kez çalıştırdığımızda, bir güncelleme performansı ile A_2 yi üretebiliriz. Üçüncü kez çalıştırdığımızda ise daha farklı güncelleme sayıları elde edebiliriz. Algoritmayı her çalıştırdığımızda uygulama rassal seçimlerin yapılmasına bağlı olarak değişir ve algoritmanın bir önceki uygulamasından farklıdır. Bu algoritma ve rassal bir çok başka algoritma için hiçbir kısmi girdi *en kötü davranış* ortaya çıkarmaz. Rassal permütasyon girdi sırasını ilişkili yapmadığı için en kötü düşmanın olmadıkça kötü bir girdi dizisi üretilmeyecektir. Eğer rassal numara üretici ‘şanssız’ permütasyon üretirse rassal algoritma kötü performans verir.

Kiralama problemi için koddaki gereken tek değişiklik dizinin rastgele sırasını değiştirmektir.

RANDOMIZED-HIRE-ASSISTANT (n)

```

1 adaylar listesini rastgele karıştır
2  $eniyi = 0$  // 0 nolu aday en az yetenekli kukla adaydır
3 for  $i = 1$  to  $n$ 
4    $i$  nolu adayla mülakat yap
5   if  $i$  nolu aday  $eniyi$  adaydan iyiyse
6      $eniyi = i$ 
7    $i$  nolu adayı işe al

```

Bu basit değişiklikle, adayların performansı rastgele sıralandığı kabul edilerek elde edilenle eşleşen bir rassal algoritma yaratmış olduk.

Lemma 5.3

RANDOMIZED-HIRE-ASSISTANT prosedürünün beklenen işe alım maliyeti $O(c_h \ln n)$ dir.

Kanıt Girdi dizisinin sıralamasını değiştirdikten sonra, HIRE-ASSISTANT ın olasılıksal analiziyle aynı davranışı gösteren bir durum elde ettik. ■

Lemma 5.2 ve 5.3 ün en önemli bölümleri karşılaştırıldığında farklılık olasılıksal analiz ve rassal algoritmalar. Lemma 5.2 de girdiyle ilgili bir varsayım yapılırken Lemma 5.3 de girdiyi rastgeleleştirmek ekstra zaman alsa da herhangi bir varsayım yapmıyoruz. Terminolojideki tutarsızlığı önlemek için Lemma 5.2 yi ortalama durum işe alım maliyeti açısından, Lemma 5.3 ü beklenen işe alım maliyeti açısından ifade ediyoruz. Bu bölümün kalan kısmında, rastgele sıralanan girdilerle ilgilenen bazı konuları ele aldık.

Rassal sıralaması değişen diziler

Birçok rassal algoritma verilen girdi dizisinin sırasının değiştirilmesiyle rastgeleleştirilir. (Rastgeleleştirilmeyi kullanmanın başka yolları da vardır) Burada iki yöntemi tartışacağız. Varsayalım ki, genellemeleri kaybetmeden 1 den n e kadar elemanları içeren A dizisi verilsin. Amacımız bir dizinin rassal permütasyonunu üretmektir.

Genel bir yöntem olarak, rastgele öncelikli $P[i]$ olan dizinin her elemanını $A[i]$ olarak atayalım ve daha sonra bu önceliklere göre A nın elemanlarını sınıflandıralım. Örneğin; eğer başlangıç dizisi $A = \langle 1, 2, 3, 4 \rangle$ olursa ve rassal öncelikleri $P = \langle 36, 3, 62, 19 \rangle$ olarak seçersek $B = \langle 2, 4, 1, 3 \rangle$ dizisini üretiriz. İkinci öncelik en küçüktür, sonrakiler ise sırasıyla dördüncü, birinci ve üçüncüdür. Bu prosedüre PERMUTE-BY-SORTING deriz:

PERMUTE-BY-SORTING (A)

```

1   $n = A.length$ 
2   $P[1..n]$  yeni bir dizi olsun
3  for  $i = 1$  to  $n$ 
4       $P[i] = \text{RANDOM}(1, n^3)$ 
5   $P$  sıralama anahtarlarını kullanarak  $A$  yı sırala

```

Satır 4, 1 ile n^3 arasında bir rassal sayı seçer. P nin önceliklerini yegane yapmak için 1 den n^3 e kadar olan sırayı kullanıyoruz. (Alıştırma 5.3-5 girişlerin yegane olduğu olasılığın en az $1 - 1/n$ olduğu durumu kanıtlamanızı istiyor ve alıştırma 5.3-6 iki yada daha fazla öncelik aynı olsa bile algoritmaların nasıl uygulandığını kanıtlamamızı istiyor.) Önceliklerin tümünün yegane olduğunu kabul edelim.

Zaman tüketme basamağı bu prosedürde Satır 5 deki sıralamadır. Bölüm 8 de göreceğimiz gibi karşılaştırma sıralamasını kullanırsak sıralama $\Omega(n \lg n)$ kadar zaman alır. Birleştirerek sıralama $\Theta(n \lg n)$ kadar zaman aldığı için, alttan sınırlamasını yapabiliriz. (Bölüm 2 de $\Theta(n \lg n)$ kadar zaman alan karşılaştırma sıralamasını da göreceğiz. Alıştırma 8.3-4 $O(n)$ zamanda 0 ile $n^3 - 1$ aralığında çok benzer numara sıralama problemi çözmenizi istiyor. Sıraladıktan sonra eğer $P[i]j$. en küçük öncelik ise $A[i]$ çıktının j nin pozisyonunda yer alır. Bu nedenle bir permütasyon elde ediyoruz. Bu prosedürün **düzgün rassal permütasyonu** kanıtlamasını engeller bu da prosedürün 1 den n e kadar olan sayıların her permütasyonu üretmesine denktir.

Lemma 5.4

PERMUTE-BY-SORTING prosedürü tüm önceliklerin ayrık olduğunu kabul ederek bir düzgün rassal permütasyon üretir.

Kanıt Her $A[i]$ elemanının i en küçük önceliği alan kısmi permütasyonunu göz önüne alarak başlıyoruz. Bu permütasyonun kesinlikle $1/n!$ olasılıkla meydana geleceğini göstereceğiz. $i = 1, 2, \dots, n$ için $A[i]$ i.de en küçük önceliği alan bir E_i olayı olsun. Öyleyse her i için E_i nin gerçekleşme olasılığı olan $\Pr\{E_1 \cap E_2 \cap E_3 \cap \dots \cap E_{n-1} \cap E_n\}$ yi hesaplamak istiyoruz.

Alıştırma C.2-5 i kullanarak bu olasılık

$$\Pr\{E_1\} \cdot \Pr\{E_2 \mid E_1\} \cdot \Pr\{E_3 \mid E_2 \cap E_1\} \cdot \Pr\{E_4 \mid E_3 \cap E_2 \cap E_1\} \\ \dots \Pr\{E_i \mid E_{i-1} \cap E_{i-2} \cap \dots \cap E_1\} \dots \Pr\{E_n \mid E_{n-1} \cap \dots \cap E_1\}$$

olur.

$\Pr\{E_1\} = 1/n$ olduğunu biliyoruz. Çünkü bu olasılık n kümesi dışında rassal seçilen bir öncelik ve en küçük önceliktir. Sonrasında $\Pr\{E_2 \mid E_1\} = 1/(n-1)$ olduğunu

elde ediyoruz çünkü verilen $A[1]$ elemanı en küçük önceliğe sahip ve geriye kalan $n - 1$ elemanın ikinci en küçük önceliğe sahip olma olasılığı eşittir. Genelde $i = 2, 3, \dots, n$ için $\Pr\{E_i \mid E_{i-1} \cap E_{i-2} \cap \dots \cap E_1\} = 1/(n - i + 1)$ çünkü $A[1]$ den $A[i - 1]$ e verilen elemanlar $i - 1$ en küçük önceliğe sahip (sıralı) geriye kalan tüm $n - (i - 1)$ elemanın i . en küçük önceliğe sahip olma şansı eşittir. Böylelikle

$$\begin{aligned} \Pr\{E_1 \cap E_2 \cap E_3 \cap \dots \cap E_{n-1} \cap E_n\} &= \left(\frac{1}{n}\right) \left(\frac{1}{n-1}\right) \dots \left(\frac{1}{2}\right) \left(\frac{1}{1}\right) \\ &= \frac{1}{n!} \end{aligned}$$

ve birim permütasyonu elde etme olasılığı $1/n!$ olur.

Bu kanıtı önceliklerin herhangi bir permütasyonu için çalışması adına genişletebiliriz. Herhangi bir ayarlanmış permütasyonu düşünelim. $\sigma = \langle \sigma(1), \sigma(2), \dots, \sigma(n) \rangle$ $\{1, 2, \dots, n\}$ setinin $A[i]$ elemanına atanmış bir r_i öncelik sırası olsun. $A[i]$ $\sigma(i)$ veya $r_i = \sigma(i)$ en küçük öncelik olacak şekilde E_i olayı tanımlarsak, aynı kanıt uygulanır. Bu yüzden herhangi bir permütasyonu elde etme olasılığını hesaplırsak, hesaplama üstteki ile aynı olur. Böylece bu permütasyonu elde etme olasılığı da $1/n!$ olur. ■

Bu permütasyonun düzgün rassal permütasyon olduğunu kanıtlamak için her $A[i]$ elemanın j ye kurulu pozisyonda olasılığı $1/n$ olduğunu düşünebilirsiniz. Alıştırma 5.3-4 bu zayıf koşulun gerçekte yeterli olmadığını gösterir.

Bu rassal permütasyonu genellemek için daha iyi bir yol ise verilen dizinin sırasını değiştirtirmektir. RANDOMIZE-IN-PLACE prosedürü bunu $O(n)$ kez yapar. i . iterasyonda $A[i]$ den $A[n]$ e kadar olan elemanlar arasında $A[i]$ elemanını rastgele seçer. i . iterasyonu takip ederek, $A[i]$ değiştirilemez.

RANDOMIZE-IN-PLACE(A)

```

1   $n = A.length$ 
2  for  $i = 1$  to  $n$ 
3       $A[i]$  ile  $A[RANDOM(i, n)]$  yerlerini değiştir.
```

RANDOMIZE-IN-PLACE prosedürünün bir düzgün rassal permütasyon ürettiğini göstermek için döngü kullanacağız. n elemanlı bir küme üzerinde bir ***k-permütasyonu*** k yı içeren bir dizidir ve tekrarsızdır. (Ek C ye bakınız). $n!/(n - k)!$ olası k permütasyonu vardır.

Lemma 5.5

RANDOMIZE-IN-PLACE prosedürü bir düzgün rassal permütasyon hesaplar.

Kanıt Takip eden döngüyü kullanacağız.

Satır 2-3 deki **for** döngüsünün i . iterasyonundan önce, n elemanın her $(i - 1)$ olası permütasyonunu için altdizi olan $A[1 \dots i-1]$ $(n - i + 1)/n!$ olasılıkla bu $(i - 1)$ permütasyonunu içerir.

Bu değişmezinin ilk iterasyondan önce doğru olduğunu, döngünün her bir iterasyonunun değişmezi koruduğunu ve döngü sonlandığında değişmezin doğruluğu göstermek için kullanışlı bir özelliği sağladığını göstermemiz gerekir.

Başlangıç: Birinci döngü iterasyonundan önce $i = 1$ durumunu düşünelim. Döngü değişmezi her olası 0-permütasyonu için altdizi $A[1..0]$ $(n - i + 1)/n!$ olasılıkla bu 0-permütasyonunun içerildiğini söyler. Altdizi $A[1..0]$ boş bir altdizidir ve bir 0-permütasyonun elemanı yoktur. Bu yüzden $A[1..0]$, 1 olasılıkla herhangi 0-permütasyonu içerir ve döngü değişmezi ilk iterasyondan önce korunur..

Yineleme: i . iterasyondan önce her olası $(i - 1)$ permütasyonunun $(n - i + 1)/n!$ olasılıkla altdizi $A[1 \dots i - 1]$ de görüneceğini varsayalım ve i . iterasyondan sonra her olası i . permütasyonunun $(n - i)/n!$ olasılıkla $A[1 \dots i]$ altdizisinde yer alacağını göstereceğiz. i yi bir sonraki iterasyon için arttırınca döngü değişmezi korunur.

i . iterasyonu inceleyelim. Elemanları $\langle x_1, x_2, \dots, x_i \rangle$ ile gösterilen kısmi bir i -permütasyonunu düşünelim. Bu permütasyon $(i - 1)$ permütasyondan $\langle x_1, \dots, x_{i-1} \rangle$ oluşur. Arkasınca i . yere $A[i]$ değeri konur. E_1 olayı olarak ilk $i - 1$ adımda $A[1 \dots i - 1]$ değerlerini oluşturan $\langle 1, \dots, i - 1 \rangle$ permutasyonunu işaretleyelim. Döngü değişmezine göre $\Pr\{E_1\} = (n - 1)/n!$. i . iterasyonda $A[i]$ değeri olarak x_i değerinin alınması E_2 olayı olsun. i değerli $\langle x_1, \dots, x_i \rangle$ permutasyonu kesin olarak E_1 ve E_2 olaylarının gerçekleşmesini gerektiriyor. Böylece biz $\Pr\{E_1 \cap E_2\}$ olasılığını hesaplamalıyız. C.14 eşitliğini kullanarak aşağıdakini alırsız:

$$\Pr\{E_2 \cap E_1\} = \Pr\{E_2 \mid E_1\} \Pr\{E_1\}$$

$\Pr\{E_2 \mid E_1\}$ olasılığı $1/(n - i + 1)$ e eşittir çünkü satır 3 de $A[1 \dots n]$ pozisyonunda $n - i + 1$ değerinden x_i rastgele seçilir. Böylece

$$\begin{aligned}
\Pr\{E_2 \cap E_1\} &= \Pr\{E_2 \mid E_1\} \Pr\{E_1\} \\
&= \frac{1}{n-i+1} \cdot \frac{(n-i+1)!}{n!} \\
&= \frac{(n-i)!}{n!}
\end{aligned}$$

olur.

Sonlandırma: Sonlandırmada, $i = n + 1$ ve $A[1 \dots n]$ alt dizisine $(n - (n + 1) + 1)/n! = 0!/n! = 1/n!$ olasılıkla verilen n -permütasyonu vardır.

Böylelikle RANDOMIZE-IN-PLACE düzgün rassal permütasyon üretir. ■

Bir rassal algoritma genellikle bir problemi çözmek için en kolay ve en etkili yoldur. Bu kitap boyunca rassal algoritma da kullanacağız.

Alıştırmalar

5.3-1

Professor Marceau Lemma 5.5 deki kanıtta kullanılan döngü değişmezine karşı çıkar. Birinci iterasyon öncesinde doğru olup olmadığı sorgular. Birinci iterasyon öncesinde doğru olup olmadığını sorgular. Kolayca boş bir altdizinin 0-permütasyonunu içermediğini ifade eder. Bu yüzden 0-permütasyonu içeren bir boş altdizinin olasılığı 0 dir. Böylece ilk iterasyon öncesinde sabit döngü geçersiz sayılır. RANDOMIZE-IN-PLACE prosedürünü yeniden yazarak ilişkilendirildiği döngü değişmezini ilk iterasyon öncesinde boş olmayan altdiziye uygulayın ve Lemma 5.5 in kanıtını kendi prosedürünüze göre değiştirin.

5.3-2

Professör Kelp özdeş permütasyondan başka herhangi bir permütasyon üreten bir prosedür yazmaya karar verir. Aşağıdaki prosedürü sunar.

PERMUTE-WITHOUT-IDENTITY (A)

```

1   $n = A.length$ 
2  for  $i = 1$  to  $n - 1$ 
3       $A[i]$  ile  $A[\text{RANDOM}(i + 1, n)]$  yi yer değiştir
```

Bu kod Professör Kelp'in istediği kod mudur?

5.3-3

Diyelim ki $A[i]$ elemanını dizideki rassal bir elemanla değiştirmek yerine dizideki herhangi bir yerde olan rassal bir elemanla değiştirdik.

PERMUTE-WITH-ALL (A)

```

1   $n = A.length$ 
2  for  $i = 1$  to  $n$ 
3       $A[i]$  ile  $A[\text{RANDOM}(1, n)]$  yi yer değiştir

```

Bu kod bir düzgün rassal permütasyon üretir mi? Neden?

5.3-4

Professör Armstrong bir düzgün rassal permütasyonu genellemek için sıradaki prosedürü öneriyor.

PERMUTE-BY-CYCLIC (A)

```

1   $n = A.length$ 
2   $B[1..n]$  yeni bir dizi olsun
3   $offset = \text{RANDOM}(1, n)$ 
4  for  $i = 1$  to  $n$ 
5       $dest = i + offset$ 
6      if  $dest > n$ 
7           $dest = dest - n$ 
8       $B[dest] = A[i]$ 
9  return  $B$ 

```

Her $A[i]$ elemanın herhangi kısmi B pozisyonunda $1/n$ olasılığa sahip olduğunu gösteriniz. Sonrasında Professör Armstrong'un sonuç permütasyonunun düzgün rassal olmadığını göstererek yanlış yaptığını gösteriniz.

5.3-5 ★

PERMUTE-BY-SORTING prosedüründeki P dizisinde her elemanın olasılığının yegane ve en az $1 - 1/n$ olduğunu kanıtlayınız.

5.3-6

İki veya daha çok önceliğin özdeş olduğu durumların üstesinden gelmek için PERMUTE-BY-SORTING algoritmasının nasıl uygulanacağını açıklayınız. Algoritmamız iki veya daha çok önceliğin özdeş olması durumunda da düzgün rassal permütasyon üretmelidir.

5.3-7

Diyelim ki $\{1, 2, 3, \dots, n\}$ kümesinden bir **rassal örnek** yaratmak istiyoruz. Yani, $0 \leq m \leq n$ için m -elemanlı S altkümesini alalım öyle ki her bir m -altkümesi eşit bir şekilde oluşturabilsin. $i = 1, 2, 3, \dots, n$ için $A[i] = i$ olarak ayarlanması bir yoldur ve RANDOMIZE-IN-PLACE(A) denir. Sonrasında sadece ilk m dizi elemanını alalım. Bu yöntem RANDOM prosedürünü n kez çağırır. Eğer n, m den yeteri kadar

büyükse daha az RANDOM çağrıyla gelişi güzel bir örnek tasarlanabilir. Sıradaki özyinelemeli prosedürün $\{1, 2, 3, \dots, n\}$ den rastgele m -altkümeye S döndürdüğünü gösterin.

```

RANDOM-SAMPLE( $m, n$ )
1  if  $m == 0$ 
2      return  $\emptyset$ 
3  else  $S = \text{RANDOM-SAMPLE}(m - 1, n - 1)$ 
4       $i = \text{RANDOM}(1, n)$ 
5      if  $i \in S$ 
6           $S = S \cup \{n\}$ 
7      else  $S = S \cup \{i\}$ 
8      return  $S$ 

```

* 5.4 Olasılıksal analiz ve rassal gösterge değişkenlerinin ek kullanımları

Bu ileri düzeydeki kısım, dört örnekle birlikte olasılıksal analizi örnekler. Birincisi bir odada ikisi aynı gün doğmuş olan k kişinin olasılığını belirler. İkinci örnek, tesadüfi olarak kutulara toplar atıldığında ne olacağını inceler. Üçüncüsü, metal parayı fırlattığımızda ardışık turaların sayısını sorgular. Son örnek ise tüm adaylarla görüşme yapmadan karar vermek zorunda olunan kiralama probleminin değişkenini analiz eder.

5.4.1 Doğumgünü paradoksu

Birinci örneğimiz **doğumgünü paradoksu**. %50 şans olmadan önce yılın aynı gününde doğmuş olan iki kişinin aynı odada olması için gereken kişi sayısı nedir? Cevap sürpriz şekilde az kişidir. Gerçekten de bir yıl içindeki gün sayısından veya ileride göreceğimiz gibi bir yıl içindeki gün sayısının yarısından bile hayli azdır.

Bu soruyu cevaplamak için, odadaki insanları $1, 2, \dots, k$ tam sayıları ile gösteririz burada k odadaki insan sayısıdır. Artık yılı yok sayıyor ve her yılın 365 gün olduğunu kabul ediyoruz. $i = 1, 2, \dots, k$ için $1 \leq b_i \leq n$ olma koşuluyla b_i , i nin doğumgününe düşen yılın bir günü olsun. Ayrıca doğumgünlerinin n gün içinde düzgün dağıldığını kabul ediyoruz. Öyleyse $i = 1, 2, \dots, k$ ve $r = 1, 2, \dots, n$ için $\Pr \{ b_i = r \} = 1/n$ olur.

Verilen iki kişi için i ve j diyelim, doğumgünlerinin eşleşme olasılığı doğumgünlerinin rassal seçiminin bağımsızlığıyla ilgilidir. Doğumgünlerinin bağım-

sızlığını kabul ettiğimiz için i nin ve j nin ikisinde doğumgünlerinin düşme olasılığı r

$$\begin{aligned}\Pr\{b_i = r \text{ ve } b_j = r\} &= \Pr\{b_i = r\} \Pr\{b_j = r\} \\ &= 1/n^2\end{aligned}$$

olur. Bu yüzden ikisinin de aynı güne düşme olasılığı

$$\begin{aligned}\Pr\{b_i = b_j\} &= \sum_{r=1}^n \Pr\{b_i = r \text{ ve } b_j = r\} \\ &= \sum_{r=1}^n (1/n^2) \\ &= 1/n\end{aligned}$$

olur.

(5.6)

Daha sezgisel olarak, b_i seçildikten sonra b_j nin aynı gün seçilme olasılığı $1/n$ dir. Böylece i ve j nin aynı doğum günlerine sahip olma olasılıkları ikisinden birinin doğumgünlerinin verilen günde olasılığı ile aynıdır. Fakat bu tesadüf doğumgünlerinin bağımsız olması varsayımına bağlıdır.

Tümleyen olaya bakarak k kişi arasındaki en az 2 kişinin doğumgünlerinin eşleşme olasılığını analiz edebiliriz. En az 2 kişinin doğumgününün eşleşme olasılığı 1 eksi tüm doğumgünlerinin farklı olma olasılığına eşittir. k kişinin ayrı doğumgünlerinin olma olayı

$$B_k = \bigcap_{i=1}^k A_i$$

dir. Burada tüm $j < i$ için A_i olayı i kişinin doğumgününün j kişinininkinden farklı olma olayıdır. $B_k = A_k \cap B_{k-1}$, yazabildiğimiz için (C.16) denkleminde şu özyinelemeyi elde ederiz.

$$\Pr\{B_k\} = \Pr\{B_{k-1}\} \Pr\{A_k \mid B_{k-1}\} \quad (5.7)$$

burada $\Pr\{B_1\} = \Pr\{A_1\} = 1$ başlangıç koşuludur. Başka bir deyişle $b_1, b_2, \dots, b_k$ farklı doğumgünleri olma olasılığı $b_1, b_2, \dots, b_{k-1}$ kere $i = 1, 2, \dots, k-1$ için ve $b_1, b_2, \dots, b_{k-1}$ farklı verildiğinde $b_k \neq b_i$ olasılığıdır.

Eğer $b_1, b_2, \dots, b_{k-1}$ farklı ise n gün arasında $n - (k-1)$ gün alınmadığı için koşullu olasılık $i = 1, 2, \dots, k-1$ için $b_k \neq b_i \Pr\{A_k \mid B_{k-1}\} = (n - k + 1)/n$ dir. Özyineleme (5.7) yi uygulayıp

$$\begin{aligned}
\Pr\{B_k\} &= \Pr\{B_{k-1}\} \Pr\{A_k \mid B_{k-1}\} \\
&= \Pr\{B_{k-2}\} \Pr\{A_{k-1} \mid B_{k-2}\} \Pr\{A_k \mid B_{k-1}\} \\
&\vdots \\
&= \Pr\{B_1\} \Pr\{A_2 \mid B_1\} \Pr\{A_3 \mid B_2\} \cdots \Pr\{A_k \mid B_{k-1}\} \\
&= 1 \cdot \left(\frac{n-1}{n}\right) \left(\frac{n-2}{n}\right) \cdots \left(\frac{n-k+1}{n}\right) \\
&= 1 \cdot \left(1 - \frac{1}{n}\right) \left(1 - \frac{2}{n}\right) \cdots \left(1 - \frac{k-1}{n}\right)
\end{aligned}$$

elde edilir.

Eşitsizlik (3.12), $1+x \leq e^x$ bize $-k(k-1)/2n \leq \ln(1/2)$

$$\begin{aligned}
\Pr\{B_k\} &\leq e^{-1/n} e^{-2/n} \cdots e^{-(k-1)/n} \\
&= e^{-\sum_{i=1}^{k-1} i/n} \\
&= e^{-k(k-1)/2n} \\
&\leq 1/2
\end{aligned}$$

verir. k doğumgününün farklı olma olasılığı $k(k-1)$ için en fazla $1/2$ olur veya $k \geq (1 + \sqrt{1 + (8 \ln 2)n})/2$ sağlandığı zamanlar için kuadratik denklem çözülür. $n = 365$ için $k \geq 23$ olmalıdır. Bu yüzden eğer odada en az 23 kişi varsa, en az iki kişinin aynı doğumgününe sahip olma olasılığı en az $1/2$ dir. Marsta bir yıl 669 Mars günü uzunluğundadır: Bu yüzden aynı etkiyi almak 31 Mars günü sürer.

Rassal gösterge değişkenleri kullanılarak analiz etme

Rassal gösterge değişkenlerini, doğumgünü paradoksunun daha basit fakat yaklaşıklık analizini elde etmek için kullanabiliriz.

$X_{ij} = I\{i. \text{ kişi ile } j. \text{ kişi aynı doğumgününe sahip}\}$

$$= \begin{cases} 1 & i. \text{ kişi ile } j. \text{ kişi aynı doğumgününe sahipse,} \\ 0 & \text{aksi halde,} \end{cases}$$

(5.6) denkleminde, iki kişinin doğumgünlerinin eşleşme olasılığı $1/n$ ve böylece Lemma 5.1 ile

$$\begin{aligned}
E[X_{ij}] &= \Pr\{i. \text{ kişi ile } j. \text{ kişi aynı doğumgününe sahip}\} \\
&= 1/n
\end{aligned}$$

olur.

X aynı doğumgünlerine sahip olan kişilerin çiftlerinin sayılarını sayan rassal değişken olursa

$$X = \sum_{i=1}^k \sum_{j=i+1}^k X_{ij}$$

elde ederiz.

Her iki taraftan da beklenti alınıp beklentinin doğrusallığı uygulandığında

$$\begin{aligned} E[X] &= E \left[\sum_{i=1}^k \sum_{j=i+1}^k X_{ij} \right] \\ &= \sum_{i=1}^k \sum_{j=i+1}^k E[X_{ij}] \\ &= \binom{k}{2} \frac{1}{n} \\ &= \frac{k(k-1)}{2n} \end{aligned}$$

elde ederiz.

$k(k-1) \geq 2n$ olduğunda aynı doğumgünündeki kişilerin çiftlerinin beklenen sayısı en az 1 dir. Bu yüzden odada en az $\sqrt{2n} + 1$ kadar kişi varsa en az iki kişinin aynı doğumgününe sahip olmasını bekleyebiliriz. $n = 365$ için eğer $k = 28$ ise aynı doğumgünleri olan çiftlerin beklenen sayısı $(28 \cdot 27)/(2 \cdot 365) \approx 1.0356$ olur. Bu yüzden en az 28 kişi ile en az bir tane eşleşen doğumgünü bulmayı bekleyebiliriz. Bir yıl 669 Mars günü uzunluğunda olan Marsta en az 38 Mars gününe gerek duyarız.

Sadece olasılıkların kullanıldığı ilk analiz doğumgünü çiftlerinin eşleştiği $1/2$ yi aşan olasılık için gereken kişi sayısını belirledi ve rassal gösterge değişkenlerinin kullanıldığı ikinci analizde eşleşen doğumgünlerinin beklenen sayısının 1 olduğu belirlendi. İki durum için kesin kişi sayıları değişkenlik göstermesine rağmen, asimptotik olarak aynıdırlar: $\Theta(\sqrt{n})$.

5.4.2 Toplar ve kutular

1 den b ye kadar numaralandırılmış b kutuya rastgele olacak şekilde özdeş top- lar attığımızı düşünelim. Atışlar bağımsızdır ve her atışta top herhangi bir kutu içine girer. Atılan topun herhangi verilen bir kutuya girme olasılığı $1/b$ dir. Bu yüzden top atım süreci başarı olasılığı $1/b$ olan Bernoulli deneme dizisidir (Ek C.4'e bakınız). Burada başarı topun verilen kutuya düşmesi demektir. Bu model karmaşıklığı analiz etmek için kısmen faydalıdır (Bölüm 11 e bakınız) ve top atışı hakkında bir çok değişik soruyu cevaplayabiliriz. (Problem C.1 top- lar ve kutular hakkında ek sorular içerir).

Kaç top verilen kutunun içine düşer? Verilen kutunun içine düşen top sayısı $b(k; n, 1/b)$ binomial dağılımını takip eder. Eğer n top atılırsa C.37 denkleminde verilen kutunun içine düşen topların beklenti sayısı n/b olur.

Verilen kutunun bir top içermesi için ortalama kaç top atmalıyız? Verilen kutunun bir top içermesi için yapılan atış sayısı geometrik dağılımı $1/b$ olasılıkla takip eder ve C.32 denkleminde başarıya ulaşana kadarki atışın beklenti sayısı $1/(1/b) = b$ olur.

Her kutunun en az bir top içermesi için kaç top atılmalıdır? Bir topun boş bir kutuya girmesi için yapılan atışa ‘isabet’ diyelim. b isabeti elde etmek için gereken atışların beklenti numarası n i bilmek istiyoruz.

İsabetleri kullanarak n atışı bölümlere ayırabiliriz. i . basamak, i . isabete kadar olan $(i - 1)$. isabetten sonraki atışlardan oluşur. Tüm kutular boşken isabet yapma garantilendiği için ilk basamak ilk atıştan sonra oluşur. Her atış için i . basamak boyunca $i - 1$ kutu top içerir ve $b - i + 1$ kutu boştur. Böylece her atış için i . basamakta bir isabet elde etme olasılığı $(b - i + 1)/b$ dir.

n_i i . basamaktaki atış sayısını gösterebilir. Böylelikle b isabet elde etmek için gereken atış sayısı $n = \sum_{i=1}^b n_i$ olur. Her rassal değişken n_i için $(b - i + 1)/b$ başarı olasılığıyla bir geometrik dağılım vardır, C.32 denkleminde

$$E[n_i] = \frac{b}{b - i + 1}$$

olur. Beklentinin doğrusallığından ise

$$\begin{aligned} E[n] &= E\left[\sum_{i=1}^b n_i\right] \\ &= \sum_{i=1}^b E[n_i] \\ &= \sum_{i=1}^b \frac{b}{b - i + 1} \\ &= b \sum_{i=1}^b \frac{1}{i} \\ &= b(\ln b + O(1)) \quad (\text{Denklem (A.7) ye göre}) \end{aligned}$$

olur.

Bu yüzden her kutunun bir top içermesini beklemeden önce yaklaşık olarak $b \ln b$ atış gerekir. b tane farklı kupon toplamaya çalışan bir kişinin bunu başarması için yaklaşık $b \ln b$ kadar rastgele kupon elde etmesi gerektiğini anlatan problem **kupon toplayıcısı problemi** olarak da bilinir.

5.4.3 Ardışık tekrarlar

Diyelim ki bir metal parayı n kez atıyorsunuz. Turalar en uzun ne kadar ard arda gelmesini beklersiniz? Cevap sıradaki çözümlerin göstereceği gibi $\Theta(\lg n)$ olur.

Öncelikle turanın en fazla ard arda gelmesinde beklenen uzunluğun $O(\lg n)$ olduğunu kanıtlıyoruz. Her metal para atılışında tura gelme olasılığı $1/2$ dir. A_{ik} en az k , i . para atılışıyla başlayan uzunluğun turanın en fazla ard arda gelmesi olayı olsun? Daha ayrıntılı olarak k tane ardışık para atışı olayı $i, i+1, \dots, i+k-1$ sadece tura sonucu verir. Burada $1 \leq k \leq n$ ve $1 \leq i \leq n-k+1$ Para atışları karşılıklı olarak bağımsız olduğu için tüm k atışlarının tura gelme olasılığı

$$\Pr \{A_{ik}\} = 1/2^k. \quad (5.8)$$

$k = 2 \lceil \lg n \rceil$ için,

$$\begin{aligned} \Pr \{A_{i,2\lceil \lg n \rceil}\} &= 1/2^{2\lceil \lg n \rceil} \\ &\leq 1/2^{2\lg n} \\ &= 1/n^2 \end{aligned}$$

olur ve böylece uzunluğun turanından ardışık tekrarı en az ve i pozisyonunda başlar. Uzunluğun turanın en fazla ard arda gelmesinin olasılığı en az $2 \lceil \lg n \rceil$ herhangi bir yerden başlar bu yüzden

$$\begin{aligned} \Pr \left\{ \bigcup_{i=1}^{n-2\lceil \lg n \rceil+1} A_{i,2\lceil \lg n \rceil} \right\} &\leq \sum_{i=1}^{n-2\lceil \lg n \rceil+1} 1/n^2 \\ &< \sum_{i=1}^n 1/n^2 \\ &= 1/n \end{aligned} \quad (5.9)$$

olur. Çünkü Boole eşitsizliğinden (C.19) olayların birleşiminin olasılığı en fazla ayrı olayların olasılıkların toplamı kadardır. (Boole eşitsizliği, bu gibi bağımsız olmayan olaylar için eşitsizliği korur.)

En uzun ard arda gelme uzunluğunu sınırlamak için (5.9) eşitsizliğini kullanıyoruz. $j = 0, 1, 2, \dots, n$ için turanın en uzun ardışık tekrarı j olacak şekilde L_j olayı olsun ve L en uzun ardışık tekrarın uzunluğu olsun. Beklenen değer tanımından

$$E[L] = \sum_{j=0}^n j \Pr \{L_j\} \quad (5.10)$$

olur.

Bu toplam üst sınır kullanarak her $\Pr \{L_j\}$ üstünde (5.9) eşitsizliğinde hesaplandığı gibi çalışabiliriz. Maalesef, bu yöntem zayıf sınırlarla sonuçlanır. İyi bir sınır elde etmek için yukarıdaki analizlerden bazı sezgileri kullanabiliriz. (5.10) denklemindeki toplamda hiçbir ayrı terim için j ve $\Pr \{L_j\}$ faktörlerinin genişliğini gözlemleyemedik. Neden ? $j \geq 2 \lceil \lg n \rceil$ olduğunda ise j epeyce küçük olur. Usule uygun şekilde $j = 0, 1, \dots, n$ için L_j olayları ayrıktır ve böylece ard arda gelen turaların uzunluğu en az $\sum_{j=2\lceil \lg n \rceil}^n \Pr \{L_j\}$ olur. (5.9) eşitsizliğinden $\sum_{j=2\lceil \lg n \rceil}^n \Pr \{L_j\} < 1/n$ ayrıca $\sum_{j=0}^n \Pr \{L_j\} = 1$ hatırlatılarak $\sum_{j=0}^{2\lceil \lg n \rceil-1} \Pr \{L_j\} \leq 1$ olur. Böylece

$$\begin{aligned}
 E[L] &= \sum_{j=0}^n j \Pr \{L_j\} \\
 &= \sum_{j=0}^{2\lceil \lg n \rceil-1} j \Pr \{L_j\} + \sum_{j=2\lceil \lg n \rceil}^n j \Pr \{L_j\} \\
 &< \sum_{j=0}^{2\lceil \lg n \rceil-1} (2 \lceil \lg n \rceil) \Pr \{L_j\} + \sum_{j=2\lceil \lg n \rceil}^n n \Pr \{L_j\} \\
 &= 2 \lceil \lg n \rceil \sum_{j=0}^{2\lceil \lg n \rceil-1} \Pr \{L_j\} + n \sum_{j=2\lceil \lg n \rceil}^n \Pr \{L_j\} \\
 &< 2 \lceil \lg n \rceil \cdot 1 + n \cdot (1/n) \\
 &= O(\lg n)
 \end{aligned}$$

elde edilir.

Turanın ard arda gelmesi sayısının $r \lceil \lg n \rceil$ aşmasıyla olasılık hızla r yle azalır. $r \geq 1$ için en az $r \lceil \lg n \rceil$ turanın bir ardışık tekrarının i pozisyonunda başlattığı olasılık

$$\begin{aligned}
 \Pr \{A_{i,r\lceil \lg n \rceil}\} &= 1/2^{r\lceil \lg n \rceil} \\
 &\leq 1/n^r
 \end{aligned}$$

olur.

Böylece en uzun ard arda gelişin en az $r \lceil \lg n \rceil$ olma olasılığı en fazla $n/n^r = 1/n^{r-1}$ olur veya aynı şekilde en uzun ard arda gelişin $r \lceil \lg n \rceil$ den daha az olma olasılığı en az $1 - 1/n^{r-1}$ olur.

Örnek olarak $n = 1000$ para atışı için en az $2 \lceil \lg n \rceil = 20$ ardışık tekrara sahip olma olasılığı en fazla $1/n = 1/1000$ dir. $3 \lceil \lg n \rceil = 30$ dan fazla ard arda geliş elde etme ise en fazla $1/n^2 = 1/1.000.000$ dur.

Şimdi ise tümleyen alt sınırı kanıtlayacağız: en uzun ardışık tekrarının beklenen uzunluğu n atışta $\Omega(\lg n)$ dir. Bu sınırı kanıtlamak için eğer $s = \lfloor (\lg n)/2 \rfloor$

seçilirse bu gruplardan en az birinin tamamının tura olacağını ve en uzun ard arda gelişin en az $j = \Omega(\lg n)$ gösterebiliriz. Sonrasında ise en uzun ardışık tekrarının beklenen uzunluğu $\Omega(\lg n)$ olur.

n para atışını en az $\lfloor n / \lfloor (\lg n) / 2 \rfloor \rfloor$ ardışık atışın en az $\lfloor (\lg n) / 2 \rfloor$ grubu olarak ayırırız ve hiçbir grubun tümünün tura geleyecek şekilde olasılığı sınırlarız. (5.8) denkleminde i pozisyonunda başlayan grubun tümünün tura gelme olasılığı

$$\begin{aligned} \Pr \{A_{i, \lfloor (\lg n) / 2 \rfloor}\} &= 1/2^{\lfloor (\lg n) / 2 \rfloor} \\ &\geq 1/\sqrt{n} \end{aligned}$$

olur.

Ard arda tura gelme olasılığı olan $\lfloor (\lg n) / 2 \rfloor$ i . pozisyonundan başlamaz, dolayısıyla en fazla $1 - 1/\sqrt{n}$ olur. $\lfloor n / \lfloor (\lg n) / 2 \rfloor \rfloor$ grup karşılıklı olarak özel, bağımsız para atışlarıyla biçimlendirildiği için bu gruplardan her birinin $\lfloor n / \lfloor (\lg n) / 2 \rfloor \rfloor$ uzunluklu ard arda atış olamama olasılığı en fazla

$$\begin{aligned} (1 - 1/\sqrt{n})^{\lfloor n / \lfloor (\lg n) / 2 \rfloor \rfloor} &\leq (1 - 1/\sqrt{n})^{n / \lfloor (\lg n) / 2 \rfloor - 1} \\ &\leq (1 - 1/\sqrt{n})^{2n / \lg n - 1} \\ &\leq e^{-(2n / \lg n - 1) / \sqrt{n}} \\ &= O(e^{-\lg n}) \\ &= O(1/n) \end{aligned}$$

olur.

Bu iddia için (3.12) eşitsizliğini kullandık. $1 + x \leq e^x$ ve kontrol etmek istediğiniz gerçek yeterli büyüklükteki n için $(2n / \lg n - 1) / \sqrt{n} \geq \lg n$ olur.

Böylece, en uzun ard arda gelme sayısı $\lfloor (\lg n) / 2 \rfloor$ yi aşar.

$$\sum_{j=\lfloor (\lg n) / 2 \rfloor}^n \Pr \{L_j\} \geq 1 - O(1/n) . \quad (5.11)$$

Şimdi (5.10) denkleminle başlayarak en uzun ardışık tekrarların beklenen uzunluğundaki bir alt sınır hesaplayabiliriz ve üst sınır analizindeki gibi ilerleriz.

$$\begin{aligned}
E[L] &= \sum_{j=0}^n j \Pr\{L_j\} \\
&= \sum_{j=0}^{\lfloor (\lg n)/2 \rfloor - 1} j \Pr\{L_j\} + \sum_{j=\lfloor (\lg n)/2 \rfloor}^n j \Pr\{L_j\} \\
&\geq \sum_{j=0}^{\lfloor (\lg n)/2 \rfloor - 1} 0 \cdot \Pr\{L_j\} + \sum_{j=\lfloor (\lg n)/2 \rfloor}^n \lfloor (\lg n)/2 \rfloor \Pr\{L_j\} \\
&= 0 \cdot \sum_{j=0}^{\lfloor (\lg n)/2 \rfloor - 1} \Pr\{L_j\} + \lfloor (\lg n)/2 \rfloor \sum_{j=\lfloor (\lg n)/2 \rfloor}^n \Pr\{L_j\} \\
&\geq 0 + \lfloor (\lg n)/2 \rfloor (1 - O(1/n)) \quad ((5.11) \text{ eşitsizliğinden}) \\
&= \Omega(\lg n) .
\end{aligned}$$

Doğumgünü paradoksunda olduğu gibi rassal gösterge değişkenleri kullanılarak daha basit fakat yaklaşık bir analiz elde edebiliriz. $X_{ik} = I\{A_{ik}\}$ turanın i . ara atışıyla başlayan, turanın en az k uzunluklu bir ardışık tekrarına bağlı bir rassal gösterge değişkeni olsun. Ardışık tekrarların toplam sayısını hesaplamak için

$$X = \sum_{i=1}^{n-k+1} X_{ik}$$

tanımlanır. Beklentiler alınıp beklentinin doğrusallığı kullanılarak

$$\begin{aligned}
E[X] &= E\left[\sum_{i=1}^{n-k+1} X_{ik}\right] \\
&= \sum_{i=1}^{n-k+1} E[X_{ik}] \\
&= \sum_{i=1}^{n-k+1} \Pr\{A_{ik}\} \\
&= \sum_{i=1}^{n-k+1} 1/2^k \\
&= \frac{n-k+1}{2^k}
\end{aligned}$$

elde edilir.

k için değişik değerler yerleştirilerek k uzunluğundaki ardışık tekrarların beklenti sayısını hesaplayabiliriz. Eğer bu sayı büyükse (1 den büyükse) k uzunlu-

ğundaki bir çok ard arda gelişin meydana geleceğini ve oluşan olasılığın yüksek olacağını bekleyebiliriz. Eğer c pozitif sabiti için $k = c \lg n$ ise

$$\begin{aligned}
 E[X] &= \frac{n - c \lg n + 1}{2^{c \lg n}} \\
 &= \frac{n - c \lg n + 1}{n^c} \\
 &= \frac{1}{n^{c-1}} - \frac{(c \lg n - 1)/n}{n^{c-1}} \\
 &= \Theta(1/n^{c-1})
 \end{aligned}$$

elde edilir.

Eğer c büyükse $c \lg n$ uzunluktaki ard arda gelişin beklenen sayısı küçük olur ve olası olmayan bir şekilde meydana geleceği sonucuna varılır. Öte yandan eğer $c = 1/2$ ise $E[X] = \Theta(1/n^{1/2-1}) = \Theta(n^{1/2})$ elde edilir ve $(1/2) \lg n$ uzunluğundaki ardışık tekrarların geniş sayılar olacağını bekleyebiliriz. O halde, bu uzunlukta bir ard arda gelişin olması olasıdır. Bu kaba tahminlerden şu sonuca varabiliriz: beklenen en uzun ard arda geliş $\Theta(\lg n)$ dir.

5.4.4 Çevrimiçi kiralama problemi

Son örnek olarak kiralama probleminin bir değişkeni üzerinde duracağız. Diyelim ki en iyi adayı bulmak için tüm adaylarla görüşmeyi istemiyoruz. Daha ve daha iyi başvuruları bulmak için işe alıp işten çıkarmayı da istemiyoruz. Bunun yerine bir kez işe alım yaparak en iyi adayı yerleştirmeyi istiyoruz. Bir şirket talebine uymalıyız: Her görüşmeden sonra çok kısa süre içinde aday işe alınmalı veya başvurusu reddedilmelidir. Görüşme sayısının en aza indirilmesi ile işe alınan adayın niteliği arasındaki denge nasıl kurulur?

Bu problemi takip eden şekilde modelleyebiliriz. Bir adayla görüştüktan sonra her birine puan verilebilir; $score(i)$, i . başvurana verilen puanı gösterebilir ve iki başvuranın aynı puanı almadığını kabul edelim. j başvurana görüştüktan sonra j adaydan hangisinin en yüksek puanı aldığını biliyoruz fakat kalan $n - j$ başvuranın daha yüksek bir puan alıp alamayacağını bilmiyoruz. $k(n)$ olacak şekilde bir pozitif sayı seçip, ilk k başvurana görüşüp reddettikten sonra öncekilerden daha yüksek puanlı ilk başvurana işe almak yönünde bir strateji geliştirmeye karar veriyoruz. Eğer en iyi nitelikli başvuran ilk k görüşülen arasındaysa n . başvurana işe alıyoruz. Bu stratejiyi işe almak istediğimiz adayı döndüren ON-LINE-MAXIMUM (k, n) prosedürüyle biçimlendiriyoruz.

ON-LINE-MAXIMUM(k, n)

```

1  bestscore =  $-\infty$ 
2  for  $i = 1$  to  $k$ 
3      if  $score(i) > bestscore$ 
4          bestscore =  $score(i)$ 
5  for  $i = k + 1$  to  $n$ 
6      if  $score(i) > bestscore$ 
7          return  $i$ 
8  return  $n$ 

```

Her olası k değeri için en nitelikli başvuranın olasılığına karar vermek istiyoruz. En iyi olası k yi seçip bu değer için stratejiyi uyguluyoruz. Şu anda k nin ayarlanmış olduğunu varsayalım. $M(j) = \max_{1 \leq i \leq j} \{score(i)\}$, 1 den j ye uzanan başvurular arasındaki en yüksek puanı gösterebilir. S en nitelikli adayın seçilmesinin başarılmaması olayı ve S_i en nitelikli başvuranın bulunmamasının i . görüşülen adayda başarılmaması olayıdır. Çeşitli S_i olayları ayrık olduğu için $\Pr\{S\} = \sum_{i=1}^n \Pr\{S_i\}$ elde edilir. En iyi adayın ilk başvurular arasında olmadığı hatırlatılarak, $i = 1, 2, \dots, k$ için $\Pr\{S_i\} = 0$ olur. Böylece

$$\Pr\{S\} = \sum_{i=k+1}^n \Pr\{S_i\} \quad (5.12)$$

elde ederiz.

Şimdi ise $\Pr\{S_i\}$ değerini hesaplıyoruz. En iyi başvuranın i . olması için iki şey olmalı. İlk olarak B_i olarak gösterilen olay en iyi başvuran i pozisyonunda olmalı. İkinci olarak ise algoritma $k + 1$ den $i - 1$ e uzanan pozisyonunda olan başvuruları seçmemelidir. Bu durum her j için $k + 1 \leq j \leq i - 1$ durumu için gerçekleşir. Satır 6 da $score(j) < bestscore$ olduğunu fark ediyoruz. (Puanlar yegane olduğu için $score(j) = bestscore$ durumunu göz ardı edebiliriz). Bir başka deyişle $score(k + 1)$ den $score(i)$ ye kadar olan değerler $M(k)$ den küçük olmalıdır; eğer herhangi biri $M(k)$ den büyük ise büyük olan birincinin indeksine geri döneriz. O_i , $k + 1$ den $i - 1$ pozisyonuna kadar olan adaylardan hiçbirinin seçilmeme olayını gösterebilir. Maalesef B_i ve O_i bağımsızdır. O_i sadece 1 den $i - 1$ e uzanan pozisyonundaki değerlerin aralarında sıralanmasına bağlıdır oysa B_i i pozisyonundaki adayların diğer pozisyonlardakilerin değerlerinden büyük olup olmamasına bağlıdır. 1 den $i - 1$ e uzanan pozisyonlardaki değerlerin sıralamaları, i pozisyonundaki değerin diğerlerinden büyük olmasını etkilemez ve i pozisyonundaki değer 1 den $i - 1$ e uzanan pozisyonundaki değerlerin sıralamalarını etkilemez. (C.15) denklemini

$\Pr\{S_i\} = \Pr\{B_i \cap O_i\} = \Pr\{B_i\} \Pr\{O_i\}$ uygulayabiliriz.

Maksimum n pozisyonundaki herhangi birine eşit olduğu için $\Pr\{B_i\}$ olasılığının $1/n$ olduğu açıktır. O_i olayı için 1 den $i - 1$ e uzanan pozisyonundaki maksimum değer $i - 1$ pozisyonundakilerden herhangi birine eşit olan ilk k pozisyonundakilerden biri olmalıdır. Sonuç olarak $\Pr\{O_i\} = k/(i - 1)$ ve $\Pr\{S_i\} = k/(n(i - 1))$ olur. (5.12) denklemi kullanılarak

$$\begin{aligned} \Pr\{S\} &= \sum_{i=k+1}^n \Pr\{S_i\} \\ &= \sum_{i=k+1}^n \frac{k}{n(i - 1)} \\ &= \frac{k}{n} \sum_{i=k+1}^n \frac{1}{i - 1} \\ &= \frac{k}{n} \sum_{i=k}^{n-1} \frac{1}{i} \end{aligned}$$

elde edilir.

Bu toplamı alttan ve üstten sınırlamak için integral ile yaklaşıyoruz. (A.12) eşitsizliklerinden

$$\int_k^n \frac{1}{x} dx \leq \sum_{i=k}^{n-1} \frac{1}{i} \leq \int_{k-1}^{n-1} \frac{1}{x} dx$$

elde edilir. Bu belirli integrallerin sonuçları bize

$$\frac{k}{n}(\ln n - \ln k) \leq \Pr\{S\} \leq \frac{k}{n}(\ln(n - 1) - \ln(k - 1))$$

sınırları verir. Bu sınır $\Pr\{S\}$ için daha sıkı bir sınır sunar. Çünkü başarı olasılığımızı en yüksek değere ulaştırmak istiyoruz. $\Pr\{S\}$ deki alt sınırı en yüksek değere çıkaran k değerine odaklanalım. (Ayrıca alt sınır ifadesini en yüksek değere çıkarmak üst sınır ifadesinden daha kolaydır). $(k/n)(\ln n - \ln k)$ ifadesinin k ye göre integralini alarak

$$\frac{1}{n}(\ln n - \ln k - 1) \text{ elde edilir.}$$

Türev 0 a eşit olacak şekilde ayarlanarak $k = \ln n - 1 = \ln(n/e)$ veya $k = n/e$ olduğunda olasılıktaki alt sınırı en yüksek değere çıkarıyoruz. Böylece eğer $k = n/e$ ile stratejimizi uygularsak en az $1/e$ olasılıkla en nitelikli başvurayı işe almaya başlamış oluruz.

Alıştırılmalar**5.4-1**

En az $1/2$ olasılıkla aynı doğumgününe sahip olan iki kişi olması için bir odada kaç kişi olmalıdır. En az iki kişinin 4 Temmuz'da doğma olasılığının $1/2$ den büyük olması için kaç kişi olmalıdır?

5.4-2

Diyelim ki, bazı kutular iki top içerece kadar b kutuya top atıyoruz. Her atış bağımsız ve her top kutuya giriyor. Top atışlarının beklenti sayısı nedir?

5.4-3 *

Doğumgünü paradoksunun analizi için doğumgünlerinin karşılıklı olarak bağımsız olması önemli midir veya bağımsızlık yeterli midir? Cevabınızı doğrulayın.

5.4-4 *

3 kişinin aynı doğumgününe sahip olması için bir partiye kaç kişi davet edilme-lidir?

5.4-5 *

n boyutlu bir küme üzerinde tanımlanan bir k -karakter dizisinin bir k -permütasyonunu oluşturma olasılığı nedir? Bu sorunun doğumgünü paradoksuyla ilişkisi nedir?

5.4-6 *

Diyelim ki n top n kutuya atılıyor. Burada tüm atışlar bağımsızdır ve top herhangi bir kutuya girer. Boş kutunun beklenti sayısı nedir? Tam olarak 1 top olan kutuların beklenti sayısı nedir?

5.4-7 *

Hilesiz madeni bir paranın n defa atılmasıyla elde edilen ard arda geliş sayısının alt sınırını net bir şekilde belirleyin. Bu olasılık $1/n$ den azdır çünkü hiç bir ard arda geliş sayısı $\lg n - 2 \lg \lg n$ den büyük olamaz.

Problemler

5-1 Olasılıksal sayma

Bir b -bit sayaç ile $2^b - 1$ e kadar sayabiliriz. R. Morris'in *olasılıksal sayması* ile biraz kesinlikten feda ederek daha büyük sayılara kadar sayabiliriz.

$i = 0, 1, \dots, 2^b - 1$ olacak şekilde i sayaç değerinin negatif olmayan ve artan sırada yazılmış n_i yi sayan bir değer olduğunu düşünelim. Bu değerlerin ilk sayacının değerini 0 olarak kabul ediyoruz, yani $n_0 = 0$. INCREMENT işlemi i değerini içeren bir sayaç üzerinde olasılıksal şekilde çalışır. Eğer $i = 2b - 1$ olursa, o zaman işlem taşma hatası mesajı verir. Diğer durumlarda, INCREMENT işlemi sayacı $1 - 1/(n_{i+1} - n_i)$ olasılıkla 1 artırır ve sayacı da değiştirmeden $1 - 1/(n_{i+1} - n_i)$ olasılıkla bırakır.

Eğer tüm $i \geq 0$ için $n_i = i$ seçersek, sayaç normal bir sayaç halini alır. Ama eğer tüm $i > 0$ için $n_i = 2^{i-1}$ veya $n_i = F_i$ (i . Fibonacci sayısı – bkz. Kısım 3.2) olarak alırsak o zaman daha ilginç sonuçlar ortaya çıkmaktadır.

Bu problem için, n_{2b-1} in fazlasıyla büyük olduğunu ve taşma hatasının göz ardı edildiğini varsayın.

- n adet INCREMENT işlemi sonunda sayacın göstereceği beklenen değerin tam olarak n olacağını gösterin.
- Sayacın gösterdiği sayının varyans analizi, n_i dizisine bağlıdır. Şu basit durumu ele alalım: tüm $i \geq 0$ için $n_i = 100i$ olsun. n adet INCREMENT işlemi sonunda kaydedicinin göstereceği değerin varyansını tahmin edin.

5-2 Sıralı olmayan diziyi arama

Bu problem, n elemanlı ve sıralı olmayan A dizisindeki bir x değerini arayan üç farklı algoritmayı incelemektedir.

Rastgele oluşturulmuş şu stratejiyi ele alalım: A dan rastgele bir i indisi seçin. Eğer $A[i] = x$ ise, o zaman programı sonlandırın. Değilse, $A[j] = x$ olacak şekilde ya da A nın tüm elemanları kontrol edilene kadar, A da rastgele yeni bir indisi seçerek devam edelim. Dikkat ettiyseniz, verilen bir elemanı birden fazla kez inceleyebilmemiz için her seferinde tüm indis kümesinden seçim yapıyoruz.

- Yukarıda bahsettiğimiz stratejiye uygulayabileceğiniz bir kabakod RANDOM-SEARCH prosedürü yazınız. Ancak, algoritmanızın A nın tüm indisleri seçildikten sonra sonlanmış olmasına dikkat edin.

- b.** Diyelim ki, $A[i] = x$ olacak şekilde sadece tek bir i indisi vardır. x i bulana ya da RANDOM-SEARCH algoritması sonlanana kadar seçebileceğimiz kaç adet indis vardır?
- c.** (b) şıkında elde ettiğiniz çözümü geliştiriniz. Burada, $A[i] = x$ olacak şekilde $k \geq 1$ adet i indisiniz olduğunu kabul edin. Peki şimdi x i bulana ya da RANDOM-SEARCH algoritması sonlanana kadar seçebileceğimiz kaç adet indis vardır? Cevabınız n ve k dan oluşan bir fonksiyon olmalıdır.
- d.** Diyelim ki, $A[i] = x$ olacak şekilde hiç indisiniz yok. O halde x i bulana ya da RANDOM-SEARCH algoritması sonlanana kadar seçebileceğimiz kaç adet indis vardır?

Şimdi, deterministik bir doğrusal arama algoritması düşünün. Biz buna DETERMINISTIC-SEARCH algoritması diyeceğiz. Bu algoritma, özellikle, $A[i] = x$ i bulana veya dizinin sonuna varana kadar x i A nın içinde sırasıyla, yani $A[1], A[2], A[1], \dots, A[n]$ olacak şekilde, aramaktadır. Giriş dizisinin tüm permütasyonlarının eşit olasılıkla gelebileceğini kabul edin.

- e.** Diyelim ki, $A[i] = x$ olacak şekilde sadece tek bir i indisi vardır. DETERMINISTIC-SEARCH algoritmasının ortalama çalışma zamanı nedir? DETERMINISTIC-SEARCH algoritmasının en kötü-durum çalışma zamanı nedir?
- f.** (e) şıkında elde ettiğiniz çözümü geliştiriniz. Burada, $A[i] = x$ olacak şekilde $k \geq 1$ adet i indisiniz olduğunu kabul edin. Bu durumda, DETERMINISTIC-SEARCH algoritmasının ortalama çalışma zamanı nedir? DETERMINISTIC-SEARCH algoritmasının en kötü-durum çalışma zamanı nedir? Cevabınız n ve k dan oluşan bir fonksiyon olmalıdır.
- g.** Diyelim ki, $A[i] = x$ olacak şekilde hiç indisiniz yok. Bu durumda, DETERMINISTIC-SEARCH algoritmasının ortalama çalışma zamanı nedir? DETERMINISTIC-SEARCH algoritmasının en kötü-durum çalışma zamanı nedir?

Son olarak, rassal bir algoritma olan SCRAMBLE-SEARCH ele alın. Bu algoritma, öncelikle giriş dizisini rassal bir permütasyon dizisi olarak oluşturur ve ardından yukarıda bahsettiğimiz ve bir permütasyon dizisi üreten deterministik doğrusal arama algoritmasını çalıştırır.

- h.** $A[i] = x$ olacak şekilde, i indislerinin sayısı k olsun. $k = 0$ ve $k = 1$ olan durumları arayan SCRAMBLE-SEARCH algoritmasının beklenen ve en kötü-durum çalışma sürelerini bulun. Bulduğunuz çözümü $k \geq 1$ i sağlayacak şekilde geliştirin.
- i.** Bu üç arama algoritmasından hangisini kullanırdınız? Cevabınızı açıklayın.

Bölüm notları

Bollobas [53], Hofri [174] ve Spencer [321] zengin bir ileri olasılıklı teknik içerir. Rassal algoritmaların avantajları tartışıldı ve Karp[200] ve Robin[288] tarafından incelendi. Matwani ve Raghavan'ın kitabı rassal algoritmalar için kapsamlı bir işleyiş sunar.

Kiralama probleminin birkaç değişkeni geniş olarak çalışıldı. Bu problemler genellikle “sekreter problemleri” olarak adlandırılırdı. Bu alandaki çalışma örneği Ajtai, Meggido ve Waarts[11] tarafından sunulmuştur.

II Sıralama ve Sıra İstatistikleri

Giriş

Bu bölüm aşağıdaki *sıralama problemlerini* çözen çeşitli algoritmalar sunmaktadır:

Girdi: Bir n elemanlı sayı dizisi $\langle a_1, a_2, \dots, a_n \rangle$.

Çıktı: $\langle a'_1, a'_2, \dots, a'_n \rangle$ girdi dizisinin $a'_1 \leq a'_2 \leq \dots \leq a'_n$ permütasyonu.

Girdi dizisi genelde n elemanlı bir dizidir; buna rağmen bağlı liste gibi farklı şekillerde de gösterilebilir.

Verilerin yapısı

Uygulamada sıralanacak sayılar nadiren izole edilmiş değerlerdir. Her biri genelde *kayıt* olarak adlandırılmış veri koleksiyonunun bir parçasıdır. Her kayıt sıralanacak değer olan bir *anahtar* içerir. Kaydın geri kalan bölümü genelde anahtarla birlikte taşınan *uydu verilerden* oluşur. Uygulamada sıralama algoritmaları anahtarları değiştirirken uydu verilerini de değiştirmek zorundadır. Eğer bir kayıt büyük miktarda uydu verisi içeriyorsa veri hareketini azaltmak için kayıtların kendilerini değiştirmek yerine kaydı gösteren işaretçilerin(pointer) dizisini değiştiririz.

Bir biçimde bir algoritmayı tamamen açılmış bir programdan ayıran bu uygulama detaylarıdır. Bir sıralama algoritması bireysel sayıları da sıralasak bir çok uydu verisi biti içeren geniş kayıtları da sıralasak sıralama düzenini belirleyerek yöntemi belirler. Bu nedenle sıralama problemine odaklanırken girdinin sadece sayılardan oluştuğunu varsayabiliriz. Sayıları sıralayan bir algoritmayı kayıtları sıralayan bir programa çevirmek oldukça basittir; buna rağmen verilen bir mühen-

dislik durumunda ince noktalar gerçek programlama görevini zorlu bir göreve dönüştürebilir.

Neden sıralama?

Birçok bilgisayar bilimcisi sıralama işlemini algoritmalar üzerinde çalışılan en temel problem olarak görmektedirler. Bunun çeşitli nedenleri bulunmaktadır:

- Bazen bir uygulamanın doğası gereği bilgileri sıralaması gerekir. Örnek olarak; bir bankanın müşterisinin bilgilerini hazırlamak için çekleri çek numarasına göre sıralaması gerekir.
- Algoritmalar sık sık sıralamayı bir altprogram olarak kullanır. Örnek olarak; birbirleri üzerinde katmanlı halde bulunan grafiksel objeleri işleyen bir program, bu objeleri tabandan tepeye kadar çizebilmesi için objeleri “above” bağlantısına göre sıralaması gerekmektedir.
- Sıralama algoritmalarını geniş bir yelpazede hazırlayabiliriz ve bu algoritmalar zengin bir işlem kümesi kullanır. Aslında algoritma tasarımı sırasında kullanılan bir çok önemli işlem, yıllar içinde geliştirilmiş sıralama algoritmaları şeklinde ortaya çıkar.
- Sıralama işlemi için zor bir alt sınır kanıtlayabiliriz (Bölüm 8 de yaptığımız gibi). En iyi üst sınırımız alt sınır ile asimptotik olarak eşleşmektedir; ve bu sayede sıralama algoritmalarımızın asimptotik en uygun olduklarını biliyoruz.
- Birçok mühendislik meselesi, sıralama algoritmalarını uygularken öne çıkmaktadır. Belirli bir durum için en hızlı sıralama programı; anahtarlar ve uydu verileri hakkındaki öncel bilgiler, bir sunucu bilgisayarının bellek hiyerarşisi ve yazılım ortamı gibi birçok faktöre bağlı olabilir. Bu konuların birçoğu, kodda değişiklikler yapmak yerine algoritmik düzeyde en iyi halledilir.

Sıralama algoritmaları

Bölüm 2 de n tane gerçel sayıyı sıralamayı sağlayan iki algoritma tanıttık. Araya sokma sıralaması en kötü durumda $\Theta(n^2)$ zamanda çalışır. Çünkü iç döngüleri sıkıdır; buna rağmen küçük sayıda girdi için hızlı bir sıralama algoritmasıdır. Birleştirerek sıralama algoritması $n \lg n$ gibi daha iyi bir asimptotik çalışma zamanına sahiptir.

Bu bölümde, rastgele gerçel sayıları sıralayan ikiden fazla algoritma tanıtacağız. Bölüm 6 da sunduğumuz bellek yığını sıralaması, n sayıyı yerinde $O(n \log n)$ sürede sıralar. Bellek yığını sıralaması, öncelikli kuyrukta dakullanılan önemli bir veri yapısı olan öncelikli kuyrukta da kullanır.

Bölüm 7 deki hızlı sıralamada yerinde sıralama yapar; ancak en kötü-durum çalışma süresi $\Theta(n^2)$ dir. Çalışma süresinin $\Theta(n \log n)$ olması beklenir; buna rağmen hızlı sıralama pratikte bellek yığını sıralamasından daha iyi çalışır. Hızlı sıralama, araya sokma sıralamasında olduğu gibi sıkı bir koda sahiptir; ve çalışma süresinde mevcut olan gizli sabit faktörü küçüktür. Hızlı sıralama büyük girdili diziler için popüler bir algoritmadır.

Araya sokma sıralaması, birleştirerek sıralama, bellek yığını sıralaması ve hızlı sıralamalarının tümü karşılaştırma sıralamalarıdır. Bu sıralamalar girdi dizisinin sıralanmış düzenine, elemanları karşılaştırarak karar verir. Bölüm 8 karşılaştırma algoritmalarının performans sınırlandırmalarını değerlendirmek için karar ağacı modelini tanıtarak başlar. Bu modeli kullanarak n girdide en kötü-durum çalışma süresi $\Omega(n \log n)$ olan herhangi bir karşılaştırma sıralamasının alt sınırını tanımladık. Böylece bellek yığını sıralaması ve birleştirerek sıralamanın asimptotik olarak en uygun karşılaştırma sıralamaları olduklarını göstermiş olduk.

Eğer elemanları karşılaştırmak yerine girdilerin sıralama düzeni hakkında bilgi toplayabilirsek o zaman bölüm 8 $\Omega(n \log n)$ in bu alt sınırını aşabileceğimizi gösterir. Sayma sıralama algoritması, örnek olarak girdi sayılarının $\{0, 1, \dots, k\}$ setinde olduğunu varsayar. Sayma sıralaması göreceli sıralamaya karar vermek için dizi indislemesini bir araç gibi kullanarak n sayıyı $\Theta(k + n)$ süresinde sıralayabilir. Bu nedenle sayma sıralaması girdi sayısının doğrusal olduğu zamanda çalışabilir. Bir göreceli sıralama olan taban sıralaması, sayma sıralamasının aralığını genişletmek için kullanılabilir. Eğer sıralanacak d haneli n tane tam sayı varsa ve her tam sayı k kadar farklı değer alabiliyorsa taban sıralaması bu sayıları $\Theta(d(n + k))$ sürede sıralayabilir. d bir sabit ve $k = O(n)$ olduğunda buket sıralaması doğrusal zamanda çalışır. Üçüncü bir sıralama olan kova sıralaması, girdi dizisindeki sayıların olasılıksal dağılım bilgilerine ihtiyaç duyar. Kova algoritması n tane gerçel sayıyı ortalama çalışma süresi $O(n)$ olan yarı açık aralıklı $[0, 1)$ düzgün dağıtılmış olarak sıralayabilir.

Aşağıdaki tablo, Bölüm 2 ve 6-8 den sıralama algoritmalarının çalışma sürelerini özetler. Her zamanki gibi n sıralanacak sayı adedini temsil etmektedir. Sayma algoritması için sıralanacak sayılar $\{0, 1, \dots, k\}$ kümesindeki sayılardır. Taban sıralaması için sıralanacak sayılar d haneli, k tane farklı değer alan tam sayılardır. Kova sıralaması için anahtarların yarı açık aralıkta düzgün olarak dağıtılmış gerçel sayılar olduğu varsayılır. En sağda bulunan kolon sütun en kötü-durum çalışma süresi dışında, ortalama veya beklenen çalışma süresini verir. Bellek yığını sıralamasının çalışma zamanını bu kitapta analiz etmediğimiz için ortalama çalışma süresi üzerinde durmadık.

Algoritma	En kötü-durum çalışma süresi	Ortalama-durum/beklenen çalışma süresi
Araya sokma Sıralaması	$\Theta(n^2)$	$\Theta(n^2)$
Birleştirerek Sıralama	$\Theta(n \lg n)$	$\Theta(n \lg n)$
Bellek yığını	$O(n \lg n)$	—
Hızlı Sıralama	$\Theta(n^2)$	$\Theta(n \lg n)$ (beklenen)
Sayma Sıralaması	$\Theta(k + n)$	$\Theta(k + n)$
Taban Sıralaması	$\Theta(d(n + k))$	$\Theta(d(n + k))$
Kova Sıralaması	$\Theta(n^2)$	$\Theta(n)$ (ortalama-durum)

Sıra İstatistikleri

n adet sayı kümesinin i . sıra düzen istatistiği o kümedeki i . en küçük sayıdır. Girdileri sıralayarak ve çıktının i . elemanını indisleyerek i . sıra istatistiğini seçebiliriz. Bu işlem Bölüm 8 de alt sınırı kanıtladığımız üzere girdi dağılımı hakkında hiçbir varsayımda bulunmadan $\Omega(n \log n)$ sürede çalışır.

Bölüm 9 da elemanlar rasgele gerçel sayılar olsa bile i nci en küçük sayıyı $O(n)$ sürede bulabileceğimizi göstermekteyiz. En kötü-durum çalışma süresi $\Theta(n^2)$ olan ancak beklenen çalışma süresi $O(n)$ olan sıkı kabakoda sahip rassal bir algoritma sunmaktayız. Ayrıca en kötü-durum çalışma süresi $O(n)$ den daha karmaşık bir algoritma da sunmaktayız.

Geçmiş

Bu bölümün birçok kısmı zor matematik bilgisine dayanmamasına rağmen bazı kısımlar matematiksel kapsamlılık gerektirmektedir. Özellikle Ek C de ve Bölüm 5 teki olasılıksal analiz ve rassal algoritma materyallerinde incelenmiş hızlı sıralama, kova sıralaması ve sıra istatistiği algoritmaları analizi olasılık kullanmaktadır. Sıra istatistiği için en kötü-durum doğrusal zaman algoritması analizi bir şekilde bu bölümdeki diğer en kötü-durum analizlerden daha kapsamlı bir matematik bilgisini içerir.

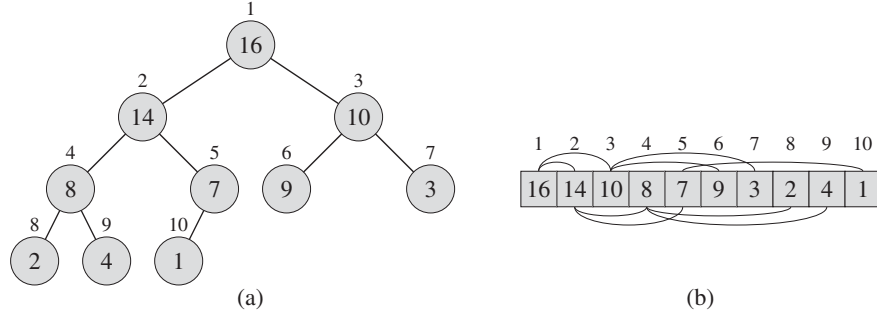
Bu bölümde, diğer bir sıralama algoritması olan bellek yığını sıralamasını tanıtmaktayız. Bellek yığını sıralaması, araya sokma sıralamasının aksine birleştirerek sıralamada olduğu gibi $O(n \lg n)$ zamanda çalışır. Bellek yığını sıralaması birleştirerek sıralamanın aksine araya sokma sıralamasında olduğu gibi yerinde sıralama yapar. Yani herhangi bir zamanda sadece sabit bir sayıda dizi elemanı girdi dizisinin dışında saklanır. Böylece bellek yığını sıralaması şimdiye kadar gördüğümüz diğer iki sıralama algoritmasının daha iyi olan özelliklerini birleştirir.

Ayrıca bellek yığını sıralaması bilgileri yönetmeye yarayan yığın dediğimiz yeni bir algoritma tasarım tekniği tanıtmaktayız. Bellek yığını veri yapısı sadece bellek yığını sıralaması için değil, etkili bir öncelikli kuyruk yapmak için de kullanışlıdır. Bellek yığını veri yapısını daha sonraki bölümlerde göreceğiz.

Bellek yığını terimi temelde bellek yığını sıralaması bağlamında türetilmiştir, ancak bellek yığını Java, Lisp gibi programlama dillerinde çöp toplama deposu (garbage collected storage) olarak geçmektedir. Bellek yığını veri yapımız çöp toplama deposu değildir ve bu kitaptaki her yerde bellek yığını olarak adlandıracağız, çöp koleksiyonunun görünümünden ziyade bir veri yapısını kasteteceğiz.

6.1 Bellek yığınları

Bir *(ikili) bellek yığını* veri yapısı Şekil 6.1 de görebileceğimiz gibi neredeyse tam ikili ağaç gibi görülebilen bir dizi objesidir. Ağacın her bir düğümü bir dizi elemanına denk gelmektedir. Ağaç sol alttan bir noktaya dolan muhtemelen en düşük seviyede hariç tüm seviyeler tamamen doldurulur. Bir bellek yığını temsil eden A dizisinin iki özelliği vardır: $A.length$, dizideki eleman sayısını veren *uzunluk* ve $A.heap-size$, A dizisi içinde bellek yığının kaç elemanının depolandığını gösteren *bellek yığını boyutu*. Yani, $A[1..A.length]$ sayıları içermesine rağmen $0 \leq A.heap-size \leq A.length$ olmak üzere sadece $A[1..A.heap-size]$ daki elemanlar bellek yığınının geçerli elemanlarıdır. Ağacın kökü (root) $A[1]$ dir ve bir düğümün i indisi verilerek o düğümün ebeveyninin, sol çocuğunun ve sağ çocuğunun indislerini kolayca hesaplayabiliriz:



Şekil 6.1 İkili ağaç ve dizi gibi görülen maks-bellek yığını. Ağaç düğümlerinin içindeki sayılar, bulunulan düğümde saklanan sayıdır. Düğüm üzerindeki sayılar dizi indislerine karşılık gelmektedir. Üstteki ve alttaki diziler ebeveyn-çocuk ilişkilerini gösteren satırlardır. Ebeveynler her zaman çocuklarının solundadır. Ağacın yüksekliği üçtür; dördüncü indisteki (8 değeri ile) düğümün yüksekliği birdir.

PARENT(i)
1 **return** $\lfloor i/2 \rfloor$

LEFT(i)
1 **return** $2i$

RIGHT(i)
1 **return** $2i + 1$

Birçok bilgisayarda LEFT prosedürü i nin ikili gösterimini bir bit sola kaydırarak $2i$ yi bir komutta hesaplayabilir. Benzer olarak, RIGHT prosedürü $2i + 1$ inin ikiligösterimini bir bit pozisyonunda sola kaydırıp bir alt-sıra bit olarak 1 e ekleyerek hızlı bir şekilde hesaplanabilir. PARENT prosedürü $\lfloor i/2 \rfloor$ yi i yi 1 bit pozisyonunda sağa kaydırarak hesaplayabilir. Bellek yığını sıralamasının iyi bir uygulaması bu prosedürleri makro veya satırıçi(inline) olarak uygular.

İki çeşit ikili bellek yığını vardır, bunlar maks-bellek yığını ve min-bellek yığınıdır. İki türde de düğümler içindeki değerler bellek yığını türüne göre çeşitlilik gösteren bellek yığını özelliğini karşılar. **Maks-bellek yığınının maks-bellek yığını özelliği** kök dışında her i düğüm için

$$A[\text{PARENT}(i)] \geq A[i]$$

dir, yani düğümün değeri en fazla ebeveyninin değeri kadardır. Böylece ağaçtaki en büyük değer kökündür ve bir düğümden türeyen bir altağacın değeri kendi dü-

ğümünde bulunandan daha büyük değildir. **Min-bellek yığını** ise bunun tam tersi şekilde organize edilmiştir. **Min-bellek yığını özelliği** kök dışında her düğüm i düğümü için

$$A[\text{PARENT}(i)] \leq A[i]$$

dir. Min-bellek yığınınındaki en küçük eleman köktür.

Bellek yığını sıralama algoritması için maks-bellek yığını kullanmaktayız. Min-bellek yığınları genelde bölüm 6.5. de gösterilen öncelikli kuyruklar için kullanılır. Herhangi bir uygulama için maks-bellek yığınına mı min-bellek yığınına mı ihtiyaç duyduğumuzu belirlerken dikkatli olmalıyız ve bir özellik maks-bellek yığınının veya min-bellek yığınının her ikisine uygulanırken sadece bellek yığını terimini kullanırız.

Bellek yığını bir ağaç gibi görerek, bir bellek yığınınındaki düğümün **yüksekliğini** düğümün yaprağa aşağı doğru uzanan en uzun basit yoldaki kenar sayısı olarak tanımladık. Ve bunun yanında bellek yığınının yüksekliğini kendi kökünün yüksekliği olarak tanımladık. n elemandan oluşan bir düğüm tam bir ikili ağaca dayandığından dolayı bellek yığınının yüksekliği $\Theta(\lg n)$ dir. (Alıştırma 6.1-2 yi görün). Bellek yığınınındaki basit işlemlerin ağacın yüksekliğinin en çok orantılı olduğu zamanda çalıştığını görebiliriz ve bu nedenle işlemler $O(\lg n)$ zamanda çalışır. Bu bölümün geri kalan kısmı bazı basit işlemler sunar ve sıralama algoritmalarında ve öncelikli-kuyruk veri yapısında nasıl kullanıldıklarını gösterir.

- $O(\lg n)$ sürede çalışan MAX-HEAPIFY prosedürü maks-bellek yığını özelliğinin sağlanmasında anahtar niteliğindedir.
- Doğrusal zamanda çalışan BUILT-MAX-HEAP prosedürü sıralanmamış girdi dizisinden maks-bellek yığını üretir.
- $O(n \lg n)$ zamanda çalışan HEAPSORT prosedürü bir diziyi yerinde sıralar.
- $O(\lg n)$ zamanda çalışan MAX-HEAP-INSERT, HEAP-EXTRACT-MAX, HEAP-INCREASE-KEY ve HEAP-MAXIMUM prosedürleri bellek yığını veri yapısının öncelikli kuyruk uygulamasına izin verir.

Alıştırmalar

6.1-1

Yüksekliği h olan bir bellek yığınının en büyük ve en küçük eleman sayısı nedir?

6.1-2

n elemanlı bir bellek yığınının yüksekliğinin $\lfloor \lg n \rfloor$ olduğunu gösterin.

6.1-3

Bir maks-bellek yığınının herhangi altağacında, altağacın kökünün bu ağacın herhangi yerinde meydana gelen en büyük sayıyı içerdiğini gösterin.

6.1-4

Her elemanın farklı bir sayı olduğunu varsayarak maks-bellek yığınındaki en küçük sayı nerede bulunur?

6.1-5

Sıralanmış biçimdeki bir dizi min-bellek yığını mıdır?

6.1-6

$\langle 23, 17, 14, 6, 13, 10, 1, 5, 7, 12 \rangle$ değerlerini bulunduran bir dizi maks-bellek yığını mıdır?

6.1-7

n elemanlı bir bellek yığını depolaması için dizi gösterimini kullanarak, yaprakların, $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \dots, n$ ile indislenen düğümler olduğunu gösteriniz."

6.2 Bellek yığını özelliğini sağlama

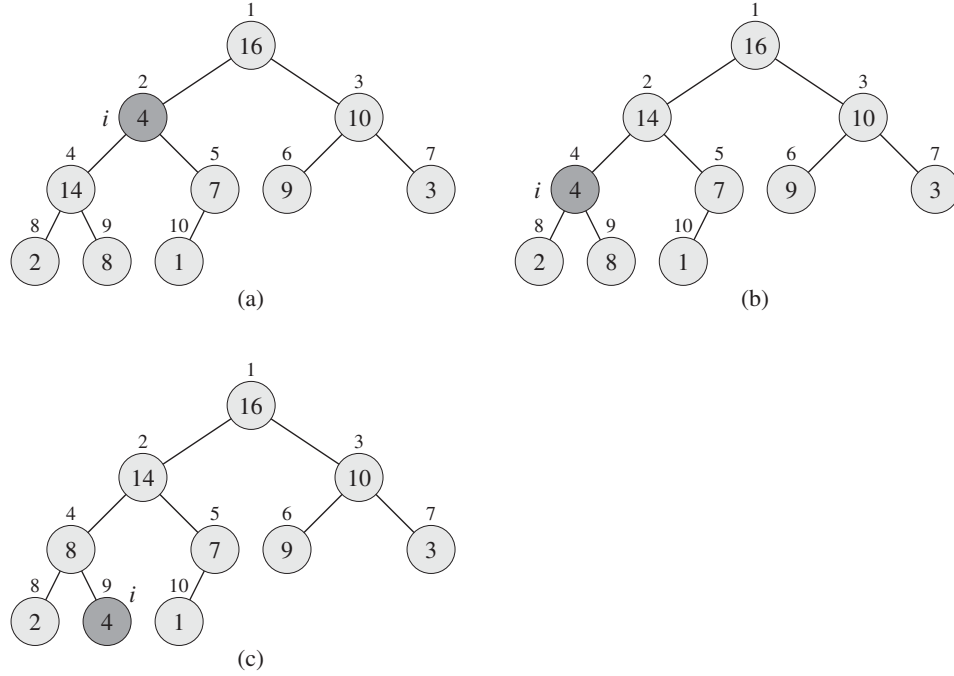
Maks-bellek yığını özelliğini sağlamak için MAX-HEAPIFY prosedürünü çağırırız. Girdiler bir dizi ve bu diziye atanan i değişkenidir. MAX-HEAPIFY çağrıldığı zaman, LEFT(i) de ve RIGHT(i)de köklenen ikili ağacın maks-bellek yığını olduğunu varsayar ancak $A[i]$ çocuğundan daha küçük olabilir; bu nedenle maks-bellek yığını özelliğini ihlal eder. MAX-HEAPIFY $A[i]$ deki değerini aşağı inmesini sağlayarak i indisinde köklenmiş altağacın maks-bellek yığını özelliğini korumasını sağlar.

MAX-HEAPIFY(A, i)

```

1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  ve  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  ve  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9       $A[i]$  yi  $A[\text{largest}]$  ile değiştir
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

Şekil 6.2. max-yığınlamanın özelliğini örnekler. Her aşamada $A[i]$ $A[\text{LEFT}(i)]$ ve $A[\text{RIGHT}(i)]$ elemanlarının en büyüğüne karar verilir ve indisi largest de saklanır. Eğer $A[i]$ en büyükse, o zaman, i düğümünde köklenmiş altağaç halihazırda bir maks-bellek yığınıdır ve prosedür sonlanır. Aksi halde iki çocuktan biri en büyük değere sahiptir ve $A[i]$, $A[\text{largest}]$ ile değiştirilerek i düğümü ve çocukları-



Şekil 6.2 A. $heap-size = 10$ olan MAX-HEAPIFY nin hareketi. **(a)** Başlangıç düzeni, yığın $i = 2$ deki $A[2]$ iki çocuğundan da daha büyük olmadığı için max-yığın özelliğini ihlal eder. Bellek yığını 2 için maks-bellek yığını özelliği. **(b)** de $A[2]$ ile bellek yığını 4 için max-bellek yığını özelliğini yok eden $A[4]$ ün yerlerini değiştirerek yeniden sağlanır. Şimdi özyinelemeli çağrı MAX-HEAPIFY (A.4) $i = 4$ e sahiptir. **(c)**de gösterildiği gibi $A[4]$ ile $A[9]$ yer değiştirerek bellek yığını 4 düzeltilir ve özyinelemeli çağrı MAX-HEAPIFY (A.9) veri yapısında daha fazla değişikliğe sebep olmaz.

nın maks-bellek yığını özelliğini sağlamasına neden olur. Bellek yığını en büyük tarafından indislenir yine de şimdi asıl değer $A[i]$ ye sahiptir. Böylece en büyüğe köklü altağaç maks-bellek yığını özelliğini ihlal edebilir. Sonuç olarak MAX-HEAPIFY yi altağaç üzerinde olarak çağırırız.

Verilen bir i düğümünde köklenmiş n boyutlu bir altağaç üzerinde MAX-HEAPIFY nin çalışma süresi $A[i]$, $A[\text{LEFT}(i)]$ ve $A[\text{RIGHT}(i)]$ elemanları arasındaki ilişkileri sağlayan $\Theta(1)$ süredir, ayrıca bu zaman i düğümünün çocuklarından birinde köklenmiş altağaç üzerinde MAX-HEAPIFY yi çalıştırmak içindir (özyinelemeli çağrının gerçekleştiğini varsayarak). Çocuklarının altağaçlarının her biri en fazla $2n/3$ boyutuna sahiptir—ağacın alt seviyesi tam olarak yarı dolu olduğunda en kötü-durum çalışma süresi meydana gelir ve bu nedenle MAX-HEAPIFY nin çalışma zamanını yenileyerek

$$T(n) \leq T(2n/3) + \Theta(1) \text{ olarak tanımlarız.}$$

Bu özyinelemeye çözüm, ana teoreminin ikinci durumu (Teorem 4.1), $T(n) = O(\lg n)$ dir. Alternatif olarak yüksekliği h olan bir a düğümünde MAX-HEAPIFY nin çalışma zamanını $O(h)$ olarak karakterize ederiz.

Alıştırmalar

6.2-1

Şekil 6.2 yi bir model olarak kullanarak $A = \langle 27, 17, 3, 16, 13, 10, 1, 5, 7, 12, 4, 8, 9, 0 \rangle$ dizisinde MAX-HEAPIFY($A, 3$) işlemini örnekleyiniz.

6.2-2

MAX-HEAPIFY prosedüründen başlayarak, min-bellek yığınındaki uygun manipülasyonu yerine getiren MIN-HEAPIFY(A, i) prosedürü için bir yazınız. MIN-HEAPIFY nin çalışma süresi MAX-HEAPIFY ninkile karşılaştırıldığında nasıldır?

6.2-3

$A[i]$ elemanı, çocuğundan daha büyük olduğu zaman MAX-HEAPIFY(A, i) yi çağırmanın etkisi nedir?

6.2-4

$i > A.heap-size = 2$ için MAX-HEAPIFY(A, i) nin etkisi nedir?

6.2-5

MAX-HEAPIFY nin birkodu muhtemelen 10. satırda bazı derleyicilerin verimsiz kod oluşturmaya neden olan özyinelemeli çağrı hariç sabit faktörler bazında oldukça verimlidir. Özyinelemeli yerine iteratif kontrol yapısı (döngü) kullanan etkin bir MAX-HEAPIFY yazınız.

6.2-6

Boyutu n olan bir bellek yığınında MAX-HEAPIFY nin en kötü-durum çalışma süresinin $\Omega(\lg n)$ olduğunu gösteriniz. (*İpucu:* n düğüme sahip bir bellek yığını için MAX-HEAPIFY nin bir yapraktan aşağı basit bir yol üzerinde her düğümün özyinelemeli çağrılmasına sebep olan düğüm değerini verir.)

6.3 Bir bellek yığını oluşturma

MAX-HEAPIFY prosedürü $n = A.length$ olan bir $A[1..n]$ dizisinden bir maks-bellek yığınına aşağıdan yukarı bir şekilde çevirmek için kullanabiliriz. Alıştırma 6.1-7 de altdizideki bütün $A[(\lfloor n/2 \rfloor + 1) .. n]$ elemanların tümü ağacın yaprak-

larıdır ve her biri 1-eleman ile başlayan bir bellek yığınıdır. BUILD-MAX-HEAP prosedürü ağacın geri kalan düğümleri üzerinden gider ve her birinde MAX-HEAPIFY yi çalıştırır.

BUILD-MAX-HEAP(A)

```

1   $A.heap-size = A.length$ 
2  for  $i = \lfloor A.length/2 \rfloor$  downto 1
3      MAX-HEAPIFY( $A, i$ )
```

Şekil 6.3 BUILD-MAX-HEAP in hareketinin örneğini göstermektedir.

BUILD-MAX-HEAP in neden doğru bir şekilde çalıştığını göstermek için sıradaki döngü değişmezini kullandık:

2. ve 3. satırdaki **for** döngüsünün her iterasyonunda, her $i + 1, i + 2, \dots, n$ düğümü maks-bellek yığınının köküdür.

Döngünün her bir iterasyonun değişmezliğini koruduğu önceliklerde ilk döngü için bu değişmezliğin korunduğunu göstermeye ihtiyaç vardır ve bu değişmezlik döngü sonlandığı zaman için uygunluğu göstermede kullanışlı bir araçtır.

Başlangıç: Döngünün ilk iterasyonundan önce $i = \lfloor n/2 \rfloor$. Her düğüm $\lfloor n/2 \rfloor + 1, \lfloor n/2 \rfloor + 2, \dots, n$ düğümü bir yapraktır ve bu nedenle basit bir maks-bellek yığınının köküdür.

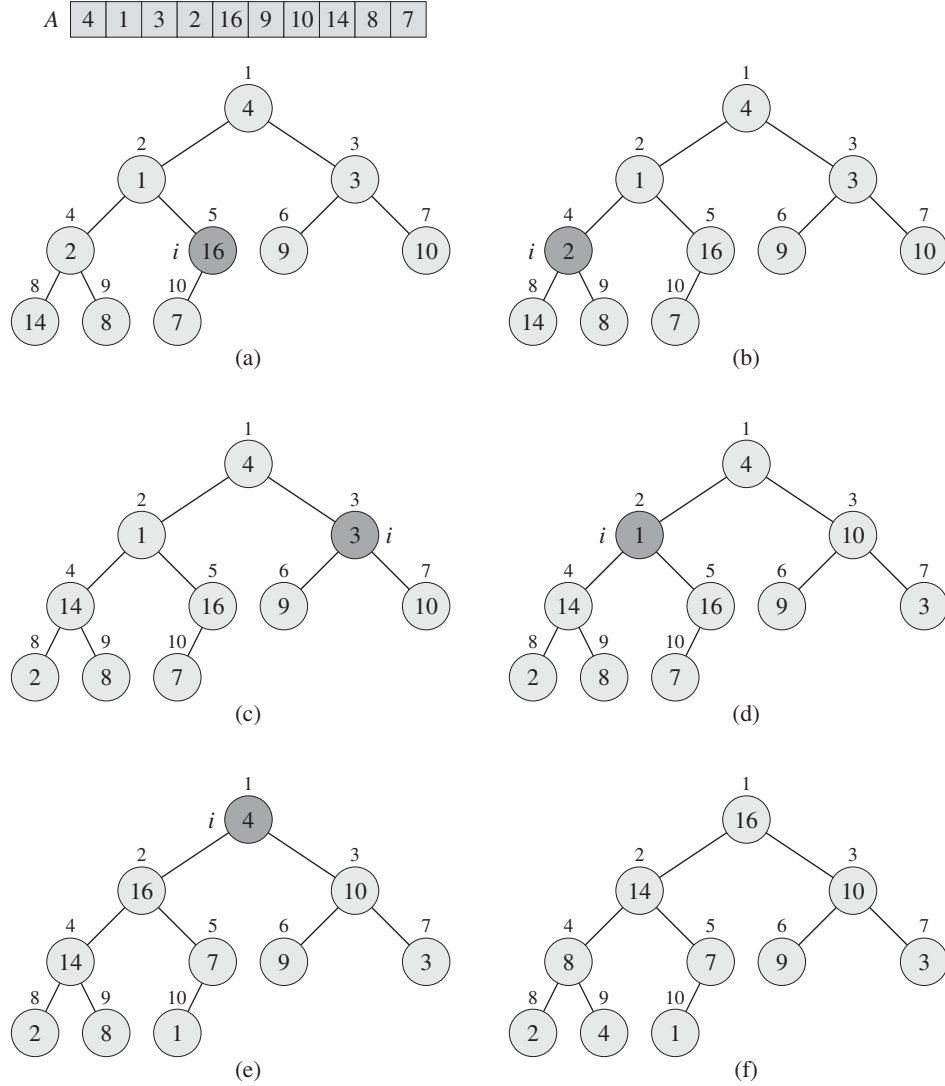
Yineleme: Her iterasyonun döngü değişmezini yinelemesini sağladığını görmek için i düğümünün çocuklarının i den daha büyük numaralandırıldığını gözlemliyoruz. Döngü değişmezi nedeniyle ikisi de maks-bellek yığınının köküdür. Bu tam olarak, i düğümünü max-düğüm kökü yapmak adına MAX-HEAPIFY çağrısı için gereken koşuldur. Dahası MAX-HEAPIFY çağrısı $i + 1, i + 2, \dots, n$ düğümlerinin hepsinin maks-bellek yığınlarının kökü olması özelliğinin korunmasını sağlar. i yi **for** döngüsünde azaltmak, döngü değişmezini bir sonraki iterasyon için yeniden sağlar.

Sonlandırma: Sonlandırmada $i = 0$ dır. Döngü değişmeziyle her $1, 2, \dots, n$ düğümü maks-bellek yığınının bir köküdür. Pratikte bu düğüm 1 dir.

BUILD-MAX-HEAP çalışma zamanının üst sınırını aşağıdaki gibi hesaplayabiliriz. MAX-HEAPIFY a her çağrı $O(\lg n)$ zamana mal olur ve BUILD-MAX-HEAP $O(n)$ zamanlı çağrı yapar. Böylece çalışma zamanı $O(n \lg n)$ olur. Bu üst sınır doğru olsa da asimptotik olarak sıkı değildir.

MAX-HEAPIFY in ağaçtaki düğümün yüksekliğine göre çeşitlilik gösteren düğümde çalıştırmak için gereken zamanı gözlemleyerek daha sıkı bir üst sınır çizebiliriz ve birçok düğümün yüksekliği küçüktür. Daha sıkı analizimiz bir n elemanlı düğümün $\lfloor \lg n \rfloor$ (Alıştırma 6.1-2) ve en fazla $\lceil n/2^{h+1} \rceil$ olan düğümün herhangi bir yüksekliğinin h (Alıştırma 6.3-3 ü görün) olması özelliğine dayanır.

MAX-HEAPIFY in herhangi bir h yüksekliğinde çağrıldığı zaman gerekli süre $O(h)$ dir ve MAX-HEAP in toplam maliyetini yukarıdan sınırlandırıldığı gibi açıklayabiliriz



Şekil 6.3 BUILD-MAX-HEAP in 3. satırında MAX-HEAPIFY ı çağırmadan önceki veri yapısını gösteren BUILD-MAX-HEAP işlemi. **(a)** 10 elamanlı girdi dizisi A ve temsil ettiği ikili ağaç. Şekil döngü indisi i nin MAX-HEAPIFY çağırısından önce düğüm 5 i kastedtiğini gösterir. **(b)** Sonuç veren veri yapısı Döngü indisi i bir sonraki iterasyon için düğüm 4 ü kasteder. **(c)-(e)** BUILD-MAX-HEAP teki for döngüsünün takip eden iterasyonları. MAX-HEAPIFY bir düğümde ne zaman çağırılsa bu düğümün iki alt ağacının da maks-bellek yığını olduğunu gözlemleriz. **(f)** BUILD-MAX-HEAP dan sonraki maks-bellek yığını sonlanır.

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right).$$

Son toplamı $x = 1/2$ de formülde(A.8) yerine koyarak

$$\begin{aligned} \sum_{h=0}^{\infty} \frac{h}{2^h} &= \frac{1/2}{(1 - 1/2)^2} \\ &= 2 \end{aligned}$$

hesaplayabiliriz.

Böylece BUILD-MAX-HEAP in çalışma zamanını aşağıdaki gibi sınırlandırabiliriz.

$$\begin{aligned} O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h}\right) &= O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) \\ &= O(n). \end{aligned}$$

Bundan dolayı doğrusal zamanda düzensiz bir diziyi maks-bellek yığınına çevirebiliriz.

BUILD-MAX-HEAP ile aynı ancak 3. satırda MAX-HEAPIFY yerine MIN-HEAPIFY ile değiştirilmiş BUILD-MIN-HEAP prosedürü ile min-bellek yığını inşa edebiliriz. BUILD-MIN-HEAP doğrusal zamanda düzensiz bir diziden min-bellek yığını üretilir.

Alıştırmalar

6.3-1

Bir model olarak Şekil 6.3 ü kullanarak $A = \langle 5, 3, 17, 10, 84, 19, 6, 22, 9 \rangle$ dizisinde BUILD-MAX-HEAP işlemini gösteriniz.

6.3-2

BUILD-MIN-HEAP in ikinci satırındaki döngünün i indisinin 1 den $\lfloor A.length/2 \rfloor$ ye artışı yerine $\lfloor A.length/2 \rfloor$ ye arttırmamızın nedeni nedir?

6.3-3

Herhangi bir n elemanlı bellek yığnında h yükseklikli en fazla $\lceil n/2^{h+1} \rceil$ düğümün olduğunu gösteriniz.

6.4 Bellek yığını sıralama algoritması

Bellek yığını sıralama algoritması, $n = A.length$ olduğu bir girdi dizisi üzerinde maks-bellek yığını inşa etmek için BUILD-MAX-HEAP kullanarak başlar. Dizinin en büyük elemanı $A[1]$ kökünde depolandığından dolayı bunu $A[1..n]$ yer

değiştirerek doğru son pozisyonuna koyabiliriz. Eğer şimdi n düğümünü bellek yığınının çıkarırsak—ve bunu basitçe $A.heap-size$ azaltarak yapabiliriz — kökün çocuğunun maks-bellek yığını kaldığını gözlemleyiz; ancak yeni kök elemanı maks-bellek yığını özelliğini ihlal edebilir. Tüm yapmamız gereken maks-bellek yığını özelliğini tekrar sağlamak; ancak bu $A[1..n-1]$ içinde maks-bellek yığını bırakan MAX-HEAPIFY($A, 1$) i çağırır. Daha sonra bellek yığını sıralama algoritması bu prosedürü boyutu $n-1$ den bellek yığını boyutu 2 ye indirmek için tekrarlar. (Tam bir döngü değişmezi için Alıştırma 6.4-2 ye bakınız.)

HEAPSORT(A)

```

1  BUILD-MAX-HEAP( $A$ )
2  for  $i = A.length$  downto 2
3       $A[1]$  ile  $A[i]$  değiştir
4       $A.heap-size = A.heap-size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```

Şekil 6.4 satır 1 ilk maks-bellek yığını inşa ettikten sonraki HEAPSORT işleminin örneğini gösterir. Şekilde 2-5 satırları arasındaki **for** döngüsünün ilk iterasyonundan önce maks-bellek yığını gösterir.

BUILD-MAX-HEAP çağırısı $O(n)$ zaman aldığından ve MAX-HEAPIFY a her bir $n-1$ çağırısı $O(\lg n)$ zaman aldığından; HEAPSORT prosedürü $O(n \lg n)$ kadar zaman alır.

Alıştırmalar

6.4-1

Şekil 6.4 ü model olarak kullanarak, $A = \langle 5, 13, 2, 25, 7, 17, 20, 8, 4 \rangle$ dizisinde HEAPSORT işlemini gösteriniz.

6.4-2

Aşağıdaki döngü değişmezini kullanarak HEAPSORT un doğruluğunu tartışınız:

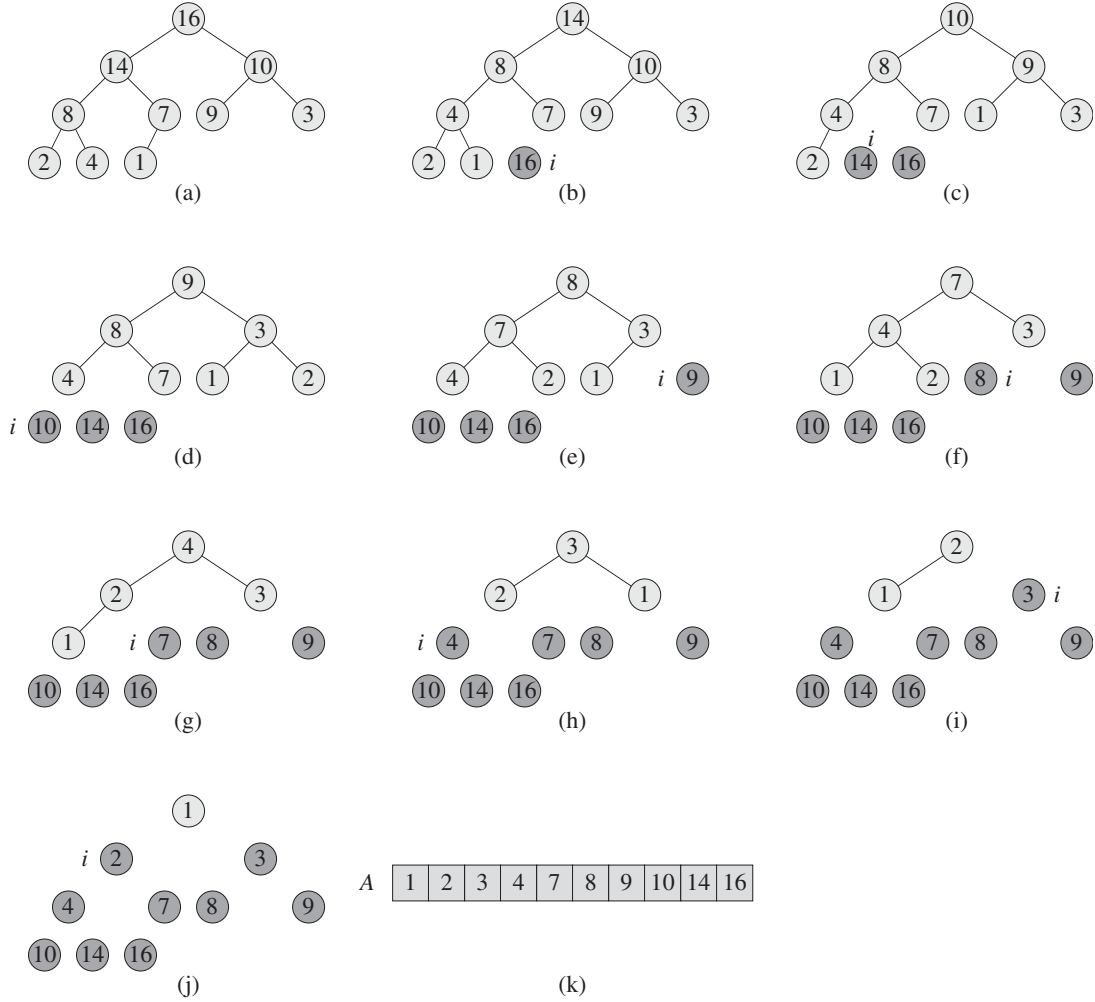
2. ve 5. satırlar arasındaki **for** döngüsünün her iterasyonun başlangıcında altdizi $A[1..n]$ i nin $A[1..n]$ nin en küçük elemanı olduğu maks-bellek yığınıdır ve $A[i+1..n]$ alt dizisi $A[1..i]$ dizisinin $n-i$ inci en büyük elemanını içerir, sıralıdır.

6.4-3

Hali hazırda artan bir şekilde sıralı olan n uzunluktaki bir A dizisinde HEAPSORT un çalışma süresi nedir?

6.4-4

HEAPSORT un en kötü-durum çalışma süresinin $\Omega(n \lg n)$ olduğunu gösteriniz.



Şekil 6.4 Bellek yığını sıralama algoritması. (a) BUILD-MAX-HEAP satır 1 de inşa ettikten hemen sonra maks-bellek yığını veri yapısı. (b)-(j) 5.satırdaki MAX-HEAPIFY in her çağrısından hemen sonra max-yığın i nin değerini gösterirken. Sadece açık gri düğümler bellek yığını içinde kalır. (k) Sıralanmış olarak kalan A dizisi.

6.4-5 ★

Her elemanı farklı olduğunda HEAPSORT un en iyi-durum çalışma zamanının $\Omega(n \lg n)$ olduğunu gösteriniz.

6.5 Öncelikli kuyruklar

Bellek yığını sıralaması çok iyi bir algoritmadır, fakat Bölüm 7 de sunulan hızlı sıralama algoritmasının iyi bir uygulaması pratikte bellek yığını sıralamasından daha iyidir. Yine de bellek yığını sıralamasının bir çok kullanımı bulunmaktadır. Bu bölümde bellek yığınının en popüler uygulamalarından verimli bir öncelikli kuyruğu sunmaktayız. Bellek yığınlarında olduğu gibi öncelikli kuyruğun iki çeşidi vardır: Maks-öncelikli kuyruklar ve min-öncelikli kuyruklar. Burada maks-bellek yığınlarının sıra tabanlı hali olan maks-öncelikli kuyruklar üzerinde duracağız. Alıştırma 6.5-3 min-öncelikli kuyruklar için prosedür yazmanızı istemektedir.

Bir **öncelikli kuyruk** her biri *anahtar* adı verilen bir değerle bağdaştırılmış bir S kümesinin elemanlarını sürdüren bir veri yapısıdır. Bir **maks-öncelikli kuyruk** aşağıdaki işlemleri destekler:

INSERT (S, x), x elemanını S kümesine ekler, $S = S \cup \{x\}$ işlemine eşittir.

MAXIMUM (S), S kümesinin en büyük anahtarlı elemanını döndürür.

EXTRACT-MAX (S), S kümesinin en büyük elemanını kümeden kaldırıp, döndürür.

INCREASE-KEY (S, x, k), en az x in şu anki anahtar değeri kadar büyük olduğu varsayılan x elemanının anahtarını yeni k değerine arttırır.

Diğer uygulamaları arasında, paylaşılan bir bilgisayarda işleri çizelgelemek için maks-öncelikli kuyrukları kullanabiliriz. Maks-öncelikli kuyruklar yapılacak işleri ve göreceli önceliklerini takip eder. Bir iş bittiğinde veya yarıda kesildiğinde çizelgeleyici bekleyen işlerden en büyük öncelikli olanı EXTRACT-MAX çağrısıyla seçer. Çizelgeleyici INSERT çağrısı ile herhangi bir zamanda kuyruğa yeni bir iş ekleyebilir.

Alternatif olarak **min-öncelikli kuyruk** INSERT, MINIMUM, EXTRACT-MIN, ve DECREASE-KEY. işlemlerini destekler. Min-öncelikli kuyruk olay güdümlü simülatörlerde kullanılabilir. Kuyruktaki parçalar her biri bir anahtar niteliği gören oluşma zamanı ile bağdaştırılmış simüle edilecek olaylardır. Olaylar oluşma zamanlarına göre simüle edilmelidirler; çünkü olayın simülasyonu ilerde başka olaylara neden olabilir. Simülasyon programı simüle edilecek sıradaki olayı seçmek için her adımda EXTRACT-MIN ı çağırırlar. Yeni olaylar oluştuğunda simülâtör bu olayları min-öncelikli kuyruğa INSERT çağrısıyla eklenir. Min-öncelikli

kuyruklarla ilgili DECREASE-KEY işlemi üzerinde duran konuları Bölüm 23 ve 24 de göreceğiz.

Şaşırtıcı olmayan bir şekilde bellek yığınlarını öncelikli kuyrukları uygulamak için kullanabiliriz. İş çizelgeleme veya olay güdümlü simülasyon gibi verilen uygulamalarda, öncelikli kuyruk içinde bulunan elemanlar uygulamadaki objelere karşılık gelir. Genelde hangi objelerin verilen öncelikli kuyrukta hangi elemana karşılık geleceğine karar vermeye ihtiyaç duyarız ya da tam tersini. Öncelikli kuyruğu uygulamak için bellek yığını kullandığımız zaman her bellek yığını elemanı içinde uygulama objesine karşılık gelen bir *kola* (handle) ihtiyaç duyarız. Kolun tam özyapısı (işaretçi (pointer) veya bir sayma sayısı) uygulamaya bağlıdır. Benzer bir şekilde her bir uygulama objesi içinde bellek yığını elemanına karşılık gelen bir kol saklamamız gerekir. Burada kol tipik olarak bir dizi indisidir. Bellek yığını elemanları bellek yığını işlemleri sırasında dizi içinde yer değiştirdiklerinden dolayı yığın elemanını yeniden yerleştirme üzerine gerçek bir uygulamanın, uygun uygulama objesinde dizi indisini yenilemesi zorunludur. Uygulama objesine ulaşmanın detayları büyük ölçüde uygulamaya ve programlamasına bağlı olduğundan dolayı onları burada pratik dışında devam ettiremeyiz ve bu kollar doğru bir şekilde sağlanmalıdır.

Şimdi bir maks-öncelikli kuyruğun nasıl uygulanacağını tartışacağız. HEAP-MAXIMUM prosedürü MAXIMUM işlemi $\Theta(1)$ zamanında uygular.

HEAP-MAXIMUM(A)

1 **return** $A[1]$

HEAP-EXTRACT-MAX prosedürü EXTRACT-MAX işlemi uygular. Bu HEAP-SORT prosedürünün **for** döngüsüne (satır 3-5 arası) benzerdir.

HEAP-EXTRACT-MAX(A)

```

1 if  $A.heap-size < 1$ 
2   error “bellek yığını aşağı taşması”
3  $max = A[1]$ 
4  $A[1] = A[A.heap-size]$ 
5  $A.heap-size = A.heap-size - 1$ 
6 MAX-HEAPIFY( $A, 1$ )
7 return  $max$ 
```

HEAP-INCREASE-KEY için $O(\lg n)$ zamanın tepesinde sadece sabit miktarda iş yaptığı için çalışma süresi $O(n \lg n)$ dir.

HEAP-INCREASE-KEY prosedürü INCREASE-KEY işlemi uygular. Dizi içindeki bir i indisi arttırmak istediğimiz öncelikli kuyruk elemanını tanımlar. Prosedür öncelikle eleman $A[i]$ nin değerini yeni değerine günceller. Çünkü $A[i]$ nin anahtarını arttırmak maks-bellek yığını özelliğini ihlal edebilir, daha sonra prose-

dür Kısım 2.1 den HEAP-INCREASE-KEY in araya sokma döngüsünün hatırlatıcı davranışında yeni arttırılmış anahtara yer bulmak için bu düğümden köke doğru basit bir yolu dolaşır. HEAP-INCREASE-KEY bu yolu dolaştığında tekrar eden bir şekilde bir elemanın ebeveyniyle karşılaştırır, anahtarlarını birbirleriyle değiştirir, eğer elemanın anahtarı daha büyükse işleme devam eder ve maks-bellek yığını özelliği sağlandığından eğer elemanın anahtarı küçükse işlemi bitirir.

HEAP-INCREASE-KEY(A, i, key)

```

1  if  $key < A[i]$ 
2      error “yeni anahtar şu anki anahtardan küçüktür”
3   $A[i] = key$ 
4  while  $i > 1$  and  $A[PARENT(i)] < A[i]$ 
5       $A[i]$  ile  $A[PARENT(i)]$  yi değiştir
6       $i = PARENT(i)$ 

```

Şekil 6.5 HEAP-INCREASE-KEY işleminin bir örneğini gösterir. 3. satırda güncellenen düğümden köke kadar takip edildiğinde yolun çalışma süresi $O(\lg n)$ olduğundan eleman sayısı n olan bir bellek yığnında HEAP-INCREASE-KEY çağrısının çalışma süresi $O(\lg n)$ dir.

MAX-HEAP-INSERT prosedürü INSERT işlemini uygular. Bu prosedür maks-bellek yığnına eklenecek yeni elemanın anahtarını girdi olarak alır. Prosedür ilk önce ağaca anahtarı $-\infty$ olan bir yaprak ekleyerek maks-bellek yığnını genişletir. Sonra bu yeni düğümün anahtarını gerçek değerine atamak ve maks-bellek yığını özelliğini korumak için MAX-HEAP-INSERT prosedürünü çağırır.

MAX-HEAP-INSERT(A, key)

```

1   $A.heap-size = A.heap-size + 1$ 
2   $A[A.heap-size] = -\infty$ 
3  HEAP-INCREASE-KEY( $A, A.heap-size, key$ )

```

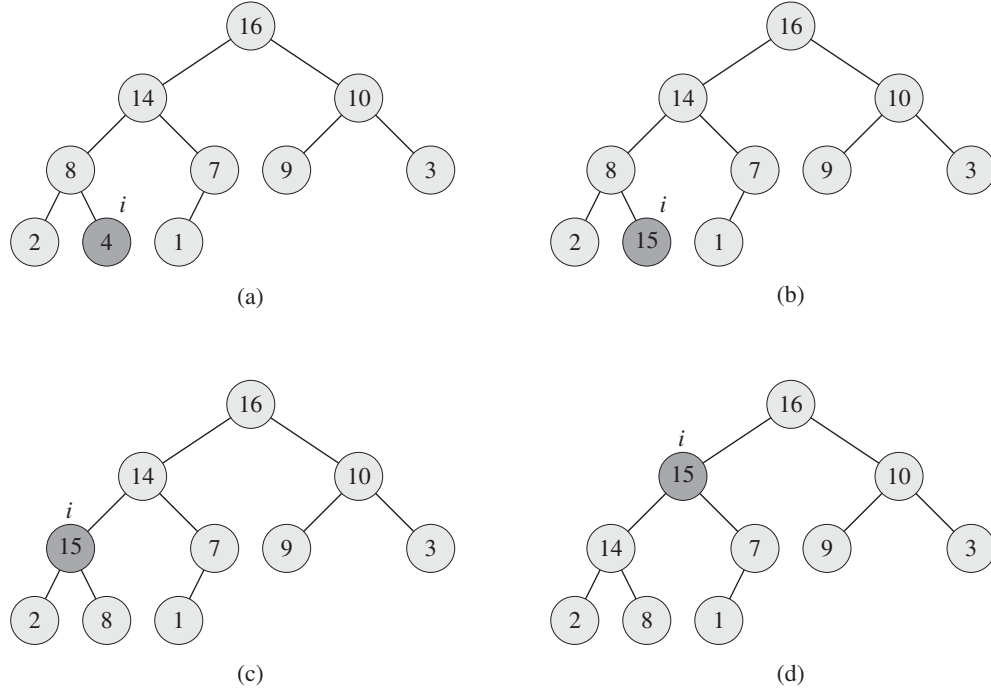
Eleman sayısı n olan bellek yığnında MAX-HEAP-INSERT prosedürünün çalışma süresi $O(\lg n)$ dir.

Özetle, bir bellek yığını $O(\lg n)$ sürede n boyutlu bir küme üzerindeki herhangi bir öncelikli-kuyruk işlemini destekleyebilir.

Alıştırmalar

6.5-1

HEAP-EXTRACT-MAX işlemini $A = \langle 15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$ bellek yığnında gösteriniz.



Şekil 6.5 HEAP-INCREASE-KEY işlemi. **(a)** Şekil 6.4 te koyu gri renkte indisi i olan bir düğüm ile maks-bellek yığını. **(b)** Bu düğümün anahtarı 15 e yükseltilmiştir. **(c)** 4-6 satırlardaki **while** döngüsünün bir iterasyonundan sonra düğüm ve ebeveyni anahtarlarını değiştirmişlerdir ve i indisi ebeveyne doğru ilerlemiştir. **(d)** **while** döngüsünün birden fazla iterasyonundan sonra maks-bellek yığını. Bu noktada $A[\text{PARENT}(i)] \geq [i]$ dir. Maks-bellek yığını özelliği şimdi sağlanır ve işlem sonlandırılır.

6.5-2

MAX-HEAP-INSERT işlemini $A = \langle 15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1 \rangle$ bellek yığnında gösteriniz.

6.5-3

Min-bellek yığını ile min-öncelikli kuyruğu uygulayan HEAP-MINIMUM, HEAP-EXTRACT-MIN, HEAP-DECREASE-KEY ve MIN-HEAP-INSERT prosedürleri için kabakod yazınız.

6.5-4

Yaptığımız bir sonraki adım anahtar değerini istediğimiz bir derece arttırdığında MAX-HEAP-INSERT işleminin 2. satırında eklenen düğümün anahtarının $-\infty$ olarak ayarlanması neden rahatsız eder?

6.5-5

Aşağıdaki döngü değişmezini kullanarak HEAP-INCREASE-KEY in doğruluğunu tartışınız:

4-6 satırlar arasındaki **while** döngüsünün her iterasyonun başında, $A[\text{PARENT}(i)] = A[\text{LEFT}(i)]$ ve $A[\text{PARENT}(i)] = A[\text{RIGHT}(i)]$, eğer bu düğümler $A[i]$ $A[\text{PARENT}(i)]$ den daha büyük olabilmesi ihlali dışında altdizi $A[1 \dots A.\text{heap-size}]$ maks-bellek yığını özelliğini sağlar.

Altdizi $A[1 \dots A.\text{heap-size}]$ maks-bellek yığını özelliğini HEAP-INCREASE-KEY çağrıldığında sağlar.

6.5-6

HEAP-INCREASE-KEY in 5. satırındaki her değiştirme işlemi üç atama işlemi gerektirmektedir. Üç atama işlemini bir atama işlemine indirmek için araya sokma sıralamasının içsel döngü fikrinin nasıl kullanıldığını gösteriniz.

6.5-7

İlk giren ilk çıkan kuyruk öncelikli kuyrukla nasıl uygulanır gösteriniz. Öncelikli kuyrukla bir yığın nasıl uygulanır gösteriniz (Kuyruklar ve yığınlar Bölüm 10 da anlatılmıştır).

6.5-8

HEAP-DELETE (A, i) işlemi i düğümündeki elemanı bir bellek yığınından siler. n elemanlı bir maks-bellek yığını için $O(\lg n)$ de çalışan bir HEAP-DELETE uygulamasını gösteriniz.

6.5-9

Tüm girdi listelerinde n nin toplam eleman sayısı olduğu k adet sıralı listeyi $O(n \lg k)$ zamanda tek bir listede birleştiren bir algoritma yazınız (*İpucu: k-adetli birleştirme için min-bellek yığını kullanınız*).

Problemler
6-1 Araya sokma kullanarak bir bellek yığını oluşturma

Bir bellek yığınına elemanlar eklemek için MAX-HEAP-INSERT i tekrarlı bir biçimde çağırarak bir bellek yığını inşa edebiliriz. BUILD-MAX-HEAP prosedüründe varyasyonu dikkate alınız:

BUILD-MAX-HEAP' (A)

```

1   $A.heap-size = 1$ 
2  for  $i = 2$  to  $A.length$ 
3      MAX-HEAP-INSERT( $A, A[i]$ )

```

- a. Aynı girdi dizisi üzerinde çalıştırılırken BUILD-MAX-HEAP ve BUILD-MAX-HEAP' prosedürleri her zaman aynı bellek yığını yaratırlar mı?
- b. En kötü-durum çalışma zamanında n elemanlı bir bellek yığını oluşturmak için BUILD-MAX-HEAP prosedürünün $\Theta(n \lg n)$ zaman gerektirdiğini gösteriniz.

6-2 d -li bellek yığınının analizi

Bir d -li bellek yığını ikili bellek yığınına benzerdir ancak (tek bir istisna dışında) yaprak olmayan düğümler 2 çocuk yerine d çocuğa sahiptir.

- a. Bir d -li bellek yığını bir dizide nasıl gösterirdiniz?
- b. n elemanı olan bir d -li bellek yığınının d ve n türünden yüksekliği nedir?
- c. d -li maks-bellek yığınının EXTRACT-MAX ın verimli bir uygulamasını veriniz. Çalışma zamanı n ve d türünden analiz edin.
- d. d -li maks-bellek yığınının INSERT in verimli bir uygulamasını veriniz. Çalışma zamanı n ve d türünden analiz edin.
- e. Eğer $k < A[i]$ ise bir uyarı veren ama diğer türlü $A[i] = k$ yapan ve d -li maks-bellek yığını yapısını uygun şekilde güncelleyen, verimli bir INCREASE-KEY uygulamasını yapınız. Çalışma zamanını d ve n cinsinden analiz ediniz.

6-3 Young tabloları

Bir $m \times n$ Young tablosu her satırın girdileri soldan sağa sıralanmış ve her kolondaki girdiler yukarıdan aşağı sıralanmış bir $m \times n$ matrisidir. Young tablosu'nda bazı girdiler ∞ olabilir, bunlar var olmayan elemanlar gibi davranılır. Bu nedenle bir Young tablosu $r \leq m \cdot n$ sınırlı sayı tutmak için kullanılır.

- a. $\{9, 16, 3, 2, 4, 8, 5, 14, 12\}$ elemanlarını içeren 4×4 lük bir Young tablosu çizin.
- b. Eğer $Y[1] = \infty$ ise $m \times n$ lik bir Young tablosu'nun boş olduğunu tartışınız. Eğer $Y[m, n] < \infty$ ise dolu ($m \cdot n$ eleman içerir) olduğunu tartışınız.

- c. Boş olmayan $m \times n$ Young tablosu'nda $O(m + n)$ zamanda çalışan EXTRACT-MIN i uygulamak için bir algoritma veriniz. Algoritmanız özyinelemeli biçimde $(m - 1) \times n$ veya $m \times (n - 1)$ altproblemde $m \times n$ problemini çözen özyinelemeli bir altprogram kullanmalıdır. (*İpucu:* MAX-HEAPIFY i düşünün.) $p = m + n$ olduğu $m \times n$ boyutlu herhangi bir Young tablosu'nda EXTRACT-MIN in çalışma zamanını $T(p)$ yi tanımlayın. $O(m + n)$ zaman sınırlı olan bir $T(p)$ özyinelemesini ver ve çöz.
- d. $O(m + n)$ çalışma zamanı olan bir $m \times n$ boyutlu young tablosu'na yeni bir eleman nasıl eklenir gösteriniz.
- e. Başka bir sıralama altprogramı kullanmadan $n \times n$ boyutlu bir Young tablosu n^2 sayıyı $O(n^3)$ çalışma zamanında nasıl sıralar gösteriniz.
- f. Verilen bir sayı Young tablosu'nda saklandığına karar veren $O(m + n)$ çalışma zamanı olan bir algoritma oluşturunuz.

Bölüm notları

Bellek yığını sıralama algoritması bir bellek yığını ile bir öncelikli kuyruğun nasıl uygulanacağını da tanımlayan Williams [357] tarafından ortaya konmuştur. BUILD-MAX-HEAP prosedürü Floyd[106] tarafından bulunmuştur.

Bölüm 16, 23 ve 24 te min-öncelikli kuyruğu uygulamak için min-bellek yığını kullandık. Ayrıca Bölüm 19 da belirli işlemler için geliştirilmiş zaman sınırı ile bir uygulama verdik ve Bölüm 20 de anahtarların negatif olmayan tam sayıların sınırlandırılmış kümesinden çekildiğini varsaydık.

Eğer veriler b -bitlik tam sayılarsa ve bilgisayar hafızası adreslendirilebilir b -bitlik kelimelerden oluşmuşsa, Fredman ve Willard[115] $O(1)$ zamanda MINIMUM ve $O(\sqrt{\lg n})$ zamanda INSERT ve EXTRACT-MIN i nasıl uygulanacağını göstermişlerdir. Throup[337] $O(\sqrt{\lg n})$ çalışma süresini $O(\lg \lg n)$ ile sınırlandırarak geliştirmiştir. Bu sınır n de sınırlandırılmamış alan kullanmaktadır; rassal çarpılama hashing kullanılarak düzlemsel alanda uygulanabilir.

Öncelikli kuyrukların önemli özel bir durumu EXTRACT-MIN işleminin dizisi **monoton** olduğu zaman oluşur. Bu başarılı bir EXTRACT-MIN işleminde döndürülen değerlerin zaman içinde monotonik olarak artmasıdır. Bu durum Bölüm 24 te tartışılan Dijkstra'nın tek kaynaklı en kısa yol algoritması ve olay tabanlı simülasyon gibi çeşitli örnekleri bulunan uygulamalarda meydana gelir. Dijkstra algoritması için DECREASE-KEY işleminin verimli olarak uygulanması özellikle önemlidir. Monoton durumlar için eğer veriler $1, 2, \dots, C$ aralığında tam sayılarsa Ahuja, Mehlhorn ve Tarjan[8] $O(\lg C)$ amortize zaman (amortize analizi hakkın-

da daha fazla bilgi için Bölüm 17 ye bakınız) zamanda EXTRACT-MIN ve INSERT in ve DECREASE-KEY in $O(1)$ sürede taban bellek yığını olarak adlandırılan bir veri yapısı kullanarak nasıl implement edildiğini tanımlamışlardır. $O(\lg C)$ sınırı $O(\sqrt{\lg C})$ ye taban bellek yığınları ile bağlaç içinde Fibonacci bellek yığınları kullanılarak geliştirilebilir. Cherkassy, Goldberg ve Silverstein [65] Donerdo ve Fox'un [85] çok seviyeli kovalama yapısını ve daha önce belirtilen Throup'un bellek yığınına birleştirerek beklenen çalışma zamanını $O(\lg^{1/3+\epsilon} C)$ sınırına geliştirmişlerdir. Raman[291] herhangi bir belirli $\epsilon > 0$ için çalışma zamanını daha da geliştirerek $O(\min(\lg^{1/4+\epsilon} C, \lg^{1/3+\epsilon} n))$ e taşımıştır.

Hızlı sıralama algoritması eleman sayısı n olan bir girdi dizisinde n^2 en kötü-durum çalışma zamanına sahiptir. Bu yavaş en kötü-durum çalışma zamanına rağmen genelde sıralama için kullanılan en iyi seçimdir. Çünkü ortalamada önemli bir biçimde verimlidir: beklenen çalışma süresi $n \lg n$ ve $n \lg n$ gösteriminde gizli sabit faktörü oldukça küçüktür. Ayrıca yerinde sıralamada da avantajı vardır (sayfa 17 ye bakınız) ve yapay hafıza ortamında bile iyi çalışır.

Kısım 7.1 algoritmayı ve paylaşım için hızlı algoritma tarafından kullanılan altprogramları tanımlar. Hızlı sıralamanın davranışı karmaşık olduğundan Kısım 7.2 de hızlı sıralamanın sezgisel tartışmasıyla başladık ve kesin analizini bölüm sonuna erteledik. Kısım 7.3 te hızlı sıralamanın rassal örnekleme kullanan versiyonunu inceledik. Bu algoritma iyi bir beklenen çalışma zamanına sahiptir ve belirli bir girdi en kötü-durum davranışına neden olur. Kısım 7.4 te rassal algoritmasını analiz etmektedir, en kötü zamanda çalışma süresini $\Theta(n^2)$ olduğunu göstermektedir ve farklı elemanlar için beklenen çalışma zamanının $O(n \lg n)$ olduğunu varsaydık.

7.1 Hızlı sıralamanın tanımı

Hızlı sıralama, birleştirme sıralaması gibi Kısım 2.3.1 de tanıtılmış böl-ve-yönet paradigmasını uygulamaktadır. Tipik bir $A[p..r]$ altdizisi için 3 adımlı böl-ve-yönet işlemi:

Böl: $A[p..r]$ dizisi her bir elemanı $A[q]$ elemanından küçük-eşit olan $A[p..q-1]$ ve her bir elemanı $A[q]$ den büyük-eşit olacak biçimde $A[p..q-1]$ ve $A[q+1..r]$ altdizilerine ayrılır.

Yönet: $A[p..q-1]$ ve $A[q+1..r]$ altdizilerini hızlı sıralamaya özyinelemeli çağrılar yaparak sıralar.

Birleştir: Altdiziler hali hazırda sıralandığından bunları birleştirmek için fazladan bir işe gerek yoktur: Tüm $A[p \dots r]$ dizisi şimdi sıralıdır.

Aşağıdaki prosedür hızlı sıralamayı uygular:

QUICKSORT(A, p, r)

```

1  if  $p < r$ 
2       $q = \text{PARTITION}(A, p, r)$ 
3      QUICKSORT( $A, p, q - 1$ )
4      QUICKSORT( $A, q + 1, r$ )

```

Bütün A dizisini sıralamak için başlangıç çağrısı QUICKSORT($A, 1, A.length$) dır.

Diziyi parçalama

Algoritma anahtar altdizi $A[p \dots r]$ yi yerinde sıralayan PARTITION prosedürüdür.

PARTITION(A, p, r)

```

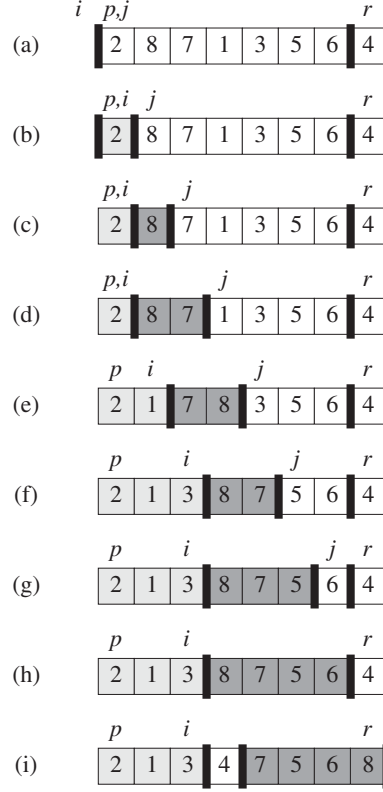
1   $x = A[r]$ 
2   $i = p - 1$ 
3  for  $j = p$  to  $r - 1$ 
4      if  $A[j] \leq x$ 
5           $i = i + 1$ 
6           $A[i]$  ile  $A[j]$  yi değiştir
7   $A[i + 1]$  ile  $A[r]$  yi değiştir
8  return  $i + 1$ 

```

Şekil 7.1 PARTITION un 8 elemanlı bir dizide nasıl çalıştığını gösterir. $A[p \dots r]$ altdizisini ayıran PARTITION prosedürü her zaman bir $x = A[r]$ elemanını **pivot** olarak seçer. Prosedür çalıştığında, diziyi dört bölgeye (boş da olabilir) ayırır. Şekil 7.2 de gösterilen Satır 3-6 daki **for** döngüsünün her iterasyonunun başında, bölgeler belli koşulları sağlar. Bu özellikleri döngü değişmezi olarak belirtiriz:

Satır 3-6 nın döngüsünün her bir iterasyonunun başında, her dizi indisi k için

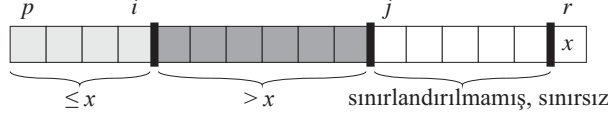
- 1 Eğer $p \leq k \leq i$ ise, o zaman $A[k] \leq x$.
 2. Eğer $i + 1 \leq k \leq j - 1$, ise, ozaman $A[k] > x$.
 3. Eğer $k = r$, ise o zaman $A[k] = x$
- dır.



Şekil 7.1 Bir örnek dizide PARTITION işlemi. Dizi girdisi $A[r]$ pivot elemanı x olur. Açık gri elemanların her biri x den küçük değerlerle ilk parçada bulunur. Koyu gri elemanlar x den büyük değerlerle ikinci parçada bulunur. Beyaz elemanlar daha hiçbir parçaya girmemiş elemanlardır. En son beyaz eleman pivottur. **(a)** İlk dizi ve ayarları. Elementlerin hiçbirisi herhangi bir parçaya ait değildir. **(b)** 2 değeri “kendisiyle yer değiştirir” ve küçük elemanların bulunduğu parçaya eklenir. **(c)-(d)** 8 ve 7 değerleri büyük elemanların bulunduğu parçaya eklenir. **(e)** 1 ve 8 değerleri yer değiştirir ve küçük değerlerin bulunduğu parça genişler. **(f)** 3 ve 7 değerleri yer değiştirir ve küçük değerlerin bulunduğu parça genişler. **(g)-(h)** Büyüklerin bulunduğu parça 5 ve 6 değerlerini ekleyerek döngüyü bitirir. **(i)** 7. ve 8. satırda pivot elemanı değiştirilir, böylece iki parça arasına girmiş olur.

j ve $r - 1$ arasındaki indisler bu 3 durum tarafından da kapsanmamaktadır ve bu girdiler içindeki değerlerin pivot ile belirli ilişkileri yoktur.

Bu döngü değişmezinin ilk iterasyondan önce doğru olduğunu, döngünün her bir iterasyonunun değişmezi koruduğunu ve döngü sonlandığında değişmezini doğruluğu göstermek için kullanışlı bir özelliği sağladığını göstermemiz gerekir.



Şekil 7.2 Dört bölge bir $A[p..r]$ altdizisinde PARTITION prosedürü tarafından sağlanır. $A[p..r]$ değerlerinin tümü x ten küçük veya eşittir $A[i+1..j-1]$ elemanlarının tümü x ten büyüktür ve $A[r] = x$ tir. $A[j..r-1]$ dizisi herhangi bir değer alabilir.

Başlangıç: Döngünün ilk iterasyonundan önce $i = p - 1$ ve $j = p$ dir. Hiçbir değer p ve i arasında ve $i + 1$ ve $j - 1$ arasında bulunmadığından dolayı döngü sabitinin ilk iki koşulu basitçe sağlanır. İlk satırdaki atama işlemi 3. koşulu sağlar.

Yineleme: Şekil 7.3 ün gösterdiği gibi 4. satırdaki testin sonucuna göre 2 durumu değerlendirmekteyiz. Şekil 7.3 (a) $A[j] > x$ olduğundaki durumu gösterir; döngüdeki tek hareket j yi arttırmaktır. j artırıldıktan sonra $A[j-1]$ için 2. koşul tutulur ve diğer bütün girdiler değişmeden kalır. Şekil 7.3 (b) $A[j] \leq x$ olduğunda ne olduğunu gösterir; döngü i yi artırır, $A[i]$ ve $A[j]$ yer değiştirir ve j artırılır. Yer değiştirme nedeniyle $A[j] \leq x$ geçerlidir ve 1. koşul sağlanır. Benzer olarak ayrıca $A[j-1] > x$ durumu da vardır, döngü değişmezi tarafından $A[j-1]$ de değiştirildiğinden x den büyüktür.

Sonlandırma: Sonlandırmada $j = r$ dir. Bu nedenle dizideki bütün girdiler döngü değişmezi tarafından üç kümeden birinin içindedir ve dizideki değerleri üç kümeye böldük: x ten küçük veya eşit olanlar, x den büyük olanlar ve x i içeren tekli bir küme.

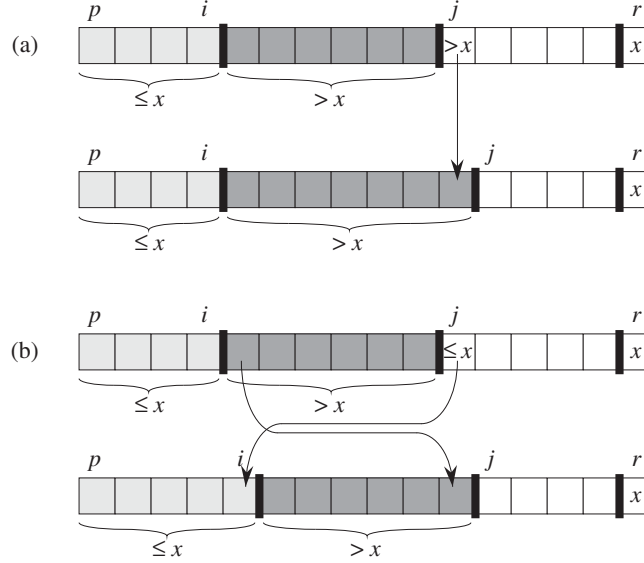
PARTITION un son iki satırı pivotu x den büyük en soldaki eleman ile değiştirerek işlemini bitirir; dolayısıyla pivot parçalanmış dizide doğru yerine kaymış ve pivotun yeni indisi döndürülmüş olur. PARTITION un çıktısı şimdi bölme aşaması için verilen belirtileri karşılar. Aslında bu durum daha güçlü olan koşulu da sağlar: Hızlı sıralamanın 2. satırından sonra $A[q]$ $A[q+1..r]$ nin her elemanından daha azdır.

PARTITION un çalışma zamanı $A[p..r]$ altdizisinde $n = r - p + 1$ olduğunda (Alıştırma 7.1-3 e bakınız) $\Theta(n)$ dir.

Alıştırmalar

7.1-1

Şekil 7.1 i model alarak PARTITION işlemini $A = \langle 13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11 \rangle$ dizisinde gösteriniz.



Şekil 7.3 PARTITION prosedürünün bir yinelenmesi için iki durum. **(a)** Eğer $A[j] > x$ ise yapılacak tek hareket j yi arttırmaktır, bu döngü değişmezini sağlar. **(b)** Eğer $A[j] \leq x$ ise i indisi arttırılmış, $A[i]$ ve $A[j]$ yer değiştirmiş ve sonra j arttırılmıştır. Döngü değişmezi yenilenmiştir.

7.1-2

Bir $A[p..r]$ dizisinin bütün elemanları aynı değere sahipse PARTITION q nun hangi değerini geri döndürür? PARTITION u $A[p..r]$ dizisindeki bütün elemanlar aynı olduğunda $q = \lfloor (p + r)/2 \rfloor$ olacak şekilde değiştirin.

7.1-3

Boyutu n olan bir altdizide PARTITION un çalışma zamanının $\Theta(n)$ olduğunu kısaca açıklayınız.

7.1-4

QUICKSORT u artmayacak sırada sıralamak için nasıl düzenlersiniz?

7.2 Hızlı sıralamanın performansı

Hızlı sıralamanın çalışma zamanı, parçalama için kullanılan elemanlara bağlı olduğundan sırasıyla parçalamanın dengeli ya da dengesiz olup olmasına bağlıdır. Eğer parçalama dengeliyse, algoritma asimptotik olarak birleştirerek sıralamaka-

dar hızlı çalışır. Ancak eğer parçalama dengesizse asimptotik olarak araya sokma sıralaması kadar yavaş çalışır. Bu kısımda, hızlı sıralamanın dengesiz parçalamaya karşı dengeli parçalamanın varsayımları altında nasıl işlediğini inceleyeceğiz.

En kötü-durum parçalaması

Hızlı sıralamanın en kötü-durum davranışı, parçalama programı, $n - 1$ elemanla ve 1 tane 0 elemanla altproblemler oluşturduğu zaman ortaya çıkar (Bunu 7.4.1 de kanıtladık). Bu dengesiz parçalamanın her bir özyinelemeli çağrıda ortaya çıktığını varsayalım. Parçalama $\Theta(n)$ zaman harcar. Boyutu 0 olan bir dizide özyinelemeli çağrı döndüğünden $T(0) = \Theta(1)$ dir ve çalışma zamanı için özyineleme

$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= T(n-1) + \Theta(n) \end{aligned}$$

dir.

Sezgisel olarak, eğer özyinelemenin her bir seviyede oluşan maliyetini toplarsak, $\Theta(n^2)$ olarak hesaplanan bir aritmetik seri elde ederiz. Gerçekten de özyineleme $T(n) = T(n-1) + \Theta(n)$ nin $T(n) = \Theta(n^2)$ çözümüne sahip olduğunu kanıtlamak için yerine koyma yöntemini kullanmak oldukça basittir (Alıştırma 77.2-1 e bakınız).

Böylece, parçalama algoritması her bir özyinelemeli seviyede maksimal dengesizdir. Çalışma zamanı $\Theta(n^2)$ dir. Bu nedenle hızlı sıralamanın en kötü-durum çalışma süresi araya sokma sıralamasından daha iyi değildir. Dahası bu $\Theta(n^2)$ çalışma zamanı dizi neredeyse sıralı olduğu durumda meydana gelir. Araya sokma sıralamasının $O(n)$ zamanda çalıştığı sıradan bir durumdur.

En iyi-durum parçalaması

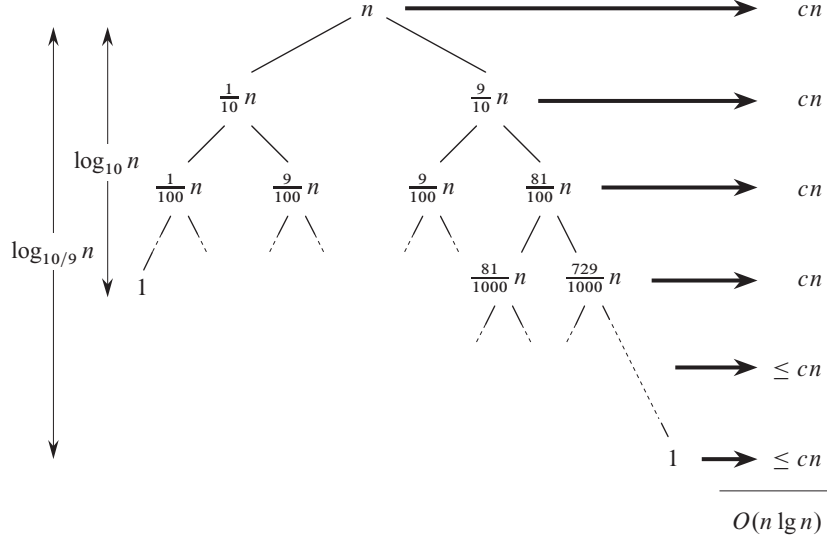
En olağan muhtemel ayırımında, PARTITION iki altproblem yaratır, birinin boyutu $\lfloor n/2 \rfloor$ ve birinin boyutu $\lceil n/2 \rceil - 1$ olduğundan, her birinin boyutu $n/2$ den fazla değildir. Bu durumda çalışma zamanı için özyineleme

$$T(n) = 2T(n/2) + \Theta(n)$$

dir, burada zemin ile tavanı ve 1 çıkarmayı gözardı ederek eğimliliğe izin veririz. Ana teoremin 2. durumundan bu özyineleme $T(n) = \Theta(n \lg n)$ çözüme sahiptir. Özyinelemenin her seviyesinde parçalamanın iki tarafını da eşit olarak dengeleyerek, asimptotik olarak daha hızlı bir algoritma elde ederiz.

Dengeli parçalama

Hızlı sıralamanın ortalama durumda çalışma zamanı kısım 7.4 te analizi gösterildiği gibi en iyi-duruma en kötü-durumdan daha yakındır. Parçalamanın dengesi-



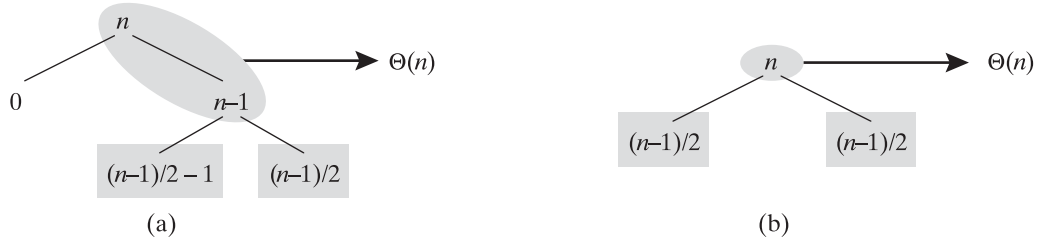
Şekil 7.4 QUICKSORT için parçalamanın her zaman 9-a-1 ayırma ürettiği çalışma zamanı $O(n \lg n)$ olan özyinelemenin ağacı. Düğümler her bir seviye maliyeti sağda olacak şekilde altproblemin boyutunu gösterir. Her bir seviye maliyeti $\Theta(n)$ teriminde c sabitini içerir.

nin nasılını niçinini anlamanın anahtarı çalışma zamanını tanımlayan bir özyinelemede yansıtılır.

Varsayalım ki, örneğin, parçalama algoritması her zaman ilk bakışta oldukça dengesiz görünen 9-a-1 orantılı ayrımlar (splits) üretir. Sonra hızlı sıralamanın çalışma zamanında $\Theta(n)$ içinde gizlenen c sabitini açıkça eklediğimiz

$$T(n) = T(9n/10) + T(n/10) + cn$$

özyinelemesini elde ederiz. 7.4 te bu özyineleme için, özyineleme ağacını göstermektedir. Özyineleme, $\log_{10} n = \Theta(\lg n)$ derinliğinde sınırlı koşullara ulaşana kadar ağacın her seviyesi cn maliyete sahiptir. Ve sonra seviyeler en fazla cn maliyete sahiptir. Özyineleme, $\log_{10/9} n = \Theta(\lg n)$ de sonlandırılır. Bundan dolayı hızlı sıralamanın toplam maliyeti $O(n \lg n)$ olur. Böylece sezgisel olarak dengesiz olan özyinelemenin her seviyesinde 9-a-1 oranlı ayırmayla hızlı sıralama $O(n \lg n)$ sürede çalışır—ayırma ortanın sağ aşağısındaymış gibi asimptotik olarak aynıdır. 99-a-1 lik bir ayırma bile $O(n \lg n)$ çalışma zamanına sahip olurdu. Hatta, her bir seviyesinin maliyeti $O(n)$ olan sabit oranlı her ayırım $\Theta(\lg n)$ derinliğe sahip bir özyineleme ağacını sağlar. Bundan dolayı çalışma zamanı, ayırma sabit bir orana sahip olduğunda $O(n \lg n)$ olur.



Şekil 7.5 (a) Hızlı sıralama için 2 seviyeli özyineleme ağacı. Kökteki parçalama maliyeti n dir ve “kötü” bir ayırma üretir: boyutu 0 ve $n - 1$ olan 2 alt dizi. Boyutu $n - 1$ olan alt diziyi parçalamak “iyi” bir ayırma verir: boyutu $(n - 1)/2 - 1$ olan alt dizi ve $n - 1/2$ olan bir alt dizi. (b) Bir özyineleme ağacının tek seviyesi çok iyi dengelidir. İki kısımda da altproblem için parçalama maliyeti elips gölgelendirmeyle gösterilmiş şekilde n dir. Yine de çözülmeyi bekleyen altproblemlerden (a) da kare gölgelendirmeyle gösterilmiş altproblem (b) de çözülmeyi bekleyen altproblemlerden daha büyük değildir.

Ortalama durum için sezgi

Hızlı sıralamanın rastgele davranışlarının kavramını açıkça geliştirmek için çeşitli girdilerle ne kadar sıklıkta başa çıkacağımızı beklediğimizi varsaymak zorundayız. Hızlı sıralamanın davranışı dizi içinde girdi olarak verilen elemanların değerlerinin göreceli sıralamasına bağlıdır. Kısım 5.2 deki kiralama probleminin olasılıksal analizindeki gibi, şimdilik girdi sayılarının permütasyonlarının hepsinin eşit olarak benzediğini varsayalım.

Rassal girdili bir dizide hızlı sıralama algoritmasını çalıştırdığımızda, resmi olmayan analizimizin varsayımına göre her seviyede aynı şekilde olması yüksek oranda olası değildir. Ayırımların bazılarının makul bir şekilde dengeli bazı ayırımların da oldukça dengesiz olmasını bekleriz. Örneğin alıştırma 7.2-6 PARTITION un yüzde 80 inin 9-a-1 den daha dengeli bir ayırım oluşturmasını ve yüzde 20 sinin 9-a-1 den daha az dengeli bir ayırım oluşturmasını göstermenizi istemektedir.

Ortalama durumda, PARTITION karışık bir şekilde “iyi” ve “kötü” ayırımlar oluşturur. PARTITION un ortalama durumlu çalışması için özyineleme ağacında, iyi ve kötü ayırımlar ağaçta rastgele olarak dağılmıştır. Sezgisel olarak hatrı için iyi ve kötü ayırımlar ağaçta seviyeyi değişimli olarak yaptığını ve en iyi ayırımların en iyi-durumlu ayırım ve kötü ayırımların en kötü-durumlu ayırım olduğunu varsayalım. Şekil 7.5 (a) özyineleme ağacında ardarda gelen iki seviyede ayırımı gösterir. Ağacın kökünde maliyet ayırma için n dir ve alt dizilerin boyutu $n - 1$ ve 0 olarak üretilmiştir: en kötü durum. Bir sonraki seviyede boyutu $n - 1$ olan alt dizi, boyutu $(n - 1)/2 - 1$ ve $(n - 1)/2$ olan alt dizilere olacak şekilde en iyi-durum ayrılmasına maruz kalır.

Boyutu 0 olan altdizi için sınır durumu maliyetinin 1 olduğunu varsayalım. İyi bir ayırım tarafından takip edilen kötü ayırımın kombinasyonu $\Theta(n) + \Theta(n - 1) = \Theta(n)$ nin birleşik parçalama maliyetinde, $(n - 1)/2 - 1$ ve $(n - 1)/2$ olan 0 boyutlu üç altdizi üretir. Kuşkusuz Şekil 7.5 (b) de boyutu $(n - 1)/2$ maliyeti $\Theta(n)$ olan iki altdizi üreten tek seviyeli parçalama olarak adlandırılan durumdan daha kötü değildir. Yine de bu sonraki durum dengelidir! Sezgisel olarak kötü ayırımın $\Theta(n - 1)$ maliyeti, iyi ayırımın $\Theta(n)$ maliyetine absorbe edilmiş olabilir ve ayırımın iyi olmasıyla sonuçlanır. Böylece hızlı sıralamanın çalışma zamanı seviyeler iyi ve kötü ayırım arasında değiştirildiğinde iyi ayırımın çalışma zamanı gibi olur: $O(n \lg n)$ ama O -notasyonu içinde gizli sabiti biraz daha büyük olarak. Kısım 7.4.2 de hızlı sıralamanın rassal versiyonunun beklenen çalışma zamanının kesin bir analizini verebilmeliyiz.

Alıştırmalar

7.2-1

Yerine koyma yöntemini kullanarak Kısım 7.2 de iddia edildiği gibi özyineleme $T(n) = T(n - 1) + \Theta(n)$ nin $T(n) = \Theta(n^2)$ çözümlerine sahip olduğunu kanıtlayınız.

7.2-2

A dizisinin bütün elemanları aynı değere sahiplerken hızlı sıralama algoritmasının çalışma zamanı nedir?

7.2-3

A dizisinin tüm elemanları farklı ve azalan bir şekilde sıralı olduğunda QUICK-SORT algoritmasının çalışma zamanının $\Theta(n^2)$ olduğunu gösteriniz.

7.2-4

Bankalar sık sık işlem zamanına göre bir hesabın işlemlerinin kaydını tutarlar ancak bir çok insan hesap özetlerini çek numarasına göre listelenmiş çeklerle almayı isterler. İnsanlar genelde çeklerini çek numarasıyla yazarlar ve satıcılar bunları makul bir göndermeyle paraya çevirirler. İşlemlerin zamanı sıralamasını çek numarasına çevirme problemi bu nedenle neredeyse sıralı girdiyi sıralama problemidir. INSERTION-SORT prosedürünün QUICKSORT prosedürünü yenme eğilimi göstermesini tartışınız.

7.2-5

Hızlı sıralamanın her seviyesindeki ayırımlar $0 < \alpha \leq 1/2$ sabitken $1 - \alpha$ dan α , oranında olduğunu varsayalım. Özyineleme ağacında bir yaprağın en düşük derinliğinin yaklaşık olarak $-\lg n / \lg \alpha$ olduğunu ve en büyük değerinin $-\lg n / \lg(1 - \alpha)$ olduğunu gösteriniz (Tam sayı yuvarlaması hakkında endişelenmeyin).

7.2-6 *

Herhangi bir rassal girdili dizide $0 < \alpha \leq 1/2$ sabiti için olasılığın yaklaşık $1 - 2\alpha$ olduğunda PARTITION un $1 - \alpha$, daha dengeli bir ayırım oluşturduğunu tartışınız. Hızlı sıralamanın ortalama durum davranışını araştırırken, tüm girdi sayılarının

7.3 Hızlı sıralamanın rassal versiyonu

permütasyonlarının eşit olarak benzeştiğini varsaydık. Ancak bir mühendislik durumunda varsayımların her zaman geçerli olacağını bekleyemeyiz (Alıştırma 7.3 e bakınız). Kısım 5.3 de gördüğümüz gibi bütün girdiler üzerinde iyi beklenen bir performans almak için bazen rastgeleliği algoritmaya ekleriz. Birçok insan hızlı sıralamanın rassal versiyonunun sonucuna göre, büyük girdiler için bu algoritmayı sıralama algoritması olarak dikkate almıştır.

Kısım 5.3 de algoritmamızı girdileri açıkça değiştirerek rastgeleleştirdik. Bunu hızlı sıralama için de yapabiliriz; ancak **rassal örnekleme** adı verilen daha basit analiz sunan başka bir rastgeleleştirme tekniği ile her zaman $A[r]$ yi pivot olarak kullanmak yerine $A[p \dots r]$ alt dizisinde rassal herhangi bir elemanı seçeriz. Bunu ilk önce $A[r]$ elemanını $A[p \dots r]$ dizisinin rassal seçilmiş bir elemanı ile değiştiririz. $p, \dots, r$ aralığını rastgele örnekleyerek pivot elemanı $x = A[r]$ ın alt dizideki herhangi bir $r - p + 1$ elemanı ile eşit olarak benzer olmasını sağlarız. Pivot elemanını rastgele seçtiğimizden girdi dizisi ayrımları ortalamada makul bir şekilde iyi dengelenmiştir.

PARTITION ve QUICKSORT un değişiklikleri küçüktür. Yeni parçalama prosedüründe yer değiştirme işlemini parçalamadan önce yaparız.

RANDOMIZED-PARTITION (A, p, r)

- 1 $i = \text{RANDOM}(p, r)$
- 2 $A[r]$ ile $A[i]$ yi değiştir
- 3 **return** PARTITION (A, p, r)

PARTITION yerine yeni hızlı çağırma çağırısı RANDOMIZED-PARTITION:

RANDOMIZED-QUICKSORT(A, p, r)

- 1 **if** $p < r$
- 2 $q = \text{RANDOMIZED-PARTITION}(A, p, r)$
- 3 RANDOMIZED-QUICKSORT($A, p, q - 1$)
- 4 RANDOMIZED-QUICKSORT($A, q + 1, r$)

Bir sonraki kısımda bu algoritmayı analiz edeceğiz.

Alıştırmalar

7.3-1

Neden rassal algoritmanın en kötü-durum çalışma zamanını değil de beklenen çalışma zamanını araştırırız?

7.3-2

RANDOMIZED-QUICKSORT çalıştığında en kötü-durumda rassal sayı üretici RANDOM için kaç defa çağrı yapılıyor? Peki en iyi-durumda kaç defa çağrı yapılıyor? Cevaplarınızı $\Theta(n)$ notasyonunda yazınız.

7.4 Hızlı sıralamanın analizi

Kısım 7.2 hızlı sıralamanın en kötü-durum davranışı ve neden hızlı çalıştığını beklediğimiz için bazı sezgiseller verir. Bu kısımda hızlı sıralamanın davranışını daha dikkatli bir biçimde analiz ettik. QUICKSORT veya RANDOMIZED-QUICKSORT a uygulanan en kötü-durum analizi ile başlayıp, RANDOMIZED-QUICKSORT un beklenen çalışma zamanının analiziyle tamamladık.

7.4.1 En kötü-durum analizi

Kısım 7.2 de hızlı sıralamada özyinelemenin her seviyesinde en kötü-durum ayrımı sezgisel olarak algoritmanın en kötü-durum çalışma zamanı olan $\Theta(n^2)$ çalışma zamanı oluşturduğunu gördük. Şimdi bu tezi kanıtlayacağız.

Yerine koyma yöntemini kullanarak (Kısım 7.3 e bakınız.) hızlı sıralamanın çalışma zamanının $O(n^2)$ olduğunu gösteririz. Hızlı sıralama prosedürü için n boyutlu girdi durumunda en kötü-durum çalışma zamanı $T(n)$ olsun.

PARTITION prosedürü toplam boyutu $n - 1$ olan iki altproblem yarattığından dolayı q parametresi 0 ile $n - 1$ aralığında bulunacak şekilde

$$T(n) = \max_{0 \leq q \leq n-1} (T(q) + T(n - q - 1)) + \Theta(n) \quad (7.1)$$

özyinelemesine sahibiz.

$T(n) \leq cn^2$ nin bazı c sabiti için olduğunu varsayıyoruz. Bu tahmini (7.1) özyinelemesinde yerine koyarak:

$$\begin{aligned} T(n) &\leq \max_{0 \leq q \leq n-1} (cq^2 + c(n - q - 1)^2) + \Theta(n) \\ &= c \cdot \max_{0 \leq q \leq n-1} (q^2 + (n - q - 1)^2) + \Theta(n) \end{aligned}$$

ifadelerini elde ederiz.

$q^2 + (n - q - 1)^2$ ifadesi $0 \leq q \leq n - 1$ aralığındaki yada bitiş noktasındaki parametrelerden en büyüğüne sahiptir. Bu iddiayı doğrulamak için q ya

göre ifadenin ikinci türevi pozitifdir (Alıştırma 7.4-3'ü bakınız.). Bu gözlem bize $\max_{0 \leq q \leq n-1} (q^2 + (n - q - 1)^2) \leq (n - 1)^2 = n^2 - 2n + 1$ sınırını verir. $T(n)$ in sınırlanmasına devam ederek, yeterince büyük c sabitini seçebildiğimizden $c(2n - 1)$ ifadesi n ifadesini baskılar böylece

$$\begin{aligned} T(n) &\leq cn^2 - c(2n - 1) + \Theta(n) \\ &\leq cn^2, \end{aligned}$$

ifadesini elde ederiz. Bu nedenle $T(n) = O(n^2)$ olur. Kısım 7.2 de hızlı sıralamanın $\Omega(n^2)$ zaman aldığı belirli bir durum gördük: ayırım dengesiz olduğu zaman. Alternatif olarak Alıştırma 7.4-1 özyineleme (7.1) in $T(n) = \Omega(n^2)$ çözümüne sahip olduğunu göstermenizi istemektedir. Bundan dolayı hızlı sıralamanın (en kötü-durum) çalışma zamanı $\Theta(n^2)$ dir.

7.4.2 eklenen çalışma süresi

RANDOMIZED-QUICKSORT un beklenen çalışma zamanının neden $O(n \lg n)$ olduğunun arkasındaki sezgiyi zaten gördük: eğer özyinelemenin her seviyesinde RANDOMIZED-QUICKSORT tarafından uyarılan ayırımların bir tarafında elemanların her sabit parçacığını koyabilirse o zaman özyineleme ağacı $\Theta(\lg n)$ derinliğe sahiptir ve her seviyede $O(n)$ iş yapılır. Bu seviyeler arasındaki mümkün en dengesiz ayırma sahip olan seviyeleride eklersek toplam süre $O(n \lg n)$ olur. RANDOMIZED-QUICKSORT algoritmasını ilk önce parçalama prosedürünün nasıl işlediğini anlayarak ve sonra bunu beklenen çalışma zamanında bir $O(n \lg n)$ sınır türeterek analiz edebiliriz. Kısım 7.2 de gördüğümüz $\Theta(n \lg n)$ en iyi durum sınırı ile birleştirilen beklenen çalışma zamanındaki bu üst sınır $\Theta(n \lg n)$ beklenen çalışma zamanına sahiptir. Sıralanan elemanlarının değerlerinin her birinin ayrı olduğunu baştan varsayalım.

Çalışma zamanı ve karşılaştırmalar

QUICKSORT ve RANDOMIZED-QUICKSORT prosedürleri sadece pivot seçimini nasıl seçecekleri konusunda farklılık gösterir; diğer bütün şeylerde aynıdır. Bu nedenle RANDOMIZED-QUICKSORT analizini QUICKSORT ve PARTITION prosedürlerini tartışarak ifade ederiz. Ancak bunu pivot elemanların RANDOMIZED-QUICKSORT a gönderilen alt diziden rassal sayılarla yapalım.

QUICKSORT un çalışma zamanı PARTITION prosedürü harcanan zaman tarafından baskın gelmiştir. PARTITION prosedürü her defasında çağrıldığında bir pivot elemanı seçer ve bu eleman hiçbir zaman ilerideki her özyinelemeli çağrıdaki QUICKSORT ve PARTITION a çağrılan yineleme içine eklenmemiştir. Böylece hızlı sıralama algoritmasının bütün çalışması boyunca PARTITION a en fazla n tane çağrı yapılır. PARTITION ya bir çağrı $O(1)$ zaman alır ve ayrıca satır 3-6 daki **for**

döngüsünün iterasyonlarının sayısıyla orantılı olan zaman kadar süre alır. Bu **for** döngüsünün her iterasyonunda satır 4 te ki pivot elemanı ile A dizisinin diğer elemanları karşılaştırılır. Bundan dolayı eğer satır 4 ün toplam çalışma sayısını sayarsak, hızlı sıralamanın çalışması boyunca **for** döngüsü için harcanan zamanı sınırlandırabiliriz.

Lemma 7.1

X bütün hızlı sıralama çalışması boyunca n elemanlı bir dizide PARTITION da satır 4 te gerçekleştirilen karşılaştırma sayısı olsun. QUICKSORT un çalışma zamanı $O(n + X)$ dir.

Kanıt Yukarıdaki tartışmaya göre algoritma PARTITION a en fazla n çağrı yapar, bunların her biri sabit miktarda iş yapar ve **for** döngüsünü belli bir sayıda çalıştırırız. **for** döngüsünün her bir iterasyonu 4. Satırda çalışır. ■

Bundan dolayı amacımız PARTITION a yapılan bütün çağrılar boyunca yapılan toplam karşılaştırma sayısı X i hesaplamaktır. PARTITION a yapılan her çağrıda kaç karşılaştırma yapıldığı analizini yapmayacağız. Bunun yerine toplam karşılaştırma sayısına ortalama bir sınır türeteceğiz. Bunu yapmak için algoritmanın ne zaman iki dizi elemanını karşılaştırdığını ve ne zaman karşılaştırmadığını anlamalıyız. Analiz kolaylığı için A dizisinin elemanlarını $z_1, z_2, \dots, z_n, z_i$ ile i 'inci en küçük eleman olarak yeniden adlandırdık. Aynı zamanda z_i ile z_j arasında dahil edecek şekilde $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$ kümesini tanımladık.

Algoritma z_i ve z_j yi ne zaman karşılaştırır? Bu soruyu cevaplamak için her eleman çiftinin en fazla bir kere karşılaştırıldığını gözlemledik. Neden? Elemanlar sadece pivot elemanı ile karşılaştırılır ve PARTITION un kısmi çağrısı bittiğinde bu çağrıda kullanılan pivot elemanı bir daha diğer elemanlarla karşılaştırılmaz.

Analizimiz rassal gösterge değişkenlerini kullanır (bakınız Kısım 5.2).

Karşılaştırmamızın, PARTITION un bir çağrısı ya da bir iterasyonu sırasında değil algoritmanın çalışması sırasında herhangi bir zamanda yer alıp almadığını değerlendirdiğimiz

$X_{ij} = I\{z_i, z_j \text{ ile karşılaştırılır}\}$ ı tanımlarız.

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$$

iki tarafın da beklentisini alarak ve beklentinin doğrusallığı ile ve Lemma 5.1 i kullanarak

$$E[X] = E \left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \right]$$

$$\begin{aligned}
&= \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}] \\
&= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr \{z_i, z_j \text{ ile karşılaştırılır}\}
\end{aligned}$$

yi elde ederiz.

$\Pr \{z_i \text{ ile } z_j \text{ karşılaştırılır}\}$ yi hesaplamak aynı kalır. Analizimiz RANDOMIZED-PARTITION prosedürünün her pivotu rastgele ve bağımsız seçtiğini varsayar.

Elemanın karşılaştırılmadığı durumu düşünelim. 1 den 10 a kadar olan sayıların (herhangi bir sırada) hızlı sıralama için girdi olduklarını düşünelim ve ilk pivot elemanın 7 olduğunu varsayalım. PARTITION a yapılan ilk çağrı sayıları iki kümeye ayırır: $\{1, 2, 3, 4, 5, 6\}$ ve $\{8, 9, 10\}$. Bunu yaparak pivot elemanı 7 bütün diğer elemanlarla karşılaştırılır; ancak ilk kümedeki hiçbir eleman (örneğin 2) ikinci kümedeki elemanlarla (örneğin 9) karşılaştırılmaz.

Genelde elemanların değerlerinin farklı olduğunu düşündüğümüzden, pivot $x < z_i < z_j$ ile seçildiğinde z_i ve z_j elemanlarının hiçbir alt durumda karşılaştırılmadıklarını biliyoruz. Diğer taraftan eğer Z_{ij} deki herhangi bir elemandan önce z_i pivot olarak seçilirse z_i kendisi hariç bütün elemanlarla karşılaştırılır. Benzer olarak eğer Z_{ij} deki herhangi bir elemandan önce z_j pivot olarak seçilirse z_j kendisi hariç bütün elemanlarla karşılaştırılır. Bizim örneğimizde 7 ve 9 değerleri karşılaştırılır çünkü 7, $Z_{7,9}$ da seçilen ilk pivot 7 dir. Karşılık olarak 2 ve 9 hiçbir zaman karşılaştırılmazlar; çünkü $Z_{7,9}$ seçilen ilk pivot elemanı 7 dir. Bu nedenle z_i ve z_j sadece Z_{ij} de pivot olarak seçilen ilk eleman z_i veya z_j olduğunda karşılaştırılır.

Şimdi bu olayın oluşma olasılığını hesaplayabiliriz. Z_{ij} de bir eleman pivot olarak seçilmeden önce bütün Z_{ij} kümesi aynı parçadadır. Bundan dolayı Z_{ij} nin herhangi bir elemanı pivot olarak seçilmek için eşit olasılıklara sahiptirler. Z_{ij} kümesi $j - i + 1$ elemanına sahip olduğundan ve pivotlar rastgele ve bağımsız seçildiklerinden herhangi bir elemanın ilk olarak pivot seçilme olasılığı $1/(j - i + 1)$ dir.

$$\begin{aligned}
\Pr \{z_i, z_j \text{ ile karşılaştırılır}\} &= \Pr \{z_i \text{ veya } z_j \text{ } Z_{ij} \text{ den seçilen ilk pivottur}\} \\
&= \Pr \{z_i \text{ } Z_{ij} \text{ den seçilen ilk pivottur}\} \\
&\quad + \Pr \{z_j \text{ } Z_{ij} \text{ den seçilen ilk pivottur}\} \\
&= \frac{1}{j - i + 1} + \frac{1}{j - i + 1} \\
&= \frac{2}{j - i + 1} .
\end{aligned} \tag{7.3}$$

İkinci satırı takip eder çünkü iki olay birbirinden bağımsızdır. Denklem (7.2) ve denklem (7.3) ü karşılaştırır ve sıradaki ifadeyi elde ederiz:

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1}.$$

Bu toplamı ($k = j - i$) değişkenlerinin değişimini ve (A.7) denklemindeki harmonik seriler üzerindeki sınırları kullanarak hesaplayabiliriz:

$$\begin{aligned} E[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} \\ &= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} \\ &< \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} \\ &= \sum_{i=1}^{n-1} O(\lg n) \\ &= O(n \lg n). \end{aligned} \tag{7.4}$$

Bu nedenle, RANDOMIZED-PARTITION u kullanmak, hızlı çalışmanın beklenen çalışma zamanı elemanların değerleri ayrı olduğunda $O(n \lg n)$ olur.

Alıştırmalar

7.4-1

$$T(n) = \max_{0 \leq q \leq n-1} (T(q) + T(n-q-1)) + \Theta(n)$$

özyinelemesinde $T(n) = \Omega(n^2)$ olduğunu gösteriniz.

7.4-2

Hızlı sıralamanın en iyi-durum çalışma zamanının $\Omega(n \lg n)$ olduğunu gösteriniz.

7.4-3

$q^2 + (n - q - 1)^2$ ifadesinin $q = 0$ veya $q = n - 1$ olduğunda $q = 0, 1, \dots, n - 1$ üzerinde en fazla değere ulaştığını gösteriniz.

7.4-4

RANDOMIZED-QUICKSORT un beklenen çalışma zamanının $\Omega(n \lg n)$ olduğunu gösteriniz.

7.4-5

Uygulamada hızlı sıralamanın çalışma zamanını sayılar neredeyse sıralı olduğunda araya sokma sıralamasının hızlı çalışma zamanının avantajını kullanarak geliştirebiliriz. k den daha az elemana sahip bir dizide hızlı sıralamayı çağırarak basitçe sayıları sıralamadan geri döndürelim. Hızlı sıralamaya yapılan üst seviye çağrı döndüğünde sıralama işlemini bitirmek için bütün dizide araya sokma sıralaması çalıştırılır. Bu sıralama algoritmasının beklenen $O(nk + n \lg(n/k))$ zamanında çalıştığını tartışınız Hem teoride hem de uygulamada k yı nasıl seçmeliyiz?

7.4-6 *

A dizisinden rastgele üç eleman çekerek ve bunların medyanından (üç elemanın orta değeri) ayırarak PARTITION prosedürünü değiştirmeyi değerlendiririz. $0 < \alpha < 1$ aralığında α nın bir fonksiyonu olarak en kötü α -ya- $(1 - \alpha)$ ayırımı alma olasılığına yakınsayınız.

Problemler
7-1 Hoare ayırımının doğruluğu

PARTITION un bu bölümde verilen versiyonu orijinal parçalama algoritması değildir. C.A.R Hoare tarafından sunulan orijinal parçalama algoritması aşağıdadır:

HOARE-PARTITION(A, p, r)

```

1   $x = A[p]$ 
2   $i = p - 1$ 
3   $j = r + 1$ 
4  while TRUE
5      repeat
6           $j = j - 1$ 
7      until  $A[j] \leq x$ 
8      repeat
9           $i = i + 1$ 
10     until  $A[i] \geq x$ 
11     if  $i < j$ 
12          $A[i]$  ile  $A[j]$  yi değiştir
13     else return  $j$ 
```

- a. Dizinin değerlerini ve 4-13 satırlarındaki **while** döngüsünün her bir iterasyonundan sonraki yardımcı değerleri gösteren $A = \langle 13, 19, 9, 5, 12, 8, 7, 4, 11, 2, 6, 21 \rangle$ dizisindeki HOARE-PARTITION nun işlemini gösteriniz.

Sıradaki üç soru sizden HOARE-PARTITION prosedürünün doğru olduğunu gösteren dikkatli argümanlar vermenizi istemektedir. Alt dizi $A[p \dots r]$ en az iki eleman içerdiğini varsayarak sıradakileri kanıtlayınız:

- b.** i ve j indisleri $A[p \dots r]$ dizisi dışında A dizisinin elemanlarının hiçbir zaman ulaşılmayacağı gibidir.
- c.** HOARE-PARTITION bittiğinde $p \leq j < r$ olacak şekilde j değerini döndürür.
- d.** $A[p \dots j]$ nin her elemanı $A[j + 1 \dots r]$ nin her elemanından küçük veya eşittir.

Kısım 7.1 deki PARTITION prosedürü oluşturduğu iki parçalamadan pivot değerini (asıl olarak $A[p]$) ayırır. Diğer tarafından HOARE-PARTITION pivot değerini her zaman $A[p \dots j]$ ve $A[j + 1 \dots r]$ içine yerleştirir. $p \leq j < r$ olduğundan bu parçalama her zaman aşıkâr (açık) değildir.

- e.** HOARE-PARTITION u kullanarak QUICKSORT prosedürünü tekrar yazınız.

7-2 Eşit değerli eleman ile hızlı sıralama

Kısım 7.4.2 deki rassal hızlı sıralamanın beklenen çalışma zamanının analizi tüm elemanlarının değerlerinin eşit olduğunu varsayar. Bu problemde aynı olmadıklarında ne olacağını inceliyoruz.

- a.** Bütün elemanların eşit olduğunu varsayalım. Rassal hızlı sıralamanın bu durumdaki çalışma zamanı ne olur?
- b.** PARTITION prosedürü $A[p \dots q - 1]$ nin her elemanının $A[q]$ dan daha küçük veya eşit olduğu ve $A[q + 1 \dots r]$ nin her elemanının $A[q]$ dan büyük veya eşit olduğu q indisini geri döndürür. PARTITION' prosedürünü $p \leq q \leq t \leq r$ olduğu
 - $A[q \dots t]$ nin her elemanı büyük
 - $A[p \dots q - 1]$ in her elemanı $A[q]$ dan daha küçük ve
 - $A[t + 1 \dots r]$ in her elemanı $A[q]$ dan daha büyük olacak şekilde

$A[p \dots r]$ elemanlarını değiştiren ve q ve t indislerini geri döndüren $\Theta(r - p)$ zaman gerektiren PARTITION' prosedürünü düzenleyin.

- c.** RANDOMIZED-PARTITION prosedürünü PARTITION' u çağırarak ve yeni prosedürü RANDOMIZED-PARTITION' olarak adlandırarak şekilde düzenleyin. Daha sonra hızlı sıralama prosedürünü RANDOMIZED-QUICKSORT' u çağırarak ve sa-

dece birbirlerinin eşit olduğunu bilmeyen eleman parçalanmasında tekrarlayan QUICKSORT' (p, r) prosedürünü oluşturacak şekilde değiştirin.

- d. QUICKSORT' u kullanarak her elemanın ayrı olduğu varsayımından kaçınmak için Kısım 7.4.2 deki analizi nasıl düzenleriz?

7-3 Alternatif hızlı sıralama analizi

Rassal hızlı sıralamanın alternatif bir çalışma zamanı analizi yapılan karşılaştırma sayısındana RANDOMIZED-QUICKSORT a yapılan her bir özyinelemeli çağrının beklenen çalışma zamanına odaklanmıştır.

- a. Boyutu n olan bir dizide belirli bir elemanın pivot olarak seçilme olasılığının $1/n$ olduğunu tartışınız. Bunu rassal gösterge değişkenleri $X_i = I_{\{i \text{ inci en küçük elemanın pivot olarak seçilmiştir}\}}$ şeklinde tanımlamak için kullanın. $E[X_i]$ ne olur?
- b. $T(n)$ boyutlu bir dizide hızlı sıralamanın çalışma zamanını belirten bir değişken olsun.

$$E[T(n)] = E \left[\sum_{q=1}^n X_q (T(q-1) + T(n-q) + \Theta(n)) \right] \quad (7.5)$$

olduğunu tartışınız.

- c. (7.5)deki denklemin

$$E[T(n)] = \frac{2}{n} \sum_{q=2}^{n-1} E[T(q)] + \Theta(n) \quad (7.6)$$

şeklinde tekrar yazılabileceğini gösteriniz.

- d.

$$\sum_{k=2}^{n-1} k \lg k \leq \frac{1}{2} n^2 \lg n - \frac{1}{8} n^2 \quad (7.7)$$

olduğunu gösteriniz. (*İpucu:* toplamayı iki parçaya ayırın bir tane $k = 2, 3, \dots, \lceil n/2 \rceil - 1$ için ve bir tane $k = \lceil n/2 \rceil, \dots, n-1$ için)

- e. Denklem 7.7 deki sınırı kullanarak denklem 7.6 daki özyinelemenin $E[T(n)] = \Theta(n \lg n)$ çözüme sahip olduğunu gösteriniz. (*İpucu:* Yeterince büyük ve bazı pozitif a sabiti için değişken dönüşümüyle $E[T(n)] \leq an \lg n$ olduğunu gösteriniz.)

7-4 Hızlı sıralama için yığın derinliği

Kısım 7.1 deki hızlı sıralama algoritması iki özyinelemeli çağrı içerir. QUICKSORT, PARTITION u çağırdıktan sonra, sol altdiziyi özyinelemeli olarak sıralar ve sonra özyinelemeli olarak sağ altdiziyi sıralar. QUICKSORT taki ikinci özyineleme gerekli değildir; bundan iteratif kontrol yapısı kullanarak kaçınılırız. **Kuyruk özyinelemesi** olarak adlandırılan bu teknik iyi derleyiciler tarafından otomatik olarak sağlanmaktadır. Kuyruk özyinelemesini simule eden sıradaki hızlı hızlı sıralamanın aşağıdaki versiyonunu dikkate alınız:

TAIL-RECURSIVE-QUICKSORT(A, p, r)

```

1  while  $p < r$ 
2      // Sol altdiziyi parçala ve sırala
3       $q = \text{PARTITION}(A, p, r)$ 
4      TAIL-RECURSIVE-QUICKSORT( $A, p, q - 1$ )
5       $p = q + 1$ 
```

- a. TAIL-RECURSIVE-QUICKSORT un A dizisini doğru bir şekilde sıraladığını tartışınız.

Derleyiciler özyinelemeli prosedürler için genelde her özyinelemeli çağrı için parametre değerlerinin de dahil olduğu ilgili bilgi içeren bir **yığın** kullanarak çalıştırılır. En yakındaki çağrı için gereken bilgiler yığının tepesindedir ve başlangıçtaki çağrılar için veriler yığının tabanındadır. Bir prosedürün çağrılması üzerine bilgisi yığına **ittirilir(pushes)**; bittiği zaman bu bilgi yığından **çıkarılır (popped)**. Dizi parametrelerinin işaretçiler tarafından fazla temsil edildiğini varsaydığımızdan beri; yığına yapılan her bir prosedür çağrısı $O(1)$ kadar yığın alanı gerektirmektedir. **Yığın derinliği** hesaplama sırasında herhangi bir zamanda kullanılan yığın alanının en büyük miktarıdır.

- b. n elemanlı bir girdi dizisinde TAIL-RECURSIVE-QUICKSORT nın yığın derinliğinin $\Theta(n)$ olduğu bir senaryo belirtin.
- c. TAIL-RECURSIVE-QUICKSORT için verilen kodu en kötü durumda yığın derinliği $\Theta(\lg n)$ olacak şekilde değiştirin. Algoritmanın beklenen çalışma süresini $O(n \lg n)$ olacak şekilde sağlayın.

7-5 3-ün-medyanı parçalaması

RANDOMIZED-QUICKSORT prosedürünü geliştirmek için bir yol pivotu bir dizi elemanları arasından rastgele seçmekten daha dikkat ederek seçmektir. Yaygın bir yöntem **3-ün-medyanı** yöntemidir: pivotu altdiziden rassal seçilmiş 3 elemanın kümesinin medyanını (orta eleman) seçmek (Alıştırma 7.4-6 yı görün). Bu problem için girdi dizisi $A[1..n]$ deki elemanlar farklı ve $n \geq 3$ olduğunu var-

sayalım. Sıralı çıktı dizisini $A'[1..n]$ olarak belirtiriz. 3-ün-medyanı yöntemini kullanarak pivot elemanı 3'ü seçiniz, $p_i = \Pr\{x = A'[i]\}$ yi tanımlayın.

- a. $i = 2, 3, \dots, n-1$ için n nin bir fonksiyonu olarak p_i nin tam formülünü verin. ($p_1 = p_n = 0$ dır.)
- b. Normal bir uygulamaya göre $x = A'[\lfloor (n+1)/2 \rfloor]$, $A[1..n]$ in medyanı, pivot olarak seçme olasılığı hangi oranda arttırılmıştır? $n \rightarrow \infty$ olduğunu varsayın ve olasılığın sınırlandırma oranını verin.
- c. Eğer “iyi” bir ayrımı $n/3 \leq i \leq 2n/3$ olduğunda pivotu $x = A'[i]$ olarak seçmek olarak tanımlarsak iyi bir ayrım olasılığını normal bir uygulamayla karşılaştırdığımızda ne kadar arttırmış oluruz? (*İpucu:* Toplamı integral ile yakınsayınız).
- d. Hızlı sıralamanın $\Omega(n \lg n)$ çalışma zamanında 3-ün-medyanının sadece sabit faktörü etkilediğini tartışınız.

7-6 Aralıkların bulanık (fuzzy) sıralanması

Sayılarını tam olarak bilmediğimiz bir sıralama problemini düşünelim. Bunun yerine her sayı için gerçek çizgisinde ait olduğu bir aralığı biliriz. Bu bize verilen $a_i \leq b_i$ olduğu $[a_i, b_i]$ türünden n kapalı aralıktır. Bu aralıkları **bulanık sıralamak** isteriz; örneğin aralıkların bir $\langle i_1, i_2, \dots, i_n \rangle$ permütasyonu üretmeye $j = 1, 2, \dots, n$, için $c_1 \leq c_2 \leq \dots \leq c_n$ karşılayan $c_j \in [a_{i_j}, b_{i_j}]$ vardır.

- a. n tane aralığı bulanık sıralamak için bir rassal algoritma tasarlayın. Algoritmanız sol bitiş noktasını (a_i değeri) hızlı sıralayan bir algoritmanın genel yapısına sahip olmalıdır; ancak çalışma zamanını geliştirmek için üst üste gelen aralıkların avantajını kullanmalıdır (Aralıklar daha çok üst üste bindikçe aralıkların bulanık hızlı sıralaması gelişerek kolaylaşır. Algoritmanız üst üste gelen aralıkların avantajını kullanmalıdır).
- b. Algoritmanızın beklenen çalışma zamanının genelde $\Theta(n \lg n)$ olduğunu ancak bütün aralıklar üst üste geldiğinde $\Theta(n)$ olduğunu tartışınız (Örneğin bir x değeri varsa bütün i değerleri için $x \in [a_i, b_i]$ dir). Algoritmanız açıkça bu durumu kontrol etmez; bunun yerine performansı doğal olarak üst üste binme sayısı arttıkça gelişir.

Bölüm notları

Hızlı sıralama prosedürü Hoare [170] tarafından geliştirilmiştir; Hoare'nin sürümü Problem 7-1 de karşımıza çıkar. Kısım 7.1 de verilen PARTITION prosedürü N. Lomuto'nundur. Kısım 7.4 deki analiz Avrim Blum'undur. Sedgewick [305] ve Bentley [43] uygulamanın detayları ve nasıl farkettileri hakkında iyi referans sağlarlar.

McIlroy [248] neredeyse herhangi bir hızlı sıralama uygulamasının $\Theta(n^2)$ zaman alan bir dizi üreten “öldürücü düşmanın” (killer adversary) yapılabileceğini göstermektedir. Eğer uygulama rassalsa, düşman hızlı sıralama algoritmasının rassal seçimlerini gördükten sonra diziyi üretir.

n adet sayıyı $O(n \lg n)$ zamanda sıralayan çeşitli algoritmalar tanıttık. Birleştirerek sıralama ve bellek yığını sıralaması bu üst sınıra en kötü durumda ulaşır; hızlı sıralama ortalama sürede buna ulaşır. Dahası bu algoritmaların her biri için algoritmanın $\Omega(n \lg n)$ sürede çalışmasına sebep olan n girdi sayılı bir sıra üretebiliriz.

Bu algoritmalar ilginç bir özellik paylaşırlar: *belirledikleri sıralı düzen sadece girdi elemanları arasındaki karşılaştırmaya dayanır*. Bu tür sıralama algoritmaları **karşılaştırma sıralaması** olarak çağrılır. Şimdiye kadar tanıtılan algoritmaların tümü karşılaştırma algoritmalarıdır.

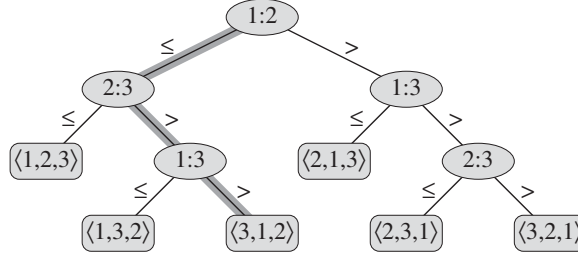
Kısım 8.1 de n sayıda elemanı karşılaştırarak sıralamak için en kötü durumda $\Omega(n \lg n)$ karşılaştırma yaptığımızı kanıtlamalıyız. Böylece birleştirerek sıralama ve bellek yığını sıralaması asimptotik olarak en uygun ve hiçbir karşılaştırma sıralaması sabit bir faktörden daha hızlı değildir.

Kısım 8.2, 8.3 ve 8.4 üç sıralama algoritması incelemektedir: doğrusal zamanda çalışan sayma sıralaması, taban sıralaması ve kova sıralaması. Bu sıralama algoritmaları tabi ki sıralı düzene karar vermek için karşılaştırma dışında işlemler kullanır. Sonuç olarak $\Omega(n \lg n)$ alt sınır onlara uygulanmaz.

8.1 Sıralama için alt sınırlar

Karşılaştırma sıralamasında, bir $\langle a_1, a_2, \dots, a_n \rangle$ girdi dizisi hakkında düzen bilgisi elde etmek için sadece elemanlar arasındaki karşılaştırmalar kullanılır. Verilen a_i ve a_j elemanları, göreceli düzene karar vermek için $a_i < a_j$, $a_i \leq a_j$, $a_i = a_j$, $a_i \geq a_j$ veya $a_i > a_j$ testlerinden birini gerçekleştirir. Başka hiçbir şekilde elemanların değerlerini inceleyemeyiz veya onlar hakkında düzen bilgisi elde edemeyiz.

Bu kısımda, bütün girdi elemanları farklı olan genelleme kaybı olmadığını varsayalım. $a_i = a_j$ biçimlerinin karşılaştırması kullanışsızdır varsayımı verildiğinde, bu biçimlerin hiçbir karşılaştırmasının yapılmadığını varsayalım. Ayrıca $a_i \leq a_j$, $a_i \geq a_j$, $a_i > a_j$, ve $a_i < a_j$ karşılaştırmalarının tümü a_i ve a_j nin göreceli



Şekil 8.1 Üç tane eleman üzerinde işlem yapan araya sokma sıralaması için karar ağacı. $i : j$ tarafından ek açıklama yapılmış iç düğüm, a_i ve a_j arasındaki bir karşılaştırmayı gösterir. $\langle \pi(1), \pi(2), \dots, \pi(n) \rangle$ permütasyonu tarafından ek açıklama yapılmış bir yaprak $a_{\pi(1)} \leq a_{\pi(2)} \leq \dots \leq a_{\pi(n)}$ sıralamasını belirtir. Gölgelendirilmiş yol, girdi dizisi $\langle a_1 = 6, a_2 = 8, a_3 = 5 \rangle$ sıralanırken yapılan kararı belirtir; yapraktaki $\langle 3, 1, 2 \rangle$ permütasyonu sıralanmış düzenin $a_3 = 5 \leq a_1 = 6 \leq a_2 = 8$ olduğunu belirtir. Burada girdi elemanlarının $3! = 6$ olası permütasyonu vardır, ve bu nedenle karar ağacı en az 6 yaprağa sahip olmalıdır.

düzenleri hakkında aynı bilgileri sağlayacak şekilde eşit olduğunu not ederiz. Bu nedenle tüm karşılaştırmaların $a_i \leq a_j$ biçimine sahip olduğunu varsayabiliriz.

Karar ağacı modeli

Karşılaştırma sıralamalarını karşılaştırma ağacı koşullarında soyut olarak görebiliriz. Bir **karar ağacı** verilen boyuttaki girdilerde işleyen çeşitli sıralama algoritmaları tarafından gerçekleştirilen elemanlar arasındaki karşılaştırmayı sunan tam ikili ağaçtır. Kontrol, veri akışı ve algoritmanın diğer tüm özellikleri yok sayılmıştır. Şekil 8.1, Kısım 2.1 den üç elemanlı bir girdi dizisi üzerinde işlem yapan araya sokma sıralama algoritmasına karşılık gelen karar ağacını göstermektedir.

Bir karar ağacında, her bir iç düğüme n in girdi dizisindeki eleman sayısı olduğu $1 \leq i, j \leq n$ aralığında bazı i ve j değerleri için $i : j$ ile ek açıklama koyarız. Ayrıca her yaprağa $\langle \pi(1), \pi(2), \dots, \pi(n) \rangle$ permütasyonu ile ek açıklama koyarız (Permutasyon hakkında genel bilgileri görmek için Kısım C.1 e bakınız). Sıralama algoritmasının çalışması, karar ağacının kök düğümünden bir yaprağa doğru basit bir yol izlemeye karşılık gelir. Her iç düğüm bir $a_i \leq a_j$ karşılaştırmasını belirtir. Sol altağaç $a_i \leq a_j$ olarak bildiğimiz daha sonraki karşılaştırmaları gerektirir ve sağ alt ağaç $a_i > a_j$ nin bilindiği sonraki karşılaştırmaları gerektirir. Bir yaprağa geldiğimizde, sıralama algoritması $a_{\pi(1)} \leq a_{\pi(2)} \leq \dots \leq a_{\pi(n)}$ sıralamasını oluşturmuştur. Herhangi bir doğru sıralama algoritması girdilerinin her bir permütasyonunu sağlamak zorunda olduğundan, n elemandaki her $n!$ permütasyonlarının her biri karşılaştırma sıralamasının doğru olması için karar ağacının yapraklarından biri gibi görünmek zorundadır. Dahası bu yaprakların her

biri karşılaştırma sıralamasının gerçek çalışmasına karşılık gelen kökten aşağı doğru inen yoldan ulaşılabilir olmalıdır (Bu tip yaprakları “ulaşılabilir” olarak kastederiz). Böylece sadece her permütasyonun ulaşılabilir bir yaprak olarak görülen karar ağaçlarını değerlendirmeliyiz.

En kötü durum için alt sınır

Bir karar ağacının kökünden ulaşılabilir yapraklara en uzun basit yolun uzunluğu, uygun sıralama algoritmasının gerçekleştirdiği en kötü-durum karşılaştırma sayısını temsil eder. Sonuç olarak, verilen bir karşılaştırma sıralaması algoritması için en kötü-durum karşılaştırma sayısı karar ağacının yüksekliğine eşittir. Bu nedenle her bir permütasyonun ulaşılabilir bir yaprak gibi görüldüğü bütün karar ağaçlarının yüksekliklerindeki bir alt sınır her karşılaştırma sıralaması algoritmasının çalışma zamanında bir alt sınırdır. Aşağıdaki teorem böyle bir alt sınırı oluşturur.

Teorem 8.1

Her bir karşılaştırma sıralama algoritması en kötü durumda $\Omega(n \lg n)$ karşılaştırma gerektirir.

Kanıt Önceki tartışmalardan, her permütasyonu ulaşılabilir bir yaprak olarak ortaya çıkan karar ağacı, ağacın yüksekliğini belirlemek için yeterlidir. l Ulaşılabilir yaprağı bir karşılaştırma sıralamasına karşılık gelen yüksekliği h olan n elemanlı bir karar ağacını ele alalım. Girdinin her bir $n!$ permütasyonu yaprak olarak ortaya çıktığından $n! \leq l$ e sahip oluruz. Yüksekliği h olan bir ikili ağaç 2^h den fazla yaprağa sahip olamayacağından $n! \leq l \leq 2^h$ ye sahip oluruz. Bunun logaritmasını alarak

$$\begin{aligned} h &\geq \lg(n!) \quad (\lg \text{ fonksiyonu monotonik olarak arttığından dolayı}) \\ &= \Omega(n \lg n) \quad (\text{Denklemler 3.19 dan}) \end{aligned}$$

elde ederiz. ■

Sonuç 8.2

Bellek yığını sıralaması ve birleştirerek sıralama asimptotik optimal karşılaştırma sıralamalarıdır.

Kanıt Bellek yığını sıralaması ve birleştirerek sıralamanın çalışma zamanındaki $O(n \lg n)$ üst sınırları, Teorem 8.1 deki $\Omega(n \lg n)$ en kötü-durum alt sınırıyla eşleşir. ■

Alıştırma

8.1-1

Bir karşılaştırma sıralaması için bir karar ağacında en küçük muhtemel yaprak derinliği nedir?

8.1-2

$\lg(n!)$ üzerinde asimptotik sıkı sınırları Stirling yaklaşımı kullanmadan elde ediniz. Bunun yerine Kısım A.2 deki tekniği kullanarak $\sum_{k=1}^n \lg k$ toplamını değerlendiriniz.

8.1-3

Uzunluğu n olan $n!$ girdinin en az yarısı için çalışma zamanı doğrusal olan hiç bir karşılaştırma sıralaması algoritması olmadığını gösteriniz. Uzunluğu n olan girdilerin $1/n$ kesri olursa ne olur? $1/2^n$ kesri olursa ne olur?

8.1-4

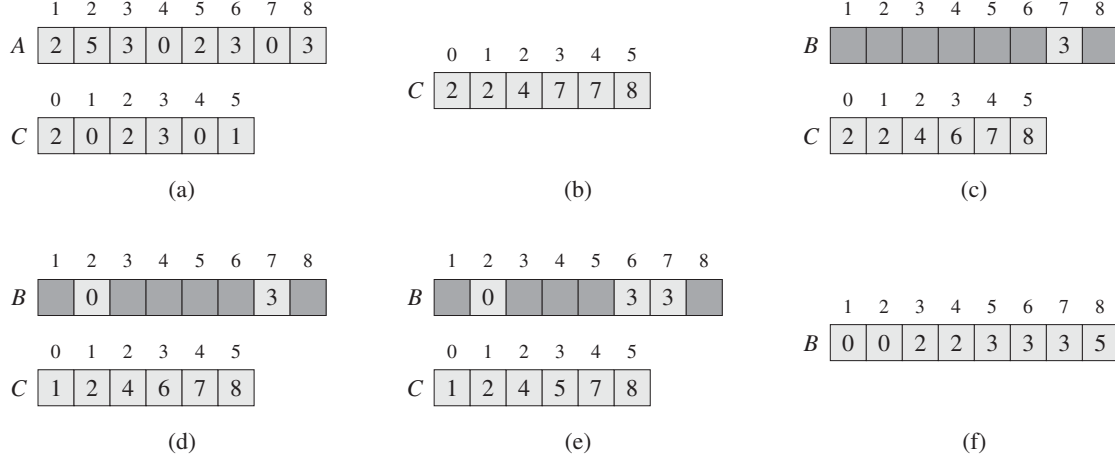
Size n elemanı olan bir dizinin sıralanmak için verildiğini varsayalım. Girdi dizisi n/k tane altdiziye sahiptir, her biri k eleman içerir. Verilen altdizideki elemanların hepsi sonra gelen altdizideki elemanlardan daha küçüktür ve öncesinde gelen alt dizi elemanlarından daha büyüktür. Bundan dolayı uzunluğu n olan bir dizinin tamamını sıralamak için tüm gereken n/k altdizisindeki her bir elemanı sıralamaktır. Sıralama probleminin bu türü için gereken karşılaştırma sayısında $\Omega(n \lg k)$ alt sınırı gösteriniz (*İpucu:* Bireysel altdiziler için basitçe alt sınırları birleştirmek sıkı değildir).

8.2 Sayma sıralaması

Sayma sıralaması n girdi elemanlarının her birinin $0 - k$ arasında bir tam sayı olduğunu varsayar. $k = O(n)$ olduğunda, sıralama $\Theta(n)$ zamanda çalışır.

Sayma sıralaması, herbir girdi elemanı x için, x den küçük eleman sayısını belirler. Bu bilgiyi x elemanını çıktığı dizisinde tam yerine koymak için kullanır. Örneğin eğer 17 tane eleman x den daha küçükse x in çıktığı dizisindeki yeri 18. pozisyonudur. Bu şemayı bir elemandan birkaç tane olduğu durumlarla başa çıkabilmek, aynı elemanları aynı pozisyona koymayı engellemek için biraz değiştirmemiz gerekmektedir.

Sayma sıralaması kodunda girdi bir $A[1..n]$ dizisidir ve bundan dolayı $A.length = n$ dir. Bunun dışında iki diziye daha ihtiyaç duyarız: dizi $B[1..n]$ sıralanmış çıktıyı tutar ve dizi $C[0..k]$ geçici çalışan depolama alanı sağlar.



Şekil 8.2 Her bir elemanı $k = 5$ den daha büyük olmayan pozitif tam sayı değerleri olan bir girdi dizisi $A[1..8]$ de COUNTING-SORT işlemi. **(a)** A dizisi ve 5. satırdan sonra yardımcı C dizisi. **(b)** 8. satırdan sonra C dizisi. **(c)-(e)** Çıktı dizisi B ve satır 10-12 arasındaki döngünün üç iterasyonundan sonra yardımcı dizi C . Sadece açık renkte gölgelendirilmiş B dizisinin elemanları doldurulmuştur. **(f)** Son çıktı dizisi B .

COUNTING-SORT(A, B, k)

```

1  let  $C[0..k]$  yeni bir dizi olsun
2  for  $i = 0$  to  $k$ 
3       $C[i] = 0$ 
4  for  $j = 1$  to  $A.length$ 
5       $C[A[j]] = C[A[j]] + 1$ 
6  //  $C[i]$  şimdi  $i$  ye eşit olan elemanların sayısını içeriyor.
7  for  $i = 1$  to  $k$ 
8       $C[i] = C[i] + C[i - 1]$ 
9  //  $C[i]$  şimdi  $i$  den küçük veya eşit olan elemanların sayısını içeriyor.
10 for  $j = A.length$  downto 1
11      $B[C[A[j]]] = A[j]$ 
12      $C[A[j]] = C[A[j]] - 1$ 

```

Şekil 8.2 sayma sıralamasını göstermektedir. Satır 2-3 teki **for** döngüsü C dizisinin bütün elemanlarını 0 a eşitledikten sonra, Satır 4-5 teki **for** döngüsü tüm girdi elemanlarını inceler. Eğer bir girdi elemanının değeri i ise $C[i]$ bir artırılır. Böylece 5. Satırdan sonra $C[i]$ her $i = 0, 1, \dots, k$ tam sayısı için i ye eşit olan girdi elemanlarının sayısını tutar. Satır 7-8 her $i = 0, 1, \dots, k$ için C dizisinin çalışan toplamını saklayarak kaç girdi elemanı i den küçük veya eşit kaç girdi elemanı olduğuna karar verir.

Son olarak Satır 10-12 deki **for** döngüsü her $A[j]$ elemanını çıktı dizisi B de doğru pozisyonuna yerleştirir. Eğer tüm n elemanları birbirinden farklıysa satır 10 a ilk geldiğimizde $A[j]$ den küçük veya eşit $C[A[j]]$ eleman olduğundan her $A[j]$ için $C[A[j]]$ değeri çıktı dizisindeki son doğru yeridir. Elemanlar birbirinden farklı olmayabileceğinden $C[A[j]]$ değerini B dizisine her $A[j]$ eklediğimizde azaltırız. $C[A[j]]$ yi azaltmak, $A[j]$ ye eşit bir sonraki girdi elemanın, eğer varsa, çıktı dizisinde $A[j]$ den hemen önceki pozisyona yerleşmesine neden olur.

Sayma sıralaması ne kadar zaman alır? Satır 2-3 deki **for** döngüsü $\Theta(k)$ kadar zaman alır, satır 4-5 deki **for** döngüsü $\Theta(n)$ kadar zaman alır, Satır 7-8'deki **for** döngüsü $\Theta(k)$ kadar zaman alır ve Satır 10-12 deki **for** döngüsü $\Theta(n)$ kadar zaman alır. Böylece ortalama zaman $\Theta(k + n)$ kadardır. Uygulamada sayma sıralamasını genelde $k = O(n)$ e sahip olduğumuzda çalışma zamanı $\Theta(n)$ olduğu durumda kullanırız.

Sayma sıralamasının alt $\Omega(n \lg n)$ in alt sınırını geçtiğini Kısım 8.1 de kanıtladık çünkü bir karşılaştırma sıralaması değildir. Aslında girdi elemanları arasında kodda hiçbir karşılaştırma yapılmaz. Bunun yerine, sayma sıralaması elemanların gerçek değerlerini bir diziye indislemek için kullanır. $\Omega(n \lg n)$ alt sınırı karşılaştırma sıralaması modelinden ayrıldığımızda sıralama için uygulanmaz.

Sayma sıralamasının önemli bir özelliği **kararlı** olmasıdır: Aynı değere sahip sayılar çıktı dizisinde girdi dizisindeki aynı düzende belirirler. Bu iki sayı arasındaki bağı hangi numara girdi dizisinde ilk belirirse çıktı dizisinde ilk belirir kuralıyla bozar. Normalde kararlılık özelliği sadece uydu verileri sıralanan elemanla birlikte taşınınca önemlidir. Sayma sıralamasının kararlılık özelliği başka bir nedenle de önemlidir: sayma sıralaması sık sık taban sıralaması için bir alt rutin olarak kullanılır. Bir sonraki kısımda göreceğimiz gibi taban sıralamasının düzgün çalışabilmesi için sayma sıralaması kararlı olmalıdır.

Alıştırmalar

8.2-1

Şekil 8.2 yi bir model olarak kullanarak $A = \langle 0, 2, 0, 1, 3, 4, 6, 1, 3, 2 \rangle$ dizisinde COUNTING-SORT işlemini gösteriniz.

8.2-2

COUNTING-SORT un kararlı olduğunu kanıtlayınız.

8.2-3

COUNTING-SORT ta 10. Satırdaki **for** döngüsünün başlığını

for $j = 1$ **to** $A.length$

olarak yeniden yazdığımızı varsayalım. Algoritmanın hala düzgün çalıştığını gösteriniz. Değiştirilmiş algoritma kararlı mıdır?

8.2-4

0 dan k ye aralığında verilen n değişkeni girdilerini önceden işleyen ve n tam sayının kaçının $[a..b]$ aralığına girdiği hakkındaki sorguları cevaplayan $O(1)$ sürede çalışan bir algoritma tanımlayınız. Algoritmanız $\Theta(n + k)$ işlem zamanı kullanmalı.

8.3 Taban sıralaması

Taban sıralaması şimdilerde sadece bilgisayar müzelerinde bulabileceğimiz kart sıralama makineleri tarafından kullanılan bir algoritmadır. Kartlarda 80 tane sütun bulunmaktadır ve her sütunda makine 12 yerden birinde delik açabilir. Sıralayıcı her bir kart destesindeki her kartın verilen sütunlarını incelemek ve kartı hangi yerinden delindiğine bağlı olarak 12 kutudan birine dağıtmak için mekanik olarak programlanmıştır. Daha sonra bir operatör kartları kutu kutu toplar böylece kartlar sıralı bir şekilde toplanmış olur.

Onluk haneler için her sütun sadece 10 yer kullanır (Diğer 2 yer nümerik olmayan karakterleri şifrelemek için ayrılmıştır). Bir d haneli sayı d kadar yer kaplardı. Kart sıralayıcı bir zamanda sadece bir sütuna bakabildiğinden d haneli n kartı sıralama problemi bir sıralama algoritması gerektirmektedir.

Sezgisel olarak sayıları en anlamlı (*significant*) hanesine göre sıralar, sonuç veren tüm kutuları özyineleyen şekilde sıralar ve sonra kart destelerini birleştirir. Ne yazık ki 10 kutudan 9 unda ki kartlar her bir kutunun sıralanabilmesi için bir kenara konmalıdır. Bu işlem kaydı tutulması gereken bir çok kart yığını oluşturur (Alistırma 8.3-5 e bakınız).

Taban sıralaması kart sıralama problemini—sezgilere aykırı bir şekilde—en az anlamlı olacak şekilde önce sıralayarak çözer. Algoritma daha sonra kartları önce sıfırıncı kutudaki kartlar sonra birinci kutudaki kartlar daha sonra ikinci kutudaki kartlar bu şekilde tek bir kart destesinde birleştirir. Daha sonra bütün kart destesini en az anlamlı ikinci haneyi dikkate alarak sıralar ve aynı şekilde kartları birleştirir. Bu işlem d haneli sayıların tümü sıralanana kadar devam eder. Önemli bir şekilde bu noktada tüm kartlar d haneli sayıda tam olarak sıralanmışlardır. Böylece sadece sıralama için d sayıda geçiş gereklidir. Şekil 8.3 yedi tane 3-haneli sayının bir kart destesinde taban sıralamasının nasıl işlediğini göstermektedir.

Taban sıralamasının doğru bir şekilde çalışması için hane sıralamaları kararlı olmalıdır. Bir kart sıralayıcı tarafından yapılan sıralama kararlıdır; ancak operatör bir kutu içindeki tüm kartlar seçilen sütunda aynı haneye sahip olsalar bile kartlar kutudan çıktığındaki gibi kartların sırasını değiştirmemeye dikkat etmelidir.

329	720	720	329
457	355	329	355
657	436	436	436
839	457	839	457
436	657	355	657
720	329	457	720
355	839	657	839

Şekil 8.3 Yedi adet 3-haneli sayıların listesinde taban sıralaması işlemi. En soldaki sütun girdidir. Geri kalan sütunlar listenin başarılı bir sıralamadan sonra artan şekilde önemli hanelerde göstermektedir. Gölgelendirmeler bir önceki sıralamada üretilmiş listedeki sıralanmış hane pozisyonunu göstermektedir.

Ard arda rassal-erişim makinesi olan tipik bir bilgisayarda, taban sıralamasını bazen çoklu alanlar tarafından anahtarlandırılmış bilgi kayıtlarını sıralamak için kullanırız. Örneğin, tarihleri Üç anahtara göre sıralamak isteyebiliriz: Yıl, ay ve gün. Verilen iki tarihte yılları karşılaştırır ve eğer aynıysa ayları karşılaştırır ve aylar da aynıysa günleri karşılaştıran bir karşılaştırma fonksiyonu ile bir sıralama algoritmasını çalıştırabiliriz. Alternatif olarak bilgiyi üç kere kararlı sıralama ile sıralayabilirdik: önce gün, sonra ay ve en son yıl olarak.

Taban sıralamasının kodu basittir. Sıradaki prosedür n elemanlı A dizisinde her elemanın 1 in en düşük seviyesi d nin en yüksek seviyesi olan d hanesi olduğunu varsayar.

RADIX-SORT (A, d)

- 1 **for** $i = 1$ **to** d
- 2 A dizisini i hanesinde sıralamak için kararlı sıralamayı kullan

Lemma 8.3

Her bir hanenin k farklı muhtemel değer aldığı verilen n tane d -haneli sayıyı RADIX-SORT doğru bir şekilde $\Theta(d(n + k))$ sürede, eğer dengeli sıralama kullanıyorsa $\Theta(n + k)$ sürede gerçekleştirir.

Kanıt Taban sıralamasının doğruluğu sıralanan sütunda tümevarım tarafından izlenir (Alistırma 8.3 ü görün). Çalışma zamanı analizi orta düzeyde sıralama algoritması olarak kullanılan kararlı sıralamaya bağlıdır. Her hane $0-(k-1)$ aralığındayken (böylece k farklı değer alır) ve k çok büyük değilken sayma sıralaması bariz bir seçimdir. n tane d hane üzerinden geçmek $\Theta(n + k)$ zaman alır. Burada d geçiş vardır ve taban sıralaması için gereken toplam süre $\Theta(d(n + k))$ kadardır. ■

d sabit olduğunda ve $k = O(n)$ ise taban sıralaması düzlemsel zamanda çalışır. Daha genel olarak her bir anahtarı hanelere nasıl ayıracağımız ile ilgili bazı esnekliğe sahibiz.

Lemma 8.4

Verilen n tane b -bitlik sayı ve herhangi bir pozitif tam sayı $r \leq b$ için RADIX-SORT bu sayıları doğru bir biçimde $\Theta((b/r)(n + 2^r))$ zamanda sıralar. Eğer kararlı sıralama kullanırsa $\Theta(n + k)$ zamanda $0 - k$ girdi aralığında sıralama yapar.

Kanıt Bir $r \leq b$ değeri için her anahtar r bitin her birinin $d = \lceil b/r \rceil$ hanesine sahipmiş gibi görürüz. Her bit 0 ile $2^r - 1$ aralığında bir tam sayıdır. Böylece sayma sıralamasını $k = 2^r - 1$ ile kullanabiliriz (Örneğin 32 bitlik bir kelimeyi dört tane 8-bitlik hane gibi görebiliriz, böylece $b = 32, r = 8, k = 2^r - 1 = 255$ ve $d = b/r = 4$). Sayma sıralamasının her geçişi $\Theta(n + k) = \Theta(n + 2^r)$ zaman alır ve d geçiş için toplam çalışma zamanı $\Theta(d(n + 2^r)) = \Theta((b/r)(n + 2^r))$ dir. ■

Verilen n ve b değerleri için r nin değerini $(b/r)(n + 2^r)$ ifadesine azaltacak şekilde seçmek isteriz. Eğer $b < \lfloor \lg n \rfloor$ ise; $r \leq b$ nin herhangi bir değeri için $(n + 2^r) = \Theta(n)$ e sahip oluruz. Böylece $r = b$ yi seçmek asimptotik optimal $(b/b)(n + 2^b) = \Theta(n)$ çalışma zamanını olmasını sağlar. Eğer $b \geq \lfloor \lg n \rfloor$ ise $r = \lfloor \lg n \rfloor$ yi seçmek sırada göreceğimiz gibi sabit faktör içinde en iyi zamanı verir. $r = \lfloor \lg n \rfloor$ yi seçmek çalışma zamanının $\Theta(bn / \lg n)$ olmasını sağlar. r yi $\lfloor \lg n \rfloor$ üzerinde arttırdığımızda paydaki 2^r terimi paydadaki r teriminden daha hızlı artar ve böylece r yi $\lfloor \lg n \rfloor$ üzerinde arttırmak $\Omega(bn / \lg n)$ çalışma zamanının olmasını sağlar. Eğer bunun yerine r yi $\lfloor \lg n \rfloor$ nin altına azaltsaydık b/r terimi artardı ve $n + 2^r$ terimi $\Theta(n)$ de kalırdı.

Buket sıralaması hızlı sıralama gibi bir karşılaştırma tabanlı sıralamaya tercih edilir mi? Eğer $b = O(\lg n)$ ise, genelde bu şekilde olduğu gibi ve $r \approx \lg n$ seçersek taban sıralamasının çalışma zamanı hızlı sıralamanın beklenen çalışma zamanı $\Theta(n \lg n)$ den daha iyi olduğu gözlenen n çalışma zamanına sahiptir. Ancak Θ -notasyonunda gizli olan sabit faktörü farklılık gösterir. Taban sıralaması n anahtar üzerinde hızlı sıralamadan daha az geçiş yapmasına rağmen taban sıralamasının her geçişi belirgin bir şekilde daha uzundur. Hangi sıralama algoritmasını seçeceğimiz uygulamanın karakteristiğine ve altında yatan makinenin (örneğin hızlı sıralama donanım önbelleğini taban sıralamasından daha etkili kullanır) girdi verisine bağlıdır. Dahası sayma sıralamasını orta seviye kararlı sıralama olarak kullanan taban sıralaması sürümü yerinde sıralama yapmaz. Böylece birincil bellek deposu en üst seviyedeysen hızlı sıralama gibi yerinde sıralama algoritması tercih ederiz.

Alıştırmalar**8.3-1**

Şekil 8.3 ü model olarak kullanarak RADIX-SORT işlemini takip eden İngilizce kelime listesinde gösteriniz: COW, DOG, SEA, RUG, ROW, MOB, BOX, TAB, BAR, EAR, TAR, DIG, BIG, TEA, NOW, FOX.

8.3-2

Verilen sıralama algoritmalarından hangileri kararlıdır: araya sokma sıralaması, birleştirerek sıralama, bellek yığını sıralaması ve hızlı sıralama? Herhangi bir sıralama algoritmasını kararlı yapan bir şema veriniz. Şemanız fazladan ne kadar çalışma zamanı ve alan (space) gerektirdi?

8.3-3

Taban sıralamasının çalışmasını kanıtlamak için tümevarım kullanınız. Kanıtınız orta sıralamanın kararlı olduğu varsayımına nerede ihtiyaç duydu?

8.3-4

n tam sayıyı $0-(n^3 - 1)$ aralığında $O(n)$ zamanda nasıl sıraladığını gösteriniz.

8.3-5 *

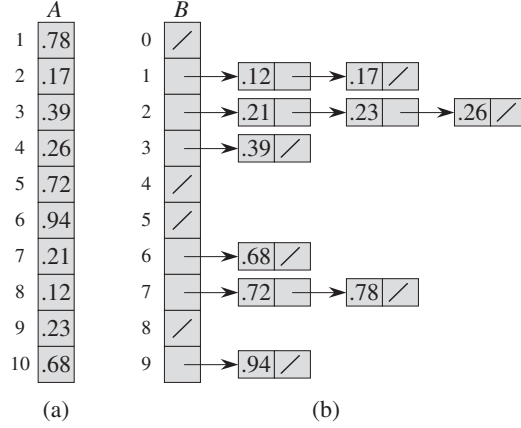
Bu kısımdaki ilk kart sıralama algoritmasında d haneli on tabanlı bir sayıyı en kötü durumda sıralamak için tam olarak kaç sıralama geçişine ihtiyaç vardır? En kötü durumda operatör kaç kart yığınının kaydını tutmaya ihtiyacı vardır?

8.4 Kova sıralaması

Kova sıralaması girdilerin düzgün (uniform) dağılımından çekildiğini ve ortalama durum çalışma zamanının $O(n)$ olduğunu varsayar. Sayma sıralaması gibi kova sıralaması da hızlıdır çünkü girdiler hakkında birşeyler tahmin eder. Sayma sıralaması küçük aralıktaki tam sayılardan oluştuğunu varsayarken kova sıralaması girdilerin $[0, 1)$ aralığı üzerinde elemanları düzgün ve bağımsız bir şekilde dağıtan rassal bir işlem ile yaratıldığını varsayar (Düzgün dağılımın tanımı görmek için Kısım 2.2 ye gidin).

Kova sıralaması $[0, 1)$ aralığını n eşit boyutlu alt aralıklara, veya **kovalara**, böler ve sonra n girdi sayısını kovalara dağıtır. Sayılar $[0, 1)$ aralığında düzgün ve bağımsız bir biçimde dağıtıldığından bir çok sayının tüm kovalara düşmemesini bekleriz. Çıktıyı üretmek için her kovadaki elemanları basitçe sıralarız ve sonra kovalar üzerinden sırayla her kovadaki sayıyı listeleyerek geçeriz.

Kova sıralaması kodu için girdimiz n -elemanlı A dizisidir ve her A dizisindeki her $A[i]$ elemanı $0 \leq A[i] < 1$ i sağlar. Kod bağlı listenin listelerin (kovaların) yardımcı dizisi $B[0..n-1]$ ni gerektirir ve böyle bir listeyi sağlayacak bir mekanizma olduğunu varsayar (Kısım 10.2 bağlı listede basit işlemlerin nasıl uygulanacağını açıklar).



Şekil 8.4 $n = 10$ için kova sıralaması işlemi. (a) Girdi dizisi $A[1..10]$ (b) Algoritmanın 8. Satırından sonraki sıralı listenin $B[0..9]$ dizisi. i kovası yarı-açık aralıktaki $[i/10, (i+1)/10)$. Sıralanmış çıktı $B[0], B[1], \dots, B[9]$ listelerinin ard arda eklenmelerini içermektedir.

BUCKET-SORT (A)

```

1   $n = A.length$ 
2   $B[0..n-1]$  yeni bir dizi olsun
3  for  $i = 0$  to  $n-1$ 
4       $B[i]$  yi boş liste yap bir dizi olsun
5  for  $i = 1$  to  $n$ 
6       $A[i]$  yi  $B[\lfloor nA[i] \rfloor]$  listesine ekle
7  for  $i = 0$  to  $n-1$ 
8       $B[i]$  listesini araya sokma sıralamasıyla sırala
9   $B[0], B[1], \dots, B[n-1]$  listelerini sıralı olarak ard arda ekle

```

Şekil 8.4 10 elemanlı bir girdi dizisinde kova sıralamasının işlemlerini göstermektedir.

Bu algoritmanın çalıştığını görmek için herhangi iki $A[i]$ ve $A[j]$ elemanını düşünün. $A[i] \leq A[j]$ herhangi bir genelleme kaybetmediğini varsayın. $\lfloor nA[i] \rfloor \leq \lfloor nA[j] \rfloor$ olduğundan ya $A[i]$ $A[j]$ ile aynı kovaya gider ya da daha düşük indisli kovaya gider. Eğer $A[i]$ ve $A[j]$ aynı kovaya giderlerse Satır 7-8'deki for döngüsü onları uygun yerlere koyar. Eğer $A[i]$ ve $A[j]$ farklı kovalara giderlerse satır 9 onları uygun yerlere koyar. Böylece kova sıralaması düzgün çalışır.

Çalışma zamanını analiz etmek için satır 8 dışındaki tüm satırların en kötü durumda $O(n)$ zamanda çalıştığını gözlemledik. Satır 8'deki ekleme sıralamasına yapılan n çağrının aldığı toplam zamanı analiz etmeliyiz.

Araya sokma sıralamasına yapılan çağrının maliyetini analiz etmek için, n_i kova $B[i]$ nin içindeki konmuş eleman sayısını belirten bir rassal değişken olsun. Araya sokma sıralaması ikinci dereceden zamanda çalıştığından (Kısım 2.2 ye bakınız) kova sıralamasının çalışma zamanı sıradaki gibidir:

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2) .$$

Şimdi kova sıralamasının ortalama durum çalışma zamanını, çalışma zamanının beklenen değerini hesaplayarak, girdi dağılımı üzerindeki beklentiye alarak analiz edeceğiz. İki tarafın da beklentisini alarak ve beklentinin doğrusallığını kullanarak

$$\begin{aligned} E[T(n)] &= E \left[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2) \right] \\ &= \Theta(n) + \sum_{i=0}^{n-1} E[O(n_i^2)] \quad (\text{Beklentinin doğrusallığından}) \\ &= \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2]) \quad (\text{Denklem C.22 den}) \end{aligned} \tag{8.1}$$

elde ederiz. $i = 0, 1, \dots, n-1$ için

$$E[n_i^2] = 2 - 1/n \tag{8.2}$$

olduğunu varsaydık. Girdi dizisi A daki her değerin herhangi bir kovaya düşmesi eşit olasılıklı olduğundan her kova i nin aynı $E[n_i^2]$ değerine sahip olması sürpriz değildir. Denklem 8.2 yi kanıtlamak için $i = 0, 1, \dots, n-1$ ve $j = 1, 2, \dots, n$ olduğunda rassal gösterge değişkeni $X_{ij} = I \{A[j], i \text{ kovaasına düşmüştür}\}$ yi tanımlarız}. Böylece

$$n_i = \sum_{j=1}^n X_{ij}$$

olur.

$E[n_i^2]$ yi hesaplamak için sıradaki kareleri genişletir ve ifadeleri yeniden gruplandırırız:

$$\begin{aligned}
E[n_i^2] &= E\left[\left(\sum_{j=1}^n X_{ij}\right)^2\right] \\
&= E\left[\sum_{j=1}^n \sum_{k=1}^n X_{ij} X_{ik}\right] \\
&= E\left[\sum_{j=1}^n X_{ij}^2 + \sum_{1 \leq j \leq n} \sum_{\substack{1 \leq k \leq n \\ k \neq j}} X_{ij} X_{ik}\right] \\
&= \sum_{j=1}^n E[X_{ij}^2] + \sum_{1 \leq j \leq n} \sum_{\substack{1 \leq k \leq n \\ k \neq j}} E[X_{ij} X_{ik}] , \tag{8.3}
\end{aligned}$$

burada son satır beklentinin doğrusallığı tarafından takip edilir. İki toplamayı ayrı olarak değerlendiririz. Rassal gösterge değişkeni X_{ij} $1/n$ olasılığıyla 1 dir, diğer türlü 0 dir; bu nedenle

$$\begin{aligned}
E[X_{ij}^2] &= 1^2 \cdot \frac{1}{n} + 0^2 \cdot \left(1 - \frac{1}{n}\right) \\
&= \frac{1}{n} .
\end{aligned}$$

$k \neq j$ olduğunda X_{ij} ve X_{ji} değişkenleri bağımsızdır ve bu nedenle;

$$\begin{aligned}
E[X_{ij} X_{ik}] &= E[X_{ij}] E[X_{ik}] \\
&= \frac{1}{n} \cdot \frac{1}{n} \\
&= \frac{1}{n^2} .
\end{aligned}$$

Denklem 8.3 deki 2 beklenen değeri yerine koyduğumuzda Denklem 8.2 yi kanıtlayan aşağıdaki ifadeyi elde ederiz:

$$\begin{aligned}
E[n_i^2] &= \sum_{j=1}^n \frac{1}{n} + \sum_{1 \leq j \leq n} \sum_{\substack{1 \leq k \leq n \\ k \neq j}} \frac{1}{n^2} \\
&= n \cdot \frac{1}{n} + n(n-1) \cdot \frac{1}{n^2} \\
&= 1 + \frac{n-1}{n} \\
&= 2 - \frac{1}{n} .
\end{aligned}$$

Denklem 8.1 deki bu beklenen değeri kullanarak kova sıralamasının ortalama durum çalışma süresinin $\Theta(n) + n \cdot O(2 - 1/n) = \Theta(n)$ olduğu sonucuna vardık.

Girdiler düzgün dağılımdan çekilmese bile kova sıralaması yine de doğrusal zamanda çalışır. Girdi toplam eleman sayısında kova boyutunun karelerinin toplamının doğrusal olma özelliğine sahip olduğu sürece Denklem 8.1 bize kova algoritmasının doğrusal zamanda çalıştığını anlatır.

Alıştırmalar

8.4-1

Şekil 8.4 ü model olarak kullanarak $A = \langle .79, .13, .16, .64, .39, .20, .89, .53, .71, .42 \rangle$ dizisinde kova sıralamasını gösteriniz.

8.4-2

Kova sıralamasının en kötü-durum çalışma zamanının neden $\Theta(n^2)$ olduğunu açıklayınız. Algoritmaya yapılan hangi basit değişiklik doğrusal ortalama çalışma zamanını korur ve en kötü-durum çalışma zamanını $O(n \lg n)$ yapar?

8.4-3

X iki kere atılmış hilesiz bir madeni paranın tura gelme sayısına eşit rassal değişken olsun. $E[X^2]$ ne olur? $E^2[X]$ ne olur?

8.4-4 *

Birim daire $p_i = (x_i, y_i)$ 'de $i = 1, 2, \dots, n$ için $0 < x_i^2 + y_i^2 \leq 1$ olacak şekilde bize n nokta verilmiştir. Noktaların eşit oranda dağıtıldığını varsayalım; yani dairenin herhangi bir bölgesinde bir nokta bulmanın olasılığı bu bölgenin alanıyla orantılıdır. Ortalama durum çalışma zamanı $\Theta(n)$ olan orijin noktasına olan uzaklıkları $d_i = \sqrt{x_i^2 + y_i^2}$ olan n noktayı sıralamak için bir algoritma tasarlayın (*İpucu*: BUCKET-SORT da kova boyutlarını birim dairede noktaların eşit oranlı dağılımını yansıtacak şekilde tasarlayın).

8.4-5 *

Bir rassal değişken X için *olasılık dağılımı fonksiyonu* $P(x) = \Pr\{X \leq x\}$ tarafından tanımlanmıştır. $O(1)$ zamanda hesaplanan devam eden olasılık dağılım fonksiyonundan n rassal değişken $X_1, X_2, \dots, X_n$ in listesini yazdığımızı varsayalım. Bu sayıları doğrusal ortalama durumda sıralayacak bir algoritma veriniz.

Problemler

8-1 Karşılaştırma sıralamasında (comparison sorting) olası alt sınırlar (lower bounds)

Bu problemde, n sayıda belli girdi elemanının herhangi bir kararlı (deterministic) ya da rassal (randomized) olarak yapılan karşılaştırma sıralaması sürecinde bir olası $\Omega(n \lg n)$ alt sınırını kanıtıyoruz. Kararlı bir A karşılaştırma sıralamasını, T_A karar ağacıyla (decision tree) inceleyerek başlıyoruz. A nın girdilerinin her permütasyonunun eşit olasılıklı (equally likely) olduğunu varsayıyoruz.

- a. T_A nın her bir yaprağının (leaf) rastgele bir girdi verilerek ulaşılmış birer olasılıkla etiketlendiğini düşünün. Tam olarak $n!$ yaprağın $1/n!$ şeklinde etiketlendiğini ve kalanının 0 olarak etiketlendiğini kanıtlayın.
- b. $D(T)$ bir T karar ağacının dış yol uzunluğu (external path length) olsun; öyle ki, $D(T)$ T nin bütün yapraklarının borçlarının toplamını göstere. T , $k > 1$ uzantıya sahip bir karar ağacı olsun ve LT ve RT T nin sağ ve sol alt ağaçları olsun. $D(T) = D(LT) + D(RT) + k$ yı gösterin.
- c. $d(k)$, $k > 1$ yapraklarına sahip bütün T karar ağaçlarında $D(T)$ nin en küçük değeri olsun. $d(k) = \min_{1 \leq i \leq k-1} \{d(i) + d(k-i) + k\}$ yi gösterin. (İpucu: k sayıda yaprağı olan ve minimumu sağlayan bir T karar ağacı düşünün. i_0 LT deki ve $k - i_0$ RT deki yaprak sayısı olsun.)
- d. $k > 1$ in belirli bir değeri ve i nin $1 \leq i \leq k - 1$ aralığında olduğu bir durum için, $i \lg i + (k - i) \lg(k - i)$ fonksiyonunun $i = k/2$ seviyesine minimize edildiğini kanıtlayın. $d(k) = \Omega(k \lg k)$ şeklinde sonuçlandırın.
- e. $D(T_A) = \Omega(n! \lg(n!))$ ı kanıtlayın ve n sayıda elemanı sıralamanın ortalama süresinin $\Omega(n \lg n)$ olduğunu gösterin.

Şimdi bir B rassal karşılaştırma sıralamasını düşünelim. Karar ağacı modelini iki çeşit düğümü birleştirerek *randomizasyonu* (randomization) karşılayacak şekilde genişletebiliriz: sıradan karşılaştırma düğümleri ve ‘randomizasyon’ düğümleri. Bir randomizasyon düğümü B algoritmasıyla yapılmış RANDOM $(1, r)$ biçimindeki rassal bir seçimi modeller; devrenin algoritmanın uygulanması sırasında her bir eşit seçim şansına sahip r sayıda çocuğu vardır.

- f. Bütün rassal B karşılaştırma sıralamaları için beklenen karşılaştırmaları B tarafından yapılandırılan daha fazla olmayan kararlı bir A karşılaştırma sıralaması olduğunu gösterin.

8-2 Doğrusal zamanda yerinde sıralama (sorting in place in linear time)

Sıralamamız gereken n sayıda veri kaydının olduğu bir dizimiz olduğunu ve her bir kayıt anahtarının 0 ya da 1 değerine sahip olduğunu düşünün. Bu tarz bir kayıt kümesini sıralayacak bir algoritma aşağıda istenen üç karakteristikten bir kısmına sahip olabilir:

1. Algoritma $O(n)$ zamanda çalışır.
 2. Algoritma kararlıdır.
 3. Algoritma orijinal diziye ek olarak sabit bir bellek miktarından fazlasını kullanmayacak şekilde yerinde sıralama yapar.
- a. 1. ve 2. kriterleri sağlayan bir algoritma verin.
 - b. 1. ve 3. kriterleri sağlayan bir algoritma verin.
 - c. 2. ve 3. kriterleri sağlayan bir algoritma verin.
 - d. RADIX-SORT un n sayıdaki kaydı $O(bn)$ zamanda b -bit anahtarlarıyla sıralayacağı şekilde, yukarıdaki algoritmalarından herhangi birisini RADIX-SORT un 2. satırında kullanılan sıralama yöntemi olarak kullanabilir misiniz? Nasıl yapabileceğinizi ya da neden yapamayacağınızı açıklayın.
 - e. n sayıdaki kaydın 1 ile k arasında anahtarlara sahip olduğunu düşünün. Sayma sıralamasını kayıtları $O(n + k)$ zamanda yerinde sıralayacak şekilde nasıl düzenleyeceğinizi gösterin. $O(k)$ belleğini girdi dizisi dışında kullanabilirsiniz. Algoritmanız sabit midir? (İpucu: $k = 3$) için bunu nasıl yaptınız?)

8-3 Değişken-uzunluk öğelerini sıralama

- a. Farklı tam sayıların farklı basamak sayılarına sahip olabileceği ama dizideki tüm tam sayıların üzerindeki toplam basamak sayısının n olduğu bir diziniz var. Dizin $O(n)$ zamanda nasıl sıralanacağını gösterin.
- b. Farklı dizgilerin farklı karakter sayılarına sahip olabileceği ama tüm dizgiler üzerindeki toplam karakter sayısının n olduğu bir diziniz var. Dizin $O(n)$ zamanda nasıl sıralanacağını gösterin.
(Beklenen sıralamanın alfabetik sıralama olduğunu unutmayın; örneğin, $a < ab < b.$)

8-4 Su testileri

Her biri farklı şekil ve boyutta n sayıda kırmızı ve n sayıda mavi su testileriniz olduğunu düşünün. Bütün kırmızı testilerde farklı miktarlarda su var, aynı şekilde bütün mavi testilerde de öyle. Üstelik, her kırmızı testi için, aynı miktarda su taşıyan bir mavi testi ve her mavi testi için, aynı miktarda su taşıyan bir kırmızı testi de mevcuttur.

Göreviniz aynı miktarda su taşıyan testleri eşleştirecek gruplamalar yapmaktır. Bunu başarmak için şu uygulamayı yapabilirsiniz: birer kırmızı ve mavi testi alın, kırmızı testi suyla doldurup sonra bu suyu mavi testie boşaltın. Bu size kırmızı ve mavi testinin aynı miktarda su alıp almadığını ya da birinin diğerinden daha büyük ya da küçük olduğunu gösterecektir. Bu uygulamanın bir birim zaman aldığını düşünün. Hedefiniz bu gruplamayı bitirmek için gereken en az sayıdaki karşılaştırmayı sağlayacak bir algoritma bulmak. Direkt olarak iki mavi ya da iki kırmızı testi kıyaslayamayacağınızı unutmayın.

- a. Testleri gruplandırmak için $\Theta(n^2)$ karşılaştırmasını kullanan kararlı (deterministik) bir algoritma tanımlayın.
- b. Bu problemi çözen bir algoritmanın yapması gereken karşılaştırma sayısı için bir $\Omega(n \lg n)$ alt sınırı gösterin.
- c. Beklenen karşılaştırma sayısı $O(n \lg n)$ olan rassal bir algoritma gösterin ve bu sınırın doğru olduğunu kanıtlayın. Algoritmanızın en kötü durumdaki karşılaştırma sayısı kaçtır?

8-5 Ortalama sıralama (Average sorting)

Bir dizinin sıralanması yerine, elemanların ortalama artış göstermesini istediğimizi düşünün. Daha kesin konuşursak, n elemanlı bir A dizisini, eğer aşağıdakileri tüm $i = 1, 2, \dots, n - k$ lar için sağlarsa ***k-sıralanmış*** olarak adlandırıyoruz:

$$\frac{\sum_{j=i}^{i+k-1} A[j]}{k} \leq \frac{\sum_{j=i+1}^{i+k} A[j]}{k}$$

- a. Bir dizi için 1-sıralanmış ne demektir?
- b. 1, 2, ..., 10 sayıları için 2-sıralanmış fakat sıralı olmayan bir permütasyon belirtiniz.
- c. n elemanlı bir dizinin ancak ve ancak bütün $i = 1, 2, \dots, n - k$ lar $A[i] \leq A[i + k]$ ise k -sıralanmış olduğunu kanıtlayınız.
- d. Bir n elemanlı diziyi $O(n \lg(n/k))$ süresinde k -sıralayan bir algoritma belirtiniz.

k nın sabit olduğu durumlarda bir k -sıralanmış dizi yaratmak için zamanda bir alt sınır da gösterebiliriz.

- e. $O(n \lg k)$ zamanda n uzunluktaki bir diziyi k -sıralanmış yapabileceğimizi gösterin. (*İpucu:* Alıştırma 6.5-9 daki çözümü kullanın).
- f. k nın sabit olması durumunda, n elemanlı bir diziyi k -sıralamanın $\Omega(n \lg n)$ zaman gerektirdiğini gösterin. (*İpucu:* karşılaştırma sıralamasında alt sınır konusunun anlatıldığı bir önceki kısımdaki çözümü kullanın.)

8-6 Sıralı listelerin birleştirilmesinde (merging sorted lists) alt sınır

Sıralı iki listenin birleştirilmesi problemiyle sıkça karşılaşılır. Kısım 2.3.1 de MERGE altprogramı isimli bir prosedür görmüştük. Bu problemde, her biri n madde içeren iki sıralı listenin birleştirilmesi için en kötü durumda gerekecek karşılaştırma sayısını $2n - 1$ alt sınırıyla kanıtlayacağız.

Öncelikle, bir karar ağacı kullanarak $2n - O(n)$ sayıdaki karşılaştırmaların en alt sınırını göstereceğiz.

- a. Elinizdeki $2n$ sayıyı her biri n sayı içerecek iki sıralı listeye bölebileceğiniz muhtemel yol sayısını hesaplayın.
- b. Bir karar ağacı ve (a) sorusuna verdiğiniz cevabı kullanarak, sıralı iki listeyi doğru şekilde birleştiren tüm algoritmaların en az $2n - O(n)$ sayıda karşılaştırma yapması gerekeceğini gösterin.

Şimdi biraz daha sıkı bir $2n - 1$ sınırı göstereceğiz.

- c. Eğer iki eleman farklı listelerden geliyorlar ve sıralı haldeler iken ardışıklarsa onları karşılaştırmamız gerektiğini gösterin.
- d. Üstteki cevabınızı iki listeyi birleştirmek için $2n - 1$ karşılaştırmasının alt sınırını göstermek için kullanın.

8-7 0-1 Sıralama lemması ve sütun sıralaması (columnsort)

$i < j$ olduğu durumda, $A[i]$ ve $A[j]$ şeklindeki iki dizi elemanı arasındaki bir karşılaştırma-değişimi uygulaması aşağıdaki şekildedir:

COMPARE-EXCHANGE (A, i, j)

- 1 **if** $A[i] > A[j]$
- 2 $A[i]$ ile $A[j]$ yi değiştir

Karşılaştırma-değişimi uygulamasından sonra $A[i] \leq A[j]$ olduğunu biliyoruz.

Kayıtsız karşılaştırma-değişimi algoritması sadece önceden belirlenmiş bir karşılaştırma-değişimi uygulamalar dizisiyle çalışır. Dizide kıyaslanmış durum indisleri önceden belirlenmiş olmalıdır ve her ne kadar sıralı eleman sayısına bağlı olsalar da, ne sıralanan değerlere ne de daha önce yapılmış herhangi bir karşılaştırma-değişimi sonucuna bağlı değillerdir. Örneğin, kayıtsız karşılaştırma-değişimi algoritması olarak ifade edilen bir araya sokma sıralaması aşağıdaki gibidir:

INSERTION-SORT (A)

- 1 **for** $j = 2$ **to** $A.length$
- 2 **for** $i = j - 1$ **downto** 1
- 3 COMPARE-EXCHANGE($A, i, i + 1$)

0-1 sıralama lemması, kayıtsız karşılaştırma-değişimi algoritmasının sıralı bir sonuç ürettiğini kanıtlamanın etkili bir yöntemidir. Bu yöneme göre, eğer kayıtsız karşılaştırma-değişimi algoritması 0 lar ve 1 lerden oluşan bütün girdi dizilerini düzgün bir şekilde sıralarsa keyfi değerleri içeren bütün girdileri de düzgün bir şekilde sıralar.

Burada, aksini kullanarak 0-1 sıralama lemmasını kanıtlayacaksınız: eğer bir kayıtsız karşılaştırma-değişimi algoritması rastgele değerleri içeren bir girdiyi sıralamakta başarısız olursa o zaman 0-1 girdisinin sıralamasında da başarısız olacaktır. Kayıtsız bir X karşılaştırma-değişimi algoritmasının $A[1..n]$ dizisini sıralayamadığını düşünün. $A[p]$ X algoritmasının yanlış yere koyduğu A nın en küçük değeri ve $A[q]$ X algoritmasının $A[p]$ nin olması gerektiği yere koyduğu yanlış değer olsun. 0 lar ve 1 lerden oluşan bir $B[1..n]$ dizisini aşağıdaki gibi ifade edin:

$$B[i] = \begin{cases} 0 & A[i] \leq A[p] \text{ ise,} \\ 1 & A[i] > A[p] \text{ ise.} \end{cases}$$

- a. $A[q] > A[p]$, dolayısıyla $B[p] = 0$ ve $B[q] = 1$ dizisini tartışın.
- b. 0-1 sıralama lemmasının kanıtını tamamlamak için X algoritmasının B dizisini düzgün bir şekilde sıralayamadığını kanıtlayın.

Şimdi 0-1 sıralama lemmasını belirli bir sıralama algoritmasının düzgün çalıştığını kanıtlamak için kullanacaksınız. Algoritma, **sütun sıralaması**, n sayıda elemanın oluşturduğu dikdörtgen bir dizide çalışır. Dizinin r satır s sütunu (böylece $n = rs$) ve üç kısıtlaması vardır:

- r çift sayı olmalıdır
- s r nin tam böleni olmalıdır, ve
- $r \geq 2s^2$

Sütun sıralaması tamamlandığında, dizi **sütun-öncelikli sıralanmıştır**: sütunları aşağı doğru ve soldan sağa okudukça elemanlar monoton bir şekilde artar.

Sütun sıralaması n in değerinden bağımsız olarak 8 adımda çalışır. Tek sayıda ki adımların hepsi aynıdır: her bir sütunu bireysel olarak sıralarlar. Çift sayıdaki her adım bir sabit permütasyonla (fixed permutation) çalışır. Adımlar şöyledir:

1. Her bir sütunu sırala.
2. Dizinin transpozunu al, ama r satır s sütun olacak şekilde yeniden şekillendir. Bir başka deyişle, en soldaki sütunu sıralı şekilde ilk r/s satırına çevir; bir sonraki sütunu sıralı şekilde bir sonraki r/s satırına çevir; ve böyle devam eder.
3. Her bir sütunu sırala.
4. 2. adımdaki permütasyonun tersini uygula.

10	14	5	4	1	2	4	8	10	1	3	6	1	4	11
8	7	17	8	3	5	12	16	18	2	5	7	3	8	14
12	1	6	10	7	6	1	3	7	4	8	10	6	10	17
16	9	11	12	9	11	9	14	15	9	13	15	2	9	12
4	15	2	16	14	13	2	5	6	11	14	17	5	13	16
18	3	13	18	15	17	11	13	17	12	16	18	7	15	18
(a)			(b)			(c)			(d)			(e)		

1	4	11	5	10	16	4	10	16	1	7	13
2	8	12	6	13	17	5	11	17	2	8	14
3	9	14	7	15	18	6	12	18	3	9	15
5	10	16	1	4	11	1	7	13	4	10	16
6	13	17	2	8	12	2	8	14	5	11	17
7	15	18	3	9	14	3	9	15	6	12	18
(f)			(g)			(h)			(i)		

Şekil 8.5 Sütun sıralamasının adımları. **(a)** 6 satır ve 3 sütundan oluşan girdi dizisi. **(b)** 1. adımda her bir sütunu sıraladıktan sonra. **(c)** 2. adımda transpozunu alıp tekrar şekillendirdikten sonra. **(d)** 3. adımda her bir sütunu sıraladıktan sonra. **(e)** 4. adımda 2. adımdaki permütasyonun tersini uyguladıktan sonra. **(f)** 5. adımda her bir sütunu sıraladıktan sonra. **(g)** 6. adımda her sütunun yarısını yer değiştirdikten sonra. **(h)** 7. adımda her bir sütunu sıraladıktan sonra. **(i)** 6. adımdaki permütasyonu tersine çeviren 8. adımdaki işlemi tamamladıktan sonra. Dizi artık sütun-öncelikli sıradadır.

5. Her bir sütunu sırala.
6. Her bir sütunun üst yarısını alt yarısıyla değiştir, ve her bir sütunun alt yarısını sağındaki sütunun üst yarısıyla değiştir. Sol sütunun üst yarısını boş bırak. En sondaki sütunun alt yarısını en sağdaki yeni sütunun üst yarısıyla değiştir ve bu yeni sütunun alt yarısını boş bırak.
7. Her bir sütunu sırala.
8. 6. adımdaki permütasyonun tersini uygula.

Şekil 8.5 $r = 6$ ve $s = 3$ ile sütun sıralaması adımlarının bir örneğidir (Her ne kadar bu örnek $r \geq 2s^2$ gerekliliğini ihlal etse de, çalışıyor gibi görünüyor).

- c. Tek sayıdaki adımları hangi sıralama yöntemlerini kullandıklarını bilmesek bile sütun sıralama yöntemini bir kayıtsız karşılaştırma-değişimi algoritması olarak kullanabileceğimizi açıklayınız.

Her ne kadar sütun sıralamanın gerçekten sıralama yapabildiğine inanmak güç olsa da, bunu yapabildiğini kanıtlamak için 0-1 sıralama lemmasını kullanacaksınız. 0-1 sıralama lemması burada kullanılabilir çünkü sütun sıralamasını bir kayıtsız karşılaştırma-değişimi algoritması olarak kullanabiliriz. Birkaç tanım 0-1

sıralama lemmasını uygulamanızda yardımcı olacaktır. Bir dizi alanının 0 'lar ya da 1 lerden oluştuğunu biliyorsak ona **temiz** bir dizi alanı deriz. Diğer türlü, dizi alanı karışık 0 lar ve 1 lerden oluşabilir ki buna da **kirli** deriz. Bu noktadan itibaren, girdi dizisinin sadece 0 lar ve 1 lerden oluştuğunu ve onu r satır ve s sütunluk bir dizi olarak kabul edebileceğimizi varsayın.

- d.** 1-3. adım lardan sonra dizinin en üstte 0 lardan oluşan temiz satırlara, en altta 1 lerden oluşan temiz satırlara ve ortada en fazla s sayıda kirli satıra sahip olduğunu kanıtlayın.
- e.** 4. adımdan sonra sütun-öncelikli sıralamada okunduğunda dizinin 0 lardan oluşan temiz bir alanla başladığını, 1 lerden oluşan temiz bir alanla sonlandığını ve ortada en çok s^2 elemana sahip kirli alanın olduğunu kanıtlayın.
- f.** 5 – 8. adımlarının tamamen sıralı (fully sorted) bir 0 – 1 çıktısı ürettiğini kanıtlayın. Sütun sıralamasının rastgele değerler de dahil olmak üzere düzgün sıralama yaptığı çıkarımına ulaşın.
- g.** Şimdi s in r yi sağlamadığını düşünün. 1 – 3. adımlardan sonra dizinin en üstte 0 lardan oluşan temiz satırları olduğunu, en altta 1 lerden oluşan temiz satırları olduğunu ve aralarında en fazla $2s - 1$ tane kirli satır olduğunu ispatlayın. s r yi tam bölmediğinde sütun sıralamasının düzgün çalışabilmesi için s e kıyasla r nin ne kadar büyük olması gerekir?
- h.** s in r yi bölmediği durumda bile 1. adımda yapılacak basit bir değişiklik sunarak $r \geq 2s^2$ yi sağlayın ve bu yaptığınız değişiklikle, sütun sıralamasının düzgün çalıştığını kanıtlayın.

Bölüm notları

Karşılaştırma sıralamalarını çalışmada karar ağacı modeli Ford ve Johnson [110] tarafından ortaya konulmuştur. Knuth'un sıralama konusundaki kapsamlı çalışmaları [211] burada sıralamanın karmaşıklığı üzerine verilen bilgi-kuramsal alt sınır da dahil olmak üzere sıralama probleminin bir çok varyasyonunu kapsamaktadır. Ben-Or [39] karar ağacı modelinin genellemelerini kullanarak sıralama için alt sınırları çalışmıştır.

Knuth 1954 te sayma sıralamasını bulduğu ve taban sıralamasını sayma sıralamasıyla birleştirme fikrini ortaya koyduğu için H. H. Stewart'a saygılarını sunmaktadır. En değersiz basamaktan başlayan sayı tabanı sıralaması mekanik kart sıralama makinesi operatörleri tarafından yaygın olarak kullanılan basit bir algoritma gibi görünmektedir. Knuth'a göre bu yönteme değinen yayınlanmış ilk referans L. J. Comrie'nin delikli kart ekipmanını (punch card equipment) anlattığı 1929 tarihli bir dökümandır. Kova sıralaması 1956 da E.J. Isaac ve R. C. Singleton [188] bu basit fikri ortaya koyduklarından beri kullanılmaktadır.

Munro ve Raman [263] $0 < \epsilon \leq 1$ in herhangi bir belirlenmiş sabiti iken, en kötü durumda $O(n^{1+\epsilon})$ karşılaştırmasını yapan kararlı bir sıralama algoritması

sundular. Her ne kadar $O(n \lg n)$ -zaman algoritmalarının herhangi biri daha az karşılaştırma yapıyor olsa da, Munro ve Raman'ın sunduğu algoritma veriyi sadece $O(n)$ kere taşır ve daha etkili çalışır.

n sayıda b -bit tam sayıyı $O(n \lg n)$ zamanda sıralamak bir çok araştırmacı tarafından düşünülmüştür. Her biri ölçüm modeliyle ilgili küçük farklarla ayrılan varsayımlara dayalı ve algoritmalara getirilmiş sınırlamalarla yapılarak bir çok pozitif sonuç elde edilmiştir. Bütün sonuçlar bilgisayar hafızasının adreslenebilir b -bit kelimelere ayrılmış olduğunu varsayar. Fredman ve Willard [115] füzyon ağacı (fusion tree) veri yapısını öne sürmüşler ve n sayıdaki tam sayıyı $O(n \lg n / \lg \lg n)$ zamanda sıralamak için kullanmışlardır. Bu sınır daha sonra Andersson [16] tarafından $O(n \sqrt{\lg n})$ zamana geliştirilmiştir. Bu algoritmalar çarpma işleminin ve bir çok önceden hesaplanmış sabitlerin kullanılmasını gerektirir. Andersson, Hagerup, Nilsson ve Raman [17] n sayıdaki tam sayıyı $O(n \lg \lg n)$ zamanda çarpma işlemini kullanmadan nasıl sıralanabileceğini göstermişlerdir ancak onların yöntemi de da n açısından sınırsız olabilecek bir bellek gerektirmektedir. Hesaba dayalı çarpımsal çırpılamayı (multiplicative hashing) kullanarak, $O(n)$ için gereken belleği azaltabiliriz, fakat o durumda da $O(n \lg \lg n)$ in işlem sürecindeki en kötü durum sınırı beklenen bir zaman sınırına dönüşür. Andersson'un [16] üssel arama ağaçlarını genelleyerek, Thorup [335] çarpma ya da randomizasyon kullanmayan ve doğrusal uzay kullanan bir $O(n(\lg \lg n)^2)$ -zaman sıralama algoritması vermiştir. Bu teknikleri bazı yeni fikirlerle birleştirerek, Han [158] sıralama sınırını $O(n \lg \lg n \lg \lg \lg n)$ zaman noktasına iyileştirmiştir. Her ne kadar bu algoritmalar önemli teorik buluşlar olsalar da, her biri son derece komplikedir ve günümüzde kullanılan sıralama algoritmalarıyla yarışmaları pek de olası değildir.

Problem 8-7 de Leighton'ın [227] sütun sıralama algoritması vardır.

n elemanlı bir kümenin i inci **sıra istatistiği** i inci en küçük elemanıdır. Örneğin, bir elemanlar kümesinin **en küçükü** ilk sıra istatistiğidir ($i = 1$) ve **en büyüğü** n inci sıra istatistiğidir ($i = n$). Bir **medyan**, ya da bir ortanca (median), isimlendirecek olursak, bir kümenin “orta noktası”dır. n tek sayı ise, medyan tek-tir ve $i = (n + 1)/2$ noktasında bulunur. n çift sayı ise, iki medyan vardır ve $i = n/2$ ve $i = n/2 + 1$ noktalarında bulunur. Dolayısıyla, n nin eşliğinden bağımsız olmak üzere, medyanlar $i = \lfloor (n + 1)/2 \rfloor$ (**alt medyan** (lower median)) ve $i = \lceil (n + 1)/2 \rceil$ (**üst medyan** (upper median)) noktalarında bulunurlar. Bu yazının basit olması açısından, tüm anlatım boyunca alt medyan kavramı için medyan terimini kullanacağız.

Bu bölüm n adet farklı sayıdan oluşan bir kümenin i inci sıra istatistiğini bulma problemiyle ilgilidir. Her ne kadar yaptığımız her şey kümenin tekrar eden değerleri içerdiği durumu kapsasa da, kolay olması açısından kümenin farklı sayılardan oluştuğunu varsayıyoruz. **Seçme problemi** (selection problem) formal olarak şu şekilde tanımlıyoruz:

Girdi: n farklı sayıdan oluşan ve i tam sayısını içeren ve $1 \leq i \leq n$ i sağlayan bir A kümesi

Çıktı: A kümesinin tam olarak $i - 1$ diğer elemanından daha büyük bir $x \in A$ elemanı

Bellek yığını sıralama ya da birleştirerek sıralama yöntemlerini kullanarak sayıları sıralayabileceğimiz ve sonra çıktı dizisinde i inci elemanı basit bir şekilde indisleyebileceğimiz için seçme problemini $O(n \lg n)$ zamanda çözebiliriz. Bu bölüm daha hızlı algoritmalar sunmaktadır.

Kısım 9.1 de bir elemanlar kümesinin minimum ve maksimumunu seçme problemini inceliyoruz. Devam eden iki kesimde incelediğimiz genel seçme problemi ise daha da ilginçtir. Kısım 9.2 farklı elemanlar varsayarak $O(n)$ beklenen çalışma zamanı ortaya koyan pratik bir rassal algoritmayı incelemektedir. Kısım 9.3 ise en kötü durumdaki $O(n)$ çalışma zamanını ortaya koyan daha teorik içerikli bir algoritmayı içermektedir.

9.1 Minimum ve maksimum

n elemanlı bir kümenin minimumunu belirlemek için kaç karşılaştırma yapmak gerekir? Kolay bir şekilde $n - 1$ üst sınırını elde edebiliriz: kümedeki her elemanı sırayla inceleyin ve o ana kadar gördüğünüz en küçük elemanı not edin. Aşağıdaki uygulamada kümenin $A.length = n$ olduğu A dizisinde bulunduğunu varsayıyoruz.

```

MINIMUM ( $A$ )
1   $min = A[1]$ 
2  for  $i = 2$  to  $A.length$ 
3      if  $min > A[i]$ 
4           $min = A[i]$ 
5  return  $min$ 

```

Aynı zamanda elbette maksimumu da $n - 1$ karşılaştırma ile bulabiliriz.

Peki yapabileceğimizin en iyisi bu mu? Minimum bulma problemi için $n - 1$ alt karşılaştırmalarının alt sınırını bulabileceğimizden, evet yapabileceğimizin en iyisi budur. Minimum bulan bütün algoritmaları elemanlar arasında bir turnuva gibi düşünün. Her karşılaştırma ikiliden küçük olanın kazandığı bir maçıdır. Kazanan haricinde her bir elemanın en az bir maç kaybetmesi gerektiğine göre, minimumu belirlemek için $n - 1$ karşılaştırma yapmanın şart olduğu sonucuna varabiliriz. Dolayısıyla, MINIMUM algoritması yapılan karşılaştırmaların sayısı bakımından optimaldir.

Eşzamanlı minimum ve maksimum

Bazı uygulamalarda n elemanlı bir kümenin hem minimum hem de maksimumunu bulmamız gerekir. Örneğin, bir grafik programı, dikdörtgen bir ekrana ya da grafik bir çıktı aracına yerleştirmek için bir (x, y) veri kümesini ölçeklendirme ihtiyacı duyabilir. Bunu yapmak için, program öncelikle her koordinatın maksimum ve minimum değerlerini bulmak zorundadır.

Bu noktada, n elemanın minimum ve maksimumunu $\Theta(n)$ karşılaştırmasıyla belirlemek açık olmalıdır ki bu karşılaştırmalar asimptotik olarak optimaldir: basitçe, minimum ve maksimumu birbirinden bağımsız olarak her biri için $n - 1$ karşılaştırma, yani toplamda $2n - 2$ karşılaştırma ile bulur.

Aslında, hem maksimum hem de minimumu en fazla $3 \lfloor n/2 \rfloor$ karşılaştırmayla bulabiliriz. Bu şekilde her bir girdi elemanını var olan minimum ve maksimum ile, iki katı kıyaslama yaparak karşılaştırmak yerine, çiftler halinde karşılaştırıyoruz. Girdideki eleman çiftlerini öncelikle birbirleriyle karşılaştırıyoruz, sonra,

toplamda her 2 eleman için 3 karşılaştırma yapmış olacak şekilde, büyük olanı maksimumla, küçük olanı minimumla karşılaştırıyoruz.

Mevcut minimum ve maksimumun başlangıç değerlerini nasıl belirlediğimiz n nin tek mi çift mi olduğuna göre değişiyor. Eğer n tek ise, hem maksimum hem minimumu ilk elemanın değerine eşliyoruz ve kalan çiftleri onlara göre karşılaştırıyoruz. Eğer n çift ise, ilk iki eleman arasında bir karşılaştırma yaparak minimum ve maksimumun başlangıç değerlerini belirliyoruz, sonra kalan çiftlerin karşılaştırmalarını n nin tek olduğu durumdaki gibi yaparak devam ediyoruz.

Şimdi toplam karşılaştırma sayılarını inceleyelim. n tek ise, $3 \lfloor n/2 \rfloor$ sayıda karşılaştırma yapıyoruz. n çift ise, önce bir başlangıç karşılaştırması yapıyor, sonra bunu $3(n-2)/2$ sayıda karşılaştırmayla, toplamda $3n/2 - 2$ olacak şekilde devam ettiriyoruz. Dolayısıyla, her iki durumda da toplam karşılaştırma sayımız en fazla $3 \lfloor n/2 \rfloor$ olmaktadır.

Alıştırmalar

9.1-1

n elemanlı bir kümenin ikinci en küçük elemanının en kötü durumda $n + \lceil \lg n \rceil - 2$ karşılaştırma ile bulunabileceğini gösterin (*İpucu*: en küçük elemanı da bulun).

9.1-2 ★

n adet sayının minimum ve maksimumunu belirlemek için en kötü durumdaki $\lceil 3n/2 \rceil - 2$ nin alt sınırını ispatlayın (*İpucu*: kaç tane sayının potansiyel olarak aynı zamanda maksimum ve minimum olabileceğini düşünüp bir karşılaştırmının bu sayımları nasıl etkilediğini araştırın).

9.2 Beklenen doğrusal zamanda seçim

Genel seçme problemi basit bir minimumu bulma probleminden daha zor gibi görünmektedir. Ancak, ilginç bir şekilde, her iki problem için de asimptotik çalışma zamanı aynıdır: $\Theta(n)$. Bu kısımda seçme problemi için bir böl-ve-yönet algoritması sunuyoruz. RANDOMIZED-SELECT algoritması Bölüm 7 deki hızlı sıralama algoritmasından sonra modellenmiştir. Hızlı sıralamadaki gibi, girdi dizisini tekrar tekrar parçalıyoruz. Ancak parçalamanın her iki tarafını da tekrar tekrar işleyen hızlı ayıklamanın tersine, RANDOMIZED-SELECT parçalamanın tek tarafında çalışır. Bu fark analizde ortaya çıkar: hızlı sıralamanın beklenen bir $\Theta(n \lg n)$ çalışma zamanı varken, RANDOMIZED-SELECT in beklenen çalışma zamanı, elemanları farklı olduğu varsayıldığında, $\Theta(n)$ dır.

RANDOMIZED-SELECT Kısım 7.3 te anlatılan RANDOMIZED-PARTITION yöntemini kullanır. Dolayısıyla, RANDOMIZED-QUICKSORT gibi, davranışı kısmen rassal sayı üreticinin çıktılarına dayandığı için rassal bir algoritmadır. RANDOMIZED-SELECT için aşağıda gösterilen kod $A[p \dots r]$ dizisinin i inci en küçük elemanını bulur.

```

RANDOMIZED-SELECT( $A, p, r, i$ )
1  if  $p == r$ 
2      return  $A[p]$ 
3   $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
4   $k = q - p + 1$ 
5  if  $i == k$           // cevap, pivot değeridir
6      return  $A[q]$ 
7  elseif  $i < k$ 
8      return RANDOMIZED-SELECT( $A, p, q - 1, i$ )
9  else return RANDOMIZED-SELECT( $A, q + 1, r, i - k$ )

```

RANDOMIZED-SELECT prosedürü aşağıdaki gibi çalışır: 1.satır alt dizisi $A[p \dots r]$ sadece bir elemandan oluştuğunda özyinelemenin temelini kontrol eder. Bu durumda, i 1 e eşit olmalıdır ve basitçe $A[p]$ ı 2.satıra i nin en küçük elemanı olarak döndürürüz. Aksi halde, RANDOMIZED-PARTITION u işleme sokmak için 3. satır $A[p \dots r]$ dizisini $A[p \dots q - 1]$ ve $A[q + 1 \dots r]$ şeklinde iki alt diziyeye, $A[p \dots q - 1]$ in her bir elemanının $A[q]$ ya eşit ya da daha küçük olacak şekilde böler, ki bu da sırasıyla $A[q + 1 \dots r]$ nin her bir elemanından küçüktür. Hızlı sıralamadaki gibi, $A[q]$ a pivot eleman diyeceğiz. 4.satır, $A[p \dots q]$ alt dizisinin k eleman sayısını ölçer ki bu da parçalamanın alt tarafındaki eleman sayısı artı pivot için bir eleman demektir. 5.satır $A[q]$ nin i inci en küçük eleman olup olmadığını kontrol eder. Eğer öyleyse, 6.satır $A[q]$ sonucunu verir. Değilse, algoritme i inci en küçük elemanın $A[p \dots q - 1]$ ve $A[q + 1 \dots r]$ alt dizilerinden hangisinde olduğuna karar verir. Eğer $i < k$ ise, aranan eleman parçalamanın alt tarafındadır ve 8.satır özyinelemeli bir şekilde onu alt diziden seçer. Diğer taraftan, eğer $i > k$ ise, o zaman aranan eleman parçalamanın üst tarafındadır. K nin $A[p \dots r]$ dizisinin i inci en küçük elemanının — diğer bir deyişle $A[p \dots q]$ elemanının — değeri olduğunu bildiğimiz için, aranan eleman, 9.satırın tekrarlayarak seçtiği, $A[q + 1 \dots r]$ dizisinin $(i - k)$ ncı en küçük elemanıdır. Kod o elemanlı alt dizilere özyinelemeli çağrılar (recursive calls) yapıyor gibi görünmektedir ancak Alıştırma 9.2-1 sizden bu durumun olamayacağını göstermenizi istemektedir.

RANDOMIZED-SELECT in en kötü durumdaki çalışma zamanı, minimumu bulmak için bile, $\Theta(n^2)$ zamandır çünkü son derece şanssız olabilir ve sürekli olarak geriye kalan en büyük elemanın parçalamasını yapıyor olabiliriz ve bu parçalama $\Theta(n)$ zaman sürmektedir. Yine de, algoritmanın doğrusal bir beklenen çalışma

zamanına sahip olduğunu göreceğiz çünkü rassaldır. Hiç bir belli girdi en kötü durum davranışını ortaya koymaz.

RANDOMIZED-SELECT in beklenen çalışma zamanını analiz etmek için n elemanlı bir $A[p..r]$ girdi dizisinin çalışma zamanının $T(n)$ ile göstereceğimiz rassal bir değişken olmasına izin veririz ve $E[T(n)]$ üzerinde aşağıdaki gibi bir üst sınır elde ederiz. RANDOMIZED-PARTITION prosedürü de herhangi bir elemanı pivot seçmeye eşit derecede yatkındır. Dolayısıyla, $1 \leq k \leq n$ deki gibi her k için, $A[p..q]$ altdizisinin $1/n$ olasılıklı (hepsi pivota eşit ya da daha küçük) k elemanı vardır. $k = 1, 2, \dots, n$ için, $X_k =$ durumunda, rassal gösterge değişkenlerini X_k olarak belirleriz. Eğer $A[p..q]$ altdizisinin tam olarak k elemanı var ise, bu elemanların farklı olduğunu varsayarak,

$$E[X_k] = 1/n$$

dur.

(9.1)

RANDOMIZED-SELECT yapacağımız zaman ve $A[q]$ yu pivot seçtiysek, —doğru cevaba hemen ulaşacaksak— altdizi $A[p..q-1]$ ya da altdizi $A[q+1..r]$ üzerinde bir öncül bir özyineleme belirleyemeyiz. Bu karar i inci en küçük elemanın $A[q]$ e göre nerede olduğuna göre değişir. $T(n)$ nin monoton olarak arttığını varsayarsak, özyinelemeli çağrısı için gereken üst sınır zamanını mümkün olan en büyük girdinin özyinelemeli çağrısı üzerinden belirleyebiliriz. Diğer bir deyişle, bir üst sınır elde etmek için, i inci elemanın her zaman parçalamanın en fazla elemana sahip tarafında olduğunu varsayabiliriz. Belirli bir RANDOMIZED-SELECT çağrısı için, rassal gösterge değişkeni X_k k nın kesin olarak bir değeri için 1 dir ve diğer bütün k lar için 0 dir. $X_k = 1$ olduğunda, üzerinde özyineleme yapabileceğimiz iki altdizinin boyutları $k-1$ ve $n-k$ dir. Dolayısıyla şu özyinelemeler olacaktır:

$$\begin{aligned} T(n) &\leq \sum_{k=1}^n X_k \cdot (T(\max(k-1, n-k)) + O(n)) \\ &= \sum_{k=1}^n X_k \cdot T(\max(k-1, n-k)) + O(n) . \end{aligned}$$

Beklenen değerleri alırsak, elimizde

$$\begin{aligned}
& E[T(n)] \\
& \leq E \left[\sum_{k=1}^n X_k \cdot T(\max(k-1, n-k)) + O(n) \right] \\
& = \sum_{k=1}^n E[X_k \cdot T(\max(k-1, n-k))] + O(n) \quad (\text{beklentinin doğrusallığından}) \\
& = \sum_{k=1}^n E[X_k] \cdot E[T(\max(k-1, n-k))] + O(n) \quad (\text{C.24) denkleminde}) \\
& = \sum_{k=1}^n \frac{1}{n} \cdot E[T(\max(k-1, n-k))] + O(n) \quad ((9.1) denkleminde)
\end{aligned}$$

olur.

Denklemleri uygulamak için (C.24), X_k ve $T(\max(k-1, n-k))$ ın bağımsız rassal değişkenler olması gerekir. Alıştırma 9.2-2 sizden bu savı doğrulamanızı istemektedir.

$\max(k-1, n-k)$ ifadesini düşünelim. Elimizde aşağıdakiler vardır.

$$\max(k-1, n-k) = \begin{cases} k-1 & k > \lceil n/2 \rceil \text{ ise,} \\ n-k & k \leq \lceil n/2 \rceil \text{ ise.} \end{cases}$$

Eğer n çift sayı ise, $T(\lceil n/2 \rceil)$ den $T(n-1)$ e kadar bütün değerler toplamda tam olarak ikişer kez görünür ve eğer n tek sayı ise, bütün bu terimler ikişer kez görünür ve $T(\lfloor n/2 \rfloor)$ bir kez görünür. Dolayısıyla elimizde olan şudur:

$$E[T(n)] \leq \frac{2}{n} \sum_{k=\lceil n/2 \rceil}^{n-1} E[T(k)] + O(n).$$

$E[T(n)] = O(n)$ yi yerine koyma yöntemiyle gösteririz. Özyinelemenin başlangıç şartlarını karşılayan sabit bir c yi içeren $E[T(n)] \leq cn$ olduğunu varsayalım. n in bazı sabitlerden küçük olduğu bir $T(n) = O(1)$ varsayıyoruz; bu sabiti daha sonra belirleyeceğiz. Ayrıca, üstteki $O(n)$ terimiyle belirlenmiş (ki bu algoritmanın çalışma zamanının özyinelemeli olmayan bileşenini göstermektedir) fonksiyonu ‘tüm $n > 0$ için’ sınırlamasıyla n ile üstten ayrılan bir sabit seçiyoruz. Bu tümevarım hipotezi ile elimizde olan şudur:

$$\begin{aligned}
E[T(n)] & \leq \frac{2}{n} \sum_{k=\lceil n/2 \rceil}^{n-1} ck + an \\
& = \frac{2c}{n} \left(\sum_{k=1}^{n-1} k - \sum_{k=1}^{\lceil n/2 \rceil - 1} k \right) + an
\end{aligned}$$

$$\begin{aligned}
&= \frac{2c}{n} \left(\frac{(n-1)n}{2} - \frac{(\lfloor n/2 \rfloor - 1) \lfloor n/2 \rfloor}{2} \right) + an \\
&\leq \frac{2c}{n} \left(\frac{(n-1)n}{2} - \frac{(n/2 - 2)(n/2 - 1)}{2} \right) + an \\
&= \frac{2c}{n} \left(\frac{n^2 - n}{2} - \frac{n^2/4 - 3n/2 + 2}{2} \right) + an \\
&= \frac{c}{n} \left(\frac{3n^2}{4} + \frac{n}{2} - 2 \right) + an \\
&= c \left(\frac{3n}{4} + \frac{1}{2} - \frac{2}{n} \right) + an \\
&\leq \frac{3cn}{4} + \frac{c}{2} + an \\
&= cn - \left(\frac{cn}{4} - \frac{c}{2} - an \right) .
\end{aligned}$$

Kanıtı tamamlamamız için, yeterince büyük bir n için bu son ifadenin en fazla cn olduğunu ya da eşit şekilde, $cn/4 - c/2 - an \geq 0$ olduğunu göstermemiz gerekir. Eğer her iki tarafa da $c/2$ ekler ve n leri sadeleştirirsek, sonuç $n(c/4 - a) \geq c/2$ olacaktır. Sabitimiz olan c yi $c/4 - a > 0$, yani, $c > 4a$ şeklinde seçersek her iki tarafı da $c/4 - a$ ile bölebiliriz ki bu da bize şu sonucu verecektir:

$$n \geq \frac{c/2}{c/4 - a} = \frac{2c}{c - 4a} .$$

Dolayısıyla, $n < 2c/(c - 4a)$ için $T(n) = O(1)$ ı düşünürsek, o zaman $E[T(n)] = O(n)$ dir. Buradan elemanların farklı olduğunu varsaydığımızda herhangi bir sıra istatistiği ve özellikle medyanı, beklenen doğrusal zamanda bulabileceğimiz sonucuna varabiliriz.

Alıştırmalar

9.2-1

RANDOMIZED-SELECT in 0 uzunluklu bir dizide asla özyinelemeli çağrı yapmadığını gösteriniz.

9.2-2

Rassal gösterge değişkeni X_k ve $T(\max(k-1, n-k))$ değerlerinin bağımsız olduğunu açıklayınız.

9.2-3

RANDOMIZED-SELECT in özyinelemeli bir versiyonunu yazınız.

9.2-4

RANDOMIZED-SELECT yönetimini $A = \langle 3, 2, 9, 0, 7, 5, 4, 8, 6, 1 \rangle$ i dizisinin en küçük elemanını bulmak için kullandığınızı varsayın. RANDOMIZED-SELECT in en kötü durum performansının sonuçlarını gösteren bir parçalama sırası tarif ediniz.

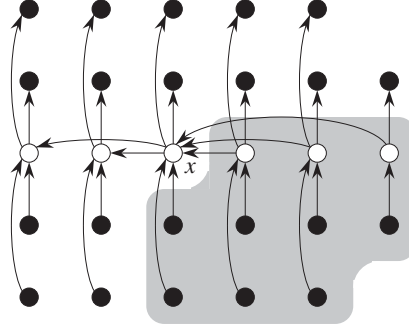
9.3 En kötü-durumlu doğrusal zamanda seçim

Şimdi en kötü durumdaki çalışma zamanı $O(n)$ olan bir seçme algoritmasını inceleyeceğiz. RANDOMIZED-SELECT gibi, SELECT algoritması da girdi dizisinde özyinelemeli parçalamalar yaparak aranan elemanı bulmaktadır. Ancak burada, diziyi parçalamada iyi bir bölmeyi *garanti ediyoruz*. SELECT hızlı sıralamadan (bkz: Kısım 7.1) kararlı parçalama algoritması PARTITION u kullanmaktadır ancak elemanı parçalamaya bir girdi parametresi olarak sokmaya ayarlanmıştır.

SELECT algoritması, $n > 1$ sayıdaki farklı elemandan oluşan bir girdi dizisinin i inci en küçük elemanını şu adımları izleyerek belirler. (Eğer $n = 1$ ise, SELECT tek girdisini i inci en küçük eleman olarak geri döndürür.)

1. Girdi dizisinin n elemanını, her birinde 5 eleman bulunan ve en fazla 1 gruba $n \bmod 5$ eleman içerecek şekilde $\lfloor n/5 \rfloor$ sayıda gruba böl.
2. Her gruba (ki her birinde en fazla 5 eleman vardır) önce araya sokma sıralaması uygulayıp sonra grup elemanlarının sıralanmış listesinden medyanları seçerek her $\lfloor n/5 \rfloor$ grubunun medyanını bul.
3. 2. Adımda bulunan $\lfloor n/5 \rfloor$ medyanlarının x medyanını bulmak için SELECT i özyineli kullan.
4. PARTITION un düzenlenmiş versiyonunu kullanarak girdi dizisini medyanların medyanı x in etrafında parçala. x k inci en küçük eleman ve parçalamanın üst kısmında $n - k$ eleman olacak şekilde, k parçalamanın alt kısmındaki eleman sayısından bir fazla olsun.
5. Eğer $i = k$ ise, x i döndür. Değilse, $i < k$ için alt kısımdaki i inci en küçük elemanı, $i > k$ için üst kısımdaki $(i - k)$ inci elemanı bulmak için SELECT i özyinelemeli kullan.

SELECT in çalışma zamanını analiz etmek için, parçalama elemanı x ten daha büyük olan eleman sayısı üzerinde bir alt sınır belirliyoruz. Şekil 9.1 bunu göstermektedir. 2. adımda bulunan medyanların en az yarısı medyanların medyanı x e



Şekil 9.1 SELECT algoritmasının analizi. n sayıdaki elemanlar küçük halkalarla belirtilmişlerdir ve 5'er elemandan oluşan her grup birer sütunda bulunmaktadır. Grupların medyanları beyazla gösterilmektedir ve medyanların medyanı x olarak belirtilmiştir (çift sayıda elemanın medyanını bulurken alt medyanı kullanıyoruz). Oklar büyük elemandan küçük elemana doğrudur; bundan x 'in sağındaki 5 elemanlı her grubun 3 elemanının x 'ten büyük olduğunu ve solundaki 5 elemanlı her grubun 3 elemanının x 'ten küçük olduğunu görebiliriz. x 'ten büyük olduğu bilinen elemanlar gölgeli kısımdadır.

eşit ya da daha büyüktür.¹ Dolayısıyla, 5'in n 'yi tam bölmediği durumlarda ortaya çıkan ve 5'ten az elemanı olan tek grup ve x 'in olduğu tek grup haricindeki bütün $\lceil n/5 \rceil$ gruplarının en az yarısı x 'ten büyük en az 3 eleman katmaktadır. Bu iki grubu düşüğümüzde ortaya x 'ten büyük eleman sayısının en az şu kadar olduğu çıkar:

$$3 \left(\left\lceil \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rceil - 2 \right) \geq \frac{3n}{10} - 6.$$

Benzer şekilde, en az $3n/10 - 6$ sayıda eleman x 'ten küçüktür. Dolayısıyla, en kötü durumda, 5. adım SELECT'i en fazla $7n/10 + 6$ elemanda özyinelemeli çağırır.

Şimdi SELECT algoritmasının en kötü çalışma zamanı $T(n)$ de bir özyineleme geliştirebiliriz. 1, 2 ve 4. adımlar $O(n)$ zaman alır. (2. adım $O(1)$ boyuttaki kümelerle $O(n)$ araya sokma sıralaması çağrılarından oluşur) 3. adım $T(\lceil n/5 \rceil)$ zaman alır ve 5. adım, T 'nin monoton olarak arttığını varsayarak, en fazla $T(7n/10 + 6)$ zaman alır. Başta gerekçesiz görünen, 140'tan az elemanlı bütün girdilerin $O(1)$ zaman alacağı varsayımını yaparız; mucizevi sabit 140'ın kaynağını birazdan açıklayacağız. Dolayısıyla şu özyinelemeyi elde ederiz.

¹Sayıların farklı olduğu varsayımımız yüzünden, x dışındaki bütün medyanlar x 'ten büyük ya da küçüktür.

$$T(n) \leq \begin{cases} O(1) & n < 140 \text{ ise,} \\ T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n) & n \geq 140 \text{ ise.} \end{cases}$$

Çalışma zamanını yerine koyma yöntemi ile doğrusal olduğunu gösteriyoruz. Daha belirgin olarak, uygun büyüklükteki bir c sabiti ve bütün $n > 0$ durumları için $T(n) \leq cn$ i göstereceğiz. Yeterli büyüklükte bir c sabiti ve bütün $n < 140$ durumları için $T(n) \leq cn$ olduğunu varsayarak başlıyoruz: bu varsayım c yeterince büyükse geçerlidir. Ayrıca, üstteki $O(n)$ terimiyle belirlenmiş (ki bu bir algoritmanın işlem süresinin tekrarlanmayan içeriğini göstermektedir) fonksiyonu ‘tüm $n > 0$ durumları için’ sınırlamasıyla üstten ayrılan bir sabit seçiyoruz. Bu tümevarımsal hipotezi özyinelemenin sağ tarafıyla değiştirmek bize şunu verir:

$$\begin{aligned} T(n) &\leq c \lceil n/5 \rceil + c(7n/10 + 6) + an \\ &\leq cn/5 + c + 7cn/10 + 6c + an \\ &= 9cn/10 + 7c + an \\ &= cn + (-cn/10 + 7c + an), \end{aligned}$$

ki bu da en fazla cn dir eğer

$$-cn/10 + 7c + an \leq 0. \quad (9.2)$$

Eşitsizlik (9.2), $n > 70$ durumunda $c \geq 10a(n/(n-70))$ eşitsizliğine denktir. $n + 140$ ı varsaydığımız için, $n/(n-70) \leq 2$ ye sahibiz ve böylece $c \geq 20a$ yı seçmek eşitsizlik (9.2) yi sağlayacaktır. (140 sabitiyle alakalı özel bir durum olmadığına dikkat edin; 70 ten kesin büyük olan herhangi bir tam sayı seçip c yi ona uygun seçebilirdik.) SELECT in en kötü durum işlem süreci bu durumda doğrusaldır.

Karşılaştırma sıralamasında olduğu gibi (bkz: Kısım 8.1), SELECT ve RANDOMIZED-SELECT elemanların bağıl sıralarıyla alakalı bilgiyi sadece elemanları karşılaştırarak belirler. Bölüm 8 den sıralamanın karşılaştırma modelinde, ortalamada bile (bkz: Kısım 8.1) $\Omega(n \lg n)$ zaman gerektirdiğini hatırlayın. Bölüm 8 deki doğrusal zamanlı sıralama algoritmaları girdi ile ilgili varsayımlar yapar. Tam tersine, bu bölümdeki doğrusal zamanlı seçme algoritmalarının girdiyle alakalı varsayımlara ihtiyacı yoktur. $\Omega(n \lg n)$ alt sınırına bağlı değildir çünkü seçme problemini sıralama kullanmadan çözerler. Dolayısıyla, seçme problemini sıralama ve indisleme ile çözmek, bölümün girişinde de bahsedildiği gibi, asimpotik olarak yetersizdir.

Alıştırmalar

9.3-1

SELECT algoritmasında girdi elemanları 5'erli gruplara ayrılmışlardır. 7'şerli gruplara ayrılırsalar algoritma yinede doğrusal zamanda çalışır mıydı? Eğer 3'erli gruplara ayrılırsalar SELECT in doğrusal zamanda çalışmayacağını gösterin.

9.3-2

SELECT i $n \geq 140$ durumunda en az $\lceil n/4 \rceil$ elemanın medyanların-medyanı x ten büyük olacağını ve en az $\lceil n/4 \rceil$ elemanın x ten küçük olacağını gösterecek şekilde analiz ediniz.

9.3-3

Bütün elemanların farklı olduğunu varsayarak, hızlı sıralamanın en kötü durumda $O(n \lg n)$ zamanda nasıl çalışacağını gösteriniz.

9.3-4 ★

Bir algoritmanın n elemanlı bir kümede i inci en küçük elemanı bulmak için sadece karşılaştırmaları kullandığını farzedin. Ek karşılaştırmalar kullanmadan algoritmanın aynı zamanda $i - 1$ daha küçük elemanları ve $n - i$ daha büyük elemanları da bulabileceğini gösterin.

9.3-5

Elinizde bir en kötü durum doğrusal zaman medyan altprogramını “kara kutu” olduğunu farzedin. Gelişi güzel bir sıra istatistiği için seçme problemini çözecek basit bir doğrusal zaman algoritması yazın.

9.3-6

n elemanlı bir kümenin k inci *yüzdelik dilimleri* sıralı bir kümeyi k eşit-boyutta kümelere (1 de dahil olmak üzere) bölen $k - 1$ sıralı istatistikleridir. Bir kümenin k inci yüzdelik dilimlerini listeyecek $O(n \lg k)$ zamanlı bir algoritma yazın.

9.3-7

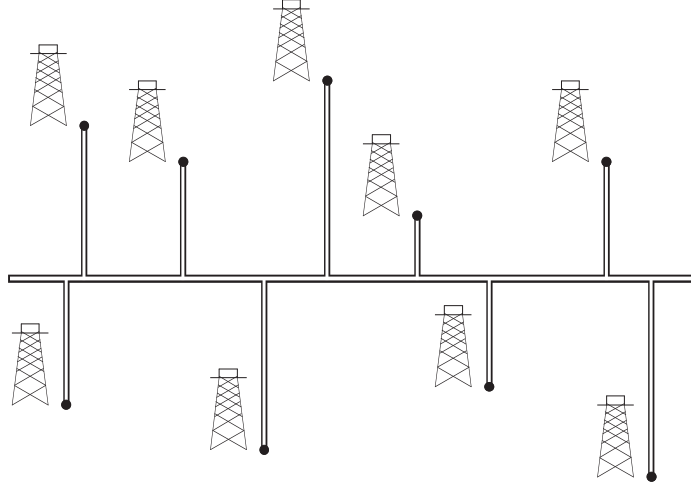
n farklı sayıdan ve bir $k \leq n$ pozitif tam sayısından oluşan bir kümede, S in medyanını en yakın olan S teki k sayılarını belirleyen bir $O(n)$ zamanlı algoritma tanımlayın.

9.3-8

$X[1..n]$ ve $Y[1..n]$, her biri hali hazırda sıralı n sayı içeren birer dizi olsunlar. X ve Y dizilerindeki tüm $2n$ elemanın medyanlarını bulacak $O(\lg n)$ zamanlı bir algoritma yazın.

9.3-9

Profesör Olay, n sayıda kuyu içeren bir petrol alanı boyunca doğudan batıya uzanan büyük bir boru hattı inşa etmek isteyen bir petrol şirketi için danışmanlık



Şekil 9.2 Profesör Olay, kuzey-güney yönlerindeki ara hatların toplam uzunluğunu minimize edecek doğu-batı yönündeki ana boru hattının yerini belirlemek zorunda.

yapmaktadır. Şirket Şekil 9.2 de gösterildiği gibi, her kuyudan doğrudan ve en kısa yoldan (güney ya da kuzey) ana hatta bağlanan birer ara hat bağlamak istemektedir. Kuyuların x ve y koordinatları verildiğine göre, profesör ara hatların toplam uzunluğunu minimize edecek optimal ana hat konumunu nasıl belirleyebilir? Optimal konumu doğrusal zamanda nasıl belirleyeceğinizi gösteriniz.

Problemler

9-1 Sıralı düzende en büyük i sayıları

Elimizdeki n elemanlı bir kümede karşılaştırma-tabanlı bir algoritma kullanarak sıralı düzende en büyük i yi bulmak istiyoruz. Aşağıdaki en iyi asimptotik en kötü durum çalışma zamanı içeren yöntemleri uygulayan algoritmayı bulunuz ve algoritmaların çalışma zamanlarını n ve i cinsinden analiz ediniz.

- Sayıları sıralayın ve i en büyüğü listeleyin.
- Sayılardan bir maksimum öncelik sırası yapın ve i kere EXTRACT-MAX işlemini uygulayın.
- i inci en büyük sayıyı bulmak için bir sıra-istatistiği algoritması kullanın, bu sayının etrafında parçalama yapın ve i en büyük sayıyı sıralayın.

9-2 Ağırlıklı medyan

$\sum_{i=1}^n w_i = 1$, oluşturacak şekilde $w_1, w_2, \dots, w_n$ pozitif ağırlıklara sahip $x_1, x_2, \dots, x_n$ şeklindeki n farklı eleman için **ağırlıklı (alt) medyan**

$$\sum_{x_i < x_k} w_i < \frac{1}{2}$$

ve

$$\sum_{x_i > x_k} w_i \leq \frac{1}{2}$$

sağlayan x_k elemanıdır.

Örneğin, eğer elemanlar 0.1, 0.35, 0.05, 0.1, 0.15, 0.5, 0.2 ve her eleman kendi ağırlığına eşitse (bu ağırlıklar $i = 1, 2, \dots, 7$) için $w_i = x_i$ dir, o zaman medyan 0.1 dir ancak ağırlıklı medyan 0.2 dir.

- $x_1, x_2, \dots, x_n$ serisinin medyanının $i = 1, 2, \dots, n$ için $w_i = 1/n$ ile x_i nin ağırlıklı medyanı olduğunu açıklayınız.
- Sıralamayı kullanarak n elemanlı bir kümenin ağırlıklı medyanının $O(n \lg n)$ en kötü zamanda nasıl hesaplanacağını gösteriniz.
- Ağırlıklı medyanın $\Theta(n)$ en kötü zamanda, Kısım 9.3 teki SELECT algoritması gibi bir doğrusal-zaman medyan algoritması kullanarak nasıl hesaplanacağını gösteriniz.

Postane konumlandırma problemi şu şekilde tanımlanır. Elimizde n sayıda $p_1, p_2, \dots, p_n$ noktası var ve bunlar $w_1, w_2, \dots, w_n$ ağırlıklarına sahipler. a ve b arasındaki uzaklığın $d(a, b)$ olduğu durumda $\sum_{i=1}^n w_i d(p, p_i)$ toplamını minimize edecek bir p noktası bulmak istiyoruz (bu noktanın girdi noktalarından birisi olması şart değildir).

- Noktaların sadece reel sayılar olduğu ve a ve b arasındaki uzaklığın $d(a, b) = |a - b|$ olduğu tek boyutlu postane konumlandırma problemi için ağırlıklı medyan yönteminin en iyi çözümlerden biri olduğunu açıklayınız.
- Noktaların (x, y) koordinat eşleri olduğu ve $a = (x_1, y_1)$ ve $b = (x_2, y_2)$ noktaları arasındaki uzaklığın $d(a, b) = |x_1 - x_2| + |y_1 - y_2|$ ce verilen **Manhattan uzaklığı** olduğu 2 boyutlu postane konumlandırma problemi için en iyi çözümü bulunuz.

9-3 Küçük sıra istatistikleri

n sayıdan i nci sıra istatistiğini seçmek için SELECT tarafından kullanılan karşılaştırmaların en kötü durumdaki sayısı $T(n)$ in $T(n) = \Theta(n)$ ı karşıladığını göstermiştik ancak Θ -notasyonunda saklanan sabit oldukça büyüktür. i nin n ye kıyasla küçük olduğu durumlarda SELECT i bir altyordam olarak kullanan ama en kötü durumda daha az karşılaştırma yapan farklı bir prosedür uygulayabiliriz.

$$a. \quad U_i(n) = \begin{cases} T(n) & i \geq n/2 \text{ ise,} \\ \lfloor n/2 \rfloor + U_i(\lceil n/2 \rceil) + T(2i) & \text{aksi durumda} \end{cases}$$

olduğunda, n elemanın i nci en küçükünü bulmak için $U_i(n)$ sayıda karşılaştırma yapan bir algoritma tanımlayın.

(İpucu: $\lfloor n/2 \rfloor$ ayrık ikili karşılaştırmalarıyla başlayan ve her ikiliden küçük olanı dahil ederek küme üzerinde özyineleme yaparak devam edin).

b. $i < n/2$ ise, $U_i(n) = n + O(T(2i) \lg(n/i))$ olduğunu gösterin.

c. i $n/2$ den küçük bir sabit ise, $U_i(n) = n + O(\lg n)$ olduğunu gösterin.

d. $k \geq 2$ için $i = n/k$ ise, $U_i(n) = n + O(T(2n/k) \lg k)$ olduğunu gösterin.

9-4 Rassal seçimin alternatif analizi

Bu problemde, RANDOMIZED-SELECT prosedürünü analiz etmek için, rassal gösterge değişkenlerini Kısım 7.4.2 de RANDOMIZED-QUICKSORT analizimizde kullandığımızı benzer bir şekilde kullanıyoruz.

Hızlı sıralama analizinde yaptığımız gibi, bütün elemanların farklı olduğunu varsayıyor ve A girdi dizisinin elemanlarını, z_i en küçük i inci eleman olacak şekilde, $z_1, z_2, \dots, z_n$ şeklinde yeniden isimlendiriyoruz. Dolayısıyla, $(A, 1, n, k)$ şeklindeki RANDOMIZED-SELECT işlemi bize z_k yı verir.

$1 \leq i \leq j \leq n$ için, $X_{ijk} = \{\text{algoritmanın çalışması sırasında zaman zaman } z_k \text{ yi bulmak için çalışırken } z_i \text{ zaman zaman } z_j \text{ ile karşılaştırılır}\}$ olsun:

a. $E[X_{ijk}]$ için kesin bir ifade sunun (İpucu: i, j ve k nin değerlerine bağlı olarak ifadeniz farklı değerler alabilir).

b. X_k, z_k yı bulurken A dizisinin elemanları arasında yapılan toplam karşılaştırma sayısı olsun. Aşağıdakini gösteriniz:

$$E[X_k] \leq 2 \left(\sum_{i=1}^k \sum_{j=k}^n \frac{1}{j-i+1} + \sum_{j=k+1}^n \frac{j-k-1}{j-k+1} + \sum_{i=1}^{k-2} \frac{k-i-1}{k-i+1} \right)$$

c. $E[X_k] \leq 4n$ yi gösteriniz.

d. A dizisinin tüm elemanlarının farklı olduğu varsayıldığında, RANDOMIZED-SELECT in beklenen $O(n)$ zamanda çalıştığı sonucuna ulaşılır.

Bölüm notları

En kötü-durumlu doğrusal-zaman medyan-bulma algoritması Blum, Floyd, Pratt, Rivest ve Tarjan tarafından bulunmuştur [50]. Hızlı rassal versiyonu Hoare'e aittir [169]. Floyd ve Rivest [108] elemanların küçük bir örneğinden seçilmiş bir elemanın etrafında özyinelemeli parçalama yapan, daha gelişmiş bir rassal versiyonu geliştirmişlerdir.

Medyanı belirlemek için kaç tane karşılaştırma yapmak gerektiği halen kesin olarak bilinmemektedir. Bent ve John [41] medyan bulmak için $2n$ karşılaştırmanın alt sınırını belirlemiş ve Schönhage, Paterson ve Pippenger [302] $3n$ için bir üst sınır belirlemişlerdir. Dor ve Zwick bu her iki sınırı da geliştirmişlerdir. Onların belirlediği üst sınır [93] $2.95n$ den biraz daha az ve alt sınır [94] $(2 + \epsilon)n$ dir. Bu sınırlar küçük bir pozitif sabit olan ϵ içindir ki böylece Dor ve arkadaşlarının [92] ilgili çalışmasını birazcık geliştirmektedir. Peterson [272] bu sonuçların bazılarını diğer ilgili çalışmalarında açıklamıştır.



III Veri Yapıları

Giriş

Kümeler matematik için olduğu kadar bilgisayar bilimleri içinde temel yapılardır. Matematikte tanımlı kümeler değişmezken, bilgisayar bilimlerinde algoritmalar sayesinde kümeler genişletilip büzülebilir veya zamanla değiştirilebilir. Bu tip kümeleri **dinamik** kümeler olarak adlandırırız. Takip eden beş bölümde sonlu dinamik kümeleri temsil etmek ve bilgisayar üzerinde işlemek için temel teknikler sunulacaktır.

Algoritmalar, kümeler üzerinde farklı tipte ve çeşitli işlemlerin uygulanmasını gerektirebilir. Örneğin, çoğu algoritma sadece kümeye eleman ekleme, kümeden eleman silme ve küme için üyelik testi işlemlerini yapabilmeye ihtiyaç duyar. Bu işlemleri destekleyen dinamik kümeleri **sözlük** olarak adlandırırız. Bazı algoritmalarsa daha karmaşık işlemlere gereksinim duyarlar. Örneğin Bölüm 6 da verilen bellek yığını veri yapılarında tanıtılan minimum-öncelikli kuyruklar, dinamik küme içerisine eleman ekleme ve kümedeki en küçük elemanın kümeden çıkarılması işlemlerine dayanır. Dolayısıyla dinamik bir kümeyi uygulamanın en iyi yolu bu kümenin desteklediği işlemleri tanımlamadan geçer.

Dinamik bir kümenin elemanları

Dinamik bir kümenin tipik bir uygulamasında, kümenin her bir elemanı nitelikleri denetlenebilen, işleme sokulabilen ve bir işaretçi tarafından gösterilebilen bir nesne tarafından ifade edilir. (Kısım 10.3 de programlama ortamlarında temel veri türleri arasına girmeyen nesneler ve işaretçilerin nasıl ifade edildiğini tartışacağız.) Bazı dinamik kümeler, nesnenin niteliklerinden birinin tanımlayıcı **anahtar** nitelik olduğunu varsayarlar. Eğer tüm anahtarlar birbirinden farklıysa, dinamik kümeyi anahtar nitelik değerlerinden oluşan bir küme olarak düşünebiliriz.

Nesne, başka bir nesnenin özellikleri etrafında yapılan ama kümenin ifadesinde başka türlü kullanılabılır olmayan **uydu verisini** içeriyor olabilir. Bununla birlikte nesne, küme işlemlerince elde edilen özelliklere sahip olabilir; bu özellikler verileri veya diğer nesneleri gösteren işaretçileri içerebilir.

Bazı dinamik kümeler baştan, anahtar niteliklerin gerçel sayılar gibi tam sıralı bir kümeden veya alfabetik olarak sıralanmış sözcükler kümesinden elde edilmiş olduğunu farz ederler. Tam sıralama, kümenin en küçük elemanını tanımlamamıza, veya örneğin bir küme içinde verilen bir elemandan sonra yer alan elemanın daha büyük olduğundan bahsetmemize izin verir.

Dinamik kümeler üzerindeki işlemler

Dinamik kümeler üzerindeki işlemler iki kategoride gruplandırılabilir: bir küme hakkında basitçe bir bilgi döndüren **sorgular** ve küme üzerinde değişikliğe neden olan **işlemleri düzenleme**. Dinamik kümeler üzerindeki bazı tipik işlemlerin bir listesi aşağıda sunulmuştur. Özel bir dinamik küme uygulaması genellikle belirtilen bu işlemlerin bir kısmının gerçekleştirilmesini gerektirir.

SEARCH(S, k)

S kümesi ve k anahtar değeri için $x.key = k$ olacak şekilde veya eğer S kümesi böyle bir elemanı içermiyorsa NIL değerine eşit olacak şekilde S kümesinin bir elemanını gösteren x işaretçisini geri döndüren sorgudur.

INSERT(S, x)

x işaretçisinin gösterdiği eleman ile S kümesini genişleten bir değişiklik yapma işlemidir. Genellikle x işaretçisinin gösterdiği elemanın S kümesinin tanımıda yer alan tüm niteliklerine bir ilk değer atandığını varsayınız.

DELETE(S, x)

x işaretçisinin gösterdiği S kümesinin elemanını kümeden çıkaran bir değişiklik yapma işlemidir. (Bu işlemin parametre olarak bir anahtar değer almadığına, bir işaretçi aldığını dikkat ediniz).

MINIMUM(S)

Tam sıralı bir S kümesinin en küçük anahtar değerine sahip elemanını gösteren işaretçiyi geri döndüren sorgudur.

MAXIMUM(S)

Tam sıralı bir S kümesinin en büyük anahtar değerine sahip elemanını gösteren işaretçiyi geri döndüren sorgudur.

SUCCESSOR(S, x)

Tam sıralı bir S kümesinin bir x elemanı verildiğinde S kümesindeki bir sonraki elemanı gösteren işaretçiyi geri döndüren sorgudur. Eğer x elemanı S kümesinin en büyük elemanı ise NIL değeri geri döndürülür.

PREDECESSOR(S, x)

Tam sıralı bir S kümesinin bir x elemanı verildiğinde S kümesindeki bir önceki elemanı gösteren işaretçiyi geri döndüren sorgudur. Eğer x elemanı S kümesinin en küçük elemanı ise NIL değeri geri döndürülür.

Bazı durumlarda farklı olmayan anahtara sahip kümeler üzerinde de uygulanabilmesi açısından SUCCESSOR ve PREDECESSOR sorgularını genişletebiliriz. Normal olan varsayım, n anahtarlı bir küme için, bir adet MINIMUM çağrısını takip eden $n - 1$ adet SUCCESSOR un çağırılması kümedeki elemanların sıralı biçimde sayılmasıdır.

Genellikle bir küme işleminin gerçekleştirilmesi için harcanan süre kümenin boyutuna bağlı olarak ölçülür. Örneğin, Bölüm 13 de n elemanlı bir küme üzerinde yukarıda listelenen tüm küme işlemlerinin $O(\log n)$ sürede gerçekleştirilebileceği bir veri yapısı tanımlanmıştır.

Kısım III e genel bakış

Dinamik kümeleri uygulamak için Bölüm 10–14 ile çeşitli veri yapılarını tanıtacağız; bunların bir çoğunu çeşitli problemlerin çözümü için etkili algoritmalar oluşturmada kullanacağız. Bölüm 6 da bellek yığını adı ile tanıttığımız önemli bir veri yapısını zaten görmüştük.

Bölüm 10, yığınlar, kuyruklar, bağlı listeler ve köklü ağaçlar gibi basit veri yapılarıyla çalışmanın temellerini sunar. Aynı zamanda programlama ortamlarında ilkel veri türü olarak tanımlanmayan nesne ve işaretçileri nasıl ifade edeceğimizi gösterir. Eğer başlangıç düzeyinde bir programlama dersi aldıysanız bahsedilenlerin çoğu size oldukça aşina gelecektir.

Bölüm 11, INSERT, DELETE ve SEARCH sözlük işlemlerini destekleyen kıyımlı arama tablolarını tanıtır. En kötü durumda çırpılama SEARCH işlemini çalıştırmak için $\Theta(n)$ süreye ihtiyaç duyar, ancak çırpı-tablosu işlemlerinin gerçekleştirilebilmesi için beklenen süre $O(1)$ dir. Çırpılama işleminin analizi olasılığa dayanır, ancak çoğu bölümün anlaşılabilmesi için bu konuda bir alt yapıya gerek yoktur.

Bölüm 12 de incelenen İkili Arama Ağaçları yukarıda listelenen tüm dinamik küme işlemlerini sağlarlar. En kötü durumda n elemanlı bir ağaçta her bir işlem $\Theta(n)$ sürede tamamlanır, fakat bir rassal ikili arama ağacında her bir küme işlemi için beklenen süre $O(\log n)$ dir. İkili arama ağaçları çoğu diğer veri yapısı için temel teşkil eder.

Bölüm 13, ikili arama ağaçlarının değişik bir türü olan Kırmızı–Siyah Ağaçlarını tanıtır. Alışlagelen ikili arama ağaçlarının aksine, kırmızı–siyah ağaçları iyi bir performansla çalışmak için garanti verirler: küme işlemleri en kötü durumda $O(\log n)$ sürede tamamlanır. Bir kırmızı–siyah ağacı bir dengeli arama ağacıdır; B-ağacı adıyla bilinen diğer bir dengeli arama ağacı Kısım V, Bölüm 18 de sunulmuştur. Kırmızı–Siyah ağaçlarının mekanizması biraz karmaşık olmasına rağmen, bu ağaçların çoğu özelliğini çalışma mekanizmasını detaylı olarak çalışmanıza gerek kalmadan bu bölümden derleyebilirsiniz. Bununla beraber, kod üzerinden giderseniz muhtemelen bu bölümü oldukça öğretici bulabilirsiniz.

Bölüm 14 de yukarıda listelenen temel küme işlemlerinden başka işlemleri destekleyecek şekilde kırmızı–siyah ağaçlarının nasıl güçlendirildiğini göstereceğiz. Öncelikle, bir anahtarlar kümesi için sıra istatistiklerini dinamik olarak elde edebilmek amacıyla ağaçları büyüteceğiz. Ardından gerçel sayı aralıklarını sağlamak için farklı yolla ağaçları güçlendireceğiz.

Bu bölümde, işaretçileri kullanan basit veri yapılarıyla dinamik kümelerin nasıl temsil edilebileceğini inceleyeceğiz. İşaretçileri kullanarak bir çok karmaşık veri yapısının oluşturabilmesine rağmen, bu bölümde sadece yığınlar, kuyruklar, bağlı listeler ve köklü ağaçlar gibi ilkel veri yapılarını sunacağız. Aynı zamanda dizilerden nesne ve işaretçileri sentezleme yollarını göstereceğiz.

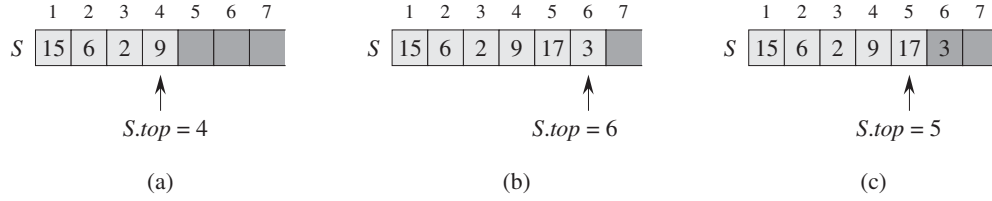
10.1 Yığınlar ve kuyruklar

Yığınlar ve kuyruklar önceden belirlenmiş bir DELETE işlemi ile elemanları kümeden silinebilen dinamik kümelerdir. Bir *yığın*da kümeden silinecek eleman daima kümeye en son eklenen elemandır: Yığınlar “*son giren, ilk çıkan (LIFO)*” politikasını uygular. Benzer biçimde, bir *kuyruk*ta kümeden silinecek eleman daima kümeye ilk eklenen elemandır: Dolayısıyla kuyruklarda ise “*ilk giren, ilk çıkan (FIFO)*” politikası uygulanır. Yığın ve kuyrukları çeşitli ve etkili yollarla bir bilgisayarda kodlamak mümkündür. Bu kısımda yığın ve kuyruk yapılarını oluşturabilmek için basit bir diziyi nasıl kullanacağımızı göstereceğiz.

Yığınlar

Bir yığın üzerinde INSERT işlemi sıklıkla PUSH ve herhangi bir parametre almayan DELETE işlemi ise POP olarak adlandırılır. Bu isimler kafeteryalarda yer alan yaylı tabak yığını sistemleri gibi fiziksel yığımlardan esinlenerek verilmiştir. Tabak yığınının tabakları alma sırası, tabakları sisteme koyma sırasının tersidir ve daima en üstteki tabak erişilebilirdir.

Şekil 10.1 de görüldüğü gibi, en çok n eleman içeren bir yığını $S[1..n]$ dizisi ile ifade edebiliriz. Dizi, $S.top$ özelliğine sahiptir ve bu nitelik diziyi en son ek-



Şekil 10.1 S yığınının bir dizi ile gösterilmesi. Yığın elemanları sadece açık gölgeli konumlarda yer alır. **(a)** S yığını 4 elemana sahiptir. En üstteki eleman 9 dur. **(b)** $PUSH(S, 17)$ ve $PUSH(S, 3)$ çağrılarının sonra S yığını. **(c)** $POP(S)$ çağrısı S yığınının 3 elemanı geri döndürülür. 3 elemanı halen dizi üzerinde olmasına rağmen S yığınının bir elemanı değildir, yığının üst elemanı 17 olur.

lenen elemanın indisini belirtir. Dolayısıyla yığının en altındaki eleman dizinin $S[1]$ elemanı ve yığının en üstünde yer alan eleman ise dizinin $S[S.top]$ elemanı olmak üzere yığın $S[1..S.top]$ elemanlarından oluşur.

$S.top = 0$ olduğunda yığın hiç bir eleman içermez ve **boş** olarak adlandırılır. Yığının boş olup olmadığını $STACK-EMPTY$ sorgu işlemi ile test edebiliriz. Eğer boş bir yığından eleman çekilmeye çalışılırsa, yığında **alttan taşma** hatası oluşur. Eğer n elemanlı bir yığına eleman eklenmeye çalışılırsa yığında üste **üstten taşma** hatası oluşur (Kabakod gösteriminde yığında taşma olup olmadığı denetlenmemiştir).

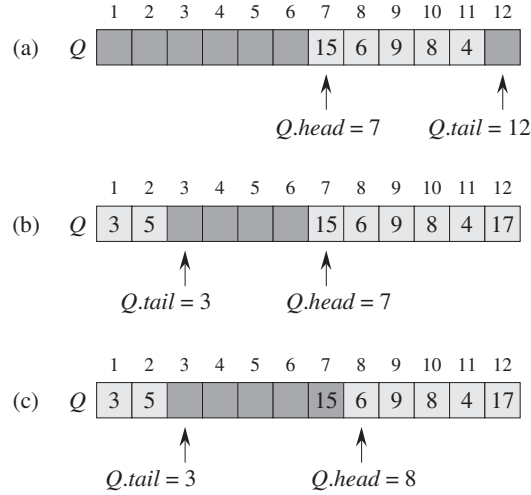
Her yığın işlemini bir kaç satırlık kodla gerçekleştirebiliriz.

```
STACK-EMPTY (S)
1  if  $S.top == 0$ 
2      return TRUE
3  else return FALSE
```

```
PUSH(S, x)
1   $S.top = S.top + 1$ 
2   $S[S.top] = x$ 
```

```
POP(S)
1  if STACK-EMPTY (S)
2      error "alttan taşma"
3  else  $S.top = S.top - 1$ 
4      return  $S[S.top + 1]$ 
```

Şekil 10.1 $PUSH$ ve POP işlemlerinin etkilerini gösterir. Üç yığın işlemi de $O(1)$ sürede tamamlanır.



Şekil 10.2 $Q[1..12]$ dizisi kullanılarak bir kuyruğun temsili. Kuyruk elemanları dizinin sadece açık gölgeli konumlarında bulunur. **(a)** Kuyruk, $Q[7..11]$ konumlarında 5 eleman içerir. **(b)** $ENQUEUE(Q, 17)$, $ENQUEUE(Q, 3)$ ve $ENQUEUE(Q, 5)$ fonksiyon çağrıları sonraki kuyruğun durumu. **(c)** $DEQUEUE(Q)$ çağrısı sonucu 15 anahtar değerinin geri döndürülmesiyle kuyruğun son durumu. Kuyruğun yeni başlığı 6 anahtar değerine sahiptir.

Kuyruklar

Kuyruğa INSERT işlemi ENQUEUE ve kuyruktan DELETE işlemi DEQUEUE ile gerçekleştirilir. Yığınlardaki POP işleminde olduğu gibi DEQUEUE fonksiyonu herhangi bir parametre almaz. Kuyruk yapılarındaki FIFO özelliği, bir veznede ödeme yapmak için sıraya giren müşterilere benzetilebilir. Kuyruk yapısı, **başlık** ve **kuyruk** niteliklerine sahiptir. Kuyruk yapısına bir eleman ekleneceği zaman ekleme işlemi daima kuyruğun sonuna yapılır, yani ödeme yapmak için vezneye gelen bir müşteri sıranın sonuna geçer. Kuyruk yapısından bir eleman silinirken de kuyruğun başından silinir, yani ödeme yapma sırası gelen müşteri vezneden ayrılır.

Şekil 10.2 en çok $n - 1$ eleman içerebilen bir kuyruk yapısını $Q[1..n]$ dizisini kullanarak oluşturmanın bir yolunu göstermektedir. Kuyruk yapısı, kuyruğun başını yani kuyruğun ilk elemanının indisini gösteren $Q.head$ niteliğine sahiptir. $Q.tail$ niteliği ise kuyruk yapısına en son eklenen elemanın indisini belirtir. Kuyruğun elemanları, dizinin $Q.head, Q.head + 1, \dots, Q.tail - 1$ indislerinde konumlanır. Bu elemanları dizinin n indisli elemanını dizinin 1. elemanı takip edecek şekilde dairesel sırada sarmallayabiliriz. $Q.head = Q.tail$ olduğunda kuyruk boşdur. Başlangıçta $Q.head = Q.tail = 1$ dir. Eğer boş bir kuyruktan eleman silinmeye çalışılırsa, “alttan taşma” hatası oluşur. $Q.head = Q.tail + 1$ olduğunda kuyruk

doludur ve bu durumda kuyruğa yeni bir eleman eklenmeye çalışılırsa, “taşma” hatası oluşur.

Kitapta ENQUEUE ve DEQUEUE prosedürlerinde taşma hataları gözardı edilmiştir. (Alıştırma 10.1-4 de bu hata denetimlerini gerçekleyen kod okuyucuya bırakılmıştır). Kabakodda $n = Q.length$ olduğu varsayılmıştır.

```

ENQUEUE ( $Q, x$ )
1   $Q[Q.tail] = x$ 
2  if  $Q.tail == Q.length$ 
3       $Q.tail = 1$ 
4  else  $Q.tail = Q.tail + 1$ 

```

```

DEQUEUE ( $Q$ )
1   $x = Q[Q.head]$ 
2  if  $Q.head == Q.length$ 
3       $Q.head = 1$ 
4  else  $Q.head = Q.head + 1$ 
5  return  $x$ 

```

Şekil 10.2 de ENQUEUE ve DEQUEUE işlemleri gösterilmektedir. Her iki işlemde $O(1)$ sürede tamamlanır.

Alıştırmalar

10.1-1

Şekil 10.1 de verilen ve başlangıçta boş olan bir S yığınının $S[1..6]$ dizisinde temsil eden modeli kullanarak $PUSH(S, 4)$, $PUSH(S, 1)$, $PUSH(S, 3)$, $POP(S)$, $PUSH(S, 8)$ ve $POP(S)$ işlemlerini sırasıyla uygulayarak her işlemin sonucunda yığını görüntüleyiniz.

10.1-2

Yığınların içerdiği toplam eleman sayısı n olmadığı sürece her iki yığında da üste taşma hatası oluşmayacak şekilde $A[1..n]$ gibi n elemanlı bir dizi üzerinde iki farklı yığını nasıl oluşturabileceğimizi açıklayınız. $PUSH$ ve POP işlemleri $O(1)$ sürede tamamlanmalıdır.

10.1-3

Şekil 10.2 ile verilen modeli kullanarak başlangıçta boş olan ve $Q[1..6]$ dizisi üzerinde tutulan bir kuyruk üzerinde $ENQUEUE(Q, 4)$, $ENQUEUE(Q, 1)$, $ENQUEUE(Q, 3)$, $DEQUEUE(Q)$, $ENQUEUE(Q, 8)$ ve $DEQUEUE(Q)$ işlemlerini sırasıyla uygulayarak her işlem sonucunda kuyruğu görüntüleyiniz.

10.1-4

Bir kuyruk yapısındaki taşma ve aşağı taşma hatalarını denetleyebilecek şekilde $ENQUEUE$ ve $DEQUEUE$ yi yeniden yazınız.

10.1-5

Yığın yapısında ekleme ve silme işlemleri yığının sonunda gerçekleştirilirken, kuyruk yapısında ekleme işlemi kuyruğun sonundaki uçta, silme işlemi ise kuyruğun diğer ucunda gerçekleşir. Çift taraflı **kuyruk** yapılarında ekleme ve silme işlemlerinin kuyruğun her iki ucunda da gerçekleşmesine izin verilir. Bir dizi üzerinde tanımlı iki sonlu kuyruğun herhangi bir ucuna ekleme veya silme işlemini $O(1)$ sürede tamamlayan 4 adet prosedür yazınız.

10.1-6

İki yığın kullanarak bir kuyruk yapısının nasıl ifade edilebileceğini gösteriniz. Kuyruk işlemlerinin çalışma zamanlarını analiz ediniz.

10.1-7

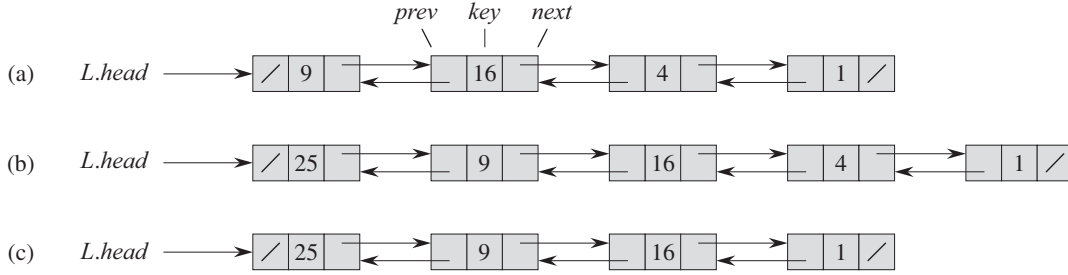
İki kuyruk kullanarak bir yığın yapısının nasıl ifade edilebileceğini gösteriniz. Yığının işlemlerinin çalışma zamanlarını analiz ediniz.

10.2 Bağlı listeler

Bağlı listeler, nesnelerin doğrusal bir sırada yerleştirildiği veri yapılarıdır. Ancak doğrusal sıra, bir dizinin aksine dizinin indisi ile değil de her bir nesneyi gösteren işaretçilerle belirlenir. Bağlı listeler sayfa 230 da listelenen tüm küme işlemlerini destekleyecek biçimde dinamik kümeler için basit ve esnek bir gösterilim sağlarlar.

Şekil 10.3 de gösterildiği gibi, **çift yönlü bir L bağlı listesinin** her bir elemanını *key* nitelik ile birlikte *prev* ve *next* işaretçilerini nitelik olarak barındıran bir nesnedir. Nesne bu nitelikler haricinde uydu verileri de içerebilir. Listenin bir x elemanı verildiğinde $x.next$ işaretçisi listenin bir sonraki nesnesini (elemanını) gösterir ve $x.prev$ işaretçisi listenin bir önceki elemanını gösterir. Eğer $x.prev = NIL$ ise x elemanının bir öncülü yoktur ve bundan dolayı x listenin ilk elemanı yani **kafası**dır. Benzer biçimde eğer $x.next = NIL$ ise x elemanının bir ardılı yoktur ve bundan dolayı x listenin son elemanı yani **kuyruğudur**. $L.head$ niteliği bağlı listenin ilk elemanını gösteren işaretçidir. Eğer $L.head = NIL$ ise liste boştur.

Bir bağlı liste, tek başına veya çift yönlü, sıralı veya sırasız, dairesel veya dairesel olmayan gibi farklı formlara sahip olabilir. Eğer liste **tek başına bağlı** ise her nesnenin *prev* niteliği kaldırılır. Eğer liste **sıralı** ise doğrusal sıra liste elemanlarının barındırdığı *key* niteliklerinin sırası ile ilintilidir. Anahtar nitelik değerinin minimum olduğu eleman listenin başında ve anahtar nitelik değerinin maksimum olduğu eleman listenin kuyruğunda yer alır. Eğer liste **sırasız**, elemanlar herhangi bir sıra gözetilmeksizin listeye yerleştirilir. Bir **dairesele listede** listenin başındaki elemanın *prev* niteliği listenin kuyruğunu işaret eder ve benzer olarak listenin



Şekil 10.3 (a) $\{1, 4, 9, 16\}$ dinamik kümesini temsil eden çift yönlü bağlı liste. Dinamik kümenin her elemanı, listede *key* niteliği ile bir önceki ve bir sonraki nesneleri gösteren işaretçi (oklarla gösterilen) niteliklerini içeren bir nesnedir. Liste kuyruğunun *next* niteliği ve liste başının önceki niteliği/işareti ile temsil edilen NIL değeridir. *L.head* listenin başını gösterir. (b) $x.key = 25$ olacak şekilde $LIST-INSERT(L, x)$ fonksiyonunun çalıştırılmasıyla, listenin başına *key* niteliğinin değeri 25 olan yeni bir nesne eklenir. Eklenen bu yeni nesne *key* niteliğinin değeri 9 olan listenin eski başını gösterir. (c) x , *key* değeri 4 olan nesneyi gösteren işaretçi olmak üzere bir önceki işlemi takip eden $LIST-DELETE(L, x)$ fonksiyon çağrısının sonucu.

kuyruğundaki elemanın *next* niteliği listenin başını gösterir. Dairesel bağlı listeyi bir nesneler halkası olarak düşünebiliriz. Bu bölümün kalan kısmında sırasız ve çift yönlü listelerle çalıştığımızı varsayacağız.

Bağlı listelerde arama

$LIST-SEARCH(L, k)$ prosedürü L listesinde k anahtarına sahip ilk elemanı basit doğrusal arama ile bularak bu nesneyi gösteren bir işaretçi döndürür. Eğer liste üzerinde k anahtarına sahip herhangi bir eleman bulunmuyorsa, prosedür NIL değerini geri döndürür. Şekil 10.3 (a) ile verilen bağlı liste üzerinde $LIST-SEARCH(L, 4)$ fonksiyonu listenin üçüncü elemanını gösteren bir işaretçi döndürürken $LIST-SEARCH(L, 7)$ fonksiyonu NIL değer döndürür.

$LIST-SEARCH(L, k)$

- 1 $x = L.head$
- 2 **while** $x \neq NIL$ and $x.key \neq k$
- 3 $x = x.next$
- 4 **return** x

n elemanlı bir listede, $LIST-SEARCH$ prosedürü en kötü durumda listenin tamamı üzerinde arama yapılacağından $\Theta(n)$ sürede tamamlanır.

Bağlı listeye eleman ekleme

Key niteliğinin değeri önceden atanmış bir x nesnesi verildiğinde $LIST-INSERT$ prosedürü Şekil 10.3 (b) deki gibi x elemanını listenin önüne ekler ve listenin başı olarak belirlenir.

$LIST-INSERT(L, x)$

```

1   $x.next = L.head$ 
2  if  $L.head \neq \text{NIL}$ 
3       $L.head.prev = x$ 
4   $L.head = x$ 
5   $x.prev = \text{NIL}$ 

```

(Nesne niteliklerinin kademeli olarak kullanılabildiğini hatırlayalım. Böylece $L.head.prev$ niteliği $L.head$ niteliğinin işaret ettiği nesnenin $prev$ niteliğini temsil eder). n elemanlı bir listede LIST-INSERT fonksiyonu $O(1)$ sürede tamamlanır.

Bağlı listeden eleman silme

LIST-DELETE prosedürü bir L listesinden bir x elemanını siler. Bu işlem için x elemanını gösteren bir işaretçi verilmek zorundadır ve bu işlem listedeki işaretçileri güncelleyerek x elemanını listeden çıkarır. Eğer anahtar niteliğinin değeri belirlenmiş bir eleman silinmek istenirse öncelikle LIST-SEARCH i çağırarak silinecek elemanı gösteren işaretçiyi belirlemek zorunda kalırız.

LIST-DELETE (L, x)

```

1  if  $x.prev \neq \text{NIL}$ 
2       $x.prev.next = x.next$ 
3  else  $L.head = x.next$ 
4  if  $x.next \neq \text{NIL}$ 
5       $x.next.prev = x.prev$ 

```

Şekil 10.3(c) bağlı listeden bir elemanın nasıl silineceğini gösterir. LIST-DELETE işlemi $O(1)$ sürede tamamlanır, ancak anahtar niteliğinin değeri belli olan bir elemanı silmek istersek, en kötü durumda öncelikle LIST-SEARCH i çağırıp silinecek elemanı liste üzerinde bulmak zorunda olduğumuzdan silme işlemi $\Theta(n)$ sürede tamamlanır.

Nöbetçiler

Eğer listenin başı ve kuyruğundaki sınır koşullarını gözardı edebilirsek, LIST-DELETE in kodu daha basit bir hale gelir:

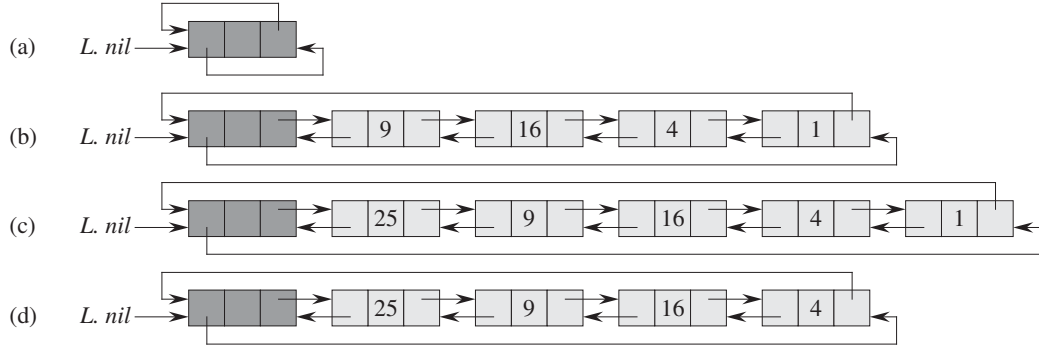
LIST-DELETE' (L, x)

```

1   $x.prev.next = x.next$ 
2   $x.next.prev = x.önceki$ 

```

Bir **nöbetçi**, sınır koşullarını sadeleştirmemize izin veren sahte bir nesnedir. Örneğin L listesinde $L.nil$ nesnesini, değerini temsil eden ve diğer tüm nesnelerin



Şekil 10.4 Dairesel, nöbetçili çift yönlü bağlı liste. Nöbetçi $L.nil$ başlık ile kuyruk arasında yer alır. $L.nil.next$ nöbetçisiyle liste başına erişilebildiğinden $L.head$ niteliğine daha fazla gerek yoktur. (a) Boş liste. (b) Listenin başı 9 anahtar değerine sahip nesne ve kuyruğu 1 anahtar değerine sahip nesne olacak şekilde Şekil 10.3 ile verilen liste. (c) $x.key = 25$ olacak şekilde $LIST-INSERT'(L, x)$ çağrısından sonra listenin son durumu. Yeni nesne listenin başı olur. (d) Anahtar değeri 1 olan nesnenin silinmesi sonrası listenin son hali. Listenin yeni kuyruğu 4 anahtar değerine sahip nesnedir.

sahip olduğu niteliklere sahip olan bir nesne olarak aldığımızı varsayalım. Liste kodları içerisinde her ne kadar NIL nesnesine referans vermiş olsak da, NIL yerine $L.nil$ nesnesini referans gösteren bir işaretçi kullanabiliriz. Şekil 10.4 ile gösterildiği gibi nöbetçi kullanma işlemi sıradan **çift yönlü bir bağlı listeyi dairesel** çift yönlü bağlı listeye dönüştürerek, $L.nil$ nöbetçinin listenin başlığı ve kuyruğu arasında konumlanmasını sağlar. $L.nil.next$ niteliği listenin başlığını işaret ederken $L.nil.prev$ listenin kuyruğunu gösterir. Benzer olarak, liste kuyruğunun $next$ niteliği ile liste başlığını $prev$ niteliği $L.nil$ nöbetçi nesnesini gösterir. $L.nil.next$ listenin başlığını gösterdiğinden $L.head$ niteliğini elimine ederek bunun yerine $L.nil.next$ niteliğini kullanabiliriz. Şekil 10.4(a) sadece bir nöbetçi nesne içeren boş bir bağlı liste göstermektedir. $L.nil$ gözcü nesnesinin $prev$ ve $next$ nitelikleri $L.nil$ nesnesinin kendisini gösterir.

$LIST-SEARCH$ prosedürünün kodu daha önceki ile hemen hemen aynı kalır, sadece NIL ve $L.head$ aşağıda verildiği gibi değiştirilir.

$LIST-SEARCH'(L, k)$

```

1   $x = L.nil.next$ 
2  while  $x \neq L.nil$  and  $x.key \neq k$ 
3       $x = x.next$ 
4  return  $x$ 
```

Listeden bir eleman silmek için önceki iki satırlık $LIST-DELETE'$ prosedürünü kullanırız. Aşağıda verilen fonksiyon ise listeye bir eleman ekler.

LIST-INSERT' (L, x)

- 1 $x.next = L.nil.next$
- 2 $L.nil.next.prev = x$
- 3 $L.nil.next = x$
- 4 $x.prev = L.nil$

Şekil 10.4, LIST-INSERT' ve LIST-DELETE' in basit bir liste üzerindeki etkilerini örnekler.

Nöbetçiler veri yapısı işlemlerinin asimptotik süre sınırlarını nadiren azaltır, ancak sabit çarpanları küçültebilir. Döngüler içinde nöbetçileri kullanmanın sağlayacağı kazanç genellikle hızdan ziyade kodun daha açık olmasını sağlama bakımından önemlidir. Nöbetçiler kullanıldığında bağlı liste kodu sadeleşir, ancak sadece LIST-INSERT' ve LIST-DELETE' prosedürlerinin çalışma zamanı analizinde $O(1)$ sürede tasarruf sağlandığı görülür. Diğer durumlarda, nöbetçileri kullanmak kodun tek bir döngü içerisinde yer almasını sağlar. Buda n veya n^2 cinsinden ifade edilen çalışma zamanının sadece katsayısında azalmaya yol açar.

Nöbetçileri akla yatkın olarak kullanırız. Çok sayıda küçük bağlı listeler mevcutken nöbetçilerin kullanımı ekstra bellek kullanımı gerektirir. Bu kitapta nöbetçiler sadece kodu basitleştirmek için kullanılmıştır.

Alıştırmalar

10.2-1

Tek başına bağlı listelerde $O(1)$ sürede tamamlanacak bir dinamik-küme işlemi INSERT yazabilir misiniz? DELETE için bu mümkün müdür?

10.2-2

Tek başına bir bağlı liste kullanarak bir yığın yapısı oluşturunuz. PUSH ve POP işlemleri $O(1)$ sürede tamamlanmalıdır.

10.2-3

Tek başına bir bağlı liste kullanarak bir kuyruk yapısı oluşturunuz. ENQUEUE ve DEQUEUE işlemleri $O(1)$ sürede tamamlanmalıdır.

10.2-4

LIST-SEARCH' prosedürü çalışırken döngünün her iterasyonu iki teste gereksinim duyar: Bunlardan biri $x \neq L.nil$ ve diğeri $x.key \neq k$ olup olmadığıdır. Döngü içindeki $x \neq L.nil$ testini nasıl yok edebiliriz?

10.2-5

Tek başına dairesel liste kullanarak INSERT, DELETE ve SEARCH sözlük işlemlerini oluşturunuz. Bu prosedürlerin çalışma sürelerini inceleyiniz.

10.2-6

S_1 ve S_2 iki ayrık küme olmak üzere dinamik-küme işlemi UNION girdi olarak bu iki kümeyi alır ve $S = S_1 \cup S_2$ kümesini geri döndürür. İşlem sonunda S_1 ve S_2 kümeleri yok edilir. Uygun bir bağlı liste veri yapısı kullanarak UNION nun $O(1)$ sürede nasıl tamamlanabileceğini gösteriniz.

10.2-7

n elemanlı tek başına bir bağlı listeyi $\Theta(n)$ sürede tersine döndüren özyinelemeli olmayan bir prosedür yazınız. Prosedür listenin kendisi için ayrılan sabit depolama alanının haricinde ekstra bir bellek alanı kullanmamalıdır.

10.2-8 ★

Çift yönlü listelerde her nesnedeki *prev* ve *next* işaretçileri yerine sadece tek bir *x.np* işaretçisi kullanılarak nasıl oluşturulabileceğini açıklayınız. Tüm işaretçi değerlerinin k -bitlik tam sayılar olarak yorumlanabileceğini ve $x.np = x.prev \text{ XOR } x.next$ olarak tanımlanabileceğini varsayınız. (NIL nesnesinin değeri 0 ile gösterilir.) Listenin başına erişebilmek için gerekli bilgiyi tanımladığınızdan emin olunuz. SEARCH, INSERT ve DELETE işlemlerini bu liste üzerinde nasıl uygulayabileceğimizi gösteriniz. Ayrıca listeyi $O(1)$ sürede nasıl tersine çevirebileceğimizi gösteriniz.

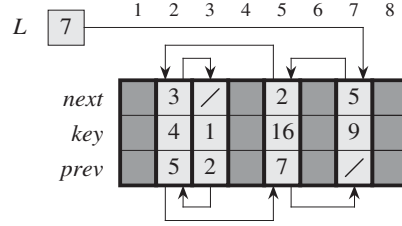
10.3 İşaretçileri ve nesneleri uygulama

İşaretçi ve nesnelerin desteklenmediği dillerde onları nasıl uygulayabiliriz? Bu kısımda işaretçi veri türünün açık olarak tanımlanmadığı dillerde liste veri yapılarını oluşturmanın iki farklı yolunu göreceğiz. Nesne ve işaretçileri, diziler ve dizi indislerinden yola çıkarak sentezleyeceğiz.

Nesnelerin birden çok diziyle temsili

Bir nesneler koleksiyonunu, nesnelerin her bir niteliği için ayrı bir dizi kullanarak ifade edebiliriz. Örnek olarak, Şekil 10.5 de üç farklı dizi ile Şekil 10.3(a) ile verilen bağlı listenin nasıl oluşturulabileceği gösterilmiştir. *key* dizisi, dinamik küme elemanlarının anahtar niteliklerinin değerlerini içerir ve dinamik küme elemanlarındaki işaretçiler *prev* ve *next* dizilerinde tutulur. Verilen x dizi indisi için *key*[x], *next*[x] ve *prev*[x] birlikte bağlı listenin bir nesnesini ifade eder. Bu yorumlama altında, x işaretçisi basitçe *key*, *prev* ve *next* dizilerinin ortak indisidir.

Şekil 10.3(a) da, 4 anahtar değerine sahip nesne, bağlı listede anahtar değeri 16 olan nesneyi takip eder. Şekil 10.5 de, 4 anahtar değeri anahtar dizisinin ikinci



Şekil 10.5 Şekil 10.3(a) ile verilen bağlı listenin *key*, *next* ve *prev* dizileri tarafından temsili. Dizilerin her bir dikey dilimi tek bir nesneyi ifade eder. Saklı işaretçiler, üstte gösterilen dizi indisleri ile ilişkilidir; oklar onları nasıl yorumlamamız gerektiğini gösterir. Açık gölgeli nesne konumları listenin elemanlarını içerir. *L* değişkeni listenin başlığının indisini tutar.

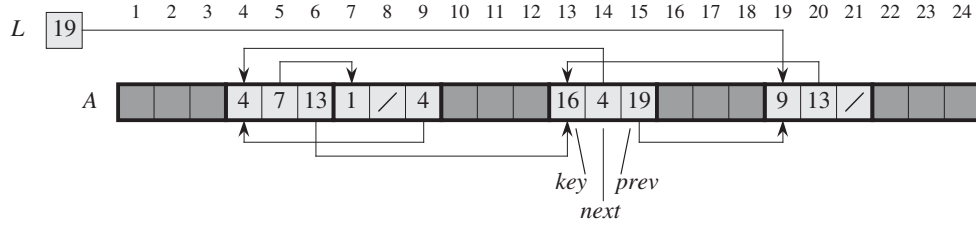
elemanı olan *key*[2] ve 16 anahtar değerine sahip olan nesne anahtar dizisinin 5. elemanı olan anahtar[5] elemanıdır. Böylece *next*[5] = 2, *prev*[2] = 5 olur. Listenin kuyruk elemanının *next* niteliği ve listenin başlığının *prev* niteliği her ne kadar NIL olsa da, genellikle bu değeri temsil edebilmek için dizilerin mümkün olan indislerinden farklı olacak biçimde 0 veya -1 gibi bir tam sayı kullanırız. *L* değişkeni listenin başlığının indisini tutar.

Nesnelerin tek bir dizi ile temsili

Bilgisayar belleğinde *M* uygun bir tam sayı olmak üzere sözcükler tipik olarak 0 dan *M* - 1 e kadar tam sayı değerlerle adreslenirler. Çoğu programlama dilinde, bir nesne bilgisayar belleğinde bitişik bellek konumlarında tutulur. Bir işaretçi basitçe nesnenin bellek konumunun başlangıcını gösteren bir adrestir ve nesne içindeki diğer adres konumlarına yani nesnenin niteliklerine işaretçiye bir tamsayı (ofset) ekleyerek ulaşabiliriz.

Benzer yaklaşımı işaretçi veri türlerinin desteklenmediği dillerde nesneleri ifade edebilmek için kullanabiliriz. Örneğin, Şekil 10.3(a) ve Şekil 10.5 de verilen bağlı listenin tek bir A dizisi içinde nasıl tutulabileceği Şekil 10.6 da gösterilmiştir. Bir nesne A dizisinin *A*[*j*.*k*] altdizisinde tutulur. Nesnelerin her bir niteliği 0 ile *k* - *j* aralığındaki ofset değeriyle ilintilidir ve nesneyi gösteren eden işaretçi *j* indisidir. Şekil 10.6 da verilen nesnelerin *key*, *prev* ve *next* niteliklerine sırasıyla 0, 1 ve 2 ofset değerleriyle ulaşılabilir. Verilen *i* işaretçisi için *i*.*prev* niteliğinin değerine, *i* işaretçisine 2 ofset değeri eklenerek yani *A*[*i* + 2] elemanı ile ulaşılabilir.

Farklı uzunluktaki nesneleri aynı dizi üzerinde tutmaya izin verdiğinden nesnelerin tek bir dizi üzerindeki gösterilimi oldukça esnekler. Bu tarz bir heterojen nesneler koleksiyonunu yönetme problemi, bütün nitelikleri aynı olan homojen bir nesneler koleksiyonunu idare etmekten daha zordur. Dikkate alacağımız çoğu veri yapısı, homojen elemanların birleşiminden oluşacağından nesnelerin birden çok dizi ile gösterilimi amacımız için yeterli olacaktır.



Şekil 10.6 Şekil 10.3(a) ve Şekil 10.5 de verilen bağlı listenin tek bir A dizisinde gösterilimi. Listenin her bir elemanı dizi üzerinde boyutu 3 olan bitişik altdizilere yerleştirilen birer nesnedir. Nesnelerin *key*, *next* ve *prev* nitelikleri sırasıyla 0, 1 ve 2 offset değerleri ile ilişkilidir. Bir işaretçi nesnenin ilk elemanının indisini gösterir. Nesneler açık gölgeli dizi konumlarında liste elemanlarını içerir ve oklar liste sırasını gösterir.

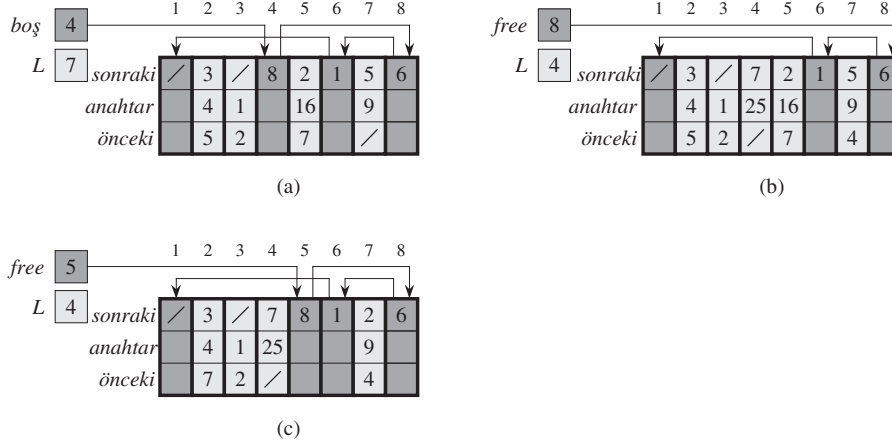
Nesnelerin tahsisi ve serbest bırakılması

Çift yönlü bağlı liste tarafından ifade edilen bir dinamik kümeye anahtar eklemek amacıyla öncelikle kullanılmayan nesneyi gösteren bir işaretçi için bellekte yer tahsisi yapmak zorunda kalırız. Bu işlem, bağlı liste gösteriliminde yer almayan nesnelerin depolanmasını yönetmek için yararlıdır. Bazı sistemlerde, hangi nesnelerin kullanılmadığını belirleme işlemi bir **çöp toplayıcı**nın sorumluluğundadır. Ancak çoğu uygulama, depolama yöneticisine kullanılmayan bir nesneyi döndürmek için sorumluluğu taşıyabilecek düzeyde basittir. Bu kısımda birden çok dizi ile gösterilen çift yönlü bağlı bir liste örneği kullanarak homojen nesneler için bellek tahsisi ve nesnelerin serbest bırakılması problemini araştıracağız.

Birden çok dizi kullanılarak ifade edilen bağlı listeyi oluşturan dizi boyutlarının m ve aynı zamanda dinamik kümenin $n \leq m$ olacak şekilde n eleman içerdiğini varsayalım. Bu durumda n adet nesne dinamik kümenin elemanlarıdır ve geri kalan $m - n$ adet nesne ise **serbesttir**; serbest nesneler dinamik kümeye gelecekte eklenecek elemanları temsil etmek için mevcuttur.

Serbest liste olarak adlandırdığımız tek başına bir bağlı listelerde serbest nesneler tutulur. Serbest liste, sadece listenin sonraki elemanlarını gösteren işaretçileri içeren “*sonraki*” dizisini kullanır. Serbest listenin başı, global olarak tanımlı *serbest* değişkeninde tutulur. Bir L bağlı listesi ile ifade edilen dinamik küme serbest değilken, serbest liste Şekil 10.7 de gösterildiği gibi L bağlı listesi ile iç içe olabilir. Gösterimde her bir nesnenin ya serbest listede ya da L bağlı listesinde olduğunu ancak ikisinde birden yer almadığına dikkat ediniz.

Serbest listeler, bir yığın gibi davranırlar: sonraki nesnesi için tahsis edilen alan en son serbest nesnedir. Bellekte nesneler için alan tahsis etmek ve önceden ayrılan bellek alanını boşaltan prosedürleri oluşturabilmek için, yığınlardaki POP ve PUSH dinamik küme işlemlerini bağlı listeleri kullanarak gerçekleştirebiliriz. Global bir değişken olan *serbest* değişkenini aşağıda verilen prosedürlerde serbest listenin ilk elemanını göstermek için kullandığımızı varsayacağız.



Şekil 10.7 *ALLOCATE-OBJECT* ve *FREE-OBJECT* fonksiyon çağrılarının etkisi. (a) Şekil 10.5 ile verilen bağlı liste açık gölgeli ve boş liste ise koyu gölgeli hücrelerde konumlanmıştır. Oklar boş-liste yapısını gösterir. (b) *anahtar*[4] değerini 25 olarak ayarlayarak *ALLOCATE-OBJECT* çağrısı (4 indisini döndürür) ve *LIST-INSERT*(*L*, 4) çağrısının sonucu. Yeni boş-liste başı boş listede *sonraki*[4] olan 8 nesnesidir. (c) *LIST-DELETE*(*L*, 5) fonksiyonunun çalışmasından sonra, *FREE-OBJECT*(5) çağrısı gerçekleştirilir. Boş liste üzerinde 8 nesnesinin takip ettiği 5 nesnesi boş-listenin yeni başı olur.

ALLOCATE-OBJECT ()

```

1  if free == NIL
2      error "bellekte yeterli alan yok"
3  else x = free
4      free = x.next
5      return x

```

FREE-OBJECT(*x*)

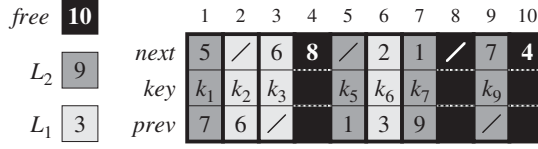
```

1  x.next = free
2  free = x

```

Serbest liste başlangıçta bellekte yer ayrılmayan n adet nesnenin tümünü kapsar. Serbest liste bitince *ALLOCATE-OBJECT* prosedürü hata üretir. Sadece tek bir bağlı liste ile bir kaç bağlı listeyi bile oluşturabiliriz. Şekil 10.8, *key*, *prev* ve *next* dizileri boyunca birbirine geçmiş iki bağlı liste ve bir boş listeyi gösterir.

Yukarıdaki iki prosedür oldukça pratik biçimde $O(1)$ sürede tamamlanır. Bu prosedürleri herhangi homojen nesneler koleksiyonu için çalışacak biçimde nesnenin niteliklerinden birini serbest listenin *next* niteliği gibi davranmaya izin vererek değiştirebiliriz.



Şekil 10.8 L_1 (açık gölgeli), L_2 (koyu gölgeli) iki bağlı listesi ve bir serbest listenin (çok koyu gölgeli) iç içe geçmesi.

Alıştırmalar

10.3-1

$\langle 13, 4, 8, 19, 5, 11 \rangle$ dizisini, birden fazla dizi kullanılarak oluşturulan çift yönlü bir bağlı listede depolanacak şekilde çizimle gösteriniz. Aynı işlemi tek bir dizi kullanılarak oluşturulan bağlı liste için tekrarlayınız.

10.3-2

Homojen bir nesneler koleksiyonu için tek bir dizi gösterilimi ile oluşturulan bağlı liste için ALLOCATE-OBJECT ve FREE-OBJECT prosedürlerini yazınız.

10.3-3

ALLOCATE-OBJECT ve FREE-OBJECT prosedürlerinde, nesnelerin *prev* ve *next* niteliklerini ayarlamaya neden ihtiyaç duymayız?

10.3-4

Çift yönlü bir bağlı listenin tüm elemanlarının birden çok dizi kullanılarak gösteriliminde örneğin ilk m indisli konumunda kompakt bir biçimde tutulması sıklıkla istenir (Bu durum sayfalı, sanal-bellek hesaplama ortamlarında görülür). ALLOCATE-OBJECT ve FREE-OBJECT prosedürlerini kompakt olacak şekilde nasıl oluşturabiliriz? Listenin kendisi dışında bağlı liste elemanlarını gösteren herhangi bir işaretçi olmadığını varsayınız. (*İpucu:* Yığının dizi ile gösterilimini kullanınız).

10.3-5

L , m uzunluklu *key*, *prev* ve *next* dizileri üzerinde depolanan ve uzunluğu n olan çift yönlü bir bağlı liste olsun. Bu dizilerin, F çift yönlü serbest listesini tutan ALLOCATE-OBJECT ve FREE-OBJECT prosedürleri ile yönetildiğini varsayınız. Aynı zamanda m nesnenin n adedinin L listesinde ve geri kalan $m - n$ adedinin ise serbest liste üzerinde olduğunu varsayınız. Bir L listesi ve F serbest listesi verildiğinde, L listesinin elemanlarını dizinin $1, 2, \dots, n$ indisli konumlarında ve serbest liste elemanlarını dizinin $n + 1, n + 2, \dots, m$ indisli konumlarında tutulacak şekilde yerleştiren COMPACTIFY-LIST(L, F) adlı bir prosedür yazınız. Yazdığınız prosedür $\Theta(n)$ sürede tamamlanmalıdır ve sadece fazladan sabit miktarda bir bellek alanı kullanılmalıdır. Yazmış olduğunuz prosedürün doğruluğunu tartışınız.

10.4 Köklü ağaçların gösterimi

Listeleri temsil etmek için bir önceki kısımda verilen yöntemler herhangi bir homojen veri yapısına genişletilir. Bu kısımda, özellikle bağlı liste veri yapılarından köklü ağaçları temsil etme problemine göz atacağız. Öncelikle ikili ağaçları inceleyeceğiz ve ardından keyfi sayıda çocuğa sahip düğümlerden oluşan köklü ağaçlar için bir yöntem sunacağız.

Ağacın her bir düğümünü bir nesneyle temsil ederiz. Bağlı listelerde olduğu gibi, ağacın her bir düğümünün bir *key* niteliğe sahip olduğunu varsayalım. İlgili kalan diğer nitelikler farklı düğümleri gösteren işaretçilerdir ve ağacın türüne göre değişiklik gösterirler.

İkili ağaçlar

Şekil 10.9 bir T ikili ağacında her bir düğümün ebeveyn, sol çocuk ve sağ çocuğunu gösteren işaretçileri bellekte saklamak için p , $left$ ve $right$ niteliklerini nasıl kullanacağımızı gösterir. Eğer $x.p = \text{NIL}$ ise x düğümü ağacın köküdür. Eğer x düğümünün sol çocuğu yoksa $x.left = \text{NIL}$ dir ve benzeri sağ çocuk içinde geçerlidir. T ağacının kökü $T.root$ niteliği tarafından işaret edilir. Eğer $T.kök = \text{NIL}$ ise ağaç boştur.

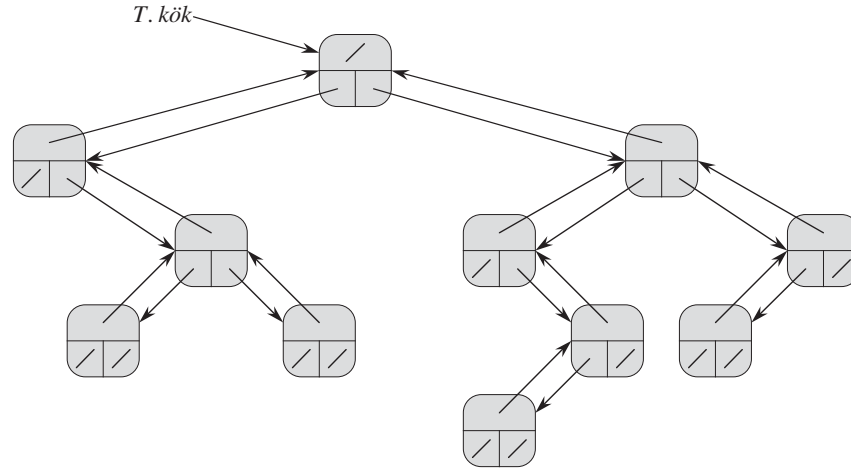
Sınırsız dallanmalı köklü ağaçlar

İkili ağaçlar için belirtilen gösterilimi, ağacın her bir düğümünü en fazla sabit k adet çocuk içerecek biçimde genişletebiliriz: düğümün $left$ ve $right$ nitelikleri yerine $child$, $child$, $\dots$, $child$ eklenebilir. Bir düğümün çocuk sayısı sınırsız olduğunda önceden ne kadarlık bir bellek alanı tahsis etmemiz gerektiğini bilemediğimizden bu tasarım çalışmaz. Diğer yandan, düğümlerin çocuk sayısı k büyük bir sabitle sınırlanmış olsa bile, çoğu düğümün az sayıda çocuğu olabileceği düşünülürse çok büyük bir bellek alanını gereksiz olarak boşa harcamış oluruz.

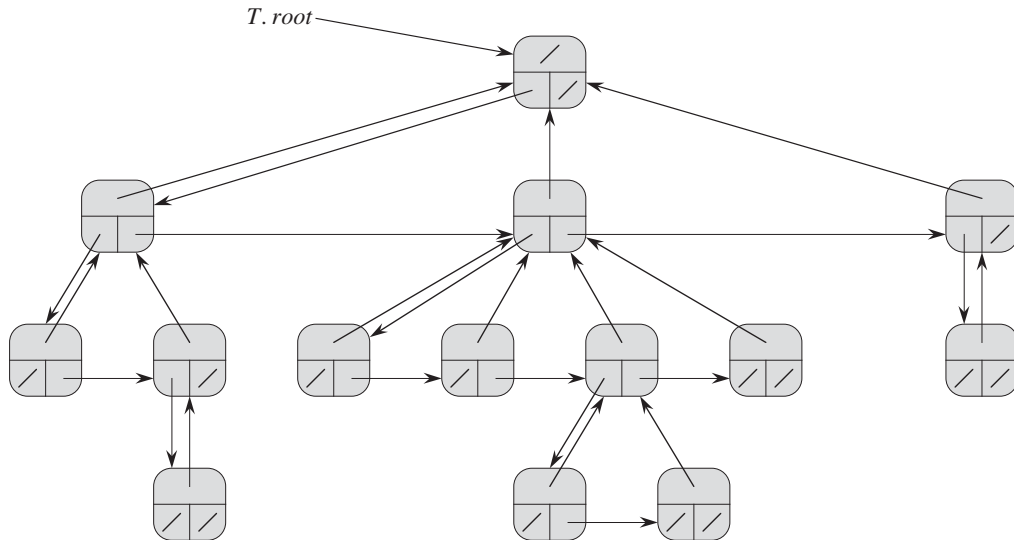
Keyfi sayıda çocuğa sahip düğümlerden oluşan ağaçları temsil etmek için daha zekice bir tasarım mevcuttur. Bu tasarım herhangi bir n -düğümlü köklü ağaç yapısı için sadece $O(n)$ alan kullanma avantajına sahiptir. Şekil 10.10 da ***sol-çocuk, sağ-kardeş gösterilimi*** görüntülenmiştir. Önceden olduğu gibi, her düğüm p ebeveyn işaretçisini içerir ve $T.root$ niteliği T ağacının kökünü işaret eder. Her bir çocuğu gösteren işaretçiler yerine, her bir x düğümü sadece iki işaretçiye sahiptir:

1. $x.left-child$ işaretçisi x düğümünün en solundaki çocuğunu gösterir ve
2. $x.right-sibling$ işaretçisi x düğümünün hemen sağında yer alan kardeş düğümünü gösterir.

Eğer x düğümü bir çocuğa sahip değilse $x.left-child = \text{NIL}$ ve eğer x kendi ebeveyn düğümünün en sağındaki çocuğu ise $x.right-sibling = \text{NIL}$ dir.



Şekil 10.9 *T* ikili ağacının temsili. Her x düğümü $x.p$ (üst), $x.left$ (sol alt) ve $x.right$ (sağ alt) niteliklerine sahiptir. Ağaçta *key* nitelikler görüntülenmemiştir.



Şekil 10.10 *T* ikili ağacının sol-çocuk, sağ-kardeş temsili. Her x düğümü $x.p$ (üst), $x.left-child$ (alt sol) ve $x.right-sibling$ (alt sağ) niteliklerine sahiptir. Ağaçta *key* nitelikler görüntülenmemiştir.

Diğer ağaç gösterilimleri

Bazen köklü ağaçları farklı yollarla temsil ederiz. Bölüm 6 da tek bir diziye ek olarak bellek yığınının son düğümünün indisi ile oluşturulan tam bir ikili ağaca dayalı bellek yığınının göstermiştik. Bölüm 21 de, sadece kök üzerinden geçilen ve böylece sadece ebeveyni gösteren işaretçileri içeren, çocukları gösteren işaretçilerin olmadığı ağaçlar verilmiştir. Diğer birçok farklı tasarım da olasıdır. Hangi tasarımın en iyi olduğu uygulamaya bağlıdır.

Alıştırmalar

10.4-1

Aşağıda verilen listedeki 6 indisli nesneyi kök kabul eden ikili ağacı çiziniz.

<i>indis</i>	<i>key</i>	<i>left</i>	<i>right</i>
1	12	7	3
2	15	8	NIL
3	4	10	NIL
4	10	5	9
5	2	NIL	NIL
6	18	1	4
7	7	NIL	NIL
8	14	6	2
9	21	NIL	NIL
10	5	NIL	NIL

10.4-2

n -düğümlü bir ikili ağacın düğümlerinin anahtar nitelik değerlerini ekranda görüntüleyen $O(n)$ süreli özyinelemeli bir prosedür yazınız.

10.4-3

n -düğümlü bir ikili ağacın düğümlerinin anahtar nitelik değerlerini ekranda görüntüleyen $O(n)$ süreli özyinelemeli olmayan bir prosedür yazınız. Bir yığın yapısını yardımcı veri yapısı olarak kullanınız.

10.4-4

sol-çocuk ve *sağ-kardeş* gösterimi ile temsil edilen n -düğümlü bir ikili ağacın düğümlerinin anahtar nitelik değerlerini ekranda görüntüleyen $O(n)$ süreli bir prosedür yazınız.

10.4-5 ★

n -düğümlü bir ikili ağacın düğümlerinin anahtar nitelik değerlerini ekranda görüntüleyen $O(n)$ süreli özyinelemeli olmayan bir prosedür yazınız. Ağacın kendi-

si haricinde sabit miktarda bir bellek alanından daha fazlasını kullanmayınız ve geçici olarak dahi olsa ağacı değiştirmeyiniz.

10.4-6 *

sol-çocuk ve *sağ-kardeş* gösterimi ile temsil edilen bir ikili ağacın düğümlerinde üç işaretçi kullanılır: *left-child*, *right-sibling* ve *parent*. Herhangi bir düğümden ebeveyn düğüme sabit sürede ulaşılabilir ve ebeveyn düğümün tüm çocuklarına doğrusal süre içinde erişilebilir. Her düğümden sadece iki işaretçi ve bir ikili değer kullanarak bir düğümün ebeveynine veya tüm çocuklarına doğrusal süre içinde erişilebileceğini gösteriniz.

Problemler

10-1 Liste yapıları arasında karşılaştırma

Aşağıda verilen tabloda listelenen dört bağlı liste türü için her bir dinamik küme işleminin en kötü durumda asimptotik çalışma süreleri ne olur?

	sırasız, tek başına bağlı liste	sıralı, tek başına bağlı liste	sırasız, çift yönlü bağlı liste	sıralı, çift yönlü bağlı liste
SEARCH (L, k)				
INSERT (L, x)				
DELETE (L, x)				
SUCCESSOR (L, x)				
PREDECESSOR (L, x)				
MINIMUM (L)				
MAXIMUM (L)				

10-2 Bağlı listeleri kullanarak birleştirilebilir bellek yığınları

Bir *birleştirilebilir bellek yığını* şu işlemleri destekler: MAKE-HEAP (boş bir bellek yığını yaratır), INSERT, MINIMUM, EXTRACT-MIN.¹ Aşağıda belirtilen her bir durum için birleştirilebilir bellek yığınlarının nasıl oluşturulabileceğini gösteriniz. Her bir işlemin mümkün olduğu kadar verimli olmasına çalışınız. Üzerinde uygulanan dinamik kümelerin boyutu bakımından her bir işlemin çalışma süresini analiz ediniz.

- a. Listeler sıralıdır.
- b. Listeler sırasızdır.
- c. Listeler sırasızdır ve birleştirilecek olan dinamik kümeler ayrıktır.

10-3 Kompakt sıralı bir listede arama

Alıştırma 10.3-4 de n -elemanlı bir listenin elemanlarının bir dizinin ilk n konumunda yer alacak biçimde kompakt olarak nasıl oluşturulabileceği sorulmuştu. Tüm anahtar değerlerinin birbirinden farklı olduğunu ve aynı zamanda kompakt listenin sıralı olduğunu, yani $i = 1, 2, \dots, n$ için $next[i] \neq NIL$ olacak biçimde $key[i] < key[next[i]]$ olduğunu varsayalım. Bununla birlikte, L değişkeni listenin ilk elemanının indisini içersin. Bu varsayımlar altında, $O(\sqrt{n})$ beklenen zamanda listede arama yapmak için aşağıda verilen rassal algoritmayı kullanabileceğimizi gösteriniz.

COMPACT-LIST-SEARCH (L, n, k)

```

1   $i = L$ 
2  while  $i \neq NIL$  ve  $key[i] < k$ 
3       $j = \text{RANDOM}(1, n)$ 
4      if  $key[i] < key[j]$  ve  $key[j] \leq k$ 
5           $i = j$ 
6      if  $key[i] == k$ 
7          return  $i$ 
8       $i = next[i]$ 
9  if  $i == NIL$  veya  $key[i] > k$ 
10     return  $NIL$ 
11 else return  $i$ 
```

Prosedürdeki 3–7 nolu satırları gözardı edersek i indisinin listenin her bir konumunu işaret ettiği sıralı bir bağlı listede arama işlemi için sıradan bir algoritma

¹Birleştirilebilir bellek yığını, MINIMUM ve EXTRACT-MIN i destekleyecek biçimde tanımladığımızdan *birleştirilebilir minimum bellek yığını* olarak adlandırabiliriz. Alternatif olarak, eğer MAXIMUM ve EXTRACT-MAX i desteklenirse *birleştirilebilir maksimum bellek yığını* elde edilir.

elde ederiz. Arama işlemi, i indisi listenin sonuna ulaştığında veya $key[i] \geq k$ olduğunda sona erer. İkinci durumda, eğer $key[i] = k$ ise aranan anahtar k değeri ile bulunmuştur. Ancak $key[i] > k$ ise k değeri ile bir anahtara ulaşmamız asla mümkün değildir, bu nedenle arama sonlandırılır.

Satır 3–7 arasında rassal seçilen j indisli konuma atlanmaya çalışılır. Bu tarz bir atlamanın sağladığı fayda şudur. Eğer $key[j]$ değeri, $key[i]$ değerinden daha büyükse ve k değerinden daha büyük değilse; bu durumda, j listede i indisinin sıradan aramada ulaşmak zorunda olduğu konumu işaretler. Liste kompakt olduğundan, j değerinin 1 ile n arasındaki herhangi bir seçimi bazı nesnelerin serbest listeden ziyade liste üzerinde olduğunu bilmemizi sağlar.

COMPACT-LIST-SEARCH'ın direkt olarak performans analizini yapmak yerine ilişkili olarak iki ayrı döngü içeren COMPACT-LIST-SEARCH' algoritmasının analizini yapabiliriz. Bu algoritma ek olarak ilk döngüdeki iterasyon sayısının üst sınırını belirleyen t parametresine ihtiyaç duyar.

COMPACT-LIST-SEARCH' (L, n, k, t)

```

1    $i = L$ 
2   for  $q = 1$  to  $t$ 
3        $j = \text{RANDOM}(1, n)$ 
4       if  $key[i] < key[j]$  ve  $key[j] \leq k$ 
5            $i = j$ 
6           if  $key[i] == k$ 
7               return  $i$ 
8   while  $i \neq \text{NIL}$  ve  $key[i] < k$ 
9        $i = \text{next}[i]$ 
10  if  $i == \text{NIL}$  veya  $key[i] > k$ 
11      return  $\text{NIL}$ 
12  else return  $i$ 
```

COMPACT-LIST-SEARCH ve COMPACT-LIST-SEARCH' algoritmalarını karşılaştırmak için, $\text{RANDOM}(1, n)$ un çağrılarının her iki algoritma içinde aynı tam sayı dizisini geri döndürdüğünü varsayınız.

a. COMPACT-LIST-SEARCH (L, n, k) algoritmasında 2-8 satırları arasındaki **while** döngüsünün t iterasyonda tamamlanacağını varsayınız. COMPACT-LIST-SEARCH' (L, n, k, t) algoritmasının da aynı sonucu döndüreceğini ve bu algoritmadaki **for** ve **while** döngülerinin her ikisinin toplam iterasyon sayısının en az t olacağını gösteriniz.

COMPACT-LIST-SEARCH' (L, n, k, t) fonksiyon çağrısında, X_i değişkeni i indisli konumdan satır 2–7 arasında yer alan **for** döngüsünün t iterasyon boyunca çalışmasından sonra görülen k anahtar değeri boyunca bağlı liste içindeki uzaklığı tanımlayan (yani *next* işaretçilerinin zinciri boyunca) rassal bir değişken olsun.

- b.** COMPACT-LIST-SEARCH' (L, n, k, t) için beklenen çalışma zamanının $O(t + E[X_t])$ olacağını gösteriniz.
- c.** $E[X_t] \leq \sum_{r=1}^n (1 - r/n)^t$ olduğunu gösteriniz. (*İpucu:* (C.25) eşitliğini kullanınız).
- d.** $\sum_{r=0}^{n-1} r^t \leq n^{t+1}/(t+1)$ olduğunu gösteriniz.
- e.** $E[X_t] \leq n/(t+1)$ olduğunu kanıtlayınız.
- f.** COMPACT-LIST-SEARCH' (L, n, k, t) nin $O(\sqrt{n})$ sürede çalışacağını gösteriniz.
- g.** COMPACT-LIST-SEARCH ın $O(\sqrt{n})$ sürede çalışacağı sonucuna varınız.
- h.** COMPACT-LIST-SEARCH algoritmasında neden tüm anahtarların birbirinden farklı olduğunu varsayınız? Liste tekrarlı anahtar değerleri içerdiğinde rassal atlama işleminin asimptotik olarak yardımcı olmayacağını iddia ediniz.

Bölüm notları

Aho, Hopcroft, ve Ullman [6] ve Knuth [209] temel veri yapıları için mükemmel kaynaklardır. Diğer çoğu kaynak temel veri yapıları ile bir programlama dilindeki uygulamalarını birlikte içerir. Bu tip kaynaklara Goodrich ve Tamassia [147], Main [241], Shaffer [311] ve Weiss [352, 353, 354] birer örnektir. Gonnet [145] çoğu veri yapısı işlemlerinin performansı üzerinde deneysel veri sağlar.

Bilgisayar bilimlerinde, sayısal bilgisayarların tanıtılmasından önce ilgili notasyonlar matematik ve işletme uygulamalarında zaten görüldüğünden yığın ve kuyruk yapılarının kökeni çok net değildir. Knuth [209], yığınların altprogram ile oluşturulması için 1947 yılında A. M. Turing'in çalışmasını referans verir.

Ayrıca işaretçi-tabanlı veri yapıları da anonim çalışmalar olarak görünmektedir. Knuth'a göre, işaretçiler tambur bellekli bilgisayarlarda kullanılmıştır. 1951 yılında G. M. Hopper tarafından geliştirilen A-1 dili ikili ağaçları cebirsel formüllerle sunmuştur. 1956 yılında A. Newell, J. C. Shaw ve H. A. Simon tarafından geliştirilen IPL-II diliyle işaretçilerin öneminin farkına varıldığını ve kullanımının desteklediği yine Knuth tarafından belirtilmiştir. Aynı isimlerin 1957 yılında geliştirdikleri IPL-III dili açık olarak yığın işlemlerini desteklemiştir.

Birçok uygulama, sadece INSERT, SEARCH ve DELETE sözlük işlemlerini destekleyen dinamik bir küme gerektirir. Örneğin, programlama dilini çeviren derleyici, içindeki elemanların anahtarlarının dildeki belirleyicilere tekavül eden, keyfi karakter katarları olan bir sembol tablosu tutar. Çırpı (hash) tablosu, sözlük uygulamaları için etkili bir veri yapısıdır. Çırpı tablosunda bir elemanı aramak, bir bağlı liste içinde eleman aramak kadar uzun sürse de (pratikte en kötü ihtimalle $\Theta(n)$ zamanda), çırpılama çok iyi işler. Mantıklı varsayımlar üzerinde, çırpı tablosunda ortalama bir eleman arama süresi $O(1)$ dir.

Çırpı tablosu daha basit sıradan dizi fikrini genelleştirir. Sıradan bir dizi içinde doğrudan adresleme, $O(1)$ zamanda dizi içinde keyfi bir pozisyonu inceleme kabiliyetimizi verimli bir şekilde kullanır. Kısım 11.1, doğrudan adreslemeyi daha detaylı anlatır. Her muhtemel anahtar için bir pozisyonu olan diziye yer açabildiğimizde doğrudan adreslemeden yararlanabiliriz.

Anlık saklanan anahtar sayısı, tüm muhtemel anahtar sayısından göreceli olarak küçük olduğunda, çırpı tabloları doğrudan adreslemeye göre daha iyi bir alternatif haline gelir. Çünkü çırpı tablosu, boyutu anlık saklanan anahtar sayısıyla doğru orantılı olan bir dizi kullanır. Anahtarı doğrudan dizi indisi olarak kullanmak yerine, dizi indisi anahtardan hesaplanır. Kısım 11.2, birden fazla dizi anahtarın aynı dizi indisiyle eşleştiği “çarpmaları” önleme yolu olarak, “zincirleme” üzerine odaklanan genel fikirleri sunar. Kısım 11.3, çırpı fonksiyonlarını kullanarak anahtarlardan nasıl dizi indisi hesaplayabildiğimizi anlatır. Genel konu üzerinde birkaç çeşitlemeyi analiz eder ve sunarız. Kısım 11.4, çarpışmalarla baş etmenin diğer yolu olan açık adreslemeye bakar. Sonuç olarak, çırpı olayı gayet etkili ve verimli bir tekniktir. Temel sözlük işlemleri ortalama $O(1)$ zaman alır. Kısım 11.5, anahtar kümesi statik olarak saklandığında, (yani anahtar kümesi bir kere saklanıp bir daha değişmediğinde) *en kötü-durumda* ‘kusursuz çırpı fonksiyonunun (çırpılamanın)’ nasıl $O(1)$ zamanda arama yaptığını açıklar.

11.1 Doğrudan-adres tabloları

Doğrudan adresleme, anahtar evreni mantıklı derecede küçük olduğunda iyi çalışan basit bir yöntemdir. Varsayın ki, bir uygulama, her bir elemanın, m 'nin çok büyük olmadığı $U = \{0, 1, \dots, m-1\}$ evreninden çekilen bir anahtara sahip olduğu dinamik bir kümeye ihtiyaç duysun. Hiçbir iki elemanın aynı anahtara sahip olmadığını varsayalım.

Dinamik kümeyi temsil etmek için, her bir pozisyonun U evreninde bir anahtara denk geldiği, $T[0 \dots m-1]$ şeklinde gösterilen bir **doğrudan-adres tablosu** kullanırız. Şekil 11.1, yaklaşımı gösterir. Pozisyon k , k anahtarlı küme içinde bir elemanı gösterir. Eğer küme, k anahtarı içeren bir eleman barındırmıyorsa, o zaman $T[k] = \text{NIL}$ dir.

Sözlük işlemleri uygulamak için gereksizdir.

DIRECT-ADDRESS-SEARCH (T, k)

1 **return** $T[k]$

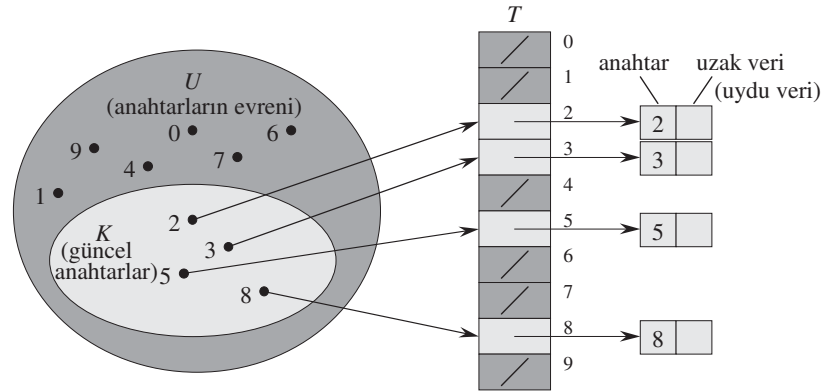
DIRECT-ADDRESS-INSERT (T, x)

1 $T[x, \text{key}] = x$

DIRECT-ADDRESS-DELETE (T, x)

1 $T[x, \text{key}] = \text{NIL}$

Her bir işlem $O(1)$ zaman alır.



Şekil 11.1 Doğrudan-adres tablosu T ile nasıl dinamik bir küme uygulandığı. $U = (0, 1, \dots, 9)$ evrenindeki her bir küme tablodaki bir indise denk gelir. Güncel anahtarların kümesi $K = \{2, 3, 5, 8\}$, elemanların işaretçilerini içeren tablodaki pozisyonlarını belirler. Gölgeleştirilmiş diğer pozisyonlar boştur.

Bazı uygulamalar için, doğrudan-adres tablosunun kendisi dinamik küme içindeki elemanları tutar. Yani, doğrudan-adres tablosuna, harici nesnenin uydu verisi ve elemanın anahtarını saklamak yerine, nesneye tablodaki pozisyonundan bir işaretçi sayesinde, nesnenin kendisini pozisyonun içinde saklayabiliriz. Böylelikle yerden tasarruf ederiz. Boş bir pozisyonu göstermek için nesnenin içinde özel bir anahtar kullanabiliriz. Dahası, nesnenin anahtarını saklamak çoğu zaman gereksizdir. Çünkü tablodaki nesnenin indisine sahipsek, onun anahtarına da sahibizdir. Çünkü anahtarlar saklanmazsa bir şekilde pozisyonun boş olup olmadığını söylemek için bir yol bulmalıyız.

Alıştırılmalar

11.1-1

Varsayın ki dinamik küme S , boyutu m olan T tablosuyla temsil ediliyor. S nin en büyük elemanını bulan bir prosedür tanımlayın. Prosedürünüzün en kötü-durum-performansı ne olur?

11.1-2

Bit vektörü, basitçe bitlerden (0 lar ve 1 lerden) oluşan bir dizidir. m boyutlu bir bit vektör, m işaretçili bir diziden daha az yer kaplar. Uydu verisi olmayan, farklı elemanlardan oluşan dinamik bir kümeyi temsil etmek için bit vektörün nasıl kullanıldığını açıklayın. Sözlük işlemleri $O(1)$ zamanda yapılmalıdır.

11.1-3

Saklanan elemanların anahtarlarının farklı olmak zorunda olmadığı ve elemanların uydu verisi alabileceği doğrudan-adres tablosunun nasıl uygulandığını önerin. Tüm üç sözlük işlemleri (INSERT, SEARCH ve DELETE) $O(1)$ zamanda yapılmalıdır. (Unutmayın ki DELETE silinecek nesneyi gösteren işaretçiyi girdi olarak alır, anahtarı değil.)

11.1-4 ★

Çok büyük bir dizi üzerinde doğrudan adresleme kullanarak bir sözlük oluşturmak istiyoruz. Başlangıçta, dizi girdileri çöp içerebilir ve diziyeye başlangıç değerleri vermek boyutundan dolayı pratik olmaz. Çok büyük bir dizi üzerinde DOĞRUDAN-ADRES uygulayan bir şema tarif edin. Saklanan her nesne $O(1)$ yer kullanmalıdır; SEARCH, INSERT ve DELETE işlemleri $O(1)$ zaman almalıdır; ve veri yapısını başlatma $O(1)$ zaman almalıdır. (*İpucu*: Fazladan bir dizi kullanın. Boyutu sözlükte saklanan anahtarların sayısı kadar olan bir yığın gibi davranın. Böylece çok büyük diziyeye verilen girdinin geçerli olup olmadığını belirlemeye yardım etsin).

11.2 Çarpı tabloları

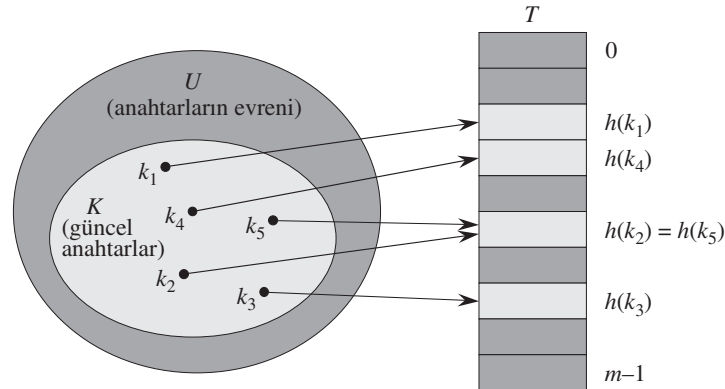
Doğrudan adreslemenin dezavantajı açıktır: eğer U evreni çok büyük olursa, $|U|$ boyutunda bir T tablosu saklamak, tipik bilgisayardaki mevcut hafıza ile uygulanabilir olmaz, hatta imkansız olur. Dahası, saklanan anahtar kümesi K , U ya göre oldukça küçük olabilir, böylece T için ayrılan yerin çoğu boşa harcanır.

Sözlükte saklanan anahtarların kümesi K , tüm muhtemel anahtarların evreni U dan göreceli olarak küçük olduğunda, çarpı tablosu doğrudan-adres tablosuna göre daha az yere ihtiyaç duyar. Özellikle, çarpı tablosunun, bir eleman aramak için $O(1)$ zaman kullanma faydasını sürdürürken, yer kaplama ihtiyacını da $\Theta(|K|)$ ya düşürebiliriz. Önemli nokta şu ki, bu sınır, *ortalama-durum* zamanı için geçerlidir, fakat doğrudan adreslemede *en kötü-durum* zamanı için geçerlidir.

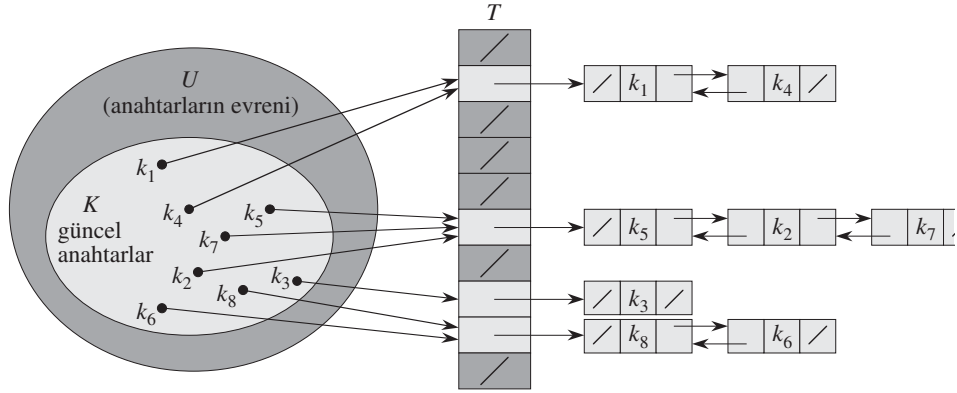
Doğrudan adresleme ile k pozisyonunda k anahtarlı bir eleman saklanır. Çarpılama ile, bu eleman $h(k)$ pozisyonunda saklanır. Yani, k anahtarından pozisyon hesaplamak için h **çarpı fonksiyonunu** kullanırız. Burada h , **çarpı tablosu** $T[0..m-1]$ pozisyonlarının içindeki anahtarlar evreni U ya eşleme yapar.

$$h: U \rightarrow \{0, 1, \dots, m-1\}$$

çarpı tablosunun boyutu m , tipik olarak $|U|$ dan çok daha az olduğunda. K anahtarlı bir eleman $h(k)$ pozisyonuna **çarpılar**, diye söyleriz. Ayrıca, $h(k)$ nın k anahtarının **çarpı değeri** olduğunu da söyleriz. Şekil 11.2, genel fikri görselleştirir. Çarpı fonksiyonu dizi indisi aralığını azaltır ve böylece dizinin boyutunu da azaltır. Evrenin boyutunun $|U|$ yerine, dizi m boyutunda olabilir.



Şekil 11.2 h çarpı fonksiyonu kullanarak anahtarların çarpı tablosu pozisyonlarına eşlenmesi. k_2 ve k_5 aynı pozisyonlarda olduğu için çarpışır.



Şekil 11.3 Zincirleme ile çarpışma çözümü. Her bir çarpı tablosu, pozisyonu $T[j]$, çarpı değeri j olan tüm anahtarların bağlı listesini içerir. Örneğin, $h(k_1) = h(k_4)$ ve $h(k_5) = h(k_7) = h(k_2)$. Bağlı liste tekli veya çiftli olabilir. Biz çiftli liste olarak gösteriyoruz. Çünkü silme işlemi daha hızlı.

Bir önemli nokta şu ki: iki anahtar aynı pozisyona çarpılayabilir. Bu durumu **çarpışma** olarak adlandırırız. Neyse ki, çarpışmalardan doğacak çatışmayı çözmek için etkili tekniklerimiz var.

Tabii ki, en ideal çözüm çarpışmalardan tamamen kaçınmak olacaktır. Bu amaca ulaşmak için uygun çarpı fonksiyonu h yi seçmeyi deneyebiliriz. Bir fikir, h yi “rassal” göstermektir. Böylece çarpışmadan kaçınılır veya en azından çarpışma sayısı en aza indirilir. Çarpı kelimesi, rassal karışım görüntülerinin ortaya çıkması demektir ve bu yaklaşımın ruhunu yakalar. (Tabii ki çarpı fonksiyonu h belirleyici olmalıdır ki verilen girdi k , her zaman aynı çıktı $h(k)$ yı üretebilsin.) Yine de, $|U| > m$ olduğu için, en azından aynı çarpı değerine sahip 2 anahtar olmasıdır. Bu yüzden tamamıyla çarpışmadan kaçınmak imkansızdır. Bu nedenle, iyi tasarlanmış “rassal” arayan bir çarpı fonksiyonu çarpışma sayısını en aza indirirken, hala çarpışmaları çözmek için bir yönteme ihtiyaç duyarız.

Bu bölümün kalan kısmı, en basit çarpışma çözücü yöntemini, zincirlemeyi, sunar. Kısım 11.4, açık adresleme diye adlandırılan çarpışma önleyici alternatif yöntemi gösterir.

Çarpışmanın zincirleme tarafından yeniden çözülmesi

Zincirlemede, aynı pozisyona çarpılayan bütün elemanları bir bağlı listeye yerleştiririz, Şekil 11.3 de gösterildiği gibi. j pozisyonu, j ye çarpılayan tüm kayıtlı elemanların listesinin başını gösteren işaretçi içerir. Eğer böyle elemanlar yoksa, j pozisyonu NIL içerir.

T çırpı tablosu üzerindeki sözlük operasyonları, zincirlemeyle çarpışmalar çözüldüğünde uygulaması kolay hale gelir:

CHAINED-HASH-INSERTION(T, x)

1 x i listenin sonuna ekle $T[h(x, key)]$

CHAINED-HASH-SEARCH(T, k)

1 listede k anahtarlı bir eleman arama $T[h(k)]$

CHAINED-HASH-DELETION(T, x)

1 listeden x i silme $T[h(x, key)]$

Araya sokmadaki en kötü-durum çalışma zamanı $O(1)$ dir. Araya sokma prosedürü kısmen hızlıdır. Çünkü eklenecek eleman x in listede zaten var olmadığı varsayılır. Gerekirse, araya sokmadan önce bu varsayımı, x anahtarlı bir elemanı aratarak test edebiliriz. Aramalarda, en kötü-durum çalışma süresi listenin büyüklüğüyle doğru orantılıdır. Aşağıda bu işlemi daha detaylı inceleyeceğiz. Eğer liste çift bağlı ise bir elemanı $O(1)$ zamanda silebiliriz, Şekil 11.3 de gösterildiği gibi. (Dikkat edin ki CHAINED-HASH-DELETION, girdi olarak eleman x i alır. Onun anahtarı k yı değil. Bu yüzden ilk önce x i aramamıza gerek kalmaz. Çırpı tablosu silmeyi desteklerse, o zaman onun bağlı listesi çift olmak zorundadır ki elemanı kolayca silebilelim. Eğer liste yalnızca tek yönlü ise, bir elemanı silmek için önce listeden $T[h(x, key)]$ x i bulmalıyız ki x in atasının next özelliğini güncelleyebilelim. Tek yönlü listelerle silme ve arama aynı asimptotik çalışma süresine sahiptirler).

Zincirleme ile çırpılama analizi

Zincirleme ile çırpılama nasıl iyi çalışır? Özellikle, verilen anahtarlı bir elemanı aramak ne kadar sürer?

n tane eleman saklayan, m tane pozisyonlu olarak verilen çırpı tablosu T için, n/m olarak T için bir **yükleme faktörü**, α , tanımlarız. Yani, Zincirde saklanan ortalama eleman sayısı. Analizimiz, 1 den küçük, 1 e eşit veya 1 den büyük olabilecek olan α cinsindendir.

Zincirleme ile çırpılamanın en kötü-durum davranışı çok kötüdür. n boyutlu bir liste yaratarak tüm n anahtarları aynı pozisyona çırpılar. Bu yüzden bir eleman aramak için en kötü-durum zamanı, $\Theta(n)$ ve çırpı fonksiyonunu hesaplamak için geçen süredir. Tüm elemanları saklamak için bir bağlı liste oluşturmaktan daha iyi değildir. Açıkça, çırpı tablolarını, onların en kötü-durumları için kullanmayız. (Kısım 11.5 de gösterilen kusursuz çırpılama, anahtar kümesi statik olduğunda, iyi bir en kötü-durum performansı sağlar.)

Çırpılamanın ortalama-durum performansı, ortalama olarak m tane pozisyon arasında saklanan anahtar kümesine çırpı fonksiyonu h nin nasıl iyi dağıttığına

bağlıdır. Kısım 11.3 bu konuları tartışır, fakat şimdilik biz diğer elemanın çırpılamasından bağımsız olarak verilen, herhangi bir elemanın m pozisyonundan birine eşit şekilde çırpıldığını varsayacağız. Bu varsayımı **basit düzgün çırpılama** olarak adlandırırız.

$j = 0, 1, \dots, m - 1$ için, $T[j]$ listesinin boyutunu n_j olarak gösterelim ki

$$n = n_0 + n_1 + n_2 + \dots + n_{m-1} \quad (11.1)$$

ve n_j nin beklenen değeri $E[n_j] = \alpha = n/m$ olsun.

Çırpı değeri $h(k)$ yı hesaplamak için $O(1)$ zamanın yeterli olduğunu varsayıyoruz ki k anahtarlı bir elemanı aramak için gereken süre $T[h(k)]$ listesinin $nh(k)$ boyutuna doğrusal olarak bağlı olsun. Çırpı fonksiyonunu hesaplamak için gereken zamana $O(1)$ koyarak ve $h(k)$ pozisyonuna erişmek için, arama algoritması tarafından incelenen elemanların beklenen sayısını, yani $T[h(k)]$ listesindeki k anahtarına sahip olup olmadığı algoritma tarafından kontrol edilen eleman sayısını düşünelim. İki durum düşüneceğiz. Öncelikle, arama başarısızdır. Listede k anahtarlı hiçbir eleman yok. İkinci durumda, arama başarılı bir şekilde k anahtarlı elemanı bulur.

Teorem 11.1

Çarpışmaların zincirleme ile çözüldüğü bir çırpı tablosunda, başarısız bir arama, basit düzgün çırpılama varsayımı altında ortalama $\Theta(1+\alpha)$ zaman alır.

Kanıt Basit düzgün çırpılama varsayımı altında, tabloda önceden kayıtlı olmayan herhangi bir k anahtarı, m pozisyonundan herhangi birine çırpılama eğilimindedir. k anahtarını başarısız bir şekilde aramak için beklenen süre $T[h(k)]$, listesinin sonunu aramak için beklenen süredir. Beklenen boyut $E[n_{h(k)}] = \alpha$ dir. Bu yüzden, başarısız bir aramada incelenen elemanların beklenen sayısı α dir. Toplam gereken zaman $\Theta(1 + \alpha)$ dir. ($h(k)$ yı hesaplamak için gereken süre dahil) ■

Başarılı bir arama durumu biraz farklıdır. Çünkü her liste aramak için eşit benzerlikte değildir. Bunun yerine, bir listenin aranma olasılığı, onun içerdiği eleman sayısı ile doğru orantılıdır. Yine de, beklenen arama zamanı hala $\Theta(1 + \alpha)$ civarındadır.

Teorem 11.2

Çarpışmaların zincirlemeyle çözüldüğü bir çırpı tablosunda, başarılı bir arama, basit düzgün çırpılama varsayımıyla ortalama $\Theta(1+\alpha)$ zaman alır.

Kanıt Aranacak elemanın, tabloda bulunan herhangi bir elemanla eşit derecede benzer olduğunu varsayalım. Bir x elemanı için başarılı bir arama süresince incelenen eleman sayısı, c nin listesindeki x ten önce görülen eleman sayısından 1

fazladır. Yeni eklenen elemanlar listenin önüne yerleştirildiğinden, listede x den önceki elemanlar, x eklendikten sonra eklenir. Beklenen incelenen eleman sayısını bulmak için, tablodaki n tane x elemanı üzerindeki listeye x eklendikten sonra x in listesine eklenen eleman sayısının 1 fazlasının ortalamasını alırız. x_i nin tabloya eklenen i nci eleman olduğunu varsayalım. $i = 1, 2, 3, \dots, n$ için, $k_i = x$ olsun. K_i ve K_j anahtarları için rassal gösterge değişkeni $X_{ij} = I\{h(k_i) = h(k_j)\}$ yi ifade eder. Basit düzgün çırpılama varsayımı altında, $\Pr\{h(k_i) = h(k_j)\} = 1/m$ ve Lemma 5.1 ile $E[X_{ij}] = 1/m$ olur. Bu yüzden, başarılı bir aramada beklenen incelenen eleman sayısı

$$\begin{aligned}
& E \left[\frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n X_{ij} \right) \right] \\
&= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n E[X_{ij}] \right) \quad (\text{beklentinin doğrusallığından}) \\
&= \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n \frac{1}{m} \right) \\
&= 1 + \frac{1}{nm} \sum_{i=1}^n (n - i) \\
&= 1 + \frac{1}{nm} \left(\sum_{i=1}^n n - \sum_{i=1}^n i \right) \\
&= 1 + \frac{1}{nm} \left(n^2 - \frac{n(n+1)}{2} \right) \quad (\text{denklem (A.1) den}) \\
&= 1 + \frac{n-1}{2m} \\
&= 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}
\end{aligned}$$

dir. Bu nedenle, başarılı bir arama için gereken toplam zaman, (çırpı fonksiyonunu hesaplamak için gereken süre de dahil) $\Theta(2 + \alpha/2 - \alpha/2n) = \Theta(1 + \alpha)$ dır. ■

Bu analiz ne anlama gelir? Çırpı tablosu hücreleri sayısı en azından tabloda ki eleman sayısı ile doğru orantılı ise, $n = O(m)$ dir ve sonuç olarak, $a = n/m = O(m)/m = O(1)$ dir. Bu yüzden, arama ortalama sabit zaman alır. Liste çift bağlı liste olduğunda, en kötü-durumda araya sokma ve silme $O(1)$ zaman aldığından, tüm sözlük işlemlerini ortalama $O(1)$ zamanda destekleyebiliriz.

Alıştırmalar

11.2-1

m boyutunda T dizisinin içinde n tane farklı anahtar çırplamak için h çırpı fonksiyonunu kullandığınızı varsayın. Tek biçim basit çırplama varsayımıyla, beklenen çarpışma sayısı nedir? Daha açık ifadeyle, $\{\{k, l\} : k \neq l \text{ ve } h(k) = h(l)\}$ kümesinin beklenen eleman sayısı nedir?

11.2-2

Çarpışmaların zincirlemeyle çözüldüğü bir çırpı tablosuna 5, 28, 19, 15, 20, 33, 12, 17, 10 anahtarları eklendiğinde ne olduğunu gösteriniz. Tablonun 9 hücresi olduğunu ve çırpı fonksiyonunun $h(k) = k \bmod 9$ olduğunu düşünün.

11.2-3

Profesör Marley, her listeyi sıralı düzende tutacak şekilde zincirleme mekanizmasını değiştirerek iyi bir performans alacağını hipotezler. Profesörün değişikliği, başarılı aramaların, başarısız aramaların, araya sokmaların ve silmelerin çalışma sürelerini nasıl etkiler?

11.2-4

Kullanılmayan tüm hücreleri serbest listeye bağlayarak çırpı tablosunun kendisi içindeki elemanların belleğe nasıl yerleştirilip nasıl çıkarılacağını öneriniz. Bir hücrenin bir bayrak, bir eleman bir gösterici ya da iki gösterici taşıdığını varsayın. Tüm sözlük ve serbest liste işlemleri $O(1)$ beklenen zamanda çalışmalıdır. Serbest liste çift bağlı mı yoksa tek bağlı serbest liste mi olmalıdır?

11.2-5

m boyutunda bir çırpı tablosunda n anahtarlı bir küme sakladığınızı varsayalım. $|U| > nm$ olduğu U evreninden anahtarlar çekilirse, U nun aynı hücreye çırplayan anahtarlardan oluşan n boyutunda bir alt kümesi olduğunu ve bu yüzden zincirlemeyle çırplama için en kötü-durum çalışma zamanının $\Theta(n)$ olduğunu gösteriniz.

11.2-6

m boyutunda bir çırpı tablosuna, çarpışmaların zincirlemeyle çözüldüğü, n anahtarlı bir küme sakladığınızı düşünelim. Her zincirin boyutunu ve en uzun zincirin L boyunda olduğunu biliyoruz. Çırpı tablosundaki anahtarlar arasından rassal olarak düzgün şekilde bir anahtar seçen ve onu $O(L \cdot (1 + 1/\alpha))$ beklenen zamanda döndüren bir prosedür yazınız.

11.3 Çırpı fonksiyonları

Bu bölümde, iyi çırpı fonksiyonu tasarımına yönelik bazı konular tartışırız ve onların yaratımı için 3 şema sunarız. Bu şemaların ikisi, çarpma ve bölmeyle çırpılama, doğası gereği sezgiseldir, diğer yandan üçüncü şema, evrensel çırpılama, muhtemelen iyi performansı sağlamak için randomizasyon kullanır.

İyi çırpı fonksiyonunu ne sağlar?

İyi bir çırpı fonksiyonu basit düzgün çırpılama özelliğini doğrulamalıdır. m tane hücreden herhangi birine çırpılanacak her bir anahtar, başka bir anahtarın oraya çırpılmasından bağımsız olarak, eşit şekilde benzerdir. Maalesef, bu şartı kontrol etme şansımız yoktur. Çünkü seçilen anahtarların dağılım olasılığını çok nadiren biliyoruz. Dahası, anahtarlar bağımsız olarak seçilmeyebilir.

Zaman zaman, dağılmayı bilmiyoruz. Örneğin, anahtarların (k) bağımsız rasal reel sayılar olduğunu ve $0 \leq k < 1$ arasında düzgün şekilde dağıldığını biliyorsak, çırpı fonksiyonu, $h(k) = [km]$ basit düzgün çırpılama koşulunu sağlar.

Pratikte, iyi performans gösteren çırpı fonksiyonu yaratmak için genelde sezgisel teknikler kullanırız. Anahtarların dağılımı hakkında niteleyici bilgi bu tasarımda yararlı olabilir. Örneğin, derleyicinin sembol tablosunu düşünelim. Yakın ilişkili semboller, pt ve pts gibi, genelde aynı programda gerçekleşirler. İyi bir çırpı fonksiyonu, bu gibi değişkenlerin aynı hücreye çırpılama şansını en aza indirir.

İyi bir yaklaşım, veride yer alabilecek herhangi bir kalıptan bağımsız çırpı değerini tüderir. Örneğin, (Kısım 11.3.1 de tartışılan) bölme yöntemi çırpı değerini, anahtar belli bir asal sayıya bölündüğünde elde edilen kalan olarak hesaplar. Bu yöntem, anahtarların dağılım kalıplarından herhangi biriyle alakasız bir asal sayı seçtiği varsayılarak sıklıkla iyi sonuçlar verir.

Sonuç olarak, bazı çırpı fonksiyon uygulamaları, basit düzgün çırpılamanın sağladığından daha güçlü özelliklere ihtiyaç duyabilir. Örneğin, çırpı değerlerinin birbirinden uzak olmasını sağlayacak manada birbirine yakın olan anahtarlar istenebiliriz (Bu özellik Kısım 11.4 de anlatılan doğrusal araştırma kullandığımızda özellikle istenilebilir). Evrensel çırpılama, Kısım 11.3.3 de tartışılan, genellikle istenilen özellikleri sağlar.

Anahtarları doğal sayılar olarak yorumlamak

Çoğu çırpı fonksiyonu, anahtarlar evreninin doğal sayılar kümesi olduğunu varsayar. Bu yüzden, anahtarlar doğal sayılar olmazsa, onları doğal sayılar gibi yorumlamanın yolunu buluruz. Örneğin, bir karakteri uygun taban yazımında bir tam sayı olarak yorumlayabiliriz. Bu yüzden, pt belirtecini tam sayı çifti olarak (112, 116) yorumlayabiliriz. Çünkü ASCII karakter kümesinde $p = 112$ ve $t = 116$. O zaman taban-128 tam sayısı olarak ifade edilen pt $(112 \cdot 128) + 116 = 14452$ olur. Verilen uygulamanın kavramında, bu tip bazı yöntemleri, genellikle her bir anahtar oldukça büyük doğal sayı gibi yorumlamak için tasarlarız. Konunun devamında, anahtarların doğal sayı olduğunu varsayalım.

11.3.1 Bölme yöntemi

Çırpı fonksiyonu yaratmak için **bölme yönteminde**, bir k anahtarını, k nın m ye bölümünden kalanı alarak m hücreden birine eşleştiririz. Yani çırpı fonksiyonu,

$$h(k) = k \bmod m$$

olur.

Örneğin, çırpı tablosu $m = 12$ boyutundaysa ve anahtar $k = 100$ ise, $h(k) = 4$ olur. Yalnızca tek bir bölme işlemi gerektiğinden, bölmeyle çırpılama oldukça hızlıdır.

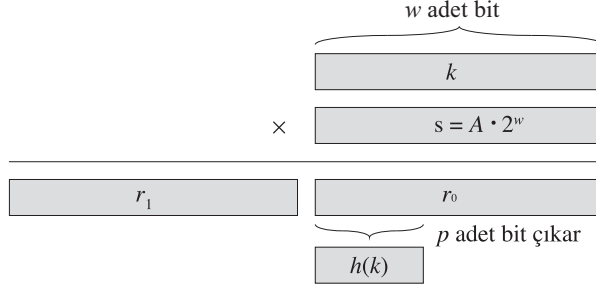
Bölme yöntemi kullanıldığında, m için bazı değerlerden kaçınılırız. Örneğin, m 2 nin bir kuvveti olmamalıdır. Çünkü $m = 2^p$ olursa, o zaman $h(k)$, k nın p -en düşük-sıra bitleri olur. Tüm en düşük-sıra p -bit kalıplarının eşit şekilde benzer olduğunu bilmezsek, anahtarın tüm bitlerine bağlı olan daha iyi bir çırpı fonksiyonu tasarlarız. Alistırma 11.3-3 ün göstermemizi istediği gibi, $k 2^p$ de yorumlanmış bir karakter katarı olduğunda $m = 2^p - 1$ i seçmek kötü bir seçim olabilir. Çünkü k nın karakterlerinin sırasını değiştirmek onun çırpı değerini değiştirmez.

2 nin tam kuvvetlerine çok fazla yakın olmayan bir asal sayı, m için iyi bir seçim olur. Örneğin, varsayın ki karakterlerin 8 bit olduğu kabaca $n = 2000$ karakter katarlarını tutmak için çarpışmaların zincirlemeyle çözüldüğü bir çırpı tablosu oluşturmak istiyoruz. Başarısız bir aramada 3 elemanın ortalamasını incelemeyi önemsemeyiz ve bu yüzden $m = 701$ boyutunda bir çırpı tablosu oluştururuz. $m = 701$ seçebiliriz. Çünkü m nin bu değeri $2000/3$ e yakın bir asal sayı ve 2 nin kuvvetine yakın değil. Her bir k anahtarına tam sayı gibi davranarak, çırpı fonksiyonumuzu

$$h(k) = k \bmod 701 \text{ yapabiliriz.}$$

11.3.2 Çarpma yöntemi

Çırpı fonksiyonlarını yaratmak için **çarpma yöntemi** iki aşamada gelişir. Birincisi, anahtar k yı, $0 < A < 1$ aralığında bir sabit A ile çarpar ve A nın kesirli kısmını



Şekil 11.4 Çırpılamanın çarpma yöntemi. k anahtarının w -bit temsili, w -bit $s = (A \cdot 2^w)$. değeriyle çarpılır. Sonucun düşük w -bitinin bit yarısının p en yüksek-sıra bitleri, istenilen $h(k)$ çırpı değerini oluşturur.

çıkartırız. Bu değeri m ile çarpar ve sonucun tabanını alırız. Kısaca çırpı fonksiyonu

$$h(k) = \lfloor m(kA \bmod 1) \rfloor.$$

olur ve burada $kA \bmod 1$, kA nın kesirli kısmını ifade eder. Yani $kA - \lfloor kA \rfloor$ dır.

Çarpma yönteminin avantajı şudur ki, m değeri kritik değildir. Onu tipik olarak 2 nin kuvveti seçeriz (bazı p tam sayıları için $m = 2^p$) Çünkü böylelikle gösterileceği gibi çoğu bilgisayarda bu fonksiyonu kolayca uygulayabiliriz. Varsayın ki makinenin kelime boyutu w bit olsun ve k bu tek kelimeye sığsın. A yı, $s/2^w$ nin kesiri olmasına kısıtlarız. s burada $0 < s < 2^w$ aralığındadır. Şekil 11.4 ü baz alarak, k yı önce w -bit tam sayıyla çarparsak $s = A \cdot 2^w$. Sonuç, r_1 in sonucun yüksek-sıra kelimesi, r_0 in ise sonucun düşük-sıra kelimesi olduğu $2w$ bit değer olan $r_1 2^w + r_0$ dır. İstenilen p -bit çırpı değeri r_0 in en önemli p bitlerinden oluşur.

Bu yöntem herhangi bir A sabit değeriyle çalışmasına rağmen, bazı değerlerde diğer değerlere göre daha iyi çalışır. Bu optimum seçim çırpılanan verinin özelliklerine bağlıdır. Knuth [211] önerir ki

$$A \approx (\sqrt{5} - 1)/2 = 0.6180339887 \dots \quad (11.2)$$

genellikle iyi çalışmaya adaydır.

Örnek olarak, $k = 123456$, $p = 14$, $m = 2^{14} = 16384$ ve $w = 32$ olduğunu düşünelim. Knuth'un önerisini uyarlayarak, a yı $(\sqrt{5} - 1)/2$ ye en yakın $s/2^{32}$ nin kesir kısmı olarak seçelim. Böylece $a = 2654435769/2$ olur³². Böylece, $k \cdot s = 327706022297664 = (76300 \cdot 2^{32}) + 17612864$ olur ve $r^1 = 76300$ ve $r_0 = 17612864$ olur. r_0 in 14 en önemli biti $h(k) = 67$ değerini sağlar.

★ 11.3.3 Evrensel çırpılama

Eğer dezavantajlı kötü bir seçim bazı sabit çırpı fonksiyonuyla çırpılacak anahtarları seçerse, bu kötü seçim, $\Theta(n)$ zamanda ortalama alımı destekleyecek, aynı hücreye çırpılayan n anahtar seçebilir. Herhangi bir sabit çırpı fonksiyonu bu tarz bir en kötü-durum davranışına maruz kalır. Durumu iyileştirmenin en verimli yolu güncel olarak saklanacak anahtarlardan tamamen bağımsız bir şekilde rastgele bir çırpı fonksiyonu seçmektir. Bu yaklaşım, **evrensel çırpılama**, hangi anahtarların seçildiği fark etmeden ortalama iyi sonuçlar verir.

Evrensel çırpılamada, uygulamanın başlangıcında dikkatlice tasarlanmış sınıf fonksiyonları arasından rassal bir çırpı fonksiyonu seçeriz. Hızlı sıralamada olduğu gibi, randomizasyon, hiçbir girdinin her zaman en kötü-durum davranışa sebep olmayacağını garanti eder. Çırpı fonksiyonunu rastgele seçtiğimiz için, algoritma herhangi bir girdi için ortalama performansı garanti ederek, aynı girdi de dahil, her bir uygulamada farklı davranış gösterecektir. Derleyicinin sembol tablosundaki örneğe dönecek olursak, programcının belirleyici seçiminin artık tutarlı olarak kötü çırpılama performansına sebep olmayacağını fark ederiz. Kötü performans yalnızca, derleyici, kötü çırpılayan bir belirteç kümesine sebep olan rassal bir çırpı fonksiyonu seçtiğinde olur, fakat bu durumun olma ihtimali düşüktür ve aynı boyuttaki her bir belirteç kümesi için aynıdır.

$\{0, 1, \dots, m - 1\}$ aralığındaki verilen anahtarlar evreni, U ya eşleyen çırpı fonksiyonlarının sonlu topluluğu $\mathcal{H}$ olsun. $k, l \in U$ her bir farklı anahtar çifti için, $h(k) = h(l)$ i sağlayan $h \in \mathcal{H}$ çırpı fonksiyonlarının sayısı en fazla $|\mathcal{H}|/m$ ise bu topluluk **evrensel**dir. Diğer bir deyişle, $\mathcal{H}$ den seçilen rastgele bir çırpı fonksiyonuyla, farklı k ve l anahtarları arasındaki çarpışma şansı, $h(k)$ ve $h(l)$ $\{0, 1, \dots, m - 1\}$ kümesinden bağımsız ve rastgele olarak seçildiğindeki çarpışmanın $1/m$ ye eşit olan şansından büyük değildir.

Aşağıdaki teorem çırpı fonksiyonlarının evrensel sınıfının iyi bir ortalama-durum performansı verdiğini gösterir. N_i nin $T[i]$ listesinin uzunluğunu temsil ettiğini unutmayalım.

Teorem 11.3

h çırpı fonksiyonunun, çırpı fonksiyonlarının evrensel topluluğundan rastgele seçildiğini ve çarpışma çözmek için zincirleme kullanılan m boyutunda T tablosuna n anahtar çırpılamak için kullanıldığını varsayalım. k anahtar tabloda yok ise, anahtar k nın çırpıladığı listenin beklenen uzunluğu $E[n_{h(k)}]$ en fazla yükleme faktörü $a = n / m$ kadar olur. k tabloda ise, k yı içeren listenin beklenen uzunluğu $E[n_{h(k)}]$ en fazla $1 + a$ olur.

Kanıt Buradaki beklentilerin çırpı fonksiyonu seçiminin üstünde olduğunu ve anahtarların dağılımı hakkında hiçbir varsayıma bağlı olmadığını fark ederiz. Her bir anahtar çifti k ve l için rassal gösterge değişkenini $X_{kl} = I\{h(k) = h(l)\}$

diye tanımlarız. Çırpı fonksiyonlarının evrensel topluluğunun tanımından, tek bir anahtar çifti en fazla $1/m$ olasılığı ile çarpışır, $\Pr\{h(k) = h(l)\} \leq 1/m$ olur. Lemma 5.1 den, bu yüzden $E[X_{kl}] \leq 1/m$ olur.

Daha sonra, her bir k anahtarı için Y_k rassal değişkenini k ile aynı hücreye çarpılayan k dan farklı anahtarların sayısı olarak tanımlarız. Böylece,

$$Y_k = \sum_{\substack{l \in T \\ l \neq k}} X_{kl} .$$

dır. Buradan

$$\begin{aligned} E[Y_k] &= E\left[\sum_{\substack{l \in T \\ l \neq k}} X_{kl}\right] \\ &= \sum_{\substack{l \in T \\ l \neq k}} E[X_{kl}] \quad (\text{beklentinin doğrusallığından}) \\ &\leq \sum_{\substack{l \in T \\ l \neq k}} \frac{1}{m} \end{aligned}$$

yı elde ederiz. Kanıtın geri kalanı k nın T tablosunda olmasına bağlıdır.

- $k \notin T$ ise, $n_{h(k)} = Y_k$ ve $|\{l : l \in T \text{ ve } l \neq k\}| = n$ dir. Bu yüzden, $E[n_{h(k)}] = E[Y_k] \leq n/m = \alpha$.
- $k \in T$ ise, k anahtarı $T[h(k)]$ listesinde yer aldığından ve Y_k , k yı içermediğinden, $n_{h(k)} = Y_k + 1$ ve $|\{l : l \in T \text{ ve } l \neq k\}| = n - 1$. Bu yüzden, $E[n_{h(k)}] = E[Y_k] + 1 \leq (n-1)/m + 1 = 1 + \alpha - 1/m < 1 + \alpha$ dır. ■

Aşağıdaki sonuç, evrensel çarpılamanın beklenen sonucu sağladığını söyler. Artık, en kötü durum çalışma zamanına zorlayan işlem sırasının seçmek olanaksızdır. Çalışma anında çırpı fonksiyonu seçimini zekice rastgele hale getirmekle, her işlem sırasının iyi ortalama-durum çalışma zamanıyla işleyeceğini garanti ederiz.

Sonuç 11.4

Evrensel çarpılamayı ve m hücreli başlangıçta boş tabloda zincirlemeyle çarpışma çözmeyi kullanarak, $\Theta(m)$ INSERT işlemi içeren, herhangi n sayıda SEARCH, INSERT ve DELETE işlemlerinin, $O(n)$ zaman alması beklenir.

Kanıt Araya sokma sayısı $O(m)$ olduğu için, $n = O(m)$ dir ve böylece $\alpha = O(1)$ dir. INSERT ve DELETE işlemleri sabit zaman alır ve Teorem 11.3 ile her bir SE-

ARCH için beklenen zaman $O(1)$ dir. Bu yüzden, beklentinin doğrusallığına dayanarak, n işlemin tüm sıralaması için beklenen zaman $O(n)$ dir. Herbir işlem $\Omega(1)$ zaman aldığından, sınır $\Theta(n)$ olur. ■

Çırpı fonksiyonlarının evrensel sınıfını tasarlama

Çırpı fonksiyonları evrensel sınıfını tasarlamak oldukça kolaydır. Küçük sayı teorisi bize bunu kanıtlamaya yardım edecek. Sayı teorisine aşına değilseniz öncelikle Bölüm 31 i inceleyebilirsiniz.

0 ve $(p - 1)$ in dahil olduğu aralıkta her bir muhtemel k anahtarının yer alması için yeterince büyük p asal sayısını seçerek başlarız. $\mathbb{Z}_p$ nin $\{0, 1, \dots, p - 1\}$ kümesini temsil ettiğini, $\mathbb{Z}_p^*$ nin ise $\{1, 2, \dots, p - 1\}$ kümesini temsil ettiğini düşünelim. p asal olduğundan, modulo p denklemlerini, Bölüm 31 de verilen yöntemlerle çözebiliriz. Anahtarlar evreninin boyutunun, çırpı tablosundaki hücre sayısından fazla olduğunu varsaydığımız için, $p > m$ diyebiliriz.

Modulo p azaltmalarını takip eden doğrusal bir dönüşüm kullanarak, $b \in \mathbb{Z}_p$ ve $a \in \mathbb{Z}_p^*$ için h_{ab} çırpı fonksiyonu tanımlarız ve o zaman modulo m :

$$h_{ab}(k) = ((ak + b) \bmod p) \bmod m \quad (11.3)$$

dir. Örneğin, $p = 17$ ve $m = 6$ ile, $h_{3,4}(8) = 5$ olur. Tüm bu çırpı fonksiyonlarının ailesi,

$$\mathcal{H}_{pm} = \{h_{ab} : a \in \mathbb{Z}_p^* \text{ ve } b \in \mathbb{Z}_p\} \quad (11.4)$$

dir. Her bir çırpı fonksiyonu h_{ab} , $\mathbb{Z}_p$ den $\mathbb{Z}_m$ ye çırpılar. Bu çırpı fonksiyonları sınıfı güzel bir özelliğe sahiptir. Çıktı aralığı boyutu m , rassal olabilir. Asal olmak zorunda değildir. Kısım 11.5 de kullanacağımız bir özellik de budur. b için p ve a için $p - 1$ seçeneğimiz olduğundan, $\mathcal{H}_{pm}$ topluluğu, $p(p - 1)$ çırpı fonksiyonu içerir.

Teorem 11.5

(11.3) ve (11.4) denklemlerinde tanımlanan çırpı fonksiyonlarının sınıfı $\mathcal{H}_{pm}$ evrenselidir.

Kanıt $k \neq l$ olmak üzere, $\mathbb{Z}_p$ den iki farklı anahtar olan k ve l yi düşünün. Verilen bir h_{ab} çırpı fonksiyonu için

$$r = (ak + b) \bmod p,$$

$$s = (al + b) \bmod p$$

dir. Önce $r \neq s$ olduğunu anlarız. Neden?

$$r - s = a(k - l) \bmod p$$

olduğunu gözlemleyin. Bu $r \neq s$ olduğunu gösterir çünkü p asaldır ve hem a hem de $(k - l) \bmod p$ de 0 dan farklıdır ve bu yüzden Teorem 31.6 ya göre onların çarpımı da mod p de 0 dan farklı olmalıdır. Böylece $h_{ab} \in \mathcal{H}_{pm}$ yi hesaplarken, k

ve l farklı girdileri, $\text{mod } p$ de r ve s farklı değerleriyle eşleşir. $\text{Mod } p$ seviyesinde henüz çarpışma yoktur. Dahası, $a \neq 0$ olmak üzere (a, b) çifti için her bir olası $p(p-1)$ seçenekleri, $r \neq s$ olmak üzere farklı (r, s) çifti üretir. Bunun sebebi verilen r ve s den, a ve b için çözüm yapabilecek olmamızdır:

$$\begin{aligned} a &= ((r-s)(k-l)^{-1} \text{ mod } p) \text{ mod } p \\ b &= (r-ak) \text{ mod } p \end{aligned}$$

burada $((k-l)^{-1} \text{ mod } p)$ $k-l$ nin modulo p ye göre tek çarpımsal tersini gösterir. $r \neq s$ olmak üzere her muhtemel (r, s) çifti için sadece $p(p-1)$ seçenek olduğu için, $a \neq 0$ olmak üzere (a, b) ve $r \neq 0$ olmak üzere (r, s) çiftleri arasında birebir bağıntı kurulabilir. Bu yüzden, verilen her bir k ve l girdi çiftleri için, $\mathbb{Z}_p^* \times \mathbb{Z}_p$ den rassal düzgün biçimde bir (a, b) seçersek, oluşan (r, s) çifti, $\text{mod } p$ de her bir farklı değer çiftiyle eşit şekilde benzer olur.

Bu nedenle, farklı k ve l anahtarlarının çarpışma olasılığı, r ve s nin $\text{mod } p$ de farklı değerler olarak rastgele seçildiğindeki $r = s \text{ (mod } m)$ olasılığıyla aynıdır. s için $(p-1)$ kalan olası değerlerden verilen r değeri için, $s \neq r$ ve $s = r \text{ (mod } m)$ yi sağlayacak s değerleri sayısı en fazla

$$\begin{aligned} \lceil p/m \rceil - 1 &\leq ((p+m-1)/m) - 1 \quad (3.6) \text{ eşitsizliğinden} \\ &= (p-1)/m. \end{aligned}$$

m moduna düşürüldüğünde r ile s nin çarpışma olasılığı en fazla

$$((p-1)/m)/(p-1) = 1/m$$

dir. Bu yüzden her bir $k, l \in \mathbb{Z}_p$ farklı değer çiftleri için,

$$\Pr \{h_{ab}(k) = h_{ab}(l)\} \leq 1/m \text{ olur.}$$

$\mathcal{H}_{pm}$ tabii ki evrensel olur. ■

Alıştırılmalar

11.3-1

n boyunda bir bağlı listede arama yapmak istediğimizi farz edelim. Her eleman $h(k)$ çırpı değeriyle bir k anahtarı içersin. Her anahtar uzun karakter katarıdır. Verilen anahtara sahip bir elemanı listede ararken, çırpı değerlerinden nasıl faydalanabiliriz?

11.3-2

Taban-128 sayıları gibi düşünerek ve sonra bölme yöntemini uygulayarak, m tane hücreye r karakter katarını çarpıladığımızı varsayalım. M sayısını kolayca 32-bit bilgisayar kelimesi olarak temsil edebiliriz, fakat taban-128 sayısı gibi davranılan r karakterli karakter katarı, çok kelime alır. Karakter katarının kendisinin dışındaki bellekteki sabit kelime sayısından fazla kullanmadan karakter katarının çırpı değerini hesaplamak için bölme yöntemini nasıl kullanırız?

11.3-3

$m = 2^p - 1$ ve taban 2^p karakter katarı olduğu, $h(k) = k \bmod m$ bölme yöntemini düşünelim. x ve y nin aynı değere çarpılaması için, karakterlerinin sırasını değiştirerek karakter katarı y den karakter katarı x türetebileceğimizi gösterin. Çarpı fonksiyonunda bu özelliğin istenmediği bir uygulama örneği verin.

11.3-4

$m = 1000$ boyutundaki çarpı tablosunu ve $A = (\sqrt{5} - 1)/2$ için $h(k) = \lfloor m(kA \bmod 1) \rfloor$ i düşünelim. 61, 62, 63, 64, 65 anahtarlarının eşleneceği yerleri hesaplayınız.

11.3-5 *

Sonlu küme U dan sonlu küme B ye çarpı fonksiyonları ailesi $\mathcal{H}$ olsun. U daki her farklı k ve l elemanları için

$\Pr\{h(k) = h(l)\} \leq \epsilon$ sağlanırsa, $\mathcal{H}$ ye **-evrensel** diyelim.

O halde aşağıdaki eşitsizliğin sağlanmak zorunda olduğunu gösterin.

$$\epsilon \geq \frac{1}{|B|} - \frac{1}{|U|}$$

11.3-6 *

$U, \mathbb{Z}_p$ den çekilen n -tuple değer kümesi, p asal sayı ve $b = \mathbb{Z}_p$ olsun.

$$h_b(\langle a_0, a_1, \dots, a_{n-1} \rangle) = \left(\sum_{j=0}^{n-1} a_j b^j \right) \bmod p$$

olarak U dan $\langle a_0, a_1, \dots, a_{n-1} \rangle$ n -tuple girdide $b \in \mathbb{Z}$ için $h_b: U \rightarrow B$ çarpı fonksiyonu tanımlayınız. $\mathcal{H} = \{h_b: b \in \mathbb{Z}_p\}$ olsun. Alıştırma 11.3-5 deki ϵ -evrensel tanımına göre $\mathcal{H}$ nin $((n-1)/p)$ -evrensel olduğunu tartışın (*İpucu*: Alıştırma 31.4-4 e bakın).

11.4 Açık adresleme

Açık adreslemede, tüm elemanlar çarpı tablosunun kendisinde yer alır. Yani, her tablo girişi ya NIL ya da dinamik kümeden bir eleman içerir. Bir eleman ararken, sistematik olarak istenilen elemanı bulana veya elemanın tabloda olmadığından emin olana kadar tablo hücrelerini inceleriz. Zincirlemeden farklı olarak, hiçbir

liste yada hiçbir eleman tablonun dışında saklanmaz. Bu yüzden, açık adreslemede, fazladan araya sokma yapılamaması için çırpı tablosu doldurulur. Bunun bir sonucu olarak a yükleme faktörü asla 1 i geçemez.

Tabi ki, bağlı listeleri çırpı tablosunun içinde zincirleme için saklayabilirdik, (diğer seçenekte kullanılmamış çırpı tablosu hücreleri (Alıştırma 11.2-4)), fakat açık adreslemenin faydası şu ki işaretçilerden tamamen kaçınır. İşaretçileri takip etmek yerine, incelenecek hücre topluluğunu hesaplarız. İşaretçiler saklanmadığından oluşan ilave hafıza, aynı hafıza miktarı için daha fazla hücreli çırpı tablosu sağlar, potansiyel olarak daha az çarpışma ve daha fazla bilgi alımını destekler.

Açık adresleme kullanarak araya sokma yapabilmek için, anahtarı koyacak boş bir hücre bulana kadar çırpı tablosunu başarılı bir şekilde inceleriz veya **deneniz**. $\Theta(n)$ arama zamanı gerektiren $0, 1, \dots, m-1$ sabit sırasında olmak yerine, incelenen pozisyonların sırası, eklenecek anahtara bağlıdır. Hangi hücreye yerleştirileceğini saptamak için, ikinci girdi olarak 0 dan başlayarak deneme sayısını içermek için çırpı fonksiyonunu genişletiriz. Bu yüzden çırpı fonksiyonu

$$h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\}$$

dır. Açık adresleme ile, her k anahtarı için,

$$\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$$

deneme dizisinin $(0, 1, \dots, m-1)$ in permütasyonu olmasına ihtiyaç duyarız Böylece her çırpı tablosu pozisyonu, tablo doldukça yeni anahtar için en sonunda bir hücre olarak düşünülür. Aşağıdaki kabakodda, T çırpı tablosundaki elemanların uydu verisi olmayan anahtarlar olduğunu varsayabiliriz. k anahtarı k anahtarını içeren elemanla aynıdır. Her bir hücre ya anahtar ya da NIL içerir. (hücre boşsa). HASH-INSERT prosedürü çırpı tablosu H yi ve anahtar k yı girdi olarak alır. k anahtarının saklandığı yerdeki hücre numarasını ya da çırpı tablosu zaten dolu olduğundan hata bayraklarını döndürür.

HASH-INSERT (T, k)

```

1   $i = 0$ 
2  repeat
3       $j = h(k, i)$ 
4      if  $T[j] == \text{NIL}$ 
5           $T[j] = k$ 
6          return  $j$ 
7      else  $i = i + 1$ 
8  until  $i == m$ 
9  error “çırpı tablosu aşağı taşması”
```

k anahtarını arayan algoritma, k anahtarı eklendiğinde incelenen araya sokma algoritmasıyla aynı hücre sırasını inceler. Bu yüzden, arama boş bir hücre bulduğunda başarısız olarak biter. Çünkü k oraya önceden eklenmiş olabilir ve

daha sonra deneme dizisine eklenmez (bu aranan anahtarların çarpı tablosundan silinmediğini varsayar). HASH-SEARCH prosedürü, çarpı tablosu H yi ve anahtar k yi girdi olarak alır ve k anahtarını içeren hücre bulursa j yi bulamazsa NIL i döndürür.

HASH-SEARCH (T, k)

```

1   $i = 0$ 
2  repeat
3     $j = h(k, i)$ 
4    if  $T[j] = k$ 
5      return  $j$ 
6     $i = i + 1$ 
7  until  $T[j] = \text{NIL}$  veya  $i = m$ 
8  return NIL

```

Açık-adresli çarpı tablosunda silme zordur. i hücresinden anahtar sildiğimizde, orda NIL saklayarak bu hücreyi basitçe boş olarak işaretleyemeyiz. Bunu yaparsak, araya sokmasını incelediğimiz ve hücresini dolu bulduğumuz herhangi bir k anahtarını çekemeyebiliriz. Bu sorunu, hücreyi işaretleyerek, oraya NIL yerine DELETED yazdırarak çözebiliriz. O zaman, HASH-INSERT prosedürünü üzerine yeni bir eleman ekleyebileceğimiz için bu hücreyi, içi boş hücre gibi davranmak adına değiştirebiliriz. HASH-SEARCH ı değiştirmemize gerek yoktur. Çünkü arama sırasında DELETED yazılı hücreleri direk geçecektir. Özel değer DELETED ı kullandığımızda, yine de, arama zamanları artık yükleme faktörü α ya bağlı olmaz ve bu sebeple anahtarların silinmesi gerektiğinde zincirleme, çarpışma çözme tekniği olarak daha yaygınca seçilir.

Analizimizde, **düzgün çarpılama** yi varsaydık. Her bir anahtarın deneme dizisi $\langle 0, 1, \dots, m-1 \rangle$ in $m!$ permütasyonlarından herhangi biriyle eşit şekilde benzerdir. Düzgün çarpılama, sadece tek bir sayı değil, tüm deneme dizisini üreten çarpı fonksiyonu üzerinde önceden tanımlanmış basit düzgün çarpılama fikrini genelleştirir. Doğru düzgün çarpılamayı uygulamak zordur. Yine de, uygun pratik yaklaşımlar (düzgün çarpılamayı) kullanılır.

Açık adresleme için gereken deneme dizilerini hesaplamak için yaygın olarak kullanılan 3 tekniği inceleyeceğiz. (Doğrusal deneme, ikinci dereceden deneme ve çift çarpılama). Tüm bu teknikler, $\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$ in her bir k anahtarı için $\langle 0, 1, \dots, m-1 \rangle$ in permütasyonu olduğunu garantiler. Yine de, bu tekniklerin hiçbiri düzgün çarpılama varsayımını doğrulamaz. Çünkü hiçbiri m^2 farklı deneme teknikleri üretemez (düzgün çarpılamanın gerektirdiği $m!$ yerine). Çift çarpılama en fazla sayıda deneme dizisine sahiptir ve birinin bekleyebileceği gibi, en iyi sonuçları göstermeye adaydır.

Doğrusal deneme

Yardımcı çırpı fonksiyonu olarak da adlandırdığımız sıradan çırpı fonksiyonu $h' : U \rightarrow \{0, 1, \dots, m-1\}$ için, **doğrusal deneme** yöntemi $i = 0, 1, \dots, m-1$ için

$$h(k, i) = (h'(k) + i) \bmod m$$

çırpı fonksiyonunu kullanır. Verilen k anahtarı için, önce $T[h'(k)]$ yı inceleriz. Yani yardımcı çırpı fonksiyonu tarafından verilen hücreyi inceleriz. Sonra, $T[h'(k) + 1]$ hücrelerini inceleriz ve böylece $T[m-1]$ hücrelerine kadar hepsini inceleriz. Sonra, $T[0], T[1], \dots$ hücrelerini toparlarız ta ki $T[h'(k) - 1]$ hücrelerine ulaşana kadar. Başlangıç denemesi tüm deneme dizisini belirlediğinden, sadece m tane farklı deneme dizisi vardır.

Doğrusal denemenin uygulaması kolaydır, fakat **birincil kümeleme** adı verilen sorunda zorlanır. İşgal edilmiş hücrelerin uzun sürede inşa edilmesi ortalama arama zamanını yükseltir. i tane dolu hücrenin arkasına gelen boş hücre $(i + 1)/m$ olasılığı ile doldurulduğundan yığılmalar oluşur. Dolmuş hücrelerin çalışma uzunluğu daha da artar ve ortalama arama zamanı yükselir.

İkinci dereceden deneme

İkinci dereceden deneme şu çırpı fonksiyonunu kullanır:

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m. \quad (11.5)$$

h' yardımcı çırpı fonksiyonu, c_1 ve c_2 pozitif yardımcı sabitler ve $i = 0, 1, \dots, m-1$ dir. Başlangıçta denenen pozisyon $T[h'(k)]$ dır. Daha sonra pozisyonlar, deneme sayısı i üzerindeki karesel davranışa bağlı olan miktarlar tarafından dengelenir. Bu yöntem doğrusal denemeden daha iyi çalışır, fakat çırpı tablosunun tamamının kullanımı için, c_1 , c_2 ve m değerleri sınırlandırılmalıdır. Problem 11-3, bu parametreleri seçmenin bir yolunu gösterir. Ayrıca, iki anahtar aynı başlangıç deneme pozisyonuna sahipse, onların deneme dizileri de aynıdır. Çünkü, $h(k_1, 0) = h(k_2, 0)$, $h(k_1, i) = h(k_2, i)$ yi verir. Bu özellik, daha zayıf yığılma formuna yol açar. **İkincil kümeleme** olarak adlandırılır. Doğrusal denemede olduğu gibi, başlangıç denemesi, tüm diziyi belirler ve böylece m tane farklı deneme dizisi kullanılır.

Çift çırpılama

Çift çırpılama açık adresleme için uygun olan en iyi yöntemlerden birini sunar. Çünkü, üretilen permütasyonlar, rastgele seçilen permütasyonların birçok özelliğine sahiptir. **Çift çırpılama** şu çırpı fonksiyonunu kullanır:

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m.$$

h_1 ve h_2 yardımcı çırpı fonksiyonlarıdır. Başlangıç denemesi $T[h_1(k)]$ pozisyonuna gider. Ardışık denemesi pozisyonları önceki denemesi pozisyonlarıyla, $h_2(k)$

0	
1	79
2	
3	
4	69
5	98
6	
7	72
8	
9	14
10	
11	50
12	

Şekil 11.5 Çift çarpılama ile araya sokma. $h_1(k) = k \bmod 13$ ve $h_2(k) = 1 + (k \bmod 11)$ olan 13 elemanlı bir çarpı tablomuz var. $14 \equiv 3 \pmod{11}$ olduğundan, boş olan 9. hücreye 14 anahtarını, 1. ve 5. hücreleri inceleyip dolu olduklarını gördükten sonra ekleriz.

miktarı modül m tarafından dengelenir. Bu yüzden, doğrusal ve ikinci dereceden denemeden farklı olarak, deneme dizisi burda k anahtarı üzerinde 2 yola bağlıdır. Çünkü başlangıç deneme pozisyonu, ofset, ya da her ikisi değişebilir. Şekil 11.5 çift çarpılama yoluyla araya sokma örneği verir.

$h_2(k)$ değeri, aranacak tüm çarpı tablosu için m boyutundaki çarpı tablosuna göreceleri olarak asaldır. (alıştırma 11.4-4). Bu şartı kesinleştirmenin uygun yolu, m nin 2 nin kuvveti olmasını sağlamak ve h_2 yi her zaman tek sayı üretecek şekilde tasarlamaktır. Diğer yolu ise m yi asal yapmak ve h_2 yi daima m den düşük pozitif tam sayı döndürecek şekilde tasarlamaktır. Örneğin, m asal sayısını seçebiliriz ve:

$$h_1(k) = k \bmod m.$$

$$h_2(k) = 1 + (k \bmod m').$$

m' , m den çok az miktarda küçük olarak seçilir (örneğin $m - 1$). Örneğin, $k = 123456$, $m = 701$ ve $m' = 700$ ise $h_1(k) = 80$ ve $h_2(k) = 257$ olur. Böylece, önce pozisyon 80 i inceleriz ve sonra anahtarı bulana veya tüm hücreleri inceleyene kadar modül m den her 257 inci hücreyi inceleriz.

m , 2 nin kuvveti olduğunda, çift çarpılama doğrusal ve ikinci dereceden deneme algoritmalarından daha iyi çalışır. $\Theta(m^2)$ deneme dizisi yerine $\Theta(m)$ deneme dizisi kullanılır. Çünkü her muhtemel $(h_1(k), h_2(k))$ çiftlisi farklı deneme dizisi

kullanır. Sonuç olarak, bu tip m değerleri için, çift çırpılama performansı, tek biçim çırpılama “ideal” şemasının performansına oldukça yakın gözüktür.

m nin 22 nin kuvveti ve asal olma dışındaki tüm değerleri prensipte çift çırpılama kullanılmasına rağmen, pratikte kısmen m ye göre göreceleri olarak asal olmasını garanti edecek şekilde verimli olarak $h_2(k)$ üretmek daha zor hale gelir. Çünkü bu tip üyelerin $\phi(m)/m$ göreceli yoğunluğu küçük olabilir (denklem 31.24)).

Açık-adres çırpılamanın analizi

Zincirleme analizimizde olduğu gibi, açık adres analizimizi çırpı tablosunun $\alpha = n/m$ yükleme faktörü cinsinden ifade ederiz. Tabii ki, açık adreslemede, her hücreyi en fazla bir eleman doldurur ve bu yüzden $n \leq m$ ve $\alpha \leq 1$ dir.

Düzgün çırpılamayı kullandığımızı varsayalım. Bu idealleştirilmiş şemada, herhangi bir k anahtarını aramak ya da onu eklemek için kullanılan deneme dizisi $\langle h(k, 0), h(k, 1), \dots, h(k, m-1) \rangle$, $\{0, 1, \dots, m-1\}$ in herhangi bir permütasyonuna eşit derecede benzerdir. Taii ki, verilen bir anahtar kendisiyle bağdaştırılmış tek bir sabit deneme dizisine sahiptir. Burada söylemek istediğimiz şey, anahtarlar uzayındaki dağılım olasılığını ve anahtarlar üzerindeki çırpı fonksiyon işlevlerini düşünerek, her bir muhtemel deneme dizisi benzerdir.

Şimdi, tek biçim çırpılama varsayımı altında açık adresleme ile çırpılama için beklenen deneme sayısını, başarısız bir aramada yapılan deneme sayısı analizinden başlayarak analiz edebiliriz.

Teorem 11.6

$\alpha = n/m < 1$ yükleme faktörüyle verilen açık-adres çırpı tablosunda, başarısız bir aramada beklenen deneme sayısı, düzgün çırpılamayı varsayarak, en fazla $1/(1 - \alpha)$ olur.

Kanıt Başarısız bir aramada, istenilen anahtarı içermeyen işgal edilmiş hücreye son erişimler hariç her deneme ve denenen son hücre boştur. Rassal değişken X i, başarısız bir aramada yapılan deneme sayısı olarak tanımlayalım. $i = 1, 2, \dots$ için A_i olayı, i inci denemenin olduğu olay ve işgal edilmiş hücre olsun. O zaman, $\{X \geq i\}$ olayı, $A_1 \cap A_2 \cap \dots \cap A_{i-1}$ olaylarının kesişimi olur. $\Pr\{X \geq i\}$ ’yi $\Pr\{A_1 \cap A_2 \cap \dots \cap A_{i-1}\}$ tarafından sınırlayacağız. Alistırma C.2-5 ten,

$$\Pr\{A_1 \cap A_2 \cap \dots \cap A_{i-1}\} = \Pr\{A_1\} \cdot \Pr\{A_2 \mid A_1\} \cdot \Pr\{A_3 \mid A_1 \cap A_2\} \cdots \Pr\{A_{i-1} \mid A_1 \cap A_2 \cap \dots \cap A_{i-2}\} .$$

n eleman ve m hücre olduğundan, $\Pr\{A_1\} = n/m$ dir. $j > 1$ için j inci deneme olması ve o hücrenin dolu olma olasılığı, $j-1$ denemenin dolu olduğu verilerek, $(n-j+1)/(m-j+1)$ dir. Bu olasılık takip eder. Çünkü $(m-(j-1))$ incelenmemiş hü-

reden birinde kalan $(n - (j - 1))$ elemanlardan birini bulacağız ve düzgün çırpılma varsayımıyla, olasılık bu miktarların oranıdır. $n < m$ nin $0 \leq j \leq m$ olduğu tüm j ler için $(n - j)/(m - j)$ yi gözlemeleyerek, $1 \leq i \leq m$ olduğu tüm i lerimiz vardır,

$$\begin{aligned} \Pr\{X \geq i\} &= \frac{n}{m} \cdot \frac{n-1}{m-1} \cdot \frac{n-2}{m-2} \cdots \frac{n-i+2}{m-i+2} \\ &\leq \left(\frac{n}{m}\right)^{i-1} \\ &= \alpha^{i-1}. \end{aligned}$$

Şimdi, beklenen deneme sayısını sınırlamak için, (C.25) denklemini kullanırız:

$$\begin{aligned} E[X] &= \sum_{i=1}^{\infty} \Pr\{X \geq i\} \\ &\leq \sum_{i=1}^{\infty} \alpha^{i-1} \\ &= \sum_{i=0}^{\infty} \alpha^i \\ &= \frac{1}{1-\alpha}. \end{aligned}$$

■

$1/(1-\alpha) = 1 + \alpha + \dots$ nin sınırının sezgisel bir yorumu vardır. Her zaman ilk denemeyi yaparız. Yaklaşık α olasılığıyla, ilk deneme işgal edilmiş bir hücre bulur. Böylece ikinci kez denemeye ihtiyaç duyarız. Yaklaşık α^2 olasılığıyla, ilk iki hücre doldurulur ve böylece üçüncü denemeyi yaparız. Böyle gider.

α bir sabitse, Teori 11.6 başarısız bir aramanın $O(1)$ zamanda çalışacağını tahmin eder. Örneğin, çırpı tablosu yarı dolu ise, başarısız bir aramada ortalama deneme sayısı en fazla $1/(1-0.5) = 2$ dir. Yüzde 90 dolu ise, ortalama deneme sayısı en fazla $1/(1-0.9) = 10$ dur.

Teorem 11.6, neredeyse hemen HASH-INSERT prosedürünün performansını verir.

Sonuç 11.7

α yükleme faktörlü bir açık-adres çırpı tablosuna eleman eklemek düzgün çırpılamayı varsayarak en fazla ortalama $1/(1-\alpha)$, deneme gerektirir.

Kanıt Bir eleman yalnızca tabloda yer varsa eklenebilir ve bu yüzden $\alpha < 1$ dir. Anahtar eklemek, bulunan ilk boş hücreye anahtar yerleştirmeyi takip eden bir başarısız arama gerektirir. Bu yüzden, beklenen deneme sayısı en fazla $1/(1 - \alpha)$ olur. ■

Başarılı bir arama için beklenen deneme sayısını hesaplamak için biraz daha uğraşmamız gerekir.

Teorem 11.8

$\alpha < 1$ yükleme faktörüyle verilen açık-adres çırpı tablosunda başarılı bir aramada beklenen deneme sayısı en fazla

$$\frac{1}{\alpha} \ln \frac{1}{1 - \alpha} \text{ dır.}$$

Kanıt Anahtar k yı aramak, k elemanı eklendiğinde oluşan aynı deneme dizisini türetir. Sonuç 11.7 den, k , çırpı tablosuna eklenen $(i + 1)$ inci anahtarsa, k yı aramak için yapılan beklenen deneme sayısı en fazla $1/(1 - i/m) = m/(m - i)$ dır. Çırpı tablosundaki tüm n anahtarların ortalamasını almak, başarılı bir aramadaki deneme sayısını bize verir:

$$\begin{aligned} \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m - i} &= \frac{m}{n} \sum_{i=0}^{n-1} \frac{1}{m - i} \\ &= \frac{1}{\alpha} \sum_{k=m-n+1}^m \frac{1}{k} \\ &\leq \frac{1}{\alpha} \int_{m-n}^m (1/x) dx \quad (\text{denklem (A.12) den}) \\ &= \frac{1}{\alpha} \ln \frac{m}{m - n} \\ &= \frac{1}{\alpha} \ln \frac{1}{1 - \alpha} . \end{aligned}$$

■

Çırpı tablosu yarı dolu ise, başarılı bir aramada beklenen deneme sayısı 1.387 den azdır. Çırpı tablosu yüzde 90 dolu ise, beklenen deneme sayısı 2.559 dan azdır.

Alıřtırmalar

11.4-1

Yardımcı ırpı fonksiyonu $h'(k) = k$ ile açık adresleme kullanarak $m = 11$ uzunluğunda bir ırpı tablosuna 10, 22, 31, 4, 15, 28, 17, 88, 59 anahtarlarını eklediğimizi düşünelim. Doğrusal deneme, $c_1 = 1$ ve $c_2 = 3$ kullanan ikinci dereceden deneme, $h_1(k) = k$ ve $h_2(k) = 1 + (k \bmod (m - 1))$ içeren çift ırpılama kullanarak bu araya sokmaların sonucunu şekillendiriniz.

11.4-3

Düzgün ırpılama ile bir açık-adres ırpı tablosu düşünün. Yükleme faktörü $3/4$ ve $7/8$ olduğunda başarılı bir aramadaki beklenen deneme sayılarına ve başarısız bir aramadaki beklenen deneme sayılarına üst sınırlar verin.

11-4.4 ★

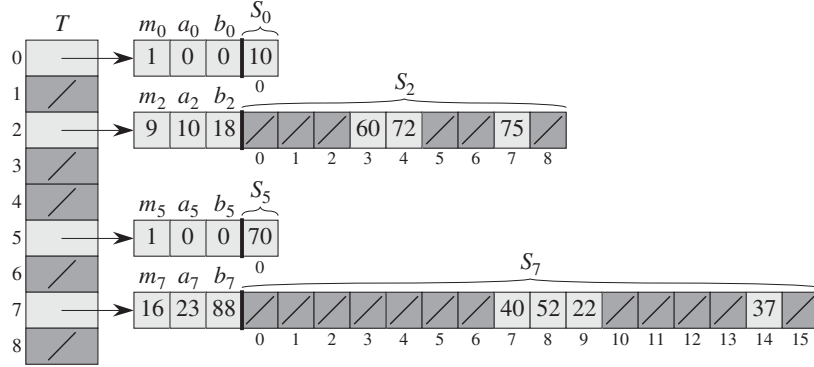
Çarpışmaları çözmek için çift ırpılama kullandığımızı varsayalım. Yani, $h(k, i) = (h_1(k) + ih_2(k)) \bmod m$ ırpı fonksiyonunu kullanırız. m ve $h_2(k)$ nın ortak bölüneni bazı k anahtarları için $d \geq 1$ olursa, k anahtarı için başarısız aramanın $h_1(k)$ hücresine döndürmeden önce ırpı tablosunun $(1/d)$ inci hücresini inceleyeceğini gösterin. Bu yüzden, $d = 1$ olduğunda, m ve $h_2(k)$ aralarında asal olur. Arama tüm ırpı tablosunu inceleyebilir (*ipucu*: Bölüm 31).

11.4-5 ★

Yükleme faktörü α olan açık-adres ırpı tablosunu düşünün. Başarısız bir aramadaki beklenen deneme sayısının, başarılı aramadakinin iki katı olduğu durum için 0 olmayan α değerini bulun. Bu beklenen deneme sayıları için Teorem 11.6 ve Teorem 11.8 de verilen üst sınırları kullanın.

★ 11.5 Kusursuz ırpılama

Kusursuz ortalama-durum performansı için genelde ırpılama iyi bir tercih olmasına rağmen, anahtarlar kümesi **durağan** olduğunda (anahtar kümesi asla değişmediğinde), en kötü durumda da kusursuz performans sağlayabilir. Bazı uygulamalar doğal olarak durağan anahtar kümesine sahiptir. Programlama dilindeki saklı kelimeler kümesini ya da CD-ROM daki dosya ismi kümelerini düşünün.



Şekil 11.6 $K = \{10, 22, 37, 40, 52, 60, 70, 72, 75\}$ kümesinde saklamak için kusursuz çırpılama-yı kullanmak. Dış çırpı fonksiyonu $a = 3$, $b = 42$, $p = 101$ ve $m = 9$ olduğu $h(k) = ((ak + b) \bmod p) \bmod m$ dir. Örneğin, $h(75) = 2$ ve bu yüzden anahtar 75 T tablosunun 2. hücreğine çırpılır. İkincil çırpı tablosu S_j ise j hücreğine çırpılan tüm anahtarları saklar. S_j çırpı tablosunun boyutu $m_j = n_j^2$ ve ona ilişkin çırpı fonksiyonu $h_j(k) = ((a_j k + b_j) \bmod p) \bmod m_j$. $H_2(75) = 7$ olduğundan, anahtar 75, ikincil çırpı tablosu S_2 nin 7. hücreğinde saklanır. İkincil çırpı tablosunun hiçbirinde çarpışma olmaz ve bu yüzden arama en kötü-durumda sabit zaman alır.

En kötü-durumda arama gerçekleştirmek için $O(1)$ bellek erişimi gerekiyorsa, bu çırpılama tekniğine **kusursuz çırpılama** deriz.

Kusursuz çırpılama şeması yaratmak için, her aşamada evrensel çırpılama ile, iki aşamalı çırpılama kullanırız. Şekil 11.6 yaklaşımı şekillendirir.

İlk aşama zincirlemeyle çırpılamada yapıldığı gibi aynı yapılıdır. Evrensel çırpı fonksiyonları ailesinden dikkatlice seçilmiş çırpı fonksiyonu h yi kullanarak m tane hücreye n tane anahtarı çırpılıyoruz.

j hücreğine çırpılayan anahtarların bağlı listesini yapmak yerine, yine de, Çağırışım yapan çırpı fonksiyonu h_j ile küçük **ikincil çırpı tablosu** kullanırız. Çırpı fonksiyonu h_j yi dikkatlice seçerek, ikincil aşamada hiçbir çarpışma olmayacağını garanti ederiz.

Yine de, ikincil aşamada hiçbir çarpışma olmayacağını garanti etmek için, S_j çırpı tablosunun boyutunun m_j olmasına izin verimiz. m_j nin n_j üzerinde karesel bağımlılığının çok fazla bellekte yer işgaline yol açıyor gibi görüldüğünü düşünseniz bile, birinci-aşama çırpı fonksiyonunu iyi seçerek, $O(n)$ de kullanılan beklenen yer miktarını kısıtlayabileceğimizi göstereceğiz.

Kısım 11.3.3 deki çırpı fonksiyonlarının evrensel sınıflarından seçilmiş çırpı fonksiyonları kullanırız. Kısım 11.3.3 deki olduğu gibi birinci aşama çırpı fonksiyonları $\mathcal{H}_{pm}$ sınıfından gelir. p herhangi bir anahtar değerinden büyük olan asal

sayıdır. j hücresine ırpılayan bu anahtarlar, $\mathcal{H}_{p,m,j}$ sınıfından seilen h_j ırpı fonksiyonu kullanarak, m_j boyundaki S_j ikincil ırpı tablosuna tekrar ırpılanır. (1)

İki aşamada ilerleyeceğiz. Birincisi, ikincil ırpı tablosunun nasıl arpışma içermeyeceğini garanti etmeyi belirleyeceğiz. İkincisi, birincil ırpı tablosu ve tüm ikincil ırpı tabloları için beklenen kullanılan tüm bellek miktarının $O(n)$ olduğunu göstereceğiz.

Teorem 11.9

ırpı fonksiyonlarının evrensel sınıfından rastgele seilmiş h ırpı fonksiyonunu kullanarak $m = n^2$ boyutunda bir ırpı tablosuna n tane anahtar yerleřtirdiğimizi varsayın. O zaman, hiç arpışma olmama olasılığı $1/2$ den az olur.

Kanıt arpışabilecek $C\binom{n}{2}$ tane anahtar vardır. h , ırpı fonksiyonları evrensel ailesi $\mathcal{H}$ den rastgele seilmişse, her bir çift $1/m$ olasılığı ile arpışır. X , arpışma sayısını sayan rassal bir deėişken olsun. $m = n^2$ olduğunda, beklenen arpışma sayısı,

$$\begin{aligned} E[X] &= \binom{n}{2} \cdot \frac{1}{n^2} \\ &= \frac{n^2 - n}{2} \cdot \frac{1}{n^2} \\ &< 1/2 \end{aligned}$$

dır. (Bu analiz, Kısım 5.4.1 deki doğum günü paradoksu analizine benzerdir). Markov'un eşitsizliğini uygularsak, $t = 1$ olduğunda $\Pr\{X \geq 1\} \leq E[X]/t$ kanıtı tamamlar. ■

$m = n^2$ olduğu Teorem 11.9 da tarif edilen durumda, $\mathcal{H}$ den rasgele seilen ırpı fonksiyonu h , arpışma içermeye daha yatkındır. ırpılanacak n tane anahtarın kümesi K verildiğine göre (K nın durağan olduğunu hatırlayın), birkaç rassal denemeye arpışma olmayan bir ırpı fonksiyonu bulmak kolay hale gelir.

Yine de, n çok büyük olduğunda, $m = n^2$ boyutlu ırpı tablosu aşırı genişler. Bu yüzden, iki aşamalı ırpılama yaklaşımı benimseriz ve yalnızca her bir hücredeki girdilere ırpılmak için Teorem 11.9 un yaklaşımını kullanırız. $m = n$ hücreye anahtarları ırpılmak için birinci aşama ya da dış ırpı fonksiyonu kullanırız. Böylece arpışma olmayan sabit zamanda işi tamamlarız.

¹ $n_j = m_j = 1$ olduğunda, j hücresi için ırpı fonksiyonuna ihtiyaç duymayız. Her bir hücre için $h_{ab}(k) = ((ak + b) \bmod p) \bmod m_j$ ırpı fonksiyonunu setiğimizde, yalnızca $a = b = 0$ ı kullanırız.

Şimdi kullanılan tüm bellek miktarının $O(n)$ olması konumuza dönüyoruz. j inci ikincil çırpı tablosunun m_j boyutu, saklanan n_j anahtar sayısı ile karesel olarak büyüdüğünden, kullanılan tüm bellek miktarının çok fazla olması riskini taşıyız.

İlk aşama tablosunun boyutu $m = n$ ise, o zaman birincil çırpı tablosu için, ikincil çırpı tablolarının m_j boyutlarının belleği için ve Kısım 11.3.3 ün $\mathcal{H}_{p,m,j}$ sınıfından çekilen ($n_j = 1$ olduğu durum ve $a = b = 0$ kullandığımız durum hariç) h_j ikincil çırpı fonksiyonlarını tanımlayan a_j ve b_j parametrelerinin bellekte saklanması için kullanılan bellek miktarı $O(n)$ dir. Aşağıdaki teorem ve sonuç, tüm ikincil çırpı tablolarının beklenen birleşmiş boyutlarına sınır koyma imkanı sağlar. İkinci sonuç, tüm ikincil çırpı tablolarının birleşmiş boyutunun süperdoğrusal olma olasılığını sınırlar (aslında, $4n$ e eşit veya ondan büyüktür).

Teorem 11.10

Çırpı fonksiyonlarının evrensel sınıfından rastgele seçilen h çırpı fonksiyonunu kullanarak $m = n$ boyutunda bir çırpı tablosunda n tane anahtar tuttuğumuzu varsayın. O zaman ;

$$E \left[\sum_{j=0}^{m-1} n_j^2 \right] < 2n$$

yi elde ederiz, burada n_j, j hücrelerine çırpılayan anahtarların sayısıdır.

Kanıt Negatif olmayan her a tam sayısı için çalışan aşağıdaki eşitlikle başlarız:

$$a^2 = a + 2 \binom{a}{2}. \quad (11.6).$$

$$\begin{aligned} E \left[\sum_{j=0}^{m-1} n_j^2 \right] &= E \left[\sum_{j=0}^{m-1} \left(n_j + 2 \binom{n_j}{2} \right) \right] && \text{(denklem (11.6))} \\ &= E \left[\sum_{j=0}^{m-1} n_j \right] + 2 E \left[\sum_{j=0}^{m-1} \binom{n_j}{2} \right] && \text{(beklenti doğrusallığı)} \\ &= E[n] + 2 E \left[\sum_{j=0}^{m-1} \binom{n_j}{2} \right] && \text{(denklem (11.1))} \end{aligned}$$

$$= n + 2 E \left[\sum_{j=0}^{m-1} \binom{n_j}{2} \right] \quad (n \text{ rassal deęiřken olmadıęı iin})$$

elde ederiz.

Bu toplamı deęerlendirmek iin, arpıřan ırpı tablosundaki anahtar iftleri-
nin toplam sayısını inceleriz. Evrensel ırpılama zellięinden, bu toplamın bek-
lenen deęeri en fazla

$$\begin{aligned} \binom{n}{2} \frac{1}{m} &= \frac{n(n-1)}{2m} \\ &= \frac{n-1}{2} \end{aligned}$$

dir, ünkü $m = n$ dir. Bu yzden,

$$\begin{aligned} E \left[\sum_{j=0}^{m-1} n_j^2 \right] &\leq n + 2 \frac{n-1}{2} \\ &= 2n - 1 \\ &< 2n \end{aligned}$$

dir. ■

Sonuç 11.11

ırpı fonksiyonları evrensel sınıfından rastgele seilmiř ırpı fonksiyonu h yi kullanarak $m = n$ boyutundaki ırpı tablosuna n tane anahtar yerleřtirdięimizi varsayın. Her bir ikincil ırpı tablosundaki boyutları, $j = 0, 1, \dots, m-1$ iin $m_j = (n_j^2)$ yaptığımızı dřnelim. O zaman, Kusursuz ırpılama řemasındaki tm ikin-
cil ırpı tabloları iin gereken belleęin beklenen miktarı $2n$ den az olur.

Kanıt $j = 0, 1, \dots, m-1$ iin $m_j = n_j^2$ olduęundan, Teorem 11.10 kanıtı tamam-
layan

$$\begin{aligned} E \left[\sum_{j=0}^{m-1} m_j \right] &= E \left[\sum_{j=0}^{m-1} n_j^2 \right] \\ &< 2n \end{aligned} \quad (11.7)$$

kanıtı tamamlayan eřitsizlięi verir. ■

Sonuç 11.12

ırpı fonksiyonları evrensel sınıfından rassal ırpı fonksiyonu h yi kullanarak $m = n$ boyutundaki ırpı tablosuna n tane anahtar yerleřtirdięimizi varsayın. Her bir ikincil ırpı tablosundaki boyutları, $j = 0, 1, \dots, m-1$ iin $m_j = n_j^2$ yaptığımızı dřnelim. O zaman, ikincil ırpı tabloları iin kullanılan toplam belleęin $4n$ ye eřit olma veya ondan byk olma olasılıęı $1/2$ den azdır.

Kanıt Yine, Markov'un eşitsizliğini uyguluyoruz. $\Pr\{X \geq t\} \leq E[X]/t$ dir. Bu kez

$X = \sum_{j=0}^{m-1} m_j$ ve $t = 4n$:

$$\begin{aligned} \Pr\left\{\sum_{j=0}^{m-1} m_j \geq 4n\right\} &\leq \frac{E\left[\sum_{j=0}^{m-1} m_j\right]}{4n} \\ &< \frac{2n}{4n} \\ &= 1/2 \end{aligned}$$

dir. ■

Sonuç 11.12 den evrensel aileden rassal birkaç çırpı fonksiyonunu test edersek, mantıklı miktarda bellek kullanan birini kolayca bulacağız.

Alıştırmalar

11.5-1 ★

Düzgün çırpılama ve açık adresleme ile m boyutunda bir çırpı tablosuna n tane anahtar yerleştirdiğimizi varsayın. $p(n, m)$, hiçbir çarpışma olmama olasılığı olsun. $p(n, m) \leq e^{-n(n-1)/2m}$ olduğunu gösterin. (*İpucu:* denklem 3.12 ye bakın.) $n, \sqrt{m}$ den büyük olduğunda, çarpışmadan kaçınma olasılığının hızla 0 a doğru gittiğini tartışın.

Problemler

11-1 Çırpılama için en uzun-deneme sınırı

$n \leq m/2$ elemanı saklamak için kullanılan m boyutundaki açık adresli çırpı tablosunu kullandığımızı varsayın.

- a. Düzgün çırpılama varsayımıyla, $i = 1, 2, \dots, n$ için, i inci araya sokmanın katiyetle k dan daha fazla deneme gerektirme olasılığının en fazla 2^{-k} olduğunu gösterin.
- b. $i = 1, 2, \dots, n$ için, i inci araya sokmanın $2 \lg n$ den daha fazla deneme gerektirme olasılığının $O(1/n^2)$ olduğunu gösterin.

X_i rastgele değişkeninin, i inci ekleme tarafından istenen deneme sayısı olduğunu varsayın. $p\{X_i > 2 \lg n\} = O(1/n^2)$ olduğunu b kısmında gösterdiniz. Rassal değişken $X = \max_{1 \leq i \leq n} X_i$, n tane araya sokmanın herhangi biri tarafından ihtiyaç duyulan maximum deneme sayısını gösterdiğini düşünün.

- c. $\Pr\{X > 2 \lg n\} = O(1/n)$ olduğunu gösterin.
- d. En uzun deneme dizisinin beklenen uzunluğunun $E[X]$, $O(\lg n)$ olduğunu gösterin.

11-2 Zincirleme için sınırı hücre-boyutu

Çarpışmaların zincirlemeyle çözüldüğü n hücreli bir ırpı tablomuz olduğunu ve bu tabloya n anahtar eklendiğini varsayın. Her bir hücreye ırpılanacak anahtarlar eşit şekilde benzerdir. M , tüm anahtarlar eklendikten sonra herhangi bir hücredeki maksimum anahtar sayısıdır. Sizin göreviniz, M nin beklenen değerinin $E[M]$ üst sınırının $O(\lg n / \lg \lg n)$ olduğunu kanıtlamaktır.

a. Belli bir hücreye tam tamına k anahtarın ırpılama olasılığının:

$$Q_k = \left(\frac{1}{n}\right)^k \left(1 - \frac{1}{n}\right)^{n-k} \binom{n}{k} \text{ olduğunu gösteriniz.}$$

b. P_k , $M = k$ nin olasılığı, yani, en çok anahtarı taşıyan hücrenin k anahtar taşıma olasılığı olsun. $P_k \leq nQ_k$ olduğunu gösterin.

c. $Q_k < e^k / k^k$ olduğunu göstermek için Stirlin'in yaklaşımını kullanın (Denklem 3.18).

d. $k_0 = \text{clg } n / \lg \lg n$ için $Q_{k_0} < 1/n^3$ olacak şekilde $c > 1$ i sağlayan bir c sabiti olduğunu gösterin. $k \geq k_0 = \text{clg } n / \lg \lg n$ için $P_k < 1/n^2$ sonucuna ulaşın.

e. $E[M] \leq \Pr \left\{ M > \frac{c \lg n}{\lg \lg n} \right\} \cdot n + \Pr \left\{ M \leq \frac{c \lg n}{\lg \lg n} \right\} \cdot \frac{c \lg n}{\lg \lg n}$ olduğunu gösterin..

$E[M] = O(\lg n / \lg \lg n)$ sonucuna ulaşın.

11-3 İkinci dereceden Deneme

Pozisyonları $0, 1, \dots, m-1$ olan bir ırpı tablosunda verilen k anahtarını aradığımızı varsayın. Anahtar uzayından $\{0, 1, \dots, m-1\}$ kümesine eşleme yapan bir h ırpı fonksiyonumuz olsun. Arama şeması şöyle devam eder:

1. $j = h(k)$ değerini hesaplayın ve $i = 0$ yapın.
2. İstenen anahtar k için j pozisyonunda deneyin. Onu bulursanız, ya da bu pozisyon boşsa, aramayı bitirin.
3. $i = i + 1$ yapın. Şimdi $i = m$ olduysa, tablo doludur. Böylece aramayı bitirin. Aksi halde, $j = (i + j) \bmod m$ yapın ve 2. adıma dönün.

m nin, 2 nin bir kuvveti olduğunu varsayın.

a. Denklem 11.5 için uygun c_1 ve c_2 yi göstererek, bu şemanın, genel “ikinci dereceden” şemasının bir örneği olduğunu gösterin.

b. Bu algoritmanın en kötü-durum her tablo pozisyonunu incelediğini kanıtlayın.

11-4 Çırpılama ve Kimlik Doğrulama

$\mathcal{H}$, her çırpı fonksiyonu $h \in \mathcal{H}$ nin anahtarlar evreni U dan $\{0, 1, \dots, m-1\}$ eşleme yaptığı çırpı fonksiyonları sınıfı olsun. k farklı anahtarların $\langle x^{(1)}, x^{(2)}, \dots, x^{(k)} \rangle$ her sabit dizisi ve $\mathcal{H}$ den rastgele seçilen her h için, sıralama $\langle h(x^{(1)}), h(x^{(2)}), \dots, h(x^{(k)}) \rangle$, $\{0, 1, \dots, m-1\}$ den çekilen elemanlarla m^k sayıda k boyutundaki dizilerin herhangi birine eşit şekilde benzerse, $\mathcal{H}$ nin ***k-evrensel*** olduğunu söyleriz.

- a. Çırpı fonksiyonları ailesi $\mathcal{H}$ 2-evrenselse, onun aynı zamanda evrensel olduğunu gösterin.
- b. Evren U nun, p nin asal olduğu, $\mathbb{Z}_p = \{0, \dots, p-1\}$ den çekilen değerlerin n -tuple kümesi olduğunu gösterin.
 $x = \langle x_0, x_1, \dots, x_{n-1} \rangle \in U$ olduğunu düşünün. Herhangi bir n -tuple $a = \langle a_0, a_1, \dots, a_{n-1} \rangle \in U$ için, h_a çırpı fonksiyonunu şöyle tanımlayın:

$$h_a(x) = \left(\sum_{j=0}^{n-1} a_j x_j \right) \bmod p$$

$\mathcal{H} = \{h_a\}$ olsun. $\mathcal{H}$ nin evrensel olduğunu, fakat 2-evrensel olmadığını gösterin (*İpucu: $\mathcal{H}$ deki tüm çırpı fonksiyonlarının aynı değer ürettiği bir anahtar bulun*).

- c. Herhangi bir $a \in U$ ve herhangi bir $b \in \mathbb{Z}_p$ için b bölümündeki $\mathcal{H}$ yi biraz değiştirdiğimizi varsayın.

$$h'_{ab}(x) = \left(\sum_{j=0}^{n-1} a_j x_j + b \right) \bmod p$$

$\mathcal{H}$ nin 2-evrensel olduğunu söyleyin (*İpucu: Bazı i ler için $x_i \neq y$ ile $x \in U$ ve $y \in U$ sabit n -tuple larını düşünün. a_i ve $b, \mathbb{Z}_p$ nin etrafında değer aldığında $h'_{ab}(x)$ e ne olur?*).

- d. Varsayın ki Alice ve Bob gizlice $\mathcal{H}$ çırpı fonksiyonlarının 2 evrensel ailesinden h çırpı fonksiyonu üzerinde anlaşmaya varırlar. Her $h \in \mathcal{H}$, p nin asal olduğu, U anahtarlar evreninden $\mathbb{Z}_p$ ye çırpılır. Da ha sonra, Alice, internet üzerinden Bob'a m mesajını, $m \in U$, gönderir. $T = h(m)$ yetki etiketi gönderecek, Bob'a bu mesajın yetkisini de verir ve Bob aldığı (m, t) çiftinin $t = h(m)$ ile uyumunu kontrol eder. Varsayın ki, bir rakip (m, t) nin yol üzerinde önünü kesiyor ve (m, t) mesajını başka bir (m', t') ile değiştirerek Bob'u şaşırtmaya çalışıyor. Bob'un yanlış mesajı kabul ederek yanılma olasılığının en fazla $1/p$ olduğunu gösterin.

Bölüm notları

Knuth [211] ve Gonnet [145], ırpı algoritmaları için harika referanslardır. Knuth, arpışma özmek için kullanılan zincirleme yöntemlerle beraber ırpı tabloları tasarlanması için H.P. Luhn (1953) e arka ıkmıştır. Aynı zamanda G.M. Amdahl açık adresleme fikrini ortaya sunmuştur.

Carter ve Wegman, 1979 da ırpı fonksiyonlarının evrensel sınıfları fikrini ortaya atmıştır [59].

Fredman, Komlos ve Szemerédi [112], Kısım 11.5 de sunulan sabit kümeler için kusursuz ırpılama şeması geliştirdi. Araya sokma ve silmelerin amortize beklenen zamanının $O(1)$ olmasını sağlayarak, onların yönteminin dinamik kümelere genişletilmesi, Dietzfelbinger tarafından verilmiştir. [86].

Arama Ağacı veri yapısı birçok dinamik küme işlemlerini destekler, SEARCH, MINIMUM, MAXIMUM, PREDECESSOR, SUCCESSOR, INSERTION ve DELETION gibi. Bu yüzden, bir arama ağacını hem sözlük hem de öncelikli kuyruk gibi kullanabiliriz.

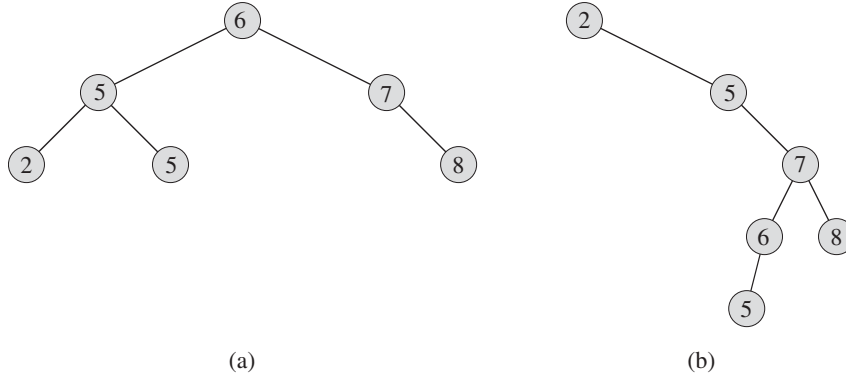
Arama ağacı üzerinde temel işlemler ağacın boyutuyla doğru orantılı olacak biçimde zaman alır. n düğümlü bütün bir arama ağacı için, bu tarz işlemler en kötü durumda $\Theta(\log n)$ zaman alır. Eğer ağaç n düğümlü doğrusal bir zincir ise, aynı işlemler en kötü durumda $\Theta(n)$ zaman alır. Kısım 12.4 de rassal oluşturulmuş ikili arama ağacının beklenen yüksekliğinin $O(\log n)$ olduğunu göreceğiz. Bu yüzden bu ağaçta temel dinamik küme işlemleri ortalama $\Theta(\log n)$ zaman alır.

Uygulamada, ikili arama ağaçlarının rassal oluşturulacağını garanti edemeyiz, fakat temel işlemler üzerinde iyi garantilenmiş en kötü durum performanslı ikili arama ağacı çeşitlendirmeleri tasarlayabiliriz. Bölüm 13, bu tarz çeşitlendirmeler yani yüksekliği $O(\log n)$ olan kırmızı siyah ağaçları sunar. Bölüm 18, ikincil disk üzerindeki veri tabanlarını tutmak için iyi olan B-ağaçlarını anlatır.

İkili arama ağaçları üzerindeki temel işlemleri sunduktan sonra, ileriki bölüm, ikili arama ağacındaki bir değer nasıl arandığını, en küçük ve en büyük elemanın nasıl bulunduğunu, bir elemanın öncül ve ardılının nasıl bulunduğunu, nasıl eleman eklendiğini ve silindiğini ve içindeki değerleri sıralı biçimde yazdırmak için ikili arama ağacında nasıl dolaşıldığını gösterir. Ek B de temel matematiksel özellikler gösterilir.

12.1 İkili arama ağacı nedir?

Şekil 12.1 de gösterildiği gibi isminden de anlaşılacağı gibi ikili ağaç içinde düzenlenmiş bir ağaçtır. Her düğümün nesne olduğu bir bağlı veri yapısıyla bu ağacı temsil edebiliriz. *Key* ve uydu verisine ek olarak, her düğümün sol çocuğunu, sağ çocuğunu ve ebeveynini gösteren *left*, *right* ve *p* işaretçileri gibi özellikleri vardır.



Şekil 12.1 İkili arama ağaçları, her hangi bir x düğümü için, sol altağacındaki anahtarlar en fazla $x.key$ dir ve sağ altağacındaki anahtarlar en az $x.key$ dir. Farklı ikili arama ağaçları aynı değer kümesiyle temsil edilir. Çoğu arama ağaçları işlemleri için en kötü durum çalışma süresi ağacın yüksekliğiyle doğru orantılıdır. (a) yüksekliği 2 olan 6 düğüm üstünde bir ikili arama ağacı. (b) aynı anahtarları içeren boyu 4 olan daha az verimli bir ikili arama ağacı.

Eğer bir çocuk veya ebeveyn kayıpsa, uygun özellik NIL değeri alır. Ağaçta ebeveyni NIL olan tek düğüm kök düğümdür.

İkili arama ağacındaki anahtarlar her zaman **ikili-arama-ağacı özelliğini** gerçekleyecek biçimde saklanır.

x ikili arama ağacında bir düğüm olsun. Eğer y , x in sol altağacında bir düğüm ise, $y.key \leq x.key$ dir. Eğer y , x in sağ altağacında bir düğüm ise $y.key \geq x.key$.

Bu yüzden, Şekil 12.1(a) da, kökün anahtarı 6, sol altağacıdaki anahtarlar 2, 5 ve 5, 6 dan büyük değillerdir ve sağ alt ağacıdaki anahtarlar 7 ve 8, 6 dan küçük değillerdir. Aynı özellik ağaçtaki her düğüm için geçerlidir. Örneğin, kökün sol çocuğundaki anahtar 5, bu düğümün sol altağacındaki düğümün anahtarı 2 den küçük değildir ve sağ altağacındaki düğümün anahtarı 5 den büyük değildir.

İkili-arama-ağacı özelliği bize düzenli ağaç adımı olarak adlandırılan basit bir özyinelemeli algoritmayla sıralı bir ikili arama ağacının tüm anahtarlarını yazdırma olanağı verir ve **sıralı ağaç yürüyüşü** olarak adlandırılır. Bu algoritma böylele adlandırılmıştır çünkü önce sol altağacın değerlerini yazdırır, ardından kökün anahtarını ve en son sağ altağacın değerlerini yazdırır (Benzer olarak, **ön sıra ağaç yürüyüşü** altağaçlardan önce kökü yazar, **son sıra ağaç yürüyüşü** altağaçlardan sonra kökü yazdırır). İkili arama ağacı T deki tüm elemanları yazdırmak amacıyla sıradaki prosedürü kullanmak için, INORDER-TREE-WALK(T , $root$) u çağırırız.

INORDER-TREE-WALK (x)

```

1  if  $x \neq \text{NIL}$ 
2      INORDER-TREE-WALK ( $x.\text{left}$ )
3      print  $x.\text{key}$ 
4      INORDER-TREE-WALK ( $x.\text{right}$ )

```

Örnek olarak, sıralı ağaç yürüyüşü Şekil 12.1 deki her bir ikili arama ağacının anahtarlarını şu sırayla yazar: 2, 5, 5, 6, 7, 8. İkili arama ağacı özelliğinden direkt olarak çıkarımla algoritmanın doğruluğu kanıtlanır.

n düğümlü bir ikili arama ağacında yürümek $\Theta(n)$ zaman alır. Çünkü başlangıç çağrısından sonra, ağaçtaki her bir düğüm için fonksiyon özyinelemeli olarak 2 kez çağırır. Bir kez sol çocuğu bir kez de sağ çocuğu. Sıralı ağaç yürüyüşünün doğrusal zaman aldığını resmi olarak kanıtlayan teorem aşağıdadır.

Teorem 12.1

Eğer x , n düğümlü bir altağacın kökü ise, INORDER-TREE-WALK (x) $\Theta(n)$ zaman alır.

Kanıt n düğümlü bir altağacın kökü üzerinde çağrıldığında INORDER-TREE-WALK un alacağı zaman $T(n)$ olsun. INORDER-TREE-WALK altağaçtaki tüm düğümleri ziyaret ettiğinden, $T(n) = \Omega(n)$ dir. Geriye $T(n) = O(n)$ i göstermek kalır.

Boş bir altağaçta INORDER-TREE-WALK küçük ve sabit bir zaman aldığından, ($x \neq \text{NIL}$ testi için), $c > 0$ olan herhangi bir sabit sayı için $T(0) = c$ olur.

$n > 0$ için, varsayın ki INORDER-TREE-WALK sol altağacının k düğümü, sağ altağacının $n - k - 1$ düğümü bulunan x düğümü üzerinde çağırılsın. INORDER-TREE-WALK(x) i çalıştırmak için gereken zaman, özyinelemeli çağrılarda harcanan zamana ek olarak, INORDER-TREE-WALK(x) in kod kısmını çalıştırmak için gereken zamanın üst sınırını yansıtan sabit sayı $d > 0$ için $T(n) \leq T(k) + T(n - k - 1) + d$ tarafından sınırlanır.

$T(n) \leq (c + d)n + c$ yi kanıtlayarak $T(n) = O(n)$ olduğunu göstermek için yerine koyma yöntemini kullanırız. $n = 0$ için $(c + d) \cdot 0 + c = T(0)$. $n > 0$ için;

$$\begin{aligned}
 T(n) &\leq T(k) + T(n - k - 1) + d \\
 &= ((c + d)k + c) + ((c + d)(n - k - 1) + c) + d \\
 &= (c + d)n + c - (c + d) + c + d \\
 &= (c + d)n + c
 \end{aligned}$$

kanıtı tamamlar. ■

Alıştırmalar

12.1-1

$\{1, 4, 5, 10, 16, 17, 21\}$ anahtarlarının kümesi için, 2, 3, 4, 5 ve 6 yüksekliğinde ikili arama ağacı çizin.

12.1-2

Min-bellek yığını özelliğiyle ikili-arama ağacı özelliği arasındaki fark nedir? (sf. 153) Min-bellek yığını özelliği $O(n)$ zamanda sıralı biçimde n düğümlü bir ağacın anahtarlarını yazdırmak için kullanılabilir mi? Öyleyse neden, değilse neden açıklayın.

12.1-3

Düzenli-ağaç adımı yapan özyinelemeli olmayan bir algoritma önerin. (*İpucu:* yardımcı veri yapısı olarak yığın kullanmak kolay çözümdür. Daha karmaşık fakat kullanışlı bir çözüm ise yığın kullanmadan iki işaretçiyi eşitlik için test edenidir.)

12.1-4

n düğümlü bir ağaç üstünde $\Theta(n)$ zamanda ön sıra ve son sıra ağaç yürüyüşü yapan özyinelemeli algoritmalar önerin.

12.1-5

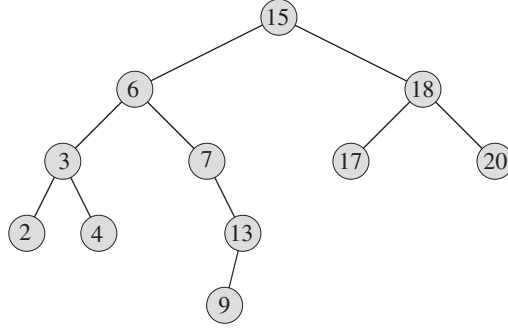
Karşılaştırma modelinde en kötü durumda, n elemanı sıralamak $\Omega(n \log n)$ zaman aldığından, n elemanlı rassal bir listeden ikili arama ağacı oluşturmak için kullanılan herhangi bir karşılaştırma tabanlı algoritma en kötü durumda $\Omega(n \log n)$ zaman aldığını gösteriniz.

12.2 Bir ikili arama ağacını sorgulama

İkili arama ağacında saklanan bir anahtarı sıklıkla ararız. SEARCH işleminin yanında, ikili arama ağaçları, MINIMUM, MAXIMUM, PREDECESSOR gibi sorguları da destekler. Bu bölümde, bu işlemleri inceleyeceğiz ve h yüksekliğinde bir ikili arama ağacında $O(h)$ zamanda her bir işlemin nasıl desteklendiğini göreceğiz.

Arama

İkili arama ağacında verilen anahtarlı bir düğümü aramak için şu prosedürü kullanırız. Ağacın kökünü gösteren gösteren işaretçi ve bir k anahtarı verilmiştir. TREE-SEARCH eğer varsa k anahtarlı bir düğümü gösteren işaretçi döndürür, yoksa NIL döndürür.



Şekil 12.2 İkili arama ağacı üzerinde sorgular. Ağaçtaki 13 anahtarını aramak için, kökten itibaren $15 \rightarrow 6 \rightarrow 7 \rightarrow 13$ yolunu takip ederiz. Ağaçtaki en düşük anahtar 2, kökten *sol* işaretçiyi takip ederek bulunabilir. En büyük anahtar 20, kökten *sağ* işaretçileri takip ederek bulunur. 15 anahtarlı düğümün ardılı 17 anahtarlı düğümdür. Çünkü 15 altağaçlı düğümdeki en küçük anahtardır. 13 anahtarlı düğümün sol altağacı yoktur ve bu yüzden onun ardılı, sol çocuğu da ata olan en düşük atasıdır. Bu durumda, 15 anahtarlı düğüm onun ardılıdır.

TREE-SEARCH(x, k)

```

1  if  $x == \text{NIL}$  veya  $k == x.key$ 
2      return  $x$ 
3  if  $k < x.key$ 
4      return TREE-SEARCH( $x.left, k$ )
5  else return TREE-SEARCH( $x.right, k$ )
  
```

Prosedür aramaya kökten başlar ve ağaçta aşağı doğru basit bir yol izler. Şekil 12.2 de gösterildiği gibi. Karşılaştığı her x düğümü için, anahtar k yi $x.key$ ile karşılaştırır. Eğer iki anahtar eşitse, arama biter. Eğer k , $x.key$ den küçük ise arama sağ altağaçta devam eder. Ağacın kökünden aşağı basit yol üzerindeki özyineleme süresince düğümler karşılaşır ve bu yüzden arama ağacının çalışma süresi $O(h)$ dir. h burada ağacın yüksekliğidir.

Bir **while** döngüsü içinde özyinelemeyi çözerek iteratif yöntemle bu prosedürü yeniden yazabiliriz. Birçok bilgisayarda, iteratif yöntem daha verimlidir.

ITERATIVE TREE-SEARCH (x, k)

ITERATIVE-TREE-SEARCH(x, k)

```

1  while  $x \neq \text{NIL}$  ve  $k \neq x.\text{key}$ 
2      if  $k < x.\text{key}$ 
3           $x = x.\text{left}$ 
4      else  $x = x.\text{right}$ 
5  return  $x$ 

```

Minimum ve maksimum

NIL ile karşılaşana kadar kökten itibaren *sol* çocuk işaretçilerini takip ederek anahtarı minimum olan bir elemanı ikili arama ağacında her zaman bulabiliriz. Şekil 12.2 de gösterildiği gibi. NIL olmamasını varsaydığımız verilen x düğümünde köklenmiş altağaçtaki minimum elemanı gösteren işaretçiyi aşağıdaki prosedür döndürür.

TREE-MINIMUM(x)

```

1  while  $x.\text{left} \neq \text{NIL}$ 
2       $x = x.\text{left}$ 
3  return  $x$ 

```

İkili-arama-ağacı özelliği, TREE-MINIMUM un doğru olduğunu garantiler. X düğümünün hiç sol altağacı yoksa, o zaman x in sağ altağacındaki her anahtar en az $x.\text{key}$ kadar büyük olduğundan, x de köklenmiş alt ağaçtaki minimum anahtar $x.\text{key}$ dir. X düğümünün sol altağacı varsa, o zaman sağ altağacında hiçbir düğüm $x.\text{key}$ den küçük olmadığından ve sol altağaçtaki her anahtar $x.\text{key}$ den büyük olmadığından, x de köklenmiş altağaçtaki en düşük anahtar $x.\text{left}$ te köklenmiş altağaçta bulunur.

TREE-MAXIMUM'un taslak kodu simetriktir:

TREE-MAXIMUM(x)

```

1  while  $x.\text{right} \neq \text{NIL}$ 
2       $x = x.\text{right}$ 
3  return  $x$ 

```

h yüksekliğindeki bir ağaçta her iki prosedür de $O(h)$ zamanda çalışır. Çünkü TREE-SEARCH da olduğu gibi karşılaşılan düğümlerin sırası kökten aşağı doğru basit bir yol oluşturur.

Ardıl ve öncül

İkili arama ağacında verilen bir düğüm için, sıralı ağaç yürüyüşü ile belirlenen sıralı biçimde onun ardılını bulmaya bazen ihtiyaç duyabiliriz. Eğer tüm anahtarlar

farklı ise, x düğümünün ardılı $x.key$ den büyük en küçük anahtarlı düğümdür. İkili arama ağacı yapısı, anahtarları hiç karşılaştırmadan düğümün ardılını belirlememize olanak tanır. Aşağıdaki prosedür ikili arama ağacında eğer varsa x düğümünün ardılı döndürür, eğer x , ağaçtaki en büyük anahtara sahipse NIL döndürür.

TREE-SUCCESSOR(x)

```

1  if  $x.right \neq \text{NIL}$ 
2      return TREE-MINIMUM( $x.right$ )
3   $y = x.p$ 
4  while  $y \neq \text{NIL}$  ve  $x == y.right$ 
5       $x = y$ 
6       $y = y.p$ 
7  return  $y$ 

```

TREE-SUCCESSOR için kodu iki duruma ayırırız. x düğümünün sağ altağacı boş değilse, x in ardılı x in altağacındaki en solda bulunan düğümdür. Onu da TREE-MINIMUM($x, right$) ı çağırarak ikinci satırda bulabiliriz. Örneğin, Şekil 12.2 deki 15 nolu düğümün ardılı, 17 nolu düğümdür.

Öte yandan, alıştırma 12.2-6 nın gösterdiği gibi, düğüm x in altağacı boşsa ve x in ardılı y ise, sol çocuğu x in atası olan y , x in en düşük atasıdır. Şekil 12.2 de 13 nolu düğümün ardılı 15 nolu düğümdür. y yi bulmak için x ten itibaren ağaçta ebeveyninin sol çocuğu olan bir düğümle karşılaşınca kadar ilerleriz. TREE-SUCCESSOR deki 3-7 satırları bununla ilgilenir.

h yüksekliğinde bir ağaçta TREE-SUCCESSOR in çalışma süresi $O(h)$ dir. Çünkü ya ağaçta yukarı doğru basit bir yol izleriz ya da aşağı doğru izleriz. TREE-PREDECESSOR prosedürü, TREE-SUCCESSOR in simetriğidir ve $O(h)$ zamanda çalışır.

Anahtarlar farklı olmadığı halde, herhangi bir x düğümün öncülü ve ardılı TREE-PREDECESSOR(x) ve TREE-SUCCESSOR(y) den dönen değerler olarak tanımlarız.

Özetle, aşağıdaki teoremi kanıtladık.

Teorem 12.2

h yüksekliğinde bir ikili arama ağacı için her bir işlemin $O(h)$ zamanda çalışacağı SEARCH, MAXIMUM, SUCCESSOR ve PREDECESSOR gibi dinamik küme işlemlerini geliştirebiliriz. ■

Alıştırmalar

12.2-1

Bir ikili arama ağacında 1 den 1000 e kadar sayılar olduğunu varsayın ve 363 ü arıyoruz. Aşağıdaki dizilerden hangisi incelenen düğümlerin dizisi olamaz?

- a. 2, 252, 401, 398, 330, 344, 397, 363.
- b. 924, 220, 911, 244, 898, 258, 362, 363.
- c. 925, 202, 911, 240, 912, 245, 363.
- d. 2, 399, 387, 219, 266, 382, 381, 278, 363.
- e. 935, 278, 347, 621, 299, 392, 358, 363.

12.2-2

TREE-MINIMUM ve TREE-MAXIMUM un özyinelemeli versiyonlarını yazın.

12.2-3

TREE-PREDECESSOR prosedürünü yazın.

12.2-4

Profesör Bunyan ikili arama ağacının önemli bir özelliğini keşfetti. İkili arama ağacında k anahtarını aramanın bir yaprakta sonlandığını varsayın. 3 kümeyi düşünün: A , arama yolunun solundaki anahtarlar; B , arama yolundaki anahtarlar; ve C , arama yolunun sağındaki anahtarlar. Profesör Bunyan, $a \in A$, $b \in B$ ve $c \in C$ gibi herhangi 3 anahtarın $a \leq b \leq c$ olması gerektiğini iddaa ediyor. Onun iddaasına en küçük muhtemel karşı iddaayı söyleyin.

12.2-5

İkili arama ağacındaki düğümün iki çocuğu varsa, ardılının sol çocuğu olmadığını ve öncülünün da sağ çocuğu olmadığını gösterin.

12.2-6

Anahtarları farklı olan ikili arama ağacı T yi düşünün. T deki x düğümünün sağ alt ağacı boş ise ve x in y adında bir ardılı var ise, y nin, sol çocuğu da x in atası olan düğüm olduğunu ve x in en küçük atası olduğunu gösterin (her bir düğümün kendisinin atası olduğunu unutmayın).

12.2-7

n düğümlü bir ikili arama ağacında gezen alternatif bir düzenli-ağaç-adımı yöntemi TREE-MINIMUM u çağırarak ve TREE-SUCCESSOR e $n - 1$ çağrı yaparak ağaçtaki en küçük elemanı bulur. Bu algoritmanın $O(n)$ zamanda çalıştığını ispatlayın.

12.2-8

h yüksekliğe sahip bir ikili arama ağacında hangi düğümden başlanıldığı fark etmeden, TREE-SUCCESSOR e k tane ardışık çağrı yapılmasının $O(h + k)$ zaman aldığını kanıtlayın.

12.2-9

T nin anahtarları farklı olan bir ikili arama ağacı olduğunu varsayın. x in yaprak düğüm olduğunu y nin ise onun ebeveyni olduğunu varsayın. $y.key$ in, ya $x.key$ den büyük en küçük anahtar olduğunu ya da $x.key$ den küçük en büyük anahtar olduğunu gösterin.

12.3 Araya sokma ve silme

Araya sokma ve silme işlemleri ikili arama ağacıyla temsil edilen dinamik kümenin değişmesine yol açar. Veri yapısı bu değişikliği yansıtmak için düzenlenmelidir. Fakat değişikliğin ikili-arama-ağacı özelliğine uyum sağlaması gerekir. Göreceğimiz gibi yeni eleman eklemek için ağacı değiştirmek göreceli olarak kolaydır. Fakat silme ile başa çıkmak biraz karmaşıktır.

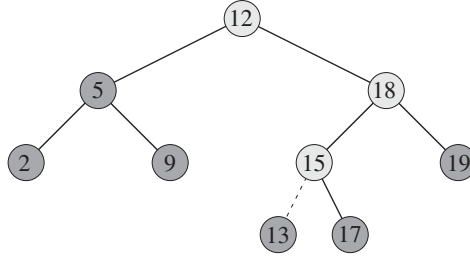
Araya sokma

İkili arama ağacı T ye yeni değer v yi eklemek için, TREE-INSERT prosedürünü kullanırız. Prosedür $z.key = v$, $z.left = \text{NIL}$ ve $z.right = \text{NIL}$ olan bir z düğümü alır. T yi ve z nin bazı özelliklerini z yi ağaçtaki uygun yere eklemek adına değiştirir.

TREE-INSERT (T, z)

```

1   $y = \text{NIL}$ 
2   $x = T.root$ 
3  while  $x \neq \text{NIL}$ 
4       $y = x$ 
5      if  $z.key < x.key$ 
6           $x = x.left$ 
7      else  $x = x.right$ 
8   $z.p = y$ 
9  if  $y == \text{NIL}$ 
10      $T.root = z$     // ağaç  $T$  boştu.
11 elseif  $z.key < y.key$ 
12      $y.left = z$ 
13 else  $y.right = z$ 
```



Şekil 12.3 İkili arama ağacına 13 anahtarlı bir eleman eklemek. Sönük gölgelendirilmiş düğümler kökten elemanın ekleneceği pozisyona kadar olan basit yolu gösterir. Noktalı çizgiler, elemanı eklemek için ağaca eklenmiş bağlantıyı gösterir.

Şekil 12.3, TREE-INSERT nin nasıl çalıştığını gösterir. TREE-SEARCH ve ITERATIVE-TREE-SEARCH prosedürlerindeki gibi, TREE-INSERT de ağacın kökünden başlar ve girdi elemanı z ile değiştirmek için NIL bulana kadar yol üzerinde aşağı iner. Prosedür, *izleyen işaretçi* y yi x in ebeveyni olarak tutar. Başlangıç değeri verildikten sonra, 3-7 satırları arasındaki **while** döngüsü bu iki işaretçinin ağacın aşağısına doğru gitmesine yol açar, x , NIL olana kadar $z.key$ ve $x.key$ karşılaşmasının sonucuna göre sağa veya sola götürür. Bu NIL, bizim z girdi elemanımızı eklemek izlediğimiz pozisyonu işgal eder. y izleyen işaretçisine ihtiyaç vardır. Çünkü z nin ait olduğu NIL i bulduğumuz anda, arama değiştirilecek olan düğümün bir aşama arkasına ilerler. 8-13 arası satırları, z yi eklemek için işaretçileri düzenler.

Ağaç aramalarındaki diğer ilkel işlemler gibi, TREE-INSERT prosedürü de h yüksekliğe sahip bir ağaçta $O(h)$ zamanda çalışır.

Silme

İkili arama ağacı T den nod z yi silmek için genel strateji 3 duruma sahiptir fakat göreceğimiz gibi, bu durumlardan biri biraz hilelidir.

- z nin hiç çocuğu yoksa, z nin ebeveyninde z çocuğunu NIL ile değiştirerek basitçe silebiliriz.
- z nin bir çocuğu varsa, z nin çocuğuyla z yi değiştirmek için z nin ebeveynini düzenleyerek z nin ağaçtaki pozisyonunu almak amacıyla bu çocuğu yükseltiriz.
- z nin iki çocuğu varsa, z nin sağ altağacında olması gereken z nin ardılı y yi buluruz ve y nin ağaçtaki z nin pozisyonunu almasını sağlarız. z nin orijinal sağ altağacının kalanı y nin yeni sağ altağacı olur ve z nin sol altağacı y nin yeni sol altağacı olur. Bu durum hileli olandır. Çünkü göreceğimiz gibi y , z nin sağ çocuğu olduğunda fark eder.

İkili arama ağacı T den verilen z düğümünü silen prosedür T ye ve z ye giden işaretçileri girdi olarak alır. Şekil 12.4 de gösterilen 4 durumu göz önüne alarak daha önce anlatılan 3 durumdan, durumları birazcık farklı düzenler.

- z nin sol çocuğu yoksa(Şeklin a kısmı), NIL olan veya olmayan z nin sağ çocuğuyla kendisini değiştiririz. z nin sağ çocuğu NIL olduğunda, bu durum z nin çocuğu olmama durumuyla çözülür. z nin sağ çocuğu NIL olmadığında, z nin sadece sağ çocuğu olduğu durum gibi çözülür.
- z nin bir sol çocuğu varsa, (Şeklin b kısmı) z yi onun sol çocuğuyla değiştiririz.
- Diğer türlü ise, z hem sağ hem sol çocuğa sahipse, z nin sağ altağacında bulunan ve çocuğu olmayan z nin ardılı y yi buluruz. (alıştırma 12.2-5) y yi güncel pozisyonunun dışına bağlamak ve onu z ile değiştirmek isteriz.
 - y , z nin sol çocuğu ise (kısım c), y nin sağ çocuğunu yalnız bırakarak, y yi z ile değiştiririz.
 - Diğer türlü, y z nin sağ altağacında, fakat z nin sağ çocuğu değildir (kısım d). Bu durumda, önce y yi kendi sağ çocuğuyla değiştiririz ve sonra y yi z ile değiştiririz.

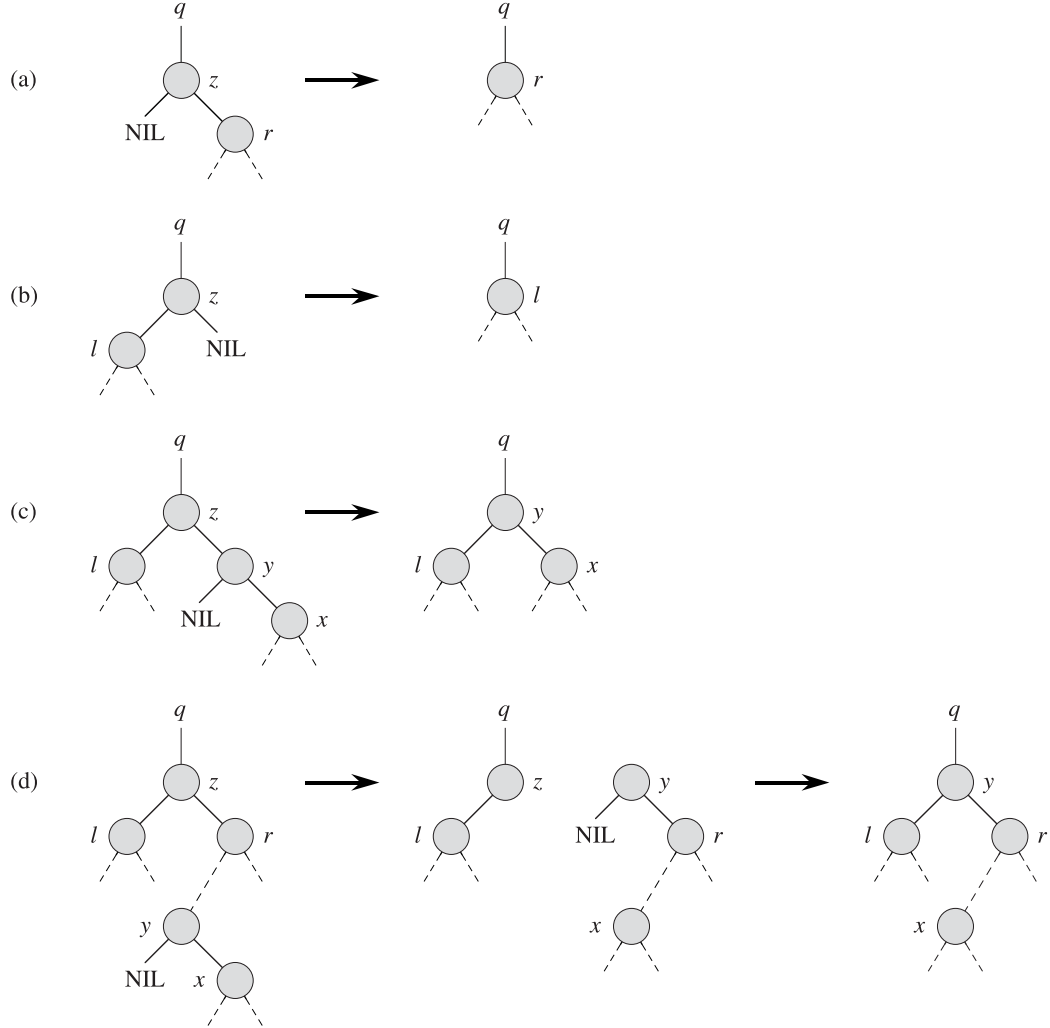
İkili arama ağacı içinde altağaçları oynatmak için, TRANSPLANT adında bir altprogram tanımlarız. Bir altağacı ebeveyninin çocuğu gibi başka bir altağaçla değiştirir. Düğüm v de köklenmiş bir altağaçla düğüm u da köklenmiş bir altağacı değiştirdiğinde, düğüm u nun ebeveyni düğüm v nin ebeveyni olur ve u nun ebeveyni v yi uygun çocuğu olarak alır ve TRANSPLANT sonlanır.

TRANSPLANT (T, u, v)

```

1  if  $u.p = \text{NIL}$ 
2       $T.root = v$ 
3  elseif  $u = u.p.left$ 
4       $u.p.left = v$ 
5  else  $u.p.right = v$ 
6  if  $v \neq \text{NIL}$ 
7       $v.p = u.p$ 
```

1-2 satırları u nun T nin kökü olduğu durumlarla ilgilenir. Diğer türlü, u ya ebeveyninin sol çocuğu ya da sağ çocuğudur. 3-4 satırları u sol çocuk ise $u.p.left$ i güncellemeyle ilgilenir ve 5. satır u sağ çocuk ise $u.p.right$ i günceller. V nin NIL olmasına izin veririz. 6-7 satırları v NIL değil ise $v.p$ yi günceller. TRANSPLANT $v.left$ ve $v.right$ i güncellemeye çalışmaz. Bunu yapmak veya yapmamak TRANSPLANT in çağırıcısının görevidir.



Şekil 12.4 İkili arama ağacından z düğümünü silme. Düğüm z kök q nun sol çocuğu ya da q nun sağ çocuğu olabilir. (a) düğüm z nin sol çocuğu yoktur. NIL olan veya olmayan r nin sağ çocuğu ile z yi değiştiririz. (b) düğüm z nin l adında sol çocuğu vardır, fakat sağ çocuğu yoktur. z yi l ile değiştiririz. (c) düğüm z nin iki çocuğu vardır. Sol çocuğu l düğümü, sağ çocuğu ise y nin ardılıdır ve y nin sağ çocuğu x düğümüdür. y nin sol çocuğunu l yaparak y yi z ile değiştiririz. Fakat x i y nin sağ çocuğu olarak bırakırız. (d) Düğüm z nin iki çocuğu vardır (sol çocuk l , sağ çocuk r) ve onun ardılı $y \neq r$, r de köklenmiş ağaç üzerinde bulunur. y yi kendi sağ çocuğu x ile değiştiririz ve y yi r nin ebeveyni yaparız. y yi q nun çocuğu, l nin ebeveyni yaparız.

Eldeki TRANSPLANT prosedürüyle, ikili arama ağacı t den düğüm z yi silen prosedür aşağıdadır:

```

TREE-DELETE( $T, z$ )
1  if  $z.left == \text{NIL}$ 
2      TRANSPLANT( $T, z, z.right$ )
3  elseif  $z.right == \text{NIL}$ 
4      TRANSPLANT( $T, z, z.left$ )
5  else  $y = \text{TREE-MINIMUM}(z.right)$ 
6      if  $y.p \neq z$ 
7          TRANSPLANT( $T, y, y.right$ )
8           $y.right = z.right$ 
9           $y.right.p = y$ 
10     TRANSPLANT( $T, z, y$ )
11      $Y.left = z.left$ 
12      $Y.left.p = y$ 

```

TREE-DELETE prosedürü görüldüğü gibi 4 durumu yürütür. 1-2 satırları z nin çocuğu olmadığı durumu inceler ve 3-4 satırları z nin sol çocuğunun olduğu fakat sağ çocuğunun olmadığı durumu inceler. 5-12 satırları z nin 2 çocuğu olduğu kalan iki durumla ilgilenir. 5. satır z nin ardılı olan y düğümünü bulur. Z nin sağ altağacı boş olmadığı için, onun ardılı en küçük anahtarlı altağactaki düğüm olmalıdır. Bu yüzden TREE-MINIMUM($z, right$) a çağrı yapılır. Daha önce fark ettiğimiz gibi, y nin sol çocuğu yoktur. y yi, şu anki pozisyonunun dışına bağlamak istiyoruz. Ağactaki z ile de değiştirilmelidir. y, z nin sağ çocuğu ise, 10-12 satırları y tarafından z yi ebeveyninin çocuğu olarak değiştirir ve y nin sol çocuğuyla z nin sol çocuğunu değiştirir. y, z nin sol çocuğu değilse, 7-9 satırları y nin sağ çocuğu ile y yi ebeveyninin çocuğu olarak değiştirir ve z nin sağ çocuğunu y nin sağ çocuğu yapar. 10-12 satırları y ile z yi ebeveyninin çocuğu olarak değiştirir ve y 'nin sol çocuğuyla z nin sol çocuğunu değiştirir.

TREE-DELETE in her satırı, TRANSPLANT a yapılan çağrılar dahil, 5. satırdaki TREE-DELETE ye yapılan çağrı hariç sabit zaman alır. Bu yüzden, TREE-DELETE h yüksekliğinde bir ağaçta $O(h)$ zamanda çalışır.

Özetle, aşağıdaki teoremi kanıtladık.

Teorem 12.3

h yüksekliğinde bir ağaçta her biri $O(h)$ zamanda çalışan INSERT ve DELETE gibi dinamik küme işlemlerini gerçekleştirebiliriz. ■

Alıştırmalar

12.3-1

TREE-INSERT prosedürünün özyinelemeli versiyonunu yazın.

12.3-2

Tekrarlı olarak farklı değerler ekleyerek bir ikili arama ağacı yarattığımızı varsayın. Ağaçtaki bir değeri aramak için incelenen düğümlerin sayısının, ağaca düğüm ilk eklendiğinde incelenen düğümlerin sayısından bir fazla olduğunu açıklayın.

12.3-3

Verilen n tane sayıyı içeren ikili arama ağacını oluşturarak bu sayıları sıralayabiliriz (elemanları tek tek eklemek için tekrarlı olarak TREE-INSERT yi kullanarak). Bu sayıları düzenli-ağaç-adımı ile yazdırabiliriz. Bu sıralama algoritmasının en iyi ve en kötü durumdaki çalışma zamanı nedir?

12.3-4

İkili arama ağacından x i ve ardından y yi silmenin ağacı önce y yi ardından x i silme durumuyla aynı şekilde bırakması açısından silme işlemi yer değiştirmeli midir? Öyleyse neden olduğunu söyleyin ya da bir karşı örnek verin.

12.3-5

Her bir x düğümünün, x in ebeveynini gösteren $x.p$ işaretçisini tutması yerine x in ardılını gösteren $x.succ$ işaretçisini tuttuğunu varsayın. Bu temsili kullanarak ikili arama ağacı T de SEARCH, INSERT ve DELETE için kabakod yazın. İkili arama ağacı T nin h yüksekliğinde olduğu durumda bu kodlar, $O(h)$ zamanda çalışmalı (*İpucu*: düğümün ebeveynini döndüren bir altprogram yazabilirsiniz).

12.3-6

TREE-DELETE deki z düğümünün 2 çocuğu varsa, y düğümünü onun ardılı seçmek yerine öncülünü seçebiliriz. Bunu yaparsak TREE-DELETE de başka ne değişiklikler yapmalıyız? Bazıları, öncüle ve ardıla eşit öncelik vermenin daha iyi deneysel performans göstereceğini savunur. Böyle adil bir strateji uygulamak için TREE-DELETE nasıl değiştirilebilir?

* 12.4 Rassal oluşturulmuş ikili arama ağacı

İkili arama ağacında her bir temel işlemin $O(h)$ zamanda çalıştığını gördük. h ağacın yüksekliğidir. İkili arama ağacının yüksekliği eleman eklendikçe ya da

çıkarıldıkça değişir. Örneğin, artan sıralamayla n tane eleman eklenirse, ağaç $n - 1$ boyunda bir zincir olur. Öte yandan, Alıştırma B.5-4 gösterir ki $h \geq \lceil \log n \rceil$ dir. Hızlı sıralamada olduğu gibi, ortalama durumun davranışı en iyi duruma, en kötü duruma olduğundan daha yakındır.

Maalesef, ikili arama ağacını yaratmak için araya sokma ve silme kullanıldığında, onun ortalama boyu hakkında çok az şey bilinir. Ağaç yalnızca araya sokmayla yaratıldığında, analiz daha izlenebilir hale gelir. Bu yüzden n anahtar üzerinde **rassal oluşturulmuş bir ikili arama ağacı** tanımlayalım. Bunu başlangıçta boş olan ağaca rastgele sırayla eleman eklemek gibi düşünebiliriz. Girdi anahtarlarının $n!$ permütasyonları eşit olarak benzerdir (Alıştırma 12.4-3 bu fikrin n anahtar üzerinde her bir ikili arama ağacının eşit şekilde benzer olduğu varsayımından farklı olduğunu göstermemizi ister). Bu bölümde, aşağıdaki teoremi kanıtlayacağız.

Teorem 12.4

n farklı anahtar üzerinde rassal oluşturulmuş bir ikili arama ağacının beklenen yüksekliği $O(\log n)$ dir.

Kanıt Rassal oluşturulmuş ikili arama ağacının yüksekliğini ölçmeye yardım eden 3 rassal değişken tanımlayarak başlayalım. Rassal oluşturulmuş ikili arama ağacının yüksekliğini X_n , **üstel yükseklik** $Y_n = (2^{X_n})$ olarak tanımlayalım. n anahtar üzerinde ikili arama ağacı oluştururken, bir anahtarı kök olarak seçeriz ve n anahtarlı bir küme içinde bu anahtarın **rankını** tutan rassal değişkeni R_n olarak tanımlarız. Yani, anahtarlar kümesi sıralanmış olsa bu anahtarın işgal edeceği pozisyonu R_n tutar. Kümenin $[1, 2, \dots, n]$ herhangi bir elemanına eşit şekilde benzer değere sahip olan R_n dir. $R_n = i$ ise, kökün sol altağacı, $i - 1$ anahtar üzerinde rassal oluşturulmuş ikili arama ağacıdır ve sağ altağaç $n - 1$ anahtarda rassal oluşturulmuş ikili arama ağacıdır. İkili ağacın yüksekliği kökün iki altağacının büyük olanının boyundan 1 fazla olduğu için, ikili ağacın üstel yüksekliği, iki alt ağacının büyüğünün üstel yüksekliğinin iki katıdır. $R_n = i$ olduğunu biliyorsak,

$$Y_n = 2 \cdot \max(Y_{i-1}, Y_{n-i})$$

elde edilir. Temel şartlar olarak, $Y_1 = 1$ dir. Çünkü 1 düğümlü ağacın üstel yüksekliği $2^0 = 1$ ve uygunluk için $Y_0 = 0$ tanımlarız.

$$Z_{n,i} = I\{R_n = i\}$$

olan $Z_{n,1}, Z_{n,2}, \dots, Z_{n,n}$ rassal gösterge değişkenleri tanımlansın. $R_n \in \{1, 2, \dots, n\}$ nin herhangi bir elemanına eşit şekilde benzer olduğundan

$$\Pr\{R_n = i\} = 1/n, i = 1, 2, \dots, n \text{ elde edilir ve bu yüzden Lemma 5.1 e uyarak} \\ E[Z_{n,i}] = 1/n \text{ dir.} \quad (12.1)$$

$Z_{n,i}$ nin sadece bir değeri 1 olduğu ve diğerleri 0 olduğu için,

$$Y_n = \sum_{i=1}^n Z_{n,i} (2 \cdot \max(Y_{i-1}, Y_{n-i}))$$

dir. $E[Y_n]$ nin n de bir polinom olduğunu göstereceğiz, en sonunda $E[X_n] = O(\log n)$ olduğunu göstereceğiz.

Rassal gösterge değişkeni $Z_{n,i} = I\{R_n = i\}$ nin, Y_{i-1} ve Y_{n-i} nin değerlerinden bağımsız olduğunu iddaa ediyoruz. $R_n = i$ seçerek, sol altağaç (üstel yüksekliği Y_{i-1}) dereceleri i den az olan $i-1$ anahtar üzerinde rassal oluşturulmuştur. Bu altağaç, $i-1$ anahtar üzerinde rassal oluşturulmuş herhangi bir ikili ağaç gibidir. Taşıdığı anahtar sayıları dışında, altağacın yapısı $R_n = i$ tercihinden hiç etkilenmez ve bu yüzden Y_{i-1} ve $Z_{n,i}$ rassal değişkenleri bağımsızdır. Benzer olarak, sağ altağaç, üstel yüksekliği Y_{n-i} olan, seviyeleri i den büyük olan $n-1$ anahtar üzerinde rassal oluşturulmuştur. Yapısı R_n değerinden bağımsızdır ve bu yüzden rassal değerler Y_{n-i} ve $Z_{n,i}$ bağımsızdır. Böylelikle,

$$\begin{aligned} E[Y_n] &= E \left[\sum_{i=1}^n Z_{n,i} (2 \cdot \max(Y_{i-1}, Y_{n-i})) \right] \\ &= \sum_{i=1}^n E[Z_{n,i} (2 \cdot \max(Y_{i-1}, Y_{n-i}))] \quad (\text{beklenti doğrusallığından}) \\ &= \sum_{i=1}^n E[Z_{n,i}] E[2 \cdot \max(Y_{i-1}, Y_{n-i})] \quad (\text{bağımsızlıktan}) \\ &= \sum_{i=1}^n \frac{1}{n} \cdot E[2 \cdot \max(Y_{i-1}, Y_{n-i})] \quad (\text{denklem (12.1)}) \\ &= \frac{2}{n} \sum_{i=1}^n E[\max(Y_{i-1}, Y_{n-i})] \quad (\text{denklem (C.22)}) \\ &\leq \frac{2}{n} \sum_{i=1}^n (E[Y_{i-1}] + E[Y_{n-i}]) \quad (\text{Alistırma C13-4}) \end{aligned}$$

$E[Y_0], E[Y_1], \dots, E[Y_{n-1}]$ den her bir terim en son toplamada bir kez $E[Y_{i-1}]$ olarak bir kez de $E[Y_{n-i}]$ olarak iki kez gözüktüğünden,

$$E[Y_n] \leq \frac{4}{n} \sum_{i=0}^{n-1} E[Y_i] \quad (12.2)$$

özyinelemesini elde ederiz.

Yerine koyma yöntemini kullanarak, n tane tüm pozitif sayılar için, (12.2) özyinelemesinin çözümünün şu olduğunu göstereceğiz:

$$E[Y_n] \leq \frac{1}{4} \binom{n+3}{3}.$$

Bunu yaparak, şu eşitliği kullanacağız:

$$\sum_{i=0}^{n-1} \binom{i+3}{3} = \binom{n+3}{4}. \quad (12.3)$$

(Alıştırma 12.4-1 sizden bu eşitliği kanıtlamanızı ister).

Temel şartlar için, sınırların $0 = Y_0 = E[Y_0] \leq (1/4)\binom{3}{3} = 1/4$ ve $1 = Y_1 = E[Y_1] \leq (1/4)\binom{1+3}{3} = 1$ olduğunu fark ederiz. Tümevarım yöntemi için,

$$\begin{aligned} E[Y_n] &\leq \frac{4}{n} \sum_{i=0}^{n-1} E[Y_i] \\ &\leq \frac{4}{n} \sum_{i=0}^{n-1} \frac{1}{4} \binom{i+3}{3} \quad (\text{tümevarım hipotezinden}) \\ &= \frac{1}{n} \sum_{i=0}^{n-1} \binom{i+3}{3} \\ &= \frac{1}{n} \binom{n+3}{4} \quad (\text{denklem (12.3)}) \\ &= \frac{1}{n} \cdot \frac{(n+3)!}{4! (n-1)!} \\ &= \frac{1}{4} \cdot \frac{(n+3)!}{3! n!} \\ &= \frac{1}{4} \binom{n+3}{3} \end{aligned}$$

dir. $E[Y_n]$ i sınırladık, fakat bizim en son amacımız $E[X_n]$ i sınırlamaktır. Alıştırma 12.4-4 ün sizden göstermenizi istediği gibi, $f(x) = 2^x$ fonksiyonu dışbükeydir. (sayfa 1199 a bakınız). Bu yüzden,

$$\begin{aligned} 2^{E[X_n]} &\leq E[2^{X_n}] \\ &= E[Y_n] \end{aligned}$$

olduğunu belirten (C.26) Jensen eşitsizliğini uygulayabiliriz. Aşağıdaki gibidir:

$$2^{E[X_n]} \leq \frac{1}{4} \binom{n+3}{3}$$

$$\begin{aligned}
&= \frac{1}{4} \cdot \frac{(n+3)(n+2)(n+1)}{6} \\
&= \frac{n^3 + 6n^2 + 11n + 6}{24}.
\end{aligned}$$

Her iki tarafın logaritmasını alırsak $E[X_n] = O(\log n)$ dir. ■

Alıştırmalar

12.4-1

(12.3) denklemini kanıtlayın.

12.4-2

Ağaçtaki düğümün ortalama derinliği $\Theta(\log n)$ olan n düğümlü bir ikili arama ağacı tasarlayın, fakat ağacın yüksekliği $w(\log n)$ olsun. Bir düğümün ortalama derinliği $\Theta(\log n)$ olan n düğümlü ikili arama ağacının yüksekliğinde bir asimptotik üst sınır verin.

12.4-3

n anahtarlı rassal seçilmiş ikili arama ağacı fikrinin, her bir n anahtarlı ikili arama ağacına eşit şekilde benzer seçilir. Bu bölümde verilen rassal oluşturulmuş ikili arama ağacı fikrinden farklı olduğunu gösterin (*İpucu: $n = 3$ olduğundaki olasılıkları listeleyin*).

12.4-4

$f(x) = 2^x$ fonksiyonunun dışbükey olduğunu gösterin.

12.4-5 *

n farklı girdi üzerinde işleyen RANDOMIZED-QUICKSORT u düşünün. Her $k > 0$ sabit sayısı için, $n!$ girdi permütasyonlarının $O(1/n^k)$ sının $O(n \log n)$ zamanda çalışacağını kanıtlayın.

Problemler

12-1 Eşit anahtarlara sahip ikili arama ağaçları

Eşit anahtarlar ikili arama ağaçları uygulama sorunu doğurur.

- a. Başlangıçta boş olan bir ikili arama ağacına n tane eşit anahtar eklendiğinde TREE-INSERT nin asimptotik performansı ne olur?

TREE-INSERT yi, $z.key = x.key$ olduğunu görmek için önce 5. satırı test etmeyi ve $z.key = y.key$ olduğunu görmek için de önce 11. satırı test etmeyi öneririz.

Eşitlik sağlanırsa, aşağıdaki stratejilerden birini uyguluyoruz. Her bir strateji için, başlangıçta boş olan ikili arama ağacına n tane aynı eleman eklemenin asimptotik performansını bulun. (Stratejiler 5. satırda açıklanır, z ve x i karşılaştırırız. 11. satırdaki stratejilere varmak için y yi x ile değiştiririz.)

- b.* Düğüm x de bir boolean bayrak $x.b$ tutun, $x.b$ nin değerine bağlı olarak x i ya $x.left$ e ya da $x.right$ a eşitleyin. Bu değer, x ile aynı anahtarlı bir düğüm eklerken x i her ziyaret ettiğimizde FALSE ve TRUE arasında değişir.
- c.* x ile aynı anahtara sahip düğümlerin listesini tutun ve z yi bu listeye ekleyin.
- d.* x i rastgele ya $x.left$ e ya da $x.right$ a ekleyin (En kötü durum performansını bulun ve beklenen çalışma süresini türetin).

12-2 Taban Ağaçları

Her bir a_i ve b_j nin bazı sıralı karakter kümesinde olduğu verilen $a = a_0a_1\dots a_p$ ve $b = b_0b_1\dots b_q$ iki karakter katarı üzerinde, a karakter katarının b karakter katarından **sözlüksel olarak daha küçük** olduğunu söyleriz.

1. Her $i = 0, 1, \dots, j-1$ için $a_i = b_i$ ve $a_j < b_j$ olacak şekilde $0 \leq j \leq \min(p, q)$ olan bir j sayısı vardır ya da
2. tüm $i = 0, 1, \dots, p$ için $p < q$ ve $a_i = b_i$ dir.

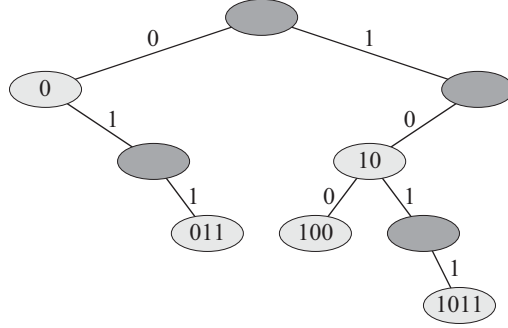
Örneğin, a ve b bit karakter katarları ise, 1. kuralla $10100 < 10110$ ($j = 3$ olduğunda) ve 2. kuralla $10100 < 101000$ dir. Bu sıralama İngilizce sözlüklerde kullanılaninkine benzerdir.

Şekil 12.5 te gösterilen **taban ağacı** veri yapısı 1011, 10, 011, 100 ve 0 bit karakter dizilerini saklar. $a = a_0a_1\dots a_p$ anahtarını ararken, $a_i = 0$ ise i derinlikli düğümde sola, $a_i = 10$ ise sağ gideriz. Boyutlarının toplamı n olan farklı bit karakter dizilerine sahip bir kümeye S diyelim. $\Theta(n)$ zamanda S kümesini sözlüksel olarak sıralamak için taban ağacını nasıl kullanacağımızı gösterin. Şekil 12.5 teki örnekteki gibi, sıralamanın çıktısı 0, 011, 10, 100, 1011 dizisi olmalıdır.

12-3 Rassal oluşturulmuş bir ikili arama ağacında ortalama düğüm derinliği

Bu sorunda, n düğümlü rassal oluşturulmuş bir ikili arama ağacında ortalama düğüm derinliğinin $O(\log n)$ olduğunu kanıtlarız. Bu sonuç, Teorem 12.4 tekinden zayıf olsa dahi, kullandığımız teknik, ikili arama ağacı oluşturmakla Kısım 7.3 teki RANDOMIZED-QUICKSORT u uygulamak arasındaki şaşırtıcı benzerliği gösterir.

Bir T ikili ağacının **toplam yol uzunluğu** $P(T)$, yi tüm x düğümleri üzerinde, $d(x, T)$ ile gösterdiğimiz x düğümünün derinliğinin toplamı olarak tanımlarız.



Şekil 12.5 1011, 10, 011, 100 ve 0 bit katarlarını saklayan bir taban ağacı. Her bir düğümün anahtarını, kökten bu düğüme giden basit yolu takip ederek belirleyebiliriz. Bu yüzden, düğümlerde anahtarları saklamaya gerek yoktur. Burdaki anahtarlar görsel amaçlar için görünür. Anahtarların uyduğu düğümler ağaçta değilse, bu düğümler koyu bir biçimde gölgelenmiştir. Bu tip düğümler yalnızca diğer düğümlere yol oluşturmak için yer alırlar.

a. T deki bir düğümün ortalama derinliğinin

$$\frac{1}{n} \sum_{x \in T} d(x, T) = \frac{1}{n} P(T)$$

olduğunu tartışın. Bu yüzden, $P(T)$ nin beklenen değerinin $O(n \lg n)$ olduğunu göstermek isteriz.

b. T_L ve T_R nin T ağacının sol ve sağ altağaçlarını ifade ettiğini varsayalım. T nin n tane düğümü varsa, o zaman

$$P(T) = P(T_L) + P(T_R) + n - 1$$

olduğunu tartışın.

c. $P(n)$ nin, n düğümlü rassal oluşturulmuş bir ikili arama ağacının ortalama toplam yol uzunluğu olduğunu varsayalım.

$$P(n) = \frac{1}{n} \sum_{i=0}^{n-1} (P(i) + P(n-i-1) + n-1)$$

olduğunu gösterin

d. $P(n)$ nin

$$P(n) = \frac{2}{n} \sum_{k=1}^{n-1} P(k) + \Theta(n)$$

olarak nasıl yeniden yazıldığını gösterin.

e. Problem 7.3 te verilen hızlı sıralamanın rassal versiyonunun alternatif analizini hatırlayarak, $P(n) = O(n \lg n)$ olduğu sonucuna varın.

Hızlı sıralamanın her bir özyinelemeli yürütmesinde, sıralanan elemanlar kümesi parçasına rassal bir pivot eleman seçeriz. İkili arama ağacının her düğümü, o düğümde köklenmiş bir altağacı elemanlar kümesine ayırır.

f. Elemanların kümesinin sıralamak için yapılan karşılaştırmaların, bir ikili arama ağacına eleman eklemek için yapılan karşılaştırmalarla tamamen aynı olduğu bir hızlı sıralama uygulaması tarif edin (Karşılaştırmaların yapılış sırası farklılaşabilir, fakat aynı karşılaştırmalar gerçekleşmelidir).

12-4 Farklı ikili ağaçların sayısı

b_n nin n düğümlü farklı ikili ağaç sayısını temsil ettiğini varsayalım. Bu problemde, b_n için asimptotik bir tahminin yanında bir formül bulacaksınız.

a. $b_0 = 1$ ve $n \geq 1$ için

$$b_n = \sum_{k=0}^{n-1} b_k b_{n-1-k}$$

olduğunu gösterin.

b. Problem 4-4 deki üreteç fonksiyonunun tanımına dayanarak, $B(x)$

$$B(x) = \sum_{n=0}^{\infty} b_n x^n$$

üreteç fonksiyonu olsun.

$B(x) = xB(x)^2 + 1$ olduğunu ve bu yüzden $B(x)$ in kapalı formunun

$$B(x) = \frac{1}{2x} (1 - \sqrt{1 - 4x})$$

biçiminde ifade edildiğini gösterin.

x de hesaplanmış f nin k ıncı türevinin $f^{(k)}(x)$ olarak ifade edildiği $x = a$ noktası çevresindeki $f(x)$ in **Taylor açılımı**

$$f(x) = \sum_{k=0}^{\infty} \frac{f^{(k)}(a)}{k!} (x - a)^k$$

biçiminde verilir.

$$\mathbf{c.} \quad b_n = \frac{1}{n+1} \binom{2n}{n}$$

(n inci **Katalan Sayısı**) $x = 0$ etrafındaki $\sqrt{1-4x}$ in Taylor açılımını kullanarak yukarıdaki denklemi gösterin. (Dilerseniz, Taylor açılımı kullanmak yerine, herhangi bir k sayısı ve n reel sayısı için, integral olmayan n üsleminde binom açılımının bir genellemesini kullanabilirsiniz. $k \geq 0$ ise, $\binom{n}{k}$ yı $n(n-1)\cdots(n-k+1)/k$ olarak, değilse 0 olarak ifade ederiz.

d.

$$b_n = \frac{4^n}{\sqrt{\pi n^{3/2}}} (1 + O(1/n))$$

olduğunu gösterin.

Bölüm notları

Knuth [211] birçok çeşitlemeyle beraber basit ikili arama ağaçlarının iyi tartışmalarını da içerir. İkili arama ağaçları, 1950 lerin sonlarında birçok insan tarafından bağımsız olarak keşfedilmiş görünür. Taban ağaçları sıklıkla, "*retrieval*" kelimesinin orta harflerinden gelen "*tries*" olarak çağrılır. Knuth [211] onları da tartışır.

Birçok metin, bu kitabın ilk iki düzenlemesi de dahil, iki çocuklu düğümleri ikili arama ağacından silmek için biraz daha basit yöntemler sunar. z düğümünü ardılı y ile değiştirmek yerine, y düğümünü siler fakat onun anahtarını ve uydu verisini z düğümüne kopyalarız. Bu yaklaşımın kötü yanı ise güncel olarak silinen düğüm, silme prosedürüne gönderilen düğüm olmayabilir. Programın diğer bileşenleri ağaçtaki düğümleri gösteren işaretçileri saklarsa, yanlışlıkla silinen düğümlere işaretçi tutmuş olabilirler. Kitabın bu basımında sunulan silme yöntemi biraz daha karmaşık olsa da z düğümünü silmek için çağrılan prosedürün yalnızca z düğümünü sileceğini garanti eder.

Kısım 15.5, ağacı oluşturmadan önce arama frekanslarını bildiğimizde nasıl optimal bir ikili arama ağacı oluşturacağımızı gösterecektir. Yani, herbir anahtarın arama frekansı ve ağaçtaki anahtarlar arasına düşen değerlerin arama frekansları verildiğinde, en az sayıda düğümleri inceleyen bu frekansları takip eden bir arama kümesi için bir ikili arama ağacı inşa ederiz.

Rassal bir şekilde oluşturulmuş bir ikili arama ağacının beklenen yüksekliğini sınırlayan Kısım 12.4 teki kanıt Aslam[24] sayesinde. Martinez ve Roura [243] ikili arama ağacında araya sokma ve silme işlemlerinin herbirinin sonucunun rassal bir ikili arama ağacı olduğu rassal algoritmalar verirler. Yine de, onların rassal ikili arama ağacı tanımı birazcık farklılaşır.

Bölüm 12 de gösterildiği gibi h yüksekliğine sahip bir ikili arama ağacı $O(h)$ zamanında çalışan bazı basit dinamik-küme işlemlerini desteklemektedir. Bunlar, SEARCH, PREDECESSOR, SUCCESSOR, MINIMUM, MAXIMUM, INSERT ve DELETE gibi ve benzeri işlemlerdir. Bunun sonucu olarak eğer arama ağacımızın yüksekliği ne kadar az ise bu küme işlemleri de bir o kadar hızlı çalışmaktadır. Ağacın yüksekliği fazla olursa bu işlemler beklenen verimi sağlayamayacak ve hatta bağlı listeye sahip küme işlemleri daha yavaş çalışacaklardır. Kırmızı-siyah ağaçlar bu işlemler en kötü durumda dahi $O(\lg n)$ zamanında çalışmasını garantileyen bazı dengeli arama ağaçlarından (Balanced Search Tree) bir tanesidir.

13.1 Kırmızı-siyah ağaçların özellikleri

Kırmızı-siyah ağaçlar her düğüm de fazladan bir bit saklayabilen ikili arama ağaçlarıdır. Bu bit bulunduğu düğümün **rengini** yani kırmızı ya da siyah olduğunu belirten bir değişkendir. Kırmızı-siyah ağaçlarda kökten (root) yapraklara (leaf) kadar olan yolların hiçbirisi bir diğerinden ikiden fazla düğüm içeremez ve bu sayede kırmızı siyah ağaçlar diğer ikili ağaçlara göre nazaran daha **dengeli** bir şekle gelirler.

Ağacın her düğümü *color*, *key*, *left*, *right* ve *p* (renk, anahtar, sol, sağ ve *p*) adında özellikler barındırır. Eğer bir düğümün ebeveyn ya da çocuğu yoksa bunlara karşılık gelen işaretçi (pointer) değişkenlerine karşılık NIL (*null*) değeri atanır. Bu NIL düğümlerini ikili arama ağacının yaprak düğümlerini gösteren pointer değişkenler olarak değerlendirmeliyiz ve ayrıca anahtar ilişkili düğümleri de ikili arama ağacının iç düğümleri olarak değerlendirmeliyiz.

Kırmızı-siyah ağaçlar aşağıdaki **kırmızı-siyah özellikleri** sağlayan ikili ağaçlardır.

1. Her düğüm siyah ya da kırmızı olmalıdır.
2. Kök her zaman siyahtır.
3. Her yaprak (NIL) siyah renklidir.
4. Eğer düğüm kırmızı ise her iki çocuğu da siyah renkli olmalıdır
5. Her düğüm için, o düğümün soyundan gelen ve yapraklara kadar ilerleyen tüm basit yollardaki siyah düğüm sayısı eşit olmalıdır.

Şekil 13.1(a) da kırmızı-siyah ağaçlara bir örnek gösterilmiştir.

Kırmızı-siyah ağaç kodunda sınırlayıcı durumlara çözüm getirebilmek için nöbetçileri (sentinel) NIL olarak gösteririz (sayfa 238). T kırmızı siyah ağacında ki *T.nil* nöbetçisi diğer sıradan düğümlerde olan değişkenlere sahip bir nesnedir. Bu renk niteliği SIYAHTIR ve diğer niteliklerde *-p*, *left*, *right* ve *key*- isteğe bağlı değerler alırlar. Şekil 13.1(b) nin gösterdiğine göre NIL gösteren tüm işaretçiler nöbetçi *T.nil* ler ile değiştirilmiştir.

Nöbetçileri *x* düğümünün NIL çocuklarına ebeveyni *x* olan normal bir düğüm gibi davranabilmek için kullanıyoruz. Ağaçlardaki NIL değerlerini her seferinde ayrı bir nöbetçi tanımlamak yerine kullanıyoruz çünkü diğer yöntem ile her NIL değerlerinin iyi tanımlanmış ebeveynleri gereksiz alan kullanımına neden olurlar. Bunun yerine tüm NIL değerlerini temsil etmek adına tek bir *T.nil* nöbetçisi kullanılır. Nöbetçinin *p*, *left*, *right* ve *key* değişkenlerinin değerleri önemsizdir bu sayede bu değişkenlere kolaylık olsun diye işlemin seyri sırasında atama yapılabiliriz.

Genellikle anahtar değerlerini tuttukları sürece kırmızı-siyah ağaçlardaki iç düğümler için olan ilgimizi sınırlı tutarız. Unutmamak gerekir ki Şekil 13.1(c) de de olduğu gibi bu başlıkta kırmızı-siyah ağaçları çizerken yaprakları göz ardı edeceğiz.

Her hangi düğümden aşağıya doğru inen basit yoldaki siyah renkli düğüm sayısını **siyah-yükseklik (black-height)** olarak adlandırırız ancak bu düğüm sayısına yolun başlangıcındaki düğüm dahil değildir, siyah yüksekliği $bh(x)$ diye tanımlarız. Özellik 5 e göre siyah-yükseklik kavramı gayet iyi tanımlanmıştır, herhangi bir düğümden çıkan ve aşağıya doğru giden tüm yollardaki siyah düğüm sayısı aynı olmalıdır. Kırmızı-siyah ağaçların siyah-yüksekliğini kökün siyah yüksekliği olarak tanımlarız.

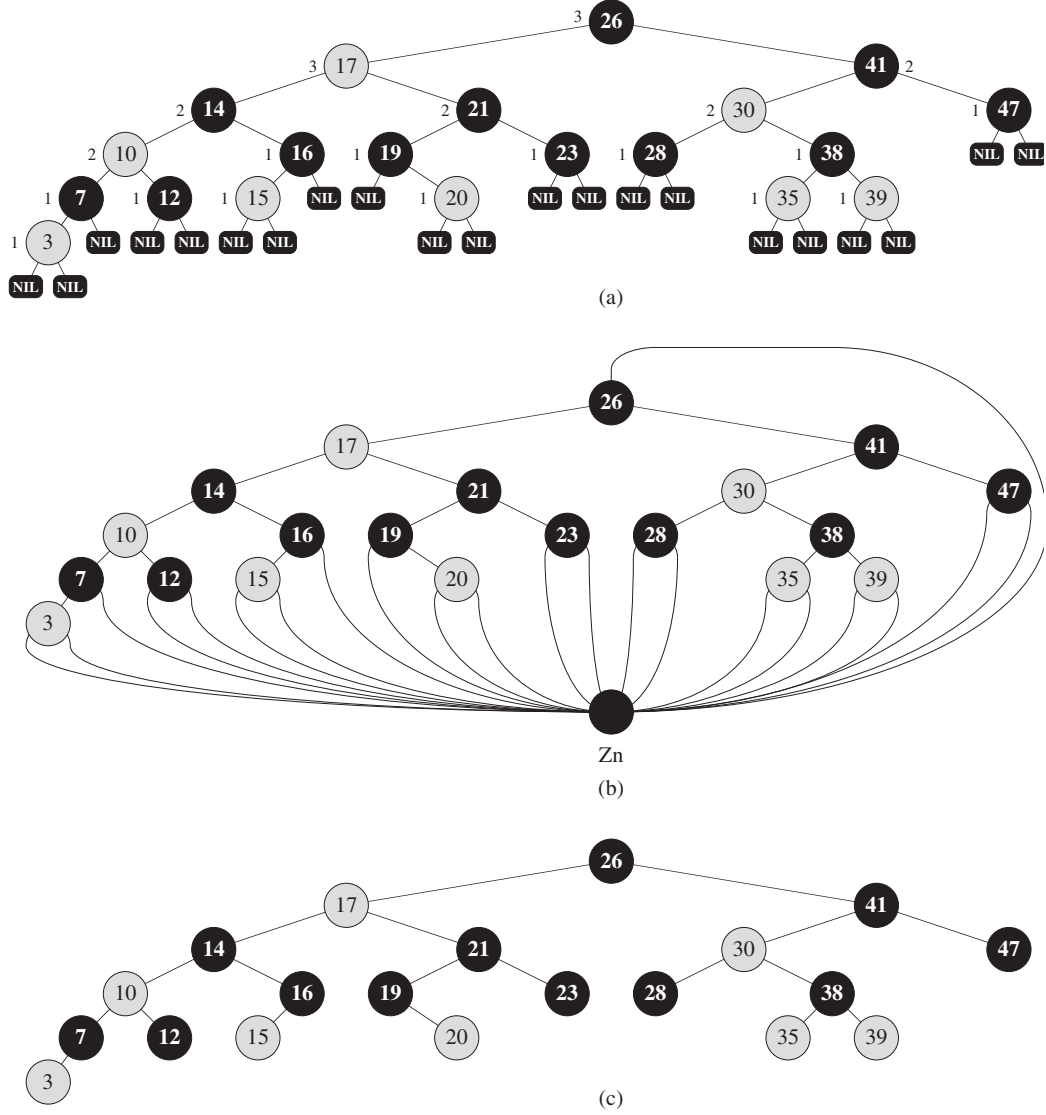
Şimdiki lemma neden kırmızı-siyah ağaçların iyi bir arama ağacı olduğunu gösterecektir.

Lemma 13.1

İçinde n düğüm bulunduran kırmızı-siyah ağacın yüksekliği en fazla $2\lg(n + 1)$ olacaktır.

Kanıt Herhangi bir düğümden çıkan altağacın en az $2^{bh(x)} - 1$ kadar düğüme sahip olduğunu göstererek başlayacağız. Bu iddiayı x in yüksekliği üzerinden tümevarım yöntemiyle kanıtlayacağız. Eğer x in yüksekliği sıfır ise, x yaprak (*T.nil*) olmalıdır ve x in kök olarak kullanıldığı altağaçlarda en az $2^{bh(x)} - 1 = 2^0 - 1 = 0$ kadar düğüm bulunur. Kanıt için bu adım da x i pozitif bir yüksekliği olan 2 çocuklu bir düğüm olarak düşünelim. İki çocuğunda siyah-yüksekliği ya $bh(x)$ ya da $bh(x) - 1$ olarak düşünülür ve bu durum sırasıyla renklerinin kırmızı ya da siyah olmasına bağlıdır. x in çocuğunun yüksekliği x in kendisinin yüksekliğinden az olduğu sürece, tümevarım hipotezinin sonucu olarak diyebiliriz ki, her çocuk en az $2^{(bh(x)-1)} - 1$ düğüme sahiptir. Bunun sonucu olarak x in kök olduğu alt ağacın bulundurduğu düğüm sayısı $(2^{(bh(x)-1)} - 1) + (2^{(bh(x)-1)} - 1) + 1 = 2^{bh(x)} - 1$ denktir ve bu da teoremi kanıtlar niteliktedir.

İddianın kanıtını tamamlamak için h ı ağacın yüksekliği olarak kabul edelim. Özellik 4 e göre kökten herhangi bir yapağa olan tüm basit yollardaki düğümler-



Şekil 13.1 Şekilde kırmızı-siyah ağacın siyah düğümleri karartılmış ve kırmızı düğümleri gölgeleştirilmiştir. Kırmızı-siyah ağaçtaki her düğüm kırmızı ve ya siyah renklidir, kırmızı düğümlerin çocuklarının ikisi de siyah renklidir ve bir düğümden başlayan ve yaprağa giden her basit yoldaki siyah düğüm sayısı eşittir. **(a)** Tüm yapraklar NIL olarak gösterilmiş ve siyah renklidir. Tüm yaprak olmayan (non-NIL) düğümler siyah-yükseklikleriyle birlikte gösterilmiştir. NIL düğümlerin siyah yüksekliği sıfırdır. **(b)** Aynı kırmızı-siyah ağaç ancak tüm NIL düğümler tek bir nöbetçi *T.nil* ile yer değiştirilmiştir. Kökün ebeveyni de nöbetçidir. **(c)** Aynı kırmızı-siyah ağaç ancak yapraklar ve kökün ebeveyni tamamiyle çıkarılmıştır. Bu çizim biçimini kitabın bu bölümü için aklımızda tutmalıyız.

rin yarısı kök dâhil olamadan siyah olmalıdır. Kökün siyah-yüksekliği en azından $h/2$ olmalıdır bundan dolayı da

$n \geq 2^{h/2} - 1$ olacaktır.

1 i sol tarafa taşırsak ve iki tarafında logaritmasını alırsak sonuç olarak $\lg(n + 1) \geq h/2$ ya da $h \leq 2\lg(n + 1)$ elde ederiz. ■

Bu lemmanın şimdiki sonuçlarına göre dinamik küme işlemlerini SEARCH, MINIMUM, MAKSIMUM, SUCCESSOR ve PREDECESSOR uygulayacak olursak eğer $O(\lg n)$ zamanda gerçekleşecektir, h yüksekliğindeki ikili arama ağaçlarında ise bu işlemler (Bölüm 12 de gösterilen) $O(h)$ zamanda gerçekleşir. Buna rağmen Bölüm 12 de gösterilen TREE-INSERT ve TREE-DELETE işlemleri $O(\lg n)$ zamanda çalışacaktır ama ikili arama ağaçları yerine kırmızı-siyah ağaçları girdi olarak verirken çalışacağını garanti edemeyiz. Ancak bu iki işlemin $O(\lg n)$ zamanda nasıl çalışacağını Kısım 13.3 ve 13.4 te göstereceğiz.

Alıştırmalar

13.1-1

Anahtarları $\{1, 2, \dots, 15\}$ olan ve yüksekliği 3 olan ikili arama ağacını Şekil 13.1(a) da gösterilen şekilde çiziniz. Kırmızı-siyah ağacın siyah-yüksekliği 2, 3 ve 4 olacak şekilde NIL yapraklarını 3 farklı şekilde ekleyiniz.

13.1-2

Anahtar 36 ile Şekil 13.1 deki ağaçta TREE-INSERT çağrıldıktan sonra ortaya çıkan kırmızı-siyah ağacı çiziniz. Eğer eklenmiş düğümün rengi kırmızı ise oluşan ağaç kırmızı-siyah ağaç mıdır açıklayınız. Eğer rengi siyah ise ne olacaktır açıklayınız.

13.1-3

1, 3, 4 ve 5 numaralı kırmızı siyah özelliklerini sağlayan ikili arama ağacını **gevşetilmiş kırmızı-siyah ağaç olarak** tanımlayalım. Diğer bir şekilde söylemek gerekirse kökü kırmızı veya siyah olabilen kırmızı-siyah ağaç diyebiliriz. T yi kökü kırmızı olan gevşetilmiş kırmızı-siyah ağaç olarak kabul edelim. Eğer T nin kökünün rengini siyaha yapar ancak T ağacına başka bir değişiklikte bulunmazsak sonuç olarak ortaya çıkan ağaç kırmızı-siyah ağaç mıdır?

13.1-4

Kırmızı-siyah ağaçtaki tüm kırmızı düğümlerin siyah ebeveynlere doğru “yutulduğunu” düşünelim böylece kırmızı düğümlerin çocukları siyah ebeveynin çocukları olacaktır.(Anahtarlara ne olduğu yok sayılacaktır.) Tüm kırmızı çocukla-

rın yutulduktan sonrasında ki siyah düğümün olası derecesi nedir? Sonuç olarak oluşan ağacın yapraklarının derinliği konusunda ne söyleyebilirsiniz?

13.1-5

Kırmızı-siyah ağaçtaki x ten başlayıp yine x in torunu olan yapraklara giden en kısa basit yolun uzunluğunun iki katı olan en uzun x ten başlayıp yine x in torunu olan yapraklara doğru giden en uzun yolu gösteriniz.

13.1-6

Siyah yüksekliği k olan bir kırmızı-siyah ağaçta olabilecek en fazla düğüm sayısı kaçtır? En az düğüm sayısı kaçtır?

13.1-7

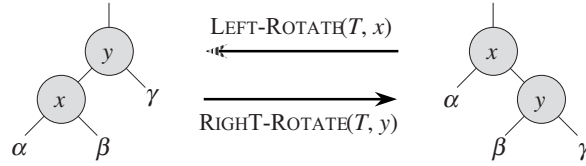
n anahtarda en yüksek mümkün kırmızı iç düğüm-siyah iç düğüm oranını sağlayabilen bir kırmızı-siyah ağaç tanımlayınız. Oran nedir? Nasıl bir ağaç en az orana sahiptir ve oran nedir?

13.2 Döndürmeler

Arama-ağacı işlemleri olan TREE-INSERT ve TREE-DELETE işlemlerini n anahtara sahip bir kırmızı-siyah ağaçta çalıştırdığımızda $O(\lg n)$ zaman sürecektir çünkü bu işlemler ağacı değiştirecektir bunun sebebi ise sonuç olarak oluşan ağaç kırmızı-siyah özelliklerinin bazılarını ihlal etme ihtimalidir. Bu özellikleri tekrar sağlamak için bazı düğümlerin rengini ve işaretçi yapısını değiştiririz.

İşaretçi yapısını ikili-arama-ağaç-özelliğini koruyan, arama ağacında yerel bir uygulama olan **döndürme** ye değiştiririz. Sağ döndürme ve sol döndürme olarak iki tip döndürme yöntemi olduğunu Şekil 13.2 de gösterilmektedir. x e sola döndürme işlemi yaparken sağ çocuk olan y nin $T.nil$ olmadığından emin olmalıyız. x sağ çocuğu $T.nil$ olmayan ağaçtaki herhangi bir düğüm olabilir. Sola döndürmede “pivot” x ten y ye olan bağlantı çevresindedir. Bu y yi altağacın kökü yaparken x i y nin sol çocuğu y nin sol çocuğunu da x in sağ çocuğu yapar.

LEFT-ROTATE için olan kabakodda x in sağ çocuğunun $T.nil$ olmadığı ve kökün ebeveyni $T.nil$ olarak kabul edilmiştir.



Şekil 13.2 İkili arama ağaçlarındaki döndürme işlemleri. LEFT-ROTATE operasyonu sağ taraftaki yapıyı belirli sayıda işaretçileri değiştirerek sol taraftaki yapı haline getirir. Tam ters operasyon olan RIGHT-ROTATE ise sol taraftaki yapıyı değiştirerek sağ taraftaki yapı haline getirir. α , β ve γ ras-sal altağaçları temsil eder. α , β , γ harfleri keyfi altağaçlardır. Bir döndürme uygulaması ikili arama ağaç özelliklerini korur. α daki anahtarlar $x.key$ in öncülü olur ki bu da β deki anahtarlara öncülük eder, bu da $y.key$ e öncülük eder ki bu da γ daki anahtarlara öncülük eder.

```

LEFT-ROTATE( $T, x$ )
1   $y = x.right$            //  $y$  ye atar
2   $x.right = y.left$        //  $y$  nin sol alt ağacını  $x$  in sağ çocuğuna eşitler
3  if  $y.left \neq T.nil$ 
4       $y.left.p = x$        //  $x$  in ebeveynini  $y$  nin ebeveynine eşitler
5   $y.p = x.p$ 
6  if  $x.p == T.nil$ 
7       $T.root = y$ 
8  elseif  $x == x.p.left$ 
9       $x.p.left = y$ 
10 else  $x.p.right = y$ 
11  $y.left = x$            //  $x$  i  $y$  nin sol çocuğu olarak atar
12  $x.p = y$ 

```

Şekil 13.3 LEFT-ROTATE nin ikili arama ağacını nasıl değiştirdiğine örnek verir. RIGHT-ROTATE nin kodu simetriktir. LEFT-ROTATE ve RIGHT-ROTATE nin ikisi de $O(1)$ zamanda çalışır. Döndürme işlemleri ile sadece işaretçiler değişir diğer tüm nitelikler aynı kalır.

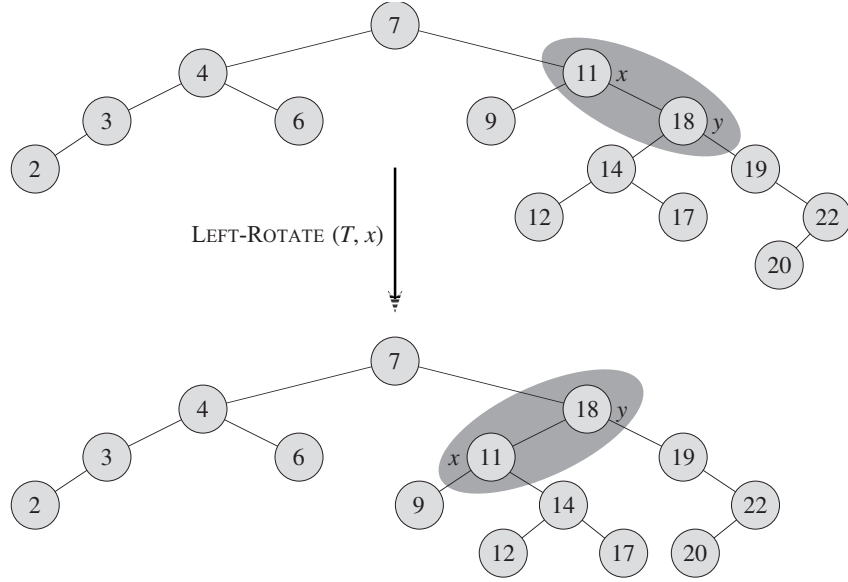
Alıştırmalar

13.2-1

RIGHT-ROTATE için kabakod yazın.

13.2-2

n -düğüm sayısına sahip bir ikili arama ağacında $n - 1$ tane döndürmenin yapılıp yapılamayacağını tartışınız.



Şekil 13.3 LEFT-ROTATE prosedürünün ikili arama ağacında nasıl değişiklikler yarattığını gösteren bir örnek. Verilen ağacın düzenli ağaç adımları ve modifiye edilmiş ağaç anahtar değerlerin aynı listelenmesini üretir.

13.2-3

a , b ve c sırasıyla Şekil 13.2'deki α , β ve γ sol ağacın altağaçlardaki rassal düğümler olsun. Şekildeki x düğümüne sola döndürme uygulandığında a , b , c düğümlerinin derinlikleri nasıl olacaktır?

13.2-4

Keyfi bir n -düğümlü ikili arama ağacı $O(n)$ döndürme işlemleri kullanılarak farklı bir n -düğümlü ikili arama ağacına dönüştürülebileceğini gösteriniz. (İpucu: İlk olarak en fazla $n - 1$ sayıda yapılan sağ-döndürme işlemi ile ağacı sağ çocuklardan oluşan bir zincir haline getirmeye çalışın)

13.2-5 *

T_1 ağacına RIGHT-ROTATE uygulayarak T_2 elde edilebiliyorsa T_1 ağacı **sağ-dönüştürülmüş** ve T_2 haline getirilmiştir diyebiliriz. Sağ-dönüştürme uygulanarak T_2 haline getirilemeyecek T_1 ve T_2 ağaçları örnek veriniz. Sonrasında eğer T_1 sağ-dönüştürülerek T_2 olabilecek bir ağaç ise $O(n^2)$ RIGHT-ROTATE ile dönüştürülebileceğini gösteriniz.

13.3 Araya sokma

n sayıda düğümü olan kırmızı-siyah ağaçlara $O(\lg n)$ zamanda bir düğüm ekleyebiliriz. Kısım 12.3 te gösterilen TREE-INSERT prosedürünün değiştirilmiş bir versiyonunu z düğümünü T ye sanki normal bir ikili arama ağacına düğüm eklemeye benzer şekilde kullanırız ve z düğümünün rengini kırmızı olarak tanımlarız. (Alıştırma 13.3-1 de neden rengi siyah değil de kırmızı yaptığımızı açıklamanızı isteyeceğiz.) Kırmızı-siyah ağaç özellikleri garanti etmek için renk değişimleri ve döndürmeler RB-INSERT-FIXUP prosedürü ile yapılmaktadır. RB-INSERT (T, z) nin çağırılması; anahtarı çoktan doldurulmuş farzedilen z düğümü kırmızı-siyah T ağacına yerleşir.

```

RB-INSERT ( $T, z$ )
1    $y = T.nil$ 
2    $x = T.root$ 
3   while  $x \neq T.nil$ 
4        $y = x$ 
5       if  $z.key < x.key$ 
6            $x = x.left$ 
7       else  $x = x.right$ 
8    $z.p = y$ 
9   if  $y == T.nil$ 
10       $T.root = z$ 
11  elseif  $z.key < y.key$ 
12       $y.left = z$ 
13  else  $y.right = z$ 
14   $z.left = T.nil$ 
15   $z.right = T.nil$ 
16   $z.color = RED$ 
17  RB-INSERT-FIXUP ( $T, z$ )

```

TREE-INSERT ve RB-INSERT prosedürleri arasında dört adet farklılık vardır. Birincisi TREE-INSERT teki NIL değerleri $T.nil$ olarak değiştirilmiştir. İkinci olarak ağaç yapısını korumak için RB-INSERT in 14.-15. satırlarında $z.left$ i ve $z.right$ i $T.nil$ e eşitleriz. Üçüncü olarak ise 16. satırda z nin rengini belirleriz. Son olarak da z yi renklendirmek ağacın kırmızı-siyah özelliklerini bozmuş olabilir bu durumu düzeltmek ve kontrol etmek için 17. satırda RB-INSERT-FIXUP çağırılır.

RB-INSERT-FIXUP (T, z)

```

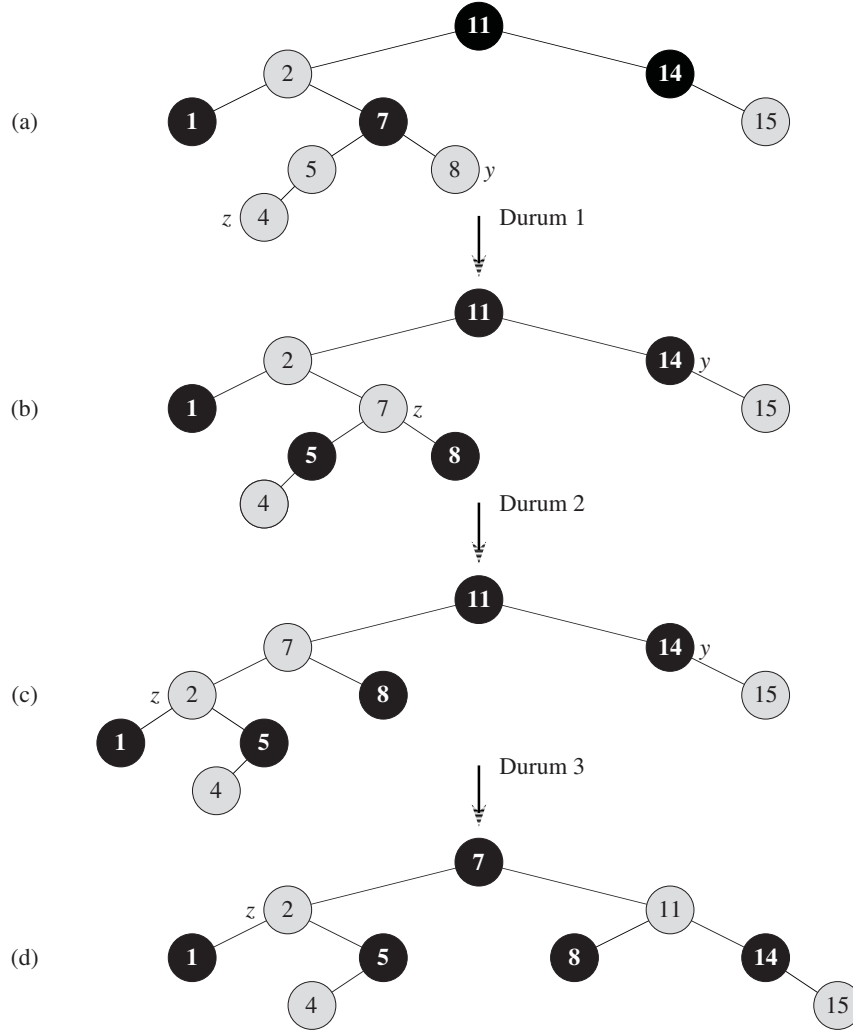
1  while  $z.p.color == RED$ 
2      if  $z.p == z.p.p.left$ 
3           $y = z.p.p.right$ 
4          if  $y.color == RED$ 
5               $z.p.color = BLACK$  // durum 1
6               $y.color = BLACK$  // durum 1
7               $z.p.p.color = RED$  // durum 1
8               $z = z.p.p$  // durum 1
9          elseif  $z == z.p.right$ 
10              $z = z.p$  // durum 2
11             LEFT-ROTATE(  $T, z$ ) // durum 2
12              $z.p.p.color = BLACK$  // durum 3
13              $z.p.p.color = RED$  // durum 3
14             RIGHT-ROTATE( $T, z.p.p$ ) // durum 3
15     else (then koşulu gibi aynı ancak “sağ” ve
           “sol” yer değiştirmiş)
16      $T.root.color = BLACK$ 

```

RB-INSERT-FIXUP in nasıl çalıştığını anlamak için incelememizi 3 ana adıma bölmeliyiz. Birincisi RB-INSERT ile z düğümü eklendiğinde ve z düğümüne kırmızı renk atandığında kırmızı siyah özelliklerin hangilerine ters düştüğünü belirlemeliyiz. İkinci olarak 1-15. satırlar arasındaki **while** döngüsünün ne amaçla kullanıldığını anlamalıyız. Son olarak da **while** döngüsünün içindeki 3 durumu da incelemeliyiz ve amacımızı nasıl gerçekleştirdiğini anlamaya çalışmalıyız.¹ Şekil 13.4 RB-INSERT-FIXUP in nasıl çalıştığını örnek bir kırmızı-siyah ağaç üzerinde göstermektedir.

RB-INSERT-FIXUP çağırıldığında hangi kırmızı-siyah özellikler ihlal edilmiştir? Yeni eklenen düğümün çocukları $T.nil$ olduğu sürece özellik 1 ve özellik 3 te sağlanmaya devam edecektir. Özellik 5 de söylendiği gibi seçilen düğümünden çıkan tüm basit yollarda ki siyah düğüm sayısı eşittir çünkü eklenen düğüm (siyah) nöbetçilerin yerine eklenmiştir ve kırmızıdır ayrıca çocukları da nöbetçidir. İhlal edilmiş olabilecek özellikler kökün siyah olması gerektiğini söyleyen özellik 2 ve kırmızı düğümlerin çocuklarının olamayacağını belirten özellik 4 tür. Eğer z kök ise özellik 2 ihlal edilir ve eğer z nin ebeveyni kırmızı ise özellik 4 ihlal edilir. Şekil 13.4(a) z düğümünün eklendikten sonra özellik 4 ün ihlal edilmesini göstermektedir.

¹2 durumu 3 durumu kapsamına giriyor ve böylece bu iki durum karşılıklı dışlayan değildir.



Şekil 13.4 RB-INSERT-FIXUP işlemi. **(a)** Araya sokmadan sonra z . Çünkü hem z hem de z nin ebeveyni kırmızı renkli bu da özellik 4 ü ihlal eder. z nin amcası olan y kırmızı olduğu sürece kod durum 1 üzerinden devam eder. Düğümleri tekrar renklendiririz ve z nin işaretçisini ağacın üst kısmına taşırız bu işlemden sonraki ağacın durumu **(b)** de gösterilmiştir. Eğer z ve z nin ebeveyni kırmızı renkliyse ancak z nin amcası olan y siyah renkliyse bir kez daha incelememiz gerekir ve bu durumda eğer z z nin ebeveyninin sağ çocuğuysa durum 2 uygulanır. Sola döndürme uygularız ve sonucu da **(c)** de görebiliriz. Eğer z ebeveyninin sol çocuğuysa durum 3 uygulanır. Renklendirme ve sağa döndürme yapılır bunun sonucu da **(d)** de gösterilmiştir.

1-15. satırlar arasındaki **while** döngüsü döngünün her bir iterasyonu başladığında aşağıdaki üç-kısımlı değişmezi korur:

- z düğümü kırmızı renklidir.
- Eğer $z.p$ kök ise $z.p$ siyah renklidir.
- Eğer ağaç kırmızı-siyah özelliklerinden bazılarını ihlal ediyorsa bu özellik 2 ya da özellik 4 tür. Eğer ağaç özellik 2 yi ihlal ediyorsa bunu sebebi z düğümünün kök ve kırmızı olmasıdır. Eğer ağaç özellik 4 ü ihlal ediyorsa bunun sebebi z ve z nin ebeveyni olan düğümün ikisinin birden kırmızı olmasıdır.

(c) kısmı kırmızı-siyah özelliklerini ihlal eden durumları ve RB-INSERT-FIXUP un çalışma prensibini anlamamız için (a) ve (b) kısımlarından çok daha yararlıdır. Çünkü z düğümü ve z düğümüne yakın olan düğümler üzerine odaklanacağız. Bu da (a) kısmından bildiğimiz gibi z nin kırmızı olduğunu anlamamıza yardımcı olacaktır. (b) kısmını da $z.p.p$ düğümünün var olduğunu göstermek için kullanacağız (Satır 2, 3, 7, 8, 13 ve 14 referans alınacak.)

Hatırlayacak olursak eğer döngü değişmezlerinin döngünü ilk iterasyonunda birinci öncelik olduğunu göstermemiz gerekir, her iterasyonda döngü değişmezlerini sağlamamız gerekir ve bu döngü değişmezi döngüyü sonlandırmamız için işe yarar bir özellik sunacaktır.

Başlatma ve sonlandırma argümanları ile başlayacağız. Sonrasında döngünün nasıl çalıştığını daha detaylı inceleyeceğiz, her iterasyonda döngünün nasıl değişmeden kaldığını tartışmalıyız. Açıklama boyunca her tekrarlama iki olası sonuç olduğunu göstereceğiz, bunlar z ağaçta yukarı taşınır ya da bazı döndürme işlemleri yapılır sonunda da döngü sonlanır.

Başlangıç: Döngünün ilk iterasyonundan önce hiçbir özellik ihlali olmayan kırmızı-siyah ağaç ve bu ağaca yeni eklenmiş kırmızı renkli z düğümü ile başlıyoruz. RB-INSERT-FIXUP çağırıldığında değişmeden kalan kısımlarını gösterecek olursak:

- RB-INSERT-FIXUP çağırıldığında z kırmızı renklidir ve ağaca eklenmiştir.
- Eğer $z.p$ kök ise ve $z.p$ siyah olarak gözüküyorsa RB-INSERT-FIXUP un çağırılma önceliğinde değişiklik olmaz.
- RB-INSERT-FIXUP çağırıldığında özellik 1, özellik 3 ve özellik 5 aynı kaldığını zaten anlatmıştık.

Eğer ağaç özellik 2 yi ihlal ediyorsa kök kırmızı renklidir demektir ve bu da z düğümünün ağaçtaki tek düğüm olduğunu gösterir çünkü z düğümünün hem iki çocuğu ve hem de ebeveyni nöbetçidir ve siyah renklidir, ayrıca bu durum özellik 4 ü ihlal etmez. Nitekim bu ihlal tüm kırmızı-siyah ağaç içindeki tek özellik ihlalidir.

Eğer ağaç özellik 4 ü ihlal ediyorsa demektir ki z düğümünün çocuklarının ikisi de siyah renkli nöbetçilerdir ve ağaç z düğümü eklenmeden önce

herhangi bir özellik ihlaline sahip değildir bunun sonucunda anlıyoruz ki özellik ihlaline neden olan durum hem z hem de $z.p$ düğümlerinin kırmızı renkli olmasıdır. Ekleyecek olursak eğer ağaçta başka bir özellik ihlali olmadığını söyleyebiliriz.

Sonlandırma: $z.p$ siyah olduğu için döngü sonlanır. (eğer z kök ise z nin ebeveyni $T.nil$ olan bir nöbetçidir ve siyah renklidir.) Nitekim ağaç döngü sonlandığında özellik 4 ü ihlal etmemiştir. Döngü sonlanırken hala sağlanmama ihtimali olan özellik ancak özellik 2 olabilir. Satır 16 da bu durumu düzeltir ve RB-INSERT-FIXUP operasyonu sonlandığında tüm kırmızı siyah özellikleri sağlanmış olur.

Yineleme: Bu olayı analiz etmek için **while** döngüsünün içindeki 6 farklı durumu göz önünde bulundurmalıyız. Bu durumların 3 tanesi zaten diğer 3 ü ile simetrik, bu ayırmda 2 numaralı satırda z nin ebeveynin z nin büyükbabasının sağ ve ya sol çocuk olmasına göre belirlenir. Verdiğimiz kod sadece $z.p$ düğümünün sol çocuk olduğu durumu ele almıştır. Eğer $z.p$ kök ise $z.p$ siyahtır. Eğer döngü iterasyonuna giriyorsak bu sadece z nin ebeveyninin kırmızı olmasıyla mümkün olabilir bundan dolayı da $z.p$ kök değildir ve buradan da anlıyoruz ki z nin büyükbabası vardır.

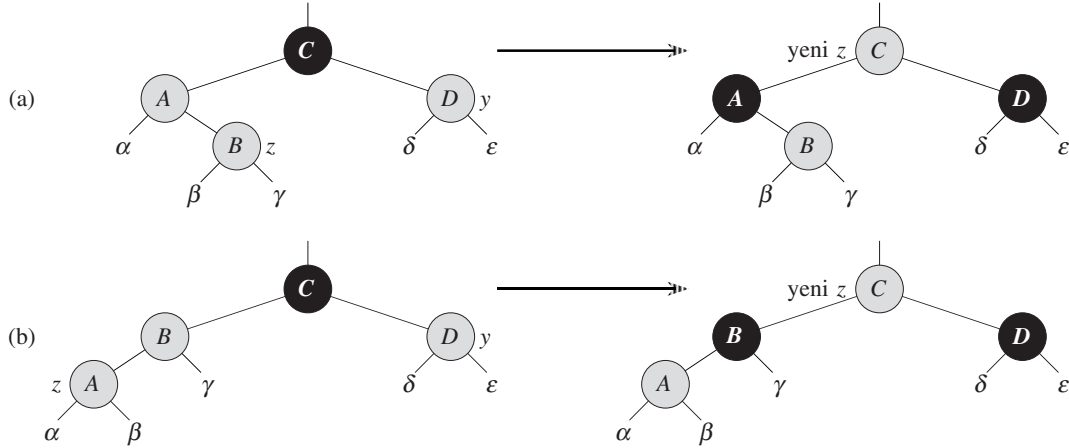
Durum 1, durum 2 ve durum 3 ü z nin ebeveyninin kardeşinin ya da z nin amcasının rengi üzerinden tartıştık. 3. satır y nin z nin amcasını göstermesini sağlar. ($z.p.p.sag$) 4. satırda y nin rengini test eder. Eğer y nin rengi kırmızıysa durum 1 i çalıştırırız. Başka türlü kontrol durum 2 ve durum 3 e geçecektir. Tüm durumlarda z nin ebeveyni kırmızı olduğu sürece z nin büyükbabası $z.p.p$ siyahtır. Özellik 4 z düğümü ve z nin ebeveyni arasında ihlal edilir.

Durum 1: z nin amcası y kırmızıdır

Şekil 13.5 gösteriyor ki durum 1 sadece z nin amcası ve y kırmızı olduğunda çalışır. $z.p$ ve y yi siyaha boyayarak bu iki düğümden oluşan sorunu çözebiliriz sonrasında da z nin büyükbabasını kırmızıya boyayarak özellik 5 i de sağlamış oluruz. Daha sonra **while** döngüsünü z nin büyükbabasını z olarak atadıktan sonra tekrarlarız bu şekilde ağaçta yukarıya doğru ilerlemiş oluruz.

Şimdi ise durum 1 in döngü bir sonraki iterasyonuna başlarken döngü değişmezlerini nasıl koruduğunu göstereceğiz. z yi o anki döngü sırasında z düğümünü göstermek için kullanırız ve z yi o anki iterasyonda z düğümünü göstermek için ve $z' = z.p.p$ yi bir sonraki iterasyonda satır 1 deki testte z düğümü denilecek olan düğümü göstermek için kullanırız.

- Bu iterasyon $z.p.p$ yi kırmızı renklendirdiğinden, z' düğümü bir sonraki iterasyonun başında kırmızıdır.
- Bu iterasyonda $z'.p, z.p.p.p$ ye eşittir ve bu düğümün rengi değişmez. Eğer bu düğüm kök ise bu iterasyondan önce siyahtı ve bir sonraki iterasyonun başında siyah kalacaktır.
- Durum 1 in özellik 5 i nasıl koruduğunu tartıştık ve bu özellik 1 ve özellik 3 ü nasıl ihlal edildiğini göstermemektedir.



Şekil 13.5 RB-INSERT-FIXUP prosedürünün durum 1 i. z ve ebeveyni $z.p$ nin ikisinin de kırmızı olduğundan özellik 4 ihlal edilir. (a) z sağ çocukta olsa (b) z sol çocukta olsa aynı yolu izleriz. Tüm $\alpha, \beta, \gamma, \delta$ altağaçlarının siyah kökleri vardır ve her biri aynı siyah yüksekliğine sahiptir. Durum 1 in kodu özellik 5 i korurken bazı düğümlerin rengini değiştirir. Bir yaprağa bir düğümden giden tüm aşağı yönlü basit yollar aynı sayıda siyah sayısına sahiptir. **While** döngüsü z düğümünün büyükbabası yeni z olarak $z.p.p$ ile devam eder. Özellik 4 ün herhangi bir ihlali sadece, kırmızı olan yeni z ile eğer kırmızıysa büyükbabası arasında oluşabilir.

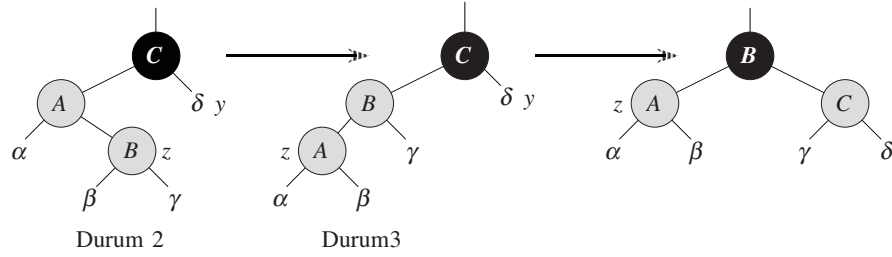
Eğer z' düğümü sonraki iterasyonda kök durumunda ise durum 1 özellik 4 ü sadece bu iterasyon için doğrular. z' kırmızı renkli ve kök konumunda olduğu sürece sadece özellik 2 yi ihlal eder ve bu ihlal direkt olarak z' ile ilgilidir.

Eğer z' düğümü bir sonraki tekrarda kök değilse durum 1 özellik 2 yi ihlal etmeyecektir. Durum 1, özellik 4 ün ihlalini bu iterasyonun başında doğrulayacaktır. Sonrasında z' düğümünü kırmızı yapıp $z'.p$ düğümünü yalnız bırakacaktır. Eğer $z'.p$ siyah renkli ise özellik 4 ün ihlali yoktur. Eğer $z'.p$ kırmızı renkli ise z' düğümünü kırmızıya boyamak $z'.p$ ve z' arasında özellik 4 ün ihlaline neden olacaktır.

Durum 2: z düğümünün amcası y siyah ve z sağ çocuktur.

Durum 3: z düğümünün amcası y siyah ve z sol çocuktur

Durum 2 ve durum 3 ün z düğümünün amcasının rengi siyahtır. İki durum z düğümünün sağ çocuk ve ya sol çocuk olup olmadığına göre ayrılır. 10 ve 11 numaralı satırlar Şekil 13.6 da durum 3 ile beraber gösterilen durum 2 yi oluştururlar. Durum 2 de z düğümü ebeveynin sağ çocuğudur. Hemen yapıyı z düğümünün bir sol çocuk olduğu durum 3 e değiştirmek için bir sol döndürme



Şekil 13.6 RB-INSERT-FIXUP prosedürünün durum 2 ve durum 3 ü. Durum 1 de durum 2 ve durum 3 ten farklı olarak özellik 4 ihlal edilir çünkü $z.p$ ve z düğümlerinin ikisi de kırmızı renklidir. α, β, γ ve δ altağaçlarının her birin kökleri siyah renklidir ve her biri aynı siyah yüksekliğine sahiptirler. durum 2 yi sol döndürme ile durum 3 haline getiririz ve bu özellik 5 i korur. Durum 3 bazı renk değişikliklerini ve sağa döndürmeyi gerçekleştirir. **While** döngüsü sonra erdiğinde özellik 4 sağlanmış olur ve alt alta iki tane kırmızı düğüm bulunmaz.

uygularız. Hem z hem de $z.p$ düğümleri kırmızı olduğu için döndürme ne düğümlerin siyah yüksekliğini ne de özellik 5 i etkilemez. Eğer durum 3 e direkt olarak ya da durum 2 den geçerek z nin amcası olan y siyah olacaktır aksi halde durum 1 i çalıştırmamız gerekir. Ek olarak 2 ve 3 numaralı satırlarda $z.p.p$ düğümünün var olduğuna kanaat getirdikten sonra 10 numaralı satırda z yi bir basamak yukarı taşıyıp 11 numaralı satırda da bir basamak alta bile çeksek $z.p.p$ nin kimliği değişmeden kalacaktır. Durum 3 te özellik 5 i sağlayan bazı renk değişiklikleri ve sağ döndürme gerçekleştirdikten sonra artık art arda iki adet kırmızı düğüm ağaçta yer almayacaktır. $z.p$ siyah olduğu sürece artık **while** döngüsü tekrarlamayacaktır.

Şimdi ise durum 2 ve durum 3 ün döngü değişmezlerini nasıl koruduğunu göstereceğiz. (Daha yeni tartıştığımız gibi, $z.p$ satır 1 de bir sonraki testte siyah olacaktır ve döngü gövdesi tekrar uygulanmayacaktır.)

- Durum 2 z nin kırmızı olan $z.p$ yi işaret etmesini sağlar. z yi değiştirmeye ihtiyaç yoktur ya da z nin rengi durum 2 ve 3 de ortaya çıkar.
- Durum 3 $z.p$ nin siyah olmasını sağlar. Eğer bir sonraki iterasyonda $z.p$ kök olarsa siyah olması sağlanmış olur.
- Durum 1 de olduğu gibi durum 2 ve durum 3 te de özellik 1 özellik 3 ve özellik 5 te sağlanmış olur.

z düğümünün durum 2 ve durum 3 te kök olmadığını bildiğimiz sürece özellik 2 nin ihlal edilmediğini söyleyebiliriz. Durum 2 ve durum 3 özellik 2 için bir ihmal oluşturmaz bu nedenle kırmızı yapılan tek düğüm durum 3 teki döndürme ile bir siyah düğümün çocuğu haline gelir.

Durum 2 ve durum 3, özellik 4 ün tek ihlalini düzeltir ve başka bir ihlal göstermez.

RB-INSERT-FIXUP un kırmızı-siyah özelliklerini doğru bir biçimde düzelttiğini gösterdik.

Analiz

RB-INSERT in çalışma zamanı nedir? n sayıda düğümün olduğu bir kırmızı-siyah ağacın yüksekliği $O(\lg n)$ olacağından, bu ağaç için RB-INSERT 1-16 satırları $O(\lg n)$ zaman alır. RB-INSERT-FIXUP da **while** döngüsü sadece durum 1 gerçekleşirse tekrarlar ve z ağaçta 2 basamak yukarı çıkar. **While** döngüsünün toplam uygulanabilme adedi bu nedenle $O(\lg n)$ dir. bu sebeple RB-INSERT toplam $O(\lg n)$ zamanda çalışacaktır. Dahası hiçbir zaman ikiden fazla döndürme sergileyemez. Bu nedenle **while** döngüsü durum 2 ve durum 3 uygulanırsa durur.

Alıştırmalar

13.3-1

RB-INSERT in 16. satırında yeni eklenmiş z düğümünün rengi kırmızıya eşitlenir. z düğümünün rengini siyah seçtiğimizde özellik 4 ün ihlal edilmediğini gözleyiniz. Kodda neden z düğümünün rengini siyah seçmedik ?

13.3-2

41, 38, 31, 12, 19, 8 anahtarını boş bir kırmızı-siyah ağaca başarılı bir şekilde ekledikten sonra ağacın son halini gösteriniz.

13.3-3

Şekil 13.5 ve 13.6 daki $\alpha, \beta, \gamma, \delta, \epsilon$, siyah yükseklikleri k olan altağaçlar olsun. Her düğümün yanına siyah yüksekliğini yazınız ve belirtilen modifikasyonun özellik 5 i sağladığını gösteriniz.

13.3-4

Profesör Teach RB-INSERT-FIXUP un durum 1 çalışırken z düğümü kök konumundayken $T.nil$ nöbetçisinin renginin kırmızı olarak değiştirilmesinden endişelidir. Profesöre endişesinin yersiz olduğunu ve RB-INSERT-FIXUP un asla $T.nil.color = 1$ KIRMIZI yapmayacağını gösteriniz.

13.3-5

RB-INSERT ile eklenen n sayıda düğümle oluşturulmuş bir kırmızı-siyah ağacı düşünün. Eğer $n > 1$ ve ağaç en azından bir kırmızı düğüme sahipse ne olur tartışınız.

13.3-6

Eğer kırmızı siyah-ağaçlar için gösterim ebeveyn işaretçiler için hiç depo içermiyorsa RB-INSERT in etkin olarak nasıl yerine getirileceği hakkında öneride bulunun.

13.4 Silme

n -düğümlü bir kırmızı-siyah ağaçtaki diğer basit işlemler gibi, bir düğümü silmede $O(\lg n)$ zamanda gerçekleşir. Kırmızı-siyah ağaçlarda bir düğümü silmek araya sokmaya göre daha karmaşıktır.

Kırmızı siyah ağaçlardan bir düğümü silmek için prosedür temel olarak TREE-DELETE (Kısım 12.3) operasyonuna dayanmaktadır. İlk önce TRANSPLANT altprogramını değiştirmemiz gerekir ki TREE-DELETE yi kırmızı-siyah ağaçlarda kullanabilelim.

RB-TRANSPLANT (T, u, v)

```

1  if  $u.p == T.nil$ 
2       $T.root = v$ 
3  elseif  $u == u.p.left$ 
4       $u.p.left = v$ 
5  else  $u.p.right = v$ 
6   $v.p = u.p$ 

```

RB-TRANSPLANT (T, u, v) prosedürü TRANSPLANT dan iki şekilde farklıdır. İlk olarak, 1 numaralı satırda NIL yerine $T.nil$ i test eder. İkinci olarak ise 6 numaralı satırda v nöbetçiyi işaretliyor olsa bile $v.p$ ye atama yapılır. v nin $T.nil$ e eşit olduğu durumda $v.p$ ye değer atama imkanımızdan fazlasıyla faydalanacağız.

RB-DELETE prosedürü TREE-DELETION prosedürüne oldukça benzemektedir ancak kabakoduna birkaç ekleme yapılmıştır. Bu satırlar y düğümünü gözleyerek kırmızı-siyah özelliklerini ihlal etme ihtimalini inceler. İki çocuktan daha az çocuğa sahip bir z düğümünü sildiğimizde y düğümünün z olmasını isteriz. z nin iki çocuğu olduğunda y nin z nin ardılı olması gerekir ve y z nin pozisyonuna taşınır. Ayrıca yeri değiştirilen ya da ağaçtan çıkartılan y düğümünün rengini hatırlamamız ve y düğümünün orijinal yerine yerleştirilen x düğümünü takip etmemiz gerekir çünkü x kırmızı-siyah özelliklerini ihlal ediyor olabilir. z düğümünü sildikten sonra RB-DELETE ağacı düzenlemek için yardımcı bir prosedür olan RB-DELETE-FIXUP u çağıracaktır.

```

RB-DELETE( $T, z$ )
1   $y = z$ 
2   $y\text{-original-color} = y.\text{color}$ 
3  if  $z.\text{left} == T.\text{nil}$ 
4       $x = z.\text{right}$ 
5      RB-TRANSPLANT ( $T, z, z.\text{right}$ )
6  elseif  $z.\text{right} == T.\text{nil}$ 
7       $x = z.\text{left}$ 
8      RB-TRANSPLANT ( $T, z, z.\text{left}$ )
9  else  $y = \text{TREE-MINIMUM}(z.\text{right})$ 
10      $y\text{-original-color} = y.\text{color}$ 
11      $x = y.\text{right}$ 
12     if  $y.p == z$ 
13          $x.p = y$ 
14     else RB-TRANSPLANT ( $T, y, y.\text{right}$ )
15          $y.\text{right} = z.\text{right}$ 
16          $y.\text{right}.p = y$ 
17     RB-TRANSPLANT ( $T, z, y$ )
18      $y.\text{left} = z.\text{left}$ 
19      $y.\text{left}.p = y$ 
20      $y.\text{color} = z.\text{color}$ 
21 if  $y\text{-original-color} == \text{BLACK}$ 
22     RB-DELETE-FIXUP ( $T, x$ )

```

RB-DELETE, TREE-DELETE den kabakodda iki kat daha fazla satır sayısı içersede iki prosedür de aynı temel yapıya sahiptir. TREE-DELETE deki benzer durumlar altında çalıştırılan her bir satır RB-DELETE de de yer alır (NIL yerine $T.\text{nil}$, TRANSPLANT yerine RB-TRANSPLANT değişiklikleriyle birlikte).

Bu iki prosedür arasındaki diğer farklar ise :

- Ağaçtan çıkartılan yada yeri değiştirilen y düğümü korunmalıdır. 1. satırda y, z düğümünü işaretlemektedir. z nin ikiden az çocuğu olduğunda y ağaçtan kaldırılacaktır. z nin iki tane çocuğu olduğunda ise 9. satır ile y z nin ardılını işaretler ve TREE-DELETE de olduğu gibi y z nin yerine yerleştirilir.
- y nin rengi değişebileceği için $y\text{-original-color}$ değişkeni y nin renginin herhangi bir değişiklik oluşmadan önceki halini saklar. 2. ve 10. satırlar y de ki değişikliklerden hemen sonra bu değişkene atama yaparlar. z nin iki çocuğa sahip olduğu ve $z \neq y$ durumda y düğümü z düğümün orijinal pozisyonuna taşınır. 20. satırda y düğümüne z nin orijinal rengini verir. y nin orijinal rengini RB-DELETE nin sonunda karşılaştırma yapabilmek için saklamamız gerekir;

eğer rengi siyah ise düğümü silme ya da yerini değiştirme işlemleri kırmızı-siyah özelliklerine aykırı bir durum oluşturabilir.

- Bahsedildiği gibi, y nin orijinal yerine taşınan x düğümünün izi takip edilir. 4. 7. ve 11. satırların görevleri ise x i y nin tek çocuğunu ya da y nin çocuğu yoksa $T.nil$ i işaretleyecek şekilde gerekli değişikliği yapmaktır.
- x $T.nil$ olsa bile y nin orijinal pozisyonuna yerleştirildikten sonra $x.p$ y nin ebeveynini gösterecek şekilde ayarlanmalıdır. z y nin orijinal ebeveyni olmadıkça (bu sadece z nin ardılı olan y nin z nin sağ çocuğu olduğu ve z nin iki çocuğunun bulunduğu durumlarda gözlenir) $x.p$ RB-TRANSPLANT kodunun 6. satırında yerini alacaktır.(RB-TRANSPLANT ın 5, 8, veya 14. Satırları çağrıldığı zaman, geçen üçüncü parametrenin x ile aynı olduğu gözlemlenir.)
 y nin orijinal ebeveyni z dir fakat biz y nin orijinal ebeveynine $x.p$ nin işaret etmesini istemiyoruz, bu yüzden düğümü ağaçtan kaldırırız. Çünkü y düğümü z düğümünün yerine çıkmak üzere, satır 13 te $x.p$ yi y ye düzenleyerek ağaçta yukarı ilerler bu da $x.p$ yi y nin ebeveyninin orijinal pozisyonuna işaret etmesine sebep olur.
- Son olarak y düğümü siyah renkli ise kırmızı-siyah özelliklerinden bir kaçını ihlal etmiş olabilir bunu düzeltmek için 22. satırda RB-DELETE-FIXUP u çağırılacaktır. Eğer y kırmızı renkli ise, kırmızı-siyah özellikleri y nin yer değiştirmesine veya ağaçtan çıkarılmasına bakılmaksızın aşağıdaki nedenlerden dolayı korunur.

1. Ağaçta hiçbir siyah yüksekliği değişmemiştir.
2. Hiçbir kırmızı düğüm bitişik olmamıştır. Çünkü z nin renginden dolayı y z nin yerini aldığından y nin yeni konumunda iki bitişik kırmızı düğüm olmasına izin veremeyiz. Ek olarak eğer y z nin sağ çocuğu değilse o zaman y nin orijinal sağ çocuğu olan x y ile yer değiştirir. Eğer y kırmızı ise x in siyah olması gerekir o zaman y ile x i yer değiştirdiğimizde her hangi iki kırmızı düğümün bitişik olması beklenmez.
3. y hiçbir zaman kök olamaz çünkü y kırmızı renklidir ve kök her zaman siyah olmalıdır.

Eğer y düğümü siyah renkli ise üç tane problem ortaya çıkacaktır ve bunlara çözüm olacak olansa RB-DELETE-FIXUP çağrısıdır. Birincisi, eğer y kök ise ve y nin kırmızı renkli çocuğu yeni kök olduysa bu durum özellik 2 yi ihlal eder. İkincisi, x ve $x.p$ nin ikisinde kırmızı renkli ise bu durum özellik 4 ü ihlal eder. Üçüncüsü ise y düğümünün yerini değiştirdiğimizde y düğümünün bulunduğu önceki basit yol bir siyah düğüm kaybedecek ve siyah sayısı bir azalacaktır. Bu nedenle y nin herhangi bir atası özellik 5 i ihlal edecektir. y nin orijinal pozisyonunda yer alan x için “ekstra” siyah ibaresi kullanarak bu ihlali giderebiliriz. Eğer x düğümünü içeren herhangi bir basit yolun siyah düğüm miktarına 1 eklersek özellik 5 i sağlamış oluruz. Siyah renkli y düğümünü kaldırırsak veya yerini değiştirirsek y nin siyahlığını x e vermiş oluruz. Problem artık x in siyah yada kırmızı olması

değildir böylece sadece özellik 1 ihlal edilmiş olur. Bunun yerine, x düğümü ya “ikili siyah” ya da “kırmızı-siyah” tır ve x düğümünü içeren herhangi bir basit yolun siyah düğüm miktarına sırasıyla ya 2 ya da 1 katkıda bulunur. x in renk özelliği ya KIRMIZI (eğer x kırmızı-siyah ise) ya da SİYAH (eğer x ikili siyah ise) olacaktır. Diğer bir deyişle, bir düğüm üzerindeki ekstra siyah, x in işaret ettiği düğümün renk özelliği yerine yansıtılır.

Artık RB-DELETE-FIXUP prosedürünün kırmızı-siyah özelliklerini nasıl koruduğunu görebiliriz.

```

RB-DELETE-FIXUP ( $T, x$ )
1  while  $x.T.root$  ve  $x.color == BLACK$ 
2      if  $x == x.p.left$ 
3           $w = x.p.right$ 
4          if  $w.color == RED$ 
5               $w.color = BLACK$  // durum 1
6               $x.p.color = RED$  // durum 1
7              LEFT-ROTATE( $T, x.p$ ) // durum 1
8               $w = x.p.right$  // durum 1
9          if  $w.left.color == BLACK$  ve  $w.right.color == BLACK$ 
10              $w.color = RED$  // durum 2
11              $x = x.p$  // durum 2
12         elseif  $w.right.color == BLACK$ 
13              $w.left.color = BLACK$  // durum 3
14              $w.color = RED$  // durum 3
15             RIGHT-ROTATE ( $T, w$ ) // durum 3
16              $w = x.p.right$  // durum 3
17              $w.color = x.p.color$  // durum 4
18              $x.p.color = BLACK$  // durum 4
19              $w.right.color = BLACK$  // durum 4
20             LEFT-ROTATE ( $T, x.p$ ) // durum 4
21              $x = T.root$  // durum 4
22         else ( then koşulu gibi aynı ancak “sağ” ve “sol” yer değiştirmiş)
23      $x.color = BLACK$ 

```

RB-DELETE-FIXUP prosedürü özellik 1, özellik 2 ve özellik 4 ün restore edilmesini sağlar. Alıştırma 13.4-1 ve 13.4-2 sizden fonksiyonun özellik 2 ve özellik 4 ü nasıl restore ettiğini göstermenizi isteyecek bu yüzden biz bu bölümün kalanında özellik 1 in nasıl restore edildiğini göstereceğiz. 1-22 numaralı satırlar arasındaki **while** döngüsünün amacı aşağıdaki şartlar sağlanana kadar ekstra siyah düğümü ağaçta yukarıya doğru hareket ettirmektir.

1. x in kırmızı ve siyah düğümü işaretlediği durumda x in rengini 23. satırda siyah yaparız.
2. x in kökü işaretlediği durumda basitçe ekstra siyah olan düğümü kaldırırız ya da
3. uygun olan döndürme ve renklendirme işlemlerini uygular, döngüden çıkarız.

While döngüsünde x her zaman kök olmayan ikili siyah düğümü işaretler. 2. satırda x in sol ya da sağ çocuk olduğuna karar veririz. w işaretçisinin x in kardeşini işaretlemesini sağlarız. x ikili siyah düğüm olduğu sürece w düğümü $T.nil$ olamaz çünkü $x.p$ den yaprak olan w düğümüne giden basit yoldaki siyah sayısı $x.p$ den x e giden basit yoldaki siyah sayısından az olacaktır.

Koddaki dört durum² Şekil 13.7 de gösterilmiştir. Her durumu detaylıca incelemeden önce özellik 5 i koruyan değişikliklerin nasıl yapıldığına genel olarak bakalım. Her durumdaki anahtar fikir; uygulanan değişikliğin köklerden $\alpha, \beta, \dots, \zeta$ altağaçlarına kadar olan siyah sayısını nasıl koruduğuyla ilgilidir. Böylece eğer özellik 5 değişiklikten önce korunuyorsa değişiklikten sonra da korunmaya devam edecektir. Örneğin Şekil 13.7(a) da kökten α, β altağaçlarına kadar olan siyah sayısı hem değişiklikten önce hem de sonra 3 tür. Benzer olarak kökten γ, δ, ϵ ve ζ kadar olan basit yollardaki siyah sayıları hem değişiklikten önce hem de sonra 2 dir. Şekil 13.7(b) de hesaplamaya kökün rengini gösteren c değerini kırmızı veya siyah olmasına bakılmaksızın dahil edilmesi gerekir. $sayı(KIRMIZI) = 0$ ve $sayı(SİYAH) = 1$ diye tanımladığımızda değişiklikten önce ya da sonra kökten α' 'ya olan basit yoldaki siyah düğüm sayısı $2 + sayı(c)$ ye eşittir. Bu durumda değişiklikten sonra yeni x düğümü c renk değişkenine sahip olacaktır ama bu düğüm (eğer $c = KIRMIZI$ ise) kırmızı-ve-siyah sırasında yada (eğer $c = SİYAH$) alt alta iki siyah şeklinde görülür. Diğer durumları da aynı şekilde doğrulayabilirsiniz. (Alıştırma 13.4-5 bakınız.)

Durum 1: x in kardeşi w kırmızı ise

Durum 1 w düğümü x in kardeşi ve kırmızı renkli olduğunda gerçekleşir. w düğümünün siyah renkli çocuğu olması gerektiği sürece hiçbir özelliği ihlal etmeden w ve $x.p$ düğümlerinin renklerini kendi aralarında değiştirip $x.p$ ye sol-döndürme uygulayabiliriz. x in yeni kardeşi aynı zamanda döndürmeden önce w düğümünün kardeşi olan düğüm artık siyah renklidir ve böylece durum 1 i durum 2, 3 ya da 4 e dönüştürmüş oluruz.

Durum 2, 3 ve 4 w düğümü siyah renkli olduğunda gerçekleşir; bu durumlar w düğümünün çocuklarının renklerine göre ayrılırlar.

²RB-INSERT-FIXUP da olduğu gibi, RB-DELETE-FIXUP daki durumlar karşılıklı dışlayan değildir.

Durum 2: x in kardeşi w siyah renkli ve w düğümünün her iki çocuğuda siyah renkli ise

Durum 2 de w düğümünün her iki çocuğu da siyah renklidir. Ayrıca w siyah olduğu sürece x ve w düğümlerinden bir tane siyah düğümü alırız ve x bu şekilde bir siyah ile yalnız bırakılır, w da kırmızı olarak bırakılır. x ve w düğümlerinden bir siyah düğümün alınmasını karşılamak amacıyla $x.p$ düğümüne ekstra siyah ekleriz ve bu siyah düğüm orjinalinde kırmızı ya da siyahtır. Sonrasında **while** döngüsünü $x.p$ düğümü ile gerçekleştiririz. Eğer durum 2 ye durum 1 içinden girecek olursak $x.p$ kırmızı olduğu sürece, x yeni düğümünün kırmızı ya da siyah olması farketmez. Bu nedenle yeni x düğümünün renk değişkeni olan c değeri kırmızıdır ve döngü döngü şartını test ettiği anda sonlandırılır. Sonrasında yeni x düğümünün rengini 23. satırda (tek başına) siyah yaparız.

Durum 3: x in kardeşi w siyah, w nun sol çocuğu kırmızı ve w nun sağ çocuğu siyah ise

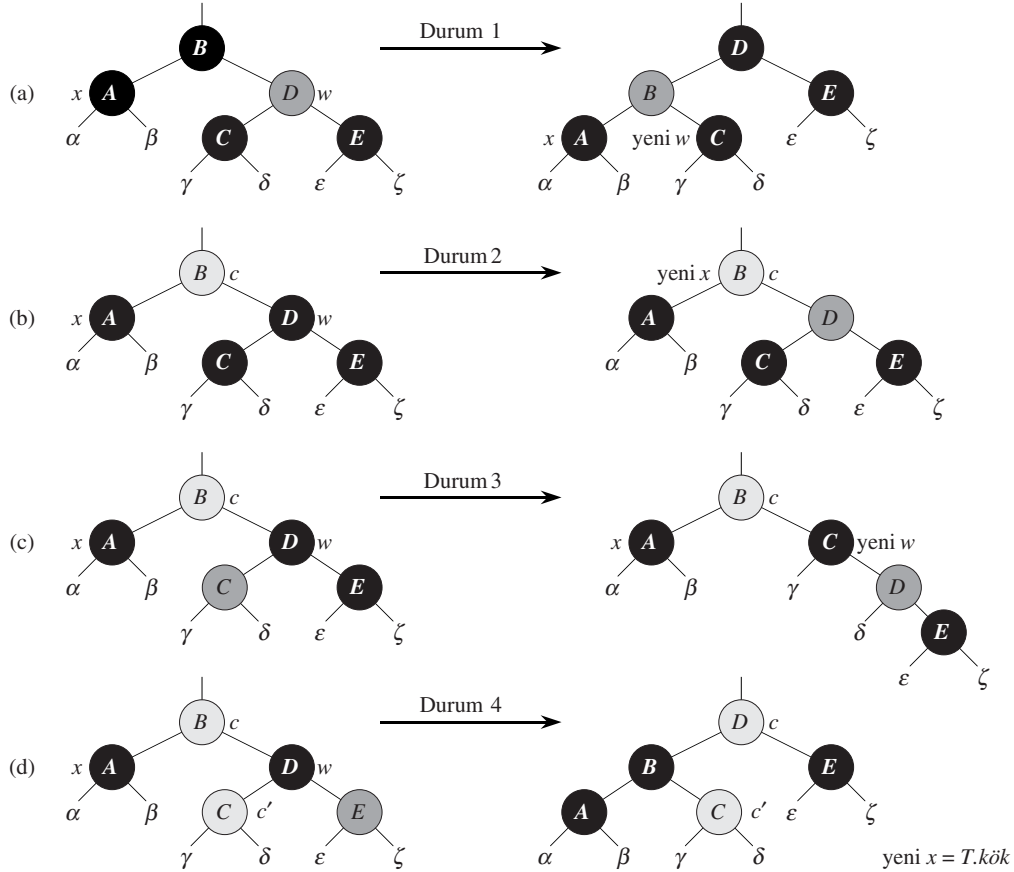
Durum 3 (13-16 satırlar arasında Şekil 13.7(c)) w siyah renkli, w nun sol çocuğu kırmızı ve yine w nun sağ çocuğu siyah olduğunda gerçekleşir. w ve w nun sol çocuğu olan $w.left$ düğümünün renklerini değiştiririz ve sonrasında w düğümüne sağ döndürme uygularız bu işlem hiçbir kırmızı-siyah özelliğini ihlal etmeyecektir. Artık x in kardeşi sağ çocuğu kırmızı, siyah renkli w düğümüdür ve bu şekilde durum 3 durum 4 e dönüştürülmüştür.

Durum 4: x in kardeşi w siyah ve w nin sağ çocuğu kırmızı ise

Durum 4 (17-21 satırlar arasında Şekil 13.7(d)) x in kardeşi olan w siyah ve yine w nun sağ çocuğu kırmızı olduğunda gerçekleşir. Bazı renk değişiklikleri ve $x.p$ üzerine sol döndürme işlemleri yaparak x üzerindeki ekstra siyah düğümü kaldırabiliriz bu yaptığımız işlemlerin hiçbirisi kırmızı-siyah özelliğini ihlal etmeyecektir. x i kök olarak ayarlamak **while** döngüsü test edilirken sonlanmasına neden olacaktır.

Analiz

RB-DELETE nin çalışma zamanı nedir? n düğüm sayısına sahip bir kırmızı-siyah ağacın yüksekliği $O(\lg n)$ olduğu sürece prosedürün toplam maliyeti RB-DELETE-FIXUP çağırılmadan $O(\lg n)$ zaman alacaktır. RB-DELETE-FIXUP un içindeki durum 1, durum 2 ve durum 4 ün her biri sabit sayıda renk değişimi ve en fazla üç farklı döndürme yaptıktan sonra sonlandırılacaklardır. Durum 2 **while** döngüsünün tekrarlanabildiği tek durumdur ve daha sonrasında x işaretçisi döndürme uygulanmadan ağaçta en fazla $O(\lg n)$ kadar yukarı doğru hareket eder. Böylece RB-DELETE-FIXUP prosedürü $O(\lg n)$ zaman alır ve en fazla üç döndürme yapar, bu nedenle RB-DELETE de $O(\lg n)$ zaman alır.



Şekil 13.7 RB-DELETE-FIXUP prosedürünün **while** döngüsü içindeki durumlar. Karartılmış düğümler siyah, gölgelendirilmişler kırmızı ve hafif gölgelendirilmişler c ve c' renk değişkenlerine sahiptirler. $\alpha, \beta, \dots, \zeta$ harfleri herhangi bir altağacı tanımlar. Her durum ağacı soldaki dizilimden bazı renk değişiklikleri ve/veya döndürmeler kullanarak sağ taraftaki dizilime getirir. x ile işaretlenen herhangi bir düğüm ekstra siyah düğüme sahiptir ve bu da ikili siyah ya da kırmızı-siyah düğüm dizilimine neden olur. Sadece durum 2 döngünün tekrarlanmasına neden olur. (a) Durum 1 B ve D düğümlerinin renklerini kendi aralarında değiştirip sol döndürme uygulandıktan sonra durum 2, durum 3 yada durum 4'e döner. (b) Durum 2'de ekstra siyah x işaretçisi ile gösterilir ve x D düğümünün rengi değiştirilerek B düğümünü gösterecek şekilde ağaçta yukarıya taşınır. Eğer durum 2'ye durum 1 üzerinden gelindiye döngü sonlandırılır çünkü yeni x düğümü artık kırmızı-siyah dizilimindedir ve bu nedenle c renk değişkeninin değeri kırmızı olur. (c) Durum 3 C ve D düğümlerinin renklerini kendi aralarında değiştirip sağ döndürme uygulanarak durum 4 haline getirir. (d) Durum 4 bazı renk değişiklikleri ve sol döndürme uygulayarak ekstra siyah x düğümünü yok eder ve sonrasında döngü sonlandırılır.

Alıştırmalar

13.4-1

RB-DELETE-FIXUP çalıştıktan sonra ağacın kökünün siyah renkli olup olmadığını tartışınız.

13.4-2

Eğer RB-DELETE deki x ve $x.p$ kırmızı olsaydı kırmızı-siyah özelliklerindeki özellik 4 RB-DELETE-FIXUP ile sağlanır mıydı, tartışınız.

13.4-3

Alıştırma 13.3-2 de başlangıçtaki boş ağaca başarılı bir şekilde 41, 38, 31, 12, 19, 8 anahtarlarını ekleyerek bir kırmızı-siyah ağaç oluşturmuştunuz. Şimdi ise sonuç olarak çıkan ağaçtan sırasıyla 8, 12, 19, 31, 38, 41 düğümlerini siliniz.

13.4-4

RB-DELETE-FIXUP için kodun hangi satırlarında $T.nil$ nöbetçisini inceleriz veya değiştiririz, gösteriniz.

13.4-5

Şekil 13.7 deki durumların herbirinde, gösterilen altağacın kökünden $\alpha, \beta, \dots, \zeta$ altağaçlarının her birine kadar olan siyah düğümlerin sayısını belirleyiniz ve dönüştürmeden sonra her bir sayının aynı kaldığını doğrulayınız. Bir düğüm c veya c' renk niteliğine sahip olduğunda sayınızda sembolik olarak $\text{sayı}(c)$ veya $\text{sayı}(c')$ notasyonunu kullanınız.

13.4-6

Profesör Skelton ve Baron RB-DELETE-FIXUP un 1. durumunun başlangıcından dolayı endişeliler ve $x.p$ düğümünün siyah olmayabileceğini düşünüyorlar. Eğer profesörler doğru düşünüyorlarsa 5. ve 6. satırlar yanlıştır. Durum 1 in başındaki $x.p$ düğümünün siyah olması gerektiğini ve profesörlerin endişelenmesine gerek olmadığını gösteriniz.

13.4-7

Varsayalım ki RB-INSERT ile x düğümünü kırmızı-siyah ağaca ekleyelim ve hemen RB-DELETE ile silelim. Sonuç olarak ortaya çıkan kırmızı-siyah ağaç başlangıçtaki eski ağaç ile aynı mıdır? Cevabınızı kanıtlayınız.

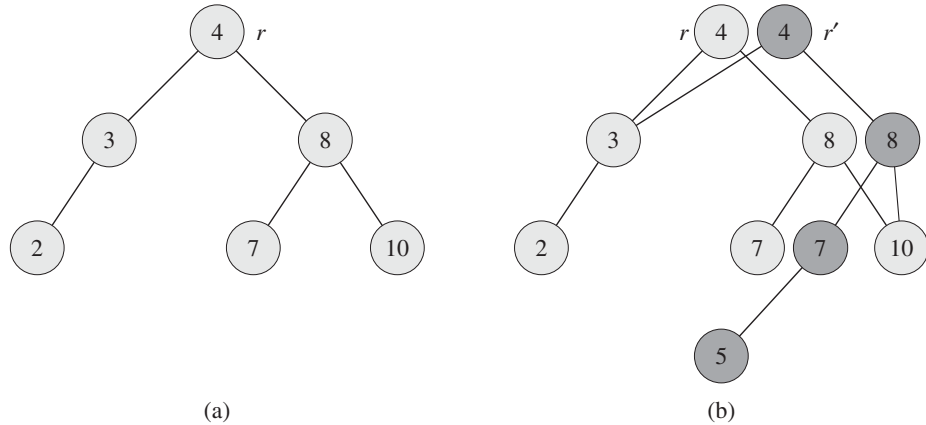
Problemler

13-1 Kalıcı dinamik kümeler

Algoritma dersi sırasında bazen güncellenmiş dinamik kümelerin önceki versiyonlarını saklamak gerektiğini fark ederiz. Bu kümelere **kalıcı** deriz. Kalıcı kümeleri uygulamanın bir yolu kümeyi değiştireceğimiz sırada tüm kümeyi kopyalamak olacaktır ancak bu durum programı oldukça yavaşlatacak ayrıca fazladan bir alan kaplayacaktır. Bazen bundan çok daha iyisini de yapabiliriz.

INSERT, DELETE ve SEARCH işlemlerine sahip ve Şekil 13.8(a) da da gösterildiği gibi ikili-arama ağaçları kullanarak gerçekleştirdiğimiz S kalıcı kümesini düşünelim. Kümenin her versiyonu için ayrı bir kök sağlarız. 5 anahtarını kümeye eklemek için üzerinde 5 anahtarı olan bir düğüm yaratırız. Bu düğüm üzerinde 7 anahtarı olan düğümü değiştiremediğimiz sürece üzerinde 7 anahtarı olan yeni düğümün sol çocuğu olacaktır. Benzer olarak üzerinde 7 anahtarı olan yeni düğüm 8 anahtarına sahip yeni düğümün sol çocuğu olacaktır ve sağ çocuk da var olan 10 anahtarına sahip düğümdür. 8 anahtarına sahip yeni düğüm, sol çocuğu var olan 3 anahtarının üzerinde 4 anahtarını barındıran r' kökünün sağ çocuğudur. Böylece Şekil 13.8(b) de gösterildiği gibi var olan ağacın sadece bir kısmını kopyalayıp düğümlerin bir kısmını gerçek ağaç ile paylaşarak kullanırız.

Her düğümün *key*, *left* ve *right* değişkenlerine sahip ancak *parent* değişkenine sahip olmadığını varsayalım.



Şekil 13.8 (a) 2, 3, 4, 7, 8, 10, anahtarlarına sahip bir ikili-arama ağacı. (b) 5 anahtarını ekledikten sonra oluşan kalıcı ikili-arama ağacı. Ağacın en güncel versiyonuna r' kökünden ve bir önceki versiyonuna da r kökünden ulaşılabilir. Yüksek miktarda gölgelendirilmiş düğümler 5 anahtarı eklenirken eklenmiş olan düğümlerdir.

- a. Genel bir kalıcı ikili-arama ağacı için ekleyeceğimiz anahtarları k ya da sileceğimiz düğümleri y olarak tanımlayınız.
- b. Verilen ağaç T , eklenecek anahtar k , T ağacına k anahtarı eklenerek ortaya çıkan kalıcı ağaç ise T' olmak üzere PERSISTENT-TREE-INSERT prosedürü yazınız.
- c. Eğer kalıcı ikili-arama ağacının yüksekliği h ise PERSISTENT-TREE-INSERT in uygulaması ne kadar zaman alır ve alan gereksinimi ne kadardır? (Alan gereksinimi yeni düğüm sayısı ile doğru orantılıdır.)
- d. Her düğüme bir ebeveyn değişkeni eklediğimizi düşünelim. Bu durumda PERSISTENT-TREE-INSERT ek bir kopyalama gerektirir mi? Bu değişiklikten sonra n sayıda düğüme sahip ağaca PERSISTENT-TREE-INSERT uygulanırsa $\Omega(n)$ zaman ve alan gerektireceğini kanıtlayınız.
- e. Her araya sokma ve silme işleminde kırmızı-siyah ağaçların en kötü-durumda $O(\lg n)$ zamanı ve alanı nasıl garanti ettiğini gösteriniz.

13-2 Kırmızı-Siyah ağaçlarda birleşim işlemi

Birleşim işlem için iki dinamik küme S_1 ve S_2 ayrıca her hangi bir $x_i \in S_i$ ve $x_j \in S_j$ olan x elementi gerekir, $x_i.key \leq x.key \leq x_j.key$ durumu vardır. Bu bize $S = S_1 \cup \{x\} \cup S_2$. Bu problemde birleşim işleminin kırmızı-siyah ağaçlara nasıl uygulanacağını incelememiz gerekir.

- a. Verilen kırmızı-siyah ağaç T olsun, siyah-yüksekliğini yeni bir değişken olan $T.sy$ ye yazalım. RB-DELETE ve RB-INSERT in sy değişkenini ağacın düğümleri içinde ekstra bir alana ihtiyacı olmadan ve asimptotik çalışma zamanlarını arttırmadan saklayıp saklayamayacağını tartışalım. T ile azalan şekilde her düğümün siyah-yüksekliğini $O(1)$ zamanda her düğümü ziyaret ederek belirlenebileceğini gösteriniz.

T_1 ve T_2 ağaçlarını yok ederek sonuç olarak $T = T_1 \cup \{x\} \cup T_2$ kırmızı-siyah ağacını döndüren RB-JOIN(T_1, x, T_2) işlemini oluşturmak istiyoruz. n , T_1 ve T_2 ağaçlarının toplam düğüm sayısı olsun.

- b. $T_1.sy \geq T_2.sy$ olarak kabul edelim. T_1 içinde siyah yüksekliği $T_2.sy$ olan en büyük anahtar değerine sahip y siyah düğümünü bulan bir algoritma tasarlayalım.
- c. T_y kökü y olan bir alt ağaç olsun. $T_y \cup \{x\} \cup T_2$ ağacının T_y altağacı ile $O(1)$ zamanda her hangi bir kırmızı-siyah özelliğini ihlal etmeden nasıl yer değiştirdiğini belirtiniz.
- d. x i hangi renk yapmalıyız ki özellik 1, özellik 3 ve özellik 5 ihlal edilmesin? Özellik 2 ve özellik 4 ün nasıl $O(\lg n)$ zamanda uygulandığını açıklayınız.

- e. Kısım (b) de yapılan varsayımın nasıl hiçbir genelliği kaybetmediğini tartışınız. $T_1.sy \leq T_2.sy$ olduğunda ortaya çıkan simetrik durumu açıklayınız.
- f. $O(\lg n)$ zaman alan RB-JOIN operasyonu tartışınız.

13-3 AVL ağaçları

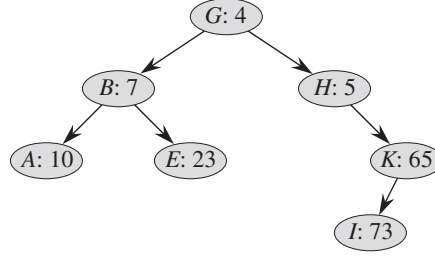
AVL ağaçları *yükseklik dengeli* ikili-arama ağaçlarıdır: her x düğümü için x in sağ ve sol altağaçlarının yüksekliği en fazla 1 farklıdır. AVL ağaç uygulaması için her düğüm için ekstra bir değişken oluşturmamız gerekir: $x.y$ x düğümünün yüksekliğini gösterir. Diğer herhangi bir ikili-arama ağacında olduğu gibi, $T.root$ ağacın kökünü işaretler.

- a. n sayıda düğümü olan bir AVL ağacının yüksekliğinin $O(\lg n)$ olduğunu kanıtlayınız. (*İpucu*: yüksekliği h olan ve F_h sayıda (h . Fibonacci sayısı) düğümü olan bir AVL ağacı üzerinde kanıtlayınız).
- b. AVL ağacında araya sokma yapmak için öncelikle düğümü ikili-arama ağacı düzeninde uygun olan bir yere ekleriz. Bu işlemten sonra artık ağacın yüksekliği dengeli olmayabilir. Özellikle bazı düğümlerin sağ ve sol çocuklarının yükseklikleri arasındaki fark 2 olabilir. İçine sağ ve sol çocuklarının yükseklik farkı en fazla 2 olan $|x.right.h - x.left.h|$ bir x düğümü alan ve kökü x olan alt ağaç üzerinde yüksekliği dengelemek için değişiklikler yapan bir BALANCE(x) prosedürünü tanımlayalım. (*İpucu*: döndürmeleri kullanın.)
- c. Kısım (b) yi kullanarak AVL-INSERT(x, z) isimli içine AVL ağacı üzerinde bulunan x düğümü, yeni oluşturulmuş z düğümü alan ve kökü x düğümü olan ağaca z düğümünü ekleyen ayrıca x düğümünü AVL ağacının kökü olarak belirten kuralı koruyan özyinelemeli prosedür AVL-INSERT(x, z) yi tanımlayınız. Kısım 12.3 de gördüğümüz TREE-INSERT de olduğu gibi $z.anahtar$ değişkeninin dolu olduğunu ayrıca $z.sol = \text{NIL}$ ve $z.sağ = \text{NIL}$ olduğundan dolayı $z.y = 0$ olduğunu kabul etmeliyiz. Bundan dolayı z düğümünü T AVL ağacına eklemek için AVL-INSERT($T.root, z$) yi çağırırız.
- d. AVL-INSERT in n sayıda düğüme sahip bir AVL ağacında $O(\lg n)$ zaman alacağını ve $O(1)$ miktarda döndürme yapacağını gösteriniz.

13-4 Treaps

Eğer n sayıda düğümü ikili-arama ağacına eklersek sonuç olarak ortaya çıkan ağaç korkunç şekilde dengesiz olabilir ve bu durum da uzun arama zamanlarına neden olur. Kısım 12.4 de gördüğümüz üzere rassal oluşturulmuş ikili-arama ağaçları dengeli olma eğilimindedirler. Bu nedenle düzenlenmiş değerlerden ortalama dengeli bir ikili-arama ağacı oluşturmak için rassal sıralanmış bu değerleri sıralarını bozmadan eklemek bir stratejidir.

Tüm bu değerlere aynı anda sahip olmasak ne olurdu? Bu değerlere tek tek erişme imkanımız olsaydı bunlardan halâ rassal bir ikili-arama ağacı oluşturabilir miydik?



Şekil 13.9 Bir treap. Her x düğümü $x.key$: $x.priority$ ile etiketlenmiştir. Örneğin kök G anahtarına ve 4 önceliğine sahiptir.

Bu sorulara olumlu cevap verebilecek bir veri yapısını inceleyeceğiz. **Bir treap**, düğümleri sıralama şekli ile değiştirilmiş bir ikili-arama ağacıdır. Örnek olarak Şekil 13.9 gösterilmiştir. Normalde de olduğu gibi ağaçtaki her $x.düğümü$ bir adet anahtara sahiptir. Ek olarak her düğüm için bağımsız olmak üzere rastgele sayılar $x.priority$ değişkenine atanmıştır. Tüm önceliklerin ve tüm anahtarların farklı olduğunu kabul ederiz. Bir treap ın düğümleri, anahtarları ikili-arama-ağacı sıralama özelliğine uyacak şekilde sıralanmıştır ve önceliklerde min-bellek yığını sıralama uyacak şekilde sıralanmıştır:

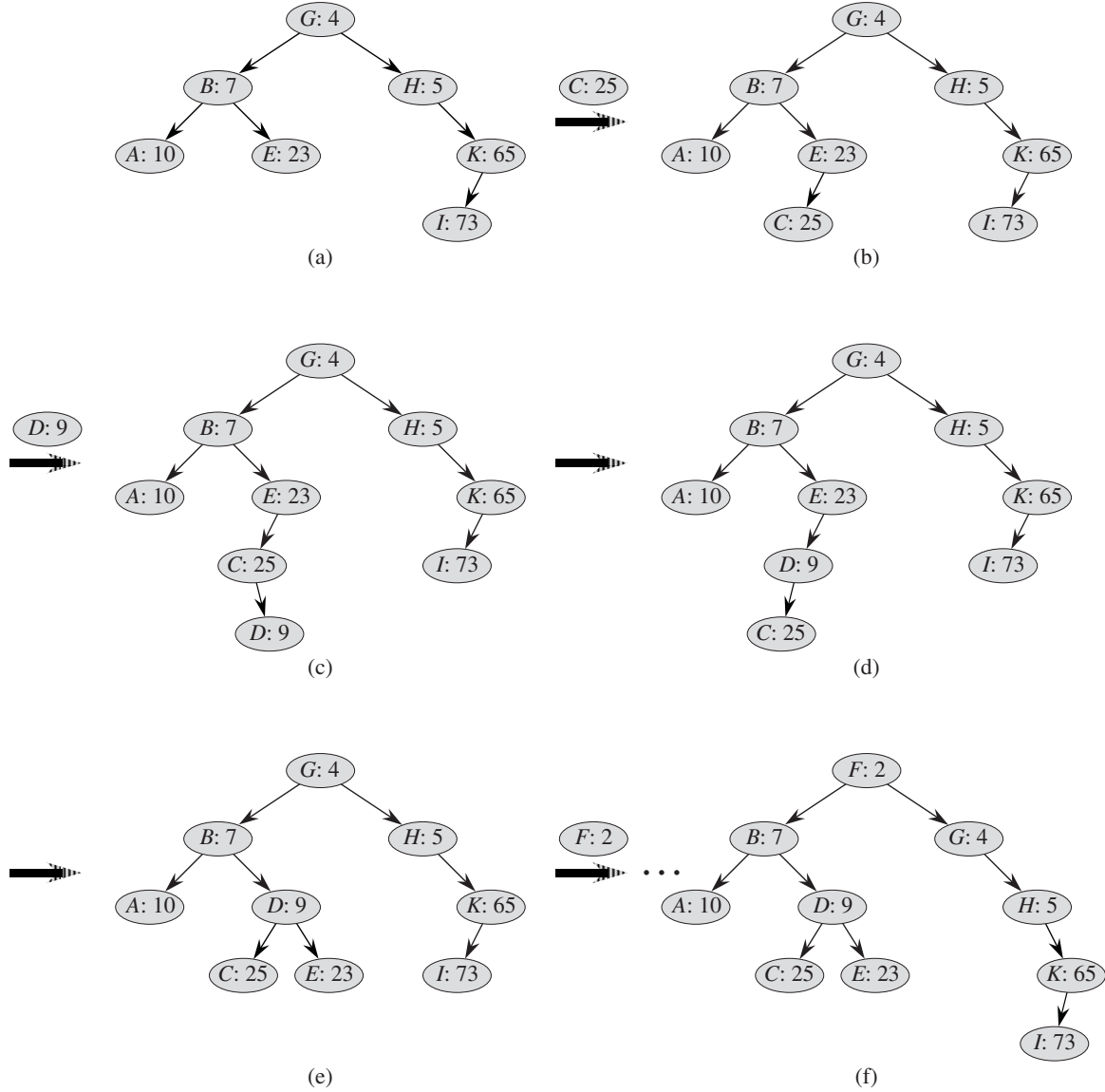
- Eğer v u nun sol çocuğu ise $v.key < u.key$ dir.
- Eğer v u nun sağ çocuğu ise $v.key > u.key$ dir.
- Eğer v u nun çocuğu ise $v.priority > u.priority$ dir.

(Bu ağaç yapısına neden “treap” denildiğini soracak olursak yukarıda belirtilen özelliklerin kombinasyonu olması diyebiliriz: hem ikili-arama ağacı özelliklerini hem de bellek yığını özelliklerini barındırmaktadır.)

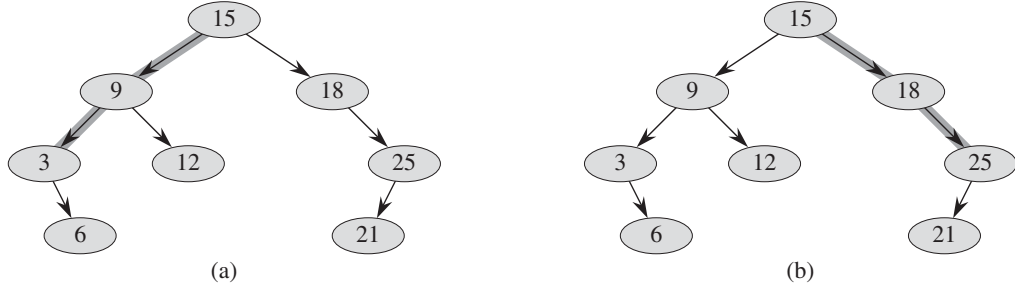
Anlatılacak olan bu yol treaps ı daha iyi anlamınıza yardımcı olacaktır. Aralarında ilişkili anahtarlara sahip $x_1, x_2, \dots, x_n$ düğümlerini treap a ekleyelim. Sonuç olarak çıkan treap düğümlerin aldıkları treap göre yerleştirilmiş bir ikili-arama ağacından farklı olmaması gerekir. Örnek olarak $x_i.priority < x_j.priority$ ise x_i ağaca x_j den önce yerleştirilmiştir.

- Aralarında birbirinden farklı ancak ilişkili anahtarlara ve önceliklere sahip $x_1, x_2, \dots, x_n$ kümesi ile treap arasında benzersiz bir ilişki olduğunu gösteriniz.
- Treap ın beklenen yüksekliğinin $\Theta(\lg n)$ ve bu sebeple de beklenen treap içindeki değerleri arama zamanının $\Theta(\lg n)$ olduğunu gösteriniz.

Var olan bir treap a yeni bir düğümün nasıl eklendiğine bakalım. Birinci olarak yapacağımız şey yeni düğüme rastgele bir öncelik değeri atamak olacaktır. Sonrasında Şekil 13.10 da gösterilen ve TREAP-INSERT olarak adlandırdığımız araya sokma algoritmasını çağırırız.



Şekil 13.10 TREAP-INSERT işlemi. **(a)** Araya sokma öncesindeki orijinal treap. **(b)** Üzerinde C anahtarı ve öncelik bulunduran düğüm eklendikten sonraki treap. **(c)-(d)** D anahtarı ve öncelik barındıran düğüm eklendiği ara aşamalar. **(e)** (c)-(d) kısımları bittikten sonraki treap görünümü. **(f)** F anahtarı ve 2 önceliğini barındıran düğüm eklendikten sonraki treap.



Şekil 13.11 İkili-arama ağacının kolları. Sol kol (a) kısmında gölgelendirilmiştir, sağ kol da (b) kısmında gölgelendirilerek gösterilmiştir.

- c. TREAP-INSERT in nasıl çalıştığını açıklayınız. Arkasında bulunan fikri Türkçe olarak açıklayınız ve kabakodda oluşturunuz. (İpucu: Öncelikle ikili-arama-ağacı araya sokma prosedürünü çalıştırınız ve sonrasında min-bellek yığını sıralama özelliğini restore etmek için döndürmeleri çalıştırınız.)
- d. TREAP-INSERT in beklenen çalışma zamanının $\Theta(\lg n)$ olduğunu gösteriniz.

TREAP-INSERT bir arama ve bir dizi döndürme yapar. Pratikte bu iki işlemin de farklı maliyetleri olsa bile, aynı çalışma zamanında gerçekleşmesi beklenmektedir. Arama, treap ı değiştirmeden ağaç üzerinden bilgi okur. Buna zıt olarak bir döndürme, ebeveyn ve çocuk işaretçilerini değiştirerek treap ın farklılaşmasına neden olurlar. Genel olarak çoğu bilgisayar yapısında okuma işlemleri yazma işlemlerinden hızlıdır. Bu nedenle TREAP-INSERT in en az miktarda döndürme yapmasını isteriz. Beklenen döndürme sayısının sabit bir değerle sınırlandırılmasını göstereceğiz.

Bunu yapmak için Şekil 13.11de tasvir edilen bazı tanımlara ihtiyacımız vardır. T ikili arama ağacının **sol kolu** kökten en düşük anahtar değerine sahip olan düğüme kadar olan altağaçtır. Bir diğer bakış açısıyla sol kola kökten başlayıp içinde sadece düğümlerin sol çocuklarını barındıran bir alt ağaç diyebiliriz. Simetrik olarak düşünersek, T ağacının **sağ kolu** da kökten başlayıp düğümlerin sadece sağ çocuklarını barındıran bir altağaçtır. Bu kolların **uzunlukları** barındırdığı düğüm sayısıdır.

- e. T treap ının TREAP-INSERT in x . düğümünü ekledikten hemen sonraki halini göz önünde bulunduralım. C x in sol altağacının sağ omurgasının uzunluğu olsun. D ise x in sağ altağacının sol kolunun uzunluğu olsun. x düğümünün eklenmesi sırasında yapılan döndürme sayısının $C + D$ olduğunu kanıtlayınız.

Şimdi ise C ve D nin beklenen değerlerini hesaplayacağız. Genelliği kaybetmek-sizin birbirleri ile karşılaştırdığımız zaman anahtarların $1, 2, \dots, n$ olduğunu kabul edelim.

T treap deki x ve y düğümleri için $y \neq x$, $k = x.key$ ve $i = y.key$ tanımlamalarını yapalım. Rassal gösterge değişkenlerini tanımlarız.

$X_{ik} = \mathbf{I} \{y, x \text{ in sol altağacının sağ kolunda bulunur.}\}$

f. $y.priority > x.priority$, $y.key < x.key$ durumunda $X_{ik} = 1$ olduğunu gösteriniz.

g. Aşağıdaki eşitliği gösteriniz.

$$\begin{aligned} \Pr \{X_{ik} = 1\} &= \frac{(k-i-1)!}{(k-i+1)!} \\ &= \frac{1}{(k-i+1)(k-i)} . \end{aligned}$$

h. Aşağıdaki eşitliği gösteriniz.

$$\begin{aligned} E[C] &= \sum_{j=1}^{k-1} \frac{1}{j(j+1)} \\ &= 1 - \frac{1}{k} . \end{aligned}$$

i. Aşağıdaki eşitliği göstermek için simetrik argümanlar kullanınız.

$$E[D] = 1 - \frac{1}{n-k+1} .$$

j. Bir treap ın düğüm eklerken beklenen döndürme sayısının 2 den az olduğuna dair bir karar verirsiniz.

Bölüm notları

Arama ağaçlarını dengeli yapma fikri 1962 de problem 13-3 gösterilen AVL ağaçlarını tanıtan Adel'son-Vel'ski'ı ve Landis [2] ye dayanır. Bir diğer arama ağaçları sınıfı ise 2–3 ağaçları J. E. Hopcroft tarafından 1970 de tanıtıldı. 2–3 ağaçları düğümlerin derecelerini manipüle ederek dengeyi korumaya çalışan ağaçlardır. Bölüm 18 2–3 ağaçlarının genel özelliklerini kapsayarak Bayer ve McCreight [35] tarafından tanıtılmış B-ağaçları hakkındadır.

Kırmızı-siyah ağaçlar “simetrik ikili B-ağaçları” ismi ile Bayer [34] tarafından geliştirilmiştir. Guibas ve Sedgewick [155] uzunluk üzerine özellikler konusunda çalışmışlardır ve kırmızı/siyah renk düzenini tanıtmışlardır. Andersson

[15] kırmızı-siyah ağaçların kodlanması için daha basit başka bir form sunmuştur. Weiss [351] bu formu AA-ağaçları olarak adlandırmıştır. AA-ağaçları kırmızı-siyah ağaçlar ile benzerlik gösterir ancak AA-ağaçlarında sol çocuklar hiçbir zaman kırmızı olamazlar.

Treaps Seidel ve Aragon tarafından sunulan ve Problem 13–4 konu edilen yapılardır. İyi-uygulanmış veri yapıları ve algoritmalar bütünü olarak LEDA [253] sözlüğünün varsayılan uygulamalarıdır.

Dengeli ikili ağaçlar için birçok farklı form bulunur bunlara ağırlık-dengeli ağaçlar [264], k -komşu ağaçları [245] ve kendini dengeleyen ağaçlar [127] dâhildir. Belki de en merak uyandırıcı olanı ise kendi kendini düzenleyebilen Sleator ve Tarjan [320] tarafından tanıtılmış olan “yayvan ağaçlardır” (Yayvan ağaçların iyi bir gösterimi için Tarjan [330] a bakınız.). Yayvan ağaçlar dengeyi ayrı, örneğin renk gibi, bir denge şartına ihtiyaç duymadan sağlarlar. Bunun yerine “yayvan işlemler” olarak adlandırılan ve ağaca her erişildiğinde çalıştırılan işlemlerle dengeyi sağlarlar. n -düğümlü bir ağaçtaki her bir işlemin amortize maliyeti $O(\lg n)$ zaman alır.

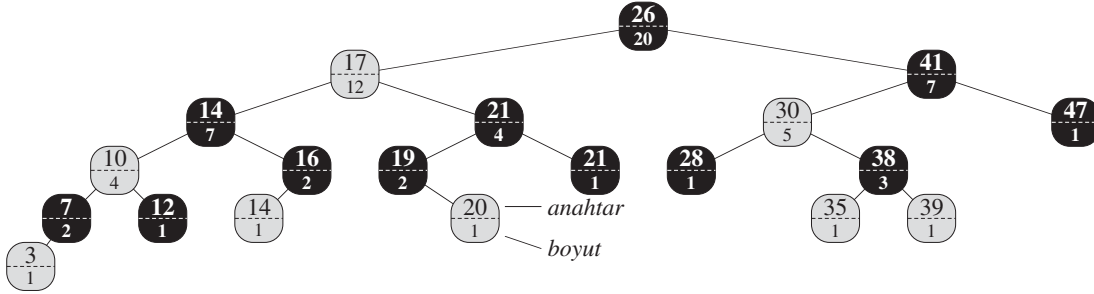
Atlama listeleri [286] dengeli ağaçlara başka bir alternatiftir. Bir atlama listesi ek işaretçilerin sayısal olarak büyümesi ile oluşan bir bağlı-listedir. n birimli bir atlama listesinde her bir işlem $O(\lg n)$ zaman alır.

Bazı mühendislik işlemleri bir veri yapısı ders kitabından daha fazlasına ihtiyaç duymaz—örneğin çift bağlı liste, çarpı tablosu ya da ikili arama ağacı—fakat diğerlerinin çoğu belli bir yaratıcılık çizgisine ihtiyaç duyar. Yine de sadece az rastlanan durumlarda, tamamen yeni bir veri yapısı tasarlamaya ihtiyaç duyulur. Çoğunlukla, bir veri yapısı ders kitabını, içeriğine ilave bilgiler ekledikten sonra güçlendirme yeterli olur. Böylece veri yapıları için yeni işlemler programlayıp, yapılması istenen uygulamayı geliştirebilirsiniz. Güçlendirilen bir veri yapısı her zaman açıkça anlaşılabilir bir şekilde olmayabilir, bununla birlikte, eklenen bilgiler veri yapısındaki olağan işlemler tarafından güncellenmeli ve korunmalıdır.

Bu bölümde, kırmızı-siyah ağaçları güçlendirerek düzenlediğimiz iki veri yapısından söz edilecektir. Kısım 14.1, bir dinamik küme üzerindeki, genel sıra istatistiği işlemlerini destekleyen bir veri yapısını anlatır. Böylece, bir kümedeki i inci en küçük sayıyı veya kümenin toplam sırasında verilen elemanın rankını kolaylıkla bulabiliriz. Kısım 14.2, bir veri yapısının güçlendirilmesi işlemini özetler ve kırmızı-siyah ağaçların güçlendirilmesi işlemini kolaylaştıran bir teoremden bahseder. Kısım 14.3 bu teoremi, aralıkların (zaman aralıkları gibi) dinamik kümelerini koruyan veri yapılarının tasarlanmasına yardım etmek için kullanır. Bu sayede bir sorgu aralığı verildiğinde, kümede onunla çakışan bir aralığı kolayca bulabiliriz.

14.1 Dinamik sıra istatistikleri

Bölüm 9 da sıra istatistiği kavramına giriş yapılmıştı. Özellikle, n elemanlı bir kümenin i inci sıra istatistiği, $i \in \{1, 2, \dots, n\}$ kümesinin bir elemanı olmak üzere, basitçe i inci en küçük anahtar ile birlikte kümedeki bir elemandır. Sırasız bir kümenin $O(n)$ zamanda sıra istatistiğini nasıl belirlediğimizi görmüştük. Bu bölümde, kırmızı-siyah ağaçları nasıl düzenleyeceğimizi göreceğiz; dolayısıyla bir dinamik kümenin $O(\lg n)$ zamanda sıra istatistiğini belirleyebiliriz. Ayrıca, $O(\lg n)$ zamanda bir elemanın—eleman doğrusal sıralı bir kümede yer almalı—*rank*ının nasıl hesaplanacağını da göreceğiz.



Şekil 14.1 Genişletilmiş bir kırmızı-siyah ağaç şeklinde sıra istatistiği ağacı. Gölgeleştirilmiş düğümler kırmızı, karartılmış olanlar ise siyahtır. Ek olarak, x ten başlayan altağaçta, başlangıç düğüm noktası dışındaki her x düğümü, düğümlerin sayısını gösteren $x.size$ özelliğine sahiptir.

Şekil 14.1 hızlı sıra-istatistiği işlemlerini destekleyebilen bir veri yapısını göstermektedir. Bir *T sıra-istatistiği ağacı* basitçe, her düğümde artan şekilde bilgi depolayan bir kırmızı-siyah ağaçtır. x düğüm noktasında genel kırmızı-siyah ağaç özellikleri olan; $x.key$, $x.color$, $x.p$, $x.left$ ve $x.right$ in yanında, başka bir özellik olan $x.size$ özelliğine de sahibiz. Bu özellik, x ten başlayan (x in kendisini içeren) altağaçtaki (iç) düğümlerin sayısını, yani altağacın boyutunu içerir. Başlangıç boyutunu 0 olarak belirlersek—yani $T.nil.size$ 0 a ayarlarsak—şu eşitlikle karşılaşırız:

$$x.size = x.left.size + x.right.size + 1$$

Sıra-istatistiği ağacında farklı anahtarlara ihtiyaç duymayız (Örneğin, Şekil 14.1 deki ağaç değer 14 ve değer 21 de ikişer tane anahtara sahiptir). Birbirine eşit anahtarların varlığında, sözü geçen rank kavramı iyi tanımlanamaz. Bir sıra-istatistiği ağacı için bu belirsizliği, düzenli ağaç adımıyla elemanın rankını, elemanın yazılacağı yerde tanımlayarak ortadan kaldırırız. Örneğin Şekil 14.1 de, siyah düğümdeki anahtar 14 ün rankı 5 tir ve kırmızı düğümdeki anahtar 14 ün rankı 6 dır.

Rankı verilen bir elemanı almak

Bu tip bir bilgiyi araya sokma veya çıkarma işlemleri boyunca nasıl korumak gerektiğini göstermeden önce, bu ek bilgiyi kullanan iki sıra istatistiği sorgusu uygulamasını inceleyelim. Rankı verilen bir elemanın alınmasıyla başlayalım. OS-SELECT(x, i) prosedürü, x ten başlayan altağaçtaki i ninci en küçük anahtarı içeren düğüme bir işaretçi döndürür. Bir T sıra-istatistiği ağacındaki i ninci en küçük anahtarın yer aldığı düğümü bulmak için, OS-SELECT($T.root, i$) ı çağırırız.

```

OS-SELECT ( $x, i$ )
1   $r = x.left.size + 1$ 
2  if  $i == r$ 
3      return  $x$ 
4  else if  $i < r$ 
5      return OS-SELECT( $x.left, i$ )
6  else return OS-SELECT( $x.right, i - r$ )

```

OS-SELECT in birinci satırında, r yi hesaplarız, yani x ten başlayan altağaçtaki x düğümünün rankını. $x.left.size$ in değeri, x ten başlayan altağacın sırayla taranmasında, x ten önce gelen düğümlerinin sayısıdır. Dolayısıyla, $x.left.size + 1$, x ten başlayan altağacın kapsamındaki x in rankıdır. Eğer $i = r$ ise, bu durumda x düğümü u i . en küçük elemandır ve üçüncü satırdaki x e geri döneriz. Eğer $i < r$ ise, i . en küçük eleman x in sol alt ağacındadır, dolayısıyla beşinci satırdaki $x.left$ e geliriz. Eğer $i > r$ ise, bu kez i . en küçük eleman x in sağ altağacında yer alır. x ten başlayan altağaç, düzenli ağaç adımımda x in sağ altağacından önce gelen r tane eleman içerdiği için, x ten başlayan altağaçta i . en küçük eleman, $x.right$ dan başlayan altağaçta $(i - r)$ inci en küçük elemana eşittir. 6. satır bu elemanı özyinelemeli olarak belirler.

OS-SELECT in nasıl çalıştığını görmek için, Şekil 14.1 deki sıra-istatistiği ağacındaki 17. en küçük eleman için bir arama düşünelim. Anahtarı 26 olan x i başlangıç olarak alalım ve $i = 17$ olsun. 26. sol altağacın boyutu 12 olduğu için, rankı 13 tür. Dolayısıyla, 26. sağ altağaçta rankı 17 olan düğüm noktasının, $17 - 13 = 4$ üncü en küçük elemana eşit olduğunu biliyoruz. Bir özyinelemeli çağrıdan sonra, x anahtarı 41 olan düğümdür ve $i = 4$ tür. 41. sol altağacın boyutu 5 e eşit olduğundan, x in bulunduğu altağaçtaki rankı 6 dır. Dolayısıyla, rankı 4 e eşit olan düğüm noktasının 41. sol altağaçtaki 4 üncü en küçük elemana eşit olduğunu biliyoruz. Bir özyinelemeli çağrıdan sonra, x anahtarı 30 a eşit olan düğümdür ve x in bulunduğu altağaçtaki rankı 2 dir. Dolayısıyla, anahtarı 38 olan düğümden başlayan altağaçta, $4 - 2 = 2$ nci en küçük elemanı bulmak için bir kez daha yineleriz. Artık onun sol altağacının boyutunun 1 e eşit olduğunu bulmayı biliyoruz ve 1 aynı zamanda en küçük ikinci eleman anlamına gelir. Bu sebeple, prosedür anahtarı 38 olan düğüm noktasına bir işaretçi döndürür.

Sıra-istatistiği ağacında her bir özyinelemeli çağrı bir seviye aşağı indiği için, OS-SELECT için toplam süre, ağacın yüksekliğiyle ters orantılıdır. Ağaç bir kırmızı-siyah ağaç olduğundan, yüksekliği $O(\lg n)$ e eşittir, n burada toplam düğüm sayısıdır. Yani, OS-SELECT için çalışma süresi n elemanlı bir dinamik küme için $O(\lg n)$ dir.

Bir elemanın rankını belirleme

Bir T sıra-istatistiği ağacında x düğümü için bir işaretçi verilmiş olsun; OS-RANK prosedürü, x e T nin düzenli ağaç adımı tarafından belirlenen doğrusal sıralı bir yer döndürür.

```

OS-RANK ( $T, x$ )
1   $r = x.left.size + 1$ 
2   $y = x$ 
3  while  $y \neq T.root$ 
4      if  $y = y.p.right$ 
5           $r = r + y.p.left.size + 1$ 
6       $y = y.p$ 
7  return  $r$ 

```

Prosedür aşağıdaki gibi çalışmaktadır. x node unun rankını, düzenli ağaç adımında x ten önce gelen düğümlerinin sayısı + 1 olarak düşünebiliriz. OS-RANK aşağıdaki döngü değişmezini korur:

3-6. satırlardaki **while** döngüsünün her bir iterasyonunun başlangıcında, r , y düğümünden dan başlayan altağactaki $x.key$ in rankıdır.

Bu değişmezi döngüyü OS-RANK ın doğru bir şekilde çalıştığını göstermek için aşağıdaki gibi kullanırız

Başlangıç: Öncelikle ilk iterasyon, 1.satır x ten başlayan altağacın içindeki $x.key$ in rankını r ye eşit olacak şekilde ayarlar. 2.satırı $y = x$ olacak şekilde ayarlamak, 3.satırın düzgün çalıştığını ilk kez test etmek içindir.

Yineleme: **while** döngüsünün her bir iterasyonunun bitişinde, $y = y.p$ olacak şekilde ayarlarız. Dolayısıyla, eğer döngü gövdesinin başlangıcında y den başlayan altağaçta $x.key$ i rankı r ise, döngü gövdesinin bitişinde $y.p$ den başlayan altağaçta $x.key$ in rankının yine r ye eşit olacağını göstermek zorundayız. **While** döngüsünün her bir iterasyonunda, $y.p$ den başlayan altağacı göz önünde bulundurmalıyız. Düzenli ağaç adımında y den başlayan altağaçta ve x ten önce gelen düğümlerinin sayısını zaten belirlemiştik ve bu yüzden düzenli ağaç adımında x ten önce gelen y ile kardeş olan düğümünden başlayan altağactaki düğümleri de eklememiz gerekir, eğer $y.p$ de x ten önce geliyorsa + 1 ekleriz. Eğer y sol çocuk ise, bu durumda ne $y.p$ ne de $y.p$ nin sağ altağacında herhangi bir düğüm x ten önce gelmez ve bu yüzden r yi yalnız başına bırakırız. Diğer türlü, y sağ çocuktur ve $y.p$ nin kendisi de aynı şekilde olmak üzere, $y.p$ nin sağ altağacındaki bütün düğümler x ten önce gelir. Bu nedenle, 5. satırda, r ye $y.p.left.size + 1$ in değerini de ekleriz.

Sonlandırma: $y = T.root$ olduğu zaman döngü sonlanır, onun için y den başlayan altağaç, altağacın tamamıdır. Bu nedenle, r nin değeri altağacın tamamındaki $x.key$ in rankına eşittir.

Örnek olarak; Şekil 14.1 deki sıra-istatistiği ağacında, anahtarı 38 olan düğümün rankını bulmak için OS-RANK ı çalıştırdığımızda, **while** döngüsünün en başında $y.key$ ve r için aşağıdaki değerler dizisini elde ederiz:

<i>İterasyon</i>	<i>y.key</i>	<i>r</i>
1	38	2
2	30	4
3	41	4
4	26	17

Prosedür rankı 17 olarak döndürür.

While döngüsünün her bir iterasyonu $O(1)$ zaman aldığı ve her bir iterasyonla birlikte y nin ağaçtaki seviyesi yükseldiği için, OS-RANK ın çalışma süresi ağacın yüksekliğiyle ters orantılıdır. Düğüm sayısı n olan bir sıra-istatistiği ağacı için çalışma süresi $O(\lg n)$ denebilir.

Altağacın boyutlarının korunması

Her düğümde verilen boyut özelliklerine göre, OS-SELECT ve OS-RANK sıra istatistiği bilgisini hızlıca hesaplayabilir. Bu özellikleri kırmızı-siyah ağaçtaki temel düzenlenmiş işlemlerin içerisinde gereken şekilde koruyamadığımız sürece, çalışmamız boşa gitmiş olacaktır. Şimdi, araya sokma ve çıkarma işlemleri sırasında altağacın boyutlarını iki işlemin de asimptotik çalışma sürelerini etkilemeyecek şekilde nasıl koruyacağımızı gösterelim.

Kısım 13.3 te kırmızı-siyah ağaçta araya sokma işleminin iki fazdan oluştuğunu belirtmiştik. Birinci faz başlangıçtan itibaren, var olan düğümlere yeni bir çocuk ekleyerek ağaçta aşağı doğru iner. İkinci faz kırmızı-siyah özellikleri korumak için, renkleri ve çalışma yönlerini değiştirerek ağaçta yukarı doğru çıkar.

Birinci fazda altağacın boyutlarını korumak için, sadece kökten yapraklara doğru çizilen yoldaki her x düğümü için $x.key$ i artırırız. Eklenen yeni düğüm boyutu olarak 1 i alır. İzlenen yolda $O(\lg n)$ tane düğüm noktası olduğundan, boyut özelliklerini korumak için gereken ek zaman $O(\lg n)$ dir.

İkinci fazda, temel kırmızı-siyah ağaç için tek yapısal değişimler en fazla iki tane olan döndürmelerden kaynaklanır. Bunun yanında, döndürme bir yerel işlemdir ve buna göre sadece iki düğüm boyut özelliklerini iptal eder. Döndürmenin gerçekleştirildiği bağın etrafı bu iki düğüme bağlıdır. Kısım 13.2 deki LEFT-ROTATE (T, x) kodunu baz alarak, aşağıdaki satırları ekleriz:

```

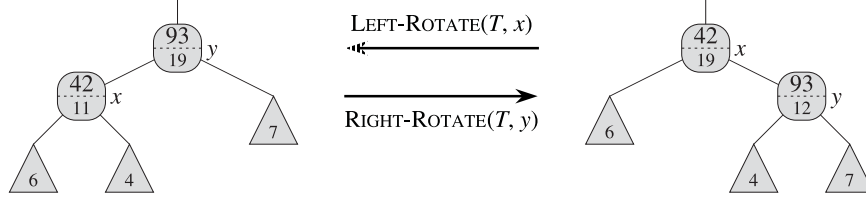
13  y.size = x.size
14  x.size = x.left.size + x.right.size + 1

```

Şekil 14.2 özelliklerin nasıl güncelleneceğini gösterir. RIGHT-ROTATE için değişim simetriktr.

Kırmızı-siyah ağaca araya sokma yapılırken en fazla iki döndürme söz konusu olduğundan, ikinci fazda boyut özelliklerini güncellemek için fazladan sadece $O(1)$ zaman harcarız. Böylece, n düğümlü bir sıra-istatistiği ağacına araya sokma yapmak için gereken toplam süre $O(\lg n)$ dir ve bu sıradan kırmızı-siyah ağaçta gereken süreyle hemen hemen aynıdır.

Kırmızı-siyah ağaçtan silme işlemi de iki fazdan oluşur: birinci faz temel arama ağacı üzerinde çalışır ve ikinci faz en fazla 3 döndürmeye neden olur, bunun dışında yapısal değişiklikler gerçekleştirmez (Kısım 13.4 e göz atınız). Birinci faz, ya ağaçtan bir y düğümünü kaldırır ya da onu ağacın içinde daha yukarı taşır. Altağacın boyutlarını güncellemek için, y düğümünden başlangıç noktasına



Şekil 14.2 Döndürmeler sırasında altağaçların güncellenmesi. Döndürdüğümüz bağı (link) etrafı, boyut özellikleri güncellenmesi gereken iki düğüme bağlıdır. Güncellemeler yereldir, sadece x, y de depolanan boyut bilgisini ve üçgenlerde gösterilen altağaçların köklerini bilmemiz gerekir.

(köke kadar) üzerindeki her düğümün boyut özelliğini azaltarak basit bir yol çizeriz (düğümün ağacın içindeki ilk pozisyonundan başlarız). n düğümlü bir kırmızı-siyah ağaçta bu yolun uzunluğu $O(\lg n)$ olduğundan, birinci fazda boyut özelliklerini korumak için harcanan ekstra zaman $O(\lg n)$ dir. Araya sokma işleminde olduğu gibi ikinci fazdaki silme işlemindeki $O(1)$ döndürmelerinin üstesinden gelebiliriz. Böylece, n düğümlü bir kırmızı siyah ağaç için, boyut özelliklerini koruma işlemini içeren silme ve araya sokma, $O(\lg n)$ zamanda gerçekleşir.

Alıştırmalar

14.1-1

Şekil 14.1 deki T sıra-istatistiği ağacında $\text{OS-SELECT}(T.\text{root}, 10)$ un nasıl çalıştığını gösteriniz.

14.1-2

Şekil 14.1 deki T sıra-istatistiği ağacında, $x.\text{key} = 35$ olan x düğümünde, $\text{OS-RANK}(T, x)$ in nasıl çalıştığını gösteriniz.

14.1-3

OS-SELECT in özyinelemeli olmayan versiyonunu yazınız.

14.1-4

Girişini T sıra-istatistiği ağacından alan ve anahtarı k olan, T ile gösterilen dinamik kümede k rankını döndüren, özyinelemeli bir $\text{OS-KEY-RANK}(T, k)$ prosedürü yazınız. T nin anahtarlarının farklı olduğunu varsayınız.

14.1-5

n düğümlü bir sıra-istatistiği ağacında bir x elemanı ve doğal sayı olan i verilmiş olsun, doğrusal sıralı bir ağaçta $O(\lg n)$ zamanda x ten sonra gelen i . elemanı nasıl belirleyebiliriz?

14.1-6

OS-SELECT veya OS-RANK ta bir düğümü referans aldığımızda, onu sadece rankı hesaplamak için ne zaman kullanabileceğimizi gözlemleyiniz. Buna göre, kendi başlangıç altağacında her düğümün kendi rankının saklandığını varsayalım. Silme ve araya sokma sırasında bu bilgileri nasıl koruyacağımızı gösteriniz (Bu iki işlemin döndürmeye sebep olduğunu hatırlayınız).

14.1-7

$O(n \lg n)$ zamanda n boyutlu bir dizide ters çevirmelerin sayısını hesaplamak için sıra-istatistiği ağacının nasıl kullanılacağını gösteriniz (Problem 2-4 e göz atınız).

14.1-8 *

Her biri kendi uç noktalarından tanımlanan n kirişli bir çember düşünelim. Çemberi iç bölgesinde kesen kiriş çiftlerinin sayısını belirlemek için gereken $O(n \lg n)$ zaman algoritması tanımlayınız (Örnek olarak, eğer n tane kiriş merkezde bulunan çaplar oluşturuyor ise, doğru cevap olacaktır). İki kirişin aynı uç noktayı paylaşmadığını varsayalım.

14.2 Bir veri yapısı nasıl genişletilir

Temel bir veri yapısının genişletilmesi işlemi, algoritma tasarımında ekstra işlevsellik oluşturmak için sıklıkla yapılır. Gelecek bölümde, aralıklarda işlem yapmayı destekleyen bir veri yapısı tasarlamak için bu işlemi tekrar kullanacağız. Bu bölümde, bu tip genişletme işlemiyle ilgili adımları inceleyeceğiz. Ayrıca çoğu durumda kırmızı-siyah ağaçları kolayca genişletmemize imkan sağlayacak bir teoremi de göstereceğiz.

Veri yapısının güçlendirmesi işlemini dört adıma bölebiliriz:

1. Temel bir veri yapısı seçin.
2. Temel veri yapısındakileri güncellemek için gereken ek bilgileri belirleyin.
3. Temel veri yapısı üzerinde ayarlanmış temel işlemler için ek bilgileri güncelleyebileceğimizi doğrulayın.
4. Yeni işlemler geliştirin.

Her kurallı tasarım yönteminde olduğu gibi, yukarda verilen adımları körü körüne izlememelisiniz. Tasarım işlemlerinin çoğu bir deneme ve hata elemanı içerir. Basamaklardaki işlemler bir yandan da devam eder. Örneğin, eğer ek bilgileri gerektiği şekilde koruyamazsak, ek bilgileri tanımlamanın ve yeni işlemler geliştirmenin (2 ve 4. basamaklar) anlamı yoktur. Yine de, bir veri yapısını güçlendirmeye çalıştığınızda bu dört adım sizin için iyi bir odak noktası oluşturabilir. Ayrıca, genişletilmiş bir veri yapısını belgelemek için de iyi bir yoldur.

Kısım 14.1 de sıra-istatistiği ağaçlarımızı tasarlamak için bu adımları takip ettik. Adım 1 için, temel veri yapısı olarak kırmızı-siyah ağaçları seçtik. Kırmızı-siyah ağaçların uygunluğuna bir işaret olarak, diğer dinamik küme işlemlerini verimli bir şekilde desteklemelerini gösterebiliriz. Örnek olarak, MINIMUM, MAXIMUM, SUCCESSOR ve PREDECESSOR verilebilir.

Adım 2 için, x ten başlayan altağacın boyutunu saklayan her x düğümüne boyut özelliğini ekledik. Genellikle ek bilgi, işlemleri daha verimli hale getirir. Örneğin; OS-SELECT ve OS-RANK ı sadece ağaçta saklanan anahtarları kullanarak uygulayabilmıştık fakat $O(n \lg n)$ zamanda çalışmayacaklardı. Ek bilgi zaman zaman bir veriden ziyade Örnek 14.1-2 de olduğu gibi bir işaretçi bilgidir.

Adım 3 için, hâlâ $O(n \lg n)$ zamanda çalışırken silme ve araya sokmanın boyut özelliklerini koruyabilmesini sağladık. İdealde, ek bilgiyi korumak için veri yapısının sadece birkaç elemanını güncellemeye ihtiyaç duymamız gerekir. Örnek olarak, eğer basitçe ağaçtaki her düğümün rankını kendisinde saklasaydık, OS-SELECT ve OS-RANK prosedürleri hızlıca çalışacaktı, ancak en az bir yeni eleman eklemek ağacın her düğümdeki bilginin değişmesine sebep olacaktı. Onun yerine altağaçtaki boyutları sakladığımızda, yeni bir eleman eklemek sadece $O(\lg n)$ tane düğümdeki bilginin değişmesine sebep olacaktır.

Adım 4 için, OS-SELECT ve OS-RANK işlemlerini geliştirdik. Buna rağmen, yeni işlemlere olan ihtiyaç bir veri yapısını çoğaltırken zorlanmamızın en temel sebebidir. Bazen de yeni işlemler geliştirmekten ziyade, Alıştırma 14.1-2 de olduğu gibi, mevcut olanları geliştirmek için ek bilgiyi kullanırız.

Kırmızı-siyah ağaçların (red-black trees) genişletilmesi

Genişletilmiş bir veri yapısının alt yapısını kırmızı-siyah ağaçlar oluşturduğunda, silme ve araya sokmanın belirli türlerdeki ek bilgileri her zaman etkili bir şekilde koruyabileceğini kanıtlayabiliriz, böylece Adım 3 kolayca yapılır. Aşağıdaki teoremin kanıtı, Kısım 14.1 de işlediğimiz sıra-istatistiği ağaçları için boyut özelliğini koruyabilmemizle ilgili olan konuyla benzerdir.

Teorem14.1 (Kırmızı-siyah ağaçların genişletilmesi)

f, n düğümlü bir T kırmızı-siyah ağaçları genişleten bir özellik olsun ve her x düğümündeki f in değerinin, x düğümlerindeki tek bilgi olan, belki $x.left.f$ ve $x.right.f$ i de içeren, $x.left$ ve $x.right$ a bağlı olduğunu varsayalım. O halde, T nin bütün düğümlerindeki f değerlerini, silme ve araya sokma sırasında bu işlemlerin $O(\lg n)$ performansında asimptotik bir etki olmadan güncelleyebiliriz.

Kanıt Kanıtın ana fikri, x düğümündeki f özelliğinde bir değişme olduğunda, bunun yalnızca x in ağaçtaki atalarına yayılmasıdır. Şöyle ki; $x.f$ yi değiştirmek için $x.p.f$ yi güncelleştirmek gerekebilir, ama yalnızca; $x.p.f$ yi güncelleştirmek $x.p.p.f$ yi güncelleştirmeyi gerektirir ve bu ağacın tepesine kadar böyle devam eder. Daha

öncesinde $T.root.f$ i güncelleştirmiştik, başka bir düğüm yeni değere bağlı olacak, ve işlem sonlanacaktır. Kırmızı-siyah ağacın yüksekliği $O(\lg n)$ olduğundan, bir düğümdeki f özelliğini değiştirmek, değişime bağlı tüm düğümleri güncelleştirirken $O(\lg n)$ zamanda gerçekleşir.

T nin içerisine x node u ekleme işlemi iki fazdan oluşur (Kısım 13.3 e göz atınız). İlk faz x i mevcut olan bir $x.p$ düğümüne yeni bir çocuk olarak ekler. $x.f$ in değerini tahminen $O(1)$ zamanda hesaplayabiliriz, bu yalnızca x üzerindeki diğer özelliklerin bilgisine ve x in sibling lerinin bilgisine bağlıdır, ancak x in çocuklarının ikisi de $T.nil$ nöbetçisidir. $x.f$ yi daha önce hesaplamıştık, değişim ağacın tepesine doğru yayılır. Böylece, araya sokmanın birinci fazı toplamda $O(\lg n)$ zamanda gerçekleşir. İkinci faz boyunca, ağacın tek yapısal değişikliği döndürmelerden kaynaklanır. Bir döndürmede sadece iki düğüm değişebildiğinden, f özellikleri için toplam güncelleştirme süresi her döndürme için $O(\lg n)$ dir. Araya sokma sırasındaki döndürmelerin sayısı en fazla iki olacağı için, araya sokma için toplam süre $O(\lg n)$ dir.

Araya sokmada olduğu gibi, silme de iki fazdan oluşur (Kısım 13.4 e göz atınız). Birinci fazda, silinen düğüm ağaçtan kaldırıldığı zaman ağaçta değişiklikler meydana gelir. Silinen düğüm o anda iki kardeşe sahipse, bu durumda onun ardılı silinen düğümün yerini alır. Bu değişikliklerden kaynaklanan güncelleştirmelerin f e doğru yayılması, en fazla $O(\lg n)$ zaman alır çünkü değişiklikler ağacın belli bölgelerini değiştirir. İkinci faz boyunca kırmızı-siyah ağacı düzenlemek, en fazla üç döndürme gerektirir ve her döndürme güncelleştirmelerin f e doğru yayılması için en fazla $O(\lg n)$ zamana ihtiyaç duyar. Bu durumda, araya sokmada olduğu gibi silmenin de toplam süresi $O(\lg n)$ dir. ■

Çoğu durumda, sıra-istatistiği ağaçlarında boyut özelliklerinin korunması gibi, Teorem 14.1 de türetilen $O(\lg n)$ den çok, bir döndürmenin alacağı zaman $O(1)$ dir. Alıştırma 14.2-3 buna örnek verilebilir.

Alıştırmalar

14.2-1

Düğümlere işaretçiler ekleyerek, genişletilmiş bir sıra-istatistiği ağacında en az $O(1)$ zaman içinde, dinamik küme sorguları olan MINIMUM, MAXIMUM, SUCCESSOR ve PREDECESSOR dan her birinin nasıl destekleneceğini gösteriniz. Diğer işlemlerin sıra-istatistiği ağaçları üzerindeki asimptotik performansının etkilenmemesi gerekir.

14.2-2

Bir kırmızı-siyah ağaçtaki düğümlerin siyah-yüksekliklerini, ağacın düğümlerindeki özellikleri olarak, hiçbir kırmızı-siyah ağaç işleminin asimptotik performansına etki etmeden güncelleyebilir miyiz? Nasıl olacağını gösteriniz ya da neden olmayacağını belirtiniz. Düğümlerin derinliği nasıl güncellenebilir?

14.2-3 *

“ $\otimes$ ” bir birleşmeli ikili operatör ve a kırmızı-siyah ağacın her düğümünde güncellenen bir özellik olsun. Her x düğümünün ek bir özellik olan f i içermesini istediğimizi varsayalım. Öyle ki; $x.f = x_1.a \otimes x_2.a \otimes \dots \otimes x_m.a$, ve $x_1, x_2, \dots, x_m, x$ den başlayan altağaçta sıralanmış düğümleri olsun. $O(1)$ zamanda bir döndürmeden sonra f özelliklerini nasıl güncelleyebileceğimizi gösteriniz. Sıra-istatistiği ağaçlarındaki boyut özelliklerine uygulamak üzere değişkeninizi basitçe düzenleyiniz.

14.2-4 *

Kırmızı-siyah ağaçları, x ten başlayan ağaçta bütün k anahtarlarının çıktısını $a \leq k \leq b$ olacak şekilde alan bir işlem olan RB-ENUMERATE(x, a, b) işlemi ile genişletmek istiyoruz. RB-ENUMERATE yi $\Theta(m + \lg n)$ zamanda nasıl uygulayacağımızı belirtiniz. Buradaki m , çıktındaki anahtarların sayısı ve n ağacın iç kısmındaki düğümlerinin sayısıdır (*İpucu*: ağaca yeni özellikler eklemenize gerek yoktur).

14.3 Aralık ağaçları

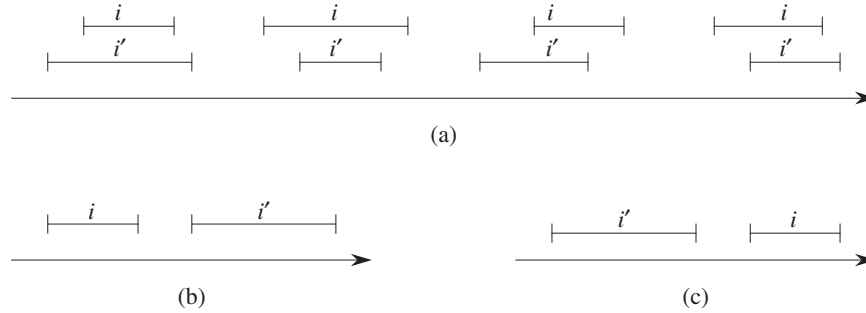
Bu bölümde, kırmızı-siyah ağaçları (red-black trees) aralıkların dinamik kümeleri üzerindeki işlemlerini desteklemek için güçlendireceğiz. **Kapalı aralık**, düzgün sıralanmış bir çift reel sayıyla $[t_1, t_2]$ şeklinde ifade edilir ve $t_1 \leq t_2$ olmalıdır. $[t_1, t_2]$ aralığı; $\{t \in \mathbb{R} : t_1 \leq t \leq t_2\}$ kümesini temsil eder. **Açık ve yarı-açık** aralıklar, sırasıyla iki uç noktayı ve tek uç noktayı kümeye dahil etmezler. Bu bölümde, aralıkların kapalı olduğunu varsayacağız; sonuçları açık ve yarı-açık aralıklara genişletmek kavramsal olarak basittir.

Aralıklar, zamanın belirli periyotlarında meydana gelen olayları temsil etmek için uygundur. Örneğin, zaman aralıkları veri tabanını, verilen aralıkta hangi olayın meydana geldiğini bulmak için sorgulamak isteyebiliriz. Bu bölümdeki veri yapısı, böyle bir aralık veri yapısının korunması için oldukça kolaylık sağlar.

$[t_1, t_2]$ aralığını, $i.low = t_1$ (**düşük uç nokta**) ve $i.high = t_2$ (**yüksek uç nokta**) özelliklerine sahip bir i nesnesi olarak gösterebiliriz. Eğer $i \cap i' \neq \emptyset$ ise, yani $i.low \leq i'.high$ ve $i'.low \leq i.high$ ise i ile i' örtüşür diyebiliriz. Şekil 14.3 te gösterildiği gibi, aşağıdaki üç özellikten birini tam olarak sağlayan herhangi iki i ve i' aralığı **aralığı üçe ayırmaya** uyum sağlar:

- i ve i' örtüşür,
- i, i' nün solundadır (örneğin; $i.high < i'.low$),
- i, i' nün sağındadır (örneğin; $i'.high < i.low$).

Aralık ağacı, elemanların dinamik kümelerini koruyan bir kırmızı-siyah ağaçtır ve her x elemanı $x.int$ aralığını içerir. Aralık ağaçları aşağıdaki işlemleri destekler:



Şekil 14.3 i ve i' kapalı aralıkları için aralığı üçe ayırma. (a) eğer i ve i' örtüşüyorsa, dört durum söz konusudur; her birinde, $i.low \leq i'.high$ ve $i'.low \leq i.high$. (b) Aralıklar örtüşmez ve $i.high \leq i'.low$. (c) Aralıklar örtüşmez ve $i'.high \leq i.low$.

INTERVAL-INSERT (T, x); int özelliğinin bir aralık içerdiğini kabul eden x elemanını T aralık ağacına ekler.

INTERVAL-DELETE (T, x); x elemanını T aralık ağacından kaldırır.

INTERVAL-SEARCH (T, i); $x.int$ ile i aralığının örtüştüğü T aralık ağacındaki x elemanına bir işaretçi döndürür, eğer kümede böyle bir eleman yoksa nöbetçi $T.nil$ e bir işaretçi döndürür.

Şekil 14.4 te aralıklar kümesinin bir aralık ağacı ile nasıl gösterildiği yer almaktadır. Kısım 14.2 deki dört-adım yöntemini izleyerek aralık ağacının tasarımını ve üzerinde yapabileceğimiz işlemleri özetleyeceğiz.

Adım 1: Temel veri yapısı

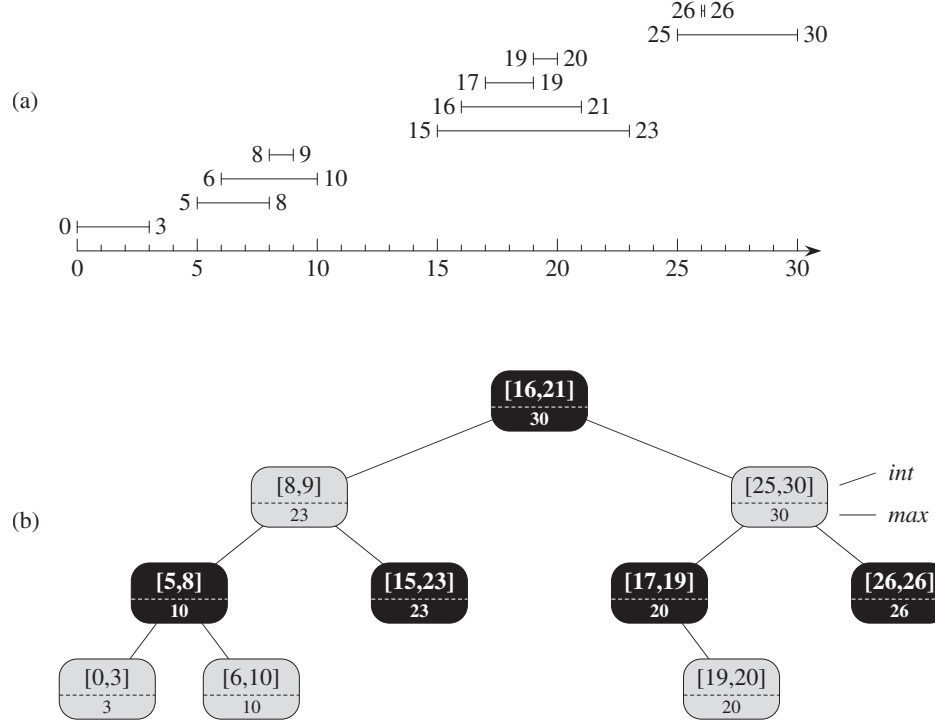
Her x düğümünde bir $x.int$ aralığı içeren ve x in anahtarının aralığın düşük uç noktasının, $x.int.low$ olduğu bir kırmızı-siyah ağaç seçeriz. Böylece, veri yapısının düzenli ağaç adımı düşük uç noktaya göre düzenlemiş aralıkları listeler.

Adım 2: Ek bilgi

Aralıkların kendilerine ek olarak, her x düğümü bir $x.max$ değeri içerir ve bu değer, x ten başlayan altağaçta depolanan herhangi bir aralığın uç noktasının maksimum değeridir.

Adım 3: Bilginin güncellenmesi

n düğümlü bir aralık ağacında, araya sokma ve silmenin $O(\lg n)$ zaman aldığını denetlememiz gerekir. $x.max$ ı, verilen aralıkta $x.int$ ı ve x in çocuklarının maksimum değerlerini belirleyebiliriz:



Şekil 14.4 Bir aralık ağacı. (a) En alttan tepeye doğru sol uç noktalara göre düzenlenmiş 10 aralıktan oluşan bir küme gösterilmiştir. (b) Aralık ağacı onları temsil etmektedir. Kesik çizgili hattın üzerinde gösterilen her x düğümü bir aralık içerir ve x ten başlayan altağaçtaki herhangi bir aralığın maksimum değeri kesik çizgili hattın altında gösterilir. Düzenli ağaç adımı, düğümleri sol uç noktalara göre düzenlenmiş olarak listeler.

$$x.max = \max(x.int.high, x.left.max, x.right.max)$$

Bu durumda, Teorem 14.1 e göre, araya sokma ve silme $O(\lg n)$ zaman boyunca çalışır. Aslında, Alıştırma 14.2-3 ve Alıştırma 14.3-1 de gösterildiği şekilde, $O(1)$ zamanda bir döndürmeden sonra max özelliklerini güncelleştirebiliriz.

Adım 4: Yeni işlemlerin geliştirilmesi

İhtiyaç duyduğumuz tek yeni işlem, aralığı i aralığıyla örtüşen T ağacındaki bir düğümü bulan $INTERVAL-SEARCH(T, x)$ dır. Eğer ağaçta i ile örtüşen hiçbir aralık yoksa, prosedür nöbetçi $T.nil$ e bir işaretçi döndürür.

INTERVAL-SEARCH(T, x)

```

1   $x = T.root$ 
2  while  $x \neq T.nil$  ve  $i$  ile  $x.int$  örtüşmez
3      if  $x.sol \neq T.nil$  ve  $x.left.max \geq i.low$ 
4           $x = x.left$ 
5      else  $x = x.right$ 
6  return  $x$ 

```

i ile örtüşen bir aralık araştırması ağacın kökü olan x ile başlar ve aşağı doğru ilerler. Örtüşen bir aralık bulunduğunda ya da x nöbetçi $T.nil$ e bir işaretçi gönderdiğinde işlem sonlanır. Temel döngünün her bir iterasyonu $O(1)$ zaman aldığından ve n düğümlü bir kırmızı-siyah ağacın yüksekliği $O(\lg n)$ olduğundan, INTERVAL-SEARCH prosedürü $O(\lg n)$ zaman alır.

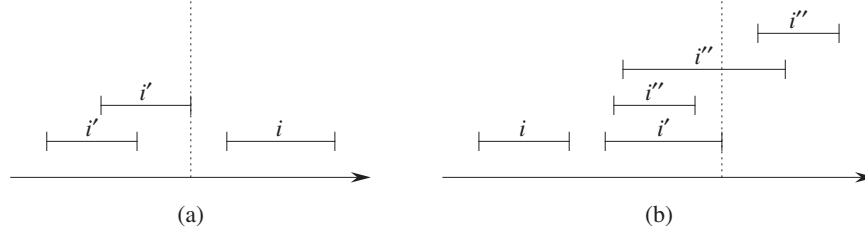
INTERVAL-SEARCH ın uygun olduğuna bakmadan önce, Şekil 14.4 teki aralık ağacı üzerinde nasıl çalıştığını bir inceleyelim. $i = [22, 25]$ aralığı ile örtüşen bir aralık bulmak istediğimizi varsayalım. i ile örtüşmeyen ve $[16, 21]$ i içeren x i kök olarak başlayalım. $x.left.max = 23$ $i.low = 22$ den büyük olduğu için, döngü $[8, 9]$ u içeren düğümün-kökün sol çocuğu olan x ile devam eder ve yine i ile örtüşmez. Bu sefer, $x.left.max = 10$ dur ve $i.low = 22$ den küçüktür, dolayısıyla döngü yeni x olarak x in sağ çocuğunu kabul ederek devam eder. Bu düğümde depo edilen $[15, 25]$ aralığı i ile örtüştüğü için, prosedür bu düğümü döndürür.

Başarısız bir araştırma örneği olarak, Şekil 14.4 teki aralık ağacında $i = [11, 14]$ aralığıyla örtüşmesini istediğimiz bir aralık bulmak istediğimizi varsayalım. Bu kez de x i kök olarak başlayalım. Kökün aralığı olan $[16, 21]$ i ile örtüşmeyeceği ve $x.left.max = 23$ $i.low = 11$ den daha büyük olduğu için, $[8, 9]$ aralığını içeren düğüm için sola gideriz. $[8, 9]$ aralığı i ile örtüşmez ve $x.left.max = 10$ $i.low = 11$ den daha küçüktür, dolayısıyla sağa gideriz (Sol altağaçta i ile örtüşen hiçbir aralık olmadığına dikkat ediniz). $[15, 23]$ aralığı i ile örtüşmez ve onun sol çocuğu $T.nil$ dir, bu yüzden yine sağa gideriz, döngü sonlanır ve nöbetçi $T.nil$ i döndürürüz.

INTERVAL-SEARCH ın neden uygun olduğunu görmek için, kökten çizilmiş tek bir yolu incelerken onun neden yeterli olduğunu anlamamız gerekir. Ana fikir şöyledir: herhangi bir x düğümü, eğer $x.int$ ile i örtüşmüyorsa, araştırma şu yolda muhakkak ilerler: eğer ağaç örtüşen bir aralık içeriyorsa araştırma onu kesinlikle bulacaktır. Aşağıdaki teorem bu özelliği daha net bir şekilde açıklamaktadır.

Teorem 14.2

INTERVAL-SEARCH (T, i) nin herhangi bir uygulaması, hem aralığı i ile örtüşen bir düğüm noktası döndürür, hem de T ağacı i ile örtüşen bir düğüm içermediğinde $T.nil$ i döndürür.



Şekil 14.5 Teorem 14.2 deki aralıkların kanıtı. $x.left.max$ ın değeri her durumda kesik çizgili olarak gösterilmiştir. (a) Arama sağa gider. x in sol altağacında i ile örtüşebilen hiçbir i' aralığı yoktur. (b) Arama sola gider. x in sol altağacı i ile örtüşen bir aralık içerir (bu durum gösterilmemiştir) ya da x in sol altağacı $i'.high = x.left.max$ gibi bir i' aralığı içerir. i ile i' örtüşmediği için, aynı zamanda x in sağ altağacındaki herhangi bir i'' aralığıyla da örtüşmez çünkü $i'.low \leq i''.low$.

Kanıt 2-5 satırlarındaki **while** döngüsü, $x = T.nil$ olduğunda ya da i ile $x.int$ örtüştüğünde sonlanır. İkinci durumda, x e dönmesi kesinlikle doğrudur. Bu nedenle, $x = T.nil$ olması nedeniyle döngünün son bulduğu ilk durum üzerine odaklanacağız.

2-5 satırlarındaki **while** döngüsü için aşağıdaki değişmezi kullanırız:

Eğer T ağacı i ile örtüşen bir aralık içeriyorsa, o zaman x ten başlayan altağaç böyle bir aralık içerir.

Bu döngü değişmezini aşağıdaki gibi kullanırız:

Başlangıç: Öncelikle ilk iterasyon, 1. satır x i T ağacının başlangıcı olarak ayarlar, onun için değişmez tutulur.

Yineleme: **while** döngüsünün her bir iterasyonu ya 4. ya da 5. satır çalıştırır. Her iki durumun da döngü değişmezinde korunduğunu gösterelim.

Eğer 5. satır çalıştırıldıysa, buna göre 3. satırdaki dallanma koşulundan dolayı, $x.left = T.nil$ ya da $x.left.max < i.low$ elde ederiz. Eğer $x.left = T.nil$ olursa, $x.left$ dan başlayan altağaç hiç şüphesiz i ile örtüşen bir aralık içermeyecektir, dolayısıyla x i $x.right$ şeklinde ayarlamak değişmezi koruyacaktır. Bu nedenle $x.left \neq T.nil$ ve $x.left.max < i.low$ olduğunu varsayalım. Şekil 14.5(a) da gösterildiği gibi, x in sol altağacındaki her i' aralığı için;

$$\begin{aligned} i'.high &\leq x.left.max \\ &< i.low \end{aligned}$$

elde ederiz. Aralığı üçe ayırmaya göre, i ile i' bu nedenle örtüşmez. Böylece, x in sol altağacı i ile örtüşen bir aralık içermez, dolayısıyla x i $x.right$ a ayarlamak değişmezi korur.

Diğer yandan, eğer 4.satır çalıştırılırsa, o zaman döngü değişmezinin devriğinin tutulacağını göstereceğiz. Yani, eğer $x.left$ dan başlayan altağaç i ile örtüşen bir aralık içermiyorsa, bu durumda ağacın hiçbir yerinde i ile örtüşen bir aralık yoktur. 4.satır çalıştırıldığı için, 3.satırdaki dallanma koşulundan ötürü, $x.left.max \geq i.low$ elde ederiz. Dahası, max özelliğinin tanımından dolayı, x in sol altağacı (aşağıda gösterildiği gibi) bazı i' aralıkları içermelidir:

$$\begin{aligned} i'.high &= x.left.max \\ &\geq i.low \end{aligned}$$

(Şekil 14.5(b) bu durumu örneklemektedir). i ile i' örtüşmediğinden ve $i'.high < i.low$ doğru olmadığından, arama aralığı üçe ayırmaya göre $i.high < i'.low$ eşitsizliğini takip eder. Aralık ağaçları aralıkların düşük uç noktaları üzerinde anahtarlandırılırlar ve buna göre arama ağacı özelliği, x in sağ altağacındaki herhangi bir i'' aralığı için

$$\begin{aligned} i.high &< i'.low \\ &\leq i''.low \end{aligned}$$

olduğunu ifade eder. Aralığı üçe ayırmaya göre, i ile i'' örtüşmez, x in sol altağacındaki herhangi bir aralığın i ile örtüşüp örtüşmediği, x i $x.right$ a ayarlananın değişmezi koruyacağı çıkarımlarında bulunabiliriz.

Sonlandırma: Eğer $x = T.nil$ olduğunda döngü sonlanıyorsa, bu durumda x ten başlayan altağaçta i ile örtüşen bir aralık yoktur. Döngü değişmezinin devriği T nin i ile örtüşen bir aralık içermediğini ifade eder. Bundan dolayı, $x = T.nil$ e dönmek doğru olacaktır. ■

Böylece INTERVAL-SEARCH düzgün bir şekilde çalışır.

Alıştırmalar

14.3-1

Bir aralık ağacı üzerindeki düğümler üzerinde çalışan ve $O(1)$ zaman içindeki max özelliklerini artıran LEFT-ROTATE için bir kabakod yazınız.

14.3-2

Bütün aralıklar açık aralık olduğunda düzgün bir şekilde çalışabilmesi için INTERVAL-SEARCH kodunu yeniden yazınız.

14.3-3

Verilen bir i aralığı için, minimum düşük uç noktaya sahip ve i ile örtüşen bir aralık döndüren ya da böyle bir aralık yoksa $T.nil$ döndüren etkili bir algoritma geliştiriniz.

14.3-4

Bir T aralık ağacı ve i aralığı verilmiş olsun, k çıktı listesindeki aralıkların sayısı olmak üzere $O(\min(n, k, \lg n))$ zamanda T ağacındaki i aralığıyla örtüşen bütün aralıkları nasıl listeleyebileceğimizi belirtiniz (*İpucu*: Basit bir yöntem birçok sorgu oluşturacaktır, o da ağaçla sorgular arasını ayarlayacak. Biraz daha karışık bir yöntem ağacı düzenlemeyecektir).

14.3-5

T bir aralık ağacı ve i bir aralık olmak üzere, yeni bir işlem olan INTERVAL-SEARCH-EXACTLY(T, i) yi desteklemesi için aralık ağacı prosedürlerinde yapılabilecek değişiklikleri belirtiniz. Bu işlem, $x.int.low = i.low$ ve $x.int.high = i.high$ olduğunda T deki x düğümüne ya da T böyle bir x içermiyorsa $T.nil$ e bir işaretçi göndermelidir. INTERVAL-SEARCH-EXACTLY dahil tüm şlemler n düğümlü bir aralık ağacı üzerinde $O(\lg n)$ zaman içinde çalışmalıdır.

14.3-6

MIN-GAP işlemini destekleyen bir küme olan, Q dinamik sayı kümesini nasıl koruyabileceğimizi gösteriniz. MIN-GAP, Q daki birbirine en yakın iki sayının farkının büyüklüğünü veren bir işlemdir. Örneğin, $Q = \{1, 5, 9, 15, 18, 22\}$ ise, MIN-GAP $18-15 = 3$ ü döndürür çünkü 15 ile 18 Q daki birbirine en yakın sayılardır. MIN-GAP, INSERT, DELETE ve SEARCH işlemlerinin olabildiğince etkili bir şekilde yapınız ve bunların çalışma sürelerini analiz ediniz.

14.3-7 *

VLSI veritabanları genellikle bir entegre devreyi dikdörtgenlerden oluşan bir liste şeklinde ifade eder. Her dikdörtgenin doğrusal bir şekilde yönlendirildiğini varsayalım (kenarları x ve y eksenlerine paralel). Bu durumda bir dikdörtgeni maksimum ve minimum x - y koordinatlarına göre ifade ederiz. Örtüşen iki dikdörtgen içeren, n kadar dikdörtgenin gösterildiği bir kümenin olup olmadığını belirlemek için bir $O(n \lg n)$ -zaman algoritması geliştiriniz. Algoritmanızın bütün kesişen çiftleri göstermesine gerek yoktur, ancak eğer varsa, bir dikdörtgen diğerini tamamen kaplıyorsa, sınır çizgileri kesişmiyor olsa bile algoritmanız bunu bildirmelidir (*İpucu*: Dikdörtgenler kümesinin karşına bir “tarama” çizgisi taşıyın).

Problemler
14-1 Maksimum çakışma noktası

Varsayalım ki aralıklar kümesinde bir **maksimum çakışma noktası** ’nın izini korumak istiyoruz.—kümedeki aralıkların en büyük sayısıyla birlikte onunla çakışan bir nokta.

- a. Parçalardan birinin uç noktasının her zaman bir maksimum çakışma noktası olacağını gösteriniz.

- b.** Maksimum çakışma noktasını döndüren, INTERVAL-INSERT, INTERVAL-DELETE ve FIND-POM işlemlerini verimli bir şekilde destekleyen bir veri yapısı tasarlayınız (*ipucu*: Bütün uç noktalar için bir kırmızı-siyah ağaç tutun. Her sol uç noktayı $+1$ değeri ile ilişkilendirin ve her sağ uç noktayı -1 değeri ile ilişkilendirin. Maksimum çakışma noktasını korumak için ağacın her düğümü bazı ek bilgilerle genişletin).

14-2 Josephus permütasyonu

Josephus problemi ni aşağıda gösterildiği gibi tanımlarız. n kişiden oluşan bir insan çemberi olduğunu ve bize $m \leq n$ olacak şekilde pozitif tam sayılar verildiğini varsayalım. Seçilmiş ilk insanla başlayalım, her m inci insanı kaldırarak çemberin etrafında ilerleyelim. Her insan kaldırıldıktan sonra, çemberin etrafında kalanları sayma işlemi devam eder. Bu işlem n kişiyi kaldırmış oluncaya kadar bu şekilde devam eder. Çemberden kaldırılan insanların sırası 1 den n e kadar olan tam sayıların **(n, m) -Josephus permütasyonu** ile belirlenir. Örneğin, $(7, 3)$ -Josephus permütasyonu $(3, 6, 2, 7, 5, 1, 4)$ dir.

- a.** m nin bir sabit olduğunu varsayınız. Verilen bir n tam sayısı için, çıkış olarak (n, m) -Josephus permütasyonunu veren bir $O(n)$ -zaman algoritması geliştiriniz.
- b.** m nin bir sabit olmadığını varsayınız. Verilen n ve m tamsayıları için, çıkış olarak (n, m) -Josephus permütasyonunu veren bir $O(n \lg n)$ -zaman algoritması geliştiriniz.

Bölüm notları

Kendi kitaplarında Preparata ve Shamos [282], H. Edelsbrunner (1980) ve E. M. McCreight (1981) den alıntılar yaparak yazılı kaynaklarda görülen birçok aralık ağacını tanımlamışlardır. Kitap, $O(k + \lg n)$ zamanda verilen bir sorgu aralığıyla çakışan bütün k aralıklarının numaralandırmamıza imkân veren, n tane aralığın durağan veri tabanı verilmiş, bir aralık ağacını ayrıntılı bir şekilde anlatmıştır.



IV İleri Tasarım ve Analiz Teknikleri

Giriş

Bu bölüm, verimli algoritmaların analiz ve tasarımında kullanılan üç önemli tekniği kapsamaktadır: dinamik programlama (Bölüm 15), açgözlü algoritmalar (Bölüm 16) ve amortize analiz (Bölüm 17). Daha önceki bölümler, böl-ve-yönet, randomizasyon ve özyinelemelerin nasıl çözüleceği gibi diğer geniş uygulanabilir teknikleri sunmuştur. Bu bölümdeki teknikler daha gelişmiştir, fakat birçok hesaplama problemi ile uğraşmak için bize yardımcı olurlar. Bu bölümde anlatılan şablonlara, kitabın daha sonraki bölümlerinde atıf yapılacaktır.

Dinamik programlama, genellikle, bir optimal çözüme ulaşmak için seçimlerin bir kümesine sahip olduğumuz optimizasyon problemlerine uygulanır. Her seçim yaptığımızda, aynı formdaki altproblemler artar. Verilen bir altproblem seçimlerin kısmi bir kümesinden öne çıktığında dinamik programlama verimlidir; anahtar yöntem, tekrar ortaya çıkacak diye her bir altproblem sonucunu saklamaktır. Bölüm 15, bu basit fikrin nasıl bazen üstel-zamanlı algoritmaların polinom-zamanlı algoritmalara dönüştüğünü gösterir.

Dinamik programlama algoritmaları gibi, açgözlü algoritmalar genellikle, bir optimal çözüme ulaşmak için seçimlerin bir kümesine sahip olduğumuz, optimizasyon problemlerine uygulanır. Bir açgözlü algoritmanın mantığı, her seçimi yerel optimal davranışta yapmaktır. Basit bir örnek, madeni para bozdurmadır: istenen miktarda değişim için gerekli madeni paraların sayısını minimize etmek üzere, kalanların miktarından daha büyük olmayacak şekilde en büyük değerli madeni parayı tekrar tekrar seçebiliriz. Bir açgözlü yaklaşımın verimli olup olmayacağını her zaman kolayca söyleyemeyiz. Bölüm 16, bir açgözlü algoritma-

nın bir optimal çözüm oluşturduğunu göstermemize yardım edecek bir matematiksel kural sağlayan, matroid teoriye giriş yapar.

Benzer işlemlerin bir serisini çalıştırdığımız kesin algoritmaları analiz etmek için amortize analizi kullanırız. İşlemler dizisinin maliyetini sınırlamak yerine ayrı olarak her bir işlemin asıl maliyetini sınırlayarak, bir amortize analiz mevcut serinin asıl maliyetinde bir sınır oluşturur. Bu yöntemin bir avantajı, bazı işlemler pahalı olmasına rağmen, diğerlerinin ucuz olmasıdır. Bir başka deyişle, en kötü durumda birçok işlem iyi çalışabilir. Amortize analiz, sadece bir analiz aracı değildir; bir algoritmanın tasarımı ve çalışma zamanının analizinin iç içe geçmesinden dolayı, algoritmaların tasarımı hakkında bir düşünme yoludur. Bölüm 17, bir algoritmanın bir amortize analizini çalıştırmak için üç yol sunmaktadır.

Dinamik programlama (Dynamic programming), böl-ve-yönet (divide-and-conquer) yöntemi gibi, altproblemlerin çözümlerini birleştirerek problemleri çözer. (“Programlama” bu içerikte, bilgisayar kodunu yazmak değil, bir listeleme yöntemini ifade eder.) Bölüm 2 ve 4 te gördüğümüz gibi, böl-ve-yönet algoritmaları bir problemi ayrık altproblemlere parçalar, altproblemleri özyinelemeli olarak çözer ve bunların çözümlerini asıl problemi çözmek için birleştirir. Bunun aksine, dinamik programlama altproblemler çakıştığında, yani, altproblemler alt altproblemleri ortak kullandığında etkili olur. Bu anlamda, böl-ve-yönet algoritması, tekrar tekrar ortak alt altproblemleri çözerek, gerekenden daha fazla çalışmaktadır. Bir dinamik programlama algoritması, her bir alt altproblemi sadece bir defa çözer ve bunun cevabını bir tabloda tutar, böylece her alt altproblemin cevabının tekrar hesaplanma yükünden kaçınılır.

Genel anlamda dinamik programlamayı *optimizasyon problemlerine* uygularız. Bu problemler birçok olanaklı çözüme sahip olabilirler. Her çözümün bir değeri vardır ve optimal (en büyük veya en küçük) değerli bir çözüm bulmak isteriz. Optimal değeri elde eden birkaç çözüm olabileceğinden, böyle bir çözümün problemin optimal çözümüne karşılık gelen problemin bir optimal çözümü olarak isimlendiririz.

Bir dinamik programlama algoritması geliştirirken şu dört adımı izleriz:

1. Optimal çözümün yapısını karakterize et.
2. Özyinelemeli olarak bir optimal çözümün değerini tanımla.
3. Genel olarak aşağıdan yukarıya olacak şekilde, bir optimal çözümün değerini hesapla.
4. Hesaplanan bilgidен bir optimal çözümü oluştur.

1-3 adımları, bir problem için bir dinamik programlama çözümünün temelini oluşturur. Eğer bir optimal çözümün kendisine değil sadece optimal çözümün değerine ihtiyaç duyarsak, o zaman adım 4 ü atlarız. Adım 4 ü çalıştırdığımızda, kimi zaman adım 3 süresince ayrıntılı bilgi elde ederiz, böylece bir optimal çözümü kolayca oluşturabiliriz.

Aşağıdaki bölümler bazı optimizasyon problemlerini çözmek için dinamik programlama yöntemini kullanır. Kısım 15.1, toplam değeri en büyük olacak şe-

kilde bir çubuğun daha küçük çubuklara kesilmesi problemini inceler. Kısım 15.2, en az toplam skaler çarpım işlemi yaparken bir matris dizisini nasıl çarpacağımızı ister. Dinamik programlamanın bu örnekleri göz önüne alındığında, Kısım 15.3, dinamik programlamanın uygulanabilir bir çözüm tekniği olduğu bir problemin iki önemli özelliğini ele alır. Ardından, Kısım 15.4, dinamik programlamayla iki dizinin en uzun ortak altdizisinin nasıl bulunacağını gösterir. Son olarak, Kısım 15.5, arama yapmak için bilinen bir anahtar dağılımı verildiğinde, dinamik programlamayı optimal ikili arama ağaçları oluşturmak için kullanır.

15.1 Çubuk kesme

İlk örneğimiz, çelik çubukları nereden keseceğimize karar vermeyele ilgili basit bir problemi çözmek için dinamik programlamayı kullanır. Serling Şirketi, uzun çelik çubuklar satın almakta ve onları satmak için daha kısa çubuklara kesmektedir. Her kesimin maliyeti sıfırdır. Serling Şirketi yönetimi, çubukları kesmenin en iyi yolunu bulmak istemektedir.

Farz edelim ki, Serling Şirketi'nin $i = 1, 2, \dots$ için i inç uzunluğundaki bir çubuğu p_i dolar olarak ücretlendirdiğini bilelim. Çubuk uzunlukları her zaman tam sayı (küsüratsız) inç uzunluktadır. Örnek bir ücret tablosu Şekil 15.1 de verilmiştir.

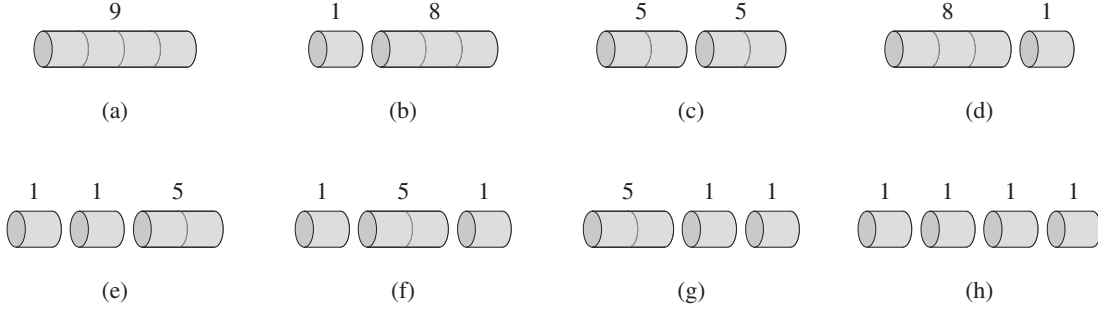
Çubuk kesme problemi (rod-cutting problem) aşağıdaki gibidir. n inç uzunluğunda bir çubuk ve $i = 1, 2, \dots, 3$ için p_i ücretler tablosu verilsin. Çubuğun kesilmesi ve parçaların satılması ile elde edilebilir en büyük r_n kazancını belirleyin. Eğer n uzunluğundaki bir çubuk için p_n ücreti yeteri kadar büyük ise, bir optimal çözümün hiç bir kesim gerektirmeyebileceğine dikkat edin.

$n = 4$ durumunu dikkate alalım. Şekil 15.2, hiç kesim olmama durumu dahil, 4 inçlik bir çubuğun tüm kesim yollarını gösterir. Görürüz ki, 4 inçlik bir çubuğu 2 inçlik iki parçaya bölmek $p_2 + p_2 = 5 + 5 = 10$ kazancını verir ve bu da optimaldir.

n uzunluklu bir çubuğu, $i = 1, 2, \dots, n - 1$ için sol uçtan i inç uzaklıkta bir kesim yapma veya yapmama gibi bir bağımsız seçeneğimiz olduğu için, 2^{n-1} farklı

uzunluk i	1	2	3	4	5	6	7	8	9	10
ücret p_i	1	5	8	9	10	17	17	20	24	30

Şekil 15.1 Çubuklar için örnek bir ücret tablosu. i inç uzunluklu her bir çubuk şirkete p_i dolar kazanç sağlar.



Şekil 15.2 4 uzunluklu çubuğun 8 tane olası kesim yolu. Şekil 15.1 deki örnek ücret tarifesine göre her parçanın değeri üstündedir. Optimal yol (c) seçeneğidir; çubuk 2 uzunluklu iki parçaya kesilmiştir ve toplam değeri 10 dur.

yolla kesebiliriz¹. Sıradan bir toplama notasyonu kullanarak kesilen parçaların bir dağılımını göstereceğiz: $7 = 2 + 2 + 3$, 7 inç uzunluğundaki bir çubuğun iki tane 2 uzunluğunda ve bir tane 3 uzunluğundaki üç parçaya kesildiğini gösterir. Eğer bir optimal çözüm çubuğu k parçaya keserse, $1 \leq k \leq n$ için parçaların uzunlukları $i_1, i_2, \dots, i_k$ olacak şekilde çubuğun bir

$$n = i_1 + i_2 + \dots + i_k$$

optimal dağılımına uygun olarak en fazla

$$r_n = p_{i_1} + p_{i_2} + \dots + p_{i_k}$$

kazancını üretir.

Örnek problemimizde, $i = 1, 2, \dots, 10$ için r_i optimal kazanç şekillerini deneyerek belirleyebiliriz. Uygun optimal dağılımlar aşağıdaki gibidir:

¹Eğer parçaları artmayan boyut sırasında kesmek gerekseydi, dikkate alınacak daha az yol olurdu. $n = 4$ için Şekil 15.2 de (a), (b), (c), (e) ve (h) olmak üzere sadece 5 yol hesaba katılacaktı. Bu yolların sayısı **parçalama fonksiyonu** (partition function) olarak isimlendirilir ve yaklaşık olarak $e^{\pi\sqrt{2n/3}}/4n\sqrt{3}$ 'e eşittir. Bu eşitlik 2^{n-1} den daha azdır, fakat n in herhangi bir polinomundan daha büyüktür. Ancak, daha detaylı incelemede bu yolu takip etmeyeceğiz.

$$\begin{aligned}
r_1 &= 1, \text{ çözümü } 1 = 1 \text{ (kesim yok),} \\
r_2 &= 5, \text{ çözümü } 2 = 2 \text{ (kesim yok),} \\
r_3 &= 8, \text{ çözümü } 3 = 3 \text{ (kesim yok),} \\
r_4 &= 10, \text{ çözümü } 4 = 2 + 2, \\
r_5 &= 13, \text{ çözümü } 5 = 2 + 3, \\
r_6 &= 17, \text{ çözümü } 6 = 6 \text{ (kesim yok),} \\
r_7 &= 18, \text{ çözümü } 7 = 1 + 6 \text{ veya } 7 = 2 + 2 + 3, \\
r_8 &= 22, \text{ çözümü } 8 = 2 + 6, \\
r_9 &= 25, \text{ çözümü } 9 = 3 + 6, \\
r_{10} &= 30, \text{ çözümü } 10 = 10 \text{ (kesim yok).}
\end{aligned}$$

Daha genel olarak, $n > 1$ için r_n değerlerini daha kısa çubukların optimal kazançları olarak aşağıdaki gibi düzenleyebiliriz:

$$r_n = \max(p_n, r_1 + r_{n-1}, r_2 + r_{n-2}, \dots, r_{n-1} + r_1) . \quad (15.1)$$

İlk değişken, p_n , hiç kesim yapılmaması ve n uzunluklu çubuğun olduğu gibi satılması durumudur. Diğer $n - 1$ tane değişken, her bir $i = 1, 2, \dots, n-1$ için i ve $n - i$ boyutlu iki parçalık başlangıç kesimi yaparak elde edilen maksimum kazanca denk gelir ve ardından bu iki parçadan r_i ve r_{n-i} kazançlarını elde edip bu parçaları optimal şekilde keser. Hangi i değerinde kazancı optimize ettiğimizi zaman gelmeden bilemeyeceğimiz için, i nin tüm mümkün değerlerini kontrol etmek ve kazancı maksimum yapan birini seçmek zorundayız. Keza, eğer çubuğu kesmeden yaptığımız satışla daha fazla kazanç elde edersek, hiçbir i seçmeme opsiyonuna da sahip oluruz.

n boyutlu asıl problemi çözmek için, aynı tipte ama daha küçük boyuttaki daha küçük problemleri çözeriz. Çubuk kesme probleminde ilk kesimi yaptığımızda, iki parçayı bağımsız olarak dikkate alabiliriz. Bu iki parçanın her birinden kazancı maksimum yaparak bu iki altproblemin optimal çözümleri genel optimal çözümü oluşturur. Çubuk kesme problemi aşağıdaki **optimal altyapıyı** gösterir: bir problemin optimal çözümleri, bağımsız olarak çözebildiğimiz ilgili altproblemlerin optimal çözümlerinin birleştirilmesidir.

Benzer fakat daha basit bir yol ile, çubuk kesme problemine özyinelemeli bir yapı oluşturmak için bir ayrıştırma göstereceğiz: sol taraftan kesilen i uzunluklu ilk parça ve ardından kalan $n - i$ uzunluklu sağ taraf. Sadece son kalan parça (ilk parça değil) daha sonrası için bölünebilir. n uzunluklu bir çubuğun her ayrıştırmasını şu şekilde gösterebiliriz: bir ilk parça ve devamında kalanın bazı ayrıştırmaları. Böyle yaptığımızda, çözümü hiçbir kesim olmadan, yani ilk parça $i=n$ olduğunda, kazanç p_n iken ve kalan parça 0 boyutlu, $r_0 = 0$ kazançlı olarak ifade edebiliriz. Bundan dolayı denklem (15.1) in daha basit bir versiyonunu elde ederiz:

$$r_n = \max_{1 \leq i \leq n} (p_i + r_{n-i}) . \quad (15.2)$$

Bu formülasyonda, bir optimal çözüm iki problem yerine sadece –geriye kalan– bir ilgili altproblemi kapsar.

Özyinelemeli yukarıdan-aşağıya uygulama

Aşağıdaki prosedür, denklem (15.2) deki kapalı hesaplamayı anlaşılır, yukarıdan-aşağıya ve özyinelemeli olarak gerçekleştirir.

CUT-ROD(p, n)

```

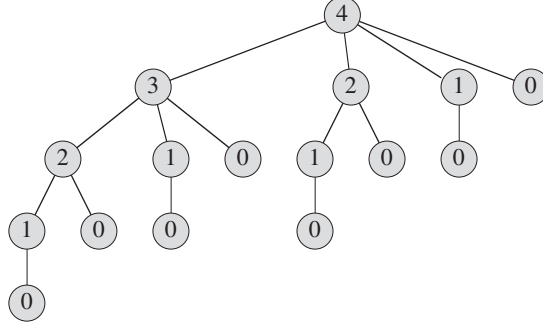
1  if  $n == 0$ 
2      return 0
3   $q = -\infty$ 
4  for  $i = 1$  to  $n$ 
5       $q = \max(q, p[i] + \text{CUT-ROD}(p, n - i))$ 
6  return  $q$ 
```

CUT-ROD prosedürü girdi olarak bir $p[1..n]$ ücret dizisi ve bir n tam sayısı alır ve n uzunluklu bir çubuk için mümkün en büyük kazancı geri döndürür. Eğer $n = 0$ ise hiç bir kazanç mümkün değildir ve böylece CUT-ROD algoritması 2. satırda 0 döndürür. 4-5 satırlarındaki **for** döngüsü $q = \max_{1 \leq i \leq n} (p_i + \text{CUT-ROD}(p, n-i))$ olarak doğru bir şekilde hesaplasın diye 3. satır en büyük q kazancını $-\infty$ olarak ilklendirir ve daha sonra 6. satır bu değeri geri döndürür. n için basit bir tümevarım, denklem (15.2) yardımıyla bu cevabın istenilen r_n cevabına eşit olduğunu ispatlar.

Eğer CUT-ROD prosedürünü sık kullandığınız programlama dilinde kodlar ve bilgisayarınızda çalıştırırsanız, girdi boyutu kısmen büyük olduğunda programın çalışması uzun zaman alabilir. $n = 40$ için, program en azından birkaç dakika, büyük ihtimalle bir saatten daha fazla zaman alabilir. Aslında, n i her 1 arttırdığınızda, programınızın çalışma zamanı yaklaşık olarak ikiye katlanır.

Niçin CUT-ROD çok verimsizdir? Problem, CUT-ROD un kendini aynı parametre değerleri ile üst üste özyinelemeli olarak çağırmasıdır. Aynı altproblemleri tekrar tekrar çözer. Şekil 15.3, $n = 4$ için ne olduğunu göstermektedir: CUT-ROD(p, n), $i = 1, 2, \dots, n$ için CUT-ROD($p, n - i$) yi çağırır. Denk olarak, CUT-ROD(p, n), her $j = 0, 1, \dots, n - 1$ için CUT-ROD(p, j) yi çağırır. Bu süreç özyinelemeli olarak açıldığı zaman, yapılan işin miktarı, n in bir fonksiyonu olarak şiddetli bir şekilde büyür.

CUT-ROD prosedürünün çalışma zamanını analiz etmek için, $T(n)$, ikinci parametre n olduğunda CUT-ROD un yaptığı çağrılarının toplam sayısını gösterebilir. Bu ifade, özyineleme ağacında kökü n ile etiketli bir altağaçtaki düğümlerin sayısına denktir. Bu sayı, kendi kökündeki başlangıç çağrısını içermektedir. Bundan



Şekil 15.3 $n = 4$ için bir CUT-ROD(p, n) çağrısının sonucundaki özyinelemeli çağrılarını gösteren özyineleme ağacı. Her bir düğüm etiketi, altprobleme karşılık gelen n boyutunu verir, böylece s etiketli ebeveynden t etiketli çocuğa bir ayırıt, $s - t$ boyutunda bir başlangıç parçası kesmeye ve t boyutlu kalan bir altproblemi atmaya uygun gelir. Kökten bir yaprağa bir yol, n uzunluklu bir çubuğun kesilmesinin 2^{n-1} yolundan birine denk gelir. Genel olarak, özyineleme ağacı 2^n düğüm ve 2^{n-1} yaprağa sahiptir.

$$T(n) = 1 + \sum_{j=0}^{n-1} T(j) . \quad (15.3)$$

den dolayı, $T(0) = 1$ ve baştaki 1 kökteki çağrıdır ve $T(j)$ terimi $j = n - i$ olduğunda CUT-ROD($p, n - i$) çağrısından dolayı (özyinelemeli çağrılar dahil) çağrılarının sayısını verir. Alıştırma 15.1-1 in kanıtlamak istediği gibi,

$$T(n) = 2^n \quad (15.4)$$

dir ve böylece CUT-ROD un çalışma zamanı n e bağlı olarak üsteldir.

Geriye bakıldığında, bu üstel çalışma zamanı çok şaşırtıcı değildir. CUT-ROD açık olarak, n uzunluklu bir çubuğun kesimindeki tüm 2^{n-1} mümkün yolu hesaba katar. Özyinelemeli çağrılar ağacı, çubuğu keserken her mümkün yol için bir tane olmak üzere, 2^{n-1} yaprağa sahiptir. Kökten bir yaprağa kadar olan basit yoldaki etiketler, kesim yapmadan önce, her bir kalan sağ parçanın boyutunu verir. Yani, etiketler çubuğun sağ ucundan ölçülmüş uygun düşen kesim noktalarını verir.

Optimal çubuk kesme için dinamik programlama kullanımı

Şimdi, dinamik programlamayı kullanarak CUT-ROD prosedürünün nasıl verimli bir algoritmaya dönüştüğünü göstereceğiz.

Dinamik programlama yöntemi aşağıdaki gibidir. Tekrar tekrar aynı altproblemleri çözdüğü için basit bir özyinelemeli çözümün verimsizliği fark edildiğinden, her bir altproblemin çözümünü kaydederek sadece bir defa çözülmesini sağlayacağız. Eğer daha sonra tekrar bu altproblemin çözümüne başvurmak gereki-

yorsa, yeniden hesaplamak yerine sadece aramak yeterli olabilir. Bundan dolayı, dinamik programlama hesaplama zamanından kazanmak için ilave bellek kullanır; **zaman-bellek maliyet dengesinin** bir örneğidir. Yapılan kazanımlar çarpıcı olabilir: üstel-zamanlı bir çözüm polinom-zamanlı bir çözüme dönüştürülebilir. İçerilen *farklı* altproblemlerin sayısı girdi boyutuna göre polinom olduğunda ve böyle her altproblemi polinom zamanda çözebildiğimizde, bir dinamik-programlama yaklaşımı polinom zamanda çalışır.

Bir dinamik programlama yaklaşımını uygulamak için genellikle iki eşdeğer yol vardır. Çubuk-kesme örneğimiz ile bunların ikisini de göstereceğiz.

İlk yaklaşım **ön belleğe alma ile yukarıdan aşağı** yöntemidir. Bu yaklaşımda doğal bir şekilde özyinelemeli olarak prosedürü yazarız fakat her altproblemin sonucunu (genellikle bir dizide veya ırpı tablosunda) kaydetmek için düzenleriz. Şimdi bu prosedür ilk olarak bu altproblemin daha önce çözümlü çözümlendiğini kontrol eder. Eğer önceden çözüldüyse, bu seviyede ilave hesaplama kazancı sağlayarak kaydedilmiş değeri döndürür; eğer önceden çözülmeyse, prosedür olağan şekilde istenen değeri hesaplar. Bu özyinelemeli prosedürün **ön belleğe** alma işlemini yaptığını; önceden hesaplanan sonuçları “hatırladığını” söyleyebiliriz.

İkinci yaklaşım **aşağıdan yukarıya yöntemidir**. Bu yaklaşım genellikle bir altproblemin “boyutunun” doğal eğilimine bağlıdır, öyle ki herhangi bir belirli altproblemin çözümü sadece daha küçük altproblemlerin çözümüne bağlıdır. Altproblemleri boyuta göre sıralarız ve en küçüğü ilk önce olacak şekilde boyut sırasında çözeriz. Belirli bir altproblemi çözerken, çözümünün bağlı olduğu daha küçük tüm altproblemleri çoktan çözmüş ve onların çözümlerini kaydetmiş oluruz. Her altproblemi sadece bir kez çözeriz ve ilk defa karşılaştığımızda onun ön koşullu altproblemlerinin tümünü çoktan çözmüş oluruz.

Yukarıdan aşağı yaklaşımda tüm mümkün altproblemleri test etmek için yinelenmenin yapılmadığı sıra dışı durumlar hariç, bu iki yaklaşım aynı asimptotik çalışma zamanlı algoritmaları verir. Aşağıdan yukarıya yaklaşımı prosedür çağrılarında daha az yük getirdiğinden çoğu kez daha iyi sabit çarpanlara sahiptir.

Ön belleğe alma işleminin eklendiği yukarıdan-aşağıya CUT-ROD prosedürünün kabakod aşağıdadır.

MEMOIZED-CUT-ROD (p, n)

1. $r[0..n]$ yeni bir dizi olsun
- 2 **for** $i = 0$ **to** n
- 3 $r[i] = -\infty$
- 4 **return** MEMOIZED-CUT-ROD-AUX(p, n, r)

²Bu bir yanlış yazım değildir. Ön belleğe alma (*memoization*) *memo*’dan gelir, çünkü daha sonra bakabilmek için bir değeri kaydeden bir tekniktir.

```

MEMOIZED-CUT-ROD-AUX( $p, n, r$ )
1  if  $r[n] \geq 0$ 
2      return  $r[n]$ 
3  if  $n = 0$ 
4       $q = 0$ 
5  else  $q = -\infty$ 
6      for  $i = 1$  to  $n$ 
7           $q = \max(q, p[i] + \text{MEMOIZED-CUT-ROD-AUX}(p, n - i, r))$ 
8   $r[n] = q$ 
9  return  $q$ 

```

Burada, MEMOIZED-CUT-ROD ana prosedürü, “bilinmeyi” göstermek için uygun bir seçenek olarak $-\infty$ değerli yeni bir yardımcı $r[0..n]$ dizisi başlatır. (Bilinen kazanç değerleri hiçbir zaman negatif değildir.) Ardından, MEMOIZED-CUT-ROD-AUX yardımcı programını çağırır.

MEMOIZED-CUT-ROD-AUX prosedürü (sadece) önceki CUT-ROD prosedürümüzün ön belleğe alan versiyonudur. İlk olarak, 1. satırda istenen değerin önceden bilinip bilinmediği kontrol edilir ve eğer biliniyorsa 2. satır bu değeri geri döndürür. Aksi halde, 3-7 satırları istenen q değerini olağan olarak hesaplar, 8. satır bu değeri $r[n]$ de kaydeder ve 9. satır ise bunu geri döndürür.

Aşağıdan yukarıya versiyonu daha kolaydır:

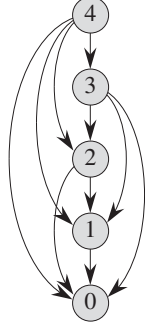
```

BOTTOM-UP-CUT-DOWN
1.  $r[0..n]$  yeni bir dizi olsun
2.  $r[0] = 0$ 
3. for  $j = 1$  to  $n$ 
4.      $q = -\infty$ 
5.     for  $i = 1$  to  $j$ 
6.          $q = \max(q, p[i] + r[j - i])$ 
7.      $r[j] = q$ 
8. return  $r[n]$ 

```

Aşağıdan-yukarıya dinamik programlama yaklaşımı için, BOTTOM-UP-CUT-DOWN, altproblemlerin doğal sıralamasını kullanır: eğer $i < j$ ise i boyutlu bir problem j boyutlu bir problemten daha küçüktür. Bundan dolayı, bu prosedür, ilgili sırada, $j = 0, 1, \dots, n$ boyutlu altproblemleri çözer.

BOTTOM-UP-CUT-DOWN prosedürünün 1. satırı, altproblemlerin sonuçlarını kaydetmek için yeni bir $r[0..n]$ dizisi oluşturur ve 2. satır 0 (sıfır) uzunluklu bir çubuk hiç bir kazanç sağlamayacağından $r[0]$ ı 0 olarak ilklendirir. 3-6 satırları, artan sırada $j = 1, 2, \dots, n$ için j boyutlu her bir altproblemi çözer. $j - i$ boyutlu altproblemi çözmek için özyinelemeli bir çağrı yapmak yerine $r[j - i]$ dizi ele-



Şekil 15.4 $n = 4$ için çubuk kesme probleminin altproblem çizgesi. Tepe etiketleri, ilgili altproblemlerin boyutlarını verir. Yönlü bir (x, y) ayrıtı, x altproblemünü çözerken y altproblemünün bir çözümüne gerek duyduğumuzu gösterir. Bu çizge, Şekil 15.3 teki ağacın indirgenmiş bir versiyonudur. Burada aynı etiketli tüm düğümler tek bir tepeye toplanmıştır ve tüm ayrıtlar ebeveynden çocuğa gider.

manını referans gösteren 6. satır hariç olmak üzere, özel bir j boyutlu problemi çözmek için kullanılan bu yaklaşım CUT-ROD tarafından kullanılanla aynıdır. 7. satır j boyutlu altproblem çözümünü $r[j]$ ye kaydeder. Son olarak, 8. satır, en iyi r_n değerine denk olarak $r[n]$ değerini geri döndürür.

Aşağıdan-yukarıya ve yukarıdan-aşağıya versiyonları aynı asimptotik çalışma zamanına sahiptir. BOTTOM-UP-CUT-DOWN prosedürünün çalışma zamanı, ikili iç içe döngü yapısından dolayı $\Theta(n^2)$ dir. İç **for** döngüsündeki iterasyonların sayısı, 5-6 satırlarındaki, bir aritmetik seri oluşturur. Yukarıdan-aşağıya karşılığının (MEMOIZED-CUT-ROD) çalışma zamanı, çalışma zamanı biraz daha kötü olmasına rağmen, $\Theta(n^2)$ dir. Daha önce çözülmüş bir altproblem çözmek için bir özyinelemeli çağrı hemen geri döndüğü için MEMOIZED-CUT-ROD her bir altproblem sadece bir defa çözer. 0, 1, ..., n boyutlu altproblemleri çözer. n boyutlu bir altproblem çözmek için, 6-7 satırlarındaki **for** döngüsü n defa yinelenir. Bu nedenle, bu **for** döngüsünün toplam iterasyon sayısı, MEMOIZED-CUT-ROD un tüm özyinelemeli çağrılarının üstünde, BOTTOM-UP-CUT-DOWN un iç **for** döngüsü gibi toplam $\Theta(n^2)$ iterasyon ile bir aritmetik seri oluşturur (Aslında burada toplama analizinin bir formunu kullanıyoruz. Toparlama analizini Kısım 17.1 de detaylı olarak göreceğiz).

Altproblem çizgeleri

Bir dinamik programlama problemini düşündüğümüzde, altproblemlerin kümesini ve altproblemlerin nasıl bir değerine bağlı olduğunu anlamalıyız.

İlgili problem için **altproblem çizgesi** tam olarak bu bilgiyi içerir. Şekil 15.4, $n = 4$ için çubuk kesme probleminin altproblem çizgesini gösterir. Her bir farklı altproblem için bir tepe içeren yönlü bir çizgedir. Eğer x altproblem için bir en iyi

çözümün belirlenmesi doğrudan y altproblemi için bir en iyi çözümün hesaba katılmasına bağlıysa, altproblem çizgesi x altproblem tepesinden y altproblem tepesine yönlü bir ayrıta sahiptir. Örneğin, eğer x i çözmek için bir yukarıdan-aşağıya özyinelemeli prosedür y yi çözmek için doğrudan kendini çağırır ise altproblem çizgesi x den y ye bir ayrıta sahiptir. Aynı altproblem için tüm düğümleri tek bir tepeye topladığımızda ve ebeveyninden çocuğa tüm ayrıtları yönlendirdiğimizde, altproblem çizgesini, yukarıdan-aşağıya özyinelemeli yöntemin öz yineleme ağacının bir “indirgemesi” veya “katlanması” olarak düşünebiliriz.

x altproblemini çözmeden önce, bu x altproblemine bitişik y altproblemlerini çözdüğümüzden dolayı dinamik programlama için aşağıdan-yukarıya yöntemi altproblem çizgesinin tepelerini dikkate alır (Kısım B.4 den, bitişiklik bağıntısının simetrik olma zorunluluğunun olmadığını hatırlayınız). Bölüm 22 deki terminolojiyi kullanarak, bir aşağıdan-yukarıya dinamik programlama algoritmasında, altproblem çizgesinin “tranpozesinin topolojik sırasında” (Kısım 22.4 e bakınız) veya “ters topolojik sırasında” altproblem çizgesinin tepeleri dikkate alınır. Başka bir deyişle, bağlı altproblemlerin tümü çözülene kadar hiçbir altproblem hesaba katılmaz. Benzer şekilde, aynı bölümdeki kavramları kullanarak, dinamik programlama için yukarıdan-aşağı yöntemi (önbellekleme ile) altproblem çizgesinin bir “derinliğine araması” olarak görülebilir (Kısım 22.3 e bakınız).

$G = (V, E)$ altproblem çizgesinin boyutu dinamik programlama algoritmasının çalışma zamanını belirlememize yardımcı olabilir. Her altproblemi sadece bir defa çözdüğümüzden, çalışma zamanı her altproblemi çözmek için gerekli zamanların toplamıdır. Genellikle, bir altproblem çözümünü hesaplama zamanı, altproblem çizgesinde ilgili tepelerin derecesi (çıkan ayrıtların sayısı) ile orantılıdır ve altproblemlerin sayısı altproblem çizgesindeki tepelerin sayısına eşittir. Bu ortak durumda, dinamik programlamanın çalışma zamanı ayrıt ve tepelerin sayısına bağlı olarak doğrusaldır.

Bir çözümün yeniden oluşturulması

Çubuk kesme problemi için dinamik programlama çözümlerimiz bir optimal çözümün değerini döndürür fakat bir gerçek çözüm, yani parça boyutlarının bir listesini, döndürmez. Sadece her altproblem optimal *değerini* değil aynı zamanda optimal değeri yönlendiren bir *seçenek* ile dinamik programlama yaklaşımını genişletebiliriz. Bu bilgi ile, kolayca bir optimal çözümü yazabiliriz.

Her j çubuk boyutunu, sadece en büyük kazanç r_j yi değil aynı zamanda s_j yi, kesilecek ilk parçanın optimal uzunluğunu hesaplayan BOTTOM-UP-CUT-DOWN prosedürünün genişletilmiş bir versiyonu aşağıdadır:

EXTENDED-BOTTOM-UP-CUT-ROD(p, n)

```

1.  $r[0..n]$  ve  $s[0..n]$  yeni diziler olsun.
2.  $r[0] = 0$ 
3. for  $j = 1$  to  $n$ 
4.    $q = -\infty$ 
5.   for  $i = 1$  to  $j$ 
6.     if  $q < p[i] + r[j - i]$ 
7.        $q = p[i] + r[j - i]$ 
8.        $s[j] = i$ 
9.    $r[j] = q$ 
10. return  $r$  and  $s$ 

```

Bu prosedür, 1. satırda s dizisini oluşturması ve i boyutlu altproblemi çözerken kesilecek ilk parçanın i optimal boyutunu tutmak için 8. satırda $s[j]$ yi güncellemesi hariç olmak üzere BOTTOM-UP-CUT-DOWN prosedürüne benzemektedir.

Aşağıdaki prosedür bir p ücret tablosu ve bir n çubuk boyutunu girdi olarak alır. Optimal ilk parçalı boyutların dizisi $s[1..n]$ i hesaplamak için EXTENDED-BOTTOM-UP-CUT-ROD prosedürünü çağırır ve n uzunluklu bir çubuğun bir optimal ayrışımındaki parça boyutlarının tam listesini yazdırır:

PRINT-CUT-ROD-SOLUTION(p, n)

```

1.  $(r, s) = \text{EXTENDED-BOTTOM-UP-CUT-ROD}(p, n)$ 
2. while  $n > 0$ 
3.   print  $s[n]$ 
4.    $n = n - s[n]$ 

```

Bizim çubuk kesme örneğimizde, EXTENDED-BOTTOM-UP-CUT-ROD($p, 10$) çağırısı aşağıdaki dizileri döndürecektir:

i	0	1	2	3	4	5	6	7	8	9	10
$r[i]$	0	1	5	8	10	13	17	18	22	25	30
$s[i]$	0	1	2	3	2	2	6	1	2	3	10

PRINT-CUT-ROD-SOLUTION($p, 10$) prosedürü çağırıldığında sadece 10 yazdıracaktır. Fakat $n = 7$ için bir çağrı, daha erken elde edilen r_7 için ilk optimal ayrışımaya uygun gelen 1 ve 6 kesimleri yazdırılacaktır.

Alıştırılmalar

15.1-1

Denklem (15.3) ve başlangıç koşulu $T(0) = 1$ i kullanarak denklem (15.4) ü elde ediniz.

15.1-2

Aksi örnek yardımıyla aşağıdaki “açgözlü” yöntemin çubukları kesme için her zaman bir optimal yol olmadığını gösteriniz. p_i/i olarak i uzunluklu bir çubuğun **yoğunluğunu**, yani inç başına değerini, tanımlayınız. n uzunluklu bir çubuk için ilgili açgözlü yöntemi, $1 \leq i \leq n$ için en çok yoğunluğa sahip i uzunluklu bir ilk parçayı keser. Ardından, $n-i$ uzunluklu kalan parça için ilgili açgözlü yöntemi uygulanmaya devam edilir.

15.1-3

Her çubuk için bir p_i ücretine ilave olarak her kesim sabit bir c maliyetinde olsun. Bir çözüm ile ilgili kazanç parçaların ücretlerinin toplamı ile kesme maliyetlerinin farkıdır. Bu değiştirilmiş problemi çözmek için bir dinamik programlama algoritması önerin.

15.1-4

Sadece değeri değil kesin çözümü de döndürmesi için MEMOIZED-CUT-ROD prosedürünü modifiye ediniz.

15.1-5

Fibonacci sayıları özyineleme (3.22) ile tanımlanmıştır. n . Fibonacci sayısını bulmak için $O(n)$ zamanlı bir dinamik programlama algoritması öneriniz. Altproblem çizgesini çiziniz. Bu çizgede kaç tepe ve ayrıt vardır?

15.2 Matris-zinciri çarpımı

Sıradaki dinamik programlama örneği matris zinciri çarpımı problemini çözen bir algoritmadır. Çarpılmak için n matrisli bir $\langle A_1, A_2, \dots, A_n \rangle$ dizisi (zinciri) verilsin ve aşağıdaki çarpımı hesaplamak isteyelim:

$$A_1 A_2 \cdots A_n . \quad (15.5)$$

(15.5) ifadesini, hangi matrislerin birlikte çarpılacağı ile ilgili belirsizlikleri çözmek için parantezleme yaptığımız zaman standart matris çiftlerinin çarpımı algoritmasını bir altprogram olarak çalıştırıp hesaplayabiliriz. Matris çarpımı birleşmelidir ve bu nedenle tüm farklı parantezlemeler aynı çarpımı vermektedir. Eğer tekil bir matris veya parantezlenmiş iki matris çarpımının çarpımı varsa bir matris çarpımı **tam anlamıyla parantezlenmiştir**. Örneğin, $\langle A_1, A_2, A_3, A_4 \rangle$ matris zinciri için $A_1 A_2 A_3 A_4$ çarpımı beş farklı şekilde parantezlenebilir:

$$\begin{aligned}
& (A_1(A_2(A_3A_4))) , \\
& (A_1((A_2A_3)A_4)) , \\
& ((A_1A_2)(A_3A_4)) , \\
& ((A_1(A_2A_3))A_4) , \\
& (((A_1A_2)A_3)A_4) .
\end{aligned}$$

Bir matris zincirini nasıl parantezleyeceğimiz, çarpımın hesaplama maliyetinde belirgin bir etki yaratabilir. İlk olarak, iki matrisin çarpımının maliyetini dikkate alalım. Standart algoritma aşağıdaki kabakod ile verilmiştir. MATRIX-MULTIPLY (Matris-Çarp) algoritması Kısım 4.2 deki SQUARE-MATRIX-MULTIPLY prosedüründen genelleştirilmiştir. *rows* ve *columns* değişkenleri matristeki satır ve sütun sayılarını tutmak için kullanılmıştır.

MATRIX-MULTIPLY (*A*, *B*)

```

1  if A.columns ≠ B.rows
2.    error “uyumsuz boyutlar”
3.  else C yeni bir A.rows × B.columns boyutlu matris olsun
4    for i = 1 to A.rows
5      for j = 1 to B.columns
6        cji = 0
7        for k = 1 to A.columns
8          cij = cji + aik · bkj
9    return C

```

İki *A* ve *B* matrisini çarpabilmemiz için bunların **uygun** olması gereklidir: *A* nın sütun sayısı *B* nin satır sayısına eşit olmalıdır. Eğer *A* $p \times q$ boyutlu ve *B* $q \times r$ boyutlu matrisler ise sonuç matrisi *C* $p \times r$ boyutludur. *C* yi hesaplamak için gerekli zaman 8. satırdaki skaler çarpımların sayısı (pqr) belirlenir. Sonrasında, maliyetleri skaler çarpımların sayısı cinsinden ifade edeceğiz.

Bir matris çarpımının farklı parantezlemelerinden kaynaklanan farklı maliyetlerini göstermek için üç matrisin $\langle A_1, A_2, A_3 \rangle$ zinciri problemini dikkate alalım. Farz edelim ki, matrislerin boyutları sırasıyla 10×100 , 100×5 ve 5×50 olarak verilsin. Eğer $((A_1A_2)A_3)$ şeklinde parantezlersek, $10 \cdot 100 \cdot 5 = 5000$ skaler çarpma işlemini 10×5 lik A_1A_2 çarpımı için ve $10 \cdot 5 \cdot 50 = 2500$ skaler çarpma işlemini bu çarpım ile A_3 ü çarparak ve böylece toplamda 7500 skaler çarpma işlemi gerçekleştiririz. Bunun yerine $(A_1(A_2A_3))$ parantezlemesini kullanırsak, $100 \cdot 5 \cdot 50 = 25000$ skaler çarpma işlemini 100×50 lik çarpımını hesaplamak için ve $10 \cdot 100 \cdot 50 = 50000$ skaler çarpma işlemini A_3A_4 ile bu matrisi çarpmak için ve toplam olarak 75000 skaler çarpma gerçekleştiririz. Bu nedenle, ilk parantezasyon 10 kat daha hızlıdır.

Matris-zinciri çarpım problemi şu şekilde tanımlayabiliriz: *n* matrisli $\langle A_1, A_2, \dots, A_n \rangle$ zinciri verilsin. Burada $i = 1, 2, \dots, n$ için, A_i matrisi $p_{i-1} \times p_i$

boyutlu olsun. $A_1 A_2 \cdots A_n$ çarpımını, skaler çarpımların sayısı en az olacak şekilde parantezleyiniz.

Matris-zinciri çarpım probleminde, tam olarak matrislerin çarpımıyla ilgilenmemektedir. Amaç sadece, en az maliyete sahip matris çarpımının bir düzenini belirlemektir. Genellikle, bu optimal sırayı belirlemede geçen zaman, matris çarpımını gerçekleştirdiğimizde kazandığımız zamandan daha fazladır (75000 yerine 7500 skaler çarpmayı gerçekleştirdiğimiz gibi).

Parantezlemelerin sayısının hesaplanması

Dinamik programlama ile matris-zinciri çarpım problemini çözmeden önce, tüm mümkün parantezlemeleri kontrol etmenin verimli bir algoritma olmayacağını bilelim. $P(n)$ ile n matrisli bir dizinin alternatif parantezlemelerinin sayısını gösterelim. $n = 1$ iken, sadece bir matrisimiz olur ve matris çarpımını parantezlemek için sadece bir yol olur. $n \geq 2$ iken, bir tam parantezli matris çarpımı iki tam parantezli matris altçarpımının çarpımıdır ve iki altçarpım arasındaki ayıraç, $k = 1, 2, \dots, n-1$ için k . ve $(k+1)$. matrisler arasında olmalıdır. Bundan dolayı, şu özyinelemeyi elde ederiz:

$$P(n) = \begin{cases} 1, & n = 1 \text{ ise,} \\ \sum_{k=1}^{n-1} P(k)P(n-k), & n \geq 2 \text{ ise.} \end{cases} \quad (15.6)$$

Problem 12-4 benzer bir özyineleme için çözümü sorar: $\Omega(4^n/n^{3/2})$ ile artan **Katalan sayıları**. Daha basit bir alıştırma (Alıştırma 15.2-3 e bakınız): (15.6) özyinelemesinin çözümü $\Omega(2^n)$ dir. Çözümlerin sayısı üsteldir ve detaylı arama için kaba kuvvet yöntemi bir matris zincirinin optimal parantezlemesini belirlemede zayıf bir strateji olarak kalır.

Dinamik programlamanın uygulanması

Bir matris zincirini nasıl optimal olarak parantezleyeceğimizi belirlemek için dinamik programlama kullanacağız. Bunu yaparken, bu bölümün başında kurduğumuz dört adımlı diziyi takip edeceğiz:

1. Bir optimal çözümün yapısını karakterize et.
2. Bir optimal çözümün değerini özyinelemeli olarak tanımla.
3. Bir optimal çözümün değerini hesapla.

4. Hesaplanan bilgidен bir optimal çözüm oluřtur.

Her adımı probleme nasıl uyguladığımız açıkça göstererek, bu adımları sırasıyla izleyeceğiz.

Adım 1: Bir optimal parantezlemenin yapısı

Dinamik programlama paradigmasında ilk adım için, optimal altyapıyı bulur ve ardından altproblemlerin optimal çözümlerinden problem için bir optimal çözüm kurmak için bunu kullanır. Matris zinciri çarpım probleminde, bu adım řu şekilde çalışır: kolay ifade edebilmek için, $A_i A_{i+1} \cdots A_j$ çarpımı sonucu oluřan matriste $A_{i..j}$ ($i \leq j$) notasyonunu kullanalım. Gösterin ki, eğer problem çözölmesi zor, yani $i < j$ ise $A_i A_{i+1} \cdots A_j$ çarpımını parantezlemek için $i \leq k < j$ aralığında bazı k tam sayıları için A_k ve A_{k+1} çarpımını ayırmalıyız. Yani, bazı k deęerleri için ilk olarak $A_{i..k}$ ve $A_{k+1..j}$ matrislerini hesaplar ve sonra bunları çarparak $A_{i..j}$ sonucunu üretiriz. Bu yolla parantezlemenin maliyeti; $A_{i..k}$ matrisini hesaplama maliyeti, $A_{k+1..j}$ matrisini hesaplama maliyeti ve bunların birlikte çarpımı maliyetinin toplamına eşittir.

Bu problemin optimal altyapısı řu şekildedir: farz edelim ki, $A_i A_{i+1} \cdots A_j$ yi optimal olarak parantezlemek için A_k ve A_{k+1} çarpımını ayırırız. $A_i A_{i+1} \cdots A_j$ optimal parantezlemesi için $A_i A_{i+1} \cdots A_k$ ön altzincirinin optimal parantezlemesi seçilmedir. Neden? Eđer daha az maliyetli bir $A_i A_{i+1} \cdots A_k$ parantezlemesi varsa, optimumdan daha az maliyetli $A_i A_{i+1} \cdots A_j$ parantezlemesini bir başka şekilde üreterek bunu optimal parantezlemeye yerleřtirirdik: çeliřki. Benzer bir gösterim, $A_i A_{i+1} \cdots A_j$ optimal parantezlemesinde $A_{k+1} A_{k+2} \cdots A_j$ altzincirinin nasıl parantezleneceğine dayanmaktadır: $A_{k+1} A_{k+2} \cdots A_j$ parantezlemesi optimal olmalıdır.

řimdi optimal altyapıyı altproblemlerin optimal çözümlerinden problemin bir optimal çözümünü oluřturmak için kullanacaęız. Görürüz ki, matris zinciri çarpım probleminin zor bir örneęinin herhangi bir çözölü, çarpımı parçalamamızı gerektirir ve herhangi bir optimal çözüm altproblem örneklerinin optimal çözümlerini içerir. Bu nedenle, örnek bir matris zinciri çarpım probleminin bir optimal çözölü, problemin iki altprobleme parçalanması ($A_i A_{i+1} \cdots A_k$ ve $A_{k+1} A_{k+2} \cdots A_j$ nin optimal parantezlemesi), altproblem örneklerinin optimal çözümlerinin bulunması ve sonra bu optimal altproblem çözümlerinin kombine edilmesiyle oluřturulabilir. Çarpımı parçalamak için doęru yeri ararken tüm mümkün pozisyonlara baktığımızı garanti etmeliyiz. Böylece optimal bir çözölü incelediğimize emin oluruz.

Adım 2: Özyinelemeli bir çözüm

Altproblemlerin optimal çözümlerine dayanarak, bir optimal çözümün maliyetini özyinelemeli olarak tanımlayabiliriz. Matris zinciri çarpım problemi için altproblemlerimizi, $1 \leq i \leq j \leq n$ için $A_i A_{i+1} \cdots A_j$ parantezlemesinin minimum maliyetini belirleyen problemler olarak ayırabiliriz. $m[i, j]$, $A_{i..j}$ matrisini hesaplamak için gerekli minimum skaler çarpma sayısı olsun: büyük problem için $A_{1..n}$ i hesaplamak için en düşük maliyet $m[1, n]$ olurdu.

$m[i, j]$ yi özyinelemeli olarak aşağıdaki gibi tanımlayabiliriz. Eğer $i = j$ ise problem basittir: Zincir sadece bir matris $A_{i..i} = A_i$ içerir ve çarpımı hesaplamak için hiçbir skaler çarpmaya gerek yoktur. Bu nedenle, $i = 1, 2, \dots, n$ için $m[i, j] = 0$ dır. $i < j$ iken $m[i, j]$ yi hesaplamak için adım 1 deki optimal çözümün yapısından faydalanabiliriz. Farz edelim ki, optimal olarak parantezlemek için, $i \leq k < j$ iken $A_i A_{i+1} \cdots A_j$ çarpımını A_k ve A_{k+1} arasından ayıralım. Sonra $m[i, j]$, $A_{i..k}$ ve $A_{k+1..j}$ altçarpımlarının minimum maliyetleri ve bu iki matrisin birlikte çarpım maliyeti birbirine eşittir. Her A_i matrisi için $p_{i-1} \times p_i$ yi kullanarak $A_{i..k} A_{k+1..j}$ matris çarpımının hesabı $p_{i-1} p_k p_j$ skaler çarpma yapar. Buradan,

$$m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$$

elde ederiz.

Bu özyinelemeli denklem, k nın değerini bildiğimizi varsayar. k için sadece $j - i$ mümkün değer vardır, yani $k = i, i + 1, \dots, j - 1$. Optimal parantezlemenin bu k değerlerinden bir tanesini kullanması gerektiğinden, sadece en iyisini bulmak için bu değerleri kontrol ederiz. Bu nedenle, $A_i A_{i+1} \cdots A_j$ çarpımını parantezlemenin minimum maliyeti için özyinelemeli tanımımız:

$$m[i, j] = \begin{cases} 0, & i = j \text{ ise,} \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j\}, & i < j \text{ ise.} \end{cases} \quad (15.7)$$

$m[i, j]$ değerleri altproblemlerin optimal çözümlerinin maliyetlerini verir fakat bir optimal çözümü kurmak için ihtiyaç duyulan tüm bilgiyi sağlamaz. Böyle yapmak için, bir optimal parantezlemede $A_i A_{i+1} \cdots A_j$ çarpımını ayırdığımız k değerini $s[i, j]$ olarak tanımlarız. Yani $s[i, j]$, $m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$ deki k değerine eşittir.

Adım 3: Optimal maliyetlerin hesaplanması

Bu aşamada, $A_1 A_2 \cdots A_n$ çarpımında $m[1, n]$ minimum maliyetini hesaplamak için (15.7) özyinelemesine dayalı bir özyinelemeli algoritma kolaylıkla yazılabilir. Çubuk kesme probleminde gördüğümüz gibi ve Kısım 15.3 te göreceğimiz gibi, bu özyinelemeli algoritma çarpımı parantezlemenin her bir yolunu kontrol eden kaba kuvvet yönteminden daha iyi olmayacak şekilde üstel zamanlıdır.

Nispeten farklı birkaç altproblemimiz olsun: her i ve j için $1 \leq i \leq j \leq n$ veya $\binom{n}{2} + n = \Theta(n^2)$ olacak bir altproblem. Bir özyinelemeli algoritma, her altproblemi, özyineleme ağacının farklı dallarında birçok defa kullanabilir. Bu özellik (altproblemlerin çalışması), dinamik programlamanın uygulanmasında ikinci bir ayırıcı özelliktir (ilki optimal altyapının bulunmasıdır).

(15.7) özyinelemesiyle çözüm hesaplamak yerine, optimal maliyeti aşağıdan-yukarı yaklaşımı ile bir tablo kullanarak hesaplarız. (benzer bir yukarıdan-aşağıya yaklaşımı (ön bellekleme kullanarak) Kısım 15.3 te sunulacaktır.)

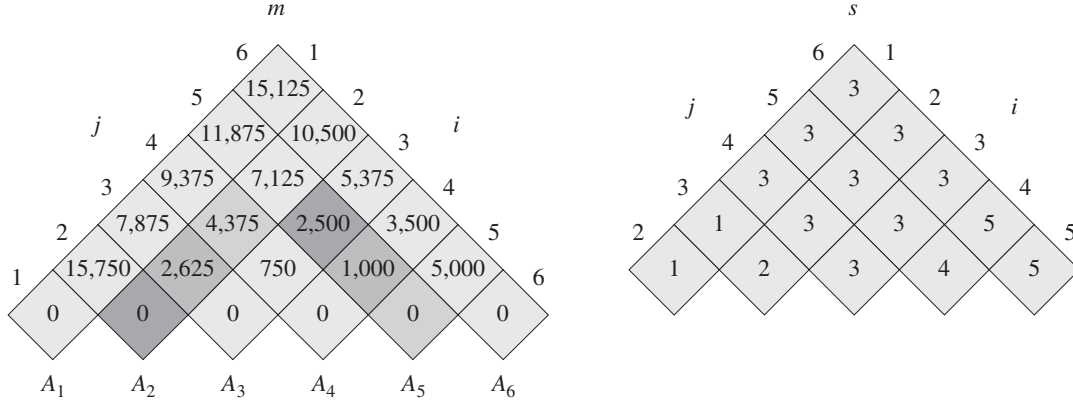
Aşağıdaki MATRIX-CHAIN-ORDER prosedüründeki aşağıdan-yukarı yöntemiyle tabloyu dolduracağız. Bu prosedür, $i = 1, 2, \dots, n$ için A_i matrisinin boyutlarını $p_{i-1} \times p_i$ olarak varsayar. Girdisi $p.length = n + 1$ olmak üzere bir $p = \langle p_0, p_1, \dots, p_n \rangle$ dizisidir. Bu prosedür, $m[i, j]$ maliyetlerini tutmak için yardımcı bir $m[1..n, 1..n]$ tablosu ve $m[i, j]$ yi hesaplamada optimal maliyeti veren k indisini kaydetmeye yardımcı bir $s[1..n - 1, 2n]$ tablosu kullanır. s tablosunu optimal çözümü kurmak için kullanacağız.

Aşağıdan-yukarı yaklaşımını uygulamak için $m(i, j)$ yi hesaplarken kullanacağımız tablo kayıtlarını belirlemeliyiz. (15.7) denklemi gösterir ki, $j - i + 1$ tane matrisin bir matris zinciri çarpım hesabının $m[i, j]$ maliyeti, sadece $j - i + 1$ matristen daha az matris zinciri çarpımının hesaplama maliyetine bağlıdır. Yani, $k = i, i + 1, \dots, j - 1$ için $A_{i..k}$ matrisi $k - i + 1 < j - i + 1$ matrislerinin bir çarpımı ve $A_{k+1..j}$ matrisi $j - k < j - i + 1$ matrislerinin bir çarpımıdır. Bundan dolayı, algoritma, artan uzunluktaki matris zincirlerinin parantezlenmesi problemini çözmek anlamına gelecek şekilde, m tablosunu doldurmalıdır. $A_i A_{i+1}, \dots, A_j$ zincirinin optimal olarak parantezlenmesi altproblemi için zincirin $j - i + 1$ uzunluklu altproblem boyutunu düşüneceğiz.

MATRIX-CHAIN-ORDER (p)

```

1   $n = p.length - 1$ 
2   $m[1..n, 1..n]$  ve  $s[1..n-1, 2..n]$  yeni tablolar olsun
3  for  $i = 1$  to  $n$ 
4       $m[i, i] = 0$ 
5  for  $l=2$  to  $n$       //  $l$  zincir uzunluğudur.
6      for  $i = 1$  to  $n - l + 1$ 
7           $j = i + l - 1$ 
8           $m[i, j] = \infty$ 
9          for  $k = i$  to  $j - 1$ 
10              $q = m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$ 
11             if  $q < m[i, j]$ 
12                  $m[i, j] = q$ 
13                  $s[i, j] = k$ 
14 return  $m$  ve  $s$ 
```



Şekil 15.5 $n = 6$ ve aşağıdaki matris boyutları için MATRIX-CHAIN-ORDER algoritması ile hesaplanan s ve m tabloları.

matris	A_1	A_2	A_3	A_4	A_5	A_6
boyut	30×35	35×15	15×5	5×10	10×20	20×25

Tablolar, ana köşegen yatay olacak şekilde döndürülmüştür. m tablosu sadece ana köşegeni ve üst üçgeni, s tablosu sadece üst üçgeni kullanır. 6 matrisi çarpmak için minimum skaler çarpma sayısı $m[1,6] = 15.125$ tir. Daha koyu olan hücrelerden, aynı renkli olanlar, $m[2,5]$ hesaplanırken 10. satırda birlikte dikkate alınır.

$$\begin{aligned}
 m[2,5] &= \min \begin{cases} m[2,2] + m[3,5] + p_1 p_2 p_5 = 0 + 2500 + 35 \cdot 15 \cdot 20 = 13,000, \\ m[2,3] + m[4,5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \cdot 5 \cdot 20 = 7125, \\ m[2,4] + m[5,5] + p_1 p_4 p_5 = 4375 + 0 + 35 \cdot 10 \cdot 20 = 11,375 \end{cases} \\
 &= 7125.
 \end{aligned}$$

Algoritma, 3-4. satırlarda, ilk önce $m[i, i] = 0$ $i = 1, 2, \dots, n$ hesaplar (1 uzunluklu zincirlerin minimum maliyetleri). Sonra, 5-13 satırlarındaki **for** döngüsünün ilk çalıştırılması süresince (15.7) özyinelemesi kullanılarak $m[i, i+1]$ $i = 1, 2, \dots, n-1$ hesaplanır ($l = 2$ uzunluklu zincirler için minimum maliyetler) ve bu şekilde devam eder. 10-13 satırlarında hesaplanan $m[i, j]$ maliyeti her seferinde daha önceden hesaplanan $m[i, k]$ ve $m[k+1, j]$ tablo kayıtlarına bağlıdır.

Şekil 15.5, bu prosedürün çalışmasını $n = 6$ matrisli bir zincir örneği üzerinde gösterir. $m[i, j]$ yi sadece $i \leq 6$ için tanımladığımızdan, m tablosunda sadece ana köşegenin üst kısmı kullanılır. Şekil, ana köşegeni yatay olacak şekilde döndürülmüş tabloyu gösterir. matris zinciri de altında listelenmiştir. Bu yerleşimi kullanarak, kuzeydoğudan A_i ve kuzeybatıdan A_j satırlarının kesişiminde, matrislerin $A_i A_{i+1} \dots A_j$ alt zincirinin çarpımı için $m[i, j]$ minimum maliyetini bulabiliriz.

Tablodaki her bir yatay satır aynı uzunluktaki matris zincirlerinin kayıtlarını tutar. MATRIX-CHAIN-ORDER, satırları aşağıdan yukarıya ve her satırı soldan sağa doğru hesaplar. Her bir $m[i, j]$ kaydını $p_{i-1}p_kp_j$ $k = i, i + 1, \dots, j - 1$ çarpımını kullanarak, ve $m[i, j]$ deki tüm kayıtlar güneybatıdan güneydoğuya olacak şekilde hesaplar.

MATRIX-CHAIN-ORDER prosedürünün iç içe döngü yapısı algoritmaya $O(n^3)$ çalışma zamanı kazandırır. Döngüler iç içe 3 seviyedir ve her döngü indisi (i, i, k) en çok $n - 1$ değer alır. Alıştırma 15.2-5, algoritmanın çalışma zamanının $\Omega(n^3)$ olduğunu göstermemizi ister. Algoritma s ve m tablolarını tutmak için $\Theta(n^2)$ alana ihtiyaç duyar. Bu nedenle, MATRIX-CHAIN-ORDER algoritması, tüm mümkün parantezlemeleri sıralayan ve her birini kontrol eden üstel zamanlı yöntemden çok daha verimlidir.

Adım 4: Bir optimal çözümün kurulması

MATRIX-CHAIN-ORDER algoritması bir matris zinciri çarpımını hesaplamak için gerekli skaler çarpımların optimal sayısını belirlemesine rağmen matrisleri nasıl çarptığını doğrudan göstermez. $s[1..n - 1, 2..n]$ tablosu, bunu yapabilmek için gerekli bilgiyi verir. Her bir $s[i, j]$ hücresi bir k değeri tutar. Bu $A_iA_{i+1}\dots A_j$ nin bir optimal parantezlemesinde, A_k ve A_{k+1} arasından çarpımı ayırır. Bu nedenle, biliriz ki $A_{1..n}$ hesabındaki matris çarpımının sonucu $A_{i..s[1, n]}A_{s[1, n] + 1..n}$ dir. $A_{i..s[1, n]}$ hesaplanırken $s[1, s[1, n]]$ son matris çarpımını belirlediğinden ve $A_{s[1, n] + 1..n}$ hesaplanırken $s[s[1, n] + 1, n]$ son matris çarpımını belirlediğinden, daha önceki matris çarpımlarını özyinelemeli olarak belirleyebiliriz. MATRIX-CHAIN-ORDER algoritması tarafından oluşturulan s tablosu ile i ve j indisleri verildiğinde aşağıdaki özyinelemeli prosedür $\langle A_i, A_{i+1}, \dots, A_j \rangle$ nin bir optimal parantezlemesini yazdırır. PRINT-OPTIMAL-PARENS($s, 1, n$) başlangıç çağrısı $\langle A_1, A_2, \dots, A_n \rangle$ in bir optimal parantezlemesini yazdırır.

PRINT-OPTIMAL-PARENS($s, 1, n$)

```

1  if  $i = j$ 
2    print " $A_i$ "
3  else print "("
4    PRINT-OPTIMAL-PARENS( $s, i, s[i, j]$ )
5    PRINT-OPTIMAL-PARENS( $s, s[i, j] + 1, j$ )
6    print ")"
```

Şekil 15.5 teki örnekle, PRINT-OPTIMAL-PARENS($s, 1, 6$) çağrısı $((A_1(A_2A_3))((A_4A_5)A_6))$ parantezlemesini yazdırır.

Alıştırmalar

15.2-1

Boyutları $\langle 5, 10, 3, 12, 5, 50, 6 \rangle$ olan bir matris zinciri çarpımının optimal parantezlemesini bulun.

15.2-2

Özyinelemeli bir MATRIX-CHAIN-MULTIPLY(A, s, i, j) algoritması önerin. Bu algoritma, $\langle A_1, A_2, \dots, A_n \rangle$ matris dizisi, MATRIX-CHAIN-ORDER tarafından hesaplanan s tablosu ve i ve j indisleri verildiğinde optimal matris zinciri çarpımını gerçekleştirsin. (Başlangıç çağrısı MATRIX-CHAIN-MULTIPLY($A, s, 1, n$) olabilir.)

15.2-3

Yerine koyma yöntemini kullanarak (15.6) özyinelemesinin çözümünün $\Omega(2^n)$ olduğunu gösterin.

15.2-4

n uzunluklu bir girdi zinciri ile matris zinciri çarpımı için altproblem çizgesini tanımlayınız. Kaç tepeye sahiptir? Kaç ayrıtı vardır ve bunlar hangi ayrıtlardır?

15.2-5

Bir MATRIX-CHAIN-ORDER çağrısında diğer tablo kayıtları hesaplanırken $m[i, j]$ tablo kaydının referans gösterilme sayısı $R(i, j)$ olsun. Mevcut tablo için referansların toplam sayısının

$$\sum_{i=1}^n \sum_{j=i}^n R(i, j) = \frac{n^3 - n}{3}$$

olduğunu gösterin.

(İpucu: (A.3) denklemi faydalı olabilir.)

15.2-6

Bir n elemanlı ifadenin tam parantezlemesi tam olarak $n - 1$ parantez çiftine sahiptir. Gösteriniz.

15.3 Dinamik programlamanın elemanları

Dinamik programlama yönteminin sadece iki örneği ile çalışmış olmamıza rağmen, bu yöntemin ne zaman uygulanacağını merak edebilirsiniz. Bir mühendislik bakış açısıyla, bir problem için ne zaman bir dinamik programlama çözümü aramalıyız? Bu kısımda bir optimizasyon probleminin dinamik programlamaya

uygulanması için gerekli iki anahtar bileşeni inceleyeceğiz: optimal altyapı ve örtüşen altproblemler. Bir aşağıdan-yukarı özyinelemeli yaklaşımında önbelleklemenin örtüşen altproblemler özelliğinde sağladığı avantajı daha iyi tartışacağız.

Optimal altyapı

Dinamik programlama ile bir optimizasyon problemini çözmede ilk adım bir optimal çözümün yapısını karakterize etmektir. Eğer problemin bir optimal çözümü altproblemlerin optimal çözümlerini içeriyorsa bu problemin **optimal altyapıyı** gösterdiğini hatırlayınız. Bir problem optimal altyapıyı ortaya koyduğunda dinamik programlamayı uygulamak için iyi bir ipucuna sahip oluruz. (Bölüm 16'nın ele aldığı gibi, bu aynı zamanda bir açgözlü stratejisinin uygulanması anlamına gelir.) Dinamik programlamada altproblemlerin optimal çözümlerinden problemin bir optimal çözümünü oluştururuz. Sonuç olarak, ele aldığımız altproblemler aralığının bir optimal çözümde kullanıldığını garanti ettiğimizi unutmamalıyız.

Bu bölümde ele aldığımız iki problemde de optimal altyapıyı ortaya çıkardık. Kısım 15.1'de n uzunluklu bir çubuğu kesmenin optimal yolunun, ilk kesimin sonucundaki kalan iki parçanın optimal kesimini kapsadığını gördük. Kısım 15.2'de çarpımı A_k ve A_{k+1} arasından ayrılan $A_i A_{i+1} \cdots A_j$ nin bir optimal parantezlemesinin, $A_i A_{i+1} \cdots A_k$ ve $A_{k+1} A_{k+2} \cdots A_j$ parantezleme problemlerinin optimal çözümünü içerdiğini gösterdik.

Optimal altyapı araştırılırken aşağıdaki ortak kalıp kendiliğinden bulunabilir:

1. Problemin bir çözümünün, bir çubuktaki başlangıç kesiminin seçimi veya matris zincirini ayırmak için indis seçimi şeklinde bir seçim yapmaktan oluştuğunu gösterirsiniz. Bu seçimin yapılması, çözülmesi için bir veya daha çok altproblemi oluşturur.
2. Verilen bir problem için bir optimal çözüme götürecek bir seçim yapıldığını kabul edersiniz. Bu seçimin nasıl belirlendiği ile ilgilenmezsiniz. Bu seçimin size verildiğini farz edersiniz.
3. Bu seçim verildiğinde hangi altproblemin ortaya çıkacağını ve altproblemlerin uzayını nasıl en iyi karakterize edeceğinizi belirlersiniz.
4. Bir optimal çözümde kullanılan altproblem çözümlerinin bir “kes-ve-yapıştır” tekniği kullanılarak optimal olması gerektiğini gösterirsiniz. Bunu altproblem çözümlerinin her birinin optimal olmadığını kabul edip ardından bir çelişki türeterek yaparsınız. Özellikle, her bir altproblemin optimal olmayan çözümünün kesip çıkarılması ve optimal olanın eklenmesi ile orijinal problem için daha iyi bir çözüm elde edebildiğini gösterirsiniz. Bundan dolayı, bir optimal çözüm oluştuğu varsayımımız çelişkiye düşer. Eğer bir optimal çözüm

bir altproblemden daha fazlasını oluşturursa, bunlar genellikle çok benzerdir. Kes-ve-yapıştır işlemi ile birini diğerlerine kolayca uyarlayarak modifiye edebilirsiniz.

Altproblem uzayını karakterize etmek için, pratik bir yol uzayı mümkün mer-tebe basit tutmayı ve gerektiği kadar genişletmeyi denemektir. Örneğin, çubuk kesme problemi için değerlendirdiğimiz altproblem uzayı, her bir i boyutu için i uzunluklu bir çubuğu optimal olarak kesme problemlerini içermiştir. Bu altproblem uzayı iyi çalışmıştır ve altproblemlerin daha genel bir uzayını denemeye gerek kalmamıştır.

Diğer taraftan, matris zinciri çarpımı için altproblem uzayımızı $A_1 A_2 \cdots A_j$ formundaki matris çarpımına kısıtlamayı denediğimizi farz edelim. Önceki gibi, bir optimal parantezleme, bazı $1 \leq k < j$ için A_k ve A_{k+1} arasında kalan bu çarpımı ayırmaktır. $k = j - 1$ olduğunu garanti ettiğimizde, $A_1 A_2 \cdots A_k$ ve $A_{k+1} A_{k+2} \cdots A_j$ formunda altproblemlere sahip olduğumuzu ve kalan problemin $A_1 A_2 \cdots A_j$ formunda olmadığını elde ederiz. Bu problem için, $A_1 A_{i+1} \cdots A_j$ altproblemde çeşitliliği sağlamak için i ve j yi serbest bırakmaya gerek duyduk.

Optimal altyapı, problem alanlarına göre iki şekilde değişiklik gösterir:

1. Orijinal problemin bir optimal çözümü kaç tane altproblem kullanır ve
2. Bir optimal çözümde, altproblem(ler)i belirlemede kaç seçim kullanmak zordur.

Çubuk kesme probleminde, n boyutlu bir çubuğu kesmek için bir optimal çözüm sadece bir altproblem ($n - i$ boyutunda) kullanır. Fakat, hangisinin bir optimal çözümü oluşturacağını belirlemek için, i için n seçimi dikkate almalıyız. $A_i A_{i+1} \cdots A_j$ altzinciri için matris zinciri çarpımı iki altprobleme bir örnektir ve $j - i$ seçim vardır. Çarpımı parçalamak için verilen bir A_k matrisi için, $A_i A_{i+1} \cdots A_k$ ve $A_{k+1} A_{k+2} \cdots A_j$ parantezlemesi olmak üzere iki altproblem vardır ve bunların ikisini de optimal olarak çözmeliyiz. Bir defa, altproblemler için optimal çözümleri belirlediğimizde k indisi için $j - i$ adayın arasından seçim yaparız.

İnformal olarak, dinamik programlama algoritmasının çalışma zamanı iki kat-sayının çarpımına bağlıdır: tüm altproblemlerin sayısı ve her altproblemde kaç seçeneğe bakacağımız. Çubuk kesmede, toplam $\Theta(n)$ altproblem ve $O(n^2)$ zaman alan her biri için incelenmesi gereken en çok n seçim vardır. Matris zinciri çarpımı, toplam $\Theta(n^2)$ altprobleme sahiptir ve her birinde en çok $n - 1$ seçim vardır. Bu da $O(n^3)$ zamanda gerçekleşir (aslında $\Theta(n^3)$ çalışma zamanı, Alıştırma 15.2-5).

Genellikle, altproblem çizgesi aynı analizi işletmek için alternatif bir yoldur. Her tepe bir altprobleme karşılık gelir ve bir altproblemin seçimleri, bu altproble-

me bağlı ayrıtlardır. Çubuk kesmeyi hatırlarsak, altproblem çizgesi n tepe ve tepe başına en çok n ayrıta sahiptir, $O(n^2)$ zamanda. Matris zinciri çarpımında, eğer altproblem çizgesini çizmek istersek, $\Theta(n^2)$ tepe ve her tepe için derece en çok $n - 1$ dir, tepeler ve ayrıtlar için toplamda $O(n^3)$ zaman alır.

Dinamik programlama, optimal yapıyı sıklıkla aşağıdan-yukarı tarzda kullanır. Yani, ilk önce, altproblemlerin optimal çözümlerini ve çözülen altproblemler yardımı ile problemin optimal çözümünü buluruz. Problemin bir optimal çözümünü bulmak, problemi çözmede kullanacağımız altproblemlerden bir seçim yapmamızı gerektirir. Problem çözümünün maliyeti genellikle, altproblem maliyetleri ve seçime dayalı maliyetin toplamıdır. Çubuk kesmede, esnek olarak, ilk önce $i = 0, 1, \dots, n - 1$ için i uzunluklu çubukların kesimi için optimal yolların belirlenmesi altproblemlerini çözdük ve (15.2) denklemini kullanarak, n uzunluklu bir çubuk için böyle altproblemin bir optimal çözüm teşkil ettiğini belirledik. Seçime dayalı maliyet (15.2) denkleminde p_i ile gösterilmiştir. Matris zinciri çarpımında, $A_i A_{i+1} \dots A_j$ altzincirinin optimal parantezlemesini belirledik ve çarpımı ayıracak A_k matrisini seçtik. Seçime dayalı maliyet $p_{i-1} p_k p_j$ dir.

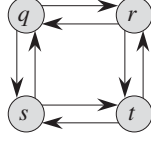
Bölüm 16 da dinamik programlamaya birçok benzerliği olan “açgözlü algoritmaları” (greedy algorithms) inceleyeceğiz. Özellikle, açgözlü algoritmaların uygulandığı problemler optimal altyapıya sahiptir. Açgözlü algoritma ile dinamik programlama arasındaki önemli bir fark, ilk önce optimal çözümlerin bulunması ve ardından bir seçim yapılması yerine açgözlü algoritmalar ilk olarak açgözlü bir seçim yapar —o andaki en iyisi—ve sonucundaki bir altproblemi çözer. Tüm mümkün ilgili altproblemlere bakmaz. Şaşırtıcı bir şekilde, bu strateji bazı durumlarda çalışır.

Farklılıklar

Optimal altyapı olmadığında, optimal yapının uygulandığını varsaymamaya dikkat etmeliyiz. Yönlü bir $G = (V, E)$ çizgesi ve $u, v \in V$ ayrıtları verildiğinde aşağıdaki iki problemi düşünelim.

Ağırlıksız en kısa yol:³ u dan v ye en az ayrıt içeren bir yol bulunuz. Böyle bir yol basit yol olmalıdır. Çünkü bir yoldan bir tur çıkarıldığında daha az ayrıtlı bir yol üretilir.

³“ağırlıksız” terimini bu problemi, Bölüm 24 ve 25 te göreceğimiz ağırlıklı ayrıtlı en kısa yolu bulma probleminden ayırmak için kullanırız. Ağırlıksız problemi çözmek için Bölüm 22 deki genişlik öncelikli tekniği kullanabiliriz.



Şekil 15.6 Ağırlıksız yönlü bir çizgede, bir en uzun basit yol bulma problemi ifade eden bir yönlü çizge optimal altyapıyı içermez. $q \rightarrow r \rightarrow t$, q dan t ye bir en uzun yoldur, fakat, ne $q \rightarrow r$ alt yolu q dan r ye bir en uzun basit yol ne de $r \rightarrow t$ alt yolu r den t ye bir en uzun basit yoldur.

Ağırlıksız en uzun basit yol: En fazla ayrıtı içeren u dan v ye bir basit yol bulunuz. Basit olma şartını sağlamak zorundadır, çünkü aksi halde, çok fazla sayıda ayrıtıla çalışırken birkaç defa çapraz geçiş yapan bir tur oluşturulabilir.

Ağırlıksız en kısa yol problemi aşağıdaki şekilde optimal altyapıyı gösterir. Farz edelim ki, $u \neq v$ yani problem zor olsun. u dan v ye herhangi bir p yolu bir w ara tepesini içermelidir. (w , u veya v olabilir.) Bu nedenle, $u \xrightarrow{p} v$ yolunu $u \xrightarrow{p_1'} w \xrightarrow{p_2} v$ alt yollarına ayrıştırabiliriz. Açık şekilde, p deki ayrıtların sayısı, p_1 ve p_2 deki ayrıtların toplamına eşittir. İddia ederiz ki, eğer p , u dan v ye bir optimal (en kısa) yol ise, p_1 , u dan w ya bir en kısa yol olmalıdır. Neden? Bir “kes-ve-yapıştır” işlemi kullanırız: eğer u dan w ya p_1 den daha az ayrıtılı başka bir p_1' yolu olsaydı, p den daha az ayrıtılı $u \xrightarrow{p_1'} w \xrightarrow{p_2} v$ yolunu üretmek için p_1 i kesip yerine p_1' yolunu yapıştırırız. Bundan dolayı p ’nin optimallliği çelişkiye düşer. Benzer biçimde, p_2 , w dan v ye bir en kısa yol olmalıdır. Bu nedenle, u dan w ya ve w dan v ye bir en kısa yol bularak ve en kısa yolu verecek bir w ara tepesini seçerek u dan v ye bir en kısa yol bulabiliriz. Kısım 25.2 de, ağırlıklı yönlü bir çizgede tepelerin her çifti arasında bir en kısa yol bulmak için optimal altyapının değişik bir bakışını kullanacağız.

Bir ağırlıksız en uzun yol bulma probleminin optimal altyapıyı gösterdiğini varsaymaya yanlış yönlendirilmiş olabilirsiniz. Buna rağmen bir $u \xrightarrow{p} v$ en uzun basit yolunu $u \xrightarrow{p_1} w \xrightarrow{p_2} v$ alt yollarına ayrıştırırsak o zaman p_1 , u dan w ya ve p_2 , w dan v ye bir en uzun yol olmamalı mıdır? Cevap hayırdır! Şekil 15.6, bir örnekle bunu gösterir. $q \rightarrow r \rightarrow t$ yolunu dikkate alalım, bu q dan t ye bir en uzun basit yoldur. $q \rightarrow r$, q dan r ye bir en uzun basit yol mudur? Hayır, $q \rightarrow s \rightarrow t \rightarrow r$ yolu daha uzun basit bir yoldur. $r \rightarrow t$, r den t ye bir en uzun basit yol mudur? Hayır, $r \rightarrow q \rightarrow s \rightarrow t$ daha uzun basit bir yoldur.

Bu örnek, en uzun basit yollar için, problemin sadece optimal altyapı ihtiyacı duymakla kalmadığını, aynı zamanda altproblemlerin çözümlerinden problemin bir geçerli çözümünü oluşturmadığını gösterir. eğer, $q \rightarrow s \rightarrow t \rightarrow r$ ve $r \rightarrow q \rightarrow s \rightarrow t$ en uzun basit yollarını birleştirirsek basit olmayan $q \rightarrow s \rightarrow t \rightarrow r \rightarrow q \rightarrow s \rightarrow t$ yolunu elde ederiz. Aslında, bir ağırlıksız en uzun basit yol bulma problemi, optimal altyapının herhangi bir türüne sahip olduğunu göstermez. bu problem için şimdiye kadar hiç verimli dinamik programlama algoritması bulunmamıştır. Aslında, bu problem NP-tam dır –Bölüm 34 te göreceğimiz gibi– yani polinomial zamanda çözülmesi mümkün değildir.

Neden en uzun basit yolun yapısı en kısa yolunkinden çok farklıdır? En uzun ve en kısa yol problemlerinin ikisi de çözümde iki altproblem kullanmasına rağmen en uzun basit yolun arandığı altproblemler **bağımsız** değildir, aksine en kısa yol problemlerinde bağımsızdır. Altproblemler için bağımsız olmak ne anlama gelir? Aynı problemin bir altprobleminin çözümü başka bir altprobleminin çözümünü etkilemez. Şekil 15.6 örneğinde, q dan t ye en uzun basit yolu bulma probleminde iki altproblemimiz vardır: q dan r ye ve r den t ye en uzun basit yolları bulma. Bu altproblemlerin ilkinde, $q \rightarrow s \rightarrow t \rightarrow r$ yolunu seçeriz ve böylece s ve t tepelerini kullanmış oluruz. Artık bu tepeleri ikinci altproblemde kullanamayız. Çünkü altproblemlerin çözümlerinin birleşimi basit bir yol olmayacaktır. Eğer t yi ikinci problemde kullanmazsak bu problemi çözemeyiz. Çünkü t bulacağımız yol için gereklidir ve bu tepe altproblem sonuçlarını birleştiren tepe (yani r tepesi olarak) değildir. Bu altproblemin çözümünde s ve t tepelerini kullandığımız için bunları başka bir altproblem çözümünde kullanamayız. Diğer altproblemi çözmek için bunlardan en az birini kullanmalıyız fakat optimal olarak çözmek için ikisini birden kullanmalıyız. Bundan dolayı bu altproblemlerin bağımsız olmadığını söyleyebiliriz. Bir başka yoldan baktığımızda, bir altproblemi çözmede kaynakların kullanımı (burada tepeler kaynaklardır) diğer problemleri çözülemez yapabilir.

Bir en kısa yolu bulmak için neden altproblemler bağımsızdır? Cevap: doğası gereği, altproblemler kaynakları paylaşmazlar. İddia ederiz ki, eğer w tepesi u dan v ye bir en kısa p yolunda ise herhangi bir $u \xrightarrow{p_1} w$ en kısa yolu ile $w \xrightarrow{p_2} v$ en kısa yolunu u dan v ye en kısa yolu oluşturmak için birleştirebiliriz. w nun dışındaki hiçbir tepe aynı anda p_1 ve p_2 de bulunamaz. Neden? Farz edelim ki, kimi $x \neq w$ tepeleri p_1 ve p_2 de bulunsun. Yani p_1 i $u \xrightarrow{p_{ux}} x \rightsquigarrow w$ ve p_2 yi $w \rightsquigarrow x \xrightarrow{p_{xv}} v$ olarak ayıralım. Bu problemin optimal altyapısında p yolu p_1 ve p_2 kadar çok ayrıta sahiptir; p nin ayrıt sayısı e olsun. u dan v ye bir $p' = u \xrightarrow{p_{ux}} x \xrightarrow{p_{xv}} v$ yolunu kuralım. Her biri en az bir ayrıta sahip x ten w ya ve w dan x e yollarını çıkardığımızdan dolayı p' yolu en çok $e - 2$ ayrıt içerir. Bu da p nin en kısa yol varsayımı

ile çelişir. Bu nedenle, en kısa yol probleminin altproblemlerinin bağımsız olduğunu garanti ederiz.

Kısım 15.1 ve Kısım 15.2 de incelenen problemlerin ikisi de bağımsız altproblemler içerir. Matris zinciri problemlerinde altproblemler $A_1, A_{i+1}, \dots, A_k$ ve $A_{k+1}, A_{k+2}, \dots, A_j$ altzincirlerinin çarpılmasıdır. Bu altzincirler ayrıktır yani hiçbir matris ikisini birden içermez. Çubuk kesmede, n uzunluklu bir çubuğu kesmenin en iyi yolunu belirlemek için $i = 0, 1, \dots, n - 1$ olmak üzere i uzunluklu çubukların kesilmesi için en iyi yolları bulmalıyız. n uzunluklu problemin bir optimal çözümü bu altproblemler çözümlerinden sadece bir tanesini içerdiğinden (ilk parçayı kestikten sonra), altproblemlerin bağımsızlığı sorun değildir.

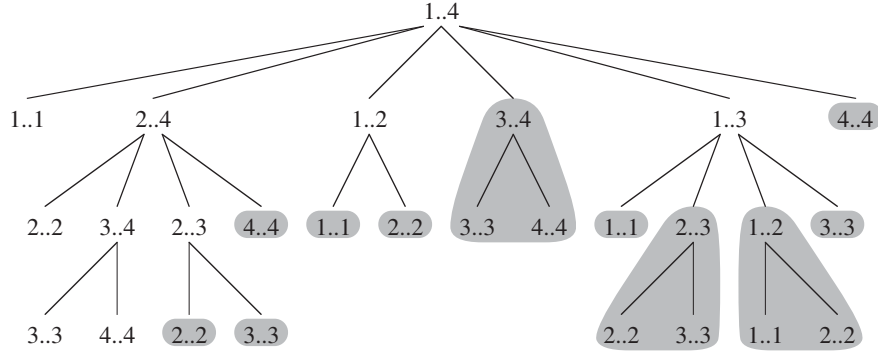
Altproblemlerin çakışması

Bir optimizasyon probleminin dinamik programlama uygulamasına sahip olması için ikinci unsur, özyinelemeli algoritmaların her zaman yeni altproblemler üretmesinden ziyade aynı altproblemleri tekrar tekrar çözmesi bakımından, altproblem uzayının “küçük” olmasıdır. Genellikle, farklı altproblemlerin toplam sayısı, girdi boyutuna bağlı olarak bir polinomdur. Bir özyinelemeli algoritma aynı problemi tekrar ele aldığı anda, bu optimal problem **altproblemlerin çakışmasına**⁴ sahiptir. Aksine bir böl-ve-yönet yaklaşımının uygun olduğu bir problem genellikle özyinelemenin her adımında yeni dallanmış problemler oluşturur. Dinamik programlama algoritmaları, genellikle, her altproblemin sadece bir kez çözümlenmesinden ve çözümün bir tabloda saklanmasından dolayı altproblem örtüşmesinde avantajlıdır. Bu tablo, ihtiyaç duyulduğunda sabit zamanda bakabildiğimiz bir tablodur.

Kısım 15.1 de, kısaca, daha küçük altproblemlerin çözümlerini bulmak için çubuk kesmenin bir özyinelemeli çözümünün nasıl üstel çoklukta çağrı yaptığını inceledik. Bizim dinamik programlama çözümümüz, üstel zamanlı özyinelemeli algoritmadan kuadratik zamana düşürülmüştür.

Çakışan-altproblemler özelliğini daha büyük bir örnekte göstermek için matris zinciri çarpım problemini yeniden inceleyelim. Şekil 15.5 i kastederek, MATRIX-CHAIN-ORDER algoritmasının daha büyük satırlı altproblemler çözülürken daha küçük satırlı altproblemlerin çözümüne baktığını gösteriniz. Örneğin, $m[3,4]$ kaydına 4 defa referans yapılmıştır: $m[2, 4]$, $m[1, 4]$, $m[3, 5]$ ve $m[3, 6]$

⁴Dinamik programlamanın bağımsız ve çakışan altproblemlere bağlı olması garip gelebilir. Bu gereksinimler çelişkili görünmesine rağmen, aynı eksenlerdeki iki noktadan ziyade iki farklı kavramı tanımlar. Eğer kaynakları paylaşmazlarsa, aynı problemin iki altproblemi bağımsızdır. Farklı problemlerin bir altproblemi olarak, eğer aynı altproblemler ise, iki altproblem örtüşür.



Şekil 15.7 RECURSIVE-MATRIX-CHAIN($p, 1, 4$) ün hesabı için özyineleme ağacı. Her düğüm i ve j değerlerini içerir. Bir gölgeli ağaçta yapılan hesaplamalar MEMOIZED-MATRIX-CHAIN tarafından bakılan tek bir tabloda tutulur.

nın hesaplanması sırasında. Eğer $m[3,4]$ değerine bakmak yerine her seferinde yeniden hesaplasaydık, çalışma zamanı kokutucu şekilde artardı. Nasıl olduğunu görmek için, $A_{i..j} = A_i A_{i+1} \dots A_j$ matris zinciri çarpımını hesaplamak için gerekli skaler çarpımların minimum sayısı, $m[i,j]$ yi belirleyen aşağıdaki (verimsiz) özyinelemeli prosedürü dikkate alalım. Bu prosedür doğrudan (15.7) özyinelemesine dayanmaktadır.

```

RECURSIVE-MATRIX-CHAIN( $p, i, j$ )
1  if  $i == j$ 
2      return 0
3   $m[i, j] = \infty$ 
4  for  $k = i$  to  $j - 1$ 
5       $q = \text{RECURSIVE-MATRIX-CHAIN}(p, i, k)$ 
           $+ \text{RECURSIVE-MATRIX-CHAIN}(p, k + 1, j)$ 
           $+ p_{i-1} p_k p_j$ 
6      if  $q < m[i, j]$ 
7           $m[i, j] = q$ 
8  return  $m[i, j]$ 

```

Şekil 15.7, RECURSIVE-MATRIX-CHAIN($p, 1, 4$) çağrısı tarafından oluşturulmuş özyineleme ağacını gösterir. Her düğüm i ve j ile etiketlenmiştir. Bazı ikililerin birkaç defa kullanıldığına dikkat ediniz.

Aslında, bu özyinelemeli prosedür tarafından $m[1, n]$ i hesaplama zamanının en azından n e bağlı olarak üstel olduğunu gösterebiliriz. $T(n)$, n matrisli bir zincirin bir optimal parantezlemesini hesaplamak için RECURSIVE-MATRIX-CHAIN prosedürünün çalışma zamanı olsun. 1–2 ve 6–7 satırlarının her birinin çalışması,

5. satırdaki çarpma işlemi gibi, en azından birim zaman alacağından, bu prosedür

$$T(1) \geq 1,$$

$$T(n) \geq 1 + \sum_{k=1}^{n-1} (T(k) + T(n-k) + 1) \quad n > 1 \text{ için}$$

özyinelemesini verir.

$i = 1, 2, \dots, n-1$ için, her $T(i)$ terimi bir defa $T(k)$ ve bir defa da $T(n-k)$ olarak kullanılır ve $n-1$ tane 1 ile öndeki 1 in toplanmasıyla;

$$T(n) \geq 2 \sum_{i=1}^{n-1} T(i) + n \quad (15.8)$$

özyinelemesini yazabiliriz.

Yerine koyma yöntemi ile $T(n) = \Omega(2^n)$ i kanıtlayacağız. Özellikle her $n \geq 1$ için $T(n) \geq 2^{n-1}$ i göstereceğiz. $T(1) \geq 1 = 2^0$ dir. Tümevarımla, $n \geq 2$ için

$$\begin{aligned} T(n) &\geq 2 \sum_{i=1}^{n-1} 2^{i-1} + n \\ &= 2 \sum_{i=0}^{n-2} 2^i + n \\ &= 2(2^{n-1} - 1) + n \quad (\text{A.5 denklemei ile}) \\ &= 2^n - 2 + n \\ &\geq 2^{n-1} \end{aligned}$$

bularak kanıtı tamamlarız. Bundan dolayı, $\text{RECURSIVE-MATRIX-CHAIN}(p, 1, n)$ çağırısı tarafından yapılan işin toplam boyutu en azından n e bağlı olarak üsteldir.

Bu yukarıdan-aşağıya özyinelemeli algoritmayı (önbellekleme olmadan) aşağıdan-yukarı dinamik programlama algoritmasıyla kıyaslayalım. Sonuncusu daha verimlidir çünkü örtüşen altproblemler özelliğinden dolayı avantajlıdır. Matris zinciri çarpımı sadece $\Theta(n^2)$ farklı altprobleme sahiptir ve dinamik programlama her birini tam olarak bir defa çözer. Özyinelemeli algoritma, diğer taraftan, özyineleme ağacındaki her altproblemi her karşısına çıktığında tekrar çözer. Bir problemin doğal özyineleme çözümünün özyineleme ağacı aynı altproblemi birkaç defa içeriyorsa ve farklı altproblemlerin toplam sayısı küçük ise, dinamik programlama verimliliği bazen çarpıcı biçimde arttırabilir.

Bir optimal çözümün oluşturulması

Pratik olarak, bir tabloda her altproblem için yaptığımız seçimi tutmak isteriz. Böylece, yeniden bu bilgiyi bulma maliyetinden kurtulmuş oluruz.

Matris zinciri çarpımı için bir optimal çözüm oluşturulduğunda, $s[i, j]$ tablosu bizi önemli bir iş yükünden kurtarır. $s[i, j]$ tablosunu tutmadığımızı, optimal problem maliyetini içeren $m[i, j]$ tablosunu tuttuğumuzu farz edelim. $A_i A_{i+1} \cdots A_j$ parantizasyonunun bir optimal çözümünde hangi altproblemleri kullanacağımızı belirlediğimizde $j - i$ ihtimalden seçim yaparız ve $j - i$ sabit bir sayı değildir. Bu nedenle, verilen problemin bir çözümü için seçeceğimiz altproblemleri yeniden oluşturmak $\Theta(j - i) = w(1)$ zaman alır. $A_i A_{i+1} \cdots A_j$ çarpımını ayıracağımız matris indisini $s[i, j]$ de tutarak, her seçimi $O(1)$ zamanda yeniden kurabiliriz.

Önbelleğe alma

Çubuk-kesme probleminde gördüğümüz gibi, dinamik programlamaya alternatif bir yaklaşım vardır. Bu yaklaşım, yukarıdan-aşağı stratejisini kullanırken aşağıdan-yukarı dinamik programlama yaklaşımı verimliliğindedir. Bu fikir, doğal fakat verimsiz özyineleme algoritmasını **önbelleğe alma** işlemidir. Aşağıdan-yukarı yaklaşımındaki gibi altproblem çözümlerinden bir tablo elde ederiz fakat tabloyu doldurmadaki kontrol yapısı daha çok özyineleme algoritması gibidir.

Bir önbelleklenmiş özyineleme algoritması, her altproblem için bir tabloda bir kayıt tutar. Her bir tablo kaydı, başlangıçta doldurulması gereken kaydı gösteren özel bir değeri içerir. Özyinelemeli algoritma açılırken altproblemle ilk karşılaşıldığında çözüm hesaplanır ve tabloda tutulur. Bu altproblemle bundan sonra her karşılaştığımızda tabloda tutulan değere bakıp bunu geri döndürürüz.⁵

RECURSIVE-MATRIX-CHAIN algoritmasının önbelleklenmiş versiyonu aşağıdadır. Çubuk-kesme probleminin önbelleklenmiş yukarı-aşağı yöntemine benzediğine dikkat edin.

⁵Bu yaklaşım, tüm mümkün altproblem parametrelerinin kümesini bildiğimizi ve tablo konumları ve altproblemler arasında bağlantı kurduğumuzu kabul eder. Bir başka daha genel yaklaşım, anahartlar olarak altproblem parametreleri ile çırpılama kullanarak önbelleğe almaktır.

```

MEMOIZED-MATRIX-CHAIN( $p$ )
1   $n = p.length - 1$ 
2  let  $m[1..n, 1..n]$  yeni bir tablo olsun
3  for  $i = 1$  to  $n$ 
4      for  $j = i$  to  $n$ 
5           $m[i, j] = \infty$ 
6  return LOOKUP-CHAIN( $m, p, 1, n$ )

```

```

LOOKUP-CHAIN( $m, p, i, j$ )
1  if  $m[i, j] < \infty$ 
2      return  $m[i, j]$ 
3  if  $i == j$ 
4       $m[i, j] = 0$ 
5  else for  $k = i$  to  $j - 1$ 
6       $q = \text{LOOKUP-CHAIN}(m, p, i, k)$ 
           $+ \text{LOOKUP-CHAIN}(m, p, k + 1, j) + p_{i-1} p_k p_j$ 
7      if  $q < m[i, j]$   $q =$ 
8           $m[i, j]$ 
9  return  $m[i, j]$ 

```

MEMOIZED-MATRIX-CHAIN prosedürü, MATRIX-CHAIN-ORDER gibi, $A_{i..j}$ matrisini hesaplamak için gerekli skaler çarpımların minimum sayısını, hesaplanan $m[i, j]$ değerlerinin bir $m[1..n, 1..n]$ tablosunu elde eder. Her tablo kaydı, doldurulacak anlamında, ∞ değerini içerir. LOOKUP-CHAIN(m, p, i, j) çağırıldığında 1. satır $m[i, j] < \infty$ bulunursa, prosedür basit bir şekilde 2. satırdaki önceden hesaplanmış $m[i, j]$ maliyetini döndürür. Aksi halde, maliyet RECURSIVE-MATRIX-CHAIN de olduğu gibi hesaplanır, $m[i, j]$ ye aktarılır ve geri döndürülür. Bu nedenle LOOKUP-CHAIN(m, p, i, j) her zaman $m[i, j]$ değerini döndürür, fakat ilk defa verilen i ve j değerleri için hesaplama yapar.

Şekil 15.7, MEMOIZED-MATRIX-CHAIN prosedürünün RECURSIVE-MATRIX-CHAIN ile kıyaslandığında nasıl zaman kazandığını gösterir. Gölge altı, tekrar hesaplamak yerine kullandığı değerleri gösterir.

Aşağı-yukarı dinamik programlama algoritması MATRIX-CHAIN-ORDER gibi MEMOIZED-MATRIX-CHAIN prosedürü $O(n^3)$ zamanda çalışır. MEMOIZED-MATRIX-CHAIN in 5. satırı $O(n^2)$ zamanda çalışır. LOOKUP-CHAIN çağrılarını iki türde sınıflandırabiliriz:

1. $m[i, j] = \infty$ daki çağrılar (3-9 satırları çalışır) ve
2. $m[i, j] < \infty$ daki çağrılar (LOOKUP-CHAIN, 2. satırda geri döndürür).

Her tablo kaydı başına, ilk tipte $\Theta(n^2)$ çağrı vardır. İkinci türün tüm çağrıları, birinci tür çağrılar tarafından özyinelemeli olarak yapılır. Her ne zaman verilen bir LOOKUP-CHAIN çağrısı özyinelemeli çağrılar yaparsa $O(n)$ zaman alır. Bu nedenle, ikinci tür çağrılarının tamamı, $O(n^3)$ tür. İkinci tür her çağrı $O(1)$ zaman alır ve her birinci tür çağrı $O(n)$ zaman ve özyinelemeli çağrılar için harcanan zamanın toplamıdır. Bundan dolayı toplam zaman $O(n^3)$ tür. Önbelleğe alma, $\Omega(2^n)$ zamanlı algoritmadan $O(n^3)$ zamanlı algoritmaya dönüşür.

Özetle, matris zinciri çarpımı problemini, yukarıdan-aşağı önbelleklenmiş dinamik programlama algoritması veya aşağıdan-yukarı dinamik programlama algoritması ile $O(n^3)$ zamanda çözebiliriz. İki yöntem de örtüşen altproblemler özelliğinden dolayı avantajlıdır. Toplamda sadece $\Theta(n^2)$ farklı altproblem vardır ve bu yöntemlerin ikisi de her altproblemın sonuncusunu sadece bir defa hesaplar. Ön belleğe alma olmadan, doğal özyinelemeli algoritma üstel zamanda çalışır. Çünkü çözülen altproblemleri tekrar çözer.

Uygulamada, eğer tüm altproblemler en azından bir kez çözümlerse aşağı-yukarı dinamik programlama algoritması yukarı-aşağı önbelleklenmiş algoritmasından genellikle sabit katsayı farkıyla daha iyi performans gösterir. Çünkü aşağı-yukarı algoritması gereksiz özyineleme yapmaz ve tabloyu oluşturmak için daha az ek yük getirir. Buna ek olarak, bazı problemler için alan veya zaman gereksinimini düşürmek için dinamik programlama algoritmasında tablo erişimlerinin düzenli bir kalıbından faydalanabiliriz. Alternatif olarak, eğer altproblem uzayındaki bazı altproblemlerin çözümlmesine gerek yoksa önbelleklenmiş çözüm sadece gerekli altproblemleri çözmesi ile avantaj kazanır.

Alıştırıcılar

15.3-1

Bir matris zinciri probleminde çarpımların optimal sayısını belirlemek için hangi yol daha verimlidir: çarpımın parantezlenmesinin tüm yollarını sayımlamak ve her biri için çarpımların sayısını hesaplamak mı yoksa RECURSIVE-MATRIX-CHAIN algoritmasını çalıştırmak mı? Cevabınızı doğrulayınız.

15.3-2

16 elemanlı bir dizi üzerinde, Kısım 2.3.1 deki MERGE-SORT prosedürü için özyineleme ağacını çiziniz. Neden önbellekleme MERGE-SORT gibi iyi bir bölve-yönet algoritmasını hızlandırmak için başarısız olacağını açıklayınız.

15.3-3

Matris zinciri çarpımı probleminin farklı bir türünü düşünelim. Amaç, matris di-

zisindeki çarpımları minimize etmek yerine maksimize etmek için parantezlemek olsun. Bu problem, optimal altyapıyı sağlar mı?

15.3-4

Belirtildiği gibi dinamik programlamada ilk önce altproblemleri çözeriz, sonra problemin optimal çözümünde hangisini kullanacağımızı seçeriz. Profesör Capulet iddia eder ki, bir optimal çözümü bulmak için her zaman tüm altproblemleri çözmek zorunda değiliz. Altproblemleri çözmeden önce, $A_i A_{i+1} \dots A_j$ altçarpımını parçalamayan A_k matrisini seçerek $(p_{i-1} p_k p_j)$ miktarını minimize eden k yı seçerek) matris zinciri probleminin bir optimum çözümünü bulabiliriz. Bu açgözlü yaklaşımının bir alt optimum çözümü verdiği matris-zinciri çarpımı probleminin bir örneğini bulunuz.

15.3-5

Farz edelim ki, Kısım 15.1 deki çubuk kesme probleminde, $i = 1, 2, \dots, n$ için üretilen i uzunluklu parçaların sayısında l_i limiti olsun. Kısım 15.1 de tanımlı optimal altyapı özelliğinin artık sağlanmadığını gösteriniz.

15.3-6

Bir parayı başka bir birime çevireceğinizi düşünün. Bir dövizden diğerine çevirmek yerine, diğer dövizlere dönüşüm serisi oluşturmak daha iyi olabilir. Farz edelim ki $1, 2, \dots, n$ ile etiketlenmiş n farklı döviz olsun, dönüşümü 1. dövizden başlayıp n . dövizde bitirmek isteyelim. Her i ve j döviz çifti için dönüşüm oranı r_{ij} olsun, yani d birim i dövizini r_{ij} dönüşüm oranıyla $d \cdot r_{ij}$ birim j dövizine dönüştürebiliriz. Bir değişim serisi bir komisyon gerektirebilir. c_k , k ya takas yapmak istediğimiz komisyon olsun. Her $k = 1, 2, \dots, n$ için $c_k = 0$ ise 1. dövizden n . dövize dönüşümün en iyi dizilimini bulma probleminin optimal altyapıyı sağladığını gösteriniz. Ardından, c_k komisyonları keyfi değerler ise 1. dövizden n . dövize dönüşümün en iyi serisini bulma probleminin optimal altyapıyı sağlamasına gerek olmadığını gösteriniz.

15.4 En uzun ortak altdizi

Biyolojik uygulamalar genellikle, iki veya daha fazla farklı organizmanın DNA'larının kıyaslanmasını gerektirir. Bir DNA dizilimi **baz** diye isimlendirilen molekül dizisinden oluşur: adenin (adenine), guanin (guanine), sitozin (cytosine) ve timin

(thymine). Bu bazları baş harfleri ile isimlendirerek, bir DNA dizilimini $\{A, C, G, T\}$ sonlu kümesi üzerinde bir katar olarak ifade edebiliriz. (bir katar tanımı için Ek C ye bakınız.) Örneğin, bir organizmanın DNA'sı $S_1 = \text{ACCGGTCGAGTGC-GCGGAAGCCGGCCGAA}$ ve bir diğeri $S_2 = \text{GTCGTTGCGAATGCCGTTGCTCT-GTAAA}$ olabilir. İki DNA diziliminin nasıl benzer olduğunu belirlemek için iki dizilimin kıyaslanması bir nedeni, iki organizmanın ne kadar yakından alakalı olduğunu ölçmek olabilir. Benzerliği birkaç farklı yolla belirleyebiliriz. Örneğin, eğer biri diğerrinin altkatarı ise, iki DNA dizilimini benzediğini söyleyebiliriz. (Bölüm 32 bu problemi çözecek algoritmaları araştırır) Bizim örneğimizde S_1 ve S_2 birbirinin altkatarı değildir. Alternatif olarak, birini diğerrine çevirmek için gerekli değişikliklerin sayısı az ise bu iki dizilime benzer diyebiliriz (Problem 15.5 bu mantıktadır). S_1 ve S_2 dizilimlerinin benzerliklerini ölçmek için bir diğerr yol, S_1 ve S_2 nin her birinde bazları var olan S_3 üçüncü dizilimini bulmaktır; bu bazlar aynı sırada olmalıdır fakat ardışık olmak zorunda değildir. Bulabileceğimiz daha uzun S_3 dizilimi daha benzer S_1 ve S_2 demektir. Örneğimizdeki en uzun S_3 dizilimi $\text{GTCGTCGGAAGCCGGCCGAA}$ dır.

Son benzerlik kavramını en uzun ortak alt dizi olarak isimlendiririz. Verilen bir dizinin altkatarı sıfır veya daha fazla elemanlı bir dizidir. Formal olarak, bir $X = \langle x_1, x_2, \dots, x_m \rangle$ dizisi ve eğer X in indislerinin $\langle i_1, i_2, \dots, i_k \rangle$ artan dizisi varken $j = 1, 2, \dots, k$ için $x_{i_j} = z_j$ oluyorsa bir $Z = \langle z_1, z_2, \dots, z_k \rangle$ dizisi X in **altdizisi**dir. Örneğ olarak, $Z = \langle B, C, D, B \rangle$ dizisi, $X = \langle A, B, C, B, D, A, B \rangle$ dizinin bir altdizisidir ve ilgili indis dizisi $\langle 2, 3, 5, 7 \rangle$ dir.

X ve Y dizileri verilsin. Eğer Z dizisi, X ve Y nin bir altdizisi ise Z ye X ve Y nin bir **ortak altdizisi** denir. Örneğin, eğer $X = \langle A, B, C, B, D, A, B \rangle$ ve $Y = \langle B, D, C, A, B, A \rangle$ ise $\langle B, C, A \rangle$ dizisi X ve Y nin bir ortak altdizisidir. $\langle B, C, A \rangle$ dizisi en büyük ortak altdizi (longest common subsequence, LCS) değildir. Çünkü uzunluğu 3 tür. X ve Y de ortak olan $\langle B, C, B, A \rangle$ nın uzunluğu 4 tür. $\langle B, C, B, A \rangle$, $\langle B, D, A, B \rangle$ gibi X ve Y nin en uzun altdizisidir çünkü 5 veya daha büyük uzunluklu ortak altdiziye sahip değildir.

En uzun-ortak-altdizi probleminde, $X = \langle x_1, x_2, \dots, x_m \rangle$ ve $Y = \langle y_1, y_2, \dots, y_n \rangle$ iki dizi olsun ve maksimum uzunluklu ortak altdiziyi bulmak isteyelim. Bu kısım, dinamik programlama kullanarak LCS yi nasıl verimli şekilde çözeceğimizi gösterir.

Adım 1: Bir en uzun ortak altdizinin karakterize edilmesi

LCS problemini çözmek için kaba-kuvvet yaklaşımında, X in tüm altdizilerini sayımlayabilir ve her altdizinin Y nin de bir altdizisi olup olmadığını kontrol edebiliriz. X in herhangi bir alt dizisi $\{1, 2, \dots, m\}$ ile indislensin. X , 2^m altdiziye sahip olduğu için bu yaklaşım üstel zamanlıdır.

LCS problemleri aşağıdaki teoremden gösterildiği gibi bir optimum altyapı özelliğine sahiptir. Altproblemlerin doğal sınıfları, iki girdi dizisinin örneklerinin ikililerine karşılık gelir. Tam olarak, bir $X = \langle x_1, x_2, \dots, x_m \rangle$ dizisi verilsin. X in i . önekini $X_i = \langle x_1, x_2, \dots, x_i \rangle$, $i = 0, 1, \dots, m$ olarak tanımlarız. Örneğin, eğer $X = \langle A, B, C, B, D, A, B \rangle$ ise $X_4 = \langle A, B, C, B \rangle$ dir ve X_0 boş dizidir.

Teorem 15.1 (bir LCS'nin optimal altyapısı)

$X = \langle x_1, x_2, \dots, x_m \rangle$ ve $Y = \langle y_1, y_2, \dots, y_n \rangle$ diziler ve $Z = \langle z_1, z_2, \dots, z_k \rangle$ X ve Y nin herhangi bir LCS'si olsun.

1. $x_m = y_n$ ise $z_k = x_m = y_n$ ve Z_{k-1} , X_{m-1} ve Y_{n-1} in bir LCS'sidir.
2. $x_m \neq y_n$ ise $z_k \neq x_m$ olduğundan Z , X_{m-1} ve Y nin bir LCS'sidir.
3. $x_m \neq y_n$ ise $z_k \neq y_n$ olduğundan Z , X ve Y_{n-1} in bir LCS'sidir.

Kanıt (1) Eğer $z_k \neq x_m$ ise $x_m = y_n$ i Z ye ekleyerek X ve Y nin $k + 1$ uzunluğunda bir ortak altdizisini elde edebiliriz. Bu Z nin, X ve Y nin bir en uzun ortak altdizisi olma kabulüyle çelişir. Bu nedenle, $z_k = x_m = y_n$ olmalıdır. Artık, Z_{k-1} öneki, X_{m-1} ve Y_{n-1} in $k - 1$ uzunluklu ortak altdizisidir. Bunun bir LCS olduğunu gösterelim. Tersini kabul ederek, X_{m-1} ve Y_{n-1} in $k - 1$ den daha uzun bir ortak altdizisi W olsun. $x_m = y_n$ i W ya ekleyerek X ve Y nin k dan daha uzun bir altdizisi oluşturulur. Bu bir çelişkidir.

(2) Eğer $z_k \neq x_m$ ise, Z , X_{m-1} ve Y nin bir ortak alt dizisidir. Eğer X_{m-1} ve Y nin k dan daha uzun bir W ortak altdizisi varsa W aynı zamanda X_m ve Y nin bir ortak altdizisidir. Bu Z nin, X ve Y nin bir LCS kabulüyle çelişir.

(3) Bu ispat (2) ile benzerdir. ■

Teorem 15.1 in en uzun ortak altdizileri karakterize etme yöntemi, bize iki dizinin bir LCS'sinin, bu iki dizini örneklerinin bir LCS'sini içerdiğini ifade etmek-

tedir. Bu nedenle, LCS problemi optimal altyapı özelliğini sağlar. Bir özyinelemeli çözüm, birazdan göreceğimiz gibi çakışan altproblemler özelliğini de sağlar.

Adım 2: Bir özyinelemeli çözüm

Teorem 15.1, $X = \langle x_1, x_2, \dots, x_m \rangle$ ve $Y = \langle y_1, y_2, \dots, y_n \rangle$ nin bir LCS'sini bulurken bir veya iki altproblemi incelememiz gerektiğini gösterir. Eğer $x_m = y_n$ ise, X_{m-1} ve Y_{n-1} in bir LCS'sini bulmalıyız. $x_m = y_n$ i bu LCS'ye ekleyerek X ve Y nin bir LCS'sini elde ederiz. Eğer $x_m \neq y_n$ ise, şu iki altproblemi çözmeliyiz: X_{m-1} ve Y nin bir LCS'sini bulma ve X ve Y_{n-1} in bir LCS'sini bulma. Bu iki LCS'nin hangisi büyükse o X ve Y nin bir LCS'sidir. Bu durumda, tüm ihtimalleri örttüğü için, optimal altproblem çözümlerinden birinin X ve Y nin bir LCS'si olduğunu biliriz.

LCS problemlerinde, kolayca çakışan altproblemler özelliğini görebiliriz. X ve Y nin bir LCS'sini bulmak için, X ve Y_{n-1} in ve X_{m-1} ve Y nin LCS'lerini bulma gereksinimi duyarız. Fakat, bu altproblemlerin her biri X_{m-1} ve Y_{n-1} in bir LCS'sini bulma altproblemini içerir. Diğer birçok altproblem alt altproblemleri paylaşır.

Matris-zinciri çarpım problemindeki gibi, LCS problemlerinin özyinelemeli çözümü, bir optimum çözümün değeri için bir özyineleme oluşturmayı gerektirir. X_i ve Y_j dizilerinin bir LCS'sinin uzunluğunu $c[i, j]$ olarak tanımlayalım. Eğer $i = 0$ veya $j = 0$ ise dizilerden biri sıfır uzunluktadır ve böylece LCS sıfır uzunluktur. LCS'nin optimum altyapısı aşağıdaki özyinelemeli formülü verir:

$$c[i, j] = \begin{cases} 0, & i = 0 \text{ veya } j = 0 \text{ ise,} \\ c[i - 1, j - 1] + 1, & i, j > 0 \text{ ve } x_i = y_j \text{ ise,} \\ \max(c[i, j - 1], c[i - 1, j]) & i, j > 0 \text{ ve } x_i \neq y_j \text{ ise.} \end{cases} \quad (15.9)$$

Bu özyinelemeli formülasyonda, problemin bir koşulunun hangi altproblemleri dikkate alabileceğimizi sınırlandırdığını gözlemleyiniz. $x_i = y_i$ olduğunda, X_{i-1} ve Y_{j-1} in bir LCS bulma altproblemini dikkate almalıyız. Aksi halde, X_i ve Y_{j-1} in ve X_{i-1} ve Y_j nin bir LCS'sinin bulunacağı iki altproblemi dikkate alırız. Çubuk kesme ve matris zinciri çarpımında incelediğimiz önceki dinamik programlama algoritmalarında, problemdeki koşullara göre hiçbir altproblemi göz ardı etmedik. Bir LCS bulma, sadece problemdeki koşullara dayalı olarak altproblemleri göz ardı eden dinamik programlama algoritmasına bağlı değildir. Örnek olarak, mesafe-düzenleme problemi (Problem 15.5 e bakınız) bu karakteristiktir.

Adım 3: Bir LCS'nin hesaplanması

Denklem (15.9) a dayanarak, iki dizinin bir LCS'sini hesaplamak için üstel zamanlı bir algoritmayı kolayca yazabiliriz. LCS problemi sadece $\Theta(m \cdot n)$ fark-

lı altprobleme sahip olduğu için, çözümleri hesaplayacak aşağı-yukarı dinamik programlama algoritması kullanabiliriz.

LCS-LENGTH prosedürü $X = \langle x_1, x_2, \dots, x_m \rangle$ ve $Y = \langle y_1, y_2, \dots, y_n \rangle$ dizilerini girdi olarak alır. $c[i, j]$ değerlerini $c[0..m, 0..n]$ tablosunda tutar ve kayıtları **satır-önce-likli** sırada tutar. (Yani prosedür, c nin ilk satırını soldan sağa doldurur, ardından ikinci satıra geçer ve böyle devam eder.) Prosedür aynı zamanda bir optimum çözüm oluşturmamız için $b[1..m, 1..n]$ tablosunu da tutar. Sezgisel olarak, $b[i, j]$ ve $c[i, j]$ hesaplanırken seçilen optimal altproblemin çözümüne uygun gelen tablo kaydını işaret eder. Prosedür, b ve c tablolarını geri döndürür; $c[m, n]$, X ve Y nin bir LCS'sinin uzunluğunu saklar.

LCS-LENGTH(X, Y)

```

1   $m = X.length$ 
2   $n = Y.length$ 
3   $b[1..m, 1..n]$  ve  $c[0..m, 0..n]$  yeni tablolar olsun
4  for  $i = 1$  to  $m$ 
5       $c[i, 0] = 0$ 
6  for  $j = 0$  to  $n$ 
7       $c[0, j] = 0$ 
8  for  $i = 1$  to  $m$ 
9      for  $j = 1$  to  $n$ 
10         if  $x_i == y_j$ 
11              $c[i, j] = c[i - 1, j - 1] + 1$ 
12              $b[i, j] = \nwarrow$ 
13         elseif  $c[i - 1, j] \geq c[i, j - 1]$ 
14              $c[i, j] = c[i - 1, j]$ 
15              $b[i, j] = \uparrow$ 
16         else  $c[i, j] = c[i, j - 1]$ 
17              $b[i, j] = \leftarrow$ 
18  return  $c$  ve  $b$ 
```

Şekil 15.8, $X = \langle A, B, C, B, D, A, B \rangle$ ve $Y = \langle B, D, C, A, B, A \rangle$ dizilerinde LCS-LENGTH tarafından üretilen tabloları gösterir. Bu prosedürün çalışma zamanı $\Theta(m, n)$ dir, çünkü her tablo kaydını hesaplamak için $\Theta(1)$ zaman harcanır.

Adım 4: Bir LCS'nin oluşturulması

LCS-LENGTH tarafından döndürülen b tablosu, $X = \langle x_1, x_2, \dots, x_m \rangle$ ve $Y = \langle y_1, y_2, \dots, y_n \rangle$ in bir LCS'sini hızlıca oluşturmamız için olanak sağlar. $b[m, n]$ den başlar ve okları takip ederek tabloda ilerleriz. $b[i, j]$ de bir $\nwarrow$ ile karşılaştığımızda, $x_i = y_j$ LCS'nin bir elemanıdır. Bu metotla, ters sırada bu LCS'nin

		j	0	1	2	3	4	5	6
i	x_i	y_j	B	D	C	A	B	A	
0			0	0	0	0	0	0	0
1	A		0	0	0	0	1	1	1
2	B		0	1	1	1	1	2	2
3	C		0	1	1	2	2	2	2
4	B		0	1	1	2	2	3	3
5	D		0	1	2	2	2	3	3
6	A		0	1	2	2	3	3	4
7	B		0	1	2	2	3	4	4

Şekil 15.8 c ve b tabloları $X = \langle A, B, C, B, D, A, B \rangle$ ve $Y = \langle B, D, C, A, B, A \rangle$ üzerinde LCS-LENGTH tarafından hesaplanmıştır. i . satır ve j . sütundaki kare, $c[i, j]$ değerini ve $b[i, j]$ nin değeri için uygun oku tutar. $c[7, 6]$ daki 4 kaydı – tablonun sağ altındaki – X ve Y nin bir $\langle B, C, B, A \rangle$ LCS'sinin uzunluğudur. $i, j > 0$ için, $c[i, j]$ kaydı sadece, $x_i = y_j$ ve $c[i-1, j]$ den önce hesaplanan $c[i-1, j]$, $c[i, j-1]$ ve $c[i-1, j-1]$ kayıtlarına bağlıdır. Bir LCS'nin elemanlarını tekrar kurmak için, sağ alt köşeden $b[i, j]$ oklarını takip edin; seri gölgelendirilmiştir. Gölgeli serideki her “↖”, bir LCS'nin üyesi olan $x_i = y_j$ için işaretlenmiş bir kayda karşılık gelir.

elemanlarını buluruz. Aşağıdaki özyinelemeli prosedür X ve Y nin bir LCS'sini baştan sona düzgün olarak yazdırır. İlk çağrı, PRINT-LCS($b, X, X.length, Y.length$) dir.

PRINT-LCS(b, X, i, j)

```

1  if  $i == 0$  veya  $j == 0$ 
2      return
3  if  $b[i, j] == \text{“↖”}$ 
4      PRINT-LCS( $b, X, i - 1, j - 1$ )
5      print  $x_i$ 
6  elseif  $b[i, j] == \text{“↑”}$ 
7      PRINT-LCS( $b, X, i - 1, j$ )
8  else PRINT-LCS( $b, X, i, j - 1$ )

```

Şekil 15.8 deki b tablosu için bu prosedür, $BCBA$ yazdırır. Prosedür $O(m + n)$ zaman alır çünkü her özyinelemeli çağrıda i ve j den biri azalmaktadır.

Kodun iyileştirilmesi

Bir algoritma geliştirildiğinde, genellikle kullanılan zaman veya alanda iyileştirme yapılabilir. Bazı değişiklikler kodu basitleştirebilir ve sabit katsayı ile iyileştirme yapabilir fakat diğer taraftan performansta hiç asimptotik iyileşme olmayabilir. Bazılarında, zaman ve alanda, azımsanmayacak asimptotik kazançlar sağlayabilir.

LCS algoritmasında, örnek olarak, b tablosunu tamamen eleyebiliriz. Her bir $c[i, j]$ kaydı sadece 3 diğer c tablo kaydına bağlıdır: $c[i - 1, j - 1]$, $c[i - 1, j]$ ve $c[i, j - 1]$. $c[i, j]$ değeri için b tablosunu incelemeden, $c[i, j]$ yi hesaplamak için bu üç değerden hangisini kullanacağımızı belirlemek $O(1)$ zaman alır. Bu nedenle, PRINT-LCS ye benzer bir prosedür kullanarak bir LCS'yi $O(m + n)$ zamanda oluşturabiliriz. (Alıştırma 15.4-2, bu kabakodu istemektedir.) Bu yöntemle $\Theta(m \cdot n)$ alan kazanmamıza rağmen, bir LCS hesabı için yedek alan ihtiyacı, asimptotik olarak azalmaz çünkü c tablosu için $\Theta(m \cdot n)$ alana ihtiyaç duyarız.

PRINT-LCS için asimptotik alan ihtiyacını indirgeyebiliriz çünkü bir seferde c tablosunun sadece iki satırına gerek duyar: hesaplanan satır ve önceki satır. (Aslında, Alıştırma 15.4-4 ün istediği gibi, bir LCS'nin uzunluğunu hesaplamak üzere c nin bir satırının alanından biraz daha fazlasını kullanabiliriz.) Eğer sadece bir LCS'nin uzunluğunu hesaplamaya gerek varsa, bu iyileştirme çalışır; eğer bir LCS'nin elemanlarını oluşturmak gerekirse, bu daha küçük tablo $O(m + n)$ zamanda adımları izlemek için yeterli bilgiyi tutamaz.

Alıştırmalar

15.4-1

$\langle 1, 0, 0, 1, 0, 1, 0, 1 \rangle$ ve $\langle 0, 1, 0, 1, 1, 0, 1, 1, 0 \rangle$ nin bir LCS'sini belirleyiniz.

15.4-2

b tablosunu kullanmadan, $O(m + n)$ zamanda $X = \langle x_1, x_2, \dots, x_m \rangle$ ve $Y = \langle y_1, y_2, \dots, y_n \rangle$ orijinal dizileri ve tamamlanmış c tablosundan bir LCS oluşturmak için kabakod yazınız.

15.4-3

$O(mn)$ zamanda çalışan LCS-LENGTH in bir ön belleğe almalı versiyonunu yazınız.

15.4-4

$O(1)$ ilave alan ile, sadece c tablosundaki $2 \cdot \min(m, n)$ kaydı kullanarak bir LCS'nin uzunluğunun nasıl hesaplanacağını gösteriniz. Sonra, $O(1)$ ilave alan ve $\min(m, n)$ kayıt kullanarak aynı işlemin nasıl yapılacağını gösteriniz.

15.4-5

n adet sayı içeren bir dizinin en uzun monoton artan altdizisini bulmak için $O(n^2)$ zamanlı bir algoritma yazınız.

15.4-6 *

n adet sayı içeren bir dizinin en uzun monoton artan altdizisini bulmak için $O(n \lg n)$ zamanlı bir algoritma yazınız. (*İpucu:* Dikkat ediniz ki, i uzunluklu aday alt dizisinin son elemanı en azından $i - 1$ uzunluklu aday altdizisinin son elemanı kadar büyüktür. Aday altdizileri, girdi dizisini referans göstererek elde ediniz.)

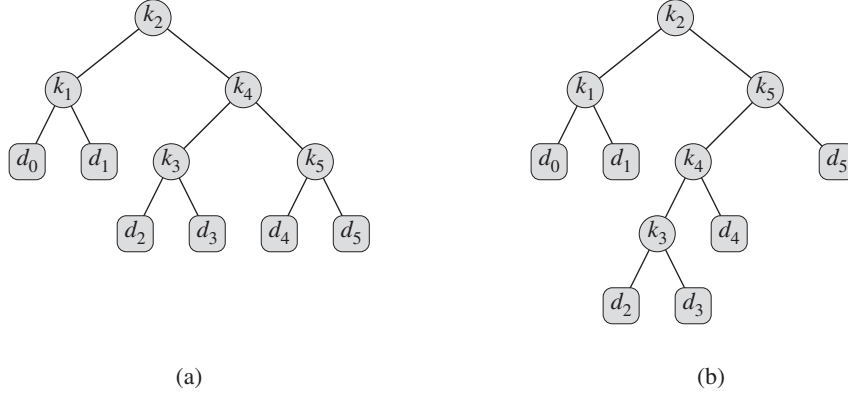
15.5 Optimal ikili arama ağaçları

Farz edelim ki, İngilizce'den Fransızca'ya metin dönüştürecek bir program tasarlıyorsunuz. Metindeki her bir İngilizce kelime için Fransızca karşılığına bakma gereksinimi duyarız. Bu bakma işlemlerini n İngilizce kelimenin anahtar ve Fransızca karşılıklarının uzak veri olduğu bir ikili arama ağacı kurarak gerçekleştirebiliriz. Metindeki her bir bireysel kelime için ağacı arayacağımızdan dolayı aramada harcanan toplam zamanın mümkün mertebe az olmasını isteriz. Bir kırmızı-siyah ağaç veya herhangi diğer dengeli ikili arama ağacı kullanarak kelime başına $O(\lg n)$ arama zamanını garanti edebiliriz. Kelimeler farklı frekanstadır ve nadir kullanılan *machicolation* kelimesi köke yakın görünürken sık kullanılan *the* kelimesi kökten uzak görünür. Bu tür bir işlem çeviriyi yavaşlatabilir çünkü ikili arama ağacında bir anahtar için arama yaparken ziyaret edilen düğümlerin sayısı, 1 ile arama yapılan anahtarı içeren düğümün derinliğinin toplamına eşittir. Metinde sıkça kullanılan kelimelerin köke daha yakın yerleştirilmesini isteriz.⁶ Ayrıca, metindeki bazı kelimeler Fransızca karşılığa sahip olmayabilir,⁷ ve bu tür kelimeler ikili arama ağacında yer almaz. Her kelimenin ne sıklıkta olduğunu bilsek, tüm aramalarda ziyaret edilen düğümlerin sayısını minimize etmek için bir ikili arama ağacını nasıl organize ederiz.

İhtiyaç duyduğumuz bir **optimal ikili arama ağacıdır**. Sıralanmış n farklı anahtarlı bir $K = \langle k_1, k_2, \dots, k_n \rangle$ dizisi verilsin ($k_1 < k_2 < \dots < k_n$) ve bu anahtarlardan bir ikili arama ağacı kurmayı isteyelim. Her bir k_i anahtarı için, bir p_i arama olasılığımız olsun. Bazı aramalar, K da olmayan değerler için olabilir ve böylece $n + 1$ “yapay anahtarımız” $d_0, d_1, d_2, \dots, d_n$ olur. Bu yapay anahtarlar, K da olmayan değerleri gösterir. Özel olarak, d_0, k_1 den küçük tüm değerleri; d_n, k_n den büyük

⁶Eğer metnin öznesi kale mimarisi olsaydı, *machicolation*'u köke yakın görmek isteyebilirdik.

⁷Evet, *machicolation* bir Fransızca karşılığa sahiptir: *mâchicoulis*.



Şekil 15.9 Aşağıdaki olasılıklarla $n = 5$ anahtarlı bir küme için iki adet ikili arama ağacı:

i	0	1	2	3	4	5
p_i		0.15	0.10	0.05	0.10	0.20
q_i	0.05	0.10	0.05	0.05	0.05	0.10

(a) 2.80 beklenen arama maliyetli bir ikili arama ağacı. (b) 2.75 beklenen arama maliyetli bir ikili arama ağacı. Bu optimaldir.

tüm değerleri ve $i = 1, 2, \dots, n-1$ için d_i yapay anahtarı k_i ve $k_i + 1$ arasındaki tüm değerleri ifade eder. Her bir d_i yapay anahtarı için, d_i yi aramaya karşılık gelen q_i olasılığı vardır. Şekil 15.9, $n = 5$ anahtarlı bir küme için 2 ikili arama ağacı gösterir. Her bir k_i anahtarı bir iç düğümdür ve her bir d_i yapay anahtarı bir yapraktır. Her arama ya başarılıdır (k_i anahtarını bularak) veya başarısızdır (d_i yapay anahtarını bularak) ve böylece

$$\sum_{i=1}^n p_i + \sum_{i=0}^n q_i = 1 \quad (15.10)$$

dir.

Her bir anahtar ve her bir yapay anahtar için aramaların olasılıklarına sahip olduğumuzdan, verilen bir T ikili arama ağacında bir aramanın beklenen maliyetini belirleyebiliriz. Farz edelim ki, bir aramanın gerçek maliyeti incelenen düğümlerin sayısına eşit olsun. Yani, T deki arama ile bulunan düğümün derinliği ile 1 in toplamına eşittir. O zaman, T deki bir aramada beklenen maliyet

$$\begin{aligned} E[T \text{ deki arama maliyeti}] &= \sum_{i=1}^n (\text{depth}_T(k_i) + 1) \cdot p_i + \sum_{i=0}^n (\text{depth}_T(d_i) + 1) \cdot q_i \\ &= 1 + \sum_{i=1}^n \text{depth}_T(k_i) \cdot p_i + \sum_{i=0}^n \text{depth}_T(d_i) \cdot q_i \end{aligned} \quad (15.11)$$

dir. depth_T , T ağacındaki bir düğümün derinliğini gösterebilir. Son eşitlik, (15.10) denkleminde elde edilir. Şekil 15.9(a) da beklenen arama maliyeti düğümünün düğümle şöyle hesaplayabiliriz:

düğüm	derinlik	olasılık	katkı
k_1	1	0.15	0.30
k_2	0	0.10	0.10
k_3	2	0.05	0.15
k_4	1	0.10	0.20
k_5	2	0.20	0.60
d_0	2	0.05	0.15
d_1	2	0.10	0.30
d_2	3	0.05	0.20
d_3	3	0.05	0.20
d_4	3	0.05	0.20
d_5	3	0.10	0.40
Total			2.80

Verilen bir olasılık kümesi için, beklenen maliyeti en küçük olan bir ikili arama ağacı kurmak isteyelim. **Optimal ikili arama ağacı** gibi bir ağacı çağırırız. Şekil 15.9(b), şeklin altında verilen olasılıklar için bir optimal ikili arama ağacı gösterir; beklenen maliyet 2.75 tir. Bu örnek, bir optimum ikili arama ağacı için yüksekliği en küçük olan bir ağacın gerekli olmadığını gösterir. Her zaman en yüksek olasılıklı anahtar köke koyarak da bir optimum ikili arama ağacı kuramayız. Burada k_5 anahtarı tüm anahtarlardan en yüksek arama olasılığına sahiptir lakin, optimum ikili arama ağacının kökünde k_2 vardır. (k_5 in kökte olduğu herhangi bir ikili arama ağacının beklenen en düşük maliyeti 2.85 tir.)

Matris-zinciri çarpımındaki gibi tüm ihtimallerin tam bir kontrolü verimli bir algoritma oluşturmayı engeller. Bir ikili arama ağacı oluşturmak için herhangi n düğümlü ikili ağaç düğümlerini $k_1, k_2, \dots, k_n$ anahtarları ile etiketleyebiliriz ve yapay anahtarları yaprak olarak ekleyebiliriz. Problem 12-4 te n düğümlü ikili ağaçların sayısının $\Omega(4^n/n^{3/2})$ olduğunu gördük. Böylece tam bir aramada üstel sayıdaki ikili arama ağaçlarını incelemek zorunda kalacaktık. Normal bir şekilde bu problemi dinamik programlama ile çözecektik.

Adım 1: Bir optimal ikili arama ağacının yapısı

Optimal ikili arama ağacının optimal altyapısını karakterize etmek için altağaçları inceleyeceğiz. Bir ikili ağacın herhangi bir altağacını düşünelim. Bu bir $1 \leq i \leq j \leq n$ için $k_i, \dots, k_j$ komşu aralığında anahtarlar içermelidir. İlave olarak, $k_i, \dots, k_j$ anahtarlarını içeren bir altağaç, aynı zamanda onun $d_{i-1}, \dots, d_j$ yapay anahtarlarını yapraklar olarak içermelidir.

Artık optimal altyapıyı kurabiliriz: eğer bir T optimal ikili arama ağacı, $k_i, \dots, k_j$ anahtarlarını içeren bir T' altağacına sahip ise, bu T' altağacı $d_{i-1}, \dots, d_j$

yapay anahtarlı ve $k_p, \dots, k_j$ anahtarlı altproblemler gibi optimal olmalıdır. Sıradan kes-ve-yapıştır işlemine başvurulur. Eğer beklenen maliyeti T' den daha küçük bir T'' altağacı olsaydı, T den T'' nü keser ve T'' nü yapıştırırdık, T den daha az beklenen maliyetli bir ikili arama ağacını elde etmiş olurduk. Bu, T nin optimalliği ile çelişir.

Altproblemlerin optimal çözümlerinden problemin bir optimal çözümünü kullanabilmeyi göstermek için optimal altyapıyı kullanma ihtiyacı duyarız. $k_p, \dots, k_j$ anahtarları verilsin. Bu anahtarlardan biri, k_r ($i \leq r \leq j$), bu anahtarları içeren bir optimal altağacın köküdür. k_r kökünün sol altağacı $k_p, \dots, k_{r-1}$ anahtarlarını (ve $d_{i-1}, \dots, d_{r-1}$ yapay anahtarlarını) ve sağ altağacı $k_{r+1}, \dots, k_j$ anahtarlarını (ve $d_r, \dots, d_j$ yapay anahtarlarını) içerir. $i \leq r \leq j$ için tüm k_r aday köklerini incelediğimiz, $k_p, \dots, k_{r-1}$ ve $k_{r+1}, \dots, k_j$ yi içeren tüm optimal ikili arama ağaçlarını belirlediğimiz sürece bir optimal ikili arama ağacını bulacağımızı garantilemiş oluruz.

Boş altağaçlar ile ilgili kayda değer bir ayrıntı vardır. $k_p, \dots, k_j$ anahtarlı bir altağaçta k_i yi kök olarak seçelim. Yukarıdaki argümanı kullanarak, k_i nin sol altağacı $k_p, \dots, k_{i-1}$ anahtarlarını içerir. Bu diziyi hiç anahtar içermemiş olarak değerlendiririz. Fakat altağaçların aynı zamanda yapay anahtarları da içerdiklerini unutmayalım. $k_p, \dots, k_{i-1}$ anahtarlarını içeren bir alt ağacın hiçbir gerçek anahtara sahip olmadığını tek bir d_{i-1} yapay anahtarını içerdiğini kabul ederiz. Benzer şekilde, eğer k_j yi kök olarak seçersek k_j nin sağ alt ağacı $k_{j+1}, \dots, k_j$ anahtarlarını içerir; bu sağ altağaç gerçek anahtar içermez, fakat d_j yapay anahtarını içerir.

Adım 2: Bir özyinelemeli çözüm

Bir optimal çözümün değerini özyinelemeli olarak tanımlamaya hazırız. Altproblem tanım kümemizi, $i \geq 1, j \leq n$ ve $j \geq i - 1$ iken $k_p, \dots, k_j$ anahtarlarını içeren bir optimal ikili arama ağacını aramadaki gibi belirleriz. ($j = i - 1$ olduğunda, hiçbir gerçek anahtar yoktur, sadece d_{i-1} yapay anahtarına sahip oluruz.) $e[i, j]$ yi, $k_p, \dots, k_j$ anahtarlarını içeren bir optimal ikili arama ağacını aramanın beklenen maliyeti olarak tanımlayalım. Nihayetinde, $e[1, n]$ i hesaplamayı umalım.

Bu basit durum, $j = i - 1$ iken oluşur. Ardından, sadece d_{j-1} yapay anahtarına sahip oluruz. Beklenen arama maliyeti $e[i, i - 1] = q_{i-1}$ dir.

$j \geq i$ iken, $k_p, \dots, k_j$ den bir k_r kökü seçme gereği duyarız ve sol altağacı olarak $k_p, \dots, k_{r-1}$ anahtarlı bir optimal ikili arama ağacı ve sağ altağacı olarak $k_r, \dots, k_j$ anahtarlı bir optimal ikili arama ağacı oluştururuz. Bir düğümün bir altağacı olduğunda, bir altağacın beklenen arama maliyeti ne olur? Altağaçtaki her düğümün derinliği 1 birim artar. Denklem (15.11) den, bu altağacın beklenen arama maliyeti, altağaçtaki tüm olasılıkların toplamı kadar artar. $k_p, \dots, k_j$ anahtarlı bir altağaç için, bu olasılıkların toplamını

$$w(i, j) = \sum_{l=i}^j p_l + \sum_{l=i-1}^j q_l \quad (15.12)$$

olarak gösterelim.

Bu nedenle, eğer $k_r, k_p, \dots, k_j$ anahtarlarını içeren bir optimal altağacın kökü ise

$e[i, j] = p_r + (e[i, r-1] + w(i, r-1)) + (e[r+1, j] + w(r+1, j))$ 'yi elde ederiz.

$$w(i, j) = w(i, r-1) + p_r + w(r+1, j) ,$$

için $e[i, j]$ yi

$$e[i, j] = e[i, r-1] + e[r+1, j] + w(i, j) . \quad (15.13)$$

olarak tekrar elde ederiz.

(15.13) özyinelemeli denklemi, kök olarak herhangi bir k_r düğümünü kullandığımızı bildiğimizi varsayar. En düşük beklenen arama maliyetini veren kökü seçeriz ve son özyinelemeli formülasyon aşağıdaki şekli alır:

$$e[i, j] = \begin{cases} q_{i-1} & j = i - 1 \text{ ise,} \\ \min_{i \leq r \leq j} \{e[i, r-1] + e[r+1, j] + w(i, j)\} & i \leq j \text{ ise.} \end{cases} \quad (15.14)$$

$e[i, j]$ değerleri, optimal ikili arama ağaçlarında beklenen arama maliyetlerini verir. Optimal ikili arama ağaçlarının yapısını izlememiz yardımcı olmak üzere, $k_p, \dots, k_j$ anahtarlarını içeren bir optimal ikili arama ağacının kökü k_r olduğu yerdeki r indisi için, $root[i, j]$ yi $1 \leq i \leq j \leq n$ olmak üzere tanımlarız. $root[i, j]$ nin değerlerinin nasıl hesaplanacağını göreceğimize rağmen, Alıştırma 15.5-1 deki gibi bu değerlerden bir optimal ikili arama ağacını oluşturulması bırakılacaktır.

Adım 3: Bir optimal ikili arama ağacının beklenen arama maliyetinin hesaplanması

Bu aşamada, optimal ikili arama ağaçlarını karakterize etmemiz ile matris zinciri çarpımı arasında bazı benzerlikleri fark edebilirsiniz. Problem tanım kümelerinin ikisi için de, altproblemimiz komşu indis aralıklarından oluşur. (15.14) denkleminin direkt özyinelemeli bir uygulaması, direkt özyinelemeli bir matris zinciri çarpımı algoritması olarak verimsiz olabilir. Bunun yerine $e[i, j]$ değerlerini bir $e[1..n+1, 0..n]$ tablosunda tutarız. İlk indis n yerine $n+1$ e kadar çalışır çünkü sadece d_n yapay anahtarını içeren bir altağaca sahip olmak için $e[n+1, n]$ i hesaplamalı ve saklamalıyız. $j \geq i-1$ iken sadece $e[i, j]$ kayıtlarını kullanırız. $k_p, \dots, k_j$ anahtarlarını içeren alt ağacın kökünü kaydetmek için de bir $root[i, j]$ tablosunu kullanırız. Bu tablo, sadece $1 \leq i \leq j \leq n$ olduğundaki kayıtları kullanır.

Verimlilik için bir başka tabloya gerek duyacağız. Her $e[i, j]$ yi hesaplarken $\Theta(j-i)$ toplama maliyeti alır. $w(i, j)$ nin değerini hesaplamak yerine bu değerleri

bir $w[1..n+1, 0..n]$ tablosunda tutarız. Bu durumda $w[i, i-1] = q_{i-1}$ ($1 \leq i \leq n+1$) yi hesaplarız. $j \geq i$ için,

$$w[i, j] = w[i, j-1] + p_j + q_j \quad (15.15)$$

dir. Bu nedenle, her bir $\Theta(1)$ zaman alan $w[i, j]$ nin $\Theta(n^2)$ değerini hesaplayabiliriz.

Aşağıdaki kabakod, girdi olarak, $p_1, \dots, p_n$ ve $q_0, \dots, q_n$ olasılıklarını ve n boyutunu alır ve e ve $root$ tablolarını geri döndürür.

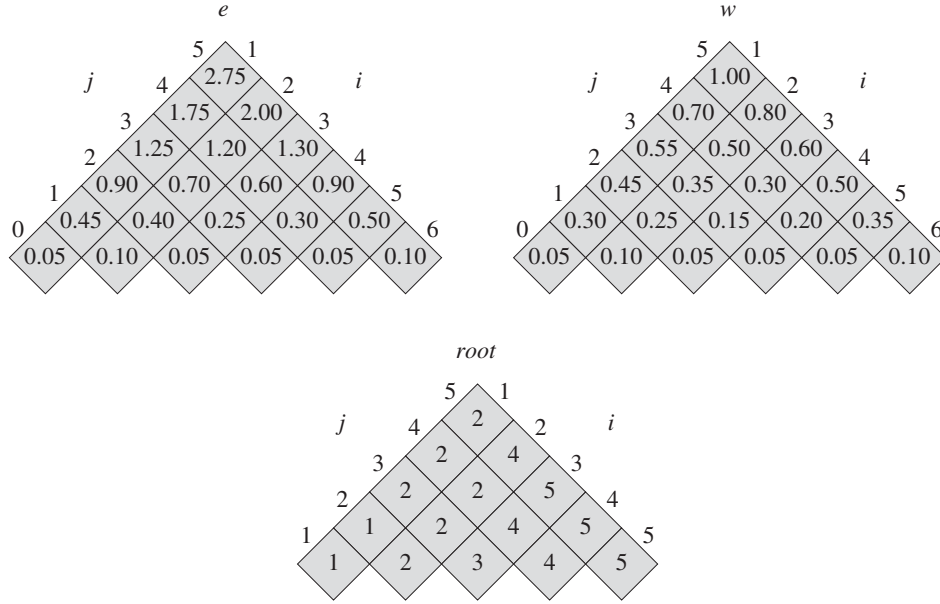
OPTIMAL-BST(p, q, n)

```

1   $e[1..n+1, 0..n], w[1..n+1, 0..n]$ 
   ve  $root[1..n, 1..n]$  yeni tablolar olsun
2  for  $i = 1$  to  $n+1$ 
3       $e[i, i-1] = q_{i-1}$ 
4       $w[i, i-1] = q_{i-1}$ 
5  for  $l = 1$  to  $n$ 
6      for  $i = 1$  to  $n-l+1$ 
7           $j = i+l-1$ 
8           $e[i, j] = \infty$ 
9           $w[i, j] = w[i, j-1] + p_j + q_j$ 
10         for  $r = i$  to  $j$ 
11              $t = e[i, r-1] + e[r+1, j] + w[i, j]$ 
12             if  $t < e[i, j]$ 
13                  $e[i, j] = t$ 
14                  $root[i, j] = r$ 
15 return  $e$  ve  $root$ 
```

Yukarıdaki tanımdan ve Kısım 15.2 deki MATRIX-CHAIN-ORDER e benzerliğinden, bu prosedürü oldukça anlaşılır bulmalısınız. 2-4 satırlarındaki **for** döngüsü, $e[i, i-1]$ ve $w[i, i-1]$ değerlerini ilklendirir. 5-14 satırlarındaki **for** döngüsü, tüm $1 \leq i \leq j \leq n$ lerde $e[i, j]$ ve $w[i, j]$ yi hesaplamak için (15.14) ve (15.15) öz-yinelemelerini kullanır. İlk iterasyonda $l = 1$ iken, döngü $i = 1, 2, \dots, n$ için $e[i, i]$ ve $w[i, i]$ yi hesaplar. İkinci iterasyon, $l = 2$, $i = 1, 2, \dots, n-1$ için $e[i, i+1]$ ve $w[i, i+1]$ i hesaplar ve böyle devam eder. 10-14 satırlarındaki, en içteki **for** döngüsü, $k, \dots, k_j$ anahtarlarını içeren bir optimal ikili arama ağacının kökü olarak kullanılacak k_r anahtarını belirlemek için her bir r aday indisini dener. Bu **for** döngüsü, kök olarak kullanmak için, daha iyi bir anahtar bulduğu zaman r indisinin mevcut değerini saklar.

Şekil 15.10, Şekil 15.9 da gösterilen anahtar dağılımındaki OPTIMAL-BST prosedürü tarafından hesaplanan $e[i, j]$, $w[i, j]$ ve $root[i, j]$ tablolarını gösterir.



Şekil 15.10 Şekil 15.9 da gösterilen anahtar dağılımındaki OPTIMAL-BST prosedürü tarafından hesaplanan $e[i,j]$, $w[i,j]$ ve $root[i,j]$ tabloları. Tablolar, köşegen yatay olacak şekilde döndürülmüştür.

Şekil 15.5 teki matris zinciri çarpımı örneğindeki gibi, tablolar köşegenler yatay olacak şekilde döndürülmüştür. OPTIMAL-BST, her bir satır için satırları aşağıdan yukarıya ve soldan sağa hesaplar.

OPTIMAL-BST prosedürü $\Theta(n^3)$ zaman alır. MATRIX-CHAIN-ORDER gibi **for** döngüleri iç içe üç tane olduğundan ve her birinin indisi en çok n defa çalışacağı için, çalışma zamanının $O(n^3)$ olduğunu kolayca söyleyebiliriz. MATRIX-CHAIN-ORDER de olduğu gibi OPTIMAL-BST deki döngü indisleri tam olarak aynı sınırlara sahip değildir. Bu nedenle, MATRIX-CHAIN-ORDER gibi, OPTIMAL-BST prosedürü $\Omega(n^3)$ zaman alır.

Alıştırmalar

15.5-1

$root$ tablosu verildiğinde, bir optimal ikili arama ağacının yapısını veren CONSTRUCT-OPTIMAL-BST($root$) prosedürünün kaba-kodunu yazınız. Şekil 15.10 daki örnek için prosedürünüz, Şekil 15.9(b) de gösterilen optimal ikili arama ağacına uygun gelen

k_2 kök
 k_1, k_2 nin sol çocuğu
 d_0, k_1 in sol çocuğu
 d_1, k_1 in sağ çocuğu
 k_3, k_2 nin sağ çocuğu
 k_4, k_3 in sol çocuğu
 k_3, k_4 ün sol çocuğu
 d_2, k_3 ün sol çocuğu
 d_3, k_3 ün sağ çocuğu
 d_4, k_4 ün sağ çocuğu
 d_5, k_5 in sağ çocuğu

yapısını yazdırmalıdır.

15.5-2

Aşağıdaki olasılıklara sahip, $n = 7$ anahtarlı bir küme için bir optimal ikili arama ağacının yapısını ve maliyetini belirleyiniz:

i	0	1	2	3	4	5	6	7
p_i		0.04	0.06	0.08	0.02	0.10	0.12	0.14
q_i	0.06	0.06	0.06	0.06	0.05	0.05	0.05	0.05

15.5-3

$w[i, j]$ tablosunu tutmak yerine, OPTIMAL-BST nin 9. satırında $w(i, j)$ değerini (15.12) denkleminle doğrudan hesapladığımızı ve hesaplanan bu değeri 11. satırda kullandığımızı farz edelim. Bu değişiklik OPTIMAL-BST nin asimptotik çalışma zamanını nasıl etkiler?

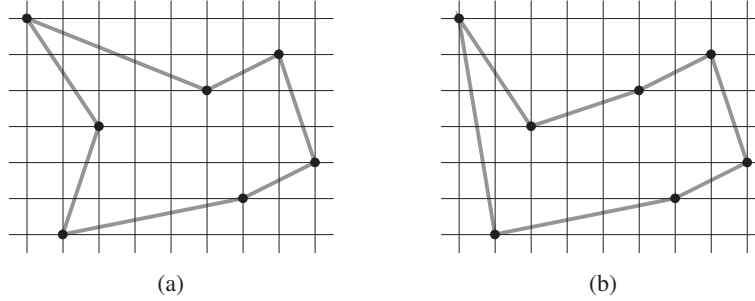
15.5-4 *

Knuth [212], tüm $1 \leq i < j \leq n$ ler için $root[i, j - 1] \leq root[i, j] \leq root[i + 1, j]$ yi sağlayan optimal altağaçların köklerinin her zaman var olduğunu göstermiştir. Bu durumu kullanarak, OPTIMAL-BST prosedürünü $\Theta(n^2)$ zamanda çalışması için düzenleyiniz.

Problemler

15-1 Bir yönlü çevrimsiz çizgede en uzun basit yol

Farz edelim ki, gerçek değerli ayrıt ağırlıkları olan bir yönlü çevrimsiz $G = (V, E)$ çizgesi ile iki ayrı s ve t tepeleri verilsin. s den t ye bir en uzun ağırlıklı basit yol bulmak için bir dinamik programlama yaklaşımı tanımlayın. Altproblem çizgesi nasıl görünür? Algoritmanızın verimliliği nedir?



Şekil 15.11 Birim sistemde gösterilen, yüzey üzerinde yedi nokta. (a) Yaklaşık 24.89 uzunluklu en kısa kapalı tur. Bu tur bitonik değildir. (b) Aynı noktalar kümesi için en kısa bitonik tur. Uzunluğu yaklaşık 25.58 dir.

15-2 En uzun palindrom alt dizisi

Bir *palindrom*, baştan ve sondan okunuşu aynı olan, bir alfabe üzerindeki boştan farklı bir dizidir. Palindrom örnekleri: 1 uzunluklu tüm katarlar, civic, racecar, aibohphobia (palindrom korkusu).

Verilen bir katarın içindeki en uzun palindromu bulmak için verimli bir algoritma öneriniz. Örneğin, character girdisi için algoritmanız carac ı döndürmelidir. Algoritmanızın çalışma zamanı nedir?

15-3 Bitonik Öklid gezgin satıcı problemi

Öklid gezgin satıcı probleminde, yüzey üzerinde n noktalı bir küme verilir ve n noktayı birleştiren en kısa kapalı turu bulmak isteriz. Şekil 15.11(a), 7 noktalı problemin çözümünü göstermektedir. Genel problem NP-zordur ve çözümünün bu nedenle polinom zamandan daha fazlasını gerektirdiğine inanılır. (Bölüm 34 e bakınız.)

J. L. Bentley, dikkatimizi *bitonik turlara* sınırlayarak problemi basitleştirebileceğimizi önermiştir. Yani, bu turlar, en sol noktadan başlar ve en sağdaki noktaya doğru ilerler ve daha sonra başlangıç noktasında doğru sol yönde gider. Şekil 15.11(b), aynı 7 noktanın en kısa bitonik turunu gösterir. Bu şekilde, polinom zamanlı algoritma mümkündür.

Bir optimal bitonik tur belirlemek için $O(n^2)$ zamanlı bir algoritma öneriniz. Aynı x koordinatına sahip iki nokta olmadığını ve reel sayılar üzerindeki tüm işlemlerin birim zaman aldığını farz edebilirsiniz. (*İpucu*: turun ilk kısmı için optimal olasılıkları koruyarak soldan sağa doğru tarama yapınız.)

15-4 Düzgün baskı

Bir yazıcıda, her karakteri aynı genişlikteki bir yazı tipi ile bir paragrafı düzgün bir biçimde yazdırma problemini düşünün. Girdi boyutu, karakterlerle ölçülmüş

$l_1, l_2, \dots, l_n$ uzunluklu n kelimenin bir dizisidir. Her biri en çok M karakter yer tutan birkaç satırda bu paragrafı düzgünce yazdırmak isteriz. Kriterimiz, aşağıdaki gibi “düzgünlüktür”. Eğer verilen bir satır $i \leq j$ iken i den j ye kadar sözcükleri içeriyorsa ve sözcükler arasında tam olarak bir boşluk bırakıyorsak, satır sonundaki fazladan boşlukların sayısı $M - j + i - \sum_{k=i}^j l_k$ dır ve sözcükleri satıra sığdırmak için negatif olmamalıdır. Son satır hariç tüm satırlar için, satır sonlarındaki fazladan boşluk karakterinin sayılarının toplamını minimize etmek isteriz. Bir yazıcıda, n kelimeli bir paragrafı düzgünce yazdırmak için bir dinamik programlama algoritması öneriniz. Algoritmanızın, çalışma zamanını ve bellek gereksinimini analiz ediniz.

15-5 Mesafe düzenleme

Bir $x[1..m]$ kaynak katarını $y[1..n]$ hedef katarına dönüştürmek için birkaç dönüşüm işlemi uygulayabiliriz. Amacımız, x ve y verildiğinde, x i y ye dönüştürecek bir dönüşüm serisi oluşturmaktır. Ara sonuçları tutmak için bir z dizisi—gerekli olduğu kadar karakter tutmak için yeteri kadar geniş kabul edilir- kullanırız. Başlangıçta z boştur ve program sonlandığında $j = 1, 2, \dots, n$ için $z[j] = y[j]$ ye sahip olmalıyız. i yi x den ve j yi z den elde ederiz ve z yi ve bu indisleri değiştirmek için operasyonlar işletilir. İlk olarak, $i = j = 1$ dir. Dönüşüm süresince x deki her bir karakteri incelememiz gereklidir ve dönüşüm işlemlerinin sonuna geldiğimizde $i = m + 1$ e sahip oluruz.

Altı dönüşüm işleminden seçim yapabiliriz:

Kopyala $z[j] = x[i]$ ile x den z ye bir karakter kopyala ve sonra i ve j yi arttır. Bu işlem $x[i]$ yi inceler.

Yer değiştir $z[j] = c$ yaparak x den bir karakteri c karakteri ile yer değiştir ve sonra i ve j yi arttır. Bu işlem $x[i]$ yi inceler.

Sil j yi yalnız bırakıp i yi arttırarak x den bir karakter sil. Bu işlem $x[i]$ yi inceler.

Ekle $z[j] = c$ yaparak z ye bir c karakteri ekle ve sonra i yi yalnız bırakarak j yi arttır. Bu işlem x deki hiçbir karakteri incelemeyiz.

Çevir x den z ye ters sırada kopyalayarak sıradaki iki karakteri çevir; $z[j] = x[i + 1]$ ve $z[j + 1] = x[i]$ yaparız ve $i = i + 2$ ve $j = j + 2$ olarak arttırırız. Bu işlem $x[i]$ ve $x[i + 1]$ i inceler.

Yok et $i = m + 1$ yaparak x den kalanları yok et. Bu işlem x de henüz incelenmemiş karakterleri inceler. Eğer çalıştırılırsa, bu işlem son operasyon işlem.

Bir örnek olarak, `algorithm` kaynak katarını `altruistic` e dönüştürmek için bir yol, aşağıdaki işlemler serisini kullanmaktır. Burada altı çizili karakterler işlemden sonraki $x[i]$ ve $z[j]$ dir:

İşlem	x	y
<i>başlangıç katarları</i>	<u>a</u> lgorithm	_
<i>kopyala</i>	a <u>l</u> gorithm	a_
<i>kopyala</i>	al <u>g</u> orithm	al_
<i>t ile yer değiştir</i>	alg <u>o</u> rithm	alt_
<i>sil</i>	algor <u>i</u> thm	alt_
<i>kopyala</i>	algori <u>t</u> hm	altr_
<i>u yu ekle</i>	algori <u>t</u> hm	altru_
<i>i yu ekle</i>	algori <u>t</u> hm	altrui_
<i>s yu ekle</i>	algori <u>t</u> hm	altruis_
<i>çevir</i>	algorith <u>m</u>	altruisti_
<i>c yu ekle</i>	algorith <u>m</u>	altruistic_
<i>yok et</i>	algorithm_	altruistic_

`algorithm` i `altruistic` e dönüştürmek için birkaç başka dönüşüm işlem serisi daha vardır.

Dönüşüm işlemlerinin her biri bir maliyete sahiptir. Bir işlemin maliyeti, o işlemin özel uygulamasına bağlıdır fakat her işlemin maliyetini sabit kabul ederiz. Bunun yanında, kopyala ve yer değiştir işlemlerinin maliyetlerinin, sil ve ekle işlemlerinin toplam maliyetlerinden daha az olduğunu kabul ederiz; aksi halde, kopyala ve yer değiştir işlemleri kullanılamazdı. Verilen bir dönüşüm işlemleri serisinin maliyeti serideki bireysel işlemlerin maliyetleri toplamına eşittir. Yukarıdaki seri için, `algorithm` i `altruistic` e dönüştürme maliyeti

$$(3 \cdot \text{maliyet}(\text{kopyala})) + \text{maliyet}(\text{yer değiştir}) + \text{maliyet}(\text{sil}) + (4 \cdot \text{maliyet}(\text{ekle})) + \text{maliyet}(\text{çevir}) + \text{maliyet}(\text{yok et})$$

dir.

- a. $x[1..m]$ ve $y[1..n]$ iki dizisi ve dönüşüm işlem maliyetlerinin kümesi verilsin, x den y ye **mesafe düzenleme**, x den y ye dönüşüm yapan en az pahalı olan işlem serisinin maliyetidir. $x[1..m]$ 'den $y[1..n]$ e mesafe düzenlemeyi bulan ve bir optimal işlem serisi yazdıran bir dinamik programlama algoritması tanımlayınız. Algoritmanızın çalışma zamanını ve bellek gereksinimini analiz ediniz.

Mesafe düzenleme problemi, iki DNA diziliminin sıralanması probleminin genelleştirilmesidir (örnek olarak Setubal ve Meidanis [310], (Kısım 3.2 ye bakınız). İki DNA dizilimini sıralarken benzerliklerini ölçmek için birçok yöntem vardır. x ve y dizilerini sıralamak için bir yöntem, iki dizideki keyfi konumlara

boşluklar ekleyerek (her ikisinin de sonunu içerecek şekilde) oluşturulur, böylece x' ve y' dizileri aynı uzunluktadır fakat aynı konuma karşılık gelecek bir boşluğa sahip değildirler (yani, hiçbir j konumu için, $x'[j]$ ve $y'[j]$ boşluk değildir). Ardından, her konuma bir “puan” atarız. j konumu aşağıdaki şekilde puanlanmaktadır:

- +1, eğer $x'[j] = y'[j]$ ve hiçbir boşluk değilse,
- -1, eğer $x'[j] \neq y'[j]$ ve hiçbir boşluk değilse,
- -2, ya $x'[j]$ ya da $y'[j]$ boşluk ise.

Sıralama için puan, bireysel konumların puanlarının toplamıdır. Örneğin, $x = \text{GATCGGCAT}$ ve $y = \text{CAATGTGAATC}$ dizilimleri verilsin, bir sıralama şöyledir:

```
G  ATCG  GCAT
CAAT  GTGAATC
- * * * * * + - + * *
```

Bir konumun altındaki +, bu konum için +1 puanı gösterir, -, -1 puanını ve *, -2 puanını gösterir. Böylece, bu sıralama $6 \cdot 1 - 2 \cdot 1 - 4 \cdot 2 = -4$ toplam puanına sahiptir.

- b.** Bir optimal sıralama bulma problemini, kopyala, yer değiştir, sil, ekle, çevir ve yok et dönüşüm işlemlerinin bir altkümesini kullanan mesafe düzenleme problemine nasıl dönüştürüleceğini açıklayınız.

15-6 Bir şirket partisi planlama

Profesör Stewart, bir şirket partisi planlayan bir kurumun genel müdürünün danışmanlığını yapmaktadır. Şirket hiyerarşik bir yapıdadır; yani, genel müdür kökte olacak şekilde yönetici bağlantıları bir ağaç oluşturmaktadır. Personel bürosu, her çalışanı (reel sayı olarak) eğlence derecesi ile sıralamıştır. Tüm katılanlar için partiyi eğlenceli yapmak üzere, genel müdür, bir çalışanın ve onun doğrudan bir yöneticisinin ikisinin de partiye katılmasını istememektedir.

Profesör Stewart’a, Kısım 10.4 te tanımlanan sol-çocuk ve sağ-kardeş gösterilimlerini kullanarak, kurumun yapısını tanımlayan ağaç verilmiştir. Ağacın her düğümü, bir çalışanın adını ve onun eğlence derecesini tutar. Misafirlerin eğlence derecelerinin toplamını maksimize edecek şekilde misafir listesini oluşturmak için bir algoritma öneriniz. Algoritmanızın çalışma zamanını analiz ediniz.

15-7 Viterbi algoritması

Ses tanıma için, bir $G = (V, E)$ yönlü çizgesinde dinamik programlama kullanabiliriz. Her $(u, v) \in E$ ayrıtı, seslerin bir sonlu Σ kümesinden bir $\sigma(u, v)$ sesi ile etiketlidir. Etiketlenmiş çizge, sınırlı bir dil konuşan bir insan için formal bir

modeldir. Ayrılmış bir $v_0 \in V$ tepesinden başlayarak çizgedeki her bir yol, model tarafından üretilen seslerin mümkün bir serisine uygun düşer. Yönlü bir yolun etiketini, bu yoldaki ayrıtların etiketlerinin birleştirilmesiyle tanımlarız.

- a.** Verimli bir algoritma tasarlayınız. v_0 ayrılmış tepeli bir ayrıt-etiketli G çizgesi ve Σ dan seslerin bir $s = \langle s_1, s_2, \dots, s_k \rangle$ dizisi verildiğinde, eğer böyle bir yol varsa v_0 dan başlayan ve etiket olarak s i kullanan, G de bir yol geri döndürür. Aksi halde, algoritma NO-SUCH-PATH cevabını döndürmelidir. Algoritmanızın çalışma zamanını analiz ediniz. (*İpucu:* Bölüm 22 deki kavramları yararlı bulabilirsiniz.)

Artık, her $(u, v) \in E$ ayrıtının, u tepesinden (u, v) ayrıtını geçmek için bir negatif olmayan $p(u, v)$ olasılığına sahip olduğunu ve uygun sesi ürettiğini kabul edelim. Herhangi bir tepenin atıldığı ayrıtların olasılıklarının toplamı 1 e eşittir. Bir yolun olasılığı, ayrıtların olasılıklarının çarpımı olarak tanımlıdır. v_0 dan başlayan bir yolun olasılığını, v_0 dan başlayan bir “rastgele yürüyüşün” belirli bir yolu izleyeceği olasılık olarak görebiliriz. Burada, u tepesini terk ederken hangi ayrıtı seçeceğimizi rastgele olarak seçeriz.

- b.** Eğer bir yol döndürürse bunun v_0 tepesinden başlayan ve s etiketine sahip en olası yol olacak şekilde, (a) şıkkındaki cevabınızı geliştiriniz. Algoritmanızın çalışma zamanını analiz ediniz.

15-8 Tabaka oyma ile görüntü sıkıştırma

Piksellerin bir $m \times n$ boyutlu $A[1..m, 1..n]$ matrisinden oluşan bir renkli resim verilsin. Burada her piksel, kırmızı (red), yeşil (green) ve mavi (blue) renkleri (RGB) ile belirtilir. Bu resmi biraz sıkıştırmak istediğimizi kabul edelim. Özellikle, tüm remi bir piksel daha küçültmek için m satırın her birinden bir piksel atmak isteriz. Bununla birlikte, görsel etektlere zarar vermemek için, iki komşu satırdan silinen piksellerin aynı veya komşu sütunlarda olmalarını isteriz; silinen pikseller, üst satırdan alt satıra kadar bir “tabaka” oluşturur, burada tabakadaki ardışık pikseller, dikey veya köşegensel olarak komşudurlar.

- a.** $n > 1$ kabul ederek, böyle mümkün tabakaların sayısı m e bağlı olarak en azından üsteldir.
- b.** Her $A[i, j]$ pikseli için, $A[i, j]$ pikselini sildiğimizde nasıl bozulduğunu göstermek için reel değerli bir $d[i, j]$ bozulma ölçüsü hesapladığımızı kabul edelim. Piksellerinin bozulma ölçülerinin toplamı olarak bir tabakanın bozulma ölçüsünü tanımladığımızı kabul edelim.

En düşük bozulma ölçüsüne sahip bir tabaka bulmak için bir algoritma öneriniz. Algoritmanız ne kadar verimlidir?

15-9 Bir katarın parçalanması

Belirli bir katar-işleme dili, bir programcıya bir katarı iki parçaya bölme imkanı verir. Bu işlem katarı kopyaladığı için, n karakterli bir katarın iki parçaya bölünmesi n zaman maliyetlidir. Programcının, bir katarı daha fazla parçaya bölmek istediğini kabul edelim. Parçalanmaların yapıldığı sıra, kullanılan toplam zaman miktarını etkileyebilir. Örneğin, programcı, 20 karakterli bir katarı 2, 8 ve 10 karakterlerinden sonra parçalamak istesin (karakterler soldan sağa artan şekilde ve 1 den başlayarak numaralanmış olsun). Eğer programcı, parçalanmaları soldan sağa sırada olacak şekilde programlarsa, ilk parçalanma 20 zaman birim maliyetli, ikinci parçalanma 18 zaman birim maliyetli (3 ten 20 ye kadarki karakterlerden 8. karakterde katarı parçalamak için) ve üçüncü parçalanma 12 zaman birim maliyetlidir. Toplamda 50 zaman birim maliyet olur. Bununla birlikte, eğer programcı, parçalanmaları sağdan sola sırada olacak şekilde programlarsa, ilk parçalanma 20 zaman birim maliyetli, ikinci parçalanma 10 zaman birim maliyetli ve üçüncü parçalanma 8 zaman birim maliyetlidir. Toplamda 38 zaman birim maliyet olur. Başka bir düzende, programcı, ilk kesimi 8 de (20 maliyetli), sonra sol parçayı 2 de (8 maliyetli) ve son olarak sağ parçayı 10 da (12 maliyetli) keser ve toplam maliyet 40 olur.

Kesim yapılacak karakterlerin numaraları verildiğinde, bu kesimlerin serisi için bir en az maliyetli yolu belirleyecek bir algoritma veriniz. Daha formal olarak, n karakterli bir S katarı ve kesim noktalarını içeren $L[1..m]$ dizisi verildiğinde, kesimlerin bir dizisi için en düşük maliyeti hesaplayın.

15.10 Bir yatırım stratejisinin planlanması

Algoritma bilginiz, 10000\$ işe başlama ikramiyesi ile Acme Bilgisayar Şirketinde heyecan verici bir işi elde etmenize yardım etsin. 10 yılın sonunda maksimize etmek amacıyla bu para ile yatırım yapmaya karar verin. Yatırımlarınızı yönetmesi için, Amalgamated Yatırım Şirketini kullanmaya karar verin. Şirket size, 1 den n e kadar numaralanmış n tane yatırım önerir. Her j yılında, i yatırımının r_{ij} oranında getirisi olsun. Başka bir deyişle, j yılında i yatırımına d dolar yatırırsanız, j yılın sonunda dr_{ij} dolar kazanırsınız. Geri dönüş oranları garantidir yani, her yatırım için önümüzdeki 10 yılın geri dönüş oranları verilmiştir. Yatırım kararlarınızı her yıl sadece bir defa yapabilirsiniz. Her yılın sonunda, önceki yıldan elde edilen parayı aynı yatırımlarda bırakabilir veya parayı, mevcut yatırımlar arasında kaydırarak ya da yeni bir yatırıma taşıyarak, diğer yatırımlara aktarabilirsiniz. $f_2 > f_1$ olmak üzere, eğer paranızı yer değiştirirseniz f_2 dolar ücret öderken, eğer paranızı ardışık yıllar arasında yer değiştirmezseniz, f_1 dolar ücret ödersiniz.

- a. Kurulan problem, her yıl çoklu yatırımlarda paranızı yatırım yapmanıza imkan verir. Her yıl, tüm paranızı tek bir yatırıma koyduğunuz bir optimal yatırım stratejisinin olduğunu kanıtlayınız. (bir optimal yatırım stratejisinin 10 yıl sonra para miktarını maksimize ettiğini ve minimizasyon riski gibi diğer amaçlarla ilgili olmadığını hatırlayınız.)
- b. Optimal yatırım stratejinizin planlanması probleminin optimal altyapıyı sağladığını kanıtlayınız.
- c. Optimal yatırım stratejinizi planlayan bir algoritma öneriniz. Algoritmanızın çalışma zamanı nedir?
- d. Amalgamated Yatırım Şirketinin, herhangi bir yatırımda herhangi bir zamanda 15000\$ dan fazla paraya sahip olamama gibi bir kısıtlama dayattığını kabul edin. Artık, 10 yılın sonunda gelirini maksimize etme probleminin optimal altyapıyı sağladığını gösteriniz.

15-11 Envanter planlama

Rinky Dink Firması, buz pistlerini kaplayan makinalar üretmektedir. Böyle ürünlerin talebi aydan aya değişiklik gösterir ve bunun için firma değişen fakat tahmin edilebilir üretim planı için bir strateji geliştirme ihtiyacı duymaktadır. Firma, önümüzdeki n ay için bir plan tasarlamayı arzu etmektedir. Her bir i ayı için, firma d_i talebini, yani satılacak makine sayısını, bilir. $D = \sum_{i=1}^n d_i$, n ayın toplam talebi olsun. Firma, ayda m makine üretecek iş gücünü sağlamak için tam zamanlı bir personel çalıştırmaktadır. Eğer firma, verilen bir ayda m makinadan fazlasını üretmeye geresinin duyarsa, makine başına c dolara çalışacak maliyette ilave kısmi zamanlı iş gücü kiralayabilir. Ayrıca, eğer bir ay sonunda, firmada satılmamış makina bulunduruyorsa, envanter maliyetini ödemelidir. $1 \leq j \leq D$ için $h(j) \geq 0$ ve $1 \leq j \leq D-1$ için $h(j) \leq h(j+1)$ olmak üzere, $j = 1, 2, \dots, D$ için j tane makine bulundurma maliyeti $h(j)$ fonksiyonu olarak verilmiştir.

Firma için, tüm talebi karşılarken maliyetleri minimize edecek bir planı hesaplayacak bir algoritma öneriniz. Çalışma zamanı, n ve D ye bağlı olarak bir polinom olmalıdır.

15-12 Serbest beysbol oyuncularını ile sözleşme imzalama

Bir birinci lig beysbol takımı için genel müdür olduğunuzu kabul edin. Sezon dışında, takımınız için, bazı serbest oyuncularla sözleşme imzalamaya gerek olsun. Takım sahibi size, serbest oyunculara harcamak üzere X \$ bütçe ayırsın. Tamamına X \$ dan daha az para harcayabilirsiniz, fakat, X \$ dan daha fazla harcarsanız takım sahibi sinirlenecektir.

N farklı pozisyon düşünmekteyiz ve her pozisyon için oynayabilecek P serbest oyuncu olsun.⁸ Herhangi bir pozisyonda çok fazla oyuncuyla listenizi taşırmak istemediğiniz için, her pozisyonda en çok bir oyuncu ile sözleşme imzalayabilirsiniz. (Eğer özel bir pozisyonda herhangi bir oyuncu ile sözleşme yapmak istemezseniz, bu pozisyondaki daha önce sahip olduğunuz oyuncularla plan yaparsınız.)

Nasıl değerli bir oyuncu olacağını belirlemek için, “VORP” veya “değiştirilen oyuncunun değeri” (value over replacement player) olarak bilinen bir sabermetrik istatistiği⁹ kullanmaya karar verirsiniz. Yüksek VORP lu bir oyuncu, düşük VORP lu olandan daha değerlidir. Yüksek VORP lu bir oyuncu, düşük VORP lu bir oyuncudan daha pahalı olmak zorunda değildir, çünkü bir oyuncunun değerinin dışındaki başka faktörler, maliyetinin ne kadar olacağını belirler.

Her uygun serbest oyuncu için, üç bilgiye sahipsiniz:

- oyuncunun pozisyonu,
- oyuncu ile sözleşme yapılacak para miktarı ve
- oyuncunun VORP u.

X \$ dan daha fazla harcamamak için, sözleşme yapacağınız oyuncunun toplam VORP unu maksimize edecek bir algoritma tasarlayınız. her oyuncunun 100000\$ ın katlarında sözleşme imzaladığını kabul edebilirsiniz. Algoritmanız, sözleşme yapacağınız oyuncuların toplam VORP unu, harcadığınız toplam para miktarını ve sözleşme yapacağınız oyuncuların listesini çıktı olarak vermelidir. Algoritmanızın çalışma zamanını ve bellek gereksinimini analiz ediniz.

Bölüm notları

R. Bellman, dinamik programlamanın sistematik araştırmasına 1955 te başladı. Burada ve doğrusal programlamada “programlama” kelimesi, bir tabu çözüm yöntemi kullanmayı ifade eder. Dinamik programlamanın elemanlarını kapsayan optimizasyon tekniklerinin daha önceden bilinmesine rağmen, Bellman keskin matematiksel kurallı bir alan oluşturdu [37].

⁸Bir beysbol takımında 9 pozisyon olmasına rağmen, bazı genel direktörler pozisyonlar için özel yöntemlere sahip oldukları için N 9 a eşit olmak zorunda değildir. Örneğin, bir genel direktör, atıcılardan, uzun atıcılardan (birkaç defa atış yapabilen atıcı) ve kısa atıcılardan (en çok bir defa atış yapabilen atıcı) başlayarak, sağ elli ve sol elli atıcıları ayrı pozisyonlarda düşünebilir,

⁹*Sabermetrik*, beysbol kayıtlarının istatistiksel analiz uygulamasıdır. Bireysel oyuncuların oransal değerlerini kıyaslamak için birkaç yol sağlar.

Galil ve Park [125], tablo boyutuna ve her kaydın bağlı olduğu diğer tablo kayıtlarının sayısına göre dinamik programlama algoritmalarını sınıflandırdı. Eğer, tablo boyutu $O(n')$ ve her kayıt $O(n^e)$ diğer kayıta bağlı ise bu dinamik programlama algoritmasını tD/eD olarak isimlendirdiler. Örneğin, Kısım 15.2'deki matris zinciri çarpım algoritması $2D/1D$ dir ve Kısım 15.4'teki en uzun ortak alt dizi algoritması $2D/0D$ dir.

Hu ve Shing [182, 183], matris zinciri çarpımı problemi için $O(n \lg n)$ zamanlı bir algoritma vermiştir.

En uzun ortak alt dizi problemi için $O(mn)$ zamanlı algoritma, geleneksel bir algoritma olarak görünür. Knuth [70], LCS problemi için alt kuadratik algoritmaların olup olmadığını sorusunu sormaktadır. Masek ve Paterson [244], bu soruyu $O(mn/\lg n)$ zamanda çalışan bir algoritma vererek cevaplamıştır, burada $n \leq m$ dir ve diziler sınırlı boyutlu bir kümeden alınmıştır. Bir girdi dizisinde birden fazla eleman olmadığı özel durum için, Szymanski [326], problemin $O((m + n) \lg(m + n))$ zamanda nasıl çözüleceğini gösterir. Bu sonuçların birçoğu, katar mesafe düzenleme hesabı problemine genişletilebilir (Problem 15-5).

Gilbert ve Moore [133] nin değişken uzunluklu ikili kodlama üzerine eski bir makalesi, tüm p_i olasılıklarının 0 olduğu durumlar için optimal ikili arama ağaçları oluşturan uygulamalar içermektedir; bu makale $O(n^3)$ zamanlı bir algoritma içermektedir. Aho, Hopcroft ve Ullman [5], Kısım 15.5'te bir algoritma önermiştir. Alıştırma 15.5-4, Knuth [212] dendir. Hu ve Tucker [184], $O(n^2)$ zaman ve $O(n)$ bellek kullanan tüm p_i olasılıklarının 0 olduğu durum için bir algoritma tasarlamış; ardından, Knuth [211], bu algoritmayı $O(n \lg n)$ e indirgemıştır.

Problem 15-8, Avidan ve Shamir [27] dendir ve bu görüntü sıkıştırma tekniğini gösteren muhteşem bir videoyu webde paylaşmışlardır.

Optimizasyon problemleri için oluşturulan algoritmalar genellikle, her adımda bir seçim kümesi kullanarak bir adımlar dizisini çalıştırır. Birçok optimizasyon problemi için, dinamik programlama kullanarak optimal seçenekleri belirleme işlemi aşırı şekilde işlem yükü getirir; daha basit, daha verimli algoritmalar geliştirilmelidir. Bir **açgözlü (greedy) algoritma** her zaman o anda optimal görünen seçimi yapar. Yani, bir global optimal çözüme götürecek ümidiyle bir yerel optimal seçimi yapar. Bu bölüm optimal çözümleri sağlayan açgözlü algoritmalar için optimizasyon problemlerini araştırır. Bu bölümü okumadan önce, dinamik programlama ile ilgili Bölüm 15 i, özellikle 15.3 Kısımını okumalısınız.

Açgözlü algoritmalar her zaman optimal çözümleri getirmezler, ama birçok problem için bunu yaparlar. İlk olarak Kısım 16.1 deki basit ama aşıkâr olmayan bir problem olan etkinlik-seçim problemi için bir açgözlü algoritmanın optimal çözümü verimli bir şekilde hesaplayabildiğini inceleyeceğiz. Öncelikle bir dinamik-programlama yaklaşımını göz önünde bulundurarak ve sonra bir optimal çözüme ulaşmak için her zaman açgözlü seçimler yapabildiğimizi göstererek açgözlü algoritmaya ulaşacağız. Kısım 16.2, açgözlü algoritmanın doğruluğunu ispatlamak için direkt bir yaklaşım olarak verilen açgözlü yaklaşımın temel unsurlarını yeniden inceler. Kısım 16.3 açgözlü tekniğinin önemli bir uygulaması olan veri-sıkıştırma(Huffman) kodlarını tasarlamayı tanıtır. Kısım 16.4 te her zaman optimal çözüm üreten bir açgözlü algoritma için “matroidler” olarak adlandırılan kombinatorik yapıların altında yatan bazı teoriler araştırılır. Son olarak, Kısım 16.5 zaman sınırı ve cezalara sahip birim-zamanlı görevlerin planlanması probleminin çözümünü için matroidleri kullanır.

Açgözlü yöntemi oldukça güçlüdür ve çeşitli sayıda problem için iyi çalışır. Sonraki bölümler açgözlü yöntemin uygulamaları olarak gördüğümüz bir çok algortimayı sunar: minimum-kapsayan-ağaç algoritması (Bölüm 23), tekil kaynaktan en kısa yollar için Dijkstra algoritması (Bölüm 24) ve Chvatal ın açgözlü küme örtüsü (set-covering) sezgiseli (Bölüm 35). Minimum-kapsayan-a-

ğaç algoritmaları açgözlü yönteminin klasik bir örneğidir. Bu bölüm ve Bölüm 23 birbirinden bağımsız okunabilmesine rağmen, birlikte okunması daha faydalı olacaktır.

16.1 Etkinlik-seçim problemi

İlk örneğimiz karşılıklı uyum içindeki etkinliklerin maksimum büyüklükteki bir kümesini seçme amacı ile ortak bir kaynağın kullanımı dışında gerekli olan birçok rekabetçi etkinliğin çizelgelenmesi problemidir. $S = \{a_1, a_2, \dots, a_n\}$ yi bir seferde yalnızca bir etkinliğin uygulanabildiği bir derslik için önerilen n **etkinliğin** kümesi olduğunu kabul edelim. Her bir a_i etkinliği, $0 \leq s_i < f_i < \infty$ iken bir s_i **başlangıç zamanına** ve bir f_i **bitiş zamanına** sahip olsun. Eğer seçilmiş ise, a_i etkinliği $[s_i, f_i)$ yarı açık zaman aralığında meydana gelir. Eğer $[s_i, f_i)$ ve $[s_j, f_j)$ aralıkları örtüşmüyorsa a_i ve a_j etkinlikleri **uygundur**. Yani, $s_i \geq f_j$ veya $s_j \geq f_i$ ise a_i ve a_j uygundur. **Etkinlik-seçim probleminde**, karşılıklı uygun etkinliklerin bir maksimum büyüklükteki alt kümesini seçmeyi isteriz. Etkinliklerin bitiş zamanlarına göre monoton artan olarak sıralandığını kabul edelim:

$$f_1 \leq f_2 \leq f_3 \leq \dots \leq f_{n-1} \leq f_n . \quad (16.1)$$

(Bu kabulün sağlandığı avantajı daha sonra göreceğiz.) Örneğin, aşağıdaki etkinliklerin S kümesi dikkate alınırsa:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

Bu örnek için, $\{a_3, a_9, a_{11}\}$ alt kümesi karşılıklı uygun etkinliklerden oluşur ve $\{a_1, a_4, a_8, a_{11}\}$ alt kümesi daha büyük olduğundan bu maksimum bir alt küme değildir. Aslında, $\{a_1, a_4, a_8, a_{11}\}$ karşılıklı uygun etkinliklerin bir en büyük alt kümesidir; diğer en büyük alt kümesi $\{a_2, a_4, a_9, a_{11}\}$ dir.

Bu problemi bir kaç adımda çözeceğiz. Bir optimal çözümde kullanılacak alt problemleri belirlerken birkaç seçeneğin göz önünde bulundurulduğu bir dinamik programlama çözümünü düşünerek başlarız. O zaman yalnızca bir seçimi—açgözlü seçimi—göz önünde bulundurmamız gerektiğini ve açgözlü seçimi elde ettiğimizde geriye sadece bir alt problem kalacağını göstereceğiz. Bu gözlemlere dayanarak, etkinlik çizelgeleme problemini çözmek için bir özyinelemeli(recursive) açgözlü algoritması geliştireceğiz. Özyinelemeli algoritmayı iteratif hale dönüştürerek bir açgözlü çözümün gelişme sürecini tamamlayacağız. Bir açgözlü algoritmayı geliştirirken bu bölümde gözden geçireceğimiz adımlar sıradan olandan biraz daha karmaşık olmasına rağmen, bu adımlar açgözlü algoritmalar ve dinamik programlama arasındaki ilişkiyi gösterir.

Etkinlik-seçim probleminin optimal altyapısı

Etkinlik-seçim probleminin optimal altyapısını göstermeyi kolayca kanıtlayabiliriz. a_i etkinliğinin bitmesinden sonra başlayan ve a_j etkinliğinin başlamasından önce biten etkinliklerin kümesini S_{ij} ile gösterelim. S_{ij} deki karşılıklı uygun etkinlerin bir maksimum kümesini bulmak istediğimizi kabul edelim ve dahası bazı a_k etkinlerini içeren bir maksimum küme olarak A_{ij} yi kabul edelim. Bir optimal çözümde a_k yı dahil ettiğimizde geriye iki altproblem kalır: S_{ik} kümesindeki karşılıklı uygun etkinliklerin bulunması (a_i etkinliğinin bitmesinden sonra başlayan ve a_k etkinliği başlamasından önce biten etkinlikler) ve S_{kj} kümesindeki karşılıklı uygun etkinliklerin bulunması (a_k etkinliğinin bitmesinden sonra başlayan ve a_j etkinliğinin başlamasından önce biten etkinlikler). $A_{ik} = A_{ij} \cap S_{ik}$ ve $A_{kj} = A_{ij} \cap S_{kj}$ olsun. Böylece A_{ik} , a_k nın başlamasından önce bitenlerin olduğu A_{ij} deki etkinlikleri içerir ve A_{kj} , a_k nın bitmesinden sonra başlayanların olduğu A_{ij} deki etkinlikleri içerir. Böylece $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$ ye sahibiz, o halde S_{ij} deki karşılıklı uygun etkinliklerinin A_{ij} kümesinin maksimum büyüklüğü $|A_{ij}| = |A_{ik}| + |A_{kj}| + 1$ etkinliklerinden oluşur.

Sıradan kes-yapıştır değişkeni, A_{ij} optimal çözümünün aynı zamanda S_{ik} ve S_{kj} altproblemlerinin optimal çözümünü içermek zorunda olduğunu gösterir. Eğer $|A'_{kj}| > |A_{kj}|$ olduğunda S_{kj} deki karşılıklı uygun etkinliklerin bir A'_{kj} kümesini bulabilirsek, S_{ij} altproblemi için bir çözümde A_{kj} yerine A'_{kj} yi kullanabiliriz. Böylece A_{ij} nin bir optimal çözüm olduğu kabulümüzle çelişen $|A_{ik}| + |A'_{kj}| + 1 > |A_{ik}| + |A_{kj}| + 1 = |A_{ij}|$ karşılıklı uygun etkinliklerinin bir kümesini oluşturmuş olacaktık. Bir simetrik argüman, S_{ik} daki etkinlikler için uygulanır.

Optimal altyapı tanımlamanın bu yolu dinamik programlama ile etkinlik-seçim problemini çözmemizi önerir. Eğer S_{ij} kümesi için bir optimum çözümün büyüklüğünü $c[i, j]$ olarak gösterirsek

$$c[i, j] = c[i, k] + c[k, j] + 1$$

yinelemesine sahip oluruz.

Elbette, S_{ij} kümesi için bir optimal çözümün a_k etkinliğini içerdiği bilinmiyor ise olacak şekilde hangisini seçeceğini bulmak için S_{ij} deki bütün etkinlikleri incelemek zorundayız.

$$c[i, j] = \begin{cases} 0, & S_{ij} = \emptyset \text{ ise,} \\ \max_{a_k \in S_{ij}} \{c[i, k] + c[k, j] + 1\}, & S_{ij} \neq \emptyset \text{ ise.} \end{cases} \quad (16.2)$$

Bu durumda, özyinelemeli bir algoritma geliştirebilir ve bunun hafızada tutulmasını veya aşağıdan-yukarı çalıştırabilir ve aynı anda tablodaki kayıtların doldurulmasını sağlayabilirdik. Ancak, bizim büyük bir avantaj için kullanabilecek olduğumuz etkinlik-seçim probleminin diğer önemli özelliğini göz ardı etmiştik.

Agözlü seimi yapmak

İlk olarak tüm alt problemleri çözmeye gerek kalmadan optimal çözümümüze eklemek için bir etkinlik seçebilir miyiz? Bu bizi, (16.2) de var olan tüm seçeneklerin hesaba katılmasından kurtarır. Aslında, etkinlik-seim problemi için sadece bir seimi dikkate almalıyız: açgözlü seimi.

Etkinlik-seim problemi için açgözlü seimi ile ne demek istiyoruz? Sezgi, mümkün olduđu kadar diğerk birok etkinlik için kullanılabilir kaynak bırakan bir etkinliğı seçmemizi önerir. Şimdi, etkinliklerin seimi bittiğinde onlardan bir tanesi, sonlandırmak için birinci olmak zorundadır. Sezgimiz, böylece, S deki en erken bitiş zamanlı etkinliğı seçmek için, mümkün olduđu kadar bunu takip eden diğerk etkinliklerin çoğı için kullanılabilir kaynak bırakabilecek olanı seçmemizi söyler. (S de birden fazla etkinlik en erken bitiş zamanına sahipse bunun gibi herhangi bir etkinliğı seçebiliriz.) Başka bir deyişle, etkinlikler bitiş zamanlarına göre monoton artan sıralandığından açgözlü seimi a_1 etkinliğidir. Sonlandırma için ilk etkinliğı seçmek, bu problem için bir açgözlü seimi yapmayı düşünmenin tek yolu değildir; Alıştırma 16.1-3 diğerk olasılıkları bulmamızı sormaktadır.

Eğer açgözlü seimini elde edersek, çözmek için geriye sadece bir altproblem kalır: a_1 in bitişinden sonra başlayan etkinliğı bulmak. Niin a_1 in başlamasından önce biten etkinliğı dikkate almıyoruz? $s_1 < f_1$ olduğuna sahibiz ve f_1 herhangi bir etkinliğın en erken bitiş zamanıdır bu nedenle hiçbir etkinlik s_1 e eşit veya daha küçük bir bitiş zamanına sahip olamaz. Böylece a_1 etkinliğı ile uyumlu bütün etkinlikler a_1 bittikten sonra başlamak zorundadır.

Bundan başka, etkinlik-seim probleminin optimal altyapıyı gösterdiğini şimdiden saptadık. a_k etkinliğinin bitmesinden sonra başlayan etkinliklerin kümesi $S_k = \{a_i \in S : s_i \geq f_k\}$ olsun. a_1 etkinliğinin açgözlü seimini elde edersek çözmek için altproblem olarak geriye sadece S_1 kalır.¹ Optimal altyapı, a_1 optimal çözümün içinde ise, asıl problem için bir optimal çözüm a_1 etkinliğı ve S_1 altproblemi için bir optimal çözümdeki bütün etkinliklerden oluşur.

Geriye bir tane büyük soru kalır: sezgimiz doğru mu? Açgözlü çözümü—birtirmek için seçtiğimiz ilk etkinlik—her zaman optimal çözümün bir parçası mıdır? Aşağıdaki teorem bunu gösterir.

¹Bazen S_k kümelerinden etkinliklerin kümesi gibi bahsetmektense altproblemeler olarak bahsederiz. S_k nın etkinliklerin bir kümesi ya da bu kümenin girdilerinin bir altproblemi olarak bahsedilip bahsedilmediğı metinde her zaman açık olacaktır.

Teorem 16.1

S_k yı boştan farklı herhangi bir altproblem kabul edelim ve a_m , S_k kümesinde en erken bitiş zamanına sahip bir etkinlik olsun. O halde a_m , S_k kümesinin karşılıklı uygun etkinliklerinin maksimum büyüklükteki alt kümesi tarafından içerilir.

Kanıt A_k , S_k kümesinde karşılıklı uygun etkinliklerin maksimum büyüklükteki bir alt kümesi ve a_j , A_k kümesinde en erken bitiş zamanına sahip bir etkinlik olsun. Eğer $a_j = a_m$ ise, a_m in S_k kümesinin karşılıklı uygun etkinliklerin maksimum büyüklükteki alt kümesinde olduğunu gösterdiğimizden bu koşul altında ispat doğrudur. $a_j \neq a_m$ ise, a_j yerine a_m koyarak $A'_k = A_k - \{a_j\} \cup \{a_m\}$ kümesi A_k olsun. A'_k kümesindeki etkinliklerin ayrıık olması, a_j nin bitiş için A_k daki ilk etkinlik olması ve $f_m \leq f_j$ olmasından A'_k kümesindeki etkinlikler ayrııktır. $|A'_k| = |A_k|$ olduğundan A_k nın, S_k kümesinin karşılıklı uygun etkinliklerinin maksimum büyüklükteki bir alt kümesi olduğunu ve a_m yi içerdiği sonucuna varırız. ■

Böylece, dinamik programlama ile etkinlik-seçim problemini çözmenin mümkün olmasına rağmen buna gerek olmadığını gördük. (Bunun yanı sıra, etkinlik-seçim probleminin çakışan altproblemlere sahip olup olmadığını henüz incelemedik.) Bunun yerine, ilk önce biten etkinliği tekrar tekrar seçebilir, sadece bu etkinlik ile uyumlu etkinlikleri tutabilir ve hiç etkinlik kalmayınca kadar tekrar edebiliriz. Dahası, her zaman en erken bitiş zamanlı etkinliği seçtiğimizden seçtiğimiz etkinliklerin bitiş zamanları kesin artan olmak zorundadır. Bitiş zamanlarının monoton artan dizilişinde her bir etkinliği genel olarak sadece bir kez düşünebiliriz.

Etkinlik-seçim problemini çözmek için bir algoritmanın bir tablo tabanlı dinamik programlama algoritması gibi aşağıdan-yukarıya çalışmasına gerek yoktur. Bunun yerine, bu algoritma, optimal çözüm içine katmak için bir etkinliği seçerek ve ardından önceden seçilmiş ve uygun olanlardan etkinlikleri seçme altproblemünü çözerek yukarıdan-aşağıya çalışabilir. Açgözlü algoritmalar genellikle bu yukarıdan-aşağı tasarıma sahiptir: bir seçim yapmadan önce altproblemlerin çözümünün aşağıdan-yukarıya tekniği yerine, bir seçim yapar ve sonra bir altproblem çözer.

Özyinelemeli açgözlü algoritma

Artık, dinamik programlama yaklaşımını nasıl atlattığımızı ve bunun yerine yukarıdan-aşağıya açgözlü algoritmayı nasıl kullandığımızı gördük. Etkinlik-seçim problemini çözmek için bir özyinelemeli yöntemini açıkça yazabiliriz. RECURSIVE-ACTIVITY-SELECTOR prosedürü, aktivitelerin başlangıç ve bitiş zamanları olarak s ve f ² dizilerini, S_k alt problemi tanımlayan k indi-

²Kabakod diziler gibi s ve f aldığından, alt indistense kare parantez ile indislenir.

sini ve asıl problemin boyutu olan n i alır. Bu, S_k daki karşılıklı uygun etkinliklerin bir maksimum büyüklükteki kümesini geri döndürür. n tane girdi etkinliğinin, denklem (16.1) e göre monoton artan bitiş zamanı ile önceden sıralanmış olduğunu kabul edelim. Eğer değilse, bağlantıları keyfi olarak kaldırarak onları $O(n \log n)$ zaman karmaşıklığında sıralayabiliriz. Başlamak için, $f_0 = 0$ olan hayali bir a_0 etkinliğini ekleriz, böylece S_0 altproblemi, S etkinliklerinin mevcut kümesi olur. Mevcut problemi çözen başlangıç çağrısı RECURSIVE-ACTIVITY-SELECTOR($s, f, 0, n$) dir.

RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)

```

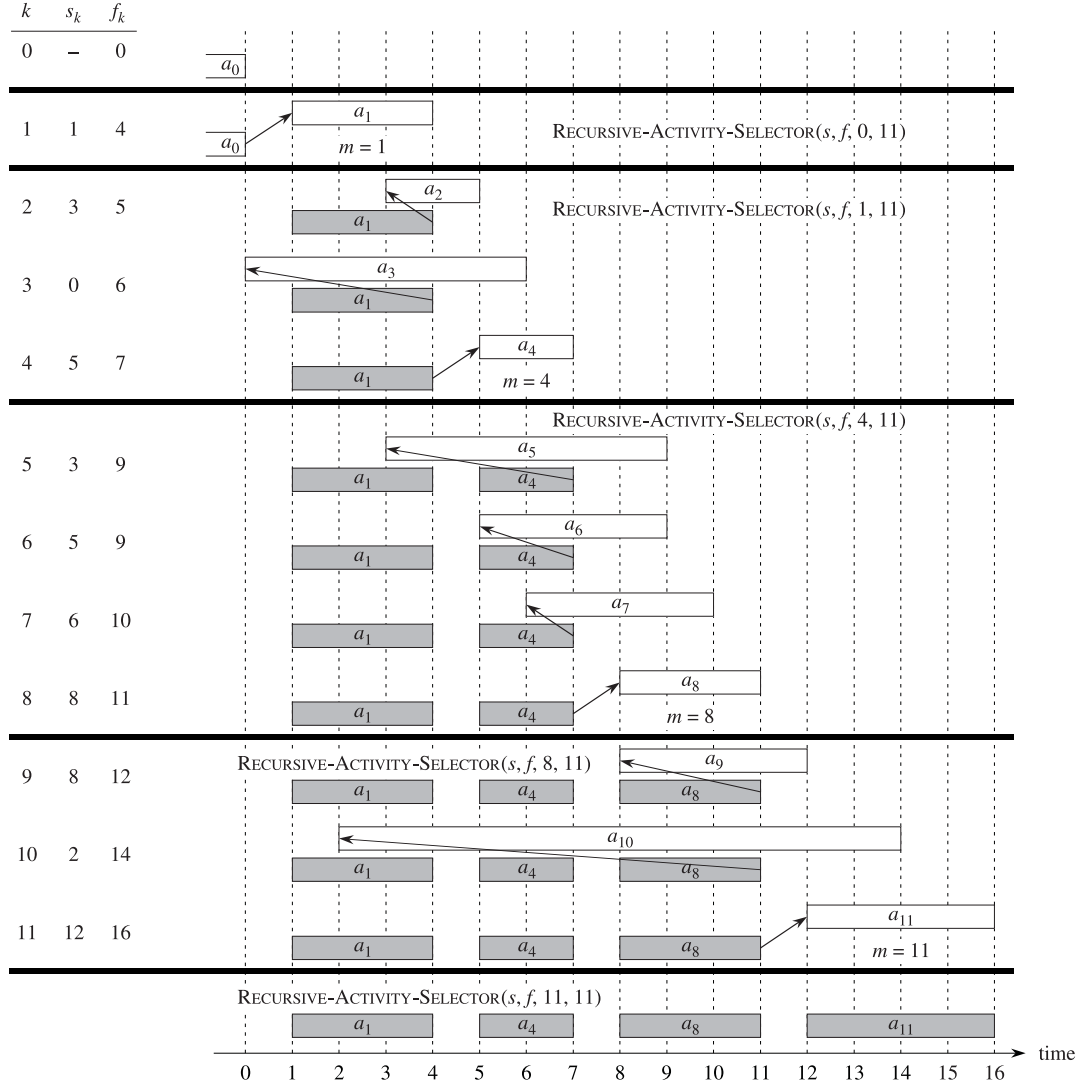
1   $m = k + 1$ 
2  while  $m \leq n$  ve  $s[m] < f[k]$       // bitirmek için  $S_k$  daki ilk aktiviteyi bul
3       $m = m + 1$ 
4  if  $m \leq n$ 
5      return  $\{a_m\} \cup \text{RECURSIVE-ACTIVITY-SELECTOR}(s, f, m, n)$ 
6  else return  $\emptyset$ 
```

Şekil 16.1 algoritmanın işleyişini gösterir. Verilen bir RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n) çağrısında, 2-3 satırlarındaki **while** döngüsü S_k kümesindeki ilk etkinliği bitirmek için arama yapar. Bu döngü $s_m \geq f_k$ ya sahip bir etkinlik gibi, a_k ye uygun ilk a_m etkinliğini buluncaya kadar $a_{k+1}, a_{k+2}, \dots, a_n$ leri inceler. Eğer döngü böyle bir etkinliği bulduğu için sonlanırsa, 5. satır, RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n) özyinelemeli çağrısı tarafından döndürülen S_m in maksimum büyüklükteki alt kümesi ile $\{a_m\}$ in birleşimini geri döndürür. Alternatif olarak, a_k ile uyumlu olarak bulunan dışında, S_k kümesindeki bütün etkinlikleri incelediğimizde $m > n$ için döngü sonlanabilir. Bu durumda, $S_k = \emptyset$ dır ve böylece prosedür 6. satırda, $\emptyset$ döndürür.

Etkinliklerin bitiş zamanlarına göre önceden sıralandığını kabul edelim, RECURSIVE-ACTIVITY-SELECTOR($s, f, 0, n$) çağrısının çalışma zamanı aşağıda görebildiğimiz gibi $\Theta(n)$ dir. Bütün özyinelemeli çağrılarda, her bir etkinlik, 2. satırın While döngü testinde, tam olarak bir kere incelenir. Özellikle, a_i etkinliği $k < i$ olduğunda yapılan enson çağrıda incelenir.

İteratif açgözlü algoritma

Özyinelemeli prosedürü iteratif şekle kolayca dönüştürebiliriz. RECURSIVE-ACTIVITY-SELECTOR prosedürü hemen hemen “kuyruk özyinelemesi”dir (Problem 7-4 bakınız): Bir birleşim işlemiyle birlikte kendisine özyinelemeli bir çağrı ile sonlanır. Bir kuyruk özyinelemesi prosedürünü iteratif forma dönüştürmek genellikle basit bir iştir; aslında bazı programlama dilleri için kimi derleyiciler bu görevi otomatik bir şekilde gerçekleştirir. Daha önce yazıldığı gibi RECURSIVE-ACTIVITY-SELECTOR, S_k altproblemleri için çalışır, burada S_k bitirmek için en son etkinliklerden oluşan altproblemlerdir.



Şekil 16.1 Daha önceden verilen 11 etkinlik üzerindeki RECURSIVE-ACTIVITY-SELECTOR un işleyişi. Her özyinelemeli çağrıda ele alınan etkinlikler yatay satırlar arasında görünür. Hayali a_0 etkinliği 0 zamanda biter ve RECURSIVE-ACTIVITY-SELECTOR($s, f, 0, 11$) ilk çağrısı a_1 etkinliğini seçer. Her bir özyinelemeli çağrıda önceden seçilmiş olan etkinlikler gölgelidir ve beyaz renkle gösterilen etkinlik dikkate alınır. Eğer bir etkinliğin başlangıç zamanı, son zamanda eklenen etkinliğin (aralarındaki ok solu gösterir) bitiş zamanından önce meydana geliyor ise reddedilir. Aksi halde (okların ucu doğrudan yukarıya ve sağa işaret eder) seçilir. Son özyinelemeli çağrı, RECURSIVE-ACTIVITY-SELECTOR($s, f, 11, 11$), $\emptyset$ döndürür. Seçilen etkinliklerin sonucunda oluşan küme $\{a_1, a_4, a_8, a_{11}\}$ dir.

GREEDY-ACTIVITY-SELECTOR yöntemi RECURSIVE-ACTIVITY-SELECTOR yönteminin bir iteratif versiyonudur. Yine girdi etkinlerinin bitiş zamanlarına göre monoton artan olarak sıralandığını kabul edelim. Bu prosedür seçilen etkinlikleri bir A kümesinde tutar ve algoritma tamamlandığında bu kümeyi geri döndürür.

GREEDY-ACTIVITY-SELECTOR(s, f)

```

1   $n = s.length$ 
2   $A = \{a_1\}$ 
3   $k = 1$ 
4  for  $m = 2$  to  $n$ 
5      if  $s[m] \geq f[k]$ 
6           $A = A \cup \{a_m\}$ 
7           $k = m$ 
8  return  $A$ 
```

Bu prosedür aşağıdaki gibi çalışır. k değişkeni, A ya en son eklenen özyinelemeli versiyondaki a_k etkinliğine işaret eder. Etkinlikleri monoton artan sırada bitiş zamanına göre dikkate aldığımızdan f_k her zaman A kümesindeki herhangi bir etkinlik için maksimum bitiş zamanıdır. Yani,

$$f_k = \max \{f_i : a_i \in A\} . \quad (16.3)$$

2-3 satırları a_1 etkinliğini seçer, sadece bu etkinliği içeren A kümesi ilklendirilir ve bu etkinliği indisleme için k ilklendirilir. 4-7 satırlarındaki **for** döngüsü, bitirmek için S_k daki en erken etkinliği bulur. Döngü, devirdeki her bir a_m etkinliğini dikkate alır ve a_m , önceden seçilmiş bütün etkinlikler ile uyumlu yani S_k daki bitime en yakın etkinlik ise, A ya eklenir. a_m etkinliğinin A da mevcut durumdaki her bir etkinlik ile uyumlu olup olmadığını görmek için, s_m başlangıç zamanının A ya en son eklenen f_k bitiş zamanlı etkinlikten daha önce olmadığını doğrulamak için (5. satır), (16.3) denkleminde bakmak yeterli olur. Eğer a_m etkinliği uyumlu ise, 6-7 satırları A ya a_m etkinliğini ekler ve k ya m değerini atar. GREEDY-ACTIVITY-SELECTOR(s, f) çağrısı tarafından geri döndürülen A kümesi, tam olarak RECURSIVE-ACTIVITY-SELECTOR($s, f, 0, n$) çağrısı tarafından geri döndürülen kümedir.

Özyinelemeli versiyon gibi, GREEDY-ACTIVITY-SELECTOR etkinliklerin ilk olarak bitiş zamanlarına göre sıralandığını farz ederek $\Theta(n)$ zamanda n etkinlikli bir kümeyi çizelgeler.

Alıştırmalar

16.1-1

(16.2) özyinelemesine dayalı etkinlik-seçim problemi için bir dinamik programlama algoritması yazınız. Algoritmanız yukarıdaki gibi tanımlı $c[i, j]$ değerlerini hesaplasın ve karşılıklı uygun etkinliklerin maksimum büyüklükte alt kümesini

de üretsın. Girdilerin (16.1) denklemindeki gibi sıralandığını kabul edelim. Çözümünüzün çalışma zamanı ile GREEDY-ACTIVITY-SELECTOR un çalışma zamanını kıyaslayınız.

16.1-2

Her zaman bitirmek için ilk etkinliği seçmek yerine, başlamak için önceden seçilmiş tüm etkinlikler ile uyumlu olan son etkinliği seçtiğimizi kabul edelim. Bu yaklaşımın nasıl bir açgözlü algoritma olduğunu tanımlayınız ve bir optimal çözüm sağladığını ispatlayınız.

16.1-3

Etkinlik-seçim problemi için, herhangi bir açgözlü yaklaşım karşılıklı uygun etkinliklerin maksimum boyutlu bir kümesini üretmez. Önceden seçilmiş etkinliklerle uyumlu olanlar arasından en az zamanlı etkinlik-seçim yaklaşımının çalışmadığını gösteren bir örnek veriniz. En az geriye kalan diğer etkinliklerin içinden uygun etkinliğin seçimi ve en erken başlangıç zamanlı geriye kalan uygun etkinliğin seçimi yaklaşımları için aynı işlemleri yapınız.

16.1-4

Herhangi bir amfide herhangi bir etkinliğin olabildiği, çok sayıda amfi arasında çizelgeleme için etkinliklerin bir kümesine sahip olduğumuzu kabul edelim. Mümkün olan en az sayıda amfiyi kullanarak bütün etkinlikleri çizelgelemek isteriz. Hangi etkinliğin hangi amfiyi kullandığını belirlemek için verimli bir açgözlü algoritma yazınız.

(Bu problem *aralık-çizgesi renklendirme problemi* olarak bilinir. Ayrıtlarının uyumsuz etkinlikleri bağladığı ve tepeleri verilen etkinlikler olan bir aralık çizgesi oluşturabiliriz. Birbirine komşu olmayan iki tepe aynı renkte olacak şekilde her tepayı boyamak için gereken en az renk sayısı, verilen bütün etkinlikleri çizelgelemek için gereken en az amfiyi bulmaya karşılık gelir.)

16.1-5

Etkinlik seçim problemine dair bir değişiklik düşünelim: Herhangi bir a_i etkinliği, başlangıç ve bitiş zamanına ilave olarak bir v_i değerine sahip olsun. Amaç çizelgelenen etkinliklerin sayısını maksimum yapmak yerine çizelgelenen etkinliklerin toplam değerini maksimum yapmaktan fazlası değil. Yani $\sum_{a_k \in A} v_k$ maksimum olacak şekilde uyumlu etkinliklerin bir A kümesini seçmek isteriz. Bu problem için bir polinom-zamanlı algoritma yazınız.

16.2 Açgözlü stratejinin elemanları

Bir açgözlü algoritma, seçimlerin bir dizisini oluşturarak bir problem için bir optimal çözüm elde eder. Her bir karar noktasında, algoritma o an için optimal görünen seçimi yapar. Bu sezgisel strateji her zaman bir optimal çözüm üretmez ancak etkinlik-seçim probleminde gördüğümüz gibi bazen üretebilir. Bu bölüm, açgözlü yöntemlerin genel özelliklerinin bazılarını ele almaktadır.

Bir açgözlü algoritma geliştirmek için, Kısım 16.1 de izlediğimiz süreç normalden biraz daha fazladır. Aşağıdaki adımlar üzerinde duracağız:

1. Problemin optimal altyapısını tanımla.
2. Bir özyinelemeli çözüm geliştir. (Etkinlik-seçim problemi için (16.2) özyinelemesini formüle ettik, ancak bu özyinelemeye dayalı bir özyinelemeli algoritma geliştirmeyi atladık.)
3. Açgözlü seçimi elde edersek, geriye sadece bir altproblem kaldığını göster.
4. Bunun her zaman açgözlü seçimi elde etmek için güvenli olduğunu ispatla. (3 ve 4 adımları herhangi bir sırada olabilir.)
5. Açgözlü yöntemini sağlayan özyinelemeli bir algoritma geliştir.
6. Özyinelemeli algoritmayı bir iteratif algoritmaya dönüştür.

Bu adımlar incelendiğinde, bir açgözlü algoritmanın dinamik programlamayı destekleyici unsurlarını son derece detaylı bir biçimde gördük. Örneğin, etkinlik-seçim probleminde, ilk olarak hem i hemde j nin değişken olduğu S_{ij} altproblemlerini tanımladık. Her zaman açgözlü seçimini elde edersek S_k formunda olan alt problemlere sınırlayabildiğimizi bulduk.

Alternatif olarak, akılda kalan bir açgözlü seçimi ile optimal altyapımızı biçimlendirebildik, böylece bu seçim geriye çözmek için sadece bir altproblem bırakır. Etkinlik-seçim probleminde, ikinci altindisi atarak ve S_k formundaki alt problemleri tanımlayarak başlayabilirdik. Sonra, uyumlu etkinliklerin geride kalan S_m kümesi için bir optimal çözüm ile birleştirilmiş bir açgözlü seçiminin (a_m , S_k daki bitirmek için ilk etkinliktir) S_k için bir optimal çözüm olduğunu ispatlayabilirdik. Daha genel olarak, aşağıdaki adımların sırasına göre açgözlü algoritmayı tasarlarız:

1. Optimizasyon problemini, karar vermemizi gerektiren ve çözülmesi gereken bir altproblem içeren bir probleme dönüştür
2. Açgözlü seçimi yapan orijinal problem için her zaman bir optimal çözümün olduğunu böylece açgözlü seçimin her zaman güvenilir olduğunu kanıtlayınız.

3. Optimal altyapıyı gösteriniz: Ağgözlü seçim yapıldığında; yaptığımız ağgözlü seçim ile bu alt problemin bir optimal çözümünü birleştirirsek, orijinal problem için bir optimal çözüme ulaştığımız bir altproblem geriye kalır.

Bu bölümün sonraki kısımlarında daha açık olan bu işlemi kullanacağız. Buna rağmen, her ağgözlü algoritma altında neredeyse her zaman daha kullanışsız bir dinamik programlama çözümü vardır.

Bir ağgözlü algoritmanın özel bir optimizasyon problemini çözüp çözemeyeceğini nasıl söyleyebiliriz? Tüm yöntemler her zaman çalışmaz, ancak ağgözlü seçim özelliği ve optimal altyapı iki anahtar etmendir. Eğer problemin bu özelliklere sahip olduğunu gösterebilirsek, bu problem için bir ağgözlü algoritma geliştirme için doğru yoldayız demektir.

Ağgözlü seçim özelliği

İlk anahtar unsur ***ağgözlü-seçim özelliği(greedy-choice property)***dir: lokal optimal (ağgözlü) seçimler yaparak, bir global optimal çözümü oluşturabiliriz. Başka bir deyişle; hangi seçimi yapacağımızı düşündüğümüz zaman, altproblemlerden gelen sonuçları dikkate almadan mevcut problemde optimal görünen seçimi yaparız.

Ağgözlü algoritmalarının dinamik programlamadan farkı budur. Dinamik programlamada her adımda bir seçim yaparız, ama bu seçim genellikle alt problemler için çözümlere bağlıdır. Sonuç olarak, biz genellikle dinamik programlama problemlerini en küçük altproblemlerinden en büyük altproblemlere ilerleyen aşağıdan-yukarı yönteminde çözeriz. (Alternatif olarak, onları yukarıdan-aşağıda çözebiliriz fakat ön belleğe alma işlemi yaparız. Tabi ki, kod yukarıdan aşağıya çalışsa bile biz yine de bir seçim yapmadan önce altproblemi çözmeliyiz.) Bir ağgözlü algoritmada, o anda optimal gibi görünen herhangi bir seçimi yaparız ve sonra kalan alt problemi çözeriz. Bir ağgözlü algoritma ile yapılan seçim şimdiye kadar yapılan seçimlere bağlı olabilir, ancak herhangi bir gelecek seçime veya alt problemler için çözümlere bağlı değildir. Böylece birinci seçimi yapmadan önce altproblemleri çözen dinamik programlamanın tersine bir ağgözlü algoritma herhangi bir altproblem çözmeden önce ilk seçimini yapar. Bir ağgözlü strateji, bir seçimden sonra başka bir ağgözlü seçim yaparak ve her verilen problem örneğini daha küçüğüne indirgeyerek genellikle yukarıdan-aşağıya tarzda hareket ederken, bir dinamik programlama algoritması aşağıdan yukarıya devam eder.

Tabi ki her adımdaki bir ağgözlü seçiminin bir global optimal çözüm verdiğini kanıtlamalıyız. Genellikle Teorem 16.1 deki durumdaki gibi kanıt bazı altproblemler için bir global optimal çözümü inceler. Daha sonra herhangi bir diğer seçim için ağgözlü seçiminin yerine geçecek çözümü nasıl değiştirdiğimizi gösterir, benzer fakat daha küçük altproblem elde edilir.

Biz genellikle ağgözlü seçimi, seçimlerin daha geniş bir kümesini dikkate almak zorunda olduğumuz zamandan daha etkili bir şekilde elde edebiliriz. Ör-

neğin, etkinlik-seçim probleminde etkinliklerin bitiş zamanlarına göre monoton artan sırayla dizildiğini kabul ettik, bunun için her bir etkinliği bir kereye mahsus incelememiz gerekti. Uygun bir veri yapısı kullanarak (çoğu kez bir öncelikli kuyruk) veya girdiye ön işlem gerçekleştirerek sıkça hızlı bir şekilde açgözlü seçimler yapabiliriz, böylece etkili bir algoritma oluşturulur.

Optimal altyapı

Eğer problem için bir optimal çözüm, altproblemleri için optimal çözümleri içeriyorsa bir problem *optimal altyapı(optimal substructure)*yı sağlar. Bu özellik açgözlü algoritma kadar dinamik programlamanın uygulanabilirliğinin değerlendirilmesinin temel bir unsurudur. Optimal altyapının bir örneği olarak, S_{ij} altproblemi için bir optimal çözüm, bir a_k etkinliğini içerirse S_{ik} ve S_{kj} altproblemleri içinde optimal çözümleri içermesi gerektiğini Kısım 16.1 de nasıl gösterdiğimizi hatırlayın. Bu optimal altyapı verildiğinde, a_k olarak hangi etkinliği kullandığımızı bilirse S_{ik} ve S_{kj} altproblemleri için optimal çözümlerdeki bütün etkinlikler ile birlikte a_k yı seçerek S_{ij} için bir optimal çözümün inşa edilebildiğimizi iddia ettik. Optimal altyapının bu yorumuna dayanarak, bir optimal çözümün değerinin tanımlandığı (16.2) özyinelenmesini tasarlayabilirdik.

Genellikle, açgözlü algoritmalara uygulandığında optimal altyapıyla ilgili olarak daha açık bir yaklaşım kullanırız. Yukarıda belirtildiği gibi, orijinal problemde açgözlü seçim yapılarak bir altprobleme ulaştığımızı varsayma lüksüne sahibiz. Hepimizin önceden yapılmış açgözlü seçim ile oluşturulmuş alt problem için bir optimal çözümün orijinal problem için bir optimal çözüm verdiğini iddia etmemiz gerekir. Bu tasarı tam olarak, her adımda açgözlü seçimin elde edilmesinin bir optimal çözüm ürettiğini kanıtlamak için altproblemler üzerinde tümevarımı kullanır.

Açgözlüye karşı dinamik programlama

Hem açgözlü hem de dinamik programlama yöntemleri optimal altyapıdan faydalandığı için, bir açgözlü çözüm yeterli olduğunda bir problem için bir dinamik programlama çözümü üretmek cazip olabilir yada aksine, aslında bir dinamik programlama çözümü gerekli olduğunda bir açgözlü çözümün çalıştığını yanlışlıkla düşünebilirsiniz. İki teknik arasındaki ince farkı örneklemek için, iki değişkenli klasik bir optimizasyon problemini inceleyelim.

0-1 sırt çantası problemi(0-1 knapsack problem) şöyledir. Bir dükkanı soy-maya çalışan hırsız n eşya arar. v_i ve w_i tam sayı olmak üzere i . ürün v_i dolar değerinde ve w_i pound ağırlığındadır. Hırsız mümkün olduğu kadar değerli bir yük almak ister fakat bazı W tam sayısı için çantasında en fazla W pound ağırlık taşıyabilir. Hırsız hangi parçaları almalıdır? (Hırsız her bir parçayı ya yanına al-

mak yada bırakmak zorundadır; kesirli miktarda bir parça yada bir kereden fazla bir parça alamadığından buna 0-1 sırt çantası problemi deriz.)

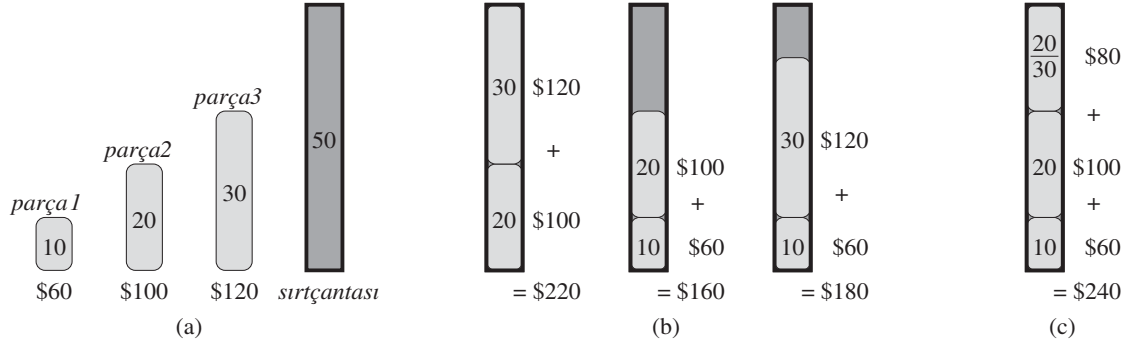
Kesirli sırt çantası problemi(fractional knapsack problem)inde kurulumu aynıdır, ancak hırsız her bir parça için (0-1) ikili seçimi yapmaktansa kesirli parçalar alabilir. (0-1) sırt çantası probleminde bir parçayı altın külçesi olarak ve kesirli sırt çantası problemindeki bir parçayı ise altın tozu olarak düşünebilirsiniz.

Her iki sırt çantası problemi optimal altyapı özelliğini sağlar. 0-1 problemi için ağırlık en fazla W pound olacak şekilde en değerli yüklemeyi dikkate alın. Eğer bu yükten j parçasını çıkarırsak kalan yük, hırsızın j haricindeki $n - 1$ asıl parçadan alabildiği en fazla $W - w_j$ ağırlığındaki en değerli yük olmalıdır. Kıyaslanabilir Kesirli problem için, optimal yükten bir j parçasının bir w miktarını çıkarırsak, kalan yük, hırsızın $n - 1$ orijinal parçaya ilave j elemanın $w_j - w$ pound miktarından seçebildiği en fazla $W - w$ ağırlığında olan en değerli yük olmalıdır.

Problemler benzer olmasına rağmen kesirli sırt çantası problemini açgözlü yöntemi ile çözebiliriz ama 0-1 problemi bu yöntem ile çözemeyiz. Kesirli problem çözmek için ilk olarak her parça için pound başına v_i/w_i değerini hesaplarız. Açgözlü yöntemine uyarak, hırsız pound başına en büyük değerli olan parçanın mümkün olduğu kadar en çok alınmasıyla başlar. Eğer mevcut parça biter ve hırsız daha fazlasını taşıyabilirse, pound başına sonraki en değerli parçadan mümkün olduğu kadar çok alır ve benzeri işlem ağırlık sınırı W olana kadar devam eder. Böylece, pound başına düşen değeri ile parçaları sıralayarak açgözlü Algoritma $O(n \lg n)$ zamanda çalışır. Kesirli sırt çantası probleminin açgözlü seçme özelliğine sahip olmasının kanıtı Alıştırma 16.2-1 de istenmektedir.

Bu açgözlü tekniğinin 0-1 sırt çantası problemi için çalışmadığını görmek için Şekil 16.2(a) da gösterilen örnek problemi düşünün. Bu örnekte üç parça ve 50 pound taşıyabilen bir sırt çantası vardır. Birinci parça 10 pound ağırlığında ve değeri 60 dolar, İkinci parça 20 pound ağırlığında ve değeri 100 dolar, üçüncü parça 30 pound ağırlığında ve 120 dolar değerindedir. Böylece birinci parçanın pound başına değeri ikinci parça (pound başına 5 dolar) veya üçüncü parça(pound başına 4 dolar)den daha büyüktür, 6 dolardır. Açgözlü yöntemi, bu nedenle birinci parçayı alacaktır. Şekil 16.2(b) deki durum analizinden görebildiğiniz gibi şu var ki, optimal çözüm birinci parçayı geride bırakarak 2. ve 3. parçaları alır. Birinci parçayı alan iki olası çözümün her ikisi de optimalin altındadır.

Karşılaştırılabilir kesir problemi için, ilk olarak birinci parçayı alan açgözlü yöntemi Şekil 16.2(c) de gösterildiği gibi bir optimal çözüm sağlar. Birinci parçayı almak 0-1 probleminde çalışmaz çünkü hırsız çantanın kapasitesini dolduramaz ve boş alan yükünün pound başına verimlilik değerini düşürür. 0-1 probleminde sırt çantasında bir parçanın olup olmadığını düşündüğümüzde; seçim yapmadan önce, bu parçayı içeren altproblemin çözümü ile parçayı içermeyen alt problemin çözümünü kıyaslamalıyız. Bu yolla formüle edilen problem birçok ça-



Şekil 16.2 Açgözlü algoritmanın 0-1 sırt çantası problemi için çalışmadığını gösteren bir örnek. (a) Hırsız ağırlıkları 50 poundu geçmeyecek şekilde üç parçanın bir alt kümesini seçmelidir. (b) Optimal alt küme 2. ve 3. parçaları içerir. 1. parça pound başına en büyük değere sahip olmasına rağmen 1. parçaya sahip herhangi bir çözüm optimalin altındadır. (c) Kesirli sırt çantası problemi için, pound başına en büyük değere sahip parçaları sırasıyla almak bir optimal çözümü oluşturur.

kışan altproblemlere neden olur—dinamik programlamanın bir ayırıcı özelliği ve aslında, Alıştırma 16.2-2 nin göstermemizi istediği gibi, 0-1 problemini çözmek için dinamik programlamayı kullanabiliriz.

Alıştırmalar

16.2-1

Kesirli sırt çantası probleminin açgözlü seçim özelliğine sahip olduğunu kanıtlayınız.

16.2-2

n parçaların sayısı ve W hırsızın sırt çantasına koyabileceği parçaların maksimum ağırlığı olmak üzere $O(n \cdot W)$ çalışma zamanlı 0-1 sırt çantası problemi için bir dinamik programlama çözümü yazınız.

16.2-3

0-1 sırt çantası probleminde ağırlıkların artan sıralandığında parçaların sırası ile azalan değeri ile sıralandığındaki sırasının aynı olduğunu kabul edelim. Sırt çantası probleminin bu türünün bir optimal çözümünü bulmak için verimli bir algoritma öneriniz ve algoritmanızın doğruluğunu tartışınız.

16.2-4

Profesör Gekko her zaman kuzey Dakota boyunca tekerlikli paten yapma hayalini kurardı. Profesör Gekko, Minnesota ile doğu sınırında olan Grand Forks dan, Montana ile batı sınırına komşu Williston'a kadar olan U.S. 2 karayolu üzerin-

deki eyaletleri geçmeyi planlamaktadır. Profesör iki litre su taşıyabilir ve suyu bitmeden önce m mil kayabilir. (Kuzey Dakota oldukça düz olduğundan, profesör yokuş yukarı kısımlarda, düz ya da yokuş aşağı kısımlara oranla daha fazla su tüketilmesi hakkında endişelenmek zorunda değil.) Profesör, Grand Forks tan dolu iki litre su ile başlayacaktır. Onun resmi Kuzey Dakota eyaleti haritası U.S. 2 boyunca yeniden su doldurabileceği bütün yerleri ve bu yerler arasındaki uzunlukları göstermektedir.

Profesörün amacı eyaletleri geçerken güzergâhı boyunca su duraklarının sayısını en az yapmaktır. Hangi su duraklarında duracağını belirleyebilen verimli bir yöntem yazınız. Stratejinizin optimal çözüm oluşturduğunu ispatlayınız ve çalışma zamanını yazınız.

16.2-5

Gerçel sayı doğru üzerindeki noktaların bir $\{x_1, x_2, \dots, x_n\}$ kümesi verilesin. Bu noktaları içeren kapalı aralıkların birim uzunluğunun en küçük kümesini belirleyen verimli bir algoritma tanımlayınız. Algoritmanın doğruluğunu tartışınız.

16.2-6 *

$O(n)$ zamanda kesirli sırt çantası probleminin nasıl çözüleceğini gösteriniz.

16.2-7

Her biri n tane pozitif tam sayı içeren A ve B kümelerinin verildiğini kabul edelim. Ancak istediğiniz gibi her kümeyi yeniden sıralamayı seçebiliyorsunuz. Yeniden sıraladıktan sonra, A kümesinin i . elemanı a_i ve B kümesinin i . elemanı b_i olsun. Sonra $\prod_{i=1}^n a_i^{b_i}$ nin bir sonucunu alırsınız. Bu işleminizin sonucunu maksimum yapacak olan bir algoritma yazınız. Algoritmanızın, sonucu maksimum yaptığını kanıtlayınız ve çalışma zamanını belirtiniz.

16.3 Huffman kodları

Huffman kodları verileri çok etkili bir şekilde sıkıştırır: sıkıştırılacak olan verinin karakterine bağlı olarak %20 ile %90 arasında kazanç normaldir. Verinin, bir karakter dizisi olduğunu dikkate alırız. Huffman'ın açgözlü algoritması, ikili karakter katarı (string) olarak her bir karakteri temsil etmenin bir optimal yolunu geliştirmek için her bir karakterin hangi sıklıkla oluştuğunu veren bir tablo kullanır.

Sıkıştırılmış bir halde depolamak istediğimiz 100.000 karakterli bir veri dosyasına sahip olduğumuzu kabul edelim. Dosyadaki karakterlerin, Şekil 16.3 ile verilen tekrarlanma sıklıkları ile meydana geldiğini gözlemliyoruz. Yani, sadece 6 farklı karakter görünür, ve a karakteri 45.000 defa meydana gelir.

Bir bilgi dosyasını nasıl temsil etmemiz gerektiği ile ilgili birçok seçeneğe sahibiz. Burada, herbir karakteri, **kod sözcüğü (codeword)** olarak adlandırılan

	a	b	c	d	e	f
Frekans (x 1000)	45	13	12	16	9	5
Sabit-uzunluklu kod sözcüğü,	000	001	010	011	100	101
Değişken-uzunluklu kod sözcüğü	0	101	100	111	1101	1100

Şekil 16.3 Bir karakter-kodlama problemi. 100.000 karakterli bir veri dosyası sadece frekansları ile verilmiş a–f karakterlerini içerir. Her bir karaktere 3-bitlik kod sözcüğü atarsak, dosyayı 300.000 bit ile kodlarız. Gösterilen değişken-uzunluklu kodu kullanarak dosyayı 224.000 bit ile kodlayabiliriz.

tek bir ikili katar ile temsil ettiği **bir ikili karakter kod** (yada kısa **kod**) tasarlama problemini düşünürüz. Eğer bir **sabit-uzunluklu (fixed-lenght) kod** kullanırsak, 6 karakteri temsil etmek için 3 bite ihtiyacımız vardır: a = 00, b = 001, ..., f = 101. Bu metod mevcut dosyayı kodlamak için 300.000 bit gerektirir. Daha iyisini yapabilir miyiz?

Bir değişken-uzunluklu kod (variable-lenght code), sık görülen karakterli kısa şifreler ve nadir görülen karakterli uzun şifreler vererek sabit-uzunluklu bir koddan çok daha iyisini yapabilir. Şekil 16.3 böyle bir kodu gösterir; 1-bitlik 0 katarı, a yı temsil eder ve 4-bitlik 1100 string, f yi temsil eder. Bu kod dosyayı temsil etmek için

$$(45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4) \cdot 1,000 = 224,000 \text{ bite}$$

ihtiyaç duyar, yaklaşık %25 kazanç yapar. Aslında, ileride göreceğimiz gibi, bu dosya için bir optimal karakter koddur.

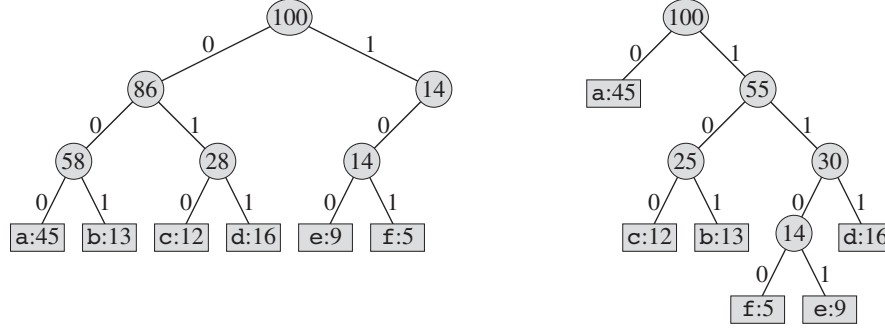
Önek kodları

Burada bir kod sözcüğünün başka diğer kod sözcüklerinin bir öneki olmadığı kodları dikkate alırız. Bu kodlar **önek kodları (prefix codes)** olarak adlandırılır. 3 Burada bunu kanıtlayamayacağımıza rağmen bir önek kodu her zaman bazı karakter kodları içinde optimal veri sıkıştırmayı elde edebilir ve böylece dikkatimizi önek kodlarına sınırlandırarak genelliği bozmayız.

Şifreleme herhangi bir ikili karakter kodu için her zaman basittir; sadece dosyanın her bir karakterini temsil eden kod sözcüklerini birleştiririz. Örneğin, Şekil 16.3 ün değişken-uzunluklu önek kodu ile, 3-karakterli abc dosyasını $0 \cdot 101 \cdot 100 = 0101100$ olarak kodlarız burada “.” birbirine bağlama anlamına gelir.

Önek kodları caziptir çünkü onlar deşifre etmeyi kolaylaştırır. Hiçbir kod sözcüğü herhangi diğerinin bir öneki olmadığı için, şifrelenmiş bir dosyanın başladığı kod sözcüğü anlaşılırdır. Başlangıç kod sözcüğünü basit bir şekilde tanımlaya-

³Belki “önek-serbest kodu” daha iyi bir isim olabilir, ancak literatürde “önek kodu” terimi standarttır.



Şekil 16.4 Şekil 16.3 teki şema kodlamasına uygun gelen ağaçlar. Her bir yaprak, bir karakter ve onun mevcut frekansı ile etiketlidir. Her bir iç düğüm, altağacındaki yaprakların frekansların toplamı ile etiketlidir. (a) Sabit-uzunluklu koda uygun gelen ağaç $a = 000, \dots, f = 101$. (b) Optimal örnek koduna uygun gelen ağaç $a = 0, b = 101, \dots, f = 1100$.

biliriz, bunu orijinal karakterlere çevirebilir ve şifrelenmiş dosyanın geri kalanı üzerinde şifre çözme işlemini tekrar edebiliriz. Örneğimizde 001011101 katarı, şifresi aabe olarak çözülen $0 \cdot 0 \cdot 101 \cdot 1101$ olarak tek bir şekilde parçalanır.

Başlangıç kod sözcüğünü kolaylıkla ayırabilmek için, şifre çözme işlemi örnek kodu için elverişli bir gösterimi gerektirir. Yaprakları, verilen karakterler olan bir ikili ağaç böyle bir gösterimi sağlar. Bir karakter için, kökten bu karaktere kadar olan basit yol olarak ikili kod sözcüğünü yorumlarız, burada 0 “sol çocuğa git” ve 1 “sağ çocuğa git” anlamına gelmektedir. Şekil 16.4 örneğimizin iki kodu için oluşturulmuş ağaçlarını gösterir. Yaprakların sıralanmış sırada yer almasına gerek olmadığından ve iç düğümler karakter anahtarlarını içermediğinden bunların ikili arama ağaçları olmadığına dikkat ediniz.

Bir dosya için bir optimal kod her zaman, her yapraksız düğüm iki tane çocuğa sahip olan bir *tam(full)* ikili ağaç tarafından temsil edilir (Alıştırma 16.3-2 bakınız). Örneğimizdeki sabit uzunluklu kod optimal değildir çünkü şekil 16.4(a) da gösterilen ağaç tam bir ikili ağaç değildir: bu 10... ile başlayan kod sözcüklerini içerir ama hiçbiri 11... ile başlamaz. Tam ikili ağaçlar için dikkatimizi sınırlandırabildiğimizden, eğer çizilmiş karakterlerin alfabesi C ve bütün karakterlerin frekansları pozitif ise, bir optimal örnek kodu için ağacın, alfabeadaki her bir karakter için tam olarak $|C|$ sayıda yaprak ve $|C| - 1$ sayıda iç düğüm içerdiğini söyleyebiliriz (Alıştırma B.5-3 bakınız).

Bir örnek koduna karşılık gelen bir T ağacı verilsin, bir dosyayı şifrelemek için gerekli bitlerin sayısını kolaylıkla hesaplayabiliriz. C alfabesindeki her bir c karakteri için, $c.freq$ özelliği dosyadaki c nin frekansı anlamına gelsin ve $d_T(c)$

ağaçtaki c nin yaprağının derinliğini belirtsin. Aynı zamanda $d_T(c)$ nin c karakteri için kod sözcüğünün uzunluğu olduğunu da dikkat ediniz. Bir dosyayı şifrelemek için gereken bitlerin sayısı böylece

$$B(T) = \sum_{c \in C} c.freq \cdot d_T(c) \quad (16.4)$$

T ağacının **maliyeti(cost)** olarak tanımlanır.

Bir Huffman kodun inşası

Huffman, **Huffman kod(huffman code)** olarak adlandırılan bir optimal örnek kodu oluşturan bir açgözlü algoritmayı buldu. Kısım 16.2 deki gözlemlerimiz ile ilgili doğrultuda, doğruluğunun kanıtı açgözlü-seçme özelliğine ve optimal altyapıya dayanır. Bu özellikleri sağladığını göstermek ve sonrasında sözde kod geliştirmektense ilk kabakodu ortaya koyarız. Bunu yapmak, algoritmanın açgözlü seçimlerini nasıl yaptığını açıklamada yardımcı olacaktır.

Aşağıdaki kabakodda, C nin n karakterli bir küme olduğunu ve her bir $c \in C$ nin frekansının $c.freq$ ile gösterilen bir nesnesi olduğunu kabul ederiz. Bu algoritma aşağıdan-yukarı yönteminde optimal koda karşılık gelen T ağacını inşa eder. $|C|$ yapraklı bir kümesi ile başlar ve sonuç ağacını oluşturmak için $|C| - 1$ sayıda “birleştirme” işlemi dizisi çalıştırır. Algoritma, birleştirilecek iki en az frekanslı objeyi belirlemek için $freq$ özelliği ile anahtarlanmış bir minimum-öncelikli Q kuyruğunu kullanır. İki nesneyi birleştirdiğimizde, sonuç frekansı birleştirilen iki nesnenin frekansları toplamı olan frekansa sahip bir yeni nesnedir.

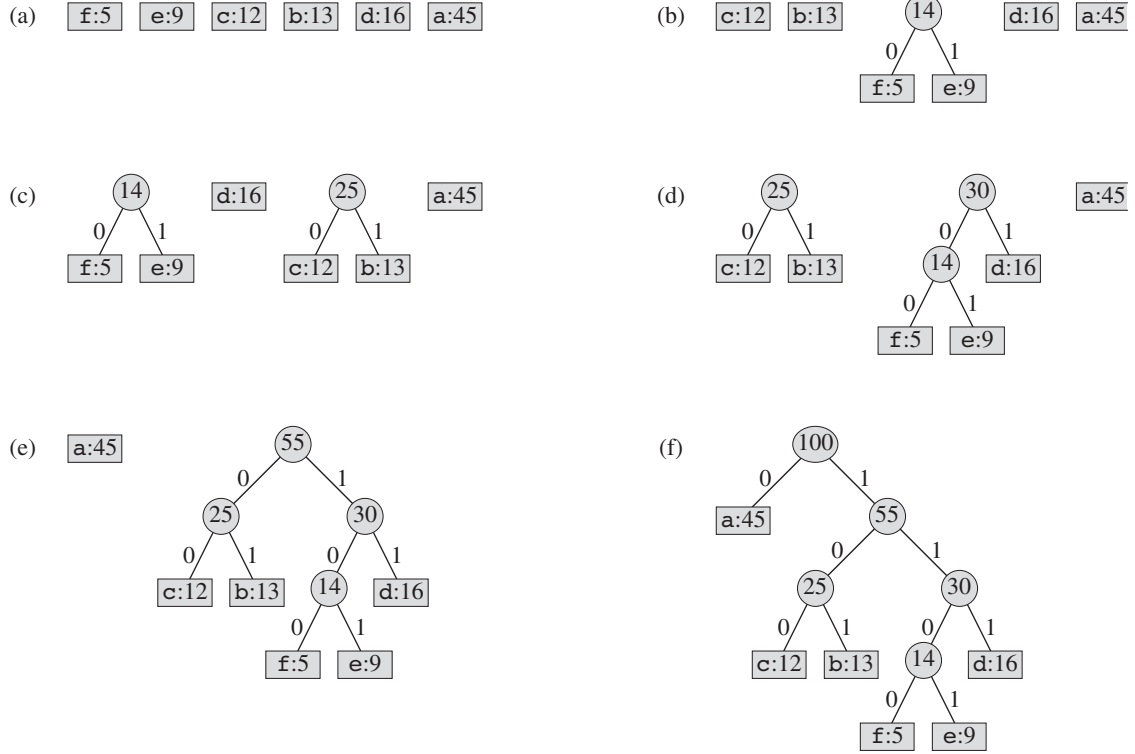
HUFFMAN(C)

```

1   $n = |C|$ 
2   $Q = C$ 
3  for  $i = 1$  to  $n - 1$ 
4      yeni bir  $z$  düğümü için yer ayır
5       $z.left = x = \text{EXTRACT-MIN}(Q)$ 
6       $z.right = y = \text{EXTRACT-MIN}(Q)$ 
7       $z.freq = x.freq + y.freq$ 
8       $\text{INSERT}(Q, z)$ 
9  return  $\text{EXTRACT-MIN}(Q)$     // ağacın kökünü geri döndür
```

Örneğimiz için Huffman'ın algoritması Şekil 16.5 de gösterildiği gibi çalışır. Alfabe 6 harf içerdiğinden başlangıç kuyruk boyutu $n = 6$ dır ve 5 birleştirme adımı ağacı inşa eder. Sonuç ağacı, optimal örnek kodunu gösterir. Bir harf için kod sözcüğü kökten bu harfe kadar olan basit yol üzerindeki ayrıtların etiketlerinin dizisidir.

2. satır C deki karakterler ile minimum-öncelikli Q kuyruğunu ilklendirir. 3-8 satırlarındaki **for** döngüsü durmadan, kuyruktan en küçük frekanslı x ve y dü-



Şekil 16.5 Şekil 16.3 te verilen frekanslar için Huffman algoritmasının adımları. Her bir kısım frekansa göre artan sırada sıralanmış kuyruğun içeriğini gösterir. Her bir adımda en düşük frekanslı iki ağaç birleştirilir. Yapraklar, bir karakter ve onun frekansını içeren dikdörtgenler olarak gösterilir. İç düğümler, çocuklarının frekanslarının toplamını içeren çemberler olarak gösterilir. Eğer bir ayrıt bir sol çocuk ise, bir iç düğümü çocuğu ile birleştiren bir ayrıt 0 ile etiketlenir ve sağ çocuğa bir ayrıt varsa 1 ile etiketlenir. Bir harfin kod sözcüğü, bu harf için köktü yaprağa birleştiren ayrıtlardaki etiketlerin dizisidir. (a) $n = 6$ düğümlü başlangıç kümesi, her bir harfe bir tane. (b)–(e) ara aşamalar (f) sonuç ağacı.

gümlelerini atar ve bunların birleşimini gösteren yeni bir z düğümü ile kuyrukta onların yerine geçer. z nin frekansı 7. satırda x ve y nin frekanslarının toplamı olarak hesaplanır. z düğümü x sol çocuğuna ve y sağ çocuğuna sahiptir (bu sıra gelişigüze; herhangi bir düğümün sağ ve sol çocuklarının yer değiştirmesi, aynı maliyette farklı bir kodu verir.) $n - 1$ birleşmeden sonra 9. satır kod ağacının kökü olan kuyruğun bir sol düğümünü geri döndürür.

Algoritma aynı sonucu üretmesine rağmen x ve y değişkenlerini alırsak 5. ve 6. satırlarda $z.left$ ve $z.right$ a direk olarak atayarak ve 7. satırda $z.freq = z.left.freq$

+ $z.right.freq$ şeklinde değiştirerek-doğruluğun kanıtında x ve y düğüm isimlerini kullanacağız. Bundan dolayı onların içinde bırakmak için uygun olanı buluruz.

Huffman algoritmasının çalışma zamanını analiz etmek için, Q nun bir ikili minimum-bellek yığını olarak uygulandığını kabul ederiz (Bölüm 6 ya bakınız). n karakterli bir C kümesi için Kısım 6.3 te tartışılan BUILD-MIN-HEAP işlemini kullanarak $O(n)$ zamanda satır 2 deki Q yı ilklendirebiliriz. 3-8 satırlarındaki **for** döngüsü tam olarak $n - 1$ kez çalışır ve her bir bellek yığını işlemi $O(\lg n)$ zamanda gerektirdiğinden döngü $O(n \lg n)$ çalışma zamanına sahip olur. Böylece n karakterli bir küme üzerindeki HUFFMAN ın toplam çalışma zamanı $O(n \lg n)$ dir. İkili min-bellek-yığını bir Van Emde Boas ağacı (Bölüm 20 ye bakınız) ile yer değiştirerek çalışma zamanını $O(n \lg \lg n)$ e düşürebiliriz.

Huffman algoritmasının doğruluğu

Açgözlü Huffman algoritmasının doğru olduğunu kanıtlamak için bir optimal örnek kodunun tanımlanması probleminin, açgözlü seçimi ve optimal altyapı özelliklerini sağladığını gösteririz. Bir sonraki lemma açgözlü-seçim özelliğinin sağlandığını gösterir.

Lemma 16.2

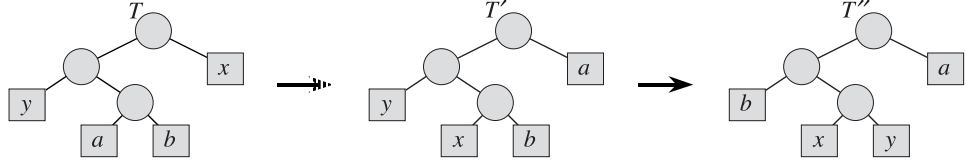
C her bir $c \in C$ karakterinin $c.freq$ frekansa sahip olduğu bir alfabe olsun. x ve y , C de en düşük frekansa sahip iki karakter olsun. C için bir optimal örnek kodu vardır, burada x ve y nin kod sözcükleri aynı uzunluktadır ve sadece son bitleri farklıdır.

Kanıt Kanıtta, keyfi bir optimal örnek kodu ile temsil edilen T ağacını alınacak ve bunu, yeni ağaçta maksimum derinlikteki kardeş yapraklar olarak görünen x ve y karakterleri olacak şekilde başka bir optimal örnek koduyla temsil edilen bir ağaç yapmak için düzenlenecektir. Böyle bir ağaç oluşturabilirsek x ve y için kod sözcükleri sadece son bitinde farklı ve aynı uzunluğa sahip olacaktır.

a ve b , T de maksimum derinlikteki kardeş yapraklar olan iki karakter olsun. Genelliği kaybetmeden $a.freq \leq b.freq$ ve $x.freq \leq y.freq$ olduğunu kabul edelim. $x.freq$ ve $y.freq$ en küçük iki yaprak frekansı olduğundan sırasıyla $a.freq$ ve $b.freq$ iki keyfi frekanslardır, sırasıyla $x.freq \leq a.freq$ ve $y.freq \leq b.freq$ sağlanır.

Kanıtın geri kalanında, $x.freq = a.freq$ ya da $y.freq = b.freq$ olduğu mümkündür. Ancak, $x.freq = b.freq$ sağlansaydı $a.freq = b.freq = x.freq = y.freq$ e sahip olurduk (Aıştırma 16.3-1 bakınız) ve lemma açık bir şekilde doğru olurdu. Bu nedenle $x \neq b$ olduğunda $x.freq \neq b.freq$ olduğunu kabul edeceğiz.

Şekil 16.6 nın gösterdiği gibi, x ve y maksimum derinlikteki kardeş yapraklar olduğunda T' ağacını üretmek için a ve x in T deki konumlarını değiştirir ve sonra



Şekil 16.6 Lemma 16.2 nin kanıtındaki anahtar adımın bir örnekleme. Optimal T ağacında, x ve y yaprakları maksimum derinlikteki iki kardeş yapraktır. x ve y yaprakları en düşük frekansa sahip iki karakterdir; bunlar T de keyfi yerlerde bulunurlar. $x \neq b$ olduğunu varsayarak, a ve x yapraklarının karşılıklı yer değiştirmesi T' ağacını üretir ve sonra b ve y yapraklarının karşılıklı yer değiştirmesi T'' ağacını üretir. Her değişim maliyeti artırmadığından, T'' sonuç ağacı aynı zamanda bir optimal ağaçtır.

bir T'' ağacını üretmek için b ve y nin T' ndeki konularını değiştiririz. ($x = b$ ama $y \neq a$ ise T'' ağacı maksimum derinlikteki kardeş yapraklar olarak x ve y ye sahip değildir. $x \neq y$ kabul ettiğimizden bu durum oluşmaz.) Denklem (16.4) ile T ve T'' arasındaki maliyet farkı, hem $a.freq - x.freq$ hemde $d_T(a) - d_T(x)$ negatif olmadığından

$$\begin{aligned}
 B(T) - B(T') &= \sum_{c \in C} c.freq \cdot d_T(c) - \sum_{c \in C} c.freq \cdot d_{T'}(c) \\
 &= x.freq \cdot d_T(x) + a.freq \cdot d_T(a) - x.freq \cdot d_{T'}(x) - a.freq \cdot d_{T'}(a) \\
 &= x.freq \cdot d_T(x) + a.freq \cdot d_T(a) - x.freq \cdot d_T(a) - a.freq \cdot d_T(x) \\
 &= (a.freq - x.freq)(d_T(a) - d_T(x)) \\
 &\geq 0
 \end{aligned}$$

dır. Daha özel olarak, x minimum frekanslı bir yaprak olduğundan $a.freq - x.freq$ negatif değildir ve a , T de maksimum derinlikte bir yaprak olduğundan $d_T(a) - d_T(x)$ negatif değildir. Benzer şekilde, y ve b nin yer değiştirmesiyle maliyet artmaz bu nedenle $B(T') - B(T'')$ negatif değildir. Dolayısıyla $B(T'') \leq B(T)$ dir ve T optimal olduğundan $B(T) \leq B(T'')$ olması $B(T'') = B(T)$ sonucunu getirir. Böylece T'' , bir sonraki lemmadan maksimum derinlikteki kardeş yapraklar olan x ve y nin olduğu bir optimal ağaçtır. ■

Lemma 16.2, birleşme işlemleriyle bir optimal ağaç oluşturma sürecinin genelliği bozmadan en düşük frekanslı iki karakterin birleştirilmesinin ağgözlü seçimi ile başlayabildiğini gösterir. Niçin bu bir ağgözlü seçimdir? Biz birleştirilen iki ögenin frekansları toplamı olarak tek bir birleşme maliyetini görebiliriz. Alıştırma 16.3-4, birleşmelerinin maliyetlerinin toplamı oluşturulan ağacın toplam maliyetine eşit olduğunu gösterir. Her adımdaki mümkün birleşmelerin içinden, HUFFMAN en az maliyetli olanı seçer.

Sonraki lemma optimal örnek kodlarının oluşturulması probleminin optimal altyapı özelliğine sahip olduğunu gösterir.

Lemma 16.3

C , her bir $c \in C$ karakteri için tanımlanmış $c.freq$ frekans ile verilen bir alfabe olsun. x ve y , C de minimum frekanslı iki karakter olsun. $C' = C - \{x, y\} \cup \{z\}$ olacak şekilde C den x ve y nin silinip, yeni bir karakter z nin eklenmesi ile oluşan bir alfabe olsun. $z.freq = x.freq + y.freq$ i göz ardı ederek C de olduğu gibi C' için f i tanımlayınız. T' , C' alfabesi için bir optimal örnek kodunu gösteren herhangi bir ağaç olsun. O halde, çocuk olarak x ve y ye sahip bir iç düğüm olan z ile bir yaprak düğümün yer değiştirilmesi ile T' den elde edilen T ağacı, C alfabesi için bir optimal örnek kodunu ifade eder.

Kanıt İlk olarak, (16.4) denklemindeki maliyetin bileşenleri göz önüne alınırsa T' ağacının $B(T')$ maliyeti açısından T ağacının $B(T)$ maliyetinin nasıl ifade edildiğini gösteririz. Her bir $c \in C - \{x, y\}$ karakteri için, $d_T(c) = d_{T'}(c)$ dir ve bundan dolayı $c.freq \cdot d_T(c) = c.freq \cdot d_{T'}(c)$ dir. $d_T(x) = d_T(y) = d_{T'}(z) + 1$ olduğundan

$$\begin{aligned} x.freq \cdot d_T(x) + y.freq \cdot d_T(y) &= (x.freq + y.freq)(d_{T'}(z) + 1) \\ &= z.freq \cdot d_{T'}(z) + (x.freq + y.freq) \end{aligned}$$

dir ve buradan

$$B(T) = B(T') + x.freq + y.freq \text{ ya da buna eşit olarak } B(T') = B(T) - x.freq - y.freq \text{ sonucuna varırız.}$$

Şimdi lemmayı çelişkiyle kanıtlayalım. T nin C için bir optimal örnek kodunu göstermediğini kabul edelim. Bu durumda $B(T'') < B(T)$ olacak şekilde bir optimal T'' ağacı vardır. Genelliği bozmadan (Lemma 16.2 ile), T'' , kardeş olarak x ve y ye sahiptir. T''' , $z.freq = x.freq + y.freq$ frekanslı bir z yaprağı ile x ve y nin ortak ebeveyninin yer değiştirdiği T'' ağacı olsun. Bu durumda

$$\begin{aligned} B(T''') &= B(T'') - x.freq - y.freq \\ &< B(T) - x.freq - y.freq \\ &= B(T'), \end{aligned}$$

T' nün, C' için bir optimal örnek kodunu temsil ettiği kabulümüz ile bir çelişki oluşturur. Böylece T , C alfabesi için bir optimal örnek kodunu ifade etmelidir. ■

Teorem 16.4

HUFFMAN prosedürü bir optimal örnek kodu üretir.

Kanıt Lemma 16.2 ve 16.3 den doğrudan elde edilir. ■

Alıştırılmalar

16.3-1

Lemma 16.2 nin kanıtında geçen $x.freq = b.freq$ ise $a.freq = b.freq \Rightarrow x.freq = y.freq$ eşitliğine sahip olmamızın nedenini açıklayınız.

16.3-2

Tam olmayan bir ikili ağacın bir optimal örnek koduna karşılık gelemeyeceğini kanıtlayınız.

16.3-3

İlk 8 fibonacci sayısına dayanan, aşağıdaki frekansların kümesi için bir optimal Huffman kodu nedir?

a:1 b:1 c:2 d:3 e:5 f:8 g:13 h:21

Frekansları ilk n fibonacci sayısı olduğunda optimal kodu bulmak için cevabınızı genelleştirebilir misiniz?

16.3-4

Bütün iç düğümler üzerinde, düğümün iki çocuğunun frekanslarının birleşiminin toplamı olarak bir kod için bir ağacın toplam maliyetini de ifade edebildiğimizi kanıtlayınız.

16.3-5

Alfabadeki karakterleri frekansları monoton azalan olacak şekilde sıralarsak, kod sözcüğünün uzunluğunun monoton artan olan bir optimal kodunun var olduğunu kanıtlayınız.

16.3-6

Karakterlerinin bir $C = \{0, 1, \dots, n-1\}$ kümesi üzerinde optimal bir örnek koduna sahip olduğumuzu kabul edelim ve mümkün olduğu kadar az bit kullanarak bu kodu aktarmak isteyelim. Sadece $2n - 1 + n \lceil \lg n \rceil$ bit kullanarak C üzerinde herhangi bir optimal örnek kodunun nasıl temsil edildiğini gösteriniz. (İpucu: Ağacın bir yürüyüşü ile gösterildiği gibi ağacın yapısını belirtmek için $2n-1$ bit kullanınız.)

16.3-7

Üç terimli kod sözcükleri için Huffman algoritmasını genelleştiriniz.(yani, 0,1 ve 2 sembolleri kullanılan kod sözcükleri) ve bunun optimal üç terimli kodları oluşturduğunu kanıtlayınız.

16.3-8

Tüm 256 karakterin, neredeyse eşit olduğu 8 bit lik karakterlerin bir dizisini içeren bir veri dosyasını kabul edelim: Maksimum karakter frekansı, minimum karakter frekansının iki katından daha azdır. Bu durumda Huffman kodlamasının, sıradan bir 8-bitlik sabit-uzunluklu kod kullanımından daha verimli olmadığını kanıtlayınız.

16.3-9

Hiçbir sıkıştırma şemasının rastgele seçilen 8 bit karakterli bir dosyayı tek bir bitle sıkıştıramadığını gösteriniz. (*İpucu:* Mümkün şifrelenmiş dosyaların sayısı ile mümkün dosyaların sayısını kıyaslayınız.)

*** 16.4 Matroidler ve ağgözlü yöntemler**

Bu kısımda ağgözlü algoritmalar hakkında güzel bir teori çiziyoruz. Bu teori ağgözlü yönteminin optimal çözümleri oluşturduğu birçok durumu tanımlar. Bu “matroidler” olarak bilinen kombinatorik yapılarla ilgilidir. Bu teori ağgözlü yöntemin uygulandığı tüm durumları kapsamamasına rağmen (örneğin, Kısım 16.1 in etkinlik-seçim problemini ya da Kısım 16.3 ün huffman kodlama problemini kapsamaz.), birçok uygulama örneğini kapsar. Buna ek olarak, bu teori birçok uygulamayı kapsamayı için genişletilmiştir; referanslar için bu bölümün sonundaki notlara bakınız.

Matroidler

Bir **matroid** aşağıdaki koşulları sağlayan bir $M = (S, \mathcal{I})$ sıralı ikilisidir.

1. S sonlu bir kümedir.
2. $\mathcal{I}; B \in \mathcal{I}$ ve $A \subseteq B$ ise $A \in \mathcal{I}$ olacak şekilde S nin **bağımsız**(*independent*) alt kümeleri olarak adlandırılan S nin alt kümelerinin boştan farklı bir ailesidir. Eğer bu özelliği sağlarsa $\mathcal{I}$ nin **kalıtsal** olduğunu söyleriz. $\emptyset$ kümenin $\mathcal{I}$ nin zorunlu bir üyesi olduğuna dikkat ediniz.
3. $A \in \mathcal{I}, B \in \mathcal{I}$ ve $|A| < |B|$ ise $A \cup \{x\} \in \mathcal{I}$ olacak şekilde en az öyle bir $x \in B - A$ elemanı vardır. M nin **özellik değiştirmeyi** (*exchange property*) sağladığını söyleriz.

“Matroid” kelimesi Hassler Whitney den gelir. Hassler Whitney, S nin elemanları verilen bir matrisin satırlarından oluşan ve bunlar her zamanki anlamda lineer bağımsız ise satırların bir kümesinin de bağımsız olduğu **matrik matroid** (*matric matroid*)leri çalışıyordu. Alıştırma 16.4-2 göstermemiz için sorduğu gibi, bu yapı bir matroid tanımlar.

Matroidlerin diğer örneği olarak, verilen bir yönsüz çizge $G = (V, E)$ açısından aşağıdaki gibi tanımlı **çizgesel matroid**(*graphic matroid*) $M_G = (S_G, \mathcal{I}_G)$ yi dikkate alınız:

- S_G kümesi, G nin ayrıtlarının kümesi E olarak tanımlanmaktadır.
- A, E nin bir alt kümesi ise $A \in \mathcal{I}_G$ dir ancak ve ancak A çevrimsizdir. Yani A nin ayrıtlarının bir kümesi bağımsızdır ancak ve ancak $G_A = (V, A)$ altçizgesi bir orman oluşturur.

Çizgesel matroid M_G , Bölüm 23 te ayrıntılı olarak bulunan minimum kapsayan ağaç problemi ile yakından ilişkilidir.

Teorem 16.5

$G = (V, E)$ yönsüz bir çizge ise $M_G = (S_G, \mathcal{I}_G)$ bir matroidtir.

Kanıt $S_G = E$ nin sonlu bir küme olduğu açıktır. Buna ek olarak, bir ormanın alt kümesi bir orman olduğundan $\mathcal{I}_G$ kalıtsaldır. Başka bir deyişle, ayrıtların çevrimsiz bir kümesinden ayrıtlar çıkarıldığında devre oluşturulamaz.

Böylece geriye M_G nin özellik değiştirmeyi sağladığını göstermek kalır. $G_A = (V, A)$ ve $G_B = (V, B)$ G nin ormanları olduğunu ve $|B| > |A|$ olduğunu kabul edelim. Yani, A ve B , ayrıtların devre oluşturmayan kümeleridir ve B, A nın içerdiklerinden daha fazla ayrıt içerir.

Bir $F = (V_F, E_F)$ ormanının tam olarak $|V_F| - |E_F|$ ağaç içerdiğini iddia ederiz. Nedeninin görmek için, F nin t ağaç içerdiğini kabul edelim burada i . ağaç v_i tane tepe ve e_i tane ayrıt içerir. O halde, $t = |V_F| - |E_F|$ yi sonucunu veren

$$\begin{aligned} |E_F| &= \sum_{i=1}^t e_i \\ &= \sum_{i=1}^t (v_i - 1) \quad (\text{Teorem B.2 den}) \\ &= \sum_{i=1}^t v_i - t \\ &= |V_F| - t \end{aligned}$$

işlemine sahip oluruz. Böylece G_A ormanı $|V| - |A|$ ağaç içerir ve G_B ormanı $|V| - |B|$ ağaç içerir.

G_B ormanı, G_A ormanında olandan daha az ağaç içerdiğinden, G_B ormanı, tepeleri G_A ormanındaki iki farklı ağaç olan bir T ağacını içermelidir. Dahası, T bağlantılı olduğundan, u ve v tepeleri G_A ormanındaki farklı ağaçlarda olacak şekilde bir (u, v) ayrıtını içermelidir. (u, v) ayrıtı G_A ormanındaki iki farklı ağaçtaki tepeleri bağladığından, (u, v) ayrıtını bir çevrim oluşturmada G_A ormanına ekleyebiliriz. Böylece M_G özellik değiştirmeyi sağlar, M_G nin bir matroid olduğu kanıtı tamamlanır. ■

Bir $M = (S, \mathcal{I})$ matroidi verilsin, eğer bağımsızlık korunurken x i A ya ekleyebilirsek, bir $x \notin A$ elemanını $A \in \mathcal{I}$ nin bir **genişlemesi(extension)** olarak adlandırırız; yani $A \cup \{x\} \in \mathcal{I}$ ise x, A nın bir genişlemesidir. Örnek olarak, bir M_G çizgesel matroidini dikkate alalım. A , ayrıtların bağımsız bir kümesi ise, e ayrıtı A nın bir genişlemesidir ancak ve ancak e, A da değildir ve e nin A ya eklenmesi bir çevrim oluşturmaz.

A , bir M matroidinde bağımsız bir alt küme ise, A hiçbir genişlemeye sahip olmadığında A **maksimal(maximal)**dir deriz. Yani M nin herhangi bir daha büyük alt kümesinde içerilmiyorsa A maksimaldir. Aşağıdaki özellik sık sık kullanılır.

Teorem 16.6

Bir matroidteki bütün maksimal bağımsız alt kümeler aynı boyuttur.

Kanıt Aksini yani, A nın M nin bir maksimal bağımsız alt kümesi olduğunu ve M nin bir diğer daha büyük maksimal alt kümesinin B olduğunu kabul edelim. Sonra, özellik değiştirme, bazı $x \in B - A$ için A yı daha büyük bağımsız $A \cup \{x\}$ kümesine genişletebilmemizi sağlar. Bu ise A nın maksimal olması ile çelişir. ■

Bu teoremin bir örneği olarak, bağlantılı ve yönsüz bir G çizgesi için bir M_G çizgesel matroidini düşünün. M_G nin her maksimal bağımsız alt kümesi, G nin bütün tepelerini birleştiren tam olarak $|V| - 1$ tane ayrıtı olan serbest bir ağaç olmalıdır. Böyle bir ağaç, G nin bir **kapsayan ağacı** (**spanning tree**) olarak adlandırılır.

Eğer her bir $x \in S$ e bir pozitif $w(x)$ ağırlığını atayan bir w ağırlık fonksiyonu varsa bu $M = (S, \mathcal{I})$ matroidine **ağırlıklı** (**weighted**) matroid deriz. w ağırlık fonksiyonu, her $A \subseteq S$ için S nin alt kümelerine aşağıdaki toplam ile genişletir:

$$w(A) = \sum_{x \in A} w(x)$$

Örnek olarak, $w(e)$ bir M_G çizgesel matroidindeki bir e ayrıtının ağırlığını gösterirse, $w(A)$, A ayrıt kümesindeki ayrıtıların toplam ağırlığıdır.

Bir ağırlıklı matroid üzerinde açgözlü algoritmalar

Bir açgözlü yaklaşımın optimal çözümleri sağladığı bir çok problem, bir ağırlıklı matroidteki maksimum-ağırlıklı bağımsız bir alt kümeyi bulmak açısından formüle edilebilir. Yani, bir ağırlıklı $M = (S, \mathcal{I})$ matroidi bize verilsin ve $w(A)$ nın maksimum olduğu bağımsız bir $A \in \mathcal{I}$ kümesini bulmak isteriz. Maksimum olası ağırlığa sahip ve bağımsız olan bu gibi alt kümeye matroidin bir **optimal** alt kümesi deriz. Herhangi bir $x \in S$ nin $w(x)$ ağırlığı pozitif olduğundan bir optimal alt küme her zaman bir maksimal bağımsız alt kümedir—bu her zaman mümkün olduğu kadar büyük A yı elde etmede yardımcı olur.

Örneğin, **minimum-kapsayan ağaç problemi**nde, bağlantılı yönsüz bir $G = (V, E)$ çizgesi ve $w(e)$, e ayrıtının pozitif uzunluğu olacak şekilde bir w uzunluk fonksiyonu verilsin. (“uzunluk” burada çizge için orijinal ayrıtın ağırlığını işaret eden terim olarak kullanılır. İlgili matroidte ise “ağırlık” terimi yine ağırlığa karşılık gelir.) Minimum toplam uzunluğa sahip ve bütün tepeleri ile bağlantılı olan ayrıtıların bir alt kümesini bulmak isteriz. Bir matroidin optimal bir alt kümesini bulma problemini incelemek için, w' ağırlık fonksiyonuna sahip M_G ağırlıklı matroidini düşünelim, burada $w'(e) = w_0 - w(e)$ ve w_0 herhangi bir ayrıtın maksimum uzunluğundan daha fazladır. Bu ağırlıklı matroidte, bütün ağırlıklar pozitif ve iyi alt küme, orijinal çizgedeki minimum toplam uzunluğa sahip bir kapsayan ağaçtır. Daha belirgin olarak, her maksimal bağımsız A alt kümesi, $|V| - 1$ tane

ayrıta sahip bir kapsayan ağaca karşılık gelir ve herhangi bir bağımsız maksimal A alt kümesi için

$$\begin{aligned}
 w'(A) &= \sum_{e \in A} w'(e) \\
 &= \sum_{e \in A} (w_0 - w(e)) \\
 &= (|V| - 1)w_0 - \sum_{e \in A} w(e) \\
 &= (|V| - 1)w_0 - w(A)
 \end{aligned}$$

olduğundan, $w'(A)$ miktarını maksimum yapan bağımsız bir alt küme, $w(A)$ yı minimum yapmalıdır. Böylece keyfi bir matroidteki optimal bir A alt kümesini bulabilen herhangi bir algoritma minimum-kapsayan-ağaç problemini çözebilir.

Bölüm 23 minimum-kapsayan-ağaç-problemi için algoritmalar verir fakat biz burada herhangi bir ağırlıklı matroid için çalışan bir açgözlü algoritması veriyoruz. Algoritma bir ilgili pozitif ağırlık fonksiyonu w ile $M = (S, \mathcal{I})$ ağırlıklı matroidi girdi olarak alır ve bir optimal A alt kümesini geri döndürür. Kabakodumuzda, w ile ağırlık fonksiyonunu ve $M.S$ ve $M.\mathcal{I}$ ile M nin bileşenlerini gösterir. Algoritma açgözlüdür, çünkü monoton azalan sırasında her bir $x \in S$ elemanını dikkate alır ve $A \cup \{x\}$ bağımsız ise bunu A kümesine ekler.

GREEDY (M, w)

1. $A = \emptyset$
2. $M.S$ yi w ağırlığının monoton azalan sırasında sırala
3. **for** $w(x)$ ağırlığının monoton azalan sırasında alınan her bir $x \in M.S$ için
4. **if** $A \cup \{x\} \in M.\mathcal{I}$
5. $A = A \cup \{x\}$
6. **return** A

4. satır A ya eklenen herbir x elemanının A yı bağımsız bir küme olarak sürdürüp sürdürmediğini kontrol eder. Eğer A bağımsız devam ederse, 5. satır x i A ya ekler. Aksi halde x hesaba katılmaz. Boş küme bağımsız olduğundan ve **for** döngüsünün her bir iterasyonu A ları bağımsız devam ettirdiğinden tümevarımdan A alt kümesi her zaman bağımsızdır. Bu nedenle, GREEDY daima bağımsız bir A alt kümesini döndürür. Birazdan A nın mümkün maksimum ağırlıkta bir alt küme olduğunu göreceğiz, böylece A optimal bir alt kümedir.

GREEDY nün çalışma zamanını analiz etmek kolaydır. $n, |S|$ yi gösterebilir. GREEDY nün sıralama aşaması $O(n \lg n)$ zaman alır. S nin her bir elemanı için bir defa olacak şekilde 4. satır tam olarak n defa çalışır. 4. satırın her bir çalışması $A \cup \{x\}$ kümesinin bağımsız olup olmadığının kontrolünü gerektirir. Bu tür her kontrol $O(f(n))$ zaman alırsa, bütün algoritma $O(n \lg n + nf(n))$ zamanda çalışır.

Şimdi GREEDY'nün bir optimal alt küme döndürdüğünü kanıtlayacağız.

Lemma 16.7 (Matroidler açgözlü-seçim özelliğini gösterir)

$M = (S, \mathcal{I})$ nın w ağırlık fonksiyonlu bir ağırlıklı matroid olduğunu ve S nin ağırlığına göre monoton azalan sırada sıralandığını kabul edelim. $\{x\}$ bağımsız olacak şekilde en az öyle bir x var ise, x , S nin ilk elemanı olsun. Eğer böyle bir x var ise, x i içeren S nin optimal bir A alt kümesi vardır.

Kanıt Böyle bir x yok ise bağımsız alt küme sadece boş kümedir ve lemma doğrudur. Aksi halde B boştan farklı herhangi bir alt küme olsun. $x \notin B$ olduğunu kabul edelim; aksi halde $A = B$ olması, x i içeren S nin bir optimal alt kümesini verir.

B nin hiçbir elemanı $w(x)$ den daha büyük ağırlığa sahip değildir. Nedenini görmek için, $y \in B$, $\{y\}$ nin bağımsız olmasını işaret ettiğini inceler, çünkü $B \in \mathcal{I}$ dir ve $\mathcal{I}$ kalıtsaldır. Bizim seçimimiz x böylece herhangi bir $y \in B$ için $w(x) \geq w(y)$ olduğunu garantiler.

A kümesini aşağıdaki gibi oluşturun. $A = \{x\}$ ile başla. x in seçiminden A kümesi bağımsızdır. Özellik değiştirmeyi kullanarak, A nın bağımsızlığı korunduğu sürece, $|A| = |B|$ oluncaya kadar A ya ekleyebilmek için B nin yeni bir elemanını bul. Bu noktada, A ve B , A nın x e sahip olması ve B nin başka diğer bir eleman y ye sahip olması dışında aynıdır. Yani, herhangi bir $y \in B$ için, $A = B - \{y\} \cup \{x\}$ dir, bu nedenle

$$\begin{aligned} w(A) &= w(B) - w(y) + w(x) \\ &\geq w(B) \end{aligned}$$

dir. B kümesi optimal olduğundan x i içeren A kümesi optimal olmalıdır. ■

Sonra, eğer bir eleman başlangıçta bir seçenek değilse daha sonrası içinde bir seçenek olamadığını göstereceğiz.

Lemma 16.8

$M = (S, \mathcal{I})$ herhangi bir matroid olsun. Eğer x , S nin bir elemanı ise, yani S nin herhangi bir bağımsız A alt kümesinin bir genişlemesi ise, o zaman x , $\emptyset$ nin bir genişlemesidir.

Kanıt x , A nın bir genişlemesi olduğundan $A \cup \{x\}$ in bağımsız olduğuna sahibiz. $\mathcal{I}$ kalıtsal olduğundan, $\{x\}$ bağımsız olmalıdır. Böylece x , $\emptyset$ nin bir genişlemesidir. ■

Sonuç 16.9

$M = (S, \mathcal{I})$ herhangi bir matroid olsun. x , $\emptyset$ nin bir genişlemesi olmayacak şekilde S nin bir elemanı ise, x , S nin herhangi bir bağımsız A alt kümesinin bir genişlemesi değildir.

Kanıt Bu sonuç, basit olarak Lemma 16.8 in karşıt tersidir. ■

Sonuç 16.9, direk olarak kullanılamayan herhangi bir elemanın hiçbir zaman kullanılamayacağını söyler. Bu nedenle, GREEDY, $\emptyset$ kümenin bir genişlemesi olmayan S kümesindeki bazı başlangıç elemanlarını yok sayarak hata yapamaz, çünkü bu elemanlar hiçbir zaman kullanılamaz.

Lemma 16.10 (Matroidler optimal-altıyapı özelliğini gösterir)

Ağırlıklı $M = (S, \mathcal{I})$ matroidi için GREEDY ile seçilen S nin ilk elemanı x olsun. x i içeren maksimum-ağırlıklı bağımsız bir alt küme bulmanın kalan problemi, $M' = (S', \mathcal{I}')$ ağırlıklı matroidinin maksimum-ağırlıklı bağımsız bir alt kümesini bulmaya indirgenir, burada

$$\begin{aligned} S' &= \{y \in S : \{x, y\} \in \mathcal{I}\}, \\ \mathcal{I}' &= \{B \subseteq S - \{x\} : B \cup \{x\} \in \mathcal{I}\} \end{aligned}$$

ve M' için ağırlık fonksiyonu, S' ye kısıtlanmış M nin ağırlık fonksiyonudur. (M' nü, x elemanı tarafından M nin **büzülmesi**(*contraction*) olarak adlandırırız.)

Kanıt Eğer A , x i içeren M nin herhangi bir maksimum ağırlıklı bağımsız alt kümesi ise $A' = A - \{x\}$, M' nün bağımsız bir alt kümesidir. Tam tersine, M' nün herhangi bir bağımsız A' alt kümesi, M nin bağımsız bir $A = A' \cup \{x\}$ alt kümesini oluşturur. $w(A) = w(A') + w(x)$ olan her iki duruma sahip olduğumuzdan, x i içeren M de bir maksimum-ağırlıklı çözüm, M' de bir maksimum ağırlıklı çözümü oluşturur ve tam tersidir. ■

Teorem 16.11 (Matroidler üzerindeki açgözlü algoritmanın doğruluğu)

Eğer $M = (S, \mathcal{I})$, w ağırlık fonksiyonlu bir ağırlıklı matroid ise GREEDY(M, w) bir optimal alt küme döndürür.

Kanıt Sonuç 16.9 ile, $\emptyset$ nin genişlemesi olmadıklarından GREEDY nin ilk başlangıçta yok saydığı bazı elemanlar unutulabilir çünkü bunlar hiçbir zaman kullanılmaz. GREEDY birinci eleman x i seçtiğinde, Lemma 16.7, algoritmanın x i A ya ekleyerek hata vermediğini gösterir çünkü x i içeren bir optimal alt küme vardır. Son olarak, Lemma 16.10, kalan problemin, x tarafından M nin büzülmesi olan M' matroidindeki bir optimal alt kümenin birini bulmak olduğunu gösterir. GREEDY prosedürü A yı $\{x\}$ e eşitledikten sonra tüm geriye kalan adımları $M' = (S', \mathcal{I}')$ matroidinde uygulandığı gibi yorumlayabiliriz çünkü tüm $B \in \mathcal{I}'$ kümeleri için B , M' nde bağımsızdır ancak ve ancak $B \cup \{x\}$ kümesi M de bağımsızdır. Böylece GREEDY nin bir sonraki işlemi M' için maksimum-ağırlıklı bağımsız bir alt küme bulmaktır ve GREEDY nin genel olarak işlemi M için maksimum-ağırlıklı bağımsız bir alt küme bulmaktır. ■

Alıştırmalar

16.4-1

S herhangi bir sonlu küme ve $\mathcal{I}_k$, büyüklüğü en fazla k olan S nin bütün alt kümelerinin bir kümesi iken $(S, \mathcal{I}_k)$ nin bir matroid olduğunu gösteriniz, burada $k \leq |S|$ tir.

16.4-2 *

Bir cisim (Reel sayılar gibi) üzerinde $m \times n$ lik bir T matrisi verilsin, $(S, \mathcal{I})$ nin bir matroid olduğunu gösteriniz burada S, T matrisinin sütunlarının kümesi ve $A \in \mathcal{I}$ dir ancak ve ancak A daki sütunlar lineer bağımsızdır

16.4-3 *

$(S, \mathcal{I})$ bir matroid ise $(S, \mathcal{I}')$ bir matroidtir, burada

$$\mathcal{I}' = \{A' : S - A' \text{ bazı } A \in \mathcal{I} \text{ maksimalini içerir}\}.$$

Yani, $(S, \mathcal{I}')$ nün maksimal bağımsız kümeleri, sadece $(S, \mathcal{I})$ nin maksimal bağımsız kümelerinin tümleyenleridir.

16.4-4 *

S sonlu bir küme ve $S_1, S_2, \dots, S_k$, boştan farklı ayrık alt kümeleri içinde S nin bir parçalanışı olsun. $\mathcal{I} = \{A : |A \cap S_i| \leq 1, i = 1, 2, \dots, k\}$ olması koşulunda $(S, \mathcal{I})$ yapısını tanımlayınız. $(S, \mathcal{I})$ nin bir matroid olduğunu gösteriniz. Yani, parçalanıştaki her bir alt kümenin en fazla bir elemanını içeren bütün A kümelerinin kümesi bir matroidin bağımsız kümelerini belirler.

16.4-5

Bir ağırlıklı matroid probleminin ağırlık fonksiyonuna nasıl dönüştüğünü gösteriniz, burada istenen optimal çözüm, bir standart ağırlıklı-matroid problemini üretmek, bir minimum-ağırlıklı maksimal bağımsız alt kümedir. Dönüşümünün doğru olduğunu dikkatlice tartışınız.

* 16.5 Bir matroid olarak görev-çizelgeleme problemi

Matroidleri kullanarak çözebileceğimiz ilginç bir problem, optimal şekilde tek bir işlemci üzerinde birim-zaman görevleri çizelgeleme problemidir, burada her görev bir bitiş zamanına sahiptir, bununla birlikte eğer görev bitiş zamanını kaçırırsa bir ceza öder. Problem anlaşılması zor gibi görünür, ancak bu problemi bir matroid gibi düşünerek şaşırtıcı bir şekilde basit bir yöntemde ve bir açgözlü algoritma kullanarak çözebiliriz.

Bir **birim-zaman görevi**(*unit-time task*) bir iştir, yani tam olarak bir birim sürede tamamlanmayı gerektiren bir bilgisayarda çalışan. bir program gibidir. Birim-zaman görevlerinin sonlu bir S kümesi verilsin, S için bir **çizelge**(*schedule*)

bu görevleri yerine getirmek için belirlenen sırada S nin bir permütasyonudur. Çizelgedeki ilk görev 0 zamanda başlar ve 1 zamanda biter, ikinci görev 1 zamanda başlar ve 2 zamanda biter ve böyle devam eder.

Tek bir işlemci için bitiş zamanı(deadline) ve cezalara(penalties) sahip olan birim-zaman görevlerinin(unit-time tasks) çizelgeleme problemi aşağıdaki gir-dilere sahiptir:

- $S = \{a_1, a_2, \dots, a_n\}$ kümesi n tane birim-zaman görevlerinin bir kümesidir.
- n tane tam sayı **bitiş zamanlarının** bir kümesi $d_1, d_2, \dots, d_n$ burada her d_i , $1 \leq d_i \leq n$ dir ve a_i görevi d_i zamanda tamamlanması beklenir, ve
- Eğer a_i görevi d_i zamanda bitmezse, bir w_i cezasına maruz kalan ve eğer bir görev bitiş zamanında biterse hiçbir cezaya maruz kalmayacak şekilde n tane negatif olmayan $w_1, w_2, \dots, w_n$ ağırlıklarının veya **cezalarının(penalties)** bir kümesidir.

Bitiş zamanını kaçıranlar için maruz kalınan toplam cezayı minimum yapan S için bir çizelge bulmak isteriz.

Verilen bir çizelgeyi düşünün. Eğer görev, bitiş zamanından sonra biterse bu göreve bu çizelgede **son** deriz. Aksi halde bu görev **erken** deriz. Keyfi bir çizelgeyi her zaman **erken-ilk biçim(early-first form)**e dönüştürebiliriz, burada erken görevler son görevlerden önce gelir. Nedenini görmek için, eğer herhangi bir erken görev a_i , herhangi bir son görev a_j yi takip ediyorsa, a_i ve a_j nin konumlarını değiştirebiliriz, ve a_i hala erken ve a_j hala son olacaktır.

Ayrıca, erken görevler son görevlerden önce geldiğinde ve erken görevler bitiş zamanlarının monoton artan sırasında çizelgelendiğinde keyfi bir çizelge **doğal biçime(canonical form)** keyfi bir çizelgeye her zaman dönüştürebileceğimizi iddia ederiz . Böyle yapabilmek için, çizelgeyi erken-ilk biçime dönüştürürüz. Sonra, $d_j < d_i$ olacak şekilde çizelgede sırasıyla k ve $k + 1$ bitiş zamanlı iki erken görev a_i ve a_j varsa, a_i ve a_j nin konumlarını değiştiririz. Yer değiştirmeden önce a_j erken olduğundan, $k + 1 \leq d_j$ dir. Bu nedenle, $k + 1 \leq d_i$ dir ve böylece yer değiştirmeden sonra hala a_i erkendir. a_j görevi çizelgede daha erkene alındığı için, yer değiştirmeden sonra erken olarak kalır.

Bir optimal çizelge için arama böylece optimal çizelgede erken olarak atayabildiğimiz görevlerin bir A kümesini bulmaya indirgenir. A yı belirlemede, A kümesinin elemanlarının bitiş zamanlarına göre monoton artan sırada listelenmesiyle asıl çizelgeyi oluşturabiliriz, sonra herhangi bir sırada son görevler (yani, $S - A$) listelenir, optimal çizelgenin bir doğal sıralaması üretilir.

Eğer hiçbir görev son olmayacak şekilde görevler için bir çizelge var ise, görevlerin bir A kümesi **bağımsızdır(independent)** deriz. Açık bir şekilde, bir çizelgenin erken görevlerinin kümesi, görevlerin bağımsız bir kümesini oluşturur. Görevlerin bütün bağımsız kümelerinin kümesini $\mathcal{I}$ olarak gösterelim.

Görevlerin verilen bir A kümesinin bağımsız olup olmadığını tanımlama problemini dikkate alalım. $t = 0, 1, 2, \dots, n$ için, $N_t(A)$ bitiş zamanı t veya daha erken olan A daki görevlerin sayısını gösterebiliriz. Herhangi bir A kümesi için $N_0(A) = 0$ olarak dikkate alın.

Lemma 16.12

A görevlerin herhangi bir kümesi için, aşağıdaki durumlar denktir.

1. A kümesi bağımsızdır.
2. $t = 0, 1, 2, \dots, n$ için, $N_t(A) \leq t$ vardır.
3. A daki görevler bitiş zamanlarına göre monoton artan sırada çizelgelenirse, hiçbir görev son olmaz.

Kanıt (1) gerektirir (2) yi göstermek için, karşıt tersini kanıtlarız: her hangi bir t için $N_t(A) > t$ ise, A kümesi için son görev olmayan bir çizelge yapmanın yolu yoktur, çünkü t adetten daha fazla görev t zamandan önce bitmelidir. Bu nedenle (1) gerektirir (2) dir. Eğer (2) sağlanırsa (3) vardır: (2), i . en büyük bitiş zamanının en azından i olduğunu gösterdiğinden, görevler bitiş zamanlarına göre monoton artan sırada çizelgelendiğinden burada problem çıkaracak bir durum yoktur. Sonuç olarak (3) aşık bir şekilde (1) i gerektirir. ■

Lemma 16.12 nin 2. özelliğini kullanarak görevlerin verilen bir kümesinin bağımsız olup olmadığını kolayca hesaplayabiliriz. (Alıştırma 16.5-2 ye bakınız.)

Son görevlerin cezalarının toplamını minimum yapma problemi erken görevlerin cezalarının toplamını maksimum yapma problemi ile aynıdır. Aşağıdaki teorem böylece maksimum toplam cezalı görevlerin bağımsız bir A kümesini bulmak için açgözlü algoritmayı kullanabildiğimizi kesinleştirir.

Teorem 16.13

Eğer S bitiş zamanlarına sahip birim-zaman görevlerinin bir kümesi ve $\mathcal{I}$ görevlerin tüm bağımsız kümelerinin kümesi ise o zaman $(S, \mathcal{I})$ sistemi bir matroidtir.

Kanıt Görevlerin bağımsız bir alt kümesinin her alt kümesi kesinlikle bağımsızdır. Özellik değiştirmeyi kanıtlamak için B ve A nın görevlerin bağımsız kümeleri olduğunu ve $|B| > |A|$ olduğunu kabul edelim. k , $N_t(B) \leq N_t(A)$ olacak şekilde en büyük t olsun. $N_0(A) = N_0(B) = 0$ olduğundan böyle bir t değeri vardır.) $N_n(B) = |B|$ ve $N_n(A) = |A|$, fakat $|B| > |A|$ olduğundan $k < n$ olduğuna ve $k + 1 \leq j \leq n$ aralığındaki tüm j ler için $N_j(B) > N_j(A)$ olduğuna sahip olmalıyız. Bu nedenle, B, A dan $k + 1$ bitiş zamanlı olan daha fazla görevlere sahiptir. a_i , $B-A$ daki $k + 1$ bitiş zamanlı bir görev olsun. $A' = A \cup \{a_i\}$ olsun.

Biz şimdi Lemma 16.12 nin 2. özelliğini kullanarak, A' nün bağımsız olmak zorunda olduğunu gösterelim. $0 \leq t \leq k$ için, $N_t(A') = N_t(A) \leq t$ ye sahibiz, çünkü A bağımsızdır. $k < t \leq n$ için $N_t(A') \leq N_t(B) \leq t$ sahibiz, çünkü B bağımsız. Bu nedenle, A' bağımsızdır, $(S, \mathcal{I})$ nın bir matroid olduğu ispatımız tamamlanır. ■

Teorem 16.11 ile, A kümesinin görevlerinin maksimum-ağırlıklı bağımsız bir kümesini bulmak için bir açgözlü algoritma kullanabiliriz. Sonra, A nın erken görevlerinde olduğu gibi A daki görevleri içeren bir optimal çizelge oluşturabili-

	Görev						
a_i	1	2	3	4	5	6	7
d_i	4	2	4	3	1	4	6
w_i	70	60	50	40	30	20	10

Şekil 16.7 Tek işlemci için bitiş zamanlarının ve cezaları olan birim-zamanlı görevlerin çizelgelenmesi probleminin bir örneği.

riz. Bu yöntem, tek işlemci için bitiş zamanlarının ve cezaları olan birim-zaman görevlerinin çizelgelenmesi için verimli bir algoritmadır. GREEDY i kullanarak çalışma zamanı $O(n^2)$ dir, çünkü bu algoritma tarafından yapılan $O(n)$ bağımsız kontrollerinin herbiri $O(n)$ zaman alır.(Alıştırma 16.5-2 bakınız.). Problem 16-4 daha hızlı bir uygulama vermektedir.

Şekil 16.7, tek işlemci için bitiş zamanlarının ve cezaları olan birim-zaman görevlerinin çizelgelenmesi probleminin bir örneğini gösterir. Bu örnekte, açgözlü algoritma sırasıyla a_1, a_2, a_3 ve a_4 görevlerini seçer, sonra a_5 (çünkü $N_4(\{a_1, a_2, a_3, a_4, a_5\}) = 5$) tir.) ve a_6 yı $N_4(\{a_1, a_2, a_3, a_4, a_6\}) = 5$) reddeder ve son olarak a_7 yi kabul eder. Son optimal çizelge $w_5 + w_6 = 50$ toplam ceza değerine sahip

$$\langle a_2, a_4, a_1, a_3, a_7, a_5, a_6 \rangle$$

dır.

Alıştırmalar

16.5-1

Şekil 16.7de verilen çizelgeleme probleminin bir örneğini çözünüz, ama her w_i cezasını $80 - w_i$ ile yer değiştiriniz.

16.5-2

Görevlerin verilen bir A kümesinin bağımsız olup olmadığını $O(|A|)$ zamanda saptamak için Lemma 16.12 nin 2. özelliğinin nasıl kullanıldığını gösteriniz.

Problemler

16-1 Madeni para değişimi

En az sayıda madeni parayı kullanarak n sent için değişim(bozdurma) yapma problemini düşünelim. Her bir madeni paranın değerinin bir tam sayı olduğunu kabul edelim.

- Yirmi beş sent, on beş sent, beş sent ve sentlerden oluşan değişimi(bozdurma) yapmak için bir açgözlü algoritma tanımlayınız. Algoritmanızın bir optimal çözüm oluşturduğunu kanıtlayınız.

- b. Mevcut madeni paralar, c nin kuvvetleri olan değer birimlerindendir, yani, değer birimleri bazı $c > 1$ ve $k \geq 1$ tam sayıları için $c^0, c^1, \dots, c^k$ dir. Açgözlü algoritmanın her zaman bir optimal çözüm oluşturduğunu gösteriniz.
- c. Açgözlü algoritmanın bir optimal çözümü vermediği madeni para değer birimlerinin bir kümesini yazınız. Bu küme bir sent içermelidir öyle ki n nin her değeri için bir çözüm vardır.
- d. Madeni paraların birinin bir sent olduğu kabulüyle k farklı madeni para gösterimlerinin herhangi bir kümesi için değişim(bozdurma) yapan $O(nk)$ -zamanlı bir algoritma yazınız.

16-2 Ortalama tamamlama süresini minimum yapmak için çizelgeleme

Görevlerin bir $S = \{a_1, a_2, \dots, a_n\}$ kümesinin verildiğini kabul edelim, burada a_i görevini tamamlamak için p_i birim işlem zamanı gerekir. Bu görevleri çalıştırmak için bir bilgisayara sahipsiniz ve bilgisayar bir zamanda sadece bir görev çalıştırabilmektedir. c_i , a_i görevinin **tamamlama süresi(completion time)** olsun, yani, a_i görevinin işlemini tamamladığı süre. Amacınız ortalama tamamlama süresini minimum yapmaktır, yani, $(1/n) \sum_{i=1}^n c_i$ minimum yapmaktır. Örneğin, $p_1 = 3$ ve $p_2 = 5$ olan iki görev a_1 ve a_2 olsun ve a_1 tarafından izlenen, a_2 nin birinci olarak çalıştığı çizelgeyi düşünelim. Sonra $c_2 = 5$, $c_1 = 8$ ve ortalama tamamlama süresi $((5+8))/2 = 6.5$ tur. Eğer a_1 görevi birinci olarak çalışırsa, ancak, $c_1 = 3$, $c_2 = 8$ dir ve ortalama tamamlama süresi $((3 + 8))/2 = 5.5$ tur.

- a. Ortalama tamamlama süresini minimum yapacak şekilde görevleri çizelgeleme yapan bir algoritma yazınız. Her bir görev öncelikli olmayan şekilde çalışmalıdır, yani, a_i görevi bir defa başladığında p_i birim zaman sürekli bir şekilde çalışmalıdır. Algoritmanızın ortalama tamamlama süresini minimum yaptığını ispatlayınız ve algoritmanızın çalışma zamanını oluşturunuz.
- b. Görevlerin aynı anda uygun olmadığını kabul edelim. Yani her bir görev **serbest bırakma zamanına(release time)** r_i ye kadar başlayamaz. **Önalıma(pre-emption)** da izin verdiğimizizi kabul edelim böylece bir görev askıya alınabilir ve daha geç bir zamanda yeniden başlayabilir. Örneğin, $p_i = 6$ işlem süresi ve $r_i = 1$ serbest bırakma zamanına sahip bir a_i görevi 1 zamanında çalışmaya başlayabilir ve 4 zamanında önalınabilir. Sonra 10 zamanında yeniden başlayabilir ama 11 zamanında önalınabilir, ve son olarak 13 zamanında yeniden başlayabilir ve 15 zamanında tamamlanır. a_i görevi toplam 6 birim sürede çalıştı ancak çalışma zamanı üç parçaya ayrılmıştır. Bu senaryoda, a_i nin tamamlama zamanı 15 tir. Bu yeni senaryoda ortalama tamamlama süresini minimum yapmak için görevleri çizelgeleyen bir algoritma yazınız. Algoritmanızın ortalama tamamlama süresini minimum yaptığını kanıtlayınız ve algoritmanızın çalışma zamanını oluşturunuz.

16-3 Çevrimsiz altçizgeler

- a. Bir yönsüz $G = (V, E)$ çizgesi için **sıklık matrisi(incidence matrix)**, e ayrıtı v tepesi üzerine gelen ise $M_{ve} = 1$, aksi halde $M_{ve} = 0$ olacak şekilde bir $|V| \times |E|$ boyutlu, M matrisidir. M nin sütunlarının bir kümesi mod 2 ye göre tam sayılar cismi üzerinde lineer bağımsızdır gerek ve yeter koşul ayrıtların uygun kümesi çevrimsizdir. Sonra, (a) kısmındaki $(E, \mathcal{I})$ nin bir matroid olduğunun alternatif kanıtını sağlamak için Alıştırma 16.4-2 nin sonucunu kullanınız.
- b. Bir yönsüz $G = (V, E)$ çizgesinde her bir ayrıtı, negatif olmayan bir $w(e)$ ağırlığı ile ilişkilendirdiğimizi kabul edelim. E nin maksimum toplam ağırlıkta çevrimsiz bir alt kümesini bulmak için verimli bir algoritma yazınız.
- c. $G = (V, E)$ keyfi bir yönlü çizge olsun ve $(E, \mathcal{I})$ şöyle tanımlansın: $A \in \mathcal{I}$ dir ancak ve ancak A herhangi bir yönlü çevrim içermez. Birleşmeli $(E, \mathcal{I})$ sistemi bir matroid olmayacak şekilde, G yönlü çizgesinin bir örneğini veriniz. Bir matroid için tanımlanan koşullardan hangisini sağlamadığını belirtiniz.
- d. Kendi döngüsü olmayan yönlü bir $G = (V, E)$ çizgesi için sıklık matrisi, bir $|V| \times |E|$ boyutlu, M matrisidir öyleki eğer e ayrıtı v tepesinden ayrılıyor ise $M_{ve} = -1$ dir yada e ayrıtı v tepesine giriyorsa $M_{ve} = 1$ aksi halde $M_{ve} = 0$ dir. M nin sütunlarının bir kümesi lineer bağımsız ise ayrıtlara karşılık gelen kümesinin yönlü bir çevrimi içermediğini tartışınız.
- e. Alıştırma 16.4-2 bize herhangi bir M matrisinin sütunlarının lineer bağımsız kümelerinin bir kümesinin bir matroid olduğunu söyler. (d) ve (e) kısımlarının sonuçlarının niçin birbiriyle çelişmediğini dikkatlice açıklayınız. Çevrimsiz olan ayrıtların bir kümesinin kavramı ile lineer bağımsız olan sıklık matrisinin sütunlarının birleşmeli kümesi kavramı arasında mükemmel bir bağlantı nasıl kurulabilir?

16-4 Varyasyonları çizelgeleme

Bitiş zamanı ve cezalara sahip birim-zaman görevlerinin çizelgelenmesinin kısım 16.5 teki problemi için aşağıdaki algoritmayı göz önünde bulunduralım. n tane dilim zamanın hepsi başlangıçta boş olsun, burada i . dilim zaman, i zamanda biten birim- uzunluklu dilim zamandır. Görevlerin cezalara göre monoton azalan sırada olduğunu düşünelim. a_j görevini dikkate aldığımızda, eğer hala boş olan a_j nin bitiş zamanı d_j de ve ya bundan önce bir dilim zaman varsa, a_j bu en son dilim zaman atanır böylece bu dilim zaman doldurulur. Eğer hiç böyle bir dilim zaman yoksa a_j görevini henüz doldurulmamış dilim zamanların en sonuncusuna ata.

- a. Bu algoritmanın her zaman bir optimal cevabı verdiğini tartışınız.
- b. Algoritmayı verimli bir şekilde uygulamak için Kısım 21.3 te gösterilen hızlı ayrık-küme ormanını kullanın. Girdi olarak verilen görevlerin kümesinin

önceden cezalarına göre monoton azalan sırada sıralandığını kabul edelim. Uygulamanızın çalışma zamanını analiz edin.

16-5 Çevrimdışı önbelleğe alma

Modern bilgisayarlar bir hızlı bellekte küçük miktarda bir veriyi depolamak için bir önbellek kullanır. Bir program, büyük miktarda veriye ulaşabilmesine rağmen tüm erişim zamanını, küçük fakat daha hızlı bir bellek olan **önbellekteki(cache)** ana belleğin bir alt kümesinde tutarak büyük oranda azaltabilir. Bir bilgisayar programı çalıştığı zaman, n tane bellek isteğinin bir $\langle r_1, r_2, \dots, r_n \rangle$ dizisi oluşur, burada her istek, özel bir veri elemanı içindir. Örneğin, 4 farklı $\{a, b, c, d\}$ elemanına ulaşan bir program, $\langle d, b, d, b, d, a, c, d, b, a, c, b \rangle$ istekler dizisini oluşturabilir. Önbelleğin boyutu k olsun. Ön bellek k eleman içerdiğinde ve program $(k + 1)$. elemanı istediğinde, sistem, bu ve önbelleğinde k tane eleman içeren her bir alt dizi isteği için karar vermelidir. Daha doğrusu, her bir r_i isteği için önbellek-yönetimli algoritma r_i elemanını önceden bellekte olup olmadığını kontrol eder. Eğer varsa, bir **önbelleğe rastlama yüzdesine(cache hit)** sahibizdir; aksi halde bir **önbelleğe rastlamama yüzdesine(cache miss)** sahibizdir. Bir önbelleğe rastlamama yüzdesine göre, sistem ana bellekten r_i yi alır ve önbellek-yönetimli algoritma önbellekte r_i yi tutup tutmayacağına karar vermelidir. Eğer r_i yi tutmaya karar verirse ve önbellek çoktan k elemanı bulunduruyorsa r_i ye yer açmak için bir elemanı çıkarmalıdır. Önbellek-yönetimli algoritma isteklerin bütün dizisi üzerinden önbelleğe rastlama yüzdesinin sayısını minimum yapmak amacı ile verileri çıkarır.

Genellikle, önbelleğe alma bir çevirim-içi problemdir. Yani, gelecek istekleri bilmeksizin önbellekteki hangi veriyi tutmaya karar vermek zorundayız. Ancak, burada, bu problemin çevrimdışı versiyonunu düşünürüz, bu problemde önbelleğin boyutu k ve n tane isteğin bütün dizisi önceden verilir ve önbelleğe rastlamama yüzdesinin toplam sayısını minimum yapmak isteriz.

Bu çevrim-dışı problemi, **en uzak gelecek(furthest-in-future)** olarak adlandırılan bir açgözlü stratejisi ile çözebiliriz. Bu strateji, istek dizisindeki bir sonraki erişimin gelecekte en uzakta gelen önbellekteki bir elemanı çıkarmayı tercih eder.

- En uzak gelecek stratejisini kullanan bir önbellek yöneticisi için sözde kod yazınız. Girdi isteklerinin bir $\langle r_1, r_2, \dots, r_n \rangle$ dizisi ve bir önbelleğin boyutu k olmalıdır ve çıktı, her bir isteğe göre hangi veri elemanını çıkarılacağı hakkındaki kararların bir dizisi olmalıdır. Algoritmanızın çalışma zamanı nedir?
- Çevrim-dışı önbelleğe alma probleminin optimal altyapıyı sergilediğini gösteriniz.
- En uzak geleceğin, önbelleğe rastlamama yüzdesinin minimum olası sayısını ürettiğini kanıtlayınız.

Bölüm notları

Ağgözlü Algoritma ve Matroidler üzerine daha fazla materyal Lawler[224], Papadimitriou ve Steiglitz [271] de bulunabilir.

Matroidler Teorisi Whitney [355] tarafından 1935 te yazılan bir makaleden eskiye dayanmasına rağmen, ağgözlü algoritma ilk olarak Edmonds[101] tarafından yazılan 1971 deki Kombinatorik optimizasyon notlarında yer alır.

Etkinlik-seçim problemi için ağgözlü algoritmanın doğruluğu kanıtımız Gavril [131] dayanır. Görev-çizelgeleme problemi Lawler[224]; Horowitz, Sahni, ve Rajasekaran [181]; ve Brassard ve Bratley[54] de çalışılmıştır.

Huffman kodları 1952 [185] de bulundu; Lelewer ve Hirschberg[231] veri-sıkıştırma tekniklerini 1987 tarihinde araştırdılar.

Matroid teorisinin bir genişlemesi olan Greedoid teori, burada sunulan teoremi geniş ölçüde genelleştiren Korte ve Lovaz [216, 217, 218, 219] tarafından öncülük edildi.

Bir **amortize analizde**, bir veri yapısı işlemi gerçekleştirmenin tüm işlemler içinde ortalama ne kadar zaman gerektireceğini buluruz. Amortize analizle, her ne kadar bir dizinin içindeki bir işlem pahalı olsa bile, eğer o işlemler dizisinin ortalamasını alırsak, bir işlemin ortalama maliyetinin düşük olduğunu gösterebiliriz. Amortize analizi ortalama analizinden olasılıkları içermemesiyle ayrılır; bir amortize analizi *en kötü durumda her işlem için ortalama performansı* ortaya koymayı garanti eder.

Bu bölümdeki ilk üç kısım amortize analizde en çok kullanılan üç yöntemi işlemektedir. Kısım 17.1, n işlemlili bir dizinin toplam maliyetinde bir $T(n)$ üst sınır belirlediğimiz toparlama analiziyle (aggregate analysis) başlar. Bu durumda işlem başına ortalama maliyet $T(n) = n$ dir. Her bir işlem için ortalama maliyeti amortize maliyet olarak alırız ki bütün işlemlerin amortize maliyetleri aynı olsun.

Kısım 17.2, her bir işlemin amortize maliyetini belirlediğimiz hesaplama yöntemini işler. Birden fazla çeşitte işlem varsa, her bir çeşit işlemin farklı amortize maliyeti olabilir. Hesaplama yöntemi dizinin başlarında bazı işlemlerin fiyatını yükseltir ve böylece kendine veri yapısındaki belirli nesneler için ön ödemeli bir kredi yaratmış olur. Bu kredi, dizinin ilerleyen kısımlarında maliyetlerinden daha az miktarla karşılanan işlemlerin eksiklerini kapatır.

Kısım 17.3, hesaplama yöntemi gibi her işlemin amortize maliyetini belirleyip, dizinin ilerisinde gerekebilecek denkleştirme için dizi başında bazı işlemlere yüksek fiyat biçebileceğimiz potansiyel yöntemidir (potential method). Potansiyel yöntem bu krediyi veri yapısının “potansiyel enerji”si gibi görür ve yapı içindeki bireysel nesnelere paylaştırmadan bir bütün olarak saklar.

Bu üç tekniği açıklamak için iki örnek kullanmalıyız. Bunlardan biri ek bir MULTIPOP işlemi olan bir yığındır ve bu POP işlemi tek seferde birçok nesneyi çıkarır. Diğerisi ise tek bir INCREMENT işlemi ile 0 dan başlayarak ikili değerleri sayan bir sayaçtır.

Bu bölümü okurken, bir amortize analiz esnasında kullanılan ücretlerin sadece analiz amaçlı kullanıldığını aklınızda tutun. Kodda bulunmalarına gerek yoktur – ve bulunmamalıdır da. Örneğin, hesaplama yöntemini kullanırken x nesnesine bir kredi atarsak, kodun içersinde $x.credit$ şeklinde uygun bir değer atamak zorunda değiliz.

Amortize analizi gerçekleştirirken, sıklıkla belirli bir veri yapısına dair bir içgörü sahibi oluruz ve bu içgörü tasarımı en iyilememize yardım edebilir. Örneğin, Kısım 17.4 te, dinamik olarak genişleyen ve daralan bir tabloyu analiz etmek için potansiyel yöntemi kullanabiliriz.

17.1 Toparlama analizi

Toparlama analizinde, her n için, n li bir işlemler dizisinin toplamda $T(n)$ en kötü durum zamanı aldığını gösteriyoruz. Dolayısıyla, en kötü durumda, işlem başına ortalama maliyet ya da *amortize maliyet* $T(n) = n$ dir. Bu **amortize maliyetin**, dizide farklı türden işlemler olsa bile, her bir işlem için geçerli olduğunu unutmayın. Bu bölümde işleyeceğimiz diğer iki yöntem olan hesaplama yöntemi ve potansiyel yöntem farklı türlerdeki işlemlere farklı amortize maliyetler atayabilir.

Yığın İşlemleri

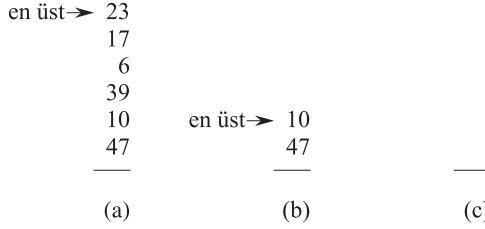
Toparlama analizindeki ilk örneğimizde, yeni bir işlemle genişletilmiş yığınları analiz edeceğiz. Kısım 10.1 her biri $O(1)$ zaman alan iki temel yığın işlemini tanıtmıştı:

PUSH(S, x) x nesnesini S yığına iter.

POP(S) S yığınının tepesini açar ve ortaya çıkan nesneyi getirir. Boş bir yığına POP işlemi uygulamak hata verir.

Bu işlemlerin her biri $O(1)$ zaman aldığı için, her birinin maliyetinin 1 olduğunu düşünelim. Dolayısıyla, n sayıda PUSH ve POP işlemler dizisinin toplam maliyeti n ve bu n işlem için gerçek çalışma zamanı $\Theta(n)$ olacaktır.

Burada bir yığın işlemi olan MULTIPOP(S, k) yı ekliyoruz. Eğer yığında k dan az sayıda nesne varsa, bu işlem bütün yığını açarak S yığınının en üstündeki k sayıda nesneyi kaldırır. Elbette, k nın pozitif olduğunu varsayıyoruz; aksi halde, MULTIPOP işlemi yığını olduğu gibi bırakır. Aşağıdaki kabakodda, yığında hali-hazırda hiç nesne yoksa STACK-EMPTY işlemi TRUE, aksi halde FALSE döndürecek.



Şekil 17.1 S yığınındaki bir MULTIPOP işlemi en başta (a) da gösterilmiştir. En üstteki 4 nesne MULTIPOP ($S, 4$) tarafından açılmış ve sonuç (b) de gösterilmiştir. Bir sonraki işlem, 7 den az sayıda nesne kaldığı için yığını boşaltan, MULTIPOP ($S, 7$) dir ve (c) de gösterilmiştir

MULTIPOP (S, k)

- 1 **while** not STACK-EMPTY (S) and $k > 0$
- 2 POP (S)
- 3 $k = k - 1$

Şekil 17.1 MULTIPOP ın bir örneğini gösterir.

S nesnelli bir yığında MULTIPOP (S, k) işleminin çalışma zamanı nedir? Gerçek çalışma zamanı, gerçekleştirilmiş POP işlemleri sayısı ile doğrusaldır ve dolayısıyla MULTIPOP u her bir PUSH ve POP için 1 soyut maliyeti (abstract cost) üzerinden analiz edebiliriz. **While** döngüsü sırasındaki iterasyon sayısı yığından çıkarılan nesnelerin min (s, k) sayısıdır. Dolayısıyla, MULTIPOP un toplam maliyeti min (s, k) dır ve gerçek çalışma zamanı bu maliyetin doğrusal bir fonksiyonudur.

n sayıda PUSH, POP ve MULTIPOP işlemlerini başlangıçta boş olan bir yığın üzerinde analiz edelim. Dizideki bir MULTIPOP işleminin en kötü durum maliyeti, yığın en fazla n büyüklüğünde olduğuna göre, $O(n)$ dir. Dolayısıyla herhangi bir yığın işleminin en kötü durum zamanı $O(n)$ dir ve dolayısıyla, her biri $O(n)$ tutan $O(n)$ MULTIPOP işlemi yaptığımızı göre, n işlemli bir dizinin maliyeti $O(n^2)$ dir. Her ne kadar bu analiz doğru olsa da, her bir işlemin bireysel olarak en kötü durum zamanını göz önünde bulundurarak elde ettiğimiz bu $O(n^2)$ sonucu kesin olarak sıkı değildir.

Toparlama analizini kullanarak, n li işlem dizisinin tamamını hesaba katan daha iyi bir üst sınır elde ederiz. Aslında, tek bir MULTIPOP işlemi pahalı olabileceği de, başlangıçta boş olan bir yığın üzerinde uygulanacak herhangi bir n sayıda PUSH, POP ve MULTIPOP işlemler dizisi en fazla $O(n)$ a mal olabilir. Neden? Bir nesneyi yığına her bir sokuşumuz için en fazla bir kere çıkarabiliriz. Dolayısıyla, MULTIPOP içindeki çağrılar da dahil olmak üzere, boş olmayan bir yığında yapılabilecek POP çağrılarının sayısı en fazla PUSH işlemi sayısı kadar olabilir ki bu da n dir. n nin tüm değerleri için, tüm n PUSH, POP ve MULTIPOP işlemler dizisi toplamda $O(n)$ zaman alır. Bir işlemin ortalama maliyeti $O(n)/n = O(1)$ dir.

Toparlama analizinde, her işlemin amortize maliyetini ortalama maliyet olarak alırız. Bu örnekte, dolayısıyla, üç yığın işleminin de amortize maliyeti $O(1)$ dir.

Her ne kadar bir yığın işleminin ortalama maliyeti ve dolayısıyla çalışma zamanının $O(1)$ olduğunu göstermiş olsak da şunu vurgulamak isteriz ki olasılıksal akıl yürütmeyi (probabilistic reasoning) kullanmadık. Aslında n işlemli bir dizide $O(n)$ in en kötü durum sınırını gösterdik. Bu maliyeti n e bölmek, bize işlem başına ortalama ya da amortize maliyeti verdi.

Bir ikili sayacı artırma

Toparlama analizinin bir diğer örneği olarak, 0 dan başlayarak sayan bir k -bit ikili sayacını uygulama problemini düşünelim. $A.length = k$ olduğunda bir $A[0 \dots k-1]$ bit dizisini sayaç olarak kullanalım. Sayaçta bulunan x ikili sayısının (binary number) en düşük-sıralı biti $A[0]$ dadır ve en yüksek-sıralı biti $A[k-1]$ dadır. Öyle ki: $x = \sum_{i=0}^{k-1} A[i] \cdot 2^i$. Başlangıçta, $x = 0$ ve dolayısıyla $i = 0, 1, \dots, k-1$ için $A[i] = 0$. Sayaçtaki değere 1 ekleyin (mod 2^k), aşağıdaki prosedürü kullanınız.

INCREMENT(A)

```

1   $i = 0$ 
2  while  $i < A.length$  and  $A[i] == 1$ 
3       $A[i] = 0$ 
4       $i = i + 1$ 
5  if  $i < A.length$ 
6       $A[i] = 1$ 
```

Şekil 17.2, 0 dan başlayıp 16 ya kadar bir ikili sayacı 16 kere artırdığımızda ne olduğunu göstermektedir. 2-4 üncü satırlardaki while döngüsünün her iterasyonunun başlangıcında i pozisyonuna 1 eklemek istiyoruz. Eğer $A[i] = 1$ ise, o zaman 1 eklemek biti i pozisyonunda 0 a çevirir ve döngünün bir sonraki iterasyonunda $i + 1$ pozisyonuna eklenmek üzere 1 artan getirir. Aksi halde, döngü sona erer ve sonra eğer $i < k$ ise, $A[i] = 0$ olduğunu biliriz, öyle ki, 6. Satır i pozisyonuna bir 1 ekleyerek 0 ı 1 e çevirir. Her bir INCREMENT işleminin maliyeti çevrilen bitlerin sayısında doğrusaldır.

Yığın örneğinde olduğu gibi, baştan savma bir analiz doğru olan ama kesin olmayan sonuçlar doğurur. Tamamı 1 lerden oluşan bir A dizisinde INCREMENT işleminin bir kere uygulanması en kötü durumda $\Theta(k)$ zaman alır. Dolayısıyla, 0'dan başlayan bir sayaçta uygulanacak bir INCREMENT işlemler dizisi en kötü durumda $O(nk)$ zaman alacaktır.

Her INCREMENT çağrısında bütün bitlerin çevrilmediğini gözlemleyerek bir n sayıda INCREMENT işlemi dizisi için bir $O(n)$ en kötü durum maliyeti verecek şekilde analizimizi pekiştirebiliriz. Şekil 17.2 nin gösterdiği gibi, $A[0]$, her INCREMENT çağrısında çevrilmektedir. Sıradaki bit $A[1]$ ise her diğer seferde çev-

Sayaç değeri	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	Toplam maliyet
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

Şekil 17.2 Değeri 18 INCREMENT işlemiyle 0 dan 16 ya çıkan bir 8 bit ikili sayacı. Bir sonraki değeri gerçekleştirmek için çevrilen bitler gölgelendirilmiştir. Bitleri çevirmek için gereken işlem maliyeti sağda gösterilmiştir. Toplam maliyetin her zaman yapılan INCREMENT işleminin iki katından küçük olduğuna dikkat ediniz.

riilmektedir: Sıfırdan başlayan bir sayaçtaki n INCREMENT işlem sayılı bir dizi $A[1]$ ın $\lfloor n/2 \rfloor$ sefer çevrilmesine sebep olmaktadır. Benzer şekilde, $A[2]$ biti sadece her dört seferde ya da başka bir deyişle, dizide $\lfloor n/4 \rfloor$ kere çevrilmektedir. Genel olarak, $i = 0, 1, \dots, k-1$ için, $A[i]$ biti sıfırdan başlayan bir sayaçtaki n INCREMENT işlemli bir dizide $\lfloor n/2^i \rfloor$ kere çevrilmektedir. $i \geq k$ için $A[i]$ biti bulunmamaktadır ve dolayısıyla çevreilemez. Dolayısıyla, dizideki toplam çevrilme sayısı

$$\sum_{i=0}^{k-1} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} = 2n,$$

denklemindeki gibi olacaktır (A.6). Dolayısıyla, sıfırdan başlayan bir sayaçtaki n sayıda INCREMENT işleminden oluşan bir dizi için en kötü durum zamanı $O(n)$ dir. Her işlemin ortalama maliyeti ve dolayısıyla amortize maliyeti ise $O(n)/n = O(1)$ dir.

Alıştırmalar

17.1-1

Eğer yığın işlemleri kümesinde k sayıda nesneyi yığına sokan bir MULTIPUSH işlemi olsaydı, yığın işlemlerindeki amortize maliyet sınırı $O(1)$ hala geçerli olacak mıydı?

17.1-2

k -bit sayaç örneğine bir DECREMENT işlemi dahil edilseydi, n işlemin en fazla $\Theta(nk)$ zaman alabileceğini gösteriniz.

17.1-3

Bir veri yapısı üzerinde n işlemli bir dizi uyguladığımızı düşünün. Bu dizinin i inci işleminin maliyeti, i eğer 2 nin tam kuvvetiyse i , değilse 1 olsun. İşlem başına amortize maliyeti belirlemek için toparlama analizini kullanın.

17.2 Hesaplama yöntemi

Amortize analizinin **hesaplama yöntemi**nde, farklı işlemlere farklı fiyatlar atıyoruz ve bu operasyonlardan bazıları gerçek değerlerinden daha yüksek ya da düşük fiyatlar alıyorlar. Bir işleme atadığımız fiyata o işlemin **amortize maliyeti** diyoruz. Bir işlemin amortize maliyeti gerçek maliyetini geçtiğinde, aradaki farklı veri yapısındaki belli nesnelere **kredi** olarak atıyoruz. Kredi daha sonraki işlemlerin ödemesinde, amortize maliyet gerçek maliyetin altında olduğunda yardımcı olabilir. Dolayısıyla, bir işlemin amortize maliyetini işlemin gerçek maliyetiyle kullanılmış ya da saklanmış kredisi arasında paylaştırılıyor gibi düşünebiliriz. Farklı işlemlerin farklı amortize maliyetleri olabilir. Bu yöntem bütün işlemlerin aynı amortize maliyete sahip olduğu toparlama analizinden farklıdır.

İşlemlerin amortize maliyetlerini dikkatle seçmeliyiz. Amortize maliyetleri analiz ederek en kötü durumda işlem başına ortalama maliyetin küçük olduğunu göstermek istiyorsak, işlemler dizisinin toplam amortize maliyetinin, dizinin toplam gerçek maliyeti üzerinde bir üst sınır belirleyebildiğinden emin olmalıyız. Dahası, toparlama analizindeki gibi, bu ilişki bütün işlemler dizileri için geçerli olmalıdır. i inci işlemin gerçek maliyetine c_i ve i inci işlemin amortize maliyetine $\hat{c}_i$ dersek, n işlemli tüm diziler için şu denkleme ihtiyaç duyarız:

$$\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i. \quad (17.1)$$

Veri yapısında saklanan toplam kredi toplam amortize maliyet ve toplam gerçek maliyet arasındaki farka, ya da başka bir deyişle, $\sum_{i=1}^n \hat{c}_i - \sum_{i=1}^n c_i$ e eşittir.

Eşitsizlikle (17.1), veri yapısına ilişkin toplam kredi tüm şartlarda negatif olmayan bir değer olmalıdır. Toplam kredinin negatif olmasına olanak sağlasaydık (daha sonra ödeme yapmak üzere altta değişen ilk işlemlerin sonucu), bu şartlarda oluşan toplam amortize maliyet oluşan toplam gerçek maliyetin altında olurdu; o ana kadar olan işlemler dizisi için, toplam amortize maliyet toplam gerçek maliyet üzerinde bir üst sınır olamazdı. Dolayısıyla, veri yapısındaki toplam kredinin asla negatif olmamasına özen göstermeliyiz.

Yığın işlemleri

Amortize analizinin hesaplama yöntemini açıklamak için yığın örneğine dönelim. k nın MULTIPOP için sağlanan değişken ve s nin bu çağrıdaki yığın boyutu olduğunu düşündüğümüzde, işlemlerin gerçek maliyetlerinin şöyle olduğunu hatırlayın:

PUSH	1 ,
POP	1 ,
MULTIPOP	$\min(k, s)$.

Aşağıdaki amortize maliyetleri atayalım:

PUSH	2 ,
POP	0 ,
MULTIPOP	0 .

Gerçek maliyet değişkenken, MULTIPOP nın amortize maliyetinin bir sabit (0) olduğuna dikkat edin. Burada, her üç amortize maliyet de sabittir. Genel olarak, ele alınan işlemlerin amortize maliyetleri birbirinden farklı olabilir ve hatta asimptotik olarak da farklı olabilirler.

Şimdi amortize maliyetleri fiyatlandırarak herhangi bir yığın işlemini dizisini ödeyebileceğimizi göstermeliyiz. Her birim maliyeti 1 dolarla temsil ettiğimizi düşünelim. Kesim 10.1 deki, yığın veri yapısı ve bir kafeteryadaki tabak yığını arasındaki benzerliği hatırlayalım. Yığına bir tabak ittiğimizde, itmenin gerçek maliyetini ödemek için 1 dolar kullanıyoruz ve elimizde 1 dolarlık kredi kalıyor (toplamda yapılan 2 dolarlık fiyatlandırma üzerinden) ve bu krediyi tabağın üzerinde bırakıyoruz.

Tabak üzerinde saklanan dolar onu yığından çıkarmanın önden ödenen bedeli olarak işe yarar. Bir POP işlemi gerçekleştirdiğimizde, işlemi ücretlendirmiyoruz ve gerçek maliyetini yığında saklanan krediden ödüyoruz. Bir tabağı çıkarmak için, tabaktaki 1 dolarlık krediyi alıyoruz ve gerçek maliyeti ödemek için kullanıyoruz. Dolayısıyla, PUSH işlemini biraz daha yüksek fiyatlandırarak, POP işlemi için hiçbir şey ödemeyebiliriz.

Dahası, MULTIPOP işlemi için de hiçbir şey ödemeyebiliriz. İlk tabağı çıkarmak için, tabağın üzerindeki kredi dolarını alıyoruz ve POP işleminin gerçek maliyetini ödemek için kullanıyoruz. İkinci tabağı çıkarmak için, gerçek maliyeti ödemek için yine tabakta bir dolarlık bir kredimiz var ve bu böyle devam ediyor. Dolayısıyla, MULTIPOP işlemi ödemek için her zaman önden ödeme yapmış durumdayız. Bir başka deyişle, yığındaki her tabağın üzerinde 1'er dolarlık kredi olduğu ve yığında her zaman negatif olmayan bir sayıda tabak olduğu için, kredi miktarımızın asla negatif olmayacağını garanti altına almış olduk. Dolayısıyla, n sayıda PUSH, POP ve MULTIPOP işlemlerinden oluşan *herhangi bir dizi* için, toplam amortize maliyet, toplam gerçek maliyetin üzerinde bir üst sınırdır. Toplam amortize maliyet $O(n)$ olduğuna göre, toplam gerçek maliyet de öyledir.

Bir ikili sayacı artırma

Hesaplama yönteminin bir başka açıklayıcısı olarak, INCREMENT işlemini sıfırdan başlayan bir ikili sayaç üzerinde analiz edeceğiz. Daha önce gözlemlediğimiz gibi, bu işlemin çalışma zamanı çevrilen bitlerle orantılıdır ki bunu bu örneğimizde bizim maliyetimiz olarak kullanmalıyız. Bir kere daha, her maliyet birimine 1 dolarlık bir değer atayalım (bu örnek için çevrilen her bite).

Amortize analizi için, bir biti 1'e eşlemek için bir amortize maliyeti 2 dolarlık fiyatlandıralım. Bir bit eşlendiğinde, bir doları (2 doların içersinden) bitin gerçek eşlemesi için kullanırız ve diğer doları bitin üzerine daha sonra bit tekrar 0'a çevrilirken kullanmak üzere kredi olarak bırakırız. Zaman içersinde herhangi bir noktada, sayaçtaki her 1'in üzerinde bir dolarlık bir kredi vardır ve dolayısıyla bir biti tekrar 0'a çevirmek için herhangi bir ödeme yapmamıza gerek kalmaz; geri çevrim için daha önceden bıraktığımız krediyi kullanırız.

Şimdi INCREMENT'in amortize maliyetini belirleyebiliriz. **while** döngüsü içersinde birleri tekrar çevirmenin bedelini tekrar çevrilen bitlerin üzerindeki dolarlarla öderiz. INCREMENT prosedürü 6. satırdaki en fazla bir bite eşlenir ve dolayısıyla INCREMENT işleminin amortize maliyeti en fazla 2 dolar olacaktır. Sayaçtaki 1'lerin değeri asla negatife dönmez ve dolayısıyla kredi miktarı her zaman negatif olmayan bir değer olarak kalır. Dolayısıyla, n sayıda INCREMENT işlemi için, toplam amortize maliyet $O(n)$ diğer ki bu da toplam gerçek maliyeti sınırlar.

Alıştırmalar

17.2-1

Boyutun k yı asla geçmediği bir yığın üzerinde bir yığın işlemleri dizisi uyguladığınızı düşünün. Her k işleminden sonra, yedekleme amacıyla bütün yığının bir kopyasını alıyoruz. Yığının kopyalama işlemi de dahil olmak üzere tüm n yığın işlemlerinin $O(n)$ olduğunu, çeşitli yığın işlemlerine uygun amortize maliyetler atayarak gösteriniz.

17.2-2

Problem 17.1-3 ü, analizin bir hesaplama yöntemini kullanarak tekrar yapınız.

17.2-3

Sadece sayacı artırmak değil, aynı zamanda onu sıfıra da döndürmek (yani içindeki bütün bitleri 0 yapmak) istediğimizi düşünün. Bitleri incelemek ya da değiştirmek için $\Theta(1)$ şeklinde sayarak, bir sayacı, herhangi bir n sayıdaki INCREMENT ve RESET işlemler dizisinin sıfırdan başlayan bir sayaçta $O(n)$ zaman alacağı şekilde, bir bitler dizisi olarak nasıl uygulayabileceğiniz gösteriniz (*İpucu:* Yüksek-sıra 1 e bir işaretçi yerleştirin).

17.3 Potansiyel yöntem

Önceden ödenmiş işleri veri yapısı içinde belirli nesnelerde depolanmış krediler şeklinde göstermek yerine, amortize analizin **potansiyel yöntemi** önceden ödenmiş işleri “potansiyel enerji” ya da sadece “potansiyel” olarak nitelendirir. Bunun sebebi bunların daha sonraki işlemler için ödeme olabilmesidir. Potansiyeli, veri yapısı içindeki belirli nesnelerden çok, veri yapısının bütünüyle ilişkilendiririz.

Potansiyel yöntem şu şekilde çalışır. Başlangıç veri yapısı D_0 olacak şekilde başlayarak n sayıda işlem gerçekleştireceğiz. Her bir $i = 1, 2, \dots, n$ için, c_i nin i inci işlemin gerçek maliyeti olmasını ve D_i nin, i inci işlemi D_{i-1} veri yapısına uygulayınca ortaya çıkan sonuç olmasına imkan sağlıyoruz. Bir **potansiyel fonksiyon** Φ her bir veri yapısını gerçek bir $\Phi(D_i)$ sayısına yönlendirir, ki bu sayı veri yapısı D_i ile ilişkilendirilmiş **potansiyeldir**. Potansiyel fonksiyon bakımından i inci işlemin **amortize maliyeti** $\hat{c}_i$ şu şekilde tanımlanır:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) . \quad (17.2)$$

Dolayısıyla, her bir işlemin amortize maliyeti, gerçek maliyetine ek olarak işleme göre potansiyelde oluşan değişimdir. 17.2 deki denklemlerle, n işlemin toplam amortize maliyeti şöyledir:

$$\begin{aligned} \sum_{i=1}^n \hat{c}_i &= \sum_{i=1}^n (c_i + \Phi(D_i) - \Phi(D_{i-1})) \\ &= \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0) . \end{aligned} \quad (17.3)$$

Denklemden (A.9) ikinci eşitlik çıkar çünkü $\Phi(D_i)$ terimleri iç içe geçmiştir.

Eğer $\Phi(D_n) \geq \Phi(D_0)$ ı oluşturacak bir potansiyel fonksiyon Φ oluşturursak, toplam amortize maliyet $\sum_{i=1}^n \hat{c}_i$, toplam gerçek maliyet olan $\sum_{i=1}^n c_i$ üzerinde bir üst sınır oluşturur.

Pratikte, toplamda kaç işlem yapılacağını her zaman bilemeyiz. Dolayısıyla, her i için $\Phi(D_i) \geq \Phi(D_0)$ ihtiyaç duyarsak, muhasebe yöntemindeki gibi, önceden ödediğimizi garanti ederiz. Genellikle $\Phi(D_0)$ yu 0 olarak tanımlar ve sonra bütün i ler için $\Phi(D_i) \geq 0$ ı gösteririz. $\Phi(D_0) \neq 0$ şeklindeki durumları çözmenin kolay bir yol için bkz: Problem 17.3-1).

Sezgisel olarak, i inci işlemin potansiyel farkı $\Phi(D_i) - \Phi(D_{i-1})$ pozitifdir, dolayısıyla amortize maliyet $\hat{c}_i$ i inci işlem üzerindeki bir şişirmeyi (overcharge) temsil eder ve veri yapısının potansiyeli artar. Potansiyel farkı negatif ise, o zaman amortize maliyet i inci işlem üzerinde bir düşük fiyatlandırma var demektir ve potansiyeldeki bu düşüş işlemin gerçek maliyetinin bedelini öder.

17.2 ve 17.3 teki denklemlerde tanımlanan amortize maliyet potansiyel fonksiyon Φ seçimine bağlıdır. Farklı potansiyel fonksiyonları farklı amortize maliyetler verebilir ancak yine de gerçek fiyatın üzerinde üst sınırlar olacaklardır. Sıklıkla potansiyel fonksiyonlarını seçerken yapabileceğimiz değişik tokuşlarla karşılaşırız; kullanılacak en iyi potansiyel fonksiyonu istenen zaman sınırlarına bağlıdır.

Yığın işlemleri

Potansiyel yöntemi açıklamak için, bir kere daha PUSH, POP ve MULTIPOP yığın işlemleri örneğine döneceğiz. Bir yığın üzerindeki potansiyel fonksiyon Φ u yığındaki nesnelerin sayısı olacak şekilde tanımlarız. Kullanarak başlayacağımız D_0 boş yığını için elimizde $\Phi(D_0) = 0$ var. Bir yığındaki nesnelerin sayısı asla negatif olmayacağı için, i inci işleminin sonucu olarak ortaya çıkacak D_i yığınının negatif olmayan bir potansiyeli vardır ve dolayısıyla,

$$\begin{aligned}\Phi(D_i) &\geq 0 \\ &= \Phi(D_0) \text{ dır.}\end{aligned}$$

Dolayısıyla, Φ bakımından n işlemin toplam amortize maliyeti gerçek maliyet üzerinde bir üst sınır teşkil eder.

Şimdi çeşitli yığın işlemlerinin amortize maliyetlerini hesaplayalım. Eğer s nesne içeren bir yığının içindeki i inci işlem bir PUSH işlemi ise, o zaman potansiyel fark

$$\begin{aligned}\Phi(D_i) - \Phi(D_{i-1}) &= (s + 1) - s \\ &= 1 \text{ dır.}\end{aligned}$$

17.2 deki denkleme göre, bu PUSH işleminin amortize değeri

$$\begin{aligned}\hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= 1 + 1 \\ &= 2 \text{ dır.}\end{aligned}$$

Yığılının üzerindeki i inci işlemin bir MULTIPOP (S, k) işlemi olduğunu düşünelim. bu işlem $k' = \min(k, s)$ nesnenin yığından açılmasını sağlar. İşlemin gerçek maliyeti k' dir ve potansiyel fark bu durumda:

$$\Phi(D_i) - \Phi(D_{i-1}) = -k' \text{ dir.}$$

Dolayısıyla MULTIPOP işleminin amortize maliyeti

$$\begin{aligned} \hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &= k' - k' \\ &= 0 \text{ dir.} \end{aligned}$$

Benzer şekilde, sonradan bir POP işleminin amortize maliyeti de 0 dir.

Üç işlemin her birinin amortize maliyeti $O(1)$ dir ve dolayısıyla n işlemlili bir dizinin toplam amortize maliyeti $O(n)$ dir. $\Phi(D_i) \geq \Phi(D_0)$ olduğunu zaten belirttiğimizden, n işlemin toplam amortize maliyeti toplam gerçek maliyet üzerinde bir üst sınır teşkil eder. n işlemin en kötü durum maliyeti bu durumda $O(n)$ dir.

Bir ikili sayacı artırma

Potansiyel yöntemin bir başka örneği olarak, bir kere daha bir ikili sayacı artırma örneğine döneceğiz. Bu sefer, sayacın i inci INCREMENT işleminden sonraki potansiyelini b_i , i inci işlemden sonra sayaçtaki 1 lerin sayısı, olarak tanımlayacağız.

Bir INCREMENT işleminin amortize maliyetini hesaplayalım. i INCREMENT işleminin t_i sayıda biti resetlediğini düşünelim. Bu durumda, t_i sayıda biti resetlemeye ek olarak en fazla bir biti 1 e eşitleyeceğinden, işlemin gerçek maliyeti en fazla $t_i + 1$ dir. $b_i + 0$ ise, bu durumda i inci işlem bütün k bitleri resetler, ve böylece, $b_{i-1} = t_i = k$ olur. Eğer $b_i > 0$ ise, bu durumda $b_i = b_{i-1} - t_i + 1$ olur. Her iki durumda da, $b_i \leq b_{i-1} - t_i + 1$ ve potansiyel fark şöyle olur:

$$\begin{aligned} \Phi(D_i) - \Phi(D_{i-1}) &\leq (b_{i-1} - t_i + 1) - b_{i-1} \\ &= 1 - t_i . \end{aligned}$$

Dolayısıyla amortize maliyet de şu şekilde olacaktır:

$$\begin{aligned} \hat{c}_i &= c_i + \Phi(D_i) - \Phi(D_{i-1}) \\ &\leq (t_i + 1) + (1 - t_i) \\ &= 2 . \end{aligned}$$

Eğer sayaç sıfırdan başlarsa o zaman $\Phi(D_0) = 0$ olur. Bütün i ler için $\Phi(D_i) \geq 0$ olduğundan, n INCREMENT işlemlili bir dizinin toplam amortize maliyeti toplam gerçek maliyetin üzerinde bir üst sınır teşkil eder ve dolayısıyla, n INCREMENT işlemin en kötü durum maliyeti $O(n)$ dir.

Potansiyel yöntem sayaç sıfırdan başlamasa bile bize sayacı analiz etmenin kolay bir yöntemini sunar. Sayaç b_0 1s ile başlar ve n INCREMENT işleminden

sonra, $0 \leq b_0, b_n \leq k$ durumunda, b_n 1s olur (k nın sayaçtaki bit sayısı olduğunu hatırlayın) Denklemi şu şekilde tekrar yazabiliriz:

$$\sum_{i=1}^n c_i = \sum_{i=1}^n \hat{c}_i - \Phi(D_n) + \Phi(D_0) \quad (17.4)$$

Bütün $1 \leq i \leq n$ ler için elimizde $\hat{c}_i \leq 2$ bulunmaktadır. $\Phi(D_0) = b_0$ ve $\Phi(D_n) = b_n$ olduğu için, n INCREMENT işleminin toplam gerçek maliyeti şöyledir:

$$\begin{aligned} \sum_{i=1}^n c_i &\leq \sum_{i=1}^n 2 - b_n + b_0 \\ &= 2n - b_n + b_0 . \end{aligned}$$

Özellikle dikkat edin ki; $k = O(n)$ olduğu sürece $b_0 \leq k$ olduğu için, toplam gerçek maliyet $O(n)$ dır. Bir başka deyişle, en az $n = \Omega(k)$ sayıda INCREMENT işlemi gerçekleştirirsek, sayacın hangi başlangıç değerinde başladığından bağımsız olarak, toplam gerçek maliyet $O(n)$ dır.

Alıştırmalar

17.3-1

Bütün i ler için $\Phi(D_i) \geq \Phi(D_0)$ olan ama aynı zamanda $\Phi(D_0) \neq 0$ öngören bir potansiyel Φ fonksiyonumuz olsun. Bütün $i \geq 1$ ler için $\Phi'(D_0) = 0$ ı sağlayan $\Phi'(D_i) \geq 0$ şeklinde bir Φ' fonksiyonu olduğunu ve bu fonksiyonla elde edilen amortize maliyetlerin Φ fonksiyonuyla elde edilenle aynı olduğunu gösteriniz.

17.3-2

Problem 17.3-1 i bir potansiyel yöntemi kullanarak tekrar yapınız.

17.3-3

n elemanlı ve $O(\lg n)$ en kötü durum zamanında INSERT ve EXTRACT-MIN komutlarını destekleyen sıradan bir ikili min-bellek yığını veri yapısı (min-heap data structure) düşünün. INSERT işleminin amortize maliyetinin $O(\lg n)$, EXTRACT-MIN işleminin amortize maliyetinin $O(1)$ olacağı bir potansiyel fonksiyon Φ yazın ve fonksiyonun çalıştığını gösterin.

17.3-4

s_0 ile başlayıp s_n ile bittiğini varsaydığımız bir yığının n sayıda PUSH, POP ve MULTIPOP işlemlerini uygulamanın toplam maliyeti nedir?

17.3-5

Bir sayacın kendi ikili gösteriminde (binary representation) 0 yerine b 1s li bir sayıda başladığını varsayalım. n sayıda INCREMENT işlemi uygulamanın maliyetinin, $n = \Omega(b)$ durumunda $O(n)$ olacağını gösterin (b nin sabit olduğunu varsaymayın).

17.3-6

Sıradan iki yığından oluşan bir kuyruğu (Problem 10.1-6) ENQUEUE ve DEQUEUE işlemlerinin amortize maliyeti $O(1)$ olacak şekilde nasıl uygulayacağınızı gösteriniz.

17.3-7

Çifte değerlere izin veren bir dinamik tam sayı çoklu küme S için aşağıdaki işlemleri destekleyen bir veri yapısı oluşturun:

INSERT (S, x) x i S e dahil eder.

DELETE-LARGER-HALF (S) en büyük $\lceil |S|/2 \rceil$ elemanları S den siler.

Herhangi bir m sayıda INSERT ve DELETE-LARGER-HALF işlemler dizisinin $O(m)$ zamanda çalışacağı şekilde bu veri yapısını nasıl uygulayabileceğinizi açıklayın. Uygulamanızın aynı zamanda S in elemanlarını $O(|S|)$ zamanda ortaya çıkaracak bir yöntem içermelidir.

17.4 Dinamik tablolar

Bazı uygulamaların kaç nesneyi tabloda saklayacaklarını her zaman önceden bilmeyiz. Tabloda belli bir yer tahsis edip sonradan bu yerin yetersiz olduğunu anlayabiliriz. Daha sonra tabloyu daha büyük bir tabloyla değiştirip orijinal tablodaki bütün nesneleri bu yeni ve daha büyük tabloya kopyalamamız gerekir. Benzer şekilde, eğer tablodan bir çok nesne silinmişse, tabloyu daha küçüğüyle değiştirmek de önemli olabilir. Bu kısımda, bu dinamik şekilde genişleyen ve daralan tablo problemini inceleyeceğiz. Amortize analizi kullanarak, işlemin gerçek maliyeti genişleme ve daralmayı tetiklediğinde büyük olsa bile, araya sokma ve silmenin amortize maliyetinin sadece $O(1)$ olduğunu göstermeliyiz. Dahası, dinamik bir tablodaki kullanılmayan alanın hiç bir zaman toplam alanın sabit bir bölümünü geçmeyeceğini nasıl garanti edeceğimizi de göstermeliyiz.

Dinamik tablonun TABLE-INSERT ve TABLE-DELETE işlemlerini desteklediğini varsayıyoruz. TABLE-INSERT, tek bir dilim zamanı işgal eden — dilim zaman tek bir öğeye ayrılan alandır — bir öğeyi tabloya ekler. Benzer şekilde, TABLE-DELETE işlemi bir öğeyi siler ve dilim zamanı boşaltır. Tabloyu organize etmek için kullanılan veri-yapılandırma yönteminin ayrıntıları önemsizdir; bir yığın (Kısım 10.1), bir bellek yığını (Bölüm 6) ya da bir çırpı tablosu (Bölüm 11) kullanabiliriz. Aynı şekilde, Kısım 10.3 te yaptığımız gibi, nesne belleği uygulamak için bir dizi ya da diziler kümesi de kullanabiliriz.

Çırpılama analizimizde tanıttığımız bir kavramı kullanmak uygun olabilir (Bölüm 11). Boş olmayan bir tablo olan T nin **yükleme faktörü** $\alpha(T)$ ı tablodaki öğe sayısının tablonun boyutuna (dilim zaman sayısı) bölümü olarak tanımlarız. Boş bir tabloyu (hiç öğesi olmayan bir tablo) 0 boyutlu olarak atar ve bu tablonun

yükleme faktörünü 1 olarak tanımlarız. Eğer bir dinamik tablonun yük çarpanı sabit bir sayıyla alttan sınırlanmışsa, tablodaki kullanılmayan alan kullanılan tüm alanın bir sabit kesrinden asla büyük değildir.

Sadece öge eklemesi yaptığımız bir dinamik tablonun analiziyle başlayalım. Öğeleri eklediğimiz ve sildiğimiz daha genel bir durumu daha sonra değerlendiririz.

17.4.1 Tablo genişlemesi

Bir tablo için ayrılan belleği bir dilim zamanlar dizisi gibi tahsis edilmiş düşünelim. Bir tablo tüm dilim zamanlar kullanıldığında ya da, eşit bir biçimde, yükleme faktörü 1 olduğunda dolar.¹ Bazı yazılım durumlarında dolu bir tabloya bir madde ekleme girişiminin tek alternatifi hata ile işlemi iptal etmektir. Ancak, bizim yazılımımızın, tıpkı diğer birçok yazılım gibi, isteğe bağlı olarak bellek parçalarını tahsis edebilen ya da boşaltabilen bir bellek yönetim sistemine sahip olduğunu düşünelim. Dolayısıyla, dolu bir tabloya bir öge eklediğimizde, tabloyu, eskisinden daha fazla dilim zamana sahip bir tabloya dönüştürerek **genişletebiliriz**. Her zaman tablonun bitişik bellekte bulunmasını istediğimiz için, yeni tablo için yeni bir dizi tahsis etmeli ve sonra öğeleri eski tablodan yenisine kopyalamalıyız.

Genel bir sezgisel yazılım yeni tabloya eskidekinin tam iki katı dilim zaman açar. Tablo işlemleri sadece araya sokma işlemleriyse, o zaman tablonun yükleme faktörü her zaman en azından $1/2$ dir ve dolayısıyla boşa harcanan alan hiç bir zaman tablodaki toplam alanın yarısını geçmez.

Aşağıdaki kabakodda, T nin tabloyu temsil eden bir nesne olduğunu varsayıyoruz. $T.table$ özniteliği tabloyu temsil eden bellek bloğundaki bir işaretçiyi; $T.num$ tablodaki öge sayısını ve $T.size$ tablodaki toplam dilim zaman sayısını içerir. Başlangıç olarak, tablo boştur: $T.num = T.size = 0$.

TABLE-INSERT(T, x)

```

1  if  $T.size == 0$ 
2      1 dilim zamana sahip  $T.table$  1 tahsis et
3       $T.size = 1$ 
4  if  $T.num == T.size$ 
5       $2 \cdot T.size$  dilim zamana sahip  $new-table$  yi tahsis et
6       $T.table$  deki bütün öğeleri  $new-table$  ye ekle
7       $T.table$  yi boşalt
8       $T.table = new-table$ 
9       $T.size = 2 \cdot T.size$ 
10  $x$  i  $T.table$ 
11  $T.num = T.num + 1$ 
```

¹Açık-adresli çırpı tablosunda olduğu gibi bazı durumlarda, yükleme faktörü 1 den kesin olarak küçük bir sabite eşitse tabloyu dolu kabul edebiliriz (bkz: Problem 17.4-1).

Burada iki “araya sokma” prosedürü kullandığımıza dikkat edin: TABLE-INSERT prosedürünün kendisi ve tablodaki 6-10. satırlara uyguladığımız **temel araya sokma** prosedürüdür. Her bir temel araya sokma işlemine 1 maliyetini atayarak temel araya sokmalar açısından TABLE-INSERT işleminin çalışma zamanını analiz edebiliriz. 2. Satırda bir başlangıç tablosu tahsis etmenin ek yükünün sabit olacağı ve 5 ve 7. Satırlarda belleği boşaltmak ve tahsis etmek için gereken ek yükün 6. Satırdaki öğeleri taşıma maliyetiyle belirleneceği şekilde TABLE-INSERT in bireysel öğeler eklemek için harcadığı gerçek çalışma zamanının doğrusal olduğunu varsayıyoruz. 5-9. satırlarda gerçekleşen işlemlere **genişleme** diyoruz.

Başlangıçta boş olan bir tablo üzerinde n sayıda TABLE-INSERT işlemi dizisini inceleyelim. i inci işlemin maliyeti c_i nedir? Eğer elimizdeki tabloda yeni bir öğe için boş yer varsa (ya da bu daha ilk işlemse), o zaman, 10. Satırda sadece bir temel araya sokma yapmamız gerektiği için, $c_i = 1$ dir. Ancak elimizdeki tablo doluysa ve bir genişleme olacaksa, o zaman $c_i = i$ dir: 10. satırdaki temel eklemenin maliyeti 1 dir artı 6. satırda eski tablodan yenisine kopyalamamız gereken öğeler için $i - 1$ dir. Eğer n işlem gerçekleştirirsek, bir işlemin en kötü durum maliyeti $O(n)$ dır ki bu da n işlem için toplam çalışma zamanında $O(n^2)$ bir üst sınır oluşturur.

Bu sınır kesin değildir çünkü bir tabloyu nadiren n TABLE-INSERT işlemiyle genişletiriz. Özellikle, i inci işlem sadece $i - 1$ 2 nin tam kuvveti olduğunda bir genişlemeye sebep olur. Bir işlemin amortize maliyeti aslında toparlama analizini kullanarak gösterebileceğimiz gibi $O(1)$ ’dır. i ninci işlemin maliyeti şöyledir:

$$c_i = \begin{cases} i, & i - 1 \text{ 2 nin tam kuvvetiyse,} \\ 1, & \text{aksi halde.} \end{cases}$$

Dolayısıyla n TABLE-INSERT işleminin toplam maliyeti de şöyledir:

$$\begin{aligned} \sum_{i=1}^n c_i &\leq n + \sum_{j=0}^{\lfloor \lg n \rfloor} 2^j \\ &< n + 2n \\ &= 3n, \end{aligned}$$

çünkü en fazla n sayıda işlemin maliyeti 1 dir ve kalan işlemlerin maliyeti geometrik bir seri oluşturur. n sayıda TABLE-INSERT işleminin toplam maliyeti $3n$ ile sınırlandırılmış için, bir işlemin amortize maliyeti en fazla 3 tür.

Hesaplama yöntemini kullanarak TABLE-INSERT işleminin amortize maliyetinin neden 3 olması gerektiğini daha iyi anlayabiliriz. Sezgisel olarak, her öğe 3 temel araya sokma için ödeme yapar: kendini var olan tabloya eklemek için, tablo genişlediğinde kendini taşımak için ve tablo genişlediğinde zaten taşınmış olan bir başka öğeyi taşımak için. Örneğin, bir tablonun boyutunun genişlemeden hemen sonra m olduğunu düşünün. Bu durumda tablo $m/2$ öğeye sahiptir ve hiç

kredi içermemektedir. Her araya sokmayı 3 dolarla yükleriz. Hemen gerçekleşen temel araya sokma işlemi 1 dolar tutar. Öğenin üzerine kredi olarak 1 dolar daha bırakırız. Hâlihazırda tabloda bulunan $m/2$ öğelerden birinin üzerine kredi olarak üçüncü bir dolar daha bırakırız. Tablo, biz başka bir $m/2 - 1$ öge ekleyene kadar dolmayacaktır ve dolayısıyla, tabloda m öge olup da tablo dolduğunda, her bir öğenin üzerinde genişleme esnasında gerçekleşecek tekrar araya sokma işlemini ödemek için bir dolar bırakmış halde oluruz.

n sayıda TABLE-INSERT işlemleri dizisini analiz etmek için potansiyel yöntemi kullanabilir ve Kesim 17.4.2 de de $O(1)$ amortize maliyeti olan TABLE-DELETE işlemini tasarlamak için bunu kullanmalıyız. Genişlemeden hemen sonra 0 olan ancak tablo dolduğunda tablonun boyutuna ulaşan bir potansiyel fonksiyon Φ tanımlayarak başlıyoruz ki potansiyelle oluşacak bir sonraki genişlemeyi ödeyebilelim.

$$\Phi(T) = 2 \cdot T.num - T.size \quad (17.5)$$

fonksiyonu olabilir.

Bir genişlemeden hemen sonra elimizde $T.num = T.size/2$ ve dolayısıyla istenen şekilde $\Phi(T) = 0$ olacaktır. Genişlemeden hemen önce ise elimizde $T.num = T.size$ ve dolayısıyla istenen şekilde $\Phi(T) = T.num$ olacaktır. Potansiyelin başlangıç değeri 0 dır ve tablo her zaman en azından yarı dolu olduğu için, $T.num \geq T.size/2$ olur ki bu da $\Phi(T)$ ın her zaman pozitif bir değer olduğunu gösterir. Dolayısıyla, n sayıda TABLE-INSERT işleminin amortize maliyetinin toplamı gerçek maliyetin toplamına bir üst sınır getirir.

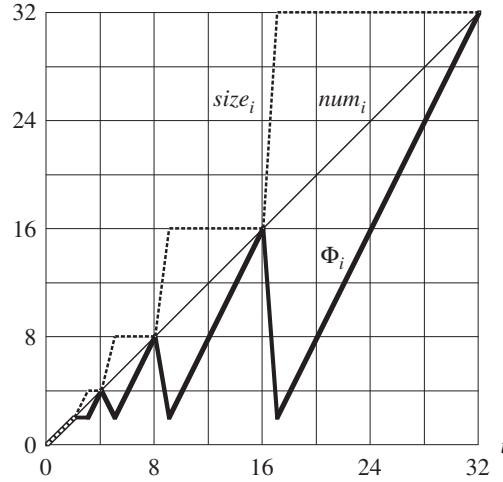
i inci TABLE-INSERT işleminin amortize maliyetini analiz etmek için num_i nin i inci işlemden sonra tabloda saklanan öge sayısı olduğunu, $size_i$ nin i inci işlemden sonra tablonun toplam boyutu olduğunu ve Φ_i nin i inci işlemden sonraki potansiyel olduğunu düşünüyoruz. Başlangıçta elimizde $num_0 = 0$, $size_0 = 0$, ve $\Phi_0 = 0$ var.

Eğer i inci TABLE-INSERT işlemi bir genişlemeyi tetiklemese, o zaman $size_i = size_{i-1}$ e sahibiz ve işlemin amortize maliyeti şöyle demektir:

$$\begin{aligned} \hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (2 \cdot num_i - size_i) - (2 \cdot num_{i-1} - size_{i-1}) \\ &= 1 + (2 \cdot num_i - size_i) - (2(num_i - 1) - size_i) \\ &= 3. \end{aligned}$$

Eğer i inci işlem bir genişlemeyi tetiklerse o zaman elimizde $size_i = 2 \cdot size_{i-1}$ ve $size_{i-1} = num_{i-1} = num_i - 1$ ve bu şunu gösterir $size_i = 2 \cdot (num_i - 1)$ Dolayısıyla, işlemin amortize maliyeti şöyledir:

$$\begin{aligned} \hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= num_i + (2 \cdot num_i - size_i) - (2 \cdot num_{i-1} - size_{i-1}) \\ &= num_i + (2 \cdot num_i - 2 \cdot (num_i - 1)) - (2(num_i - 1) - (num_i - 1)) \\ &= num_i + 2 - (num_i - 1) \\ &= 3. \end{aligned}$$



Şekil 17.3 Tablodaki num_i sayıda öğeye uygulanan n sayıda TABLE-INSERT işleminin, $size_i$ sayıda dilim zamanının ve $\Phi_i = 2 \cdot num_i - size_i$ potansiyelinin, her biri i inci işlemten sonra ölçülmüş haldeki etkisi. İnce çizgi num_i yi, kesikli çizgi $size_i$ yi ve kalın çizgi Φ_i yi gösterir. Genişlemeden hemen önce potansiyelin tablodaki öğe sayısına ulaştığına ve dolayısıyla bütün öğelerin yeni tabloya taşınmasını ödeyebileceğine dikkat edin. Sonrasında, potansiyel 0 a düşer ancak genişlemeye sebep olan öğe eklenir eklenmez 2 ye yükselir.

Şekil 17.3 num_i , $size_i$, ve Φ_i nin i karşısındaki değerini gösterir. Potansiyelin tabloyu ödemek için nasıl yükseldiğine dikkat edin.

17.4.2 Tablo genişlemesi ve daralması

Bir TABLE-DELETE işlemi uygulamak için belirli bir öğeyi tablodan kaldırmak yeterince kolaydır. Ancak, boşa harcanan alanı sınırlandırmak için, yük çarpamı çok küçük olduğunda tabloyu **daraltmak** isteyebiliriz. Tablo daraltmak tablo genişletmenin benzeridir: tablodaki öğelerin sayısı çok alt seviyelere düştüğünde, daha küçük, yeni bir tablo tahsis eder ve öğeleri eski tablodan yeni tabloya kopyalarız. Sonrasında, eski tabloyu bellek yönetim sistemine göndererek, eski tablonun belleğini boşaltabiliriz. İdeal olarak, iki özelliği korumak isteriz:

- Dinamik tablonun yükleme faktörü pozitif bir değerle alttan sınırlıdır ve
- Bir tablo işleminin amortize maliyeti bir sabitle üstten sınırlıdır.

Maliyeti temel araya sokma ve silme işlemleri bağlamında hesapladığımızı varsayıyoruz.

Dolu bir tabloya bir öge eklediğimizde tablo boyutunu iki katına çıkarmamız gerektiğini ya da bir madde silmenin tabloyu yarıdan daha az yapacağı durumlarda tablo boyutunu yarıya indirmemiz gerektiğini düşünebilirsiniz. Bu strateji tablonun yük çarpanının asla $1/2$ değerinin altına düşmemesini garanti eder, ancak ne yazık ki, bir işlemin amortize maliyetinin çok büyük olmasına sebep olabilir. Şu senaryoyu düşünün. Bir T tablosunda n sayıda işlem yapıyoruz ve $n/2$ 'nin tam kuvveti. İlk $n/2$ işlemler araya sokma işlemleri ve daha önceki analizimizde bulduğumuz üzere bunun maliyeti $\Phi(n)$ dır. Bu araya sokmalar dizisinin sonunda, $T.num = T.size = n/2$ olur. İkinci $n/2$ işlemleri için, aşağıdaki diziyi uyguluyoruz:

ekle, sil, sil, ekle, ekle, sil, sil, ekle, ekle,.....

İlk araya sokma tablonun n boyutuna genişlemesine sebep olur. Takip eden 2 silme tabloyu tekrar $n/2$ boyutuna daraltır. Sonraki araya sokmalar bir başka genişlemeye sebep olur ve bu böyle devam eder. Her bir genişleme ve daralmanın maliyeti $\Theta(n)$ ve toplamda $\Theta(n)$ sayıda işlem var. Dolayısıyla, n işlemin toplam maliyeti $\Theta(n^2)$ olur ve bu da bir işlemin amortize maliyetini $\Theta(n)$ yapar.

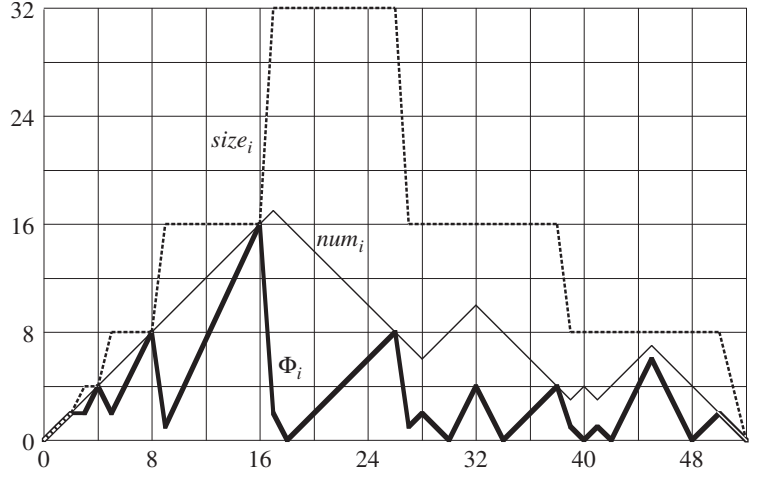
Bu stratejinin dezavantajı gayet açıktır: tabloyu genişlettikten sonra, daraltmayı ödemek için yeterince öge silmiyoruz. Aynı şekilde, tabloyu daralttıktan sonra, genişlemeyi ödemek için yeterince öge eklemiyoruz.

Tablonun yükleme faktörünün $1/2$ seviyesinin altına inmesine izin vererek bu stratejiyi geliştirebiliriz. Özellikle, dolu bir tabloya bir öge ekleyerek tablo boyutunu iki katına çıkarmaya devam ederiz, ancak, bir öge silmenin, daha önceki gibi tablonun $1/2$ doluluk seviyesinin altına düşmesi yerine, $1/4$ doluluk seviyesinin altına düşmesine sebep açtığı durumlarda tablo boyutunu yarıya indiririz. Dolayısıyla tablonun yükleme faktörü $1/4$ sabitiyle alttan sınırlı olacaktır.

Sezgisel olarak, $1/2$ yükleme faktörünü ideal kabul ederiz ve tablonun potansiyeli bu durumda 0 olacaktır. Yükleme faktörü $1/2$ den saptığında, potansiyel öyle yükselir ki tabloyu genişlettiğimiz ya da daralttığımız zaman tablo yeni tahsis edilmiş tabloya bütün öğeleri kopyalamayı ödeyecek kadar potansiyel kazanmış olur. Dolayısıyla, yükleme faktörü 1 e yükseldiği ya da $1/4$ seviyesine indiği zaman $T.num$ a ulaşmış olacak bir potansiyel fonksiyonuna ihtiyacımız vardır. Tabloyu daralttıktan ya da genişlettikten sonra, yükleme faktörü $1/2$ ye ve tablonun potansiyeli de tekrar geriye 0'a döner.

TABLE-INSERT in benzeri olduğu için TABLE-DELETE kodunu yazmıyoruz. Analizimiz için, tablodaki öge sayısının 0 a düştüğü her durumda tablo için bellegi boşaltmış olduğumuzu varsayıyoruz. Bu da şu demek oluyor, eğer $T.num = 0$ ise $T.size = 0$ dır.

Şimdi, n sayıda bir TABLE-INSERT ve TABLE-DELETE işlemleri dizisinin maliyetini analiz etmek için potansiyel yöntemini kullanabiliriz. Genişleme ya da daralmadan hemen sonra 0 olan ve yükleme faktörü 1 e yükseldiğinde ya da $1/4$



Şekil 17.4 tablodaki num_i sayıda öğeye uygulanan n sayıda TABLE-INSERT işleminin, $size_i$ sayıda dilim zamanının ve

$$\Phi_i = \begin{cases} 2 \cdot num_i - size_i & \alpha_i \geq 1/2 \text{ ise,} \\ size_i/2 - num_i & \alpha_i < 1/2 \text{ ise,} \end{cases}$$

potansiyelinin, her biri i inci işlemden sonra ölçülmüş haldeki etkisi. İnce çizgi num_i yi, kesikli çizgi $size_i$ yi ve kalın çizgi Φ_i yi gösterir. Genişlemeden hemen önce potansiyelin tablodaki öğe sayısına ulaştığına ve dolayısıyla bütün öğelerin yeni tabloya taşınmasını ödeyebileceğine dikkat edin. Sonrasında, potansiye 1 0 a düşer ancak genişlemeye sebep olan öğe eklenir eklenmez 2 ye yükselir.

e düştüğünde oluşan bir potansiyel fonksiyonu Φ tanımlayarak başlıyoruz. Boş olmayan bir tablonun yükleme faktörüne $\alpha(T) = T.num / T.size$ diyelim. Boş bir tablo için $T.num = T.size = 0$ ve $\alpha(T) = 1$ olduğundan, tablo boş olsa da olmasa da elimizde her zaman $T.num = \alpha(T) \cdot T.size$ vardır. Potansiyel fonksiyonumuz şöyle olmalıdır:

$$\Phi(T) = \begin{cases} 2 \cdot T.num - T.size & \alpha(T) \geq 1/2 \text{ ise,} \\ T.size/2 - T.num & \alpha(T) < 1/2 \text{ ise.} \end{cases} \quad (17.6)$$

Tablonun potansiyelinin 0 olduğuna ve potansiyelin asla negatif olmadığına dikkat edin. Dolayısıyla, fonksiyon bakımından işlemler dizisinin amortize maliyeti dizinin gerçek maliyeti üzerinde bir üst sınır sağlar.

Kesin bir analizle devam etmeden önce, potansiyel fonksiyonunun 17.4 te gösterildiği gibi bazı özelliklerine göz atmak için duruyoruz. Yükleme faktörünün $1/2$ olduğu durumda potansiyelin 0 olduğuna dikkat edin. Yükleme faktörü 1 olduğunda elimizde $T.size = T.num$ vardır ki bu da $\Phi(T) = T.num$ gösterir ve dolayısıyla potansiyel, eğer bir araya sokma yapılırsa, genişlemeyi karşılayabilir.

Yükleme faktörünün $1/4$ olduğu durumda elimizde $T.size = 4 \cdot T.num$ vardır ki bu da $\Phi(T) = T.num$ gösterir ve dolayısıyla potansiyel, eğer bir silme yapılırsa, daralmayı karşılayabilir.

n sayıda bir TABLE-INSERT ve TABLE-DELETE işlemler dizisini analiz etmek istersek, i inci işlemin gerçek maliyetine c_i , Φ bakımından bunun amortize maliyetine, i inci işlemden sonra tabloda saklanan öge sayısına num_i , i inci işlemden sonraki tablo büyüklüğüne $size_i$, i inci işlemden sonraki yüklem faktörüne α_i ve i inci işlemden sonraki potansiyele Φ_i diyelim. Başlangıçta $num_0 = 0$, $size_0 = 0$, $\alpha_0 = 1$ ve $\Phi_0 = 0$ dır.

i inci işlemin TABLE-INSERT işlemi olduğu durumla başlayalım. Eğer $\alpha_{i-1} \geq 1/2$ ise, analiz Kısım 17.4 te tablo genişlemesi için yapılanla tıpatıp aynıdır. Tablo genişlese de genişlemese de, amortize maliyet $\hat{c}_i$ en fazla 3 tür. Eğer $\alpha_{i-1} < 1/2$ ise, tablo sadece $\alpha_{i-1} = 1$ olduğunda genişleyebildiği için, işlemin sonucu olarak genişleyemez. Ayrıca eğer $\alpha_i < 1/2$ ise, i inci işlemin amortize

$$\begin{aligned}\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (size_i/2 - num_i) - (size_{i-1}/2 - num_{i-1}) \\ &= 1 + (size_i/2 - num_i) - (size_i/2 - (num_i - 1)) \\ &= 0.\end{aligned}$$

Eğer $\alpha_{i-1} < 1/2$ ama $\alpha_i \geq 1/2$ ise

$$\begin{aligned}\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (2 \cdot num_i - size_i) - (size_{i-1}/2 - num_{i-1}) \\ &= 1 + (2(num_{i-1} + 1) - size_{i-1}) - (size_{i-1}/2 - num_{i-1}) \\ &= 3 \cdot num_{i-1} - \frac{3}{2}size_{i-1} + 3 \\ &= 3\alpha_{i-1}size_{i-1} - \frac{3}{2}size_{i-1} + 3 \\ &< \frac{3}{2}size_{i-1} - \frac{3}{2}size_{i-1} + 3 \\ &= 3.\end{aligned}$$

maliyeti şöyledir:

Dolayısıyla bir TABLE-INSERT işleminin amortize maliyeti en fazla 3 tür.

Şimdi, işlemin TABLE-DELETE olduğu duruma dönelim. Bu durumda, $num_i = num_{i-1} - 1$ dir. $\alpha_{i-1} < 1/2$ ise, işlemin tablonun daralmasına yol açıp açmadığına bakmalıyız. Eğer açmıyorsa, $size_i = size_{i-1}$ durumu vardır ve işle-

$$\begin{aligned}\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= 1 + (size_i/2 - num_i) - (size_{i-1}/2 - num_{i-1}) \\ &= 1 + (size_i/2 - num_i) - (size_i/2 - (num_i + 1)) \\ &= 2.\end{aligned}$$

min amortize maliyeti şöyledir:

$\alpha_{i-1} < 1/2$ ise ve i inci işlem bir daralmayı tetikliyorsa, o zaman işlemin gerçek maliyeti, bir öge sildiğimiz ve num_i öge taşıdığımız için $c_i = num_i + 1$ dir. Elimizde $size_i/2 = size_{i-1}/4 = num_{i-1} = num_i + 1$ vardır ve işlemin amortize maliyeti şöyledir:

$$\begin{aligned}\hat{c}_i &= c_i + \Phi_i - \Phi_{i-1} \\ &= (num_i + 1) + (size_i/2 - num_i) - (size_{i-1}/2 - num_{i-1}) \\ &= (num_i + 1) + ((num_i + 1) - num_i) - ((2 \cdot num_i + 2) - (num_i + 1)) \\ &= 1.\end{aligned}$$

i inci işlem bir TABLE-DELETE işlemi ve $\alpha_{i-1} \geq 1/2$ ise, amortize maliyet bir sabitle üstten sınırlanmıştır. Analizi, Problem 17.4-2 deki gibi bırakırız.

Özetle, her bir işlemin amortize maliyeti bir sabitle üstten sınırlı olduğu için, dinamik tablo üzerinde n operasyonlu herhangi bir dizi için gerçek zaman $O(n)$ dır.

Alıştırmalar

17.4-1

Dinamik, açık-adresli bir çırpı tablosu yapmak istediğimizi düşünün. Yükleme faktörü α 1 den kesin olarak daha az bir değere ulaştığında neden tablonun dolu olduğunu düşünmemiz gerekir? Amortize maliyetin araya sokma başına beklenen/ortalama değerinin $O(1)$ olacağı şekilde çalışan bir dinamik, açık-adres çırpı tablosuna nasıl araya sokma yapılacağını kısaca açıklayınız. Neden araya sokma başına gerçek maliyetin beklenen değeri bütün araya sokma için kesinlikle $O(1)$ değildir?

17.4-2

$\alpha_{i-1} \geq 1/2$ ve dinamik tablo üzerinde i inci işlem TABLE-DELETE ise, işlemin amortize maliyetinin potansiyel fonksiyon bakımından (17.6) bir sabitle üstten sınırlı olduğunu gösterin.

17.4-3

Yükleme faktörünün $1/4$ ün altına düştüğü durumda bir tabloyu boyutunu yarıya indirerek daraltmak yerine, yük çarpanının $1/3$ ün altına düştüğünde tablo boyutunu $2/3$ ile çarparak daralttığımızı düşünün.

$$\Phi(T) = |2 \cdot T.num - T.size|,$$

potansiyel fonksiyonunu kullanarak bu stratejiyi kullanan bir TABLE-DELETE işleminin amortize maliyetinin bir sabitle üstten sınırlı olduğunu gösterin.

Problemler

17-1 Bit düzeyinde ters çevrilmiş ikili sayaç

Bölüm 30, hızlı Fourier dönüşümü ya da FFT diye adlandırılan önemli bir algoritmayı incelemektedir. FFT algoritmasının ilk adımı, uzunluğu negatif olmayan bir tam sayı olan k için $n = 2^k$ olan $A[0..n-1]$ girdi dizisinin üzerinde bir **bit düzeyinde ters çevrilmiş permütasyon** uygular. Bu permütasyon indeksleri birbirinin tersi olan ikili gösterimlere sahip elemanları değiş tokuş eder.

$a = \sum_{i=0}^{k-1} a_i 2^i$ olduğunda her bir a indisini k -bitlik $\langle a_{k-1}, a_{k-2}, \dots, a_0 \rangle$ dizisi şeklinde ifade edebiliriz. $\text{rev}_k(\langle a_{k-1}, a_{k-2}, \dots, a_0 \rangle) = \langle a_0, a_1, \dots, a_{k-1} \rangle$ şeklinde tanımlarız, dolayısıyla

$$\text{rev}_k(a) = \sum_{i=0}^{k-1} a_{k-i-1} 2^i \text{ dir.}$$

Örneğin, 3'ün 4 bitli tasarımı 0011 olduğu ve bunun zıttı 1100 ve böylece 12'nin 4 bit tasarımı olduğu için, $n = 16$ (ya da, eşit şekilde, $k = 4$) ise $\text{rev}_k(3) = 12$ dir.

a. $\Theta(k)$ zamanda çalışan bir rev_k fonksiyonu dikkate alarak, $O(nk)$ zamanda $n = 2^k$ uzunluğundaki bir dizinin bit düzeyinde ters çevrilmiş permütasyonunu uygulamak için bir algoritma yazın.

Bit düzeyinde ters çevrilmiş permütasyonun çalışma zamanını geliştirmek için, amortize analiz üzerine kurulu bir algoritma yazabiliriz. Bit düzeyinde ters çevrilmiş sayaca a değerini verdiğimizde $\text{rev}_k(\text{rev}_k(a) + 1)$ sonucunu üreten bir BIT-REVERSED-INCREMENT prosedürü ve “bit düzeyinde ters çevrilmiş sayaç” kullanırız. Örneğin, $k = 4$ ise ve bit düzeyinde ters çevrilmiş sayaç 0 dan başlıyorsa o zaman BIT-REVERSED-INCREMENT e yapılan ardışık çağrılar şu diziye oluşturur.

0000, 1000, 0100, 1100, 0010, 1010, ... = 0, 8, 4, 12, 2, 10,

b. Bilgisayarınızdaki kelimelerin k -bit değer taşıdığını ve birim zaman başına bilgisayarınızın rassal sayılarda sağa sola kaydırma, bitwise-AND, bitwise-OR gibi işlemlerle ikili değerleri değiştirebildiğini düşünün. BIT REVERSED INCREMENT prosedürünün n elemanlı bir dizide zıt yönlü bit permütasyonunu toplamda $O(n)$ zamanda yapabilecek bir uygulamasını tanımlayın.

c. Bir kelimeyi birim zaman başına sağa ya da sola sadece bir birim kaydırabildiğinizi düşünün. $O(n)$ -zaman bit düzeyinde ters çevrilmiş permütasyonunu uygulamak hala mümkün müdür?

17-2 İkili aramayı dinamikleştirmek

Sıralı bir dizinin ikili araması logaritmik arama zamanı alır, ancak, dizinin boyutuna yeni bir öge eklemek doğrusaldır. Araya sokma için gereken zamanı bir çok sıralı diziyi tutarak geliştirebiliriz.

Özellikle, n elemanlı bir kümenin üzerinde SEARCH ve INSERT işlemlerini desteklemek istiyoruz. $k = \lceil \lg(n+1) \rceil$ olsun ve n nin ikili gösterimi $\langle n_{k-1}, n_{k-2}, \dots, n_0 \rangle$ olsun. $i = 0; 1, \dots, k-1$ için dizinin uzunluğunun A_i is 2^i olduğu bir $A_0, A_1, \dots, A_{k-1}$ k sıralı dizimiz var. Her dizi sırasıyla $n_i = 1$ ya da $n_i = 0$ a göre dolu ya da boştur. Bütün k dizilerindeki toplam eleman sayısı dolayısıyla $\sum_{i=0}^{k-1} n_i 2^i = n$ dir. Her ne kadar her bir dizi sıralı olsa da, farklı dizilerdeki elemanların birbirleriyle hiçbir özel bağıntısı yoktur.

- Bu veri yapısı için SEARCH işlemini nasıl gerçekleştireceğinizi açıklayınız. Bu işlemin en kötü durumdaki çalışma zamanını analiz ediniz.
- INSERT işlemini nasıl gerçekleştireceğinizi açıklayınız. En kötü durum ve amortize işlem sürelerini analiz ediniz.
- DELETE işlemini nasıl uygulayacağınızı açıklayınız.

17-3 Amortize ağırlık-dengeli ağaçlar

x kökünün altağaçlarında saklanan anahtar sayılarının verilmesiyle her x düğümüne $x.size$ değerini iliştirerek artırılmış sıradan bir ikili arama ağacı düşünün. $1/2 \leq \alpha < 1$ aralığında bir sabit değer olsun. Eğer $x.left.size \leq \alpha \cdot x.size$ ve $x.right.size \leq \alpha \cdot x.size$ ise verilen x düğümü α -**dengeli**dir. Eğer ağaçtaki düğüm α -**dengeli** ise, bütün ağaç α -**dengeli**dir. Ağırlık-dengeli ağaçların sağlanmasına dair aşağıdaki amortize yaklaşım G. Varghese tarafından öne sürülmüştür.

- 1/2-dengeli ağacı, bir anlamda, olabildiğince dengelidir. Rassal bir ikili arama ağacının içindeki bir x düğümünü ele alarak, x köküne bağlı bir altağacı 1/2 dengeli hale gelecek şekilde nasıl yeniden inşa edebileceğinizi gösterin. Algoritmanız $\Theta(x.size)$ zamanda çalışmalıdır ve $O(x.size)$ yardımcı belleğini kullanabilir.
- n -düğümlü bir α -dengeli ikili arama ağacında gerçekleştirilecek bir aramanın $O(\lg n)$ en kötü durum zamanında çalışacağını gösteriniz.

Bu problemin artanı olarak α sabitinin $1/2$ den kesinlikle büyük olduğunu varsayın. n -düğümlü bir ikili arama ağacı için INSERT ve DELETE işlemlerini her zaman ki gibi yaptığımızı düşünün. Bunun tek istisnası şudur: bu işlemlerin her birinden sonra ağaçtaki herhangi bir düğüm α -dengeli değilse, o zaman altağacı ağaçtaki benzer en yüksek düğümünde köklenmiş olacak şekilde tekrar yapılandırırız ki 1/2- dengeli hale gelsin.

Bu yeniden yapılandırma düzenini potansiyel yöntemini kullanarak analiz etmeliyiz. Bir T ikili arama ağacındaki x düğümü için şunu tanımlarız

$$\Delta(x) = |x.left.size - x.right.size|$$

ve $c \alpha$ e bağlı yeterince büyük bir sabit olduğunda, T nin potansiyelini

$$\Phi(T) = c \sum_{x \in T: \Delta(x) \geq 2} \Delta(x),$$

olarak tanımlarız.

- c. Bütün ikili arama ağaçlarının negatif olmayan bir potansiyele sahip olduğunu ve 1/2-dengeli bir ağacın potansiyelinin 0 olduğunu tartışınız.
- d. m birim potansiyelin m -düğümlü altağacın yeniden yapılandırmasına karşılayabildiğini düşünün. α -dengeli olmayan bir altağacı yeniden yapılandırmak için gereken $O(1)$ amortize zaman alacak bir $c \alpha$ bakımından ne kadar büyük olmalıdır?
- e. n -düğümlü bir α -dengeli ağaca bir düğüm araya sokmanın ya da bu ağaçtan bir düğüm silmenin $O(\lg n)$ amortize zamana mal olacağını gösterin.

17-4 Kırmızı-siyah ağaçların yeniden yapılandırılma maliyeti

Kırmızı-siyah ağaçlar üzerinde *yapısal değişiklik* yapan dört basit işlem vardır: düğüm araya sokmaları, düğüm silmeleri, döndürmeler ve renk değişiklikleri. RB-INSERT ve RB-DELETE işlemlerinin kırmızı-siyah özelliklerini korumak için sadece $O(1)$ döndürme, düğüm araya sokması ve düğüm silmesi yaptığını, ancak çok daha fazla renk değişikliği yapabileceğini görmüştük.

- a. $(n + 1)$ st düğümünü araya sokmak için yapılacak RB-INSERT çağrısının $\Omega(\lg n)$ renk değişimine sebep olacağı geçerli bir kırmızı-siyah ağacı tanımlayın. Daha sonra, belirli bir düğüm üzerinde RB-DELETE çağrısının $\Omega(\lg n)$ renk değişimine sebep olacağı n düğümlü bir geçerli kırmızı-siyah ağacı tanımlayın.

İşlem başına en kötü durum renk değişimi sayısı logaritmik olabilse de, başlangıçta boş olan bir kırmızı-siyah ağacı üzerinde yapılacak m sayıda RB-INSERT ve RB-DELETE işlemler dizisinin en kötü durumda $O(m)$ yapısal değişiklik yaratacağını kanıtlamalıyız. Şimdi her bir renk değişimini yapısal bir değişiklik olarak sayıyoruz.

- b. Hem RB-INSERT-FIXUP hem de RB-DELETE-FIXUP işlemlerinin kodlarının ana döngüsü tarafından halledilen bazı durumlar *sonlandırıcıdır (terminating)*: gerçekleştiklerinde, belli bir sabit ek işlem sayısından sonra döngünün sonlanmasına sebep olurlar. Her bir RB-INSERT-FIXUP ve RB-DELETE-FIXUP durumları için hangilerinin sonlandırıcı olup hangilerinin olmayacağını belirtin (*İpucu*: Şekil 13.5, 13.6 ve 13.7 ye bakın).

Öncelikle sadece Araya sokmaların yapıldığı durumlarda gerçekleşen yapısal değişiklikleri analiz etmeliyiz. T bir kırmızı-siyah ağaç olsun ve $\Phi(T)$ T içindeki kırmızı düğümler olacak şekilde tanımlayın. Bir birim potansiyelin RB-INSERT-FIXUP un üç durumları tarafından gerçekleştirilen yapısal değişiklikleri karşılayabileceğini varsayın.

- c. T' RB-INSERT-FIXUP işleminin T ye uygulanmasının birinci durumunun sonucu olsun. $\Phi(T') = \Phi(T) - 1$ olduğunu tartışın.
- d. RB-INSERT işlemini kullanarak bir düğümü bir kırmızı-siyah ağacına eklediğimizde, işlemi üç parçaya bölebiliriz. RB-INSERT in 1-16 Satır'larından, RB-INSERT-FIXUP un sonlandırmayan durumlarından ve RB-DELETE-FIXUP un sonlandıran durumlarından kaynaklanan yapısal değişiklikleri ve potansiyel değişimlerini listeleyin.
- e. (d) bölümünü kullanarak RB-INSERT in her çağrısı için yapılan yapısal değişikliklerin amortize zamanının $O(1)$ olduğunu tartışın.

Şimdi, hem araya sokma hem de silme olduğunda $O(m)$ yapısal değişiklik olduğunu ispatlamak istiyoruz. Her bir x düğümü için şunu tanımlayalım:

$$w(x) = \begin{cases} 0, & x \text{ kırmızı ise,} \\ 1, & x \text{ siyah ve } x \text{ kırmızı çocuğa sahip değil ise,} \\ 0, & x \text{ siyah ve } x \text{ bir kırmızı çocuğa sahip ise,} \\ 2, & x \text{ siyah ve } x \text{ 2 kırmızı çocuğa sahip ise.} \end{cases}$$

Şimdi, kırmızı-siyah T ağacının potansiyelini şu şekilde tekrar tanımlıyoruz:

$$\Phi(T) = \sum_{x \in T} w(x),$$

ve T' , RB-INSERT-FIXUP ve RB-DELETE-FIXUP işlemlerinin sonlandırmayan her durumunun uygulanmasından kaynaklanan ağaç olsun.

- f. RB-INSERT-FIXUP un sonlandırmayan her durumu için $\Phi(T') \leq \Phi(T) - 1$ olduğunu gösterin. RB-INSERT-FIXUP un her çağrısı tarafından gerçekleştirilen yapısal değişikliklerin amortize sayısının $O(1)$ olduğunu tartışın.
- g. RB-DELETE-FIXUP un sonlandırmayan her durumu için $\Phi(T') \leq \Phi(T) - 1$ olduğunu gösterin. RB-DELETE-FIXUP un her çağrısı tarafından gerçekleştirilen yapısal değişikliklerin amortize sayısının $O(1)$ olduğunu tartışın.
- h. En kötü durumda her m RB-INSERT ve RB-DELETE işlemleri dizisinin $O(m)$ yapısal değişiklik yaptığını kanıtlayarak tamamlayın.

17-5 Öne-taşıma ile kendi kendini düzenleyen listelerin rekabetçi analizi

Bir **kendi kendini-düzenleyen liste** her bir elemanın benzersiz bir anahtarının olduğu n elemanlı bağlantılı bir listedir. Listede bir elemanı aradığımızda, bilgi-sayar bize bir anahtar verir ve elemanı o anahtarla buluruz.

Bir kendi kendini düzenleyen listenin iki önemli özelliği vardır:

1. Listede anahtarı verilmiş bir elemanı bulmak için, listeyi başlangıcından anahtarı verilmiş elemanı bulana kadar dikkatle incelemeliyiz. Eğer o eleman listenin başlangıcından itibaren k ıncı elemansa, o elemanı bulmanın maliyeti k dır.
2. Verilmiş bir kural ve verilmiş bir maliyete göre, listedeki elemanları her işlemde sonra tekrar sıralayabiliriz. Listeyi nasıl yeniden sıralayacağımıza karar vermek için istediğimiz bir sezgisel yöntemi seçebiliriz.

n elemanlı bir listeyle başladığımızı ve bulmak için sırasıyla $\sigma = \langle \sigma_1, \sigma_2, \dots, \sigma_m \rangle$ anahtarlar dizisine erişimimiz olduğunu düşünün. Dizinin maliyeti, dizideki tekil erişim maliyetlerinin toplamıdır.

Bir işlemde sonra listeyi tekrar sıralamanın mümkün olan çeşitli yöntemleri arasından, problem bitişik liste elemanlarının yerlerini, her bir değiştirme işlemi için bir birim maliyet ile tersine çevirme – listedeki yerlerini değiştirme – konusuna odaklanmaktadır. Bir potansiyel fonksiyon yardımıyla, listeyi yeniden sıralamak için belirli bir sezgisel yaklaşım olan öne-taşıma yaklaşımının, diğer yaklaşımlar dizi erişimini önceden biliyor olsalar bile, listenin sırasını koruyacak diğer hiçbir yaklaşımın maliyetinin 4 katından fazla olmadığını göstereceksiniz. Bu tarz bir analize bir **rekabetçi analiz** diyoruz.

Bir H sezgisel yaklaşımı ve listenin verilmiş başlangıç dizilimi için, σ dizisinin erişim maliyetine $C_H(\sigma)$ deyin. m σ daki erişim sayısı olsun.

- a. H sezgisel yaklaşımı erişim sırasını önceden bilmiyorsa en kötü durum maliyetinin $C_H(\sigma) = \Omega(mn)$ olduğunu tartışın.

Öne-taşıma yaklaşımıyla, bir x elemanını aradıktan hemen sonra, x i listedeki ilk pozisyona (yani listenin başına) taşırız.

$\text{rank}_L(x)$, x elemanının listedeki sıralaması yani L listesinde x in pozisyonu olsun. Örneğin, eğer x listedeki dördüncü elemansa, o zaman $\text{rank}_L(x) = 4$ dür. c_i öne-taşıma yaklaşımını kullanan σ_i erişiminin maliyeti olsun ki bu maliyet elemanı listede bulma maliyeti ve onu bir dizi bitişik liste elemanının yerlerini değiştirme işlemiyle listenin önüne taşımanın maliyetini de içerir.

- b. Eğer σ_i L listesindeki x elemanına öne-taşıma yaklaşımını kullanarak erişirse, $c_i = 2 \cdot \text{rank}_L(x) - 1$ olduğunu gösterin.

Şimdi öne-taşımayı yukarıdaki iki özelliğe göre erişim dizisini işleyen her diğer H yaklaşımıyla karşılaştıracacağız. H yaklaşımı listedeki elemanları istediği gibi değiştirebilir ve hatta bütün erişim dizisini önceden bilebilir.

L_i, σ_i erişimi öne-taşıma yaklaşımını kullandıktan sonraki liste ve L_i^*, σ_i erişimi H yaklaşımını kullandıktan sonraki liste olsun. σ_i erişiminin öne-taşıma maliyetine c_i , H yaklaşımı maliyetine c_i^* diyelim. H yaklaşımının σ_i erişimi boyunca t_i^* değiştirme yaptığını varsayalım.

c. (b) bölümünde $c_i = 2 \cdot \text{rank}_{L_{i-1}}(x) - 1$ olduğunu göstermişsiniz. Şimdi $c_i^* = \text{rank}_{L_{i-1}^*}(x) + t_i^*$ olduğunu gösterin.

L_i listesindeki bir **tersine çevirme** işlemini L_i de y nin z den önce geldiği, L_i^* te de z nin y den önce geldiği bir y ve z elemanlar çiftiyle tanımlarız. L_i listesinin erişim dizisi $\langle \sigma_1, \sigma_2, \dots, \sigma_i \rangle$ i uyguladıktan sonra q_i tersi olduğunu düşünün. Bu durumda, L_i listesini gerçek bir sayıya eşleyen Φ potansiyel fonksiyonunu $\Phi(L_i) = 2q_i$ ile tanımlarız. Örneğin, eğer $L_i \langle e, c, a, d, b \rangle$ elemanlarından ve $L_i^* \langle c, a, b, d, e \rangle$ elemanlarından oluşuyorsa L nin 5 tersi $((e, c), (e, a), (e, d), (e, b), (d, b))$ vardır ve aynı zamanda $\Phi(L_i) = 10$ dır. Her i için $\Phi(L_i) \geq 0$ olduğuna ve eğer öne-taşı ve H sezgisel yaklaşımı aynı L_0 listesiyle başlarsa $\Phi(L_0) = 0$ olduğuna dikkat edin.

d. Bir yer değiştirme işleminin potansiyeli 2 artırdığına ya da 2 azalttığına açıklık getirin.

σ_i erişiminin x elemanını bulduğunu varsayın. Potansiyelin σ_i ye göre nasıl değiştiğini anlamak için, x dışındaki elemanları i inci erişimden hemen önce listede bulundukları yere göre 4 kümeye bölelim:

- A kümesi hem L_{i-1} hem de L_{i-1}^* listelerinde σ ten önce gelen elemanlardan oluşur.
 - B kümesi L_{i-1} listesinde σ ten önce gelen, L_{i-1}^* listesinde σ i takip eden elemanlardan oluşur.
 - C kümesi L_{i-1} listesinde σ i takip eden, L_{i-1}^* listesinde σ den önce gelen elemanlardan oluşur.
 - D kümesi hem L_{i-1} hem de L_{i-1}^* listelerinde x i takip eden elemanlardan oluşur.
- e. $\text{rank}_{L_{i-1}}(x) = |A| + |B| + 1$ ve $\text{rank}_{L_{i-1}^*}(x) = |A| + |C| + 1$ olduğunu açıklayın.
- f. σ_i erişim listesinin, daha önce olduğu gibi H yaklaşımının erişimi boyunca t^x yer değiştirme yaptığı durumda $\Phi(L_i) - \Phi(L_{i-1}) \leq 2(|A| - |B| + t_i^*)$ potansiyelinde bir değişime sebep olduğunu gösterin.
 σ_i erişiminin $\hat{c}_i$ amortize maliyetini $\hat{c}_i = c_i + \Phi(L_i) - \Phi(L_{i-1})$ ile tanımlayın.
- g. σ_i erişiminin $\hat{c}_i$ amortize maliyetinin $4c_i^*$ ile üstten sınırlandırıldığını gösterin.
- h. Öne-taşı kullanan σ erişim dizisinin $C_{\text{MTF}}(\sigma)$ maliyetinin diğer her H yaklaşımının $C_H(\sigma)$ maliyetinin en fazla 4 katı olduğunu her iki yaklaşımın da aynı listeye başladığını varsayarak gösteriniz.



V İleri Veri Yapıları

Giriş

Bu bölüm dinamik kümeler üzerindeki işlemleri destekleyen veri yapılarını çalış-
ma getirisi sağlar fakat, Bölüm III den daha ileri bir seviyededir. Örneğin, bölüm-
lerin ikisi Bölüm 17 de gördüğümüz amortize analiz tekniklerinin geniş kapsamlı
kullanımını sağlar.

Bölüm 18, özellikle diskler üzerinde depolanmak için tasarlanmış dengeli ara-
ma ağaçları olan B-ağaçları sunar. Diskler, rastgele-erişimli bellekten çok daha
yavaş işlem yaptığından, yalnızca dinamik-küme işlemlerinin ne kadar hesap-
lama zamanı tükettiğine bakarak değil, aynı zamanda işlem yaptıkları kaç disk
erişimi olduğuna bakarak B-ağaçların performansını ölçtük. Her B-ağaç işlemi
için disk erişimi sayısı B-ağacın yüksekliği ile birlikte artar fakat B-ağaç işlem-
leri yüksekliği düşük tutar.

Bölüm 19, INSERT, MINIMUM, EXTRACT-MIN ve UNION işlemlerini destekle-
yen bir birleştirilebilir yığın uygulamasını verir.¹ UNION işlemi iki yığını bağlar,
veya birleştirir. Fibonacci bellek yığınları – Bölüm 19 da ki veri yapısı – aynı
zamanda DELETE ve DECREASE-KEY işlemlerini de destekler. Fibonacci bellek
yığınlarının performansını hesaplamak için amortize zaman sınırlarını kullandı-
k. Fibonacci bellek yığınları üzerinde INSERT, MINIMUM ve UNION işlemleri
yalnızca $O(1)$ gerçek ve amortize zaman almıştır ve EXTRACT-MIN ve DELETE
işlemleri $O(\lg n)$ amortize zaman almıştır. Bununla birlikte, Fibonacci bellek yı-
ğınlarının en önemli avantajı, DECREASE-KEY işleminin yalnızca $O(1)$ amortize
zaman almasıdır. DECREASE-KEY işlemi sabit amortize zaman aldığından, Fibo-

¹Problem 10-2 de olduğu gibi, MINIMUM ve EXTRACT-MIN işlemlerini desteklemesi için bir birleştirilebilir yığın tanımladık ve böylece aynı zamanda bunu *birleştirilebilir min-bellek yığını* olarak atayabiliriz. Bundan farklı olarak, eğer MAXIMUM ve EXTRACT-MAX işlemleri desteklenirse, bu *birleştirilebilir max-bellek yığını* olarak alınacaktır. Aksini açıkça belirtmedikçe, birleştirilebilir bellek yığınları birleştirilebilir min-bellek yığınları olarak dikkate alınacaktır.

nacci yığınları bugüne kadarki çizge problemleri için asimptotik olarak en hızlı algoritmaların bazılarının anahtar bileşenleridir.

Sınırlı bir aralıkta anahtarlar tam sayı olduğunda $\Omega(n \lg n)$ alt sınırını aşabileceğimizi kaydederek, Bölüm 20 anahtarlar sınırlı bir aralıkta tam sayılar olduğunda $o(\lg n)$ zamanda SEARCH, INSERT, DELETE, MINIMUM, MAXIMUM, SUCCESSOR ve PREDECESSOR dinamik-küme işlemlerini destekleyen bir veri yapısı tasarlayıp tasarlayamayacağımızı sorar. Cevap olarak, van Emde Boas ağacı olarak bilinen bir özyinelemeli veri yapısını kullanarak yapabildiğimiz ortaya çıkar. $u \geq 2$ nin tam bir kuvveti olmak üzere, anahtarlar $\{0, 1, 2, \dots, u - 1\}$ kümesinden çekilmiş tek türlü tam sayılar ise, o zaman van Emde Boas ağaçları $O(\lg \lg u)$ zamanda yukarıdaki işlemlerin her birini destekler.

Son olarak, Bölüm 21 ayrık kümeler için veri yapıları sunar. Dinamik kümeler içine bölünmüş bir n elemanlı evrene sahibiz. İlk olarak, her eleman kendi tek öğeli kümesine aittir. UNION işlemi iki kümeyi birleştirir ve bu anda FIND-SET sorgusu belli bir elemanı içeren tek türlü kümeyi belirler. Her kümeyi kökü olan basit bir ağaç olarak belirleyerek, şaşırtıcı şekilde hızlı işlemler elde ederiz: $\alpha(n)$ inanılmaz şekilde yavaş biçimde artan bir fonksiyon olmak üzere, m işlemler dizisi $O(m \alpha(n))$ zamanda çalışır — $\alpha(n)$ her olası uygulamada en fazla 4 tür. Bu zaman sınırını ortaya koyan amortize analizi karmaşık olduğu kadar bir veri yapısı olarak basittir.

Bu bölümde kapsanmış başlıklar kesinlikle yalnızca “ileri” veri yapılarının örnekleri değildir. Diğer ileri veri yapıları aşağıdakileri içerir:

- **Dinamik ağaçlar**, Sleator ve Tarjan [319] tarafından sunulmuş ve Tarjan [330] tarafından tartışılmıştır, bir ayrık köklü ağaçlar ormanını sürdürür. Her ağaçtaki her bir ayrıntı gerçek-değerli bir maliyete sahiptir. Dinamik ağaçlar ataları, kökleri, ayrıntı maliyetlerini ve basit bir yol üzerinde tepedeki bir düğümden bir köke kadar minimum ayrıntı maliyetini bulmak için sorguları destekler. Ağaçlar, ayrıntılar kesilerek, tepedeki bir düğümden köke kadar olan basit bir yol üzerindeki tüm ayrıntı maliyetleri güncellenerek, başka bir ağaç içindeki bir köke bağlanarak ve bir düğümü içinde bulunduğu ağacın bir kökü yapılarak işlenmiş olabilir. Dinamik ağaçların bir uygulaması her işlem için $O(\lg n)$ amortize zaman sınırını verir; daha komplike bir uygulama $O(\lg n)$ en kötü-durum zaman sınırlarına uyum sağlar. Dinamik ağaçlar asimptotik olarak en hızlı ağ-akış algoritmalarının bazılarında kullanılır.
- **Yayvan ağaçlar**, Sleator ve Tarjan [320] tarafından geliştirilmiş ve tekrar, Tarjan [330] tarafından tartışılmıştır, $O(\lg n)$ amortize zamanda çalışan standart arama ağacı işlemleri üzerine ikili arama ağacının bir formudur. Yayvan ağaçların bir uygulaması dinamik ağaçları sadeleştirir.
- **Kalıcı** veri yapıları sorgulara ve bir veri yapısının eski versiyonlarında olduğu gibi bazen güncellemelere olanak sağlar. Driscoll, Sarnak, Sleator ve Tarjan [97] yalnızca küçük bir zaman ve alan maliyeti ile bağlantılı veri yapılarını

kalıcı yapmak için teknikler sunar. Problem 19-1 kalıcı dinamik bir kümenin basit bir örneğini verir.

- Bölüm 20 de olduğu gibi, birkaç veri yapısı anahtarların sınırlı bir evreni için sözlük işlemlerinin (INSERT, DELETE ve SEARCH) daha hızlı bir uygulamasına olanak sağlar. Bu kısıtların avantajını elde ederek, karşılaştırma-tabanlı veri yapılarından daha iyi en kötü-durum asimptotik çalışma zamanları elde edebilirler. Fredman ve Willard, evren tam sayılara daraltıldığında daha hızlı sözlük işlemlerine olanak sağlaması amacıyla ilk veri yapısı olan **füzyon ağaçlarını** [115] sunmuştur. Onlar bu işlemleri $O(\lg n / \lg \lg n)$ zamanda nasıl uygulanacağını göstermişlerdir. Bundan sonraki **üssel arama ağaçlarını** içeren birkaç veri yapısı aynı zamanda sözlük işlemlerinin bazıları veya tümü üzerinde iyileştirilmiş sınırlar vermiştir ve bu kitap boyunca bölüm notlarında bahsedilmektedir.
- **Dinamik çizge veri yapıları** silme veya ekleme işlemleri vasıtasıyla tepeler veya ayrıtları değiştirmek için bir çizge yapısına olanak sağlarken, çeşitli sorguları destekler. Desteklediği sorgu örnekleri tepe bağlantısı [166], ayrıt bağlantısı, minimum örten ağaçlar [165], çift bağlanabilirlik ve geçişli kapanmayı [164] içerir.

Bu kitap boyunca bölüm notları ek veri yapılarından bahseder.

B-ağaçları diskler ya da diğer doğrudan erişimli ikincil depolama cihazları üzerinde iyi çalışması için tasarlanmış dengeli arama ağaçlarıdır. B-ağaçları kırmızı-siyah ağaçlara benzer (Bölüm 13), fakat disk giriş/çıkış işlemlerini en aza indirmede daha iyilerdir. Birçok veritabanı sistemleri bilgi saklamak için B-ağaçlarını ya da B-ağaçları türevlerini kullanmaktadırlar.

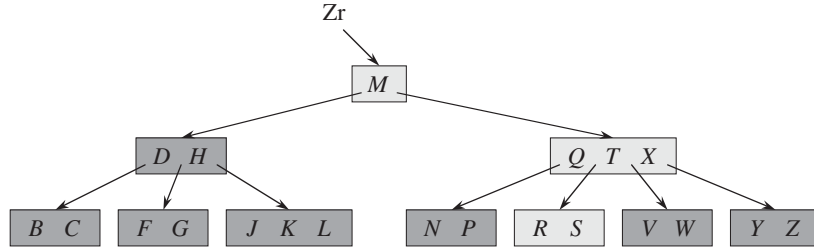
B-ağaçlarının kırmızı-siyah ağaçlardan farkı B-ağaçlarının düğümlerinin birkaç taneden binlerce taneye erişebilen sayıda çocuklarının olabilmesidir. Diğer bir deyişle, bir B-ağacının “dallanma faktörü”, genellikle kullanılan disk biriminin özelliklerine bağlı olmasına rağmen, oldukça geniş olabilir. B-ağaçları, her n -düğümlü B-ağacının yüksekliğinin $O(\log(n))$ olması dolayısıyla kırmızı-siyah ağaçlara benzerlik gösterirler. Bir B-ağacının tam yüksekliği bir kırmızı-siyah ağacın yüksekliğinden çok daha az olabilir, ancak, dallanma faktöründen ve yüksekliğini ifade eden logaritma tabanından dolayı daha büyük de olabilir. Bu yüzden B-ağaçları birçok dinamik-küme işlemini $O(\log(n))$ zamanda gerçekleştirebilmek için kullanılabilir.

B-ağaçları doğal anlamda ikili arama ağaçlarını genelleştirirler. Şekil 18.1 basit bir B-ağacını göstermektedir. Bir B-ağacının iç düğümü olan x , $x.n$ tane anahtar içerirse, x düğümünün $x.n+1$ tane çocuk düğümü vardır. x düğümündeki anahtarlar, her biri x in bir çocuk düğümünü ele alacak olan bu anahtarları $x.n+1$ alt aralığa ayırmak için belirlenecek aralıkları tespit edecek bölünme noktaları olarak görev alırlar. Bir B-ağacında anahtar aranırken, x düğümünde saklanan $x.n$ anahtar ile karşılaştırmaya bağlı olarak $(x.n + 1)$ yönlü karar verilir. Yaprak düğümlerin yapısı iç düğümlerden farklıdır (bu farklar Bölüm 18.1 de incelenecektir).

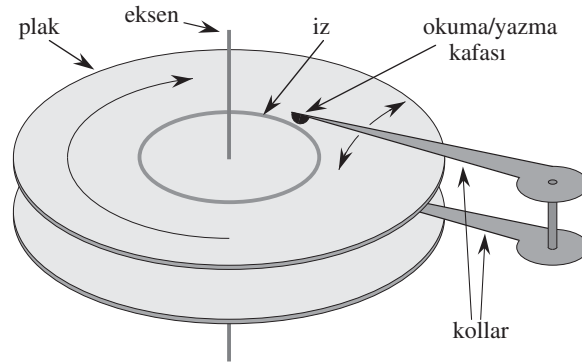
Kısım 18.1 B-ağaçlarının kesin bir tanımını vermekte ve B-ağaçlarının yüksekliğinin içerdiği düğüm sayısı ile sadece logaritmik olarak büyüdüğünü kanıtlamaktadır. Kısım 18.2 bir anahtarın B-ağacında nasıl arandığını ve B-ağacına nasıl eklendiğini açıklamakta ve Kısım 18.3 ise silme işleminden söz etmektedir. Başlamadan önce, neden ana sıradan-erişimli bellek üzerinde çalışmak üzere tasarlanmış veri yapılarından farklı olarak bir disk üzerinde çalışmak üzere tasarlanmış veri yapılarının incelendiğinin sorulması gereklidir.

İkincil depolama üzerindeki veri yapıları

Bilgisayar sistemleri bellek kapasitesi sağlayan çeşitli teknolojilerin avantajına sahiptirler. Bir bilgisayar sisteminin *birincil belleği* (ya da *ana hafızası*) normal-



Şekil 18.1 Anahtarları İngilizcedeki sessiz harfler olan bir B-ağacı. $x.n$ anahtarı olan iç düğüm x , $x.n + 1$ adet çocuğa sahiptir. Ağaçtaki tüm yapraklar aynı derinlikte yer alır. Açık renk düğümler R harfinin aranmasında incelenen düğümlerdir.



Şekil 18.2 Tipik bir disk sürücüsü. Bir mil etrafında dönen bir veya daha fazla sayıda (burada iki tane gösterilmiştir) plaktan oluşur. Her plak, bir kolun ucunda bulunan bir kafa ile okunur ve yazılır. Kollar ortak sabit bir eksen etrafında dönerler. Okuma/yazma kafası sabit iken altından geçen alana iz adı verilir.

de silikon bellek çiplerinden oluşur. Bu teknoloji teyp ya da diskler gibi manyetik depolama teknolojisinden çok, saklanan bit başına daha pahalı bir maliyete sebep olmaktadır. Birçok bilgisayar sisteminde, manyetik disklerle dayalı ikincil depolama da bulunur ve bunun gibi **ikincil depolama** miktarları en az karesi kadar büyüklükte olacak kadar birincil bellek miktarını aşmaktadır.

Şekil 18.2 tipik bir disk sürücüsünü göstermektedir. Sürücü bir **eksen** etrafında sabit hızda dönen bir ya da daha fazla sayıda **plaklar** içerir. Her bir plak manyetize edilebilen bir kaplama ile kaplanmıştır. Sürücü bir **kolun** ucunda bulunan **kafa** yardımı ile okuma ve yazma işlemini gerçekleştirir. Kollar kafaları mile

veya dışa doğru hareket ettirebilir. Belirli bir kafa hareketsizken kafanın altından geçen yüzeye *iz* adı verilir. Çok plak olması disk sürücüsünün sadece kapasitesini artırır, performansını artırmaz.

Diskler, ana hafızadan daha ucuz ve daha yüksek kapasiteli olmalarına rağmen, hareket eden mekanik kısımlarından dolayı çok çok yavaşlardır¹. Mekanik hareketin iki bileşeni vardır: plağın dönüşü ve kol hareketi. Bu metin yazıldığı sırada ticari diskler dakikada 5400–15,000 dolanım (RPM) dönüş hızında çalışmaktadırlar. 15,000 RPM hızlar genellikle sunucu düzeyli sürücülerde, 7200 RPM hızlar masaüstü bilgisayar sürücülerinde ve 5400 RPM hızlar ise dizüstü bilgisayarlarda görülmektedirler. 7200 RPM hızlı görünmesine rağmen bir tur 8.33 milisaniye sürer ki bu da silikon bellekler için bulunan yaklaşık 50 nanosaniyelik erişim zamanından 5 mertebeden (10^5) daha uzundur. Diğer bir deyişle belirli bir kaydın okuma/yazma kafası altına gelmesi için bir tam tur beklemek için geçen sürede ana hafızaya 100.000 kereden fazla sayıda erişilebilir. Ortalama olarak yarım tur beklenilmektedir ancak yine de disklerle silikon belleklerin erişim zamanları karşılaştırıldığında aradaki fark muazzamdır.

Mekanik hareketleri beklerken geçen zamanı telafi etmek için diskler aynı anda bir değil birden çok nesneye erişir. Bilgi, izlerde ardışık olarak görünen eşit büyüklükteki bit *sayfalarına* bölünmüştür ve her disk okuma ve yazması bir ya da daha fazla tam sayfasını içerir. Tipik bir disk için bir sayfanın uzunluğu 2^{11} – 2^{14} byte arasında değişebilir. Okuma/yazma kafası doğru olarak yerleştiğinde ve disk istenen sayfanın başlangıcına döndüğünde, okuma ve yazma işlemi tamamen elektronik olarak gerçekleşir ve böylece disk hızlı bir şekilde büyük miktarlardaki veriyi okuyup yazabilir.

Sıklıkla, bir bilgi sayfasına erişim ve diskten okumak, okunan bütün bilgiyi gözden geçirmekten daha uzun sürer. Bu sebeple, bu bölümde çalışma zamanının iki temel bileşeni ayrı ayrı incelenecektir:

- Disk erişim sayısı ve
- CPU (hesaplama) zamanı

Disk erişim sayıları diskten okunması ya da diske yazılması gereken bilgi sayfalarının sayısı cinsinden ölçülür. Disk erişim zamanı sabit değildir—mevcut iz ile istenen iz arasındaki uzaklığa ve diskin başlangıçtaki döngüsel konumuna

¹Bu kitap yazılırken Solid-State sürücüler (SSD) tüketici marketlerine yeni gelmekteydi. SSD sürücüler mekanik disk sürücülerinden daha hızlı olmalarına karşın, gigabyte başına maliyetleri daha fazladır ve mekanik disk sürücülerinden daha düşük kapasiteye sahiplerdir.

bağlıdır. Yine de disk erişiminde toplam harcanan sürenin ilk-sıra yaklaşımında okunan ya da yazılan sayfa sayısı kullanılacaktır.

Tipik bir B-ağacı uygulamasında, ele alınan veri miktarı o kadar büyüktür ki tüm veri aynı anda ana belleğe sığmamaktadır. B-ağacı algoritmaları seçilen sayfaları gerektiğinde diskten ana belleğe kopyalar ve değişen sayfaları diske geri yazar. B-ağacı algoritmaları bir anda sadece belirli sayıda sayfayı ana bellekte tutar; bu yüzden ana belleğin büyüklüğü uygulanacak olan B-ağacının büyüklüğünü kısıtlamaz.

Disk işlemleri kabakodda aşağıdaki gibi modellenmektedir. x bir nesnenin işaretçisi olarak varsayalım. Eğer nesne mevcut durumda ana bellekte ise, nesnenin özelliklerine her zamanki gibi başvurulabilir: örneğin $x:key$ gibi. Eğer x ile ilişkilendirilen nesne disk üzerinde ise, bu durumda, özelliklerine erişebilmeden önce x nesnesini ana belleğe okuyabilmek için $DISK-READ(x)$ işlemi gerçekleştirilmelidir. (x zaten ana bellekte ise, o zaman $DISK-READ(x)$ disk erişimine gerek duymaz: “işlem-yok” olduğu varsayılmaktadır. Benzer olarak, $DISK-WRITE(x)$ işlemi de x nesnesinin özelliklerinde yapılan değişiklikleri saklamak için kullanılır. Yani bir nesne ile çalışmanın tipik şekli aşağıdaki gibidir:

```

 $x$  = bir nesnenin işaretçisi
DISK-READ( $x$ )
 $x$  in özelliklerine erişen ve/veya değişiklik yapan işlemler
DISK-WRITE( $x$ )      //  $x$  in özelliklerinde değişiklik yoksa göz ardı edilir
 $x$  e erişen fakat özelliklerini değiştirmeyen işlemler

```

Sistem aynı anda sadece sınırlı sayıda sayfayı ana bellekte tutabilir. Sistemin artık kullanılmayan sayfaları bellekten sileceği varsayılmaktadır; burada işlenen B-ağacı algoritmaları bu konuyu göz ardı etmektedir.

Birçok sistemde B-ağacı algoritmasının çalışma zamanı temel olarak gerçekleştirdiği $DISK-READ$ ve $DISK-WRITE$ işlemi sayısına bağlı olmasından dolayı, bu işlemlerin mümkün olduğunca çok bilgi okuyup yazması istenir. Bu sebeple bir B-ağacı düğümü genellikle bir disk sayfası kadar büyüktür ve bu büyüklük de B-ağacı düğümünün sahip olabileceği çocuk sayısını kısıtlamaktadır.

Diskte saklanan büyük bir B-ağacı için dallanma faktörü, sayfa büyüklüğüne göre anahtarın büyüklüğüne dayandığından sıklıkla 50 ile 2000 arasında görülmektedir. Büyük dallanma faktörü hem ağacın yüksekliğini hem de herhangi bir anahtarı bulmak için gerekli disk erişim sayısını önemli ölçüde azaltır. Şekil 18.3 bir milyanın üzerinde anahtar saklayabilen, 2 yükseklik değerine ve 1001 dallanma faktörüne sahip bir B-ağacını göstermektedir; bununla birlikte bu ağaçta en fazla 2 disk erişimi ile herhangi bir anahtar bulunabilir.

3. $x.key$ anahtarları her bir altağaçta depolanan anahtar aralıklarını ayırır: k_i kökü $x.c_i$ olan bir altağaçta saklanan bir anahtar ise

$$k_1 \leq x.key \leq x.key \leq \dots \leq x.key \leq k_{x.n+1}.$$

4. Bütün yapraklar aynı yüksekliğe sahiptirler, o da ağacın yüksekliği olan h değeridir.
5. Düğümlerin içerebileceği anahtar sayısında üst ve alt limitler vardır. Bu sınırlar B-ağacının **en küçük derecesi** olarak adlandırılan sabit bir tamsayı $t \geq 2$ ile ifade edilir:
- a. Kök dışında her düğümün en az $t - 1$ adet anahtarı bulunmalıdır. Bu yüzden kök dışındaki her iç düğüm t adet çocuk düğüme sahiptir. Eğer ağaç boş değilse kökte en az bir anahtar bulunmalıdır.
 - b. Her düğüm en fazla $2t - 1$ adet anahtar içerebilir. Bu sebeple bir iç düğümün en çok $2t$ çocuğu olabilir. Bir düğüm tam olarak $2t - 1$ adet anahtar içeriyorsa bu düğüm **tam** olarak adlandırılır².

En basit B-ağacı $t = 2$ olduğu zaman oluşur. Bu durumda her iç düğümün 2, 3 ya da 4 çocuğu olur böylece **2-3-4 ağacı** elde edilmiş olur. Pratikte t nin daha büyük değerleri, küçük yükseklikli B-ağaçlarını meydana getirirler.

B-ağacının yüksekliği

Bir B-ağacındaki birçok işlem için gerekli disk erişim sayısı, B-ağacının yüksekliği ile orantılıdır. Şimdi bir B-ağacının en-kötü yüksekliği analiz edilecektir.

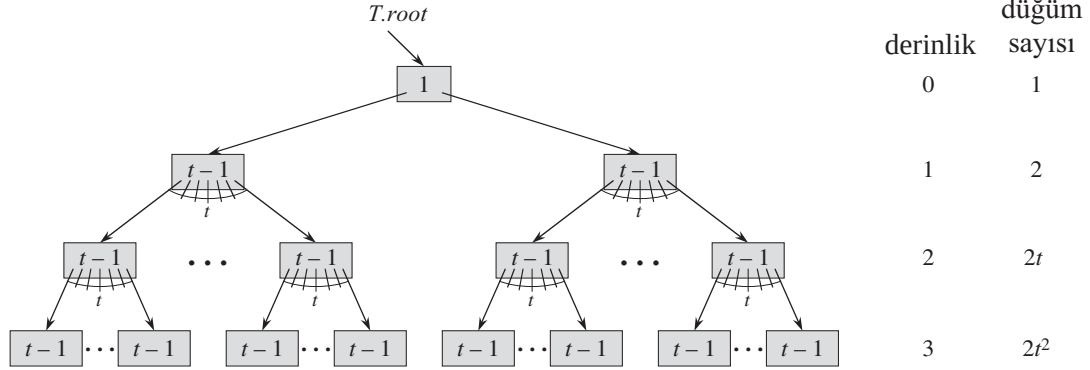
Teorem 18.1

Eğer $n \geq 1$ ise, en-küçük derecesi $t \geq 2$ olan, h yüksekliğindeki n -anahtarlı T B-ağacı için

$$h \leq \log_t \frac{n + 1}{2}.$$

Kanıt T B-ağacının kökü en az bir anahtar ve tüm diğer düğümler en az $t - 1$ anahtar içerir. Bu yüzden yüksekliği h olan T 1 derinliğinde en az 2 düğüm, 2 derinliğinde en az $2t$ düğüm, 3 derinliğinde en az $2t^2$ düğüm vs. bulunmaktadır. h derinliğine kadar en az $2t^{h-1}$ adet düğüm bulunur. Yüksekliği $h = 3$ olan böyle bir ağaç Şekil 18.4 de gösterilmektedir. Bu sebeple anahtar sayısı n aşağıdaki

²B-ağaçlarının **B*-ağacı** olarak bilinen bir türevi her ara düğümün en az 2/3 dolu olmasını gerektirir. (B-ağaçları en azından yarı dolu olmasını gerektirir.)



Şekil 18.4 3 yüksekliğinde olası en az sayıda anahtar içeren bir B-ağacı. Her x düğümünün içinde gösterilen $x.n$ dir.

eşitsizliği sağlamaktadır:

$$\begin{aligned}
 n &\geq 1 + (t-1) \sum_{i=1}^h 2t^{i-1} \\
 &= 1 + 2(t-1) \left(\frac{t^h - 1}{t - 1} \right) \\
 &= 2t^h - 1.
 \end{aligned}$$

Basit cebir ile $t^h \leq (n+1)/2$ elde edilebilir. Her iki tarafın t tabanında logaritmaları alındığında teorem kanıtlanmış olur. ■

Bu durumda kırmızı-siyah ağaçlarla karşılaştırıldığında B-ağaçlarının gücü görülmektedir. Bir çok durumda ağacın yüksekliği $O(\log(n))$ olarak büyüse de (t nin sabit olduğunu hatırlayın) B-ağaçlarının logaritma tabanı kat be kat büyük olabilir. Bu yüzden B-ağaçları kırmızı-siyah ağaçlara göre birçok ağaç işleminde ele alınan düğüm sayısı açısından $\lg t$ kat kazanç sağlar. Ağaçtaki herhangi bir düğümü değerlendirmek için genellikle diske erişim gerektiğinden B-ağaçları önemli sayıda disk erişiminden kaçınabilmektedir.

Alıştırmalar

18.1-1

Neden minimum derece $t = 1$ olmasına izin verilmez?

18.1-2

Şekil 18.1 deki ağacın geçerli bir B-ağacı olması için t hangi değerleri almalıdır?

18.1-3

$\{1, 2, 3, 4, 5\}$ ifade eden minimum derecesi 2 olan tüm geçerli B-ağaçlarını gösteriniz.

18.1-4

Minimum derece t nin bir fonksiyonu olarak h yüksekliğindeki bir ağacında saklanabilecek en yüksek anahtar sayısı nedir?

18.1-5

Bir kırmızı-siyah ağaçta her siyah düğüm kendi kırmızı çocuklarını ve onun çocuklarını yutsaydı sonuçta oluşacak veri yapısını tanımlayınız.

18.2 B-ağaçları üzerinde temel işlemler

Bu kısımda B-TREE-SEARCH, B-TREE-CREATE ve B-TREE-INSERT işlemlerinin detayları sunulmaktadır. Bu prosedürlerde 2 kabule uyulmaktadır:

- B-ağacının kökü her zaman ana bellektedir, bu yüzden kök üzerinde DISK-READ işlemine gerek yoktur. Ancak kök düğümün değişmesi durumunda kök için DISK-WRITE işlemi gerçekleştirilmek zorundadır.
- Parametre olarak aktarılan düğümler üzerinde DISK-READ işlemi yapılmış olmalıdır.

Burada gösterilen prosedürlerin hepsi, ağacın kökünden altlara doğru ilerleyen geriye dönmeyi gerektirmeyen “tek-geçiş” algoritmalarıdır.

B-ağacında arama

B-ağaçlarında arama ikili ağaçlarda aramaya benzer, tek farkı her düğümde ikili ya da iki yönlü dallanma kararı vermek yerine düğümün çocuk sayısına göre çok yönlü dallanma kararı verilmesidir. Tam olarak her iç düğüm x de $(x.n + 1)$ yönlü dallanma kararı verilir.

B-TREE-SEARCH prosedürü ikili arama ağaçları için tanımlanan TREE-SEARCH prosedürünün genelleştirilmiştir. B-TREE-SEARCH, girdi olarak bir altağacın kök düğümü x i gösteren işaretçi ve o altağaçta aranacak olan anahtar k yı alır. Bu sebeple bu prosedürün en üst çağırımı B-TREE-SEARCH($T.root, k$) biçimindedir. Eğer k B-ağacı içinde bulunuyorsa, B-TREE-SEARCH prosedürü k yı içeren düğüm olan y ve indis i den ($y.key = k$) oluşan (y, i) sıralı ikilisini döndürür. Aksi halde prosedüre NIL değeri döndürür.

```

B-TREE-SEARCH ( $x, k$ )
1   $i = 1$ 
2  while  $i \leq x.n$  ve  $k > x.key_i$ 
3       $i = i + 1$ 
4  if  $i \leq x.n$  ve  $k = x.key_i$ 
5      return ( $x, i$ )
6  else eğer  $x.leaf$ 
7      return NIL
8  else DISK-READ( $x, c_i$ )
9      return TREE-SEARCH ( $x.c_i, k$ )

```

Bir doğrusal arama prosedürü kullanılırsa 1–3. satırlar, $k \leq x.key$ koşulunu sağlayan en küçük indis i yi bulur, aksi durumda i değeri $x.n + 1$ değerini alacak şekilde ayarlanır. 4–5. satırlar anahtar değerin bulunup bulunmadığını kontrol eder, bulunduyorsa sonuç döndürülür. Diğer durumlarda 6–9. satırlar ya aramayı başarısız olarak sonlandırır (eğer x yaprak düğümse), ya da o çocuk düğüm üzerinde gerekli DISK-READ işlemini gerçekleştirdikten sonra aramayı x in uygun altağaçlarında devam ettirir.

Şekil 18.2 B-TREE-SEARCH işlemini örnekle açıklamaktadır. Prosedür, R anahtarını arama sırasında açık renkle işaretlenmiş düğümleri incelemektedir.

İkili arama ağaçlarındaki TREE-SEARCH prosedüründe olduğu gibi özyineleme sırasında ele alınan düğümler ağacın kökünden alt seviyelere doğru basit bir yol oluştururlar. Bu yüzden B-TREE-SEARCH prosedürü $O(h) = O(\log_t n)$ disk sayfasına erişir (h , B-ağacının yüksekliğini, n ise B-ağacındaki anahtar sayısını temsil etmektedir). $x.n < 2t$ olduğu için 2–3. satırlardaki (**while**) döngüleri her bir düğüm için $O(t)$ kadar zaman alır ve toplam CPU zamanı da $O(th) = O(t \log_t n)$ kadardır.

Boş bir B-ağaç oluşturma

Bir T B-ağacını oluşturmak için ilk önce boş bir kök düğüm oluşturmak üzere B-TREE-CREATE kullanılır ve daha sonra yeni anahtarlar eklemek için B-TREE-INSERT çağrılır. Her iki prosedür de $O(1)$ zamanda yeni bir düğüm olarak kullanılacak disk sayfasını tahsis eden ALLOCATE-NODE yardımcı prosedürünü kullanır. Geçerli düğüm için henüz kullanılabilir bilgi olmadığından ALLOCATE-NODE tarafından yaratılmış bir düğümün DISK-READ işlemine ihtiyaç duymadığı varsayılabilir.

```

B-TREE-CREATE( $T$ )
1   $x = \text{ALLOCATE-NODE}()$ 
2   $x.leaf = \text{TRUE}$ 
3   $x.n = 0$ 
4  DISK-WRITE ( $x$ )
5   $T.root = x$ 

```

B-TREE-CREATE için $O(1)$ disk işlemi ve $O(1)$ CPU zamanı gerekmektedir.

B-ağacına bir anahtar ekleme

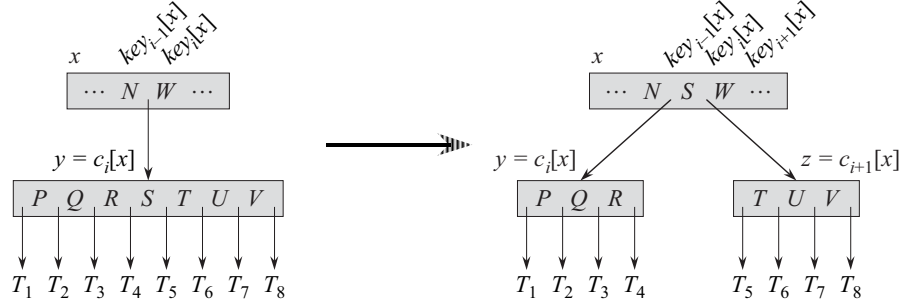
B-ağacına bir anahtar ekleme, ikili arama ağacına bir anahtar ekleme işleminden önemli derecede daha karmaşık bir işlemdir. İkili arama ağaçlarında olduğu gibi yeni anahtarın ekleneceği bir yaprak konumu araştırılır. Ancak bir B-ağacında, basitçe bir yaprak düğüm oluşturulup içine anahtar ekleme yapılamaz, bu durumda ağaç, geçerli bir B-ağacı olmayacaktır. Onun yerine yeni anahtar var olan bir yaprak düğüme eklenir. Tam olan bir yaprak düğüme anahtar eklenemeyeceği için, tam olan ($2t - 1$ sayıda anahtarı olan) bir y yaprak düğümünü **medyan anahtarı** olan $y.key_t$ den bölerek her birinde $t - 1$ adet anahtar bulunan iki düğüm oluşturacak **ayırma** işlemi tanımlanmalıdır. Medyan anahtar, iki yeni ağacın bölünme noktasını belirtmek üzere y nin ebeveyn düğüme çıkar. Ancak eğer y nin ebeveynide tam ise o da yeni anahtar eklenmeden önce bölünmelidir ve bu yüzden tüm tam düğümlerin bölünme işleminin ağacın en üstüne kadar yapılmasını gerektirebilir.

İkili arama ağaçlarında olduğu gibi yeni anahtar B-ağacına kökten yaprağa kadar tek geçişte eklenebilir. Bunu yapmak için ekleme yapmak amacıyla gerçekten tam bir düğümün bölünmesi gerekliliğinin bulunması için beklenmez. Onun yerine, yeni anahtarın ait olduğu konumu bulmak için ağaç üzerinde aşağıya doğru dolaşırken, yaprak dâhil yol üzerinde karşılaşılan her dolu düğüm bölünür. Bu yüzden dolu düğüm y nin bölünmesi istendiğinde ebeveyninin dolu olmadığı garanti edilmiş olur.

B-ağacında bir düğümü ayırma

B-TREE-SPLIT-CHILD prosedürü girdi olarak ana bellekte olduğu varsayılan tam olmayan iç düğüm x i ve yine ana bellekte olduğu varsayılan ve x in çocuklarından dolu olan $x.c_i$ düğümünün indis değeri i yi alır. Prosedür bu çocuk düğümü ikiye ayırır ve x i ek bir çocuğu olacak şekilde düzenler. Tam bir düğümü bölmek için ilk önce kök yeni bir boş kökün çocuğu yapılır, böylece B-TREE-SPLIT-CHILD kullanılabilir. Bu sayede ağacın yüksekliği 1 artmış olur; bölünme ağacın büyüdüğü anlamına gelir.

Şekil 18.5 bu süreci göstermektedir. Tam düğüm $y = x.c_p$, y nin üst düğümü olan x e taşınan medyan anahtarı S den bölünür. y düğümündeki medyan anahtardan büyük olan anahtarlar, x in yeni çocuğu olan yeni düğüm z ye taşınır.



Şekil 18.5 $t = 4$ olan bir düğümün ayrımı. Düğüm $y = x.c_i$ ve z olarak iki düğüme ayrılır ve y nin medyan anahtarı S , üst düğüm olan y nin ebeveynine taşınır.

B-TREE-SPLIT-CHILD(x, i)

```

1   $z = \text{ALLOCATE-NODE}()$ 
2   $y = x.c_i$ 
3   $z.\text{leaf} = y.\text{leaf}$ 
4   $z.n = t - 1$ 
5  for  $j = 1$  to  $t - 1$ 
6       $z.\text{key}_j = y.\text{key}_{j+t}$ 
7  if not  $y.\text{leaf}$ 
8      for  $j = 1$  to  $t$ 
9           $z.c_j = y.c_{j+t}$ 
10  $y.n = t - 1$ 
11 for  $j = x.n + 1$  downto  $i + 1$ 
12      $x.c_{j+1} = x.c_j$ 
13  $x.c_{i+1} = z$ 
14 for  $j = x.n$  downto  $i$ 
15      $x.\text{key}_{j+1} = x.\text{key}_j$ 
16  $x.\text{key}_i = y.\text{key}_t$ 
17  $x.n = x.n + 1$ 
18 DISK-WRITE( $y$ )
19 DISK-WRITE( $z$ )
20 DISK-WRITE( $x$ )

```

B-TREE-SPLIT-CHILD doğrudan “kesip yapıştırarak” çalışır. Burada x ayrılan düğüm ve y de (2. satırda tanımlanan) x in i . çocuğudur. y düğümünde $(2t - 1)$ anahtarlı $2t$ adet çocuk vardır ancak bu işlem ile $(t - 1)$ anahtarlı t adet çocuğa indirgenmiştir. z düğümü y den en büyük t çocuğu alır ve x in çocuk tablosunda y

den hemen sonra yerleşen çocuğu olur. y düğümünün medyan anahtarı x de y ve z yi ayıran anahtar olmak üzere üste (x e) çıkar.

1-9. satırlar z düğümünü oluşturur ve en büyük $t - 1$ anahtar ile y nin karşılık gelen y adet çocuğunu z ye verir. 10. satır y için anahtar sayısını düzenler. Son olarak 11-17. satırlar z yi x in bir çocuğu olarak ekler, medyan anahtarı y yi z den ayırmak için y den x e taşır ve x in anahtar sayısını ayarlar. 18-20. satırlar düzenlenen tüm disk sayfalarını yazarlar. B-TREE-SPLIT-CHILD tarafından kullanılan CPU süresi 5-6 ve 8-9. Satırlardaki döngülerden dolayı $\Theta(t)$ kadardır (Diğer döngüler $O(t)$ iterasyon için çalışır). Prosedür $O(1)$ disk işlemi gerçekleştirir.

Bir B-ağacına ye ağaçtan aşağıya tek geçişte anahtar ekleme

h yüksekliğindeki bir T B-ağacına bir k anahtar eklenmek istendiğinde $O(h)$ disk erişimi gereklidir. Gereken CPU zamanı ise $O(th) = O(t \log_e n)$ dir. B-TREE-INSERT prosedürü dolu bir düğüme inmemeyi garantilemek için B-TREE-SPLIT-CHILD prosedürünü kullanır.

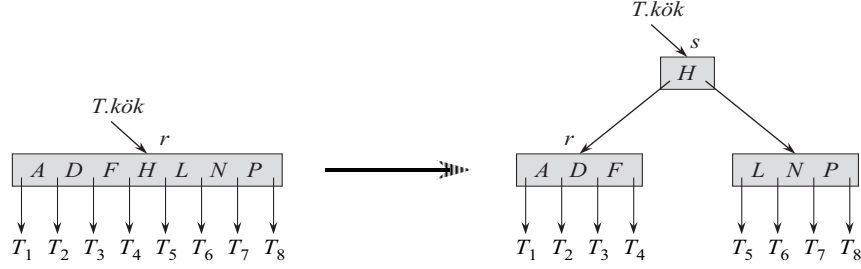
B-TREE-INSERT (T, k)

```

1   $r = T.root$ 
2  if  $r.n == 2t - 1$ 
3       $s = \text{ALLOCATE-NODE}()$ 
4       $T.root = s$ 
5       $s.leaf = \text{FALSE}$ 
6       $s.n = 0$ 
7       $s.c_1 = r$ 
8      B-TREE-SPLIT-CHILD ( $s, 1$ )
9      B-TREE-INSERT-NONFULL ( $s, k$ )
10 else B-TREE-INSERT-NONFULL( $r, k$ )
```

3-9. satırlar kök düğüm r nin tam olduğu durumları ele alırlar: kök düğüm ayrılır ve iki çocuğu olan yeni bir düğüm s kök olur. Kök düğümü bölmek bir B-ağacının yüksekliğini artırmanın tek yoludur. Şekil 18.6 bu durumu göstermektedir. İkili arama ağacından farklı olarak B-ağacının yüksekliği en alttan değil, en üstten artırılır. Prosedür k anahtarını tam dolu olmayan kök düğümden başlayan ağaca eklemek için B-TREE-INSERT-NONFULL prosedürünü çağırarak sonlanır. B-TREE-INSERT-NONFULL prosedürü, her seferinde yinelendiği düğümün dolu olmadığını sağlamak için gerektiğinde B-TREE-SPLIT-CHILD prosedürünü çağırarak ağaçta altlara doğru kendini geri çağırır.

Yardımcı özyinelemeli prosedür B-TREE-INSERT-NONFULL, k anahtarını, prosedür çağrıldığında tam olmadığı varsayılan x düğüme ekler. B-TREE-INSERT işlemi ve özyinelemeli B-TREE-INSERT-NONFULL işlemi bu varsayımın doğru olduğunu garantilemektedir.



Şekil 18.6 $t = 4$ olan kökün ayrılması. Kök düğüm r ikiye ayrıştırılır ve yeni kök düğüm s oluşturulur. Yeni kök r nin medyan anahtarını içerir ve r nin iki yarısını çocuk olarak yaratır. B-ağacının yüksekliği kök ayrışınca bir artar.

B-TREE-INSERT-NONFULL (x, k)

```

1   $i = x.n$ 
2  if  $x.leaf$ 
3      while  $i \geq 1$  ve  $k < x.key_i$ 
4           $x.key_{i+1} = x.key_i$ 
5           $i = i - 1$ 
6       $x.key_{i+1} = k$ 
7       $x.n = x.n + 1$ 
8      DISK-WRITE( $x$ )
9  else while  $i \geq 1$  ve  $k < x.key_i$ 
10      $i = i - 1$ 
11      $i = i + 1$ 
12     DISK-READ( $x.c_i$ )
13     if  $x.c_i.n = 2t - 1$ 
14         B-TREE-SPLIT-CHILD( $x, i$ )
15         if  $k > x.key_i$ 
16              $i = i + 1$ 
17     B-TREE-INSERT-NONFULL ( $x, c_i, k$ )

```

B-TREE-INSERT-NONFULL prosedürü şu şekilde çalışır: Satır 3-8, k anahtarını yaprak düğüm olan x e ekleme işlemini gerçekleştirir. Eğer x bir yaprak düğüm değilse, k değerinin x ten başlayan altağaçlarındaki uygun olan yaprak düğüme eklenmesi gerekir. Bu durumda Satır 9-11 x in çocuklarından uygun olanı bulmak için özyinelemeyi gerçekleştirir. Satır 13 döngünün tam çocuk olup olmadığını kontrol eder, Satır 14 B-TREE-SPLIT-CHILD prosedürünü kullanarak o çocuğu iki tam olmayan çocuğa ayırır ve Satır 15-16 bu çocuklardan hangisinin doğru dalanma olup olmadığına karar verir. (Bu aşamada Satır 16 i değerini bir artırdıktan sonra döngü B-TREE-SPLIT-CHILD tarafından oluşturulan çocuklardan birisine

ineceği için $\text{DISK-READ}(x.c_i)$ prosedürüne gerek yoktur). Satır 13-16'nın etkileri prosedürün bir tam düğüme dönmemesini garantiler. Satır 17 k değerini uygun altağaca eklemek için prosedürü özyineler. Şekil 18.7 bir B-ağacına ekleme işleminin çeşitli durumlarını göstermektedir.

h yüksekliğindeki bir B-ağacı için B-TREE-INSERT, B-TREE-INSERT-NON-FULL a yapılan çağrılar arasında $O(1)$ DISK-READ ve DISK-WRITE işlemleri olduğu için $O(h)$ disk erişimi gerçekleştirir. Kullanılan toplam CPU zamanı $O(th) = O(t \log t n)$ kadardır. B-TREE-INSERT-NONFULL prosedürü kuyruk-özyinelemeli olduğu için bir **while** döngüsü ile gerçekleştirilebilir, bu sebeple herhangi bir zamanda bellekte bulunması gereken sayfa sayısı $O(1)$ kadardır.

Alıştırmalar

18.2-1

En küçük derecesi 2 olan boş bir B-ağacına aşağıdaki anahtarları eklemenin sonuçlarını gösteriniz. Bir düğüm ayrılmadan hemen önceki ayarlamaları ve son ağaç düzenini çiziniz.

$F, S, Q, K, C, L, H, T, V, W, M, R, N, P, A, B, X, Y, D, Z, E$

18.2-2

B-TREE-INSERT çağrısının hangi durumlarında gereksiz DISK-READ ve DISK-WRITE işlemlerinin oluşabileceği durumları açıklayınız (Gereksiz DISK-READ işlemi, zaten bellekte olan bir sayfaya DISK-READ işlemi uygulanmasıdır. Gereksiz DISK-WRITE, zaten bellekte tutulan bir bilgi sayfasının belleğe yazılmasıdır).

18.2-3

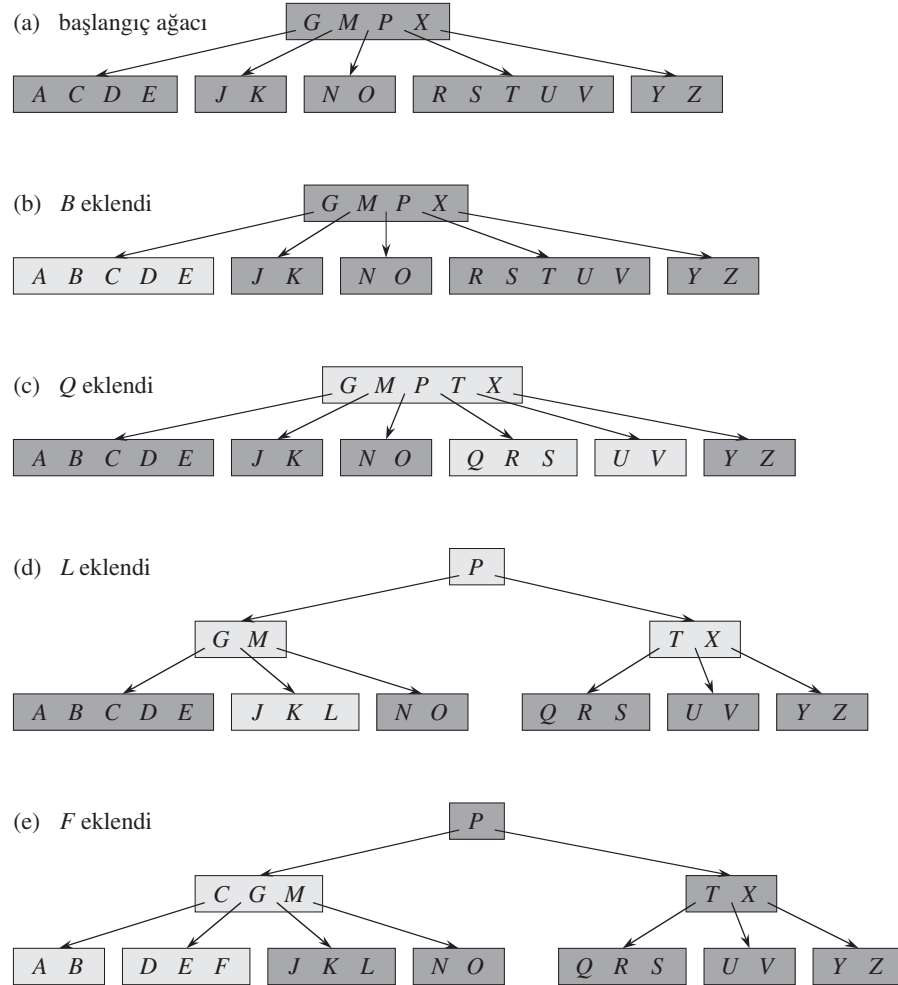
Bir B-ağacında saklanan en küçük anahtarın nasıl bulunacağını ve verilen bir anahtarın ardılının nasıl bulunacağını açıklayınız.

18.2-4 *

$\{1, 2, \dots, n\}$ anahtarlarının minimum 2 dereceli bir B-ağacına eklendiği durumda sonuçta oluşan ağacın kaç düğümü olur?

18.2-5

Yaprak düğümlerde çocuk düğümler için bir işaretçi gerekmediği için aynı disk sayfası büyüklüğündeki ara düğümlerden daha büyük bir t değeri kullanabilir. Bu durumda prosedürlerin B-ağacı yaratma ve ekleme yapılabilmesi için nasıl düzenlenmesi gerektiğini gösteriniz.



Şekil 18.7 B-ağacına anahtar ekleme. Bu B-ağacı için minimum derece 3 tür, bu yüzden bir düğüm en fazla 5 anahtar tutabilir. Ekleme süresi tarafından düzenlenen düğümler açık renkle gösterilmiştir. (a) Bu örnek için başlangıç ağacı. (b) Başlangıç ağacına *B* eklenmesinin sonucu, basit bir yaprak düğüme ekleme işlemi. (c) Bir önceki ağaca *Q* eklenmesi. *RSTUV* düğümü *RS* ve *UV* yi içeren iki düğüme ayrılır, *T* düğümü köke çıkar ve *Q* iki yarının soldakine (*RS* düğümüne) eklenir. (d) Bir önceki ağaca *L* ekleme. Kök ikiye ayrılır çünkü doludur ve B-ağacının yüksekliği bir artar. Daha sonra *L*, *JK* düğümüne eklenir. (e) Bir önceki ağaca *F* ekleme. *ABCDE* düğümü, *F* iki yarının sağdakine (*DE* düğümüne) eklenmeden önce ikiye ayrılır.

18.2-6

Her bir düğümde doğrusal arama yerine ikili arama kullanabilmek için B-TREE-SEARCH yapılması isteniyor. Bu değişikliğin, t nin n in bir fonksiyonu olarak seçilmesinden bağımsız olarak CPU zamanı gereksiniminin $O(\log n)$ olduğunu gösteriniz.

18.2-7

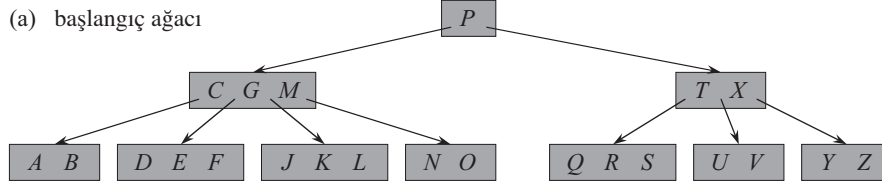
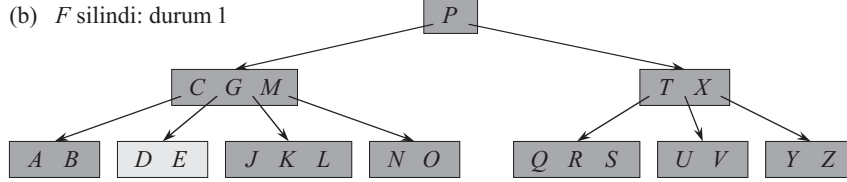
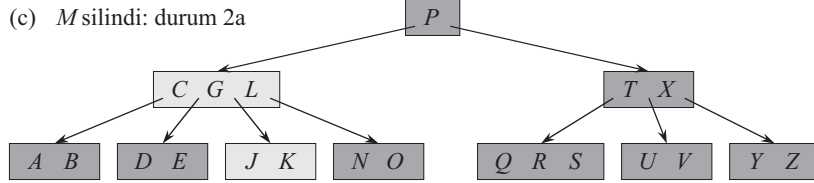
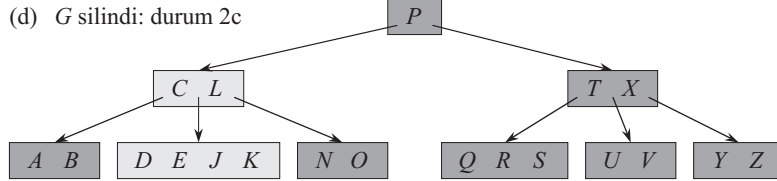
Disk donanımının disk büyüklüğünü isteğe bağlı olarak değiştirebilmeye olanak sağladığı ancak disk sayısını okuma zamanının $a + bt$ (a ve b tanımlanmış sabitler, t seçilen büyüklükteki sayfaları kullanan B-ağacının minimum derecesi) olduğunu varsayınız. B-ağacı arama zamanının en aza indirilebilmesi için t değerin nasıl seçilmesi gerektiğini açıklayınız. $a = 5$ milisaniye ve $b = 10$ mikrosaniye değerleri için en uygun t değerini öneriniz.

18.3 B-ağacından bir anahtar silme

B-ağacından silme işlemi, ekleme işlemine benzer ancak biraz daha karmaşıktır, çünkü sadece yapraklar değil herhangi bir düğüm de silinmek istenilebilir ve bir ara düğümünden bir anahtar silindiğinde o düğümün çocuklarının da tekrar düzenlenmesi gerekir. Eklemede olduğu gibi, silme durumunda yapısı B-ağacı özelliklerine uymayan bir ağaç oluşması engellenmelidir. Ekleme sonunda düğümün çok büyük olmamasını sağladığımız gibi silme durumunda da düğümün çok küçük olmasını önlememiz gerekir (kök düğümün $t - 1$ anahtardan az sayıda anahtar içermesine izin verilmesi durumu hariç). Basit bir ekleme algoritmasının eklenecek anahtarın yolu üzerindeki düğümlerden birinin tam olması durumunda yedeklenmesi gerektiği gibi, silme yaklaşımında da kök dışındaki bir ara düğümün minimum sayıdan az sayıda anahtarı olan düğümün de yedeklenmesi gerekir.

B-TREE-DELETE prosedürü k anahtarını x düğümünden dallanan altağaçtan siler. Bu prosedürün x düğümü üzerinde kendini geri çağırması durumunda x düğümündeki anahtar sayısının en azından t minimum derecesinde olduğunun garantilenmesi gerekir. Bu durum genel B-ağacı koşullarınca gereken minimum anahtar sayısından bir fazlasını gerektirir. Bu yüzden bazen özyineleme bir çocuk düğüme gelmeden anahtarın o çocuk düğüme aktarılması gerekebilir. Bu güçlendirilmiş koşul yedek almadan tek bir aşağıya doğru geçiş ile bir düğümün silinmesine olanak sağlar (daha sonra açıklanacak bir istisna dışında). Eğer bir kök düğüm x , anahtarı olmayan bir ara düğüm olursa, x düğümü silinir ve x in tek çocuğu $x.c1$ ağacın yeni kökü olur. Bu da ağacın yüksekliğini 1 eksiltir ve ağaç kökünün en az bir anahtara sahip olması durumu (ağaç boş değil ise) korunmuş olur.

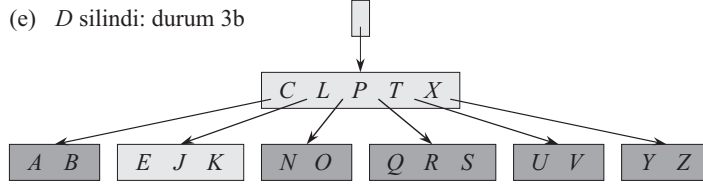
(a) başlangıç ağacı

(b) F silindi: durum 1(c) M silindi: durum 2a(d) G silindi: durum 2c

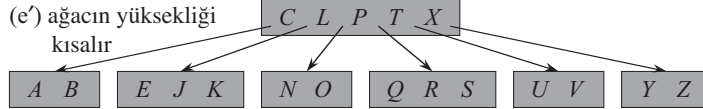
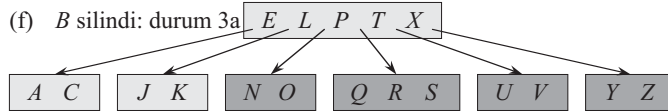
Şekil 18.8 B-ağacından anahtar silme. Bu ağaç için minimum derece $t = 3$ tür yani kök dışında bir düğüm 2 den az anahtara sahip olamaz. Düzenlenen düğümler açık renkle gösterilmiştir. (a) Şekil 18.7(e) deki B-ağacı. (b) F nin silinmesi. Durum 1: bir yapraktan basit silme. (c) M nin silinmesi. Durum 2a: M nin öncülü L , M nin yerini almak üzere bir üste çıkar. (d) G nin silinmesi. Durum 2c: $DEGJK$ düğümünü oluşturmak üzere G aşağıya itilir daha sonra G yapraktan silinir (durum 1).

Silme işleminin nasıl çalıştığı aşağıda kabakod yerine taslak olarak verilmektedir. Şekil 18.8 bir B-ağacından anahtar silme işleminin çeşitli durumlarını göstermektedir.

1. Eğer k anahtarı x düğümünde ve x bir yaprak düğümse, k anahtarını x ten sil.
2. Eğer k anahtarı x düğümünde ve x bir ara düğümse, şunları yap:

(e) D silindi: durum 3b

(e') ağacın yüksekliği kısalmır

(f) B silindi: durum 3a

Şekil 18.8, devamı (e) D nin silinmesi. Durum 3b: özyineleme CL düğümüne kadar gitmez çünkü onun sadece 2 anahtarı vardır. Bu yüzden P aşağıya itilir, CL ile TX birleştirilir ve $CLPTX$ oluşturulur. Daha sonra yapraklardan D silinir (durum 1). (e') (e) den sonra kök silinir ve ağacın yüksekliği bir eksiltir. (f) B nin silinmesi. Durum 3a: C, B nin yerini doldurmak üzere yer değiştirir ve E, C nin yerine gelir.

- Eğer x düğümündeki k ya öncülük eden y çocuğunun en az t anahtarı varsa, y den dallanan altağaçtaki k nın öncülü olan k' değerini bul. Özyinelemeli olarak k' değerini sil ve x düğümünde k yerine k' değerini yerleştir (k' değeri tek bir geçiş ile bulunup silinebilir).
 - Eğer y t sayıda anahtardan daha az anahtar içeriyorsa, simetrik olarak x düğümünde k yi takip eden z çocuğunu incele. Eğer z en az t anahtara sahipse, z düğümünden dallanan altağaçta k nın ardılı olan k' değerini bul. Özyinelemeli olarak k' değerini sil ve x düğümünde k yerine k değerini koy. (k' değeri tek bir geçiş ile bulunup silinebilir).
 - Aksi halde, y ve z düğümleri $t-1$ anahtar içeriyorsa, k ve z nin tüm anahtarlarını y düğümünde birleştir, böylece x , k anahtarını ve z işaretçisini kaybeder ve y de $2t - 1$ adet anahtar olur. Daha sonra z yi serbest bırak ve özyinelemeli olarak y den k anahtarını sil.
- Eğer k anahtarı ara düğüm x de yoksa, k değerini içermesi gereken altağacın kökü olan $x.c_i$ düğümünü belirle. Eğer $x.c_i$ $t - 1$ anahtara sahip ise, en az t anahtarı olan bir düğüme geçmeyi garantilemek için 3a ya da 3b adımlarından gerekli olanı çalıştır. Daha sonra özyinelemeyi x in uygun olan çocuğunda sonlandır.

- a. Eğer $x.c_i$ sadece $t - 1$ anahtara sahip ancak en az t anahtara sahip kardeşi varsa, $x.c_i$ düğümüne x düğümünden bir anahtar aktar, $x.c_i$ nin sol ya da sağ kardeşinden bir anahtar alarak yukarı x düğümüne taşı ve uygun çocuk işaretçisini de kardeş düğümünden $x.c_i$ ye taşı.
- b. Eğer $x.c_i$ ve kardeşi $t - 1$ anahtara sahip ise, $x.c_i$ düğümünü bir kardeşi ile birleştir, x düğümünden bir anahtarı aşağıya yeni birleştirilen düğümde medyan (ortanca) anahtar olacak şekilde taşı.

B-ağaçlarındaki anahtarların çoğunluğu yaprak düğümlerde olduğundan, pratikte silme işlemlerinin sıklıkla yapraklarda gerçekleştiği düşünülebilir. Bu durumlarda B-TREE-DELETE prosedürü yedekleme yapmadan tek geçişte çalışır. Ara düğümünden bir anahtar silerken prosedür silinen anahtarın olduğu düğüme geri dönmek zorunda kalabilir. (2a ve 2b deki gibi).

Bu prosedür karmaşık görünmesine rağmen, h yüksekliğindeki bir ağaçta DISK-READ ve DISK-WRITE çağrıları özyinelemeler arasında $O(1)$ kez gerçekleşeceğinden sadece $O(h)$ disk işlemi gerektirir. Gereken CPU zamanı ise $O(th) = (t \log_e n)$ kadardır.

Alıştırmalar

18.3-1

Şekil 18.8(f) teki ağaçtan sırasıyla C , P ve V değerlerini silme sonuçlarını gösteriniz.

18.3-2

B-TREE-DELETE için kabakodunu yazınız.

Problemler

18-1 İkincil depolamada yığınlar

Küçük miktarda hızlı birincil belleği ve büyük miktarda yavaş disk depolaması olan bir bilgisayarda bir yığın işlemini göz önüne alın. PUSH ve POP işlemleri tekli kelime değerleri üzerinde çalışırlar. Bizim desteklemek istediğimiz yığın belleğe sığabileceğinden daha fazla büyüyebilir ve bu yüzden çoğunluğu disk üzerinde saklanmalıdır.

Basit ama etkin olmayan yığın uygulaması tüm yığını disk üzerinde tutmaktır. Bellekte, yığının en üstündeki elemanın disk adresini gösteren bir yığın işaretçisi temin edilir. İşaretçinin değerine p , sayfa başına kelime sayısına m dersek, en üstteki eleman diskteki $\lfloor p/m \rfloor$ sayfasının $(p \bmod m)$ nci kelimesidir.

PUSH işlemini uygulamak için yığın işaretçisini artırıp, diskten uygun sayfayı belleğe okuyup eklenecek elemanı sayfadaki uygun kelimeye kopyalar ve sayfayı diske geri yazarız. POP işlemi de benzerdir. Yığın işaretçisi eksiltilir, diskten uygun sayfa okunur ve yığının en üstü döndürülür. Değiştirilmediği için sayfayı geri yazmaya gerek yoktur.

Disk işlemleri maliyetli olduğu için herhangi bir uygulama için 2 maliyet ele alınır: disk erişiminin toplam sayısı ve toplam CPU zamanı. m kelimeli herhangi bir sayfaya disk erişimi 1 disk erişimine ve $\Theta(m)$ CPU zamanına mal olur.

- a.** Asimptotik olarak bu basit uygulama kullanıldığında n yığın işlemi için en kötü durum disk erişim sayısı nedir? n yığın işlemi için CPU zamanı nedir? (Bu ve bundan sonraki kısımlar için cevabınızı m ve n terimleri cinsinden veriniz).

Şimdi yığının bir sayfasının bellekte tutulduğu yığın uygulamasını göz önünde bulundurun (Hangi sayfanın bellekte tutulduğunu izlemek için de bellekten küçük bir miktarı kullanacağız). Bir yığın işlemi, sadece ilgili disk sayfası bellekteyse gerçekleştirebileceğiz. Eğer gerekirse, bellekteki mevcut sayfayı diske yazıp diskten yeni sayfayı belleğe okuyabiliriz. Eğer ilgili disk sayfası zaten bellekte ise, o zaman disk erişimine gerek duyulmayacaktır.

- b.** n adet PUSH işlemi için gerekli en kötü durum disk erişim sayısı nedir? CPU zamanı nedir?
- c.** n adet yığın işlemi için gerekli en kötü durum disk erişim sayısı nedir? CPU zamanı nedir?

Şimdi yığın uygulamasını bellekte 2 sayfa tutarak gerçekleştirdiğimizi varsayın (ek olarak az sayıda kelime ile yönetimi sağlayarak)

- d.** Herhangi bir yığın işleminin dönümlü disk erişim sayısının $O(1/m)$ ve sönümlü CPU zamanının $O(1)$ olması için yığın sayfalarının ne şekilde idare edilmesi gerektiğini açıklayınız.

18-2 2-3-4 ağaçlarını birleştirme ve ayırma

Birleştirme işlemi iki dinamik küme S' ve S'' ile x elemanını alır. Herhangi $x' \in S'$ ve $x'' \in S''$ için $x'.key < x.key < x''.key$ durumu söz konusudur. Bu işlem $S = S' \cup \{x\} \cup S''$ kümesini döndürür. **Ayırma** işlemi ise “ters” birleştirme gibidir: verilen dinamik S kümesi ve $x \in S$ elemanı için, $S - \{x\}$ içindeki tüm elemanlardan anahtarları $x.key$ den küçük anahtarlı elemanları olan S' kümesini ve $S - \{x\}$ içindeki tüm elemanlardan anahtarları $x.key$ den büyük anahtarlı elemanları olan S'' kümesini oluşturur. Bu problemde, bu işlemleri 2-3-4 ağaçlarında nasıl uygulayaca-

bileceğimizi araştıracağız. Elemanların sadece anahtarlardan oluştuğunu ve tüm anahtar değerlerin birbirlerinden farklı olduğunu varsayalım.

- a. 2-3-4 ağacının her düğümü için x düğümünden dallanan (kökü x olan) alt-ağacın yüksekliğinin $x.height$ şeklinde bir özellik olarak nasıl oluşturulacağını gösteriniz.
- b. Birleştirme işleminin nasıl uygulanacağını gösteriniz. Verilen T' ve T'' 2-3-4 ağaçları ile k anahtarına göre birleştirme işlemi $O(1 + |h' - h''|)$ zamanda gerçekleşmelidir (h' ve h'' , T' ve T'' ağaçlarının yüksekliğini göstermektedir).
- c. Bir 2-3-4 ağacı T nin kökünden verilen bir k anahtarına giden yolun p olduğunu, T deki k dan küçük anahtarlar kümesinin S' ve k dan büyük anahtarlar kümesinin S'' olduğunu varsayın. p nin S' kümesini $\{T'_0, T'_1, \dots, T'_m\}$ ağaçlar kümesine ve $\{k'_1, k'_2, \dots, k'_m\}$ anahtarlar kümesine ayırdığını ve $i = 1, 2, \dots, m$ için herhangi $y \in T'_{i-1}$ ve $z \in T'_i$ için $y < k'_i < z$ olduğunu gösterin. T'_{i-1} ve T'_i ağaçlarının yükseklikleri arasındaki ilişki nedir? p nin S'' kümesini nasıl ağaçlar ve anahtarlar kümelerine böldüğünü açıklayın.
- d. Ayırma işleminin T üzerinde nasıl uygulanacağını gösterin. S' kümesindeki anahtarların tek bir 2-3-4 ağacı T' ile S'' kümesindeki anahtarların tek bir 2-3-4 ağacı T'' yi bir araya getirmek için birleştirme işlemini kullanın. Ayırma işleminin çalışma zamanı $O(\log(n))$ olmalıdır ($n = T$ deki anahtar sayısı) (*İpucu*: birleştirme maliyeti dahil edilmelidir).

Bölüm notları

Knuth [211], Aho, Hopcroft ve Ullman [5] ile Sedgewick [306] dengeli ağaç şemaları ve B-ağaçları üzerine daha ileri çalışmalarda bulunmuşlardır. Comer [74] B-ağaçları üzerine kapsamlı araştırmalar öngörür. Guibas ve Sedgewick [155], kırmızı-siyah ağaçları ve 2-3-4 ağaçlarını da içeren çeşitli dengeli ağaç şema türleri arasındaki ilişkileri tartışmıştır.

1970 te J. E. Hopcroft B-ağaçlarına ve 2-3-4 ağaçlarına öncülük eden ve her bir ara düğümü iki ya da üç çocuğu olan 2-3 ağaçlarını keşfetmiştir. Bayer ve McCreight [35] 1972 de B-ağaçları tanıtmış, isim tercihlerini açıklamamışlardır.

Bender, Demaine ve Farach-Colton [40] bellek hiyerarşisi etkilerinin varlığında B-ağaçlarının daha iyi işletilmesine çalışmıştır. **Önbellek-unutkan** algoritmaları, bellek hiyerarşisindeki veri transfer büyüklüğünü bilmeksizin etkin olarak çalışmaktadır.

Fibonacci Bellek yığınları veri yapısı iki amaca hizmet eder. Birincisi, “birleştirilebilir bellek yığınları” olarak bilinen yapıyı oluşturan işlemler kümesini destekler. İkinci olarak bazı Fibonacci-bellek yığını işlemleri sabit amortize zamanda çalışır, bu da bu veri yapısını bu tür işlemleri sık kullanan uygulamalar için elverişli kılar.

Birleştirilebilir bellek yığınları

Birleştirilebilir bellek yığını aşağıdaki her elemanı bir anahtara sahip olan 5 işlemi gerçekleştirmeyi destekleyen veri yapısıdır:

MAKE-HEAP() elemanı olmayan yeni bir bellek yığını yaratıp döndürür.

INSERT(H, x) anahtarı önceden belirlenmiş bir x elemanını H bellek yığına ekler.

MINIMUM(H) H bellek yığınındaki anahtarı en küçük elemanın işaretçisini döndürür.

EXTRACT-MIN (H) en küçük anahtarlı elemanı H bellek yığından siler ve bu elemanın işaretçisini döndürür.

UNION (H_1, H_2) H_1 ve H_2 bellek yığınlarının tüm elemanlarını içeren bir bellek yığını oluşturur ve döndürür. H_1 ve H_2 bu işlemle yok edilir.

Yukarıdaki birleştirilebilir bellek yığını işlemlerine ek olarak Fibonacci bellek yığınları aşağıdaki işlemleri de destekler:

DECREASE-KEY(H, x, k) mevcut değerinden daha büyük olmadığını varsaydığımız yeni anahtar değeri k y H bellek yığınındaki x elemanına atama işlemini gerçekleştirir.¹

DELETE (H, x) x elemanını H bellek yığından siler.

¹Kısım V in girişinde de bahsedildiği gibi birleştirilebilir bellek yığınlarımız birleştirilebilir min-bellek yığınlarıdır ve bu sebeple MINIMUM, EXTRACT-MIN ve DECREASE-KEY uygulanabilir. Alternatif olarak MAXIMUM, EXTRACT-MAX ve INCREASE-KEY işlemleriyle uygulanabilecek bir **birleştirilebilir maks-bellek yığını** da tanımlayabilirdik.

Prosedür	İkili bellek yığını (en kötü durum)	Fibonacci bellek yığını (amortize)
MAKE-HEAP	$\Theta(1)$	$\Theta(1)$
INSERT	$\Theta(\log n)$	$\Theta(1)$
MINIMUM	$\Theta(1)$	$\Theta(1)$
EXTRACT-MIN	$\Theta(\log n)$	$O(\log n)$
UNION	$\Theta(n)$	$\Theta(1)$
DECREASE-KEY	$\Theta(\log n)$	$\Theta(1)$
DELETE	$\Theta(\log n)$	$O(\log n)$

Şekil 19.1 Birleştirilebilir bellek yığınlarının iki uygulaması üzerindeki işlemler için çalışma zamanları. Bir işlem anında bellek yığnında bulunan eleman sayısı n ile gösterilmiştir.

Şekil 19.1 de gösterildiği gibi UNION işlemine gerek duymazsak sıradan ikili bellek yığınları—bellek yığını sıralamasındaki gibi (Bölüm 6)—oldukça iyi çalışır. UNION dışındaki diğer işlemler ikili bellek yığınlarında en kötü zaman olan $O(\log n)$ zamanında çalışır. UNION işlemine ihtiyaç duyulursa ikili bellek yığınları zayıf performans gösterir. İkili bellek yığınlarını tutan dizileri birleştirip BUILD-MIN-HEAP (bkz. Kısım 6.3) çalıştırılırsa UNION işlemi en kötü durumda $\Theta(n)$ zaman alır.

Fibonacci bellek yığınları diğer yandan INSERT, UNION ve DECREASE-KEY işlemlerinde ikili bellek yığınlarından daha iyi asimptotik zaman sınırlarına sahiptir. Diğer işlemler için aynı zaman sınırları söz konusudur. Hatırlatılmalıdır ki Şekil 19.1 de gösterilen çalışma zamanları işlem başına en kötü durum değil, amortize zaman sınırlarıdır. UNION işlemi Fibonacci bellek yığınlarında sabit amortize zaman alır ki bu da ikili bir bellek yığını için gerekli olan doğrusal en kötü durum zamanından belirgin şekilde daha iyidir (elbette ki amortize zaman sınırının yeterli olduğu varsayıldığında).

Teoride ve pratikte Fibonacci bellek yığınları

Teorik açıdan bakıldığında Fibonacci bellek yığınları özellikle EXTRACT-MIN ve DELETE işlemlerinin sayısı diğer işlemlere oranla daha az olduğu durumlarda tercih edilendir. Bu durum birçok uygulamada ortaya çıkar. Örneğin çizge problemleri için bazı algoritmalar DECREASE-KEY işlemini her kenar için bir kez çağırabilir. Birçok kenarı olan yoğun çizgeler için her bir DECREASE-KEY çağrısının $\Theta(1)$ amortize zamanı ikili bellek yığınlarının en kötü durum zamanı olan $\Theta(\log n)$ için bir geliştirme getirmektedir. En küçük kapsayan ağaçlarda hesaplama problemleri gibi hızlı algoritmalar ve tek kaynak en kısa yol bulma çözümleri Fibonacci bellek yığınlarının kullanımı gerekli kılar.

Ancak pratik açıdan bakıldığında Fibonacci bellek yığınlarının sabit etkenleri ve programlama karmaşıklığı, çok büyük verileri yöneten belirli uygulamalar dışında sıradan ikili (ya da k -lı) bellek yığınlarındaki uygulamalar için Fibonacci bellek yığınları tercih edilmeyebilir. Bu sebeple Fibonacci bellek yığınları baskın olarak teorik çalışmaların ilgisini çekmektedir. Fibonacci bellek yığınlarıyla aynı amortize zaman sınırları olan daha basit bir veri yapısı geliştirilseydi, bu veri yapısı daha pratik olabilirdi.

İkili bellek yığınları ve Fibonacci bellek yığınlarının her ikisi de SEARCH işlemine destekleri açısından etkin değillerdir, verilen bir anahtarı bulmak zaman almaktadır. Bu yüzden DECREASE-KEY ve DELETE gibi verilen bir elemanı ilgilendiren işlemler için o elemanı gösteren bir işaretçiyi girdilerinin bir parçası olarak tutmak zorunluluğu vardır. Kısım 6.5 te sözü edilen öncelikli-kuyruklarda tartıştığımız gibi bir uygulamada birleştirilebilir bellek yığını kullanıldığında hem her uygulama nesnesinin hem de o nesneye karşılık gelen birleştirilebilir bellek yığını elemanının bir bilgisi tutulmaktadır. Bu bilgilerin tam yapısı uygulamaya ve gerçekleştirimine bağlıdır.

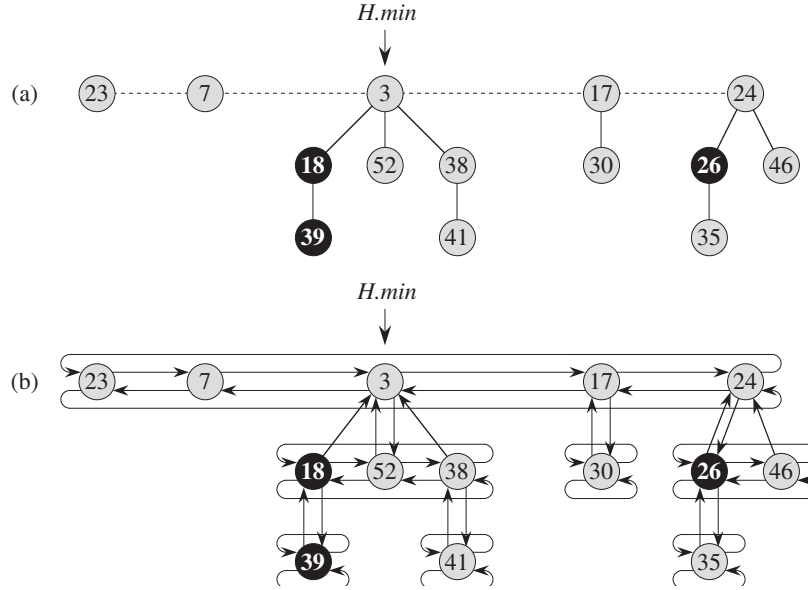
Daha önce gördüğümüz bazı veri yapıları gibi Fibonacci bellek yığınları köklü ağaçlar üzerine kurulmuştur. Her eleman ağaçtaki bir düğüm ile ifade edilir ve her düğümün bir anahtar özelliği vardır. Bu bölümün geri kalanı için “eleman” terimi yerine “düğüm” terimi kullanılmıştır. Ayrıca ekleme öncesi yer ayırma ve silme sonrası belleği boşaltma işlemleri de bellek yığını prosedürlerini çağıran kodun bu detayları ayarlayacağı varsayılarak göz ardı edilmiştir.

Kısım 19.1 Fibonacci bellek yığınlarını tanımlar, onların nasıl ifade edildiğini konu eder ve amortize analizleri için kullanılan potansiyel fonksiyonları göstermektedir. Kısım 19.2 birleştirilebilir bellek yığını işlemlerinin nasıl gerçekleştirildiğini ve Şekil 19.1 de gösterilen amortize zaman sınırlarının nasıl elde edildiğini gösterir. Kalan iki işlem, DECREASE-KEY ve DELETE, Kısım 19.3 ün içeriğini teşkil eder. Son olarak Kısım 19.4 analizin anahtar kısmını tamamlar ve veri yapısının garip ismini açıklar.

19.1 Fibonacci bellek yığınlarının yapısı

Bir *Fibonacci bellek yığını*, *min-bellek yığını sıralı* köklü ağaçlar topluluğudur. Yani her ağaç *min-bellek yığını özelliğine* uyar: bir düğümün anahtarı kendi ebeveyninin anahtarından büyük ya da eşittir. Şekil 19.2(a) örnek bir Fibonacci bellek yığını göstermektedir.

Şekil 19.2(b) de gösterildiği gibi her x düğümü kendi ebeveynini gösteren bir $x.e$ işaretçisi ve çocuklarından birini gösteren $x.child$ işaretçisi içerir. x düğümünün çocukları dairesel olarak birbirine çift yönlü bağlı listeler ile bağlıdır, bu listeye x in *çocuk listesi* denir. Çocuk listesindeki her çocuk y de sol ve sağ kardeşlerini gösteren $y.right$ ve $y.left$ işaretçileri bulunur. Eğer y düğümü tek çocuk ise, $y.left = y.right = y$ dir. Kardeşler bir çocuk listesinde herhangi bir sırada gözükabilirler.



Şekil 19.2 (a) Beş adet min-bellek yığını-sıralı ağacı ve 14 düğümü olan bir Fibonacci bellek yığını. Kesik çizgiler kök listesini göstermektedir. Bellek yığınının minimum düğümü 3 anahtarını içeren düğümdür. Siyah düğümler işaretli düğümlerdir. Bu Fibonacci bellek yığınının potansiyeli $5 + 2 \cdot 3 = 11$ dir. (b) İşaretçilerle birlikte tam gösterim. p (yukarı ok), $child$ (aşağı ok) ve $left\ right$ ve $sağ$ (yan oklar). Bu bölümdeki diğer şekiller, burada gösterilen bütün bilgi (a) dan çıkarılabildiği için bu detayları göz ardı etmektedir.

Çift yönlü dairesel bağlı listelerinin Fibonacci bellek yığınlarında kullanımının iki avantajı vardır. İlki, dairesel çift yönlü bağlı listelerde herhangi bir yere bir düğüm eklemek ya da herhangi bir yerden bir düğüm çıkarmak $O(1)$ zaman alır. İkincisi, böyle iki liste $O(1)$ zamanda tek bir dairesel çift yönlü bağlı listede birleştirilebilir (birbirine eklenebilir). Fibonacci bellek yığını işlemlerinin tanımlarında bu işlemlere az da olsa değinilecek, uygulamaların detaylarının doldurulması kullanıcıdan istenecektir.

Her düğümde iki özellik daha bulunur. $x.degree$ özelliğinde x in çocuk listesindeki çocuk sayısı saklanır. Boole değerli özellik $x.mark$ x in başka bir düğümün çocuğu yapıldığı andan itibaren bir çocuk kaybedip etmediğinin bilgisini tutar. Yeni yaratılmış düğümler işaretlenmemiştir ve bir x düğümü başka bir düğümün çocuğu olduğunda işaretsiz olur. Kısım 19.3'teki DECREASE-KEY işlemine değininceye kadar tüm işaret özelliklerini FALSE olarak ayarlayacağız.

H Fibonacci bellek yığınının erişim en küçük anahtar değeri içeren ağacın kök düğümünü gösteren işaretçi $H.min$ ile sağlanır; bu düğüme Fibonacci bellek

yığınının **minimum düğümü** denir. Birden çok kök en küçük değerli anahtara sahipse, bu köklerden herhangi birisi minimum düğüm olarak hizmet eder. H Fibonacci bellek yığını boş ise $H.min$ NIL değerini alır.

Bir Fibonacci bellek yığınınındaki bütün ağaçların kökleri *left* ve *right* işaretçileri ile dairesel çift yönlü bağlı liste şeklinde birbirlerine bağlıdır. Bu listeye Fibonacci bellek yığınının kök listesi adı verilir. $H.min$ işaretçisi bu durumda kök listesindeki anahtarı en küçük olan düğümü gösterir. Bir kök listesindeki ağaçlar herhangi bir sırada gözükabilir.

Bir H Fibonacci bellek yığını için kullanacağımız bir diğer özellik ise $H.n$: mevcut durumda H bellek yığınınındaki düğüm sayısıdır.

Potansiyel fonksiyon

Daha önce konu edildiği gibi, Kısım 17.3 teki potansiyel fonksiyonu Fibonacci bellek yığını işlemlerinin performanslarını analiz etmek için kullanacağız. Verilen bir H Fibonacci bellek yığını için $t(H)$ değeri ile H bellek yığınınındaki kök listesinin ağaç sayısını ve $m(H)$ ile de H bellek yığınınındaki işaretli düğümlerin sayısını göstermek için kullanacağız. Bu durumda bir H Fibonacci bellek yığınının potansiyel fonksiyonu $\Phi(H)$ şöyle tanımlanır.

$$\Phi(H) = t(H) + 2m(H) \quad (19.1)$$

(Bu fonksiyon için Kısım 19.3 te biraz daha bilgi edineceğiz.) Örneğin Şekil 19.2 de verilen Fibonacci bellek yığınının potansiyel fonksiyonu $5 + 2 \cdot 3 = 11$ dir. Bir Fibonacci bellek yığını kümesinin potansiyeli her bir Fibonacci bellek yığınının potansiyelleri toplamına eşittir. Bir birim potansiyelin sabit bir iş miktarını karşıladığı varsayılır. Buradaki sabit, spesifik sabit zamanlı iş parçalarının her birinin maliyetini kapsayacak kadar büyüktür.

Fibonacci bellek yığını uygulamalarının bir bellek yığını ile başlamadığı varsayılır. Başlangıç potansiyeli 0 değerini alır. Potansiyel, tüm zamanlarda negatif olmayan değerler alır. Denklem 17.3 ten toplam amortize maliyetinin üst sınırı işlem sıralarının toplam gerçek maliyeti için bir üst sınır belirler.

Maksimum derece

Bundan sonraki bölümlerde kullanacağımız amortize analizler n düğümlü bir Fibonacci bellek yığınınında herhangi bir düğümün maksimum derecesi için üst sınır olan $D(n)$ değerini bildiğimizi varsaymaktadır. Bunu kanıtlamayacağız ama sadece birleştirilebilir bellek yığını işlemleri desteklendiğinde $D(n) \leq \lceil \lg n \rceil$ olduğunu belirtelim. (Problem 19.2(d) bu özelliği sizin kanıtlamanızı beklemektedir.) Kısım 19.3 ve 19.4 te DECREASE-KEY ve DELETE desteklendiğinde $D(n) = O(\lg n)$ olduğunu göstereceğiz.

19.2 Birleştirilebilir bellek yığını işlemleri

Fibonacci bellek yığınlarındaki birleştirilebilir bellek yığını işlemleri mümkün olduğunca çalışır. Çeşitli işlemlerde performans ödünleri verilir. Örneğin bir düğümü kök listesine ilave ederek ekleyebiliriz ki bu belirli bir zaman alır. Boş bir Fibonacci bellek yığını ile başlar ve k tane düğüm eklersek, Fibonacci bellek yığını k düğümden oluşan bir kök listesinden ibaret olur. Buradaki ödün şudur: Eğer H Fibonacci bellek yığında EXTRACT-MIN işlemi gerçekleştirecek olursak, $H.min$ değerinin gösterdiği düğümü çıkardıktan sonra, yeni minimum düğümü bulmak için kök listesinde kalan $k - 1$ tane düğüme de bakmamız gerekir. EXTRACT-MIN işlemi sırasında kök listesinin tamamında dolaştığımız sürece kök listesinin boyunu azaltmak için düğümleri min-bellek yığını-sıralı ağaçlar haline çevirmek için düzenlememiz gerekir. Göreceğiz ki, EXTRACT-MIN işleminden önce kök listesinin şekli ne olursa olsun, işlemden sonra kök listesindeki her düğüm listedeki diğer düğümlerden farklı olacak şekilde bir derece değerine sahip olacak. Bu da en fazla $D(n) + 1$ büyüklüğünde bir kök listesi oluşturacak.

Yeni bir Fibonacci bellek yığını oluşturma

Boş bir Fibonacci bellek yığını oluşturmak için, MAKE-FIB-HEAP prosedürü çalışır ve $H.n = 0$, $H.min = \text{NIL}$ olan bir H Fibonacci bellek yığını nesnesi döndürür. H nin içinde hiç ağaç yoktur. $t(h) = 0$ $m(H) = 0$ olduğu için, boş ağacın potansiyeli $\Phi(H) = 0$ dır. Bu yüzden MAKE-FIB-HEAP işleminin amortize maliyeti $O(1)$ gerçek maliyetine eşittir.

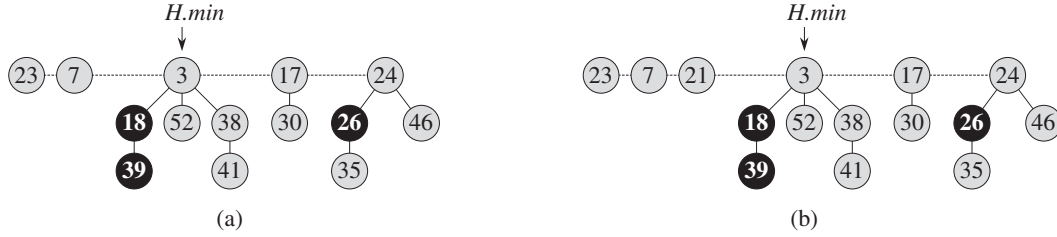
Düğüm ekleme

Aşağıdaki prosedür x düğümünü h Fibonacci bellek yığına ekler. Düğümün önceden yaratıldığı ve $x.key$ değerinin tanımlandığı varsayılmaktadır.

FIB-HEAP-INSERT (H, x)

```

1   $x.degree = 0$ 
2   $x.e = \text{NIL}$ 
3   $x.child = \text{NIL}$ 
4   $x.mark = \text{FALSE}$ 
5  if  $H.min == \text{NIL}$ 
6       $H$  için, sadece  $x$  i içeren bir kök listesi yarat
7       $H.min = x$ 
8  else  $x$  i  $H$  nin kök listesine ekle
9      if  $x.key < H.min.key$ 
10          $H.min = x$ 
11   $H.n = H.n + 1$ 
```



Şekil 19.3 Fibonacci bellek yığınına bir düğüm ekleme. (a) H Fibonacci bellek yığını. (b) Anahtar değeri 21 olan bir düğüm eklendikten sonraki H Fibonacci bellek yığını. Düğüm kendisinin min-yığın sıralı ağacı olur ve daha sonra kökün sol kardeşi olacak şekilde kök listesine eklenir.

Satır 1-4 x düğümünün bazı yapısal özelliklerini belirler. Satır 5 H Fibonacci bellek yığınının boş olup olmadığını test eder. Eğer boş ise, Satır 6-7 x i H nin kök listesindeki tek düğüm olacak ve $H.min$ x i gösterecek şekilde düzenler. Değilse, Satır 8-10 x i H nin kök listesine ekler ve eğer gerekiyorsa $H.min$ i günceller. Son olarak Satır 11 yeni düğümün eklendiğini belirtmek üzere $H.min$ değerini artırır. Şekil 19.3, 21 anahtar değerli bir düğümün Şekil 19.2 deki bellek yığınına eklenmesini göstermektedir.

FIB-HEAP-INSERT işleminin amortize maliyetini belirlemek için, giriş Fibonacci bellek yığını H , sonuç Fibonacci bellek yığını da H' olsun. Bu durumda $t(H') = t(H) + 1$ $m(H') = M(H)$ olur. Potansiyeldeki artış ise

$$((t(H) + 1) + 2m(H)) - (t(H) + 2m(H)) = 1$$

olur.

Gerçek maliyet $O(1)$ olduğu için, amortize maliyet de $O(1) + 1 = O(1)$ dir.

Minimum düğümü bulma

H Fibonacci bellek yığınının minimum düğümü $H.min$ işaretçisi ile verilmiştir, o yüzden minimum düğümü $O(1)$ gerçek zamanında bulabiliriz. H nin potansiyeli değişmeyeceği için, bu işlemin amortize maliyeti de $O(1)$ gerçek maliyetine eşittir.

İki Fibonacci bellek yığını birleştirme

Aşağıdaki prosedür H_1 ve H_2 Fibonacci bellek yığınlarını birleştirir ve süreç içinde H_1 ve H_2 yi yok eder. Basitçe H_1 ve H_2 nin kök listelerini bir araya getirip yeni minimum düğümü belirler. Bundan sonra H_1 ve H_2 nesnelerinin gösterdiği nesneler bir daha kullanılamaz.

FIB-HEAP-UNION (H_1, H_2)

```

1   $H = \text{MAKE-FIB-HEAP}$ 
2   $H.min = H_1.min$ 
3   $H_2$  nin kök listesini  $H$  nin kök listesi ile birleştir
4  if ( $H_1.min == \text{NIL}$ ) veya ( $H_2.min \neq \text{NIL}$  ve  $H_2.min.key < H_1.min.key$ )
5       $H.min = H_2.min$ 
6   $H.n = H_1.n + H_2.n$ 
7  return  $H$ 

```

Satır 1-3 H_1 ve H_2 nin kök listelerini yeni bir H kök listesinde birleştirir. Satır 2, 4, 5 H nin minimum düğümünü ayarlar ve Satır 6 $H.n$ değerini düğümlerin toplamı olarak belirler. Satır 7 oluşan Fibonacci bellek yığını döndürür. FIB-HEAP-INSERT prosedüründe olduğu gibi tüm kökler kök olarak kalmaya devam eder.

Potansiyeldeki değişim:

$$\begin{aligned}
 \Phi(H) - (\Phi(H_1) + \Phi(H_2)) \\
 &= (t(H) + 2m(H)) - ((t(H_1) + 2m(H_1)) + (t(H_2) + 2m(H_2))) \\
 &= 0
 \end{aligned}$$

olur çünkü $t(H) = t(H_1) + t(H_2)$ ve $m(H) = m(H_1) + m(H_2)$ dir. Bu sebeple FIB-HEAP-UNION nun amortizemaliyeti kendisinin $O(1)$ gerçek maliyetine eşittir.

Minimum düğümü çıkarma

Minimum düğümü çıkarma işlemi, bu kısımda anlatılacak olan işlemlerden en karmaşığıdır. Ayrıca işlem sonunda oluşacak olan kök listesindeki ağaçların birleştirilmesinin gerçekleştiğı gecikmeli iş yükünün olduğu kısımdır. Aşağıdaki prosedür minimum düğümü çıkarır. Bağlı listeden bir düğüm çıkarıldığında listede kalan işaretçilerin güncellendiğı ancak çıkarılan düğümdeki işaretçilerin değişmeden bırakıldığı varsayılmaktadır. Ayrıca kısaca göreceğimiz CONSOLIDATE yardımcı prosedürünü de çağırmaktadır.

FIB-HEAP-EXTRACT-MIN (H)

```

1   $z = H.min$ 
2  if  $z \neq \text{NIL}$ 
3      for  $z$  nin her bir çocuğu  $x$ 
4           $x$  i  $H$  nin kök listesine ekle
5           $x.e = \text{NIL}$ 
6       $z$  yi  $H$  nin kök listesinden çıkar
7      if  $z == z.right$ 
8           $H.min = \text{NIL}$ 
9      else  $H.min = z.right$ 
10     CONSOLIDATE( $H$ )
11      $H.n = H.n - 1$ 
12 return  $z$ 

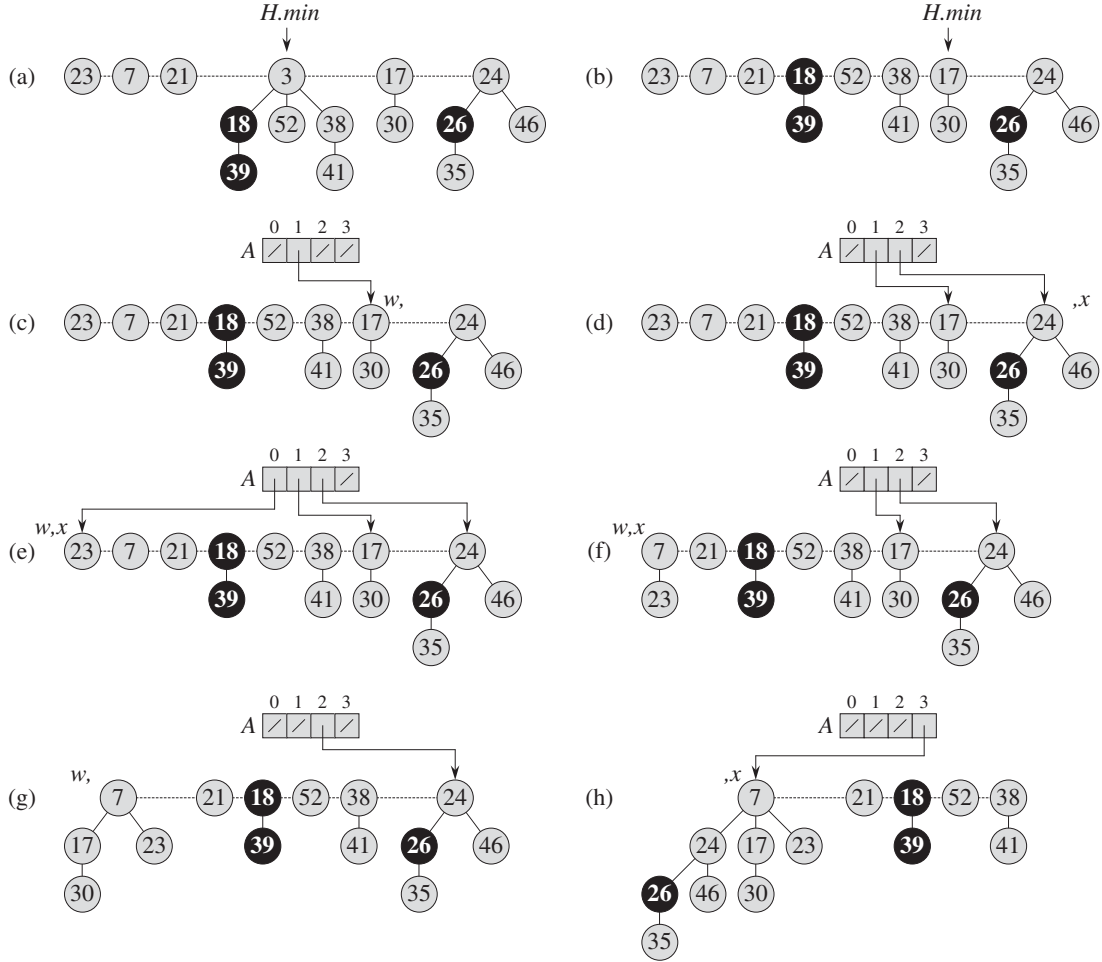
```

Şekil 19.4 te gösterildiği gibi, FIB-HEAP-EXTRACT-MIN ilk önce minimum düğümün çocuklarından birini kök yaparak ve daha sonra minimum düğümü kök listesinden çıkararak çalışır. Daha sonra her dereceden en fazla bir kök kalıncaya kadar eşit dereceli kökleri bağlayarak düzenler.

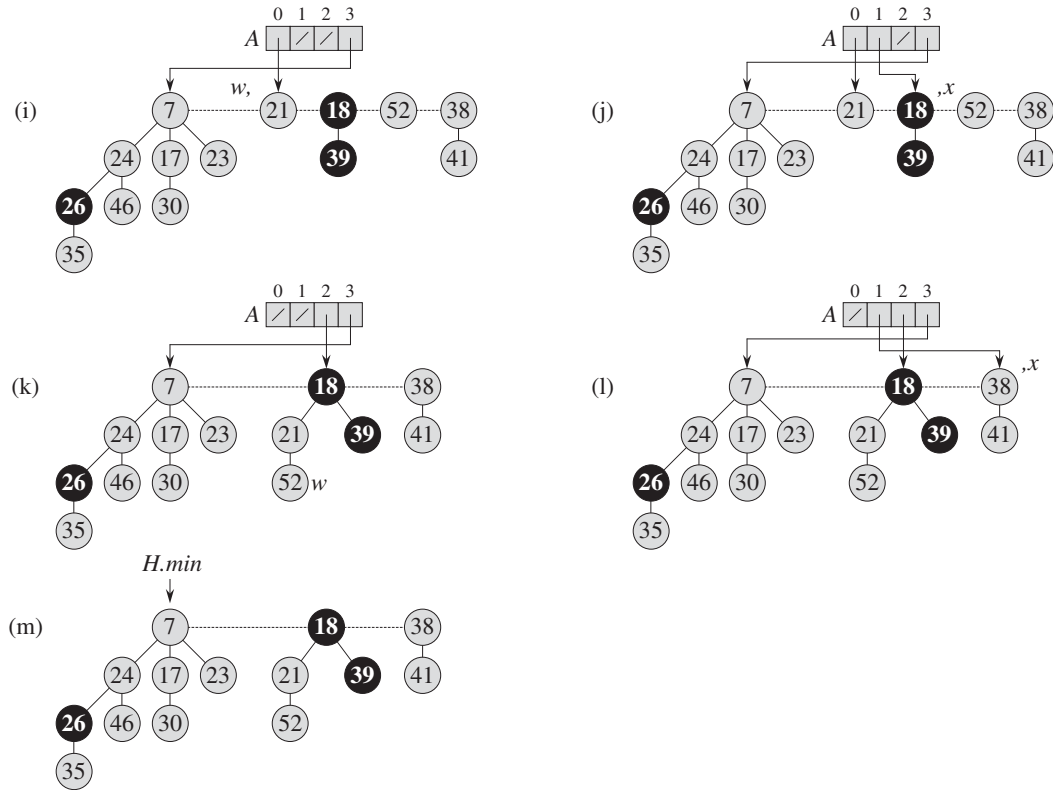
Satır 1 de z işaretçisini minimum düğüm olarak saklayarak başlanır, prosedür en sonda bu işaretçiyi döndürür. Eğer z NIL değerini almışsa H Fibonacci bellek yığını zaten boştur ve yapılacak başka bir şey yoktur. Değilse, Satır 3-5 te z nin çocuklarının hepsi H nin kökü yapılarak (kök listesine eklenerek) ve Satır 6 da z yi listeden çıkararak z düğümü silinir. Satır 6 dan sonra eğer z kendi sağ kardeşi ise, z kök listesindeki tek düğüm demektir ve çocuğu yoktur, bu yüzden tek yapılması gereken Satır 8 deki Fibonacci bellek yığını boşaltmak ve z yi döndürmektir. Değilse, $H.min$ değeri kök listesindeki z den başka bir kökü (bu durumda z nin sağ kardeşi) gösterecek şekilde düzenlenir, FIB-HEAP-EXTRACT-MIN bittikten sonra yeni minimum düğüm olmasına gerek yoktur. Şekil 19.4(b), Şekil 19.4(a) daki Fibonacci bellek yığınının Satır 9 çalıştıktan sonraki halini göstermektedir.

Fibonacci bellek yığınındaki ağaç sayısını azaltacağımız bir sonraki adım H nin kök listesini CONSOLIDATE(H) çağrısı ile başarılan *sağlamlaştırma*dır. Kök listesinin sağlamlaştırılması aşağıdaki adımların kök listesindeki her kökün farklı bir değer alıncaya kadar tekrar edilmesiyle sağlanır:

1. Kök listesinde aynı dereceye sahip iki kök bul (x ve y). Genellik kaybı olmadan $x.key \leq y.key$ olsun.
2. y yi x e *bağla*: y yi kök listesinden çıkar, FIB-HEAP-LINK prosedürü ile y yi x in çocuğu yap. Bu prosedür $x.degree$ değerini artırıp y deki işareti temizler.



Şekil 19.4 FIB-HEAP-EXTRACT-MIN işleyişi. (a) H Fibonacci bellek yığını, (b) Minimum düğüm z kök listesinden çıkarılıp çocuklarının kök listesine eklenmesinden sonraki durum. (c) – (e) CONSOLIDATE prosedürünün Satır 4-14 ündeki **for** döngüsünün 3. iterasyonundan her birinden sonraki A dizisi ve ağaçlar. Prosedür kök listesini işlemeye $H.min$ ile gösterilen düğümden başlar ve sağ işaretçilerle devam eder. Her kısım bir iterasyondan sonraki w ve x değerlerini göstermektedir. (f)–(h) Satır 7–13 teki **while** döngüsünün her bir iterasyonundan sonra gösterilen w ve x değerleri ile **for** döngüsünün bir sonraki iterasyonu. (f) **while** döngüsünün ilk seferindeki durumu göstermektedir. 23 anahtar değerli düğüm, x in şimdi gösterdiği 7 anahtar değerli düğüme bağlanmıştır. (g) de 17 anahtar değerli düğüm halen x in gösterdiği 7 anahtar değerli düğüme bağlanmıştır. (h) 24 anahtar değerli düğüm 7 anahtar değerli düğüme bağlanmıştır. $A[3]$ tarafından daha önceden hiçbir düğüm gösterilmediği için, **for** döngüsünün sonunda $A[3]$ son durumdaki ağacın kökünü gösterecek şekilde ayarlanmıştır.



Şekil 19.4 devamı (i)–(l) for döngüsünün sonraki dört iterasyonunun her birinden sonraki durum. (m) A dizisinden kök listesinin yeniden oluşturulmasından ve yeni $H.min$ işaretçisinin belirlenmesinden sonraki H Fibonacci bellek yığını.

CONSOLIDATE prosedürü, derecelerine göre köklerin izini tutan yardımcı olarak $A[0..D(H.n)]$ dizisini kullanır. Eğer $A[i] = y$ ise y düğümü $y.degree = i$ olacak şekilde bir kök düğüm demektir. Tabi ki, diziyi belleğe yerleştirmek için maksimum derecede üst sınır olan $D(H.n)$ değerinin nasıl hesaplanacağını bilmemiz gerekir, ancak bunun nasıl yapıldığını Kısım 19.4 te göreceğiz.

```

CONSOLIDATE( $H$ )
1   $A[0..D(H.n)]$  yeni bir dizi olsun
2  for  $i = 0$  to  $D(H.n)$ 
3       $A[i] = \text{NIL}$ 
4  for  $H$  nin kök listesindeki her bir  $w$  düğümü için
5       $x = w$ 
6       $d = x.degree$ 
7      while  $A[d] \neq \text{NIL}$ 
8           $y = A[d]$  //  $x$  ile aynı dereceli başka bir düğüm
9          if  $x.key > y.key$ 
10              $x$  ile  $y$  yi yer değiştir
11             FIB-HEAP-LINK( $H, y, x$ )
12              $A[d] = \text{NIL}$ 
13              $d = d + 1$ 
14       $A[d] = x$ 
15   $H.min = \text{NIL}$ 
16  for  $i = 0$  to  $D(H.n)$ 
17      if  $A[i] \neq \text{NIL}$ 
18          if  $H.min == \text{NIL}$ 
19               $H$  için sadece  $A[i]$  yi içeren bir kök listesi oluştur
20               $H.min = A[i]$ 
21          else  $A[i]$  yi  $H$  nin kök listesine ekle
22              if  $A[i].anahtar < H.min.key$ 
23                   $H.min = A[i]$ 

FIB-HEAP-LINK
1   $y$  yi  $H$  nin kök listesinden çıkar
2   $y$  yi  $x$  in çocuğu yap,  $x.degree$  yi artır
3   $y.mark = \text{FALSE}$ 

```

Detaylı anlatmak gerekirse, CONSOLIDATE prosedürü şöyle çalışır: Satır 1-3 her bir elemanı NIL olan A dizisini oluşturur. Satır 4-14 teki **for** döngüsü kök listesindeki her w kökünü işler. Kökleri birbirine bağladıkça w bir başka düğüme bağlanabilir ve artık kök olmaktan çıkar. Yine de w , kendisi olan ya da olmayan bir x düğümünden dallanan bir ağaçta yer alır. Her dereceden en çok bir kök olmasını istediğimiz için, x ile aynı dereceye sahip bir y kökü olup olmadığını belirlemek için A dizisine bakarız. Eğer varsa, x ve y e köklerini bağlarız ama bağlandıktan sonra x in hala kök kaldığını garanti etmemiz gerekir. Yani y yi x e, y nin anahtarı x ten küçükse iki kökün işaretçilerini ilk değiştirmeden sonra bağlarız. y x e bağlandıktan sonra, x in derecesi bir artırılır ve bu süreç x ve derecesi x in yeni derecesine eşit olan başka bir köke bağlayarak x ile aynı dereceli kök

kalmayıncaya kadar devam eder. Daha sonra A nın ilgili elemanını x i gösterecek şekilde düzenleyerek, böylece daha sonra kökleri işlerken, x i işlediğimiz kökler arasında kendi derecesinde tek kök olarak kaydederiz. Bu **for** döngüsü sona erdiğinde her dereceden tek bir kök kalacak ve A dizisi kalan kökleri gösterecektir.

Satır 7-13 teki **while** döngüsü, tekrarlı olarak w düğümünü içeren ağacın x kökünü, aynı dereceli başka kök kalmayıncaya kadar x ile aynı dereceli köke sahip başka bir ağaca bağlar. Bu **while** döngüsü aşağıdaki değişmezliği korur:

while döngüsünün her iterasyonunun başlangıcında $d = x.degree$ dir

Bu döngüdeki değişmezlik şu şekilde kullanılır:

Başlangıç: Satır 6, döngüye ilk girildiğindeki döngü değişmezinin gerçekleşmesini sağlar.

Yineleme: **while** döngüsünün her iterasyonunda $A[d]$ bir y kökünü işaret eder. $d = x.degree = y.degree$ olduğu için x ile y yi bağlamak isteriz. Bağlama işlemi sonunda x veya y den küçük anahtara sahip olan ötekine ebeveyn olur, böylece Satır 9-10 gerek duyulursa x ile y nin işaretçilerini değiştirir. Sonra Satır 11 deki FIB-HEAP-LINK(H, y, x) çağrısı ile y, x e bağlanır. Bu çağrı $x.degree$ değerini artırır ancak $y.degree, d$ olarak kalır. y düğümü artık bir kök değildir ve Satır 12 A dizisindeki onu gösteren işaretçiyi kaldırır. FIB-HEAP-LINK çağrısı $x.derece$ değerini artırdığı için Satır 13 $d = x.degree$ işlemi ile değişmezliği korumuş olur.

Sonlandırma: **while** döngüsü, $A[d] = \text{NIL}$ olana kadar, yani x ile aynı dereceli başka kök kalmayıncaya kadar sürdürülür.

While döngüsü bittikten sonra Satır 14 te $A[d]$ x e set edilir ve **for** döngüsünün bir sonraki iterasyonu gerçekleştirilir.

Şekil 19.4 (c) – (e) A dizisini ve Satır 4-14 teki **for** döngüsünün ilk üç iterasyonundan sonra oluşan ağaçları göstermektedir. **For** döngüsünün bir sonraki iterasyonunda 3 bağ oluşur. Bu bağların sonuçları Şekil 19.4 (f)-(h) de gösterilmektedir. Şekil 19.4(i)-(l) **for** döngüsünün sonraki dört iterasyonunu göstermektedir.

Geriye kalan temizleme işlemidir. Satır 4-14 teki **for** döngüsü tamamlandığında Satır 15 kök listesini boşaltır ve Satır 16-23, A dizisinden listeyi tekrar oluşturur. Oluşan Fibonacci bellek yığını Şekil 19.4(m)de görülmektedir. Kök listesi sağlamalaştırıldıktan sonra FIB-HEAP-EXTRACT-MIN Satır 11 de $H.n$ yi eksilterek ve Satır 12 de silinen z düğümünü gösteren işaretçiyi geri döndürerek sonlanır.

Artık n düğümlü, bir Fibonacci bellek yığınının minimum düğümün çıkarılmasının amortize maliyetinin $O(D(n))$ olduğunu gösterebiliriz. H , FIB-HEAP-EXTRACT-MIN işlemine girmeden önceki Fibonacci bellek yığını ifade etsin.

Minimum düğümün gerçek maliyetini hesaplamakla başlayalım. $O(D(n))$ katkısı CONSOLIDATE prosedürünün 2-3. ve 16-23. satırlarında minimum düğümün

en fazla $D(n)$ çocuğu üzerinde çalışmasından gelir. Geriye bir toparlama analizi kullanacağımız, CONSOLIDATE ın Satır 4-14 ünden gelen katkıyı analiz etmek kalır. CONSOLIDATE çağrıldığında kök listesinin uzunluğu, $t(H)$ kök listeli düğümden, eksi çıkarılan kök düğümden, artı çıkarılan düğümün en fazla $D(n)$ sayıdaki çocuklarından oluştuğu için en çok $D(n) + t(H) - 1$ olabilir. **For** döngüsünün 4-14. Satırlarındaki herhangi bir iterasyonunda, 7-13. Satırlardaki **while** döngüsünün iterasyon sayısı kök listesine bağlıdır. Ancak biliyoruz ki **while** döngüsünün her seferinde köklerden birisi bir diğerine bağlanır ve bu yüzden **for** döngüsünün tüm iterasyonlarındaki **while** döngüsünün iterasyon sayısı en fazla kök listesindeki köklerin sayısı kadardır. Bu sebeple **for** döngüsünde gerçekleştirilen toplam iş miktarı en fazla $D(n) + t(H)$ ile orantılıdır. Bu sebeple minimum düğümü çıkarmadaki gerçek iş $O(D(n) + t(H))$ kadardır.

En fazla $D(n) + 1$ kök kaldığı ve işlem sırasında hiçbir düğüm işaretlenmediği için minimum düğümü çıkarmadan önceki potansiyel $t(H) + 2m(H)$ ve sonraki potansiyel ise $(D(n) + 1) + 2m(H)$ kadardır. $O(t(H))$ taki gizli sabiti baskılamak için potansiyel birimlerini yükseltebildiğimiz için amortize maliyet en çok

$$\begin{aligned} & (O(D(n) + t(H)) + ((D(n) + 1) + 2m(H)) - (t(H) + 2m(H))) \\ &= O(D(n)) + O(t(H)) - t(H) \\ &= O(D(n)) \end{aligned}$$

kadar olur. Öngörülse olarak her bağı gerçekleştirmesindeki maliyet linkin kök sayısını bir azaltmasına bağlı olarak potansiyeldeki azalmayla karşılanır. Kısım 19.4 te göreceğiz ki $D(n) = O(\log n)$ ve bu yüzden minimum düğüm çıkarmanın amortize maliyeti $O(\log n)$ dir.

Alıştırmalar

19.2-1

Şekil 19.4(m) de gösterilen Fibonacci bellek yığınının FIB-HEAP-EXTRACT-MIN çağrıldıktan sonra oluşan Fibonacci bellek yığını gösteriniz.

19.3 Anahtar azaltma ve düğüm silme

Bu kısımda bir Fibonacci bellek yığınındaki bir düğümün anahtarının $O(1)$ amortize zamanda nasıl azaltılacağını ve n düğümlü bir Fibonacci bellek yığınının herhangi bir düğümü $O(D(n))$ amortize zamanda nasıl silineceğini göstereceğiz.

Kısım 19.4 te FIB-HEAP-EXTRACT-MIN ve FIB-HEAP-DELETE işlemlerinin $O(\log n)$ amortize zamanda çalıştığını vurgulayan $D(n)$ maksimum derecesinin $O(\log n)$ olduğunu göstereceğiz.

Anahtar azaltma

Aşağıdaki FIB-HEAP-DECREASE-KEY işlemi için yazılmış kaba kodda, bağlı listedeki bir düğümü çıkarmanın çıkarılan düğümdeki yapısal özelliklerde bir değişiklik olmadığını varsayıyoruz.

FIB-HEAP-DECREASE-KEY

```

1  if  $k > x.key$ 
2      hata “yeni anahtar mevcut anahtardan büyük”
3   $x.key = k$ 
4   $y = x.e$ 
5  if  $y \neq \text{NIL}$  ve  $x.key < y.key$ 
6      CUT ( $H, x, y$ )
7      CASCADING-CUT( $H, y$ )
8  if  $x.key < H.min.key$ 
9       $H.min = x$ 
```

CUT(H, x, y)

```

1   $y$  nin çocuk listesinden  $x$  i çıkar,  $y.degree$  yi azalt
2   $x$  i  $H$  nin kök listesine ekle
3   $x.e = \text{NIL}$ 
4   $x.mark = \text{FALSE}$ 
```

CASCADING-CUT (H, y)

```

1   $z = y.e$ 
2  if  $z \neq \text{NIL}$ 
3      if  $y.mark == \text{FALSE}$ 
4           $y.mark = \text{TRUE}$ 
5      else CUT ( $H, y, z$ )
6      CASCADING-CUT ( $H, z$ )
```

FIB-HEAP-DECREASE-KEY prosedürü şöyle çalışır: Satır 1-3 yeni anahtarın x in mevcut anahtar değerinden daha büyük olmamasını sağlar ve sonra yeni anahtarı x e atar. Eğer x bir kök ise veya $x.key \geq y.key$ ise, min-bellek yığını sırası ihlal edilmediği için yapısal bir değişiklik yapılmasına gerek yoktur. Satır 4-5 bu koşulu test eder.

Eğer min-bellek yığını sırası ihlal edilmiş ise, birçok değişiklik gerçekleşir. Satır 6 da x i **keserek** başlarız. CUT prosedürü, x ile ebeveyni y arasındaki bağı keserek x i bir kök yapar.

İstenilen zaman sınırlarını elde etmek için işaret özelliklerini kullanacağız. Bu özellik, her bir düğüm geçmişinin bir parçasını kaydeder. Aşağıdaki olayların x düğümünde gerçekleştiğini düşünelim:

1. Herhangi bir zamanda x bir kök düğümdü,
2. Daha sonra x , başka bir düğüme bağlandı (o düğümün çocuğu oldu),
3. x in iki çocuğu kesilmelerle ayrıldı.

İkinci çocuk kaybolur kaybolmaz, x i ebeveyninden ayırarak onu yeni bir kök haline getiririz. Eğer adım 1 ve adım 2 gerçekleştiyse ve x in bir çocuğu ayrılmışsa $x.mark$ özelliği TRUE dir. Bu yüzden CUT prosedürü Satır 4 te $x.mark$ değerini temizler (FIB-HEAP-LINK daki Satır 3 ün $y.mark$ ı neden temizlediğini şimdi görebiliriz: y düğümü başka bir düğüme bağlanmaktadır ve bu yüzden Adım 2 işlenmektedir. Bir sonraki y nin çocuğu kesildiğinde $y.mark$ TRUE olarak atanacaktır).

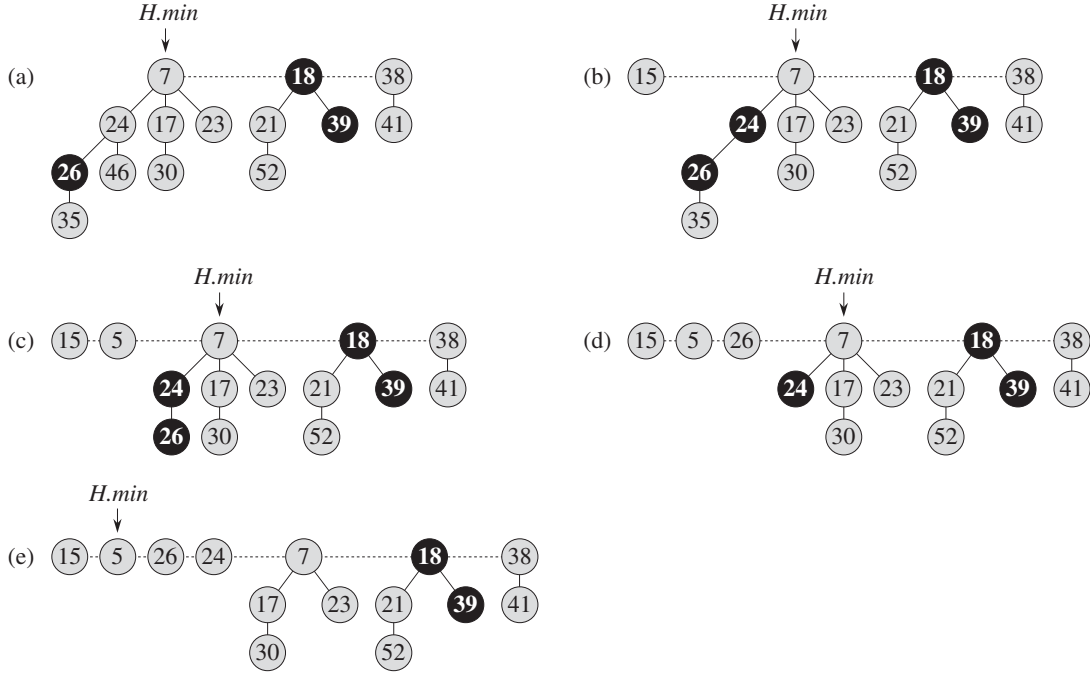
Henüz her şey bitmedi çünkü x , y başka bir düğüme bağlandıktan sonra ebeveyni y den kesilen ikinci çocuk olabilir. Bu sebeple FIB-HEAP-DECREASE-KEY Satır 7, y üzerinde **basamaklı-kesme** işlemi gerçekleştirmeye çalışır. Eğer y bir kök düğüm ise, CASCADING-CUT Satır 2 deki test prosedürün hemen dönmesine sebep olur. Eğer y işaretlenmemiş ise, ilk çocuğu kesildiği için prosedür onu 4. Satırda işaretler ve geri döner. Eğer y işaretlenmiş ise ikinci çocuğunu yeni kaybetmiş demektir ve y 5. Satırda kesilir ve CASCADING-CUT 6. Satırda y nin ebeveyni z için özyinelemeli olarak kendisini tekrar çağırır. CASCADING-CUT prosedürü yukarıya doğru bir kök ya da işaretlenmemiş bir düğüm buluncaya kadar kendini çağırmaya devam eder.

Bütün basamaklı kesmeler gerçekleştirildiğinde, FIB-HEAP-DECREASE-KEY daki 8-9. Satırlar, $H.min$ değerini (gerekliyorsa) güncelleyerek işlemi sonlandırır. Anahtarı değiştirilen tek düğüm, anahtarı azaltılan x düğümüdür. Bu yüzden yeni en küçük düğüm ya orijinal en küçük düğümdür ya da x düğümüdür.

Şekil 19.5 te FIB-HEAP-DECREASE-KEY prosedürünün Şekil 19.5(a) da gösterilen Fibonacci bellek yığını üzerinde iki kez çalıştırılması gösterilmektedir. İlk çağrıldığında hiçbir basamaklı kesme yoktur (Şekil 19.5(b)). İkincide ise iki adet basamaklı kesme yer almaktadır (Şekil 19.5(c)-(e)).

Şimdi FIB-HEAP-DECREASE-KEY işleminin amortize maliyetinin $O(1)$ olduğunu göstereceğiz. Öncelikle gerçek maliyeti belirlemekle başlayalım. FIB-HEAP-DECREASE-KEY, $O(1)$ artı basamaklı kesme işlemleri için geçen süre kadar zaman alır. Örnek olarak bir FIB-HEAP-DECREASE-KEY isteğinin c adet CASCADING-CUT çağrısı yaptığı varsayalım (FIB-HEAP-DECREASE-KEY 7. Satırdan yapılan ve takip eden $c - 1$ adet CASCADING-CUT özyinelemesi). Her bir CASCADING-CUT çağrısı, özyinelemeler hariç $O(1)$ zaman alır. Bu sebeple FIB-HEAP-DECREASE-KEY in gerçek maliyeti özyinelemeli çağrılar dahil $O(c)$ dir.

Daha sonra potansiyeldeki değişimi hesaplayalım. H , FIB-HEAP-DECREASE-KEY işleminden hemen önceki Fibonacci bellek yığını olsun. FIB-HEAP-DEC-



Şekil 19.5 FIB-HEAP-DECREASE-KEY in iki çağırımı. (a) Başlangıç Fibonacci bellek yığını. (b) Anahtar değeri 46 olan düğüm anahtar değerini 15 e düşürdü. Düğüm kök olur ve daha önceden işaretlenmemiş olan ebeveyni (24 anahtar değerli) işaretli hale gelir. (c)–(e) 35 anahtarlı düğüm anahtar değerini 5 e düşürdü. (c) de şimdi 5 anahtar değerli olan düğüm kök olur. Onun 26 anahtar değerli ebeveyni işaretlenir, böylece basamaklı kesme oluşur. 26 anahtar değerli düğüm ebeveyninden kesilir ve (d) de işaretsiz bir kök olur. 24 anahtar değerli düğüm de işaretli olduğu için başka bir basamaklı kesme oluşur. Bu düğüm ebeveyninden kesilir ve (e) de işaretsiz kök yapılır. Basamaklı kesme bu noktada durur çünkü y anahtar değerli düğüm bir köktür (Bu düğüm kök olmasaydı bile işaretsiz olduğu için basamaklı kesme duracaktı). (e) yeni minimum düğümü gösteren $H.min$ ile FIB-HEAP-DECREASE-KEY işleminin sonucunu göstermektedir.

REASE-KEY 6. Satırdaki CUT çağırısı, kök düğümü x olan yeni bir ağaç oluşturur ve x in işaret bitini temizler (zaten FALSE değeri atanmış olabilir). Sonuncusu hariç her bir CASCADING-CUT çağırısı, işaretli bir düğümü keser ve işaret bitini temizler. Daha sonra Fibonacci bellek yığını $t(H) + c$ tane ağaç (orijinal $t(H)$ ağaç, basamaklı kesmelerle oluşan $c - 1$ adet ağaç ve x köklü ağaç) ve en çok $m(H) - c + 2$ adet işaretli düğüm ($c - 1$ tanesi basamaklı kesmelerle temizlenmiş ve BCASCADING-CUT in son çağırımı ile işaretlenmiş olabilen bir düğüm) içerir. Bu yüzden potansiyeldeki değişim en çok

$$((t(H) + c) + 2(m(H) - c + 2)) - (t(H) + 2 m(H)) = 4 - c.$$

Bu yüzden FIB-HEAP-DECREASE-KEY için amortize maliyet en çok

$$O(c) + 4 - c = O(1)$$

olur, çünkü $O(c)$ içindeki gizli sabiti bastırmak için potansiyel birimleri artırılabilir.

Şimdi işaretli düğüm sayısının iki katı olan bir terimi içermesi için neden potansiyel fonksiyonu tanımladığımızı görebilirsiniz. İşaretli bir y düğümü basamaklı kesme ile kesildiğinde, işaret biti temizlenir bu da potansiyeli 2 birim azaltır. Bir birim potansiyel kesme ve işaret bitini temizlemek için, diğer birim ise y düğümünün kök düğüm olması sebebiyle potansiyeldeki artış içindir.

Düğüm Silme

Aşağıdaki kabakod n düğümlü bir Fibonacci bellek yığınının $O(D(n))$ amortize zamanda bir düğüm siler. Mevcut Fibonacci bellek yığınının $-\infty$ değerli anahtar bulunmadığı varsayılmaktadır.

FIB-HEAP-DELETE(H, x)

1 FIB-HEAP-DECREASE-KEY ($H, x, -\infty$)

2 FIB-HEAP-EXTRACT-MIN (H)

FIB-HEAP-DELETE x düğümünün anahtarını $-\infty$ değerini vererek Fibonacci bellek yığını içindeki en küçük düğüm olmasını sağlar. FIB-HEAP-EXTRACT-MIN prosedürü x düğümünü Fibonacci bellek yığınının çıkarır. FIB-HEAP-DELETE in amortize zamanı, FIB-HEAP-DECREASE-KEY in $O(1)$ amortize zamanı ile FIB-HEAP-EXTRACT-MIN in $O(D(n))$ amortize zamanlarının toplamı kadardır. Kısım 19.4 te gösterileceği gibi $D(n) = O(\log(n))$ olduğundan, FIB-HEAP-DELETE in amortize zamanı $O(\log(n))$ dir.

Alıştırmalar

19.3-1

Bir Fibonacci bellek yığınının x kökünün işaretli olduğu varsayalım. x düğümünün nasıl işaretli bir kök olduğunu açıklayınız. Başka bir düğüme bağlandığında ve sonra bir çocuğunu kaybettiğinde bile bir kök değilken x in işaretlenmesinin analiz için önemli olmadığını tartışınız.

19.3-2

Birleştirme analizi kullanarak işlem başına ortalama maliyet olarak FIB-HEAP-DECREASE-KEY amortize zamanının $O(1)$ olduğunu doğrulayınız.

19.4 Maksimum dereceyi sınırlama

FIB-HEAP-EXTRACT-MIN ve FIB-HEAP-DELETE in amortize zamanlarının $O(\log n)$ olduğunu kanıtlamak için n düğümlü bir Fibonacci bellek yığınınındaki herhangi bir düğümün derecesinin üst sınırı $D(n)$ nin $O(\log(n))$ olduğunu göstermemiz gerekir. Özellikle göstermeliyiz ki, $D(n) \leq \lfloor \log_\phi n \rfloor$ dir. Buradaki ϕ altın oran denklem (3.24) te aşağıdaki gibi tanımlanmıştır.

$$\phi = (1 + \sqrt{5})/2 = 1.61803 \dots$$

Analizin anahtarı şu şekildedir. Bir Fibonacci bellek yığınınındaki her x düğümü için, x köklü altağaçlarda bulunan ve x in kendisini de içere düğüm sayılarını gösteren size (x) tanımlanır. (x in kök listesinde olması gerekmediğini belirtelim – herhangi bir düğüm olabilir.) ölçü(x) in $x.degree$ de üssel olduğunu göstermeliyiz. Akılda tutulmalıdır ki $x.degree$ her zaman x in derecesinin kesin sayısı olarak elde edilir.

Lemma 19.1

x , bir Fibonacci bellek yığınınında herhangi bir düğüm olsun ve $x.degree = k$ olduğu varsayalım. $y_1, y_2, \dots, y_k$ de x e bağlanma sırasıyla çocuklarının gösterimi olsun. Bu durumda $y_1.degree \geq 0$ ve $y_i.degree \geq i - 2$ ($i = 2, 3, \dots, k$) olur.

Kanıt $y_1.degree \geq 0$ olduğu açıktır.

$i \geq 2$ için, söyleyebiliriz ki y_i x e bağlandığında, $y_1, y_2, \dots, y_{i-1}$ x in çocuklarıydı, bu yüzden $x.degree \geq i - 1$. y_i düğümü sadece eğer $x.degree = y_i.degree$ ise x e bağlandığı için aynı zamanda $y_i.degree \geq i - 1$ olmalıdır. Ancak ondan sonra y_i en fazla bir çocuk kaybetmiştir, yoksa eğer 2 çocuk kaybetmiş olsaydı x ten CASCA-DING-CUT ile kesilecektir. Bu da demektir ki $y_i.degree \geq i - 2$ olduğuna karar verilebilir. ■

Son olarak analizin “Fibonacci bellek yığınları” adının açıklandığı kısmına geldik. Kısım 3.2 den hatırlanabileceği gibi $k = 0, 1, 2, \dots$ için k .ncı Fibonacci sayısı şu yineleme ile tanımlanır:

$$F_k = \begin{cases} 0 & k = 0 \text{ ise,} \\ 1 & k = 1 \text{ ise,} \\ F_{k-1} + F_{k-2} & k \geq 2 \text{ ise.} \end{cases}$$

Aşağıdaki lemma F_k yı tanımlamanın başka bir yolunu vermektedir.

Lemma 19.2

$k \geq 0$ şartını sağlayan tüm tam sayılar için,

$$F_{k+2} = 1 + \sum_{i=0}^k F_i .$$

Kanıt Kanıt, k dan tümevarım yoluyla gösterilir. $k = 0$ olduğunda

$$\begin{aligned} 1 + \sum_{i=0}^0 F_i &= 1 + F_0 \\ &= 1 + 0 \\ &= F_2 . \end{aligned}$$

Şimdi tümevarımsal hipotez $F_{k+1} = 1 + \sum_{i=0}^{k-1} F_i$ i varsayarak şunu elde ederiz:

$$\begin{aligned} F_{k+2} &= F_k + F_{k+1} \\ &= F_k + \left(1 + \sum_{i=0}^{k-1} F_i \right) \\ &= 1 + \sum_{i=0}^k F_i . \end{aligned}$$

■

Lemma 19.3

$k \geq 0$ şartını sağlayan tüm tam sayılar için, $(k + 2)$ nci Fibonacci sayısı $F_{k+2} \geq \phi^k$ eşitsizliğini sağlar.

Kanıt Kanıt, k dan tümevarım yoluyla gösterilir. Temel olgu $k = 0$ ve $k = 1$ içindir. $k = 0$ olduğunda $F_2 = 1 = \phi^0$ ve $k = 1$ olduğunda $F_3 = 2 > 1.619 > \phi^1$ elde edilir. Tümevarımsal adım $k \geq 2$ olduğu durumlar içindir ve $i = 0, 1, \dots, k-1$ için $F_{i+2} > \phi^i$ olduğu varsayılır. Hatırlayın ki ϕ , denklem (3.23) deki $x^2 = x + 1$ in pozitif köküdür. Bu sebeple şunu elde ederiz:

$$\begin{aligned} F_{k+2} &= F_{k+1} + F_k \\ &\geq \phi^{k-1} + \phi^{k-2} \quad (\text{tümevarımsal hipotez ile}) \\ &= \phi^{k-2}(\phi + 1) \\ &= \phi^{k-2} \cdot \phi^2 \quad (\text{denklem (3.23) ten}) \\ &= \phi^k . \end{aligned}$$

■

Aşağıdaki lemma ve onun sonucu da analizi tamamlamaktadır.

Lemma 19.4

x , bir Fibonacci bellek yığnında herhangi bir düğüm olsun ve $k = x.degree$ olsun. Bu durumda $size(x) \geq F_{k+2} \geq \phi^k$ ve $\phi = (1 + \sqrt{5})/2$ dir.

Kanıt Herhangi bir Fibonacci bellek yığnındaki herhangi bir k dereceli düğümün minimum olası büyüklüğü s_k ile tanımlansın. Önemli olarak $s_0 = 1$ ve $s_1 = 2$ olsun. s_k sayısı en çok $size(x)$ kadardır ve bir düğüme çocuk eklemek o düğümün büyüklüğünü azaltamayacağı için s_k nın değeri k ile monoton bir şekilde artar. Herhangi bir Fibonacci monoton $z.degree = k$ ve $size(z) = s_k$ olan bir z düğümünü ele alalım. $s_k \leq size(z)$ olduğu için, $size(z)$ üzerine, s_k üzerindeki bir alt sınır bulunarak alt sınır hesaplanır. Lemma 19.1 deki gibi $y_1, y_2, \dots, y_k, z$ nin kendisine sırayla bağlanan çocukları olsun. s_k değerini sınırlamak için, z nin kendisini bir ve ilk çocuğu y_1 i ($size(y_1) \geq 1$ olan) bir sayarız. Böylece

$$\begin{aligned} size(x) &\geq s_k \\ &\geq 2 + \sum_{i=2}^k s_{y_i.degree} \\ &\geq 2 + \sum_{i=2}^k s_{i-2} , \end{aligned}$$

elde edilir. Son satır Lemma 19.1 den ($y_i.degree \geq i-2$) ve s_k nın monotonluğundan ($s_{y_i.degree} \geq s_{i-2}$) devam eder.

Şimdi tümevarım yoluyla k nın negatif olmayan tüm tam sayı değerleri için $s_k \geq F_{k+2}$ olduğunu gösterelim. Yine $k=0$ ve $k=1$ için temel olgular önemsizdir. Tümevarım adımı için $k \geq 2$ ve $s_i \geq F_i + 2$ ($i=0, 1, \dots, k-1$) olduğunu varsayalım.

$$\begin{aligned} s_k &\geq 2 + \sum_{i=2}^k s_{i-2} \\ &\geq 2 + \sum_{i=2}^k F_i \\ &= 1 + \sum_{i=0}^k F_i \\ &= F_{k+2} \quad (\text{Lemma 19.2}) \\ &\geq \phi^k \quad (\text{Lemma 19.3}) \end{aligned}$$

Böylece $size(x) \geq s_k \geq F_{k+2} \geq \phi^k$ olduğunu göstermiş oluruz. ■

Sonuç 19.5

n düğümlü herhangi bir Fibonacci bellek yığınındaki herhangi bir düğümün maksimum derecesi $D(n) = O(\log(n))$ dir.

Kanıt x , n düğümlü herhangi bir Fibonacci bellek yığınındaki herhangi bir düğüm olsun ve $k = x.degree$ olsun. Lemma 19.4 e göre $n \geq \text{size}(x) \geq \phi^k$ dir. Her iki tarafın ϕ tabanlı logaritması alınırsa $k \leq \log_{\phi} n$ elde edilir. (Aslında k bir tam sayı olduğu için, $k \leq \lfloor \log_{\phi} n \rfloor$ dir). O yüzden herhangi bir düğümün maksimum derecesi $D(n)$, $O(\log(n))$ olur. ■

Alıştırılmalar**19.4-1**

Profesör Pinokyo n düğümlü bir Fibonacci düğümünün yüksekliğinin $O(\log(n))$ olduğunu iddia etmektedir. Herhangi bir pozitif tam sayı n için, bir dizi Fibonacci bellek yığını işlemi ile n düğümlü doğrusal bir zincir olan bir ağacı içeren bir Fibonacci bellek yığını oluşturulabileceğini sergileyerek yanıldığını gösteriniz.

19.4-2

Basamaklı-kesme kuralını bir x düğümünü k .nci çocuğunu kaybeder etmez ebeveyninden ayrılacak şekilde genelleştirdiğimizi varsayın (Kısım 19.3'teki kural $k = 2$ yi kullanır.) k nın hangi değerleri için $D(n) = O(\log(n))$ olur?

Problemler**19-1 Silme işleminin alternatif uygulaması**

Profesör Pisano silinecek düğüm $H.min$ ile gösterilen düğüm olmadığı durumlarda daha hızlı çalıştığını iddiasıyla FIB-HEAP-DELETE prosedürünün aşağıdaki türevini önermiştir.

PISANO-DELETE(H, x)

```

1  if  $x == H.min$ 
2      FIB-HEAP-EXTRACT-MIN( $H$ )
3  else  $y = x.p$ 
4      if  $y \neq \text{NIL}$ 
5          CUT( $H, x, y$ )
6          CASCADING-CUT( $H, y$ )
7       $x$  in çocukları listesini  $H$  nin kök listesine ekle
8       $x$  i  $H$  nin kök listesinden çıkar.
```

- a. Profesörün bu prosedürün daha hızlı olduğu iddiası kısmen Satır 7 nin $O(1)$ gerçek zamanında işletilebilmesi varsayımına dayanmaktadır. Bu varsayım-daki yanlış nedir?
- b. x $H.min$ olmadığı durum için PISANO-DELETE in gerçek zamanı üzerine iyi bir üst sınır belirleyiniz. Belirttiğiniz bu sınır $x.degree$ ve CASCADING-CUT prosedürünün c sayıda çağırımı cinsinden olmalıdır.
- c. Varsayın ki PISANO-DELETE(H, x) i çağırdık ve sonuç olarak da H' Fibonacci bellek yığını döndü. x düğümünün bir kök olmadığını farz ederek H' potansiyelini $x.degree, c, t(H)$ ve $m(H)$ cinsinden sınırlayınız.
- d. PISANO-DELETE için amortize zamanın $x \neq H.min$ olduğu durumlarda bile asimptotik olarak FIB-HEAP-DELETE den daha iyi olmadığını gösteriniz.

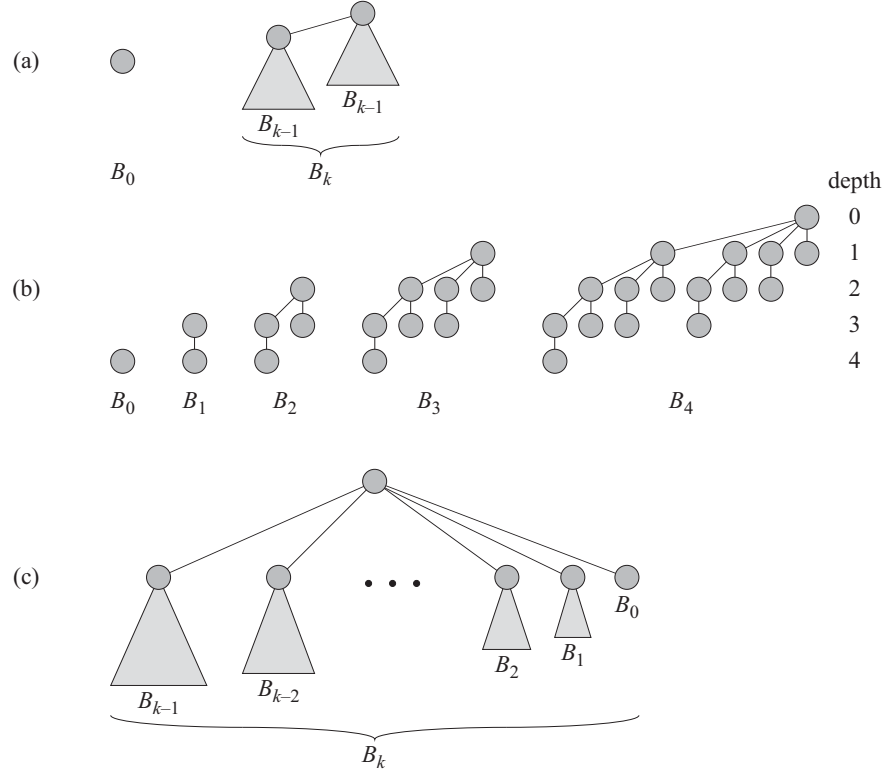
19-2 Binom ağaçları ve Binom bellek yığınları

B_k **binom ağacı** özyinelemeli tanımlanan bir sıralı ağaçtır (bkz. Kısım B.5.2). Şekil 19.6(a) da gösterildiği gibi binom ağacı B_0 tek bir düğüm içerir. Binom ağacı B_k , birinin kökü diğerinin kökünün en soldaki çocuğu olacak şekilde birbirlerine bağlı iki adet binom ağacı B_{k-1} içerir. Şekil 19.6(b) B_0 dan B_4 e kadarki binom ağaçlarını göstermektedir.

- a. Binom ağacı B_k için aşağıdakileri gösteriniz:
 1. 2^k tane düğüm vardır.
 2. ağacın yüksekliği k dır.
 3. $i = 0, 1, 2, \dots, k$ için her i derinliğinde tam olarak düğüm vardır.
 4. kökün derecesi, diğer herhangi bir düğümün derecesinden büyük olan k dır, dahası, Şekil 19.6(c) deki gibi, eğer kökün çocuklarını soldan sağa doğru $k-1, k-2, \dots, 0$ şeklinde numaralayacak olursak bu durumda i .nci çocuk B_i altağacının köküdür.

Bir **binom bellek yığını** H aşağıdaki özellikleri sağlayan binom ağaçları kümesidir.

1. Her düğümün bir anahtarı vardır (Fibonacci bellek yığınındaki gibi)
2. H deki her binom ağacı min-bellek yığını özelliğine uyar.
3. Herhangi bir negatif olmayan k tam sayısı için H bellek yığınında kökü k dereceye sahip en fazla bir binom ağacı vardır.
- b. Bir H binom bellek yığınında n adet düğüm olduğunu varsayın. H nin içerdiği binom ağaçları arasındaki ilişkiyi ve n nin ikili gösterimini tartışınız. H nin en çok $\lfloor \log(n) \rfloor + 1$ adet binom ağacı içerebileceğini gösteriniz.



Şekil 19.6 (a) B_k binom ağacının özyinelemeli ifadesi. Üçgenler köklü altağaçları göstermektedir. (b) B_0 dan B_4 e kadar olan binom ağaçları. B_4 te düğüm derinlikleri gösterilmektedir. (c) B_k ağacının başka bir ifade şekli.

Bir binom bellek yığını şu şekilde ifade ettiğimizi düşünelim. Kısım 19.4 te belirtilen sol-çocuk, sağ-kardeş şeması bir binom bellek yığınındaki her bir binom ağacını temsil eder. Her düğümde o düğümün anahtarı, ebeveynini, en soldaki çocuğunu ve kendi sağındaki kardeşini gösteren işaretçiler (uygunsa bu işaretçiler NIL değerini alır) ve o düğümün derecesi (Fibonacci bellek yığınlarındaki gibi, kaç çocuğu olduğu) bulunur. Kökler, kök derecelerine göre alçaktan yükseğe doğru sıralı tekli bağlı kök listesi oluştururlar ve bu binom ağacına kök listesindeki ilk düğümü gösteren bir işaretçi aracılığıyla erişiriz.

- c. Bir binom bellek yığını gösterimin tanımlamasını tanımlayınız (ör: özellikleri isimlendirin, özelliklerin ne zaman NIL değerini alacağını tanımlayın, kök listesinin nasıl organize edileceğini belirtin) ve bu bölümde Fibonacci bellek yığınlarının işletimindeki gibi belirtilen yedi işlemin binom bellek yığınla-

rında nasıl yürütüleceğini gösterin. Her işlem $O(\log(n))$ (n = binom bellek yığınınındaki düğüm sayısı, UNION işleminde birleştirilen iki binom bellek yığınınındaki düğüm sayısı) en kötü zamanında çalışmalıdır. MAKE-HEAP işlemi sabit zaman almaktadır.

- d. Bir Fibonacci bellek yığınınında sadece birleştirilebilir-bellek yığını işlemlerini gerçekleştirebildiğimizi varsayalım (ör: DECREASE-KEY ya da DELETE işlemlerini kullanmayacağız). Fibonacci bellek yığınınındaki ağaçlar nasıl binom ağacındaki ağaçlara benzer? Nasıl farklılık gösterir? n düğümlü bir Fibonacci bellek yığınınındaki maksimum derecenin en çok $\lfloor \log(n) \rfloor$ olduğunu gösterin.
- e. Profesör McGee Fibonacci bellek yığınlarını temel alan yeni bir veri yapısı geliştirmiştir. Bir McGee bellek yığını Fibonacci bellek yığını ile aynı yapıya sahip ve sadece birleştirilebilir-bellek yığını işlemlerini desteklemektedir. İşlemlerin uygulaması, ekleme ve birleştirmenin son adımları olarak kök listesini oluşturması dışında Fibonacci bellek yığınları ile aynıdır. McGee bellek yığınları üzerindeki işlemlerin en kötü çalışma zamanları nelerdir?

19-3 Diğer Fibonacci-bellek yığını işlemleri

Bir Fibonacci bellek yığını H yi, diğer bütün Fibonacci-bellek yığını işlemlerinin amortize çalışma zamanlarını değiştirmeden 2 yeni işlemi destekleyecek şekilde güçlendirmek istiyoruz.

- a. FIB-HEAP-CHANGE-KEY (H, x, k) işlemi, x düğümünün anahtarını verilen k değeri ile değiştirir. FIB-HEAP-CHANGE-KEY için etkin bir uygulama belirleyin ve uygulamanızın amortize çalışma zamanını k nın $x.key$ değerinden büyük, küçük ve ona eşit olduğu durumlar için analiz edin.
- b. H bellek yığınınından $q = \min(r, H.min)$ olan düğümleri silen FIB-HEAP-PRUNE(H, r) için etkin bir uygulama belirleyin. Silmek için herhangi bir q seçebilirsiniz. Uygulamanızın amortize çalışma zamanını analiz edin (*İpucu*: veri yapısını ve potansiyel fonksiyonu değiştirmeniz gerekebilir).

19-4 2-3-4 bellek yığını

Bölüm 18 de, her ara düğümün (kök dışındaki düğümlerin) iki, üç ya da dört çocuğunun olduğu ve her yaprağın aynı derinlikte olduğu 2-3-4 ağaçları tanıtılmıştı. Bu problemde birleştirilebilir-bellek yığını işlemlerini destekleyen **2-3-4 bellek yığınlarını** uygulayacağız.

2-3-4 bellek yığınları, 2-3-4 ağaçlarından şu yönlerde farklılık gösterir. 2-3-4 bellek yığınlarında sadece yapraklar anahtar saklar ve her yaprak $x, x.key$ özelliğinde tam olarak bir anahtar bulundurulur. Yapraklardaki anahtarlar herhangi bir sırada görülebilir. Her bir iç düğüm $x, x.small$ adında bir değer içerir ve bu da x kökünden dallanan altağaçtaki herhangi bir yaprakta saklanan en küçük anahtara eşittir. Kök r de ağacın yüksekliğini veren $r.height$ adında bir özellik bulunur. Son

olarak 2-3-4 bellek yığınları ana bellekte tutulmak üzere tasarlanmışlardır, bu yüzden disk okuma ve yazmaya gerek yoktur.

Aşağıdaki 2-3-4 bellek yığını işlemlerini gerçekleştiriniz. (a) – (e) kısımlarındaki her işlem n elemanlı 2-3-4 bellek yığnında $O(\log(n))$ zamanında çalışmalıdır. (f) kısmındaki UNION işlemi $O(\log(n))$ zamanda çalışmalıdır (buradaki n iki girdi bellek yığnındaki elemanların sayısıdır).

- a.** En küçük anahtarlı yaprağı gösteren işaretçiyi döndüren MINIMUM.
- b.** Verilen bir x yaprağındaki anahtarı verilen $k \leq x.key$ değerine indiren DECREASE-KEY
- c.** Anahtarı k olan bir x yaprağını ekleyen INSERT.
- d.** Verilen x yaprağını silen DELETE.
- e.** En küçük anahtarlı yaprağı çıkaran EXTRACT-MIN.
- f.** Verilen 2-3-4 bellek yığınlarını birleştirerek tek bir 2-3-4 bellek yığını döndüren ve girdi bellek yığınlarını yok eden UNION.

Bölüm notları

Fredman ve Tarjan [114] Fibonacci bellek yığınlarını tanıttılar. Makaleleri ayrıca Fibonacci bellek yığınlarının tek kaynaklı en kısa yol, tüm en kısa yol çiftleri, ağırlıklı ikili eşleştirme ve minimum kapsayan ağaç problemlerindeki uygulamalarını da açıklamaktadır.

Onların ardından Driscoll, Gabow, Shrairman ve Tarjan [96], Fibonacci bellek yığınlarına alternatif olarak “gevşetilmiş bellek yığın”larını geliştirdiler. Gevşetilmiş bellek yığınlarının iki çeşidini tasarladılar. Bir tanesi Fibonacci bellek yığınları ile aynı amortize zamanı vermektedir. Diğer DECREASE-KEY in $O(1)$ en kötü (amortize olmayan) zamanında çalışmasını ve EXTRACT-MIN ve DELETE in $O(\log(n))$ en kötü zamanında çalışmasını sağlamaktadır. Gevşetilmiş bellek yığınları paralel programlamada Fibonacci bellek yığınlarından daha fazla avantaja sahiptir.

EXTRACT-MIN çağrılarınca döndürülen değer dizilerinin zamanla monoton artması ve verilerin belirli bir aralıktaki tam sayılar olası durumlarında daha hızlı çalışan DECREASE-KEY işlemlerini destekleyen veri yapıları için Bölüm 6 nın bölüm notlarına bakınız.

Önceki bölümlerde bir öncelikli-kuyruğun işlemlerini destekleyen veri yapısını gördük —Bölüm 6 daki ikili bellek yığınları. Bölüm 13 deki kırmızı-siyah ağaçlar,¹ ve Bölüm 19 daki Fibonacci bellek yığınları. Tüm bu veri yapılarında, amortize ya da en kötü durumda en az bir önemli işlem $O(\lg n)$ zaman almaktaydı. Aslında, bu veri yapılarının tümü kararlarını anahtar karşılaştırmaya dayandırdığı için, Kısım 8.1 de sıralamanın alt sınırı olarak verilen $\Omega(n \lg n)$ bize en az bir işlemin $\Omega(\lg n)$ zaman alacağını söylüyor. Neden? Eğer hem INSERT hem de EXTRACT-MIN işlemlerini $o(\lg n)$ zamanda gerçekleştirebilseydik, n adet anahtarı önce n INSERT işlemi, ardından n EXTRACT-MIN işlemi yaparak $o(n \lg n)$ zamanda sıralayabilirdik.

Ancak Bölüm 8 de anahtarlar hakkında bazı ek bilgileri kullanarak $o(n \lg n)$ zamanda sıralama yapabileceğimizi görmüştük. Özel durumda, her biri $0-k$ aralığında bir tam sayı olan n adet anahtarı, sayma sıralaması kullanarak $\Theta(n + k)$ zamanda sıralayabiliriz. Bu zaman da $k = O(n)$ için $\Theta(n)$ olur.

Anahtarlar sınırlı bir aralıkta olduğunda, $\Omega(n \lg n)$ alt sınırını altedebildiğimize göre, tüm öncelikli-kuyruk işlemlerini benzer biçimde $o(\lg n)$ zamanda yapıp yapamayacağımızı merak ediyor olabilirsiniz. Bu bölümde yapabileceğimizi göreceğiz: van Emde Boas ağaçları öncelikli-kuyruk işlemlerini ve diğer birkaç işlemi, en kötü durumda $O(\lg \lg n)$ zamanda destekler. Burada gerek koşul, anahtarların $0-(n-1)$ aralığında, birbirlerinden farklı tam sayılar olmasıdır.

Özel olarak van Emde Boas ağaçları, sayfa 230 da sıralanmış olan dinamik küme işlemlerini —SEARCH, INSERT, DELETE, MINIMUM, MAXIMUM, SUCCESSOR ve PREDECESSOR — $O(\lg \lg n)$ zamanda destekler. Bu bölümde, uydu verilerini bir kenara bırakıp yalnızca anahtarların saklanmasına yoğunlaşacağız. Anahtarlara konsantre olacağımız ve aynı anahtarların saklanmasına izin veremeyeceğimiz için, SEARCH işlemini tanımlamak yerine, x değerinin S dinamik kü-

¹Bölüm 13 EXTRACT-MIN ve DECREASE-KEY işlemlerinin nasıl uygulanacağını doğrudan açıklamamaktadır, ancak bu işlemleri MINIMUM, DELETE ve INSERT işlemlerini destekleyen herhangi bir veri yapısı için kolayca oluşturabiliriz.

mesinde olup olmadığını belirten bir Boole değer döndüren, daha basit MEMBER (ÜYE) (S, x) işlemini uygulayacağız.

Şimdiye kadar n parametresini iki ayrı amaçla kullandık: dinamik kümenin eleman sayısı, ve mümkün değerlerin aralığı. Karışıklığı önlemek için, bundan sonra kümenin eleman sayısı olarak n , mümkün değerlerin aralığı olarak u kullanacağız. O halde, tüm van Emde Boas ağacı işlemleri $O(\lg \lg u)$ zamanda çalışır. $\{0, 1, 2, \dots, u-1\}$ kümesini saklanabilecek tüm değerlerin **evreni**, u değerini de **evrenin boyutu** olarak adlandırıyoruz. Bu bölüm boyunca u değerinin 2'nin bir tam kuvveti olduğunu, yani $k \geq 1$ bir tamsayı olmak üzere $u = 2^k$ olduğunu kabul ediyoruz.

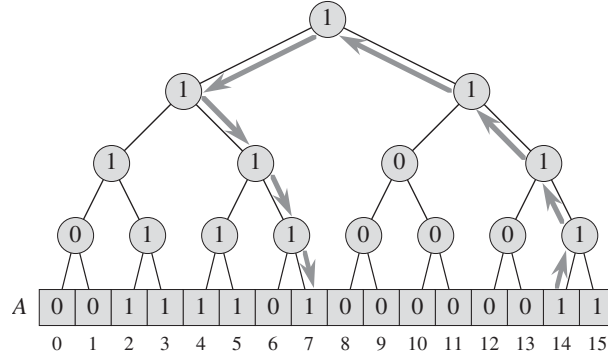
Kısım 20.1 bizi doğru istikamete yönlendirecek bazı basit yaklaşımları inceleyerek başlıyor. Bu yaklaşımları Kısım 20.2 de, hedefimiz olan $O(\lg \lg u)$ zamanı sağlayamayan özyinelemeli ön van Emde Boas yapılarını açıklayarak geliştiriyoruz. Kısım 20.3 ise ön van Emde Boas yapılarında değişiklikler yaparak van Emde Boas ağaçlarını oluşturuyor ve tüm işlemlerin $O(\lg \lg u)$ zamanda nasıl uygulanacağını gösteriyor.

20.1 Ön yaklaşımlar

Bu kısımda, bir dinamik kümeyi saklamak için çeşitli yaklaşımları inceleyeceğiz. Arzuladığımız $O(\lg \lg u)$ zaman sınırlarını hiçbiri sağlayamasa da, bu bölümde daha sonra karşılaştığımızda van Emde Boas ağaçlarını anlamamıza yardımcı olacak sezgileri kazanacağız.

Doğrudan adresleme

Kısım 11.1 de gördüğümüz gibi doğrudan adresleme, bir dinamik kümeyi saklamak için en basit yaklaşımı sağlar. Bu bölümde yalnızca anahtarların saklanmasıyla ilgilendiğimizden, adresleme yaklaşımını Alistırma 11.1-2 de ele alındığı gibi, dinamik kümeyi bir bit vektörü olarak saklamak üzere basitleştirebiliriz. $\{0, 1, 2, \dots, u-1\}$ evreninden değerleri bir dinamik kümede saklamak için, u bitten oluşan bir $A[0..u-1]$ dizisi oluşturuyoruz. $A[x]$ ögesi, eğer x değeri dinamik kümedeyse 1, aksi halde 0 değerini saklar. INSERT, DELETE ve MEMBER işlemlerini bir bit vektörü ile $O(1)$ zamanda yapabiliyor olsak da, geri kalan işlemlerin —



Şekil 20.1 $u = 16$ için $\{2, 3, 4, 5, 7, 14, 15\}$ kümesini temsil eden, bir bit vektörü üzerine eklenmiş bir ikili ağaç. Her iç düğüm ancak ve ancak kendi altağacı 1 içeriyorsa 1 değerini alır. Oklar kümedeki 14 değerinin öncülünü bulmak için izlenen yolu gösteriyor.

MINIMUM, MAXIMUM, SUCCESSOR ve PREDECESSOR— hepsi en kötü durumda $\Theta(u)$ zaman almaktadır, çünkü $\Theta(u)$ adet elemanı taramak zorunda kalabiliriz.² Örneğin, bir küme yalnızca 0 ve $u - 1$ değerlerini içeriyorsa, 0 değerinin ardılını bulmak için, $A[u - 1]$ ögesindeki 1 değerini bulana kadar 1 den başlayıp $u - 2$ adet ögeyi taramamız gerekir.

İkili ağaç yapısı ekleme

Bit vektöründeki uzun taramaları, vektörün üzerine bitlerden oluşan bir ikili ağaç ekleyerek kısaltabiliriz. Şekil 20.1 bunun bir örneğini gösteriyor. Bit vektörünün öğeleri ikili ağacın yapraklarını oluşturmaktadır, ve her iç düğüm ancak ve ancak altağacındaki yapraklardan herhangi biri 1 ise kendisi de 1 değerini alır. Diğer bir deyişle, bir iç düğümde saklanan bit, iki çocuğunun mantıksal-veyasıdır.

Yalın bir bit vektörü uygulaması ile en kötü durumda $\Theta(u)$ zaman alan işlemler, artık ağaç yapısını kullanıyor:

- Kümedeki minimum değeri bulmak için, kökten başla, ve her zaman, 1 içeren düğümlerden en soldakini alarak yapraklara doğru in.
- Kümedeki maksimum değeri bulmak için, kökten başla ve her zaman, 1 içeren düğümlerden en sağdakini alarak yapraklara doğru in.

²Bu bölüm boyunca, MINIMUM ve MAXIMUM prosedürlerinin, dinamik küme boş ise, LD (listede değil) döndürdüğünü ve SUCCESSOR ve PREDECESSOR prosedürlerinin, verilen elemanın sırasıyla, ardılı veya öncülü yok ise, LD döndürdüğünü kabul ediyoruz.

- x in ardılını bulmak için, x ile indislenmiş yapraktan başla, ve soldaki düğümlerden, sağ çocuğu $z + 1$ içeren birine girene kadar köke doğru çık. Sonra da her zaman, 1 içeren düğümlerden en soldakini alarak z düğümünden aşağı in (yani, kökü o sağ çocuk z olan altağaçtaki minimum değeri bul).
- x in öncülünü bulmak için, x ile indislenmiş yapraktan başla ve sağdaki düğümlerden, sol çocuğu $z + 1$ içeren birine girene kadar köke doğru çık. Sonra da her zaman, 1 içeren düğümlerden en sağdakini alarak z düğümünden aşağı in (yani, kökü o sol çocuk z olan alt ağaçtaki maksimum değeri bul).

Şekil 20.1, 14 değerinin öncülü olan 7 değerini bulmak için gidilen yolu gösteriyor.

INSERT ve DELETE işlemlerini de sorunsuz şekilde ilave edebiliriz. Bir değeri eklerken, uygun yapraktan köke doğrudan giden yoldaki her düğümden 1 saklarız. Bir değeri silerken, yol üzerindeki her iç düğümün sakladığı bit o düğümün iki çocuğunun mantıksal-veyası olacak şekilde, uygun yapraktan köke doğru gideriz.

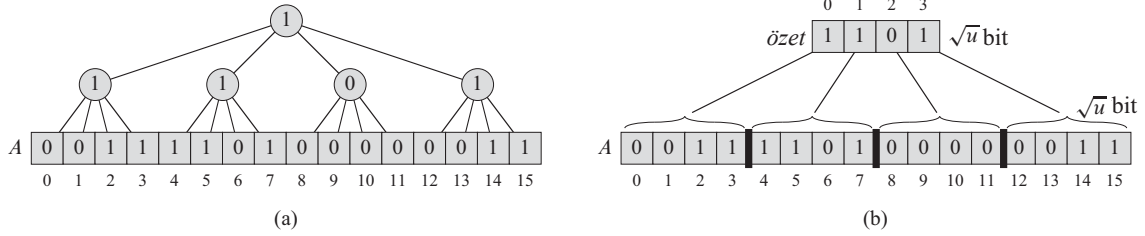
Ağacın yüksekliği $\lg u$ olduğundan ve yukarıdaki işlemlerin her biri ağaç üzerinden en fazla bir defa aşağı ve en fazla bir defa da yukarı doğru geçtiğinden, her bir işlem en kötü durumda $O(\lg u)$ zaman alır.

Bu yaklaşım, kırmızı-siyah ağaç kullanmaktan biraz daha iyidir. Yine de MEMBER işlemini $O(1)$ zamanda gerçekleştirebiliriz. Oysa ki, bir kırmızı-siyah ağaçta arama yapmak $O(\lg n)$ zaman alır. Öte yandan, saklanan elemanların sayısı n evrenin boyutu u dan çok daha küçük ise, tüm diğer işlemler için kırmızı-siyah ağaç daha hızlı olacaktır.

Sabit yükseklikli ağaç ekleme

Daha yüksek dereceli bir ağaç eklersek ne olur? Evrenin boyutunun, k bir tam sayı olmak üzere $u = 2^{2^k}$ olduğunu, böylece $\sqrt{u}$ nun da bir tam sayı olduğunu varsayalım. Bit vektörünün üzerine bir ikili ağaç eklemek yerine, $\sqrt{u}$ derecesinden bir ağaç ekleyelim. Şekil 20.2(a) da Şekil 20.1 deki aynı bit vektörü üzerinde böylesi bir ağaç gösteriliyor. Ortaya çıkan ağacın yüksekliği her zaman 2 dir.

Önceden olduğu gibi, her iç düğüm kendi altağacındaki bitlerin mantıksal veyasını saklıyor. Böylece, 1 derinliğindeki $\sqrt{u}$ adet iç düğüm, $\sqrt{u}$ değerden oluşan her bir grubu özetliyor. Şekil 20.2(b) de görüldüğü gibi bu düğümleri, *summary*[i] ancak ve ancak altdizi $A[i\sqrt{u} \dots (i+1)\sqrt{u} - 1]$ de 1 varsa 1 içermek üzere bir *özet*[$0 \dots \sqrt{u} - 1$] dizisi gibi düşünebiliriz. Bu $\sqrt{u}$ -bitlik A altdizisine i . **küme** adını veriyoruz. Verilen bir x değeri için, $A[x]$ biti, numarası $\lfloor x/\sqrt{u} \rfloor$ olan kümede yer alır. Artık INSERT $O(1)$ zamanlı bir işleme dönüştü: x i eklemek



Şekil 20.2 (a) Şekil 20.1 deki aynı bit vektörünün üzerine eklenmiş $\sqrt{u}$ derecesinden bir ağaç. Her iç düğüm kendi altağacındaki bitlerin mantıksal-veyası saklıyor. (b) Aynı yapının başka bir görünüşü. Burada $\text{özet}[i]$, $A[i\sqrt{u} \dots (i+1)\sqrt{u} - 1]$ alt dizisinin mantıksal-veyası olmak üzere, 1 derinliğindeki iç düğümler bir $\text{özet}[0 \dots \sqrt{u} - 1]$ dizisi olarak değerlendiriliyor.

için, $A[x]$ ve $\text{özet}[\lfloor x/\sqrt{u} \rfloor]$ e 1 ata. MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR ve DELETE işlemlerinin her birini $O(\sqrt{u})$ zamanda yapmak için özet dizisinden yararlanabiliriz:

- Minimum (maksimum) değeri bulmak için, özet dizisinde 1 içeren öğelerden en soldakini (en sağdakini) bul; diyelim ki bu $\text{özet}[i]$ olsun. Sonra da i . kümede, en soldaki (en sağdaki) 1 için doğrusal arama yap.
- x in ardılını (öncülünü) bulmak için, önce kendi kümesi içerisinde sağa (sola) doğru arama yap. Eğer 1 bulunursa, o pozisyon sonucu verir. Aksi halde, $i = \lfloor x/\sqrt{u} \rfloor$ olmak üzere i indisinden başlayarak özet dizisi içinde sağa (sola) doğru arama yap. 1 içeren ilk yer bir kümenin indisidir. O kümenin en solundaki (en sağdaki) 1 i bul. O pozisyon i in ardılıdır (öncülüdür).
- x değerini silmek için, önce $A[x]$ e 0, sonra da $i = \lfloor x/\sqrt{u} \rfloor$ olmak üzere $\text{özet}[i]$ ye i . kümedeki bitlerin mantıksal-veyasını ata.

Yukardaki işlemlerin her birinde, $\sqrt{u}$ bitlik kümelerden en çok ikisinde ve bir de özet dizisinde arama yaptığımızdan, her bir işlem $O(\sqrt{u})$ zaman alır.

İlk bakışta olumsuz bir gelişme kaydetmişiz gibi görünüyor. İkili ağaç eklemek bize, $O(\sqrt{u})$ zamandan asimptotik olarak daha hızlı olan $O(\lg u)$ zamanlı işlemler sağlamıştı. Ancak, $\sqrt{u}$ derecesinden bir ağaç kullanmak van Emde Boas Ağaçları için kilit bir nokta olarak karşımıza çıkar. Sonraki kısımda bu yolda ilerlemeye devam ediyoruz.

Alıştırmalar

20.1-1

Bu kısımdaki veri yapılarını birbirleriyle aynı anahtarları destekleyecek şekilde düzenleyin.

20.1-2

Bu kısımdaki veri yapılarını kendilerine bağlı uydu verileri olan anahtarları destekleyecek şekilde düzenleyin.

20.1-3

Bu kısımdaki yapıları kullanarak bir x değerinin ardıl ve öncülleri bulmamız, x in herhangi bir zamanda o kümede olup olmamasına bağlı değildir. Bir ikili arama ağacındaki x in ardılının x ağaçta saklanmıyor olduğunda nasıl bulunabileceğini gösterin.

20.1-4

$\sqrt{u}$ derecesinden bir ağaç eklemek yerine, $k > 1$ bir sabit olmak üzere, $u^{1/k}$ derecesinden bir ağaç eklediğimizi düşünün. Böyle bir ağacın yüksekliği ne olurdu ve her bir işlem ne kadar zaman alırdı?

20.2 Özyinelemeli bir yapı

Bu kısımda, bit vektörünün üzerine $\sqrt{u}$ derecesinden bir ağaç ekleme fikrini biraz değiştiriyoruz. Önceki kısımda, her bir ögenin $\sqrt{u}$ boyutunda başka bir yapıyı işaret ettiği $\sqrt{u}$ boyutlu özet bir yapı kullandık. Şimdi bu yapıyı, her yinelemede evreni kare kökü boyutuna düşürecek şekilde özyinelemeli hale getiriyoruz. Boyutu u olan bir evrenden başlayarak, önce $\sqrt{u} = u^{1/2}$ öge saklayan bir yapı oluşturuyoruz; bu ögelerin kendileri de $u^{1/4}$ adet öge saklıyor ve yine bu ögeler de $u^{1/8}$ adet öge saklıyor. Bu yapı, taban olan 2 ye ulaşıncaya kadar devam ediyor.

Kolaylık sağlaması adına bu kısımda, $u, u^{1/2}, u^{1/4}, \dots$ sayılarının birer tam sayı olması için, k bir tam sayı olmak üzere $u = 2^{2^k}$ olduğunu kabul ediyoruz. Pratikte bu kısıtlama bir hayli sert olur, u değerlerinin 2, 4, 16, 256, 65536, dizisinden seçilmesini gerektirirdi. Bir sonraki kısımda bu kabulu genişleterek, k bir tam sayı olmak üzere $u = 2^k$ kabulüne nasıl dönüştüreceğimizi göreceğiz. Bu kısımda incelediğimiz yapı, van Emde Boas Ağacı yapısının yalnızca bir öncüsü olduğu ve konuyu kavramamıza yardımcı olduğundan bu kısıtlamayı hoşgöreceğiz.

Hedefimizin, işlemleri $O(\lg \lg u)$ zamanda gerçekleştirmek olduğunu hatırlayarak, böylesi çalışma zamanlarını nasıl elde edebileceğimiz üzerine kafa yoralım. Kısım 4.3 ün sonunda, değişkenleri değiştirerek

$$T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n \quad (20.1)$$

özyinelemesinin $T(n) = O(\lg n \lg \lg n)$ çözümü olduğunu gördük. Benzer, ancak daha basit bir özyinelemeyi ele alalım:

$$T(u) = T(\sqrt{u}) + O(1) . \quad (20.2)$$

Aynı tekniği kullanarak, değişkenleri değiştirerek, (20.2) özyinelemesinin $T(u) = O(\lg \lg u)$ çözümü olduğunu gösterebiliriz. $m = \lg u$ olsun. O zaman elimizde $u = 2^m$ ve dolayısıyla

$$T(2^m) = T(2^{m/2}) + O(1)$$

olur. Şimdi $S(m) = T(2^m)$ diyerek yeni bir özyineleme veriyoruz:

$$S(m) = S(m/2) + O(1).$$

Ana yöntemin 2. durumundan, bu özyinelemenin $S(m) = O(\lg m)$ çözümü vardır. $S(m)$ den $T(u)$ ya dönerek $T(u) = T(2^m) = S(m) = O(\lg m) = O(\lg \lg u)$ olduğunu gösteririz.

(20.2) özyinelemesi, veri yapısı arayışımızda bize yol gösterecek. Her özyineleme seviyesinde $\sqrt{u}$ kat küçülen, özyinelemeli bir veri yapısı tasarlayacağız. Bir işlem bu yapıyı dolaşırken, her bir seviyede, bir aşağıdaki seviyeye geçmeden önce sabit bir zaman harcayacak. (20.2) özyinelemesi de işlemin çalışma zamanını belirleyecek.

İşte $\lg \lg u$ ifadesinin nasıl (20.2) özyinelemesinin çözümü olup çıkacağını başka bir izahıdır. Özyinelemeli veri yapısının her seviyesinde evrenin boyutuna baktığımızda $u, u^{1/2}, u^{1/4}, u^{1/8}, \dots$ dizisini görüyoruz. Her seviyedeki evreni saklamak için kaç bite ihtiyaç duyduğumuzu düşünelim; en üst seviyede $\lg u$ bite ve sonraki seviyelerin her birinde de bir önceki seviyedekinin yarısı kadar bite ihtiyaç duyarız. Genel olarak, b adet bite başlar ve bit sayısını her seviyede yarılar, $\lg b$ seviye sonra da tek bir bite kadar inmiş oluruz. $b = \lg u$ olduğundan, $\lg \lg u$ seviye sonra boyutu 2 olan bir evrenimiz kaldığını görürüz.

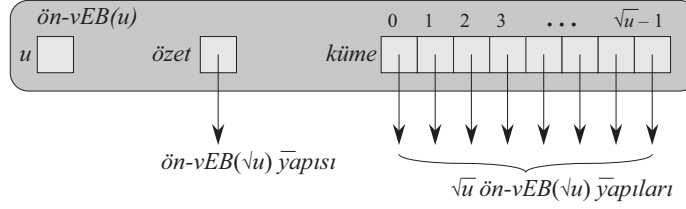
Şekil 20.2 deki veri yapısına tekrar baktığımızda, küme numarası $\lfloor x / \sqrt{u} \rfloor$ ifadesinde verilen bir x değerinin yer aldığını görüyoruz. Eğer x değerini $\lg u$ -bitlik bir ikili tam sayı olarak görürsek, küme numarası $\lfloor x / \sqrt{u} \rfloor$ ifadesi en anlamlı $(\lg u)/2$ bit tarafından belirlenir. x değeri, bulunduğu küme içinde, kendisinin en az anlamlı $(\lg u)/2$ bitiyle belirlenen $x \bmod \sqrt{u}$ pozisyonunda yer alır. Bu yolla bir indisleme yapmamız gerekecek. Bunun için de bazı fonksiyonlar tanımlayalım:

$$\text{high}(x) = x \sqrt{u} + y.$$

$$\text{low}(x) = x \bmod \sqrt{u},$$

$$\text{index}(x, y) = x \sqrt{u} + y.$$

$\text{high}(x)$ fonksiyonu, x in en anlamlı $(\lg u)/2$ bitini vererek x in kümesinin numarasını üretir. $\text{low}(x)$ fonksiyonu, x in en az anlamlı $(\lg u)/2$ bitini vererek x in bulunduğu küme içindeki pozisyonunu temin eder. $\text{index}(x, y)$ fonksiyonu da x ve y değerlerinden bir eleman numarası üretir. Bunun için x değerini elemanın en anlamlı $(\lg u)/2$ biti, y değerini de en az anlamlı $(\lg u)/2$ biti olarak kullanır. Şu



Şekil 20.3 $u \geq 4$ olduğunda bir $\text{ön-vEB}(u)$ yapısında bulunan bilgi. Yapı, evren boyutu u yu, bir $\text{ön-vEB}(\sqrt{u})$ yapısına işaret eden *özet* işaretçisini ve $\text{ön-vEB}(\sqrt{u})$ yapılarına işaret eden $\sqrt{u}$ adet işaretçinin bulunduğu bir *grup* $[0 \dots \sqrt{u} - 1]$ dizisini içerir.

eşitliğe sahibiz: $x = \text{index}(\text{high}(x), \text{low}(x))$. Bu fonksiyonların her biri tarafından kullanılan u değeri her zaman, fonksiyonu çağırdığımızdaki veri yapısının özyinelemeli yapıdan aşağı inildikçe değişen evren boyutu olacak.

20.2.1 Ön van Emde Boas yapıları

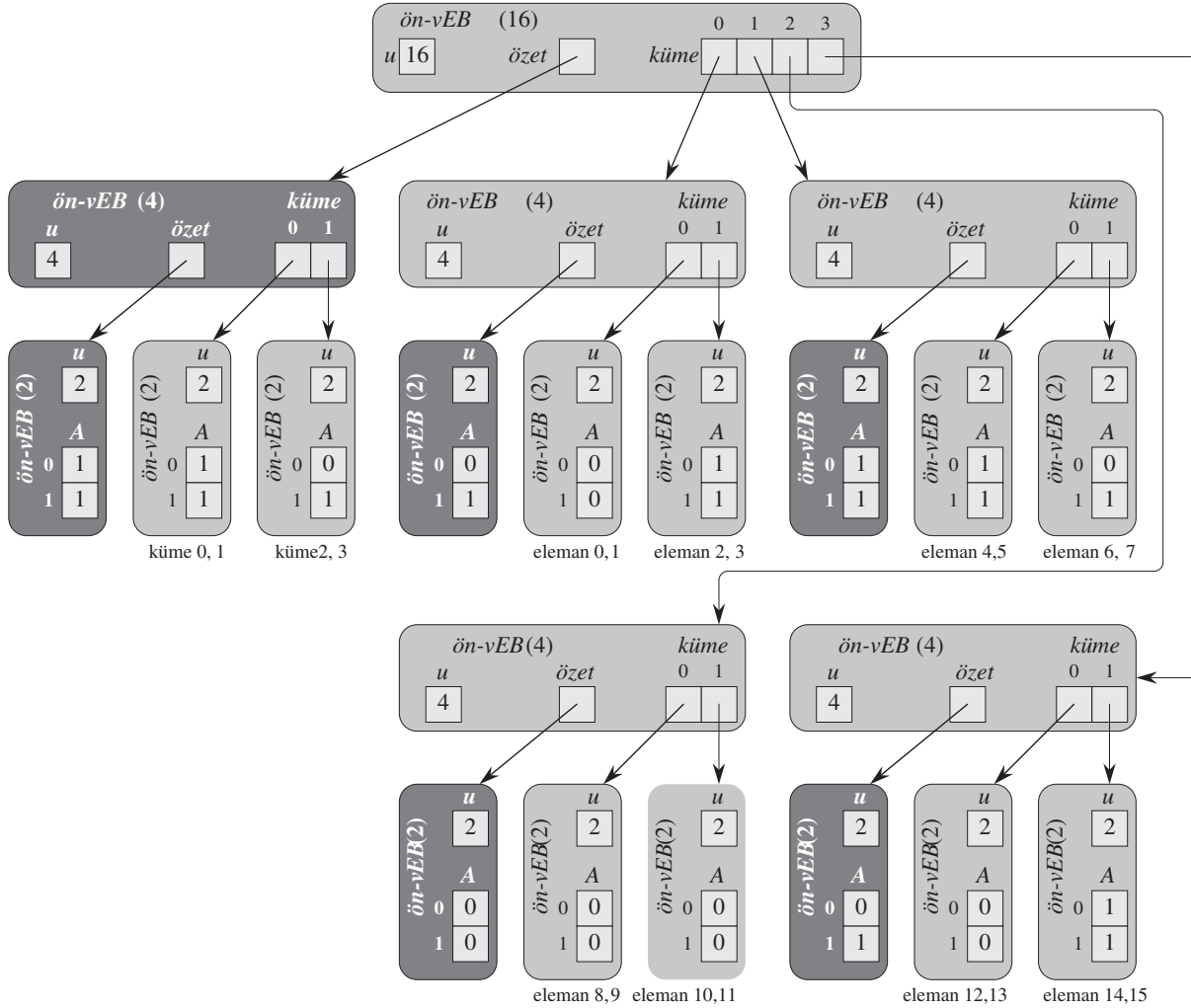
(20.2) özyinelemesinden ipucu alarak, istediğimiz işlemleri destekleyen, özyinelemeli bir veri yapısı tasarlayalım. Bu veri yapısı, hedefimiz olan $O(\lg \lg u)$ $\text{ön-vEB}(\sqrt{u})$ zamanı bazı işlemler için sağlayamayacak olsa da, Kısım 20.3 te göreceğimiz van Emde Boas ağaçları için bir temel oluşturacak.

$\{0, 1, 2, \dots, u - 1\}$ evreni için, $\text{ön-vEB}(u)$ ile gösterdiğimiz, özyinelemeli bir **ön van Emde Boas yapısı** veya **ön-vEB** yapısı tanımlıyoruz: Her $\text{ön-vEB}(\sqrt{u})$ yapısı, evreninin boyutunu veren bir u özelliğine sahiptir. Ek olarak şunları da içerir:

- Eğer $u = 2$ ise, bu taban boyutudur ve iki bitten oluşan bir $A[0..1]$ dizisi içerir.
- Aksi halde, $k \geq 1$ bir tam sayı olmak üzere, $u \geq 4$ olacak şekilde $u = 2^{2^k}$ olur. $\text{ön-vEB}(u)$ yapısı, evren boyutuna ek olarak Şekil 20.3 te resmedildiği gibi, şunları da içerir:
 - Bir $\text{ön-vEB}(u)$ yapısına işaret eden *özet* isimli bir işaretçi ve
 - $\sqrt{u}$ adet işaretçiden oluşan bir Her biri bir $\text{ön-vEB}(\sqrt{u})$ yapısına işaret eden $[0 \dots \sqrt{u} - 1]$ dizi kümesi.

$0 \leq x < u$ için x elemanı, $\text{high}(x)$ numaralı kümenin $\text{low}(x)$ elemanı olarak özyinelemeli biçimde saklanır.

Önceki kısımdaki iki-seviyeli yapıda, her düğümde her bir ögesinin bir bit sakladığı $\sqrt{u}$ boyutunda bir *özet* dizisi tutuluyor. Her ögenin indisinden, bitin özetlediği $\sqrt{u}$ boyutlu altdizinin başlangıç indisini hesaplayabiliriz. Ön-vEB yapısında, indis hesaplamaları yerine doğrudan işaretçiler kullanıyoruz. Kullanılan



Şekil 20.4 $\{2, 3, 4, 5, 7, 14, 15\}$ kümesini temsil eden bir $\text{ön-vEB}(4)$ yapısı. Yapıda, $\text{küme} [0..3]$ dizisindeki dört $\text{ön-vEB}(2)$ yapısına ve kendisi de bir $\text{ön-vEB}(4)$ yapısı olan bir özet yapıya işaret ediliyor. Her bir $\text{ön-vEB}(2)$ yapısında da, $[0..1]$ dizisindeki iki $\text{ön-vEB}(2)$ yapısına ve bir $\text{ön-vEB}(2)$ özetine işaret ediliyor. Her $\text{ön-vEB}(2)$ yapısı, yalnızca, iki bitlik bir $A[0..1]$ dizisini içeriyor. “eleman i, j ”nin üstündeki $\text{ön-vEB}(2)$ yapıları asıl dinamik kümenin i ve j bitlerini, ve “küme i, j ”nin üstündeki $\text{ön-vEB}(2)$ yapıları da en üst seviyedeki $\text{ön-vEB}(16)$ yapısındaki i ve j kümeleri için özet bitlerini saklıyorlar. Kolay anlaşılması adına, koyu gölgelemeler kendi ebeveyn yapısı hakkında özet bilgi saklayan bir ön-vEB yapısının üst seviyesini belirtmektedir; bunun dışında böylesi bir proto-vEB yapısının aynı evren boyutuna sahip herhangi bir başka ön-vEB yapısından farkı yoktur.

özet dizisi, bir ön-vEB yapısında özyinelemeli olarak saklanan özet bitlerini, ve küme dizisi $\sqrt{u}$ adet işaretçiyi içeriyor.

Şekil 20.4, $\{2, 3, 4, 5, 7, 14, 15\}$ kümesini temsil eden, tamamıyla açılmış bir ön-vEB(16) yapısını gösteriyor. Eğer i değeri özet tarafından işaret edilen ön-vEB yapısının içerisindeyse, i . küme temsil edilen kümeden bir değer saklıyor demektir. Sabit boylu ağaçta olduğu gibi, $küme[i]$ i . kümesi oluşturacak şekilde $i\sqrt{u}$ dan $(i + 1)\sqrt{u} - 1$ e kadar olan değerleri temsil eder.

Taban seviyesinde, asıl dinamik kümenin elemanları bazı ön-vEB(2) yapılarında saklanır, ve geri kalan ön-vEB(2) yapıları da özet bitlerini saklar. Özet olmayan taban yapılarının altında, o yapılarda hangi bitlerin saklandığı gösteriliyor. Örneğin, “eleman 6,7” ile etiketlenmiş ön-vEB(2) yapısı, $A[0]$ da 6. bitini (0, çünkü 6 dinamik kümenin elemanı değil) ve $A[1]$ de 7. bitini (1, çünkü 7 dinamik kümenin elemanı) saklıyor.

Kümeler gibi her özet de $\sqrt{u}$ evren boyutlu bir dinamik kümedir ve bu yüzden her özeti bir ön-vEB($\sqrt{u}$) yapısı şeklinde temsil ediyoruz. Ana ön-vEB(16) yapısının dört özet biti en soldaki ön-vEB(4) yapısındadır ve sonunda iki ön-vEB(2) yapısında yer alırlar. Örneğin, “küme 2,3” ile etiketlenmiş ön-vEB(2) yapısında, ön-vEB(16) yapısının 2. grubunda (8, 9, 10, 11 elemanlarını içeren) yalnızca 0 olduğunu belirten $A[0] = 0$, ve 3. kümesinde (12, 13, 14, 15 elemanlarını içeren) en az bir adet 1 olduğunu belirten $A[1] = 1$ vardır. Her ön-vEB(4) yapısı kendi özetine işaret eder, ki o da bir ön-vEB(2) yapısı şeklinde temsil edilir. Örneğin, “eleman 0,1” ile etiketlenenin solundaki ön-vEB(2) yapısına bakın. Bize $A[0]$ ı 0 olduğundan, “eleman 0,1” yapısının tamamıyla 0 olduğunu ve $A[1]$ i 1 olduğundan, “eleman 2,3” yapısında en az bir adet 1 olduğunu söylüyor.

20.2.2 Ön van Emde Boas yapısında işlemler

Şimdi bir ön-vEB yapısında işlemlerin nasıl gerçekleştirileceğini tarif edeceğiz. Öncelikle ön-vEB yapısını değiştirmeyen sorgu işlemlerini —MEMBER, MINIMUM, MAXIMUM ve SUCCESSOR— inceliyoruz. Daha sonra INSERT ve DELETE i ele alıyoruz. MAXIMUM ve PREDECESSOR işlemlerini sırasıyla, MINIMUM ve SUCCESSOR işlemlerinin simetrikleri oldukları için Alıştırma 20.2-1 e bırakıyoruz.

MEMBER, SUCCESSOR, PREDECESSOR, INSERT ve DELETE işlemlerinin hepsi girdi olarak bir ön-vEB yapısı olan V ile beraber bir x parametresi alıyor. Tüm işlemlerde $0 \leq x < Vu$ olduğu kabul ediliyor.

Bir değer kümede olup olmadığını belirleme

MEMBER (x) işlemini gerçekleştirmek için, uygun ön-vEB(2) yapısında x e denk gelen biti bulmamız gerekir. Bunu, özet yapılarını toptan atlayarak $O(\lg \lg u)$

zamanda yapabiliriz. Aşağıdaki prosedür bir V ön-vEB yapısını ve bir x değerini alıyor ve V nin sakladığı dinamik kümede x in olup olmadığını belirten bir bit döndürüyor.

PROTO-vEB-MEMBER(V, x) z

- 1 **if** $V.u = 2$
- 2 **return** $V.A[x]$
- 3 **else return** PROTO-vEB-MEMBER($V.küme[high(x)], low(x)$)

PROTO-vEB-MEMBER prosedürü şöyle çalışıyor. Satır 1, V nin bir ön-vEB(2) yapısı olduğu taban durumunda olup olmadığını test ediyor. Satır 2, A dizisinin ilgili bitini döndürerek taban olma durumunu hallediyor. Satır 3, uygun küçük ön-vEB yapısına doğru “inerek” özyinelemeli durumla ilgileniyor. $high(x)$ değeri hangi ön-vEB($\sqrt{u}$) yapısına uğradığımızı söylüyor ve $low(x)$ de o ön-vEB($\sqrt{u}$) yapısının hangi elemanını sorguladığımızı belirliyor.

Şekil 20.4 teki ön-vEB(16) yapısında PROTO-vEB-MEMBER($V, 6$) prosedürünü çağırdığımız zaman neler olduğuna bir göz atalım. $u = 16$ için $low(6) = 1$ olduğundan, sağ üstteki ön-vEB(4) yapısına iniyoruz ve o yapının $low(6) = 2$ numaralı elemanını sorguluyoruz. Bu özyinelemeli çağrıda $u = 4$ olur ve bir kez daha çağrı yapmamız gerekir. $u = 4$ için $high(2) = 1$ ve $high(2) = 0$ olduğundan, sağ üstteki ön-vEB(2) yapısının 0. elemanını sorguluyoruz. Bu özyinelemeli çağrı bir taban olma durumudur, ve dolayısıyla özyineleme zincirinden geriye doğru $A[0] = 0$ değerini döndürür. Böylece, PROTO-vEB-MEMBER($V, 6$) prosedürünün, 6 nın kümede yer almadığını belirtmek üzere 0 döndürdüğünü görmüş oluruz.

PROTO-vEB-MEMBER nin çalışma zamanını belirlemek üzere, $T(u)$ bir ön-vEB(u) yapısının çalışma zamanını ifade etsin. Her özyinelemeli çağrı, çağırdığı özyinelemenin zamanı dışında sabit zamanda çalışır. PROTO-vEB-MEMBER özyinelemeli bir çağrı yaptığında, ön-vEB($\sqrt{u}$) yapısına bir çağrı yapar. Böylece çalışma zamanını, özyineleme (20.2) olarak gördüğümüz $T(u) = T(\sqrt{u}) + O(1)$ ile ifade edebiliriz. Çözümü $T(u) = O(\lg \lg u)$ olduğundan, PROTO-vEB-MEMBER işleminin $O(\lg \lg u)$ zamanda çalıştığı sonucuna varırız.

En küçük elemanı bulma

Şimdi MINIMUM işleminin nasıl yapılacağını inceleyelim. PROTO-vEB-MINIMUM(V) prosedürü, V ön-vEB yapısının en küçük elemanını veya V boş ise LD (Listede Değil) döndürür.

```

PROTO-VEB-MINIMUM( $V$ )
1  if  $V.u == 2$ 
2      if  $V.A[0] == 1$ 
3          return 0
4      elseif  $V.A[1] == 1$ 
5          return 1
6      else return LD
7  else  $min\_cluster = \text{PROTO-VEB-MINIMUM}(V.summary)$ 
8      if  $min\_cluster == LD$ 
9          return LD
10     else  $konum = \text{PROTO-VEB-MINIMUM}(V.cluster[min\_cluster])$ 
11     return index ( $min\_cluster, offset$ )

```

Bu prosedür şöyle çalışıyor. Satır 1 taban durumunu kontrol ediyor ve Satır 2-6 bu durumu kaba kuvvet (brute force) ile hallediyor. Satır 7-11 özyinelemeli durumu ele alıyor. Öncelikle Satır 7, kümenin bir elemanını içeren ilk kümenin numarasını buluyor. Bunu, bir $\text{ön-veB}(\sqrt{u})$ yapısı olan $V.summary$ üzerinde PROTO-VEB-MINIMUM özyinelemeli çağrısı yaparak gerçekleştiriyor. Satır 7 bu küme numarasını $min_küme$ değişkenine atıyor. Eğer küme boş ise, özyinelemeli çağrı ve de Satır 9 LD döndürür. Aksi halde, kümenin en küçük elemanı $min_küme$ numaralı grubun içinde bir yerlerdedir. Satır 10 daki özyinelemeli çağrı en küçük elemanın, bulunduğu kümedeki konumunu buluyor. Son olarak Satır 11, küme numarasını ve konumu kullanarak en küçük elemanın değerini döndürüyor.

Her ne kadar özet bilgiyi sorgulamak en küçük elemanın bulunduğu kümesi hızla bulmamızı sağlasa da, prosedür $\text{ön-veB}(\sqrt{u})$ üzerinde iki kez özyinelemeli çağrı yaptığı için, en kötü durumda $O(\lg \lg u)$ zamanda çalışmaz. $T(u)$ bir $\text{ön-veB}(u)$ yapısı üzerinde PROTO-VEB-MINIMUM prosedürünün en kötü çalışma zamanını ifade etmek üzere, elimizde aşağıdaki özyineleme var:

$$T(u) = 2T(\sqrt{u}) + O(1) . \quad (20.3)$$

Bu özyinelemeyi çözmek için yine $m = \lg u$ olacak şekilde değişken değiştirmeyi kullanarak şu çözümü elde ederiz:

$$T(2^m) = 2T(2^{m/2}) + O(1)$$

$$S(m) = T(2^m) \text{ diyerek elde edeceğimiz}$$

$$S(m) = 2S(m/2) + O(1)$$

ana yöntemin 1. durumundan $S(m) = \Theta(m)$ çözümüne sahiptir. $S(m)$ den $T(u)$ ya geri döndüğümüzde $T(u) = T(2^m) = S(m) = \Theta(m) = \Theta(\lg u)$ elde ederiz. Böylece, ikinci özyinelemeli çağrıdan ötürü, PROTO-VEB-MINIMUM, $O(\lg \lg u)$ yerine $\Theta(\lg u)$ zamanda çalışır.

Ardıl bulma

SUCCESSOR işlemi daha da kötüdür. En kötü durumda, iki özyinelemeli çağrının yanında PROTO-VEB-MINIMUM çağrısı da yapar. PROTO-VEB-SUCCESSOR(V, x) prosedürü, V ön-veb yapısındaki x ten büyük en küçük elemanı veya böyle bir eleman yoksa LD döndürür. x kümenin elemanı olmak zorunda değildir, ancak $0 \leq x < V.u$ olduğu kabul edilmektedir.

```

PROTO-VEB-SUCCESSOR ( $V.x$ )
1  if  $V.u == 2$ 
2      if  $x == 0$  ve  $V.A[1] == 1$ 
3      return 1
4      else return LD
5  else  $offset = \text{PROTO-VEB-SUCCESSOR}(V.cluster[high(x)], low(x))$ 
6      if  $offset \neq \text{LD}$ 
7          return index ( $high(x), konum$ )
8      else  $succ-cluster = \text{PROTO-VEB-SUCCESSOR}(V.cluster, high(x))$ 
9      if  $succ-cluster == \text{LD}$ 
10         return LD
11     else  $offset = \text{PROTO-VEB-MINIMUM}(V.cluster, [succ-cluster])$ 
12     return index ( $succ-cluster, offset$ )

```

PROTO-VEB-SUCCESSOR prosedürü şöyle çalışıyor. Alışıldığı gibi, Satır 1 taban durumunu kontrol ediyor ve Satır 2-4 bu durumu kaba kuvvet ile hallediyor: x in ardılının bir ön-veb(2) yapısında olmasının tek yolu $x = 0$ ve $A[1]$ in 1 olmasıdır. Satır 5-12 özyinelemeli durumla ilgileniyor. Satır 5 x in grubu içerisinde ardılını arayıp sonucu $offset$ e atıyor. Satır 6 x in kendi kümesi içerisinde bir ardılı olup olmadığına bakıyor, eğer varsa, o zaman Satır 7 ardılı hesaplayıp döndürüyor. Aksi halde, diğer kümeleri de araştırmamız gerekir. Satır 8 özet bilgisini kullanarak sonraki boş olmayan kümesi numarasını bulup $succ-cluster$ e atıyor. Satır 9 $succ-cluster$ in LD olup olmadığını kontrol ederek takip eden tüm kümeler boş ise Satır 10 ile LD döndürüyor. Eğer $succ-cluster$ LD değilse, Satır 11 o kümedeki ilk elemanı $offset$ e atıyor ve Satır 12 o kümedeki en küçük elemanı hesaplayıp döndürüyor.

En kötü durumda PROTO-VEB-SUCCESSOR ön-veb($\sqrt{u}$) üzerinde iki kez özyinelemeli çağrı yapar ve onlar da ön-veb($\sqrt{u}$) üzerinde bir kez PROTO-VEB-MINIMUM'u çağırır. Dolayısıyla, PROTO-VEB-SUCCESSOR un en kötü durumda çalışma zamanı olan $T(u)$ için özyineleme şöyledir:

$$\begin{aligned}
 T(u) &= 2T(\sqrt{u}) + \Theta(\lg \sqrt{u}) \\
 &= 2T(\sqrt{u}) + \Theta(\lg u) .
 \end{aligned}$$

Bu özyinelemenin $T(u) = \Theta(\lg u \lg \lg u)$ çözümü olduğunu göstermek için özyineleme (20.1) de kullandığımız tekniğin aynısını kullanabiliriz. Yani, PROTO-VEB-SUCCESSOR prosedürü PROTO-VEB-MINIMUM dan asimptotik olarak daha yavaştır.

Eleman ekleme

Bir elemanı eklemek için, onu uygun gruba eklememiz ve o grubun özet bitini de 1 yapmamız gerekir. PROTO-VEB-INSERT (V, x) prosedürü, V PROTO-VEB yapısına x değerini ekler.

PROTO-VEB-SUCCESSOR(V, x)

```

1  if  $V.u = 2$ 
2       $V.A[x] = 1$ 
3  else PROTO-VEB-INSERT( $(V.cluster[high(x)], low(x))$ )
4      PROTO-VEB-INSERT( $(V.summary, high(x))$ )
```

Taban durumunda, Satır 2 A dizisindeki ilgili biti 1 yapar. Özyinelemeli durumda, Satır 3 teki özyinelemeli çağrı x i uygun kümeye ekler ve Satır 4 o küme için tutulan özet bitini 1 yapar.

PROTO-VEB-INSERT en kötü durumda iki özyinelemeli çağrı yaptığından, (20.3) özyinelemesi çalışma zamanını belirler. Bu nedenle, PROTO-VEB-INSERT $\Theta(\lg u)$ zamanda çalışır.

Eleman silme

DELETE işlemi eklemekten daha karışıktır. Ekleme yaparken özet bitini her zaman 1 yapıyorken, silme işleminde her zaman o özet bitini 0 yapamayız. İlgili gruptaki herhangi bir bitin 1 olup olmadığını da belirlememiz gerekir. Bunun için, ön-vEB yapılarını tanımladığımız haliyle, bir kümenin içerisindeki tüm $\sqrt{u}$ adet bitin içinde herhangi birinin 1 olup olmadığını sınılamamız gerekirdi. Alternatif olarak, ön-vEB yapısına, kaç adet elemanı olduğunu saklayan bir n özelliği de ekleyebiliriz. PROTO-VEB-DELETE in uygulanmasını Alıştırma 20.2-2 ve 20.2-3 e bırakıyoruz.

Belli ki ön-vEB yapısını, her işlemde en fazla bir kez özyinelemeli çağrı yapılacak şekilde düzenlememiz gerekiyor. Bir sonraki kısımda bunu nasıl yapacağımızı göreceğiz.

Alıştırmalar

20.2-1

PROTO-VEB-MAXIMUM ve PROTO-VEB-PREDECESSOR prosedürleri için kaba-kod yazın.

20.2-2

PROTO-VEB-DELETE için kabakod yazın. Grup içindeki ilgili bitleri tarayarak uygun özet bitini değiştirmelidir. Prosedürünüzün en kötü durumda çalışma zamanı nedir?

20.2-3

Her bir ön-vEB yapısına temsil ettiği kümenin eleman sayısını veren n özelliğini ekleyin, ve özet bitini ne zaman 0 yapacağını n özelliğini kullanarak belirleyen PROTO-VEB-DELETE işleminin kabakodunu yazın. Prosedürünüzün en kötü durumda çalışma zamanı nedir? Bu yeni özellik sebebiyle başka hangi prosedürleri değiştirmek gerekir? Bu değişimler çalışma zamanlarını etkiler mi?

20.2-4

ön-vEB yapılarını eş anahtarları destekleyecek şekilde düzenleyin.

20.2-5

ön-vEB yapılarını uydu verilere sahip anahtarları destekleyecek şekilde düzenleyin.

20.2-6

Bir ön-vEB(u) yapısı yaratan bir prosedür için kabakod yazın.

20.2-7

PROTO-VEB-MINIMUM un Satır 9 u çalıştırıldığında, ön-vEB yapısının boş olması durumunu tartışın.

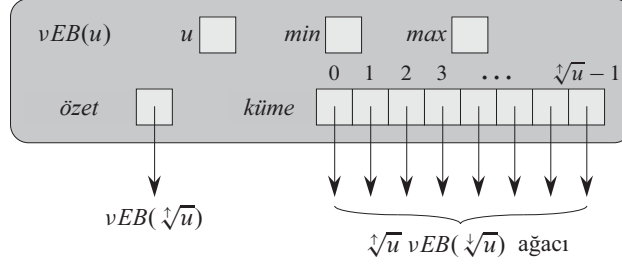
20.2-8

Tüm *küme* dizilerinin yalnızca $u^{1/4}$ eleman içerdiği bir ön-vEB yapısı tasarladığımızı varsayalım. Her bir işlemin çalışma zamanı ne olurdu?

20.3 van Emde Boas ağacı

Önceki kısımdaki ön-vEB yapısı ulaşmamız gereken $O(\lg \lg u)$ çalışma zamanına yakındı. Yetersiz kaldı, çünkü birçok işlemde çok fazla özyineleme yapmamız gerekiyordu. Bu kısımda, ön-vEB yapısına benzer, ancak biraz daha fazla bilgi saklayarak bu özyinelemelerden kurtulmamızı sağlayacak bir veri yapısı tasarlayacağız.

Kısım 20.2 de, evren boyutu hakkında yaptığımız kabulün — k bir tam sayı olmak üzere $u = 2^{2^k}$ olması— aşırı derecede kısıtlayıcı olduğunu, mümkün u değerlerini oldukça seyrek bir uzaya hapsettiğini gözlemledik. Bu noktadan itibaren, evren boyutu u nun 2 nin herhangi bir tam kuvveti olmasına izin verecek, ve $\sqrt{u}$ bir tam sayı olmadığında —yani, u 2 nin bir tek kuvveti ise ($k \geq 0$) bir



Şekil 20.5 $u > 2$ olduğunda bir $vEB(u)$ da mevcut olan bilgi. Yapı, evren boyutu olan u , min ve max elemanları, bir $vEB(\sqrt{u})$ ağacına işaret eden bir $özet$ işaretçisi ve $vEB(\sqrt{u})$ ağaçlarına işaret eden $vEB(\sqrt{u})$ adet işaretçiden oluşan bir $grup [0.. \sqrt{u}-1]$ dizisinden meydana geliyor.

tam sayı olmak üzere 2 ($u = 2^{2k+1}$)— bir sayının $\lg u$ bitini en anlamlı $\lceil (\lg u)/2 \rceil$ bitine ve en az anlamlı $\lfloor (\lg u)/2 \rfloor$ bitine ayıracağız. Kolaylık adına, $2^{\lceil (\lg u)/2 \rceil}$ yi (u nun “üst kare kökü”) $\sqrt{u}$ ile ve $2^{\lfloor (\lg u)/2 \rfloor}$ yi (u nun “alt kare kökü”) $\sqrt[2]{u}$ ile göstereceğiz. Böylece, $u = \sqrt{u} \cdot \sqrt[2]{u}$ ve u 2 nin bir çift kuvveti ise (k bir tam-sayı olmak üzere $u = 2^{2k}$) $\sqrt{u} = \sqrt[2]{u} = \sqrt{u}$ olur. Artık u ya 2 nin bir tek kuvveti olma iznini verdiğimizize göre, Kısım 20.2 deki faydalı fonksiyonlarımızı yeniden tanımlamalıyız:

$$\begin{aligned} \text{high}(x) &= \lfloor x / \sqrt{u} \rfloor, \\ \text{low}(x) &= x \bmod \sqrt{u}, \\ \text{index}(x, y) &= x \sqrt{u} + y. \end{aligned}$$

20.3.1 van Emde Boas ağaçları

van Emde Boas ağacı veya **vEB ağacı**, ön-vEB yapısını modifiye eder. Evren boyutu u olan bir vEB ağacını $vEB(u)$ ile gösteriyoruz ve u taban boyutu olan 2 ye eşit değilse, $özet$ işaretçisi bir $vEB(\sqrt{u})$ ağacına ve $grup [0.. \sqrt{u}-1]$ dizisi $\sqrt{u}$ adet $vEB(\sqrt{u})$ ağacına işaret ediyor. Şekil 20.5 de görüldüğü üzere, bir vEB ağacı bir ön-vEB yapısında bulunmayan iki özellik içerir:

- min vEB ağacındaki en küçük elemanı saklar ve
- max vEB ağacındaki en büyük elemanı saklar.

Ayrıca, min de saklanan eleman, küme dizisinin işaret ettiği hiçbir özyinelemeli $vEB(\sqrt{u})$ ağacında yer almaz. Dolayısıyla, bir $V vEB(u)$ ağacında saklanan elemanlar, $V.min$ artı $V.cluster [0.. \sqrt{u}-1]$ tarafından işaret edilen $vEB(\sqrt{u})$ ağaçlarında özyinelemeli olarak saklanan tüm elemanlardır. Bir vEB ağacı iki veya daha fazla eleman içerdiğinde, min ve max a farklı davrandığımızı dikkat

edin: *min* in sakladığı eleman grupların hiçbirinde bulunmaz, ancak *max* ın sakladığı eleman gruplarda yer alır.

Taban boyutu 2 olduğundan, bir $vEB(2)$ yapısının *ön- $vEB(2)$* yapısındaki gibi A dizisine ihtiyacı yokur. Bunun yerine, yapının elemanlarını *min* ve *max* özelliklerinden elde edebiliriz. Elemanı olmayan bir vEB ağacında, u dan bağımsız olarak hem *min* hem de *max* özellikleri LD olur.

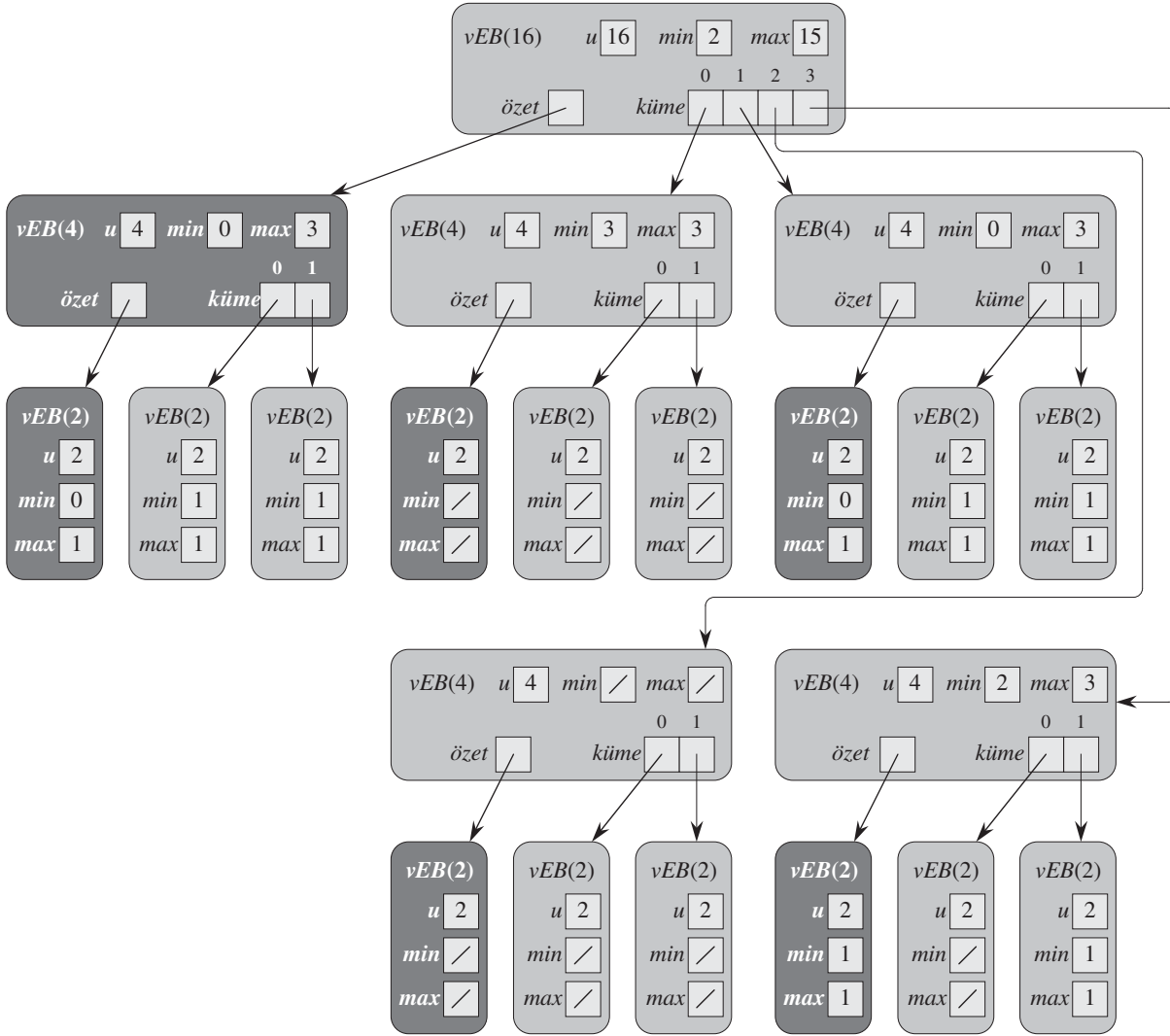
Şekil 20.6 $\{2, 3, 4, 7, 14, 15\}$ kümesini saklayan bir $vEB(16)$ ağacını gösteriyor. En küçük eleman 2 olduğu için, $V.min$ 2 ye eşittir ve $heigh(2) = 0$ olmasına rağmen, 2 elemanı $V.cluster[0]$ tarafından işaret edilen $vEB(2)$ ağacında görülmemektedir: $V.cluster[0].min$ in 3 e eşit olduğuna, ve 2 nin bu vEB ağacında yer almadığına dikkat edin. Benzer şekilde, $V.cluster[0].min$ 3 e eşit olduğu, ve 2 ve 3 $V.cluster[0]$ ın yegâne elemanları oldukları için, $V.cluster[0]$ içerisindeki $vEB(2)$ yapıları boştur.

vEB ağaçlarındaki özyinelemeli çağrı sayısının azaltılmasının anahtarı *min* ve *max* özellikleri olacak. Bu özellikler bize dört yönden yardım edecek:

1. MINIMUM ve MAXIMUM işlemleri özyinelemeye ihtiyaç dahi duymayacak, çünkü yalnızca *min* ve *max* değerlerini döndürecekler.
2. SUCCESSOR işlemi, x değerinin ardılının $high(x)$ içerisinde olup olmadığını belirlemek için özyinelemeli bir çağrı yapmaktan kurtulabilecek. Bunun nedeni, x in ardılının ancak ve ancak x , grubun *max* özelliğinden küçükse o grupta bulunabiliyor olmasıdır. Simetrik bir tez PREDECESSOR ve *min* için de geçerlidir.
3. Bir vEB ağacının hiç elemanı olmadığını, yalnızca bir elemanı olduğunu veya en az iki elemanı olduğunu *min* ve *max* değerlerine bakarak sabit zamanda söyleyebiliriz. Bu da bize INSERT ve DELETE işlemlerinde yardımcı olacak. Eğer hem *min* hem de *max* LD ise, o zaman vEB ağacının hiç elemanı yoktur. Eğer *min* ve *max* LD değil ancak birbirlerine eşitse, o zaman vEB ağacının yalnızca bir elemanı vardır. Aksi halde, hem *min* ve *max* LD değil, hem de eşit değillerse, o zaman vEB ağacının iki veya daha fazla elemanı vardır.
4. Eğer bir vEB ağacının boş olduğunu biliyorsak, yalnızca *min* ve *max* özelliklerini değiştirerek bir eleman ekleyebiliriz. Böylece, boş bir vEB ağacına sabit zamanda ekleme yapabiliriz. Benzer şekilde, vEB ağacının yalnızca bir elemanı olduğunu biliyorsak, bu elemanı yalnızca *min* ve *max* özelliklerini değiştirerek sabit zamanda silebiliriz. Bu özellikler özyineleme zincirini kısa kesmemize olanak sağlayacak.

Evren boyutu u 2 nin bir tek kuvveti olsa dahi, özet vEB ağacının ve grupların boyutları arasındaki fark vEB -ağacı işlemlerinin asimptotik çalışma zamanlarını etkilemeyecektir. vEB -ağacı işlemlerini uygulayan özyinelemeli prosedürlerin hepsi şu özyineleme tarafından belirlenen çalışma zamanlarına sahip olacaklardır

$$T(u) \leq T(\sqrt[4]{u}) + O(1) . \quad (20.4)$$



Şekil 20.6 Şekil 20.4 teki ön- vEB ağacına karşılık gelen bir $vEB(16)$ ağacı. $\{2, 3, 4, 5, 7, 14, 15\}$ kümesini saklıyor. Slash işaretleri LD değerlerini gösteriyor. Bir vEB ağacının min özelliğinde saklanan değer, gruplarının hiçbirinde bulunmuyor. Koyu gölgelemelerin anlamı burada da Şekil 20.4 teki ile aynı.

Bu özyineleme (20.2) özyinelemesine benziyor, ve bunu da benzer yolla çözeceğiz. $m = \lg u$ olmak üzere şu şekilde yeniden yazalım

$$T(2^m) \leq T(2^{\lceil m/2 \rceil}) + O(1).$$

Tüme $m \geq 2$ değerleri için $\lceil m/2 \rceil \leq 2m/3$ olduğunu dikkate aldığımızda

$$T(2^m) \leq T(2^{2m/3}) + O(1) \text{ olur.}$$

$S(m) = T(2^m)$ diyerek şu son özyinelemeyi elde ederiz

$$S(m) \leq S(2m/3) + O(1).$$

Bunun da, ana yöntemin 2. durumundan, $S(m) = O(\lg m)$ çözümü vardır. (Asimptotik çözüm açısından, $2/3$ kesirinin $1/2$ kesirinden farkı yoktur, çünkü ana yöntemi uyguladığımızda, $\log_{3/2} 1 = \log_2 1 = 0$ olduğunu görürüz.) Böylece, $T(u) = T(2^m) = S(m) = O(\lg m) = O(\lg \lg u)$ olur.

Bir van Emde Boas ağacını kullanmadan önce, en başta boş bir kümeyi temsil edecek uygun boyutta bir van Emde Boas ağacı yaratabilmek için, evren boyutu u yu bilmemiz gerekir. Problem 20-1 in sizden göstermesini istediği üzere, bir van Emde Boas ağacının toplam alan gereksinimi $O(u)$ dur ve boş bir ağacı $O(u)$ zamanda kolayca yaratılabilir. Öte yandan, boş bir kırmızı-siyah ağacı sabit zamanda yaratabiliriz. Bu yüzden, az sayıda işlem yapacağımızda, veri yapısını oluşturmada geçen zaman işlemlerden kazandığımız zamanı aşacağından, bir van Emde Boas ağacı kullanmak istemeyebiliriz. Bu dezavantaj genelde önemsizdir. Çünkü yalnızca birkaç elemanı olan bir kümeyi temsil etmek istediğimizde, tipik olarak dizi veya bağlantılı liste gibi basit veri yapılarını kullanırız.

20.3.2 van Emde Boas ağacında işlemler

Artık van Emde Boas ağacı üzerinde işlemler gerçekleştirmeye hazırız. Ön van Emde Boas ağaçları için yaptığımız gibi, önce sorgulama işlemlerini, ardından da INSERT ve DELETE işlemlerini ele alacağız. Bir vEB ağacının en küçük ve en büyük elemanları arasındaki küçük asimetri nedeniyle —vEB ağacında en az iki eleman olduğunda, en küçük eleman bir grup içinde yer almaz ancak en büyük eleman bir gruptadır— tüm beş sorgu işlemi için de kabakod vereceğiz. Aynı proto van Emde Boas yapılarındaki işlemlerde olduğu gibi, buradaki işlemler de, V bir van Emde Boas ağacı ve x de $0 \leq x \leq V.u$ olacak şekilde bir eleman olmak üzere, V ve x parametrelerini alıyor.

En büyük ve en küçük elemanları bulma

En büyük ve en küçük elemanları *min* ve *max* özelliklerinde sakladığımızdan, iki işlem de çok kısadır ve sabit zaman alırlar:

VEB-TREE-MINIMUM(V)

1 **return** $V.min$

VEB-TREE-MAXIMUM(V)

1 **return** $V.max$

Bir değerin kümede olup olmadığını belirleme

VEB-TREE-MEMBER(V, x) prosedüründe PROTO-VEB-MEMBER da olduğu gibi özyinelemeli bir durum söz konusudur, fakat taban durumu biraz farklıdır. x in doğrudan en küçük veya en büyük eleman olup olmadığını kontrol etmemiz gerekir. Bir vEB ağacında, bir ön-vEB yapısındaki gibi bitler saklanmadığı için, VEB-AĞACI-MEMBER 1 1 veya 0 yerine TRUE veya FALSE döndürecek şekilde tasarlıyoruz.

VEB-TREE-MEMBER(V)

1 **if** $x == V.min$ veya $x == V.max$

2 **return** TRUE

3 **elseif** $V.u == 2$

4 **return** FALSE

5 **else return** VEB-TREE-MEMBER($V.cluster[high(x)], low(x)$)

Satır 1 x in en küçük veya en büyük elemana eşit olup olmadığını kontrol ediyor. Eğer eşitse, Satır 2 TRUE döndürüyor. Aksi halde, Satır 3 taban durumunu test ediyor. Bir $vEB(2)$ ağacında min ve max dışında bir eleman olmadığından, eğer taban durumuysa, Satır 4 FALSE döndürüyor. Diğer olasılıkla da —taban durumu değil ve x ne min ne de max a eşitse— Satır 5 teki özyinelemeli çağrı ilgileniyor.

Özyineleme (20.4) VEB-TREE-MEMBER prosedürünün çalışma zamanını belirlediğinden, bu prosedür $O(\lg \lg u)$ zaman alır.

Ardıl ve öncül bulma

Sırada SUCCESSOR işleminin nasıl uygulanacağını var. PROTO-VEB-SUCCESSOR(V, x) prosedürünün iki özyinelemeli çağrı yapabileceğini hatırlayın. Biri x in ardılının x ile aynı grupta olup olmadığını belirlemek, diğeri de aynı grupta değilse, x in ardılını içeren grubu bulmak için. Artık bir vEB ağacının en büyük değerine çabucak ulaşabildiğimizden, iki özyinelemeli çağrı yapmaktan kaçınılabilir, ve bunun yerine ya yalnızca grup ya da yalnızca özet üzerinde bir çağrı yapabiliriz.

```

VEB-TREE-SUCCESSOR( $V, x$ )
1  if  $V.u == 2$ 
2      if  $x == 0$  ve  $V.max == 1$ 
3          return 1
4      else return LD
5  elseif  $V.max \neq \text{LD}$  ve  $x < V.min$ 
6      return  $V.min$ 
7  else  $max-low = \text{VEB-TREE-MAXIMUM}(V.cluster[high(x)])$ 
8      if  $max-low \neq \text{LD}$  ve  $low(x) < max-low$ 
9           $offset = \text{VEB-TREE-SUCCESSOR}(V.cluster[high(x)], low(x))$ 
10         return index ( $high(x), offset$ )
11     else  $succ-cluster = \text{VEB-TREE-SUCCESSOR}(V.summary, high(x))$ 
12         if  $succ-cluster == \text{LD}$ 
13             return LD
14         else  $offset = \text{VEB-TREE-MINIMUM}(V.cluster[succ-cluster])$ 
15         return index ( $succ-cluster, offset$ )

```

Bu prosedürde altı **return** ifadesi ve birkaç farklı durum bulunuyor. Satır 2-4 te taban durumuyla başlıyoruz. Eğer 0'ın ardılı arıyorsak ve 1 değeri 2 elemanlı kümenin içerisindeyse, Satır 3 te 1 döndürülüyor; aksi halde, Satır 4 te LD döndürülüyor.

Taban durumunda değilsek, x in en küçük elemandan küçük olup olmadığına bakıyoruz. Eğer öyleyse, en küçük elemanı Satır 6 da döndürüyoruz.

Satır 7 ye ulaştığımızda, taban durumunda olmadığımızı ve x in V vEB ağacındaki en küçük elemandan büyük veya ona eşit olduğunu biliyoruz demektir. Satır 7 $max-low$ değişkenine x in grubundaki en büyük elemanı atıyor. x in kümesi, x ten büyük bir eleman içeriyorsa, x in ardılının bu grubun içerisinde olduğunu da biliriz. Satır 8 bu durumu kontrol ediyor. Eğer x in ardılı, x in kümesindeyse, o zaman Satır 9 onun kümedeki yerini buluyor, ve Satır 10 da, PROTO-VEB-SUCCESSOR un Satır 7 siyle aynı şekilde ardılı döndürüyor.

x kendi kümesindeki en büyük elemandan daha büyük veya ona eşitse, Satır 11 e geliriz. Bu durumda, Satır 11-15 de, PROTO-VEB-SUCCESSOR'un Satır 8-12'siyle aynı şekilde x in ardılını buluyor.

Özyineleme (20.4) ün nasıl VEB-TREE-SUCCESSOR un çalışma zamanını belirlediğini görmek kolaydır. Satır 7 deki kontrolün sonucuna göre, prosedür ya Satır 9 da (evren boyutu $\sqrt[4]{u}$ olan bir vEB ağacı üzerinde) ya da Satır 11 de (evren boyutu $\sqrt[4]{u}$ olan bir vEB ağacı üzerinde) kendisini çağırıyor. İki durumda da özyinelemeli çağrı, evren boyutu en fazla $\sqrt[4]{u}$ olan bir vEB ağacı üzerinde yapılacaktır. Prosedürün geri kalanı, VEB-TREE-MINIMUM ve VEB-TREE-MAXIMUM çağrıları dahil, $O(1)$ zaman alır. Dolayısıyla, VEB-TREE-SUCCESSOR en kötü durumda $O(\lg \lg u)$ zamanda çalışır.

VEB-TREE-PREDECESSOR prosedürü vEB-TREE-SUCCESSOR prosedürüne simetrik, ancak ilave bir durumla:

```

VEB-TREE-SUCCESSOR( $V, x$ )
1  if  $V.u == 2$ 
2      if  $x == 0$  ve  $V.min == 1$ 
3          return 0
4      else return LD
5  elseif  $V.max \neq \text{LD}$  ve  $x > V.max$ 
6      return  $V.max$ 
7  else  $min-low = \text{VEB-TREE-MAXIMUM}(V.cluster[high(x)])$ 
8      if  $min-low \neq \text{LD}$  ve  $düşük(x) > min-low$ 
9           $offset = \text{VEB-TREE-PREDECESSOR}(V.cluster[high(x)], low(x))$ 
10         return  $\text{index}(high(x), offset)$ 
11     else  $pred-cluster = \text{VEB-TREE-PREDECESSOR}(V.summary, high(x))$ 
12         if  $pred-cluster = \text{LD}$ 
13             if  $V.min \neq \text{LD}$  ve  $x > V.min$ 
14                 return  $V.min$ 
15             else return LD
16     else  $offset = \text{VEB-TREE-MAXIMUM}(V.cluster[pred-cluster])$ 
17         return  $\text{index}(pred-cluster, offset)$ 

```

Satır 13-14 ilave durumu oluşturuyor. Bu durum, eğer varsa, x in ardılı x in grubunun dışında olduğunda ortaya çıkıyor. VEB-TREE-SUCCESSOR da, x in ardılının x in kümesinin dışında olduğuna emin olduğumuzda, daha yüksek numaralı bir kümenin içinde olması gerektiğini de biliriz. Fakat, x in öncülü V vEB ağacının en küçük değeri ise, o zaman ardıl hiçbir grupta yer almıyordur. Satır 13 bu durumu kontrol ediyor ve koşul sağlanıyorsa, Satır 14 en küçük değeri döndürüyor.

Bu ilave durum, VEB-TREE-PREDECESSOR un VEB-TREE-SUCCESSOR a göre asimptotik çalışma zamanını etkilemediğinden, VEB-TREE-PREDECESSOR en kötü durumda $O(\lg \lg u)$ zamanda çalışır.

Eleman ekleme

Şimdi bir vEB ağacına nasıl eleman ekleneceğini inceliyoruz. PROTO-VEB-INSERT'ün iki özyinelemeli çağrı yaptığını hatırlayın: biri elemanı eklemek, diğeri de o elemanın küme numarasını özete eklemek için. VEB-TREE-INSERT prosedürü ise yalnızca bir özyinelemeli çağrı yapacak. Tek bir çağrı nasıl yeterli olabilir? Bir elemanı eklerken, gideceği kümede ya başka bir eleman da vardır, ya da yoktur. Eğer kümede zaten başka bir eleman varsa, grup numarası önceden özete kaydedilmiş demektir ve bu yüzden bir özyinelemeli çağrı yapmamıza ge-

rek kalmaz. Eğer grupta başka eleman yoksa, eklenen eleman o kümenin yegâne elemanı olur, ve bizim de boş bir vEB ağacına eleman eklemek için özyineleme yapmamız gerekmez:

VEB-EMPTY-TREE-INSERT (V, x)

```
1   $V.min = x$ 
2   $V.max = x$ 
```

Bu prosedür cepte. Bu da x in V vEB ağacının temsil ettiği kümede olmadığını kabul eden VEB-TREE-INSERT (V, x) için kabakod:

VEB-TREE-INSERT (V, x)

```
1  if  $V.min == LD$ 
2      VEB-EMPTY-TREE-INSERT ( $V, x$ )
3  else if  $x < V.min$ 
4       $x$  i  $V.min$  ile değiştir
5      if  $V.u > 2$ 
6          if VEB-TREE-MINIMUM( $V.cluster[high(x)]$ ) == LD
7              VEB-TREE-INSERT( $V.summary[high(x)]$ )
8              VEB-EMPTY-TREE-INSERT( $V.cluster[high(x)], low(x)$ )
9          else VEB-TREE-INSERT( $V.cluster[high(x)], low(x)$ )
10 if  $x > V.max$ 
11      $V.max = x$ 
```

Bu prosedür şöyle çalışıyor. Satır 1 V nin boş bir vEB ağacı olup olmadığını kontrol ediyor, ve öyleyse, Satır 2 bu kolay durumla ilgileniyor. Satır 3-11 V nin boş olmadığını ve V nin kümelerinden birine bir eleman ekleneceğini kabul ediyor. Fakat bu eleman, VEB-TREE-INSERT e gönderilecek x elemanı olmayabilir. Satır 3 te kontrol edildiği gibi $x < min$ ise, o zaman x in yeni min olması gerekir. Ancak orijinal min i de kaybetmek istemediğimizden, onu V nin kümelerinden birine eklemeliyiz. Bu durumda, Satır 4 orijinal min i V nin kümelerinden birine ekleyebilmemiz için x i min ile değiştiriyor.

Satır 6-9 u yalnızca, V taban durumunda bir vEB ağacı değilse çalıştırıyoruz. Satır 6 x in gideceği grubun o anda boş olup olmadığını belirliyor. Öyleyse, Satır 7 x in grup numarasını özete ekliyor ve Satır 8 x i boş bir kümeye ekleme durumunu hallediyor. Eğer x in kümesi o an boş değilse, Satır 9 x i o kümeye ekliyor. Bu durumda, x in küme numarası zaten özetin bir üyesi olduğundan, özeti güncellememize gerek yoktur.

Son olarak, Satır 10-11 $x > max$ ise max değerini güncelliyor. Eğer V taban durumunda boş bir vEB ağacı ise, Satır 3-4 ve Satır 10-11 in min ve max ı uygun şekilde güncelleyeceğine dikkat edin.

Bir kez daha, özyineleme (20.4) ün çalışma zamanını belirlediğini kolayca görebiliriz. Satır 6 daki kontrolün sonucuna göre, ya Satır 7 deki (evren boyutu $\sqrt[4]{u}$ olan bir vEB ağacı üzerinde çalışan) ya da Satır 9 daki (evren boyutu $\sqrt[4]{u}$ olan bir vEB ağacı üzerinde çalışan) özyinelemeli çağrı yürütülüyor. İki durumda da özyinelemeli çağrı, evren boyutu en fazla $\sqrt[4]{u}$ olan bir vEB ağacı üzerinde yapılacaktır. VEB-AĞACI-EKLE nin geri kalanı $O(1)$ zaman aldığından, özyineleme (20.4) devreye girer ve çalışma zamanı $O(\lg \lg u)$ olur.

Eleman silme

Son olarak, bir vEB ağacından nasıl eleman silineceğine bakıyoruz. VEB-TREE-DELETE(V, x) prosedürü x in V vEB ağacı tarafından temsil edilen kümenin bir elemanı olduğunu kabul ediyor.

```

VEB-TREE-DELETE( $V, x$ )
1  if  $V.min == V.max$ 
2       $V.min = LD$ 
3       $V.max = LD$ 
4  elseif  $V.u == 2$ 
5      if  $x == 0$ 
6           $V.min = 1$ 
7      else  $V.min = 0$ 
8           $V.max == V.min$ 
9  else if  $x == V.min$ 
10      $first-cluster = VEB-TREE-MINIMUM(V.summary)$ 
11      $x = index(first-cluster,$ 
12          $VEB-TREE-MINIMUM(V.cluster[first-cluster]))$ 
13      $V.min = x$ 
14     VEB-TREE-DELETE( $V.cluster[high(x)], low(x)$ )
15     if VEB-TREE-MINIMUM( $V.cluster[high(x)]$ ) == LD
16         VEB-TREE-DELETE( $V.summary, high(x)$ )
17         if  $x == V.max$ 
18              $summary-max = VEB-TREE-MAXIMUM(V.summary)$ 
19             if  $summary-max = LD$ 
20                  $V.max = V.min$ 
21             else  $V.max = index(summary-max,$ 
22                  $VEB-TREE-MAXIMUM(V.cluster[summary-max]))$ 
23     elseif  $x == V.max$ 
24          $V.max = index(high(x)$ 
25              $VEB-TREE-MAXIMUM(V.cluster[high(x)]))$ 

```

VEB-TREE-DELETE prosedürü şöyle çalışıyor. Eğer vEB ağacı V de yalnızca bir eleman varsa, o elemanı silmek, boş bir vEB ağacına eleman eklemek kadar kolaydır: yalnızca $\min$ ve $\max$ a LD atanır. Satır 1-3 bu durumla ilgileniyor. Aksi halde, V nin en az iki elemanı vardır. Satır 4 V nin taban durumunda bir vEB olup olmadığını kontrol ediyor, ve öyleyse, Satır 5-8 $\min$ ve $\max$ a geriye kalan tek elemanı atıyor.

Satır 9-22 V nin iki veya daha fazla elemanı olduğunu ve $u \geq 4$ olduğunu kabul ediyor. Bu durumda, bir kümenin elemanlarından birini silmemiz gerekecek. Ancak gruptan sileceğimiz eleman x olmayabilir, çünkü x $\min$ e eşitse, x i sildiğimizde V nin kümelerinden birindeki başka bir eleman yeni $\min$ olur ve o elemanı da kümesinden silmemiz gerekir. Eğer Satır 9 daki kontrolde bu durumda olduğumuz sonucu çıkarsa, Satır 10 *first-cluster* e $\min$ den sonraki en küçük elemanın bulunduğu kümenin numarasını atar, ve Satır 11 de x e o kümenin en küçük elemanını atar. Bu eleman, Satır 12 de yeni $\min$ olarak atanır ve değerine x i atadığımızdan, gruptan silinecek olan da bu eleman olur.

Satır 13 e ulaştığımızda, x VEB-TREE-DELETE e yollanmış olan orijinal değer de olsa, yeni en küçük eleman olarak belirleniyor da olsa, artık x 'in grubundan silinmesi gerektiğini biliyoruz. Satır 13 x i grubundan siler. Bu grup artık boşalmış olabilir, ki Satır 14 bunu test eder ve öyleyse, x in grup numarasını özetten çıkarmamız gerekir. Bunu da Satır 15 hallediyor. Özeti güncelledikten sonra, $\max$ 'ı da güncellememiz gerekebilir. Satır 16 silinen elemanın V deki en büyük eleman olup olmadığını kontrol eder, ve öyleyse, Satır 17 *summary-max* a boş olmayan en yüksek numaralı grubun numarasını atar. (VEB-TREE-MAXIMUM (V .özet) çalışıyor, çünkü zaten V .summary üzerinde bir VEB-TREE-DELETE özyinelemeli çağrısı yapmıştık, ve bu yüzden de V .summary.max gerektiği gibi güncellenmişti.) Eğer V nin bütün kümeleri boşsa, V nin geriye kalan tek elemanı $\min$ dir; Satır 18 bu durumu inceliyor, ve Satır 19 sonuca göre $\max$ ı güncelliyor. Aksi durumda, Satır 20 en yüksek numaralı grubun en büyük elemanına atıyor. (Eğer bu küme elemanın silindiği kümeysen, yine Satır 13 teki özyinelemeli çağrının, kümenin $\max$ özelliğini düzelttiğini biliriz).

Son olarak, x in silinmesine karşın grubunun boşalmadığı durumu halletmemiz gerekiyor. Bu durumda özet güncellememiz gerekmesen de $\max$ ı güncellememiz gerekebilir. Satır 21 bu durumu test eder ve $\max$ ı güncellememiz gerekiyorsa, Satır 22 bunu yapar (yine özyinelemeli çağrının kümedeki $\max$ ı düzelttiğini bilerek).

Şimdi VEB-TREE-DELETE in en kötü durumda $O(\lg \lg u)$ zamanda çalıştığını gösterelim. İlk bakışta özyineleme (20.4) ün her zaman geçerli olmadığını düşünebilirsiniz. Çünkü tek bir VEB-TREE-DELETE çağrısı, iki özyinelemeli çağrı yapabilir: biri Satır 13, biri de Satır 15 te. Prosedürde bu iki çağrı yapılabilir olsa da, böyle bir durumda neler olacağını düşünelim. Satır 15 teki özyinelemeli çağrının gerçekleşmesi için, Satır 14 teki testin x in kümesinin boş olduğunu

göstermesi gerekir. x in kümesinin boş olmasının tek yolu Satır 13 teki çağrıyı yaptığımızda x in kümesindeki yegâne eleman olmasıdır. Fakat x grubundaki yegâne elemansa, bu özyinelemeli çağrı $O(1)$ zaman alır, çünkü yalnızca Satır 1-3 ü çalıştırır. Yani iki ayrık durum söz konusu:

- Satır 13 teki özyinelemeli çağrı sabit zaman alır,
- Satır 15 teki özyinelemeli çağrı gerçekleşmez.

Her iki durumda da, özyineleme (20.4) vEB-TREE-DELETE in çalışma zamanını belirler, ve bu da en kötü durumda $O(\lg \lg u)$ dur.

Alıştırmalar

20.3-1

vEB ağaçlarını, eş anahtarları destekleyecek şekilde düzenleyin.

20.3-2

vEB ağaçlarını, kendilerine bağlı uydu verileri olan anahtarları destekleyecek şekilde düzenleyin.

20.3-3

Boş bir van Emde Boas ağacı yaratan bir prosedür için kaba kod yazın.

20.3-4

vEB-TREE-DELETE prosedürünü zaten vEB ağacında yer alan bir elemanla çağırırsanız ne olur? vEB-TREE-DELETE prosedürünü vEB ağacında yer almayan bir elemanla çağırırsanız ne olur? Prosedürlerin bu davranışları niçin sergilediğini açıklayın. vEB ağacını ve işlemlerini, bir elemanın ağaçta olup olmadığını sabit zamanda kontrol edebileceğimiz şekilde düzenleyin.

20.3-5

Evren boyutu $\sqrt[k]{u}$ olan $\sqrt[k]{u}$ adet grup yerine, $k > 1$ bir sabit olmak üzere, evren boyutu $u^{1-1/k}$ olan $u^{1/k}$ adet kümesi olan vEB ağaçları oluşturduğumuzu düşünün. İşlemleri uygun şekilde düzenlersek, çalışma zamanları ne olurdu? Analizin amacına yönelik olarak, $u^{1/k}$ ve $u^{1-1/k}$ değerlerinin her zaman tam sayı olduğunu kabul edin.

20.3-6

Evren boyutu u olan bir vEB ağacını yaratmak $O(u)$ zaman gerektirir. Varsayalım ki doğrudan bu zamanı açıklamak istiyoruz. Bir vEB ağacındaki her bir işlemin amortize zamanının $O(\lg \lg u)$ olduğu en küçük n işlem sayısı kaçtır?

Problemler

20-1 van Emde Boas Ağaçları için alan gereksinimleri

Bu problem van Emde Boas ağaçları için alan gereksinimlerini inceler ve veri yapısında, alan gereksinimi evren boyutu u yerine, ağaçta saklanan elemanların sayısı olan n e bağlı olacak şekilde bir düzenleme önerir. Kolaylık adına, $\sqrt{u}$ nun her zaman bir tam sayı olduğunu kabul edelim.

- a. Evren boyutu u olan bir van Emde Boas ağacının alan gereksinimi $P(u)$ olmak üzere, şu özyinelemenin neden alan gereksinimini belirlediğini açıklayın:

$$P(u) = (\sqrt{u} + 1)P(\sqrt{u}) + \Theta(\sqrt{u}) \quad (20.5)$$

- b. Özyineleme (20.5) in $P(u) = O(u)$ çözümü olduğunu kanıtlayın.

Alan gereksinimlerini azaltmak için, bir **indirgenmiş-uzaylı van Emde Boas ağacı** veya **RS-vEB ağacı**, tanımlayalım. Bu ağaç, aşağıdaki değişikliklerin uygulandığı bir V vEB ağacı olsun:

- $V.cluster$ özelliği, evren boyutu $\sqrt{u}$ olan vEB ağaçlarına işaret eden işaretçilerden oluşan basit bir dizi yerine, dinamik bir tablo olarak (bkz. Kısım 17.4) oluşturulmuş bir çırpı tablosu (bkz. Bölüm 11) şeklinde saklanır. $V.cluster$ in dizi versiyonuna karşılık gelecek şekilde, çırpı tablosu, evren boyutu $\sqrt{u}$ olan RS-vEB ağaçlarına işaret eden işaretçileri saklar. i . kümesi bulmak için, çırpı tablosundaki i anahtarını ararız. Böylece i . kümesi, özet tablosunda tek bir aramayla bulmuş oluruz.
- Çırpı tablosu yalnızca boş olmayan kümelerle işaret eden işaretçileri saklar. Çırpı tablosunda boş bir küme için arama yapıldığında LD döndürülür, ki bu kümenin boş olduğunu belirtir.
- Tüm kümeler boş ise, $V.summary$ özelliği LD olur. Aksi halde, $V.summary$ evren boyutu $\sqrt{u}$ olan bir RS-vEB ağacına işaret eder.

Çırpı tablosu bir dinamik tablo ile uygulandığından, gerek duyduğu alan boş olmayan kümelerle orantılıdır.

Boş olmayan bir RS-vEB ağacına eleman eklememiz gerektiğinde, u parametresi RS-vEB ağacının evren boyutu olmak üzere, aşağıdaki prosedürü çağırarak RS-vEB ağacını oluştururuz:

```

CREATE-NEW-RS-vEB-TREE( $u$ )
1  yeni bir  $V$  vEB ağacı tahsis et
2   $V.u = u$ 
3   $V.min = LD$ 
4   $V.max = LD$ 
5   $V.summary = LD$ 
6   $V.cluster$  i, boş bir dinamik çırpı tablosu olarak yarat
7  return  $V$ 

```

- a. vEB-TREE-INSERT prosedürünü, CREATE-NEW-RS-vEB-TREE yi gerektiği gibi çağırarak x i V RS-vEB ağacına ekleyen RS-vEB-TREE-INSERT (V, x) prosedürünün kabakodunu elde etmek üzere düzenleyin.
- b. vEB-TREE-SUCCESSOR prosedürünü, x in V AA-vEB ağacındaki ardılını, veya x in ardılı V de yoksa LD döndüren AA-vEB-TREE-SUCCESSOR(V, x) prosedürünün kabakodunu elde etmek üzere düzenleyin.
- c. Basit düzgün çarpılama (simple uniform hashing) olduğu varsayımı altında, RS-vEB-TREE-INSERT ve RS-vEB-TREE prosedürlerinin beklenen $O(\lg \lg u)$ zamanda çalıştığını kanıtlayın.
- d. Bir vEB ağacından hiç eleman silinmediğini kabul ederek, n RS-vEB ağacında saklanan elemanların sayısı olmak üzere, RS-vEB ağacının alan gereksiniminin $O(n)$ olduğunu kanıtlayın.
- e. RS-vEB ağaçlarının vEB ağaçlarına göre bir başka avantajı daha vardır: yaratılmaları daha az zaman gerektirir. Boş bir RS-vEB ağacını yaratmak ne kadar zaman alır?

20-2 y -hızlı taban ağaçları(trieler)

Bu problem D. Willard'ın “ y -hızlı taban ağaçları”nı inceler. Onlar da van Emde Boas ağaçları gibi, MEMBER, MINIMUM, MAXIMUM, SUCCESSOR ve PREDECESSOR işlemlerinin her birini, boyutu u olan bir evrenden çekilen elemanlar üzerinde, en kötü durumda $O(\lg \lg u)$ zamanda gerçekleştirir. INSERT ve DELETE işlemleri ise $O(\lg \lg u)$ amortize zamanda çalışır. Azaltılmış-alanlı van Emde Boas ağaçları (bkz. Problem 20-1) gibi, y -hızlı taban ağaçları da n elemanı saklamak için $O(n)$ alan kullanır. y -hızlı taban ağaçlarının tasarımı, kusursuz çarpılamaya (perfect hashing) (bkz. Kısım 11.5) bağlıdır.

Ön bir yapı olarak, yalnızca bir dinamik kümedeki tüm elemanları değil, her elemanın ikili temsiline örneklerini de saklayan bir kusursuz çarpı tablosu yarattığımızı düşünelim. Örneğin, $u = 6$, dolayısıyla $\lg u = 4$ ise ve $x = 13$ kümedeyse, 13'ün ikili temsili 1101 olduğundan, kusursuz çarpı tablosunda 1, 11, 110 ve 1101 saklanır. Çarpı tablosuna ek olarak, kümede o an bulunan elemanların artan sırada bir çift yönlü bağlı listesini oluşturuyoruz.

- a. Bu yapı ne kadar alana gereksinim duyar?
- b. Nasıl MINIMUM ve MAXIMUM işlemlerinin $O(1)$ zamanda; MEMBER, SUCCESSOR ve PREDECESSOR işlemlerinin $O(\lg \lg u)$ zamanda; ve INSERT ve DELETE işlemlerinin $O(\lg u)$ zamanda çalıştığını gösterin.

Alan gereksinimini $O(n)$ 'e düşürmek için, veri yapısına aşağıdaki değişiklikleri yapıyoruz:

- n elemanı, $\lg u$ boyutunda $n/\lg u$ adet gruba topluyoruz (Şimdilik $\lg u$ 'nun n i böldüğünü varsayalım). İlk grup kümenin en küçük $\lg u$ elemanından oluşuyor, ikinci grup sonraki $\lg u$ en küçük elemandan oluşuyor ve böyle devam ediyor.
- Her grup için bir “temsilci” değer atıyoruz. i . grubun temsilcisi en az i . gruptaki en büyük eleman kadar büyük ve $(i + 1)$ grubun her elemanından küçüktür. (Son grubun temsilcisi mümkün en büyük eleman olan $i - 1$ olabilir). Temsilcinin o an kümede bulunmayan bir değer olabileceğine dikkat edin.
- Her gruptaki $\lg u$ adet elemanı, kırmızı-siyah ağaç gibi bir dengeli ikili arama ağacında tutuyoruz. Her temsilci grubunun dengeli ikili arama ağacına, ve her dengeli ikili arama ağacı da grubunun temsilcisine işaret eder.
- Kusursuz çırpı tablosu yalnızca, çift yönlü bağlı listede artan sırada saklanan temsilcileri saklar.

Bu yapıya bir **y-hızlı taban ağacı(trie)** diyoruz.

- Bir y-hızlı önek ağacının, n adet elemanı saklamak için $O(n)$ alana gereksinim duyduğunu gösterin.
- MINIMUM ve MAXIMUM işlemlerinin, bir y-hızlı taban ağacıyla nasıl $O(\lg \lg u)$ zamanda gerçekleştirileceğini gösterin.
- MEMBER işleminin nasıl $O(\lg \lg u)$ zamanda gerçekleştirileceğini gösterin.
- PREDECESSOR ve SUCCESSOR işlemlerinin nasıl $O(\lg \lg u)$ zamanda gerçekleştirileceğini gösterin.
- INSERT ve DELETE işlemlerinin neden $O(\lg \lg u)$ zaman aldığını açıklayın.
- y-hızlı taban ağacındaki her grubun tam $\lg u$ adet eleman içermesi gereksiniminin, INSERT ve DELETE işlemleri $O(\lg \lg u)$ amortize zamanda çalışacak şekilde ve diğer işlemlerin asimptotik çalışma zamanlarını etkilemeden, nasıl gevşetileceğini gösterin.

Bölüm notları

Bu bölümdeki veri yapısı adını 1975 yılında ilk fikri ortaya koyan P. van Emde Boas'tan almaktadır [339]. van Emde Boas'ın [340] ve van Emde Boas, Kaas, ve Zijlstra'nın [341] sonraki yayınları, fikri ve anlatımı geliştirdiler. Sonrasında

Mehlhorn ve Naeher [252] fikri, asal boyuttaki evrenlerde uygulanacak şekilde genişletti. Mehlhorn'un kitabı [249] van Emde Boas ağaçlarına, bu bölümdekinden biraz daha farklı bir şekilde yaklaşır.

Dementiev et al. [83], van Emde Boas ağaçlarının arkasındaki fikri kullanarak, özyinelemeli olmayan, ve kendi denemelerinde van Emde Boas ağaçlarından hızlı çalışan, üç seviyeli bir arama ağacı geliştirdi.

Wang ve Lin [347] her işlem için sabit bir amortize zamana sahip ve düzenek içerisinde $O(\lg \lg u)$ aşaması olan, donanım-düzenekli bir van Emde Boas ağacı tasarladı.

Patrascu ve Thorup [273, 274] tarafından verilen bir alt sınır, rassallığa izin verildiğinde dahi, van Emde Boas ağaçlarının, öncül işlemi için optimal olduğunu göstermektedir.

Bazı uygulamalarda, n farklı eleman ayrık kümelerin bir koleksiyonunda toplanır. Bu uygulamalar genelde iki işleme ihtiyaç duyar: verilen bir elemanı içeren tek (yegâne) kümeyi bulmak ve iki kümeyi birleştirmek. Bu bölüm, bahsedilen işlemleri destekleyen bir veri yapısını sağlamak için gerekli yöntemleri inceliyor.

Kısım 21.1 bir ayrık küme veri yapısının desteklediği işlemleri açıklıyor ve basit bir uygulama tanıtıyor. Kısım 21.2 de, ayrık kümeler için basit bir bağlantılı liste uygulamasına bakıyoruz. Kısım 21.3 köklü ağaçları kullanan daha etkili bir temsil yöntemini anlatıyor. Ağaç temsili kullanıldığında, çalışma zamanı teorik olarak doğrusal üstüdür, ancak tüm pratik amaçlar için doğrusaldır. Kısım 21.4 ağaç tabanlı uygulamada işlemlerin çalışma zamanlarında karşımıza çıkan, kendi çok hızlı büyüyen ve tersi çok yavaş büyüyen bir fonksiyonu tanımlıyor ve tartışıyor. Sonra da karmaşık bir amortize analiz ile, çok çok az doğrusal üstü olan çalışma zamanı için bir üst sınırı kanıtlıyor.

21.1 Ayrık-küme işlemleri

Bir *ayrık-küme veri yapısı*, bir $\mathcal{S} = \{S_1, S_2, \dots, S_k\}$ ayrık dinamik küme koleksiyonunu saklar. Her kümeyi bir *temsilci* ile saptarız, ki o da kümenin bir elemanıdır. Bazı uygulamalarda, hangi elemanın temsilci olarak kullanıldığı önemli değildir; yalnızca, dinamik kümeden iki defa bir temsilci istediğimizde, istekler arasında kümeyi düzenlemediysek, ikisinde de aynı cevabı alıp almadığımızı dikkat ederiz. Farklı uygulamalar temsilci seçimi için, en küçük elemanın seçilmesi gerektiği gibi (tabii ki elemanların sıralanabilir olduklarını varsayarak) önceden tanımlı kurallar getirebilir.

İncelediğimiz diğer dinamik küme uygulamalarında olduğu gibi, kümenin her bir elemanını bir nesne ile temsil ediyoruz. x bir nesneyi ifade etmek üzere, şu işlemleri desteklemek istiyoruz:

MAKE-SET (x) biricik elemanı (dolayısıyla temsilcisi) x olan yeni bir küme yaratır. Kümeler ayrık olduğundan, x in halihazırda başka bir kümede yer almayor olması gerekir.

UNION(x, y) x ve y yi içeren dinamik kümeleri, diyelim ki S_x ve S_y olsunlar, yeni bir küme olarak birleştirir. İşlem öncesinde iki kümenin ayrık olduğunu kabul ediyoruz. Oluşan kümenin temsilcisi $S_x \cup S_y$ nin herhangi bir elemanı olabilir. Ancak UNION ın çoğu uygulamasında yeni temsilci olarak ya S_x in, ya da S_y nin temsilcisi seçilir. Koleksiyondaki kümelerin ayrık olması gerektiğinden, S_x ve S_y kümelerini $\mathcal{S}$ koleksiyonundan çıkararak kavramsal olarak yok ederiz. Pratikte, genellikle bir kümenin elemanlarını diğerinin içine alırız.

FIND-SET (FIND-SET)(x) x i içeren yegâne kümenin temsilcisini işaret eden bir işaretçi döndürür.

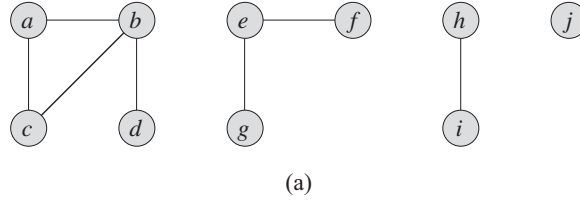
Bu bölüm boyunca, ayrık küme veri yapılarının çalışma zamanlarını iki parametreye göre analiz edeceğiz: n , MAKE-SET işlemlerinin sayısı ve m , MAKE-SET, UNION ve FIND-SET işlemlerinin toplam sayısı. Kümeler ayrık olduğundan, her UNION işlemi küme sayısını bir azaltır. Bu yüzden, $n - 1$ UNION işleminden sonra yalnızca bir küme kalır. UNION işlemlerinin sayısı da, doğal olarak, en fazla $n - 1$ olabilir. MAKE-SET işlemleri, tüm işlemlerin sayısı olan m e dahil olduğundan, $m \geq n$ olduğunu da göz önünde bulundurun. n adet MAKE-SET işleminin, ilk gerçekleştirilen n işlem olduğunu varsayıyoruz.

Ayrık küme veri yapılarının bir uygulaması

Ayrık küme veri yapılarının birçok uygulamasından biri yönsüz bir çizgenin bağlantılı bileşenlerini belirlerken ortaya çıkar (bkz. Kısım B.4). Örneğin, Şekil 21.1(a), dört bağlantılı bileşeni olan bir çizgeyi gösteriyor.

Aşağıdaki CONNECTED-COMPONENT prosedürü, bir çizgenin bağlantılı bileşenlerini bulmak için ayrık küme işlemlerini kullanıyor. CONNECTED-COMPONENTS çizgeyi ön işlemden geçirdikten sonra, SAME-COMPONENT prosedürü iki tepenin aynı bağlantılı bileşende olup olmadığına dair sorguya cevap veriyor.¹ (Kabakod, G çizgesinin tepeler kümesini $G.V$ ile, ayrıtlar kümesini de $G.E$ ile ifade ediyoruz).

¹Çizgenin ayrıtları statik olduğunda —zaman içinde değişmediğinde—, bağlantılı bileşenleri derinlik-öncelikli arama ile daha hızlı hesaplayabiliriz (Alıştırma 22.3-12). Yine de bazen, ayrıtlar dinamik olarak eklenir ve onlar eklendikçe yeni bağlantılı bileşenleri bulmamız gerekir. Bu durumda, her yeni ayrıt için yeni bir derinlik öncelikli arama yapmaktansa, burada verilen uygulamayı kullanmak daha etkin olabilir.



İşlenen ayrıt	Ayrık kümelerin koleksiyonu									
Başlangıç kümeleri	{a}	{b}	{c}	{d}	{e}	{f}	{g}	{h}	{i}	{j}
(b,d)	{a}	{b,d}	{c}		{e}	{f}	{g}	{h}	{i}	{j}
(e,g)	{a}	{b,d}	{c}		{e,g}	{f}		{h}	{i}	{j}
(a,c)	{a,c}	{b,d}			{e,g}	{f}		{h}	{i}	{j}
(h,i)	{a,c}	{b,d}			{e,g}	{f}		{h,i}		{j}
(a,b)	{a,b,c,d}				{e,g}	{f}		{h,i}		{j}
(e,f)	{a,b,c,d}				{e,f,g}			{h,i}		{j}
(b,c)	{a,b,c,d}				{e,f,g}			{h,i}		{j}

(b)

Şekil 21.1 (a) Dört bağlantılı bileşeni olan bir çizge: $\{a, b, c, d\}$, $\{e, f, g\}$, $\{h, i\}$, ve $\{j\}$. (b) Tüm ayrıtlar işlendikten sonra ayrık kümelerin koleksiyonu.

CONNECTED-COMPONENTS(G)

```

1  for her tepe  $v \in G.V$ 
2      MAKE-SET( $v$ )
3  for her ayrıt  $(u, v) \in G.V$ 
4      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5          UNION( $u, v$ )

```

SAME-COMPONENT(u, v)

```

1  if FIND-SET( $u$ ) == FIND-SET( $v$ )
2      return TRUE
3  else return FALSE

```

CONNECTED-COMPONENTS prosedürü başta her v tepesini kendi kümesine yerleştiriyor. Sonra, her (u, v) ayrıtı için, u ve v tepelerini içeren kümeleri birleştiriyor. Alistırma 21.1-2 de verildiği gibi, tüm ayrıtlar işlendikten sonra, iki tepe ancak ve ancak karşılık geldikleri nesneler aynı kümedeyse aynı bağlantılı bileşenin içerisinde oldukları. Böylece, CONNECTED-COMPONENTS kümeleri, iki tepenin aynı bileşende olup olmadığını belirler. SAME-COMPONENT prosedürü tarafından belirlene-

bilecek hale getirir. Şekil 21.1(b) CONNECTED-COMPONENTS prosedürünün ayrık kümeleri nasıl hesapladığını gösteriyor.

Bu bağlantılı bileşenler algoritmasının gerçek bir uygulamasında, çizgenin temsilleri ile ayrık küme veri yapısının birbirlerine referans göstermeleri gerekirdi. Yani, bir tepeyi temsil eden nesnede, o tepeye karşılık gelen ayrık küme nesnesine işaret eden ve aynı şekilde bir de ters yöne birer işaretçi olmalıydı. Bu programlama detayları uygulama diline bağlıdır, ve bunları burada irdelemeyeceğiz.

Alıştırmalar

21.1-1

CONNECTED-COMPONENTS'ın, $G = \{a, b, c, d, e, f, g, h, i, j, k\}$ ve E ayrıtlar kümesi $(d, i), (f, k), (g, i), (b, g), (a, h), (i, j), (d, k), (b, j), (d, f), (g, j), (a, e)$ şeklinde dizilmiş olmak üzere, bir $G = (V, E)$ yönsüz çizgesi üzerinde çalıştırılıyor olsun. Satır 3-5 teki her iterasyondan sonra, her bir bağlantılı bileşendeki tepeleri listeleyin.

21.1-2

Tüm ayrıtlar CONNECTED-COMPONENTS tarafından işlendikten sonra, iki tepenin ancak ve ancak aynı kümenin elemanlarıysa aynı bağlantılı bileşende bulunduklarını gösterin.

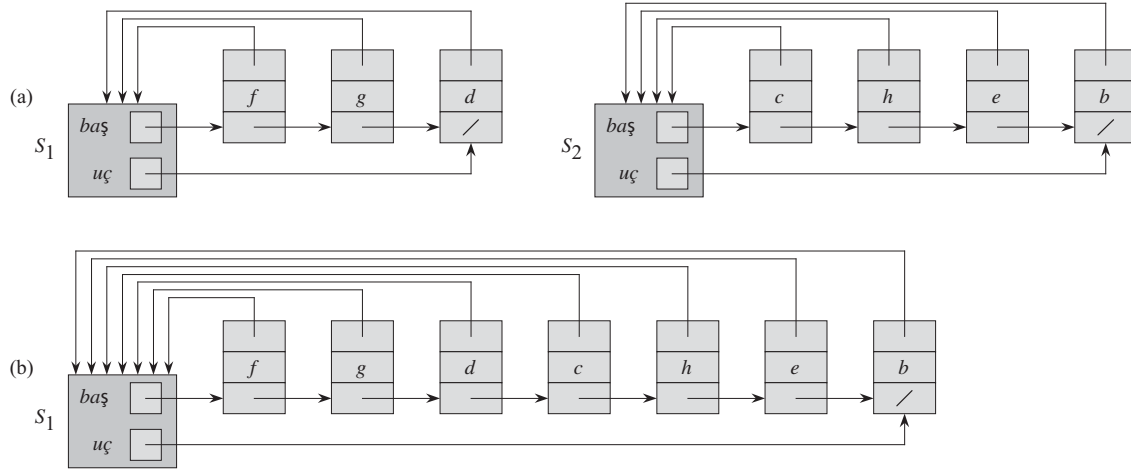
21.1-3

k adet bağlantılı bileşeni olan bir yönsüz $G = (V, E)$ çizgesinin üzerinde CONNECTED-COMPONENTS ı çalıştırırken, FIND-SET prosedürü kaç kez çağırılır? UNION kaç kez çağırılır? Cevaplarınızı $|V|$, $|E|$ ve k cinsinden ifade edin.

21.2 Ayrık-kümelerin bağlı-liste gösterimi

Şekil 21.2(a) bir ayrık küme veri yapısını uygulamanın kolay bir yolunu gösteriyor: her küme kendi bağlantılı listesi tarafından temsil ediliyor. Her kümenin nesnesi, listedeki ilk elemana işaret eden *baş* ve son elemana işaret eden *uç* özelliklerine sahip. Her bir bağlantılı listede, nesneler herhangi bir sırada yer alabilir. Temsilci, liste içindeki ilk nesnede bulunan küme elemanıdır.

Bu bağlantılı liste temsiliyle, MAKE-SET ve FIND-SET çok kolaydır ve $O(1)$ zamanda çalışırlar. MAKE-SET(x) ı gerçekleştirmek için, tek nesnesi x olan yeni bir bağlantılı liste yaratırız. FIND-SET(x) için, x ten küme nesnesine işaret eden işaretçiyi takip eder ve *baş* tarafından işaret edilen nesnenin elemanını döndürürüz. Örneğin, Şekil 21.2(a) da, FIND-SET(g) çağırısı yaparsak, geriye f i döndürür.



Şekil 21.2 (a) İki kümenin bağlantılı liste temsili. S_2 kümesi, f temsilci olmak üzere d , f ve g elemanlarını, S_2 kümesi, c temsilci olmak üzere b , c , e ve h elemanlarını içeriyor. Listedeki her nesnede bir küme elemanı, listedeki bir sonraki nesneye bir işaretçi ve küme nesnesine geri işaret eden bir başka işaretçi bulunuyor. Her küme nesnesi, sırasıyla ilk ve son nesnelere işaret eden *baş* ve *uç* işaretçilerini içeriyor. (b) $\text{UNION}(g, e)$ nin sonucunda e yi içeren bağlantılı liste g yi içeren bağlantılı listeye ekleniyor. Sonuç kümesinin temsilcisi ise f dir. e nin listesinin küme nesnesi, S_2 , yok ediliyor.

Birleşimin basit bir uygulaması

UNION işlemi, küme temsili için bağlantılı liste kullanılan en basit uygulamasında, MAKE-SET ve FIND-SET ten epey fazla zaman alır. Şekil 21.2(b) nin gösterdiği gibi, $\text{UNION}(x, y)$ işlemini, y nin listesini x in listesinin üzerine ekleyerek gerçekleştiriyoruz. x in listesinin temsilcisi sonuç kümesinin temsilcisi oluyor. x in listesindeki *tail* işaretçisini, y nin listesini nereye ekleyeceğimizi kolayca bulmak için kullanıyoruz. y nin listesindeki tüm elemanlar x in listesine katıldığından, y nin listesi için kullanılan küme nesnesini yok edebiliriz. Örneğin Şekil 21.2 de, $\text{UNION}(g, e)$ işlemi b , c , e ve h nin nesnelerindeki işaretçilerin güncellenmesine sebep oluyor.

Aslında, n nesne üzerinde m işlemden oluşan bir diziyi, $\Theta(n^2)$ zamanda çalışacak biçimde kolayca kurabiliriz. $x_1, x_2, \dots, x_n$ nesnelerimiz olduğunu düşünelim. Şekil 21.3 de gösterildiği gibi, n adet MAKE-SET işlemini takiben $n - 1$ adet UNION işleminden oluşan, yani $m = 2n - 1$ olacak şekilde, bir işlemler dizisini çalıştırıyoruz. n adet MAKE-SET işlemi için $\Theta(n)$ zaman harcıyoruz. i . UNION işlemi i adet nesneyi güncellediğinden, toplam $n - 1$ UNION işlemi tarafından güncellenen nesnelerin sayısı

İşlem	Güncellenen nesne sayısı
MAKE-SET(x_1)	1
MAKE-SET(x_2)	1
$\vdots$	$\vdots$
MAKE-SET(x_n)	1
UNION(x_2, x_1)	1
UNION(x_3, x_2)	2
UNION(x_4, x_3)	3
$\vdots$	$\vdots$
UNION(x_n, x_{n-1})	$n - 1$

Şekil 21.3 Basit bir UNION uygulaması ve bağlı-liste küme temsili kullanılarak n adet nesne üzerinde yapılan, toplam $\Theta(n^2)$ veya işlem başına ortalama $\Theta(n)$ zaman alan $2n - 1$ işleminden oluşan bir işlem dizisi.

$$\sum_{i=1}^{n-1} i = \Theta(n^2)$$

olur. İşlemlerin toplam sayısı $2n-1$ dir ve böylece her işlem ortalamada $\Theta(n)$ zamana ihtiyaç duyar. Yani, bir işlemin amortize zamanı $\Theta(n)$ olur.

Bir ağırlıklı-birleşim sezgiseli

En kötü durumda, UNION prosedürünün yukarıdaki gibi bir uygulaması her çağrı için ortalama $\Theta(n)$ zamana ihtiyaç duyar, çünkü daha küçük bir listenin üzerine daha büyük bir liste ekliyor olabiliriz; uzun listedeki her eleman için küme nesnesine işaret eden işaretçiyi güncellememiz gerekir. Bunun yerine her listenin aynı zamanda listenin uzunluğunu da içerdiğini (ki bunu kolayca sağlayabiliriz) ve her zaman, bağlantıları gelişigüzel kopararak, daha küçük bir listeyi daha büyük bir listenin üzerine eklediğimizi düşünün. Bu basit **ağırlıklı-birleşim sezgiseli** ile, yine de, her iki kümede de $\Omega(n)$ adet eleman varsa, tek bir UNION işlemi $\Omega(n)$ zaman alabilir. Ancak aşağıdaki teoremin gösterdiği üzere, n tanesi MAKE-SET işlemi olan m adet MAKE-SET, UNION ve FIND-SET işleminden oluşan bir işlem dizisi, $O(m + n \lg n)$ zaman alır.

Teorem 21.1

Ayrık kümelerin bağlı liste temsili ve ağırlıklı-birleşim sezgiseli kullanıldığında, n tanesi MAKE-SET işlemi olan m adet MAKE-SET, UNION ve FIND-SET işleminin oluşan bir işlem dizisi, $O(m + n \lg n)$ zaman alır.

Kanıt Her bir UNION işlemi iki ayrık kümeyi birleştirdiğinde, toplamda en fazla $n - 1$ UNION işlemi yapabiliriz. Şimdi, bu UNION işlemlerinin aldığı toplam zaman için bir sınır bulalım. Her nesne için, nesneden küme nesnesine işaret eden işaretçilerin güncellenme sayısı için bir üst sınır belirleyerek başlıyoruz. Belirli bir x nesnesini düşünelim. x in işaretçisi her güncellendiğinde, x in başta küçük kümede olduğunu biliriz. Bu nedenle, x in işaretçisi ilk kez güncellendiğinde, sonuç kümesinin en az 2 üyesinin olması gerekirdi. Benzer şekilde, x in işaretçisi bir sonraki kez güncellendiğinde, sonuç kümesinin en az 4 elemanı olması gerekirdi. Bu şekilde devam ederek, her $k \leq n$ için, x in işaretçisi $\lceil \lg k \rceil$ defa güncellendikten sonra, sonuç kümesinin en az k elemanı olması gerekir. En büyük kümede en fazla n eleman olabileceği için, her nesnenin işaretçisi, tüm UNION işlemleri boyunca en fazla $\lceil \lg n \rceil$ defa güncellenebilir. Dolayısıyla, tüm UNION işlemleri boyunca nesne işaretçilerinin güncellenmesi için geçen zaman $O(n \lg n)$ dir. *tail* işaretçilerinin güncellenmesini de göz önünde tutmalıyız, ki bu da her UNION işleminde yalnızca $\Theta(1)$ zaman alır. Sonuçta tüm UNION işlemlerinde harcanan zaman $O(n \lg n)$ olur.

m adet işlemden oluşan işlem dizisinin tamamı için zaman kolayca çıkarılır. Her MAKE-SET ve FIND-SET işlemi $O(1)$ zaman alır ve bunlardan $O(m)$ adet vardır. Bū yüzden de, tüm dizi için zaman $O(m + n \lg n)$ dir. ■

Alıştırımlar

21.2-1

Bağılantılı liste temsili ve ağırlıklı-birleşim sezgiselini kullanan MAKE-SET, FIND-SET ve UNION işlemleri için kaba kod yazın.

21.2-2

Aşağıdaki programın sonucunda çıkan veri yapısını ve FIND-SET işlemleri tarafından döndürülen cevapları gösterin. Bağlı liste temsilini ağırlıklı-birleşim sezgisiyle kullanın.

```

1  for  $i = 1$  to 16
2      MAKE-SET( $x_1$ )
3  for  $i = 1$  to 15 by 2
4      UNION( $x_1, x_{i+1}$ )
5  for  $i = 1$  to 13 by 4
6      UNION( $x_1, x_{i+2}$ )
7  UNION( $x_1, x_5$ )
8  UNION( $x_{11}, x_{13}$ )
9  UNION( $x_1, x_{10}$ )
10 FIND-SET( $x_2$ )
11 FIND-SET( $x_9$ )

```

x_i ve x_j elemanlarını içeren kümelerin boyutu aynıysa, $\text{UNION}(x_i, x_j)$ işleminin, x_j nin listesini x_i nin listesinin üzerine eklediğini kabul edin.

21.2-3

Teorem 21.1 in toplam kanıtını (aggregate proof), MAKE-SET ve FIND-SET için $O(1)$ ve UNION için $O(\lg n)$ amortize zaman elde edecek şekilde uyarlayın. Bağlantılı liste temsili ve ağırlıklı-birleşim sezgiselini kullanın.

21.2-4

Bağlı liste temsili ve ağırlıklı-birleşim sezgiselinin kullanıldığını varsayarak Şekil 21.3 teki işlemler dizisinin çalışma zamanı üzerinde sıkı bir asimptotik sınır verin.

21.2-5

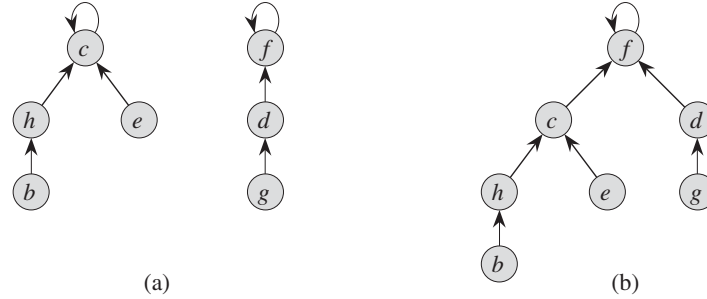
Profesör Gompers, her bir liste elemanındaki işaretçilerin sayısı iki kalmak üzere, her küme nesnesinin içinde iki işaretçi yerine (*head ve tail*) tek bir işaretçinin kullanılabileceğini düşünüyor. Profesör'ün düşüncesinin sağlam temelleri olduğunu, işlemler bu bölümde anlatıldığı kadar zaman alacak şekilde, kümelerin bağlı liste ile nasıl temsil edilebileceğini açıklayarak gösterin. İşlemlerin nasıl çalıştığını da açıklayın. Tasarımınız, bu bölümde açıklandığı etkiyi yaratacak şekilde ağırlıklı-birleşim sezgiselinin kullanımına müsait olmalıdır. (*İpucu*: bağlı listenin ucunu, o kümenin temsilcisi olarak kullanın).

21.2-6

UNION prosedürü üzerinde, bağlı liste temsiliinde her listedeki son nesneye işaret eden *tail* işaretçisine ihtiyaç kalmayacak şekilde basit bir değişim önerin. Ağırlıklı-birleşim sezgiseli kullanılsın veya kullanılsın, değişiminiz UNION prosedürünün asimptotik çalışma zamanını değiştirmemeleridir (*İpucu*: Bir listeyi diğerine eklemek yerine, ikisini birbirine bağlayın).

21.3 Ayırık-küme ormanları

Ayrık kümelerin daha hızlı bir uygulamasında kümeleri, her düğüm bir eleman içerecek ve her ağaç bir kümeyi temsil edecek şekilde, köklü ağaçlarla temsil ederiz. Şekil 21.4(a) da gösterildiği üzere, bir **ayırık küme ormanında**, her üye yalnızca ebeveynine işaret eder. Her ağacın kökü temsilciyi içerir ve kendisinin ebeveynidir. Göreceğimiz gibi, bu temsili kullanan düz bir algoritma, bağlantılı liste temsili kullananlardan daha hızlı olmasa da, iki sezgisel —“rank ile birleşim” ve “yol sıkıştırma”— sunarak, asimptotik olarak optimal bir ayırık küme veri yapısına ulaşabiliriz.



Şekil 21.4 Bir ayrık küme ormanı. (a) Şekil 21.2 nin iki kümesini temsil eden iki ağaç. Soldaki ağaç, c temsilci olmak üzere, $\{b, c, e, h\}$ kümesini, sağdaki ağaç da, f temsilci olmak üzere, $\{b, f, g\}$ kümesini temsil ediyor. (b) $\text{UNION}(e, g)$ nin sonucu.

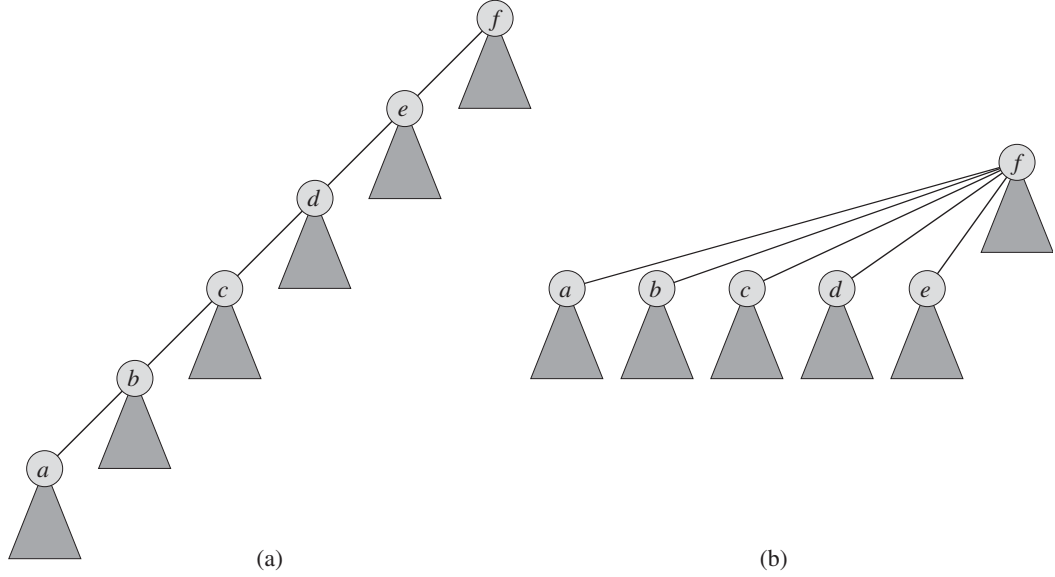
Üç ayrık küme işlemini şöyle gerçekleştiriyoruz. **MAKE-SET** işlemi tek bir düğümü olan bir ağaç yaratıyor. **FIND-SET** işlemini, ağacın kökünü bulana kadar ebeveyn işaretçilerini takip ederek gerçekleştiriyoruz. Köke doğru giden bu basit yolda ziyaret edilen tepeler **yol bulma** oluşturuyor. Şekil 21.4(b) de gösterilen **UNION** işlemi, bir ağacın kökünün diğerinin köküne işaret etmesine yol açıyor.

Çalışma zamanını iyileştirmek için sezgiseller

Buraya kadar bağlı liste uygulamasını iyileştiremedik. $n - 1$ adet **UNION** işlemi n düğümden oluşan doğrusal bir zincirden meydana gelen bir ağaç oluşturabilir. Ancak iki sezgisel kullanarak, tüm işlemlerin sayısı olan m cinsinden, neredeyse doğrusal bir çalışma zamanına erişebiliriz.

İlk sezgisel, **rank ile birleşim**, bağlı liste temsiline kullandığımız ağırlıklı birleşim sezgiseline benzerdir. Aşık bir yaklaşım, az düğümü olan ağacın kökünü çok düğümü olan ağacın köküne bağlamak olurdu. Her düğümün altağacının boyutunun kaydını tutmaktansa, analizi kolaylaştıracak bir yaklaşım kullanacağız. Her düğüm için bir rank saklıyoruz, ki bu da düğümün boyu için bir üst sınırdır. **Ranka** ile birleşimde, rank puanı düşük olan köke yüksek ranka sahip kökü işaret ettiriyoruz.

İkinci sezgisel olan **yol sıkıştırma** da gayet kolay ve oldukça etkilidir. Bu sezgiseli, Şekil 21.5 te gösterildiği gibi, **FIND-SET** işlemleri sırasında yol bulma üzerindeki her düğümü doğrudan kökü işaret ettirmek için kullanıyoruz. Yol sıkıştırma herhangi bir rankı değiştirmiyor.



Şekil 21.5 FIND-SET işlemi sırasında yol sıkıştırma. Oklar ve köklerdeki burgular hariç bırakılmış. (a) FIND-SET(a) çalıştırılmadan önceki kümeyi temsil eden bir ağaç. Üçgenler, kökleri gösterilen tepeler olan altağaçları temsil ediyor. (b) Aynı kümenin FIND-SET(a) çalıştırıldıktan sonraki durumu. Bulma yolu üzerindeki her tepe artık doğrudan köke işaret ediyor.

Ayrık küme ormanları için kabakod

Rank ile birleşim sezgiseliyle bir ayrık küme ormanını uygulamak için, rankların kaydını tutmalıyız. Her bir x tepesi ile, x in yüksekliği (x ile altındaki yapraklardan biri arasındaki en uzun basit yoldaki ayrıtların sayısı) üzerinde bir üst sınır olan $x.rank$ tam sayı değerini tutuyoruz. MAKE-SET tekil bir küme yarattığında, o kümeye karşılık gelen ağaçtaki tek tepe 0 başlangıç rankını alır. FIND-SET işlemleri rankları değiştirmez. UNION işleminde, ağaçların köklerinin aynı ranka sahip olup olmamasına göre iki durum vardır. Eğer köklerin rankı farklıysa, yüksek rankı kökü düşük ranka sahip kökün ebeveyni yaparız, ancak ranklar değişmez. Eğer köklerin rankları eşitse, keyfi olarak köklerden birini ebeveyn seçeriz ve onun rankını artırırız.

Bu yöntemi kabakoda dökelim. x tepesinin ebeveynini $x.p$ ile gösteriyoruz. UNION tarafından çağırılan LINK alt prosedürü, iki köke işaret eden işaretçileri girdi olarak alıyor.

MAKE-SET(x)

```
1   $x.p = x$ 
2   $x.rank = 0$ 
```

UNION(x, y)

```
1  LINK(FIND-SET( $x$ ), FIND-SET( $y$ ))
```

LINK(x, y)

```
1  if  $x.rank > y.rank$ 
2       $y.p = x$ 
3  else  $x.p = y$ 
4      if  $x.rank == y.rank$ 
5           $y.rank = y.rank + 1$ 
```

Yol sıkıştırma ile FIND-SET prosedürü oldukça basittir:

FIND-SET(x)

```
1  if  $x \neq x.p$ 
2       $x.p = \text{FIND-SET}(x.p)$ 
3  return  $x.p$ 
```

FIND-SET prosedürü bir **iki-geçişli yöntem**dir: özyineleme yaptığında, kökü bulmak için bulma yolundan yukarıya doğru bir geçiş yapar ve özyineleme geriye doğru sararken her tepeyi doğrudan köke işaret edecek şekilde güncellemek üzere bulma yolundan aşağıya bir geçiş yapar. FIND-SET(x) her çağrıldığında, Satır 3 te $x.p$ döndürür. Eğer x kök ise, o zaman FIND-SET Satır 2 yi atlar ve zaten x in aynısı olan $x.p$ yi döndürür; bu durumda özyineleme dibe kadar iner. Aksi halde, Satır 2 çalışır ve $x.p$ parametresiyle yapılan özyinelemeli çağrı köke işaret eden bir işaretçi döndürür. Satır 2 x tepesini, doğrudan kökü işaret edecek şekilde günceller ve Satır 3 bu işaretçiyi döndürür.

Sezgisellerin çalışma zamanı üzerindeki etkisi

Ayrık küme ormanları üzerindeki işlemlerin çalışma zamanını, ayrı ayrı, rank ile birleşim veya yol sıkıştırma sezgiselleri iyileştirir ve bu iyileşme iki sezgiseli beraber kullanırsak daha da büyük olur. Tek başına rank ile birleşim $O(m \lg n)$ çalışma zamanına sahiptir (bkz. Alıştırma 21.4-4) ve bu sınır sıkıdır (bkz. Alıştırma 21.3-3). Burada kanıtlamayacak olsak da, n adet MAKE-SET (ve dolayısıyla en fazla $n - 1$ UNION) ve f adet FIND-SET işlemlerinden oluşan bir işlem dizisi için, tek başına yol sıkıştırma sezgiseli en kötü durumda $\Theta(n + f \cdot (1 + \log_{2+f/n} n))$ çalışma zamanı verir.

Hem rank ile birleşimi hem de yol sıkıştırmayı kullandığımızda, en kötü durumda çalışma zamanı, $\alpha(m)$ Kısım 21.4 te tanımladığımız gibi çok yavaş büyüyen bir fonksiyon olmak üzere, $O(m \alpha(n))$ olur. Akla gelebilecek tüm ayrık küme veri yapısı uygulamalarında $\alpha(n) \leq 4$ tür; bu sebeple, tüm pratik durumlarda çalışma zamanına m cinsinden doğrusal gözüyle bakabiliriz. Ancak, kurallara göre konuşmak gerekirse, doğrusal üstüdür. Kısım 21.4 te, bu üst sınırı kanıtlıyoruz.

Alıştırmalar

21.3-1

Rank ile birleşim ve yol sıkıştırma ile ayrık küme ormanı kullanarak Alıştırma 21.2-2 yi tekrar yapın.

21.3-2

Yol sıkıştırma ile FIND-SET in özyinelemeli olmayan bir versiyonunu yazın.

21.3-3

Yalnızca rank ile birleşimi kullandığımızda $\Omega(m \lg n)$ zaman alan, n tanesi MAKE-SET işlemi olan m adet MAKE-SET, UNION ve FIND-SET işleminden oluşan bir işlem dizisi verin.

21.3-4

PRINT-SET(x) işlemini eklemek istediğimizi düşünün. Bu işlem verilen bir x kümesinin tüm elemanlarını herhangi bir sırada yazdırıyor. Diğer işlemlerin asimptotik çalışma zamanlarını değiştirmeden, PRINT-SET(x) ın x kümesinin elemanları cinsinden doğrusal bir zamanda çalışmasını ayrık küme ormanının her bir tepesine tek bir özellik ekleyerek nasıl sağlayabileceğimizi gösterin.

21.3-5 *

Tüm LINK işlemleri bir FIND-SET işleminden sonra yapılmak üzere, m adet MAKE-SET, FIND-SET ve LINK işleminden oluşan herhangi bir işlem dizisinin, hem rank ile birleşim hem de yol sıkıştırma kullanırsak $O(m)$ zaman aldığını gösterin. Aynı durumda yalnızca yol sıkıştırma sezgiseli kullanırsak ne olur?

* 21.4 Yol sıkıştırma ile bir rank tarafından birleşimin analizi

Kısım 21.3 te belirtildiği gibi, rank ile birleşim ve yol sıkıştırma sezgiselleri beraber, n eleman üzerindeki m adet ayrık küme işlemi için $O(m \alpha(n))$ zamanda çalışıyor. Bu kısımda, ne kadar yavaş büyüdüğünü görmek için α fonksiyonunu inceleyeceğiz. Sonra da amortize analizin potansiyel yöntemini kullanarak bu çalışma zamanını kanıtlayacağız.

Çok hızlı büyüyen bir fonksiyon ve onun çok yavaş büyüyen tersi

$k \geq 0$ ve $j \geq 1$ tam sayıları için, $A_k(j)$ fonksiyonunu,

$$A_k(j) = \begin{cases} j + 1 & k = 0 \text{ ise,} \\ A_{k-1}^{(j+1)}(j) & k \geq 1 \text{ ise,} \end{cases}$$

şeklinde tanımlıyoruz. Burada $A_{k-1}^{(j+1)}(j)$ ifadesi, Kısım 3.2 de verilen fonksiyonel iterasyon gösterimini kullanıyor. Özel olarak, $i \geq 1$ için $A_{k-1}^{(0)}(j) = j$ ve $A_{k-1}^{(i)}(j) = A_{k-1}(A_{k-1}^{(i-1)}(j))$ dir. k parametresine A fonksiyonunun *seviyesi (level)* diyoruz.

$A_k(j)$ fonksiyonu hem j , hem de k ile sıkı şekilde artar. Bu fonksiyonun ne kadar hızlı büyüdüğünü görmek adına, önce $A_1(j)$ ve $A_2(j)$ için kapalı form ifadeler elde ediyoruz.

Lemma 21.2

Herhangi bir $j \geq 1$ tam sayısı için, $A_1(j) = 2j + 1$ dir.

Kanıt Önce $A_0^{(i)}(j) = j + i$ olduğunu göstermek için i üzerinde tümevarımı kullanıyoruz. Temel durumu için $A_0^{(0)}(j) = j = j + 0$ dir. Tümevarım adımı için, $A_0^{(i-1)}(j) = j + (i-1)$ olduğunu varsayalım. O halde $A_0^{(i)}(j) = A_0(A_0^{(i-1)}(j)) = (j + (i-1)) + 1 = j + i$ dir. Sonuç olarak, $A_1(j) = A_0^{(j+1)}(j) = j + (j+1) = 2j + 1$ olur. ■

Lemma 21.3

Herhangi bir $j \geq 1$ tamsayısı için, $A_2(j) = 2^{j+1}(j+1) - 1$ dir.

Kanıt Önce $A_1^{(i)}(j) = 2^i(j+1) - 1$ olduğunu göstermek için i üzerinde tümevarımı kullanıyoruz. Temel durumu için $A_1^{(0)}(j) = j = 2^0(j+1) - 1$ dir. Tümevarım adımı için, $A_1^{(i-1)}(j) = 2^{i-1}(j+1) - 1$ olduğunu varsayalım. O halde $A_1^{(i)}(j) = A_1(A_1^{(i-1)}(j)) = A_1(2^{i-1}(j+1) - 1) = 2 \cdot (2^{i-1}(j+1) - 1) + 1 = 2^i(j+1) - 2 + 1 = 2^i(j+1) - 1$ dir. Sonuç olarak, $A_2(j) = A_1^{(j+1)}(j) = 2^{j+1}(j+1) - 1$ olur. ■

Artık $k = 0, 1, 2, 3, 4$ seviyeleri için $A_k(1)$ i inceleyerek $A_k(j)$ nin ne kadar hızlı büyüdüğünü görebiliriz. $A_0(k)$ tanımından ve yukarıdaki lemmalardan, elimizde $A_0(1) = 1 + 1 = 2$, $A_1(1) = 2 \cdot 1 + 1 = 3$ ve $A_2(1) = 2^{1+1} \cdot (1+1) - 1 = 7$ var.

Ayrıca elimizde

$$\begin{aligned}
 A_3(1) &= A_2^{(2)}(1) \\
 &= A_2(A_2(1)) \\
 &= A_2(7) \\
 &= 2^8 \cdot 8 - 1 \\
 &= 2^{11} - 1 \\
 &= 2047
 \end{aligned}$$

ve

$$\begin{aligned}
 A_4(1) &= A_3^{(2)}(1) \\
 &= A_3(A_3(1)) \\
 &= A_3(2047) \\
 &= A_2^{(2048)}(2047) \\
 &\gg A_2(2047) \\
 &= 2^{2048} \cdot 2048 - 1 \\
 &> 2^{2048} \\
 &= (2^4)^{512} \\
 &= 16^{512} \\
 &\gg 10^{80}
 \end{aligned}$$

de vardır ki bu da gözlemlenebilen evrendeki tüm atomların tahmini sayısına eşittir ($\gg$ sembolü “çok daha büyük” bağıntısını ifade ediyor).

$n \geq 0$ tam sayısı için $A_k(n)$ fonksiyonunun tersini

$$a(n) = \min \{k : A_k(1) \geq n\}$$

ile tanımlıyoruz. Sözel olarak $\alpha(n)$, $A_k(1)$ in en az n olduğu en düşük k seviyesidir. $A_k(1)$ in yukarıdaki değerlerinden görüyoruz ki

$$\alpha(n) = \begin{cases} 0 & 0 \leq n \leq 2 \text{ için,} \\ 1 & n = 3 \text{ için,} \\ 2 & 4 \leq n \leq 7 \text{ için,} \\ 3 & 8 \leq n \leq 2047 \text{ için,} \\ 4 & 2048 \leq n \leq A_4(1) \text{ için.} \end{cases}$$

Yalnızca n in “astronomik” terimi bile hafif kalan, çok büyük (dev bir sayı olan $A_4(1)$ den büyük) değerleri için $\alpha(n) > 4$ olur ve bu yüzden de tüm pratik amaçlar için $\alpha(n) \geq 4$ tür.

Rankların özellikleri

Bu kısmın geri kalanında, Rankların birleşim ve yol sıkıştırma ile ayrık küme işlemlerinin çalışma zamanı için bir $O(m \alpha(n))$ sınırı kanıtıyoruz. Bu sınırı kanıtlamak için öncelikle rankların bazı basit özelliklerini kanıtlayalım.

Lemma 21.4

Tüm x tepeleri için, $x \neq x.p$ sıkı eşitsizliği ile $x.rank \leq x.p.rank$ elimizde var. $x.rank$ in değeri başlangıçta 0 dır ve $x \neq x.p$ olana kadar zamanla artar; o andan sonra da $x.rank$ değişmez. $x.p.rank$ in değeri zamanla monoton olarak artar.

Kanıt Kanıt, Kısım 21.3 te gördüğümüz MAKE-SET, UNION, ve FIND-SET uygulamaları kullanılarak, işlem sayısı üzerinde doğrudan tümevarımla yapılır. Kanıtı Alıştırma 21.4-1 olarak bırakıyoruz. ■

Sonuç 21.5

Herhangi bir tepeden bir köke doğru giden yolu takip ederken, tepe rankları sıkı olarak artar. ■

Lemma 21.6

Bir tepenin rankı en fazla $n - 1$ olabilir.

Kanıt Her tepenin rankı 0 dan başlar ve yalnızca LINK işlemlerinde artar. En fazla $n - 1$ UNION işlemi olduğundan, LINK işlemi de en fazla $n - 1$ defa gerçekleştirilir. Her bir LINK işlemi ya hiçbir rankı değiştirmez ya da bir tepenin rankını 1 arttırır. Bu nedenle bir rank en fazla $n - 1$ olabilir. ■

Lemma 21.6 ranklar üzerinde zayıf bir sınır sağlıyor. Aslında, bir tepenin rankı en fazla $\lceil \lg n \rceil$ olabilir (bkz. Alıştırma 21.4-2). Ancak, bizim amaçlarımız doğrultusunda, Lemma 21.6 nın daha gevşek sınırı yeterli olacak.

Zaman sınırını kanıtlama

$O(m \alpha(n))$ zaman sınırını kanıtlamak için, amortize analizin potansiyel yöntemini (bkz. Kısım 17.3) kullanacağız. Amortize analizi gerçekleştirirken UNION işlemi yerine LINK işlemini çağırdığımızı kabul etmenin daha uygun olduğunu göreceğiz. Yani, LINK prosedürünün parametreleri iki köke işaret eden işaretçiler olduğundan, uygun FIND-SET işlemlerini ayrı ayrı uyguluyormuşuz gibi davranacağız. Aşağıdaki lemma, UNION işlemlerinin sebep olduğu ekstra FIND-SET işlemlerini saysak bile, asimptotik çalışma zamanının değişmediğini gösteriyor.

Lemma 21.7

m' adet MAKE-SET, UNION ve FIND-SET işleminden oluşan bir S' işlem dizisini, her bir UNION ı iki FIND-SET ve bir LINK ile değiştirerek m adet MAKE-SET, LINK ve FIND-SET işleminden oluşan bir s işlem dizisine dönüştürdüğümüzü kabul edelim. O halde, eğer s dizisi $O(m \alpha(n))$ zamanda çalışıyorsa, S' dizisi de $O(m' \alpha(n))$ zamanda çalışır.

Kanıt S' dizisindeki her bir UNION işlemi S dizisinde üç adet işleme dönüştürüldüğünden, elimizde $m' \leq m \leq 3m'$ var. $m = O(m')$ olduğundan, dönüştürülmüş S dizisi için bir $O(m \alpha(n))$ sınırı varsa, orijinal S' dizisi için de bir $O(m' \alpha(n))$ zaman sınırı vardır. ■

Bu kısmın geri kalanında, m' adet MAKE-SET, UNION ve FIND-SET işleminin oluşan başlangıç dizisinin m adet MAKE-SET, LINK ve FIND-SET işleminden oluşan bir işlem dizisine dönüştürüldüğünü kabul edeceğiz. Şimdi dönüştürülmüş işlem dizisi için bir $O(m \alpha(n))$ zaman sınırı ispatlayıp m' işlemlik orijinal dizi için $O(m' \alpha(n))$ çalışma zamanını kanıtlamak için Lemma 21.7 ye başvuralım.

Potansiyel fonksiyonu

Kullandığımız potansiyel fonksiyonu, ayrık küme ormanındaki her bir x tepesine q adet işlemden sonra bir $\phi_q(x)$ potansiyeli atar. Tüm ormanın potansiyeli için de tepe potansiyellerini toplarız: $\Phi_q = \sum_x \phi_q(x)$. Burada ϕ_q , ormanın q adet işlemin ardından sahip olduğu potansiyeli ifade ediyor. İlk işlem öncesinde orman boştur, ve keyfi olarak $\phi_0 = 0$ alabiliriz. Bir ϕ_q potansiyeli negatif olamaz.

$\phi_q(x)$ in değeri, x in q . işlemden sonra bir ağacın kökü olup olmamasına bağlıdır. Eğer öyle, veya $x.rank = 0$ ise, o zaman $\phi_q(x) = \alpha(n) \cdot x.rank$ olur.

Şimdi q . işlemden sonra x in bir kök olmadığını ve $x.rank \geq 1$ olduğunu kabul edelim. $\phi_q(x)$ i tanımlayabilmemiz için x üzerinde iki yardımcı fonksiyon tanımlamamız gerekiyor. Öncelikle şu fonksiyonu tanımlayalım

$$\text{level}(x) = \max \{k : x.p. \text{ rank} \geq A_k(x.rank)\}.$$

Yani, $\text{level}(x)$, x in rankına uygulanan A_k için, x in ebeveyninin rankından yüksek olmayan en büyük k seviyesidir.

$$0 \leq \text{level}(x) < \alpha(n) \quad (21.1)$$

olduğunu, aşağıda gösterildiği şekilde iddia ediyoruz. Elimizde

$$\begin{aligned} x.p. \text{ rank} &\geq x.rank + 1 && \text{(Lemma 21.4 ten)} \\ &= A_0(x.rank) && (A_0(j)) \text{ nin tanımından} \end{aligned}$$

var. Bu, $\text{level}(x) \geq 0$ anlamına geliyor. Ayrıca elimizde

$$\begin{aligned}
A_{\alpha(n)}(x.rank) &\geq A_{\alpha(n)}(1) && (A_k(j) \text{ kesin olarak arttığından}) \\
&\geq n && (\alpha(n) \text{ in tanımından}) \\
&> x.p.rank && (\text{Lemma 21.6 dan})
\end{aligned}$$

de var. Bu da, $\text{level}(x) < \alpha(n)$ anlamına geliyor. $x.p.rank$ zaman içinde monoton olarak arttığı için, $\text{level}(x)$ in de aynı şekilde artacağına dikkat edin.

İkinci yardımcı fonksiyon $x.p.rank \geq 1$ olduğunda devreye giriyor:

$$\text{iter}(x) = \max \{i : x.p.rank \geq A_{\text{level}(x)}^{(i)}(x.rank)\} .$$

Yani $\text{iter}(x)$, $A_{\text{level}(x)}$ i, en başta x in rankına uygulamak üzere, x in ebeveyninden daha büyük bir değer elde etmeden önce kaç kez iteratif olarak uyguladığımızı veriyor.

$x.rank \geq 1$ olduğunda,

$$1 \leq \text{iter}(x) \leq x.rank \quad (21.2)$$

olduğunu, aşağıda gösterildiği şekilde iddia ediyoruz. Elimizde

$$\begin{aligned}
x.p.rank &\geq A_{\text{level}(x)}(x.rank) \quad (\text{Lemma 21.4 ten}) \\
&= A_{\text{level}(x)}^{(i)}(x.rank) \quad (\text{fonksiyonel iterasyonun tanımından})
\end{aligned}$$

var. Bu $\text{iter}(x) \geq 1$ anlamına geliyor. Ayrıca elimizde

$$\begin{aligned}
A_{\text{level}(x)}^{(x.rank+1)}(x.rank) &= A_{\text{level}(x)}(x) + 1(x.rank) && (A_k(i) \text{ nin tanımından}) \\
&> x.p.rank && (\text{level}(x) \text{ in tanımından})
\end{aligned}$$

de var. Bu da, $\text{iter}(x) < x.rank$ anlamına geliyor. $x.p.rank$ zaman içinde monoton olarak arttığından, $\text{iter}(x)$ in azalması için, $\text{level}(x)$ artmalıdır. $\text{level}(x)$ değişmediği sürece, $\text{iter}(x)$ ya azalmalı ya da değişmemelidir.

Bu yardımcı fonksiyonlar ile birlikte, q işleminden sonra x tepesinin potansiyelini tanımlamaya hazırız:

$$\phi_q(x) = \begin{cases} \alpha(n) \cdot x.rank & x \text{ bir kök veya } x.rank = 0 \text{ ise,} \\ (\alpha(n) - \text{level}(x)) \cdot x.rank - \text{iter}(x) & x \text{ bir kök değil ve } x.rank \geq 1 \text{ ise} \end{cases}$$

Artık tepe potansiyellerinin bazı kullanışlı özelliklerini inceleyebiliriz.

Lemma 21.8

Tüm x tepeleri ve q işlem sayıları için aşağıdaki ifade doğrudur;

$$0 \leq \phi_q(x) \leq \alpha(n) \cdot x.rank.$$

Kanıt x bir kök veya $x.rank = 0$ ise, o halde tanımdan $\phi_q(x) = \alpha(n) \cdot x.rank$ tır. Şimdi, x in bir kök olmadığını ve $x.rank \geq 1$ olduğunu kabul edelim. $level(x)$ ve $iter(x)$ i maksimize ederek $\phi_q(x)$ üzerinde bir alt sınır elde edebiliriz. (21.1) sınırından $level(x) \leq \alpha(n) - 1$ ve (21.2) sınırından $iter(x) \leq x.rank$ elimizde var. Dolayısıyla,

$$\begin{aligned}\phi_q(x) &= (\alpha(n) - level(x)) \cdot x.rank - iter(x) \\ &\geq (\alpha(n) - (\alpha(n) - 1)) \cdot x.rank - x.rank \\ &= x.rank - x.rank \\ &= 0.\end{aligned}$$

Benzer şekilde, $level(x)$ ve $iter(x)$ i minimize ederek $\phi_q(x)$ üzerinde bir üst sınır elde edebiliriz. (21.1) sınırından $level(x) \geq 0$ ve (21.1) sınırından $iter(x) \geq 1$ elimizde var. Dolayısıyla,

$$\begin{aligned}\phi_q(x) &\leq (\alpha(n) - 0) \cdot x.rank - 1 \\ &= \alpha(n) \cdot x.rank - 1 \\ &< \alpha(n) \cdot x.rank\end{aligned}$$

■

Sonuç 21.9

Eğer x tepesi bir kök değil ve $x.rank > 0$ ise, $\phi_q(x) < \alpha(n) \cdot x.rank$ tır. ■

Potansiyel değişimleri ve işlemlerin amortize maliyetleri

Artık ayrık küme işlemlerinin tepe potansiyellerini nasıl etkilediğini incelemek için hazırız. Potansiyeldeki değişimin her bir işlemle ilişkisini anlayarak işlemle-rim amortize maliyetini belirleyebiliriz.

Lemma 21.10

x in kök olmayan bir tepe olduğunu ve q . işlemin LINK ya da FIND-SET işlemlerinden biri olduğunu kabul edelim. O halde q . işlemin ardından $\phi_q(x) \leq \phi_{q-1}(x)$ tir. Hatta eğer $x.rank \geq 1$ ise ve $level(x)$ ya da $iter(x)$ q . işlemden dolayı değişiyorsa, o zaman $\phi_q(x) \leq \phi_{q-1}(x) - 1$ olur. Yani, x in potansiyeli artmaz ve eğer pozitif bir rankı varsa ve $level(x)$ ya da $iter(x)$ değişiyorsa, o zaman x in potansiyeli en az 1 azalır.

Kanıt x bir kök olmadığı, x işlem $x.rank$ ı değiştirmedeği ve n başlangıçtaki n adet MAKE-SET işleminden sonra değişmediği için, $\alpha(n)$ de değişmeden kalır. Bu sebeple, x in potansiyeli için yazılan formülün bileşenleri q . işlemin ardından aynı kalır. Eğer $x.rank = 0$ ise, $\phi_q(x) = \phi_{q-1}(x) = 0$ dir. Şimdi $x.rank \geq 1$ olduğunu kabul edelim.

$level(x)$ in zaman içerisinde monoton olarak arttığını hatırlayın. Eğer q . işlem $level(x)$ i değiştirmezse, $iter(x)$ ya artar ya da değişmez. Eğer hem $level(x)$ hem de

$\text{iter}(x)$ değişmezse, o zaman $\phi_q(x) = \phi_{q-1}(x)$ dir. Eğer $\text{level}(x)$ değişmez ve $\text{iter}(x)$ artarsa, en az 1 artacaktır, ve bu yüzden $\phi_q(x) \leq \phi_{q-1}(x) - 1$ olur.

Son olarak, eğer q . işlem $\text{level}(x)$ i arttırırsa, en az 1 arttırır, öyle ki $(\alpha(n) - \text{level}(x)) \cdot x.\text{rank}$ ifadesi en az $x.\text{rank}$ kadar azalır. $\text{level}(x)$ arttığı için, $\text{iter}(x)$ in değeri düşebilir, ancak sınır (21.2) ye göre bu düşüş en fazla $x.\text{rank} - 1$ kadardır. Bu yüzden, potansiyelde $\text{iter}(x)$ yüzünden gerçekleşen artış, $\text{level}(x)$ yüzünden yaşanan düşüşten azdır ve sonuç olarak $\phi_q(x) \leq \phi_{q-1}(x) - 1$ dir. ■

Son üç lemmamız her bir MAKE-SET, LINK ve FIND-SET işleminin amortize maliyetinin $O(\alpha(n))$ olduğunu gösteriyor. Eşitlik (17.2) den, her bir işlemin amortize maliyetinin, asıl maliyet artı işlemde kaynaklanan potansiyel artış olduğunu hatırlayın.

Lemma 21.11

Her bir MAKE-SET işleminin amortize maliyeti $O(1)$ dir.

Kanıt q . işlemin MAKE-SET(x) olduğunu varsayalım. Bu işlem, rankı 0 olan x tepesini yaratır, öyle ki $\phi_q(x) = 0$ dir. Başka bir rank veya potansiyel değişmez ve dolayısıyla $\phi_q(x) = \phi_{q-1}(x)$ dir. MAKE-SET işleminin asıl maliyetinin $O(1)$ olduğunun belirtilmesiyle de kanıt tamamlanır. ■

Lemma 21.12

Her bir LINK işleminin amortize maliyeti $O(1)$ dir.

Kanıt q . işlemin LINK(x, y) olduğunu varsayalım. LINK işleminin asıl maliyeti $O(1)$ dir. Genellikten ödün vermeden, LINK işleminin y yi x in ebeveyni yaptığını düşünelim.

Potansiyeldeki LINK nedeniyle gerçekleşen değişimi belirlemek için, potansiyeli değişebilecek tepelerin yalnızca x, y ve y nin işlemde hemen önceki çocukları olduğunu belirtelim. LINK işlemi yüzünden potansiyeli düşebilecek tek tepenin y olduğunu, ve bu düşüşün en fazla $\alpha(n)$ kadar olduğunu göstereceğiz:

- Lemma 21.10 dan, LINK dan hemen önce y nin çocuğu olan hiçbir tepenin potansiyeli LINK sebebiyle artamaz.
- $\phi_q(x)$ in tanımından görüyoruz ki, x q işlemde hemen önce bir kök olduğundan, $\phi_q(x) = \alpha(n) \cdot x.\text{rank}$ tır. Eğer $x.\text{rank} = 0$ ise, o zaman $\phi_q(x) \leq \phi_{q-1}(x) = 0$ dir. Aksi halde,

$$\begin{aligned} \phi_q(x) &\leq \alpha(n) \cdot x.\text{rank} \quad (\text{Sonuç 21.9 dan}) \\ &= \phi_{q-1}(x), \end{aligned}$$

ve dolayısıyla x in potansiyeli azalır.

- LINK dan önce y bir kök olduğu için, $\phi_{q-1}(y) = \alpha(n) \cdot y.rank$ tır. LINK işlemi y yi kök olarak bırakır ve ya y nin rankını değiştirmez ya da y nin rankını 1 arttırır. O halde, ya $\phi_q(y) = \phi_{q-1}(y)$ ya da $\phi_q(y) = \phi_{q-1}(y) + \alpha(n)$ dir.

Böylece LINK işlemi sebebiyle potansiyelde gerçekleşen artış en fazla $\alpha(n)$ olur. LINK işleminin amortize maliyeti $O(1) + \alpha(n) = O(\alpha(n))$ dir. ■

Lemma 21.13

Her bir FIND-SET işleminin amortize maliyeti $O(\alpha(n))$ dir.

Kanıt q . işlemin FIND-SET olduğunu ve bulma yolunun s tepe içerdiğini varsayalım. LINK işleminin asıl maliyeti $O(s)$ dir. Hiçbir tepenin potansiyelinin FIND-SET sebebiyle artmadığını ve bulma yolu üzerindeki en az $\max(0, s - (\alpha(n)) + 2)$ tepenin potansiyelinin en az 1 azaldığını göstereceğiz.

Hiçbir tepenin potansiyelinin artmadığını görmek için, öncelikle kökten farklı tüm tepeler için Lemma 21.10 a başvuruyoruz. Eğer x kök ise, potansiyeli $\alpha(n) \cdot x.rank$ tır, ki bu da değişmez.

Şimdi, en az $\max(0, s - (\alpha(n)) + 2)$ adet tepenin potansiyellerinin 1 azaldığını gösterelim. x , yol bulma üzerinde, $x.rank > 0$ olmak üzere bir tepe olsun, ve x , yolun bir yerinde kök olmayan başka bir y tepesinin ardından gelsin, öyle ki FIND-SET işleminden hemen önce $level(x) = level(y)$ olsun. (y tepesi x in hemen ardından gelmek zorunda değil.) Yol bulma üzerinde tüm tepeler değil, en fazla $\alpha(n) + 2$ tepe x üzerindeki bu kısıtları sağlıyor. Kısıtları sağlamayanlar, bulma yolu üzerindeki ilk tepe (eğer rankı 0 ise), yoldaki son tepe (yani kök), ve her $k = 1, 2, \dots, \alpha(n) - 1$ değeri için $level(w) = k$ olmak üzere yoldaki son w tepesidir.

Böyle bir x tepesi seçelim ve x in potansiyelinin en az 1 azaldığını gösterelim. $k = level(x) = level(y)$ olsun. FIND-SET ten dolayı gerçekleşen yol sıkıştırma işleminden hemen önce elimizde

$$\begin{aligned} x.p.rank &\geq A_k^{(iter(x))}(x.rank) && (\text{iter}(x) \text{ in tanımından}), \\ y.p.rank &\geq A_k(y.rank) && (\text{level}(y) \text{ nin tanımından}), \\ y.rank &\geq x.p.rank && (\text{Sonuç 21.5 ten ve } y \text{ yol bulmada} \\ &&& x \text{ in ardından geldiğinden}) \end{aligned}$$

olur. Bu eşitsizlikleri bir araya getirdiğimizde ve i iter(x) in yol sıkıştırmadan önceki değeri olmak üzere elimizde

$$\begin{aligned} y.p.rank &\geq A_k(y.rank) \\ &\geq A_k(x.p.rank) && (A_k(j) \text{ kesin olarak arttığından}) \\ &\geq A_k(A_k^{(iter(x))}(x.rank)) \\ &= A_k^{(i+1)}(x.rank) \end{aligned}$$

olur. Yol sıkıştırma x ve y nin aynı ebeveyne sahip olmasını sağlayacağı için, yol sıkıştırmanın ardından $x.p.rank = y.p.rank$ olduğunu ve yol sıkıştırmanın $y.p.rank$ ı azaltmayacağını biliyoruz. $x.rank$ değişmediğinden, yol sıkıştırmanın ardından $x.p.rank \geq A_k^{(i+1)}(x.rank)$ olur. Dolayısıyla, yol sıkıştırma ya $iter(x)$ in artmasına (en az $i + 1$ 'e) ya da $level(x)$ 'in artmasına ($iter(x)$ en az $x.rank + 1$ e çıkarsa) yol açacak. İki durumda da, Lemma 21.10'dan, elimizde $\phi_q(x) \leq \phi_{q-1}(x) - 1$ var. Bu yüzden, x in potansiyeli en az 1 düşer.

FIND-SET işleminin amortize maliyeti, asıl maliyet artı potansiyeldeki değişimdir. Asıl maliyeti $O(s)$ tir ve potansiyelin en az $\max(0, s - (\alpha(n) + 2))$ kadar azaldığını gösterdik. Neticede, potansiyel birimlerini $O(s)$ te saklı olan sabitlere baskın gelecek şekilde ölçekleyebileceğimizden, amortize maliyet en fazla $O(s) - (s - (\alpha(n) + 2)) = O(s) - s + O(\alpha(n)) = O(\alpha(n))$ dir. ■

Önceki lemmaları bir araya getirdiğimizde aşağıdaki teorem ortaya çıkıyor.

Teorem 21.14

n tanesi MAKE-SET işlemi olan m adet MAKE-SET, UNION ve FIND-SET işleminin oluşan bir işlem dizisi, rank ile birleşim ve yol sıkıştırma ile bir ayrık küme ormanında, en kötü durumda $O(m \alpha(n))$ zamanda gerçekleştirilebilir.

Kanıt Lemma 21.7, 21.11, 21.12 ve 21.13 ten ispat açıktır. ■

Alıştırmalar

21.4-1

Lemma 21.4 ü kanıtlayın.

21.4-2

Bir tepenin rankının en fazla $\lfloor \lg n \rfloor$ olduğunu kanıtlayın.

21.4-3

Alıştırma 21.4-2 nin ışığında, her bir x tepesinde $x.rank$ ı saklamak için kaç bit gereklidir?

21.4-4

Alıştırma 21.4-2 yi kullanarak, rank ile birleşim kullanılan ama yol sıkıştırma kullanılmayan bir ayrık küme ormanında işlemlerin $O(m \lg n)$ zamanda çalıştığına dair basit bir ispat verin.

21.4-5

Profesör Dante, tepe rankları, köke giden basit bir yol boyunca kesin olarak arttığı için, tepe seviyelerinin yol boyunca monoton şekilde artması gerektiği sonucu-

na varmaktadır. Diğer bir deyişle, eğer $x.rank > 0$ ve $x.p$ bir kök değil ise, o halde $level(x) \leq level(x.p)$ 'dir. Profesör haklı mı?

21.4-6 *

$\alpha'(n)$ min $\{k : A_k(1) \geq \lg(n+1)\}$ fonksiyonunu düşünelim. n in pratikteki tüm değerleri için $\alpha'(n) \leq 3$ olduğunu, ve Alıştırma 21.4-2 yi kullanarak n tanesi MAKE-SET işlemi olan m adet MAKE-SET, UNION ve FIND-SET işleminden oluşan bir işlem dizisini, rank ile birleşim ve yol sıkıştırma ile uygulanan bir ayrık küme ormanı üzerinde, en kötü durumda $O(m \alpha'(n))$ zamanda yapacak şekilde potansiyel fonksiyonu argümanını nasıl değiştirebileceğimizi gösterin.

Problemler

21-1 Çevrimdışı minimum

Çevrimdışı minimum problemi bizden, INSERT ve EXTRACT-MIN işlemleri altında $\{1, 2, \dots, n\}$ tanım kümesinin elemanlarından oluşan bir T dinamik kümesi sağlamamızı ister. $\{1, 2, \dots, n\}$ deki her bir anahtar tam olarak bir kez eklenmek üzere, n adet INSERT ve m adet EXTRACT-MIN çağrısından oluşan bir S dizisi verilsin. Her EXTRACT-MIN çağrısında hangi anahtarın döndürüldüğünü belirlemek istiyoruz. Özelde, $i = 1, 2, \dots, m$ için çıkarılan $[i]$, i . EXTRACT-MIN çağrısının döndürdüğü anahtar olacak şekilde *çıkarılan* $[1..m]$ dizisini doldurmak istiyoruz. Problem, döndürülen anahtarlardan herhangi birini belirlemeden önce S dizisi üzerinde işlem yapmamıza izin verildiği için, “çevrimdışıdır”.

a. Çevrimdışı minimum probleminin aşağıdaki örneğinde, her $INSERT(i)$ işlemi i nin değeriyle ve her EXTRACT-MIN işlemi de E harfiyle gösterilmiştir:

4, 8, E, 3, E, 9, 2, 6, E, E, E, 1, 7, E, 5 .

çıkarılan dizisinin elemanlarını doğru biçimde doldurun.

Bu problem için bir algoritma geliştirirken, S dizisini homojen altdizilere parçalıyoruz. Yani, S i şu şekilde temsil ediyoruz:

$I_1, E, I_2, E, I_3, \dots, I_m, E, I_{m+1}$

burada her E tek bir EXTRACT-MIN çağrısını ve her I_j (muhtemelen boş olan) bir INSERT çağrılar dizisini temsil ediyor. Her I_j altdizisi için, öncelikle bir K_j kümesine, bu işlemler tarafından eklenen anahtarları yerleştiriyoruz; ki I_j boşsa K_j de boştur. Daha sonra şunu yapıyoruz:

OFF-LINE-MINIMUM (m, n)

```

1  for  $i = 1$  to  $n$ 
2       $i \in K_j$  olacak şekilde  $j$  belirle
3      if  $j \neq m + 1$ 
4           $\text{çıkarılan}[j] = i$ 
5          öyle bir  $K_l$  kümesi olsun ki  $l$  değeri  $j$  den
              büyük olan en küçük değer olsun
6           $K_l = K_j \cup K_l, K_j$  yi yok et
7  return  $\text{çıkarılan}$ 

```

- b. OFF-LINE-MINIMUM tarafından döndürülen çıkarılan dizisinin doğruluğunu tartışın.
- c. OFF-LINE-MINIMUM u bir ayırık küme veri yapısıyla nasıl etkin biçimde uygulayabileceğimizi açıklayın. Uygulamanızın en kötü durumda çalışma zamanı için sıkı bir sınır verin.

21-2 Derinliğine karar

Derinliğine karar probleminde, köklü ağaçların bir $\mathcal{F} = \{T_i\}$ ormanını üç işlem altında sağlıyoruz:

MAKE-TREE (v) yegâne tepesi v olan bir ağaç yaratır.

FIND-DEPTH (v) v nin, bulunduğu ağaç içerisindeki derinliğini döndürür.

GRAFT(r, v) bir ağacın kökü olduğu kabul edilen r tepesini, r den farklı bir ağaçta bulunduğu kabul edilen ancak kök olması gerekmeyen bir v tepesinin çocuğu yapar.

- a. Ayırık küme ormanına benzer bir ağaç temsili kullandığımızı varsayalım: $v.p$ v nin ebeveyn tepesidir; yalnızca v kök ise $v.p = v$ olur. Ayrıca, GRAFT(r, v) yi $r.p = v$ atamasını yaparak ve FIND-DEPTH(v) yi köke çıkan yolu takip edip v hariç karşılaşılan tüm tepelerin sayısını döndürerek uyguladığımızı düşünelim. m adet MAKE-TREE, FIND-DEPTH ve GRAFT işleminden oluşan bir işlem dizisi için en kötü durumda çalışma zamanının $\Theta(m^2)$ olduğunu gösterin.

Rank ile birleşim ve yol sıkıştırma sezgisellerini kullanarak, en kötü durumda çalışma zamanını düşürebiliriz. Her bir S_i kümesinin (o da bir ağaçtır) $\mathcal{F}$ ormanındaki bir T_i ağacına karşılık geldiği bir $\mathcal{S} = \{S_i\}$ ayırık küme ormanı kullanırız. Öte yandan, bir S_i kümesi içindeki ağaç yapısı T_i ağacının yapısıyla aynı olmak durumunda değildir. Aslında, S_i uygulamasında ebeveyn-çocuk ilişkisinin kaydı tam olarak tutulmaz, ama yine de T_i içerisindeki herhangi bir tepenin derinliğini ölçmemize olanak sağlar.

Kilit nokta, her v tepesi içerisinde bir $v.d$ “sözde-uzaklık” değeri saklamaktır. Sözde-uzaklık, v den bulunduğu S_i kümesinin köküne giden yoldaki sözde-uzaklıkların toplamı, v nin T_i deki derinliğine eşit olacak şekilde tanımlanır. Yani, v den S_i deki köküne giden basit yol, $v_0 = v$ ve v_k kök olmak üzere $v_0, v_1, \dots, v_k$ ise, o zaman v ’nin T_i deki derinliği $\sum_{j=0}^k v_j.d$ dir.

- b.** MAKE-TREE nin bir uygulamasını oluşturun.
- c.** FIND-DEPTH i uygulamak için FIND-SET i nasıl düzenlemek gerektiğini gösterin. Uygulamanız yol sıkıştırmayı kullanmalı, ve çalışma zamanı, bulma yolunun uzunluğuyla doğrusal olmalıdır. Uygulamanızın sözde-uzaklıkları doğru şekilde güncellediğinden emin olun.
- d.** r ve v yi içeren kümeleri birleştiren GRAFT(r, v) yi, UNION ve LINK işlemlerini düzenleyerek nasıl uygulayabileceğimizi gösterin. Uygulamanızın sözde-uzaklıkları doğru şekilde güncellediğinden emin olun. S_i kümesinin kökünün T_i ağacının köküne karşılık gelmeyebileceğini hesaba katın.
- e.** n tanesi MAKE-TREE işlemi olan m adet MAKE-TREE, FIND-DEPTH ve GRAFT işleminden oluşan bir işlem dizisinin en kötü durumda çalışma zamanı için sıkı bir sınır verin.

21-3 Tarjan’ın çevrimdışı en küçük-ortak-atalar algoritması

Bir T köklü ağacındaki iki u ve v tepesinin **en küçük ortak atası**, hem u hem de v nin atası olan ve T içerisinde en büyük derinliğe sahip bir w tepesidir. **Çevrimdışı en küçük ortak atalar** probleminde (*off-line least-common-ancestors problem*), bize bir T köklü ağacı ve T deki sırasız tepe çiftlerinden seçilen keyfi bir $P = \{\{u, v\}\}$ kümesi veriliyor, ve bizden P deki her çiftin en küçük ortak atasını bulmamız isteniyor.

Çevrimdışı ortak atalar problemini çözmek için aşağıdaki prosedür, başlangıçta $LCA(T.root)$ çağrısı yaprak, T ağacında bir yürüyüş gerçekleştiriyor. Yürüyüş öncesinde tüm tepelerin BEYAZ olduğunu kabul ediyoruz.

LCA(u)

- 1 MAKE-SET(u)
- 2 FIND-SET(u).ancestor = u
- 3 **for** u nun T deki her bir v çocuğu için
- 4 LCA(v)
- 5 UNION(u, v)
- 6 FIND-SET(u).ancestor = u
- 7 $u.color = \text{SİYAH}$
- 8 **for** $\{u, v\} \in P$ olmak üzere her v tepesi
- 9 **if** $v.color = \text{SİYAH}$
- 10 yazdır: u “ve” v “nin en küçük ortak atası” FIND-SET(v).ancestor dır.”

- a. Satır 10 un, her bir $\{u, v\} \in P$ çifti için yalnızca bir kere çalışacağını tartışın.
- b. $LCA(u)$ çağrısı sırasında, ayrık küme veri yapısındaki kümelerin sayısının T deki u tepesinin derinliğine eşit olduğunu tartışın.
- c. LCA nın, her bir $\{u, v\} \in P$ çifti için, doğru en küçük ortak atayı yazdıracağını ispatlayın.
- d. $LCAA$ nın çalışma zamanını Kısım 21.3 teki ayrık veri yapısı uygulamasını kullandığımızı kabul ederek analiz edin.

Bölüm notları

Ayrık küme veri yapısının önemli neticelerinin çoğu, en azından kısmen R. E. Tarjan sayesinde. Tarjan [328, 330] toparlama analizi kullanarak, Ackermann'ın fonksiyonunun çok yavaş büyüyen tersi $\hat{\alpha}(m, n)$ için ilk sıkı üst sınırı verdi. (Kısım 21.4 te verilen $A_k(j)$ fonksiyonu Ackermann'ın fonksiyonunun bir benzeridir, ve $\alpha(n)$ de tersinin benzeridir. Hem $\alpha(n)$ hem de $\hat{\alpha}(m, n)$ m ve n in tüm makul değerleri için en çok 4 değerini alır.) Bir $O(m \lg^* n)$ üst sınırı Hopcroft ve Ullman [5, 179] tarafından daha önce ispatlanmıştı. Kısım 21.4 teki yaklaşım, Kozen'in [220] bir analizine dayanan, Tarjan'ın [332] daha sonraki bir analizinden uyarlanmıştır. Harfst ve Reingold [161], Tarjan'ın önceki sınırının potansiyel tabanlı bir versiyonunu verir.

Tarjan ve van Leeuwen [333], bazı durumlarda iki-geçişli yöntemlerden daha iyi bir sabit çarpan veren “tek-geçişli yöntemler” de dahil olmak üzere, yol sıkıştırma sezgisellerinin varyantlarını tartışırlar. Tarjan'ın önceki temel yol sıkıştırma sezgiseli analizlerinde olduğu gibi, Tarjan ve van Leeuwen'in analizleri de toplam analizidir. Harfst ve Reingold [161] daha sonra potansiyel fonksiyonunda küçük bir değişiklik yaparak tek-geçişli varyantları için yol sıkıştırma analizlerini uyarladılar. Gabow ve Tarjan [121] bazı uygulamalarda, ayrık küme işlemlerinin $O(m)$ zamanda çalıştırılabileceğini gösterirler.

Tarjan [329], belirli teknik şartları sağlayan herhangi bir ayrık küme veri yapısı üzerindeki işlemlerin zamanı için, bir $\Omega(m \hat{\alpha}(m, n))$ alt sınırı olduğunu gösterdi. Bu alt sınır daha sonra, en kötü durumda $\Omega(m \hat{\alpha}(m, n))$ adet $(\lg n)$ -bit sözlük belleğe erişilmesi gerektiğini gösteren Fredman ve Saks [113] tarafından genelleştirildi.



VI Çizge Algoritmaları

Giriş

Çizge problemleri bilgisayar bilimlerinde yaygındır ve onlar için çalışan algoritmalar bu alan için temeldir. Yüzlerce ilginç hesaplamalı problem çizgelerde ifade edilir. Bu bölümde biz en önemli birkaç tanesine değineceğiz.

Bölüm 22 bir bilgisayarda bir çizgeyi nasıl ifade edeceğimizi gösterir ve bir çizge aramasına dayalı enine arama yada derinliğine aramalardan birini kullanarak algoritmalar tartışılır. Bölüm derinliğine aramanın iki uygulamasını verir: topolojik sıralanmış yönlü çevrimsiz bir çizge ve yönlü bir çizgesi güçlü bağlantılı bileşenlerine ayırmak.

Bölüm 23 bir çizgenin minimum-ağırlıklı kapsayan ağacını nasıl hesaplayacağımızı ifade eder: her ayrıta bir ağırlık karşılık geldiğinde tüm tepeleri en az-ağırlıklı yolla birleştirebilmek. Minimum kapsayan ağaçları hesaplamak için olan algoritmalar, açgözlü algoritmaların iyi örnekleri olarak görev yapar (Bölüm 16 ya bakınız).

Bölüm 24 ve 25 de her ayrıta bir uzunluk veya "ağırlık" karşılık geldiğinde tepeler arasındaki en kısa yolların nasıl hesaplanacağı incelenecektir. Bölüm 24 verilen bir kaynak tepeden diğer tüm tepelere olan en kısa yolların nasıl bulunacağını gösterir ve Bölüm 25 her tepe çifti arasındaki en kısa yolu hesaplayan yöntemleri inceler.

Son olarak, Bölüm 26 bir akış ağında, yönlü bir çizge, malzemenin belirlenmiş bir kaynak tepesine, belirlenmiş bir çıkış düğümlü tepe noktasına ve malzemenin miktarına karşılık gelen her yönlü ayrıttan iki yönde de geçebilen belirlenmiş kapasitelere sahip olduğunda malzemenin maksimum akışının nasıl hesaplanacağını gösterir. Bu genel problem pek çok biçimde ortaya çıkar ve maksimum akışı hesaplayan iyi bir algoritma ilgili çeşitli problemleri etkin bir şekilde çözmeye yardımcı olur.

Verilen bir $G = (V, E)$ çizgesi üzerindeki bir çizge algoritmasının çalışma zamanını tanımladığımızda genellikle çizgenin $|V|$ tepe sayısı ve $|E|$ ayrıt sayısı cinsinden girdinin büyüklüğünü ölçeriz. Yani, girdinin boyutunu sadece bir tane ile değil, iki parametre ile tanımlarız. Bu parametreler için uzlaşmış ortak bir notasyon belirleriz. Asimptotik notasyonda (O -notasyonu ya da Θ -notasyonu gibi) ve bu notasyonun sadece içinde, V sembolü $|V|$ ile gösterilir ve E sembolü $|E|$ ile gösterilir. Örneğin “algoritma $O(VE)$ zamanda çalışır” dediğimizde algoritmanın $O(|V||E|)$ zamanda çalıştığı anlamına gelir. Bu seçim belirsizlik riski olmadan, çalışma zamanı formüllerini okumayı daha da kolaylaştırır.

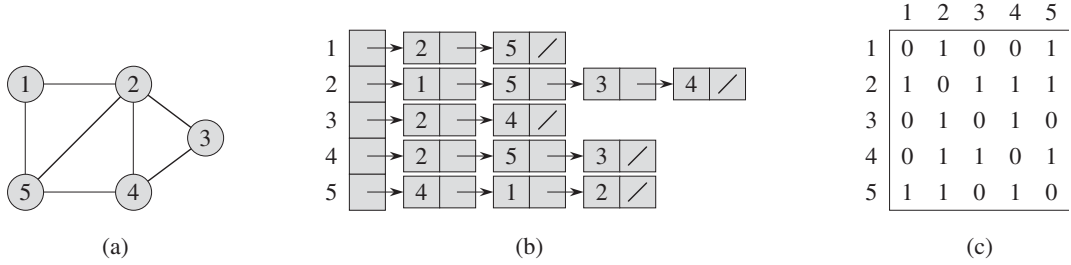
Benimsediğimiz bir diğer seçim kabakodda görünür. G çizgesinin tepe kümesini $G.V$ ve ayrıt kümesini $G.E$ ile gösteririz. Yani kabakod tepe ve ayrıt kümelerini bir çizgenin niteliği olarak görür.

Bu bölümde bir çizgenin gösterimi ve bir çizgedeki aramalar için gerekli olan yöntemler sunulacaktır. Bir çizgedeki aramalar, çizgenin ayrıtlarını sistematik bir şekilde takip ederek çizgenin tepelerini gezme anlamına gelir. Bir çizge-arama algoritması çizgenin yapısı hakkında bilgiler elde edilebilir. Birçok algoritma bu yapısal bilgileri elde etmek için çizge girdilerini araştırmayla işe başlar. Diğer bazı çizge algoritmaları çizge çizge aramalarının detaylarına girerler. Bir çizgedeki arama teknikleri, çizge algoritmaları alanın en temel yerindedir.

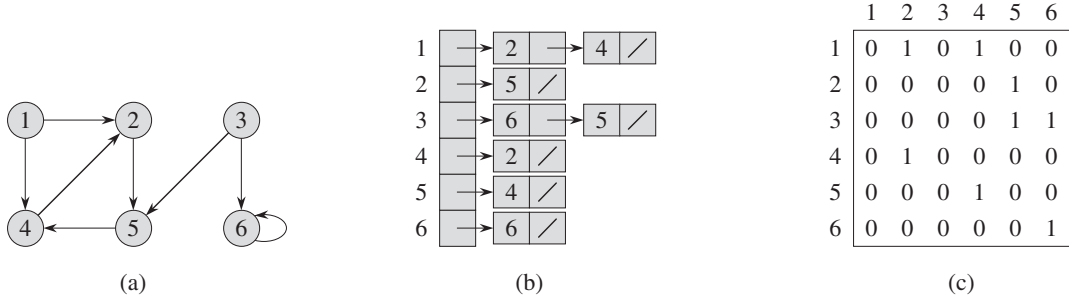
Kısım 22.1 çizgelerin en yaygın iki hesaplamalı gösterimini anlatır: bitişiklik listeleri ve bitişiklik matrisleri. Kısım 22.2 enine arama olarak adlandırılan basit bir çizge-arama algoritması sunar ve enine bir ağacın nasıl oluşturulacağını gösterir. Kısım 22.3 derinliğine aramayı sunar ve derinliğine arama ile tepeleri sıralı olarak ziyaret etmek için bazı standart sonuçları ispatlar. Kısım 22.4 derinliğine aramaya ait bizim ilk gerçek uygulamamızı sunar: çevre içermeyen yönlü çizgelerde topolojik sıralama. Derinliğine aramanın ikinci bir uygulaması, Kısım 22.5 in bir konusu olan, yönlü bir çizgenin güçlü bağlantılı bileşenlerinin bulunmasıdır.

22.1 Çizgelerin gösterimi

Bir $G = (V, E)$ çizgesinin gösterimi için standart iki yoldan bir tanesini seçebiliriz: bitişiklik listelerinin bir koleksiyonu veya bir bitişiklik matrisi olarak. Her iki yol yönlü veya yönsüz çizgelerde uygulanabilir. $|E|$ nin, $|V|^2$ den çok daha az olduğu **seyrek** çizgelerin gösteriminde sıkı bir yol sunduğundan, genellikle yöntem olarak bitişiklik-listeleri gösterimi tercih edilir. Bu kitaptaki çizge algoritmalarının çoğu çizge girdisi olarak bitişiklik-listesi formunu sunar. Ancak çizge **yoğun** - $|E|$, $|V|^2$ ye yakın- olduğunda veya verilen iki tepeyi birleştiren bir ayrıtlın olup olmadığını hızlı bir şekilde söyleyebilmek istediğimizde bitişiklik-matrisi gösterimini



Şekil 22.1 Yönsüz bir çizgenin iki gösterimi. (a) 5 tepeli ve 7 ayrıtlı bir G yönsüz çizge (b) G nin bitişiklik-listesi gösterimi (c) G nin bitişiklik-matrisi gösterimi.



Şekil 22.2 Yönlü bir çizgenin iki gösterimi (a) 6 tepeli ve 8 ayrıtlı bir G yönlü çizgesi (b) G nin bitişiklik-listesi gösterimi (c) G nin bitişiklik-matrisi gösterimi.

tercih edebiliriz. Örneğin, Bölüm 25 de verilen en kısa yol algoritmalarının iki-sinde de çizge girdileri bitişiklik matrisleri ile gösterilmiştir.

Bir $G = (V, E)$ çizgesinin **bitişiklik-listesi gösterimi**, herbiri V deki her tepe için $|V|$ adet listenin bir Adj dizisinden oluşur. Her $u \in V$ için $Adj[u]$ bitişiklik-listesi bir $(u, v) \in E$ ayrıtı var olacak şekilde tüm v tepelerini içerir. Yani, $Adj[u]$, G çizgesinde u tepesine komşu olan tüm tepeleri içerir (Alternatif olarak bu tepelere bir işaret vermiş olur). Bitişiklik listeleri bir çizgedeki ayrıtları gösterdiğinden, kabakod içinde Adj dizisi çizgenin bir niteliği gibi tanımlanır, tıpkı ayrıt kümesini E ile tanımladığımız gibi. Bu durumda, kabakod içinde G $Adj[u]$ şeklinde bir gösterim göreceğiz. Şekil 22.1(b), Şekil 22.1(a) daki yönsüz çizgenin bir bitişiklik-listesi gösterimidir. Benzer şekilde, Şekil 22.2(b), Şekil 22.2(a) daki yönlü çizgenin bir bitişiklik-listesi gösterimidir.

Eğer, G yönlü bir çizge ise, tüm bitişiklik listelerinin uzunlukları toplamı $|E|$ dir, çünkü (u, v) formundaki bir ayrıt, $Adj[u]$ içindeki v nin görünmesi halinde gösterilir. Eğer G yönsüz bir çizge ise, tüm bitişiklik listelerinin uzunlukları top-

lamı $2|E|$ dir, çünkü eğer (u, v) yönsüz bir ayrıtsa u, v nin bitişiklik listesinde, v de u nun bitişiklik listesinde vardır. Yönlü ve yönsüz çizgelerin her ikisi için bitişiklik-listesi gösterimi, $\Theta(V + E)$ bellek miktarına gerek duyduğundan dolayı cazip bir özelliğe sahiptir.

Bitişiklik listelerini, **ağırlıklı çizgelerin** gösterimlerine kolaylıkla uyarlayabiliriz; her ayrıta ilişkilendirilmiş **ağırlıkların** olduğu çizgelerde bir **ağırlık fonksiyonu** $w : E \rightarrow \mathbb{R}$ ile verilir. Örneğin, $G = (V, E)$, w ağırlık fonksiyonlu ağırlıklı bir çizge olsun. $(u, v) \in E$ ayrıtının $w(u, v)$ ağırlığı, u nun bitişiklik listesindeki v tepesi ile saklanır. Bitişiklik listesi gösterimi oldukça güçlüdür, bu nedenle diğer birçok çizge değişkenlerini desteklemek için bu gösterimde değişiklik yapabiliriz.

Bitişiklik listesinin gösterimindeki potansiyel bir dezavantaj, çizgede olan bir (u, v) ayrıtı verildiğinde $Adj[u]$ bitişiklik listesindeki v leri araştırmaktan daha hızlı bir yol yoktur. Bu dezavantaj, asimptotik olarak fazla bellek kullanımı maliyetiyle, çizgenin bitişiklik matrisi gösterimiyle kaldırılabilir. (Bitişiklik listeleri üzerinde ayrıtların daha hızlı bir şekilde aranıp bulunduğu değişimlere ait tavsiyeler için Alıştırma 22.1-8 e bakınız.)

Bir $G = (V, E)$ çizgesinin **bitişiklik-matrisi gösterimi** için tepelerin keyfi olarak $1, 2, \dots, |V|$ olarak numaralandırıldığını kabul edelim. Bir G çizgesinin bitişiklik matrisi gösterimi $|V| \times |V|$ boyutunda bir $A = (a_{ij})$ matrisinden oluşur, öyle ki:

$$a_{ij} = \begin{cases} 1 & (i, j) \in E \text{ ise,} \\ 0 & \text{aksi halde.} \end{cases}$$

Şekil 22.1(c) ve Şekil 22.2(c) sırasıyla Şekil 22.1(a) ve Şekil 22.2(a) daki yönsüz ve yönlü çizgelerin bitişiklik matrisleridir. Bir çizgenin bitişiklik matrisi, çizgedeki ayrıt sayısından bağımsız olarak $\Theta(V^2)$ bellek gerektirir.

Şekil 22.1(c) deki bitişiklik matrisinin ana köşegeni boyunca simetrisini ele alalım. Bir çizgede (u, v) ile (v, u) aynı ayrıtı gösterdiğinden yönsüz bir A matrisinin transpozesi kendine eşittir: $A = A^T$. Bazı uygulamalarda sadece üst girişler için yer ayrılır ve bitişiklik matrisinin köşegeninin üstü kesilip çizgenin yarısı saklanarak gereken hafıza yarıya düşürülür.

Bir çizgenin bitişiklik-listesi gösteriminde olduğu gibi, bir bitişiklik matrisi de ağırlıklı bir çizgesi temsil edebilir. Örneğin, eğer $G = (V, E)$ çizgesi, bir w ayrıt-ağırlık fonksiyonlu ağırlıklı bir çizge ise $(u, v) \in E$ ayrıtının $w(u, v)$ ağırlığı bitişiklik matrisinin u satır, v sütunundaki giriş olarak basitçe gösterilir. Eğer bir ayrıt yoksa, ilgili matris girdisi olarak pek çok problemde 0 veya ∞ gibi değerlerin kullanılması daha uygun olmasına rağmen NIL değeri olarak da saklayabiliriz.

Bitişiklik-listesi gösteriminin, asimptotik olarak en az bitişiklik-matrisi gösterimi kadar yer verimliliği olsa da, çizgeler epey küçük olduğunda daha basit

olacağından dolayı bitişiklik matrisi tercih edilir. Dahası, ağırlıksız çizgeler için bitişiklik matrisleri bir avantaj taşırlar: onlar her girdi için sadece bir bit kullanır.

Niteliklerin gösterimi

Çizgeler üzerinde çalışan pek çok algoritma tepeler ve/veya ayrıtlar için nitelikleri sürdürmeye ihtiyaç duyar. Bu nitelikleri alıştığımız gösterimleri kullanarak gösteririz, sözgelimi bir v tepesinin d niteliği $v.d$ olarak gösterilir. Ayrıtları tepe çiftleri olarak gösterdiğimizde, benzer gösterim stilini kullanırız. Örneğin, ayrıtlar bir f niteliğine sahipse, (u, v) ayrıtı için bu niteliği $(u,v).f$ biçiminde ifade ederiz. Algoritmaların sunumunda ve anlaşılmasında, bizim nitelik gösterimimiz yeterlidir.

Gerçek programlara tepe ve ayrıt niteliklerini uygulamak tümüyle başka bir hikayedir. Tepe ve ayrıtların niteliklerini tutmak ve erişmek için bir tane en iyi yol yoktur. Verilen bir durum için kararınız kullandığınız programlama diline, uyguladığınız algoritmaya, programınızın geri kalanının çizgeyi nasıl kullandığına bağlıdır. Bir çizgeyi bitişiklik listesi ile gösteriyorsanız, bir tasarım tepe niteliklerini ilave dizilerde gösterir, örneğin $d[1..|V|]$ dizisi Adj satırına paraleldir. u ya komşu olan tepeler $Adj[u]$ içindeyse $u.d$ niteliği satırda $d[u]$ olarak saklanır. Nitelikleri uygulamanın diğer pek çok yolu mümkündür. Örneğin, nesne tabanlı programlama dilinde tepe nitelikleri Vertex sınıfının bir alt sınıfı içinde durum değişkeni olarak gösterilebilir.

Alıştırmalar

22.1-1

Verilen yönlü bir çizgede bitişiklik-listesi gösterimi ile her tepenin dış derecesini hesaplamak ne kadar sürer? İç dereceleri hesaplamak ne kadar sürer?

22.1-2

7 tepeli tam bir ikili ağaç için bitişiklik-listesi temsillerini veriniz. Bir denk bitişiklik-matris gösterimini veriniz. Tepelerin 1 den 7 ye kadar bir ikili bellek yığı-
nında numaralandığını kabul ediniz.

22.1-3

Yönlü bir $G = (V, E)$ çizgenin *transpozesi*, $E^T = \{(v, u) \in V \times V : (u, v) \in E\}$ olacak şekilde $G^T = (V, E^T)$ çizgesidir. Bundan dolayı, G^T , tersi alınmış bütün ayrıtları ile G dir. G den G^T yi hesaplamak için G nin bitişiklik-listesi ve bitişiklik-matrisi gösterimlerinin her ikisi için de, verimli algoritmalar tanımlayınız. Algoritmalarınızın çalışma zamanını analiz ediniz.

22.1-4

$G = (V, E)$ çoklu çizgesinin bitişiklik liste gösterimi verilsin, “denk” yönsüz $G' = (V, E')$ çizgesinin bitişiklik-listesi gösterimini hesaplamak için $O(V + E)$ -zaman algoritmasını tanımlayınız, öyle ki E', E deki iki tepe arasında bulunan bütün çoklu ayrıtların tek bir ayrıt ile değiştirilmesi ve bütün döngülerin atılmasıyla elde edilen ayrıtlardan oluşur.

22.1-5

Yönlü bir $G = (V, E)$ çizgesinin **karesi** $G^2 = (V, E^2)$ çizgesidir, öyle ki G ve v arasında en çok iki ayrıtlı bir yol içerir ancak ve ancak $(u, v) \in E^2$ olur. G nin bitişiklik listesi ve bitişiklik matris gösterimlerinin her ikisi için G den G^2 yi hesaplayan verimli algoritmalar tanımlayınız. Algoritmalarınızın çalışma zamanını analiz ediniz.

22.1-6

Birçok çizge algoritması bitişiklik-matris gösteriminin girdi için gereken zamanını $\Omega(V^2)$ olarak alır, fakat bazı istisnalar vardır. Eğer yönlü bir G çizgesi **evrensel çukur** — $|V| - 1$ iç dereceli ve 0 dış dereceli bir tepe—içeriyorsa, G için verilen bir bitişiklik-matrisinin $O(V)$ zamanda nasıl belirleneceğini gösteriniz.

22.1-7

Kendine döngü içermeyen yönlü bir $G = (V, E)$ çizgesinin $B = (b_{ij})$ **çakışıklık matrisi** $|V| \times |E|$ matrisidir, öyle ki;

$$b_{ij} = \begin{cases} -1, & j \text{ ayrıtı } i \text{ tepesinden çıkıyor ise,} \\ 1, & j \text{ ayrıtı } i \text{ tepesine giriyor ise,} \\ 0, & \text{aksi halde.} \end{cases}$$

B^T, B nin transpozesi olmak üzere BB^T matris çarpımının girdilerinin neyi gösterdiklerini açıklayınız.

22.1-8

Bağlı bir liste yerine, $(u, v) \in E$ iken v tepelerini içeren her dizi ögesi $Adj[u]$ nun bir çırpı tablosu olduğunu varsayalım. Eğer bütün ayrıt aramaları eşit olasılıklı ise, bir ayrıtın çizgede olup olmadığını belirlemek için beklenen zaman nedir? Bu planın sahip olduğu dezavantaj nedir? Her ayrıt listesi için bu problemleri çözen alternatif bir veri yapısı önerin. Çırpı tablosu ile karşılaştırıldığında sizin alternatifinizin dezavantajları var mıdır?

22.2 Enine arama

Enine arama, bir çizge aramasındaki en basit algoritmalarından biridir ve pek çok önemli çizge algoritmasının ilk örneğidir. Enine arama, Prim'in minimum kapsayan ağaç algoritması (Kısım 23.2) ve Dijkstra'nın tek-kaynaklı en kısa yollar algoritması (Bölüm 24.3) ile benzer fikirleri kullanır.

Verilen bir $G = (V, E)$ çizgesi ve seçilmiş bir s **kaynak** tepesi ile, enine arama s den ulaşılabilen her tepeyi “bulmak” için G nin ayrıtlarını sistematik bir şekilde araştırır. Bu algoritma s den ulaşılabilen her tepeye olan uzaklığı (en küçük ayrıt sayısını) hesaplar. Ayrıca s kaynağı ve tüm ulaşılabilen tepeleri içeren “enine ağaç” üretir. s den ulaşılabilen herhangi bir v tepesi için enine ağaçta s den v ye olan basit yol, G de s den v ye bir “en kısa yola”, yani en az sayıda ayrıt içeren bir yola karşılık gelir. Algoritma yönlü veya yönsüz çizgelerin her ikisi için de çalışır.

Algoritmaların bu şekilde isimlendirilmesinin sebebi bulunan ve bulunmayan tepeleri arasındaki sınırın tek bir şekilde enine genişlemesidir. Bu da şu demektir ki; algoritma s den başlayarak $k + 1$ uzaklıklı tepeleri gezmeden önce k uzaklıklı diğer tüm tepeleri dolaşır.

İlerleme yolunu bulmak için algoritma her tepeyi beyaz, gri veya siyah renkle boyar. Tüm tepeler beyaz ile başlar, sonradan gri ve en son siyah olur. Arama sırasında bir tepe ile ilk defa karşılaşıldığında o tepe **bulunan (discovered)** olur ve o andan itibaren tepe artık beyaz renkli değildir. Gri ve siyah tepeler bulunan tepelerdir, fakat enine ilerlemek için bunlar arasından hangisinin seçileceğine karar verilir.¹ Eğer, $(u, v) \in E$ ve u tepesi siyah ise, v tepesi de siyah veya gridir, böylece siyah tepelere bitişik olan tüm tepeler gezilir. Gri tepeler bazı beyaz tepelerle bitişik olabilir; bulunan ve bulunmayan tepeler arasındaki sınırı gösterirler.

Enine arama başlangıçta sadece s kaynak tepesi olan kökü içererek enine ağacı oluşturur. Daha önce bulunan u tepesinin bitişiklik listesi gezinirken beyaz v tepesi bulunduğunda v tepesi ve (u, v) ayrıtı ağaca eklenir. u tepesine v 'nin **öncülü (predecessor)** ya da **ebeveyni (parent)** denir. Bir tepe en fazla bir kez bulunduğundan, en fazla bir ebeveyni olur. Enine ağaçta s köküne bağlı olarak ata (ancestor) ve torun (descendant) ilişkisi şu şekilde tanımlanır: eğer ağaçta s kökünden v tepesine olan basit yolda u tepesi varsa, bu durumda u , v nin atası, v de u nun torunu olarak isimlendirilir.

¹Gri ve siyah tepeleri ayırmamız enine aramanın nasıl çalıştığını anlayabilmemize yardımcı olur. Hatta, Alıştırma 22.2-3 ün gösterdiği gibi, gri ve siyah tepeleri ayırmamak bile aynı sonucu elde ederiz.

Aşağıda yer alan BFS enine arama işlemi, girdi çizgeyi $G = (V, E)$ nin bitişiklik listesi kullanılarak gösterildiğini varsayar. Çizgede her tepeye birkaç ek özellik ilave eder. Her $u \in V$ tepesinin rengini $u.color$ özelliği ile ve u nun öncülünü (predecessor) ise $u.\pi$ özelliği içinde saklarız. Eğer u , öncül tepeye sahip değilse (örneğin eğer $u = s$ veya u henüz bulunmayan bir tepeyse) bu durumda $u.\pi = \text{NIL}$ olur. $u.d$ özelliği ile algoritma tarafından hesaplanan s kaynağından u tepesine olan uzaklık tutulur. Gri tepelerin kümesini kontrol etmek için algoritma ayrıca ilk-giriş, ilk-çıkış kuyruğu Q yu (Bölüm 10.1 e bakınız) kullanır.

```

BFS(G,s)
1  for her  $u \in G.V - \{s\}$  tepesi
2       $u.color = \text{WHITE}$ 
3       $u.d = \infty$ 
4       $u.\pi = \text{NIL}$ 
5   $s.color = \text{GRAY}$ 
6   $s.d = 0$ 
7   $s.\pi = \text{NIL}$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11      $u = \text{DEQUEUE}(Q)$ 
12     for her  $v \in G.Adj[u]$ 
13         if  $v.color == \text{WHITE}$ 
14              $v.color = \text{GRAY}$ 
15              $v.d = u.d + 1$ 
16              $v.\pi = u$ 
17             ENQUEUE( $Q, v$ )
18      $u.color = \text{BLACK}$ 

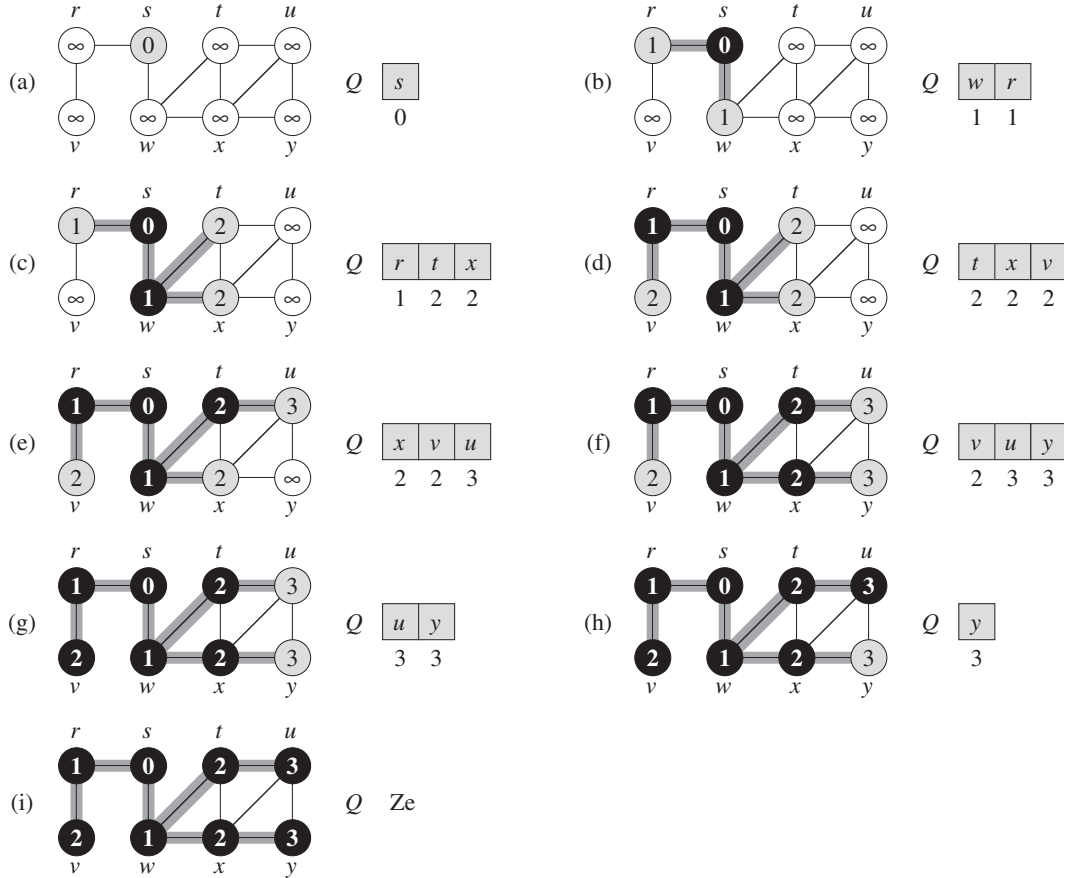
```

Şekil 22.3 örnek bir çizge üzerinde BFS nin ilerleme durumunu göstermektedir..

BFS işlemi aşağıdaki gibi çalışmaktadır. s kaynak tepesi hariç, satır 1-4 her tepeyi beyaza boyar, her u tepesi için $u.d$ yi sonsuz olarak belirler ve her tepenin ebeveynini (parent) NIL olarak belirler. Satır 5, işlemler başladığında bulunan olarak düşündüğümüz s tepesini griye boyar. Satır 6, $s.d$ yi 0 olarak başlatır ve satır 7 kaynağın öncülünü (predecessor) NIL olarak belirler. Satır 8-9 sadece s tepesini içeren Q kuyruğunu başlatır.

Satır 10-18 deki **while** döngüsü, bitişiklik listesi tam olarak elden geçirilmemiş bulunan tepeler yani gri tepeler kaldıkça tekrarlanır. Bu **while** döngüsü aşağıdaki değişmezi korur:

Satır 10 daki testte, Q kuyruğu gri tepeler kümesinden oluşur.



Şekil 22.3 Yönsüz bir çizge üzerinde BFS işlemi. BFS ile üretilen üç ayrıntı gölgeli olarak gösterilmiştir. Her u tepesi içinde $u.d$ değeri görünmektedir. Satır 10-18'deki **while** döngüsünün her adımının başlangıcında Q kuyruğu gösterilmiştir. Tepe uzaklıkları kuyruk içindeki tepelerin altında görünmektedir.

Bu döngü değişmezlerinin doğruluğunu kanıtlamaya gerek duymamakla birlikte, ilk iterasyona bağlı olarak ve döngü iterasyonları ile değişmez korunur. İlk iterasyona göre Q kuyruğundaki tek gri tepe ve tek tepe, s kaynak tepesidir. Satır 11, Q kuyruğunun en başındaki u gri tepesini ele alarak onu Q dan atar. Satır 12-17'deki **for** döngüsü u nun bitişiklik listesindeki her v tepesini inceler. Eğer v beyaz ise, henüz keşfedilmemiştir, ve Satır 14-17'nin çalıştırılması ile işlem onu keşfeder. İşlem v tepesini griye boyar, uzaklığını $v.d$ iken $u.d + 1$ ye olarak belirler, u nun ebeveyni olarak $v.\pi$ yi kaydeder ve onu Q kuyruğunun en sonuna yerleştirir. u nun bitişiklik listesindeki tüm tepeler işlem tarafından bir kez daha incelenir.

incelenmez satır 18 onu siyaha boyar. Döngü değişmezi korunur çünkü bir tepe griye boyandığında (satır 14 de) o aynı zamanda sıraya alınır (satır 17 de), ve bir tepe kuyruktan çıkarıldığında (satır 11 de) o aynı zamanda siyaha boyanır (satır 18 de).

Enine aramanın sonuçları verilen bir tepenin komşularının satır 12 de ziyaret edilme derecesine bağlı olabilmektedir; enine ağaç farklı olabilir, fakat algoritma ile hesaplanmış olan d uzaklıkları farklı değildir (Aıştırma 22.2-5 e bakınız).

Analiz

Enine aramanın birçok özelliğini kanıtlamadan önce, bir $G=(V, E)$ girdi çizgesinin çalışma zamanını analiz eden daha kolay bir iş ile başlayalım. Kısım 17.1 de gördüğümüz gibi, toparlama analizi kullanırız. Başlangıçtan sonra enine arama bir tepelyi hiçbir zaman beyazlaştırmaz ve satır 13 deki test her tepenin kuyruğa en fazla bir kez girdiğini ve en fazla bir kez çıktığını garanti eder. Bu giriş ve çıkış operasyonları $O(1)$ zaman alır ve kuyruk işlemleri için toplam $O(V)$ zaman ayrılır. Çünkü işlem her tepenin bitişiklik listesini, sadece tepe kuyruktan çıkarılırken tarar, her bitişiklik listesini en fazla bir defa tarar. Tüm bitişiklik listesinin uzunluğunun toplamı $\Theta(E)$ olduğundan, toplam bitişiklik listesinin taranması için en fazla $O(E)$ zaman harcanır. Başlangıç için üst sınır $O(V)$ dir ve böylece BFS işleminin toplam çalışma zamanı $O(V + E)$ olur. Sonuç olarak, enine arama G nin bitişiklik-listesi gösteriminin büyüklüğüne bağlı olarak doğrusal zamanda çalışır.

En Kısa Yollar

Bu kısmın başında, bir $G=(V, E)$ çizgesinde verilen bir $s \in V$ kaynak tepesinden başlanarak erişilebilen her tepelye olan uzaklığı enine aramanın bulduğunu iddia etmiştik. s tepesinden v tepesine olan yollar içinde en az sayıda ayrıta sahip olan yol, s den v ye **en kısa yol uzunluğu** $\delta(s, v)$ olarak tanımlanır; s den v ye hiç yol yoksa $\delta(s, v) = \infty$ olur. s den v ye olan bir yolun uzunluğu $\delta(s, v)$ ise s den v ye bir **en kısa yol**² vardır. Enine aramanın en-kısa yolların uzunluklarını doğru bir şekilde hesapladığını göstermeden önce, en kısa yolların uzunluğu ile önemli bir özelliği incelememiz gerekmektedir.

²Bölüm 24 ve 25 de, her ayrıtan gerçel-değerli bir ağırlığa sahip olduğu ve bir yolun ağırlığının o yolu oluşturan ayrıtların ağırlıkları toplamı olduğu ağırlıklı çizgeler üzerinde en kısa yollar çalışmamızı genelleştirmeliyiz. Şimdi içinde bulunulan bölümdeki çizgeler ağırlıksızdır veya eşit bir anlamda tüm ayrıtları birim ağırlığa sahiptir.

Lemma 22.1

$G = (V, E)$ yönlü veya yönsüz bir çizge ve $s \in V$ rassal seçilmiş bir tepe olsun. Bu durumda, herhangi bir $(u, v) \in E$ ayrıtı için

$$\delta(s, v) \leq \delta(s, u) + 1$$

olur.

Kanıt Eğer u tepesine, s den ulaşılabilirse, v tepesine de ulaşılabilir. Bu durumda, s den v ye olan en kısa yol, (u, v) ayrıtını takip ederek s den u ya olan en kısa yoldan daha uzun olamaz ve böylece eşitsizlik geçerli olur. Eğer, u tepesine s den ulaşamıyorsa bu durumda $\delta(s, u) = \infty$ olur ve eşitsizlik geçerli olur. ■

BFS nin her $v \in V$ tepesi için $v.d = \delta(s, v)$ yi doğru hesapladığını göstermek istiyoruz. Öncelikle $v.d$ nin, $\delta(s, v)$ yi üstten sınırladığını gösterelim.

Lemma 22.2

$G = (V, E)$ yönlü veya yönsüz bir çizge olsun ve BFS nin, verilen bir $s \in V$ kaynak tepesinden başlayarak G çizgesi üzerinde çalıştığını kabul edelim. Her $v \in V$ tepesi için BFS ile hesaplanan $v.d$ değeri $v.d \geq \delta(s, v)$ yi sağlar.

Kanıt ENQUEUE işlemlerinin sayısı üzerinde tümevarım kullanacağız. Tümevarım hipotezimiz, tüm $v \in V$ için $v.d \geq \delta(s, v)$ dir.

Tümevarımın temeli BFS nin 9. satırında s nin kuyruğa alınmasından hemen sonra başlar. Tümevarım hipotezi burada gerçekleşir, çünkü $s.d = 0 = \delta(s, s)$ ve her $v \in V - \{s\}$ için $v.d = \infty \geq \delta(s, v)$ dir.

Tümevarımın ilk adımı için bir u tepesinden başlanan arama sırasında karşılaşılan beyaz bir v tepesini düşünelim. Tümevarım hipotezi $u.d \geq \delta(s, u)$ yi gerektirir. Satır 15 ile yapılan atamadan ve Lemma 22.1 den,

$$\begin{aligned} v.d &= u.d + 1 \\ &\geq \delta(s, u) + 1 \\ &\geq \delta(s, v). \end{aligned}$$

Daha sonra v tepesi kuyruğa eklenir ve gri rengini aldığından dolayı bir daha kuyruğa alınmaz, satır 14-17 arasındaki **then** komutu sadece beyaz tepeler için çalıştırılır. Böylece, $v.d$ değeri bir daha asla değişmez ve tümevarımın hipotezi korunmuş olur. ■

$v.d = \delta(s, v)$ yi kanıtlayabilmek için ilk olarak BFS nin akışı esnasında Q kuyruğunun nasıl eksiksiz bir biçimde çalıştığını göstermeliyiz. Bir sonraki Lemma, kuyrukta her zaman en fazla iki farklı d değerinin tutulduğunu göstermektedir.

Lemma 22.3

Bir $G = (V, E)$ çizgesi üzerinde BFS nin çalışması sırasında v_1 Q nun başı ve v_r kuyruk sonu olmak üzere, Q kuyruğu $\langle v_1, v_2, \dots, v_r \rangle$ tepelerini içerir. Bu durumda tüm $i = 1, 2, \dots, r - 1$ için $v_r.d \leq v_i.d + 1$ ve $v_i.d \leq v_{i+1}.d$ dir.

Kanıt Kanıt, kuyruk işlemlerinin sayısı üzerinden tümevarımla yapılır. Başlangıçta, kuyruk sadece s yi içerdiğinden lemma kesinlikle gerçekleşir.

Tümevarımın ilk adımı için bir tepenin kuyruğa hem girişi hem de çıkışından sonra lemmanın gerçekleştiğini kanıtlamalıyız. Eğer kuyruğun başı v_1 kuyruktan çıktıysa, yeni baş v_2 olur. (Eğer kuyruk boş hale geldiyse kendiliğinden gerçekleşir). Tümevarım hipotezi ile, $v_1.d \leq v_2.d$ dir. Fakat bu durumda, $v_r.d \leq v_1.d + 1 \leq v_2.d + 1$ olur ve kalan eşitsizlikler etkilenmez. Böylece lemma baş olarak v_2 ile devam eder.

Bir tepenin kuyruğa eklenmesi üzerine neler olabileceğini anlayabilmek için kodu daha yakından incelememiz gerekmektedir. BFS nin 17. satırında bir v tepesini kuyruğa eklediğimizde bu tepe v_{r+1} olur. Aynı anda, bitişiklik listesi Q kuyruğu tarafından hali hazırda taranmış ve tümevarım hipotezi ile yeni baş tepe v_1 in $v_1.d \geq u.d$ yi sağladığı bir anda u tepesini çıkarmış oluruz. Öyleyse, $v_{r+1}.d = v.d = u.d + 1 \leq v_1.d + 1$ dir. Tümevarım hipotezinden, ayrıca $v_r.d \leq u.d + 1$, ve $v_r.d \leq u.d + 1 = v.d = v_{r+1}.d$ olup kalan eşitsizlikler bu durumdan etkilenmezler. Böylece, v kuyruğa eklendiğinde lemma gerçekleşir. ■

Sıradaki sonuç, tepelerin kuyruğa eklenmesi sırasında, d değerlerinin zamana bağlı bir biçimde monotonik arttığını göstermektedir.

Sonuç 22.4

BFS nin çalışması sırasında, v_i, v_j den daha önce olacak şekilde v_i ve v_j tepelerinin kuyruğa eklendiği kabul edilsin. v_j nin kuyruğa eklenmesiyle $v_i.d \leq v_j.d$ olur.

Kanıt BFS sürecinin çalışması sırasında her tepenin en fazla bir kez sonlu bir d değerini elde etmesi özelliğinden ve Lemma 22.3 den kanıt tamamlanır. ■

Şimdi, enine aramanın, en kısa yol uzunluklarını doğru bir biçimde bulduğunu kanıtlayabiliriz.

Teorem 22.5 (Enine aramanın doğruluğu)

$G = (V, E)$ yönlü ya da yönsüz çizge olsun ve BFS nin verilen bir $s \in V$ kaynak tepesinden başlayarak G üzerinde çalıştığını varsayalım. Daha sonra, çalışması sırasında, BFS, s kaynak tepesinden ulaşılabilen tüm $v \in V$ tepelerini keşfeder ve sonlanma sırasında tüm $v \in V$ ler için $v.d = \delta(s, v)$ olur. Ayrıca, s den ulaşılabilen

herhangi bir $v \neq s$ tepesi için s den v ye olan en kısa yollardan biri, (v, π, v) ayrıtıyla izlenen s den v, π ye olan en kısa yoldur.

Kanıt Çelişki amaçlandığı için bazı tepelerin en kısa yol uzaklığına eşit olmayan bir d değerini elde etmiş olduğu kabul edilsin. $\delta(s, v)$ minimum değerli ve yanlış d değerli v tepesi için açıktır ki, $v \neq s$ dir. Lemma 22.2 den, $v.d \geq \delta(s, v)$ olduğundan $v.d > \delta(s, v)$ e sahibiz. v tepesine, s den erişilebilinmelidir, aksi halde, $\delta(s, v) = \infty \geq v.d$ dir. s den v ye olan en kısa yol üzerinde, v den hemen önce gelen tepe u ise, $\delta(s, v) = \delta(s, u) + 1$ dir. $\delta(s, u) < \delta(s, v)$ olduğundan ve v yi nasıl seçtiğimize göre, $u.d = \delta(s, u)$ vardır. Bu özellikleri bir araya getirdiğimizde,

$$v.d > \delta(s, v) = \delta(s, u) + 1 = u.d + 1 \quad (22.1)$$

elde edilir.

Şimdi de Q nun 11. satırında atılan u tepesinin seçimi esnasında BFS nin harcadığı zamanı değerlendirelim. Bu sırada, v tepesi beyaz, gri veya siyah olabilir. Bu durumların herbiri için (22.1) eşitsizliği ile bir çelişki elde edeceğimizi göstermeliyiz. Eğer v beyaz ise, satır 15, (22.1) eşitsizliği ile çelişki oluşturarak $v.d = u.d + 1$ i belirler. Eğer v siyah ise, zaten kuyruktan atılmıştır ve Sonuç 22.4 e göre, (22.1) eşitsizliği ile çelişki oluşturarak, $v.d \leq u.d$ elde edilir. Eğer v gri ise, u 'dan önce Q dan atılacak herhangi bir w tepesi şartıyla ve $v.d = w.d + 1$ eşitliğiyle gri renkli kalır. Fakat, Sonuç 22.4 ile $w.d \leq u.d$ olup, $v.d = w.d + 1 \leq u.d + 1$ elde edilir ki, bir kez daha (22.1) eşitsizliği ile çelişkiye düşülür.

Sonuç olarak, tüm $v \in V$ ler için $v.d = \delta(s, v)$ eşitliğinin çıkarımını yapabiliriz. s den ulaşılabilen tüm v tepeleri keşfedilmiş olmalıdır, aksi halde $\infty = v.d > \delta(s, v)$ bulunur. Teoremin kanıtını sonlandırabilmek için eğer $v, \pi = u$ ise $v.d = u.d + 1$ olduğunu kabul edelim. Bu durumda, s den v ye olan en kısa yollardan biri, s den v, π ye olan en kısa yolun alınması ve (v, π, v) ayrıtının üzerinden geçilmesiyle bulunabilir. ■

Enine ağaçlar

Şekil 22.3 de de gösterildiği üzere, BFS işlemleri çizgede arama yaparken bir enine bir ağaç oluşturur. Ağaç, π simgesiyle ilişkilendirilir. Daha resmi olarak, s kaynaklı bir $G = (V, E)$ çizgesi için

$$V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\}$$

ve

$$E_\pi = \{(v, \pi, v) : v \in V_\pi - \{s\}\}$$

olmak üzere, G nin **öncül altçizgesi** $G_\pi = (V_\pi, E_\pi)$ biçiminde tanımlanır. Eğer V_π , tüm $v \in V$ için s den ulaşılabilen tepelerden oluşuyorsa ve s den v ye olan

G deki en kısa yol aynı zamanda G_π deki tek basit yol ise bu durumda öncül altçizge G_π **enine bir ağaçtır**. Enine ağaç aslında bir ağaçtır, çünkü bağlantılıdır ve $|E_\pi| = |V_\pi| - 1$ dir (Teorem B.2 ye bakınız). E_π içindeki ayrıtlar **ağaç ayrıtları** olarak adlandırılır.

Aşağıdaki lemma BFS çalışması ile oluşan öncül altçizgenin enine bir ağaç olduğunu göstermektedir.

Lemma 22.6

π yi oluşturan BFS adımlarının yönlü veya yönsüz $G = (V, E)$ çizgesine uygulanmasıyla oluşan $G_\pi = (V_\pi, E_\pi)$ öncül altçizgesi, enine bir ağaçtır.

Kanıt Ancak ve ancak $(u, v) \in E$ ve $\delta(s, v) < \infty$ dur, —yani v ye s den erişilebilir olduğunda—ancak ve ancak 16. satırından, $v.\pi = u$ sağlanır ve V_π , V içindeki s den erişilebilen tepelerden oluşur. G_π bir ağaç biçiminde olduğundan, Teorem B.2 ile, V_π deki her tepeye s den erişilebilir tek basit bir yol içerir. Teorem 22.5 in uygulanmasıyla tümevarımsal olarak bu şekildeki tüm yollar G deki en kısa yollardır. ■

BFS nin, enine bir ağacı hesapladığı düşünüldüğünde, s den v ye olan en kısa yol üzerindeki tepeler aşağıdaki adımlarla yazdırılır:

```

PRINT-PATH ( $G, s, v$ )
1  if  $v = s$ 
2      print  $s$ 
3  elseif  $v.\pi = \text{NIL}$ 
4      print  $s$  “den”  $v$  “ye doğru olan bir yol yoktur”
5  else PRINT-PATH ( $G, s; v.\pi$ )
6      print  $v$ 

```

Bu adımlar yazdırılan yoldaki tepe sayısı ile doğrusal zamanda çalışır, çünkü yol için olan her özyinelemeli çağrı bir tepe daha kısadır.

Alıştırmalar

22.2-1

Tepe 3 ü kaynak olarak kullanarak, Şekil 22.2(a) daki yönsüz çizgede yapılan enine aramadan d ve π değerlerini gösteriniz.

22.2-2

u tepesini kaynak olarak kullanarak, Şekil 22.3 deki yönlü çizgede yapılan enine aramadan d ve π değerlerini gösteriniz.

22.2-3

Tek bit kullanarak her bir tepenin rengini saklamanın yeterli olduğunu BFS adımlarının satır 5 ve 14 silinirse de aynı sonucu üreteceğini tartışarak gösteriniz.

22.2-4

Eğer BFS nin girdi çizgesini bitişiklik matrisi ile gösterirsek ve algoritmayı bu formattaki girdiyi tutmak için değiştirirsek çalışma zamanı ne olur?

22.2-5

Enine aramada tepelerin her bir bitişiklik listesinde bulunduğu durumda $u.d$ değerinin sıralamadan bağımsız u tepesi değerine atanmasını tartışınız. Şekil 22.3 ü örnek olarak kullanarak, BFS tarafından hesaplanan enine ağacın bitişiklik listesindeki sıraya bağlı olduğunu gösteriniz.

22.2-6

Kaynak tepesinin $s \in V$ ve ağaç kümelerinin $E_\pi \subseteq E$ olduğu, her $v \in V$ tepesi için s den v ye (V, E_π) çizgesindeki yolun G deki en kısa yol olduğu, fakat E_π nin ayrıştırılabilir kümesi G üzerinde herbir bitişiklik listesinde tepeler sıralansa bile BFS çalıştırılarak üretilemez olduğu yönlü bir $G = (V, E)$ çizgesine örnek veriniz.

22.2-7

Profesyonel güreşçilerin iki tipi vardır: “bebek yüzlü” (“iyi adamlar”) ve “kalleş kimseler” (“kötü adamlar”). Profesyonel güreşçilerin eşleşmelerinde rekabet olabilir ya da olmayabilir. n tane profesyonel güreşçimiz olduğunu ve çekişmeli r tane eşleşme listemiz olduğunu farz edelim. Bazı güreşçilerin bebek yüzlü ve geri kalanların kalleş olduğunu, öyle ki her bir rekabetin bebek yüzlü ve kalleş arasında olduğu tasarlamanın mümkün olup olmadığını belirleyen $O(n + r)$ zamanlı bir algoritma veriniz. Eğer bu şekilde bir tasarım oluşturmak mümkün ise, algoritmanız bunu üretmelidir.

22.2-8 *

$T = (V, E)$ ağacının **çapı** (*diameter*) $\max_{u,v \in V} \delta(u, v)$ olarak tanımlanır, yani bu ağaçtaki en büyük en kısa yolların mesafesidir. Bir ağacın çapını hesaplamak için verimli bir algoritma veriniz ve algoritmanızın çalışma zamanını analiz ediniz.

22.2-9

$G = (V, E)$ bağlantılı, yönsüz bir çizge olsun. Her yönde kesinlikle bir tane olan E deki herbir ayrıttan çapraz geçen G deki yolu hesaplayan $O(V + E)$ zamanlı bir algoritma veriniz. Eğer yüksek miktarda kuruş tedarikiniz verilmişse, labirentten çıkmak için yolunuzu nasıl bulacağınızı tarif ediniz.

22.3 Derinliğine arama

Derinliğine arama, adından da anlaşıldığı üzere, çizgede mümkün olduğunca “daha derine” arama yapar. Derinliğine aramada en son bulunan v tepesinden çıkan henüz keşfedilmemiş ayrıtlar araştırılır. v nin tüm ayrıtları keşfedilmişse, arama aynı yoldan ayrıtları keşfetmeye geri döner. Bu süreç, orjinal kaynak tepeden ulaşılabilen tüm tepeler bulununcaya kadar devam eder. Eğer, bulunmayan bir tepe kaldıysa, derinliğine arama bu tepelerden birini yeni kaynak olarak seçer ve arama bu kaynak tepeden tekrarlanır. Algoritma, tüm bu süreci her yeni bulunduğu tepe için tekrarlar.³

Enine aramada olduğu gibi, derinliğine aramada da yeni uğranmış u tepesine ait bitişiklik listesindeki tepelerin taranmasıyla v tepesi bulunur ve aynı zamanda bu aramada, v nin öncülü $v. \pi$ yi u ya bağlar. Öncül altçizgesi ağaç formunda olan enine aramanın tersine, derinliğine arama tarafından oluşturulan öncül altçizgesi birkaç ağaçtan oluşabilir, çünkü arama birden çok kaynaktan tekrarlanmış olabilir. Bu nedenle derinlik aramanın **öncül altçizgesi**ni enine aramadan kısmen farklı bir biçimde tanımlarız:

$E_{\pi} = \{(v. \pi, v) : v \in V \text{ ve } v. \pi \neq \text{NIL}\}$ olacak şekilde,

$G_{\pi} = (V, E_{\pi})$ tanımlanır.

Derinliğine aramanın öncül altçizgesi pekçok **derinliğine ağaç**tan oluşan **derinliğine bir orman** formundadır. E_{π} içindeki ayrıtlar **ağaç ayrıtları**dır.

Enine aramada olduğu gibi, derinliğine aramada da tepelerin yerini belirlemek için arama boyunca tepeler boyanır. Her tepe başlangıçta beyazdır, aramada bulununca gri olur ve bittiğinde yani bitişiklik listesi tamamlandığında siyah olur. Bu teknik her tepenin kesinlikle derinliğine bir ağaçta bitmesini garantiler, bu da ağaçların ayrık olduğunu gösterir.

Derinliğine bir ormanı oluşturma yanısıra, derinliğine arama aynı zamanda her tepenin bir **uğranma zamanını** bulur. Her v tepesinin iki zaman pulu vardır: ilk uğranma zamanını $v.d$, v ilk kez keşfedildiğinde (ve gri olduğunda) kaydeder, ikinci zaman pulunu $v.f$, v nin bitişiklik listesinin incelenmesi tamamlandığında

³Enine arama sadece bir kaynak ile sınırlı iken, derinliğine aramanın birden fazla kaynaktan arama yapabilmesi keyfi görünebilir. Fakat, kavramsal olarak enine arama birden fazla kaynaktan ilerleyebilir ve derinliğine arama bir kaynak ile sınırlandırılabilir, bu aramaların sonuçlarının nasıl olduğuna bağlı olarak genellikle hangisinin kullanılacağı bizim yaklaşımımıza yansır. Enine arama en kısa yol uzunluklarını (ve ilişkili öncül altçizgesi) genellikle verilen tek bir kaynaktan bularak çalışır. Derinliğine arama, bu bölümün ilerleyen kısımlarında göreceğimiz gibi, diğer algoritmanın içinde çoğunlukla bir alt-programdır.

(v siyah olduğunda) kaydeder. Bu zaman pulları çizgenin yapısı hakkında önemli bilgiler verir ve derinliğine arama davranışı hakkında yardımcı olur.

Aşağıdaki DFS adımları $u.d$ değişkeni içinde u keşfedildiğinde kaydedilmeye başlanır ve $u.f$ değişkeni içinde u tepesi bulunduğunda sonlanır. Bu zaman pulları, 1 ile $2 \lfloor V \rfloor$ tam sayıları arasındadır, çünkü her $\lfloor V \rfloor$ tepesi için bir bulunma olayı ve bir bitiş olayı vardır. Her u tepesi için

$$u.d < u.f \quad (22.2)$$

dir.

u tepesi $u.d$ zamanından önce WHITE dır, $u.d$ zamanı ve $u.f$ zamanı arasında GRAY, ve sonrasında ise BLACK dır.

Aşağıdaki kabakod basit bir derinliğine arama algoritmasıdır. G girdi çizgesi yönlü veya yönsüz olabilir. $Time$ değişkeni, evrensel bir değişken olup biz onu zaman pulu için kullanırız.

DFS (G)

```

1  for her  $u \in G.V$  tepesi
2       $u.color = \text{WHITE}$ 
3       $u.\pi = \text{NIL}$ 
4   $time = 0$ 
5  for her  $u \in G.V$  tepesi
6      if  $u.color == \text{WHITE}$ 
7          DFS-VISIT( $G, u$ )

```

DFS-VISIT(G, u)

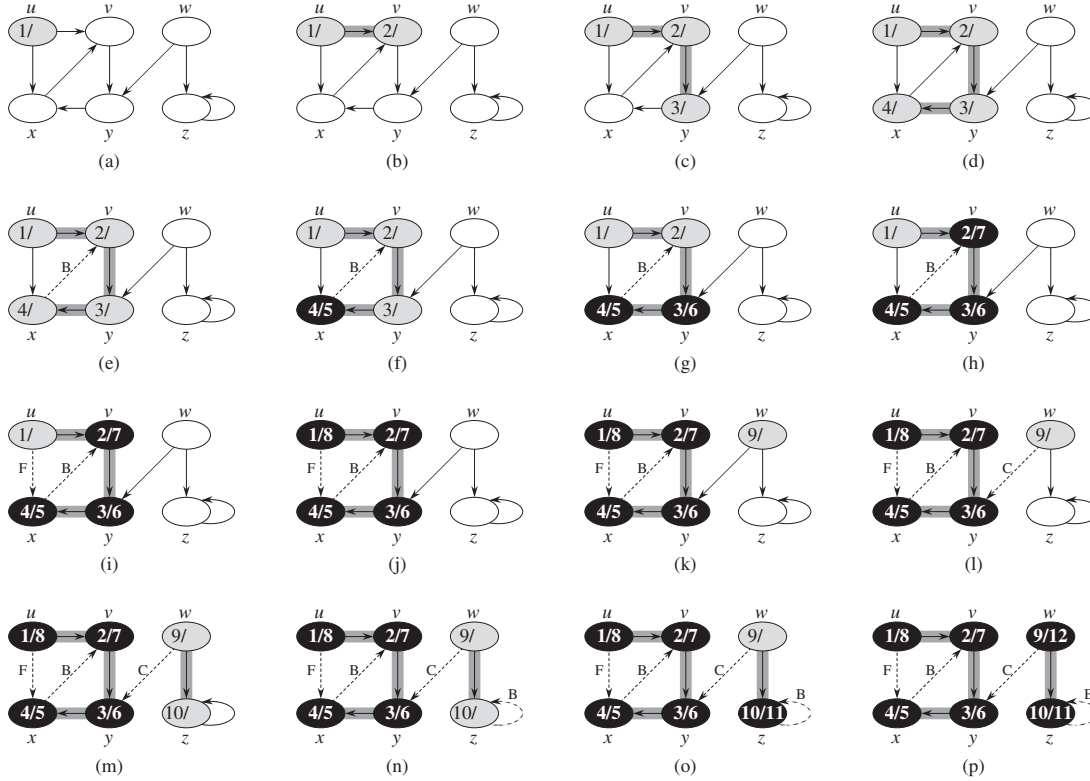
```

1   $time = time + 1$                                 //  $u$  beyaz tepesi henüz bulundu
2   $u.d = time$ 
3   $u.color = \text{GRAY}$ 
4  for her  $v \in G$  tepesi,  $Adj[u]$                     //  $(u, v)$  ayrıtını keşfet
5      if  $v.color == \text{WHITE}$ 
6           $v.\pi = u$ 
7          DFS-VISIT( $G, v$ )
8   $u.color = \text{BLACK}$                                 // siyahlanan  $u$ ; bitti
9   $time = time + 1$ 
10  $u.f = time$ 

```

Şekil 22.2 deki çizgenin DFS adımları Şekil 22.4 de gösterilmiştir.

DFS süreci şu şekilde çalışmaktadır. Satır 1-3 tüm tepeleri beyaza boyar ve onların π değişkeni NIL ile başlar. Satır 4 evrensel zaman sayacını sıfırlar. Satır 5-7 V deki her tepayı sırasıyla kontrol eder ve beyaz bir tepe bulduğunda, o te-



Şekil 22.4 Yönlü çizgedeki derinliğine arama DFS nin ilerleme durumu. Algoritma tarafından ayrıtlar keşfedildiğinde, ya gölgeli (eğer üç ayrıt ise) yada noktalı (aksi halde) olarak gösterilmiştir. Ağaç olmayan ayrıtlar geri, kesen veya ileri ayrıt olup olmadığına göre B, C veya F ile isimlendirilmiştir. Tepelerdeki zaman bulma zamanı/bitiş zamanlarını belirtmektedir.

peye DFS-VISIT ile uğrar. Satır 7 de her seferinde $\text{DFS-VISIT}(G, u)$ çağrılır ve u tepesi derinliğine ormanın yeni bir ağacının kökü olur. DFS geri döndüğünde, her u tepesine bir $u.d$ bulma zamanı ve $u.f$ bitiş zamanı atanır.

$\text{DFS-VISIT}(G, u)$ nin her çağrılışında, u tepesi başlangıçta beyazdır. Satır 1 evrensel değişken $time$ ı artırır, satır 2 $time$ ın yeni değerini $u.d$ bulma zamanı olarak kaydeder ve satır 3 u yu griye boyar. Satır 4-7'de u ya komşu olan her v tepesi incelenir ve eğer v beyazsa tekrar tekrar ziyaret edilir. Her $v \in \text{Adj}[u]$ tepesi satır 4 de dikkate alındığından, (u, v) ayrıtı derinliğine arama ile **keşfedilmiştir (explored)** deriz. Sonuç olarak, u dan ayrılan her ayrıt keşfedildikten sonra sonra, satır 8-10 u yu siyah olarak boyar, zamanı artırır ve $u.f$ deki bitiş zamanını kaydeder.

Derinliğine aramanın sonuçları, DFS nin satır 5 inde tepelerin incelenme sırasına ve satır 4 ünde DFS-VISIT in bir tepenin komşularını ziyaret etme sıra-

sına bağlıdır. Herhangi bir derinliğine aramanın sonuçlarını verimli bir şekilde kullanabildiğimizden ve genellikle eşit sonuçlar elde edildiğinden, farklı ziyaret sıraları pratikte bir probleme neden olmaz.

DFS nin çalışma zamanı nedir? DFS nin satır 1-3 ve satır 5-7 deki döngüleri, DFS-VISIT aramalarının çalıştırılması sırasında geçen zaman hariç, $\Theta(V)$ zaman alır. Enine aramada yaptığımız gibi, toplam analizi kullanırız. DFS-VISIT adımları her bir tepe $v \in V$ için kesinlikle bir defa çağırılır, DFS-VISIT çağırıldığında u tepesi beyaz olmalı ve DFS-VISIT in yapacağı ilk iş u tepesini gri ile boyamaktır. DFS-VISIT(G, v) nin çalıştırılması sırasında, satır 4-7 deki döngü $|Adj[v]|$ kere çalışır.

$$\sum_{v \in V} |Adj[v]| = \Theta(E)$$

olduğundan, DFS-VISIT in satır 4-7 de çalışmasının toplam maliyeti $\Theta(E)$ dir. BFS nin çalışma zamanı bundan dolayı $\Theta(V + E)$ dir.

Derinliğine arama özellikleri

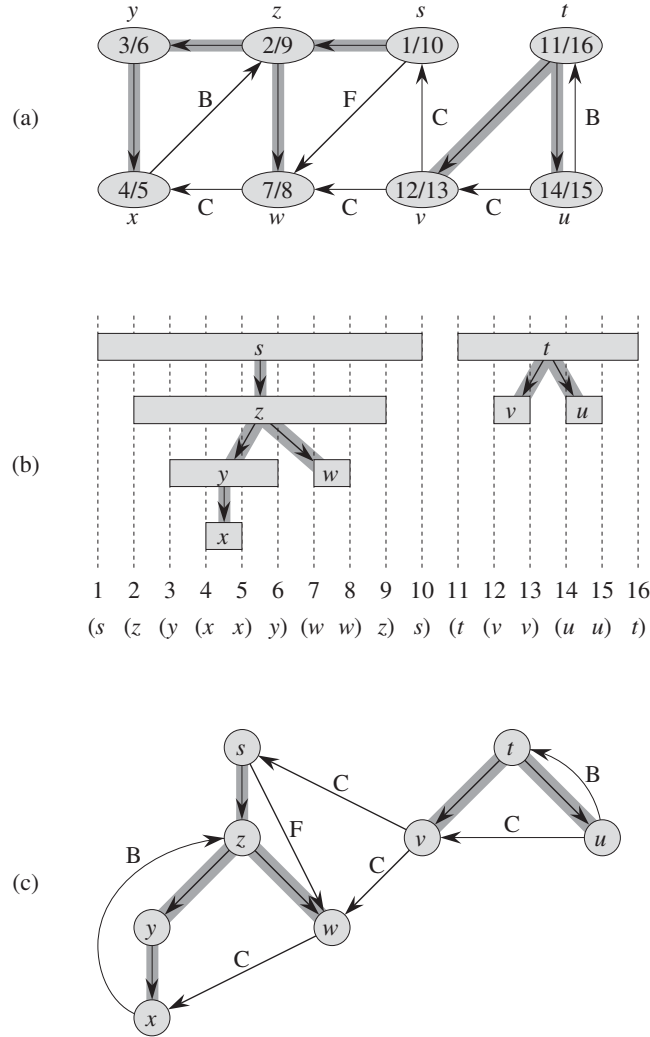
Derinliğine arama bir çizgenin yapısı hakkında değerli bilgiler verir. Belki de derinliğine aramanın en önemli ana özelliği aslında ağaçların bir orman yapısında olan G_π öncül altçizgesidir, çünkü derinliğine ağaçların yapısı tam olarak DFS-VISIT özyinelemeli çağrılarının yapısını yansıtır. Bu da $u = v.\pi$ ancak ve ancak u nun bitişlik listesinin araması sırasında DFS-VISIT(G, v) çağırıldığında olur. Ek olarak, v tepesi derinliğine ormanda u tepesinin torunudur, ancak ve ancak u gri iken v keşfedildiği zaman.

Derinliğine aramanın diğer bir önemli özelliği, keşif ve bitiş zamanlarının **parantez yapısına** (parenthesis structure) sahip olmasıdır. Eğer u tepesinin keşfini bir sol parantez ile “(“ u ” ve bitişini bir sağ parantez ile “)” gösterirsek, bu durumda parantezlerin uygun bir şekilde içiçe kullanımıyla keşifler ve bitişlerin tarihinde iyi bir şekilde ifade edilebilir. Örneğin, Şekil 22.5(a) daki derinliğine aramaya karşılık gelen parantez yapısı Şekil 22.5(b) de gösterilmiştir. Aşağıdaki teorem parantez yapısını karakterize etmenin farklı bir yolunu göstermektedir.

Teorem 22.7 (Parantez teoremi)

$G = (V, E)$ çizgesinin (yönlü veya yönsüz) herhangi bir derinliğine aramasında, herhangi iki u ve v tepeleri, tam olarak aşağıdaki üç koşulu sağlar:

- $[u.d, u.f]$ ve $[v.d, v.f]$ aralıkları tümüyle ayrıktır ve derinliğine ağaçta ne u ne de v birbirinin torunudur,
- $[u.d, u.f]$ aralığı tümüyle $[v.d, v.f]$ aralığı tarafından içeriliyorsa ve derinliğine ağaçta u, v nin bir torunudur, veya
- $[v.d, v.f]$ aralığı tümüyle $[u.d, u.f]$ aralığı tarafından içeriliyorsa ve derinliğine ağaçta v, u nun bir torunudur.



Şekil 22.5 Derinliğine aramanın özellikleri. (a) Yönlü bir çizgedeki derinliğine arama sonuçları. Tepeler zaman bilgili ve ayırt tipleri Şekil 22.4 de belirtilmiş. (b) Gösterilmiş parantezleme ile ilgili herbir tepenin keşif zamanı ve bitiş zamanı için aralıklar. Her dikdörtgen ilgili tepenin bulma ve bitiş zamanları ile verilen aralığı kapsar. Sadece üç ayırt gösterilmiştir. Eğer iki aralık çalışırsa, biri diğerinin içine geçmiştir ve küçük aralığa karşılık gelen tepe büyük aralığa karşılık gelen tepenin torunudur. (c) (a) kısmındaki çizge derinliğine ağaca giden bütün ağaç ve ileri ayırtları yeniden çizilmiştir ve bütün geri ayırtlar torundan ataya yükselmiştir.

Kanıt $u.d < v.d$ durumuyla başlayalım. Düşüneceğimiz iki alt durum vardır, $v.d < u.f$ olması ya da olmaması durumları. İlk alt durum olan $v.d < u.f$, u hala gri iken v bulunduğu, yani v , u nun bir torunu olduğunda sağlanır. Hatta v , u dan sonra bulunduğu için v nin tüm çıkış ayrıtları keşfedilmiştir, arama geri dönmeden ve u yu bitirmeden önce v bitmiştir. Bu durumda, dolayısıyla, $[v.d, v.f]$ aralığı tamamen $[u.d, u.f]$ aralığı tarafından içerilir. Diğer alt durumda, $u.f < v.d$ ve 22.2 eşitsizliğinden, $u.d < u.f < v.d < v.f$ olduğundan $[u.d, u.f]$ ve $[v.d, v.f]$ aralıkları ayrıktır. Aralıklar ayrık olduğundan, diğeri gri iken bulunan tepe de, biri diğerrinin torunu olan tepe de yoktur.

Yukarıdaki kanıtta bulunan u ve v nin rolleri yer değiştirildiğinde de $v.d < u.d$ durumu aynıdır. ■

Sonuç 22.8 (Torunların aralıklarının içiçe konulması)

G (yönlü veya yönsüz) çizgesi için derinliğine ormanda v tepesinin u tepesinin uygun bir torunu olması için gerek ve yeter şart $u.d < v.d < v.f < u.f$ dir.

Kanıt Teorem 22.7 den doğrudan görülür. ■

Sonraki teoremden derinliğine ormanda bir tepenin diğerrinin torunu olmasının diğerr bir önemli nitelemesi verilmiştir.

Teorem 22.9 (Beyaz-yol teoremi)

$G = (V, E)$ (yönlü veya yönsüz) çizgesinin derinliğine ormanında, v tepesinin u tepesinin bir torunu olması için gerek ve yeter şart, u yu bulan arama $u.d$ zamanında ise, v tepesine u tepesinden gidilen tamamen beyaz tepelerden oluşan bir yol vardır.

Kanıt $\Rightarrow$: Eğer $v = u$ ise, u dan v ye olan yol $u.d$ değerini belirlediğimizde hala beyaz olan sadece u tepesini içerir. Şimdi, derinliğine ormanda v nin u nun uygun bir torunu olduğunu kabul edelim. Sonuç 22.8 e göre $u.d < v.d$ ve $u.d$ zamanında v beyazdır. v , u nun herhangi bir torunu olduğundan, derinliğine ormanda $u.d$ zamanında u dan v ye olan tek basit bir yol üzerindeki tüm tepeler beyazdır.

$\Leftarrow$: Kabul edelim ki, $u.d$ zamanında u dan v ye olan bir yoldaki tepeler beyazdır, fakat derinliğine ağaçta v , u nun bir torunu olmaz. Genelliği bozmadan, yol boyunca kabul edelim ki v dışındaki diğerr tüm tepeler u nu torunu olur. (aksi halde yol boyunca v , u tepesine en yakın tepe olduğunda v , u nun torunu olmaz). Yol üzerinde w , v nin bir öncülü olsun, böylece w , u nun torunu olur (w ve u aslında aynı tepe olabilirler). Sonuç 22.8 e göre, $w.f \leq u.f$. Çünkü v , u bulunduktan sonra, ancak w nun bitmesinden önce bulunmalıdır, böylece $u.d < v.d < w.f \leq u.f$ dir. Te-

orem 22.7 $[v.d, v.f]$ aralığının tamamen $[u.d, u.f]$ aralığında içerildiğini vurgular. Sonuç 22.8 e göre v tüm torunlardan sonra u nun bir torunu olmalıdır. ■

Ayrıtların sınıflandırılması

Derinliğine aramanın bir diğer önemli özelliği $G = (V, E)$ girdi çizgesinin ayrıtlarının sınıflandırılmasında kullanılmasıdır. Her ayrıtlın türü, bir çizge hakkında önemli bilgiler sağlar. Örneğin, bir sonraki kısımda, yönlü bir çizgenin çevrimsiz olması için gerek ve yeter şart derinliğine aramada geri ayrıtların olmamasıdır ifadesini göreceğiz (Lemma 22.11).

G üzerinde bir derinliğine aramayla oluşturulan G_π derinliğine ormanında 4 tür ayrıtl tanımlayabiliriz:

1. **Ağaç ayrıtlar** G_π derinliğine ormanın ayrıtlarıdır. Eğer v tepesi ilk olarak (u, v) ayrıtlının incelenmesiyle keşfedilmişse, (u, v) ayrıtı bir ağaç ayrıttır.
2. **Arka ayrıtlar** Derinliğine bir ağaçta bir u tepesini, atası olan v tepesine bağlayan (u, v) ayrıtlarıdır. Yönlü çizgelerde olan döngüler, geri ayrıtlar olarak düşünülür.
3. **İleri ayrıtlar** Derinliğine bir ağaçta bir u tepesini, torunu olan bir v tepesine bağlayan ağaç olmayan (u, v) ayrıtlarıdır.
4. **Çapraz ayrıtlar** diğer tüm ayrıtlardır. Aynı derinliğine ağaçta bir tepenin diğerinin atası olmayan tepeler arasında ya da farklı derinliğine ağaçlardaki tepeler arasında gidip gelebilir.

Şekil 22.4 ve 22.5 deki ayrıtların etiketleri ayrıtların türünü göstermektedir. Şekil 22.5(a) daki çizgenin derinliğine ağacında tüm ağaç ve ileri ayrıtların aşağı doğru ve tüm geri ayrıtların yukarı doğru nasıl yeniden çizildiği Şekil 22.5(c) de gösterilmektedir. Bu biçimde, herhangi bir çizge yeniden çizilebilir.

DFS algoritması, ayrıtlarla karşılaştığından dolayı, bazı ayrıtları sınıflandırmak için yeterli bilgiye sahiptir. Burada kilit nokta, (u, v) ayrıtlının ilk kez incelenmesinde, v tepesinin renginin ayrıtl hakkında birşeyler ifade etmesidir:

1. WHITE ağaç ayrıtı belirtir,
2. GRAY arka ayrıtı belirtir,
3. BLACK ileri veya çapraz ayrıtı belirtir.

İlk durum algoritmanın özelliğinden hemen gelir. İkinci durum için kabul edelim ki, gri tepeler aktif DFS-VISIT çağrılarının yığını ile ilişkili torunların daima bir lineer zinciri şeklindedir; gri tepelerin sayısı derinliğine ormanda son keşfedilmiş tepenin derinliğinden bir fazladır. Arama her zaman en derindeki gri tepeden ilerler, böylece başka bir gri tepeye ulaşan bir ayrıtl bir ataya ulaşmış olur. Üçüncü

durum kalan olasılıkla ilgilidir; Alıştırma 22.3-5, bir (u, v) ayrıtı vardır ki, eğer $u.d < v.d$ ise bu ayrıtı ileri ayrıtı, $u.d > v.d$ ise çapraz ayrıtı olduğunu gösterin sorusunu bize sormaktadır.

Yönsüz bir çizgede, (u, v) ve (v, u) nun tam olarak aynı ayrıtı olması, ayrıtıların sınıflandırılmasında bazı belirsizliklere yol açabilir. Böyle bir durumda, uygulanan sınıflandırma listesi içinde ayrıtı birinci tür olarak sınıflandırılır. Eşit bir şekilde (Alıştırma 22.3-6 ya bakınız), aramada (u, v) veya (v, u) dan hangisine ilk olarak rastlanırsa o ayrıtı sınıflandırılır.

Şimdi, yönsüz bir çizgedeki derinliğine aramada ileri ve çapraz ayrıtıların asla bulunmayacağını göstereceğiz.

Teorem 22.10

Yönsüz bir G çizgesinin derinliğine aramasında, G nin her ayrıtı ya bir ağaç ayrıtı ya da geri ayrıttır.

Kanıt (u, v) , G nin rassal seçilmiş bir ayrıtı olsun ve genelliği bozmadan $u.d < v.d$ olduğunu kabul edelim. Bu durumda, arama v yi, u yu bitirmeden (u hala gri iken) keşfeder ve bitirir, çünkü v, u nun bitişiklik listesindedir. Eğer, arama u dan v ye doğru olan yönde ilk olarak (u, v) ayrıtını keşfederse, zira v o zamana kadar keşfedilmemiştir (beyaz), aksi durumda arama bu ayrıtı v den u ya doğru olan yönde keşfetmiş olur. O halde, (u, v) bir ağaç ayrıtı olur. Eğer arama v dan u ya doğru olan yönde ilk olarak (u, v) ayrıtını keşfederse, (u, v) bir geri ayrıtı olur, çünkü ayrıtının ilk keşfedildiği anda u hala gridir. ■

Bu teoremlerin birçok uygulamasını sonraki kısımlarda göreceğiz.

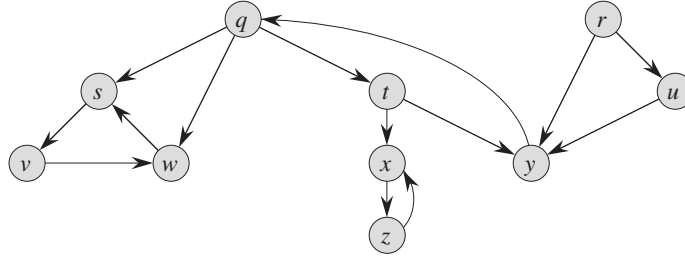
Alıştırmalar

22.3-1

Kolon ve satır adları WHITE, GRAY ve BLACK olan 3 e 3 lük bir çizelge yapın. Her bir hücre (i, j) de, yönlü çizgenin derinliğine arama sırasında herhangi bir noktada tepe rengi i den tepe rengi j ye olan olup olamayacağını belirtiniz. Her bir olası ayrıtı için hangi ayrıtı tipi olduğunu belirtiniz. Yönsüz çizge için aynı şekilde ikinci bir çizelge yapın.

22.3-2

Şekil 22.6 daki çizgede derinliğine aramanın nasıl çalıştığını gösteriniz. DFS adımlarının satır 5-7 deki **for** döngüsü tepelerin alfabetik sıraya göre olduğunu düşündüğünü farzediniz ve her bir bitişiklik listesinin alfabetik sırada olduğunu farzediniz. Her bir tepe için bulma ve bitiş zamanlarını gösteriniz ve her bir ayrıtının sınıflandırılmasını gösteriniz.



Şekil 22.6 Alıştırma 22.3-2 ve 22.5-2 de kullanılmak için yönlü bir çizge.

22.3-3

Şekil 22.4 deki derinliğine aramanın parantez yapısını gösteriniz.

22.3-4

Tek bit kullanarak her bir tepenin rengini tutmanın yeterli olduğunu tartışarak DFS adımlarının eğer DFS-VISIT in 3. satırı silinirse aynı sonucu üreteceğini gösteriniz.

22.3-5

(u, v) ayrıtının

- bir ağaç ayrıt veya ileri ayrıt ancak ve ancak $u.d < v.d < v.f < u.f$,
- bir geri ayrıt ancak ve ancak $v.d \leq u.d \leq u.f \leq v.f$, ve
- bir kesen ayrıt ancak ve ancak $v.d < v.f < u.d < u.f$, olduğunu gösteriniz.

22.3-6

Yönsüz çizgede, (u, v) ayrıtının ağaç ayrıt ya da geri ayrıt olarak sınıflandırılması, (u, v) ya da (v, u) nun derinliğine arama sırasında ilk olarak karşılaşılması, sınıflandırma çizelgesinde dört tipe göre sıralanarak sınıflandırılmasına eşdeğerdir.

22.3-7

Yığın kullanarak tekrarları elemek için DFS adımlarını tekrar yazınız.

22.3-8

Eğer yönlü G derinliğine u dan v ye bir yol içeriyorsa ve eğer G nin derinliğine aramasında $u.d < v.d$ ise o zaman derinliğine ormanın ürettiği v, u nun neslinden gelir, varsayımına karşı örnek veriniz.

22.3-9

Eğer yönlü çizge G u dan v ye bir yol içeriyorsa, o zaman herhangi derinliğine arama $v.d \leq u.f$ ile sonuçlanmalıdır, varsayımına karşı örnek veriniz.

22.3-10

Derinliğine arama kabakodunu G yönlü çizgesindeki her ayrıtı tipleriyle birlikte yazdıracak şekilde değiştiriniz. Hangi değişiklikler yaptığınızı, varsa eğer G 'yi yönsüz yapmak gerekirse gösteriniz.

22.3-11

Yönlü çizgenin u tepesinin sadece u yu barındıran derinliğine ağaçta u gelen ve giden ayrıtlar olsa bile nasıl sonuçlandığını açıklayınız.

22.3-12

G nin bağlantılı bileşenlerini belirlemek için yönsüz G çizgesi için derinliğine aramayı kullanabileceğimizi gösteriniz ve derinliğine orman G nin bağlı bileşenler kadar çok ağaçlar barındırsın. Daha kesin olarak, derinliğine aramanın nasıl değiştirildiğini gösteriniz. Böylece her v tepesi tam sayı etiket $v.cc$, 1 ve k , arasında olacak şekilde atanır, burada k G nin bağlı bileşenlerinin sayısıdır, öyle ki ancak ve ancak u ve v aynı bağlı bileşende ise $u.cc = v.cc$ dir.

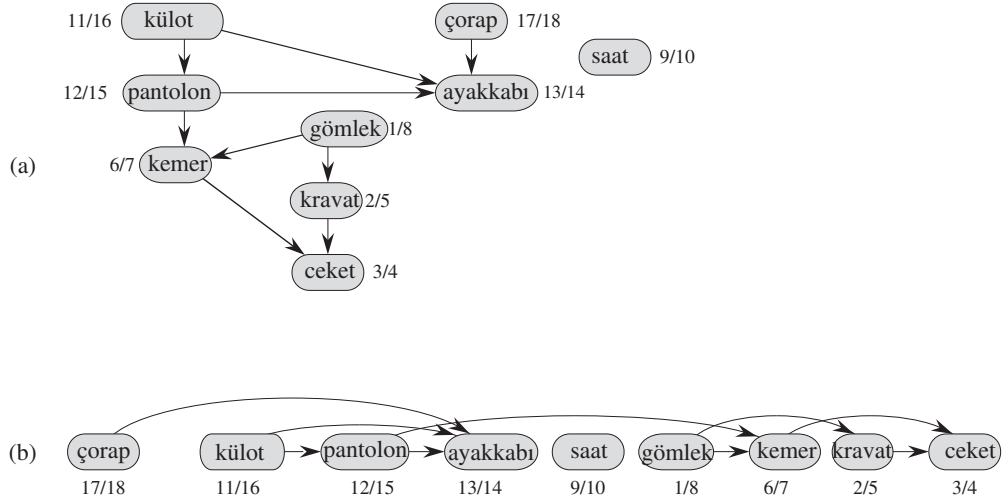
22.3-13 *

Yönlü bir $G = (V, E)$ çizgesi **tek başına bağlı**dır, eğer $u \rightsquigarrow v$ ile, G nin u dan v ye bütün $u, v \in V$ tepeleri için en çok bir basit yolu içerdiği ifade ediliyorsa. Yönlü bir çizgenin tek başına bağlı olması ya da olmamasını belirlemek için verimli bir algoritma veriniz.

22.4 Topolojik sıralama

Bu kısım, kimi kez “işaret” olarak isimlendirilen, yönlü çevrimsiz bir çizgenin topolojik sıralamasını sunabilmek için derinliğine aramanın nasıl kullanılacağını göstermektedir. Bir $G = (V, E)$ çizgesinin **topolojik sıralaması**nın işareti, tüm tepelerin bir doğrusal sıralaması olup, G çizgesinin bir (u, v) ayrıtını içermesi halinde yapılan sıralamada u, v den önce görünmelidir. (Eğer, çizge bir çevre içeriyorsa bir lineer sıralama mümkün değildir.) Bir çizgenin topolojik sıralaması, çizgenin tepelerini yatay bir doğru boyunca, tüm yönlü ayrıtlar soldan sağa olacak şekilde sıralanması şeklinde görülebilir. Topolojik sıralama, II. kısımda incelenen alışlagelmiş “sıralama” türünden biraz farklıdır.

Birçok uygulama, olayları öncelik sırasında göstermek için yönlü çevrimsiz çizgeleri kullanır. Profesör Bumstead’in sabahları giyinirken karşılaştığı durum bir örnek olarak Şekil 22.7 de verilmiştir. Profesör bazı giysileri (örnek olarak, çoraplar, ayakkabılardan önce) diğerlerinden önce giymelidir. Diğer parçalar, her-



Şekil 22.7 (a) Profesör Bumstead üstünü giyinirken topolojik olarak kıyafetlerini sıralamaktadır. Her yönlü ayrıt (u, v) nin anlamı, giysi u nun giysi v den önce giyilmesi. Derinliğine aramanın bulma ve bitiş zamanları bir sonraki herbir tepede gösterilmiştir. (b) Bitiş zamanını azaltmak için tepeleri soldan sağa düzenlenmiş topolojik olarak sıralanmış aynı çizge gösterilmiştir. Bütün yönlü ayrıtlar soldan sağa gitmektedir.

hangi bir sırada giyilebilir (örnek olarak, çoraplar ve pantolonlar). Şekil 22.7(a) nin işaretindeki yönlü bir (u, v) ayrıtı, u giysisinin v giysisinden önce giyilmesi gerektiğini gösterir. Bu işaretin topolojik sıralaması, giyinmenin bir sırasını vermektedir. Şekil 22.7(b) de tepelerin yatay bir doğru boyunca, tüm yönlü ayrıtlar soldan sağa olacak şekilde sıralanması, topolojik sıralanmış işareti göstermektedir.

Bir sonraki temel algoritma bir işareti topolojik olarak sıralar:

TOPOLOGICAL-SORT(G)

- 1 her v tepesi için bitiş zamanı $v.f$ yi hesaplamak için DFS(G) yi çağır
- 2 her tepe bittikçe, onu bağlı listenin önüne ekle
- 3 **return** tepelerin bağlı listesi

Şekil 22.7(b) topolojik olarak sıralanmış tepelerin bitiş zamanlarına göre sondan başa doğru sıralanmasının nasıl görüldüğünü göstermektedir.

Topolojik sıralamanın çalışma zamanı $\Theta(V + E)$ dir, çünkü derinliğine arama $\Theta(V + E)$ zaman alır ve her bir $|V|$ tepenin bağlı listenin önüne eklenmesi de $O(1)$ zaman alır.

Bu algoritmanın doğruluğu, yönlü çevrimsiz çizgeleri ayırt eden sıradaki anahtar lemma ile kanıtlanır.

Lemma 22.11

Yönlü bir G çizgesinin çevrimsiz olması için gerek ve yeter şart G nin derinliğine aramasında geri ayırt olmamasıdır.

Kanıt $\Rightarrow$: Derinliğine aramanın bir (u, v) geri ayırtını ürettiğini kabul edelim. Bu durumda, derinliğine ormanda v tepesi, u tepesinin bir atası olur. Böylece, G çizgesi v den u ya bir yol içerir ve (u, v) geri ayırtı bir çevreyi tamamlar.

$\Leftarrow$: G nin bir c çevresi içerdiğini kabul edelim. G nin bir derinliğine aramasının geri ayırt içerdiğini göstereceğiz. c içinde bulunan ilk tepe v olsun ve (u, v) ayırtı c içinde en önce gelen ayırt olsun. $v.d$ zamanında, c nin tepeleri, v den u ya olan beyaz tepelerin bir yolu formundadır. Beyaz yol teoreminden, u tepesi derinliğine ormanda v nin bir torunu haline gelir. Böylece, (u, v) bir geri ayırttır. ■

Teorem 22.12

Yönlü çevrimsiz çizgenin topolojik sıralamasını, çıktılar olarak üreten TOPOLOGICAL-SORT dur.

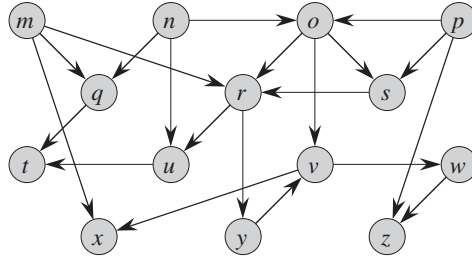
Kanıt Verilen bir $G = (V, E)$ işaretinde tepelerin bitiş zamanını belirlemek için DFS çalıştırılsın. Herhangi $u, v \in V$ ayrık tepe çifti için eğer G , u dan v ye bir ayırt içeriyorsa, $v.f < u.f$ nin olduğunu göstermek yeterlidir. DFS(G) ile keşfedilmiş herhangi bir (u, v) ayırtını ele alalım. Bu ayırt keşfedildiğinde, v gri olamaz, çünkü öyle bir durumda v , u nun bir atası olmalıydı ve (u, v) bir geri ayırt olmalıydı ki, bu durum Lemma 22.11 ile çelişir. Öyleyse v ya beyaz ya da siyah olmalıdır. Eğer v beyaz ise, u nun bir torunu olur ve $v.f < u.f$ bulunur. Eğer v siyah ise, önceden bitirilmiş ve $v.f$ daha önceden kaydedilmiştir. Hala u dan başlanarak arama yapıldığı için $u.f$ ye bir zaman pulu atanmalıdır ve bu bir kez yapıldığında $v.f < u.f$ elde edilmiş olur. Sonuçta, işaret içindeki herhangi bir (u, v) ayırtı için $v.f < u.f$ elde edildiyse, teorem kanıtlanmıştır. ■

Alıştırmalar**22.4-1**

Alıştırma 22.3-2 deki varsayım altında, Şekil 22.8 deki işaret üzerinde çalıştığınızda, TOPOLOGICAL-SORT tarafından üretilen tepelerin sıralamasını gösteriniz.

22.4-2

Yönlü çevrimsiz $G = (V, E)$ çizgesini ve iki tepe s ve t yi girdi olarak alan ve G deki s den t ye olan basit yolların sayısını döndüren doğrusal zamanlı bir algoritma veriniz. Örneğin, Şekil 22.8 deki yönlü çevrimsiz çizge tepe p den tepe v ye kesinlikle dört basit yol içermektedir: $pow, poryv, posryv$, ve $psryv$ (Algoritmanız sadece basit yolları saymalıdır, listeleme yapmamalıdır).



Şekil 22.8 Topolojik sıralama için bir işaret.

22.4-3

Verilen çevre barındıran ya da barındırmayan yönsüz $G = (V, E)$ çizgesini saptayan bir algoritma veriniz. Algoritmanız $|E|$ den bağımsız olarak $O(V)$ zamanda çalışmalıdır.

22.4-4

Kanıtlayın ya da aksini kanıtlayın: Eğer yönlü G çizgesi çevre içeriyorsa, TOPOLOGICAL-SORT(G) üretilen sıralama ile uymayan kötü ayrıtları minimize eden tepe sıralaması üretir.

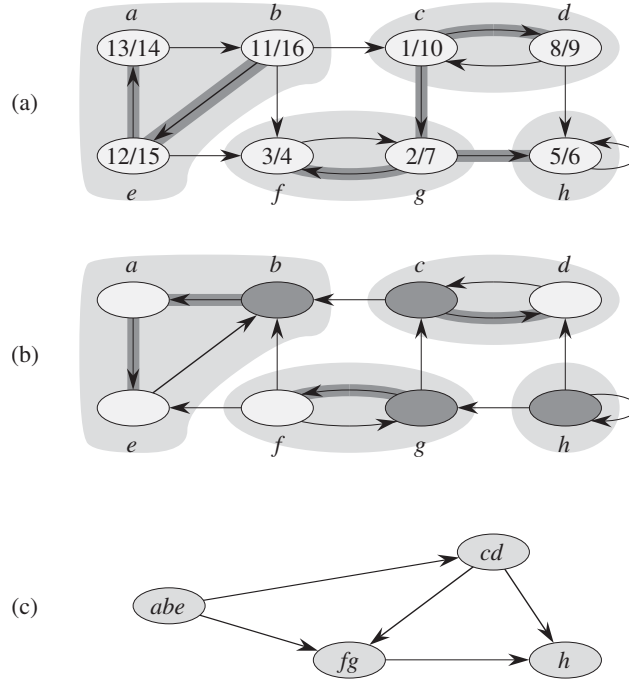
22.4-5

Yönlü çevrimsiz $G = (V, E)$ çizgesinde topolojik sıralamayı gerçekleştirmenin bir diğer yolu, iç derecesi 0 olan tepeleri tekrarlı olarak bulmak, yazdırmak ve çizgenin üzerindeki bütün ayrıtlar ile bu tepeleri silmektir. Bu fikrin nasıl uygulanacağını açıklayınız, öyle ki $O(V + E)$ zamanda çalışsın. Eğer G çevrimli ise bu algoritmaya ne olur?

22.5 Güçlü bağlantılı bileşenler

Derinliğine aramanın klasik bir uygulamasını ele alalım: yönlü bir çizgeyi güçlü bağlantılı bileşenlerine ayırma. Bu kısım, bunun iki tane derinliğine arama kullanılarak nasıl yapılacağını gösterir. Yönlü çizgeler ile çalışan birçok algoritma, böyle bir ayırmayla başlar. Çizge, güçlü bağlantılı bileşenlerine ayrıldıktan sonra bu algoritmalar her biri için ayrı ayrı çalışır ve bileşenler arasındaki bağlantının yapısı ile alakalı olarak sonuçlar birleştirilir.

Ek B den, yönlü bir $G = (V, E)$ çizgesinin güçlü bağlantılı bileşeni, $C \subseteq V$ tepelerinin maksimal kümesidir, öyle ki C deki tüm u ve v tepe çiftleri için $u \rightsquigarrow v$ ve $v \rightsquigarrow u$ vardır ve u ve v tepeleri için birinden diğerine ulaşılabilir. Şekil 22.9 bir örnek göstermektedir.



Şekil 22.9 (a) Yönlü G çizgesi. Her gölgeli bölge G nin güçlü bağlantılı bileşenidir. Her tepe derinliğine aramada bulma ve bitiş zamanlarına göre isimlendirilmiştir ve üç ayrıt gölgelenmiştir. (b) G^T çizgesi G nin hesaplanan derinliğine orman ile satır 3 de STRONGLY-CONNECTED-COMPONENTS inde gösterilen transpozeseidir ve üç ayrıt gölgelenmiştir. Her güçlü bağlantılı bileşen bir derinliğine ağaca karşılık gelir. Koyu olarak gölgelenmiş tepeler b , c , g ve h G^T nin derinliğine aramasından üretilmiş derinliğine ağacın kökleridir. (c) Çevrimsiz bileşenli çizge G^{SCC} G nin güçlü bağlantılı her bileşeninin bütün ayrıtlarının büzüşmesi ile elde edilmiştir; bundan dolayı her bileşende sadece tek bir tepe kalmıştır.

$G = (V, E)$ çizgesinin güçlü bağlantılı bileşenlerini bulmak için algoritmamız, Alıştırma 22.1-3 de tanımlanan, $E^T = \{(u, v) : (v, u) \in E\}$ ile G nin transpozesei olan $G^T = (V, E^T)$ çizgesini kullanır. Yani E^T , G çizgesinin yönlerinin tersi alınmış ayrıtlarından oluşmaktadır. G nin verilen bir bitişliklik listesi gösterimi ile G^T yi oluşturmak için geçen zaman $O(V+E)$ dir. G ve G^T nin aynı güçlü bağlantılı bileşenlere sahip olduğunu görmek ilginçtir: G içinde u ve v nin birbirlerine ulaşılabilirliğinin gerek ve yeter koşulu G^T de de bu iki tepenin birbirine ulaşılabilir olmasıdır. Şekil 22.9(b), Şekil 22.9(a) daki çizgenin transpozeseini göstermektedir.

Aşağıdaki doğrusal zamanlı (örneğin, $\Theta(V + E)$ zamanlı) algoritma $G = (V, E)$ yönlü çizgesinin güçlü bağlantılı bileşenlerini iki tane derinliğine arama kullanarak biri G üzerinde, diğeri G^T üzerinde olacak şekilde hesaplar.

STRONGLY-CONNECTED-COMPONENTS(G)

- 1 Her u tepesi için $u.f$ bitiş zamanını hesaplamak için DFS(G) çağrılır
- 2 G^T yi hesapla
- 3 DFS(G^T) yi çağır, fakat DFS nin ana döngüsünde, $u.f$ nin azalan sırada olan tepelerini göz önüne al (satır 1’de hesaplandığı gibi)
- 4 Satır 3 ün derinliğine ormanındaki her ağacın tepelerinin çıktısı, bir ayrık güçlü bağlantılı bileşen gibidir.

Bu algoritmanın arkasındaki fikir, aşağıda tanımlanan $G^{\text{SCC}} = (V^{\text{SCC}}, E^{\text{SCC}})$ **bileşen çizgesinin** anahtar özelliğinden gelir. G nin güçlü bağlantılı bileşenleri $C_1, C_2, \dots, C_k$ olsun. Tepe kümesi $V^{\text{SCC}}, \{v_1, v_2, \dots, v_k\}$ dir ve G nin her C_i güçlü bağlantılı bileşeni için bir v_i tepesi içermektedir. Bazı $x \in C_i$ ve bazı $y \in C_j$ için eğer G , yönlü bir (x, y) ayrıtını içeriyorsa, bir $(v_i, v_j) \in E^{\text{SCC}}$ ayrıtı vardır. Başka bir açıdan bakıldığında, G nin aynı güçlü bağlantılı bileşenleri içinde olan tüm ayrıtların bitişik tepeleri ile çelişerek elde edilen çizge G^{SCC} olur. Şekil 22.9(c), Şekil 22.9(a) daki çizgenin bileşen çizgesini göstermektedir.

Bir sonraki lemmada belirtildiği gibi, bileşen çizgenin bir işaret olduğu anah-tar bir özelliktir.

Lemma 22.13

C ve C' $G = (V, E)$ yönlü çizgesinin ayrık güçlü bağlantılı bileşeni olsun, $u, v \in C$, $u, v \in C'$ olsun ve $G, u \rightsquigarrow u'$ ya bir yol içersin. G ayrıca $v' \rightsquigarrow v$ yolunu içerebilir.

Kanıt Eğer $G, v' \rightsquigarrow v$ ye bir yol içeriyorsa $u \rightsquigarrow u' \rightsquigarrow v'$ ve $v' \rightsquigarrow v \rightsquigarrow u$ yollarını içersin. Böylece, u ve v' birbirinden ulaşılabilir, dolayısıyla C ve C' güçlü bağlantılı bileşen varsayımı ile çelişir. ■

Birinci derinliğine aramada hesaplanan bitiş zamanının azalan sırasını, ikinci derinliğine aramada tepeleri ele alarak görmeliyiz, esas itibariyle bileşen çizgenin tepelerini (her biri G nin güçlü bağlantılı bileşenine karşılık gelir) topolojik sıralanmış şekilde ziyaret ederiz.

Çünkü STRONGLY-CONNECTED-COMPONENTS süreci iki tane derinliğine arama gerçekleştirir, $u.d$ veya $u.f$ yi tartıştığımızda belirsizlik için bir potansiyel vardır. Bu kısımda, bu değerler her zaman satır 1 de DFS nin ilk çağrısında hesaplanan bulma ve bitiş zamanlarına karşılık gelir.

Tepelerin kümeleri bulma ve bitiş zamanları için gösterimi değiştirebiliriz. Eğer $U \subseteq V$ ise, $d(U) = \min_{u \in U} \{u.d\}$ ve $f(U) = \max_{u \in U} \{u.f\}$ tanımlanabilir. Bu, $d(U)$ ve $f(U)$, U daki herhangi bir tepenin sırasıyla erken bulma zamanı ve geç bitiş zamanıdır.

Aşağıdaki lemma ve onun sonucu güçlü bağlantılı bileşenler ile ilgili anahtar bir özellik ve bitiş zamanları ilk derinliğine aramadaki bitiş zamanlarını verir.

Lemma 22.14

C ve C' , $G = (V, E)$ yönlü çizgesinde ayrık güçlü bağlantılı bileşenler olsun. $u \in C$ ve $v \in C'$ olduğunda, $(u, v) \in E$ şeklinde bir ayrıt olduğunu kabul edelim. Bu durumda, $f(C) > f(C')$ dür.

Kanıt Derinliğine arama sırasında ilk bulunan tepeye sahip olan güçlü bağlantılı bileşenden birisi C veya C' ye bağlı olarak iki durum düşünülmelidir.

Eğer $d(C) < d(C')$ ise, C de bulunan ilk tepe x olsun. $x.d$ zamanında, C ve C' deki tüm tepeler beyazdır. Aynı zamanda, G çizgesi x ten C deki tüm tepelere sadece beyaz tepelerden oluşan bir yol içerir. Çünkü $(u, v) \in E$ olduğundan, herhangi bir $w \in C'$ tepesi için G çizgesinde $x.d$ zamanında x den w ya sadece beyaz tepelerden oluşan bir yol vardır: $x \rightsquigarrow u \rightarrow v \rightsquigarrow w$. Beyaz yol teoreminden, C ve C' içindeki tüm tepeler derinliğine ağaçta x in torunları olur. Sonuç 22.8 den, x torunlarına göre en geç bitiş zamanını elde eder ve $x.f = f(C) > f(C')$ olur.

Eğer $d(C) > d(C')$ ise, y tepesi C' içinde ilk bulunan tepe olsun. $y.d$ zamanında, C'' içindeki tüm tepeler beyazdır ve G, y den başlayan ve sadece beyaz tepelerden oluşan C' içindeki tüm tepeleri kapsayan bir yolu içerir. Beyaz yol teoremine göre, C' içindeki tüm tepeler derinliğine aramada y nin torunu haline gelir ve Sonuç 22.8 e göre $y.f = f(C')$ olur. $y.d$ zamanında, C deki tüm tepeler beyazdır. C den C' ye bir (u, v) ayrıt olduğundan, Sonuç 22.13 C' den C ye bir yol olamayacağını belirtir. Bundan dolayı, C deki hiçbir tepe y 'den erişilebilir değildir. Öyleyse, $y.f$ zamanında, C deki tüm tepeler hala beyazdır. Sonuç olarak, herhangi bir $w \in C$ tepesi için $f(C) > f(C')$ anlamına gelen $w.f > y.f$ elde edilir. ■

Sıradaki sonuç, farklı güçlü bağlantılı bileşenlere giden G^T üzerindeki her ayrıtın, erken bitiş zamanlı (ilk derinliğine arama içinde) bileşenden daha geç bitiş zamanlı bir bileşene gitmesini ifade eder.

Sonuç 22.15

$G = (V, E)$ yönlü çizgesindeki farklı güçlü bağlantılı bileşenler C ve C' olsun. $u \in C$ ve $v \in C'$ iken $(u, v) \in E^T$ biçiminde bir ayrıt olsun. Bu durumda $f(C) < f(C')$ olur.

Kanıt $(u, v) \in E^T$ olduğundan, $(v, u) \in E$ vardır. G ve G^T nin güçlü bağlantılı bileşenleri aynı olduğundan, Lemma 22.14, $f(C) < f(C')$ olduğuna işaret eder. ■

Sonuç 22.15 güçlü bağlantılı bileşenler algoritmasının neden çalıştığını anlamaya yarayan anahtarı sağlamaktadır. G^T üzerinde, ikinci derinliğine arama işlediğinde neler olduğunu değerlendirelim. Bitiş zamanı maksimum $f(C)$ olan C güçlü bağlantılı bileşeni ile başlayalım. Arama herhangi bir $x \in C$ tepesinden başlar ve c deki bütün tepeleri ziyaret eder. Sonuç 22.15 e göre G^T , G nin C den diğer herhangi güçlü bağlantılı bir bileşenine olan hiçbir ayrıtını içermez ve x ten başlayan arama diğer hiçbir bileşenin tepesini ziyaret etmez. Sonuç olarak, x köklü ağaç C nin tepelerini kesinlikle içerir. C nin bütün tepelerinin ziyareti bittiğinde, satır 3 teki arama bir tepeyi kök olarak seçer, C' diğer bazı güçlü bağlantılı bileşenden bitiş zamanı $f(C')$, C den tüm diğer bileşenler üzerinden maksimum olur. Tekrar, arama C' deki tüm tepeleri ziyaret eder, fakat Sonuç 22.15 e göre, sadece G^T deki ayrıtlar C' den diğer herhangi bir bileşene yani daha önce ziyaret ettiğimiz C ye de gitmelidir. Genellikle, satır 3 teki G^T nin derinliğine araması herhangi güçlü bağlantılı bileşeni ziyaret ettiğinde, bu bileşenin dışındaki herhangi ayrıtlar daha önce ziyaret edilen bu bileşene doğru olmalıdır. Her derinliğine ağaç bundan dolayı kesinlikle güçlü bağlantılı bir bileşen olacaktır. Sıradaki teorem, bu argümanı formülize eder.

Teorem 22.16

Yönlü G çizgesinin güçlü bağlantılı bileşenlerini girdi olarak doğru bir şekilde STRONGLY-CONNECTED-COMPONENTS adımları hesaplar.

Kanıt Her ağacın tepelerinin güçlü bağlantılı bir bileşene denk geldiği, G^T nin derinliğine aramasının satır 3 ünden elde edilen derinliğine ağaçların sayısı üzerinde tümevarım yapılması gerektiğini iddia edelim. Tümevarım hipotezine göre, satır 3 de üretilen ilk k ağaç, güçlü bağlantılı bileşendir. Tümevarımın temeli için $k = 0$ olduğunda, durum aşıkardır.

Tümevarım basamağında satır 3 de üretilen ilk k tane derinliğine ağacın her birinin güçlü bağlantılı bir bileşen olduğunu kabul edelim ve $(k + 1)$ ağacın üretildiğini düşünelim. Bu ağacın kökü u tepesi olur ve u , C deki güçlü bağlantılı bileşen içindedir. Derinliğine aramanın satır 3 ünde kökleri nasıl seçeceğimiz verildiğinden, C den farklı henüz ziyaret edilen herhangi bir güçlü bağlantılı bileşen C' için $u.f = f(C) > f(C')$ dir. Tümevarım hipotezine göre arama u yu ziyaret ettiğinde, C nin diğer tüm tepeleri beyazdır. Beyaz yol teoremine göre bu durumda C nin tüm diğer tepeleri derinliğine ağacında u nun torunlarıdır. Hatta, tümevarım hipotezi ve Sonuç 22.15 e göre C yi terk eden G^T içindeki herhangi ayrıtlar önceden ziyaret edilmiş olan güçlü bağlantılı bileşenlere doğru olmalıdır. Sonuç olarak, C

dışındaki herhangi güçlü bağlantılı bileşen içindeki hiç bir tepe, G^T derinliğine araması esnasında u nun torunu olmayacaktır. Böylece tümevarım adımını ve ispatı bitiren u yu kök kabul eden G^T içindeki derinliğine ağacın tepeleri bir tane güçlü bağlantılı bileşen oluşturur. ■

Burada ikinci derinliğine arama işlemlerinin nasıl olduğuna başka bir yoldan bakılmıştır. G^T nin bileşen çizgesi $(G^T)^{SCC}$ olsun. İkinci derinliğine aramada ziyaret edilen her güçlü bağlantılı bileşen $(G^T)^{SCC}$ bir tepesine atanır, topolojik sıralanmış derecenin tersi ile $(G^T)^{SCC}$ nin tepelerini ikinci derinliğine arama ziyaret eder. $(G^T)^{SCC}$ nin ayrıtlarının tersini aldığımızda, $((G^T)^{SCC})^T$ çizgesini elde etmiş oluruz. $((G^T)^{SCC})^T = G^{SCC}$ olduğundan (Alıştırma 22.5-4 e bakınız), ikinci derinliğine arama, topolojik sıralanmış derece ile G^{SCC} nin tepelerini ziyaret eder.

Alıştırmalar

22.5-1

Eğer yeni ayrıt eklenirse, çizgenin güçlü bağlantılı bileşenlerinin sayısı nasıl değişir?

22.5-2

Şekil 22.6 daki çizge üzerinde çalışan STRONGLY-CONNECTED-COMPONENTS adımlarının nasıl çalıştığını gösteriniz. Özel olarak, satır 1 de hesaplanan bitiş zamanını ve satır 3 de üretilen ormanı gösteriniz. DFS'de satır 5-7 de yer alan tepelerin alfabetik sırada olduğunu ve dolayısıyla bitişliklik listesinin de alfabetik sırada olduğunu varsayalım.

22.5-3

Profesör Bacon güçlü bağlantılı bileşenlerin algoritmasının eğer ikinci derinliğine aramada orijinal çizge (transpozese yerine) kullanıldığı ve bitiş zamanını arttıran tepeler tarandığı bir algoritma iddia etmektedir. Bu basit algoritma her zaman doğru sonuçlar verir mi?

22.5-4

Herhangi yönlü bir G çizgesi için $((G^T)^{SCC})^T = G^{SCC}$ olduğunu kanıtlayınız. G^T çizgesinin bileşenlerinin transpozese G çizgesinin bileşenlerinde olduğu gibidir.

22.5-5

Yönlü bir $G = (V, E)$ çizgesinin bileşenlerini hesaplamak için $O(V + E)$ zamanlı bir algoritma veriniz. Algoritmanızın en çok iki tepe arasında bir ayrıt üreteceğinden emin olunuz.

22.5-6

Yönlü bir $G = (V, E)$ çizgesi verildiğinde, başka $G' = (V, E')$ çizgesinin nasıl oluşturulacağını açıklayınız, öyle ki (a) G' , G ile aynı güçlü bağlantılı bileşenlere sahip, (b) G' , G ile aynı bileşen çizgeye sahip, (c) E' mümkün olduğu kadar küçük. G' yü hesaplamak için hızlı bir algoritma tanımlayınız.

22.5-7

Eğer, tüm $u, v \in V$ tepe çiftleri için $u \rightsquigarrow v$ ya da $v \rightsquigarrow u$ var ise, $G = (V, E)$ yönlü çizgesi **yarı-bağlantılı**dır. G nin yarı-bağlantılı olup olmadığını inceleyen verimli bir algoritma veriniz. Algoritmanızın doğru olduğunu kanıtlayınız ve çalışma zamanını analiz ediniz.

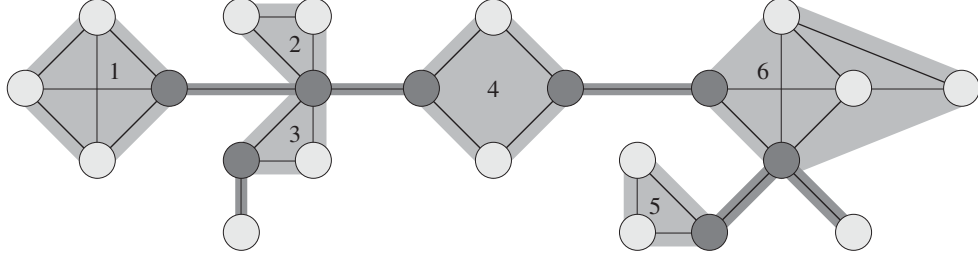
Problemler
22-1 Enine arama ile ayrıtların sınıflandırılması

Bir derinliğine orman, bir çizgenin ayrıtlarını ağaç, arka, ileri ve kesen ayrıtlar olarak sınıflandırır. Bir enine ağaç aynı dört kategori içindeki arama kaynağından ulaşılabilen ayrıtları sınıflandırmak içinde kullanılabilir.

- a.** Yönsüz çizgenin enine aramasında, aşağıdaki özelliklerin tutulduğunu kanıtlayınız:
 1. Hiç arka ayrıt ve ileri ayrıt yoktur.
 2. Her ağaç ayrıt (u, v) için $v.d = u.d + 1$ dir.
 3. Her kesen ayrıt (u, v) için $v.d = u.d$ veya $v.d = u.d + 1$ dir.
- b.** Yönlü çizgenin enine aramasında, aşağıdaki özelliklerin tutulduğunu kanıtlayınız:
 1. Hiç ileri ayrıt yoktur.
 2. Her ağaç ayrıt (u, v) için $v.d = u.d + 1$ dir.
 3. Her kesen ayrıt (u, v) için $v.d \leq u.d + 1$ dir.
 4. Her arka ayrıt (u, v) için $0 \leq v.d \leq u.d$ dir.

22-2 Eklem noktaları, köprüler ve iki-bağlantılı bileşenler

$G = (V, E)$ bağlantılı, yönsüz bir çizge olsun. G nin **eklem noktaları** G den ayrılıp silinen tepelerdir. G nin **köprüsü** G den ayrılıp silinen ayrıtlardır. G nin **iki-bağlantılı bileşeni** ayrıtların azami kümeleridir, öyle ki herhangi iki ayrıt ortak basit çevre üzerinde uzanır. Şekil 22.10 da bu tanımlar gösterilmiştir.



Şekil 22.10 Problem 22-2 de kullanılması için bağlantılı, yönsüz çizgenin eklem noktaları, köprüleri ve iki bağlantılı bileşenleri. Eklem noktaları koyu gölgelenmiş bölgelerdir, köprüler koyu gölgelenmiş ayrıtlardır ve iki bağlantılı bileşenler *bcc* numaralandırmasıyla gösterilmiş gölgeli bölgedeki ayrıtlardır.

Eklem noktaları, köprüler, ve iki-bağlantılı bileşenleri enine arama ile belirleyebiliriz. $G_\pi = (V, E_\pi)$ G nin enine ağacı olsun.

- G_π nin kökünün G nin eklem noktası noktasıdır ancak ve ancak G_π de en az iki ardıla sahiptir ifadesini kanıtlayınız.
- V , G_π nin kökü olmayan bir tepesi olsun. v nin G nin eklem noktası olduğunu kanıtlayınız ancak ve ancak v s ardılına sahip olsun öyle ki s den veya s nin herhangi soyundan v nin uygun atası olan arka ayrıt yoktur.
- Varsayalım

$$v.low = \min \begin{cases} v.d, \\ w.d : (u, w) \text{ } v \text{ nin bazı } u \text{ torunu için bir arka ayrıttır} \end{cases}$$

olsun. $O(E)$ zamanda bütün tepeler $v \in V$ için $v.low$ un nasıl hesaplandığını gösteriniz.

- $O(E)$ zamanda bütün eklem noktalarının nasıl hesaplandığını gösteriniz.
- G nin ayrıtının köprü olduğunu ancak ve ancak G nin herhangi bir basit döngüsünde uzanmadığında.
- $O(E)$ zamanda bütün köprülerin nasıl hesaplandığını gösteriniz.
- İki-bağlantılı bileşenlerin bölünmesi G nin köprü olmayan ayrıtlarını oluşturduğunu kanıtlayınız.
- G nin her e ayrıtını isimlendirmek için pozitif tam sayı $e.bcc$ ile $O(E)$ zamanlı bir algoritma veriniz öyle ki $e.bcc = e'.bcc$ olsun ancak ve ancak e ve e' aynı iki-bağlantılı bileşende ise.

22-3 Euler Turu

Güçlü bağlantılı, yönlü $G = (V, E)$ çizgesinin **Euler turu** G nin her bir ayrıntından bir kere çapraz geçen çevredir, ancak tepeyi birden fazla ziyaret edebilir.

- a. G nin Euler turu olduğunu gösteriniz ancak ve ancak her bir $v \in V$ tepesi için $\text{in-degree}(v) = \text{out-degree}(v)$ dir.
- b. Eğer varsa G nin Euler turunu bulmak için $O(E)$ zamanlı algoritmayı tanımlayınız. (*ipucu*: bütün ayrıntıları birleştiriniz.)

22-4 Ulaşılabilirlik

$G = (V, E)$ her bir $u \in V$ tepesi $\{1, 2, \dots, |V|\}$ kümesinden tek bir tam sayı $L(u)$ ile isimlendirilmi yönlü bir çizge olsun. Her bir $u \in V$ tepesi için $R(u) = \{v \in V : u \rightsquigarrow v\}$ u dan ulaşılabilen tepelerin kümesi olsun. Etiket minimum olan $R(u)$ daki tepe olması için $\min(u)$ yu tanımlayınız demek $\min(u)$ v tepesidir öyle ki $L(v) = \min \{L(w) : w \in R(u)\}$ dir. $u. \in V$ olan bütün tepeler için $\min(u)$ yu hesaplayan $O(V + E)$ zamanlı bir algoritma veriniz.

Bölüm notları

Even[103] ve Tarjan [330] çizge algoritması için mükemmel referanslardır.

Enine arama labirente yol bulma konusunda Moore[260] tarafından keşfedilmiştir. Lee[226] bağımsız olarak devre kartlarında kabloların yolları konusunda aynı algoritmayı keşfetmiştir.

Hopcroft ve Tarjan[178] seyrek çizgeler için bitişiklik matrisi gösterimi üzerinde bitişiklik listesi gösterimini savunmuşlardır ve enine aramanın algoritmik önemini farketmişlerdir. Enine arama 1950 lerin sonuna kadar özellikle yapay zeka programlarında yaygın olarak kullanılmıştır.

Tarjan [327] güçlü bağlantı elemanları bulmak için doğrusal zamanlı bir algoritma vermiştir. Kısım 22.5 teki güçlü bağlantılı elemanlar için olan algoritma Aho, Hopft, ve Ullman[6] dan adapte edilmiştir, övgüsü S. R. Kosaraju [basılmamış] ve M. Sharir[314] e gitmiştir. Gabow[119] güçlü bağlantı elemanlar için büzüşen çevrimler temelli ve doğrusal zaman için iki yığın kullanan bir algoritmayı ayrıca geliştirmiştir. Knuth[209] topolojik sıralama için doğrusal zamanlı algoritmayı veren ilk kişidir.

Elektronik devre tasarımları genellikle çeşitli bileşenlerin uçlarının elektriksel olarak üretiminin, bu bileşenlerin birlikte kablolanmasına eşdeğer olmayı gerektirir. n tane ucu birbirine bağlarken, herbiri iki ucu birbirine bağlayan $n - 1$ tane kablo kullanabiliriz. Tüm bu düzenlemelerle en çok arzu edilen en az miktarda tel kullanılmasıdır.

Bu kablolama problemini, V uçların kümesi, E uç çiftlerinin olası bağlantı kümesi olmak üzere ve her $(u, v) \in E$ ayrıtı için u ve v nin bağlantı maaliyeti (gerekli olan kablo miktarı) $w(u, v)$ yoğunluğu olmak üzere, bağlantılı ve yönsüz bir $G = (V, E)$ çizgesi ile modelleyebiliriz. Daha sonra,

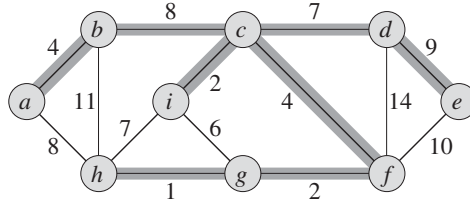
$$w(T) = \sum_{(u,v) \in T} w(u, v)$$

toplam yoğunluğu minimize edilmiş ve bütün tepelerinin birleştirilmiş olduğu çevrimsiz bir alt küme olan $T \subseteq E$ yi bulmak isteriz. T çevrimsiz ve bütün tepeleri birleştirilmiş olduğundan, G çizgesini “kapsadığı” için **kapsayan ağaç** olarak isimlendirdiğimiz bir ağaç oluşturmak gerekir. T ağacını belirleme problemi, **minimum kapsayan ağaç problemi**¹ olarak isimlendirilir. Şekil 23.1 bağlantılı bir çizge ve minimum kapsayan ağaç örneğini göstermektedir.

Bu bölümde, minimum kapsayan ağaç problemini çözmek için iki algoritmayı inceleyeceğiz: Kruskal algoritması ve Prim algoritması. Sıradan ikili bellek yığınları kullanarak kolaylıkla herbirinin çalışma zamanının $O(E \lg V)$ olmasını sağlayabiliriz. Eğer $|V|$, $|E|$ den çok küçükse, ikili-bellek yığınları vasıtasıyla geliştirilen Fibonacci ikili-bellek yığınları kullanılmasıyla Prim algoritmasının çalışma zamanı $O(E + V \lg V)$ olur.

Bölüm 16 da tanımlandığı gibi iki algoritma da açgözlü algoritmalar. Açgözlü algoritmaların her bir basamağı birkaç olası seçenekten birisini yapmalıdır. Açgözlü stratejisi o anda n en iyi seçimi yapmayı savunur. Böyle bir strateji ge-

¹“Minimum kapsayan ağaç” ifadesi “minimum-ağırlıklı kapsayan ağaç” ifadesinin kısaltılmış halidir. Örnek olarak, Teorem B.2 ye göre bütün kapsayan ağaçlar tam olarak olarak $|V| - 1$ ayrıtı sahip olduğu için, T deki ayrıtı sayısını minimize etmeyiz.



Şekil 23.1 Bağlantılı bir çizge için minimum kapsayan bir ağaç. Ağırlıklar ayrıtlar üzerinde gösterilmiştir ve minimum kapsayan ağaçdaki ayrıtlar gölgelendirilmiştir. Ağacın toplam ağırlığı 37 olarak gösterilmiştir. Bu kapsayan ağaç tek değildir: (b, c) ayrıtının silinmesi ve (a, h) ile değiştirilmesi ağırlığı 37 olan başka bir kapsayan ağaç verecektir.

nellikle problemlere her zaman global optimum çözümler bulunacağını garanti etmez. Ancak minimum kapsayan ağaç problemi için belirli açgözlü stratejiler elde edileceğini kanıtlayabiliriz. Bu bölümün Bölüm 16 dan bağımsız olarak okunabilmesine rağmen, burada sunulan açgözlü yöntemler o bölümde tanıtılan teorik kavramların klasik uygulamasıdır.

Kısım 23.1 de bir seferde kapsayan ağaca bir ayrıt eklenmesiyle büyüyen “genel” kapsayan bir ağaç tanıtılmaktadır. Kısım 23.2 de genel yöntem uygulanmış iki algoritma verilmiştir. Birinci algoritma Kruskal’dan dolayı, Kısım 21.1 deki bağlantılı bileşen algoritmasıyla benzerdir. İkincisi Prim’den dolayı, Dijkstra’nın en kısa yol algoritmasını andırmaktadır (Kısım 24.3).

Bir ağaç bir tür çizge olduğundan dolayı, kesin olması için, bir ağacı sadece ayrıtları cinsinden değil, tepeleri cinsinden de tanımlamalıyız. Bu bölüm ayrıtları cinsinden ağaçlar ile ilgilenmesine rağmen, bir T ağacının bazı ayrıtlarıyla bitişik olan T nin tepeleri ile de çalışılacaktır.

23.1 Minimum kapsayan ağacı büyütme

Varsayalım ki, elimizde $w : E \rightarrow \mathbb{R}$ ağırlık fonksiyonlu bağlantılı yönsüz bir $G = (V, E)$ çizgesi olsun ve G için minimum kapsayan bir ağaç bulunmak istensin. Bu bölümde ele aldığımız iki algoritmada problem açgözlü bir yaklaşım kullanmaktadır, fakat algoritmalar bu yaklaşımın uygulanış biçimleri bakımından birbirlerinden farklıdır.

Bu açgözlü strateji, minimum kapsayan ağaca her seferinde bir ayrıt eklenmesiyle büyümesini sağlayan aşağıdaki genel yöntem ile yakalanır. Bu genel yöntem aşağıdaki döngü değişmezini koruyarak ayrıtların bir kümesi olan A yı yönetir :

Her iterasyon öncesi, A herhangi bir minimum kapsayan ağacın alt kümesidir.

Her adımda, bu değişmezi ihlal etmeyerek A ya eklenebilecek (u, v) ayrıtının belirlenmesiyle $A \cup \{(u, v)\}$ aynı zamanda minimum kapsayan bir ağacın bir alt

kümesi olmalıdır. Değişmezliği sağlarken ayrıtı A ya güvenli olarak ekleyebildiğimiz için bu gibi ayrıtları **güvenli ayrıt** olarak adlandırırız.

GENERIC-MST(G, w)

```

1   $A = \emptyset$ 
2  while  $A$  kapsayan bir ağaç formunda değil
3       $A$  için güvenli olan bir  $(u, v)$  ayrıtı bul
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 

```

Döngü değişmezini aşağıdaki gibi kullanırız:

Başlangıç: 1. satırdan sonra, A kümesi döngü değişmezini önemsiz bir şekilde sağlar.

Yineleme: 2-4. satırlardaki döngü sadece güvenli ayrıtları ekleyerek değişmezi muhafaza eder.

Sonlandırma: A ya eklenen tüm ayrıtlar minimum kapsayan bir ağaç içindedir ve böylece 5. satırda dönen A kümesi de minimum kapsayan bir ağaç olmalıdır.

Elbette ki işin ince kısmı 3. satırda güvenli bir ayrıt bulmaktır. 3. satır çalıştırıldığında $A \subseteq T$ olacak şekilde bir T kapsayan ağacın varlığı değişmez tarafından dikte edilmektedir. **While** döngüsü içindeyken, T nin uygun bir alt kümesi A olur ve burada $(u, v) \notin A$ koşulunu sağlayan öyle bir $(u, v) \in T$ ayrıtı vardır ki, (u, v) ayrıtı A için güvenlidir.

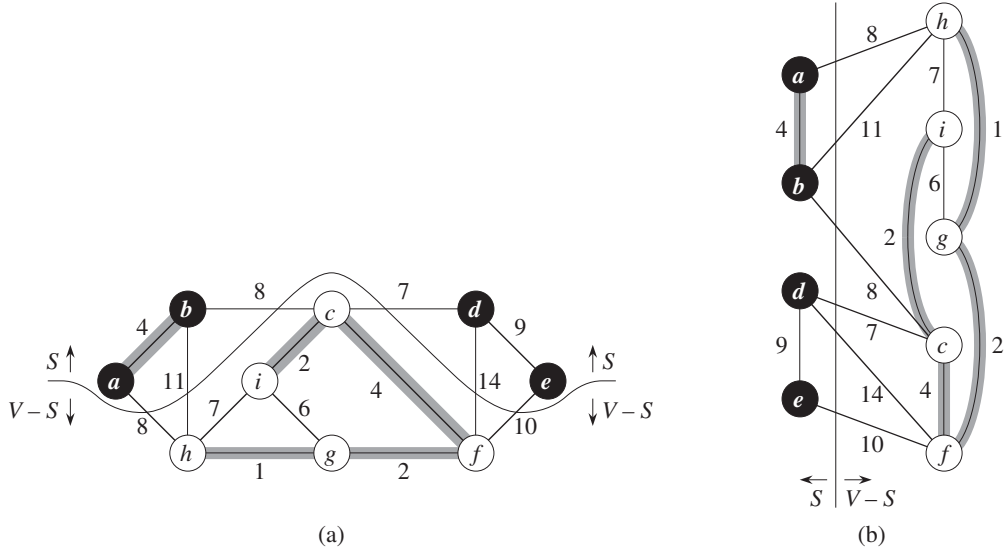
Bu bölümün geri kalanında ise, güvenli ayrıtları tanımlamak için bir kural (Teorem 23.1) sağlayacağız. Bir sonraki bölümde bu kuralı etkin olarak kullanarak güvenli ayrıtları bulan iki algoritma tanımlanacaktır.

Öncelikle bazı tanımlamalara ihtiyacımız vardır. Yönsüz bir $G = (V, E)$ çizgesinin V ye göre parçalanışına bir $(S, V - S)$ **kesimi** denir. Şekil 23.2 bu kavramı göstermektedir. Eğer bir $(u, v) \in E$ ayrıtının bir uç noktası S de diğeri $V - S$ de ise bu ayrıt $(S, V - S)$ kesiminden **geçer** deriz. Eğer A içinde kesimden geçen ayrıtlar yoksa, kesim A ayrıtlar kümesine uyar. Kesimden geçen ayrıtlardan ağırlığı minimum olan herhangi bir ayrıt **hafif ayrıt** olarak isimlendirilir. Bağlantı çeşitleri içinde, kesimden geçen birden fazla hafif ayrıt olabilir. Daha genel olarak diyebiliriz ki, herhangi bir ayrıt için yoğunluğu minimum olma özelliği sağlanıyorsa, o ayrıt bir **hafif ayrıttır**.

Güvenli ayrıtları tanımlamak için kuralımız aşağıdaki teoremle verilir.

Teorem 23.1

$G = (V, E)$ çizgesi, gerçel değerli w ağırlık fonksiyonunun E üzerinde tanımlandığı bağlantılı, yönsüz bir çizge olsun. E nin, G için minimum kapsayan bir ağaç içeren bir alt kümesi A olsun, A ile uyumlu olacak şekilde G nin herhangi bir kesmesi $(S, V - S)$ olsun ve (u, v) , $(S, V - S)$ yi kesen hafif bir ayrıt olsun. Böylece ayrıt (u, v) ayrıtı A için güvenli olur.



Şekil 23.2 Şekil 23.1 deki çizgenin bir $(S, V-S)$ kesimini görmek için iki yol. (a) Siyah tepeler S kümesinin içinde ve beyaz tepeler $V-S$ nin içindedir. Kesimden geçen ayrıtlar beyaz tepeleri siyah tepelere bağlar. (d, c) ayrıtı kesimden geçen tek hafif ayrıttır. Ayrıtların bir altkümesi olan A kümesi gölgelenmiştir: A nın hiç bir ayrıtı kesimden geçmediği için, $(S, V-S)$ kesiminin A ya uyduğu dikkate alınmalıdır. (b) S kümesindeki tepelere sahip aynı çizge sol tarafta ve $V-S$ kümesindeki tepelere sahip aynı çizge sağ taraftadır. Eğer sol taraftaki tepe sağ taraftaki tepeye bağlanırsa, ayrıtı kesimden geçmiş olur.

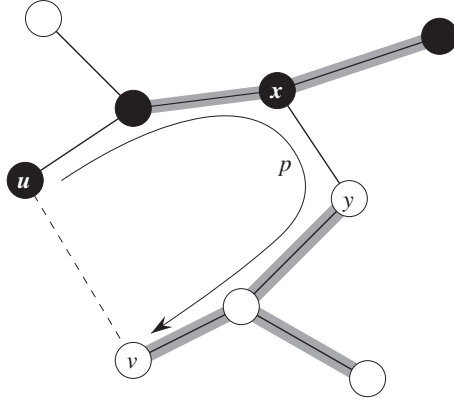
Kanıt A yı içeren minimum kapsayan bir ağaç T olsun ve kabul edelim T ağacı (u, v) hafif ayrıtını içermesin. (u, v) nin A için güvenli ayrıt olduğunu gösterebilmek için, kes-ve-yapıştır tekniği kullanılarak $A \cup \{(u, v)\}$ yı içeren yeni bir T' bir minimum kapsayan ağacı oluşturulmalıdır.

Şekil 23.3 de gösterildiği gibi, (u, v) ayrıtı T içinde u dan v ye basit bir p yolu üzerinde kenarları olan bir çevre oluşturur. u ve v ayrıtları $(S, V-S)$ kesiminin zıt taraflarında olduğundan, T deki en az bir ayrıt basit p yolu üzerinde bulunur ve ayrıca kesimden geçer. Kabul edelim ki (x, y) bu şekildeki herhangi bir ayrıt olsun. (x, y) ayrıtı A nın içinde değildir, çünkü kesim A ya uyar. (x, y) ayrıtı T de u dan v ye tek bir basit yol üzerinde olduğundan, (x, y) yi ortadan kaldırmak T yi iki bileşene ayırır. (u, v) yi eklemek bu bileşenleri yeniden birleştirir ve yeni bir $T' = T - \{(x, y)\} \cup \{(u, v)\}$ kapsayan ağacını oluşturur.

Daha sonra T' nün minimum kapsayan bir ağaç olduğunu göstereceğiz. (u, v) $(S, V-S)$ yi kesen hafif ayrıt (u, v) olduğundan ve (x, y) de bu kesimden geçtiğinden, $w(u, v) \leq w(x, y)$ dir. Bundan dolayı,

$$\begin{aligned} w(T') &= w(T) - w(x, y) + w(u, v) \\ &\leq w(T) \end{aligned}$$

dir.



Şekil 23.3 Teorem 23.1 in kanıtı. Siyah tepeler S içinde ve beyaz tepeler ise $V - S$ içindedir. T minimum kapsayan ağacı içindeki ayrıtlar gösterilmiştir, fakat G çizgesi içindeki ayrıtlar gösterilmemiştir. A daki ayrıtlar gölgelenmiştir ve $(S, V - S)$ kesiminden geçen hafif bir ayrıt (u, v) dir. T içindeki u dan v ye olan tek basit yol olan p üzerindeki ayrıt (x, y) ayrıtıdır. (u, v) yi içeren T' minimum kapsayan ağacına biçim vermek için, T den (x, y) ayrıtını sileriz ve (u, v) ayrıtını ekleriz.

Ancak T minimum kapsayan bir ağaçtır, bundan dolayı $w(T) \leq w(T')$ dir, bu nedenle, T' da minimum kapsayan bir ağaç olmalıdır.

Bu bize (u, v) ayrıtının A için gerçekten güvenli bir ayrıt olduğunu göstermektedir. $A \subseteq T$ ve $(x, y) \notin A$ olduğundan, $A \subseteq T'$ dir; bundan dolayı, $A \cup \{(u, v)\} \subseteq T'$ dir. Sonuç olarak, T' minimum kapsayan bir ağaç olduğundan, (u, v) de A için güvenlidir. ■

Teorem 23.1 bağlantılı bir $G = (V, E)$ çizgesi üzerinde uygulanan GENERIC-MST yöntemini daha iyi anlamamızı sağlar. Yöntem ilerledikçe, A kümesi her zaman çevrimsizdir; aksi halde, A yı kapsayan minimum kapsayan bir ağaç çevre barındırır ki bu bir çelişkidir. Uygulamanın herhangi bir noktasında, $G_A = (V, A)$ çizgesi bir orman ve G_A nın herbir bağlantılı bileşeni bir ağaç olur. (Bazı ağaçlar sadece bir tepe barındırır. Örneğin yöntem başladığında: A boştur ve orman her tepe için bir tane olacak şekilde $|V|$ ağaç barındırır.) Diğer taraftan, A için güvenli herhangi (u, v) ayrıtı, $A \cup \{(u, v)\}$ çevrimsiz olması gerektiğinden G_A nın ayırık bileşenlerini bağlar.

GENERIC-MST içindeki 2-4 satırlarındaki **while** döngüsü $|V| - 1$ defa çalışır. Çünkü her iterasyonda minimum kapsayan ağacın $|V| - 1$ ayrıtından yalnızca birini bulur. Başlangıçta $A = \emptyset$ olduğunda, G_A içinde $|V|$ ağaç vardır ve her iterasyonda 1 eksilir. Orman sadece tek bir ağaç barındırdığında, yöntem sonlanır.

Kısım 23.2 içindeki iki algoritma şimdi verilecek olan Teorem 23.1 in sonucunu kullanır.

Sonuç 23.2

E kümesi üzerinde, w gerçel değerli ağırlık fonksiyonu ile tanımlanan bağlantılı, yönsüz bir çizge $G = (V, E)$ olsun. G için bir minimum kapsayan ağaç içeren A kümesi E nin bir alt kümesi olsun ve $G_A = (V, A)$ ormanı içindeki bağlantılı bileşen (ağaç) $C = (V_C, E_C)$ olsun. Eğer (u, v) C yi G_A içindeki diğer herhangi bir bileşene bağlayan hafif bir ayrıt ise (u, v) ayrıtı A için güvenlidir.

Kanıt $(V_C, V - V_C)$ kesimi A ya uyar ve (u, v) ayrıtı bu kesim için hafif bir ayrıttır. Bundan dolayı, (u, v) ayrıtı A için güvenlidir. ■

Alıştırmalar**23.1-1**

Bağlantılı bir G çizgesinde (u, v) ayrıtı minimum ağırlıklı bir ayrıt olsun. (u, v) ayrıtının G nin bazı minimum kapsayan ağacına ait olduğunu gösteriniz.

23.1-2

Profesör Sabatier Teorem 23.1 in tersini şu şekilde varsaymaktadır. E kümesi üzerinde, w gerçel değerli ağırlık fonksiyonu tanımlanan bağlantılı, yönsüz bir çizge $G = (V, E)$ olsun. G için bir minimum kapsayan ağaç içeren A kümesi E nin bir alt kümesi olsun, A ya uyan G nin herhangi bir kesimi, $(S, V - S)$ olsun ve $(S, V - S)$ yi kesen A için güvenli bir ayrıt (u, v) olsun. Böylece (u, v) ayrıtı kesim için hafif bir ayrıt olur. Profesörün varsayımının yanlış olduğunu karşı bir örnek vererek gösteriniz.

23.1-3

Eğer (u, v) ayrıtı herhangi bir minimum kapsayan ağaç tarafından içeriliyorsa, çizgenin herhangi bir kesiminden geçen hafif bir ayrıt olduğunu gösteriniz.

23.1-4

Bağlantılı bir çizgenin $\{(u, v): \text{öyle bir } (S, V - S) \text{ kesimi vardır ki, } (S, V - S) \text{ den geçen } (u, v) \text{ ayrıtı hafif bir ayrıttır}\}$ şeklinde tanımlanan ayrıtlarının kümesinin minimum kapsayan bir ağaç oluşturmadığına basit bir örnek veriniz.

23.1-5

$G = (V, E)$ bağlantılı çizgesinde herhangi bir çevre üzerindeki maksimum ağırlıklı ayrıt e olsun. G nin aynı zamanda minimum kapsayan bir ağacı olan $G' = (V, E - \{e\})$ çizgesinin minimum kapsayan bir ağaç olduğunu kanıtlayınız. Burada G nin minimum kapsayan bir ağacı vardır ki bu ağaç e yi içermemektedir.

23.1-6

Eğer çizgenin her kesimi için kesimden geçen tek bir hafif ayrıt varsa, çizge tek bir minimum kapsayan ağaca sahiptir. Karşıt örnek vererek bunun zıttının doğru olmadığını gösteriniz.

23.1-7

Eğer çizgenin bütün ayrıtlarının ağırlıkları pozitif ise, tüm tepeleri birbirine bağlayan ve toplam ağırlığı minimum olan herhangi bir ayrıt kümesi mutlaka bir ağaç olur ifadesini tartışınız. Bazı ağırlıkların pozitif olmaması halinde aynı sonucun elde edilemeyeceğini göstermek için bir örnek veriniz.

23.1-8

G çizgesinin minimum kapsayan bir ağacı T olsun ve T nin ayrıtlarının ağırlıklarının sıralanmış bir listesi L olsun. G çizgesinin bir diğer minimum kapsayan ağacı T' için L listesinin aynı zamanda T' nün sıralanmış ayrıt ağırlığı listesi olduğunu gösteriniz.

23.1-9

$G = (V, E)$ çizgesinin minimum kapsayan bir ağacı T olsun ve V nin bir alt kümesi V' olsun. T nin V' tarafından indirgenmiş bir altçizgesi T' olsun ve G nin V' tarafından indirgenmiş bir altçizgesi G' olsun. Eğer T' bağlantılı ise, G' nün minimum kapsayan bir ağacının T' olduğunu gösteriniz.

23.1-10

Bir G çizgesi ve T minimum kapsayan ağacı verilsin. T içindeki ayrıtlardan birinin ağırlığını azalttığımızı farzedelim. G çizgesi için T nin hala minimum kapsayan bir ağaç olduğunu gösteriniz. Daha resmi bir biçimde, G çizgesi için ayrıt ağırlıkları w ağırlık fonksiyonu ile verilen minimum kapsayan bir ağaç T olsun. Bir $(x, y) \in T$ ayrıtı ve pozitif bir sayı olarak k seçilsin ve w' ağırlık fonksiyonu aşağıdaki gibi tanımlansın:

$$w'(u, v) = \begin{cases} w(u, v) & (u, v) \neq (x, y) \text{ ise,} \\ w(x, y) - k & (u, v) = (x, y) \text{ ise.} \end{cases}$$

T nin G için ayrıt ağırlıkları w' ile verilen minimum kapsayan ağaç olduğunu gösteriniz.

23.1-11 *

Bir G çizgesi ve T minimum kapsayan ağacı verilsin, T içinde olmayan ayrıtlardan birisinin ağırlığını azalttığımızı farzedelim. Değiştirilmiş bu çizgenin minimum kapsayan ağacını bulan bir algoritma veriniz.

23.2 Kruskal ve Prim algoritmaları

Bu bölümde, genel yöntem üzerinde tanımlanan iki minimum kapsayan ağaç algoritmasının ayrıntıları üzerinde durulacaktır. İkisi de kendine özgü kurallarla GENERIC-MST nin 3. satırındaki güvenli ayırtı saptamaktadır. Kruskal algoritmasında; A kümesi verilen çizgenin tüm tepelerini içeren bir ormandır. İki ayrık bileşeni birleştiren ayırtlardan en düşük ağırlıklısının eklenmesi, A ya güvenli bir ayırtın eklenmesi anlamına gelir.. Prim algoritmasında; A kümesi tek bir ağaç formundadır. A ya güvenli ayırt ekleme, ağaca bağlı olmayan bir tepenin en düşük ağırlıklı ayırt ile ağaca eklenmesidir.

Kruskal algoritması

Kruskal algoritması ormandaki herhangi iki ağacı birbirine bağlayan en düşük ağırlıklı tüm (u, v) ayırtlarını bularak ve bu güvenli ayırtı ekleyerek ormanı genişletir. (u, v) ayırtı ile birbirine bağlanan iki ağaç C_1 ve C_2 olsun. C_1 i başka bir ağaca bağlayan hafif ayırt bir (u, v) olduğundan, Sonuç 23.2 ye göre C_1 için (u, v) güvenli bir ayırttır. Kruskal algoritması açgözlü algoritma olarak nitelendirilir, çünkü her adımında ormana mümkün olan en düşük olası ağırlıklı ayırtı ekler.

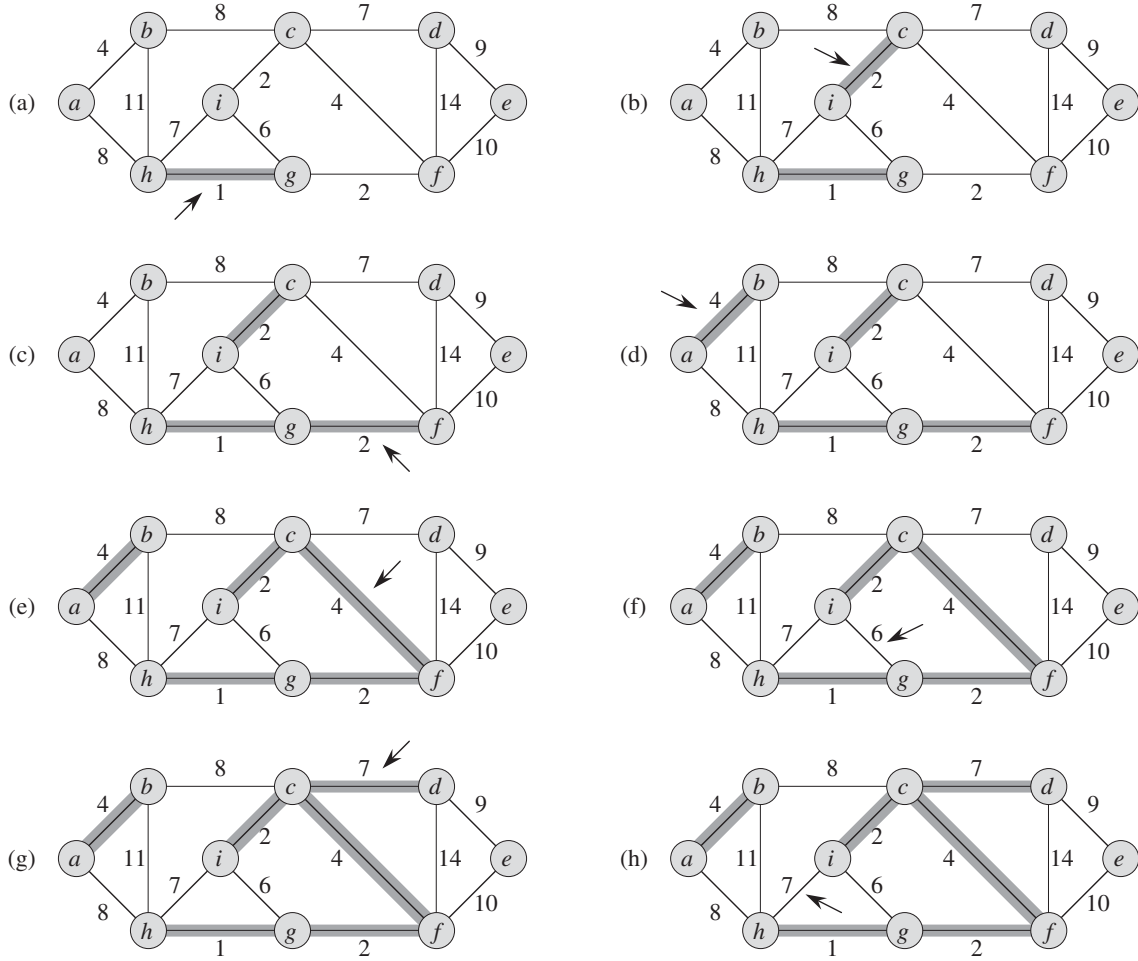
Kruskal algoritmasının uygulaması Kısım 21.1 deki bağlantılı bileşenleri hesaplama algoritmasına benzer. Bu algoritma elemanların çeşitli ayrık kümelerini korumak için ayrık-küme veri yapısını kullanır. Her küme ormandaki bir ağacın içindeki tepeleri içerir. FIND-SET(u) adımı ile u yu içeren bir kümeden temsili bir eleman döndürülür. Böylece u ve v tepelerinin aynı ağaca ait olup olmadığı, FIND-SET(u) nun FIND-SET(v) ye eşitliğinin test edilmesiyle belirlenir. Ağaçları birleştirmek için, Kruskal algoritması UNION işlemini çağırır.

MST-KRUSKAL (G, w)

```

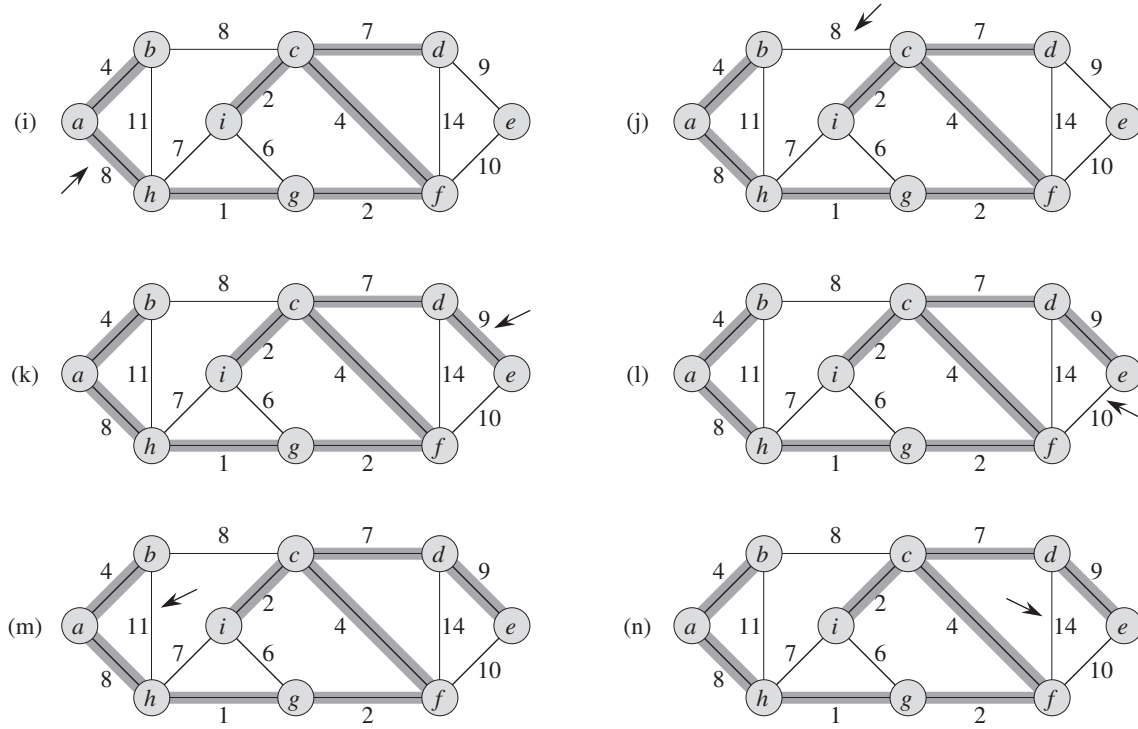
1   $A = \emptyset$ 
2  for her  $v \in G.V$  tepesi
3      MAKE-SET( $v$ )
4   $G.E$  nin ayırtlarını  $w$  ağırlığına göre azalmayan düzende sırala
5  for her  $(u, v) \in G.E$ , ayırtını ağırlığına göre azalmayan düzende içeri al
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Şekil 23.4 Kruskal algoritmasının nasıl çalıştığını göstermektedir. 1-3 satırları A kümesini boş olarak ayarlar ve her biri her tepeyi içerecek şekilde $|V|$ ağaç oluşturur. 5-8 satırlarındaki **for** döngüsü ağırlıkları en düşükten en yükseğe olacak



Şekil 23.4 Şekil 23.1 deki çizge üzerinde Kruskal algoritmasının çalışması. A ormanına ait gölgeyi ayrıtlar kalınlaştırılmıştır. Algoritma her ayrıtlın ağırlık sırasına göre sıralandığını düşünür. Algoritmanın her adımında inşa halindeki ayrıt bir ok ile gösterilmektedir. Eğer ayrıt ormandaki iki farklı ağacı birleştirirse, ormana eklenir, böylece iki ağaç birleşmiş olur.

şekilde ayrıtları inceler. Döngü her (u, v) ayrıtı için, u ve v nin uç noktalarının aynı ağaca ait olup olmadığını kontrol eder. Eğer aynı ağaca aitse, (u, v) ayrıtı çevre oluşturmayacak şekilde ormana eklenmez ve ayrıt atılır. Aksi halde, iki tepe farklı ağaçlara aittir. Bu durumda, 7. satır (u, v) ayrıtını A ya ekler ve 8. satır iki ağaçtaki tepeleri birleştirir.



Şekil 23.4 devamı Kruskal algoritmasının çalışmasındaki ileri adımlar.

$G = (V, E)$ çizgesi için Kruskal algoritmasının çalışma zamanı ayrık-küme veri yapısını nasıl kullandığımıza bağlı olarak değişir. Asimptotik olarak bilinen en hızlı uygulama olmasından dolayı, Kısım 21.3 ün ayrık küme ormanı uygulamasını rank-ile-birleşim ve yol-sıkıştırma-sezgileri ile kullandığımızı varsayalım. 1. satırda A kümesini başlatmak $O(1)$ zamanını alır ve 4. satırda ayrıtları sıralamak $O(E \lg E)$ zamanını alır. (2-3. satırdaki **for** döngüsünde $|V|$ MAKE-SET işlemlerinin maliyetini hemen şimdi hesaba katacağız). 5-8. satırlardaki **for** döngüsü ayrık-küme ormanı üzerindeki FIND-SET ve UNION işlemleri $O(E)$ zamanda gerçekleşir. Kısım 21.4 de tanımlanan çok yavaş büyüyen bir fonksiyon $O((V + E) \alpha(V))$ olmak üzere $|V|$ MAKE-SET işlemleri toplam $O((V + E) \alpha(V))$ zaman alır, G nin birleştirilmiş olduğunu kabul ettiğimizden, elimizde $|E| \geq |V| - 1$ vardır ve bundan dolayı ayrık-küme işlemleri $O(E \alpha(V))$ zaman alır. Hatta, $\alpha(|V|) = O(\lg V) = O(\lg E)$ olduğundan, Kruskal algoritmasının toplam çalışma zamanı $O(E \lg E)$ olur. $|E| < |V|^2$ gözlemlendiğinde, $\lg |E| = O(\lg V)$ olacağından, Kruskal algoritmasının çalışma zamanını yeniden $O(E \lg V)$ olarak yapılandırabiliriz.

Prim Algoritması

Kruskal algoritması gibi Prim algoritması da Kısım 23.1 deki genel minimum kapsayan ağaç yönteminin özel bir halidir. Prim algoritmasının çalışması Kısım 24.3 de göreceğimiz çizgelerde en kısa yolları bulan Dijkstra algoritmasına daha çok benzer. Prim algoritmasının bir özelliği, A kümesi içindeki ayrıtların her zaman tek bir ağaç biçiminde olmasıdır. Şekil 23.5 de gösterildiği gibi, ağaç keyfi bir kök olan r tepesinden başlayarak V içindeki tüm tepeleri kapsayana kadar büyür. A ağacına her adımda, A nın hiçbir ayrıtına bağlı olmayan bir izole tepeyle birleştirecek olan hafif bir ayrıt eklenir. Sonuç 23.2 ye göre, bu kural yalnızca A için güvenli olan ayrıtları ekler; bundan dolayı, algoritma sonlandığında A daki ayrıtlar minimum kapsayan ağaç halini alır. Bu strateji açgözlü olarak sınıflanır, çünkü her adımında ağacın ağırlığına en az katkıda bulunacak bir ayrıt ağaca eklenir.

Prim algoritmasını verimli bir şekilde uygulamak için A içindeki ayrıtlar ile kurulacak olan ağaca eklenecek yeni ayrıtı hızlı şekilde seçtiren bir yola ihtiyacımız vardır. Aşağıdaki kabakodda G çizgesi bağlantılıdır ve algoritmanın girdileri ile genişleyecek minimum kapsayan ağacın kökü r dir. Algoritmanın çalıştırılması sırasında, ağaçta olmayan tüm tepeler bir “key” simgesine dayanarak min-öncelikli bir Q kuyruğunda tutulur. Her v tepesi için ağaçtaki bir tepeyi v ile birleştiren herhangi bir ayrıtın minimum ağırlığı $v.key$ simgesidir; eğer böyle bir ayrıt yoksa $v.key = \infty$ dur. $v.\pi$ simgesi ise v nin ağaçtaki atasının adıdır. Algoritma kesin olarak A kümesini GENERIC-MST den

$$A = \{(v, v.\pi) : v \in V - \{r\} - Q\}$$

şeklinde sürdürür.

Algoritma sonlandığında, min-öncelikli Q kuyruğu boştur. G için minimum kapsayan ağaç A ;

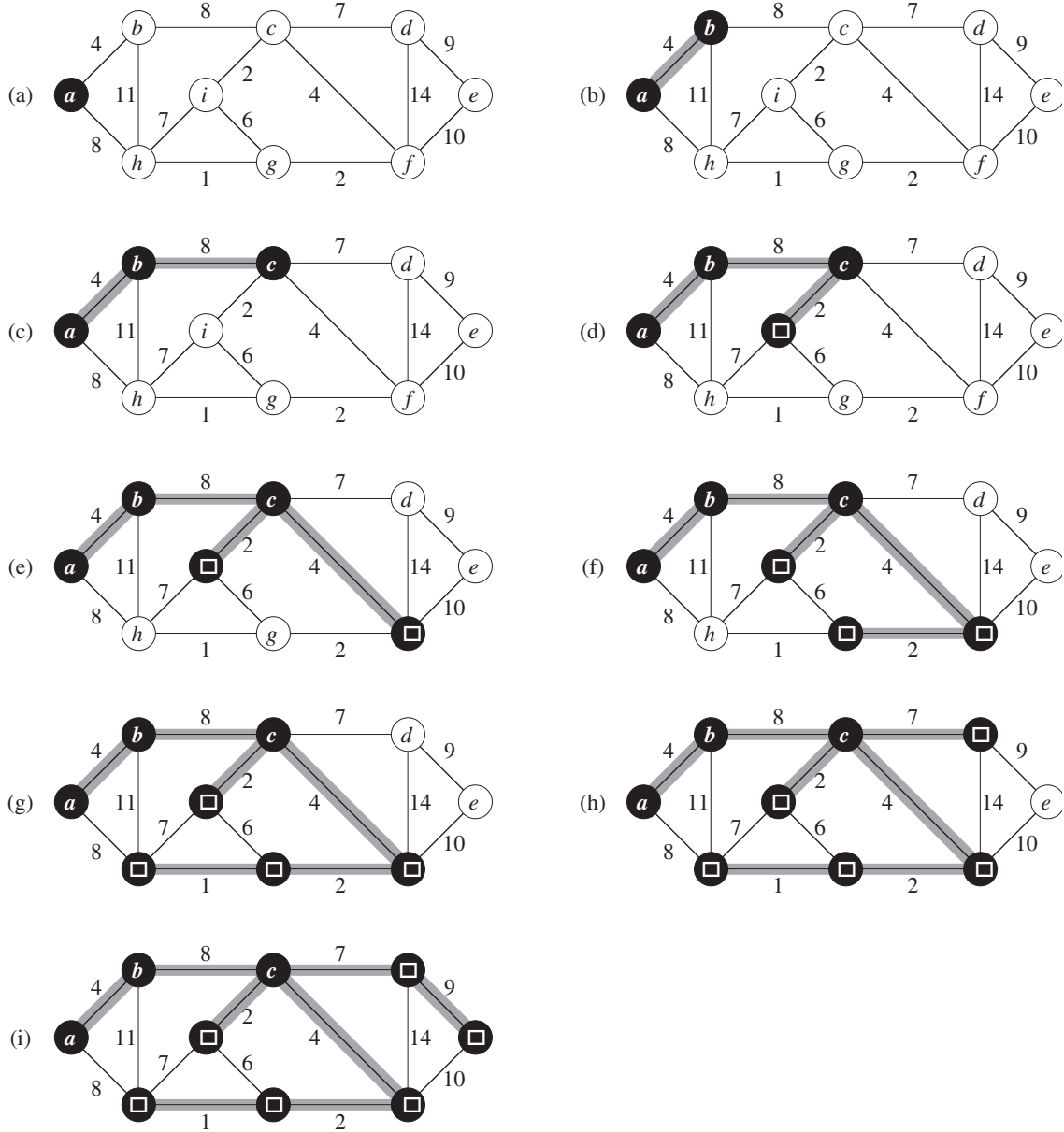
$$A = \{(v, v.\pi) : v \in V - \{r\}\}$$

dir.

MST-PRIM (G, w, r)

```

1  for her  $u \in G.V$ 
2       $u.key = \infty$ 
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = G.V$ 
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8      for her  $v \in G.Adj[u]$ 
9          if  $v \in Q$  her  $w(u, v) < v.key$ 
10              $v.\pi = u$ 
11              $v.key = w(u, v)$ 
```



Şekil 23.5 Şekil 23.1 deki çizgenin üzerinde Prim algoritmasının çalışması. Kök tepe *a* dır. Gölgeleştirilen ayrıtlar büyüyen ağacın içindedir ve siyah tepeler ağacın içindedir. Algoritmanın her adımında, ağaçtaki tepeler çizgenin kesimini belirler ve kesimden geçen hafif bir ayrıtl ağaca eklenir. İkinci basamakta, örneğin, her ikisi de kesimden geçen hafif ayrıtlar olduğundan, algoritma (*b*,*c*) ayrıtlarını yada (*a*, *h*) ayrıtlarını ağaca ekleme seçimine sahiptir.

Şekil 23.5 de Prim algoritmasının nasıl çalıştığı gösterilmektedir. 1-5. satırlarda her bir tepenin key değeri ∞ atanır (key değeri 0 olarak atanan r kökü hariç, bunun nedeni kökün işlenecek ilk tepe olmasıdır), her tepenin ata tepesi NIL olarak atanır ve bütün tepeleri kapsamak için min-öncelikli Q kuyruğu başlatılır. Algoritma aşağıdaki üç parça döngü değişmezini sürdürür:

6-11. satırlardaki her **while** döngü iterasyonu öncesi,

1. $A = \{(v, v.\pi) : v \in V - \{r\} - Q\}$.
2. $V - Q$ içindeki tepeler hali hazırda minimum kapsayan ağaçta bulunan tepelerdir.
3. $v \in Q$ olan bütün tepeler için eğer $v.\pi \neq \text{NIL}$ ise $v.\text{key} < \infty$ ve $v.\text{key}$, v yi minimum kapsayan ağaç içinde bulunan bir tepeye bağlayan $(v, v.\pi)$ hafif ayrıtının ağırlığı olarak atanır.

7. satır $u \in Q$ olacak şekilde $(V - Q, Q)$ kesiminden geçen hafif ayrıta bağlı olan tepeyi tanımlar (4. satırdan dolayı $u = r$ olduğundan, ilk iterasyonun dışında).. Ağaçta u yu Q kümesinden çıkartmak ve $V - Q$ kümesine eklemek, $(u, u.\pi)$ yi A ya eklemek demektir. 8-11. satırlar arasındaki **for** döngüsü, u ya komşu olan ancak ağaçta bulunmayan her v tepesinin key ve π simgelerini günceller, böylece döngü değişmezinin üçüncü kısmı sağlanmış olur.

Prim algoritmasının çalışma zamanı min-öncelikli Q kuyruğunu nasıl uyguladığımıza bağlıdır. Eğer Q için ikili min-bellek yığını uygulanacaksa (Bölüm 6 ya bakınız) 1-5. satırlar arasında $O(V)$ zamanda çalışan BUILD-MIN-HEAP yöntemini kullanabiliriz. **While** döngüsünün gövdesi $|V|$ kez çalışır ve her iterasyonda EXTRACT-MIN işlemi $O(\lg V)$ zaman alır, böylece EXTRACT-MIN nin tüm çağrıları için toplam zaman $O(V \lg V)$ olur. 8-11. satırlardaki **for** döngüsü, bütün komşuluk listelerinin uzunlukları toplamı $2|E|$ olduğu için, hep birlikte $O(E)$ zamanda çalışır. 9. satırdaki **for** döngüsünün içinde, Q için üyelik testi uygulanabilir, her bir tepe için bir bit tutarak sabit zamanda Q da olup olmadığını söyler, tepe Q dan atıldığında bit güncellenir. 11. satırdaki atama $O(\lg V)$ zamanını destekleyen min-bellek yığını üzerinde üstü kapalı bir DECREASE-KEY işlemini içerir. Bundan dolayı Kruskal algoritmasında yaptığımız uygulamanın asimptotik olarak aynısı olan Prim algoritması için toplam zaman $O(V \lg V + E \lg V) = O(E \lg V)$ olur.

Prim algoritmasının asimptotik çalışma zamanı Fibonacci bellek yığınları kullanılarak geliştirilebilir. Bölüm 19 da eğer bir Fibonacci bellek yığını $|V|$ eleman tutuyorsa, EXTRACT-MIN işleminin amortize $O(\lg V)$ zaman ve DECREASE-KEY işleminin (11. satırın uygulaması) amortize $O(1)$ zaman aldığını gösterir. Bu yüzden, min-öncelikli Q kuyruğunun uygulaması için Fibonacci bellek yığınları kullanılacaksa, Prim algoritmasının çalışma zamanı $O(E + V \lg V)$ olarak geliştirilmiş olur.

Alıştırılmalar

23.2-1

Kruskal algoritması ayrıtlar düzenli sıralandığında, bağların nasıl kırıldığına bağlı olarak aynı G çizgesi girdisi için farklı kapsayan ağaçlar verebilir. G nin her minimum kapsayan T ağacı için Kruskal algoritmasının T yi vermesi ve algoritmada G çizgesinin ayrıtlarını sıralamak için bir yol olduğunu gösteriniz.

23.2-2

Bir $G = (V, E)$ çizgesini komşuluk matrisi biçiminde ifade ettiğimizi varsayalım. Prim algoritmasının $O(V^2)$ zamanda çalışan basit bir uygulamasını veriniz.

23.2-3

$|E| = \Theta(V)$ olan seyrek bir $G = (V, E)$ çizgesi için Fibonacci bellek yığını, Prim algoritması uygulaması asimptotik olarak ikili bellek yığını uygulamasından daha hızlı mıdır? $|E| = \Theta(V^2)$ olan yoğun çizge için ne söylenebilir? Fibonacci bellek yığını uygulamasının asimptotik olarak ikili bellek yığını uygulamasından daha hızlı olması için $|E|$ ve $|V|$ nin boyutları ne olmalıdır?

23.2-4

Bir çizgenin bütün ayrıt ağırlıklarının 1 ile $|V|$ arasında tam sayı olduğunu kabul edelim. Kruskal algoritmasının çalışmasını ne kadar hızlı yapabilirsiniz? W herhangi bir tam sayı olmak üzere ayrıt ağırlıkları 1 ile W arasında olsa ne olurdu?

23.2-5

Bir çizgenin bütün ayrıt ağırlıklarının 1 ile $|V|$ arasında tam sayı olduğunu kabul edelim. Prim algoritmasının çalışmasını ne kadar hızlı yapabilirsiniz? W herhangi bir tam sayı olmak üzere ayrıt ağırlıkları 1 ile W arasında olsa ne olurdu?

23.2-6 *

Ayrıt ağırlıklarının bir çizgede düzgün olarak $[0,1)$ yarı açık aralığında dağıldığını varsayalım. Kruskal veya Prim algoritmasından hangisi çalışmayı daha hızlı yapar?

23.2-7 *

Minimum kapsayan ağacı önceden hesaplanmış bir G çizgesi olduğunu varsayalım. Eğer bu çizgeye yeni bir tepe ve bu tepeye bitişik ayrıtları eklersek, minimum kapsayan ağaç ne kadar hızlı güncellenebilir?

23.2-8

Profesör Borden minimum kapsayan ağaçları hesaplamak için aşağıdaki gibi tanımlanan yeni bir böl-ve-yönet algoritması önermiştir. Verilen bir $G = (V, E)$ çizgesinin V tepeler kümesi, $|V_1|$ ve $|V_2|$ değeri birbirlerinden en fazla 1 fazla olacak

biçimde, V_1 ve V_2 şeklinde iki kümeye ayrılışın. E_1 sadece V_1 deki tepelere komşu olan ayrıtların kümesi olsun ve E_2 sadece V_2 deki tepelere komşu olan ayrıtların kümesi olsun. Minimum kapsayan ağaç problemini özyinelemeli bir şekilde $G_1 = (V_1, E_1)$ ve $G_2 = (V_2, E_2)$ altçizgelerinin herbiri için çözünüz. Son olarak, (V_1, V_2) kesiminden geçen E içindeki minimum ağırlıklı ayrıtı seçiniz ve bu ayrıtı kullanarak en son çıkan minimum kapsayan iki ağacı tek bir kapsayan ağaçta birleştiriniz.

G nin minimum kapsayan bir ağacının algoritma ile doğru hesaplandığını ya da algoritmanın başarısız olduğunu gösteren bir örnek veriniz.

Problemler

23-1 İkinci en iyi minimum kapsayan ağaç

$G = (V, E)$ yönsüz, bağlantılı ve ağırlık fonksiyonu $w : E \rightarrow \mathbb{R}$ olan bir çizge olsun, ve bütün ayrıtlarının ağırlıklarının farklı ve $|E| \geq |V|$ olduğunu kabul edelim.

İkinci en iyi minimum kapsayan ağacı aşağıdaki gibi tanımlarız. $\mathcal{T}$ G nin bütün minimum kapsayan ağaçlarının kümesi T ve G nin minimum kapsayan bir ağacı T' olsun. O zaman $w(T) = \min_{T'' \in \mathcal{T} - \{T'\}} \{w(T'')\}$ olmak üzere **ikinci en iyi minimum kapsayan ağaç** bir T kapsayan ağacıdır.

- Minimum kapsayan ağacın tek olduğunu fakat ikinci en iyi minimum kapsayan ağacın tek olmak zorunda olmadığını gösteriniz.
- G nin minimum kapsayan ağacı T olsun. G nin ikinci en iyi minimum kapsayan ağacı $(u, v) \in T$ ve $(x, y) \notin T$ yi ayrıtlarını içerir öyle ki $T - \{(u, v)\} \cup \{(x, y)\}$ olduğunu kanıtlayınız.
- T G nin minimum kapsayan bir ağacı T olsun ve herhangi iki $u, v \in V$ tepesi için $\max[u, v]$ T deki u ve v arasındaki tek basit yol üzerindeki ayrıtın maksimum ağırlığını simgelesin. Verilen T için bütün $u, v \in V$ için $\max[u, v]$ yi hesaplayan $O(V^2)$ zamanlı bir algoritma tanımlayınız.
- G nin ikinci en iyi minimum kapsayan ağacını hesaplamak için etkili bir algoritma veriniz.

23-2 Seyrek çizgelerde minimum kapsayan ağaç

Bağlantılı çok seyrek bir $G = (V, E)$ çizgesi için Prim algoritması çalışmadan önce $(E + V \lg V)$ üzerine Fibonacci bellek yığınları ön işleminden geçerek G nin tepe sayısı azaltılıp Prim algoritmasının $(E + V \lg V)$ olan çalışma zamanı geliştirilmiş olur. Özellikle, her u tepesi için, u ya komşu olan minimum ağırlıklı (u, v) ayrıtını seçeriz ve (u, v) yi yapım aşamasındaki minimum kapsayan ağaca

ekleriz. Sonra seçilen bütün ayrıtları büzeriz (Kısım B.4 e bakınız). Her seferinde bu ayrıtları büzüştürmektense, öncelikle aynı yeni tepede birleşmiş tepeler kümesini tanımlarız. Daha sonra bu ayrıtların her seferinde büzüşmesinden elde edilen çizgesi yaratırız fakat bu işlemi bu kümelerin uç noktalarının bulunduğu yere göre yeniden isimlendirerek yaparız. Orijinal çizgeden bir kaç ayrıt birbiri ile aynı isimlendirilmiş olabilir. Bu durumda, sadece bir ayrıt sonuç olur ve bu ayrıtın ağırlığı ilişkili orijinal ayrıtların ağırlığının minimumu olur.

Başlangıçta, minimum kapsayan ağaç T yi boş olarak oluşturalım ve her $(u, v) \in E$ ayrıtı için $(u, v).orig = (u, v)$ ve $(u, v).c = w(u, v)$ özelliklerini başlangıç durumuna getirelim. *Orig* özelliğini büzüşmüş çizgedeki ayrıt ile ilişkili olan başlangıçtaki ayrıt için kullanırız. c özelliği bir ayrıtın ağırlığını tutar ve ayrıtlar büzüşmüşse ayrıt ağırlıklarını seçmek için yukarıdaki şemaya göre güncellenir. MST-REDUCE işlemi G ve T girdilerini alır ve $orig'$ ve c' güncellenmiş özellikleri ile G' büzüşmüş çizgesini verir. İşlem ayrıca G nin ayrıtlarını T minimum kapsayan ağacı içinde biriktirir. biriktirir.

```

MST-REDUCE( $G, T$ )
1  for her  $v \in G.V$ 
2       $v.mark = FALSE$ 
3      MAKE-SET( $v$ )
4  for her  $u \in G.V$ 
5      if  $u.mark = FALSE$ 
6           $(u, v).c$  yi minimum olacak şekilde  $v \in G.Adj[u]$  seç
7          UNION( $u, v$ )
8           $T = T \cup \{(u, v).orig\}$ 
9           $u.mark = v.mark = TRUE$ 
10  $G'.V = \{FIND-SET(v) : v \in G.V\}$ 
11  $G'.E = \emptyset$ 
12 for her  $(x, y) \in G.E$ 
13      $u = FIND-SET(x)$ 
14      $v = FIND-SET(y)$ 
15     if  $(u, v) \notin G'.E$ 
16          $G'.E = G'.E \cup \{(u, v)\}$ 
17          $(u, v).orig' = (x, y).orig$ 
18          $(u, v).c' = (x, y).c$ 
19     else if  $(x, y).c < (u, v).c'$ 
20          $(u, v).orig' = (x, y).orig$ 
21          $(u, v).c' = (x, y).c$ 
22  $G'$  için  $G'Adj$  bitişiklik listelerini yapılandır
23 return  $G'$  ve  $T$ 

```

- a. MST-REDUCE tarafından verilen ayrıtların kümesi T olsun ve $\text{MST-PRIM}(G', c', r)$ çağırılarak oluşturulan G' çizgesinin minimum kapsayan ağacı olsun. $G'.E$. W ayrıtları üzerindeki ağırlık özelliği A c' ve $G'.V$ içindeki herhangi bir tepe r olsun. $T \cup \{(x, y).orig': (x, y) \in A\}$ nun G nin minimum kapsayan bir ağacı olduğunu kanıtlayınız.
- b. $|G'.V| \leq |V|/2$ olmasını tartışınız.
- c. MST-REDUCE un $O(E)$ zamanda çalışması için nasıl uygulanması gerektiğini gösteriniz. (*İpucu:* Basit veri yapıları kullanın).
- d. G nin girdileri ile bir faz tarafından üretilen G' çıktılarını sonraki faza kadar kullanarak ve T deki ayrıtları biriktirerek MST-REDUCE un k fazlarını çalıştırdığımızı varsayalım. k fazları için toplam çalışma zamanının $O(kE)$ olmasını tartışınız.
- e. MST-REDUCE ın k fazlarını çalıştırdıktan sonra, d maddesinde olduğu gibi, Prim algoritmasını $\text{MST-PRIM}(G', c', r)$ yi çağırarak çalıştırdığımızı varsayalım. Burada G' , c' ağırlık özelliği c' ile son faz tarafından döndürülür ve r tepesi $G'.V$ deki herhangi tepedir. Toplam çalışma zamanı $O(E \lg \lg V)$ olacak şekilde k yı nasıl seçtiğinizi gösteriniz. k seçiminizin toplam asimptotik çalışma zamanını minimum'a indirdiğini tartışınız.
- f. $|E|$ nin hangi değerleri ($|V|$ cinsinden) için asimptotik olarak ön işlem yapan Prim algoritması, ön işlem yapmayan Prim algoritmasını yener?

23-3 Darboğaz kapsayan ağaç

Yönsüz bir G çizgesinin T **darboğaz kapsayan ağacı**, G nin bütün kapsayan ağaçları arasında en büyük ayrıt ağırlığı minimum olan ağaçtır. Darboğaz kapsayan ağacın değeri T içindeki maksimum ağırlıklı ayrıtın ağırlığıdır.

- a. Minimum kapsayan bir ağacın darboğaz kapsayan ağaç olduğunu tartışınız.
 - (a) maddesi darboğaz kapsayan ağacı bulmanın minimum kapsayan bir ağacı bulmaktan zor olmadığını göstermektedir. Geri kalan maddelerde ise, darboğaz kapsayan ağacı doğrusal zamanda nasıl bulacağımızı göstereceğiz.
- b. Verilen bir G çizgesi ve b tam sayısı için darboğaz kapsayan ağacın değerinin en çok b olduğu doğrusal zamanlı bir algoritma veriniz.
- c. (b) maddesindeki algoritmanızı darboğaz kapsayan ağaç problemi için doğrusal zamanlı algoritma içinde bir altprogram olarak kullanınız. (*İpucu:* Problem 23-2 deki MST-REDUCE işleminde tanımlandığı gibi ayrıt kümelerini büzüştüren altprogramı kullanmak isteyebilirsiniz).

23-4 Alternatif minimum kapsayan ağaç algoritmaları

Bu problemde üç farklı algoritma için kabakod vereceğiz. Herbiri bağlantılı bir çizge ve bir ağırlık fonksiyonunu girdi olarak alır ve bir T ayrıtlar kümesini verir. Her algoritma için, T nin minimum kapsayan bir ağaç olduğunu veya T nin minimum kapsayan bir ağaç olmadığını kanıtlayınız. Ayrıca minimum kapsayan bir ağacı hesaplasın ya da hesaplamasın, her algoritmanın en etkin uygulamasını tanımlayınız.

a. MAYBE-MST-A(G, w)

```

1   $w$  ayrıt ağırlıklarının artmayan sırasına göre ayrıtları sırala
2   $T = E$ 
3  for her  $e$  ayrıtı, ağırlığına göre artmayan sırada al
4      if  $T - \{e\}$  bağlantılı bir çizge
5           $T = T - \{e\}$ 
6  return  $T$ 
```

b. MAYBE-MST-B(G, w)

```

1   $T = \emptyset$ 
2  for her  $e$  ayrıtı keyfi sırada alınsın
3      if  $T \cup \{e\}$  çevre içermez
4           $T = T \cup \{e\}$ 
5  return  $T$ 
```

c. MAYBE-MST-C(G, w)

```

1   $T = \emptyset$ 
2  for her  $e$  ayrıtı keyfi sırada alınsın
3       $T = T \cup \{e\}$ 
4      if  $T$   $c$  çevresi içeri
5           $c$  üzerindeki maksimum-ağırlıklı ayrıt  $e'$  olsun
6           $T = T - \{e'\}$ 
7  return  $T$ 
```

Bölüm notları

Tarjan [330] minimum kapsayan ağaç problemini araştırmıştır ve mükemmel gelişmiş malzeme sağlamıştır. Graham ve Hell [151] minimum kapsayan ağaç probleminin tarihçesini derlemiştir.

Tarjan ilk minimum kapsayan ağaç algoritmasını 1926 yılında O. Boruvka tarafından yazılmış bir makaleye dayandırmıştır. Boruvka'nın algoritması Prob-

lem 23-2 de tanımlanan MST-REDUCE işleminin $O(\lg V)$ iterasyonlarının çalışmasından oluşmaktadır. Kruskal algoritması Kruskal [222] tarafından 1956'da sunulmuştur. Prim algoritması olarak bilinen algoritmanın aslında Prim [285] tarafından bulunduğu düşünülür, fakat aslında bu algoritma çok daha önceden 1930 yılında V. Jarnik tarafından bulunmuştur.

Minimum kapsayan ağaç bulmada açgözlü algoritmaların etkili olmasında altının çizilmesi gereken neden; bir çizgesinin ormanlarının kümesinin çizgesel matroid şeklinde olmasıdır. (Bknz. Kısım 16.4)

$|E| = \Omega(V \lg V)$ olduğunda, Fibonacci bellek yığınları ile uygulanmış Prim algoritması $O(E)$ zamanda çalışır. Seyrek çizgeler için Prim algoritması, Kruskal algoritması ve Boruvka algoritmalarının fikirlerinin kombinasyonu ile ileri veri yapılarını birlikte kullanarak, Fredman ve Tarjan [114] $O(E \lg^* V)$ zamanda çalışan bir algoritma vermişlerdir. Gabow, Galil, Spencer ve Tarjan [120] bu algoritmayı $O(E \lg \lg^* V)$ zamanda çalışacak şekilde geliştirmişlerdir. Chazelle [60], $\hat{\alpha}(E, V)$ Ackermann fonksiyonunun fonsiyonel tersi olmak üzere $O(E \hat{\alpha}(E, V))$ zamanda çalışan algoritmayı vermiştir. (Ackermann fonksiyonu ve tersinin özeti için Bölüm 21 deki bölüm notlarına bakınız.) Önceki minimum kapsayan ağaç algoritmalarının aksine, Chazelle algoritması açgözlü yöntemini kullanmaz.

İlişkili bir problem olan **kapsayan ağaç doğrulaması**nda, verilen bir $G = (V, E)$ çizgesi ve bir $T \subseteq E$ ağacı olduğunda T nin G grafinin minimum kapsayan ağacı olup olmadığını belirlemek istiyoruz. King [203] kapsayan ağacı doğrulamak için, Komlos [215] ve Dixon, Rauch ve Tarjan [90] ın önceki işlerini derleyerek doğrusal zamanlı bir algoritma vermiştir.

Yukarıdaki algoritmalar deterministik algoritmalar ve Bölüm 8 de tanımlanmış karşılaştırma tabanlı modelde yer almaktadır. Karger, Klein ve Tarjan [195] $O(V + E)$ beklenen zamanda çalışan rassal minimum kapsayan ağaç algoritması vermişlerdir. Bu algoritma Kısım 9.3'deki doğrusal-zaman seçim algoritması ile benzer tavrıda özyineleme kullanmaktadır: dış problemdeki bir özyineleme çağrısı herhangi bir minimum kapsayan ağaç içinde olmayan ayrıtlarının bir alt kümesi olan E' yü tanımlar. tanımlar. $E - E'$ üzerine yapılan diğer özyinelemeli bir çağrıdan sonra minimum kapsayan ağaç bulunur. Algoritma ayrıca kapsayan ağaç doğrulaması için Boruvka algoritması ve King algoritmalarının fikirlerini kullanır.

Fredman ve Willard [116] karşılaştırma tabanlı olmayan deterministik bir algoritma kullanarak $O(V + E)$ zamanda minimum kapsayan ağacın nasıl bulunacağını göstermiştir. Onların algoritmaları verilerin b -bit tam sayı olduğunu ve bilgisayar hafızasının adreslenebilir b -bit kelimelerden oluştuğunu kabul etmektedir.

Profesör Patrik Phoenix'den Indianapolis'e en kısa güzergâhı bulmak ister. Her komşu kavşak çiftinin arasındaki uzaklığın işaretlendiği Birleşik Devletlerin bir yol haritası verilerek, Profesör bu en kısa güzergâhı nasıl belirleyebilir?

Her güzergâhın uzaklığını toplayarak ve en kısa olanı seçerek Phoenix'den Indianapolis'e olan tüm güzergâhları numaralandırmak olası bir yol olabilir. Bunu görmek kolay, fakat geçit vermeyen çevrim içeren güzergâhlar, Profesör Patrick'in incelemek zorunda olduğu çok sayıda olasılığın basitçe birçoğu göz önüne almaya değer değildir. Örneğin, Seattle'dan geçen Phoenix'den Indianapolis'e olan güzergâhlardan biri açıkça zayıf bir seçenektir çünkü Seattle yolun birkaç yüz mil dışındadır.

Bu bölümde ve Bölüm 25 de bu tür problemlerin verimli bir şekilde çözümünü gösteriyoruz. Bir **en kısa-yol problem**inde (shortest-paths problem), ağırlıklı, yönlü $G = (V, E)$ çizgesinin $w : E \rightarrow \mathbb{R}$ ağırlık fonksiyonun değerleri gerçek ayrıt ağırlıklarıyla eşlenir. $p = \langle v_0, v_1, \dots, v_k \rangle$ yolunun $w(p)$ **ağırlığı** (weight) yolu oluşturan her ayrıtın ağırlıkları toplamıdır:

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

u dan v ye olan $\delta(u, v)$ **en kısa-yolun ağırlığı** (*shortest-path weight*) aşağıdaki gibi tanımlanır:

$$\delta(u, v) = \begin{cases} \min\{w(p) : u \xrightarrow{p} v\}, & u \text{ ile } v \text{ arasında yol varsa,} \\ \infty, & \text{aksi halde,} \end{cases}$$

u tepesinden v tepesine olan bir **en kısa-yol** (shortest path) ağırlığı $w(p) = \delta(u, v)$ olan herhangi bir p yolu olarak tanımlanır.

Phoenix'den Indianapolis'e olan örnekte yol haritasını çizge olarak modelleyebiliriz: tepeler kavşakları, ayrıtlar kavşaklar arasındaki yol kısımlarını ve ayrıt ağırlıkları yol uzunluklarını gösterir. Bizim hedefimiz Phoenix'te verilen bir kavşaktan Indianapolis'te verilen bir kavşağa olan en kısa yolu bulmaktır.

Ayrıt ağırlıkları uzaklıklardan farklı başka ölçüleri gösterebilir, örneğin zaman, maliyet, genişlik, zarar veya yol boyunca doğrusal olan birikimler ve minimize etmek istediğimiz herhangi bir nicelik olabilir.

Kısım 22.2 den enine-arama algoritması ağırlıklı çizgelerde çalışan yani her ayrıtın birim ağırlığının olduğu bir en kısa yol algoritmasıdır. Çünkü enine-aramadan birçok kavram ağırlıklı çizgelerde en kısa yol çalışmalarında ortaya çıkmıştır, ilerlemeden önce Kısım 22.2 yi incelemek isteyebilirsiniz.

Varyantlar

Bu bölümde, **tek-kaynaklı en-kısa-yol problemlerine** (single-source shortest-paths problem) odaklanacağız: verilen bir $G = (V, E)$ çizgesi için verilen bir $s \in V$ **kaynak** (source) tepeden diğer bütün $v \in V$ tepelerine olan en kısa yolu bulmak istiyoruz. Tek-kaynaklı problemi için olan algoritma aşağıdaki seçenekleri içeren diğer birçok problemi çözebilir.

Tek-hedefli en-kısa-yol problemi (Single-destination shortest-paths problem):

Her v tepesinden verilen t **hedef** (destination) tepeye en kısa yolu bulmaktır. Çizgedeki tüm ayrıtların yönleri geri çevrilerek bu problem tek-kaynaklı probleme indirgenebilir.

Tek-ikili en-kısa-yol problemi (Single-pair shortest-path problem): Verilen u ve

v tepeleri arasındaki en kısa yolu bulmaktır. Eğer u kaynak tepesi için tek-kaynaklı problemi çözersek bu problemi de çözmüş oluruz. Ayrıca bu problem için bilinen tüm algoritmalar en iyi tek-kaynaklı algoritmalar ile aynı en-kötü-durum asimptotik çalışma sürelerine sahiptir.

Tüm-ikililerin en kısa-yol problemi (All-pairs shortest-paths problem): Her u

ve v tepe ikilileri için u dan v ye en kısa yolu bulmaktır. Bu problemi her tepe için bir kere tek-kaynaklı algoritmasını çalıştırarak çözebilmemize rağmen, biz genellikle bunu daha hızlı çözeriz. Ek olarak, yapısı kendi içinde ilginçtir. Bölüm 25 de her-ikili probleminin detayı gösterilmiştir.

En kısa yolun en uygun altyapısı

Genellikle en-kısa-yol algoritmaları iki tepe arasındaki en kısa yol içinde diğer en kısa yolları içermesi özelliğine dayanır. (Bölüm 26 deki Edmonds-Karp'ın maksimum-akış algoritması da bu özelliğe dayanır.) Hatırlarsak, en uygun alt yapı için dinamik program (Bölüm 15) ve açgözlü yöntem (Bölüm 16) tek anahtar gösterge olarak uygulanabilir. Kısım 24.3 te görülebilen Dijkstra'nın algoritması bir açgözlü algoritmadır ve Floyd-Warshall algoritması her tepe ikilisi arasındaki en kısa yolları bulan (bakınız Kısım 25.2) dinamik-programlama algoritmasıdır. Aşağıdaki lemma en kısa yolların daha açık olarak en-uygun-altyapı özelliğini belirler.

Lemma 24.1 (En kısa yolların alt yolları en kısa yollardır)

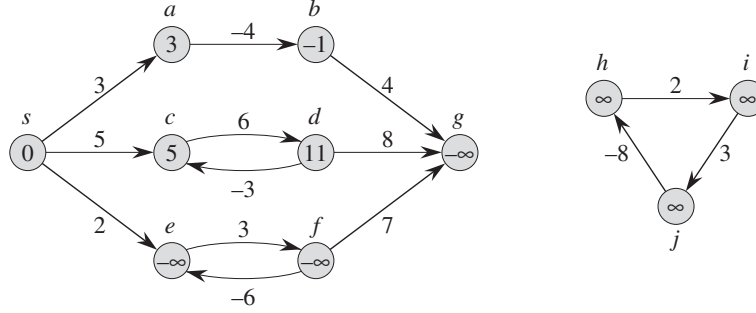
$w : E \rightarrow \mathbb{R}$ ağırlık fonksiyonu ile verilen ağırlıklı, yönlü $G = (V, E)$ çizgesinde $p = \langle v_0, v_1, \dots, v_k \rangle$ yolu v_0 tepesinden v_k tepesine en kısa yol olsun ve her i ve j için $0 \leq i \leq j \leq k$ için $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ v_i tepesinden v_j tepepsine p nin alt yolu olsun. O halde p_{ij} yolu v_i den v_j ye bir en kısa yoldur.

Kanıt Eğer p yolunu $v_0 \xrightarrow{p_{0i}} v_i \xrightarrow{p_{ij}} v_j \xrightarrow{p_{jk}} v_k$ olacak şekilde ayrıştırırsak o zaman $w(p) = w(p_{0i}) + w(p_{ij}) + w(p_{jk}) =$ olur. Şimdi v_i den v_j ye ağırlığı $w(p'_{ij}) < w(p_{ij})$ olan p'_{ij} yolu olduğunu kabul edelim. O halde, v_0 dan v_k ya $v_0 \xrightarrow{p_{0i}} v_i \xrightarrow{p'_{ij}} v_j \xrightarrow{p_{jk}} v_k$ ağırlığı $w(p)$ den daha az olan $w(p_{0i}) + w(p'_{ij}) + w(p_{jk})$ ağırlıklı bir yoldur ki bu p nin v_0 dan v_k ya en kısa yol olduğu varsayımıyla çelişir. ■

Negatif-ağırlıklı ayrıtlar

Bazı tek-kaynaklı en-kısa-yol problemi örnekleri negatif ağırlıklı ayrıtları içerebilir. Eğer $G = (V, E)$ çizgesi s kaynağından ulaşılabilir negatif olmayan ağırlıklı çevrimler içermez ise, o halde her $v \in V$ için negatif değeri olsa bile $\delta(s, v)$ en-kısa-yol ağırlığının kalanı iyi tanımlanmıştır. Eğer çizgede s den ulaşılabilen negatif ağırlıklı çevrimler içerir ise, bununla birlikte en-kısa-yolların ağırlıkları iyi tanımlanmış olmaz. s den bir tepeye çevrim üzerindeki hiçbir yol en-kısa-yol değildir – sürekli “en kısa” yolu ve sonra negatif-ağırlıklı çevrim bağlantısını önerme yoluyla en küçük ağırlıklı yolu bulmaya çalışıyoruz. Eğer s den v ye yollar üzerinde bazı negatif-ağırlıklı çevrimler var ise, $\delta(s, v) = -\infty$ olarak tanımlarız.

Şekil 24.1 negatif ağırlıkların etkilerini ve en-kısa-yol ağırlıkları üzerindeki negatif-ağırlıklı çevrimlerini örnekler. Çünkü s den a ya yalnızca bir yol vardır ($\langle s, a \rangle$ yolu), $\delta(s, a) = w(s, a) = 3$ tür. Benzer bir şekilde, s den b ye yalnız bir tane yol vardır ve $\delta(s, b) = w(s, a) + w(a, b) = 3 + (-4) = -1$ dir. s den c ye ise çok sayıda yollar vardır: $\langle s, c \rangle, \langle s, c, d, c \rangle, \langle s, c, d, c, d, c \rangle$ ve devam eder gider. Çünkü $\langle c, d, c \rangle$ çevrimi $6 + (-3) = 3 > 0$ ağırlığına sahiptir, $\langle s, c \rangle$ ise s den c ye ağırlığı $\delta(s, c) = w(s, c) = 5$ olan en kısa yoldur. Benzer biçimde, s den d ye olan en kısa yol ağırlığı $\delta(s, d) = w(s, c) + w(c, d) = 11$ olan $\langle s, c, d \rangle$ yoludur. Benzer olarak, s den e ye de birçok yol vardır: $\langle s, e \rangle, \langle s, e, f, e \rangle, \langle s, e, f, e, f, e \rangle$ ve devam eder gider. Çünkü $\langle e, f, e \rangle$ çevrimi $3 + (-6) = -3 < 0$ ağırlığına sahiptir fakat s den e ye herhangi bir en kısa yol yoktur. Negatif-ağırlıklı $\langle e, f, e \rangle$ çevrimini rastgele birçok kez geçme yoluyla geliş güzel büyük negatif ağırlıklı s den e ye olan yollar bulunabilir ve $\delta(s, e) = -\infty$ dur. Benzer biçimde $\delta(s, f) = -\infty$ dur. Çünkü g ye f den ulaşılabilir, ayrıca geliş güzel büyük negatif ağırlıklı s den g ye olan yollar bulunabilir ve $\delta(s, g) = -\infty$ dur. Ayrıca h, i ve j tepeleri negatif-ağırlıklı çevrimlerini oluşturur. Onlar s den ulaşılabilir değildir, bununla birlikte, $\delta(s, h) = \delta(s, i) = \delta(s, j) = \infty$ dur.



Şekil 24.1 Yönlü bir çizgede negatif ayrıt ağırlıkları. s kaynağından en-kısa yol ağırlığı her tepe içinde görünmektedir. e ve f tepelerinin s tepesinden ulaşılabilirliği negatif-ağırlıklı çevre olduğundan onların en-kısa-yol ağırlıkları $-\infty$ dur. g tepesi en-kısa-yol ağırlığı $-\infty$ olan tepeden ulaşılabilir olduğundan, o nun dahi en-kısa-yol ağırlığı $-\infty$ dur. h , i ve j gibi tepeler s den ulaşılabilir değildir ve bundan dolayı onların en-kısa-yol ağırlıkları ∞ dur, buna rağmen onlarda negatif-ağırlık çevresinde yer alırlar.

Yol haritaları örneğinde olduğu gibi bazı en-kısa-yol algoritmaları, örneğin Dijkstra'nın algoritması gibi, girilen çizgenin bütün ayrıt ağırlıklarının negatif olmadığını kabul eder. Bazıları, örneğin Bellman-Ford algoritması gibi, girilen çizgenin ayrıtlarının negatif-ağırlıklı olmasına izin verirler ve kaynaktan ulaşılabilenlere negatif-ağırlıklı çevrimleri olmayana kadar doğru cevap üretirler. Benzer bir şekilde, eğer negatif-ağırlıklı çevrimleri var ise algoritma onu algılayabilir ve var olduğunu bildirir.

Çevrimler

En-kısa-yollar çevrim içerebilir mi? Görüldüğü üzere, negatif-ağırlıklı çevrim içeremez. Nede pozitif-ağırlıklı çevrim içerir, mademki yoldan çevrimler çıkarılıyor, aynı kaynak ve hedef tepeler ve küçük yol ağırlığı ile yol üretilir. Bu, eğer $p = \langle v_0, v_1, \dots, v_k \rangle$ bir yol ve $c = \langle v_i, v(i+1), \dots, v_j \rangle$ bu yolun pozitif-ağırlıklı bir çevrimi olsun (bununla birlikte $v_i = v_j$ ve $w(c) > 0$), o zaman $p' = \langle v_0, v_1, \dots, v_i, v(j+1), v(j+2), \dots, v_k \rangle$ yolu $w(p') = w(p) - w(c) < w(p)$ ağırlığına sahiptir, ve bundan dolayı p v_0 dan v_k ya olan en kısa yol olmayabilir.

Bu sadece 0-ağırlıklı çevrimlerde ayrılır. Ağırlığı aynı olan diğer bir yolu üreten herhangi bir yoldan 0-ağırlıklı çevrimi kaldırabiliriz. Böylece, eğer kaynak tepe s den hedef tepe v ye en kısa yol 0-ağırlıklı çevrimi içerir ise s den v ye bu çevrim olmaksızın başka bir en kısa yol vardır. En kısa yol 0-ağırlıklı çevrimleri içerdiği sürece, biz çevrim-bağımsız en kısa yola sahip olana kadar yoldaki bu çevrimleri durmadan kaldırabiliriz. Bu nedenle, belirsizliği kaybetmeden en kısa yolları bulurken çevrim içermediklerini örneğin onların basit yollar olduğunu kabul edebiliriz. Çünkü $G = (V, E)$ çizgesindeki herhangi bir çevrimsiz yol en çok

$|V|$ tane ayrık tepe içerir, bu aynı zamanda en çok $|V| - 1$ tane ayrıt içerir. Dolayısıyla dikkatimizi en çok $|V| - 1$ ayrıtlı en kısa yollarla sınırlayabiliriz.

En kısa yolların gösterimi

Genellikle sadece en kısa yolların ağırlığının hesaplanmasını değil en kısa yollar üzerindeki tepelerin bilinmesini umarız. Kısım 22.2 de enine ağaçların gösteriminde en kısa yolların nasıl olduğunu benzer şekilde gösteririz. Verilen bir $G = (V, E)$ çizgesinde, her $v \in V$ için ya NIL ya da diğer tepe olan bir $v.\pi$ **öncülü** yönetiriz. Bu bölümdeki en-kısa-yol algoritmaları s den v ye en kısa yollar boyunca v tepesinden başlayarak geriye doğru çalıştırılınca öncülerin zincirinin π ye dayandığını gösterir. Bu nedenle, $v.\pi \neq \text{NIL}$ olan verilen v tepesi için Kısım 22.2 deki PRINT-PATH (G, s, v) yöntemi s den v ye en kısa yolu yazacaktır.

En-kısa yol algoritmasının yürütülmesi esnasında yine de π değerleri en kısa yolları göstermez. Enine aramadaki gibi π değerleriyle türetilen $G_\pi = (V_\pi, E_\pi)$ **öncül altçizgesiyle (predecessor subgraph)** ilgilenmiş olacağız. Burada tekrar, NIL-olmayan öncüler ile G nin tepelerinin kümesi olan tepe kümesi V_π yi s kaynağına ek olarak tanımlarsak:

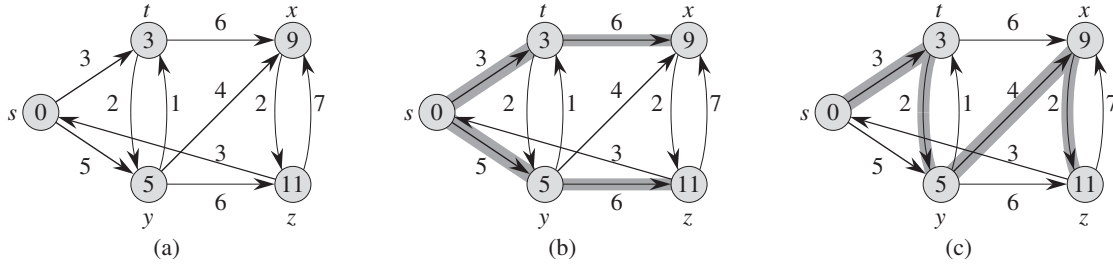
$$V_\pi = \{v \in V : v.\pi \neq \text{NIL}\} \cup \{s\} .$$

Yönlü ayrıtların kümesi E_π V_π tepeleri için π değerleri tarafından türetilen ayrıtlar kümesidir.

$$E_\pi = \{(v.\pi, v) \in E : v \in V_\pi - \{s\}\} .$$

G_π sonlandırmasında bu bölümdeki algoritma tarafından üretilen π değerleri “En-kısa-yol ağacı”dır özelliğine sahip olduğunu, kayıt dışı olarak s den ulaşılabilen her tepeye s kaynağından en kısa yolu içeren köklü ağaç olduğunu kanıtlamalıyız. En-kısa-yol ağacı Kısım 22.2 deki enine-arama ağaç ile benzerdir, fakat ayrıtların sayısı yerine ayrıtların ağırlıkları cinsinden tanımlanan kaynaktan en-kısa-yolları içerir. Tam olarak, $G = (V, E)$ $w : E \rightarrow \mathbb{R}$ fonksiyonuyla ağırlıklı yönlü çizge olsun ve kabul edelim ki G çizgesi $s \in V$ kaynak tepesinden ulaşılabilen G negatif-ağırlıklı çevrimler içermesin, böylece en kısa yollar iyi tanımlanır. Kökü s olan bir **en-kısa-yol ağacı** yönlü bir $G' = (V', E')$ altçizgesidir, $V' \subseteq V$ ve $E' \subseteq E$, öyle ki

1. V' G deki s den ulaşılabilen tepeler kümesidir,
2. G' s kökü ile köklü bir ağaç oluşturur ve
3. Her $v \in V'$ için G' de s den v ye tek basit yol G de s den v ye en kısa yoldur.



Şekil 24.2 (a) s kaynağından en kısa yol ağırlıklarına sahip, ağırlıklı ve yönlü çizge. (b) gölgeli ayrıtlar s kaynağında kökleşmiş ağaç bir en-kısa-yol ağacı oluşturur. (c) aynı köke sahip diğer en-kısa-yol ağacı.

Şekil 24.2 En-kısa-yol tek olmayabilir ve nede en-kısa-yol ağacı değildir. Örneğin, Şekil 24.2 de ağırlıklı, yönlü çizge ve aynı kök ile iki en-kısa-yol ağaçları görülür.

Gevşetme

Bu bölümdeki algoritmalar **gevşetme (relaxation)** tekniğini kullanır. Her $v \in V$ tepesi için s kaynağından v ye bir en-kısa-yolun ağırlıkça üst sınırı olan bir $v.d$ niteliğini savunuruz. $v.d$ yi **en-kısa-yol ölçümü (shortest-path estimate)** olarak adlandırırız. En kısa-yol tahminlerini ve öncülleri aşağıdaki $\Theta(V)$ -zamanlı prosedür ile başlatırız.

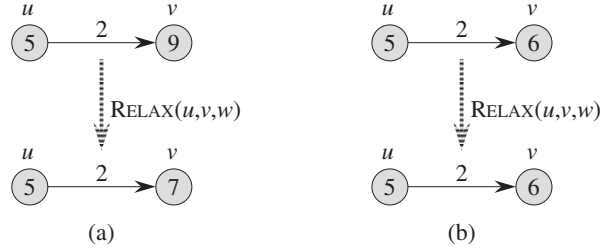
INITIALIZE-SINGLE-SOURCE (G, s)

- 1 **for** $v \in G.V$ her tepesi
- 2 $v.d = \infty$
- 3 $v.\pi = \text{NIL}$
- 4 $s.d = 0$

Başlattıktan sonra, her $v \in V$ için elimizde $v.\pi = \text{NIL}$ vardır, $s.d = 0$, ve $v \in V - \{s\}$ için $v.d = \infty$ dur.

Bir (u, v) ayrıtları **gevşetme (relaxing)** işlemi v ye u üzerinden gidilerek o ana kadar bulunan en kısa yolu iyileştirip iyileştiremeyeceğimizi test etmekten oluşur, eğer öyleyse $v.\pi$ ve $v.d$ güncellenir. Gevşetme adımı¹ en-kısa-yolun $v.d$

¹“gevşetme” üst sınırı daraltma operasyonu olarak kullanılan ilginç bir terim olarak görülebilir. Terimin kullanımı tarihseldir. Gevşetme adımının çıktısı $v.d \leq u.d + w(u, v)$ kısıtının gevşetmesi ki bu üçgen eşitsizliği (Lemma 24.10) den izlenebilir, eğer $u.d = \delta(s, v)$ ve $v.d = \delta(s, v)$ karşılamalı ise. Bu da, eğer $v.d \leq u.d + w(u, v)$, bu kısıtı karşılamak için “baskı” yoktur, dolayısıyla kısıt “gevşetilir”.



Şekil 24.3 Ağırlığı $w(u, v) = 2$ olan (u, v) ayrıtının gevşetilmesi. Her tepe için en kısa-yol tahmini tepe içinde görünür. **(a)** $v.d$ değeri düşer çünkü gevşetmeden önce $v.d > u.d + w(u, v)$. **(b)** Burada, ayrıtı gevşetmeden önce $v.d \leq u.d + w(u, v)$ ve bu yüzden gevşetme adımı $v.d$ yi değiştirmeden bırakır.

tahmin değerini ve v nin öncülünün $v.\pi$ ye atıfta bulunarak güncellemesiyle düşürebilir. Aşağıdaki kod (u, v) ayrıtındaki gevşetmeyi $O(1)$ zamanda gerçekleştirir:

```

RELAX( $u, v, w$ )
1  if  $v.d > u.d + w(u, v)$ 
2       $v.d = u.d + w(u, v)$ 
3       $v.\pi = u$ 

```

Şekil 24.3 te bir ayrıtın gevşetilmesinin iki örneği gösterilmektedir, birinde en kısa-yol tahmini düşürülür ve diğerinde tahmin değişmez.

Bu bölümdeki her algoritma INITIALIZE-SINGLE-SOURCE u çağırır ve sonra ayrıtları tekrar tekrar gevşetir. Ayrıca, gevşetme sadece en kısa yolun tahmini ve öncüllerin değişimi anlamına gelir. Bu bölümdeki algoritmalar her ayrıtın gevşetmesinin kaç kere yapıldığı ve ayrıtların gevşetilmesi sırasına göre farklılık gösterir. Dijkstra'nın algoritması ve yönlü ve çevrimsiz çizgelerin en kısa yol algoritmasında her ayrıt tam bir kere gevşetilir. Bellman-Ford algoritmasında her ayrıt $|V| - 1$ kere gevşetilir.

En kısa-yolların ve gevşetmenin özellikleri

Bu bölümde algoritmaların doğruluğunu kanıtlamak için en kısa-yolların ve gevşetmenin birçok özelliğine başvuracağız. Bu özellikleri burada belirtiyoruz ve Kısım 24.5 te bunlar biçimsel olarak kanıtlanır. Bilginize, burada belirtilen her özellik uygun lemma ya da Kısım 24.5 den sonuç numarası içerir. Bu özelliklerin son 5 tanesi en kısa-yol tahmini veya öncül altçizgesinden bahseder, tamamen çizgenin INITIALIZE-SINGLE-SOURCE(G, s) ile başlatılmış olduğu kabul edilir, ayrıca tek en kısa-yol tahmini ve öncül altçizge değişimi bazı sıralı gevşetme adımlarıyla olur.

Üçgen eşitsizliği (Lemma 24.10)

Her $(u, v) \in E$ ayrıtı için $\delta(s, v) \leq \delta(s, u) + \delta(u, v)$ değeri vardır.

Üst-sınır özelliği (Lemma 24.11)

Her $v \in V$ tepesi için her zaman $v.d \geq \delta(s, v)$ ve bir defa $v.d = \delta(s, v)$ değerine ulaşır ve asla değişmez.

Yol-olmama özelliği (Sonuç 24.11)

Eğer s den v ye yol yok ise, o zaman daima $v.d = \delta(s, v) = \infty$ değerini alır.

Yakınsaklık özelliği (Lemma 24.14)

Eğer bazı $u, v \in V$ için $s \rightsquigarrow u \rightarrow v$ G de bir en kısa yol ise ve eğer (u, v) ayrıtının gevşetmeden önceki herhangi bir zamanda $u.d = \delta(s, u)$ ise, o halde her zaman $v.d = \delta(s, v)$ dir.

Yol-gevşetme özelliği (Lemma 24.15)

Eğer $p = \langle v_0, v_1, \dots, v_k \rangle$ $s = v_0$ dan v_k ya en kısa yol ise ve $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$ düzenindeki p nin ayrıtılarını gevşetirsek, o zaman $v_k.d = \delta(s, v_k)$ dir. Ne olursa olsun p nin ayrıtılarının gevşetmesinin birbirleriyle karışmış olması durumunda bile bu özellik diğer tüm gevşetme adımları oluşmasa da geçerlidir.

Öncül-altçizge özelliği (Lemma 24.17)

Her $v \in V$ için $v.d = \delta(s, v)$ de, öncül altçizge s de köklü en-kısa-yol ağacıdır.

Bölüm taslağı

Kısım 24.1 negatif ağırlıklı ayrıtıları olan tek-kaynaklı en-kısa-yol problemlerini genel durumlarda çözen Bellman-Ford algoritmasını gösterir. Kaynaktan ulaşılabilen negatif-ağırlıklı çevrim olmadığı halde Bellman-Ford algoritması önemli biçimde kolay, daha da ötesinde algılamaya faydalıdır. Kısım 24.2 yönlü çevrimsiz çizgede tek-kaynaktan hesaplanan en-kısa-yol için doğrusal-zamanlı bir algoritma verir. Kısım 24.3 Bellman-Ford algoritmasından daha düşük çalışma zamanına sahip fakat negatif olmayan ayrıtı ağırlıklarını gerektiren Dijkstra'nın algoritmasını kapsar. Kısım 24.4 Bellman-Ford algoritmasını doğrusal programlamanın özel bir durumunu çözmek için nasıl kullanabileceğimizi gösterir. Sonuç olarak, Kısım 24.5 en-kısa-yolların özelliklerini ve yukarıda belirtilen gevşetmeyi kanıtlar.

Bizim sonsuzlukla yapılan işlemleri tartışmamız gereklidir. Kabul edelim ki herhangi bir gerçek sayı $a \neq -\infty$ için, $a + \infty = \infty + a = \infty$ dir. Ayrıca, kanıtlarımızı yapmak için negatif-ağırlıklı çevrimler karşısında, kabul edelim ki herhangi bir gerçek sayı $a \neq \infty$ için $a + (-\infty) = (-\infty) + a = -\infty$ vardır.

Bu bölümdeki tüm algoritmaların G yönlü çizmeyi temsili bitişiklik-listesinde depolandığını kabul eder. Ek olarak her ayrıtı ağırlığı ile depolanır, böylece her bitişiklik listesini dikkatle incelersek, ayrıtı ağırlıklarını her bir ayrıtı için $O(1)$ zamanda belirlenebilir.

24.1 Bellman-Ford algoritması

Bellman-Ford algoritması (Bellman-Ford algorithm) tek-kaynaklı en-kısa-yol problemini ayırıt ağırlıklarının negatif olabileceği genel durumda çözer. Verilen ağırlıklı, s kaynaklı yönlü ve $w : E \rightarrow \mathbb{R}$ ağırlık fonksiyonlu $G = (V, E)$ çizgesi, kaynaktan ulaşılabilir bir negatif-ağırlıklı çevrim olsa da Bellman-Ford algoritması mantıksal değer üretir. Eğer böyle bir çevrim varsa algoritma çözümün var olmadığını belirtir. Eğer böyle bir çevrim yoksa algoritma en kısa yolları ve onların ağırlıklarını üretir.

Algoritma ayrıtları asıl en-kısa-yol ağırlığı $\delta(s, v)$ ye ulaşana dek her $v \in V$ tepesine s kaynağından bir en kısa yolun ağırlığı üzerindeki $v.d$ nin tahminini gittikçe azalarak gevşetir. Algoritma ancak ve ancak kaynaktan ulaşılabilir negatif-ağırlıklı çevrimler içermez ise TRUE çıktısı verir.

BELLMAN-FORD (G, w, s)

```

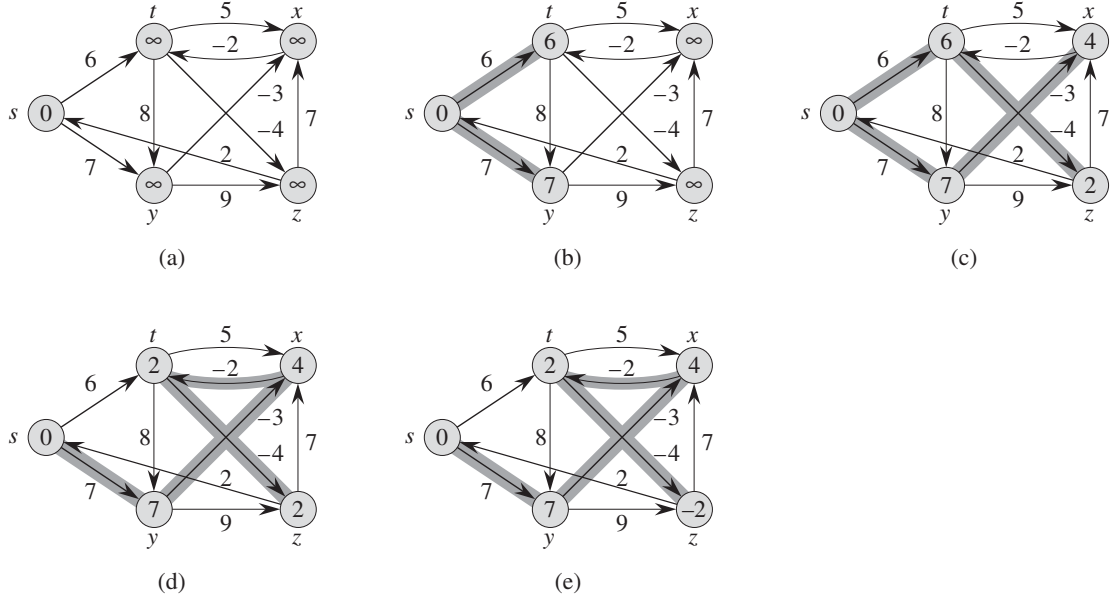
1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2  for  $i = 1$  to  $|G.V| - 1$ 
3      for her ayrıt  $(u, v) \in G.E$ 
4          RELAX( $u, v, w$ )
5  for her ayrıt  $(u, v) \in G.E$ 
6      if  $v.d > u.d + w(u, v)$ 
7          return FALSE
8  return TRUE

```

Şekil 24.4 5 tepeli çizgede Bellman-Ford algoritmasının uygulamasını gösterir. Satır 1 deki bütün tepelerin d ve π değerlerinin sıfırlanmasından sonra, algoritma çizgenin ayrıtlarını üzerinden $|V| - 1$ geçiş üretir. Her geçiş satır 2-4 deki **for** döngüsünün bir tekrarıdır ve çizgenin her ayrıtının bir kez gevşetilmesinden oluşur. Şekil 24.4 (b)-(e) ayrıtlar üzerinden dört kez geçişten sonra algoritmanın durumunu gösterir. $|V| - 1$ geçişin yapılmasından sonra, satır 5-8 ler negatif – ağırlıklı çevreleri kontrol eder ve uygun mantıksal değeri geri bildirir. (kısa süre sonra neden bu kontrol çalışmasını yaptığımızı göreceğiz.)

Bellman-Ford algoritması $O(VE)$ zamanda çalışır, satır 1 deki başlangıç $\Theta(V)$ zaman alır, 2-4 satırlarındaki ayrıtlar üzerinden her $|V| - 1$ geçiş $\Theta(E)$ zaman alır, ve 5-7 satırlarındaki **for** döngüleri $O(E)$ zaman alır.

Bellman-Ford algoritmasının doğruluğunu kanıtlamak için, eğer negatif-ağırlıklı çevre yok ise algoritma en-kısa-yol ağırlıklı çevrimleri kaynaktan ulaşılabilen her tepe için doğru hesapladığını göstermekle başlanır.



Şekil 24.4 Bellman–Ford algoritmasının uygulaması. Kaynak s tepesidir. d değerleri tepeler içinde görünür, ve gölgeli ayrıtlar öncül değerleri gösterir: eğer (u, v) ayrıtı gölgeli ise, o zaman $v.\pi = u$ dir. Bu özel örnekte, her geçiş (t, x) , (t, y) , (t, z) , (x, t) , (y, z) , (z, x) , (z, s) , (s, t) , (s, y) sıralı ayrıtları gevşetir. Ayrıtlar üzerinden hemen önceki ilk geçiş durumu **(a)** dır. Ayrıtlar üzerinden ardışık her geçiş sonrası durumu **(b)–(e)** dir. Kısım **(e)** deki d ve π değerleri son değerlerdir. Bu örnekte Bellman-Ford algoritması TRUE çıktı üretir.

Lemma 24.2

$G = (V, E)$ çizgesi ağırlıklı, s kaynaklı yönlü ve ağırlık fonksiyonu $w : E \rightarrow \mathbb{R}$ ve G nin s den ulaşılabilen negatif-ağırlıklı çevrimler içermediğini kabul edelim. O zaman, BELLMAN-FORD un 2–4 satırlarındaki **for** döngülerinin $|V| - 1$ tane iterasyonundan sonra s den ulaşılabilen her v tepesi için $v.d = \delta(s, v)$ vardır.

Kanıt Lemmayı yol gevşetme özelliğine başvurarak kanıtlarız. s den ulaşılabilen herhangi bir v tepesini ve burada $v_0 = s$ den $v_k = v$ ye olan en kısa yol $p = \langle v_0, v_1, \dots, v_k \rangle$ yolu olsun. Çünkü en kısa yollar kolaydır, p en fazla $|V| - 1$ tane ayrıtı, ve aynı zamanda $k \leq |V| - 1$ dir. 2–4 satırlarının **for** döngüleri her birinin $|V| - 1$ tane iterasyonundan sonra bütün $|E|$ ayrıtları gevşetilir. i inci iterasyon gevşetilmiş ayrıtlar arasında $i = 1, 2, \dots, k$ için (v_{i-1}, v_i) dir. Yol-gevşetme özelliğinden dolayı $v.d = v_k.d = \delta(s, v_k) = \delta(s, v)$ dir. ■

Sonuç 24.3

$G = (V, E)$ ağırlıklı s kaynak tepeli yönlü çizge ve ağırlık fonksiyonu $w : E \rightarrow \mathbb{R}$ ve G nin s den ulaşılabilen negatif-ağırlıklı çevrimler içermediğini kabul edelim. O zaman her $v \in V$ tepesi için s den v ye bir yol vardır ancak ve ancak BELLMAN-FORD G 'de çalıştığında $v.d < \infty$ ile sınırlar ise.

Kanıt Bu kanıt Alıştırma 24.1-2 ye bırakılmıştır. ■

Teorem 24.4 (Bellman–Ford algoritmasının doğruluğu)

BELLMAN-FORD ağırlıklı, yönlü s kaynaklı ve ağırlık fonksiyonu $w : E \rightarrow \mathbb{R}$ olan $G = (V, E)$ çizgesinde çalışsın. Eğer G , s den ulaşılabilen negatif-ağırlıklı olmayan çevrimler içerirse, o zaman algoritma TRUE çıktı verir, her $v \in V$ tepesi için $v.d = \delta(s, v)$ vardır ve öncül altçizge G_π s de köklü bir en-kısa-yol ağacıdır. Eğer G s den ulaşılabilen negatif-ağırlıklı çevrimler içerirse o zaman, algoritma FALSE çıktı verir.

Kanıt Kabul edelim ki G , s den ulaşılabilen negatif-ağırlıklı olmayan çevrimler içersin. İlk olarak her $v \in V$ tepesi için $v.d = \delta(s, v)$ sınırı kanıtlanmalıdır. Eğer v s den ulaşılabilir ise o zaman, Lemma 24.2 bu iddiayı kanıtlar. Eğer v s den ulaşılabilir değil ise o zaman, iddia yol-olmama özelliğinden devam eder. Bu durumda iddia kanıtlanır. Öncül-altçizge özelliği iddia ile birlikte, G_π nin en-kısa-yol ağacı olduğunu belirtir. Şimdi BELLMAN-FORD un TRUE çıktı verdiği gösterimi iddia-sını kullanacağız. Sınırdaki, her $(u, v) \in E$ ayrıtı için

$$\begin{aligned} v.d &= \delta(s, v) \\ &\leq \delta(s, u) + w(u, v) \quad (\text{üçgen eşitsizliğinden}) \\ &= u.d + w(u, v) \end{aligned}$$

ve bu yüzden 6 ıncı satırdaki hiçbir test BELLMAN-FORD un FALSE çıktı vermesine sebep olmaz. Bundan dolayı, TRUE çıktı verir.

Şimdi, G nin s den ulaşılabilen negatif-ağırlıklı çevrimler içerdiğini kabul edelim; ve bu $v_0 = v_k$ nin olduğu $c = \langle v_0, v_1, \dots, v_k \rangle$ çevrimi olsun. O zaman,

$$\sum_{i=1}^k w(v_{i-1}, v_i) < 0. \quad (24.1)$$

Bellman–Ford algoritmasında çelişkinin amacının kabulü için TRUE çıktı vermeli. Bundan dolayı, $i = 1, 2, \dots, k$ için $v_i.d \leq v_{i-1}.d + w(v_{i-1}, v_i)$ dir. c çevriminin etrafındaki eşitsizliklerin toplamı

$$\begin{aligned}
\sum_{i=1}^k v_i \cdot d &\leq \sum_{i=1}^k (v_{i-1} \cdot d + w(v_{i-1}, v_i)) \\
&= \sum_{i=1}^k v_{i-1} \cdot d + \sum_{i=1}^k w(v_{i-1}, v_i)
\end{aligned}$$

verir.

$v_0 = v_k$ olduğundan beri, c deki her tepe $\sum_{i=1}^k v_i \cdot d$ ve $\sum_{i=1}^k v_{i-1} \cdot d$ her bir toplamda açıkça bir kere görünür ve de

$$\sum_{i=1}^k v_i \cdot d = \sum_{i=1}^k v_{i-1} \cdot d.$$

Buna ek olarak, Sonuç 24.3 ten $v_i \cdot d_i = 1, 2, \dots, k$ için sonsuzdur. Bundan dolayı,

$$0 \leq \sum_{i=1}^k w(v_{i-1}, v_i)$$

ki bu (24.1) deki eşitsizlikle çelişir. Sonuçta Bellman-Ford algoritması TRUE geri bildirimde bulunur eğer G çizgesi kaynaktan ulaşılabilen negatif-ağırlıklı çevrimler içermez ise, aksi halde FALSE tur. ■

Alıştırmalar

24.1-1

Şekil 24.4 teki yönlü çizgenin üzerinde Bellman-Ford algoritmasını z tepesini kaynak olarak çalıştırınız. Her geçişte, şekilde aynı düzendeki gevşetilmiş ayrıtları ve d ve π değerlerini her geçişten sonra gösterin. Şimdi, (z, x) ayrıtının ağırlığını 4 olarak değiştirin ve algoritmayı tekrar çalıştırın, s yi kaynak olarak kullanın.

24.1-2

Sonuç 24.3 ü kanıtlayın.

24.1-3

Verilen ağırlıklı, yönlü negatif-ağırlıklı çevrimler içermeyen $G = (V, E)$ çizgesi için, s kaynağından v ye olan en kısa yollardaki ayrıt sayılarının en küçüğünün $v \in V$ nin bütün tepeleri üzerindeki en büyük değeri m olsun. (Burada, ağırlık bakımından en kısa yol, ayrıtların sayısı değil.) Önceden m bilinmese bile, Bellman-Ford algoritmasında $m + 1$ geçişle sınırlamaya izin veren basit değişiklik önerin.

24.1-4

Kaynaktan v ye bazı yollarda negatif-ağırlıklı bir çevrim üzerindeki tüm v tepeleri için $v \cdot d$ yi $-\infty$ olacak şekilde Bellman-Ford algoritmasını değiştirin.

24.1-5 *

Ağırlıklı, yönlü ve ağırlık fonksiyonu $w : E \rightarrow \mathbb{R}$ olan çizge $G = (V, E)$ olsun. $v \in V$ nin her tepesi için $\delta^*(v) = \min_{u \in V} \{\delta(u, v)\}$ değerini bulan $O(VE)$ -zaman algoritması verin.

24.1-6 *

Ağırlıklı, yönlü $G = (V, E)$ çizgesinin negatif-ağırlık çevrimlerine sahip olduğunu varsayalım. Çevrimlerden birinin tepelerini listeleyen verimli bir algoritma verin. Algoritmanızın doğruluğunu kanıtlayın.

24.2 Yönlü, çevrimsiz çizgelerde tek-kaynaklı en kısa yollar

Ağırlıklı dag (directed acyclic graph) $G = (V, E)$ nin ayrıtlarını gevşeterek, tepelerinin bir topolojik sıralamasına göre tek bir kaynaktan en kısa yolları $\Theta(V + E)$ zamanda hesaplayabiliriz. Yönlü, çevrimsiz çizgelerde en kısa yollar her zaman iyi tanımlıdır. Hatta eğer negatif ağırlıklı ayrıtlar varsa bile, negatif ağırlıklı çevrimler olmayabilir.

Tepelerde doğrusal bir sıralamayı (Kısım 22.4 e bakınız.) uygulamak için, algoritma yönlü çevre içermeyen çizgelerin topolojik sıralamasıyla başlar. Eğer yönlü, çevrimsiz çizge u tepesinden v tepesine yol içeriyorsa u v den topolojik sıralamaya göre önde gidiyordur. Topolojik sıralamaya göre tepeler üzerinden sadece bir kez geçilir. Her bir tepeden geçerken, o tepeyi terk ediyor gibi her bir ayrıt gevşetilir.

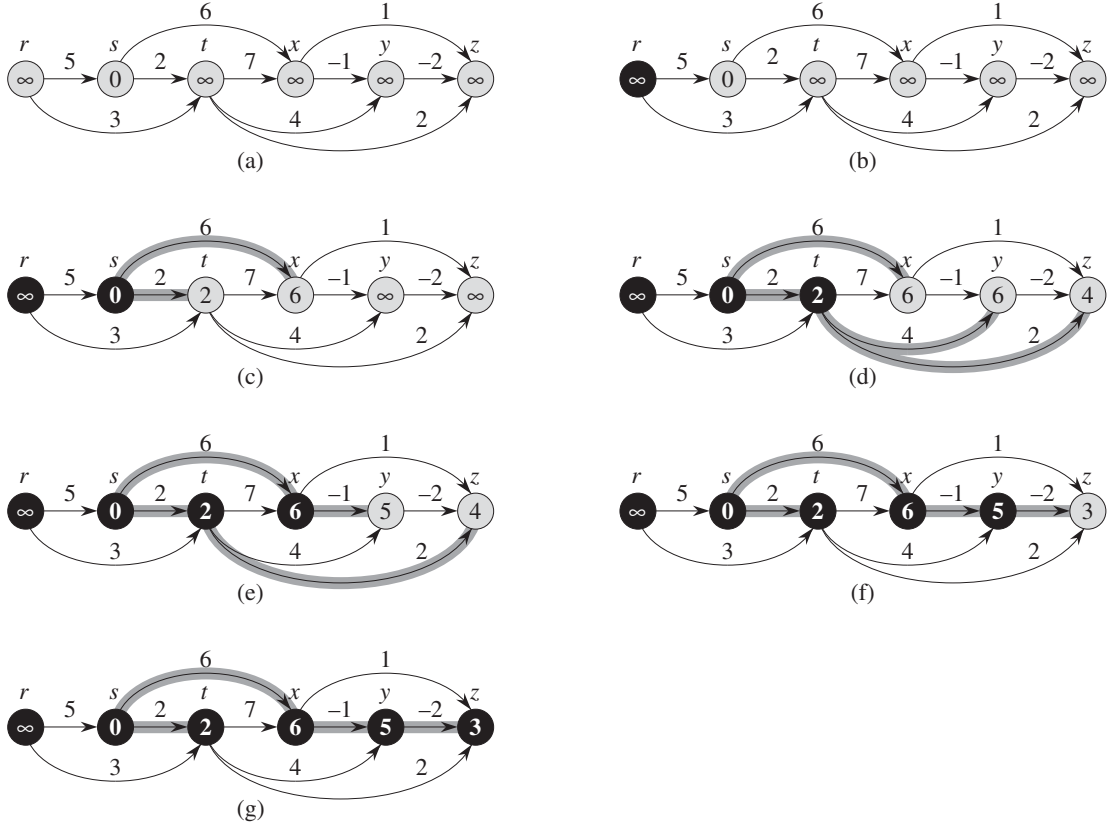
DAG-SHORTEST-PATHS (G, w, s)

- 1 G nin tepelerini topolojik olarak sırala
- 2 INITIALIZE-SINGLE-SOURCE(G, s)
- 3 **for** Topolojik sıralama düzeninden seçilen her u tepesi
- 4 **for** $Adj[u]$ ya bitişik her $v \in G$
- 5 RELAX(u, v, w)

Bu algoritmanın çalıştırılması Şekil 24.5 de gösterilmektedir.

Bu algoritmanın çalışma süresini analiz etmek kolaydır. Kısım 22.4 de gösterildiği üzere topolojik sıralamaya göre 1. satır $\Theta(V + E)$ zaman alır. 2. satırdaki INITIALIZE-SINGLE-SOURCE $\Theta(V)$ zaman alır. 3-5 satırlarındaki **for** döngüsü her tepe için bir iterasyon yapar. 4-5 satırlarının **for** döngüsü tümüyle her ayrıtı tam bir kere gevşetir. (Burada toparlama analizini kullandık.) İçteki **for** döngüsünün her iterasyonu $\Theta(1)$ zaman aldığı için, toplam çalışma süresi çizgenin bitişiklik listesi gösterimde büyüklüğü doğrusal olan $\Theta(V + E)$ dir.

Aşağıdaki teorem yönlü DAG-SHORTEST-PATHS prosedürünün en kısa yolları doğru hesapladığını göstermektedir.



Şekil 24.5 Yönlü, çevrimsiz bir çizgede en kısa yollar için algoritmanın çalıştırılması. Tepeler soldan sağa topolojik sıralanmıştır. Kaynak tepe s dir. d değerleri tepelerin içinde ve gölgeli ayrıtlarda π değerleri ile gösterilmektedir. (a) 3-5 satırlarının **for** döngüsünün birinci iterasyonundan önceki durumu (b)-(g) 3-5 satırlarının **for** döngüsünün her bir iterasyonunda sonraki durumu. Her iterasyonda yeni karartılmış tepe o iterasyonda u olarak kullanılmıştır. (g) kısmında görünen değerler, son değerlerdir.

Teorem 24.5

Eğer ağırlıklı, yönlü $G = (V, E)$ çizgesinin kaynak tepesi s ve çevrimsizse DAG-SHORTEST-PATHS algoritmasının sonucunda tüm $v \in V$ tepeleri için $v.d = \delta(s, v)$ ve G_π öncül altçizgesi en kısa-yol ağacıdır.

Kanıt Öncelikle tüm $v \in V$ tepeleri için $v.d = \delta(s, v)$ olduğu gösterilecektir. Eğer v ye s den ulaşamıyorsa, yol yok özelliğinden $v.d = \delta(s, v) = \infty$ dir. Şimdi v ye s den ulaşılabilir olduğu kabul edilsin, $v_0 = s$ ve $v_k = v$ olduğu yerde en kısa yol $p = \langle v_0, v_1, \dots, v_k \rangle$ dır. Tepeler topolojik sıralı düzende incelendiğinden p deki

ayrıntlar $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$ sırasında gevşetilir. Yol-gevşetme özelliği $i = 0, 1, \dots, k$ için $v_i.d = \delta(s, v_i)$ yi simgeler. Sonunda, öncül altçizge özelliğinden G_π en kısa yol ağacıdır. ■

Bu algoritmanın ilgi çekici bir uygulaması **PERT diyagramı**² (**PERT chart**) analizinde kritik yolları belirlemede ortaya çıkar. Ayrıntlar, gerçekleşecek işleri ve ayrıt ağırlıkları, gerçekleşecek işler için gerekli zamanı gösterir. Eğer (u, v) ayrıt v tepesine girip, (v, x) ayrıt v tepesinden ayrılıyorsa (u, v) işi (v, x) işinden önce gerçekleşmektedir. Yönlü, çevrimsiz çizgedeki yol belirli sırada gerçekleşecek olan işlerin sırasını gösterir. **Kritik yol (critical path)**, yönlü çevre içermeyen çizgede herhangi bir işin sırasını gerçekleştiren en uzun zamana ilişkin *en uzun* yoldur. Böylece kritik yolun ağırlığı tüm işleri gerçekleştirmek için toplam zamanda alt sınırı sağlar.

- Ayrıt ağırlıklarını negatif yaparak ve yönlü, DAG-SHORTEST-PATHS çalıştırarak, ya da
- DAG-SHORTEST-PATHS 2. satırdaki INITIALIZE-SINGLE-SOURCE daki “ ∞ ” u “ $-\infty$ ” ile ve RELAX prosedüründeki “ $>$ ” ü “ $<$ ” ile değiştirerek çalışır.

Alıştırmalar

24.2-1

Şekil 24.5 deki yönlü çizgede DAG-SHORTEST-PATHS algoritmasını r tepesini kaynak alarak çalıştırın.

24.2-2

DAG-SHORTEST-PATHS algoritmasının 3.satırını

3 **for** topolojik sıralı düzende alınan ilk $|V|-1$ tepe

şeklinde değiştirdiğini kabul edin. Yöntemin doğru kalacağını gösterin.

24.2-3

Yukarıda verilen PERT diyagramı formülasyonu biraz doğaya aykırıdır. Daha doğal bir yapıda, tepeler işleri ve ayrıntlar sıralı kısıtları gösterir; yani (u, v) ayrıt u işinin v işinden önce gerçekleşeceğini gösterir. Bundan sonra ağırlıklar ayrıntlara değil tepelere atanır. DAG-SHORTEST-PATHS prosedürünü tepeleri ağırlıklı, yönlü, çevrimsiz bir çizgede bir en uzun yolu doğrusal zamanda bulması için değiştirin.

²PERT” (program evaluation and review technique) “program değerlendirme ve inceleme tekniği” nin kısaltmasıdır.

24.2-4

Yönlü, çevrimsiz bir çizgede yolların toplam sayısını sayan verimli bir algoritma verin. Algoritmanızı analiz edin.

24.3 Dijkstra algoritması

Dijkstra algoritması ağırlıklı, yönlü, bir $G = (V, E)$ çizgesinde tek-kaynaklı en kısa yollar problemini, tüm ayrıt ağırlıklarının negatif olmadığı durumu için çözer. Bu yüzden, bu bölümde her $(u, v) \in E$ ayrıtı için $w(u, v) \geq 0$ kabul edilir. İyi bir gerçekleştirim ile Dijkstra Algoritmasının çalışma zamanının Bellman-Ford algoritmasından daha düşük olduğu görülecektir.

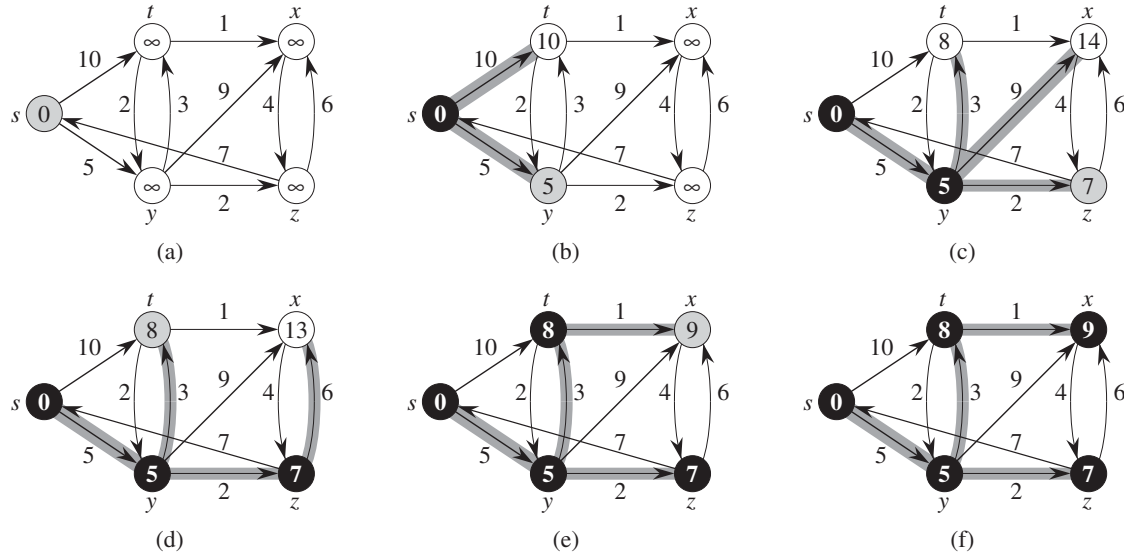
Dijkstra algoritması s kaynağından bulunan son en kısa yol ağırlıklarının S tepeler kümesini korur. Algoritma $u \in V - S$ tepesini minimum en kısa yol tahmini ile tekrarlı bir şekilde seçer, S ye u yu ekler, u dan ayrılan tüm ayrıtları gevşetir. Aşağıdaki gerçekleştirimde, d değerleri ile uyumlu tepelerin min-öncelikli bir Q kuyruğu kullanılır.

Dijkstra (G, w, s)

```

1  INITIALIZE-SINGLE-SOURCE( $G, s$ )
2   $S = \emptyset$ 
3   $Q = G.V$ 
4  while  $Q \neq \emptyset$ 
5       $u = \text{EXTRACT-MIN}(Q)$ 
6       $S = S \cup \{u\}$ 
7      for her  $v \in G.Adj[u]$ 
8          RELAX( $u, v, w$ )
```

Dijkstra algoritması Şekil 24.6 da gösterildiği üzere ayrıtları gevşetir. 1. satır d ve π değerlerini alışılmış bir şekilde başlatır ve 2. satır S kümesini boş küme olarak başlatır. Algoritma, 4-8 satırlarındaki **while** döngüsünün her iterasyon başlangıcında $Q = V - S$ değişmezini yürütür. 3. satır min-öncelikli Q kuyruğunu V deki tüm tepeleri içerecek şekilde başlatır. $S = \emptyset$ olduğu zaman, 3. satırdan sonra değişmez doğrudur. 4-8 satırlarındaki **while** döngüsünün her defasında, 5. satır bir u tepesini $Q = V - S$ den çıkarır ve 6. satır bunu S kümesine ekler, dolayısı ile değişmez korunur. (Bu döngünün ilk seferinde $u = s$ dir.) Bu sebepten, u tepesi $V - S$ deki herhangi bir tepeye en küçük en-kısa yol tahminine sahiptir. Daha sonra, 7-8 satırları u dan ayrılan her (u, v) ayrıtını gevşetir, böylece o ana kadar u dan v ye bulunan en kısa yolu geliştirebilirsek $v.d$ tahmini ve $v.\pi$ öncülü güncellenir. Algoritmanın 3. satırdan sonra Q ya hiçbir zaman tepe eklenmediğini ve her te-



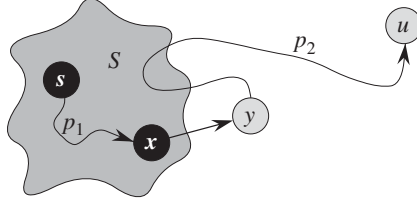
Şekil 24.6 Dijkstra algoritmasının çalışması. Kaynak s en soldaki tepedir. En kısa yol tahminleri tepelerin içinde görülmektedir ve gölgeli ayrıtlar öncül değerleri belirtmektedir. Siyah tepeler S kümesindedir ve beyaz tepeler min-öncelikli $Q = V - S$ kuyruğundadır. (a)–(f) satırlarındaki **while** döngüsünün ilk iterasyonundan hemen önceki durum. Gölgeli tepe minimum d değerine sahiptir ve 5. satırda u tepesi olarak seçilir. (b)–(f) **while** döngüsünün birbirini takip eden her iterasyonundan sonraki durum. Her parçadaki gölgeli tepe sonraki iterasyonun 5. satırında u tepesi olarak seçilir. (f) parçasında gösterilen d değerleri ve öncüller son değerlerdir.

penin sadece bir kez Q dan çıkartılıp S ye eklendiğini gözlemleyin. Böylece, 4-8 satırlarındaki **while** döngüsü $|V|$ defa tekrarlanır.

Dijkstra algoritması her zaman $V-S$ deki “en hafif” veya “en yakın” tepeyi S kümesine eklemek için seçtiğinden açgözlü bir strateji kullandığını söyleriz. 16. Bölüm açgözlü stratejileri detaylı olarak anlatmaktadır fakat Dijkstra algoritmasını anlamak için okumaya ihtiyacınız yok. Açgözlü stratejiler her zaman en iyi sonuçları vermez fakat aşağıdaki teorem ve onun sonucu aslında Dijkstra algoritmasının en kısa yolları hesapladığını gösterir. Fikir her defasında bir u tepesini S kümesine eklediğini göstermektir, $u.d = \delta(s, u)$ dir.

Teorem 24.6 (Dijkstra Algoritmasının Doğruluğu)

Dijkstra algoritması negatif olmayan w ağırlık fonksiyonu ve s kaynağına sahip ağırlıklı, yönlü $G = (V, E)$ çizgesinde çalışır, tüm $u \in V$ tepeleri için $u.d = \delta(s, u)$ ile sonlanır.



Şekil 24.7 Teorem 24.6'nın kanıtı. S kümesine v tepesi eklenmeden önce S boştur. y , S de olmayan yol üzerindeki ilk tepe ve $x \in S$ y den hemen önceki olmak üzere, s kaynağından u tepesine olan en kısa p yolunu $s \xrightarrow{p_1} x \rightarrow y \xrightarrow{p_2} u$ şeklinde ayrıştırırız. x ve y tepeleri ayrıktır fakat $s = x$ veya $y = u$ olabilir. p_2 yolu tekrar S kümesine girebilir yada girmeyebilir.

Kanıt Aşağıdaki döngü değişmezini kullanırız.

4-8 satırlarının **while** döngüsünün her iterasyonunun başlangıcında her $v \in S$ tepesi için $v.d = \delta(s, v)$ dir.

Her bir $u \in V$ tepesi için, S kümesine u eklendiğinde, $u.d = \delta(s, u)$ nin varlığını göstermek yeterlidir. $u.d = \delta(s, u)$ olduğunu gösterir göstermez eşitliğin bundan sonra her zaman geçerli olduğunu göstermek için üst sınır özelliğine dayanırız.

Başlangıç: Başlangıçta, $S = \emptyset$ ve sabit açıkça doğrudur.

Yineleme: Her iterasyonda, S kümesine eklenen tepe için $u.d = \delta(s, u)$ olduğunu göstermek isteriz. Çelişkinin amacı için, u ilk tepe olmak üzere, S kümesine eklendiğinde $u.d \neq \delta(s, u)$ olsun. u nun S ye eklendiği **while** döngüsünün iterasyonunun başlangıcındaki duruma odaklanmalı ve s den u ya en kısa yolu inceleme ile $u.d = \delta(s, u)$ olduğu çelişkidenden elde edilmelidir. S kümesine eklenen s ilk tepe olduğundan $u \neq s$ vardır ve o zaman $s.d = \delta(s, s) = 0$ dır. $u \neq s$ olduğundan, u , S ye eklenmeden önce $S \neq \emptyset$ vardır. S den u ya yol olmalıdır yoksa yol-olmama özelliğinden $u.d = \delta(s, u) = \infty$ ki bu $u.d \neq \delta(s, u)$ ile çelişir. Çünkü en az bir yol vardır bu da s den u ya p yoludur. S ye u yu eklemekten önce, s diye adlandırılan S deki bir tepelyi, u diye adlandırılan $V - S$ deki bir tepelyi p yolu bağlar. p boyunca ilk tepe $y \in V - S$ kabul edilsin ve $x \in S$ y nin p boyunca önde geleni olsun. Böylece Şekil 24.7 de örneklendiği üzere, $s \xrightarrow{p_1} x \rightarrow y \xrightarrow{p_2} u$ ya ayrılabilir. (p_1 veya p_2 yollarının ayrışmaları olmayabilir.) u , S ye eklendiğinde $y.d = \delta(s, y)$ olduğu iddia edilmektedir. Bu iddiayı kanıtlamak için $x \in S$ incelenir. Dolayısıyla $u.d \neq \delta(s, u)$ için S ye eklenen u ilk tepe seçildiğinden, x , S ye eklendiğinde $x.d = \delta(s, x)$ vardır.

O zaman (x, y) ayrıtı gevşer ve iddia yakınsaklık özelliğinden anlaşılır. $u.d = \delta(s, u)$ yu kanıtlamak için şimdi çelişki elde edilebilir. Çünkü s den u ya en kısa yol üzerinde y u dan önce bulunur ve tüm ayrıt ağırlıkları negatif olmayandır. (açıkça bunlar p_2 üzerindedir.) $\delta(s, y) \leq \delta(s, u)$ vardır ve böylelikle

$$\begin{aligned} y.d &= \delta(s, y) \\ &\leq \delta(s, u) \\ &\leq u.d \quad (\text{üst-sınır özelliğinden}). \end{aligned} \tag{24.2}$$

Fakat 5. satırda u seçildiğinde u ve y tepeleri $V - S$ de var olduğundan $u.d \leq y.d$ vardır. Böylece bu ve 24.2 deki iki eşitsizlik aslında eşitlik içinde

$$y.d = \delta(s, y) = \delta(s, u) = u.d$$

yi verir.

Sonuç olarak, $u.d = \delta(s, u)$ u nun seçimiyle çelişir. S ye u eklendiğinde $u.d = \delta(s, u)$ olduğu sonuçlanır ve bu eşitlik bundan sonra tüm zamanlarda korunur.

Sonlandırma: Bitişte $Q = \emptyset$ önceki $Q = V - S$ sabiti ile birlikte $S = V$ yi içerir. Böylece $u \in V$ tepeleri için $u.d = \delta(s, u)$ dur. ■

Sonuç 24.7

Dijkstra algoritması ağırlıklı, yönlü, negatif olmayan w ağırlık fonksiyonu ve s kaynaklı $G = (V, E)$ çizgesinde çalışır, o halde bitişte, öncül altçizge G_π , kökü s olan, en kısa yolların ağacıdır.

Kanıt Teorem 24.6 dan ve öncül altçizge özelliğinden açıktır. ■

Analiz

Dijkstra algoritması ne kadar hızlı? Üç tane öncelikli-kuyruk işlemlerinin çağrılmasıyla min-öncelikli Q kuyruğunu korur: INSERT (3. satırda ifade edilen), EXTRACT-MIN (5. satırda) ve DECREASE-KEY (8. satırda adlandırılan, RELAX de ifade edilen). Algoritma her tepe başına INSERT ve EXTRACT-MIN i çağırır. Çünkü S kümesine tam olarak bir kez eklenen $u \in V$ tepesi, $Adj[u]$ bitişiklik listesindeki her ayrıt 7-8 satırlarının **for** döngüsünde algoritma programı boyunca incelenmiştir. Tüm bitişiklik listelerindeki toplam ayrıt sayısı $|E|$ olduğundan, bu **for** döngüsü toplam $|E|$ kez tekrarlar ve böylece algoritma en fazla $|E|$ kez DECREASE-KEY diye adlandırılır.

Dijkstra algoritmasının çalışma zamanı min-öncelikli kuyruğu nasıl gerçekleştirdiğimize bağlıdır. 1 den $|V|$ ye numaralandırılmış tepelerden yararlanarak

min-öncelikli kuyruğunun nasıl korunduğu durumu ilk olarak düşünülür. Basitçe $v.d$ dizinin v inci girdisi olarak saklanır. Her bir INSERT ve DECREASE-KEY işlemleri $O(1)$ zaman alır ve en az EXTRACT-MIN işlemi $O(V)$ zaman alır (tüm sıra boyunca incelenmelidir), $O(V^2 + E)$ nin toplam zamanı $O(V^2)$ dir.

Eğer çizge yeterince seyrek ise – özellikle, $E = O(V^2/\lg V)$ – min-öncelikli kuyruğu bir ikili min-bellek yığını ile gerçekleştirerek algoritmayı geliştirebiliriz. (Kısım 6.5 de tartışıldığı gibi uygulama tepeleri ve ilgili bellek yığını elemanları işleçlerini korumak için birbirlerinden emin olmalıdır). Dolayısıyla her EXTRACT-MIN işlemi $O(\lg V)$ zaman alır. Eskisi gibi bu işlemler $|V|$ dir. İkili min-bellek yığını oluşturmak için zaman $O(V)$ dir. Her DECREASE-KEY işlemi $O(\lg V)$ zamanı alır ve bu işlemlerden en fazla $|E|$ tane vardır. Bu nedenle, eğer tüm tepeler kaynaktan ulaşılabiliriyorsa toplam zaman $O(E \lg V)$ olan $O((V + E)\lg V)$ dir. Eğer $E = O(V^2/\lg V)$ ise bu çalışma zamanı $O(V^2)$ zaman uygulamasından iyisini yapar.

Aslında Fibonacci bellek yığını ile min-öncelikli kuyruk uygulamasıyla $O(V \lg V + E)$ çalışma zamanı elde edilir (Bölüm 19 a bakın). Her $|V|$ tane EXTRACT-MIN işleminin amortize maliyeti $O(\lg V)$ dir ve her DECREASE-KEY denilen, en fazla $|E|$ tane olan, sadece $O(1)$ amortize zaman alır. Tarihsel olarak, Fibonacci bellek yığınlarının gelişimi Dijkstra algoritmasının DECREASE-KEY işlemini, EXTRACT-MIN işleminden çok daha fazla çağırmasını gözlemleyerek harekete geçmiştir. Böylece her bir DECREASE-KEY işleminin amortize süresini, EXTRACT-MIN işleminin amortize süresini arttırmadan $O(\lg V)$ ye düşürmek için kullanılan tüm yöntemler ikili bellek yığınlı yöntemlere göre asimptotik olarak çok daha hızlı bir uygulama sağlayacaktır.

Dijkstra algoritması en az dallanmış ağaçları bulan “enine-arama” (BFS – breadth-first search) (Kısım 22.2 ye bakın) ve Prim algoritmalarına (Kısım 23.2 ye bakın) benzemektedir. “Enine-arama” benzer şöyle ki S kümesi “enine-aramasındaki” siyah tepelerin kümesine karşılık gelir; tıpkı S deki tepelerin son en kısa yol ağırlıkları olduğu gibi “enine-aramasındaki” siyah tepelerinde enine-arama uzaklıkları vardır. Dijkstra algoritması Prim algoritmasına benzer şöyle ki her iki algoritmada verilen küme dışında en hafif tepeyi bulmak için en az öncelikli sırayı kullanır, (Dijkstra algoritmasındaki S kümesi ve Prim algoritmasında oluşan ağaç) bu tepeyi kümeye ekler ve kalan tepelerin ağırlıklarını küme dışına göre ayarlar.

Alıştırmalar

24.3-1

Şekil 24.2 deki yönlü çizgesinde önce s tepesini kaynak olarak ve sonra z tepesini kaynak olarak Dijkstra algoritmasını çalıştırın. Şekil 24.6 tarzında d ve π değerlerini ve **while** döngüsünün her iterasyonundan sonra S kümesindeki tepelerini gösterin.

24.3-2

Ayrıtları negatif ağırlıklı, yönlü çizge için Dijkstra algoritmasının yanlış cevaplar ürettiğini gösteren bir basit örnek verin. Negatif-ağırlıklı ayrıtlara izin verildiğinden neden Teorem 24.6'nın kanıtlanmamıştır?

24.3-3

Dijkstra algoritmasının 4. satırını

4 **while** $|Q| > 1$

şeklinde değiştiğini farz edin. Bu değişiklik **while** döngüsünün $|V|$ zaman yerine $|V| - 1$ zamanda yapmasına sebep olur. Bu önerilen algoritma doğru mudur?

24.3-4

Profesör Gaedel, Dijkstra algoritmasının uygulamalarını gerektiren bir program yazmıştır. Bu program $v \in V$ tepesi için $v.d$ ve $v.\pi$ üretir. Profesörün programının çıktılarını kontrol etmek için $O(V + E)$ zaman verilmiştir. d ve π nin bazı en kısa yollar ağacına eşleme yapıp yapmayacağı belirlenmelidir. Tüm ayrıt ağırlıklarının negatif olmadığını farz edin.

24.3-5

Profesör Newman, Dijkstra algoritmasının daha basit kanıtı için çalışabildiğini düşünüyor. Dijkstra algoritmasının çizgedeki her kısa yolun ayrıtları yolda görüldüğü sırada gevşer ve bu sebeple kaynaktan ulaşılan her tepeye yol gevşetme özelliği uygulanır. Profesörün hata yaptığını Dijkstra algoritmasının sırasız en kısa yolların ayrıtlarını gevşettiğini yönlü çizge oluşturarak gösterin.

24.3-6

u dan v ye iletişim yolunun güvenilirliğini gösteren $r(u, v)$, $0 \leq r(u, v) \leq 1$ aralığında reel sayısı verilen yönlü $G = (V, E)$ çizgesinin her $(u, v) \in E$ ayrıtı üzerindedir. $r(u, v)$, u dan v ye olan yolun başarısız olmayacağı olasılığı olarak yorumlanır ve bu olasılıkların bağımsız olduğu varsayılır. Verilen iki tepe arasında en güvenilir yolu bulmak için etkili bir algoritma verin.

24.3-7

$G = (V, E)$ ağırlıklı, yönlü çizgesi, $\omega : E \rightarrow \{1, 2, \dots, W\}$, W pozitif tam sayısı için pozitif ağırlık fonksiyonu verilsin ve s tepesi kaynağından herhangi iki tepenin aynı kısa yol ağırlıklarına sahip olmadığını farz edin. Şimdi her $(u, v) \in E$ ayrıtının, $\omega(u, v)$ birim-ağırlıklı ayrıtlarla seri olarak yer değiştirerek $G' = (V \cup V', E')$ ağırlıksız yönlü çizgesini farz edin. G' nün kaç tane tepesi vardır? G' de bir “eni-

ne-aramanın” çalıştırıldığını farz edin. G' nün “enine-aramasının” V deki tepeleri siyaha boyama sırasının, Dijkstra algoritmasının G de çalıştığında V den tepelerin öncelikli kuyruğa göre ayıklama sırasının aynı olduğunu gösterin.

24.3-8

$G = (V, E)$ ağırlıklı, yönlü çizgesi, $\omega: E \rightarrow \{0, 1, 2, \dots, W\}$, W pozitif tam sayısı için negatif olmayan şekilde ağırlık fonksiyonu verilsin. $O(WV + E)$ zamanda verilen s tepesi kaynağından en kısa yolları hesaplayan Dijkstra algoritmasını değiştirin.

24.3-9

Alıştırma 24.3 den 24.8 e kadar olan algoritmayı $O((V + E) \lg W)$ zamanda çalıştırmak için değiştirin. (*İpucu:* Zamanda herhangi bir noktada $V - S$ de kaç tane farklı en kısa-yol tahmini vardır?)

24.3-10

Verilen ağırlıklı, yönlü $G = (V, E)$ çizgesinin s tepesi kaynağından ayrılan ayrıtların negatif ağırlıklı olabileceğini diğer ayrıtların negatif olamadığını ve negatif ağırlıklı çevrimlerin var olmadığını farz edin. Dijkstra algoritmasının bu çizgede s den en kısa yolları doğru bir şekilde bulunduğunu tartışın.

24.4 Fark kısıtları ve en kısa yollar

29. Bölüm bir doğrusal fonksiyonu bir dizi doğrusal eşitsizliklere bağlı olarak optimize etmek istediğimiz genel doğrusal programlama problemini inceler. Bu bölümde doğrusal programlamanın özel durumu araştırılmaktadır ki tek kaynaktan olan en kısa yolları bulma indirgensin. Tek-kaynaklı en kısa yollar problemini Bellman-Ford algoritmasının çalışmasıyla sonuçlanan tek-kaynaklı en kısa yollar problemi çözülür dolayısıyla doğrusal programlama problemi de çözülür.

Doğrusal Programlama

Genelde *doğrusal-programlama problemi (linear-programming problem)* verilen A matrisi $m \times n$ dir, m -vektörü b ve n -vektörü c dir. $Ax \leq b$ ile verilen m kısıtların bağlı $\sum_{i=1}^n c_i x_i$ *amaç fonksiyonunu (objective function)* maksimum yapmak için n elemanlı x vektörü bulunmak istenmektedir.

Bölüm 29 da geçen simpleks algoritması girdi boyutlarına göre her zaman polinom zamanda çalışmamasına rağmen, polinom zamanda çalışan başka doğrusal programlama algoritmaları vardır. Doğrusal programlama problemlerinin kurulmasını anlamak için burada iki düşünce önerilmektedir. Öncelikle eğer verilen

problemi polinom boyutlu doğrusal programlama problemine çevirmeyi biliyorsa problemi polinom zamanlı algoritma ile hemen çözülür. İkinci daha hızlı algoritma ise doğrusal programlamanın birçok özel durumları için vardır. Örneğin tek-ikili en kısa yol problemi (Örnek 24.4-4) ve maksimum akış problemi (Örnek 26.1-5) doğrusal programlamanın özel durumları için vardır.

Bazen amaç fonksiyonlarıyla ilgilenmeyip **olurlu çözüm** için $Ax \leq b$ yi sağlayan herhangi bir x vektörünü yada olurlu çözümün olmadığını belirlemek isteriz. Böyle bir **elverişlilik problemi** odaklanmalıyız.

Fark kısıtları sistemleri

Fark kısıtlarının sisteminde (system of difference constraints) doğrusal programlama matrisi A nın her satırı bir tane 1, bir tane -1 ve diğer tüm girdileri 0 içerir. Böylece $Ax \leq b$ biçiminde verilen kısıtlar, her kısıtlama şeklinde basit bir doğrusal eşitsizlik olduğu n bilinmeyenler içeren m fark kısıtları kümesidir.

$1 \leq i, j \leq n, i \neq j$ ve $1 \leq k \leq m$ için $1 \leq k \leq m$ dir.

Örneğin, $x = (x_i)$ yi sağlayan 5 vektörü bulmayı düşünün.

$$\begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & -1 \\ 0 & 1 & 0 & 0 & -1 \\ -1 & 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & -1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} \leq \begin{pmatrix} 0 \\ -1 \\ 1 \\ 5 \\ 4 \\ -1 \\ -3 \\ -3 \end{pmatrix} .$$

Bu problem 8 fark kısıtını sağlayan x_1, x_2, x_3, x_4, x_5 bilinmeyen değerleri bulmaya karşılık gelir.

$$x_1 - x_2 \leq 0, \quad (24.3)$$

$$x_1 - x_5 \leq -1, \quad (24.4)$$

$$x_2 - x_5 \leq 1, \quad (24.5)$$

$$x_3 - x_1 \leq 5, \quad (24.6)$$

$$x_4 - x_1 \leq 4, \quad (24.7)$$

$$x_4 - x_3 \leq -1, \quad (24.8)$$

$$x_5 - x_3 \leq -3, \quad (24.9)$$

$$x_5 - x_4 \leq -3. \quad (24.10)$$

Bu probleme bir çözüm, her bir eşitsizlik kontrol edilerek $x = (-5, -3, 0, -1, -4)$ olduğu doğrulanır. Aslında bu problemin birden fazla çözümü vardır. Bir başka çözümü $x' = (0, 2, 5, 4, 1)$ dir. İki çözüm bağlantılıdır şöyle ki, x' nün her bir birleşeni x in her bir birleşenine karşılık gelenden 5 fazladır. Bu gerçek tesadüfi değildir.

Lemma 24.8

$x = (x_1, x_2, \dots, x_n)$, $Ax \leq b$ sisteminin fark kısıtlarının bir çözümü olsun ve d herhangi bir sabit olsun. Böylece $x + d = (x_1 + d, x_2 + d, \dots, x_n + d)$, $Ax \leq b$ nin de çözümüdür.

Kanıt Her x_i ve x_j için $(x_j + d) - (x_i + d) = x_j - x_i$ vardır. Böylece x , $Ax \leq b$ yi sağlar, $x + d$ de sağlar. ■

Fark kısıtlarının sistemleri birçok farklı uygulamalarda görülür. Örneğin, olayların gerçekleşmesi için x_i bilinmeyenleri zamanı gösterebilir. Her kısıt iki olay arasında geçen en az belirli bir zamanı ya da en çok belirli bir zamanı belirtir. Belki olaylar ürünün montajı esnasında gerçekleşecek işlerdir. x_1 zamanda uygulaması 2 saat alan yapıştırıcı ve x_2 zamanda bir kısmının kuruması beklendiğinde elde edilen kısıt $x_2 \geq x_1 + 2$ ya da buna eş $x_1 - x_2 \leq -2$ dir. Alternatif olarak, yapıştırıcının yarısından fazlasını uygulamadan parça takılmak istenebilir. Bu durumda $x_2 \geq x_1$ ve $x_2 \leq x_1 + 1$ çift kısıtları ya da buna eş $x_1 - x_2 \leq 0$ ve $x_2 - x_1 \leq 1$ kısıtları elde edilir.

Kısıt Çizgeler

Fark kısıtlarının sistemleri teorik çizge bakımından yorumlanabilir. Fark kısıtları $Ax \leq b$ sisteminde, n tepeli, m ayrıtlı çizge için bitişiklik matrisinin transpozesi olan $m \times n$ doğrusal programlama matrisi A dır (Örnek 22.1-7 ye bakınız.). Çizgedeki her v_i tepesi $i = 1, 2, \dots, n$ için n tane bilinmeyen, değişken x_i nin bir tanesine karşılık gelir. Çizgedeki her yönlü ayrıt, 2 tane bilinmeyen içeren bir tane m eşitsizliğine karşılık gelir.

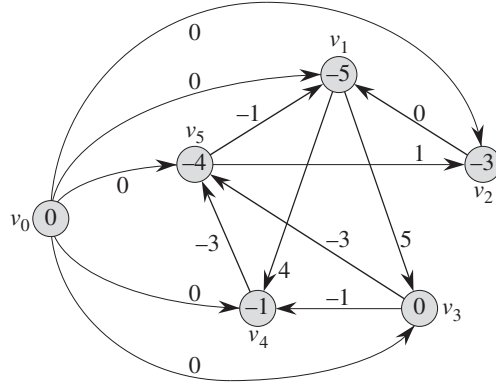
Daha şeklen, fark kısıtları $Ax \leq b$ sistemi, ağırlıklı, yönlü $G = (V, E)$ çizgesinin **kısıt çizgesine**

$$V = \{v_0, v_1, \dots, v_n\}$$

ve

$$E = \{(v_i, v_j) : x_j - x_i \leq b_k, \text{ bir kısıttır}\} \\ \cup \{(v_0, v_1), (v_0, v_2), (v_0, v_3), \dots, (v_0, v_n)\}$$

durumunda karşılık gelir.



Şekil 24.8 Fark kısıtları (24.3) – (24.10) sistemlerine karşılık gelen kısıt çizgesi $\delta(v_0, v_i)$ değeri, her v_i tepesinden bulunur. Sistemin uygulanabilir bir çözümü $x = \{-5, -3, 0, -1, -4\}$ tür.

Kısaca görüleceği gibi diğer tüm tepelere ulaşabilen bazı tepeleri garantiye almak için kısıt çizgesi v_0 ilave tepesini içerir. Böylece bilinmeyen her x_i için tepeler kümesi V bir v_i tepesinden oluşur ilave olarak v_0 tepesi eklenir. Her fark kısıtı için ayrıt kümesi E bir ayrıt içerir ve her bilinmeyen x_i için (v_0, v_i) ayrıtı eklenir. Eğer $x_j - x_i \leq b_k$ farklı bir kısıt ise (v_i, v_j) ayrıtının ağırlığı $w(v_i, v_j) = b_k$ dir. v_0 dan ayrılan her ayrıtın ağırlığı 0 dir. Şekil 24.8 (24.3)-(24.10) sistemlerinin fark kısıtları için kısıt çizgesini gösterir.

Aşağıdaki teoremde kısıt çizgeye karşılık gelen en kısa yolların ağırlıklarını kullanarak fark kısıtlarının bir sistemine çözüm bulunabileceği gösterilmiştir.

Teorem 24.9

$G = (V, E)$, fark kısıtları olan bir $Ax \leq b$ sistemine karşılık gelen kısıt çizgesi olsun. Eğer G negatif olmayan-ağırlıklı çevrimler içeriyorsa, o zaman

$$x = (\delta(v_0, v_1), \delta(v_0, v_2), \delta(v_0, v_3), \dots, \delta(v_0, v_n)) \quad (24.11)$$

ifadesi bu sistem için uygun bir çözümdür. Eğer G negatif-ağırlıklı çevrim içeriyorsa, o zaman bu sistem için uygun bir çözüm yoktur.

Kanıt Biz ilk olarak eğer kısıt çizgesi negatif olmayan-ağırlıklı çevrim içeriyorsa, (24.11) denkleminin uygun bir çözüm verdiğini göstereceğiz. Herhangi bir $(v_i, v_j) \in E$ ayrıtını düşünelim. Üçgen eşitsizliğinden, $\delta(v_0, v_j) \leq \delta(v_0, v_i) + w(v_i, v_j)$ ya da eşdeğer olarak, $\delta(v_0, v_j) - \delta(v_0, v_i) \leq w(v_i, v_j)$ dir. Böylece $x_j = \delta(v_0, v_j)$ ve

$x_j = \delta(v_0, v_j)$ alarak, (v_i, v_j) ayrıtına karşılık gelen $x_j - x_i \leq w(v_i, v_j)$ fark kısıtını elde ederiz.

Şimdi, eğer kısıt çizgesi negatif-ağırlıklı çevrim içeriyorsa, o zaman fark kısıtlarının sisteminin elverişli bir çözüme sahip olmadığını gösterelim. Genelliği kaybetmeden, $v_1 = v_k$ iken, $c = \langle v_1, v_2, \dots, v_k \rangle$ negatif-ağırlıklı çevrim olsun. (v_0 tepesi bir c çevrimi üzerinde olamaz çünkü v_0 'a giren ayrıt yoktur.) c çevrimi aşağıdaki fark kısıtlarına karşılık gelir:

$$\begin{aligned} x_2 - x_1 &\leq w(v_1, v_2) , \\ x_3 - x_2 &\leq w(v_2, v_3) , \\ &\vdots \\ x_{k-1} - x_{k-2} &\leq w(v_{k-2}, v_{k-1}) , \\ x_k - x_{k-1} &\leq w(v_{k-1}, v_k) . \end{aligned}$$

Biz x in bu k tane eşitsizliğin her birini sağlayan bir çözümü olduğunu varsayalım ve buna göre çelişki türetelim. Bu çözüm ayrıca k tane eşitsizliği taraf tarafa topladığımızda elde ettiğimiz eşitsizliği de sağlamalıdır. Eğer sol tarafları toplarsak, her bilinmeyen x ; bir kere eklenir ve bir kere çıkarılır. ($v_1 = v_k$ nın $x_1 = x_k$ ı kastettiğini hatırlayın) böylece sol tarafın toplamı 0 dır. Sağ tarafın toplamı $w(c)$ olsun. Dolayısıyla $0 \leq w(c)$ yi elde ederiz. Fakat c negatif-ağırlıklı çevre olduğundan, $w(c) < 0$ dır ve dolayısıyla $0 \leq w(c) < 0$ çelişkisini elde ederiz. ■

Fark kısıtlarının sistemlerini çözme

Teorem 24.9 bize, fark kısıtlarının bir sistemini çözerken Bellman-Ford algoritmasını kullanabileceğimizi söyler. Çünkü kısıt çizgesi v_0 kaynak tepesinden diğer tüm tepelere olan ayrıtları içerdiği için, kısıt çizgesindeki herhangi bir negatif ağırlıklı çevrimi v_0 dan ulaşılabilir. Eğer Bellman-Ford algoritmasının geri bildirimi TRUE ise, kısa yol ağırlıkları sistemi için uygulanabilir bir çözüm verir. Şekil 24.8 de, örneğin; kısa yol ağırlıkları $x = (-5, -3, 0, -1, k-4)$ uygulanabilir, çözüm sağlar ve Lemma 24.8 de $x = (d-5, d-3, d, d-1, d-4)$ herhangi bir sabit d için uygulanabilir çözümdür. Eğer Bellman-Ford algoritmasının geri bildirimi FALSE ise, burada fark kısıtlarının sisteminin elverişli çözümü yoktur.

m kısıtlı n bilinmeyenli fark kısıtlarının sistemi $n + 1$ tepeli ve $n + m$ ayrıtlı bir çizge üretir. Böylece, Bellman-Ford algoritmasını kullanarak sistemi $O((n + 1)(n + m)) = O(n^2 + mn)$ zamanda çözebiliriz. Alıştırma 24.4-5 te, eğer m, n den çok daha az olsa dahi algoritmayı $O(mn)$ zamanda çalıştırmak için bir değişiklik yapıp yapamayacağınız soruluyor.

Alıştırmalar**24.4-1**

Aşağıdaki fark kısıtlarının sistemleri için elverişli bir çözüm bulun ya da elverişli bir çözüm olmayacağını belirleyin.

$$x_1 - x_2 \leq 1 ,$$

$$x_1 - x_4 \leq -4 ,$$

$$x_2 - x_3 \leq 2 ,$$

$$x_2 - x_5 \leq 7 ,$$

$$x_2 - x_6 \leq 5 ,$$

$$x_3 - x_6 \leq 10 ,$$

$$x_4 - x_2 \leq 2 ,$$

$$x_5 - x_1 \leq -1 ,$$

$$x_5 - x_4 \leq 3 ,$$

$$x_6 - x_3 \leq -8 .$$

24.4-2

Aşağıdaki fark kısıtlarının sistemleri için elverişli bir çözüm bulun ya da uygun bir çözüm olmayacağını belirleyin.

$$x_1 - x_2 \leq 4 ,$$

$$x_1 - x_5 \leq 5 ,$$

$$x_2 - x_4 \leq -6 ,$$

$$x_3 - x_2 \leq 1 ,$$

$$x_4 - x_1 \leq 3 ,$$

$$x_4 - x_3 \leq 5 ,$$

$$x_4 - x_5 \leq 10 ,$$

$$x_5 - x_3 \leq -4 ,$$

$$x_5 - x_4 \leq -8 .$$

24.4-3

Bir kısıt çizgesindeki yeni bir v_0 tepesinden elde edilen herhangi bir en kısa yol ağırlığı pozitif olabilir mi? Açıklayınız.

24.4-4

Tek-ikili en kısa yol problemini doğrusal bir program olarak ifade ediniz.

24.4-5

Bellman-Ford algoritmasını m eşitsizlikli n bilinmeyenli fark kısıtlarının sistemini çözmek için kullandığımızda çalışma zamanını $O(nm)$ yapmak için yapılması gereken ufak değişikliği gösteriniz.

24.4-6

$x_i = x_j + b_k$ **eşitlik kısıtları (equality constraints)** yapısı aynı kalmak üzere, bir fark kısıtlarının sistemine ek yapacağımızı kabul edelim. Bu tür bir kısıt sistemini çözmek için Bellman-Ford algoritmasını nasıl uyarlamak gerektiğini gösteriniz.

24.4-7

Ek bir v_0 tepesi olmadan bir kısıt çizgesinde, fark kısıtlarının sisteminin Bellman-Ford-benzeri algoritma tarafından nasıl çözüleceğini gösteriniz.

24.4-8 *

$A_x \leq b$ n bilinmeyenli m fark kısıtlı bir sistem olsun. Bellman-Ford algoritmasını sisteme karşılık gelen kısıt çizgesinde, her x_i için $x_i \leq 0$ ve $A_x \leq b$ kısıtları altında $\sum_{i=1}^n x_i$ yi maksimum yapmak için çalıştırınız.

24.4-9 *

$Ax \leq b$ fark kısıt sistemi için belirlenen kısıt çizgesi üzerinde Bellman-Ford algoritmasını $Ax \leq b$ kısıtı altında $(\max\{x_i\} - \min\{x_i\})$ miktarını minimum yapmak için çalıştırınız. Eğer algoritma yapılacak işleri çizelgelemek için kullanılırsa bu durumun işe yarayıp yaramayacağını açıklayınız.

24.4-10

$Ax \leq b$ doğrusal programının A matrisinin her bir satırının $x_i \leq b_k$ formundaki tek değişkenli veya $-x_i \leq b_k$ formundaki tek değişkenli bir fark kısıtına karşılık geldiğini varsayalım. Bellman-Ford algoritmasını böyle bir sisteme nasıl uyarlayacağımızı gösteriniz.

24.4-11

Tüm b elemanları gerçek-değerli ve tüm x_i bilinmeyenleri tam sayı olmak zorundayken $Ax \leq b$ fark kısıtlarının sistemini çözmek için verimli bir algoritma öneriniz.

24.4-12 *

Tüm b elemanları gerçek-değerli ve x_i bilinmeyenlerinin bazı özel alt kümeleri (ama hepsi değil) tam sayı olmak zorundayken $Ax \leq b$ fark kısıtlarının sistemini çözmek için verimli bir algoritma öneriniz.

24.5 En kısa yolların özelliklerinin kanıtları

Bu bölüm boyunca, bizim doğruluk kanıtlarımız üçgen eşitsizliği, üst-sınır özelliği, yol-olmama özelliği, yakınsaklık özelliği, yol-gevşetme özelliği ve öncül-alt-çizge özelliğine dayalıdır. Bu bölümün başında, bu özellikleri kanıt olmadan belirttik. Bu bölümde onları kanıtlayacağız.

Üçgen eşitsizliği

Enine-aramayı (Kısım 22.2) ilk çalışırken, Lemma 22.1 de ağırlıksız çizgelerde kısa uzaklıkların bir basit özelliğini kanıtladık. Üçgen eşitsizliği bu özelliği ağırlıklı çizgelerde genelleştirir.

Lemma 24.10 (Üçgen eşitsizliği)

$G = (V, E)$ s kaynak tepeli, $w : E \rightarrow \mathbb{R}$ ağırlık fonksiyonu ile yönlü, ağırlıklı bir çizge olsun. Daha sonra, tüm $(u, v) \in E$ ayrıtları için

$$\delta(s, v) \leq \delta(s, u) + w(u, v)$$

vardır.

Kanıt p nin s kaynağından v tepesine kadar en kısa yol olduğunu kabul edelim. p s den v ye olan diğer tüm yollardan daha fazla ağırlığa sahip değildir. Özellikle, p yolu, s kaynağından u tepesine olan en kısa yolu ve (u, v) ayrıtını alan belirli bir yoldan daha fazla ağırlığa sahip değildir.

Alıştırma 24.5-3, size s den v ye en kısa yol olmaması durumunu ele almayı sorar. ■

En-kısa-yol tahminlerinde gevşetmenin etkileri

Sonraki lemma grupları en kısa yol tahminlerinin, çizge INITIALIZE-SINGLE-SOURCE tarafından başlatılan yönlü, ağırlıklı çizgenin ayrıtlarının gevşetme adımlarının art arda uygulanması esnasında nasıl etkilendiğini tanımlar.

Lemma 24.11 (Üst-sınır özelliği)

$G = (V, E)$ ağırlıklı, yönlü ve ağırlık fonksiyonu $w : E \rightarrow \mathbb{R}$ olan çizge olsun. $s \in V$ kaynak tepe ve çizge INITIALIZE-SINGLE-SOURCE (G, s) ile başlatılsın. Her $v \in V$ için $v.d \geq \delta(s, v)$ ve bu sabit, G ayrıtları üzerinde herhangi art arda gevşetme adımları üzerinde sürdürülmesidir. Ayrıca, $\delta(s, v)$ nin alt sınırı $v.d$ elde edilir, bu hiçbir zaman değişmez.

Kanıt $v.d \geq \delta(s, v)$ sabitini bütün $v \in V$ tepeleri için gevşetme adımlarının sayısını tümevarımla kanıtlarız.

Temel olarak, $v.d \geq \delta(s, v)$ başlangıç sonrası kesinlikle doğrudur, çünkü $v.d = \infty$ her $v \in V - \{s\}$ için $v.d \geq \delta(s, v)$ ve $s.d = 0 \geq \delta(s, s)$ anlamına gelmektedir. (Eğer s negatif-ağırlıklı çevrim üzerindeyse ve aksi halde 0 ise $\delta(s, s) = -\infty$ olduğu dikkate alınmalıdır).

Tümevarım adımı için (u, v) ayrıtını gevşetmeyi düşünün. Tümevarımsal hipotez aracılığıyla, gevşemeden önce her $x \in V$ için $x.d \geq \delta(s, x)$ dir. Sadece d değeri $v.d$ olarak değişebilir. Eğer değişirse, sahip olduğumuz

$$\begin{aligned} v.d &= u.d + w(u, v) \\ &\geq \delta(s, u) + w(u, v) && \text{(Tümevarımsal Hipotez aracılığıyla)} \\ &\geq \delta(s, v) && \text{(Üçgen Eşitsizliği aracılığıyla)} \end{aligned}$$

ve bu nedenle sabit korunur.

$v.d$ değerinin hiç değişmediğini görmek için önce $v.d = \delta(s, v)$, elde olan alt sınıra dikkat, $v.d$ azalmaz çünkü henüz $v.d \geq \delta(s, v)$ olduğunu gösterdik ve bu değişmez çünkü gevşetme adımları d değerini artırmaz. ■

Sonuç 24.12 (Yol-olmama Özelliği)

Kabul edelim ki ağırlık fonksiyonu $w : E \rightarrow \mathbb{R}$ olan yönlü, ağırlıklı $G = (V, E)$ çizgesi, $s \in V$ kaynak tepe den verilen $v \in V$ tepeye yol bağlantısı olmasın. O zaman, çizge INITIALIZE-SINGLE-SOURCE(G, s) ile başlatıldıktan sonra, biz $v.d = \delta(s, v) = \infty$ sahibiz ve bu eşitsizlik G ayrıtları üzerinde gevşetme adımı dizilerini sabit olarak muhafaza eder.

Kanıt Üst-Sınır özelliği tarafından, biz her zaman $\infty = \delta(s, v) \leq v.d$ olduğunu bilmekteyiz ve bu nedenle $v.d = \infty = \delta(s, v)$. ■

Lemma 24.13

$G = (V, E)$ ağırlık fonksiyonu $w : E \rightarrow \mathbb{R}$ olan ağırlıklı bir çizge ve $(u, v) \in E$ olsun. O zaman, RELAX(u, v, w) nin uygulanmasıyla (u, v) ayrıtının gevşetilmesinden hemen sonra, $v.d \leq u.d + w(u, v)$ bilmekteyiz.

Kanıt Eğer, (u, v) ayrıtının gevşetilmesinden önce, $v.d > u.d + w(u, v)$, o zaman sonra $v.d = u.d + w(u, v)$ dir. Eğer, yerine, gevşetmeden hemen önce $v.d \leq u.d + w(u, v)$ dir, o zaman ne $u.d$, ne de $v.d$ değişir ve sonradan $v.d \leq u.d + w(u, v)$ dir. ■

Lemma 24.14 (Yakınsaklık Özelliği)

$G = (V, E)$ çizgesi ağırlık fonksiyonu $w : E \rightarrow \mathbb{R}$ olan ağırlıklı, yönlü bir çizge, $s \in V$ bir kaynak tepe ve bazı $u, v \in V$ tepeliri için $s \rightsquigarrow u \rightarrow v$ yolu G deki en

kısa yol olsun. Kabul edelim ki G INITIALIZE-SINGLE-SOURCE(G, s) ile başlatılsın ve sonra G nin tepelerinde gevşetme adımları RELAX(u, v, w) prosedürü çağrılarak uygulansın. Eğer çağırılmadan önce herhangi bir zamanda $u.d = \delta(s, u)$, o zaman çağırma sonrasında her zaman $v.d = \delta(s, v)$ olur.

Kanıt Üst-Sınır özelliği tarafından eğer (u, v) ayrıtının gevşetilmesinden önce bazı noktalarda $u.d = \delta(s, u)$ dır, o zaman bu eşitlik buradan sonra kurulur. Özellikle (u, v) ayrıtı gevşetildikten sonra,

$$\begin{aligned} v.d &\leq u.d + w(u, v) && \text{(Lemma 24.13 aracılığıyla)} \\ &= \delta(s, u) + w(u, v) \\ &= \delta(s, v) && \text{(Lemma 24.1 aracılığıyla) vardır.} \end{aligned}$$

Üst-sınır özelliği tarafından, $v.d \geq \delta(s, v)$ bizim sonuçlandırdığımız $v.d = \delta(s, v)$ ve bu eşitlik bundan sonra muhafaza edilir. ■

Lemma 24.15 (Yol-Gevşetme Özelliği)

Ağırlık fonksiyonu $w : E \rightarrow \mathbb{R}$, yönlü, ağırlıklı bir $G = (V, E)$ çizgesi ve $s \in V$ kaynak tepesi olsun. Herhangi bir $s = v_0$ dan v_k ya olan en-kısa yol $p = \langle v_0, v_1, \dots, v_k \rangle$ olsun. Eğer G , INITIALIZE-SINGLE-SOURCE(G, s) ile başlatılırsa ve sonra bir dizi gevşetme adımları sırasıyla $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$ ayrıtlarına uygulanırsa, o zaman bu gevşetmeler sonrasında $v_k.d = \delta(s, v_k)$ ve sonra her zaman böyledir. Bu özellik diğer ayrıt gevşetmeleri ne olursa olsun böyledir, p nin ayrıtlarının gevşetmeleri ile karışmış olan gevşetmeler dahil.

Kanıt Tümevarım ile p yolunun i inci ayrıtının gevşetilmesinden sonra $v_i.d = \delta(s, v_i)$ bildiğimizi gösteririz. Temel olarak, $i = 0$ ve önce p nin herhangi bir ayrıtı gevşetilmeden $v_0.d = s.d = 0 = \delta(s, s)$ başlangıcına sahibiz. Üst-sınır özelliği tarafından başlangıç sonrası $s.d$ değeri hiç değişmez.

Tümevarımsal adım için $v_{i-1}.d = \delta(s, v_{i-1})$ olduğunu varsayalım ve gevşetme ayrıtı (v_{i-1}, v_i) olduğunda ne olduğunu inceleyelim. Yakınsama özelliği tarafından, bu ayrıt gevşetildikten sonra $v_i.d = \delta(s, v_i)$ vardır ve bu eşitlik bundan sonra tüm zamanlarda muhafaza edilir. ■

Gevşetme ve En-Kısa-Yol Ağaçları

Şimdi, bir kere gevşetmeler dizisi en-kısa yolun tahmini en-kısa yol ağırlıklarına yakınsamasına neden olmasını gösterelim, π değeri sonuçları tarafından oluşan öncül altçizge G_π , G için en-kısa yol ağacıdır. Takip eden lemma, öncül altçizge her zaman kimin kök kaynağından köklü ağaç oluşturduğunu göstermeye başlıyoruz.

Lemma 24.16

$G = (V, E)$, ağırlık fonksiyonu $w : E \rightarrow \mathbb{R}$ olan yönlü ağırlıklı bir çizgedir, $s \in V$ kaynak tepesi ve kabul edelim ki G, s den ulaşılabilen negatif ağırlıklı çevrim içermesin. O zaman çizge INITIALIZE-SINGLE-SOURCE(G, s) ile başlandıktan sonra öncül altçizge G_π s kökü ile köklü bir ağaç ile oluşur ve G nin ayrıştılarındaki gevşetme adımı dizinleri bu özelliği sabit olarak muhafaza eder.

Kanıt İlk olarak, G_π deki tek tepe kaynak tepedir ve lemma açıkça doğrudur. Bir dizi gevşeme adımı sonucu ortaya çıkan öncül altçizge G_π yi dikkate alalım. Biz ilk olarak G_π nin çevrim içermediğini kanıtlamalıyız. Bazı gevşetme adımlarının G_π çizgesinde çevrim oluşturduğunu çelişki uğruna varsayalım. $c = \langle v_0, v_1, \dots, v_k \rangle$ çevrimi $v_k = v_0$ olan bir çevrim olsun. Sonra $i = 1, 2, \dots, k$ için $v_i.\pi = v_{i-1}$ dir ve genelliği kaybetmeden (v_{k-1}, v_k) ayrıştıını gevşetme G_π de çevrim oluşturacağını varsayabiliriz.

c çevrimi üzerindeki tüm tepelere s kaynağından ulaşılabilirliğini iddia edilir. Neden? c üzerindeki her tepe non-NIL öncüle sahiptir ve c üzerindeki her tepeye non-NIL π değeri atandığında sonlu en-kısa-yol tahmini atanır. Üst-sınır özelliği ile c çevrimi üzerindeki her tepe s den ulaşılabilen sonlu ağırlıklı en kısa yola sahip olduğunu gösterir.

RELAX(v_{k-1}, v_k, w) i çağırmadan hemen önce c üzerinde en-kısa-yol tahmini araştırabilir ve c nin negatif ağırlıklı çevrim olduğu gösterilir, dolayısıyla çelişen varsayım kaynaktan ulaşılabilen G nin içerdiği negatif ağırlıklı olmayan çevrimlerdir. Çağırmadan sadece biraz önce $i = 1, 2, \dots, k-1$ için $v_i.\pi = v_{i-1}$ vardır. Böylece, $i = 1, 2, \dots, k-1$ için $v_i.d$ nin en son güncellemesi $v_i.d = v_{i-1}.d + w(v_{i-1}, v_i)$ atamasıyladır. Eğer $v_{i-1}.d$ değiştiği zaman o azalır. Bundan dolayı RELAX(v_{k-1}, v_k, w) çağırmadan biraz önce

$$i = 1, 2, \dots, k-1 \text{ için } v_i.d \geq v_{i-1}.d + w(v_{i-1}, v_i) \quad (24.12)$$

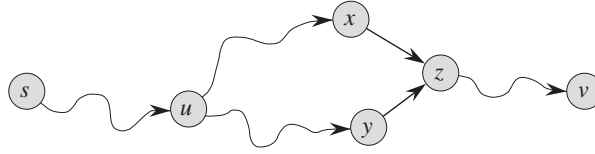
vardır.

Çağırmayla $v_k.\pi$ değiştiği için, hemen öncesinde mutlak eşitsizlikte vardır.

$$v_k.d > v_{k-1}.d + w(v_{k-1}, v_k).$$

Bu mutlak eşitsizliğin (24.12) deki $k-1$ eşitsizlikleri toplamıyla, c çevrimi etrafında en-kısa-yol tahmininin toplamı elde edilir:

$$\begin{aligned} \sum_{i=1}^k v_i.d &> \sum_{i=1}^k (v_{i-1}.d + w(v_{i-1}, v_i)) \\ &= \sum_{i=1}^k v_{i-1}.d + \sum_{i=1}^k w(v_{i-1}, v_i) . \end{aligned}$$



Şekil 24.9 s kaynağından v tepesine G_π deki temel yolun tek olduğunu gösterir. $x \neq y$ olduğunda eğer iki yol $p_1 (s \rightsquigarrow u \rightsquigarrow x \rightarrow z \rightsquigarrow v)$ ve $p_2 (s \rightsquigarrow u \rightsquigarrow y \rightarrow z \rightsquigarrow v)$ varsa $z.\pi = x$ ve $z.\pi = y$ bir çelişkidir.

Ama

$$\sum_{i=1}^k v_i \cdot d = \sum_{i=1}^k v_{i-1} \cdot d,$$

c çevrimindeki her tepe her toplamda tam bir kez ortaya çıktığından bu eşitlik

$$0 > \sum_{i=1}^k w(v_{i-1}, v_i)$$

yı belirtir.

Böylece istenen çelişkiyi sağlayan c çevrimi etrafındaki ağırlıkların toplamı negatifdir.

G_π yönlü çevrimsiz çizge olduğu kanıtlanmıştır. s kökü ile köklü bir ağacı oluşturduğunu göstermek için her $v \in V_\pi$ tepesi için kanıtlanması yeterlidir (bakınız Alıştırma B.5-2), G_π de s den v ye temel bir tek yol vardır.

Öncelikle V_π deki her tepe için, s den bir yol olduğu gösterilmelidir. V_π deki tepeler non-NIL π değerlerine ek s tepeleri vardır. Buradaki düşünce tümevarımla s den V_π deki tüm tepelere bir yol olduğunu kanıtlamaktır. Detaylar alıştırma 24.5-6 dadır.

Lemmanın kanıtını bitirmek için herhangi $v \in V_\pi$ nin her bir tepesi için G_π çizgesinin s den v ye en fazla bir tane basit yol içerdiği gösterilmelidir. Diğer türlü varsayalım. Şöyle, Şekil 24.9 da gösterildiği gibi, G_π s den v tepesine 2 tane basit yol içermektedir: $p_1, s \rightsquigarrow u \rightsquigarrow x \rightarrow z \rightsquigarrow v$ ve $p_2, s \rightsquigarrow u \rightsquigarrow y \rightarrow z \rightsquigarrow v$ şeklinde $x \neq y$ durumunda parçalanır. (u, s ve z, v olabilir.) Fakat sonra $x = y$ çelişkisini veren $z.\pi = x$ ve $z.\pi = y$ dir. G_π nin s den v ye tek basit yol içerdiği ve böylece G_π nin s köklü ağaç oluşturduğu hesaplanır. ■

Gevşetme adımlarının sıralaması uygulandıktan sonra tüm tepelerin en kısa yol ağırlıkları ile işaretlendiği böylece öncül altçizge G_π nin en kısa yollu ağaç olduğu gösterilebilir.

Lemma 24.17 (Öncül-altçizge Özelliği)

$G = (V, E)$ ağırlıklı, $w : E \rightarrow \mathbb{R}$ ağırlık fonksiyonu ile yönlü çizge olsun, $s \in V$ kaynak tepe, s den ulaşılabilen G nin negatif-olmayan-ağırlıklı çevrimler içerdiğini kabul edilsin. INITIALIZE-SINGLE-SOURCE (G, s) çağırılsın ve sonra her $v \in V$ için G nin ayrıtları üzerindeki gevşetme adımlarının herhangi bir dizisini çalıştırmak için $v.d = \delta(s, v)$ üretilir. O zaman, öncül altçizge G_π s de köklü bir en kısa-yol ağacıdır.

Kanıt G_π için sağlayan sayfa 647 de verilen en kısa-yol ağacının 3 tane özelliği kanıtlanmalıdır. İlk özelliği göstermek için V_π s den ulaşılabilen tepelerin kümesi olduğu gösterilmelidir. Tanım aracılığı ile $\delta(s, v)$ ağırlıklı bir en-kısa-yol ancak ve ancak s den v ye ulaşabilirse sonludur ve böylece s den ulaşılabilen tepeler tam olarak sonlu d değerleridir. Ancak bir $v \in V - \{s\}$ tepesine $v.d$ için bir sonlu değer atanmışsa ancak ve ancak $v.\pi \neq \text{NIL}$ dir. Böylece s den ulaşılabilen V_π deki tepeler tam olarak bunlardır.

İkinci özellik Lemma 24.16 dan takip edilir.

En-kısa-yolların ağaçlarının son özelliğini kanıtlamak için her $v \in V_\pi$ tepesi için s den v ye G deki en kısa yol G_π deki $s \xrightarrow{p} v$ tek temel yoldur. $v_0 = s$ ve $v_k = v$ iken $p = \langle v_0, v_1, \dots, v_k \rangle$ olsun. Hesaplanan $w(v_{i-1}, v_i) \leq \delta(s, v_i) - \delta(s, v_{i-1})$ sonucunda $i = 1, 2, \dots, k$ için $v_i.d = \delta(s, v_i)$ ve $v_i.d \geq v_{i-1}.d + w(v_{i-1}, v_i)$ vardır. p yolu boyunca toplanan ağırlıklar

$$\begin{aligned}
 w(p) &= \sum_{i=1}^k w(v_{i-1}, v_i) \\
 &\leq \sum_{i=1}^k (\delta(s, v_i) - \delta(s, v_{i-1})) \\
 &= \delta(s, v_k) - \delta(s, v_0) \quad (\text{çünkü toplam iç içe geçer}) \\
 &= \delta(s, v_k) \quad (\text{Çünkü } \delta(s, v_0) = \delta(s, s) = 0)
 \end{aligned}$$

dır.

Böylece $w(p) \leq \delta(s, v_k)$ dir. s den v_k ya herhangi bir yolun ağırlığının $\delta(s, v_k)$ alt sınırı için $w(p) = \delta(s, v_k)$ hesaplanır ve böylece p s den $v = v_k$ ya en kısa yoldur. ■

Alıştırmalar**24.5-1**

Şekil 24.2 de (sayfa 648) yönlü çizge için gösterilenden farklı iki tane en kısa-yol ağacını veriniz.

24.5-2

Ağırlıklı, yönlü, $w : E \rightarrow \mathbb{R}$ ağırlık fonksiyonlu $G = (V, E)$ çizgesi ve s kaynak tepesi ile G nin şu özellikleri sağladığı bir örnek veriniz: her $(u, v) \in E$ ayrıtı için (u, v) yi içeren s köklü en kısa-yol ağaçları ve (u, v) yi içermeyen diğer s köklü en kısa-yol ağaçları.

24.5-3

En kısa-yol ağırlıklarının $-\infty$ ya da ∞ olduğu durumların üstesinden gelebilmek için Lemma 24.10 nun kanıtını düzenleyiniz.

24.5-4

$G = (V, E)$ çizgesi s kaynak tepesi ile ağırlıklandırılmış, yönlü olsun ve G INITIALIZE-SINGLE-SOURCE(G, s) ile başlatılmış olsun. Eğer gevşetme dizisi adım adım $s.\pi$ den non-NIL değere ise G nin negatif ağırlıklı döngü içerdiğini kanıtlayınız.

24.5-5

$G = (V, E)$ negatif olmayan ayrıt ağırlıklı, yönlü bir çizge olsun. $s \in V$ kaynak tepe ve s kaynağından v ye v üzerinde herhangi bir en kısa yol v nin öncülü olan $v.\pi$ ye izin verelim eğer $v \in V - \{s\}$ için s den ulaşılabilir ve aksi halde NIL olacağını kabul edelim. Böyle bir G çizgesine ve G_π de bir çevre üreten π değerlerinin atanmasına bir örnek veriniz. (Lemma 24.16 daki, gevşetme adım dizileri ile bu atama üretilemez.)

24.5-6

$G = (V, E)$ $w : E \rightarrow \mathbb{R}$ ağırlık fonksiyonu ve negatif-ağırlıklı-çevreleri olmayan ağırlıklı ve yönlü bir çizge olsun. $s \in V$ kaynak tepesi ve G INITIALIZE-SINGLE-SOURCE(G, s) ile başlatılmış olsun. Her $v \in V_\pi$ için, G_π de s ten v ye bir yol vardır ve bu özellik herhangi bir gevşetme ayrıtında değişmez olarak korunur olduğunu kanıtlayınız.

24.5-7

$G = (V, E)$, negatif ağırlıklı çevrimsiz ağırlıklı, yönlü bir çizge olsun. $s \in V$ kaynak tepesi ve G INITIALIZE-SINGLE-SOURCE(G, s) ile başlatılmış olsun. Her $v \in V$ için $v.d = \delta(s, v)$ üreten $|V| - 1$ gevşetme adımlı bir dizinin var olduğunu kanıtlayınız.

24.5-8

G rassal ağırlıklı, s kaynağından ulaşılabilen negatif-ağırlık çevrimli yönlü çizge olsun. G nin ayrıtalarının sonsuz gevşetme dizisinin nasıl düzenlendiğini gösteriniz öyle ki; her gevşetme en-kısa-yol tahmini değişimine neden olmaktadır.

Problemler

24-1 Bellman-Ford'a Yen'in iyileştirmesi

Bellman-Ford algoritmasının her bir geçişinde ayrıt gevşetmelerinin aşağıdaki gibi sıralandığını varsayalım. İlk geçişten önce, girdi çizgesi $G = (V, E)$ nin tepeleri için rassal doğrusal sıralamayı $v_1, v_2, \dots, v_{|V|}$ olarak belirleyelim. Daha sonra ayrıt kümesi E yi $E_f = \{(v_i, v_j) \in E : i < j\}$ ve $E_b = \{(v_i, v_j) \in E : i > j\}$ olmak üzere $E_f \cup E_b$ şeklinde parçalayalım. (G nin kendi kendine döngüleri içermediğini varsayalım, bu durumda her ayrıt ya E_f de ya da E_b dedir.) $G_f = (V, E_f)$ ve $G_b = (V, E_b)$ olarak tanımlanır.

- a. $\langle v_1, v_2, \dots, v_{|V|} \rangle$ topolojik sıralaması ile G_f nin çevrimsiz olduğunu ve $\langle v_{|V|}, v_{|V|-1}, \dots, v_1 \rangle$ topolojik sıralaması ile G_b nin çevrimsiz olduğunu kanıtlayınız.

Bu şekilde Bellman-Ford algoritmasının her bir geçişinde uyguladığımızı varsayalım. Tepelerden ayrılan E_f gevşeme ayrıtları $v_1, v_2, \dots, v_{|V|}$ sırasıyla her bir tepeye uğratırız. Her bir tepeye uğradıktan sonra tepelerden ayrılan E_b gevşetme ayrıtları $v_{|V|}, v_{|V|-1}, \dots, v_1$ sırası ile her bir tepeye uğrar.

- b. Bu düzende, eğer G kaynak tepesi s den ulaşılabilen negatif ağırlıklı olmayan çevrimler içeriyorsa ve daha sonra ayrıtlardan sadece $\lceil |V|/2 \rceil$ tanesinden geçtiğinde, her $v \in V$ için $v.d = \delta(s, v)$ olduğunu kanıtlayınız.
- c. Bu düzen Bellman-Ford algoritmasının asimptotik çalışma süresini artırır mı?

24-2 İç içe konan kutular

Eğer $\{1, 2, \dots, d\}$ üzerinde $x_{\pi(1)} < y_1, x_{\pi(2)} < y_1, \dots, x_{\pi(d)} < y_1$ gibi bir π permütasyonu varsa $(x_1, x_2, \dots, x_d)$ boyutlarıyla bir d -boyutlu kutu $(y_1, y_2, \dots, y_d)$ boyutlu diğer kutunun içine **konur**.

- a. İç içe konma bağıntısının geçişken olduğunu tartışınız.
- b. Bir d -boyutlu kutunun içine diğerinin yerleştirilip yerleştirilememesiyle ilgili etkili bir yöntem belirleyip tanımlayınız.
- c. Size verilen $\{B_1, B_2, \dots, B_n\}$ n tane d -boyutlu kutular kümesi olduğunu kabul edelim. $B_{ij} j = 1, 2, \dots, k-1$ için B_{i_j+1} in içine yerleşen en uzun $\langle B_{i_1}, B_{i_2}, \dots, B_{i_k} \rangle$ dizisinin kutularını bulan etkili bir algoritma verin. n ve d açısından algoritmanızın çalışma zamanını ifade ediniz.

24-3 Kur farklılıklarından yararlanarak para alıp satma

Kur farklılıklarından yararlanarak para alıp satma, bir para biriminin aynı miktarda birden fazla para birimine dönüşümündeki cari döviz kurlarındaki farklılıklardan yararlanır. Örneğin, 1 Amerikan Dolarının 49 Hint Rupisi, 1 Hint Rupisinin 2 Japon Yeni, ve 1 Japon Yeninin 0.0107 Amerikan Doları aldığını varsayalım. Tüccar bu para dönüşümü ile 1 Amerikan Doları ile başlayarak $49 \times 2 \times 0,0117 = 1,0486$ Amerikan Doları alabilir, böylelikle %4.86 kazanç elde eder.

Bir c_i para birimi ile c_j para birimi $R[i, j]$ birim satın alan, n adet $c_1, c_2, \dots, c_n$ paranın ve döviz kuru R nin $n \times n$ lik bir tablo olduğunun bize verildiğini varsayalım.

- a. $R[i_1, i_2] \cdot R[i_2, i_3] \cdots R[i_{k-1}, i_k] \cdot R[i_k, i_1] > 1$ gibi bir $\langle c_{i_1}, c_{i_2}, \dots, c_{i_k} \rangle$ para birimi dizisi olup olmadığını belirlemek için etkili bir algoritma yazınız.

Algoritmanızın çalışma süresini inceleyiniz.

- b. Eğer varsa, böyle bir dizi yazdırmak için etkin bir algoritma yazınız. Algoritmanızın çalışma süresini inceleyiniz.

24-4 Tek-kaynaklı en kısa yollar için Gabow'un ölçeklendirme algoritması

Ölçeklendirme (scaling) algoritması, başlangıçta her bir ilgili girdi değerinin (ayrıt ağırlığı gibi) sadece en yüksek mertebeli parçasını ele alarak problemi çözer. Daha sonra başlangıç çözümü iki en yüksek mertebeli basamağa (bit) bakılarak düzelir. Tüm basamakları inceleyene ve doğru çözümü hesaplayana kadar, her seferinde çözümü düzenleyerek, giderek daha yüksek mertebeli basamağa bakar.

Bu problemde, ayrıt ağırlıklarını ölçeklendirerek tek bir kaynaktan en kısa yolu hesaplamak için bir algoritma inceliyoruz. $G = (V, E)$ yönlü çizgesinin ayrıt ağırlıkları negatif olmayan bir tam sayı olan w ve $W = \max_{(u,v) \in E} \{w(u, v)\}$ olsun. Amacımız $O(E \lg W)$ zamanda çalışan bir algoritma geliştirmek. Tüm tepelerin kaynaktan ulaşılabilir olduğunu varsayalım.

Algoritma, en önemli basamaktan en önemsiz basamağa kadar ayrıt ağırlıklarının ikili gösteriminde basamakları birer birer ortaya çıkarır. Özellikle W nin ikili ifadelerinde $k = \lceil \lg(W + 1) \rceil$ basamak sayısı ve $i = 1, 2, \dots, k$ için $w_i(u, v) = \lceil w(u, v) / 2^{k-i} \rceil$ olsun. Bu durumda verilen i , $w(u, v)$ nin en önemli basamağı olmak üzere; $w_i(u, v)$, $w(u, v)$ nin “ufaltılmış” (scaled-down) sürümüdür. (Böylelikle, her $(u, v) \in E$ için $w_k(u, v) = w(u, v)$ dir.) Örneğin ikili ifadesi $\langle 1101 \rangle$ olduğunda $k = 5$ iken $w(u, v) = 25$ ise, $w_3(u, v) = \langle 110 \rangle = 6$ olur. $k = 5$ olan başka bir örnekte eğer $w(u, v) = \langle 00100 \rangle = 4$ ise $w_3(u, v) = \langle 001 \rangle = 1$ dir. $\delta_i(u, v)$ yi, w_i ağırlık fonksiyonunu kullanarak u tepesinden v tepesine en kısa yol olarak tanımlayalım. Böylelikle her $u, v \in V$ için $\delta_k(u, v) = \delta(u, v)$ dir. Verilen s tepe kaynağı için ölçeklendirme algoritması her $v \in V$ için ilk olarak en kısa yol algoritması $\delta_1(s, v)$ yi hesaplar, daha sonra her $v \in V$ için $\delta_1(s, v)$ yi hesaplar, her $v \in V$ için $\delta_k(s, v)$

yi hesaplayana kadar bu şekilde devam eder. $|E| \geq |V| - 1$ süresince devam ettiğini varsayalım ve δ_{i-1} den δ_i nin $O(E)$ zamanda hesaplanacağını göreceğiz, böylece tüm algoritma $O(k, E) = O(E \lg W)$ zaman alır.

a. Her $v \in V$ tepeleri için $\delta(s, v) \leq E$ olduğunu varsayınız. Her $v \in V$ için $O(E)$ zamanda $\delta(s, v)$ yi hesaplayabileceğimizi gösteriniz.

b. Her $v \in V$ için $O(E)$ zamanda $\delta_1(s, v)$ yi hesaplayabileceğimizi gösteriniz.

Şimdi δ_{i-1} den δ_i yi hesaplayabileceğimiz üzerine yoğunlaşınız.

c. $i = 2, 3, \dots, k$ için ya $w_i(u, v) = 2w(i-1)(u, v)$ ya da $w_i(u, v) = 2w(i-1)(u, v) + 1$ olduğunu kanıtlayınız. Daha sonra, her $v \in V$ için

$$2\delta_{i-1}(s, v) \leq \delta_i(s, v) \leq 2\delta_{i-1}(s, v) + |V| + 1$$

olduğunu kanıtlayınız.

d. $i = 1, 2, \dots, k$ ve her $u, v \in V$ için

$$\hat{w}_i(u, v) = w_i(u, v) + 2\delta_{i-1}(s, u) - 2\delta_{i-1}(s, v)$$

olarak tanımlayınız.

$i = 1, 2, \dots, k$ ve her $u, v \in V$ için $\hat{w}_i(u, v)$ nin (u, v) ayrıtının “yeniden ağırlıklandırılmış” değerinin negatif olmayan bir tamsayı olduğunu kanıtlayınız.

e. $\hat{\delta}_i(s, v)$ yi ağırlık fonksiyonu $\hat{w}_i$ yi kullanarak s den v ye en kısa yol olarak tanımlayalım. $i = 1, 2, \dots, k$ ve her $v \in V$ için

$$\delta_i(s, v) = \hat{\delta}_i(s, v) + 2\delta_{i-1}(s, v) \text{ ve } \hat{\delta}_i(s, v) \leq |E| \text{ olduğunu kanıtlayınız.}$$

f. Her $v \in V$ için $O(E)$ zamanda $\delta_{i-1}(s, v)$ den $\delta_i(s, v)$ yi nasıl hesapladığımızı ve sonunda her $v \in V$ için $O(E \lg W)$ zamanda $\delta(s, v)$ yi hesaplayabileceğimizi gösteriniz.

24-5 Karp’ın minimum ortalama-ağırlıklı çevrim algoritması

Ağırlık fonksiyonu $w : E \rightarrow \mathbb{R}$ ile $G = (V, E)$ yönlü bir çizge ve $n = |v|$ olsun. E ayrıtında $c = \langle e_1, e_2, \dots, e_k \rangle$ çevrimlerindeki **ortalama-ağırlığı** (*mean weight*)

$$\mu(c) = \frac{1}{k} \sum_{i=1}^k w(e_i)$$

olarak tanımlarız.

c nin aralığı G deki bütün yönlü çevrimleri kapsıyor iken $\mu^* = \min_c \mu(c)$ olsun. $\mu(c) = \mu^*$ olan c çevresi **minimum ortalama-ağırlıklı çevrim (minimum mean-weight cycle)** olarak adlandırılır. Bu problem μ^* ı incelemek için etkili bir algoritma inceler.

Genelliği kaybetmeden, her $v \in V$ tepesinin kaynak tepesi $s \in V$ den ulaşılabilir olduğunu varsayalım. $\delta(s, v)$ s den v ye en kısa yolun ağırlığı ve $\delta_k(s, v)$ tam k tane ayrıt içeren s den v ye en kısa yolun ağırlığı olsun. s den v ye tam k tane ayrıt içeren bir yol yoksa $\delta_k(s, v) = \infty$ dur.

a. $\mu^* = 0$ olduğunda; G nin negatif ağırlıklı çevrim içermediğini ve tüm $v \in V$ tepeleri için $\delta_k(s, v) = \min_{0 \leq k \leq n-1} \delta_k(s, v)$ olduğunu gösteriniz.

b. $\mu^* = 0$ olduğunda; her $v \in V$ tepeleri için

$$\max_{0 \leq k \leq n-1} \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k} \geq 0$$

olduğunu gösteriniz. (*İpucu:* (a) kısmındaki iki özelliği de kullanınız.)

c. 0-ağırlıklı çevre olsun ve u ve v c de herhangi iki tepe olsun. $\mu^* = 0$ ve çevre boyunca u dan v ye basit bir yolun ağırlığının x olduğunu varsayınız. $\delta(s, v) = \delta(s, u) + x$ olduğunu kanıtlayınız. (*İpucu:* çevrim boyunca v den u ya basit yolların ağırlığı $-x$ tir.)

d. $\mu^* = 0$ olduğunda, her bir minimum ortalama-ağırlıklı çevriminin üzerinde

$$\max_{0 \leq k \leq n-1} \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k} = 0$$

olan bir v tepesi olduğunu gösteriniz. (*İpucu:* Çevrim üzerindeki bir sonraki tepedeki en kısa yolu elde etmek için, çevrim boyunca en küçük ortalama-ağırlıklı çevresini üzerindeki herhangi bir tepe için en kısa yolun nasıl genişletildiğini gösteriniz.)

e. $\mu^* = 0$ olduğunda

$$\min_{v \in V} \max_{0 \leq k \leq n-1} \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k} = 0$$

olduğunu gösteriniz.

f. G deki herhangi bir ayrıt ağırlığına t sabitini eklersek, μ^* ın t ile arttığını gösteriniz. Bu bulguyu,

$$\mu^* = \min_{v \in V} \max_{0 \leq k \leq n-1} \frac{\delta_n(s, v) - \delta_k(s, v)}{n - k}$$

eşitliğini göstermek için kullanınız.

g. μ^* 'ı hesaplamak için $O(VE)$ -zamanlı bir algoritma veriniz.

24-6 Çiftdüze (bitonic)

Bir dizi sürekli artıyor ve sonra sürekli azalıyor ya da, döngüsel öteleme ile sürekli artıyor ve sonra sürekli azalıyor ise **çiftdüze**dir. Örneğin, $\langle 1, 4, 6, 8, 3, -2 \rangle$, $\langle 9, 2, -4, -10, -5 \rangle$ ve $\langle 1, 2, 3, 4 \rangle$ dizileri çiftdüzedir, ancak $\langle 1, 3, 12, 4, 2, 0 \rangle$ içiftdüze değildir. (çiftdüze Öklit gezgin-satıcı problemi için Problem 15-3 e bakınız.)

Tüm ayrıt ağırlıkları bir değerli olan, $w : E \rightarrow \mathbb{R}$ ağırlık fonksiyonu ile $G = (V, E)$ yönlü çizgesi verildiğini ve s tepe kaynağından tek-kaynaklı en kısa yolu bulmayı istediğimizi varsayalım. Bir ek bilgi-birim verilir: her bir $v \in V$ tepesi için, s den v ye herhangi bir yol boyunca ayrıt ağırlıkları çiftdüze bir dizi oluşur.

Bu problemi çözebilmek için en etkili algoritmayı veriniz ve çalışma süresini analiz ediniz.

Bölüm notları

Dijkstra algoritması [88] 1959 da ortaya çıkmıştır, ancak içeriğinde öncelik sırasına değinilmemiştir. Bellman-Ford algoritması Bellman [38] ve Ford [109] tarafından farklı algoritmalara dayanmaktadır. Bellman farklı kısıtlamalarla en kısa yol bağlantısını tanımlar. Halk bilgisi kısmını düşünen Lawler [224] bir dag'da en kısa yol için doğrusal zaman algoritmasını tanımlar.

Ayrıt ağırlıkları nispeten küçük negatif olmayan tam sayı olduğunda, tek-kaynaklı en kısa yol problemini çözmek için daha etkili bir algoritma elde ederiz. Dijkstra algoritmasında EXTRACT-MIN aramalar ile döndürülen değerler dizisi monoton zamanla artar. Bölüm 6 notlarında tartışıldığı gibi, bu durumda birçok veri yapısı çeşitli öncelik-sırası işlemleri ikili yığın yada Fibonacci bellek yığınlarının daha etkili bir şekilde uygulanabilir. Ahuja, Mehlhorn, Orlin ve Tarjan [8] çizgelerde herhangi bir ayrıtının en büyük ağırlığı W olan, negatif olmayan ayrıt ağırlığı ile çizge üzerinde $O(E + V \sqrt{\lg W})$ zamanda çalışan bir algoritma verir. En iyi sınırlar Thorup [337] tarafından verilen $O(E \lg \lg V)$ zamanda çalışan algoritma ve Raman [291] tarafından verilen $O(E + V \min \{(\lg V)^{1/3+\epsilon}, (\lg W)^{1/4+\epsilon}\})$ zamanda çalışan algoritmadır. Bu iki algoritma esas mekanizmanın sözcük uzunluğuna bağlı olan boşluk miktarını kullanır. Girdinin boyutunda boşluk miktarı sınırlandırılmamış olmasına rağmen rassal hesaba dayalı adresleme kullanılarak girdi büyüklüğü doğrusala indirgenebilir.

Tam sayı ağırlıkları olan yönsüz çizgelerde Thorup [336] tek-kaynaklı en kısa yol için $O(E + V)$ zaman algoritmasını verir. Önceki paragrafta açıklanan algo-

ritmanın aksine, EXTRACT-MIN aramalar ile döndürülen değerler dizisi monoton zamanla artmadığından bu algoritma Dijkstra algoritmasının uygulaması değildir.

Negatif ayrıt ağırlıklı çizgeler için, Gabow ve Tarjan[122] algoritması $O(\sqrt{V}E \lg(VW))$ zamanda çalışır ve Goldberg [137] nin $W = \max_{(u,v) \in E} \{|w(u, v)|\}$ olan bir algoritması $O(\sqrt{V}E \lg W)$ zamanda çalışır.

Cherkassky, Goldberg, ve Radzik [64] çeşitli kısa yol algoritmaları karşılaştıran geniş deneyler yaptılar.

Bu bölümde, bir çizgedeki tepelerin tüm ikilileri arasında en kısa yolları bulan problemi ele alacağız. Bu problem bir yol atlası için şehirlerin tüm ikilileri arasındaki mesafe tablosunu yaparken ortaya çıkabilir. 24. Bölüm de olduğu gibi, ayrıtları gerçel değerli ayrıtlarla eşleştiren $w : E \rightarrow \mathbb{R}$ ağırlık fonksiyonlu bir ağırlıklandırılmış, yönlü bir $G = (V, E)$ çizgesi verilsin. Her $u, v \in V$ tepe ikilileri için u dan v ye en kısa (en az ağırlıklı) bir yol bulmak istiyoruz. Bir yolun ağırlığı kendisini oluşturan ayrıtların ağırlıkları toplamıdır. Genellikle tablo şeklinde bir çıktı isteriz: u satırı ve v sütunundaki eleman u dan v ye en kısa bir yolun ağırlığı olmalıdır.

Tüm-ikililerin en kısa yolları problemini bir tek-kaynaklı en kısa yollar algoritmasını her tepeyi bir kez kaynak seçip $|V|$ defa çalıştırarak çözebiliriz. Eğer tüm ayrıt ağırlıkları negatif değil ise, Dijkstra'nın algoritmasını kullanabiliriz. Eğer min-öncelikli kuyruğun doğrusal-dizi uygulamasını kullanırsak çalışma zamanı $O(V^3 + VE) = O(V^3)$ dür. Min-öncelikli kuyruğun ikili min-bellek yığını uygulaması ise $O(VE \lg V)$ çalışma zamanını verir, ki eğer çizge seyrek ise bir iyileştirmedir. Alternatif olarak, min-öncelikli kuyruğu $O(V^2 \lg V + VE)$ çalışma zamanını veren Fibonacci bellek yığını ile uygulayabiliriz.

Eğer çizge negatif-ağırlıklı ayrıtlara sahip ise, Dijkstra'nın algoritmasını kullanamayız. Yerine, daha yavaş olan Bellman-Ford algoritmasını her tepe için bir kez çalıştırmalıyız. Ortaya çıkan çalışma zamanı $O(V^2E)$ dir, yoğun çizgeler de ise $O(V^4)$ dir. Bu bölümde nasıl daha iyi yapılacağını göreceğiz. Ayrıca tüm-ikililerin en kısa yolları probleminin matris çarpımı ile ilişkisini araştırıp bunun cebirsel yapısını çalışacağız.

Çizgenin bitişiklik-listesi gösterimini kabul eden tek-kaynaklı algoritmaların tersine, bu bölümdeki algoritmaların çoğu bitişiklik-matrisi gösterimini kullanır. (Kısım 25.3 deki Johnson'un algoritması seyrek çizgeler için bitişiklik listelerini kullanır.) Kolaylık sağlamak için, tepelerin $1, 2, \dots, |V|$ ile numaralandırıldığını kabul edelim, öyle ki girdi $n \times n$ lik n -tepeli yönlü bir $G = (V, E)$ çizgesinin ayrıt ağırlıklarını temsil eden bir W matrisidir. Yani, $W = (w_{ij})$, öyle ki

$$w_{ij} = \begin{cases} 0, & i = j \text{ ise,} \\ (i, j) \text{ yönlü ayrıtının ağırlığı,} & i \neq j \text{ ve } (i, j) \in E \text{ ise,} \\ \infty, & i \neq j \text{ ve } (i, j) \notin E \text{ ise.} \end{cases} \quad (25.1)$$

Negatif-ağırlıklı ayrıtlara izin veriyoruz, fakat şimdilik girdi çizgesinin negatif-ağırlıklı çevrimler içermediğini kabul ediyoruz.

Bu bölümde sunulan tüm-ikililerin en kısa yolları algoritmalarının tablo şeklindeki çıktısı $n \times n$ lik bir $D = (d_{ij})$ matrisidir, öyle ki d_{ij} ögesi i tepesinden j tepesine en kısa bir yolun ağırlığını içermektedir. Yani, eğer i tepesinden j tepesine en kısa yolun ağırlığını $\delta(i, j)$ ile tanımlarsak (Bölüm 24 te olduğu gibi), neticede $d_{ij} = \delta(i, j)$ olur.

Bir girdi bitişiklik matrisi üzerinde tüm-ikililerin en kısa yolları problemini çözmek için sadece en kısa-yol ağırlıklarının değil aynı zamanda bir $\Pi = (\pi_{ij})$ **öncül matrisini (predecessor matrix)** hesaplayama da ihtiyacımız vardır. Eğer $i = j$ ise veya i den j ye bir yol yok ise π_{ij} NIL dir, aksi halde π_{ij} i den olan bir en kısa yol üzerinde j 'nin öncülüdür. Tıpkı Bölüm 24 te G_π öncül altçizgesinin verilen bir kaynak tepesi için bir en kısa-yol ağacı olması gibi. Π matrisinin i . satırı tarafından indirgenen altçizge kökü i olan bir en kısa-yol ağacıdır. Her $i \in V$ tepesi için G nin **öncül altçizgesini** $G_{\pi,i} = (V_{\pi,i}, E_{\pi,i})$ olarak tanımlarız, öyle ki

$$V_{\pi,i} = \{j \in V : \pi_{ij} \neq \text{NIL}\} \cup \{i\}$$

ve

$$E_{\pi,i} = \{(\pi_{ij}, j) : j \in V_{\pi,i} - \{i\}\}$$

dir.

Eğer $G_{\pi,i}$ bir en kısa-yol ağacı ise, Bölüm 22 deki PRINT-PATH prosedürünün değiştirilmiş bir versiyonu olan aşağıdaki prosedür i tepesinden j tepesine en kısa bir yolu yazar.

PRINT-ALL-PAIRS-SHORTEST-PATH(Π, i, j)

```

1  if  $i == j$ 
2      print  $i$ 
3  elseif  $\pi_{ij} == \text{NIL}$ 
4      print  $i$  “den”  $j$  “ye yol yoktur”
5  else PRINT-ALL-PAIRS-SHORTEST-PATH( $\Pi, i, \pi_{ij}$ )
6      print  $j$ 
```

Bu bölümdeki tüm-ikililer algoritmalarının temel özelliklerini vurgulamak için, Bölüm 24 de öncül altçizgelerle kapsamlı bir şekilde ilgilendiğimiz gibi öncül matrislerin özelliklerini ve oluşturulmasını ele almayacağız. Alıştırmaların bazıları en temel özellikleri kapsamaktadır.

Bölüm taslağı

Kısım 25.1 tüm-ikililerin en kısa-yolları problemini çözmek için matris çarpımı tabanlı bir dinamik-programlama algoritması sunar. “Tekrarlı kareleme” tekniğı kullanarak $\Theta(V^3 \lg V)$ çalışma zamanına erişebiliriz. Kısım 25.2 $\Theta(V^3)$ zamanında çalışan bir diğer dinamik-programlama algoritması Floyd-Warshall algoritmasını verir. Ayrıca, Kısım 25.2 tüm-ikililerin en kısa-yolları problemi ile ilişkisi olan yönlü bir çizgenin geçişli kapanışını bulma problemini kapsamaktadır. Son olarak, Kısım 25.3 tüm-ikililerin en kısa-yolları problemini $\Theta(V^2 \lg V + VE)$ zamanında çözen ve büyük, seyrek çizgeler için iyi bir seçim olan Johnson algoritmasını tanıtır.

İlerlemeden önce, bitişiklik-matrisi gösterimleri için bazı yazı biçimlerine ihtiyacımız var. Öncelikle, $G = (V, E)$ girdi çizgesinin n tepeye sahip olduğunu kabul edeceğiz. Yani $n = |V|$ olacak. İkinci olarak, matrisleri W, L veya D gibi büyük harflerle ve matrislerin elemanlarını w_{ij}, l_{ij} , veya d_{ij} gibi alt-indisli küçük harfler ile temsil edeceğiz. Bazı matrisler iterasyonları belirtmek için $L^{(m)} = (l_{ij}^{(m)})$ veya $D^{(m)} = (d_{ij}^{(m)})$ deki gibi parentezlenmiş üst-indislere sahip olacaktır. Son olarak, verilen bir $n \times n$ lik A matrisi için n değerinin $A.rows$ niteliğı içerisinde saklandığını kabul edeceğiz.

25.1 En kısa yollar ve matris çarpımı

Bu bölüm yönlü bir $G = (V, E)$ çizgesi üzerinde tüm-ikililerin en kısa yolları problemi için bir dinamik programlama algoritması sunar. Dinamik programın her ana döngüsü matris çarpımına çok benzer bir işlemi çalıştıracaktır öyle ki algoritma tekrarlı matris çarpımına benzeyecektir. Tüm-ikililerin en kısa yolları problemi için bir $\Theta(V^4)$ zamanlı algoritma geliştirerek başlayıp daha sonra bunun çalışma zamanını $\Theta(V^3 \lg V)$ olarak iyileştireceğiz.

İlerlemeden önce, bir dinamik-programlama algoritması geliştirmek için Bölüm 15’te verilen adımları kısaca tekrarlayalım.

1. Optimal bir çözümün yapısının karakteristiğı ortaya çıkarmak.
2. Özyinelemeli olarak optimal bir çözümün değerini tanımlamak.
3. Aşağıdan-yukarıya (bottom-up) mantığı ile optimal bir çözümün değerini hesaplamak.

Dördüncü adımı “hesaplanan bilgilerden optimal bir çözüm oluşturmak” alıştır-malar için ayırıyoruz.

Bir en kısa yolun yapısı

Optimal bir çözümün yapısını karakterize ederek başlıyoruz. Bir $G = (V, E)$ çizgesi üzerinde tüm-ikililerin en kısa yolları problemi için, bir en kısa yolun tüm alt-yolları gene en kısa yollardır (Lemma 24.1) olduğunu daha önce kanıtladık. Çizgesi bir $W = (w_{ij})$ bitişiklik-matrisi ile temsil ettiğimizi kabul edelim. i tepesinden j tepesine en kısa bir p yolunu ele alalım ve p nin en fazla m adet ayrıt içerdiğini kabul edelim. Negatif-ağırlıklı çevrim olmadığını, m nin sonlu olduğunu varsayalım. Eğer $i = j$ ise p nin ağırlığı 0 ve ayrıt içermez. Eğer i ve j tepeleri farklı ise p yolunu $i \xrightarrow{p'} k \rightarrow j$ olarak parçalarız, p' yolu şimdi $m - 1$ ayrıt içermektedir. Lemma 24.1 den p' yolu i den k ya bir en kısa yoldur ve böylece $\delta(i, j) = \delta(i, k) + w_{kj}$ dir.

Tüm-ikililerin en kısa yolları problemine özyinelemeli bir çözüm

Şimdi, i tepesinden j tepesine en fazla m ayrıt içeren herhangi bir yolun minimum ağırlığı $l_{ij}^{(m)}$ olsun. $m = 0$ olduğunda ancak ve ancak $i = j$ ise i den j ye ayrıtları olmayan bir en kısa yol vardır. Bu yüzden,

$$l_{ij}^{(0)} = \begin{cases} 0, & i = j \text{ ise,} \\ \infty, & i \neq j \text{ ise,} \end{cases}$$

$m \geq 1$ için $l_{ij}^{(m)}$ yi $l_{ij}^{(m-1)}$ in (i den j ye en fazla $m - 1$ ayrıt içeren bir en kısa yolun ağırlığı) minimumu olarak hesaplarız ve j nin tüm mümkün k öncüllerine bakarak elde edilen i den j ye en fazla m ayrıt içeren herhangi bir yolun minimum ağırlığı olarak hesaplarız. Bu yüzden özyinelemeli olarak

$$\begin{aligned} l_{ij}^{(m)} &= \min \left(l_{ij}^{(m-1)}, \min_{1 \leq k \leq n} \{ l_{ik}^{(m-1)} + w_{kj} \} \right) \\ &= \min_{1 \leq k \leq n} \{ l_{ik}^{(m-1)} + w_{kj} \} \end{aligned} \quad (25.2)$$

tanımlarız. İkinci eşitlik tüm j ler için $w_{jj} = 0$ olduğundan doğrudur.

Gerçek en kısa-yol ağırlıkları $\delta(i, j)$ nelerdir? Eğer çizge negatif-ağırlıklı çevrim içermiyorsa, $\delta(i, j) < \infty$ olan her i ve j tepe ikilileri için i den j ye en fazla $n - 1$ ayrıt içeren bir en kısa yol vardır. i tepesinden j tepesine $n - 1$ den daha fazla ayrıt içeren bir yol i den j ye bir en kısa yoldan n daha az ağırlığa sahip olamaz. Bu yüzden gerçek en kısa-yol ağırlıkları şu şekilde verilmiştir

$$\delta(i, j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = l_{ij}^{(n+1)} = \dots \quad (25.3)$$

En kısa yol ağırlıklarını aşağıdan-yukarıya hesaplama

Girdi olarak $W = (w_{ij})$ matrisini alalım, şimdi $L^{(1)}, L^{(2)}, \dots, L^{(n-1)}$ matrisler serisini hesaplıyoruz, öyle ki $m = 1, 2, \dots, n - 1$ için $L^{(m)} = (l_{ij}^{(m)})$ dir. Son $L^{(n-1)}$ matrisi gerçek en kısa-yol ağırlıklarını içermektedir. Tüm $i, j \in V$ tepeleri için $l_{ij}^{(1)} = w_{ij}$ olduğunu gözlemleyin ve böylece $L^{(1)} = W$.

Algoritmanın kalbi verilen $L^{(m-1)}$ ve W matrisleri için $L^{(m)}$ matrisini döndüren aşağıdaki prosedürdür. Yani algoritma o ana kadar hesaplanan en kısa yolları bir ayırt daha kullanarak genişletir.

EXTEND-SHORTEST-PATHS(L, W)

```

1   $n = L.rows$ 
2   $L' = (l'_{ij})$   $n \times n$  lik yeni bir matris olsun
3  for  $i = 1$  to  $n$ 
4      for  $j = 1$  to  $n$ 
5           $l'_{ij} = \infty$ 
6          for  $k = 1$  to  $n$ 
7               $l'_{ij} = \min(l'_{ij}, l_{ik} + w_{kj})$ 
8  return  $L'$ 
```

Prosedür en sonunda geri döndürdüğü bir $L' = (l'_{ij})$ matrisi hesaplar. Bunu tüm i ve j ler için (25.2) eşitliğini hesaplayarak, $L^{(m-1)}$ için L yi $L^{(m)}$ için L' nü kullanarak yapar (Girdi ve çıktı matrislerini m 'den bağımsız yapmak için üst-indisler yazılmamıştır). İç içe üç for döngüsünden dolayı çalışma zamanı $\Theta(n^3)$ tür.

Şimdi matris çarpımı ile olan ilişkiyi görebiliriz. İki tane $n \times n$ lik A ve B matrisinin çarpımı olan $C = A \cdot B$ matrisini hesaplamak istediğimizi varsayalım. O zaman $i, j = 1, 2, \dots, n$ için

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj} \quad (25.4)$$

hesaplarız.

Aşağıdaki yerine koymaları (25.2) eşitliğinde yaparsak

$$\begin{aligned}
 l^{(m-1)} &\rightarrow a, \\
 w &\rightarrow b, \\
 l^{(m)} &\rightarrow c, \\
 \min &\rightarrow +, \\
 + &\rightarrow \cdot
 \end{aligned}$$

(25.4) eşitliğini elde edeceğimizi gözlemleyin. Bu yüzden, bu değişiklikleri EXTEND-SHORTEST-PATHS te yaparsak ve aynı zamanda ∞ u (min için kimlik) 0 (+

için kimlik) ile değiştirirsek, Kısım 4.2 de gördüğümüz kare matrislerin çarpımındaki $\Theta(n^3)$ zaman prosedürünün aynısını elde ederiz.

SQUARE-MATRIX-MULTIPLY(A, B)

```

1   $n = A.rows$ 
2   $C$   $n \times n$  lik yeni bir matris olsun
3  for  $i = 1$  to  $n$ 
4      for  $j = 1$  to  $n$ 
5           $c_{ij} = 0$ 
6          for  $k = 1$  to  $n$ 
7               $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$ 
8  return  $C$ 
```

Tüm-ikililerin en kısa yolları problemine geri dönerek, en kısa-yol ağırlıklarını en kısa yolları ayırıt ayırıt genişleterek hesaplarız. $A \cdot B$, EXTEND-SHORTEST-PATHS(A, B) ile dönen matris “çarpımını” temsil etsin. $n - 1$ matris dizisini

$$\begin{aligned}
 L^{(1)} &= L^{(0)} \cdot W = W, \\
 L^{(2)} &= L^{(1)} \cdot W = W^2, \\
 L^{(3)} &= L^{(2)} \cdot W = W^3, \\
 &\vdots \\
 L^{(n-1)} &= L^{(n-2)} \cdot W = W^{n-1}
 \end{aligned}$$

hesaplarız. Yukarıda tartıştığımız gibi $L^{(n-1)} = W^{n-1}$ matrisi en kısa-yol ağırlıklarını içerir. Aşağıdaki prosedür bu diziyi $\Theta(n^4)$ zamanında hesaplar.

SLOW-ALL-PAIRS-SHORTEST-PATHS(W)

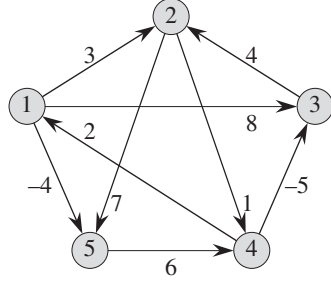
```

1   $n = W.rows$ 
2   $L^{(1)} = W$ 
3  for  $m = 2$  to  $n - 1$ 
4       $L^{(m)}$   $n \times n$  lik yeni bir matris olsun
5       $L^{(m)} = \text{EXTEND-SHORTEST-PATHS}(L^{(m-1)}, W)$ 
6  return  $L^{(n-1)}$ 
```

Şekil 25.1 bir çizgeyi SLOW-ALL-PAIRS-SHORTEST-PATHS prosedürü ile hesaplanan $L^{(m)}$ matrislerini ve bir çizgeyi göstermektedir.

Çalışma zamanını iyileştirmek

Amacımız yine de tüm $L^{(m)}$ matrislerini hesaplamak değildir: sadece $L^{(n-1)}$ matrisi ile ilgileniyoruz. Negatif-ağırlıklı çevrimlerin yokluğunda (25.3) eşitliğinin



$$L^{(1)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \quad L^{(2)} = \begin{pmatrix} 0 & 3 & 8 & 2 & -4 \\ 3 & 0 & -4 & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & \infty & 1 & 6 & 0 \end{pmatrix}$$

$$L^{(3)} = \begin{pmatrix} 0 & 3 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \quad L^{(4)} = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix}$$

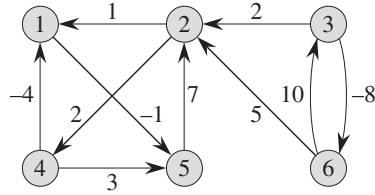
Şekil 25.1 Yönlü bir çizge ve SLOW-ALL-PAIRS-SHORTEST-PATHS ile hesaplanan $L^{(m)}$ matrisler dizisi. $L^{(4)} \cdot W$ olarak tanımlanan $L^{(5)}$ in $L^{(4)}$ e eşit olduğunu doğrulamak isteyebilirsiniz. Böylece, tüm $m \geq 4$ değerleri için $L^{(m)} = L^{(4)}$ tür.

tüm $m \geq n - 1$ tamsayıları için $L^{(m)} = L^{(n-1)}$ i içerdiğini hatırlayın. Geleneksel matris çarpımının birleşme özelliği olduğu gibi EXTEND-SHORTEST-PATHS prosedürü ile tanımlanan matris çarpımı da öyledir (Alıştırma 25.1-4 e bakınız). Bu yüzden, $L^{(n-1)}$ i aşağıdaki diziyi hesaplayarak sadece $\lceil \lg(n-1) \rceil$ adet matris çarpımı ile hesaplayabiliriz.

$$\begin{aligned} L^{(1)} &= W, \\ L^{(2)} &= W^2 = W \cdot W, \\ L^{(4)} &= W^4 = W^2 \cdot W^2, \\ L^{(8)} &= W^8 = W^4 \cdot W^4, \\ &\vdots \\ L^{(2^{\lceil \lg(n-1) \rceil})} &= W^{2^{\lceil \lg(n-1) \rceil}} = W^{2^{\lceil \lg(n-1) \rceil - 1}} \cdot W^{2^{\lceil \lg(n-1) \rceil - 1}}. \end{aligned}$$

$2^{\lceil \lg(n-1) \rceil} \geq n - 1$ olduğu için son $L^{(2^{\lceil \lg(n-1) \rceil})}$ çarpımı $L^{(n-1)}$ e eşittir.

Aşağıdaki prosedür **tekrarlı kareleme (repeated squaring)** tekniğini kullanarak yukarıdaki matrisler dizisini hesaplar.



Şekil 25.2 Alıştırma 25.1-1, 25.2-1 ve 25.3-1 de kullanmak için ağırlıklı, yönlü bir çizge.

FASTER-ALL-PAIRS-SHORTEST-PATHS(W)

```

1   $n = W.rows$ 
2   $L^{(1)} = W$ 
3   $m = 1$ 
4  while  $m < n - 1$ 
5       $L^{(2m)}$   $n \times n$  lik yeni bir matris olsun
6       $L^{(2m)} = \text{EXTEND-SHORTEST-PATHS}(L^{(m)}, L^{(m)})$ 
7       $m = 2m$ 
8  return  $L^{(m)}$ 

```

4-7 satırlarındaki while döngüsün her iterasyonunda, $m = 1$ den başlayarak $L^{(2m)} = (L^{(m)})^2$ hesaplarız. Her iterasyonun sonunda, m nin değerini 2 ile çarpıyoruz. Son iterasyon bazı $n - 1 \leq 2m \leq 2n - 2$ değerleri için $L^{(2m)}$ i hesaplayarak $L^{(n-1)}$ i hesaplar. (25.3) eşitliğinden $L^{(2m)} = L^{(n-1)}$ dir. Sonra satır 4 teki test gerçekleştirilir, m 2 ile çarpılmıştır, bu yüzden $m \geq n - 1$ dir, test başarısız olur ve prosedür hesaplanan son matrisi geri döndürür.

$\lceil \lg(n - 1) \rceil$ adet matris çarpımlarının her biri $\Theta(n^3)$ zaman aldığı için FASTER-ALL-PAIRS-SHORTEST-PATHS prosedürü $\Theta(n^3 \lg n)$ zamanında çalışır. Kodun sade ve ayrıntılı veri yapıları içermediğini görebilirsiniz. Bu yüzden Θ -notasyonu içerisindeki gizli sabit küçüktür.

Alıştırmalar

25.1-1

Şekil 25.2 deki ağırlıklı, yönlü çizge üzerinde SLOW-ALL-PAIRS-SHORTEST-PATHS i döngünün her iterasyonunda elde edilen matrisleri göstererek çalıştırın. Daha sonra aynısını FASTER-ALL-PAIRS-SHORTEST-PATHS için de yapın.

25.1-2

Tüm $1 \leq i \leq n$ için neden $w_{ii} = 0$ gereklidir?

25.1-3

En kısa yollar algoritmalarında kullanılan

$$L^{(0)} = \begin{pmatrix} 0 & \infty & \infty & \cdots & \infty \\ \infty & 0 & \infty & \cdots & \infty \\ \infty & \infty & 0 & \cdots & \infty \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \infty & \infty & \infty & \cdots & 0 \end{pmatrix}$$

matrisi düzenli matris çarpımında neye karşılık gelmektedir?

25.1-4

EXTEND-SHORTEST-PATHS ile tanımlanan matris çarpımının birleşme özelliğine sahip olduğunu gösteriniz.

25.1-5

Tek-kaynaklı en kısa yollar probleminin matrislerin ve bir vektörün çarpımı olarak nasıl ifade edileceğini gösterin. Bu çarpımın Bellman-Ford benzeri bir algoritmaya nasıl karşılık geldiğini açıklayın (Kısım 24.1 e bakınız).

25.1-6

Bu bölümdeki algoritmalarda en kısa yollar üzerinde aynı zamanda tepeleri de elde etmeyi istediğimizi varsayalım. Hesaplanan L en kısa-yol ağırlıkları matrisinden Π öncül matrisinin $\Theta(n^3)$ zamanda nasıl hesaplanacağını gösterin.

25.1-7

En kısa yol ağırlıklarını hesapladığımız gibi aynı zamanda en kısa yollar üzerindeki tepeleri de hesaplayabiliriz. i den j ye en fazla m ayrıt içeren herhangi minimum-ağırlıklı bir yol üzerinde j tepesinin öncülünü $\pi_{ij}^{(m)}$ olarak tanımlayın. $L^{(1)}, L^{(2)}, \dots, L^{(n-1)}$ matrislerinin hesaplandığı gibi $\Pi^{(1)}, \Pi^{(2)}, \dots, \Pi^{(n-1)}$ matrislerini hesaplamak için EXTEND-SHORTEST-PATHS ve SLOW-ALL-PAIRS-SHORTEST-PATHS prosedürlerini modifiye edin.

25.1-8

FASTER-ALL-PAIRS-SHORTEST-PATHS prosedürü daha önce yazıldığı gibi her biri n^2 eleman içeren $\lceil \lg(n-1) \rceil$ adet matrisin saklanması için toplam $\Theta(n^2 \lg n)$ alana gerek duyar. Bu prosedürü sadece $\Theta(n^2)$ alana gerek duyması için yalnız iki adet $n \times n$ lik matris kullanarak modifiye edin.

25.1-9

FASTER-ALL-PAIRS-SHORTEST-PATHS prosedürünü çizgenin negatif-ağırlıklı çevrim içerip içermediğini tespit etmesi için modifiye edin.

25.1-10

Bir çizgede minimum-uzunluklu negatif-ağırlıklı bir çevrimin uzunluğunu (ayrıtların sayısı) bulan verimli bir algoritma verin.

25.2 Floyd-Warshall algoritması

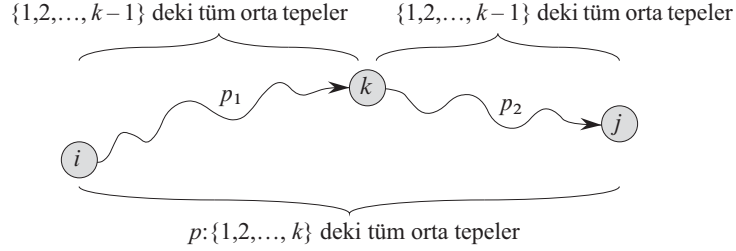
Bu bölümde, yönlü bir $G = (V, E)$ çizgesi üzerinde tüm-ikililerin en kısa-yolları problemini çözmek için farklı bir dinamik programlama formülasyonu kullanacağız. Elde edilen algoritma **Floyd-Warshall algoritması** olarak bilinir ve $\Theta(n^3)$ zamanında çalışır. Önceden olduğu gibi negatif-ağırlıklı ayrıtlar var olabilir fakat negatif-ağırlıklı çevrimlerin olmadığını kabul ediyoruz. Kısım 25.1 de olduğu gibi algoritmayı geliştirmek için dinamik-programlama sürecini izliyoruz. Elde edilen algoritmayı çalıştırdıktan sonra, yönlü bir çizgenin geçişli kapanışını bulmak için benzer bir yöntem sunacağız.

Bir en kısa yolun yapısı

Floyd-Warshall algoritmasında bir en kısa yolun yapısını Kısım 25.1 de karakterize ettiğimizden farklı bir şekilde karakterize ediyoruz. Floyd-Warshall algoritması bir en kısa yolun orta tepelerini ele alır. Basit bir $p = \langle v_1, v_2, \dots, v_l \rangle$ yolunun bir **orta** tepesi p nin v_1 ve v_l tepelerinden farklı herhangi bir tepesidir. Yani, $\{v_2, v_3, \dots, v_{l-1}\}$ kümesinden herhangi bir tepedir.

Floyd-Warshall algoritması aşağıdaki gözlemlere dayanır. G nin tepelerinin $V = \{1, 2, \dots, n\}$ olduğu varsayarak, herhangi bir k için tepelerin bir $\{1, 2, \dots, k\}$ altkümesini ele alalım. Herhangi $i, j \in V$ ikilileri için tüm orta tepeleri $\{1, 2, \dots, k\}$ kümesinden çizilen i den j ye tüm yolları ele alalım ve p bunların içinde minimum-ağırlıklı bir yol olsun. (p yolu basittir.) Floyd-Warshall algoritması p yolu ve tüm orta tepeleri $\{1, 2, \dots, k-1\}$ kümesinde olan i den j ye en kısa yollar arasındaki bir ilişkiden faydalanır. Bu ilişki k nın p yolunun orta tepesi olup olmadığına bağlıdır.

- k eğer p yolunun bir orta tepesi değil ise, p yolunun tüm orta tepeleri $\{1, 2, \dots, k-1\}$ kümesindedir. Bu yüzden, i tepesinden j tepesine tüm orta tepeleri $\{1, 2, \dots, k-1\}$ kümesinde olan bir en kısa yol aynı zamanda i den j ye tüm orta tepeleri $\{1, 2, \dots, k\}$ kümesinde olan bir en kısa yoldur.
- k eğer p yolunun bir orta tepesi ise, p yi Şekil 25.3'te gösterildiği gibi $i \xrightarrow{p_1} k \xrightarrow{p_2} j$ biçiminde parçalarız. Lemma 24.1 ten p_1 tüm orta tepeleri $\{1, 2, \dots, k\}$ kümesinde olan i den k ya bir en kısa yoldur. Aslında, biraz daha güçlü bir ifade elde edebiliriz. k tepesi p_1 yolunun bir orta tepesi olmadığı için p_1 in tüm orta tepeleri $\{1, 2, \dots, k-1\}$ kümesindedir. Bundan dolayı, p_1 tüm orta



Şekil 25.3 p yolu i tepesinden j tepesine bir en kısa yoldur ve k ise p nin en büyük numaralı orta tepesidir. p_1 yolu p yolunun i tepesinden k tepesine olum bölümüdür ve tüm orta tepeleri $\{1, 2, \dots, k-1\}$ kümesindedir. Aynısı k tepesinden j tepesine olan p_2 yolu için de geçerlidir.

tepeleri $\{1, 2, \dots, k-1\}$ kümesinde olan i den k ya bir en kısa yoldur. Benzer bir şekilde p_2 tüm orta tepeleri $\{1, 2, \dots, k-1\}$ kümesinde olan i den j ye bir en kısa yoldur.

Tüm-ikililerin en kısa-yolları problemine özyinelemeli bir çözüm

Yukarıdaki gözlemlere dayanılarak, Bölüm 25.1 dekinden farklı olarak en kısa yolun özyinelemeli bir formülasyonunu tanımlıyoruz. $d_{ij}^{(k)}$ i tepesinden j tepesine tüm orta tepeleri $\{1, 2, \dots, k\}$ kümesinde olan bir en kısa yolun ağırlığı olsun. $k=0$ iken, i tepesinden j tepesine 0 dan büyük numaralı orta tepesi olmayan bir yolun hiç orta tepesi yoktur. Böyle bir yolun en fazla bir ayrıtı vardır, bundan dolayı $d_{ij}^{(0)} = w_{ij}$ dir. Yukarıdaki tartışmayı takiben, $d_{ij}^{(k)}$ yı

$$d_{ij}^{(k)} = \begin{cases} w_{ij}, & k=1 \text{ ise,} \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}), & k \geq 1 \text{ ise} \end{cases} \quad (25.5)$$

ile özyinelemeli olarak tanımlarız.

Herhangi bir yol için tüm orta tepeler $\{1, 2, \dots, k\}$ kümesinde olduğundan dolayı $D^{(n)} = (d_{ij}^{(n)})$ matrisi şu son cevabı verir: tüm $i, j \in V$ için $d_{ij}^{(n)} = \delta(i, j)$ dir.

En kısa-yol ağırlıklarını aşağıdan yukarıya hesaplama

(25.5) özyinelemesini temel alarak, aşağıdaki aşağıdan-yukarıya prosedürünü k nin artan değerlerine göre $d_{ij}^{(k)}$ değerlerini hesaplamak için kullanabiliriz. Girdi (25.1) eşitliğinde tanımlandığı gibi $n \times n$ lik bir W matrisidir. Prosedür $D^{(n)}$ en kısa yol ağırlıkları matrisini geri döndürür.

FLOYD-WARSHALL(W)

```

1   $n = W.rows$ 
2   $D^{(0)} = W$ 
3  for  $k = 1$  to  $n$ 
4       $D^{(k)} = (d_{ij}^{(k)})_{n \times n}$  lik yeni bir matris olsun
5      for  $i = 1$  to  $n$ 
6          for  $j = 1$  to  $n$ 
7               $d_{ij}^{(k)} = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$ 
8  return  $D^{(n)}$ 

```

Şekil 25.4, Şekil 25.1 deki çizge için Floyd-Warshall algoritması ile hesaplanan $D^{(k)}$ matrisini göstermektedir.

Floyd-Warshall algoritmasının çalışma zamanı 3-7 satırlarındaki üçlü iç içe **for** döngüsü ile belirlenir. Satır 7 nin her çalışması $O(1)$ zaman aldığı için algoritma $\Theta(n^3)$ zamanda çalışmaktadır. Bölüm 25.1 deki son algoritmada olduğu gibi kod sade ve ayrıntılı veri yapıları içermez ve bu yüzden Θ -notasyonu içerisindeki gizli sabit küçüktür. Böylece, Floyd-Warshall algoritması orta-büyükölçekteki çizge girdileri için bile oldukça pratiktir.

Bir en kısa yolun oluşturulması

Floyd-Warshall algoritmasında en kısa yolları oluşturmak için çeşitli farklı yollar bulunmaktadır. Bunlardan biri, D en kısa-yol ağırlıkları matrisini hesaplamak ve daha sonra D matrisinden Π öncül matrisini oluşturmaktır. Alıştırma 25.1-6 bu yöntemin $O(n^3)$ zamanda çalışması için gerçekleştirilmesini sormaktadır. Öncül matrisi verildiğinde, PRINT-ALL-PAIRS-SHORTEST-PATH prosedürü verilen bir en kısa yol üzerindeki tepeleri yazacaktır.

Alternatif olarak, algoritma $D^{(k)}$ matrislerini hesaplarken Π öncül matrisini hesaplayabiliriz. Daha belirgin olarak, $\Pi = \Pi^{(n)}$ olduğu $\Pi^{(0)}, \Pi^{(1)}, \dots, \Pi^{(n)}$ matrisler dizisini hesaplarız ve $\pi_{ij}^{(k)}$ yı i tepesinden tüm orta tepeleri $\{1, 2, \dots, k\}$ kümesinde olan en kısa bir yol üzerinde j tepesinin öncülü olarak tanımlarız.

$\pi_{ij}^{(k)}$ nin özyinelemeli bir formülasyonunu verebiliriz. $k = 0$ iken i den j ye bir en kısa yolun hiç orta tepesi yoktur. Bu yüzden,

$$\pi_{ij}^{(0)} = \begin{cases} \text{NIL}, & i = j \text{ veya } w_{ij} = \infty \text{ ise,} \\ i, & i \neq j \text{ ve } w_{ij} < \infty \text{ ise.} \end{cases} \quad (25.6)$$

$k \geq 1$ için eğer $i \rightsquigarrow k \rightsquigarrow j$ yolunu alırsak ($k \neq j$), j nin seçtiğimiz öncülü tüm orta tepeleri $\{1, 2, \dots, k-1\}$ kümesinde olan k dan en kısa bir yol üzerinde j nin daha önce seçtiğimiz öncülü ile aynıdır. Aksi halde, i den tüm orta tepeleri

$$\begin{aligned}
D^{(0)} &= \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} & \Pi^{(0)} &= \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & \text{NIL} & 4 & \text{NIL} & \text{NIL} \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix} \\
D^{(1)} &= \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} & \Pi^{(1)} &= \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix} \\
D^{(2)} &= \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} & \Pi^{(2)} &= \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix} \\
D^{(3)} &= \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} & \Pi^{(3)} &= \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix} \\
D^{(4)} &= \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} & \Pi^{(4)} &= \begin{pmatrix} \text{NIL} & 1 & 4 & 2 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix} \\
D^{(5)} &= \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} & \Pi^{(5)} &= \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}
\end{aligned}$$

Şekil 25.4 Şekil 25.1'deki çizge için Floyd-Warshall algoritması ile hesaplanan $D^{(k)}$ ve $\Pi^{(k)}$ matrisler dizisi.

$\{1, 2, \dots, k-1\}$ kümesinde olan en kısa bir yol üzerinde daha önce seçtiğimiz j nin aynı öncülünü seçeriz. Biçimsel olarak, $k \geq 1$ için

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)}, & d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \text{ ise,} \\ \pi_{kj}^{(k-1)}, & d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \text{ ise} \end{cases} \quad (25.7)$$

dir.

$\Pi^{(k)}$ matrislerinin hesaplanması işini Alıştırma 25.2-3 olarak FLOYD-WARSHALL prosedürüne bırakıyouz. Şekil 25.4, Şekil 25.1 deki çizge için elde edilen algoritmanın hesapladığı $\Pi^{(k)}$ matrisler dizisini göstermektedir. Alıştırma aynı zamanda daha zor bir görevi; $G_{\pi,i}$ öncül altçizgesinin kökü i olan bir en kısa-yol ağacı olduğunun kanıtını istemektedir. Alıştırma 25.2-7 en kısa yolların tekrar bulunması için başka bir yol istemektedir.

Yönlü bir çizgenin geçişli kapanışı

Tepe kümesi $V = \{1, 2, \dots, n\}$ olan verilen yönlü bir $G = (V, E)$ çizgesi için tüm $i, j \in V$ tepe ikilileri için G nin i den j ye bir yol içerip içermediğini belirlemek isteyebiliriz. G nin **geçişli kapanışını** (*transitive closure*) $G^* = (V, E^*)$ olarak tanımlarız, öyle ki $E^* = \{(i, j) : G \text{ de } i \text{ tepesinden } j \text{ tepesine bir yol vardır}\}$.

$\Theta(n^3)$ zamanda bir çizgenin geçişli kapanışını hesaplayan bir yol E nin her ayrıtına 1 ağırlığını atamak ve Floyd-Warshall algoritmasını çalıştırmaktır. Eğer i tepesinden j tepesine bir yol var ise $d_{ij} < n$ elde ederiz. Aksi durumda, $d_{ij} = \infty$ elde ederiz.

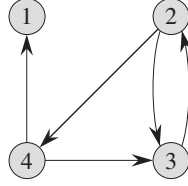
$\Theta(n^3)$ zamanda G nin geçişli kapanışını hesaplayan pratikte zaman ve alan(space) kazandırabilecek benzer bir yol daha vardır. Bu yöntem Floyd-Warshall algoritmasında min ve + aritmetik işlemleri için mantıksal $\vee$ (mantıksal VEYA) ve $\wedge$ (mantıksal VE) işlemlerini yerine koyar. $i, j, k = 1, 2, \dots, n$ için, tüm orta tepeleri $\{1, 2, \dots, k\}$ kümesinde olan i tepesinden j tepesine bir yol var ise $t_{ij}^{(k)}$ yı 1 aksi halde 0 olarak tanımlarız. $t_{ij}^{(n)} = 1$ dir ancak ve ancak $G^* = (V, E^*)$ geçişli kapanışını (i, j) ayrıtını E^* içine ekleyerek oluştururuz. $t_{ij}^{(k)}$ nin özyinelemeli bir tanımı (25.5) özyinelemesine benzer bir şekilde

$$t_{ij}^{(0)} = \begin{cases} 0, & i \neq j \text{ ve } (i, j) \notin E \text{ ise,} \\ 1, & i = j \text{ ise } (i, j) \in E \text{ ise} \end{cases}$$

ve $k \geq 1$ için

$$t_{ij}^{(k)} = t_{ij}^{(k-1)} \vee (t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)}) \text{ dir.} \quad (25.8)$$

Floyd-Warshall algoritmasında olduğu gibi $T^{(k)} = (t_{ij}^{(k)})$ matrislerini k nın artan değerlerine göre hesaplarız.



$$T^{(0)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} \quad T^{(1)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} \quad T^{(2)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix}$$

$$T^{(3)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \quad T^{(4)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

Şekil 25.5 Yönlü bir çizge ve geçişli kapanış algoritması ile hesaplanan $T^{(k)}$ matrisleri.

TRANSITIVE-CLOSURE(G)

```

1   $n = |G.V|$ 
2   $T^{(0)} = (t_{ij}^{(0)})$   $n \times n$  lik yeni bir matris olsun
3  for  $i = 1$  to  $n$ 
4      for  $j = 1$  to  $n$ 
5          if  $i == j$  or  $(i, j) \in G.E$ 
6               $t_{ij}^{(0)} = 1$ 
7          else  $t_{ij}^{(0)} = 0$ 
8  for  $k = 1$  to  $n$ 
9       $T^{(k)} = (t_{ij}^{(k)})$   $n \times n$  lik yeni bir matris olsun
10     for  $i = 1$  to  $n$ 
11         for  $j = 1$  to  $n$ 
12              $t_{ij}^{(k)} = t_{ij}^{(k-1)} \vee (t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)})$ 
13 return  $T^{(n)}$ 

```

Şekil 25.5 örnek bir çizge üzerinde TRANSITIVE-CLOSURE prosedürü ile hesaplanan $T^{(k)}$ matrislerini göstermektedir. TRANSITIVE-CLOSURE prosedürü Floyd-Warshall algoritması gibi $\Theta(n^3)$ zamanda çalışır. Bazı bilgisayarlarda tek-bit değerleri üzerindeki mantıksal işlemler tam sayı kelimeleri üzerindeki aritmetik işlemlerinden daha hızlı çalışır. Bunun ötesinde, geçişli-kapanış algoritması tam sayı değerlerinden ziyade sadece boole değerlerini kullandığı için algoritma-

nın bellek gereksinimi Floyd-Warshall algoritmasının gereksiniminden bilgisayar belleğinde bir kelimenin büyüklüğüne karşılık gelen bir çarpan kadar daha azdır.

Alıştırmalar

25.2-1

Şekil 25.2 deki ağırlıklı, yönlü çizge üzerinde Floyd-Warshall algoritmasını çalıştırın. Dış döngünün her iterasyonunda elde edilen $D^{(k)}$ matrisini gösterin.

25.2-2

Kısım 25.1 deki tekniği kullanarak geçişli kapanışın nasıl hesaplanacağını gösterin.

25.5-3

Floyd-Warshall prosedürünü (25.6) ve (25.7) eşitliklerine göre $\Pi^{(k)}$ matrislerini hesaplamak için modifiye edin. Tüm $i \in V$ için $G_{\pi,i}$ öncül altçizgesinin kökü i olan bir en kısa-yollar ağacı olduğunu dikkatli bir şekilde kanıtlayınız. (*İpucu:* $G_{\pi,i}$ nin çevrimsiz olduğunu göstermek için önce $\pi_{ij}^{(k)} = l$ eşitliğinin $\pi_{ij}^{(k)}$ nin tanımına göre $d_{ij}^{(k)} \geq d_{il}^{(k)} + w_{lj}$ eşitsizliğini beraberinde getirdiğini gösterin. Daha sonra Lemma 24.16 nın kanıtını uyarlayın.

25.2-4

Yukarıda görüldüğü üzere, $i, j, k = 1, 2, \dots, n$ için $d_{ij}^{(k)}$ hesapladığımızdan dolayı Floyd-Warshall algoritması $\Theta(n^3)$ alana gerek duymaktadır. Sadece tüm üst-indisleri iptal eden aşağıdaki prosedürün doğru olduğunu ve bu sayede sadece $\Theta(n^2)$ alana gerek duyulduğunu gösterin.

FLOYD-WARSHALL'(W)

```

1   $n = W.rows$ 
2   $D = W$ 
3  for  $k = 1$  to  $n$ 
4      for  $i = 1$  to  $n$ 
5          for  $j = 1$  to  $n$ 
6               $d_{ij} = \min(d_{ij}, d_{ik} + d_{kj})$ 
7  return  $D$ 
```

25.2-5

(25.7) eşitliğindeki yöntemi değiştirdiğimizi varsayalım:

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)}, & d_{ij}^{(k-1)} < d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \text{ ise,} \\ \pi_{kj}^{(k-1)}, & d_{ij}^{(k-1)} \geq d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \text{ ise.} \end{cases}$$

Π öncül matrisinin bu alternatif tanımı doğru mudur?

25.2-6

Negatif-ağırlıklı bir çevrimin varlığını tespit etmek için Floyd-Warshall algoritmasının çıktısını nasıl kullanabiliriz?

25.2-7

Floyd-Warshall algoritmasında en kısa yolları yeniden yapılandıran bir diğer yol $i, j, k = 1, 2, \dots, n$ için $\phi_{ij}^{(k)}$ değerlerini kullanır. $\phi_{ij}^{(k)}$ tüm orta tepeleri $\{1, 2, \dots, k\}$ kümesinde olan i den j ye bir en kısa yolun en büyük-numaralı orta tepesidir. $\phi_{ij}^{(k)}$ için özyinelemeli bir formülasyon verin, FLOYD-WARSHALL prosedürünü $\phi_{ij}^{(k)}$ değerlerini hesaplamak için modifiye edin ve PRINT-ALLPAIRS-SHORTEST-PATH prosedürünü $\Phi = (\phi_{ij}^{(n)})$ matrisini girdi olarak alması için yeniden yazın. Φ matrisi Kısım 15.2 deki matris-zinciri çarpım problemindeki s tablosuna ne kadar benzemektedir?

25.2-8

Yönlü bir $G = (V, E)$ çizgesinin geçişli kapanışını hesaplamak için $O(V E)$ zamanlı bir algoritma verin.

25.2-9

Yönlü çevrimsiz bir çizgenin geçişli kapanışını $f(|V|, |E|)$ zamanda hesaplayabildiğimizi varsayalım. f fonksiyonu $|V|$ ve $|E|$ 'ye bağlı monoton artan bir fonksiyondur. Yönlü, genel bir $G = (V, E)$ çizgesinin $G^* = (V, E^*)$ geçişli kapanışını hesaplamak için $f(|V|, |E|) + O(V + E^*)$ zaman gerektiğini gösterin.

25.3 Seyrek çizgeler için Johnson algoritması

Johnson algoritması tüm ikililer arasındaki en kısa yolları $O(V^2 \lg V + VE)$ zamanda bulur. Seyrek çizgeler için ya matrislerin tekrarlı kare alınmasından ya da Floyd-Warshall algoritmasından asimptotik olarak daha hızlıdır. Algoritma ya tüm tepelerin ikilileri için en kısa-yol ağırlıklarının matrisini döndürür ya da girdi çizgesinin negatif-ağırlıklı bir çevrim içerdiğini raporlar. Johnson algoritması Bölüm 24 te açıklanan Bellman-Ford algoritmasını ve Dijkstra algoritmasının her ikisini de altprogramlar olarak kullanır.

Johnson algoritması aşağıdaki şekilde çalışan **yeniden ağırlıklandırma (reweighting)** tekniğini kullanır. Eğer G çizgesinde w ayrıt ağırlıklarının hiçbiri negatif değil ise her tepeden bir kez Dijkstra algoritmasını çalıştırarak tüm tepelerin ikilileri arasında en kısa yolları bulabiliriz; Fibonacci-bellek yığını min-öncelikli kuyruk ile bu tüm-ikililerin algoritması $O(V^2 \lg V + VE)$ zamanda çalışır. Eğer G negatif-ağırlıklı ayrıtlar içeriyor ancak negatif-ağırlıklı çevrimler içermiyor ise, basitçe negatif olmayan ayrıt ağırlıklarının yeni bir kümesini hesaplarız, ki bu

bize aynı metodu kullanmamıza izin verir. Ayırt ağırlıklarının yeni kümesi $\hat{w}$ iki önemli özelliği sağlamalıdır:

1. Tüm $u, v \in V$ tepe ikilileri için bir p yolu w ağırlık fonksiyonunu kullanarak u dan v ye bir en kısa yoldur ancak ve ancak p aynı zamanda $\hat{w}$ ağırlık fonksiyonunu kullanarak u dan v ye bir en kısa yoldur.
2. Tüm (u, v) ayırtları için yeni $\hat{w}(u, v)$ ağırlığı negatif değildir.

Biraz sonra göreceğimiz üzere yeni $\hat{w}$ ağırlık fonksiyonu belirlemek için G yi $O(VE)$ zamanda önışleyebiliriz.

Yeniden ağırlıklandırarak en kısa yolları koruma

Aşağıdaki lemma yukarıdaki birinci özelliği sağlamak için ayırtları kolayca nasıl yeniden ağırlıklandırabildiğimizi göstermektedir. w ağırlık fonksiyonundan türetilen en kısa-yol ağırlıklarını belirlemek için delta, $\hat{w}$ ağırlık fonksiyonundan türetilen en kısa-yol ağırlıklarını belirlemek için $\hat{\delta}$ kullanırız.

Lemma 25.1 (Yeniden ağırlıklandırma en kısa yolları değiştirmez)

$w : E \rightarrow \mathbb{R}$ ağırlık fonksiyonlu, yönlü, ağırlıklı bir $G = (V, E)$ çizgesi verilsin, $h : V \rightarrow \mathbb{R}$ tepeleri gerçel sayılara eşleyen herhangi bir fonksiyon olsun. Her $(u, v) \in E$ ayırtı için

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v) \quad (25.9)$$

tanımlansın. $p = \langle v_0, v_1, \dots, v_k \rangle$, v_0 tepesinden v_k tepesine herhangi bir yol olsun. O zaman p yolu v_0 dan v_k ye w ağırlık fonksiyonlu bir en kısa yoldur ancak ve ancak p yolu $\hat{w}$ ağırlık fonksiyonlu bir en kısa yoldur. Yani, $w(p) = \delta(v_0, v_k)$ ancak ve ancak $\hat{w}(p) = \hat{\delta}(v_0, v_k)$. dir. Bunun da ötesinde, G çizgesi $\hat{w}$ ağırlık fonksiyonunu kullanarak negatif-ağırlıklı bir çevrim içeriyordur ancak ve ancak, G çizgesi w ağırlık fonksiyonunu kullanarak negatif-ağırlıklı bir çevrim içeriyordur.

$$\textbf{Kanıt} \quad \hat{w}(p) = w(p) + h(v_0) - h(v_k) \quad (25.10)$$

eşitliğini göstererek başlıyoruz.

$$\begin{aligned} \hat{w}(p) &= \sum_{i=1}^k \hat{w}(v_{i-1}, v_i) \\ &= \sum_{i=1}^k (w(v_{i-1}, v_i) + h(v_{i-1}) - h(v_i)) \\ &= \sum_{i=1}^k w(v_{i-1}, v_i) + h(v_0) - h(v_k) \quad (\text{çünkü toplam iç içe geçer}) \\ &= w(p) + h(v_0) - h(v_k) . \end{aligned}$$

Bu yüzden, v_0 dan v_k ya her hangi bir p yolunda $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$ dir. $h(v_0)$ ve $h(v_k)$ yola bağlı olmadığı için eğer v_0 dan v_k ya bir yol w ağırlık fonksiyonunu kullanarak diğerinden daha kısa ise aynı zamanda $\hat{w}$ yi kullanarak ta daha kısadır. Böylece, $w(p) = \delta(v_0, v_k)$ ancak ve ancak $\hat{w}(p) = \hat{\delta}(v_0, v_k)$ dir.

Son olarak, G çizgesi $\hat{w}$ ağırlık fonksiyonunu kullanarak negatif-ağırlıklı bir çevrim içeriyorsa ancak ve ancak G çizgesinin w ağırlık fonksiyonunu kullanarak negatif-ağırlıklı bir çevrim içerdiğini gösterelim. Herhangi bir $c = \langle v_0, v_1, \dots, v_k \rangle$ çevrimini ele alalım, yani $v_0 = v_k$. (25.10) eşitliğinden

$$\begin{aligned}\hat{w}(c) &= w(c) + h(v_0) - h(v_k) \\ &= w(c)\end{aligned}$$

olur ve bu yüzden c , ancak ve ancak w yi kullanarak negatif ağırlık içerir ancak ve ancak $\hat{w}$ yi kullanarak negatif ağırlık içerir. ■

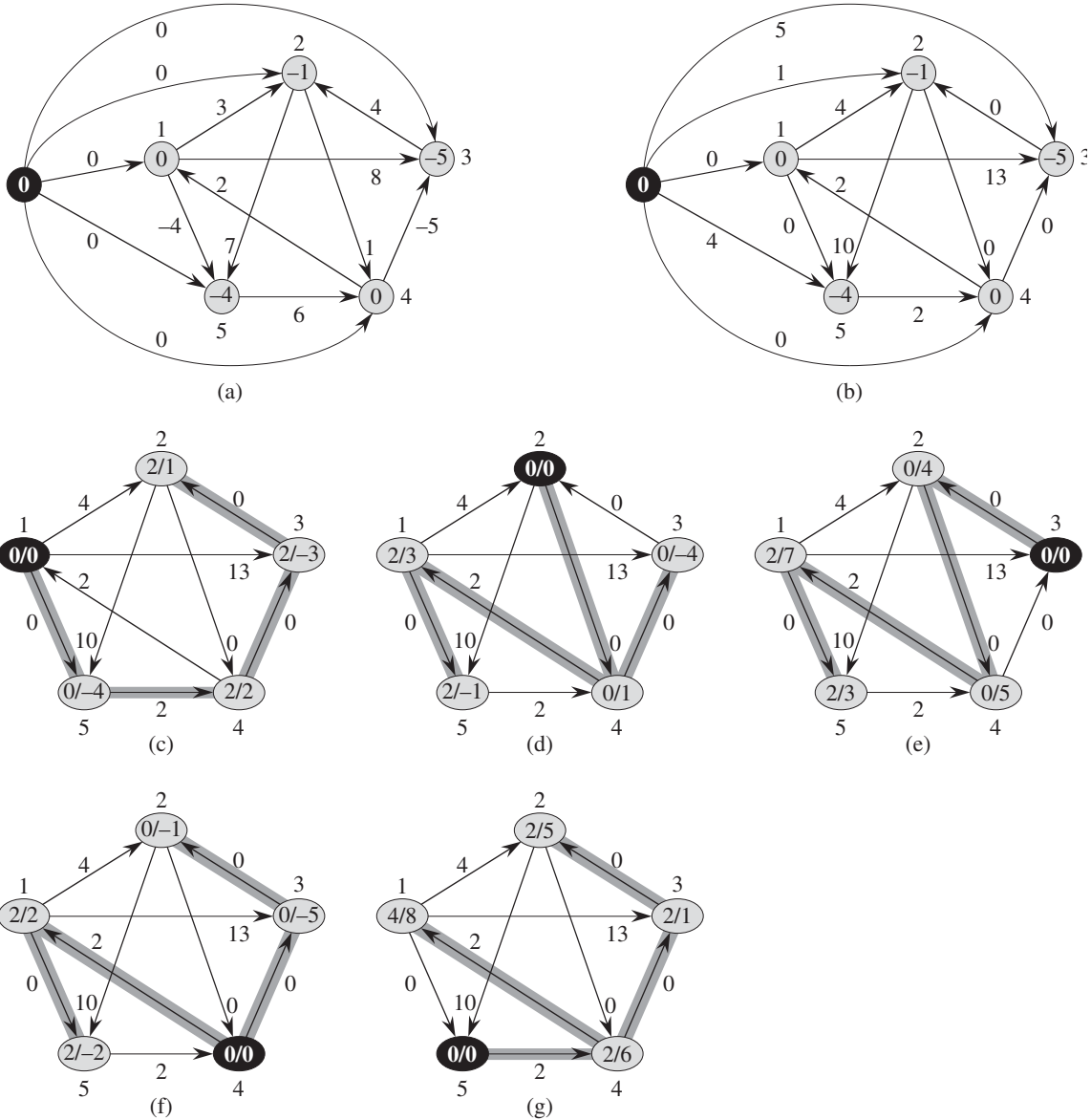
Yeniden ağırlıklandırarak negatif olmayan ağırlıklar üretmek

Bir sonraki amacımız ikinci özelliğin sağlandığından emin olmaktır: Tüm $(u, v) \in E$ ayrıtları için $\hat{w}(u, v)$ nin negatif olmamasını istiyoruz. $w : E \rightarrow \mathbb{R}$ ağırlık fonksiyonlu, yönlü, ağırlıklı bir $G = (V, E)$ çizgesi verilsin. Yeni bir $G' = (V', E')$ çizgesi yapıyoruz, öyle ki bazı yeni $s \notin V$ tepeleri için $V' = V \cup \{s\}$ ve $E' = E \cup \{(s, v) : v \in V\}$ dir. Tüm $v \in V$ için $w(s, v) = 0$ olsun diye w ağırlık fonksiyonunu genişletiyoruz. s ye gelen ayrıtlar olmadığı için G' da s kaynaklı yollar dışında s yi içeren en kısa yollar yoktur. Ayrıca, G negatif-ağırlıklı çevrimlere sahip değildir ancak ve ancak G' negatif-ağırlıklı çevrimlere sahip değildir. Şekil 25.6(a) Şekil 25.1 deki G çizgesine karşılık gelen G' çizgesini göstermektedir.

Şimdi G ve G' çizgelerinin negatif-ağırlıklı çevrimler içermediğini varsayalım. Tüm $v \in V'$ için $h(v) = \delta(s, v)$ eşitliğini tanımlayalım. Üçgen eşitsizliğinden (Lemma 24.10) tüm $(u, v) \in E'$ ayrıtları için $h(v) \leq h(u) + w(u, v)$ dir. Bu yüzden, (25.9) daki eşitliğe göre yeniden ağırlıklandırma yaparak yeni $\hat{w}$ ağırlıklarını tanımlarsak $\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$ elde ederiz ve ikinci özelliği sağladık. Şekil 25.6(b) Şekil 25.6(a) daki G' grafinin yeniden ağırlıklandırılmış halini göstermektedir.

Tüm-ikililerin en kısa yollarını hesaplama

Johnson'un algoritması tüm-ikililerin en kısa yollarını hesaplamak için Bellman-Ford algoritmasını (Kısım 24.1) ve Dijkstra'nın algoritmasını (Kısım 24.3) altprogramlar olarak kullanır. Açık bir şekilde ayrıtların komşuluk listelerinde tutulduğunu kabul eder. Algoritma $|V| \times |V|$ lik $D = d_{ij}$ matrisini $d_{ij} = \delta(i, j)$ döndürür veya girdi çizginin negatif-ağırlıklı bir çevrim içerdiğini raporlar. Bir tüm-ikililerin en kısa yolları algoritması için tipik olarak teperin 1 den $|V|$ ye kadar numarandırıldığını kabul ediyoruz.



Şekil 25.6 Johnson'nın tüm ikililerin en kısa yolları algoritması Şekil 25.1'deki çizge üzerinde çalışır. (a) Orijinal w ağırlık fonksiyonlu G' çizgesi. Yeni s tepesi siyahtır. Her v tepesi içerisinde $h(v) = \delta(s, v)$ dir. (b) Her (u, v) ayrıtını $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ ağırlık fonksiyonu ile yeniden ağırlıklandırıldıktan sonra. (c) (g) G nin her tepesi üzerinde $\hat{w}$ ağırlık fonksiyonunu kullanarak Dijkstra'nın algoritmasının çalışmasının sonucu. Her bölümde u kaynak tepesi siyahtır ve taranmış ayrıtlar algoritma tarafından hesaplanan en kısa yollar ağacı içindedir. Her v tepesi içerisinde $\hat{\delta}(u, v)$ ve $\hat{\delta}(u, v)$ değerleri bölü işareti ile ayrılarak belirtilmiştir. $d_{uv} = \delta(u, v)$ değeri $\hat{\delta}(u, v) + h(v) - h(u)$ ye eşittir.

JOHNSON (G, w)

```

1   $G'$  hesapla, öyle ki  $G'.E = G.E \cup \{(s, v) : v \in G.V\}$ 
    $G'.V = G.V \cup \{s\}$  ve tüm  $v \in G.V$  ler için  $w(s, v) = 0$ 
2  if BELLMAN-FORD( $G', w, s$ ) == FALSE
3      print “girdi çizgesi negatif-ağırlıklı bir çevrim içermektedir”
4  else for her  $v \in G'.V$  tepesi için
5       $h(v)$  yi Bellman-Ford algoritması ile
       hesaplanan  $\hat{\delta}(u, v)$  değerine ata
6      for her  $(u, v) \in G'.E$  ayrıtı için
7           $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$ 
8       $D = (d_{uv}) n \times n$  lik yeni bir matris olsun
9      for her  $u \in G.V$  tepesi için
10         tüm  $v \in G.V$  ler için  $\hat{\delta}(u, v)$  yi hesaplamak için DIJKSTRA( $G, \hat{w}, u$ )
           çalıştır
11         for her  $v \in G.V$  tepesi için
12              $d_{uv} = \hat{\delta}(u, v) + h(v) - h(u)$ 
13     return  $D$ 

```

Bu kod basit bir şekilde daha önce belirttiğimiz işleri gerçekleştirir. Satır 1 G' çizgesini üretir. Satır 2 w ağırlık fonksiyonlu ve s kaynak tepeli G' üzerinde Bellman-Ford algoritmasını çalıştırır. Eğer G' ve bundan dolayı G negatif-ağırlıklı bir çevrim içeriyorsa, satır 3 bu problemi bildirir. 4-12 satırları G' çizgesinin negatif-ağırlıklı bir çevrim içermediğini kabul eder. 4-5 satırları her $v \in V'$ ler için $h(v)$ yi Bellman-Ford algoritması ile hesaplanan en kısa-yol ağırlığı $\hat{\delta}(u, v)$ ya eşitler. 6-7 satırları yeni $\hat{w}$ ağırlıklarını hesaplar. Her $u, v \in G.V$ tepe ikilileri için, 9-12 satırlarının **for** döngüsü V' deki her tepeden bir kez Dijkstra'nın algoritmasını çağırarak en kısa-yol ağırlığı olan $\hat{\delta}(u, v)$ yi hesaplar. Satır 12 (25.10) eşitliği kullanılarak hesaplanan gerçek en kısa-yol ağırlığı $\delta(u, v)$ yi d_{uv} matris elemanında saklar. Son olarak, satır 13 tamamlanan D matrisini döndürür. Şekil 25.6 Johnson'un algoritmasının çalışmasını resmetmektedir.

Eğer Dijkstra'nın algoritmasında min-öncelikli kuyruğu Fibonacci bellek yığını ile gerçekleştirsek, Johnson'un algoritması $O(V^2 \lg V + VE)$ zamanda çalışır. Daha basit olan min-bellek yığını uygulaması ise eğer çizge gevşekse Floyd-Warshall algoritmasından hala asimptotik olarak daha hızlı $O(VE \lg V)$ çalışma zamanını verir.

Alıştırmalar

25.3-1

Şekil 25.2 deki çizgenin tüm tepe ikilileri arasındaki en kısa yolları bulmak için Johnson'un algoritmasını kullanın. Algoritma ile hesaplanan h ve $\hat{w}$ nin değerlerini gösterin.

25.3-2

Yeni s tepesini V ye eklemenin (bu V' tepe kümesini verir) amacı nedir?

25.3-3

Tüm $(u, v) \in E$ ayrıtları için $w(u, v) \geq 0$ olduğunu kabul edin. w ve $\hat{w}$ ağırlık fonksiyonları arasındaki ilişki nedir?

25.3-4

Profesör Greenstreet ayrıtları yeniden ağırlıklandırmak için Johnson'un algoritmasındaki yöntemden daha basit bir yol olduğunu iddia etmektedir. $w^* = \min_{(u,v) \in E} \{w(u, v)\}$ yaparak tüm $(u, v) \in E$ ayrıtları için $\hat{w}(u, v) = w(u, v) - w^*$ tanımlayın. Profesörün yeniden ağırlıklandırma yöntemindeki yanlış nedir?

25.3-5

w ağırlık fonksiyonlu yönlü bir G çizgesi üzerinde Johnson'un algoritmasını çalıştırdığımızı kabul edin. Eğer G , 0-ağırlıklı bir c çevrimi içeriyorsa c deki her (u, v) ayrıtı için $\hat{w}(u, v) = 0$ olduğunu gösterin.

25.3-6

Profesör Michener JOHNSON'un 1. satırındaki yeni bir kaynak tepesi oluşturmaya gerek olmadığını iddia etmektedir. Bunun yerinde sadece $G' = G$ yi kullanabileceğimizi ve s nin herhangi bir tepe olabileceğini iddia etmektedir. Profesörün fikrini JOHNSON içine dahil edip yanlış cevaplara neden olan ağırlıklı, yönlü bir G çizgesi örneği verin. Daha sonra, eğer G güçlü bağlantılı (her tepe diğer her tepeden ulaşılabilir) ise profesörün değişiklikleri ile birlikte JOHNSON tarafından geriye dönen sonuçların doğru olduğunu gösterin.

Problemler
25-1 Dinamik bir çizgenin geçişli kapanışı

E ye ayrıtları eklediğimiz gibi yönlü bir $G = (V, E)$ çizgesinin geçişli kapanışını devam ettirmek istediğimizi varsayın. Yani, her ayrıt eklendikten sonra o ana kadar eklenen ayrıtların geçişli kapanışını güncellemek istiyoruz. G çizgesinin başlangıçta ayrıtları olmadığını ve geçişli kapanışı boole matrisi ile gösterdiğimizizi kabul edelim.

- a.** Bir $G = (V, E)$ çizgesinin $G^* = (V, E^*)$ geçişli kapanışını G ye yeni bir ayrıt eklendiğinde $O(V^2)$ zamanda nasıl güncelleneceğini gösterin.
- b.** Bir G çizgesi ve e ayrıtı örneği verin öyle ki e yi G ye ekledikten sonra geçişli kapanışı güncellemek için gereken zaman $\Omega(V^2)$ olsun. Hangi algoritmanın kullanıldığı önemli değildir.

- c. Ayrıtlar çizgeye eklenirken geçişli kapanışı güncellemek için verimli bir algoritma açıklayın. n ekleme için algoritmanız toplamda $\sum_{i=1}^n t_i = O(V^3)$ zamanda çalışmalıdır, t_i i . ayrıt eklendiğinde geçişli kapanışı güncelleme zamanıdır. Algoritmanızın bu zaman sınırını aldığını kanıtlayınız.

25-2 ϵ -yoğun çizgelerde en kısa yollar

Bir $G = (V, E)$ çizgesi eğer $0 < \epsilon \leq 1$ aralığındaki bazı sabit ϵ için $|E| = \Theta(V^{1+\epsilon})$ ise **ϵ -yoğundur**. ϵ -yoğun çizgeler üzerinde en kısa-yollar algoritmalarında d -li min-bellek yığınları (Problem 6-2 ye bakınız) kullanılarak, karmaşık bir veri yapısı kullanmadan Fibonacci bellek yığını tabanlı algoritmalarının çalışma zamanlarını yakalayabiliriz.

- a. INSERT, EXTRACT-MIN ve DECREASE-KEY için bir d -li min-bellek yığınındaki elemanların sayısı n ve d cinsinden bir fonksiyon olarak çalışma zamanları nelerdir? Eğer bazı $0 < \alpha \leq 1$ sabiti için $d = \Theta(n^\alpha)$ seçersek bu çalışma zamanları ne olur? Bir Fibonacci bellek yığını için bu işlemlerin çalışma zamanları ile amortize maliyetlerini karşılaştırın.
- b. ϵ -yoğun yönlü, negatif-ağırlıklı ayrıtlar içermeyen bir $G = (V, E)$ çizgesi üzerinde tek bir kaynaktan en kısa yolları $O(E)$ zamanda nasıl hesaplandığını gösterin (*İpucu*: d yi ϵ cinsinden alın).
- c. ϵ -yoğun yönlü, negatif-ağırlıklı ayrıtlar içermeyen bir $G = (V, E)$ çizgesi üzerinde tüm-ikililerin en kısa yolları problemini $O(VE)$ zamanda nasıl çözüldüğünü gösterin.
- d. ϵ -yoğun yönlü, negatif-ağırlıklı ayrıtlar içerebilen ancak negatif-ağırlıklı çevrimler içermeyen bir $G = (V, E)$ çizgesi üzerinde tüm-ikililerin en kısa yolları problemini $O(VE)$ zamanda nasıl çözüldüğünü gösterin.

Bölüm notları

Lawler [224] seyrek çizgeler için çözümleri analiz etmemesine rağmen tüm-ikililerin en kısa yolları probleminin iyi bir tartışmasına sahiptir. Matris çarpım algoritmasını halkbilimine bağlamaktadır. Floyd-Warshall algoritması boole matrislerin kapalı geçişlerinin nasıl hesaplanacağını açıklayan Warshall'ın bir teoremine [349] bağlı kalan Floyd'a [105] dayanmaktadır. Johnson'un algoritması [192] den alınmıştır.

Birçok araştırmacı en kısa yolları hesaplamak için matris çarpımı aracılığıyla algoritmalar geliştirmiştir. Fredman [111] tüm-ikililerin en kısa yolları probleminin nasıl çözüleceğini ayrıt ağırlıklarının toplamaları arasında $O(V^{5/2})$ karşılaştırma

kullanarak göstermiştir ve $O(V^3(\lg \lg V/\lg V)^{1/3})$ zamanda çalışan bir algoritma elde etmiştir ki Floyd-Warshall algoritmasının çalışma zamanından biraz daha iyidir. Han [159] çalışma zamanını $O(V^3(\lg \lg V/\lg V)^{5/4})$ e indirmiştir. Bir diğer araştırma hızlı matris çarpımı algoritmalarını (Bölüm 4 deki bölüm notlarına bakınız) tüm-ikililerin en kısa yolları problemine uygulayabileceğimizi göstermektedir. $O(n^\omega)$ $n \times n$ matrislerini çarpan en hızlı algoritmanın çalışma zamanı olsun: şu anki $\omega < 2.376$ [78]. Galil ve Margalit [123, 124] ve Seidel [308] tüm-ikililerin en kısa yolları problemini yönsüz, ağırlıksız çizgelerde ($V^\omega p(V)$) zamanda çözen algoritmalar tasarlamışlardır, $p(n)$ polilogaritmik sınırlı olan belirli bir fonksiyonu temsil eder. Yoğun çizgelerde, bu algoritmalar $|V|$ defa genişlik-öncelikli aramaları (BFS) gerçekleştirmek için gerekli olan $O(VE)$ zamanından daha hızlıdır. Birçok araştırmacı tüm-ikililerin en kısa yolları problemini ayırt ağırlıkları $\{1, 2, \dots, W\}$ aralığında tam sayılar olan yönsüz çizgelerde çözen algoritmalar vermesi için bu sonuçları genişlettiler. Shoshan ve Zwick [316] tarafından verilen asimptotik olarak en hızlı böyle bir algoritma $O(WV^\omega p(VW))$ zamanda çalışır.

Karger, Koller ve Phillips [196] ve bağımsız olarak McGeoch [247] bazı en kısa yollarda iştirak eden E nin ayırtlar kümesi olan E^* a bağlı olan bir zaman sınırı vermişlerdir. Negatif olmayan ayırt ağırlıklı bir çizge verilsin, onların algoritmaları $O(VE^* + V^2 \lg V)$ zamanda çalışır ve $|E^*| = o(E)$ olduğunda Dijkstra'nın algoritmasını $|V|$ kat kadar iyileştirir.

Baswana, Hariharan ve Sen [33] tüm-ikililerin en kısa yollarını ve geçişli kapanış bilgisini yürütmek için azalan (decremental) algoritmaları incelemişlerdir. Azalan algoritmalar bir dizi ayırt silme ve sorgu karışımlarına izin verir; kıyas yaparak, ayırtların eklendiği Problem 25-1 azalan bir algoritma istemektedir. Baswana, Hariharan ve Sen tarafından önerilen algoritmalar rassaldır ve bir yol var olduğu zaman onların geçişli-kapanış algoritmaları gelişigüzel bir $c > 0$ için $1/n^c$ olasılıkla bunu rapor edemeyebilirler. Sorgu zamanları çok yüksek ihtimal ile $O(1)$ dir. Geçişli kapanış için her güncellemenin amortize zamanı $O(V^{4/3} \lg^{1/3} V)$ dir. En kısa-yol ağırlıklarını veren sorgular için güncelleme başına amortize zamanı $O(V^3/E \lg^2 V)$ dir. Gerçek en kısa yolu bildirmek için amortize güncelleme zamanı $\min(O(V^{3/2} \sqrt{\lg V}), O(V^3/E \lg^2 V))$ dir. Demetrescu ve Italiano [84] verilen her ayırt gerçel sayıların mümkün değerlerinin sınırlı bir aralığına sahip olduğu sürece ayırtlar eklendiği ve silindiği zaman güncelleme ve sorgu işlemlerinin nasıl ele alınacağını göstermişlerdir.

Aho, Hopcroft ve Ullman [5] yönlü çizgelerde yol problemlerini çözmek için genel bir çerçeve olarak hizmet veren “çizgelerde” olarak bilinen cebirsel bir yapıyı tanımladılar. Kısım 25.2 deki Floyd-Warshall algoritması ve geçişli-kapanış algoritmaları bir tüm-ikililer algoritmasının kapalı yarı halka tabanlı örneklemeleleridir. Maggs ve Plotkin [240] bir kapalı yarı halka kullanarak minimum kapsayan ağaçlarının nasıl bulunacağını göstermişlerdir.

Bir noktadan başka bir noktaya olan en kısa yolu bulmak için yönlü bir çizge olarak bir yol haritasını modelleyebileceğimiz gibi, ayrıca bir yönlü çizgeyi “akış ağı” (flow network) olarak da yorumlayabiliriz ve bunu malzeme akışlarıyla ilgili soruları cevaplamak için kullanabiliriz. Malzemenin üretildiği bir kaynaktan, tüketildiği bir çıkış düğümü sistemi aracılığıyla akan bir malzeme olduğunu farz edelim. Kaynak, malzemeyi belirli bir sabit oranda üretir ve çıkış düğümü, malzemeyi aynı oranda tüketir. Sistemde her noktadaki malzeme “akışı” sezgisel olarak malzemenin hareket ettiği orandadır. Akış ağı, borulardan akan akışkanları, montaj hattı boyunca parçaları, elektrik ağından geçen akımı ve iletişim ağından akan bilgiyi de içeren birçok problemi modelleyebilir.

Akış ağı içerisindeki her yönlü ayrıtı malzeme için bir iletim hattı olarak düşünebiliriz. Her iletim hattı, malzemenin hat boyunca akabildiği maksimum oran olarak verilmiş belirli bir kapasiteye sahiptir, örneğin bir borudan geçen saatte 200 galon akışkan ya da bir kablodan geçen 20 amper elektrik akımı gibi. Tepe noktaları iletim hattının birleşme yerleridir ayrıca kaynak ile çıkış düğümü haricinde, malzeme tepe noktalarında birikmeksizin doğrudan akar. Başka bir ifadeyle, malzemenin tepe noktasına girdiğindeki oran, tepe noktasının çıkışındakine eşit olmalıdır. Bu özelliği “akış korunumu” olarak adlandırırız ve malzeme elektrik akımı iken bu Kirchhoff’un akım yasasına eşittir.

Maksimum-akış probleminde, kapasite kısıtlarını bozmaksızın kaynaktan çıkış düğümüne gönderebildiğimiz en büyük oranı hesaplamayı isteriz. Bu akış ağıyla ilgili en basit problemlerden biridir ve bu bölümde göreceğimiz gibi, bu problem etkili algoritmalarla çözülebilir. Üstelik diğer akış-ağı problemlerini çözmek için, maksimum-akış algoritmalarında kullanılan temel teknikleri uyarlayabiliriz.

Bu bölüm maksimum-akış problemlerini çözmeye yarayan iki genel yöntemi sunar. Kısım 26.1 akış ağları ve akışlar kavramlarını şekillendirir, maksimum-akış problemini resmi olarak tanımlar. Kısım 26.2 maksimum akışları bulmaya yarayan Ford ve Fulkerson’nın klasik yöntemini tanımlar. Bu yöntemin bir uygulaması, yönsüz iki parçalı (bipartite) çizgede maksimum bir eşleme bulmak,

Kısım 26.3 de görülecektir. Kısım 26.4, akış-ağı problemleri için en hızlı algoritmaların bir çoğunun altında yatan “itme-yeniden etiketleme” yöntemini sunar. Kısım 26.5, $O(V^3)$ zamanda yürütülen itme-yeniden etiketleme yönteminin özel bir gerçekleştirimi olan “öne-yeniden etiketleme” (relabel-to-front) algoritmasını kapsar. Bu algoritmanın bilinen en hızlı algoritma olmamasına rağmen, asimptotik olarak hızlı algoritmalarda kullanılan bazı teknikleri ortaya koyar ve bu uygulamalarda oldukça etkilidir.

26.1 Akış ağları

Bu bölümde akış ağının çizge-teorik tanımını veriyor, özelliklerini tartışıyor ve maksimum-akış problemini tam olarak tanımlıyoruz. Ayrıca bazı faydalı gösterimleri de tanıtıyoruz.

Akış ağları ve akışlar

Bir $G = (V, E)$ **akış ağı (flow network)** her $(u, v) \in E$ ayrıtı negatif olmayan $c(u, v) \geq 0$ **kapasitesine** sahip yönlü bir çizgedir. Eğer E , bir (u, v) ayrıtı içeriyorsa ters yönde (v, u) ayrıtı yoktur (Bu kısıtlamalar üzerine nasıl çalışıldığını kısaca göreceğiz). Eğer $(u, v) \notin E$ ise, bu durumda kolaylık sağlaması için $c(u, v) = 0$ tanımlarız ve kendi kendine döngüleri (self-loops) geçersiz sayarız. Akış ağında iki tepeyi ayırt ederiz: bir **kaynak (source)** s ve bir **çıkış düğümü (sink)** t . Kolaylık olması için, her tepe noktasının kaynaktan çıkış düğümüne giden güzergah üzerinde olduğunu farz ederiz. Yani, her $v \in V$ tepesi için akış ağı bir güzergah $s \rightsquigarrow v \rightsquigarrow t$ içerir. Çizge bu yüzden bağlantılıdır ve s den başka her tepe en az bir giriş (entering) ayrıtına sahip olduğu için $|E| \geq |V| - 1$ dir. Kısım 26.1 akış ağının bir örneğini gösterir.

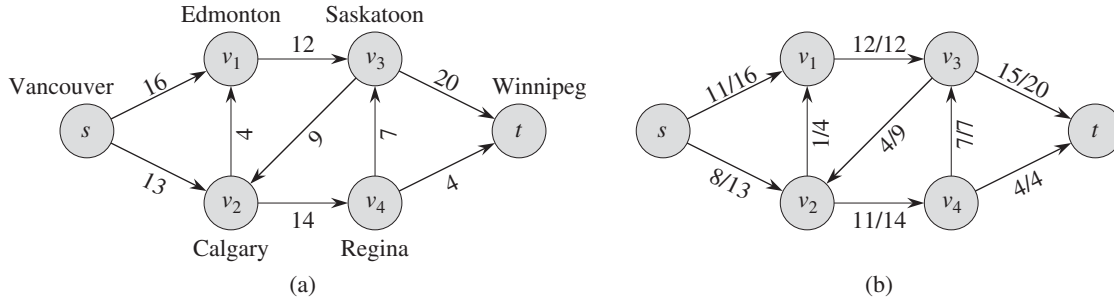
Şimdi akışları resmi olarak tanımlamaya hazırız. $G = (V, E)$ kapasite fonksiyonu c olan bir akış ağı olsun. i ağın kaynağı ve t de çıkış düğümü (sink) olsun. G deki bir **akış (flow)** reel değerli bir fonksiyondur $f : V \times V \rightarrow \mathbb{R}$ ki bu aşağıdaki iki özelliği sağlar:

Kapasite kısıtı: Her $u, v \in V$ için, $0 \leq f(u, v) \leq c(u, v)$ ye gerek duyarız.

Akış korunumu: Her $u \in V - \{s, t\}$ için,

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v) \text{ gerek duyarız.}$$

$(u, v) \notin E$ iken, u dan v ye akış olamaz ve $f(u, v) = 0$ dir.



Şekil 26.1 (a) Lucky Puck Şirketi'nin nakliyat problemi için bir $G = (V, E)$ akış ağı. Vancouver fabrikası s kaynağı ve Winnipeg deposu t çıkış düğümüdür. Şirket buz hokeyi toplarını aradaki şehirler aracılığıyla nakletmektedir fakat günde sadece $c(u, v)$ sandık u şehrinde v şehrine gidebilmektedir. Her ayrıntı kendi kapasitesi ile etiketlenmiştir. (b) $|f| = 19$ değerine sahip G deki f akışı. Her (u, v) ayrıntı $f(u, v)/c(u, v)$ ile etiketlenmiştir. Bölü gösterimi yalnızca akış ve kapasiteyi ayırmakta olup, bölümü belirtmemektedir.

Negatif olmayan $f(u, v)$ miktarını, u tepesinden v tepesine akış olarak adlandırırız. f akışının $|f|$ değeri (value) aşağıdaki gibi tanımlanır:

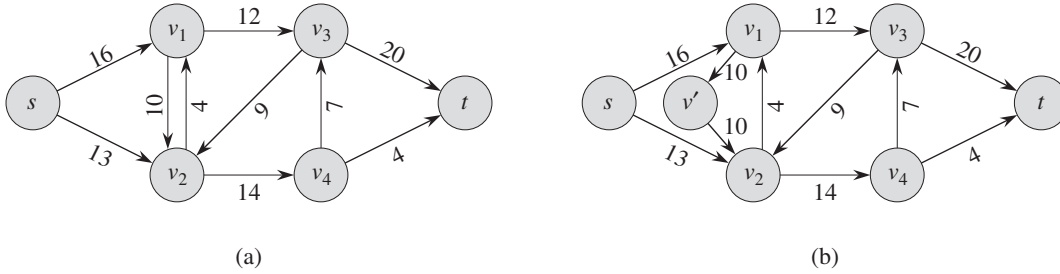
$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s), \quad (26.1)$$

yani, kaynaktan dışa doğru olan toplam akış eksi kaynağa doğru olan akıştır (Burada, $|\cdot|$ notasyonu akış değerini ifade etmekte olup, mutlak değer veya kardinalite ifade etmemektedir). Genel olarak bir akış ağı kaynağa doğru herhangi bir ayrıta sahip olmayacaktır ve $\sum_{v \in V} f(v, s)$ toplamı ile verilen kaynağa doğru olan akış 0 olacaktır. Buna rağmen onu dahil ettik, çünkü bu bölümde daha sonra artık ağları tanıttığımız zaman kaynağa doğru olan akış önemli hale gelecektir. **Maksimum-akış problemi (maximum-flow problem)**, s kaynaklı ve t çıkış düğümlü bir G akış ağı verilmektedir ve biz maksimum değere sahip bir akış bulmak isteriz.

Bir ağ-akış problem örneğini görmeden önce akışın tanımını ve iki akış özelliğini kısaca açıklayalım. Kapasite kısıtı basit olarak, bir tepeden diğerine olan akışın negatif olmayacağı ve verilen kapasite değerini aşmayacağını söylemektedir. Akış-korunum özelliği, kaynak ve çıkış düğümü dışındaki bir tepeden içeriye doğru olan toplam akışın bu tepeden dışarıya doğru olan akışa eşit olacağını söylemektedir—gayri resmi olarak “giren akış çıkan akışa eşittir.”

Bir akış örneği

Bir akış ağı Şekil 26.1 de gösterilen nakliyat problemini modelleyebilir. Lucky Puck Şirketi'nin Vancouver'da hokey topu üreten bir fabrikası (kaynak s) ve bunları depoladığı Winnipeg'de bir deposu (çıkış düğümü) vardır. Lucky Puck hokeyi toplarını fabrikadan depoya nakletmek için başka bir şirketten kamyonlarda yer



Şekil 26.2 Anti-paralel ayrıtlı bir ağın denk paralel ayrıtlı bir ağı dönüştürülmesi. **(a)** (v_1, v_2) ve (v_2, v_1) ayrıtlarının her ikisini de içeren bir akış ağı. **(b)** Anti-paralel ayrıtları bulunmayan bir denk ağ. v' yeni tepesini ekledik ve (v_1, v_2) ayrıtlını, her ikisi de (v_1, v_2) ile aynı kapasiteye sahip, (v_1, v') ve (v', v_2) ayrıtlı çifti ile değiştirdik.

kiralamaktadır. Kamyonlar şehirler (tepeler) arasında belirlenen yollardan (ayrıtlar) gittiği ve sınırlı kapasiteye sahip oldukları için, Lucky Puck Şekil 26.1 (a) daki her u ve v şehir çifti arasında günde en fazla $c(u, v)$ sandık nakledebilmektedir. Lucky Puck'ın bu yollar ve kapasiteler üzerinde kontrolü bulunmamaktadır ve dolayısıyla Şekil 26.1(a) da yer alan akış ağını değiştiremez. Deposuna nakledebileceğinden fazla hokey topu üretmesinin bir anlamı olmayacağından dolayı, günde nakledebilecekleri en fazla sandık sayısı p yi belirlemeye ve bu miktarda üretmeye ihtiyaçları bulunmaktadır. Lucky Puck bir hokey topunun fabrikadan depoya ulaşmasının ne kadar zaman alacağı ile ilgilenmemektedir; sadece fabrikadan günde çıkan p sandık ve depoya günde giren p sandık ile ilgilenmektedirler.

Bir şehirden başka bir şehire nakledilen sandık sayısı kapasite kısıtına tabi olduğundan, nakliyat akışlarını bu ağdaki bir denk olarak modelleyebiliriz. Ek olarak, model akış korunumuna uymalıdır, kararlı bir durum için, ara şehire giren hokey topu oranının, çıkışındaki orana eşit olması gerekmektedir. Aksi takdirde sandıklar ara şehirlerde birikir.

Problemleri anti-paralel ayrıtlar ile modelleme

Kamyon firmasının Lucky Puck'a Edmonton'dan Calgary'ye giden kamyonlarda 10 sandıklık yer kiralama fırsatı sunduğunu varsayalım. Bu fırsatı kendi örneğimize eklememiz ve Şekil 26.2(a) da gösterilen ağı oluşturmamız doğal gözükmemektedir. Bu ağ bir sorundan muzdariptir, halbuki, eğer bir (v_1, v_2) ayrıtlı $\in E$ ise $(v_2, v_1) \in E$ şeklindeki ilk varsayımımızı ihlal etmektedir. (v_1, v_2) ve (v_2, v_1) ayrıtlarını **anti-paralel (antiparallel)** olarak adlandırırız. Böylece, eğer bir akış problemini anti-paralel ayrıtlar ile modellemek istersek, ağı anti-paralel ayrıtlar

içermeyen bir dengine dönüştürmemiz gerekmektedir. Şekil 26.2(b) bu denk ağı göstermektedir. İki anti-paralel ayrıttan birini seçeriz, bu durumda (v_1, v_2) dir, yeni bir v' tepesi ekleyerek ve (v_1, v_2) ayrıtını (v_1, v') ve (v', v_2) ayrıt çifti ile değiştirerek onu böleriz. Ayrıca her iki yeni ayrıtın kapasitesini ilk ayrıtın kapasitesiyle aynı olacak şekilde ayarlarız. Elde edilen ağ eğer bir ayrıt ağda ise, ters ayrıt ağda değildir özelliğini sağlamaktadır. Alıştırma 26.1-1 sizden elde edilen ağı ilk ağa denk olduğunu kanıtlamanızı istemektedir.

Böylece, gerçek dünyadaki akış probleminin, bir anti-paralel ayrıtlı ağ ile en doğal şekilde modellenebileceğini görmüş olduk. Anti-paralel ayrıtlara izin vermemek uygun olacaktır, buna rağmen, anti-paralel ayrıtlara sahip bir ağı anti-paralel ayrıtı olmayan denk bir ağa çevirmenin basit bir yoluna sahibiz.

Çoklu kaynaklara ve çıkış düğümlerine sahip ağlar

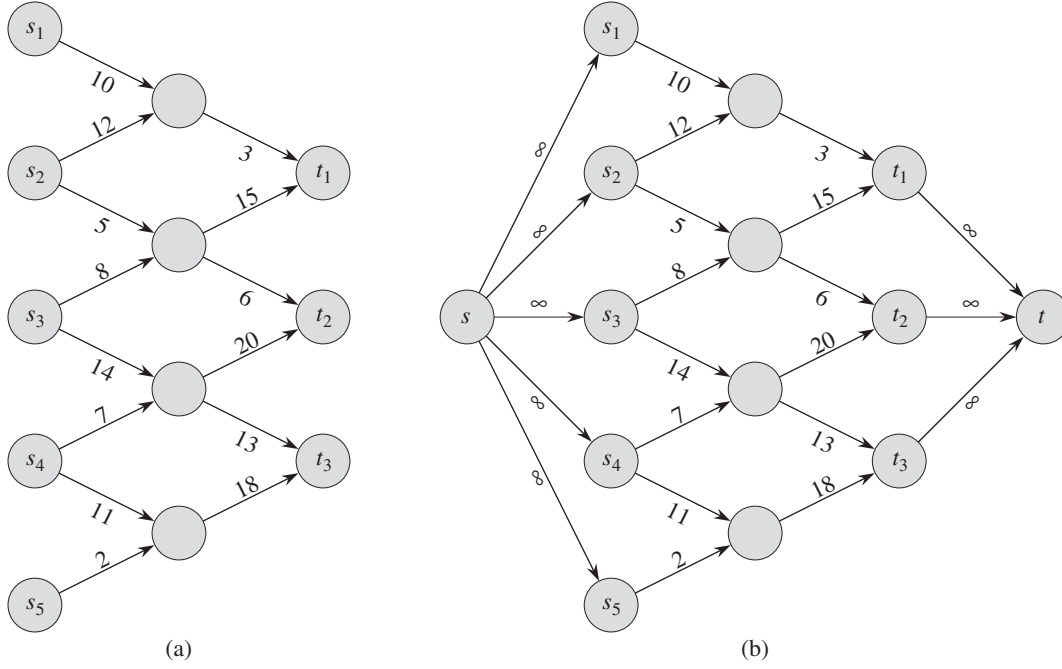
Bir maksimum akış problemi herbirinden bir tane olması yerine çeşitli kaynaklara ve çıkış düğümlerine sahip olabilir. Şekil 26.3(a) da gösterildiği gibi örneğin Lucky Puck Şirketi, aslında, m fabrika $\{s_1, s_2, \dots, s_m\}$ ve n depo $\{t_1, t_2, \dots, t_n\}$ ya sahiptir. Neyse ki, bu problem alışılmış maksimum akıştan daha zor değildir.

Çoklu kaynaklar ve çoklu çıkış düğümlerine sahip bir ağdaki maksimum akışı belirleme problemini alışılmış bir maksimum akış problemine indirgeyebiliriz. Şekil 26.3(b) (a) daki ağı nasıl tek kaynak ve tek çıkış düğümlü alışılmış bir akış ağına çevrileceğini göstermektedir. Bir **süper kaynak (supersource)** ve her $i = 1, 2, \dots, m$ için $c(s, s_i) = \infty$ kapasiteli bir yönlü (s, s_i) ayrıtı ekleriz. Ayrıca bir **süper çıkış düğümü (supersink)** yaratırız ve her $i = 1, 2, \dots, n$ için $c(t_i, t) = \infty$ kapasiteli bir yönlü (t_i, t) ayrıtı ekleriz. Sezgisel olarak, (a) da yer alan ağdaki herhangi bir akış, (b) de yer alan ağdaki bir akışa karşılık gelmektedir veya tam tersi de geçerlidir. Tek kaynak s basitçe çoklu s_i kaynakları için arzu edildiği kadar akışları sağlamaktadır ve aynı şekilde tek t çıkış düğümü çoklu t_i çıkış düğümleri için arzu edilen akışları tüketmektedir. Alıştırma 26.1-2 her iki problemin aynı olduğunu resmi olarak kanıtlamanızı istemektedir.

Alıştırmalar

26.1-1

Bir akış ağındaki bir ayrıtı bölmenin bir denk ağla sonuçlanacağını gösteriniz. G akış ağıının, (u, v) ayrıtını içerdiğini varsayalım ve yeni bir x tepesi yaratarak ve $c(u, x) = c(x, v) = c(u, v)$ olmak üzere (u, v) 'yi yeni (u, x) ve (x, v) ayrıtları ile değiştirerek yeni bir G' akış ağı yaratalım. G' içindeki maksimum akışın G 'deki maksimum akış ile aynı değere sahip olduğunu gösteriniz.



Şekil 26.3 Çoklu kaynak ve çoklu çıkış düğümü maksimum akış problemini, bir tek kaynak ve bir tek çıkış düğümü problemine dönüştürmek. (a) Beş kaynaklı $S = \{s_1, s_2, s_3, s_4, s_5\}$ ve üç çıkış düğümü $T = \{t_1, t_2, t_3\}$ bir akış ağı (b) Eşdeğer tek kaynaklı, tek çıkış düğümlü akış ağı. Bir süper kaynak s ve s den çoklu kaynakların her birine giden sonsuz kapasiteli ayrıtlar ekleriz. Ayrıca bir süper çıkış düğümü t ve çoklu çıkış düğümlerinin her birinden t ye giden sonsuz kapasiteli ayrıtlar ekleriz.

26.1-2

Akış özelliklerini ve tanımlamalarını çoklu kaynak, çoklu çıkış düğümü problemine genişletiniz. Çoklu kaynak, çoklu çıkış düğümü akış ağı içerisindeki herhangi bir akışın süper kaynak ve süper çıkış düğümünün eklenmesiyle elde edilen tekli kaynak, tekli çıkış düğümü ağı içerisindeki denk bir akışa karşılık geldiğini gösteriniz.

26.1-3

Bir $G = (V, E)$ akış ağının ağın her $v \in V$ tepesi için bir $s \rightsquigarrow v \rightsquigarrow t$ yolu içerdiği varsayımını ihlal ettiğini kabul edelim. u bir tepe olsun öyle ki bir $s \rightsquigarrow u \rightsquigarrow t$ yolu olmasın. G de her $v \in V$ tepesi için $f(u, v) = f(v, u) = 0$ olan maksimum bir f akışının olması gerektiğini gösteriniz.

26.1-4

f bir ağda bir akış olsun ve α reel bir sayı olsun. αf ile temsil edilen **skaler akış çarpımı (scalar flow product)**, $V \times V$ den $\mathbb{R}$ ye $(\alpha f)(u, v) = \alpha \cdot f(u, v)$ ile tanımlanan bir fonksiyon olsun. Bir ağdaki akışların bir **dışbükey küme (convex set)** oluşturduğunu kanıtlayınız. Yani, f_1 ve f_2 akış ise $0 \leq \alpha \leq 1$ aralığındaki tüm α değerleri için $\alpha f_1 + (1 - \alpha) f_2$ nin de akış olduğunu gösterin.

26.1-5

Maksimum akış problemini bir doğrusal programlama problemi olarak ifade edin.

26.1-6

Profesör Adam'ın maalesef birbirini sevmeyen iki çocuğu vardır. Problem öyle zor ki, çocuklar sadece okula birlikte yürümeyi reddetmiyor aynı zamanda biri herhangi bir binaya yürümeyi reddettiğinde diğer çocuk o gün o binaya basıp gidiyor. Çocukların yollarının bir tepede kesişmesinde bir sorun yok. Neyse ki, hem profesörün evi hem de okul tepelerde fakat bunun dışında profesör çocuklarının her ikisini aynı okula göndermesinin mümkün olabileceğinden emin değil. Profesörde kasabasının bir haritası var. Her iki çocuğunun aynı okula gidebileceğini maksimum akış problemi olarak nasıl formülleşebileceğini gösterin.

26.1-7

Farz edelim ki, ayrıt kapasitelerine ek olarak bir akış ağı **tepe kapasitelerine (vertex capacities)** sahiptir. Yani her bir v tepesi, v üzerinden ne kadar akışın geçebileceği üzerine bir $l(v)$ limitine sahiptir. Tepe kapasiteli bir $G = (V, E)$ akış ağının tepe kapasitesiz bir $G' = (V', E')$ akış ağına nasıl dönüştürüldüğünü gösterin, öyle ki, G' de maksimum bir akış G de aynı değerli maksimum bir akış olsun. G' kaç tane tepe ve ayrıta sahiptir?

26.2 Ford-Fulkerson yöntemi

Bu bölüm maksimum akış problem çözümleri için Ford-Fulkerson yöntemini sunar. Farklı çalışma zamanlarında bir çok uygulamayı kapsamaması nedeniyle “Algoritma” yerine “yöntem” terimi olarak isimlendirdik. Ford-Fulkerson yöntemi kendini baskın kılan 3 önemli fikre bağlıdır ve bunlar birçok bir çok akış algoritmaları ve problemleri ile ilgilidir: artık ağlar, genişletilmiş yollar ve kesimler. Bu fikirler önemli maks-akış min-kesim teoreminin (Teorem 26.6) temelini oluşturur ki akış ağlarının kesimleri açısından bir maksimum akışın değerini karakterize

eder. Bu bölümü Ford-Fulkerson yönteminin özel bir gerçekleştirimini tanıtarak ve bunun çalışma zamanını analiz ederek bitiriyoruz.

Ford-Fulkerson yöntemi akışının değerini tekrarlayarak artırır. Biz o değerin bir başlangıç akışını vererek tüm $u, v \in V$ için $f(u, v) = 0$ ile başlayalım. Her bir iterasyonda, bir bağıntılı “Artık ağ” G_f ’de bir “Genişletilmiş yol” bularak G ’de akış değerini arttırırız. Bir de G_f ’inde bir genişletilmiş yolunun kenarlarını biliyoruz, akışını değiştirebileceğimiz yani akış değerini arttırabileceğimiz G ’de belirli ayrıtları rahatlıkla tanımlayabiliriz. Ford-Fulkerson yönteminin her bir iterasyonunda akış değerini arttırıyor olmasına rağmen, artan veya azalan G ’nin herhangi bir özel ayrıtındaki akışı görelim; aynı ayrıt üzerindeki akış azalması kaynaktan çıkış düğümüne daha fazla akış göndermek için bir algoritmayı mümkün kılacak bazı ayrıtlar üzerinde akış azaltılması gerekli olabilir. Artık ağ daha fazla genişletilmiş yolları bulundurmaması duruma kadar biz tekrarlı olarak akışı genişletebiliriz. Maks-akış min-kesim teoremi terminasyon üzerinde gösterir ki, bu işlem maksimum akış kazancı sağlar.

FORD-FULKERSON-METHOD(G, s, t)

- 1 f akışını 0 ile başlat
- 2 **while** G_f ’artık ağında genişletilmiş bir p yolu var
- 3 p boyunca f akışını arttır
- 4 **return** f

Ford-Fulkerson yöntemini uygulamak ve analiz etmek için birçok ilave kavramlar tanıtılmalıdır.

Artık ağlar

Sezgisel olarak, verilen bir G akış ağı ve bir f akışı için, G_f ’artık ağı G ’nin ayrıtları üzerindeki akışı nasıl değiştirebileceğimizi temsil eden kapasiteli ayrıtları içerir. Akış ağının bir ayrıtı, eklenen akış miktarının ayrıtın kapasitesi ile o ayrıttaki akışın farkına eşit olmasına izin verir. Eğer bu değer pozitif ise, ise, bu ayrıtı $c_f(u, v) = c(u, v) - f(u, v)$ “artık kapasite” li G_f ’ine yerleştiririz. G ’in sadece G_f ’de bulunan ayrıtları daha çok akış kabul edilebilir olanlardır; akışları kapasitelerine eşit olan bu (u, v) ayrıtları için $c_f(u, v) = 0$ ’dır ve bunlar G_f ’in içinde değildir.

Bununla birlikte, artık G_f ’ağı aynı zamanda G ’inde olmayan ayrıtları içerebilir. Bir algoritma akışı yönlendirdiği gibi, toplam akışı yükseltmesi amacıyla, belirli bir ayrıt üzerindeki akışı da düşürmek gerekebilir. G ’deki bir ayrıtın pozitif $f(u, v)$ akışının muhtemel azalmasını temsil etmesi için, G_f ’ine $c_f(v, u) = f(u, v)$ artık kapasiteli bir (v, u) ayrıtı yerleştiririz—yani, ters (u, v) yönünde akışa izin verebilen, en fazla (u, v) ’deki akışı sıfırlayan bir ayrıt. Artık ağdaki bu ters ayrıtlar

bir ayrıt boyunca zaten gönderilmiş akışın geri gönderilmesi için bir algoritmaya imkan verir. Bir ayrıt boyunca akışı geri göndermek ayrıttaki akışı azaltmakla denktir ki bu çoğu algoritmada gerekli bir işlemdir.

Daha resmi şekilde, s kaynağı ve t çıkış düğümü olan bir akış ağı $G = (V, E)$ ye sahip olduğumuzu farz edebiliriz. f G de bir akış olsun ve $u, v \in V$ olan bir çift tepe düşünelim $c_f(u, v)$ **artık kapasitesini (residual capacity)** şu şekilde tanımlayabiliriz:

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v), & (u, v) \in E \text{ ise,} \\ f(v, u), & (v, u) \in E \text{ ise,} \\ 0, & \text{aksi halde.} \end{cases} \quad (26.2)$$

Çünkü bizim $(u, v) \in E$ varsayımımız $(v, u) \notin E$ anlamına gelir, (26.2) eşitliğindeki tam olarak bir durum.

Denklem 26.2 ye örnek olarak, $c(u, v) = 16$ ve $f(u, v) = 11$ ise, sonra biz $f(v, u)$ yi (u, v) ayrıtında kapasite sınırını aşmadan $c_f(u, v) = 5$ birim oluncaya kadar artırabiliriz. Aynı zamanda algoritmanın v den u ya 11 birim akış döndürmesine izin vermek isteriz, bu yüzden $c_f(u, v) = 11$ dir.

$G = (V, E)$ akış ağı ve f akışı verilsin, G nin f tarafından tetiklenen **artık ağı (residual network)** $G_f = (V, E_f)$ dir.

$$E_f = \{(u, v) \in V \times V : c_f(u, v) > 0\} \quad (26.3)$$

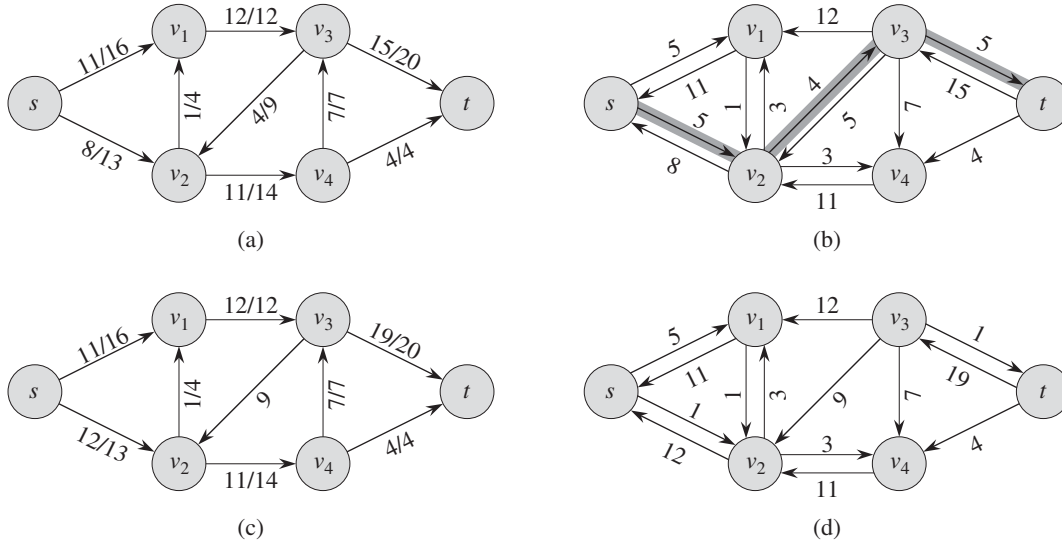
yani yukarıda taahhüt edildiği gibi artık ağın her ayrıtı veya **artık ayrıt (residual edge)** 0 dan büyük bir akışa izin verir. Şekil 26.4(a) Şekil 26.1(b) deki G akış ağını ve f akışını tekrar eder ve Şekil 26.4(b) buna karşılık gelen G_f artık ağını gösterir. E_f deki ayrıtlar ya E deki ayrıtlardır ya da onların tersleridir ve bu yüzden $|E_f| \leq 2 |E|$

olur.

G_f artık ağının c_f de verilen akış ağı kapasiteleriyle aynı olduğu fark edilmelidir. Bu bizim akış ağı tanımımıza uymaz çünkü bu (u, v) ve bunun tersi (v, u) ayrıtlarının ikisini birden içerebilir. Ayrıca bu fark dışında, bir artık ağ akış, ağ ile aynı özelliklere sahip G_f artık ağ içindeki akışı akış tanımlarını dikkate alarak tanımlayabiliriz fakat G_f ağındaki c_f kapasitelerine uymalıyız.

Artık ağdaki bir akış, orijinal akış ağına akış eklemede yol haritası sağlar. Eğer f G de bir akış ve f' buna uygun düşen G_f artık ağındaki akış ise, biz f' tarafından f deki akış artışını $f \uparrow f'$ olarak tanımlarız, bu $V \times V$ den $\mathbb{R}$ ye bir fonksiyondur, şöyle tanımlanır

$$(f \uparrow f')(u, v) = \begin{cases} f(u, v) + f'(u, v) - f'(v, u), & (u, v) \in E \text{ ise,} \\ 0, & \text{aksi halde.} \end{cases} \quad (26.4)$$



Şekil 26.4 (a) Şekil 26.1 (b) nin G akış ağı ve f akışı. (b) Gölge genişletilmiş p yoluna sahip artık G_f ağı; artık kapasitesi $c_f(p) = c_f(v_2, v_3) = 4$ tür. Örneğin (v_1, v_3) gibi artık kapasitesi 0 olan ayrıtlar gösterilmez, bu bölümün geri kalanında izleyeceğimiz bir kuraldır. (c) kendi artık kapasitesi 4 ün p yolu boyunca genişletilmesiyle sonuçlanan G deki akış., (v_3, v_2) gibi akış taşımayan ayrıtlar sadece kendi kapasiteleri ile etiketlenirler, baştan sona izleyeceğimiz bir başka kural. (d) (c) deki akış ile oluşan artık ağı.

Bu tanımın arkasındaki önsezi artık ağ tanımını izler. (u, v) üzerindeki akışı $f'(u, v)$ kadar artırırız ancak $f'(v, u)$ kadar azaltırız, çünkü artık ağdaki ters ayrıt üzerinde akış itmek orijinal ağdaki akışın azalmasına işaret eder. Artık ağdaki ters ayrıt üzerinde akış itmek aynı zamanda **iptal (cancellation)** olarak da bilinir. Örneğin, eğer u dan v ye 5 kasa hokey topu gönderirsek ve v den u ya 2 kasa gönderirsek, u dan v ye sadece (sonuç açısından) 3 kasa göndermiş ve v den u ya hiç göndermemiş oluruz. Bu tip bir iptal her maksimum-akış algoritması için önemlidir.

Lemma 26.1

$G = (V, E)$, s kaynaklı ve t çıkış düğümlü bir akış ağı olsun ve f , G de bir akış olsun. G_f ile oluşturulan G nin artık ağı olsun ve f' , G_f de bir akış olsun. Böylece, (26,4) eşitliğinde tanımlanan $f \uparrow f'$ fonksiyonu G de $|f \uparrow f'| = |f| + |f'|$ değerine sahip bir akıştır. .

Kanıt Öncelikle $f \uparrow f'$ nin E deki her ayrıt için kapasite kısıtına ve $V - \{s, t\}$ deki her tepe için akış korunumuna uyduğunu doğrularız.

Kapasite kısıtı için öncelikle eğer $(u, v) \in E$ ise o zaman $c_f(v, u) = f(u, v)$ olduğunu gözlemleyin. Böylece, $f'(v, u) \leq c_f(v, u) = f(u, v)$ dir ve bundan dolayı

$$\begin{aligned} (f \uparrow f')(u, v) &= f(u, v) + f'(u, v) - f'(v, u) \text{ (denklem (26.4) den)} \\ &\geq f(u, v) + f'(u, v) - f(u, v) \text{ (çünkü } f'(v, u) \leq f(u, v) \text{ dir)} \\ &= f'(u, v) \\ &\geq 0 \end{aligned}$$

dır. Ek olarak,
 $(f \uparrow f')(u, v)$

$$\begin{aligned} &= f(u, v) + f'(u, v) - f'(v, u) \text{ (denklem (26.4) den)} \\ &\leq f(u, v) + f'(u, v) \text{ (çünkü akışlar negatif değil)} \\ &\leq f(u, v) + c_f(u, v) \text{ (kapasite kısıtı)} \\ &= f(u, v) + c(u, v) - f(u, v) \text{ (} c_f \text{'nin tanımı)} \\ &= c(u, v) \text{ dir} \end{aligned}$$

Akış korunumu için f ve f' akış korunumuna uyduklarından dolayı, tüm $u \in V - \{s, t\}$ için

$$\begin{aligned} \sum_{v \in V} (f \uparrow f')(u, v) &= \sum_{v \in V} (f(u, v) + f'(u, v) - f'(v, u)) \\ &= \sum_{v \in V} f(u, v) + \sum_{v \in V} f'(u, v) - \sum_{v \in V} f'(v, u) \\ &= \sum_{v \in V} f(v, u) + \sum_{v \in V} f'(v, u) - \sum_{v \in V} f'(u, v) \\ &= \sum_{v \in V} (f(v, u) + f'(v, u) - f'(u, v)) \\ &= \sum_{v \in V} (f \uparrow f')(v, u) \end{aligned}$$

öyle ki, akış korunumundan üçüncü satır ikinciye takip eder.

Sonuç olarak, $f \uparrow f'$ değerini hesaplayabiliriz. G de anti-paralel ayrıtlara izin vermediğimizi hatırlayalım (ama G_f de değil) ve dolayısıyla her $v \in V$ tepesi için, bir (s, v) veya (v, s) ayrıtı olabileceğini ancak ikisinin birlikte asla olmayacağını biliyoruz. $V_1 = \{v : (s, v) \in E\}$ kümesini s den ayrıtlara sahip tepeler kümesi olarak ve $V_2 = \{v : (v, s) \in E\}$ kümesini s ye ayrıtları olan tepelerin kümesi olarak tanımlarız. $V_1 \cup V_2 \subseteq V$ olduğu ve anti-paralel ayrıtlara izin vermediğimiz için $V_1 \cap V_2 = \emptyset$ dir. Şimdi hesaplayabiliriz

$$\begin{aligned} |f \uparrow f'| &= \sum_{v \in V} (f \uparrow f')(s, v) - \sum_{v \in V} (f \uparrow f')(v, s) \\ &= \sum_{v \in V_1} (f \uparrow f')(s, v) - \sum_{v \in V_2} (f \uparrow f')(v, s) \end{aligned} \tag{26.5}$$

öyleki, eğer $(w, x) \notin E$ ise $(f \uparrow f')(w, x) = 0$ olduğu için ikinci satır devam eder. Şimdi denklem (26.5) e $f \uparrow f'$ tanımını uyguluyoruz ve sonra aşağıdaki denklemi elde etmek için terimleri yeniden sıralayıp, gruplarız:

$$\begin{aligned}
 |f \uparrow f'| &= \sum_{v \in V_1} (f(s, v) + f'(s, v) - f'(v, s)) - \sum_{v \in V_2} (f(v, s) + f'(v, s) - f'(s, v)) \\
 &= \sum_{v \in V_1} f(s, v) + \sum_{v \in V_1} f'(s, v) - \sum_{v \in V_1} f'(v, s) \\
 &\quad - \sum_{v \in V_2} f(v, s) - \sum_{v \in V_2} f'(v, s) + \sum_{v \in V_2} f'(s, v) \\
 &= \sum_{v \in V_1} f(s, v) - \sum_{v \in V_2} f(v, s) \\
 &\quad + \sum_{v \in V_1} f'(s, v) + \sum_{v \in V_2} f'(s, v) - \sum_{v \in V_1} f'(v, s) - \sum_{v \in V_2} f'(v, s) \\
 &= \sum_{v \in V_1} f(s, v) - \sum_{v \in V_2} f(v, s) + \sum_{v \in V_1 \cup V_2} f'(s, v) - \sum_{v \in V_1 \cup V_2} f'(v, s). \quad (26.6)
 \end{aligned}$$

Denklem (26.6) da, V üzerinde toplamak için dört toplamı da genişletebiliriz, çünkü her bir ilave terim 0 değerine sahiptir. (Alıştırma 26.2-1 bunu kanıtlamanızı ister.) Böylece

$$\begin{aligned}
 |f \uparrow f'| &= \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{v \in V} f'(s, v) - \sum_{v \in V} f'(v, s) \quad (26.7) \\
 &= |f| + |f'|
 \end{aligned}$$

elde ederiz. ■

Genişletilmiş yollar

Bir $G = (V, E)$ akış ağı ve bir f akışı verilsin, bir p **genişletilmiş yolu** (*augmenting path*) G_f artık ağında s den t ye basit bir yoldur. Artık ağ tanımına göre, bir genişletilmiş yolun bir (u, v) ayrıtındaki akışı, orijinal G akış ağında (u, v) ve (v, u) nun herhangi biri üzerindeki kapasite kısıtlarını aşmadan, $c_f(u, v)$ kadar artırabiliriz.

Şekil 26.4(b) deki gölgeli yol bir genişletilmiş yoldur. Şekildeki G_f artık ağına bir akış ağı gibi davranarak, bir kapasite kısıtını ihlal etmeden bu yolun her bir ayrıtı boyunca akışı 4 birim artırabiliriz, çünkü bu yoldaki en küçük artık kapasite $c_f(v_2, v_3) = 4$ tür. Bir artan p yolunun her bir ayrıtı üzerindeki artırabileceğimiz akışın maksimum miktarını p nin **artık kapasitesi** (*residual capacity*) olarak adlandırırız, şu şekilde verilmiştir

$$c_f(p) = \min \{c_f(u, v) : (u, v) \text{ } p \text{ üzerindedir}\}.$$

Kanıtı Alıştırma 26.2-7 olarak bıraktığımız aşağıdaki lemma, yukarıdaki argümanı daha kesin kılmaktadır.

Lemma 26.2

$G = (V, E)$ bir akış ağı olsun, f G de bir akış olsun ve p G_f de genişletilmiş bir yol olsun. Bir $f_p : V \times V \rightarrow \mathbb{R}$ fonksiyonunu

$$f_p(u, v) = \begin{cases} c_f(p), & (u, v) \text{ } p \text{ üzerinde ise,} \\ 0, & \text{aksi halde} \end{cases} \quad (26.8)$$

ile tanımlayın. O halde, f_p G_f de $|f_p| = c_f(p) > 0$ değerine sahip bir akıştır. ■

Aşağıdaki sonuç eğer f akışı f_p kadar artırılırsa, G de değeri maksimuma daha yakın başka bir akış elde ettiğimiz göstermektedir. Şekil 26.4(c), Şekil 26.4(a) daki f akışı Şekil 26.4(b) deki f_p akışı kadar artırıldığında elde edilen sonucu göstermektedir ve Şekil 26.4(d) sonraki artık ağı göstermektedir.

Sonuç 26.3

$G = (V, E)$ bir akış ağı olsun, f G de bir akış olsun ve p G_f de genişletilmiş bir yol olsun. f_p (26.8) denklemindeki gibi tanımlanmış olsun ve f yi f_p kadar artırdığımızı kabul edelim. O halde, $f \uparrow f_p$ fonksiyonu G de $|f \uparrow f_p| = |f| + |f_p| > |f|$ değerine sahip bir akıştır.

Kanıt Doğrudan 26.1 ve 26.2 lemmalarından. ■

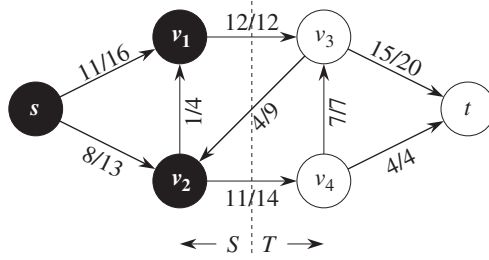
Akış ağlarının kesimleri

Ford-Fulkerson yöntemi maksimum bir akış bulununcaya kadar akışı genişletilmiş yollar boyunca tekrar tekrar artırır. Algoritma sona erdiğinde gerçekten maksimum bir akış bulup bulmadığımızı nasıl bilebiliriz? Biraz sonra kanıtlayacağımız maks-akış min-kesim teoremi, bir akışın ancak ve ancak artık ağın genişletilmiş yol içermediğinde maksimum olacağını bize söylemektedir. Bu teoremi kanıtlamak için öncelikle bir akış ağının bir kesimi kavramını incelemeliyiz.

$G = (V, E)$ akış ağının bir (S, T) **kesimi** V nin S ve $T = V - S$ ye bölünmesidir öyle ki $s \in S$ ve $t \in T$ dir. (Bu tanım, Bölüm 23 de minimum kapsayan ağaçlar için kullandığımız “kesim” tanımına benzerdir, ancak burada bir yönsüz çizge yerine bir yönlü çizgeyi kesiyoruz ve $s \in S$ ile $t \in T$ olmasını ısrar ediyoruz.) Eğer f bir akış ise, o zaman (S, T) kesimi boyunca $f(S, T)$ **ağ akışı**:

$$f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) \quad (26.9)$$

olarak tanımlanır.



Şekil 26.5 $S = \{s, v_1, v_2\}$ ve $T = \{v_3, v_4, t\}$ olan, Şekil 26.1(b) deki akış ağındaki bulunan bir (S, T) kesimi. S deki tepeler siyah ve T deki tepeler beyazdır. (S, T) boyunca net akış $f(S, T) = 19$ ve kapasite $c(S, T) = 26$ dir.

(S, T) kesiminin **kapasitesi**

$$c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v) \quad (26.10)$$

dir.

Bir ağı bir **minimum kesimi**, ağıdaki tüm kesimler içinden kapasitesi minimum olan bir kesimdir.

Akışın ve bir kesimin kapasitesinin tanımları arasındaki asimetri kasıtlı ve önemlidir. Kapasite için sadece S den T ye giden ayrıtların kapasitelerini hesaplar, ters yöndeki ayrıtları göz ardı ederiz. Akış için S den T ye giden akışı göz önüne alır, T den S ye giden ters yöndeki akışı çıkarırız. Bu farkın nedeni bu bölümde daha sonra anlaşılabacaktır.

Şekil 26.5, Şekil 26.1(b) nin akış ağındaki $(\{s, v_1, v_2\}, \{v_3, v_4, t\})$ kesimini göstermektedir. Bu kesim boyunca ağ akışı

$$\begin{aligned} f(v_1, v_3) + f(v_2, v_4) - f(v_3, v_2) &= 12 + 11 - 4 \\ &= 19 \end{aligned}$$

ve bu kesimin kapasitesi

$$\begin{aligned} c(v_1, v_3) + c(v_2, v_4) &= 12 + 14 \\ &= 26 \end{aligned}$$

dır.

Aşağıdaki lemma, verilen bir f akışı için herhangi bir kesim boyunca ağ akışının aynı olduğunu ve bunun da akışının değeri olan $|f|$ ye eşit olduğunu göstermektedir.

Lemma 26.4

s kaynaklı ve t çıkış düğümlü bir $G = (V, E)$ akış ağındaki f bir akış olsun ve (S, T) G 'nin herhangi bir kesimi olsun. O halde, (S, T) boyunca ağ akışı $f(S, T) = |f|$ dir.

Kanıt Herhangi bir $u \in V - \{s, t\}$ düğümü için akış korunumu şartını aşağıdaki gibi yeniden yazabiliriz:

$$\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) = 0. \quad (26.11)$$

(26.1) denklemindeki $|f|$ nin tanımını alarak ve (26.11) denkleminin sıfıra eşit olan sol tarafını $S - \{s\}$ deki tüm tepeler üzerindeki toplamını ekleyerek aşağıdaki elde edilir:

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{u \in S - \{s\}} \left(\sum_{v \in V} f(u, v) - \sum_{v \in V} f(v, u) \right).$$

Sağ taraftaki toplamın açılması ve terimlerin yeniden gruplandırılması aşağıdaki verir:

$$\begin{aligned} |f| &= \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) + \sum_{u \in S - \{s\}} \sum_{v \in V} f(u, v) - \sum_{u \in S - \{s\}} \sum_{v \in V} f(v, u) \\ &= \sum_{v \in V} \left(f(s, v) + \sum_{u \in S - \{s\}} f(u, v) \right) - \sum_{v \in V} \left(f(v, s) + \sum_{u \in S - \{s\}} f(v, u) \right) \\ &= \sum_{v \in V} \sum_{u \in S} f(u, v) - \sum_{v \in V} \sum_{u \in S} f(v, u). \end{aligned}$$

$V = S \cup T$ ve $S \cap T = \emptyset$ olduğu için V üzerindeki her toplamı S ve T üzerindeki toplamlara ayırıp aşağıdakini elde edebiliriz:

$$\begin{aligned} |f| &= \sum_{v \in S} \sum_{u \in S} f(u, v) + \sum_{v \in T} \sum_{u \in S} f(u, v) - \sum_{v \in S} \sum_{u \in S} f(v, u) - \sum_{v \in T} \sum_{u \in S} f(v, u) \\ &= \sum_{v \in T} \sum_{u \in S} f(u, v) - \sum_{v \in T} \sum_{u \in S} f(v, u) \\ &\quad + \left(\sum_{v \in S} \sum_{u \in S} f(u, v) - \sum_{v \in S} \sum_{u \in S} f(v, u) \right). \end{aligned}$$

Parantezler içindeki iki toplam aslında birbirinin aynısıdır, çünkü tüm $x, y \in V$ tepeleri için $f(x, y)$ terimi her toplamda bir kere gözükmektedir. Dolayısıyla bu toplamlar iptal olur ve

$$\begin{aligned} |f| &= \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) \\ &= f(S, T) \end{aligned}$$

dir. ■

Lemma 26.4 ün bir sonucu, bir akışın değerini sınırlamak için kesim kapasitelerini nasıl kullanabileceğimizi göstermektedir

Sonuç 26.5

Bir G akış ağında herhangi bir f akışının değeri G nin herhangi bir kesiminin kapasitesi ile üstten sınırlıdır.

Kanıt (S, T) G nin herhangi bir kesimi olsun ve f herhangi bir akış olsun. Lemma 26.4 ve kapasite kısıtı ile

$$\begin{aligned}
 |f| &= f(S, T) \\
 &= \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u) \\
 &\leq \sum_{u \in S} \sum_{v \in T} f(u, v) \\
 &\leq \sum_{u \in S} \sum_{v \in T} c(u, v) \\
 &= c(S, T)
 \end{aligned}$$

■

dir.

Sonuç 26.5 bir ağdaki maksimum akışın değeri ağın bir minimum kesiminin kapasitesi ile üstten sınırlı olduğu sonucunu doğrudan vermektedir. Şimdi ifade edeceğimiz ve kanıtlayacağımız çok önemli maks-akış min-kesim teoremi bir maksimum akışın değerinin aslında bir minimum kesimin kapasitesine eşit olduğunu söylemektedir.

Teorem 26.6 (maks-akış min-kesim teoremi)

s kaynaklı ve t çıkış düğümlü bir $G = (V, E)$ akış ağında, eğer f bir akış ise o zaman aşağıdaki koşullar denktir:

1. f akışı G de bir maksimum akıştır.
2. G_f artık ağı genişletilmiş yollar içermez.
3. G nin bazı (S, T) kesimleri için $|f| = c(S, T)$ dir.

Kanıt (1) $\Rightarrow$ (2): Çelişki olsun diye f nin G de maksimum bir akış olduğunu ama G_f nin bir genişletilmiş p yoluna sahip olduğunu kabul edelim. Sonra, Sonuç 26.3 ile, f yi (26.8) denkleminde verilen f_p kadar artırarak elde edilen akış $|f|$ den kesinlikle daha büyük değerli G de bir akıştır. Bu da f nin maksimum bir akış olduğu kabulü ile çelişmektedir.

(2) $\Rightarrow$ (3): G_f nin genişletilmiş yola sahip olmadığını yani G_f nin s den y ye yol içermediğini kabul edelim.

$S = \{ v \in V : G_f \text{ de } s \text{ den } v \text{ ye bir yol vardır} \}$

ve $T = V - S$ yi tanımlayın. (S, T) parçası bir kesimdir: açıkça $s \in S$ ve $t \notin S$ dir, çünkü G_f de s den t ye yol yoktur. Şimdi bir çift $u \in S$ ve $v \in T$ tepesi ele alalım.

Eğer $(u, v) \in E$ ise $f(u, v) = c(u, v)$ olmalıdır, çünkü aksi halde $(u, v) \in E_f$ dir, bu da v yi S kümesine yerleştirecektir. Eğer $(u, v) \in E$ ise $f(v, u) = 0$ olmalıdır, çünkü aksi halde $c_f(u, v) = f(v, u)$ pozitif ve $(u, v) \in E_f$ olacaktır, bu da v yi S ye yerleştirecektir. Şüphesiz, ne (u, v) ne de $(v, u) \in E$ de değil ise, o zaman $f(u, v) = f(v, u) = 0$ dir. Bu yüzden,

$$\begin{aligned} f(S, T) &= \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{v \in T} \sum_{u \in S} f(v, u) \\ &= \sum_{u \in S} \sum_{v \in T} c(u, v) - \sum_{v \in T} \sum_{u \in S} 0 \\ &= c(S, T) . \end{aligned}$$

dir.

26.4 lemması ile bu yüzden $|f| = f(S, T) = c(S, T)$ dir.

(3) $\Rightarrow$ (1): Sonuç 26.5 den tüm (S, T) kesimleri için $|f| \leq c(S, T)$ dir. $|f| = c(S, T)$ koşulu f nin bir maksimum akış olduğunu göstermektedir. ■

Temel Ford-Fulkerson algoritması

Ford-Fulkerson yönteminin her iterasyonunda, bir *genişletilmiş* p yolu buluruz ve p yi f akışını düzenlemek için kullanırız. Lemma 26.2 ve Sonuç 26.3'ün önerdiği gibi, f yi $|f| + |f_p|$ değerine sahip yeni bir $f \uparrow f_p$ akışı ile yer değiştiririz. Yöntemin aşağıdaki uygulaması bir $G = (V, E)$ akış ağındaki maksimum akışı her $(u, v) \in E$ ayrıtı için $(u, v).f$ akış niteliğini güncelleyerek hesaplar.¹ Eğer $(u, v) \notin E$ ise, $(u, v).f = 0$ olduğunu dolaylı olarak kabul ederiz. Ayrıca, akış ağı ile birlikte $c(u, v)$ kapasitelerinin verilmiş olduğunu varsayarız ve eğer $(u, v) \notin E$ ise $c(u, v) = 0$ dir. (26.2) formülüne göre $c_f(u, v)$ artık kapasitesini hesaplarız. Koddaki $c_f(p)$ ifadesi p yolunun artık kapasitesini tutan yalnızca geçici bir değişkendir.

FORD-FULKERSON(G, s, t)

```

1  for her  $(u, v) \in G.E$  ayrıtı için
2       $(u, v).f = 0$ 
3  while artık  $G_f$  ağında  $s$  den  $t$  ye bir  $p$  yolu var
4       $c_f(p) = \min \{ c_f(u, v) : (u, v) p \text{ içindedir} \}$ 
5      for  $p$  deki her  $(u, v)$  ayrıtı için
6          if  $(u, v) \in E$ 
7               $(u, v).f = (u, v).f + c_f(p)$ 
8          else  $(v, u).f = (v, u).f - c_f(p)$ 
```

¹Kısım 22.1 den hatırlayın: (u, v) ayrıtı için bir f niteliğini diğer herhangi bir nesnenin bir niteliği için kullandığımız $(u, v).f$ gösterimi ile aynı biçimde temsil ediyoruz.

FORD-FULKERSON algoritması basitçe önceden verilen FORD-FULKERSON-METHOD kabakodunu genişletir. Şekil 26.6 bir örnek üzerinde her iterasyonun sonuçlarını gösterir. Satır 1-2, f akışını 0 dan başlatır. Satır 3-8 deki while döngüsü, tekrar tekrar G_f de genişletilmiş bir p yolu bulur ve p boyunca f akışını artık $c_f(p)$ kapasitesi kadar arttırır. p yolundaki her artık ayrıt ya orijinal ağda bir ayrıt ya da orijinal ağdaki bir ayrıtın zıttıdır. Satır 6-8 artık ayrıt orijinal bir ayrıt ise akış ekleyerek aksi durumda çıkartarak, her durumdaki akışı düzgünce yeniler. Genişletilmiş yollar olmadığında f akışı maksimum bir akıştır.

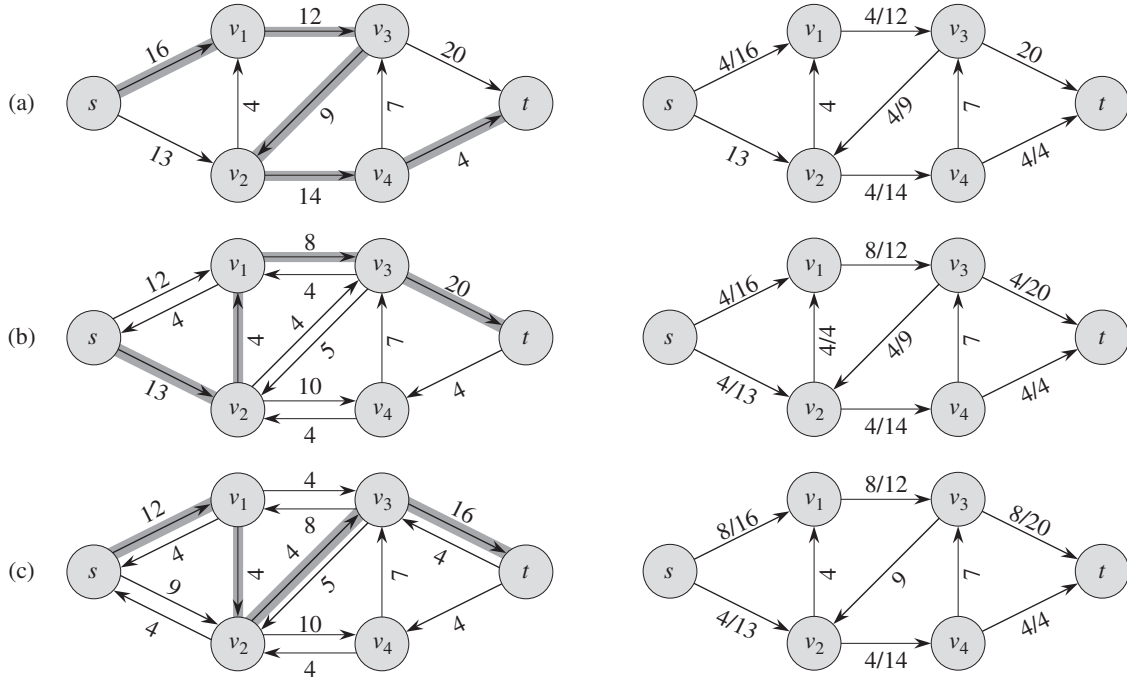
Ford-Fulkerson'ın analizi

FORD-FULKERSON'ın çalışma zamanı satır 3 teki genişletilmiş p yolunu nasıl bulduğumuza bağlıdır. Eğer onu kötü seçersek, algoritma sonlanmayabilir bile: akışın değeri ardışık eklemelerle artacaktır, ama maksimum akış değerine yakınsaması gerekmez.² Eğer genişletilmiş yolu bir enine arama (Kısım 22.2 de gördüğümüz) kullanarak bulursak algoritma polinom zamanda çalışır. Bu sonucu kanıtlamadan önce, genişletilmiş yolu keyfi olarak seçtiğimiz ve tüm kapasitelerin tam sayı olduğu durum için basit bir sınır elde ediyoruz.

Uygulamada maksimum-akış problemi genellikle tam sayı (integral) kapasiteleri ile ortaya çıkar. Eğer kapasiteler rasyonel sayılar ise, hepsini tam sayı yapmak için uygun bir ölçekleme dönüşümü uygulayabiliriz. Eğer f^* dönüştürülen ağdaki bir maksimum akışı gösteriyorsa, sonra FORD-FULKERSON'ın basit bir gerçekleştirimi akış değeri her iterasyonda en az bir birim arttığından **while** döngüsünün 3–8 satırlarını en fazla $|f^*|$ defa yürütür.

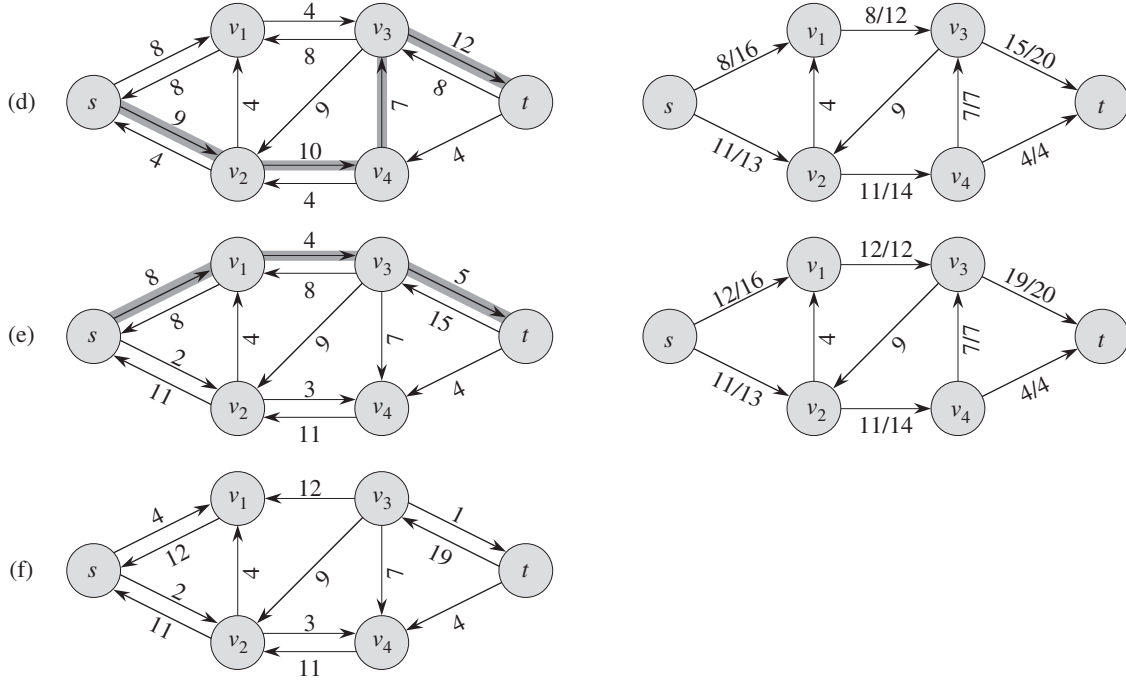
Eğer doğru veri yapısı ile $G = (V, E)$ akış ağını gerçekleştirecek ve doğrusal-zamanlı bir algoritma ile genişletilmiş bir yol bulabilirsek, **while** döngüsüyle yapılan çalışmayı verimli bir şekilde yerine getirebiliriz. Yönlü bir $G' = (V, E')$ çizgesine karşılık gelen bir veri yapısı tuttuğumuzu varsayalım, $E' = \{(u, v) : (u, v) \in E \text{ veya } (v, u) \in E\}$. G ağındaki ayrıtlar aynı zamanda G' ağının da ayrıtlarıdır ve bu yüzden bu veri yapısında kapasiteleri ve akışları kolayca sürdürebiliriz. G üzerinde bir f akışı verildiğinde, G_f artık ağdaki ayrıtlar G' nün tüm (u, v) lerini içerir, öyle ki $c_f(u, v) > 0$ burada c_f denklem (26.2) ye uygundur. Bir artık ağda bir yol bulma zamanı eğer ya derinliğine arama ya da enine arama kullanırsak $O(V + E') = O(E)$ dir. **while** döngüsünün her iterasyonu böylece $O(E)$ zaman alır, satır 1–2 de başlangıç yapıldığından FORD-FULKERSON algoritmasının toplam çalışma süresini $O(E |f^*|)$ yapar.

²Ford Fulkerson yöntemi ancak ayrıt kapasiteleri irrasyonel sayılar olursa sonlanmada başarısız olabilir.



Şekil 26.6 Temel Ford Fulkerson algoritmasının çalışması. **(a)-(e)** While döngüsünün ardışık iterasyonları. Her bir parçanın sol tarafı taralı genişletilmiş p yolu ile satır 3 ten artık G_f ağını gösterir. Her bir parçanın sağ tarafı f nin f_b kadar artışıyla elde edilen yeni akış f yi gösterir. (a) daki artık ağ G girdi ağıdır.

Kapasiteler tam sayı ve optimum akış değeri $|f^*|$ küçük olduğu zaman, Ford-Fulkerson algoritmasının çalışma süresi iyidir. Şekil 26.7(a) $|f^*|$ ı büyük olan basit bir akış ağı üzerinde neler olabileceğinin bir örneğini göstermektedir. Bu ağdaki bir maksimum akış değeri 2,000,000 dur: 1,000,000 birim akış $s \rightarrow u \rightarrow t$ yolunu gezer ve diğer bir 1,000,000 birim $s \rightarrow v \rightarrow t$ yolunu gezer. Eğer FORD-FULKERSON tarafından bulunan Şekil 26.7(a) da gösterilen ilk genişletilmiş yol $s \rightarrow u \rightarrow v \rightarrow t$ ise ilk iterasyondan sonra akışın değeri 1 dir. Elde genişletilmiş artık ağ Şekil 26.7(b) de gösterilmiştir. Eğer ikinci iterasyon Şekil 26.7(b) de gösterildiği gibi $s \rightarrow v \rightarrow u \rightarrow t$ genişletilmiş yolunu bulur ise akışın değeri 2 dir. Şekil 26.7(c) elde edilen artık ağı gösterir. Tek sayılı iterasyonlarında $s \rightarrow u \rightarrow v \rightarrow t$ genişletilmiş yolunu ve çift sayılı iterasyonlarında $s \rightarrow v \rightarrow u \rightarrow t$ genişletilmiş yolunu seçerek devam edebiliriz. Her birinde akış değerini sadece 1 birim artırarak toplam 2,000,000 artırmayı gerçekleştiririz.



Şekil 26.6 devamı (f) Son **while** döngü testindeki artık ağ. Genişletilmiş yollar yoktur ve (e) de gösterilen f akışı bu yüzden bir maksimum akıştır. Bulunan maksimum akışın değeri 23 tür.

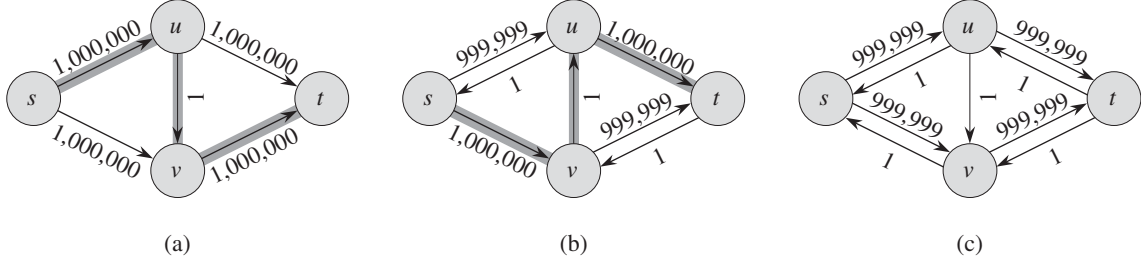
Edmonds-Karp algoritması

Satır 3 deki genişletilmiş p yolunu enine arama ile bularak FORD-FULKERSON üzerindeki sınırı geliştirebiliriz. Yani, her ayrıtın birim uzaklığa (ağırlığa) sahip olduğu artık ağda s den t ye bir en kısa yol olarak genişletilmiş yolu seçeriz. Ford-Fulkerson yöntemi **Edmonds-Karp algoritması**nın gerçekleştirilmesidir diyoruz. Şimdi Edmonds-Karp algoritmasının $O(VE^2)$ zamanda çalıştığını kanıtlıyoruz.

Analiz artık G_f ağındaki tepelerin uzaklıklarına bağlıdır. Aşağıdaki lemma, $\delta_f(u, v)$ gösterimini her ayrıtın birim mesafede olduğu G_f de u dan v ye en kısa yol için kullanıyor.

Lemma 26.7

Eğer Edmonds-Karp algoritması $G = (V, E)$ akış ağı üzerinde s kaynağı ve t çıkış düğümü için ile çalıştırılırsa, tüm $v \in V - \{s, t\}$ tepeleri için artık G_f ağındaki $\delta_f(s, v)$ en kısa-yol uzunluğu her bir akış genişlemesi ile monoton artar.



Şekil 26.7 (a) Maksimum akış, burada $|f^*| = 2,000,000$ olarak gösterilen f^* in maksimum olduğu bir akış ağı, FORD FULKERSON için $\Theta(E |f^*|)$ zamanı alabilir. Gölgeyi yol artık kapasitesi 1 olan genişletilmiş bir yoldur. (b) Artık kapasitesi 1 olan başka bir genişletilmiş yola sahip elde edilen artık ağ. (c) Elde edilen artık ağ.

Kanıt Bazı $v \in V - \{s, t\}$ tepeleri için s den v ye en kısa yol uzaklığını azaltmaya neden olan bir akış genişlemesi olduğunu varsayalım ve daha sonra bir çelişki elde edeceğiz. f bazı en kısa yol uzaklığını azaltan ilk genişlemeden hemen önceki akış olsun ve f hemen sonrasındaki akış olsun. v genişlemesi ile uzaklığı azalan minimum $\delta_{f'}(s, v)$ ye sahip tepe olsun, böylece $\delta_{f'}(s, v) < \delta_f(s, v)$. $p = s \rightsquigarrow u \rightarrow v$ yolu $G_{f'}$ de s den v ye bir en kısa yol olsun, öyle ki $(u, v) \in E_{f'}$ ve

$$\delta_{f'}(s, u) = \delta_{f'}(s, v) - 1. \quad (26.12)$$

v yi nasıl seçtiysek, kaynak s den u tepesine uzaklığın azalmadığını biliyoruz, yani

$$\delta_{f'}(s, u) \geq \delta_f(s, u). \quad (26.13)$$

$(u, v) \notin E_f$ olduğunu iddia ediyoruz. Niçin? Eğer $(u, v) \in E_f$ olsaydı aynı zamanda

$$\begin{aligned} \delta_f(s, v) &\leq \delta_f(s, u) + 1 \quad (\text{Lemma 24.10 dan, üçgen eşitsizliği}) \\ &\leq \delta_{f'}(s, u) + 1 \quad ((26.13) \text{ eşitsizliğinden}) \\ &= \delta_{f'}(s, v) \quad ((26.12) \text{ denkleminde}) \text{ sahip olurduk, bu da} \end{aligned}$$

$\delta_{f'}(s, v) < \delta_f(s, v)$ olduğu varsayımımız ile çelişmektedir.

Nasıl $(u, v) \notin E_f$ ve $(u, v) \in E_{f'}$ olabilir? Artış v den u ya akışı artırmalıdır. The Edmonds-Karp algoritması her zaman en kısa yollar boyunca akışı artırmaktadır ve bu nedenle G_f de s den u ya en kısa yol son ayrıt olarak (v, u) ya sahiptir. Bu nedenle

$$\begin{aligned} \delta_f(s, v) &= \delta_f(s, u) - 1 \\ &\leq \delta_{f'}(s, u) - 1 \quad ((26.13) \text{ eşitsizliğinden}) \\ &= \delta_{f'}(s, v) - 2 \quad ((26.12) \text{ denkleminde}), \end{aligned}$$

$\delta_{f'}(s, v) < \delta_f(s, v)$ olduğu varsayımımız ile çelişmektedir. Böyle bir v tepesinin var olduğu kabulü yanlıştır diye sonuçlandırırız. ■

Bir sonraki teorem Edmonds-Karp algoritmasının iterasyon sayısını sınırlamaktadır.

Teorem 26.8

Eğer Edmonds-Karp algoritması s kaynak ve t çıkış düğümlü bir $G = (V, E)$ akış ağı üzerinde çalışırsa, o zaman algoritma ile gerçekleştirilen akış genişlemelerinin toplam sayısı $O(VE)$ dir.

Kanıt Bir G_f artık ağındaki (u, v) ayrıtının bir p genişletilmiş yolu üzerinde **kritik** olduğunu söyleriz eğer p nin artık kapasitesi (u, v) nin artık kapasitesi ise, yani, eğer $c_f(p) = c_f(u, v)$ ise. Akışı bir genişletilmiş yol boyunca artırdıktan sonra, yol üzerindeki herhangi bir kritik ayrıt artık ağdan kaybolur. Ayrıca, herhangi bir genişletilmiş yol üzerinde en az bir ayrıt kritik olmalıdır. $|E|$ ayrıtın her birinin en çok $|V|/2$ kez kritik olabileceğini göstereceğiz.

u ve v E den bir ayrıt ile bağlı olan V deki tepeler olsun. Genişletilmiş yollar en kısa yollar olduğu için (u, v) ilk defa kritik olduğunda

$$\delta_f(s, v) = \delta_f(s, u) + 1 \text{ e sahibiz.}$$

Akış artırıldıktan sonra, (u, v) ayrıtı artık ağdan kaybolur. u dan v ye akış azalınca kadar başka bir genişletilmiş yol üzerinde tekrar görünemez. Eğer bu olay olduğunda f' G de akış ise,

$$\delta_{f'}(s, u) = \delta_{f'}(s, v) + 1$$

e sahip oluruz.

Lemma 26.7 den $\delta_f(s, v) \leq \delta_{f'}(s, v)$ olduğu için

$$\begin{aligned} \delta_{f'}(s, u) &= \delta_{f'}(s, v) + 1 \\ &\geq \delta_f(s, v) + 1 \\ &= \delta_f(s, u) + 2 \end{aligned}$$

dir.

Sonuç olarak, (u, v) nin kritik olduğu zamandan sonraki kritik hale geldiği zamana kadar, kaynaktan u nun uzaklığı en az 2 kat artar. Kaynaktan u ya uzaklık ilk olarak en az 0 dir. s den u ya en kısa yol üzerindeki orta tepeler s yi, u yu, veya t yi (genişletilmiş bir yol üzerinde (u, v) $u \neq t$ yi gerektirdiği için) içeremez. Böylece, u kaynaktan ulaşamaz hale gelinceye kadar, eğer olursa, uzaklığı en çok $|V| - 2$ dir. Böylece, (u, v) nin ilk kez kritik olduğu zamandan sonra, en çok $(|V| - 2)/2 = |V|/2 - 1$ kez daha kritik olabilir, bu da toplamda en çok $|V|/2$ kez demektir. Bir artık ağ içinde $O(E)$ tepe çifti arasında bir ayrıt olduğu için, Edmon-

ds-Karp algoritmasının tümünü yürütülmesi sırasında kritik ayrıtların toplam sayısı $O(VE)$ dir. Her genişletilmiş yol en az bir kritik ayrıta sahiptir ve böylece teorem kanıtlanır. ■

Genişletilmiş yolu enine arama ile bulduğumuzda FORD-FULKERSON'ın her iterasyonunu $O(E)$ zamanda gerçekleştirebildiğimiz için, Edmonds-Karp algoritmasının toplam çalışma zamanı $O(VE^2)$ dir. İtme-yeniden etiketleme (push-relabel) algoritmalarının daha iyi sınırlar verebileceğini göreceğiz. Kısım 26.5 deki $O(V^3)$ zamanlı algoritma için temel oluşturan, Kısım 26.4'ün algoritması $O(V^2E)$ çalışma zamanına ulaşmak için bir metod verir.

Alıştırmalar

26.2-1

Denklem (26.6) daki toplamların denklem (26.7) daki toplamlara eşit olduğunu kanıtlayın.

26.2-2

Şekil 26.1(b) de, $(\{s, v_2, v_4\}, \{v_1, v_3, t\})$ kesimi boyunca akış nedir? Bu kesimin kapasitesi nedir?

26.2-3

Şekil 26.1(a) daki akış ağı üzerinde Edmonds-Karp algoritmasının yürütülmesini gösterin.

26.2-4

Şekil 26.6 daki örnekte gösterilen, maksimum akışa karşılık gelen minimum kesim nedir? Örnekte görülen genişletilmiş yollardan hangisi akışı iptal eder?

26.2-5

Çoklu kaynak ve çoklu çıkış düğümlü bir akış ağını tek kaynak, tek çıkış düğümlü ağa sonsuz kapasite ile ayrıtlar ekleyerek dönüştüren Kısım 26.1 deki yapıyı hatırlayın. Eğer çoklu kaynak ve çıkış düğümlü orijinal ağda ayrıtlar sınırlı kapasiteye sahipse, elde edilen ağdaki her hangi bir akışın sınırlı değere sahip olduğunu kanıtlayın.

26.2-6

Çoklu kaynak ve çıkış düğümlü bir akış ağında her bir kaynak s_i nin tam olarak p_i akış birimi ürettiğini varsayalım, böylece $\sum_{v \in V} f(s_i, v) = p_i$ dir. Her t_j çıkış düğümünün tam olarak q_j birim tükettiğini de varsayalım, böylece $\sum_i p_i = \sum_j q_j$ olduğu yerde, $\sum_{v \in V} f(v, t_j) = q_j$ dir. Bir f akışı bulma problemini tek kaynak,

tek çıkış düğümlü akış ağı içinde bir maksimum akış bulma problemine bu ek kısıtlara uyarak nasıl dönüştürüleceğini gösterin.

26.2-7

Lemma 26.2 yi kanıtlayınız.

26.2-8

s içine ayrıtlara izin vermemek için artık ağı yeniden tanımladığımızı varsayalım. FORD-FULKERSON prosedürünün maksimum akış için hala doğru hesapladığını tartışın.

26.2-9

Hem f hem de f' , G ağında akışlarolduğunu ve $f \uparrow f'$ akışını hesapladığımızı varsayalım. Genişletilmiş akış, akış koruma özelliğini yerine getiriyor mu? Bu kapasite kısıtını karşılar mı?

26.2-10

Bir $G = (V, E)$ ağında en çok $|E|$ genişletilmiş yolların bir dizisi ile bir akışın nasıl bulunduğunu gösterin. (*İpucu: Maksimum akış bulduktan sonra yolları belirleyin*).

26.2-11

Yönsüz çizgenin **ayrıt bağlanabilirliği** (*edge connectivity*) çizgeyi kesmek için taşınması gereken minimum ayrıt sayısı k dır. Örneğin, bir ağacın ayrıt bağlanabilirliği 1 dir ve çevrimsel tepeler zincirinin ayrıt bağlanabilirliği 2 dir. Her biri $O(V)$ tepe ve $O(E)$ ayrıtlı en çok $|V|$ akış ağında bir maksimum akış algoritması çalıştırarak, $G = (V, E)$ yönsüz çizgesinin ayrıt bağlanabilirliğinin nasıl belirleneceğini gösterin.

26.2-12

Bir G akış ağı verildiğini, ve G nin s kaynağının girildiği ayrıtlara sahip olduğunu varsayalım. f kaynağa giren (v, s) ayrıtlarından birinde $f(s, v) = 1$ olan bir akış olsun. Başka bir f' akışının olması gerektiğini kanıtlayınız öyle ki $|f| = |f'|$ ve $f'(v, s) = 0$ olsun. Tüm ayrıt kapasitelerinin tam sayı olduğu varsayılan, verilen f için f' hesaplayan $O(E)$ zamanlı bir algoritma yazın.

26.2-13

Tam sayı kapasiteli bir G akış ağı içindeki tüm minimum kesimler arasında, en az sayıda ayrıta sahip olanı bulmak istediğinizi varsayalım. Yeni bir G' akış ağı oluşturmak için G nin kapasitelerinin nasıl değiştirileceğini gösterin. Öyle ki, G' deki herhangi minimum kesim G de en az sayıda ayrıta sahip minimum bir kesimdir.

26.3 Maksimum iki parçalı eşleme

Bazı kombinatoriyel problemler kolayca maksimum-akış problemlerine dönüştürülebilir. Kısım 26.1 den çoklu-kaynak, çoklu-çıkış düğümü maksimum-akış problemi bize bir örnek verdi. Bazı diğer kombinatoriyel problemler akış ağları ile çok az ilgisi var görünür, ama aslında maksimum-akış problemlerine indirgenebilirler. Bu bölüm böyle bir problemi sunar: iki parçalı çizgelerde maksimum eşleme bulma. Bu problemi çözmek için, Ford-Fulkerson yöntemiyle sağlanan tamlama özelliğinin avantajından yararlanacağız. Aynı zamanda bir $G = (V, E)$ çizgesi üzerinde $O(VE)$ zamanında Ford-Fulkerson yönteminin maksimum iki parçalı eşleme probleminin çözümünde nasıl kullanıldığını göreceğiz.

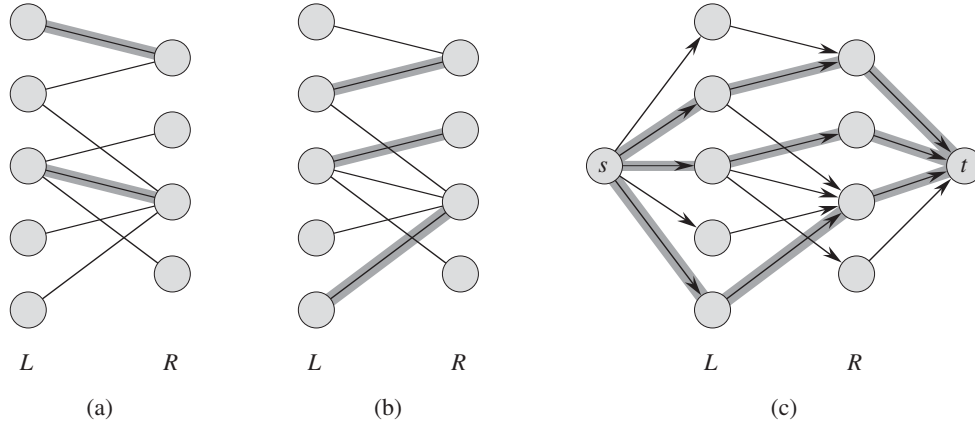
Maksimum iki parçalı eşleme problemi

Bir $G = (V, E)$ yönsüz çizgesi verildiğinde, bir *eşleme (matching)* ayrıtların bir $M \subseteq E$ alt kümesidir, öyle ki, tüm $v \in V$ tepeleri için M nin en fazla bir ayrıtı v ye bağlıdır. Bir $v \in V$ tepesi eğer M deki bazı ayrıtlar v ye bağlı ise M **maksimum eşleme (maximum matching)** ile **eşlenmiş (matched)**, aksi halde ise v **eşleştirilmemiş (unmatched)** deriz. Bir **maksimum eşleme (maximum matching)** maksimum büyüklükte bir eşlemedir, yani, bir M eşlemesi herhangi bir M' eşlemesi için, $|M| \geq |M'|$ dür. Bu bölümde, iki parçalı çizgelerde maksimum eşleme bulmak için dikkatimizi vereceğiz: tepe kümesinin $V = L \cup R$ şeklinde bölünebildiği ve E deki tüm ayrıtların L ile R arasında olduğu çizgeler iki parçalı çizgelerdir. V deki her tepenin en az bir bağlı ayrıta sahip olduğunu varsayalım. Şekil 26.8 bir iki parçalı çizgede eşleme kavramını göstermektedir.

İki parçalı çizgede maksimum eşleme bulma probleminin birçok pratik uygulaması vardır. Örnek olarak, makinelerin L kümesi ile aynı anda yapılacak görevlerin R kümesinin eşlemesini düşünebiliriz. E deki (u, v) ayrıtının varlığını, belirli bir $u \in L$ makinesinin belirli bir $v \in R$ görevin gerçekleştirimi için uygun anlamında alabiliriz. Maksimum bir eşleme mümkün olduğunca çok makinenin çalışmasını sağlar.

Maksimum iki parçalı eşleme bulma

Yönsüz iki parçalı bir $G = (V, E)$ çizgesinde maksimum bir eşleme bulmak için Ford-Fulkerson yöntemini $|V|$ ve $|E|$ cinsinden polinom zamanda kullanabiliriz. Şekil 26.8. (c) de gösterildiği gibi püf noktası, akışların eşlemelere karşılık geldiği bir akış ağ oluşturmaktır. İki parçalı G çizgesi için $G' = (V', E')$ **uygun akış ağını** aşağıdaki gibi tanımlarız. s kaynak ve t çıkış düğümü V de olmayan yeni



Şekil 26.8 $V = L \cup R$ tepe bölmeli iki parçalı bir $G = (V, E)$ çizgesi. (a) Gölgeyi ayrıtlarla gösterilmiş, 2 büyüklüğünde bir eşleme. (b) 3 büyüklüğünde bir maksimum eşleme. (c) Maksimum akışlı akış ağına karşılık gelen gösterilmektedir. Her ayrıt birim kapasiteye sahiptir. Gölgeyi ayrıtlar bir akışa sahiptir, diğerleri hiç akış taşımaz. L den R ye gölgeyi ayrıtlar (b) den maksimum eşleme olanlara karşılık gelir.

tepeler olsun ve $V = V \cup \{s, t\}$ olsun. Eğer G nin tepe bölütü $V = L \cup R$ ise, $|V|$ adet yeni yönlü ayrıtlar boyunca, L den R ye yönlendirilen, G' nün yönlü ayrıtları E nin ayrıtlarıdır:

$$E' = \{(s, u) : u \in L\} \cup \{(u, v) : (u, v) \in E\} \cup \{(v, t) : v \in R\}$$

Yapıyı tamamlamak için E' deki her ayrıta birim kapasite atıyoruz. V deki her tepenin en az bir bağlı ayrıtı olduğundan, $|E| \geq |V|/2$. Böylece, $|E| \leq |E'| = |E| + |V| \leq 3|E|$ ve bu yüzden $|E'| = \Theta(E)$.

Aşağıdaki lemma G' akış ağına karşılık gelen G deki bir akışa doğrudan te- kabul eden G deki eşlemeyi göstermektedir. Bir $G = (V, E)$ akış ağı üzerindeki f akışı, eğer $f(u, v)$ tüm $(u, v) \in V \times V$ ler için tam sayı ise **tam sayı-değerlidir** deriz.

Lemma 26.9

$G = (V, E)$, $V = L \cup R$ tepe bölmeli bir iki parçalı çizge olsun ve $G' = (V', E')$ ona karşılık gelen akış ağı olsun. Eğer M , G de bir eşleme ise G' de bir f tam sayı-değerli akış vardır, $|f| = |M|$. Diğer taraftan eğer f , G' de bir tam sayı-değerli akış ise G de $|M| = |f|$ kardinaliteli bir M eşlemesi vardır.

Kanıt Öncelikle G deki bir M eşlemesinin, G' deki tam sayı-değerli bir f akışına karşılık geldiğini gösteririz. f yi aşağıdaki gibi tanımlayın. Eğer $(u, v) \in M$ ise $f(s, u) = f(u, v) = f(v, t) = 1$ dir. Diğer tüm $(u, v) \in E'$ ayrıtları için $f(u, v) = 0$ tanımlarız. f in kapasite kısıtını ve akış korunumunu sağladığını doğrulamak basittir.

Sezgisel olarak, her $(u, v) \in M$ ayrıtı $s \rightarrow u \rightarrow v \rightarrow t$ yolunu gezen G' deki bir birim akışa karşılık gelmektedir. Ayrıca M deki ayrıtlar tarafından oluşturulan yollar, s ve t dışında, tepe-ayrıktırlar (vertex-disjoint). $(L \cup \{s\})$, $R \cup \{t\}$ kesiminden (cut) geçen ağ akışı $|M|$ ye eşittir; böylece Lemma 26.4 den, akış değeri $|f| = |M|$ olur.

Tersini kanıtlamak için f , G' de bir tam sayı-değerli akış olsun ve

$$M = \{(u, v): u \in L, v \in R, \text{ ve } f(u, v) > 0\}$$

olsun.

Her $u \in L$ tepesi (s, u) olarak adlandırılan sadece bir giriş ayrıtına sahiptir ve kapasitesi 1 dir. Bu yüzden her $u \in L$ en fazla bir birim akış girişine sahiptir ve eğer bir birim akış girerse, akış korunumuna göre, bir birim akış ayrılmak zorundadır. Ayrıca, f tam sayı-değerli olduğundan her $u \in L$ için bir birim akış en fazla bir ayrıtı girebilir ve en fazla bir ayrıttan çıkabilir. Bu yüzden, sadece $f(u, v) = 1$ olan bir $v \in R$ tepesi vardır ve her $u \in L$ den çıkan en fazla bir ayrıtı pozitif akış taşır ancak ve ancak u ya bir birim akış girer. Simetrik bir argüman her $v \in R$ için de geçerlidir. M kümesi bu yüzden bir eşlemedir.

$|f| = |M|$ olduğunu görmek için eşleşen her $u \in L$ tepesi için $f(s, u) = 1$ e sahip olduğumuzu ve her $(u, v) \in E - M$ ayrıtı için $f(u, v) = 0$ a sahip olduğumuzu gözlemleyin. Sonuçta, $(L \cup \{s\})$, $R \cup \{t\}$ kesiminden geçen $f(L \cup \{s\})$, $R \cup \{t\}$ akışı $|M|$ ye eşittir. Lemma 26.4 ü uygulayarak $|f| = f(L \cup \{s\}, R \cup \{t\}) = |M|$ yi elde ederiz ■

Lemma 26.9 a dayanarak, iki parçalı bir G çizgesindeki maksimum bir eşlemin karşılık gelen G' akış ağındaki bir maksimum akışa karşılık geldiği sonucunu çıkarmak istiyoruz ve bu yüzden G deki maksimum bir eşlemeyi G' üzerinde bir maksimum-akış algoritmasını çalıştırarak hesaplayabiliriz. Bu düşüncedeki tek aksaklık, $|f|$ akış değeri tam sayı olsa bile, maksimum-akış algoritması G' de bazı $f(u, v)$ için bir tam sayı olmayan akış döndürebilir. Aşağıdaki teorem eğer Ford-Fulkerson yöntemini kullanırsak, bu zorluğun ortaya çıkamayacağını göstermektedir.

Teorem 26.10 (Bütünlük teoremi)

Eğer c kapasite fonksiyonu sadece tam sayı (integral) değerler alıyorsa, Ford-Fulkerson yöntemi tarafından üretilen f maksimum akışı $|f|$ nin bir tam sayı olduğu özelliğine sahiptir. Hatta tüm u ve v tepeleri için $f(u, v)$ değeri bir tam sayıdır.

Kanıt Kanıt iterasyonların sayısı üzerinden tümevarım yoluyla yapılır. Bunu Alıştırma 26.3-2 olarak bırakıyoruz. ■

Şimdi Lemma 26.9 un aşağıdaki sonucunu kanıtlayabiliriz.

Sonuç 26.11

Bir iki parçalı G çizgesindeki bir M maksimum eşlemesinin kardinalitesi G ye karşılık gelen G' akış ağındaki bir maksimum f akışının değerine eşittir.

Kanıt Lemma 26.9 daki terminolojiyi kullanıyoruz. M nin G de bir maksimum eşleme olduğunu ve G' deki karşılık gelen f akışının maksimum olmadığını farz edin. O zaman G' de bir maksimum f' akışı vardır öyle ki $|f'| > |f|$. G deki kapasiteler tam sayı-değerli oldukları için Teorem 26.10 ile, f' yi tam sayı-değerli kabul edebiliriz. Bu yüzden f' , G de kardinalitesi $|M'| = |f'| > |M| = |f|$ olan bir M' eşlemesine karşılık gelmektedir ki bu M nin bir maksimum eşleme olduğu varsayımımız ile çelişmektedir. Benzer bir şekilde, eğer f G' de bir maksimum akış ise, ona karşılık gelen eşlemenin G de bir maksimum eşleme olduğunu gösterebiliriz. ■

Bu yüzden, verilen iki parçalı yönsüz bir G çizgesi için bir maksimum eşlemeyi G' akış ağını oluşturarak bulabiliriz şöyle ki: Ford-Fulkerson yöntemini çalıştırır ve bulunan tam sayı-değerli f maksimum akışından doğrudan bir M maksimum eşleme elde ederiz. İki parçalı bir çizgedeki herhangi bir eşleme en fazla $\min(L, R) = O(V)$ kardinalitesine sahip olduğu için G' deki maksimum akışın değeri $O(V)$ dir. $|E'| = \Theta(E)$ olduğundan dolayı iki parçalı bir çizgede $O(VE') = O(VE)$ zamanında bir maksimum eşleme bulabiliriz.

Alıştırmalar**26.3-1**

Ford-Fulkerson algoritmasını Şekil 26.8(c) deki akış ağında çalıştırın ve her akış genişlemesinden sonra artık (residual) ağı gösterin. L deki tepeleri üstten alta 1 den 5 e ve R deki tepeleri üstten alta 6 dan 9 a kadar numaralandırın. Her iterasyon için, sözlüksel olarak en küçük olan genişletilmiş yolu alın.

26.3-2

Teorem 26.10 u kanıtlayınız.

26.3-3

$G = (V, E)$, $V = L \cup R$ tepe bölmeli bir iki parçalı çizge olsun ve G' ona karşılık gelen akış ağı olsun. FORD-FULKERSON un çalıştırılması süresince G' de bulunan herhangi bir genişletilmiş yolun uzunluğuna iyi bir üst sınır verin.

26.3-4 *

Bir **kusursuz eşleme (perfect matching)** bütün tepelerin eşleştiği bir eşlemedir. $G = (V, E)$, $V = L \cup R$ tepe bölmeli yönsüz bir iki parçalı çizge olsun öyle ki $|L| = |R|$. Herhangi bir $X \subseteq V$ için X in **komsuluğunu (neighborhood)**

$$N(X) = \{y \in V : (x, y) \in E \text{ bazı } x \in X \text{ için}\}$$

şeklinde tanımlayın, yani X in bazı üyelerine komşu tepelerin kümesidir. **Hall’ın teoremini** kanıtlayın: Her $A \subseteq L$ alt kümesi için $|A| \leq |N(A)|$ dır ancak ve ancak G de kusursuz bir eşleme bulunur.

26.3-5 *

Eğer her $v \in V$ tepesi tam olarak d derecesine sahip ise, bir iki parçalı $G = (V, E)$ çizgesi, öyle ki $V = L \in R$, **d -düzenlidir (d -regular)** deriz. Bütün d -düzenli iki parçalı çizgeler $|L| = |R|$ eşitliğine sahiptirler. Bütün d -düzenli iki parçalı çizgelerin bir $|L|$ kardinaliteli eşlemeye sahip olduklarını, karşılık gelen akış ağının bir minimum kesiminin $|L|$ kapasitesine sahip olduğunu tartışarak kanıtlayınız.

* 26.4 İtme-yeniden etiketleme algoritmaları

Bu bölümde maksimum akışı hesaplamak için “itme-yeniden etiketleme (push-relabel)” yaklaşımını sunacağız. Bugüne kadar, asimptotik olarak en hızlı maksimum-akış algoritmalarının bir çoğu itme-yeniden etiketleme algoritmalarıdır ve maksimum-akış algoritmalarının en hızlı mevcut gerçekleştirmeleri itme-yeniden etiketleme yöntemine dayanmaktadır. Ayrıca itme-yeniden etiketleme yöntemleri minimum-maliyetli akış problemi gibi diğer akış problemlerini etkin şekilde çözmektedir. Bu bölüm Edmonds-Karp algoritmasının $O(VE^2)$ sınırının geliştirilmesiyle, $O(V^2E)$ zamanında çalışan basit bir gerçekleştirmesi olan, Goldberg’ın “genel” maksimum-akış algoritmasını tanıtmaktadır. Kısım 26.5 genel algoritmayı $O(V^3)$ zamanında çalışan başka bir itme-yeniden etiketleme algoritması elde etmek için geliştirmektedir.

İtme-yeniden etiketleme algoritmaları Ford-Fulkerson yönteminden daha lokal tarzda çalışmaktadırlar. Bir genişletilmiş yol bulmak için bütün artık ağı incelemekten ziyade, sadece artık ağdaki tepenin komşularına bakarak itme-yeniden etiketleme algoritmaları bir seferde bir tepe üstünde çalışırlar. Ayrıca, Ford-Fulkerson yönteminin aksine, itme-yeniden etiketleme algoritmaları çalışmaları boyunca akış-korunumu özelliğini sürdürememektedir. Bununla birlikte onlar, kapasite kısıtını ve aşağıdaki akış korunumu gevşetmesini sağlayan bir $f : V \times V \rightarrow \mathbb{R}$ fonksiyonu olan **önakış (preflow)** sürdürmektedir:

$$\sum_{v \in V} f(v, u) - \sum_{v \in V} f(u, v) \geq 0$$

tüm $u \in V - \{s\}$ tepeleri için. Yani, bir tepeye gelen akış çıkan akışı aşabilir.

$$e(u) = \sum_{v \in V} f(v, u) - \sum_{v \in V} f(u, v) \quad (26.14)$$

miktarını u tepesine gelen **aşırı akış(excess flow)** olarak adlandırırız. Bir tepedeki aşırılık içeri akışın dışarı akıştan fazla olan miktardır. Eğer $e(u) > 0$ ise, bir $u \in V - \{s, t\}$ tepesi **taşmıştır (overflowing)** deriz.

Bu bölüme itme-yeniden etiketleme yönteminin arkasındaki sezgiyi açıklayarak başlayacağız. Daha sonra yöntem tarafından kullanılan iki işlemi: önakış “itme” ve bir tepeyi “yeniden etiketleme”, inceleyeceğiz. Son olarak bir genel itme-yeniden etiketleme algoritması sunup, doğruluğunu ve çalışma zamanını analiz edeceğiz.

Sezgi

İtme-yeniden etiketleme yönteminin arkasındaki sezgiyi sıvı akışlar üzerinden anlayabilirsiniz: bir $G = (V, R)$ akış ağını, kapasiteleri verilmiş birbirine bağlı borular sistemi olarak ele alırız. Bu benzetimi Ford-Fulkerson yöntemine uygulayarak, ağdaki her genişletilmiş yol, dallanma olmayan kaynaktan hedefe akan bir ilave sıvı akışına artış vermektedir diyebiliriz. Ford-Fulkerson yöntemi iteratif olarak eklenemeyince kadar bir dizi akış ekler.

Genel itme-yeniden etiketleme algoritması oldukça farklı bir sezgiye sahiptir. Önceki gibi, yönlü ayrıtlar borulara karşılık gelmektedir. Boruların kesişme noktası olan tepeler iki ilginç özelliğe sahiptirler. İlki, aşırı akışı barındırmak için her tepe sıvı biriktirebilen yeterince genişlikte bir rezervuara yönlendirilen bir çıkış-akış borusuna sahiptir. İkincisi, her tepe, rezervuarı ve tüm boru bağlantıları algoritma ilerledikçe yüksekliği artan bir platformun üstünde oturmaktadır.

Tepe yükseklikleri akışın nasıl itileceğini belirler: akışı sadece daha yüksek bir tepeden daha alçak bir tepeye doğru, yokuş aşağı iteriz. Daha alçak bir tepeden daha yüksek bir tepeye olan akış pozitif olabilir, ama akış itme işlemleri sadece yokuş aşağı iterse. Kaynak yüksekliğini $|V|$ ve çıkış düğümü yüksekliğini 0 olarak düzeltiriz. Bütün diğer tepe yükseklikleri 0 dan başlar ve zamanla artarlar. Algoritma önce kaynaktan çıkış düğümüne doğru yokuş aşağı mümkün olduğu kadar çok akış gönderir. Gönderdiği miktar tam olarak kaynaktan kapasiteye her çıkan boruyu doldurmak için yeterlidir; yani, $(s, V - \{s\})$ kesiminin kapasitesini gönderir. Akış ilk kez bir ara tepeye girdiğinde, tepenin rezervuarında birikir. Onu sonuçta oradan yokuş aşağı iteriz.

Sonuçta, sadece bir u tepesinden çıkan ve akış ile doymamış olan boruların, u gibi aynı seviyedeki ya da u dan yukarıdaki tepeler ile bağlantı kurduklarını bulabiliriz. Bu durumda, taşan bir u tepesini aşırı akışından kurtarmak için onun yüksekliğini artırmak zorundayız – u tepesini “yeniden etiketleme” denilen bir işlem. Onun yüksekliğini doymamış bir boruya sahip komşularından en düşük olanının yüksekliğinden bir birim fazla artırırız. Bir tepe yeniden etiketlendikten sonra, böylece, daha fazla akış itebileceğimiz en az bir çıkan boruya sahip olur.

Sonuçta, mümkün olduğunca çıkış düğümüne ulaşabilen bütün akışlar oraya varır. Daha fazla varış olmadığında, borular kapasite kısıtlarına uyduklarından dolayı; herhangi bir kesim boyunca akış miktarı hala kesim kapasitesi ile sınırlıdır. Önakışı bir “yasal(legal)” akış yapmak için algoritma taşan tepelerin rezervuarlarında birikmiş aşırılığı yukarıdaki kaynağın düzeltilmiş $|V|$ yüksekliği için

tepeleri yeniden etiketlemeye devam ederek kaynağa geri yollar. Anlayacağımız gibi, bir kez bütün rezervuarları boşaltınca, önakış sadece bir “yasal” akış değil, ayrıca bir maksimum akıştır.

Temel işlemler

Önceki tartışmadan, bir itme-yeniden etiketleme algoritmasının iki temel işlemi yerine getirdiğini anladık: aşırı akışı bir tepeden bir komşusuna itme ve bir tepeyi yeniden etiketleme. Şimdi açık bir şekilde tanımlayacağımız bu işlemlerin uygulandığı durumlar tepelerin yüksekliklerine bağlıdır.

$G = (V, E)$, s kaynağı ve t çıkış düğümü olan bir akış ağı olsun ve f , G de bir önakış olsun. Eğer $h(s) = |V|$, $h(t) = 0$ ve her $(u, v) \in E_f$ artık(residual) ayrıtı için

$$h(u) \leq h(v) + 1$$

ise $h : V \rightarrow \mathbb{N}$ fonksiyonu bir *yükseklik fonksiyonu (height function)* dur.³ Hemen aşağıdaki lemmayı elde ederiz.

Lemma 26.12

$G = (V, E)$ bir akış ağı olsun, f , G de bir önakış olsun ve h , V de bir yükseklik fonksiyonu olsun. Herhangi bir $u, v \in V$ iki tepesi için eğer $h(u) > h(v) + 1$ ise (u, v) artık ağda bir ayrıtı değildir. ■

İtme işlemi

PUSH(u, v) temel işlemi eğer u taşan bir tepe, $c_f(u, v) > 0$ ve $h(u) = h(v) + 1$ ise uygulanır. Aşağıdaki kabakod u ve v için f önakışını ve aşırı akışları günceller. Verilen c ve f ile $c_f(u, v)$ artık kapasitesini sabit zamanda hesaplayabileceğimizi kabul eder. Bir u tepesinde depolanan aşırı akışı $u.e$ niteliği olarak ve u nun yüksekliğini $u.h$ niteliği olarak sürdürürüz. $\Delta_f(u, v)$ ifadesi u dan v ye itebildiğimiz akış miktarını depolayan geçici değişkendir.

³Literatürde, bir yükseklik fonksiyonuna genellikle bir “uzaklık fonksiyonu” denilmektedir ve bir tepenin yüksekliğine bir “uzaklık etiketi” denilmektedir. “uzaklık” terimini kullanıyoruz çünkü daha çok algoritmanın arkasındaki sezgiyi düşündürmektedir. Bir tepenin yüksekliğini artıran işlemi kastetmek için “yeniden etiketleme” teriminin kullanımını sürdürürüz. Bir tepenin yüksekliği, G^T transpozisinin bir enine aramasında bulunabilen, t çıkış düğümüne olan uzaklığıyla ilişkilidir.

PUSH(u, v)

```

1 // Applies when:  $u$  taşıyor,  $c_f(u, v) > 0$  ve  $u.h = v.h + 1$ .
2 // Action:  $u$  dan  $v$  ye  $\Delta_f(u, v) = \min(u.e, c_f(u, v))$  birim akışı itmek.
3  $\Delta_f(u, v) = \min(u.e, c_f(u, v))$ 
4 if  $(u, v) \in E$ 
5      $(u, v).f = (u, v).f + \Delta_f(u, v)$ 
6 else  $(v, u).f = (v, u).f - \Delta_f(u, v)$ 
7  $u.e = u.e - \Delta_f(u, v)$ 
8  $v.e = v.e + \Delta_f(u, v)$ 

```

PUSH kodu aşağıdaki gibi işlemektedir. u tepesi pozitif bir $u.e$ aşırılığına sahip olduğu ve (u, v) nin artık kapasitesi pozitif olduğu için akışı u dan v ye, $u.e$ nin negatif olmasına ya da $c(u, v)$ kapasitesinin aşmasına neden olmadan, $\Delta_f(u, v) = \min(u.e, c_f(u, v))$ kadar artırabiliriz. 3. satır $\Delta_f(u, v)$ değerini hesaplar, 4-6 satırları f yi günceller. Akışı aynı zamanda orijinal bir ayrıt olan artık ayrıt üzerinden ittiğimizden dolayı 5. satır (u, v) ayrıtındaki akışı artırır. 6. satır (v, u) ayrıtındaki akışı azaltır, çünkü artık ayrıt aslında orijinal ağdaki bir ayrıtın tersidir. Son olarak, 7-8 satırları u ve v tepelerine gelen aşırı akışları günceller. Bu yüzden, eğer PUSH çağırılmadan önce f bir önakış ise sonrasında yine bir önakış olarak kalır.

PUSH kodundaki hiçbir şeyin u ve v nin yüksekliklerine bağlı olamadığını gözlemleyin. Yine de $u.h = v.h + 1$ olmadıkça çağırılmasını yasaklarız. Bu yüzden, aşırı akışı sadece 1 yükseklik farkıyla yokuş aşağı iteriz. Lemma 26.12 den yükseklik farkları 1 den daha fazla olan iki tepe arasında artık ayrıt bulunmamaktadır ve bu yüzden, h niteliği gerçekten bir yükseklik fonksiyonu oldukça, 1 den daha fazla yükseklik farkıyla akışın yokuş aşağı itilmesine izin vererek hiçbir şey elde edemeyiz.

PUSH(u, v) işlemini u dan v ye bir *itme* olarak isimlendiririz. Eğer bir itme işlemi bir u tepesinden çıkan bazı (u, v) ayrıtlarına uygulanırsa, aynı zamanda itme işlemi u ya uygulanır deriz. Eğer artık ağdaki (u, v) ayrıtı *doymuş (saturated)* (sonradan $c_f(u, v) = 0$) olursa o bir *doyuran itme (saturating push)* dir; aksi halde bir *doyurmayan itme (nonsaturating push)* dir. Eğer bir ayrıt doymuş olursa, artık ağdan kaybolur. Basit bir lemma bir doyurmayan itmenin bir sonucunu karakterize etmektedir.

Lemma 26.13

u dan v ye bir doyurmayan itmeden sonra, u tepesi artık taşmış değildir.

Kanıt İtme doyurmayan olduğundan, asıl itilen $\Delta_f(u, v)$ akış miktarı itmeden önceki $u.e$ ye eşit olmalıdır. $u.e$ bu miktar kadar azaltıldığı için itmeden sonra 0 olur. ■

Yeniden etiketleme işlemi

RELABEL(u) temel işlemi eğer u taşmış ise ve tüm $(u, v) \in E_f$ ayrıtları için $u.h \leq v.h$ ise uygulanır. Başka bir deyişle, u dan v ye artık kapasitesi olan her v tepesi için eğer akış u dan v ye itilemez ise taşmış bir u tepesini yeniden etiketleyebiliriz, çünkü v u 'dan aşağıda değildir. (Tanıma göre hatırlayın, ne s kaynağı ne de t çıkış düğümü taşmış olabilir ve bu yüzden s ve t yeniden etiketleme için uygunsuzdur).

RELABEL(u)

- 1 // **Applies when:** u taşıyor ve $(u, v) \in E_f$ olacak şekilde her $v \in V$ için $u.h \leq v.h$ a sahibiz.
- 2 // **Action:** u nun yüksekliğini artır.
- 3 $u.h = 1 + \min \{v.h : (u, v) \in E_f\}$

RELABEL(u) işlemi çağırduğumuzda, u tepesinin **yeniden etiketlendiğini (relabelled)** söyleriz. u yeniden etiketlendiğinde, E_f en az bir tane u dan çıkan ayrıt içermelidir içermelidir, öyle ki koddaki minimizasyon boş olmayan bir küme ile biter. Bu özellik u taşıyor kabulünden anlaşılmaktadır, bir de bize aşağıdakini söyler:

$$u.e = \sum_{v \in V} f(v, u) - \sum_{v \in V} f(u, v) > 0 .$$

Bütün akışlar negatif olmadığı için $(v, u).f > 0$ olan en az bir v tepesine sahip olmalıyız. Fakat $(u, v) \in E_f$ olmasını gerektiren $c_f(u, v) > 0$ olmasıdır. RELABEL(u) işlemi böylece u ya yükseklik fonksiyonlarındaki kısıtların izin verdiği en büyük yüksekliği verir.

Soysal algoritma

Genel itme-yeniden etiketleme algoritması akış ağında başlangıç önakışı oluşturmak için aşağıdaki alt programı kullanır.

INITIALIZE-PREFLOW(G, s)

- 1 **for** her $v \in G.V$ tepesi için
- 2 $v.h = 0$
- 3 $v.e = 0$
- 4 **for** her $(u, v) \in G.E$ tepesi için
- 5 $(u, v).f = 0$
- 6 $s.h = |G.V|$
- 7 **for** her $v \in s.Adj$ tepesi için
- 8 $(s, v).f = c(s, v)$
- 9 $v.e = c(s, v)$
- 10 $s.e = s.e - c(s, v)$

INITIALIZE-PREFLOW bir başlangıç f önakışını

$$(u, v).f = \begin{cases} c(u, v), & u = s \text{ ise,} \\ 0, & \text{aksi halde} \end{cases} \quad (26.15)$$

tanımlayarak oluşturmaktadır.

Yani, s kaynağından ayrılan her ayrıtı kapasitesi ile doldururuz ve diğer tüm ayrıtlar akış taşımazlar. Kaynağa komşu her v tepesi için, ilk olarak $v.e = c(s, v)$ sahibiz ve $s.e$ 'yi bu kapasitelerin toplamının negatifi ile başlatırız. Genel algoritma aynı zamanda

$$u.h = \begin{cases} |V|, & u = s \text{ ise,} \\ 0, & \text{aksi halde} \end{cases} \quad (26.16)$$

ile verilen bir h başlangıç yükseklik fonksiyonu ile başlar.

Eşitlik (26.16) bir yükseklik fonksiyonu tanımlar çünkü sadece $u.h > v.h + 1$ olan (u, v) ayrıtları $u = s$ olanlardır ve bu ayrıtlar doymuşlardır, bu ise artık ağda olmadıkları manasına gelir.

İklendirme belirli bir sırada çalıştırılmayan bir dizi itme ve yeniden etiketleme işlemleri ardından GENERIC-PUSH-RELABEL algoritmasını verir:

GENERIC-PUSH-RELABEL (G)

- 1 INITIALIZE-PREFLOW (G, s)
- 2 **while** uygulanabilir bir itme veya yeniden etiketleme işlemi vardır
- 3 yerine uygulanabilir bir itme veya yeniden etiketleme işlemi seç ve gerçekleştir

Aşağıdaki lemma bir taşan tepe var oldukça, iki temel işlemten en az birinin uygulandığını anlatıyor.

Lemma 26.14 *(Bir taşan tepe itilmiş ya da yeniden etiketlenmiş olabilir)*

$G = (V, E)$, s kaynaklı ve t çıkış düğümlü bir akış ağı olsun, f bir önakış olsun ve h, f için herhangi bir yükseklik fonksiyonu olsun. Eğer u herhangi bir taşan tepe ise ya bir itme ya da bir yeniden etiketleme işlemi uygulanır.

Kanıt Herhangi bir (u, v) artık ayrıtı için $h(u) \leq h(v) + 1$ e sahibiz çünkü h bir yükseklik fonksiyonudur. Eğer bir itme işlemi bir taşmış u tepesine uygulanmıyorsa, tüm (u, v) artık ayrıtları için $h(u) \leq h(v)$ yi işaret eden $h(u) \leq h(v) + 1$ e sahip olmalıyız. Bu yüzden bir yeniden etiketleme işlemi u ya uygulanır. ■

İtme-yeniden etiketleme yönteminin doğruluğu

Genel itme-yeniden etiketleme algoritmasının maksimum-akış problemini çözdüğünü göstermek için, öncelikle algoritma eğer sonlanırsa, f önakışının bir maksimum akış olduğunu kanıtlayacağız. Daha sonra algoritmanın sonlandığını kanıtlayacağız. h yükseklik fonksiyonu hakkındaki bazı gözlemlerle başlıyoruz.

Lemma 26.15 (Tepe yükseklikleri hiçbir zaman azalmaz)

$G = (V, E)$ akış ağı üzerinde GENERIC-PUSH-RELABEL prosedürü yürütülürken, her $u \in V$ tepesi için $u.h$ yüksekliği hiçbir zaman azalmaz. Dahası, ne zaman bir u tepesine yeniden etiketleme işlemi uygulansa, $u.h$ yüksekliği en az 1 kadar artar.

Kanıt Tepe yükseklikleri sadece yeniden etiketleme işlemi sırasında değiştiği için, lemmanın ikinci önermesinin kanıtlanması yeterlidir. Eğer u ters yönde yeniden etiketlendiyse, o zaman $(u, v) \in E_f$ olan tüm v tepeleri için $u.h \leq v.h$ dir. Böylece $u.h < 1 + \min \{v.h : (u, v) \in E_f\}$ olur ve bu yüzden işlem $u.h$ a artmalıdır. ■

Lemma 26.16

$G = (V, E)$, s kaynağı ve t çıkış düğümüne sahip bir akış ağı olsun. Daha sonra G üzerinde GENERIC-PUSH-RELABEL uygulanması yükseklik fonksiyonu olarak h niteliğini sürdürür.

Kanıt Gerçekleştirilen temel işlemlerin sayısı üzerinde tümevarım ile kanıt yapılacaktır. Başlangıç olarak h zaten gözlenmiş olan bir yükseklik fonksiyonudur.

h bir yükseklik fonksiyonu ise o zaman RELABEL(u) işlemi h yükseklik fonksiyonunu bırakır diye iddia ediyoruz. Eğer u dan çıkan $(v, u) \in E_f$ artık ayrıtına bakarsak, RELABEL(u) işlemi $u.h \leq v.h + 1$ olduğundan emin olur. Şimdi u ya giren bir (w, u) artık ayrıtı varsayalım. Lemma 26.15 ile RELABEL(u) işleminden önceki $w.h \leq u.h + 1$, sonrasında $w.h \leq u.h + 1$ olmasını gerektirir. Böylece RELABEL(u) işlemi h yi bir yükseklik fonksiyonunu olarak bırakır.

Şimdi bir PUSH(u, v) işlemi ele alalım. Bu işlem (v, u) ayrıtını E_f ye ekleyebilir ve (u, v) ayrıtını E_f den çıkarabilir. Önceki durumda, $v.h = u.h - 1 < u.h + 1$ sahibiz ve böylece h bir yükseklik fonksiyonu olarak kalır. Sonraki durumda, artık ağdan (u, v) yi kaldırmak karşılık gelen kısıtı kaldırır ve h yeniden bir yükseklik fonksiyonu olarak kalır. ■

Aşağıdaki lemma yükseklik fonksiyonlarının önemli bir özelliğini verir.

Lemma 26.17

$G = (V, E)$, s kaynaklı ve t çıkış düğümlü bir akış ağı olsun, f G de bir önakış olsun ve h , V üzerinde bir yükseklik fonksiyonu olsun. O zaman G_f artık ağında, s kaynağından t çıkış düğümüne bir yol yoktur.

Kanıt Çelişki olsun diye G_f nin s den t ye bir p yolu içerdiğini varsayalım, öyle ki $p = \langle v_0, v_1, \dots, v_k \rangle$, $v_0 = s$ ve $v_k = t$ dir. Genelliği kaybetmeden, p basit bir yoldur ve bu yüzden $k < |V|$ dir. $i = 0, 1, \dots, k - 1$ için, (v_i, v_{i+1}) ayrıtı $\in E_f$ dir. h bir yükseklik fonksiyonu olduğundan, $i = 0, 1, \dots, k - 1$ için $h(v_i) \leq h(v_{i+1}) + 1$ dir. p yolu üzerinde bu eşitsizlikleri birleştirmek, $h(s) \leq h(t) + k$ yı verir. Fakat,

$h(t) = 0$ olduğu için bir yükseklik fonksiyonunda $h(s) = |V|$ gereksinimi ile çelişen $h(s) \leq k < |V|$ elde ederiz. ■

Şimdi, eğer genel itme-yeniden etiketle algoritması sonlanırsa hesapladığı önakışın maksimum bir akış olduğunu göstermeye hazırız.

Teorem 26.18 (Genel itme-yeniden etiketle algoritmasının doğruluğu)

Eğer GENERIC-PUSH-RELABEL algoritması s kaynak ve t çıkış düğümlü bir $G = (V, E)$ akış ağı üzerinde çalışırken sonlanırsa, o zaman hesapladığı f önakışı G için maksimum bir akıştır.

Kanıt Aşağıdaki döngü değişmezini kullanacağız:

GENERIC-PUSH-RELABEL içindeki ikinci satırda **while** döngüsü testinin her yürütülüşünde, f bir önakıştır.

Başlangıç: INITIALIZE-PREFLOW, f ’yi bir önakış yapar.

Sürdürme: Yalnızca 2-3 satırlarındaki **while** döngüsü içindeki işlemler itme ve yeniden etiketledir. Yeniden etiketle işlemleri akış değerlerini değil sadece yükseklik niteliklerini etkiler; bu yüzden f ’nin bir önakış olup olmadığını etkilemezler. Sayfa 739 da tartışıldığı gibi eğer f bir koyma işleminde önceden bir önakış ise, sonrasında da bir önakış olarak kalır.

Sonlandırma: Sonlandırmada, Lemma 26.14 den ve f ’nin her zaman bir önakış olduğu değişmezinden dolayı $V - \{s, t\}$ deki her tepe 0 aşırılığa sahip olmalıdır, taşan tepeler yoktur. Bu yüzden f bir akıştır. Lemma 26.16 sonlandırmada h ’nin bir yükseklik fonksiyonu olduğunu gösterir ve böylece Lemma 26.17 bize G_f ’artık ağında s den t ye bir yol olmadığını söyler. Maks-akış min-kesme teoreminden (Teorem 26.6) f maksimum bir akıştır. ■

İtme-yeniden etiketle metodunun analizi

Genel itme-yeniden etiketle algoritmasının aslında sonladığını göstermek için gerçekleştirdiği işlemlerin sayısını sınırlandırmalıyız. Yeniden etiketleme, doyuran itme ve doyurmayan itme işlemlerinin üç türünün her birini ayrı ayrı sınırlandırırız. Bu sınırların bilgisiyle, $O(V^2 E)$ zamanında çalışan bir algoritma geliştirmek basit bir problemdir. Ancak analize başlamadan önce, önemli bir lemmayı kanıtlıyoruz. Artık ağda kaynağa gelen ayrıtlara izin verdiğimizizi hatırlayalım.

Lemma 26.19

$G = (V, E)$, s kaynaklı ve t çıkış düğümlü bir akış ağı olsun ve f G ’de bir önakış olsun. O zaman taşmış herhangi bir x tepesi için, G_f ’artık ağında x den s ye basit bir yol vardır.

Kanıt Taşmış bir x tepesi için, $U = \{v : G_f \text{ de } x \text{ den } v \text{ ye basit bir yol vardır}\}$ olsun ve çelişki olsun diye $s \notin U$ olduğunu varsayın. $\overline{U} = V - U$ olsun.

$$\begin{aligned}
 \sum_{u \in U} e(u) &= \sum_{u \in U} \left(\sum_{v \in V} f(v, u) - \sum_{v \in V} f(u, v) \right) \\
 &= \sum_{u \in U} \left(\left(\sum_{v \in U} f(v, u) + \sum_{v \in \overline{U}} f(v, u) \right) - \left(\sum_{v \in U} f(u, v) + \sum_{v \in \overline{U}} f(u, v) \right) \right) \\
 &= \sum_{u \in U} \sum_{v \in U} f(v, u) + \sum_{u \in U} \sum_{v \in \overline{U}} f(v, u) - \sum_{u \in U} \sum_{v \in U} f(u, v) - \sum_{u \in U} \sum_{v \in \overline{U}} f(u, v) \\
 &= \sum_{u \in U} \sum_{v \in \overline{U}} f(v, u) - \sum_{u \in U} \sum_{v \in \overline{U}} f(u, v) .
 \end{aligned}$$

elde etmek için; (26.14) eşitliğinden aşırının tanımını aldık, U daki tüm tepeleri topladık, $V = U \cup \overline{U}$ olduğunu dikkate aldık.

$e(x) > 0$, $x \in U$ olduğu, s haricinde tüm tepeler negatif olmayan aşırılığa sahip olduğu için ve $s \notin U$ varsayımıyla, $\sum_{u \in U} e(u)$ değerinin pozitif olması gerektiğini biliyoruz. Bu yüzden,

$$\sum_{u \in U} \sum_{v \in \overline{U}} f(v, u) - \sum_{u \in U} \sum_{v \in \overline{U}} f(u, v) > 0 . \quad (26.17)$$

sahibiz. Bütün ayrıt akışları negatif değildir ve bu nedenle (26.17) eşitliğini elde etmek için $\sum_{u \in U} \sum_{v \in \overline{U}} f(v, u) > 0$ olmalıdır. Bu yüzden $f(v', u') > 0$ olan en az bir $u' \in U$ ve $v' \in \overline{U}$ tepe çifti olmalıdır. Fakat eğer $f(v', u') > 0$ ise, x den v' ne basit bir yol ($x \rightsquigarrow u' \rightarrow v'$) yolu olduğu anlamına gelen, böylece U nun tanımıyla çelişen bir (u', v') artık ayrıtı olmalıdır. ■

Bir sonraki lemma tepelerin yüksekliklerini sınırlar ve bunun sonucu olarak toplamda gerçekleştirilen yeniden etiketleme işlemlerinin sayısını sınırlar.

Lemma 26.20

$G = (V, E)$, s kaynaklı ve t çıkış düğümlü bir akış ağı olsun, G üzerinde GENE-RIC-PUSH-RELABEL in yürütülmesinin herhangi bir anında, her $u \in V$ tepesi için $u.h \leq 2|V| - 1$ olur.

Kanıt s kaynağı ve t çıkış düğümünün yükseklikleri asla değişmez çünkü bu tepeler tanım gereği taşmazlar. Böylece her zaman her ikisi de $2|V| - 1$ den büyük olmayan $s.h = |V|$ ve $t.h = 0$ a sahip olurlar.

Şimdi herhangi bir $u \in V - \{s, t\}$ tepesini ele alalım. Başlangıç olarak $u.h = 0 \leq 2|V| - 1$ dir. Her yeniden etiketleme işleminden sonra, hala $u.h \leq 2|V| - 1$ olduğu-

nu göstereceğiz. u yeniden etiketlendiği zaman bir taşandır ve Lemma 26.19, G_f de u dan s ye basit bir p yolu olduğunu söyler. $v_0 = u, v_k = s$ iken $p = \langle v_0, v_1, \dots, v_k \rangle$ olsun ve p basit olduğundan $k \leq |V| - 1$ dir. $i = 0, 1, \dots, k - 1$ için, $(v_i, v_{i+1}) \in E_f$ dir ve böylece Lemma 26.16 ile $v_i \cdot h \leq v_{i+1} \cdot h + 1$ dir. Bu eşitsizlikleri p yolu üzerine genişletmek $u \cdot h = v_0 \cdot h \leq v_k \cdot h + k \leq s \cdot h + (|V| - 1) = 2|V| - 1$ verir. ■

Sonuç 26.21 (Yeniden etiketle işlemlerinde sınır)

$G = (V, E)$, s kaynaklı ve t çıkış düğümlü bir akış ağı olsun. O zaman G üzerinde GENERIC-PUSH-RELABEL yürütümü boyunca yeniden etiketleme işlemlerinin sayısı her bir tepe için en çok $2|V| - 1$ ve toplamda en çok $(2|V| - 1)(|V| - 2) < 2|V|^2$ dir.

Kanıt $V - \{s, t\}$ de sadece $|V| - 2$ tepe yeniden etiketlenebilir. $u \in V - \{s, t\}$ olsun. RELABEL(u) işlemi $u \cdot h$ yi artırır. $u \cdot h$ nin başlangıç değeri 0 dır ve Lemma 26.20 den en fazla $2|V| - 1$ e artar. Böylece her $u \in V - \{s, t\}$ tepesi en çok $2|V| - 1$ zamanda etiketlenir ve gerçekleştirilen yeniden etiketleme işlemlerinin toplam sayısı en çok $(2|V| - 1)(|V| - 2) < 2|V|^2$ dir. ■

Lemma 26.20 ayrıca doyuran itmelerin sayısını sınırlandırmamıza yardımcı eder.

Lemma 26.22 (Doyuran itmelerde sınır)

Herhangi bir $G = (V, E)$ akış ağında GENERIC-PUSH-RELABEL yürütümü boyunca doyuran itmelerin sayısı $2|V||E|$ den azdır.

Kanıt Herhangi $u, v \in V$ tepe çifti için u dan v ye ve v den u ya doyuran itmeleri birlikte sayacağız, onları u ile v arasında doyuran itmeler olarak adlandırıyoruz. Eğer herhangi böyle itmeler varsa, (u, v) ve (v, u) nun en az biri aslında E de bir ayrıttır. Şimdi, u dan v ye bir doyuran itme meydana geldiğini kabul edelim. O zaman $v \cdot h = u \cdot h - 1$ dir. u dan v ye daha sonra bir başka itmenin gerçekleşmesi için, algoritma $v \cdot h = u \cdot h + 1$ olana kadar önce v den u ya akış itmelidir. $u \cdot h$ hiç azalmadığından, $v \cdot h = u \cdot h + 1$ olması için, $v \cdot h$ değeri en az 2 arttırılmalıdır. Benzer şekilde, v den u ya doyuran itmeler arasında $u \cdot h$ değeri en az 2 arttırılmalıdır. Yükseklikler 0 dan başlar ve Lemma 26.20 den $2|V| - 1$ i asla geçmez bu da yüksekliği 2 arttırılmış herhangi bir tepenin arttırılma sayısının $|V|$ den az olduğunu ima eder. u ile v arasındaki herhangi iki doyuran itme arasında $u \cdot h$ ve $v \cdot h$ nin en az biri 2 arttırılması gerektiği için, u ile v arasında $2|V|$ den daha az doyuran itme vardır. Ayrıtların sayısını çarpmak toplam doyuran itme sayısı üzerinde $2|V||E|$ den daha az bir sınır verir. ■

Aşağıdaki lemma genel itme-yeniden etiketle algoritmasında doyurmayan itmelerin sayısını sınırlar.

Lemma 26.23 (Doyurmayan itmelerde sınır)

Herhangi bir $G = (V, E)$ akış ağında GENERIC-PUSH-RELABEL yürütümü boyunca doyurmayan itmelerin sayısı $4|V|^2 (|V| + |E|)$ den azdır.

Kanıt Potansiyel bir $\Phi = \sum_{v:e(v)>0} v \cdot h$ fonksiyonu tanımlayın. Başlangıç olarak $\Phi = 0$ ve her yeniden etiketleme, doyuran itme ve doyurmayan itme sonrasında Φ nin değeri değişebilir. Doyuran itmelerin ve yeniden etiketlemelerin Φ nin artmasına katkı yapabileceği miktarı sınırlandıracağız. Sonra her doyurmayan itmenin Φ yi en az 1 azaltması gerektiğini göstereceğiz ve doyurmayan itmelerin sayısında bir üst sınır elde etmek için bu sınırları kullanacağız.

Φ nin artabileceği iki yolu inceleyelim. Birincisi, toplamı alınan küme aynı olduğu için ve yeniden etiketleme u nun yüksekliğini mümkün olan maksimum yüksekliğinden, Lemma 26.20 ye göre $2|V| - 1$, daha fazla arttıramayacağı için, bir u tepesini yeniden etiketlemek Φ 'yi $2|V|$ den daha az artırır. İkincisi, yükseklikler değişmediğinden ve yalnızca yüksekliği en çok $2|V| - 1$ olan v tepesi muhtemelen taşan olabileceğinden dolayı, bir u tepesinden bir v tepesine bir doyuran itme Φ yi $2|V|$ den daha az artırır.

Şimdi u dan v ye bir doyurmayan itmenin Φ yi en az 1 azalttığını göstereceğiz. Neden? Doyurmayan itmeden önce, u taşan idi ve v taşan olmuş olabilir veya olmayabilir. Lemma 26.13 den, u itmeden sonra artık taşan değildir. Ayrıca v kaynak değilse, itme sonrasında taşan olabilir veya olmayabilir. Bu yüzden potansiyel Φ fonksiyonu tam olarak $u \cdot h$ kadar azalmıştır hem de ya 0 ya da $v \cdot h$ kadar artmıştır. $u \cdot h - v \cdot h = 1$ olduğu için, potansiyel fonksiyonun en az 1 azalmış olması net etkidir.

Böylece algoritma yürütme süresince, Φ deki toplam artış miktarı yeniden etiketlemeler ve doymuş itmelere bağlıdır ve Sonuç 26.21 ile Lemma 26.22 artışın $(2|V|)(2|V|^2) + (2|V|)(2|V||E|) = 4|V|^2 (|V| + |E|)$ den daha az olmasını engeller. $\Phi \geq 0$ olduğu için, azalışın toplam miktarı ve bu yüzden doyurmayan itmelerin toplam sayısı $4|V|^2 (|V| + |E|)$ den azdır. ■

Yeniden etiketlemeler, doyuran itmeler ve doyurmayan itmelerin sayılarının sınırlı olması ve itme-yeniden etiketle metodu tabanlı her hangi bir algoritmadan dolayı GENERIC-PUSH-RELABEL işleminin aşağıdaki analizi için zemin hazırladık.

Teorem 26.24

Herhangi bir $G = (V, E)$ akış ağında GENERIC-PUSH-RELABEL yürütümü boyunca temel işlemlerin sayısı $O(V^2 E)$ dir.

Kanıt Sonuç 26.21 ve Lemma 26.22 ile 26.23 den aşıkardır. ■

Bu yüzden, algoritma $O(V^2 E)$ işleminden sonra sonlanır. Her bir işlemin gerçekleştirimi için ve çalıştırmaya uygun bir işlem seçmek için verimli bir metot vermek bütün bunlardan arta kalandır.

Sonuç 26.25

Herhangi bir $G = (V, E)$ akış ağında $O(V^2 E)$ zamanda çalışan genel itme-yeniden etiketle algoritmasının bir gerçekleştirimi vardır.

Kanıt Alıştırma 26.4-2 genel algoritmasının her yeniden etiketleme işlemi için $O(V)$ ve her itme için $O(1)$ ek yük ile nasıl gerçekleştirileceğini göstermenizi ister. Ayrıca $O(1)$ zamanda uygulanabilir bir işlem seçmenize izin veren bir veri yapısı tasarlamanızı ister. Sonuç o zaman ortaya çıkar.

Alıştırmalar

26.4-1

INITIALIZE-PREFLOW(G, s) işleminin sonlanmasından sonra, G için f^* maksimum bir akış olduğunda $s.e \leq -|f^*|$ a sahip olduğumuzu kanıtlayın.

26.4-2

Her yeniden etiketleme işlemi için $O(V)$ zaman, her itme için $O(1)$ zaman ve uygulanabilir bir işlem seçmek için $O(1)$ zaman, toplamda ise $O(V^2 E)$ zaman kullanarak genel itme-yeniden etiketle algoritmasının nasıl gerçekleştirileceğini gösterin.

26.4-3

Genel itme-yeniden etiketle algoritmasının tüm $O(V^2)$ yeniden etiketle işlemlerinin yapılmasında toplamda sadece $O(VE)$ zaman harcadığını kanıtlayın.

26.4-4

Bir itme-yeniden etiketle algoritması kullanarak bir $G = (V, E)$ akış ağında maksimum bir akış bulduğumuzu kabul edin. G de minimum bir kesme bulmak için hızlı bir algoritma yazın.

26.4-5

İki parçalı bir çizgede maksimum eşleme bulmak için verimli bir itme-yeniden etiketle algoritması yazın. Algoritmanızı analiz edin.

26.4-6

Bir $G = (V, E)$ akış ağında tüm ayrıt kapasitelerinin $\{1, 2, \dots, k\}$ kümesinde olduğunu kabul edin. Genel itme-yeniden etiketle algoritmasının çalışma zamanını $|V|$, $|E|$ ve k cinsinden analiz edin. (*İpucu:* Her ayrıt doyurmayan bir itmeyi doymuş hale gelmeden önce kaç defa destekler?)

26.4-7

INITIALIZE-PREFLOW un 6. satırını, genel itme-yeniden etiketle algoritmasının doğruluğu veya asimptotik performansını etkilemeden

$$6 \quad s.h = |G.V| - 2$$

şeklinde değiştirebileceğimizi gösterin.

26.4-8

G_f artık ağında u dan v ye uzaklık (ayrıntıların sayısı) $\delta_f(u, v)$ olsun. GENERIC-PUSH-RELABEL işleminin şu özellikleri sürdürdüğünü gösterin: $u.h < |V|$, $u.h \leq \delta_f(u, t)$ olmasını belirtir ve $u.h \geq |V|$, $u.h - |V| \leq \delta_f(u, s)$ olmasını belirtir.

26.4-9 *

Önceki alıştırmadaki gibi, G_f artık ağında u dan v ye uzaklık $\delta_f(u, v)$ olsun. Genel itme-yeniden etiketle algoritmasının şu özelliği sürdürmek için nasıl değiştirileceğini gösterin: $u.h < |V|$, $u.h = \delta_f(u, t)$ olmasını belirtir ve $u.h \geq |V|$, $u.h - |V| = \delta_f(u, s)$ olmasını belirtir. Gerçekleştirmenizin bu özelliği sürdürmek için adadığı toplam zaman $O(VE)$ olmalıdır.

26.4-10

Bir $G = (V, E)$ akış ağında GENERIC-PUSH-RELABEL işlemi ile yapılan doyurmayan itmelerin sayısının $|V| \geq 4$ için en çok $4|V|^2 |E|$ olduğunu gösteriniz.

★ 26.5 Öne-yeniden-etiketle algoritması

İtme-yeniden etiketle metodu hiçbir sıra olmaksızın temel işlemleri uygulamamıza imkan verir. Ancak sırayı dikkatlice seçerek ve ağ veri yapısını verimli yöneterek maksimum akış problemini Sonuç 26.25 ile verilen $O(V^2 E)$ sınırından daha hızlı çözebiliriz. Şimdi öne-yeniden etiketle algoritmasını, asimptotik olarak en az $O(V^2 E)$ kadar iyi ve hatta yoğun ağlar için daha iyi, çalışma zamanı $O(V^3)$ olan bir itme-yeniden etiketle algoritmasını inceleyeceğiz.

Öne-yeniden etiketle algoritması ağdaki tepelerin bir listesini yönetir. Önden başlayarak algoritma listeyi tarar, taşan u tepesini tekrar tekrar seçerek ve sonra onu boşaltarak, yani u pozitif aşırılığa sahip olmayana kadar itme-yeniden etiketle işlemlerini yürütür. Ne zaman bir tepeyi yeniden etiketlesek, onu listenin başına taşırız (bu yüzden isim “öne-yeniden etiketle”) ve algoritma onu yeniden taramaya başlar.

Öne-yeniden etiketle algoritmasının doğruluğu ve analizi “kabul edilebilir” ayrıtlar kavramına bağlıdır: bir artık ağdaki bu ayrıtlar akışın itilebildiği ayrıtlardır. Kabul edilebilir ayrıtların ağı ile ilgili bazı özelliklerin kanıtlanmasından sonra boşaltma işlemini inceleyeceğiz ve daha sonra öne-yeniden etiketle algoritmasının kendisini tanıtıp analiz edeceğiz.

Kabul edilebilir ayrıtlar ve ağlar

Eğer $G = (V, E)$, s kaynaklı ve t çıkış düğümlü bir akış ağı, fG de bir önakış ve h bir yükseklik fonksiyonu ise o zaman $c_f(u, v) > 0$ ve $h(u) = h(v) + 1$ iken (u, v) bir **kabul edilebilir ayrıttır** deriz. Aksi halde, (u, v) **kabul edilemezdir**. $E_{(f,h)}$ kabul edilebilir ayrıtların bir kümesi iken, **kabul edilebilir ağ** $G_{(f,h)} = (V, E_{(f,h)})$ dir.

Kabul edilebilir ağ akış itebileceğimiz ayrıtları içerir. Aşağıdaki lemma bu ağın yönlü çevrimsiz bir çizge olduğunu gösterir.

Lemma 26.26 (Kabul edilebilir ağ çevrimsizdir)

Eğer $G = (V, E)$ bir akış ağı, fG de bir önakış ve h G üzerinde bir yükseklik fonksiyonu ise o zaman kabul edilebilir $G_{(f,h)} = (V, E_{(f,h)})$ ağı çevrimsizdir.

Kanıt Kanıt çelişki ile yapılır. $v_0 = v_k$ ve $k > 0$ iken $G_{(f,h)}$ nin bir $p = \langle v_0, v_1, \dots, v_k \rangle$ çevrimi içerdiğini kabul edelim. p deki her ayrıt kabul edilebilir olduğundan dolayı $i = 1, 2, \dots, k$ için $h(v_{i-1}) = h(v_i) + 1$ dir. Çevrim toplamı

$$\begin{aligned} \sum_{i=1}^k h(v_{i-1}) &= \sum_{i=1}^k (h(v_i) + 1) \\ &= \sum_{i=1}^k h(v_i) + k \end{aligned}$$

olur. p çevrimindeki her tepe, toplamada bir kere görüldüğü için $0 = k$ çelişkisini elde ederiz. ■

Aşağıdaki iki lemma itme ve yeniden etiketleme işlemlerinin kabul edilebilir ağı nasıl değiştirdiğini göstermektedir.

Lemma 26.27

$G = (V, E)$ bir akış ağı olsun, fG de bir önakış olsun ve h niteliğinin bir yükseklik fonksiyonu olduğunu kabul edelim. Eğer bir u tepesi taşmaysa ve (u, v) bir kabul edilebilir ayrıt ise o zaman PUSH (u, v) uygulanır. İşlem yeni bir kabul edilebilir ayrıt üretmez, fakat (u, v) nin kabul edilemez olmasına sebep olabilir.

Kanıt Kabul edilebilir ayrıtın tanımından akışı u dan v ye itebiliriz. u taşan olduğu için $PUSH(u, v)$ işlemi uygulanır. Akışı u dan v ye itmenin yaratabileceği tek artık ayrıt, (v, u) dur. $v.h = u.h - 1$ olduğu için (v, u) ayrıtı kabul edilebilir olamaz. Eğer işlem doyuran bir itme ise, sonradan $c_f(u, v) = 0$ olur ve (u, v) kabul edilemez hale gelir. ■

Lemma 26.28

$G = (V, E)$ bir akış ağı olsun, f de G de bir ön-akış olsun ve h niteliğinin bir yükseklik fonksiyonu olduğunu varsayalım. Eğer bir u tepesi taşan ise ve u ya terk eden kabul edilebilir ayrıtlar yoksa, o zaman $RELABEL(u)$ uygulanır. Yeniden etiketleme işleminden sonra, u dan ayrılan en az bir kabul edilebilir ayrıt vardır, ancak u ya giren kabul edilebilir ayrıtlar yoktur.

Kanıt Eğer u taşan ise, o zaman Lemma 26.14 e göre ona ya bir itme ya da bir yeniden etiketleme işlemi uygulanır. Eğer u' an ayrılan kabul edilebilir ayrıtlar yoksa, o zaman u dan akış itilemez ve dolayısıyla $RELABEL(u)$ uygulanır. Yeniden etiketleme işleminden sonra, $u.h = 1 + \min\{v.h : (u, v) \in E_f\}$ olur. Böylece, eğer v bu kümedeki minimumu gerçekleştiren bir tepeyse, (u, v) ayrıtı kabul edilebilir olur. Bundan dolayı, yeniden etiketlemeden sonra, u dan ayrılan en az bir kabul edilebilir ayrıt vardır.

Bir yeniden etiketleme işleminden sonra u ya giren kabul edilebilir hiçbir ayrıt olmadığını göstermek için, bir v tepesi olduğunu varsayalım, öyle ki (v, u) kabul edilebilir olsun. O halde, yeniden etiketlemeden sonra $v.h = u.h + 1$ olur ve dolayısıyla yeniden etiketlemeden hemen önce $v.h > u.h + 1$ olur. Ancak Lemma 26.12 den, yükseklikleri farkı 1 den fazla olan tepeler arasında artık ayrıtlar olamaz. Üstelik, bir tepayı yeniden etiketlemek artık ağı değiştirmez. Bu nedenle (v, u) artık ağda değildir ve böylece kabul edilebilir ağda olamaz. ■

Komşu listeleri

Öne-yeniden-etiketleme algoritmasındaki ayrıtlar “komşu listeleri” içine organize edilirler. Verilen bir $G = (V, E)$ akış ağında, bir $u \in V$ tepesi için $u.N$ **komşu listesi** G de u nun komşularının tek bağlı bir listesidir. Böylece v tepesi, eğer $(u, v) \in E$ veya $(v, u) \in E$ ise $u.N$ listesinde görünür. $u.N$ komşu listesi (u, v) artık ayrıtları için kesinlikle v tepelerini içermektedir. $u.N.head$ niteliği $u.N$ deki ilk tepeye işaret eder ve $v.next-neighbor$ bir komşu listesindeki v yi izleyen tepeye işaret eder; eğer v komşu listesindeki son tepeyse bu işaretçi NIL dir.

Öne-yeniden-etiketleme algoritması, her komşu listesi boyunca algoritmanın yürütülmesi süresince sabitlenen keyfi bir düzende döner. Her u tepesi için $u.current$ niteliği $u.N$ de o anda değerlendirme altındaki tepeye işaret eder. Başlangıçta, $u.current$, $u.N.head$ e ayarlanır.

Taşan bir tepeyi boşaltmak

Taşan bir u tepesi tüm aşırı akışını komşu tepelere olan kabul edilebilir ayrıtlar üzerinden iterek ve u 'dan çıkan ayrıtların kabul edilebilir hale gelmesi için u yu yeniden etiketleyerek **boşaltır** (*discharged*). Kabakod, aşağıdaki gibidir.

DISCHARGE(u)

```

1  while  $u.e > 0$ 
2       $v = u.current$ 
3      if  $v == \text{NIL}$ 
4          RELABEL( $u$ )
5           $u.current = u.N.head$ 
6      elseif  $c_f(u, v) > 0$  and  $u.h == v.h + 1$ 
7          PUSH( $u, v$ )
8      else  $u.current = v.next-neighbor$ 
```

Şekil 26.9, u tepesi pozitif aşırılığa sahip olduğu sürece çalışan 1-8 satırlarındaki **while** döngüsünün birkaç iterasyonunu adım adım gösterir. Her bir iterasyon tam olarak üç eylemden birini, $u.N$ komşu listesinde o anki v tepesine bağlı olarak yürütür.

1. Eğer v NIL se, o zaman $u.N$ nin sonunu geçiririz. Satır 4 u tepesini yeniden etiketler ve sonra satır 5 u nun mevcut komşusunu, $u.N$ de birinci olması için resetler (Aşağıdaki Lemma 26.29 bu durumda yeniden etiketleme işleminin uygulanmasını belirtir).
2. Eğer v NIL-olmayan ve (u, v) kabul edilebilir bir ayrıtsa (satır 6 daki test tarafından belirlenmiş), o halde satır 7 u nun bazı (ya da muhtemelen tüm) aşırılığını tepe v ye ekler.
3. Eğer v NIL-olmayan, ancak (u, v) kabul edilemezse, o zaman satır 8 $u.current$ 'i $u.N$ komşu listesinde bir konum ilerletir.

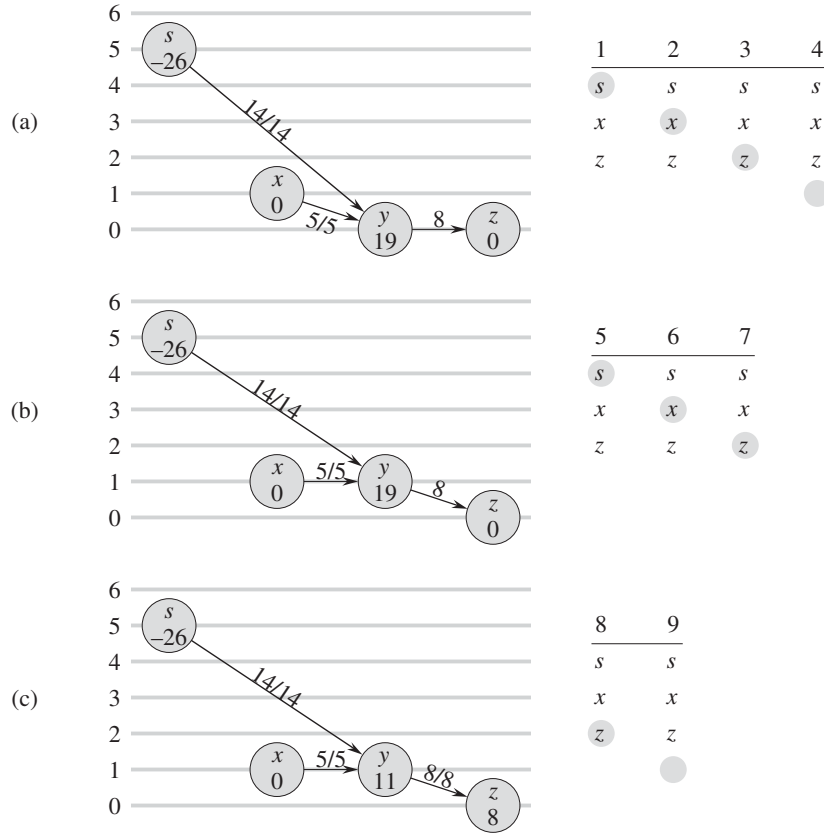
Taşan bir u tepesi üzerinde DISCHARGE çağırıldığında DISCHARGE tarafından yürütülen son eylemin u dan bir itme olması gerektiğini gözlemleyin. Neden? Prosedür sadece $u.e$ sıfır olduğunda biter ve ne yeniden etiketleme işlemi ne de $u.current$ işaretçisini ilerletmek $u.e$ nin değerini etkiler.

DISCHARGE tarafından PUSH ya da RELABEL çağırıldığında, işlemin uygulanmışından emin olmalıyız. Bir sonraki lemma bu gerçeği kanıtlar.

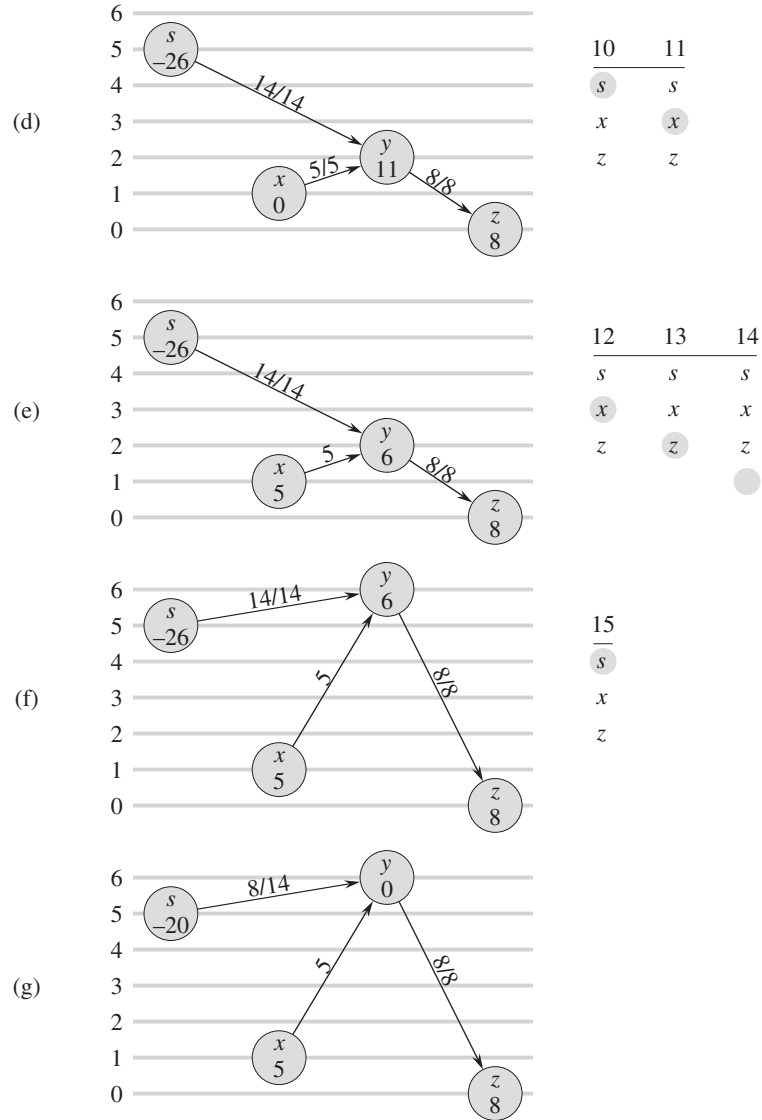
Lemma 26.29

Eğer DISCHARGE satır 7 deki PUSH(u, v) yi çağırırsa, (u, v) ye bir itme işlemi uygulanır. Eğer DISCHARGE satır 4 teki RELABEL(u) yu çağırırsa, u ya bir yeniden etiketleme işlemi uygulanır.

Kanıt Satır 1 ve 6 daki testler, bir itme işleminin sadece işlem uygulandığında gerçekleştiğini garanti ederler bu da lemmadaki birinci ifadeyi kanıtlar.



Şekil 26.9 Bir y tepesinin boşaltılması. Tüm aşırı akışı y den itmek için DISCHARGE in **while** döngüsü 15 iterasyon sürer. Sadece y ye giren ya da ondan ayrılan akış ağının ayrıtları ve y 'nin komşuları gösterilir. Şeklin her parçasında, her tepenin içindeki sayı o parçada gösterilen ilk iterasyonunun başlangıcındaki aşırılığıdır ve her tepe parça boyunca yüksekliğinde gösterilir. Her iterasyonun başlangıcındaki komşu listesi $y.N$ sağda üstte iterasyon sayısı ile gösterilir. Gölgelemiş komşu, $y.current$ dir. (a) başlangıçta, y den ekleyecek itmek için 19 birim aşırılık vardır ve $y.current = 8$ dir. 1., 2. ve 3. iterasyonlar y den çıkan kabul edilebilir ayrıtlar olmadığı için sadece $y.current$ 1 ilerletir. 4. iterasyonda $y.current = NIL$ dir (komşu listesinin altındaki gölgelemeler tarafından gösterilen) ve dolayısıyla y yeniden etiketlenir ve $y.current$ komşu listesinin başı olarak resetlenir. (b) Yeniden etiketlemeden sonra, y tepesi 1 yüksekliğine sahiptir. 5. ve 6. iterasyonlarda (y, s) ve (y, x) ayrıtları kabul edilemez bulunur, fakat 7. iterasyon y den z ye 8 birim aşırı akış iter. İtmeden dolayı $y.current$ bu iterasyonda ilerlemez. (c) 7. iterasyondaki itme (y, z) ayrıtlarını doyurduğu için 8. iterasyonda kabul edilemez bulunur. 9. iterasyonda $y.current = NIL$ dir ve bu yüzden y tepesi tekrar etiketlenir ve $y.current$ resetlenir.



Şekil 26.9. devamı (d) 10 iterasyonda (y, s) kabul edilebilirdi fakat 11. iterasyon y den x e 5 birim aşırı akış iter. (e) $y.current$ 11. iterasyonda ilerlemediğinden, 12. iterasyon (y, x) in kabul edilemez olduğunu bulur. 13. iterasyon (y, z) nin kabul edilemez olduğunu bulur ve 14. iterasyon y tepesini yeniden etiketlendirir ve $y.current$ i resetler. (f) 15. iterasyon 6 birim aşırı akışı y den s ye iter. (g) y tepesinin şimdi aşırı akışı yoktur ve DISCHARGE sona erer. Bu örnekte DISCHARGE komşu listesinin başındaki mevcut işaretçi ile hem başlar hem de biter ancak genelde bu durum böyle değildir.

İkinci ifadeyi kanıtlamak için, satır 1 deki test ve Lemma 26.28 e göre, sadece u dan çıkan tüm ayrıtların kabul edilemez olduklarını göstermemiz gerekiyor. Eğer $\text{DISCHARGE}(u)$ ya bir çağrı u nun komşu listesinin başındaki $u.\text{current}$ işaretçisi ile başlıyor ve listenin sonunda onunla son buluyorsa, o halde u dan çıkan tüm ayrıtlar kabul edilemez ayrıtlardır ve bir yeniden etiketleme işlemi uygulanır. Ne var ki, bir $\text{DISCHARGE}(u)$ çağrısı boyunca, $u.\text{current}$ işaretçisinin sadece prosedür geri dönmenden önceki listenin parçasını dolaşması olasıdır. O halde diğer tepelerdeki DISCHARGE çağrıları gerçekleşebilir, ancak $u.\text{current}$ $\text{DISCHARGE}(u)$ ya bir sonraki çağrı süresince liste boyunca ilerlemeye devam edecektir. Şimdi listenin tümü geçilirken yani $u.N$ in başında başlayıp $u.\text{current} = \text{NIL}$ ile biterken ne olduğunu ele alacağız. $U.\text{current}$ bir kez listenin sonuna ulaştı mı, prosedür u yu yeniden etiketlendirir ve yeni bir geçiş başlar. $u.\text{current}$ işaretçisinin, geçiş süresince $v \in u.N$ tepesini geçmeye ilerler, (u, v) ayrıtı satır 6 daki test tarafından kabul edilemez hale getirilmek zorundadır. Böylece, zamanla geçiş tamamlanır, u dan ayrılan her ayrıt geçiş süresinde bazı noktalarda kabul edilemez olarak belirlenir. Temel gözlem, geçişin sonundadır, u dan ayrılan her ayrıt hala kabul edilemezdir. Neden? Lemma 26.27 ye göre, akışın hangi tepeden itildiğine bakmadan, itmeler herhangi bir kabul edilebilir ayrıt yaratamazlar. Böylece, herhangi bir kabul edilebilir ayrıt bir yeniden etiketleme işlemi tarafından yaratılmak zorundadır. Ancak tepe u , geçiş boyunca yeniden etiketlenmez ve Lemma 26.28 e göre, geçiş süresince yeniden etiketlenen başka herhangi bir v tepesi (bir $\text{DISCHARGE}(v)$ çağrısından sonuçlanan) yeniden etiketlemeden sonra kabul edilebilir gelen ayrıtlara sahip değildir. Böylece, geçişin sonunda, u dan ayrılan tüm ayrıtlar kabul edilemez kalır ve bu da kanıtı tamamlar. ■

Öne-yeniden etiketleme algoritması

Öne-yeniden etiketleme algoritmasında, $V - \{s, t\}$ deki tüm tepeleri içeren bir bağlı L listesini sürdürürüz. Aşağıdaki döngü değişmezinde göreceğimiz gibi, L deki tepelerin kabul edilebilir ağa göre topolojik sıralı olması temel bir özelliktir. (Kabul edilebilir ağın bir yönlü çevrimsiz çizge olduğunu Lemma 26.26 dan hatırlayalım).

Öne-yeniden etiketleme algoritması için kabakod, $u.N$ komşu listelerinin her u tepesi için oluşturulmuş olduğunu kabul eder. Ayrıca $u.\text{next}$ in L listesindeki u yu takip eden tepeye işaret ettiğini kabul eder ve her zamanki gibi eğer u listedeki son tepeyse $u.\text{next} = \text{NIL}$ olduğunu varsayar.

```

RELABEL-TO-FRONT( $G, s, t$ )
1  INITIALIZE-PREFLOW( $G, s$ )
2   $L = G.V - \{s, t\}$ , herhangi bir sırada
3  her  $u \in G.V \setminus \{s, t\}$  tepesi
4       $u.current = u.N.head$ 
5   $u = L.head$ 
6  while  $u \neq \text{NIL}$ 
7       $old-height = u.h$ 
8      DISCHARGE( $u$ )
9      if  $u.h > old-height$ 
10          $u$  yu  $L$  listesinin ön tarafına taşı
11      $u = u.next$ 

```

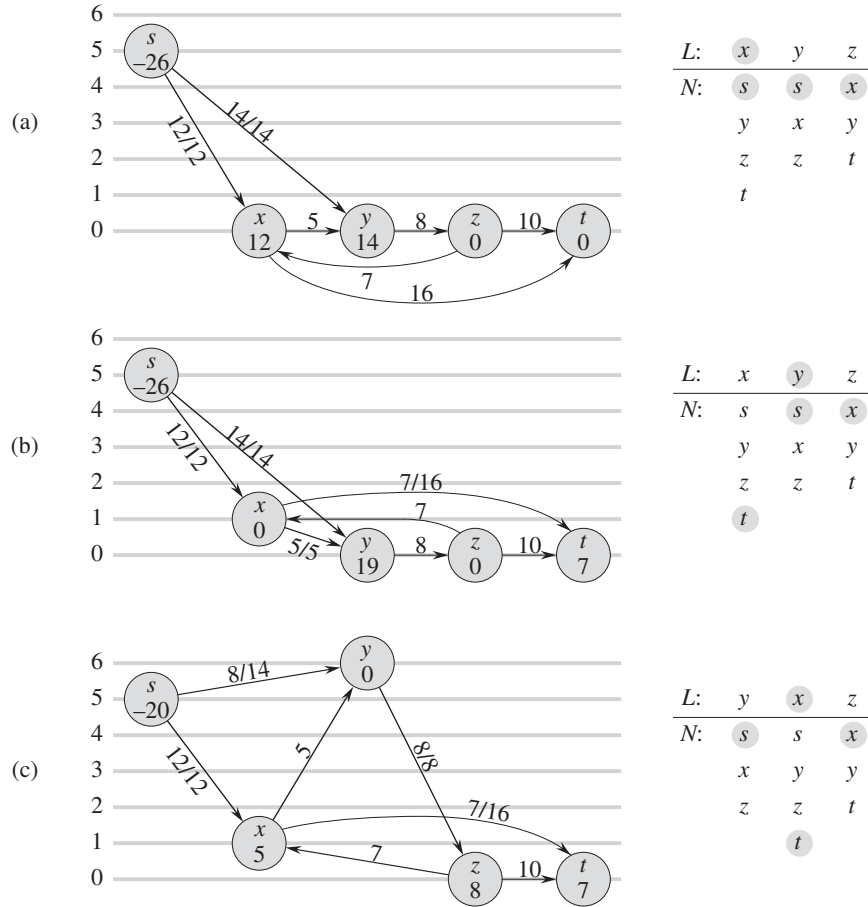
Öne-yeniden etiketleme algoritması şu şekilde çalışır. Satır 1 ön-akışı ve yükseklikleri genel itme-yeniden etiketleme algoritmasındaki aynı değerlerle başlatır. Satır 2 L listesini tüm potansiyel aşan tepeleri, herhangi bir sırada içermesi için başlatır. Satır 3-4, her u tepesinin current işaretçisini u nun komşu listesindeki ilk tepesi ile başlatır.

Şekil 26.10 un yansıttığı gibi, 6-11 satırlarının **while** döngüsü L listesi boyunca işlerken, tepeleri boşaltır. Satır 5 listedeki ilk tepe ile başlatır. Döngünün her iterasyonunda satır 8 bir u tepesini boşaltır. Eğer u DISCHARGE prosedürü tarafından yeniden etiketlenirse, satır 10 onu L listesinin önüne taşır. u nun yeniden etiketlenip etiketlenmediğini boşaltma işleminden önce satır 7 de $old-height$ değişkeninde saklanan yüksekliği ile hemen sonrasında satır 9 daki yüksekliğini kıyaslayarak belirleyebiliriz. Satır 11 L listesindeki u yu takip eden tepeyi kullanarak **while** döngüsünün sonraki iterasyonunu üretir. Eğer satır 10 u yu listenin önüne taşırsa, bir sonraki iterasyonda kullanılan tepe listedeki yeni konumunda u dan bir sonradır.

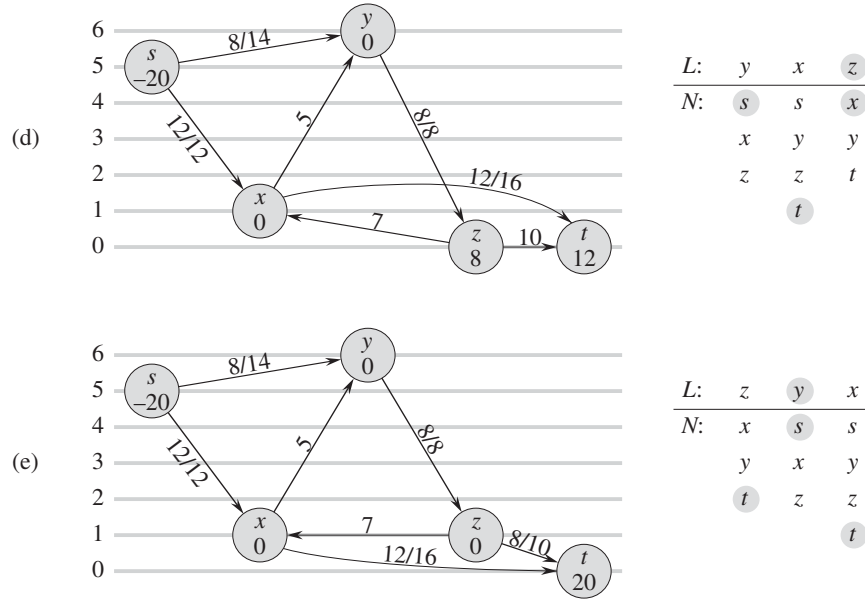
RELABEL-TO-FRONT un maksimum bir akış hesapladığını göstermek için, onun genel itme-yeniden etiketleme algoritmasının bir gerçekleştirimi olduğunu göstereceğiz. Önce, onun itme ve yeniden etiketleme işlemlerini sadece uygulandıklarında yürüttüğünü gözlemleyin. Çünkü Lemma 26.29, DISCHARGE ın sadece uygulandıklarında onları yürüttüğünü garanti eder. Geriye RELABEL-TO-FRONT sonlandığında hiçbir temel işlemin uygulanmadığını göstermek kalıyor. Doğruluk argümanının geri kalanı, aşağıdaki döngü değişmezine dayanır:

RELABEL-TO-FRONT un satır 6 daki her testinde, L listesi kabul edilebilir $G_{f,h} = (V, E_{f,h})$ ağındaki tepelerin topolojik bir sıralamasıdır ve listede u dan önceki hiçbir tepenin aşırı akışı yoktur.

Başlangıç: INITIALIZE-PREFLOW çalıştırıldıktan hemen sonra, $s.h = |V|$ ve tüm $v \in V - \{s\}$ için $v.h = 0$ olur. $|V| \geq 2$ olduğu için (çünkü V en az s ve t yi içerir)



Şekil 26.10 RELABEL-TO-FRONT icraatı. **(a)** **While** döngünün ilk iterasyonundan hemen önceki bir akış ağı. Başlangıçta, 26 birimlik akış kaynak s den ayrılır. Sağda, başlangıç listesi $L = \{x, y, z\}$ gösterilir, burada başlangıçta $u = x$ tir. L listesindeki her tepenin altı onun komşu listesidir ve mevcut komşu gölgelenmiştir. Tepe x boşaltılmıştır. 1 yüksekliğine yeniden etiketlenir, 5 birim aşırı akış y ye itilir ve kalan 7 birim aşırılık t çıkış düğümüne itilir. x yeniden etiketlenildiği için L nin başına taşınır ki bu durum L nin yapısını değiştirmez. **(b)** x ten sonra, L de boşaltılan bir sonraki tepe y dir. Şekil 26.9 bu durumdaki y yi boşaltmanın detaylı eylemini gösterir. y yeniden isimlendirildiğinden; L nin başına taşınır. **(c)** x tepesi şimdi L de y den sonradır bu yüzden 5 birim aşırı akışının tümünü t ye iterek yeniden boşaltılır. x tepesi bu boşaltım işleminde yeniden etiketlenmediği için L listesinde yerinde kalır.



Şekil 26.10 devamı (d) z tepesi L de x tepesinden sonra olduğu için boşaltılır. Yükseklik 1 e yeniden etiketlenir ve 8 birim aşırı akışının tümü t ye itilir. z yeniden etiketlendiği için L nin başına taşınır. (e) y tepesi L de z tepesinden sonra gelir ve bu yüzden boşaltılır. Ancak y aşırılığa sahip değildir, DISCHARGE hemen döner ve y L de yerinde kalır. Daha sonra x tepesi boşaltılır. O da aşırılığa sahip değildir, DISCHARGE gene döner ve x L de yerinde kalır. RELABEL-TO-FRONT L listesinin sonuna ulaşmıştır ve sonlanır. Taşan tepeler yoktur ve ön-akış maksimum bir akıştır.

hiçbir ayrıt kabul edilir olamaz. Böylece, $E_{f,h} = \emptyset$ ve $V - \{s, t\}$ nin herhangi bir sırası $G_{f,h}$ nin topolojik bir sıralamasıdır.

u başlangıçta L listesinin başı olduğu için ondan önce tepe yoktur ve bu yüzden ondan önce aşırı akışlı tepe de yoktur.

Yineleme: while döngüsünün her iterasyonunun topolojik sıralamayı sürdürdüğünü görmek için kabul edilebilir ağı sadece itme ve yeniden etiketleme işlemleri tarafından değiştirildiğini gözlemleyerek başlarız. Lemma 26.27 gereği, itme işlemleri ayrıtların kabul edilebilir olmasına neden olmaz. Bu yüzden, sadece yeniden etiketleme işlemleri kabul edilebilir ayrıtlar yaratabilir. Bir u tepesi yeniden isimlendirildikten sonra, bununla birlikte, Lemma 26.28, u ya giren tutarlı ayrıtların olmadığını ancak u dan ayrılan kabul edilebilir ayrıtların olabileceğini gösterir. Böylece, u yu L nin önüne taşıyarak, algoritma u dan çıkan herhangi bir kabul edilebilir ayrıtların topolojik sıralamayı sağladığını garanti eder.

L de u dan önce gelen hiçbir tepenin aşırı akışı olmadığını görmek için, sonraki iterasyonda u olacak tepelyi u' ile temsil ederiz. Bir sonraki iterasyonda u' nden önce gelen tepeler mevcut u yu içerir (satır 11 nedeniyle) ayrıca ya diğer tepeleri içermez (eğer u yeniden etiketlenirse) ya da önceden olduğu gibi aynı tepeleri içerir (eğer u yeniden etiketlenmezse). u boşaltıldığında, artık aşırı akışı yoktur. Böylece, eğer u boşaltım süresince yeniden etiketlenirse u' den önceki hiçbir tepenin aşırı akışı yoktur. Eğer u boşaltım süresince yeniden etiketlenmezse, listede ondan önceki hiçbir tepe bu boşaltım süresince aşırı akış kazanmaz çünkü L boşaltım boyunca her zaman topolojik olarak sıralanmış olmaya devam eder (tam dikkat çekildiği gibi, kabul edilebilir ayrıtlar yalnızca yeniden etiketlemeyle yaratılabilir, itme ile değil) ve dolayısıyla her itme işlemi aşırı akışın listenin daha da altındaki tepelere ilerlemesine neden olur (ya da s veya t ye). Yine, u' den önceki hiçbir tepenin aşırı akışı yoktur.

Sonlandırma: Döngü sona erdiğinde, u tam L nin sonuna geçer ve dolayısıyla döngü değişmezi, her tepenin aşımının 0 olmasını sağlar. Böylece, hiçbir temel işlem uygulanmaz.

Analiz

Şimdi RELABEL–TO–FRONT un herhangi bir $G = (V, E)$ akış ağında $O(V^3)$ zamanında çalıştığını göstereceğiz. Algoritma, genel itme-yeniden etiketleme algoritmasının bir yürütmesi olduğu için, Sonuç 26.21'in avantajını alacağız ki bu da tepe başına yürütülen yeniden etiketleme işlemlerinin sayısı üzerine bir $O(V)$ ve ortalama yeniden etiketleme işlemlerinin toplam sayısı üzerine bir $O(V^2)$ sınırı sağlar. Buna ek olarak Alıştırma 26.4-3 yeniden etiketleme işlemlerini yürütmek için harcanan toplam zaman üzerine bir $O(VE)$ sınırı sağlar ve Lemma 26.22 böylesi işlemlerin doğunluğunun toplam sayısı üzerine bir $O(VE)$ sınırı sağlar.

Teorem 26.30

Bir $G = (V, E)$ akış ağı üzerinde RELABEL–TO–FRONT un çalışma zamanı $O(V^3)$ tür.

Kanıt Ardışık iki yeniden etiketleme işlemi arasındaki zamanın öne-yeniden etiketleme algoritmasının bir “faz”ı olduğunu düşünelim. $O(V^2)$ yeniden etiketleme işlemi olduğu için $O(V^2)$ faz vardır. Her safha DISCHARGE’a en çok $|V|$ çağrısı içerir ki bunları aşağıda görebiliriz. Eğer DISCHARGE bir yeniden etiketleme işlemi yürütmezse, o zaman bir sonraki DISCHARGE çağrısı L listesinde daha da aşağı iner ve L nin uzunluğu $|V|$ den daha az olur. Eğer DISCHARGE bir yeniden etiketleme yürütürse sonraki DISCHARGE çağrısı farklı bir faza ait olur. Her faz DISCHARGE için en çok $|V|$ çağrısı içerdiğinden ve $O(V^2)$ faz olduğundan, RELABEL–TO–FRONT in 8. satırında çağrılan DISCHARGE sayısı $O(V^3)$ olur. Böy-

lece, RELABEL–TO–FRONT taki **while** döngüsü tarafından yürütülen toplam iş, DISCHARGE içinde yürütülen iş hariç olmak üzere, en çok $O(V^3)$ olur.

Şimdi, algoritmanın yürütülmesi süresince DISCHARGE içinde yürütülen çalışmayı sınırlamak zorundayız. **while** döngüsünün DISCHARGE içindeki her iterasyonu üç eylemden birisini yürütür.

Yeniden etiketleme işlemleriyle (4-5 satırları) başlarız. Örnek 26.4-3 yürütülen tüm $O(V^2)$ yeniden etiketlemeler üzerinde bir $O(VE)$ zaman sınırı sağlar.

Şimdi, eylemin satır 8 deki $u.current$ işaretçisini güncellediğini varsayalım. Bu eylem, bir u tepesinin yeniden etiketlendiği her zaman, $O(derece(u))$ defa ve tepe için de toplamda $O(V \cdot derece(u))$ defa olur. Tüm tepeler için dolayısıyla, komşu listelerindeki işaretçileri ilerletmede yapılan işin toplam miktarı tokalaşma lemmasından $O(VE)$ dir. (Alıştırma B.4-1).

DISCHARGE tarafından yürütülen üçüncü eylem türü, bir itme işlemidir (satır 7). Doyuran itme işlemlerinin toplam sayısının $O(VE)$ olduğunu zaten biliyoruz. Doyurmayan bir itme işlemi yürütülürse, DISCHARGE nın hemen döndüğünü gözlemleriz çünkü itme aşımı 0 a düşürür. Böylece, DISCHARGE çağrısı başına en çok bir doyurmayan itme olabilir. Gözlemlemiş olduğumuz gibi DISCHARGE $O(V^3)$ defa çağrılır ve böylece doyurmayan itmeler için harcanan toplam zaman $O(V^3)$ olur.

RELABEL–TO–FRONT un işlem zamanı bu yüzden, $O(V^3 + VE)$ dir ve bu da $O(V^3)$ tür. ■

Alıştırmalar

26.5-1

Şekil 26.1(a) daki akış ağı için Şekil 26.10 a göre RELABEL–TO–FRONT un çalışma şeklini gösteriniz. L deki tepelerin başlangıç sıralaması $\langle v_1, v_2, v_3, v_4 \rangle$ şeklindedir ve komşuluk listesi de aşağıdaki gibidir:

$$\begin{aligned} v_1.N &= \langle s, v_2, v_3 \rangle, \\ v_2.N &= \langle s, v_1, v_3, v_4 \rangle, \\ v_3.N &= \langle v_1, v_2, v_4, t \rangle, \\ v_4.N &= \langle v_2, v_3, t \rangle. \end{aligned}$$

26.5-2 *

Taşan tepelerin ilk-giren ilk-çıkan kuyruk yapısını sürdürdüğümüz bir itme-yeniden etiketleme algoritması uygulamak istiyoruz. Algoritma sürekli olarak kuyruğun başındaki tepeyi boşaltmaktadır ve boşaltmadan önce taşmayan tepeler değil boşaltmadan sonra taşan tepeler kuyruğun en sonunda yer almaktadır. Kuyruğun başındaki tepe boşaltıldıktan sonra o tepe çıkartılır. Kuyruk tamamen boşaldığın-

da algoritma sonlanmaktadır. Bu algoritmanın maksimum bir akışı $O(V^3)$ zamanda hesaplaması için nasıl uygulandığını gösteriniz.

26.5-3

RELABEL $u.h = u.h + 1$ formülünü basitçe hesaplayarak $u.h$ yi güncellerse soysal algoritmanın hala çalıştığını gösteriniz. Bu değişiklik RELABEL-TO-FRONT analizini nasıl etkilemektedir?

26.5-4 *

Eğer her zaman taşan en yüksek tepeyi boşaltırsak, itme-yeniden etiketleme yöntemini $O(V^3)$ zamanda çalıştırabileceğimizi gösteriniz.

26.5-5

İtme-yeniden etiketleme algoritmasının yürütülmesinin bir yerinde $v.h = k$ olmadığı bir tepe için $0 < k < |V| - 1$ aralığında bir tam sayı olduğunu varsayın. $v.h > k$ koşulunu sağlayan bütün tepelerin minimum kesimin kaynak tarafında olduğunu gösteriniz. Eğer böyle bir k mevcut ise, **aralık sezgiseli (gap heuristic)** $v.h = \max(v.h, |V| + 1)$ değerini hesaplayabilmek için $v.h > k$ durumundaki $v \in V - \{s\}$ olan her tepeyi güncellemektedir. Elde edilen h niteliğinin bir yükseklik fonksiyonu olduğunu gösteriniz (Aralık sezgiseli, itme-yeniden etiketleme yönteminin etkili bir şekilde gerçekleştirilebilmesi için çok önemlidir).

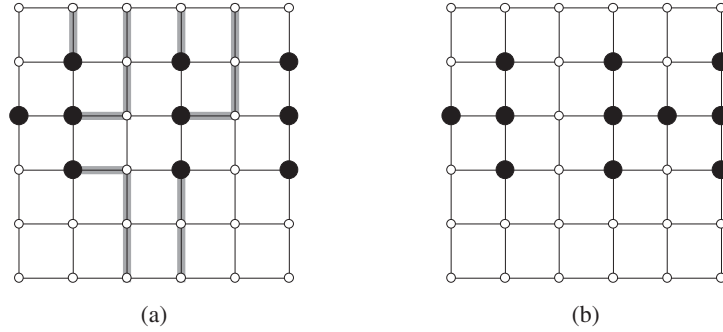
Problemler

26-1 Kaçış problemi

$n \times n$ lik bir **ızgara (grid)** n adet satır ve n adet sütun içeren yönsüz yönsüz çizgedir, Şekil 26.11 de görüldüğü gibi. i . satır j . sütundaki tepeyi (i, j) ile ifade ediyoruz. Bir ızgaradaki bütün tepelerin ayrıtları hariç tam olarak dört adet komşusu vardır, bu tepeler $i = 1, i = n, j = 1$ ya da $j = n$ durumundaki (i, j) noktalarıdır.

Izgarada $m \leq n^2$ iken $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$ başlangıç noktaları verildiğinde, **kaçış problemi (escape problem)** sınırdaki başlangıç noktasından herhangi m sayıda farklı noktaya tepe-ayrık yolun olup olmadığını belirlemektedir. Örnek olarak, Şekil 26.11(a) daki ızgara bir kaçışa sahiptir, fakat Şekil 26.11(b) deki ızgara ise sahip değildir.

- a. Tepelerin ve aynı zaman ayrıtların kapasiteleri olduğu bir akış ağı olduğunu varsayın. Yani, verilen herhangi bir tepeye girilen toplam pozitif akış, kapasite kısıtına bağlıdır. Ayrıt ve tepe kapasiteleri olan bir ağdaki maksimum akışın, karşılaştırılabilir boyutta bir akış ağı üzerinde sıradan bir maksimum-akış problemine indirgenebildiğini gösteriniz.



Şekil 26.11 Kaçış problemi için ızgaralar. Başlangıç noktaları siyah ve diğer ızgara tepeleri beyazdır. (a) Gölge yollar ile gösterilen kaçışlı ızgara. (b) Kaçış olmayan bir ızgara.

- b.** Kaçış problemini çözen etkili bir algoritma tanımlayın, ve çalışma zamanını da analiz ediniz.

26-2 Minimum yol örtüsü

Yönlü bir $G = (V, E)$ çizgesinin bir **yol örtüsü (path cover)** tepe-ayrık yollarının P kümesidir öyle ki V deki her tepe P de tam olarak tek bir yola dahildir. Yollar başlar ve herhangi bir yerde biter ve bu yollar herhangi bir uzunlukta olabilir, 0 da dahil. G nin bir **minimum yol örtüsü (minimum path cover)** en az olası yolları içeren yol örtüsüdür.

- a.** Yönlü çevrimsiz bir $G = (V, E)$ çizgesini minimum yol örtüsünü bulan etkili bir algoritma veriniz. (İpucu: $V = \{1, 2, \dots, n\}$ olduğunu varsayın, $G' = (V', E')$ çizgesini yapılandırın, öyle ki

$$V' = \{x_0, x_1, \dots, x_n\} \cup \{y_0, y_1, \dots, y_n\},$$

$$E' = \{(x_0, x_i) : i \in V\} \cup \{(y_i, y_0) : i \in V\} \cup \{(x_i, y_j) : (i, j) \in E\}$$

olsun ve maksimum-akış algoritmasını çalıştırın).

- b.** Geliştirdiğiniz algoritma çevrimler içeren yönlü bir çizge için çalışır mı? Açıklayınız.

26-3 Algoritmik danışmanlık

Profesör Gore algoritmik danışmanlık şirketi açmayı istemektedir. Profesör $A = \{A_1, A_2, \dots, A_n\}$ kümesi ile temsil ettiği algoritmaların n adet önemli alt alanlarını (kabaca, bu ders kitabının farklı bölümlerine karşı gelen) belirlemiştir. farklı bölümüne karşı gelen). Her A_k alt alanlarına, c_k dolar karşılığında uzmanlar tutmuştur. Danışmanlık şirketi potansiyel işlerin bir $J = \{J_1, J_2, \dots, J_m\}$ kümesini düzenlemiştir. J_i işini gerçekleştirmek için şirketin alt alanlarının bir $R_i \subseteq A$ altkümesi kadar uzman tutmaya ihtiyacı vardır. Her uzman eşzamanlı

olarak birden çok işte çalışabilir. Eğer şirket J_i işini kabul etmeyi seçtiğinde, R_i deki tüm alt alanlarda uzman tutmak zorundadır ve p_i dolar kadar gelir elde edecektir.

Profesör Gore'un işi, hangi alt alanlara uzman tutulacağını, net gelirleri maksimum yapmak için hangi işlerin kabul edileceğini, kabul edilen işlerin toplam gelirinden çalışan uzmanların toplam giderlerini çıkararak belirlemektir.

Aşağıdaki G akış ağını ele alalım. Bu ağ bir kaynak s tepesi, $A_1, A_2, \dots, A_n$ tepeleri, $J_1, J_2, \dots, J_m$ tepeleri ve bir de t çıkış düğümü içermektedir. $k = 1, 2, \dots, n$ için akış ağı $c(s, A_k) = c_k$ kapasitesinde (s, A_k) ayrıtı içermektedir ve $i = 1, 2, \dots, m$ için akış ağı $c(J_i, t) = p_i$ kapasitesinde (J_i, t) ayrıtı içermektedir. $k = 1, 2, \dots, n$ ve $i = 1, 2, \dots, m$ için eğer $A_k \in R_i$ ise, $c(A_k, J_i) = \infty$ kapasitesinde (A_k, J_i) ayrıtı içermektedir.

- Eğer G nin sonlu kapasiteli bir (S, T) kesimi için $J_i \in T$ ise, her $A_k \in R_i$ için $A_k \in T$ olduğunu gösteriniz.
- G nin bir minimum kesimin kapasitesinden ve verilen p_i değerlerinden maksimum net gelirin nasıl hesaplandığını gösteriniz.
- Hangi işlerin kabul edileceğini ve hangi uzmanların tutulacağını belirleyen etkili bir algoritma belirleyiniz. Geliştirdiğiniz algoritmanın m, n ve $r = \sum_{i=1}^m |R_i|$ cinsinden çalışma zamanını analiz ediniz.

26-4 Maksimum akışı güncelleme

$G = (V, E)$ kaynağı s , çıkış düğümü t ve tam sayılı kapasiteleri olan bir akış ağı olsun. Bize G de maksimum akış verildiğini varsayalım.

- $(u, v) \in E$ olan tek bir ayrıtın kapasitesini 1 kadar arttırdığımızı varsayalım. Maksimum akışı güncelleyen $O(V + E)$ zamanında çalışan bir algoritma veriniz.
- $(u, v) \in E$ olan tek bir ayrıtın kapasitesini 1 kadar azalttığımızı varsayalım. Maksimum akışı güncelleyen $O(V + E)$ zamanında çalışan bir algoritma veriniz.

26-5 Ölçekleme ile maksimum akış

$G = (V, E)$ kaynağı s , çıkış düğümü t ve her $(u, v) \in E$ ayrıtı üzerinde tam sayılı $c(u, v)$ kapasiteleri olan bir akış ağı olsun. $C = \max_{(u,v) \in E} c(u, v)$ olsun.

- G nin bir minimum kesiminin en fazla $C|E|$ kapasitesine sahip olduğunu tartışın.
- Verilen K sayısı için $O(E)$ zamanında kapasitesi en az K olan genişletilmiş bir yolun nasıl bulunduğunu gösteriniz, eğer böyle bir yol varsa.

G deki maksimum akışı bulmak için aşağıdaki FORD-FULKERSON-METHOD un değiştirilmiş şeklini kullanabiliriz.

MAX-FLOW-BY-SCALING(G, s, t)

```

1   $C = \max_{(u,v) \in E} c(u, v)$ 
2   $f$  akışını 0 olarak başlat
3   $K = 2^{\lceil \lg C \rceil}$ 
4  while  $K \geq 1$ 
5      while en az  $K$  kapasiteli genişletilmiş bir  $p$  yolu var
6           $p$  boyunca  $f$  akışını artır
7       $K = K/2$ 
8  return  $f$ 
```

- c. MAX-FLOW-BY-SCALING in bir maksimum akış döndürdüğünü tartışınız.
- d. G_f artık ağın minimum kesim kapasitesinin 4. satırın her çalışmasında en fazla $2K |E|$ kadar olduğunu gösteriniz.
- e. 5-6 satırlarındaki iç içe **while** döngüsü K nın her değeri için $O(E)$ zamanda çalıştığını tartışınız.
- f. MAX-FLOW-BY-SCALING in $O(E^2 \lg C)$ zamanda çalıştığının uygulanabildiğini sonuçlandırınız.

26-6 Hopcroft-Karp iki parçalı eşleme algoritması

Bu problemde, Hopcroft ve Karf'a göre iki parçalı çizgelerde maksimum eşleşmeyi bulan daha hızlı bir algoritma tanımlıyoruz. Bu algoritma $O(\sqrt{V}E)$ zamanda çalışmaktadır. $V = L \cup R$ olduğu durumda yönsüz iki parçalı bir çizge $G = (V, E)$ verilsin ve bütün ayrıtlar L de tam olarak bir bitiş noktası, M ise G de bir eşleme olsun. G de basit bir P yolu eğer L de eşlenmemiş bir tepeden başlar ve R de eşlenmemiş bir tepede biterse, M ye göre bir **genişletilmiş yoldur (augmenting path)** ve o yolun ayrıtları değişimli olarak M ve $E - M$ ye aittir. (Genişletilmiş yolun bu tanımı bir akış ağındaki genişletilmiş yol ile ilişkili fakat farklıdır.) Bu problemde, bir yolu tepeler dizisinden ziyade ayrıtlar dizisi olarak işliyoruz. Bir M eşlemesine göre en kısa genişletilmiş bir yol, minimum ayrıtlı genişletilmiş bir yoldur.

A ve B kümeleri verilsin, $A \oplus B$ **simetrik farkı (symmetric difference)** $(A - B) \cup (B - A)$ ile tanımlanır, yani elemanlar iki kümeden yalnızca birinde bulunmaktadır.

- a. Eğer M bir eşleme ve P M ye göre genişletilmiş bir yol ise, $M \oplus P$ simetrik farkının bir eşleme ve $|M \oplus P| = |M| + 1$ olduğunu gösteriniz. Eğer $P_1, P_2, \dots, P_k$ M ye göre tepe-ayrık genişletilmiş yollar ise, $M \oplus (P_1 \cup P_2 \cup \dots \cup P_k)$ simetrik farkının kardinalitesi $|M| + k$ olan bir eşleme olduğunu gösteriniz.

Algoritmamızın genel yapısı şöyledir:

HOPCROFT-KARP(G)

```

1   $M = \emptyset$ 
2  repeat
3       $\mathcal{P} = \{P_1, P_2, \dots, P_k\}$   $M$  ye göre tepe-ayrık en kısa genişletilmiş
        yolların maksimal bir kümesi olsun.
4       $M = M \oplus (P_1 \cup P_2 \cup \dots \cup P_k)$ 
5  until  $\mathcal{P} == \emptyset$ 
6  return  $M$ 
```

Problemin geri kalan kısmına göre algoritmadaki iterasyon sayısını bulunuz (**repeat** döngüsü içindeki iterasyon sayısı) ve 3. satırın uygulanmasını tanımlayınız.

- b. G de M ve M^* eşlemeleri verilsin, $G' = (V, M \oplus M^*)$ çizgesindeki her tepenin derecesinin en fazla 2 olduğunu gösteriniz. G' basit yolların ya da çevrimlerin ayrık bir birleşimidir sonucuna varınız. Böyle her basit yoldaki veya çevrimdeki ayrıtlar değişimli olarak M veya M^* ne ait olduğunu tartışınız. Eğer $|M| \leq |M^*|$ ise, $M \oplus M^*$ nün M ye göre en az $|M^*| - |M|$ kadar tepe-ayrık genişletilmiş yollar içerdiğini kanıtlayınız.

l bir M eşlemesine göre en kısa genişletilmiş bir yolun uzunluğu olsun ve $P_1, P_2, \dots, P_k$ ise M ye göre tepe-ayrık eklenmiş yolların l uzunluğundaki maksimum kümesi olsun. $M' = M \oplus (P_1 \cup \dots \cup P_k)$ olsun ve M' ne göre en kısa genişletilmiş yolun P olduğunu varsayın.

- c. Eğer P yolu $P_1, P_2, \dots, P_k$ dan tepe-ayrık ise, P nin l adet ayrıttan daha fazlasına sahip olduğunu gösteriniz..
- d. P yolunun $P_1, P_2, \dots, P_k$ dan tepe-ayrık olmadığını varsayın. $A, (M \oplus M') \oplus P$ ayrıtlarının bir kümesi olsun. $A = (P_1 \cup P_2 \cup \dots \cup P_k) \oplus P$ ve $|A| \geq (k+1)l$ olduğunu gösteriniz. P nin l den daha fazla ayrıtı olduğunu sonuçlandırınız.
- e. Eğer M ye göre en kısa genişletilmiş bir yolun l adet ayrıtı var ise, maksimum eşlemenin boyutunun $|M| + |V| / (l+1)$ olduğunu kanıtlayınız.

- f. Algoritmadaki **repeat** döngü iterasyonlarının sayısının en fazla $2\sqrt{|V|}$ olduğunu gösteriniz. (*İpucu:* $\sqrt{|V|}$ iterasyondan sonra M ne kadar büyüyebilir?
- g. Verilen bir M eşlemesi için tepe-ayrık en kısa genişletilmiş $P_1, P_2, \dots, P_k$ yollarının maksimal bir kümesini bulmak için $O(E)$ zamandan çalışan bir algoritma veriniz. HOPCROFT-KARP algoritmasının toplam çalışma zamanının $O(\sqrt{V}E)$ olduğu sonucuna varın.

Bölüm notları

Ahuja, Magnanti, ve Orlin [7], Even [103], Lawler [224], Papadimitriou ve Steiglitz [271] ve Tarjan [330] ağ akışı ve ilgili algoritmalar için iyi referanslardır. Goldberg, Tardos, ve Tarjan [139] ağ-akış problemleri için çalışan algoritmalar hakkında iyi araştırmalar sağlamışlardır, ve Schrijver [304] ağ akışı alanının tarihsel gelişme süreci hakkında ilginç derlemeler yazmıştır.

Ford-Fulkerson yöntemi, ağ akışı alanında maksimum-akış ve iki parçalı eşleme problemleri de dahil birçok problemin formal çalışmasına kaynak oluşturan Ford ve Fulkerson [109] a bağlıdır. Ford-Fulkerson yönteminin daha önceki uygulamaları, enine arama kullanılarak genişletilmiş yolu bulmuştur; Edmonds ve Karp [102] ve bunlardan bağımsız olarak Dinic[89], bu stratejinin polinom-zamanlı algoritma sonucunu ortaya çıkardığını kanıtlamışlardır. Bununla ilgili bir fikir, “akışı durdurma,” kullanılarak üretilen bu sonuç ilk olarak Dinic [89] tarafından geliştirilmiştir. Karzanov [202] önakışlar fikrini geliştiren ilk kişidir. İtme-yeniden etiketleme yöntemi, Goldberg [136] ve Goldberg ve Tarjan [140] a dayanmaktadır. Goldberg ve Tarjan, taşan tepelerin kümesini devam ettirerek kuyruk yapısını kullanan $O(V^3)$ zamanda çalışan, hem de $O(VE \lg(V^2/E + 2))$ çalışma zamanına ulaşabilmek için dinamik ağaçları kullanan bir algoritma ortaya koymuşlardır. Diğer bazı araştırmacılar itme-yeniden etiketleme maksimum-akış algoritmaları geliştirmişlerdir. Ahuja ve Orlin [9] ve Ahuja, Orlin, ve Tarjan [10] ölçeklendirme kullanan algoritmalar vermişlerdir. Cheriyan ve Meheshwari [62] maksimum yükseklikteki taşan tepeden dolayı itici akış ileri sürmüşlerdir. Cheriyan ve Hagerup [61] komşuluk listesini rassal olarak değiştirdiğini öne sürmüşlerdir ve bazı araştırmacılar [14, 204, 276] bu fikir üzerinden hızlı algoritmalar ile zeki rassallaştırma geliştirmişlerdir. King, Rao ve Tarjan [204] ın algoritması en hızlı çalışan algoritmadır ve $O(VE \log_{E/(V \lg V)} V)$ zamanda çalışır.

Maksimum-akış problemi için asimptotik olarak hali hazırda en hızlı çalışan algoritma, Goldberg ve Rao [138] tarafından geliştirilmiş, $O(\min(V^{2/3}, E^{1/2})E \lg(V^2/E + 2) \lg C)$ zamanda çalışmaktadır öyle ki $C = \max_{(u,v) \in E} c(u, v)$. Bu algoritma itme-yeniden etiketleme yöntemini kullanmamaktadır fakat onun yerine akışları bloke etmeyi bulma tabanlıdır. Bu bö-

lümdekiler de dahil, daha önceki bütün maksimum-akış algoritmaları her ayrıta 1 uzunluğu atanmış bir mesafe kavramını (itme-yeniden etiketleme benzer yükseklik kavramını kullanır) kullanır. Bu yeni algoritma farklı bir yaklaşım vermektedir ve yüksek kapasiteli ayrıtlara 0 uzunluğunu ve düşük kapasiteli ayrıtlara 1 uzunluğunu atamaktadır. Gayri resmi olarak, bu uzunluklara bağlı olarak, kaynaktan çıkış düğümüne en kısa yol yüksek kapasiteli olma eğilimindedir, bu daha az iterasyon gerektirir.

Uygulamada, itme-yeniden etiketleme algoritması maksimum-akış problemleri için, genişletilmiş-yol ya da doğrusal-programlama tabanlı algoritmalara göre daha üstündür. Cherkassky ve Goldberg [63] tarafından yapılan bir çalışmaya göre bir itme-yeniden etiketleme algoritması uygulandığı zaman iki sezgisel yöntem kullanmanın önemi vurgulanmaktadır. İlk sezgisel yöntem daha kesin yükseklik değerleri elde etmek için periyodik olarak artık ağın enine arama yöntemini gerçekleştirir. İkinci sezgisel yöntem aralık sezgiselidir, Alıştırma 26.5-5 de tanımlandığı gibi. Cherkassky ve Goldberg, maksimum yükseklikteki taşan tepeleri boşaltmak için itme-yeniden etiketleme çeşitlerinden en iyisini seçerek sonuçlandırmaktadırlar.

Maksimum iki parçalı eşleme için hali hazırdaki $O(\sqrt{V}E)$ zamanda çalışan en iyi algoritma, Hopcroft ve Karp [176] tarafından bulunmuştur ve Problem 26-6 da tanımlanmıştır. Lovasz ve Plummer [239] un kitabı eşleme problemleri için mükemmel bir referanstır.



VII Seçme Konular

Giriş

Bu bölüm, kitaptaki daha önceki materyali genişleten ve tamamlayan algoritmik konuların bir seçimini içerir. Bazı bölümler devreler veya paralel bilgisayarlar gibi hesaplamanın yeni modellerini tanıtır. Diğer bölümler, hesaplanabilir geometri veya sayılar teorisi gibi uzmanlık alanlarını kapsar. Kitabın son iki bölümü verimli algoritmaların tasarımının bilinen bazı sınırlamalarını tartışır ve bu sınırlamalar ile başa çıkma tekniklerini tanıtır.

Bölüm 27 temeli dinamik çoklu iş parçacığı kullanımı olan paralel hesaplama için algoritmik bir model sunar. Bu bölüm çalışma ve yayılma ölçüleri açısından paralelliğin nasıl ölçüleceğini gösteren söz konusu modelin temellerini tanıtır. Daha sonra da matris çarpımını ve sıralama birleştirme algoritmaları dahil olmak üzere çoklu iş parçacıklı birçok ilginç algoritmayı inceler.

Bölüm 28 matrisler üzerinde işlem yapmak için etkili algoritmaları anlatır. Bu bölüm $O(n^3)$ işlem sayısı zamanında Gauss yok etme yöntemiyle doğrusal denklemleri çözmek için LU ayrışımı(LU decomposition) ve LUP ayrışımı(LUP decomposition) olarak adlandırılan iki genel yöntem sunar. Ayrıca, matrisin tersinin bulunmasının ve matris çarpımının eşit hızda gerçekleştirilebileceğini gösterir. Bölüm, doğrusal denklemler sisteminin kesin çözümünün olmadığı durumda sistemin en küçük kareler yaklaşık çözümünün nasıl hesaplanacağını göstererek biter.

Bölüm 29, verilen sınırlı kaynaklar ve rekabet kısıtlarının doğrusal programlanmasını -ki burada bir fonksiyonu en üst düzeye çıkarmayı veya en aza indirmeyi istiyoruz- anlatır. Doğrusal programlama pratik uygulama alanlarının bir çeşidinde ortaya çıkar. Bu bölüm doğrusal programların nasıl formüle edileceğini ve çözüleceğini kapsar. Çözüm yöntemi doğrusal programlama için en eski algoritma olan simpleks algoritmasıdır. Bu kitaptaki birçok algoritmanın aksine, simplex algoritması en kötü durumda polinom zamanında çalışmaz, ancak o oldukça etkilidir ve uygulamada yaygın olarak kullanılır.

Bölüm 30 polinomlar üzerindeki işlemleri inceler ve n -dereceden iki polinomu $O(n \log n)$ süresinde çarpmak için iyi bilinen bir sinyal işleme tekniğinin — hızlı Fourier dönüşümü (the fast Fourier transform) (FFT)—nasıl kullanılacağını gösterir. Ayrıca bir paralel devre de dahil olmak üzere FFT nin verimli uygulamalarını araştırır.

Bölüm 31 sayılar teorisi algoritmalarını sunar. Temel sayılar teorisini gözden geçirdikten sonra, en büyük ortak böleni hesaplamak için Euclid algoritmasını sunar. Daha sonra modüler doğrusal denklemleri çözen ve bir sayıyı başka bir sayının kuvvet moduna yükselten algoritmaları anlatır. O zaman sayı teorisi algoritmalarının önemli bir uygulamasını keşfeder: RSA ortak anahtarlı şifreleme sistemi (RSA public-key cryptosystem). Bu şifreleme sadece mesajları şifrelemek için değil-ki böylelikle rakip bir kişi onları okuyamaz- aynı zamanda dijital imzalar sağlamak için de kullanılabilir. Bölüm böylece RSA sistemi için temel bir gereklilik olan etkili bir şekilde büyük asal sayıları bulabileceğimiz Miller-Rabin rassal asallık testini (Miller-Rabin randomized primality test) sunar. Son olarak, bölüm tam sayıların asal çarpanlarına ayrışımı için Pollard'ın "rho" sezgisel algoritmasını kapsar ve tam sayının çarpanlarına ayrışımı sanatının durumunu tartışır.

Bölüm 32 metin düzenleme programlarında sıkça ortaya çıkan, verilen bir metin katarındaki belirli bir desen katarının tüm tekrarlarını bulma problemini verir. Naif yaklaşımı inceledikten sonra, Bölüm Rabin ve Karp tarafından verilen zarif bir yaklaşımı sunar. O zaman, sonlu otomata dayalı etkin bir çözüm gösterdikten sonra, bölüm bu desenin akıllı ön işlenmesiyle yer kazanmak için otomat-tabanlı algoritmayı değiştiren Knuth-Morris-Pratt algoritmasını sunar.

Bölüm 33 hesaplanabilir geometrinin birkaç problemini ele alır. hesaplanabilir geometrinin temel ilkelerini tartıştıktan sonra bölüm, doğru parçalarının bir kümesinin herhangi bir kesişme içerip içermediğini etkili bir şekilde belirlemek için "tarama" yönteminin nasıl kullanılacağını gösterir. Bir nokta kümesinin dış-bükey örtüsünün bulunması için Graham'ın tarama (Graham's scan) ve Jarvis (Jarvis's march) adı verilen iki akıllı algoritma, süpürme yöntemlerinin gücünü göstermektedir. Bu bölüm düzlemde verilen bir nokta kümesi içinde en yakın çifti bulmak için etkin bir algoritma ile kapanır.

Bölüm 34 NP-tam(NP-complete) problemleri verir. Birçok ilginç hesaplama problemi NP-tamdır. Ancak onların herhangi birinin çözümü için bilinen polinom zamanlı bir algoritma yoktur. Bu bölüm bir problem NP-tam olduğunda onun belirlenebilmesi için teknikler sunar. Birkaç klasik problemin NP-tam olduğu kanıtlanır: Bir çizgenin bir hamiltonian döngüsü olup olmadığının belirlenmesi, bir Boolean formülünün sağlatılabilir olup olmadığının belirlenmesi ve verilen bir sayı kümesinin verilen bir hedef değere kadar toplanan bir alt kümesinin olup olmadığının belirlenmesi. Bu bölüm aynı zamanda ünlü gezgin satıcı probleminin NP-tamk olduğunu kanıtlar.

Bölüm 35 yaklaşım algoritmalarını kullanarak NP-tam problemler için yaklaşık çözümlerin etkili şekilde nasıl bulunacağını gösterir. Bazı NP-tam problemler

için optimale yakın yaklaşık çözümler üretmek oldukça kolaydır, ama diğerleri için çalıştığı bilinen en iyi yaklaşım algoritmaları bile problemin boyutu arttıkça giderek daha kötü çalışır. O zaman, daha iyi yaklaşık çözümler karşılığında hesaplama zamanının artan miktarına yatırım yapabileceğimiz bazı problemler vardır. Bu bölüm tepe-örtüsü problemi (ağırlıksız ve ağırlıklı versiyonları), 3-CNF gerçekleştirilebilirliğinin bir optimizasyon versiyonu, gezgin-satıcı problemi, küme-örtüsü problemi ve altküme-toplam problemi gibi bu olanakları açıklamaktadır.

27 Çoklu İş Parçacıklı Algoritmalar

Bu kitaptaki algoritmaların büyük bir çoğunluğu bir seferde tek bir komut yürüten tek işlemcili bilgisayarda çalıştırmak için uygun *seri algoritmalar (serial algorithms)* dır. Bu bölümde, algoritmik modelimizi aynı anda birden fazla komutu yürüten çok işlemcili bir bilgisayarda çalıştırabileceğimiz kapsamlı *paralel algoritmalara (parallel algorithms)* genişleteceğiz. Özellikle, algoritmik tasarım ve analiz, hem de uygulamada etkin bir şekilde yürütülmesi uygun olan dinamik çoklu iş parçacıklı algoritmaların şık modelini keşfedeceğiz.

Paralel bilgisayarlar—çok işlemcili bilgisayarlar—giderek daha yaygın hale geldi ve performans ve geniş bir fiyat aralığı oluşturdu. Nispeten ucuz masaüstü ve dizüstü *çok işlemcili çip (chip multiprocessors)*, her biri bir ortak belleğe erişebilen tam teşekküllü bir işlemci olan ve içinde çoklu işlem “çekirdeklerini” bulunduran entegre-devre çipli bir tek *çok çekirdekli (multicore)* içerir. Bir ara fiyat / performans noktası kişisel bilgisayarlar- özel bir ağ ile birbirine bağlı—genellikle basit PC sınıfı makinalardan oluşan kümelerdir. Fiyatı en yüksek olan makineler genellikle saniyede yürütülen yönergeler açısından en yüksek performansını sunmak için özel mimarileri ve özel ağları bir arada kullanan süperbilgisayarlardır.

Çok işlemcili bilgisayarlar bir biçim veya diğeri şeklinde yıllarca var olmuştur. Bilgisayar bilimleri tarihinin başlarında bilgisayar topluluğu, seri işlem için rassal-erişimli makine modelinde yerleşmiş olsa da, paralel hesaplama için tek bir model geniş kabul görmemiştir. Bunun önemli bir nedeni satıcıların paralel bilgisayarlar için tek bir mimari model üzerinde anlaşamamalarıdır. Örneğin, bazı paralel bilgisayarlar, her işlemcinin belleğin herhangi bir yerine doğrudan erişebildiği *paylaşılmış bellek (shared memory)* özelliğine sahiptir. Diğer paralel bilgisayarlar her işlemcinin belleğinin özel olduğu ve bir işlemci başka belleğe erişmek için işlemciler arasında açık bir mesajın gönderilmesinin gerektiği, *dağıtılmış bellek (distributed memory)* kullanır. Çok çekirdekli teknoloji ile birlikte, ancak, her yeni dizüstü ve masaüstü makine artık paylaşılmış bellekli paralel bil-

gisayardır ve eğilim paylaşılmış bellekli çoklu işlemcilere doğru gibi görünüyor. Zaman gösterecek olsa da, bu bölümde ele alacağımız yaklaşım budur.

Programlama çok işlemcili çipler ve diğer paylaşılmış bellekli paralel bilgisayarlara bir ortak aracı “sanal işlemcinin” bir yazılım soyutlamasını sağlayan **durağan iş parçacığını (static threading)** veya bir ortak hafızayı paylaşan **iş parçacıklarını (threads)** kullanmaktır. Herbir iş parçacığı ilişkili bir program sayacı sağlar ve diğer iş parçacıklarından bağımsız olarak kodu çalıştırabilir. İşletim sistemi yürütülmesi için bir işlemci üzerine bir iş yükler ve başka bir iş parçacığını çalıştırmak için gerektiğinde dışarı geçer. İşletim sistemi programcılara konuları oluşturmak ve yok etmek için izin verse de, bu işlemler nispeten yavaştır. Bu nedenle, çoğu uygulama için iş parçacıkları, onları “durağan” olarak isimlendirmemizin nedeni olan bir hesaplama süresince çalışır.

Ne yazık ki, doğrudan durağan iş parçacıklarını kullanarak paylaşılmış bellekli paralel bilgisayarın programlanması zor ve hata eğilimlidir. Bunun bir nedeni iş yükünün, iş parçacıkları arasında dinamik olarak parçalanmasıdır, böylelikle yaklaşık aynı yük alan herbir iş parçacığı iş yükü bölümlemenin karmaşık bir girişim olduğunu ortaya çıkarır. Herhangi biri ama uygulamaların en basiti için, programcı iş yükü yüklemek-dengelemek için bir çizelgeleyici uygulamak için karmaşık iletişim protokollerini kullanması gerekir. Bu durum koordinatları, programları, ve paralel bilgi işlem kaynakları yöneten bir yazılım katmanı sağlamak **eşzamanlı platformlar (concurrency platforms)** oluşturulmasına doğru yol açmıştır. Bazı eşzamanlı platformlar çalışma zamanı kütüphaneleri olarak inşa edilir, ama diğerleri derleyici ve çalışma zamanı desteği ile tam teşekküllü paralel dilleri sağlamaktadır.

Dinamik çoklu iş parçacıklı programlama

Eşzamanlı platformun önemli bir sınıfı bu kısımda uyarlayacağımız model olan **dinamik çoklu iş parçalama (dynamic multithreading)** dir. Dinamik çoklu iş parçalama, programcılara iletişim protokolleri, yük dengeleme ve durağan-iş parçacığı programlamanın diğer kaprisleri endişesi olmadan uygulamalarında paralellik belirlemelerini sağlar. Eşzamanlı platform, hesaplamayı otomatik olarak yük dengeleyen bir çizelgeleyici içerir, böylece programcının angaryasını büyük ölçüde basitleştirir. Dinamik çoklu iş parçalama ortamlarının işlevselliği hala gelişmesine rağmen hemen hemen tümü iki özelliği destekler: iç içe paralellik ve paralel döngüler. Doğurulan altprogram işlem sonucunu hesaplarken, iç içe paralellik, işlemdeki çağırıcıya izin vererek “doğurulan (spawned)” bir alt programa izin verir. Paralel döngü, döngünün özyinelemelerini aynı anda çalıştırabilmesi hariç sıradan bir **for** döngüsü gibidir.

Bu iki özellik, bu kısımda çalışacağımız dinamik çoklu iş parçalama için modelin temelini oluşturur. Bu modelin bir önemli yönü, programcının bir hesaplama içinde sadece mantıksal paralellik belirtmek ve kendi aralarında temel eşzamanlı platform programı ve yük dengesi hesaplama içindeki konulara ih-

tiyaç duymasıdır. Biz temel eşzamanlı platformun hesaplamaları nasıl verimli bir şekilde çizelgeleyebileceği kadar, bu model için yazılmış çoklu iş parçacıklı algoritmaları araştıracağız.

Dinamik çoklu iş parçacığı için modelimiz birçok önemli avantajlar sunar:

- Seri programlama modelimizin basit bir uzantısıdır. Kabakodumuza **Parallel**, **spawn**, ve **sync** şeklinde sadece üç “eşzamanlı” anahtar kelimelerini ekleyerek çoklu iş parçacıklı bir algoritma tanımlayabiliriz. Üstelik, eğer bu eşzamanlı anahtar kelimelerini çoklu iş parçacıklı kabakodumuzdan silersek sonuç metni, çoklu iş parçacıklı algoritmanın serileştirilmesi adı verilen aynı problem için seri kabakoddur.
- Bu “iş yükü (work)” ve “süre (span)” kavramlarına dayalı paralellik ölçmek için teorik olarak kusursuz bir yol sağlar.
- İç içe paralellik içeren birçok çoklu algoritma, böl-ve-yönet paradigmasını doğal olarak takip eder. Ayrıca, sadece seri böl-ve-yönet algoritmaları özyinelemeleri çözerek kendilerini analize ve böylece de çoklu iş parçacıklı algoritmalara ödünç verir.
- Model, parallel-hesaplama alıştırmasının nasıl gelişmekte olduğuna bağlıdır. parallel-hesaplama giderek artan sayısı bir varyant veya Cilk [51, 118], Cilk++ [71], OpenMP [59], Parelel Görev Kütüphanesi (Task Parallel Library) [230] ve İş parçacığı İnşa blokları (Threading Building Blocks) [292] içeren bir başka dinamik çoklu iş parçalamayı destekler.

Kısım 27.1 dinamik çoklu iş parçacıklı modeli tanıttır ve çoklu iş parçacıklı algoritmaların analizi için kullanacağımız işin metrikleri ve paralellliği sunar. Kısım 27.2 çoklu iş parçalama ile matrislerin nasıl çarpıklacağını araştırır ve Kısım 27.3 çoklu iş parçacıklı birleştirerek sıralamanın zor problemini ele alır.

27.1 Dinamik çoklu iş parçalamanın temelleri

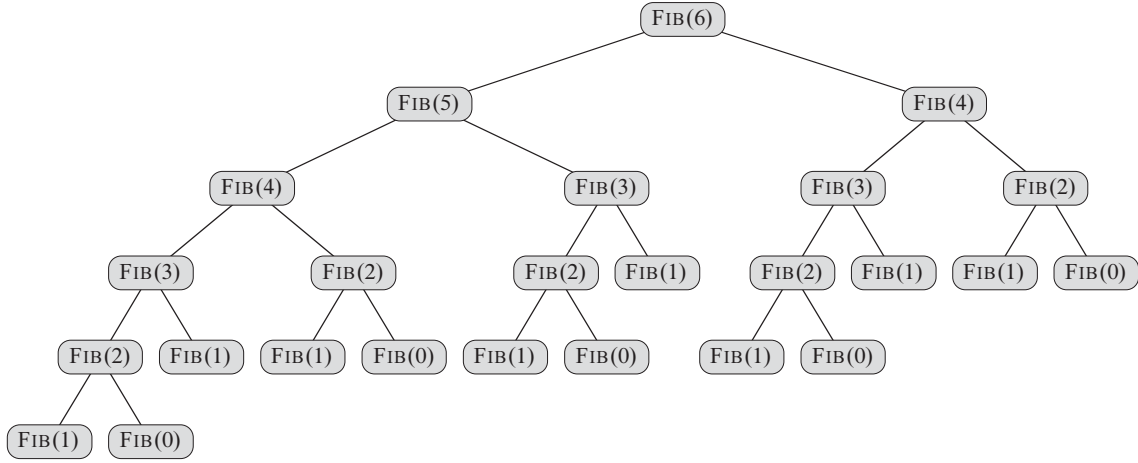
Fibonacci sayılarının ardışık olarak hesaplanması örneğini kullanarak dinamik çoklu iş parçalama keşifimize başlayacağız. Fibonacci sayılarının (3.22) özyinelemesiyle tanımlandığını hatırlayınız: $i \geq 2$ için

$$F_0 = 0 ,$$

$$F_1 = 1 ,$$

$$F_i = F_{i-1} + F_{i-2} .$$

İşte n . Fibonacci sayısını hesaplamak için basit, özyinelemeli, seri bir algoritma:



Şekil 27.1 FIB(6) nın hesaplanmasındaki özyinelemeli prosedür örneklerinin ağacı. FIB in aynı argümanlı her örneği aynı sonucu üretmek için aynı işi yapar, Fibonacci sayıları hesaplamak için etkisiz ama ilginç bir yol sağlar.

```

FIB(n)
1  if n ≤ 1
2      return n
3  else x = FIB(n - 1)
4      y = FIB(n - 2)
5      return x + y
  
```

Gerçekte büyük Fibonacci sayılarını bu şekilde hesaplamak istemezsiniz, çünkü bu hesaplama çok iş tekrarı yapıyor. Şekil 27.1, F_6 hesabında ortaya çıkan özyinelemeli prosedür örneklerinin ağacını gösteriyor. Örneğin, bir FIB(6) çağrısı ardışık olarak FIB(5) ve sonrasında FIB(4) ü çağırır. Ama bir FIB(5) çağrısı aynı zamanda bir FIB(4) çağrısıyla sonuçlanır. FIB(4) ün her iki örneği de aynı sonucu geri döndürür ($F_4 = 3$). FIB süreci ezberlemediğinden, FIB(4) ün ikinci çağrısı ilk çağrının gerçekleştirdiği işi tekrarlar.

$T(n)$, FIB(n) nın çalışma süresini gösterebilir. FIB(n), iki özyinelemeli çağrıya ek olarak bir sabit ekstra çalışma miktarı içerdiği için

$$T(n) = T(n - 1) + T(n - 2) + \Theta(1)$$

özyinelemesini elde ederiz. Bu özyineleme, yerine koyma yöntemini kullanarak gösterebileceğimiz $T(n) = \Theta(F_n)$ çözümüne sahiptir. Bir tümevarım hipotezi için $a > 1$ ve $b > 0$ sabitler olmak üzere $T(n) \leq aF_n - b$ olduğunu varsayalım.

Eğer, $\Theta(1)$ içindeki sabiti domine etmek için b yi yeterince büyük seçersek, yerine koymayla

$$\begin{aligned} T(n) &\leq (aF_{n-1} - b) + (aF_{n-2} - b) + \Theta(1) \\ &= a(F_{n-1} + F_{n-2}) - 2b + \Theta(1) \\ &= aF_n - b - (b - \Theta(1)) \\ &\leq aF_n - b \end{aligned}$$

elde ederiz. O zaman başlangıç koşulunu sağlayan yeterince büyük a seçebiliriz.

$\phi = (1 + \sqrt{5})/2$ altın oran olmak üzere, böylece denklem (3.25) den analitik sınır

$$T(n) = \Theta(\phi^n) \quad (27.1)$$

bulunur. F_n , n e göre üstel şekilde büyüdüğünden, Fibonacci sayılarını hesaplamak için bu prosedürü bilhassa yavaş bir yoldur (Daha hızlı yollar için Problem 31-3 e bakınız).

Fibonacci sayılarını hesaplamak için FIB prosedürü zayıf bir yol olmasına rağmen, çoklu iş parçacıklı algoritmaların analizinde anahtar kavramları göstermesi bakımından iyi bir örnek oluşturur. FIB(n) den görüleceği gibi satır 3 ve 4 de FIB($n-1$) ve FIB($n-2$) için karşılıklı olarak iki özyinelemeli çağırma birbirinden bağımsızdır: Onlar herhangi bir sırada çağrılabilir, ve hiçbir şekilde önce gerçekleştirilen hesaplama ötekini etkilemez. Böylece, iki özyinelemeli çağrı paralel olarak çalışabilir.

Biz **spawn** ve **sync** eşzamanlı anahtar sözcükleri ekleyerek kabakodumuzu işaret edilen paralellığe genişletiyoruz. İşte dinamik çoklu iş parçalamayı kullanmak için FIB prosedürü nasıl yeniden yazabileceğimiz:

P-FIB(n)

```

1  if  $n \leq 1$ 
2      return  $n$ 
3  else  $x = \text{spawn P-FIB}(n-1)$ 
4       $y = \text{P-FIB}(n-2)$ 
5      sync
6      return  $x + y$ 
```

spawn ve **sync** eşzamanlı anahtar sözcükleri P-FIB den sildiğimizde sonuç kabakod metninin FIB a özdeş olacağına dikkat ediniz (başlığındaki ve iki özyinelemeli çağrıdaki prosedürü yeniden adlandırma işlemi dışında). Bir çoklu iş parçacıklı algoritmanın *serileştirilmesini* (*serialization*) çoklu iş parçacıklı anahtar kelimeler olan **spawn**, **sync** ve paralel döngüleri incelediğimizde **parallel** sözcüklerini silmemizin sonucu olan seri algoritma olarak tanımlarız. Gerçekten de, bizim çoklu iş parçacıklı kabakodumuz, bir serileştirme her zaman aynı problemi çözen sıradan seri kabakoddur gibi hoş bir özelliğe sahiptir.

Satır 3 de olduğu gibi **spawn** anahtar kelimesi bir prosedür çağrısından (procedure call) önce geldiğinde *iç içe paralellik* (*Nested parallelism*) ortaya çıkar.

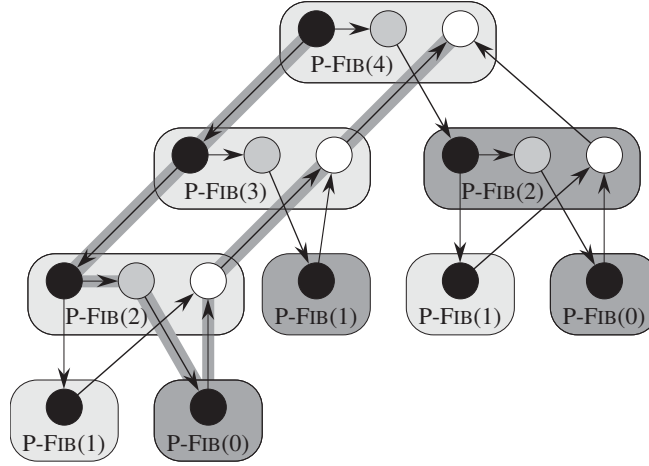
Bir seri yürütmeye normalde olduğu gibi, bir yumurtanın semantiği yumurtayı yürüten o prosedür örneğindeki sıradan bir prosedür çağrısından farklıdır yumurta—*ebeveyn (parent)*—oluşmuş alt programa- onun *çocuğu (child)*—bitirmek için çocuğu beklemek yerine paralel olarak yürütmeye devam edebilir. Bu durumda, doğurulan çocuk $P-FIB(n-1)$ i hesaplarken, satır 4 de doğurulan çocuğa paralel olarak ebeveyn $P-FIB(n-2)$ yi hesaplamaya devam edebilir. $P-FIB$ prosedürü özyinelemeli olduğundan tüm hesaplamalara paralel olarak, bu iki altprogram, çocuklarının yaptığı gibi, iç içe paralellliği oluşturan kendilerini çağırır, böylece alt hesaplamaların potansiyel olarak büyük bir ağacı oluşturulur.

Bununla birlikte **spawn** anahtar kelimesi bir prosedür onun doğurulan çocuğu ile eş zamanlı olarak yürütmesi gerektiğini söylemez, sadece bu kadar olabilir. Eşzamanlılık anahtar kelimeleri paralel olarak hesaplamanın hangi parçalarının yürütülebileceğini gösteren hesaplamanın *mantıksal paralellliğini (logical parallelism)* ifade eder. Çalışma zamanında, hangi althesaplamaların hesaplama izlerken, onları kullanılabilir işlemcilerle eşleştirerek gerçekten eşzamanlı olarak çalışabileceğini belirlemek için bir *zamanlayıcı* kadardır. Çizelgeleyicinin arkasındaki teoriyi kısaca tartışacağız.

Bir prosedür satır 5 de olduğu gibi, o bir **sync** deyimini yürütme sonrasına kadar çocuğundan döndürülen değeri güvenli bir şekilde kullanamaz. **sync** anahtar kelimesi, prosedürün onun tüm doğurulan çocuklarının eşitlemeden sonra ifadeye geçmeden önce tamamlanması için yeterince beklemesini işaret eder. $P-FIB$ prosedüründe, x hesaplanmadan önce eğer x ve y toplansaydı, oluşan anomali önlemek için satır 6 daki **return** ifadesinden önce bir **sync** gerekir. **sync** ifadesinin sağladığı açık eşitlemeye ek olarak, geri döndürülmeden önce her prosedür dolaylı olarak bir **sync** yürütür, böylece o yapmadan önce onun tüm çocuklarının sonlandırılması sağlanır.

Çoklu iş parçacığı yürütmek için bir model

Bir *çoklu iş parçacıklı hesaplamayı* —çoklu iş parçacıklı bir programın adına bir işlemci tarafından yürütülen çalışma zamanı talimatlar seti—*hesaplanabilir yönlü çevrimsiz çizge* olarak isimlendirilen bir hesaplanabilir çevrimsiz yönlü çizge $G = (V, E)$ olarak düşünmeye yardımcı olur. Örneğin, Şekil 27.2 $P-FIB(4)$ ün hesaplanmasından ortaya çıkan hesaplanabilir yönlü çevrimsiz çizgeyi gösterir. Kavramsal olarak, V deki tepeler talimatlardır ve E deki ayrıtlar talimatları arasındaki bağımlılıkları temsil eder, burada $(u, v) \in F$, u talimatı v talimatından önce yürütülmelidir anlamındadır. Bununla birlikte, kolaylık sağlamak için eğer bir talimatlar zinciri hiçbir paralel kontrol (**spawn**, **sync**, ya da bir yumurtadan **return**-ya açık bir **return** deymi veya bir prosedürün sonuna ulaşan örtülü meydana gelen bir dönüş) içermiyorsa, biz onları tek bir *dizilim (strand)* halinde, ki bunların her biri, bir ya da daha fazla talimat temsil eder, gruplayabiliriz. Paralel kontrolü içeren talimatlar dizilimlere dahil değildir, ancak yönlü çevrimsiz çizge yapısında temsil edilirler. Örneğin, bir dizilimin iki ardılı varsa, bunlardan biri doğurulan olmalıdır ve çoklu öncüllerle bir öncüllerle bir **sync** komutuyla bağlandığını gösterir. Böylece genel durumda, V kümesi dizilimlerin kümesini şe-



Şekil 27.2 P-FIB(4) ün hesaplanmasını gösteren bir yönlü çevrimsiz çizge. Herbir daire bir dizilimi gösterir, siyah dairelerle ya temel durumlar ya da satır 3 deki P-FIB($n - 1$) in doğumunun olduğu prosedürün(örnek) bir kısmını gösterilir, gölgeli daireler satır 5 deki **sync** e kadar, burada o P-FIB($n - 1$) in doğumu geri dönene kadar askıya alır, satır 4 de P-FIB($n - 2$) yi çağırarak prosedürün bir parçasını gösterir, ve beyaz daireler sonucu döndüren noktaya kadar x ve y yi toplayan **sync** den sonraki prosedürün parçasını gösterir. Aynı prosedüre ait olan dizilimlerin her bir grubu, doğmuş prosedürler için açık gölgeli ve çağırılan prosedürler için koyu gölgeli yuvarlak köşeli bir dikdörtgenle çevrilidir. Üreten ayrıtlar ve çağırılan ayrıtlar aşağı yöne bakar, devam eden ayrıtlar yatay olarak sağa bakar ve geri döndürülen ayrıtlar yukarı bakar. Her dizilimin birim zaman sürdüğünü varsayarak 17 dizilim olduğu için çalışma, 17 zaman birimine eşittir ve gölgeli ayrıtlarla gösterilen kritik yol 8 dizilim içerdiğinden yayılma 8 birim zamandır.

killedir, ve yönlü ayrıtların E kümesi paralel kontrol ile indüklenen dizilimler arasında bağımlılıkları temsil eder. G , u diziliminden v dizilimine bir yönlü yol içerdiğinde biz iki dizilim (**mantıksal olarak**) **seridir** deriz. Aksi durumda u ve v dizilimleri (**mantıksal olarak**) **paraleldir**.

Prosedür örneklerinin bir ağacı içine gömülü dizilimlerin bir yönlü çevrimsiz çizge olarak, bir çoklu iş parçacıklı hesaplamanın resmini yapabiliriz. Örneğin, Şekil 27.1 dizilimleri gösteren detaylı yapı olmaksızın P-FIB(6) için prosedür örnekleri ağacını gösterir. Şekil 27.2 herbir prosedürü oluşturan dizilimleri gösteren, bu ağacın bir bölümü üzerinde yakınlaştırma yapar. Dizilimleri birleştiren tüm yönlü ayrıtlar ya bir prosedürle veya prosedür ağacındaki yönsüz ayrıtlarla beraber çalışır.

Biz çeşitli dizilimler arasındaki bağımlılıkların türünü belirtmek için bir yönlü çevrimsiz çizgenin ayrıtlarını sınıflandırabiliriz. Şekil 27.2 de yatay çizilen bir **devam eden ayrıt** (u, u'), aynı prosedür örneğiyle bir u dizilimini onun ardılı olan u' dizilimine birleştirir. Bir u dizilimi bir v dizilimini ürettiğinde yönlü çevrimsiz çizge şeklinde aşağı yönü gösteren bir **doğuran ayrıt** (u, v) içerir. Normal prosedür çağrılarını gösteren **çağrı ayrıtları** da aşağı yönü gösterir. v dizilimini üreten u dizilimi v yi çağırarak u dan farklıdır öyle ki bir yumurta onun prosedüründeki u yu

takip eden u' den u ya giden bir yatay devam eden ayrıtları indükler, aynı zamanda v de olduğu gibi u' yürütmede serbesttir halbuki bir çağrı böyle hiçbir ayrıtları indüklemeyebilir. Bir u dizilimini kendi çağırma prosedürünü geri döndürdüğünde ve x çağırma prosedüründeki bir sonraki **sync** i hemen takip eden dizilimini olduğunda, yönlü çevrimsiz çizge yukarıyı gösteren **return edge**(u, x) i içerir. Bir hesaplama tek bir **başlangıç dizilimi** —Şekil 27.2 de P-FIB(4) etiketli prosedürdeki siyah tepe— ile başlar ve tek bir **son dizilim** —Şekil 27.2 de P-FIB(4) etiketli prosedürdeki beyaz tepe— ile sonlanır.

Biz bir işlemci kümesi ve bir **dizisel tutarlı** paylaşımlı bellekten oluşan **uygun bir paralel bilgisayarda** çoklu parçacıklı algoritmaların yürütülmesini çalışacağız. Dizisel tutarlılık, aynı anda işlemcilerden saklanan ve birçok yüklemeyi gerçekte yerine getiren paylaşımlı hafızanın her bir adımda olduğu gibi aynı sonucu üretmesi anlamındadır. Yani, hafıza, her işlemcinin kendi talimatlarını verdiği bireysel siparişleri koruyan bazı küresel doğrusal emirlere göre sırayla yürütülüyormuş gibi davranır. Eş zamanlı platformlar tarafından otomatik olarak işlemciler üzerinde çizelgelenen dinamik çoklu iş parçacıklı hesaplamalar için paylaşımlı hafıza çoklu iş parçacıklı hesaplamaların talimatları hesaplanabilir çevrimsiz yönlü çizgenin kısmi sırasını koruyan bir doğrusal düzeni sağlayacak veri ile denetim bilgisini karıştırmış gibi davranır. Çizelgelemeye bağlı olarak, düzenleme programın bir çalışmasından diğerine farklı olabilir ama herhangi bir yürütmenin davranışı hesaplanabilir çevrimsiz yönlü çizgeli bazı doğrusal tutarlı düzen içinde talimatların yürütüldüğü varsayımı ile anlaşılabilir.

Semantikler hakkındaki varsayımları üretmeye ek olarak, uygun-paralel-bilgisayar modeli bazı performans varsayımları yapar. Özel olarak, makinedeki her bir işlemcinin eşit hesaplama gücüne sahip olduğunu varsayar ve çizelgeleme maliyetini ihmal eder. Bu son varsayım iyimser görünse de yeterli “paralellik” (yeri geldiğinde tam olarak tanımlayacağımız bir terim) algoritmalar için çizelgeleme yükünün pratikte genellikle az olduğu ortaya çıkar.

Performans ölçüleri

Biz iki metrik kullanarak çoklu iş parçacıklı bir algoritmanın teorik verimliliğini ölçebiliriz: “iş yükü” ve “süre”. Bir çoklu iş parçacıklı hesaplamanın **iş yükü**, bir işlemci üzerindeki tüm hesaplamanın yürütülmesi için geçen toplam süredir. Bir başka deyişle iş yükü, çalışma dizilimlerinin her biri tarafından kullanılan sürelerin toplamıdır. Her bir dizilimin birim zaman sürdüğü bir hesaplanabilir çevrimsiz yönlü çizge için iş yalnızca çizgedeki tepelerin sayısıdır. **Süre**, çizgedeki herhangi bir yol boyunca dizilimlerin yürütülmesi için geçen en uzun zamandır. Tekrar, her bir dizilimin birim zaman kullandığı bir çizge için süre, çizgedeki **kritik yol** veya bir en uzun yol üzerindeki tepe sayısına eşittir. (Kısım 24.2 den $\Theta(V + E)$ süresinde bir $G = (V, E)$ çizgesinde bir kritik yol bulabileceğimizi hatırlayın). Ör-

neğin, Şekil 27.2 deki çizge tümü 17 tepe ve onun kritik yolu üzerindeki 8 tepeye sahiptir, böylelikle, eğer her bir dizilim birim zaman kullanırsa onun iş yükü 17 birim zaman ve onun süresi 8 birim zamandır.

Bir çoklu iş parçacıklı hesaplamanın gerçek çalışma zamanı sadece onun iş ve süresine değil, kaç tane işlemcinin kullanılabilir olduğuna ve çizelgeleyicinin dizilimleri işlemcilere nasıl paylaştığına da bağlıdır. P işlemci üzerindeki bir çok iş parçacıklı hesaplamanın çalışma süresini göstermek için P altindisini kullanacağız. Örneğin, P işlemci üzerindeki bir algoritmanın çalışma süresini T_P ile gösterebiliriz. İş yükü, tek bir işlemci üzerindeki çalışma süresidir veya T_1 dir. Eğer onun kendi işlemcileri üzerindeki her bir dizilimini çalıştırabilseydik—bir başka deyişle, eğer limitsiz sayıda işlemciye sahip olsaydık süre, çalışma zamanı olurdu ve böylelikle süreyi T_∞ ile gösterirdik.

İş yükü ve süre, P işlemci üzerindeki bir çoklu iş parçacıklı hesaplamanın T_P çalışma süresinin alt sınırlarını verir:

- Bir adımda, P işlemcili uygun bir paralel bilgisayar en fazla P birim iş yükü yapabilir, ve böylece T_P zamanında, o en fazla PT_P iş yükü gerçekleştirebilir. Yapılacak toplam iş T_1 olduğundan $PT_P \geq T_1$ elde ederiz. P ile sadeleştirildiğinde **iş yükü yasası** (*work law*) elde edilir:

$$T_P \geq T_1/P. \quad (27.2)$$

- P -işlemcili uygun bir paralel bilgisayar herhangi limitsiz işlemcili bir makineden daha hızlı çalışmaz. Başka bir açıdan bakıldığında, limitsiz işlemcili bir makine sadece onun P işlemcisini kullanarak P -işlemcili bir makineyi taklit edebilir. Böylelikle **süre yasası** (*span law*) şu şekilde elde edilir:

$$TP \geq T_\infty. \quad (27.3)$$

P işlemci üzerindeki hesaplamanın 1 işlemci üzerindekiinden kaç defa daha hızlı olduğunu söyleyen T_1/T_P oranını kullanarak P işlemci üzerindeki bir hesaplamanın **hızlanmasını** (*speedup*) tanımlarız. İş yükü yasasıyla, $T_1/T_P \leq P$ eşitsizliğini gerektiren $T_P \geq T_1/P$ eşitsizliğini elde ederiz. Böylece P işlemci üzerindeki hızlanma en fazla P olabilir. Hızlanma, işlemci sayısına göre doğrusal olduğunda, yani $T_1/T_P = \Theta(P)$, hesaplama **doğrusal hızlanma** sergiler, ve $T_1/T_P = P$ olduğunda **mükemmel doğrusal hızlanma** elde ederiz.

Süre için T_1/T_∞ çalışma oranı çoklu iş parçacıklı hesaplamanın **paralelliğini** verir. Paralelliği 3 perspektiften görüntüleyebiliriz. Bir oran olarak, paralellik kritik yol boyunca her bir adım için paralel olarak gerçekleştirilen ortalama iş yükü miktarını gösterir. Bir üst sınır olarak, paralellik herhangi sayıda işlemci üzerinde ulaşılan olası maksimum hızlanmayı verir. Son olarak ve belki de en önemli olanı, paralellik mükemmel doğrusal hızlanmaya ulaşma olasılığı üzerinde bir limit sağlar. Özel olarak, işlemci sayısı paralelliği aştığında, hesaplamanın mükemmel

doğrusal hızlanmaya ulaşması mümkün olamaz. Bu son noktayı görmek için süre kanununun gerektirdiği hızlanmanın $T_1/T_P \leq T_1/T_\infty < P$ eşitsizliğini sağlaması durumunda, $P > T_1/T_\infty$ olduğunu varsayalım. Üstelik, ideal bir paralel bilgisayardaki P işlemci sayısı paralellliği büyük ölçüde aşarsa — yani $P \gg T_1/T_\infty$ olursa — o zaman $T_1/T_P \ll P$ dir ve böylece hızlanma işlemci sayısından çok daha azdır. Bir başka deyişle, paralellik ötesinde kullandığımız daha fazla işlemci, daha az mükemmel hızlanma.

Bir örnek olarak, Şekil 27.2 deki P-FIB(4) hesaplamasını ele alalım, ve her bir dizilimin birim zaman sürdüğünü varsayalım. İş $T_1 = 17$ ve süre $T_\infty = 8$ olduğundan, paralellik $T_1/T_\infty = 17/8 = 2.125$ dir. Sonuç olarak 2 katdan daha fazla hızlanmaya ulaşmak mümkün değildir, hesaplamayı yürütmek için kaç işlemci kullandığımızın bir önemi yoktur. Bununla birlikte, daha büyük giriş boyutları için P-FIB(n) in önemli paralellik sergilediğini göreceğiz.

P işlemcili bir ideal paralel bilgisayar üzerinde yürütülen bir çoklu iş parçacıklı hesaplamasının **paralel gevşeklik** (*parallel slackness*) kavramını $(T_1/T_\infty)/P = T_1/(PT_\infty)$ oranı olarak tanımlarız ki bu makinedeki işlemci sayısını aşan hesaplamasının paralellliği ile çarpımıdır. Buna göre eğer gevşeklik 1'den küçükse mükemmel doğrusal hızlanmayı elde etmeyi umamayız, çünkü $T_1/(PT_\infty) < 1$ ve P işlemci üzerindeki hızlanmadan kaynaklanan süre kanunu $T_1/T_P \leq T_1/T_\infty < P$ eşitsizliğini sağlar. Gerçekten, gevşeklik 1 den 0 a doğru azalırken hesaplamasının hızlanması mükemmel doğrusal hızlanmadan git gide uzaklaşır. Bununla beraber, eğer gevşeklik 1 den büyük olursa işlemci başına düşen iş yükü sınırlayan kısıttır. Göreceğimiz gibi, gevşeklik 1 den artarken, iyi bir çizelgeleyici mükemmel doğrusal hızlanmaya git gide yaklaşmayı başarabilir.

Çizelgeleme

İyi performans sadece iş yükü ve süreyi küçültmekten daha fazlasına bağlıdır. Dizilimlerin de paralel makinanın işlemcileri üzerine mükemmel bir şekilde çizelgelenmiş olmalıdır. Bizim çoklu iş parçacıklı programlama modelimiz hangi dizilimin hangi işlemcide yürütüleceğini belirleyen hiçbir yol sağlamaz. Bunun yerine biz tek işlemciye dinamik şekilde ortaya çıkan hesaplamayı planlamak için eşzamanlı platformun çizelgeleyicisine güveniriz. Pratikte, çizelgeleyici dizilimler durağan dizilimlere eşler ve işletim sistemi dizilimleri işlemcilerin kendileri üzerinde durağan, ama bu yönlendirme ekstra seviyesi çizelgeleme anlayışımız için gereksizdir. Eşzamanlılık platformu zamamlayıcısının dizilimleri işlemcilere doğrudan eşleştirdiğini sadece hayal edebiliriz.

Bir çoklu iş parçacıklı çizelgeleyici dizilimler ortaya çıktığında veya onlar tamamladığında ön bilgisiz hesaplamayı çizelgelemelidir — o, **çevrimiçi** çalışmalıdır. Üstelik, iyi bir çizelgeleyici dağıtılmış bir şekilde çalışır ki burada çizelgeleyiciyi uygulayan dizilimler hesaplama yük-dengesi için işbirliği yapar. İyi kanıtlanabilir çevrimiçi, dağıtılmış çizelgeleyiciler vardır ancak onların analizi karmaşıktır.

Bunun yerine, analizimizi basit tutmak için biz verilen herhangi bir zamanda hesaplamanın küresel durumunu bilen bir çevrimiçi **merkezi** çizelgeleyiciyi araştıracağız. Özellikle, her bir zaman adımında mümkün oldukça birçok dizilimi işlemcilerle eşleştiren **açgözlü çizelgeleyicileri** analiz edeceğiz. En az P dizilimi bir zaman adımı sırasında çalıştırmak için hazırsa, biz adımın bir **tamamlama adımı** olduğunu söyleriz, ve bir açgözlü çizelgeleyici hazır herhangi P dizilimi işlemcilerle eşleştirir. Aksi takdirde, P den daha az dizilim çalıştırmak için hazırsa, bu durumda biz adımın bir **eksik adım** olduğunu söyleriz, ve bir çizelgeleyici hazır her bir dizilimi kendi işlemcisiyle eşleştirir.

İş yükü yasasına göre, P işlemci üzerinde umabildiğimiz en iyi çalışma süresi $T_P = T_1/P$ dir ve süre yasasına göre, umduğumuzun en iyisi $T_P = T_\infty$ dir. Aşağıdaki teorem umduğumuzun en iyisi onun bir üst sınır olarak bu iki alt sınırın toplamına ulaşması için iyi kanıtlanabilir olduğunu gösteriyor.

Teorem 27.1

P işlemcili bir uygun bilgisayar üzerinde bir açgözlü çizelgeleyici T_1 iş ve T_∞ süreli bir çoklu iş parçacıklı hesaplamayı

$$T_P \leq T_1/P + T_\infty \quad (27.4)$$

zamanında çalıştırır.

Kanıt Tamamlama adımları dikkate alarak başlıyoruz. Herbir tamamlama adımında, P işlemci bir P iş yükünün tamamını birlikte gerçekleştirir. Çelişki amacı için tamamlama adımlarının sayısının $\lfloor T_1/P \rfloor$ den büyük olduğunu varsayalım. O zaman tamamlama adımlarının toplam iş yükü en azından

$$\begin{aligned} P \cdot (\lfloor T_1/P \rfloor + 1) &= P \lfloor T_1/P \rfloor + P \\ &= T_1 - (T_1 \bmod P) + P \quad ((3.8) \text{ denkleminde}) \\ &> T_1 \quad ((3.9) \text{ eşitsizliğinden}) \end{aligned}$$

olur. Böylece P işlemci tamamlama adımlarının sayısının en fazla $\lfloor T_1/P \rfloor$ olduğu sonucuna ulaşmamızı sağlayan hesaplamanın gerektirdiği iş yükünden daha fazlasını gerçekleştirdi çelişkisini elde ederiz.

Şimdi, tamamlanmamış adımlar göz önüne alalım. G , tüm hesaplamayı gösteren yönlü çevrimsiz çizge olsun, ve genelliği bozmadan, her bir dizilimin birim zaman sürdüğünü varsayalım. (Biz daha uzun her bir dizilim birim zaman süren dizilimlerin bir zinciri ile değiştirebiliriz) Tamamlanmamış adımın henüz başında yürütülecek G nin altçizgesi G' olsun ve tamamlanmamış adımdan sonra yürütülecek kalan altçizge G'' olsun. Bir çevrimsiz yönlü çizgedeki bir en uzun yol mutlaka 0-derecedeki bir tepe ile başlamalıdır. Bir açgözlü çizelgeleyicinin bir tamamlanmamış adımı G' deki 0-derecedeki tüm dizilimleri yürüttüğü için G'' deki bir en uzun yolun boyu G' deki bir en uzun yolun boyundan 1 az olmalıdır. Başka bir deyişle, bir tamamlanmamış adım, yürütülmemiş çevrimsiz yönlü çizgenin süresini 1 azaltır. Bundan dolayı, tamamlanmamış adımların sayısı en fazla T_∞ olur.

Mademki her bir adım ya tamamlama ya da tamamlanmamıştır, teorem kanıtlandı. ■

Teorem 27.1 in aşağıdaki doğal sonucu bir ağgözlü çizelgeleyicinin her zaman iyi bir performans sergilediğini gösterir.

Sonuç 27.2

P işlemcili uygun bir bilgisayar üzerinde ağgözlü bir çizelgeleyici tarafından çizelgelenmiş herhangi çoklu iş parçacıklı hesaplamanın T_P çalışma süresi optimal sürenin 2 katıdır.

Kanıt T_P^* P işlemcili bir makine üzerinde optimal bir çizelgeleyici tarafından üretilen çalışma süresi, T_1 ve T_∞ ise hesaplamanın sırasıyla iş ve süresi olsun. İş yükü ve süre yasası —(27.2) ve (27.3) eşitsizlikleri— bize $T_P^* \geq \max(T_1/P, T_\infty)$ olduğunu verdiğinden Teorem 27.1

$$\begin{aligned} T_P &\leq T_1/P + T_\infty \\ &\leq 2 \cdot \max(T_1/P, T_\infty) \\ &\leq 2T_P^* \end{aligned}$$

■

olmasını gerektirir.

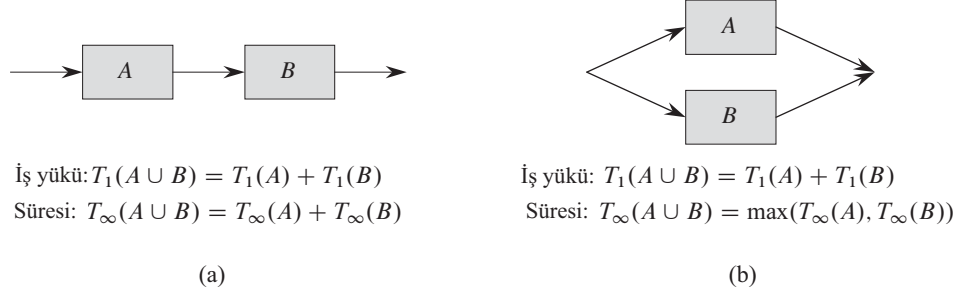
Sonraki sonuç bir ağgözlü çizelgeleyicinin gerçekte gevşeklik artarken herhangi çoklu iş parçacıklı hesaplama üzerinde mükemmelliğe yakın doğrusal hızlanmayı elde ettiğini gösterir.

Sonuç 27.3

T_P , P işlemcili bir uygun bilgisayar üzerinde bir ağgözlü çizelgeleyici tarafından üretilmiş bir çoklu iş parçacıklı hesaplamanın çalışma zamanı ve T_1 ve T_∞ ise hesaplamanın sırasıyla iş yükü ve süresi olsun. O zaman, eğer $P \ll T_1/T_\infty$ ise $T_P \approx T_1/P$ olduğunu veya buna denk yaklaşık olarak P kadarlık bir hızlanma elde ederiz.

Kanıt Eğer $P \ll T_1/T_\infty$ olduğunu varsayarsak, o zaman $T_\infty \ll T_1/P$ olduğunu da elde ederiz, ve dolayısıyla Teorem 27.1 bize $T_P \leq T_1/P + T_\infty \approx T_1/P$ olduğunu verir. (27.2) iş yükü yasasının $T_P \geq T_1/P$ olduğunu dikte ettiğinden, $T_P \approx T_1/P$ sonucuna veya buna denk olarak hızlanmanın $T_1/T_P \approx P$ olduğu sonucuna ulaşırız. ■

$\ll$ sembolü “daha az,” anlamına gelir ama “daha az” ne kadardır? Pratik bir kural olarak, en az 10 luk bir—yani, işlemcilerden 10 kat daha fazla paralellik—genelde iyi hızlanma elde etmek için yeterlidir. Daha sonra, ağgözlülük sınırındaki süre terimi, (27.4) eşitsizliği, çoğu mühendislik durumları için yeterince iyi olan işlemci-başına-düşen-iş yükü teriminin %10 dan daha azdır. Örneğin, eğer bir hesaplama sadece 10 veya 100 işlemci üzerinde çalışırsa, 10.000 paralellik üzerinde 1.000.000 desek, hatta 100 fark faktörü ile bile paralellığe değer biçmek mantıklı değildir. Problem 27-2 nin gösterdiği gibi, bazen aşırı paralellliği diğer kaygılara göre daha iyi olan ve hala makul işlemci sayısı üzerinde iyi şekilde ölçeklenen algoritmaları elde edebiliriz.



Şekil 27.3 Oluşturulmuş alt hesaplamaların iş yükü ve süresi. **(a)** İki alt hesaplama seri olarak birleştirildiğinde, bileşimin iş yükü, onların iş yüklerinin toplamıdır ve bileşimin süresi, onların sürelerinin toplamıdır. **(b)** İki alt hesaplama paralel olarak birleştirildiğinde, bileşimin iş yükü, onların işlerinin toplamı kalır, ama bileşimin süresi, onların sürelerinin sadece maksimumudur.

Çoklu iş parçacıklı algoritmaların analizi

Artık çoklu iş parçacıklı algoritmaları analiz etmek için gerekli tüm aletlere sahibiz ve onların çeşitli sayıda işlemci üzerindeki çalışma sürelerinin sınırlarını iyice sağlarız. Sıradan bir seri algoritmanın çalışma süresinin analizinden ki bu ders kitabının çoğunun onun hakkında olmasından ona aşina olmalısınız! —yani, çoklu iş parçacıklı algoritmaların serileştirilmesi— başka bir şey tutmadığı için iş yükü analizi oldukça basittir. Süre analizi daha ilginçtir, ama genellikle onun anlamını bir kere aldığınızda zor değildir. Biz P-FIB programını kullanarak temel fikirleri araştıracağız.

P-FIB(n) in $T_1(n)$ iş yükünün analizi hiçbir engel teşkil etmez, çünkü biz onu zaten yapmıştık. Orijinal FIB prosedürü aslında P-FIB in serileştirmesidir ve bundan dolayı denklem (27.1) den $T_1(n) = T(n) = \Theta(\phi^n)$ dir.

Şekil 27.3 süreyi nasıl analiz edeceğimizi gösteriyor. Eğer iki alt hesaplama paralel olarak birleştirilirse, onların bileşiminin süresi bu iki alt hesaplamaların sürelerinin maksimumu oluyorken, eğer iki alt hesaplama seri olarak birleştirilirse, onların süresi onların bileşiminin süresini şekillendirmek için toplanır. P-FIB(n) için satır 3 deki P-FIB($n-1$) e oluşturulan çağrı satır 4 deki P-FIB($n-2$) çağrısıyla paralel çalışır. Bundan dolayı P-FIB(n) in süresini $T_\infty(n) = \Theta(n)$ çözümüne sahip olan

$$\begin{aligned} T_\infty(n) &= \max(T_\infty(n-1), T_\infty(n-2)) + \Theta(1) \\ &= T_\infty(n-1) + \Theta(1) \end{aligned}$$

özyinelemesi olarak ifade edebiliriz.

P-FIB(n) in paralelliği n büyürken önemli ölçüde büyüyen $T_1(n)/T_\infty(n) = \Theta(\phi^n/n)$ dir. Nitekim, en büyük paralel bilgisayarlar üzerinde bile n için mü-

tevazi bir değer, $P\text{-FIB}(n)$ için mükemmel doğrusal hızlanmanın yakına ulaşmak için yeterlidir, çünkü bu prosedür önemli paralel gevşeklik sergiler.

Paralel döngüler

Birçok algoritma tüm özyinelemelerin paralel olarak yürütüldüğü döngüler içerir. Göreceğimiz gibi, biz, **spawn** ve **sync** anahtar kelimelerini kullanarak bu döngüleri paralelleştirebiliriz, ama böyle döngüleri bunların özyinelemelerinin eşzamanlı çalıştırabildiği algoritmalar olarak doğrudan belirtmek için çok daha uygundur. Bizim kabakodumuz, bir **for** döngü deyimindeki **for** anahtar kelimesinin önüne geçen **parallel** eşzamanlı anahtar sözcüğüyle bu işlevselliği sağlar.

Bir örnek olarak, $n \times n$ boyutlu $A = (a_{ij})$ matrisinin n -boyutlu $x = (x_j)$ vektörüyle çarpılması problemini ele alalım. n -boyutlu $y = (y_i)$ sonuç vektörü $i = 1, 2, \dots, n$ için

$$y_i = \sum_{j=1}^n a_{ij}x_j$$

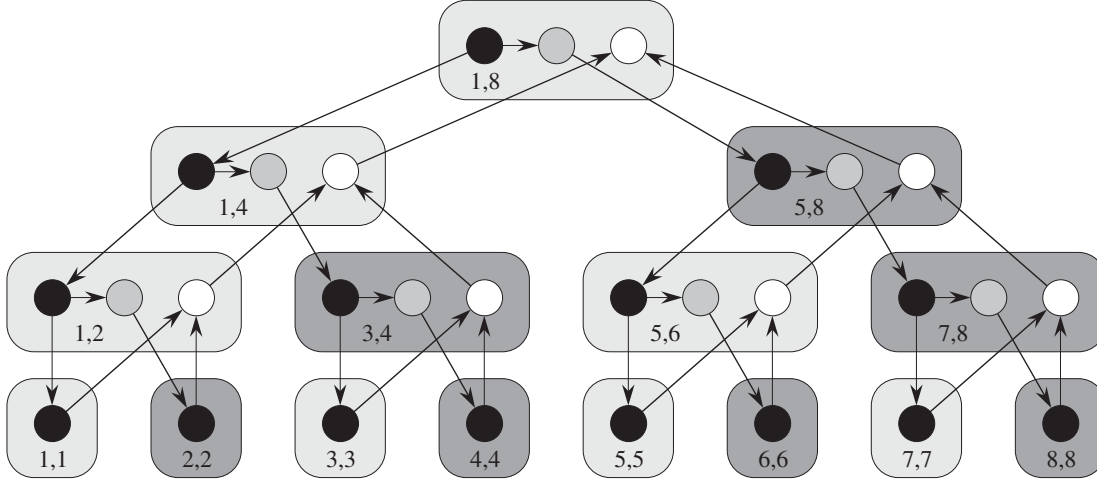
denklemleriyle verilir. y nin tüm elemanlarını paralel olarak aşağıdaki gibi hesaplayarak matris-vektör çarpımını gerçekleştirebiliriz:

```

MAT-VEC( $A, x$ )
1   $n = A.rows$ 
2   $y$   $n$  uzunluğunda yeni bir vektör olsun
3  parallel for  $i = 1$  to  $n$ 
4       $y_i = 0$ 
5  parallel for  $i = 1$  to  $n$ 
6      for  $j = 1$  to  $n$ 
7           $y_i = y_i + a_{ij}x_j$ 
8  return  $y$ 

```

Bu kodda, 3 ve 5 nci satırlardaki **parallel for** anahtar kelimesi ilgili döngülerin özyinelemelerinin eşzamanlı çalışabildiğini gösterir. Bir derleyici iç içe paralelliği kullanarak her bir **parallel for** döngüsünü böl-ve-yönet alt programı olarak uygulayabilir. Örneğin, 5-7 satırlardaki **parallel for** döngüsü, derleyicinin aşağıdaki gibi MAT-VEC-MAIN-LOOP yardımcı alt programını ürettiği MAT-VEC-MAIN-LOOP ($A, x, y, n, 1, n$) çağrısıyla gösterilebilir.



Şekil 27.4 MAT-VEC-MAIN-LOOP($A, x, y, 8, 1, 8$) in hesaplanmasını gösteren bir yönlü çevrimsiz çizge. Herbir yuvarlak köşeli dikdörtgendeki iki sayı prosedürün çağrısındaki (yumurta veya çağrı) son iki parametrenin (prosedür başlığındaki i ve i') değerini verir. Siyah daireler ya temel duruma ya da 5 nci satırdaki MAT-VEC-MAIN-LOOP in yumurtasına kadar olan prosedürün bir parçasına karşılık gelen dizimlileri gösterir; gölgeli daireler 6 ncı satırdaki MAT-VEC-MAIN-LOOP u çağrısından 7 nci satırdaki **sync** e kadar olan prosedürün parçasına karşılık gelen dizimlileri gösterir ki burada o, 5 nci satırdaki doğurulan altprogram geri dönene kadar askıda kalır; ve beyaz daireler **sync** den sonra onun geri döndüğü noktaya kadar olan prosedürün (önemsiz) parçasına karşılık gelen dizimlileri gösterir.

MAT-VEC-MAIN-LOOP(A, x, y, n, i, i')

```

1  if  $i == i'$ 
2      for  $j = 1$  to  $n$ 
3           $y_i = y_i + a_{ij}x_j$ 
4  else  $mid = \lfloor (i + i')/2 \rfloor$ 
5      spawn MAT-VEC-MAIN-LOOP( $A, x, y, n, i, mid$ )
6      MAT-VEC-MAIN-LOOP( $A, x, y, n, mid + 1, i'$ )
7      sync
```

Bu kod Şekil 27.4 de gösterildiği gibi, arta kalanların özel döngü iterasyonları olduğu bir ikilik yürütme ağacını oluşturarak özyinelemelerin ikinci yarısını paralel olarak yürütmek için döngü iterasyonlarının birinci yarısını özyinelemeli olarak doğurur ve sonra bir **sync** i yürütür.

$n \times n$ boyutlu bir matris üzerinde MAT-VEC 'in $T_1(n)$ işini hesaplamak için **parallel for** döngüsünü sıradan **for** döngüsüyle yer değiştirerek elde ettiğimiz onun serileştirilmesinin çalışma süresini basitçe hesaplarız. Böylece $T_1(n) = \Theta(n^2)$ eşitliğini elde ederiz, çünkü 5-7 satırlarındaki iki kat iç içe döngülerin ikinci de-

receden çalışma süresi baskın olur. Bununla beraber bu analiz paralel döngülerin yerine getirilmesinde özyinelemeli doğurma için yükü önemsemez gibi görünür. Aslında, özyinelemeli doğrumanın yükü paralel bir döngünün iş yükünü onun serileştirilmesiyle karşılaştırıldığında arttırır, ama asimptotik olarak değil. Nedenini görmek için özyinelemeli prosedür örneklerinin ağacı tam ikili ağaç olduğu için iç düğüm noktası sayısının yaprak sayısından (Alistırma B.5-3) 1 az olduğunu gözlemlemeliyiz. Her iç düğüm noktası özyineleme aralığı bölmek için sabit bir çalışma yapar, ve her yaprak, döngünün en azından sabit zaman alan (bu durum için $\Theta(n)$ süresi) bir özyinelemesine karşılık gelir. Böylece, genel çalışmaya en çok bir sabit çarpan kadar katkıda bulunarak, özyinelemelerin iş yüküne karşı özyinelemeli doğrumanın iş yükünü amortize edebiliriz.

Pratik bir mesele olarak, dinamik çoklu iş parçacıklı eşzamanlı platformlar bazen tek bir yaprakta birkaç iterasyon yürüterek ya otomatik olarak ya da programcının kontrolü altında özyineleme yapraklarını *irileştirir*, böylece özyinelemeli doğurmaların iş yükünü azaltır. Bununla birlikte, bu azaltılmış iş yükü, aynı zamanda paralelliği azaltmak pahasına geliyor, ama hesaplama yeterli paralel şekliğe sahipse, mükemmele yakın doğrusal hızlanma fedakarlık gerektirmez.

Paralel döngü yapısının süresini analiz ederken ayrıca özyinelemeli doğurmayı da göz önüne almalıyız. Özyinelemeli çağrı derinliği özyineleme sayısı bakımından logaritmik olduğundan, i . iterasyonu $iter_{\infty}(i)$ süresine sahip olan n iterasyonlu paralel bir döngü için süre

$$T_{\infty}(n) = \Theta(\lg n) + \max_{1 \leq i \leq n} iter_{\infty}(i)$$

olur. Örneğin, $n \times n$ boyutlu bir matris üzerinde MAT-VEC için 3–4 satırlarındaki paralel başlangıç döngüsü $\Theta(\lg n)$ süresine sahiptir, çünkü özyinelemeli doğurma her özyinelemenin sabit zamanlı iş yükünü bastırır. 5-7 satırlarındaki ikili iç içe döngülerin süresi $\Theta(n)$ dir, çünkü dıştaki **parallel for** döngüsü içteki (seri) **for** döngüsünün n iterasyonunu içerir. Prosedürdeki kodun kalanının süresi sabittir, ve böylece süre, tüm prosedür için genel bir $\Theta(n)$ süresini üreten iki kat iç içe döngüler ile bastırılır. İş yükü, $\Theta(n^2)$ olduğundan paralellik $\Theta(n^2)/\Theta(n) = \Theta(n)$ dir (Alistırma 27.1-6 daha da paralellik ile bir uygulama sağlamak için size sorar).

Yarış koşulları

Eğer çoklu iş parçacıklı bir algoritma talimatların çok çekirdekli bilgisayarda nasıl planlandığının önemi olmaksızın aynı giriş üzerinde her zaman aynı şeyi yaparsa o *belirleyicidir*. Eğer onun davranışı çalışmadan çalışmaya değişiyorsa o *belirsizdir*. Genellikle, belirleyici olması amaçlanan bir çoklu iş parçacıklı algoritma başarısız olur çünkü o bir “belirleme yarışı” içerir.

Yarış koşulları eşzamanlı felakettir. Ünlü yarış hataları üç kişiyi öldüren ve birkaç kişiyi yaralayan Therac-25 radyasyon tedavi makinesini ve 50 milyonu

aşkın kişiyi enerjisiz bırakan 2003 ün Kuzey Amerikan karartmasını içerir. Bu zararlı hataları bulmak herkesin bildiği gibi zordur. Sadece yazılımınızın alanda aralıklı şekilde çıktığını keşfettiğiniz testleri günlerce laboratuvarda bir başarısızlık olmadan çalıştırabilirsiniz.

İki mantıksal paralel talimatlar aynı bellek konumuna erişebildiğinde ve talimatların en az biri bir yazma gerçekleştirdiğinde bir **belirleme yarışı** oluşur. Aşağıdaki prosedür, bir yarış koşulunu göstermektedir:

RACE-EXAMPLE()

```

1   $x = 0$ 
2  parallel for  $i = 1$  to 2
3       $x = x + 1$ 
4  yaz  $x$ 

```

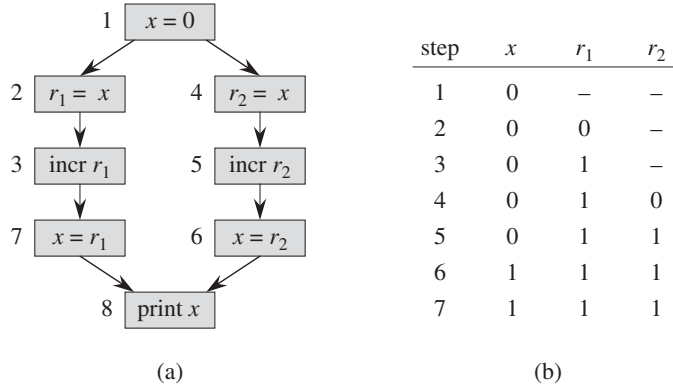
satır 1 deki x e 0 atamasından sonra RACE-EXAMPLE satır 3 de herbiri x i arttıran iki paralel dizilim oluşturur. RACE-EXAMPL her zaman 2 değerini yazdırıyor gibi görünse de (onun serileştirme kesinlikle öyledir), yerine o 1 değerini yazdırır. Bu anomalinin nasıl oluştuğunu görelim.

Bir işlemci x i arttırırken işlem bölünmez değildir, ama bir dizi talimattan oluşur:

1. Bellekten işlemcinin kayıtlarının birine x i oku.
2. Kayıttaki değeri arttır.
3. Kayıttaki değeri bellekteki x 'e geri yaz.

Şekil 27.5(a) dizilimlerin tek tek bireysel talimatlara kırıldığı RACE-EXAMPL nin çalışmasını gösteren bir yönlü hesaplama çizgesini örnekliyor. Hatırlayın ki, bir uygun paralel bilgisayar dizisel tutarlılığı desteklediği için, yönlü çizgedeki bağımlılıkları baz alan dönüşümlü talimatları olarak çoklu iş parçacıklı bir algoritmanın paralel yürütülüşünü görüntüleyebiliriz. Şeklin (b) bölümü anomali ortaya çıkaran hesaplamanın bir yürütülüşündeki değerleri gösteriyor. x değeri bellekte saklanır, ve r_1 ve r_2 işlemci kayıtlarıdır. Adım 1 de işlemcilerden biri 0 ı x e atar. r_1 deki 1 değeri üretilirken adım 2 ve 3 de, işlemci 1 bellekten x i ve onun r_1 kaydını okur ve onu arttırır. O noktada, işlemci 2, 4–6 talimatlarını yürüterek resme girer. r_2 deki 1 değeri üretilirken işlemci 2 bellekten x i ve onun r_2 kaydını okur ve onu arttırır, ve o zaman 1 i x e atayarak bu değeri x de saklar. Şimdi, işlemci 1, r_1 deki 1 değerini x in değerini değişmeden bırakan x de saklayarak adım 7 ile kaldığı yerden devam eder. Böylece adım 8, serileştirme yazmış gibi, 2 yerine 1 değerini yazar.

Ne olduğunu görebiliyoruz. Paralel yürütmenin etkisi, işlemci 2 den önce işlemci 1 onun tüm talimatlarını yürütmesi şeklinde olsaydı, 2 değeri yazılırdı.



Şekil 27.5 RACE-EXAMPLE ndeki belirleme yarışının örneklenmesi. (a) Bir yönlü çevrimsiz hesaplama çizgesinin bireysel talimatlar arasındaki bağımlılığı göstermesi. İşlemci kayıtları r_1 ve r_2 dir. Döngü kontrolünün uygulanması gibi yarış ile bağlantılı olmayan, talimatlar ihmal edilir. (b) Yürütme dizisindeki her adım için hatayı ortaya çıkaran n yürütme dizisi, bellekteki x değerini ve r_1 ve r_2 kayıtlarını göstermektedir.

Tersine, Etki, işlemci 1 den önce işlemci 2 tüm talimatları yürütsün şeklinde olsaydı, 2 değeri hala yazılıyor olurdu. Bununla birlikte, iki işlemcinin talimatları aynı anda yerine getirildiğinde, bu örnekteki yürütmede olduğu gibi, x in güncelleştirmelerinin birinin kaybolması mümkündür.

Elbette, birçok yürütme hata ortaya çıkartmaz. Örneğin, yürütme sırası $\langle 1, 2, 3, 4, 5, 6, 7, 8 \rangle$ veya $\langle 1, 4, 5, 6, 2, 3, 7, 8 \rangle$ şeklinde olsaydı, biz doğru sonucu elde ederdik. Belirleme yarışı problemi budur. Genellikle, birçok sıralama doğru sonucu üretir— sol taraftaki herhangi bir talimatın sağ tarftakilerden önce yürütülmesi gibi veya tersine. Ama bazı sıralamalar talimatlar geri döndürüldüğünde yanlış sonuçlar üretir. Sonuç olarak, yarışları test etmek son derece zor olabilir. Günlerce testleri çalıştırabilir ve sonuç kritik olduğunda alandaki bir felaket sistem çökmesi deneyimi dışında asla hatayı göremezsiniz.

Çeşitli yollarla yarışların üstesinden gelmemize rağmen, amacımıza yönelik karşılıklı içerme kilitleri (mutual-exclusion locks) ve diğer eşitleme yöntemlerinin kullanımını içererek, biz basitçe şundan emin olacağız ki paralel olarak yürütülen dizilimler **bağımsızdır**: onların aralarında hiçbir belirlilik yarışı yoktur. Böylece inşa edilen bir **parallel for**’da tüm iterasyonlar bağımsız olmalıdır. Bir **spawn** ve buna karşılık gelen **sync** arasında, doğan çocuğun kodu doğurulan veya çağrılmış ek çocuk tarafından gerçekleştirilen kodu içeren ebeveynin kodundan bağımsız olmalıdır. Not edelim ki asıl doğum oluşmadan önce doğurulan bir çocuğun argümanları ebeveynde değerlendirilir, ve böylece doğurulan bir alt programın argümanlarının değerlendirilmesi doğumdan sonra o argümanlara herhangi erişimler ile sıralıdır.

Yarışlı kodları üretmenin ne kadar kolay olduğuna örnek olarak, işte iç **for** döngüsünü paralelleştirerek bir $\Theta(\lg n)$ süresine ulaşan çoklu iş parçacıklı matris-vektör çarpımının hatalı bir uygulaması:

MAT-VEC-WRONG (A, x)

```

1   $n = A.rows$ 
2   $y, n$  uzunluğunda yeni bir vektör olsun
3  parallel for  $i = 1$  to  $n$ 
4       $y_i = 0$ 
5  parallel for  $i = 1$  to  $n$ 
6      parallel for  $j = 1$  to  $n$ 
7           $y_i = y_i + a_{ij} x_j$ 
8  return  $y$ 
```

Maalesef bu prosedür j nin tüm n değerleri için eş zamanlı yürütülen satır 7 de y_i nin güncellenmesindeki yarışlar yüzünden doğru değildir. Alıştırma 27.1-6 sizden $\Theta(\lg n)$ süreli doğru bir uygulamayı vermenizi istiyor.

Yarışları olan bir çoklu iş parçacıklı algoritma bazen doğru olabilir. Örneğin, iki paralel dizilim paylaşımlı bir değişkende aynı değeri saklayabilir ve hangi değer in önce saklandığının bir önemi olmaz. Bununla beraber, genellikle yarış içeren kodu usulsüz kabul edeceğiz.

Bir satranç dersi

Bu bölümü dünya-sınıfı çoklu iş parçacıklı satranç-oynayan program olan *Socrates [80] in aşağıdaki zamanlamaları fuar için basitleştirilmiş olmasına rağmen gelişimi sırasında meydana gelen gerçek bir hikaye ile kapatıyoruz. Program, 32-işlemcili bir bilgisayar için prototipleştirildi ancak sonuçta 512 işlemcili bir süper bilgisayarda çalıştırıldı. Bir noktada, geliştiriciler programa onun 32-işlemcili makine üzerindeki çalışma süresini $T_{32} = 65$ saniyeden $T'_{32} = 40$ saniyeye önemli ölçüde azaltan bir optimizasyon ekledi. Ancak, geliştiriciler, 32 işlemci üzerindeki daha hızlı olan yeni versiyonun 512 işlemci üzerindeki orijinal versiyonundan aslında daha yavaş olduğu sonucuna varmak için, iş ve süre performans ölçülerini kullandı. Sonuç olarak, onlar “optimizasyon” yıkımına uğradılar.

İşte onların analizi. Programın orijinal versiyonu $T_1 = 2048$ saniye işine ve $T_\infty = 1$ saniye süresine sahipti. (27.4) eşitsizliğini bir denklem olarak düşünersek $T_P = T_1/P + T_\infty$ ve onu P işlemci üzerindeki çalışma süresinin yaklaşımı olarak kullanırsak, gerçekten $T_{32} = 2048/32 + 1 = 65$ olduğunu görürüz. Optimizasyon ile, iş $T'_1 = 1024$ saniye ve süre $T'_\infty = 8$ saniye oldu. Tekrar yaklaşımımızı kullanarak, $T'_{32} = 1024/32 + 8 = 40$ elde ederiz.

Bununla birlikte, 512 işlemci üzerindeki çalışma süresini hesapladığımızda, iki versiyonun bağıl hızları geçiş yapar. Özel durumda, $T_{512} = 2048/512 + 1 = 5$

saniye ve $T'_{512} = 1024/512 + 8 = 10$ saniye elde ederiz. 32 işlemci üzerindeki programı hızlandıran optimizasyon 512 işlemci üzerindeki programı 2 kat yavaşlatmaktadır! 32 işlemci üzerindeki çalışma süresi açısından baskın terim olmayan optimize edilen versiyonun 8 olan süresi daha çok işlemci kullanımı avantajını sıfırlayarak 512 işlemci üzerinde baskın terim oldu.

Hikayenin kıssadan hissesi şudur ki iş yükü ve süre hesaplanabilir çalışma sürelerinden tahmini performansın daha iyi bir yolunu sağlayabilir.

Alıştırmalar

27.1-1

P-FIB ın 4. satırındaki P-FIB($n - 2$) i kodda yapıldığı gibi onu çağırarak yerine doğurduğumuzu varsayalım. Asimptotik iş yükü, süre ve paralellik üzerindeki etkisi nedir?

27.1-2

P-FIB(5) in yürütülmesinden kaynaklanan yönlü çevrimsiz hesaplama çizgesini çiz. Hesaplamadaki her dizilimin birim zaman sürdüğünü varsayarsak, hesaplamanın iş yükü, süre ve paralelliği nedir? Her dizilim yürütülme sırasına göre zaman adımlarıyla etiketleyerek, 3 işlemci üzerindeki yönlü çizgenin açgözlü çizelgeleyiciyi kullanarak nasıl çizelgeneceğini göster.

27.1-3

Bir açgözlü çizelgeleyicinin Teorem 27.1 de kanıtlanan sınırdan az daha güçlü aşığıdaki zaman sınırına ulaştığını kanıtlayınız:

$$T_P \leq \frac{T_1 - T_\infty}{P} + T_\infty \quad (27.5)$$

27.1-4

Bir açgözlü çizelgeleyicinin bir yürütmesinin aynı sayıda işlemci üzerinde açgözlü bir çizelgeleyicinin bir diğer yürütme zamanından nerdeyse 2 kat sürebildiğini gösteren yönlü çevrimsiz hesaplama çizgesi oluşturun. İki yürütmenin nasıl devam edebildiğini açıkla.

27.1-5

Profesör Karan açgözlü bir çizelgeleyici kullanarak 4, 10 ve 64 işlemcili ideal bir paralel bilgisayarda belirleyici çoklu iş parçacıklı algoritmasını ölçüyor. 3 çalıştırmanın $T_4 = 80$ saniye, $T_{10} = 42$ saniye ve $T_{64} = 10$ saniye ile sonuçlandığını iddia ediyor. Profesörün beceriksiz biri mi yoksa yalan mı söylediğini tartışınız. (İpucu: (27.2) iş kanununu, (27.3) süre kanununu ve Alıştırma 27.1-3 deki eşitsizliğini kullanınız).

27.1-6

$n \times n$ boyutlu bir matris ile bir n -vektörünün $\Theta(n^2)$ iş yükü sağlanırken $\Theta(n^2 / \lg n)$ paralelliğine ulaşan, bir çoklu iş parçacıklı çarpım algoritmasını veriniz.

27.1-7

$n \times n$ boyutlu A matrisinin transpozisini hesaplayan aşağıdaki çoklu iş parçacıklı kabakod veriliyor:

P-TRANSPOSE (A)

```

1   $n = A.rows$ 
2  parallel for  $j = 2$  to  $n$ 
3      parallel for  $i = 1$  to  $j - 1$ 
4           $a_{ij}$  ile  $a_{ji}$  yi yerdeğiştir
```

Bu algoritmanın iş yükü, süre ve paralelliğini analiz ediniz.

27.1-8

P-TRANSPOSE un 3. satırındaki **parallel for** döngüsünü sıradan bir **for** döngüsüyle yerdeğiştirdiğimizi varsayın (Alıştırma 27.1-7 e bakınız). Sonuç algoritmanın iş yükü, süre ve paralelliğini analiz ediniz.

27.1-9

$T_P = T_1/P + T_\infty$ olduğunu varsayarak, satranç programının iki versiyonu kaç tane işlemci için eşit şekilde hızlı çalışır?

27-2 Çoklu iş parçacıklı matris çarpımı

Bu bölümde, matris çarpımının nasıl çoklu iş parçacıklı yapılacağını, Kısım 4.2 de gördüğümüz seri çalışma süresi problemini araştıracağız. Böl-ve-yönet algoritmaları kadar standart üç kat iş işe döngülere dayanan çoklu iş parçacıklı algoritmaları da göreceğiz.

Çoklu iş parçacıklı matris çarpımı

Göreceğimiz ilk algoritma sayfa 75 deki SQUARE-MATRIX-MULTIPLY yordamındaki döngülerin paralelleştirilmesine dayanan basit algoritmadır:


```

P-SQUARE-MATRIX-MULTIPLY( $A, B$ )
1   $n = A.rows$ 
2   $C, n \times n$  boyutlu yeni bir matris olsun
3  parallel for  $i = 1$  to  $n$ 
4      parallel for  $j = 1$  to  $n$ 
5           $c_{ij} = 0$ 
6          for  $k = 1$  to  $n$ 
7               $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$ 
8  return  $C$ 

```

Bu algoritmayı analiz etmek için algoritmanın serileştirilmesinin sadece SQUARE-MATRIX-MULTIPLY olduğunu görürüz, böylece iş yükü basitçe SQUARE-MATRIX-MULTIPLY nin çalışma süresindekine benzer $T_1(n) = \Theta(n^3)$ dir. Süre $T_\infty(n) = \Theta(n)$ dir, çünkü o başlangıcı satır 3 de olan **parallel for** döngüsü için özyineleme ağacı altında bir yol izler, sonra başlangıcı satır 4 de olan **parallel for** döngüsü için özyineleme ağacı altında bir yol izler ve sonra satır 6'da başlayan sıradan **for** döngüsünün tüm n iterasyonunu yürütür, sonuç olarak toplam süre $\Theta(\lg n) + \Theta(\lg n) + \Theta(n) = \Theta(n)$ olur. Böylece paralellik $\Theta(n^3)/\Theta(n) = \Theta(n^2)$ olur. Alıştırma 27.2-3, sizden yarışlar yaratacağımızdan **parallel for** kullanarak basitçe yapamayacağımız $\Theta(n^3/\lg n)$ paralelliğini elde etmek için içteki döngüyü paralelleştirmenizi istiyor.

Matris çarpımı için bir böl-ve-yönet çoklu iş parçacıklı algoritması

Kısım 4.2 de öğrendiğimiz gibi, böyle bir çoklu iş parçacıklı algoritmayı gözden geçirmemiz için bizi motive eden Strassen'in böl-ve-yönet stratejisini kullanarak $n \times n$ boyutlu matrisleri $\Theta(n^{\lg 7}) = O(n^{2.81})$ süresinde seri olarak çarpabiliriz. Kısım 4.2 de söylediğimiz gibi, çoklu iş parçacıklı daha basit bir böl-ve-yönet algoritmasıyla başlıyoruz.

Sayfa 77 deki $n \times n$ boyutlu C matrisini üretmek için $n \times n$ boyutlu iki A ve B matrislerini çarpan SQUARE-MATRIX-MULTIPLY-RECURSIVE prosedürünün, her üç matrisin $n/2 \times n/2$ boyutlu dört altmatrise parçalanışına dayandığını hatırlayın:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, \quad C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

O zaman matris çarpımını aşağıdaki gibi yazabiliriz

$$\begin{aligned} \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} &= \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} \\ &= \begin{pmatrix} A_{11}B_{11} & A_{11}B_{12} \\ A_{21}B_{11} & A_{21}B_{12} \end{pmatrix} + \begin{pmatrix} A_{12}B_{21} & A_{12}B_{22} \\ A_{22}B_{21} & A_{22}B_{22} \end{pmatrix}. \end{aligned} \quad (27.6)$$

Böylelikle, iki $n \times n$ boyutlu matrisleri çarpmak için $n/2 \times n/2$ boyutlu matrislerin sekiz tane çarpımını ve $n \times n$ boyutlu matrislerin bir tane toplamını gerçekleştirebiliriz.

tiririz. Aşağıdaki kabakod iç içe paralelliği kullanan bu böl-ve-yönet stratejisini uygular. Bunu temel alan SQUARE-MATRIX-MULTIPLY-RECURSIVE prosedürünün aksine, P-MATRIX-MULTIPLY-RECURSIVE, gereksiz yere matrisleri tahsis etmeyi önlemek için çıkış matrisini bir parametre olarak alır.

P-MATRIX-MULTIPLY-RECURSIVE(C, A, B)

```

1   $n = A.rows$ 
2  if  $n == 1$ 
3       $c_{11} = a_{11}b_{11}$ 
4  else  $n \times n$  boyutlu yeni bir matris olsun
5       $A, B, C$  ve  $T$  matrislerini  $n/2 \times /2$  boyutlu altmatrislere böl
           $A_{11}, A_{12}, A_{21}, A_{22}; B_{11}, B_{12}, B_{21}, B_{22}; C_{11}, C_{12}, C_{21}, C_{22};$ 
          and  $T_{11}, T_{12}, T_{21}, T_{22};$  respectively
6      spawn P-MATRIX-MULTIPLY-RECURSIVE( $C_{11}, A_{11}, B_{11}$ )
7      spawn P-MATRIX-MULTIPLY-RECURSIVE( $C_{12}, A_{11}, B_{12}$ )
8      spawn P-MATRIX-MULTIPLY-RECURSIVE( $C_{21}, A_{21}, B_{11}$ )
9      spawn P-MATRIX-MULTIPLY-RECURSIVE( $C_{22}, A_{21}, B_{12}$ )
10     spawn P-MATRIX-MULTIPLY-RECURSIVE( $T_{11}, A_{12}, B_{21}$ )
11     spawn P-MATRIX-MULTIPLY-RECURSIVE( $T_{12}, A_{12}, B_{22}$ )
12     spawn P-MATRIX-MULTIPLY-RECURSIVE( $T_{21}, A_{22}, B_{21}$ )
13     P-MATRIX-MULTIPLY-RECURSIVE( $T_{22}, A_{22}, B_{22}$ )
14     sync
15     parallel for  $i = 1$  to  $n$ 
16         parallel for  $j = 1$  to  $n$ 
17              $c_{ij} = c_{ij} + t_{ij}$ 

```

Satır 3, 1×1 matrisleri çarptığımız temel durumu ele alır. 4–17 satırlarında özyinelemeli durumu ele alırız. Satır 4 de geçici bir T matrisini tahsis ederiz ve satır 5 A, B, C ve T matrislerinin her birini $n/2 \times n/2$ altmatrislerine parçalar. (sayfa 77 de SQUARE-MATRIX-MULTIPLY-RECURSIVE da olduğu gibi, bir matrisin altmatris bölümlerini göstermek için indis hesaplamalarını nasıl kullanacağımız gibi küçük sorunları örtbas ederiz). Satır 6 da özyinelemeli çağrı C_{11} altmatrisini $A_{11}B_{11}$ çarpım altmatrisine eşitler, böylelikle C_{11} , denklem (27.6) da onun toplamını oluşturan iki terimden birincisine eşit olur. Benzer şekilde 7–9 satırları C_{12}, C_{21} ve C_{22} yi, denklem (27.6) da onların toplamına eşit olan iki terimden ilkinde eşitler. Satır 10, T_{11} altmatrisini $A_{12}B_{21}$, çarpım altmatrisine eşitler, böylelikle T_{11}, C_{11} in toplamını oluşturan iki terimden ikincisine eşit olur. 11–13 satırları T_{12}, T_{21} ve T_{22} yi, sırasıyla C_{12}, C_{21} ve C_{22} nin toplamalarını oluşturan iki terimden ikincisine eşitler. İlk yedi özyinelemeli çağrı doğrudur, ve sonuncusu ana dizilimde çalışır. Satır 14 deki **sync** ifadesi 6–13 satırlarındaki tüm altmatris çarpımlarının hesaplandığını

garanti eder, daha sonra 15–17 satırlarındaki ikili iç içe **parallel for** döngülerini kullanarak T deki çarpımları C ye ekleriz.

İlk olarak atası SQUARE-MATRIX-MULTIPLY-RECURSIVE in seri çalışma zamanını yankılayan P-MATRIX-MULTIPLY-RECURSIVE prosedürünün $M_1(n)$ iş yükünü analiz edeceğiz. Özyinelemeli durumda, $\Theta(1)$ zamanında parçalarız, $n / 2 \times n / 2$ boyutlu matrislerin sekiz özyinelemeli çarpımını gerçekleştiririz, ve $n \times n$ boyutlu iki matrisin toplamından kaynaklanan $\Theta(n^2)$ iş yüküyle bitiririz. Böylece, ana teoremin durum 1 inden $M_1(n)$ iş yükü için özyineleme

$$\begin{aligned} M_1(n) &= 8M_1(n/2) + \Theta(n^2) \\ &= \Theta(n^3) \end{aligned}$$

olur. Diğer bir deyişle, çoklu iş parçacıklı algoritmamızın iş yükü asimptotik olarak Kısım 4.2 deki onun üçlü iç içe döngülü SQUARE-MATRIX-MULTIPLY yordamının çalışma zamanıyla aynıdır.

P-MATRIX-MULTIPLY-RECURSIVE in $M_\infty(n)$ süresini belirlemek için ilk olarak 15-17 satırlarındaki ikili iç içe **parallel for** döngülerinin $\Theta(\lg n)$ süresi tarafından domine edilen parçalanmanın süresinin $\Theta(1)$ olduğunu gözlemliyoruz. Çünkü sekiz özyinelemeli çağrılarının hepsi aynı boyutlu matrisler üzerinde işlem yapar, herhangi özyinelemeli çağrının maksimum süresi yalnızca herhangi birinin süresi kadardır. Dolayısıyla P-MATRIX-MULTIPLY-RECURSIVE in $M_\infty(n)$ özyineleme süresi aşağıdaki gibi olur.

$$M_\infty(n) = M_\infty(n/2) + \Theta(\lg n) \quad (27.7)$$

Bu özyineleme ana teoremin olgularının herhangi birinin altında girmemektedir, ama Alıştırma 4.6-2 nin koşulunu karşılar. Nitekim, Alıştırma 4.6-2 ye göre (27.7) özyinelemesinin çözümü $M_\infty(n) = \Theta(\lg^2 n)$ dir.

Madem ki P-MATRIX-MULTIPLY-RECURSIVE in iş yükü ve süresini biliyoruz, onun çok yüksek olan paralelliliğini $M_1(n)/M_\infty(n) = \Theta(n^3 / \lg^2 n)$, olarak hesaplayabiliriz.

Strassen'in çoklu iş parçacıklı yöntemi

Çoklu iş parçacıklı Strassen'in algoritmasına göre, sadece iç içe paralelliliği kullanarak sayfa 79 daki aynı genel adımları takip ederiz:

1. A ve B giriş matrisi ve C çıkış matrisini (27.6) denklemindeki gibi $n / 2 \times n / 2$ boyutlu altmatrislerine böl. Bu adım indis hesabına göre $\Theta(1)$ iş ve süresi kadar zaman alır.
2. Herbiri $n / 2 \times n / 2$ boyutlu olan ve adım 1 de oluşturulan iki matrisin toplamı veya farkı olan 10 tane $S_1, S_2, \dots, S_{10}$, matrislerini oluştur. İkili iç içe **parallel for** döngülerini kullanarak 10 matrisin tümünü $\Theta(n^2)$ işi ve $\Theta(\lg n)$ süresinde oluşturabiliriz.

3. Adım 1 de oluşturulan alt matrisleri ve adım 2 de oluşturulan 10 matrisi kullanarak, özyinelemeli olarak $n / 2 \times n / 2$ boyutlu yedi matris çarpımı olan $P_1, P_2, \dots, P_7$ matrislerinin hesaplanmasını doğurur.
4. Birkez daha ikili iç içe **parallel for** döngülerini kullanarak, P_i matrislerinin çeşitli kombinasyonlarını toplayarak veya çıkartarak, C matrisinin istenen $C_{11}, C_{12}, C_{21}, C_{22}$ altmatrislerini hesapla. Dört altmatrisin tümünü $\Theta(n^2)$ işi ve $\Theta(\lg n)$ süresinde hesaplayabiliriz.

Bu algoritmayı analiz etmek için ilk önce, serileştirme orijinal seri algoritma ile aynı olduğundan iş yükünün, sadece serileştirmenin çalışma süresi olan, $\Theta(n^{\lg 7})$ olduğunu gözlemleriz. P-MATRIX-MULTIPLY-RECURSIVE e gelince, süre için bir özyineleme geliştirebiliriz. Bu durumda, yedi özyineleme paralel olarak yürütülür, ama onların hepsi aynı boyutlu matrisler üzerinde işlem yaptığından, $\Theta(\lg^2 n)$ çözümüne sahip P-MATRIX-MULTIPLY-RECURSIVE için yaptığımız gibi, aynı (27.7) özyinelemesini elde ederiz. Böylece, çoklu iş parçacıklı Strassen'in yönteminin paralellliği, P-MATRIX-MULTIPLY-RECURSIVE nin paralellüğinden bir miktar daha az olan, yüksek $\Theta(n^{\lg 7} / \lg^2 n)$ değeridir.

Alıştırmalar

27.2-1

Diyagramınızdaki tepelerin algoritmanın yürütülmesindeki dizilimlere nasıl karşılık geldiğini etiketleyerek, 2×2 matrisler üzerinde P-SQUARE-MATRIX-MULTIPLY ı hesaplayan yönlü çevrimsiz hesaplama çizgesini çizin. Doğurulan ayrıtlar ve çağrı ayrıtları aşağı yönü gösterir, devam eden ayrıtlar yatay olarak sağ yönü gösterir ve döndürme ayrıtları yukarı yönü gösterir kuralını kullanın. Her bir dizilimin birim zaman sürdüğünü varsayarak, bu hesaplamanın işi, süresi ve paralellliğini analiz edin.

27.2-2

P-MATRIX-MULTIPLY-RECURSIVE için Alıştırma 27.2-1 i tekrarlayın.

27.2-3

İki $n \times n$ boyutlu matrisi $\Theta(n^3)$ işinde ama sadece $\Theta(\lg n)$ süresinde çarpan çoklu iş parçacıklı algoritmanın kabakodunu verin. Algoritmanızı analiz edin.

27.2-4

$p \times q$ boyutlu bir matrisi $q \times p$ boyutlu bir matrisle çarpan bir etkili çoklu iş parçacıklı algoritmanın kabakodunu verin. p , q ve r nin herhangi biri 1 olsa bile algoritmanız son derece paralel olmalıdır. Algoritmanızı analiz edin.

27.2-5

Matrisi özyinelemeli olarak $n/2 \times n/2$ boyutlu dört altmatrise bölmek için böl-ve-yönet kullanarak $n \times n$ boyutlu bir matrisin transpozisini yerinde alan bir etkili çoklu iş parçacıklı algoritmanın kabakodunu verin. Algoritmanızı analiz edin.

27.2-6

Ayrıt-ağırlıklı bir çizgede, tüm tepe çiftleri arasındaki en kısa yolu hesaplayan Floyd-Warshall algoritmasının (Kısım 25.2 ye bakın) bir etkili çoklu iş parçacıklı uygulaması için kabakodu verin. Algoritmanızı analiz edin.

27.3 Çoklu iş parçacıklı birleştirerek sıralama

Birleştirerek sıralamayı ilk olarak Kısım 2.3.1 de gördük, ve Kısım 2.3.2 de onun çalışma süresini analiz ettik ve onun $\Theta(n \lg n)$ olduğunu gösterdik. Birleştirerek sıralama zaten böl-ve-yönet paradigmasını kullandığı için o, iç içe paralelliği kullanarak çoklu iş parçalama için müthiş bir aday gibi görünür. Kabakodu kolaylıkla değiştirebiliriz böylece ilk özyinelemeli çağrı doğurulur:

MERGE-SORT'(A, p, r)

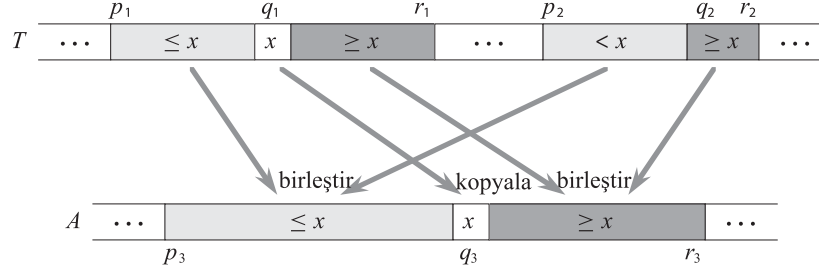
```

1  if p < r
2      q = ⌊(p + r)/2⌋
3      spawn MERGE-SORT'(A, p, q)
4      MERGE-SORT'(A, q + 1, r)
5      sync
6      MERGE(A, p, q, r)
```

Kendi seri meslektaşısı gibi MERGE-SORT' $A[p..r]$ altdizisini sıralar. Satır 5 deki **sync** ifadesi tarafından sağlanan satır 3 ve 4 deki özyinelemeli iki altprogram tamamlandıktan sonra sayfa 31 de olduğu gibi MERGE-SORT' aynı MERGE prosedürünü çağırır.

Şimdi MERGE-SORT' e göz atalım. Bunu yapabilmek için ilk önce MERGE yi analiz etmeliyiz. n elemanı birleştirmek için onun çalışma süresinin $\Theta(n)$ olduğunu hatırlayınız. MERGE seri olduğu için onun her iki iş yükü ve süresi $\Theta(n)$ dir. Böylelikle aşağıdaki özyineleme, birleştirerek sıralamanın seri çalışma süresiyle aynı olan n eleman üzerinde MERGE-SORT' ün $MS'_1(n)$ iş yükünü karakterize eder.

$$\begin{aligned}
 MS'_1(n) &= 2 MS'_1(n/2) + \Theta(n) \\
 &= \Theta(n \lg n),
 \end{aligned}$$



Şekil 27.6 İki sıralanmış $T[p_1 \dots r_1]$ ve $T[p_2 \dots r_2]$ altdizisinin $A[p_3 \dots r_3]$ altdizisi olarak çoklu iş parçacıklı birleştirilmesinin arkasındaki düşünce. $T[p_1 \dots r_1]$ in medianının $x = T[q_1]$ ve q_2 nin $T[p_2 \dots r_2]$ deki yeri olduğunu varsayarak, öyleki x , $T[q_2 - 1]$ ve $T[q_2]$ arasında düşecek, $T[p_1 \dots q_1 - 1]$ ve $T[p_2 \dots q_2 - 1]$ (açık gölgeli) altdizilerindeki her eleman x den küçük veya eşittir ve $T[q_1 + 1 \dots r_1]$ ve $T[q_2 + 1 \dots r_2]$ (koyu gölgeli) altdizilerindeki her eleman x den küçüktür. Birleştirmek için, x in $A[p_3 \dots r_3]$ deki yeri olan q_3 indisini hesaplarız, x i $A[q_3]$ e kopyalıyoruz ve sonra özyinelemeli olarak $T[p_1 \dots q_1 - 1]$ ile $T[p_2 \dots q_2 - 1]$ yi $A[p_3 \dots q_3 - 1]$ olarak $T[q_1 + 1 \dots r_1]$ ile $T[q_2 \dots r_2]$ yi $A[q_3 + 1 \dots r_3]$ olarak birleştiririz.

MERGE-SORT' un özyinelemeli iki çağrısı paralel olarak çalışabildiği için MS'_∞ süresi

$$\begin{aligned} MS'_\infty(n) &= MS'_\infty(n/2) + \Theta(n) \\ &= \Theta(n) \end{aligned}$$

özyineleme formülüyle verilir. Böylece MERGE-SORT' un paralellliği, paralellik için bir etkileyici miktar olan $MS'_1(n)/MS'_\infty(n) = \Theta(\lg n)$ olarak bulunur. Örneğin 10 milyon elemanı sıralamak için o birkaç işlemci üzerindeki doğrusal hızlanma elde edilebilir, ama yüzlerce işlemciye etkili olarak büyütmek olmaz.

Siz büyük olasılıkla paralellik darboğazının bu çoklu iş parçacıklı birleştirerek sıralamanın neresinde olduğunu zaten anladınız: Seri MERGE yordamı. Birleştirme başlangıçta doğal olarak seri gibi görünüyor olsa da, biz iç içe paralellliği kullanarak aslında, bunun bir çoklu iş parçacıklı versiyonunu yapabiliriz.

Şekil 27.6 da gösterilen çoklu iş parçacıklı birleştirme için bizim bölve-yönet stratejimiz bir T dizisinin altdizileri üzerinde işlem yapar. İki sıralı $n_1 = r_1 - p_1 + 1$ uzunluklu $T[p_1 \dots r_1]$ altdizisi ile $n_2 = r_2 - p_2 + 1$ uzunluklu $T[p_2 \dots r_2]$ altdizisini $n_3 = r_3 - p_3 + 1 = n_1 + n_2$ uzunluklu diğer bir $A[p_3 \dots r_3]$ altdizisinde birleştirdiğimizi varsayalım. Genelliği bozmadan, basitlik için $n_1 \geq n_2$ varsayımını yapıyoruz.

İlk olarak $T[p_1 \dots r_1]$ altdizisinin $x = T[q_1]$ orta elemanını buluruz, burada $q_1 = \lfloor (p_1 + r_1)/2 \rfloor$ dir. Altdizi sıralı olduğu için x , $T[p_1 \dots r_1]$ in bir medianıdır: $T[p_1 \dots q_1 - 1]$ nin her elemanı x den büyük değildir ve $T[q_1 + 1 \dots r_1]$ nin her elemanı x den küçük değildir. Sonra $T[p_2 \dots r_2]$ altdizisinde q_2 indeksini

bulmak için ikili aramayı kullanırız, böylece $T[q_2 - 1]$ ve $T[q_2]$ arasına x i sokarsak altdizi yine de sıralı kalır.

Sonraki adımda orijinal $T[p_1 \dots r_1]$ ve $T[p_2 \dots r_2]$ altdizilerini $A[p_3 \dots r_3]$ de aşağıdaki gibi birleştiririz:

1. $q_3 = p_3 + (q_1 - p_1) + (q_2 - p_2)$ olarak ayarla.
2. x i $A[q_3]$ e kopyala.
3. Özyinelemeli olarak $T[p_1 \dots q_1 - 1]$ ve $T[p_2 \dots q_2 - 1]$ yi birleştir ve sonucu $A[p_3 \dots q_3 - 1]$ altdizisine yerleştir.
4. Özyinelemeli olarak $T[q_1 + 1 \dots r_1]$ ve $T[q_2 \dots r_2]$ yi birleştir ve sonucu $A[q_3 + 1 \dots r_3]$ altdizisine yerleştir.

q_3 ü hesapladığımızda, $q_1 - p_1$ sayısı $T[p_1 \dots q_1 - 1]$ altdizisindeki eleman sayısıdır ve $q_2 - p_2$ sayısı $T[p_2 \dots q_2 - 1]$ altdizisindeki eleman sayısıdır. Bu nedenle, bunların toplamı $A[p_3 \dots r_3]$ alt dizisinde x den önce oluşan elemanları sayısıdır.

İki boş altdiziyi birleştirmek için yapılacak bir şeyin olmadığı $n_1 = n_2 = 0$, olduğunda taban durum meydana gelir. $T[p_1 \dots r_1]$ altdizisinin uzunluğunun en az $T[p_2 \dots r_2]$ kadar, yani $n_1 \geq n_2$ olduğunu varsaydığımızdan dolayı sadece $n_1 = 0$ olup olmadığına bakarak taban durumunu kontrol edebiliriz. Ayrıca iki altdiziden sadece biri, $n_1 \geq n_2$ varsayımımıza göre bu $T[p_2 \dots r_2]$ altdizisi olmalıdır, boş olduğundaki durumu özyinelemenin uygun şekilde ele aldığından da emin olmalıyız.

Şimdi bu düşünceleri kabakoda yerleştirelim. Seri şekilde ifade ettiğimiz ikili arama ile başlıyoruz. BINARY-SEARCH(x, T, p, r) prosedürü bir x anahtarı ve bir $T[p \dots r]$ altdizisini alır, ve aşağıdakilerden birini geri döndürür:

- Eğer $T[p \dots r]$ boş ($r < p$) ise, o zaman p indisini geri döndürür.
- Eğer $x \leq T[p]$, yani $T[p \dots r]$ nin tüm elemanlarından küçük veya eşitse o zaman p indeksini geri döndürür.
- Eğer $x > T[p]$ ise o zaman $p < q \leq r + 1$ aralığında $T[q - 1] < x$ olan en büyük q indisini geri döndürür.

İşte kabakod:

BINARY-SEARCH(x, T, p, r)

```

1  low = p
2  high = max(p, r + 1)
3  while low < high
4      mid = ⌊(low + high)/2⌋
5      if x ≤ T[mid]
6          high = mid
7      else low = mid + 1
8  return high
```

BINARY-SEARCH(x, T, p, r) çağrısı en kötü durumda $\Theta(\lg n)$ seri zamanı kadar sürer, burada $n = r - p + 1$, onun üzerinde çalıştığı altdizinin boyutudur. (Alistırma 2.3-5 e bakınız.) BINARY-SEARCH bir seri prosedür olduğundan onun kötü-durum yükü ve süresinin her ikisi $\Theta(\lg n)$ dir.

Şimdi çok iş parçacıklı birleştirme prosedürünün kendisinin psudokodunu yazmak için hazırlanalım. Sayfa 31 deki MERGE prosedüründe olduğu gibi P-MERGE yordamı iki altdizinin aynı dizide birleştirilmiş olduğunu varsayar. Bununla beraber, MERGE ün aksine P-MERGE birleştirilecek iki altdizinin dizi içinde bitişik olduğunu kabul etmez. (yani P-MERGE $p_2 = r_1 + 1$ olmasını gerektirmez.) MERGE ve P-MERGE arasındaki bir diğer fark P-MERGE ün birleştirilmiş değerlerin saklandığı bir A çıktı altdizisini bir argüman olarak almasıdır. P-MERGE($T, p_1, r_1, p_2, r_2, A, p_3$) çağrısı sıralanmış $T[p_1 \dots r_1]$ ve $T[p_2 \dots r_2]$ altdizilerini $A[p_3 \dots r_3]$ altdizisinde birleştirir, burada $r_3 = p_3 + (r_1 - p_1 + 1) + (r_2 - p_2 + 1) - 1 = p_3 + (r_1 - p_1) + (r_2 - p_2) + 1$ dir ve bir girdi olarak sağlanmaz.

P-MERGE($T, p_1, r_1, p_2, r_2, A, p_3$)

```

1   $n_1 = r_1 - p_1 + 1$ 
2   $n_2 = r_2 - p_2 + 1$ 
3  if  $n_1 < n_2$                                 //  $n_1 \geq n_2$  olduğundan emin olun
4      yerdeğiştir  $p_1$  ile  $p_2$ 
5      yerdeğiştir  $r_1$  ile  $r_2$ 
6      yerdeğiştir  $n_1$  ile  $n_2$ 
7  if  $n_1 == 0$                                 // ikisi birden boş?
8      return
9  else  $q_1 = \lfloor (p_1 + r_1)/2 \rfloor$ 
10  $q_2 = \text{BINARY-SEARCH}(T[q_1], T, p_2, r_2)$ 
11  $q_3 = p_3 + (q_1 - p_1) + (q_2 - p_2)$ 
12  $A[q_3] = T[q_1]$ 
13 spawn P-MERGE( $T, p_1, q_1 - 1, p_2, q_2 - 1, A, p_3$ )
14 P-MERGE( $T, q_1 + 1, r_1, q_2, r_2, A, q_3 + 1$ )
15 sync
```

P-MERGE prosedürü aşağıdaki şekilde çalışır. Satır 1–2 sırasıyla $T[p_1 \dots r_1]$ ve $T[p_2 \dots r_2]$ altdizilerinin n_1 ve n_2 uzunluklarını hesaplar. Satır 3–6 $n_1 \geq n_2$ varsayımını gerçekleştirir. Satır 7, $T[p_1 \dots r_1]$ in boş olduğu (buna göre $T[p_2 \dots r_2]$ de öyledir) taban durumunu test eder, bu durumda basitçe geri döneriz. Satır 9–15 böl-ve-yönet stratejisini uygular. Satır 9 $T[p_1 \dots r_1]$ in orta noktasını hesaplar, ve satır 10 $T[p_2 \dots q_2 - 1]$ deki tüm elemanların $T[q_1]$ (x e karşılık gelir)’den küçük ve $T[q_2 \dots r_2]$ deki tüm elemanların en azından $T[q_1]$ kadar büyük

olduğu $T[p_2 \dots r_2]$ deki q_2 noktasını bulur. Satır 11 $A[p_3 \dots r_3]$ çıktı altdizisini $A[p_3 \dots q_3 - 1]$ ve $A[q_3 + 1 \dots r_3]$ altdizilerine bölen elemanın q_3 indisini hesaplar ve sonra satır 12, $T[q_1]$ yi doğrudan $A[q_3]$ içine kopyalar.

Daha sonra, iç içe paralellik kullanarak yineleriz. Satır 14 paralel olarak ikinci problemi çağırırken, satır 13 ilk problem oluşturur. Satır 15 deki **sync** ifadesi yordam geri döndürülmeden önce altproblemlerin bitmesini sağlar (Her yordam geri dönmeye önce kapalı olarak bir **sync** yürüttüğü için, biz satır 15 deki **sync** deyimini ihmal edebiliriz, fakat onu içermek pratikte iyi bir kodlamadır). Kodlamada, $T[p_2 \dots r_2]$ altdizisi boş olduğunda kodun doğru çalışmasını sağlayan birtakım akıllar vardır. Onun çalışmasını sağlayan yol, her bir özyineleme çağrısında, taban duruma neden olan $T[p_1 \dots r_1]$ in kendisinin en sonunda boş olana kadar $T[p_1 \dots r_1]$ in bir medyanının çıktı altdizisine yerleştirilmesidir.

Çoklu iş parçacıklı birleştirmenin analizi

İlk iki altdizinin toplamda $n = n_1 + n_2$ sayıda elemanı içerdiği P-MERGE ün $PM_\infty(n)$ süresi için bir özyineleme türeteceğiz. Satır 13 deki spawn ve satır 14'deki çağrı paralel mantıkla işlem yaptığı için biz iki çağrıdan sadece masraflı olanı incelememiz gerekir. Anahtar, en kötü durumda, aşağıdaki göreceğimiz üzere özyinelemeli çağrılarının birinde elemanlarının maksimum sayısının en fazla $3n/4$ olabileceğini anlamaktır. Çünkü 3–6 satırları $n_2 \leq n_1$ olmasını sağlar, buradan da $n_2 = 2n_2/2 \leq (n_1 + n_2)/2 = n/2$ olduğu elde edilir. En kötü durumda iki özyinelemeli çağrılarının biri $T[p_1 \dots r_1]$ in $\lfloor n_1/2 \rfloor$ elemanını $T[p_2 \dots r_2]$ nin n_2 elemanlarının tümüyle birleştirir ve dolayısıyla çağrıda kapsanan eleman sayısı

$$\begin{aligned} \lfloor n_1/2 \rfloor + n_2 &\leq n_1/2 + n_2/2 + n_2/2 \\ &= (n_1 + n_2)/2 + n_2/2 \\ &\leq n/2 + n/4 \\ &= 3n/4 \end{aligned}$$

olur. Çağrısının $\Theta(\lg n)$ deki maliyetini satır 10 daki BINARY-SEARCH e ekleyerek en kötü-durum süresi için aşağıdaki özyineleme formülünü elde ederiz:

$$PM_\infty(n) = PM_\infty(3n/4) + \Theta(\lg n) \quad (27.8)$$

(Taban durum için, 1–8 satırları sabit zamanda çalıştığından süre $\Theta(1)$ dir.) Bu özyineleme ana teoremin herhangi bir durumunun altına girmez, ama Alıştırma 4.6-2 nin koşuluyla uyusur. Böylelikle (27.8) özyinelemesinin çözümü $PM_\infty(n) = \Theta(\lg^2 n)$ olur.

Şimdi n eleman üzerinde P-MERGE ün $\Theta(n)$ ile gösterilen $PM_1(n)$ işini analiz edeceğiz. n elemanın her biri T dizisinden A dizisine kopyalanması gerektiğinden $PM_1(n) = \Omega(n)$ buluruz. Böylece sadece $PM_1(n) = O(n)$ olduğunu göstermek kalıyor.

İlk olarak en-kötü-durum iş yükü için bir özyineleme türeteceğiz. Satır 10'daki ikili arama özyinelemeli çağrılarının dışındaki diğer iş yükleri bastıran en kötü durumda $\Theta(\lg n)$ a mal olur. Özyinelemeli çağrılar için 13 ve 14 satırlarındaki özyinelemeli çağrılar farklı sayıda elemanı birleştirmesine rağmen iki özyinelemeli çağrı birlikte en fazla n elemanı birleştirdiğini gözlemleyin. ($T[q_1]$ özyinelemeli çağrıya katılmadığı için aslında $n - 1$ eleman). Üstelik, süreyi analiz ederken gördüğümüz gibi, bir özyinelemeli çağrı en fazla $3n/4$ elemanı işler. Böylece

$$PM_1(n) = PM_1(\alpha n) + PM_1((1 - \alpha)n) + O(\lg n) \quad (27.9)$$

özyinelemesini elde ederiz, burada α , $1/4 \leq \alpha \leq 3/4$ aralığındadır, ve buradan anlıyoruz ki α nın gerçek değeri özyinelemenin her aşamasında değişebilir.

Yerine koyma yöntemiyle (27.9) özyinelemesinin $PM_1 = O(n)$ çözümüne sahip olduğunu kanıtlayacağız. Bazı c_1 ve c_2 pozitif sabitleri için $PM_1(n) \leq c_1 n - c_2 \lg n$ olduğunu varsayalım. $c_2(\lg n + \lg(\alpha(1 - \alpha)))$, $\Theta(\lg n)$ i domine edecek şekilde c_2 yi yeterince büyük seçebileceğimizden yerine koyma bize

$$\begin{aligned} PM_1(n) &\leq (c_1 \alpha n - c_2 \lg(\alpha n)) + (c_1 (1 - \alpha)n - c_2 \lg((1 - \alpha)n)) + \Theta(\lg n) \\ &= c_1(\alpha + (1 - \alpha))n - c_2(\lg(\alpha n) + \lg((1 - \alpha)n)) + \Theta(\lg n) \\ &= c_1 n - c_2(\lg \alpha + \lg n + \lg(1 - \alpha) + \lg n) + \Theta(\lg n) \\ &= c_1 n - c_2 \lg n - (c_2(\lg \alpha + \lg(1 - \alpha))) + \Theta(\lg n) \\ &\leq c_1 n - c_2 \lg n \end{aligned}$$

olduğunu verir. Üstelik özyinelemenin taban koşullarını sağlaması için c_1 i yeterince büyük seçebiliriz. P-MERGE ün $PM_1(n)$ işi hem $\Omega(n)$ hem de $O(n)$ olduğu için $PM_1(n) = \Theta(n)$ elde ederiz.

P-MERGE in paralelliği $PM_1(n)/PM_\infty(n) = \Theta(n/\lg^2 n)$ dir.

Çoklu iş parçacıklı birleştirerek sıralama

Şimdi bir güzel paralelleştirdiğimiz çoklu iş parçacıklı birleştirme prosedürünü, bir çoklu iş parçacıklı birleştirerek sıralama içine dahil edebiliriz. Birleştirerek sıralamanın bu versiyonu daha önce gördüğümüz MERGE-SORT' prosedürüne benzerdir, ama MERGE-SORT' aksine, sıralanmış sonucu tutan B alt dizisini bir çıktı argümanı olarak alır. Özel olarak, P-MERGE-SORT. (A, p, r, B, s) , $A[p \dots r]$ nin elemanlarını sıralar ve onları $B[s \dots s + r - p]$ de saklar.

P-MERGE-SORT(A, p, r, B, s)

```

1   $n = r - p + 1$ 
2  if  $n == 1$ 
3       $B[s] = A[p]$ 
4  else  $T[1..n]$  yeni bir dizi olsun
5       $q = \lfloor (p + r)/2 \rfloor$ 
6       $q' = q - p + 1$ 
7      spawn P-MERGE-SORT( $A, p, q, T, 1$ )
8      P-MERGE-SORT( $A, q + 1, r, T, q' + 1$ )
9      sync
10     P-MERGE( $T, 1, q', q' + 1, n, B, s$ )
```

Satır 1 $A[p..r]$ giriş altdizisindeki elemanların n sayısını hesapladıktan sonra, satır 2-3 dizi sadece 1 elemana sahip olduğunda taban durumu ele alır. 4–6 satırları, satır 7 deki özyinelemeli doğurmayı ayarlar ve paralel olarak işletilen satır 8 deki çağrıyı yapar. Özel olarak, satır 4, özyinelemeli birleştirerek sıralamanın sonuçlarını saklamak için n elemanlı geçici bir T dizisini ayırır. Satır 5, elemanları ardışık olarak sıralanacak iki $A[p..q]$ ve $A[q + 1..r]$ altdizilerine bölen $A[p..r]$ nin q indisini hesaplar, ve satır 6, birinci $A[p..q]$ altdizisinin q' eleman sayısını hesaplayarak devam eder, satır 8 sıralanmış sonuç $A[q + 1..r]$ dizisinin nerede saklanacağını gösteren T deki başlangıç indisini belirlemeye çalışır. Bu noktada, doğurma ve özyinelemeli çağrı üretilir, doğurulan prosedür yapılarına kadar beklemesini zorlayan satır 9 daki **sync** ile devam eder. Son olarak, satır 10, sıralanmış $T[1..q']$ ve $T[q' + 1..n]$ altdizilerini $B[s..s + r - p]$ çıktı altdizisinde birleştirmek için P-MERGE ü çağırır.

Çoklu iş parçacıklı birleştirerek sıralamanın analizi

P-MERGE ün iş yükü analizinden çok daha önemli olan P-MERGE-SORT un $PM-S_1(n)$ işini analiz ederek başlayacağız. Aslında iş yükü,

$$\begin{aligned}
 PMS_1(n) &= 2PMS_1(n/2) + PM_1(n) \\
 &= 2PMS_1(n/2) + \Theta(n)
 \end{aligned}$$

özyinelemesiyle verilir. Bu özyineleme Kısım 2.3.1 deki sıradan MERGE-SORT için verilen (4.4) özyinelemesiyle aynıdır ve ana teorem durum 2 ye göre $PMS_1(n) = \Theta(n \lg n)$ çözümüne sahiptir.

Şimdi en-kötü-durum süresi $PMS_\infty(n)$ için bir özyinelemeyi türeteceğiz ve analiz edeceğiz. Satır 7 ve 8 deki P-MERGE-SORT ın iki özyinelemeli çağrısı mantıksal paralel olarak işlem yaptığı için

$$\begin{aligned}
PMS_{\infty}(n) &= PMS_{\infty}(n/2) + PM_{\infty}(n) \\
&= PMS_{\infty}(n/2) + \Theta(\lg^2 n)
\end{aligned} \tag{27.10}$$

özyinelemesini elde ederek onların birini ihmal edebiliriz. (27.8) özyinelemesindeki gibi, ana teorem (27.10) özyinelemesini sağlamaz, ama Alıştırma 4.6-2 nin sağlar. Çözüm $PMS_{\infty}(n) = \Theta(\lg^3 n)$ dir ve böylece P-MERGE-SORT un süresi $\Theta(\lg^3 n)$ dir.

Paralel birleştirme, MERGE-SORT' üzerinde P-MERGE-SORT a önemli bir paralellik avantajı verir. Seri MERGE prosedürünü çağıran MERGE-SORT' un paralelliğinin yalnızca $\Theta(\lg n)$ olduğunu hatırlayın. P-MERGE-SORT için paralellik hem teoride hem de pratikte çok daha iyi olan

$$\begin{aligned}
PMS_1(n)/PMS_{\infty}(n) &= \Theta(n \lg n)/\Theta(\lg^3 n) \\
&= \Theta(n/\lg^2 n)
\end{aligned}$$

dir. Pratikte iyi bir uygulama asimptotik notasyon tarafından gizlenen sabitleri azaltmak için temel durumu irileştirerek bazı paralelliği feda ederdi. Dizinin boyutu yeterince küçük olduğunda, temel durumu irileştirmenin basit yolu sıradan bir seri sıralamaya belki de hızlı sıralamaya geçmektir.

Alıştırmalar

27.3-1

P-MERGE ün ana durumunun nasıl irileştirileceğini açıklayınız.

27.3-2

P-MERGE ün yaptığı gibi, daha büyük altdizide bir medyan elemanı bulmak yerine Alıştırma 9.3-8 in sonucunu kullanarak iki sıralanmış altdizideki tüm elemanların bir medyan elemanının bulunması seçeneğini dikkate alınız. Bu medyan-bulma prosedürünü kullanan etkili bir çoklu iş parçacıklı birleştirme prosedürü için kaba kod veriniz. Algoritmanızı analiz ediniz.

27.3-3

Sayfa 171 deki PARTITION prosedürünün yaptığı gibi, bir pivot etrafında bir diziyi parçalamak için etkili bir çoklu iş parçacıklı algoritma veriniz. Diziyi yerinde parçalamanız gerekmez. Algoritmanızı mümkün olduğunca paralel yapınız. Algoritmanızı analiz ediniz (*İpucu*: Bir yardımcı dizi gerekebilir ve giriş elemanları üzerinde birden fazla geçiş yapmak gerekebilir).

27.3-4

Sayfa 911 deki RECURSIVE-FFT nin bir çoklu iş parçacıklı sürümünü verin. Uygulamanızı mümkün olduğunca paralel yapınız. Algoritmanızı analiz ediniz.

27.3-5 *

Sayfa 216'daki RANDOMIZED-SELECT'in bir çoklu iş parçacıklı sürümünü verin. Uygulamanızı mümkün olduğunca paralel yapınız. Algoritmanızı analiz ediniz. (*İpucu:* Alistırma 27.3-3'deki parçalama algoritmasını kullanınız).

27.3-6 *

Kısım 9.3'deki SELECT'in nasıl çoklu iş parçacıklı yapılacağını gösteriniz. Uygulamanızı mümkün olduğunca paralel yapınız. Algoritmanızı analiz ediniz.

Problemler
27-1 İççe paralelliği kullanarak paralel döngülerin uygulanması

Toplamlarının $C[1..n]$ de saklandığı n -elemanlı $A[1..n]$ ve $B[1..n]$ dizilerinin eleman eleman toplamını gerçekleştiren aşağıdaki çoklu iş parçacıklı algoritmayı ele alalım :

SUM-ARRAYS(A, B, C)

```
1  parallel for  $i = 1$  to  $A.length$ 
2       $C[i] = A[i] + B[i]$ 
```

a. MAT-VEC-MAIN-LOOP'in durumundaki iççe paralelliği (**spawn** ve **sync**) kullanarak SUM-ARRAYS'deki paralel döngüyü tekrar yazınız.

Belirlenecek bir *grain-size* değerini içeren paralel döngünün aşağıdaki alternatif uygulamasını ele alalım:

SUM-ARRAYS'(A, B, C)

```
1   $n = A.length$ 
2   $grain-size = ?$  // belirlenecek
3   $r = \lceil n/grain-size \rceil$ 
3   $r = \lceil n/grain-size \rceil$ 
4  for  $k = 0$  to  $r - 1$ 
5      spawn ADD-SUBARRAY( $A, B, C, k \cdot grain-size + 1,$ 
                            $\min((k + 1) \cdot grain-size, n)$ )
6  sync
```

ADD-SUBARRAY(A, B, C, i, j)

```
1  for  $k = i$  to  $j$ 
2       $C[k] = A[k] + B[k]$ 
```

- b.** *grain-size* = 1 olarak ayarladığımızı varsayalım. Bu uygulamanın paralelliği nedir?
- c.** SUM-ARRAYS' nin süresi için n ve *grain-size* cinsinden bir formül veriniz. Paralelliği maksimum yapmak için *grain-size* nin en iyi değerini türetiniz.

27-2 *Matris çarpımında geçici yerden tasarruf*

P-MATRIX-MULTIPLY-RECURSIVE prosedürü Θ -notasyonu tarafından gizlenen sabitleri olumsuz etkileyen $n \times n$ boyutlu geçici bir T matrisini paylaştıran dezavantaja sahiptir. Bununla beraber, P-MATRIX-MULTIPLY-RECURSIVE yüksek paralelliğe sahiptir. Örneğin, Θ -notasyonundaki sabitleri göz ardı ederek, 1000×1000 boyutlu matris çarpımının paralelliği, $\lg 1000 \approx 10$ olduğu için yaklaşık olarak $1000^3/10^2 = 10^7$ kadardır. paralel bilgisayarların çoğu 10 milyon işlemciden daha az işlemciye sahiptir.

- a.** Süreyi $\Theta(n)$ e artırma ücretinde geçici T matrisine duyulan ihtiyacı eleyen bir özyinelemeli çoklu iş parçacıklı algoritmayı tanımlayınız. (*İpucu:* P-MATRIX-MULTIPLY-RECURSIVE in genel stratejisini takip eden $C = C + AB$ yi hesaplayınız, ama paralel olarak C yi başlatınız ve bir mantıklı seçilen konuma bir **sync** ekleyiniz).
- b.** Uygulamanızın iş yükü ve süresi için özyinelemeler veriniz ve çözünüz.
- c.** Uygulamanızın paralelliğini analiz ediniz. Θ -notasyonundaki sabitleri ihmal ederek, 1000×1000 boyutlu matrislerin paralelliğini değerlendiriniz. P-MATRIX-MULTIPLY-RECURSIVE in paralelliği ile karşılaştırınız.

27-3 *Çok Çoklu iş parçacıklı matris algoritmaları*

- a.** Bu algoritmanın bir çoklu iş parçacıklı versiyonu için kabakodu vererek, sayfa 821 deki LU-DECOMPOSITION prosedürünü paralelleştiriniz. Mümkün olduğunca uygulamanızı paralel olarak yapınız ve onun iş yükünü, süresini ve paralelliğini analiz ediniz.
- b.** Aynısını sayfa 824 deki LUP-DECOMPOSITION için yapınız.
- c.** Aynısını sayfa 817 deki LUP-SOLVE için yapınız.
- d.** Simetrik pozitif-tanımlı bir matrisin tersinin bulunması için (28.13) denklemini dayanan bir çoklu iş parçacıklı algoritma için aynısını yapınız.

27-4 Çoklu iş parçacıklı indirgemeler ve örnek hesaplamalar

Bir $x[1..n]$ dizisinin bir $\otimes$ -**indirgemesi**, burada $\otimes$ birleştirme operatörüdür,

$$y = x[1] \otimes x[2] \otimes \cdots \otimes x[n]$$

değeridir. Aşağıdaki prosedür, bir $x[i..j]$ altdizisinin $\otimes$ -indirgemesini seri olarak hesaplar.

REDUCE(x, i, j)

```

1   $y = x[i]$ 
2  for  $k = i + 1$  to  $j$ 
3       $y = y \otimes x[k]$ 
4  return  $y$ 
```

- a. $\Theta(n)$ iş yükü ve $\Theta(\lg n)$ süreli benzer fonksiyonu gerçekleştiren bir çoklu iş parçacıklı P-REDUCE algoritmasına uygulamak için iç içe paralelliği kullanınız. Algoritmanızı analiz ediniz.

Bağlantılı bir problem, bazen bir $\otimes$ -**tarama** olarak adlandırılan, bir $x[1..n]$ dizisi üzerinde bir $\otimes$ **ön ek hesaplamanın** hesaplanmasıdır, burada $\otimes$ yine birleştirme(associative) operatörüdür. $\otimes$ -tarama

$$\begin{aligned}
 y[1] &= x[1], \\
 y[2] &= x[1] \otimes x[2], \\
 y[3] &= x[1] \otimes x[2] \otimes x[3], \\
 &\vdots \\
 y[n] &= x[1] \otimes x[2] \otimes x[3] \otimes \cdots \otimes x[n],
 \end{aligned}$$

ile verilen $y[1..n]$ dizisini üretir, yani x dizinin tüm örnekleri $\otimes$ operatörü kullanılarak “toplanır”. Aşağıdaki SCAN seri prosedürü bir $\otimes$ -ön ek hesaplama gerçekleştirir:

SCAN(x)

```

1   $n = x.length$ 
2   $y[1..n]$  yeni bir dizi olsun
3   $y[1] = x[1]$ 
4  for  $i = 2$  to  $n$ 
5       $y[i] = y[i - 1] \otimes x[i]$ 
6  return  $y$ 
```

Ne yazık ki, çoklu iş parçacıklı SCAN kolay değildir. Örneğin, döngü gövdesindeki her iterasyon, önceki iterasyona bağlı olduğu için **for** döngüsünü bir **parallel for** döngüsüne değiştirmek yarışları yaratır. Aşağıdaki P-SCAN-1 prosedürü, verimsiz de olsa, paralel olarak bir $\otimes$ -ön ek hesaplama gerçekleştirir:

P-SCAN-1(x)

```

1   $n = x.length$ 
2   $y[1..n]$  yeni bir dizi olsun
3  P-SCAN-1-AUX( $x, y, 1, n$ )
4  return  $y$ 

```

P-SCAN-1-AUX(x, y, i, j)

```

1  parallel for  $l = i$  to  $j$ 
2       $y[l] = \text{P-REDUCE}(x, 1, l)$ 

```

b. P-SCAN-1 in iş yükünü, süresini ve paralelliğini analiz ediniz.

İç içe paralelliği kullanarak, daha etkili bir $\otimes$ -önek hesaplama elde edebiliriz:

P-SCAN-2(x)

```

1   $n = x.length$ 
2   $y[1..n]$  yeni bir dizi olsun
3  P-SCAN-2-AUX( $x, y, 1, n$ )
4  return  $y$ 

```

P-SCAN-2-AUX(x, y, i, j)

```

1  if  $i == j$ 
2       $y[i] = x[i]$ 
3  else  $k = \lfloor (i + j)/2 \rfloor$ 
4      spawn P-SCAN-2-AUX( $x, y, i, k$ )
5      P-SCAN-2-AUX( $x, y, k + 1, j$ )
6      sync
7      parallel for  $l = k + 1$  to  $j$ 
8           $y[l] = y[k] \otimes y[l]$ 

```

c. P-SCAN-2 nin doğruluğunu tartışınız, ve onun iş yükünü, süresini ve paralelliğini analiz ediniz.

Veriler üzerinde iki farklı geçiş ile $\otimes$ -önek hesaplamayı gerçekleştirerek P-SCAN-1 ve P-SCAN-2 in her ikisini de geliştirebiliriz. İlk geçişte, geçici bir t dizisi içine x in çeşitli bitişik altdizileri için terimleri toplarız ve ikinci geçişte, nihai sonuç y yi hesaplamak için t deki terimleri kullanırız. Aşağıdaki kabakod, bu stratejiyi uygular, ama belli ifadeler ihmal edilmiştir:

P-SCAN-3(x)

```

1   $n = x.length$ 
2   $y[1..n]$  ve  $[1..n]$  yeni diziler olsun
3   $y[1] = x[1]$ 
4  if  $n > 1$ 
5      P-SCAN-UP( $x, t, 2, n$ )
6      P-SCAN-DOWN( $x[1], x, t, y, 2, n$ )
7  return  $y$ 

```

P-SCAN-UP(x, t, i, j)

```

1  if  $i == j$ 
2      return  $x[i]$ 
3  else
4       $k = \lfloor (i + j)/2 \rfloor$ 
5       $t[k] = \text{spawn P-SCAN-UP}(x, t, i, k)$ 
6       $right = \text{P-SCAN-UP}(x, t, k + 1, j)$ 
7      sync
8      return _____ // boşluğu doldurun

```

P-SCAN-DOWN(v, x, t, y, i, j)

```

1  if  $i == j$ 
2       $y[i] = v \otimes x[i]$ 
3  else
4       $k = \lfloor (i + j)/2 \rfloor$ 
5      spawn P-SCAN-DOWN(_____,  $x, t, y, i, k$ ) // boşluğu doldurun
6      P-SCAN-DOWN(_____,  $x, t, y, k + 1, j$ ) // boşluğu doldurun
7      sync

```

d. P-SCAN-UP'ın 8. satırındaki ve P-SCAN-DOWN'ın 5. ve 6. satırlarındaki üç eksik ifadeyi tamamlayınız. Yazdığımız ifadeler ile P-SCAN-3'ün doğruluğunu tartışınız. (*İpucu:* P-SCAN-DOWN(v, x, t, y, i, j) de geçen v değerinin $v = x[1] \otimes x[2] \otimes \dots \otimes x[i - 1]$ yi sağladığını kanıtlayınız.)

e. P-SCAN-3'ün iş yükünü, süresini ve paralelliğini analiz ediniz.

27-5 Çoklu iş parçacıklı basit bir hesaplama şablonu

Hesaplanabilir Bilim, hesaplama boyunca değişmeyen diğer bilgilerle birlikte önceden hesaplanmış belirli komşu elemanların değerlerine bağlı olan değerlerle dolu bir dizinin elemanlarını gerektiren algoritmalar ile doludur. Komşu girişlerin deseni hesaplama sırasında değişmez ve bir **şablon** olarak adlandırılır. Örneğin,

Bölüm 15.4, $c[i, j]$ elemanındaki değer, girdi olarak verilen iki dizi ile birlikte x_i ve y_i elemanlarına olduğu kadar, $c[i-1, j]$, $c[i, j-1]$ ve $c[i-1, j-1]$ deki değerlerine bağlı olan en uzun ortak altdiziyi hesaplamak için bir şablon algoritma sunar. Girdi dizileri sabittir, ama algoritma iki boyutlu c dizisini doldurur böylelikle o, $c[i-1, j]$, $c[i, j-1]$ ve $c[i-1, j-1]$ elemanlarının tümünü hesapladıktan sonra $c[i, j]$ elemanını hesaplar.

Bu problemde, içiçe paralellliği $n \times n$ boyutlu A matrisi üzerinde bir çoklu parçacıklı basit şablon hesaplamasında nasıl kullanacağımızı inceleyeceğiz öyleki A daki değerlerin, $A[i, j]$ değeri yalnızca $A[i', j']$ değerine bağlıdır, burada $i' \leq i$ $j' \leq j$ dir (ve elbette $i' \neq i$ veya $j' \neq j$). Diğer bir deyişle, dizinin dışındaki statik bilgiler ile birlikte, bir elemanın değeri sadece onun üzerindeki ve/veya onun solundaki elemanların değerlerine bağlıdır. Üstelik, bu problem boyunca, $A[i, j]$ ye bağlı olan elemanları bulmadan önce, $A[i, j]$ elemanının $\Theta(1)$ süresinde bulabileceğimizi varsayıyoruz (Kısım 15.4 ün LCS-LENGTH prosedüründe olduğu gibi).

$n \times n$ boyutlu A matrisini $n/2 \times n/2$ boyutlu dört alt matrislere aşağıdaki gibi parçalayabiliriz:

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \quad (27.11)$$

Şimdi özyinemeli olarak A_{11} altmatrisini tamamlayabileceğimizi görün, çünkü o diğer üç altmatrisin elemanlarına bağlı değildir. Önce A_{11} tamamlanır. Paralel şekilde ardışık olarak A_{12} ve A_{21} i tamamlamaya devam edebiliriz. Çünkü, onların ikisi de A_{11} e bağlı olmasına rağmen birbirlerine bağlı değildirler. Son olarak, özyinemeli olarak A_{22} yi tamamlayabiliriz.

- a. Yukarıdaki tartışmaya ve (27.11) ayrışımına dayanan bir böl-ve-yönet SIMPLE-STENCIL algoritmasını kullanarak bu basit şablon hesaplamasını gerçekleştiren çoklu iş parçacıklı kabakodu verin. (özel şablona bağlı olan temel durumun detaylarını düşünmeyin.) n açısından bu algoritmanın iş yükü ve süresi için özyineleme verin ve çözün. Paralellik nedir?
- b. Tekrar mümkün olduğu kadar paralellik ile özyineleyerek, bir $n \times n$ matrisini $n/3 \times n/3$ boyutlu dokuz alt matrise bölmek için çözümünüzü (a) kısmına uyarlayın. Bu algoritmayı analiz edin. Bu algoritma ne kadar çok veya az paralellığe sahiptir, (a) kısmındaki algoritma ile karşılaştırınız?
- c. Aşağıdaki şekilde çözümlerinizi kısım (a) ve (b) ye genelleştiriniz. $b \geq 2$ olacak şekilde bir tam sayı seçiniz. Mümkün olduğu kadar paralellik ile özyineleyerek, bir $n \times n$ matrisini, herbirinin boyutu $n/b \times n/b$ olan, b^2 sayıda alt matrise bölünüz. n ve b açısından algoritmanızın iş yükü, süre ve paralellliği nedir? Bu yaklaşımı kullanarak $b \geq 2$ nin hangi seçimi için paralellüğün $o(n)$ olacağını tartışınız (*İpucu:* Bu son argüman için, paralellikteki n nin üssü $b \geq 2$ nin hangi seçimi için kesin olarak 1 den küçüktür).

- d. $\Theta(n / \lg n)$ paralelliğe ulaşan bu basit şablon hasaplaması için bir çok iş parçacıklı algoritmanın kabakodunu veriniz. İş yükü ve süre kavramlarını kullanarak problemin gerçekte $\Theta(n)$ doğal paralelliğe sahip olmasını tartışınız. Bakılırsa, bizim çok iş parçacıklı kabakodumuzun böl-ve-yönet doğası bu maksimum paralelliğe ulaşmamıza imkan vermiyor.

27-6 Rassal çoklu iş parçacıklı algoritmalar

Sadece sıradan seri algoritmalar olduğu gibi, bazen rassal çoklu iş parçacıklı algoritmaları uygulamak isteriz. Bu problem, algoritmaların beklenen davranışıyla başa çıkmak için çeşitli performans ölçütlerine nasıl uyarlanacağını araştırıyor. Ayrıca rassal hızlı sıralama (quicksort) için çok evreli bir algoritma tasarlama ve analiz etmenizi ister.

- a. T_p , T_1 ve T_∞ un tümü rassal değişkenler olduğunda beklentileri karşılamak için (27.2) iş yükü yarasını, (27.3) süre yarasını ve (27.4) açgözlü çizelgeleyiciyi sınırının nasıl değiştirileceğini açıklayın.
- b. Zamanın %1 i için $T_1 = 10^4$ ve $T_{10,000} = 1$ olarak elde ettiğimiz, ama zamanın %99 i için $T_1 = T_{10,000} = 10^9$ olarak bulduğumuz bir rassal çoklu iş parçacıklı algoritmayı dikkate alalım. Rassal çok iş parçacıklı algoritmanın **hızlanışını** $E[T_1/T_P]$ yerine $E[T_1]/E[T_P]$ şeklinde tanımlanmasını tartışın.
- c. Rassal çoklu iş parçacıklı algoritmanın **paralelliğinin** $E[T_1]/E[T_\infty]$ oranı olarak tanımlanmasını tartışın.
- d. İç içe paralelliği kullanarak sayfa 179 daki RANDOMIZED-QUICKSORT algoritmasını çok iş parçacıklı yapınız. (RANDOMIZED-PARTITION'ı paralelleştirme-yin.) P-RANDOMIZED-QUICKSORT algoritmanızın sözdekodunu verin.
- e. Rastsallaştırılmış hızlısıralama (quicksort) için çoklu iş parçacıklı algoritmanızı analiz edin (*İpucu*: Sayfa 216 daki RANDOMIZED-SELECT in analizini gözden geçirin).

Bölüm notları

Paralel bilgisayarlar, paralel bilgisayarlar için modeller, ve paralel bilgisayarlar için algoritmik modeller yıllardır çeşitli şekillerde var olmuştur. Bu kitabın önceki sürümlerinde, ağları sıralama ve PRAM (Paralel Rastgele- Erişimli Makine) modeli alanında malzeme vardı. Veri-paralel modeli [48, 168], ilkel olarak vektörler ve matrisler üzerindeki işlemleri karakterize eden bir diğer popüler algoritmik programlama modelidir.

Graham [149] ve Brent [55], Teorem 27.1 nin sınırlarına ulaşan zamanlayıcıların varlığını gösterdi. Eager, Zahorjan, ve Lazowska [98], herhangi açgözlü zamanlayıcının bu sınırlara ulaşacağını gösterdi ve paralel algoritmaları analiz etmek için iş yükü ve süre kullanımı yöntemini önerdi (her ne kadar kendi isimlerini vermeselerde). Blelloch [47], veri-paralel programlama için iş ve süreye dayanan (hesaplamanın “derinliği” adını verdiği) algoritmik programlama modeli geliştirdi. Blumofe ve Leiserson [52], rassal “iş-çalmaya (work-stealing)” dayalı dinamik çoklu iş parçalama için dağıtılmış bir çizelgeleme algoritması verdi ve onun $E[T_P] \leq T_1/P + O(T_\infty)$ sınırına ulaştığını gösterdi. Arora, Blumofe, ve Plaxton [19] ve Blelloch, Gibbons, ve Matias [49] ayrıca dinamik çoklu iş parçacıklı hesaplamaları zamanlaması için kolayca kanıtlanabilir algoritmalar sağladı.

Çoklu iş parçacıklı kabakod ve programlama modeli ağırlıklı olarak MIT’de projelendirilen Cilk [51, 118]’den etkilendi, ve Cilk++ [71] in C++ a genişletilmesi, Cilk Arts, Inc. tarafından dağıtıldı. Bu kısımdaki çoklu iş parçacıklı algoritmaların birçoğu C.E. Leiserson ve H. Prokop tarafından basılmamış ders notları olarak ortaya çıktı ve Cilk veya Cilk++ ile uygulamaya konuldu. Çok iş parçacıklı birleştirerek-sıralama algoritması Akl [12] nin bir algoritmasından esinlenmiştir.

Dizisel tutarlılık kavramı Lamport [223] den kaynaklanmaktadır.

Matrisler üzerinde işlemler bilimsel hesaplamanın özünde yattığı için matrisler ile çalışan etkili algoritmalar birçok pratik uygulamaya sahiptir. Bu bölüm matrislerin nasıl çarpılacağını ve doğrusal denklem sistemlerinin nasıl çözüleceğine odaklanır. Ek Bölüm D matrislerin temellerini gözden geçirir.

Kısım 28.1, LUP ayrışımı kullanılarak doğrusal denklem sistemlerinin nasıl çözüleceğini gösterir. Daha sonra, Kısım 28.2, Çarpma ve matrislerin terslerinin bulunması arasındaki yakın ilişkiyi araştırır. Son olarak, Kısım 28.3, simetrik pozitif tanımlı matrislerin önemli sınıfını ele alır ve üst belirlenmiş (overdetermined) bir doğrusal denklem sisteminin en küçük kareler çözümünü bulmak için onları nasıl kullanacağımızı gösterir.

Uygulamada ortaya çıkan bir önemli konu *sayısal kararlılık (numerical stability)*tır. Sayısal hesaplamalardaki yuvarlama hataları sayısal hesaplama boyunca güçlendirilebilse de, günümüz bilgisayarlarındaki kayan nokta gösteriminin sınırlı kesinliği yüzünden yanlış sonuçlar ortaya çıkmaktadır; Biz bu tür hesaplamaları, *sayısal olarak kararsız (numerically unstable)* olarak isimlendireceğiz. Bu bölümde fırsat buldukça kısaca sayısal kararlılığı ele almamıza rağmen, ona odaklanmayacağız. Biz size kararlılık konularının ayrıntılı bir tartışması için Golub ve Van Loan'ın [144] mükemmel kitabını öneririz.

28.1 Doğrusal denklem sistemlerini çözme

Birçok uygulama doğrusal denklem sistemlerinin çözümünü gerektirir. Biz, bir doğrusal sistemi, elemanları tipik olarak $\mathbb{R}$, reel sayılar gibi bir cisme ait olan matris ve vektörle, bir matris denklemi olarak formüle edebiliriz. Bu bölüm LUP ayrışımı adı verilen yöntemi kullanarak bir doğrusal denklem sisteminin nasıl çözüleceğini ele alır.

$x_1, x_2, \dots, x_n$ bilinmeyenler olmak üzere n bilinmeyenli bir doğrusal denklem sistemi ile başlıyoruz:

$$\begin{aligned}
a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n &= b_1, \\
a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n &= b_2, \\
&\vdots \\
a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n &= b_n.
\end{aligned} \tag{28.1}$$

(28.1) denklemlerinin bir **çözümü (solution)**, tüm denklemleri aynı anda sağlayan $x_1, x_2, \dots, x_n$ bilinmeyenlerinin bir değer kümesidir. Bu bölümde, yalnızca tam olarak n -bilinmeyen ve n -denklemin olduğu durumları ele alacağız.

(28.1) denklemlerini uygun biçimde, matris-vektör denklemi

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

veya buna denk olarak $A = (a_{ij})$, $x = (x_i)$, $b = (b_i)$ olmak üzere

$$Ax = b \tag{28.2}$$

şeklinde yeniden yazabiliriz.

Eğer A tekil değilse A^{-1} tersine sahiptir ve

$$x = A^{-1}b \tag{28.3}$$

çözüm vektörüdür. x çözümünün (28.2) denkleminin tek çözümü olduğunu şu şekilde kanıtlayabiliriz. x ve x' iki çözüm olsun. O zaman $Ax = Ax' = b$ eşitliği doğrudur ve I birim matris olmak üzere,

$$\begin{aligned}
x &= Ix \\
&= (A^{-1}A)x \\
&= A^{-1}(Ax) \\
&= A^{-1}(Ax') \\
&= (A^{-1}A)x' \\
&= x'
\end{aligned}$$

elde edilir.

Bu bölümde ağırlıklı olarak A matrisinin tekil olmadığı veya (Teorem D.1 den) A matrisinin rankının n bilinmeyen sayısına eşit olduğu durumlarla ilgileneceğiz. Bununla beraber kısa bir tartışmayı hakeden diğer olasılıklar da vardır. Eğer denklem sayısı, n bilinmeyen sayısından az ise veya daha genel olarak A matrisinin rankı n den küçük ise o zaman sistem **belirsizdir (underdetermined)**. Denklemler tutarsız (inconsistent) olduğunda sistemin çözümü olmamasına rağmen, belirsiz bir sistem tipik olarak sonsuz çözüme sahiptir. Eğer denklem sayısı,

n bilinmeyen sayısından fazla ise, sistem **aşırı belirsizdir (overdetermined)** ve sistemin herhangi bir çözümü olmayabilir. Kısım 28.3, aşırı belirsiz denklem sistemlerinin iyi yaklaşık çözümlerinin bulunması problemini anlatır.

Şimdi n denklemden oluşan n bilinmeyenli $Ax = b$ sisteminin çözümü problemimize geri dönelim. A^{-1} matrisini hesaplayabilsek o zaman (28.3) denklemini kullanarak b yi A^{-1} ile çarparak $x = A^{-1}b$ çözümüne ulaşabiliriz. Bu yaklaşım uygulamada sayısal kararsızlığa neden olur. Neyse ki, başka bir yaklaşım—LUP ayrışımı—sayısal karardır ve uygulamada daha hızlı olma avantajına sahiptir.

LUP ayrışımına genel bakış

LUP ayrışımı yönteminin arkasındaki fikir

$$PA = LU \quad (28.4)$$

olacak şekilde üç $n \times n$ boyutlu L , U ve P matrislerinin bulunmasıdır, burada

- L , birim alt-üçgensel matris,
- U , alt-üçgensel matris ve
- P , permütasyon matrisidir.

(28.4) denklemini sağlayan L , U ve P matrislerine A matrisinin bir **LUP ayrışımı** denir. Biz tekil olmayan her A matrisinin böyle bir ayrışımına sahip olacağını göstereceğiz.

A matrisinin bir LUP ayrışımı hesabı, bu matrisler üçgensel matris olduklarında doğrusal sistemin çözümünü kolaylaştıran avantajlara sahiptir. Önce A matrisinin bir LUP ayrışımını buluruz, O zaman (28.2) ile verilen $Ax = b$ denklem sistemini sadece üçgensel doğrusal sistemleri çözerek aşağıdaki gibi çözebiliriz. $Ax = b$ sisteminin her iki yanını P matrisi ile çarparak bu sisteme eşdeğer $PAx = Pb$ sistemini buluruz. (28.4) ayrışımını kullanırsak

$$LUx = Pb$$

denklemini elde ederiz. Böylelikle iki doğrusal üçgensel sistemi çözerek bu denklemi çözebiliriz. x aranan çözüm vektörü olmak üzere $y = Ux$ tanımlamasını yapalım. Önce “ileri yerine koyma” adı verilen yöntemi kullanarak

$$Ly = Pb \quad (28.5)$$

alt üçgen sistemini çözerek y vektörünü buluruz. O zaman y çözümünü kullanarak x çözümünü bulmak için “geri yerine koyma” adı verilen yöntemle

$$Ux = y \quad (28.6)$$

üst üçgen sistemini çözeriz. P permütasyon matrisi tersinir olduğundan (Alıştırma D.2-3) (28.4) denkleminin her iki yanını P^{-1} ile çarparak $P^{-1}PA = P^{-1}LU$ buradan da

$$A = P^{-1}LU \quad (28.7)$$

bulunur. Bu nedenle x vektörü $Ax = b$ sisteminin aradığımız çözümüdür.

$$\begin{aligned} Ax &= P^{-1}LUx \quad ((28.7) \text{ denkleminde}) \\ &= P^{-1}Ly \quad ((28.6) \text{ denkleminde}) \\ &= P^{-1}Pb \quad ((28.5) \text{ denkleminde}) \\ &= b. \end{aligned}$$

Bir sonraki adımımız ileri ve geri yerine koymanın nasıl çalıştığını göstermek ve LUP ayrışımının kendisini hesaplama problemine geçmektir.

İleri ve geri yerine koyma

İleri yerine koyma (forward substitution), verilen L, P ve b için $\Theta(n^2)$ süresinde (28.5) alt-üçgensel sistemini çözebilir. Kolaylık sağlamak için P permütasyon matrisini kompakt olarak $\pi[1..n]$ dizisi ile gösterelim. $i = 1, 2, \dots, n$ için $\pi[i]$ elemanı, $P_{i,\pi[i]} = 1$ ve $j \neq \pi[i]$ için $P_{ij} = 0$ olduğunu gösterir. Böylelikle PA matrisi, i satır ve j sütununda $a_{\pi[i],j}$ elemanına ve Pb matrisi ise i satırda $b_{\pi[i]}$ elemanına sahiptir. L alt-üçgensel matris olduğundan (28.5) denklemini aşağıdaki şekilde yeniden yazabiliriz:

$$\begin{aligned} y_1 &= b_{\pi[1]}, \\ l_{21}y_1 + y_2 &= b_{\pi[2]}, \\ l_{31}y_1 + l_{32}y_2 + y_3 &= b_{\pi[3]}, \\ &\vdots \\ l_{n1}y_1 + l_{n2}y_2 + l_{n3}y_3 + \dots + y_n &= b_{\pi[n]}. \end{aligned}$$

Birinci denklem bize $y_1 = b_{\pi[1]}$ olduğunu anlatır. Bulunan y_1 değerini ikinci denklemde yerine yazarak ikinci değer bulunur:

$$y_2 = b_{\pi[2]} - l_{21}y_1.$$

Şimdi y_1 ve y_2 üçüncü denklemde yerine yazılarak üçüncü değeri bulabiliriz:

$$y_3 = b_{\pi[3]} - (l_{31}y_1 + l_{32}y_2).$$

Genel olarak y_i değerini bulmak için $y_1, y_2, \dots, y_{i-1}$ değerleri “ileri doğru” i denklemde yerine yazılır:

$$y_i = b_{\pi[i]} - \sum_{j=1}^{i-1} l_{ij} y_j .$$

y elde edilerek ileri yerine koymaya benzer olarak **geri yerine koyma (back substitution)** yöntemini kullanarak (28.6) denkleminde x_i i çözeriz. Burada önce n .nci denklemi çözeriz ve birinci denkleme doğru geriye doğru çalışırız. İleri yerine koymaya benzer olarak, bu prosedür $\Theta(n^2)$ süresinde çalışır. U üst-üçgensel matris olduğundan (28.6) sistemini aşağıdaki şekilde yeniden yazabiliriz:

$$\begin{aligned} u_{11}x_1 + u_{12}x_2 + \cdots + u_{1,n-2}x_{n-2} + u_{1,n-1}x_{n-1} + u_{1n}x_n &= y_1 , \\ u_{22}x_2 + \cdots + u_{2,n-2}x_{n-2} + u_{2,n-1}x_{n-1} + u_{2n}x_n &= y_2 , \\ &\vdots \\ u_{n-2,n-2}x_{n-2} + u_{n-2,n-1}x_{n-1} + u_{n-2,n}x_n &= y_{n-2} , \\ u_{n-1,n-1}x_{n-1} + u_{n-1,n}x_n &= y_{n-1} , \\ u_{n,n}x_n &= y_n . \end{aligned}$$

Böylece $x_n, x_{n-1}, \dots, x_1$ i ardışık olarak aşağıdaki gibi çözebiliriz:

$$\begin{aligned} x_n &= y_n / u_{n,n} , \\ x_{n-1} &= (y_{n-1} - u_{n-1,n}x_n) / u_{n-1,n-1} , \\ x_{n-2} &= (y_{n-2} - (u_{n-2,n-1}x_{n-1} + u_{n-2,n}x_n)) / u_{n-2,n-2} , \\ &\vdots \end{aligned}$$

veya genel olarak,

$$x_i = \left(y_i - \sum_{j=i+1}^n u_{ij}x_j \right) / u_{ii}$$

bulunur.

Verilen P, L, U ve b için LUP-SOLVE prosedürü ileri ve geri yerine koyma yöntemini birleştirerek x çözümünü bulur. Aşağıdaki kabakoddaki $L.rows$ özelliği L nin satır sayısını, n boyutu olarak kabul eder ve permütasyon matrisi, π dizisi ile gösterilir.

LUP-SOLVE(L, U, π, b)

```

1   $n = L.rows$ 
2   $x$  ve  $n$  uzunluğunda yeni vektörler olsun
3  for  $i = 1$  to  $n$ 
4       $y_i = b_{\pi[i]} - \sum_{j=1}^{i-1} l_{ij} y_j$ 
5  for  $i = n$  downto 1
6       $x_i = (y_i - \sum_{j=i+1}^n u_{ij} x_j) / u_{ii}$ 
7  return  $x$ 
```

LUP-SOLVE prosedürü, 3-4 satırlarında ileri yerine koymayı kullanarak y çözümünü ve sonra 5-6 satırlarında geri yerine koymayı kullanarak da x çözümünü bulur. **for** döngülerindeki her bir toplam bir kapalı döngü içerdiği için çalışma süresi $\Theta(n^2)$ dir.

Bu yöntemlere örnek olarak aşağıdaki gibi tanımlı doğrusal denklem sistemini ele alalım

$$\begin{pmatrix} 1 & 2 & 0 \\ 3 & 4 & 4 \\ 5 & 6 & 3 \end{pmatrix} x = \begin{pmatrix} 3 \\ 7 \\ 8 \end{pmatrix}$$

burada

$$A = \begin{pmatrix} 1 & 2 & 0 \\ 3 & 4 & 4 \\ 5 & 6 & 3 \end{pmatrix},$$

$$b = \begin{pmatrix} 3 \\ 7 \\ 8 \end{pmatrix},$$

şeklinde ve biz x bilinmeyen vektörünü bulmak istiyoruz. LUP ayrışımı

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 0.2 & 1 & 0 \\ 0.6 & 0.5 & 1 \end{pmatrix},$$

$$U = \begin{pmatrix} 5 & 6 & 3 \\ 0 & 0.8 & -0.6 \\ 0 & 0 & 2.5 \end{pmatrix},$$

$$P = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

olarak elde edilir ($PA = LU$ olduğunu sağlatabilirsiniz). İleri yerine koyma yöntemini kullanarak y için $Ly = Pb$ eşitliğini çözeriz.

$$\begin{pmatrix} 1 & 0 & 0 \\ 0.2 & 1 & 0 \\ 0.6 & 0.5 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 3 \\ 7 \end{pmatrix}$$

Önce y_1 , sonra y_2 ve son olarak da y_3 ü hesaplayarak

$$y = \begin{pmatrix} 8 \\ 1.4 \\ 1.5 \end{pmatrix}$$

elde edilir. Geri yerine koyma yöntemi kullanılarak, x için $Ux = y$ eşitliğini çözeriz:

$$\begin{pmatrix} 5 & 6 & 3 \\ 0 & 0.8 & -0.6 \\ 0 & 0 & 2.5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 1.4 \\ 1.5 \end{pmatrix},$$

böylece önce x_3 , sonra x_2 ve son olarak da x_1 hesaplanarak aranan x çözümü

$$x = \begin{pmatrix} -1.4 \\ 2.2 \\ 0.6 \end{pmatrix}$$

olarak elde edilir.

LU ayrışımının hesaplanması

Tekil olmayan A matrisinin bir LUP ayrışımını elde ettiğimizde ileri ve geri yerine koyma yöntemlerinin $Ax = b$ doğrusal denklem sistemini çözebileceğini yukarıda gösterdik. Şimdi A matrisinin bir LUP ayrışımının etkili şekilde nasıl hesaplanacağını göstereceğiz. A matrisinin $n \times n$ boyutlu tekil olmadığı ve P nin var olmadığı (veya denk olarak $P = I_n$) olduğu durumu ele alarak başlayacağız. Bu durumda A yı $A = LU$ şeklinde yazabiliriz. L ve U matrislerine, A matrisinin bir **LU ayrışımı** diyeceğiz.

LU ayrışımını oluşturmak için **Gauss yok etme** olarak bilinen süreci uygulayırız. Her bir denklemdeki ilk değişkeni yok etmek için, birinci denklemin katlarını o denklemlerden çıkartarak başlarız. O zaman, ikinci denklemin katlarını üçüncü denklem ve alt denklemlerden çıkartırız ve böylelikle birinci ve ikinci değişkenler o denklemlerden yok edilir. Sistem bir üst-üçgensel yapıya dönüşene kadar bu prosedüre devam ederiz ki bu oluşan aslında U matrisidir. L matrisi yok edilen değişkenlerin satır çarpanlarından oluşur.

Bu stratejiyi uyguladığımız algoritma özyinelemeli(recursive)dir. Tekil olmayan, $n \times n$ boyutlu A matrisinin bir LU ayrışımını oluşturmak istiyoruz. $n = 1$ olduğunda $L = L_1$ ve $U = A$ seçerek bunu yaparız. $n > 1$ olduğunda A matrisini aşağıdaki şekilde dört parçaya ayırırız,

$$\begin{aligned} A &= \left(\begin{array}{c|ccc} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{array} \right) \\ &= \begin{pmatrix} a_{11} & w^T \\ v & A' \end{pmatrix}, \end{aligned}$$

burada v , $(n - 1)$ boyutlu sütun vektörü, w^T , $(n - 1)$ boyutlu satır vektörü ve A' ise $(n - 1) \times (n - 1)$ boyutlu matristir. O zaman matris cebirine göre (çarparak

denklemleri sağlatın) A matrisini aşağıdaki gibi yazabiliriz

$$\begin{aligned} A &= \begin{pmatrix} a_{11} & w^T \\ v & A' \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & A' - vw^T/a_{11} \end{pmatrix}. \end{aligned} \quad (28.8)$$

(28.8) denkleminin birinci ve ikinci matrislerindeki 0 lar sırasıyla $(n-1)$ boyutlu satır ve sütun vektörleridir. v ve w vektörlerinin dış çarpımınının herbir teriminin a_{11} e bölümünden elde edilen vw^T/a_{11} terimi, $(n-1) \times (n-1)$ boyutlu matristir. Sonuç olarak elde edilen $(n-1) \times (n-1)$ boyutlu

$$A' - vw^T/a_{11} = L'U' \quad (28.9)$$

matrisine A matrisinin a_{11} e göre **Schur tümleyeni** adı verilir.

İddia ediyoruz ki A matrisi tekil olmayan matris olduğunda onun Schur tümleyeni de tekil olmayan matristir. Neden? Schur tümleyeninin $(n-1) \times (n-1)$ boyutlu tekil matris olduğunu varsayalım. O zaman Teorem D.1 e göre onun satır rankı $n-1$ den kesin olarak küçüktür.

$$\begin{pmatrix} a_{11} & w^T \\ 0 & A' - vw^T/a_{11} \end{pmatrix}$$

matrisinin birinci sütununun altındaki $n-1$ elemanın hepsi sıfır olduğundan bu matrisin alttaki $n-1$ satırının satır rankı $n-1$ den küçük olmalıdır. Buna göre tüm matrisin satır rankı kesinlikle n den küçüktür. (28.8) denkleminde Alıştırma D.2-8 i uygulayarak A matrisinin rankının kesin olarak n den küçük olduğu ve Teorem D.1 den A matrisinin tekil olduğu sonucu elde edilir ki bu bir çelişkidir. Schur tümleyeni tekil olmadığından şimdi biz özyinelemeli olarak onun için bir LU ayrışımı bulabiliriz. L' , alt-üçgensel matris, U' ise üst-üçgensel matris olmak üzere

$$A' - vw^T/a_{11} = L'U'$$

olduğunu kabul edelim. O zaman matris işlemlerini kullanarak

$$\begin{aligned} A &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & A' - vw^T/a_{11} \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & L'U' \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ v/a_{11} & L' \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & U' \end{pmatrix} \\ &= LU, \end{aligned}$$

eşitliğini ve böylelikle de LU ayrışımını elde ederiz (L' alt-üçgensel olduğundan L alt-üçgensel ve U' üst-üçgensel olduğundan U üst-üçgensel matris olduğuna dikkat ediniz).

$a_{11} = 0$ olduğunda sıfıra bölme olacağından bu yöntemin çalışmayacağı açıktır. $A' - vw^T/a_{11}$ Schur tümleyeninin sol üst elemanının sıfır olması durumunda da bir sonraki aşamada yine sıfıra bölme olacağından yöntem yine çalışmaz. LU ayrışımı süresince bölen bu elemanlara **pivot** adı verilir ve bunlar U matrisinin köşegen elemanlarını oluştururlar. LUP ayrışımı sürecinde P permütasyon matrisi kullanılmasının sebebi sıfıra bölmelerden kaçınmak içindir. Sıfıra (veya sayısal kararsızlığa neden olan küçük sayılara) bölmeyi önlemek için permütasyonlar kullandığımızda **pivottlama** yapmış oluruz.

LU ayrışımının düzgün çalıştığı önemli bir matris sınıfı, pozitif tanımlı simetrik matrisler sınıfıdır. Bu tür matrisler pivottlama işlemini gerektirmez ve böylece sıfıra bölme korkusunu yaşamadan yukarıda özetlenen özyinelemeli stratejiyi uygulayabiliriz. Bu ve diğer birkaç sonucu Kısım 28.3 de kanıtlayacağız.

A matrisinin LU ayrışımı için kodumuz, özyinelemenin iterasyon döngüsüyle yerdeğiştirdiği durum haricinde, özyinelemeli stratejiyi takip eder. (Bu dönüşüm bir “kuyruk-özyinelemeli” prosedür-ki bu son işlemin, kendisine bir özyinelemeli çağrıdır- için standart bir optimizasyondur. Problem 7-4 e bakınız). $A.rows$ özelliği, A nın boyutunu verir. Başlangıç olarak U nun köşegenin altında kalan elemanlarını 0 ve L nin köşegen elemanlarını 1, köşegenin üstünde kalanları 0 ahyoruz.

LU-DECOMPOSITION(A)

```

1   $n = A.rows$ 
2   $L$  ve  $U$   $n \times n$  boyutlu yeni matrisler olsun
3   $U$  matrisini köşegen altını 0 alarak başlat
4   $L$  matrisini köşegen üzerini 1 ler ve köşegen üstünü 0 lar alarak başlat
5  for  $k = 1$  to  $n$ 
6       $u_{kk} = a_{kk}$ 
7      for  $i = k + 1$  to  $n$ 
8           $l_{ik} = a_{ik}/a_{kk}$            //  $a_{ik}$  sağlar  $v_i$ 
9           $u_{ki} = a_{ki}$                //  $a_{ki}$  sağlar  $w_i$ 
10     for  $i = k + 1$  to  $n$ 
11         for  $j = k + 1$  to  $n$ 
12              $a_{ij} = a_{ij} - l_{ik}u_{kj}$ 
13  return  $L$  ve  $U$ 
```

Satır 5 de başlayan dış **for** döngüsü, her bir özyineleme adımında birkez çalışır. Bu döngü ile satır 6, pivotu $u_{kk} = a_{kk}$ olarak belirler. 7-9 satırlarındaki **for** döngüsü (ki $k = n$ olduğunda çalışmaz), L ve U matrislerini oluşturmak için v ve w^T vektörlerini kullanır. Satır 8, v vektörünün v_i elemanlarını belirler ki o l_{ik} da saklanır, satır 9 w^T vektörünün w_i^T elemanlarını hesaplar ki o u_{ki} de saklanır. Son olarak satır 10-12 Schur tamlayanın elemanlarını hesaplar v tekrar onları A mat-

(a) $\begin{pmatrix} 2 & 3 & 1 & 5 \\ 6 & 13 & 5 & 19 \\ 2 & 19 & 10 & 23 \\ 4 & 10 & 11 & 31 \end{pmatrix}$

(b) $\begin{pmatrix} \textcircled{2} & 3 & 1 & 5 \\ 3 & 4 & 2 & 4 \\ 1 & 16 & 9 & 18 \\ 2 & 4 & 9 & 21 \end{pmatrix}$

(c) $\begin{pmatrix} 2 & 3 & 1 & 5 \\ 3 & \textcircled{4} & 2 & 4 \\ 1 & 4 & 1 & 2 \\ 2 & 1 & 7 & 17 \end{pmatrix}$

(d) $\begin{pmatrix} 2 & 3 & 1 & 5 \\ 3 & 4 & 2 & 4 \\ 1 & 4 & \textcircled{1} & 2 \\ 2 & 1 & 7 & 3 \end{pmatrix}$

(e)
$$\begin{pmatrix} 2 & 3 & 1 & 5 \\ 6 & 13 & 5 & 19 \\ 2 & 19 & 10 & 23 \\ 4 & 10 & 11 & 31 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 3 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 2 & 1 & 7 & 1 \end{pmatrix} \begin{pmatrix} 2 & 3 & 1 & 5 \\ 0 & 4 & 2 & 4 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 3 \end{pmatrix}$$

Şekil 28.1 LU-DECOMPOSITION işlemi. **(a)** A matrisi. **(b)** Siyah dairedeki $a_{11} = 2$ elemanı pivottur, gölgeli sütun v/a_{11} ve gölgeli satır w^T dir. U nun buraya kadar hesaplanan elemanları yatay çizginin üzerinde ve L nin elemanları düşey çizginin solundadır. Schur tümleyen matrisi $A' - vw^T/a_{11}$ sağ alt taraftadır. **(c)** Şimdi (b) bölümünde üretilen Schur tümleyen matrisi üzerinde işlem yapıyoruz. Siyah dairedeki $a_{22} = 4$ elemanı pivottur, ve gölgeli sütun ve satır sırasıyla v/a_{22} ve w^T (Schur tümleyeninin parçalanışındaki) dir. Çizgiler, verilen matrisi, U nun o ana kadar hesaplanan elemanlarına ayırır (üst), L nin o ana kadar hesaplanan elemanları (sol) ve yeni Schur tümleyeni (sağ alt). **(d)** Bir sonraki adımdan sonra, A matrisi ayrıştırılmıştır. (Özyineleme sonlandığında, yeni Schur tümleyenindeki 3 elemanı U nun parçası olur.) **(e)** $A = LU$ ayrışımı.

risine geri yerleştirerek saklar. (12. satırda a_{kk} ile bölmemiz gerekmiyor çünkü 8. satırda l_{ik} yı hesaplarken bunu yapmıştık). 12. satır üçlü iç içe olduğundan LU DECOMPOSITION $\Theta(n^3)$ süresinde çalışır.

Şekil 28.1, LU-DECOMPOSITION işlemini görüntüler. L ve U matrislerinin önemli unsurlarını A matrisinde sakladığımız prosedürün standart optimizasyonunu gösterir. Yani, biz her bir a_{ij} elemanını ile ya l_{ij} ($i > j$ ise) veya u_{ij} ($j \leq i$ ise) elemanı arasında bir eşleştirme oluşturabilir ve işlem sonlandığında U ve L matrisinin her ikisini de tutan A matrisini yenileyebiliriz. Yukarıdaki kabakoddan bu optimizasyonun kabakodunu elde etmek için sadece her bir l veya u referansını a ile yer değiştiririz; Bu dönüşümün gerçekliği koruduğunu kolaylıkla gösterebilirsiniz.

LUP ayrışımının hesaplanması

Genel olarak, $Ax = b$ lineer denklem sistemi çözerken sıfıra bölmeden kaçınmak için A matrisinin köşegen elemanları üzerinde pivotlama işlemi yapmalıyız. 0 ile bölme, elbette, felaket olurdu. A tekil olmayan bir matris olsa bile çok küçük bir

sayıya bölmeden de kaçınmak istiyoruz çünkü bu sayısal kararsızlıklara neden olur. Bundan dolayı pivot olarak büyük bir değeri almaya çalışırız.

LUP ayrışımının arkasındaki matematik, LU ayrışımındakine benzerdir. Hatırlayın, bize $n \times n$ boyutlu tekil olmayan bir A matrisi verilmişti ve $PA = LU$ eşitliğini sağlayan bir P permütasyon matrisi, L alt-üçgensel matrisi ve U alt-üçgensel matrisini bulmak istemiştik. A matrisini, LU ayrışımında olduğu gibi, parçalamadan önce sıfırdan farklı herhangi bir a_{k1} elemanını birinci sütunda matrisin (1,1) konumuna taşıyoruz. Sayısal kararlılık için a_{k1} elemanını birinci sütunda mutlak değerce en büyük eleman olacak şekilde seçeriz. (Birinci sütun tamamıyla 0 lardan oluşamaz, çünkü o zaman Teorem D.4 ve D.5 e göre determinantı sıfır olacağından A matrisi tekil olurdu.). Denklem grubunu korumak amacıyla, A matrisini soldan bir Q permütasyon matrisiyle çarpmaya denk olan 1. ve k . satırları yer değiştiririz (Alıştırma D.1-4). Böylece QA matrisini

$$QA = \begin{pmatrix} a_{k1} & w^T \\ v & A' \end{pmatrix}$$

şeklinde yazabiliriz, burada a_{k1} ; $w^T = (a_{k2}, a_{k3}, \dots, a_{kn})^T$; a_{11} ile a_{k1} yer değiştirmesi hariç, $v = (a_{21}, a_{31}, \dots, a_{n1})$ ve A' , $(n-1) \times (n-1)$ boyutlu matristir. Şimdi, $a_{k1} \neq 0$ olduğu için LU ayrışımındaki benzer cebirsel işlemleri sıfıra bölme tehlikesini garantileyerek gerçekleştirebiliriz:

$$\begin{aligned} QA &= \begin{pmatrix} a_{k1} & w^T \\ v & A' \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & A' - vw^T/a_{k1} \end{pmatrix}. \end{aligned}$$

LU ayrışımında gördüğümüz üzere A tekil olmayan matris olduğunda $A' - vw^T/a_{k1}$ Schur tümleyeni de tekil olmayan matristir. Böylece

$$P'(A' - vw^T/a_{k1}) = L'U'$$

olacak şekilde L' alt-üçgensel matrisi, U' alt-üçgensel matrisi ve P' permütasyon matrisi için özyinelemeli olarak bir LUP ayrışımı bulabiliriz.

Aşağıdaki matrisini tanımlayalım:

$$P = \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} Q$$

burada P , iki permütasyon matrisinin çarpımından oluştuğundan kendisi de bir permütasyon matrisidir (Alıştırma D.1-4). Şimdi LUP ayrışımını üreterek

$$\begin{aligned}
PA &= \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} QA \\
&= \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} \begin{pmatrix} 1 & 0 \\ v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & A' - vw^T/a_{k1} \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & P' \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & A' - vw^T/a_{k1} \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & P'(A' - vw^T/a_{k1}) \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & L'U' \end{pmatrix} \\
&= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & L' \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & U' \end{pmatrix} \\
&= LU,
\end{aligned}$$

elde ederiz. L' birim alt-üçgensel matris olduğundan L alt-üçgensel matris ve U' alt-üçgensel matris olduğundan U alt-üçgensel matristir.

Bu üretimde, LU ayrışımının aksine, v/a_{k1} sütun vektörü ile $A' - vw^T/a_{k1}$ Schur tümleyeninin her ikisini de P permütasyon matrisi ile çarpmak gerektiğine dikkat ediniz. İşte LUP ayrışımının kabakodu:

LUP-DECOMPOSITION(A)

```

1   $n = A.rows$ 
2   $\pi[1..n]$  yeni bir matris olsun
3  for  $i = 1$  to  $n$ 
4       $\pi[i] = i$ 
5  for  $k = 1$  to  $n$ 
6       $p = 0$ 
7      for  $i = k$  to  $n$ 
8          if  $|a_{ik}| > p$ 
9               $p = |a_{ik}|$ 
10              $k' = i$ 
11      if  $p == 0$ 
12          error “tekil matris”
13      yerdeğiştir  $\pi[k]$  ile  $\pi[k']$ 
14      for  $i = 1$  to  $n$ 
15          yerdeğiştir  $a_{ki}$  ile  $a_{k'i}$ 
16      for  $i = k + 1$  to  $n$ 
17           $a_{ik} = a_{ik}/a_{kk}$ 
18          for  $j = k + 1$  to  $n$ 
19               $a_{ij} = a_{ij} - a_{ik}a_{kj}$ 

```


LU-DECOMPOSITION prosedürüne benzer olarak LUP-DECOMPOSITION prosedürümüz bir özyinelemeyi bir iterasyon döngüsüyle yer değiştirir. Özyinelemenin doğrudan uygulanmasında bir iyileştirme yaparak, dinamik olarak P permütasyon matrisini bir π dizisi olarak saklarız, burada $\pi[i] = j$, P permütasyon matrisinin i . satırının j . sütununun 1 e eşit olduğu anlamına gelmektedir. Ayrıca L ve U matrislerini A matrisinin içinde “ilgili yerde” hesaplamak için kodu uygulayız. Bu yüzden, işlem sona erdiğinde

$$a_{ij} = \begin{cases} l_{ij}, & i > j \text{ ise,} \\ u_{ij}, & i \leq j \text{ ise,} \end{cases}$$

olur.

Şekil 28.2, LUP-DECOMPOSITION un bir matrisi nasıl çarpanlara ayırdığını gösteriyor. 3-4 satırlarında birim permütasyon matrisi π dizisi ile gösterilir. Satır 5 deki dış **for** döngüsü, özyinelemeyi uygular. Dış döngü süresince her defasında, 6-10 satırları LUP ayrışımını yaptığımız $(n - k + 1) \times (n - k + 1)$ matrisinin ilk sütunundaki (sütun k) mutlak değerce en büyük elemanı $a_{k'k}$ olarak belirler. Eğer birinci sütundaki tüm elemanlar sıfır ise, 11-12 satırları matrisin tekil matris olduğunu söyler. Pivotlama için, biz 13. satırda $\pi[k']$ ile $\pi[k]$ yı ve 14-15. satırlarda da A matrisinin k ve k' satırlarını değiştiriyoruz, böylece pivot elemanı a_{kk} yapıyoruz (Yukarıda verilen yöntemde, P' matrisiyle sadece $A' - v w^T / a_{k1}$ değil v / a_{k1} de çarpıldığından tüm satırlar değiştirilir). Son olarak, LU-DECOMPOSITION algoritmasının 7-12 satırlarındakine benzer olarak Schur tümleyeni, 16-19 satırlarında bir farkla aynı yolla hesaplanır ki burada işlem yerinde gerçekleştirilir.

LU-DECOMPOSITION da olduğu gibi, sahip olduğu iç içe üçlü döngü yapısı sebebiyle LUP-DECOMPOSITION un da çalışma süresi $\Theta(n^3)$ dir. Böylece, pivotlama bize, zaman olarak en çok bir sabit çarpanı kadar süreye mal olur.

Alıştırmalar

28.1-1

İleri yerine koyma yöntemini kullanarak

$$\begin{pmatrix} 1 & 0 & 0 \\ 4 & 1 & 0 \\ -6 & 5 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 3 \\ 14 \\ -7 \end{pmatrix}$$

denklemini çözünüz.

28.1-2

$$\begin{pmatrix} 4 & -5 & 6 \\ 8 & -6 & 7 \\ 12 & -7 & 12 \end{pmatrix}$$

matrisi için bir LU ayrışımı bulunuz.

(a)
$$\begin{array}{c|cccc} 1 & 2 & 0 & 2 & 0.6 \\ 2 & 3 & 3 & 4 & -2 \\ 3 & 5 & 5 & 4 & 2 \\ 4 & -1 & -2 & 3.4 & -1 \end{array}$$

(b)
$$\begin{array}{c|cccc} 3 & 5 & 5 & 4 & 2 \\ 2 & 3 & 3 & 4 & -2 \\ 1 & 2 & 0 & 2 & 0.6 \\ 4 & -1 & -2 & 3.4 & -1 \end{array}$$

(c)
$$\begin{array}{c|cccc} 3 & 5 & 5 & 4 & 2 \\ 2 & 0.6 & 0 & 1.6 & -3.2 \\ 1 & 0.4 & -2 & 0.4 & -0.2 \\ 4 & -0.2 & -1 & 4.2 & -0.6 \end{array}$$

(d)
$$\begin{array}{c|cccc} 3 & 5 & 5 & 4 & 2 \\ 2 & 0.6 & 0 & 1.6 & -3.2 \\ 1 & 0.4 & -2 & 0.4 & -0.2 \\ 4 & -0.2 & -1 & 4.2 & -0.6 \end{array}$$

(e)
$$\begin{array}{c|cccc} 3 & 5 & 5 & 4 & 2 \\ 1 & 0.4 & -2 & 0.4 & -0.2 \\ 2 & 0.6 & 0 & 1.6 & -3.2 \\ 4 & -0.2 & -1 & 4.2 & -0.6 \end{array}$$

(f)
$$\begin{array}{c|cccc} 3 & 5 & 5 & 4 & 2 \\ 1 & 0.4 & -2 & 0.4 & -0.2 \\ 2 & 0.6 & 0 & 1.6 & -3.2 \\ 4 & -0.2 & 0.5 & 4 & -0.5 \end{array}$$

(g)
$$\begin{array}{c|cccc} 3 & 5 & 5 & 4 & 2 \\ 1 & 0.4 & -2 & 0.4 & -0.2 \\ 2 & 0.6 & 0 & 1.6 & -3.2 \\ 4 & -0.2 & 0.5 & 4 & -0.5 \end{array}$$

(h)
$$\begin{array}{c|cccc} 3 & 5 & 5 & 4 & 2 \\ 1 & 0.4 & -2 & 0.4 & -0.2 \\ 4 & -0.2 & 0.5 & 4 & -0.5 \\ 2 & 0.6 & 0 & 1.6 & -3.2 \end{array}$$

(i)
$$\begin{array}{c|cccc} 3 & 5 & 5 & 4 & 2 \\ 1 & 0.4 & -2 & 0.4 & -0.2 \\ 4 & -0.2 & 0.5 & 4 & -0.5 \\ 2 & 0.6 & 0 & 0.4 & -3 \end{array}$$

(j)
$$\begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 2 & 0 & 2 & 0.6 \\ 3 & 3 & 4 & -2 \\ 5 & 5 & 4 & 2 \\ -1 & -2 & 3.4 & -1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0.4 & 1 & 0 & 0 \\ -0.2 & 0.5 & 1 & 0 \\ 0.6 & 0 & 0.4 & 1 \end{pmatrix} \begin{pmatrix} 5 & 5 & 4 & 2 \\ 0 & -2 & 0.4 & -0.2 \\ 0 & 0 & 4 & -0.5 \\ 0 & 0 & 0 & -3 \end{pmatrix}$$

Şekil 28.2 LUP-DECOMPOSITION işlemi. **(a)** Soldaki satırların birim permütasyonlu A girdi matrisi. Algoritmanın birinci adımı, üçüncü satırda siyah çember içindeki 5 elemanını birinci sütunun pivotu olarak belirler. **(b)** Satır 1 ve 3 yer değiştirilir ve permütasyon güncellenir. Gölge sütun ve satır v ve w^T yi gösterir. **(c)** v vektörü $v/5$ ile yer değiştirilir ve matrisin sağ altı Schur tümleyeni olarak yenilenir. Çizgiler matrisi üç bölgeye parçalar: U nun elemanları (üst), L nin elemanları (sol), ve Schur tümleyeninin elemanları (sağ alt). **(d)–(f)** İkinci adım. **(g)–(i)** Üçüncü adım. Dördüncü (son) adımda başka bir değişiklik olmaz. **(j)** $PA = LU$, LUP ayrışımı.

28.1-3

LUP ayrışımını kullanarak

$$\begin{pmatrix} 1 & 5 & 4 \\ 2 & 0 & 3 \\ 5 & 8 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 12 \\ 9 \\ 5 \end{pmatrix}$$

denklemini çözünüz.

28.1-4

Bir köşegen matris için LUP ayrışımını açıklayınız.

28.1-5

Bir A permütasyon matrisi için LUP ayrışımını açıklayarak, bu ayrışımın tek olduğunu kanıtlayınız.

28.1-6

Her $n \geq 1$ için bir LU ayrışımına sahip $n \times n$ boyutlu tekil bir matrisin olduğunu gösteriniz.

28.1-7

LU-DECOMPOSITION algoritmasında $k = n$ durumunda en dıştaki **for** iterasyon döngüsünün gerçekleştirilmesi gerekli midir? LUP-DECOMPOSITION için durum ne olur?

28.2 Matrislerin tersini alma

Doğrusal denklem sistemlerinin çözümü için pratikte ters matrisleri genellikle kullanmamamıza rağmen, LUP ayrışımı gibi sayısal olarak daha kararlı teknikleri tercih ederek bazen bir matrisin tersini hesaplamaya ihtiyaç duyarız. Bu bölümde, bir matrisin tersini hesaplamak için LUP ayrışımını nasıl kullanacağımızı göstereceğiz. Ayrıca matris çarpımının ve bir matrisin tersinin hesaplanmasının denk zorlukta problemler olduğunu kanıtlayacağız öyle ki (teknik koşullara dayanarak) birini çözmek için kullandığımız bir algoritmayı aynı asimptotik çalışma süresinde diğeri için de kullanabiliriz. Nitekim, bir matrisin tersini hesaplamak için matris çarpımını Strassen algoritmasıyla (Kısım 4.2 ye bakınız) bulabiliriz. Gerçekten, Strassen'in orijinal makalesi, bir doğrusal denklem sisteminin sıradan yöntemlerden daha hızlı çözülebileceğini gösteren problemle motive edilmiştir.

LUP ayrışımıyla bir matrisin tersinin hesaplanması

$PA = LU$ olacak şekilde L , U ve P matrislerinin oluşturduğu A matrisinin bir LUP ayrışımına sahip olduğumuzu varsayalım. LUP-SOLVE algoritmasını kullanarak $Ax = b$ sisteminin çözümünü $\Theta(n^2)$ süresinde çözebiliriz. LUP ayrışımı b ye değil A matrisine bağlı olduğundan LUP-SOLVE algoritmasını ikinci bir $Ax = b'$ sistemi için ek $\Theta(n^2)$ süresinde çalıştırabiliriz. Genel olarak, A matrisinin bir LUP ayrışımına sahip olduğumuz sürece her farklı b için $Ax = b$ denkleminin k farklı versiyonunu $\Theta(kn^2)$ süresinde çözebiliriz.

A matrisinin tersi olan X matrisini tanımlayan

$$AX = I_n, \quad (28.10)$$

denklemini, $Ax = b$ şeklinde n farklı denklem kümesi olarak düşünebiliriz. Daha açık olmak gerekirse, X_i vektörü X in i . sütun vektörünü gösterecek ve e_i birim vektörü I_n matrisinin i . sütunu olsun. O zaman A nın bir LUP ayrışımını kullanarak her bir

$$AX_i = e_i$$

denklemden X_i yi çözerek 28.10 denkleminin X çözümünü elde edebiliriz. LUP ayrışımına birkez sahip olduğumuzda, her bir n -tane X_i sütunlarını $\Theta(n^2)$ süresinde hesaplayabilir ve böylelikle A nın bir LUP ayrışımından X çözümünü $\Theta(n^3)$ süresinde hesaplayabiliriz. A nın LUP ayrışımını $\Theta(n^3)$ süresinde bulabildiğimiz için A nın A^{-1} tersini de $\Theta(n^3)$ süresinde hesaplayabiliyoruz.

Matris çarpımı ve matrisin tersinin bulunması

Şimdi matris çarpımı için elde edilen teorik hızlanmanın matris tersinin bulunmasına dönüşeceğini göstereceğiz. Aslında, daha güçlü bir şey kanıtlayacağız: Matrisin tersinin bulunması, aşağıdaki anlamda matris çarpımına eşdeğerdir. $M(n)$, $n \times n$ boyutlu iki matrisin çarpım süresi olsun, o zaman $n \times n$ boyutlu tekil olmayan bir matrisinin tersini $O(M(n))$ süresinde hesaplayabiliriz. Üstelik, $I(n)$, $n \times n$ boyutlu tekil olmayan matrisin tersinin bulunması süresi ise, o zaman $n \times n$ boyutlu iki matrisi biz, $O(I(n))$ süresinde çarpabiliriz. Bu sonuçları iki ayrı teorem olarak kanıtlayacağız.

Teorem 28.1 (Matrislerin çarpımı, matrisin tersinin bulunmasından zor değildir) $I(n) = \Omega(n^2)$ ve $I(n)$ süresi, $I(3n) = O(I(n))$ düzenlilik koşulunu sağlaması kaydıyla, $n \times n$ boyutlu bir matrisin tersini $I(n)$ süresinde bulabilirsek, o zaman $n \times n$ boyutlu iki matrisi $O(I(n))$ süresinde çarpabiliriz.

Kanıt A ve B $n \times n$ boyutlu matrisler ve C hesaplayacağımız çarpım matrisi olsun. $n \times n$ boyutlu D matrisini tanımlayalım

$$D = \begin{pmatrix} I_n & A & 0 \\ 0 & I_n & B \\ 0 & 0 & I_n \end{pmatrix}.$$

D matrisinin tersi

$$D^{-1} = \begin{pmatrix} I_n & -A & AB \\ 0 & I_n & -B \\ 0 & 0 & I_n \end{pmatrix}$$

matrisidir ve böylece AB çarpım matrisini D^{-1} matrisinin sağ üst köşesinde $n \times n$ boyutlu altmatris oluşturarak hesaplayabiliriz.

D matrisini $\Theta(n^2)$ süresinde oluşturabiliriz ki $I(n) = \Omega(n^2)$ varsayımına göre bu $O(I(n))$ süresidir ve D matrisinin tersini düzenlilik koşuluna göre $O(I(3n)) = O(I(n))$ süresinde hesaplayabiliriz. Böylece $M(n) = O(I(n))$ elde ederiz. ■

Keyfi $c > 0$ ve ≥ 0 sabitleri için $I(n)$ süresinin $I(n) = \Theta(n^c \lg^d n)$ düzgünlük koşulunu her zaman sağlayacağını not ediniz.

Matrisin tersinin bulunmasının matrislerin çarpımından zor olmadığına kanıtı simetrik pozitif-tanımlı matrislerin Kısım 28.3 de kanıtlayacağımız bazı özelliklerine dayanır.

Teorem 28.2 (Matrisin tersinin hesaplanması, matrislerin çarpımından zor değildir)

$n \times n$ boyutlu iki matrisi $M(n)$ süresinde çarpabildiğimizi varsayalım, burada $M(n) = \Omega(n^2)$ ve $M(n)$, $0 \leq k \leq n$ olmak üzere keyfi k için $M(n+k) = O(M(n))$ ve $c < 1/2$ için $M(n/2) \leq c M(n)$ olmak üzere iki düzenlilik koşulunu sağlar. O zaman gerçel, tekil olmayan $n \times n$ boyutlu keyfi matrisin tersini $O(M(n))$ süresinde hesaplayabiliriz.

Kanıt Biz burada teoremi gerçel matrisler için kanıtlayacağız. Alıştırma 28.2-6 da karmaşık matrisler için kanıtın genelleştirilmesi sorulmaktadır.

Keyfi $k > 0$ için

$$\begin{pmatrix} A & 0 \\ 0 & I_k \end{pmatrix}^{-1} = \begin{pmatrix} A^{-1} & 0 \\ 0 & I_k \end{pmatrix}$$

olduğundan, n nin 2 nin bir tam kuvveti olduğunu kabul edebiliriz. Böylelikle $n + k$, 2 nin bir kuvveti olacak şekilde k yı seçerek matrisin boyutunu 2 nin bir sonraki kuvvetine genişletiriz ve aranan A^{-1} cevabını genişletilmiş problemin cevabından elde ederiz. $M(n)$ üzerindeki birinci düzenlilik koşulu, bu genişlemenin çalışma süresinin bir sabit çarpandan daha fazlasına arttırmayacağını garanti eder.

Bir an için $n \times n$ boyutlu A matrisinin simetrik ve pozitif tanımlı olduğunu varsayalım. A matrisini ve onun A^{-1} tersinin $n/2 \times n/2$ boyutlu dört alt matrise parçalayalım:

$$A = \begin{pmatrix} B & C^T \\ C & D \end{pmatrix} \quad \text{ve} \quad A^{-1} = \begin{pmatrix} R & T \\ U & V \end{pmatrix}. \quad (28.11)$$

O zaman

$$S = D - CB^{-1}C^T \quad (28.12)$$

matrisini A matrisinin B ye göre Schur tümleyeni olarak kabul edersek (Schur tümleyeninin bu şeklini, Kısım 28.3 de daha kapsamlı göreceğiz),

$$A^{-1} = \begin{pmatrix} R & T \\ U & V \end{pmatrix} = \begin{pmatrix} B^{-1} + B^{-1}C^TS^{-1}CB^{-1} & -B^{-1}C^TS^{-1} \\ -S^{-1}CB^{-1} & S^{-1} \end{pmatrix} \quad (28.13)$$

elde ederiz ki matris çarpımını gerçekleştirerek $AA^{-1} = I_n$ olduğunu sağlatabilirsiniz. A , simetrik ve pozitif tanımlı olduğundan Kısım 28.3 deki Lemmalar 28.4 ve 28.5 e göre B ve S nin her ikisi de simetrik ve pozitif tanımlıdır. Böylece Kısım 28.3 deki Lemma 28.3 e göre B^{-1} ve S^{-1} ters matrisleri vardır ve Alıştırma D.2-6 ya göre B^{-1} ve S^{-1} simetrik matrislerdir, yani $(B^{-1})^T = B^{-1}$ ve $(S^{-1})^T = S^{-1}$ olur. Nitekim A^{-1} matrisinin $n/2 \times n/2$ boyutlu R , T , U ve V matrislerini aşağıdaki şekilde hesaplayabiliriz

1. A matrisinin B , C , C^T ve D altmatrislerini oluştur.
2. Özyinelemeli olarak B matrisinin B^{-1} tersini hesapla
3. $W = CB^{-1}$ matris çarpımını ve sonra transpozesi $B^{-1}C^T$ ye eşit olan W^T yi hesapla (Alıştırma D.1-2 ye göre $(B^{-1})^T = B^{-1}$)
4. Eşiti $CB^{-1}C^T$ olan $X = WC^T$ matris çarpımını hesapla ve sonra $S = D - X = D - CB^{-1}C^T$ matrisini hesapla.
5. Özyinelemeli olarak S matrisinin S^{-1} tersini hesapla ve S^{-1} e V de.
6. Eşiti $S^{-1}CB^{-1}C$ matrisi olan $Y = S^{-1}W$ matris çarpımını hesapla ve sonra transpozesi $B^{-1}C^TS^{-1}$ olan YT yi hesapla (Alıştırma D.1-2 ye göre $(B^{-1})^T = B^{-1}$ ve $(S^{-1})^T = S^{-1}$). $-Y^T$ e T ve $-Y$ e U de.
7. Eşiti $B^{-1}C^TS^{-1}CB^{-1}$ olan $Z = W^TY$ matris çarpımını hesapla ve $B^{-1} + Z$ ye R de.

Böylece, 2. ve 5. adımlarda $n/2 \times n/2$ boyutlu iki matrisin tersini hesaplayarak; 3, 4, 6, ve 7. adımlarda $n/2 \times n/2$ boyutlu dört çarpımı gerçekleştirerek; A matrisinden altmatrisleri çıkartarak, A^{-1} e altmatrisleri ekleyerek ve $O(n^2)$ ek hesaplama yaparak ve $n/2 \times n/2$ boyutlu matrisler üzerindeki bir sabit kadar toplama, çıkartma ve transpoze işlemlerini gerçekleştirerek $n \times n$ boyutlu simetrik pozitif-tanımlı bir matrisin tersini bulabiliriz.

$$\begin{aligned} I(n) &\leq 2I(n/2) + 4M(n/2) + O(n^2) \\ &= 2I(n/2) + \Theta(M(n)) \\ &= O(M(n)) \end{aligned}$$

özyinelemesini elde ederiz.

Teoremin ifadesindeki ikinci düzenlilik koşulu $4M(n/2) < 2M(n)$ koşulunu gerektirir ve $M(n) = \Omega(n^2)$ kabul ettiğimiz için ikinci satır sağlanır. İkinci düzenlilik koşulu ana teoremin (Teorem 4.1) 3. durumunu uygulamamıza imkan tanıdığı için üçüncü satır devam eder.

Simetrik pozitif-tanımlı olmayan A matrisi tersinir olduğunda matrisin tersinin bulunmasında olduğu gibi matris çarpımı için de benzer asimptotik çalışma süresini elde edebileceğimizi kanıtlamak mümkündür. Temel düşünce keyfi tekil olmayan A matrisi için $A^T A$ matrisi simetrik (Alıştırma D.1-2) ve pozitif-tanımlı (Teorem D.6) olmasıdır. O zaman hile, A nın tersinin bulunması problemini $A^T A$ nın tersinin bulunması problemine indirgemektir.

İndirgeme, A , $n \times n$ boyutlu tekil olmayan matris olduğunda, $((A^T A)^{-1} A^T) A = (A^T A)^{-1} (A^T A) = I_n$ ve bir matrisin tersinin tekliliğinden

$$A^{-1} = (A^T A)^{-1} A^T$$

eşitliğinin elde edilebileceği gözlemine dayanır. Nitekim, $A^T A$ yı elde etmek için önce, A^T yi A ile çarpıp sonra yukarıdaki böl-ve-yönet algoritmasını kullanarak simetrik pozitif-tanımlı $A^T A$ matrisinin tersini hesaplayarak ve son olarak elde edilen sonucu A^T ile çarparak A^{-1} yi hesaplayabiliriz. Bu üç adımın her biri $O(M(n))$ kadar süreyi kapsar ve böylece keyfi tekil olmayan reel değerli matrisin tersini $O(M(n))$ süresinde bulabiliriz. ■

Teorem 28.2 nin kanıtı, A tekil olmayan matris olduğu sürece, pivotlama yapmaksızın LU ayrışımını kullanarak $Ax = b$ denkleminin çözüm araçlarını gösterir. $(A^T A)x = A^T b$ eşitliğini elde etmek için denklemin her iki yanını A^T ile çarpılır. A^T tersinir matris olduğundan, bu dönüşümden x çözümü etkilenmez, ve böylece bir LU ayrışımını hesaplayarak simetrik pozitif-tanımlı $A^T A$ matrisini çarpanlarına ayırabiliriz. O zaman x çözümünü bulmak için $A^T b$ sağ tarafı ile birlikte ileri ve geri yerine koyma yöntemini kullanabiliriz. Bu yöntem teorik olarak doğru olmasına rağmen, pratikte LUP-DECOMPOSITION algoritması daha iyi çalışır. LUP ayrışımı bir sabit çarpan kadar daha az aritmetik işlem gerektirir, ve biraz daha iyi sayısal özelliklere sahiptir.

Alıştırmalar

28.2-1

$M(n)$, $n \times n$ boyutlu iki matrisin çarpım süresi olsun ve $S(n)$, $n \times n$ boyutlu bir matrisi kareleştirmek için gerekli süreyi gösterson. Matrisleri çarpma ve karelemenin aslında aynı zorluğa sahip olduğunu gösteriniz: Yani, bir $M(n)$ -zamanlı matris çarpım algoritması, bir $O(M(n))$ -zamanlı kareleme algoritmasını gerektirir ve bir $S(n)$ -zamanlı kareleme algoritması, bir $O(S(n))$ -zamanlı matris çarpım algoritmasını gerektirir.

28.2-2

$M(n)$ $n \times n$ boyutlu iki matrisin çarpımı için gereken zaman olsun. $M(n)$ -zamanlı matrisi çarpımı algoritmasının bir $O(M(n))$ -zamanlı LUP-ayrışımı algoritmasını gerektirdiğini gösteriniz.

28.2-3

$M(n)$, $n \times n$ boyutlu iki matrisin çarpım süresi ve $D(n)$, boyutlu bir matrisinin determinantını hesaplama süresi olsun. Matrisleri çarpma ve determinantını hesaplamanın aslında aynı zorluğa sahip olduğunu gösteriniz: Yani, bir $M(n)$ -zamanlı matris çarpım algoritması, bir $O(M(n))$ -zamanlı determinant hesaplama algoritmasını gerektirir ve bir $D(n)$ -zamanlı determinant hesaplama algoritması, bir $O(D(n))$ -zamanlı matris çarpım algoritmasını gerektirir.

28.2-4

$M(n)$, $n \times n$ boyutlu iki boolean matrisin çarpım süresi ve $T(n)$, $n \times n$ boyutlu bir boole matrisinin geçişli-kapanışının bulunma süresi olsun. (Kısım 25.2 ye bakınız). Bir $M(n)$ -zamanlı boole matris çarpım algoritmasının, bir $O(M(n) \lg n)$ - geçişli-kapanma algoritması süresini gerektirdiğini ve bir $T(n)$ -zamanlı geçişli-kapanış algoritmasının, bir $O(T(n))$ -zamanlı boole matris çarpım algoritmasını gerektirdiğini gösteriniz.

28.2-5

Matris elemanlarının tam sayıların mod 2 cisminden oluşması durumunda, Teorem 28.2 ye dayanan matrisin tersinin bulunması algoritması çalışır mı? Açıklayınız.

28.2-6 *

Teorem 28.2 ile verilen matrisin tersinin bulunması algoritmasını kompleks sayılardan oluşan matrisler için genelleştiriniz ve, bu genelleştirmenin doğru çalıştığını kanıtlayınız. (*İpucu:* A matrisinin transpozu yerine A matrisin her elemanının eşlenik transpozese alınmasıyla elde edilen A^* **eşlenik transpozese** matrisini kullanınız. Simetrik matrisler yerine $A = A^*$ eşitliği ile verilen **Hermityen** matrisleri dikkate alınız.

28.3 Simetrik pozitif-tanımlı matrisler ve en küçük-kareler yaklaşımı

Simetrik pozitif-tanımlı matrisler birçok ilginç ve çekici özelliklere sahiptir. Örneğin, onlar tekil olmayan matrislerdir ve onlar üzerinde herhangi bir sığa bölme endişesi yaşamadan LU ayrışımı yöntemini gerçekleştirebiliriz. Bu bölümde,

simetrik pozitif-tanımlı matrislerin diğer birkaç önemli özelliğini kanıtlayacağız ve bir en küçük kareler yaklaşımıyla eğri uydurma problemi için ilginç bir uygulama göstereceğiz

Kanıtlayacağımız ilk özellik belki de en temel olanıdır.

Lemma 28.3

Keyfi pozitif-tanımlı matris tekil olmayan matristir.

Kanıt A matrisi tekil matris olsun. O zaman Sonuç D.3 e göre, $Ax = 0$ olacak şekilde sıfırdan farklı bir x vektörü vardır. Dolayısıyla, $x^T Ax = 0$ ve A pozitif-tanımlı olamaz. ■

Simetrik pozitif-tanımlı bir matris için sıfıra bölme olmaksızın LU ayrışımını gerçekleştirebileceğimiz bir kanıt daha karmaşıktır. A matrisinin belli altmatrisleri hakkındaki özellikleri kanıtlayarak başlıyoruz. A matrisinin ilk k satır ve ilk k sütunun kesişmesiyle oluşan k . **ön altmatris (leading submatrix)** A_k olarak tanımlayalım.

Lemma 28.4

A matrisi bir simetrik pozitif-tanımlı matris ise o zaman A matrisinin her ön altmatrisi simetrik ve pozitif-tanımlıdır.

Kanıt Herbir A_k ön altmatrisinin simetrik olacağı açıktır. A_k nın pozitif-tanımlılığını göstermek için tersini varsayalım ve çelişki elde etmeye çalışalım. A_k pozitif-tanımlı olmasın, o zaman $x_k \neq 0$ olacak şekilde bir k -vektör vardır ve $x_k^T A_k x_k \leq 0$. A , $n \times n$ boyutlu matris olsun, ve A matrisinin B $((n-k) \times k)$ boyutlu ve A $((n-k) \times (n-k))$ boyutlu altmatrisleri için

$$A = \begin{pmatrix} A_k & B^T \\ B & C \end{pmatrix} \quad (28.14)$$

eşitliği sağlansın. $x = (x_k^T \ 0)^T$ n -vektörü tanımlayalım, burada x_k dan sonra gelen 0 , $n-k$ tane sıfırı göstermektedir. O zaman

$$\begin{aligned} x^T Ax &= (x_k^T \ 0) \begin{pmatrix} A_k & B^T \\ B & C \end{pmatrix} \begin{pmatrix} x_k \\ 0 \end{pmatrix} \\ &= (x_k^T \ 0) \begin{pmatrix} A_k x_k \\ B x_k \end{pmatrix} \\ &= x_k^T A_k x_k \\ &\leq 0 \end{aligned}$$

eşitsizliğini elde ederiz ki bu ise A matrisinin pozitif-tanımlı olmasıyla çelişir. ■

Şimdi **Schur tümleyeninin** bazı temel özelliklerine döneceğiz. A simetrik, pozitif-tanımlı bir matris, ve A_k ise A matrisinin $k \times k$ boyutlu bir ön altmatrisi olsun. A matrisinin (28.14) denklemine göre tekrar parçalanışını ele alalım. A matrisinin A_k ya göre S Schur tümleyenini tanımlamak için (28.9) denklemini aşağıdaki gibi genelleştiririz:

$$S = C - BA_k^{-1}B^T \quad (28.15)$$

(Lemma 28.4 e göre, A_k simetrik ve pozitif-tanımlıdır, dolayısıyla Lemma 28.3 e göre A_k^{-1} vardır ve S iyi tanımlıdır.) Schur tümleyeninin, (28.9) ile verilen daha önceki tanımının $k = 1$ için (28.15) denklemi ile aynı olacağına dikkat ediniz.

Bir sonraki lemma simetrik pozitif-tanımlı matrislerin Schur tümleyen matrislerinin kendilerinin de simetrik pozitif-tanımlı olduğunu gösteriyor. Bu sonucu Teorem 28.2’de kullandık, ve simetrik pozitif-tanımlı matrislerin LU ayrışımının doğruluğunu kanıtlamak için onun doğal sonucuna ihtiyacımız var.

Lemma 28.5 (Schur tümleyeni lemması)

A , simetrik pozitif-tanımlı bir matris ve A_k , A matrisinin $k \times k$ boyutlu ön altmatrisi ise, o zaman A nın A_k ya göre S Schur tümleyeni simetrik pozitif-tanımlıdır.

Kanıt A simetrik olduğu için C altmatrisi de simetriktir. Alıştırma D.2-6 dan $BA_k^{-1}B^T$ çarpımı ve Alıştırma D.1-1’e göre de S simetrik matristir.

Geriye S matrisinin pozitif-tanımlılığını göstermek kalıyor. (28.14) denkleminin verilen A matrisinin parçalanışını ele alalım. A matrisinin pozitif tanımlı olması varsayımından sıfırdan farklı keyfi x vektörü için $x^T Ax > 0$ elde ederiz. A_k ve C matrisleriyle karşılıklı uyumlu olacak şekilde x vektörünü y ve z olarak iki altvektöre ayıralım. A^{-1} , var olduğundan, matris sihiriyle

$$\begin{aligned} x^T Ax &= \begin{pmatrix} y^T & z^T \end{pmatrix} \begin{pmatrix} A_k & B^T \\ B & C \end{pmatrix} \begin{pmatrix} y \\ z \end{pmatrix} \\ &= \begin{pmatrix} y^T & z^T \end{pmatrix} \begin{pmatrix} A_k y + B^T z \\ B y + C z \end{pmatrix} \\ &= y^T A_k y + y^T B^T z + z^T B y + z^T C z \\ &= (y + A_k^{-1} B^T z)^T A_k (y + A_k^{-1} B^T z) + z^T (C - BA_k^{-1} B^T) z \end{aligned} \quad (28.16)$$

eşitliğini elde ederiz (doğruluğunu çarpımları yaparak sağlattırın). Bu son denklem, kuadratik ifadenin “kareye tamamlanması” anlamına gelmektedir (Alıştırma 28.3-2 ye bakınız).

Sıfırdan farklı keyfi x vektörü için $x^T Ax > 0$ doğru olduğundan, sıfırdan farklı keyfi z vektörünü belirleyerek ve sonra (28.16) denkleminin ilk terimini sıfırlayan $y = -A_k^{-1} B^T z$ seçimini yapıp eşitliğin değerini

$$z^T (C - BA_k^{-1} B^T) z = z^T S z$$

olarak bulunur. Dolayısıyla $z \neq 0$ için $z^T S z = x^T Ax > 0$, elde ederiz ve böylece S , pozitif tanımlıdır. ■

Sonuç 28.6

Simetrik pozitif-tanımlı bir matrisin LU ayrışımı asla sıfırla bölmeye neden olmaz.

Kanıt A , simetrik pozitif-tanımlı matris olsun. Bu sonucun ifadesinden daha güçlü bir şey kanıtlayacağız: her pivot pozitifdir. İlk pivot a_{11} dir. e_1 ile birinci birim vektörü gösterelim, buna göre $a_{11} = e_1^T A e_1 > 0$ olur. A matrisinin $A_1 = (a_{11})$ e göre Schur tümleyeni, LU ayrışımının ilk basamanda üretildiğinden tümevarım yöntemiyle Lemma 28.5 den tüm pivotların pozitif olduğunu elde ederiz. ■

En küçük-kareler yaklaşımı

Simetrik pozitif-tanımlı matrislerin önemli bir uygulaması, verilen noktalar kümesine en iyi uyan eğrinin bulunması problemlerinde ortaya çıkar. Bize m nokta kümesinin verildiğini varsayalım

$$(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$$

burada y_i değerleri ölçüm hataları içerir. $i = 1, 2, \dots, m$ olmak üzere

$$\eta_i = F(x_i) - y_i \quad (28.17)$$

yaklaşım hataları küçük olacak şekilde bir $F(x)$ fonksiyonunu belirlemek istiyoruz. F fonksiyonunun yapısı ele alınan probleme bağlıdır. Burada, onun yapısının

$$F(x) = \sum_{j=1}^n c_j f_j(x)$$

doğrusal ağırlıklı toplam şeklinde olduğunu varsayıyoruz. Toplanan terim sayısını ifade eden n ve f_i **baz fonksiyonları** ele alınan problemin bilgisine dayanılarak seçilir. Yaygın seçim $f_j(x) = x^{j-1}$ şeklindedir ki bu, $F(x)$ fonksiyonunun x değişkenine göre $(n-1)$. dereceden bir polinom olması anlamına gelir

$$F(x) = c_1 + c_2 x + c_3 x^2 + \dots + c_n x^{n-1}.$$

Böylelikle $(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)$ şeklinde verilen m tane noktayla, $\eta_1, \eta_2, \dots, \eta_m$ yaklaşım hatalarını minimize eden n tane $c_1, c_2, \dots, c_n$ katsayılarını hesaplamak isteriz.

$n = m$ seçimiyle, (28.17) denklemindeki her bir y_i değerini *kesin olarak* hesaplayabiliriz. Böyle yüksek dereceli bir F , veriye olduğu kadar “hatayı da uyar”, ancak, x in önceden bilinmeyen değerleri için tahmini y değerleri kullanıldığında genellikle zayıf sonuç verir. n yi m den daha küçük seçmek genellikle daha iyidir ve c_j katsayıları iyi seçilerek hatanın aşırılığına dikkat etmeksizin veri noktalarındaki önemli kalıpları bulan bir F fonksiyonunun bulabileceği umut edilir. n yi

seçmek için bazı teorik ilkeler vardır, ancak onlar bu metnin kapsamı dışındadır. Herhangi bir durumda, m den daha küçük bir n değeri seçmekle, yaklaşık olarak çözmek istediğimiz bir doğrusal denklem sistemini oluşturmuş oluruz. Şimdi bunun nasıl yapılacağını gösterelim.

$$A = \begin{pmatrix} f_1(x_1) & f_2(x_1) & \dots & f_n(x_1) \\ f_1(x_2) & f_2(x_2) & \dots & f_n(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_1(x_m) & f_2(x_m) & \dots & f_n(x_m) \end{pmatrix}$$

matrisi baz fonksiyonlarının verilen noktalardaki değerlerinin matrisini göster-sin: yani $a_{ij} = f_j(x_i)$. $c = (c_k)$, aranan katsayıların n -vektörü olsun. O zaman

$$\begin{aligned} Ac &= \begin{pmatrix} f_1(x_1) & f_2(x_1) & \dots & f_n(x_1) \\ f_1(x_2) & f_2(x_2) & \dots & f_n(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_1(x_m) & f_2(x_m) & \dots & f_n(x_m) \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{pmatrix} \\ &= \begin{pmatrix} F(x_1) \\ F(x_2) \\ \vdots \\ F(x_m) \end{pmatrix} \end{aligned}$$

y için “tahmini değerlerin” m -vektörüdür. Nitekim

$$\eta = Ac - y$$

yaklaşım hatalarının m -vektörüdür.

Yaklaşım hatalarını minimize etmek için bize bir **en küçük kareler çözümünü** veren η hata vektörünün normunu minimize etmeyi seçeriz,

$$\|\eta\| = \left(\sum_{i=1}^m \eta_i^2 \right)^{1/2}.$$

$$\|\eta\|^2 = \|Ac - y\|^2 = \sum_{i=1}^m \left(\sum_{j=1}^n a_{ij} c_j - y_i \right)^2$$

olduğundan,

$\|\eta\|^2$ yı her bir c_k ya göre diferansiyelleyip sonucu sıfıra eşitleyerek $\|\eta\|$ yi minimize edebiliriz:

$$\frac{d \|\eta\|^2}{dc_k} = \sum_{i=1}^m 2 \left(\sum_{j=1}^n a_{ij} c_j - y_i \right) a_{ik} = 0 . \quad (28.18)$$

O zaman $k = 1, 2, \dots, n$ olmak üzere (28.18) denklemleri tek bir matris denkleme

$$(Ac - y)^T A = 0$$

veya buna denk olarak (Aıştırma D.1-2 yi kullanarak)

$$A^T (Ac - y) = 0$$

matris denklemine denktir ki buradan

$$A^T Ac = A^T y \quad (28.19)$$

elde edilir.

İstatistikte buna **normal denklemler** adı verilir. Aıştırma D.1-2 ye göre $A^T A$ matrisi simetriktir ve eğer A matrisinin rankı n ise, o zaman Teorem 6 ya göre $A^T A$ aynı zamanda pozitif tanımlıdır. Dolayısıyla, $(A^T A)^{-1}$ vardır ve (28.19) denkleminin çözümü

$$\begin{aligned} c &= ((A^T A)^{-1} A^T) y \\ &= A^+ y \end{aligned} \quad (28.20)$$

dir, burada $A^+ = ((A^T A)^{-1} A^T)$ matrisi, A matrisinin **sözdetersidir (pseudoinverse)**.

Sözdeters A , kare matris olmadığında matris tersi notasyonunu doğal olarak genelleştirir. $(Ac = y)$ denkleminin (28.20) yaklaşık çözümünü, $Ax = b$ denkleminin $A^{-1}b$ gerçek çözümü ile karşılaştır.

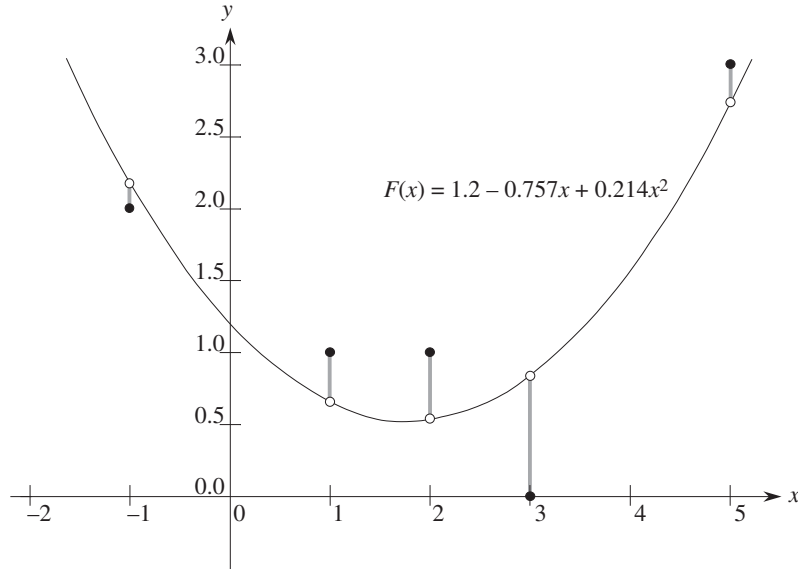
En küçük kareler çözümünü oluşturmaya örnek olarak Şekil 28.3'de siyah noktalar olarak gösterilen

$$\begin{aligned} (x_1, y_1) &= (-1, 2) , \\ (x_2, y_2) &= (1, 1) , \\ (x_3, y_3) &= (2, 1) , \\ (x_4, y_4) &= (3, 0) , \\ (x_5, y_5) &= (5, 3) , \end{aligned}$$

noktalarına sahip olduğumuzu varsayalım. Bu noktaları

$$F(x) = c_1 + c_2 x + c_3 x^2$$

şeklinde bir ikinci dereceden polinomla yaklaştıralım.



Şekil 28.3 Beş noktalı $\{(-1, 2), (1, 1), (2, 1), (3, 0), (5, 3)\}$ veri kümesinin ikinci dereceden polinomla en küçük kareler yaklaşımı. Siyah noktalar veri noktalarıdır, ve beyaz noktalar onların $F(x) = 1.2 - 0.757x + 0.214x^2$ polinomuyla bulunmuş yaklaşık değerleridir ki bu ikinci dereceden polinom kareleri alınmış hatalar toplamını minimize eder. Her bir gölgeli çizgi o noktadaki hatayı gösterir.

Sözdetersi

$$A^+ = \begin{pmatrix} 0.500 & 0.300 & 0.200 & 0.100 & -0.100 \\ -0.388 & 0.093 & 0.190 & 0.193 & -0.088 \\ 0.060 & -0.036 & -0.048 & -0.036 & 0.060 \end{pmatrix}$$

olan

$$A = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \\ 1 & x_4 & x_4^2 \\ 1 & x_5 & x_5^2 \end{pmatrix} = \begin{pmatrix} 1 & -1 & 1 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \\ 1 & 5 & 25 \end{pmatrix}$$

baz fonksiyonlarının verilen noktalardaki değerlerinin matrisi ile başlıyoruz. y yi A^+ ile çarparak ikinci dereceden polinomuna karşılık gelen katsayılar vektörünü

$$c = \begin{pmatrix} 1.200 \\ -0.757 \\ 0.214 \end{pmatrix}$$

olarak ve en küçük kareler anlamında verilen noktaları en iyi yaklaştıran ikinci dereceden polinomu ise

$$F(x) = 1.200 - 0.757x + 0.214x^2$$

şeklinde elde ederiz.

Pratik bir yol olarak, y yi A^T ile çarparak ve sonrasında $A^T A$ matrisinin bir LU ayrışımını bularak (28.19) normal denklemlerini çözeriz. Eğer A matrisinin rankı n ise, $A^T A$ matrisinin tekil olmadığı garantilenir, zira o simetrik ve pozitif tanımlıdır. (Alıştırma D.1-2 ve Teorem D.6 ya bakınız.)

Alıştırmalar

28.3-1

Simetrik pozitif-tanımlı matrisin her köşegen elemanının pozitif olduğunu kanıtlayınız.

28.3-2

$A = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$, simetrik pozitif-tanımlı 2×2 boyutlu bir matris olsun. Lemma 28.5 in kanıtında kullanılan bir duruma benzer şekilde “kareye tamamlayarak” onun determinantı olan $ac - b^2$ nin pozitif olduğunu kanıtlayınız.

28.3-3

Simetrik pozitif-tanımlı bir matrisin maksimum elemanının köşegen üzerinde bulunacağını kanıtlayınız.

28.3-4

Simetrik pozitif-tanımlı bir matrisin her bir ön altmatrisinin determinantının pozitif olacağını kanıtlayınız.

28.3-5

A_k simetrik pozitif-tanımlı bir A matrisin k . ön altmatrisi olsun. LU ayrışımında $\det(A_k) / \det(A_{k-1})$ nin k . pivot olduğunu kanıtlayınız, burada $\det(A_0) = 1$ dir.

28.3-6

(1,1), (2,1), (3,3), (4,8) noktalarına en iyi uyan en küçük kareler yaklaşımını

$$F(x) = c_1 + c_2 x \lg x + c_3 e^x$$

formunda bulunuz.

28.3-7

A^+ , sözdeters matrisinin aşağıdaki dört denklemi sağladığını gösteriniz:

$$\begin{aligned} AA^+A &= A, \\ A^+AA^+ &= A^+, \\ (AA^+)^T &= AA^+, \\ (A^+A)^T &= A^+A. \end{aligned}$$

Problemler**28-1 Tridiagonal doğrusal denklem sistemleri**

Aşağıdaki tridiagonal matris verilsin

$$A = \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 2 \end{pmatrix}.$$

- a. A matrisinin bir LU ayrışımını bulunuz.
- b. İleri ve geri yerine koymayı kullanarak $Ax = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \end{pmatrix}^T$ denklemini çözünüz.
- c. A matrisinin tersini bulunuz.
- d. Herhangi $n \times n$ boyutlu, simetrik pozitif-tanımlı, tridiagonal A matrisi ve b n -vektörü için LU ayrışımını gerçekleştirerek $O(n)$ süresinde $Ax = b$ denkleminin nasıl çözüleceğini gösteriniz. En kötü durumda, A^{-1} matrisinin oluşturulmasına dayalı herhangi yöntemin asimptotik olarak daha masraflı olmasını tartışınız.
- e. Herhangi $n \times n$ boyutlu, tekil olmayan, tridiagonal A matrisi ve b n -vektörü için LUP ayrışımını gerçekleştirerek $O(n)$ süresinde $Ax = b$ denkleminin nasıl çözüleceğini gösteriniz.

28-2 Spline Fonksiyonları

Bir veri kümesini bir eğri ile interpolasyonunu yapmanın pratik yöntemi **kübik spline** fonksiyonlarını kullanmaktır. $x_0 < x_1 < \dots < x_n$ olmak üzere $\{(x_i, y_i) : i = 0, 1, \dots, n\}$ şeklinde $n + 1$ tane nokta-değer çifti kümesi verilsin. Parçalı-kübik eğri (spline) fonksiyonunu verilen noktalara uydurmak isteyelim. Buna göre, $f(x)$ eğrisi $i = 0, 1, \dots, n - 1$ olmak üzere n

tane kübik $f_i(x) = a_i + b_i x + c_i x^2 + d_i x^3$ polinomundan oluşur, burada eğer $x_i \leq x \leq x_{i+1}$ ise o zaman eğrinin değeri $f(x) = f_i(x - x_i)$ ile verilir. Kübik polinomların birbirine eklendiği x_i noktalarına **düğüm noktaları (knots)** denir. Basitlik için, $i = 0, 1, \dots, n$ olmak üzere $x_i = i$ olduğunu kabul edelim.

$$\begin{aligned} f(x) \text{ fonksiyonunun sürekliliğinin temini için } i = 0, 1, \dots, n-1 \text{ olmak üzere} \\ f(x_i) &= f_i(0) = y_i, \\ f(x_{i+1}) &= f_i(1) = y_{i+1} \end{aligned}$$

koşullarının sağlanması gerekir. $f(x)$ fonksiyonunun yeterince pürüzsüzlüğü için ise her bir düğüm noktasında birinci türevinin sürekliliğini şart koşarız:

$$i = 0, 1, \dots, n-2 \text{ için}$$

$$f'(x_{i+1}) = f'_i(1) = f'_{i+1}(0).$$

- a.** $i = 0, 1, \dots, n$ için, her bir düğüm noktasında sadece $\{(x_i, y_i)\}$ nokta-değer çifti değil $D_i = f'(x_i)$ birinci türev değerlerinin de verildiğini varsayalım. Herbir a_i, b_i, c_i ve d_i katsayılarını y_i, y_{i+1}, D_i ve D_{i+1} değerleri cinsinden ifade edelim ($x_i = i$ olduğunu hatırlayalım.). Nokta-değer çifti ve türev değerlerinden $4n$ tane katsayıyı ne kadar hızlı hesaplayabiliriz?

Soru, $f(x)$ in düğüm noktalarındaki birinci türevlerinin nasıl seçileceği ile ilgilidir. Bir yöntem, düğüm noktalarında ikinci türevlerin sürekli olması koşuldur:

$$i = 0, 1, \dots, n-2 \text{ için}$$

$$f''(x_{i+1}) = f''_i(1) = f''_{i+1}(0).$$

İlk ve son düğüm noktasında, $f''(x_0) = f''_0(0) = 0$ ve $f''(x_n) = f''_{n-1}(1) = 0$ olduğunu varsayalım; Bu varsayımlar $f(x)$ eğrisini **doğal (natural)** bir kübik spline yapar.

- b.** $i = 1, 2, \dots, n-1$ olmak üzere

$$D_{i-1} + 4D_i + D_{i+1} = 3(y_{i+1} - y_{i-1}) \quad (28.21)$$

olduğunu göstermek için ikinci türev üzerindeki süreklilik kısıtını kullanınız.

$$\mathbf{c.} \quad 2D_0 + D_1 = 3(y_1 - y_0), \quad (28.22)$$

$$D_{n-1} + 2D_n = 3(y_n - y_{n-1}) \quad (28.23)$$

olduğunu gösteririz.

- d.** (28.21)–(28.23) denklemlerini $D = \langle D_0, D_1, \dots, D_n \rangle$ bilinmeyen vektörünü içeren matris denklemi olarak tekrar yazarız. Denkleminizde matris olması ne ifade eder?

- e.** Bir doğal kübik spline fonksiyonunun $n+1$ tane nokta-değer çifti kümesini $O(n)$ süresinde interpolasyon yapabilir mi tartışınız (Problem 28-1 e bakınız.).

f. x_i düğüm noktalarının i ye eşit olmasının gerekmediği durumda, $x_0 < x_1 < \dots < x_n$ olmak üzere $n + 1$ adet (x_i, y_i) noktalı kümeyi interpolasyon yapan bir doğal kübik spline fonksiyonunun nasıl belirleneceğini gösteriniz. Yönteminiz nasıl matris denklem çözmelidir, ve sizin algoritmanız ne kadar hızlı çalışır?

Bölüm notları

Mükemmel metinlerin çoğu bizim burada verdiğimizden çok daha fazla detaylı olarak sayısal ve bilimsel hesaplamayı açıklar. Özellikle aşağıdakiler okunabilir: George and Liu [132], Golub and Van Loan [144], Press, Teukolsky, Vetterling, ve Flannery [283, 284] ve Strang [323, 324].

Golub ve Van Loan [144] sayısal kararlılığı tartışır. Onlar $\det(A)$ sayısının neden bir A matrisin kararlılığının gerekli iyi bir göstergesi olmadığını $\|A\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |a_{ij}|$ olmak üzere bunun yerine $\|A\|_\infty \|A^{-1}\|_\infty$ sayısını kullanmayı önererek gösterdiler. Onlar aynı zamanda gerçekte A^{-1} matrisini hesaplamadan bu sayının nasıl hesaplanacağı sorusunu açıkladılar.

LU ve LUP ayrışımına dayanan Gauss yok etme, doğrusal denklem sistemlerinin çözümü için ilk sistematik yöntem oldu. Aynı zamanda, ilk sayısal algoritmaların biriydi. Daha önce bilinmesine rağmen, onun keşfi yaygın olarak C. F. Gauss'a (1777-1855) atfedilir. Meşhur makalesinde [325], Strassen $n \times n$ boyutlu bir matrisin $O(n^{\lg 7})$ süresinde tersinin alınabileceğini gösterdi. Winograd [358] orijinal olarak matris çarpımının matrisin tersinin bulunmasından zor olmadığını gösterdi ve bunun tersini de Aho, Hopcroft, and Ullman [5] gösterdi. Bir diğer önemli matris ayrışımı **tekil değer ayrıştırması (singular value decomposition)** veya **SVD** dir. SVD, $m \times n$ boyutlu bir A matrisini $A = Q_1 \Sigma Q_2^T$ şeklinde çarpanlara parçalar, burada Σ , $m \times n$ boyutlu sıfırdan farklı değerleri sadece köşegen üzerinde olan matris, Q_1 , $m \times m$ boyutlu sütunları ortonormal olan matris, ve Q_2 de, $n \times n$ boyutlu sütunları ortonormal olan matristir. İç çarpımları sıfır ve her birinin normu 1 olan iki vektör **ortonormaldir (orthonormal)**. Strang [323, 324], Golub ve Van Loan [144] tarafından yazılmış olan kitaplar SVD nin iyi uygulamalarını içerir.

Strang [324], simetrik pozitif-tanımlı matrisler ve genel olarak lineer cebirin mükemmel bir sunumuna sahiptir.

Birçok problem, verilen sınırlı kaynaklar ve rakip kısıtlar ile amacın minimize veya maksimize edildiği biçimde şekillenir. Eğer amacı, belli değişkenlerin bir doğrusal fonksiyonu gibi ve kaynak kısıtları, o değişkenler üzerinde eşitlikler veya eşitsizlikler şeklinde tanımlayabilirsek sonuçta bir ***doğrusal-programlama problemi (linear-programming problem)*** elde etmiş oluruz. Doğrusal programlar çeşitli kullanışlı uygulamalarda ortaya çıkmaktadır. Seçim politikalarından bir uygulamayı çalışarak başlayacağız.

Siyasal bir problem

Senin seçimi kazanmaya çalışan bir politikacı olduğunu varsayalım. Seçim bölgen üç farklı bölgeye—şehir, şehrin çevresindeki yerleşim alanları varoşlar ve köy—sahiptir. Bu bölgeler sırasıyla 100,000, 200,000 ve 50,000 kayıtlı seçmene sahiptir. Tüm kayıtlı seçmenler seçim bürolarına gitmesede etkili bir biçimde idare etmeye karar veriyorsun ve üç farklı bölgede de kayıtlı seçmenlerin en az yarısının senin için oy vermesini istiyorsun. Onurlu bir kişinin ve inanmadığın politikaları kesinlikle desteklemiyorsun. Bununla birlikte biliyorsun ki belli bölgelerden oy almak için belli sorun çözümleri daha etkili olabiliyor. Senin öncelikli sorunların, daha fazla yol inşa etmek, silah kontrolü, çiftlik sübvansiyonları ve gelişmiş toplu taşımaya hitap eden benzin vergisi. Senin seçim kampanyanda çalışanların araştırmalarına göre herbir nüfus bölgeden kazanacağın veya kaybedeceğin oy sayısını, herbir sorun için yapacağın reklamlarda \$1000 harcayarak tahmin edebilirsin. Bu bilgi Şekil 29.1 de yer alan tabloda verilmektedir. Bu tabloda, herbir girdi, belli bir sorunun reklamlarında harcanan \$1000 ile herbir şehir, varoş ve köy seçmen gruplarından kazanılacak oy sayısını belirtir. Negatif girdiler kaybedilen oyları belirtmektedir. Senin için 50,000 şehir oyu, 100,000 varoş oyu ve 25,000 köy oyu kazanmak için en az ne kadar harcaman gerektiğini ifade etmektedir.

Denemeler ve hatalar ile, gerekli oy sayısını kazandıran bir strateji geliştirebilirsin ancak, bulduğun bu strateji en az para harcanan olmayabilir. Örneğin yol yapım reklamları için 20,000\$ silah kontrolü için 0\$ çiftlik sübvansi-

politika	şehir	varoş	köy
yol inşası	-2	5	3
silah kontrolü	8	2	-5
çiftlik sübvansiyonları	0	0	10
benzin vergisi	10	0	-2

Şekil 29.1 Oy verenler üzerinde politikanın etkisi. Herbir girdi belli bir konuda bir politikanın reklam desteğinden 1000\$ harcanarak kazanılabilecek binlerce şehir, varoş ve köy oy sayısını belirtir. Negatif girdiler kaybedilecek oyları gösterir.

yonları için 4000\$ ve benzin vergileri için 9000\$ ayırabilirsin. Bu durumda $20(-2) + 0(8) + 4(0) + 9(10) = 50$ bin şehir oyu, $20(5) + 0(2) + 4(0) + 9(0) = 100$ bin varoş oyu ve $20(3) + 0(-5) + 4(10) + 9(-2) = 82$ bin köy oyu kazanırsın. Varoş ve şehir bölgelerinden tam istediğin sayıda ve köylerden istediğinden fazla sayıda oy kazanabilirsin. (Gerçekten, köy bölgesinde, seçmen sayısından daha fazla sayıda oy alabilirsin). Bu oyları kazanmak için reklam harcamaların $20 + 0 + 4 + 9 = 33$ bin dolardır.

Doğal olarak, bu stratejinin en iyi strateji olup olmadığını merak edebilirsin. Yani, reklam için daha az para harcayarak amaçlarına ulaşabilir misin? Ek bir deneme ve hata, bu soruya cevap vermene yardımcı olabilir, ancak bu tarz sorulara cevap vermek için daha sistematik bir yöntemin olsa daha iyi olmaz mı? Bir tane geliştirmek için bu soruyu matematiksel formül halinde ifade etmeliyiz. 4 değişken tanımlansın:

- x_1 yol yapım reklamlarında harcanan bin dolar sayısı,
- x_2 silah kontrol reklamlarında harcanan bin dolar sayısı,
- x_3 çiftlik sübvansiyonları reklamlarında harcanan bin dolar sayısı,
- x_4 benzin vergi reklamlarında harcanan bin dolar sayısı.

En az 50,000 şehirli oyu almamız için gerekenleri aşağıdaki şekilde yazabiliriz:

$$-2x_1 + 8x_2 + 0x_3 + 10x_4 \geq 50 . \quad (29.1)$$

Benzer olarak, en az 100,000 varoş ve 25,000 köylü oyu kazanmak için gerekenleri aşağıdaki şekilde yazabiliriz:

$$5x_1 + 2x_2 + 0x_3 + 0x_4 \geq 100 \quad (29.2)$$

ve

$$3x_1 - 5x_2 + 10x_3 - 2x_4 \geq 25 . \quad (29.3)$$

(29.1)-(29.3) eşitsizliklerini sağlayan x_1, x_2, x_3, x_4 değişkenler dizisi, herbir tip oydan yeterli sayıda elde etmeyi sağlar. Giderleri mümkün olduğunca az tutmak

için reklam için harcanan parayı minimize etmek isteyebilirsiniz. Yani, aşağıdaki ifadeyi minimize etmek isteyebilirsiniz

$$x_1 + x_2 + x_3 + x_4 . \quad (29.4)$$

Her ne kadar negatif reklamlarla politik kampanyalarda sık sık karşılaşılsada, negatif reklam gideri gibi birşey yoktur. Sırasıyla aşağıdakilere gereksinim duyulur

$$x_1 \geq 0, x_2 \geq 0, x_3 \geq 0, \text{ ve } x_4 \geq 0 . \quad (29.5)$$

(29.1)–(29.3) ve (29.5) eşitsizlikleri (29.4) minimize etmek amacı ile birleştirildiğinde “doğrusal program” olarak bilinen ifadeyi elde ederiz. Bu problem aşağıdaki şekilde ifade edilir:

$$\text{Minimize} \quad x_1 + x_2 + x_3 + x_4 \quad (29.6)$$

Kısıtlar

$$-2x_1 + 8x_2 + 0x_3 + 10x_4 \geq 50 \quad (29.7)$$

$$5x_1 + 2x_2 + 0x_3 + 0x_4 \geq 100 \quad (29.8)$$

$$3x_1 - 5x_2 + 10x_3 - 2x_4 \geq 25 \quad (29.9)$$

$$x_1, x_2, x_3, x_4 \geq 0 . \quad (29.10)$$

Bu doğrusal program çözümü en iyi stratejiyi verir.

Genel doğrusal programlar

Genel bir doğrusal programlama probleminde doğrusal eşitsizlikler dizisine göre doğrusal bir fonksiyon optimize etmek istenir. Verilen $a_1, a_2, \dots, a_n$ gerçel sayılar ve $x_1, x_2, \dots, x_n$ değişkenler dizisi ile bu değişkenler üzerinde bir *doğrusal fonksiyonu (linear function)* aşağıdaki şekilde tanımlanır:

$$f(x_1, x_2, \dots, x_n) = a_1x_1 + a_2x_2 + \dots + a_nx_n = \sum_{j=1}^n a_jx_j .$$

Eğer b gerçel sayı ve f doğrusal fonksiyon ise aşağıdaki denklem

$$f(x_1, x_2, \dots, x_n) = b$$

bir *doğrusal eşitlik* ve aşağıdaki eşitsizlikler

$$f(x_1, x_2, \dots, x_n) \leq b$$

ve

$$f(x_1, x_2, \dots, x_n) \geq b$$

doğrusal eşitsizliklerdir. **Doğrusal kısıtlar** genel terimini, doğrusal eşitlik veya doğrusal eşitsizlikleri ifade etmede kullanırız. Doğrusal programlamada katı eşitsizliklere izin verilmez. Formal olarak bir **doğrusal programlama problemi**, sonlu sayıda doğrusal kısıtlara göre doğrusal fonksiyonun minimize etmesi veya maksimize etmesi problemidir. Eğer minimizasyon yapılıyorsa doğrusal program bir **minimizasyon doğrusal programı**, eğer maksimizasyon yapılıyorsa **maksimizasyon doğrusal programı** olarak adlandırılır.

Bu bölümün kalanı doğrusal programların çözümü ve formülüzasyonunu içerir. Her ne kadar doğrusal programlar için çeşitli polinomiyal zamanlı algoritmalar geliştirilsede bu bölümde onları çalışmayacağız. Onun yerine en eski doğrusal programlama algoritması olan simpleks algoritmaları çalışmalıyız. Simpleks algoritması en kötü durumda polinomiyal zamanda sonlanmaz ancak oldukça verimlidir ve uygulamada yaygın biçimde kullanılır.

Doğrusal programlamaya genel bakış

Doğrusal programların özellikleri ve bu programlar için tasarlanan algoritmaları tanımlamak için bu programları doğal biçimde ifade etmenin daha elverişli olacağını düşündük. Bu bölümde iki form kullanmalıyız; **standart** ve **gevşek** biçimler. Kısım 29.1 de bu biçimleri eksiksiz olarak tanımlayacağız. Gayri resmi olarak, standart biçimdeki doğrusal bir program doğrusal **eşitsizliklere** bağlı bir doğrusal fonksiyonun maksimizasyon iken, gevşek biçimdeki doğrusal bir program doğrusal eşitliklere bağlı bir doğrusal fonksiyonun maksimizasyonudur. Genel anlamda doğrusal programları ifade etmek için standart biçimleri kullanmalıyız fakat simpleks algoritmaların detaylarını açıklarken gevşek biçimi kullanmak daha elverişli olacaktır. Şimdi, dikkatimizi m adet doğrusal eşitsizliğe bağlı n değişken üzerinde tanımlı bir doğrusal fonksiyonun maksimizasyonuna verelim.

Öncelikle 2 değişkenli aşağıdaki doğrusal programı ele alalım:

$$\text{Maksimize } x_1 + x_2 \quad (29.11)$$

Kısıtlar

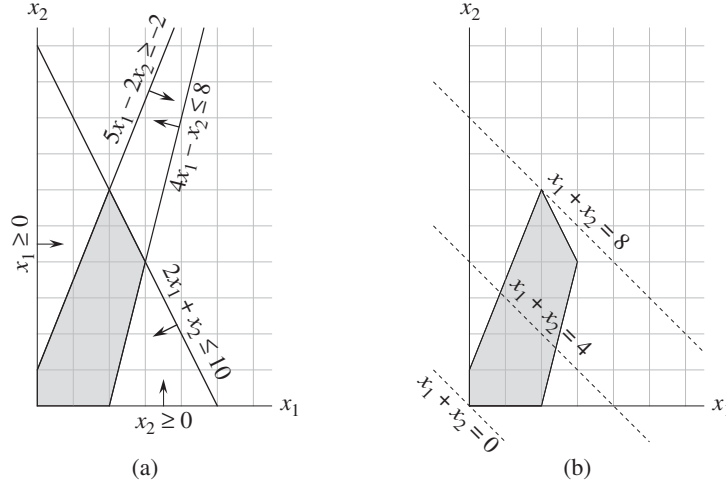
$$4x_1 - x_2 \leq 8 \quad (29.12)$$

$$2x_1 + x_2 \leq 10 \quad (29.13)$$

$$5x_1 - 2x_2 \geq -2 \quad (29.14)$$

$$x_1, x_2 \geq 0. \quad (29.15)$$

(29.12)-(29.15) aralığındaki tüm kısıtları sağlayan x_1, x_2 değişkenler setini doğrusal programa **elverişli çözüm** olarak adlandırırız. Kısıtları (x_1, x_2) kartezyen koordinat sisteminde çizersek, Şekil 29.2(a) da olduğu gibi, görüyoruz ki elve-



Şekil 29.2 (a) (29.12)-(29.15) ile verilen doğrusal program. Her bir kısıt bir doğru ve bir yön ile temsil edilir. Elverişli bölgeyi veren kısıtlar kesişimi taranmıştır. (b) Noktalı doğrular sırasıyla 0,4 ve 8 amaç değerli noktalarıdır. Doğrusal programa en iyi çözüm 8 amaç değeri ile $x_1=2$ ve $x_2=6$ 'dır.

rişli çözümler kümesi (şekilde taranan alan) iki boyutlu uzayda dışbükey bölge¹ oluşturur. Bu dışbükey bölge **elverişli bölge** ve maksimize etmeye çalıştığımız fonksiyon **amaç fonksiyonu** olarak adlandırılır. Kavramsal olarak $x_1 + x_2$ amaç fonksiyonunu elverişli bölgede her noktada hesaplayabiliriz; amaç fonksiyonunun belli bir noktadaki değerini **amaç değeri** olarak adlandırırız. Sonrasında maksimum amaç değerini alan noktayı optimal çözüm olarak tanımlayabiliriz. Bu örnek (ve çoğu doğrusal programlar) için elverişli bölge sonsuz sayıda nokta içerir, bu nedenle elverişli bölgede yer alan her noktada amaç fonksiyonunu hesaplamaya gerek olmadan maksimum amaç değerini veren noktayı bulmak için daha elverişli bir yol tanımlamaya ihtiyacımız vardır.

İki boyutlu durumlarda grafiksel bir prosedür ile optimize edebiliriz. $x_1 + x_2 = z$, her z için noktalar kümesi -1 eğimli bir doğrudur. $x_1 + x_2 = 0$ ı çizersek, merkezden geçen -1 eğimli, Şekil 29.2(b) de olduğu gibi, bir doğru elde ederiz. Elverişli bölge ve bu doğrunun kesişimi 0 amaç değerine sahip elverişli çözümler kümesidir. Bu durumda, doğrunun elverişli bölge ile kesişimi sadece $(0,0)$ noktasıdır. Daha genel olarak, her z için $x_1 + x_2 = z$ doğrusu ve elverişli bölgenin

¹Dışbükey bölgenin sezgisel tanımında bölgedeki herhangi iki nokta için, aralarında çizilen doğru üzerindeki tüm noktalar bölge içerisinde olma gerekliliğini sağlamalıdır.

kesişimi z amaç değerine sahip elverişli çözümler kümesidir. Şekil 29.2(b) $x_1 + x_2 = 0$, $x_1 + x_2 = 4$ ve $x_1 + x_2 = 8$ doğrularını göstermektedir. Şekil 29.2'deki elverişli bölge sınırlı olduğu için $x_1 + x_2 = z$ doğrusu ve elverişli bölge kesişimlerinin boş küme olmadığı bazı maksimum z değerleri olmalıdır. Bu durumun olduğu herhangi bir nokta doğrusal programa optimal çözümdür, bu durumda nokta 8 amaç değerini veren $x_1 = 2$ ve $x_2 = 6$ dır.

Doğrusal programın optimal çözümünün elverişli bölgenin bir tepesinde bulunması bir hata değildir. $x_1 + x_2 = z$ doğrusu ve elverişli bölgenin kesiştiği maksimum z değeri, elverişli bölgenin sınırında yer almalıdır ve bu nedenle bu doğrunun elverişli bölge sınırı ile kesişimi ya tek bir tepe ya da bir doğru parçasıdır. Eğer kesişim tek bir k tepe ise sadece bir en iyi çözüm vardır ve bu da o tepedir. Eğer kesişim bir doğru parçası ise doğru parçası üzerindeki her bir nokta aynı amaç değerine sahip olmalıdır; özellikle, doğru parçasının her iki uç noktasıda optimal çözümlerdir. Doğru parçasının her iki uç noktasıda bir tepe olduğu için yine bu durumda da optimal çözüm bir tepede yer almaktadır.

Herne kadar ikiden fazla değişkene sahip doğrusal programları kolayca çizemesekte, aynı sezgi geçerlidir. Eğer üç değişkenimiz varsa, her bir kısıt üçboyutlu uzayda bir yarı-uzaya denk gelir. Bu yarı-uzayların kesişimi elverişli bölgeyi oluşturur. Amaç fonksiyonunun verilen z değerini elde ettiği noktalar kümesi şimdi bir düzlemdir (dejenere koşulların olmadığı varsayılır). Amaç fonksiyonunun tüm katsayıları pozitif ise ve merkez doğrusal programa elverişli bir çözüm ise bu düzlemi merkezden hareket ettirdiğimizde, amaç fonksiyonunun normal yönünde, amaç değerini arttıran noktalar buluruz. (Eğer merkez elverişli değil veya amaç fonksiyonundaki bazı katsayılar negatif ise sezgisel resim biraz daha karmaşılaşır.) İki boyutluda olduğu gibi, elverişli bölge dışbükey olduğu için, en iyi amaç değerini elde eden noktalar kümesi elverişli bölgenin bir tepesini içermelidir. Benzer olarak eğer n değişkenimiz varsa, her bir kısıt n boyutlu uzayda bir yarı-uzayı temsil eder. Bu yarı-uzayların kesişimi ile şekillenen elverişli bölgeyi **simpleks** ile adlandırırız. Amaç fonksiyonu şimdi bir hiperdüzlemdir, dışbükeylik sebebiyle, bir optimal çözüm hala simpleksin bir tepesinde yer alır.

Simpleks algoritması girdi olarak doğrusal programı alır ve çıktı olarak optimal çözümü verir. Simpleksin bazı tepelerinden başlar ve iterasyonlar dizisi ile devam eder. Her bir iterasyonda belli bir köşeden, amaç değeri bu tepeden daha küçük olmayacak şekilde (ve genellikle daha büyüktür), komşu bir tepeye simpleksin bir ayrıtı boyunca hareket ettirilir. Simpleks algoritma yerel en büyüğe ulaştığı zaman sona erer, öyle ki bu bir tepedir ve diğer tüm komşu tepeler daha küçük amaç değerine sahiptir. Elverişli bölge dışbükey ve amaç fonksiyonu doğrusal olduğu için bu yerel optimum, çoğunlukla genel optimumdur. Kısım 29.4'te simpleks algoritma ile bulunan çözümün gerçekten optimal olduğunu göstermek için “dualite(duality)” olarak adlandırılan kavramı kullanmalıyız.

Her ne kadar geometrik gösterim simpleks algoritma işlemlerinin iyi sezgisel bir biçimde anlayışını sağlasada, Kısım 29.3 te simpleks algoritma detaylarını geliştirirken tamamıyla ona bağlı kalmamalıyız. Onun yerine cebirsel bakışı kullanmalıyız. Öncelikle verilen doğrusal programı doğrusal eşitlikler kümesi olan gevşek biçimde yazalım. Bu doğrusal değişkenler “temel değişkenler” olarak adlandırılan bazı değişkenleri “temel olmayan değişkenler” cinsinden ifade eder. Bir tepeden diğer tepeye, temel değişkeni temel olmayan, temel olmayan değişkeni ise temel yaparak, hareket ettiriyoruz. Bu işleme “pivot” adı verilir, cebirsel bakış açısıyla, aslında doğrusal programı, eşdeğer gevşek biçimde tekrar yazmaktan ibarettir.

Yukarıda tanımlanan iki değişkenli örnek basittir. Bu bölümde daha çeşitli detaylara değinmeliyiz. Bu durumlar çözümü olmayan, sonlu optimal çözüme sahip olmayan ve merkezi (orijin) elverişli çözüm olmayan, doğrusal programları tanımlamayı içerir.

Doğrusal programlama uygulamaları

Doğrusal programlama çok sayıda uygulamaya sahiptir. Bir işlem araştırmalarını konu alan herhangi bir kitap doğrusal programlama örnekleriyle doludur ve çoğu işletme okulunda doğrusal programlama öğrencilere öğretilen standart bir araç olmuştur. Seçim senaryosu tipik bir örnektir. Doğrusal programlamanın başka iki örneği aşağıda verilmiştir:

- Bir havaalanı uçuş personeli için takvim belirlemek istiyor. Federal Havacılık Yönetimi, herbir uçuş görevlisinin ard arda gelen saatlerde çalışmasına sınır koyma, ve belli uçuş görevlilerinin herbir ay süresince sadece bir model uçakta çalışma gerekliliği gibi, çeşitli kısıtlar dayatmaktadır. Havaalanı mümkün olduğunca en az sayıda çalışan kullanarak herbir uçuş için görevli takvimi belirlemek istiyor.
- Bir petrol şirketi yağ için nerede sondajlama yapacağına karar vermek istiyor. Belli bir yere sondaj konumlandırmasının belirlenmiş bir maliyeti vardır ve jeolojik araştırmalara ve birkaç petrol varilinden umulan ödemeye bağlıdır. Yeni sondajları yerleştirmek için şirketin sınırlı bir bütçesi vardır ve bu sınırlı bütçeyle en yüksek miktarda petrol bulmayı istemektedir.

Doğrusal programlar ile, bu kitaptada bulabileceğiniz gibi, çizimleri ve kombinatorik problemleri de modelleyip çözümleyebiliriz. Kısım 24.4 te, fark kısıtları sistem çözümünde kullanılan doğrusal programlamanın, özel bir durumunu görmüştük. Kısım 29.2 de çeşitli çizelge ve ağ akış problemlerini nasıl doğrusal programlar şeklinde formüle edeceğimizi çalışmalıyız. Kısım 35.4 te başka bir çizelge problemine yaklaşık bir çözüm bulmak için araç olarak doğrusal programlamayı kullanmalıyız.

Doğrusal programlama için algoritmalar

Bu bölüm simpleks algoritmayı çalışır. Bu algoritma, dikkatlice uygulandığında, uygulamada genel doğrusal programları kolaylıkla çözer. Bununla birlikte, dikkatlice bulunmuş girdiler ile simpleks algoritma üstel zamana gereksinim duyabilir. Doğrusal programlama için ilk polinomiyal zamanlı algoritma **elipsoid algoritması**dır, uygulamada çok yavaş çalışır. Polinomiyal zamanlı algoritmaların ikinci bir sınıfı **iç-nokta yöntemi** olarak bilinen algoritmalarlardır. Elverişli bölgenin dışı boyunca hareket eden ve her bir adımda simpleksin bir tepesi olan elverişli çözüm ile devam eden simpleks algoritmalarına karşıt olarak bu algoritmalar elverişli bölgenin içi boyuca hareket eder. Algoritma süresince elde edilen çözümler, elverişli iken, simpleksin tepeleri olmak zorunda değildir, ancak sonuçta elde edilen çözüm bir tepedir. Çok sayıda girdi için iç nokta algoritmaları simpleks kadar hızlı, bazen daha hızlı, çalışabilir. Bölüm notları bu algoritmalar ile ilgili seni daha fazla bilgiye götürür.

Eğer doğrusal programa ek bir gereklilik olarak tüm değişkenlerin tam sayı değer alma zorunluluğunu getirirsek, bir **tam sayılı doğrusal program** elde ederiz. Alıştırma 34.5-3 bu probleme elverişli bir çözüm bulmanın NP- zor bir problem, olduğunu göstermeni istiyor; çünkü herhangi NP- zor bir problem için hiçbir polinomiyal zamanlı algoritma bilinmiyor, tamsayılı doğrusal programlama için polinomiyal zamanlı bir algoritma yoktur. Karşıt olarak, genel doğrusal programlama problemini polinomiyal zamanda çözebiliriz.

Bu bölümde, eğer $x = (x_1, x_2, \dots, x_n)$ değişkenleri ile tanımlı bir doğrusal programımız var ise ve değişkenlerin özel bir setinden söz etmek istiyorsak, $\bar{x} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ gösterimini kullanmalıyız.

29.1 Standart ve gevşek biçimler

Bu bölümde doğrusal programları belirlerken ve çalışırken kullanışlı olacak standart ve gevşek biçimler tanımlanacaktır. Standart biçimde, tüm kısıtlar eşitsizlikler iken gevşek biçimde tüm kısıtlar eşitliklerdir (değişkenlerin negatif olmama gereksinimi dışında).

Standart biçim

Standart biçimde, n adet $c_1, c_2, \dots, c_n$ gerçel sayıları; m adet $b_1, b_2, \dots, b_m$ gerçel sayıları; ve mn adet $i = 1, 2, \dots, m$ ve $j = 1, 2, \dots, n$ için a_{ij} gerçel sayıları verilmiş olsun. n adet $x_1, x_2, \dots, x_n$ gerçel sayıları bulunmaya çalışılır öyle ki:

$$\text{Maksimize} \quad \sum_{j=1}^n c_j x_j \quad (29.16)$$

Kısıtlar

$$\sum_{j=1}^n a_{ij} x_j \leq b_i \quad i = 1, 2, \dots, m \quad (29.17)$$

$$x_j \geq 0 \quad j = 1, 2, \dots, n. \quad (29.18)$$

İki değişkenli doğrusal program için tanımladığımız terminolojiyi genel-leştirerek, (29.16) ifadesini **amaç fonksiyonu** ve (29.17) ile (29.18) de yeralan $n + m$ adet eşitsizliği **kısıtlar** olarak adlandırırız. (29.18) satırındaki n adet kısıt **negatif olmama kısıtlarıdır**. Keyfi bir doğrusal programın negatif olmama kısıt-larına sahip olması gerekmez ancak standart biçimde olmalıdır. Bazen doğrusal bir programı daha kompakt formda ifade etmeyi elverişli buluruz. Eğer $A = (a_{ij})$ $m \times n$ matris, $b = (b_j)$ m -vektör, $c = (c_j)$ n -vektör ve $x = (x_j)$ n -vektörü yarattıy-sak, (29.16)-(29.18) de tanımlanan doğrusal programı aşağıdaki şekilde yeniden yazabiliriz:

$$\text{Maksimize} \quad c^T x \quad (29.19)$$

Kısıtlar

$$Ax \leq b \quad (29.20)$$

$$x \geq 0. \quad (29.21)$$

(29.19) satırında yer alan $c^T x$ iki vektörün iç çarpımıdır. (29.20) eşitsizliğinde, Ax matris vektör çarpımı, (29.21) eşitsizliğinde yer alan $x \geq 0$ ise her bir x vektör girdisinin negatif olmaması gerektiğini belirtir. Görüyoruz ki standart biçimdeki bir doğrusal programı (A, b, c) üçlüsü ile ifade edebiliriz, ve A , b ve c nin yukarı-da belirtilen boyutlarda olma kuralını kabul etmeliyiz.

Şimdi doğrusal programlara çözümleri tanımlayan terminolojiyi tanıtacağız. Bu terminolojinin bir kısmını önceki iki değişkenli doğrusal programlama örne-ğinde kullandık. Tüm kısıtları sağlayan belli bir $\bar{x}$ değişkenler dizisini **elverişli çözüm** olarak adlandırırız öyleki en az bir kısıtı sağlamayan $\bar{x}$ değişkenler dizisi ise **elverişsiz çözüm (infeasible solution)**dür. Bir $\bar{x}$ çözümü $c^T \bar{x}$ amaç değerine sahiptir. Amaç değeri tüm elverişli çözümlerin en büyüğü olan elverişli çözüm **optimal çözüm (optimal solution)** dür ve amaç değeri $c^T \bar{x}$ **optimal amaç değeri (optimal objective value)**dir. Eğer bir doğrusal programın hiçbir elverişli çözümü yoksa **mümkün olmayan (infeasible)**; varsa **elverişli (feasible)** doğrusal program olarak adlandırılır. Eğer bir doğrusal programın bazı elverişli çözümleri var an-cak sonlu optimal amaç değeri yok ise bu doğrusal program **sınırsız (unbounded)** bir doğrusal programdır. Alıştırma 29.1-9 da doğrusal programın elverişli bölgesi sınırsız olduğu durumda bile sonlu optimal amaç değerine sahip olabileceğini göstermeniz istenmiştir.

Doğrusal programları standart forma dönüştürme

Doğrusal kısıtlara bağlı bir doğrusal fonksiyonu minimize veya maksimize bir doğrusal programı standart biçime dönüştürmek her zaman için mümkündür. Bir doğrusal program aşağıda belirtilen 4 sebepten herhangi biri nedeniyle standart formda olmayabilir:

1. Amaç fonksiyonu maksimizasyon yerine minimizasyon olabilir.
2. Negatif olmama kısıtlarına sahip olmayan değişkenler olabilir.
3. Küçük eşit yerine eşitlik işaretine sahip *eşitlik kısıtları* olabilir.
4. Küçük eşit yerine büyük eşit işaretine sahip *eşitsizlik kısıtları* olabilir.

Bir L doğrusal programını başka bir L' doğrusal programına dönüştürürken L' elverişli çözümünün L ninde elverişli çözümü olması beklenir. Bu düşünceyle, eğer L nin her bir, z amaç değerli $\bar{x}$ elverişli çözümü için benzer, L' nin de z amaç değerli bir $\bar{x}'$ elverişli çözümü var ise; ve L' nin her bir z amaç değerli $\bar{x}'$ elverişli çözümü için benzer z amaç değerli $\bar{x}$ elverişli çözümü var ise L ve L' maksimizasyon doğrusal programlarına *denktir* denir. (Bu tanım elverişli çözümler arasında bire birlik kuralına tekabül etmez.) Eğer L' nin her bir, z amaç değerli $\bar{x}$ elverişli çözümü için benzer, L' nin $-z$ amaç değerli bir $\bar{x}'$ elverişli çözümü var ise; ve L' nin her bir z amaç değerli $\bar{x}'$ elverişli çözümü için benzer $-z$ amaç değerli $\bar{x}$ elverişli çözümü var ise L minimizasyon doğrusal programı ile L' maksimizasyon doğrusal programı denktir.

Şimdi yukarıdaki listede yer alan her bir mümkün problemi, bir bir, nasıl kaldıracağımızı göstereceğiz. Her birini kaldırdıktan sonra yeni doğrusal programın eski doğrusal programa denk olup olmadığını tartışmalıyız.

Bir L minimizasyon doğrusal programını denk bir L' maksimizasyon doğrusal programına dönüştürmek için basitçe amaç fonksiyonundaki katsayıların işaretlerini değiştiririz. L ve L' benzer elverişli çözüm dizilerine sahip olduğu için ve her bir elverişli çözüm için L deki amaç değeri L' amaç değerinin negatifi olduğu için bu iki doğrusal program birbirine denktir. Örneğin aşağıdaki doğrusal programı ele alalım:

$$\begin{array}{ll} \text{Minimize} & -2x_1 + 3x_2 \\ \text{Kısıtlar} & \\ & x_1 + x_2 = 7 \\ & x_1 - 2x_2 \leq 4 \\ & x_1 \geq 0 \end{array}$$

ve amaç fonksiyon katsayılarının işaretlerini değiştirelim.

$$\text{Maksimize} \quad 2x_1 - 3x_2$$

Kısıtlar

$$\begin{aligned} x_1 + x_2 &= 7 \\ x_1 - 2x_2 &\leq 4 \\ x_1 &\geq 0. \end{aligned}$$

Sonrasında, negatif olmama kısıtı olmayan değişkenlere sahip doğrusal programı, her bir değişkeni negatif olmama kısıtına sahip doğrusal programa nasıl dönüştüreceğimizi göstereceğiz. Varsayalım ki bazı x_j değişkenleri negatif olmama kısıtına sahip olmasın. Böyle bir durumda, her bir x_j , $x'_j - x''_j$ ile yer değiştirilir ve $x'_j \geq 0$ ve $x''_j \geq 0$ negatif olmama kısıtları eklenir. Bu nedenle, eğer amaç fonksiyonu $c_j x_j$ terimine sahip ise $c_j x'_j - c_j x''_j$ ile yer değiştirilir ve eğer i kısıtı $a_{ij} x_j$ terimine sahipse $a_{ij} x'_j - a_{ij} x''_j$ ile yer değiştirilir. Doğrusal programın herhangi bir $\hat{x}$ elverişli çözümü $\bar{x}_j = \hat{x}'_j - \hat{x}''_j$ ve aynı amaç değeri ile orijinal doğrusal programın $\bar{x}$ elverişli çözümüne denk gelir. Ayrıca orijinal doğrusal programın $\bar{x}$ elverişli çözümü eğer $\bar{x}_j \geq 0$ ise $\hat{x}'_j = \bar{x}_j$, $\hat{x}''_j = 0$ ile ve eğer $\bar{x}_j < 0$ ise $\hat{x}'_j = -\bar{x}_j$, $\hat{x}''_j = 0$ ile yeni doğrusal programın $\hat{x}$ elverişli çözümüne denk gelir. İki doğrusal program $\bar{x}_j$ nin işaretine bakmaksızın aynı amaç değerine sahiptir. Bu nedenle iki doğrusal program birbirine denktir. Bu dönüşüm şemasını, tüm değişkenleri negatif olmama kısıtına sahip denk bir doğrusal programa ulaşmak için negatif olmama kısıtına sahip olmayan her bir değişkene uygularız.

Örneğe devam ederek, her bir değişkenin negatif olmama kısıtına sahip olmasını sağlayalım. x_1 değişkeni böyle bir kısıta sahip iken x_2 değişkeni sahip değildir. Bu nedenle x_2 değişkeni x'_2 ve x''_2 ile yer değiştirilir ve doğrusal program değiştirilerek aşağıdaki biçim elde edilir

$$\text{En büyük} \quad 2x_1 - 3x'_2 + 3x''_2$$

Kısıtlar

$$\begin{aligned} x_1 + x'_2 - x''_2 &= 7 \\ x_1 - 2x'_2 + 2x''_2 &\leq 4 \\ x_1, x'_2, x''_2 &\geq 0. \end{aligned} \tag{29.22}$$

Sonrasında, eşitlik kısıtlarını eşitsizlik kısıtlarına dönüştürürüz. Varsayalım ki bir doğrusal program $f(x_1, x_2, \dots, x_n) = b$ eşitlik kısıtına sahip olsun. $x = y$ ancak ve ancak $x \leq y$ ve $x \geq y$ olduğu için bu eşitlik kısıtını $f(x_1, x_2, \dots, x_n) \leq b$ ve $f(x_1, x_2, \dots, x_n) \geq b$ iki eşitsizlik kısıtları ile yer değiştirebiliriz. Bu dönüşümü her bir eşitlik kısıtına uygulayarak her bir kısıtı eşitsizlik olan doğrusal programa ulaşıyoruz.

Son olarak, büyük-veya-eşit kısıtlarını küçük-veya-eşit yapmak için bu kısıtları -1 ile çarpıyoruz. Öyle ki her bir

$$\sum_{j=1}^n a_{ij}x_j \geq b_i$$

biçimde ifade edilen eşitsizlik

$$\sum_{j=1}^n -a_{ij}x_j \leq -b_i$$

ile denktir. Böylece her bir a_{ij} katsayısını $-a_{ij}$ ve her bir b_i değerini $-b_i$ ile değiştirerek denk bir küçük eşit kısıtına sahip oluruz.

Örneği bitirmek üzere, (29.22) kısıtındaki eşitliği iki eşitsizlik ile değiştirerek aşağıdaki formu elde ederiz

$$\begin{array}{l} \text{En büyükle} \quad 2x_1 \quad - \quad 3x_2' \quad + \quad 3x_2'' \\ \text{Kısıtlar} \quad \begin{array}{rclcl} x_1 & + & x_2' & - & x_2'' & \leq & 7 \\ x_1 & + & x_2' & - & x_2'' & \geq & 7 \\ x_1 & - & 2x_2' & + & 2x_2'' & \leq & 4 \\ x_1, x_2', x_2'' & & & & & \geq & 0 \end{array} \end{array} \quad (29.23)$$

Son olarak, (29.23) kısıtını negtaifiyle değiştiririz. Değişken isimlerindeki süreklilik için x_2', x_2 ile x_2'' ise x_3 ile değiştirilir ve aşağıdaki standart biçim elde edilir

$$\text{Maksimize} \quad 2x_1 \quad - \quad 3x_2 \quad + \quad 3x_3 \quad (29.24)$$

$$\text{Kısıtlar} \quad \begin{array}{rclcl} x_1 & + & x_2 & - & x_3 & \leq & 7 \end{array} \quad (29.25)$$

$$-x_1 \quad - \quad x_2 \quad + \quad x_3 \leq -7 \quad (29.26)$$

$$x_1 \quad - \quad 2x_2 \quad + \quad 2x_3 \leq 4 \quad (29.27)$$

$$x_1, x_2, x_3 \geq 0 \quad (29.28)$$

Doğrusal programları gevşek biçime dönüştürme

Verimlilik için bir doğrusal programı simpleks algoritma ile çözerken, doğrusal programı kısıtların bir kısmı eşitlik kısıtı olan biçimde ifade etmeyi tercih ederiz. Daha detaylı olarak, onu öyle bir forma dönüştürmeliyiz ki tüm eşitsizlik kısıtları negatif olmama kısıtları olurken kalan kısıtlar eşitlik kısıtları olsun.

$$\sum_{j=1}^n a_{ij}x_j \leq b_i \quad (29.29)$$

bir eşitsizlik kısıtı olsun. Yeni bir s değişkeni tanımlayalım ve (29.29) eşitsizliğini aşağıdaki iki kısıt ile yeniden yazalım

$$s = b_i - \sum_{j=1}^n a_{ij}x_j, \quad (29.30)$$

$$s \geq 0. \quad (29.31)$$

“ s ” i **gevşek değişken** olarak tanımlayalım çünkü (29.29) denkleminin sağ taraf ve sol tarafı arası farklılığı ve **gevşemeyi** ölçer (İleride kısıtı neden sadece sol taraftaki gevşek değişken ile ifade etmenin gerekli olduğunu görmeliyiz). (29.29) eşitsizliği ancak ve ancak her iki (29.30) eşitliği ve (29.31) eşitsizliği doğru olduğunda sağlanıyor olması sebebiyle, negatif olmama kısıtları sadece eşitsizlik kısıtları olan denk bir doğrusal program elde etmek için, doğrusal programda yer alan her bir eşitsizlik kısıtını bu yolla dönüştürebiliriz. Standarttan gevşek biçime dönüşüm yaparken i . eşitsizlikle bağlantılı gevşek değişkeni belirtmek için (s yerine) x_{n+i} kullanmalıyız. Böylece i . kısıt $x_{n+i} \geq 0$ negatif olmama şartı ile aşağıdaki biçime ifade edilir

$$x_{n+i} = b_i - \sum_{j=1}^n a_{ij}x_j \quad (29.32)$$

Standart biçimdeki doğrusal programın her bir kısıtını dönüştürerek farklı bir biçime doğrusal program elde ederiz. Örneğin (29.24)-(29.28) de tanımlanan doğrusal program için x_4 , x_5 ve x_6 gevşek değişkenlerini tanıtarak aşağıdaki biçimi elde ederiz

$$\text{En büyükle} \quad 2x_1 - 3x_2 + 3x_3 \quad (29.33)$$

Kısıtlar

$$x_4 = 7 - x_1 - x_2 + x_3 \quad (29.34)$$

$$x_5 = -7 + x_1 + x_2 - x_3 \quad (29.35)$$

$$x_6 = 4 - x_1 + 2x_2 - 2x_3 \quad (29.36)$$

$$x_1, x_2, x_3, x_4, x_5, x_6 \geq 0. \quad (29.37)$$

Bu doğrusal program içinde negatif olmama kısıtları dışındaki tüm kısıtlar eşitliktir ve her bir değişken negatif olmama kısıtına sahiptir. Her bir eşitlik kısıtını eşitliğin sol tarafında bir değişken ve sağ tarafında diğer tüm değişkenler olacak şekilde yazalım. Dahası, her bir denklemde sağ tarafta aynı değişkenler yer alır ve bu değişkenler sadece amaç fonksiyonunda yer alan değişkenlerdir. Eşitliğin sol tarafında yer alan değişkenleri **temel değişkenler**, sağ taraftakileri ise **temel olmayan** değişkenler olarak adlandırırız.

Bu koşulları sağlayan doğrusal programlar için bazen “en maksimizasyon” ve “kısıtlar” kelimelerini çıkartabiliriz aynı şekilde negatif olmama kısıtları da aşikardır. Ayrıca amaç fonksiyon değerini belirtirken z değişkenini kullanabiliriz.

Sonuçta elde edilen yapı **gevşek biçim** olarak adlandırılır. Eğer (29.33)-(29.37) verilen doğrusal programı gevşek biçimde yazarsak aşağıdaki ifadeyi elde ederiz

$$z = 2x_1 - 3x_2 + 3x_3 \quad (29.38)$$

$$x_4 = 7 - x_1 - x_2 + x_3 \quad (29.39)$$

$$x_5 = -7 + x_1 + x_2 - x_3 \quad (29.40)$$

$$x_6 = 4 - x_1 + 2x_2 - 2x_3. \quad (29.41)$$

Standart biçimde olduğu gibi, gevşek biçimi tanımlamak için de daha kısa bir gösterim kullanmak daha elverişlidir. Kısım 29.3 te görüldüğü üzere, temel ve temel olmayan değişkenler simpleks algoritma süresince değişecektir. Temel olmayan değişken indisleri kümesi N ile temel değişken indisler kümesi ise B ile belirtilir. Her zaman için $|N| = n - |B| = m$, $N \cup B = \{1, 2, \dots, n + m\}$ olur. Denklemler B girdileri ile sağ taraftaki değişkenler ise N girdileri ile indislenir. Standart biçimde olduğu gibi sabit terimleri ve katsayıları belirtmek için b_i , c_j ve a_{ij} kullanılır. Ayrıca amaç fonksiyonundaki tercihe bağlı sabit terimleri belirtmek için de v kullanılır (sonra, amaç fonksiyonunda sabit terim kullanımının amaç fonksiyon değerini belirlemede kolaylık sağladığı görülmelidir). Böylece gevşek biçim kısaca (N, B, A, b, c, v) değişkenler grubu ile tanımlanabilir, tanımlanan gevşek biçimde

$$z = v + \sum_{j \in N} c_j x_j \quad (29.42)$$

$$x_i = b_i - \sum_{j \in N} a_{ij} x_j \quad i \in B \text{ için} \quad (29.43)$$

tüm x değişkenleri negatif olmama zorunluluğu ile kısıtlanmıştır. Gerçekten 29.43 teki $\sum_{j \in N} a_{ij} x_j$ toplamını çıkardığımız için a_{ij} değerleri gevşek biçimde görüldüğü gibi katsayıların negatiftir.

Örneğin, aşağıdaki gevşek biçimde

$$\begin{aligned} z &= 28 - \frac{x_3}{6} - \frac{x_5}{6} - \frac{2x_6}{3} \\ x_1 &= 8 + \frac{x_3}{6} + \frac{x_5}{6} - \frac{x_6}{3} \\ x_2 &= 4 - \frac{8x_3}{3} - \frac{2x_5}{3} + \frac{x_6}{3} \\ x_4 &= 18 - \frac{x_3}{2} + \frac{x_5}{2}, \end{aligned}$$

$B = \{1, 2, 4\}$ $N = \{3, 5, 6\}$ dır.

Denklem,

$$A = \begin{pmatrix} a_{13} & a_{15} & a_{16} \\ a_{23} & a_{25} & a_{26} \\ a_{43} & a_{45} & a_{46} \end{pmatrix} = \begin{pmatrix} -1/6 & -1/6 & 1/3 \\ 8/3 & 2/3 & -1/3 \\ 1/2 & -1/2 & 0 \end{pmatrix},$$

$$b = \begin{pmatrix} b_1 \\ b_2 \\ b_4 \end{pmatrix} = \begin{pmatrix} 8 \\ 4 \\ 18 \end{pmatrix},$$

$c = (c_3 \ c_5 \ c_6)^T = (-1/6 \ -1/6 \ -2/3)^T$ ve $v=28$ dir. A , b ve c indislerinin bitişik tam sayılar kümesi olmasına gerek olmadığına dikkat edilmelidir; bu indisler B ve N indis kümelerine bağlıdır. A girdilerinin gevşek biçim görüntüsünde katsayıların negatifi oluşuna örnek vermek gerekirse, x_1 için denklem $x_3/6$ terimini içerir, lakin a_{13} katsayısı gerçekten $+1/6$ değil $-1/6$ dır.

Alıştırmalar

29.1-1

Eğer (29.24)–(29.28) da yer alan doğrusal programı, (29.19)–(29.21) kısa gösterimi ile ifade edersek, n , m , A , b ve c nedir?

29.1-2

(29.24)–(29.28) deki doğrusal program için üç adet elverişli çözüm veriniz. Herbirinin amaç değeri nedir?

29.1-3

(29.38)–(29.41) gevşek biçimi için N , B , A , b , c ve v nedir?

29.1-4

Aşağıdaki doğrusal programı standart biçime dönüştürünüz:

$$\text{Maksimize } 2x_1 + 7x_2 + x_3$$

Kısıtlar

$$\begin{array}{rclcl} x_1 & & - & x_3 & = & 7 \\ 3x_1 & + & x_2 & & \geq & 24 \\ & x_2 & & & \geq & 0 \\ & & & x_3 & \leq & 0 . \end{array}$$

29.1-5

Aşağıdaki doğrusal programı gevşek biçime dönüştürünüz:

$$\begin{array}{ll}
 \text{Maksimize} & 2x_1 \quad \quad \quad - \quad 6x_3 \\
 \text{Kısıtlar} & \\
 & x_1 + x_2 - x_3 \leq 7 \\
 & 3x_1 - x_2 \geq 8 \\
 & -x_1 + 2x_2 + 2x_3 \geq 0 \\
 & x_1, x_2, x_3 \geq 0 .
 \end{array}$$

Temel ve temel olmayan değişkenler nelerdir?

29.1-6

Aşağıdaki doğrusal programın elverişsiz olduğunu gösteriniz:

$$\begin{array}{ll}
 \text{Maksimize} & 3x_1 - 2x_2 \\
 \text{Kısıtlar.} & \\
 & x_1 + x_2 \leq 2 \\
 & -2x_1 - 2x_2 \leq -10 \\
 & x_1, x_2 \geq 0 .
 \end{array}$$

29.1-7

Aşağıdaki doğrusal programın sınırsız olduğunu gösteriniz:

$$\begin{array}{ll}
 \text{Maksimize} & x_1 - x_2 \\
 \text{Kısıtlar} & \\
 & -2x_1 + x_2 \leq -1 \\
 & -x_1 - 2x_2 \leq -2 \\
 & x_1, x_2 \geq 0 .
 \end{array}$$

29.1-8

n değişken ve m kısıt ile genel bir doğrusal programımız olduğunu ve gevşek biçime dönüştürdüğümüzü varsayalım. Son elde edilen doğrusal programda değişken ve kısıt sayılarının üst sınırını belirtiniz.

29.1-9

Elverişli bölgenin sınırlı olmadığı ancak optimal amaç değerinin sonlu olduğu bir doğrusal program örneği veriniz.

29.2 Problemleri doğrusal programlar olarak formüle etme

Bu bölümde simpleks algoritma üzerinde odaklanmamız gereksede, problemi bir doğrusal program gibi formüle etme işlemini hatırlamamız da çok önemlidir. Bir kere problemi polinomiyal zamanlı bir doğrusal program gibi ifade edebilirsek, elipsoid algoritması veya iç nokta yöntemleriyle polinomiyal zamanda çözebiliriz. Çeşitli doğrusal programlama yazılım paketleri ile bu problemler verimli bir şekilde çözülebilirler, bu nedenle problem bir doğrusal program biçiminde olduğunda, böyle bir paket bu problemi çözebilir.

Çeşitli somut doğrusal programlama örneklerine bakmalıyız. Önceden çalıştığımız iki problem ile başlayalım: tek-kaynaklı en kısa yol problemi (bkz. Bölüm 24) ve maksimum-akış problemi (bkz. Bölüm 26). Sonrasında minimum-maliyet akışı problemini tanımlayacağız. Her ne kadar minimum-maliyet akışı problemi polinomiyal zamanlı doğrusal programlama temelli olmayan bir algoritmaya sahip olsa da, bu algoritmayı tanımlamayacağız. Son olarakta tek bilinen polinomiyal zamanlı algoritması doğrusal programlama temelli olan çoklu madde akış problemi tanımlanacaktır.

Bölüm VI da çizge problemlerini çözerken $v.d$ ve $(u, v).f$ gibi özellik gösterimlerini kullandık. Doğrusal programlar tipik olarak bağlı özellikli nesneler yerine altsimgeli değişkenler kullanır. Bu nedenle doğrusal programda değişkenler ifade edilirken altsimgeler ile tepe ve ayrıtları belirtmeliyiz. Örneğin, v tepesi için en-kısa yol ağırlığını $v.d$ ile değil d_v ile gösteririz. Benzer olarak u tepesinden v tepesine akışı $(u, v).f$ ile değil f_{uv} ile belirtiriz. Problem girdisi şeklinde verilen miktarlar için ayrıt ağırlıkları veya kapasiteler gibi, $w(u, v)$ ve $c(u, v)$ gibi gösterimler kullanılmaya devam edilir.

En kısa yollar

Tek-kaynaklı en kısa yollar problemini doğrusal program gibi formüle edebiliriz. Bu bölümde, Alıştırma 29.2-3 örneğindeki gibi daha genel tek-kaynaklı en kısa yol problemi genişletmelerini bırakarak, tek-ikili en kısa yol problemini nasıl formüle edeceğimize odaklanmalıyız.

Tek-ikili en kısa yol probleminde, hedef tepesi t , kaynak tepesi s , ayrıtları gerçek değerli ağırlıklara izdüşüren $w : E \rightarrow \mathbb{R}$ ağırlık fonksiyonu ile ağırlıklı, yönlü $G = (V, E)$ çizgesi verilir. s den t ye en kısa yol ağırlığını belirten d_t değerini hesaplamak isteriz. Bu problemi doğrusal program gibi ifade etmek için s den t ye en kısa yolu tanımlayan değişkenler ve kısıtlar kümesini belirtmeye ihtiyacımız vardır. Neyse ki, Bellman-Ford algoritması tam olarak bunu yapar. Bellman-Ford algoritması durduğunda, her bir v tepesi için bir d_v değeri (özellik gösterimi yerine altsimga gösterimi kullanarak) öyle ki her bir ayrıt $(u, v) \in E$ için $d_v \leq d_u + w(u, v)$ elde edilir.

Başlangıçta, kaynak tepe hiçbir zaman değişmeyen $d_s=0$ değerini alır. Böylece s den t ye en kısa yol ağırlığını hesaplamak için aşağıdaki doğrusal programı elde ederiz:

$$\text{Maksimize } d_t \quad (29.44)$$

$$\text{Kısıtlar } d_v \leq d_u + w(u, v) \quad (u, v) \in E \text{ ayrıtı için,} \quad (29.45)$$

$$d_s = 0. \quad (29.46)$$

En kısa yol hesaplanmak istenirken, bu doğrusal programın amaç fonksiyonunu maksimize edilmesi şaşırtıcı gözükmemektedir. Amaç fonksiyonunu minimize etmek istemiyoruz çünkü her $v \in V$ için $\bar{d}_v = 0$ en kısa yol problemini çözmeksizin doğrusal programa optimal çözümünü vermektedir. Maksimize ediyoruz çünkü en kısa yol problemine optimal çözüm, $\bar{d}_v$ yi $\min_{u:(u,v) \in E} \{\bar{d}_u + w(u, v)\}$ e götürür, böylece $\bar{d}_v$, $\{\bar{d}_u + w(u, v)\}$ kümesindeki tüm elemanlara küçük veya eşit olan maksimum değerini alır. s den t ye en kısa yol üzerinde tüm v tepeleri için d_v yi maksimize etmek isityoruz tüm v tepelerinde bu kısıtlar altında d_t yi maksimize etme bu amacı başarır.

Bu doğrusal program her bir $v \in V$ için $|V|$ adet d_v değişkenine sahiptir. Ayrıca her bir ayrıtı için kaynak tepenin en kısa yol ağırlığının her zaman 0 olması kısıtı ile $|E| + 1$ kısıta sahiptir.

Maksimum akış (maximum flow)

Sonrasında maksimum akış problemini bir doğrusal program gibi ifade edeceğiz. Tekrardan yönlü bir $G = (V, E)$ çizgesi verildiğini varsayalım, burada her bir $(u, v) \in E$ ayrıtı $c(u, v) \geq 0$, negatif olmayan kapasiteye ve s kaynak ve t çıkış düğümü olmak üzere iki farklı tepeye sahiptir. Kısım 26.1 de tanımlandığı gibi bir akış, kapasite kısıtı ve akış korunumunu sağlayan $f : V \times V \rightarrow \mathbb{R}$ negatif olmayan gerçek değerli bir fonksiyondur. Maksimum akış bu kısıtları sağlayan ve kaynaktan çıkan toplam akış eksi kaynağa gelen toplam akış değerini maksimize eden akıştır. Böylece akış doğrusal kısıtları sağlar ve akış değeri doğrusal bir fonksiyondur. Ayrıca tekrardan, eğer $(u, v) \notin E$ ise $c(u, v) = 0$ ve birbirine paralel olmayan ayrıtarın olmadığını varsayalım. Maksimum akış problemini aşağıdaki doğrusal program gibi ifade edebiliriz:

$$\text{Maksimize } \sum_{v \in V} f_{sv} - \sum_{v \in V} f_{vs} \quad (29.47)$$

Kısıtlar

$$f_{uv} \leq c(u, v) \quad \text{her } u, v \in V \quad \text{için,} \quad (29.48)$$

$$\sum_{v \in V} f_{vu} = \sum_{v \in V} f_{uv} \quad \text{her } u \in V - \{s, t\} \quad \text{için,} \quad (29.49)$$

$$f_{uv} \geq 0 \quad \text{her } u, v \in V \quad \text{için.} \quad (29.50)$$

Bu doğrusal program her bir köşe ikilisi arasındaki akışa karşılık gelen $|V|^2$ değişkene ve $2|V|^2 + |V| - 2$ sayıda kısıta sahiptir.

Genelde küçük boyutlu doğrusal programları çözmek daha verimlidir. (29.47)-(29.50) deki doğrusal program, gösterim kolaylığı için $(u, v) \notin E$ için her bir u, v köşeler ikilisi için kapasitenin 0 olduğu bir akışa sahiptir. Doğrusal programı tekrardan yazmak daha verimli olacaktır, öyle ki $O(V + E)$ kısıta sahiptir. Alıştırma 29.2-5 te böyle yapmanız istenmiştir.

Minimum-maliyet akışı (minimum-cost flow)

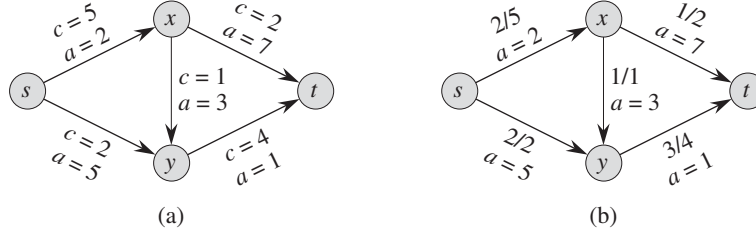
Bu bölümde, bildiğimiz algoritmalarla verimli şekilde çözülebilen problemleri çözmek için doğrusal programlamayı kullandık. Gerçekten, özellikle bir problem için verimli bir algoritma tasarlanmış olabilir, örneğin tek-kaynaklı en kısa yol problemi için Dijkstra algoritması, veya maksimum akış için itme-yeniden etiketleme yöntemi, her iki teori ve uygulamada da doğrusal programlamadan daha verimli sonuçlar verir.

Doğrusal programlamanın gerçek gücü yeni problemleri çözme yeteneğinden gelmektedir. Bu bölümün başında gördüğümüz siyasi problemi tekrardan ele alalım. Fazla para harcamadan yeterli sayıda oy toplayabilme problemi bu kitapta şu ana kadar çalıştığımız algoritmalarından herhangi biriyle çözülemez, ama doğrusal programlama ile çözülebilir. Kitaplarda, doğrusal programlama ile çözülebilen bu gibi gerçek hayat problemlerine çok rastlanır. Doğrusal programlama aynı zamanda çözümü için verimli bir algoritması olmayan çeşitli problemler için çok kullanışlıdır.

Örneğin aşağıdaki maksimum akış probleminin genelleştirmesini ele alalım. Varsayalım ki, her bir (u, v) ayrıtı için $c(u, v)$ kapasitesine ek olarak gerçel $a(u, v)$ maliyet değeri verilsin. Maksimum akış probleminde olduğu gibi eğer $(u, v) \notin E$ ise $c(u, v) = 0$ ve paralel olmayan ayrıtların olmadığı varsayılsın. Eğer (u, v) ayrıtı üzerinden f_{uv} birimlerini gönderirsek $a(u, v)f_{uv}$ maliyeti ortaya çıkar. Ayrıca d akış talebi verilmiştir. Akış ile ortaya çıkan $\sum_{(u,v) \in E} a(u, v)f_{uv}$ toplam maliyetini minimize ederek s den t ye akışta d birimin gönderimi istenmektedir. Bu problem **minimum-maliyet-akışı problemi** olarak bilinir.

Şekil 29.3(a) da minimum-maliyet-akışı problemine bir örnek gösterilmiştir. Minimum toplam maliyet ile s den t ye 4 birim akışı istenmektedir. Herhangi bir özel yasal akış, yani (29.48)-(29.49) kısıtlarını sağlayan f fonksiyonu, $\sum_{(u,v) \in E} a(u, v)f_{uv}$ toplam maliyetini ortaya çıkartır. Bu maliyeti minimize edenn özel 4 birimlik bir akış istenmektedir. Şekil 29.3(b), $\sum_{(u,v) \in E} a(u, v)f_{uv} = (2 \cdot 2) + (5 \cdot 2) + (3 \cdot 1) + (7 \cdot 1) + (1 \cdot 3) = 27$ toplam maliyet ile en iyi çözümü belirtir.

Özellikle minimum maliyet akışı problemleri için hazırlanmış polinomiyal zamanlı algoritmalar vardır, ancak bu kitabın kapsamına alınmamıştır. Bununla birlikte minimum maliyet akışı problemi bir doğrusal program gibi ifade edilebilir.



Şekil 29.3 (a) Minimum maliyet akışı problemi örneği. Kapasiteler c , maliyetler a ile gösterilmiştir. s tepesi kaynak t tepesi ise çıkış düğümüdür ve s den t ye 4 birimlik akış gönderimi istenmektedir. (b) s den t ye 4 birimlik akışın gönderildiği minimum maliyet akışı probleminin çözümü. Her bir ayrıntı için akış ve kapasiteler akış/kapasite ile yazılmıştır.

Doğrusal program, akış değerinin tam d birim olduğu ek kısıtı ve yeni minimize eden amaç fonksiyonu ile bir maksimum akış problemine benzerdir:

$$\text{Minimize} \quad \sum_{(u,v) \in E} a(u,v) f_{uv} \quad (29.51)$$

Kısıtlar

$$\begin{aligned} f_{uv} &\leq c(u,v) && \text{her } u, v \in V \text{ için,} \\ \sum_{v \in V} f_{vu} - \sum_{v \in V} f_{uv} &= 0 && \text{her } u, v \in V - \{s, t\} \text{ için,} \\ \sum_{v \in V} f_{sv} - \sum_{v \in V} f_{vs} &= d, \\ f_{uv} &\geq 0 && \text{her } u, v \in V \text{ için.} \end{aligned} \quad (29.52)$$

Çoklu ürün akışı

Son bir örnek olarak, başka bir akış problemi ele alınmıştır. Varsayalım ki Kısım 26.1 den Lucky Puck şirketi ürün yelpazesi ve gönderimini sadece hokey topu ile kısıtlamayıp hokey çubukları ve kaskları ile genişletmek istesin. Her bir ürünün parçası kendi fabrikalarında üretilmektedir, kendi depoları vardır ve bu parçalar hergün depodan fabrikaya nakledilmelidir. Çubuklar Vancouver'da üretilmektedir ve Saskatoon'a gönderilmelidir. Kasklar ise Edmonton'da üretilmektedir ve Regina'ya gönderilmelidir. Gönderim ağ kapasiteleri değişmez bununla birlikte farklı parçalar ya da **ürünler (commodities)** aynı ağı paylaşmalıdır.

Bu örnek bir **çoklu ürün-akışı problemi (multicommodity-flow problem)** örneğidir. Bu problemde tekrardan her bir $(u, v) \in V$ ayrıtı $c(u, v) \geq 0$ negatif olmayan kapasiteli yönlü $G = (V, E)$ çizgesi verilmiştir. Maksimum akış probleminde olduğu gibi tam olarak $(u, v) \notin E$ için $c(u, v) = 0$ ve çizgenin paralel olmayan

ayrıtlarının olmadığı varsayılmıştır. Ek olarak her bir i ürününün $K_i = (s_i, t_i, d_i)$ üçlüsü ile belirtildiği k farklı $K_1, K_2, \dots, K_k$ ürünleri verilmiştir. Burada s_i tepesi i ürününün kaynağı, t_i tepesi i ürününün çıkış düğümü yeri ve d_i s_i den t_i ye istenilen ürün akış değeri yani ürününün talep miktarıdır. i ürünü için akış f_i (öyle ki f_{iuv} u dan v ye i ürününün akışıdır) ile tanımlanır ve f_i kapasite kısıtlarını ve akış korunumunu sağlayan gerçel değerli bir fonksiyondur. Şimdi f_{uv} , **toparlama akış**, çeşitli ürün akışlarının toplamı, $f_{uv} = \sum_{i=1}^k f_{iuv}$ tanımlanacaktır. (u, v) ayrıtı üzerindeki agrega akışı (u, v) ayrıtı kapasitesinden fazla olamaz. Bu problemde bir amaç fonksiyonu minimize etmeye çalışılmaz; böyle bir akışın var olduğu gösterilmelidir. Bu nedenle 0 amaç fonksiyonu ile bir doğrusal program yazılmıştır:

Minimize

$$0$$

kısıtlar

$$\begin{aligned} \sum_{i=1}^k f_{iuv} &\leq c(u, v) && \text{her } u, v \in V \text{ için} \\ \sum_{v \in V} f_{iuv} - \sum_{v \in V} f_{ivu} &= 0 && \text{her } i = 1, 2, \dots, k \text{ ve} \\ &&& \text{her } u \in V - \{s_i, t_i\} \text{ için} \\ \sum_{v \in V} f_{i, s_i, v} - \sum_{v \in V} f_{i, v, s_i} &= d_i && \text{her } i = 1, 2, \dots, k \text{ için} \\ f_{iuv} &\geq 0 && \text{her } u, v \in V \text{ ve} \\ &&& \text{her } i = 1, 2, \dots, k \text{ için.} \end{aligned}$$

Bu problem için tek bilinen polinomiyal zamanlı algoritmada problem doğrusal program şeklinde ifade edilir ve sonrasında polinomiyal zamanlı doğrusal programlama algoritması ile çözülür.

Alıştırmalar

29.2-1

(29.44)-(29.46) daki tek-ikili en kısa yol doğrusal programını standart formda ifade ediniz.

29.2-2

Şekil 24.2(a) da gösterilen s düğümünden y düğümüne en kısa yolu bulmayı sağlayan doğrusal programı açık bir şekilde yazınız.

29.2-3

Tek-kaynaklı en kısa yol probleminde, s tepesinden tüm $v \in V$ tepelerine en kısa yol ağırlıkları bulunmak isteniyor. Verilen bir G çizgesi ile, bir doğrusal program yazınız öyle ki çözümde, d_v , s den her bir $v \in V$ tepesine en kısa yolu belirtsin.

29.2-4

26.1(a) şeklindeki maksimum akış problemini ifade eden doğrusal programı açık şekilde yazınız.

29.2-5

(29.47)-(29.50) maksimum akış doğrusal programını tekrardan yazınız öyle ki sadece $O(V + E)$ kısıtlarını kullansın.

29.2-6

Öyle bir doğrusal program yazınız ki, verilen iki parçalı $G = (V, E)$ çizgesi ile maksimum iki parçalı eşleme problemini çözsün.

29.2-7

Minimum-maliyet çoklu ürün-akışı probleminde her bir $(u, v) \in E$ ayrıtı $c(u, v) \geq 0$ negatif olmayan kapasite ve $a(u, v)$ maliyetine sahip bir $G = (V, E)$ yönlü çizgesi verilsin. Çoklu ürün akışı probleminde olduğu gibi i ürününün $K_i = (s_i, t_i, d_i)$ üçlüsü ile belirtildiği k farklı $K_1, K_2, \dots, K_k$ ürünü verilsin. Çoklu ürün akış probleminde olduğu gibi i ürünü için f_i akışı ve (u, v) ayrıtı üzerinde f_{uv} toparlama akışı tanımlanmıştır. Elverişli akış öyle bir akıştır ki, her bir (u, v) ayrıtı üzerindeki bütün akış (u, v) ayrıt kapasitesinden fazla değildir. Akış maliyeti $\sum_{u,v \in V} a(u, v) f_{uv}$ ve amaç minimum maliyetli elverişli akışı bulmaktır. Bu problemi doğrusal program şeklinde ifade ediniz.

29.3 Simpleks algoritma

Simpleks algoritma doğrusal programları çözmek için klasik bir yöntemdir. Bu kitaptaki diğer tüm algoritmalara karşıt olarak en kötü durumda çalışma zamanı polinomial değildir. Bu doğrusal programları çalışmaya yöneliktir ancak uygulamada simpleks algoritma önemli derecede hızlıdır.

Önceden bu bölümde tanımlanan geometrik gösterime ek olarak, simpleks algoritma Kısım 28.1 de çalışılan Gauss yok etme, yöntemi ile benzerlikler taşımaktadır. Gauss yok etme, çözümü bilinmeyen doğrusal eşitlikler sistemi ile başlar. Her bir iterasyonda bu sistem ek yapılara sahip denk biçimde tekrardan yazılır. Birkaç iterasyondan sonra sistemi tekrardan yazarız öyle ki çözümü elde etmek kolaydır. Simpleks algoritma da benzer şekilde süre gelir ve eşitsizlikler için gauss yok etme gibi incelenebilir.

Şimdi simpleks algoritmanın bir iterasyonundaki temel düşünce tanımlanacaktır. Herbir iterasyon ile ilişkili bir temel çözüm olacaktır öyle ki doğrusal programın gevşek biçimiyle kolayca elde edilebilir: her bir temel olmayan değişkeni 0 olarak tanımlar ve eşitlik kısıtlarındaki temel değişken değerlerini hesaplar. Bir iterasyon bir gevşek biçimi denk başka bir gevşek biçime dönüştürür. İlgili temel değişken çözümünün amaç değeri önceki iterasyondakilerden küçük olmayacak ve çoğunlukla büyük olacaktır. Amaç değerindeki bu artışı sağlamak için bir temel olmayan değişken seçilir öyle ki, eğer bu değişkenin değeri 0 dan arttırılabiliyorsa amaç değeri de birlikte artış gösterir. Değişkendeki artım miktarı diğer kısıtlar ile sınırlandırılmıştır. Özellikle bazı temel değişken değerleri 0 olana kadar arttırılır. Sonrasında gevşek biçim, temel değişken ve seçilen temel olmayan değişken rolleri yer değiştirilerek tekrardan yazılır. Algoritmaya yol göstermek için belli değişken kümelerinin kullanılması ve bunları kanıtlarda kullanmamız gerekse de, algoritma açık olarak bu çözümü sürdürmez. Doğrusal programı, optimal çözümü “açık” olana dek basitçe tekrardan yazar.

Simpleks algoritma örneği

Genişletilmiş bir örnek ile başlayalım. Aşağıdaki standart formulu doğrusal programı ele alalım:

$$\text{Maksimize } 3x_1 + x_2 + 2x_3 \quad (29.53)$$

kısıtlar

$$x_1 + x_2 + 3x_3 \leq 30 \quad (29.54)$$

$$2x_1 + 2x_2 + 5x_3 \leq 24 \quad (29.55)$$

$$4x_1 + x_2 + 2x_3 \leq 36 \quad (29.56)$$

$$x_1, x_2, x_3 \geq 0 . \quad (29.57)$$

Simpleks algoritmayı kullanmak için doğrusal program gevşek biçime dönüştürülmelidir: Kısım 29.1 de nasıl olacağı açıklanmıştır. Cebirsel manipülasyona ek olarak, gevşeklik kullanışlı algoritmik bir kavramdır. Kısım 29.1 e geri dönersek, herbir değişken ilişkili bir negatif olmama kısıtına sahiptir. Eğer kısıtların temel değişken değerlerinin 0 olmasına sebep oluyorsa, bir eşitlik kısıtı, ilişkili temel olmayan değişkenler kümesi için *sıkı* dır. Benzer olarak, temel değişken değerini negatif yapan temel olmayan değişkenler kümesi bu kısıtı *ihlal eder*. Bu nedenle, gevşek değişkenler açık bir şekilde herbir kısıtın sıkılıktan uzaklığını kalıcı hale getirir ve böylelikle hiçbir kısıtı ihlal etmeden temel olmayan değişken değerlerinin ne kadar arttırılması gerektiğinin belirlenmesine yardımcı olurlar.

x_4, x_5, x_6 gevşek değişkenlerini sırasıyla (29.54)-(29.56) eşitsizliklerinde kullanılarak ve gevşek biçimdeki doğrusal programa yerleştirilerek aşağıdaki ifadeleri elde ederiz.

$$z = 3x_1 + x_2 + 2x_3 \quad (29.58)$$

$$x_4 = 30 - x_1 - x_2 - 3x_3 \quad (29.59)$$

$$x_5 = 24 - 2x_1 - 2x_2 - 5x_3 \quad (29.60)$$

$$x_6 = 36 - 4x_1 - x_2 - 2x_3 . \quad (29.61)$$

(29.59)-(29.61) kısıtlar sistemi 3 denklem ve 6 değişkenden oluşur. Herhangi x_p , x_s , x_3 değişkenler kümesi x_p , x_s , x_6 değerlerini tanımlar; bu nedenle bu denklemler sisteminin sonsuz sayıda çözümü vardır. Eğer tüm x_p , x_s , $\dots$, x_6 negatif değil ise çözüm elverişlidir ve böylece sonsuz sayıda elverişli çözüm elde edilebilir. Bu gibi sistemlere sonsuz sayıda mümkün çözümler sonraki kanıtlarda kullanışlı olacaktır. **Temel çözüme** odaklanalım: sağ taraftaki tüm temel olmayan değişkenleri 0 yapalım ve sonrasında sol taraftaki temel değişkenleri hesaplayalım. Bu örnekte temel çözüm $(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_6) = (0, 0, 0, 30, 24, 36)$ dır ve amaç değeri $z = (3 \cdot 0) + (1 \cdot 0) + (2 \cdot 0) = 0$ dır. Herbir $i \in B$ için $\bar{x}_i = b_i$ temel çözümleri incelenir. Simpleks algoritmanın bir iterasyonu sağ tarafa farklı değişkenleri koymak için denklemler kümesini ve amaç fonksiyonunu tekrardan yazar. Böylece tekrardan yazılan probleme ilişkin farklı temel çözümler elde edilir. Tekrardan yazım hiçbir şekilde ana doğrusal programlama problemini değiştirmez; bir iterasyondaki problem bir önceki iterasyondaki problemle aynı elverişli çözümler kümesine sahiptir. Bununla birlikte problem, bir önceki iterasyondakinden farklı bir temel çözüme sahiptir.

Eğer temel çözüm aynı zamanda uygunsa, **temel elverişli çözüm** olarak adlandırılır. Simpleks algoritma çalıştırıldığında temel çözüm her zaman temel elverişli çözümdür. Kısım 29.5 e bakıldığında bununla birlikte simpleks algoritmanın ilk birkaç iterasyonu için temel çözüm elverişli olmayabilir.

Amacımız, herbir iterasyonda, temel çözüm değeri daha büyük amaç değeri alacak şekilde doğrusal programı tekrar formüle etmektir. Amaç fonksiyonundaki katsayısı pozitif olan x_e temel olmayan değişken seçilir, ve kısıtları ihlal etmeden x_e değeri artırılır. x_e değişkeni temel yapılır ve diğer x_l değişkenlerinin bazıları temel olmayan yapılır. Diğer temel değişkenlerin ve amaç fonksiyonunun değerleri değişmeyebilir.

Örneğe devam etmek için x_l in değerini arttırmaya çalışalım. x_l i arttırdığımız sürece x_p , x_s , x_6 değerleri düşer. Çünkü herbir değişken için negatif olmama kısıtı vardır, herhangi birinin negatif olmasına izin verilmez. Eğer x_l 30 un üzerine çıkarsa x_4 negatif olur ve x_l 12 ve 9 un üzerine çıkarsa sırasıyla x_5 ve x_6 negatif olur. Üçüncü (29.61) kısıtı en sıkı kısıttır ve x_l i ne kadar arttırmamız gerektiğinin sınırlandırır. Bu nedenle x_l ve x_6 nin rolleri değiştirilir. x_l için (29.61) denklemi çözülür ve aşağıdaki eşitlik elde edilir:

$$x_l = 9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4} \quad (29.62)$$

Sağ taraftaki x_6 ile diğer denklemleri tekrardan yazmak için (29.62) denklemini kullanarak x_1 yerine yazılır. (29.59) için de yapılarak aşağıdaki ifade elde edilir:

$$\begin{aligned} x_4 &= 30 - x_1 - x_2 - 3x_3 \\ &= 30 - \left(9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4}\right) - x_2 - 3x_3 \\ &= 21 - \frac{3x_2}{4} - \frac{5x_3}{2} + \frac{x_6}{4}. \end{aligned} \quad (29.63)$$

Benzer olarak doğrusal programı aşağıdaki biçimde tekrardan yazmak için (29.62) denklemini (29.60) kısıtı ve (29.58) amaç fonksiyonu ile kombinlenir:

$$z = 27 + \frac{x_2}{4} + \frac{x_3}{2} - \frac{3x_6}{4} \quad (29.64)$$

$$x_1 = 9 - \frac{x_2}{4} - \frac{x_3}{2} - \frac{x_6}{4} \quad (29.65)$$

$$x_4 = 21 - \frac{3x_2}{4} - \frac{5x_3}{2} + \frac{x_6}{4} \quad (29.66)$$

$$x_5 = 6 - \frac{3x_2}{2} - 4x_3 + \frac{x_6}{2}. \quad (29.67)$$

Bu işlem **pivot** olarak adlandırılır. Yukarıda gösterildiği gibi bir pivot, temel olmayan bir x_i **giren değişken** olarak adlandırılır, ve temel değişken x_l **çıkan değişken** olarak adlandırılır, seçer ve rollerini değiştirir.

(29.64)–(29.67) denklemleri ile tanımlanan doğrusal program (29.58)–(29.61) denklemleri ile tanımlı doğrusal programa denktir. Simpleks algoritmada iki işlem uygulanır: değişkenler sol taraf ve sağ tarafta yer değiştirecek şekilde denklemleri tekrardan yazma ve bir denklemin yerine diğerini yazma. İlk işlemde önemsiz bir şekilde denk bir problem yaratılır, ve ikincide basit doğrusal cebir ile denk bir problem yaratılır (Alıştırma 29.3-3 e bkz).

Bu denkliği göstermek için orijinal temel çözümün (0, 0, 0, 30, 24, 36) (29.65)–(29.67) yeni denklemleri sağladığını ve amaç değerinin $27 + (1/4) \cdot 0 + (1/2) \cdot 0 - (3/4) \cdot 36 = 0$ olduğunu görelim. Yeni doğrusal programın temel çözümü (9, 0, 0, 21, 6, 0) dır ve temel olmayan değerler 0 ve $z = 27$ olur. Basit aritmetik gösteriyor ki bu çözüm aynı zamanda (29.59)–(29.61) denklemlerini sağlar ve (29.58) amaç fonksiyonuna uygulandığında amaç değeri $(3 \cdot 9) + (1 \cdot 0) + (2 \cdot 0) = 27$ olur.

Örneğe devam edecek olursak, değerini arttırmak istediğimiz yeni bir değişken bulunur. x_6 arttırılmak istenmez çünkü x_6 arttığında amaç değeri düşer. x_2 veya x_3 ü arttırmayı deneyebiliriz; x_3 ü seçtiğimizi varsayalım. Hiçbir kısıtı ihlal etmeden x_3 ü ne kadar arttırabiliriz? Kısıt (29.65) 18 e kadar, kısıt (29.66) 42/5 e kadar ve kısıt (29.67) 3/2 ye kadar limit koyar. Üçüncü kısıt yine en sıkı olanıdır bu nedenle x_3 sol tarafta x_5 sağ tarafta olacak şekilde üçüncü kısıt tekrardan

yazılır. Sonrasında bu yeni denklem, $x_3 = 3/2 - 3x_2/8 - x_5/4 + x_6/8$ (29.64)-(29.66) denklemleri yerine yazılır ve yeni denk aşağıdaki sistem elde edilir:

$$z = \frac{111}{4} + \frac{x_2}{16} - \frac{x_5}{8} - \frac{11x_6}{16} \quad (29.68)$$

$$x_1 = \frac{33}{4} - \frac{x_2}{16} + \frac{x_5}{8} - \frac{5x_6}{16} \quad (29.69)$$

$$x_3 = \frac{3}{2} - \frac{3x_2}{8} - \frac{x_5}{4} + \frac{x_6}{8} \quad (29.70)$$

$$x_4 = \frac{69}{4} + \frac{3x_2}{16} + \frac{5x_5}{8} - \frac{x_6}{16} . \quad (29.71)$$

Bu sistemin temel çözümü $(33/4, 0, 3/2, 69/4, 0, 0)$ amaç değeri ise $111/4$ tür. Şimdi amaç değerini arttırmanın tek yolu x_2 yi arttırmaktır. Üç kısıt sırasıyla 132,4 ve sonsuz üst sınırlarını verir ((29.71). Kısıtından sonsuz üst sınırı elde edilir çünkü x_2 arttığı sürece x_4 temel değişkeni de artar. Bu kısıt bu nedenle x_2 artışına hiçbir sınır getirmez). x_2 4 e arttırılır ve temel olmayan değişken yapılır. Sonrasında x_2 için (29.70) denklemi çözülür ve diğer denklemler yerine yazılarak aşağıdaki denklemler elde edilir:

$$z = 28 - \frac{x_3}{6} - \frac{x_5}{6} - \frac{2x_6}{3} \quad (29.72)$$

$$x_1 = 8 + \frac{x_3}{6} + \frac{x_5}{6} - \frac{x_6}{3} \quad (29.73)$$

$$x_2 = 4 - \frac{8x_3}{3} - \frac{2x_5}{3} + \frac{x_6}{3} \quad (29.74)$$

$$x_4 = 18 - \frac{x_3}{2} + \frac{x_5}{2} . \quad (29.75)$$

Bu noktada, amaç fonksiyonundaki tüm katsayılar negatiftir. Bu bölümün sonrasında da görüleceği gibi bu durum sadece doğrusal programı tekrardan yazdığımızda ve böylece temel çözüm optimal çözüm olduğunda ortaya çıkar. Bu nedenle bu problem için çözüm $(8, 4, 0, 18, 0, 0)$ ve amaç değeri 28 optimaldir. Şimdi (29.53)–(29.57) de verilen orijinal doğrusal programa dönebiliriz. Orijinal doğrusal programdaki değişkenler x_1, x_2 ve x_3 tür bu nedenle çözümümüz $x_1 = 8, x_2 = 4$ ve $x_3 = 0$ ile amaç değeri $(3 \cdot 8) + (1 \cdot 4) + (2 \cdot 0) = 28$ dir. Son çözümdeki gevşek değişken değerleri her bir eşitsizlikte ne kadar gevşek kalacağını ölçer. x_4 gevşek değişkeni 18 dir ve (29.54) eşitsizliğinde sol taraf $8 + 4 + 0 = 12$ değeri ile 30 değerli sağ taraftan 18 azdır. x_5 ve x_6 gevşek değişkenleri 0 dır ve gerçekten (29.55) ve (29.56) eşitsizliklerinde sol taraf ve sağ taraf birbirine eşittir. Orijinal gevşek biçimdeki katsayılar integral olduğu halde diğer doğrusal programlardaki katsayıların integral olması gibi bir gereklilik yoktur ve ortadaki çözümlerin de integral olması gerekmez. Buna ek olarak, doğrusal programın son çözümünün

integral olmasına gerek yoktur; sadece rastlantısal olarak bu örnek çözümü integreldir.

Pivotlama

Şimdi pivotlama süreci biçimlendirilecektir. PIVOT süreci, verilen (N, B, A, b, c, v) üçlüsü, x_l çıkan değişkenin l indisi ve giren değişken x_e nin e indisi ile gevşek biçimi girdi olarak alır. Bu üçlü yeni gevşek biçimi tanımlayan $(\hat{N}, \hat{B}, \hat{A}, \hat{b}, \hat{c}, \hat{v})$ üçlüsüne döner. ($m \times n$ boyutlu A ve $\hat{A}$ matris girdileri gerçekten gevşek biçimde görülen katsayıların negatifleridir).

PIVOT (N, A, B, b, c, v, l, e)

- 1 // Yeni x_e temel değişkeni için denklemin katsayılarını hesapla.
- 2 $\hat{A}$ yeni $m \times n$ lik matris olsun
- 3 $\hat{b}_e = b_l / a_{le}$
- 4 **for** her $j \in N - \{e\}$ için
- 5 $\hat{a}_{ej} = a_{lj} /$
- 6 $\hat{a}_{el} = 1 / a_{le}$
- 7 // Kalan kısıtların katsayılarını hesapla.
- 8 **for** her $i \in B - \{l\}$ için
- 9 $\hat{b}_i = b_i - a_{ie} \hat{b}_e$
- 10 **for** her $j \in N - \{e\}$ için
- 11 $\hat{a}_{ij} = a_{ij} - a_{ie} \hat{a}_{ej}$
- 12 $\hat{a}_{il} = -a_{ie} \hat{a}_{el}$
- 13 // Amaç fonksiyonunu hesapla.
- 14 $\hat{v} = v + c_e \hat{b}_e$
- 15 **for** her $j \in N - \{e\}$ için
- 16 $\hat{c}_j = c_j - c_e \hat{a}_{ej}$
- 17 $\hat{c}_l = -c_e \hat{a}_{el}$
- 18 // Yeni temel ve temel olmayan değişkenler kümelerini hesapla.
- 19 $\hat{N} = N - \{e\} \cup \{l\}$
- 20 $\hat{B} = B - \{l\} \cup \{e\}$
- 21 **return** $(\hat{N}, \hat{B}, \hat{A}, \hat{b}, \hat{c}, \hat{v})$

PIVOT şu şekilde işler. 3-6 satırlarında, sol tarafta x_e yerine x_l olan denklem yeniden yazılarak x_e için yeni denklemdeki katsayılar hesaplanır. 8-12 satırlarında her bir x_e için bu yeni denklemin sağ tarafı yerine koyularak kalan denklemler güncellenir. 14-17 satırlarında aynı yerine koyma işlemi amaç fonksiyonu için

yapılır ve 19 ve 20 satırlarında temel ve temel olmayan değişken kümeleri güncellenir. 21 satırında yeni gevşek biçime dönülür. Verildiği gibi, eğer $a_{le} = 0$ ise PIVOT 0 a bölme sebebiyle hata verebilir, fakat Lemma 29.2 ve 29.12 kanıtlarından görmeliyiz ki, PIVOT sadece $a_{le} \neq 0$ ise çağırılır.

Şimdi PIVOT un temel çözümde değişken değerlerine olan etkisi incelenecektir.

Lemma 29.1

$a_{le} \neq 0$ olan PIVOT (N, B, A, b, c, v, l, e) u ele alalım. Bu işlemten geri dönen değerler $(\hat{N}, \hat{B}, \hat{A}, \hat{b}, \hat{c}, \hat{v})$ olsun ve işlemten sonra temel çözüm $\bar{x}$ ile gösterilsin. Sonrasında

1. $\bar{x}_j = 0$ her $j \in \hat{N}$ için.
2. $\bar{x}_e = b_l / a_{le}$
3. $\bar{x}_i = b_i - a_{ie} \hat{b}_e$ her $i \in \hat{B} - \{e\}$ için.

Kanıt İlk ifade doğrudur çünkü temel çözüm her zaman tüm temel olmayan değişkenleri 0 yapar.

$$x_i = \hat{b}_i - \sum_{j \in \hat{N}} \hat{a}_{ij} x_j ,$$

kısıtında her bir temel olmayan değişkeni 0 yaptığımızda, her $i \in \hat{B}$ için $\bar{x}_i = \hat{b}_i$ olur. $e \in \hat{B}$ olduğu sürece PIVOT un 3 satırı ikinci ifadeyi kanıtlayan

$$\bar{x}_e = \hat{b}_e = b_l / a_{le}$$

elde edilir. Benzer olarak her bir $i \in \hat{B} - \{e\}$ için üçüncü ifadeyi kanıtlayan

$$\bar{x}_i = \hat{b}_i = b_i - a_{ie} \hat{b}_e$$

elde edilir. ■

Formal Simpleks algoritma

Şimdi örnekle gösterdiğimiz simpleks algoritmayı formalize etmeye hazırız. Bu örnek özel ve güzel bir örnekti ve çeşitli başka sorunlarla da karşılaşabilirdik:

- Bir doğrusal programın elverişli olduğunu nasıl belirleriz?
- Eğer doğrusal program elverişli ancak başlangıç temel çözüm elverişli değil ise ne yaparız?
- Bir doğrusal programın sınırsız olduğunu nasıl belirleriz?
- Giren ve çıkan değişkenleri nasıl seçeriz?

Kısım 29.5 de bir problemin elverişli olduğunun nasıl belirleneceği ve eğer elverişliyse başlangıç temel çözümü elverişli olan gevşek biçimin nasıl bulunacağı gösterilmelidir. Bu nedenle $m \times n$ $A = (a_{ij})$ matrisi, m -vektör $b = (b_i)$ ve n -vektör $c = (c_j)$ ile standart biçimdeki doğrusal program girdisini veren INITIALİZE-SIMPLEX (A, b, c) prosedürünün varolduğu düşünülür. Eğer problem elverişli değil ise prosedür programın elverişsiz olduğuna dair mesaj gönderir ve prosedürü sonlandırır. Elverişliyse başlangıç temel çözümü elverişli olan gevşek biçimi verir.

SIMPLEX prosedürü önceden tanımlandığı gibi standart biçimde doğrusal programı girdi olarak alır. (29.19)-(29.21) de tanımlanan doğrusal programa optimal çözüm olan n -vektör $\bar{x} = (\bar{x}_j)$ yi verir.

```

SIMPLEX( $A, b, c$ )
1  ( $N, B, A, b, c, v$ ) = INITIALIZE-SIMPLEX ( $A, b, c$ )
2   $\Delta$ ,  $n$  uzunluklu yeni bir vektör olsun.
3  while bazı  $j \in N$  için  $c_j > 0$ 
4       $c_e > 0$  olan bir  $e \in N$  indisi seç.
5      for her  $i \in N$  indisi için
6          if  $a_{ie} > 0$  ise
7               $\Delta_i = b_i / a_{ie}$ 
8          else  $\Delta_i = \infty$ 
9       $\Delta_i$  yi minimize eden bir  $l \in B$  indisi seç
10     if  $\Delta_l == \infty$  ise
11         return "sınırsız"
12     else ( $N, B, A, b, c, v$ ) = PIVOT( $N, B, A, b, c, v, l, e$ )
13 for  $i=1$  to  $n$ 
14     if  $i \in B$  ise
15          $\bar{x}_i = b_i$ 
16     else  $\bar{x}_i = 0$ 
17 return ( $\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n$ )

```

SIMPLEX süreci şu şekilde işler. 1 satırında yukarıda tanımlanan ve doğrusal programın elverişsiz olduğunu belirten veya temel çözümü elverişli olan gevşek biçimi döndüren INITIALİZE-SIMPLEX (A, b, c) çağırılır. 3-12 satırlarından oluşan **while** döngüsü algoritmanın temel bölümünü oluşturur. Eğer amaç fonksiyonundaki tüm katsayılar negatif ise **while** döngüsü sonlanır. Değilse 4 satırında amaç fonksiyonundaki katsayısı pozitif olan x_e değişkeni giren değişken olarak seçilir. Her ne kadar herhangi bir değişkeni giren değişken olarak seçebilsekte, bazı önceden belirlenmiş rassal olmayan bir kuralı kullandığımız varsayılır. Sonrasında 5-9 satırlarında herbir kısıt kontrol edilir ve herhangi bir negatif olmama kısıtını ihlal etmeden x_e yi artırma miktarını en ağır şekilde sınırlandıran kısıt seçilir:

bu kısıt ile ilişkili temel değişken x_i dir. Tekrardan çıkan değişken olarak çeşitli değişkenlerden bir tanesini serbestçe seçilir ancak önceden belirlenmiş rassal olmayan bir kuralın kullanıldığı varsayılır. Eğer kısıtlardan hiçbiri giren değişken artış miktarına sınır koymuyorsa algoritma 11 satırında “sınırsız” mesajını döndürür. Değilse 12 satırında yukarıda tanımlandığı gibi, $\text{PIVOT}(N, B, A, b, c, v, l, e)$ çağırılarak giren ve çıkan değişken rolleri değiştirilir. 13-16 satırında, tüm temel olmayan değişkenler 0 ve her bir temel değişken $\bar{x}_i$ b_i yapılarak orijinal doğrusal programlama değişkenleri için $\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n$ çözümü hesaplanır ve 17 satırında bu değerler döndürülür.

SIMPLEX in doğru olduğunu göstermek için, öncelikle eğer SIMPLEX bir başlangıç elverişli çözümüne sahipse ve sonunda duruyorsa bir elverişli çözüm döndürdüğü veya doğrusal programın sınırsız olduğunu belirttiği gösterilir. Sonrasında SIMPLEX in sonlandırıldığı gösterilir. En son olarak, Kısım 29.4 (Teorem 29.10) da döndürülen çözümün optimal olduğu gösterilir.

Lemma 29.2

Verilen bir (A, b, c) doğrusal programı olsun. Varsayalım ki SIMPLEX in 1 satırında çağırılan INITIALIZE-SIMPLEX temel çözümü elverişli olan gevşek biçimi döndürsün. Sonrasında eğer 17 satırında bir çözüm döndürülüyorsa bu çözüm doğrusal programın elverişli çözümüdür. Eğer SIMPLEX 11 satırında “sınırsız” mesajı veriyorsa, bu doğrusal program sınırsızdır.

Kanıt Aşağıdaki üç parçalı döngü değişmezi kullanılır:

- 3-12 satırlarından oluşan **while** döngüsünün her bir iterasyon başlangıcında,
1. gevşek biçim INITIALIZE-SIMPLEX ile döndürülen gevşek biçime denktir.
2. her bir $i \in B$ için $b_i \geq 0$ dır ve
3. gevşek biçime ilişkin temel çözüm elverişlidir.

Başlangıç: Gevşek biçimlerin denkliği ilk iterasyon için aşıkardır. Lemmadan, SIMPLEX in 1 satırındaki BAŞLANGIÇ-SIMPLEX çağrısının, temel çözümü elverişli olan gevşek formu döndürdüğü varsayılır. Bu nedenle sabtin üçüncü kısmıda doğrudur. Temel çözüm elverişli olduğu için her bir temel değişken x_i negatif değildir. Ayrıca, temel çözüm her bir x_i temel değişkenini b_i yaptığı sürece, her $i \in B$ için $b_i \geq 0$ olur. Böylece değişmezin ikinci kısmıda sağlanmış olur.

Yineleme: 11 satırındaki **return** ifadesinin uygulanmadığı varsayılarak **while** döngüsünün her bir iterasyonunun, döngü değişmezini koruduğu gösterilmelidir. Sonlandırma durumu tartışılırken 11 satırındaki uygulama durumu

ele alınmalıdır. **While** döngüsünün her bir iterasyonu PIVOT sürecini çağırarak temel ve temel olmayan değişken rollerini değiştirir. Alıştırma 29.3-3 ile, gevşek biçim, başlangıç gevşek biçimine denk olan, döngü değişmezi ile, bir önceki iterasyondakine denktir.

Şimdi döngü değişmezinin ikinci bölümü gösterilecektir. **While** döngüsünün her bir itirasyon başlangıcında her $i \in B$ için $b_i \geq 0$ olduğu varsayalım. Bu eşitsizliklerin 12 satırındaki PIVOT çağırımından sonra da doğru olduğu gösterilmelidir. Sadece b_i değişkenlerine ve temel değişkenler kümesi B ye uygulanan değişiklikler bu süreçte yapılması, 12 satırının değişmezin bu bölümünü koruduğunu göstermek için yeterlidir. b_i , a_{ij} , ve B PIVOT çağırımından önce varolan değerler ve b_i PIVOT tan sonra döndürülen değerler olsun.

Öncelikle, $\hat{b}_e \geq 0$ elde edilir çünkü döngü değişmezi ile $b_i \geq 0$ dır, SIMPLEX in 6 ve 9 satırları ile $a_{ie} > 0$ dır ve PIVOT un 3 satırı ile $\hat{b}_e = b_l / a_{le}$ olur.

Kalan $i \in B - \{l\}$ indisleri için aşağıdaki ifadeler sahip oluruz:

$$\begin{aligned} \hat{b}_i &= b_i - a_{ie} \hat{b}_e && \text{(PIVOT un 9. satırı aracılığıyla)} \\ &= b_i - a_{ie} (b_l / a_{le}) && \text{(PIVOT un 3. satırı aracılığıyla)} \end{aligned} \quad (29.76)$$

Ele alınacak $a_{ie} > 0$ veya $a_{ie} < 0$ a bağlı iki durum vardır. Eğer $a_{ie} > 0$ ise her $i \in B$ için

$$b_l / a_{le} \leq b_i / a_{ie} \quad \text{her } i \in B \text{ için} \quad (29.77)$$

olacak şekilde l seçildiği sürece

$$\begin{aligned} \hat{b}_i &= b_i - a_{ie} (b_l / a_{le}) && \text{(denklem (29.76) dan)} \\ &\geq b_i - a_{ie} (b_i / a_{ie}) && \text{(eşitsizlik (29.77) den)} \\ &= b_i - b_i \\ &= 0, \end{aligned}$$

elde edilir ve bu nedenle $\hat{b}_i \geq 0$ dır. Eğer $a_{ie} \leq 0$ ise, a_{ie} , b_i ve b_l negatif olmadıği için denklem (29.76) gösteriyor ki b_i de negatif olmamalıdır.

Şimdi temel çözümün elverişli olduğu, yani tüm değişkenlerin neaktif olmayan değerler aldığı durum tartışılacaktır. Temel olmayan değişkenler 0 yapılmıştır bu nedenle hiçbir negatif değildir. Her bir x_i temel değişkeni

$$x_i = b_i - \sum_{j \in N} a_{ij} x_j$$

ile tanımlanır. Temel çözüm $\bar{x}_i = b_i$ olur. Döngü değişmezinin ikinci kısmı kullanılarak her bir $\bar{x}_i$ temel değişkeninin negatif olmadığı sonucuna varılır.

Sonlandırma: **While** döngüsü iki farklı yolla sonlanabilir. Eğer 3. satırdaki kısıttan dolayı duruyorsa, o anda var olan temel çözüm elverişlidir ve 17. satırda bu sonuç döndürülür. Sonlandırmanın diğer yolu ise 11. satırdaki sınırsız gönderimi ile olur. Bu durumda 5-8 satırlarındaki **for** döngüsünün her bir iterasyonu için $a_{ie} \leq 0$ bulunur. Aşağıdaki şekilde tanımlanmış $\bar{x}$ çözümünü ele alalım:

$$\bar{x}_i = \begin{cases} \infty, & i = e \quad \text{ise,} \\ 0, & i \in N - \{e\} \text{ ise,} \\ b_i - \sum_{j \in N} a_{ij} \bar{x}_j, & i \in B \quad \text{ise.} \end{cases}$$

Şimdi bu çözümün elverişli olduğunu yani tüm değişkenlerin negatif olmadığını gösterelim. $\bar{x}_e$ dışındaki temel olmayan değişkenler 0 değildir ve $\bar{x}_e = \infty > 0$ dır; bu nedenle tüm temel olmayan değişkenler negatif değildir. Herbir $\bar{x}_i$ temel değişkeni için

$$\begin{aligned} \bar{x}_i &= b_i - \sum_{j \in N} a_{ij} \bar{x}_j \\ &= b_i - a_{ie} \bar{x}_e \end{aligned}$$

olur. Döngü değişmezi ile $b_i \geq 0$ sağlanır ve $a_{ie} \leq 0$, $\bar{x}_e = \infty > 0$ olur. Bu nedenle $\bar{x}_i \geq 0$ dır.

Şimdi $\bar{x}$ çözümü için amaç değerinin sınırsız olduğunu gösterelim. (29.42) denkleminde amaç değeri aşağıdaki şekilde tanımlanır:

$$\begin{aligned} z &= v + \sum_{j \in N} c_j \bar{x}_j \\ &= v + c_e \bar{x}_e \end{aligned}$$

(SIMPLEX in 4. satırından) $c_e > 0$ ve $\bar{x}_e = \infty$ olduğu için amaç değeri ∞ dur ve bu nedenle doğrusal program sınırsızdır. ■

SIMPLEX in sonlandığını ve sonlandığında döndürülen çözümün optimal olduğunu göstermek kaldı. Kısım 29.4 de optimallik açıklanacaktır. Şimdi sonlandırma ele alınsın.

Sonlandırma

Bu bölümün başında verilen örnekte, simpleks algoritmasının her bir iterasyonu temel çözüme ilişkin amaç değerini arttırmıştır. Alıştırma 29.3-2 de kanıtlanmasının istendiği gibi, SIMPLEX in hiçbir iterasyonu temel çözümle ilişkili amaç değerini azaltamaz. Ne yazık ki, herhangi bir iterasyonda amaç değerinin değişmeden kalması mümkündür. Bu olay *yozlaşma* (*degeneracy*) olarak adlandırılır ve şimdi daha detaylı olarak incelenmelidir.

PIVOT un 14. satırındaki $\hat{v} = v + c_e \hat{b}_e$ gösterimi amaç değerini değiştirir. SIMPLEX sadece $c_e > 0$ iken PIVOT u çağırdığı için amaç değerinin değişmeden kalmasının (yani $\hat{v} = v$) tek yolu $\hat{b}_e$ nin 0 olmasıdır. Bu değer PIVOT un 3. satırında $\hat{b}_e = b_l / a_{le}$ olarak belirlenir. $a_{le} \neq 0$ olduğu tüm durumlarda PIVOT çağırıldığı için $\hat{b}_e$ nin 0 a eşit olduğu bilindiğinde, amaç değeri değişmemiştir ve $b_l = 0$ olmalıdır.

Gerçekten bu durum oluşabilir. Aşağıdaki doğrusal programı ele alalım:

$$\begin{aligned} z &= x_1 + x_2 + x_3 \\ x_4 &= 8 - x_1 - x_2 \\ x_5 &= x_2 - x_3. \end{aligned}$$

Varsayalım ki giren değişken x_1 ve çıkan değişken x_4 seçilsin. Pivotlama sonrası aşağıdaki ifadeler elde edilir:

$$\begin{aligned} z &= 8 + x_3 - x_4 \\ x_1 &= 8 - x_2 - x_4 \\ x_5 &= x_2 - x_3. \end{aligned}$$

Bu noktada tek seçenek x_3 giren ve x_5 çıkan ile pivotlama yapmaktır. $b_5 = 0$ olduğu için 8 amaç değeri pivotlama sonrası değişmeden sabit kalır:

$$\begin{aligned} z &= 8 + x_2 - x_4 - x_5 \\ x_1 &= 8 - x_2 - x_4 \\ x_3 &= x_2 - x_5. \end{aligned}$$

Amaç değeri değişmemiş ancak gevşek biçim değişime uğramıştır. Neyse ki, eğer tekrar x_2 giren ve x_1 çıkan ile pivot yaparsak, amaç değeri (16 ya) artar ve simpleks algoritması devam edebilir.

Yozlaşma simpleks algoritmanın sonlandırılmasını önleyebilir, çünkü **döngü (cycling)** olarak adlandırılan olaya sebep olabilir: SIMPLEX in iki farklı iterasyondaki gevşek biçimler aynıdır. Yozlaşma sebebiyle, SIMPLEX, birbiri ardına gelen gevşek biçim tekrarlarına rağmen, amaç değerini değişmeden sabit bırakan pivot işlemler dizisini seçebilmiştir. SIMPLEX deterministik bir algoritma olduğu için, eğer devir yaparsa, gevşek formların aynı serileri ile hiçbir zaman sonlanmadan sürekli döngü devam eder.

Döngü (cycling), SIMPLEX in sonlanmaması için tek sebeptir. Bu durumu göstermek için ilk olarak bazı ek düzenekler geliştirilmelidir.

Her bir iterasyonda SIMPLEX N ve B kümelerine ek olarak A , b , c ve v nin devamlılığını sağlar. Simpleks algoritmayı verimli olarak uygulayabilmek için açık bir şekilde A , b , c ve v nin devamlılığının sağlanması gerektiği halde, devamlılıkları sağlanmadan da uygulanabilir. Diğer bir deyişle, temel ve temel olmayan değişkenler tek bir gevşek biçim tanımlanmasına yeterli olur. Bu olayı kanıtlamadan önce kullanışlı cebirsel bir lemma kanıtlanacaktır.

Lemma 29.3

I indisler kümesi, her bir $j \in I$ için a_j ve b_j reel sayılar ve x_j reel-değerli bir değişken olsun. γ herhangi reel bir sayı olsun. Varsayalım ki herhangi bir x_j dizisi için

$$\sum_{j \in I} \alpha_j x_j = \gamma + \sum_{j \in I} \beta_j x_j \quad (29.78)$$

olsun. Bu durumda her bir $j \in I$ için α_j ve β_j ve $\gamma = 0$ dır.

Kanıt x_j nin herhangi bir değeri için denklem (29.78) sağlandığı sürece, α_j ve β_j ve γ hakkında sonucu varmak için özel değerler kullanılabilir. Eğer her bir $j \in I$ için $x_j = 0$ ise $\gamma = 0$ sonucu elde edilir. Şimdi keyfi bir $j \in I$ indisi seçilsin, $x_j = 1$ ve tüm $k \neq j$ için $x_k = 0$ yapılsın. Sonrasında $\alpha_j = \beta_j$ olur. j, I kümesinden herhangi bir indis olarak seçildiği sürece her bir $j \in I$ için $\alpha_j = \beta_j$ olacağı sonucuna varılır. ■

Özel bir doğrusal program birçok farklı gevşek biçime sahiptir: her bir gevşek biçimin orijinal doğrusal programla aynı elverişli ve optimal çözümler kümesine sahip olduğunu hatırlayalım. Şimdi doğrusal programın gevşek biçiminin tek biçimde, temel değişkenler kümesi ile tanımlandığını görelim. Öyle ki, verilen temel değişkenler kümesi ile tek gevşek biçim (tek sağ taraf ve katsayılar kümesi) bu temel değişkenler ile ilişkilidir.

Lemma 29.4

(A, b, c) standart biçimde bir doğrusal program olsun. Verilen B temel değişkenler kümesi ile ilişkili bir tek gevşek biçim tanımlanır.

Kanıt Çelişkinin amacı için varsayalım ki aynı B temel değişkenler kümesi ile iki farklı gevşek biçim tanımlı olsun. Gevşek biçimler aynı zamanda aynı $N = \{1, 2, \dots, n + m\} - B$ temel olmayan değişkenler kümesine sahip olmalıdır. İlk gevşek biçim aşağıdaki şekilde

$$z = v + \sum_{j \in N} c_j x_j \quad (29.79)$$

$$x_i = b_i - \sum_{j \in N} a_{ij} x_j \quad i \in B, \quad (29.80)$$

ve ikinci gevşek biçim aşağıdaki şekilde ifade edilsin:

$$z = v' + \sum_{j \in N} c'_j x_j \quad (29.81)$$

$$x_i = b'_i - \sum_{j \in N} a'_{ij} x_j \quad i \in B. \quad (29.82)$$

(29.80) satırındaki ilişkili denklemden (29.82) satırındaki her bir denklemi çıkartarak oluşturulan denklemler sistemi ele alınsın. Elde edilen sistem

$$0 = (b_i - b'_i) - \sum_{j \in N} (a_{ij} - a'_{ij})x_j \quad \text{her } i \in B \text{ için}$$

veya eşdeğer olarak,

$$\sum_{j \in N} a_{ij}x_j = (b_i - b'_i) + \sum_{j \in N} a'_{ij}x_j \quad \text{her } i \in B \text{ için}$$

şeklinde ifade edilir.

Şimdi her $i \in B$ için $\alpha_j = a_{ij}$, $\beta_j = a'_{ij}$, $\gamma = b_i - b'_i$ ile Lemma 29.3 uygulan-
sın. $\alpha_j = \beta_j$ olduğu sürece her $j \in N$ için $a_{ij} = a'_{ij}$ olur ve $\gamma = 0$ olduğu süre-
ce $b_i = b'_i$ dir. Bu nedenle her iki gevşek biçim için A ve b , A' ve b' ile ay-
nıdır. Benzer görüşü kullanarak, Alıştırma 29.3-1 gösteriyor ki, aynı zamanda
 $c = c'$ ve $v = v'$ olmalıdır ve sonuç olarak gevşek biçimler aynıdır. ■

Şimdi döngünün SIMPLEX in sonlanması için tek mümkün yol olduğu göste-
rilebilir.

Lemma 29.5

Eğer SIMPLEX en fazla $\binom{n+m}{m}$ iterasyonda sonlanmıyorsa döngüye(cycling) gir-
miş demektir.

Kanıt Lemma 29.4 den B temel değişkenler kümesi tek bir gevşek form tanım-
lar. $n+m$ değişken vardır ve $|B| = m$ dir bu nedenle B yi seçmek için en fazla
 $\binom{n+m}{m}$ yol vardır. Böylece sadece en fazla $\binom{n+m}{m}$ gevşek form bulunur. Sonuç ola-
rak, eğer SIMPLEX $\binom{n+m}{m}$ iterasyondan fazla uygulanırsa, döngüye girmelidir. ■

Döngü teorikte mümkündür ancak çok ender rastlanır. Giren ve çıkan deęiş-
kenler daha dikkatli seçilerek bu durum önlenabilir. Bir seçenek girdiyi kısmen
bozmaktır böylece aynı amaç değeri ile iki farklı çözüm bulmak imkansızdır. Di-
ğer bir seçenek her zaman en küçük indisli deęişkeni seçerek eşitliği bozmaktır,
bu strateji **Bland's kuralı** olarak adlandırılır. Bu stratejilerin döngüyü engellediğı
kanıtı atlanmıştır.

Lemma 29.6

Eğer SIMPLEX in 4 ve 9 satırları her zaman en küçük indisli deęişkeni seçerek
eşitliği bozuyorsa SIMPLEX in sonlanacağı kesindir. ■

Bu bölüm aşağıdaki lemma ile bitirilecektir.

Lemma 29.7

INITIALIZE-SIMPLEX in uygun temel çözümlü bir gevşek biçim döndürdüğü varsayıldığında, SIMPLEX, doğrusal programın sınırsız olduğunu bildirir ya da en fazla $\binom{n+m}{m}$ iterasyonda elverişli bir çözümle sonlanır.

Kanıt Yardımcı teorem 29.2 ve 29.6 gösteriyor ki eğer INITIALIZE-SIMPLEX uygun temel çözümlü bir gevşek biçim döndürüyorsa, SIMPLEX, doğrusal programın sınırsız olduğunu bildirir veya elverişli bir çözümle sonlanır. Lemma 29.5 devriği ile, eğer SIMPLEX elverişli bir çözümle sonlanıyorsa en fazla $\binom{n+m}{m}$ iterasyon uygulanmıştır. ■

Alıştırmalar**29.3-1**

$c = c'$ ve $v = v'$ olduğunu göstererek Lemma 29.4 ün kanıtını tamamlayınız.

29.3-2

SIMPLEX in 12. satırındaki PIVOT çağrımının hiçbir zaman v değerini azaltmadığını gösteriniz.

29.3-3

PIVOT sürecinde verilen gevşek biçim ve prosedürün döndürdüğü gevşek biçimin aynı olduğunu gösteriniz.

29.3-4

Standart biçimdeki (A, b, c) doğrusal programın gevşek biçime dönüştürüldüğünü varsayalım. Temel çözümün ancak ve ancak $b_i \geq 0$, $i = 1, 2, \dots, m$ ise elverişli olduğunu gösteriniz.

29.3-5

SIMPLEX i kullanarak aşağıdaki doğrusal programı çözünüz:

Maksimize $18x_1 + 12.5x_2$

kısıtlar

$$\begin{array}{rclcl} x_1 & + & x_2 & \leq & 20 \\ x_1 & & & \leq & 12 \\ & & x_2 & \leq & 16 \\ x_1, x_2 & & & \geq & 0 \end{array} .$$

29.3-6

SIMPLEX i kullanarak aşağıdaki doğrusal programı çözünüz:

Maksimize $5x_1 - 3x_2$

kısıtlar

$$\begin{aligned} x_1 - x_2 &\leq 1 \\ 2x_1 + x_2 &\leq 2 \\ x_1, x_2 &\geq 0 . \end{aligned}$$

29.3-7

SIMPLEX i kullanarak aşağıdaki doğrusal programı çözünüz:

Minimize $x_1 + x_2 + x_3$

kısıtlar

$$\begin{aligned} 2x_1 + 7.5x_2 + 3x_3 &\geq 10000 \\ 20x_1 + 5x_2 + 10x_3 &\geq 30000 \\ x_1, x_2, x_3 &\geq 0 . \end{aligned}$$

29.3-8

Lemma 29.5 kanıtında B temel değişkenler kümesi seçiminin en fazla $\binom{m+n}{n}$ yolla olabileceği öne sürülmüştü. B kümesini seçmek için $\binom{m+n}{n}$ dan az yol olan bir doğrusal program örneği veriniz.

29.4 Dualite

Temel varsayımlar altında SIMPLEX in sonladığı kanıtlanmıştır. Ancak bununla birlikte gerçekten doğrusal programa optimal çözümü bulduğu gösterilmedi. Bunu yapmak için güçlü bir kavram olan **doğrusal programlamadaki dualite** tanımlanmıştır.

Dualite, çözümün gerçekten optimal olduğunun kanıtlanmasını sağlar. Bölüm 26 Teorem 26.6, maks-akış min-kesim teoreminde, bir dualite örneği görülmüştü. Varsayalım ki verilen bir maksimum akış probleminde, $|f|$ değeri ile bir f akışı bulunsun. f in maksimum akış olduğunu nasıl bileceğiz? Maks-akış min-kesim probleminden yola çıkarak, eğer değeri yine $|f|$ olan bir kesim (cut) bulabiliyorsak, f in gerçekten maksimum akış olduğunu doğrulamış oluruz. Bu bağlantı bir ikilik örneği verir: verilen bir maksimizasyon problemine ilişkili bir minimizasyon problemi tanımlanır öyle ki bu iki problem aynı optimal amaç değerlerine sahiptir.

Amacı maksimizasyon olan bir doğrusal programda, amacı minimizasyon olan ve optimal değeri orjinal doğrusal program ile aynı olan bir **dual** doğrusal

programı nasıl formüle edeceğimizi tanımlamalıyız. Dual doğrusal programlardan bahsedilirken, orijinal doğrusal program **asal** olarak adlandırılır.

(29.16)-(29.18) de olduğu gibi standart biçimde verilen asal doğrusal programın dual doğrusal programı aşağıdaki şekilde tanımlanmıştır:

$$\text{Minimize } \sum_{i=1}^m b_i y_i \quad (29.83)$$

kısıtlar

$$\sum_{i=1}^m a_{ij} y_i \geq c_j \quad j = 1, 2, \dots, n \text{ için}, \quad (29.84)$$

$$y_i \geq 0 \quad i = 1, 2, \dots, m \text{ için}. \quad (29.85)$$

Duali biçimlendirmek için minimizasyonu maksimizasyon ile değiştirme, amaç fonksiyonu ve sağ taraf katsayı rollerini yer değiştirme ve her küçük-veya-eşiti büyük-veya-eşit ile değiştirme işlemini yaparız. Asaldaki her bir m kısıtı dualde bir y_i değişkeni ile ve dualdeki her bir n kısıtı asaldaki x_j değişkeni ile ilişkilidir. Örneğin (29.53)-(29.57) de verilen doğrusal program ele alınsın. Bu doğrusal programın duali aşağıda verilmiştir:

$$\text{Minimize } 30y_1 + 24y_2 + 36y_3 \quad (29.86)$$

kısıtlar

$$y_1 + 2y_2 + 4y_3 \geq 3 \quad (29.87)$$

$$y_1 + 2y_2 + y_3 \geq 1 \quad (29.88)$$

$$3y_1 + 5y_2 + 2y_3 \geq 2 \quad (29.89)$$

$$y_1, y_2, y_3 \geq 0. \quad (29.90)$$

Teorem 29.10 da dual doğrusal programın optimal değeri ile asal doğrusal programın en iyi değerinin birbirine eşit olduğunu göstermeliyiz. Buna ek olarak, simpleks algoritması gerçekten tam olarak asal ve dual doğrusal programların her ikisini benzer şekilde çözer ve böylece optimalliğin (optimality) kanıtı sağlanır.

Asal doğrusal programa herhangi bir elverişli çözüm değerinin, dual doğrusal programa herhangi bir elverişli çözüm değerinden daha fazla olmayacağını belirten **zayıf dualite** göstererek başlayacağız.

Lemma 29.8 (Zayıf doğrusal programlama ikililiği)

$\bar{x}$, (29.16)-(29.18) deki asal doğrusal programa elverişli çözüm ve $\bar{y}$, (29.83)-(29.85) de ki dual doğrusal programa elverişli çözüm olsun. Sonrasında aşağıdaki ifade elde edilir:

$$\sum_{j=1}^n c_j \bar{x}_j \leq \sum_{i=1}^m b_i \bar{y}_i.$$

Kanıt Aşağıdaki ifadeler elde edilir:

$$\begin{aligned}
 \sum_{j=1}^n c_j \bar{x}_j &\leq \sum_{j=1}^n \left(\sum_{i=1}^m a_{ij} \bar{y}_i \right) \bar{x}_j \quad ((29.84) \text{ eşitsizliği ile}) \\
 &= \sum_{i=1}^m \left(\sum_{j=1}^n a_{ij} \bar{x}_j \right) \bar{y}_i \\
 &\leq \sum_{i=1}^m b_i \bar{y}_i \quad ((29.17) \text{ eşitsizliği ile}) . \quad \blacksquare
 \end{aligned}$$

Sonuç 29.9

$\bar{x}$, (A, b, c) asal doğrusal programa elverişli çözüm ve $\bar{y}$, elverişli dual doğrusal programa elverişli çözüm olsun. Eğer

$$\sum_{j=1}^n c_j \bar{x}_j = \sum_{i=1}^m b_i \bar{y}_i ,$$

ise $\bar{x}$ ve $\bar{y}$ sırasıyla asal ve dual doğrusal programa optimal çözümlerdir.

Kanıt Lemma 29.8'den, asala elverişli çözümün amaç değeri, dualin elverişli çözümünü aşamaz. Asal doğrusal program bir maksimizasyon problemi, dual ise minimizasyon problemidir. Bu nedenle eğer $\bar{x}$ $\bar{y}$ elverişli çözümleri aynı amaç değerine sahipse her ikisi de geliştirilemez. $\blacksquare$

Herzaman için değeri optimal asal çözüme eşit olan bir dual çözüm vardır kanıtlanmadan önce böyle bir çözümü nasıl bulacağımızı anlatacağız. (29.53)–(29.57) de yer alan doğrusal program üzerinde simpleks algoritmayı çalıştırdığımızda, son iterasyon amaç $z = 28 - x_3/6 - x_5/6 - 2x_6/3$, $B = \{1, 2, 4\}$ ve $N = \{3, 5, 6\}$ ile (29.72)–(29.75) gevşek biçimine yöneltir. Aşağıda göstermemiz gerektiği gibi, sonuç gevşek biçimi ile ilişkili temel çözüm gerçekten doğrusal programa optimal çözümdür; (29.53)–(29.57) doğrusal programına bir optimal çözüm bu nedenle amaç değeri $(3 \cdot 8) + (1 \cdot 4) + (2 \cdot 0) = 28$ ile $(\bar{x}_1, \bar{x}_2, \bar{x}_3) = (8, 4, 0)$ dır. Ayrıca aşağıda gösterdiğimiz gibi, optimal dual çözümü şu şekilde okuyabiliriz: birincil amaç fonksiyonunun negatif katsayıları, dual değişken değerleridir. Daha açık olarak, asalın son gevşek biçiminin aşağıdaki gibi olduğunu düşünelim:

$$\begin{aligned}
 z &= v' + \sum_{j \in N} c'_j x_j \\
 x_i &= b'_i - \sum_{j \in N} a'_{ij} x_j \quad i \in B \text{ için.}
 \end{aligned}$$

Optimal dual çözümü oluşturmak için aşağıdaki düzenlemeyi yaparız:

$$\bar{y}_i = \begin{cases} -c'_{n+i}, & (n+i) \in N \text{ ise,} \\ 0, & \text{aksi halde.} \end{cases} \quad (29.91)$$

Böylece (29.86)-(29.90) da tanımlanan dual doğrusal programa optimal çözüm $\bar{y}_1 = 0$ ($n+1 = 4 \in B$ olduğu için), $\bar{y}_2 = -c'_5 = 1/6$, ve $\bar{y}_3 = -c'_6 = 2/3$ olur. (29.86) dual amaç fonksiyonu hesaplandığında, asalın amaç değerinin gerçekten dualin amaç değerine eşit olduğunu kanıtlayan $(30 \cdot 0) + (24 \cdot (1/6)) + (36 \cdot (2/3)) = 28$ amaç değeri elde edilir. Lemma 29.8 ile bu hesaplamalar birleştirildiğinde asal doğrusal programın optimal amaç değerinin 28 olduğu kanıtlanmış olur. Şimdi bu yaklaşımın genele uygulamasını göstereceğiz: duale en iyi çözümü bulabiliriz ve aynı zamanda asal çözümün en iyi olduğunu kanıtlayabiliriz.

Teorem 29.10 (Doğrusal programlama dualitesi)

Asal doğrusal program (A, b, c) için SIMPLEX in $\bar{x} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ değerlerini döndürdüğünü düşünelim. N ve B son gevşek biçim için temel olmayan ve temel değişkenleri, c' son gevşek biçimdeki katsayıları temsil etsin ve $\bar{y} = (\bar{y}_1, \bar{y}_2, \dots, \bar{y}_m)$ (29.91) denklemi ile tanımlansın. Sonrasında $\bar{x}$ doğrusal programa optimal çözüm ve $\bar{y}$ dual doğrusal programa optimal çözümdür ve aşağıdaki denklem sağlanır:

$$\sum_{j=1}^n c_j \bar{x}_j = \sum_{i=1}^m b_i \bar{y}_i. \quad (29.92)$$

Kanıt Sonuç 29.9 dan eğer (29.92) denklemini sağlayan $\bar{x}$ ve $\bar{y}$ elverişli çözümlerini bulabiliyorsak, $\bar{x}$ ve $\bar{y}$ optimal asal ve ikincil dual olmalıdır. Şimdi teoremden tanımlanan $\bar{x}$ ve $\bar{y}$ çözümlerinin (29.92) denklemini sağladığını göstermeliyiz.

SIMPLEX i (29.16)-(29.18) satırlarında verildiği gibi asal doğrusal program üzerinde çalıştırdığımızı düşünelim. Algoritma bir dizi gevşek biçim oluşturur ta ki

$$z = v' + \sum_{j \in N} c'_j x_j. \quad (29.93)$$

Amaç fonksiyonlu son gevşek biçim ile sonlanana kadar. SIMPLEX bir çözüm ile sonlandığında, 3 satırındaki koşuldan biliyoruz ki

$$c'_j \leq 0 \text{ her } j \in N \text{ için.} \quad (29.94)$$

Eğer

$$c'_j = 0 \text{ her } j \in B \text{ için} \quad (29.95)$$

şeklinde tanımlama yaparsak, (29.93) denklemini tekrardan aşağıdaki şekilde yazabiliriz:

$$\begin{aligned} z &= v' + \sum_{j \in N} c'_j x_j \\ &= v' + \sum_{j \in N} c'_j x_j + \sum_{j \in B} c'_j x_j \text{ (çünkü eğer } j \in B \text{ ise } c'_j = 0 \text{ dır.)} \\ &= v' + \sum_{j=1}^{n+m} c'_j x_j \text{ (çünkü } N \cup B = \{1, 2, \dots, n+m\} \text{ dir.)} \end{aligned} \quad (29.96)$$

Bu son gevşek biçim ile ilişkili $\bar{x}$ temel çözümü için $\bar{x}_j = 0$ olur $j \in N$ ve $z = v'$ için. Tüm gevşek biçimler birbirine eşit olduğu için eğer $\bar{x}$ üzerinde orijinal amaç fonksiyonunu hesaplırsak aynı amaç değerini elde etmeliyiz:

$$\sum_{j=1}^n c_j \bar{x}_j = v' + \sum_{j=1}^{n+m} c'_j \bar{x}_j \quad (29.97)$$

$$\begin{aligned} &= v' + \sum_{j \in N} c'_j \bar{x}_j + \sum_{j \in B} c'_j \bar{x}_j \\ &= v' + \sum_{j \in N} (c'_j \cdot 0) + \sum_{j \in B} (0 \cdot \bar{x}_j) \\ &= v' . \end{aligned} \quad (29.98)$$

Şimdi (29.91) denklemi ile tanımlanan $\bar{y}$ nin, dual doğrusal program için elverişli olduğunu göstermeliyiz yani onun amaç değeri $\sum_{i=1}^m b_i \bar{y}_i$ eşittir $\sum_{j=1}^n c_j \bar{x}_j$. Denklem (29.97) söylüyor ki $\bar{x}$ de hesaplanan ilk ve son gevşek biçimler eşittir. Daha genel olarak, tüm gevşek biçimlerin eşit olması ile herhangi bir $x = (x_1, x_2, \dots, x_n)$ değerler kümesi için aşağıdaki ifade elde edilir:

$$\sum_{j=1}^n c_j x_j = v' + \sum_{j=1}^{n+m} c'_j x_j .$$

Böylece, herhangi bir $\bar{x} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ değerler kümesi için aşağıdaki ifadeler elde edilir:

$$\begin{aligned}
& \sum_{j=1}^n c_j \bar{x}_j \\
&= v' + \sum_{j=1}^{n+m} c'_j \bar{x}_j \\
&= v' + \sum_{j=1}^n c'_j \bar{x}_j + \sum_{j=n+1}^{n+m} c'_j \bar{x}_j \\
&= v' + \sum_{j=1}^n c'_j \bar{x}_j + \sum_{i=1}^m c'_{n+i} \bar{x}_{n+i} \\
&= v' + \sum_{j=1}^n c'_j \bar{x}_j + \sum_{i=1}^m (-\bar{y}_i) \bar{x}_{n+i} \quad ((29.91) \text{ ve } (29.95) \text{ denklemlerinden}) \\
&= v' + \sum_{j=1}^n c'_j \bar{x}_j + \sum_{i=1}^m (-\bar{y}_i) \left(b_i - \sum_{j=1}^n a_{ij} \bar{x}_j \right) \quad ((29.32) \text{ denkleminde}) \\
&= v' + \sum_{j=1}^n c'_j \bar{x}_j - \sum_{i=1}^m b_i \bar{y}_i + \sum_{i=1}^m \sum_{j=1}^n (a_{ij} \bar{x}_j) \bar{y}_i \\
&= v' + \sum_{j=1}^n c'_j \bar{x}_j - \sum_{i=1}^m b_i \bar{y}_i + \sum_{j=1}^n \sum_{i=1}^m (a_{ij} \bar{y}_i) \bar{x}_j \\
&= \left(v' - \sum_{i=1}^m b_i \bar{y}_i \right) + \sum_{j=1}^n \left(c'_j + \sum_{i=1}^m a_{ij} \bar{y}_i \right) \bar{x}_j ,
\end{aligned}$$

Böylece

$$\sum_{j=1}^n c_j \bar{x}_j = \left(v' - \sum_{i=1}^m b_i \bar{y}_i \right) + \sum_{j=1}^n \left(c'_j + \sum_{i=1}^m a_{ij} \bar{y}_i \right) \bar{x}_j . \quad (29.99)$$

Lemma (29.3) (29.99) denkleminde uygulandığı aşağıdaki ifadeleri elde ederiz

$$v' - \sum_{i=1}^m b_i \bar{y}_i = 0 , \quad (29.100)$$

$$c'_j + \sum_{i=1}^m a_{ij} \bar{y}_i = c_j \quad j = 1, 2, \dots, n \text{ için.} \quad (29.101)$$

(29.100) denkleminde, $\sum_{i=1}^m b_i \bar{y}_i = v'$ elde edilir ve bu yüzden asılın amaç değeri $\left(\sum_{i=1}^m b_i \bar{y}_i \right)$ asılınke (v') eşittir. $\bar{y}$ çözümünün dual problem için uygun

olduğunu göstermek kalmıştır. (29.94) eşitsizliği ve (29.95) denkleminde her $j = 1, 2, \dots, n + m$ için $c'_j \leq 0$ elde edilir. Böylece, herhangi $j = 1, 2, \dots, n$ için (29.101) denklemleri, dualin (29.84) kısıtlarını sağlayan aşağıdaki ifadelere sebep olur:

$$\begin{aligned} c_j &= c'_j + \sum_{i=1}^m a_{ij} \bar{y}_i \\ &\leq \sum_{i=1}^m a_{ij} \bar{y}_i \quad . \end{aligned}$$

Sonuç olarak, her $j \in N \cup B$ için $c'_j \leq 0$ olduğu sürece, $\bar{y}$ yi denklem (29.91) e göre kurarsak her $\bar{y}_i \geq 0$ elde ederiz ve böylece negatif olmama kısıtları da sağlanmış olur. ■

Gösterdik ki, verilen bir doğrusal programda eğer INITIALIZE-SIMPLEX elverişli çözüm döndürürse ve eğer SIMPLEX “sınırsız” çıktısı olmadan sonlanırsa dönen çözüm gerçekten optimal çözümdür. Aynı zamanda dual doğrusal programa optimal çözümün nasıl yapılandırılacağını gösterdik.

Alıştırmalar

29.4-1

Alıştırma 29.3-5 de verilen doğrusal programın dualini oluşturunuz.

29.4-2

Varsayalım ki standart biçimde olmayan bir doğrusal programımız olsun. Duali için onu ilk önce standart biçime dönüştürüp sonrasında dualini alabilirdik. Duali direkt oluşturma tabii ki daha elverişli olurdu. Herhangi bir doğrusal programın dualini direkt olarak nasıl oluşturabileceğimizi açıklayınız.

29.4-3

Sayfa 860 ta (29.47)–(29.50) satırlarında verilen maksimum akış doğrusal programının dualini yazınız. Bu formülasyonu, bir min-kesim problemine nasıl çevirebileceğimizi açıklayınız.

29.4-4

Sayfa 862 de (29.51)–(29.52) satırlarında verilen minimum-maliyet-akışı doğrusal programının dualini yazınız. Bu problemi, çizgeler ve akışlar ile nasıl yorumlayabileceğimizi açıklayınız.

29.4-5

Bir doğrusal programın dualinin asal doğrusal program olacağını gösteriniz.

29.4-6

Bölüm 26'nın hangi sonucu, maksimum akış problemi için zayıf dualiteyi açıklar?

29.5 Başlangıç temel elverişli çözüm

Bu bölümde, ilk olarak bir doğrusal programın elverişli olup olmadığını nasıl test edeceğimizi ve eğer elverişli ise, temel çözümü elverişli olan için gevşek biçimi nasıl oluşturacağımızı tanımlayacağız. Doğrusal programlamanın temel teoremi olan, SIMPLEX prosedürü her zaman doğru sonucu üretir teoremini kanıtlayarak sonuçlandıracağız.

Bir başlangıç çözümü bulma

Kısım 29.3'te bir doğrusal programın herhangi bir elverişli çözüme sahip olup olmadığını belirten INITIALIZE-SIMPLEX prosedürünün olduğunu varsaydık ve eğer böyle bir çözüm varsa temel çözümü elverişli olan bir gevşek biçim verir. Bu prosedür burada tanımlanacaktır.

Bir doğrusal program başlangıç temel çözümü elverişli olmadan da elverişli olabilir. Örneğin aşağıdaki doğrusal program ele alınsın:

$$\text{Maksimize } 2x_1 - x_2 \quad (29.102)$$

kısıtlar

$$2x_1 - x_2 \leq 2 \quad (29.103)$$

$$x_1 - 5x_2 \leq -4 \quad (29.104)$$

$$x_1, x_2 \geq 0. \quad (29.105)$$

Eğer bu doğrusal program gevşek biçime dönüştürülecekse, temel çözüm $x_1 = 0$ ve $x_2 = 0$ yapılır. Bu çözüm (29.104) kısıtını sağlamaz bu nedenle elverişli çözüm değildir. Bu yüzden INITIALIZE-SIMPLEX sadece açık gevşek biçimi döndüremez. Bir doğrusal programın herhangi bir elverişli çözüme sahip olup olmadığını belirlemek için bir *yardımcı doğrusal program* formüle edeceğiz. Bu yardımcı doğrusal program için temel çözümü elverişli olan bir gevşek biçim bulabiliriz (ufak bir uğraşla). Ayrıca, bu yardımcı doğrusal programın çözümü başlangıç doğrusal programının elverişli olup olmadığını belirler ve öyleyse, elverişli çözümü sağlıyorsa SIMPLEX'i başlatabiliriz.

Lemma 29.11

L , (29.16)-(29.18) da verildiği gibi standart biçimde bir doğrusal program olsun. x_0 yeni bir değişken, ve L_{aux} $n + 1$ değişken ile aşağıdaki doğrusal program olsun:

$$\text{Maksimize} \quad -x_0 \quad (29.106)$$

kısıtlar

$$\sum_{j=1}^n a_{ij}x_j - x_0 \leq b_i \quad i = 1, 2, \dots, m \text{ için}, \quad (29.107)$$

$$x_j \geq 0 \quad j = 0, 1, \dots, n \text{ için}. \quad (29.108)$$

Öyleyse, L ancak ve ancak L_{aux} un optimal amaç değeri 0 ise elverişlidir.

Kanıt Varsayalım ki $\bar{x} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n)$ L nin elverişli çözümü olsun. Öyleyse $\bar{x}$ ile birleştirilen $\bar{x}_0 = 0$ çözümü 0 amaç değeri ile L_{aux} a elverişli çözümdür. $x_0 \geq 0$ L_{aux} un kısıtı ve amaç fonksiyonu $-x_0$ ı maksimizasyon iken bu çözüm L_{aux} için optimal olmalıdır.

Tam tersi L_{aux} un optimal amaç değerinin 0 olduğunu varsayalım. Öyleyse $\bar{x}_0 = 0$ ve $\bar{x}$ nin kalan çözüm değerleri L nin kısıtlarını sağlar. ■

Şimdi standart biçimde bir doğrusal program için başlangıç temel elverişli çözüm bulma stratejimizi açıklayacağız.

INITIALIZE-SIMPLEX (A, b, c)

- 1 k en küçük b_i nin indisi olsun.
- 2 **if** $b_k \geq 0$ ise // başlangıç temel çözüm uygun mu?
- 3 **return** ($\{1, 2, \dots, n\} \setminus \{n+1, n+2, \dots, n+m\}, A, b, c, 0$)
- 4 her bir kısıtın sol tarafına $-x_0$ ekleyerek ve amaç fonksiyonunu $-x_0$ yaparak, L_{aux} u formüle et.
- 5 (N, B, A, b, c, v) L_{aux} un son gevşek formu olsun.
- 6 $l = n + k$
- 7 // L_{aux} $n + 1$ temel olmayan ve m temel değişkene sahip
- 8 $(N, B, A, b, c, v) = \text{PIVOT}(N, B, A, b, c, v, l, 0)$
- 9 // Temel çözüm şimdi L_{aux} için elverişlidir.
- 10 L_{aux} a en iyi çözüm bulunana kadar SIMPLEX in 3-12 satırlarını **while** döngüsüne sok
- 11 **if** L_{aux} a optimal çözüm $\bar{x}_0 = 0$ yaparsa
- 12 **eğer** $\bar{x}_0$ temel ise
- 13 temel olmayan yapmak için bir (yozlaşmış) PIVOT uygula.
- 14 L_{aux} un son gevşek biçim kısıtlarından x_0 ı sil ve L nin orijinal amaç fonksiyonunu geri getir fakat bu amaç fonksiyonundaki her bir temel değişkeni sağ taraftaki ilişkili kısıtı ile yer değiştir.
- 15 **return** değiştirilmiş son gevşek biçim
- 16 **else return** “elverişsiz”

INITIALIZE-SIMPLEX şu şekilde çalışır. 1-3 satırlarında $N = \{1, 2, \dots, n\}$ $B = \{n+1, n+2, \dots, n+m\}$ $\bar{x}_i = b_i$ $\bar{x}_j = 0$ ile verilen L için başlangıç gevşek biçimine temel çözüm basitçe test edilir. (gevşek biçimi yaratmak fazla güç gerektirmez çünkü A , b ve c değerleri her iki gevşek ve standart biçimde aynıdır.) Eğer satır 2 temel çözümün elverişli olduğunu bulursa-yani $\bar{x}_i \geq 0 \in N \cup B$ - 3 satırı gevşek biçimi döndürür. Değilse 4 satırında Lemma 29.11 de verilen yardımcı doğrusal program L_{aux} ü formüle edilir. L ye başlangıç temel çözüm elverişli değil ise L_{aux} için başlangıç temel çözümde elverişli değildir. Temel elverişli çözüm bulmak için tek bir pivot işlemi uygulanır. 6 satırı, önümüzdeki pivot prosedüründe giden değişken olacak temel değişken indisini $l = n+k$ seçer. Temel değişkenler $x_{n+1}, x_{n+2}, \dots, x_{n+m}$ olduğu sürece x_l çıkan değişken en negatif değerdir. 8 satırında x_0 giren ve x_l çıkan değişkenler ile PIVOT çağrısı yapılır. Kısaca görmeliyiz ki, bu PIVOT çağrısının döndürdüğü sonuç elverişli olacaktır. Şimdi temel çözümü elverişli olan bir gevşek biçimi sahibiz 10 satırında yardımcı doğrusal programı tamamıyla çözmek için tekrar tekrar PIVOT çağrısı yapabiliriz. 11 satırındaki testin gösterdiği gibi eğer 0 amaç değeri ile $L\bar{x}_0 = 0$ a optimal çözüm bulunursa, 12-14 satırlarında temel çözümü elverişli olan L için bir gevşek biçim yaratırız. Bunu yapmak için öncelikle 12-13 satırlarında x_0 ın $\bar{x}_0 = 0$ değeri ile hala temel olabileceği yozlaşmış durumu ele alırız. Bu durumda temelden x_0 ı atmak için giren değişken olarak $a_{0e} \neq 0$ olan $e \in N$ kullanarak bir pivot adımı uygulayız. Yeni temel çözüm elverişli kalır; yozlaşmış pivot hiçbir değişken değerini değiştirmez. Sonrasında kısıtlardan tüm x_0 değerlerini sileriz, ve L için orijinal amaç fonksiyonunu tekrardan oluştururuz. Orijinal amaç fonksiyonu her iki temel ve temel olmayan değişkenleri içerebilir. Bu nedenle amaç fonksiyonunda her bir temel değişkeni ilişkili kısıtın sağ tarafı ile yer değiştiririz. Sonrasında 15 satırı bu geliştirilen gevşek biçimi döndürür. Eğer diğer yandan, 11 satırı L orijinal doğrusal programın elverişsiz olduğunu belirlerse, 16 satırı bu bilgiyi döndürür.

Şimdi, (29.102)-(29.105) yer alan doğrusal program üzerinde INITIALIZE-SIMPLEX işlemini göstereceğiz. Eğer x_1 ve x_2 için (29.103) ve (29.104) eşitsizliklerini sağlayan negatif olmayan değerleri bulabilirsek, bu doğrusal program elverişlidir. Lemma 29.11 i kullanarak, yardımcı doğrusal program formüle edilmiştir:

$$\text{Maksimize} \quad -x_0 \quad (29.109)$$

kısıtlar

$$2x_1 - x_2 - x_0 \leq 2 \quad (29.110)$$

$$x_1 - 5x_2 - x_0 \leq -4 \quad (29.111)$$

$$x_1, x_2, x_0 \geq 0 .$$

Lemma 29.11 den, eğer yardımcı doğrusal programın optimal amaç değeri 0 ise, orijinal doğrusal program elverişli çözüme sahiptir. Eğer yardımcı doğrusal prog-

ramın optimal amaç değeri negatif ise orijianl doğrusal programın elverişli çözümü yoktur.

Bu doğrusal programı gevşek biçimde yazarsak aşağıdaki ifadeyi elde ederiz:

$$\begin{aligned} z &= & -x_0 \\ x_3 &= 2 - 2x_1 + x_2 + x_0 \\ x_4 &= -4 - x_1 + 5x_2 + x_0 . \end{aligned}$$

Daha tehlikeden kurtulmadık çünkü $x_4 = -4$ olan temel çözüm yardımcı doğrusal program için elverişli değildir. Bununla birlikte PIVOT a bir çağrı ile bu gevşek formu temel çözümü elverişli olan bir gevşek biçime dönüştürebiliriz. 8 satırında gösterdiği gibi, giren değişken olarak x_0 ı seçeriz. 6 satırında temel çözüm değeri en negatif temel değişken x_4 ü çıkan değişken olarak seçeriz. Pivotlama sonrası aşağıdaki gevşek biçime sahip oluruz:

$$\begin{aligned} z &= -4 - x_1 + 5x_2 - x_4 \\ x_0 &= 4 + x_1 - 5x_2 + x_4 \\ x_3 &= 6 - x_1 - 4x_2 + x_4 . \end{aligned}$$

Elverişli olan ilişkil temel çözüm $(\bar{x}_0, \bar{x}_1, \bar{x}_2, \bar{x}_3, \bar{x}_4) = (4, 0, 0, 6, 0)$ dir. Şimdi L_{aux} a optimal çözümü bulana kadar tekrar tekrar PIVOT u çağıracağız. Bu durumda x_2 giren, x_0 çıkan değişken ile PIVOT a bir çağrı aşağıdaki ifadeye götürür:

$$\begin{aligned} z &= & -x_0 \\ x_2 &= \frac{4}{5} - \frac{x_0}{5} + \frac{x_1}{5} + \frac{x_4}{5} \\ x_3 &= \frac{14}{5} + \frac{4x_0}{5} - \frac{9x_1}{5} + \frac{x_4}{5} . \end{aligned}$$

Bu gevşek biçim yardımcı probleme son çözümdür. Bu çözüm $x_0 = 0$ olduğu için biliyoruz ki, bizim başlangıç problemimiz elverişlidir. Bu yüzden $x_0 = 0$ olduğu sürece bunu kısıtlar kümesinden silebiliriz. Sonrasında orijinal amaç fonksiyonunu sadece temel olmayan değişkenleri içermek amacıyla yapılan elverişli yer değiştirmeler ile tekrardan yapılandırırız. Bizim örneğimizde aşağıdaki amaç fonksiyonu elde edilir:

$$2x_1 - x_2 = 2x_1 - \left(\frac{4}{5} - \frac{x_0}{5} + \frac{x_1}{5} + \frac{x_4}{5} \right) .$$

$x_0 = 0$ yapıp basitleştirerek, aşağıdaki amaç fonksiyonu

$$-\frac{4}{5} + \frac{9x_1}{5} - \frac{x_4}{5}$$

ve gevşek biçim

$$\begin{aligned}
z &= -\frac{4}{5} + \frac{9x_1}{5} - \frac{x_4}{5} \\
x_2 &= \frac{4}{5} + \frac{x_1}{5} + \frac{x_4}{5} \\
x_3 &= \frac{14}{5} - \frac{9x_1}{5} + \frac{x_4}{5}
\end{aligned}$$

elde edilir.

Bu gevşek biçim bir elverişli temel çözüme sahiptir ve bunu SIMPLEX işlemine döndürebiliriz.

Şimdi formal olarak INITIALIZE-SIMPLEX in doğruluğunu göstereceğiz.

Lemma 29.12

Eğer bir L doğrusal programının hiçbir elverişli çözümü yoksa, INITIALIZE-SIMPLEX “elverişsiz”, var ise de temel çözümü elverişli olan geçerli bir gevşek form döndürür.

Kanıt Öncelikle L doğrusal programının hiçbir elverişli çözümünün olmadığını varsayalım. Sonrasında yardımcı teorem 29.11’den, (29.106)-(29.108) de tanımlı L_{aux} un optimal amaç değerinin 0 dan farklı bir değer olması ve x_0 üzerindeki negatif olmama kısıtı sebebiyle optimal amaç değeri negatif olmalıdır. Dahası bu amaç değeri sonlu olmalıdır çünkü $i = 1, 2, \dots, n$ ve $x_0 = |\min_{i=1}^m \{b_i\}|$ elverişlidir ve bu çözüm $-|\min_{i=1}^m \{b_i\}|$ amaç değerine sahiptir. Bu nedenle BAŞLANGIÇ-SIMPLEX in 10 satırı pozitif olmayan bir amaç değeri ile bir çözüm bulur. $\bar{x}_0$ son gevşek biçim ile ilişkili temel çözüm olsun. $\bar{x}_0 = 0$ olamaz çünkü olursa L_{aux} un amaç değeri 0 olur ve bu da amaç değerinin negatif olmasıyla çakışır. Bu nedenle 11 satırındaki test 16 satırında “elverişsiz” ile döndürülür.

Şimdi ise L doğrusal programının bir elverişli çözümünün olduğunu varsayalım. Alistırma 29.3-4 den biliyoruz ki eğer $i = 1, 2, \dots, m$ için $b_i \geq 0$ ise başlangıç gevşek biçimine ilişkin temel çözüm elverişlidir. Bu durumda 2-3 satırı girdiye ilişkin gevşek formu döndürür (Standart biçimi gevşek biçime çevirmek kolaydır çünkü A, b, c her ikisinde de aynıdır).

İspatın geri kalanında doğrusal programın elverişli olduğu ancak 3. satırda döndürmediğimiz durumu ele alacağız. Bu durumda 4-10 satırları 0 amaç değeri ile L_{aux} a elverişli çözüm bulur. Öncelikle 1-2 satırları ile

$$b_k < 0$$

ve

$$b_k \leq b_i \text{ her bir } i \in B \text{ için} \quad (29.112)$$

elde edilmelidir. 8 satırında, x_l (hatırlayalım ki $l = n + k$ olduğundan $b_l < 0$) çıkan değişkeni minimum b_i li denklemin sol tarafı ve x_0 giren değişkeni, ekstradan ek-

lenen değişken olan bir pivot işlemi uyguluyoruz. Şimdi göstereceğiz ki, bu pivottan sonra tüm b girdiler negatif değildir ve bu nedenle L_{aux} a temel çözüm elverişlidir. PIVOT a çağrımdan sonra temel çözüm $\bar{x}$ ve PIVOT ile döndürülen değerler $\hat{b}, \hat{B}$ yapılırsa Lemma 29.1 aşağıdaki ifadelere götürür:

$$\bar{x}_i = \begin{cases} b_i - a_{ie}\hat{b}_e, & i \in \hat{B} - \{e\} \text{ ise,} \\ b_i/a_{ie}, & i = e \text{ ise.} \end{cases} \quad (29.113)$$

8. satırda PIVOT a çağrıda $e = 0$ dir. Eğer a_{i0} katsayılarını içermesi için (29.107) eşitsizliğini tekrardan yazarsak,

$$\sum_{j=0}^n a_{ij}x_j \leq b_i \quad i = 1, 2, \dots, m \text{ için,} \quad (29.114)$$

olur ve buradan

$$a_{i0} = a_{ie} = -1 \text{ her bir } i \in B \text{ için} \quad (29.115)$$

elde edilir.(bu bağlamda a_{i0} katsayısının negatifi değil, (29.114) eşitsizliğinde görülen x_0 ın katsayılarıdır, çünkü L_{aux} gevşek yerine standart biçimdedir.) $i \in B$ olduğu için aynı zamanda $a_{ie} = -1$ dir. Bu nedenle $b_i/a_{ie} > 0$ ve $\bar{x}_e > 0$ dir. Kalan temel değişkenler için

$$\begin{aligned} \bar{x}_i &= b_i - a_{ie}\hat{b}_e && \text{(denklem (29.113) ten)} \\ &= b_i - a_{ie}(b_l/a_{le}) && \text{(PIVOT un 3. satırından)} \\ &= b_i - b_l && \text{(denklem (29.115) ve } a_{ie} = -1 \text{ den)} \\ &\geq 0 && \text{((29.112) eşitsizliğinden)} \end{aligned}$$

Eşitliği elde edilir ve burada her temel değişken şimdi negatif değildir. Bu nedenle 8 satırındaki PIVOT a çağrımdan sonra temel çözüm elverişlidir. Sonrasında L_{aux} u çözdüren 10 satırını çalıştıracaktır. L nin elverişli çözüme sahip olduğunu varsaydığımız için, Lemma 29.11 den L_{aux} un en iyi çözümü 0 amaç değerlidir. Tüm gevşek biçimler eşit olduğu için L_{aux} a son temel çözümde $\bar{x}_0 = 0$ olmalıdır ve doğrusal programdan x_0 silindikten sonra L için elverişli olan gevşek biçimi elde ederiz. Sonrasında 15 satırında bu gevşek biçim döndürülür. ■

Doğrusal programlamanın temel teoremi

SIMPLEX prosedürünün çalıştığını göstererek bu bölümü sonlandıracağız. Özellikle herhangi bir doğrusal program elverişsizdir, sınırsızdır veya sonlu amaç değeri ile optimal çözüme sahiptir. Her bir durum için SIMPLEX elverişli şekilde çalışır.

Teorem 29.13 (Doğrusal programlamanın temel teoremi)

Standart biçimde verilen herhangi bir L doğrusal programı aşağıdaki 3 seçenekten birini sağlar.

1. sonlu amaç değeri ile optimal çözüme sahip
2. elverişsiz
3. sınırsız

Eğer L elverişsiz ise SIMPLEX “elverişsiz” döndürür. Eğer L sınırsız ise SIMPLEX “sınırsız” döndürür. Aksi takdirde ise SIMPLEX sonlu amaç değeri ile en iyi çözümleri döndürür.

Kanıt Lemma 29.12 den, eğer L doğrusal programı elverişsiz ise SIMPLEX “elverişsiz” döndürür. Şimdi L doğrusal programının elverişli olduğunu varsayalım. Lemma 29.12 den, INITIALIZE-SIMPLEX temel çözümü elverişli olan bir gevşek biçim döndürür. Bu nedenle, Lemma 29.7 den SIMPLEX sınırsız döndürür veya elverişli çözüm ile sonlanır. Eğer sonlu çözüm ile sonlanırsa Teorem 29.10 bu çözümün optimal olduğunu söyler. Diğer yandan, eğer SIMPLEX “sınırsız” döndürürse, Lemma 29.2 L doğrusal programının gerçekten sınırsız olduğunu söyler. SIMPLEX her zaman bu yollardan biriyle sonlanacağı için kanıt tamamlanmıştır. ■

Alıştırmalar**29.5-1**

INITIALIZE-SIMPLEX in 5 ve 14 satırlarını uygulayan detaylı bir kabakod ver.

29.5-2

SIMPLEX in ana döngüsü INITIALIZE-SIMPLEX ile çalıştırıldığında “sınırsız” döndüremeyeceğini göster.

29.5-3

Standart formda bir L doğrusal programının verildiğini ve her iki L ve L nin duali için başlangıç gevşek biçimlerine ilişkin temel çözümlerin elverişli olduğunu varsayalım. L nin en optimal değerinin 0 olduğunu gösteriniz.

29.5-4

Doğrusal programda katı eşitsizliklere izin verdiğimiz varsayalım. Bu durumda doğrusal programlamanın temel teoreminin sağlanmayacağını gösteriniz.

29.5-5

SIMPLEX i kullanarak aşağıdaki doğrusal programı çözünüz:

Maksimize $x_1 + 3x_2$

kısıtlar

$$\begin{aligned} x_1 - x_2 &\leq 8 \\ -x_1 - x_2 &\leq -3 \\ -x_1 + 4x_2 &\leq 2 \\ x_1, x_2 &\geq 0 . \end{aligned}$$

29.5-6

SIMPLEX i kullanarak aşağıdaki doğrusal programı çözünüz:

Maksimize $x_1 - 2x_2$

kısıtlar

$$\begin{aligned} x_1 + 2x_2 &\leq 4 \\ -2x_1 - 6x_2 &\leq -12 \\ x_2 &\leq 1 \\ x_1, x_2 &\geq 0 . \end{aligned}$$

29.5-7

SIMPLEX i kullanarak aşağıdaki doğrusal programı çözünüz:

Maksimize $x_1 + 3x_2$

kısıtlar

$$\begin{aligned} -x_1 + x_2 &\leq -1 \\ -x_1 - x_2 &\leq -3 \\ -x_1 + 4x_2 &\leq 2 \\ x_1, x_2 &\geq 0 . \end{aligned}$$

29.5-8

(29.6)-(29.10) da verilen doğrusal programı çözünüz.

29.5-9

Aşağıdaki P olarak adlandırdığımız 1-değişkenli doğrusal programı ele alalım:

Maksimize tx

$$\begin{aligned} \text{kısıtlar} \quad rx &\leq s \\ x &\geq 0 , \end{aligned}$$

burada r, s ve t keyfi reel sayılardır. D, L nin duali olsun. r, s ve t nin hangi değer-

leri için aşağıdaki 4 maddeyi savunabileceğini belirt.

1. Heriki P ve D sonlu amaç değerleri ile optimal çözümlere sahiptir.
2. P elverişli fakat D elverişsizdir.
3. D elverişli fakat P elverişsizdir.
4. Ne P ne de D elverişlidir.

Problemler

29-1 Doğrusal-eşitsizlik elverişliliği (feasibility)

Verilen n adet $x_1, x_2, \dots, x_n$ değişkenleri üzerinde, m adet doğrusal eşitsizlik kümesi ile, **doğrusal-eşitsizlik elverişliliği problemi** aynı anda herbir eşitsizliği sağlayan değişkenler kümesi olup olmadığını sormaktadır.

- a. Göster ki, eğer doğrusal programlama için bir algoritmaya sahipsek, bu algoritmayı doğrusal-eşitsizlik elverişliliği problem çözümünde de kullanabiliriz. Doğrusal-programlama probleminde kullanacağınız değişkenler ve sabitlerin sayısı n ve m için polinomiyal olmalıdır.
- b. Göster ki, eğer doğrusal-eşitsizlik elverişliliği problemi için bir algoritmaya sahipsek, doğrusal programlama problem çözümünde de kullanabiliriz. Doğrusal-eşitsizlik elverişliliği probleminde kullanacağınız değişkenler ve doğrusal eşitsizliklerin sayısı, doğrusal programdaki değişken ve kısıtların sayısı olan n ve m için polinomiyal olmalıdır.

29.2 Tümleyen gevşeklik

Tümleyen gevşeklik, asal değişkenlerin değerleri ile dual kısıtları arasında ve dual değişkenlerin değerleri ile asal kısıtları arasındaki ilişkiyi tanımlar. $\bar{x}$, (29.16)-(29.18) da verilen asal doğrusal programa elverişli çözüm ve $\bar{y}$, (29.83)-(29.85) de verilen dual doğrusal programa elverişli çözüm olsun. Tümleyen gevşeklik, $\bar{x}$ ve $\bar{y}$ nin optimal olması için, aşağıdaki koşulların gerekli ve yeterli olduğunu belirtir:

$$\sum_{i=1}^m a_{ij} \bar{y}_i = c_j \text{ veya } \bar{x}_j = 0 \quad j = 1, 2, \dots, n \text{ için}$$

ve

$$\sum_{j=1}^n a_{ij} \bar{x}_j = b_i \text{ veya } \bar{y}_i = 0 \quad i = 1, 2, \dots, m \text{ için.}$$

- a. Tümleyen gevşekliğin, (29.53)-(29.57) satırlarındaki doğrusal program için sağlandığını doğrulayınız.
- b. Tümleyen gevşekliğin, herhangi bir doğrusal program ve duali için sağlandığını kanıtla.
- c. (29.16)-(29.18) de verilen asal doğrusal programa $\bar{x}$ elverişli çözümü optimaldir ancak ve ancak $\bar{y} = (\bar{y}_1, \bar{y}_2, \dots, \bar{y}_m)$ değerleri varsa öyle ki
 1. $\bar{y}$, (29.83)-(29.85) de verilen dual doğrusal programa elverişli çözümdür,
 2. $\sum_{i=1}^m a_{ij} \bar{y}_i = c_j \quad \bar{x}_j > 0$ ve
 3. $\bar{y}_i = 0 \quad \sum_{j=1}^n a_{ij} \bar{x}_j < b_i$.

29-3 Tam sayılı doğrusal programlama

Bir **tam sayılı doğrusal programlama problemi**, x değişkenlerinin tam sayı değerleri alma zorunluluğu ek kısıtı ile bir doğrusal programlama problemidir. Alıştırma 34.5-3 gösteriyor ki bir tam sayılı doğrusal programın elverişli çözümünün olup olmadığını belirleme NP-zordur yani; bu problem için bilinen polinomial zamanlı algoritma yoktur.

- a. Zayıf-dualite (weak duality) (Lemma 29.8) in bir tamsayılı doğrusal program için sağlandığını gösteriniz.
- b. Bir tam sayılı doğrusal program için dualitenin (Teorem 29.10) her zaman sağlanmadığını gösteriniz.
- c. Standart biçimde verilen bir asal doğrusal program için P asal doğrusal programın optimal amaç değeri, D dualinin optimal amaç değeri, IP asalının (değişkenlerin tam sayı değer alma kısıtı eklenen asal) tam sayılı versiyonu için optimal amaç değeri ve ID dualin tam sayılı versiyonu için optimal amaç değeri olsun. Her iki asal tam sayılı program ve dual tam sayılı programın elverişli ve sınırlı olduğunu varsayarak aşağıdaki ifadeyi gösteriniz:

$$IP \leq P = D \leq ID.$$

29-4 Farka'nın lemması

A , $m \times n$ matris ve c , n -vektör olsun. Farka'nın lemması tam olarak aşağıdaki sistemlerden birinin

$$Ax \leq 0 ,$$

$$c^T x > 0$$

ve

$$A^T y = c ,$$

$$y \geq 0$$

çözülebilir olduğunu belirtir, burada x n -vektör, y m -vektördür.

29-5 Minimum-maliyet sirkülasyonu (minimum-cost circulation)

Bu problemde, Kısım 29.2 den talep, kaynak veya çıkış düğümü verilmeyen çeşitli minimum-maliyet akışı problemleri ele alınmıştır. Bunun yerine, önceden olduğu gibi, bir ağ akışı ve ayrıt maliyeti $a(u,v)$ verilmiştir. Bir akış, eğer her bir ayrıt kapasite kısıtını ve her tepede akış korunumunu sağlıyor ise elverişlidir. Amaç tüm elverişli akışlar arasından minimum maliyetli olanını bulmaktır. Bu problem **minimum-maliyet-sirkülasyon problemi** olarak adlandırılır.

- a. Minimum-maliyet-sirkülasyon problemini bir doğrusal program gibi formüle ediniz.
- b. Tüm $(u, v) \in E$ ayrıtları için $a(u, v) > 0$ olsun. Minimum-maliyet-sirkülasyon problemi için bir optimal çözüm tanımlayın.
- c. Minimum-akış problemini, minimum-maliyet-sirkülasyon problemi doğrusal programı gibi formüle ediniz. s kaynak, t çıkış düğümü, c ayrıt kapasiteleri ile verilen $G = (V, E)$ örneği için, c' ayrıt kapasiteleri, a' ayrıt maliyetleri ile (mümkün farklı) $G' = (V', E')$ ağını vererek bir minimum-maliyet-sirkülasyon problemi yaratın öyle ki; minimum-maliyet-sirkülasyon problemine bir çözümünden maksimumk-akış problemine bir çözüm algılayabilin.
- d. Tek-kaynaklı en kısa-yol problemini minimum-maliyet-sirkülasyon problemi doğrusal programı gibi formüle ediniz.

Bölüm notları

Bu bölüm geniş alanda doğrusal programlamayı sadece çalışmaya başlamıştır. Chvatal [69], Gass [130], Karloff [197], Schrijver [303] ve Vanderbei [344] de dahil olmak üzere birkaç kitap sadece doğrusal programlamaya adanmıştır. Papadimitriou ve Steiglitz [271] ve Ahuja, Magnanti ve Orlin [7] de dahil olmak üzere diğer birçok kitap, doğrusal programlamanın iyi bir kapsamını verir. Bu bölüm içeriğinde Chavatal dan alınan yaklaşımdan yararlanılmıştır.

Doğrusal programlama için simpleks algoritma 1947 de G. Dantzig tarafından bulunmuştur. Kısa bir süre sonra, araştırmacılar çeşitli alanlardan birkaç problemin doğrusal programlar gibi nasıl formüle edileceğini bulmuşlardır ve sonra onları simpleks algoritma ile çözmüşlerdir. Sonuç olarak, doğrusal programlama uygulamaları çeşitli algoritmalarla gelişmiştir. Simpleks algoritma çeşitleri, doğrusal programlama çözümleri için en popüler yöntemler olarak kalmıştır. Bu tarih, [69] ve [197] de ki notlarda dahil olmak üzere birkaç yerde bulunmaktadır.

Elipsoid algoritması, 1979 da L. G. Khachian a göre doğrusal programlama için ilk polinomial zamanlı algoritmadır; temeli önceki çalışmalardan N. Z. Shor, D. B. Judin ve A. S. Nemirovskii ye dayanmaktadır. Grötschel, Lovasz ve Schrijver [154] kombinatoriyal optimizasyonda çeşitli problem çözümlerinde elipsoid algoritmasının nasıl kullanılacağını tanımlamışlardır. Şimdiye kadar, uygulamada elipsoid algoritması, simpleks algoritma ile rakip görülmemiştir.

Karmarkar'ın makalesi [198] ilk interior-point (iç-nokta) algoritmasının tanımlamasını içerir. Birçok sonradan gelen araştırmacı iç-nokta algoritmaları tasarlamışlardır. Goldfarb ve Todd'un makalesi [141] ve Ye [361] kitabında güzel incelemeler vardır.

Simpleks algoritma analizi aktif bir araştırma alanı olarak kalmıştır. V. Klee ve G.J. Minty simpleks algoritmanın 2^n-1 iterasyon süresince çalıştığı bir örnek yapılandırmışlardır. Simpleks algoritma çoğu zaman uygulamada çok iyi çalışır ve birçok araştırmacı bu deneysel araştırmalar için teorik kanıtlar vermeye çalışmışlardır., K. H. Borgwardt ile başlayan ve diğerleri ile devam eden bir dizi araştırma gösteriyor ki girdilerdeki belli olasılıksal varsayımlar altında simpleks algoritma istenilen polinomial zamanda sonlanır. Spielman ve Teng [322] "algoritmaların yumuşatılmış (smoothed) analizi" ni tanımlayıp simpleks algoritmaya uygulayarak bu alanda ilerleme sağlamışlardır.

Belli özel durumlarda simpleks algoritmanın verimli olduğu bilinir. Özellikle dikkate değer olan ağ-simpleks (network-simplex) algoritmasıdır öyle ki; bu algoritma ağ- akış problemine özel bir simpleks algoritmadır. En-kısa yol, maksimum-akış ve minimum-maliyet akışı problemlerini de içeren belli ağ problemleri için ağ-simpleks algoritma çeşitleri polinomial zamanda sonlanır. Örneğin, Orlin [268] ve buradaki atıflara bakabilirsiniz.

Derecesi n olan iki polinomun toplanması için bilinen basit yöntemde $\Theta(n)$ süre alır, ama bunları çarpmak için bilinen basit yöntem $\Theta(n^2)$ süreye ihtiyaç duyar. Bu bölümde hızlı Fourier dönüşümü (fast Fourier transform) veya FFT ile polinomları çarpmanın süresinin $\Theta(n \lg n)$ e nasıl düşürüleceğini göstereceğiz.

Fourier dönüşümlerinin ve dolayısıyla FFT nin en yaygın kullanımı sinyal işlemedir. Zamandan genliğe fonksiyon eşlemesi ile belirtilen *zaman alanında* sinyal verilir. Fourier analizi sinyallerin değişen frekansların durumu ötelenmiş sinüs eğrilerinin ağırlıklı toplamı olarak ifade etmemize olanak sağlar. *Etki alanı sıklığında*, frekanslar ile özdeşleştirilmiş ağırlıklar ve durumlar sinyali karakterize eder. FFT nin bazı günlük uygulamalardaki kullanımları MP3 dosyaları dahil olmak üzere sayısal video ve ses dosyalarının kodlanmasındaki sıkıştırma teknikleridir. Birkaç iyi kitap sinyal işleme alanını derinlemesine araştırmıştır; bunlara bölüm sonundaki notlarda yer verilecektir.

Polinomlar

Polinom, F cebirsel cismi üzerindeki x değişkenine göre $A(x)$ fonksiyonunun gösterimi toplam biçimindedir:

$$A(x) = \sum_{j=0}^{n-1} a_j x^j .$$

$a_0, a_1, \dots, a_{n-1}$ değerlerini polinomun **katsayılar (coefficients)** ı olarak adlandırırız. Katsayılar F cisminde yer alır, genelde karmaşık sayılar kümesi C alınır. En büyük indisli a_k katsayısı 0 dan farklı ise $A(x)$ polinomunun **derecesi** k olur ve $\text{derece}(A) = k$ ile gösterilir. Polinomun derecesinden daha büyük bir tam sayı o polinomun **derece-sınırıdır (degree-bound)**. Bu nedenle, derece-sınırı n olan bir polinomun derecesi 0 ve $n-1$ aralığı içindeki bir tam sayıdır.

Polinomlar üzerinde çeşitli işlemler tanımlayabiliriz. **Polinom toplaması**, $A(x)$ ve $B(x)$ derece-sınırı n olan polinomlar ise cisimdeki her bir x için onların

toplami, derece-sınırı n olan $C(x)$ polinomudur, $C(x) = A(x) + B(x)$ Bu, $j = 0, 1, \dots, n-1$ olmak üzere $c_j = a_j + b_j$ olarak hesaplanır.

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

ve

$$B(x) = \sum_{j=0}^{n-1} b_j x^j ,$$

sonra

$$C(x) = \sum_{j=0}^{n-1} c_j x^j$$

olur.

Örneğin, $A(x) = 6x^3 + 7x^2 - 10x + 9$ ve $B(x) = -2x^3 + 4x - 5$ polinomları varsa, $C(x) = 4x^3 + 7x^2 - 6x + 4$ dir.

Polinom çarpımı, $A(x)$ ve $B(x)$ derece-sınırı n olan polinomlar ise onların **çarpımı** $C(x)$ polinomunun derece-sınırı $2n-1$ olur; öyle ki, cisimdeki her bir x için $C(x) = A(x)B(x)$ dir. Muhtemel olarak daha önce polinomların çarpımını $A(x)$ deki her bir terimi, $B(x)$ deki her terim ile çarparak ve sonra bunları eşit kuvvetler ile birbirine eşleyerek yapmışsınızdır. Örneğin, $A(x) = 6x^3 + 7x^2 - 10x + 9$ ve $B(x) = -2x^3 + 4x - 5$ polinomlarını şu şekilde çarpabiliriz:

$$\begin{array}{r} 6x^3 + 7x^2 - 10x + 9 \\ - 2x^3 \qquad \qquad + 4x - 5 \\ \hline - 30x^3 - 35x^2 + 50x - 45 \\ 24x^4 + 28x^3 - 40x^2 + 36x \\ - 12x^6 - 14x^5 + 20x^4 - 18x^3 \\ \hline - 12x^6 - 14x^5 + 44x^4 - 20x^3 - 75x^2 + 86x - 45 \end{array}$$

$C(x)$ çarpımını başka bir şekilde açıklama yöntemi:

$$C(x) = \sum_{j=0}^{2n-2} c_j x^j , \quad (30.1)$$

burada

$$c_j = \sum_{k=0}^j a_k b_{j-k} . \quad (30.2)$$

$derece(C) = derece(A) + derece(B)$ olduğu not edilmelidir ve bu A polinomunun derece-sınırı n_a ve B polinomunun derece-sınırı n_b ise, C polinomunun derece-sınırı $n_a + n_b - 1$ olduğunu anlatmaktadır. derece-sınırı k olan bir polinom aynı zamanda derece-sınırı $k + 1$ olan bir polinom olduğundan, çarpım polinomu C nin derece-sınırının $n_a + n_b$ olduğunu doğal olarak söyleyebiliriz.

Bölüm Taslağı

Kısım 30.1 polinomların temsili için iki yöntem sunmaktadır: katsayılar temsili ve nokta-değer temsili. Polinomların çarpımı için bilinen basit yöntem – (30.1) ve (30.2) denklemleri, polinomu katsayılar biçiminde ifade edersek, $\Theta(n^2)$ sürede, ancak nokta-değer temsili ile sadece $\Theta(n)$ sürede gerçekleşir. Bununla beraber, bu iki temsil arasında geçiş yaparak katsayılar temsilini kullanarak polinomların çarpımını $\Theta(n \lg n)$ sürede yapabiliriz. Bu yaklaşımın neden işlediğini anlamak için öncelikle Kısım 30.2 de anlatılan birimin karmaşık köklerini çalışmalıyız. Sonra, yine bölüm 30.2 de tanımlanan FFT ve bunun tersini dönüşümleri gerçekleştirmek için kullanırız. Kısım 30.3 de FFT nin seri ve paralel modellerde hızlı çalışacak şekilde nasıl uygulandığını gösterilmektedir.

Bu bölümde genellikle karmaşık sayılar kullanılmaktadır ve bölüm boyunca i sembolü $\sqrt{-1}$ i ifade etmektedir.

30.1 Polinomların gösterimi

Polinomların katsayılar ve nokta-değer temsilleri bir anlamda denktir, şöyle ki; nokta-değer formundaki bir polinomun katsayılar formunda tek bir karşılığı vardır. Bu bölümde, iki temsili tanıtaacağız ve derece-sınırı n olan iki polinomun $\Theta(n \lg n)$ sürede çarpabilmek için bunları nasıl ilişkilendireceğimizi göstereceğiz.

Katsayılar gösterimi

Derece-sınırı n olan $A(x) = \sum_{j=0}^{n-1} a_j x^j$ polinomunun **katsayılar temsili** $a = (a_0, a_1, \dots, a_{n-1})$ katsayılar vektörüdür. Bu bölümde, matris eşitliklerinde vektörleri genellikle sütun vektörü olarak düşünmeliyiz.

Katsayılar temsili polinomlar üzerindeki belli işlemler için uygundur. Örneğin, verilen bir x_0 noktasında $A(x)$ polinomunun **değerlendirilmesi** $A(x_0)$ değerinin hesaplanmasını teşkil eder. **Horner kuralı** ile bir polinomu $\Theta(n)$ zamanda hesaplayabiliriz:

$$A(x_0) = a_0 + x_0(a_1 + x_0(a_2 + \dots + x_0(a_{n-2} + x_0(a_{n-1}))) \dots) .$$

Benzer şekilde, katsayılar vektörleri ile temsil edilen polinomların toplamı $a = (a_0, a_1, \dots, a_{n-1})$ ve $b = (b_0, b_1, \dots, b_{n-1})$ için $\Theta(n)$ sürede hesaplanır: $j = 0, 1, \dots, n-1$ için $c_j = a_j + b_j$ olacak şekilde $c = (c_0, c_1, \dots, c_{n-1})$ katsayı vektörünü oluştururuz.

Şimdi derece-sınırı n olan iki polinomun $A(x)$ ve $B(x)$ katsayılar formundaki çarpımını ele alalım. (30.1) ve (30.2) nolu denklemlerde verilen yöntemi kullanırsak, polinomların çarpımı a vektöründeki her katsayıyı b vektöründeki her katsayı ile çarpmak zorunda olduğumuzdan $\Theta(n^2)$ süre alır. Katsayılar formundaki iki polinomun çarpımı, polinomun istenilen değerinin hesaplanmasından veya bunların toplanmasından oldukça zor gözükmemektedir. (30.2) nolu denklemdeki c katsayılar sonuç vektörü a ve b girdi vektörlerinin **evrişimi** (*convolution*) olarak isimlendirilir ve $c = a \otimes b$ ile gösterilir. Polinomların çarpımı ve girişim vektörlerinin hesaplanması uygulama açısından oldukça önemli temel hesaplama problemleri olduğundan, bu bölüm onlar için verimli algoritmalar üzerinde yoğunlaşmaktadır.

Nokta-değer gösterimi

Derece-sınırı n olan $A(x)$ polinomunun **nokta-değer temsili** n tane **nokta-değer ikilisinin** oluşturduğu bir kümedir

$$\{(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})\}$$

öyle ki; $k = 0, 1, \dots, n-1$ için ve x_k lerin herbiri birbirinden farklı olmak üzere

$$y_k = A(x_k). \quad (30.3)$$

$x_0, x_1, \dots, x_{n-1}$ gibi birbirinden farklı n noktayı temsil için baz olarak kullanabildiğimizden ötürü bir polinomun çok sayıda farklı nokta-değer temsili vardır.

Birbirinden farklı n noktayı $x_0, x_1, \dots, x_{n-1}$ seçmek ve bunları $k = 0, 1, \dots, n-1$ için $A(x_k)$ de değerlendirmek durumunda olduğumuzdan katsayılar formunda verilen bir polinom için nokta-değer temsili hesaplamak oldukça açıktır. Horner yöntemi ile n noktada bir polinomu değerlendirmek $\Theta(n^2)$ süre alır. İlerleyen bölümlerde göreceğiz ki, x_k noktalarını akıllıca seçersek bu hesaplamanın çalışma zamanını $\Theta(n \lg n)$ olarak hızlandırabiliriz.

Değerlendirme işleminin tersi başka bir deyişle, nokta-değer temsiliinden polinomun katsayılarını belirleme işlemi **interpolasyondur**. Sıradaki teorem istenilen polinomun derece-sınırı ile verilen nokta-değer ikililerinin sayısının aynı olduğu durumda interpolasyonun iyi tanımlı olduğunu göstermektedir.

Teorem 30.1 (İnterpolasyon polinomunun tekliği)

n nokta-değer ikilisinin herhangi bir $\{(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})\}$ kümesinde birbirinde farklı x_k değerleri için derece-sınırı n olan tek türlü bir $A(x)$ polinomu vardır öyle ki $k = 0, 1, \dots, n-1$ için $y_k = A(x_k)$.

Kanıt Kanıt belirli matrisin tersinin varlığına dayanmaktadır. Denklem (30.3) aşağıdaki matrise denktir.

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix}. \quad (30.4)$$

Soldaki matris $V(x_0, x_1, \dots, x_{n-1})$ ile gösterilir ve Vander-monde matrisi olarak adlandırılır. D-1 problemi gereği, bu matrisin determinanı

$$\prod_{0 \leq j < k \leq n-1} (x_k - x_j)$$

ve bundan dolayı, Teorem D.5, gereği, x_k ler birbirinden farklı ise bu matris tersinirdir (tekil olmayan). Bu yüzden, a_j katsayıları için tek türlü olarak verilen nokta-değer temsili çözebiliriz:

$$a = V(x_0, x_1, \dots, x_{n-1})^{-1} y. \quad \blacksquare$$

Teorem 30.1 in kanıtı doğrusal denklemlerin çözümüne dayanan (30.4) kümesinin interpolasyon için olan algoritmasını tanımlamaktadır. Bölüm 28 in LU ayrışım algoritmalarını kullanarak, bu denklemleri $O(n^3)$ zamanda çözebiliriz.

n -nokta interpolasyonu için daha hızlı bir algoritma **Lagrange formülüne** dayanmaktadır:

$$A(x) = \sum_{k=0}^{n-1} y_k \frac{\prod_{j \neq k} (x - x_j)}{\prod_{j \neq k} (x_k - x_j)}. \quad (30.5)$$

Denklem (30.5) in sağ tarafının derece-sınırı n olan her k için $A(x_k) = y_k$ yi sağlayan polinomu doğrulamak isteyebilirsiniz. Alıştırma 30.1-5 Lagrange formülünü kullanarak A nın katsayılarını $\Theta(n^2)$ sürede nasıl hesaplanacağını sormaktadır.

Böylece, n -nokta değerlendirmesi ve interpolasyon, polinomun katsayılar temsili ve nokta-değer temsili arasındaki dönüşümü sağlayan iyi tanımlı ters işlemlerdir.¹ Yukarıda bu problemlerin çözümü için tanımlanan algoritmalar $\Theta(n^2)$ süre içinde yanıt verir.

Nokta-değer temsili, polinomlar üzerindeki birçok işlem için oldukça uygundur. Buna ek olarak, $C(x) = A(x) + B(x)$ ise herhangi bir x_k için $C(x_k) = A(x_k) + B(x_k)$ dir. Daha açık bir ifadeyle, Eğer A için nokta-değer temsili

¹İnterpolasyon, sayısal denge bakış açısından aldatıcı bir problemdir. Burada verilen yaklaşım, matematiksel açıdan doğru olmasına rağmen, girdilerdeki küçük değişimler veya hesaplamalar esnasındaki yuvarlama hataları sonuçta büyük farklara neden olabilir.

$$\{(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})\}$$

dir ve B için

$$\{(x_0, y'_0), (x_1, y'_1), \dots, (x_{n-1}, y'_{n-1})\}$$

varsa (dikkat edilmelidir ki A ve B aynı n noktada değerlendirilmiştir), C için nokta-değer temsili

$$\{(x_0, y_0 + y'_0), (x_1, y_1 + y'_1), \dots, (x_{n-1}, y_{n-1} + y'_{n-1})\}$$

böylece, nokta-değer biçimindeki derece-sınırı n olan iki polinomun toplanması $\Theta(n)$ sürede gerçekleşir.

Benzer şekilde, nokta-değer temsili polinomların çarpımı için oldukça uygundur. $C(x) = A(x)B(x)$ ise herhangi bir x_k noktası için $C(x_k) = A(x_k)B(x_k)$ ve C nin nokta-değer temsilini elde etmek için A nin nokta-değer temsilini, B nin nokta-değer temsili ile nokta tabanlı çarpıyoruz. Bununla birlikte, $derece(C) = derece(A) + derece(B)$ problemi ile karşı karşıya geliriz; A ve B derece-sınırı n ise C nin derece-sınırı $2n$ dir. A ve B nin standart nokta-değer temsilleri her polinom için n tane nokta-değer ikilisini içerir. Bunları birbiri ile çarparken, n tane nokta-değer ikilisi elde ederiz; ancak derece-sınırı $2n$ olan tek ve eşsiz C polinomunu interpolate etmek için $2n$ tane ikiliye ihtiyaç duyarız (Bakınız Alıştırma 30.1-4). Bundan dolayı, $2n$ tane nokta-değer ikilisini içeren A ve B nin nokta-değer temsillerinin genişletilmiş versiyonu ile başlamak durumundayız. Verilen bir A için genişletilmiş nokta-değer temsili

$$\{(x_0, y_0), (x_1, y_1), \dots, (x_{2n-1}, y_{2n-1})\} ,$$

ve B için genişletilmiş nokta-değer temsili

$$\{(x_0, y'_0), (x_1, y'_1), \dots, (x_{2n-1}, y'_{2n-1})\} ,$$

sonra, C nin nokta-değer temsili

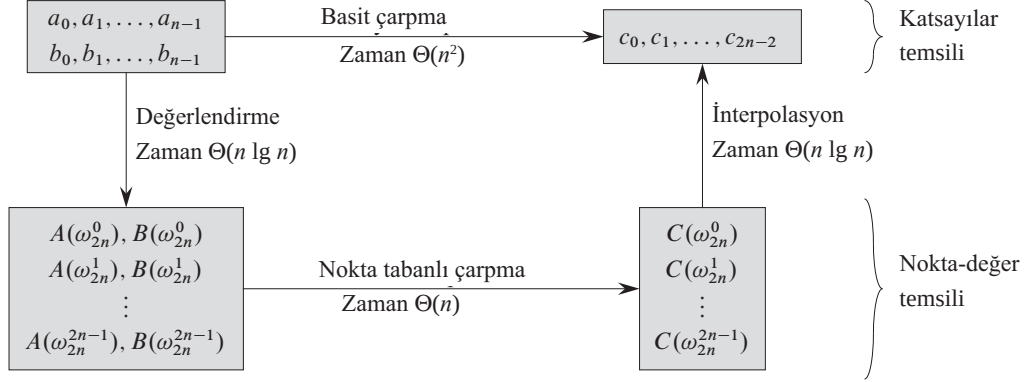
$$\{(x_0, y_0 y'_0), (x_1, y_1 y'_1), \dots, (x_{2n-1}, y_{2n-1} y'_{2n-1})\} .$$

Genişletilmiş nokta-değer biçiminde verilen iki girdi polinomunu çarpmak için ihtiyaç duyulan süre nokta-değer biçiminde $\Theta(n)$ olmaktadır ve bu değer katsayılar biçimindeki çarpmadan çok daha azdır.

Sonuç olarak, nokta-değer biçiminde verilen bir polinomu yeni verilen bir noktada nasıl değerlendirilmesini hesaba katıyoruz. Bu problem için polinomu katsayılar biçimine dönüştürüp ve sonra orada yeni noktayı değerlendirmekten başka daha kolay bir yöntem olmadığını biliyoruz.

Katsayılar biçimindeki polinomların hızlı çarpımı

Nokta-değer biçimindeki polinomları katsayılar biçimindeki polinom çarpımını hızlandıracak şekilde doğrusal zamanlı çarpma kullanabilir miyiz? Burada-



Şekil 30.1 Verimli polinom çarpma işlem sürecinin şekilsel gösterimi. Yukarıdaki gösterimler katsayılar biçiminde iken onların altındakiler nokta-değer biçimindedir. Sağdan sola doğru olan oklar çarpma işlemine karşılık gelmektedir. ω_{2n} terimleri birimlerinin $2n$. kökleridir.

ki cevabın dayanak noktası, bir polinomu hızlı bir şekilde katsayılar formundan nokta-değer biçimine (değerlendirme) ve tersi de (interpolasyon) olacak şekilde dönüştürüp dönüştüremeyeceğimizdir.

İstenilen noktaları değerlendirme noktaları olarak kullanabiliriz; ancak değerlendirme noktalarının dikkatli bir şekilde seçilmesiyle, temsiller arasındaki geçişi sadece $\Theta(n \lg n)$ sürede yapabiliriz. Kısım 30.2 de göreceğimiz gibi, “birimlerin karmaşık kökleri”ni değerlendirme noktaları olarak seçersek, katsayılar vektörünün ayrık Fourier dönüşümünü (veya DFT) alarak nokta-değer temsili üretebiliriz. Ters alma işlemini katsayılar vektörünü sağlayan nokta-değer ikililerinin “ters DFT”ini alarak gerçekleştirebiliriz. Kısım 30.2 de FFT nin DFT ve ters DFT işlemlerinin üstesinden $\Theta(n \lg n)$ sürede nasıl geldiğini anlatmaktadır.

Şekil 30.1 bu mantığı grafiksel olarak göstermektedir. Önemsiz bir detay derece-sınırları ile ilgilidir. Derece-sınırı n olan iki polinomun çarpımı derece-sınırı $2n$ olan bir polinom üretir. Bundan dolayı, A ve B girdi polinomlarını değerlendirmeden önce n tane yüksek dereceli katsayıları 0 olan elemanları ekleyerek, bunların derece-sınırlarını iki katına $2n$ e çıkartırız. Vektörler $2n$ tane elemana sahip olduğundan Şekil 30.1 de verilen ω_{2n} terimler ile gösterilen “birimin $2n$. köklerini ($2n$ th roots of unity) kullanırız.

Verilen FFT için, derece-sınırı n , girdi ve çıktı gösterimleri katsayılar biçiminde olan $A(x)$ ve $B(x)$ polinomlarının çarpma işlemini aşağıdaki yöntem ile $\Theta(n \lg n)$ sürede yaparız. n nin 2 nin bir kuvveti olduğunu varsayalım; bu durumu derece-sınırını 2 nin bir kuvveti olacak şekilde katsayıları sıfır olan yüksek dereceli terimlerin eklenmesi ile bu şartı her zaman sağlayabiliriz.

1. *Derece-sınırını iki katına çıkarmak:* derece-sınırı $2n$ olacak şekilde $A(x)$ ve $B(x)$ nin katsayıları sıfır olan yüksek dereceli terimlerin eklenmesi ile katsayılar temsiliinin oluşturulması.

2. *Değerlendirme*: Uzunluğu $2n$ olan $A(x)$ ve $B(x)$ in nokta-değer temsillerini mertebesi $2n$ olan her bir polinomun üzerinde uygulayarak hesaplanır. Bu gösterim $2n$. birim köklerinde bu iki polinomun değerlerini içerir.
3. *Nokta tabanlı çarpma*: $C(x) = A(x)B(x)$ in nokta-değer temsilini bunların nokta tabanlı çarpılmasıyla hesaplayın. Bu gösterim $C(x)$ 'in $2n$. birim köklerinin-deki değerini içerir.
4. *İnterpolasyon*: $C(x)$ polinomunun katsayılar temsilini ters DFT yi hesaplamak için FFT yi $2n$ nokta-değer çiftlerinde uygulayarak hesaplayın.

(1) ve (3) nolu adımlar $\Theta(n)$, (2) ve (4) nolu adımlar $\Theta(n \lg n)$ süre alırlar. Böylece, FFT nin nasıl kullanıldığını gösterdiğimiz zaman, aşağıdaki teoremi kanıtlamış olacağız.

Teorem 30.2

Girdi ve çıktı temsilleri katsayılar biçiminde ve derece-sınırı n olan iki polinomu $\Theta(n \lg n)$ süre içerisinde çarpabiliriz. ■

Alıştırmalar

30.1-1

Denklem (30.1) ve (30.2) yi kullanarak $A(x) = 7x^3 - x^2 + x - 10$ ve $B(x) = 8x^3 - 6x + 3$ yi çarpınız.

30.1-2

Verilen bir x_0 noktasında derece-sınırı n olan $A(x)$ polinomunu değerlendirmek için bir diğer yöntem, kalanı r ve bölüm polinomunun derece-sınırı $n - 1$ olan $q(x)$ için $A(x)$ i $(x - x_0)$ polinomuna bölmektir, öyle ki;

$$A(x) = q(x)(x - x_0) + r.$$

Açıkça, $A(x_0) = r$. $\Theta(n)$ zamanda x_0 ve A nın katsayılarını kullanarak r kalanını ve $q(x)$ in katsayılarının nasıl hesaplanacağını gösteriniz.

30.1-3

$A^{\text{rev}}(x) = \sum_{j=0}^{n-1} a_{n-1-j} x^j$ için nokta-değer temsilini $A(x) = \sum_{j=0}^{n-1} a_j x^j$ ün nokta-değer temsili ile noktaların 0 dan farklı olduğunu varsayarak türetiniz.

30.1-4

Birbirinden farklı n tane nokta-değer ikilisinin, derece-sınırı n olan polinomun tek türlü belirtilmesi gerekliliğini kanıtlayınız; başka bir deyişle; n taneden daha az birbirinden farklı nokta-değer ikilisi verilirse, derece-sınırı n olan tek türlü polinom belirlenemez (*İpucu*: Teorem 30.1'i kullanarak, $n-1$ tane nokta-değer ikilisi kümesine rassal seçilen nokta-değer ikilisi eklendiğinde ne söyleyebilirsiniz?).

30.1-5

Denklem (30.5) i denklemi kullanarak $\Theta(n^2)$ sürede interpolate edildiğini gösteriniz. (*İpucu:* Alıştırma 30.1-2 ye bakınız. Öncelikle $\prod_j (x - x_j)$ polinomunun katsayılar temsilini hesaplayın ve sonra $(x - x_k)$ ye her terimin payını bölünüz. n paydanın her birini $O(n)$ sürede hesaplayabilirsiniz.)

30.1-6

Polinom bölümünde “açık” yaklaşımın nokta-değer temsili kullanımındaki sınırların başka bir deyişle karşılık gelen y değerlerinin bölümünün ne olduğunu açıklayınız. Bölmenin tam olarak sonuçlandığı ve sonuçlanmadığı durumları ayrı bir şekilde tartışınız.

30.1-7

Her biri 0 dan $10n$ e kadar n tane tam sayı içeren A ve B kümelerini göz önüne alalım. A ve B nin **Kartezyen toplamı** şu şekilde hesaplarız:

$$C = \{x + y : x \in A \text{ ve } y \in B\}.$$

C deki tam sayıların aralığı 0 ile $20n$ arasındadır. C nin elemanlarını ve bulmak istiyoruz ve C nin her elemanının tekrarlanma sayısının A ve B nin elemanlarının toplamı şeklinde kavramaktır. Bu problemin $O(n \lg n)$ sürede nasıl çözüleceğini gösteriniz (*İpucu:* A ve B yi derecesi en fazla $10n$ olan polinomlar gibi göstermek).

30.2 DFT ve FFT

Kısım 30.1 de birimin karmaşık köklerini kullanırsak, $\Theta(n \lg n)$ sürede polinomu değerlendirebilme ve interpolate edebilmemizi iddia etmiştik. Bu bölümde, birimin karmaşık köklerini tanıttık ve onların özelliklerini çalışacağız, DFT yi tanımladıktan sonra FFT nin DFT ve onun tersini $\Theta(n \lg n)$ sürede nasıl hesaplanacağını göstereceğiz.

Birimin karmaşık kökleri

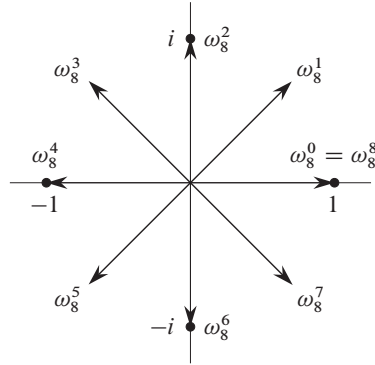
Birimin n inci karmaşık kökü karmaşık bir sayıdır w öyle ki;

$$w^n = 1.$$

Birimin n . dereceden karmaşık kök sayısı tam olarak n tane dir: $k = 0, 1, \dots, n - 1$ için $e^{2\pi i k/n}$. Bu formülü yorumlamak için karmaşık sayıların üssünün tanımını veririz:

$$e^{iu} = \cos(u) + i \sin(u).$$

Şekil 30.2 birimin n karmaşık kökünün karmaşık düzlemde, orijin merkezli, birim yarıçaplı daire etrafında eşit aralıklı olduğunu göstermektedir. Bu değer



Şekil 30.2 $\omega_8^0, \omega_8^1, \dots, \omega_8^7$ 8. temel birim kökü olmak üzere karmaşık düzlemdeki $\omega_8 = e^{2\pi i/8}$ değerler.

$$\omega_n = e^{2\pi i/n} \quad (30.6)$$

birimin n . esas (principal) kökü olan bu değer², birimin n . dereceden tüm karmaşık kökleri ω^n nin kuvvetidir.

Birimin n . dereceden n tane karmaşık kökü

$$\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}$$

çarpma işlemi altında bir grup oluşturur (Kısım 31.3 e bakınız). $\omega_n^n = \omega_n^0 = 1$ $\omega_n^j \omega_n^k = \omega_n^{j+k} = \omega_n^{(j+k) \bmod n}$ ’i gerektirdiğinden bu grup $(\mathbb{Z}_n, +)$ modülo n grubuyla aynı yapıya sahiptir. Benzer olarak, $\omega_n^{-1} = \omega_n^{n-1}$. İzleyen lemmalar (önergeler) birimin n . dereceden karmaşık kökünün gerekli özelliklerinden bazılarını sunmaktadır.

Lemma 30.3 (İptal lemması)

Herhangi $n \geq 0$, $k \geq 0$ ve $d > 0$ tam sayıları için

$$\omega_{dn}^{dk} = \omega_n^k. \quad (30.7)$$

Kanıt

$$\begin{aligned} \omega_{dn}^{dk} &= (e^{2\pi i/dn})^{dk} \\ &= (e^{2\pi i/n})^k \\ &= \omega_n^k \end{aligned}$$

olduğundan, lemma denklem (30.6) dan doğrudan ortaya çıkmaktadır. ■

²Birçok yazar ω^n yi: $\omega_n = e^{-2\pi i/n}$ şeklinde tanımlar Bu alternatif tanıma yönelmenin sebebi sinyal işleme uygulamalarındaki kullanımdır. Temeldeki matematik ω^n tanımında da büyük ölçüde aynıdır.

Sonuç 30.4

$n > 0$ çift tam sayısı için

$$\omega_n^{n/2} = \omega_2 = -1 .$$

Kanıt Alıştırma 30.2-1 de kanıt okuyucuya bırakılmıştır. ■

Lemma 30.5 (Yarıya indirme (Halving) lemması)

$n > 0$ çift tam sayı ise, birimin n . dereceden karmaşık n kökünün kareleri $n = 2$ iken birimin $(n/2)$. dereceden karmaşık $n/2$ kökleridir.

Kanıt İptal lemması ile negatif olmayan herhangi k için $(\omega_n^k)^2 = \omega_{n/2}^k$ vardır. Dikkat edilmelidir ki; birimin n . dereceden karmaşık köklerinin hepsinin karesini alırsak, aşağıdakilerden ötürü, birimin $(n/2)$. dereceden her bir kökünün tam iki defa elde edilir.

$$\begin{aligned} (\omega_n^{k+n/2})^2 &= \omega_n^{2k+n} \\ &= \omega_n^{2k} \omega_n^n \\ &= \omega_n^{2k} \\ &= (\omega_n^k)^2 . \end{aligned}$$

Böylece, ω_n^k ve $\omega_n^{k+n/2}$ aynı kareye sahip olur. $\omega_n^{n/2} = -1$, $\omega_n^{k+n/2} = -\omega_n^k$ işaret ettiğinden, Sonuç 30.4 ü bu özelliği kanıtlamak için kullanabiliriz; bu yüzden $(\omega_n^{k+n/2})^2 = (\omega_n^k)^2$. ■

Anladığımız üzere, yarıya indirme lemması özyinelemeli alt problemleri yarı genişliğe indirmeyi sağladığından, polinomların katsayı ve nokta-değer temsilleri arasındaki dönüşümde böl-ve-yönet yaklaşımı için gereklidir.

Lemma 30.6 (Toplam lemması)

Herhangi $n \geq 1$ tam sayısı ve n tarafından bölünmeyen, sıfırdan farklı k tam sayısı için

$$\sum_{j=0}^{n-1} (\omega_n^k)^j = 0 .$$

Kanıt Denklem (A.5) gerçel sayılara uygulandığı gibi karmaşık değerlere de uygulanır ve böylece şunu elde ederiz

$$\begin{aligned}
\sum_{j=0}^{n-1} (\omega_n^k)^j &= \frac{(\omega_n^k)^n - 1}{\omega_n^k - 1} \\
&= \frac{(\omega_n^n)^k - 1}{\omega_n^k - 1} \\
&= \frac{(1)^k - 1}{\omega_n^k - 1} \\
&= 0 .
\end{aligned}$$

Paydanın 0 olmadığını, k nin n tarafından bölünmediğini ve $\omega_n^k = 1$ sadece k, n tarafından bölündüğünden ötürü söyleriz. ■

DFT

Hatırlatmak gerekir ki; $\omega_n^0, \omega_n^1, \omega_n^2, \dots, \omega_n^{n-1}$ de derece-sınırı n olan polinomda değerlendirilmek istiyoruz.

$$A(x) = \sum_{j=0}^{n-1} a_j x^j$$

(Şöyle ki; birimin n .dereceden karmaşık n kökünde)³. Varsayalım ki; A katsayılar formunda verilsin: $a = (a_0, a_1, \dots, a_{n-1})$. $k = 0, 1, \dots, n-1$ için y_k sonuçlarını aşağıda verilenler ile tanımlayalım

$$\begin{aligned}
y_k &= A(\omega_n^k) \\
&= \sum_{j=0}^{n-1} a_j \omega_n^{kj} .
\end{aligned} \tag{30.8}$$

$y = (y_0, y_1, \dots, y_{n-1})$ vektörü, $a = (a_0, a_1, \dots, a_{n-1})$ katsayılar vektörünün **ayrık Fourier dönüşümüdür (DFT)**. Aynı zamanda $y = \text{DFT}_n(a)$ ile yazabiliriz.

FFT

Hızlı Fourier dönüşümü (FFT) olarak bilinen, birimin karmaşık köklerinin ayrıcalıklı özelliklere sahip yöntemi kullanarak, $\text{DFT}_n(a)$ yı $\Theta(n \lg n)$ sürede sonuç veren basit yöntemin aksine $\Theta(n^2)$ sürede hesaplayabiliriz. Başından sonuna ka-

³Değerlendirilmeden önce verilen polinomun derece-sınırını ikiye katladığımızdan, Kısım 30.1 de uzunluğu n olan için $2n$ e çıkartıyorduk. Bundan dolayı, polinom çarpımı kapsamında, birimin $(2n)$. dereceden karmaşık kökleri ile çalışmaktayız.

dar n yi 2 nin tam kuvveti olarak varsayıyoruz. 2 nin kuvveti olmayan durumlar için düşünceler olmasına rağmen, onlar bu kitabın kapsamı dışındadır.

FFT yöntemi böl-ve-yönet mantığını $A(x)$ polinomunun katsayılarını çift ve tek tam sayı olan göstergeler yardımıyla derece-sınırı $(n/2)$ olan yeni $A^{[0]}(x)$ ve $A^{[1]}(x)$ tanımlayarak çalıştırırız:

$$\begin{aligned} A^{[0]}(x) &= a_0 + a_2x + a_4x^2 + \cdots + a_{n-2}x^{n/2-1}, \\ A^{[1]}(x) &= a_1 + a_3x + a_5x^2 + \cdots + a_{n-1}x^{n/2-1}. \end{aligned}$$

Dikkat edilmelidir ki; $A^{[0]}(x)$, A nın çift tam sayı olan tüm gösterge katsayılarından meydana gelmektedir (göstergenin ikilik tabandaki temsili 0 da sonlanır). ve $A^{[1]}(x)$, tek tam sayı olan tüm gösterge katsayılarını içerir (göstergenin ikilik tabandaki temsili 1 de sonlanır). Sıradaki formülü izleriz

$$A(x) = A^{[0]}(x^2) + xA^{[1]}(x^2) \quad (30.9)$$

$A(x)$ i $A(x)$ at $\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}$ noktalarında değerlendirme problemi aşağıdakilere indirgenir.

1. derece-sınırı $n/2$ olan $A^{[0]}(x)$ ve $A^{[1]}(x)$ lerin

$$(\omega_n^0)^2, (\omega_n^1)^2, \dots, (\omega_n^{n-1})^2 \quad (30.10)$$

noktalarında değerlendirilmesi

2. sonuçları denklem (30.9) yardımıyla bir araya getirilmesi

Yarıya indirme lemması ile, (30.10) daki değerler listesi birbirinden farklı n değeri içermez, ancak bunların sadece $n/2$ tanesi tam iki kere tekrar ederek birimin $(n/2)$.dereceden karmaşık köküdür. Bundan dolayı, derece-sınırı $n/2$ olan $A^{[0]}$ ve $A^{[1]}$ polinomlarını özyinelemeli bir şekilde birimin $(n/2)$.dereceden $n/2$ karmaşık köklerinde değerlendiririz. Bu küçük problemler orijinal problem ile aynı formda olmakla birlikte, boyutları yarıya inmiştir. n elemanlı DFT_n hesabını iki tane $n/2$ elemanlı $DFT_{n/2}$ hesabına başarılı bir şekilde böldük. Bu dönüşüm; n , 2 nin bir kuvveti olmak üzere, n elemanlı $a = (a_0, a_1, \dots, a_{n-1})$ vektörünün DFT sinin hesaplanması, FFT algoritmasının özyinelemeli bir şekilde tekrar edilmesini temel almaktadır.

RECURSİVE-FFT(a)

```

1   $n = a.length$  //  $n$ , 2 nin bir kuvvetidir.
2  if  $n == 1$ 
3      return  $a$ 
4   $\omega_n = e^{2\pi i/n}$ 
5   $\omega = 1$ 
6   $a^{[0]} = (a_0, a_2, \dots, a_{n-2})$ 
7   $a^{[1]} = (a_1, a_3, \dots, a_{n-1})$ 
8   $y^{[0]} = \text{RECURSİVE-FFT}(a^{[0]})$ 
9   $y^{[1]} = \text{RECURSİVE-FFT}(a^{[1]})$ 
10 for  $k = 0$  to  $n/2 - 1$ 
11      $y_k = y_k^{[0]} + \omega y_k^{[1]}$ 
12      $y_{k+(n/2)} = y_k^{[0]} - \omega y_k^{[1]}$ 
13      $\omega = \omega \omega_n$ 
14 return  $y$  //  $y$  sütun vektörü olarak düşünülür.

```

RECURSİVE-FFT yöntemi şu şekilde işler. 2-3 nolu satırlar, özyinelemenin temelini gösterir; bir elemanın DFT si o elemanın kendisidir, bu durumda

$$\begin{aligned}
 y_0 &= a_0 \omega_1^0 \\
 &= a_0 \cdot 1 \\
 &= a_0 .
 \end{aligned}$$

6-7 nolu satırlar, $A^{[0]}$ ve $A^{[1]}$ polinomları için katsayılar vektörünü tanımlar. 4, 5 ve 13 nolu satırlar ω nın düzenli olarak güncellendiğini, böylece 11 ve 12 nolu satırlar çalıştırıldığında, $\omega = \omega_n^k$ elde ederiz (iterasyondan iterasyona ω nın değişen değerini saklamak, ω_n^k hesabında **for** döngüsü boyunca elemanı silmekten zaman kazanılmasını sağlar). 8-9 nolu satırlar özyinelemeli DFT _{$n/2$} hesaplamalarını $k = 0, 1, \dots, n/2 - 1$ için şu şekilde yapar,

$$\begin{aligned}
 y_k^{[0]} &= A^{[0]}(\omega_{n/2}^k) , \\
 y_k^{[1]} &= A^{[1]}(\omega_{n/2}^k) ,
 \end{aligned}$$

veya $\omega_{n/2}^k = \omega_n^{2k}$ olduğundan, iptal lemması ile,

$$\begin{aligned}
 y_k^{[0]} &= A^{[0]}(\omega_n^{2k}) , \\
 y_k^{[1]} &= A^{[1]}(\omega_n^{2k}) .
 \end{aligned}$$

11-12 nolu satırlar özyinelemeli $\text{DFT}_{n/2}$ hesaplamalarının sonuçları birbirine bağlar. $y_0, y_1, \dots, y_{n/2-1}$ için 11 nolu satır aşağıdakini verir

$$\begin{aligned} y_k &= y_k^{[0]} + \omega_n^k y_k^{[1]} \\ &= A^{[0]}(\omega_n^{2k}) + \omega_n^k A^{[1]}(\omega_n^{2k}) \\ &= A(\omega_n^k) \end{aligned} \quad (\text{denklem (30.9) ile})$$

$y_{n/2}, y_{n/2+1}, \dots, y_{n-1}$ için 12 nolu satır $k = 0, 1, \dots, n/2 - 1$ ile aşağıdakini verir

$$\begin{aligned} y_{k+(n/2)} &= y_k^{[0]} - \omega_n^k y_k^{[1]} \\ &= y_k^{[0]} + \omega_n^{k+(n/2)} y_k^{[1]} \quad (\omega_n^{k+(n/2)} = -\omega_n^k \text{ den dolayı}) \\ &= A^{[0]}(\omega_n^{2k}) + \omega_n^{k+(n/2)} A^{[1]}(\omega_n^{2k}) \\ &= A^{[0]}(\omega_n^{2k+n}) + \omega_n^{k+(n/2)} A^{[1]}(\omega_n^{2k+n}) \quad (\omega_n^{2k+n} = \omega_n^{2k} \text{ den dolayı}) \\ &= A(\omega_n^{k+(n/2)}) \quad (\text{denklem (30.9) ile}) \end{aligned}$$

Böylece, RECURSIVE-FFT ile elde edilen y vektörü a girdi vektörünün DFT sidir. $k = 0, 1, \dots, n/2 - 1$ için 11 ve 12 nolu satırlar $y_k^{[1]}$ her bir değerini ω_n^k ile çarpar. 11 nolu satır bu çarpımı $y_k^{[0]}$ e ekler ve 12 nolu satır ondan çıkarır. Her iki pozitif ve negatif biçimlerde her ω_n^k faktörünü kullandığımızdan, ω_n^k faktörlerini **döndürme faktörleri** olarak isimlendiririz.

RECURSIVE-FFT işleminin çalışma zamanını belirlemek için dikat edilmelidir ki; özyinelemeli özel çağrılarının her birinin yürütülmesi n girdi vektörünün uzunluğu olmak üzere, $\Theta(n)$ süre alır. Bundan dolayı, çalışma zamanı için iterasyon

$$\begin{aligned} T(n) &= 2T(n/2) + \Theta(n) \\ &= \Theta(n \lg n) . \end{aligned}$$

Böylece, hızlı Fourier dönüşümü kullanarak derece-sınırı n olan bir polinomu birimin n . dereceden karmaşık kökünde $\Theta(n \lg n)$ sürede değerlendirebiliriz.

Birimin karmaşık köklerinde interpolasyon

Nokta-değer biçiminden katsayılar biçimine geri dönmemize olanak sağlayan, birimin karmaşık köklerini nasıl interpolate edeceğimizi göstererek polinom çarpmayı şimdi tamamladık. DFT yi matris şeklinde yazarak interpolasyon yaparız ve sonra matrisin tersinin formunda bunları kontrol ederiz.

Denklem (30.4) den, V_n , ω_n nin uygun kuvvetlerini içeren Vandermonde matrisi olmak üzere DFT yi $y = V_n a$ matris çarpımı şeklinde yazabiliriz:

$$\begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_n & \omega_n^2 & \omega_n^3 & \cdots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \omega_n^6 & \cdots & \omega_n^{2(n-1)} \\ 1 & \omega_n^3 & \omega_n^6 & \omega_n^9 & \cdots & \omega_n^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \omega_n^{3(n-1)} & \cdots & \omega_n^{(n-1)(n-1)} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{pmatrix}.$$

$j, k = 0, 1, \dots, n-1$ için V_n nin (k, j) girdisi ω_n^{kj} dır. V_n nin girdilerinin üsleri çarpma tablosu oluşturur.

Ters işlem için $a = \text{DFT}_n^{-1}(y)$ yazarız, V_n nin tersi V_n^{-1} matrisi ile y yi çarparak devam ederiz.

Teorem 30.7

$j, k = 0, 1, \dots, n-1$ için V_n^{-1} (j, k) girdisi ω_n^{-kj}/n dir.

Kanıt $V_n^{-1}V_n = I_n$ nin $n \times n$ birim matris olduğunu gösterelim. $V_n^{-1}V_n$ nin (j, j') girdisini ele alalım:

$$\begin{aligned} [V_n^{-1}V_n]_{jj'} &= \sum_{k=0}^{n-1} (\omega_n^{-kj}/n)(\omega_n^{kj'}) \\ &= \sum_{k=0}^{n-1} \omega_n^{k(j'-j)}/n. \end{aligned}$$

$j' = j$ ise bu toplam 1 e eşittir ve aksi halde toplam lemmasından dolayı 0 dır. Dikkat edilmelidir ki; $-(n-1) \leq j' - j \leq n-1$ kullanmaktayız, böylece, toplam lemmasını uygulayabilmek için $j' - j$, n ile bölünememektedir. ■

Verilen V_n^{-1} ters matrisi ile $j = 0, 1, \dots, n-1$ için $\text{DFT}_n^{-1}(y)$ şu şekilde ifade edilir:

$$a_j = \frac{1}{n} \sum_{k=0}^{n-1} y_k \omega_n^{-kj} \quad (30.11)$$

(30.8) ve (30.11) nolu denklemleri karşılaştırdığımızda, FFT algoritmasını a ve y nin rollerini, ω_n yi ω_n^{-1} ile yer değiştirerek ve elde edilen sonucun her bir elemanını n ile bölecek şekilde modifiye edersek, ters DFT nin hesaplanacağını görürüz (Aıştırma 30.2-4 e bakınız). Böylece, DFT_n^{-1} yi de $\Theta(n \lg n)$ sürede hesaplayabiliriz.

FFT ve ters FFT yi kullanarak, derece-sınırı n olan bir polinomun $\Theta(n \lg n)$ sürede katsayılar temsili ve nokta-değer temsili arasında gidiş-gelişi sağlayan dönüşümü yapabileceğimizi gösterdik. Polinom çarpımı bağlamında, aşağıdakini göstermiş olduk.

Teorem 30.8 (Evrişim (Convolution) teoremi)

$n, 2$ nin bir kuvveti olmak üzere n uzunluğundaki iki a ve b vektörü için

$$a \otimes b = \text{DFT}_{2n}^{-1}(\text{DFT}_{2n}(a) \cdot \text{DFT}_{2n}(b)),$$

Burada a ve b vektörlerinin uzunluğu $2n$ olacak şekilde 0 lar eklenir ve $\cdot$ iki tane $2n$ elemanlı vektörlerin eleman tabanlı çarpma işlemini göstermektedir. ■

Alıştırmalar**30.2-1**

Sonuç 30.4 ü kanıtlayınız.

30.2-2

$(0,1,2,3)$ vektörünün DFT sini hesaplayınız.

30.2-3

Alıştırma 30.1-1 i $\Theta(n \lg n)$ sürede çalışan yöntem ile çözünüz.

30.2-4

DFT_n^{-1} i $\Theta(n \lg n)$ sürede çözecek algoritmanın kaba kodunu yazınız.

30.2-5

$n, 3$ ün kuvveti olacak şekilde FFT işleminin genelleştirilmesini tanımlayınız. Çalışma zamanı için özyineleme veriniz ve bu özyinelemeyi çözünüz.

30.2-6 *

Varsayalım ki, karmaşık sayılar cisimi üzerinde n elemanlı FFT nin uygulanması yerine, tam sayıların modülo m e göre değerlendirildiği $\mathbb{Z}_m$ halkasını kullanınız, burada $m = 2^{tn/2} + 1$ ve t herhangi bir pozitif tam sayıdır. modülo m e göre birimin n .dereceden ana kökü olarak ω_n yerine $\omega = 2^t$ yi kullanalım. Bu sistemde, DFT ve ters DFT'nin iyi tanımlı olduğunu kanıtlayınız.

30.2-7

Verilen $z_0, z_1, \dots, z_{n-1}$ değerleri (muhtemelen tekrarlı) için sadece $z_0, z_1, \dots, z_{n-1}$ noktalarında (muhtemelen tekrarlı) sıfırları ve derece-sınırı $n + 1$ olan $P(x)$ polinomunun katsayılarının nasıl bulunacağını gösteriniz. Yapacağınız işlem $O(n \lg^2 n)$ sürede çalışmalıdır. (*İpucu:* $P(x)$ in z_j de sıfırı olması ancak ve ancak $P(x)$ in $(x - z_j)$ lerin çarpımı olması ile mümkündür).

30.2-8 *

$a = (a_0, a_1, \dots, a_{n-1})$ vektörünün *civıltı (chirp) dönüşümü*, z karmaşık bir sayı ve $y_k = \sum_{j=0}^{n-1} a_j z^{kj}$ olmak üzere $y = (y_0, y_1, \dots, y_{n-1})$ vektörüdür.

Bundan dolayı, DFT, $z = \omega_n$ alınarak elde edilen cıvıltı dönüşümünün özel bir halidir. Herhangi bir karmaşık sayı için cıvıltı dönüşümünün $O(n \lg n)$ sürede nasıl değerlendirileceğini gösteriniz (*İpucu*: Cıvıltı dönüşümünü girişim olarak yorumlamak için

$$y_k = z^{k^2/2} \sum_{j=0}^{n-1} \left(a_j z^{j^2/2} \right) \left(z^{-(k-j)^2/2} \right)$$

denklemini kullanınız).

30.3 Verimli FFT uygulamaları

Sinyal işleme gibi son derece yüksek hızın talep edildiği DFT nin pratik uygulamaları olduğundan, bu bölüm iki FFT uygulamasını analiz etmektedir. Öncelikle, $\Theta(n \lg n)$ sürede çalışan ancak Θ -notasyonunda Kısım 30.2 deki özyinelemeli versiyondan farklı olarak düşük bir sabit sayı olan, FFT nin iteratif versiyonunu anlatacağız (doğru zamanlı uygulamaya bağlı olarak, özyinelemeli versiyon donanım önbelleğinde daha verimli kullanılabilir). Sonrasında, verimli paralel FFT devresi tasarlamak için bizi tekrarlamalı uygulamaya yönlendiren anlayışları kullanacağız.

İteratif FFT uygulaması

Öncelikle RECURSIVE-FFT algoritmasının 10-13 nolu satırlarındaki **for** döngüsünün $\omega_n^k y_k^{[1]}$ içerdiğini hatırlatmak isteriz. Derleme terminolojisinde, böyle bir değere **ortak alt ifade** adını veririz. t değişkenini geçici olarak saklayarak, döngüyü sadece bir kere hesaplayarak değiştirebiliriz.

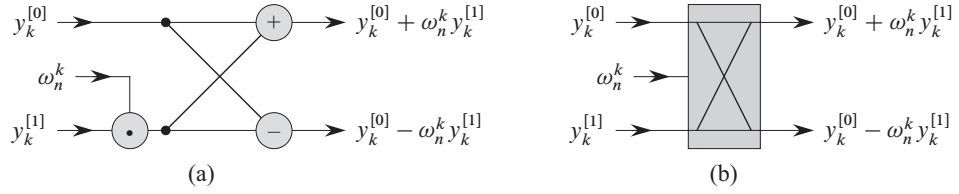
```

for  $k = 0$  to  $n/2 - 1$ 
     $t = \omega y_k^{[1]}$ 
     $y_k = y_k^{[0]} + t$ 
     $y_{k+(n/2)} = y_k^{[0]} - t$ 
     $\omega = \omega \omega_n$ 

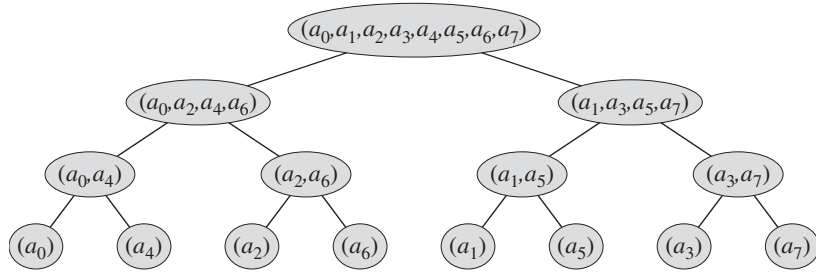
```

Döngüdeki $y_k^{[1]}$ ile $\omega = \omega_n^k$ döndürme faktörünü çarparak, çarpımı t içinde depolayarak ve $y_k^{[0]}$ den t yi toplayıp-çıkararak elde edilen bu işlem **kelebek (butterfly) işlemi** olarak bilinir ve Şekil 30.3 te şematik olarak gösterilmiştir.

Şimdi, FFT algoritmasının özyinelemeli yerine iteratif bir şekilde nasıl yapılacağını göstereceğiz. Şekil 30.4 te, $n = 8$ başlangıç değeri için ağaç yapısında RECURSIVE-FFT nin yürütülmesinde iterasyonlar için girdi vektörlerini düzenledik. Karşılık gelen girdi vektörü ile etiketlenen prosedürün her bir çağrısı için ağaç bir düğüme sahiptir. 1 elemanlı vektörden farklı elemanlar olduğu sürece,



Şekil 30.3 Kelebek işlemi. (a) İki girdi değeri soldan giriş yapar, döndürme faktörü ω_n^k , $y_k^{[1]}$ ile çarpılır ve toplam ve fark sağ tarafta çıktı olarak verilir. (b) Kelebek işleminin basitleştirilmiş çizimi. Bu gösterimi FFT nin paralel devresinde kullanacağız.



Şekil 30.4 RECURSIVE-FFT işleminin özyinelemeli çağrılardaki girdi vektörlerinin ağacı. $n = 8$ için başlangıcın yürütülmesidir.

her RECURSIVE-FFT yürütmesi iki tane yineleme çağrısı yapar. İlk çağrı sol kökte ve ikinci çağrı sağ kökte yer alır.

Ağaca baktığımız zaman, a başlangıç vektörünün yapraklardaki elemanlarının bulunuşunu düzenleyebilirsek, RECURSIVE-FFT işleminin gerçekleştirilmesini yukarıdan aşağıya yerine aşağıdan yukarıya olacak şekilde izleyebiliriz. Öncelikle, ikililerin içindeki elemanları alırız, her bir ikilinin DFT sini kelebek işlemi ile hesaplar ve bu çifti kendi DFT si ile yer değiştiririz. Sonra, bu vektör $n/2$ tane 2 elemanlı DFT lerden meydana gelir. Daha sonra, ikililerdeki bu $n/2$ tane DFT yi alıp ve bunlar iki tane 2 elemanlı DFT yi bir tane 4 elemanlı DFT ile değiştirerek ve iki kelebek işleminden gelen dört vektörün elemanlarının DFT si hesaplanır. Bu vektör, $n/4$ tane 4 elemanlı DFT leri barındırır. Bu mantığı En sondaki n elemanlı DFT yi $n/2$ kelebek işlemi kullanarak iki tane $(n/2)$ elemanlı vektörüne aktarana kadar devam ettiririz.

Yukarıdaki yaklaşımı koda dökebilmek için Şekil 30.4 teki ağacın yapraklarında yer alan a girdi vektörünün başlangıç elemanlarını tutan $A[0 \dots n - 1]$ dizisini kullanırız. (Daha sonra bitlerin ters çevrilmesi permütasyonu olarak bilinen

bu düzenin nasıl belirlendiğini göstereceğiz). Ağacın her bir aşamasında DFT leri birleştirmemiz gerektiğinden, 1 den (aşağıda 2 elemanlı DFT leri oluşturmak için ikilileri birleştirdiğimiz durum) $\lg n$ e (yukarıda son sonucu üretmek için iki tane $(n/2)$ elemanlı DFT leri bağladığımız durum) kadar değişen seviyeleri saymak için s değişkenini tanımlarız. Bundan dolayı, algoritma aşağıdaki yapıdadır.

```

1  for  $s = 1$  to  $\lg n$ 
2      for  $k = 0$  to  $n - 1$  by  $2^s$ 
3          iki tane  $2^{s-1}$ -elemanlı
               $A[k \dots k + 2^{s-1} - 1]$  ve  $A[k + 2^{s-1} \dots k + 2^s - 1]$  DFT yi
               $2^s$ -elemanlı  $A[k \dots k + 2^s - 1]$  DFT içine bağla

```

Döngünün (satır 3) yapısını daha çok kaba kod olarak açıklayabiliriz. $y^{[0]}$ ı $A[k \dots k + 2^{s-1} - 1]$ ile $y^{[1]}$ i $A[k + 2^{s-1} \dots k + 2^s - 1]$ ile belirleyerek, YİNE-LEMELİ-FFT işleminden for döngüsünü kopyalarız. Her kelebek işleminde kullanılan döndürme faktörü s değerine dayanmaktadır; bu değer $m = 2^s$ iken ω_m nin bir kuvvetidir (Anlaşılmayı arttırmak adına m değişkenini tanımlarız). Kelebek işlemini gerçekleştirmek amacıyla başka bir u geçici değişkenini tanımlarız. Satır 3 ü döngü yapısıyla yer değiştirdiğimizde, aşağıdaki kaba kodu elde ederiz, bu kod daha sonra göreceğimiz paralel uygulamanın temelini oluşturur. Başlangıç düzende ihtiyaç duyduğumuz a vektörünü A dizisi içine kopyalamak için öncelikle yardımcı işlem BIT-TERS-KOPYALAMA (BIT-REVERSE-COPY) (a , A) çağrılır.

ITERATIVE-FFT(a)

```

1  BIT-REVERSE-COPY( $a$ ,  $A$ )
2   $n = a.length$  //  $n$ , 2 nin bir kuvveti
3  for  $s = 1$  to  $\lg n$ 
4       $m = 2^s$ 
5       $\omega_m = e^{2\pi i/m}$ 
6      for  $k = 0$  to  $n - 1$  by  $m$ 
7           $\omega = 1$ 
8          for  $j = 0$  to  $m/2 - 1$ 
9               $t = \omega A[k + j + m/2]$ 
10              $u = A[k + j]$ 
11              $A[k + j] = u + t$ 
12              $A[k + j + m/2] = u - t$ 
13              $\omega = \omega \omega_m$ 
14  return  $A$ 

```

BIT-REVERSE-COPY işlemi a girdi vektörünün elemanlarını alıp nasıl A dizisinde istenilen düzende nasıl yerine koyar? Şekil 30.4 teki yapraklardaki düzen

bit-tersine çevirme permütasyonudur. Bu şu demek ki; k nin ikilik tabandaki gösterimindeki bitlerin tersi ile oluşturulan $\lg n$ bitlik tam sayı $\text{rev}(k)$ olursa, a_k vektör elemanını $A[\text{rev}(k)]$ pozisyonunda yerleştirmek isteriz. Örneğin, Şekil 30.4 te 0, 4, 2, 6, 1, 5, 3, 7 düzen yapraklarda görünür; bu dizinin ikilik gösterimi 000, 100, 010, 110, 001, 101, 011, 111 ve her bir değerin bitlerini tersten yazarsak, 000, 001, 010, 011, 100, 101, 110, 111 dizisini elde ederiz. Genel olarak bit-ters kopyalama permütasyonunu isteriz, ağacın en üst seviyesindeki indislerin en düşük mertebeli bit 0 ise sol alt ağaca, 1 ise sağ alt ağaca gidilir. Yapraklarda bit-ters kopyalama ile verilen düzeni elde edene kadar, her seviyede en düşük seviyedeki biti çıkararak bu işleme devam ederiz.

$\text{rev}(k)$ fonksiyonunun kolayca hesaplayabileceğimizden, BIT-REVERSE-COPY işlemi basittir:

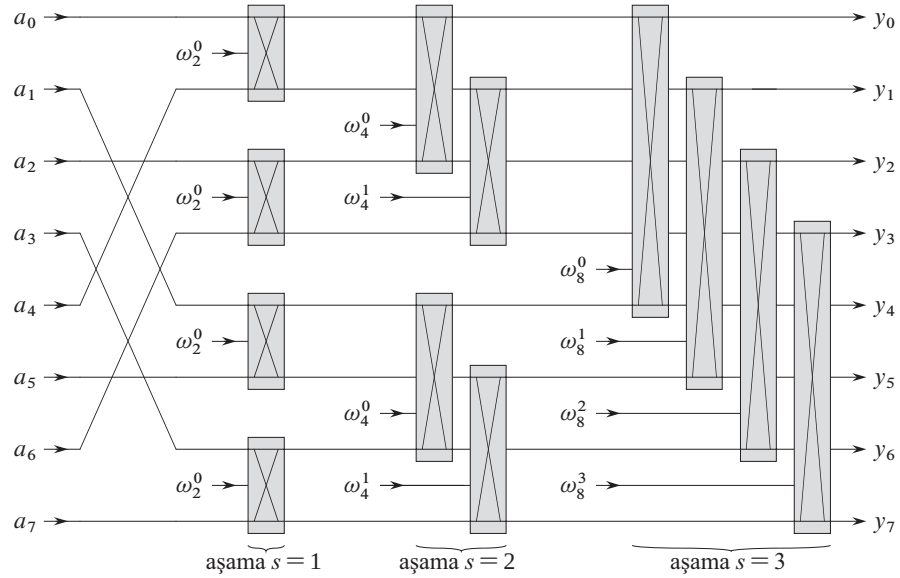
BIT-REVERSE-COPY(a, A)

```

1   $n = a.length$ 
2  for  $k = 0$  to  $n - 1$ 
3       $A[\text{rev}(k)] = a_k$ 
```

İteratif FFT uygulaması $\Theta(n \lg n)$ sürede çalışır. n defa tekrar yaptığımızdan ve 0 ve $n - 1$ arasındaki tam sayıları $\lg n$ bit ile $O(\lg n)$ sürede ters çevirebildiğimizden dolayı, BIT-REVERSE-COPY(a, A) açıkça $O(n \lg n)$ sürede çalışır (Uygulamada, genelde n nin başlangıç değerini bildiğimizden k den $\text{rev}(k)$ ye dönüşüm tablosunu muhtemelen kodlarız, BIT-REVERSE-COPY işlemi $\Theta(n)$ sürede düşük katsayılı olarak çalışır. Başka bir seçenek olarak, Soru 17-1 de tanımlanan akıllı ters ikilik sayacı kullanabiliriz). $\Theta(n \lg n)$ sürede ITERATIVE-FFT nin çalıştığının kanıtını tamamlamak için 8-13 nolu satırlardaki en iç döngünün sayılarının çalışma zamanını $L(n)$ ile gösteririz, bu $\Theta(n \lg n)$ dür. 6-13 nolu satırlardaki **for** döngüsü her bir s değeri için $n/m = n/2^s$ defa tekrar eder ve 8-13 nolu satırlardaki en iç döngü $m/2 = 2^{s-1}$ defa tekrar eder. Böylece,

$$\begin{aligned}
 L(n) &= \sum_{s=1}^{\lg n} \frac{n}{2^s} \cdot 2^{s-1} \\
 &= \sum_{s=1}^{\lg n} \frac{n}{2} \\
 &= \Theta(n \lg n) .
 \end{aligned}$$



Şekil 30.5 $n = 8$ girdi için FFT yi paralel bir şekilde hesaplayan devre. Döndürülen çarpanın yanısıra iki kanaldaki girdi değerleri kelebek işlemine sokulur ve kelebek işleminde iki kanaldaki değerler çıktı olarak üretilir. Kelebeklerin aşamaları ITERATIVE-FFT işleminin en dıştaki döngüsündeki iterasyonların etiketlenmesine karşılık gelir. Kelebek boyunca geçen en üst ve en alttaki kanallar onunla iletişime geçer; kelebeğin ortasından geçen kanal kelebek ile değiştirilen değerleri veya kelebekteki değerleri etkilemez. Örneğin, aşama 2 deki en üstteki kelebeğin kanal 1 ile bir ilgisi yoktur (kanalın çıktısı y_1 ile etiketlenir); onun girdileri ve çıktıları sadece kanal 0 ve 2 dedir (sırasıyla, y_0 ve y_2 etiketlenir). Bu devre $\Theta(\lg n)$ yoğunluğuna sahiptir ve $\Theta(n \lg n)$ kelebek işlemleri ile birlikte yürütür.

Paralel FFT devresi

FFT için verimli paralel algoritma üretmek için verimli iteratif FFT algoritmasına olarak sağlayan özelliklerin birçoğunu ortaya çıkarabiliriz. Paralel FFT algoritmasını bir devre gibi ifade edebiliriz. Şekil 30.5 $n = 8$ girdi için FFT yi hesaplayan paralel FFT devresini göstermektedir. Devre girdilerin bitleri ters kopyalama permütasyonu ile başlar, $\lg n$ aşama ile devam edilir, her bir aşama $n/2$ tane kelebek paralel olarak gerçekleştirilir. Bununla birlikte, bir devrenin **yoğunluğu** – herhangi bir çıktı ve girdi arasındaki sayısal elemanların ulaşabileceği en yüksek sayı – $\Theta(\lg n)$.

Paralel FFT devresinin en soldaki bölümü bitleri ters kopyalama işlemini gerçekleştirir ve kalan ITERATIVE-FFT işlemini kopya eder. **for** döngüsünün en dışındaki her bir iterasyon $n/2$ bağımsız kelebek işlemini yapar, devre onları paralel şekilde gerçekleştirir. ITERATIVE-FFT ile birlikte her bir iterasyon s değeri Şekil

30.5 de gösterilen kelebeklerin bir aşamasına karşılık gelir. $s = 1, 2, \dots, \lg n$ için s aşaması her biri 2^{s-1} tane kelebek içeren (ITERATIVE-FFT deki j nin her bir değerine karşılık gelmektedir) $n/2^s$ tane kelebek grubundan (ITERATIVE-FFT deki k nin her bir değerine karşılık gelir) oluşur. En içteki döngünün (ITERATIVE-FFT nin 9-12 nolu satırları) kelebek işlemlerine karşılık gelen kelebekler Şekil 30.5 te gösterilmiştir. Dikkat edilmelidir ki; kelebekler içinde kullanılan döndürme faktörleri ITERATIVE-FFT içinde kullanılanlara karşılık gelmektedir: s aşamasında $m = 2^s$ iken $\omega_m^0, \omega_m^1, \dots, \omega_m^{m/2-1}$ kullanırız.

Alıştırmalar

30.3-1

Girdi vektörünün DFT sini ITERATIVE-FFT nin nasıl hesapladığını gösteriniz (0, 2, 3, -1, 4, 5, 7, 9)

30.3-2

Bitleri tersine yazma permütasyonunu başlangıçta kullanmak yerine sonda kullanılarak FFT algoritmasının nasıl uygulanacağını gösteriniz (*İpucu*: Ters DFT yi göz önüne alınız).

30.3-3

Her bir aşamada döndürme faktörleri ITERATIVE-FFT de kaç defa hesaplanır? s aşamasında döndürme faktörleri sadece 2^{s-1} defa hesaplamak için ITERATIVE-FFT yi yeniden yazınız.

30.3-4 *

Bazı durumlarda FFT devresinin kelebek işlemleri ile birlikte toplayıcıların girdiden bağımsız olarak her zaman sıfır çıktısı üretir şekilde hatalı olduğunu kabul edelim. Sadece bir toplayıcının düzgün çalışmadığını ve bunun hangi toplayıcı olduğunu bilmediğimizi varsayalım. Düzgün çalışmayan toplayıcının hangisinin olduğunu tüm FFT devresindeki girdiler ve buna bağlı çıktıları gözleyerek tanımlayınız. Yönteminiz ne kadar verimli?

Problemler

30-1 Böl-ve-yönet (Divide-and-conquer) çarpma yöntemi

- $ax + b$ ve $cx + d$ doğrusal polinomlarını sadece üç çarpma işlemi ile nasıl çarpmasının yapılacağını gösteriniz (*İpucu*: Çarpmalardan birisi $(a + b)(c + d)$).
- $\Theta(n^{\lg 3})$ sürede derece-sınırı n olan iki polinomun çarpımını böl-ve-yönet mantığı ile yapan iki algoritma veriniz. İlk algoritmada girdi polinomunun katsayıları yüksek ve alt dereceli olacak şekilde iki parçaya ve ikinci algoritmada göstergesi tek veya çift olacak şekilde girdi polinomunun katsayıları iki parçaya ayrılmalıdır.

- c. $O(n^{\lg 3})$ adımda n bitlik iki tam sayının her adımda 1 bit değerlerinin en fazla sabit sayısının işlendiği durumda nasıl çarpılacağını gösteriniz.

30-2 Toeplitz matrisleri

Toeplitz matris $n \times n$ boyutundaki $A = (a_{ij})$ matrisi ile tanımlanır, öyle ki; $i = 2, 3, \dots, n$ ve $j = 2, 3, \dots, n$ için $a_{ij} = a_{i-1, j-1}$ 'dir.

- İki Toeplitz matrisin toplamı, Toeplitz matrisi olmak zorunda mıdır? Çarpma işlemi için ne söylenebilir?
- $n \times n$ boyutundaki iki Toeplitz matrisi $O(n)$ sürede toplayacak şekilde Toeplitz matrisi nasıl temsil edeceğimizi tanımlayınız.
- $n \times n$ boyutundaki Toeplitz matris ile n uzunluğundaki bir vektörü çarpmak için $O(n \lg n)$ sürede çalışan bir algoritma veriniz. (b) de verdiğiniz temsili kullanınız.
- $n \times n$ boyutundaki iki Toeplitz matrisi çarpmak için verimli bir algoritma veriniz. Çalışma zamanını analiz ediniz.

30-3 Çok boyutlu hızlı Fourier dönüşümü

Denklem (30.8) ile 1 boyutlu ayrık Fourier dönüşümünü d boyuta genelleştirebiliriz. Girdi boyutları $n_1 n_2 \dots n_d = n$ için $n_1, n_2, \dots, n_d$ olan d boyutlu $A = (a_{j_1, j_2, \dots, j_d})$ dizisidir. d boyutlu ayrık Fourier dönüşümünü şu denklem ile tanımlarız

$$y_{k_1, k_2, \dots, k_d} = \sum_{j_1=0}^{n_1-1} \sum_{j_2=0}^{n_2-1} \dots \sum_{j_d=0}^{n_d-1} a_{j_1, j_2, \dots, j_d} \omega_{n_1}^{j_1 k_1} \omega_{n_2}^{j_2 k_2} \dots \omega_{n_d}^{j_d k_d}$$

burada $0 \leq k_1 < n_1, 0 \leq k_2 < n_2, \dots, 0 \leq k_d < n_d$ dir.

- 1 boyutlu DFT leri sırasıyla hesaplayarak d boyutlu DFT yi hesaplayabileceğimizi gösteriniz. Şöyle ki; öncelikle n/n_1 tane ayrı 1 boyutlu DFT leri boyut 1 olduğu sürece hesaplarız. Sonra, boyut 1 olduğu sürece elde edilen DFT lerin sonucunu girdi olarak kullanarak, n/n_2 tane ayrı 1 boyutlu DFT leri boyut 2 olduğu sürece hesaplarız. Bu sonucu girdi olarak kullanarak, n/n_3 tane ayrı 1 boyutlu DFT leri boyut 3 olduğu sürece hesaplarız ve bunu boyut d olana kadar devam ettiririz.
- Boyutların düzeninin sıkıntı teşkil etmediğini gösteriniz, böylece d boyutlu DFT yi d boyutlularının herhangi bir düzeninde 1 boyutlu DFT leri hesaplayarak bulabiliriz.

- c. Hızlı Fourier dönüşümünü elde ederek her 1 boyutlu DFT yi hesaplarsak, d den bağımsız olarak d boyutlu DFT nin hesaplanması için toplam süre $O(n \lg n)$ dir.

30-4 Bir polinomun tüm türevlerinin bir noktada değerlendirilmesi

Derece-sınırı n olan $A(x)$ polinomu için onun t . türevini şöyle tanımlarız

$$A^{(t)}(x) = \begin{cases} A(x), & t = 0 \text{ ise,} \\ \frac{d}{dx} A^{(t-1)}(x), & 1 \leq t \leq n-1 \text{ ise,} \\ 0, & t \geq n \text{ ise.} \end{cases}$$

$A(x)$ in $(a_0, a_1, \dots, a_{n-1})$ olan katsayılar temsilinden ve verilen x_0 noktasından, $A^{(t)}(x_0)$ ı $t = 0, 1, \dots, n-1$ için belirlemek istiyoruz.

- a. Verilen $b_0, b_1, \dots, b_{n-1}$ katsayıları için

$$A(x) = \sum_{j=0}^{n-1} b_j (x - x_0)^j,$$

için $A^{(t)}(x_0)$ ı $O(n)$ sürede $t = 0, 1, \dots, n-1$ i nasıl hesaplanacağını gösteriniz.

- b. $k = 0, 1, \dots, n-1$ için verilen $A(x_0 + \omega_n^k)$ yi kullanarak $b_0, b_1, \dots, b_{n-1}$ leri $O(n \lg n)$ sürede nasıl bulacağımızı açıklayınız.

- c.

$$A(x_0 + \omega_n^k) = \sum_{r=0}^{n-1} \left(\frac{\omega_n^{kr}}{r!} \sum_{j=0}^{n-1} f(j) g(r-j) \right)$$

ve

$$g(l) = \begin{cases} x_0^{-l} / (-l)!, & -(n-1) \leq l \leq 0 \text{ ise,} \\ 0, & 1 \leq l \leq n-1 \text{ ise} \end{cases}$$

olduğunu kanıtlayınız. Burada $f(j) = a_j \cdot j!$

- d. $k = 0, 1, \dots, n-1$ için $A(x_0 + \omega_n^k)$ yi $O(n \lg n)$ sürede nasıl değerlendireceğimizi açıklayınız. x_0 da $A(x)$ in aşikar olmayan tüm türevlerinin $O(n \lg n)$ sürede değerlendirebileceğimiz sonucuna varınız.

30-5 Bir polinomun birden fazla noktada değerlendirilmesi

Horner kuralı kullanarak $O(n)$ sürede derece-sınırı n olan bir polinomu tek bir noktada nasıl değerlendirildiğini gördük. FFT yi kullanarak $O(n \lg n)$ sürede böyle bir polinomun birimin tüm n karmaşık köklerinde nasıl değerlendirildiğini de araştırdık. Şimdi, $O(n \lg^2 n)$ sürede derece-sınırı n olan bir polinomun rassal seçilen n noktada nasıl değerlendirileceğini göstereceğiz.

Bunu yapabilmek için böyle bir polinom başka bir polinoma $O(n \lg n)$ sürede bölünüyorsa, kalan polinomunu hesaplayabiliriz, bu sonucu kanıtsız olarak veriyoruz. Örneğin, $3x^3 + x^2 - 3x + 1$ polinomunu $x^2 + x + 2$ ye böldüğümüzde kalan

$$(3x^3 + x^2 - 3x + 1) \bmod (x^2 + x + 2) = -7x + 5$$

dir.

$A(x) = \sum_{k=0}^{n-1} a_k x^k$ polinomunun katsayı temsili ve $x_0, x_1, \dots, x_{n-1}$ gibi n nokta verildiğinde, n tane $A(x_0), A(x_1), \dots, A(x_{n-1})$ değerini hesaplamak istiyoruz. $0 \leq i \leq j \leq n-1$ için $P_{ij}(x) = \prod_{k=i}^j (x - x_k)$ ve $Q_{ij}(x) = A(x) \bmod P_{ij}(x)$ polinomlarını tanımlayalım. Dikkat edilmelidir ki; $Q_{ij}(x)$, en fazla $j-i$ derecesine sahiptir.

- a. Herhangi bir z noktası için $A(x) \bmod (x - z) = A(z)$ olduğunu kanıtlayınız.
- b. $Q_{kk}(x) = A(x_k)$ ve $Q_{0,n-1}(x) = A(x)$ olduğunu kanıtlayınız.
- c. $i \leq k \leq j$ için $Q_{ik}(x) = Q_{ij}(x) \bmod P_{ik}(x)$ ve $Q_{kj}(x) = Q_{ij}(x) \bmod P_{kj}(x)$ e sahip olduğumuzu kanıtlayınız.
- d. $A(x_0), A(x_1), \dots, A(x_{n-1})$ i $O(n \lg^2 n)$ sürede değerlendiren bir algoritma veriniz.

30-6 Modüler aritmetik kullanan FFT

Tanımlandığı gibi, ayırık Fourier dönüşümü karmaşık sayılar ile hesaplama yapmamızı gerektirir, yuvarlama hataları nedeniyle, bu sonuç hassasiyet kaybıyla sonuçlanabilir. Bazı problemler için, sadece tam sayıların içerildiği durumlar için cevap bilinmekte ve modüler aritmetiğe dayanan FFT nin bir versiyonunu kullanarak cevabın tam olarak hesaplandığını garanti ederiz. Böyle bir probleme örnek, tam sayı katsayılarla sahip iki polinomun çarpımı verilir. Alıştırma 30.2-6 bir yaklaşım verir, burada DFT yi n noktada işlemek için $\Omega(n)$ uzunluğunun modülü kullanılır. Bu problem başka bir yaklaşım verir, $O(\lg n)$ gibi daha makul bir uzunluğun modülü kullanılır; Bölüm 31 de verilen malzemelere ihtiyaç olduğu anlaşılr. $n, 2$ nin bir kuvveti olsun.

- a. $p = kn + 1$ asal sayı olacak şekilde en küçük k yi aradığımızı farzedin. Neden k nin yaklaşık olarak $\ln n$ olduğunu ummamız gerektiğini veren basit sezgisel bir yaklaşımı açıklayınız. (k değeri daha büyük veya küçük olabilir, ancak ortalama olarak k nin olası değerlerini $O(\lg n)$ ile makul bir şekilde test edebiliriz). p nin beklenen uzunluğu ile n nin uzunluğunu karşılaştırdığımızda nasıl olur?

- $g, \mathbb{Z}_p^*$ 'ın bir üretici ve $w = g^k \bmod p$ olsun.
- b.** DFT ve ters DFT nin modülo p ye ters işlemlerinin iyi tanımlı olduğunu tartışınız, burada w , birimin n . temel kökü olarak kullanılmaktadır.
 - c.** FFT ve tersinin modülo p ye göre $O(n \lg n)$ sürede nasıl çalıştığını gösteriniz, burada $O(\lg n)$ bitlerinin karakterleri üzerindeki işlemler birim zamanı almaktadır. Algoritmaya p ve w verildiğini varsayalım.
 - d.** $(0, 5, 3, 7, 7, 2, 1, 6)$ vektörünün DFT sini $p = 17$ modülüne göre hesaplayınız. $\mathbb{Z}_{17}^*$ nin üreticinin $g = 3$ olduğuna dikkat edilmelidir.

Bölüm notları

Van Loan'ın kitabı [343] hızlı Fourier dönüşümünün seçkin bir değerlendirmesini sağlamaktadır. Press, Tekolsky, Vetterling ve Flannery [283, 284] hızlı Fourier dönüşümü ve uygulamalarının güzel bir tanımına sahiptir. FFT nin popüler bir uygulama alanı olan sinyal işlemeye iyi bir giriş için, Oppenheim, Schaffer [266] ve Oppenheim, Willsky'nin [267] kitaplarına göz atılır. Oppenheim ve Schaffer kitabı n nin 2 nin bir kuvveti olmadığı durumlar için neler yapılacağını da göstermektedir.

Fourier analizi 1 boyutlu veriler ile sınırlı değildir. Görüntü işlemede veriler 2 veya daha fazla boyutlarda işlenir. Gonzalez, Woods [146] ve Pratt [281] çok boyutlu Fourier dönüşümlerini ve bunların görüntü işlemedeki kullanımlarını tartışır ve Tolimieri, An, Lu [338] tarafından ve Van Loan [343] tarafından yazılan kitaplar çok boyutlu hızlı Fourier dönüşümünün matematiğini tartışır.

Cooley ve Tukey [76] 1960'larda FFT yi planlayarak oldukça yaygın itibara sahip olmuşlardır. Aslında, FFT önceleri birçok kez bulunmuş, ancak önemi modern sayısal bilgisayarların keşfinden önce tam olarak anlaşılamamıştır. Press, Tekolsky, Vetterling ve Flannery yöntemin kaynaklarını Runge ve König'in 1924 yılındaki çalışmasına vermesine rağmen, FFT nin hikayesi Heideman, Johnson ve Burrus [163] çalışmasındaki izlere dayanarak Gauss'un 1805 yılındaki çalışmasına kadar uzanmaktadır.

Frigo ve Johnson [117], FFTW Batının en hızlı Fourier dönüşümü (fastest Fourier transform in the West) olarak adlandırılan, FFT nin hızlı ve biçimlendirilebilir bir uygulamasını geliştirmişlerdir. FFTW, aynı problem üzerinde çoklu DFT hesaplamasına ihtiyaç duyulan durumlar için tasarlanmıştır. DFT leri hesaplamadan önce, FFTW "planlayıcı" olarak çalışır, burada denemeler serisi ile ana makinede verilen problem boyutu için FFT hesaplaması nasıl en iyi şekilde parçalara ayrılması belirlenir. FFTW donanım ön belleğini verimli olarak kullanılmasına uyum sağlar ve alt problemler yeterince küçük olduğunda, FFTW iyileştirilmiş, doğrusal kod ile onları çözer. Bununla birlikte, FFTW, n büyük bir asal sayı olduğu durumda bile n boyutlu bir problem için $O(n \lg n)$ sürede alınmasının beklenmedik avantajına sahiptir.

Standart Fourier dönüşümü zaman alanında noktaların temsilinin düzgün aralıklı olarak yerleştirilmesini varsaymasına rağmen, eşit olmayan aralıklı verilerde diğer teknikler FFT ye yaklaşmaktadır. Ware [348] tarafından yazılan makale bu konuda genel bakış sağlamaktadır.

Sayılar Teorisi, soyut matematikte güzel ama kullanışsız bir konu olarak incelenmişti. Günümüzde ise sayı-teorik algoritmalar büyük asal sayılara dayanılarak geliştirilen şifreleme şemaları nedeniyle geniş alanda kullanılırlar. Büyük asal sayıları bulmanın kolay olması ve bu büyük asal sayıların çarpımlarını nasıl çarpanlarına ayıracağımızı bilmenin (veya bunlara bağlı problemlerin çözümünde, örneğin ayrık algoritmaların hesaplanması) etkili bir yolu olmamasından dolayı bu yöntemlerin uygulanabilmesi mümkündür. Bu bölüm sayılar teorisinin bir bölümünü ve uygulamaların altında yatan algoritmaları sunar.

Kısım 31.1 bölünebilme, modüler denklik, asal çarpanlarına ayırma vb. gibi sayılar teorisinin ana kavramlarını içerir. 31.2 en eski algoritmalarından olan, iki tamsayı için en büyük ortak böleni bulan Euclid Algoritmasından bahseder. 31.3 modüler aritmetiğin kavramlarını inceler. Kısım 31.4 verilen bir a sayısının katlarının kümesi, n modülü ve $ax \equiv b \pmod{n}$ denklemi için bütün çözümlerinin Euclid Algoritmasıyla nasıl bulunacağından bahseder. Kısım 31.5 te Çin kalan teoremi anlatılır. 31.6. Kısım ise; verilen a sayısının kuvvetlerini ve n modülünü göz önüne alarak, verilen a , b ve n için a^b yi etkin şekilde bulan tekrarlı-kareleme algoritmasını(repeated-squaring algorithm) inceler. Bu işlem asallık testinin ve modern kriptografinin esasını oluşturur. Kısım 31.7 ise, RSA şifreleme sistemine açıklar. 31.8 rasallaştırılan asallık testini inceler. RSA şifreleme sistemine anahtarlar oluşturmak amacıyla, büyük asal sayıları etkin bir şekilde bulmak için bu testi kullanırız. Son olarak 31.9 küçük asal sayıları çarpanlarına ayırmak için sezgisel, etkin ama basit bir yöntemi inceler. Gariptir ki, çarpanlarına ayırma insanlığının problemlili kalmasını istediği tek konudur çünkü RSA'nın güvenliği büyük tam sayıları çarpanlarına ayırmanın zorluğuna bağlıdır.

Girdilerin boyutu ve aritmetik hesaplamaların maliyeti (cost)

Büyük tam sayılarla çalıştığımız için, girdi boyutları ve temel aritmetik işlemlerin maliyetinin ne olması gerektiğini ayarlamamız gerekir.

Bu bölümde, “büyük girdi” ile ifade edilmek istenen “büyük tam sayılar” dır, “sayıca çok tam sayılar” anlamında değildir (sıralama için olduğu gibi). Bu ne-

denle, girdinin boyutunu; girdinin içerdiği tam sayı sayısına göre değil o girdiyi ifade eden *bitlerin sayısı* cinsinden değerlendirmemiz gerekir. Girdileri $a_1, a_2, \dots, a_k$ tam sayıları olan bir algoritma, eğer $\lg a_1, \lg a_2, \dots, \lg a_k$ polinomiyal zamanda çalışıyorsa yani ikili kodlanmış girdileri $\lg a_1, \lg a_2, \dots, \lg a_k$ polinomuyla sınırlı ise, **polinom-zamanlı algoritmadır**.

Bu kitabın büyük bölümünde, temel aritmetik işlemlerin (çarpma, bölme veya kalan hesaplama gibi) birim zamanda olduğunu varsaymanın uygun olduğunu düşünüyorduk. Algoritmanın çalıştırdığı buna benzer işlemleri sayarak, algoritmanın bilgisayarda gerçek çalışma zamanına yakın bir tahminde bulunmak için bir temel oluştururuz. Temel işlemler girdi boyutları büyük olduğunda, fazla zaman alan işlemler halini alabiliyorlar. Bu nedenle sayı-teorik algoritmanın ne kadar **bit işlemleri** gerçekleştirdiğiyle ilgilenmek daha yerinde olacaktır. Bu modelde, sıradan yöntemlerle iki adet β -bitlik tamsayının çarpımı $\Theta(\beta^2)$ bit işlem gerektirir. Benzer şekilde, β -bitlik bir tam sayıyı daha küçük bir tam sayıya bölmek veya β -bitlik bir tam sayının daha küçük bir tam sayıya bölünmesinden kalanı bulmak da $\Theta(\beta^2)$ bit işlem gerektirir. Bilinen daha hızlı yöntemler mevcuttur. Örneğin, basit bir böl-ve-yönet (divide-and-conquer) algoritmasıyla iki adet β -bitlik sayıyı çarpmanın çalışma zamanı $\Theta(\beta^{\lg 3})$ dır, en hızlısı ise $\Theta(\beta \lg \beta \lg \lg \beta)$ dır. Oysa ki daha pratik kullanım için, $\Theta(\beta^2)$ sıklıkla en iyisidir ve analizlerimizde bunu baz olarak alacağız.

Bu bölümde algoritmaların analizini, aritmetik işlem sayısı ve gerektirdiği bit işlemi sayısına göre yapacağız.

31.1 Temel sayı-teorik kavramlar

$\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ tam sayılar kümesi ve $\mathbb{N} = \{0, 1, 2, \dots\}$ doğal sayılar kümesi hakkında temel sayılar teorisinin kavramlarının bir incelemesini bu bölümde yapacağız.

Bölünebilirlik ve bölenler

Bir tam sayının başka bir tam sayı tarafından bölünebilir olması sayılar teorisinin anahtarıdır. $d \mid a$ gösterimi (d, a yı **böler**” şeklinde okunur) bazı k tam sayı değerleri için $a = kd$ anlamını ifade eder. Her bir tam sayı 0’ı böler. Eğer $a > 0$ ve $d \mid a$ ise o zaman $|d| \leq |a|$ dır. Eğer $d \mid a$ ise, aynı zamanda a, d nin **katıdır** diyebiliriz. Eğer d, a yı bölmüyorsa $d \nmid a$ ile ifade edilir.

Eğer $d \mid a$ ve $d \geq 0$, ise o zaman d ye a nın bir **bölenidir** denir. Dikkat ediniz ki, $d \mid a$ ancak ve ancak $-d \mid a$ dır. Yani bölenleri negatif olmayan olarak sınıflandırmak genelleme yapmamıza engel değildir; a nın herhangi bir böleninin negati-

finin de a yı böleceği aşıkardır. Sıfırdan farklı bir a tam sayısının böleni en az 1 olabilir ve $|a|$ dan büyük olamaz. Örneğin, 24 sayısının bölenleri: 1, 2, 3, 4, 6, 8, 12 ve 24 olarak belirlenir.

Her a pozitif tam sayısı **aşık**ar bölenler (*trivial divisors*) olarak adlandırılan 1 ve a ya bölünür. a sayısının non-trivial (aşıkar olmayan) bölenleri ise a nın **çarpanları** (*factors*) olarak adlandırılırlar. Örneğin 20 sayısının çarpanları 2, 4, 5 ve 10 dur.

Asal sayılar ve bileşik sayılar

Kendisinden ve 1 den başka böleni olmayan $a > 1$ tam sayısına **asal sayı** veya kısaca **asal** denir. Asal sayıların farklı birçok özellikleri mevcuttur ve sayılar kuranında önemli bir yer tutarlar. İlk 20 asal sayı aşağıdaki gibidir:

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71.

Alıştırma 31.1-2 sizden sonsuz çoklukta asal sayının olduğunu kanıtlamanızı istiyor. $a > 1$ tam sayısı asal sayı değilse, **bileşik sayıdır** veya kısaca **bileşiktir**. Örneğin, 39 sayısı bileşiktir çünkü 1 ve 39 dan farklı olarak $3 \mid 39$ dur. 1 tam sayısı **birim** (*unit*) olarak adlandırılır ve ne asal ne bileşik sayıdır. Benzer şekilde 0 ve tüm negatif tam sayılar asal veya bileşik olarak sınıflandırılmazlar.

Bölme teoremi, kalanlar ve modüler denklik

Verilen bir n tam sayısı için tam sayıları n nin katı olan tam sayılar ve n nin katı olmayan tam sayılar kısımlarına ayırarak inceleyebiliriz. Çok sayıda teori bu kısımların ayrılmasını baz alacak şekilde, n nin katı olmayan tam sayıların, n ile bölündüğündeki kalanlarına göre sınıflandırılmasına dayanır. Aşağıdaki teorem bu ayrımın temelini oluşturur. Kanıtını şimdilik geçiyoruz. (ama örneği için bkz: Niven e Zuckerman [265]).

Teorem 31.1 (Bölme Teoremi)

Herhangi bir a tam sayısı ve herhangi bir n pozitif tam sayısı için $0 \leq r < n$ ve $a = qn + r$ yi sağlayan ve tek türlü belirlenen q ve r tam sayıları vardır. ■

$q = \lfloor a/n \rfloor$ bölme işleminde **bölümü**, $r = a \bmod n$ değeri **kalanı** ifade eder. Buna göre, $n \mid a$ ancak ve ancak $a \bmod n = 0$ olduğunda sonucunu elde ederiz.

Tam sayıları, n modülünden kalanlarına göre, n denklik sınıfına ayırabiliriz. a tam sayısını içeren **modulo n denklik sınıfı** (*mod n e göre denklik sınıfı*):

$$[a]_n = \{a + kn : k \in \mathbb{Z}\} .$$

Örneğin, $[3]_7 = \{\dots, -11, -4, 3, 10, 17, \dots\}$ bu kümeyi ayrıca $[-4]_7$ ve $[10]_7$ notasyonlarıyla da ifade edebiliriz. $a \in [b]_n$ ile $a \equiv b \pmod{n}$ ifadesi aynı anlamdadır. Bu şekildeki tüm denklik sınıflarının kümesi:

$$\mathbb{Z}_n = \{[a]_n : 0 \leq a \leq n-1\} \quad (31.1)$$

dir.

$$\mathbb{Z}_n = \{0, 1, \dots, n-1\} \quad (31.2)$$

(31.2) ifadesinin (31.1) e denk olduğu görülür; burada 0 ın $[0]_n$ sınıfını, 1 in (31.1) de $[1]_n$ sınıfını ve böyle devam ederek her sınıfın kendisine ait en küçük negatif olmayan tam sayı ile ifadesinden oluştuğu anlaşılmalıdır. Ancak denklik sınıflarını akılda tutmak gereklidir; örneğin eğer $\mathbb{Z}_n$ in elemanı olan -1 den bahsediyorsak aslında ifade etmek istediğimizin $[n-1]_n$ olduğunu $-1 \equiv n-1 \pmod{n}$ denkliğinden yararlanarak hatırlamamız gerekmektedir.

Ortak bölen ve en büyük ortak bölen

Eğer d , a nın böleni ise ve d aynı anda b nin de bir böleni ise; o halde d a ile b nin **ortak böleni**dir denir. Örneğin, 30 sayısının bölenleri 1, 2, 3, 5, 6, 10, 15 ve 30 dur. Bu durumda 24 ile 30 sayılarının ortak bölenleri: 1, 2, 3, 6 olarak ifade edilir. Unutulmamalıdır ki, 1 herhangi iki tam sayı için her zaman ortak bölenidir.

Ortak bölenlerin en önemli özelliği:

$$d \mid a \text{ ve } d \mid b \text{ ise } \rightarrow d \mid (a+b) \text{ ve } d \mid (a-b) \quad (31.3)$$

dir.

Daha genel bir ifadeyle;

$$d \mid a \text{ ve } d \mid b \text{ ise } \rightarrow d \mid (ax+by) \quad (31.4)$$

her $x, y \in \mathbb{Z}$ için. Ayrıca, $a \mid b$ ise bu durumda $|a| \leq |b|$ veya $b=0$ dir. Buradan, $a \mid b$ ve $b \mid a$ ise $a = \pm b$ olduğu sonucu çıkarılır. (31.5)

a ve b sıfırdan farklı iki tam sayı olmak üzere, bu iki sayının **en büyük ortak böleni**, ortak bölenleri içerisinde en büyüğü olarak alınır; gösterimi $\text{ebob}(a, b)$ şeklindedir. Örneğin, $\text{ebob}(24, 30) = 6$, $\text{ebob}(5, 7) = 1$, $\text{ebob}(0, 9) = 0$ olarak bulunabilir. Eğer a ve b sayılarının her ikisi de sıfırdan farklı ise bu durumda $\text{ebob}(a, b)$ sonucu, 1 ile $\min(|a|, |b|)$ arasında olacaktır. $\text{ebob}(0, 0) = 0$ olarak alınır.

En büyük ortak bölen için elementer özellikler aşağıdaki gibidir:

$$\text{ebob}(a, b) = \text{ebob}(a, b) , \quad (31.6)$$

$$\text{ebob}(a, b) = \text{ebob}(a, b) , \quad (31.7)$$

$$\text{ebob}(a, b) = \text{ebob}(|a|, |b|) , \quad (31.8)$$

$$\text{ebob}(a, b) = |a| , \quad (31.9)$$

$$\text{ebob}(a, b) = |a| \quad \text{her } k \in \mathbb{Z} \text{ için.} \quad (31.10)$$

Sıradaki teorem, $\text{ebob}(a, b)$ nin daha kullanışlı bir tanımını ifade etmemizi sağlar.

Teorem 31.2

a ve b sıfırdan farklı herhangi tam sayılar olmak üzere; $\text{ebob}(a, b)$, $\{ax + by : x, y \in \mathbb{Z}\}$ ile ifade edilen a ile b nin lineer (doğrusal) kombinasyonlarından oluşan kümenin en küçük pozitif elemanıdır.

Kanıt a ve b nin en küçük pozitif doğrusal kombinasyonu s olsun. Bu durumda, $s = ax + by$, $x, y \in \mathbb{Z}$ ve $q = \lfloor a/s \rfloor$ olsun. (31.8) deki eşitlikten görülür ki,

$$\begin{aligned} a \bmod s &= a - qs \\ &= a - q(ax + by) \\ &= a(1 - qx) + b(-qy), \end{aligned}$$

Dolayısıyla, $a \bmod s$ ifadesi a ile b nin bir doğrusal kombinasyonu olduğu görülür. Fakat, $0 \leq a \bmod s < s$ ve elimizde $a \bmod s = 0$ olduğu var, çünkü s öyle bir doğrusal kombinasyonun en küçüğü idi. Bu nedenle, $s \mid a$ ve elementer özelliklerden $s \mid b$ olarak bulunur. Bu da s nin, a ile b nin bir ortak böleni olduğunu ifade eder; dolayısıyla $\text{ebob}(a, b) \geq s$ dir. $\text{ebob}(a, b)$ hem a yı hem b yi bölebildiğinden ve s de a ile b nin doğrusal kombinasyonu olduğundan dolayı, (31.4) eşitliğine göre $\text{ebob}(a, b) \mid s$ olarak bulunur. Ama $\text{ebob}(a, b) \mid s$ ve $s > 0$ olması $\text{ebob}(a, b) \leq s$ olduğunu gösterir. $\text{ebob}(a, b) \geq s$ ve $\text{ebob}(a, b) \leq s$ eşitsizliklerinden $\text{ebob}(a, b) = s$ olduğu sonucuna ulaşılır. Sonuç olarak, s a ile b nin en büyük ortak bölenidir. ■

Sonuç 31.3

Herhangi a ve b tam sayıları için eğer, $d \mid a$ ve $d \mid b$ ise o zaman $d \mid \text{ebob}(a, b)$ dir.

Kanıt Teorem (31.2) ye göre, $\text{ebob}(a, b)$ a ile b nin bir doğrusal kombinasyonu idi. Bu durumda (31.4) eşitliğinden $d \mid \text{ebob}(a, b)$ olduğu görülür. ■

Sonuç 31.4

Her a ve b tam sayısı için n negatif olmayan bir tam sayı olmak üzere, $\text{ebob}(an, bn) = n \text{ebob}(a, b)$

Kanıt Eğer $n = 0$ ise, sonuç açıktır. Eğer $n > 0$ ise, o zaman $\text{ebob}(an, bn)$ $\{anx + bny : x, y \in \mathbb{Z}\}$ kümesinin en küçük pozitif elemanıdır. Bu da $\{ax + by : x, y \in \mathbb{Z}\}$ kümesinin en küçük pozitif elemanının n katıdır. ■

Sonuç 31.5

n , a ve b pozitif tam sayıları için, eğer $n \mid ab$ ve $\text{ebob}(a, n) = 1$ ise $n \mid b$ dir.

Kanıt 31.1-5 örneğinde kanıtı var. ■

Aralarında asal sayılar

Ortak bölenleri yalnızca 1 olan, yani $\text{ebob}(a, b) = 1$ ise, a ile b tam sayıları **aralarında asaldır** denir. Örneğin, 8 ve 15 aralarında asaldır; 8 in bölenleri 1, 2, 4 ve 8 iken 15 in bölenleri 1, 3, 5 ve 15 dir. Sıradaki teorem eğer iki tam sayı p tam sayısı ile aralarında asal ise, bu iki tam sayının çarpımının da p ile aralarında asal olacağından bahseder.

Teorem 31.6

Herhangi a, b ve p tam sayıları için; $\text{ebob}(a, p) = 1$ ve $\text{ebob}(b, p) = 1$ ise $\text{ebob}(ab, p) = 1$ dir.

Kanıt Teorem 31.2 den, x, y, x' ve y' tam sayıları olmak üzere

$$ax + py = 1,$$

$$bx' + py' = 1$$

olarak ifade edilebilir. Bu denklemler düzenlendikten sonra

$$ab(xx') + p(ybx' + y'ax + pyy') = 1$$

yi elde ederiz.

Böylece 1, ab ve p nin doğrusal kombinasyonu olduğundan, Teorem 31.2 uygulanarak kanıt tamamlanır. ■

Eğer $i \neq j$ için $\text{ebob}(n_i, n_j) = 1$ ise $n_1, n_2, \dots, n_k$ tam sayıları **aralarında asal ikililerdir**.

Tek çarpanlara ayırma

Asal sayılara bölünebilmekle ilgili temel ancak önemli bir bulgu aşağıdaki gibidir.

Teorem 31.7

Tüm p asal sayıları ve tüm a ve b tam sayıları için eğer $p \mid ab$ ise o zaman $p \mid a$ veya $p \mid b$ dir (veya her ikisi de olabilir).

Kanıt $p \mid ab$ olmak üzere kabul edelim ki $p \nmid a$ ve $p \nmid b$ olsun. Bu durumda, $\text{ebob}(a, p) = 1$ ve $\text{ebob}(b, p) = 1$ dir. p nin bölenleri yalnızca 1 ve p olduğundan (p asal idi) o halde p ne a yı ne de b yi böler diyebiliriz. Teorem 31.6 dan $\text{ebob}(ab, p) = 1$ olarak bulunur. Bu ise, $p \mid ab$ iken $\text{ebob}(ab, p) = p$ olması gerektiğinden, $p \mid ab$ kabulümüz ile çelişir. Kabulümüz yanlıştır, $p \mid a$ veya $p \mid b$ olmalıdır. ■

Teorem 31.7 ye göre, herhangi bir bileşik tam sayıyı olarak asal sayıların çarpımı şeklinde tek türlü yazabiliriz.

Teorem 31.8 (Tek çarpanlara ayırma)

Herhangi bir bileşik a tam sayısını aşağıdaki çarpım formunda yazabiliriz :

$a = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r}$ ler asal sayılar, $p_1 < p_2 < \cdots < p_r$ ve e_i ler pozitif tamsayılardır.

Kanıt 31.1-11. Alistırmaya bırakılmıştır. ■

Örnek olarak 6000 sayısını, tek türlü şekilde asal çarpanlarına ayırdığımızda sonuç: $2^4 \cdot 3 \cdot 5^3$ olarak bulunacaktır.

Alistırmalar**31.1-1**

Eğer $a > b > 0$ ve $c = a + b$ ise, o zaman $c \bmod a = b$ olduğunu kanıtlayınız.

31.1-2

Sonsuz çoklukta asal sayının var olduğunu kanıtlayınız. (Yol Gösterme: $p_1, p_2, \dots, p_k$ asallarının hiçbiri $(p_1 p_2 \cdots p_k) + 1$ yı bölemez).

31.1-3

Eğer $a \mid b$ ve $b \mid c$ ise, o halde $a \mid c$ dir, kanıtlayınız.

31.1-4

Eğer p asal ve $0 < k < p$ ise, o halde $\text{ebob}(k, p) = 1$ dir kanıtlayınız.

31.1-5

Sonuç 31.5 'i kanıtlayınız.

31.1-6

Eğer p asal ve $0 < k < p$ ise, o halde $p \mid \binom{p}{k}$ dır. a ve b gibi tüm tam sayılar ve p asal sayıları için

$(a + b)^p \equiv a^p + b^p \pmod{p}$ olduğu sonucuna ulaşınız.

31.1-7

a ve b herhangi iki pozitif tam sayı ve $a \mid b$ ise, o zaman tüm x değerleri için

$$(x \bmod b) \bmod a = x \bmod a$$

olduğunu kanıtlayınız. Yine aynı değerler altında tüm x ve y tam sayı değerleri için

$$x \equiv y \pmod{b} \text{ gerektirir } x \equiv y \pmod{a}$$

olduğunu kanıtlayınız.

31.1-8

Herhangi $k > 0$ tam sayısı için $a^k = n$ eşitliğini sağlayan bir a tam sayısı bulunabiliyorsa n tam sayısı **k . kuvvettendir**. Buna ek olarak, $n > 1$ sayısı, $k > 1$ gibi herhangi bir tam sayının k . kuvveti ise, $n > 1$ **aşık-olmayan kuvvettir**. Verilen β -bitlik n tam sayısının, zaman polinomunun aşık-olmayan kuvveti olup olmadığına nasıl karar verileceğini gösteriniz. Verilen bir β -bitlik n tam sayısının, β zaman polinomunda bir aşık-olmayan kuvvet olup olmadığına nasıl karar verileceğini gösteriniz.

31.1-9

(31.6) dan (31.10) a kadar olan denklemleri/eşitlikleri kanıtlayınız.

31.1-10

ebob operatörünün birleşmeli olduğunu gösteriniz. Böylece tüm a , b ve c tam sayıları için

$$\text{ebob}(a, \text{ebob}(b, c)) = \text{ebob}(\text{ebob}(a, b), c)$$

olduğunu kanıtlayınız.

31.1-11 *

31.8 nolu teoremi kanıtlayınız.

31.1-12

β -bitlik bir tam sayının daha kısa bir tam sayıya bölümü ve β -bitlik tam sayının daha kısa bir tam sayıya bölümünden kalanının bulunması ile ilgili olarak etkili algoritmalar bulunuz. Algoritmaların çalışma zamanları $\Theta(\beta^2)$ olmalıdır.

31.1-13

β -bitlik (ikili sistemde) tam sayının onluk sistemde gösterimini hesaplayan bir algoritma bulunuz. Maksimum uzunluğu β olan tam sayıların çarpma ve bölme işlemlerinin zaman karmaşıklığı $M(\beta)$ ise, ardından ikilik sayı sisteminden onluk sayı sistemine geçerkenki karmaşıklıkları $\Theta(M(\beta) \lg \beta)$ oluyor mu, inceleyiniz. (*İpucu: böl-ve-yönet- yaklaşımını kullanın, farklı özyinelemelerle sonucun üst ve alt yarıların elde edin.*)

31.2 En büyük ortak bölen

Bu bölümde, iki tam sayının en büyük ortak böleninin hesaplanmasında daha etkili bir yöntem olan Öklid Algoritmasını tanımlayacağız. Çalışma zamanını hesaplarken, Öklid Algoritmasının en kötü durumu için girdi olarak Fibonacci Sayıları ile ilgili beklenmedik bir bağlantı bulacağız.

Bu bölümde çalışma alanımızı, negatif olmayan tam sayılarla sınırlandıracağız. Bu kısıtlama (31.8) eşitliğindeki $\text{ebob}(a, b) = \text{ebob}(|a|, |b|)$ ile doğrulanır.

İlke olarak, a ve b pozitif tam sayıları için $\text{ebob}(a, b)$ yi, a ve b nin asal çarpanları üzerinden hesaplayabiliriz. Gerçekten, eğer

$$a = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r}, \quad (31.11)$$

$$b = p_1^{f_1} p_2^{f_2} \cdots p_r^{f_r}, \quad (31.12)$$

sıfır kuvvetleri, a ve b için aynı olan $p_1, p_2, \dots, p_r$ asallar kümesini elde etmek için kullanılıyor olmak üzere ise, o halde Alıştırma 31.2-1 de göstermeniz için size sorulacak olan

$$\text{gcd}(a, b) = p_1^{\min(e_1, f_1)} p_2^{\min(e_2, f_2)} \cdots p_r^{\min(e_r, f_r)}. \quad (31.13)$$

Kısım 31.9 da gösterecek olmamıza rağmen, bugüne kadar çarpanlara ayırma için en iyi algoritma polinom zamanda çalışmaz. Böylece, en büyük ortak bölenlerin hesaplanması bu yaklaşım ile en etkin biçimde sağlanıyor denilemez.

Öklid Algoritması, en büyük ortak bölen hesabı için aşağıdaki teoreme dayanır.

Teorem 31.9 (EBOB için özyineleme teoremi)

Negatif olmayan bir a tam sayısı ve herhangi bir b pozitif tam sayısı için

$$\text{ebob}(a, b) = \text{ebob}(b, a \bmod b)$$

dir.

Kanıt $\text{ebob}(a, b)$ ve $\text{ebob}(b, a \bmod b)$ lerin, birbirlerini böldüğünü göstermemiz gerekir. Böylece, (31.5) sonucundan bu iki operatör eşit olmalıdır (her ikisi de negatif olmayan tam sayılardır olduğu için).

İlk olarak göstermemiz gereken, $\text{ebob}(a, b) \mid \text{ebob}(b, a \bmod b)$ dir. Eğer $d = \text{ebob}(a, b)$ ise, $d \mid a$ ve $d \mid b$ dir. (3.8) deki denklemden, $a \bmod b = a - qb$ ve $q = \lfloor a/b \rfloor$ dir. $a \bmod b, a$ ile b nin doğrusal kombinasyonu olduğundan, (31.4) denklemi $d \mid (a \bmod b)$ olması demektir. Böylelikle, $d \mid b$ ve $d \mid (a \bmod b)$ olduğundan sonuç (31.3) der ki $d \mid \text{ebob}(b, a \bmod b)$ veya buna denk olarak

$$\text{ebob}(a, b) = \text{ebob}(b, a \bmod b). \quad (31.14)$$

$\text{ebob}(b, a \bmod b) \mid \text{ebob}(a, b)$ olduğunu göstermek de hemen hemen aynıdır. Şimdi, $d = \text{ebob}(b, a \bmod b)$ dersek o halde $d \mid b$ ve $d \mid (a \bmod b)$ dir. $q = \lfloor a/b \rfloor$ olmak üzere, $a = qb + (a \bmod b)$ olduğundan, a, b ile $(a \bmod b)$ nin bir doğrusal kombinasyonudur. (31.4) özelliğinden $d \mid a$ olduğu sonucuna varırız. $d \mid b$ ve $d \mid a$ elimizde vardı, o halde $d \mid \text{ebob}(a, b)$ yi de (31.3/ sonucundan çıkarırız. Bu da

$$\text{ebob}(b, a \bmod b) = \text{ebob}(a, b) \quad (31.15)$$

(31.5) denklemini kullanarak (31.14) ve (31.15) ile birlikte kanıt tamamlanır. ■

Öklid algoritması

Öklid'in *elemanları*, başlangıçtan bile evvel olmasına rağmen, aşağıdaki ebob algoritmasını tanımlamışlardır. Biz Teorem 31.9 u baz alarak, Öklid algoritmasını bir özyinelemeli program olarak açıkladık. Girdi değerleri olan a ve b , keyfi negatif olmayan tam sayılardır.

```

EUCLID( $a, b$ )
1  if  $b == 0$ 
2      return  $a$ 
3  else return EUCLID( $b, a \bmod b$ )

```

EUCLID algoritmasının çalışma örneğini verirse, ebob(30,21) için:

$$\begin{aligned}
 \text{EUCLID}(30, 21) &= \text{EUCLID}(21, 9) \\
 &= \text{EUCLID}(9, 3) \\
 &= \text{EUCLID}(3, 0) \\
 &= 3
 \end{aligned}$$

Bu hesaplama, EUCLID i özyinelemeli olarak 3 kez çağrılır.

EUCLID doğrulamasını Teorem 31.9 ile yaparız ve algoritma 2. satırda a değerini döndürürse o halde $b = 0$ demektir; yani (31.9) der ki: $\text{ebob}(a, b) = \text{ebob}(a, 0) = a$. Algoritma sonsuz defa çağrılmaz, çünkü her çağırışta ikinci argümanın değeri azalmaktadır ve her zaman negatif olmayan bir değer alacaktır. O halde, EUCLID her zaman doğru sonuçlanır.

Öklid algoritmasının çalışma zamanı

EUCLID algoritmasının en kötü çalışma zamanını hesaplarken a ve b girdilerinin boyutlarına bağlı bir fonksiyonu inceleriz. Bunun için $a > b \geq 0$ giriş değerleri ile genelleme yapabiliriz. Bunu göstermek için, $b > a \geq 0$ durumuna bakalım; bu koşul altında $\text{EUCLID}(a, b)$ $\text{EUCLID}(b, a)$ özyinelemeli çağrısını yapacaktır. Yani eğer ilk argümanın değeri ikinci argümandan küçük ise EUCLID bir rekürsif çağırış daha yaparak argümanların yerini değiştirecek ve daha sonra çalışmaya devam edecektir. Benzer şekilde; eğer $b = a > 0$ ise prosedür bir özyinelemeli çağrıdan sonra duracaktır, çünkü $(a \bmod b) = 0$ olacaktır.

EUCLID in tam çalışma zamanı, yaptığı özyinelemeli çağrılarının sayısıyla orantılıdır. Analizimiz, (3.22) özyinelemesi ile tanımlı F_k Fibonacci sayılarının kullanımını yapar.

Lemma 31.10

Eğer $a > b \geq 1$ ve $\text{EUCLID}(a, b)$ prosedürü $k \geq 1$ özyinelemeli çağrı uyguluyorsa, o zaman $a \geq F_{k+2}$ ve $b \geq F_{k+1}$ dir.

Kanıt Kanıtı k üzerinde tümevarımla vereceğiz. Tümevarımın başlangıcı olarak $k = 1$ alalım. Bu durumda $b \geq 1 = F_2$ ve $a > b$ olduğundan $a \geq 2 = F_3$ dir. $b > (a \bmod b)$ olduğundan, her özyinelemeli çağrıda ilk eleman ikinci elemmandan daima büyük olacaktır, böylece $a > b$ varsayımı her özyinelemeli çağrıda gerçekleştirilmiş olur.

Şimdi $k - 1$ için doğru olsun. $k - 1$ özyinelemeli çağrı oluyorsa göstermemiz gereken k tane özyinelemeli çağrı için de doğru olduğudur. $k > 0$ olduğundan; $b > 0$ idi ve $\text{EUCLID}(a, b)$, $\text{EUCLID}(b, a \bmod b)$ yi özyinelemeli olarak çağırır. $k - 1$ tane çağrı vardı, hipotezden $b \geq F_{k+1}$ ve $(a \bmod b) \geq F_k$ olacaktır. Elimizde

$$\begin{aligned} b + (a \bmod b) &= b + (a - b \lfloor a/b \rfloor) \\ &\leq a, \end{aligned}$$

$a > b > 0$ durumu $\lfloor a/b \rfloor \geq 1$ olduğunu ifade eder. Böylece,

$$\begin{aligned} a &\geq b + (a \bmod b) \\ &\geq F_{k+1} + F_k \\ &= F_{k+2}. \end{aligned}$$

■

Aşağıdaki teorem bu lemmanın yakın bir sonucudur.

Teorem 31.11 (Lamés Teoremi)

Herhangi bir $k \geq 1$ tam sayısı için eğer $a > b \geq 1$ ve $b < F_{k+1}$ ise, $\text{EUCLID}(a, b)$ k dan daha az sayıda özyinelemeli çağrı yapar. ■

Teorem (31.11)'in üst sınırı en iyisidir, bunu da $\text{EUCLID}(F_{k+1}, F_k)$ $k \geq 2$ için tam olarak $k - 1$ özyinelemeli çağrı yaptığını göstererek ifade edebiliriz. k üzerinde tümevarımla gideceğiz. İlk adım için $k = 2$ olsun, $\text{EUCLID}(F_3, F_2)$ tam bir tane çağrı yapar ve $\text{EUCLID}(1, 0)$ olur. ($k = 2$ den başlamak zorundayız, çünkü $k = 1$ olduğunda $F_2 > F_1$ oluyor.) Tümevarım adımında, $\text{EUCLID}(F_k, F_{k-1})$ tam $(k - 2)$ özyinelemeli çağrı yapar. $k > 2$ için $F_k > F_{k-1} > 0$ ve $F_{k+1} = F_k + F_{k-1}$ elimizde var; ve böylece Alıştırma 31.1-1 den

$$F_{k+1} \bmod F_k = F_{k-1} \text{ dir. Böylece elimizde,}$$

$$\begin{aligned} \gcd(F_{k+1}, F_k) &= \gcd(F_k, F_{k+1} \bmod F_k) \\ &= \gcd(F_k, F_{k-1}). \end{aligned}$$

Böylece, $\text{EUCLID}(F_{k+1}, F_k)$ çağrısı, $\text{EUCLID}(F_k, F_{k-1})$ çağrısından bir seferden daha fazla ya da tam olarak, Teorem 31.11 in üst sınırına yaklaşık olarak, $k - 1$ kere yinelenir.

F_k yaklaşık olarak $\phi^k / \sqrt{5}$ olduğu için, burada $(1 + \sqrt{5})/2$ ile altın oran ifa-
de edilir. Bu durumda EUCLID teki özyinelemeli çağrılarının sayısı $O(\lg b)$ dir. Bu

a	b	$\lfloor a/b \rfloor$	d	x	y
99	78	1	3	-11	14
78	21	3	3	3	-11
21	15	1	3	-2	3
15	6	2	3	1	-2
6	3	2	3	0	1
3	0	—	3	1	0

Şekil 31.1 EXTENDED-EUCLID in $\text{ebob}(99,78)$ i nasıl hesapladığı. Her satır iterasyonun bir seviyesinin gösterir: a ve b girdilerin değerleri, $\lfloor a/b \rfloor$ hesaplanmış değeri ve geri dönen d , x , ve y değerleri. (d, x, y) üçlüsü, iterasyonun bir sonraki üst adımında kullanılan (d', x', y') üçlüsünü döndürür. EXTENDED-EUCLID(99,78) çağırısı, $(3, -11, 14)$ ü döndürür, öyle ki $\text{ebob}(99,78) = 3 = 99 \cdot (-11) + 78 \cdot 14$.

durumda EUCLID prosedürü iki β -bitlik sayıya uygulanıyorsa o zaman, $O(\beta)$ aritmetik işlem ve $O(\beta^3)$ bit işlemi gerekir. Burada β -bitlik sayıların çarpımlarının ve bölümlerinin $O(\beta^2)$ bit işlemi gerektirdiği varsayılmıştır. Problem 31-2 sizden $O(\beta^2)$ nin bit işlemlerinin sayısı ile sınırlı olduğunu göstermenizi ister.

Öklid algoritmasının genişletilmiş biçimi

Öklid Algoritmasını daha kullanışlı bilgiler elde etmek amacıyla yeniden yazalım. Özel olarak, algoritmayı tam sayılardan oluşan a ve b katsayılarına göre çalışacak şekilde genişletelim. Şöyle ki;

$$d = \text{ebob}(a, b) = ax + by \quad (31.16)$$

Burada a ve b katsayılarının sıfır veya negatif tam sayılar olabileceğinin altını çizmeliyiz. Bu katsayıları daha sonra modüler çarpımsal tersleri hesaplamada kullanışlı bulacağız. EXTENDED-EUCLID Algoritmasının prosedürü girdi olarak negatif olmayan iki tam sayı alır ve çıktı olarak (31.16) denklemini sağlayan (d, x, y) üçlüsünü verecektir.

EXTENDED-EUCLID(a, b)

```

1  if  $b == 0$ 
2      return  $(a, 1, 0)$ 
3  else  $(d', x', y') = \text{EXTENDED-EUCLID}(b, a \bmod b)$ 
4       $(d, x, y) = (d', y', x' - \lfloor a/b \rfloor y')$ 
5      return  $(d, x, y)$ 
```

Şekil 31.1 EXTENDED-EUCLID algoritmasının $\text{ebob}(99,78)$ için çalıştırılmış halini göstermektedir.

EXTENDED-EUCLID Algoritması, EUCLID prosedürünün bir varyasyonudur. 1 nolu satır, EUCLID in 1. satırındaki “ $b = 0$ ” testine denktir. EXTENDED-EUCLID

algoritması sadece satır 2 deki $d = a$ yı çıktı olarak vermez, ayrıca $x = 1$ ve $y = 0$ için $a = ax + by$ nin katsayılarını da gönderir. Eğer $b \neq 0$ ise, EXTENDED-EUCLID algoritması ilk olarak (d', x', y') yi hesaplar, öyle ki $d' = \text{ebob}(b, a \bmod b)$ ve

$$d' = bx' + (a \bmod b)y' \quad (31.17)$$

dir. EUCLID de olduğu gibi, bu durumda $d = \text{ebob}(a, b) = d' = \text{ebob}(b, a \bmod b)$. $d = ax + by$ için x ve y değerlerini elde etmek amacıyla (31.17) denklemini $d = d'$ eşitliğini ve (3.8). denklemi kullanırız.

$$\begin{aligned} d &= bx' + (a - b \lfloor a/b \rfloor)y' \\ &= ay' + b(x' - \lfloor a/b \rfloor y') . \end{aligned}$$

Böylelikle, $x = y'$ ve $y = x' - \lfloor a/b \rfloor y'$ seçilerek $d = ax + by$ denklemi sağlanmış olur; bu da EXTENDED-EUCLID algoritmasının doğruluğunu gösterir.

EUCLID algoritmasında yapılan özyinelemeli çağrılar ile EXTENDED-EUCLID algoritmasında yapılan özyinelemeli çağrılar eşit olduğundan, bu iki algoritmanın da çalışma zamanı aynıdır. $a > b > 0$ için özyinelemeli çağrı sayısı, $O(\lg b)$ dir.

Alıştırmalar

31.2-1

(31.11) ve (31.12) nolu denklemleri (31.13) ile ifade ederek kanıtlayınız.

31.2-2

EXTENDED-EUCLID(899,493) değerinin döndürdüğü (d, x, y) yi hesaplayınız.

31.2-3

a, k ve n tam sayıları için $\text{ebob}(a, n) = \text{ebob}(a + kn, n)$ olduğunu kanıtlayınız.

31.2-4

EUCLID'i hafızanın yalnızca belli bir miktarını kullanan iteratif biçimini yeniden yazınız (yani tam sayı değerlerinin yalnızca belli bir sayısını depolayan).

31.2-5

Eğer $a > b \geq 0$ ise, o zaman EUCLID(a, b) prosedürü en fazla $1 + \log_\phi b$ özyinelemeli çağrı yapabilir, gösteriniz. Bu sınırı, $1 + \log_\phi(b / \gcd(a, b))$ na geliştiriniz.

31.2-6

EXTENDED-EUCLID(F_{k+1}, F_k) geriye ne döndürür? Açıklayınız.

31.2-7

ebob fonksiyonunu birden fazla argümana bağlı olarak, $\gcd(a_0, a_1, \dots, a_n) = \gcd(a_0, \gcd(a_1, a_2, \dots, a_n))$ rekürsif ifadesiyle tanımlayınız. ebob fonksiyonunun, verilen argümanların sırasından bağımsız olacak şekilde aynı cevabı verdiğini gösteriniz. Ayrıca $x_0, x_1, \dots, x_n$ tam sayıları için $\gcd(a_0, a_1, \dots, a_n) = a_0x_0 + a_1x_1 + \dots + a_nx_n$ eşitliğini sağlayan x_i leri nasıl buluruz? Sizin algoritmanızla hesaplanan bölme işlemlerinin sayısının $O(n + \lg(\max\{a_0, a_1, \dots, a_n\}))$ olduğunu gösteriniz

31.2-8

$a_1, a_2, \dots, a_n$ gibi n tane tam sayı için **en küçük ortak kat** olacak şekilde $\text{lcm}(a_1, a_2, \dots, a_n)$ ifadesini, yani her a_i nin çarpımının en küçük negatif olmayan tamsayı değerini tanımlayınız. $\text{lcm } a_1, a_2, \dots, a_n$ nin, (iki argümanlı) ebob işlemini bir alt program şeklinde kullanarak nasıl hesaplandığını gösteriniz.

31.2-9

n_1, n_2, n_3 ve n_4 aralarında asal ikililerdir ancak ve ancak

$\text{ebob}(n_1, n_2, n_3, n_4) = \text{ebob}(n_1n_2n_3, n_4) = 1$ dir.

Daha genel bir ifadeyle, gösteriniz ki $n_1, n_2, \dots, n_k$ aralarında asal ikililerdir ancak ve ancak n_i den türetilmiş $\lceil \lg k \rceil$ tane sayı çiftinin bir kümesi aralarında asaldır. Modüler aritmetiği, tam sayılar üzerindeki genel aritmetik olarak düşünebi-

31.3 Modüler aritmetik

liriz, ancak istisnai olarak eğer modül n de çalışıyorsak o halde her x sonucu $\{0, 1, \dots, n-1\}$ kümesinin $x \bmod n$ e göre denk olan elemanlarının yer değiş-tirmesiyle oluşur. Eğer toplama, çıkarma ve çarpma işlemlerinden bahsediyorsak bu kuralsız model yeterlidir. Modüler aritmetiğin daha kurallı bir yöntemi ise şimdi vereceğimiz olan, grup teoreminin çalışma alanıyla açıklanabilir.

Sonlu gruplar

$(S, \oplus)$ **grubu**, S kümesi üzerinde aşağıdaki özellikleri sağlayan $\oplus$ ikili işlemini ifade eder:

1. **Kapalılık:** Her $a \oplus b \in S$ için $a \oplus b \in S$ dir.
2. **Birim Eleman:** Her $a \in S$ için $e \oplus a = a \oplus e = a$ koşulunu sağlayan $e \in S$ gibi bir eleman varsa bu elemana grubun **birimi** denir.
3. **Birleşme:** Her $a, b, c \in S$ için $(a \oplus b) \oplus c = a \oplus (b \oplus c)$ koşulu sağlanıyorsa **birleşmelidir** denir.

4. Ters Eleman : Her $a \in S$ için $a \oplus b = b \oplus a = e$ olacak şekilde öyle bir tek $b \in S$ elemanı varsa, b ye a **elemanının tersi** denir.

Örnek olarak, $(\mathbb{Z}, +)$ grubunu ele alalım, $\mathbb{Z}$ tam sayıları toplama işlemi altında bir gruptur. 0 birim elemandır, a elemanının tersi ise $-a$ ile ifade edilir. Eğer $(S, \oplus)$ grubu her $a, b \in S$ için $a \oplus b = b \oplus a$ **değişme özelliğini** sağlıyorsa, **değişmeli (abel) grup** olarak adlandırılır. Eğer $(S, \oplus)$ grubu $|S| < \infty$ koşulunu sağlıyorsa **sonlu gruptur**.

Modüler toplama ve çarpma işlemleri altında tanımlanan gruplar

n pozitif tam sayı olmak üzere; n modülüne göre toplama ve çarpma işlemlerini kullanarak iki tane sonlu grup oluşturabiliriz. Bu gruplar, Kısım 31.1 de tanımlanan n modülüne göre denklik sınıflarının elemanlarından oluşur.

$\mathbb{Z}_n$ e grup tanımlamak için uygun toplama ve çarpma işlemlerine ihtiyacımız vardır ve bunu da temel toplama ve çarpma işlemlerini yeniden tanımlayarak sağlayabiliriz. $\mathbb{Z}_n$ için toplama ve çarpma işlemlerini kolayca belirleyebiliriz çünkü iki tam sayının denklik sınıfı, toplamaları veya çarpımlarının denklik sınıflarını eşsiz olarak belirler. Bu da, eğer $a \equiv a' \pmod{n}$ ve $b \equiv b' \pmod{n}$ ise o halde,

$$\begin{aligned} a + b &\equiv a' + b' \pmod{n}, \\ ab &\equiv a'b' \pmod{n}. \end{aligned}$$

Böylece, toplama ve çıkarma işlemlerini modulo n altında $+_n$ ve $\cdot_n$ sembolleri ile gösteririz ve aşağıdaki gibi tanımlarız:

$$\begin{aligned} [a]_n +_n [b]_n &= [a + b]_n, \\ [a]_n \cdot_n [b]_n &= [ab]_n. \end{aligned} \tag{31.18}$$

($\mathbb{Z}_n$ üzerine çıkarma işlemini $[a]_n -_n [b]_n = [a - b]_n$ ile benzer şekilde gösteririz ancak bölme işlemi daha karmaşık şekilde ifade edilir.) Bu olgular; $\mathbb{Z}_n$ de hesaplamalar yaparken, her denklik sınıfındaki en küçük negatif olmayan elemanın temsilci olarak alınarak işlem yapılmasının uygun ve yaygın kullanımını destekler. Toplama, çıkarma ve çarpma işlemlerini genellikle bu temsilciler üzerinde yaparız ancak her x sonucunu temsilcisinin $x \bmod n$ ile ifade edilen sınıfı ile değiştiririz.

modulo n üzerinde toplamanın bu tanımını kullanırsak, **modülo n ye göre toplamsal grubu** $(\mathbb{Z}_n, +_n)$ ile ifade ederiz. modulo n nin toplamsal grubunun boyutu $|\mathbb{Z}_n| = n$ dir. 31.2(a) şekli $(\mathbb{Z}_6, +_6)$ grubunun işlem tablosunu gösterir.

Teorem 31.12

$(\mathbb{Z}_n, +_n)$ sonlu abel gruptur.

Kanıt (31.18) eşitliği $(\mathbb{Z}_n, +_n)$ nin kapalı olduğunu gösterir. $+_n$ için birleşme ve değişme özellikleri, $+$ işleminin birleşme ve değişme özelliklerinden hareketle gösterilir.

$+_6$	0	1	2	3	4	5
0	0	1	2	3	4	5
1	1	2	3	4	5	0
2	2	3	4	5	0	1
3	3	4	5	0	1	2
4	4	5	0	1	2	3
5	5	0	1	2	3	4

(a)

$\cdot_{15}$	1	2	4	7	8	11	13	14
1	1	2	4	7	8	11	13	14
2	2	4	8	14	1	7	11	13
4	4	8	1	13	2	14	7	11
7	7	14	13	4	11	2	1	8
8	8	1	2	11	4	13	14	7
11	11	7	14	2	13	1	8	4
13	13	11	7	1	14	8	4	2
14	14	13	11	8	7	4	2	1

(b)

Şekil 31.2 İki sonlu grup. Denklik sınıfları, temsilci elemanları ile gösterilir.

(a) $(\mathbb{Z}_6, +_6)$ grubu için. **(b)** $(\mathbb{Z}_{15}^*, \cdot_{15})$ grubu için.

$$\begin{aligned}
 ([a]_n +_n [b]_n) +_n [c]_n &= [a + b]_n +_n [c]_n \\
 &= [(a + b) + c]_n \\
 &= [a + (b + c)]_n \\
 &= [a]_n +_n [b + c]_n \\
 &= [a]_n +_n ([b]_n +_n [c]_n) ,
 \end{aligned}$$

$$\begin{aligned}
 [a]_n +_n [b]_n &= [a + b]_n \\
 &= [b + a]_n \\
 &= [b]_n +_n [a]_n .
 \end{aligned}$$

$(\mathbb{Z}_n, +_n)$ grubunun birim elemanı 0 dır ($[0]_n$). a elemanının, $[a]_n$, toplamsal tersi $[a]_n +_n [-a]_n = [a - a]_n = [0]_n$ olduğu için $[-a]_n$ veya $[n - a]_n$ dir. ■

modulo n üzerinde çarpma tanımından, **n modülünün çarpımsal grubunu** $(\mathbb{Z}_n^*, \cdot_n)$ ile ifade ederiz. Bu grubun elemanları, $\mathbb{Z}_n^*$ in n ile aralarında asal olan elemanlarından oluşur. Böylece her elemanın bir tek tersi mevcut olur.

$$\mathbb{Z}_n^* = \{[a]_n \in \mathbb{Z}_n : \gcd(a, n) = 1\}$$

kümesinin iyi tanımlı olduğunu görmek için $0 \leq a < n$ aralığında $a \equiv (a + kn) \pmod{n}$ $k \in \mathbb{Z}_n^*$ olduğu unutulmamalıdır. Alıştırma 31.2-3 den, $\text{ebob}(a, n) = 1$ gösterir ki tüm k tam sayıları için $\text{ebob}(a + kn, n) = 1$ dir. $[a]_n = \{a + kn : k \in \mathbb{Z}\}$ olduğundan, kümesi iyi tanımlıdır.

$\mathbb{Z}_{15}^*$ grubu için $\mathbb{Z}_{15}^* = \{1, 2, 4, 7, 8, 11, 13, 14\}$ dir. (Burada $[a]_{15}$ elemanı a ile gösterilmiştir, örneğin $[7]_{15}$ elemanı 7 ile gösterilmiştir). Şekil 31.2(b), $(\mathbb{Z}_{15}^*, \cdot_{15})$ grubunun işlem tablosunu gösterir. Örneğin, $\mathbb{Z}_{15}^*$ kümesinde çalışırken $8 \cdot 11 \equiv 13 \pmod{15}$ dir. Birim eleman 1 dir.

Teorem 31.13

$(\mathbb{Z}_n^*, \cdot_n)$ kümesi sonlu abel gruptur.

Kanıt Teorem 31.6 gösterir ki $(\mathbb{Z}_n^*, \cdot_n)$ kapalıdır. Birleşme ve değişme özellikleri $\cdot_n$ için Teorem 31.12 de $+_n$ için gösterildiği gibi gösterilebilir. Birim eleman $[1]_n$ dir. a , $\mathbb{Z}_n^*$ kümesinin elemanı olsun ve (d, x, y) EXTENDED-EUCLID(a, n) algoritmasının çıktısı olmak üzere ters elemanların varlığı gösterilebilir. O halde, $a \in \mathbb{Z}_n^*$ olduğundan $d = 1$ dir ve

$$ax + ny = 1 \quad (31.19)$$

veya buna denk olarak

$$ax \equiv 1 \pmod{n}$$

dir. Böylece, $[x]_n$, n modülüne göre $[a]_n$ elemanının tersi olur. Dahası, $[x]_n \in \mathbb{Z}_n^*$ dir. Çünkü, x ile n in en küçük doğrusal pozitif kombinasyonu, Eşitlik (31.19) dan, 1 dir. Böylece Teorem 31.2 gösterir ki $\text{ebob}(x, n) = 1$ dir. Tersin tekliğinin kanıtını, Sonuç (31.26) ya bırakıyoruz. ■

Çarpımsal terslerin hesaplanmasına örnek olarak, $a = 5$ ve $n = 11$ alalım. EXTENDED-EUCLID(a, n) algoritmasının çıktısı $(d, x, y) = (1, -2, 1)$ olarak döndürür. Böylece, $1 = 5 \cdot (-2) + 11 \cdot 1$ olur. Buradan, $[-2]_{11}$, $[5]_{11}$ elemanının tersidir.

Bu bölümün kalan kısmında $(\mathbb{Z}_n, +_n)$ ve $(\mathbb{Z}_n^*, \cdot_n)$ grupları üzerinde çalışırken, $+_n$ ve $\cdot_n$ operatörlerini $+$ ve $\cdot$ ile ifade edeceğiz; kalan sınıflarını temsilcilerle göstereceğiz. (veya bitişik olma, yani, $ab = a.b$) Aynı zamanda, modulo n denklilikleri $\mathbb{Z}_n$ de ki denklemler gibi yorumlanabilecektir. Örneğin,

$$\begin{aligned} ax &\equiv b \pmod{n}, \\ [a]_n \cdot_n [x]_n &= [b]_n. \end{aligned}$$

ifadeleri denktir.

Benzer şekilde, $(S, \oplus)$ grubunun ifadesi için yalnızca S şeklinde yazacağız. $(\mathbb{Z}_n, +_n)$ ve $(\mathbb{Z}_n^*, \cdot_n)$ gruplarını ise sırasıyla $\mathbb{Z}_n$ ve $\mathbb{Z}_n^*$ ile ifade edeceğiz.

a elemanının tersini $(a^{-1} \pmod{n})$ ile göstereceğiz. $\mathbb{Z}_n^*$ de bölme işlemi ise, $a/b \equiv ab^{-1}$ ile tanımlanır. Örneğin, $7^{-1} \equiv 13 \pmod{15}$ dir, çünkü $7 \cdot 13 = 91 \equiv 1 \pmod{15}$ dir. Buradan, $4/7 \equiv 4 \cdot 13 \equiv 7 \pmod{15}$.

$\mathbb{Z}_n^*$ kümesini boyutu $\phi(n)$ dir. Bu fonksiyon, **Euler ϕ fonksiyonu** olarak bilinir. Aşağıdaki eşitliği sağlar:

$$\phi(n) = n \prod_{p: p \text{ asal ve } p \mid n} \left(1 - \frac{1}{p}\right), \quad (31.20)$$

p, n yi bölen tüm asalları tarar (eğer n asal ise n nin kendisini de alır).

Kanıtını burada vermeyeceğiz. Sezgisel olarak, n kalanlarının listesiyle başlayacağız, $\{0, 1, \dots, n-1\}$ sonra n yi bölen her p asalı için p nin katlarını listeden sileceğiz. Örneğin, 45 in asal bölenleri 3 ve 5 için

$$\begin{aligned} \phi(45) &= 45 \left(1 - \frac{1}{3}\right) \left(1 - \frac{1}{5}\right) \\ &= 45 \left(\frac{2}{3}\right) \left(\frac{4}{5}\right) \\ &= 24. \end{aligned}$$

dür. Eğer p asalsa, o halde $\mathbb{Z}_p^* = \{1, 2, \dots, p-1\}$ ve

$$\begin{aligned} \phi(p) &= p \left(1 - \frac{1}{p}\right) \\ &= p-1. \end{aligned} \quad (31.21)$$

Eğer n bileşik ise, $\phi(n) < n-1$ dir, bununla birlikte gösterilebilir ki

$$\phi(n) > \frac{n}{e^\gamma \ln \ln n + \frac{3}{\ln \ln n}} \quad (31.22)$$

$n \geq 3$ için $\gamma = 0.5772156649 \dots$ **Euler sabitidir**.

$n > 5$ için oldukça basit(daha gevşek) bir alt sınır

$$\phi(n) > \frac{n}{6 \ln \ln n} \quad (31.23)$$

$$\liminf_{n \rightarrow \infty} \frac{\phi(n)}{n / \ln \ln n} = e^{-\gamma} \quad (31.24)$$

olduğundan (31.22) alt sınır en mümkün olandır.

Altgruplar

Eğer, $(S, \oplus)$ bir grupsa, $S' \subseteq S$ ve $(S', \oplus)$ de bir grup ise o halde, $(S', \oplus)$ grubu, $(S, \oplus)$ nin bir **altgrubudur** denir. Örneğin, çift sayılar tam sayılar kümesinde toplama işlemi altında bir alt gruptur. Aşağıdaki teorem altgruplar tanımımızı kolaylaştırır.

Teorem 31.14 (Sonlu bir grubun boş olmayan kapalı bir alt kümesi, altgruptur)

Eğer $(S, \oplus)$ sonlu bir grup, S' de S nin boş olmayan herhangi bir alt kümesi olmak üzere, her $a, b \in S'$ için $a \oplus b \in S'$ oluyorsa, $(S', \oplus)$ $(S, \oplus)$ nin bir altgrubudur.

Kanıt Alıştırma 31.3-3 için bırakılmıştır. ■

Örneğin, $\{0, 2, 4, 6\} \mathbb{Z}_8$ in bir altgrubudur çünkü $+$ işlemi altında kapalı ve sonludur (yani, $+$ altında kapalıdır).

Aşağıdaki teorem bir altgrubu boyutu için son derece kullanışlı bir sınırlama sağlar; kanıtına yer vermedik.

Teorem 31.15 (Lagrange teoremi)

Eğer $(S, \oplus)$ sonlu bir grup ve $(S', \oplus)$ grubu, $(S, \oplus)$ nin bir altgrubu ise; $|S'|$ değeri $|S|$ yi böler. ■

Eğer $S \neq S'$ ise S nin S' altgrubu **öz** altgrubudur. Bölüm 31.8 de Miller-Rabin Asallık Testinde aşağıdaki sonucu kullanacağız.

Sonuç 31.16

Eğer S' , S sonlu grubunun öz altgrubu ise o zaman; $|S'| \leq |S|/2$ dir. ■

Elemanlarla belirlenen altgruplar

Teorem 31.14 $(S, \oplus)$ sonlu grubunun altgruplarını belirlememiz için yol gösterir: a elemanını seçelim ve grup işlemleri altında a ile belirlenen diğer elemanları alalım. Özellikle, $k \geq 1$ için $a^{(k)}$ yı tanımlayalım:

$$a^{(k)} = \bigoplus_{i=1}^k a = \underbrace{a \oplus a \oplus \cdots \oplus a}_{k \text{ tane}} .$$

Örneğin, eğer $\mathbb{Z}_6$ için $a = 2$ elemanını alırsak $a^{(1)}, a^{(2)}, a^{(3)}, \dots$ dizisi $2, 4, 0, 2, 4, 0, 2, 4, 0, \dots$ olur.

$\mathbb{Z}_n$ grubunda $a^{(k)} = ka \pmod{n}$ dir ve $\mathbb{Z}_n^*$ grubu için $a^{(k)} = a^k \pmod{n}$ dir. **a nın ürettiği altgrubu** tanımlarken veya $(a, \oplus)$ sembolleri kullanılır ve

$\langle a \rangle = \{a^{(k)} : k \geq 1\}$ ile ifade edilir.

a , $\langle a \rangle$ altgrubunu **üretir** veya $\langle a \rangle$ altgrubunun **üretici** a dır denir. S sonlu olduğundan, $\langle a \rangle$ S nin sonlu alt kümesidir, genellikle S nin hepsini içerir. $\oplus$ nin birleşmeli olmasından dolayı

$$a^{(i)} \oplus a^{(j)} = a^{(i+j)}$$

$\langle a \rangle$ kapalıdır buradan Teorem 31.14 ile $\langle a \rangle$ S nin bir altgrubudur. Örneğin, $\mathbb{Z}_6$ için

$$\langle 0 \rangle = \{0\} ,$$

$$\langle 1 \rangle = \{0, 1, 2, 3, 4, 5\} ,$$

$$\langle 2 \rangle = \{0, 2, 4\} .$$

Benzer şekilde $\mathbb{Z}_7^*$ için

$$\langle 1 \rangle = \{1\} ,$$

$$\langle 2 \rangle = \{1, 2, 4\} ,$$

$$\langle 3 \rangle = \{1, 2, 3, 4, 5, 6\}$$

olarak belirlenir.

a nın mertebesi (S grubu içindeki) $\text{ord}(a)$ ile gösterilir ve $a^{(t)} = e$ yi sağlayan en küçük pozitif t tam sayısı ile tanımlanır.

Teorem 31.17

Herhangi sonlu $(S, \oplus)$ grubu ve $a \in S$ için a nın mertebesi ürettiği altgrup sayısına eşittir veya $\text{ord}(a) = |\langle a \rangle|$ ile gösterilir.

Kanıt $t = \text{ord}(a)$ olsun. $a^{(t)} = e$ ve $k \geq 1$ için $a^{(t+k)} = a^{(t)} \oplus a^{(k)} = a^{(k)}$ olduğundan dolayı; eğer $i > t$ ise o zaman bazı $j < i$ değerleri için $a^{(i)} = a^{(j)}$ dir. Buradan, a ile elemanlar oluşturduğumuzda $a^{(t)}$ den sonra yeni elemanlar oluşmayacağı görülür. Bu da, $\langle a \rangle = \{a^{(1)}, a^{(2)}, \dots, a^{(t)}\}$ dir böylelikle $|\langle a \rangle| \leq t$ olur. $|\langle a \rangle| \geq t$ olduğunu göstermek için $a^{(1)}, a^{(2)}, \dots, a^{(t)}$ dizisindeki tüm elemanların ayrıık olduğunu göstermek gerekir. $1 \leq i < j \leq t$ koşulunu sağlayan i ve j değerleri için $a^{(i)} = a^{(j)}$ nin çelişki olduğunu gösterelim. $k \geq 0$ için $a^{(i+k)} = a^{(j+k)}$ olur. Bu eşitlik ise, $a^{(i+(t-j))} = a^{(j+(t-j))} = e$ yi ifade eder, bu çelişkidir, çünkü $i + (t-j) < t$ dir fakat $t a^{(t)} = e$ koşulunu sağlayan en küçük pozitif değerdir. Buradan, $a^{(1)}, a^{(2)}, \dots, a^{(t)}$ dizisindeki her eleman ayrııktır ve $|\langle a \rangle| \geq t$ olur. Buradan $\text{ord}(a) = |\langle a \rangle|$ olduğu sonucunu çıkarırız. ■

Sonuç 31.18

$a^{(1)}, a^{(2)}, \dots, a^{(t)}$ dizisi $t = \text{ord}(a)$ periyotludur. Bu da $a^{(i)} = a^{(j)}$ ancak ve ancak $i \equiv j \pmod{t}$ dir. ■

Yukarıdaki sonuca uygun olarak, her $i \in \mathbb{Z}$ için. $a^{(0)} = e$ ve $a^{(i)} = a^{(i \bmod t)}$, $t = \text{ord}(a)$ dir.

Sonuç 31.19

Eğer $(S, \oplus)$ grubu, e birim elemanına sahip bir sonlu grup ise, tüm $a \in S$ için $a^{(|S|)} = e$ dir.

Kanıt Lagrange teoremi (Teorem 31.15); $\text{ord}(a) \mid |S|$ ve dolayısıyla $t = \text{ord}(a)$ durumunda $|S| \equiv 0 \pmod{t}$ olduğunu olduğunu belirtir. Bu yüzden $a^{(|S|)} = a^{(0)} = e$ dir. ■

Alıştırmalar

31.3-1

$(\mathbb{Z}_4, +_4)$ ve $(\mathbb{Z}_5^*, \cdot_5)$ grupları için grup işlem tablolarını çizin. $\alpha(a) \cdot \alpha(b) \equiv \alpha(c) \pmod{5}$ ancak ve ancak $a + b \equiv c \pmod{4}$ olacak şekilde bileşenler arasındaki birebir eşlemeyi (α) göstererek, bu grupların izomorfik olduğunu gösteriniz.

31.3-2

$\mathbb{Z}_9$ ve $\mathbb{Z}_{13}^*$ e ait tüm altgrupları listeleyin.

31.3-3

Teorem 31.14 ü kanıtlayın.

31.3-4

p nin asal sayı olması ve e nin pozitif bir tam sayı olması halinde şunu gösterin:

$$\phi(p^e) = p^{e-1}(p-1)$$

31.3-5

Herhangi bir tam sayı ($n > 1$) ve $a \in \mathbb{Z}_n^*$ durumu için $f_a(x) = ax \pmod{n}$ ile tanımlanan $f_a : \mathbb{Z}_n^* \rightarrow \mathbb{Z}_n^*$ fonksiyonunun $\mathbb{Z}_n^*$ in permütasyonu olduğunu gösterin.

31.4 Modüler doğrusal denklemleri çözme

Şimdi, $a > 0$ ve $n > 0$ olması durumu için

$$ax \equiv b \pmod{n} \tag{31.25}$$

denklemini için çözüm bulunması sorununu ele alacağız. Bu sorunun çeşitli uygulamaları vardır; örneğin bunu, Kısım 31.7 deki RSA ortak anahtarlı şifreleme sistemi dahilinde anahtar bulma prosedürünün bir parçası olarak kullanacağız. a , b ve n nin verilmiş olduğunu varsayıp denklemini tamamlayan tüm x değerlerini ve modulo n i bulmayı arzu ediyoruz (31.25). Denklem sıfır, bir ya da birden fazla olmak üzere bu tip bir çözüme sahip olabilmektedir.

$\langle a \rangle$, a tarafından oluşturulan $\mathbb{Z}_n$ alt grubunu gösterebiliriz. $\langle a \rangle = \{a^{(x)} : x > 0\} = \{ax \pmod{n} : x > 0\}$ olması dolayısıyla, ancak ve ancak $[b] \in \langle a \rangle$ durumunda denklem (31.25) bir çözüme sahip olur. Lagrange teoremi bize (Teorem 31.15) $|\langle a \rangle|$ nın, n in bir böleni olmak durumunda olduğunu söyler. Aşağıdaki teorem, bize $\langle a \rangle$ nın uygun biçimde sınıflandırılışını sunmaktadır.

Teorem 31.20

Pozitif tam sayı a ve n için $d = \text{ebob}(a, n)$ durumunda, $\mathbb{Z}_n$ içinde

$$\langle a \rangle = \langle d \rangle = \{0, d, 2d, \dots, ((n/d) - 1)d\} \quad (31.26)$$

olur ve bu yüzden $|\langle a \rangle| = n/d$ dir.

Kanıt $d \in \langle a \rangle$ olduğunu göstererek başlıyoruz. EXTENDED-EUCLID(a, n) in , $ax' + ny' = d$ olacak şekilde x' ve y' tam sayılarını oluşturacağını unutmayınız. Dolayısıyla $ax' \equiv d \pmod{n}$ olur; böylelikle $d \in \langle a \rangle$ olur. Başka bir deyişle d , $\mathbb{Z}_n$ kümesinde a nın katı olur.

$d \in \langle a \rangle$ nedeniyle, d nin her katının $\langle a \rangle$ ya ait olduğu sonucu çıkar, çünkü a nın bir katının katı, a nın katının kendisi olmaktadır. Bu yüzden, $\{0, d, 2d, \dots, ((n/d) - 1)d\}$ dahilindeki her birleşeni ihtiva etmektedir. Bu ise, $\langle d \rangle \subseteq \langle a \rangle$ demektir.

Şimdi, $\langle a \rangle \subseteq \langle d \rangle$ durumunu göstereceğiz. Eğer $m \in \langle a \rangle$ ise, x tamsayısı için $m = ax \pmod{n}$ olur ve dolayısıyla y tam sayısı için $m = ax + ny$ olur. Bununla birlikte $d | a$ ve $d | n$ ve dolayısıyla $d | m$ denklem dahilinde olur (31.4). Bu yüzden $m \in \langle d \rangle$ olur.

Bu sonuçları topladığımızda, $\langle a \rangle = \langle d \rangle$ sonucuna ulaşırız. $|\langle a \rangle| = n/d$ olduğunu görmek için 0 ve $n - 1$ ($n - 1$ dahil) arasında d nin n/d katlarının tam olarak mevcut olduğunu gözlemleyin. ■

Sonuç 31.21

$ax \equiv b \pmod{n}$ denklemi x bilinmeyeni için çözülebilirdir ancak ve ancak $d = \text{ebob}(a, n)$ iken $d | b$ ise.

Kanıt Eğer $[b] \in \langle a \rangle$ ancak ve ancak $ax \equiv b \pmod{n}$ denklemi bilinmeyen x değerleri için çözümlenebilmektedir ifadesi, teorem (31.20) belirtildiği gibi

$$(b \pmod{n}) \in \{0, d, 2d, \dots, ((n/d) - 1)d\}$$

demekle aynı anlamdadır. Üyelerinin tam olarak d nin katları olması dolayısıyla, $0 \leq b < n$ durumunda, $b \in \langle a \rangle$ dır ancak ve ancak $d | b$ olduğunda. Eğer $b < 0$ veya $b \geq n$ ise ve b ile $b \pmod{n}$ nin katları ile farklılık gösterdiğinden (kendisi d nin katlarıdır) sonuç $d | b$ dir ancak ve ancak $d | (b \pmod{n})$ gözleminden açıklanabilir. ■

Sonuç 31.22

$ax \equiv b \pmod{n}$ denklemi, $d = \text{ebob}(a, n)$ durumunda d ayrık/distinct çözümlere sahip olabilir ya da çözümsüz kalır.

Kanıt $ax \equiv b \pmod{n}$ nin bir çözüme sahip olması durumunda, $b \in \langle a \rangle$ olur. Teorem 31.17 e göre $\text{ord}(a) = |\langle a \rangle|$ olur ve Sonuç 31.18 ve Teorem 31.20, $i = 0, 1, \dots$, için $a_i \pmod{n}$ dizisinin $|\langle a \rangle| = n/d$ periyodu ile periyodik olduğunu belirtir. i değerleri, 0 dan $(n - 1)$ e kadar artış gösterirken artış gösterirken $\langle a \rangle$

değerlerinin tam olarak d kere tekrarlandığı (n/d) uzunluklu bloğundan dolayı, $b \in \langle a \rangle$ durumunda $b, ai \bmod n$ dizisi dahilinde tam d defa yer alır. $ax \bmod n = b$ için olan d konumlarının x indisleri $ax \equiv b \pmod{n}$ denklemi için çözüm olabilmektedir. ■

Teorem 31.23

$d = \text{ebob}(a, n)$ koşulunu ve bazı x' ve y' tam sayıları için $d = ax' + ny'$ olduğunu varsayın (örneğin EXTENDED-EUCLID ile hesaplandığı şekilde). $d \mid b$ olması halinde, $x_0 = x'(b/d) \bmod n$ durumunda, $ax \equiv b \pmod{n}$ denklemi kendi çözümlerinden biri olarak x_0 değerine sahip olur.

Kanıt Elimizde olan;

$$\begin{aligned} ax_0 &\equiv ax'(b/d) \pmod{n} \\ &\equiv d(b/d) \pmod{n} \quad (ax' \equiv d \pmod{n} \text{ olduğu için}) \\ &\equiv b \pmod{n}, \end{aligned}$$

$ax \equiv b \pmod{n}$ denkleminin çözümü x_0 dır. ■

Teorem 31.24

$ax \equiv b \pmod{n}$ denkleminin çözümlenebilir (yani $d = \text{ebob}(a, n)$ durumunda $d \mid b$ dir) ve bu denklemin her türlü sonucunun x_0 olacağını varsayın. O zaman bu denklem, $i = 0, 1, \dots, d-1$ için $x_i = x_0 + i(n/d)$ denklemi tarafından sunulan d farklı çözümlere sahip olur (modulo n).

Kanıt $i = 0, 1, \dots, d-1$ için; $n/d > 0$ ve $0 \leq i(n/d) < n$ olduğu için $x_0, x_1, \dots, x_{d-1}$ değerleri modulo n için her biri farklıdır. $x_0, ax \equiv b \pmod{n}$ nin bir çözümü olduğu için $ax_0 \bmod n \equiv b$ denklemini elde ederiz. Bu yüzden, $i = 0, 1, \dots, d-1$ için aşağıdakilere sahibiz;

$$\begin{aligned} ax_i \bmod n &= a(x_0 + in/d) \bmod n \\ &= (ax_0 + ain/d) \bmod n \\ &= ax_0 \bmod n \quad \text{Çünkü } d \mid a, ain/d \text{ nin } n \text{ nin} \\ &\equiv b \pmod{n}, \quad \text{bir katı olduğunu belirtmektedir} \end{aligned}$$

ve dolayısıyla x_i yi çözüm haline getiren, $ax_i \equiv b \pmod{n}$ e de sahibiz. Sonuç 31.22 itibariyle, $ax \equiv b \pmod{n}$ tamamıyla d çözümlerine sahiptir, böylelikle $x_0, x_1, \dots, x_{d-1}$ tümüyle bunlar olmalıdır. ■

Şuan itibariyle $ax \equiv b \pmod{n}$ denklemini çözmek için ihtiyaç duyulan matematiği geliştirmiş durumdayız; aşağıdaki algoritma, bu denklemin tüm çözümlerini verir. a ve n girdileri keyfi pozitif tam sayılardır ve b keyfi bir tam sayıdır.

MODULAR–LINEAR–EQUATION–SOLVER(a, b, n)

1. $(d', x', y') = \text{EXTENDED-EUCLID}(a, n)$
2. **if** $d \mid b$
3. $x_0 = x' (b / d) \bmod n$
4. **for** $i = 0$ **to** $d - 1$
5. print $(x_0 + i(n / d)) \bmod n$
6. **else** print “çözüm yoktur.”

Bu prosedürün işleyişine örnek olarak, $14x \equiv 30 \pmod{100}$ denklemini inceleyelim (Burada $a = 14$, $b = 30$ ve $n = 100$ dür). 1. sıradaki EXTENDED–EUCLID algoritmasında başvurduğumuzda, $(d', x', y') = (2, -7, 1)$ sonucunu elde ederiz. $2 \mid 30$ olması dolayısıyla, 3 ile 5. satırlar işleme girer. Satır 3, $x_0 = (-7)(15) \bmod 100 = 95$ şeklinde hesaplamada bulunur. 4 ile 5. satırlar üzerindeki döngü 95 ve 45 olmak iki çözüm çıktısı verir.

MODULAR–LINEAR–EQUATION–SOLVER prosedürü aşağıdaki gibi çalışmaktadır. 1. satır, x' değerinin $ax' \equiv b \pmod{n}$ denkleminin bir çözümü olduğunu göstererek, $d = ax' + ny'$ olacak şekilde x' ve y' olmak üzere iki değer ile birlikte $d = \text{ebob}(a, n)$ hesaplamasında bulunur. d nin b yi bölmemesi durumunda, Sonuç 31.21 e göre $ax \equiv b \pmod{n}$ denklemi çözümsüz kalır. Satır 2, $d \mid b$ koşulunu kontrol eder; koşul sağlanmazsa 6. satır çözüm olmadığını rapor eder. Aksi halde satır 3 Teorem 31.23 e göre $ax \equiv b \pmod{n}$ denklemi için x_0 çözümünü hesaplar. Tek çözüm dikkate alındığında Teorem 31.24, (n/d) nin katlarını, modülo n , eklenmesinin diğer $d - 1$ çözümlerini verimli kılacağını belirtmektedir. Satır 4-5 satırlarındaki **for** döngüsü, x_0 dan modulo n nin berisinde yer alan n/d den başlayarak tüm d çözümlerini verir.

$O(\lg n + \text{ebob}(a, n))$ EXTENDED–EUCLID $O(\lg n)$ aritmetik işlemleri icra etmesi ve 4-5 satırlarındaki **for** döngüsünün her iterasyonunun sabit bir sayıda işlem gerçekleştirmesi dolayısıyla MODULAR–LINEAR–EQUATION–SOLVER aritmetik işlemlerini gerçekleştirir.

Teorem 31.24 e ait aşağıdaki sonuçlar bazı özelleştirmeleri vermemize olanak sağlar.

Sonuç 31.25

$n > 1$ için $\text{ebob}(a, n) = 1$ olması halinde, $ax \equiv b \pmod{n}$ denklemi modulo n ye göre tek bir çözüme sahiptir. ■

$b = 1$ durumunda, aradığımız x , a nın modülo n ye göre *çarpımsal tersidir*.

Sonuç 31.26

$n > 1$ durumunda, $\text{ebob}(a, n) = 1$ olması halinde, $ax \equiv 1 \pmod{n}$ denkleminin tek bir çözümü vardır. Aksi halde yoktur. ■

$a^{-1} \bmod n$ Sonuç 31.26 sayesinde, a ve n nin aralarında asal olmaları durumunda a nın çarpımsal tersini belirtmek için notasyonunu kullanabiliriz. $\text{ebob}(a, n) = 1$ durumunda, $ax \equiv 1 \pmod{n}$ denkleminin tekil çözümü EXTENDED-EUCLID algoritmasıyla döndürülen x değeridir, bu da $\text{ebob}(a, n) = 1 = ax + ny$ denkleminin $ax \equiv 1 \pmod{n}$ ifadesinden kaynaklanır. Böylece, $a^{-1} \bmod n$ değeri EXTENDED-EUCLID ile hesaplanabilir.

Alıştırmalar

31.4-1

$35x \equiv 10 \pmod{50}$ denklemi için tüm çözümleri bulun.

31.4-2

$ax \equiv ay \pmod{n}$ nin, $\text{ebob}(a, n) = 1$ olması durumunda $x \equiv y \pmod{n}$ i belirttiğini kanıtlayın. $\text{ebob}(a, n) = 1$ koşulunun, $\text{ebob}(a, n) > 1$ ile karşı örneğini sunarak gerekli olduğunu gösterin.

31.4-3

MODULA-LINEAR-EQUATION-SOLVER prosedürünün 3. satırına yönelik aşağıdaki değişikliği değerlendiriniz:

$$3 \quad x_0 = x'(b/d) \bmod (n/d)$$

Bu komut çalışır mı? Nedenini veya neden olmadığını açıklayınız.

31.4-4 *

p asal sayı olsun ve $f(x) \equiv f_0 + f_1x + \cdots + f_tx^t \pmod{p}$ polinomial denklemi t .dereceden ve $\mathbb{Z}_p$ den alınan katsayılarıyla belirlenmiş olsun. $f(a) \equiv 0 \pmod{p}$ durumunda, a nın f fonksiyonunun *sıfırı* olduğunu söyleriz. a nın f nin sıfırı olması durumunda, $f(x) \equiv (x - a)g(x)$ nin, $t - 1$ polinom derecesinden bazı $g(x)$ ler için sağlandığını gösteriniz. t üzerinden tümevarımla, p nin asal sayı olması halinde, t polinom dereceli $f(x)$ in en çok t farklı sıfırlara (modulo n için) sahip olduğunu kanıtlayın.

31.5 Çin kalan teoremi

M.S 100. yıl civarlarında, Çinli matematikçi Sun-Tsu 3, 5 ve 7 ile bölündüğünde sırasıyla 2, 3 ve 2 kalanını veren x tam sayılarını arayan problemi çözdü. $x \equiv 23$ bu problemin bir çözümüdür; diğer tüm çözümler k keyfi bir tam sayı olmak

üzere $23 + 105k$ biçiminde yazılabilir. "Çin kalan teoremi", aralarında ikişerli asal olan sayıların modülleri ile bu sayıların çarpımlarının modülleri arasında bir denklik kurar.

Çin kalan teoreminin iki büyük uygulaması vardır. Varsayalım ki n tamsayısı n_i ler aralarında asal ikililer olmak üzere; $n = n_1 n_2 \cdots n_k$ ile ifade edilsin. İlk olarak, Çin kalan teoremi, $\mathbb{Z}_n$ halkasının yapısını $\mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times \cdots \times \mathbb{Z}_{n_k}$ Kartezyen çarpımı ile ifade etmemize yarar. İkinci olarak, bit işlemlerinde $\mathbb{Z}_{n_i}$ sınıflarında çalışmak modül n de çalışmaktan daha kolay olduğu için daha etkin algoritmalar oluşturmamızı sağlar.

Teorem 31.27 (Çin kalan Teoremi)

$n = n_1 n_2 \cdots n_k$ ler aralarında asal ikililerolsun. Aşağıdaki eşlemeyi inceleyelim:

$$a \leftrightarrow (a_1, a_2, \dots, a_k) \quad (31.27)$$

$$a \in \mathbb{Z}_n, a_i \in \mathbb{Z}_{n_i} \text{ ve}$$

$$i = 1, 2, \dots, k \text{ için } a_i = a \bmod n_i$$

(31.27) eşlemesi $\mathbb{Z}_n$ ve $\mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times \cdots \times \mathbb{Z}_{n_k}$ Kartezyen çarpımları arasında bire-bir eşlemedir (bijeksiyondur). $\mathbb{Z}_n$ nin elemanları üzerinde uygulanmış olan işlemler, bağımsız olarak uygun sistemdeki her bir koordinat pozisyonunda işlemler uygulayarak, uygun k -lı lar üzerinde eşit bir biçimde uygulanmış olabilir.

$$a \leftrightarrow (a_1, a_2, \dots, a_k)$$

$$b \leftrightarrow (b_1, b_2, \dots, b_k)$$

ise o zaman,

$$(a + b) \bmod n \leftrightarrow ((a_1 + b_1) \bmod n_1, \dots, (a_k + b_k) \bmod n_k), \quad (31.28)$$

$$(a - b) \bmod n \leftrightarrow ((a_1 - b_1) \bmod n_1, \dots, (a_k - b_k) \bmod n_k), \quad (31.29)$$

$$(ab) \bmod n \leftrightarrow (a_1 b_1 \bmod n_1, \dots, a_k b_k \bmod n_k) \quad (31.30)$$

ile ifade edilir.

Kanıt İki gösterim arasındaki dönüşüm, oldukça anlaşılırdır. a dan $(a_1, a_2, \dots, a_k)$ ya gitmek oldukça kolaydır ve yalnızca k adet "mod" işlemlerini gerektirmektedir.

$(a_1, a_2, \dots, a_k)$ girdilerinden a nın hesaplanması biraz daha karışıktır. $i = 1, 2, \dots, k$ için $m_i = n/n_i$ tanımlayarak başlıyoruz; bu yüzden n_i : $m_i = n_1 n_2 \cdots n_{i-1} n_{i+1} \cdots n_k$ lardan farklı olarak n_j lerin tümünün çarpımı olmaktadır. Sonrasında $i = 1, 2, \dots, k$ için

$$c_i = m_i(m_i^{-1} \bmod n_i) \quad (31.31)$$

dir.

(31.31) denklemi, m_i ve n_i ler aralarında asal olduğu sürece (Teorem 31.6 dan) her zaman iyi tanımlıdır. Sonuç 31.26 $m_i^{-1} \bmod n_i$ nin var olduğunu garantiler. Sonuç olarak, a değerini $a_1, a_2, \dots, a_k$ ların fonksiyonu olarak aşağıdaki şekilde ifade ederiz:

$$a \equiv (a_1c_1 + a_2c_2 + \dots + a_kc_k) \quad (31.32)$$

Şimdi denklem (31.32) nin, $i = 1, 2, \dots, k$ için $a \equiv a_i \pmod{n_i}$ sağladığını gösterelim. $j \neq i$ durumunda, $m_j \equiv 0 \pmod{n_i}$ dir, bu da $c_j \equiv m_j \equiv 0 \pmod{n_i}$ olduğunu gösterir. Denklem (31.31) itibariyle, $c_i \equiv 1 \pmod{n_i}$ olduğunu dikkate alınız. Böylece faydalı bir eşleme olan $c_i \leftrightarrow (0, 0, \dots, 0, 1, 0, \dots, 0)$ bulunur. Bu vektör i . koordinatta 1 değerini, diğer tüm koordinatlarda 0 değerini alır. c_i bir bağlamda temsil için esas teşkil eder. Her i için buradan

$$\begin{aligned} a &\equiv a_i c_i && \pmod{n_i} \\ &\equiv a_i m_i (m_i^{-1} \bmod n_i) && \pmod{n_i} \\ &\equiv a_i && \pmod{n_i} \end{aligned}$$

denklemini elde ediyoruz: a_i lerden a hesaplama yöntemimiz $i = 1, 2, \dots, k$ için kısıtlarını karşılayan bir a sonucu üretmektedir. $i = 1, 2, \dots, k$ ve her x için $x \bmod n_i = (x \bmod n)$ olduğundan, (31.28)-(31.30) denklemleri Alıştırma (31.1-7) den hareketle gösterilir. ■

Aşağıdaki sonuçlardan bu bölüm dahilinde daha sonra faydalanacağız.

Sonuç 31.28

Eğer $n = n_1 n_2 \dots n_k$ aralarında asal ikililer ve $n = n_1 n_2 \dots n_k$ ise; $a_1, a_2, \dots, a_k$ herhangi tam sayıları için eş zamanlı denklemlerin kümesi

$$x \equiv a_i \pmod{n_i},$$

$i = 1, 2, \dots, k$ bilinmeyen x için modulo n ye göre tek bir çözüme sahiptir. ■

Sonuç 31.29

$n = n_1 n_2 \dots n_k$ nin aralarında ikişerli asal olması ve $n_1, n_2, \dots, n_k$ durumunda, tüm x ve a tam sayıları için;

$$i = 1, 2, \dots, k \text{ için } x \equiv a \pmod{n_i} \text{ ancak ve ancak}$$

$$x \equiv a \pmod{n} \text{ olur.} \quad \blacksquare$$

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	40	15	55	30	5	45	20	60	35	10	50	25
1	26	1	41	16	56	31	6	46	21	61	36	11	51
2	52	27	2	42	17	57	32	7	47	22	62	37	12
3	13	53	28	3	43	18	58	33	8	48	23	63	38
4	39	14	54	29	4	44	19	59	34	9	49	24	64

Şekil 31.3 Çin kalan teoreminin $n_1 = 5$ ve $n_2 = 13$ için uygulanaşı yukarıdaki gibidir. Bu örnekte, $c_1 = 26$ ve $c_2 = 40$ tır. i . satır ve j . sütun a nın değerini modulo 65 için gösterir, öyle ki $a \bmod 5 = i$ ve $a \bmod 13 = j$. Unutmayınız ki 0. satır ve 0. sütun 0 değerini alır. Benzer şekilde, 4. satır ve 12. sütun 64 değerini içerir (-1 e denktir.) $c_1 = 26$ olduğundan bir satır aşağı inmek demek a yı 26 artırmak demektir. Yine benzer olarak, $c_2 = 40$ ifadesi sütunda bir kez sağa hareket edildiğinde a nın değerini 40 artırmak anlamındadır. a yı 1 artırmak ise, köşegen üzerinde sağa aşağı doğru hareket, alttan yukarı doğru ve sağdan sola doğru hareket ile mümkündür.

Çin kalan teoreminin bir uygulaması olarak,

$$a \equiv 2 \pmod{5},$$

$$a \equiv 3 \pmod{13},$$

denklemlerinin verildiğini varsayalım. Bu durumda, $a_1 = 2$, $n_1 = m_2 = 5$, $a_2 = 3$ ve $n_2 = m_1 = 13$ olur ve bizim hesaplamak istediğimiz $n = n_1 m_2 = 65$ olduğundan $a \bmod 65$ i bulmaktır. Çünkü $13^{-1} \equiv 2 \pmod{5}$ ve $5^{-1} \equiv 8 \pmod{13}$ tür. Böylece

$$c_1 = 13(2 \bmod 5) = 26,$$

$$c_2 = 5(8 \bmod 13) = 40,$$

ve

$$a \equiv 2 \cdot 26 + 3 \cdot 40 \pmod{65}$$

$$\equiv 52 + 120 \pmod{65}$$

$$\equiv 42 \pmod{65}$$

dir. modulo 65 için Şekil 31.3 Çin kalan teoreminin tablosunu verir.

Böylelikle, modulo n üzerinde direkt modulo n üzerinde hesaplamalarla veya ayrı modulo n_i hesaplamaları üzerinden temsiliyle çalışabiliriz. Tüm bu hesaplamalar denktir.

Alıştırırmalar

31.5-1

$x \equiv 4 \pmod{5}$ $x \equiv 5 \pmod{11}$ ve denklemlerine yönelik tüm çözümleri bulunuz.

31.5-2

Sırasıyla 9, 8, 7 ile bölündüğünde 1, 2, 3 kalanını veren tüm x tam sayılarını bulun.

31.5-3

$\gcd(a, n) = 1$ Teorem 31.27 altında, durumunda

$$(a^{-1} \bmod n) \leftrightarrow ((a_1^{-1} \bmod n_1), (a_2^{-1} \bmod n_2), \dots, (a_k^{-1} \bmod n_k))$$

olmasını tartışınız.

31.5-4

$f(x) \equiv 0 \pmod{n}$ Teorem 31.27 altında, f polinomu için denkleminin kök sayısının $f(x) \equiv 0 \pmod{n_1}$, $f(x) \equiv 0 \pmod{n_2}$, $\dots$, $f(x) \equiv 0 \pmod{n_k}$ denklemlerinin her birinin kök sayısının çarpımına eşit olduğunu kanıtlayın.

31.6 Bir elemanın kuvvetleri

Verilen a elemanın katlarını n modülünde incelediğimiz gibi, $a \in \mathbb{Z}_n^*$ için n modülünde a nın kuvvetlerinin dizisini de inceleyeceğiz.

$$a^0, a^1, a^2, a^3, \dots, \text{ modulo } n. \quad (31.33)$$

0 dan indislendiğinde, bu dizideki 0. değer $a^0 \bmod n = 1$ olur ve i . değer $a^i \bmod n$ olur. Örneğin modulo 7 de 3 ün kuvvetleri şöyle olur:

i	0	1	2	3	4	5	6	7	8	9	10	11	...
$3^i \bmod 7$	1	3	2	6	4	5	1	3	2	6	4	5	...

iken, modulo 7 de 2 nin kuvvetleri şöyle olur:

i	0	1	2	3	4	5	6	7	8	9	10	11	...
$2^i \bmod 7$	1	2	4	1	2	4	1	2	4	1	2	4	...

Bu kısımda $\langle a \rangle$ yinelenen çarpım ile a tarafından oluşturulan $\mathbb{Z}_n^*$ nin altgrubunu belirtsin ve $\text{ord}_n(a)$ nın (" a nın mertebesi, modulo n) $\mathbb{Z}_n^*$ de a nın mertebesini belirtsin. Örneğin $\mathbb{Z}_7^*$ de $\langle 2 \rangle = \{1, 2, 4\}$ ve $\text{ord}_7(2) = 3$ olur. $\mathbb{Z}_n^*$ in boyutu olarak Euler Phi fonksiyonu $\phi(n)$ nin tanımından (bkz. Kısım 31.3), Sonuç 31.19 u $\mathbb{Z}_n^*$ notasyonuna dönüştürüyoruz ki Euler Teoremi sağlansın ve p nin asal sayı olduğu durumda Fermat'ın teoremini elde etmek için onu $\mathbb{Z}_p^*$ ye özelleştiriyoruz.

Teorem 31.30 (Euler teoremi)

Her $n > 1$ tam sayısı,

ve tüm $a \in \mathbb{Z}_n^*$ için $a^{\phi(n)} \equiv 1 \pmod{n}$ dir. ■

Teorem 31.31 (Fermat teoremi)

p nin asal sayı olması durumunda,

$a \in \mathbb{Z}_p^*$ için $a^{p-1} \equiv 1 \pmod{p}$ olur.

Kanıt Denklem (31.21) e göre, $\mathbb{Z}_p$ nin asal sayı olması durumunda $\phi(p) = p - 1$ olur. ■

$0 \notin \mathbb{Z}_p^*$ nedeniyle; Fermat'ın teoremi, 0 haricinde her birleşen için geçerlidir. Bununla birlikte $a \in \mathbb{Z}_p$ için p nin asal sayı olması durumunda $a^p \equiv a \pmod{p}$ ye sahibiz.

$\text{ord}_n(g) = |\mathbb{Z}_n^*|$ durumunda, deki her birleşen g nin bir kuvveti olur (modulo n) ve g ye **ilkel kökü** ya da $\mathbb{Z}_n^*$ in **üretici** denir. Örneğin 3, modulo 7 de ilkel köktür, fakat 2 modulo 7 de ilkel kök değildir. $\mathbb{Z}_n^*$ nin ilkel bir köke sahip olması durumunda, $\mathbb{Z}_n^*$ grubu **devirlidir**. Niven ve Zuckerman [265] tarafından kanıtlanan aşağıdaki teoremin kanıtını yok sayıyoruz.

Teorem 31.32

$\mathbb{Z}_n^*$ nin devirli olduğu durumdaki $n > 1$ değerleri, tüm $p > 2$ asal sayıları ve tüm pozitif e tam sayıları için 2, 4, p^e ve $2p^e$ dir. ■

g nin $\mathbb{Z}_n^*$ nin bir ilkel kökü ve a nin $\mathbb{Z}_n^*$ kümesinin herhangi bir elemanı olması durumunda, $g^z \equiv a \pmod{n}$ şeklinde bir z mevcut olur. Bu z , g tabanına göre a nın **ayrık logaritması** ya da **indisi** olur (modulo n); bu değeri $\text{ind}_{n,g}(a)$ olarak belirteceğiz.

Teorem 31.33 (Ayrık logaritma teoremi)

g nin $\mathbb{Z}_n^*$ nin bir ilkel kökü olması durumunda, $g^x \equiv g^y \pmod{n}$ dir. Ancak ve ancak $x = y + k\phi(n)$ denklemi sağlanır.

Kanıt Öncelikle, $x \equiv y \pmod{\phi(n)}$ olduğu varsayımında bulunalım. Sonrasında k tam sayısı için $x = y + k\phi(n)$ olur. Böylece;

$$\begin{aligned} g^x &\equiv g^{y+k\phi(n)} \pmod{n} \\ &\equiv g^y \cdot (g^{\phi(n)})^k \pmod{n} \\ &\equiv g^y \cdot 1^k \pmod{n} \quad ((\text{Euler teoremine göre}) \\ &\equiv g^y \pmod{n}. \end{aligned}$$

Bunun aksine, $g^x \equiv g^y \pmod{n}$ varsayımında bulunalım. g kuvveti dizisinin, $\langle g \rangle$ ve $|\langle g \rangle| = \phi(n)$ denkleminde ait her birleşeni üretmesi nedeniyle, Sonuç 31.18, g kuvvetleri dizisinin, $\phi(n)$ periyodu ile periyodik olduğunu belirtir. Bu yüzden $g^x \equiv g^y \pmod{n}$ durumunda, $x \equiv y \pmod{\phi(n)}$ denklemini elde etmek durumundayız. ■

Şuan itibariyle, dikkatimizi 1 in modülo asal kuvvete göre kareköklerine çeviriyoruz. Aşağıdaki teorem, Kısım 31.8 deki asallık testini geliştirmemiz açısından faydalı olacaktır.

Teorem 31.34

p nin tek asal sayı olması ve $e \geq 1$ halinde,

$$x^2 \equiv 1 \pmod{p^e} \quad (31.34)$$

denklemini $x = 1$ ve $x = -1$ olmak üzere iki çözüme sahip olur.

Kanıt (31.34) denklemini aşağıdaki denkleme tekabül etmektedir:

$$p^e \mid (x - 1)(x + 1).$$

$p > 2$ olduğu için $p \mid (x - 1)$ ya da $p \mid (x + 1)$ elde ederiz, lakin ikisini birden değil (Aksi halde özellik (31.3) e göre, p bunların farkını da $(x + 1) - (x - 1) = 2$ bölecektir). $p \nmid (x - 1)$ durumunda ise $\text{ebob}(p^e, x - 1) = 1$ olur ve Sonuç 31.5'e göre, $p \nmid (x + 1)$ elde etmiş oluruz. Bu, $x \equiv -1 \pmod{p^e}$ demektir. Simetrik olarak, $p^e \mid (x + 1)$ durumunda, $\text{ebob}(p^e, x + 1) = 1$ olur ve Sonuç 31.5 $p^e \mid (x - 1)$ durumunu belirtir, böylelikle olur. Bu yüzden ya $x \equiv -1 \pmod{p^e}$ ya da $x \equiv 1 \pmod{p^e}$ olur. ■

$x^2 \equiv 1 \pmod{n}$ denklemini sağlaması durumunda x sayısı, **modülo n ye göre 1 in aşikar olmayan karekökü** olur fakat x , 1 ya da -1 olmak üzere iki “aşikar” kare köküne denk gelmeyecektir. Örneğin, modulo 35 de 6, 1 in aşikar olmayan kareköküdür. Miller-Rabin asallık testi prosedürü için Kısım 31.8 dahilinde kanıtın doğruluğu konusunda Teorem 31.34 e yönelik aşağıdaki sonuçtan faydalanacağız.

Sonuç 31.35

Modulo n ye göre 1 in aşikar olmayan karekökünün mevcut olması durumunda; n , bileşik olur.

Kanıt Teorem 31.34 ün devriğine göre, modülo n ye göre 1 in aşikar olmayan karekökünün mevcut olması durumunda n tek asal sayı ya da bir asal sayının kuvveti olamaz. Eğer $x^2 \equiv 1 \pmod{2}$ ise o halde $x \equiv 1 \pmod{2}$ ve böylece 1 in mod 2 ye göre tüm karekökleri trivialdir. Bu yüzden n asal sayı olamaz. Son olarak, mevcut olacak olan 1 in aşikar olmayan kare kökü için $n > 1$ elde etmek durumunda kalırız. Bu yüzden n bileşik olmak durumundadır. ■

Tekrarlı kareleme ile kuvvetlere yönelik artış

Bir sayının kuvvet modulünde başka bir sayıya yükseltilmesi sayı teorik hesaplamalarda sıkça vuku bulan bir işlemdir ve **modüler üs alma** olarak da bilinmektedir. Daha özel bir şekilde, a ve b nin negatif olmayan tam sayılar olması ve n nin pozitif tam sayı olması halinde $a^b \pmod{n}$ in hesaplanması için etkin bir yöntem arzu etmekteyiz. Modüler üs alma, bir çok asallık testi rutinleri ve RSA ortak anahtarlı şifreleme sistemi dahilinde gerekli bir işlem olmaktadır. **Tekrarlı kareleme** yöntemi, b nin ikili temsiline faydalanarak bu problemi verimli şekilde çözmektedir.

$\langle b_k, b_{k-1}, \dots, b_1, b_0 \rangle$ ifadesi b nin ikili temsili olsun (Bu, ikili temsilin $k + 1$ bit uzunlukta olduğu, b_k nin en soldaki bit olduğu ve b_0 in en sağdaki bit olduğu

i	9	8	7	6	5	4	3	2	1	0
b_i	1	0	0	0	1	1	0	0	0	0
c	1	2	4	8	17	35	70	140	280	560
d	7	49	157	526	160	241	298	166	67	1

Şekil 31.4 $a = 7$, $b = 56 = \langle 1000110000 \rangle$, ve $n = 561$ iken $a^b \pmod{n}$ hesaplaması sırasında MODULAR-EXPONENTIATION un sonucu. Değerler **for** döngüsünün her uygulamasından sonra gösterilir. Son sonuç 1 dir.

anlamına gelir). Sıradaki prosedür, c değerini 0 dan b ye ikiye katlamalar ve artışlarla $a^c \pmod{n}$ yi hesaplamamıza yarar.

MODULAR-EXPONENTIATION (a, b, n)

```

1   $c = 0$ 
2   $d = 1$ 
3   $\langle b_k, b_{k-1}, \dots, b_0 \rangle$ ,  $b$  nin ikili temsili olsun.
4  for  $i = k$  dan 0 kadar.
5       $c = 2c$ 
6       $d = (d \cdot d) \pmod{n}$ 
7      if  $b_i = 1$ 
8           $c = c + 1$ 
9       $d = (d \cdot a) \pmod{n}$ 
10 return  $d$ 
```

Her tekrarda 6. satırda karesinin alınması “tekrarlı kareleme” ismini açıklamaktadır. Örnek olarak $a = 7$, $b = 560$ ve $n = 561$ için algoritma Şekil 31.4 de gösterilen modulo 561 e göre değerler dizisini hesaplar; kullanılan üsler dizisi, c ile etiketlenen tablo satırında yer alır.

c değişkeni, gerçekten algoritma için gerekli değildir, fakat aşağıdaki iki parçalı döngü değişmezi için dahil edilmiştir:

4 ile 9. satırlara ait **for** döngüsünün her bir iterasyonu öncesinde,

1. c değeri, b nin ikili temsilinin öneki $\langle b_k, b_{k-1}, \dots, b_{i+1} \rangle$ ile aynı olur

2. $d = a^c \pmod{n}$

Bu döngü değişmezini aşağıdaki şekilde kullanmaktayız:

Başlangıç: Başlangıç olarak, $i = k$ dır, bu yüzden $c = 0$ a tekabül eden $\langle b_k, b_{k-1}, \dots, b_{i+1} \rangle$ öneki boştur. Bununla birlikte $d = 1 = a^0 \pmod{n}$ dur

Yineleme: c' ve d' nin **for** döngüsünün iterasyonunun sonunda c ve d değerlerini ve dolayısıyla sonraki iterasyon öncesindeki değerleri belirtsin. Her döngü $c' = 2c'$ (eğer $b_i = 0$ ise) ya da $c' = 2c + 1$ (eğer $b_i = 1$ ise) güncellemesi yapmaktadır, böylelikle sonraki döngü öncesinde c doğru olacaktır. $b_i = 0$ ise, $d' = d^2 \bmod n = (a^c)^2 \bmod n = a^{2c} \bmod n = \bmod n$ olur. $b_i = 1$ durumunda, $d' = d^2 a \bmod n = (a^c)^2 a \bmod n = a^{2c+1} \bmod n = \bmod n$ olur. Her durumda, döngü öncesinde $d = a^c \bmod n$ olur.

Sonlandırma: Sonlandırmada $i = -1$ olur. b nin ikili temsili $\langle b_k, b_{k-1}, \dots, b_0 \rangle$ örneğinin değerinin c olmasından dolayı $c = b$ olur. Bu yüzden $d = a^c \bmod n = a^b \bmod n$.

a , b ve n girdilerinin β -bit lik sayılar olması durumunda, gerekli aritmetik işlemlerin toplam sayısı $O(\beta)$ olur ve gerekli bit işlemlerinin toplam sayısı ise $O(\beta^3)$ olur.

Alıştırmalar

31.6-1

$\mathbb{Z}_{11}^*$ deki her bir bileşenin derecesini gösteren bir tablo çizin. En küçük ilkel kök g yi seçin ve tüm $x \in \mathbb{Z}_{11}^*$ için $\text{ind}_{11,g}(x)$ i belirten bir tablo hesaplaması yapın.

31.6-2

Soldan sağa yerine, sağdan sola b bitlerini inceleyen modüler bir üs alma algoritması sunun.

31.6-3

$\phi(n)$ i bildiğiniz varsayımıyla, MODULAR-EXPONENTIATION prosedüründen faydalanarak $a \in \mathbb{Z}_n^*$ için $a^{-1} \bmod n$ in nasıl hesaplandığını açıklayın.

31.7 RSA ortak-anahtarlı şifreleme sistemi

Ortak anahtarlı şifreleme sistemi ile haberleşen iki şahsın mesajlarını şifreleyebiliriz, böylece gizlice dinleyen kimse şifrelenmiş mesajları çözemez. Açık anahtarlı şifreleme sistemi, kişiye, elektronik bir mesajın sonuna sahtesi yapılamayan “sayısal imza” eklemesine olanak sağlar. Böyle bir imza kağıt üzerine atılan imzanın elektronik halidir. Bu başkası tarafından kolaylıkla kontrol edilebilir, ancak sahtesi yapılamaz; eğer mesajın herhangi bir biti değiştirilirse, geçerliliğini yitirir. Bundan dolayı, imzalanan mesajın içeriği ve imzalayanın kimliği doğrulamamıza

olanak sağlar. Elektronik olarak imzalanmış iş sözleşmeleri, elektronik çekler, elektronik cüzdan siparişleri ve diğer kişinin kimliğini doğrulamanın istendiği elektronik haberleşme uygulamaları için çok iyi bir araçtır.

RSA ortak anahtar şifreleme sistemi iki sayının çarpımını çarpanlara ayırma ve büyük asal sayıları bulmanın arasındaki etkileyici farka dayanmaktadır. Bölüm 31.8 büyük asal sayıları verimli bir şekilde bulma işlemini tanımlamaktadır ve Bölüm 31.9 büyük sayıları çarpanlara ayırma problemini tartışmaktadır.

Ortak anahtarlı şifreleme sistemleri

Ortak anahtarlı şifreleme sisteminde, her taraf *ortak anahtar* ve *gizli anahtar* ikilisine sahiptir. Her anahtar bilginin bir parçasıdır. Örneğin, RSA şifreleme sisteminde, her anahtar tam sayıların bir çiftini içerir. Taraf, “Alice” ve “Bob”, kriptografideki örneklerde geleneksel olarak kullanılır; Alice’in ortak ve gizli anahtarlarını P_A , S_A ve Bob’un ortak ve gizli anahtarlarını P_B , S_B ile gösteririz.

Her katılımcı kendi ortak ve gizli anahtarlarını oluşturur. Gizli anahtarlar gizli olarak saklanır, ancak ortak anahtarlar herkes tarafından bilinebilir veya yayımlanabilir. Aslında, herkesin ortak anahtarının herkese açık bir dizin içerisinde olması daha uygun bir yaklaşımdır, böylece herhangi bir katılımcı başka bir katılımcının ortak anahtarını kolaylıkla elde edebilir.

Ortak ve gizli anahtarlar herhangi bir mesaja uygulanabilen fonksiyonları belirtir. $\mathcal{D}$ izin verilen mesajların kümesi olsun. Örneğin, $\mathcal{D}$, sonlu uzunluklu bit dizilerinin kümesi olabilir. Ortak anahtarlı kriptografinin en basit ve özgün tanımında, ortak ve gizli anahtarlar $\mathcal{D}$ den kendisine bire-bir fonksiyonları belirtir. Alice’in ortak anahtarını P_A , bunun fonksiyonunu $P_A()$ ile gizli anahtarını S_A , buna karşılık gelen fonksiyonu $S_A()$ ile gösteririz. $P_A()$ ve $S_A()$ fonksiyonları $\mathcal{D}$ nin permütasyonlarıdır. $P_A()$ ve $S_A()$ fonksiyonlarının verilen P_A ve S_A anahtarları için verimli bir şekilde hesaplandığını kabul edelim.

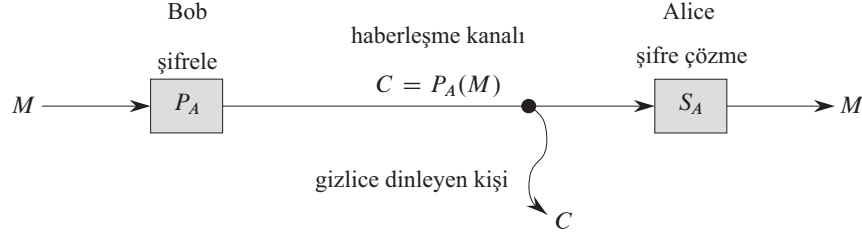
Herhangi bir katılımcının ortak ve gizli anahtarları “eşlenmiş çift” tir, öyle ki; bunlar birbirinin tersi olan fonksiyonları belirtir. Şöyle ki; herhangi bir $M \in \mathcal{D}$ mesajı için

$$M = S_A(P_A(M)) , \quad (31.35)$$

$$M = P_A(S_A(M)) \quad (31.36)$$

M mesajını P_A ve S_A anahtarları ile herhangi bir sırada dönüştürsek, başarılı bir şekilde M mesajını geri elde ederiz.

Ortak anahtarlı şifreleme sisteminde, Alice haricinde hiç kimsenin $S_A()$ fonksiyonunu kısa bir sürede çözmemesine gerek duymaktayız. Bu varsayım Alice’e gönderilen şifreli iletinin Alice’e özgü kalması açısından önemlidir ve bunu bilmek Alice’in sayısal imzasını doğrular. Alice, S_A yı gizli tutmak zorundadır; eğer tutmazsa, tekliğini kaybeder ve şifreleme sistemi ona özgü yetenekler sağlaya-



Şekil 31.5 Ortak anahtarlı sistemde şifreleme. Bob, M mesajını Alice'in ortak anahtarı P_A yı kullanarak şifreler ve karşılık gelen şifre metin $C = P_A(M)$ yi haberleşme kanalı üzerinden Alice'e gönderir. İletilen şifre metni yakalayan hattı gizlice dinleyen kişi M mesajı hakkında hiçbir bilgi elde edemez. Alice C yi kabul eder ve orijinal mesajı elde etmek için $M = S_A(C)$ ile kendi gizli anahtarını kullanarak şifreyi çözer.

maz. Alice sadece $S_A()$ yı hesaplar varsayımı P_A yı bilen herkes için geçerli olmalı ve $S_A()$ nın ters fonksiyonu ile $P_A()$ yı verimli bir hesaplayabilmelidir. Çalışabilen bir ortak anahtarlı şifreleme sistemi tasarlayabilmek için, şuna bir çözüm bulmamız gerekir: Nasıl bir P_A dönüşümünü ortaya çıkarmalıyız ki, buna karşılık gelen S_A ters dönüşümünün nasıl hesaplanabildiği elde edilmemelidir. Bu madde zorlu gözükmemektedir, ancak bunun nasıl üstesinden geleceğimizi göreceğiz.

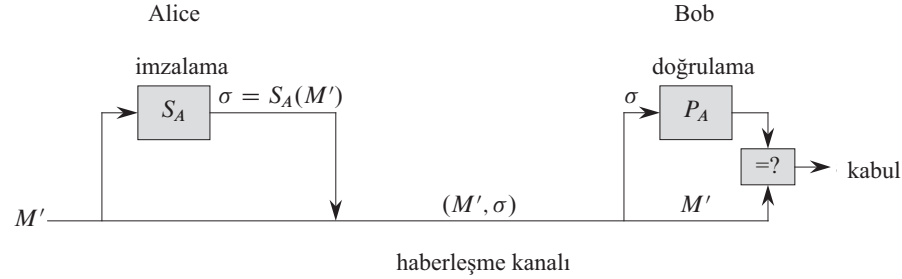
Ortak anahtarlı şifreleme sisteminde şifreleme Şekil 31.5 te gösterildiği gibi çalışır. Bob'un Alice'e şifreli M mesajını göndermek istediğini varsayalım, böylece hattı gizlice dinleyen kişi bunu anlamsız bir mesaj gibi görür. Mesaj gönderme planı şu şekilde işler:

- Bob, Alice'in ortak anahtarı P_A yı alır (doğrudan Alice'den veya herkese açık bir dizinden)
- Bob, M mesajı için gelen **şifreli metin** $C = P_A(M)$ yi hesaplar ve Alice'e gönderir.
- Alice C şifreli metnini aldığı zaman, kendi gizli anahtarı S_A ile orijinal mesajı elde eder: $S_A(C) = S_A(P_A(M)) = M$.

$S_A()$ ve $P_A()$ birbirinin tersi fonksiyonlar olduklarından, Alice, C den M yi hesaplayabilir. Sadece Alice $S_A()$ yı hesaplayabildiğinden, C den M yi hesaplayabilen tek kişidir. Bob, M yi $P_A()$ ile şifrelediği için, Alice iletilen mesajı anlayabilir.

Sayısal imzaları ortak anahtarlı şifreleme sisteminin formülasyonu ile kolayca uygulayabiliriz (sayısal imzaları inşası problemi için başka yaklaşımlar da bulunmaktadır, ancak burada onlara değinmeyeceğiz). Alice'in Bob'a sayısal olarak imzalanmış M' nü göndermek istediğini varsayalım. Şekil 31.6 sayısal imzalama senaryosunun nasıl işlediğini göstermektedir.

- Alice, M' mesajı için kendi gizli anahtarı S_A yı kullanarak $\sigma = S_A(M')$ denklemi ile kendi **sayısal imzasını** hesaplar.



Şekil 31.6 Ortak anahtarlı sistemde sayısal imzalar. Alice, M' mesajını $\sigma = S_A(M')$ ile kendi *sayısal imzasını* hesaplar, mesajın sonuna ekler. Bob'a (M', σ) mesaj/imza çiftini iletir, Bob $\sigma = S_A(M')$ denklemini kontrol ederek imzayı doğrular. Eğer bu denklem doğrulanırsa, (M', σ) ü ve bu mesajın Alice tarafından imzalandığını kabul eder.

- Alice (M', σ) mesaj/imza çiftini Bob'a gönderir.
- Bob (M', σ) ü aldığı anda, Alice'in ortak anahtarını $M' = P_A(\sigma)$ denkleminde kullanarak mesajın kaynağının Alice olduğunu doğrulayabilir. (M' nün Alice'in ismini içerdiği düşünülür, böylece Bob kimin ortak anahtarını kullanacağını bilir). Bu denklem sağlanırsa, M' nün gerçekten Alice tarafından imzalandığı sonucuna varır. Bu denklem sağlanmaz ise, M' mesajının veya σ sayısal imzasının iletim esnasında değiştirildiği veya (M', σ) çiftinin taklidinin yapılmasına çalışıldığı sonucuna varır.

Sayısal imza, imzalayanın kimliğini ve imzalanan mesajın içeriğinin doğruluğunu sağladığından, kâğıt kalem ortamında yazılan bir belgenin sonuna el ile atılan bir imzanın benzeridir.

Sayısal imza, imzayı atanın açık anahtarına erişebilen herhangi birisi tarafından doğrulanabilmelidir. İmzalanan mesaj bir kişi tarafından doğrulanabilir ve sonra bunu doğrulayabilen diğer kişilere iletilir. Örneğin, mesaj, Alice'den Bob'a bir elektronik çek olabilir. Bob Alice'in çek üzerindeki imzasını doğruladıktan sonra, bu çeki bankaya verebilir, burada banka da imzayı doğrular ve uygun miktarda transfer yetkisini alır.

İmzalanan mesajın şifrelenmesine gerek yoktur; mesaj "açık bir formda" ve değiştirilmeye karşı korunma olmaksızın gönderilebilir. Yukarıda şifreleme ve imzalama için verilen protokolleri birleştirerek, hem imzalı hem de şifrelenmiş mesajlar oluşturabiliriz. İmzalayan kişi öncelikle kendi sayısal imzasını mesaja ekler ve sonra mesaj/imza çiftini alıcının ortak anahtarı ile şifreler. Alıcı, orijinal mesaj ve buna bağlı imzayı elde edebilmek için kendi gizli anahtarı ile şifreyi çözer. Alıcı daha sonra imzayı göndericinin ortak anahtarı ile doğrular. Bu işlemin

kağıt ortamındaki karşılığı imzalanan bir belgenin sadece alıcının açabildiği bir zarf içine koyup postalama işlemidir.

RSA şifreleme sistemi

RSA ortak anahtarlı şifreleme sisteminde bir taraf kendi gizli ve ortak anahtarını aşağıda verilen işlemlere göre oluşturur:

1. $p \neq q$ olacak şekilde rassal p ve q büyük asal sayılarını seç. Bu p ve q asallarının her biri 1024 bit olabilir.
2. $n = pq$ yu hesapla.
3. (31.20) denkleminde $(p-1)(q-1)$ e eşit olan $\phi(n)$ ile aralarında asal olan küçük bir e sayısını seç.
4. $\phi(n)$ modülüne göre e nin çarpmaya göre tersi d yi hesapla. (Sonuç 31.26 d nin varlığını ve tek bir şekilde tanımladığını göstermektedir. Kısım 31.4 deki tekniği e ve $\phi(n)$ verildiğinde d yi hesaplamak için kullanabiliriz.)
5. Tarafın **RSA ortak anahtarı** olarak $P = (e, n)$ çiftini yayımla.
6. Tarafın **RSA gizli anahtarı** olarak $S = (d, n)$ çiftini sakla.

Bu tasarımda $D, \mathbb{Z}_n$ kümesidir. M mesajını $P = (e, n)$ açık anahtarı ile dönüştürmek için , aşağıdakini hesapla:

$$P(M) = M^e \bmod n \quad (31.37)$$

C şifreli metnini $S = (d, n)$ gizli anahtarı ile dönüştürmek için aşağıdakini hesapla:

$$S(C) = C^d \bmod n \quad (31.38)$$

Bu denklemler hem şifreleme hem de imzalama işlemlerinde kullanılır. İmza oluşturmak amacıyla, imzalayacak taraf şifreli metin yerine imzalanacak mesaj için kendi gizli anahtarını kullanır. İmzayı doğrulamak için, mesajı şifrelemede kullanılanın yerine imzalayanın ortak anahtarı uygulanır.

Kısım 31.6 da tanımlanan MODULAR-EXPONENTIATION işlemi ile ortak ve gizli anahtarlı işlemleri uygulayabiliriz. Bu işlemlerin çalışma zamanlarını analiz edebilmemiz için şunları varsayalım: ortak anahtar (e, n) ve gizli anahtar (d, n) ; $\lg e = O(1)$, $\lg d = O(1)$, $\lg d \leq \beta$ ve $\lg n \leq \beta$ ü sağlasın. Sonra ortak anahtarı uygularken $O(1)$ tane modüler çarpmaya ihtiyaç duyulur ve $O(\beta^2)$ tane bit işlemi kullanılır. Gizli anahtarı uygularken $O(\beta)$ tane modüler çarpmaya ihtiyaç duyulur ve $O(\beta^3)$ tane bit işlemi kullanılır.

Teorem 31.36 (RSA'nın doğruluğu)

RSA denklemleri (31.37) ve (31.38) $\mathbb{Z}_n$ nin (31.35) ve (31.36) nolu denklemlerini sağlayan ters dönüşümlerini tanımlar.

Kanıt (31.37) ve (31.38) nolu denklemlerden, herhangi bir $M \in \mathbb{Z}_n$ için

$$P(S(M)) = S(P(M)) = M^{ed} \pmod{n}$$

e ve d $\phi(n) = (p-1)(q-1)$ modülünde çarpmaya göre ters elemanlar olduklarından bazı k tam sayıları için

$$ed = 1 + k(p-1)(q-1)$$

Ancak, eğer $M \not\equiv 0 \pmod{p}$ varsa,

$$\begin{aligned} M^{ed} &\equiv M(M^{p-1})^{k(q-1)} \pmod{p} \\ &\equiv M((M \bmod p)^{p-1})^{k(q-1)} \pmod{p} \\ &\equiv M(1)^{k(q-1)} \pmod{p} \quad (\text{Teorem 31.31 den}) \\ &\equiv M \pmod{p}. \end{aligned}$$

Ayrıca, $M \equiv 0 \pmod{p}$ ise $M^{ed} \equiv M \pmod{p}$. Böylece, her M için

$$M^{ed} \equiv M \pmod{p}$$

dir. Benzer olarak, her M için

$$M^{ed} \equiv M \pmod{q}$$

olur. Bu yüzden, Çin kalan teoremi ve Sonuç 31.29 dan, her M için

$$M^{ed} \equiv M \pmod{n} \quad \blacksquare$$

RSA şifreleme sisteminin güvenliği önemli ölçüde büyük sayıların çarpanlara ayrılmasının zorluğuna dayanmaktadır. Eğer istenmeyen kişi ortak anahtar n yi çarpanlara ayırabilir ise, bu kişi p ve q asal sayılarını kullanarak aynı üretim yöntemleri ile ihtiyaç duyulan gizli anahtarı türetebilir. Bundan dolayı, büyük sayıları çarpanlara ayırmak kolay ise, RSA şifreleme sistemini kırmak da kolaydır. Bunun tersi, eğer büyük sayıları çarpanlara ayırmak zor ise, RSA yı kırmak zordur, bu kanıtlanmamıştır. Bununla birlikte, 20 yıllık araştırmalardan sonra RSA şifreleme sistemini kırmak için n modülünü çarpanlara ayırmaktan daha kolay bir yöntem bulunamamıştır. Kısım 31.9 da göreceğimiz gibi büyük sayıları çarpanlara ayırmak şaşırtıcı bir şekilde zordur. Rassal seçilen ve birbiri ile çarpılan iki 1024 bitlik sayı ile varolan teknoloji ile mümkün bir zamanda “kırılmayan” ortak anahtar üretebiliriz. Sayılar teorisi tabanlı algoritmaların tasarımında temel bir yeniliğin eksikliğinden ve önerilen standartlardaki yönlendirmeler dikkatli bir şekilde uygulandığında, RSA sistemi uygulamalarda yüksek dereceli güvenlik sağlama özelliğine sahiptir.

Bununla birlikte, RSA şifreleme sisteminde güvenliği sağlayabilmek için çarpanlara ayırma konusundaki gelişmelere karşı dirençli olabilecek düzeyde olduk-

ça büyük tam sayılar, örneğin yüzlerce hatta bin bitten daha uzun, kullanmalıyız. Bunun yazıldığı tarihte (2009), RSA, yaygın olarak modülü 768 ile 2048 bit arasında değişen uzunluklarda kullanılmaktaydı. Bu boyutlardaki modülü üretebilmek için, büyük asal sayıları verimli bir şekilde bulabilmeliyiz. Kısım 31.8 bu problemi temel almaktadır.

Ortak anahtarlı olmayan hızlı çalışan şifreleme sistemleri ile birlikte “hibrid” veya “anahtar yönetimi” modunda verimlilik sağlaması açısından RSA sıklıkla kullanılır. Böyle bir sistemde, şifreleme ve şifre çözme anahtarı aynıdır. Alice büyük bir M mesajını Bob’a gizli bir şekilde göndermek istiyorsa, Alice hızlı çalışan ortak anahtarlı olmayan şifreleme sistemleri için rassal bir anahtar seçer ve M mesajını K yi kullanarak şifreler, C şifreletnini elde eder. Burada, C nin uzunluğu M ile aynıdır, ancak K oldukça küçük boyutludur. Sonra Alice, K yi Bob’un RSA anahtarı ile şifreler. K oldukça küçük olduğundan, $P_B(K)$ yi hesaplamak hızlıdır ($P_B(M)$ yi hesaplamaktan daha hızlıdır). Alice ($C, P_B(K)$) yi Bob’a iletir, Bob $P_B(K)$ yi K yi elde etmek için çözer ve sonra K yi kullanarak M ye ulaşmak amacıyla C şifreletnini çözer.

Verimli sayısal imzalar yapabilmek için benzer hibrid yaklaşımı kullanabiliriz. Bu yaklaşımda RSA ile *çarpışma-dayanıklı çırpı fonksiyonu* h birleştirilir – hesaplama anlamında $h(M) = h(M')$ olacak şekilde farklı iki M ve M' mesajlarını bulmanın mümkün olmadığı durumda, bunların h fonksiyonunda hesabının kolay olduğu fonksiyondur. $h(M)$ değeri oldukça kısadır (örneğin 256 bit), bu M mesajının parmak izi olarak görülür. Alice, M mesajını imzalamak istiyorsa, öncelikle M yi h fonksiyonuna $h(M)$ parmak izini elde etmek için sokar, sonra kendi gizli anahtarı ile şifreler. Alice, M nin kendi imzasını bir versiyonu gibi ($M, S_A(h(M))$) yi Bob’a gönderir. Bob, $h(M)$ yi hesaplayarak imzayı doğrulayabilir, bunu, P_A yı $S_A(h(M))$ ye uyguladığında aynı $h(M)$ elde ediliyorsa doğrulama gerçekleşir. Aynı parmakizine sahip iki farklı mesaj oluşturulmadığından, imzalanan bir mesajı değiştirmek ve imzanın geçerliliğini sürdürmek hesaplama açısından mümkün değildir.

Son olarak, ortak anahtarların dağıtımını daha kolay yapabilmek için *sertifika* kullanımını göz önünde bulundurmalıyız. Örneğin, ortak anahtarı herkes tarafından bilinen “güvenilir bir makam” T olduğunu varsayalım. Alice, “Alice’in ortak anahtarı P_A dır” şeklinde T den imzalı mesaj (kendi sertifikasını) alabilir. Herkes P_T yi bildiğinden, bu sertifika “kendini doğrular”. Alice, kendi imzalı mesajları ile birlikte sertifikasını barındırabilir, böylece alıcı, Alice’in ortak anahtarına imzasını doğrulayabilmek için hemen ulaşır. Anahtarı T ile imzalandığından, alıcı Alice’in gerçekten Alice olduğunu bilmektedir.

Alıştırmalar

31.7-1

$p = 11$, $q = 29$, $n = 319$ ve $e = 3$ kümesi ile RSA yı gözönüne alalım. Gizli anahtar d nin değeri ne olmalıdır? $M = 100$ mesajının şifrelenmiş hali nedir?

31.7-2

Alice ortak üs alma elemanı $e=3$ ve gizlice dinleyen kişi $0 < d < \phi(n)$ durumunda gizli anahtarı d yi elde ediyorsa, bu durumda Alice'in n modülünü, bit uzunluğuna bağlı olarak polinom zamanda çarpanlara ayırabileceğini kanıtlayınız. ($e=3$ durumu kaldırıldığında bile bunun sağlandığını size sorulmamış olmasına rağmen gösterebilirsiniz. Miller [255] e bakınız.)

31.7-3 *

RSA'nın aşağıdakiler ışığında çarpımsal olduğunu kanıtlayınız.

$$P_A(M_1)P_A(M_2) \equiv P_A(M_1M_2) \pmod{n}$$

Bunu kullanarak, gizlice dinleyen kişi $\mathbb{Z}_n$ de P_A ile şifrelenmiş mesajların yüzde 1 ini verimli bir şekilde çözebiliyorsa, olasılıksal bir algoritma ile P_A ile şifrelenmiş her mesajı büyük bir olasılıkla çözebilir.

*** 31.8 Asallık testi**

Bu bölümde, büyük asal sayıları bulma problemini tartışacağız. Asal sayıların sıklığı ile başlayıp, bunları makul sürede ancak tam olmayan bir şekilde test etme yaklaşımlarına geçtikten sonra Miller ve Rabin'e dayanan verimli rastgele asallık testini sunacağız.

Asal sayıların yoğunluğu

Kriptografi gibi bir çok uygulamada, “rastgele” üretilen büyük asal sayılara ihtiyaç duyarız. Büyük asal sayılar şansımıza nadir değildir, böylece bir asal sayı bulana kadar, uygun olan rastgele tam sayıları test etmek mümkündür. **Asal dağılım fonksiyonu** $\pi(n)$ n den küçük eşit olan asal sayıların sayısını belirtir. Örneğin, $\pi(10) = 4$, burada 10 dan küçük 4 adet asal sayı vardır; 2,3,5 ve 7. Asal sayı teoremi $\pi(n)$ e faydalı bir benzerlik verir.

Teorem 31.37 (Asal sayı teoremi)

$$\lim_{n \rightarrow \infty} \frac{\pi(n)}{n / \ln n} = 1$$

■

Küçük çift sayı n için bile $n / \ln n$ $\pi(n)$ de makul olarak doğru tahminler verir. Örneğin, $n = 10^9$ da %6 dan daha azı kapalıdır, burada $\pi(n) = 50,847,534$

ve $n/\ln n \approx 48,254,942$ (sayılar teorisi çalışan birisi için 10^9 küçük bir sayıdır).

Rastgele bir n tam sayısının seçilmesi ve bunun Bernoulli denemesine göre asal olup olmadığını belirleme işlemini izleyebiliriz (Kısım C.4 e bakınız). Asal sayı teoreminden n nin asal sayı olma başka bir deyişle başarılı olma olasılığı yaklaşık $1/\ln n$ dir. Geometrik dağılım başarı elde edebilmek için kaç tane deneme yapmamız gerektiğini söyler ve denklem (C.32) ile beklenen deneme sayısı yaklaşık olarak $\ln n$ dir. Böylece, n ile aynı uzunlukta bir asal sayıyı bulmak için n civarında rastgele seçilen yaklaşık $\ln n$ tane tam sayı test etmeyi bekleriz. Örneğin, 1024 bitlik bir asal sayı bulmak için yaklaşık $\ln 2^{1024} \approx 710$ kadar rastgele seçilen 1024 bitlik sayıyı test etmeyi ummaktayız (tabii kide sadece tek tam sayıları seçerek bu durumu yarıya indirebiliriz).

Bu bölümün geri kalan yerinde, büyük tek sayının asal sayı olup olmadığına karar verme problemini tartışacağız. Gösterim uygunluğu açısından, burada $r \geq 1$ için $p_1, p_2, \dots, p_r$, n nin asal çarpanları ve $e_1, e_2, \dots, e_r$ pozitif tam sayılar olmak üzere n nin çarpanlara ayrılışı

$$n = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r}, \quad (31.39)$$

n nin asal sayı olabilmesi ancak ve ancak $r = 1$ ve $e_1 = 1$ olabilmesi ile mümkündür.

Asallık testi problemine basit bir yaklaşım **deneyerek bölme**dir. n sayısını her bir tam sayıya $2, 3, \dots, \lfloor \sqrt{n} \rfloor$ bölmeye çalışırız (Burada yine, 2 den büyük çift asal sayıları atlarız). n sayısının asal sayı olabilmesinin ancak ve ancak hiçbir sayının n yi bölmemesi olduğunu kolayca anlarız. Her bir deneyerek bölmenin sabit bir zamanda yapıldığı varsayılırsa, en kötü durum çalışma zamanı n 'nin uzunluğunun üssel hali olan $\Theta(\sqrt{n})$ 'dür. (Hatırlamak gerekir ki, eğer n β bit kullanılarak ikilik tabanda kodlanmış ise, $\beta = \lceil \lg(n+1) \rceil$ ve bu yüzden $\sqrt{n} = \Theta(2^{\beta/2})$). Böylece, deneyerek bölme sadece n çok küçük bir sayı ise veya küçük asal çarpanı varsa iyi çalışmaktadır. Bu yöntem çalıştığında, n nin asal sayı olup olmadığına karar vermedeki avatajının yanısıra, n bileşik bir sayı ise n nin asal çarpanlarını bulur.

Bu bölümde, verilen bir n sayısının asal sayı olup olmadığına karar vermek ile ilgileniyoruz; n bileşik bir sayı ise, onun asal çarpanları ile ilgilenmiyoruz. Kısım 31.9 da göreceğimiz gibi, bir sayının asal çarpanlarını bulmak hesaplama açısından pahalıdır. Verilen bir sayının asal sayı olup olmadığını söylemek, onun asal çarpanlarını bulmanın daha kolay olması muhtemelen şaşırtıcıdır.

Sözde asallık testi

Şimdi, “neredeyse çalışan” bir asallık test yöntemini gözönüne alacağız ve aslında bu bir çok kullanışlı uygulama için yeteri kadar iyidir. Daha sonra, bu

yöntemin küçük sorunlarını ortadan kaldıran bir iyileştirme sunacağız. $\mathbb{Z}_n^+$, $\mathbb{Z}_n$ ün sıfırdan farklı elemanlarını gösterebiliriz:

$$\mathbb{Z}_n^+ = \{1, 2, \dots, n-1\}$$

Eğer n asal ise, $\mathbb{Z}_n^+ = \mathbb{Z}_n^*$ dir.
 n bileşik bir sayı ve

$$a^{n-1} \equiv 1 \pmod{n} \quad (31.40)$$

ise n , *taban- a sözde asaldır*.

Fermat teoremi (Teorem 31.31), n asal ise, $\mathbb{Z}_n$ deki her a için n denklem (31.40) ı sağlar. Böylece, eğer n için denklem (31.40) ı sağlamayan bir $a \in \mathbb{Z}_n^+$ bulabilirsek, n kesinlikle bileşik bir sayıdır. Şaşırtıcı bir şekilde, bunun tersi hemen hemen tutmaktadır, bu yüzden, bu kriter asallık için neredeyse mükemmel bir test teşkil eder. $a = 2$ için denklem (31.40) ın sağlanıp sağlanmadığını görmek için test ederiz. Eğer sağlamazsa, n nin bir bileşik sayı olduğunu COMPOSITE şeklinde çıktı vererek açıklarız. Aksi halde, n nin asal olduğu tahminiyle, PRIME diye geri dönüş yapılır (aslında tüm bildiğimiz sayının asal veya taban-2 sözde asal olduğudur).

İzleyen işlemler, bu mantıkta n nin asallığını test etmek gibi görünmektedir. Kısım 31.6 dan MODÜLER-ÜS ALMA işlemini kullanmaktadır. n nin 2 den büyük tek bir sayı olduğunu varsayalım.

PSEUDOPRIME(n)

- 1 **if** MODULAR-EXPONENTIATION ($2, n-1, n$) $\neq 1 \pmod{n}$
- 2 **return** COMPOSITE // kesinlikle
- 3 **else return** PRIME // öyle ümit ediyoruz!

Bu işlem sadece tek tip hata vermektedir. Şöyle ki; eğer bu n nin bileşik sayı olduğunu söylerse, bu her zaman doğrudur. Eğer bu n nin asal olduğunu söylerse, sadece n nin taban-2 sözde asal olduğu durumda hata verir.

Bu hata işlemi hangi sıklıkla gerçekleşir? Şaşırtıcı olarak, nadiren. 10,000 den daha küçük n için hata veren sadece 22 değer vardır; ilk dört değer 341, 561, 645 ve 1105 tir. Bunu kanıtlamayacağız; ancak $\beta \rightarrow \infty$ iken olasılık rastgele seçilen β -bitlik sayı için sifıra gider. Pomerance'ı [279] kullanarak verilen bir uzunlukta-ki sayı için taban-2 sözde asallık testinde daha kesin karar verilir, rasgele seçilen 512 bitlik bir sayının yukarıda verilen işlem için taban-2 sözde asal olma olasılığı $1/10^{20}$ den daha azdır ve rastgele seçilen 1024 bitlik bir sayının taban-2 sözde asal olma olasılığı $1/10^{41}$ ten daha azdır. Bundan dolayı, eğer bazı uygulamalar için sadece büyük asal sayı bulmaya çalışıyorsanız, tüm pratik amaçlar için onlardan biri PSEUDOPRIME yi PRIME ye çevirene kadar, büyük sayıları rastgele seçerek hemen hemen hatalı davranmazsınız. Ancak asallık için test edilecek sayılar rast-

gele seçilmediği zaman, asallık testi için daha iyi bir yaklaşıma ihtiyaç duyarız. Göreceğimiz gibi, biraz akıllıca yaklaşım ve rastgelelik tüm girdiler üzerinde iyi çalışan sıradan asallık testi verecektir.

Ne yazık ki, **Carmicheal sayıları** olarak bilinen tüm $a \in \mathbb{Z}_n^*$ için denklem (31.40) ı sağlayan n bileşik sayıları olduğundan, $a = 3$ gibi ikinci bir baza göre denklem (31.40) ı basitçe kontrol etmek ile tüm hatalardan tam olarak kaçınamayız. (Dikkat edilmelidir ki; denklem (31.40) $\text{ebob}(a, n) > 1$ olduğunda yani $a \notin \mathbb{Z}_n^*$ durumunda sağlanmamaktadır, ancak eğer n nin sadece büyük asal çarpma olduğu durumda öyle bir a nin bulunmasının zor olduğunu kullanarak n nin bileşik sayı olduğunun gösterilmesini ümit ederiz). İlk üç Carmicheal sayıları 561, 1105 ve 1729 dur. Carmicheal sayıları oldukça nadirdir; örneğin, 100,000,000 dan daha küçük sadece 255 tane vardır. Alıştırma 31.8-2 onların neden nadiren görüldüğünü açıklamaktadır.

Asallık testimizi nasıl iyileştirebileceğimizi göstereceğiz, böylece Carmicheal sayıları tarafından kandırılmayacaktır.

Miller-Rabin rassal asallık testi

Miller-Rabin asallık testinde, basit PSEUDOPRIME testindeki problemlerin iki değişiklik ile üstesinden geliriz:

- Sadece bir taban değeri kullanmak yerine rasgele seçilen a taban değerleri denenir.
- Her bir modüler üs almayı hesaplarken, modülo n e göre 1 in aşıkır olmayan kare kökünü karelemenin son kümesi sırasında arar. Eğer bir tane bulursa, durur ve COMPOSITE olarak bildirir. Kısım 31.6 dan Sonuç 31.35 bu mantıkta bileşiklerin belirlenmesini doğrular.

Miller-Rabin asallık testi için kaba kod şu şekildedir. $n < 2$ asallığı test edilecek tek bir sayıdır ve s de $\mathbb{Z}_n^+$ dan rasgele seçilen denenecek olan elemandır. Kod sayfa 117 de tanımlanan rasgele sayı üretici RANDOM u kullanır: RANDOM $(1, n - 1)$ $1 \leq a \leq n - 1$ ü sağlayan rassal bir a tam sayısını verir. Kod yardımcı işlem WITNESS kullanır, WITNESS(a, n) sadece ve sadece a, n nin bileşik olduğuna “tanık” olması ile TRUE olur. Şöyle ki; n nin bileşik olduğunun kanıtı için a nin kullanılması mümkündür (ileride göreceğimiz bir bakış açısı). WITNESS(a, n) testi aşağıdaki gibi genişletilmiş, ancak daha verimli bir halidir

$$a^{n-1} \not\equiv 1 \pmod{n}$$

bu PSEUDOPRIME için ($a = 2$) bazını oluşturur. Öncelikle, WITNESS in inşasını sunup, doğrulayacağız ve daha sonra onu Miller-Rabin asallık testinde nasıl kullanacağımızı göstereceğiz. $t \geq 1$ iken $n - 1 = 2^t u$ ve u tek bir sayı olsun; başka bir deyişle, $n - 1$ in ikili temsili, tek tam sayı u nun ikili temsili tam t tane sıfırla takip eden temsildir. Bundan dolayı, $a^{n-1} \equiv (a^u)^{2^t} \pmod{n}$, böylece,

$a^{n-1} \bmod n$ yi öncelikle $a^u \bmod n$ i elde edip ve sonra ardışık olarak t defa kareleyerek hesaplayabiliriz.

WITNESS(a, n)

```

1   $t \geq 1, u$  tek bir sayı ve  $n-1 = 2^t u$  şeklinde  $t$  ve  $u$  olsun.
2   $x_0 = \text{MODULAR-EXPONENTIATION}(a, u, n)$ 
3  for  $i = 1$  to  $t$ 
4       $x_i = x_{i-1}^2 \bmod n$ 
5      if  $x_i \neq 1$  and  $x_{i-1} \neq 1$  and  $x_{i-1} \neq n-1$ 
6          return TRUE
7  if  $x_t \neq 1$ 
8      return TRUE
9  return FALSE

```

Bu kabakod WITNESS için $a^{n-1} \bmod n$ yi hesaplar, bunun için öncelikle $x_0 = a^u \bmod n$ yi satır 2 de hesaplar ve sonra 3-6 nolu satırlardaki **for** döngüsü içinde t kere kareler. i üzerinde tümevarımla, $x_0, x_1, \dots, x_t$ dizisi $i = 0, 1, \dots, t$ için $x_i \equiv a^{2^i u} \bmod n$ denklemini sağlayacak şekilde hesaplanır, böylece özellikle $x_t \equiv a^{n-1} \bmod n$. Bununla birlikte, 4 nolu satırdakareleme adımını gerçekleştirdikten sonra, eğer 5-6 nolu satırlarda 1'in aşikar olmayan bir kareköküne denk geldiği zaman döngü erken sonlanabilir. (bu testleri kısaca açıklayacağız). Eğer sonlanırsa, algoritma durur ve TRUE bildirilir. 7-8 nolu satırlar PSEUDOPRIME işleminde bu durumda BİLEŞİK sonuçlandığı gibi eğer $x_t \equiv a^{n-1} \bmod n$ 1 e eşit değilse TRUE döndürülür. Eğer 6 veya 8 nolu satırda TRUE cevabı vermediysek, 9 nolu satırda FALSE cevabı döner.

Şimdi, WITNESS(a, n), TRUE cevabını verirse, a yı tanık olarak kullanarak n nin BİLEŞİK olduğunun ispatını kuracağız.

8. satırdan WITNESS, TRUE döndürürse, o zaman $x_t = a^{n-1} \equiv 1 \bmod n$ olduğunu görürüz. Bununla birlikte, eğer n asal ise, o zaman Fermat teoremi aracılığıyla her $a \in \mathbb{Z}_n^+$ için $a^{n-1} \equiv 1 \bmod n$ yi elde ederiz. Bu nedenle, n asal olamaz ve $a^{n-1} \equiv 1 \bmod n$ denkleminin bu gerçeği kanıtlar.

6 nolu satırdan WITNESS, TRUE yu döndürürse, $x_{i-1} \not\equiv \pm 1 \bmod n$ ve $x_i \equiv x_{i-1}^2 \equiv 1 \bmod n$ olduğundan modülo n e göre 1 in aşikar olmayan kare kökünün x_{i-1} olduğunu buluruz. Sonuç 31.35, yalnızca n bileşik olursa, modülo n e göre 1 in bir aşikar olmayan kare kökünün olabileceğini ifade eder, bu yüzden x_{i-1} in modülo n e göre 1 in aşikar olmayan bir kare kökü olduğunu göstermek, n nin bileşik sayı olduğunu kanıtlar.

Bu WITNESS in doğruluğunun kanıtını tamamlar. Eğer WITNESS(a, n) i çağırdığımız zaman TRUE dönerse, n kuşkusuz bileşiktir a tanığı, prosedürün DOĞRU döndürme sebepleri ile birlikte (6 veya 8 nolu satırdan mı döndü?), n nin bileşik olduğunun kanıtını yapar.

Bu noktada, Miller-Rabin asallık testinin verimliliğini analiz ederken, daha sonra faydalı bulacağımız $X = \langle x_0, x_1, \dots, x_t \rangle$ dizisinin fonksiyonu gibi WITNESS için hareket tarzlarının alternatif tanımlarını kısaca sunacağız. Dikkat edilmelidir ki; bazı $0 \leq i < t$ için $x_i = 1$ ise WITNESS dizinin geriye kalanını hesaplamayabilir. Bununla birlikte, eğer hesaplasaydı, $x_{i+1}, x_{i+2}, \dots, x_t$ deki her değer 1 olacaktı ve bu durumları X dizisinin tüm elemanlarının 1 olmasını gözönüne alırız. Dört duruma sahibiz:

1. $d \neq 1$: olduğu durumda $X = \langle \dots, d \rangle$ X dizisi 1 ile bitmez. 8 nolu satırda TRUE döndür; n nin bileşik sayı olduğunun tanığı a dır. (Fermat Teoremi'nden).
2. $X = \langle 1, 1, \dots, 1 \rangle$: X dizisi tamamen 1 dir. FALSE döndür; n nin bileşik sayı olduğunun tanığı a değildir.
3. $X = \langle \dots, -1, 1, \dots, 1 \rangle$: X dizisi 1 ile biter, ancak 1 olmayan son eleman -1 'e eşittir. FALSE döndür; n 'nin bileşik sayı olduğunun tanığı a değildir.
4. $d \neq \pm 1$: olduğu durumda $X = \langle \dots, d, 1, \dots, 1 \rangle$: X dizisi 1 ile biter, ancak 1 olmayan son eleman -1 e eşit değildir. 6 nolu satırda TRUE döndür; $d, 1$ in aşikar olmayan kare kökü olduğundan n nin bileşik sayı olduğunun tanığı a dır

Şimdi, WITNESS kullanımına dayanarak Miller-Rabin asallık testini inceleyelim. Tekrar, n nin 2 de büyük tek bir tam sayı olduğunu farzedelim.

MILLER-RABIN(n, s)

```

1  for  $j = 1$  to  $s$ 
2       $a = \text{RANDOM}(1, n - 1)$ 
3      if WITNESS( $a, n$ )
4          return COMPOSITE          // kesinlikle
5  return PRIME                      // hemen hemen
```

MILLER-RABIN yöntemi n nin bileşik olduğunun kanıtını olasılıksal bir arama ile gerçekleştirir. Ana döngü (1 nolu satırdan başlayan) $\mathbb{Z}_n^+$ dan s tane rastgele a değeri alınır. Eğer seçilen a lardan birisi n nin bileşik olduğuna tanık ise MILLER-RABIN 4 nolu satırda COMPOSITE döner. Böyle bir sonuç WITNESS in geçerli olmasından ötürü her zaman doğrudur. MILLER-RABIN s denemede tanık bulamazsa, yöntem tanık olmadığını bundan dolayı ve n nin asal olduğunu varsayar. s olabildiğince büyük ise bu sonucun muhtemelen doğru olacağını göreceğiz, ancak yine de tanıkların var olmasına rağmen, yöntemin a ların seçiminde şanssız olduğu durumlardan ötürü az da olsa yanlış olabilir.

MILLER-RABIN yöntemini göstermek için n Carmichael sayısı 561 olsun, böylece $n - 1 = 560 = 2^4 35$, $t = 4$ ve $u = 35$. Yöntem baz olarak $a = 7$ yi seçerse, Kısm 31.6 da Şekil 31.4 gösterildiği gibi $\text{WITNESS } x_0 \equiv a^{35} \equiv 241 \pmod{561}$

ü ve böylece, $X = \langle 241, 298, 166, 67, 1 \rangle$ dizisini hesaplar. Bundan dolayı, $a^{280} \equiv 67 \pmod{n}$ ve $a^{560} \equiv 1 \pmod{n}$ olduğundan son aşamada, WITNESS 1 in aşıkâr olmayan kare kökünü bulur. Dolayısıyla, $a = 7$, n nin bileşik olduğuna tanıktır, WITNESS(7, n) DOĞRU döner ve MILLER-RABIN COMPOSITE sonuç verir.

Eğer n , β -bit uzunluğunda bir sayı ise, MILLER-RABIN $O(s\beta)$ tane aritmetik işleme ve $O(s\beta^3)$ bir işlemine ihtiyaç duyar, aslında asimptotik olarak s tane modüler üs almadan fazla işe ihtiyaç duyulmaz.

Miller-Rabin asallık testinin hata oranı

MILLER-RABIN PRIME cevabını veriyorsa, hata yapılmış olma olasılığı az da olsa vardır. Bununla birlikte, PSEUDOPRIME nin aksine, hata olasılığı n den bağımsızdır; bu yöntem için kötü girdi yoktur. Bu s nin boyutuna ve a tabanının seçimindeki “şans sonucuna” dayanır. Üstüne üstlük, her test denklem (31.40) ı kontrol etmekten daha katı olduğundan, genel prensipler çerçevesinde rastgele seçilen n tam sayıları için hata oranının küçük olmasını bekleriz. Sıradaki teorem daha kesin bir argümanı sunmaktadır.

Teorem 31.38

n tek bir bileşik sayı ise, n nin bileşik olduğunu göstermek için kullanılan tanıkların sayısı en az $(n - 1)/2$ dir.

Kanıt Bu kanıt, kanıt olmayanların sayısının en fazla $(n - 1)/2$ olduğunu gösterecektir ve bu da teoremi ima etmektedir.

Kanıt olmayanların $\mathbb{Z}_n^*$ da bir eleman olduğunu iddia ederek başlarız. Niçin? Kanıt olmayan bir a yı düşünelim. $a^{n-1} \equiv 1 \pmod{n}$ ü sağlamalıdır veya denk olarak $a \cdot a^{n-2} \equiv 1 \pmod{n}$. Böylece, $ax \equiv 1 \pmod{n}$ denklemi a^{n-2} olarak adlandırılan çözüme sahiptir. Sonuç 31.21 den $\text{ebob}(a, n) = 1$ i ima eden $\text{ebob}(a, n) \mid 1$ bulunur. Bundan dolayı, a , $\mathbb{Z}_n^*$ da bir elemandır; tüm kanıt olmayanlar $\mathbb{Z}_n^*$ dedir.

Kanıtı tamamlamak için, tüm kanıt olmayanların $\mathbb{Z}_n^*$ de olduklarını ve bunların $\mathbb{Z}_n^*$ nin öz altgrupları B tarafından içerildiğini gösterelim. (hatırlatmak gerekir ki; B , $\mathbb{Z}_n^*$ nin altgrubu ve $\mathbb{Z}_n^*$ ye eşit olmadığı zaman, B , $\mathbb{Z}_n^*$ nin öz altgrubudur deriz). Sonuç 31.16 dan $|B| \leq |\mathbb{Z}_n^*|/2$ e sahip oluruz. $|\mathbb{Z}_n^*| \leq n - 1$ olduğundan, $|B| \leq (n - 1)/2$ elde ederiz. Bundan dolayı, kanıt olmayanların sayısı en fazla $(n - 1)/2$ dir, böylece kanıtların sayısı en az $(n - 1)/2$ olmalıdır.

Şimdi, kanıt olmayanların hepsini içeren $\mathbb{Z}_n^*$ nin B öz altgrubunu nasıl bulacağımızı gösterelim. Kanıtı iki parçaya böleriz.

Durum 1: $x \in \mathbb{Z}_n^*$ vardır; öyle ki,

$$x^{n-1} \not\equiv 1 \pmod{n}$$

Başka bir deyişle, n Carmicheal sayısı değildir. Daha önce belirttiğimiz gibi, Carmicheal sayılarının nadir olmasından dolayı, durum 1 “uygulamada” ortaya çıkan temel sorundur (örneğin, n rastgele bir şekilde seçildiğinde ve asallık için test edildiği durumda).

$B = \{b \in \mathbb{Z}_n^* : b^{n-1} \equiv 1 \pmod{n}\}$ olsun. Açıkça, $1 \in B$ olduğundan, B boş değildir. B modülo n çarpımına göre kapalı olduğundan, Teorem 31.14 den B , $\mathbb{Z}_n^*$ nin alt grubudur. Dikkat edilmelidir ki; kanıt olmayan her eleman B ye aittir, çünkü kanıt olmayan bir a $a^{n-1} \equiv 1 \pmod{n}$ ü sağlar. $x \in \mathbb{Z}_n^* - B$ olduğundan, B nin $\mathbb{Z}_n^*$ nin öz alt grubu olduğunu anlarız.

Durum 2: Her $x \in \mathbb{Z}_n^*$ için

$$x^{n-1} \equiv 1 \pmod{n} \quad (31.41)$$

Başka bir deyişle, n Carmicheal sayısıdır. Bu durum ile uygulamada nadiren karşılaşılır. Bununla birlikte, şimdi göstereceğimiz gibi, Miller-Rabin testi (sözde asallık testinin aksine) Carmicheal sayılarının bileşik olduğunu verimli bir şekilde belirleyebilir.

Bu durumda, n bir asal sayının bir kuvveti olamaz. Bunun nedenini görmek için, tam tersini kabul edelim, şöyle ki, $n = p^e$, burada p asal ve $e > 1$ dir. Sıradaki gibi bir çelişki türeteceğiz. n nin tek sayı olduğunu varsaydığımızdan, p de tek bir sayıdır. Teorem 31.32 $\mathbb{Z}_n^*$ nin devirli bir grup olduğunu bize söyler; bu g üreticini içerir öyle ki; $\text{ord}_n(g) = |\mathbb{Z}_n^*| = \phi(n) = p^e(1 - 1/p) = (p - 1)p^{e-1}$ ($\phi(n)$ denklem (31.20) den ortaya çıkmaktadır). Denklem (31.41) den $g^{n-1} \equiv 1 \pmod{n}$. Sonra, ayrık logaritma problemi (Teorem 31.33 de $y=0$ alınır) $n - 1 \equiv 0 \pmod{\phi(n)}$ verir veya

$$(p - 1)p^{e-1} \mid p^e - 1$$

dir.

$(p - 1)p^{e-1}$, p asalı tarafından bölünebildiğinden ancak p^{e-1} tarafından bölünemediğinden $e > 1$ için bir çelişki meydana gelir. Böylece, n bir asal kuvveti değildir.

n bileşik sayısı bir asal kuvveti olmadığından, n_1 ve n_2 aralarında asal ve 1 den büyük tek sayılar olduğu durumda, $n_1 n_2$ çarpımına ayırıştırırız. (n yi ayırıştırmak için birçok yol bulunmaktadır ve bunlardan hangisini seçtiğimiz problem olmaz. Örneğin, $n = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r}$ ise $n_1 = p_1^{e_1}$ ve $n_2 = p_2^{e_2} p_3^{e_3} \cdots p_r^{e_r}$ olarak seçebiliriz).

Hatırlatmak gerekir ki; $n - 1 = 2^u$ olacak şekilde $t \geq 1$ ve u nun tek sayı olduğu durumda t ve u yu tanımladık ve a girdisi için WITNESS diziyi hesaplar

$$X = \langle a^u, a^{2u}, a^{2^2u}, \dots, a^{2^t u} \rangle$$

(tüm hesaplamalar modülo n ye göre yapılmaktadır).

Eğer $v \in \mathbb{Z}_n^*$, $j \in \{0, 1, \dots, t\}$ ve $v^{2^j u} \equiv -1 \pmod{n}$ ise (v, j) tam sayılar çiftini ***kabul edilebilir*** olarak adlandıralım.

u tek bir sayı olduğundan, kabul edilebilir çiftler kesinlikle vardır; $v = n - 1$ ve $j = 0$ şeklinde seçim yapabiliriz, böylece $(n - 1, 0)$ kabul edilebilir bir çift olur. Şimdi (v, j) kabul edilebilir çift olacak şekilde en büyük j yi alalım ve v yi düzeltelim, böylece (v, j) kabul edilebilir bir çift olur.

$$B = \{x \in \mathbb{Z}_n^* : x^{2^j u} \equiv \pm 1 \pmod{n}\}$$

olsun. B modülo n çarpımında kapalı olduğundan, $\mathbb{Z}_n^*$ nin bir alt grubudur. Bundan dolayı, Teorem 31.15 ile $|B|$, $|\mathbb{Z}_n^*|$ yi böler. Her tanık olmayan B nin elemanıdır, çünkü tanık olmayan ile üretilen X dizisinin her bir elemanı ya 1 kere ya da j nin maksimum olmasından dolayı j . elemandan sonra olmamak üzere $a-1$ taneyi içermelidir (eğer a tanık olmayan olduğunda (a, j') kabul edilebilir ise j yi nasıl seçtiğimizden $j' \leq j$ vardır).

Şimdi, $w \in \mathbb{Z}_n^* - B$ olduğunu ve dolayısıyla B nin $\mathbb{Z}_n^*$ nin bir öz alt grubu olduğunu göstermek için v nin varlığını kullanacağız. $v^{2^j u} \equiv -1 \pmod{n}$ olduğundan, Sonuç 31.29 ve Çin kalan teoreminden $v^{2^j u} \equiv -1 \pmod{n_1}$ vardır. Sonuç 31.28 yardımıyla, eş zamanlı olarak aşağıdaki denklemleri sağlayan w vardır:

$$w \equiv v \pmod{n_1},$$

$$w \equiv 1 \pmod{n_2}.$$

Böylece,

$$w^{2^j u} \equiv -1 \pmod{n_1},$$

$$w^{2^j u} \equiv 1 \pmod{n_2}.$$

Sonuç 31.29 ile $w^{2^j u} \not\equiv 1 \pmod{n}$, $w^{2^j u} \not\equiv 1 \pmod{n_1}$ ü ve $w^{2^j u} \not\equiv -1 \pmod{n}$, $w^{2^j u} \not\equiv -1 \pmod{n_2}$ ü verir. Bundan dolayı, $w^{2^j u} \not\equiv \pm 1 \pmod{n}$ ü elde ederiz ve $w \notin B$ olur.

$w \in \mathbb{Z}_n^*$ olduğunu göstermek geriye kaldı, burada, her bir modül n_1 ve n_2 için ayrı ayrı çalışacağız. n_1 modülü ile çalışırken, $v \in \mathbb{Z}_n^*$ olduğundan $\text{ebob}(v, n) = 1$ olduğunu ve $\text{ebob}(v, n_1)$ i gözlemleriz; eğer v, n ile ortak bölene sahip değil ise, kesinlikle n_1 ile de ortak bölene yoktur. $w \equiv v \pmod{n_1}$ olduğundan, $\text{ebob}(w, n_1) = 1$ olduğunu görürüz. n_2 modülü ile çalışırken, $w \equiv 1 \pmod{n_2}$ ün $\text{ebob}(w, n_2)$ yi verdiğini gözlemleriz. Bu sonuçları birleştirmek için $\text{ebob}(w, n_1 n_2) = \text{ebob}(w, n) = 1$ i veren Teorem 31.6'yı kullanırız. Bu da $w \in \mathbb{Z}_n^*$ dür.

Dolayısıyla, $w \in \mathbb{Z}_n^* - B$ L ve Durum 2'yi B nin $\mathbb{Z}_n^*$ nin bir öz alt grubu sonucunu ile bitiririz.

Her iki durumda da, n nin bileşik olduğunu gösteren tanıkların sayısının en az $(n - 1)/2$ olduğunu görürüz. ■

Theorem 31.39

Herhangi bir $n > 2$ tek tam sayısı ve s pozitif tam sayısı için MILLER-RABIN(n, s) nin hata verme olasılığı en fazla 2^{-s} dir.

Kanıt Teorem 31.38 i kullanarak, n bileşik ise, **for** döngüsünün 1-4 nolu satırlardaki her bir çalıştırılmasında, n nin bileşik olup olmadığını bulmak için x tanığının bulmanın olasılığı $1/2$ dir. MILLER-RABIN de sadece ana döngünün s iterasyonunun her birinde n nin bileşikliğine tanıklık edecek elemanın yanlış seçilmesinde durumda hata yapar. Bu tür dizi kaçırımlarının olasılığı en fazla 2^{-s} dir. ■

n asal ise, MILLER-RABIN her zaman PRIME sonucunu verir ve n bileşik ise, bunun MILLER-RABIN tarafından PRIME olarak sonuçlandırılmasının olasılığı en fazla 2^{-s} dir.

Bununla birlikte, MILLER-RABIN i rastgele seçilen büyük bir n tam sayısına uygularken, MILLER-RABIN in çıktısını doğru yorumlamak için, n nin asal olma olasılığını da göz önüne almamız gerekir. β uzunluğunu sabitlediğimizi ve β uzunluğundaki n tam sayısının asallığını test etmek için rastgele seçtiğimizi varsayalım. A , n nin asal sayı olduğu olay olsun. Asal sayı teoreminden (Teorem 31.37), n nin asal olma olasılığı yaklaşık olarak

$$\begin{aligned}\Pr\{A\} &\approx 1/\ln n \\ &\approx 1.443/\beta\end{aligned}$$

dir. Şimdi, B , MILLER-RABIN in PRIME cevabını verdiği olayı gösterebiliriz. $\Pr\{\overline{B} \mid A\} = 0$ (ya da denk olarak $\Pr\{B \mid A\} = 1$) ve $\Pr\{B \mid \overline{A}\} \leq 2^{-s}$ (ya da denk olarak $\Pr\{\overline{B} \mid \overline{A}\} > 1 - 2^{-s}$).

Ancak verilen bir n sayısı için MILLER-RABIN in PRIME dönme olasılığı $\Pr\{A \mid B\}$ nedir? Bayes teoreminin alternatif biçimi ile (denklem (C.18))

$$\begin{aligned}\Pr\{A \mid B\} &= \frac{\Pr\{A\} \Pr\{B \mid A\}}{\Pr\{A\} \Pr\{B \mid A\} + \Pr\{\overline{A}\} \Pr\{B \mid \overline{A}\}} \\ &\approx \frac{1}{1 + 2^{-s}(\ln n - 1)}\end{aligned}$$

dir. s , $\lg(\ln n - 1)$ i aşana kadar olasılık $1/2$ yi geçmez. Sezgisel olarak, n nin bileşik olması ön yargısını aşmak için n nin bileşik olduğunu veren tanığı bulmanın başarısızlığından kurtulmak için bir çok deneme yapmaya ihtiyaç duyulur. $\beta = 1024$ bitlik bir sayı için bu test yaklaşık

$$\begin{aligned}\lg(\ln n - 1) &\approx \lg(\beta/1.443) \\ &\approx 9\end{aligned}$$

denemeye ihtiyaç duyar. Her halükarda, $s = 50$ seçmek hayal edilebilir bir uygulama için yeterlidir.

Aslında, şartlar çok iyi durumda. Rastgele seçilen tek tam sayılara MILLER-RABIN uygulayarak büyük asal sayıları bulmaya çalışıyorsak, küçük bir s değeri (3 gibi) seçmek hatalı sonuç vermeye büyük ihtimalle neden olmaz,

gerçi bunu burada kanıtlamayacağız. Bunun nedeni, rastgele seçilen tek bileşik n tam sayısı için n nin bileşikliğine tanıklık etmeyen beklenen sayının büyük olasılıkla $(n - 1)/2$ den çok daha küçük olmasıdır.

Bununla birlikte, n tam sayısı rastgele seçilmediyse, Teorem 31.38 in iyileştirilmiş halini kullanarak, tanık olmayan eleman sayısının en fazla $(n - 1)/4$ olması göstermek yapabileceğimizin en iyisidir. Buna ek olarak, tanık sayısı $(n - 1)/4$ olan n tam sayıları bulunmaktadır.

Alıştırmalar

31.8-1

Tek tam sayı $n > 1$ asal veya asal kuvvet değil ise, modülo n ye göre aşikar olmayan kare kökünün olduğunu kanıtlayınız.

31.8-2 *

Euler teoremini hafifçe güçlendirmek için aşağıdaki biçimi kullanabiliriz,

her $a^{\lambda(n)} \equiv 1 \pmod{n}$ için $a \in \mathbb{Z}_n^*$

burada $n = p_1^{e_1} \cdots p_r^{e_r}$ ve $\lambda(n)$

$$\lambda(n) = \text{lcm}(\phi(p_1^{e_1}), \dots, \phi(p_r^{e_r})) \quad (31.42)$$

ile tanımlanır. $\lambda(n) \mid \phi(n)$ olduğunu kanıtlayınız. n bileşik sayısı $\lambda(n) \mid n - 1$ varsa Carmichael sayısıdır. En küçük Carmichael sayısı $561 = 3 \cdot 11 \cdot 17$ dir, burada $\lambda(n) = \text{lcm}(2, 10, 16) = 80$ ve bu 560 ı böler. Carmichael sayılarının “karesiz” (asalın karesi tarafından bölünemeyen) ve en az üç asalın çarpımı (bu nedenden dolayı, onlar çok yaygın değildir.) olduğunu kanıtlayınız.

31.8-3

x , modülo n e göre 1 in aşikar olmayan kare kökü ise, $\text{ebob}(x - 1, n)$ ve $\text{ebob}(x + 1, n)$ in her ikisinin n nin aşikar olmayan bölenleri olduğunu kanıtlayınız.

* 31.9 Tam sayıları çarpanlara ayırma

Çarpanlarına ayırmak istediğimiz; başka bir deyişle, asalların çarpımını bulacağımız bir n tam sayısına sahip olduğumuzu varsayalım. Önceki bölümdeki asallık testi bize n nin bileşik olduğunu söyler, ancak bize n nin asal çarpanlarını vermez. Büyük bir n tam sayısını çarpanlara ayırmak, n sayısının asal veya bileşik olduğunu belirlemekten çok daha zor gözükmemektedir. Bugünün süper bilgisayarları ve en iyi algoritmaları ile birlikte kullansak bile 1024 bitlik bir sayıyı uygun bir şekilde çarpanlara ayıramayız.

Pollard rho sezgiseli

R ye kadar tüm sayıların kullanılması bölme girişiminde, R^2 ye kadar olan herhangi bir sayının çarpanlarını kesin olarak vermektedir. Benzer bir çalışma için, sıradaki POLLARD-RHO işlemi R^4 e kadar olan herhangi bir sayıyı çarpanlarına ayırır (eğer şanssız olmadıkça). Bu işlem sadece sezgisel olduğundan işlem uygulamada oldukça verimli olmasına rağmen, ne çalışma zamanı ne de başarısı garanti edilemez. POLLARD-RHO nun başka bir avantajı da bellek konumunun sadece sabit sayı kadar kullanmasıdır. (eğer yapmak isterseniz, POLLARD-RHO yu programlanabilir cep hesaplayıcınızda küçük sayıların çarpanlarını bulacak şekilde kolayca uygulayabilirsiniz).

POLLARD-RHO(n)

```

1   $i = 1$ 
2   $x_1 = \text{RANDOM}(0, n - 1)$ 
3   $y = x_1$ 
4   $k = 2$ 
5  while TRUE
6       $i = i + 1$ 
7       $x_i = (x_{i-1}^2 - 1) \bmod n$ 
8       $d = \text{gcd}(y - x_i, n)$ 
9      if  $d \neq 1$  and  $d \neq n$ 
10         print  $d$ 
11     if  $i == k$ 
12          $y = x_i$ 
13          $k = 2k$ 
```

İşleyiş şu şekildedir. 1-2 nolu satırlarda i yi 1 ile başlatır ve x_1 e $\mathbb{Z}_n$ den rastgele seçilen bir atanır. 5 nolu satırda başlayan **while** döngüsü n nin çarpanlarını aramak için daima tekrar eder. **while** döngüsündeki her bir tekrar esnasında, 7 nolu satır yenilemesini

$$x_i = (x_{i-1}^2 - 1) \bmod n \quad (31.43)$$

x_i nin sonraki değerini sonsuz dizide

$$x_1, x_2, x_3, x_4, \dots, \quad (31.44)$$

elde etmek için 6 nolu satırdaki i nin uygun bir şekilde artırılması ile birlikte kullanılır. Kaba kod, anlaşılabilirlik açısından x_i değişkenleri alt indislenmiştir, ancak alt indislerin hepsi çıkarıldığında program yine aynı çalışmaktadır, çünkü x_i nin en güncel değerinin korunmuş olmasına ihtiyaç vardır. Bu güncelleme ile işlemden bellek konumunun sadece sabit sayısı kadar olanını kullanılır.

Program y değişkeninde x_i nin en güncel değerini sık sık saklar. Özellikle alt indislerinin bir kuvveti olan değerler saklanır:

$$x_1, x_2, x_4, x_8, x_{16}, \dots$$

3 nolu satırda x_1 i saklanır ve 12 nolu satırda i, k ye eşit olduğu sürece x_k yi depolanır. 4 nolu satırda, k değişkeni 2 olarak başlatılır ve 12 nolu satırda y ye yeni değer atanana kadar, 13 nolu satırda onun iki katı alınır. Bundan dolayı, $k, 1, 2, 4, 8, \dots$ dizisini izler ve daima y de saklanacak olan x_k nin sonraki değerinin alt indisini verir.

8-10 nolu satırlarda depolanan y değeri ve x_i nin o anki değeri kullanılarak n nin çarpanı bulunmaya çalışılır. Özellikle, 8 nolu satırda $d = \gcd(y - x_i, n)$. ortak bölenlerin en büyüğü hesaplanır. 9 nolu satırda, n nin aşikar olmayan bir çarpanı d bulunursa, 10 nolu satırda d yazdırılır.

Bu işlemde çarpanı bulmak başlangıçta bir şekilde gizemli olarak görülebilir. Bununla birlikte dikkat edilmelidir ki; POLLARD-RHO hiçbir zaman yanlış cevap vermez; verdiği cevap n nin aşikar olmayan bir bölenidir. POLLARD-RHO hiçbir cevap vermeyebilir, gerçi, herhangi bir böleni vereceğini garanti etmemektedir. Bununla birlikte, **while** döngüsündeki $\Theta(\sqrt{p})$ tekrardan sonra, POLLARD-RHO yönteminin n nin bir p çarpanını vermesini beklemek için önemli bir sebebe sahip olduğumuzu göreceğiz. Böylece, n bileşik ise, yaklaşık $n^{1/4}$ güncellemeden sonra n yi çarpanlara ayırmak için yeterli kadar böleni bulmayı ümit ederiz, çünkü n nin 1 den büyük her asal çarpanı p $\sqrt{n}$ den daha küçüktür.

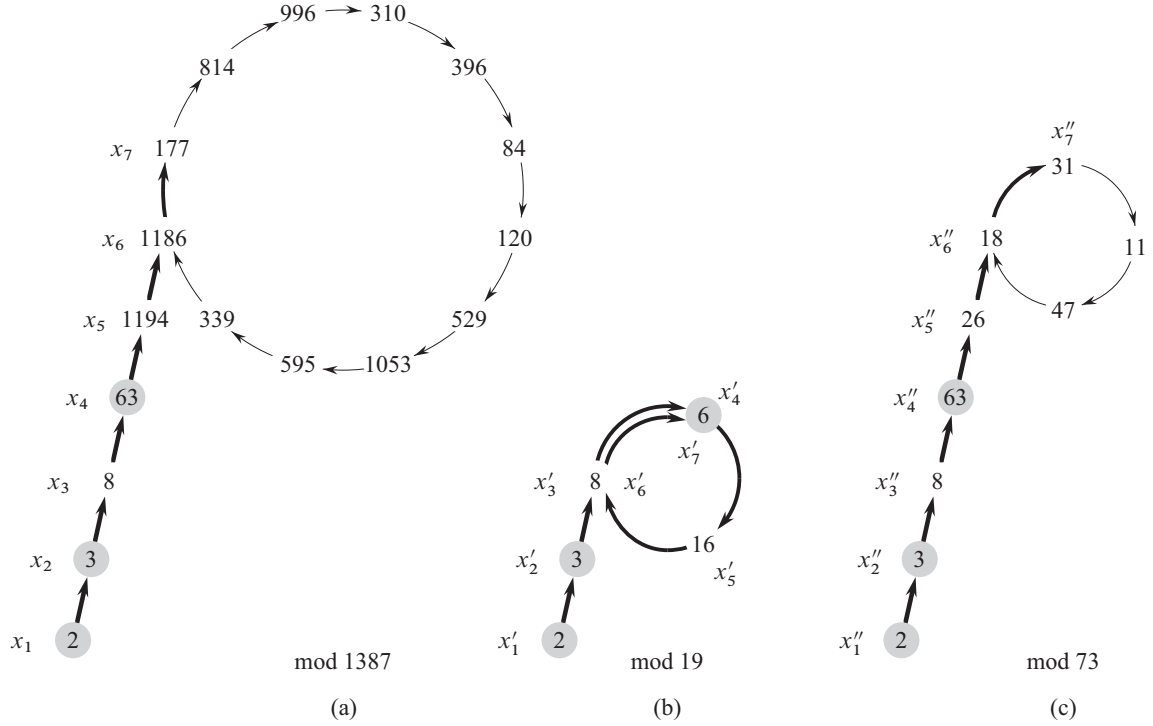
Bir değeri tekrar etmek için modülo n e göre rastgele bir dizinin uzunluğunu çalışarak bu işlemin nasıl davrandığının analizine başlayacağız. $\mathbb{Z}_n$ sonlu olduğundan ve (31.44) teki dizi her bir değer sadece öncekine bağlı olduğundan, (31.44) teki dizi en sonunda kendini tekrar eder. Bazı $j < i$ için $x_i = x_j$ olacak şekilde bir x_i ye erişildiğinde bir döngümüz olur, çünkü $x_{i+1} = x_{j+1}$, $x_{i+2} = x_{j+2}$ ve buna benzer. “rho sezgiseli” isminin verilmesinin nedeni, Şekil 31.7 de görüldüğü gibi, $x_1, x_2, \dots, x_{j-1}$ dizisini rho’nun “kuyruğu” olarak ve $x_j, x_{j+1}, \dots, x_i$ dizisini çevrimini rho’nun gövdesi olarak alacağız.

x_i nin dizisi için tekrarın ne kadar süreceği konusunu ele alalım. Bu bilgi tam olarak ihtiyacımız olan bir şey değil, ancak daha sonra bu argümanı nasıl değiştirebileceğimizi göreceğiz. Bu tahminin amacı için,

$$f_n(x) = (x^2 - 1) \bmod n$$

fonksiyonunun “rastgele” fonksiyon gibi davrandığını varsayalım. Tabii ki de, gerçek anlamda rastgele değildir, ancak bu varsayım POLLARD-RHO nun gözlenen davranışı ile tutarlı sonuçları verir. Sonra, $\mathbb{Z}_n$ deki düzgün dağılıma göre $\mathbb{Z}_n$ den bağımsızca seçilmiş her bir x_i yi göz önüne alabiliriz. Kısım 5.4.1 in doğum günü paradoksu analizi ile, dizi çevrimi öncesinde $\Theta(\sqrt{n})$ kadar aşamanın yapılması beklenir.

Şimdi ihtiyaç duyulan değişikliğe geelim. p, n nin aşikar olmayan bir çarpanı olsun, öyle ki; $\gcd(p, n/p) = 1$. Örneğin, n nin çarpanlara ayrılması $n = p_1^{e_1} p_2^{e_2} \cdots p_r^{e_r}$ ise, p yi $p_1^{e_1}$ olarak alabiliriz. (akılda kalması açısından güzel bir örnek: eğer $e_1 = 1$ ise p, n nin en küçük asal çarpanıdır).



Şekil 31.7 Pollard rho sezgiseli, (a) $x_1 = 2$ ile başlayarak, $x_{i+1} = (x_i^2 - 1) \bmod 1387$ özyinelemesi ile değerler üretilir. 1387 nin asal çarpanları, 19 ve 73. 19 çarpanının bulunmasından önce kalın çizilmiş oklar iterasyon adımlarının işletilmesini göstermektedir. İnce çizilmiş oklar tekrardaki ulaşılammış değerleri “rho” şeklini göstermek için işaret etmektedir. Gölgeleştirilmiş değerler POLLARD-RHO tarafından depolanan y değerleridir. $\text{ebob}(63-177, 1387)=19$ hesaplandığında, 19 çarpanı, $x_7 = 177$ ye erişildiğinde bulunur. Tekrarlanan ilk x değeri 1186 dır, ancak bu değer tekrarlanmasından önce 19 çarpanı bulunur. (b) Değerler modülo 19 a göre aynı özyineleme ile üretilir. (a) kısmında verilen her x_i değeri modülo 19 a göre burada gösterilen x'_i değerine denktir. Örneğin, her iki $x_4 = 63$ ve $x_7 = 177$ modülo 19'a göre 6 dır. (c) Değerler modülo 73 e göre aynı özyineleme ile üretilir. (a) kısmında verilen her x_i değeri modülo 73 e göre burada gösterilen x''_i değerine denktir. Çin kalan teoreminden, (a) kısmındaki her düğüm noktası, biri (b) den diğeri (c) den olmak üzere düğüm noktaları çiftine karşılık gelir.

$\langle x_i \rangle$ dizisi, modülo p ye göre $\langle x'_i \rangle$ dizisine karşılık gelir, burada her i için $x'_i = x_i \bmod p$ dir.

Buna ek olarak, f_n sadece modülo n de aritmetik işlemler ile tanımlandığından (kareleme ve çıkarma), x'_{i+1} i, x'_i den hesaplayabiliriz; dizinin “modülo p ” açısı-

sından değerlendirilmesi modülo n ye göre değerlendirilmesinin daha küçük bir şeklidir:

$$\begin{aligned}
 x'_{i+1} &= x_{i+1} \bmod p \\
 &= f_n(x_i) \bmod p \\
 &= ((x_i^2 - 1) \bmod n) \bmod p \\
 &= (x_i^2 - 1) \bmod p \quad (\text{Alıştırma 31.1-7 den}) \\
 &= ((x_i \bmod p)^2 - 1) \bmod p \\
 &= ((x'_i)^2 - 1) \bmod p \\
 &= f_p(x'_i).
 \end{aligned}$$

Böylece, $\langle x'_i \rangle$ dizisini açık bir şekilde hesaplamamamıza rağmen, bu dizi iyi tanımlıdır ve $\langle x'_i \rangle$ dizisindeki gibi aynı özyinelemeye uyar.

Önceki mantığı devam ettirirsek, $\langle x'_i \rangle$ dizisinin tekrarlarından önce aşamaların beklenen sayısını $\Theta(\sqrt{p})$ olarak buluruz. Eğer p , n ile karşılaştırıldığında küçük bir sayı ise, $\langle x'_i \rangle$ dizisi, $\langle x_i \rangle$ dizisinden çok daha hızlı tekrar edebilir. Aslında, Şekil 31.7 deki (b) ve (c) kısımlarının gösterdiği gibi $\langle x_i \rangle$ dizisinin ik elemanı modülo n değil de modülo p ye göre denk olduğunda, $\langle x'_i \rangle$ dizisi tekrar eder.

t , $\langle x'_i \rangle$ dizisindeki tekrar eden ilk elemanın indisi olsun ve $u > 0$ onunla ilgili üretilen çevrimin uzunluğunu gösterebilir. t ve $u > 0$ en küçük değerlerdir; öyle ki; her $i \geq 0$ için $x'_{t+i} = x'_{t+u+i}$. Yukarıdaki argümanlar ile t ve u nun beklenen değerleri $\Theta(\sqrt{p})$ dür. Dikkat edilmelidir ki; $x'_{t+i} = x'_{t+u+i}$ ise, $p \mid (x_{t+u+i} - x_{t+i})$. Böylece, $\text{ebob}(x_{t+u+i} - x_{t+i}, n) > 1$.

Dolayısıyla, POLLARD-RHO $k \geq t$ için herhangi x_k değerini y olarak sakladığında, $y \bmod p$ her zaman çevrim üzerinde modülo p dir. (yeni değer y olarak saklandığında, bu değer de çevrim üzerinde modülo p dir). sonuçta, k , u dan daha büyük bir değere atanır ve sonra, işlemde y değerini değiştirmeden tüm döngü çevrim üzerinde modülo p ye göre çalıştırılır. İşlem n nin bir çarpanını, x_i , y nin modülo p ye göre bir önceki değerine $x_i \equiv y \pmod{p}$ olduğu durumda denk gelir.

Zaman zaman p nin katı hesaplanmasına rağmen, p çarpanı bulunduğu farzedilebilir. t ve u nun beklenen değerleri $\Theta(\sqrt{p})$ ise, p çarpanını üretmek için beklenen adım sayısı $\Theta(\sqrt{p})$ dür.

Bu algoritma iki nedenden ötürü beklendiği gibi gerçekleşmeyebilir. Öncelikle, çalışma zamanının sezgisel analizi kesin değildir, ancak çevrimin modülo p ye göre değeri $\sqrt{p}$. özyinelemeyeden çok daha büyük olabilir. Bu durumda, algoritma doğru çalışmaktadır, ancak istenilenden çok daha yavaştır. Pratikte, bu sorun tartışılabilir. İkinci olarak, n nin bu algoritma tarafından bulunan aşıkâr bölenleri her zaman 1 veya n i olabilir. Örneğin, p ve q asal iken $n = pq$ olduğunu varsayalım. p için t ve u nun değerleri ile q için t ve u nun değerleri aynı çıkabilir ve böylece, p çarpanı ortak bölenlerin en büyüğü işlemindeki gibi her zaman q çarpanını verir. Her iki çarpan aynı bulunduğundan, aşıkâr çarpan $pq = n$ ortaya

çkar ki bu da gereksizdir. Tekrar söylemek gerekirse, bu problem uygulamada önemsizdir. Eğer gerekliyse, $x_{i+1} = (x_i^2 - c) \bmod n$ biçiminin farklı özyinelemesi ile sezgisel bir yaklaşıma tekrar başlayabiliriz. ($c = 0$ ve $c = 2$ değerlerinden kaçınılmamız, diğer değerlerde sakınca yoktur; detaylarına burada değinmeyeceğiz).

Tabii ki de bu analiz sezgiseldir ve kesin değildir, çünkü özyineleme gerçek anlamda “rastgele” değildir. Bununla birlikte, bu işlem pratikte düzgün çalışmaktadır ve bu sezgisel analizin belirttiği gibi verimli gözükmektedir. Büyük bir sayının küçük asal çarpanlarını bulmak için bir seçimdir. β -bitlik n tamsayısını çarpanlara ayırmak için sadece $\lfloor n^{1/2} \rfloor$ den küçük tüm asal çarpanlarını bulmamız gerekmektedir ve POLLARD-RHO nun en fazla $n^{1/4} = 2^{\beta/4}$ tane aritmetik işleme ve en fazla $n^{1/4} \beta^2 = 2^{\beta/4} \beta^2$ tane bit işlemine ihtiyaç duymaktadır. POLLARD-RHO nun aritmetik işlemlerin $\Theta(\sqrt{p})$ sayısı ile n nin p küçük çarpanını bulma özelliği genelde onun en cazip özelliğidir.

Alıştırmalar

31.9-1

Şekil 31.7(a) da gösterilen ilgili çalışmayı gözönüne alarak POLLARD-RHO, 1387 nin 73 çarpanını ne zaman verir?

31.9-2

$f : \mathbb{Z}_n \rightarrow \mathbb{Z}_n$ fonksiyonun ve $x_0 \in \mathbb{Z}_n$ başlangı değerinin bize verildiğini varsayalım. $i = 1, 2, \dots$ için $x_i = f(x_{i-1})$ i tanımlayalım. t ve $u > 0$ en küçük değerler olsun; öyle ki, $i = 0, 1, \dots$ için $x_{t+i} = x_{t+u+i}$. Pollard rho algoritmasındaki terimler ile t , kuyruğun uzunluğu ve u , rho’nun çevriminin uzunluğudur. t ve u yu tam olarak belirleyen verimli bir algoritma veriniz ve onun çalışma zamanını analiz ediniz.

31.9-3

p asal ve $e > 1$ iken, POLLARD-RHO nun p^e biçiminde çarpanını kaç aşamada bulmasını bekleriz?

31.9-4 *

POLLARD-RHO nun bir sakıncası yazıldığı gibi, özyinelemenin her aşamasında ortak bölenlerin en büyüğünün bir sefer hesaplamasına ihtiyaç duymasıdır. Bunun yerine, birkaç x_i değerini çarpımını bir satırda biriktirerek ve sonra bu çarpımı ebob hesaplamasında x_i nin yerine kullanarak ebob hesaplamalarını toplu yapabiliriz. Bu fikri nasıl uygulayabileceğinizi, neden çalıştığını ve β -bitlik bir n sayısı ile çalışırken en verimli durum için toplu iş boyutunun ne olacağını dikkatli bir şekilde ifade ediniz.

Problemler

31-1 İkili ebob algoritması

Bilgisayarların çoğu, çıkarma işlemini, ikilik tabanda verilmiş bir tam sayının eşlik testini (tek veya çift) ve ikiye bölmeyi kalanı hesaplamadan daha hızlı gerçekleştirmektedir. Bu problem **ikili ebob algoritmasını** araştırmaktadır, böylece, Euclid algoritmasında kullanılan kalan hesaplamalarından kaçınılmaktadır.

- a ve b çift ise $ebob(a, b) = 2 \cdot ebob(a/2, b/2)$ olduğunu kanıtlayınız.
- a tek ve b çift ise $ebob(a, b) = ebob(a, b/2)$ olduğunu kanıtlayınız.
- a ve b tek ise $ebob(a, b) = ebob((a - b)/2, b)$ olduğunu kanıtlayınız.
- $O(\lg a)$ sürede çalışan $a \geq b$ iken a ve b tam sayılarını girdi olarak alan verimli ikili ebob algoritması tasarlayınız. Her bir çıkarma, eşlik testi ve ikiye bölmenin birim zaman aldığını varsayalım.

31-2 Öklid algoritmasındaki bit işlemlerinin analizi

- Uzun bölme için bilinen “kağıt ve kalem” algoritmasını düşünelim: a yı b ye böldüğümüzde, q bölümünü ve r kalanını elde ederiz. Bu yöntemin $O((1 + \lg q)\lg b)$ bit işlemine ihtiyaç duyduğunu gösteriniz.
- $\mu(a, b) = (1 + \lg a)(1 + \lg b)$ yi tanımlayalım. EUCLID ile $ebob(a, b)$ hesaplama problemini $ebob(b, a \bmod b)$ ye indirgeme işleminde gereken bit işlemlerinin sayısının en fazla, bazı yeterince büyük $c > 0$ sabiti için $c(\mu(a, b) - \mu(b, a \bmod b))$ olduğunu gösteriniz.
- EUCLID(a, b) nin genelde $O(\mu(a, b))$ bit işlemine ve iki tane β bitlik girdi ile $O(\beta^2)$ bit işlemine ihtiyaç duyduğunu gösteriniz.

31-3 Fibonacci sayıları için üç algoritma

Bu problem verilen n için n . Fibonacci sayısını F_n hesaplamakta kullanılan üç yöntemin verimliliğini karşılaştırır. Sayıların büyüklüğünden bağımsız olarak iki sayıyı toplananın, çıkarmanın ve çarpmanın maliyetinin $O(I)$ olduğunu varsayalım.

- (3.22) deki özyinelemeye dayanarak F_n yi hesaplamayı basit özyineleme ile yapıldığında çalışma zamanının n de üssel olduğunu gösteriniz (örneğin, sayfa 775 teki FIB işlemine bakınız).
- F_n nin $O(n)$ sürede taramalı tablo ile nasıl hesaplanacağını gösteriniz.

- c. F_n nin $O(\lg n)$ sürede sadece tam sayı toplama ve çarpma işlemi nasıl hesaplanacağını gösteriniz. (İpucu:

$$\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$$

matrisini ve bunun kuvvetlerini göz önüne alın).

- d. Şimdi, β -bitlik iki sayıyı toplamının $\Theta(\beta)$ süre aldığını ve β -bitlik iki sayının çarpımının $\Theta(\beta^2)$ süre aldığını varsayalım. Basit aritmetik işlemler için bu daha makul maliyet ölçüsü ile bu üç yöntemin çalışma zamanları nedir?

31-4 İkinci dereceden kalanlar

p tek asal sayı olsun. Eğer $x^2 = a \pmod{p}$ bilinmeyen x için bir çözümü var ise, $a \in \mathbb{Z}_p^*$ sayısı **ikinci dereceden kalan**dır.

- a. Modülo p ye göre tam $(p-1)/2$ tane ikinci dereceden kalanın var olduğunu gösteriniz.
- b. p asal ise, modülo p ye göre a ikinci dereceden kalan ise 1, diğer durumlarda -1 olacak şekilde $a \in \mathbb{Z}_p^*$ için **Legendre sembolünü** $\left(\frac{a}{p}\right)$ tanımlarız. $a \in \mathbb{Z}_p^*$ ise

$$\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod{p}$$

olduğunu kanıtlayınız. Verilen bir a sayısının modülo p ye göre ikinci dereceden kalan olup olmadığına karar veren verimli bir algoritma yazınız. Algoritmanızın verimliliğini analiz ediniz.

- c. p , $4k+3$ formunda bir asal ve $a, \mathbb{Z}_p^*$ de ikinci dereceden kalan ise, $a^{k+1} \pmod{p}$ nin, modülo p ye göre a nın kare kökü olduğunu kanıtlayınız.. a modülo p ikinci dereceden kalanının kare kökünü bulmak için ne kadar süreye ihtiyaç vardır?
- d. Rastgele seçilen p asal modülünde ikinci dereceden olmayan kalanı bulmak için verimli rastgele algoritma; başka bir deyişle, $\mathbb{Z}_p^*$ deki bir elemanın ikinci dereceden olmayan kalan olduğunu tanımlayınız. Ortalamada algoritmanızın kaç tane aritmetik işleme ihtiyaç duyar?

Bölüm notları

Niven ve Zuckerman [265] temel sayılar teorisine mükemmel bir giriş sağlamaktadır. Knuth [210], ortak bölenlerin en büyüğünü bulmak için varolan algo-

ritmaların yanısıra diğer temel sayılar teorisi algoritmalarının iyi bir tartışmasını içermektedir. Bach [30] ve Riesel [295] hesaplanabilir sayılar teorisinin güncel araştırmalarını vermektedir. Dixon [91] çarpanlara ayırma ve asalılık testini gözden geçirmektedir. Pomerance [280] tarafından düzenlenmiş bildiri kitapları birkaç mükemmel araştırma makalelerini içermektedir. Güncel olarak, Bach ve Shalit [31] hesaplanabilir sayılar teorisinin temelleri için olağanüstü bir gözden geçirme sağlamaktadır.

Knuth [210] Euclid algoritmasının kökenini tartışmıştır. Bu 7 nolu Kitapta Önerme 1 ve 2 de Yunan matematikçi Öklid'in milattan önce 300 yılı civarında yazmış olduğu *Elemanlar* olarak yer almaktadır. Öklid'in tanımı milattan önce 375 yılı civarında Eudoxus 'a dayandırılabilir. Öklid'in algoritması aşıkâr olmayan en eski algoritma ünvanı ile ödüllendirilebilir; sadece antik Mısırlıların çarpma algoritması bununla rekabet eder. Shalit [312] Euclid algoritmasının tarihsel analizini sırayla vermektedir.

Knuth, Çin kalan teoreminin özel bir durumunu (Teorem 31.27) tarih tam kesin olmamakla beraber milattan önce 200 ile milattan sonra 200 yılları arasında bir süre yaşamış olan Çinli matematikçi Sun Tsu'ya dayandırmaktadır. Aynı özel durum Yunan matematikçi Nichomachus tarafından milattan sonra 100 yılı civarında verilmiştir. Bu durum Chhin Chiu-Shoa tarafından 1247 yılında genelleştirilmiştir. Çin kalan teoremi son olarak belirtilmiş ve tam halinin genelleştirilmesinin kanıtı 1734 yılında L. Euler tarafından verilmiştir.

Burada sunulan rassal asalılık testi algoritması Miller [255] ve Rabin'e [289] dayandırılarak verilmiştir; sabit çarpanlar ile birlikte bilinen en hızlı asalılık testi algoritmasıdır. Teorem 31.39 un kanıtı Bach'ın [29] önerdiği küçük bir uyarlamadır. MILLER-RABIN için daha güçlü sonucun kanıtı Monier [258, 259] tarafından verilmiştir. Uzun seneler için asalılık testi verimli (polinom zamanlı) algoritma elde etmek için gerekli bir koşul olarak görünen rassalılık gibi bir problemin klasik haliydi. Bununla birlikte, 2002 yılında Agrawal, Kayal ve Saxema [4] polinom zamanlı çalışan, kararlı asalılık testi algoritması önererek herkesi şaşırtmışlardır. O zamana kadar, Cohen ve Lenstra'ya [73] dayanan, bilinen en hızlı kararlı asalılık testi algoritması n girdisi için süper polinom olarak görünen $(\lg n)^{O(\lg \lg n)}$ sürede çalışır. Yine de, pratik amaçlar için rasgeleleştirilmiş asalılık testi algoritmaları daha verimli kalmakta ve tercih edilirler.

Büyük "rastgele" asalların bulunması problemi Beauchemin, Brassard, Crepeau, Goutier ve Pomerance [36] tarafından güzelce tartışılmıştır.

Ortak anahtarlı kriptografi kavramı Diffie ve Hellman'a [87] dayanmaktadır. RSA şifreleme sistemi 1977 yılında Rivest, Shamir ve Adleman [296] tarafından önerilmiştir. O zamandan beri, kriptografi alanı canlanmıştır. RSA şifreleme sistemi için bizim anlayışımız derinleşmiştir ve bugünkü uygulamalar burada sunulan temel tekniklerin önemli iyileştirilmelerini kullanmaktadır. Bunlara ek olarak, şifreleme sistemlerinin güvenli olduğunu kanıtlamak için birçok yeni tek-

nik geliştirilmiştir. Örneğin, Goldwasser ve Micali [142] gelişigüzelliğin güvenilir ortak anahtarlı şifreleme şemaları tasarlamak için verimli bir araç olduğunu gösterirler. İmzalama şemaları için Goldwasser, Micali ve Rivest [143] sayısal imzalama şeması sunmuştur; sahteciliğin her akla uygun durumu çarpanlara ayırma kadar zor olduğu kanıtlanır. Menezes, van Oorschot ve Vanstone [254] uygulamalı kriptografi için bir genel bakış sağlamaktadır.

Çarpanlara ayırma için sezgisel rho Pollard [277] tarafından düşünülmüştür. Burada sunulan versiyon Brent [56] tarafından önerilmiştir.

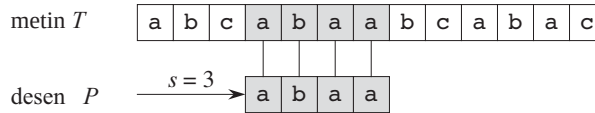
Büyük sayıları çarpanlara ayırmak için en iyi algoritmalar çarpanlarına ayrılacak n sayısının uzunluğunun küp kökü ile kabaca üssel olarak büyüyen çalışma zamanına sahiptir. Genel sayı-alan eleği çarpanlara ayırma algoritması (Buhler, Lenstra ve Pomerance [57] tarafından geliştirilen, Pollard [278] ve Lenstra ve arkadaşlarının [232] sayıalan eleği çarpanlara ayırma algoritmasındaki fikirlerinin genişlemesidir ve Coppersmith [77] ve diğerleri tarafından iyileştirilmiştir) belki de büyük girdiler için genel olarak en verimli algoritmadır. Bu algoritmanın kesin analizini vermek zor olmasına rağmen, kabul edilebilir varsayımlar altında, çalışma zamanını $L(\alpha, n) = e^{(\ln n)^\alpha (\ln \ln n)^{1-\alpha}}$ iken $L(1/3, n)^{1.902+o(1)}$ olarak türetebiliriz.

Lenstra'nın eliptik-eğri yöntemi bazı girdiler için sayı-alan eleği yönteminden daha etkili olabilir, çünkü aynı Pollard rho yönteminde olduğu gibi küçük asal çarpan p yi hızlıca bulabilir. Bu yöntemle, p yi bulmanın süresi $L(1/2, p)^{\sqrt{2}+o(1)}$ olarak değerlendirilir.

Metin düzeltme programlarının sıklıkla metindeki bir desenin tüm oluşlarını bulması gerekir. Genellikle, metin, düzeltilecek olan belge ve aranılan desen, kullanıcı tarafından sağlanan özel bir kelimedir. “katar eşleme” olarak adlandırılan bu problem için verimli algoritmaların metin düzeltme programının çabuk vermesi konusunda büyük yardımı vardır. Onların diğer uygulamaları, katar eşleme algoritmaları DNA dizisindeki özel desenleri araştırır. İnternet arama motorları sorgulardaki web sitelerini bulmak için de kullanılır.

Katar eşleme problemini aşağıdaki şekilde formülize ederiz. Metnin, uzunluğu n olan $T[1..n]$ dizisi olduğunu ve desenin uzunluğu $m \leq n$ olan $P[1..m]$ dizisi olduğunu varsayalım. Bundan başka, P ve T nin elemanlarının sonlu Σ alfabesinden karakterler olduklarını farzedelim. Örneğin, $\Sigma = \{0, 1\}$ veya $\Sigma = \{a, b, \dots, z\}$ ye sahip olabiliriz. P ve T katarları genelde karakterlerin **katarları** (*strings*) olarak adlandırılır.

Şekil 32.1 i kaynak göstererek, eğer $0 \leq s \leq n - m$ ve $T[s + 1..s + m] = P[1..m]$ (şöyle ki; $1 \leq j \leq m$ için $T[s + j] = P[j]$) ise T metninde desen P nin ***s kaydırma ile görüldüğünü*** (ya da denk olarak T metninde desen P nin $s + 1$ ***nolu pozisyonda başladığı görülür***) söyleyebiliriz. T de, P s kaydırma ile görülürse, s ye ***geçerli kaydırma*** deriz; aksi halde, s ye ***geçersiz kaydırma*** deriz. ***Katar eşleme problemi*** verilen bir T metninde verilen P desenin kaç geçerli kaydırma ile bulunması problemidir.



Şekil 32.1 Katar eşleme probleminin bir örneği, burada $T = \text{abcabaabac}$ metninde $P = \text{abaa}$ deseninin tüm görülmelerinin bulunmasını istiyoruz. Desen, geçerli bir kaydırma olan $s = 3$ kaydırma ile metin içerisinde sadece bir kez bulunur. dikey çizgi desenin her bir karakterini metindeki katar eşlemesine bağlanır ve eşlenen tüm karakterler gölgelendirilmiştir.

Algoritma	Ön işlem süresi	Eşleme zamanı
Basit	0	$O((n - m + 1)m)$
Rabin-Karp	$\Theta(m)$	$O((n - m + 1)m)$
Sonlu otomata	$O(m \Sigma)$	$\Theta(n)$
Knuth-Morris-Pratt	$\Theta(m)$	$\Theta(n)$

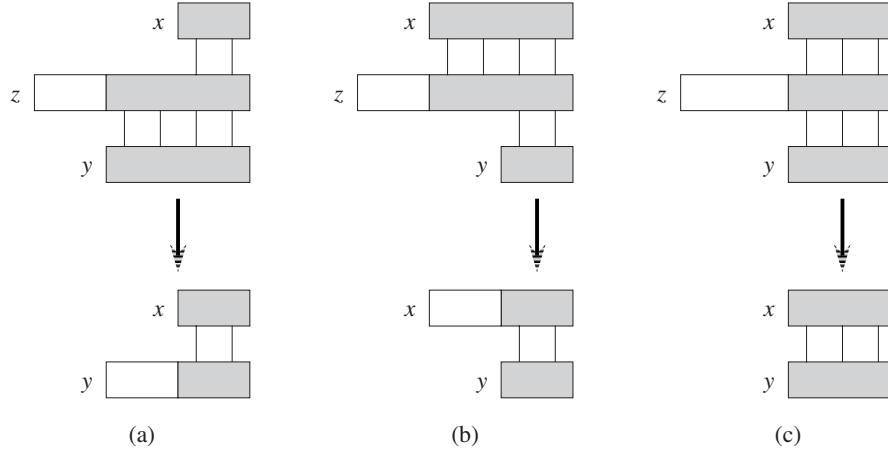
Şekil 32.2 Bu bölümdeki katar eşleme algoritmaları ve onların ön işlem ve eşleme süreleri.

Kısım 32.1 de gözden geçirdiğimiz basit kaba kuvvet algoritması hariç, bu bölümdeki her katar eşleme algoritması desene bağlı olarak bazı ön işlemler gerçekleştirir ve sonra tüm geçerli kaydırmaları bulur; bu sonraki aşamayı “eşleme” olarak adlandırırız. Şekil 32.2 bu bölümdeki her algoritmanın ön işleme ve eşleme zamanlarını gösterir. Her algoritmanın toplam çalışma zamanı ön işleme ve eşleme zamanlarının toplamıdır. Kısım 32.2 Rabin ve Karp’ın ilginç katar eşleme algoritmasını sunar. Bu algoritmanın en kötü çalışma zamanı $\Theta((n - m + 1)m)$ basit yönteminkinden daha iyi olmasa bile, uygulamada ve ortalamada daha iyidir. Ayrıca, diğer desen eşleme problemlerini güzel bir şekilde genelleştirmektedir. Kısım 32.3 katar eşleme algoritmasını tanımlar, metindeki verilen bir P deseninin görülmelerinin araştırılması için özel tasarlanan sonlu otomatanın inşası ile başlar. Bu algoritma, ön işleme için $O(m |\Sigma|)$ süre alır, ancak eşleme süresi $\Theta(n)$ ’dür. Kısım 32.4 benzer ama daha akıllıca olan Knuth-Morris-Pratt (veya KMP) algoritmasını sunar; bu algoritma aynı $\Theta(n)$ eşleme süresine sahiptir ve ön işleme süresini sadece $\Theta(m)$ süreye indirir.

Gösterim ve terminoloji

Σ alfabesindeki karakterler kullanılarak oluşturulan tüm sonlu uzunluklu karakter dizilerinin kümesini Σ^* (“sigma yıldız” olarak okuyun) ile gösteririz. Bu bölümde, sadece sonlu uzunluklu dizileri göz önünde bulunduracağız. Sıfır uzunluklu **boş katar** Σ^* ’dadır ve ε ile gösterilir. x , katarının uzunluğu $|x|$ ile gösterilir. İki katar x ve y nin **ardarda eklenmesi** ile uzunluğu $|x| + |y|$ elde edilen dizi xy ile gösterilir ve x in y ile izlendiği karakterleri içerir.

Katar w , x katarının **önekidir** (prefix) deriz, $w \sqsubset x$ ile gösterilir, bazı $y \in \Sigma^*$ katarı için $x = wy$ dir. Dikkat edilmelidir ki; eğer $w \sqsubset x$ varsa, $|w| \leq |x|$ dir. Benzer olarak, katar w , x katarının **sonekidir** (suffix) deriz, $w \sqsupset x$ ile gösterilir, bazı $y \in \Sigma^*$ katarı için $x = wy$ dir. Önekte olduğu gibi $w \sqsubset x$ $|w| \leq |x|$ i ima eder. Örneğin, $ab \sqsubset abcca$ ve $cca \sqsupset abcca$ e sahibiz. Boş katarı ε , her katarının son ve önekidir. Herhangi x ve y katarları ve herhangi bir a karakteri için ancak ve ancak $x \sqsubset y$ varsa, $xa \sqsubset ya$ e sahibiz.



Şekil 32.3 Lemma 32.1 in şekilsel kanıtı. $x \sqsubseteq z$ ve $y \sqsubseteq z$ olduğunu varsayalım. Şekilin üç kısmı lemmadaki üç durumu açıklamaktadır. Dikey çizgiler katarların eşleme bölgelerini (gölgelendirilme ile gösterilen) bağlamaktadır. **(a)** $|x| \leq |y|$ ise $x \sqsubseteq y$. **(b)** $|x| \geq |y|$ ise $y \sqsubseteq x$. **(c)** $|x| = |y|$ ise $x = y$ dir.

Aynı zamanda, $\sqsubseteq$ ve $\sqsupseteq$ ün geçiş bağıntıları olduğuna dikkat ediniz. İzleyen lemma daha sonra faydalı olacaktır.

Lemma 32.1 (Örtüşen sonek (Overlapping-suffix) lemması)

Farzedelim ki; x, y and z $x \sqsubseteq z$ ve $y \sqsubseteq z$ ü sağlayan katarları olsun. $|x| \leq |y|$ ise $x \sqsubseteq y$. $|x| \geq |y|$ ise $y \sqsubseteq x$. $|x| = |y|$ ise $x = y$ dir.

Kanıt Grafiksel kanıt için Şekil 32.3 e bakınız. ■

Gösterimin kısa olması açısında, $P[1..m]$ deseninin k karakter öneki $P[1..k]$ yi P_k ile gösteririz. Böylece, $P_0 = \varepsilon$ ve $P_m = P = P[1..m]$ dir. Benzer olarak, T metninin k karakter önekini T_k ile gösteririz. Bu gösterimi kullanarak, katar eşleme problemini $P \sqsubseteq T_{s+m}$ olarak şekilde $0 \leq s \leq n - m$ aralığındaki tüm s kaydırmalarını bulmak olarak tanımlayabiliriz.

Kabakodumuzda, basit işlem olarak eşit uzunluktaki katarının eşitlik açısından karşılaştırılmasına izin veririz. Katarları soldan sağa karşılaştırılırsa ve yanlış eşleme bulunduğu karşılaştırma duruyorsa, bulunan eşleşmiş karakterlerin sayısının doğrusal olduğu bir testin aldığı kadar süre aldığını varsayabiliriz. Kesin olmak için, “ $x == y$ ” testinin $\Theta(t + 1)$ süre aldığı varsayılır, burada t , en uzun z katarının uzunluğudur; öyle ki; $z \sqsubseteq x$ ve $z \sqsubseteq y$. ($t = 0$ durumuyla baş edebilmek için $\Theta(t + 1)$ yerine $\Theta(t)$ ü yazarız; ilk karakterler karşılaştırıldığında eşleşmiyor, ancak bu karşılaştırmayı gerçekleştirmek belli bir süre alır).

32.1 Sade katar-eşleme algoritması

Basit algoritma, s nin $n - m + 1$ tane olası değerinin herbirinin $P[1..m] = T[s + 1..s + m]$ koşulunu kontrol eden bir döngü kullanarak geçerli tüm kaydırmaları bulur.

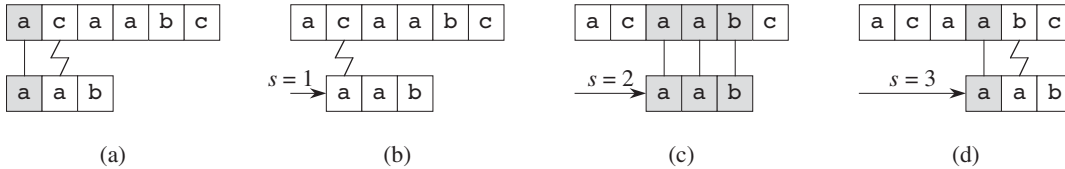
NAIVE-STRING-MATCHER (T, P)

```

1   $n = T.length$ 
2   $m = P.length$ 
3  for  $s = 0$  to  $n - m$ 
4      if  $P[1..m] == T[s + 1..s + m]$ 
5          print  $s$  “kaydırma ile desen görülür”
```

Şekil 32.4 basit katar eşleme işlemini metin üzerinde deseni içeren kaydırılan bir “kalıp (template)” gibi metindeki karşı gelen karakterlerin kalıptaki tüm karakterlerin kaydırılmasının işaretlenmesi ile betimlemiştir. 3-5 nolu satırlardaki **for** döngüsü mümkün olan her kaydırmayı açıkça gözönüne alır. 4 nolu satırdaki test o anki kaydırmanın geçerli olup olmadığına karar verir; bu test tüm konumlar tam olarak eşleşene veya bir yanlış eşleme bulunana kadar dolaylı bir şekilde ilgili karakter konumunu kontrol ederek döngüyü devam eder. 5 nolu satır geçerli her s kaydırmasını yazar.

NAIVE-STRING-MATCHER işlemi $O((n - m + 1)m)$ süre alır ve bu sınır en kötü durum için sıkıdır. Örneğin, a^n katar metnini (n tane a dan oluşan katar) ve desen a^m . s kaydırması için $n - m + 1$ olası değer her biri için, 4 nolu satırdaki dolaylı döngüde ilgili karakterleri karşılaştırmak amacıyla geçerli kaydırma m defa gerçekleştirilmelidir. Böylece, en kötü durum çalışma zamanı $\Theta((n - m + 1)m)$ dür, $m = \lfloor n/2 \rfloor$ ise $\Theta(n^2)$ dür. Ön işlemeye ihtiyaç olmadığından, NAIVE-STRING-MATCHER nin çalışma zamanı onun eşleme zamanına eşittir.



Şekil 32.4 Metin $T = acaabc$ ve desen $P = aab$ için basit katar eşlemenin işlemi. Desen P yi metinde bir sonrakine kaydırığımız kalıp gibi hayal edebiliriz. **(a)-(d)** ardışık olan sıralama basit katar eşlemesi tarafından denir. Her kısımda, dikey çizgiler eşleme için ilgili alanlar bulunduğu anda bağlama işlemini yapar (gölgelendirme ile gösterilir) ve kıvrımlı çizgi yanlış eşleşen ilk karakter bulunduğu bağlanır. (c) kısmında gösterildiği gibi $s = 2$ kaydırmasında algoritma desenin bir görülmesini bulur.

Göreceğimiz gibi, NAIVE-STRING-MATCHER bu problem için en uygun işlem değildir. Aslında, bu bölümde, Knuth-Morris-Pratt algoritmasının en kötü durumda daha iyi olduğunu göreceğiz. Basit dizi eşleme verimsizdir, çünkü s nin diğer değerleri gözönüne alındığında, s nin bir değeri için metin hakkında elde edilen bilgiyi tamamen gözardı eder. Bununla birlikte, böyle bir bilgi oldukça değerlidir. Örneğin, $P = \text{aaab}$ ve geçerli $s = 0$ bulursak, 1,2,3 kaydırmalarının hiçbirisi geçerli olmaz, çünkü $T[4] = b$ dir. İzleyen bölümlerde, bilginin bu sıralamasının kullanımını verimli yapmak için birkaç yöntemi değerlendireceğiz.

Alıştırmalar

32.1-1

Basit dizi eşlemesi $T = 000010001010001$ metnindeki $P = 0001$ deseni için yaptığı karşılaştırmaları gösteriniz.

32.1-2

P desenindeki tüm karakterlerin farklı olduğunu farzedelim. n karakterlik T metninde NAIVE-STRING-MATCHER nin çalışmasını $O(n)$ süreye nasıl hızlandırılacağını gösteriniz.

32.1-3

P deseni ve T metninin $d \geq 2$ için d elemanlı bir alfabaden $\Sigma_d = \{0, 1, \dots, d-1\}$ sırasıyla m ve n uzunluklu *rassal* seçilen katarlar olduklarını farzedelim. Basit algoritmanın 4 nolu satırındaki dolaylı döngü tarafından yapılan bu yürütmelerin karakter bazlı karşılaştırmalarının *beklenen* sayısının

$$(n - m + 1) \frac{1 - d^{-m}}{1 - d^{-1}} \leq 2(n - m + 1)$$

olduğunu gösteriniz. (basit algoritma yanlış eşleşme veya tüm deseni bulduğunda, verilen kaydırma için karakterleri karşılaştırmayı durdurduğunu varsayalım). Böylece, rassal seçilen katarlar için basit algoritma oldukça verimlidir.

32.1-4

P deseninin, karakterlerin *keyfi* bir katardaki eşlemenin olduğu (sıfır uzunluktaki katarı olsa bile) $\diamond$ **boşluk karakterinin** görülmelerine izin verdiğini farzedelim. Örneğin, $\text{ab} \diamond \text{ba} \diamond \text{c}$ deseni, cabccbacbacab metninde şu şekilde oluşur

$$\begin{array}{ccccccc} \text{c} & \text{ab} & \text{cc} & \text{ba} & \text{cba} & \text{c} & \text{ab} \\ & \text{ab} & \diamond & \text{ba} & \diamond & \text{c} & \end{array}$$

ve

$$\begin{array}{ccccccc} \text{c} & \text{ab} & \text{ccbac} & \text{ba} & & \text{c} & \text{ab} \\ & \text{ab} & \diamond & \text{ba} & \diamond & \text{c} & \end{array}$$

Dikkat edilmelidir ki; boşluk karakteri metindekilerin hepsi olmayacak şekilde desendeki tekrarların keyfi sayılarının oluşması olabilir. T metninde P deseninin görülüp görülmediğine polinom zamanda karar veren bir algoritma oluşturunuz ve bu algoritma çalışma zamanını analiz ediniz.

32.2 Rabin-Karp algoritması

Rabin ve Karp, genelleştirildiğinde iki boyutlu desen eşleme gibi pratikteki diğer ilgili problemlerde çalışabilen katar eşleme algoritması önermişlerdir. Rabin-Karp algoritması ön işleme için $\Theta(m)$ süre kullanır ve en kötü durum çalışma zamanı ise $\Theta((n - m + 1)m)$ dir. Bununla birlikte, bazı varsayımlara dayanarak bu algoritmanın ortalama durum çalışma zamanı daha iyidir.

Bu algoritma, iki sayının belli bir modüloya göre denkliği gibi basit sayılar teorisi kavramlarını kullanmaktadır. Konu ile ilgili tanımlar için Kısım 31.1 i kaynak olarak kullanabilirsiniz.

Amaçların açıklanması için, $\Sigma = \{0, 1, 2, \dots, 9\}$ olduğunu varsayalım, böylece her karakter bir rakamdır. (genel durumda, her karakterin $d = |\Sigma|$ olacak şekilde d tabanında bir basamak olduğunu farzedebiliriz). k uzunluğundaki bir ondalık sayının gösteriminde olduğu gibi bir katarını k tane ardışık karakterler topluluğu olarak görebiliriz. Böylece, 31415 katarı onluk sayı 31,415'e karşılık gelir. Girdi karakterlerini şekilsel semboller ve rakamlar olarak yorumladığımızdan dolayı, bu bölümde, standart yazı tipinde onları rakam olarak göstermeyi uygun bulduk.

Verilen bir $P[1..m]$, *deseninde* p buna karşılık gelen ondalık değer olsun. Benzer tarzda, verilen bir $T[1..n]$ metninde $s = 0, 1, \dots, n - m$ için t_s m uzunluğundaki $T[s + 1..s + m]$ alt katarının ondalık değerini gösterebilir. Şüphesiz, $t_s = p$ olabilmesi ancak ve ancak $T[s + 1..s + m] = P[1..m]$ ile mümkündür; böylece, s nin geçerli bir kaydırma olabilmesi ancak ve ancak $t_s = p$ ile mümkündür. Eğer p yi $\Theta(m)$ sürede ve her t_s değerini toplam $\Theta(n - m + 1)$ sürede hesaplayabilirsek¹, tüm s kaydırmalarını p ile tek tek karşılaştırarak $\Theta(m) + \Theta(n - m + 1) = \Theta(n)$ sürede belirleyebiliriz. (şimdilik, p ve t_s nin çok büyük sayılar olmalarının olasılığı konusunda endişelenmeyelim).

Horner kuralını (Kısım 30.1'e bakınız) kullanarak p yi $\Theta(m)$ sürede hesaplayabiliriz:

$$p = P[m] + 10(P[m - 1] + 10(P[m - 2] + \dots + 10(P[2] + 10P[1]) \dots))$$

Benzer şekilde, $T[1..m]$ den t_{0+1} $\Theta(m)$ sürede hesaplayabiliriz.

¹ s , $n - m + 1$ tane farklı değer aldığından $\Theta(n - m + 1)$ yerine $\Theta(n - m)$ yazarız. “+1” asimptotik anlamda önemlidir; çünkü $m=n$ iken tek t_s değerini hesaplamak $\Theta(0)$ süre değil, $\Theta(1)$ süre alır.

Geri kalan $t_1, t_2, \dots, t_{n-m}$ değerlerini $\Theta(n - m)$ sürede hesaplamak için

$$t_{s+1} = 10(t_s - 10^{m-1}T[s + 1]) + T[s + m + 1] \quad (32.1)$$

olduğundan t_s den t_{s+1} i sabit sürede hesaplayabileceğimizi gözlemliyoruz.

$10^{m-1}T[s + 1]$ i çıkartarak t_s den yüksek mertebeli basamak kaldırılır, 10 ile çarpmak sola doğru bir basamak kaydırmaktır, $T[s + m + 1]$ i eklemek uygun düşük mertebeli basamağı kazandırır. Örneğin, $m = 5$ ve $t_s = 31415$ ise yüksek mertebeli basamak $T[s + 1] = 3$ ü kaldırmayı isteriz ve

$$\begin{aligned} t_{s+1} &= 10(31415 - 10000 \cdot 3) + 2 \\ &= 14152 \end{aligned}$$

elde etmek için düşük mertebeli basamağı ($T[s + 5 + 1] = 2$) buluruz.

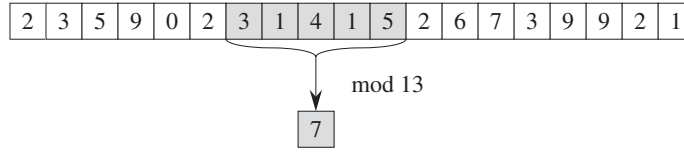
Sabit 10^{m-1} i önceden hesaplırsak (bu uygulama için kolay anlaşılır $O(m)$ süreli yöntem yeterli olmasına rağmen, bu işlemi Bölüm 31.6 daki teknikleri kullanarak $O(\lg m)$ sürede yapabiliriz), denklem (32.1) in her koşturulması sabit sayıda aritmetik işlem ile yapılır. Bundan dolayı, p yi $\Theta(m)$ sürede ve tüm $t_0, t_1, \dots, t_{n-m}$ leri $\Theta(n - m + 1)$ sürede hesaplayabiliriz. Bu nedenle, $T[1..n]$ metnindeki $P[1..m]$ deseninin tüm tekrarlarını bulmayı ön işleme aşamasını $\Theta(m)$, ve eşleme aşamasını $\Theta(n - m + 1)$ sürede yaparak gerçekleştiririz.

Şimdiye kadar, bilinçli olarak bir problemi atladık: p ve t_s nin uygun olarak çalışmak için çok olabileceği durum. P, m karakter içeriyorsa, p deki (m basamak uzunlukta) her aritmetik işlemin “sabit süre” aldığını mantık açısından varsayamayız. Şans eseri, Şekil 32.5 in gösterdiği gibi bu problem kolayca çözebiliriz: p ve t_s yi uygun bir q modülüne göre hesaplamak. p modülo q yu $\Theta(m)$ sürede ve tüm t_s değerlerinin modülo q ya göre $\Theta(n - m + 1)$ sürede hesaplayabiliriz. $10q$ bilgisayarın bir kelime uzunluğuna uyacak bir şekilde asal q modülünü seçersek, tek duyarlılıklı aritmetik ile tüm gerekli hesaplamaları gerçekleştirebiliriz. Genelde, d elemanlı alfabe $\{0, 1, \dots, d - 1\}$ ile q seçeriz; öyle ki, dq bilgisayarın bir kelime uzunluğuna sığar ve özyineleme denklemi (3.21) i modülo q ile çalışmak için düzeltilir, böylece o meydana gelir

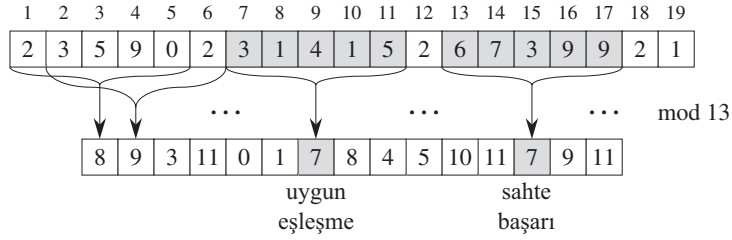
$$t_{s+1} = (d(t_s - T[s + 1]h) + T[s + m + 1]) \bmod q \quad (32.2)$$

burada m basamaklı metnin yüksek mertebeli pozisyonundaki “1” basamağının değeri $h \equiv d^{m-1}$ dür.

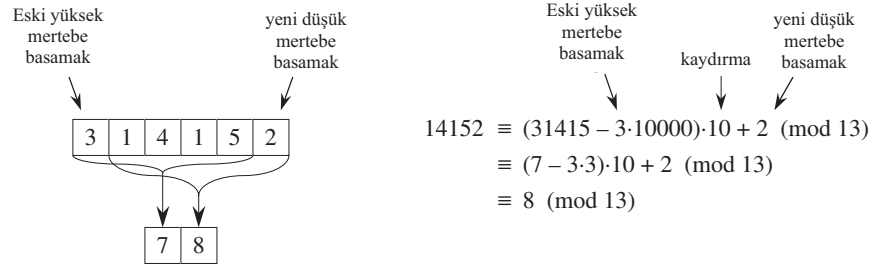
Bununla birlikte, modülo q daki çalışmaların çözümleri mükemmel değildir: $t_s \equiv p \bmod q$, $t_s = p$ yi ima etmemektedir. Öte yandan, eğer $t_s \not\equiv p \bmod q$ varsa, $t_s \not\equiv p$ ye sahibiz, böylece, s geçersizdir. Bundan dolayı, $t_s \equiv p \bmod q$ testini geçersiz s kaydırmalarını hükümsüz kılmak için sezgisel test gibi kullanırız. $t_s \equiv p \bmod q$ için herhangi s kaydırmasının gerçekten geçerli olup olmadığını



(a)



(b)



(c)

Şekil 32.5 Rabin-Karp algoritması. Her karakter ondalık basamaktır ve bunları modülo 13 e göre hesaplarız. Pencere uzunluğu 5 tir ve gölgeli gösterilmiştir. Gölgeli sayının sayısal değeri modülo 13 e hesaplanmış ve 7 değerini verir. **(b)** Pencere uzunluğu 5 in olası tüm pozisyonları için aynı katarı modülo 13 e göre hesaplanan değerleri içerir. Desenin $P = 31415$ olduğunu varsayarak, $31415 \equiv 7 \pmod{13}$ olduğundan öyle bir pencere arıyoruz ki; değeri modülo 13 e göre 7 olsun. Algoritma öyle iki pencere bulur ve şekilde gölgeli olarak gösterilmiştir. İlk olarak, metinde 7 nolu pozisyonunda başlayan eleman gerçekten desenin bir görülmesidir, bunun aksine, ikincisinde 13 nolu pozisyonunda başlayan değer sahte başarıdır. **(c)** Pencere için önceki pencerede verilen değer ile yeni değeri sabit sürede nasıl hesaplarız. İlk pencere 31415 değerini alır. Yüksek mertebeli basamak 3 ü atarak, sola doğru kaydırarak (10 ile çarpma) ve düşük mertebeli basamak olarak 2 yi eklemek bize yeni değer 14152 yi verir. Tüm hesaplamalar modülo 13 e göre gerçekleştirildiğinden, ilk pencerenin değeri 7 ve yeni pencerenin değeri 8’ dir.

veya **sahte bir vuruş** (*spurious hit*) elde edip etmediğimiz test edilmelidir. Bu ek ölçüm açıkça $P[1..m] = T[s+1..s+m]$ koşulunu kontrol eder. Eğer q yeterince büyük ise, sahte vuruşların yeterince seyrek olmasını ve böylece ek kontrol işleminin maliyetinin az olmasını ümit ederiz.

İzleyen işlem bu fikirleri kesin hale getirmektedir. İşlemin girdileri T metni, P deseni kullanılan d tabanı (genel olarak $|\Sigma|$ alınır) ve q asalıdır.

RABIN-KARP-MATCHER(T, P, d, q)

```

1   $n = T.length$ 
2   $m = P.length$ 
3   $h = d^{m-1} \bmod q$ 
4   $p = 0$ 
5   $t_0 = 0$ 
6  for  $i = 1$  to  $m$                                 // ön işleme
7       $p = (dp + P[i]) \bmod q$ 
8       $t_0 = (dt_0 + T[i]) \bmod q$ 
9  for  $s = 0$  to  $n - m$                                 // eşleme
10     if  $p == t_s$ 
11         if  $P[1..m] == T[s+1..s+m]$ 
12             yazdır  $s$  “kaydırma ile desen görülür”
13     if  $s < n - m$ 
14          $t_{s+1} = (d(t_s - T[s+1])h + T[s+m+1]) \bmod q$ 

```

RABIN-KARP-MATCHER prosedürü şu şekilde çalışır. Tüm karakterler d tabanındaki basamaklar olarak yorumlanır. t nin alt indisleri sadece anlaşılabilirlik için sağlanmış; eğer tüm alt indisler kaldırılırsa program doğru çalışır. 3 nolu satır m basamaklı pencerenin yüksek mertebeli basamağının pozisyonunun değeri h olarak başlatılır. 4-8 nolu satırlarda p değeri, $P[1..m] \bmod q$ olarak ve t_0 değeri $T[1..m] \bmod q$ olarak hesaplanır. 9-14 nolu satırlardaki **for** döngüsü olası tüm s kaydırmalarını izleyen değişmezi sabit kılarak tekrar eder:

10 nolu satır ne zaman çalıştırılırsa çalışsın, $t_s = T[s+1..s+m] \bmod q$.

10 nolu satırda $p = t_s$ (“vuruş (hit)”) ise, 11 nolu satırda $P[1..m] = T[s+1..s+m]$ olup olmadığı sahte vuruşun olasılığını hükümsüz kılmak için kontrol eder. 12 nolu satırda herhangi bir geçerli kaydırma bulunduğu çıktı verilir. Eğer $s < n - m$ (13 nolu satırda kontrol edilen) ise, **for** döngüsü en az bir kere daha çalıştırılır ve böylece 14 nolu satır, 10 nolu satıra geri döndüğümüzde, döngü değişmezinin tutulduğundan emin olmak için çalıştırılır. 14 nolu satırda denklem (32.2) kullanılarak sabit zamanda $t_s \bmod q$ dan $t_{s+1} \bmod q$ değeri hesaplanır.

RABIN-KARP-MATCHER $\Theta(m)$ ön işleme zamanında yapılır ve Rabin-Karp algoritması her geçerli kaydırmayı doğruladığından (basit katar eşleme algorit-

masında olduğu gibi), ondaki eşleme zamanı en kötü durumda $\Theta((n - m + 1)m)$ süredir. $P = a^m$ ve $T = a^n$ ise $n - m + 1$ olası kaydırmalar geçerli olduğundan, doğrulama $\Theta((n - m + 1)m)$ sürede yapılır.

Birçok uygulamada, c sabiti kadar geçerli kaydırma bekleriz. Bu tip uygulamalarda, algoritmanın beklenen eşleme zamanı sadece $O((n - m + 1) + cm) = O(n + m)$ ve bununla sahte vuruş aşaması için gereken zamanı toplarız. Değerleri mod q ya indirgemenin Σ^* den $\mathbb{Z}_q$ ya rassal bir eşleme gibi davrandığı varsayımının sezgisel analizi temel alırız. (Kısım 11.3.1 deki çırpılama için bölmenin kullanımı tartışmasına bakınız. Yapılabilir bir yaklaşım için q nun tamsayılardan uygun bir boyutta rasgele seçildiği varsayımı olmasına rağmen, öyle bir varsayımı formülize etmek ve kanıtlamak zordur. Burada, bu formülasyonun devamla etmeyeceğiz). Sahte vuruşların sayısının $O(n/q)$ olmasını bekleyebiliriz, çünkü keyfi seçilmiş t_s nin modülo q da p ye denk olma şansının $1/q$ olduğunu tahmin edebiliriz. 10 nolu satırın testi $O(n)$ pozisyonunda başarısız olduğundan ve her bir vuruş için $O(m)$ süre harcadığımızdan, Rabin-Karp algoritması ile beklenen eşleme zamanı

$$O(n) + O(m(v + n/q))$$

burada v , geçerli kaydırmaların sayısıdır. $v = O(l)$ ve $q \geq m$ seçersek, bu çalışma zamanı $O(n)$ dir. Şöyle ki; geçerli kaydırmaların beklenen sayısı küçük ($O(l)$) ise, desenin uzunluğundan daha büyük q asalını seçeriz, sonra Rabin-Karp prosedürünün sadece $O(n + m)$ eşleme zamanı kullanmayı bekleriz. $m \leq n$ ise beklenen eşleme zamanı $O(n)$ dir.

Alıştırmalar

32.2-1

$q = 11$ modülünde, $P = 26$ deseninin $T = 3141592653589793$ metninde ararken Rabin-Karp eşlemesi kaç tane sahte vuruş ile karşılaşır?

32.2-2

Rabin-Karp yöntemini, verilen k desenlerinin kümesindeki herhangi birisinin görülmesi için metinde katar arama problemine nasıl genişletebiliriz? k desenlerinin hepsinin aynı uzunlukta olduğunu varsayarak başlayalım. Daha sonra, çözümünüzü farklı uzunluklardaki desenler için genelleştiriniz.

32.2-3

$n \times n$ lik katarında $m \times m$ lik desenin aranması problemi ile baş etmek için Rabin-Karp yöntemini nasıl genişleteceğimizi gösteriniz. (desen dikey ve yatay olarak kaydırılabilir, ancak döndürülemez).

32.2-4

Alice n -bit uzunluğundaki $A = \langle a_{n-1}, a_{n-2}, \dots, a_0 \rangle$ dosyasının kopyasına sahip ve benzer olarak Bob'da n -bitlik $B = \langle b_{n-1}, b_{n-2}, \dots, b_0 \rangle$ dosyasına sahiptir. Alice ve Bob dosyalarının aynı olup olmadığını bilmek istiyor. A veya B nin hepsinin iletilmesinden kaçınmak için şu izleyen olasılıksal kontrolü kullanırlar. $q > 1000n$ asalını beraber seçerler ve $\{0, 1, \dots, q-1\}$ den x tam sayısını rassal seçerler. Sonra, Alice şunu değerlendirir

$$A(x) = \left(\sum_{i=0}^{n-1} a_i x^i \right) \bmod q$$

ve benzer olarak Bob $B(x)$ i değerlendirir. $A \neq B$ ise 1000 de en fazla 1 kere $A(x) = B(x)$ i görürüz, bunun aksine, eğer iki dosya aynı ise, $A(x)$ ister istemez $B(x)$ ile aynıdır (*İpucu:* Alıştırma 31.4-4 e bakınız).

32.3 Sonlu otomata ile katar eşleme

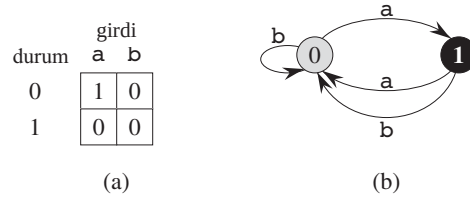
Birçok katar eşleme algoritması bilgiyi işlemek için basit bir makineyi, sonlu bir otomata (özdevinir) kurar, bu makine P deseninin tüm görülmeleri için T metin dizisini tarar. Bu bölüm, böyle bir otomatayı kurmak için bir yöntem sunar. Bu katar eşleme algoritması çok verimlidir: metnin her bir karakterini sabit sürede tam olarak bir kere denetler. Bundan dolayı, otomatayı kurmak için desenin ön işlemlerinden sonra eşleme süresi $\Theta(n)$ dir. Bununla birlikte, Σ büyük ise, otomatayı kurma süresi de büyük olabilir. Kısım 32.4 bu problem çevresinde akıllı bir yol tanımlamaktadır.

Bu bölüme sonlu otomatanın tanımı ile başlarız. Sonra, özel katar eşleme otomatasını analiz edeceğiz ve metindeki bir desenin görülmelerini bulmak için nasıl kullanıldığını göstereceğiz. Son olarak, verilen bir girdi deseni için katar eşleme otomatasını nasıl kuracağımızı göstereceğiz.

Sonlu otomata (Finite automata)

Şekil 32.6 da şekillendirildiği gibi, *sonlu otomata* M , 5 elemanlıdır $(Q, q_0, A, \Sigma, \delta)$ burada

- Q , *durumların* sonlu bir kümesidir.
- $q_0 \in Q$ *başlangıç durumudur*.
- $A \subseteq Q$ *kabul durumlarının* farkedilebilir kümesidir.
- Σ sonlu *girdi alfabesidir*.
- δ , $Q \times \Sigma$ den Q ya olan bir fonksiyondur, bu M nin *geçiş fonksiyonu* olarak adlandırılır.



Şekil 32.6 $Q = \{0, 1\}$ durum kümesi, $q_0 = 0$ başlangıç durumu ve $\Sigma = \{a, b\}$ girdi alfabesi ile iki durumlu basit bir sonlu otomata. **(a)** δ geçiş fonksiyonunun tablo gösterimi. **(b)** Tabloya denk durum geçiş şekli. Siyah ile gösterilen durum 1 sadece kabul durumudur. Yönlü oklar geçişleri gösterir. Örneğin, b ile işaretlenen durum 1 ile durum 0'ı birleştiren ok $\delta(1, b) = 0$ ı belirtir. Bu otomata tek sayıda a ile biten katarları kabul eder. Daha açık bir ifadeyle, bu otomata x katarını ancak ve ancak $x = yz$ iken kabul eder, burada $y = \varepsilon$ veya y, a, b ile biter ve k tek bir sayı olmak üzere $z = a^k$ dir. Örneğin, abaaa girdisi, başlangıç durumunu içererek, $\langle 0, 1, 0, 1, 0, 1 \rangle$ durumlarının dizisi bu otomataya girer ve böylece, bu girdi kabul edilir. abbaa girdisi için $\langle 0, 1, 0, 0, 1, 0 \rangle$ durumlarının dizisi bu otomataya girer ve böylece, bu girdi reddedilir.

Sonlu otomata q_0 durumunda başlar ve girdi dizisinin karakterlerini tek bir sürede okur. Eğer otomata q durumunda ve a girdi karakterini okursa, q durumundan $\delta(q, a)$ durumuna taşıma (“geçiş yapılır”) olur. O andaki durum q , A nın bir elemanı olduğu sürece, M makinesi o zamana kadar okunan katarını **kabul eder**. Girdi kabul edilmezse, **reddedilir**.

Son durum fonksiyonu olarak adlandırılan M sonlu otomatası ϕ fonksiyonu na neden olur, bu fonksiyon Σ^* dan Q ya olur; öyle ki; $\phi(w)$, M de w katarının taranmasından sonra sonlanma durumudur. Böylece, M , w katarını kabul eder ancak ve ancak $\phi(w) \in A$ varsa kabul eder. ϕ fonksiyonunu geçiş fonksiyonunu kullanarak özyinelemeli olarak tanımlarız:

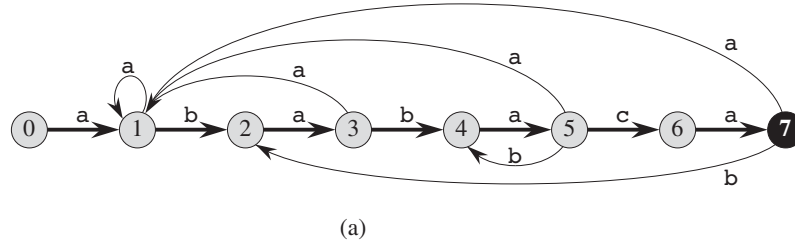
$$\begin{aligned}\phi(\varepsilon) &= q_0, \\ \phi(wa) &= \delta(\phi(w), a) \text{ her } w \in \Sigma^* \text{ için } a \in \Sigma\end{aligned}$$

Katar eşleme otomatası

Verilen bir P deseni için ön işleme aşamasını kullanmadan metin dizisinde arama yapmak amacıyla katar eşleme otomatası kurarız. Şekil 32.7 $P = ababaca$ deseni için otomatayı nasıl kuracağımızı ifade eder. Bu andan itibaren, P deseninin sabitlenmiş bir katar olduğunu varsayalım ve kısalık açısından, gösterimimizde P ye göre hareket etmeyeceğiz.

Verilen bir $P[1..m]$ ye karşılık gelen katar eşleme otomatasını ayrıntıları ile belirtmek amacıyla, P ye karşılık **sonek (suffix) fonksiyonu** olarak adlandırılan σ yardımcı fonksiyonunu tanımlarız. σ , fonksiyonu $\Sigma^* \rightarrow \{0, 1, \dots, m\}$ ye eşler; öyle ki; $\sigma(x)$, P nin en uzun önekinin uzunluğudur, böylece x in soneki olur:

$$\sigma(x) = \max \{k : P_k \sqsubseteq x\} \quad (32.3)$$



durum	girdi			P
	a	b	c	
0	1	0	0	a
1	1	2	0	b
2	3	0	0	a
3	1	4	0	b
4	5	0	0	a
5	1	4	6	c
6	7	0	0	a
7	1	2	0	

(b)

i	—	1	2	3	4	5	6	7	8	9	10	11
$T[i]$	—	a	b	a	b	a	b	a	c	a	b	a
durum $\phi(T_i)$	0	1	2	3	4	5	4	5	6	7	2	3

(c)

Şekil 32.7 (a) ababaca dizisi ile biten tüm katarları kabul eden katar eşleme otomatası için durumlara arası geçişlerin şekli. 0 başlangıç durumudur ve 7 (siyah ile boyanmış) ise sadece kabul durumudur. i durumundan j durumuna a ile işaretlenmiş yönlendirilmiş ok $\delta(i, a) = j$ yi temsil eder. Sağa yönlendirilmiş, kalınlaştırılmış oklar otomatanın “kolunu” teşkil eder, bunlar, desen ve girdi karakterleri arasındaki başarılı eşlemeye karşılık gelir. Sola yönlendirilmiş oklar ise başarısız olan eşlemelerdir. Başarısız eşleşmeye karşılık gelen bazı oklar çıkarılır; yapılagelişler gereği, eğer i durumu bazı $a \in \Sigma$ için a ile işaretlenmiş çıkışa sahip değil ise, $\delta(i, a) = 0$ dır. **(b)** Karşılık gelen δ geçiş fonksiyonu ve katar $P = ababaca$ deseni. Desen ve girdi karakterleri arasındaki başarılı eşlemelere karşılık gelen girişler gölgelendirilmiş şekilde gösterilmektedir. **(c)** $T = abababacaba$ metni üzerinde otomatanın işleyişi. Her $T[i]$ karakteri altında $\phi(T_i)$ durumu meydana gelir, otomata, T_i önekini işledikten sonra bu duruma geçer. Otomata desenin bir kere görülmesini bulur ve pozisyon 9 da biter.

σ ekleme fonksiyonu, her boş katar $P_0 = \varepsilon$ her katarının soneki olduğundan iyi tanımlıdır. Örneklerdeki gibi, $P = ab$ deseni için $\sigma(\varepsilon) = 0$, $\sigma(ccaca) = 1$ ve $\sigma(ccab) = 2$ yi buluruz. m uzunluğundaki P deseni için $\sigma(x) = m$ olabilmesi ancak ve ancak $P \sqsubseteq x$ ile mümkündür. Sonek fonksiyonunun tanımından, $x \sqsubseteq y$, $\sigma(x) \leq \sigma(y)$ i gerektirir.

Verilen $P[1..m]$ desenine karşılık gelen katar eşleme otomatasını şu şekilde tanımlarız:

- Durumlar kümesi Q , $\{0, 1, \dots, m\}$ dir. q_0 başlangıç durumu, 0 durumudur ve m durumu ise sadece kabul durumudur.
- δ geçiş fonksiyonu herhangi bir q durumu ve a karakteri için izleyen denklem ile tanımlanır:

$$\delta(q, a) = \sigma(P_q a) \quad (32.4)$$

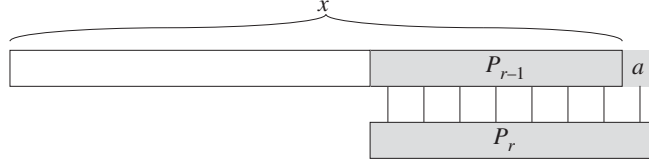
$\delta(q, a) = \sigma(P_q a)$ yı tanımlarız, çünkü T metin dizisinde P deseninin en uzun önekinin eşleştiği izi takip etmek istiyoruz. T nin en son okunan karakterlerini gözönüne alıyoruz. T nin alt katarını yani $T[i]$ ile sona eren alt katarı, P nin bazı P_j önekini eşlemek için bu P_j öneki T_i nin soneki olmak zorundadır. $q = \phi(T_i)$ olduğunu varsayalım, böylece, T_i yi okuduktan sonra, otomata q durumundadır. δ geçiş fonksiyonunu tasarlarız, böylece bu q durumu T_i nin soneki ile eşleşen P nin en uzun önekinin uzunluğunu bize söyler. Başka bir deyişle, q durumunda $P_q \supset T_i$ ve $q = \sigma(T_i)$. ($q = m$ olduğunda, P nin tüm m karakterleri T_i nin soneki ile eşleşir ve böylece bir eşlem bulmuş oluruz). Bu nedenle, $\phi(T_i)$ ve $\sigma(T_i)$, q ya eşit olduğundan, otomatının izleyen değişmezi koruduğunu göreceğiz (aşağıdaki Teorem 32.4 te):

$$\phi(T_i) = \sigma(T_i) \quad (32.5)$$

Eğer otomata q durumunda ve sonraki karakter $T[i + 1] = a$ yı okur ise, $T_i a$ nin soneki olan P nin en uzun öneğine karşılık gelmesine neden olan geçişi isteriz ve durum $\sigma(T_i a)$ dir. P_q , P nin en uzun öneki, yani T_i nin soneki olduğundan, P nin en uzun öneki yani $T_i a$ nin soneki sadece $\sigma(T_i a)$ değil ve aynı zamanda $\sigma(P_q a)$ dir. (sayfa 1000 deki, Lemma 32.3 $\sigma(T_i a) = \sigma(P_q a)$ olduğunu kanıtlar). Bundan dolayı, otomata q durumunda olduğu zaman, a karakterinde geçiş fonksiyonu otomatayı $\sigma(P_q a)$ durumuna götürür.

Burada gözönüne alınacak iki durum vardır. İlk durumda, $a = P[q + 1]$ dir, bu yüzden, a karakteri desen eşlemeye devam eder; bu durumda, $\delta(q, a) = q + 1$ ise otomatının “kolu (spine)” boyunca ilerlemeye devam eder (Şekil 32.7 deki kalınlaştırılmış çizgiler). İkinci durumda, $a \neq P[q + 1]$ dir, böylece, a deseni eşlemeye devam etmez. Burada, T_i nin soneki olan P nin daha küçük önekinin bulmak zorundayız. Katar eşleme otomatası oluştururken ön işleme aşaması deseni kendisine karşı eşlediğinden, geçiş fonksiyonu P nin daha küçük en uzun önekinin hızlıca belirler.

Bir örneğe göz atalım. Şekil 32.7 deki katar otomatası ilk durumu örnekleyen $\delta(5, c) = 6$ ya sahiptir, eşleme devam eder. İkinci durumu örneklemek için, Şekil 32.7 de otomata $\delta(5, b) = 4$ e sahiptir. Eğer otomata $q = 5$ durumunda bir b yi okursa, $P_q b = ababab$ olduğundan ve P nin en uzun öneki aynı zamanda $aba-bab$ nin soneki, $P_4 = abab$ olduğundan, bu geçişi yaparız.



Şekil 32.8 Lemma 32.2 nin kanıtı için bir açıklama. Şekil $r = \sigma(xa)$ iken $r \leq \sigma(x) + 1$ i gösterir.

Katar eşleme otomatasının işlemini açıklığa kavuşturmak için, herhangi bir girdi metni $T[1..n]$ de uzunluğu m olan P deseninin görülmelerini bulan otomatanın (δ geçiş fonksiyonu ile gösterilir) benzeri basit, verimli bir program vereceğiz. m uzunluğundaki desen için herhangi katar eşleme otomatası konusunda, durumlar kümesi $\{0, 1, \dots, m\}$, başlangıç durumu, 0 ve kabul durumu m dir.

FINITE-AUTOMATON-MATCHER (T, δ, m)

```

1   $n = T.length$ 
2   $q = 0$ 
3  for  $i = 1$  to  $n$ 
4       $q = \delta(q, T[i])$ 
5      if  $q == m$ 
6          yazdır  $i-m$  “kaydırma ile desen bulunur”
```

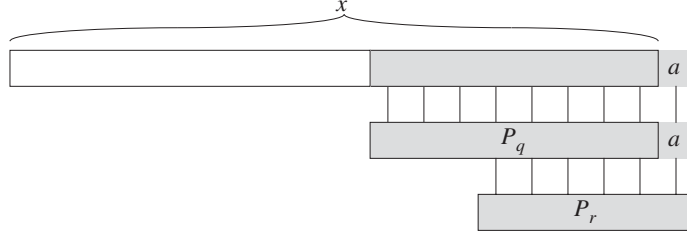
FINITE-AUTOMATON-MATCHER nin basit döngü yapısından, n uzunluğundaki metin dizisi üzerindeki onun eşlemesinin zamanının $\Theta(n)$ olduğunu kolayca görebiliriz. Bununla birlikte, bu eşleme zamanı δ geçiş fonksiyonunu hesaplamak için ihtiyaç duyulan ön işleme zamanını içermez. FINITE-AUTOMATON-MATCHER işleminin doğru çalıştığını gösterdikten sonra, bu problemi ele alacağız.

$T[1..n]$ girdi metni üzerinde otomatanın nasıl işlediğini göz önüne alalım. Otomata $T[i]$ karakterini taradıktan sonra $\sigma(T_i)$ durumunda olduğunu kanıtlayacağız. $\sigma(T_i) = m$ olması ancak ve ancak $P \sqsubset T_i$ ile mümkün olduğundan, P deseni yeni taranır ancak ve ancak bu makine m kabul durumundadır. Bu sonucu ispatlamak için, σ ek fonksiyonu hakkında izleyen iki lemmadan faydalanacağız.

Lemma 32.2 Sonek fonksiyonu eşitsizliği (Suffix-function inequality)

Herhangi x katarı ve a karakteri için $\sigma(xa) \leq \sigma(x) + 1$ vardır.

Kanıt Şekil 32.8 i baz alarak, $r = \sigma(xa)$ olsun. r ise $\sigma(xa) = r \leq \sigma(x) + 1$ durumu $\sigma(x)$ nin negatif olmaması ile açık bir şekilde sağlanır. Şimdi, $r > 0$ olduğunu varsayalım. Sonra, σ tanımından $P_r \sqsubset xa$ buluruz. Böylece, a yı P_r nin



Şekil 32.9 Lemma 32.3 ün kanıtının şekilsel gösterimi. Şekil, $q = \sigma(x)$ ve $r = \sigma(xa)$ iken $r = \sigma(P_q a)$ olduğunu gösterir.

sonundan ve xa nın sonundan silerek $P_{r-1} \sqsubset x$ elde edilir. Bundan dolayı, $r - 1 \leq \sigma(x)$ çünkü, $\sigma(x)$, $P_k \sqsubset x$ olan en büyük k dir ve bu nedenle, $\sigma(xa) = r \leq \sigma(x) + 1$ dir. ■

Lemma 32.3 (Ek fonksiyonu özyineleme (Suffix-function recursion) lemması)

Herhangi x katarı ve a karakteri için $q = \sigma(x)$ ise $\sigma(xa) = \sigma(P_q a)$ dir.

Kanıt σ nın tanımından, $P_q \sqsubset x$ vardır. Şekil 32.9'un gösterdiği gibi, $P_q a \sqsubset xa$ e sahibiz. Eğer $r = \sigma(xa)$ alırsak, $P_r \sqsubset xa$ ve Lemma 32.2 den $r \leq q + 1$ dir. Bu nedenle, $|P_r| = r \leq q + 1 = |P_q a|$ dir. $P_q a \sqsubset xa$, $P_r \sqsubset xa$ ve $|P_r| \leq |P_q a|$ olduğundan, Lemma 32.1 $P_r \sqsubset P_q a$ yı belirtir. Bundan dolayı, $r \leq \sigma(xa)$; şöyle ki; $\sigma(xa) \leq \sigma(P_q a)$ dir. Ancak, $P_q a \sqsubset xa$ olduğundan $\sigma(P_q a) \leq \sigma(xa)$ da sahibiz. Bu nedenle, $\sigma(xa) = \sigma(P_q a)$ dir. ■

Verilen bir girdi metni üzerinde katar eşleme otomatasının davranışını karakterize eden ana teoremimizin kanıtını vermeye şimdi hazırız. Yukarıda not edildiği gibi, bu teorem her aşamada, şimdiye kadar okunan desenin en uzun sonesinin aynı zamanda önekinin sadece kayıtlarını tutan otomatayı gösterir. Başka bir deyişle, bu otomata (32.5) değişmezini sürdürür.

Teorem 32.4

ϕ , verilen bir P deseni için katar otomatasının son durum fonksiyonu ve $T[1 \dots n]$ otomata için girdi metni ise,

$i = 0, 1, \dots, n$ için

$$\phi(T_i) = \sigma(T_i).$$

Kanıt i üzerinden tümevarım ile yapılır. $i = 0$ için teorem açıkça doğrudur, çünkü $T_0 = \varepsilon$ dur. Böylece, $\phi(T_0) = 0 = \sigma(T_0)$ dir.

Şimdi, $\phi(T_i) = \sigma(T_i)$ olduğunu varsayalım ve $\phi(T_{i+1}) = \sigma(T_{i+1})$ olduğunu kanıtlayacağız. $q, \phi(T_i)$ yi ve $a, T[i+1]$ i göstereyim. Sonra,

$$\begin{aligned}
 \phi(T_{i+1}) &= \phi(T_i a) && (T_{i+1} \text{ ve } a \text{ nın tanımlarından}) \\
 &= \delta(\phi(T_i), a) && (\phi \text{ nin tanımından}) \\
 &= \delta(q, a) && (q \text{ nın tanımından}) \\
 &= \sigma(P_q a) && (\delta \text{ nın (32.4) deki tanımından}) \\
 &= \sigma(T_i a) && (\text{Lemma 32.3 ve tümevarımdan}) \\
 &= \sigma(T_{i+1}) && (T_{i+1} \text{ in tanımından})
 \end{aligned}$$

■

Teorem 32.4 ile, eğer makine 4 nolu satırda q durumuna geçerse, q en büyük değerdir; öyle ki; $P_q \sqsupset T_i$. 5 nolu satırda $q = m$ ye sahip olmamız ancak ve ancak makinenin P deseninin bir görülüşünün henüz okuması ile mümkündür. FİNITE-AUTOMATON-MATCHER nin düzgün bir şekilde çalıştığı sonucuna varırız.

Geçiş fonksiyonunu hesaplama

Sıradaki işlem $P[1..m]$ deseninden δ geçiş fonksiyonunu hesaplamaktadır.

COMPUTE-TRANSITION-FUNCTION (P, Σ)

```

1   $m = P.length$ 
2  for  $q = 0$  to  $m$ 
3      for her  $a \in \Sigma$  için
4           $k = \min(m + 1, q + 2)$ 
5          repeat
6               $k = k - 1$ 
7          until  $P_k \sqsupset P_q a$ 
8           $\delta(q, a) = k$ 
9  return  $\delta$ 

```

Bu işlem, $\delta(q, a)$ yı, (32.4) deki denklemdeki tanımına göre kolay anlaşılır bir şekilde hesaplar. 2 ve 3 nolu satırlarda başlayan iç içe döngüler tüm q durumları ve tüm a karakterleri göz önünde bulundurur; ve 4-8 nolu satırlar, $\delta(q, a)$ yı $P_k \sqsupset P_q a$ olacak şekilde en büyük k olarak belirler. Kod, $\min(m, q + 1)$ olan akla uygun en büyük k değeri ile başlar. Sonra $P_k \sqsupset P_q a$ olana kadar k azaltılır, çünkü $P_0 = \varepsilon$ her katarının sonekidir.

COMPUTE-TRANSITION-FUNCTION nın çalışma zamanı $O(m^3 |\Sigma|)$ dir; çünkü, dış döngü $m |\Sigma|$ nın çarpanına katkıda bulunur, içteki **repeat** döngüsü en fazla $m + 1$ defa çalışabilir ve 7 nolu satırdaki $P_k \sqsupset P_q a$ testi karşılaştırılacak

m karaktere ihtiyaç duyabilir. Çok daha hızlı işlemler vardır; P deseni hakkında akıllıca hesaplanmış bazı bilgileri değerlendirerek (Alıştırma 32.4-8 e bakınız), δ yı hesaplamak için ihtiyaç duyulan süreyi P den $O(m |\Sigma|)$ ye iyileştirebiliriz. δ nın hesabı için bu iyileştirilmiş işlem ile, Σ alfabetini kullanan, n uzunluğundaki metinde m uzunluklu bir desenin tüm görülmelerini $O(m |\Sigma|)$ ön işleme süresinde ve $\Theta(n)$ eşleme süresinde bulabiliriz.

Alıştırmalar

32.3-1

$P = \text{aabab}$ deseni için katar eşleme otomatası kurunuz ve $T = \text{aaababaabaa-babaab}$ metin dizisi üzerinde onun işlemlerini ifade ediniz.

32.3-2

$\Sigma = \{a, b\}$ alfabeti üzerinde $\text{ababbabbababbababbabb}$ deseni için katar eşleme otomatasının durum geçiş şeklini geçiniz.

32.3-3

Eğer $P_k \sqsubset P_q$ $k = 0$ veya $k = q$ yu gerektiyorsa, P desenini **çakışmayan** olarak adlandırırız. Çakışmayan bir desen için katar eşleme otomatasının durum geçiş şeklini tanımlayınız.

32.3-4 *

Verilen iki P ve P' desenlerinden her iksinin de tüm görülmelerini bulan sonlu otomatanın nasıl kurulacağını tanımlayınız. Otomatanızdaki durumların sayısını en aza indirmeyi deneyiniz.

32.3-5

Boşluk karakterleri içere verilen bir P deseninde (Alıştırma 32.1-4 e bakınız), $n = |T|$ için T metnindeki P nin bir görülmesini $O(n)$ eşleme süresinde bulan bir otomatanın nasıl kurulacağını gösteriniz.

* 32.4 Knuth-Morris-Pratt Algoritması

Şimdi, Knuth, Morris ve Pratt'ın doğrusal zamanlı katar eşleme algoritmasını sunacağız. Bu algoritma δ geçiş fonksiyonunun bütünüyle hesaplamaktan kaçınır ve $\Theta(n)$ sürede desenden ön hesaplama yapan ve bunları $\pi[1..m]$ dizisinde depolayan π yardımcı fonksiyonunu kullanarak onun eşleme süresi $\Theta(m)$ dir. π dizisi δ geçiş fonksiyonunu ihtiyaç duyulduğu gibi verimli bir şekilde “anında” hesaplamamıza olanak sağlar. Üstünkörü bir deyişle, herhangi $q = 0, 1, \dots, m$

durumu ve herhangi $a \in \Sigma$ karakteri için $\pi[q]$ değeri a dan bağımsız hesaplanan $\delta(q, a)$ bilgisini içerir. π dizisi sadece m girdisine sahip ve δ nın $\Theta(m |\Sigma|)$ girdisi olduğundan, δ yı tercihen π yi hesaplayarak önışleme süresinde $|\Sigma|$ nın çarpanı kadar kurtarırız.

Bir desen için örnek fonksiyonu

π örnek fonksiyonu bir desen için kendisinin kaydırmalarına karşı eşlemeler hakkında bilgiyi içermektedir. Bu bilginin avantajını, sade desen eşleme algoritmasında gereksiz kaydırmaların testinden ve katar eşleme otomatasının δ tam geçiş fonksiyonu ön hesaplanmasından kaçınmak için kullanabiliriz.

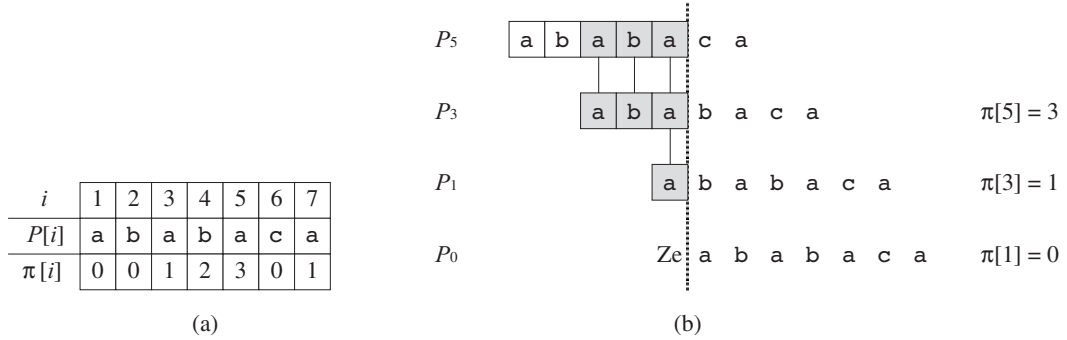
Sade katar eşleme işlemini düşünelim. Şekil 32.10(a) T metninde karşılaştırarak $P = ababaca$ desenini içeren şablonun belirli s kaydırmasını göstermektedir. Bu örnek için $q = 5$ de karakterler başarıyla eşleşmiştir, ancak 6. desen karakterine karşılık gelen metin karakteri eşlemesi başarısız olur. q tane karakterin başarı bir şekilde eşlendiği bilgisi karşılık gelen metin karakterlerini belirler. q tane metin karakterini bilmek bazı kaydırmaların geçersiz olduğunu hemen belirlememize olanak sağlar. Şekildeki örnekte, $s + 1$ kaydırması ister istemez geçersizdir; çünkü ilk desen karakteri (a), metin karakteri ile uyum sağlamaktadır, ancak bu metin karakteri ile eşleşmemektedir, ama ikinci desen karakteri (b) ile eşleşmektedir. Bununla birlikte, şeklin (b) kısmında gösterilen $s' = s + 1$ kaydırması desenin ilk üç karakterini metnin üç karakteri ile tam olarak eşleşmesi amacıyla karşılaştırır. Genel olarak, izleyen sorunun cevabını bilmek faydalı olmaktadır:

Verilen $P[1..q]$ desen karakterleri $T[s+1..s+q]$ metin karakterleri ile eşleşir, $s' + k = s + q$ iken bazı $k < q$ için en küçük kaydırma $s' > s$ nedir?

$$P[1..k] = T[s' + 1..s' + k] \quad (32.6)$$

Başka bir ifadeyle, $P_q \sqsupseteq T_{s+q}$ ü bilmek, aynı zamanda T_{s+q} nun soneki olan P_q nun P_k en uzun uygun önekini istiyoruz. ($s' + k = s + q$ olduğundan, s ve q bize verilirse, en küçük s' kaydırmasını bulmak ile k uzunluğundaki en uzun önekini bulmak aynıdır). P nin örneklerinin uzunluklarındaki $q - k$ farkını yeni s' kaydırmasına ulaşmak için s kaydırmasına ekleriz, böylece, $s' = s + (q - k)$ olur. En iyi durumda, $k = 0$ dır, böylece, $s' = s + q$ ve $s + 1, s + 2, \dots, s + q - 1$ kaydırmaları hemen geçersiz kılarız. Herhangi bir durumda, yeni s' kaydırması için, P nin ilk k karakterini T de karşılık gelen karakterler ile karşılaştırmamıza gerek duymayız; çünkü (32.6) nolu denklem onların eşleşmesini garantilemektedir.

Şekil 32.10(c) nin gösterdiği gibi deseni kendisi ile karşılaştırarak gerekli bilginin ön hesabını yapabiliriz. $T[s' + 1..s' + k]$ metnin bilinen parça-



Şekil 32.11 $P = ababaca$ deseni ve $q = 5$ için Lemma 32.5 in gösterimi. **(a)** Verilen bir desen için π fonksiyonu. $\pi[5]$, $\pi[3]$ ve $\pi[1]$ olduğundan, π yi tekrarlayarak $\pi^*[5] = \{3, 1, 0\}$ ı elde ederiz. **(b)** P desenini içeren şablonu sağa doğru kaydırırız ve P nin P_k öneki, P_5 in bazı uygun soneki ile eşleştiği zaman not ederiz; $k = 3, 1$ ve 0 iken eşleşmeleri elde ederiz. Şekilde, ilk satır P yi verir ve noktalı dikey çizgi P_5 ten hemen sonra çizilmiştir. Başarılı satırlar P nin bazı P_k öneklerinin P_5 in bazı sonekleri ile eşleşmesine neden olan P nin tüm kaydırmalarını gösterir. Başarıyla eşleşmiş karakterler gölgelendirilmiş olarak gösterilmiştir. Dikey çizgiler sıraya konulmuş eşleşmiş karakterleri bağlar. Bu nedenle, $\{k : k < 5 \text{ ve } P_k \sqsubset P_5\} = \{3, 1, 0\}$. Lemma 32.5 her q için $\pi^*[q] = \{k : k < q \text{ ve } P_k \sqsubset P_q\}$ olduğunu iddia eder.

fonksiyonunu verir. Aşağıdaki kabakod Knuth-Morris-Pratt eşleme algoritması KMP-MATCHER işlemi gibi verir. Birçok kısım için, göreceğimiz gibi işlem FINITE-AUTOMATON-MATCHER ni izler. KMP-MATCHER, COMPUTE-PREFIX-FUNCTION yardımcı işlemi π yi hesaplamak amacıyla çağırır.

KMP-MATCHER(T, P)

```

1   $n = T.length$ 
2   $m = P.length$ 
3   $\pi = \text{COMPUTE-PREFIX-FUNCTION}(P)$ 
4   $q = 0$  // eşlenen karakterlerin sayısı
5  for  $i = 1$  to  $n$  // metni soldan sağa tara
6      while  $q > 0$  ve  $P[q + 1] \neq T[i]$ 
7           $q = \pi[q]$  // eşleşmeyen sonraki karakter
8      if  $P[q + 1] == T[i]$ 
9           $q = q + 1$  // eşleşen sonraki karakter
10     if  $q == m$  //  $P$  nin hepsi eşleşti mi?
11         yazdır  $i - m$  “kaydırma ile desen görülür”
12      $q = \pi[q]$  // sonraki eşleşmenin aranması
```

COMPUTE-PREFIX-FUNCTION(P)

```

1   $m = P.length$ 
2   $\pi[1..m]$  yeni bir dizi olsun
3   $\pi[1] = 0$ 
4   $k = 0$ 
5  for  $q = 2$  to  $m$ 
6      while  $k > 0$  ve  $P[k + 1] \neq P[q]$ 
7           $k = \pi[k]$ 
8      if  $P[k + 1] == P[q]$ 
9           $k = k + 1$ 
10      $\pi[q] = k$ 
11 return  $\pi$ 

```

Bu iki işlemin çok ortak noktası vardır, çünkü her ikisi P desenine karşı katar eşlemektedir: KMP-MATCHER P ye karşı T metnini eşler ve COMPUTE-PREFIX-FUNCTION P yi kendisine karşı eşler.

Bu prosedürlerin çalışma zamanlarının analizine başlayacağız. Bu prosedürlerin doğru olduğunu kanıtlamak daha karmaşık olacaktır.

Çalışma zamanı analizi

COMPUTE-PREFIX-FUNCTION'ın çalışma zamanı amortize analizin toparlanma yöntemini kullanarak gösterdiğimiz $\Theta(m)$ dir (Kısım 17.1 e bakınız). Aldatıcı tek kısım 6-7 nolu satırlardaki **while** döngüsünün hep beraber $O(m)$ sürede yerine getirilmesidir. Onun en fazla $m - 1$ iterasyon yaptığını göstereceğiz. k hakkında bazı gözlemler yaparak başlayacağız. İlk olarak, 4 nolu satırda k , 0 da başlar ve k yi artırmanın tek yolu, 9 nolu satırdaki artırma işlemidir, bu işlem 5-10 satırlardaki **for** döngüsünün her iterasyonunda en fazla bir defa çalıştırılır. Bu nedenle, k deki toplam artış en fazla $m - 1$ dir. İkinci olarak, $k < q$ olduğunda **for** döngüsüne girdikten sonra ve döngüdeki her iterasyon q yu artırdığından dolayı, her zaman $k < q$ vardır. Bundan dolayı, 3 ve 10 nolu satırlardaki atamalar her $q = 1, 2, \dots, m$ için $\pi[q] < q$ sağlanır; bu şu demektir ki, **while** döngüsünün her bir iterasyonu k yi azaltır. Üçüncü olarak, k hiçbir zaman negatif olamaz. Bu gerçeklerin hepsini biraraya koyduğumuzda, **while** döngüsünden k nin toplam azalışının, **for** döngüsündeki tüm iterasyonlarda k deki toplam artışın $m - 1$ ile üst sınırlandırıldığını görürüz. bu nedenle, **while** döngüsü toplamda en fazla $m - 1$ defa tekrar eder ve ÖNEK-FONKSİYONU-HESAPLAMA $\Theta(m)$ sürede çalışır.

Alıştırma 32.4-4 toparlama analizinin benzeri ile KMP-MATCHER nin eşleme süresinin $\Theta(n)$ olduğunu göstermenizi istemektedir.

FINITE-AUTOMATON-MATCHER ile karşılaştırıldığında, δ yerine π kullanılması ile desenin ön işleme süresini, $\Theta(n)$ ile sınırlandırılmış gerçek eşleme süresini tutarak $O(m |\Sigma|)$ den $\Theta(m)$ e indiririz.

Önek fonksiyonunun hesaplamının doğruluğu

π önek fonksiyonu katar eşleme otomatasında δ geçiş fonksiyonunun benzerini yapmaya yardım ettiğini az sonra göreceğiz. Ancak, öncelikle, COMPUTE-PREFIX-FUNCTION'ın gerçekten önek fonksiyonunu doğru bir şekilde hesapladığını ispatlamamız gerekiyor. Bunu yapabilmek için, verilen P_q önekinin uygun sonekleri olan P_k öneklerinin hepsini bulmak durumundayız. $\pi[q]$ nun değeri öyle en uzun öneki verir, ancak Şekil 32.11 de resmedilen sıradaki lemma π önek fonksiyonunu tekrarlayarak anlatmaktadır, P_q nun uygun sonekleri olan tüm P_k öneklerini gerçekten numaralandırabiliriz.

$$\pi^*[q] = \{\pi[q], \pi^{(2)}[q], \pi^{(3)}[q], \dots, \pi^{(i)}[q]\}$$

olsun, burada $\pi^{(i)}[q]$ fonksiyonel tekrar ile tanımlanmıştır, böylece $\pi^{(0)}[q] = q$ ve $i \geq 1$ için $\pi^{(i)}[q] = \pi[\pi^{(i-1)}[q]]$ dir ve $\pi^*[q]$ daki dizi $\pi^{(i)}[q] = 0$ değerinde ulaştığı zaman sona ulaşır.

Lemma 32.5 (Önek fonksiyonu iterasyon (Prefix function iteration) lemması)

P, m uzunluğunda π önek fonksiyonuna sahip olan bir desen olsun. $q = 1, 2, \dots, m$ için $\pi^*[q] = \{k : k < q \text{ ve } P_k \sqsupset P_q\}$ vardır.

Kanıt Öncelikle, $\pi^*[q] \subseteq \{k : k < q \text{ ve } P_k \sqsupset P_q \text{ veya ona denk olan}$

$$i \in \pi^*[q] \text{ yu ima eden } P_i \sqsupset P_q \text{ ü} \quad (32.7)$$

kanıtlarız.

Eğer $i \in \pi^*[q]$ varsa, bazı $u > 0$ için $i = \pi^{(u)}[q]$. u üzerinden tümevarım ile (32.7) nolu denklemi kanıtlarız. $u = 1$ için $i = \pi[q]$ yu sahibiz ve $i < q$ ve $P_{\pi[q]} \sqsupset P_q$ olduğundan iddia π nin tanımını izler. $\pi[i] < i$ ve $P_{\pi[i]} \sqsupset P_i$ bağlantılarını kullanarak $<$ ve $\sqsupset$ geçişi $\pi^*[q]$ daki her i için iddiayı kurar. Bundan dolayı, $\pi^*[q] \subseteq \{k : k < q \text{ ve } P_k \sqsupset P_q\}$.

Şimdi, $\{k : k < q \text{ ve } P_k \sqsupset P_q\} \subseteq \pi^*[q]$ ü olmayana ergi ile kanıtlarız. Tersini yani $\{k : k < q \text{ ve } P_k \sqsupset P_q\} - \pi^*[q]$ kümesinin boş olmadığı farzedelim ve j kümedeki en büyük sayı olsun. $\pi[q] \{k : k < q \text{ ve } P_k \sqsupset P_q\}$ de en büyük değer ve $\pi[q] \in \pi^*[q]$ olduğundan, $j < \pi[q]$ vardır ve böylece, j den daha büyük j' , $\pi^*[q]$ daki en küçük tamsayıyı gösterebilir. (eğer $\pi^*[q]$ da j den daha büyük olan bir sayı yoksa, $j' = \pi[q]$ olarak seçebiliriz). $P_{j'} \sqsupset P_q$ vardır; çünkü $j \in \{k : k < q \text{ ve } P_k \sqsupset P_q\}$ ve (32.7) nolu denklemde, $j' \in \pi^*[q]$ e sahip oluruz. Bu nedenle, Lemma 32.1 ile $P_j \sqsupset P_{j'}$ ve bu özellik ile j, j' den küçük olan en büyük değerdir. Bundan dolayı, $\pi[j'] = j$ ye sahip oluruz ve $j' \in \pi^*[q]$ olduğundan, aynı zamanda $j' \in \pi^*[q]$ e sahip oluruz. Bu çelişki lemmayı kanıtlar. ■

COMPUTE-PREFIX-FUNCTION algoritması $\pi[q]$ yu $q = 1, 2, \dots, m$ için sırayla hesaplar. COMPUTE-PREFIX-FUNCTION'ın 3 nolu satırında $\pi[1]$ in 0 a ayarlanması muhakkak ki doğrudur; çünkü, her q için $\pi[q] < q$ dur. Sıradaki lemmayı

ve onun sonucunu ÖNEK-COMPUTE-PREFIX-FUNCTION nun $q > 1$ için $\pi[q]$ yu hesapladığını ispatlamak amacıyla kullanacağız.

Lemma 32.6

P, m uzunluğunda bir desen olsun ve π, P için önek fonksiyonu olsun. $q=1,2,\dots,m$ için eğer $\pi[q] > 0$ ise, $\pi[q] - 1 \in \pi^*[q-1]$.

Kanıt $r = \pi[q] > 0$ olsun, böylece $r < q$ ve $P_r \sqsupset P_q$; bu nedenle, $r-1 < q-1$ ve $P_{r-1} \sqsupset P_{q-1}$ (P_r ve P_q dan son karakteri silerek, ki bunu $r > 0$ olduğundan yapabiliriz). Lemma 32.5 ten $r-1 \in \pi^*[q-1]$. Böylece, $\pi[q] - 1 = r-1 \in \pi^*[q-1]$ e sahibiz. ■

$q=2,3,\dots,m$ için $E_{q-1} \subseteq \pi^*[q-1]$ alt kümesi aşağıdaki ile sağlanır.

$$\begin{aligned} E_{q-1} &= \{k \in \pi^*[q-1] : P[k+1] = P[q]\} \\ &= \{k : k < q-1 \text{ ve } P_k \sqsupset P_{q-1} \text{ ve } P[k+1] = P[q]\} \text{ (Lemma 32.5 ile)} \\ &= \{k : k < q-1 \text{ ve } P_{k+1} \sqsupset P_q\} . \end{aligned}$$

E_{q-1} kümesi $k < q-1$ değerlerini $P_k \sqsupset P_{q-1}$ için içerir ve $P[k+1] = P[q]$ olduğundan, $P_{k+1} \sqsupset P_q$ vardır. Böylece, E_{q-1} $k \in \pi^*[q-1]$ değerlerini içerir; öyle ki; P_k yi P_{k+1} e genişletebiliriz ve P_q nun uygun sonekini elde ederiz.

Sonuç 32.7

P, m uzunluğunda bir desen olsun ve π, P için önek fonksiyonu olsun. $q = 2,3,\dots,m$ için

$$\pi[q] = \begin{cases} 0, & E_{q-1} = \emptyset \text{ ise,} \\ 1 + \max \{k \in E_{q-1}\}, & E_{q-1} \neq \emptyset \text{ ise.} \end{cases}$$

Kanıt E_{q-1} boş ise, P_k yi P_{k+1} e genişletebiliriz ve P_q nun uygun sonekini elde edebileceğimiz öyle bir eleman $k \in \pi^*[q-1]$ yoktur ($k=0$ ı içeren). Bundan dolayı, $\pi[q] = 0$.

E_{q-1} boş değil ise, her $k \in E_{q-1}$ için $k+1 < q$ ve $P_{k+1} \sqsupset P_q$ vardır. Bundan dolayı, $\pi[q]$ nun tanımından,

$$\pi[q] \geq 1 + \max \{k \in E_{q-1}\} . \quad (32.8)$$

Dikkat edilmelidir ki; $\pi[q] > 0$ dır. $r = \pi[q] - 1$ olsun, böylece, $r+1 = \pi[q]$ ve $P_{r+1} \sqsupset P_q$. $r+1 > 0$ olduğundan, $P[r+1] = P[q]$ vardır. Bunlara ek olarak, Lemma 32.6 dan $r \in \pi^*[q-1]$ vardır. Bundan dolayı, $r \in E_{q-1}$ ve böylece $\max \{k \in E_{q-1}\}$ veya denk olarak

$$\pi[q] \leq 1 + \max \{k \in E_{q-1}\} \quad (32.9)$$

(32.8) ve (32.9) nolu denklemleri birleştirdiğimizde kanıt tamamlanır. ■

Şimdi, COMPUTE-PREFIX-FUNCTION un π yi doğru bir şekilde hesapladığının kanıtını bitireceğiz. COMPUTE-PREFIX-FUNCTION prosedüründe, 5-10 nolu satırlardaki **for** döngüsünün her bir iterasyonunun başlamasında, $k = \pi[q - 1]$ e sahibiz. Döngüye ilk kez girildiği zaman ve 10 satırdan ötürü her başarılı tekrarda doğru kaldığı zaman, bu koşul 3 ve 4 nolu satırla güçlendirilir. 6-9 nolu satırlar k yi düzeltir, böylece $\pi[q]$ nun doğru değeri olur. 6-7 nolu satırlardaki **while** döngüsü $P[k + 1] = P[q]$ olan k değerini bulana kadar, tüm $k \in \pi^*[q - 1]$ değerlerini arar.; bu noktada, k, E_{q-1} kümesindeki en büyük değerdir; böylece, Corollary 32.7 ile, $\pi[q]$ yu $k + 1$ e ayarlarız. Eğer **while** döngüsü $P[k + 1] = P[q]$ olan bir $k \in \pi^*[q - 1]$ bulamıyorsa, 8 nolu satırda $k, 0$ a eşittir. Eğer $P[1] = P[q]$ ise k ve $\pi[q]$ nuun ikisini birden 1 e ayarlamalıyız; aksi halde, k yi yalnız bırakmalı ve $\pi[q]$ yu 0 a ayarlamalıyız. 8-10 nolu satırlar k ve $\pi[q]$ yu herhangi bir durumda doğru ayarlar. Bu COMPUTE-PREFIX-FUNCTION un doğruluğu ispatımızı tamamlar.

Knuth-Morris-Pratt algoritmasının doğruluğu

KMP-MATCHER prosedürünü FINITE-AUTOMATON-MATCHER prosedürünün tekrar uygulanmış hali olarak düşünebiliriz, ancak geçiş durumlarının hesaplamak için π önek fonksiyonunu kullanırız. Belli bir amaç için, KMP-MATCHER ve FINITE-AUTOMATON-MATCHER nin her ikisinin **for** döngülerinin i nci iterasyonlarında m ile eşitliğini kontrol ettiğimiz zaman, q durumu aynı değere sahip olur (KMP-MATCHER 10 nolu satırda ve SONLU-OTOMATA-EŞLEMESİ 5 nolu satırda). KMP-MATCHER, FINITE-AUTOMATON-MATCHER in hareketini benzediğini savunduğumuz zaman, KMP-MATCHER nin doğruluğu FINITE-AUTOMATON-MATCHER nin doğruluğunu takip eder (yine de KMP-MATCHER i nde 12 nolu satırın gerekli olduğunu az sonra göreceğiz).

KMP-MATCHER nin gerektiği gibi FINITE-AUTOMATON-MATCHER ne benzediğini şekilsel olarak ispatlamadan önce, π önek fonksiyonu, δ geçiş fonksiyonunu nasıl yenilediğine bir göz atalım. Hatırlatmak gerekir ki; katar eşleme otomatası q durumunda olduğu ve $a = T[i]$ karakterini taradığı zaman, yeni $\delta(q, a)$ durumuna geçer. Eğer $a = P[q + 1]$ ise; böylece a , deseni eşlemeye devam eder, sonra $\delta(q, a) = q + 1$ olur. Aksi halde, $a \neq P[q + 1]$ dir; böylece, a , deseni eşlemeye devam etmez ve $0 \leq \delta(q, a) \leq q$ dur. İlk durumda, a , eşlemeye devam ettiğinde, KMP-MATCHER durumu π ye başvurmadan $q + 1$ e taşır: 6 nolu satırdaki **while** döngüsü testi ilk seferde yanlış olarak gelir, 8 nolu satırdaki test doğru olur ve 9 nolu satırda q artırılır.

π fonksiyonu, a karakteri deseni eşlemeye devam etmediği zaman devreye girer, böylece yeni $\delta(q, a)$ durumu q veya otomatanın omurgası boyunca q nun solundadır. KMP-MATCHER nin 6-7 nolu satırlardaki **while** döngüsü $\pi^*[q]$ daki durumlar boyunca q' durumuna geldiği zaman duracak şekilde tekrar eder; öyle

ki, a , $P[q' + 1]$ i eşler veya q' işlemin sonuna kadar 0 a iner. Eğer a , $P[q' + 1]$ i eşlerse, 9 nolu satır yeni durumu $q' + 1$ e ayarlar, bu durum $\delta(q, a)$ benzetim için doğru çalışmalıdır. Başka bir deyişle, yeni durum $\delta(q, a)$ ya 0 ya da $\pi^*[q]$ daki bazı durumlardan büyük olmalıdır.

Şekil 32.7 ve 32.11 de desenin $P = \text{ababaca}$ olduğu örneğe göz atalım. Otomatanın $q = 5$ durumunda olduğunu farzedelim; $\pi^*[q]$ daki durumlar azalan sırayla 3, 1 ve 0 dır. Bir sonraki taranan karakter c ise, her iki FINITE-AUTOMATON-MATCHER ve KMP-MATCHER nde otomatanın $\delta(5, c) = 6$ durumuna geçtiğini kolayca görebiliriz. Şimdi, b yerine sonraki karakterin tarandığını farzedelim; böylece, otomata $\delta(5, b) = 4$ durumuna taşınmalıdır. MP-MATCHER ndeki **while** döngüsü 7 nolu satır bir kere çalıştırıldığında çıkar ve $q' = \pi[5] = 3$ durumuna varır. $P[q' + 1] = P[4] = b$ olduğundan, 8 nolu satırdaki test doğru sonucuna varır ve MP-MATCHER $q' + 1 = 4 = \delta(5, b)$ yeni durumuna taşınır. Son olarak, a yerine sonraki karakterin tarandığını farzedelim; böylece, otomata $\delta(5, a) = 1$ durumuna taşınmalıdır. 6 nolu satırdaki testin ilk üç çalıştırılmasında test doğru cevabını verir. İlk seferde, $P[6] = c \neq a$ yı buluruz ve MP-MATCHER $\pi[5] = 3$ durumuna taşır. ($\pi^*[5]$ daki ilk durum). İkinci seferde, $P[4] = b \neq a$ bulunur ve $\pi[3] = 1$ durumuna taşır ($\pi^*[5]$ daki ikinci durum). Üçüncü seferde, $P[2] = b \neq a$ bulunur ve $\pi[1] = 0$ durumuna taşır ($\pi^*[5]$ daki son durum). **while** döngüsünden $q' = 0$ durumuna ulaşıldığı zaman çıkılır. Şimdi, 8 nolu satır $P[q' + 1] = P[1] = a$ yı bulur ve 9 nolu satır otomatayı yeni $q' + 1 = 1 = \delta(5, a)$ durumuna taşır.

Böylece, bizim sezgimiz MP-MATCHER, $\pi^*[q]$ daki durumların azalan bir şekilde bazı q' durumunda durarak ve durumu $q' + 1$ e taşıyarak tekrar eder. $\delta(q, a)$ hesaplamasının benzetimi için asimptotik olarak göz önüne aldığımızda çok fazla iş yapılacak olması görülmesine rağmen, MP-MATCHER, FINITE-AUTOMATON-MATCHER nden daha yavaş değildir.

Şimdi, Knuth-Morris-Pratt algoritmasının doğruluğunu şekilsel olarak ispatlamaya hazırız. Teorem 32.4 ile, her seferde FINITE-AUTOMATON-MATCHER nin 4 nolu satırını çalıştırdıktan sonra, $q = \sigma(T_i)$ ye sahip oluruz. Bundan dolayı, MP-MATCHER ndeki **for** döngüsü bakımından aynı özelliklere sahip olduklarını göstermek yeterli olacaktır. İspat döngü iterasyonlarının sayısı üzerinden tümevarım ile yapılır. Başlangıçta, ilk sefer için ilgili **for** döngülerine girildiğinden, her iki işlem q yu 0 a ayarlar. MP-MATCHER ndeki **for** döngüsünün i iterasyonlarını göz önüne alalım ve bu döngü iterasyonunun başlangıçtaki durumu q' olsun. Tümevarımsal hipotez ile, $q' = \sigma(T_{i-1})$ ye sahip oluruz. 10 nolu satırda $q = \sigma(T_i)$ yi göstermemiz gerekir (yine, 12 nolu satırı ayrı olara ele alacağız).

$T[i]$ karakterini göz önüne aldığımız zaman, P nin en uzun öneki aynı zamanda T_i nin soneki ya $P_{q'+1}$ ($P[q' + 1] = T[i]$ ise) ya da P'_q nın bazı önkidir (düzgün olmasına gerek olmamakla beraber, büyük ihtimalle boş kümedir). Üç durumu ayrı bir şekilde değerlendireceğiz: $\sigma(T_i) = 0$, $\sigma(T_i) = q' + 1$ ve $0 < \sigma(T_i) \leq q'$.

- $\sigma(T_i) = 0$ ise, $P_0 = \varepsilon$, T_i nin soneki olan P nin önekidir. 6-7 nolu satırlardaki **while** döngüsü $\pi^*[q']$ daki değerler boyunca tekrar eder, ancak her $P_q \sqsubset T_{i-1}$ için $q \in \pi^*[q']$ olmasına rağmen, döngü $P[q + 1] = T[i]$ eşitliğini sağlayan öyle bir q bulamaz. Döngü q , 0 a ulaştığında biter ve doğal olarak 9 nolu satır çalıştırılmaz. Bundan dolayı, 10 nolu satırda $q = 0$ olur; böylece, $q = \sigma(T_i)$.
- $\sigma(T_i) = q' + 1$ ise, $P[q' + 1] = T[i]$ ve 6 nolu satırdaki **while** döngüsü testi ilk deneme boyunca başarısız olur. 9 nolu satır çalıştırıldığında q artırılır; böylece, daha sonra $q = q' + 1 = \sigma(T_i)$ olur.
- $0 < \sigma(T_i) \leq q'$ ise, 6-7 nolu satırlardaki **while** döngüsü en az bir kere tekrar eder, $q \in \pi^*[q']$ deki her bir değer azalan sırasındaki kontrol, bazı $q < q'$ da durana kadar devam eder. Böylece, $P_{q'}$, $P[q + 1] = T[i]$ için $P_{q'}$ ın en uzun önekidir; böylece, **while** döngüsü sona erdiğinde, $q + 1 = \sigma(P_{q'}T[i])$. $q' = \sigma(T_{i-1})$ olduğundan, Lemma 32.3 $\sigma(T_{i-1}T[i]) = \sigma(P_{q'}T[i])$ yi gerektirir. Bu nedenle, **while** döngüsü sonlandığında,

$$\begin{aligned} q + 1 &= \sigma(P_{q'}T[i]) \\ &= \sigma(T_{i-1}T[i]) \\ &= \sigma(T_i) \end{aligned}$$

elde ederiz. 9 nolu satırda q nun artırılmasından sonra, $q = \sigma(T_i)$ elde ederiz.

12 nolu satır KMP-MATCHER nde gereklidir; çünkü aksi durum olsaydı, P nin bir görülmesini bulduktan sonra, 6 nolu satırdaki $P[m + 1]$ i ilişkilendirebiliriz. ($q = \sigma(T_{i-1})$ verisi 6 nolu satırın sonraki çalıştırılmasına kadar Alıştırma 32.4-8 de verilen ipucu ile: $\delta(m, a) = \delta(\pi[m], a)$ geçerli kalır veya denk olarak herhangi $a \in \Sigma$ için $\sigma(Pa) = \sigma(P_{\pi[m]}a)$). KMP-MATCHER, FINITE-AUTOMATON-MATCHER nin işleyişini taklit ettiğini gösterdiğimizden ötürü Knuth-Morris-Pratt algoritması için geriye kalan tartışma FINITE-AUTOMATON-MATCHER nin doğruluğundan çıkmaktadır.

Alıştırmalar

32.4-1

ababbabbabbababbabbb deseni için π önek fonksiyonunu hesaplayınız.

32.4-2

q nun fonksiyonu gibi $\pi^*[q]$ nun boyutu için bir üst sınır veriniz. Bulduğunuz bu sınırın sıkı olduğunu gösteren bir örnek veriniz.

32.4-3

P T katarı için π fonksiyonunu test ederek T metnindeki P deseninin görülme sıklığı nasıl karar verildiğini açıklayınız ($m + n$ uzunluğundaki deseninin P ve T nin ardarda eklenmesidir).

32.4-4

KMP-MATCHER nin çalışma zamanının $\Theta(n)$ olduğunu göstermek için toparlama analizini kullanınız.

32.4-5

KMP-MATCHERİ nin çalışma zamanının $\Theta(n)$ olduğunu göstermek için olası bir fonksiyon kullanınız.

32.4-6

7 nolu satırdaki π nin görülmesinin (12 nolu satırdaki değil) π' ile yer değiştirilerek KMP-MATCHERİ nin iyileştirildiğini gösteriniz; burada, $\pi', q = 1, 2, \dots, m$ için aşağıdaki denklem ile yinelemeli olarak tanımlanmıştır.

$$\pi'[q] = \begin{cases} 0, & \pi[q] = 0 \text{ ise,} \\ \pi'[\pi[q]], & \pi[q] \neq 0 \text{ ve } P[\pi[q] + \\ \pi[q], & \pi[q] \neq 0 \text{ ve } P[\pi[q] + \end{cases}$$

Düzeltilen bu algoritmanın neden doğru olduğunu açıklayınız ve hangi anlamda bu değişikliğin iyileştirme anlamına geldiğini gösteriniz.

32.4-7

T metninin başka bir katarı T' nin devirli döndürmesi olup olmadığına karar veren doğrusal zamanlı algoritma veriniz. Örneğin, `arc` ve `car`, birbirinin devirli çevrimidir.

32.4-8 *

Verilen bir P desenine karşılık gelen katar eşleme otomatası için δ geçiş fonksiyonunu hesaplayan $O(m |\Sigma|)$ zamanlı bir algoritma veriniz. (*İpucu:* $q = m$ veya $P[q + 1] \neq a$ ise $\delta(q, a) = \delta(\pi[q], a)$ olduğunu kanıtlayınız).

Problemler**32-1 Tekrarlama çarpanlarına dayanan katar eşleme**

y^i , y katarının kendisi ile i kere ardarda eklenmesini gösterebilir. Örneğin, $(ab)^3 = ababab$. Eğer bazı $x \in \Sigma^*$ ve bazı $r > 0$ için $x = y^r$ ise, $y \in \Sigma^*$ katarı tekrarlama çarpanına sahiptir deriz. $\rho(x)$ en büyük r yi gösterebilir; öyle ki, x, r **tekrarlama çarpanına** sahip olsun.

a. $P[1..m]$ deseni girdi olarak alan ve $i = 1, 2, \dots, m$ için $\rho(P_i)$ değerini hesaplayan verimli bir algoritma veriniz. Algoritmanızın çalışma zamanı nedir?

- b.** Herhangi $P[1..m]$ deseni için $\rho^*(P) = \max_{1 \leq i \leq m} \rho(P_i)$ olarak tanımlansın. Eğer P deseni m uzunluklu ikilik tabandaki tüm dizilerden oluşan bir kümeden seçilirse, $\rho^*(P)$ nin beklenen değeri $O(1)$ dir.
- c.** Aşağıdaki katar eşleme algoritmasının $T[1..n]$ metnindeki P deseninin tüm görülmelerini $O(\rho^*(P)n + m)$ sürede bulduğunu iddia ediniz ve gösteriniz:

REPETITION-MATCHER(P, T)

```

1   $m = P.length$ 
2   $n = T.length$ 
3   $k = 1 + \rho^*(P)$ 
4   $q = 0$ 
5   $s = 0$ 
6  while  $s \leq n - m$ 
7      if  $T[s + q + 1] == P[q + 1]$ 
8           $q = q + 1$ 
9          if  $q == m$ 
10              yazdır  $s$  “kaydırma ile desen görülür”
11      if  $q == m$  veya  $T[s + q + 1] \neq P[q + 1]$ 
12           $s = s + \max(1, \lceil q/k \rceil)$ 
13       $q = 0$ 
```

Bu algoritma Galil ve Seiferas’a dayanmaktadır. Bu fikirleri güzel genişlettikten sonra, P ve T için ihtiyaç duyulanın haricinde sadece $O(1)$ alan kullanan doğrusal zamanlı katar eşleme algoritmasını elde etmişlerdir.

Bölüm notları

Katar eşlemenin sonlu otomata teorisi ile olan ilişkisi Aho, Hopcroft ve Ullman [5] tarafından tartışılmıştır. Knuth-Morris-Pratt algoritması [214] birbirlerinden bağımsız olarak Knuth ve Pratt ve Morris tarafından bulunmuştur; kendi çalışmalarını ortak olarak yayınlamışlardır. Reingold, Urban ve Gries [294], Knuth-Morris-Pratt algoritmasının farklı değerlendirmesini vermiştir. Rabin-Karp algoritması, Karp ve Rabin [201] tarafından önerilmiştir. Galil ve Seiferas [126] ilginç bir şekilde desen ve metni depolamak için sadece $O(1)$ alana ihtiyaç duyan kararlı doğrusal zamanlı Katar eşleme algoritması vermiştir.

Hesaplanabilir geometri geometrik problemleri çözen algoritmalar üzerinde çalışan bilgisayar biliminin bir dalıdır. Modern mühendislik ve matematikte, hesaplanabilir geometrinin; bilgisayar grafikleri, robot bilim, VLSI dizaynı, bilgisayar destekli dizayn, moleküler modelleme, metalürji, üretim, tekstil dizaynı, ormancılık ve istatistik gibi farklı alanlarda uygulamaları vardır. Hesaplanabilir geometriye ait bir problemin girdisi genellikle; noktalar kümesi, doğru parçaları kümesi ya da bir çokgenin köşelerinin saat yönünün tersinde sıralı bir kümesi gibi geometrik cisimlerin kümesi olarak verilir. Çıktısı ise çoğu kez, doğruların kesişip kesişmediğini ya da belki de, noktalar kümesinin dışbükey örtüsü (convex hull) (en küçük dışbükey çokgen örtüsü) gibi yeni bir geometrik cismin olup olmadığı gibi geometrik cisimlerle ilgili sorulara yanıttır.

Bu bölümde iki boyutlu uzayda yani düzlemde hesaplanabilir geometriye ait bazı algoritmalara bakacağız. Her cismin girdisini $p_i = (x_i, y_i)$ ve $x_i, y_i \in \mathbb{R}$ nin olduğu $\{p_1, p_2, p_3, \dots\}$ noktalar kümesiyle göstereceğiz. Örneğin, n köşeli bir P çokgeni, köşelerinin P sınırında görünme sırasına göre $\langle p_0, p_1, p_2, \dots, p_{n-1} \rangle$ dizisiyle ile göstereceğiz. Hesaplanabilir geometri aynı zamanda üç boyutlu hatta daha yüksek boyutlu uzaya da uygulanabilir fakat bu tür problemleri ve çözümlerini canlandırmak çok zordur. Zaten iki boyutlu uzayda hesaplanabilir geometri tekniklerine ait iyi örnekler görebileceğiz.

Kısım 33.1 doğru parçalarıyla ilgili sorulara nasıl tam ve doğru cevap vereceğimizi inceliyor. Bu sorular örneğin: bir doğru parçasının ortak uç noktasının, paylaştığı diğer doğru parçasından saat yönünde mi saat yönünün tersinde mi olduğu, bitişik iki doğru parçasını çaprazlarken hangi yola döneceğimizi yada iki doğru parçasının kesişip kesişmediğidir. Kısım 33. 2 ise n doğru parçalı bir kümenin herhangi bir kesişim içerip içermediğini $O(n \lg n)$ zamanda tespit eden “sweeping” (tarama) isminde bir yöntemi inceliyor. Kısım 33.3, n noktalı bir kümenin dışbükey örtüsünü hesaplayan iki “rotational sweep” ten (dönen tarama) algoritmasından bahsediyor. Bunlar $O(n \lg n)$ zaman da çalışan Graham’s Scan (Graham taraması) algoritması ve dışbükey örtünün köşe sayısı h olduğun-

da $O(nh)$ zamanda çalışan Jarvis's March'ıdır (Jarvis Marşı). Son olarak 33.4 ise düzlemde n noktalı bir küme içerisinde en yakın iki nokta çiftini bulan $O(n \lg n)$ zamanlı zamanlı divide-and-conquer (böl-ve-yönet) algoritmasını anlatıyor.

33.1 Doğru parçasının özellikleri

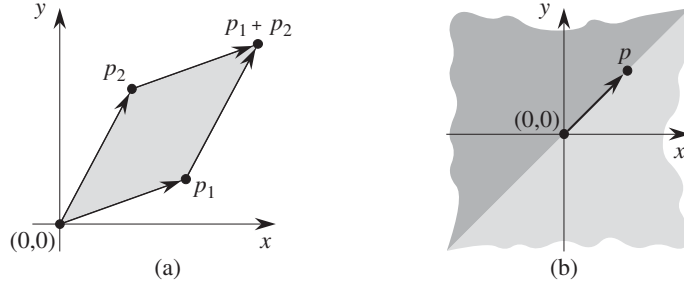
Bu bölümde bazı hesaplanabilir geometri algoritmaları, doğru parçasının özellikleri ile alakalı sorulara cevap verecek. İki farklı $p_1(x_1, y_1)$ ve $p_2(x_2, y_2)$ noktasının **dışbükey birleşimi**, bazı $0 \leq \alpha \leq 1$ değerleri için $x_3 = \alpha x_1 + (1 - \alpha)x_2$ ve $y_3 = \alpha y_1 + (1 - \alpha)y_2$ olduğundaki herhangi bir $p_3(x_3, y_3)$ noktasıdır. Ayrıca $p_3 = \alpha p_1 + (1 - \alpha)p_2$ de yazabiliriz. Sezgisel olarak, p_3 ün p_1 ve p_2 noktalarından geçen doğru üzerinde herhangi bir nokta olduğunu ve doğru üzerinde p_1 ve p_2 nin üzerinde ya da arasında olduğunu söyleyebiliriz. p_1 ve p_2 farklı 2 nokta verilsin, bu durumda $\overline{p_1 p_2}$ **doğru parçası** p_1 ve p_2 nin dışbükey birleşimlerinin kümesidir. p_1 ve p_2 , $\overline{p_1 p_2}$ doğru parçasının **uç noktalarıdır**. Bazen p_1 ve p_2 nin sıralaması önem arz eder bu durumda $\overrightarrow{p_1 p_2}$ **yönlü doğru parçası** deriz. p_1 noktası **orijin** (0, 0) ise bu durumda $\overrightarrow{p_1 p_2}$ doğru parçasını p_2 **vektörü** olarak alabiliriz.

Bu bölümde aşağıdaki özellikleri açıklayacağız:

1. Farklı iki $\overrightarrow{p_0 p_1}$ ve $\overrightarrow{p_0 p_2}$ yönlü doğru parçaları verilsin. Bu durumda $\overrightarrow{p_0 p_1}$, ortak uç noktası p_0 a göre $\overrightarrow{p_0 p_2}$ nin saat yönünde midir?
2. Farklı iki $\overline{p_0 p_1}$ ve $\overline{p_1 p_2}$ doğru parçaları verilsin. İlk önce $\overline{p_0 p_1}$ nı daha sonra $\overline{p_1 p_2}$ nı çaprazlarsak p_1 noktasında sola dönüş yapmış mı oluruz?
3. $\overline{p_1 p_2}$ ve $\overline{p_3 p_4}$ doğru parçaları kesişir mi?

Verilen noktalarda herhangi bir kısıtlama yoktur.

Biz her sorunun cevabını $O(1)$ zamanda bulabiliriz. Her sorunun girdi büyüklüğü $O(1)$ olduğu için bu cevap bizim için sürpriz değildir. Üstelik yöntemlerimiz sadece toplama, çıkarma, çarpma ve kıyaslamayı kullanır. Bizim ne bölmeye ne de trigonometrik fonksiyonlara ihtiyacımız var. Bunların her ikisi hesaplama olarak pahalıdır ve yuvarlama hataları vermeye yatkındır. Örneğin iki doğru parçasının kesişip kesişmediğini belirleyen “straightforward” (açıksözlü) yöntemi—her doğru parçası için $y = mx + b$ (m eğim, b de y eksenini kestiği nokta) biçimindeki doğru denklemini hesapla, doğruların kesişim noktalarını bul, ve bu noktanın her iki doğru parçasında olup olmadığını kontrol et—kesişim noktasını bulmak için bölme işlemini kullanır. Şayet doğru parçaları paralele yakınsa bu yöntem reel sayılar üzerinde bölme operatörünün kesinliği hakkında çok hassastır. Bu bölümde bölme işlemini kullanmayan bir yöntem çok daha doğrudur.



Şekil 33.1 (a) p_1 ve p_2 vektörlerinin vektörel çarpımı paralelkenarın taralı alanıdır. (b) Açık taralı bölgedeki vektörler p nin saat yönündedir. Koyu taralı bölgedeki vektörler ise p nin saat yönünün tersindedir.

Vektörel çarpım

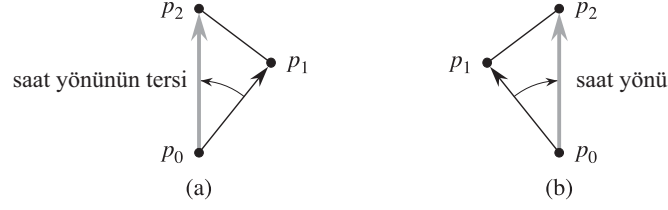
Vektörel çarpımı hesaplamak doğru-parçası yöntemlerinin tam kalbinde yer alır. Şekil 33.1(a) daki p_1 ve p_2 vektörlerini düşünün. $p_1 \times p_2$ **vektörel çarpımı**, $(q_0, 0)$, p_1, p_2 ve $p_1 + p_2 = (x_1 + x_2, y_1 + y_2)$ noktalarıyla tanımlanmış paralelkenarın alanı olarak yorumlayabiliriz. Benzer biçimde, daha da kullanışlı olan aşağıdaki tanım vektörel çarpımın bir matrisin determinantı olarak tanımlıyor¹:

$$\begin{aligned} p_1 \times p_2 &= \det \begin{pmatrix} x_1 & x_2 \\ y_1 & y_2 \end{pmatrix} \\ &= x_1 y_2 - x_2 y_1 \\ &= -p_2 \times p_1 . \end{aligned}$$

Orijini $(0, 0)$ olarak kabul edersek eğer $p_1 \times p_2$ pozitif ise p_1, p_2 nin saat yönünde, negatif ise p_1, p_2 nin saat yönünün tersindedir. (Bkz. Alıştırma 33.1-1) Şekil 33.1.(b) de p vektörüne göre saat yönündeki ve saat yönünün tersindeki noktaları görebiliriz. Vektörel çarpım 0 ise bir sınır koşul ortaya çıkar, bu durum vektörlerin **eş-doğrusal** olduklarını ve aynı yönde ya da zıt yönde bulunduklarını gösterir.

Bir $\overrightarrow{p_0 p_1}$ yönlü doğru parçasının diğer $\overrightarrow{p_0 p_2}$ yönlü doğru parçasına, p_0 ortak uç noktasına göre, saat yönünde mi yoksa saat yönünün tersinde mi yakın olduğunu tespit etmek için ilk olarak p_0 ı orijin olarak kabul ediyoruz. $x'_1 = x_1 - x_0$ ve $y'_1 = y_1 - y_0$ iken $p_1 - p_0$ ifadesi $p'_1 = (x'_1, y'_1)$ vektörünü gösterebiliriz, $p_2 - p_0$ da aynı şekilde tanımlı olsun. O zaman vektörel çarpımı

¹Aslında vektörel çarpım 3 boyutlu uzayda kullanılan bir işlemdir. Bu vektör p_1 ve p_2 nin her ikisine “sağ el kuralına” göre dik olan ve büyüklüğü $|x_1 y_2 - x_2 y_1|$ olan bir vektördür. Bu ünite de ise vektörel çarpımı $x_1 y_2 - x_2 y_1$ değeri olarak kullandık.



Şekil 33.2 Vektörel çarpımın ardışık $\overrightarrow{p_0 p_1}$ ve $\overrightarrow{p_1 p_2}$ doğru parçalarının p_1 noktasında nasıl döndüğünü belirlemek için kullanımı. $\overrightarrow{p_0 p_2}$ yönlü doğru parçasının $\overrightarrow{p_0 p_1}$ yönlü doğru parçasına göre saat yönünde mi yoksa saat yönünün tersinde mi olduğunu kontrol ediyoruz. **(a)** Eğer saat yönünün tersinde ise noktalar sola dönüş yapar **(b)** Eğer saat yönünde ise noktalar sağa dönüş yapar.

$(p_1 - p_0) \times (p_2 - p_0) = (x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0)$ olarak hesaplayabiliriz.

Vektörel çarpım pozitif ise $\overrightarrow{p_0 p_1}$, $\overrightarrow{p_0 p_2}$ nin saat yönünde, negatif ise saat yönünün tersindedir.

Ardışık doğru parçalarının sola mı yoksa sağa mı döndüğünü belirleme

Bir sonraki sorumuz, $\overrightarrow{p_0 p_1}$ ve $\overrightarrow{p_1 p_2}$ ardışık doğru parçalarının p_1 noktasında sola mı yoksa sağa mı dönüş yaptığıdır. Ayrıca verilen bir $\angle p_0 p_1 p_2$ açısının hangi yöne doğru döndüğünü gösteren bir yöntem de ihtiyaç duyuyoruz. Vektörel çarpım bu soruyu açının değerini hesaplamaksızın cevaplamamızı sağlar. Şekil 33.2 de $\overrightarrow{p_0 p_1}$ yönlü doğru parçasının $\overrightarrow{p_0 p_2}$ yönlü doğru parçasına göre saat yönünde mi yoksa saat yönünün tersinde mi olduğunu kontrol ediyoruz. Böyle yaparak $(p_2 - p_0) \times (p_1 - p_0)$ vektörel çarpımın sonucunu hesaplıyoruz. Sonuç negatif ise $\overrightarrow{p_0 p_2}$ yönlü doğru parçası $\overrightarrow{p_0 p_1}$ e göre saat yönünün tersindedir. Bu durumda p_1 de sola dönüş vardır. Sonuç pozitif ise saat yönünü ve sağa dönüşü işaret eder. Vektörel çarpımın 0 olması da p_0 , p_1 ve p_2 nin doğrusal olduğu anlamına gelir.

İki doğru parçasının kesişip kesişmediğini belirleme

İki doğru parçasının kesişip kesişmediğini tespit etmek için her bir doğru parçasının diğer doğru parçasını ihtiva eden doğruyu **straddle** (her iki yanında olmak) edip etmediğini kontrol ederiz. Eğer p_1 noktası doğrunun bir tarafında p_2 noktası doğrunun diğer tarafında ise $\overrightarrow{p_1 p_2}$ doğru parçası bu doğruya çift taraflı işlem uygular. İki doğru parçası kesişir ancak ve ancak aşağıdaki şartlardan biri veya her ikisi birden sağlanırsa:

1. Her bir doğru parçası diğer doğru parçasını ihtiva eden doğruya çift taraflı işlem uygular.
2. Bir doğru parçasının uç noktası diğerinin üzerindedir (Bu şart sınır durumundan kaynaklanır).

Aşağıdaki prosedür bu fikrin uygulamasıdır. $\overline{p_1p_2}$ ve $\overline{p_3p_4}$ doğru parçaları kesişirse SEGMENTS-INTERSECT, TRUE ya, kesişmez ise FALSE a döner. Yukarıdaki vektörel çarpım yöntemini kullanarak göreceli yönelimleri hesaplayan alt programa DIRECTION, bir noktanın bir doğru parçası ile eş doğrusal olup olmadığını yani noktanın o doğru parçasının üzerinde olup olmadığını tespit eden alt programa da ON-SEGMENT adını verelim.

```

SEGMENTS-INTERSECT( $p_1, p_2, p_3, p_4$ )
1   $d_1 = \text{DIRECTION}(p_3, p_4, p_1)$ 
2   $d_2 = \text{DIRECTION}(p_3, p_4, p_2)$ 
3   $d_3 = \text{DIRECTION}(p_1, p_2, p_3)$ 
4   $d_4 = \text{DIRECTION}(p_1, p_2, p_4)$ 
5  if(( $d_1 > 0$  ve  $d_2 < 0$ ) veya ( $d_1 < 0$  ve  $d_2 > 0$ )) ve
    (( $d_3 > 0$  ve  $d_4 < 0$ ) or ( $d_3 < 0$  ve  $d_4 > 0$ ))
6    return TRUE
7  elseif  $d_1 = 0$  ve ON-SEGMENT( $p_3, p_4, p_1$ )
8    return TRUE
9  elseif  $d_2 = 0$  ve ON-SEGMENT( $p_3, p_4, p_2$ )
10   return TRUE
11 elseif  $d_3 = 0$  ve ON-SEGMENT( $p_1, p_2, p_3$ )
12   return TRUE
13 elseif  $d_4 = 0$  ve ON-SEGMENT( $p_1, p_2, p_4$ )
14   return TRUE
15 elsereturn FALSE

```

```

DIRECTION( $p_i, p_j, p_k$ )
1  return  $(p_k - p_i) \times (p_j - p_i)$ 

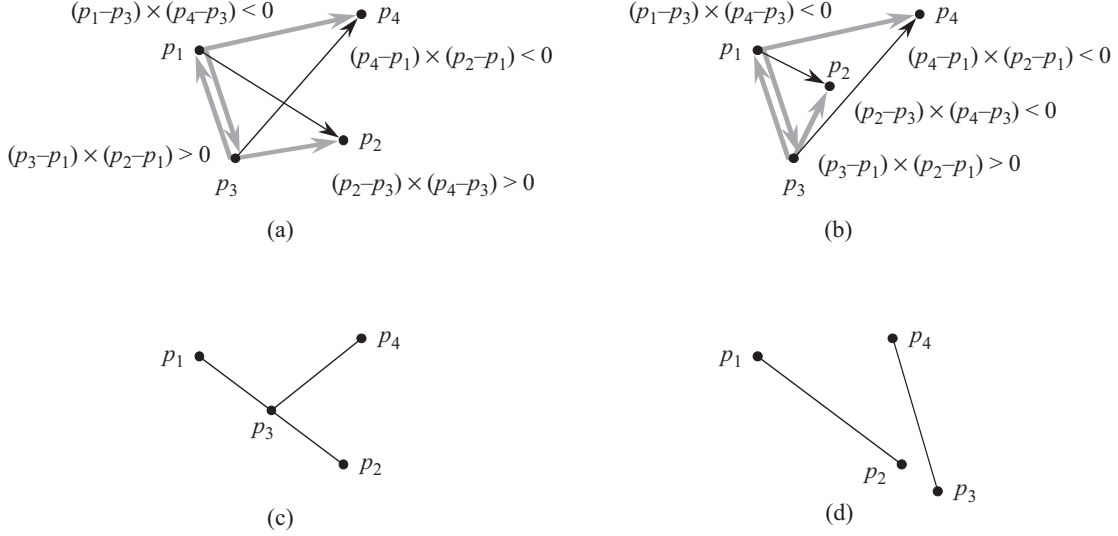
```

```

ON-SEGMENT( $p_i, p_j, p_k$ )
1  if  $\min(x_i, x_j) \leq x_k \leq \max(x_i, x_j)$  ve  $\min(y_i, y_j) \leq y_k \leq \max(y_i, y_j)$ 
2    return TRUE
3  else return FALSE

```

SEGMENTS-INTERSECT şöyle çalışır. 1-4 satırları her bir p_i uç noktasının diğer doğru parçasına göre bağlı yönlerini hesaplar. Eğer tüm bağlı yönler sıfırdan farklı ise $\overline{p_1p_2}$ ve $\overline{p_3p_4}$ doğru parçalarının kesişip kesişmediğini şu şekilde kolayca belirleyebiliriz: $\overline{p_3p_1}$ ve $\overline{p_1p_2}$ yönlü doğru parçaları $\overrightarrow{p_3p_4}$ e göre ters yöne sahipse, $\overline{p_3p_2}$ doğru parçası $\overline{p_3p_4}$ ü ihtiva eden bir doğruya çift taraflı işlem uygular. Böyle bir durumda d_1 ve d_2 nin işaretleri farklıdır. d_3 ve d_4 ün işaretleri farklı ise $\overline{p_3p_4}$, $\overline{p_1p_2}$ nı ihtiva eden doğruya çift taraflı işlem uygular. Eğer 5. Satır doğru ise, doğru parçaları birbirlerini straddle eder ve SEGMENTS-INTERSECTS,



Şekil 33.3 SEGMENTS-INTERSECT prosedüründeki durumlar. (a) $\overline{p_1p_2}$ ve $\overline{p_3p_4}$ doğru parçaları birbirini doğrusunu straddle eder. $\overline{p_3p_4}$, $\overline{p_1p_2}$ 'yi ihtiva eden bir doğruya çift taraflı işlem uyguladığından $(p_3-p_1) \times (p_2-p_1)$ ve $(p_4-p_1) \times (p_2-p_1)$ vektörel çarpımların işaretleri farklıdır. $\overline{p_1p_2}$, $\overline{p_3p_4}$ ü ihtiva eden bir doğruya çift taraflı işlem uyguladığından $(p_1-p_3) \times (p_4-p_3)$ ve $(p_2-p_3) \times (p_4-p_3)$ vektörel çarpımların işaretleri farklıdır. (b) $\overline{p_3p_4}$, $\overline{p_1p_2}$ 'yi ihtiva eden bir doğruya çift taraflı işlem uygularken $\overline{p_1p_2}, \overline{p_3p_4}$ ü ihtiva eden bir doğruya çift taraflı işlem uygulamaz. (c) P_3 noktası $\overline{p_1p_2}$ ile doğrusaldır ve p_1 ile p_2 arasındadır. (d) P_3 noktası $\overline{p_1p_2}$ ile doğrusal fakat p_1 ile p_2 arasında değildir. Bu durumda doğru parçaları kesişmez.

TRUE ya döner. Şekil 33.3 (a) bu durumu anlatıyor. Aksi halde, bir sınır durumu uygulanabilirliğine rağmen doğru parçaları birbirlerinin doğrularına çift taraflı işlem uygulamaz. Eğer hiçbir bağıl yön sıfır değilse sınır durumu uygulanmaz. O zaman 7-13. satırlar böyle bir durumda geçersiz olur ve SEGMENTS-INTERSECT 15. satırda FALSE a döner. Şekil 33.3(b) bu durumu anlatıyor.

Herhangi bir d_k bağıl yön sıfır ise bir sınır durum ortaya çıkar. Burada, p_k nın diğer doğru parçasıyla doğrusal olduğunu biliyoruz. p_k diğer doğru parçası üzerindedir ancak ve ancak p_k diğer doğru parçasının uç noktaları arasındadır. ON-SEGMENT prosedürü p_k nın 7-13. satırlarda çağrıldığında diğer doğru parçası olacak olan $\overline{p_i p_j}$ nin uç noktaları arasında olup olmamasına göre döner; prosedür p_k nin p_i ile doğrusal olduğunu varsayar. Şekil 33.3 (c) ve (d) doğrusal nokta durumlarını gösteriyor. Şekil 33.3 (c) de p_3 , $\overline{p_1p_2}$ nin üzerindedir ve böylece SEGMENTS-INTERSECT TRUE ya döner. Şekil 33.3 (d) de hiçbir uç noktası başka bir doğru parçası üzerinde değildir. ve böylelikle SEGMENTS-INTERSECT 15. satırda FALSE a döner.

Vektörel çarpımın diğer uygulamaları

Bu üniteadaki sonraki bölümler vektörel çarpımın ilave kullanımlarını anlatır. Kısım 33.3 te bir noktalar kümesini verilen bir orijin ve kutupsal açılara göre sıralamaya gerek duyacağız. Alıştırma 33.1-2, sıralama prosedüründe mukayese yapmak için vektörel çarpımın kullanabileceğini göstermemizi istiyor. Kısım 33.2 de bir doğru parçası kümesinin dikey sıralamasını yapmak için kırmızı-siyah ağaçları kullanacağız. Kırmızı-siyah ağaç kodunda birbirine kıyasladığımız açık anahtarları saklamak yerine, verilen bir dikey doğruyu kesen doğru parçasından hangisinin diğeri üzerinde olduğunun tespiti için bir vektörel çarpım hesaplayacağız.

Alıştırmalar

33.1-1

Eğer $p_1 \times p_2$ pozitif ise, p_1 vektörü, (0,0) orijinine göre, p_2 nin saat yönündedir ve negatif ise p_1 vektörü p_2 nin saat yönünün tersindedir, kanıtlayınız.

33.1-2

Profesör van Pelt ON-SEGMENT in 1. Satırında sadece x -boyutun kontrolüne gerek olduğunu ileri sürüyor. Profesör nerede yanılıyor?

33.1-3

Orijin olan bir p_0 noktasına göre p_1 noktasının **kutupsal açısı** $p_1 - p_0$ vektörünün kutupsal koordinat sistemindeki açısıdır. Örneğin; (2,4) noktasına göre (3,5) noktasının kutupsal açısı (1,1) vektörünün 45 derece ya da $\pi/4$ radyan olan açısıdır. (2,4) noktasına göre (3,3) in kutupsal açısı (1,1) vektörünün 315 derece ya da $\pi/4$ radyan olan açısıdır. n noktanın $\langle p_1, p_2, \dots, p_n \rangle$ dizisini, verilen p_0 orijin noktasına göre, kutupsal açılara göre sıralayan bir kabakod yazın. Prosedürünüzün $O(n \lg n)$ zaman sürmesi ve açıları karşılaştırmak için vektörel çarpım kullanmamız gerekmektedir.

33.1-4

n noktalı bir kümede herhangi 3 noktanın doğrusal olup olmadığını $O(n^2 \lg n)$ zamanda nasıl belirleneceğini yazın.

33.1-5

Bir **çokgen**, düzlemde parçalı-doğrusal, kapalı bir eğridir. Kendi üzerinde biten bir eğridir ve bir düz-doğru parçası dizisiyle oluşan çokgen **kenarlarına** sahiptir. Ardışık iki kenarın birleşme noktası çokgenin bir **köşesidir**. Genel kabule göre, bir çokgen kendisini kesmiyorsa **basittir**. Çokgen tarafından sınırlanan bölgede kalan noktalar kümesine çokgenin **içbölgesini**, çokgenin üzerinde kalan noktalar kümesi çokgenin **sınırını**, çokgenin dışında kalan noktalar kümesi de çokgenin

dışbölgesini oluşturur. Sınırında yada içbölgesinde alınan herhangi iki noktanın oluşturduğu doğru parçasının tüm noktaları her zaman çokgenin sınırında veya içbölgesinde kalıyorsa, bu çokgene, **dışbükey** çokgen adı verilir. Bir dışbükey çokgenin bir köşesi sınırındaki ya da iç bölgesindeki herhangi iki noktanın bir dışbükey kombinasyonu olarak ifade edilemez.

Profesör Amundsen, n noktalı bir $\langle p_0, p_1, \dots, p_{n-1} \rangle$ dizisinin dışbükey bir çokgenin ardışık köşeleri olup olmadığını tespit eden aşağıdaki yöntemi ortaya atıyor. Alt-kod toplamının büyüklüğü n iken $\{\angle p_i p_{i+1} p_{i+2} : i = 0, 1, \dots, n-1\}$ kümesi sola dönüş ve sağa dönüş içermiyor ise çıktı “evet”, aksi halde çıktı “hayır”dır. Bu yöntemin doğrusal sürede çalışmasına rağmen her zaman doğru cevap vermediğini gösterin. Profesör’ün yöntemini doğrusal sürede her zaman doğru cevabı verecek şekilde düzenleyin.

33.1-6

Verilen bir $p_0 = (x_0, y_0)$, noktası için p_0 in **sağ yatay ışını**, $\{p_i = (x_i, y_i) : x_i \geq x_0 \text{ ve } y_i = y_0\}$ noktalar kümesidir, yani bu küme p_0 ile p_0 nın sağında kalan noktalar kümesidir. Verilen bir p_0 in sağ yatay ışınının $\overline{p_1 p_2}$ doğru parçasını kesip kesmediğini, bu problemi iki doğru parçasının kesişip kesişmediğine indirgeyerek, $O(1)$ zamanda nasıl belirleneceğin gösterin.

33.1-7

Bir p_0 noktasının basit fakat dışbükey olmasına gerek olmayan bir çokgenin iç bölgesinde olup olmadığını tespit etmenin bir yolu da, p_0 in P nin sınırında olması şartıyla p_0 dan çıkan herhangi bir ışının P nin sınırını tek sayı kez kesmesidir. Bir p_0 noktasının n -köşeli P çokgeninin içinde olup olmadığı $\Theta(n)$ sürede nasıl hesaplanır? (*İpucu:* Alıştırma 33.1-6 dan faydalanın. Işının çokgeni bir köşede ve ışının çokgenin bir kenarını kestiği zaman algoritmanın doğru çalıştığına emin olun).

33.1-8

n -köşeli basit ama dışbükey olmasına gerek olmayan bir çokgenin alanının $\Theta(n)$ zamanda nasıl hesaplanacağını gösterin (bkz. Alıştırma 33.15 te çokgenle ilgili tanımlar).

33.2 Herhangi bir doğru parçası çiftinin kesişip kesişmediğinin belirlenmesi

Bu bölüm, doğru parçaları kümesinde herhangi iki doğru parçasının kesişip kesişmediğini belirleyen bir algoritmayı anlatıyor. Bu algoritma, bir çok hesaplanabilir geometri algoritmalarında ortak olan “sweeping” (tarama) adlı bir tekniği kullanıyor. Ayrıca bölüm sonundaki alışırmalarda gösterildiği gibi, bu algoritma,

ya da daha basit varyasyonları başka hesaplanabilir geometri problemlerinin çözümünde de kolaylıklar sağlar.

n doğru parçası sayısı olmak üzere bu algoritma $O(n \lg n)$ zamanda çalışır. Algoritma sadece herhangi bir kesişmenin olup olmadığını tespit eder, tüm kesişmelerin çıktısını vermez (Alistırma 33.2-1, n doğru parçalı bir kümede tüm kesişimleri bulmanın, en kötü durumda $\Omega(n^2)$ zaman aldığını gösteriyor).

Tarama tekniğinde farazi dikey bir **tarama doğrusu** genellikle soldan sağa doğru olmak üzere verilen geometrik cisimlerin arasından geçer. Tarama doğrusunun geçtiği uzaysal boyutu, bu durumda x -boyutunu, zamanın bir boyutu olarak kabul ediyoruz. Tarama tekniği geometrik cisimlerin sıralamasını, bunu genellikle bu cisimleri veri yapısına yerleştirerek yapar ve bunların arasındaki ilişkileri daha iyi bilinmesi için bir yöntem sağlar. Bu bölümdeki line-segment-intersection (doğru-parçası-kesişi) algoritması soldan sağa sıralı tüm doğru—parçası uç noktalarını gözler ve bir uç noktası ile karşılaştığı her an bir kesişim kollar.

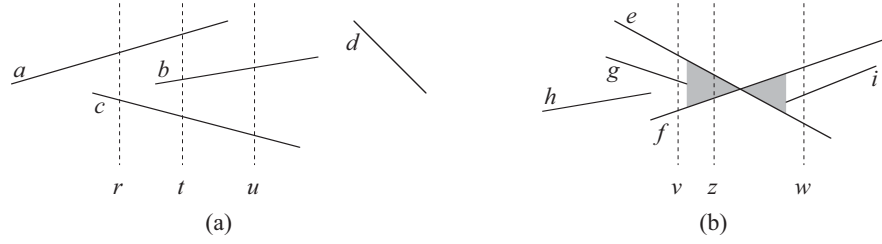
n doğru parçasında kesişen herhangi iki doğrunun olup olmadığını tespit eden algoritmayı tanımlamak ve doğrulamak için iki basit varsayım yapacağız. İlk olarak hiçbir doğru parçası girdisinin dik olmadığını varsayacağız. İkinci olarak herhangi bir noktada kesişen üç doğru parçası girdisinin olmadığını varsayacağız. Alistırma 33.2-8 ve 33.2-9 yukarıdaki varsayımlar sağlanmadığında küçük bir değişikliklerle çalışan algoritmayı soruyor. Aslında bu varsayımları çıkardığımızda sınır şartlarla işlem yapmak, hesaplanabilir geometri algoritmalarını programlamak ve bunu doğrulamak çok zor bir problem olmasına neden olur.

Doğru parçalarını sıralamak

Hiçbir dikey doğru parçası varsaymadığımızdan dolayı verilen bir dikey tarama doğrusunu kesen herhangi bir doğru parçası girdisi onu sadece bir noktada keser. Böylece, bir dikey tarama doğrusuyla kesişen doğru parçalarını, kesişim noktalarının y koordinatlarına göre sıralayabiliriz.

Daha açık bir ifadeyle s_1 ve s_2 adlı iki doğru parçası düşünün. x -koordinatlı x dikey doğru bunların her ikisini de keserse bu doğru parçalarının x te **karşılaştırılabilir** olduğunu söyleriz. Eğer s_1 ve s_2 , x te karşılaştırılabilir ve s_1 in tarama doğrusu ile x teki kesişimi, s_2 nin aynı tarama doğrusu ile kesişiminden daha yukarda ise ya da s_1 ve s_2 aynı tarama doğrusu üzerinde kesişirse s_1, s_2 nin **üstündedir** deriz ve $s_1 \geq_x s_2$ diye yazarız. Şekil 33.4 (a), örneğinde, $a \geq_r c$, $a \geq_l b$, $b \geq_l c$, $a \geq_l c$ ve $b \geq_u c$ ilişkisi vardır. d doğru parçası diğer hiçbir doğru parçası ile karşılaştırılabilir değildir.

Verilen herhangi bir x için “ $\geq_x$ ” bağıntısı, tarama doğrusu i le x te kesişen tüm doğru parçaları için tam ön sıralama bağıntısıdır (bkz. Kısım B.2) Bu şu demektir: s_1 ve s_2 doğru parçalarının her biri tarama doğrusu ile x te kesişirse bu bağıntı geçişkindir o zaman ya $s_1 \geq_x s_2$ ya da $s_2 \geq_x s_1$ dir veyahut her ikisi de geçerlidir (s_1 ve s_2 aynı tarama doğrusu ile kesişirse) Bu bağıntı ayrıca yansıyandır, fakat



Şekil 33.4 Farklı dikey tarama doğrularında kesişen doğru parçaları arasındaki sıralama. (a) $a \geq_r c$, $a \geq_t b$, $b \geq_t c$, $a \geq_u c$, ve $b \geq_u c$ dir. d doğru parçasının başka hiçbir doğru parçası ile karşılaştırılabilir olmadığını gösteriyor (b) e ve f doğru parçaları kesişirse sıralamaları değişir $e \geq_v f$ olur fakat $f \geq_w e$ dir. Taralı bölgeden geçen herhangi bir tarama doğrusu (z gibi), $\geq_z$ bağıntısı ile sıralı ardışık e ve f ye sahiptir.

simetrik ya da ters simetrik değildir. Doğru parçaları sıralamadan girip çıkarken tam ön sıralama bağıntısı x in farklı değerlerine göre farklılık gösterir. Bir doğru parçası, solundaki uç noktası tarama ile karşılaştığında sıralamaya girer, sağındaki uç noktası karşılaştığında ise sıralamadan ayrılır.

Süpürme doğrusu iki doğru parçasının kesişiminden geçerse ne olur? Şekil 33.4(b) gösterdiği gibi doğru parçaları tam ön sıralama bağıntısında yerlerini değiştirir. e ve f doğru parçalarının kesişim noktasının solunda ve sağında y ve w tarama doğruları var ve bu durumda $e \geq_y f$ ve $f \geq_w e$ dir. Herhangi üç doğru parçasının aynı noktada kesişmediğini varsaydığımızdan dolayı, $\geq_x$ tam ön sıralama bağıntısında *ardışık* olan e ve f doğru parçaları için bazı x tarama doğrusu olması gerektiğine dikkat edin. Şekil 33.4(b) de z gibi taralı bölgenin içinden geçen herhangi bir tarama doğrusu tam ön sıralama bağıntısında ardışık e ve f ye sahiptir.

Tarama doğrusunu hareket ettirme

Tarama algoritmaları tipik olarak iki veri kümesine ayrılır.

1. **Tarama-doğrusu durumları**, süpürme doğrusunun kestiği cisimler arasında bir ilişki kurar.
2. **Olay-noktası çizelgeleme**, x -koordinatlarına göre soldan sağa doğru sıraladığımız **olay noktalarının** dizisidir. Tarama soldan sağa doğru çalıştığında, süpürme doğrusu bir olay noktasının x koordinatına denk gelse tarama durur, olay noktasını işler ve daha sonra yeniden başlar. Tarama-doğrusu statüsündeki değişimler yalnızca olay noktalarında gerçekleşir.

Bazı algoritmalar için (örneğin Alıştırma 33.2-7 de sorulduğu gibi) olay-noktası çizelgeleme algoritma çalışırken dinamik olarak gelişir. Mevcut algoritma, bununla birlikte, taramadan önce tüm olay noktalarını tespit eder ve yalnızca veri

girdisinin basit özellikleri üzerine kuruludur. Daha açık bir ifadeyle, her doğru parçasının uç noktası bir olay noktasıdır. Doğru parçası uç noktalarını artan x -koordinatlarına göre sıralıyoruz ve soldan sağa doğru ilerliyoruz (eğer iki ya da daha fazla uç noktası **eşdikey** ise yani aynı x -koordinatına sahip ise eşdikey olan tüm sol uç noktalarını eşdikey sağ uç noktalarından önce koyarak bağlantıyı kesiyoruz. Bir eşdikey sol uç noktalar kümesi içinde bunları ilk olarak daha düşük y -koordinatlılar gelecek şekilde yerleştiriyoruz ve daha sonra bir eşdikey sağ uç noktaları için de aynı işi yapıyoruz). Bir doğru parçasının sol uç noktasıyla karşılaştırsak, doğru parçasını tarama-doğrusu durumuna sokuyoruz ve sağ uç noktası ile karşılaştırsak doğru parçasını tarama-doğrusu durumundan çıkarıyoruz. İki doğru parçası tam ön sıralama bağıntısında ardışık oldukları ilk anda, kesişip kesişmediğini kontrol ediyoruz.

Tarama-doğrusu durumu bir T tam ön sıralama bağıntısıdır. T için aşağıdaki işlemler gereklidir.

- $\text{INSERT}(T, s)$: s doğru parçasını T nin içine sok
- $\text{DELETE}(T, s)$: doğru parçasını T den sil
- $\text{ABOVE}(T, s)$: derhal T de s doğru parçasının yukarısında olan doğru parçasına dön
- $\text{BELOW}(T, s)$: derhal T de s doğru parçasının aşağısında olan doğru parçasına dön

s_1 ve s_2 doğru parçasının her ikisinin de tam ön sıralama bağıntısı T de birbiri üzerinde olması mümkündür; bu durum eğer s_1 ve s_2 , tam ön sıralama bağıntısı T olan tarama doğrusunda kesişirse gerçekleşir. Bu durumda iki doğru parçası da T sıralamasında gözükür.

Girdi, n doğru parçasından oluşursa INSERT , DELETE , ABOVE VE BELOW işlemlerini kırmızı-siyah ağaçları kullanarak $O(\lg n)$ zamanda yapabiliyoruz. 13. Bölüm de karşılaştırma anahtarlarını içeren kırmızı-siyah-ağaç işlemlerini hatırlayın. İki doğru parçasının göreceli sıralamasını tespit etmek için vektörel çarpımı kullanırken anahtar karşılaştırmaları yerine karşılaştırmaları kullanabiliriz (bkz 33.2-2).

Doğru parçası—kesişim kabakodu

Aşağıdaki algoritma n doğru parçalı bir S kümesini girdi olarak alır, S de herhangi bir doğru parçası çifti kesişirse boole değeri TRUE ya döndürülür aksi halde FALSE a döndürür. Bir kırmızı-siyah-ağaç tam ön sıralama bağıntısı T yi çalıştırır.

ANY-SEGMENT-INTERSECT (S)

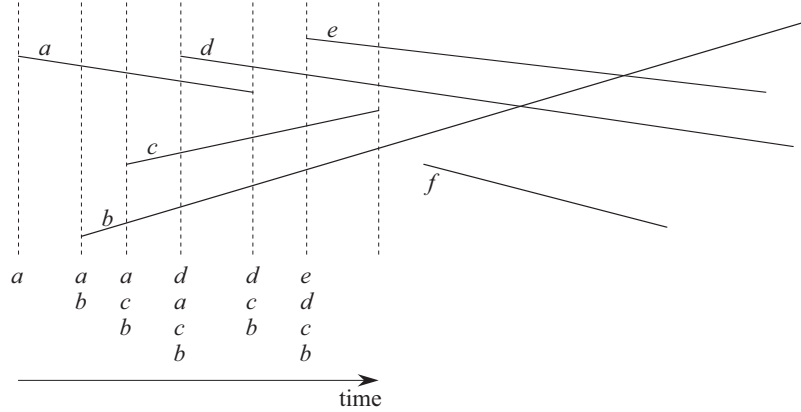
```

1   $T = \emptyset$ 
2   $S$  deki doğru parçalarının uç noktalarını soldan sağa
    doğru sırala, sağ uç noktalarından önce sol uç noktalarını
    koyarak bağlantıları kes ve noktaları ilk olarak aşağıdaki
     $y$ -koordinatlar olacak şekilde koyarak ileri bağlantıları kes
3  for uç noktalarının sıralı listesinde her  $p$  noktası
4      if  $p$ , bir  $s$  doğru parçasının sol uç noktası
5          INSERT ( $T, s$ )
6          if (ABOVE ( $T, s$ ) gerçekleşir ve  $s$  ile kesişirse)
              veya (BELOW ( $T, s$ ) gerçekleşir ve  $s$  ile kesişirse)
7              return TRUE
8      if  $p$ ,  $s$  doğru parçasının sağ uç noktası
9          if hem ABOVE( $T, s$ ) hem de BELOW( $T, s$ ) gerçekleşir
              ve ABOVE( $T, s$ ) BELOW( $T, s$ ) ile kesişirse
10             return TRUE
11             DELETE ( $T, s$ )
12  return FALSE

```

Şekil 33.5 algoritmanın nasıl çalıştığını gösteriyor. 1. satır, tam ön sıralama bağıntısını sıfır olarak başlatıyor. 2. satır, $2n$ doğru parçası uç noktalarını solda sağa sıralayarak ve yukarıda belirtilen şekilde bağlantıları keserek olay-noktası çizelgeleme belirliyor. 2. satır şöyle de gösterilebilir: x ve y birer koordinat, bir sol uç nokta için $e = 0$, bir sağ uç nokta için $e = 1$ olmak üzere uç noktalar (x, e, y) üzerinde sıralanır.

3-11 satırlarındaki **for** döngülerinin her iterasyonu bir tane p olay noktası işler. p , s doğru parçasının sol uç noktası ise 5. satır s yi tam ön sıralama bağıntısına ekler ve eğer s , p den geçen tarama doğrusu ile tanımlanan tam ön sıralama bağıntısı içinde kendisinin ardışık olduğu herhangi bir doğru parçasını keserse 6-7. satırlar TRUE döndürür. (eğer p başka bir s' doğru parçası üzerinde ise bir sınır şartı oluşur. Bu durumda bize gerekli olan şey sadece s ve s' nin T nin içinde ardışık olarak yerleşmesidir.) Eğer p , bir s doğru parçasının sağ uç noktası ise o zaman s yi tam ön sıralama bağıntısından silmemiz gerekir. Fakat ilk olarak p den geçen süpürme doğrusu ile tanımlanan tam ön sıralama bağıntısı içinde s nin etrafındaki doğru parçaları arasında bir kesişme olursa, 9-10 satırlar TRUE ya döner. Eğer bunlar kesişmez ise 11. satır, s yi tam ön sıralama bağıntısından siler. S nin çevresindeki doğru parçaları kesişirse 11. satırın işlem görmesini engelleyen 10. satırdaki **return** e dönmüş halde s yi sildikten sonra bu doğru parçaları ardışık olurlar. Bu tezin tam ön sıralama daha da açık hale getirecek olan şey aşağıda da bahsedilen s nin çevresindeki doğru parçalarını kontrol etmenin neden yeterli olduğudur. Sonuç olarak tüm $2n$ olay noktasını işledikten sonra herhangi bir kesişim bulamaz isek, 12. satır FALSE a döner.



Şekil 33.5 ANY-SEGMENT-INTERSECT gösterimi. Her bir kesik çizgi bir olay noktasındaki tarama doğrusudur. En sağdaki tarama doğrusu hariç her tarama doğrusunun altındaki doğru parçalarının adlarının sıralaması, **for** döngülerinin karşılık gelen olay noktasının işleminin sonundaki T tam ön sıralama bağıntısına karşılık gelir. C doğru parçasının sağ uç noktasını işlerken en sağdaki tarama doğrusu meydana çıkar; çünkü c nin etrafındaki d ve b doğru parçalarıdır ve birbiriyle kesişir, prosedür TRUE ya döner.

Doğruluk

ANY-SEGMENT-INTERSECTS nin doğru olduğunu göstermek için ANY-SEGMENTS-INTERSECT çağırısı TRUE ya döndüğünde ancak ve ancak S deki doğru parçaları arasında bir kesişmenin var olduğunu kanıtlayacağız.

ANY-SEGMENTS-INTERSECT in sadece girdi doğru parçaları arasında herhangi bir kesişme olduğunu bulması halinde TRUE ya döndüğünü (7. ve 10. satırlarda), görmek kolaydır. Sonuç olarak eğer TRUE ya dönerse, bir kesişme vardır.

Bunun tersini de kanıtlamamız gerekecek: bir kesişme varsa o zaman ANY-SEGMENTS-INTERSECT TRUE ya döner. En az bir tane kesişme olduğunu varsayalım. P , en soldaki kesişme noktası olsun, en düşük y -koordinatlı noktayı seçip bağlantıları keselim ve a ve b , p de kesişen doğru parçaları olsun. P nin solunda hiçbir kesişme oluşmayacağından T tarafından verilen sıralama, p nin solundaki tüm noktalar için doğrudur. Hiç bir noktada üç doğru parçası kesişmediğinden a ve b bazı z tarama doğrusunda tam ön sıralama bağıntısının içinde ardışık olurlar.² Üstelik z , p nin solunda ya da p den geçer. z tarama doğrusunda

²Eğer üç tane doğru parçasının aynı noktada kesişmesine izin verirse o zaman p noktasında hem a hem de b ile kesişen bir c doğru parçası ortaya çıkabilir. Bu; $a \geq_w b$ iken p nin solundaki tüm w tarama doğruları için $a \geq_w c$ ve $c \geq_w b$ olabilmesi demektir. Alıştırma 33.2-8, aynı noktada üç doğru parçası kesişirse dahi ANY-SEGMENTS-INTERSECT'in doğru olduğunu göstermenizi istiyor.

bazı q doğru parçası uç noktaları, a ve b nin tam ön sıralama bağıntısında ardışık olduğu yerde olay noktasıdır. p, z tarama doğrusu üzerinde ise $q = p$ dir. p, z tarama doğrusu üzerinde değil ise q, p nin solundadır. Her iki durumda da q ile karşılaşmadan hemen önce T tarafından verilen sıralama doğrudur (Sonuçta algoritma durum noktalarını işlerken sözlük sıralamasını kullanıyoruz. p en soldaki kesişme noktalarının en düşüğü olduğundan, p, z tarama doğrusu üzerinde ve bazı diğer kesişme noktası olan p', z nin üzerinde olsa bile, $q = p$ olay noktası, başka kesişme noktası p' in T tam ön sıralama bağıntısına müdahale ihtimalinden önce işlem görür. Üstelik, p, a denilen bir doğru parçasının sol uç noktası ve b denilen doğru parçasının sağ uç noktası olsa bile, sol uç nokta olay noktaları sağ uç nokta olay noktalarından önce oluştuğu için, b doğru parçası T nin içinde ilk karşılaşılan a doğru parçasının üzerinde olur.) q olay noktası ya ANY-SEGMENTS-INTERSECT tarafından işlem görür ya da işlem görmez.

Eğer q , ANY-SEGMENTS-INTERSECT tarafından işlem görürse yalnız iki olay mümkün olur:

1. Ya a ya da b T ye sokulur ve diğer doğru parçası, tam ön sıralama bağıntısının ya yukarısında ya da aşağısında olur. 4-7 satırları bu durumu inceler.
2. a ve b doğru parçaları T nin içinde hazırdır ve tam ön sıralama bağıntısı içinde bunların arasında kalan doğru parçası silinir, bu durumda a ve b ardışık olur. 8-11 satırları da bu durumu inceler.

Her iki durumda da, p kesişimini buluruz ve ANY-SEGMENTS-INTERSECT de TRUE ya döner.

q olay noktası ANY-SEGMENTS-INTERSECT tarafından işlem görmez ise prosedür tüm olay noktalarının işleminden önce dönmesi gerekir. Bu durum, ANY-SEGMENTS-INTERSECT bir kesişme bulur ve TRUE ya dönerse gerçekleşir.

Sonuç olarak, bir kesişme varsa ANY-SEGMENTS-INTERSECT TRUE ya döner. Az önce gördüğümüz gibi, ANY-SEGMENTS-INTERSECT TRUE ya dönerse bir kesişme vardır. Yani, ANY-SEGMENTS-INTERSECT her zaman doğru cevaba döndürür.

Çalışma süresi

S kümesi n doğru parçası içerirse ANY-SEGMENTS-INTERSECT $O(n \lg n)$ zamanda çalışır. 1. Satır $O(1)$ zaman sürer. 2. Satır birleştirerek sıralama veya bellek yığını sıralaması kullanarak $O(n \lg n)$ sürede çalışır. 3-11 arasındaki satırlardaki for döngüleri her olay noktası için en çok bir kez yinelenir ve böylece $2n$ olay noktalı durumda, döngü en çok $2n$ zaman da yinelenir. Her iterasyon $O(\lg n)$ zaman alır, her kırmızı-siyah-ağaç işlemi $O(\lg n)$ zaman aldığından ve Bölüm 33.1 deki yöntemi kullandığından dolayı, her kesişme testi $O(1)$ zaman alır. Böylelikle toplam zaman $O(n \lg n)$ dir.

Alıştırmalar

33.2-1

n doğru parçalı bir kümenin $\Theta(n^2)$ kesişme kapsayabileceğini gösterin.

33.2-2

x te karşılaştırılabilir iki doğru parçası a ve b olsun, $O(1)$ zamanda, $a \geq_x b$ ve $b \geq_x a$ dan hangisinin geçerli olduğunu nasıl belirleneceğini gösterin. Hiç bir doğru parçasının dikey olmadığını farz edin. (*İpucu:* eğer a ve b kesişmez ise vektörel çarpımı kullanabilirsiniz. Eğer a ve b kesişirse—sadece vektörel çarpımı kullandığınızı da söyleyebilirsiniz—yine sadece toplama, çıkarma ve çarpmayı kullanabilir, bölmeden uzak durabilirsiniz. Tabi ki burada $\geq_x$ bağıntısının içindeki uygulama kullanıldı, eğer a ve b kesişirse, durabilir ve bir kesişme olduğunu ilan ederiz.

33.2-3

Profesör Mason ANY-SEGMENTS-INTERSECT i değiştirmemizi öneriyor şöyle ki; kesişmenin üzerinde dönmektense, değiştirilen kod kesişen doğru parçalarını çıktısını verir ve **for** döngüsünün bir sonraki iterasyonuna devam eder. Profesör bu prosedüre PRINT-INTERSECTING-SEGMENTS diyor ve soldan sağa, doğru parçası kümesinde oluşan tüm kesişimlerin çıktısını verdiğini iddia ediyor. Profesör Dixon buna katılmıyor, Profesör Mason'un fikrinin doğru olmadığını savunuyor. Hangi profesör haklı? PRINT-INTERSECTING-SEGMENTS her zaman ilk olarak en soldaki kesişimi mi bulur? Her zaman tüm kesişimleri bulur mu?

33.2-4

n -köşeli bir çokgenin basit olup olmadığını belirleyen bir $O(n \lg n)$ zaman algoritması yazın.

33.2-5

Toplam n köşeli iki basit çokgenin kesişip kesişmediğini belirleyen bir $O(n \lg n)$ zaman algoritması bulun.

33.2-6

Bir disk, çember ve iç bölgesini içerir ve merkez noktası ve yarıçap ile gösterilir. İki diskin ortak bir noktası varsa kesişirler. n elemanlı bir kümede herhangi iki diskin kesişip kesişmediğini belirleyen bir $O(n \lg n)$ zaman algoritması söyleyin.

33.2-7

Toplam k kadar kesişim içeren n elemanlı bir doğru parçası kümesi veriliyor. $O(n + k \lg n)$ zamanda tüm kesişimlerin çıktısının nasıl verileceğini gösterin.

33.2-8

ANY-SEGMENTS-INTERSECT in, aynı noktada üç veya daha fazla doğru parçasının kesişmesinde bile doğru çalıştığını tartışın.

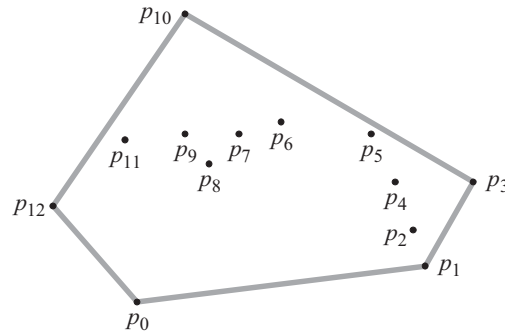
33.2-9

ANY-SEGMENTS-INTERSECT in, dikey doğru parçalarının bulunduğu yerde, bir dikey doğru parçasının, en alttaki uç noktasını bir sol uç noktası ve en üstteki uç noktasını bir sağ uç noktası olarak alsak da doğru çalıştığını gösterin. Alıştırma 33.2-2 ye verdiğiniz cevabınız, tüm dikey doğru parçalarına için izin verdiğimizde nasıl olur?

33.3 Dışbükey örtü bulma

Bir Q noktalar kümesinin **dışbükey örtüsü (convex hull)** $CH(Q)$ ile gösterilir ve en küçük P dışbükey çokgenidir öyle ki Q daki her nokta ya P nin sınırı üzerinde ya da içindedir. (dışbükey örtünün daha özel bir tanımı için bkz Alıştırma 33.1-5). Dolaylı olarak Q daki tüm noktaların tek olduğunu ve Q nun eş doğrusal olmayan en az üç nokta içerdiğini varsayıyoruz. Q daki her noktayı yüzeyden dışarı çıkan bir çivi gibi düşünebiliriz. O halde dışbükey örtü, tüm noktalarını saran sıkı bir paket lastiği şeklindedir. Şekil 33.6 bir noktalar kümesini ve onun dışbükey örtüsünü gösteriyor.

Bu bölümde, n noktalı bir kümenin dışbükey örtüsünü hesaplayan iki algoritma sunacağız. İki algoritma da dışbükey örtünün köşelerinin çıktısını saat yönünün tersinde veriyor. İlki, $O(n \lg n)$ zaman da çalışan Graham's Scan (Graham Taraması). İkincisi, $O(nh)$ zamanda çalışan Jarvis's March (Jarvis Marşı) öyle ki h , dışbükey örtünün köşe sayısı olsun. Şekil 33.6 gösterdiği gibi $CH(Q)$ nun her köşesi Q içinde bir noktadır. İki algoritma da bu özelliği, Q daki hangi köşelerin



Şekil 33.6 $Q = \{p_1, p_1, \dots, p_{12}\}$ noktalar kümesi gri renkteki $CH(Q)$ dışbükey örtü ile birlikte.

dışbükey çokgenin köşeleri olarak kabul edilip hangilerinin red edileceğinin kararını vermede kullanır.

Dışbükey çokgenleri birkaç yöntemden herhangi birisi ile $O(n \lg n)$ sürede hesaplayabiliriz. Graham Taraması ve Jarvis marşının her ikisi de “rotational sweep (dönen tarama)” adında bir teknik kullanır. Bu teknik, köşeleri, bir referans köşeye göre düzenlenmiş kutupsal açılarının sıralamasına göre işler. Diğer yöntemler ise aşağıdakileri uygular:

- **Artan yöntemde**, ilk olarak noktaları soldan sağa doğru sıralıyoruz ve $\langle p_1, p_2, \dots, p_n \rangle$ dizisini elde ediyoruz. i . aşamada dışbükey çokgenin $i - 1$ en-sol $CH(\{p_1, p_2, \dots, p_{i-1}\})$ noktalarını soldan i . noktaya göre düzenleyip, $CH(\{p_1, p_2, \dots, p_i\})$ yi elde ediyoruz. Alıştırma 33.3-6 bu yöntemi $O(n \lg n)$ zamanın tamamında nasıl uygulanacağını size soruyor.
- **Böl-ve-yönet yönteminde** n noktalı kümeyi $\Theta(n)$ sürede iki alt kümeye ayırıyoruz, bunlardan biri en-sol $\lceil n/2 \rceil$ noktayı diğeri de en sağ $\lfloor n/2 \rfloor$ noktayı ihtiva ediyor, bu alt kümelerin dış bükey çokgenlerini hesaplıyoruz ve sonra akılcı bir yöntem kullanarak, tüm örtüyü $O(n)$ sürede birleştiriyoruz. Çalışma süresi $T(n) = 2T(n/2) + O(n)$ özyineleme ile tanımlıdır ve böylece böl-ve-yönet yönteminin çalışma süresi $O(n \lg n)$ olur.
- **Buda-ve-ara yöntemi** Kısım 9.3 teki worst-case linear time median (en kötü-durum doğrusal zaman medyan) algoritmasına benzerdir. Bu yöntemle dışbükey çokgenin üst kısmını (ya da “üst zincirini”), dış bükey çokgenin yalnızca üst zinciri kalana kadar kalan noktaların sabit bir kısmını atıyoruz. Alt zincir için de aynı şeyi yapıyoruz. Bu yöntem asimptotik olarak en hızlıdır: eğer dışbükey örtünün h köşesi varsa, çalışma süresi $O(n \lg h)$ olur.

Bir noktalar kümesinin dışbükey örtüsünü hesaplamak kendi içinde ilginç bir problemdir. Üstelik, diğer bazı hesaplamalı-geometri problemlerinin algoritmaları, dışbükey örtüyü hesaplarken başlar. Düşünün, örneğin, iki boyutlu **en uzak-çift problemi (farthest-pair problem)**: bize düzlemde n noktalı bir küme veriliyor ve bizden birbiriyle uzaklığı maksimum olan iki noktayı bulmamız isteniyor. Alıştırma 33.3-3 ün sizden kanıtlamasını istediği gibi bu iki nokta dışbükey örtünün köşeleri olmak zorunda. Burada bunu kanıtlamayacağız ama n -köşeli bir dışbükey çokgenin köşelerinin en uzak çiftini $O(n)$ sürede bulabiliriz. Böylelikle, n girdi noktaların dışbükey örtüsünü $O(n \lg n)$ zamanda hesapladıktan sonra oluşan dışbükey-çokgen köşelerinin en uzak çiftini bularak, n noktalı herhangi bir kümenin en uzak çiftini $O(n \lg n)$ zamanda bulabiliriz.

Graham taraması

Graham taraması dışbükey-örtü problemini aday noktalardan oluşan bir S yığınınını işleyerek çözer. Q girdi kümesinin her bir noktasını tek seferde yığına iter, ve

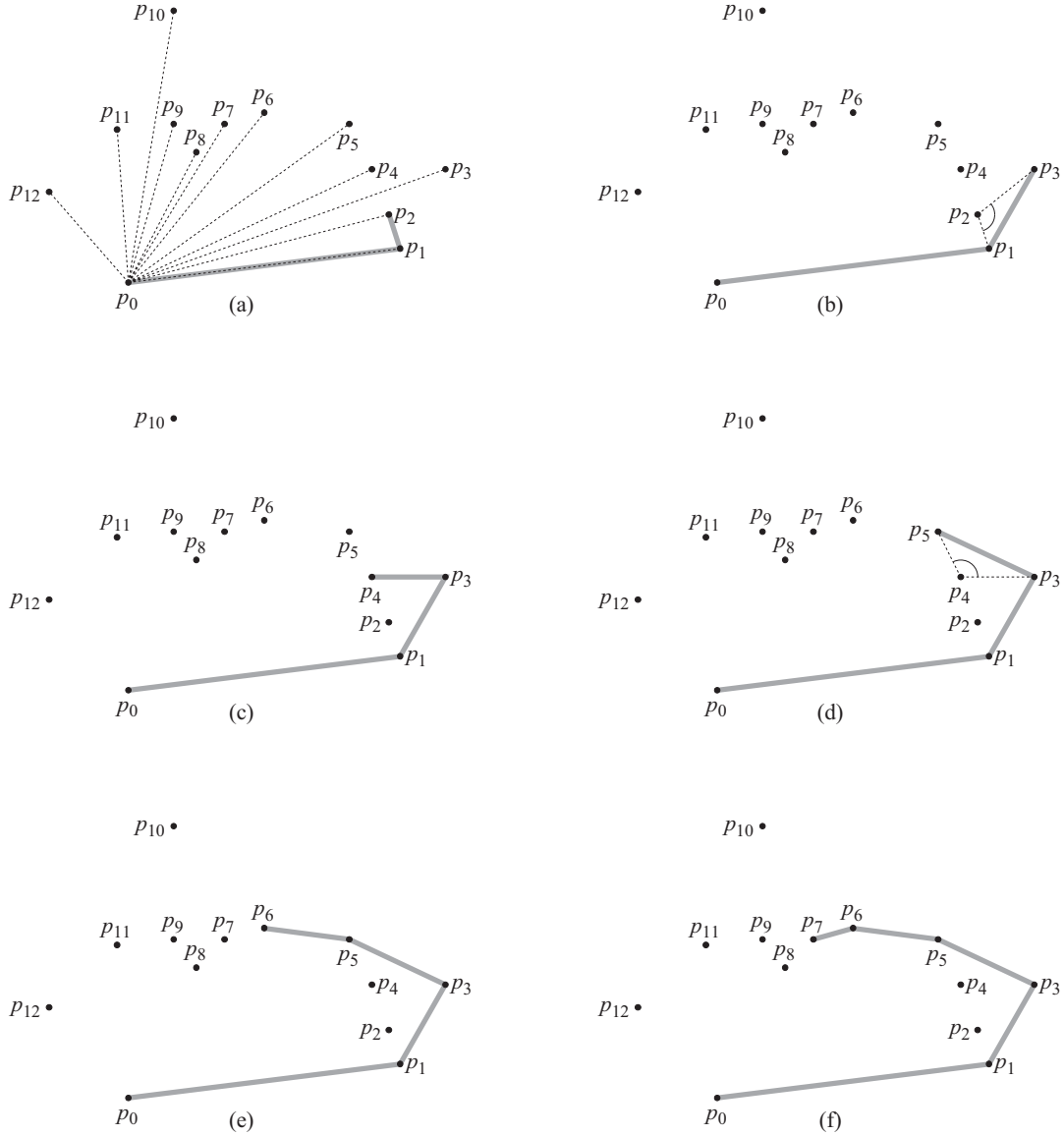
zamanla bu yığından $CH(Q)$ nun köşesi olmayan her bir noktayı çıkarır. Algoritma sona erdiğinde, S yığını tamamen $CH(Q)$ nun köşelerinden oluşur ve bunların sınır üzerinde görüntüleri saat yönünün tersinde dizilirler.

GRAHAM-SCAN prosedürü girdi olarak Q noktalar kümesini alır öyle ki $|Q| \geq 3$ dir. $TOP(S)$, S yığınıni değiştirmeden S yığınının en üstteki noktasına dönen fonksiyon ve $NEXT-TO-TOP(s)$ ise S yığınıni değiştirmeden S yığının en üstünün altındaki nokta girdisine dönen fonksiyondur. İlerde kanıtlayacağımız gibi GRAHAM-SCAN tarafından dönen S yığını, en alttan en üste tüm $CH(Q)$ köşelerini saat yönünün tersinde içerir.

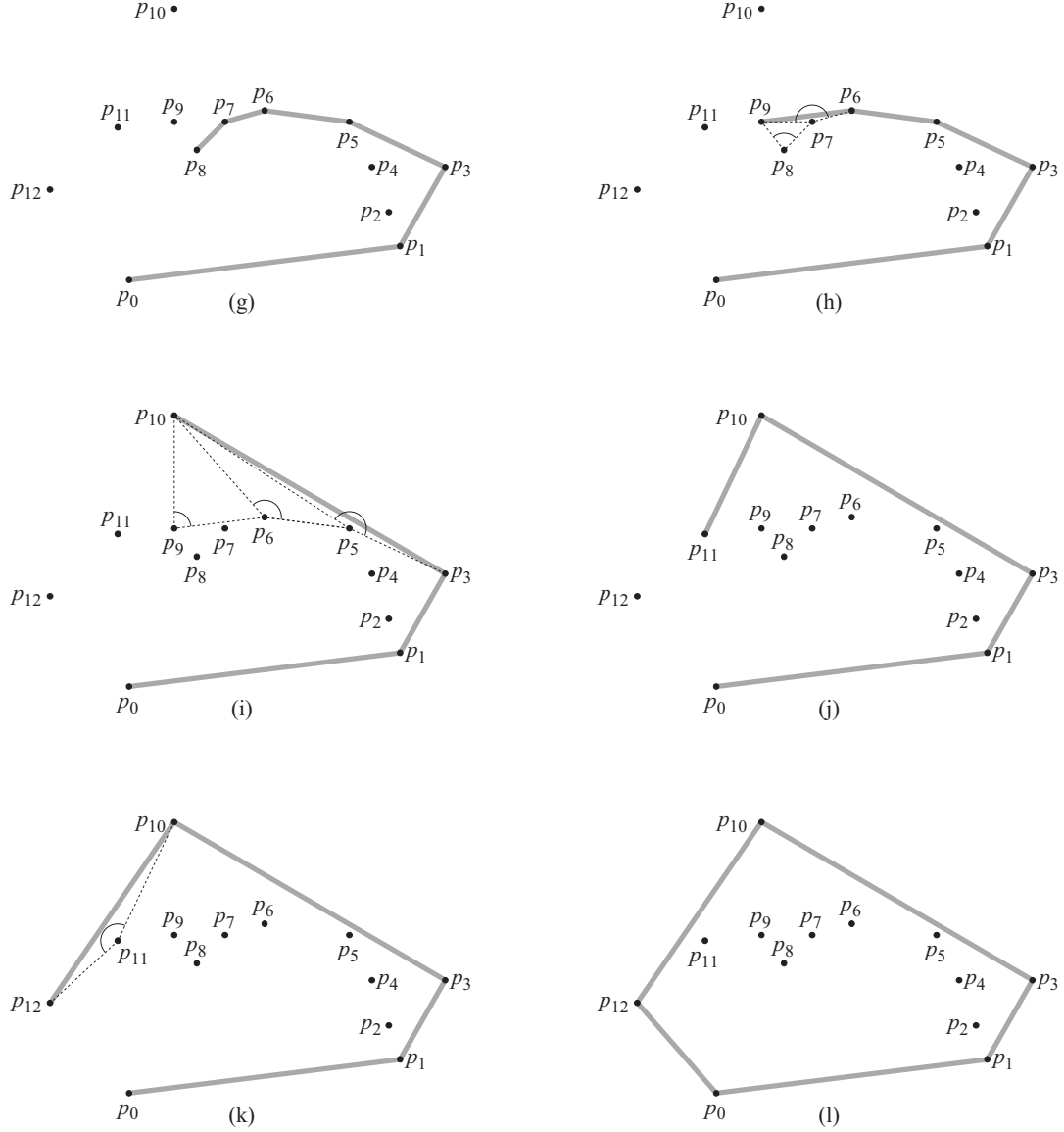
GRAHAM-SCAN(Q)

- 1 p_0 , Q içinde en küçük y -koordinatlı bir nokta ya da bir bağlantı olması durumunda en sol nokta olsun
- 2 $\langle p_1, p_2, \dots, p_m \rangle$ Q içinde kalan noktalar olsun, bunlar p_0 etrafında kutupsal açılarına göre saat yönünün tersinde sıralıdır. (eğer birden çok aynı açıya sahipse p_0 a en uzak olan hariç tüm noktaları sil)
- 3 **if** $m < 2$
- 4 **return** “dışbükey örtü boş”
- 5 **else** S boş bir yığın olsun
- 6 PUSH(p_0, S)
- 7 PUSH(p_1, S)
- 8 PUSH(p_2, S)
- 9 **for** $i = 3$ **to** m
- 10 **while** NEXT-TO-TOP(S), TOP(S) noktaları tarafından oluşturulan açı ve p_i sola dönüş yapmaz
- 11 POP (S)
- 12 PUSH(p_i, S)
- 13 **return** S

Şekil 33.7, GRAHAM-SCAN in işleyişini gösteriyor. 1. satır en küçük y -koordinatlı noktayı p_0 noktası olarak seçer, bağlantı olması halinde bu noktanın en solunu alır. Q nun içinde, p_0 ın altında hiçbir nokta olmadığı için ve aynı y -koordinatlı başka herhangi nokta da sağında olmadığı için, p_0 muhakkak $CH(Q)$ nun bir köşesi olmalıdır. 2. satır Q nun geriye kalan noktalarını p_0 a göre kutupsal açı vasıtasıyla, aynı yöntemi kullanarak-vektörel çarpımları karşılaştırarak-Alıştırma 33.1-3 teki gibi sıralar. İki veya daha fazla nokta p_0 a göre olan aynı kutupsal açıya sahip ise, en uzak nokta hariç hepsi p_0 ın ve en uzak noktanın dışbükey kombinasyonlarıdır ve böylelikle bunların tamamını gözden çıkarıyoruz. Q içinde her noktanın radyanla ölçülü p_0 a göre kutupsal açısı $[0, p)$ yarı açık aralığındadır. Noktalar, kutupsal açılarına göre sıralandığından dolayı, bunlar p_0 a göre saat yönünün tersinde sıralanmıştır. Bu sıralanmış noktalar dizisini $\langle p_1, p_2, \dots, p_m \rangle$ şeklinde düzenleyelim. p_1 ve p_m nin, $CH(Q)$ un köşeleri olduğuna dikkat edin



Şekil 33.7 Şekil 33.6'nın Q kümesindeki GRAHAM-SCAN gösterimi. S yığınındaki mevcut dışbükey örtü her adımda gri renkle gösterilmiştir. **(a)** $\langle p_1, p_2, \dots, p_{12} \rangle$ noktalar dizisi, p_0 a göre olan bağlı olan kutupsal açıların artan sırasına göre numaralandırılmıştır ve başlangıç S yığını p_0, p_1 ve p_2 yi içermektedir. **(b)-(k)** 9-12 satırlarının **for** döngülerinin her bir iterasyonu için S yığını. Kesik çizgiler, noktaların yığından çıkmasına neden olan sola dönmeyen dönüşleri göstermektedir. **(h)** de, örneğin, $\angle p_6 p_7 p_9$ açısındaki sağ dönüş p_8 in çıkmasına ve $\angle p_7 p_8 p_9$ açısındaki sağ dönüş de p_7 nin çıkmasına neden olmaktadır.



Şekil 33.7 devamı (l) prosedür tarafından oluşturulan ve Şekil 33.6 ile eşleşen dışbükey örtü.

(bkz Alıştırma 33.3-1) Şekil 33.7 (a), Şekil 33.6 da ki noktaları p_0 a göre artan kutupsal açıların sırasına göre dizide numaralandırılmıştır.

Prosedürün geri kalanı S yığınının kullanır. 5-8. satırlar yığını, en alttan en üste, ilk üç nokta p_0, p_1 ve p_2 yi kapsatmak için başlatır. Şekil 33.7(a) S başlangıç yığınının gösteriyor. 9-12. satırların **for** döngüsü, $\langle p_3, p_4, \dots, p_m \rangle$ alt dizisindeki her bir nokta için yinelenir. p_i noktasının işleminden sonra S yığının, en alttan en üste saat yönünün tersinde sıralanmış $CH(\{p_0, p_1, \dots, p_i\})$ köşelerine sahip olduğunu göreceğiz. 10-11. satırların **while** döngüsü, dışbükey örtünün köşeleri olmayan noktalar bulursa, bu noktaları yığından siler. Dışbükey örtüyü saat yönünün tersine çevirdiğimizde, her köşede sol dönüş yapmamız gerekecek. Böylece, **while** döngüsü sol dönüş yapmadığımız bir köşe bulduğu an, köşeyi yığından çıkarmış oluruz (sağ dönüş yerine sol yapmayan dönüşü kontrol ederek sonuçta oluşan dışbükey çokgenin bir köşesinde bir dik açının olması ihtimalini engelleriz. Dik açı istemiyoruz, çünkü bir dışbükey çokgenin hiçbir köşesi çokgenin diğer köşelerinin bir çokgen kombinasyonu olmayabilir) Sol dönüşü olmayan köşeleri çıkardıktan sonra p_i ye doğru ilerlerken, p_i yi yığına iteriz. Şekil 33.7 (b)-(k) s yığınının **for** döngüsünün her bir iterasyonundan sonraki halini gösteriyor. En sonda, GRAHAM-SCAN 13. satırda S yığına döner. Şekil 33.7 (l) uygun dışbükey çokgeni gösteriyor.

Bir sonraki teorem GRAHAM-SCAN in doğruluğunu kanıtlıyor.

Teorem 33.1 (Graham Taramasının doğruluğu)

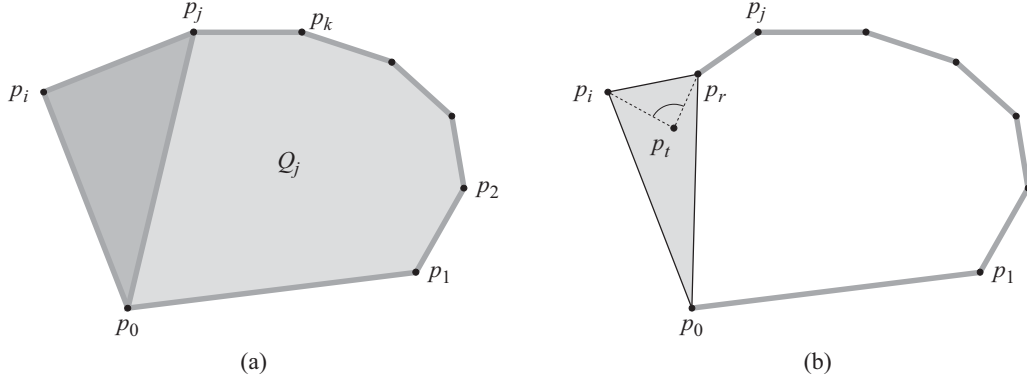
Graham taraması, $|Q| \geq 3$ olduğu, Q noktalar kümesinde işlem yaparsa, en sonunda, S yığını, en alttan en başa, kesinlikle $CH(Q)$ nun köşelerini içerir ve bunlar saat yönünün tersinde sıralıdır.

Kanıt 2. satırdan sonra elimizde $\langle p_1, p_2, \dots, p_m \rangle$ noktalar dizisi vardır. $i = 2, 3, \dots, m$ için $Q_i = \{p_0, p_1, \dots, p_i\}$ noktalarının alt kümesini tanımlayalım. $Q - Q_m$ nin içindeki noktalar silinen noktalardır çünkü bunlar Q_m deki bazı noktalar ile p_0 a göre olan aynı kutupsal açıya sahiptir; bu noktalar $CH(Q)$ içinde değildir, ve sonuçta $CH(Q_m) = CH(Q)$ dir. Bu durum GRAHAM-SCAN bittiğinde, en alttan en üste sıralandığında, S yığınının $CH(Q_m)$ nin köşelerini saat yönünün tersinde içerdiğini kanıtlamış olur. p_0, p_1 ve p_m , $CH(Q)$ nun köşeleri olduğu gibi p_0, p_1 ve p_i nin de $CH(Q_i)$ nin tüm köşeleri olduğuna dikkat edin.

Kanıt burada sıradaki döngü değişmezini kullanır:

9-12 . satırlarındaki **for** döngüsünün her iterasyonunun başlangıcında, S yığını, en alttan en üste, tamamen, saat yönünün tersinde sıralanmış olan $CH(Q_{i-1})$ 'in köşelerinden oluşur.

Başlangıç: 9. satırı işlediğimiz ilk an bu değişmez geçerli olur, çünkü o an, S yığını, tamamen, $Q_2 = Q_{i-1}$ in köşelerinden oluşur ve bu üç köşeli küme kendi



Şekil 33.8 Graham-Scan in doğruluğunun kanıtı. **(a)** p_0 a göre p_i nin kutupsal açısı p_j nin kutupsal açısından daha büyük olduğundan ve $\angle p_k p_j p_i$ açısı sola dönüş yaptığından, p_i yi $CH(Q_j)$ eklemek bize $CH(Q_j \cup \{p_i\})$ köşelerini tam tamına verir. **(b)** eğer $\angle p_r p_i p_j$ açısı sol olmayan dönüş yaparsa p_r , ya p_0 , p_r ve p_i nin oluşturduğu üçgenin iç bölgesindedir ya da üçgenin bir kenarının üzerindedir, bu, p_i nin $CH(Q_j)$ bir köşesi olamayacağı anlamına geliyor.

dışbükey örtüsünü oluşturur. Üstelik, bunlar en alttan en üste saat yönünün tersine sıralanırlar.

Yineleme: **for** döngüsünün bir iterasyonu çalıştığında, S yığnında en üstteki nokta p_{i-1} olur ki bu nokta bir önceki iterasyonda (ya da ilk iterasyondan önce, $i = 3$ olduğunda) en sonda idi. p_j , 10–11. satırları çalıştırdıktan sonra fakat 12. satırın p_i yi itmesinden önceki, S yığnında en üstteki nokta olsun ve p_k ’da S yığnında p_j nin hemen altındaki nokta olsun. p_j nin S de en üstte olduğu an ve henüz p_i yi itmemişken S yığını tam tamına **for** döngüsünün j iterasyonundan sonra sahip olduğu aynı noktaya sahiptir. Döngü değişmezi sayesinde, neticede, S , o anda tam tamına $CH(Q_j)$ nin köşelerini içerir ve en alttan en üste saat yönünün tersine sıralanmış görünürler.

p_i yi itmeden hemen önce şu önceki anın üzerinde odaklanmaya devam edelim. p_0 ’a göre p_i nin kutupsal açısının p_j nin kutupsal açısından daha büyük olduğunu ve $\angle p_k p_j p_i$ açısının sol dönüşü yaptığını (aksi halde p_j yi çıkarırdık) biliyoruz. Sonuç olarak S tamtamına $CH(Q)$ nun köşelerini içerdiğinden, Şekil 33.8 (a) dan şunu görürüz: p_i yi ittiğimizde, S yığını $CH(Q_j \cup \{p_i\})$ nin köşelerini tam tamına içerecektir yine en alttan en üste saat yönünün tersi durumuyla.

Şimdi $CH(Q_j \cup \{p_i\})$ nin $CH(Q_i)$ noktalar kümesi ile aynı olduğunu gösterebiliriz. **for** döngüsünün i iterasyonu sırasında çıkarttığımız herhangi bir p_t noktası düşünün, ve p_r de p_t çıkarıldığı an S yığnında p_t nin hemen altındaki nokta olsun (p_r , p_j de olabilir). $\angle p_r p_t p_i$ açısı sol olmayan bir dönüş yapar ve p_0 a göre p_t nin kutupsal açısı p_r nin kutupsal açısından daha büyüktür. Şekil 33.8

(b) nin de gösterdiği gibi p_i ya p_0 , p_r ve p_i nin oluşturduğu üçgenin iç bölgesindedir ya da bu üçgenin bir kenarı üzerindedir (ama kesinlikle köşesinde değil). Açık bir biçimde, p_r , Q_i nin diğer üç noktasının oluşturduğu üçgenin içinde olduğundan, $CH(Q)$ nun bir köşesi olamaz. p_r , $CH(Q)$ nun bir köşesi olmadığından da

$$CH(Q_i - \{p_r\}) = CH(Q_i) \text{ dir.} \quad (33.1)$$

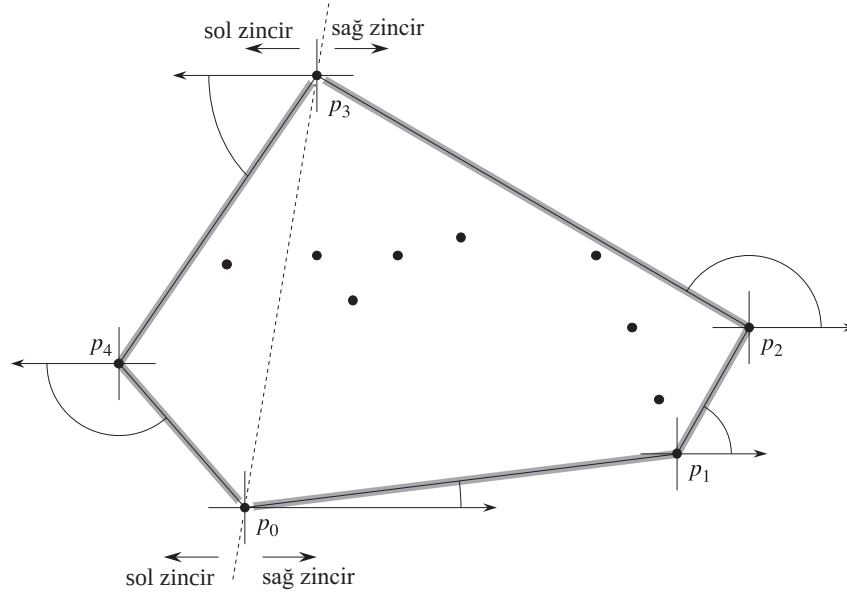
p_r , **for** döngüsünün i iterasyonu sırasında çıkartılan noktaların kümesi olsun. (33.1) denklemi p_i nin içindeki tüm noktalara uygulandığından, bu uygulamayı tekrar ederek $CH(Q_i - P_i) = CH(Q_i)$ olduğunu gösterebiliriz. Fakat $Q_i - P_i = Q_j \cup \{p_i\}$ dir ve sonuç olarak $CH(Q_j \cup \{p_i\}) = CH(Q_i - P_i) = CH(Q_i)$ yi elde ederiz.

p_i yi bir kez ittiğimizde, S yığınının, tam tamına $CH(Q)$ nun köşelerini, en alttan en üste saat yönünün tersinde sıralanmış olarak içerdiğini göstermiş olduk. i yi arttırarak değişmez döngünün bir sonraki iterasyon için geçerli olmasına neden oluruz.

Sonlandırma: Döngü bittiğinde $i = m + 1$ yi elde ederiz ve böylece değişmez döngü, S yığınının $CH(Q_m)$ nun bir başka deyişle $CH(Q)$ nun köşelerini en alttan en üste saat yönünün tersinde sıralanmış olarak içerdiğini söyler. Böylece kanıtımız tamamlanmış olur. ■

Şimdi, GRAHAM-SCAN nin çalışma süresinin, $n = |Q|$ iken, $O(n \lg n)$ olduğunu gösterelim. 1. Satır $\Theta(n)$ zaman sürer. 2. satır, birleştirerek sıralama ya da bellek yığını sıralaması kullanarak kutupsal açıları sıralayıp ve Kısım 33.1 deki vektörel çarpım yöntemini kullanıp açıları kıyaslayarak, $O(n \lg n)$ zaman sürer. (Tüm noktalar içinde, toplam $O(n)$ zamanda aynı kutupsal açılı en uzak hariç tüm noktaları silebiliriz) 5-8. satırlar $O(1)$ zaman sürer. $m \leq n - 1$ olduğundan 9-12. satırların **for** döngüsünün işlem süresi en çok $n - 3$ zaman sürer. PUSH, $O(1)$ zaman sürdüğünden her bir iterasyon 10-11. Satırların **while** döngüsü için harcanan zaman hariç $O(1)$ zaman sürer ve böylece tüm **for** döngüsü iç içe **while** döngüsü hariç $O(n)$ zaman sürmüştür.

while döngüsünün toplamda $O(n)$ zaman sürdüğünü göstermek için toplam analizi kullanacağız. $i = 0, 1, \dots, m$ için biz her bir p_i yi S yığına tam olarak bir kez itiyoruz. Kısım 17.1 in MULTIPOP prosedürünün analizinde olduğu gibi en fazla ittiğimiz nesnelerin sayısı kadar çıkarma yapabildiğimizi biliyoruz. En az üç nokta— p_0 , p_1 ve p_m —yığından hiç çıkmadıkları için aslında toplam $m - 2$ POP işlemi gerçekleşir. **while** döngüsünün her bir iterasyonu bir POP işlemi yapar ve böylece **while** döngüsü en fazla $m - 2$ işlem yapmış olur. 10. satırdaki test $O(1)$ zaman sürdüğünden, her bir POP $O(1)$ zaman sürer ve $m \leq n - 1$, **while** döngüsü toplam $O(n)$ zaman sürmüştür. Böylelikle, GRAHAM-SCAN in çalışma süresi $O(n \lg n)$ olmuş olur.



Şekil 33.9 Jarvi's March işlemi. En alttaki p_0 noktasını ilk köşe olarak seçiyoruz. Bir sonraki p_1 köşesi p_0 a göre en küçük kutupsal açıya sahip olan noktadır. Sonra p_2 de, p_1 göre en küçük kutupsal açıya sahip olan nokta olur. Sağ zincir en yüksek p_3 noktasına kadar gider. Daha sonra, sol zinciri de negatif x -eksenine göre en küçük kutupsal açıları kullanarak inşa ederiz.

Jarvis marşı

Jarvis marşı bir Q noktalar kümesinin dışbükey örtüsünü **paket sarma (package wrapping)** ya da **hediye sarma (gift wrapping)** tekniği ile hesaplar. h , $CH(Q)$ nun köşeleri iken, algoritma $O(nh)$ zaman da çalışır h , $o(\lg n)$ olursa, Jarvi Marşı, Graham Taramasından asimptotik olarak daha hızlı olur.

Jarvis marşını gergin bir kağıdın Q kümesini sarması olarak düşünebiliriz. Kağıdın ucunu kümedeki en alttaki nokta olan ve Graham Taramasına başlarken ki aynı noktaya ilişitirerek başlıyoruz. Bu noktanın dışbükey çokgenin bir köşesi olması gerektiğini biliyoruz. Kağıdı gergin olması için sağa doğru çekiyoruz, ve sonra da yukarıya doğru bir noktaya ulaşana kadar çekiyoruz. Bu nokta da dışbükey çokgenin bir köşesi olması gerekmektedir. Kağıdı gergin tutarak, bu şekilde kümeyi dolaşarak başladığımız p_0 noktasına geri geliyoruz.

Daha bilimsel olarak, Jarvis Marşı, $CH(Q)$ nun köşelerini $H = \langle p_0, p_1, \dots, p_{h-1} \rangle$ dizisi olarak inşa eder. p_0 ile başlarız. Şekil 33.9 gösterdiği gibi dışbükey çokgenin bir sonraki p_1 köşesi p_0 a göre en küçük kutupsal açıya sahip olan noktadır. (çakışma durumunda p_0 a en uzak noktayı seçiyoruz).

Benzer biçimde p_2 de p_0 a göre en küçük kutupsal açıya sahiptir ve böyle devam eder. En yüksek köşeye ulaştığımızda, bu noktaya p_k diyelim, (yine çakışma durumunda köşeye en uzak noktayı seçerek), Şekil 33.9 un da gösterdiği gibi $CH(Q)$ un **sağ zincirini** inşa ederiz. **Sol zinciri** inşa etmek için de, p_k noktasından başlarız, p_k ya göre *negatif x -ekseninden alınan* en küçük kutupsal açılı noktayı p_{k+1} olarak seçeriz. Bu şekilde kutupsal açıları negatif x -ekseninden alıp, başladığımız p_0 köşesine gelene kadar devam edip sol zinciri inşa ederiz.

Jarvis marşı, dışbükey çokgenin etrafında bir kavramsal tarama ayrık sağ ve sol zincir olmadan uygulayabiliriz. Bu tür uygulamalar genellikle, seçilen en son dışbükey-örtü yüzünün açısını takip eder ve örtü yüzlerinin açılarının dizisinin kesinlikle artan olmasını (0 dan 2π radyana kadar) gerekli kılar. Ayrık zincirleri inşa etmenin avantajı, açıları hesaplamaya bariz bir şekilde ihtiyaç duymadığımızdır, Kısım 33.1 deki teknikler açıları karşılaştırmak için yeterlidir.

Nizami olarak uygulandığında, Jarvis marşı, $O(nh)$ çalışma zamanına sahiptir. $CH(Q)$ un h köşelerinden her biri için en küçük kutupsal açılı köşeyi buluruz. Kutupsal açıların mukayesesi, Kısım 33.1 deki teknikleri kullanarak, $O(1)$ zaman sürer. Kısım 9.1 gösterdiği gibi, her bir mukayese $O(1)$ zaman sürerse n değerini $O(n)$ zamanda hesaplayabiliriz. Böylelikle Jarvis marşı, $O(nh)$ zaman sürer.

Alıştırma

33.3-1

GRAHAM-SCAN prosedüründe p_1 ve p_m nin $CH(Q)$ nun köşeleri olması gerektiğini kanıtlayınız.

33.3-2

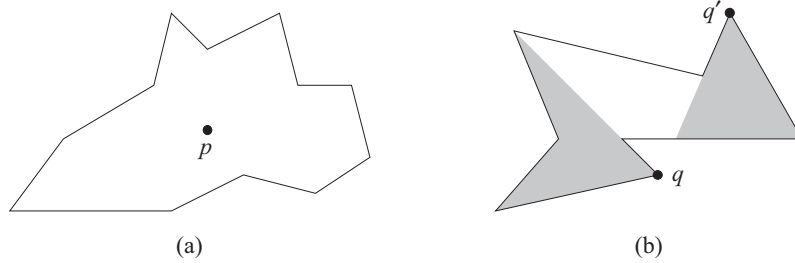
Toplama, karşılaştırma ve çarpmayı destekleyen ve n sayıyı sıralamak için bir W ($n \lg n$) alt sınırının olduğu bir bilgisayar modeli düşünün. Böyle bir modelde, n noktalı bir kümenin köşelerini sırayla hesaplamak için bir $\Omega(n \lg n)$ nin alt sınırı olması gerektiğini kanıtlayınız.

33.3-3

Q noktalar kümesi verilsin. Birbirinden en uzak noktaların oluşturduğu çiftin $CH(Q)$ nun köşeleri olması gerektiğini ispat edin.

33.3-4

Bir P çokgeni ve onun sınırında bir q noktası verilsin, q nun **gölgesi** r noktalarından oluşan bir kümedir öyle ki $\overline{qr}$ doğru parçası tamamen P nin iç bölgesinde veya sınırında olsun. Şekil 33.10 un gösterdiği gibi, P nin iç bölgesinde yani P nin sınırındaki her noktanın gölgesi içinde bir p noktası oluşursa P çokgenine **yıldız-şekilli** denir. Bu tür p noktalar kümesine P nin **çekirdeği** denir. Verilen



Şekil 33.10 Alıştırma 33.3-4 te kullanılan bir yıldız-şekilli çokgen tanımı. **(a)** Bir yıldız-şekilli çokgen. p den sınır üzerindeki herhangi bir noktaya çizilen doğru parçası sınırı yalnız q da kesiyor. **(b)** yıldız-şekilli olmayan çokgen. Soldaki taralı alan q nun gölgesi ve sağdaki taralı alan ise q' nün gölgesi. Bu bölgeler ayrık olduğundan, çekirdek boş kümedir.

n -köşeli, saat yönünün tersine sıralı köşeleri ile belirlenen P yıldız-şekilli çokgeni için $CH(P)$ nin $O(n)$ zamanda nasıl hesaplanacağını gösterin.

33.3-5

Doğru-üzerinde dışbükey-örtü problemi (On-line convex-hull problem)nde n noktalı Q kümesinden bize her bir seferde bir nokta veriliyor. Her noktayı aldıktan sonra şimdiye kadar görünen noktaların dışbükey örtüsünü hesaplıyoruz. Açık bir şekilde, her nokta için Graham taraması bir kez çalıştırdığımızda, toplam çalışma süresi $O(n^2 \lg n)$ olur. Çevrimiçi dışbükey örtü probleminin toplam $O(n^2)$ zamanda nasıl çözüleceğini gösterin.

33.3-6 *

n noktanın dışbükey örtüsünü $O(n \lg n)$ zamanda hesaplayan artan yöntemin nasıl uygulandığını gösterin.

33.4 En yakın nokta çiftini bulma

Şimdi, $n \geq 2$ noktanın olduğu Q kümesinde birbirine en yakın nokta çiftini bulma problemini düşünelim. ‘En yakın’ demek bildiğimiz Öklid uzaklığıdır : $p_1 = (x_1, y_1)$ ve $p_2 = (x_2, y_2)$ noktaları arasındaki uzaklık $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$ dir. Q kümesinde iki nokta çakışık olabilir o zaman aralarındaki uzaklık 0 olmuş olur. Bu problemin uygulamalarını, örneğin, trafik-kontrol sistemlerinde görebiliriz. Hava ve deniz trafiğinin kontrol sistemi iki en yakın aracın muhtemel çarpışmaları tespit etmek için gereklidir.

Brute-force closest-pair (kaba-kuvvet en yakın-çift) algoritması tüm $\binom{n}{2} = \Theta(n^2)$ noktalarını basitçe inceleyebilir. Bu bölümde bu problem için çalışma süresi tanıdık özyineleme $T(n) = 2T(n/2) + O(n)$ ile tanımlanan, böl-ve-

yönet algoritmasını anlatacağız. Bu algoritmanın çalıştığı süre sadece $O(n \lg n)$ zamandır.

Böl-ve-yönet algoritması

Algoritmanın her özyineleme yürütmesi, girdi olarak $P \subseteq Q$ alt kümesini ve P alt kümesinin tüm noktalarını içeren X ve Y dizilerini alır. X dizisinin noktaları sıralı olduğundan x -koordinatları monoton artandır. Benzer şekilde, Y dizisi de monoton artan y -koordinatlarına göre sıralıdır. $O(n \lg n)$ zaman sınırına ulaşmak için her özyineleme çağrısında sıralamaya yapamayacağımıza dikkat edin, eğer böyle yapsaydık, çalışma zamanı için özyineleme $T(n) = 2T(n/2) + O(n \lg n)$ olurdu, bunun da çözümü $T(n) = O(n \lg^2 n)$ dir. (Alıştırma 4.6-2 deki ana yöntemin bir versiyonunu kullanarak bu sonucu elde edebilirsiniz). İlerleyen sayfalarda, bu sıralama özelliğini, her bir özyineleme çağrısında sıralamadan, gerçekleştiren presorting (önsıralama) i nasıl kullanacağımıza dair bir konu göreceğiz.

P , X ve Y girdileriyle, verilen bir özyineleme yürütmesi ilk olarak $|P| \leq 3$ olup olmadığını kontrol eder. Eğer öyleyse, yürütme, yukarda bahsedilen sadece kaba kuvvet yöntemini kullanır: noktaların tüm $\binom{|P|}{2}$ çiftlerine bak ve sonra en yakın çifte döner. Şayet $|P| > 3$ ise özyineleme yürütme böl-ve-yönet paradigmasını aşağıdaki gibi icra eder.

Bölme(divide): P noktalar kümesini, P_L ve P_R adlı iki kümeye bölen dikey bir doğru l bul öyle ki $|P_L| = \lceil |P|/2 \rceil$, $|P_R| = \lfloor |P|/2 \rfloor$ olsun ve P_L içindeki tüm noktalar Doğru l nin üzerinde ya da solunda olsun, ve P_R içindeki tüm noktalar da Doğru l nin sağında olsun. X dizisini sırayla P_L ve P_R nin noktalarını kapsayan ve monoton artan x -koordinatına göre sıralı olan X_L ve X_R dizilerine böl. Benzer biçimde, Y dizisini de sırayla P_L ve P_R nin noktalarını kapsayan ve monoton artan y -koordinatına göre sıralı olan Y_L ve Y_R dizilerine böl.

Yönet(Conquer): P yi P_L ve P_R ye bölerek, iki özyineleme çağrısı yap, bunlardan bir tanesi P_L 'deki birbirine en yakın nokta çiftini diğeri de P_R deki birbirine en yakın nokta çiftini bulsun. İlk çağrının girdileri; P_L alt kümesi ve X_L ve Y_L dizileri; ikinci çağrının girdileri de P_R, X_R ve Y_R dir. P_L ve P_R için oluşan en yakın-çift uzaklıkları sırasıyla δ_L ve δ_R ile beraber $\delta = \min(\delta_L, \delta_R)$ olsun.

Birleştir (combine): En yakın çift, özyineleme çağrılarından biriyle bulunan ve uzaklığı δ olan çifttir ya da bir noktası P_L içinde diğeri noktası P_R içinde olan bir çifttir. Algoritma, bir noktası P_L içinde diğeri noktası P_R içinde olup uzaklığı δ en küçük bir çiftin olup olmadığını tespit eder. Eğer δ den küçük uzaklığa sahip bir çift varsa, iki noktanın da Doğru l nin δ birim civarında (δ birimden daha küçük) olmaları gerekir. Sonuç olarak, Şekil 33.11(a) da olduğu gibi, bunların ikisi de Doğru l merkezli, 2δ genişliğinde dikey bir şeridin içinde olmaları gerekir. Bu çifti bulmak için, eğer varsa, aşağıdaki işlemleri yapıyoruz:

1. Bir Y' dizisi oluştur, bu dizi 2δ -genişliğindeki dikey şeritteki Y dizisinin noktalarıdır. Y' dizisi de tıpkı Y gibi y -koordinatına göre sıralıdır.
2. Y' deki her p noktası için Y' da olup p nin δ birim etrafındaki noktaları bulmaya çalışın. Birazdan da göreceğimiz gibi Y' de p yi izleyen hesap etmeye değer yalnız 7 nokta var. p noktasından bu 7 noktanın her birine olan uzaklıklarını hesap edelim ve Y' noktalarının oluşturduğu tüm çiftler arasındaki δ uzaklığındaki en yakın çiftin yolunu saklayalım.
3. Eğer $\delta' < \delta$ ise o zaman dikey şerit, özyinelemeli çağrılarının bulunduğu çiftlerden başka bir çift içermesine ihtiyaç yoktur. Bu çiftte dön ve bunun uzaklığı δ' dür. Aksi halde, özyinelemeli çağrılarının bulunduğu ve uzaklığı δ olan en yakın çiftte döneriz.

Yukarıdaki tanım $O(n \lg n)$ çalışma süresini elde etmek için önemli olan bazı uygulama detaylarını göstermiyor. Algoritmanın doğruluğunu kanıtladıktan sonra, istenilen zaman sınırına ulaşmak için algoritmanın nasıl uygulandığını göstereceğiz.

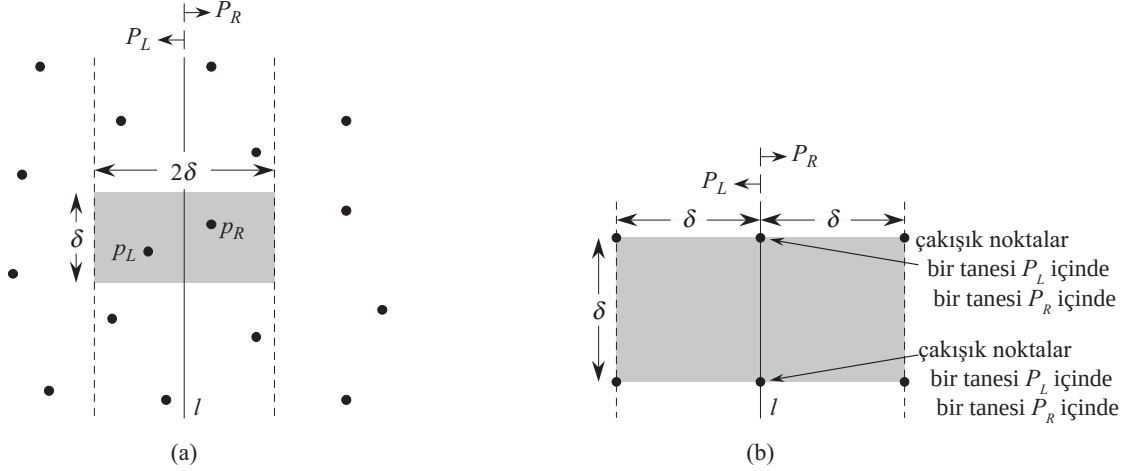
Doğruluk

İki durum hariç en yakın-çift-algoritmasının doğruluğu açıktır. Bunlardan ilki, özyinelemenin en sonunda $|P| \leq 3$ olduğunda, biz sadece bir noktayı içeren bir problemi çözmeye çalışmayacağımıza eminiz. İkinci durum ise bizim ihtiyacımız olan tek şey Y' dizisi içindeki her p yi takip eden 7 noktayı kontrol etmek. Şimdi bu durumu kanıtlayalım.

kanıtlayalım herhangi bir evresinde en yakın çiftin $p_L \in P_L$ ve $p_R \in P_R$ olduğunu farz edin. Böylelikle, p_L ve p_R arasındaki δ' uzaklığı açıkça δ den küçük olur. p_L noktası Doğru l nin üzerinde ya da solunda olması ve δ birim uzaklıktan az uzakta olması gerekir. Benzer biçimde, p_R de Doğru l nin üzerinde ya da sağında olması ve δ birim uzaklıktan küçük olması gerekir. Üstelik, p_L ve p_R dikey olarak birbirlerine δ birimin içinde yakınlıktalar. Şekil 33.11 (a) gösterdiği gibi, p_L ve p_R , doğru l merkezli, $\delta \times 2\delta$ boyutlu bir dikdörtgenin içindeler (Bu dikdörtgen içinde başka noktalar da olabilir).

Şimdi göstereceğimiz şey ise P nin en çok 8 noktası bu $\delta \times 2\delta$ dikdörtgenin içinde yer alabilmesidir. Bu dikdörtgenin sol yarısı olan $\delta \times \delta$ karesini düşünün. P_L nin içindeki tüm noktalar en az δ birim uzakta oldukları için en fazla 4 nokta bu karenin içinde olabilir; Şekil 33.11 (b) nasıl olduğunu gösteriyor. Benzer biçimde, P_R nin içindeki en fazla 4 nokta, dikdörtgenin sağ yarısı olan $\delta \times \delta$ karesinin içinde olabilir. Böylelikle P nin en fazla 8 noktası $\delta \times 2\delta$ dikdörtgenin içinde olabilir. (Doğru l nin üzerindeki noktalar ya P_L nin ya da P_R nin içinde olduklarından, doğru üzerinde en çok 4 nokta vardır. Bu koşul, iki çakışık nokta çiftinin olduğu iki çiftten birinin Doğru l nin üzerinde dikdörtgenin en üstünde, diğerinin Doğru l nin üzerinde dikdörtgenin en altında olursa gerçekleşmiş olur)

P nin en çok 8 noktasının dikdörtgende olabileceğini gördükten sonra neden sadece Y' nün içindeki her noktayı takip eden 7 noktanın kontrolüne ihtiyacımız



Şekil 33.11 En yakın-çift algoritmasının yalnızca Y' dizisi içindeki her noktayı takip eden 7 noktanın kontrolüne gerek olduğunu kanıtındaki anahtar görseller. **(a)** $p_L \in P_L$ ve $p_R \in P_R$ birbirinden δ birim uzakta iseler bunlar Doğru l merkezli $\delta \times 2\delta$ dikdörtgeninin içinde olmaları gerekir. **(b)** Aralarındaki mesafe en δ birim olan ve çiftlerden oluşan 4 noktanın hepsi nasıl $\delta \times \delta$ karesi içinde yer alır. Sol tarafta 4 nokta p_L nin içinde ve sağda 4 nokta p_R nin içindedir. Doğru l nin üzerinde olan noktalar çakışan nokta çiftlerinden biri ve bir nokta p_L de diğer nokta p_R de ise, $\delta \times 2\delta$ dikdörtgeni 8 nokta kapsayabilir.

olduğunu kolaylıkla görürüz. Yine en yakın çiftin p_L ve p_R olduğunu farz ederek ve yine genelleme yapmadan Y' dizisi içinde P_L nin P_R den önce olduğunu farz edelim. O zaman, P_L , Y' dizisi içinde mümkün olduğunca en erken ve P_R de mümkün olduğunca en geç oluşsa bile P_R , P_L yi takip eden 7 konumdan biridir. Böylece, en yakın-çift algoritmasının doğruluğunu kanıtlamış olduk.

Uygulama ve çalışma süresi

Daha önce söylediğimiz gibi, hedefimiz, n noktalı bir kümenin süresi $T(n)$ iken özyinelemenin çalışma süresinin $T(n) = 2T(n/2) + O(n)$ olmasıdır. Temel zorluk, özyinelemeli çağrılara giden X_L , X_R , Y_L ve Y_R dizilerinin uygun bir koordinata göre sıralandığını ve ayrıca Y' nün de y -koordinatına göre sıralandığını garantiye almaktır (Özyinelemeli çağrı tarafından alınan X dizisi hazırda sıralanmışsa o zaman P kümesini kolayca P_L ve P_R ye doğrusal bir zamanda bölebildiğimize dikkat edin).

Burada kilit nokta, her bir çağrıda, sıralı bir dizisinin bir sıralı alt kümesini oluşturmak istememizdir. Örneğin, belirli bir yürütme, P alt kümesi ve Y dizisini, y -koordinatına göre sıralı bir biçimde alır. P yi P_L ve P_R ye ayırdıktan sonra, y -koordinatına göre sıralı Y_L ve Y_R dizilerini doğrusal zamanda oluşturmak gerekir. Biz yönteme Kısım 2.3.1 de birleştirerek sıralama sayesinde MERGE prosedürü-

nün tersi gibi bakabiliriz: sıralı bir diziyi iki sıralı diziyeye ayırıyoruz. Aşağıdaki kabakod bize bu fikri veriyor:

```

1  diziyeye  $Y_L[1..Y.length]$  ve  $Y_R[1..Y.length]$  yeni diziler olsun
2   $Y_L.length = Y_R.length = 0$ 
3  for  $i = 1$  to  $Y.length$ 
4      if  $Y[i] \in P_L$ 
5           $Y_L.length = Y_L.length + 1$ 
6           $Y_L[Y_L.length] = Y[i]$ 
7      else  $Y_R.length = Y_R.length + 1$ 
8           $Y_R[Y_R.length] = Y[i]$ 

```

Y dizisindeki noktaları basit bir şekilde sırayla inceliyoruz. Eğer bir $Y[i]$ noktası P_L içinde ise biz onu Y_L dizisinin en sonuna ilave ediyoruz ; aksi halde onu Y_R dizisinin en sonuna iliştiyoruz. Benzer kabakod X_L, X_R ve Y' dizilerini oluşturmak için işlem yapar.

Geriye bir soru kalıyor: ilk bölgedeki noktalar sıralı olarak nasıl elde edilir? Onları **önsıralıyoruz**; yani, onları bir kez ve ilk özyineleme çağrısından önce tümü için sıralıyoruz. Bu sıralı dizileri ilk özyineleme çağrısının içine atıyoruz ve oradan onları gerekirse özyineleme çağrılarının arasından aşağı atıyoruz. Önsıralama, çalışma zamanına $O(n \lg n)$ terimini ilave ediyor, fakat şu anda özyinelemenin her bir adımı özyinelemeli çağrılar hariç doğrusal zaman sürer. Sonuç olarak, $T(n)$ her bir özyinelemeli adımın çalışma süresi ve $T'(n)$ de tüm algoritmanın çalışma süresi olursa

$$T'(n) = T(n) + O(n \lg n) \text{ ve}$$

$$T(n) = \begin{cases} 2T(n/2) + O(n), & n > 3 \text{ ise,} \\ O(1), & n \leq 3 \text{ ise.} \end{cases}$$

Böylece $T(n) = O(n \lg n)$ ve $T'(n) = O(n \lg n)$ olur.

Alıştırmalar

33.4-1

Profesör Williams Y' dizisindeki sıradaki her nokta için yalnızca 5 noktayı kontrol edilmesine imkan veren bir en yakın-çift algoritma şeması ile çıkageldi. Fikir şu: Doğru l nin üzerindeki noktaları her zaman P_L kümesine yerleştir. Böylece, Doğru l üzerinde bir tanesi P_L de diğeri P_R de olan hiçbir çakışık nokta çifti bulunamaz. Sonuçta, $\delta \times 2\delta$ dikdörtgeninde en fazla 6 nokta yer alabilir. Profesör'ün şemasındaki hata nedir?

33.4-2

Y' dizisindeki sıradaki her nokta için dizi içindeki 5 yerdeki noktayı kontrol etmenin yeterli olacağını gösterin.

33.4-3

İki nokta arasındaki uzaklığı Öklid'den başka yollarla da tanımlayabiliriz. Düzlemde, p_1 ve p_2 noktaları arasındaki L_m -uzaklığı $(|x_1 - x_2|^m + |y_1 - y_2|^m)^{1/m}$ ile veriliyor. Öklid uzaklık tabii ki L_2 -uzaklıktır. **Manhattan uzaklığı** diye de bilinen L_1 uzaklığını kullanarak en yakın-çift algoritmasında değişiklik yapın.

33.4-4

Düzlemde p_1 ve p_2 noktaları veriliyor, aralarındaki L_∞ -uzaklık $\max(|x_1 - x_2|, |y_1 - y_2|)$ ile veriliyor. L_∞ -uzaklığı kullanmak için en yakın-çift algoritmasında değişiklik yapın.

33.4-5

En yakın-çift algoritmasına verilen noktaların $\Omega(n)$ i eş-dikey olsun. P_L ve P_R kümelerini nasıl belirlersiniz ve Y nin her noktasının P_L de mi P_R de mi olduğunu nasıl belirlersiniz öyle ki en yakın-çift algoritmasının çalışma zamanı $O(n \lg n)$ olarak kalsın.

33.4-6

Y dizisine ön-sıralama yapmadan fakat çalışma zamanı $O(n \lg n)$ olacak biçimde closest pair algoritmasında bir değişiklik önerin (*İpucu*: Sıralı, Y_L ve Y_R yi, Y sıralı dizisini oluşturmak için birleştirin).

Problemler
33-1 Dışbükey Katmanlar

Düzlemde Q noktalar kümesi veriliyor, Q nun **dışbükey katmanlarını** tümevarımla açıklayacağız. Q nun ilk dışbükey katmanı, $CH(Q)$ nun köşeleri olan Q daki noktalardan oluşur. $i > 1$ için Q_i yi şöyle tanımlayın: $1, 2, \dots, i-1$ dışbükey katmanlarındaki silinen tüm noktalardan geriye kalan Q daki noktalardan oluşur. O halde Q nun i . dışbükey katmanı eğer $Q_i \neq \emptyset$ ise $CH(Q_i)$ dir aksi halde tanımlanamaz.

- a. n noktalı bir kümenin dışbükey katmanlarını oluşturan $O(n^2)$ bir algoritma yazın.
- b. n reel sayıyı sıralamak için $\Omega(n \lg n)$ zaman gerektiren herhangi bir bilgisayar modeliyle n noktalı bir kümenin dışbükey katmanlarını hesaplamak için $\Omega(n \lg n)$ zaman gerektiğini kanıtlayınız.

33.2 Maksimal katmanlar

Q , düzlemde n noktalı bir küme olsun. Eğer $x \geq x'$ ve $y \geq y'$ ise (x, y) noktası (x', y') noktasını **domine eder**. Q daki diğer noktalardan tarafından domine edilmeyen Q daki her noktaya **maksimal** denir. Q nun aşağıdaki gibi- **maksimal katmanlar** içinde düzenlenebilir bir çok maksimal nokta ihtiva edebileceğine dikkat edin. İlk maksimal L_1 katmanı Q nun maksimal noktalar kümesidir. $i > 1$ için i . L_i maksimal katmanı, $Q - \bigcup_{j=1}^{i-1} L_j$ de maksimal noktalar kümesidir.

Q nun boş olmayan k kadar maksimal katmanı olduğunu ve $i = 1, 2, \dots, k$ olduğunda y_i nin de, L_i içindeki en-sol noktanın y -koordinatı olduğunu farz edin. Şimdi de, Q nun içindeki hiçbir iki noktanın aynı x - ya da y -koordinatına sahip olmadığını farz edin.

a. $y_1 > y_2 > \dots > y_k$ olduğunu gösterin.

Q nun içindeki herhangi bir noktanın solunda olan bir (x, y) noktasını ve y nin de Q daki herhangi bir noktanın y -koordinatından farklı olduğunu düşünün. $Q' = Q \cup \{(x, y)\}$ olsun

b. $j = k + 1$ olduğu durumda, $y < y_k$ olmadıkça, j de $y_j < y$ yi sağlayan minimum indis olsun. Q' nün maksimal katmanlarının aşağıda gösterildiği gibi olduğunu kanıt edin:

- Eğer $j \leq k$ ise, o zaman Q' nün maksimal katmanları Q un maksimal katmanlarıyla- L_j nin ayrıca (x, y) yi yeni en-sol noktası olarak ihtiva ettiği durum hariç-aynıdır.
- Eğer $j = k + 1$ ise, Q' nün ilk k katmanı Q nun kilerle aynıdır, fakat bununla birlikte, Q' boş olmayan $k + 1$. maksimal katmanına, $L_{k+1} = \{(x, y)\}$ ye sahiptir.

c. n noktalı bir Q kümesinin maksimal katmanlarını hesaplayan bir $O(n \lg n)$ zamanlı bir algoritma tanımlayın (*İpucu*: Bir tarama doğrusunu sağdan sola doğru hareket ettirin).

d. Girdi noktalarının aynı x - ya da y -koordinatlı olmasına izin verdiğimizde bir problem çıkar mı? Bu tür problemleri çözmek için bir yol önerin.

33-3 Hayalet Avcıları ve Hayaletler

n hayalet avcısının oluşturduğu bir grup n hayaletle savaşıyor. Her hayalet avcısı hayalete bir akım ateşlediğinde onu yok eden bir proton çantası taşıyor. Akım düz bir çizgide gidiyor ve hayalete çarptığında sona eriyor. Hayalet avcıları aşağıdaki gibi bir strateji belirliyor: Herkes bir hayaletle eşleşiyor, n tane hayalet avcısı-hayalet çifti oluşuyor, ve sonra aynı anda her bir hayalet avcısı seçilen hayalete ateş ediyor. Akımların birbiriyle çarpışmasının çok tehlikeli olduğunu bildiğimizden, hayalet avcıları hiçbir akımın çarpışmayacağı şekilde kendilerine hayalet seçmeleri gerekiyor.

Her bir hayalet avcısının ve hayaletin konumunun düzlemde sabit bir nokta olduklarını ve hiçbir üç noktanın da doğrusal olmadığını farz edin.

- a. Bir hayalet avcısı ve bir hayaletten geçen bir doğru olduğunu ve doğrunun bir yanındaki hayalet avcı sayısının aynı yandaki hayalet sayısına eşit olduğunu tartışın. Böyle bir doğruyu $O(n \lg n)$ zaman içinde nasıl bulunacağını ifade edin
- b. Hiç bir akımın çarpışmayacağı hayalet avcısı hayalet çiftleri oluşturmak için $O(n^2 \lg n)$ zamanlı bir algoritma yazın.

33-4 Çubukları toplama

Profesör Charon'un, hepsi belli bir düzende yığılmış n çubuktan oluşan bir kümesi var. Her bir çubuk uç noktalarıyla tanımlı ve her bir çubuk koordinatı (x, y, z) olan bir sıralı üçlü. Hiçbir çubuk dik değil. Profesör çubukları bir anda toplamak istiyor fakat toplarken topladığı hiçbir çubuğun üzerinde başka bir çubuk olmamalı.

- a. Alınan iki a ve b çubuğundan, a 'nın yukarda mı aşağıda mı olduğunu ya da b ile temasının olup olmadığını bildiren bir prosedür yazın.
- b. Tüm çubukları toplamamanın mümkün olup olmadığını belirleyen ve eğer toplanıyorsa, onları toplarken doğru sırayla toplayacak etkili bir algoritma yazın.

33-5 Seyrek-örtülü (sparse-hulled) dağılımlar

Düzlemde bilinen bir rassal dağılımla çizilmiş noktalar kümesinin oluşturduğu dışbükey örtüyü hesaplayan problemi düşünün. Bazen, böyle bir dağılımla çizilmiş n noktanın dışbükey örtüsünün noktalarının sayısı, ya da boyutunun, bazı $\epsilon > 0$ sabiti için $O(n^{1-\epsilon})$ olması beklenir. Bu tür dağılıma **seyrek-örtülü** deriz. Seyrek-örtülü dağılım aşağıdaki şartlara sahiptir.

- Noktalar homojen bir biçimde bir birim yarıçaplı bir daire içinde çizilmiştir. Dışbükey örtünün boyutu $\Theta(n^{1/3})$ olması beklenir.
 - Noktalar herhangi bir k sabiti için k kenarlı dışbükey çokgenin içinde homojen olarak çizilmiştir. Dışbükey örtünün boyutu $\Theta(\lg n)$ olması beklenir.
 - Noktalar iki boyutlu normal bir dağılıma göre çizilmişlerdir. Dışbükey örtünün boyutu $\Theta(\sqrt{\lg n})$ olması beklenir.
- a. Köşe sayıları sırayla n_1 ve n_2 olan iki dışbükey çokgen verilsin. Tüm $n_1 + n_2$ noktalarının dışbükey örtüsünün $O(n_1 + n_2)$ zamanda nasıl hesaplanacağını gösterin (Çokgenler kesişebilir).
 - b. Seyrek-örtülü dağılıma göre bağımsız çizilen n noktalı kümenin dışbükey örtüsünün, $O(n)$ ortalama-durum zamanda nasıl hesaplanacağını gösterin (*İpucu:* Birinci $n/2$ noktanın dışbükey örtüsünü ve ikinci $n/2$ noktanın örtüsünü bulun ve sonra sonuçları birleştirin).

Bölüm notları

Bu ünite hesaplanabilir-geometrinin algoritma ve tekniklerini yüzeyini çok az kazımıştır. Hesaplanabilir-geometri alanındaki kitaplardan bazıları Preparata ve Shamos[282], Edelsbrunner [99] ve O'Rourke[269] dir.

Geometri antik çağdan itibaren çalışılmasına rağmen geometrik problemlerin algoritmalarının gelişimi ona göre yeni sayılır. Preparata ve Shamos bir problemin karmaşıklığı konusunun ilk olarak E.Lemoine tarafından 1902'de verildiğini söyler. Lemoine Öklid yapıları üzerinde çalışıyordu-pergel ve cetvel kullandı- 5 ilkelden oluşan bir küme tasarladı: pergelin bir ayağını verilen bir nokta üzerine, diğer ayağını da verilen bir doğru üzerine yerleştirerek, bir çember çizdi, cetvelin kenarıyla, verilen bir noktanın üzerinden geçen bir doğru çizdi. Lemoine verilen bir yapıyı meydana getirmek için gerekli olan ilkelerin sayılarıyla ilgilendi; buna yapının 'basitliği' adını verdi.

Kısım 33.2 de doğru parçalarının kesişip kesişmediğini belirleyen algoritmanın fikir babası Shamos ve Hoey' dir.

Graham Taramasının orijinal versiyonu Graham [150] tarafından yazılmıştır. Paket-sarma algoritması ise Jarvis[189] tarafından yazılmıştır. Karar-ağacı hesaplama modelini kullanan Yao [359] bir dışbükey-örtü algoritmasının çalışma zamanı için en-kötü-durum alt sınırının $\Omega(n \lg n)$ olduğunu kanıtlamıştır. Dışbükey örtünün h köşe sayısı hesaba katılınca Kirkpatrick ve Seidel[206] ın bu-da-ve-ara algoritması $O(n \lg h)$ zaman la asimptotik olarak en iyisidir.

En-yakın-çifti bulan $O(n \lg n)$ zamanlı böl-ve-yönet algoritması Shamos tarafından Preparata ve Shamos [282]'da yazılmıştır. Preparata ve Shamos bu algoritmanın karar-ağacı modeli ile asimptotik olarak en iyi sonucu verdiğini göstermiştir.

Bu bölüme kadar çalıştığımız algoritmaların neredeyse tamamı **polinom-zamanlı algoritmalar**dır: n boyutlu girdilerde en kötü çalışma süreleri bir k sabit sayısı için $O(n^k)$ dir. *Bütün* problemlerin polinom zamanda çözülüp çözilemeyeceğini merak edebilirsiniz. Cevap hayırdır. Örneğin, ne kadar süre verirse verelim, Turing’in ünlü “Durma Problemi” gibi herhangi bir bilgisayar ile çözilemeyen problemler vardır. Çözülebilir problemler de vardır; ancak çözüm süresi herhangi bir k sabit sayısı için $O(n^k)$ değildir. Genellikle polinom-zamanlı algoritmalar tarafından çözülebilen problemleri çözülebilir ya da kolay olarak, ve süperpolinom zaman gerektiren problemleri çözülemez ya da zor olarak düşünürüz.

Bununla birlikte, bu bölümün konusu, “NP-tam” problemler olarak adlandırılan ve durumu bilinmeyen ilginç bir problemler sınıfıdır. Bir NP-tam problem için henüz hiç bir polinom-zamanlı algoritma bulunmamıştır; ayrıca henüz hiç kimse herhangi biri için hiç bir polinom-zamanlı algoritmanın var olamayacağını da kanıtlanamamıştır. Bu $P \neq NP$ denilen soru, 1971 yılında ilk ortaya çıktığından beri kuramsal bilgisayar bilimlerindeki en derin, en kafa karıştırıcı açık problemlerden biri olmuştur.

Bazı NP-tam problemler özellikle boşuna umut vericidir; çünkü görünüşte polinom zamanda nasıl çözüleceğini bildiğimiz problemler ile benzer gibidirler. Aşağıdaki problem çiftlerinin her birinde problemlerden biri polinom zamanda çözülebilir, diğeri ise NP-tamdır; fakat problemler arasındaki fark önemsiz olarak görülür:

En kısa yollara karşı en uzun basit yollar: Bölüm 24 de, negatif ayrıt ağırlıklılarıyla bile yönlü bir $G = (V, E)$ çizgesindeki tek bir kaynaktan *en kısa* yolları $O(VE)$ zamanda bulabildiğimizi gördük. Ancak iki tepe arasındaki *en uzun* basit yolu bulmak zordur. Bir çizgenin en az verilmiş sayı kadar ayrıta sahip bir basit yol içerip içermediğini sadece belirlemek bile NP-tamdır.

Euler turuna karşı Hamilton çevrimi: Birleşik, yönlü bir $G = (V, E)$ çizgesinin bir *Euler turu* her bir tepeyi birden fazla ziyaret edebilmesine rağmen G nin her bir *ayrıtından* tam olarak bir kez geçen bir dögüdür. Problem 22-3 e göre bir çizgenin bir Euler turuna sahip olup olmadığını sadece $O(E)$ süresi içinde

belirleyebiliriz ve aslında Euler turunun ayrıtlarını $O(E)$ süresi içinde bulabiliriz. Yönlü bir $G = (V, E)$ çizgesinin bir **hamilton çevrimi** V deki tüm *tepeleri* içeren basit bir çevrimdir. Bir yönlü çizgenin hamilton çevrimi içerip içermediğini belirlemek NP-tamdır (Bu bölümün ilerleyen kısımlarında *yönsüz* bir çizgenin hamilton çevrimi içerip içermediğini belirlemenin NP-tam olduğunu kanıtlayacağız).

2-CNF gerçekleştirilebilirliğine karşı 3-CNF gerçekleştirilebilirliği Bir boole formülü, değeri 0 ya da 1 olan değişkenler; $\wedge$ (AND), $\vee$ (OR) ve $\neg$ (NOT) gibi boole bağlaçları ve parantezler içerir. Bir boole formülü, 0 ve 1 değerlerinin değişkenlerine 1 sonucunu veren bir ataması varsa **gerçekleştirilebilir**dir. İlerleyen kısımlarda terimleri daha resmi olarak tanımlayacağız; fakat gayri resmi olarak, bir boole formülü eğer tam olarak k tane değişkenin veya onların değillemelerinin OR larının cümlelerinin AND lenmesi ise ***k-ve-li normal form*** ya da k -CNF dir. Örneğin, $(x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_3) \wedge (\neg x_2 \vee \neg x_3)$ boole formülü 2-CNF dir. ($x_1 = 1, x_2 = 0, x_3 = 1$ gerçekleştirilebilir atamasına sahiptir.) Bir 2-CNF formülünün gerçekleştirilebilir olup olmadığını polinom zamanda belirleyebilmemize rağmen daha sonra göreceğiz ki bir 3-CNF formülünün gerçekleştirilebilir olup olmadığını belirlemek NP-tamdır.

NP-tamlık ve P ve NP sınıfları

Bu bölüm boyunca problemlerin üç sınıfından bahsedeceğiz: P, NP ve NPC - sonuncu sınıf NP-tam problemlerdir. Bunların informal tanımlarını burada vereceğiz; sonradan daha formal olarak tanımlayacağız.

P sınıfı polinom zamanda çözülebilen problemlerden oluşur. Daha açıkçası, n problemin girdi boyutu olmak üzere, bir k sabiti için $O(n^k)$ süresinde çözülebilen problemlerdir. Önceki bölümlerde incelenen problemlerin çoğu P sınıfındadır.

NP sınıfı polinom zamanda “doğrulanabilir” problemlerden oluşur. Doğrulanabilir bir problem ile neyi kastediyoruz? Eğer bize bir şekilde bir çözümün “sertifikası” verilseydi, bu sertifikanın doğru olduğunu problemin girdi boyutunun polinomu zamanında teyit edebilirdik. Örneğin, hamilton döngü probleminde, yönlü bir $G = (V, E)$ çizgesi verildiğinde sertifika, $|V|$ tepelerinin bir $\langle v_1, v_2, v_3, \dots, v_{|V|} \rangle$ dizisi olurdu. Polinom zamanda kolaylıkla $(v_i, v_{i+1}) \in E$ ve ayrıca $(v_{|V|}, v_1) \in E$, $i = 1, 2, 3, \dots, |V| - 1$, kontrol edebilirdik. Bir başka örnek olarak, 3-CNF gerçekleştirilebilirliği için bir sertifika, değerlerin değişkenlere bir ataması olacaktır. Bu atamanın boole formülünü gerçekleştirip gerçekleştirmediğini polinom zamanda kontrol edebiliriz.

P sınıfındaki herhangi bir problem aynı zamanda NP sınıfındadır; çünkü bir problem P de ise sertifika olmaksızın onu polinom zamanda çözebiliriz. Bu bölümün sonraki kısımlarında bu kavramı biçimlendireceğiz; ancak şimdilik $P \subseteq NP$ olduğuna inanabiliriz. Açık soru, P sınıfının NP nin uygun bir alt kümesi olup olmadığıdır.

İnformal olarak, eğer bir problem NP’de ise ve NP sınıfındaki herhangi bir problem kadar “zor” ise bu problem NPC – **NP-tam** yerine kullanıyoruz– sınıfındadır. NP’deki herhangi bir problem kadar zor olmanın ne anlama geldiğini formal olarak daha sonra tanımlayacağız. Bununla birlikte, eğer *herhangi bir* NP-tam problem polinom zamanda çözülebilirse NP’deki *her* problemin polinom zamanlı bir algoritması olduğunu kanıtsız olarak ifade edeceğiz. Çoğu kuramsal bilgisayar bilimci NP-tam problemlerin baş edilemez olduğuna inanır; çünkü bu zamana kadar çalışılmış çok çeşitli NP-tam problemler verildiğinde –hiçkimse, hiçbirisi için polinom-zamanlı bir çözüm keşfetmiş olmaksızın– hepsi polinom zamanda çözülebilseydi hakikaten hayret verici olurdu. Yine de NP-tam problemlerin baş edilemez olduğunu kanıtlamak için bu zamana kadar sarfedilen çaba düşünüldüğünde –kesin bir sonuç olmaksızın– NP-tam problemlerin aslında polinom zamanda çözülebilir olma olasılığını göz ardı edemeyiz.

İyi bir algoritma tasarımcısı olmak için NP-tamlık teorisinin esaslarını anlamalısınız. Bir problemi NP-tam olarak belirleyebilirsiniz onun inatçılığı için iyi kanıtlar temin edersiniz. Böylece bir mühendis olarak, zamanınızı problemi kesin çözen hızlı bir algoritma aramak için harcamaktansa bir yaklaşım algoritması (bkz. Bölüm 35) geliştirmeye harcamakla daha iyi edersiniz. Ayrıca yüzeyde sıralama, çizge arama ya da ağ akış probleminden daha zor görünmeyen çok sayıda doğal ve ilginç problem aslında NP-tamdır. Dolayısıyla problemlerin bu dikkat çekici sınıfına aşina olmalısınız.

Problemlerin NP-tamlığını göstermeye genel bakış

Belirli bir problemin NP-tam olduğunu göstermek için kullandığımız teknikler, bu kitabın genelinde algoritma tasarlamak ve analiz etmek için kullanılan tekniklerden temel olarak farklılık gösterir. Bir problemin NP-tam olduğunu gösterirken ne kadar kolay olduğundan ziyade ne kadar zor olduğu (ya da en azından ne kadar zor olduğunu düşündüğümüz) hakkında bir açıklama yapıyoruz. Etkili bir algoritmanın varlığını kanıtlamaya çalışmıyoruz; fakat onun yerine hiç bir etkili algoritmanın bulunmasının olası olmadığını göstermeye çalışıyoruz. Bu şekilde, NP-tamlık kanıtları Kısım 8.1 deki herhangi bir karşılaştırma sıralama algoritması için bir $\Omega(n \lg n)$ - zamanlı alt sınırın kanıtı ile bazı benzerlikler taşır; ancak NP-tamlığı göstermek için kullanılan bazı özel teknikler Kısım 8.1 de kullanılan karar-ağacı yönteminden farklılık gösterir.

Bir problemin NP-tam olduğunu göstermede üç ana kavrama dayanırız.

Karar problemlerine karşı optimizasyon problemleri

İlgili problemlerin çoğu her bir uygun (yani “yasal”) çözümün ilişkili bir değerinin olduğu **optimizasyon problemleridir** ve biz en iyi değere sahip uygun çözümü bulmayı diliyoruz. Örneğin, SHORTEST-PATH probleminde, yönsüz bir G çiz-

gesi ile u ve v tepeleri veriliyor ve en az sayıda ayrıt kullanarak u dan v ye bir yol bulmak istiyoruz. Başka bir deyişle, SHORTEST-PATH, ağırlıksız ve yönsüz bir çizgede tek-ikili en kısa-yol problemidir. NP-tamlık direkt olarak optimizasyon problemlerine uygun olmamakla birlikte yanıtın basitçe “evet” ya da “hayır” (ya da daha formal olarak “1” veya “0”) olduğu **karar problemlerine** uygundur.

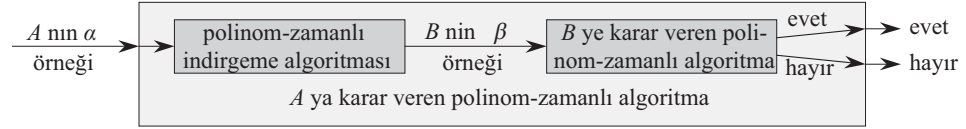
NP-tam problemler, karar problemlerinin bölgesine sınırlanmış olmasına rağmen; optimizasyon problemleri ve karar problemlerinin arasındaki uygun bir ilişkiden yararlanabiliriz. Genellikle, verilen bir optimizasyon problemini, optimize edilmek istenen değer üzerine bir sınır koyarak, ilişkili karar problemi olarak gösterebiliriz. Örneğin, SHORTEST-PATH ile ilişkili bir karar problemi PATH problemidir: yönlü bir G çizgesi, u ve v tepeleri ile k tam sayısı verildiğinde u dan v ye en fazla k tane ayrıt içeren bir yol var mıdır?

Bir optimizasyon problemi ve onun ilişkili problemi arasındaki ilişki, optimizasyon probleminin “zor” olduğunu göstermeye çalışırken bizim lehimize işler. Bunun nedeni, karar probleminin bir anlamda “daha kolay” ya da en azından “daha zor olmayan” bir problem olmasıdır. Özel bir örnek olarak, SHORTEST-PATH problemini çözerek ve ardından bulunan en kısa yoldaki ayrıtların sayısını karar-problemi parametresi k ile karşılaştırarak PATH problemini çözebiliriz. Bir başka deyişle, eğer bir optimizasyon problemi kolaysa onun ilişkili karar problemi de kolaydır. NP-tamlık ile daha fazla uygunluğa sahip bir şekilde belirtmek gerekirse, eğer bir probleminin zor olduğuna dair kanıtlar sağlayabilirsek, onun ilişkili probleminin de zor olduğuna dair kanıtlar temin ederiz. Dolayısıyla, her ne kadar dikkati karar problemlerine sınırlasa da NP-tamlık teorisinin sıklıkla optimizasyon problemleri için de uygulamaları vardır.

İndirgemeler

Yukarıdaki bir problemin diğerinden daha zor ya da daha kolay olmadığını gösterme kavramı her iki problem karar problemi olduğunda dahi uygulanır. Aşağıda olduğu gibi, neredeyse bütün NP-tamlık kanıtlarında bu fıkirden yararlanacağız. Polinom zamanda çözmek istediğimiz bir A karar problemi düşünelim. Belirli bir problemin girdilerini o problemin **örneği** olarak adlandırırız; örneğin PATH probleminde belirli bir G çizgesi, G nin belirli iki tepesi u ve v ile belirli bir k tam sayısı bir örnektir. Şimdi farzedelim ki farklı bir B probleminin polinom zamanda nasıl çözüldüğünü zaten biliyoruz. Son olarak, farzedelim ki A nın herhangi bir α örneğini B nin bir β örneğine dönüştüren ve aşağıdaki özelliklere sahip bir prosedürümüz olsun:

- Dönüşüm polinom zaman gerektirir.
- Yanıtlar aynıdır. Yani β için yanıt "evet" tir ancak ve ancak α için de yanıt “evet” tir.



Şekil 34.1 Başka bir B algoritması için karar algoritması verildiğinde bir A karar problemini çözmek için polinom-zamanlı bir indirgeme algoritmasının nasıl kullanılacağı. Polinom zamanda A nın bir α örneğini B nin bir β örneğine dönüştürürüz; B yi polinom zamanda çözeriz ve β nin yanıtını α nın yanıtı olarak kullanırız.

Böyle bir prosedürü polinom zamanlı bir **indirgeme algoritması** olarak adlandırırız ve Şekil 34.1 in gösterdiği üzere bu bize A problemini polinom zamanda çözmek için bir yol sağlar:

1. A probleminin herhangi bir α örneği verildiğinde bunu B probleminin bir β örneğine dönüştürmek için polinom-zamanlı bir indirgeme algoritması kullan.
2. Polinom-zamanlı karar algoritmasını B nin β örneği üzerinde çalıştır.
3. β nin yanıtını α nın yanıtı olarak kullan.

Bu adımların her biri polinom zaman aldığı sürece üçü birlikte yine polinom zaman alır ve dolayısıyla α ya karar vermek için polinom zamanda bir yolumuz olur. Başka bir deyişle, A problemini çözmeyi B problemini çözmeye “indirgeyerek” A nın “kolaylığını” kanıtlamak için B nin “kolaylığını” kullanırız.

NP-tamlığın, bir problemin ne kadar kolay olduğunu göstermekten ziyade ne kadar zor olduğunu göstermekle ilgili olduğunu hatırlayarak, bir problemin NP-tam olduğunu göstermek için polinom-zamanlı indirgemeleri zıt yönde kullanırız. Bu fikri bir adım daha ileri götürelim ve belirli bir B problemi için hiçbir polinom-zamanlı algoritmanın var olamayacağını göstermek için polinom-zamanlı algoritmaları nasıl kullanabileceğimizi gösterelim. Farzedelim ki onun için hiçbir polinom-zamanlı algoritmanın var olamayacağını zaten bildiğimiz bir A karar problemine sahibiz. (Şimdilik böyle bir A problemini nasıl bulacağımız ile ilgilene-meyelim.) Ayrıca A nın örneklerini B nin örneklerine dönüştüren polinom-zamanlı bir indirgemeye sahip olduğumuzu da varsayalım. Şimdi B için hiçbir polinom-zamanlı algoritmanın var olamayacağını göstermek için basit bir çelişki ile kanıt kullanabiliriz. Aksini varsayalım; yani B polinom-zamanlı bir algoritmaya sahip olsun. Bu durumda Şekil 34.1 de gösterilen yöntemi kullanarak A problemini polinom zamanda çözmek için bir yolumuz olacaktır ki bu da A için hiçbir polinom-zamanlı algoritma olmadığı varsayımımızla çelişir.

NP-tamlık için A probleminin kesinlikle hiçbir polinom-zamanlı algoritmasının olmadığını varsayamayız. Kanıt metodolojisi benzerdir; Bununla birlikte A probleminin de NP-tam olduğu varsayımı üzerine B probleminin NP-tam olduğunu kanıtlarız.

İlk NP-tam problem

İndirgeme tekniği, farklı bir problemin NP-tam olduğunu kanıtlamak için zaten NP-tam olduğu bilinen bir probleme sahip olmaya dayandığından bir “ilk” problem ihtiyacımız var. Kullanacağımız problem devresel-gerçekleştirilebilirlik problemidir; bu problemde AND, OR ve NOT kapılarından oluşmuş bir boole kombinatorik döngüsü verildiğinde, çıktısı 1 olacak şekilde bir boole girdi kümesinin var olup olmadığını bilmek isteriz. Bu ilk problemin NP-tam olduğunu Kısım 34.3 de kanıtlayacağız.

Bölüm taslağı

Bu bölüm, algoritmaların analiziyle doğrudan doğruya ilgili olan NP-tamlığın yönlerini araştırmaktadır. Kısım 34.1 de “problem” kavramımızı biçimlendireceğiz ve polinom-zamanda çözülebilir karar problemlerinin P karmaşıklık sınıfını tanımlayacağız. Ayrıca bu kavramların formal-dil teorisinin çerçevesine nasıl girdiğini göreceğiz. Kısım 34.2, çözümlerinin polinom zamanda doğrulanabilir olduğu karar problemlerinin NP sınıfını tanımlar. Ayrıca $P \neq NP$ sorusunu formal olarak ortaya çıkarır.

Kısım 34.3, polinom-zamanlı “indirgemeler” aracılığıyla problemleri ilişkilendirebileceğimizi gösterir. NP-tamlığı tanımlar ve “devresel-gerçekleştirilebilirlik” adı verilen bir problemin NP-tam olduğunun kanıtını kısaca tarif eder. Bir NP-tam problem bulduktan sonra, Kısım 34.4 de, diğer problemlerin NP-tam olduğunun indirgemeler metodolojisi ile nasıl daha basit şekilde kanıtlandığını göstereceğiz. Bu metodolojiyi iki tane formül gerçekleştirilebilirlik probleminin NP-tam olduğunu göstererek örneklendireceğiz. İlave indirgemeler ile, Kısım 34.5 de çeşitli diğer problemlerin NP-tam olduğunu göstereceğiz.

34.1 Polinom zaman

NP-tamlık çalışmamıza polinom-zamanda çözülebilir problemler kavramımız ile başlıyoruz. Genellikle, bu problemlere baş edilebilir olarak bakarız; ancak matematiksel değil filozofik nedenler için. Üç destekleyici görüş sunabiliriz.

İlk olarak, $\Theta(n^{100})$ zaman gerektiren bir problemi mantıklı olarak baş edilemez olarak görebilmemize rağmen, gerçekçi problemlerin çok azı böyle yüksek-dereceli polinom türünden zaman gerektirir. Pratikte karşılaşılan, polinom-zamanda hesaplanabilir problemler genel anlamda daha az zaman gerektirir. Tecrübeler göstermiştir ki bir problem için ilk polinom-zamanlı algoritma bir kez bulunduğu daha etkili algoritmalar sıklıkla ardından gelir. Bir problem için eldeki en iyi algoritmanın çalışma zamanı $\Theta(n^{100})$ olsa bile daha iyi çalışma zamanlı bir algoritmanın çok geçmeden bulunması olasıdır.

İkinci olarak, birçok makul hesaplama modeli için bir modelde polinom zamanda çözülebilen bir problem diğerinde de polinom zamanda çözülebilir. Örneğin, bu kitabın genelinde kullanılan seri rassal-erişimli makineler ile polinom zamanda çözülebilen problemlerin sınıfı, soyut Turing makinelerinde polinom zamanda çözülebilen problemler sınıfıyla aynıdır.¹ Ayrıca işlemci sayısı girdi boyutuyla polinomiyal olarak büyüdüğünde bir paralel bilgisayarda polinom zamanda çözülebilen problemler sınıfıyla da aynıdır.

Üçüncü olarak, polinom-zamanda çözülebilir problemlerin sınıfı iyi kapalılık özelliklerine sahiptir; çünkü polinomlar toplama, çarpma ve birleşim işlemleri altında kapalıdır. Örneğin, polinom-zamanlı bir algoritmanın çıktısı diğerinin girdisiyle beslenirse bileşik algoritma, polinomsaldır. Alıştırma 34.1-5 de eğer bir algoritma polinom-zamanlı alt programa sabit bir sayı kadar çağrı yapar ve ilave miktarda işi yine polinom-zamanda yürütürse, bileşik algoritmanın çalışma zamanının polinomsal olduğunun gösterilmesi istenmektedir.

Soyut problemler

Polinom-zamanda çözülebilir problemler sınıfını anlamak için öncelikle bir “problem”in ne demek olduğuna dair formal bir kavrama sahip olmalıyız. Bir *Q* soyut problemi problem örnekleri kümesi *I* ve problem çözümleri kümesi *S* üzerinde bir ikili bağıntı olarak tanımlarız. Örneğin, SHORTEST-PATH probleminin bir örneği, bir çizge ve 2 tepe içeren bir üçlüdür. Çizgedeki tepelerin bir dizisi belki de hiçbir yol olmadığını belirten boş dizi ile bir çözümdür. SHORTEST-PATH probleminin kendisi bir çizge ve iki tepenin her bir örnek problemi ile iki tepeyi bağlayan çizgedeki bir en kısa yolu ilişkilendirir. En kısa yollar illa ki tek olmadığından verilen bir problem örneği birden fazla çözüme sahip olabilir.

Bir soyut problemin bu formülasyonu, amaçlarımız için ihtiyaç duyduğumuzdan daha geneldir. Yukarıda gördüğümüz üzere NP-tamlık teorisi dikkati **karar problemlerine** sınırlandırır: bir evet/hayır çözümüne sahip olanlar. Bu durumda, bir soyut karar problemini, *I* örnek kümesini $\{0, 1\}$ çözüm kümesine eşleyen bir fonksiyon olarak görebiliriz. Örneğin, SHORTEST-PATH ile ilişkili bir karar problemi daha önceden gördüğümüz PATH problemidir. Eğer $\langle G, u, v, k \rangle$ PATH karar probleminin bir örneği ise *u* dan *v* ye bir en kısa yol, en fazla *k* tane ayrıta sahip ise $PATH(i) = 1$ (evet); aksi halde $PATH(i) = 0$ (hayır) dır. Soyut problemlerin çoğu, karar problemleri değil en küçüklenecek ya da en büyüklenecek bazı değerler gerektiren **optimizasyon problemleridir**. Bununla birlikte yukarıda gördüğümüz üzere, bir optimizasyon problemini bir karar problemi olarak genellikle yeniden biçimlendirebiliriz, bu daha zor değildir.

¹Turing-makine modelinin derinlemesine işlenişi için Hopcroft ve Ullman [180] ya da Lewis ve Papadimitriou [236] ya bakınız.

Kodlamalar

Bir bilgisayar programının, bir soyut problemi çözmesi için problem örneklerinin programın anlayacağı şekilde ifade etmeliyiz. Soyut nesnelerin bir S kümesinin bir **kodlaması** S den ikili katarlar kümesine bir e eşlemesidir.² Örneğin, doğal sayıların, $\mathbb{N} = \{0, 1, 2, 3, 4, \dots\}$, $\{0, 1, 10, 11, 100, \dots\}$ katarları olarak kodlanmasına hepimiz aşinayız. Bu kodlamayı kullanarak $e(17) = 10001$ dir. Eğer klavye karakterlerinin bilgisayar gösterimlerine baktıysanız, muhtemelen, A nın kodlamasının 1000001 olduğu ASCII kodu görmüşsünüzdür. Bileşik bir nesneyi, onu oluşturan parçaların gösterimlerini birleştirerek bir ikili katar olarak kodlayabiliriz. Çokgenler, çizgeler, fonksiyonlar, sıralı çiftler, programlar – tamamı ikili katarlar olarak kodlanabilirler.

Böylece, bazı soyut karar problemlerini “çözen” bir bilgisayar algoritması aslında bir problem örneğinin kodlamasını girdi olarak alır. Örnek kümesi, ikili katarların kümesi olan bir problemi **somut problem** olarak adlandırırız. Eğer bir algoritma, bir somut problemin çözümünü $O(T(n))$ zamanda üretebilirse, $n = |i|$ uzunluğunda bir i problem örneği sağlandığında, algoritma bu problemi $O(T(n))$ zamanda **çözer** deriz.³ Eğer bir somut problemi, bir k sabit sayısı için $O(n^k)$ zamanda çözen bir algoritma mevcutsa, bu somut problem **polinom-zamanlı çözülebilir**dir.

Şimdi **P karmaşıklık sınıfını** formal olarak polinom-zamanlı çözülebilen somut problemlerin kümesi olarak tanımlayabiliriz

Soyut problemleri somut problemlere eşlemek için kodlamaları kullanabiliriz. Bir I örnek kümesini $\{0,1\}^*$ kümesine eşleyen bir Q soyut karar problemi verildiğinde, bir $e : I \rightarrow \{0, 1\}^*$ kodlaması, $e(Q)$ ile gösterdiğimiz bir ilişkili somut karar problemine teşvik edebilir.⁴ Eğer bir $i \in I$ soyut-problem örneğinin çözümü $Q(i) \in \{0, 1\}$ ise $e(i) \in \{0, 1\}^*$ somut-problem örneğinin çözümü de $Q(i)$ dir. Bir teknik ayrıntı olarak, bazı ikili katarlar hiçbir anlamlı soyut-problem örneği sunmayabilirler. Uygunluk açısından bu şekildeki herhangi bir katarın keyfi olarak 0 a eşlendiğini varsayacağız. Dolayısıyla, somut problem, soyut-problem örneklerinin kodlamalarını gösteren ikili-katar örneklerinde, soyut problem ile aynı çözümleri üretir.

Polinom-zamanlı çözülebilirlik tanımını, kodlamaları köprü olarak kullanarak somut problemlerden soyut problemlere genişletmek istiyoruz; ancak tanımın

² e nin değer kümesinin *ikili* katarlar olması gerekmez; en az 2 sembol içeren sonlu bir alfabe üzerindeki herhangi bir katar kümesi yeterlidir.

³Algoritmanın çıktısının girdisinden ayrı olduğunu varsayıyoruz. Çünkü çıktının her bir bitini üretmek en az bir zaman adımı alır ve algoritma $O(T(n))$ zaman adımı alır; çıktının boyutu $O(T(n))$ dir.

⁴ $\{0,1\}^*$ ile $\{0,1\}$ kümesindeki sembollerden oluşmuş tüm katarların kümesini belirtiriz.

herhangi belirli bir kodlamadan bağımsız olmasını istiyoruz. Yani, bir problemi çözmenin etkinliği, problemin nasıl kodlandığına bağlı olmamalıdır. Ne yazık ki oldukça fazla şekilde kodlamaya bağlıdır. Örneğin, bir k tam sayısının tek girdi olarak bir algoritmaya verildiğini ve algoritmanın çalışma zamanının $\Theta(k)$ olduğunu farzedelim. Eğer k tam sayısı, birli olarak $-k$ nin bir katarı, $1s -$ verilirse algoritmanın çalışma zamanı, n -uzunluklu girdiler üzerinde $O(n)$ polinom zamandır. Bununla beraber, k tam sayısının daha doğal ikili gösterimini kullanırsak girdi uzunluğu $n = \lfloor \lg k \rfloor + 1$ dir. Bu durumda algoritmanın çalışma zamanı, girdi boyutunda üssel olan $\Theta(k) = \Theta(2^n)$ dir. Dolayısıyla, kodlamaya bağlı olduğu için algoritma polinom ya da süperpolinom zamanda çalışır.

Bir soyut problemi nasıl kodladığımız polinom zamanı nasıl anladığımızı göre oldukça önem taşır. Öncelikle bir kodlama belirlemeksizin, gerçekten bir soyut problemi çözmekten bahsedemeyiz. Yine de, uygulamada, birli olanlar gibi “pahalı” kodlamaları göz ardı edersek bir problemin asıl kodlaması, problemin polinom zamanda çözülüp çözülemeyeceğini az miktarda etkiler. Örneğin, tam sayıları birli yerine 3 tabanında göstermenin, bir problemin polinom zamanda çözülebilirliğine bir etkisi yoktur; çünkü 3 tabanında gösterilen bir tam sayıyı, 2 tabanında gösterilen bir tam sayıya polinom zamanda dönüştürebiliriz.

Herhangi bir $x \in \{0, 1\}^*$ girdisi verildiğinde $f(x)$ çıktısını üreten bir A polinom-zamanlı algoritması mevcutsa $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ fonksiyonu **polinom-zamanlı hesapedilebilirdir** deriz. Problem örneklerinin bir I kümesi için eğer herhangi bir $i \in I$ için $f_{12}(e_1(i)) = e_2(i)$ sağlayan polinom-zamanlı hesapedilebilir f_{12} ve f_{21} fonksiyonları varsa, e_1 ve e_2 kodlamaları **polinomiyal ilişkilidir** deriz.⁵ Yani bir polinom-zamanlı algoritma $e_2(i)$ kodlamasını $e_1(i)$ kodlamasından hesaplayabilir, tersi de geçerlidir. Aşağıdaki lemmanın gösterdiği üzere, eğer bir soyut problemin e_1 ve e_2 gibi iki kodlaması polinomiyal ilişkiliyse problemin polinom-zamanlı çözülebilir olup olmadığı hangi kodlamayı kullandığımızdan bağımsızdır.

Lemma 34.1

Q , bir I örnek kümesi üzerinde bir soyut karar problemi ve e_1 ile e_2 I üzerinde polinomiyal ilişkili kodlamalar olsun. Bu durumda, $e_2(Q) \in P$ dir ancak ve ancak $e_1(Q) \in P$ dir.

⁵Teknik olarak, f_{12} ve f_{21} fonksiyonlarının ayrıca “örnek olmayanlardan örnek olmayanlara eşleme” olmalarını isteriz. Bir e katarının bir **örnek olmayanı**, $e(i) = x$ için hiçbir i örneği bulunmayan bir $x \in \{0, 1\}^*$ katarıdır. e_1 kodlamasının her x örnek olmayanı için y , e_2 nin bir örnek olmayanı olmak üzere, $f_{12}(x) = y$; ve e_1 kodlamasının her x' örnek olmayanı için y' , e_1 in bir örnek olmayanı i olmak üzere, $f_{21}(x') = y'$ olmasını isteriz.

Kanıt Sadece ileri yönü kanıtlamamız gerekiyor; çünkü geri yön simetriktr. Bu nedenle, farzedelim ki $e_1(Q)$, bir k sabit sayısı için $O(n^k)$ zamanda çözülebilir. Ayrıca, herhangi bir i problem örneği için $n = |e_2(i)|$ ve c bir sabit sayı olmak üzere, $e_1(i)$ kodlamasının, $e_2(i)$ kodlamasından $O(n^c)$ zamanda çözülebildiğini varsayalım. $e_2(Q)$ problemini $e_2(i)$ girdisi ile çözmek için öncelikle $e_1(i)$ yi hesaplarız, ardından $e_1(Q)$ için algoritmayı $e_1(i)$ ile çalıştırırız. Bu ne kadar zaman alır? Kodlamaları dönüştürmek $O(n^c)$ zaman alır, dolayısıyla $|e_1(i)| = O(n^c)$; çünkü bir seri bilgisayarın çıktısı çalışma zamanından daha uzun olamaz. $e_1(i)$ için problemi çözmek $O(|e_1(i)|^k) = O(n^{ck})$ zaman alır ki bu da n ve k sabit sayı olduğundan polinom zamandır. ■

Dolayısıyla, bir soyut problemin ikili tabanda ya da 3 tabanında kodlanmış örneklerinin olması “karmaşıklığını”, yani polinom-zamanlı çözülebilir olup olmadığını etkilemez; ancak eğer örnekler birli sistemde kodlanırsa karmaşıklığı değişebilir. Bir kodlama-bağımsız biçimi tersine çevirebilmek için, özellikle belirtmediğimiz takdirde, problem örneklerinin herhangi makul, özlü bir biçimde kodlandığını varsayacağız. Açık olmak gerekirse, bir tam sayının kodlamasının, ikili gösterimine polinomiyal ilişkili olduğunu ve sonlu bir kümenin kodlamasının, küme parantezleriyle kapatılmış ve virgüllerle ayrılmış bir elemanlar listesi olarak kodlaması ile polinomiyal ilişkili olduğunu farzedeceğiz. (ASCII böyle bir kodlama şemasıdır.) Eldeki böyle bir “standart” kodlama ile değişkenler grupları, çizgeler ve formüller gibi diğer matematiksel nesnelerin uygun kodlamalarını elde edebiliriz. Bir nesnenin standart kodlamasını belirtmek için nesneyi üçgen parantezler içine yazacağız. Bu nedenle, $\langle G \rangle$, G çizgesinin standart kodlamasını belirtir.

Bu standart kodlamaya polinomiyal ilişkili bir kodlamayı tam olarak kullandığımız sürece, kodlama seçiminin soyut problemin polinom-zamanlı çözülebilirliğine hiçbir etkisi olmadığını bilerek, herhangi bir kodlamadan bahsetmeksiz doğrudan soyut problemler hakkında konuşabileceğiz. Bundan böyle, açıkça aksini belirtmedikçe, tüm problem örneklerinin standart kodlamayı kullanarak kodlanmış ikili katarlar olduğunu varsayacağız. Genel anlamda soyut ve somut problemler arasındaki farkı da ihmal edeceğiz. Bununla birlikte, standart kodlamanın açık olmadığı ve kodlamanın fark yaratacağı, uygulamadaki problemlere karşı dikkatli olmalısınız.

Bir formal-dil yapısı

Karar problemlerine odaklanarak formal-dil teorisi mekanizmasından yararlanabiliriz. O teoriden bazı tanımlamaları gözden geçirelim. Bir Σ **alfabesi**, sembollerin sonlu bir kümesidir. Σ üzerindeki bir L **dili**, Σ deki sembollerden oluşturulmuş katarların herhangi bir kümesidir. Örneğin, $\Sigma = \{0,1\}$ ise $L = \{10, 11, 101, 111, 1011, 1101, 10001, \dots\}$ kümesi asal sayıların ikili temsili dilidir. **Boş katarı** ϵ

ile, **boş dili** $\emptyset$ ile ve Σ üzerindeki tüm katarların dilini Σ^* ile gösteririz. Örneğin, $\Sigma = \{0, 1\}$ ise $\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$ tüm ikili katarların kümesidir. Σ üzerindeki her L dili, Σ^* 'ın bir altkümesidir.

Diller üzerinde çeşitli işlemler uygulayabiliriz. **Birleşim** ve **kesişim** gibi küme-teorik işlemler, doğrudan küme-teorik tanımlardan çıkar. L nin **tümleyenini** $\overline{L} = \Sigma^* - L$ olarak tanımlarız. L_1 ve L_2 gibi iki dilin $L_1 L_2$ **art arda eklenmesi**

$L = \{x_1 x_2 : x_1 \in L_1 \text{ ve } x_2 \in L_2\}$ dilidir.

Bir L dilinin **kapanışı** ya da **Kleene yıldızı**, L^k , L nin k defa ardarda eklenmesiyle elde edilmiş dil olmak üzere

$L^* = \{\epsilon\} \cup L \cup L^2 \cup L^3 \cup \dots$, dilidir.

Dil teorisi bakımından, herhangi bir Q karar problemi için örnekler kümesi, $\Sigma = \{0, 1\}$ olmak üzere, basitçe Σ^* dır. Q , bütünüyle bir 1 (evet) yanıtı üreten problem örnekleriyle tanımlandığı için Q problemini, $\Sigma = \{0, 1\}$ üzerinde

$L = \{x \in \Sigma^* : Q(x) = 1\}$

olmak üzere, bir L dili olarak görebiliriz.

Örneğin, PATH karar problemi, ona ilişkin

YOL = $\{(G, u, v, k) : G = (V, E) \text{ bir yönsüz çizge,}$
 $u, v \in V,$
 $k \geq 0 \text{ bir tam sayı, ve}$
 $G \text{ de } u \text{ dan } v \text{ ye en fazla } k \text{ tane}$
 $\text{ayrıt içeren bir yol vardır}\}.$

diline sahiptir.

(Uygun olduğu yerde, hem karar problemi hem de ilgili dilinden bahsetmek için bazen aynı ismi – bu durumda PATH – kullanacağız.)

Formal dil yapısı, karar problemleri ve onları çözen algoritmalar arasındaki bağıntıyı kısaca açıklamamıza olanak tanır. Bir x girdisi verildiğinde algoritmanın $A(x)$ çıktısı 1 oluyorsa, bir A algoritması bir $x \in \{0, 1\}^*$ katarını **kabul eder** deriz. Bir A algoritması tarafından **kabul edilen** dil, katarların bir kümesi $L = \{x \in \{0, 1\}^* : A(x) = 1\}$; yani, algoritmanın kabul ettiği katarların kümesidir. Eğer $A(x)=0$ ise A algoritması x katarını **reddeder**.

L dili, bir A algoritması tarafından kabul edilse bile; algoritma, girdi olarak sağlanan bir $x \notin L$ katarını zorunlu olarak reddetmeyecektir. Örneğin, algoritma sonsuz döngüye girebilir. Bir L dilindeki her ikili katar bir A algoritması tarafından kabul edilir ve L de olmayan her ikili katar A tarafından reddedilir ise L diline, A algoritması tarafından **karar verilir**. Bir L dili bir A algoritması tarafından kabul edilirse ve ayrıca herhangi n -uzunluklu bir $x \in L$ katarı için A algoritmasının x i $O(n^k)$ zamanda kabul edeceği şekilde bir k sabiti varsa, L dili,

A algoritması tarafından **polinom zamanlı kabul edilir**. Herhangi n -uzunluklu bir $x \in \{0, 1\}^*$ katarı için bir A algoritmasının $x \in L$ olup olmadığına $O(n^k)$ sürede doğru biçimde karar vereceği şekilde bir k sabiti mevcutsa, L diline, A algoritması tarafından **polinom zamanlı karar verilir**. Dolayısıyla, bir algoritmanın bir dili kabul etmesi için L de bir katar verildiğinde sadece bir yanıt üretmesi gerekir; ancak bir dili belirlemesi için $\{0, 1\}^*$ deki her katarı doğru şekilde kabul etmeli ya da reddetmelidir.

Örnek olarak, PATH dili polinom zamanlı kabul edilebilir. Polinom-zamanlı kabul eden bir algoritma G nin yönsüz bir çizgesi kodladığını, u ve v nin G deki tepeler olduğunu doğrular; G de, u dan v ye en kısa yolu hesaplamak için enine aramayı kullanır ve elde edilen en kısa yol üzerindeki ayrıtların sayısını k ile karşılaştırır. Eğer G , bir yönsüz çizgesi kodlarsa ve u dan v ye bulunan yol en fazla k tane ayrıt içeriyorsa, algoritma 1 çıktısını verir ve durur. Aksi halde, algoritma sonsuza kadar çalışır. Bununla birlikte, bu algoritma PATH i belirlemez; çünkü bir kısa yolun k den daha fazla ayrıta sahip olduğu örnekler için açıkça 0 çıktısını üretmez. PATH için bir karar algoritması, PATH e ait olmayan ikili katarları açık olarak reddetmelidir. PATH gibi bir karar problemi için böyle bir karar algoritması tasarlamak kolaydır: u dan v ye en fazla k ayrıtlı bir yol olmadığında sonsuza dek çalışmak yerine 0 çıktısını üretir ve durur (Girdi kodlaması hatalıysa da 0 çıktısını vermeli ve sonlanmalıdır). Turing Durma Problemi gibi diğer problemler için kabul eden bir algoritma mevcuttur; ancak bir karar algoritması yoktur.

Bir **karmaşıklık sınıfını** dillerin bir kümesi olarak informal şekilde, verilen bir x katarının L diline ait olup olmadığını belirleyen bir algoritmanın çalışma zamanı gibi bir **karmaşıklık ölçüsü** tarafından belirlenen üyeliği olarak tanımlayabiliriz. Karmaşıklık sınıfının asıl tanımı biraz daha tekniktir.⁶

Bu dil-teorik yapıyı kullanarak P karmaşıklık sınıfının alternatif bir tanımını verebiliriz:

$$P = \{L \subseteq \{0, 1\}^*: L \text{ ye polinom zamanlı karar veren bir } A \text{ algoritması vardır}\}.$$

Aslında, P ayrıca polinom zamanlı kabul edilebilen dillerin de sınıfıdır.

Teorem 34.2

$P = \{L: L, \text{ polinom-zamanlı bir algoritma tarafından kabul edilir}\}.$

Kanıt Polinom-zamanlı algoritmalar tarafından karar verilen dillerin sınıfı, polinom-zamanlı algoritmalar tarafından kabul edilen dillerin sınıfının altkütmesi olduğu için sadece, L polinom-zamanlı bir algoritma tarafından kabul edilirse,

⁶Karmaşıklık sınıfları üzerine daha fazla bilgi için Hartmanis ve Stearns'ın özgün makalesine bakınız [162].

L ye polinom-zamanlı bir algoritma tarafından karar verildiğini göstermemiz gerekmektedir. L , polinom-zamanlı bir A algoritması tarafından kabul edilen dil olsun. L ye karar veren bir başka polinom-zamanlı A' algoritması oluşturmak için klasik bir “simülasyon” argümanı kullanacağız. A, L yi bir k sabiti için $O(n^k)$ zamanda kabul ettiği için öyle bir c sabiti de vardır ki A, L yi en fazla cn^k adımda kabul eder. Herhangi bir x katar girdisi için, A' algoritması A nın cn^k adımını simüle eder. cn^k adım simülasyondan sonra A' algoritması A nın davranışını inceler. A, x i kabul etmişse A' bir 1 çıktısı üreterek x i kabul eder. A, x i kabul etmemişse A' bir 0 çıktısı üreterek x i reddeder. A yi simüle eden A' algoritmasının ek yükü çalışma zamanını bir polinom katsayıdan fazla artırmaz ve dolayısıyla A', L yei belirleyen polinom-zamanlı bir algoritmadır. ■

Dikkat edilirse Teorem 34.2 nin kanıtı yapısal değildir. Verilen bir $L \in P$ dili için L yi kabul eden A algoritmasının çalışma zamanı üzerinde aslında bir sınır bilmiyor olabiliriz. Bununla birlikte, A' algoritmasını kolaylıkla bulamasak bile, böyle bir sınırın var olduğunu ve dolayısıyla sınırı kontrol edebildiğimiz bir A' algoritmasının var olduğunu biliriz.

Alıştırmalar

34.1-1

LONGEST-PATH-LENGTH optimizasyon problemini yönsüz bir çizge ve iki tepeden oluşan her bir örnek ile iki tepe arasındaki bir en uzun basit yolu ilişkilendiren bir bağıntı olarak tanımlayınız. LONGEST-PATH = $\{\langle G, u, v, k \rangle : G = (V, E) \text{ bir yönsüz çizge; } u, v \in V, k \geq 0 \text{ bir tam sayı ve } G \text{ de } u \text{ dan } v \text{ ye en az } k \text{ ayrıt içeren bir basit yol var}\}$ karar probleminin tanımlayınız. LONGEST-PATH-LENGTH optimizasyon probleminin ancak ve ancak LONGEST-PATH $\in P$ ise polinom zamanda çözülebildiğini gösteriniz.

34.1-2

Yönsüz bir çizgede en uzun basit çevrimi bulma problemi içi formal bir tanım veriniz. İlişkili karar problemini veriniz. Karar problemine ilişkin dili veriniz.

34.1-3

Bir bitişiklik-matrisi gösterimi kullanarak, yönlü çizgelerin ikili katarlar olarak formal kodlamasını veriniz. Aynısını bir bitişiklik-listesi gösterimi kullanarak yapınız. İki gösterimin polinomiyal ilişkili olup olmadığını tartışınız.

34.1-4

0-1 sırt çantası problemi için Alıştırma 16.2-2 de sorulan dinamik-programlama algoritması polinom-zamanlı bir algoritma mıdır? Yanıtınızı açıklayınız.

34.1-5

Bir algoritma polinom-zamanlı altprogramlara sabit bir sayı kadar çağrı yapıyor ve yine polinom zaman alan ek miktarda işi yürütüyorsa, bu algoritmanın polinom zamanda çalıştığını gösteriniz. Ayrıca, polinom-zamanlı altprogramlara bir polinom sayı kadar yapılan çağrılarını, üssel-zamanlı bir algoritmaya neden olabildiğini gösteriniz.

34.1-6

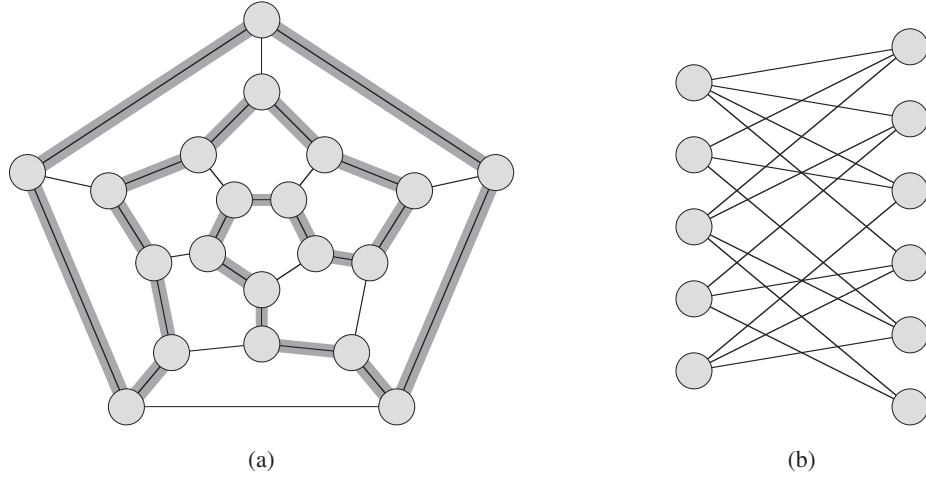
Dillerin bir kümesi olarak görülen P sınıfının birleşim, kesişim, art arda eklenme, tümleyen ve Kleene yıldızı işlemleri altında kapalı olduğunu gösteriniz. Yani $L_1, L_2 \in P$ ise $L_1 \cup L_2 \in P$, $L_1 \cap L_2 \in P$, $L_1 L_2 \in P$, $\overline{L_1} \in P$ ve $L_1^* \in P$ dir.

34.2 Polinom-zamanlı doğrulama

Şimdi dillerdeki üyeliği doğrulayan algoritmalara bakacağız. Örneğin, PATH karar probleminin verilen bir $\langle G, u, v, k \rangle$ örneği için ayrıca u dan v ye bir p yolu verildiğini farzedelim. p nin G de bir yol olup olmadığını ve p nin uzunluğunun en fazla k olup olmadığını kolaylıkla kontrol edebiliriz; ve eğer öyleyse p yi, örneğin gerçekten PATH problemine ait olduğunu belirten bir “sertifika” olarak görebiliriz. PATH karar problemi için bu sertifika bize çok şey ifade eder gibi görünmüyor. Nihayetinde PATH, P sınıfına aittir –aslında, PATH ı doğrusal zamanda çözebiliriz– ve dolayısıyla, verilen bir sertifikadan üyeliği doğrulamak, problemi başlangıçtan çözmek süresince devam eder. Şimdi kendisi için hiçbir polinom-zamanlı karar algoritmasının bilinmediği ve yine de bir sertifika verildiğinde doğrulamanın kolay olduğu bir problemi inceleyeceğiz.

Hamilton çevrimleri

Yönsüz bir çizgede bir hamilton çevrimi bulma problemi yüzyılı aşkın bir süre çalışılmıştır. Formal olarak, yönsüz bir $G = (V, E)$ çizgesinin bir **hamilton çevrimi**, V deki her bir tepeyi içeren basit bir çevrimdir. Bir hamilton çevrimi içeren bir çizge **hamilton** olarak, aksi halde **hamilton olmayan** olarak adlandırılır. Bu isim, oniki yüzlü (Şekil 34.2(a)) üzerinde bir oyuncunun herhangi sıralı beş tepeye beş iğne batırdığı ve diğer oyuncunun da tüm tepeleri içeren bir çevrim oluşturmak



Şekil 34.2 (a) Taralı ayrıtlarla gösterilmiş bir hamilton çevrimi ile birlikte bir oniki yüzlünün tepelerini, ayrıtlarını ve yüzlerini temsil eden bir çizge. (b) Tek sayıda tepeye sahip iki parçalı bir çizge. Buna benzeyen herhangi bir çizge hamilton olmayan çizgedir.

için yolu tamamlamak zorunda olduğu bir matematiksel oyunu tanımlayan W. R. Hamilton’u onurlandırır.⁷ Oniki yüzlü hamiltondur ve Şekil 34.2(a) bir hamilton çevrimini gösterir. Bununla birlikte, bütün çizgeler hamilton değildir. Örneğin Şekil 34.2(b) tek sayıda tepe içeren iki parçalı bir çizgeyi gösterir. Alıştırma 34.2-2, bunun gibi olan tüm çizgelerin hamilton olmayan çizge olduğunu göstermenizi ister.

Hamilton-çevrim problemi, “Bir G çizgesi bir hamilton çevrimine sahip midir?”, formal bir dil olarak aşağıdaki gibi tanımlayabiliriz:

$$\text{HAM-CYCLE} = \{\langle G \rangle : G \text{ bir hamilton çizge}\}.$$

Bir algoritma HAM-CYCLE dilinin nasıl belirler? Bir $\langle G \rangle$ problem örneği verildiğinde, olası bir karar algoritması G nin tüm tepelerinin bütün permütasyonlarını listeler ve her bir permütasyonu, bir hamilton yolu olup olmadığını görmek için kontrol eder. Bu algoritmanın çalışma zamanı nedir? Eğer bir çizgenin “makul” kodlamasını onun bitişiklik matrisi olarak kullanırsak, $n = |\langle G \rangle|$, G nin kodlamasının uzunluğu olmak üzere, çizgedeki tepelerin sayısı m , $\Omega(\sqrt{n})$ dir. Te-

⁷17 Ekim 1856 yılında arkadaşı John. T. Graves’e yazdığı mektupta, Hamilton [157., 624. syf] şöyle demişti: Icosion’un sunduğu yeni bir matematik oyununu oynayan gençler oldukça eğleniyordu, bu oyunda bir kişi, herhangi beş ardışık noktaya beş iğne batırıyordu ... ve diğer oyuncu ise, bu mektupta yazılı olan teoriyle paralel şekilde, diğer tüm noktaları kapatmak için başka onbeş iğneyi dairesel ardılıkla yerleştirmeyi ve rakibinin başladığı yerdeki iğnenin hemen yakınında bitirmeyi amaçlıyordu.

pelerin $m!$ tane mümkün permütasyonu vardır ve dolayısıyla çalışma zamanı, herhangi bir k sabiti için $O(n^k)$ olmayıp, $\Omega(m!) = \Omega(\sqrt{n}!) = \Omega(2^{\sqrt{n}})$ dir. Bu nedenle, bu sade algoritma polinom zamanda çalışmaz. Aslında hamilton-çevrim problemi, Kısım 34.5 de kanıtlayacağımız gibi NP-tamdır.

Doğrulama algoritmaları

Biraz daha kolay bir problem düşünün. Farzedin ki bir arkadaşınız, size verilen G çizgesinin hamilton olduğunu söylüyor ve hamilton çevrimi boyunca sıra ile tepeleri size vererek bunu kanıtlamanızı teklif ediyor. Kanıtı doğrulamak için bu kesinlikle yeteri kadar kolay olurdu: V nin tepelerinin bir permütasyonu olup olmadığını ve çevrim boyunca ardışık ayrıtların her birinin gerçekten çizgede varolup olmadığını kontrol ederek, verilen çevrimin hamilton olduğunu basitçe doğrula. n , G nin kodlamasının uzunluğu olmak üzere, bu doğrulama algoritmasını kesinlikle $O(n^2)$ zamanda çalışması için uygulayabilirsiniz. Bu durumda, bir çizgede, bir hamilton çevriminin varolmasının kanıtı polinom zamanda doğrulanabilir.

Bir **doğrulama algoritmasını**, bir argümanı sıradan bir x katar girdisi ve diğeri bir **sertifika** olarak adlandırılan y ikili katarı olmak üzere, iki-argümanlı bir A algoritması olarak tanımlarız. İki-argümanlı bir A algoritması, $A(x,y) = 1$ olacak şekilde bir y sertifikası varsa, x katar girdisini **doğrular**. Bir doğrulama algoritması tarafından **doğrulan dil**

$L = \{x \in \{0,1\}^* : A(x,y) = 1 \text{ olacak şekilde } y \in \{0,1\}^* \text{ bulunur}\}$ dir.

Sezgisel olarak, herhangi bir $x \in L$ katarı için bir A algoritmasının $x \in L$ olduğunu kanıtlamak üzere kullanabildiği bir y sertifikası varsa, A algoritması bir L dilini doğrular. Ayrıca, herhangi bir $x \notin L$ katarı için $x \in L$ olduğunu kanıtlayan hiçbir sertifika olmamalıdır. Örneğin, hamilton-çevrim probleminde sertifika, bir hamilton çevrimindeki tepelerin listesidir. Bir çizge hamilton çizgeyse, hamilton çevrimi kendisi bu gerçeği doğrulamak için yeterli bilgiyi sunar. Diğer taraftan, bir çizge hamilton değilse, doğrulama algoritması emin olmak için verilen “çevrim”i dikkatlice kontrol ettiğinden tepelerin herhangi bir listesi olmayabilir, bu durum doğrulama algoritmasını çizgenin hamilton olduğuna inanmaya sevkeder.

NP karmaşıklık sınıfı

NP karmaşıklık sınıfı, polinom-zamanlı bir algoritma tarafından doğrulanabilen dillerin sınıfıdır.⁸ Daha iyi ifade etmek gerekirse, ancak ve ancak

$$L = \{ x \in \{0,1\}^* : A(x, y) = 1 \text{ olacak şekilde} \\ \text{bir } y \text{ sertifikası, } |y| = O(|x|^c), \text{ vardır} \}$$

olacak şekilde iki-girdili polinom-zamanlı bir A algoritması ve bir c sabiti mevcut ise bir L dili NP sınıfına aittir.

A algoritması, L dilini **polinom zamanlı doğrular** deriz.

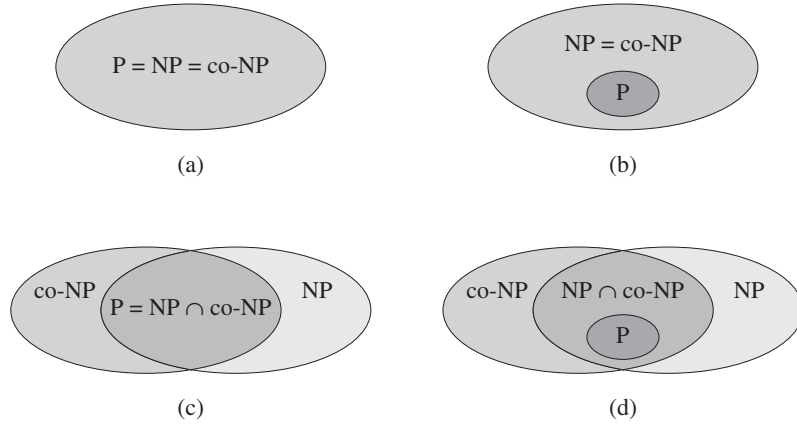
Hamilton-çevrim problemi üzerine daha önceki irdellemelerimizden yola çıkarak, şimdi $\text{HAM-CYCLE} \in \text{NP}$ olduğunu göreceğiz. (Önemli bir kümenin boş olmadığını bilmek her zaman güzeldir.) Ayrıca, $L \in \text{P}$ ise $L \in \text{NP}$ dir; çünkü L yi belirleyen polinom-zamanlı bir algoritma varsa bu algoritma, herhangi bir sertifikayı göz ardı eden ve tam olarak L de olduğunu belirlediği girdi katarlarını kabul eden iki-argümanlı bir algoritmaya kolaylıkla dönüştürülebilir. Bu nedenle, $\text{P} \subseteq \text{NP}$ dir.

$\text{P} = \text{NP}$ olup olmadığı meçhuldür; ama çoğu araştırmacı P ve NP nin aynı sınıf olmadığına inanmaktadır. Sezgisel olarak, P sınıfı hızlıca çözülebilen problemlerden oluşur. NP sınıfı, bir çözümü hızlıca doğrulanabilen problemlerden oluşur. Özellikle zaman kısıtları altında çalışırken, bir problemi sıfırdan çözmenin açıkça sunulan bir çözümü doğrulamaktan çoğunlukla daha zor olduğunu deneyimlerden öğrenmiş olabilirsiniz. Teorik bilgisayar bilimciler genellikle bu benzerliğin P ve NP sınıflarına genişlediğine ve dolayısıyla NP 'nin P 'de olmayan dilleri içerdiğine inanırlar.

$\text{P} \neq \text{NP}$ olduğuna dair daha zorlayıcı, yine de kesin olmayan bir bulgu vardır –“NP-tam” olan dillerin varoluşu. Bu sınıfı Kısım 34.3 de çalışacağız.

$\text{P} \neq \text{NP}$ sorusunun ötesinde diğer birçok temel soru çözümlenmemiş olarak kalır. Şekil 34.3 bazı olası senaryoları gösterir. Birçok araştırmacı tarafından yapılan çokça çalışmaya rağmen hiç kimse NP sınıfının tümleyen altında kapalı olup olmadığını bile bilmiyor. Başka bir deyişle, $L \in \text{NP}$, $\overline{L} \in \text{NP}$ anlamına gelir mi? **co-NP sınıfını**, $\overline{L} \in \text{NP}$ olan L dillerinin kümesi olarak tanımlayabiliriz. NP nin tümleyen altında kapalı olup olmadığı sorusunu, $\text{NP} = \text{co-NP}$ olup olmadığı olarak yeniden ifade edebiliriz. P , tümleme altında kapalı olduğu için (Alıştırma 34.1-6), Alıştırma 34.2-9 dan $\text{P} \subseteq \text{NP} \cap \text{co-NP}$ sonucu çıkar. Bununla birlikte, $\text{P} = \text{NP} \cap \text{co-NP}$ ya da $\text{NP} \cap \text{co-NP} = \text{P}$ de bir dilin var olup olmadığını yine hiç kimse bilmiyor.

⁸ “NP” ismi “belirsiz polinom zamanlı” yerine kullanılır. NP sınıfı aslen belirsizliğin içeriğinde çalışılmıştır; ama bu kitap biraz daha basit ancak eşdeğer olan doğrulama kavramını kullanır. Hopcroft ve Ullman [180], belirsiz hesaplama modelleri kullanarak NP-tamlığın iyi bir sunumunu vermiştir.



Şekil 34.3 Karmaşıklık sınıfları arasındaki ilişki için dört olasılık. Her bir grafikte, diğerini içeren bir bölge bir uygun-altküme bağıntısı belirtir. (a) $P = NP = \text{co-NP}$. Çoğu araştırmacı bu olasılığı en olası olmayan olarak görmektedir. (b) $NP = \text{co-NP}$, tümleyen altında kapalıysa $NP = \text{co-NP}$ dir; ama $P = NP$ olması gerekli değil. (c) $P = NP \cap \text{co-NP}$; ancak NP tümleyen altında kapalı değil. (d) $NP \neq \text{co-NP}$ ve $P \neq NP \cap \text{co-NP}$. Çoğu araştırmacı bu olasılığı en olası olarak görmektedir.

Dolayısıyla, P ve NP arasındaki tam ilişkiyi anlayışımız malesef eksiktir. Bununla beraber, belirli bir problemin baş edilemez olduğunu kanıtlamayı başarmasak bile, eğer NP -tam olduğunu kanıtlayabilirsek onun hakkında değerli bir bilgi elde etmiş oluruz.

Alıştırmalar

34.2-1

$\text{GRAPH-ISOMORPHISM} = \{\langle G_1, G_2 \rangle : G_1 \text{ ve } G_2 \text{ izomorfik çizgeler}\}$ dilini düşünün. Dili doğrulamak için polinom-zamanlı bir algoritma tanımlayarak $\text{GRAPH-ISOMORPHISM} \in NP$ olduğunu kanıtlayınız.

34.2-2

G , tek sayıda tepeye sahip yönsüz, iki parçalı bir çizgeyse, G nin hamilton olmayan çizge olduğunu kanıtlayınız.

34.2-3

$\text{HAM-CYCLE} \in P$ ise bir hamilton çevrimlerinin tepelerini sırayla listeleme probleminin polinom-zamanlı çözülebilir olduğunu gösteriniz.

34.2-4

Dillerin NP sınıfının birleşim, kesişim, art arda ekleme, tümleyen ve Kleene yıldızı işlemleri altında kapalı olduğunu kanıtlayınız. Tümleme altında NP nin kapanışını tartışınız.

34.2-5

NP'deki herhangi bir dilin, bir k sabiti için $2^{O(n^k)}$ zamanda çalışan bir algoritma tarafından belirlenebileceğini gösteriniz.

34.2-6

Bir çizgedeki bir *hamilton yolu* her tepeyi tam olarak bir kez ziyaret eden basit bir yoldur. $\text{HAM-PATH} = \{ \langle G, u, v \rangle : G \text{ çizgesinde } u \text{ dan } v \text{ ye bir hamilton yolu vardır} \}$ dilinin NP sınıfına ait olduğunu gösteriniz.

34.2-7

Alıştırma 34.2-6 daki hamilton-yolu probleminin yönlü çevrimsiz çizgeler üzerinde polinom zamanlı çözülebildiğini gösteriniz. Problem için etkin bir algoritma veriniz.

34.2-8

ϕ , boole girdi değişkenleri $x_1, x_2, \dots, x_k$, değiller ($\neg$), AND ler ($\wedge$), OR lar ($\vee$), ve parantezlerden inşa edilmiş bir boole formülü olsun. ϕ formülü, girdi değişkenlerinin her 1 ve 0 ataması için 1 değerini hesaplırsa ϕ bir **totolojidir**. TAUTOLOGY yi totoloji olan boole formüllerinin dili olarak tanımlayınız. TAUTOLOGY $\in$ co-NP olduğunu gösteriniz.

34.2-9

$P \subseteq \text{co-NP}$ olduğunu kanıtlayınız.

34.2-10

$\text{NP} \neq \text{co-NP}$ ise $P \neq \text{NP}$ olduğunu kanıtlayınız.

34.2-11

G , en az 3 tepeli bağlantılı, yönsüz bir çizge olsun ve G^3 , G de en fazla 3 uzunluklu bir yolla bağlantılı tüm tepe çiftlerini birleştirerek elde edilen çizge olsun. G^3 ün hamilton olduğunu kanıtlayınız (*İpucu: G için bir kapsayan ağaç inşa edin ve tümevarımsal bir argüman kullanın*)

34.3 NP-tamlık ve indirgenebilirlik

Teorik bilgisayar bilimcilerin $P \neq NP$ olduğuna inanmalarının en zorlayıcı nedeni muhtemelen “NP-tam” problemlerin sınıfının varlığından gelmektedir. Bu sınıfın şaşırtıcı bir özelliği vardır, eğer *herhangi* bir NP-tam problem polinom zamanda çözülebilirse NP’deki *her* problem polinom-zamanlı bir çözüme sahiptir, yani $P = NP$ dir. Bununla birlikte, yılların çalışmasına rağmen herhangi bir NP-tam problem için şimdiye kadar hiçbir polinom-zamanlı algoritma keşfedilememiştir.

HAM-CYCLE dili bir NP-tam problemidir. Eğer HAM-CYCLE ne polinom zamanda karar verebilseydik NP’deki her problemi polinom zamanda çözebilirdik. Aslında NP-P, boş olmayan hale dönüşseydi kesinlikle HAM-CYCLE $\in$ NP-P diyebilirdik.

NP-tam dilleri bir anlamda NP’deki “en zor” dillerdir. Bu bölümde, “polinom-zamanlı indirgenebilirlik” olarak adlandırılan belirli bir kavramı kullanarak dillerin göreceli “zorluğunun” nasıl karşılaştırılacağını göstereceğiz. Daha sonra NP-tam dilleri formal olarak tanımlayıp; CIRCUIT-SAT olarak adlandırılan böyle bir dilin NP-tam olduğunun kanıtının taslağını yaparak bitireceğiz. Kısım 34.4 ve Kısım 34.5 de, diğer birçok problemin NP-tam olduğunu göstermek için indirgenebilirlik kavramını kullanacağız.

İndirgenebilirlik

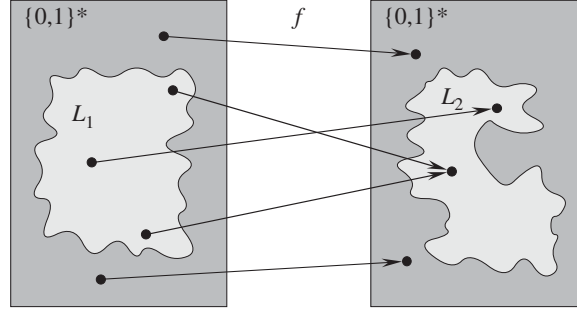
Sezgisel olarak, bir Q probleminin herhangi bir örneği, Q' probleminin bir örneği olarak “kolaylıkla ifade edilebilir” ise Q problemi, Q nun örneğine bir çözüm temin eden Q' problemine indirgenebilir. Örneğin, belirsiz bir x için doğrusal denklemleri çözme problemi, ikinci dereceden denklemleri çözme problemine indirgenir. $ax + b = 0$ örneği verildiğinde bu denklemi, çözümünü $ax + b = 0$ için bir çözüm sağlayan $0x^2 + ax + b = 0$ denklemine dönüştürürüz. Dolayısıyla, bir Q problemi bir diğer Q' problemine indirgenirse, bir anlamda, Q , Q' problemine “çözmekten daha zor değildir”.

Karar problemleri için formal-dil yapımıza dönecek olursak, eğer her $x \in \{0, 1\}^*$ için

$$x \in L_1 \text{ ancak ve ancak } f(x) \in L_2 \quad (34.1)$$

olacak şekilde, polinom-zamanda hesaplanabilir bir $f: \{0, 1\}^* \rightarrow \{0, 1\}^*$ fonksiyonu varsa bir L_1 dili bir L_2 diline **polinom-zamanda indirgenebilir** deriz. f fonksiyonunu **indirgeme fonksiyonu** olarak ve f yi hesaplayan polinom-zamanlı bir F algoritmasını bir **indirgeme algoritması** olarak adlandırırız.

Şekil 34.4, bir L_1 dilinden bir diğer L_2 diline polinom-zamanlı bir indirgeme fikrini tasvir eder. Her bir dil $\{0, 1\}^*$ kümesinin bir altkümesidir. f indirgeme fonksiyonu polinom-zamanlı bir eşleme sağlar, öyle ki $x \in L_1$ ise $f(x) \in L_2$ dir.



Şekil 34.4 Bir f indirgeme fonksiyonu aracılığıyla bir L_1 dilinden bir L_2 diline polinom-zamanlı bir indirgemenin tasviri. Herhangi bir $x \in \{0,1\}^*$ için $x \in L_1$ olup olmadığı sorusunun yanıtı $f(x) \in L_2$ olup olmadığı sorusunun yanıtıyla aynıdır.

Ayrıca $x \notin L_1$ ise $f(x) \notin L_2$ dir. Dolayısıyla, indirgeme fonksiyonu L_1 ile gösterilen karar probleminin herhangi bir x örneğini L_2 ile gösterilen problemin bir $f(x)$ örneğine eşler. $f(x) \in L_2$ olup olmadığına bir yanıt sağlamak, doğrudan $x \in L_1$ olup olmadığına bir yanıt sağlar.

Polinom-zamanlı indirgemeler, çeşitli dillerin P sınıfına ait olduğunu kanıtlamak için bize güçlü bir araç verir.

Lemma 34.3

$L_1, L_2 \subseteq \{0,1\}^*$ dilleri için $L_1 \leq_p L_2$ ise, $L_2 \in P$ olması $L_1 \in P$ olmasını gerektirir.

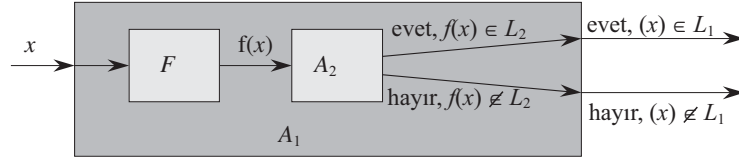
Kanıt A_2, L_2 yi belirleyen polinom-zamanlı bir algoritma ve F, f indirgeme fonksiyonunu hesaplayan polinom-zamanlı bir indirgeme algoritması olsun. L_1 e karar veren polinom-zamanlı bir A_1 algoritması oluşturacağız.

Şekil 34.5, A_1 i nasıl oluşturacağımızı tasvir eder. Verilen bir $x \in \{0, 1\}^*$ girdisi için A_1 algoritması, x girdisini $f(x)$ e dönüştürmek amacıyla F yi kullanır ve ardından $f(x) \in L_2$ olup olmadığını test etmek için A_2 yi kullanır. A_1 algoritması, A_2 algoritmasından çıktıyı alır ve kendi çıktısı olarak o yanıtı üretir.

A_1 algoritmasının doğruluğu (34.1) koşulunun sonucu olarak çıkar. Hem F hem de A_2 polinom zamanda çalıştığı için A_1 algoritması da polinom zamanda çalışır (Alıştırma 34.1-5 e bakınız). ■

NP-tamlık

Polinom-zamanlı indirgemeler, bir problemin en az diğeri kadar zor olduğunu, bir polinom-zaman faktörü dahilinde, göstermek için formal bir araç sağlar. Yani, $L_1 \leq_p L_2$ ise L_1, L_2 den bir polinomiyal faktörden daha fazla zor değildir, bu da



Şekil 34.5 Lemma 34.3 ün kanıtı. F algoritması, L_1 den L_2 ye f indirgeme fonksiyonunu polinom zamanda hesaplayan bir indirgeme algoritması ve A_2 , L_2 ye karar veren polinom-zamanlı bir algoritmadır. A_1 algoritması, herhangi bir x girdisini $f(x)$ e dönüştürmek amacıyla F yi kullanarak ve $f(x) \in L_2$ olup olmadığını belirlemek için A_2 algoritmasını kullanarak $x \in L_1$ olup olmadığını belirler.

indirgeme için “daha küçük ya da eşit” simgesinin neden anımsatıcı olduğunu belirtir. Şimdi NP’deki en zor problemler olan NP-tam problemler sınıfını tanımlayabiliriz.

Bir $L \subseteq \{0, 1\}^*$ dili,

1. $L \in \text{NP}$ ve
2. Her $L' \in \text{NP}$ için $L' \leq_p L$

koşullarını sağlıyorsa **NP-tamdır**. Bir L dili 2. özelliği sağlıyorsa, 1. özelliği sağlaması gerekli değil, L dili **NP-zordur** deriz. Ayrıca NPC’yi NP-tam dillerin sınıfı olarak tanımlarız.

Aşağıdaki teoremin gösterdiği üzere, NP-tamlık P’nin gerçekte NP’ye eşit olup olmadığını belirlemenin merkezindedir.

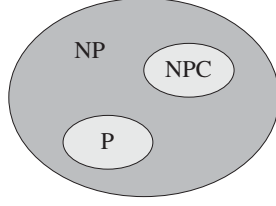
Teorem 34.4

Herhangi bir NP-tam problem polinom-zamanlı çözülebilir ise, $P = \text{NP}$ dir. Denk olarak, NP’deki herhangi bir problem polinom-zamanlı çözülemez ise hiçbir NP-tam problem polinom-zamanlı çözülebilir değildir.

Kanıt $L \in P$ ve ayrıca $L \in \text{NPC}$ olduğunu varsayalım. NP-tamlık tanımının 2. özelliğinden, herhangi bir $L' \in \text{NP}$ için $L' \leq_p L$ olur. Dolayısıyla Lemma 34.3 den, $L' \in P$ olur ve teoremin ilk ifadesi kanıtlanır.

İkinci ifadeyi kanıtlamak için onun ilk ifadenin karşıt tersi olduğuna dikkat ediniz. ■

$P \neq \text{NP}$ sorusu araştırmasının NP-tam problemler etrafında merkezlenmesi bu nedenden kaynaklanır. Çoğu kuramsal bilgisayar bilimci, P , NP ve NPC arasındaki Şekil 34.6 da gösterilen ilişkilere sebep olan $P \neq \text{NP}$ ye inanır. Ancak, bildiğimiz kadarıyla, biri yine de bir NP-tam problem için polinom-zamanlı bir algoritma ortaya koyabilir, bu sayede $P = \text{NP}$ olduğunu kanıtlar. Bununla birlikte, herhangi bir NP-tam problem için henüz hiçbir polinom-zamanlı algoritma keşfe-



Şekil 34.6 Çoğu teorik bilgisayar bilimcinin P, NP ve NPC arasındaki ilişkilere nasıl baktığı. Hem P hem de NPC, NP tarafından tamamen kapsanmaktadır ve $P \cap NPC = \emptyset$ dir.

dilmemiş olduğundan, bir problemin NP-tam olduğunun kanıtı onun başedilemez olduğuna dair kusursuz bir delil sağlar.

Devresel gerçekleştirilebilirlik

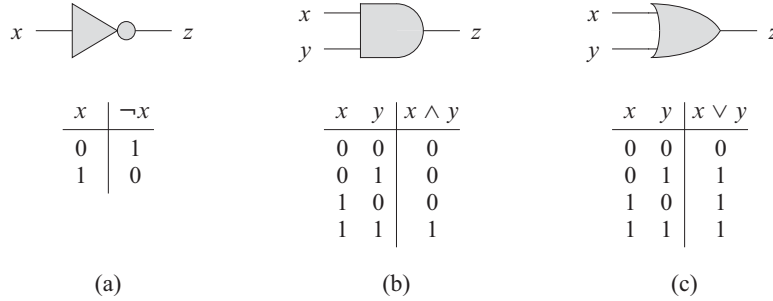
Bir NP-tam problem kavramını tanımladık; ancak buraya kadar herhangi bir problemin NP-tam olduğunu gerçekte kanıtlamadık. En az bir problemin NP-tam olduğunu bir kere kanıtladığımızda, diğer problemlerin NP-tam olduğunu kanıtlamak için polinom-zamanlı indirgenebilirliği bir araç olarak kullanabiliriz. Dolayısıyla, şimdi bir NP-tam problemin varlığını göstermeye odaklanıyoruz: devresel-gerçekleştirilebilirlik problemi.

Malesef, devresel-gerçekleştirilebilirlik probleminin NP-tam olduğunun formal kanıtı bu metnin kapsamının ötesinde teknik detaylar gerektirir. Bunun yerine, boole birleşimsel devrelerin temel bir anlayışına dayanan bir kanıtı informal olarak tanımlayacağız.

Boole birleşimsel devreler, kablolarla birbirine bağlanmış boole birleşimsel elemanlardan inşa edilir. Bir **boole birleşimsel eleman**, sabit sayıda boole girdi ve çıktılarına sahip ve iyi-tanımlı bir fonksiyonu yürüten herhangi bir devre elemanıdır. Boole değerleri, 0 ın YANLIŞ 1, 1 in DOĞRU yu ifade ettiği $\{0, 1\}$ kümesinden çekilir.

Devresel-gerçekleştirilebilirlik probleminde kullandığımız boole birleşimsel elemanlar, basit boole fonksiyonları hesaplarlar ve **mantık kapıları** olarak bilinirler. Şekil 34.7, devresel-gerçekleştirilebilirlik probleminde kullandığımız üç temel mantık kapısını gösterir: **NOT kapısı** (ya da **tersleyici**), **AND kapısı** ve **OR kapısı**. NOT kapısı, değeri 0 ya da 1 olan tek bir ikili x **girdisi** alır ve değeri, girdi değerinin tersi olan bir ikili z **çıkışı** üretir. Diğer iki kapının her biri iki tane ikili x ve y girdisi alır ve tek bir ikili z çıkışı üretir.

Her bir kapının ve herhangi bir boole birleşimsel elemanın işlemini Şekil 34.7 de her bir kapının altında gösterilen bir **doğruluk tablosu** ile tanımlarız. Bir doğruluk tablosu, girdilerin her bir mümkün kümelenmesi için birleşimsel elemanın



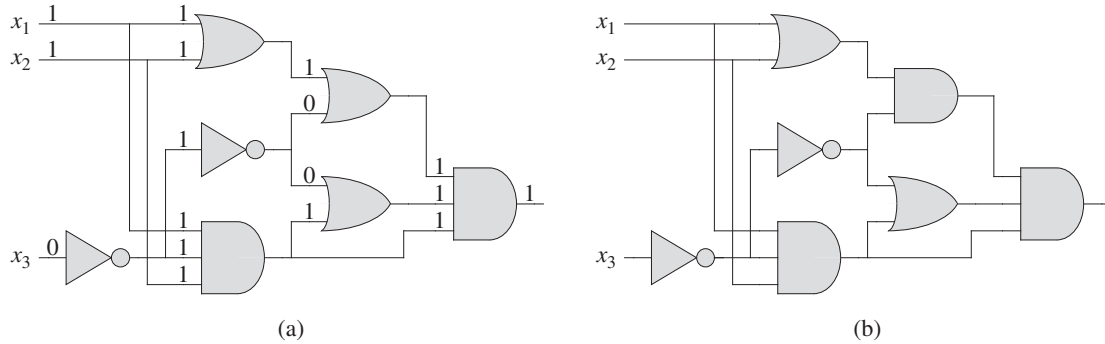
Şekil 34.7 İkili girdi ve çıktılarla üç temel mantık kapısı. Her bir kapının altı, kapının işlemini tanımlayan doğruluk tablosudur. **(a)** NOT kapısı. **(b)** AND kapısı. **(c)** OR kapısı.

çıktılarını verir. Örneğin, OR kapısı için doğruluk tablosu bize, girdiler $x=0$ ve $y=1$ olduğunda çıktı değerinin $z=1$ olduğunu söyler. $\neg$ sembolünü NOT fonksiyonunu belirtmek için $\wedge$ sembolünü AND fonksiyonunu belirtmek için ve $\vee$ sembolünü OR fonksiyonunu belirtmek için kullanırız. Bu nedenle örneğin, $0 \vee 1 = 1$.

AND ve OR kapılarını ikiden daha fazla girdi alacak şekilde genelleleyebiliriz. Bir AND kapısının çıktısı, eğer tüm girdiler 1 ise 1, aksi halde 0'dır. Bir OR kapısının çıktısı, girdilerden herhangi biri 1 ise 1, aksi halde 0'dır.

Bir **boole birleşimsel devre**, **kablolar** ile birbirine bağlanmış bir ya da daha fazla boole birleşimsel elemandan oluşur. Bir kablo, ilk elemanın çıktı değerini ikincinin girdi değeri olarak sunmak suretiyle, bir elemanın çıktısını diğerinin girdisine bağlayabilir. Şekil 34.8, sadece bir kapıda farklılaşan, iki benzer boole birleşimsel devreleri gösterir. Şeklin (a) kısmı ayrıca, $\langle x_1 = 1, x_2 = 1, x_3 = 0 \rangle$ girdisi verildiğinde birbirinden ayrı kablolar üzerindeki değerleri gösterir. Bir tekil kablo, ona bağlı birden daha fazla birleşimsel-eleman çıktısına sahip olamasa dahi, çeşitli eleman girdilerini besleyebilir. Bir kablo tarafından beslenen eleman girdilerinin sayısı, kablonun **çıkış yelpazesi** olarak adlandırılır. Hiçbir eleman çıktısı bir kabloya bağlı değilse, kablo, bir dış kaynaktan girdi değerleri kabul eden bir **devre girdisi**dir. Hiçbir eleman girdisi bir kabloya bağlı değilse, kablo, devrenin hesaplamasının sonuçlarını dış dünyaya temin eden bir **devre çıktısı**dır. (Bir dahili kablo, bir devre çıktısına da yayılabilir.) Asıl donanım tasarımında bir boole birleşimsel devresi birçok çıktıya sahip olabilmesine rağmen, devresel-gerçekleştirilebilirlik problemini tanımlama amacı için devre çıktıların sayısını 1'e kısıtlarız.

Boole birleşimsel devreleri çevrim içermez. Başka bir deyişle, her bir birleşimsel eleman için bir tepe ve çıkış yelpazesi k olan her bir kablo için k tane yönlü ayrıt ile bir yönlü $G = (V, E)$ çizgesi oluşturduğumuzu varsayalım; bir kablo u elemanının çıktısını v elemanının bir girdisine bağlarsa G çizgesi yönlü bir (u, v) ayrıtı içerir. Öyleyse G çevrimsiz olmalıdır.



Şekil 34.8 Devresel-gerçekleştirilebilirlik probleminin iki örneği. **(a)** Bu devrenin girdilere $\langle x_1 = 1, x_2 = 1, x_3 = 0 \rangle$ ataması devrenin çıktısının 1 olmasına neden olur. Devre bu nedenle tatmin edilebilir. **(b)** Bu devrenin girdilere hiçbir ataması devrenin çıktısının 1 olmasına yol açmaz. Devre dolayısıyla gerçekleştirilemezdir.

Bir boole birleşimsel devresi için bir **doğruluk ataması**, boole girdi değerlerinin bir kümesidir. Bir tek-çıkıtlı boole birleşimsel devresi, eğer bir **gerçekleşen atamaya** sahipse, **gerçekleştirilebilirdir**: devrenin çıktısının 1 olmasına yol açan bir doğruluk ataması. Örneğin, Şekil 34.8(a) daki devre $\langle x_1 = 1, x_2 = 1, x_3 = 0 \rangle$ gerçekleşen atamaya sahiptir ve dolayısıyla gerçekleştirilebilirdir. Alıştırma 34.3-1 in göstermenizi istediği gibi, x_1 , x_2 ve x_3 e hiçbir değer ataması Şekil 34.8(b) deki devrenin bir 1 çıktısı üretmesine yol açmaz ve bu nedenle de gerçekleştirilemezdir.

Devresel-gerçekleştirilebilirlik problemi, “AND, OR ve NOT kapılarından oluşmuş bir boole birleşimsel devresi verildiğinde, bu devre tatmin edilebilir midir?” dir. Bununla birlikte, bu soruyu formal olarak ortaya çıkarmak amacıyla devreler için standart bir kodlama üzerinde anlaşmalıyız. Bir boole birleşimsel devrenin boyutu, boole birleşimsel elamanların sayısı artı devredeki kabloların sayısıdır. Verilen herhangi bir C devresini, uzunluğu devrenin kendisinin boyutunda polinom olan bir ikili $\langle C \rangle$ katarına eşleyen çizge benzeri bir kodlama tasarlayabiliriz. Formal bir dil olarak, dolayısıyla

$\text{CIRCUIT-SAT} = \{ \langle C \rangle : C \text{ gerçekleştirilebilir bir boole birleşimsel devredir} \}.$ tanımlayabiliriz.

Devresel-gerçekleştirilebilirlik problemi bilgisayar-destekli donanım optimizasyonu alanında ortaya çıkar. Bir altdevre sürekli 0 üretirse, o altdevre gereksizdir; tasarımcı onu, tüm mantık kapılarını ihmal eden ve sabit 0 değerini çıktısı olarak sağlayan daha basit bir altdevre ile değiştirebilir. Bu problem için neden polinom-zamanlı bir algoritmaya sahip olmak istediğimizi anlayabilirsiniz.

Bir C devresi verildiğinde, girdilere tüm mümkün atamaları basitçe kontrol ederek, gerçekleştirilebilir olup olmadığını belirleme girişiminde bulunabiliriz.

Ne yazık ki, devre k girdiye sahipse sayısı 2^k ya varan mümkün atamaları kontrol etmeliyiz. C nin boyutu k ye bağlı polinomial olduğunda, her birini kontrol etmek, $\Omega(2^k)$ zaman alır ki bu da devrenin boyutunda superpolinomdur.⁹ Aslında, iddia ettiğimiz üzere, devresel-gerçekleştirilebilirlik problemini çözen hiçbir polinom-zamanlı algoritmanın var olmadığına dair güçlü kanıtlar vardır; çünkü devre tatmin edilebilirlik NP-tamdır. Bu olgunun kanıtını NP-tamlık tanımının iki kısmına dayanarak iki parçaya ayırırız.

Lemma 34.5

Devresel-gerçekleştirilebilirlik problemi NP sınıfına aittir.

Kanıt CIRCUIT-SAT dilini doğrulayabilen iki-girdili, polinom-zamanlı bir A algoritması sağlayacağız. A nın girdilerinden biri bir C boole birleşimsel devresidir (onun standart kodlaması). Diğer girdi, boole değerlerinin C deki kabloları bir atamasına ilişkin bir sertifikadır (Daha küçük bir sertifika için Alıştırma 34.3-4 e bakınız).

A algoritmasını aşağıdaki gibi inşa ederiz. Algoritma, devredeki her bir mantık kapısı için çıktı kablosu üzerindeki sertifika tarafından sağlanan değer, girdi kabloları üzerindeki değerlerin bir fonksiyonu olarak doğru şekilde hesaplanıp hesaplanmadığını denetler. Daha sonra, eğer bütün devrenin çıktısı 1 ise algoritma 1 çıktısını verir; çünkü C nin girdilerine atanan değerler gerçekleşen bir atama sağlar. Aksi halde, A , 0 çıktısını verir.

Gerçekleştirilebilir bir C devresi A algoritmasının girdisi olduğunda, uzunluğu C nin boyutunda polinom olan ve A nın bir 1 çıktısı vermesine neden olan bir sertifika vardır. Gerçekleştirilemez bir devre girdi olarak verildiğinde, hiçbir sertifika A yı devrenin gerçekleştirilebilir olduğuna inanması için kandıramaz. A algoritması polinom zamanda çalışır: iyi bir uygulama ile doğrusal zamanlı yeterli gelir. Dolayısıyla, CIRCUIT-SAT ı polinom zamanda doğrulayabiliriz ve CIRCUIT-SAT $\in$ NP dir. ■

CIRCUIT-SAT ın NP-tam olduğu kanıtının ikinci kısmı, dilin NP-zor olduğunu göstermektir. Yani, NP deki her dilin CIRCUIT-SAT a polinom-zamanlı indirgenebilir olduğunu göstermeliyiz. Bu olgunun asıl kanıtı teknik karmaşıklıklarla doludur, bu nedenle bilgisayar donanımı çalışmalarının bazı anlayışlarına dayanan, kanıtın bir taslağına razı olacağız.

Bir bilgisayar programı, bilgisayar belleğinde bir talimatlar dizisi olarak saklanır. Tipik bir talimat, yürütülecek bir işlemi, bellekteki işlenenlerin adreslerini ve sonucun saklanacağı bir adresi kodlar. **Program sayacı** denilen özel bir bellek

⁹Diğer yandan, C nin boyutu $\Theta(2^k)$ ise çalışma zamanı $O(2^k)$ olan bir algoritma, devre boyutunun polinomunda çalışma zamanına sahiptir. $P \neq NP$ olsa bile bu durum problemin NP-tamlığı ile çelişmeyecektir; özel bir durum için polinom-zamanlı bir algoritmanın varlığı, bütün durumlar için polinom-zamanlı bir algoritma olmasını gerektirmez.

yeri, bir sonra uygulanacak olan talimatı takip eder. Program sayacı, yerine getirilen her bir talimatta otomatik olarak artar, böylelikle bilgisayarın talimatları sırasıyla uygulamasına neden olur. Bununla birlikte, bir talimatın icrası bir değerin program sayacına yazılmasına neden olabilir; bu da normal ardışık yürütmeyi değiştirir ve bilgisayarın döngüye girmesine ve diğer dalları uygulamasına izin verir.

Bir programın yürütülmesi boyunca herhangi bir noktada, bilgisayarın belleği hesaplamaların tüm durumunu tutar. (Belleğin, programın kendisini, program sayacını, çalışma belleğini ve bir bilgisayarın saymanlık için devam ettirdiği çeşitli durum bitlerinden herhangi birini içerdiğini varsayalım). Bilgisayar belleğinin herhangi belirli bir durumunu **yapılanma** olarak adlandırırız. Bir talimatın uygulanmasını bir yapılanmadan diğerine eşleme olarak görebiliriz. Bu eşlemeyi yerine getiren bilgisayar donanımı, aşağıdaki lemmanın kanıtında M ile belirttiğimiz bir boole birleşimsel devre olarak uygulanabilir.

Lemma 34.6

Devresel-gerçekleştirilebilirlik problemi NP-zordur.

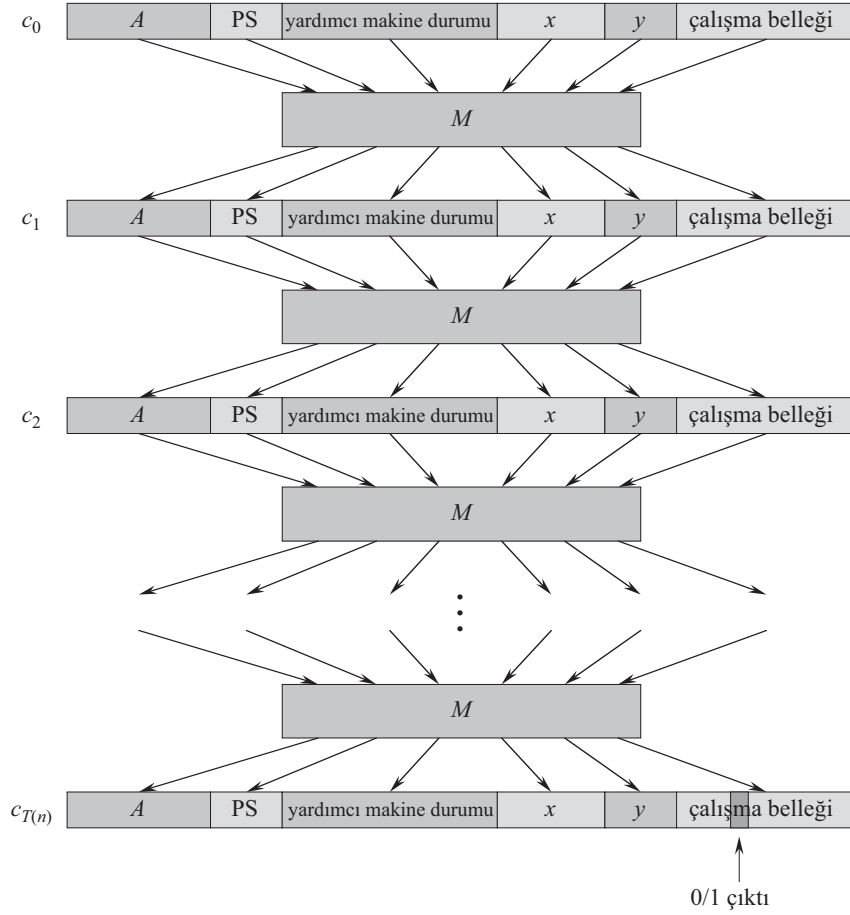
Kanıt L , NP'deki herhangi bir dil olsun. Her ikili x katarını bir $C = f(x)$ devresine eşleyen bir f indirgeme fonksiyonunu hesaplayan polinom-zamanlı bir F algoritması tanımlayacağız; öyle ki $x \in L$ ancak ve ancak $C \in \text{CIRCUIT-SAT}$ dir.

$L \in \text{NP}$ olduğundan L yi polinom zamanlı doğrulayan bir A algoritması var olmalıdır. İnşa edeceğimiz F algoritması, f indirgeme fonksiyonunu hesaplamak için iki-girdili A algoritmasını kullanır.

$T(n)$, n -uzunluklu girdi katarları için A algoritmasının en kötü çalışma zamanını belirtsin ve $k \geq 1$ bir sabit sayı olsun öyle ki $T(n) = O(n^k)$ ve sertifikanın uzunluğu $O(n^k)$. (A nın çalışma zamanı aslında toplam girdi boyutunda polinomdur, bu da hem bir girdi katarı hem de bir sertifikayı içerir; ancak sertifikanın uzunluğu, girdi katarının n uzunluğunun polinomu olduğundan çalışma zamanı n ye bağlı polinomdur.)

Kanıtın temel fikri A nın hesaplamasını yapılanmaların bir listesi olarak belirtmektir. Şekil 34.9 un açıkladığı gibi, her bir yapılanmayı, A için olan program, program sayacı ve yardımcı makine durumu, x girdisi, y sertifikası ve çalışma belleğinden oluşan parçalara ayırabiliriz. Bilgisayar donanımını uygulayan birleşimsel M devresi, c_0 başlangıç yapılanmasından başlayarak her bir c_i yapılanmasını bir sonraki c_{i+1} yapılanmasına eşler. A algoritması yürütmeyi bitirir bitirmez çıktısını -0 ya da 1 – tayin edilen bir yere yazar ve bundan sonra A nın durduğunu varsayarsak bu değer asla değişmez. Dolayısıyla, algoritma en fazla $T(n)$ adım çalışırsa, çıktı $c_{T(n)}$ deki bitlerin biri olarak görünür.

F indirgeme algoritması, verilen bir başlangıç yapılanmasından üretilen bütün yapılanmaları hesaplayan tek bir birleşimsel devreyi inşa eder. Burada fikir M



Şekil 34.9 Bir x girdisi ve y sertifikası üzerinde çalışan bir A algoritması tarafından üretilen yapılanmalar dizisi. Her bir yapılanma, hesaplamanın bir adımı için bilgisayarın durumunu ve ayrıca A , x ve y yi gösterir; program sayacı (PS), yardımcı makine durumu ve çalışma belleğini içerir. y sertifikası hariç c_0 başlangıç yapılanması sabittir. Bir boole birleşimsel M devresi her bir yapılanmayı bir sonraki yapılanmaya eşler. Çıktı, çalışma belleğinde ayrılmış bir bittir.

devresinin $T(n)$ kopyasını birlikte yapıştırmaktır. c_i yapılanmasını üreten i . devrenin çıktısı $(i + 1)$. devrenin girdisini doğrudan besler. Dolayısıyla, yapılanmalar, bilgisayarın belleğinde saklanmaktan ziyade M nin kopyalarını bağlayan kablolar üzerinde değerler olarak bulunurlar.

Polinom-zamanlı F indirgeme algoritmasının ne yapması gerektiğini hatırlayın. Bir x girdisi verildiğinde, ancak ve ancak $A(x, y) = 1$ olacak şekilde bir y sertifikası varsa gerçekleştirilebilir olan bir $C = f(x)$ devresi hesaplanabilir. F , bir x girdisi elde ettiğinde ilk olarak $n = |x|$ i hesaplar ve M nin $T(n)$ kopyasını içeren bir C' birleşimsel devresi inşa eder. C' devresinin girdisi $A(x, y)$ üzerindeki bir hesaplamaya ilişkin bir başlangıç yapılanmasıdır, çıktısı ise $c_{T(n)}$ yapılanmasıdır.

F algoritması, $C = f(x)$ devresini kurmak için C' devresini kısmen değiştirir. İlk olarak, A için olan programa ilişkin C' girdilerini, başlangıç program sayacını, x girdisini ve belleğin başlangıç durumunu doğrudan bu bilinen değerlere bağlar. Dolayısıyla, devrenin sadece kalan girdileri y sertifikasına karşılık gelir. İkinci olarak, A nın çıktısına ilişkin $c_{T(n)}$ nin tek biti hariç, C' devresinden gelen tüm çıktıları ihmal eder. Bu şekilde kurulmuş bu C devresi $O(n^k)$ uzunluklu herhangi bir y girdisi için $C(y) = A(x, y)$ hesaplar. F indirgeme algoritması, bir x katar girdisi sağlandığında böyle bir C devresini hesaplar ve çıktı olarak verir.

İki özelliği kanıtlamamız gerekiyor. Öncelikle, F nin bir f indirgeme fonksiyonunu doğru olarak hesapladığını göstermeliyiz. Yani, C devresinin ancak ve ancak $A(x, y) = 1$ olacak şekilde bir y sertifikası varsa gerçekleştirilebilir olduğunu göstermeliyiz. Ardından F nin polinom zamanda çalıştığını göstermeliyiz.

F nin bir indirgeme fonksiyonunu doğru olarak hesapladığını göstermek için $A(x, y) = 1$ olacak şekilde $O(n^k)$ uzunluklu bir y sertifikasının var olduğunu varsayalım. y nin bitlerini C nin girdilerine uygularsak C nin çıktısı $C(y) = A(x, y) = 1$ dir. Dolayısıyla, bir sertifika mevcutsa C gerçekleştirilebilir. Diğer yön için C nin gerçekleştirilebilir olduğunu farzedelim. Bu durumda, $C(y) = 1$ olacak şekilde, C ye bir y girdisi vardır, buradan $A(x, y) = 1$ sonucuna varırız. Böylece, F bir indirgeme fonksiyonunu doğru olarak hesaplar.

Kanıt taslağını tamamlamak için sadece F nin $n = |x|$ in polinomunda çalıştığını göstermemiz gerekiyor. Yaptığımız ilk gözlem, bir yapılanmayı göstermek için gerekli bitlerin sayısının n nin polinomu olmasıdır. A için olan programın kendisi, x girdisinin uzunluğundan bağımsız olan, sabit boyuta sahiptir. x girdisinin uzunluğu n ve y sertifikasının uzunluğu $O(n^k)$ dir. Algoritma en fazla $O(n^k)$ adım için çalıştığından A tarafından gerek duyulan çalışma belleği de n nin polinomudur. (Bu belleğin bitişik olduğunu farzederiz; Alıştırma 34.3-5 de, bu argümanı, A tarafından erişilen yerlerin çok daha büyük bir bellek bölgesi boyunca yayıldığı ve belirli bir yayılma deseninin her bir x girdisi için farklı olabildiği duruma genişletmeniz istenmektedir.)

Bilgisayar donanımını uygulayan birleşimsel M devresi, bir yapılanmanın uzunluğunda polinom boyuta, $O(n^k)$, sahiptir; dolayısıyla M nin boyutu n nin polinomudur. (Bu devre elemanlarının çoğu bellek sisteminin mantığını uygular.) C

devresi, M nin en fazla $t = O(n^k)$ kopyasından oluşur ve bu yüzden n nin polinomu boyuta boyuta sahiptir. F indirgeme algoritması, C yi x den polinom zamanda inşa edebilir; çünkü kurulumun her bir adımı polinom zaman alır. ■

CIRCUIT-SAT dili, dolayısıyla en az NP'deki herhangi bir dil kadar zordur ve kendisi de NP'ye ait olduğundan, NP-tamdır.

Teorem 34.7

Devresel-gerçekleştirilebilirlik problemi NP-tamdır.

Kanıt Lemma 34.5 ve 34.6 dan ve NP-tamlık tanımından gelir. ■

Alıştırmalar

34.3-1

Şekil 34.8(b) deki devrenin gerçekleştirilemez olduğunu gösteriniz.

34.3-2

$\leq_p$ bağıntısının diller üzerinde geçişli bir bağıntı olduğunu gösteriniz. Yani, $L_1 \leq_p L_2$ ve $L_2 \leq_p L_3$ ise $L_1 \leq_p L_3$ olduğunu gösteriniz.

34.3-3

$L \leq_p \bar{L}$ ancak ve ancak $\bar{L} \leq_p L$ olduğunu kanıtlayınız.

34.3-4

Lemma 34.5 in alternatif bir kanıtında bir sertifika olarak gerçekleşen bir atama kullanabildiğimizi gösteriniz.

34.3-5

Lemma 34.6 nın kanıtı, A algoritması için çalışma belleğinin polinom boyutlu bitişik bir bölgeyi işgal ettiğini varsayar. Kanıtın neresinde bu varsayımı kullanırsınız? Bu varsayımın hiçbir genellik kaybına yol açmadığını tartışınız.

34.3-6

Eğer $L \in C$ ve her $L' \in C$ için $L' \leq_p L$ ise bir L dili, polinom-zamanlı indirgemelere göre bir C dil sınıfı için **tamdır**. $\emptyset$ ve $\{0, 1\}^*$ dillerinin polinom-zamanlı indirgemelere göre P için tam olmayan P deki tek diller olduğunu gösteriniz.

34.3-7

Polinom-zamanlı indirgemelere göre (Alıştırma 34.3-6 ya bakınız) $\bar{L}$, co-NP için tamdır ancak ve ancak L nin de NP için tam olduğunu gösteriniz.

34.3-8

Lemma 34.6 daki F indirgeme algoritması, x , A ve k nin bilgisine dayalı olarak $C = f(x)$ devresini inşa eder. Professor Sartre, x katarının F nin girdisi olduğunu A , k ve $O(n^k)$ çalışma zamanının içindeki sabit çarpanın sadece varlığının F tarafından bilindiğini (L dili NP ye ait olduğundan), ancak gerçek değerlerinin bilinmediğini gözlemler. Dolayısıyla, profesör F nin muhtemelen C devresini inşa edemediğini ve CIRCUIT-SAT dilinin illa ki NP-zor olmadığı sonucuna varır. Profesörün mantığındaki hatayı açıklayınız.

34.4 NP-tamlık kanıtları

Devresel-gerçekleştirilebilirlik probleminin NP-tam olduğunu her $L \in \text{NP}$ dili için $L \leq_p \text{CIRCUIT-SAT}$ olduğunun doğrudan kanıtı ile kanıtladık. Bu bölümde, NP'deki *her* dili verilen dile doğrudan indirgemeksizin, dillerin NP-tam olduğunun nasıl kanıtlandığını göstereceğiz. Bu metodolojiyi çeşitli formül-gerçekleştirilebilirlik problemlerinin NP-tam olduğunu kanıtlayarak tasvir edeceğiz. Kısım 34.5, metodolojinin daha fazla örneğini sağlar.

Aşağıdaki lemma, bir dilin NP-tam olduğunu göstermek için kullandığımız yöntemin temelidir.

Lemma 34.8

L , bir $L' \in \text{NPC}$ için $L' \leq_p L$ sağlayan bir dil ise NP-zordur. İlaveten, eğer $L \in \text{NP}$ ise $L \in \text{NPC}$ dir.

Kanıt L' , NP-tam olduğundan, her $L'' \in \text{NP}$ için $L'' \leq_p L'$ olur. Varsayımdan, $L' \leq_p L$ ve dolayısıyla geçişlilik özelliğinden (Alıştırma 34.3-2) $L'' \leq_p L$ olur ki bu da L nin NP-zor olduğunu gösterir. $L \in \text{NP}$ ise ayrıca $L \in \text{NPC}$ dir. ■

Diğer bir deyişle, bilinen bir NP-tam L' dilini L diline indirgeyerek, dolaylı olarak NP'deki her dili L ye indirgeriz. Dolayısıyla, Lemma 34.8 bize, bir L dilinin NP-tam olduğunu kanıtlamak için bir yöntem verir:

1. $L \in \text{NP}$ olduğunu kanıtlayınız.
2. Bilinen bir NP-tam L' dili seç.

3. L' dilinin her $x \in \{0,1\}^*$ örneğini L dilinin bir $f(x)$ örneğine eşleyen bir f fonksiyonunu hesaplayan bir algoritma tanımla.
4. f fonksiyonunun, tüm $x \in \{0,1\}^*$ için $x \in L'$ ancak ve ancak $f(x) \in L$ sağladığını kanıtlayınız.
5. f 'yi hesaplayan algoritmanın polinom zamanda çalıştığını göster.

(2-5 adımları L nin NP-zor olduğunu gösterir.) Bilinen tek bir NP-tam dilden indirgeme metodolojisi, daha karmaşık olan NP'deki her dilden nasıl indirgenmediğini doğrudan gösterme sürecinden çok daha basittir. CIRCUIT-SAT $\in$ NPC olduğunu kanıtlamak bize bir "fırsat" verir. Çünkü devresel-gerçekleştirilebilirlik probleminin NP-tam olduğunu biliyoruz, şimdi diğer problemlerin NP-tam olduğunu çok daha kolayca kanıtlayabiliriz. Ayrıca, bilinen NP-tam problemlerin bir kataloğunu geliştirdiğimiz için, indirgemedede kullanılacak diller için daha da fazla seçeneğimiz olacak.

Formül-gerçekleştirilebilirliği

İndirgeme metodolojisini, bir boole formülünün, bir devre değil, gerçekleştirilebilir olup olmadığını belirleme problemi için bir NP-tamlık kanıtı vererek açıklarız. Bu problem, şimdiye kadar NP-tam olduğu gösterilen ilk problem olma tarihi onuruna sahiptir.

(Formül) gerçekleştirilebilirlik problemini SAT dili açısından aşağıdaki gibi formüle ederiz. SAT bir örneği, aşağıdakilerden oluşan bir ϕ boole formülüdür.

1. n tane boole değişkeni: $x_1, x_2, \dots, x_n$;
2. m tane boole bağlacı: $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT), $\rightarrow$ (gerektirme), $\leftrightarrow$ (ancak ve ancak) gibi bir veya iki girdi ve bir çıktılı herhangi bir boole fonksiyonu; ve
3. parantezler. (Genelliği kaybetmeden, gereksiz parantezlerin olmadığını farzederiz; yani bir formül, boole bağlacı başına en fazla bir çift parantez içerir.)

Bir ϕ boole formülünü kolayca $n + m$ nin polinomu olan bir uzunlukta kodlayabiliriz. Boole birleşimsel devrelerinde olduğu gibi, bir ϕ boole formülü için bir **doğruluk ataması**, ϕ nin değişkenleri için bir değerler kümesidir ve bir **gerçekleşen atama**, 1 hesaplamasına neden olan bir doğruluk atamasıdır. Gerçekleşen atamalı bir formül **gerçekleştirilebilir** bir formüldür. Gerçekleştirilebilirlik problemi verilen bir boole formülünün gerçekleştirilebilir olup olmadığını sorar; formal-dil ifadesinde,

$SAT = \{ \langle \phi \rangle : \phi, \text{gerçekleştirilebilir bir boole formülüdür} \}.$

Örnek olarak,

$$\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$$

formülü $\langle x_1 = 0, x_2 = 0, x_3 = 1, x_4 = 1 \rangle$ gerçekleşen atamasına sahiptir; çünkü

$$\begin{aligned} \phi &= ((0 \rightarrow 0) \vee \neg((\neg 0 \leftrightarrow 1) \vee 1)) \wedge \neg 0 \quad (34.2) \\ &= (1 \vee \neg(1 \vee 1)) \wedge 1 \\ &= (1 \vee 0) \wedge 1 \\ &= 1 \end{aligned}$$

ve dolayısıyla bu ϕ formülü SAT a aittir.

Keyfi bir boole formülünün tatmin edilebilir olup olmadığını belirleyen sade algoritma polinom zamanda çalışmaz. n değişkenli bir formülün 2^n mümkün ataması vardır. Eğer $\langle \phi \rangle$ nin uzunluğu n nin polinomu ise her atamayı kontrol etmek $\Omega(2^n)$ zaman gerektirir ki bu da $\langle \phi \rangle$ nin uzunluğunda süperpolinomdur. Aşağıdaki teoremin gösterdiği gibi, polinom-zamanlı bir algoritmanın var olması pek mümkün değildir.

Teorem 34.9

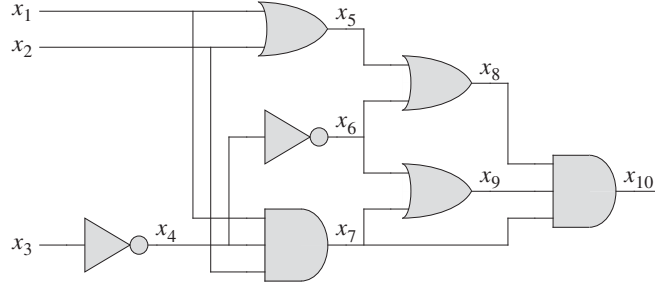
Boole formüllerinin gerçekleştirilebilirliği NP-tamdır.

Kanıt SAT $\in$ NP tartışmasıyla başlarız. Ardından, SAT ın NP-zor olduğunu CIRCUIT-SAT $\leq_p$ SAT olduğunu göstererek kanıtlarız; Lemma 34.8 ile teorem kanıtlanacaktır.

SAT ın NP'ye ait olduğunu göstermek için bir ϕ girdi formülü için bir gerçekleşen atamadan oluşan bir sertifikanın polinom zamanda doğrulanabildiğini gösteririz. Doğrulama algoritması basitçe formüldeki her bir değişkeni onun ilişkili değeriyle değiştirir ve ardından yukarıdaki (34.2) denkleminde yaptığımız gibi ifadeyi hesaplar. Bu işin polinom zamanda yapılması kolaydır. Eğer ifade 1 olarak hesaplanırsa algoritma, formülün gerçekleştirilebilir olduğunu doğrulamıştır. Dolayısıyla, Lemma 34.8 in NP-tamlık için olan ilk şartı sağlanır.

SAT ın NP-zor olduğunu kanıtlamak için CIRCUIT-SAT $\leq_p$ SAT olduğunu gösteririz. Diğer bir deyişle, devresel-gerçekleştirilebilirliğin bir örneğinin formül-gerçekleştirilebilirliğinin bir örneğine polinom zamanlı nasıl indirgendiğini göstermemiz gerekiyor. Herhangi bir boole birleşimsel devresini bir boole formülü olarak ifade etmek için tümevarım kullanabiliriz. Basitçe, devre çıktısını üreten kapıya bakarız ve kapının girdilerinin her birini formüller olarak tümevarımsal şekilde ifade ederiz. Ardından, kapının fonksiyonunu girdilerinin formüllerine uygulayan bir ifade yazarak devre için formül elde ederiz.

Ne yazık ki, bu kolay anlaşılır yöntem polinom-zamanlı indirgeme anlamına gelmez. Alıştırma 34.4-1 in göstermenizi istediği gibi, çıktı kabloları 2 ya da daha fazla çıkış yelpazesine sahip kapılardan ortaya çıkan paylaşımlı altformüller, yaratılan formülün boyutunun üssel olarak büyümesine neden olabilir. Bu nedenle, indirgeme algoritması biraz daha zeki olmalıdır.



Şekil 34.10 Devresel-gerçekleştirilebilirliği, formül-gerçekleştirilebilirliğine indirgeme. İndirgeme algoritmasıyla üretilen formül, devredeki her bir kablo için bir değişkene sahiptir.

Şekil 34.10, bu problemin üstesinden nasıl geleceğimizi Şekil 34.8(a) deki devreyi örnek olarak kullanarak tasvir eder. C devresindeki her bir x_i kablosu için ϕ formülü bir x_i değişkenine sahiptir. Şimdi, her bir kapının, kablolarının değişkenlerini içeren küçük bir formül olarak nasıl işlediğini ifade edebiliriz. Örneğin, AND çıktı kapısının işlemi $x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9)$ dur. Bu küçük formüllerin her birini bir **cümlecik** olarak adlandırırız.

İndirgeme algoritması tarafından üretilen ϕ formülü, her bir kapının işlemini tanımlayan cümleciklerin birleşimi ile devre-çıktı değişkeninin AND lenmesidir. Şekildeki devre için formül

$$\begin{aligned} \phi = & x_{10} \wedge (x_4 \leftrightarrow \neg x_3) \\ & \wedge (x_5 \leftrightarrow (x_1 \vee x_2)) \\ & \wedge (x_6 \leftrightarrow \neg x_4) \\ & \wedge (x_7 \leftrightarrow (x_1 \wedge x_2 \wedge x_4)) \\ & \wedge (x_8 \leftrightarrow (x_5 \vee x_6)) \\ & \wedge (x_9 \leftrightarrow (x_6 \vee x_7)) \\ & \wedge (x_{10} \leftrightarrow (x_7 \wedge x_8 \wedge x_9)) . \end{aligned}$$

Bir C devresi verildiğinde, böyle bir ϕ formülünü polinom zamanda üretmek basittir.

Niçin C devresi tam olarak ϕ formülü gerçekleştirilebilirse gerçekleştirilebilir? Eğer C devresi gerçekleşen bir atamaya sahipse devrenin her bir kablosu iyi-tanımlı bir değere sahiptir ve devrenin çıktısı 1 dir. Dolayısıyla, kablo değerlerini ϕ deki değişkenlere atadığımızda ϕ nin her bir cümlecigi 1 olarak değerlendirilir ve bu nedenle tamamının birleşimi 1 olarak hesaplanır. Diğer taraftan, eğer bir atama ϕ nin 1 değeri almasına neden olursa C devresi benzer bir argüman tarafından gerçekleştirilebilir. Böylece, $\text{CIRCUIT-SAT} \leq_p \text{SAT}$ olduğunu göstermiş oluruz ve kanıt tamamlanır. ■

3-CNF gerçekleştirilebilirliği

Birçok problemin NP-tam olduğunu formül-gerçekleştirilebilirlikten indirgeme yaparak kanıtlayabiliriz. İndirgeme algoritması herhangi bir girdi formülüyle başa çıkmalıdır, buna rağmen bu gereksinim düşünmemiz gereken muazzam sayıda duruma yol açabilir. Sıklıkla boole formüllerinin kısıtlanmış bir dilinden indirgeme yapmayı tercih ederiz, böylece daha az durumu düşünmemiz gerekir. Tabi ki, polinom-zamanlı çözülebilir olması için dili çok fazla kısıtlamamalıyız. 3-CNF gerçekleştirilebilirliği, ya da 3-CNF-SAT uygun bir dildir.

Takip eden terimleri kullanarak 3-CNF tatmin edilebilirliği tanımlarız. Bir boole formüldeki bir *literal* bir değişken ya da onun değeridir. Bir boole formülü, eğer her biri bir ya da daha fazla literalin OR u olan *cümleciklerin* bir AND i olarak ifade edilirse *ve-li normal biçimde* ya da *CNF* dedir. Eğer her bir cümlecik tam olarak 3 ayrı literale sahipse, bir boole formülü *3-ve-li normal biçimde* ya da *3-CNF* dedir.

Örneğin,

$$(x_1 \vee \neg x_1 \vee \neg x_2) \wedge (x_3 \vee x_2 \vee x_4) \wedge (\neg x_1 \vee \neg x_3 \vee \neg x_4)$$

boole formülü 3-CNF dedir. Üç cümlecikinden ilki $x_1, \neg x_1, \neg x_2$ literallerini içeren $(x_1 \vee \neg x_1 \vee \neg x_2)$ cümlecikidir.

3-CNF-SAT dilinde, 3-CNF de verilen bir ϕ boole formülünün gerçekleştirilebilir olup olmadığı sorulur. Aşağıdaki teorem, bu basit normal formda gösterildiklerinde bile, boole formüllerinin gerçekleştirilebilirliği belirleyebilen polinom-zamanlı bir algoritmanın bulunmasının pek mümkün olmadığını gösterir.

Teorem 34.10

3-ve-li normal biçimdeki boole formüllerinin gerçekleştirilebilirliği NP-tamdır.

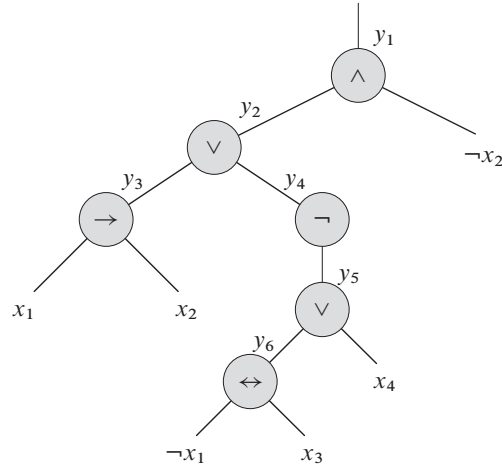
Kanıt SAT $\in$ NP olduğunu göstermek için Teorem 34.9 un kanıtında kullandığımız argüman, 3-CNF-SAT $\in$ NP olduğunu göstermek için buraya aynı derecede iyi uyar. Böylece, Lemma 34.8 den, sadece SAT $\leq_p$ 3-CNF-SAT olduğunu göstermemiz gerekiyor.

İndirgeme algoritmasını üç temel adıma ayırırız. Her bir adım ϕ girdi formülünü istenen 3-ve-li normal biçime derece derece daha yakın olarak dönüştürür.

İlk adım, Teorem 34.9 da CIRCUIT-SAT $\leq_p$ SAT olduğunu kanıtlamak için kullandığımıza benzerdir. Öncelikle ϕ girdi formülü için yapraklar olarak literal-ler ve dahili düğümler olarak bağlaçlar ile birlikte bir ikili “analiz” ağacı oluştururuz. Şekil 34.11, aşağıdaki formül için böyle bir analiz ağacını gösterir.

$$\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2 \quad (34.3)$$

Girdi formülü, çeşitli literallerin OR u gibi bir cümlecik içerirse, sonuçta oluşan ağaçtaki her dahili düğümün 1 ya da 2 çocuğu olsun diye ifadeyi tamamen parantez içine almak için birleşimliliği kullanırız. Şimdi ikili analiz ağacını, fonksiyonu hesaplayan bir devre olarak düşünebiliriz.



Şekil 34.11 $\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$ formülüne ilişkin ağaç.

Teorem 34.9 daki kanıtı taklit ederek, her bir dahili düğümün çıktısı için bir y_i değişkeni sunarız. Ardından, orijinal ϕ formülünü, kök değişkeni ile her bir düğümün işlemini tanımlayan cümleciklerin bir birleşiminin VEs'i olarak yeniden yazarız. (34.3) formülü için sonuç ifadesi,

$$\begin{aligned} \phi' = & y_1 \wedge (y_1 \leftrightarrow (y_2 \wedge \neg x_2)) \\ & \wedge (y_2 \leftrightarrow (y_3 \vee y_4)) \\ & \wedge (y_3 \leftrightarrow (x_1 \rightarrow x_2)) \\ & \wedge (y_4 \leftrightarrow \neg y_5) \\ & \wedge (y_5 \leftrightarrow (y_6 \vee x_4)) \\ & \wedge (y_6 \leftrightarrow (\neg x_1 \leftrightarrow x_3)) \text{ dir.} \end{aligned}$$

Bu şekilde elde edilen ϕ' formülünün, her biri en fazla 3 literale sahip ϕ'_i cümleciklerin bir birleşimi olduğuna dikkat ediniz. Karşılama başari olabileceğimiz tek gereklilik, her bir cümlecğin, 3 literalin bir OR u olması gerektiğidir.

İndirgemenin ikinci adımı her bir ϕ'_i cümlecğini ve-li normal biçime dönüştürür. ϕ'_i için bir doğruluk tablosunu, değişkenlerine tüm mümkün atamaları değerlendirerek oluştururuz. Doğruluk tablosunun her bir satırı, cümlecğin değişkenlerinin bir mümkün ataması ile birlikte o atamanın altında yer alan cümlecğin değerinden oluşur. 0 ı değerlendiren doğruluk-tablosu girişlerini kullanarak $\neg\phi'_i$ cümlecğine denk olan **veya-li normal biçimi** (ya da **DNF**) – AND lerin bir OR u – inşa ederiz. Ardından bu formülün değilini alırız ve tüm literalleri tümlemek, OR ları AND lere ve AND leri OR lara dönüştürmek amacıyla önermeler mantı-

y_1	y_2	x_2	$(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$
1	1	1	0
1	1	0	1
1	0	1	0
1	0	0	0
0	1	1	1
0	1	0	0
0	0	1	1
0	0	0	1

Şekil 34.12 $(y_1 \leftrightarrow (y_2 \wedge \neg x_2))$ cümlecği için doğruluk tablosu.

ğındaki **DeMorgan kurallarını**,

$$\neg(a \wedge b) = \neg a \vee \neg b ,$$

$$\neg(a \vee b) = \neg a \wedge \neg b ,$$

kullanarak bir ϕ_i'' CNF formülüne dönüştürürüz.

Örneğimizde, $\phi_1' = (y_1 \leftrightarrow (y_2 \wedge \neg x_2))$ cümlecğini CNF ye aşağıdaki gibi dönüştürürüz. ϕ_1' için doğruluk tablosu Şekil 34.12 de görülmektedir. $\neg\phi_1'$ cümlecğine denk olan DNF formülü

$$(y_1 \wedge y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge x_2) \vee (y_1 \wedge \neg y_2 \wedge \neg x_2) \vee (\neg y_1 \wedge y_2 \wedge \neg x_2) \text{dir.}$$

Değilini alıp DeMorgan kurallarını uygulayarak orijinal ϕ_1' cümlecğine denk olan,

$$\begin{aligned} \phi_1'' &= (\neg y_1 \vee \neg y_2 \vee \neg x_2) \wedge (\neg y_1 \vee y_2 \vee \neg x_2) \\ &\quad \wedge (\neg y_1 \vee y_2 \vee x_2) \wedge (y_1 \vee \neg y_2 \vee x_2) , \end{aligned}$$

CNF formülünü elde ederiz.

Bu noktada, ϕ' formülünün her bir ϕ_i' cümlecğini bir ϕ_i'' CNF formülüne dönüştürmüş olduk ve bu nedenle ϕ' , ϕ_i'' lerin birleşiminden oluşan ϕ'' CNF formülüne denktir. Ayrıca, ϕ'' formülünün her bir cümlecği en fazla 3 literale sahiptir.

İndirgemenin üçüncü ve son adımı, her bir cümlecği *tam olarak* 3 farklı literale sahip olacak şekilde formülü değiştirir. ϕ'' CNF formülünün cümleciklerinden son ϕ''' 3-CNF formülünü oluştururuz. ϕ''' formülü, p ve q diyeceğimiz iki yardımcı değişken de kullanır. ϕ'' formülünün her bir C_i cümlecği için aşağıdaki cümlecikleri ϕ''' formülüne dahil ederiz:

- C_i , 3 farklı literale sahipse, C_i yi ϕ''' formülünün bir cümlecği olarak basitçe dahil et.
- C_i , 2 farklı literale sahipse; yani l_1 ve l_2 literaller olmak üzere $C_i = (l_1 \vee l_2)$ ise $(l_1 \vee l_2 \vee p) \wedge (l_1 \vee l_2 \vee \neg p)$ 'yi ϕ''' formülünün cümlecikleri olarak dahil et. p ve $\neg p$ literalleri sadece ϕ''' formülünün her bir cümlecğinin tam

olarak 3 farklı literale sahip olması sintaktik gerekliliğini karşılar. $p = 0$ veya $p = 1$ ise cümleciklerden biri $l_1 \vee l_2$ 'ye denktir ve diğeri 1 olarak değerlendirilir ki bu da AND in özdeşliğidir.

- C_i , sadece 1 farklı l literale sahipse $(l \vee \neg p \vee q) \wedge (l \vee \neg p \vee \neg q) \wedge (l \vee p \vee q) \wedge (l \vee p \vee \neg q)$ yi ϕ''' formülünün cümlecikleri olarak dahil et. p ve q nun değerleri ne olursa olsun dört cümlecikten biri l ye denktir ve diğer 3 ü 1 olarak değerlendirilir.

Üç adımın her birini inceleyerek ϕ''' 3-CNF formülünün ancak ve ancak ϕ gerçekleştirilebilir ise her birini olduğunu görebiliriz. CIRCUIT-SAT dilinden SAT diline indirgeme gibi, ilk adımda ϕ formülünden ϕ' inşası gerçekleştirilebilirliği muhafaza eder. İkinci adım, cebirsel olarak ϕ' formülüne denk olan bir ϕ'' CNF formülü üretir. Üçüncü adım, etkin bir biçimde ϕ'' formülüne denk olan bir ϕ''' 3-CNF formülü üretir; çünkü p ve q değişkenlerine herhangi bir atama cebirsel anlamda ϕ'' formülüne denk olan bir formül üretir.

İndirgemenin polinom zamanda hesaplanabildiğini de göstermeliyiz. ϕ formülünden ϕ' inşası, ϕ formülündeki bağlaç başına en fazla 1 değişken ve 1 cümlecik sunar. ϕ' formülünden ϕ'' inşası, ϕ' formülünden her bir cümlecik için ϕ'' içine en fazla 8 cümlecik sunabilir; çünkü ϕ' formülünün her bir cümlecigi en fazla 3 değişkene sahiptir ve her bir cümlecik için doğruluk tablosu en fazla $2^3 = 8$ satır içerir. ϕ'' formülünden ϕ''' inşası, ϕ'' formülünün her bir cümlecik için ϕ''' içine en fazla 4 cümle sunar. Dolayısıyla, ϕ''' sonuç formülünün boyutu orijinal formülün uzunluğunda polinomdur. Kurulumların her biri polinom zamanda kolaylıkla yerine getirilebilir. ■

Alıştırmalar

34.4-1

Teorem 34.9 un kanıtındaki basit (polinom-zamanlı olmayan) indirgemeyi düşünün. Bu yöntem ile bir formüle dönüştürüldüğünde, boyutu n de üssel olan bir formül veren n boyutlu bir devre tanımlayınız.

34.4-2

Teorem 34.10 un yöntemini (34.3) formülü üzerinde kullandığımızda ortaya çıkan 3-CNF formülünü gösteriniz.

34.4-3

Profesör Jagger, sadece Teorem 34.10 un kanıtındaki doğruluk-tablosu tekniğini kullanıp diğer adımları kullanmayarak $\text{SAT} \leq_p 3\text{-CNF-SAT}$ olduğunu göstermeyi teklif eder. Yani profesör, bir ϕ boole formülünü almayı, onun değişkenleri için bir doğruluk tablosu oluşturmayı, doğruluk tablosundan $\neg\phi$ formülüne denk 3-DNF de bir formül elde etmeyi, ardından değilini alıp ϕ formülüne denk bir

3-CNF formül üretmek için DeMorgan kurallarını kullanmayı önerir. Bu stratejinin polinom-zamanlı bir indirgeme vermediğini gösteriniz.

34.4-4

Bir boole formülünün bir totoloji olup olmadığını belirleme probleminin co-NP için tam olduğunu gösteriniz. (İpucu: Alıştırma 34.3-7 ye bakınız.)

34.4-5

Veya-lı normal biçimdeki boole formüllerinin gerçekleştirilebilirliğini belirleme probleminin polinom-zamanda çözülebilir olduğunu gösteriniz.

34.4-6

Birinin size formül-gerçekleştirilebilirliğini belirlemek için polinom-zamanlı bir algoritma verdiğini farzedin. Polinom zamanda gerçekleşen atamalar bulmak için bu algoritmanın nasıl kullanıldığını tarif ediniz.

34.4-7

2-CNF-SAT, CNF deki cümlecik başına tam olarak 2 literalli gerçekleştirilebilir boole formüllerinin kümesi olsun. $2\text{-CNF-SAT} \in P$ olduğunu gösteriniz. Algoritmanızı mümkün olduğunca etkili yapınız. (İpucu: $x \vee y$ ifadesinin $\neg x \rightarrow y$ ifadesine denk olduğuna dikkat ediniz. 2-CNF-SAT ı yönlü bir çizge üzerinde etkin bir biçimde çözülebilir bir probleme indirgeyiniz).

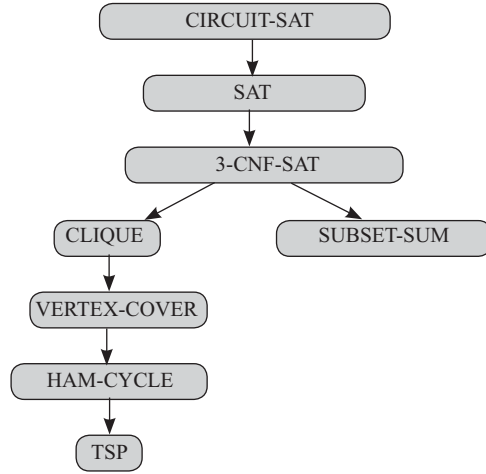
34.5 NP-tam problemler

NP-tam problemler çeşitli alanlarda ortaya çıkar: boole mantık, çizgeler, aritmetik, ağ tasarımı, kümeler ve parçalamalar, depolama ve erişim, sıralama ve çizelgeleme, matematiksel programlama, cebir ve sayılar teorisi, oyunlar ve bulmacalar, otomata ve dil teorisi, program optimizasyonu, biyoloji, kimya, fizik ve daha fazlası. Bu bölümde, çizge teori ve küme parçalamadan elde edilen çeşitli problemler için NP-tamlık kanıtları sağlamak üzere indirgeme metodolojisini kullanacağız.

Şekil 34.13, bu bölüm ve Kısım 34.4 deki NP-tamlık kanıtlarının yapısını ana hatlarıyla belirtir. Şekildeki her bir dilin NP-tam olduğunu onu gösteren dilden indirgeme yaparak kanıtlarız. Teorem 34.7 de NP-tam olduğunu kanıtladığımız CIRCUIT-SAT dili köktedir.

34.5.1 Klik problemi

Yönsüz bir $G = (V, E)$ çizgesindeki bir **klik**, her bir çifti E deki bir ayrıt ile bağlanmış tepelerin bir $V' \subseteq V$ altkümesidir. Başka bir deyişle, bir klik G nin bir tam alt çizgesidir. Bir kliğin **boyutu** içerdiği tepelerin sayısıdır. **Klik problemi**, bir çizgede maksimum boyutlu bir klik bulma optimizasyon problemidir. Bir karar



Şekil 34.13 Kısım 34.4 ve 34.5 deki NP-tamlık kanıtlarının yapısı. Tüm kanıtlar CIRCUI-T-SAT ın NP-tamlığından indirgeme ile elde edilir.

problemi olarak, basitçe, verilen bir k boyutunda bir kliğin çizgede olup olmadığını sorarız. Formal tanımı:

$\text{CLIQUE} = \{ \langle G, k \rangle : G, k \text{ boyutlu bir klik içeren bir çizgedir} \}.$

$|V|$ tepeli bir $G = (V, E)$ çizgesinin k boyutlu bir kliğe sahip olup olmadığını belirleyen basit bir algoritma, V nin tüm k -altkümelerini listeler ve bir klik oluşturup oluşturmadığını görmek için her birini kontrol eder. Bu algoritmanın çalışma zamanı, k bir sabit sayı ise polinom olan $\Omega(k^2 \binom{|V|}{k})$ dir. Bununla birlikte, genelde algoritmanın süperpolinom zamanda çalıştığı durumda k sayısı $|V|/2$ değerine yakın olabilir. Aslında, klik problemi için etkili bir algoritmanın bulunması pek mümkün değildir.

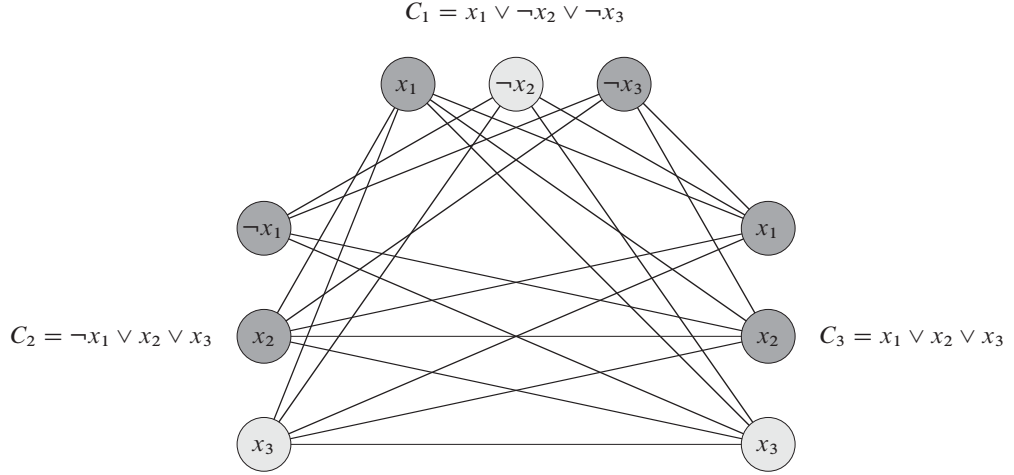
Teorem 34.11

Klik problemi NP-tamdır.

Kanıt Verilen bir $G = (V, E)$ çizgesi için $\text{CLIQUE} \in \text{NP}$ olduğunu göstermek için klikteki tepelerin $V' \subseteq V$ kümesini G için bir sertifika olarak kullanırız. Her bir $u, v \in V'$ çifti için (u, v) ayrımının E ye ait olup olmadığını kontrol ederek V' kümesinin bir klik olup olmadığını polinom zamanda kontrol edebiliriz.

Ardından, klik probleminin NP-zor olduğunu gösteren $3\text{-CNF-SAT} \leq_p \text{CLIQUE}$ ifadesini kanıtlarız. Böyle bir sonucu kanıtlayabilmemize şaşırmış olabilirsiniz; çünkü yüzeyde mantıksal formüllerin çizgelerle az ilgisi var gibi görünür.

İndirgeme algoritması 3-CNF-SAT on bir örneği ile başlar. $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$, 3-CNF de k cümleli bir boole formülü olsun. $r = 1, 2, \dots$,



Şekil 34.14 $C_1 = (x_1 \vee \neg x_2 \vee \neg x_3)$, $C_2 = (\neg x_1 \vee x_2 \vee x_3)$, $C_3 = (x_1 \vee x_2 \vee x_3)$ olmak üzere, 3-CNF-SAT ı CLIQUE e indirgemedeki $\phi = C_1 \wedge C_2 \wedge C_3$ den elde edilen G çizgesi. Formülün gerçekleşen bir ataması $x_2 = 0$, $x_3 = 1$ ve $x_1 = 0$ ya da 1 dir. Açık gri boyalı tepelerle clique e denk gelen bu atama, $\neg x_2$ ile C_1 i, x_3 ile C_2 ve C_3 ü tatmin eder.

k için her bir C_r cümlecği tam olarak üç farklı l_1^r, l_2^r ve l_3^r literale sahiptir. Bir G çizgesi oluşturacağız öyle ki, ϕ , ancak ve ancak G , k boyutlu bir kliğe sahipse ϕ gerçekleştirilebilir. ϕ deki her bir $C_r = (l_1^r \vee l_2^r \vee l_3^r)$ cümlecği için V ye bir v_1^r, v_2^r ve v_3^r tepe üçlüsü yerleştiririz. Aşağıdakilerin ikisi birden sağlanırsa v_1^r ve v_j^s tepeleri arasına bir ayırıt koyarız:

- v_i^r ve v_j^s farklı üçlülerin içindedir; yani $r \neq s$, ve
- ilgili literaller **tutarlıdır**; yani l_i^r, l_j^s nin değili değildir.

Bu çizgesi polinom zamanda ϕ den kolayca inşa edebiliriz. Bu inşanın bir örneği olarak, eğer

$$\phi = (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$$

ise G , Şekil 34.14 de gösterilen çizgedir.

ϕ nin G ye bu dönüşümünün bir indirgeme olduğunu göstermeliyiz. İlk olarak, ϕ nin gerçekleşen bir ataması olduğunu varsayalım. Bu durumda, her bir C_r cümlecği 1 atanmış en az bir l_i^r literali içerir ve bu şekildeki her bir literal bir v_i^r tepesine karşılık gelir. Her bir cümleden böyle bir “doğru” literal seçmek k tepenin bir V' kümesini verir. V' kümesinin bir klik olduğunu iddia ediyoruz. $r \neq s$ olmak üzere, herhangi iki $v_i^r, v_j^s \in V'$ tepeleri için her iki ilişkili l_i^r, l_j^s literal-

rini verilen cümleciği atama tarafından 1 e eşlenir, dolayısıyla literaller tümleyen olamazlar. Bu nedenle, G in inşasına göre, (v_i^r, v_j^s) ayrıtı E ye aittir.

Diğer taraftan, G nin k boyutlu bir V' kliğine sahip olduğunu varsayalım. G deki hiçbir ayrıt aynı üçlüdeki tepeleri birleştirmez ve dolayısıyla V' üçlü başına tam olarak bir tepe içerir. Hem bir literale hem de onun tümleyenine 1 atama korkusu olmadan, her bir l_i^r literale 1 atayabiliriz öyle ki $v_i^r \in V'$ olur; çünkü G uyumsuz literaller arasında hiçbir ayrıt içermez. Her bir cümlecik gerçekleşir ve dolayısıyla ϕ gerçekleşir (Klikteki bir tepeye karşılık gelmeyen değişkenler keyfi olarak ayarlanabilir). ■

Şekil 34.14 ün örneğinde ϕ nin gerçekleşen bir atamasında $x_2 = 0$ ve $x_3 = 1$ dir. Karşılık gelen bir $k = 3$ boyutlu klik, ilk cümlecikten $\neg x_2$ ye ilişkin, ikinci cümlecikten x_3 e ilişkin ve üçüncü cümlecikten x_3 e ilişkin tepelerden oluşur. Çünkü klik, x_1 ya da $\neg x_1$ e ilişkin hiçbir tepe içermez, bu gerçekleşen atamada x_1 i 0 ya da 1 olarak ayarlayabiliriz.

Teorem 34.11 in kanıtında, 3-CNF-SAT ın keyfi bir örneğini belirli bir yapıdaki CLIQUE in bir örneğine indirgediğimize dikkat edin. Sadece tepelerin üçlüler içinde görünmeye sınırlandığı ve aynı üçlüdeki tepeler arasında hiçbir ayrıtın olmadığı çizgelerde CLIQUE in NP-zor olduğunu gösterdiğimizi düşünmüş olabilirsiniz. Aslında, CLIQUE in sadece bu kısıtlanmış durumda NP-zor olduğunu gösterdik; ancak bu kanıt CLIQUE in genel çizgelerde NP-zor olduğunu göstermek için yeterli gelir. Neden? CLIQUE i genel çizgeler üzerinde çözen polinom-zamanlı bir algoritmamız olsaydı, CLIQUE i kısıtlanmış çizgeler üzerinde de çözerdi.

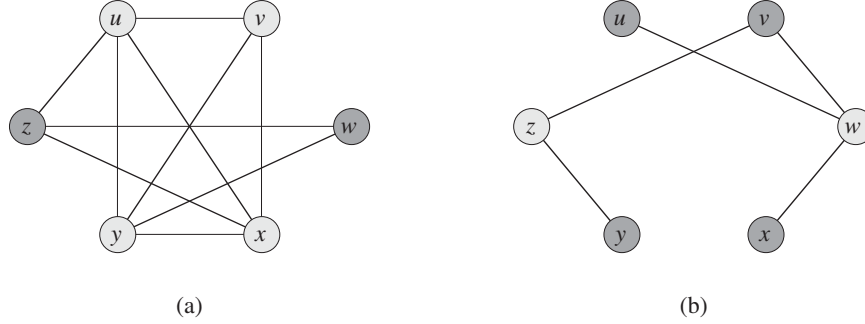
Bununla beraber, zıt yaklaşım – özel bir yapıya sahip 3-CNF-SAT ın örneklerini CLIQUE in genel örneklerine indirgeme – yeterli gelmezdi. Neden? Muhtemelen, ondan indirgemek için seçtiğimiz 3-CNF-SAT örnekleri “kolay”dı ve bu nedenle bir NP-zor problemi CLIQUE e indirgemezdik.

İndirgemenin, çözümü değil 3-CNF-SAT ın örneğini kullandığına da dikkat edin. Polinom-zamanlı indirgeme ϕ formülünün gerçekleştirilebilir olup olmadığını bilmeye dayalı olmuş olsaydı, yanılmış olurduk; çünkü ϕ nun gerçekleştirilebilir olup olmadığına polinom zamanda nasıl karar verildiğini bilmiyoruz.

34.5.2 Tepe-örtüsü problemi

Yönsüz bir $G = (V, E)$ çizgesinin bir **tepe örtüsü**, $(u, v) \in E$ ise $u \in V'$ ya da $v \in V'$ (ya da her ikisi) olacak şekilde bir $V' \subseteq V$ altkümesidir. Yani, her bir tepe bitişik ayrıtlarını “örter” ve G için bir tepe örtüsü E deki bütün ayrıtları örten tepelerin bir kümesidir. Bir tepe örtüsünün **boyutu** ondaki tepelerin sayısıdır. Örneğin, Şekil 34.15(b) deki çizge 2 boyutlu bir $\{w, z\}$ tepe örtüsüne sahiptir.

Tepe örtüsü problemi, verilen bir çizgede en küçük boyuta sahip bir tepe örtüsünü bulmaktır. Bu optimizasyon problemini bir karar problemi olarak yeniden



Şekil 34.15 CLIQUE i VERTEX-COVER a indirgeme. (a) $V' = \{u, v, x, y\}$ kligi ile yönsüz bir $G = (V, E)$ çizgesi. (b) İndirgeme algoritmasıyla üretilmiş, $V - V' = \{w, z\}$ tepe örtüsüne sahip $\overline{G}$ çizgesi.

ifade edersek, bir çizgenin verilen k boyutlu bir tepe örtüsüne sahip olup olmadığını belirlemek istiyoruz. Bir dil olarak aşağıdaki gibi tanımlarız.

$\text{VERTEX-COVER} = \{\langle G, k \rangle : G \text{ çizgesi } k \text{ boyutlu bir tepe örtüsüne sahiptir}\}$

Aşağıdaki teorem bu problemin NP-tam olduğunu gösterir.

Teorem 34.12

Tepe-örtüsü problemi NP-tamdır.

Kanıt İlk olarak $\text{VERTEX-COVER} \in \text{NP}$ olduğunu gösteririz. Bir $G = (V, E)$ çizgesi ve bir k tam sayısı verildiğini farzedelim. Seçtiğimiz sertifika $V' \subseteq V$ tepe örtüsünün kendisidir. Doğrulama algoritması $|V'| = k$ olduğunu doğrular ve ardından her bir $(u, v) \in E$ için $u \in V'$ ya da $v \in V'$ olup olmadığını kontrol eder. Sertifikayı polinom zamanlı kolayca doğrulayabiliriz.

$\text{CLIQUE} \leq_p \text{VERTEX-COVER}$ olduğunu göstererek tepe-örtüsü probleminin NP-zor olduğunu kanıtlarız. Bu indirgeme, bir çizgenin “tümleyeni” kavramına dayanır. Yönsüz bir $G = (V, E)$ çizgesi verildiğinde, G nin **tümleyenini**, $\overline{E} = \{(u, v) : u, v \in V, u \neq v \text{ ve } (u, v) \notin E\}$ olmak üzere $\overline{G} = (V, \overline{E})$ olarak tanımlarız. Diğer bir deyişle, $\overline{G}$, tam olarak G de olmayan ayrıtları içeren çizgedir. Şekil 34.15 bir çizge ile onun tümleyenini gösterir ve CLIQUE ten VERTEX-COVER e indirgemeyi tasvir eder.

İndirgeme algoritması, girdi olarak klik probleminin bir $\langle G, k \rangle$ örneğini alır. Polinom zamanda kolayca yapabildiğimiz üzere $\overline{G}$ tümleyenini hesaplar. İndirgeme algoritmasının çıktısı, tepe-örtüsü probleminin $\langle \overline{G}, |V| - k \rangle$ örneğidir. Kanıtı tamamlamak için bu dönüşümün gerçekte bir indirgeme olduğunu gösteririz:

G çizgesi k boyutlu bir kliğe sahiptir ancak ve ancak $\overline{G}$ çizgesi, $|V| - k$ boyutlu bir tepe örtüsüne sahiptir.

G nin $|V'| = k$ ile bir $V' \subseteq V$ kliğine sahip olduğunu varsayalım. $V - V'$ kümesinin $\overline{G}$ çizgesinde bir tepe örtüsü olduğunu iddia ediyoruz. $(u, v) \in \overline{E}$ de herhangi bir ayrıt olsun. Böylece, $(u, v) \notin E$ olur ki bu u ya da v den en az birinin V' kümesine ait olmadığını belirtir; çünkü V' kümesindeki her tepe çifti E nin bir ayrıtıyla birleştirilmiştir. Denk olarak, u ya da v den en az biri $V - V'$ kümesindedir, bu da (u, v) ayrıtının $V - V'$ tarafından örtüldüğü anlamına gelir. $(u, v) \in E$ den keyfi olarak seçildiği için $\overline{E}$ nin her ayrıtı $V - V'$ kümesindeki bir tepe tarafından örtülür. Dolayısıyla, boyutu $|V| - k$ olan $V - V'$ kümesi, $\overline{G}$ için bir tepe örtüsü oluşturur.

Diğer taraftan, $|V'| = |V| - k$ olmak üzere, $\overline{G}$ çizgesinin bir $V' \subseteq V$ tepe örtüsüne sahip olduğunu farzedelim. Şu halde, her $u, v \in V$ için eğer $(u, v) \notin E$ ise $u \in V'$ ya da $v \in V'$ ya da her ikisi de kümeye aittir. Bu gerektirmenin karşıtı tersi, her $u, v \in V$ için eğer $u \notin V'$ ve $v \notin V'$ ise $(u, v) \in E$ dir. Başka bir deyişle, $V - V'$ bir klikte ve $|V| - |V'| = k$ boyutuna sahiptir. ■

VERTEX-COVER, NP-tam olduğundan minimum-boyutlu bir tepe örtüsü bulmak için polinom-zamanlı bir algoritma bulmayı beklemeyiz. Bununla birlikte, Kısım 35.1, tepe-örtüsü problemi için “yaklaşık” çözümler üreten polinom-zamanlı bir “yaklaşım algoritması” sunar. Algoritma tarafından üretilen bir tepe örtüsünün boyutu, bir tepe örtüsünün minimum boyutunun en fazla iki katıdır.

Dolayısıyla, bir problem sadece NP-tam diye ümit etmeyi bırakmamalıyız. Bir optimal çözüm bulmak NP-tam olsa da yakın-optimal çözümler elde eden polinom-zamanlı bir yaklaşım algoritması tasarlayabiliriz. Bölüm 35, NP-tam problemler için çeşitli yaklaşım algoritmaları verir.

34.5.3 Hamilton-çevrim problemi

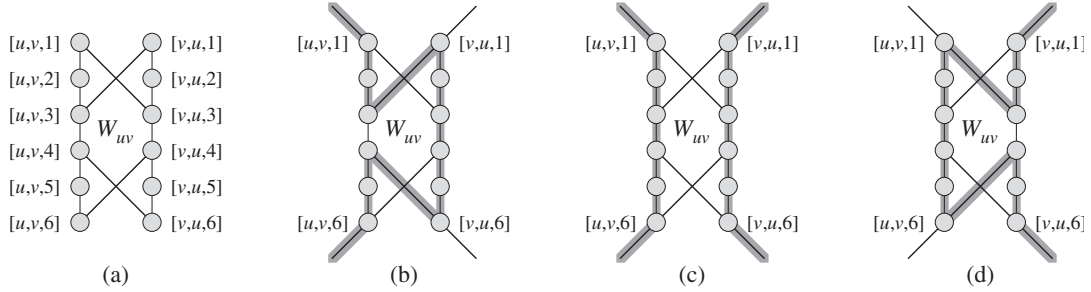
Şimdi Kısım 34.2 de tanımlanan hamilton-çevrim problemine geri dönüyoruz.

Teorem 34.13

Hamilton çevrim problemi NP-tamdır.

Kanıt Öncelikle HAM-CYCLE nin NP'ye ait olduğunu gösteririz. Bir $G = (V, E)$ çizgesi verildiğinde sertifikamız, hamilton çevrimini oluşturan $|V|$ tepenin dizisidir. Doğrulama algoritması, bu dizinin V deki her bir tepayı tam olarak bir kez içerip içermediğini ve sonda tekrarlanan ilk tepe ile birlikte G de bir çevrim oluşturup oluşturmadığını kontrol eder. Yani, ardışık tepelerin her bir çifti arasında ve ilk ile son tepeler arasında bir ayrıt var mı diye kontrol eder. Sertifikayı polinom zamanda doğrularız.

Şimdi HAM-CYCLE nin NP-tam olduğunu gösteren VERTEX-COVER $\leq_p$ HAM-CYCLE olduğunu kanıtlarız. Yönsüz bir $G = (V, E)$ çizgesi ve bir k tam sa-

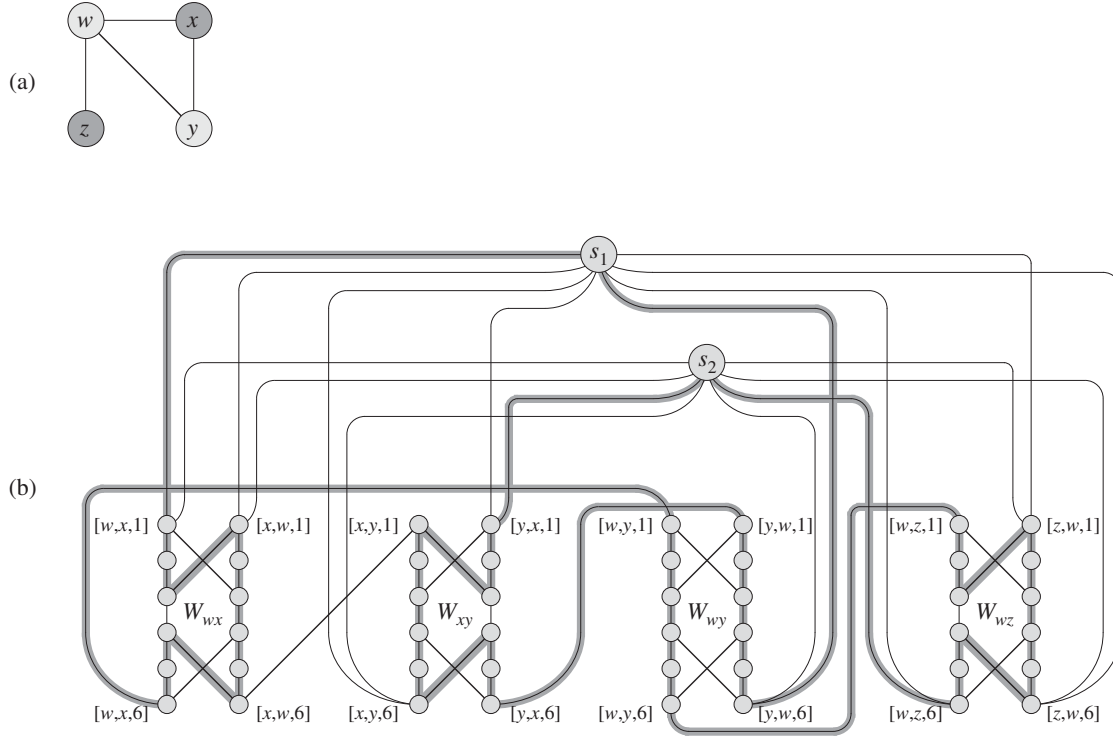


Şekil 34.16 Tepe-örtüsü problemini hamilton-çevrim problemine indirgemede kullanılan parçacık. G çizgesinin bir (u, v) ayrıtı, indirgemede yaratılan çizgesindeki W_{uv} parçacığına karşılık gelir. (a) İşaretlenmiş ayrık tepeler ile parçacık. (b)-(d) Parçacıktan G' çizgesinin geri kalanına olan bağlantıların yalnızca $[u, v, 1]$, $[u, v, 6]$, $[v, u, 1]$ ve $[v, u, 6]$ tepeleri boyunca olduğunu varsayarak, sadece gölgelendirilmiş yollar parçacık boyunca tüm tepeleri içeren mümkün yollardır.

yısı verildiğinde, ancak ve ancak G , k boyutlu bir tepe örtüsüne sahip ise hamilton çevrime sahip olan yönsüz bir $G' = (V', E')$ inşa ederiz.

Yapımız, belirli özellikleri yerine getiren, bir çizgenin bir parçası olan bir **parçacık** kullanır. Şekil 34.16(a) kullandığımız parçacığı gösterir. İnşa ettiğimiz G' çizgesi, her bir $(u, v) \in E$ ayrıtı için bu parçacığın W_{uv} ile belirttiğimiz bir kopyasını içerir. W_{uv} deki her bir tepeyi, $1 \leq i \leq 6$ olmak üzere, $[u, v, i]$ ya da $[v, u, i]$ ile gösteririz, böylece her bir W_{uv} parçacığı 12 tepe içerir. W_{uv} parçacığı ayrıca, Şekil 34.16(a) da gösterilen 14 ayrıtı içerir.

Parçacığın dahili yapısı ile birlikte, parçacık ve inşa ettiğimiz G' çizgesinin geri kalan kısmı arasındaki bağlantıları sınırlandırarak istediğimiz özellikleri yürürlüğe koyarız. Özellikle, sadece $[u, v, 1]$, $[u, v, 6]$, $[v, u, 1]$ ve $[v, u, 6]$ tepeleri W_{uv} nin dışından bitişik ayrıtlara sahip olacaktır. G' çizgesinin herhangi bir hamilton çevrimi, W_{uv} nin ayrıtlarından Şekil 34.16(b)-(d) de gösterilen üç yoldan biri ile geçmelidir. Eğer çevrim $[u, v, 1]$ tepesiyle başlarsa, $[u, v, 6]$ tepesinden çıkmalıdır ve parçacığın tepelerinin ya 12 sini birden (Şekil 34.16(b)) ya da $[u, v, 1]$ den $[u, v, 6]$ ya kadar altı tepeyi ziyaret eder (Şekil 34.16(c)). Sonraki durumda, çevrim $[v, u, 1]$ den $[v, u, 6]$ ya kadar olan tepeleri ziyaret etmek için parçacığa tekrar girmek zorunda kalacaktır. Benzer olarak, eğer çevrim $[v, u, 1]$ tepesiyle başlarsa $[v, u, 6]$ tepesinden çıkmalıdır ve parçacığın tepelerinin ya 12 sini birden (Şekil 34.16(d)) ya da $[v, u, 1]$ den $[v, u, 6]$ ya kadar altı tepeyi ziyaret eder (Şekil 34.16(c)). Parçacık boyunca 12 tepenin hepsini ziyaret eden başka hiçbir yol mümkün değildir. Özellikle, biri $[u, v, 1]$ i $[v, u, 6]$ ya bağlayan ve diğeri $[v, u, 1]$ i $[u, v, 6]$ ya bağlayan ve birleşimleri parçacığın bütün tepelerini içerecek şekilde iki tane tepe-ayrık yolun bulunması imkansızdır.



Şekil 34.17 Tepe-örtüsü probleminin bir örneğini hamilton-çevrim probleminin bir örneğine indirgeme. **(a)** Açık gölgelendirilmiş w ve y tepelerini içeren 2 boyutlu bir tepe örtüsüne sahip yönsüz bir G çizgesi. **(b)** Gölgelendirilmiş tepe örtüsüne karşılık gelen hamilton yola sahip, indirgeme ile üretilen yönsüz G' çizgesi. $\{w, y\}$ tepe örtüsü, hamilton çevrimde görülen $(s_1, [w, x, 1])$ ve $(s_2, [y, x, 1])$ ayrıtlarına karşılık gelir.

V' kümesinde parçacıklara ait olanlar dışındaki diğer tepeler *seçici tepelerdir*, $s_1, s_2, \dots, s_k$. G çizgesindeki örtünün k tane tepesini seçmek için G' çizgesindeki seçici tepelere bitişik ayrıtları kullanırız.

Parçacıklardaki ayrıtlara ilaveten, E' , Şekil 34.17 de gösterilen ayrıtların diğer iki çeşidini de içerir. İlk olarak, her bir $u \in V$ tepesi için, G de u ya bitişik ayrıtlara karşılık gelen bütün parçacıkları içeren bir yol oluşturmak amacıyla parçacıkların çiftlerine birleştirmek üzere ayrıtlar ekleriz. Her bir $u \in V$ tepesinin komşu tepelerini, keyfi olarak $u^{(1)}, u^{(2)}, \dots, u^{(\text{derece}(u))}$ şeklinde sıralarız; burada $\text{derece}(u)$, u ya komşu tepelerin sayısıdır. E' kümesine $\{([u, u^{(i)}, 6], [u, u^{(i+1)}, 1]) : 1 \leq i \leq \text{derece}(u) - 1\}$ ayrıtlarını ekleyerek, G' çizgesinde u ya bitişik ayrıtlara karşılık gelen tüm parçacıklar boyunca bir yol oluştururuz. Şekil 34.17 de, örneğin, w ye komşu tepeleri x, y, z olarak sıralarız ve böylece şeklin (b) kısmındaki G' çizgesi, $([w, x,$

$6], [w, y, 1])$ ve $([w, y, 6], [w, z, 1])$ ayrıtlarını içerir. Her bir $u \in V$ tepesi için G' çizgesindeki bu tepeler, G çizgesinde u ya bitişik ayrıtlara karşılık gelen bütün parçacıkları içeren bir yolu tamamlarlar.

Bu ayrıtların arkasındaki görüş şudur ki eğer G çizgesinin tepe örtüsünde bir $u \in V$ tepesi seçersek, G' çizgesinde $[u, u^{(1)}, 1]$ dan $[u, u^{(\text{derece}(u))}, 6]$ ya giden ve u ya bitişik ayrıtlara karşılık gelen tüm parçacıkları “örtün” bir yol inşa edebiliriz. Yani, bu parçacıkların her biri için $W_{u,u^{(i)}}$ diyelim, yol ya 12 tepenin hepsini içerir (u tepe örtüsünde ama $u^{(i)}$ değilse) ya da sadece $[u, u^{(i)}, 1], [u, u^{(i)}, 2], \dots, [u, u^{(i)}, 6]$ tepelerini içerir (hem u hem de $u^{(i)}$ tepe örtüsünde ise).

E' kümesindeki son ayrıt tipi, bu yolların her birinin $[u, u^{(1)}, 1]$ ilk tepesi ve $[u, u^{(\text{derece}(u))}, 6]$ son tepesini seçici tepelerin her biri ile birleştirir. Yani,

$$\begin{aligned} \{ (s_j, [u, u^{(1)}, 1]) : u \in V \text{ ve } 1 \leq j \leq k \} \\ \cup \{ (s_j, [u, u^{(\text{derece}(u))}, 6]) : u \in V \text{ ve } 1 \leq j \leq k \} \end{aligned}$$

ayrıtlarını dahil ederiz.

Ardından, G' çizgesinin boyutunun G çizgesinin boyutunda polinom olduğunu gösteririz ve dolayısıyla G' çizgesini G çizgesinin boyutunda polinom zamanda inşa edebiliriz. G' çizgesinin tepeleri, parçacıklarda olanlar ve aynı zamanda seçici tepelerdir. Parçacık başına 12 tepeye ek $k \leq |V|$ tane seçici tepe ile toplamda

$$\begin{aligned} |V'| &= 12 |E| + k \\ &\leq 12 |E| + |V| \end{aligned}$$

tane tepeye sahip oluruz. G' çizgesinin ayrıtları parçacıklarda olanlar, parçacıklar arasında olanlar ve seçici tepeleri parçacıklara birleştiren ayrıtlardır. Her bir parçacık için 14 ayrıt olmak üzere, tüm parçacıklar toplamda $14 |E|$ tane ayrıt içerir. Her bir $u \in V$ tepesi için G' çizgesi parçacıklar arasına giden $\text{derece}(u) - 1$ tane ayrıta sahiptir, bu durumda V deki tüm tepeler için toplamda

$$\sum_{u \in V} (\text{derece}(u) - 1) = 2 |E| - |V|$$

tane ayrıt parçacıklar arasına gider. Son olarak, G' çizgesi, bir seçici tepe ile V nin bir tepesini içeren her bir çift için iki ayrıt olmak üzere toplamda $2k |V|$ tane böyle ayrıt içerir. Dolayısıyla, G' çizgesinin toplam ayrıt sayısı

$$\begin{aligned} |E'| &= (14 |E|) + (2 |E| - |V|) + (2k |V|) \\ &= 16 |E| + (2k - 1) |V| \\ &\leq 16 |E| + (2 |V| - 1) |V|. \end{aligned}$$

Şimdi G den G' çizgesine dönüşümün bir indirgeme olduğunu göstereceğiz. Yani, ancak ve ancak G' çizgesi bir hamilton çevrime sahipse G nin k boyutlu bir tepe örtüsüne sahip olduğunu göstermeliyiz.

$G = (V, E)$ çizgesinin k boyutlu bir $V^* \subseteq V$ tepe örtüsüne sahip olduğunu varsayalım. $V^* = \{u_1, u_2, \dots, u_k\}$ olsun. Şekil 34.17 nin gösterdiği gibi, her bir $u_j \in V^*$ tepesi için aşağıda bahsedilen ayrıtları¹⁰ dahil ederek G' çizgesinde bir hamilton çevrimi oluştururuz. u_j ye bitişik ayrıtlara karşılık gelen bütün parçacıkları birleştiren $\{([u_j, u_j^{(i)}, 6], [u_j, u_j^{(i+1)}, 1]) : 1 \leq i \leq \text{derece}(u_j) - 1\}$ ayrıtlarını dahil et. Ayrıtların V^* kümesindeki bir ya da iki tepe tarafından örtülüp örtülmediğine bağlı olarak, Şekil 34.16(b)-(d) nin gösterdiği gibi bu parçacıkların içerisindeki ayrıtları da dahil ederiz. Hamilton çevrimi ayrıca

$$\begin{aligned} & \{(s_j, [u_j, u_j^{(1)}, 1]) : 1 \leq j \leq k\} \\ & \in \{(s_{j+1}, [u_j, u_j^{(\text{derece}(u_j))}, 6]) : 1 \leq j \leq k-1\} \\ & \cup \{(s_1, [u_k, u_k^{(\text{derece}(u_k))}, 6])\} \end{aligned}$$

ayıtlarını içerir.

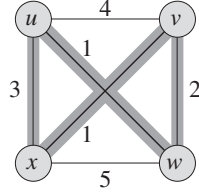
Şekil 34.17 yi inceleyerek, bu ayrıtların bir çevrim oluşturduğunu doğrulayabilirsiniz. Çevrim s_1 de başlar, u_1 e bitişik ayrıtlara karşılık gelen bütün parçacıkları ziyaret eder; ardından s_2 yi ziyaret eder, u_2 ye bitişik ayrıtlara karşılık gelen bütün parçacıkları ziyaret eder ve s_1 e dönünceye kadar bu şekilde devam eder. Çevrim her bir parçacığı, V^* kümesinin bir ya da iki tepesinin onunla ilişkili ayrıtı örtüp örtmediğine bağlı olarak, ya bir ya da iki kere ziyaret eder. Çünkü V^* , G için bir tepe örtüsüdür, E deki her bir ayrıt V^* kümesindeki bir tepeye bitişiktir ve bu nedenle çevrim G' çizgesinin her bir parçacığındaki her bir tepayı ziyaret eder. Çevrim her seçici tepayı de ziyaret ettiği için hamiltondur.

Diğer taraftan, $G' = (V', E')$ çizgesinin bir $C \subseteq E'$ hamilton çevrimine sahip olduğunu farzedelim.

$$V^* = \{u \in V : \text{bir } 1 \leq j \leq k \text{ için } (s_j, [u, u^{(1)}, 1]) \in C\} \quad (34.4)$$

kümesinin G için bir tepe örtüsü olduğunu iddia ediyoruz. Neden öyle olduğunu görmek için C nin bir seçici s_i tepesinde başlayan, bir $u \in V$ için $(s_i, [u, u^{(1)}, 1])$ ayrıtlarından geçen ve herhangi başka bir seçici tepeden geçmeksizin, bir seçici s_j tepesinde sonlanan maksimal yollara parçalanışını düşünelim. Bu şekildeki yolların her birini “örtü yolu” olarak adlandıralım. G' çizgesinin inşa edilışinden, her bir örtü yolu bir s_i den başlamalı, bir $u \in V$ için $(s_i, [u, u^{(1)}, 1])$ ayrıtlarını almalı, E de u ya bitişik ayrıtlara karşılık gelen bütün parçacıklardan geçmeli ve ardından bir seçici s_j tepesinde sonlanmalıdır. Bu örtü yolunu p_u olarak adlandırırız ve denklem (34.4) e göre u yu V^* kümesine koyarız. p_u tarafından ziyaret edilen her bir parçacık, bir $v \in V$ için W_{uv} ya da W_{vu} olmalıdır. p_u tarafından ziyaret edilen her bir parçacığın tepeleri bir ya da iki örtü yolu tarafından ziyaret edilir. Eğer bir örtü yolu tarafından ziyaret edilirlerse $(u, v) \in E$ ayrıtı, G de u tepesi tarafından örtülür. Eğer iki örtü yolu parçacığı ziyaret ederse diğer örtü yolu p_v olmalıdır ki bu

¹⁰Teknik olarak, bir çevrimi ayrıtlardan ziyade tepeler bakımından tanımlarız (Kısım B.4'e bakınız). Anlaşılr olması için burada notasyonu farklı şekilde kullanırız ve hamilton çevrimi ayrıtlar üzerinden tanımlarız.



Şekil 34.18 Gezgin-satıcı probleminin bir örneği. Gölgeleştirilmiş ayrıtlar 7 maliyeti ile en küçük-maliyetli turu gösterir.

$v \in V^*$ anlamına gelir, ve $(u, v) \in E$ ayrıtı hem u hem de v tarafından örtülür. Her bir parçadaki her bir tepe, bir örtü yolu tarafından ziyaret edildiği için E deki her bir ayrıtın V^* kümesindeki bir tepe tarafından örtüldüğünü görürüz. ■

34.5.4 Gezgin-satıcı problemi

Hamilton-çevrim problemiyle yakından ilişkili olan **gezgin-satıcı probleminde** bir satıcı n tane şehri ziyaret etmelidir. Problemi n tepeli bir tam çizge olarak modelleyerek, satıcının her bir şehri tam olarak bir kez ziyaret eden ve başladığı şehre geri dönen bir **tur**, ya da hamilton döngüsü, yapmak istediğini söyleyebiliriz. Satıcı, i şehrinden j şehrine seyahat etmek için bir negatif olmayan tam sayı maliyetine $c(i, j)$ girer ve toplam maliyeti en küçük olan turu yapmak ister; burada toplam maliyet turun ayrıtları boyunca bireysel maliyetlerin toplamıdır. Örneğin, Şekil 34.18 de en küçük-maliyetli tur, 7 maliyet ile $\langle u, w, v, x, u \rangle$ turudur. İlişkili karar problemi için formal dil aşağıdaki gibidir.

$$\begin{aligned} \text{GSP} = \{ \langle G, c, k \rangle : & G = (V, E) \text{ bir tam çizge,} \\ & V \times V \rightarrow \mathbb{N} \text{ bir fonksiyon,} \\ & k \in \mathbb{N} \text{ ve} \\ & G, \text{ en fazla } k \text{ maliyetli bir gezgin-satıcı turuna sahip} \} \end{aligned}$$

Aşağıdaki teorem gezgin-satıcı problemi için hızlı bir algoritmanın bulunmasının pek mümkün olmadığını gösterir.

Teorem 34.14

Gezgin-satıcı problemi NP-tamdır.

Kanıt Öncelikle GSP nin NP'ye ait olduğunu gösteririz. Problemin bir örneği verildiğinde sertifika olarak turdaki n tepenin dizisini kullanırız. Doğrulama algoritması, bu dizinin her bir tepeyi tam olarak bir kez içerip içermediğini kontrol eder, ayrıt maliyetlerini toplar ve toplamın en fazla k olup olmadığını kontrol eder. Bu işlem kesinlikle polinom zamanda yapılabilir.

GSP nin NP-zor olduğunu göstermek için HAM-CYCLE $\leq_p$ TSP olduğunu gösteririz. $G = (V, E)$ HAM-CYCLE ın bir örneği olsun. TSP nin bir örneğini aşağıdaki gibi inşa ederiz. $E' = \{(i, j) : i, j \in V \text{ ve } i \neq j\}$ olmak üzere tümleyen $G' = (V, E')$ çizgesini oluştururuz ve c maliyet fonksiyonunu

$$c(i, j) = \begin{cases} 0, & (i, j) \in E \text{ ise,} \\ 1, & (i, j) \notin E \text{ ise,} \end{cases}$$

ile tanımlarız. (G, yönsüz olduğu için kendi kendine döngü içermez ve bu nedenle tüm $v \in V$ için $c(u, v) = 1$ dir) Bu durumda GSP nin örneği, polinom zamanda kolayca üretebileceğimiz $\langle G', c, 0 \rangle$ dir.

Şimdi G nin bir hamilton çevrime sahip olması için gerek ve yeter koşulun G' çizgesini en fazla 0 maliyetli bir tura sahip olması olduğunu gösteririz. G çizgesinin bir h hamilton çevrime sahip olduğunu varsayalım. h deki her bir ayrıt E ye aittir ve dolayısıyla bu ayrıtlar G' çizgesinde 0 maliyete sahiptir. Bu nedenle, h , G' çizgesinde 0 maliyetli bir turdur. Diğer taraftan, G' çizgesinin en fazla 0 maliyetli bir h' turuna sahip olduğunu farzedelim. E' kümesindeki ayrıtların maliyetleri 0 ve 1 olduğu için h' turunun maliyeti tam olarak 0 dır ve turdaki her bir ayrıt 0 maliyetine sahip olmalıdır. Dolayısıyla, h' sadece E deki ayrıtları içerir. h' turunun, G çizgesinde bir hamilton çevrimi olduğu sonucuna ulaşırız. ■

34.5.5 Altküme-toplam problemi

Bu kısımda bir aritmetik NP-tam problemi ele alacağız. **Altküme-toplam probleminde**, pozitif tam sayıların sonlu bir S kümesi ve bir $t > 0$ **hedef** tam sayısı veriliyor. Elemanları toplamı t olan bir $S' \subseteq S$ altkümesinin bulunup bulunmadığını sorarız. Örneğin, $S = \{1, 2, 7, 14, 49, 98, 343, 686, 2409, 2793, 16808, 17206, 117705, 117993\}$ ve $t = 138457$ ise $S' = \{1, 2, 7, 98, 343, 686, 2409, 17206, 117705\}$ altkümesi bir çözümdür.

Her zamanki gibi, problemi bir dil olarak tanımlarız:

SUBSET-SUM = $\{\langle S, t \rangle : t = \sum_{s \in S'} s \text{ olacak şekilde bir } S' \subseteq S \text{ altkümesi vardır.}\}$

Herhangi bir aritmetik problemde olduğu gibi, standart kodlamamızın girdi tam sayılarını ikili sistemde kodlanmış olarak varsaydığını hatırlamak önemlidir. Bu varsayım ile birlikte, altküme-toplam probleminin hızlı bir algoritmaya sahip olmasının mümkün olmadığını gösterebiliriz.

Teorem 34.15

Altküme-toplam problemi NP-tamdır.

Kanıt SUBSET-SUM un NP'de olduğunu göstermek amacıyla, problemin bir $\langle S, t \rangle$ örneği için S' altkümesi bir sertifika olsun. Bir doğrulama algoritması, $t = \sum_{s \in S'} s$ olup olmadığını polinom zamanda kontrol edebilir.

Şimdi 3-CNF-SAT $\leq_p$ SUBSET-SUM olduğunu gösterelim. $x_1, x_2, \dots, x_n$ değişkenleri üzerinde, her biri tam olarak üç farklı literal içeren $C_1, C_2, \dots, C_k$

cümlecikleri ile bir 3-CNF ϕ formülü verildiğinde, indirgeme algoritması altküme-toplam probleminin bir $\langle S, t \rangle$ örneğini inşa eder. Öyle ki ϕ , ancak ve ancak toplamı tam olarak t olan bir S altkütmesi varsa gerçekleştirilebilirdir. Genelliği kaybetmeden, ϕ formülü hakkında iki basitleştirici varsayımda bulunuruz. İlk olarak, hiçbir cümlecik hem bir değişkeni hem de onun değerini içermez, böyle bir cümlecik değişkenlere herhangi bir değer ataması tarafından otomatik olarak gerçekleştirilir. İkincisi, her bir değişken en az bir cümlecikte bulunur; çünkü hiçbir cümlede bulunmayan bir değişkene hangi değer atandığı önemli değildir.

İndirgeme, her bir x_i değişkeni için S kümesinde iki sayı ve her bir C_j cümlecigi için S de iki sayı üretir. Sayıları 10 tabanında üreteceğiz; burada her bir sayı $n + k$ basamak içerir ve her bir basamak ya bir değişkene ya da bir cümlecige karşılık gelir. On tabanı (ve göreceğimiz gibi diğer tabanlar), ihtiyacımız olan, daha küçük basamaklardan büyük basamaklara eldeleri engelleme özelliğine sahiptir.

Şekil 34.19 un gösterdiği gibi, S kümesi ve t hedefini aşağıdaki gibi inşa ederiz. Her bir basamak pozisyonunu ya bir değişken ya da bir cümlecik ile etiketleriz. En az önemli k basamak, cümlecikler ile etiketlenir ve en önemli n basamak, değişkenler ile etiketlenir.

- t hedefi, bir değişken tarafından etiketlenmiş her bir basamakta bir 1 e ve bir cümlecik tarafından etiketlenmiş her bir basamakta bir 4 e sahiptir.
- S kümesi her bir x_i değişkeni için v_i ve v'_i tam sayılarını içerir. v_i ve v'_i tam sayılarının her biri, x_i ile etiketlenmiş basamakta bir 1'e ve diğer değişken basamaklarda 0 lara sahiptir. Eğer x_i literalı C_j cümleciginde bulunursa, C_j tarafından v_i de etiketlenmiş basamak bir 1 içerir. Eğer $\neg x_i$ literalı C_j cümleciginde bulunursa, C_j tarafından v'_i tam sayısında etiketlenmiş basamak bir 1 içerir. v_i ve v'_i tam sayılarında cümlecikler tarafından etiketlenmiş diğer tüm basamaklar 0 dır.

S kümesindeki tüm v_i ve v'_i değerleri tektir. Neden? $1 \neq i$ için hiçbir v_i ya da v'_i değeri en önemli n basamakta v_i ve v'_i değerlerine eşit olamaz. Ayrıca, yukarıdaki basitleştirici varsayımlarımızdan, hiçbir v_i ve v'_i değeri en az anlamlı k basamağın tamamında eşit olamaz. Eğer v_i ve v'_i değerleri eşit olsaydı x_i ve $\neg x_i$, cümleciklerin tam olarak aynı kümesinde bulunmak zorunda kalırdı. Ancak hiçbir cümlecğin x_i ve $\neg x_i$ literallerinin ikisini birden içermediğini; ayrıca ya x_i ya da $\neg x_i$ nin bir cümlecikte bulunduğunu varsayarız ve bu nedenle de v_i ve v'_i değerlerinin farklı olduğu bir C_j cümlecigi var olmalıdır.

- S kümesi her bir C_j cümlecigi için s_j ve s'_j tam sayılarını içerir. s_j ve s'_j tam sayılarının her biri, C_j ile etiketlenmiş olan dışındaki tüm basamaklarda 0 lara sahiptir. s_j için C_j basamağında bir 1 vardır ve s'_j bu basamakta bir 2 ye sahiptir. Bu tam sayılar, 4 hedef değerine eklemek için her bir cümle-etiketli basamak pozisyonunu elde etmek amacıyla kullandığımız “aylak değişkenler” dir.

Şekil 34.19 un basit irdelemesi S deki tüm s_j ve s'_j değerlerinin S kümesinde tek olduğunu gösterir.

Herhangi bir basamak pozisyonunda, basamakların en büyük toplamının 6 olduğuna dikkat ediniz. Bu durum cümlecikler ile etiketlenmiş basamaklarda or-

		x_1	x_2	x_3	C_1	C_2	C_3	C_4
v_1	=	1	0	0	1	0	0	1
v'_1	=	1	0	0	0	1	1	0
v_2	=	0	1	0	0	0	0	1
v'_2	=	0	1	0	1	1	1	0
v_3	=	0	0	1	0	0	1	1
v'_3	=	0	0	1	1	1	0	0
s_1	=	0	0	0	1	0	0	0
s'_1	=	0	0	0	2	0	0	0
s_2	=	0	0	0	0	1	0	0
s'_2	=	0	0	0	0	2	0	0
s_3	=	0	0	0	0	0	1	0
s'_3	=	0	0	0	0	0	2	0
s_4	=	0	0	0	0	0	0	1
s'_4	=	0	0	0	0	0	0	2
t	=	1	1	1	4	4	4	4

Şekil 34.19 3-CNF-SAT'ın SUBSET-SUM'un indirgemesi. $C_1 = (x_1 \vee \neg x_2 \vee \neg x_3)$, $C_2 = (\neg x_1 \vee \neg x_2 \vee \neg x_3)$, $C_3 = (\neg x_1 \vee \neg x_2 \vee x_3)$ ve $C_4 = (x_1 \vee x_2 \vee x_3)$, olmak üzere 3-CNF'deki formül $\phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$ dir. ϕ nin gerçekleşen bir ataması $\langle x_1 = 0, x_2 = 0, x_3 = 1 \rangle$ şeklindedir. İndirgeme tarafından üretilen S kümesi gösterilen 10-tabanlı sayılardan oluşur; yukarıdan aşağıya okuyarak, $S = \{1001001, 1000110, 100001, 101110, 10011, 11100, 1000, 2000, 100, 200, 10, 20, 1, 2\}$. t hedefi 1114444'dür. $S' \subseteq S$ altkümesi açık gölgelendirilmiştir ve gerçekleşen atamaya karşılık gelen v'_1, v'_2 ve v_3 değerlerini içerir. Ayrıca C_1 den C_4 e kadar olan cümlelerle etiketlenmiş basamaklarda 4 hedef değerini başarmak için $s_1, s'_1, s'_2, s_3, s_4$ ve s'_4 aylak değişkenlerini de içerir.

taya çıkar (v_i ve v'_i değerlerinden üç tane 1, s_j ve s'_j değerlerinden artı 1 ve 2). Bu sayıları 10 tabanında yazdığımızda, dolayısıyla, küçük basamaklardan büyük basamaklara hiçbir elde ortaya çıkamaz.¹¹

İndirgemeyi polinom zamanda gerçekleştirebiliriz. S kümesi, her biri $n + k$ basamağa sahip $2n + 2k$ değer içerir ve her bir basamağı üretmek için gerekli zaman $n + k$ nin polinomudur. t hedefi $n + k$ basamağa sahiptir ve indirgeme her birini sabit zamanda üretir.

Şimdi göstereceğiz ki 3-CNF ϕ formülü gerçekleştirilebilirdir ancak ve ancak toplamı t olan bir $S' \subseteq S$ altkümesi vardır. Öncelikle, ϕ formülünün gerçekleşen bir ataması olduğunu varsayalım. $i = 1, 2, \dots, n$ için bu atamada $x_i = 1$ ise v_i yi S' kümesine ekle. Aksi halde, v'_i değerini ekle. Diğer bir deyişle, S' kümesine tam

¹¹Aslında, $b \geq 7$ olan herhangi bir b tabanı işe yarar. Bu altkısımın başındaki örnek, sıralı düzende listelenmiş S kümesi ile 7 tabanında yazılmış Şekil 34.19'daki t hedefidir.

olarak, gerçekleşen atamada 1 değerli literallere karşılık gelen v_i ve v'_i değerlerini ekleriz. Tüm i değerleri için v_i ya da v'_i değerlerini, ikisini birden değil, ekleyip tüm s_j ve s'_j lerde değişkenler ile etiketlenmiş basamaklara 0 koyduktan sonra her bir değişken-etiketli basamak için S' kümesinin değerleri toplamının 1 olması gerektiğini görürüz; bu basamaklar t hedefinin o basamaklarına uyar. Her bir cümlecik gerçekleştirildiği için cümlecik 1 değerli bir literal içerir. Dolayısıyla, bir cümlecik tarafından etiketlenmiş her bir basamak, S' kümesindeki bir v_i ya da v'_i değeri tarafından toplamına katkıda bulunulmuş en az bir 1 e sahiptir. Aslında, 1, 2 ya da 3 literal her bir cümlecikte 1 olabilir ve böylece her bir cümlecikte-etiketli basamak S' kümesindeki v_i ve v'_i değerlerinden 1, 2 ya da 3 toplamına sahiptir. Örneğin, Şekil 34.19 da $\neg x_1$, $\neg x_2$ ve x_3 literalleri gerçekleşen bir atamada 1 değerine sahiptir. C_1 ve C_4 değerlerinin her biri bu literallerin tam olarak bir tanesini içerir, dolayısıyla v'_1 , v'_2 ve v_3 , C_1 ve C_4 için basamaklardaki toplama 1 katkısını sağlarlar. C_2 cümlecigi bu literallerden ikisini içerir ve v'_1 , v'_2 ile v_3 , C_2 için basamaktaki toplama 2 katkısını sağlarlar. C_3 cümlecigi bu literallerin üçünü de içerir ve v'_1 , v'_2 ile v_3 , C_3 için basamaktaki toplama 3 katkısını sağlarlar. S' kümesine $\{s_j, s'_j\}$ aylak değişkenlerinin boş olmayan uygun bir altkümesini dahil ederek C_j cümlecigi ile etiketlenmiş her bir basamakta 4 hedefine ulaşırız. Şekil 34.19 da, S' kümesi $s_1, s'_1, s'_2, s_3, s_4$ ve s'_4 değişkenlerini içerir. Toplamın bütün basamaklarında hedefe uyduğumuz için ve hiçbir eldelik ortaya çıkmadığından S' kümesinin değerleri toplamı t dir.

Şimdi toplamı t olan bir $S' \subseteq S$ altkümesi olduğunu farzedelim. S' altkümesi, her bir $i = 1, 2, \dots, n$ için v_i ve v'_i değerlerinden tam olarak birini içermeli, aksi halde değişkenler tarafından etiketli basamaklar 1 toplamını vermez. $v_i \in S'$ ise $x_i = 1$ olarak ayarlarız. Aksi halde, $v'_i \in S'$ dir ve $x_i = 0$ olarak ayarlarız. Her C_j cümleciginin, $j = 1, 2, \dots, k$, bu atama tarafından tatmin edildiğini iddia ederiz. Bu iddiayı kanıtlamak için C_j ile etiketlenmiş basamakta bir 4 toplamı elde etmek için S' altkümesinin C_j ile etiketli basamakta bir 1 e sahip olan v_i veya v'_i değerlerinden en az birini içermesi gerektiğine dikkat ediniz; çünkü s_j , s'_j aylak değişkenlerinin katkıları birlikte en fazla 3 toplamını verir. S' altkümesi, C_j nin konumunda bir 1 e sahip bir v_i içerirse, x_i literalı C_j cümleciginde bulunur. $v_i \in S'$ iken $x_i = 1$ olarak ayarladığımız için C_j cümlecigi gerçekleştirilir. S' altkümesi, o konumda bir 1 e sahip bir v'_i içerirse $\neg x_i$ literalı C_j cümleciginde bulunur. $v'_i \in S'$ iken $x_i = 0$ olarak ayarladığımız için C_j cümlecigi yine cümlecigi. Dolayısıyla, ϕ formülünün bütün cümlecikleri gerçekleştirilir ve kanıt tamamlanır. ■

Alıştırmalar

34.5-1

Altçizge-izomorfizma problemi yönsüz G_1 ve G_2 çizgelerini alır ve G_1 in G_2 nin bir altçizgesine izomorf olup olmadığını sorar. Altçizge-izomorfizma probleminin NP-tam olduğunu gösteriniz.

34.5-2

$m \times n$ tipinde bir A tam sayı matrisi ve bir b tam sayı m -vektörü verildiğinde **0-1 tam sayılı-programlama problemi**, $Ax \leq b$ olacak şekilde $\{0, 1\}$ kümesindeki

elemanlarla bir x tam sayı n -vektörünün var olup olmadığını sorar. 0-1 tam sayı-programlama probleminin NP-tam olduğunu kanıtlayınız (*İpucu*: 3-CNF-SAT tan indirgeyiniz).

34.5-3

Tam sayılı doğrusal-programlama problemi, x vektörünün değerlerinin sadece 0 ya da 1 yerine herhangi bir tam sayı olabilmesi dışında Alıştırma 34.5-2 de verilen 0-1 tam sayı-programlama problemi gibidir. 0-1 tam sayı-programlama probleminin NP-zor olduğunu varsayarak tam sayılı doğrusal-programlama probleminin NP-tam olduğunu gösteriniz.

34.5-4

t hedef değeri birli sistemde ifade edilirse altküme-toplam probleminin polinom zamanda nasıl çözüleceğini gösteriniz.

34.5-5

Küme-parçalama problemi, sayıların bir S kümesini girdi olarak alır. Bu problemde, sayılar, $\sum_{x \in A} x = \sum_{x \in \bar{A}} x$ olacak şekilde, A ve $\bar{A} = S - A$ kümelerine parçalanabilir mi diye sorulur. Küme-parçalama probleminin NP-tam olduğunu gösteriniz.

34.5-6

Hamilton-yol probleminin NP-tam olduğunu gösteriniz.

34.5-7

En uzun-basit-çevrim problemi, bir çizgede en büyük uzunluklu bir basit çevrime (tekrarlı tepeler yok) belirleme problemidir. İlişkili karar problemini formülleştiriniz ve karar probleminin NP-tam olduğunu gösteriniz.

34.5-8

Yarı 3-CNF gerçekleştirilebilirlik probleminde, n değişkenli ve m cümleli, m çift olmak üzere, bir 3-CNF ϕ formülü veriliyor. Cümleciklerin tam olarak yarısı 0 ve tam olarak yarısı 1 olarak değerlendirilecek şekilde ϕ nin değişkenlerine bir doğruluk ataması olup olmadığını belirlemek isteriz. Yarı 3-CNF gerçekleştirilebilirlik probleminin NP-tam olduğunu kanıtlayınız.

Problemler

34-1 Bağımsız küme

Bir $G = (V, E)$ çizgesinin bir **bağımsız kümesi**, tepelerin bir $V' \subseteq V$ altkümesidir öyle ki E deki her bir ayrıt V' kümesindeki en fazla bir tepeye bitişiktir. **Bağımsız-küme problemi**, G de en büyük-boyutlu bir bağımsız küme bulmaktır.

- a.** Bağımsız-küme problemi için bir ilişkili karar problemi ifade ediniz ve bu karar probleminin NP-tam olduğunu kanıtlayınız. (*İpucu:* Klik probleminden indirgeyiniz.)
- b.** (a) kısmında tanımladığınız karar problemini çözen bir “kara-kutu” alt programının size verildiğini varsayın. En büyük boyutlu bir bağımsız küme bulmak için bir algoritma veriniz. Kara kutuya yapılan sorguları tek bir adım olarak saydığınızda, algoritmanızın çalışma zamanı $|V|$ ve $|E|$ nin polinomu olmalıdır. Bağımsız-küme karar problemi NP-tam olmasına rağmen, belirli özel durumlar polinom-zamanda çözülebilir.
- c.** G deki her bir tepenin derecesi 2 olduğunda bağımsız-küme problemini çözen etkili bir algoritma veriniz.
- d.** G , iki parçalı olduğunda bağımsız-küme problemini çözen etkili bir algoritma veriniz. Çalışma zamanını analiz ediniz ve algoritmanızın doğru olarak çalıştığını kanıtlayınız (*İpucu:* Kısım 26.3 ün sonuçlarını kullanınız).

34-2 Bonnie ve Clyde

Bonnie ve Clyde az önce bir banka soydular. Bir çanta dolusu paraları var ve bunu bölüşmek istiyorlar. Aşağıdaki senaryoların her biri için ya polinom-zamanlı bir algoritma veriniz ya da problemin NP-tam olduğunu kanıtlayınız. Her bir durumdaki girdi, her birinin değeri ile birlikte çantadaki n nesnenin bir listesidir.

- a.** Çanta n tane madeni para içeriyor; ancak sadece 2 farklı değer var: bazı paralar x dolar değerinde ve bazıları y dolar değerindedir. Bonnie ve Clyde, parayı tamı tamına eşit olarak paylaşmak istiyor.
- b.** Çanta farklı değerlerin keyfi bir sayısı ile n tane madeni para içeriyor; fakat her bir değer 2 nin negatif olmayan bir tam sayı kuvveti, yani olası değerler 1 dolar, 2 dolar, 4 dolar vs. Bonnie ve Clyde parayı tamı tamına eşit olarak paylaşmak istiyor.
- c.** Çanta n tane çek içeriyor ve tesadüfe bakın ki bu çekler “Bonnie ya da Clyde” olarak yazılmış. Her biri tam olarak aynı miktarda para alacak şekilde bu çekleri paylaşmak istiyorlar.
- d.** Çanta (c) kısmındaki gibi n tane çek içeriyor; fakat bu kez Bonnie ve Clyde aradaki farkın 100 dolardan daha fazla olmayacağı bir paylaşımı kabul etmeye razılar.

34-3 Çizge renklendirme

Haritacılar bir harita üzerindeki ülkeleri renklendirirken, aynı sınırı paylaşan iki ülke aynı renk olmadığı sürece mümkün olduğu kadar az renk kullanmaya çalışırlar. Bu problemi yönsüz bir $G = (V, E)$ çizgesi ile modelleyebiliriz. Bu çizgedeki her bir tepe bir ülkeyi belirtir ve ilgili ülkeleri bir sınırı paylaşan tepeler komşudur. Bu durumda bir ***k*-renklendirme**, her $(u, v) \in E$ ayrıtı için $c(u) \neq c(v)$ olacak şekilde bir $c : V \rightarrow \{1, 2, \dots, k\}$ fonksiyonudur. Diğer bir deyişle, $1, 2, \dots, k$ sayıları k tane rengi belirtir ve komşu tepeler farklı renklerde olmalıdır. ***Çizge-renklendirme problemi***, verilen bir çizgeyi renklendirmek için gerekli renklerin en küçük sayısını belirlemektir.

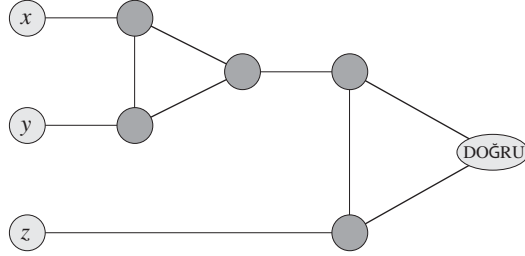
- a. Bir çizgenin bir 2-renklendirmesini belirlemek için eğer varsa, etkili bir algoritma veriniz.
- b. Çizge-renklendirme problemini bir karar problemi olarak ifade ediniz. Çizge-renklendirme problemi polinom zamanlı çözülebilir ancak ve ancak karar probleminizin de polinom zamanlı çözülebilir olduğunu gösteriniz.
- c. 3-COLOR dili 3 renkle renklendirilebilen çizgelerin kümesi olsun. Eğer 3-COLOR NP-tam ise (b) kısmındaki karar probleminizin de NP-tam olduğunu gösteriniz.

3-COLOR dilinin NP-tam olduğunu kanıtlamak için 3-CNF-SAT tan bir indirgeme kullanırız. $x_1, x_2, \dots, x_n$ gibi n değişken ve m cümleli bir ϕ formülü verildiğinde bir $G = (V, E)$ çizgesini şu şekilde oluştururuz. V kümesi her bir değişken için bir tepe, her bir değişkenin değili için bir tepe, her bir cümlecik için 5 tepe ve 3 özel tepeden oluşur: DOĞRU, YANLIŞ, KIRMIZI. Çizgenin ayrıtıları iki tiptedir: cümleciklerden bağımsız olan “literal” ayrıtılar ve cümleciklere bağlı “cümlecik” ayrıtılar. Literal ayrıtılar, özel tepeler üzerinde bir üçgen oluştururlar ve ayrıca $i = 1, 2, \dots, n$ için $x_i, \neg x_i$ ve KIRMIZI üzerinde de bir üçgen oluştururlar.

- d. Literal ayrıtılar içeren bir çizgenin herhangi bir c 3-renklendirmesinde, bir değişken ve değilinden tam olarak birinin c (DOĞRU) ve diğerinin c (YANLIŞ) olarak renklendiği üzerinde tartışınız. ϕ nin herhangi bir doğruluk ataması için sadece literal ayrıtlardan oluşan çizgenin bir 3-renklendirmesinin bulunduğu konusunu tartışınız.

Şekil 34.20 deki parçacık, bir $(x \vee y \vee z)$ cümlecğine karşılık gelen koşulu yerine getirmeye yardım eder. Her bir cümlecik, şekilde koyu olarak gölgelendirilmiş 5 tepenin tek bir kopyasına gerek duyar; cümlecğin literallerini ve özel DOĞRU tepesini gösterildiği gibi birleştirir.

- e. x, y ve z den her biri c (DOĞRU) ya da c (YANLIŞ) olarak boyalı ise, parçacığın 3-renklendirilebilir ancak ve ancak x, y ve z den en az biri c (DOĞRU) olarak boyalı olduğunu tartışınız.
- f. 3-COLOR un NP-tam olduğu kanıtını tamamlayınız.



Şekil 34.20 Problem 34-3 de kullanılan ve bir $(x \vee y \vee z)$ cümlesine karşılık gelen parçacık.

34-4 Kazançlar ve bitiş zamanları ile çizelgeleme

Bir makine ve her biri makinede zaman gerektiren $a_1, a_2, \dots, a_n$ işlerinin bir kümesine sahip olduğumuzu varsayalım. Her bir a_j işi makinede t_j zaman birimi gerektirir (onun işlem süresi), bir p_j kazancı sağlar ve bir d_j bitiş zamanı vardır. Makine bir seferde sadece bir işi gerçekleştirebilir ve a_j işi, t_j ardışık zaman birimi için kesintisiz olarak işlemelidir. Eğer a_j işini d_j bitiş zamanına kadar tamamlarsak bir p_j kazancı elde ederiz; ancak işi bitiş zamanından sonra tamamlarsak hiçbir kazanç sağlamayız. Bir optimizasyon problemi olarak şöyle ifade ederiz: n tane işin bir kümesi için işlem süreleri, kazançlar ve bitiş zamanları veriliyor; tüm işleri tamamlayan ve en büyük kazanç miktarını veren bir çizelge bulmak istiyoruz. İşlem süreleri, kazançlar ve zaman sınırlarının tümü negatif olmayan sayılardır.

- Bu problemi bir karar problemi olarak ifade ediniz.
- Karar probleminin NP-tam olduğunu gösteriniz.
- Bütün işlem sürelerinin 1 den n ye kadar tam sayılar olduğunu farzederek, karar problemi için polinom-zamanlı bir algoritma veriniz.
- Bütün işlem sürelerinin 1 den n ye kadar tam sayılar olduğunu farzederek, optimizasyon problemi için polinom-zamanlı bir algoritma veriniz.

Bölüm notları

Garey ve Johnson [129] tarafından yazılan kitap, teoriyi enine boyuna ele alması ve 1979 da NP-tam olduğu bilinen birçok problemin bir kataloğunu temin etmesiyle NP-tamlığa harika bir rehber sağlar. Teorem 34.13 ün kanıtı onların kitabından uyarlanmıştır ve Kısım 34.5 in başındaki NP-tam problem alanlarının listesi onların içindekiler tablosundan çekilmiştir. Johnson, NP-tamlıktaki yeni gelişmeleri rapor eden 23 sütunluk bir seriyi 1981 ve 1992 arasında Journal of

Algorithms’de yazdı. Hopcroft, Motwani ve Ullman [177], Lewis ve Papadimitriou [236], Papadimitriou [270] ve Sipser [317] karmaşıklık teorisinin içeriğinde NP-tamlığın iyi uygulamalarına sahiptir. NP-tamlık ve çeşitli indirgemeler ayrıca Aho, Hopcroft ve Ullman [5]; Dasgupta, Papadimitriou ve Vazirani [82]; Johnsonbaugh ve Schaefer [193] ile Kleinberg ve Tardos [208] tarafından yazılan kitaplarda da bulunurlar.

P sınıfı, 1964’de Cobham [72] tarafından ve bundan bağımsız olarak ayrıca NP sınıfını da tanıtan ve $P \neq NP$ varsayımında bulunan Edmonds [100] tarafından 1965’de takdim edildi. NP-tamlık kavramı, formül gerçekleştirilebilirlik ve 3-CNF gerçekleştirilebilirlik için ilk NP-tamlık kanıtlarını veren Cook [75] tarafından 1971 yılında önerildi. Levin [234] bağımsız olarak, bir resim döşeme problemi için bir NP-tamlık kanıtı vererek kavramı keşfetmiştir. Karp [199] indirgemeler metodolojisini 1972’de tanıttı ve NP-tam problemlerin zengin çeşitliliğini gösterdi. Karp’ın makalesi klik, tepe-örtüsü ve hamilton-çevrim problemlerinin orijinal NP-tamlık kanıtlarını içerir. O zamandan beri binlerce problemin NP-tam olduğu birçok araştırmacı tarafından kanıtlandı. Papadimitriou, 1995’de Karp’ın 60. doğumgünü kutlamasındaki bir konuşmada şöyle bir yorum yapmıştır: “her yıl yaklaşık 6000 makale, başlığında, özetinde ya da anahtar kelimeler listesinde ‘NP-tam’ terimini içerir. Bu, ‘derleyici’, ‘veritabanı’, ‘uzman’, ‘sinir ağları’ ya da ‘işletim sistemi’ terimlerinin her birinden daha fazladır.”

Karmaşıklık teorisinde son zamanlarda yapılan çalışma, yaklaşık çözümleri hesaplamanın karmaşıklığı üzerine ışık tutmuştur. Bu çalışma, “olasılıksal olarak denetlenebilir kanıtları” kullanarak NP’nin yeni bir tanımını verir. Bu yeni tanım, klik, tepe örtüsü, üçgen eşitsizlikli gezgin-satıcı problemi gibi problemler ve diğer birçok problem için iyi yaklaşık çözümler hesaplamanın NP-zor olduğunu ve dolayısıyla bunun optimal çözümleri hesaplamaktan daha kolay olmadığını belirtir. Bu alana bir giriş, Arora’nın tezinde [20]; Arora ve Lund tarafından yazılan Hochbaum [172] kitabındaki bir bölümde; Arora tarafından yapılan bir incelemede [21]; Mayr, Prömel ve Steger tarafından düzenlenen bir kitapta [246] ve Johnson tarafından yapılan bir inceleme makalesinde [191] bulunabilir.

Uygulamadaki problemlerin birçoğu NP-tamdır, buna rağmen vazgeçilmeyecek kadar önemlidirler; çünkü polinom zamanda bir optimal çözümün nasıl bulunduğunu bilmiyoruz. Bir problem NP-tam olsa bile bir ümit olabilir. NP-tamlığın üstesinden gelmek için en az üç yolumuz vardır. İlk olarak, mevcut girdiler küçükse, üssel çalışma zamanlı bir algoritma kusursuz bir şekilde memnun edici olabilir. İkincisi, polinom zamanda çözebildiğimiz önemli özel durumları ayrı tutabiliriz. Üçüncü olarak, polinom zamanda *yakın-optimal* çözümler bulmak için yaklaşımlar öne sürebiliriz (ya en kötü durumda ya da beklenen durumda). Uygulamada yakın-optimallik sıklıkla yeterince iyidir. Yakın-optimal çözümler geri döndüren bir algoritmayı bir **yaklaşım algoritması** olarak adlandırırız. Bu bölüm, çeşitli NP-tam problemler için polinom-zamanlı yaklaşım algoritmaları sunar.

Yaklaşım algoritmaları için performans oranları

Farz edelim ki, her bir olası çözümün bir pozitif maliyete sahip olduğu bir optimizasyon problemi üzerinde çalışıyoruz ve yakın-optimal bir çözüm bulmak istiyoruz. Probleme bağlı olarak, maksimum olası maliyetli ya da minimum olası maliyetli olacak şekilde bir optimal çözüm tanımlayabiliriz; yani problem ya bir maksimizasyon ya da bir minimizasyon problemidir.

Herhangi bir n boyutlu girdi için algoritma tarafından üretilen çözümün C maliyeti, bir optimal çözümün C^* maliyetinin bir $\rho(n)$ katı dahilindeyse, bir problem için bir algoritma bir $\rho(n)$ **yaklaşım oranına** sahiptir deriz:

$$\max \left(\frac{C}{C^*}, \frac{C^*}{C} \right) \leq \rho(n) . \quad (35.1)$$

Bir algoritma bir $\rho(n)$ yaklaşım oranını gerçekleştirirse onu **$\rho(n)$ -yaklaşım algoritması** olarak adlandırırız. Yaklaşım oranı ve $\rho(n)$ -yaklaşım algoritmasının tanımları, minimizasyon ve maksimizasyon problemlerinin her ikisine de uyar. Bir maksimizasyon problemi için $0 < C \leq C^*$ ve C^* / C oranı, bir optimal çözümün maliyetini yaklaşık çözümün maliyetinden daha büyük yapan katsayıyı verir.

Benzer olarak, bir minimizasyon problemi için $0 < C^* \leq C$ ve C / C^* oranı, yaklaşık çözümün maliyetini bir optimal çözümün maliyetinden daha büyük yapan katsayıyı verir. Bütün çözümlerin pozitif maliyete sahip olduğunu varsaydığımız için bu oranlar her zaman iyi tanımlıdır. Bir yaklaşım algoritmasının yaklaşım oranı asla 1 den küçük değildir; çünkü $C / C^* \leq 1$ ifadesi $C^* / C \geq 1$ anlamına gelir. Dolayısıyla, bir 1-yaklaşım algoritması¹ bir optimal çözüm üretir ve büyük yaklaşım oranlı bir yaklaşım algoritması optimalden çok daha kötü bir çözüm geri döndürebilir.

Birçok problem için küçük sabit yaklaşım oranlı polinom-zamanlı algoritmalara sahibiz, bununla birlikte diğer problemler için en iyi bilinen polinom-zamanlı yaklaşım algoritmaları, n girdi boyutunun fonksiyonları olarak büyüyen yaklaşım oranlarına sahiptir. Böyle bir problemin bir örneği, Kısım 35.3 de sunulan küme-örtüsü problemidir.

Bazı NP-tam problemler, gitgide daha fazla hesaplama zamanı kullanarak giderek daha iyi yaklaşım oranlarına ulaşabilen polinom-zamanlı yaklaşım algoritmalarına izin verirler. Yani, hesaplama zamanını yaklaşımın kalitesi ile takas edebiliriz. Kısım 35.5 de çalışılan altküme-toplamı problemi buna bir örnektir. Bu durum, kendisinin bir ismini hak etmesi için yeterince önemlidir.

Bir optimizasyon problemi için bir **yaklaşım şeması**, sadece problemin bir örneğini değil ayrıca bir $\epsilon > 0$ değerini de girdi olarak alan bir yaklaşım algoritmasıdır öyle ki herhangi belirli bir ϵ için şema bir $(1 + \epsilon)$ -yaklaşım algoritmasıdır. Herhangi belirli bir $\epsilon > 0$ için bir şema girdi örneğinin n boyutuna bağlı polinom zamanda çalışırsa bu yaklaşım şeması bir **polinom-zamanlı yaklaşım şemasıdır** deriz.

Polinom-zamanlı bir yaklaşım şemasının çalışma zamanı, ϵ azalırken çok hızlı artabilir. Örneğin, polinom-zamanlı bir yaklaşım şemasının çalışma zamanı $O(n^{2/\epsilon})$ olabilir. İdeal olarak, ϵ sabit bir çarpan ile azalırsa, istenen yaklaşımı gerçekleştiren çalışma zamanı, sabit bir çarpandan daha fazla şekilde artmamalıdır (ϵ değerinin sabit azalma katsayısıyla aynı olmak zorunda değildir).

Bir yaklaşım şeması var ve bunun çalışma zamanı $1/\epsilon$ ve girdi örneğinin n boyutunda polinomiyal ise bu yaklaşım şeması bir **tam polinom-zamanlı yaklaşım şemasıdır** deriz. Örneğin, şema bir $O((1/\epsilon)^2 n^3)$ çalışma zamanına sahip olabilir. Böyle bir şema ile, ϵ değerindeki herhangi bir sabit-çarpan azalması, çalışma zamanındaki ilişkili bir sabit-çarpan artışıyla birlikte gelir.

¹ Yaklaşım oranı n den bağımsız olduğunda, n ye bağımlı olmadığını belirterek “ p yaklaşım oranı” ve “ p -yaklaşım algoritması” terimlerini kullanırız.

Bölüm taslağı

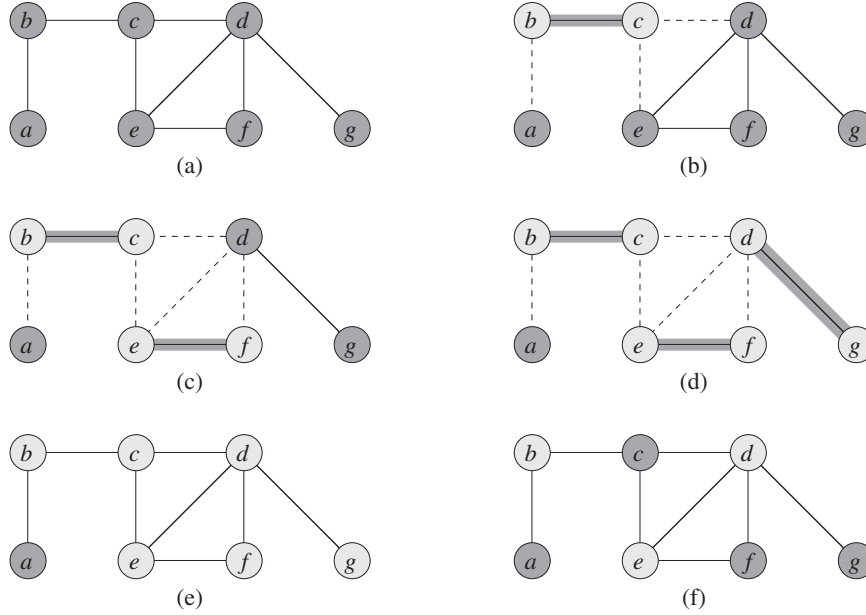
Bu bölümün ilk dört kısmı, NP-tam problemler için polinom-zamanlı yaklaşım şemalarının bazı örneklerini sunar ve beşinci kısım bir tam polinom-zamanlı yaklaşım şeması sunar. Kısım 35.1, tepe-örtüsü probleminin, bir çalışmasıyla başlar, yaklaşım oranı 2 olan bir yaklaşım algoritmasına sahip bir NP-tam minimizasyon problemi. Kısım 35.2, gezgin-satıcı probleminin, maliyet fonksiyonu üçgen eşitsizliğini sağlayan durumu için 2 yaklaşım oranlı bir yaklaşım algoritması sunar. Ayrıca herhangi bir $\rho \geq 1$ için üçgen eşitsizliği olmadan, $P = NP$ olmadıkça bir ρ -yaklaşım algoritmasının var olamayacağını da gösterir. Kısım 35.3 de, maliyeti en kötü durumda optimal maliyetten logaritmik bir çarpan kadar büyük olan bir örtü elde ederek, küme-örtüsü problemi için bir açgözlü yöntemin etkili bir yaklaşım algoritması olarak nasıl kullanıldığını gösteririz. Kısım 35.4 iki yaklaşım algoritması daha sunar. İlk olarak, 3-CNF gerçekleştirilebilirliğin optimizasyon versiyonunu çalışırız ve ümit edilen $8/7$ yaklaşım oranlı bir çözüm üreten basit bir rassal algoritma veririz. Ardından tepe-örtüsü probleminin ağırlıklı bir varyantını inceleriz ve bir 2-yaklaşım algoritması geliştirmek için doğrusal programlamanın nasıl kullanıldığını gösteririz. Son olarak, Kısım 35.5 altküme-toplam problemi için bir tam polinom-zamanlı yaklaşım şeması sunar.

35.1 Tepe-örtüsü problemi

Kısım 34.5.2 tepe-örtüsü problemini tanımlamış ve onun NP-tam olduğunu kanıtlamıştır. Yönsüz bir $G = (V, E)$ çizgesinin bir *tepe örtüsünün* (*vertex cover*) (u, v) , G nin bir ayrıtı ise $u \in V'$ ya da $v \in V'$ (ya da her ikisi) olacak şekilde bir $V' \subseteq V$ altkümesi olduğunu hatırlayınız. Bir tepe örtüsünün boyutu ondaki tepelerin sayısıdır.

Tepe-örtüsü problemi, verilen yönsüz bir çizgede bir minimum boyutlu tepe örtüsü bulmaktır. Böyle bir tepe örtüsünü bir *optimal tepe örtüsü* olarak adlandırırız. Bu problem, bir NP-tam karar probleminin optimizasyon versiyonudur.

Bir G çizgesindeki bir optimal tepe örtüsünün polinom zamanda nasıl bulunduğunu bilmesek bile yakın-optimal olan bir tepe örtüsünü etkin bir biçimde bulabiliriz. Aşağıdaki yaklaşım algoritması yönsüz bir G çizgesini girdi olarak alır ve boyutu bir optimal tepe örtüsünün boyutunun iki katından fazla olmayacağı garanti edilen bir tepe örtüsü geri döndürür.



Şekil 35.1 APPROX-VERTEX-COVER'ın işleyişi. **(a)** 7 tepesi ve 8 ayrıtı olan G girdi çizgesi. **(b)** Kalın olarak gösterilen (b, c) ayrıtı APPROX-VERTEX-COVER tarafından seçilen ilk ayrıttır. Açık gölgelendirilmiş b ve c tepeleri, oluşturulmakta olan tepe örtüsünü içeren C kümesine eklenir. Kesikli çizgi ile gösterilen (a, b) , (c, e) ve (c, d) ayrıtları çıkarılır; çünkü bu ayrıtlar şimdi C deki bir ayrıtı tarafından örtülüdür. **(c)** (e, f) ayrıtı seçilir; e ve f tepeleri C ye eklenir. **(d)** (d, g) ayrıtı seçilir; d ve g tepeleri C ye eklenir. **(e)** APPROX-VERTEX-COVER tarafından üretilen C tepe örtüsü b, c, d, e, f, g tepelerini içerir. **(f)** Bu problem için optimal tepe örtüsü sadece üç tepe içerir: b, d ve e .

APPROX-VERTEX-COVER (G)

```

1   $C = \emptyset$ 
2   $E' = G.E$ 
3  while  $E' \neq \emptyset$ 
4       $(u, v)$ ,  $E'$  kümesinin keyfi ayrıtı olsun
5       $C = C \cup \{u, v\}$ 
6       $E'$  kümesinden  $u$  ya da  $v$  ye bitişik olan bütün ayrıtları çıkar
7  return  $C$ 
```

Şekil 35.1, APPROX-VERTEX-COVER algoritmasının örnek bir çizge üzerinde nasıl çalıştığını gösterir. C değişkeni inşa edilmekte olan tepe örtüsünü içerir. Satır 1, C yi boş küme olarak başlatır. Satır 2, E' kümesini çizgenin $G.E$ ayrıtı kümesinin bir kopyası olarak ayarlar. Satır 3 ile satır 6 arasındaki döngü arka arkaya,

E' kümesinden bir (u, v) ayrıtı seçer, onun u ve v uç noktalarını C ye ekler ve u ya da v tarafından örtülen bütün ayrıtıları E' kümesinden siler. Son olarak, satır 7, C tepe örtüsünü geri döndürür. E' kümesini göstermek için bitişiklik listeleri kullanıldığında, bu algoritmanın çalışma zamanı $O(V + E)$ dir.

Teorem 35.1

APPROX-VERTEX-COVER polinom-zamanlı bir 2-yaklaşım algoritmasıdır.

Kanıt APPROX-VERTEX-COVER algoritmasının polinom zamanda çalıştığını zaten göstermiştik.

APPROX-VERTEX-COVER tarafından döndürülen tepelerin C kümesi bir tepe örtüsüdür; çünkü algoritma $G.E$ deki her ayrıtı C deki bir tepe tarafından örtülene kadar döngüye devam eder.

APPROX-VERTEX-COVER ın bir optimal örtünün boyutunun en fazla iki katı olan bir tepe örtüsü döndürdüğünü görmek için A , APPROX-VERTEX-COVER ın 4. satırının seçtiği ayrıtıların kümesini belirtsin. A daki ayrıtıları örtmek için herhangi bir tepe örtüsü – özellikle, bir C^* optimal örtüsü – A daki her bir ayrıtının en az bir uç noktasını içermelidir. A daki iki ayrıtı aynı uç noktayı paylaşmaz; çünkü bir ayrıtı 4. satırda bir kere seçildiğinde onun uç noktalarına bitişik diğer bütün ayrıtılar 6. satırda E' kümesinden silinir. Dolayısıyla A daki hiçbir iki ayrıtı C^* kümesinden aynı tepeyle örtülmez ve bir optimal tepe örtüsünün boyutu üzerinde

$$|C^*| \geq |A| \quad (35.2)$$

alt sınırına sahip oluruz. Satır 4 ün her bir yürütmesi uç noktalarının hiçbirisi zaten C de olmayan bir ayrıtı seçer ve bu döndürülen tepe örtüsünün boyutu üzerinde bir üst sınır (aslında bir tam üst sınır) verir:

$$|C| = 2 |A|. \quad (35.3)$$

(35.2) ve (35.3) denklemlerini birleştirerek

$$\begin{aligned} |C| &= 2 |A| \\ &\leq 2 |C^*| \end{aligned}$$

ifadesini elde ederiz ve böylece teorem kanıtlanır. ■

Bu kanıtı derinlemesine düşünelim. Başlangıçta, bir optimal tepe örtüsünün boyutunu bilmiyorken bile APPROX-VERTEX-COVER tarafından döndürülen tepe örtüsünün boyutunun bir optimal tepe örtüsünün boyutunun en fazla iki katı olduğunu nasıl kanıtlayabildiğimiz merak ediyor olabilirsiniz. Bir optimal tepe örtüsünün kesin boyutunu bilmek gereksinimi yerine boyut üzerindeki bir alt sınıra güveniriz. Alıştırma 35.1-2 nin göstermenizi istediği gibi, APPROX-VERTEX-COVER algoritmasının 4. satırının seçtiği ayrıtıların A kümesi aslında G çizgesinde bir maksimal eşlemedir. (Bir **maksimal eşleme**, diğer hiçbir eşlemenin uygun bir

altkümesi olmayan bir eşlemedir.) Bir maksimal eşlemenin boyutu, Teorem 35.1 in kanıtında ele aldığımız gibi, bir optimal tepe örtüsünün boyutu üzerindeki bir alt sınırdır. Algoritma, boyutu en fazla A maksimal eşlemesinin boyutunun iki katı olan bir tepe örtüsü geri döndürür. Döndürülen çözümün boyutunu alt sınır ile ilişkilendirerek yaklaşım oranımızı elde ederiz. Bu metodolojiyi daha sonraki kısımlarda da kullanacağız.

Alıştırmalar

35.1-1

APPROX-VERTEX-COVER in her zaman altoptimal bir çözüm verdiği bir çizge örneği veriniz.

35.1-2

APPROX-VERTEX-COVER in 4. satırında seçilen ayrıtların kümesinin G çizgesinde bir maksimal eşleme oluşturduğunu kanıtlayınız.

35.1-3 *

Profesör Bündchen, tepe-örtüsü problemini çözmek için aşağıdaki sezgiseli önerir. Arka arkaya maksimum dereceli bir tepeyi seç ve onun bütün bitişik ayrıtlarını çıkar. Profesörün sezgiselinin 2 yaklaşım oranına sahip olmadığını göstermek için bir örnek veriniz. (*İpucu:* Solda aynı dereceye sahip tepeler ve sağda farklı derecelere sahip tepeler ile iki parçalı bir çizgeyi deneyiniz.)

35.1-4

Bir ağaç için bir optimal tepe örtüsünü doğrusal zamanda bulan etkili bir greedy algoritması veriniz.

35.1-5

Teorem 34.12 nin kanıtından biliyoruz ki, tepe-örtüsü problemi ve NP-tam klik problemi, bir optimal tepe örtüsünün, tümleyen çizgedeki bir maksimum-boyutlu kliğin tümleyeni olması bakımından tamamlayıcıdırlar. Bu ilişki, klik problemi için sabit yaklaşım oranına sahip polinom-zamanlı bir yaklaşım algoritması olduğu anlamına gelir mi? Yanıtınızı doğrulayınız.

35.2 Gezgin-satıcı problemi

Kısım 34.5.4 de tanıtılan gezgin-satıcı probleminde, her bir $(u, v) \in E$ ayrıtı ile ilişkili bir negatif olmayan tam sayı $c(u, v)$ maliyetine sahip yönsüz bir $G = (V, E)$ tam çizgesi verilir ve G nin minimum maliyetli bir hamilton çevrimini bulmamız istenir. Gösterim biçimimizin bir genişletmesi olarak, $c(A)$, $A \subseteq E$ altkümesindeki ayrıtların toplam maliyetini belirtsin:

$$c(A) = \sum_{(u,v) \in A} c(u, v) .$$

Birçok gerçekçi durumda, bir u konumundan bir w konumuna gitmek için en az maliyetli yol, hiçbir ara adım olmaksızın doğrudan gitmektir. Bir başka deyişle, bir ara durağı çıkarmak maliyeti asla artırmaz. Eğer tüm $u, v, w \in V$ tepeleri için $c(u, w) \leq c(u, v) + c(v, w)$.

ise, c maliyet fonksiyonu **üçgen eşitsizliğini** sağlar diyerek bu kavramı biçimlendiririz.

Üçgen eşitsizliği sanki doğal olarak geçerliymiş gibi görünür ve birçok uygulamada otomatik olarak sağlanır. Örneğin, çizgenin tepeleri düzlemde noktalarsa ve iki tepe arasındaki yolculuğun maliyeti onlar arasındaki sıradan öklid uzaklığı ise üçgen eşitsizliği sağlanır. Ayrıca, öklid uzaklığından başka diğer birçok maliyet fonksiyonu da üçgen eşitsizliğini sağlar.

Alıştırma 35.2-2 nin gösterdiği gibi, maliyet fonksiyonunun üçgen eşitsizliğini sağlamasını istesek bile gezgin-satıcı problemi yine de NP-tamdir. Dolayısıyla, bu problemi kesin çözmek için polinom-zamanlı bir algoritma bulmayı beklememeliyiz. Onun yerine iyi yaklaşım algoritmaları ararız.

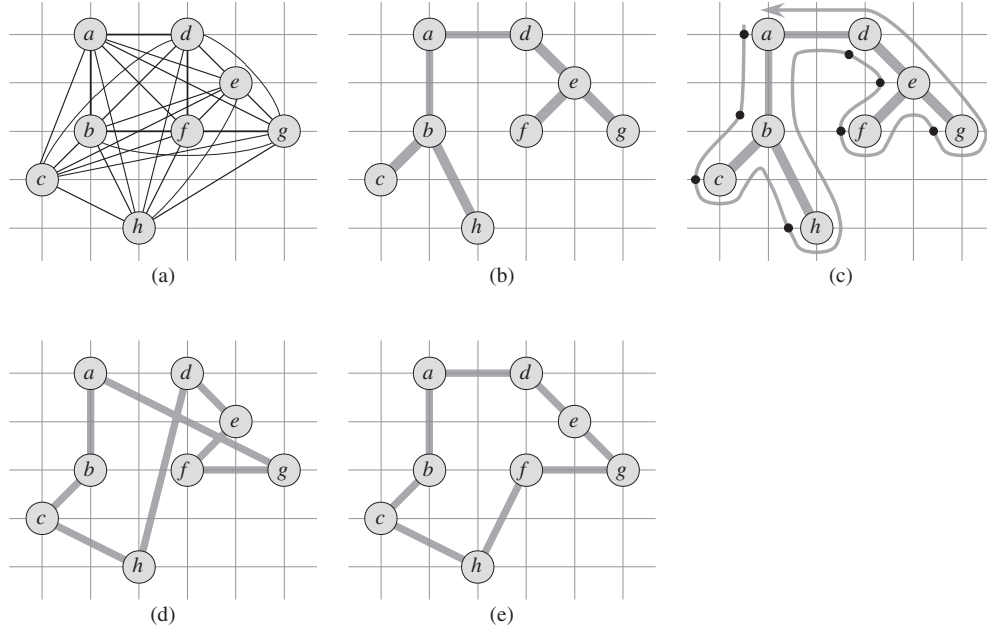
Kısım 35.2.1 de üçgen eşitsizliği ile gezgin-satıcı problemi için bir 2-yaklaşım algoritmasını inceleyeceğiz. Kısım 35.2.2 de, üçgen eşitsizliği olmadan, $P = NP$ olmadıkça sabit bir yaklaşım oranına sahip polinom-zamanlı bir yaklaşım algoritmasının var olmadığını göstereceğiz.

35.2.1 Üçgen eşitsizliği ile gezgin-satıcı problemi

Bir önceki bölümün metodolojisini uygulayarak, öncelikle ağırlıklı bir optimal gezgin-satıcı turunun uzunluğu üzerinde bir alt sınır veren bir yapıyı—bir minimum kapsayan ağaç—hesaplayacağız. Ardından, maliyet fonksiyonu üçgen eşitsizliğini sağladığı sürece, maliyeti, minimum kapsayan ağacın ağırlığının maliyetinin iki katından daha büyük olmayan bir tur oluşturmak için minimum kapsayan ağacı kullanacağız. Aşağıdaki algoritma, Kısım 23.2 deki minimum-kapsayan-ağaç algoritması MST-PRIM i bir prosedür olarak çağırarak bu yaklaşımı uygular. G parametresi yönsüz bir tam çizgedir ve c maliyet fonksiyonu üçgen eşitsizliğini sağlar.

APPROX-TSP-TOUR (G, c)

- 1 bir $r \in G.V$ tepesini bir “kök” tepe olarak seç
- 2 MST-PRIM (G, c, r) yi kullanarak G için r kökünden bir T spanning ağaç hesapla
- 3 H, T nin önsıra ağaç yürüyüşünde ilk ne zaman ziyaret edildiklerine göre sıralı bir tepeler listesi olsun
- 4 **return** H hamilton çevrimi



Şekil 35.2 APPROX-TSP-TOUR işleyişi. **(a)** Bir yönsüz tam çizge. Tepeler tam sayı ızgara doğrularının kesişiminin üzerine düşer. Örneğin, f , h den bir birim sağda ve 2 birim yukarıdadır. İki nokta arasındaki maliyet fonksiyonu sıradan öklid uzaklığıdır. **(b)** Tam çizgenin, MST-PRIM tarafından hesaplanmış bir T minimum kapsayan ağacı. a tepesi kök tepesidir. Sadece minimum kapsayan ağaçtaki ayrıtlar gösterilmiştir. Tepeler, MST-PRIM tarafından esas ağaca alfabetik sırada eklendikleri şekilde etiketlenirler. **(c)** a dan başlayan bir T yürüyüşü. Ağacın bir tam yürüyüşü bir tepeyi sadece ilk karşılaşıldığında listeler; ve her bir tepenin yanındaki noktayla gösterildiği gibi a, b, c, h, d, e, f, g sıralamasını verir. **(d)** Tepeleri önsıra yürüyüşü tarafından verilen sırada ziyaret ederek elde edilen bir tur; bu tur APPROX-TSP-TOUR tarafından geri döndürülen H turudur. Toplam maliyeti yaklaşık olarak 19.074 dür. **(e)** Orijinal tam çizge için bir H^* optimal turu. Toplam maliyeti yaklaşık olarak 14.715 dir.

Kısım 12.1 den bir önsıra ağaç yürüyüşünün, bir tepeyi çocuklarından herhangisini ziyaret etmeden önce ilk karşılaşıldığında listelediğini ve özyinelemeli olarak ağaçtaki her tepeyi ziyaret ettiğini hatırlayınız.

Şekil 35.2 APPROX-TSP-TOUR un işleyişini örneklerle açıklar. Şeklin (a) kısmı yönsüz bir tam çizmeyi gösterir ve (b) kısmı MST-PRIM tarafından a kök tepesinden büyüyen T minimum kapsayan ağacı gösterir. (c) kısmı, T nin bir önsıra yürüyüşünün tepeleri nasıl ziyaret ettiğini gösterir ve (d) kısmı APPROX-TSP-TOUR tarafından geri döndürülen ilişkili turu sergiler. (e) kısmı yaklaşık %23 daha kısa olan bir optimal turu gösterir.

Alıştırma 23.2-2 den, MST-PRIM in basit bir yürütmesiyle bile APPROX-TSP-TOUR un çalışma zamanı $\Theta(V^2)$ dir. Şimdi, gezgin-satıcı probleminin bir örneği için maliyet fonksiyonu üçgen eşitsizliğini sağlarsa, APPROX-TSP-TOUR un maliyeti bir optimal turun maliyetinin iki katından daha fazla olmayan bir tur geri döndürdüğünü göstereceğiz.

Teorem 35.2

APPROX-TSP-TOUR, üçgen eşitsizlikli gezgin-satıcı problemi için bir polinom-zamanlı 2-yaklaşım algoritmasıdır.

Kanıt APPROX-TSP-TOUR un polinom zamanda çalıştığını zaten göstermiştik.

H^* verilen tepeler kümesi için bir optimal turu belirtsin. Bir turdan herhangi bir ayrıtı silerek bir kapsayan ağaç elde ederiz ve her bir ayrıtı maliyeti negatif olmayan bir sayıdır. Dolayısıyla, APPROX-TSP-TOUR un 2. adımında hesaplanan minimum T kapsayan ağacının ağırlığı, bir optimal turun maliyeti üzerinde bir alt sınır sağlar:

$$c(T) \leq c(H^*). \quad (35.4)$$

T nin bir **tam yürüyüşü (full walk)** tepeleri ilk ziyaret edildiğinde ve ayrıca bir altağaca yapılan bir ziyaretin ardından o tepeye her geri dönüldüğünde listeler. Bu tam yürüyüşe W diyelim. Örneğimizin tam yürüyüşü,

$a, b, c, b, h, b, a, d, e, f, e, g, e, d, a$

sıralamasını verir. Tam yürüyüş T nin her ayrıtından tam olarak iki kere geçtiği için aşağıdaki denkliği elde ederiz (ayrıntıların çoklu kümeleriyle başa çıkmak için c maliyetinin tanımını doğal şekilde genişleterek)

$$c(W) = 2 c(T). \quad (35.5)$$

(35.4) eşitsizliği ve (35.5) denklemi

$$c(W) \leq 2 c(H^*) \quad (35.6)$$

anlamına gelir ve dolayısıyla W nin maliyeti bir optimal turun maliyetinin 2 katını aşmaz.

Ne yazık ki W tam yürüyüşü genelde bir tur değildir; çünkü bazı tepeleri birden fazla defa ziyaret eder. Bununla birlikte üçgen eşitsizliğinden, herhangi bir tepeye yapılan bir ziyareti W den silebiliriz ve maliyet artmaz. (Eğer u ve w ziyaretleri arasındaki bir v tepesini W den silersek, sonuçta oluşan sıralama u dan w ye doğrudan gidildiğini belirtir.) Bu işlemi arka arkaya uygulayarak, herbir tepeye yapılan ilk ziyaretler dışındaki tüm ziyaretleri W den çıkarabiliriz. Bu işlem örneğimizde

a, b, c, h, d, e, f, g

sıralamasını bırakır.

Bu sıralama, T ağacının bir önsıra yürüyüşü tarafından elde edilenle aynıdır. H , bu önsıra yürüyüşe karşılık gelen döngü olsun. Bu bir hamilton çevrimidir; çünkü

her tepe tam olarak bir kez ziyaret edilir ve aslında APPROX-TSP-TOUR tarafından hesaplanan döngüdür. H , W tam yürüyüşünden tepeleri silerek elde edildiğinden

$$c(H) \leq c(W) \quad (35.7)$$

olur. (35.6) ve (35.7) eşitsizliklerini birleştirirsek $c(H) \leq 2 c(H^*)$ olur ve kanıt tamamlanır. ■

Teorem 35.2 tarafından sağlanan iyi yaklaşım oranına rağmen APPROX-TSP-TOUR, bu problem için genelde en kullanışlı seçim değildir. Uygulamada genel anlamda daha iyi performans sergileyen başka yaklaşım algoritmaları vardır (Bu bölümün sonundaki referanslara bakınız).

35.2.2 Genel gezgin-satıcı problemi

c maliyet fonksiyonunun üçgen eşitsizliğini sağladığı varsayımını bırakırsak, $P = NP$ olmadıkça iyi yaklaşık turları polinom zamanda bulamayız.

Teorem 35.3

$P \neq NP$ ise, herhangi bir $\rho \geq 1$ için genel gezgin-satıcı probleminin ρ yaklaşım oranlı hiçbir polinom-zamanlı yaklaşım algoritması yoktur.

Kanıt Olmayana ergi yöntemiyle kanıtlarız. Varsayalım ki bir $\rho \geq 1$ için ρ yaklaşım oranlı bir A polinom-zamanlı yaklaşım algoritması olsun. Genelliği kaybetmeden, gerekirse yuvarlayarak, ρ nin bir tam sayı olduğunu varsayalım. Şimdi (Kısım 34.2 de tanımlanan) hamilton-çevrim probleminin örneklerini polinom zamanda çözmek için A nın nasıl kullanıldığını göstereceğiz. Teorem 34.13, hamilton-çevrim probleminin NP-tam olduğunu söylediği için Teorem 34.4, bu problemi polinom zamanda çözebilirsek $P = NP$ olduğunu belirtir.

$G = (V, E)$ hamilton çevrim probleminin bir örneği olsun. Varsayımında bulunan A yaklaşım algoritmasından yararlanarak, G nin bir hamilton çevrim içerip içermediğini etkili olarak belirlemek isteriz. G yi gezgin-satıcı probleminin bir örneğine aşağıdaki gibi dönüştürürüz. $G' = (V, E')$, V üzerinde bir tam çizge olsun; yani

$$E' = \{(u, v) : u, v \in V \text{ ve } u \neq v\}.$$

E' kümesindeki her bir ayrıta bir tam sayı maliyetini aşağıdaki gibi ata:

$$c(u, v) = \begin{cases} 1, & (u, v) \in E \text{ ise,} \\ \rho |V| + 1, & \text{aksi halde.} \end{cases}$$

G' ve c nin gösterimlerini G nin bir gösteriminden $|V|$ ve $|E|$ nin polinom zamanında yaratabiliriz.

Şimdi (G', c) gezgin-satıcı problemini düşünelim. Eğer orijinal G çizgesi bir H hamilton çevrimine sahipse, c maliyet fonksiyonu H nin herbir ayrıtına bir 1 maliyeti atar ve böylece (G', c) , $|V|$ maliyetli bir tur içerir. Diğer taraftan, eğer G bir hamilton çevrimi içermezse, G' çizgesinin herhangi bir turu E de olmayan bir ayrıt kullanmalıdır. Ancak, E de olmayan bir ayrıt kullanan herhangi bir turun maliyeti en az

$$\begin{aligned} (\rho |V| + 1) + (|V| - 1) &= \rho |V| + |V| \\ &> \rho |V|. \end{aligned}$$

G de olmayan ayrıtlar oldukça maliyetli olduğundan, G de bir hamilton çevrimi olan bir turun maliyeti (maliyet $|V|$) ile herhangi bir başka turun maliyeti (maliyet en az $\rho |V| + |V|$) arasında en az $\rho |V|$ kadar bir fark bulunur. Dolayısıyla, G de hamilton çevrimi olmayan bir turun maliyeti, G de hamilton çevrimi olan bir turun maliyetinden en az $\rho + 1$ in bir katı kadar büyüktür.

Şimdi A yaklaşım algoritmasını, (G', c) gezgin-satıcı problemine uyguladığımızı varsayalım. A nın, maliyeti bir optimal turun maliyetinin ρ katından daha büyük olmayan bir tur geri döndürdüğü garanti edildiğinden, eğer G bir hamilton çevrimi içerirse A onu geri döndürmelidir. Eğer G hiç hamilton çevrimi içermiyorsa, A algoritması $\rho |V|$ den daha maliyetli bir tur geri döndürür. Dolayısıyla, hamilton-çevrimi problemini polinom zamanda çözmek için A algoritmasını kullanabiliriz. ■

Teorem 35.3 ün kanıtı, bir probleme çok iyi yaklaşamadığımızın kanıtı için genel bir tekniğin bir örneği işlevini görür. Bir NP-zor X problemi verildiğinde bir Y minimizasyon problemini polinom zamanda üretebildiğimizi varsayalım. Öyle ki X in “evet” örnekleri Y nin en fazla k (herhangi bir k için) değerli örneklerine karşılık gelsin; ancak X in “hayır” örnekleri Y nin ρk den daha büyük değerli örneklerine karşılık gelsin. Bu durumda, $P = NP$ olmadığı sürece, Y problemi için hiçbir polinom-zamanlı ρ -yaklaşım algoritması bulunmadığını gösterdik.

Alıştırmalar

35.2-1

En az 3 tepeli yönsüz bir $G = (V, E)$ tam çizgesinin üçgen eşitsizliğini sağlayan bir c maliyet fonksiyonuna sahip olduğunu varsayalım. Her $u, v \in V$ için $c(u, v) \geq 0$ olduğunu kanıtlayınız.

35.2-2

Gezgin-satıcı probleminin bir örneğinin, maliyet fonksiyonu üçgen eşitsizliğini sağlayan başka bir örneğine polinom zamanda nasıl dönüştürüldüğünü gösteriniz. İki örnek, optimal turların aynı kümesine sahip olmalıdır. $P \neq NP$ olduğunu varsayarak, böyle bir polinom-zamanlı dönüşümün neden Teorem 35.3 ile çelişmediğini açıklayınız.

35.2-3

Maliyet fonksiyonu üçgen eşitsizliğini sağlayan yaklaşık bir gezgin-satıcı turu inşa etmek için aşağıdaki *en yakın-nokta sezgiselini* düşünün. Keyfi seçilmiş tek bir tepeden oluşan sıradan bir çevrim ile başla. Her bir adımda, çevrimde olmayan ama çevrimdeki herhangi bir tepeye uzaklığı minimum olan u tepesini belirle. Çevrimdeki u ya en yakın tepenin v tepesi olduğunu farzet. u tepesini v nin hemen arkasına ekleyerek çevrimi genişlet. Tüm tepeler çevrimde olana dek tekrar et. Bu sezgiselin, toplam maliyeti bir optimal turun maliyetinin iki katından daha fazla olmayan bir tur geri döndürdüğünü kanıtlayınız.

35.2-4

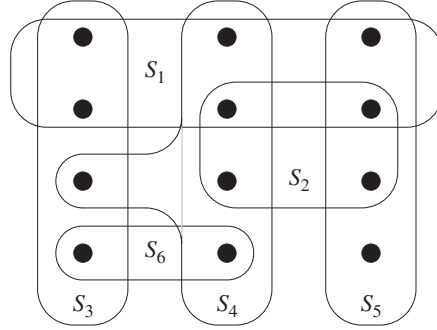
Darboğazlı gezgin-satıcı probleminde, çevrimdeki en maliyetli ayrıtın maliyetini en aza indirgeyen hamilton çevrimini bulmak isteriz. Maliyet fonksiyonunun üçgen eşitsizliğini sağladığını farzederek, bu problem için 3 yaklaşım oranına sahip polinom-zamanlı bir yaklaşım algoritmasının var olduğunu gösteriniz (*İpucu*: Problem 23-3 de ele alındığı gibi, ağacın bir tam yürüyüşünü alarak ve düğümleri atlayarak; ancak ikiden fazla ardışık ara düğümü atlamaksızın, bir darboğazlı kapsayan ağaçtaki tüm düğümleri tam olarak bir kez ziyaret edebildiğimizi özyinelemeli olarak gösteriniz. Bir darboğazlı kapsayan ağaçtaki en maliyetli ayrıtın maliyetinin, en fazla darboğazlı bir hamilton çevrimindeki en maliyetli ayrıtın maliyeti kadar olduğunu gösteriniz).

35.2-5

Gezgin-satıcı probleminin bir örneği için tepelerin düzlemde noktalar olduğunu ve $c(u, v)$ maliyetinin u ve v arasındaki öklid uzaklığı olduğunu varsayalım. Bir optimal turun asla kendisiyle çakışmadığını gösteriniz.

35.3 Küme-örtüsü problemi

Küme-örtüsü problemi, kaynaklarının paylaştırılmasını gerektiren birçok problem modelleyen bir optimizasyon problemidir. İlgili karar problemi, NP-tam tepe-örtüsü problemini genelleştirir ve dolayısıyla NP-zordur. Bununla birlikte, tepe-örtüsü probleminin üstesinden gelmek için geliştirilen yaklaşım algoritması buraya uymaz ve dolayısıyla diğer yaklaşımları denememiz gerekir. Logaritmik bir yaklaşım oranına sahip basit bir açgözlü sezgiselini inceleyeceğiz. Yani, örneğin boyutu büyüdükçe yaklaşık çözümün boyutu, bir optimal çözümün boyutuna göre artabilir. Bununla beraber, logaritma fonksiyonu bir hayli yavaş büyüdüğü için bu yaklaşım algoritması yine de yararlı sonuçlar verebilir.



Şekil 35.3 X , 12 siyah noktadan oluşan küme ve $\mathcal{F} = \{S_1, S_2, S_3, S_4, S_5, S_6\}$ olmak üzere küme-örtüsü probleminin bir $(X, \mathcal{F})$ örneği. Bir minimum-boyutlu küme örtüsü, boyutu 3 olan $\mathcal{C} = \{S_3, S_4, S_5\}$ kümesidir. Açgözlü algoritması, S_1, S_4, S_5 ve S_3 ya da S_1, S_4, S_5 ve S_6 kümelerini sırasıyla seçerek 4 boyutlu bir örtü üretir.

Küme-örtüsü probleminin bir $(X, \mathcal{F})$ örneği, bir X sonlu kümesi ve X in altkümelerinin bir $\mathcal{F}$ ailesinden oluşur, öyle ki X in her elemanı $\mathcal{F}$ deki altkümelerden en az birine aittir:

$$X = \bigcup_{S \in \mathcal{F}} S.$$

Bir $S \in \mathcal{F}$ altkümesi kendi elemanlarını **örter** deriz. Problem, elemanları X in tamamını örten minimum boyutlu bir $\mathcal{C} \subseteq \mathcal{F}$ altkümesi bulmaktır:

$$X = \bigcup_{S \in \mathcal{C}} S. \quad (35.8)$$

(35.8) denklemini sağlayan herhangi bir $\mathcal{C}$ altkümesi X i **örter** deriz. Şekil 35.3 küme-örtüsü problemini örnekler. $\mathcal{C}$ nin boyutu, kapsadığı kümelerdeki bireysel elemanların sayısından ziyade bu kümelerin sayısıdır; çünkü X i örten her $\mathcal{C}$ altkümesi $|X|$ tane bireysel elemanın tamamını içermelidir. Şekil 35.3 de minimum küme örtüsünün boyutu 3 dür.

Küme-örtüsü problemi, yaygın olarak ortaya çıkan birçok kombinatoriyal problemi özetler. Basit bir örnek olarak, X in bir problemi çözmek için gerekli yeteneklerin bir kümesini belirttiğini ve problem üzerinde çalışmaya müsait insanların bir kümesinin verildiğini farzedelim. Mümkün olduğunca az sayıda kişi içeren bir komite oluşturmak isteriz, öyle ki X deki gerekli her yetenek için komitenin en az bir üyesi o yeteneğe sahip olmalıdır. Küme-örtüsü probleminin karar versiyonunda, k problem örneğinde belirlenen ilave bir parametre olmak üzere, en fazla k boyutlu bir örtünün var olup olmadığını sorarız. Alıştırma 35.2-2 nin göstermenizi istediği üzere, problemin karar versiyonu NP-tamdır.

Bir açgözlü yaklaşım algoritması

Açgözlü yöntemi, her bir adımda, kalan örtülmemiş elemanların en çoğunu örten S kümesini seçerek çalışır.

GREEDY-SET-COVER ($X, \mathcal{F}$)

```

1   $U = X$ 
2   $S \in \mathcal{F}$ 
3  while  $U \neq \emptyset$ 
4       $|S \cap U|$  değerini maksimize eden bir  $S \in \mathcal{F}$  kümesi seç
5       $U = U - S$ 
6       $\mathcal{C} = \mathcal{C} \cup \{S\}$ 
7  return  $\mathcal{C}$  yi
```

Şekil 35.3 ün örneğinde GREEDY-SET-COVER $\mathcal{C}$ ye S_3 veya S_6 tarafından takip edilen S_1, S_4 ve S_5 kümelerini sırasıyla ekler.

Algoritma şu şekilde çalışır. U kümesi, her bir adımda, kalan örtülmemiş elemanların kümesini içerir. $\mathcal{C}$ kümesi oluşturulmakta olan örtüyü kapsar. Algoritmanın 4. satırı, mümkün olduğunca çok sayıda elemanı örten bir S altkümünün seçildiği (eşitlikleri gelişigüzel bozarak) açgözlü karar-verme adımıdır. S seçildikten sonra, 5. satır onun elemanlarını U dan çıkarır ve 6. satır S yi $\mathcal{C}$ nin içine yerleştirir. Algoritma sonlandırıldığında $\mathcal{C}$ kümesi, $\mathcal{F}$ nin X i örten bir alt ailesini içerir.

GREEDY-SET-COVER algoritmasının $|X|$ ve $|\mathcal{F}|$ nin polinom zamanında çalıştığını kolaylıkla gösterebiliriz. Döngünün 3-6 satırları üzerindeki iterasyon sayısı üstten $\min(|X|, |\mathcal{F}|)$ ile sınırlı olduğundan ve döngü gövdesinin $O(|X||\mathcal{F}|)$ zamanında çalıştığını söyleyebildiğimizden basit bir uygulama $O(|X||\mathcal{F}| \min(|X|, |\mathcal{F}|))$ zamanında çalışır. Alıştırma 35.3-3 doğrusal-zamanlı bir algoritma ister.

Analiz

Şimdi açgözlü algoritmasının, optimal bir küme örtüsünden çok büyük olmayan bir küme örtüsü geri döndürdüğünü gösterelim. Uygunluk için bu bölümde, d . harmonik $H_d = \sum_{i=1}^d 1/i$ sayısını (Kısım A.1 e bakınız) $H(d)$ ile belirtiriz. Bir sınır koşulu olarak $H(0) = 0$ olarak tanımlarız.

Teorem 35.4

$$\rho(n) = H(\max \{|S| : S \in \mathcal{F}\})$$

olmak üzere, GREEDY-SET-COVER polinom-zamanlı bir $\rho(n)$ -yaklaşım algoritmasıdır.

Kanıt GREEDY-SET-COVER algoritmasının polinom zamanda çalıştığını zaten göstermiştik.

GREEDY-SET-COVER ın bir $\rho(n)$ -yaklaşım algoritması olduğunu göstermek için algoritma tarafından seçilen her bir kümeye bir 1 maliyeti atarız, bu maliyeti ilk kez örtülen elemanlar üzerine paylaştırırız ve ardından bir $\mathcal{C}^*$ optimal küme örtüsünün boyutu ile algoritma tarafından geri döndürülen $\mathcal{C}$ küme örtüsünün boyutu arasındaki ilişkiyi elde etmek için bu maliyetleri kullanırız. S_i , GREEDY-SET-COVER tarafından seçilen i . altkümeyle belirlensin; algoritma S_i yi $\mathcal{C}$ ye eklediğinde bir 1 maliyetine girer. Bu S_i yi seçmenin maliyetini S_i tarafından ilk defa örtülen elemanlar arasında eşit olarak dağıtırız. c_x , her bir $x \in X$ için x elemanına ayrılan maliyeti belirlensin. Her bir elemene bir maliyet sadece bir kere, ilk defa örtüldüğünde atanır. Eğer x , S_i tarafından ilk defa örtülürse

$$c_x = \frac{1}{|S_i - (S_1 \cup S_2 \cup \dots \cup S_{i-1})|} \text{ olur.}$$

Algoritmanın her bir adımı maliyetin 1 birim maliyet atar ve böylece

$$|\mathcal{C}| = \sum_{x \in X} c_x. \quad (35.9)$$

Her bir $x \in X$ elemanı, $\mathcal{C}^*$ optimal küme örtüsündeki en az bir kümede bulunur ve bu nedenle

$$\sum_{S \in \mathcal{C}^*} \sum_{x \in S} c_x \geq \sum_{x \in X} c_x. \quad (35.10)$$

(35.9) denklemi ve (35.10) eşitsizliğini birleştirerek

$$|\mathcal{C}| \leq \sum_{S \in \mathcal{C}^*} \sum_{x \in S} c_x. \quad (35.11)$$

Kanıtın geri kalanı, kısaca kanıtlayacağımız aşağıdaki kilit eşitsizliğe dayanır. $\mathcal{F}$ ailesine ait herhangi bir S kümesi için

$$\sum_{x \in S} c_x \leq H(|S|) \quad (35.12)$$

dir.

(35.11) ve (35.12) eşitsizliklerinden

$$\begin{aligned} |\mathcal{C}| &\leq \sum_{S \in \mathcal{C}^*} H(|S|) \\ &\leq |\mathcal{C}^*| \cdot H(\max \{|S| : S \in \mathcal{F}\}), \end{aligned}$$

olur ve teoremin kanıtı tamamlanır.

Geriye kalan iş (35.12) eşitsizliğini kanıtlamaktır. Herhangi bir $S \in \mathcal{F}$ kümesini ve herhangi bir $i = 1, 2, \dots, |\mathcal{C}|$ düşünelim ve

$u_i = |S - (S_1 \cup S_2 \cup \dots \cup S_i)|$, algoritma $S_1, S_2, \dots, S_i$ kümelerini seçtikten sonra örtülmemiş olarak kalan S deki elemanların sayısı olsun. S nin tamamı başlangıçta örtülmemiş olan eleman-

larının sayısı olarak $u_0 = |S|$ değerini tanımlarız. $u_k = 0$ olacak şekilde en küçük indis k olsun, böylece S deki her eleman $S_1, S_2, \dots, S_k$ kümelerinden en az biri tarafından örtülür ve S deki herhangi bir eleman $S_1 \cup S_2 \cup \dots \cup S_{k-1}$ tarafından örtülmez. O halde, $u_{i-1} \geq u_i$ dir ve S nin $u_{i-1} - u_i$ tane elemanı, $i = 1, 2, \dots, k$ için S_i tarafından ilk defa örtülür. Dolayısıyla,

$$\sum_{x \in S} c_x = \sum_{i=1}^k (u_{i-1} - u_i) \cdot \frac{1}{|S_i - (S_1 \cup S_2 \cup \dots \cup S_{i-1})|}.$$

Dikkat edilirse,

$$\begin{aligned} |S_i - (S_1 \cup S_2 \cup \dots \cup S_{i-1})| &\geq |S - (S_1 \cup S_2 \cup \dots \cup S_{i-1})| \\ &= u_{i-1}, \end{aligned}$$

olur; çünkü S_i nin açgözlü seçimi S nin S_i nin yaptığından daha fazla yeni elemanı örtemediğini garantiler (aksi halde, algoritma S_i yerine S yi seçmiş olurdu). Dolayısıyla şu ifadeyi elde ederiz

$$\sum_{x \in S} c_x \leq \sum_{i=1}^k (u_{i-1} - u_i) \cdot \frac{1}{u_{i-1}}.$$

Şimdi bu niceliği aşağıdaki gibi sınırlandırırız:

$$\begin{aligned} \sum_{x \in S} c_x &\leq \sum_{i=1}^k (u_{i-1} - u_i) \cdot \frac{1}{u_{i-1}} \\ &= \sum_{i=1}^k \sum_{j=u_i+1}^{u_{i-1}} \frac{1}{u_{i-1}} \\ &\leq \sum_{i=1}^k \sum_{j=u_i+1}^{u_{i-1}} \frac{1}{j} \quad (\text{çünkü } j \leq u_{i-1}) \\ &= \sum_{i=1}^k \left(\sum_{j=1}^{u_{i-1}} \frac{1}{j} - \sum_{j=1}^{u_i} \frac{1}{j} \right) \\ &= \sum_{i=1}^k (H(u_{i-1}) - H(u_i)) \\ &= H(u_0) - H(u_k) \quad (\text{çünkü toplam iç içe geçip kısalır}) \\ &= H(u_0) - H(0) \\ &= H(u_0) \quad (\text{çünkü } H(0) = 0) \\ &= H(|S|) \end{aligned}$$

ve böylece (35.12) eşitsizliğinin kanıtı tamamlanır. ■

Sonuç 35.5

GREEDY-SET-COVER polinom-zamanlı bir $(\ln |X| + 1)$ -yaklaşım algoritmasıdır.

Kanıt (A.14) eşitsizliğini ve Teorem 35.4 ü kullanınız. ■

Bazı uygulamalarda $\max \{|S| : S \in \mathcal{F}\}$ küçük bir sabit sayıdır ve dolayısıyla GREEDY-SET-COVER tarafından döndürülen çözüm optimalden en fazla küçük bir katsayı kadar daha büyüktür. Böyle bir uygulama, bu sezgisel tepelerinin derecesi en fazla 3 olan bir çizge için yaklaşık bir tepe örtüsü bulurken ortaya çıkar. Bu durumda, GREEDY-SET-COVER tarafından bulunan çözüm bir optimal çözümün $H(3) = 11/6$ katından daha büyük değildir ki bu da APPROX-VERTEX-COVER algoritmasınınkinden kısmen daha iyi bir performans garantisidir.

Alıştırmalar**35.3-1**

Şu kelimelerin herbirini harflerin bir kümesi olarak düşünelim: {arid, dash, drain, heard, lost, nose, shun, slate, snare, thread}. Eşitlikleri, sözlükte ilk gelen kelimeyi seçerek bozduğumuzda, GREEDY-SET-COVER ın algoritmasınınkinden hangi küme örtüsünü ürettiğini gösteriniz.

35.3-2

Küme-örtüsü probleminin karar versiyonunun NP-tam olduğunu tepe-örtüsü probleminden indirgeme yaparak gösteriniz.

35.3-3

GREEDY-SET-COVER ın, çalışma zamanı $O\left(\sum_{S \in \mathcal{F}} |S|\right)$ olacak şekilde nasıl uygulandığını gösteriniz.

35.3-4

Teorem 35.4 ün aşağıdaki daha zayıf biçiminin bariz bir şekilde doğru olduğunu gösteriniz :

$$|\mathcal{C}| \leq |\mathcal{C}^*| \max \{|S| : S \in \mathcal{F}\} .$$

35.3-5

GREEDY-SET-COVER 4. satırdaki eşitlik durumlarını nasıl bozduğumuza bağlı olarak, farklı çözümlerin bir sayısını geri döndürebilir. Küme-örtüsü probleminin bir n -elemanlı örneğini geri döndüren bir BAD-SET-COVER-INSTANCE (n) prosedürü veriniz, öyle ki GREEDY-SET-COVER, 4. satırdaki eşitlik durumlarını nasıl bozduğumuza bağlı olarak, bu örnek için farklı çözümlerin n de üssel olan bir sayısını geri döndürebilir.

35.4 Rassal ve doğrusal programlama

Bu bölümde, yaklaşım algoritmaları tasarlamak için iki kullanışlı tekniği çalışacağız: rassallaştırma ve doğrusal programlama. 3-CNF gerçekleştirilebilirliğin bir optimizasyon versiyonu için basit bir rassal algoritma vereceğiz ve ardından tepe-örtüsü probleminin ağırlıklı bir versiyonu için bir yaklaşım algoritması tasarlamaya yardım etmesi amacıyla doğrusal programlamayı kullanacağız. Bu bölüm, bu iki güçlü tekniği yüzeysel olarak ele alır. Bölüm notları bu alanların daha ileri çalışmaları için kaynaklar verir.

MAX-3-CNF gerçekleştirilebilirlik için rassal yaklaşım algoritması

Nasıl ki bazı rassal algoritmalar kesin çözümler hesaplar, bazı rassal algoritmalar da yaklaşık çözümler hesaplar. Eğer, herhangi bir n boyutlu girdi için rassal algoritma tarafından üretilen çözümün *beklenen* C maliyeti, bir optimal çözümün C^* maliyetinin bir $\rho(n)$ katı dahilindeyse, bir problem için bir rassal algoritma bir $\rho(n)$ *yaklaşım oranına* sahiptir deriz:

$$\max \left(\frac{C}{C^*}, \frac{C^*}{C} \right) \leq \rho(n) . \quad (35.13)$$

Bir rassal algoritma bir $\rho(n)$ yaklaşım oranını gerçekleştirirse onu **rassal $\rho(n)$ -yaklaşım algoritması** olarak adlandırırız. Diğer bir deyişle, bir rassal yaklaşım algoritması, yaklaşım oranının beklenen bir maliyet için olması dışında deterministik bir yaklaşım algoritması gibidir.

Kısım 34.4 de tanımlanan 3-CNF gerçekleştirilebilirliğin belirli bir örneği gerçekleştirilebilir ya da gerçekleştirilemez. Gerçekleştirilebilir olması için değişkenlerinin öyle bir ataması olmalıdır ki her cümlecik 1 olarak değerlendirilsin. Bir örnek gerçekleştirilebilir değilse gerçekleştirilebilir olmaya ne kadar “yakın” olduğunu hesaplamak isteyebiliriz, yani mümkün olduğunca çok cümleciciği gerçekleştirilen değişkenlerin bir atamasını bulmak isteriz. Sonuçta ortaya çıkan maksimizasyon problemini **MAX-3-CNF gerçekleştirilebilirlik** olarak adlandırırız. MAX-3-CNF gerçekleştirilebilirliğin girdileri 3-CNF gerçekleştirilebilirliğin girdileri ile aynıdır ve amaç 1 olarak değerlendirilen cümleciklerin sayısını maksimize eden bir atama geri döndürmektir. Şimdi, her bir değişkeni 1/2 olasılıkla 1 olarak ve 1/2 olasılıkla 0 olarak rastgele şekilde ayarlamamızın bir rassal 8/7-yaklaşım algoritması verdiğini göstereceğiz. Kısım 34.4 den 3-CNF gerçekleştirilebilirliğin tanımına göre, her bir cümlecğin tam olarak üç farklı literalden oluşmasını isteriz. Ayrıca hiçbir cümlecğin bir değişken ile onun değilinin ikisini birden içermediğini varsayarız (Alıştırma 35.4-1 bu son varsayımı çıkarmanızı ister).

Teorem 35.6

MAX-3-CNF gerçekleştirilebilirliğin n değişkenli, $x_1, x_2, \dots, x_n$ ve m cümlecikli bir örneği verildiğinde, her bir değişkeni bağımsız olarak $1/2$ olasılıkla 1 olarak ve $1/2$ olasılıkla 0 olarak ayarlayan rastgele algoritma, bir rassal $8/7$ -yaklaşım algoritmasıdır.

Kanıt Her bir değişkeni bağımsız olarak $1/2$ olasılıkla 1 olarak ve $1/2$ olasılıkla 0 olarak ayarladığımızı varsayalım. $i = 1, 2, \dots, m$ için

$$Y_i = I \{i \text{ cümlecği gerçekleştirilebilir}\}$$

rassal gösterge değişkenini tanımlarız; böylece i . cümlecikteki kalıp deyimlerden en az birini 1 olarak ayarladığımız sürece $Y_i = 1$ dir. Hiçbir kalıp deyim aynı cümlecikte birden fazla görünmediği için ve hiçbir değişken ile onun değilinin aynı cümlecikte bulunmadığını varsaydığımız için her bir cümlecikteki üç kalıp deyimden ayarlanması birbirinden bağımsızdır. Eğer bir cümlecikteki kalıp deyimlerin üçü birden 0 olarak ayarlanırsa ancak o zaman o cümlecik gerçekleştirilemez ve böylece $\Pr\{i \text{ cümlecği gerçekleştirilemez}\} = (1/2)^3 = 1/8$. Dolayısıyla, $\Pr\{i \text{ cümlecği gerçekleştirilebilir}\} = 1 - 1/8 = 7/8$ olur ve Lemma 5.1 den $E[Y_i] = 7/8$. Toplamda gerçekleştirilen cümleciklerin sayısı Y olsun, böylece $Y = Y_1 + Y_2 + \dots + Y_m$. Bu durumda,

$$\begin{aligned} E[Y] &= E\left[\sum_{i=1}^m Y_i\right] \\ &= \sum_{i=1}^m E[Y_i] \quad (\text{olasılığın doğrusallığından}) \\ &= \sum_{i=1}^m 7/8 \\ &= 7m/8. \end{aligned}$$

Açıktır ki m , gerçekleştirilen cümleciklerin sayısı üzerinde bir üst sınırdır ve dolayısıyla yaklaşım oranı en fazla $m / (7m/8) = 8/7$ dir. ■

Doğrusal programlamayı kullanarak ağırlıklı tepe örtüsüne yaklaşım

Minimum-ağırlıklı tepe-örtüsü probleminde, her bir $v \in V$ tepesinin ilişkili pozitif bir $w(v)$ ağırlığına sahip olduğu yönsüz bir $G = (V, E)$ çizgesi veriliyor. Herhangi bir $V' \subseteq V$ tepe örtüsü için tepe örtüsünün ağırlığını $w(V') = \sum_{v \in V'} w(v)$ olarak tanımlarız. Amaç, minimum ağırlıklı bir tepe örtüsü bulmaktır.

Ağırlıksız tepe örtüsü için kullanılan algoritmayı uygulayamayız ya da bir rassal çözüm kullanamayız; her iki yöntem de optimalden çok uzak çözümler geri döndürebilir. Bununla birlikte, bir doğrusal program kullanarak minimum-a-

ğırlıklı tepe-örtüsünün üzerine bir alt sınır hesaplayacağız. Ardından bu çözümü “yuvarlayacağız” ve bunu bir tepe örtüsü elde etmek için kullanacağız.

Her bir $v \in V$ tepesiyle bir $x(v)$ değişkenini ilişkilendirdiğimizi varsayalım ve her bir $v \in V$ için $x(v)$ değerinin 0 ya da 1 olmasını gerekli kılalım. v yi tepe örtüsüne dahil etmek için gerek ve yeter koşul $x(v) = 1$ olmasıdır. Bu durumda, herhangi bir (u, v) ayrıtı için u ve v den en az birinin tepe örtüsünde bulunması gerektiği kısıtını $x(u) + x(v) \geq 1$ olarak yazabiliriz. Bu fikir, bir minimum-ağırlıklı tepe örtüsü bulmak için aşağıdaki **0-1 tam sayılı programı** meydana getirir:

$$\text{minimize} \quad \sum_{v \in V} w(v) x(v) \quad (35.14)$$

kısıtlar

$$x(u) + x(v) \geq 1 \quad \text{her bir } (u, v) \in E \text{ için} \quad (35.15)$$

$$x(v) \in \{0, 1\} \quad \text{her bir } v \in V \text{ için} \quad (35.16)$$

Bütün $w(v)$ ağırlıklarının 1 e eşit olduğu özel durumda, bu formül NP-zor tepe-örtüsü probleminin optimizasyon versiyonudur. Bununla beraber, $x(v) \in \{0, 1\}$ kısıtını kaldırdığımızı ve onu $0 \leq x(v) \leq 1$ ile değiştirdiğimizi varsayalım. Bu durumda **doğrusal-programlama rahatlığı** olarak bilinen aşağıdaki doğrusal programı elde ederiz:

$$\text{minimize} \quad \sum_{v \in V} w(v) x(v) \quad (35.17)$$

kısıtlar

$$x(u) + x(v) \geq 1 \quad \text{herbir } (u, v) \in E \text{ için} \quad (35.18)$$

$$x(v) \leq 1 \quad \text{herbir } v \in V \text{ için} \quad (35.19)$$

$$x(v) \geq 0 \quad \text{herbir } v \in V \text{ için.} \quad (35.20)$$

(35.14) – (35.16) satırlarındaki 0-1 tam sayılı program için herhangi bir uygun çözüm (35.17) – (35.20) satırlarındaki doğrusal program için de bir uygun çözümdür. Bu nedenle, doğrusal programın bir optimal çözüm değeri, 0-1 tam sayılı programın bir optimal çözüm değeri üzerine bir alt sınır ve dolayısıyla minimum-ağırlıklı tepe-örtüsü problemindeki optimal ağırlık üzerine bir alt sınır sağlar.

Aşağıdaki presedür, minimum-ağırlıklı tepe-örtüsü probleminin bir yaklaşık çözümünü inşa etmek için doğrusal-programlama rahatlığını kullanır:

APPROX-MIN-WEIGHT-VC (G, w)

```

1   $C = \emptyset$ 
2   $\bar{x}$  değerini, (35.17) – (35.20) satırlarındaki doğrusal programın bir
    optimal çözümü, hesapla
3  for her bir  $v \in V$ 
4      if  $\bar{x}(v) \geq 1/2$ 
5           $C = C \cup \{v\}$ 
6  return  $C$ 

```

APPROX-MIN-WEIGHT-VC presedürü şu şekilde çalışır. Birinci satır, tepe örtüsünü boş küme olarak başlatır. İkinci satır, (35.17)–(35.20) satırlarındaki doğrusal programı formülleştirir ve ardından bu doğrusal programı çözer. Bir optimal çözüm her bir v tepesine, $0 \leq \bar{x}(v) \leq 1$ olmak üzere, ilişkili bir $\bar{x}(v)$ değeri verir. Bu değeri, 3. ve 5. satırlar arasında, C tepe örtüsüne hangi tepelerin ekleneceği seçiminde yol göstermesi için kullanırız. Eğer $\bar{x}(v) \geq 1/2$ ise v tepesini C kümesine ekleriz; aksi halde eklemeyiz. Aslına bakılırsa, (35.14) – (35.16) satırlarındaki 0-1 tam sayılı programa bir çözüm elde etmek için doğrusal programın çözümündeki herbir kesirli değişkeni 0 ya da 1 e “yuvarlıyoruz”. Son olarak, altıncı satır C tepe örtüsünü geri döndürür.

Teorem 35.7

APPROX-MIN-WEIGHT-VC algoritması, minimum-ağırlıklı tepe-örtüsü problemi için polinom-zamanlı bir 2-yaklaşım algoritmasıdır.

Kanıt İkinci satırdaki doğrusal programı çözmek için polinom-zamanlı bir algoritma var olduğundan ve 3-5 satırlarındaki **for** döngüsü polinom zamanda çalıştığından APPROX-MIN-WEIGHT-VC polinom-zamanlı bir algoritmadır.

Şimdi APPROX-MIN-WEIGHT-VC nin bir 2-yaklaşım algoritması olduğunu gösterelim. C^* , minimum-ağırlıklı tepe-örtüsü probleminin bir optimal çözümü ve z^* , (35.17)–(35.20) satırlarındaki doğrusal programın bir optimal çözümünün değeri olsun. Bir optimal tepe örtüsü doğrusal programın bir uygun çözümü olduğundan z^* , $w(C^*)$ üzerinde bir alt sınır olmalıdır; yani

$$z^* \leq w(C^*). \quad (35.21)$$

Daha sonra, $\bar{x}(v)$ değişkenlerinin kesirli değerlerini yuvarlayarak, bir tepe örtüsü olan ve $w(C) \leq 2z^*$ eşitsizliğini sağlayan bir C kümesi ürettiğimizi iddia ederiz. C nin bir tepe örtüsü olduğunu görmek için herhangi bir $(u, v) \in E$ ayrıtını düşünelim. (35.18) kısıtından $x(u) + x(v) \geq 1$ olduğunu biliyoruz ve bu da $\bar{x}(u)$ ve $\bar{x}(v)$ den en az birinin en az $1/2$ olduğu anlamına gelir. Dolayısıyla, u ve v den en az biri tepe örtüsüne eklenir ve böylece tüm tepeler örtülür.

Şimdi örtünün ağırlığını düşünelim.

$$\begin{aligned}
z^* &= \sum_{v \in V} w(v) \bar{x}(v) \\
&\geq \sum_{v \in V: \bar{x}(v) \geq 1/2} w(v) \bar{x}(v) \\
&\geq \sum_{v \in V: \bar{x}(v) \geq 1/2} w(v) \cdot \frac{1}{2} \\
&= \sum_{v \in C} w(v) \cdot \frac{1}{2} \\
&= \frac{1}{2} \sum_{v \in C} w(v) \\
&= \frac{1}{2} w(C) .
\end{aligned} \tag{35.22}$$

(35.21) ve (35.22) eşitsizliklerinden

$$w(C) \leq 2z^* \leq 2w(C^*)$$

olur ve dolayısıyla APPROX-MIN-WEIGHT-VC bir 2-yaklaşım algoritmasıdır. ■

Alıştırmalar

35.4-1

Bir cümleinin bir değişken ve onun değerini aynı anda içermesine izin versek bile, her bir değişkeni 1/2 olasılıkla 1 ve 1/2 olasılıkla 0 olarak rastgele ayarlamamanın bir rassal 8/7-yaklaşım algoritması verdiğini gösteriniz.

35.4-2

MAX-CNF gerçekleştirilebilirlik problemi, her bir cümlecığı tam olarak 3 litemale sahip olması bakımından kısıtlamamakla birlikte MAX-3-CNF gerçekleştirilebilirlik problemi gibidir. MAX-CNF gerçekleştirilebilirlik problemi için bir rassal 2-yaklaşım algoritması veriniz.

35.4-3

MAX-CUT probleminde bir ağırlıksız, yönsüz $G = (V, E)$ çizgesi veriliyor. Bir $(S, V-S)$ kesimini Bölüm 23 deki gibi ve bir kesimin **ağırlığını** kesim kümesiyle çakışan ayrıtların sayısı olarak tanımlarız. Amaç maksimum ağırlıklı bir kesim kümesi bulmaktır. Her bir v tepesi için v yi 1/2 olasılıkla S kümesine ve 1/2 olasılıkla $V-S$ kümesine rassal ve bağımsız olarak yerleştirdiğimizi farzedelim. Bu algoritmanın bir rassal 2-yaklaşım algoritması olduğunu gösteriniz.

35.4-4

(35.19) satırındaki kısıtları (35.17) – (35.20) satırlarındaki doğrusal programdan çıkarırsak, sonuçta oluşan doğrusal programın herhangi bir optimal çözümü, her bir $v \in V$ için $x(v) \leq 1$ eşitsizliğini sağlamalıdır. Bu bakımdan, bu kısıtların gereksiz olduğunu gösteriniz.

35.5 Altküme-toplam problemi

Kısım 34.5.5 den, S pozitif tam sayıların bir $\{x_1, x_2, \dots, x_n\}$ kümesi ve t pozitif bir tam sayı olmak üzere, altküme-toplam probleminin bir örneğinin bir (S, t) çifti olduğunu hatırlayınız. Bu karar problemi, S nin toplamı tam olarak t hedef değerini bulan bir altkümesinin var olup olmadığını sorar. Kısım 34.5.5 de gördüğümüz üzere bu problem NP-tamdır.

Bu karar problemiyle ilişkili optimizasyon problemi gerçekçi uygulamalarda ortaya çıkar. Optimizasyon probleminde, $\{x_1, x_2, \dots, x_n\}$ kümesinin toplamı mümkün olduğunca büyük olan ama t yi aşmayan bir altkümesini bulmak isteriz. Örneğin, t pounddan daha fazlasını taşıyamayan bir kamyon ve i . kutusunun ağırlığı x_i pound olmak üzere yüklenecek n tane farklı kutumuz olabilir. Verilen ağırlık sınırını aşmadan, kamyonu mümkün olduğunca ağır bir yük ile doldurmak isteriz.

Bu bölümde, bu optimizasyon problemi için optimal değer hesaplayan üssel-zamanlı bir algoritma sunacağız ve ardından tam polinom-zamanlı bir yaklaşım şeması olması için bu algoritmanın nasıl değiştirildiğini göstereceğiz. (Tam polinom-zamanlı bir yaklaşım şemasının, $1/\epsilon$ ve ayrıca girdinin boyutunda polinom çalışma zamanına sahip olduğunu hatırlayınız.)

Bir üstel-zamanlı kesin algoritma

S nin herbir S' altkümesi için S' kümesindeki elemanların toplamını hesapladığımızı ve ardından toplamı t yi geçmeyen altkümeler arasından, toplamı t ye en yakın olanı seçtiğimizi varsayalım. Açık ki bu algoritma optimal çözüm geri döndürürdü, ancak bu üssel zaman alabilirdi. Bu algoritmayı uygulamak için $\{x_1, x_2, \dots, x_{i-1}\}$ kümesinin tüm altkümelerinin toplamalarını bir başlangıç noktası olarak kullanarak, i . iterasyonda $\{x_1, x_2, \dots, x_i\}$ kümesinin tüm altkümelerinin toplamalarını hesaplayan özyinelemeli bir prosedür kullanabilirdik. Böyle yaparak, belirli bir S' kümesi bir kere t yi aşan bir toplama sahip olduğunda onunla devam etmek için bir nedenin olmadığını anlardık; çünkü S' kümesinin hiçbir üstkümesi optimal çözüm olamazdı. Şimdi bu stratejinin bir uygulamasını vereceğiz.

EXACT-SUBSET-SUM prosedürü bir $S = \{x_1, x_2, \dots, x_n\}$ girdi kümesini ve bir t hedef değerini alır; kabakod çok geçmeden göreceğiz. Bu prosedür, özyinelemeli

olarak, $\{x_1, x_2, \dots, x_n\}$ kümesinin t yi geçmeyen tüm altkümelerinin toplamalarının listesi olan L_i yi hesaplar ve ardından L_n deki maksimum değeri geri döndürür.

L pozitif tam sayıların bir listesi ve x başka bir pozitif tam sayıysa, $L + x$, L deki her bir elemanı x ile artırarak L den türetilen listeyi belirtir. Örneğin, eğer $L = \langle 1, 2, 3, 5, 9 \rangle$ ise $L+2 = \langle 3, 4, 5, 7, 11 \rangle$ olur. Bu gösterimi kümeler için de şu şekilde kullanırız:

$$S + x = \{s + x : s \in S\}.$$

Ayrıca, bir MERGE-LISTS (L, L') yardımcı prosedürünü kullanırız. Bu prosedür, tekrarlı değerleri çıkartarak L ve L' sıralı girdi listelerinin birleşimi olan sıralı listeyi geri döndürür. Birleştirerek sıralamada kullandığımız (Kısım 2.3.1) MERGE prosedürü gibi MERGE-LISTS de $O(|L| + |L'|)$ zamanda çalışır. MERGE-LISTS için kabakodu ihmal ederiz.

EXACT-SUBSET-SUM (S, t)

```

1   $n = |S|$ 
2   $L_0 = \langle 0 \rangle$ 
3  for  $i = 1$  to  $n$ 
4       $L_i = \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x)$ 
5       $t$  den büyük olan her elemanı  $L$  den çıkar
6  return  $L_n$  deki maksimum eleman
```

EXACT-SUBSET-SUM ın nasıl çalıştığını görmek için P_i , $\{x_1, x_2, \dots, x_i\}$ kümesinin bir altkümesini (boş küme olabilir) seçerek ve onun elemanlarını toplayarak elde edilen tüm değerlerin kümesini belirtsin. Örneğin, $S = \{1, 4, 5\}$ ise

$$\begin{aligned}
 P_1 &= \{0, 1\}, \\
 P_2 &= \{0, 1, 4, 5\}, \\
 P_3 &= \{0, 1, 4, 5, 6, 9, 10\}.
 \end{aligned}$$

Aşağıdaki

$$P_i = P_{i-1} \cup (P_{i-1} + x_i) \quad (35.23)$$

özdeşliği verildiğinde (Alıştırma 35.5-1 e bakınız), L_i listesinin P_i nin değeri t den fazla olmayan her elemanını içeren sıralı bir liste olduğunu i üzerinde tümevarımla kanıtlayabiliriz. t nin $|S|$ ye bağlı polinom olduğu ya da S deki tüm sayıların $|S|$ nin bir polinomuyla sınırlı olduğu özel durumlarda polinom-zamanlı bir algoritma olmasına rağmen, L_i listesinin uzunluğu 2^i kadar olabildiği için EXACT-SUBSET-SUM genel olarak üssel-zamanlı bir algoritmadır.

Bir tam polinom-zamanlı yaklaşım şeması

Altküme-toplamı problemi için bir tam polinom-zamanlı yaklaşım şemasını, her bir L_i listesini yaratıldıktan sonra “kırparak” türetebiliriz. Kırparmanın ardındaki

fikir şudur: eğer L deki iki değer birbirlerine yakınsa, biz sadece yaklaşık bir çözüm istediğimizden bariz olarak her ikisiyle devam etmemiz gerekmez. Daha doğrusu, $0 < \delta < 1$ olacak şekilde bir kırpma parametresi kullanırız. Bir L listesini δ ile kırptığımızda L den mümkün olduğunca çok elemanı şu şekilde çıkarırız: L' , L yi kırpmanın sonucu ise, L den çıkarılan her y elemanı için L' listesinde hala y ye yaklaşık olan bir z elemanı vardır; yani

$$\frac{y}{1 + \delta} \leq z \leq y. \quad (35.24)$$

Böyle bir z elemanını yeni L' listesinde y yi “temsil eden” eleman olarak düşünebiliriz. Çıkarılan her bir y elemanı, (35.24) eşitsizliğini sağlayan bir z kalan elemanı ile temsil edilir. Örneğin, eğer $\delta = 0.1$ ve

$$L = \langle 10, 11, 12, 15, 20, 21, 22, 23, 24, 29 \rangle$$

ise

$$L' = \langle 10, 12, 15, 20, 23, 29 \rangle$$

listesini elde etmek için L listesini kırpabiliriz. Bu listede, silinen 11 değeri 10 tarafından temsil edilir, silinen 21 ve 22 değerleri 20 tarafından ve silinen 24 değeri 23 tarafından temsil edilir. Listenin kırpılmış versiyonunun her elemanı aynı zamanda listenin orijinal versiyonunun da bir elemanı olduğundan, silinen her bir eleman için yakın (ve biraz daha küçük) bir temsilci değerini listede tutarken, kırpma işlemi bu saklanan elemanların sayısını önemli ölçüde azaltabilir.

L ve δ verildiğinde ve L nin monoton olarak artan şekilde sıralı olduğunu varsayarak, aşağıdaki prosedür $L = \langle y_1, y_2, \dots, y_m \rangle$ listesini $\Theta(m)$ zamanında kırpar. Prosedürün çıktısı, kırpılmış bir sıralı listedir.

TRIM (L, δ)

```

1   $m, L$  nin uzunluğu olsun
2   $L' = \langle y_1 \rangle$ 
3   $last = y_1$ 
4  for  $i = 2$  to  $m$ 
5      if  $y_i > last \cdot (1 + \delta)$  //  $L$  sıralı olduğundan  $y_i \geq last$ 
6           $y_i$  elemanını  $L'$  listesinin sonuna ekle
7           $last = y_i$ 
8  return  $L'$ 
```

Prosedür L nin elemanlarını monoton olarak artan sırada tarar. Eğer bir sayı L nin ilk elemanıysa ya da L' listesine yerleştirilen en son sayı tarafından temsil edilemezse ancak o zaman geri döndürülen L' listesinin sonuna eklenir.

TRIM prosedürü verildiğinde yaklaşım şemamızı şu şekilde inşa edebiliriz. Bu prosedür n tam sayının bir $S = \{x_1, x_2, \dots, x_n\}$ kümesini (gelişigüzel sırada), bir t hedef değerini ve

$$0 < \epsilon < 1 \quad (35.25)$$

olmak üzere bir ϵ “yaklaşım parametresini” girdi olarak alır. Değeri, optimal çözümün $1 + \epsilon$ katı dahilinde olan bir z değerini geri döndürür.

APPROX-SUBSET-SUM (S, t, ϵ)

```

1   $n = |S|$ 
2   $L_0 = \langle 0 \rangle$ 
3  for  $i = 1$  to  $n$ 
4       $L_i = \text{MERGE-LISTS}(L_{i-1}, L_{i-1} + x_i)$ 
5       $L_i = \text{TRIM}(L_i, \epsilon / 2n)$ 
6       $t$  den büyük olan her elemanı  $L_i$  den kaldır
7   $z^*, L_n$  deki maksimum eleman olsun
8  return  $z^*$ 
```

İkinci satır, L_0 listesini sadece 0 elemanını içeren liste olarak başlatır. Üçüncü ile altıncı satırlar arasındaki **for** döngüsü, P_i kümesinin uygun şekilde kırpılmış bir versiyonunu içeren sıralı bir liste olarak ve t den büyük bütün elemanları çıkarak L_i yi hesaplar. L_i yi L_{i-1} listesinden oluşturduğumuz için tekrarlı kırpmanın çok fazla hata ortaya çıkarmadığından emin olmalıyız. Hemen şimdi APPROX-SUBSET-SUM eğer varsa doğru bir yaklaşım geri döndürdüğünü göreceğiz.

Bir örnek olarak, $t = 308$ ve $\epsilon = 0.40$ ile

$$S = \langle 104, 102, 201, 101 \rangle$$

örneğini farzedelim. δ kırpma parametresi $\epsilon / 8 = 0.05$ dir. APPROX-SUBSET-SUM aşağıdaki değerleri belirtilen satırlarda hesaplar:

$$\text{satır 2: } L_0 = \langle 0 \rangle ,$$

$$\text{satır 4: } L_1 = \langle 0, 104 \rangle ,$$

$$\text{satır 5: } L_1 = \langle 0, 104 \rangle ,$$

$$\text{satır 6: } L_1 = \langle 0, 104 \rangle ,$$

$$\text{satır 4: } L_2 = \langle 0, 102, 104, 206 \rangle ,$$

$$\text{satır 5: } L_2 = \langle 0, 102, 206 \rangle ,$$

$$\text{satır 6: } L_2 = \langle 0, 102, 206 \rangle ,$$

$$\text{satır 4: } L_3 = \langle 0, 102, 201, 206, 303, 407 \rangle ,$$

$$\text{satır 5: } L_3 = \langle 0, 102, 201, 303, 407 \rangle ,$$

$$\text{satır 6: } L_3 = \langle 0, 102, 201, 303 \rangle ,$$

$$\text{satır 4: } L_4 = \langle 0, 101, 102, 201, 203, 302, 303, 404 \rangle ,$$

$$\text{satır 5: } L_4 = \langle 0, 101, 201, 302, 404 \rangle ,$$

$$\text{satır 6: } L_4 = \langle 0, 101, 201, 302 \rangle .$$

Algoritma $z^* = 302$ değerini yanıtı olarak geri döndürür. Bu değer, $307 = 104 + 102 + 101$ optimal yanıtının $\epsilon = \%40$ dahilinde iyidir; aslında $\%2$ içindedir.

Teorem 35.8

APPROX-SUBSET-SUM, altküme-toplam problemi için bir tam polinom-zamanlı yaklaşım şemasıdır.

Kanıt Beşinci satırda L_i yi kırma ve t den büyük olan her elemanı L_i den çıkarma işlemleri L_i nin her elemanının aynı zamanda P_i nin de bir üyesi olma özelliğini devam ettirir. Dolayısıyla, 8. satırda geri döndürülen z^* değeri gerçekte S nin bir altkümesinin toplamıdır. $y^* \in P_n$, altküme-toplam probleminin bir optimal çözümünü belirtsin. Bu durumda 6. satırdan $z^* \leq y^*$ olduğunu biliyoruz. (35.1) eşitsizliğine göre, $y^* / z^* \leq 1 + \epsilon$ olduğunu göstermemiz gerekir. Ayrıca bu algoritmanın çalışma zamanının hem $1 / \epsilon$ hem de girdi boyutunda polinom olduğunu göstermeliyiz.

Alıştırma 35.5-2 nin göstermenizi istediği üzere, P_i de değeri en fazla t olan her y elemanı için

$$\frac{y}{(1 + \epsilon/2n)^i} \leq z \leq y \quad (35.26)$$

olacak şekilde bir $z \in L_i$ elemanı vardır. (35.26) eşitsizliği $y^* \in P_n$ için sağlanmalıdır ve dolayısıyla

$$\frac{y^*}{(1 + \epsilon/2n)^n} \leq z \leq y^*$$

olacak şekilde bir $z \in L_n$ elemanı vardır ve böylece

$$\frac{y^*}{z} \leq \left(1 + \frac{\epsilon}{2n}\right)^n. \quad (35.27)$$

(35.27) eşitsizliğini yerine getiren bir $z \in L_n$ elemanı var olduğundan, eşitsizlik L_n deki maksimum değer olan z^* için sağlanmalıdır; yani,

$$\frac{y^*}{z^*} \leq \left(1 + \frac{\epsilon}{2n}\right)^n. \quad (35.28)$$

Şimdi $y^*/z^* \leq 1 + \epsilon$ olduğunu gösterelim. Bunu $(1 + \epsilon/2n)^n \leq 1 + \epsilon$ olduğunu göstererek yaparız. (3.14) denkleminde göre, $\lim_{n \rightarrow \infty} (1 + \epsilon/2n)^n = e^{\epsilon/2}$ dir. Alıştırma 35.5-3,

$$\frac{d}{dn} \left(1 + \frac{\epsilon}{2n}\right)^n > 0 \quad (35.29)$$

olduğunu göstermenizi ister. Dolayısıyla, $(1 + \epsilon/2n)^n$ fonksiyonu, $e^{\epsilon/2}$ limit değerine yaklaştıkça n ile birlikte artar ve

$$\begin{aligned}
\left(1 + \frac{\epsilon}{2n}\right)^n &\leq e^{\epsilon/2} \\
&\leq 1 + \epsilon/2 + (\epsilon/2)^2 \quad ((3.13) \text{ eşitsizliğine göre}) \\
&\leq 1 + \epsilon \quad ((35.25) \text{ eşitsizliğine göre})
\end{aligned} \tag{35.30}$$

(35.28) ve (35.30) eşitsizliklerinin birleştirilmesi yaklaşım oranının analizini tamamlar.

APPROX-SUBSET-SUM ın bir tam polinom-zamanlı yaklaşım şeması olduğunu göstermek için L_i nin uzunluğu üzerinde bir sınır elde ederiz. Kırpma işleminden sonra, L_i nin ardışık z ve z' elemanları arasında $z'/z > 1 + \epsilon/2n$ ilişkisi olmalıdır. Her bir liste, dolayısıyla, 0 değerini, muhtemelen 1 değerini ve $\lfloor \log_{1+\epsilon/2n} t \rfloor$ e kadarki ilave değerleri içerir. Her bir L_i listesindeki elemanların sayısı en fazla

$$\begin{aligned}
\log_{1+\epsilon/2n} t + 2 &= \frac{\ln t}{\ln(1 + \epsilon/2n)} + 2 \\
&\leq \frac{2n(1 + \epsilon/2n) \ln t}{\epsilon} + 2 \quad ((3.17) \text{ eşitsizliğine göre}) \\
&< \frac{3n \ln t}{\epsilon} + 2 \quad ((35.25) \text{ eşitsizliğine göre})
\end{aligned}$$

dir. Bu sınır, girdinin boyutunda $-t$ yi göstermek için gerekli bitlerin sayısı $\lg t$ artı S kümesini göstermek için gerekli bitlerin sayısı, dolayısıyla (in turn) n de polinom – ve $1/\epsilon$ de polinomdur. APPROX-SUBSET-SUM ın çalışma zamanı L_i nin uzunluklarında polinom olduğundan APPROX-SUBSET-SUM ın bir tam polinom-zamanlı yaklaşım şeması olduğu sonucuna ulaşırız. ■

Alıştırmalar

35.5-1

(35.23) denklemini kanıtlayınız. Ardından, EXACT-SUBSET-SUM ın 5. satırını uyguladıktan sonra L_i nin, P_i nin değeri t den büyük olmayan her elemanını içeren bir sıralı liste olduğunu gösteriniz.

35.5-2

i üzerinde tümevarım kullanarak (35.26) eşitsizliğini kanıtlayınız.

35.5-3

(35.29) eşitsizliğini kanıtlayınız.

35.5-4

Bu bölümde sunulan yaklaşım şemasını, verilen girdi listesinin bir altkümesinin toplamı olan t sayısından küçük olmayan minimum değere iyi bir yaklaşım bulmak için nasıl değiştirirsiniz?

35.5-5

APPROX-SUBSET-SUM prosedürünü, S nin, toplamı z^* değerini veren altkümesini de geri döndürecek şekilde değiştiriniz.

Problemler
35-1 Kutu paketleme

i. nesnenin s_i boyutu $0 < s_i < 1$ olmak üzere, n tane nesneden oluşan bir küme verildiğini varsayalım. Bütün nesneleri en az sayıda bir-boyutlu kutu kullanarak paketlemek isteriz. Her bir kutu, nesnelerin toplam boyutu 1 i geçmeyen herhangi bir altkümesini tutabilir.

a. Gerekli kutuların minimum sayısını belirleme probleminin NP-zor olduğunu kanıtlayınız. (*İpucu:* Altküme-toplam probleminden indirgeyiniz).

İlk-uygun sezgiseli her bir nesneyi sırasıyla alır ve onu yerleştirebildiği ilk bidona yerleştirir. $S = \sum_{i=1}^n s_i$ olsun.

b. Gerekli bidonların optimal sayısının en az $\lceil S \rceil$ olduğu üzerinde tartışınız.

c. İlk-uygun sezgiselinin en fazla bir kutuyu yarısından daha az dolu olarak bırakması üzerine tartışınız.

d. İlk-uygun sezgiseli tarafından kullanılan kutuların sayısının asla $\lceil 2S \rceil$ den daha fazla olmadığını kanıtlayınız.

e. İlk-uygun sezgiseli için bir 2 yaklaşım oranını kanıtlayınız.

f. İlk-uygun sezgiselinin etkili bir uygulamasını veriniz ve çalışma zamanını analiz ediniz.

35-2 Bir maksimum kliğin boyutuna yaklaşım

$G = (V, E)$ yönsüz bir çizge olsun. Herhangi bir $k \geq 1$ için $G^{(k)}$ yı bir $(V^{(k)}, E^{(k)})$ yönsüz çizgesi olarak tanımlayınız. $V^{(k)}$, V deki tepelerin tüm sıralı k -bileşenlerinin kümesidir ve $E^{(k)}$ şu şekilde tanımlanır: $(v_1, v_2, \dots, v_k)$ ile $(w_1, w_2, \dots, w_k)$ nın bitişik olması için gerek ve yeter koşul, $i = 1, 2, \dots, k$ için v_i tepesinin G de w_i ye bitişik olması ya da $v_i = w_i$ olmasıdır.

- a. $G^{(k)}$ çizgesindeki maksimum kliğin boyutunun G çizgesindeki maksimum kliğin boyutunun k . kuvvetine eşit olduğunu gösteriniz.
- b. Bir maksimum-boyutlu klik bulmak için bir sabit sayı yaklaşım oranına sahip bir yaklaşım algoritması varsa, problem için polinom-zamanlı bir yaklaşım şemasının var olduğunu tartışınız.

35-3 Ağırlıklı küme-örtüsü problemi

Küme-örtüsü problemini, $\mathcal{F}$ ailesindeki her bir S_i kümesi ilişkili bir w_i ağırlığına sahip ve bir $\mathcal{C}$ örtüsünün ağırlığı $\sum_{S_i \in \mathcal{C}} w_i$ olacak şekilde genellediğimizi varsayalım. Bir minimum-k-ağırlıklı örtü belirlemek isteriz. (Kısım 35.3, tüm i değerleri için $w_i = 1$ olduğu durumu ele alır.)

Ağırlıklı küme-örtüsü probleminin herhangi bir örneği için yaklaşık bir çözüm sağlamak amacıyla, greedy küme-örtüsü sezgiselinin doğal bir şekilde nasıl genelleneceğini gösteriniz. d , herhangi bir S_i kümesinin maksimum boyutu olmak üzere, sezgiselinizin bir $H(d)$ yaklaşım oranına sahip olduğunu gösteriniz.

35-4 Maksimum eşleme

Yönsüz bir G çizgesi için bir eşlemenin, kümedeki hiçbir ayrıt çiftinin aynı tepe üzerinde bitişik olmadığı bir ayrıtlar kümesi olduğunu hatırlayınız. Kısım 26.3 de, iki parçalı bir çizgede bir maksimum eşlemenin nasıl bulunduğunu görmüştük. Bu problemde genel olarak yönsüz çizgelerdeki eşlemelere bakacağız (yani çizgelerin iki parçalı olması gerekli değil).

- a. Bir **maksimal eşleme** herhangi diğer bir eşlemenin uygun bir altkümesi olmayan bir eşlemedir. Bir maksimal eşlemenin bir maksimum eşleme olması gerekmediğini yönsüz bir G çizgesi ve G de bir maksimum eşleme olmayan bir M maksimal eşlemesi sunarak gösteriniz. (*İpucu:* Sadece dört tepeli böyle bir çizge bulabilirsiniz.)
- b. Yönsüz bir $G = (V, E)$ çizgesini düşünün. G de bir maksimal eşleme bulmak için $O(E)$ -zamanlı bir açgözlü algoritması veriniz.

Bu problemde, maksimum eşleme için polinom-zamanlı bir yaklaşım algoritması üzerinde yoğunlaşacağız. Maksimum eşleme için bilinen en hızlı algoritma superlineer (ama polinomsal) zaman alırken buradaki yaklaşım algoritması doğrusal zamanda çalışacak. Maksimal eşleme için (b) kısmındaki doğrusal-zamanlı açgözlü algoritmasının maksimum eşleme için bir 2-yaklaşım algoritması olduğunu göstereceksiniz.

- c. G de bir maksimum eşlemenin boyutunun, G için herhangi bir tepe örtüsünün boyutu üzerinde bir alt sınır olduğunu gösteriniz.

d. $G = (V, E)$ de bir M maksimal eşlemesini düşünün.

$$T = \{v \in V : M \text{ deki herhangi bir ayrınt } v \text{ ye bitişik}\}$$

olsun. G nin T de olmayan tepeleri tarafından indirgenmiş G nin bir alt çizgesi hakkında ne söyleyebilirsiniz?

e. (d) kısmından, G için bir tepe örtüsünün boyutunun $2|M|$ olduğu sonucuna ulaşınız.

f. (c) ve (e) kısımlarını kullanarak (b) kısmındaki ağgözlü algoritmasının maksimum eşleme için bir 2-yaklaşım algoritması olduğunu kanıtlayınız..

35-5 Paralel makine çizelgeleme

Paralel-makine-çizelgeleme probleminde, her bir J_k işi ilişkili bir negatif olmayan p_k işlem süresine sahip olmak üzere, n tane $J_1, J_2, \dots, J_n$ işleri verilir. Ayrıca m tane özdeş $M_1, M_2, \dots, M_m$ makineleri verilir. Herhangi bir iş herhangi bir makinede yapılabilir. Bir **çizelge**, her bir J_k işi için yürütüldüğü makineyi ve çalışma esnasındaki zaman periyodunu belirler. Her bir J_k işi bir M_i makinesinde p_k ardışık zaman birimi boyunca yürütülmelidir ve o zaman periyodu boyunca diğer hiçbir iş M_i makinesinde yapılmaz. C_k, J_k işinin **tamamlama süresi**, yani J_k işinin işlemini tamamladığı zaman olsun. Bir çizelge verildiğinde, $C_{\max} = \max_{1 \leq j \leq n} C_j$ ifadesini çizelgenin **üretim süresi** olarak tanımlarız. Amaç, üretim süresi minimum olan bir çizelgeyi bulmaktır.

Örneğin, M_1 ve M_2 gibi iki makinemiz ve $p_1 = 2, p_2 = 12, p_3 = 4$ ve $p_4 = 5$ olmak üzere J_1, J_2, J_3, J_4 işlerimizin olduğunu varsayalım. Bu durumda bir mümkün çizelge için M_1 makinesinde J_1 işi ve takibinde J_2 işi yapılır; M_2 makinesinde ise J_4 işi ve ardından J_3 işi yapılır. Bu çizelge için $C_1 = 2, C_2 = 14, C_3 = 9, C_4 = 5$ ve $C_{\max} = 14$ dür. Bir optimal çizelge, J_2 işini M_1 makinesinde ve J_1, J_3 ve J_4 işlerini M_2 makinesinde çalıştırır. Bu çizelge için $C_1 = 2, C_2 = 12, C_3 = 6, C_4 = 11$ ve $C_{\max} = 12$ dir.

Bir paralel-makine-çizelgeleme problemi verildiğinde, $C_{\max}^*$ bir optimal çizelgenin üretim süresini belirlsin.

a. Optimal üretim süresinin en az maksimum işlem süresi kadar büyük olduğunu gösteriniz, yani

$$C_{\max}^* \geq \max_{1 \leq k \leq n} p_k .$$

b. Optimal üretim süresinin en az ortalama makine yükü kadar büyük olduğunu gösteriniz, yani

$$C_{\max}^* \geq \frac{1}{m} \sum_{1 \leq k \leq n} p_k .$$

Paralel makine çizelgelemesi için şu açgözlü algoritmasını kullandığımızı varsayın: ne zaman bir makine boşta ise, henüz çizelgelenmemiş olan herhangi bir işi çizelgele.

- c. Bu açgözlü algoritmasını uygulamak için kabakod yazınız. Algoritmanızın çalışma zamanı nedir?
- d. Açgözlü algoritması tarafından geri döndürülen çizelge için

$$C_{\max} \leq \frac{1}{m} \sum_{1 \leq k \leq n} p_k + \max_{1 \leq k \leq n} p_k$$

olduğunu gösteriniz. Bu algoritmanın polinom-zamanlı bir 2-yaklaşım algoritması olduğu sonucuna ulaşınız.

35-6 Bir maksimum kapsayan ağaca yaklaşım

$G = (V, E)$, her bir $(u, v) \in E$ üzerindeki farklı $w(u, v)$ ağırlıklarıyla yönsüz bir çizge olsun. Her bir $v \in V$ tepesi için $\max(v) = \operatorname{argmax}_{(u, v) \in E} \{w(u, v)\}$, o tepeye bitişik olan maksimum-ağırlıklı ayrıt olsun. $S_G = \{\max(v) : v \in V\}$, herbir tepeye bitişik olan maksimum-ağırlıklı ayrıtların kümesi ve T_G , G nin maksimum-ağırlıklı kapsayan ağacı, yani maksimum toplam ağırlıklı spanning ağaç olsun. Ayrıtların herhangi bir $E' \subseteq E$ altkümesi için $w(E) = w(E') = \sum_{(u, v) \in E'} w(u, v)$ tanımlayınız.

- a. $S_G = T_G$ olan en az 4 tepeli bir çizgenin örneğini veriniz.
- b. $S_G \neq T_G$ olan en az 4 tepeli bir çizgenin örneğini veriniz.
- c. Herhangi bir G çizgesi için $S_G \subseteq T_G$ olduğunu kanıtlayınız.
- d. Herhangi bir G çizgesi için $w(T_G) \geq w(S_G) / 2$ olduğunu kanıtlayınız.
- e. Maksimum kapsayan ağaca bir 2-yaklaşım hesaplamak için $O(V + E)$ -zamanlı bir algoritma veriniz.

35-7 0-1 Sırt çantası problemi için bir yaklaşım algoritması

Sırt çantası problemini Kısım 16.2 den hatırlayınız. i . nesne v_i dolar değerinde ve w_i pound ağırlığında olmak üzere n tane nesne vardır. Ayrıca en fazla W pound taşıyabilen bir sırt çantası veriliyor. Burada, her bir w_i ağırlığının en fazla W olduğu ve nesnelerin, değerlerinin monoton azalan sırasında indislendiği varsayımlarını da ekleriz: $v_1 \geq v_2 \geq \dots \geq v_n$.

0-1 sırt çantası probleminde, toplam ağırlığı en fazla W olan ve toplam değeri maksimum olan nesnelerin bir altkümesini bulmak isteriz. Kesirli sırt çantası problemi 0-1 sırt çantasına benzer; ancak bu problemde her bir nesnenin hepsini ya da hiçbirini almaya kısıtlanmak yerine bir bölümünü alabiliriz. $0 \leq x_i \leq 1$ ol-

mak üzere, eğer i nesnesinin x_i kadarını alırsak, sırt çantasının ağırlığına $x_i w_i$ kadar katkı yaparız ve $x_i v_i$ değerini elde ederiz. Amacımız, 0-1 sırt çantası problemi için polinom-zamanlı bir 2-yaklaşım algoritması geliştirmektir.

Polinom-zamanlı bir algoritma tasarlamak için 0-1 sırt çantası probleminin kısıtlanmış örneklerini ele alırız. Sırt çantası probleminin bir I örneği verildiğinde, $j = 1, 2, \dots, n$ için I_j kısıtlanmış örneklerini, $1, 2, \dots, j - 1$ nesnelerini çıkartarak ve çözümün j nesnesini içermesini şart koşarak oluştururuz (hem kesirli hem de 0-1 sırt çantası problemlerindeki j nesnesinin tamamı). I_1 örneğinde hiçbir nesne çıkarılmaz. I_j örneği için P_j , 0 – 1 problemin bir optimal çözümünü ve Q_j , kesirli problemin bir optimal çözümünü belirtsin.

- a. 0-1 sırt çantası probleminin I örneğinin bir optimal çözümünün $\{P_1, P_2, \dots, P_n\}$ den biri olduğu üzerinde tartışınız.
- b. I_j örneği için j nesnesini dahil ederek ve ardından her bir adımda $\{j + 1, j + 2, \dots, n\}$ kümesindeki seçilmemiş nesnenin mümkün olduğunca fazlasını pound başına maksimum değer v_i / w_i ile aldığımız açgözlü algoritmasını kullanarak kesirli problemin bir Q_j optimal çözümünü bulabildiğimizi kanıtlayınız.
- c. I_j örneği için kesirli problemin, en fazla bir nesneyi kesirli olarak içeren bir Q_j optimal çözümünü her zaman inşa edebildiğimizi kanıtlayınız. Bir başka deyişle, muhtemelen biri hariç tüm nesneler için sırt çantasına nesnenin tamamını ekleriz ya da hiçbir kısmını eklemeyiz.
- d. I_j örneği için kesirli problemin bir Q_j optimal çözümü verildiğinde, Q_j den R_j çözümünü Q_j den bazı kesirli nesneleri silerek oluşturunuz. $v(S)$, bir S çözümlüne alınan nesnelerin toplam değerini belirtsin. $v(R_j) \geq v(Q_j) / 2 \geq v(P_j) / 2$ olduğunu kanıtlayınız.
- e. $\{R_1, R_2, \dots, R_n\}$ kümesinden bir maksimum-değerli çözüm geri döndüren polinom-zamanlı bir algoritma veriniz ve algoritmanızın 0-1 sırt çantası problemi için polinom-zamanlı bir 2-yaklaşım algoritması olduğunu kanıtlayınız.

Bölüm notları

Kesin çözümler hesaplamayan yöntemler binlerce yıldır biliniyor olmasına rağmen (örneğin, π nin değerine yaklaşmak için yöntemler), bir yaklaşım algoritması kavramı çok daha yenidir. Hochbaum [172], polinom-zamanlı bir yaklaşım algoritması kavramını biçimlendirme işini Garey, Graham ve Ullman [128] ile Johnson [190] a atfeder. Bu şekildeki ilk algoritma sıklıkla Graham [149] a mal edilir.

Bu erken çalışmadan beri çok çeşitli problemler için binlerce yaklaşım algoritması tasarlanmıştır ve bu alanda bir sürü kaynak mevcuttur. Ausiello vd. [26], Hochbaum [172] ve Vazirani [345] tarafından en son metinler, Shmoys [315] ile Klein ve Young [207] ın yaptığı incelemelerde olduğu gibi sadece yaklaşım algoritmalarından bahseder. Garey ve Johnson [129] ile Papadimitriou ve Steiglitz [271] gibi diğer birçok metin de yaklaşım algoritmalarını önemli ölçüde ele alır. Lawler, Lenstra, Rinnooy Kan ve Shmoys [225], gezgin-satıcı problemi için yaklaşım algoritmalarının geniş çaplı bir uygulamasını sağlar.

Papadimitriou ve Steiglitz APPROX-VERTEX-COVER algoritmasını F. Gavril ve M. Yannakakis'e atfeder. Tepe-örtüsü problemi kapsamlı bir şekilde çalışılmıştır (Hochbaum [172] bu problem için 16 farklı yaklaşım algoritması listeler); ancak bütün yaklaşım oranları en az $2 - o(1)$ dir.

APPROX-TSP-TOUR algoritması, Rosenkrantz, Stearn ve Lewis [298] tarafından yazılan bir makalede görülür. Christofides bu algoritmayı geliştirmiş ve üçgen eşitsizlikli gezgin-satıcı problemi için bir $3/2$ -yaklaşım algoritması vermiştir. Arora [22] ve Mitchell [257], eğer noktalar öklid düzlemde ise polinom-zamanlı bir yaklaşım şemasının olduğunu göstermişlerdir. Teorem 35.3, Sahni ve Gonzalez [301] sayesinde.

Küme-örtüsü problemi için greedy sezgiselinin analizi, daha genel bir sonucun Chvátal [68] tarafından yayınlanan kanıtından sonra modellenmiştir; burada sunulduğu gibi temel sonuç Johnson [190] ve Lovász [238] dolayısıyladır.

APPROX-SUBSET-SUM algoritması ve onun analizi, sırt çantası ve altkümeye-toplamı problemleri için Ibarra ve Kim [187] tarafından oluşturulan ilişkili yaklaşım algoritmalarından sonra genel hatlarıyla biçimlendirilmiştir.

Problem 35-7, sırt çantası-tipindeki tam sayılı programlara yaklaşma üzerine Bienstock ve McClosky [45] tarafından sunulan daha genel bir sonucunun kombinatorik bir versiyonudur.

MAX-3-CNF gerçekleştirilebilirliği için rassal algoritma Johnson [190]'ın çalışmasında saklıdır. Ağırlıklı tepe-örtüsü algoritması Hochbaum [172] tarafından. Kısım 35.4, rastgeleleştirme ve doğrusal programlamanın yaklaşım algoritmalarının tasarımındaki gücüne sadece değinir. Bu iki düşüncenin bileşimi, "rassal yuvarlama" denilen bir teknik sağlar. Bu teknik, bir problemi bir tam sayılı doğrusal program olarak ifade eder, doğrusal-programlama gevşetmesini çözer ve çözümdeki değişkenleri olasılıklar olarak yorumlar. Bu olasılıklar daha sonra orijinal problemin çözümüne yol göstermede yardım eder. Bu teknik ilk olarak Raghavan ve Thompson [290] tarafından kullanılmıştır ve sonradan gelen birçok kullanıma sahiptir. (Bir inceleme için Motwani, Naor ve Raghavan [261] a bakınız) Yaklaşım algoritmaları alanında çeşitli diğer göze çarpan düşünceler, bölve-yönet algoritmalarında, [229], kullanmak için seyrek kesimler bulan asal-dual yöntemi (bir inceleme için Goemans ve Williamson [135] a bakınız) ve yarıtanımlı programlamanın, [134], kullanımını içerir.

Bölüm 34 ün bölüm notlarında bahsedildiği gibi, olasılıksal olarak denetlenebilir kanıtlardaki en yeni sonuçlar, bu bölümdeki birkaç problem de dahil olmak üzere birçok problemin yaklaşılabirliği üzerine alt sınırlara yol açmıştır. Oradaki kaynaklara ek olarak Arora ve Lund [23] tarafından bir bölüm olasılıksal olarak denetlenebilir kanıtlar ile çeşitli problemlere yaklaşmanın zorluğu arasındaki ilişkinin iyi bir tanımını içerir.



VIII Ek: Matematiksel Altyapı

Giriş

Algoritmaları analiz ederken, matematiksel araçların büyük bir kısmından sıklıkla yararlanmamız gerekir. Bu araçlardan bazıları lise cebiri kadar kolay iken diğerleri sizin için yeni olabilir. Kısım I de, asimptotik notasyonların nasıl çalıştığını ve özyinelemeler için nasıl çözüm bulunduğunu gördük. Bu ek, diğer çeşitli kavramların bir özeti ile algoritmaları analiz etmekte kullandığımız yöntemleri içermektedir. Kısım I e girişte kaydedildiği gibi, bu kitabı okumadan önce bu ekteki materyallerin birçoğunu görmüş olabilirsiniz (kullandığımız belirli gösterimsel uzlaşımlar buradan başka bir yerde gördüklerinizden bazen farklı olabilir). Böylece, bu eki bir referans materyal olarak ele alabilirsiniz. Bununla birlikte, bu kitabın geri kalanına, bu alandaki becerilerinizi geliştirmeniz için alıştırmalar ve problemler dâhil ettik.

Ek A, algoritma analizlerinde sıklıkla ortaya çıkan toplamaların hesaplanması ve sınırlanması için yöntemler sunar. Buradaki birçok formül herhangi bir kalkülüs ders kitabında bulunur, fakat bir yerde toplu olarak bulmak için ek A yı elverişli olarak bulacaksınız.

Ek B, temel tanımları ve kümeler, bağıntılar, fonksiyonlar, çizgeler ve ağaçlar için notasyonları içerir. Bu matematiksel nesnelerin bazı temel özelliklerini de içerir.

Ek C, saymanın basit prensipleri: permütasyon, kombinasyon vb. ile başlar. Geri kalan kısım temel olasılığın tanımlarını ve özelliklerini içerir. Bu kitapta ki algoritmaların çoğu onların analizleri için olasılık gerektirmez ve bu yüzden ilk okuma için yüzeysel olarak bakmadan bile, bölümün ikinci kısmını kolayca atlayabilirsiniz. Daha sonra, daha iyi anlamak istediğin bir olasılık analiz ile karşılaştığın zaman, Ek C yi referans amaçlı olarak iyi organize edilmiş şekilde bulacaksınız.

Ek D matrisler, matris işlemleri ve onların bazı özelliklerini tanımlar. Eğer bir lineer cebir kursu aldıysan muhtemelen şimdiye kadar bu materyallerin birçoğunu görmüşsündür, fakat bir yerde toplanmış gösterimlerimize ve tanımlarımıza bakmak için ek D yi yardımcı olarak bulabilirsin.

A Toplamlar

Bir algoritma **while** veya **for** gibi iteratif kontrol yapısı içerdği zaman, algoritmanın çalışma süresi döngünün esas kısmının her bir yürütümü üzerindeki harcanan zamanın toplamı olarak ifade edebiliriz. Örneğin, Kısım 2.2 de araya sokma sıralamasının j . iterasyonu en kötü durumda j ile orantılı zaman aldığını bulduk. Her bir özyinelemenin üstüne harcanan zamanı ekleyince toplamı (ya da serileri) elde ettik

$$\sum_{j=2}^n j .$$

Bu toplam hesaplandığında, algoritmanın en kötü durumda çalışma süresinin $\Theta(n^2)$ ile sınırlandığını elde ettik. Bu örnek toplamların nasıl sınırlandığını ve çalıştığını neden öğrenmemiz gerektiğini açıklar.

Kısım A.1 toplamlar içeren çeşitli temel formülleri listeler. Kısım A.2 sınırlı toplamlar için kullanışlı teknikler sunar. Her ne kadar Kısım A.2 deki yöntemleri örneklemek için formüllerin bazılarının kanıtları bu kısımda görülüyorsa da, Kısım A.1 deki formülleri kanıtsız olarak sunduk. Diğer kanıtların çoğunu herhangi bir kalkülüs ders kitabında bulabilirsiniz.

A.1 Toplam formülleri ve özellikleri

n bir negatif olmayan tam sayısı olmak üzere verilen bir $a_1, a_2, \dots, a_n$ sayılar dizisi için $a_1 + a_2 + \dots + a_n$ sonlu toplamını

$$\sum_{k=1}^n a_k$$

şeklinde yazabiliriz.

Eğer $n = 0$ ise toplamın değeri 0 olarak tanımlanır. Bir sonlu serinin değeri her zaman iyi tanımlıdır ve her terimini herhangi bir sıra ile eklenebilir.

Verilen bir sonsuz $a_1, a_2, \dots$ sayılar dizisi için $a_1 + a_2 + \dots$ sonsuz toplamını

$$\sum_{k=1}^{\infty} a_k$$

olarak yazabiliriz ve bunu

$$\lim_{n \rightarrow \infty} \sum_{k=1}^n a_k .$$

anlamında yorumlarız. Eğer limit mevcut değilse, seri **ıraksaktır**; aksi takdirde **yakınsaktır**. $\sum_{k=1}^{\infty} a_k$ serisi yakınsak ise $\sum_{k=1}^{\infty} |a_k|$ **mutlak yakınsak seri** olarak yeniden düzenleyebiliriz.

Doğrusallık

Herhangi bir c reel sayısı ve her sonlu $a_1, a_2, \dots, a_n$ ve $b_1, b_2, \dots, b_n$, dizileri için

$$\sum_{k=1}^n (ca_k + b_k) = c \sum_{k=1}^n a_k + \sum_{k=1}^n b_k$$

Doğrusallık özelliği sonsuz yakınsak serilere de uygulanır.

Doğrusallık özelliğini birleşik asimptotik notasyonların toplamaları çalıştırmak için faydalanabiliriz. Örneğin,

$$\sum_{k=1}^n \Theta(f(k)) = \Theta\left(\sum_{k=1}^n f(k)\right)$$

dir.

Bu denklemde, sol taraftaki Θ -notasyonu k değişkenine uygular, fakat sağ taraftaki n ye uygular. Böyle çalıştırmaları sonsuz yakınsak serilere de uygulayabiliriz.

Aritmetik seriler

$$\sum_{k=1}^n k = 1 + 2 + \dots + n \text{ toplamı bir } \textbf{aritmetik seridir} \text{ ve}$$

$$\sum_{k=1}^n k = \frac{1}{2}n(n+1) \tag{A.1}$$

$$= \Theta(n^2) \tag{A.2}$$

değerine sahiptir.

Karelerin ve küplerin toplamı

$$\sum_{k=0}^n k^2 = \frac{n(n+1)(2n+1)}{6}, \quad (\text{A.3})$$

$$\sum_{k=0}^n k^3 = \frac{n^2(n+1)^2}{4}. \quad (\text{A.4})$$

Geometrik seriler

$x \neq 1$ reel sayısı için,

$$\sum_{k=0}^n x^k = 1 + x + x^2 + \cdots + x^n \text{ toplamı}$$

bir *geometrik* veya *üstel* seridir ve

$$\sum_{k=0}^n x^k = \frac{x^{n+1} - 1}{x - 1}, \quad (\text{A.5})$$

değerine sahiptir.

Toplam sonsuz ve $|x| < 1$ olduğu zaman, sonsuz azalan geometrik serisi olan

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1 - x} \quad (\text{A.6})$$

sahip oluruz.

Harmonik seriler

n pozitif tam sayıları için n . *harmonik sayısı*

$$\begin{aligned} H_n &= 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \cdots + \frac{1}{n} \\ &= \sum_{k=1}^n \frac{1}{k} \\ &= \ln n + O(1) \end{aligned} \quad (\text{A.7})$$

dir. (İlgili sınırı Kısım A.2 de kanıtlayacağız).

Serilerin toplamı ve farkı

Ek formüller yukarıdaki formülleri toplayarak ya da farkını alarak meydana gelir. Örneğin (A.6) sonsuz geometrik serisinin her iki tarafının farkını alarak ve x ile çarparak,

$$\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2} \quad (\text{A.8})$$

$|x| < 1$ için elde ettik.

İç içe geçen seriler

Herhangi bir $a_0, a_1, \dots, a_n$, dizisi için

$$\sum_{k=1}^n (a_k - a_{k-1}) = a_n - a_0 \quad (\text{A.9})$$

olur.

Çünkü $a_1, a_2, \dots, a_{n-1}$ dizisinin her bir terimi tam olarak bir defa eklenir ve tam olarak bir defa çıkarılır. Buna **İç içe geçenlerin** toplamı deriz. Benzer olarak,

$$\sum_{k=0}^{n-1} (a_k - a_{k+1}) = a_0 - a_n .$$

İç içe geçen toplama bir örnek olarak,

$$\sum_{k=1}^{n-1} \frac{1}{k(k+1)} \text{ serisini gözden önünde bulunduralım. Her bir terimi}$$

$$\frac{1}{k(k+1)} = \frac{1}{k} - \frac{1}{k+1} ,$$

olarak yeniden yazabileceğimiz için

$$\begin{aligned} \sum_{k=1}^{n-1} \frac{1}{k(k+1)} &= \sum_{k=1}^{n-1} \left(\frac{1}{k} - \frac{1}{k+1} \right) \\ &= 1 - \frac{1}{n} \end{aligned}$$

elde ederiz.

Çarpımlar

$a_1 a_2 \cdots a_n$ sonlu çarpımını

$$\prod_{k=1}^n a_k$$

olarak yazabiliriz.

Eğer $n = 0$ ise, çarpımın değeri 1 olarak tanımlanır. Bir çarpım formülünü bir toplam formülüne

$$\lg \left(\prod_{k=1}^n a_k \right) = \sum_{k=1}^n \lg a_k$$

özdeşliğini kullanarak dönüştürebiliriz.

Alıştırmalar

A.1-1

$\sum_{k=1}^n (2k - 1)$ için basit bir formül bulunuz.

A.1-2 *

Gösteriniz ki harmonik seriler çalıştırılarak $\sum_{k=1}^n 1/(2k - 1) = \ln(\sqrt{n}) + O(1)$ olur.

A.1-3

Gösteriniz ki $\sum_{k=0}^{\infty} k^2 x^k = x(1 + x)/(1 - x)^3$ for $|x| < 1$.

A.1-4 *

Gösteriniz ki $\sum_{k=0}^{\infty} (k - 1)/2^k = 0$.

A.1-5 *

$\sum_{k=1}^{\infty} (2k + 1)x^{2k}$ toplamını hesaplayınız.

A.1-6

Toplamların doğrusallık özelliğini kullanarak $\sum_{k=1}^n O(f_k(i)) = O(\sum_{k=1}^n f_k(i))$ kanıtlayınız.

A.1-7

$\prod_{k=1}^n 2 \cdot 4^k$ çarpımını hesaplayınız.

A.1-8 *

$\prod_{k=2}^n (1 - 1/k^2)$ çarpımını hesaplayınız.

A.2 Toplamların sınırlanışı

Algoritmaların çalışma süresini tanımlayan sınırlı toplamlar için hizmetimizde olan birçok tekniğe sahibiz. En sık kullanılan yöntemlerin çoğunluğu buradadır.

Matematiksel tümevarım

Bir seriyi hesaplamak için en basit yol matematiksel tümevarımı kullanmaktır. Örneğin, $\sum_{k=1}^n k$ aritmetik serisinin $\frac{1}{2}n(n + 1)$ olduğunu kanıtlayalım. İddiayı $n = 1$ için kolaylıkla denetleyebiliriz. Tümevarım varsayımının n için sağlanır

şeklinde yaparız ve $n + 1$ için sağlandığını kanıtlarız.

$$\begin{aligned}\sum_{k=1}^{n+1} k &= \sum_{k=1}^n k + (n + 1) \\ &= \frac{1}{2}n(n + 1) + (n + 1) \\ &= \frac{1}{2}(n + 1)(n + 2)\end{aligned}$$

elde ettik.

Matematiksel tümevarımı her zaman bir toplamın tam değerini tahmin etmek için kullanmak zorunda değiliz. Bunun yerine, bir toplam üzerindeki sınırı kanıtlamak için tümevarımı kullanabiliriz. Örneğin, $\sum_{k=0}^n 3^k$ geometrik serisinin $O(3^n)$ olduğunu kanıtlayalım. Daha özel olarak, bazı c sabitleri için $\sum_{k=0}^n 3^k \leq c3^n$ olduğunu kanıtlayalım. $n = 0$ başlangıç koşulu için $\sum_{k=0}^0 3^k = 1 \leq c \cdot 1$ olduğu sürece $c \geq 1$ eşitsizliğine sahibiz. n için sınırın sağladığını kabul edelim ve $n + 1$ için sağlandığını kanıtlayalım.

$$\begin{aligned}\sum_{k=0}^{n+1} 3^k &= \sum_{k=0}^n 3^k + 3^{n+1} \\ &\leq c3^n + 3^{n+1} \quad (\text{tümevarım hipoteziyle}) \\ &= \left(\frac{1}{3} + \frac{1}{c}\right) c3^{n+1} \\ &\leq c3^{n+1}\end{aligned}$$

eşitsizliğini $(1/3 + 1/c) \leq 1$ olduğu sürece veya denk olarak $c \geq 3/2$ sağlandığı sürece elde ederiz. Böylece $\sum_{k=0}^n 3^k = O(3^n)$ göstermek istediğimizdir.

Tümevarım ile sınırları kanıtlamak için asimptotik sembolü kullandığımız zaman dikkatli olmak zorundayız. $\sum_{k=1}^n k = O(n)$ olan aşağıdaki yanıltıcı kanıt gözden geçirelim. Elbette, $\sum_{k=1}^1 k = O(1)$ eşitliği vardır. Kabul edelim ki n için sınır sağlansın ve $n + 1$ için sağlandığını kanıtlayalım;

$$\begin{aligned}\sum_{k=1}^{n+1} k &= \sum_{k=1}^n k + (n + 1) \\ &= O(n) + (n + 1) \quad \Leftarrow \text{hatalı!!} \\ &= O(n + 1) .\end{aligned}$$

Argümandaki hata n ile orantılı artan büyük-O tarafından “sabit” in gizlenmesidir ve böylece sabit değildir. Biz aynı sabitin her değeri için çalıştığını göstermedik.

Terimlerin sınırlanışı

Bazen serilerin her bir terimini sınırlayarak o seri üzerinde iyi bir üst sınır elde edebiliriz ve diğerlerini sınırlamak için en büyük terimi kullanmak sıklıkla yeterli

olur. Örneğin, (A.1) aritmetik serisindeki üst sınır

$$\begin{aligned}\sum_{k=1}^n k &\leq \sum_{k=1}^n n \\ &= n^2\end{aligned}$$

olur.

Genelde, bir $\sum_{k=1}^n a_k$ serisi için eğer $a_{\max} = \max_{1 \leq k \leq n} a_k$ olarak alırsak, o zaman

$$\sum_{k=1}^n a_k \leq n \cdot a_{\max}$$

olur.

En büyük terim ile serideki her bir terimi sınırlayan teknik, seri gerçekten bir geometrik seri tarafından sınırlandırılabilirse zayıf bir yöntemdir. Kabul edelim ki $0 < r < 1$ ve $a_{k+1}/a_k \leq r$ iken her $k \geq 0$ için $\sum_{k=0}^n a_k$ serisi sabittir. $a_k \leq a_0 r^k$ için toplamı bir sonsuz azalan geometrik seri ile sınırlayabiliriz ve böylece

$$\begin{aligned}\sum_{k=0}^n a_k &\leq \sum_{k=0}^{\infty} a_0 r^k \\ &= a_0 \sum_{k=0}^{\infty} r^k \\ &= a_0 \frac{1}{1-r}\end{aligned}$$

olur.

Bu yöntemi $\sum_{k=1}^{\infty} (k/3^k)$ toplamını sınırlamak için kullanabiliriz. $k = 0$ da toplama başlamak için $\sum_{k=0}^{\infty} ((k+1)/3^{k+1})$ olarak yeniden yazarız. İlk terim olan (a_0) $1/3$ tür ve ardışık terimlerin (r) oranı her $k \geq 0$ için

$$\begin{aligned}\frac{(k+2)/3^{k+2}}{(k+1)/3^{k+1}} &= \frac{1}{3} \cdot \frac{k+2}{k+1} \\ &\leq \frac{2}{3}\end{aligned}$$

olur. Böylece

$$\begin{aligned}\sum_{k=1}^{\infty} \frac{k}{3^k} &= \sum_{k=0}^{\infty} \frac{k+1}{3^{k+1}} \\ &\leq \frac{1}{3} \cdot \frac{1}{1-2/3} \\ &= 1\end{aligned}$$

elde ettik.

Bu yöntemi uygularken yapılan yaygın hata ardışık terimlerin oranının 1 den az olduğunu göstermek ve sonra toplamın bir geometrik seri ile sınırlı olduğunu kabul etmektir. Buna bir örnek sonsuz harmonik seridir ve

$$\begin{aligned}\sum_{k=1}^{\infty} \frac{1}{k} &= \lim_{n \rightarrow \infty} \sum_{k=1}^n \frac{1}{k} \\ &= \lim_{n \rightarrow \infty} \Theta(\lg n) \\ &= \infty\end{aligned}$$

olduğu için iraksar.

Bu serideki $(k+1)$.ci ve k .ci terimin oranı $k/(k+1) < 1$ dir. Fakat seri bir azalan geometrik seri ile sınırlı değildir. Bir seriyi bir geometrik seri ile sınırlamak için öyle bir sabit $r < 1$ vardır ki tüm ardışık terim çiftlerinin oranının r yi aşmadığını göstermeliyiz. Oran rassal olarak 1 e yakın olduğu için harmonik seride böyle bir r değeri yoktur.

Toplamların ayrılması

Zorlu bir toplamın sınırlarını elde etmenin yollarından biri seriyi, indisin tanım kümesini iki ya da daha fazla serinin toplamına parçalayarak ifade etmek ve daha sonra oluşan bu serileri sınırlamaktır. Örneğin, $\sum_{k=1}^n k$ aritmetik serisi üzerinde bir alt sınır bulmaya çalışacağımızı varsayalım, zaten n^2 nin bu seri için bir üst sınır olduğunu görmüştük. En küçük terim ile toplamdaki her bir terimi sınırlamayla deneyebiliriz fakat terim 1 olduğundan toplam için n^2 nin bir üst sınırından çok farklı olan, n nin bir alt sınırını elde ederiz.

İlk toplamı ayırarak daha iyi bir alt sınır elde edebildik. Uygun olarak n nin çift olduğunu varsayalım. $\sum_{k=1}^n k = O(n^2)$ olduğu için

$$\begin{aligned}\sum_{k=1}^n k &= \sum_{k=1}^{n/2} k + \sum_{k=n/2+1}^n k \\ &\geq \sum_{k=1}^{n/2} 0 + \sum_{k=n/2+1}^n (n/2) \\ &= (n/2)^2 \\ &= \Omega(n^2)\end{aligned}$$

asimptotik sıkı sınıra sahibiz.

Bir algoritmanın analizden ortaya çıkan bir toplam için çoğu zaman toplam-ları ayırabilir ve başlangıç terimlerinin bir sabit sayısını görmezden gelebiliriz. Genel olarak, a_k toplamındaki her bir $\sum_{k=0}^n a_k$ terimi n den bağımsız olduğunda

bu tekniği uyguluyoruz. Sonra her $k_0 > 0$ sabiti için

$$\begin{aligned} \sum_{k=0}^n a_k &= \sum_{k=0}^{k_0-1} a_k + \sum_{k=k_0}^n a_k \\ &= \Theta(1) + \sum_{k=k_0}^n a_k \end{aligned}$$

toplamanın başlangıç terimlerinin tümü sabit ve onların sayısı da bir sabit olduğundan yukarıdaki gibi yazabiliriz. $\sum_{k=k_0}^n a_k$ sınırlamak için diğer yöntemleri kullanabiliriz. Bu teknik sonsuz toplamlara da uygulanır. Örneğin,

$$\sum_{k=0}^{\infty} \frac{k^2}{2^k}$$

üzerinde bir asimptotik üst sınır bulmak için eğer $k \geq 3$ ise ardışık terimlerinin oranının

$$\begin{aligned} \frac{(k+1)^2/2^{k+1}}{k^2/2^k} &= \frac{(k+1)^2}{2k^2} \\ &\leq \frac{8}{9} \end{aligned}$$

olduğunu gözlemleriz.

Böylece, toplam şu şekilde ayrılabilir:

$$\begin{aligned} \sum_{k=0}^{\infty} \frac{k^2}{2^k} &= \sum_{k=0}^2 \frac{k^2}{2^k} + \sum_{k=3}^{\infty} \frac{k^2}{2^k} \\ &\leq \sum_{k=0}^2 \frac{k^2}{2^k} + \frac{9}{8} \sum_{k=0}^{\infty} \left(\frac{8}{9}\right)^k \\ &= O(1) \end{aligned}$$

dir çünkü ilk toplam terimleri sabit bir sayıya sahiptir ve ikinci toplam bir azalan geometrik seridir.

Toplamların ayrılması tekniği çok zorlu durumlar için asimptotik sınırları saptamak için yardımcı olabilir. Örneğin, (A.7) harmonik serisinin bir sınırını $O(\lg n)$ elde edebiliriz:

$$H_n = \sum_{k=1}^n \frac{1}{k}.$$

$\lfloor \lg n \rfloor + 1$ parçalarındaki 1 den n ye kadar değer kümesinin parçalayarak ve her bir parça 1 i geçecek şekilde üstten sınırlayarak yaparız. $i = 0, 1, \dots, \lfloor \lg n \rfloor$ için

i . parça $1/2^i$ da başlayarak terimleri içerir fakat $1/2^{i+1}$ içermez. Son parça orijinal harmonik seri içinde olmayan terimleri içerebilir ve böylece

$$\begin{aligned}
 \sum_{k=1}^n \frac{1}{k} &\leq \sum_{i=0}^{\lfloor \lg n \rfloor} \sum_{j=0}^{2^i-1} \frac{1}{2^i + j} \\
 &\leq \sum_{i=0}^{\lfloor \lg n \rfloor} \sum_{j=0}^{2^i-1} \frac{1}{2^i} \\
 &= \sum_{i=0}^{\lfloor \lg n \rfloor} 1 \\
 &\leq \lg n + 1
 \end{aligned} \tag{A.10}$$

sahip oluruz.

İntegraller ile yaklaşım

$f(k)$ monoton artan bir fonksiyonken bir toplam $\sum_{k=m}^n f(k)$ formunda olduğu zaman integraller ile yaklaşımda bulunabiliriz:

$$\int_{m-1}^n f(x) dx \leq \sum_{k=m}^n f(k) \leq \int_m^{n+1} f(x) dx . \tag{A.11}$$

Şekil A.1 bu yaklaşımı gerçekler. Toplam, şekildeki dikdörtgenlerin alanı olarak sunulur ve integral eğri altında kalan gölgeli bölgedir. $f(k)$ bir monoton azalan fonksiyon olduğunda,

$$\int_m^{n+1} f(x) dx \leq \sum_{k=m}^n f(k) \leq \int_{m-1}^n f(x) dx . \tag{A.12}$$

Sınırlarını sağlamak için benzer bir yöntem kullanabiliriz.

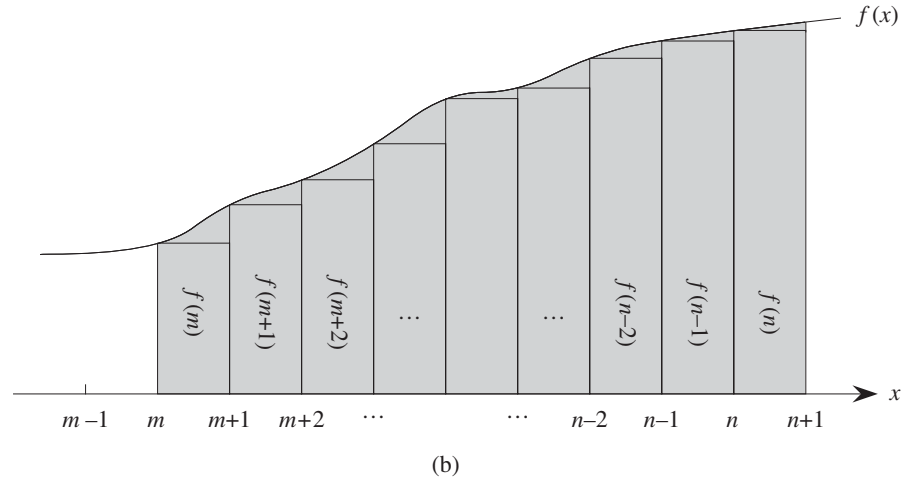
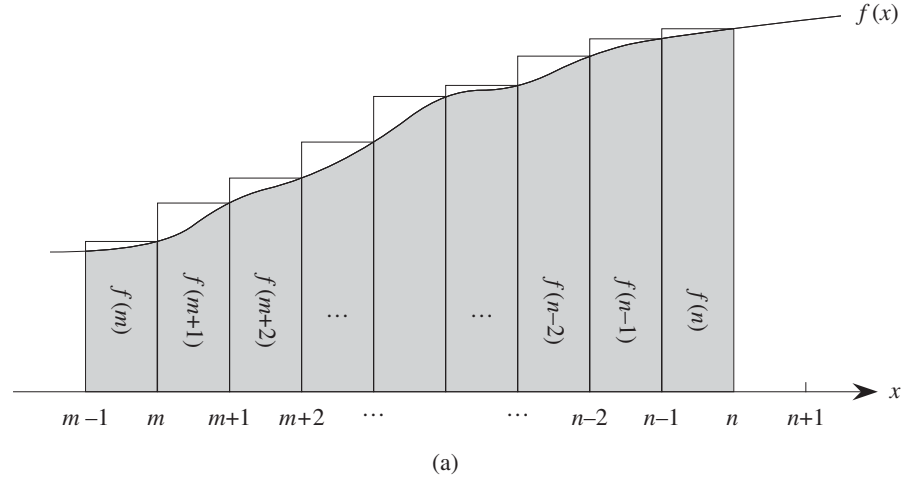
(A.12) integral yaklaşımı, n .harmonik sayısı için bir sıkı tahmin verir. Bir alt sınır için

$$\begin{aligned}
 \sum_{k=1}^n \frac{1}{k} &\geq \int_1^{n+1} \frac{dx}{x} \\
 &= \ln(n+1)
 \end{aligned} \tag{A.13}$$

elde ettik.

Bir üst sınır için aşağıdaki eşitsizliğin türevini alıp

$$\begin{aligned}
 \sum_{k=2}^n \frac{1}{k} &\leq \int_1^n \frac{dx}{x} \\
 &= \ln n
 \end{aligned}$$



Şekil A.1 İntegrallerle $\sum_{k=m}^n f(k)$ yaklaşım. Her bir dikdörtgenin alanı dikdörtgen içerisinde gösterilir ve tüm dikdörtgenin alanı toplamın alanının değeri olarak sunulur. İntegral eğri altında kalan gölgeli bölge olarak sunulur. **(a)** bölgeleri karşılaştırarak, $\int_{m-1}^n f(x) dx \leq \sum_{k=m}^n f(k)$ elde ederiz ve sonrada dikdörtgenleri bir birim sağa kaydırarak **(b)** deki $\sum_{k=m}^n f(k) \leq \int_m^{n+1} f(x) dx$ elde ederiz.

aşağıdaki sınırı ortaya çıkarınız

$$\sum_{k=1}^n \frac{1}{k} \leq \ln n + 1 \quad (\text{A.14})$$

Alıştırmalar

A.2-1

$\sum_{k=1}^n 1/k^2$ yukarıdaki sabit ile sınırlıdır, gösteriniz.

A.2-2

$\sum_{k=0}^{\lfloor \lg n \rfloor} \lceil n/2^k \rceil$ toplamı üzerinde bir asimptotik üst sınır bulunuz.

A.2-3

Ayrılan toplamların n . harmonik sayısının $\Omega(\lg n)$ olduğunu gösteriniz.

A.2-4

$\sum_{k=1}^n k^3$ toplamın bir integral ile yaklaşım yapınız.

A.2-5

Neden (A.12) integral yaklaşımını doğrudan $\sum_{k=1}^n 1/k$ üzerinde n . harmonik sayısı üzerindeki bir üst sınırı elde etmek için kullanmadık?

Problemler

A-1 Toplamların sınırlanışı

Aşağıdaki toplamlar için asipmtotik sıkı sınırlar veriniz. $r \geq 0$ ve $s \geq 0$ sabit olduğunu kabul ediniz.

a. $\sum_{k=1}^n k^r.$

b. $\sum_{k=1}^n \lg^s k$

$$c. \sum_{k=1}^n k^r \lg^s k.$$

Ek notlar

Knuth [209] burada sunulan materyal için mükemmel bir referans sağlar. Serilerin temel özelliklerini Apostol [18] veya Thomas et al. [334] gibi herhangi bir iyi kalkülüs kitabında bulabilirsiniz.

B Kümeler, vs.

Bu kitabın birçok bölümü ayrık matematiğin unsurlarından bahseder. Bu ek, kümelerin, bağıntıların, fonksiyonların, çizgelerin ve ağaçların temel özelliklerini, gösterimlerini ve tanımlarını gözden geçirir. Hali hazırda bu materyal konusunda tecrübeli iseniz, bu bölümde sadece göz gezdirebilirsiniz.

B.1 Kümeler

Bir **küme**, **eleman** veya **üye** denilen ayrık objelerin bir koleksiyonudur. Eğer bir x objesi bir S kümesinin bir elemanı ise, $x \in S$ yazarız (“ x , S nin bir elemanıdır” veya kısaca “ x , S dedir” şeklinde okuruz). Eğer x , S nin bir elemanı değil ise $x \notin S$ yazarız. Bir kümeyi elemanları açıkça listeli parantezlerin içinde listelenmiş olarak ifade edebiliriz. Örneğin, $S = \{1, 2, 3\}$ yazarak 1, 2, ve 3 sayılarını içeren bir S kümesi tanımlayabiliriz. 2, S nin bir elemanı olduğu için $2 \in S$ yazabiliriz ve 4, S nin bir elemanı olmadığı için $4 \notin S$ dir. Bir küme aynı objeyi bir defadan daha fazla içeremez¹ ve elemanlar sıralı değildir. İki A ve B kümesi aynı elemanlara sahip ise bu kümeler **eşittir** ve $A = B$ yazılır. Örneğin, $\{1, 2, 3, 1\} = \{1, 2, 3\} = \{3, 2, 1\}$.

Sık karşılaşılan kümeler için şu gösterimleri benimseyeceğiz:

- $\emptyset$ **boş küme** anlamında, hiçbir eleman içermeyen küme.
- $\mathbb{Z}$ **tam sayılar** kümesi anlamında, $\{\dots, -2, -1, 0, 1, 2, \dots\}$ kümesi.
- $\mathbb{R}$ **reel sayılar** kümesi.
- $\mathbb{N}$ **doğal sayılar** kümesi anlamında, $\{0, 1, 2, \dots\}$ ² kümesi.

¹Aynı objeyi birden fazla defa içeren bir küme varyasyonuna çoklu küme denir.

²Bazı yazarlar doğal sayılara 0 yerine 1 ile başlarlar. Modern eğilim 0 ile başlamasıdır.

Eğer bir A kümesinin tüm elemanları bir B kümesi tarafından içeriliyorsa buna $x \in A$ gerektirir $x \in B$ ise, $A \subseteq B$ yazarız ve A, B nin bir **altkümesidir** deriz. Eğer $A \subset B$ fakat $A \subseteq B$ ise A kümesi B nin **öz altkümesidir** ve $A \neq B$ yazılır. (Bazı yazarlar " $\subseteq$ " sembolünü öz altküme bağıntısından ziyade alışlagelmiş alt küme bağıntısını ifade etmek için kullanırlar.) Her A kümesi için $A \subseteq A$ vardır. Her A, B, C kümeleri için eğer $A = B$ ve $B \subseteq A$ ise o zaman $A \subseteq C$ olur. Her A kümesi için $\emptyset \subseteq A$ vardır.

Bazen kümeleri diğer kümeler üzerinden tanımlarız. Verilen bir A kümesi için, $B \subseteq A$ ü, B nin elemanlarından ayıran bir özelliği belirterek tanımlayabiliriz. Örneğin, çift tam sayılar kümesini $\{x : x \in \mathbb{Z} \text{ and } x/2 \text{ is an integer}\}$ ile tanımlayabiliriz. Buradaki iki nokta üst üste işareti "öyle ki" olarak okunur (Bazı yazarlar iki nokta üst üste işaretinin yerine dikey bir çubuk işareti kullanırlar).

Verilen iki A ve B kümesine **küme işlemleri** uygulayarak yeni kümeler tanımlayabiliriz:

- A ve B kümelerinin **kesişimi**

$$A \cap B = \{x : x \in A \text{ ve } x \in B\}$$
- A ve B kümelerinin **birleşimi**

$$A \cup B = \{x : x \in A \text{ ve } x \in B\}$$
- A ve B kümelerinin **farkı**

$$A - B = \{x : x \in A \text{ veya } x \notin B\}$$

Küme işlemleri aşağıdaki kurallara uyar:

Boş küme kuralları:

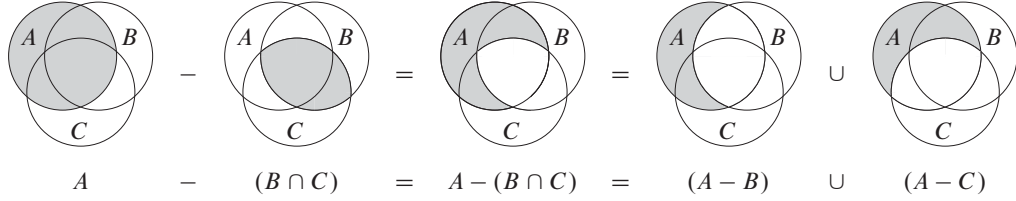
$$\begin{aligned} A \cap \emptyset &= \emptyset, \\ A \cup \emptyset &= A. \end{aligned}$$

Eşgüçlülük kuralları:

$$\begin{aligned} A \cap A &= A, \\ A \cup A &= A. \end{aligned}$$

Değişme kuralları:

$$\begin{aligned} A \cap B &= B \cap A, \\ A \cup B &= B \cup A. \end{aligned}$$



Şekil B.1 (B.2) deki DeMorgan kurallarından ilkinin resimleyen bir Venn diyagramı. A , B ve C kümelerinin her biri bir çember olarak temsil ediliyor.

Birleşme kuralları:

$$\begin{aligned} A \cap (B \cap C) &= (A \cap B) \cap C, \\ A \cup (B \cup C) &= (A \cup B) \cup C. \end{aligned}$$

Dağılma kuralları:

$$\begin{aligned} A \cap (B \cup C) &= (A \cap B) \cup (A \cap C), \\ A \cup (B \cap C) &= (A \cup B) \cap (A \cup C). \end{aligned} \tag{B.1}$$

Yutma kuralları:

$$\begin{aligned} A \cap (A \cup B) &= A, \\ A \cup (A \cap B) &= A. \end{aligned}$$

DeMorgan Kuralları:

$$\begin{aligned} A - (B \cap C) &= (A - B) \cup (A - C), \\ A - (B \cup C) &= (A - B) \cap (A - C). \end{aligned} \tag{B.2}$$

Şekil B.1, DeMorgan kurallarının ilki bir **Venn diyagramı** nı kullanarak örnekler: kümelerdeki grafiksel resim düzlemin bölgeleri olarak temsil ediliyorlar.

Göz önünde tutulan tüm kümeler **evren** olarak isimlendirilen daha büyük U kümesinin alt kümeleridir. Örneğin, sadece tam sayılardan oluşan çeşitli kümeleri göz önünde bulundurursak, tam sayılar kümesi $\mathbb{Z}$ kendine özgü bir evrendir. Verilen bir U evreni için bir A kümesinin **tümleyeni** $\overline{A} = U - A = \{x : x \in U \text{ ve } x \notin A\}$ olarak tanımlanır. Her A kümesi için aşağıdaki kurullar vardır:

$$\begin{aligned} \overline{\overline{A}} &= A, \\ A \cap \overline{A} &= \emptyset, \\ A \cup \overline{A} &= U. \end{aligned}$$

(B.2) DeMorgan kurallarını tümleyenler ile tekrar yazabiliriz. Verilen her iki $B, C \subseteq U$ kümesi için

$$\begin{aligned}\overline{B \cap C} &= \overline{B} \cup \overline{C} \\ \overline{B \cup C} &= \overline{B} \cap \overline{C} .\end{aligned}$$

dir.

Eğer $A \cap B = \emptyset$ yani verilen A ve B kümeleri hiçbir ortak elemana sahip değil ise bu iki kümeye **ayrık** kümeler denir. Boş olmayan kümelerin bir $\mathcal{S} = \{S_i\}$ koleksiyonu bir S kümesinin bir **parçalanışını** oluşturur eğer

- Kümeler **çiftler haline ayrık** yani $S_i, S_j \in \mathcal{S}$ ve $i \neq j$ gerektirir $S_i \cap S_j = \emptyset$ ve
- Onların birleşimleri S dir. yani

$$S = \bigcup_{S_i \in \mathcal{S}} S_i$$

dir.

Diğer bir deyişle S nin her bir elemanı tam olarak bir $\mathcal{S}$ ise $S_i \in \mathcal{S}$, S nin bir parçalanışını oluşturur.

Bir kümedeki elemanların sayısı o kümenin **kardinalitesidir** (veya **büyüklüğüdür**) ve $|S|$ ile gösterilir. İki küme aynı kardinaliteye sahip ise elemanları birebir eşlenebilir. Boş kümenin kardinalitesi $|\emptyset| = 0$. Bir kümenin kardinalitesi bir doğal sayısı ise kümeye **sonlu** aksi takdirde **sonsuz** deriz. $\mathbb{N}$ doğal sayılar kümesi ile birebir eşlenebilen kümeye **sayılabılır sonsuz** aksi takdirde **sayılamaz** deriz. Örneğin, $\mathbb{Z}$ tam sayılar kümesi sayılabilir fakat $\mathbb{R}$ reel sayılar kümesi sayılamazdır.

Verilen herhangi iki A ve B kümesi için aşağıdaki özdeşlik vardır

$$|A \cup B| = |A| + |B| - |A \cap B| . \quad (\text{B.3})$$

Buradan aşağıdaki eşitsizliği elde ederiz

$$|A \cup B| \leq |A| + |B| .$$

Eğer A ve B ayrık ise $|A \cap B| = 0$ olur ve böylece $|A \cup B| = |A| + |B|$ dir. Eğer $A \subseteq B$ ise o zaman $|A| \leq |B|$ olur.

n elemanlı bir sonlu kümeye bazen **n -küme** denir. Bir 1-küme ye **tekil** denir. Bir kümenin k elemanlı bir altkümesine bir **k -altküme** denir.

Boş kümeyi ve kendisini içeren bir S kümesinin tüm altkümelerinin kümesi 2^S ile gösterilir; 2^S S nin **kuvvet kümesidir** denir. Örneğin, $2^{\{a,b\}} = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}$. Bir sonlu S kümesinin altkümeleri $2^{|S|}$ kardinalitesine sahiptir (Alıştırma B.1-5 e bakınız).

Bazen elemanları sıralı olan küme benzeri yapılarla ilgileniriz. a ve b elemanlarının bir **sıralı çifti** (a, b) ile gösterilir ve formal olarak $(a, b) = \{a, \{a, b\}\}$ olarak ifade edilir. Böylece, (a, b) sıralı çifti (b, a) sıralı çifti ile aynı değildir.

A ve B gibi iki kümenin **Kartezyen çarpımı**, $A \times B$ ile gösterilir ve ilk elemanı A nın bir elemanı ve ikinci elemanı B nin bir elemanı olacak şekilde sıralı ikililer kümesidir. Daha formal olarak,

$$A \times B = \{(a, b) : a \in A \text{ ve } b \in B\} .$$

Örneğin, $\{a, b\} \times \{a, b, c\} = \{(a, a), (a, b), (a, c), (b, a), (b, b), (b, c)\}$. A ve B sonlu kümeler olduğunda onların Kartezyen çarpımlarının kardinalitesi,

$$|A \times B| = |A| \cdot |B| \quad (\text{B.4})$$

dir.

$A_1, A_2, \dots, A_n$ n adet kümenin Kartezyen çarpımı **n -katlı** kümedir yani

$$A_1 \times A_2 \times \dots \times A_n = \{(a_1, a_2, \dots, a_n) : a_i \in A_i \text{ için } i = 1, 2, \dots, n\}$$

dir ve kardinalitesi, tüm kümeler sonlu ise

$$|A_1 \times A_2 \times \dots \times A_n| = |A_1| \cdot |A_2| \cdots |A_n|$$

olur.

Bir A kümesi üzerindeki n -katlı Kartezyen çarpımı

$$A^n = A \times A \times \dots \times A$$

kümesi ile gösteririz ve kardinalitesi eğer A sonlu ise $|A^n| = |A|^n$ olur. n uzunluklu bir sonlu diziyi n -katlı olarak da görebiliriz (sayfa 1166 ya bakınız).

Alıştırmalar

B.1-1

(B.1) dağılma kuralının ilkinin örnekleyen Venn diyagramını çizin.

B.1-2

Aşağıda görülen kümelerin her sonlu koleksiyonu için DeMorgan kurallarının genellemesini kanıtlayınız.

$$\begin{aligned} \overline{A_1 \cap A_2 \cap \dots \cap A_n} &= \overline{A_1} \cup \overline{A_2} \cup \dots \cup \overline{A_n} , \\ \overline{A_1 \cup A_2 \cup \dots \cup A_n} &= \overline{A_1} \cap \overline{A_2} \cap \dots \cap \overline{A_n} . \end{aligned}$$

B.1-3 *

İçerleme ve dışlama prensibi olarak adlandırılan (B.3) eşitliğinin genelleştirmesini kanıtlayınız:

$$\begin{aligned}
 |A_1 \cup A_2 \cup \dots \cup A_n| = & \\
 & |A_1| + |A_2| + \dots + |A_n| \\
 & - |A_1 \cap A_2| - |A_1 \cap A_3| - \dots \quad (\text{tüm ikililer}) \\
 & + |A_1 \cap A_2 \cap A_3| + \dots \quad (\text{tüm üçlüler}) \\
 & \vdots \\
 & + (-1)^{n-1} |A_1 \cap A_2 \cap \dots \cap A_n| .
 \end{aligned}$$

B.1-4

Çift doğal sayılar kümesinin sayılabilir olduğunu gösteriniz.

B.1-5

Gösteriniz ki, her sonlu S kümesi için 2^S kuvvet kümesi $2^{|S|}$ elemana sahiptir (yani S nin $2^{|S|}$ ayrık altkümesi vardır).

B.1-6

Bir n -katlı için bir sıralı çift için küme teorik tanım ile genişletilen bir tümevarımsal tanım veriniz.

B.2 Bağıntılar

A ve B gibi iki küme üzerindeki bir **ikili bağıntı** $A \times B$ Kartezyen çarpımının bir altkümesidir. Eğer $(a, b) \in R$ ise bazen $a R b$ yazarız. R, A kümesi üzerinde bir ikili bağıntıdır dediğimizde, $R A \times A$ nın bir alt kümesidir anlamındadır. Örneğin, doğal sayılar üzerindeki “küçüktür” bağıntısı $\{(a, b) : a, b \in \mathbb{N} \text{ ve } a < b\}$ kümesidir. $A_1, A_2, \dots, A_n$ kümeleri üzerindeki n -ariteli bağıntı $A_1 \times A_2 \times \dots \times A_n$ kümesinin bir altkümesidir.

Eğer her $a \in A$ için $a R a$ ise, o zaman $R \subseteq A \times A$ ikili bağıntısı **yansıyandır**.

Örneğin, “=” “≤” ve $\mathbb{N}$ doğal sayılar kümesi üzerinde yansıyan bağıntılardır fakat “<” değildir. Eğer her $a, b \in A$ için $a R b$ gerektirir $b R a$ ise, o zaman R bağıntısı **simetrik**dir.

Örneğin, “=” simetrik fakat “<” ve “≤” değildir.

Her $a, b, c \in A$ için, R ilişkisi eğer $a R b$ ve $b R c$ gerektirir $a R c$ ise ikili bağıntı **geçişkendir**.

Örneğin, “<,” “≤,” ve “=” bağıntıları geçişken fakat $R = \{(a, b) : a, b \in \mathbb{N} \text{ ve } a = b - 1\}$ ve $3 R 4$ ve $4 R 5$, $3 R 5$ yi gerektirmediğinden bağıntı değildir.

Eğer bir bağıntı yansıyan, simetrik ve geçişken ise bir **denklik bağıntısıdır**. Örneğin, “=” bağıntısı doğal sayılar üzerinde bir denklik bağıntısıdır fakat “<” değildir. Eğer R bağıntısı bir A kümesi üzerinde denklik bağıntısı ise $a \in A$ için a nın **denklik sınıfı** $[a] = \{b \in A : a R b\}$ kümesidir ve bu kümenin tüm elemanları a ya denktir. Örneğin, $R = \{(a, b) : a, b \in \mathbb{N} \text{ tanımlarsak o zaman } a + a \text{ çift (yansıyan), } a + b \text{ çift gerektirir } b + a \text{ çift (simetrik), ve } a + b \text{ çift ve } b + c \text{ çift gerektirir } a + c \text{ çift (geçişken) olduğu için } R \text{ bir denklik bağıntısıdır. 4 ün denklik sınıfı denklik sınıfı } [4] = \{0, 2, 4, 6, \dots\} \text{ dir ve 3 ün denklik sınıfı } [3] = \{1, 3, 5, 7, \dots\} \text{ dir. Denklik sınıflarının bir temel teoremi aşağıdadır.}$

Teorem B.1 (Bir denklik bağıntısı bir parçalanışın aynısıdır)

A nın bir parçalanışından A kümesi üzerindeki herhangi bir R denklik bağıntısının denklik sınıfları ve A nın herhangi bir parçalanışı denklik sınıfları olan parçalanış içindeki kümeler için A üzerinde bir denklik bağıntısı tanımlar.

Kanıt Kanıtın ilk kısmı için R nin denklik sınıflarının boştan farklı, ikişer ikişer ayrık kümelerin birleşimlerinin A , olduğunu göstermeliyiz. R yansıyan, $a \in [a]$ olduğu için denklik sınıfları boş değildir; üstelik her $a \in A$ elemanı $[a]$ denklik sınıfının elemanı olduğu için, denklik sınıflarının birleşimi A kümesidir. Geriye denklik sınıflarının ikişer ikişer ayrık olduğunu göstermek kalır yani, eğer iki denklik sınıfı $[a]$ ve $[b]$ iki denklik sınıfı c gibi bir ortak elemana sahip ise o zaman bu kümeler gerçekte aynı kümelerdir. Kabul edelim ki $a R c$ ve $b R c$ olsun. Simetri ile $c R b$ ve geçişkenlik ile $a R b$ olur. Böylece keyfi bir $x \in [a]$ için, $x R b$ sahibiz ve geçişkenlik ile $x R b$ olur ve böylece $[a] \subseteq [b]$. Benzer olarak, $[b] \subseteq [a]$ ve böylece $[a] = [b]$ olur.

Kanıtın ikinci kısmı için, A nın bir parçalanışı $\mathcal{A} = \{A_i\}$ olsun ve $R = \{(a, b) : a \in A_i \text{ ve } b \in A_i \text{ olacak şekilde öyle bir } i \text{ vardır}\}$ olarak tanımlansın. R nin A üzerinde bir denklik bağıntısı olduğunu iddia edelim. $a \in A_i$ gerektirir $a R a$ olduğu için yansıma sağlanır. Eğer $a R b$ ise, a ve b A_i kümesindedir simetri ve böylece $b R a$ olduğundan geçişkenlik sağlanır. Parçalanış içindeki kümelerin R nin denklik sınıfları olduğunu görmek için, eğer $a \in A_i$ ise o zaman $x \in [a]$ gerektirir $x \in A_i$ ve $x \in A_i$ gerektirir $x \in [a]$ olduğunu gözlemliyoruz. ■

Eğer $a R b$ ve $b R a$ gerektirir $a = b$ ise bir A kümesi üzerindeki bir R ikili bağıntısı **ters simetrik** bağıntıdır

Örneğin, doğal sayılar üzerindeki “ $\leq$ ” bağntısı $a \leq b$ ve $b \leq a$ gerektirir $a = b$ olduğu için ters simetriktir. Yansıyan, ters simetrik ve geçişken bir bağntıya bir **kısmi sıralı** denir ve bir kısmi sıra üzerinde tanımlanan kümeye **kısmi sıralı küme** denir. Örneğin, “soyundan gelen kimse” bağntısı tüm insanlar kümesi üzerinde kısmi sıralıdır. (eğer her bir kişi kendi soyundan geliyorsa)

Bir kısmi sıralı A kümesinde $b R a$ iken hiçbir $b \in A$ olacak şekilde tek bir maksimum eleman olmayabilir yani $b \in A$ durumunda. Örneğin, farklı boyutlarda kutuların bir koleksiyonu diğer kutularla uyuşmayan çeşitli maksimal kutular vardır, ancak diğer hiçbir kutunun içine uyan tek bir maksimum kutu yoktur³.

Bir A kümesi üzerindeki bir R bağntısına **tam bağntı** denir eğer her $a, b \in A$ için $a R b$ veya $b R a$ sahip isek (veya her ikisine de), bu da A nın elemanlarının her çifti için A, R tarafından bağlı demektir. A kısmi sıralı kümesi tam bağntı ise A **tam sıralı veya doğrusal sıralı**dır. Örneğin, “ $\leq$ ” bağntısı doğal sayılar üzerinde tam sıralı fakat başka birinin soyundan gelmeyen bireysel kişiler olduğu için “soyundan gelen kimse” bağntısı tüm insanlar kümesi üzerinde tam sıralı değildir. Tam bağntı geçişkendir fakat yansıyan ve ters simetrik olması gerekmeyen bir **tam ön sıralı**dır.

Alıştırmalar

B.2-1

“ $\subseteq$ ” nin tüm altkümeleri üzerindeki $\mathbb{Z}$ altküme bağntısı bir kısmi sıralıdır fakat bir tam sıralı değildir, kanıtlayınız.

B.2-2

Gösteriniz ki herhangi bir pozitif n tam sayısı için “mod n ye denk” bağntısı tam sayılar üzerinde bir denklik bağntısıdır. (Eğer $a \equiv b \pmod{n}$ olacak şekilde bir q tam sayısı varsa $a - b = qn$ deriz). Bu bağntı hangi denklik sınıfları içerisinde tam sayıları parçalar?

B.2-3

Aşağıdaki bağntılara örnekler veriniz.

- Yansıyan ve simetrik fakat geçişken olmayan,
- Yansıyan ve geçişken fakat simetrik olmayan,
- Simetrik ve geçişken fakat yansıyan olmayan.

³Kesinlik adına, “içine uygun olmak” bağntısı kısmi sıralı olması için bakmamız gereken şey kendisinin içine uygun olan bir kutudur.

B.2-4

S sonlu bir küme ve $R, S \times S$ üzerinde bir denklik bağıntısı olsun. Eğer R ye anti simetrik özellik eklersek o zaman R ile ilgili S nin denklik sınıfları tek elemanlıdır, gösteriniz.

B.2-5

Profesör Narcissus bir bağıntının simetrik ve geçişken ise aynı zamanda yansıyan olduğunu iddia ediyor. Şu kanıtı teklif ediyor: simetriyle $a R b$ gerektirir $b R a$ dır. Öyleyse geçişkenlik $a R a$ yı gerektirir. Profesör haklı mıdır?

B.3 Fonksiyonlar

Verilen iki A ve B kümesi için bir **fonksiyon**, A ve B üzerindeki bir f ikili bağıntısıdır öyle ki tüm $a \in A$ için $b \in B$ olacak şekilde tam olarak bir $(a, b) \in f$ vardır. A kümesine f nin **tanım kümesi**, B kümesine f nin **değer kümesi** denir. Bazen $f: A \rightarrow B$ şeklinde yazarız; eğer $(a, b) \in f$ ise b, a tarafından seçimi tek şekilde belirlendiği için $b = f(a)$ yazarız.

Sezgisel olarak, f fonksiyonu B nin bir elemanına A nın her bir elemanı için atama yapar. A nın hiçbir elemanı B nin iki farklı elemanına atanmaz fakat B nin bir elemanına A nın iki farklı elemanına atama yapılabilir. Örneğin, her bir a doğal sıyışı için

$$f = \{(a, b) : a, b \in \mathbb{N} \text{ ve } b = a \bmod 2\}$$

olacak şekilde $f: \mathbb{N} \rightarrow \{0, 1\}$ içinde tam olarak bir tane b değeri olduğu için $\{0, 1\}$ ikili bağıntısı bir $b = a \bmod 2$. fonksiyondur. Bu örnek için, $0 = f(0), 1 = f(1), 0 = f(2)$ vs. Tersine $(1, 3)$ ve $(1, 5)$ g de oldukları ve seçilen $a = 1$ için

$$g = \{(a, b) : a, b \in \mathbb{N} \text{ ve } a + b \text{ çifttir}\}$$

olacak şekilde tam olarak bir tane b olmadığından $(a, b) \in g$ bir fonksiyon değildir.

Verilen bir $f: A \rightarrow B$ fonksiyonu için eğer $b = f(a)$ ise a ya f nin bir **argümanı** ve b ye f nin a daki **değeri** deriz. Tanım kümesinin her elemanına onların değerleri belirtilerek bir fonksiyon tanımlayabiliriz. Örneğin, $f(n) = 2n$ anlamına gelen $n \in \mathbb{N}$ için $f = \{(n, 2n) : n \in \mathbb{N}\}$ tanımlayabiliriz. Tanım kümesindeki her a için $f(a) = g(a)$ ve f ile g aynı tanım ve değer kümesine sahip ise f ve g fonksiyonları **eşittir**.

n uzunluklu bir **sonlu dizi**, tanım kümesi n tam sayılı $\{0, 1, \dots, n-1\}$ olan bir f fonksiyonudur. Bir sonlu diziyi sıklıkla değerleri listeleterek veririz: $\langle f(0), f(1), \dots, f(n-1) \rangle$. Bir **sonsuz dizi** tanım kümesi $\mathbb{N}$ doğal sayılar olan bir fonksiyondur. Örneğin, (3.22) özyineleme Fibonacci dizisi $\langle 0, 1, 1, 2, 3, 5, 8, 13, 21, \dots \rangle$ sonsuz dizidir.

Bir f fonksiyonunun tanım kümesi bir Kartezyen çarpım olduğu zaman, f nin argümanlarının etrafındaki fazladan parantezleri sıklıkla çıkarırız. f için birli argüman teknik olarak n -katlı $b = f((a_1, a_2, \dots, a_n))$ olsa da f fonksiyonu için her bir $(a_1, a_2, \dots, a_n)$ **argüman** da deriz.

Eğer $f : A \rightarrow B$ bir fonksiyon ve $b = f(a)$ ise, o zaman b , a nın f altındaki **görüntüsüdür** deriz. f altındaki bir $A' \subseteq A$ kümesinin görüntüsü $f(A') = \{b \in B : b = f(a) \text{ bazı } a \in A' \text{ için}\}$ ile tanımlanır.

f nin **rankı**, f nin tanım kümesinin görüntüsü $f(A)$ dir. Örneğin, $f : \mathbb{N} \rightarrow \mathbb{N}$ fonksiyonunun ranjı $f(n) = 2n$ ile tanımlı $f(\mathbb{N}) = \{m : m = 2n \text{ bazı } n \in \mathbb{N} \text{ için}\}$ dür, diğer bir deyişle, negatif olmayan çift tam sayılar kümesidir.

Bir fonksiyonun görüntüsü değer kümesi ise fonksiyon **örtendir**. Örneğin, $f(n) = \lfloor n/2 \rfloor$ deki her eleman bazı argümanlar için f nin değeri olarak ortaya çıktığı için $\mathbb{N}$ den $\mathbb{N}$ ye olan $\mathbb{N}$ bir örten fonksiyondur. Tersine, f için hiçbir argüman ile 3 değeri üretilmediği için $f(n) = 2n$ fonksiyonu örten değildir. Bununla birlikte $f(n) = 2n$ fonksiyonu doğal sayılardan çift sayılara örten fonksiyondur. $f : A \rightarrow B$ örten fonksiyonu bazen A dan B üzerine dönüşüm olarak tanımlanır. f üzerine dediğimizde bu onun örten olduğu anlamındadır.

Eğer farklı argümanlar farklı değerler üretiyorsa, yani, eğer $a \neq a'$, $f(a) \neq f(a')$ nü gerektiriyorsa, $f : A \rightarrow B$ fonksiyonu **içine** fonksiyondur. Örneğin, $f(n) = 2n$ fonksiyonu $\mathbb{N}$ den $\mathbb{N}$ e bir içine fonksiyondur çünkü tanım kümesindeki en fazla bir eleman olan $b/2$, her bir çift sayı b nin, f altındaki görüntüsüdür. 1 değeri hem 2 hem de 3 ile üretildiği için $f(n) = \lfloor n/2 \rfloor$ fonksiyonu içine değildir. Bir içine fonksiyona bazen **birebir** fonksiyon deriz.

Bir $f : A \rightarrow B$ fonksiyonu içine ve örten ise **birebir örten (bijeksiyon)** fonksiyon denir. Örneğin, $f(n) = (-1)^n \lfloor n/2 \rfloor$ fonksiyonu $\mathbb{Z}$ den $\mathbb{Z}$ e bir bijeksiyondur:

$$\begin{aligned} 0 &\rightarrow 0, \\ 1 &\rightarrow -1, \\ 2 &\rightarrow 1, \\ 3 &\rightarrow -2, \\ 4 &\rightarrow 2, \\ &\vdots \end{aligned}$$

$\mathbb{Z}$ nin elemanlarının hiçbir $\mathbb{N}$ nin birden fazla elemanının görüntüsü değildir. $\mathbb{Z}$ her elemanı $\mathbb{N}$ nin bazı elemanlarının görüntüsü olarak ortaya çıktığı için fonksiyon örtenidir. Böylece fonksiyon bijeksiyondur. Fonksiyon tanım ve değer kümesindeki elemanları eşlediği için bu bijeksiyona bazen **birebir eşleme** denir. Bir A kümesinden kendisine olan bijeksiyona bazen bir **permütasyon** denir.

Bir fonksiyon bijeksiyon ise onun **ters** f^{-1} fonksiyonunu, $f^{-1}(b) = a$ ancak ve ancak $f(a) = b$ şeklinde tanımlarız.

Örneğin, $f(n) = (-1)^n \lceil n/2 \rceil$ fonksiyonunun ters fonksiyonu şöyledir:

$$f^{-1}(m) = \begin{cases} 2m & m \geq 0 \text{ ise,} \\ -2m - 1 & m < 0 \text{ ise.} \end{cases}$$

Ahştırmalar

B.3.-1

A ve B sonlu kümeler ve $f : A \rightarrow B$ bir fonksiyon olsun. Gösteriniz ki

- a. f bir içine fonksiyon ise, o zaman $|A| \leq |B|$ olur;
- b. f bir örten fonksiyon ise, o zaman $|A| \geq |B|$ olur.

B.3.-2

Tanım ve değer kümesi $\mathbb{N}$ olduğu zaman, $f(x) = x + 1$ fonksiyonu bijeksiyon olur mu? Tanım ve değer kümesi $\mathbb{Z}$ olursa, bijeksiyon olur mu?

B.3.-3

Bir ikili bağıntının tersi için eğer bir bağıntı gerçekte bir bijeksiyon fonksiyon ise onun bağıntısal tersi onun fonksiyonel tersidir olacak şekilde doğal bir tanım veriniz.

B.3.-4 *

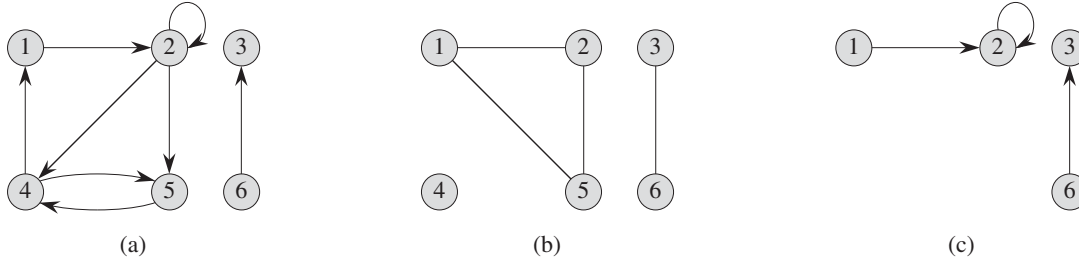
$\mathbb{Z}$ den $\mathbb{Z} \times \mathbb{Z}$ e bir bijeksiyon veriniz.

B.4 Çizgeler

Bu kısımda yönlü ve yönsüz olarak iki çeşit çizge sunar. Kaynaklardaki kesin tanımlar burada verilenlerden farklıdır fakat çok büyük bir kısmında farklılıklar azdır. Kısım 22.1 bilgisayar hafızasındaki çizgelerin nasıl temsil edildiğini gösterir.

Bir **yönlü çizge** (digraf) G , V sonlu bir küme ve E , V üzerinde tanımlı bir ikili bağıntı olmak üzere (V, E) ikilisidir. V kümesine G nin **tepe** kümesi denir ve elemanlarına onun **tepeleri** denir (tekil: tepe). E kümesine G nin **ayrıt** kümesi denir ve onun elemanlarına **ayrıtlar** denir. Şekil B.2(a), $\{1, 2, 3, 4, 5, 6\}$ tepe kümesi üzerindeki bir yönlü çizgenin resimsel temsidir. Tepeler şekil üzerinde çemberler ve ayrıtlar ok işaretleri ile temsil ediliyor. Dikkat edelim ki bir tepeden kendisine olan **öz-döngü** mümkündür.

Bir **yönsüz çizge** $G = (V, E)$, tepelerin sıralı çiftlerinden ziyade sıralı olmayan tepe çiftlerini içerir. Yani, $u, v \in V$ ve $u \neq v$ iken bir ayrıt $\{u, v\}$ kümesidir.



Şekil B.2 Yönlü ve yönsüz çizgeler. (a) $V = \{1, 2, 3, 4, 5, 6\}$ ve $E = \{(1, 2), (2, 2), (2, 4), (2, 5), (4, 1), (4, 5), (5, 4), (6, 3)\}$ iken bir yönlü çizge $G = (V, E)$ dir. (2,2) ayrıtı öz-döngüdür. (b) $V = \{1, 2, 3, 4, 5, 6\}$ ve $E = \{(1, 2), (1, 5), (2, 5), (3, 6)\}$ iken bir yönsüz çizge $G = (V, E)$ dir. Tepe 4 ayrıtıdır. (c) parça (a) da ki çizgenin alt çizgesi $\{1, 2, 3, 6\}$ tepe kümesi tarafından içerilir.

Genel kabul olarak, $\{u, v\}$ küme simgesinden ziyade bir ayrıtı için (u, v) simgesi kullanırız ve (u, v) ve (v, u) aynı ayrıtı olduğunu göz önünde bulundururuz. Bir yönsüz çizgede, öz-döngüler yasaktır ve her ayrıtı iki ayrıtı tepeden oluşur. Şekil B.2(b) $\{1, 2, 3, 4, 5, 6\}$ tepe kümesi üzerindeki bir yönsüz çizgenin bir resimsel temsidir.

İki içerikteki hali hazırdaki terimler önemsenmeyecek kadar farklı olmasına rağmen, yönlü ve yönsüz çizgeler için çoğu tanımlar aynıdır. Eğer (u, v) , $G = (V, E)$ yönlü çizgesindeki bir ayrıtı ise (u, v) tepe u **dan bağılıdır** veya **ayrılır** ve (u, v) tepe v **ye bağılıdır** veya **ulaşır**. Örneğin, şekil B.2(a) de tepe 2 den ayrılan ayrıtlar $(2,2)$, $(2,4)$ ve $(2,5)$ dir. Tepe 2 ye ulaşan ayrıtlar $(1,2)$ ve $(2,2)$ dir. Eğer (u, v) bir ayrıtı ve $G = (V, E)$ yönsüz çizge ise (u, v) u ve v üzerinden **bağılıdır** deriz. Şekil B.2(b) de tepe 2 ye bağılı olan ayrıtlar $(1,2)$ ve $(2,5)$ dir.

$G = (V, E)$ çizgesinde (u, v) bir ayrıtı ise tepe v , tepe u ya **bitişiktir** deriz. Çizge yönsüz olduğu zaman bitişiklik bağıntısı simetriktir. Çizge yönlü olduğu zaman bitişiklik bağıntısının simetrik olması gerekmez. Eğer bir yönlü çizgede v, u ya bitişik ise bazen $u \rightarrow v$ yazarız. Şekil B.2 nin (a) ve (b) kısımlarında ayrıtı $(1,2)$ her iki çizgenin de elemanı olduğu için tepe 2, tepe 1 e bitişiktir. Ayrıtı $(2,1)$ Şekil B.2(a) da ki çizgeye ait olmadığı için tepe 1, tepe 2 ye bitişik değildir.

Bir yönsüz çizgede bir tepesinin **derecesi** o tepeye bağılı olan ayrıtların sayısıdır. Örneğin, Şekil B.2(b) deki tepe 2, 2 dereceye sahiptir. Şekil B.2(b) deki tepe 4 teki gibi derecesi sıfır olan tepe **izole**dir. Bir yönlü çizgede bir tepenin **dış derecesi** tepeden ayrılan ayrıtların sayısı ve **iç derecesi** o tepeye bağlanan ayrıtların sayısıdır. Bir yönsüz çizgede bir tepenin **derecesi** onun iç dereceleri ile dış

derecelerinin toplamıdır. Şekil B.2(a) da tepe 2, 2 iç derece, 2 dış dereceye ve 5 dereceye sahiptir.

Bir $G = (V, E)$ çizgesinde bir u tepesinden bir v tepesine bir k **uzunluklu yol** $\langle v_0, v_1, v_2, \dots, v_k \rangle$ $u = v_0, u' = v_k$ ve $(v_{i-1}, v_i) \in E$ olacak şekilde tepelerin bir $i = 1, 2, \dots, k$ dizisidir. Yol uzunluğu, yoldaki ayrıtların sayıdır. Yol $v_0, v_1, \dots, v_k$ tepelerini ve $(v_0, v_1), (v_1, v_2), \dots, (v_{k-1}, v_k)$ ayrıtlarını **içerir**. (Her zaman u dan u ya 0-uzunluklu yol vardır.) Eğer u dan u' e bir p yolu ise p aracılığı ile u' e u ya **ulaşılabilir** deriz ve eğer G yönlü ise bazen $u \xrightarrow{p} u'$ olarak yazarız. Bir yoldaki tüm tepeler ayrık ise yol **basittir**⁴. Şekil B.2(a)'da, $\langle 1, 2, 5, 4 \rangle$ yolu 3 uzunluklu basit yoldur. $\langle 2, 5, 4, 5 \rangle$ yolu basit değildir.

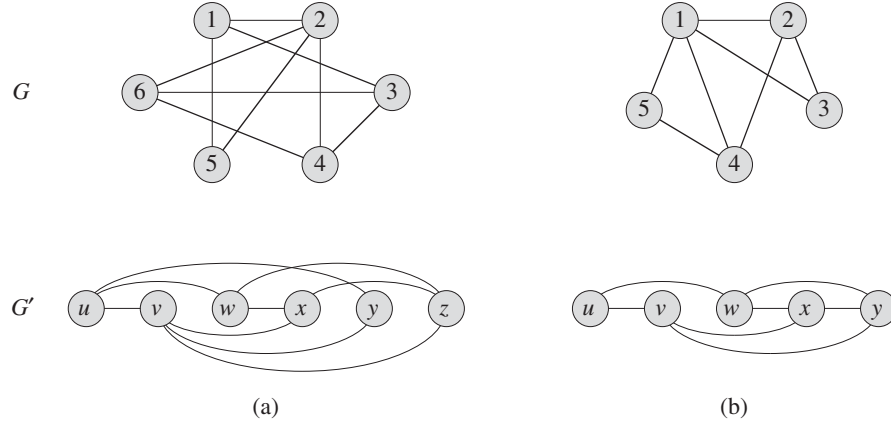
$p = \langle v_0, v_1, \dots, v_k \rangle$ yolunun bir **alt yolu** bu yolun tepelerinin bir sürekli alt dizisidir. Yani, herhangi bir $0 \leq i \leq j \leq k$ için $\langle v_i, v_{i+1}, \dots, v_j \rangle$ tepelerinin alt dizisi p nin bir alt yoludur.

Bir yönlü çizgede eğer $\langle v_0, v_1, \dots, v_k \rangle$ ve yol en az bir ayrıt içerirse yol çevrim oluşturur. Eğer çevrime ek olarak $v_1, v_2, \dots, v_k$ ayrık iseler çevrim **basittir**. Bir öz döngü 1 uzunluklu bir çevrimdir. İki yol $\langle v'_0, v'_1, v'_2, \dots, v'_{k-1}, v'_0 \rangle$ ve $\langle v_0, v_1, v_2, \dots, v_{k-1}, v_0 \rangle$ aynı çevrimi oluşturur şayet $v'_i = v_{(i+j) \bmod k}$ için $i = 0, 1, \dots, k-1$ olacak şekilde en az bir j tam sayısı varsa. Şekil B.2(a)'da, $\langle 1, 2, 4, 1 \rangle$ yolu, $\langle 2, 4, 1, 2 \rangle$ ve $\langle 4, 1, 2, 4 \rangle$ yollarıyla aynı çevrimi oluşturur fakat $\langle 1, 2, 4, 5, 4, 1 \rangle$ çevrimi değil. $\langle 2, 2 \rangle$ ayrıtı tarafından oluşturulan $\langle 2, 2 \rangle$ çevrimi öz döngüdür. Öz döngü olmayan bir yönlü çizge **basittir**. Bir yönsüz çizgede, bir $\langle v_0, v_1, \dots, v_k \rangle$ yolu, eğer $k \geq 3$ ve $v_0 = v_k$ bir çevrim oluşturur; şayet $v_1, v_2, \dots, v_k$ ayrık iseler çevrim **basittir**. Örneğin, şekil B.2(b)'de, $\langle 1, 2, 5, 1 \rangle$ yolu bir basit çevrimdir. Çevrimi olmayan çizgeye çevrimsiz denir.

Eğer bir yönsüz çizgenin her tepesi diğer tüm tepelerden ulaşılabilir ise **bağlantılıdır**. Bir çizgenin **bağlantılı bileşenleri** “den ulaşılabilirdir” bağıntısı altında tepelerin denklik sınıflarıdır. Şekil B.2(b)'deki çizge üç bağlantılı bileşene sahiptir: $\{1, 3, 5\}$, $\{3, 6\}$ ve $\{4\}$. $\{1, 2, 5\}$ deki her tepe $\{1, 2, 5\}$ içindeki diğer her tepesinden ulaşılabilirdir. Bir yönsüz çizge eğer tam olarak bir bağlantılı bileşene sahip ise bağlantılıdır. Bir bağlı bileşenin ayrıtları sadece bileşenin tepeleri üzerinden bağlıdır, diğer bir deyişle hem u hem de v bileşenin tepeleri ise (u, v) ayrıtı bağlantılı bileşenin bir ayrıtıdır.

Eğer bir yönlü çizgede her iki tepe diğer herbirinden ulaşılabilir ise bu yönlü çizge **kuvvetli bağlantılıdır**. Bir yönlü çizgenin **kuvvetli bağlantılı bileşenleri**

⁴Bazı yazarlar bir yol dediğimiz şeyi “gidiş, hareket” ve basit yol dediğimiz şeyi de sadece “yol” olarak ima ederler. “yol” ve “basit yol” ifadelerini kitap boyunca onların tanımları ile tutarlı olacak şekilde kullanıyoruz.



Şekil B.3 (a) Eşyapılı çizgelerin bir çifti. Yukarıdaki çizgenin tepeleri $f(1) = u$, $f(2) = v$, $f(3) = w$, $f(4) = x$, $f(5) = y$, $f(6) = z$ ile alttaki çizgenin tepeleri ile eşlenmiştir. (b) Yukarıdaki çizge 4 dereceli bir tepeye sahip fakat alttaki çizge sahip olmadığı için iki çizge eşyapılı değildir.

“karşılıklı ulaşılabilir” bağıntısı altında tepelerin denklik sınıflarıdır. Bir yönlü çizge sadece bir kuvvetli bağlantılı bileşene sahip ise kuvvetli bağlantılıdır. Şekil B.2(a) deki çizge üç tane kuvvetli bağlantılı bileşene sahiptir: $\{1, 2, 4, 5\}$, $\{3\}$ ve $\{6\}$. $\{1, 2, 4, 5\}$ içindeki tüm tepe çiftleri karşılıklı olarak ulaşılabilir. Tepe 6, tepe 3 ulaşılabilir olmadığı için $\{3, 6\}$ bir kuvvetli bağlantılı bileşen teşkil etmez.

İki çizge $G = (V, E)$ ve $G' = (V', E')$ **eşyapılıdır** eğer $f : V \rightarrow V'$ ancak ve ancak $(u, v) \in E$ olacak şekilde bir $(f(u), f(v)) \in E'$ bijeksiyonu varsa. Diğer bir deyişle, G ve G' deki eş ayrıtları koruyarak G nin tepelerini G' nün tepeleri olarak yeniden etiketleyebiliriz. Şekil B.3(a), $V = \{1, 2, 3, 4, 5, 6\}$ ve $V' = \{u, v, w, x, y, z\}$ tepe kümeleri esas alınarak G ve G' çizgelerinin eşyapılı olduğunu gösterir. $f(1) = u$, $f(2) = v$, $f(3) = w$, $f(4) = x$, $f(5) = y$, $f(6) = z$ ile verilen V den V' dönüşümü gerekli biyektif fonksiyonu sağlar. Şekil B.3(b)'deki çizgeler eşyapılı değildi. Her iki çizge tepe 5 ve tepe 7 ye sahip olmalarına rağmen, yukarıdaki çizge 4 dereceli ve aşağıdaki çizge 4 dereceli değildir.

Bir $G' = (V', E')$ çizgesi, $G = (V, E)$ çizgesinin **altçizgesidir** deriz eğer $V' \subseteq V$ ve $E' \subseteq E$ gerçekleşiyorsa. Verilen bir $V' \subseteq V$ kümesi için V' tarafından **indüklenen** G nin altçizgesi, $G' = (V', E')$ olduğunda $E' = \{(u, v) \in E : u, v \in V'\}$ çizgesidir.

Şekil B.2(a) daki $\{1,2,3,6\}$ tepe kümesi tarafından indüklenen altçizge Şekil B.2(c) de görünür ve $\{(1,2), (2,2), (6,3)\}$ ayrıt kümesine sahiptir.

Verilen bir $G = (V, E)$ yönsüz çizgesi için G nin **yönlü versiyonu**, $G' = (V, E')$ ancak ve ancak $(u, v) \in E'$ olduğunda $(u, v) \in E$ yönlü çizgesidir. Yani, G ye ait her yönsüz tepe (u, v) yi yönlü versiyondaki (u, v) ve (v, u) iki yönlü ayrıt ile yer değiştiririz. Verilen bir yönlü çizge $G = (V, E)$ için G nin **yönsüz versiyonu**, $G' = (V, E')$ ancak ve ancak $u \neq v$ ve $(u, v) \in E$ olduğunda $(u, v) \in E'$ yönsüz çizgesidir. Yani, yönsüz versiyon, G nin “yönleri kaldırılmış” ve öz-döngüleri elenmiş ayrıtlarını içerir (u, v) ve (v, u) bir yönsüz çizgede aynı ayrıt olduğu için, yönlü çizge her iki (u, v) ve (v, u) ayrıtı içerse bile, bir yönlü çizgenin yönsüz versiyonu onu sadece bir defa içerir.) Bir $G = (V, E)$ yönlü çizgesinde, bir u tepesinin bir **komşusu**, G nin yönsüz versiyonundaki u ya bitişik olan herhangi bir tepedir. Yani, v u nun bir komşusudur, eğer $u \neq v$ veya $(u, v) \in E$ ya da $(v, u) \in E$ ise. Bir yönsüz çizgede, eğer u ve v bitişiklerse, komşudurlar.

Bir çok çeşitli çizge özel isimlere sahiptirler. Bir **tam çizge** her tepe çifti komşu olan yönsüz çizgedir. Bir **iki parçalı çizge**, $(u, v) \in E$ gerektirir ya $u \in V_1$ ve $v \in V_2$ ya da $u \in V_2$ ve $v \in V_1$ olacak şekilde V nin V_1 ve V_2 gibi iki kümeye bölünebilen $G = (V, E)$ bir yönsüz çizgesidir. Yani, tüm ayrıtlar iki küme V_1 ve V_2 ye aracılık yaparlar. Bir çevrimsiz yönsüz çizge bir **orman** ve bir bağlantılı, çevrimsiz, yönsüz çizge bir **(serbest) ağaçtır** (Kısım B.5 e bakınız). Sık sık “yönlü çevrimsiz çizgenin ilk harflerini alır ve böyle bir çizgeye **dag** deriz.

Bazen karşılaşabileceğiniz iki çeşit çizge vardır. Bir **çoklu çizge** bir yönsüz çizge gibidir ama aynı zamanda da tepeler ve öz-döngüler arasında birden çok ayrıtı sahiğ olabilir. Bir **hiper çizge** bir yönsüz çizge gibidir fakat her bir **hiper ayrıt** iki tepenin bağlamasından ziyade tepelerin bir keyfi alt kümesini bağlar. Alışıl gelmiş yönlü ve yönsüz çizgeler için yazılan birçok algoritma bu çizge benzeri yapılar üzerinde çalışmak için uyarlanabilir.

Bir yönsüz $G = (V, E)$ çizgesinin bir $e = (u, v)$ ayrıtı ile **büzülmesi**, $G' = (V', E')$ ve x yeni bir tepe olmak üzere bir $V' = V - \{u, v\} \cup \{x\}$ çizgesidir. Ayrıtların bir E' kümesi (u, v) ayrıtı silerek ve u veya v üzerinden bağlı olan her bir w tepesi için E içinde olan hangi (u, w) ve (v, w) olursa o tepeyi silerek ve yeni (x, w) ayrıtı eklenerek, E tarafından oluşturulur. Aslında u ve v bir tek tepeye “daraltılmıştır”.

Alıştırmalar

B.4-1

Bir fakülte partisinin katılımcıları her biri diğerini selamlamak için tokalaşırlar ve her bir profesör kaç defa tokalaştığını hatırlar. Partinin sonunda, bölüm başkanı

her bir profesörün tokalaştıkları sayıyı ilave eder. Sonucu **tokalaşma lemmasını** kanıtlayarak: eğer $G = (V, E)$ yönsüz bir çizge ise o zaman

$$\sum_{v \in V} \text{derece}(v) = 2|E|$$

olduğunu gösteriniz.

B.4-2

Eğer bir yönlü veya yönsüz çizge u ve v tepeleri arasında bir yol içeriyorsa o zaman u ve v arasında bir basit yol içerir, gösteriniz. Eğer bir yönlü çizge bir çevrim içeriyorsa o zaman bir basit çevrim içerir, gösteriniz.

B.4-3

Herhangi bir bağlantılı yönsüz çizge $G = (V, E)$ nin, $|E| \geq |V| - 1$ eşitsizliğini doğruladığını gösteriniz.

B.4-4

Bir yönsüz çizgede “dan ulaşılabilir” bağıntısının çizgenin tepeleri üzerinde bir denklik bağıntısı olduğunu kanıtlayınız. Bir denklik bağıntısının üç özelliğinden hangisi bir yönlü çizgenin tepeleri üzerindeki “dan ulaşılabilir” bağıntısı için genellikle sağlanır?

B.4-4

Şekil B.2(a) daki yönlü çizgenin yönsüz versiyonu nedir? Şekil B.2(b) deki yönsüz çizgenin yönlü versiyonu nedir?

B.4-5 *

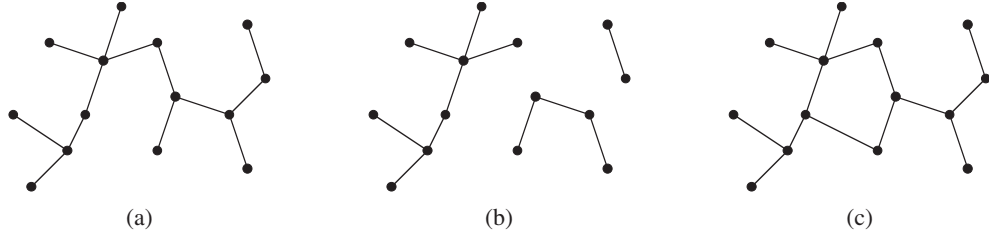
Eğer hiper çizgedeki tepelerin iki parçalı çizgedeki bitişikliğe bire bir eşlenmesine izin verirse, bir iki parçalı çizgeyi bir hiper çizge olarak temsil edebildiğimizi gösteriniz. (*İpucu:* iki parçalı çizgedeki tepelerin bir kümesini hiper çizgenin tepelerine karşılık getirelim ve iki parçalı çizgenin tepelerinin diğer kümesini hiper ayrıtlara karşılık getirelim.)

B.5 Ağaçlar

Çizgelerde olduğu gibi, az oranda farklı olarak ağaçlarla ilgili kavramlar vardır. Bu kısım ağaçların çeşitli tanımlarını ve matematiksel özelliklerini sunar. Kısımlar 10.4 ve 22.1 bilgisayar hafızasında ağaçları nasıl temsil edebileceğimizi betimler.

B.5.1 Serbest ağaçlar

Kısım B.4 de tanımlandığı gibi, bir **serbest ağaç** bir bağlantılı, çevrimsiz, yönsüz çizgedir. Bir çizge bir ağaçtır dediğimizde “serbest” sıfatını kaldırırız. Eğer bir yönsüz çizge bir çevrimsiz fakat imkân dâhilinde bağlantısız ise o bir **ormandır**.



Şekil B.4 (a) Bir serbest ağaç. (b) Bir ağaç (c) Bir çevrim içeren fakat ne bir ağaç ne de orman olan bir çizge.

Ağaçlar için çalışan birçok algoritma ormanlar için de çalışır. Şekil B.4(a) bir serbest ağacı gösterir ve Şekil B.4(b) bir orman gösterir. Şekil B.4(b) deki orman bağlantılı olmadığı için bir ağaç değildir. Şekil B.4(c) deki çizge bağlantılı fakat bir çevrim içerdiğinden ne bir ağaç ne de bir ormandır.

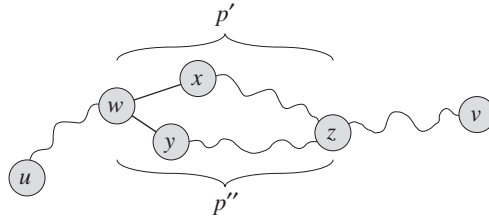
Aşağıdaki teorem serbest ağaçlarla ilgili birçok duruma dikkat çeker.

Teorem B.2 (Serbest ağaçların özellikleri)

$G = (V, E)$ bir yönsüz çizge olsun. Aşağıdaki ifadeler denktir.

1. G bir serbest ağaçtır.
2. G deki herhangi iki tepe bir tek basit yol ile bağlantılıdır.
3. G bağlantılıdır fakat E den herhangi iki ayrıt kaldırılırsa ortaya çıkan çizge bağlantısızdır.
4. G bağlantılıdır ve $|E| = |V| - 1$ dir.
5. G çevrimsizdir ve $|E| = |V| - 1$ dir.
6. G çevrimsizdir fakat E ye herhangi iki ayrıt eklenirse ortaya çıkan çizge bir çevrim içerir.

Kanıt $(1) \Rightarrow (2)$: Bir ağaç bağlantılı olduğu için G deki herhangi iki tepe en az bir basit yol ile bağlantılıdır. Kabul edelim ki, çelişki adına, şekil B.5 de gösterildiği gibi bu u ve v tepeleri p_1 ve p_2 iki ayrık basit yol ile bağlantılı olsunlar. w ise yollardaki ilk ayrılan tepe olsun; yani, $w, x \neq y$ iken p_1 üstündeki ardılı x ve p_2 üstündeki ardılı y olan hem p_1 hem de p_2 üstündeki ilk tepedir. z de yollarda birleşen ilk tepe olsun; yani, p_1 üzerindeki w yu takip eden ayrıca p_2 üzerinde olan ilk tepedir. p' , w den x e doğru uzanan, z ye ulaşan p_1 in bir alt yolu olsun ve p'' , w den y e doğru uzanan, z ye ulaşan p_2 nin bir alt yolu olsun. p' ve p'' yolları bitiş noktaları hariç hiçbir tepayı paylaşmazlar. Böylece p' ve p'' nin tersini bağlayarak



Şekil B.5 Teorem B.2 nin kanıtında bir aşama: eğer (1) G bir serbest ağaç ise o zaman (2) G deki herhangi iki tepe bir tek basit yol ile bağlantılıdır. Çelişki adına, u ve v tepeleri, iki ayrık basit yol p_1 ve p_2 ile bağlantılı olduğunu kabul edelim. Bu yollar ilk olarak w de bölünür ve onlar ilk olarak z de tekrar birleşirler. p' yolunun tersi ile art arda sıranmış p'' yolu çelişkiye yol açan bir çevrim oluşturur.

elde edilen yol bir çevrim olması G nin bir ağaç olduğu varsayımımız ile çelişir. Böylece, eğer G bir ağaç ise iki tepe arasında en fazla bir basit yol olabilir.

(2) $\Rightarrow$ (3): Eğer G içindeki herhangi iki çizge bir tek basit yol ile bağlantılı ise o zaman G bağlantılıdır. $(u, v), E$ de herhangi bir ayrıt olsun. Bu ayrıt u dan v ye bir yol ve böylece u dan v ye bir tek yol olmalıdır. G den (u, v) ayrıtını kaldırırsak u dan v ye bir yol olmaz ve böylece bu kaldırma G bağlantısızlaştırır.

(3) $\Rightarrow$ (4): Varsayım ile, G çizgesi bağlantılı ve Alıştırma B.4-3 ile $|E| \geq |V| - 1$ elde ettik. Tümevarım ile $|E| \leq |V| - 1$ kanıtlayacağız. $n = 1$ ve $n = 2$ tepeli bir bağlantılı çizge $n - 1$ ayrıta sahiptir. Kabul edelim ki, G , $n \geq 1$ tepeli ve n den az tepeye sahip (3) ü sağlayan tüm çizgeler $|E| \leq |V| - 1$ eşitsizliğini de sağlasın. G den keyfi bir ayrıtı kaldırma çizgesi $k \geq 2$ bağlantılı bileşenlere ayırır (gerçekte $k = 2$). Her bir bileşen ya (3) ü sağlar ya da başka bir G , (3) ü sağlamayacaktır. Eğer E_i ayrıt kümeli ve kendisi serbest ağaç olan her bir bağlı bileşen V_i yi incelersek, her bir bileşen $|V|$ az tepeye sahip olduğu için tümevarım hipotezi ile $|E_i| \leq |V_i| - 1$ sahibiz. Böylece, birleşik tüm bileşenlerin içindeki ayrıtların sayısı en fazla $|V| - k \leq |V| - 2$ dir. Kaldırılan ayrıtlara ekleme $|E| \leq |V| - 1$ eşitsizliğini verir.

(4) $\Rightarrow$ (5): kabul edelim ki, G bağlantılı ve $|E| = |V| - 1$ olsun. G nin çevrimsiz olduğunu göstermeliyiz. Kabul edelim ki, G , k tepe $v_1, v_2, \dots, v_k$ içeren bir çevrime sahip olsun ve genelliği kaybetmeden varsayalım ki bu çevrim basit olsun. $G_k = (V_k, E_k)$, çevrim içeren G nin altçizgesi olsun. Dikkat edelim ki, $|V_k| = |E_k| = k$ eşitliği vardır. Eğer $k < |V|$ ise G bağlantılı olduğu için bazı $v_{k+1} \in V - V_k$ tepelerine bitişik olan bir tepesi $v_i \in V_k$ olmalıdır. $G_{k+1} = (V_{k+1}, E_{k+1})$ çizgesini $V_{k+1} = V_k \cup \{v_{k+1}\}$ ve $E_{k+1} = E_k \cup \{(v_i, v_{k+1})\}$ özelliğine sahip G nin altçizgesi olarak tanımlayalım.

Dikkat edelim ki, $|V_{k+1}| = |E_{k+1}| = k + 1$ eşitliği vardır. Eğer $k + 1 < |V|$ ise, aynı şekilde G_{k+2} tanımlayarak $n = |V|$, $V_n = V$ ve $|E_n| = |V_n| = |V|$ olduğunda $G_n = (V_n, E_n)$ çizgesini elde edene kadar devam edebiliriz. G_n , G nin bir altçizgesi olduğu için $E_n \subseteq E$ sahibiz ve $|E| \geq |V|$ olur, bu da $|E| = |V| - 1$ varsayımı ile çelişir. Böylece, G bir çevrimsizdir.

(5) $\Rightarrow$ (6): Kabul edelim ki, G bir çevrimsiz ve $|E| = |V| - 1$ olsun. G nin bağlantılı bileşenlerinin sayısı k olsun. Tanımdan her bir bağlantılı bileşen bir serbest ağaçtır ve (1) gerektirir (5) olduğu için G nin tüm bağlantılı bileşenlerindeki tüm ayrıtların toplamı $|V| - k$ olur. Sonuç olarak, $k = 2$ sahip olmak zorundayız ve G gerçekte bir ağaçtır. (1) gerektirir (2) olduğu için G deki herhangi iki tepe bir tek basit yol ile bağlantılıdır. Böylece, G ye herhangi bir ayrıt ilave etme bir çevrim oluşturur.

(6) $\Rightarrow$ (1): Kabul edelim ki, G bir çevrimsiz fakat E ye herhangi bir ayrıt ilave etme bir çevrim oluştursun. G nin çevrim olduğunu göstermek zorundayız. u ve v , G deki keyfi seçilmiş tepeler olsun. Eğer u ve v halihazırda bitişik değiller ise, o zaman (u, v) ayrıtını ekleme (u, v) hariç tüm ayrıtların G de olduğu bir çevrim oluşturur. Böylece, çevrim ile (u, v) ayrıtının farkı u dan v ye bir yol içermektedir ve u ve v keyfi seçildiği için G bağlantılıdır. ■

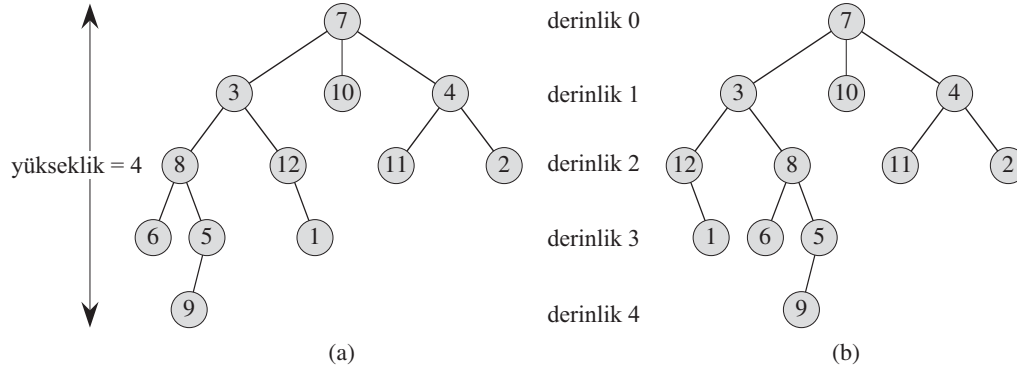
B.5.2 Köklü ve sıralı ağaçlar

Bir **köklü ağaç** tepelerinden bir tanesi diğerlerinden ayrık olan bir serbest ağaçtır. Ayrık olan tepeye ağacın **kökü** deriz. Bir köklü ağacın bir tepesini ağacın bir **düğümü**⁵ olarak adlandırırız. Şekil B.6(a), kök 7 ye sahip 12 düğümlü bir küme üzerinde bir köklü ağaç gösterir.

Kök r ye sahip T köklü ağacında x düğümünü göz önünde bulundurun. r den x e tek basit yol üstündeki her y düğüme x in **atası** deriz. Eğer y, x in bir atası ise x, y nin bir **torunudur**. (Her düğüm kendisinin hem bir atası hem de torunudur.) Eğer y, x in bir torunu ve $x \neq y$ ise y, x in bir **öz atası** ve x, y nin bir **öz torunudur**. **x de köklü alt ağaç** x de köklü, x in torunları tarafından indüklenmiş ağaçtır. Örneğin, Şekil B.6(a) da düğüm 8 de köklü alt ağaç 8, 6, 5 ve 9 düğümlerini içerir.

Bir T ağacının r kökünden bir x düğüme olan basit yol üzerindeki son ayrıt (y, x) ise o zaman y, x in bir **ebeveyni** ve x, y nin bir **çocuğudur**. Kök ebeveynsiz T içinde sadece düğümdür. İki düğüm aynı ebeveyne sahip ise, onlar **kardeştir**. Çocuksuz bir düğüm bir **dış düğümün yaprağıdır**. Bir yapraksız düğüm bir **iç düğümdür**.

⁵“düğüm” terimi çizge teori kaynakçasındaki “tepe” için bir anlamdaş olarak kullanılır. “düğüm” terimini bir köklü ağacın bir tepesi anlamına karşılık getiriyoruz.



Şekil B.6 Köklü ve sıralı ağaçlar. (a) 4 yükseklikli bir köklü ağaç. Ağaç standart yolla çizilmiştir: kök (düğüm 7) tepededir, onun çocukları (1 derinlikli düğümler) ondan aşağıdadır, onların çocukları (2 derinlikli düğümler) onların aşağısındadır, vs. Eğer ağaç sıralı ise bir düğümün çocuklarının soldan sağa sıralanma ilişkisi önemlidir; aksi takdirde değildir. (b) Bir diğer köklü ağaç. Bir köklü ağaç olarak (a) daki ağaçla özdeştir fakat düğüm 3'ün çocukları farklı bir sırada gözüktüğü için bir sıralı ağaç olarak farklıdır.

Bir köklü T ağacındaki bir x düğümünün çocuklarının sayısı x in **derecesi**⁶ eşittir. Kök r den bir x düğümüne olan basit yolun uzunluğu T içindeki x in **derinliğidir**. Bir ağacın **seviyesi** aynı derinlikteki tüm düğümlerden oluşur. Bir ağaçtaki bir düğümün **yüksekliği** düğümden bir yaprağa basit aşağıya doğru inen en uzun yol üstündeki ayrıtların sayısıdır. Bir ağacın yüksekliği ağaç içindeki her düğümden en büyük derinlikli olanın derinliğine de eşittir.

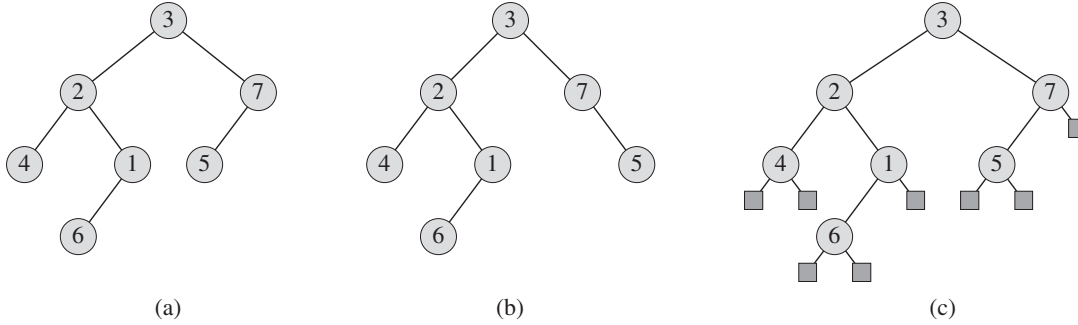
Bir **sıralı ağaç** her bir düğümün çocuklarının sıralı olduğu köklü ağaçtır. Yani, eğer bir düğüm k çocuğa sahip ise bir birinci çocuk, bir ikinci çocuk, ..., ve bir k . çocuk vardır. Şekil B.6 daki iki ağaç sıralı ağaç olarak göz önünde bulundurulursa farklı fakat sadece köklü ağaçlar olarak bakılırsa aynıdır.

B.5.3 İkili ve konumsal ağaçlar

İkili ağaçları özyineleme olarak tanımlarız. Bir **ikili ağaç** T bir sonlu düğümler kümesi üzerinde tanımlanan bir yapıdır öyle ki ya:

- Hiç bir düğüm içermez, ya da

⁶Dikkat ediniz ki, bir düğümün derecesi T nin bir köklü ağaç veya bir serbest ağaç olarak göz önünde bulundurulmasına bağlıdır. Bir serbest ağaç içindeki bir düğümün derecesi herhangi bir yönsüz ağaçtaki tepelerin bitişiklik sayısı ile aynıdır. Bir köklü ağaçta, derece çocukların sayısıdır-bir düğümün ebeveyni onun derecesi olarak sayılmaz.



Şekil B.7 İkili ağaçlar. **(a)** Standart bir yolla çizilmiş bir ikili ağaç. Bir düğümün sol çocuğu, düğümün altında ve solunda çizilmiştir. Sağ çocuk, altında ve sağında çizilmiştir. **(b)** (a) da olan ağaçtan farklı bir ikili ağaç. (a) da, 7 düğümünün sol çocuğu 5 tir ve sağ çocuğu bulunmamaktadır. (b) de 7 düğümünün sol çocuğu bulunmamakta ve sağ çocuğu 5 tir. Sıralı ağaçlardaki gibi bu ağaçlarda aynıdır, fakat ikili ağaç olarak farklıdır. (c) Bir tam ikili ağacın iç düğümleriyle temsil edilmiş (a) daki ikili ağaç: her bir düğümün 2 dereceli olduğu sıralı bir ağaç. Ağaçtaki yapraklar kareler şeklinde gösterilmiştir.

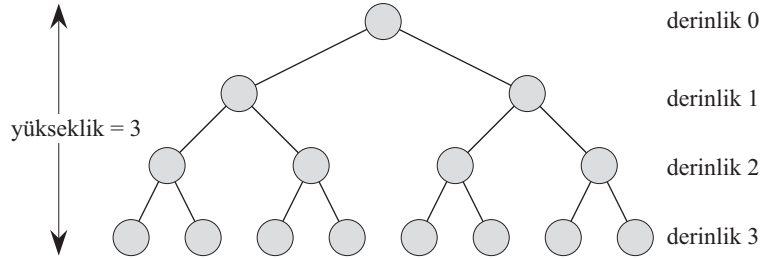
- Üç ayrık düğümler kümesinden oluşur: bir **kök** düğüm, onun **sol altağacı** denilen bir ikili ağaçtır ve onun **sağ altağacı** denilen bir ikili ağaçtır.

Düğüm içermeyen ikili ağaca **boş ağaç** ya da **değersiz ağaç** denir, bazen NIL ile ifade edilir. Eğer sol altağaç boştan farklı ise köküne tüm ağacın kökünün **sol çocuğu** denir. Aynı şekilde boştan farklı sağ altağacın kökü tüm ağacın kökünün **sağ çocuğudur**. Eğer bir altağaç boş ağaç NIL ise çocuğa **olmayan** ya da **kayıp** deriz. Şekil B.7(a) bir ikili ağacı gösterir.

Bir ikili ağacı her bir düğüm en fazla iki 2 dereceye sahip ise basitçe sıralı ağaç değildir. Örneğin, bir ikili ağaçta, bir düğüm sadece bir çocuğa sahip ise çocuğun konumu ya **sol çocuk** ya da **sağ çocuk** olur. Bir sıralı ağaçta, sol ya da sağ olması yalnızca bir çocuk için bir farklılık oluşturmaz. Şekil B.7(b) ikili ağaç, bir düğümün konumundan dolayı Şekil B.7(b) deki ağaçtan farklılık gösterir. Sıralı ağaçlar olarak dikkate alırsak, iki ağaç özdeştir.

Bir sıralı ağacın iç düğümleri ile bir ikili ağaç içindeki bilgileri konumlamayı şekil B.7(c) de gösterildiği gibi temsil edebiliriz. Fikir, hiç çocuğa sahip olmayan bir düğümlü ikili ağaç içindeki her bir kayıp çocuğu yeniden konumlandırmaktır. Bu yaprak düğümler şekildeki kareler gibi çizilir. Ortaya çıkan ağaç bir **tam ikili ağaçtır**: her bir düğüm ya bir yaprak ya da tam olarak 2 dereceye sahiptir. Orada 1 dereceli düğümler yoktur. Sonuç olarak, bir düğümün çocuklarının sırası konum bilgisini korur.

Her bir düğüm için 2 den fazla çocuklu ağaçlar için sıralı ağaçları ikili ağaçlardan ayıran bilgi konumlamayı genişletebiliriz. Bir **konumsal ağaçta**, bir düğümün



Şekil B.8 Bir 3 yükseklikli, 8 yapraklı ve 7 iç düğümlü tam ikili ağaç.

mün çocukları farklı pozitif tam sayılar ile etiketlenir. Hiçbir çocuk i tam sayısı ile etiketlenmemiş ise bir düğümün i . çocuğu *mevcut değildir*. Bir **k -ariteli** ağaç, her düğüm için k den daha büyük olan etiketli tüm çocukların kayıp olduğu bir konumsal ağaçtır. Böylece, bir ikili ağaç $k = 2$ li k -li ağaçtır.

Bir **tam k -li** ağaç, tüm yaprakları aynı derinliğe sahip ve tüm iç düğümleri k dereceli bir k -li ağaçtır. Şekil B.8, 3 derinliğe sahip bir tam ikili ağaçtır. Kaç yaprak h yükseklikli bir k -li tam ağaca sahip olur? Kök derinlik 1 de k çocuğa sahiptir, onların her biri derinlik 2 de k çocuğa sahiptir, vs. böylece, h derinlikteki yaprakların sayısı k^h dir. Sonuç olarak, n yapraklı k -li bir tam ağacın yüksekliği $\log_k n$ dir. h yükseklikli bir k -li tam ağacın iç düğümlerinin sayısı (A.5) eşitliği ile

$$\begin{aligned} 1 + k + k^2 + \dots + k^{h-1} &= \sum_{i=0}^{h-1} k^i \\ &= \frac{k^h - 1}{k - 1} \end{aligned}$$

olur. Böylece, bir tam ikili ağaç $2^h - 1$ tane iç düğüme sahiptir.

Alıştırmalar

B.5-1

Üç x , y ve z tepeden oluşan tüm serbest ağaçları çizin. x köklü, x , y ve z düğümlü tüm köklü ağaçları çizin. x köklü, x , y ve z düğümlü tüm sıralı ağaçları çizin. x köklü, x , y ve z düğümlü tüm ikili ağaçları çizin.

B.5-2

$G = (V, E)$, bir $v_0 \in V$ tepesi olan öyle ki v_0 dan her $v \in V$ tepesine tek bir olacak şekilde bir yönlü çevrimsiz çizge olsun. G nin yönsüz versiyonunun bir ağaç oluşturduğunu kanıtlayınız.

B.5-3

Her boştan farklı ikili ağaçtaki 2 dereceli düğümlerin sayısının yaprakların sayısından 1 az sayıda olduğunu tümevarım ile gösteriniz. Bir tam ikili ağaçtaki iç düğümlerin sayısının yaprakların sayısından 1 tane az olduğunu sonuçlandırınız.

B.5-4

Tümevarımı n düğümlü boştan farklı bir ikili ağacın en az $\lfloor \lg n \rfloor$ yüksekliğe sahip olduğunu göstermek için kullanınız.

B.5-5 *

Bir ful ikili ağacın *iç yol uzunluğu* ağacın tüm iç düğümlerini üstlenen her bir düğümün derinliğinin toplamıdır. Aynı şekilde, bir tam ikili ağacın *dış yol uzunluğu* ağacın tüm yapraklarını üstlenen her bir yaprağın derinliğinin toplamıdır. iç yol uzunluğu i ve dış yol uzunluğu e olan n iç düğümlü bir tam ikili ağacı göz önünde bulunduralım. $e = i + 2n$ olduğunu kanıtlayınız.

B.5-6 *

Bir ikili T ağacındaki d derinlikte her bir yaprak ile $w(x) = 2^{-d}$ “ağırlık” ile birleştirelim ve T nin yapraklarının kümesi L olsun. $\sum_{x \in L} w(x) \leq 1$ olduğunu kanıtlayınız. (Bu eşitsizlik **Kraft eşitsizliği** olarak bilinir).

B.5-7 *

Eğer $L \geq 2$ ise o zaman her L yapraklı ikili ağacın $L/3$ ile $2L/3$ arasında yaprağa sahip (ya da içeren) bir altağaç içerdiğini gösteriniz.

Problemler
B-1 Çizge renklendirme

Verilen bir yönsüz $G = (V, E)$ çizgesi için G nin bir **k -renklendirmesi** her $(u, v) \in E$ ayrıtı için $c(u) \neq c(v)$ olacak şekilde bir $c : V \rightarrow \{0, 1, \dots, k-1\}$ fonksiyonudur. Diğer bir deyişle, $0, 1, \dots, k-1$ sayıları k renkleri temsil eder ve bitişik matrisler farklı renklere sahip olmak zorundadırlar.

a. Her ağacın 2-renklendirilebilir olduğunu gösteriniz.

b. Aşağıdakilerin denk olduğunu gösteriniz:

1. G iki parçalıdır.
 2. G 2-renklendirilebilirdir.
 3. G hiçbir tek sayı uzunluğunda çevrime sahip değildir.
- c. Bir G çizgesindeki her tepenin maksimum derecesi d olsun. Kanıtlayınız ki, $d + 1$ renk ile G yi boyayabiliriz.
- d. Eğer G $O(|V|)$ ayrıta sahip ise $O(\sqrt{|V|})$ renk ile G yi renklendirebildiğimizi gösteriniz.

B-2 Dost çizgeler

Aşağıdaki her bir ifadeyi yönsüz çizgeler hakkındaki a bir teorem olarak başka şekilde ifade ediniz ve sonra kanıtlayınız. Dostluğun simetrik fakat yansımali olmadığını kabul ediniz.

- a. En az iki kişilik herhangi bir grup, grup içindeki arkadaşlarla aynı sayıda en az iki kişi içerir.
- b. 6 kişilik her grup ya en az üç ortak arkadaş ya da en az üç ortak yabancıyı içerir.
- c. Her kişinin arkadaşlarının en az yarısı, bu kişinin bir üye olmadığı alt gruba ait olacak şekilde herhangi bir insan grubu iki altgruba bölünebilir.
- d. Eğer bir gruptaki herkes, grup içindeki kişilerin en az yarısının arkadaşı ise herkes iki arkadaşı arasına oturacak şekilde grup bir masanın etrafında oturabilir.

B-3 İkiye bölünen ağaçlar

Çizgeler üzerinde çalışan çoğu böl-ve-yönet algoritmaları tepelerin bir parçalanışı tarafından indüklenen iki yaklaşık olarak eşit ölçümlü altçizgelere iki parça olarak altçizgelere gerektirir. Bu problem ayrıtların bir küçük sayısını kaldırarak oluşturulan ağaçların ikiye bölümünü inceler. Her ne zaman iki düğüm ayrıtlar kaldırıldıktan sonra aynı altağaçta son bulursa o zaman onlar aynı bölüntüde yer almak zorundadırlar olgusuna ihtiyaç duyarız.

- a. Bir tek ayrıt kaldırılarak, $|A| \leq 3n/4$ ve $|B| \leq 3n/4$ olacak şekilde her n -tepli ikili ağacın tepelerini iki A ve B kümesine bölebildiğimizi gösteriniz.
- b. Kısım (a) içindeki $3/4$ sabitinin, $|A| = 3n/4$ e sahip bir ayrıtın kaldırılması üzerinde bir basit ikili ağacın en dengeli bölüntüsünün bir örneği ile verilen en kötü durumda optimal olduğunu gösteriniz.

- c. En fazla $O(\lg n)$ ayırıt kaldırarak, $|A| = \lfloor n/2 \rfloor$ ve $|B| = \lceil n/2 \rceil$ olacak şekilde her n -tepeli ikili ağacın tepelerini iki A ve B kümesine bölebildiğimizi gösteriniz.

Ek notları

Sembolik mantığın gelişmesine G. Boole önderlik etmiş ve 1854 yılında yayınlanan bir kitabında temel küme simgelerinin pek çoğunu tanıtmıştır. Modern küme teorisi 1874-1895 dönemi boyunca G. Cantor tarafından yaratılmıştır. Cantor, ağırlıklı olarak sonsuz kardinaliteli kümeler üzerine yoğunlaşmıştır. Matematiksel formüllerin birçok çeşidini ima etmek için “fonksiyon” terimini kullanan G. W. Leibniz’e dayanmaktadır. Onun sınırlı tanımı çok defa genelleştirilmiştir. Königsberg şehrindeki yedi köprüden her birini yalnız bir kere geçerek başlangıç noktasına dönmenin imkânsız olduğunu L. Euler’in kanıtladığı 1736 yılında çizge teorisi ortaya çıkmıştır.

Harary [160] taki kitap çizge teorideki birçok tanım ve sonucun bir kullanışlı özetini temin eder.

C

Sayma ve Olasılık

Bu ek basit kombinasyonları ve olasılık teorisini incelemektedir. Eğer bu alanda iyi bir altyapıya sahipseniz, bu ekin başlangıcına çok derinlere girmeden göz gezdirip; sonraki bölümlere odaklanmak isteyebilirsiniz. Bu kitabın bölümlerinin çoğu olasılık içermez, ama bazı bölümlerde bu gereklidir.

Kısım C.1 hesaplama teorisinin, hesaplama permütasyon ve kombinasyonları için standart formüllerini içeren basit sonuçlarını inceler. Olasılık dağılımı ile ilgili olasılık aksiyomları ve temel kuramlar Kısım C.2'yi oluşturur. Rassal değişkenler, olasılık ve değişkenlik özellikleri ile incelenerek C.3'de tanıtıldı. Kısım C.4'de Bernoulli denemeleri çalışılarak çıkarılan geometrik ve binom dağılımları incelenmektedir. Binom dağılımları çalışması Kısım C.5'de devam ediyor. Dağılımlarla ilgili kalan konuları ileri düzeyde tartışılmasına devam edilmektedir.

C.1 Sayma

Hesap teorisi bütün seçenekleri gerçekten numaralandırılmaksızın “Kaç tane” sorusuna cevap vermeye başlar. Örnek olarak, “Kaç tane farklı n -bit sayı vardır?” yada “ n farklı elemanda kaç tane sıralama vardır?” diye sorabiliriz. Bu bölümde, sayma teorisinin elemanlarını gözden geçiriyoruz. Bazı materyaller setteki temel anlamlara ulaştığı için Kısım B.1'deki materyalleri gözden geçirerek başlamak isteyebilirsiniz.

Toplam ve çarpım kuralı

Bazen, saymak istediğimiz nesnelerin kümesini, ayrık kümelerin birleşimi ya da kümelerin Kartezyen çarpımı olarak ifade edebiliriz.

Toplama kuralına göre iki ayrık kümenin birinden bir eleman seçme yollarının tümü kümelerin nicelik (cardinalities) toplamıyla aynıdır. A ve B ortak elemanı olmayan iki küme ise (B.3) denklemine göre $|A \cup B| = |A| + |B|$. Örnek

olarak; araba plakası üzerindeki her bir yazı harf ya da sayıdan oluşur. Harf adına 26, rakam olarak da 10 farklı alternatif olduğundan her bir plaka için ihtimal sayısı $26 + 10 = 36$ dır.

Çarpım kuralına göre, sıralı bir çift seçmek için birinci elemanla, ikinci eleman alma ihtimallerini birbiriyle çarpmak gerekir. A ve B iki sonlu kümeysse $|A \times B| = |A| \cdot |B|$ dir ve bu aslında (B.4) denklemdir. Örneğin, eğer bir dondurmacıda 28 çeşit dondurma üzerine konulabilecek 4 farklı malzeme ile sunuluyorsa, o zaman dondurma üzeri süs olacak şekilde toplam dondurma çeşidi sayısı şu olacaktır: $28 \cdot 4 = 112$.

Karakter dizileri

Sonlu bir S kümesi üzerinde tanımlı bir **karakter dizisi**, S in elemanlarının bir dizilişinden oluşur. Örnek olarak, 3 uzunluklu ikili diziler 8 tanedir:

000, 001, 010, 011, 100, 101, 110, 111.

k uzunluklu karakter dizilerine **k -karakter dizisi** de diyebiliriz. s **karakter dizisinin** alt **karakter dizisi** olan s' , s deki ardışık elemanların sıralı dizileridir. Uzunluğu k olan alt dizilere **k - alt karakter dizisi** denir. Örneğin, 010, 01101001de 3 - karakter alt dizisidir ancak 111, 01101001de alt karakter dizisi değildir.

S üzerindeki k -karakter dizilerini, k -değişkenli S^k da Kartezyen çarpımın elemanları gibi S kümesi üzerindeki k -karakter dizileri inceleyebiliriz. Bu yüzden k -uzunluklu diziler $|S|^k$ tanedir. Örnek olarak ikili k -karakter dizilerinin sayısı 2^k dır. Sezgisel olarak n -küme üzerindeki k -karakter dizilerini oluşturmak için ilk elemanı seçmek için n yola sahibiz; bu seçeneklerden her biri için ikinci elemanı seçmek için n yola sahibiz; ve aynı şekilde k -kez devam eder. Bu yapı da bizi k -kat çarpıma yöneltir: $n \cdot n \cdots n = n^k$. Bu sayı, k -karakter dizilerinin sayısıdır.

Permütasyon

S sonlu kümesinde **permütasyon** bütün elemanların sadece bir kez kullanılması durumunda S nin bütün elemanlarının sıralı dizisidir. Örnek olarak; eğer $S = \{a, b, c\}$ ise S , 6 permütasyona sahiptir. Bunlar:

$abc, acb, bac, bca, cab, cba$ dır.

n elemanlı bir kümede $n!$ permütasyon vardır. İlk dizi elemanını n farklı yola seçebildiğimiz için ikinciye $n - 1$ yolla, üçüncüyü $n - 2$ yolla vs.

S deki **k -permütasyon**, elemanlar dizide birden fazla kullanılmaksızın S deki k elemanın sıralı dizisidir. (Bu yüzden, sıralı bir permütasyon n -kümedeki n -permütasyondur). $\{a, b, c, d\}$ kümesindeki 2-permütasyonlar

$ab, ac, ad, ba, bc, bd, ca, cb, cd, da, db, dc$ olacak şekilde 12 tanedir.

n -kümeli k -permütasyon sayısı,

$$n(n-1)(n-2)\cdots(n-k+1) = \frac{n!}{(n-k)!}, \quad (\text{C.1})$$

ilk elemanı seçerken n yola sahibiz, ikinci elemanı seçmek için $n-1$ yol ve k elemanı seçene kadar bu şekilde devam eder, en son seçim kalan $n-k+1$ elamandan yapılır.

Kombinasyon

n -kümeli S in **k -kombinasyonları** basitçe S nin k -alt kümeleridir. Örnek olarak, $\{a, b, c, d\}$ 4-kümesinin 2-kombinasyonları:

ab, ac, ad, bc, bd, cd .

(Burada 2-altküme $\{a, b\}$ yi kısaca ab ile gösteriyoruz, ve diğerleri de aynı şekilde) n -kümesinden k farklı eleman seçerek, n kümesinde k -kombinasyon kurabiliriz. Elemanları seçtiğimiz sıra önemli değildir.

n -setteki k -permütasyon sayısına göre, n kümedeki k -kombinasyon sayısını ifade edebiliriz. Her k -kombinasyon, onun elemanlarının tam olarak $k!$ permütasyona sahiptir. Bu yüzden n -kümedeki k kombinasyon sayısı, k -permütasyonun $k!$ ile bölünmesinden elde edilen sayıdır. Denklem (C.1)den bu eşitlik;

$$\frac{n!}{k!(n-k)!}. \quad (\text{C.2})$$

$k=0$ için, bu formül, $0!=1$ olduğu için, n -setten 0 eleman seçme yollarının sayısının 1 olduğunu söyler.

Binom katsayıları

$\binom{n}{k}$ formülü (“ n seç k ” diye okunur) n -kümedeki k -kombinasyon sayısını ifade eder. (C.2) denkleminden

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \text{ elde ederiz.}$$

Bu formül k ve $n-k$ da simetriktr:

$$\binom{n}{k} = \binom{n}{n-k}. \quad (\text{C.3})$$

Bu sayılar **binom açılımına** bağlı olarak, **binom katsayıları** olarakda bilinir.

$$(x + y)^n = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k} . \quad (\text{C.4})$$

$x = y = 1$ olduğunda binom açılımının özel durumu oluşur:

$$2^n = \sum_{k=0}^n \binom{n}{k} .$$

Bu formül içerdikleri 1 lerin sayısı ile 2^n ikili n -dizi hesabı için uygundur: $\binom{n}{k}$ ikili k -dizileri k tane 1 lerin içerir. Çünkü 1 leri yerleştirebileceğimiz n konumdan k adet seçim yapmak için (n, k) farklı yol vardır.

Birçok birim binom katsayılarını içerir. Bu bölümün sonundaki alıştırmalar birkaç deneme yapmanız için imkan tanır.

Binom sınırları

Bazen binom katsayılarının büyüklüğünü sınırlamaya ihtiyaç duyarız. $1 \leq k \leq n$ için alt sınırimız

$$\begin{aligned} \binom{n}{k} &= \frac{n(n-1) \cdots (n-k+1)}{k(k-1) \cdots 1} \\ &= \left(\frac{n}{k}\right) \left(\frac{n-1}{k-1}\right) \cdots \left(\frac{n-k+1}{1}\right) \\ &\geq \left(\frac{n}{k}\right)^k . \end{aligned}$$

Stirling'in yaklaşımından (3.18) türeyen $k! \geq (k/e)^k$ eşitsizliğinden yararlanarak, üst sınırı elde ederiz

$$\begin{aligned} \binom{n}{k} &= \frac{n(n-1) \cdots (n-k+1)}{k(k-1) \cdots 1} \\ &\leq \frac{n^k}{k!} \\ &\leq \left(\frac{en}{k}\right)^k . \end{aligned} \quad (\text{C.5})$$

$0 \leq k \leq n$ aralığındaki bütün k tam sayıları için sınırın sağlanmasını yapmak için tümevarım kullanabiliriz.

$$\binom{n}{k} \leq \frac{n^n}{k^k (n-k)^{n-k}}, \quad (\text{C.6})$$

$k = \lambda n$ için, $0 \leq \lambda \leq 1$ olduğu durumda, bu sınırı şu şekilde düzenleyebiliriz,

$$\begin{aligned} \binom{n}{\lambda n} &\leq \frac{n^n}{(\lambda n)^{\lambda n} ((1-\lambda)n)^{(1-\lambda)n}} \\ &= \left(\left(\frac{1}{\lambda} \right)^\lambda \left(\frac{1}{1-\lambda} \right)^{1-\lambda} \right)^n \\ &= 2^{n H(\lambda)}, \end{aligned}$$

burada

$$H(\lambda) = -\lambda \lg \lambda - (1-\lambda) \lg(1-\lambda) \quad (\text{C.7})$$

(ikili) entropi fonksiyonu olarak adlandırılmaktadır. Kolaylık için $0 \lg 0 = 0$ alıyoruz öyle ki $H(0) = H(1) = 0$ olmaktadır.

Alıştırmalar

C.1-1

Bir n -karakter kaç tane k -alt karakter dizisi vardır? (Farklı durumlardaki aynı k -alt karakter dizisinin farklı olacağını dikkate alın) Bir n karakter dizisinde toplam kaç tane alt karakter dizisi vardır?

C.1-2

Bir n -girdi, m -çıktılı **boole** fonksiyonu, $\{\text{True}, \text{False}\}^n$ den $\{\text{True}, \text{False}\}^m$ e kadar olan bir fonksiyondur. Kaç tane n -girdi, 1-çıktılı boole fonksiyonu vardır? Kaç tane n -girdili ve, m -çıktılı ikili boole fonksiyonu vardır?

C.1-3

Yuvarlak bir toplantı masası etrafında n profesör kaç şekilde oturabilir? Eğer biri ötekinin döndürülmüş şekli ise iki oturmayı aynı kabul edin.

C.1-4

$\{1, 2, \dots, 99\}$ sayı kümesinden kaç şekilde, toplamı çift olan üç ayrı numara seçebiliriz?

C.1-5

$0 < k \leq n$ için

$$\binom{n}{k} = \frac{n}{k} \binom{n-1}{k-1} \quad (\text{C.8})$$

eşitliğini kanıtlayınız.

C.1-6

$0 \leq k < n$ için

$$\binom{n}{k} = \frac{n}{n-k} \binom{n-1}{k}$$

eşitliğini kanıtlayınız.

C.1-7

n nesneden k tane seçmek için nesnelerden birinin farklı olduğunu ve farklı olan nesnenin seçilmesini düşünün. Bu yaklaşımı,

$$\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$$

i kanıtlamak için kullanınız.

C.1-8

C.1-7 alıştırmalarını kullanarak, $n = 0, 1, \dots, 6$ için ve üstte $\binom{0}{0}$, sonraki sırada $\binom{1}{0}$ ve $\binom{1}{1}$ vs. olan $0 \leq k \leq n$ aralığındaki $\binom{n}{k}$ binom katsayıları için bir tablo yapınız. Binom katsayılarının bu tablosuna **Pascal Üçgeni** denir.

C.1-9

$$\sum_{i=1}^n i = \binom{n+1}{2} \quad \text{olduğunu kanıtlayınız.}$$

C.1-10

$n \geq 0$ ve $0 \leq k \leq n$ aralığındaki herhangi bir tam sayı için $k = \lfloor n/2 \rfloor$ ya da $k = \lceil n/2 \rceil$ olduğunda $\binom{n}{k}$ ifadesinin maksimum değerini aldığını kanıtlayınız.

C.1-11 *

Herhangi bir $n \geq 0, j \geq 0, k \geq 0$ ve $j + k \leq n$ tam sayıları için

$$\binom{n}{j+k} \leq \binom{n}{j} \binom{n-j}{k} \quad (\text{C.9})$$

olup olmadığını tartışın.

Bu formül n dışında $j + k$ öge seçimi için hem cebirsel kanıt hem de bir yöntemeye dayanan çıkarımı sağlar. Bu eşitliğin sağlanmadığı bir örnek veriniz.

C.1-12 *

$0 \leq k \leq n/2$ aralığındaki bütün k tam sayıları üzerinde (C.6) eşitsizliğini kanıtlamak için tümevarım kullanınız ve $0 \leq k \leq n$ aralığındaki bütün k tam sayılarına genişletmek için (C.3) denklemini kullanınız.

C.1-13 *

$$\binom{2n}{n} = \frac{2^{2n}}{\sqrt{\pi n}} (1 + O(1/n)) \quad (\text{C.10})$$

ifadesini kanıtlamak için Stirling yaklaşımını kullanınız.

C.1-14 *

$H(\lambda)$ entropy fonksiyonunu (türevle) farklılaştırarak, $\lambda = 1/2$ de maksimum değerini aldığını gösteriniz.

C.1-15 *

Herhangi bir $n \geq 0$ tam sayısı için

$$\sum_{k=0}^n \binom{n}{k} k = n2^{n-1} \quad (\text{C.11})$$

ifadesini kanıtlayınız.

C.2 Olasılık

Tasarı, olasılık analizi ve rassal algoritmalar için olasılık önemli bir araçtır. Bu bölümde temel olasılık teorisi incelenmiştir.

Elemanları **başlangıç olayları** olarak adlandırılan S **örnek uzayı**na göre olasılık belirleriz. Her bir başlangıç olayını deneyimizin olası sonucu gibi düşünebiliriz. İki ayırt edilebilir parayı yazı tura atma deneyimiz için, her bir atış sonucu yazı(Y) ya da tura(T) gelecek şekilde, örnek uzayı $\{T, Y\}$ üzerindeki olası 2-dizilerin tamamını içerecek şekilde inceleyebiliriz:

$$S = \{TT, TY, YT, YY\}.$$

Olay örnek uzay S in alt kümesidir. Örnek olarak, yazı tura atma deneyimizde, bir yazı ve bir tura gelme olayı $\{TY, YT\}$ dır. S olayı **kesin olay** olarak adlandırılır, ve $\emptyset$ olayı **boş olay** olarak adlandırılır. $A \cap B = \emptyset$ ise A ve B olayları **ayrışık**tır. Bazen bir başlangıç olayı $s \in S$ i, $\{s\}$ olayı gibi ele alırız. Tanımdan, bütün başlangıç olayları ayrışiktır.

Olasılık aksiyomları

S örnek uzayı üzerindeki bir **olasılık dağılımı** $\Pr\{\cdot\}$, S olaylarından, aşağıdaki **olasılık aksiyomlarını** sağlayan reel sayılara kadar olan bir eşlemedir:

1. Herhangi bir A olayı için $\Pr\{A\} \geq 0$.
2. $\Pr\{S\} = 1$.
3. Ayrışık iki A ve B olayları için $\Pr\{A \cup B\} = \Pr\{A\} + \Pr\{B\}$. Daha genel olarak, ikili ayrışık olan $A_1, A_2, \dots$ olaylar dizisi (sonlu yada sayılabilir sonsuz) için

$$\Pr\left\{\bigcup_i A_i\right\} = \sum_i \Pr\{A_i\}.$$

$\Pr\{A\}$ yı A olayının **olasılığı** olarak adlandırırız. 2 aksiyomunun normalleştirme ihtiyacını dikkate alıyoruz: doğal ve uygun olması dışında, kesin olayların olasılığı 1 olmasına rağmen hiçbir olay kesin değildir.

Çeşitli sonuçlar hemen bu aksiyomlardan ve temel küme teorisinden çıkar. (B.1 bölümüne bakınız). $\emptyset$ boş olayının olasılığı $\Pr\{\emptyset\} = 0$ dır. $A \subseteq B$ olduğunda $\Pr\{A\} \leq \Pr\{B\}$ olur. $S - A$ (A nın **tümleyen**i) olayını göstermek için $\overline{A}$ kullanarak, $\Pr\{\overline{A}\} = 1 - \Pr\{A\}$ elde ederiz. Herhangi iki A ve B olayları için

$$\Pr\{A \cup B\} = \Pr\{A\} + \Pr\{B\} - \Pr\{A \cap B\} \quad (\text{C.12})$$

$$\leq \Pr\{A\} + \Pr\{B\}. \quad (\text{C.13})$$

¹Genel olasılık dağılımları için S kümesinin olay olmayan bazı alt kümeleri olabilir. Bu durum genellikle örnek uzayın sayılamayan sonsuzlukta olduğu durumlarda ortaya çıkar. Hangi alt kümelerin olay olup olmadığıyla ilgili temel gereklilik, örnek uzay olay kümesinin şu işlemlerde kapalı olmasıdır: bir olayın tümleyenini alma, sonlu ya da sayılabilir sayıdaki olayların birleşimini ve kesişimini alma. Ele alacağımız bir çok dağılım sonlu ya da sayılabilir örnek uzayıdır ve dolayısıyla bu örnek uzayların tüm alt kümelerini genel olarak olaylar olarak ele alacağız. Bu alanda tek istisna, kısaca değineceğimiz, sürekli tek tip olasılık dağılımlarıdır.

Yazı tura atma örneğimizde, her bir dört olayın olasılığını $1/4$ farz edelim. O zaman en az bir tura gelme olasılığı,

$$\begin{aligned} \Pr\{TT, TY, YT\} &= \Pr\{TT\} + \Pr\{TY\} + \Pr\{YT\} \\ &= 3/4. \end{aligned}$$

Bunun yerine, tam olarak 1 den az tura gelme olasılığı $\Pr\{YY\} = 1/4$ olduğundan, en az 1 tura gelme olasılığı $1 - 1/4 = 3/4$ tür.

Ayrık olasılık dağılımı

Bir olasılık dağılımı, eğer sonlu yada sayılabilir sonsuz örnek uzayında tanımlanmışsa, **ayrıktır**. S bir örnek uzay olsun. Herhangi bir A olayı için

$$\Pr\{A\} = \sum_{s \in A} \Pr\{s\} ,$$

başlangıç koşulunda, özellikle Anın elemanı olduğunda, ayrışıktır. Eğer S sonlu ve her $s \in S$ başlangıç koşulu

$$\Pr\{s\} = 1/|S|$$

olasılığa sahipse, o zaman S üzerinde **tek tip olasılık dağılımı** vardır. Böyle bir durumda bu deneme sıkça ‘ S den rassal bir eleman seçme’ olarak tanımlanır. Örnek olarak, yazı tura atma sürecini düşünelim, bir kere tura gelme olasılığı yazı gelme olasılığıyla aynı olduğunda bu, $1/2$ dir. Eğer n kere yazı tura atarsak, S örnek uzayında tanımlanan tek tip olasılık dağılımı $S\{T, Y\}^n$ e sahip oluruz ve eleman sayısı 2^n dir. S deki her bir başlangıç olayını $\{T, Y\}$ üzerinde dizgi uzunluğu olarak gösterebiliriz, her bir dizgi $1/2$ olasılıkla meydana gelir.

$$A = \{\text{tamamen } k \text{ tura ve tamamen } n - k \text{ yazı oluşur}\}$$

$\{T, Y\}$ üzerindeki n uzunluklu $\binom{n}{k}$ dizileri tamamen k tura içerdiğinden, $|A| = \binom{n}{k}$ ölçümlü olan A olayı, S in alt kümesidir. Bu olayların olasılığı bu yüzden $\Pr\{A\} = \binom{n}{k}/2^n$ dir.

Sürekli düzgün olasılık dağılımı

Sürekli düzgün olasılık dağılımı, örnek uzayın tüm alt kümelerini durumlar olarak kabul etmeyen bir olasılık dağılımı örneğidir. Düzgün olasılık dağılımı, $a < b$ olmak üzere bir $[a, b]$ kapalı aralığı üzerinde tanımlıdır. Beklentimiz $[a, b]$ kapalı aralığındaki her bir noktanın “eşit olasılıklı” olması gerektirir. Fakat sayılamaz çoklukta nokta olduğundan, bütün noktalara benzer sonlu, pozitif olasılık versek bile 2. ve 3. aksiyomları aynı zamanda gerçekleştiremeyiz. Bu yüzden, aksiyom-

lar bu olayları gerçekleyecek şekilde S nin bazı alt kümelerini bir olasılıkla ilişkilendirmek isteyeceğiz.

Herhangi bir $[c, d]$ kapalı aralığı için, burada $a \leq c \leq d \leq b$ olmak üzere, **sürekli düzgün olasılık dağılımı** $[c, d]$ olayının olasılığını

$$\Pr\{[c, d]\} = \frac{d - c}{b - a}$$

olacak şekilde tanımlar. Herhangi bir $x = [x, x]$ noktası için x in olasılığının 0 olduğuna dikkat edelim. Eğer $[c, d]$ kapalı aralığındaki bitiş noktalarını kaldırırsak (c, d) açık aralığını elde ederiz. $[c, d] = [c, c] \cup (c, d) \cup [d, d]$ olduğundan, aksiyom 3 vize $\Pr\{[c, d]\} = \Pr\{(c, d)\}$ eşitliğini verir. Genellikle olayların sürekli düzgün olasılık dağılım kümesi $[a, b]$ örnek uzayının açık ve kapalı aralıklarının sonlu veya sayılabilir birleşimlerinden elde edilebilenlerinin yanı sıra daha karmaşık kümeleri de içerir.

Koşullu olasılık ve bağımsızlık

Bazen bir deneyin sonucu hakkında önceden kısmi bir bilgiye sahibiz. Örneğin, bir arkadaşımızın hilesiz iki madeni para atışı yaptığının ve bunlardan en az birinin tura geldiğini, söylediğini varsayalım. İkisinin de tura gelme olasılığı nedir? Bu bilgi ikisinin de yazı gelme olasılığını eliyor. Geri kalan üç temel olayın eş olasılıklı olup, her birinin $1/3$ olasılığına sahip olduğunu çıkarabiliriz. Bu temel olaylardan yalnızca biri iki turayı gösterdiği için, sorumuzun cevabı $1/3$ olacaktır.

Koşullu olasılık, bir deneyin sonucu hakkında önceden sahip olduğumuz kısmi bilgi kavramını formülize eder. Farklı B olayında geçen bir A olayının, $\Pr = \{B\} \neq 0$ olduğu sürece, **koşullu olasılığı**

$$\Pr\{A \mid B\} = \frac{\Pr\{A \cap B\}}{\Pr\{B\}} \quad (\text{C.14})$$

olarak tanımlanır. (“ $\Pr\{A \mid B\}$ ” yi “ B verilmişken A nın olasılığı” olarak okuyoruz.) Sezgisel olarak, B olayının oluşu verildiğinden, A olayının oluşu da $A \cap B$ dir. Yani $A \cap B$, A ve B nin ikisinde de meydana gelen sonuçların kümesidir. Sonuç B de geçen temel olaylardan bir tanesi olduğundan, B deki tüm temel olayların olasılıklarını $\Pr\{B\}$ ye bölerek normalize ederiz. Dolayısıyla da, olasılıklar toplamı 1 olur. Verilen B deki A nın koşullu olasılığı, $A \cap B$ olayının olasılığının, B olayının olasılığına oranıdır. Yukarıdaki örnekte, A iki madeni paranın da tura gelme olayı ve B , en az birinin tura gelme olayı olsun. O zaman, $\Pr\{A \mid B\} = (1/4)/(3/4) = 1/3$ dür.

Eğer

$$\Pr\{A \cap B\} = \Pr\{A\} \Pr\{B\} \quad (\text{C.15})$$

sağlanırsa iki olay **bağımsızdır**. Buna denk olarak da, $\Pr = \{B\} \neq 0$ olduğunda

$$\Pr\{A \mid B\} = \Pr\{A\}$$

olmalıdır.

Örneğin, iki hilesiz madeni parayı attığımızı ve sonuçlarının bağımsız olduğunu varsayalım. O zaman ikisinin de tura gelme olasılığı $(1/2)/(1/2) = (1/4)$ dür. Şimdi bir olayda ilk paranın tura ve diğer bir olayda da paraların yüzlerinin farklı geldiğini varsayalım. Bu olayların her biri $1/2$ olasılığına sahip ve iki olayın da aynı anda olma olasılığı $1/4$ dür. Böylece, bağımsızlık tanımına göre olaylar bağımsızdırlar—iki olayında ilk paraya bağlı olduğunu düşünüyor olsanız bile. Sonuç olarak, iki parayı aynı anda attığımızı ve dolayısıyla da ikisinin aynı anda tura veya yazı gelme olasılıkları eşittir. O zaman her bir paranın tura gelme olasılığı $1/2$ dir, fakat ikisinin de tura gelme olasılığı $1/2 \neq (1/2)/(1/2)$ olacaktır. Dolayısıyla birinin tura gelmesi olayıyla, diğerinin tura gelmesi olayları bağımsız değildir.

Eğer bütün $1 \leq i \leq j \leq n$ için

$$\Pr\{A_i \cap A_j\} = \Pr\{A_i\} \Pr\{A_j\}$$

ise, $A_1, A_2, \dots, A_n$ olayların koleksiyonuna **ikili bağımsızdır** denir. Eğer koleksiyonun her $A_{i_1}, A_{i_2}, \dots, A_{i_k}$ k -altkümesi $2 \leq k \leq n$ ve $1 \leq i_1 < i_2 < \dots < i_k \leq n$ olduğunda

$$\Pr\{A_{i_1} \cap A_{i_2} \cap \dots \cap A_{i_k}\} = \Pr\{A_{i_1}\} \Pr\{A_{i_2}\} \dots \Pr\{A_{i_k}\}$$

sağlanıyorsa, koleksiyonun olayları **(karşılıklı) bağımsızdır** deriz.

Örneğin iki hilesiz madeni parayı attığımızı varsayalım. A_1 ilkinin tura gelme olayı, A_2 ikincinin tura gelme olayı ve A_3 ikisinin farklı gelme olayları olsun.

$$\Pr\{A_1\} = 1/2,$$

$$\Pr\{A_2\} = 1/2,$$

$$\Pr\{A_3\} = 1/2,$$

$$\Pr\{A_1 \cap A_2\} = 1/4,$$

$$\Pr\{A_1 \cap A_3\} = 1/4,$$

$$\Pr\{A_2 \cap A_3\} = 1/4,$$

$$\Pr\{A_1 \cap A_2 \cap A_3\} = 0$$

olaylarına sahibiz.

$1 \leq i \leq j \leq 3$ için A_1, A_2 ve A_3 olayları ikili bağımsız olduğundan, $\Pr\{A_i \cap A_j\} = \Pr\{A_i\} \Pr\{A_j\} = 1/4$, eşitliğine sahibiz. Fakat $\Pr\{A_1 \cap A_2 \cap A_3\} = 0$ ve $\Pr\{A_1\} \Pr\{A_2\} \Pr\{A_3\} = 1/8 \neq 0$ olduğundan, olaylar karşılıklı bağımsız değildir.

Bayes teoremi

(C.14) koşullu olasılık tanımı ve $A \cap B = B \cap A$ değişmelilik yasasından, ikisi de sıfır olasılığına sahip olmayan A ve B olayları için

$$\begin{aligned}\Pr\{A \cap B\} &= \Pr\{B\} \Pr\{A \mid B\} \\ &= \Pr\{A\} \Pr\{B \mid A\}\end{aligned}\tag{C.16}$$

elde edilir. $\Pr\{A \mid B\}$ yi çözmek için, **Bayes teoremi** olarak bilinen

$$\Pr\{A \mid B\} = \frac{\Pr\{A\} \Pr\{B \mid A\}}{\Pr\{B\}}\tag{C.17}$$

eşitliğini elde ederiz. $\Pr\{B\}$ böleni, aşağıdaki gibi formülize edebileceğimiz bir normalleştirme sabitidir. $B = (B \cap A) \cup (B \cap \bar{A})$ ve $B \cap A$ ile $B \cap \bar{A}$ karşılıklı ayırık olaylar olduğu için

$$\begin{aligned}\Pr\{B\} &= \Pr\{B \cap A\} + \Pr\{B \cap \bar{A}\} \\ &= \Pr\{A\} \Pr\{B \mid A\} + \Pr\{\bar{A}\} \Pr\{B \mid \bar{A}\}\end{aligned}$$

dir. Denklem (C.17) de değişken dönüşümü yaparak

$$\Pr\{A \mid B\} = \frac{\Pr\{A\} \Pr\{B \mid A\}}{\Pr\{A\} \Pr\{B \mid A\} + \Pr\{\bar{A}\} \Pr\{B \mid \bar{A}\}}\tag{C.18}$$

Bayes teoreminin denk bir formunu elde ederiz.

Bayes teoremi koşullu olasılıkların hesaplanmasını kolaylaştırabilir. Örneğin, bir hilesiz madeni paraya ve her zaman tura gelen hileli bir madeni paraya sahip olduğumuzu varsayalım. Üç bağımsız olayı içeren bir deney yürütelim: bu iki paradan birini rastgele seçelim ve iki defa art arda atalım. İki sefer de tura geldiğini varsayalım. Bunun hileli para olma olasılığı nedir?

Bu problemi Bayes teoremini kullanarak çözelim. A hileli parayı seçtiğimiz olay ve B de iki defa tura gelmesi seçilen olay olsun. $\Pr\{A \mid B\}$ yi belirlemek istiyoruz. $\Pr\{\bar{A}\} = 1/2$, $\Pr\{A\} = 1/2$, $\Pr\{B \mid A\} = 1$ ve $\Pr\{B \mid \bar{A}\} = 1/4$ değerlerini sahibiz. Dolayısıyla da

$$\begin{aligned}\Pr\{A \mid B\} &= \frac{(1/2) \cdot 1}{(1/2) \cdot 1 + (1/2) \cdot (1/4)} \\ &= 4/5\end{aligned}$$

dir.

Alıştırmalar

C.2-1

Profesör Rosencrantz hilesiz bir madeni parayı bir kez atсын. Profesör Guildenstern hilesiz bir parayı iki kez atсын. Profesör Rosencrantz'in Profesör Guildenstern'den daha fazla tura elde etme olasılığı nedir?

C.2-2

Boole eşitsizliği: $A_1, A_2, \dots$, olayların sonlu ya da sayılabilir çokluktaki bir dizisi olsun.

$$\Pr\{A_1 \cup A_2 \cup \dots\} \leq \Pr\{A_1\} + \Pr\{A_2\} + \dots \quad (\text{C.19})$$

olduğunu kanıtlayınız.

C.2-3

1 den 10 a kadar farklı sayılarla numaralandırılmış 10 kartı iyice karıştırdığımızı varsayalım. Daha sonra desteden, her seferinde bir tane olmak üzere, üç kart ayıralım. Ayırdığımız bu üç kartın sıralı(artan bir şekilde) olasılığı nedir?

C.2-4

$$\Pr\{A \mid B\} + \Pr\{\bar{A} \mid B\} = 1$$

olduğunu kanıtlayınız.

C.2-5

Herhangi bir $A_1, A_2, \dots, A_n$, olaylar koleksiyonu için

$$\Pr\{A_1 \cap A_2 \cap \dots \cap A_n\} = \Pr\{A_1\} \cdot \Pr\{A_2 \mid A_1\} \cdot \Pr\{A_3 \mid A_1 \cap A_2\} \dots \Pr\{A_n \mid A_1 \cap A_2 \cap \dots \cap A_{n-1}\}$$

olduğunu kanıtlayınız.

C.2-6 *

$0 < a < b$ olacak şekilde iki tane a ve b tam sayısını girdi olarak alan ve hilesiz madeni para atışını kullanarak $(b - a)/b$ olasılığına sahip kuyrukları ve a/b olasılığına sahip kafaları çıktı olarak üreten bir prosedür tanımlayınız. $O(1)$ olması gereken madeni para atışlarının beklenen sayısının sınırını veriniz (*İpucu:* İkili a/b yi gösteriniz).

C.2-7 *

İkili olarak bağımsız fakat $k > 2$ için karşılıklı bağımsız hiçbir alt kümesi olmayan n olayın bir kümesi nasıl inşa edilir?

C.2-8 *

Verilen C olayında, eğer

$$\Pr\{A \cap B \mid C\} = \Pr\{A \mid C\} \cdot \Pr\{B \mid C\}$$

ise, A ve B olayları **koşullu bağımsızdır**.

İki olayın bağımsız olmadığı fakat verilen bir üçüncü olayında koşullu bağımsız olduğu basit ama açık olmayan bir örnek veriniz.

C.2-9 *

Ödülün, üç perdeden birinin ardında saklı olduğu bir yarışma programında yarışmacı olduğunuzu düşünün. Ödülü kazanmak için doğru perdeyi seçmeniz ge-

rekmetektedir. Siz bir perdeyi seçtikten sonra henüz perde açılmadan şu andaki seçiminizi, sunucu geriye kalan perdelerden biriyle değiştirmek isteyip istemediğini sorar. Eğer değiştirirseniz şansınız ne kadar değişir? (Bu soru yarışmacıların genellikle bu ikilem içinde bırakan yarışma sunucusundan sonra ismi konan ünlü *Monty Hall problemidir*.)

C.2-10 *

Bir cezaevi gardiyanı üç mahkumdan birini özgür bırakmak için rastgele seçecek ve kalan ikisi idam edilecektir. Gardiyan hangisinin özgür bırakılacağını bilir ancak herhangi bir mahkuma ne olacağı ile ilgili bilgi vermesi yasaktır. Mahkumları X , Y ve Z olarak isimlendirelim. X mahkumu, Y ya da Z den hangisinin idam edileceğini, onlardan en az birinin öleceğini önceden bildiğini varsayarak, baş başayken gardiyana sorar. Ancak gardiyan X in durumuyla ilgili herhangi bir bilgiyi paylaşmazken, Y nin idam edileceğini söyler. X mahkumu şimdi daha mutludur çünkü kendisinin ya da Z mahkumunun serbest bırakılacağını anlar ve bu da kendisinin özgür kalma olasılığının $1/2$ olduğu anlamına gelir. Yaptığı bu hesap doğru mudur? Yoksa şansı hala $1/3$ müdür? Araştırınız.

C.3 Ayırık Rassal Değişkenler

Bir (*ayırık*) *rassal değişken* X , sonlu yada sayılabilir sonsuz bir örnek uzay S den reel sayılara bir fonksiyondur. Bu fonksiyon bir reel sayıyı, bir deneyin tüm muhtemel sonuçlarına ilişkilendirir, bu da bize sayıların sonuç kümesi üzerine indirgenmiş olasılık dağılımı ile çalışmamıza imkan verir. Rassal değişkenler ayrıca sayılamaz sonsuz kümeler için de tanımlanabilir, fakat bu durum teknik meseleler ortaya çıkarır ve amacımızın dışına çıkar. O halde, rassal değişkenleri ayırık olarak alacağız.

Bir rassal değişken X ve bir x reel sayısı için, $X=x$ olayını $\{s \in S : X(s) = x\}$ olarak tanımlayacağız, böylece,

$$\Pr \{X = x\} = \sum_{s \in S : X(s)=x} \Pr \{s\}$$

dir.

$f(x) = \Pr \{X = x\}$ fonksiyonu,

X rassal değişkeninin olasılık yoğunluğu fonksiyonudur. **Olasılık aksiyomlarından**, $\Pr \{X=x\} \geq 0$ ve $\sum_x \Pr \{X = x\} = 1$ dir.

Bir örnek olarak, sıradan bir 6-yüzlü zar çiftinin atılması deneyini göz önüne alalım. Örnek uzayda 36 muhtemel basit olay vardır. Kabul edelim ki, olasılık

dağılım fonksiyonu düzenli olsun, böylece her bir basit olay $s \in S$ eşit olasılıklı olarak: $\Pr\{s\} = 1/36$ dır. X rassal değişkenini, zarların üst yüze gelen değerleri maksimum olacak şekilde tanımlayalım. 36 sonucun 2 tanesi yani (1,3), (2, 3), (3, 3) (3, 2) sonuçları 3 değerini içerdiğinden, $\Pr\{X=3\} = 5/36$ dır.

Çoğunlukla aynı örnek uzay üzerinde, birçok rassal değişken tanımlarız. X ve Y rassal değişkenler ise,

$$f(x, y) = \Pr\{X=x \text{ ve } Y=y\}$$

fonksiyonu X ve Y nin **ortak olasılık yoğunluk fonksiyonu** denir. Bu sabit bir y değeri için,

$$\Pr\{Y=y\} = \sum_x \Pr\{X=x \text{ ve } Y=y\} \text{ dir}$$

ve benzer şekilde, sabit bir x değeri için,

$$\Pr\{X=x\} = \sum_y \Pr\{X=x \text{ ve } Y=y\} \text{ dir.}$$

Koşullu olasılığın (C.14) tanımını kullanarak,

$$\Pr\{X=x \mid Y=y\} = \frac{\Pr\{X=x \text{ ve } Y=y\}}{\Pr\{Y=y\}} .$$

Tüm x ve y değerleri için, $X=x$ ve $Y=y$ olayları **bağımsız** ise ya da buna denk olarak tüm x ve y değerleri için, $\Pr\{X=x \text{ ve } Y=y\} = \Pr\{X=x\} \Pr\{Y=y\}$ ise X ve Y rassal değişkenlerine, bağımsız denir.

Aynı örnek uzay üzerinde verilen bir rassal değişkenler kümesi, yeni rassal değişkenleri orjinallerinin toplamı, çarpımı ve diğer fonksiyonları olarak tanımlayabiliriz.

Bir Rassal Değişkenin Beklenen Değeri

Bir rassal değişkenin dağılımının en basit ve kullanışlı özeti aldığı değerlerin “ortalamasıdır”. Bir ayırık rassal değişken X in **beklenen değeri** (ya da eş anlamlı, **beklenti** veya **ortalama**) ,

$$E[X] = \sum_x x \cdot \Pr\{X=x\} \quad (\text{C.20})$$

şeklinde ve eğer toplam sonlu ya da mutlak yakınsak ise beklenen değer iyi tanımlıdır. Genellikle X in beklenen değeri μ_x ile gösterilir, fakat rassal değişken durumdan dolayı belli ise kısaca μ ile gösterilir.

İki adet hilesiz madeni para atma oyununu düşünelim. Her bir tura için 3\$ kazanıyor ve her bir yazı için 2\$ kaybediyorsunuz. Bu durumda rassal değişkenin

beklenen değeri, sizin kazancınızı gösterir, ve

$$\begin{aligned} E[X] &= 6 \cdot \Pr\{2 \text{ H's}\} + 1 \cdot \Pr\{1 \text{ H, } 1 \text{ T}\} - 4 \cdot \Pr\{2 \text{ T's}\} \\ &= 6(1/4) + 1(1/2) - 4(1/4) \\ &= 1. \end{aligned}$$

İki adet rassal değişkenin toplamının beklenen değeri, onların beklenen değerlerinin toplamına eşittir, yani, $E[X]$ ve $E[Y]$ iyi tanımlanmış ise,

$$E[X + Y] = E[X] + E[Y] \quad (\text{C.21})$$

dir. Bu özelliğe **beklentinin doğrusallığı** denir ve X ve Y bağımsız olmasalar dahi eşitlik sağlanır. Bu ayrıca, beklenen değerlerin sonlu ve yakınsak toplamlarına genişletilebilir. Beklentinin doğrusallığı, gösterge rassal değişkenleri kullanarak (bakınız Kısım 5.2) olasılıksal analizleri yapmamıza imkan veren önemli bir özelliktir.

Eğer X bir rassal değişken ise, herhangi bir $g(x)$ fonksiyonu yeni bir $g(X)$ rassal değişkeni tanımlar. Eğer $g(x)$ in beklentisi tanımlanmış ise,

$$E[g(X)] = \sum_x g(x) \cdot \Pr\{X = x\}$$

dir. Eğer $g(x) = ax$ olarak alırsak, herhangi bir a sabiti için

$$E[aX] = aE[X] \quad (\text{C.22})$$

dir. Sonuç olarak, beklentiler doğrusaldır: herhangi iki rassal değişken X ve Y , herhangi bir sabit a için

$$E[aX + Y] = aE[X] + E[Y] \quad (\text{C.23})$$

dir.

İki rassal değişken X ve Y bağımsız ve her biri tanımlanmış beklentiye sahip iken,

$$\begin{aligned} E[XY] &= \sum_x \sum_y xy \cdot \Pr\{X = x \text{ and } Y = y\} \\ &= \sum_x \sum_y xy \cdot \Pr\{X = x\} \Pr\{Y = y\} \\ &= \left(\sum_x x \cdot \Pr\{X = x\} \right) \left(\sum_y y \cdot \Pr\{Y = y\} \right) \\ &= E[X] E[Y]. \end{aligned}$$

Genel olarak, n rassal değişken $X_1, X_2, \dots, X_n$ karşılıklı bağımsız iseler,

$$E[X_1 X_2 \cdots X_n] = E[X_1] E[X_2] \cdots E[X_n] \quad (\text{C.24})$$

dir.

Bir X rassal değişkeni $\mathbb{N} = \{0, 1, 2, \dots\}$ doğal sayılar kümesi üzerinde değerler aldığı anda, bunun bekleneni için iyi bir formülü vardır.

$$\begin{aligned}
 E[X] &= \sum_{i=0}^{\infty} i \cdot \Pr\{X = i\} \\
 &= \sum_{i=0}^{\infty} i (\Pr\{X \geq i\} - \Pr\{X \geq i+1\}) \\
 &= \sum_{i=1}^{\infty} \Pr\{X \geq i\}
 \end{aligned} \tag{C.25}$$

dır. Çünkü her bir $\Pr\{X \geq i\}$ terimi i kere eklenip $i-1$ kere çıkarılmıştır. ($\Pr\{X \geq 0\}$ hariç, çünkü 0 kere eklenip hiç çıkarılmamıştır)

Bir içbükey fonksiyon $f(x)$ i bir X rassal değişkenine uyguladığımız zaman, **Jensen eşitsizliği** bize, beklentiler var ve sonlu olduğunda

$$E[f(X)] \geq f(E[X]) \tag{C.26}$$

yi sağlar. (Bir $f(x)$ fonksiyonu tüm x ve y için ve tüm $0 \leq \lambda \leq 1$ için, $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$ sağlanırsa içbükey fonksiyon adını alır.)

Varyans ve Standart Sapma

Bir rassal değişkenin beklenen değeri bize değerlerin nasıl “dağılıma” sahip olduğu hakkında bilgi vermez. Örneğin, eğer elimizde $\Pr\{X=1/4\} = \Pr\{X=3/4\} = 1/2$ ve $\Pr\{Y=0\} = \Pr\{Y=1\} = 1/2$ olacak şekilde X ve Y rassal değişkenleri varsa bu durumda $E[X]$ ve $E[Y]$ lerin her ikisinde $1/2$ dir, Y tarafından alınan asıl değerler ortalamadan X tarafından alınan asıl değerlerden daha uzaktır.

Varyans kavramı, matematiksel olarak bir rassal değişkenin değerlerinin ortalamadan ne kadar uzak olabileceğini açıklar. $E[X]$ beklenen değerli bir X rassal değişkeninin **varyansı**,

$$\begin{aligned}
 \text{Var}[X] &= E[(X - E[X])^2] \\
 &= E[X^2 - 2XE[X] + E^2[X]] \\
 &= E[X^2] - 2E[XE[X]] + E^2[X] \\
 &= E[X^2] - 2E^2[X] + E^2[X] \\
 &= E[X^2] - E^2[X]
 \end{aligned} \tag{C.27}$$

dir. Dikkat edilirse $E[X]$ bir reel sayı olup, bir rassal değişken olmadığından $E^2[X]$ de reel sayıdır, o halde $E[E^2[X]] = [E^2[X]]$ eşitliği gerçekleşir. Denklem (C.22)

de $a = [X]$ alınarak, $E[XE[X]] = E^2[X]$ eşitliği sağlanır. (C.27) Denklemini düzenleyerek bir rassal değişkenin karesinin beklenen değeri için:

$$E[X^2] = \text{Var}[X] + E^2[X] \quad (\text{C.28})$$

ifadesi elde edilir.

Bir X rassal değişkeninin varyansı ile aX in varyansı arasında ilişki vardır (bkz. Alıştırma C.3-10):

$$\text{Var}[aX] = a^2 \text{Var}[X].$$

X ve Y bağımsız rassal değişkenler iseler,

$$\text{Var}[X + Y] = \text{Var}[X] + \text{Var}[Y] \text{ dir.}$$

Genel olarak, eğer n rassal değişken $X_1, X_2, \dots, X_n$ karşılıklı bağımsız iseler,

$$\text{Var} \left[\sum_{i=1}^n X_i \right] = \sum_{i=1}^n \text{Var}[X_i] \quad (\text{C.29})$$

dir. Bir X rassal değişkeninin **standart sapması**, X varyansının pozitif kareköküdür. X rassal değişkeninin standart sapması genellikle σ_X ile gösterilir yada durumdan anlaşılıyorsa kısaca σ ile gösterilir. Bu notasyon ile, X in standart sapması σ^2 ile gösterilir.

Alıştırmalar

C.3-1

6-yüzlü bir standart zarı attığımızı düşünelim, üst yüze gelen iki sayının toplamının beklenen değeri nedir? Üste gelen maksimum değerlerin beklenen değeri nedir?

C.3-2

Bir $A[1..n]$ dizisi her bir permütasyonu eşit değere sahip, rastgele sıralanmış n farklı sayıyı içersin. Dizide ki en büyük indisli elemanın beklenen değeri nedir? Dizide ki en küçük indisli elemanın beklenen değeri nedir?

C.3-3

Bir karnaval oyunu bir kafeste üç zara sahiptir. Bir oyuncu 1 den 6 ya kadar sayılara bir dolar iddiada bulunabilir. Kafes sallandı ve ödül şöyle ödenecek. Eğer oyuncunun sayısı herhangi bir zarda çıkmadıysa, dolarını kaybeder. Aksi halde, $k = 1, 2, 3$ için oyuncunun sayısı k tane zarda çıkarsa, oyuncunun parası kendinde kalır ve k dolar daha kazanır. Bu karnaval oyununun bir kere oynanmasında beklenen kazanç nedir?

C.3-4

X ve Y negatif olmayan rassal değişkenler olsun. Buna göre,

$E[\max(X, Y)] \leq E[X] + E[Y]$ dir.

C.3-5 *

X ve Y bağımsız rassal değişkenler olsunlar. $f(X)$ ve $g(Y)$ nin f ve g fonksiyonlarının seçimlerinden bağımsız olduğunu kanıtlayınız.

C.3-6 *

X negatif olmayan bir rassal değişken ve $E[X]$ iyi tanımlanmış olsun.

Tüm $t > 0$ için,

$$\Pr\{X \geq t\} \leq E[X] / t \quad (\text{C.30})$$

Markov eşitsizliğini kanıtlayınız.

C.3-7 *

S bir örnek uzay olsun ve X ve X' , tüm $s \in S$ ler için $X(s) \geq X'(s)$ olacak şekilde rassal değişkenler olsunlar. Her t *sabit reel sayısı için*

$$\Pr\{X \geq t\} \geq \Pr\{X' > t\} \quad (\text{C.30})$$

olduğunu kanıtlayınız.

C.3-8

Hangisi daha büyük: bir rassal değişkenin karesinin beklenen değeri mi yoksa onun beklenen değerinin karesi mi?

C.3-9

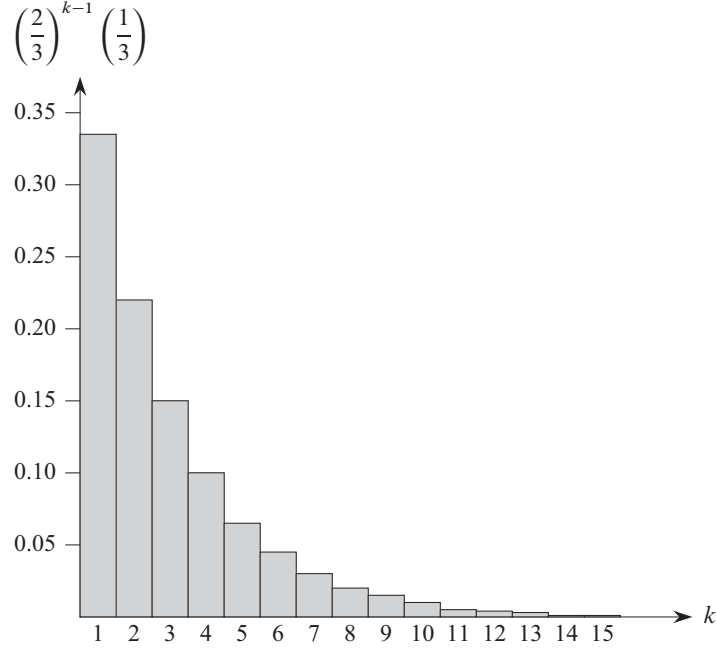
Sadece 0 ve 1 değerleri alan herhangi bir X rassal değişkeni için, $\text{Var}[X] = E[X]E[1 - X]$ olduğunu gösteriniz.

C.3-10

Varyansın (C.27) tanımından $\text{Var}[aX] = a^2\text{Var}[X]$ olduğunu gösteriniz.

C.4 Geometrik ve Binom Dağılımlar

p olasılıklı **başarı** sonucu ve $q = 1 - p$ olasılıklı **başarısızlık** sonucuna sahip iki sonuçlu bir madeni para atma deneyini, **Bernoulli denemesine** örnek olarak düşünebiliriz. Bernoulli denemelerinden bahsettiğimizde, aksi belirtilmedikçe, denemelerin karşılıklı bağımsız olduğu anlamı çıkarılacaktır, yani her bir başarı



Şekil C.1 Başarının $p = 1/3$ olasılığına ve başarısızlığın $q = 1 - p$ olasılığına sahip geometrik dağılım. Dağılımın beklentisi $1/p = 3$ tür.

olasılığı aynı şekilde p dir. Bernoulli denemelerinden iki önemli dağılım ortaya çıkar: geometrik dağılım ve binom dağılımı.

Geometrik Dağılım

Kabul edelim ki, elimizde her biri p olasılığına sahip başarı ve $q = 1 - p$ olasılığına sahip başarısızlık Bernoulli denemeleri dizisi olsun. Bir başarı elde etmeden önce kadar kaç deneme yapmak gerekir? Bir X rassal değişkenini başarı elde edesiye kadar olan denemelerin sayısı olarak tanımlayalım. Bu durumda $X, k \geq 1$ için $\{1, 2, \dots\}$ değerlerini alır. Bir başarı elde edesiye kadar $k - 1$ başarısız sonuç olacağından,

$$\Pr\{X = k\} = q^{k-1} p \quad (\text{C.31})$$

dir. (C.31) eşitliğini sağlayan bir olasılık dağılımına **geometrik dağılım** denir. Şekil C.1 bu tür bir dağılımı göstermektedir.

$q < 1$ olarak kabul edilirse, (A.8) özdeşliğini kullanarak bir geometrik dağılımın beklentisini hesaplayabiliriz:

$$\begin{aligned}
 E[X] &= \sum_{k=1}^{\infty} k q^{k-1} p \\
 &= \frac{p}{q} \sum_{k=0}^{\infty} k q^k \\
 &= \frac{p}{q} \cdot \frac{q}{(1-q)^2} \\
 &= \frac{p}{q} \cdot \frac{q}{p^2} \\
 &= 1/p.
 \end{aligned} \tag{C.32}$$

Böylece, ortalamada, başarı elde edesiye kadar $1/q$ deneme yapılır, bu sezgisel bir sonuçtur.

Varyans da benzer şekilde hesaplanır, A.1-3 ü kullanarak, X in varyansı,

$$\text{Var}[X] = q/p^2 \tag{C.33}$$

dir.

Örnek olarak, iki adet zarı üste gelen sayıların toplamı yedi veya onbir olana kadar tekrar tekrar attığımızı düşünelim. Tüm bu 36 sonuçtan, 6 tanesi yedi ve 2 tanesi onbir olarak gelir. Böylece, başarı olasılığı $p = 8/36 = 2/9$ ve ortalama $1/p = 9/2 = 4.5$ kere zar atmalıyız ki bir yedi veya onbir elde edelim.

Binom Dağılımı

Başarının p olasılıklı ve başarısızlığın $q = 1 - p$ olasılıklı olduğu n Bernoulli denemesinde kaç başarı elde edilir? X rassal değişkenini n denemede başarıların sayısı olarak tanımlayalım. Bu durumda X

$\{0, 1, \dots, n\}$ değerlerini alır ve $k = 0, 1, \dots, n$ için, n denemede k başarının seçimi $\binom{n}{k}$ yolla olacağından ve her birinin olma olasılığı $p^k q^{n-k}$ olduğundan,

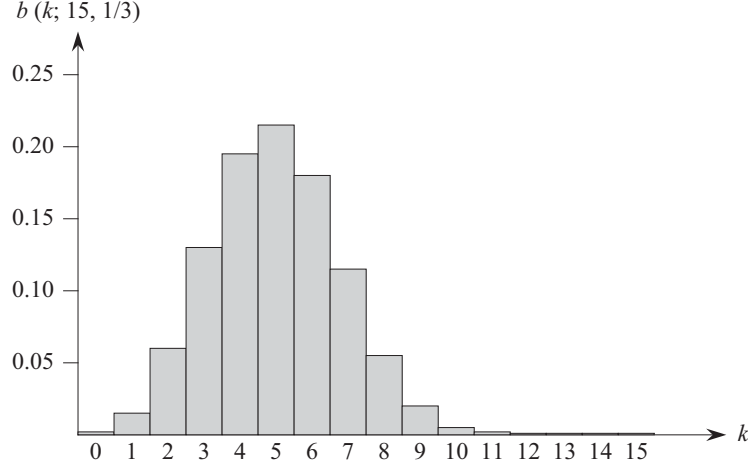
$$\Pr\{X = k\} = \binom{n}{k} p^k q^{n-k} \tag{C.34}$$

dır. (C.34) denklemini sağlayan olasılık dağılımına binom dağılımı denir. Uygun olması açısından, binom dağılımı ailesini,

$$b(k; n, p) = \binom{n}{k} p^k (1-p)^{n-k} \tag{C.35}$$

notasyonunu kullanarak tanımlayacağız.

Şekil C.2 bir **binom dağılımını** gösterir. “Binom” ismi (C.34) denkleminde eşitliğin sağ tarafından $(p + q)^n$ ifadesinin k -ıncı terimin açılımından gelmektedir.



Şekil C.2 $b(k; 15, 1/3)$ $n = 15$ Bernoulli denemesinden sonuçlanan binom dağılımı, her biri $p = 1/3$ başarı olasılığıdır. Dağılımın beklenen değeri $np = 5$ dir.

Sonuç olarak, $p + q = 1$ olduğundan, olasılık aksiyomlarından 2. aksiyomu ile birlikte,

$$\sum_{k=0}^n b(k; n, p) = 1 \quad (\text{C.36})$$

dir.

Binom dağılıma sahip bir rassal değişkenin beklenen değerini (C.8) ve (C.36) denklemlerinden hesaplayabiliriz. X bir rassal değişkeni $b(k; n, p)$ binom dağılımına sahip olsun ve $q = 1 - p$ olsun. Beklentinin tanımından,

$$\begin{aligned} E[X] &= \sum_{k=0}^n k \cdot \Pr\{X = k\} \\ &= \sum_{k=0}^n k \cdot b(k; n, p) \\ &= \sum_{k=1}^n k \binom{n}{k} p^k q^{n-k} \\ &= np \sum_{k=1}^n \binom{n-1}{k-1} p^{k-1} q^{n-k} \quad ((\text{C.8}) \text{ denkleminde}) \\ &= np \sum_{k=0}^{n-1} \binom{n-1}{k} p^k q^{(n-1)-k} \end{aligned}$$

$$\begin{aligned}
&= np \sum_{k=0}^{n-1} b(k; n-1, p) \\
&= np \quad \quad \quad ((C.36) \text{ denkleminde}) \quad \quad (C.37)
\end{aligned}$$

Beklentinin doğrusallığını kullanarak aynı sonucu daha az işlem ile de elde edebiliriz. X_i rassal değişkeni i -inci denemede başarıların sayısını ifade etsin. O halde, $E[X_i] = p \cdot 1 + q \cdot 0 = p$ dir ve beklentinin doğrusallığı ile (C.21 denklemi), n deneme için beklenen başarı sayısı,

$$\begin{aligned}
E[X] &= E\left[\sum_{i=1}^n X_i\right] \\
&= \sum_{i=1}^n E[X_i] \\
&= \sum_{i=1}^n p \\
&= np. \quad \quad \quad (C.38)
\end{aligned}$$

Benzer yaklaşımı, dağılımın varyansını hesaplarken de kullanabiliriz. (C.27) denklemi ile $\text{Var}[X_i] = E[X_i^2] - E^2[X_i]$ elde ederiz. X_i sadece 0 ve 1 değerlerini alacağından, $X_i^2 = X_i$ dir ve buradan $E[X_i^2] = E[X_i] = p$ olmasını gerektirir. Böylece,

$$\text{Var}[X_i] = p - p^2 = p(1 - p) = pq \quad \quad \quad (C.39)$$

dır.

X in varyansını hesaplamak için, n denemenin bağımsız oluşunu kullanırız; böylece (C.29) denklemi ile

$$\begin{aligned}
\text{Var}[X] &= \text{Var}\left[\sum_{i=1}^n X_i\right] \\
&= \sum_{i=1}^n \text{Var}[X_i] \\
&= \sum_{i=1}^n pq \\
&= npq. \quad \quad \quad (C.40)
\end{aligned}$$

Şekil C.2 de görüldüğü gibi, $b(k; n, p)$ binom dağılımı np ortalamasına ulaşana kadar k ile birlikte artar ve sonra düşer. başarılı terimlerin oranına bakarak, dağılımın her zaman bu şekilde davrandığını kanıtlayabiliriz:

$$\begin{aligned}
\frac{b(k; n, p)}{b(k-1; n, p)} &= \frac{\binom{n}{k} p^k q^{n-k}}{\binom{n}{k-1} p^{k-1} q^{n-k+1}} \\
&= \frac{n!(k-1)!(n-k+1)!p}{k!(n-k)!n!q} \\
&= \frac{(n-k+1)p}{kq} \\
&= 1 + \frac{(n+1)p-k}{kq}.
\end{aligned} \tag{C.41}$$

$(n+1)p-k$ tam olarak pozitif değer aldığında, oran 1 den büyüktür. Sonuç olarak, $k > (n+1)p$ için $b(k; n, p) > b(k-1; n, p)$ dir (dağılım artar) ve $k > (n+1)p$ için $b(k; n, p) < b(k-1; n, p)$ (dağılım azalır) dır. Eğer $k = (n+1)p$ bir tam sayı ise o zaman $b(k; n, p) = b(k-1; n, p)$ ve böylece dağılım $k-1 = (n+1)p$ ve $k-1 = (n+1)p-1 = np-q$ da iki maksimale sahiptir. Aksi halde, $np-q < k < (n+1)p$ aralığındaki bir tek k değerinde maksimuma ulaşır.

Sıradaki lemma binom dağılımı üzerinde bir üst sınır sağlar.

Lemma C.1

$n \geq 0$, $0 < p < 1$, $q = 1-p$ ve $0 \leq k \leq n$ olsun. Buna göre,

$$b(k; n, p) \leq \left(\frac{np}{k}\right)^k \left(\frac{nq}{n-k}\right)^{n-k} \text{ dir.}$$

Kanıt (C.6) denklemini kullanarak,

$$\begin{aligned}
b(k; n, p) &= \binom{n}{k} p^k q^{n-k} \\
&\leq \left(\frac{n}{k}\right)^k \left(\frac{n}{n-k}\right)^{n-k} p^k q^{n-k} \\
&= \left(\frac{np}{k}\right)^k \left(\frac{nq}{n-k}\right)^{n-k}.
\end{aligned}$$

■

Alıştırırmalar

C.4-1

Olasılık aksiyomlarından, aksiyom 2 yi geometrik dağılım için sağlandığını gösteriniz.

C.4-2

6 hilesiz parayı ortalama kaç kez atarsak 3 yazı 3 tura elde ederiz?

C.4-3

$q = 1 - p$ iken, $b(k; n, p) = b(n - k; n, q)$ olduğunu gösteriniz.

C.4-4

$b(k; n, p)$ binom dağılımının maksimum değerinin, $q = 1 - p$ iken, yaklaşık olarak $1/\sqrt{2\pi npq}$ olduğunu gösteriniz.

C.4-5 *

$p = 1/n$ olasılığına sahip n Bernoulli denemesinde hiç başarı olmama olasılığının $1/e$ olduğunu gösteriniz.

C.4-6 *

Profesör Rosencrantz ve Profesör Guildenstern hilesiz bir parayı n ket atıyorlar. Her ikisinin de aynı sayıda tura atma olasılıklarının $\binom{2n}{n}/4^n$ olduğunu gösteriniz. (*İpucu:* Profesör Rosencrantz için bir tura atmanın başarı olduğunu, Profesör Guildenstern için bir yazının başarı olduğunu kabul ediniz.)

İddianızı kullanarak $\sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}$ eşitliğini gerçekleyiniz.

C.4-7 *

$0 \leq k \leq n$ için,

$H(x)$ (C.7) deki entropi fonksiyonu olmak üzere,

$b(k; n, 1/2) \leq 2^{n H(k/n) - n}$ olduğunu gösteriniz.

C.4-8 *

i -inci denemesinin p_i başarı olasılığına sahip olduğu, $i = 1, 2, \dots, n$ için n tane Bernoulli denemesini ele alalım ve X toplam başarı sayısının rassal değişkenini göstereyin. Tüm $i = 1, 2, \dots, n$ ler için $p \geq p_i$ olsun. Buna göre $1 \leq k \leq n$ için,

$\Pr \{X < k\} \geq \sum_{i=0}^{k-1} b(i; n, p)$ olduğunu kanıtlayınız.

C.4-9 *

X , i -inci denemenin p_i başarı olasılığına sahip olduğu n Bernoulli denemesinin bir A kümesinde toplam başarı sayısı için rassal değişkeni olsun, ve X' de, i -inci denemenin $p'_i \geq p_i$ başarı olasılığına sahip olduğu n Bernoulli denemesinin bir A' kümesinde toplam başarı sayısı için rassal değişkeni olsun. Buna göre,

$\Pr\{X' \geq k\} \geq \Pr\{X \geq k\}$ olduğunu gösteriniz. (*Yol Gösterme: A'nın denemelerini kapsayan bir deney ile A'nde Bernoulli denemlerinin nasıl elde edileceğini gösteriniz ve alıştırmaya C.3-7'yi kullanınız.*)

* C.5 Binom dağılımın kuyrukları

Her birinin p başarı olasılığına sahip n Bernoulli denemelerindeki k başarılarına en az ya da en çok sayıda sahip olma olasılığı, kesin k başarılarına sahip olma olasılığından daha fazla ilgi çeker. Bu kısımda binom **dağılım** kuyruklarını inceleyeceğiz: $b(k; n, p)$ dağılımın iki bölgesi np ortalamasından uzaktır. Bir kuyruk üzerinde (bütün terimleri terimlerin toplamında) bazı önemli sınırları kanıtlayalım.

İlk olarak $b(k; n, p)$ dağılımının sağ kuyruğu üzerinde gerçekleyelim. Başarı ve başarısızlıkların rolünü ters çevirerek de sol kuyruk üzerindeki sınırları belirleyebiliriz.

Teorem C.2

p başarı olasılıklı n Bernoulli örneklerin bir dizisini düşünelim. X toplam gerçekleşenlerin sayısını gösteren rassal değişken olsun. O zaman, $0 \leq k \leq n$ için, en az k tanesinin gerçekleşme olasılığı

$$\begin{aligned} \Pr\{X \geq k\} &= \sum_{i=k}^n b(i; n, p) \\ &\leq \binom{n}{k} p^k \end{aligned}$$

dır.

Kanıt $S \subseteq \{1, 2, \dots, n\}$ için A_S , her $i \in S$ için i . denemede gerçekleşen olayı gösterebilir. Eğer $|S| = k$ ise $\Pr\{A_S\} = p^k$ olduğu açıktır.

$$\begin{aligned} \Pr\{X \geq k\} &= \Pr\{S \subseteq \{1, 2, \dots, n\} \text{ vardır: } |S| = k \text{ and } A_S\} \\ &= \Pr\left\{ \bigcup_{S \subseteq \{1, 2, \dots, n\}; |S|=k} A_S \right\} \\ &\leq \sum_{S \subseteq \{1, 2, \dots, n\}; |S|=k} \Pr\{A_S\} \quad ((C.19) \text{ eşitliğinden}) \\ &= \binom{n}{k} p^k. \quad \blacksquare \end{aligned}$$

Aşağıdaki sonuç binom dağılımının sol kuyruğu için teoremi yeniden ifade ediyor. Genellikle uyum sağlayabilmeniz için bir kuyruktan diğerine olan kanıtları sizlere bırakacağız.

Sonuç C.3

p başarı olasılıklı n Bernoulli örneklerin bir dizisini düşünelim. X toplam gerçekleşenlerin sayısını gösteren rassal değişken olsun. O zaman, $0 \leq k \leq n$ için, en az k tanesinin gerçekleşme olasılığı

$$\begin{aligned} \Pr \{X \leq k\} &= \sum_{i=0}^k b(i; n, p) \\ &\leq \binom{n}{n-k} (1-p)^{n-k} \\ &= \binom{n}{k} (1-p)^{n-k}. \end{aligned}$$

■

Bir sonraki sınırlamamız binom dağılımının sol kuyruğuyla ilgilidir. Yukarıdaki sonuç, ortalamadan uzak olan sol kuyruğun üstel olarak azaldığını göstermektedir.

Teorem C.4

p başarı ve $q = 1 - p$ başarısızlık olasılığına sahip n Bernoulli örneklerin bir dizisini düşünelim. X toplam gerçekleşenlerin sayısını gösteren rassal değişken olsun. O zaman, $0 < k < np$ için, k taneden daha azının gerçekleşme olasılığı

$$\begin{aligned} \Pr \{X < k\} &= \sum_{i=0}^{k-1} b(i; n, p) \\ &< \frac{kq}{np - k} b(k; n, p) \end{aligned}$$

dır.

Kanıt Sayfa 1151, Kısım A.2 deki tekniği geometrik seride kullanarak $\sum_{i=0}^{k-1} b(i; n, p)$ serilerini sınırlandıralım. (C.41) denkleminde $i = 1, 2, \dots, k$ için

$$\begin{aligned} \frac{b(i-1; n, p)}{b(i; n, p)} &= \frac{iq}{(n-i+1)p} \\ &< \frac{iq}{(n-i)p} \\ &\leq \frac{kq}{(n-k)p} \end{aligned}$$

ye sahibiz. Eğer

$$\begin{aligned}
 x &= \frac{kq}{(n-k)p} \\
 &< \frac{kq}{(n-np)p} \\
 &= \frac{kq}{nqp} \\
 &= \frac{k}{np} \\
 &< 1
 \end{aligned}$$

alırsak, buradan $0 < i \leq k$ için

$$b(i-1; n, p) < x b(i; n, p)$$

sonuçlanır. Bu eşitsizlik iteratif olarak $k-i$ defa uygulanırsa, $0 \leq i < k$ için

$$b(i; n, p) < x^{k-i} b(k; n, p)$$

elde ederiz ve dolayısıyla da

$$\begin{aligned}
 \sum_{i=0}^{k-1} b(i; n, p) &< \sum_{i=0}^{k-1} x^{k-i} b(k; n, p) \\
 &< b(k; n, p) \sum_{i=1}^{\infty} x^i \\
 &= \frac{x}{1-x} b(k; n, p) \\
 &= \frac{kq}{np-k} b(k; n, p)
 \end{aligned}$$

dır. ■

Sonuç C.5

p başarı ve $p = 1 - p$ başarısızlık olasılığına sahip n Bernoulli örneklerin bir dizisini düşünelim. O zaman, $0 < k \leq np/2$ için, k taneden daha azının gerçekleşme olasılığı $k + 1$ taneden daha azının gerçekleşme olasılığının yarısıdır.

Kanıt $k \leq np/2$ olduğu ve $q \leq 1$ olduğu için

$$\frac{kq}{np-k} \leq \frac{(np/2)q}{np - (np/2)}$$

$$\begin{aligned}
&= \frac{(np/2)q}{np/2} \\
&\leq 1
\end{aligned} \tag{C.42}$$

e sahibiz. X toplam gerçekleşenlerin sayısını gösteren rassal değişken olsun Teorem C.4 ve (C.42) eşitsizliği k taneden daha azının gerçekleşme olasılığı olan

$$\Pr\{X < k\} = \sum_{i=0}^{k-1} b(i; n, p) < b(k; n, p)$$

yi gerektirir. Böylece $\sum_{i=0}^{k-1} b(i; n, p) < b(k; n, p)$ olduğu için

$$\begin{aligned}
\frac{\Pr\{X < k\}}{\Pr\{X < k+1\}} &= \frac{\sum_{i=0}^{k-1} b(i; n, p)}{\sum_{i=0}^k b(i; n, p)} \\
&= \frac{\sum_{i=0}^{k-1} b(i; n, p)}{\sum_{i=0}^{k-1} b(i; n, p) + b(k; n, p)} \\
&< 1/2,
\end{aligned}$$

ye sahibiz. ■

Sağ kuyruk üzerindeki sınırlar da benzer şekilde takip edilecektir. Alıştırma C.5-2 de kanıtları size bırakılmıştır.

Sonuç C.6

p başarı olasılığına sahip n Bernoulli örneklerin bir dizisini düşünelim. X toplam gerçekleşenlerin sayısını gösteren rassal değişken olsun. O zaman, $np < k < n$ için, k taneden daha fazlasının gerçekleşme olasılığı

$$\begin{aligned}
\Pr\{X > k\} &= \sum_{i=k+1}^n b(i; n, p) \\
&< \frac{(n-k)p}{k-np} b(k; n, p)
\end{aligned}$$

dir. ■

Sonuç C.7

p başarı ve $q = 1 - p$ başarısızlık olasılığına sahip n Bernoulli örneklerin bir dizisini düşünelim. O zaman, $(np + n)/2 < k < n$ için, k taneden daha fazlasının gerçekleşme olasılığı $k - 1$ taneden fazlasının gerçekleşme olasılığının yarısından daha azdır. ■

Bir sonraki teorem, her $i = 1, 2, \dots, n$ için her biri p_i başarısına sahip n Bernoulli örnekleri olsun. Daha sonraki sonuçta da belirttiğimiz gibi, teoremi her bir

örnekte $p_i = p$ olarak binom dağılımın sağ kuyruğu üzerinde bir sınır bulmak için kullanabiliriz.

Teorem C.8

i sırasına denk gelen denemede $i = 1, 2, \dots, n$ için başarının p_i ve başarısızlığın $q_i = 1 - p_i$ olasılığıyla oluştuğu n Bernoulli örneklerinden bir diziyi ele alalım. X toplam gerçekleşenlerin sayısını gösteren rassal değişken ve $\mu = E[X]$ olsun. O zaman $r \geq \mu$ için

$$\Pr\{X - \mu \geq r\} \leq \left(\frac{\mu e}{r}\right)^r$$

dir.

Kanıt Daha sonra belirleyeceğimiz herhangi bir $\alpha > 0$ için, $e^{\alpha x}$ fonksiyonu x de kesin artan olduğundan

$$\Pr\{X - \mu \geq r\} = \Pr\{e^{\alpha(X-\mu)} \geq e^{\alpha r}\} \quad (\text{C.43})$$

dir. (C.30) Markov eşitsizliğini kullanarak,

$$\Pr\{e^{\alpha(X-\mu)} \geq e^{\alpha r}\} \leq E[e^{\alpha(X-\mu)}] e^{-\alpha r} \quad (\text{C.44})$$

eşitsizliğini elde ederiz.

Kanıtın genişlemesi (C.44) eşitsizliğindeki $E[e^{\alpha(X-\mu)}]$ sınırı ve uygun bir α nın değişiminden oluşur. İlk olarak $E[e^{\alpha(X-\mu)}]$ yi hesaplayalım. Rassal gösterge değişkenleri tekniğini kullanarak (bakınız Kısım 5.2), $i = 1, 2, \dots, n$ için $X_i = I\{i\}$. Bernoulli örneği doğrudur; yani X_i keyfi bir değişkendir yani i Bernoulli örneği doğruysa 1 ve yanlışsa 0 dır. Böylece,

$$X = \sum_{i=1}^n X_i$$

ve beklentinin doğrusallığından,

$$\mu = E[X] = E\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n p_i$$

dir bu da

$$X - \mu = \sum_{i=1}^n (X_i - p_i)$$

yi gerektirir.

$E[e^{\alpha(X-\mu)}]$ yü hesaplamak için, $X - \mu$ yü yerine koyduğumuzda

$$\begin{aligned} E[e^{\alpha(X-\mu)}] &= E[e^{\alpha \sum_{i=1}^n (X_i - p_i)}] \\ &= E\left[\prod_{i=1}^n e^{\alpha(X_i - p_i)}\right] \\ &= \prod_{i=1}^n E[e^{\alpha(X_i - p_i)}], \end{aligned}$$

X_i değişkenleri karşılıklı bağımsız olduğu için $e^{\alpha(X_i - p_i)}$ rassal değişkenlerinin karşılıklı bağımsız olmasını gerektireceğinden (C.24) den yukarıdaki sonuç elde edilir. (Bakınız Alıştırma C.3-5). $\exp(x)$, $\exp(x) = e^x$ üstel fonksiyonunu göstermek üzere, olasılık tanımından,

$$\begin{aligned} E[e^{\alpha(X_i - p_i)}] &= e^{\alpha(1-p_i)} p_i + e^{\alpha(0-p_i)} q_i \\ &= p_i e^{\alpha q_i} + q_i e^{-\alpha p_i} \\ &\leq p_i e^{\alpha} + 1 \\ &\leq \exp(p_i e^{\alpha}) \end{aligned} \quad (C.45)$$

dir ((C.45) eşitsizliği $\alpha > 0$, $q_i \leq 1$, $e^{\alpha q_i} \leq e^{\alpha}$ ve $e^{-\alpha p_i} \leq 1$ ve son satır (3.12) eşitsizliğinden elde edilir). Sonuç olarak $\mu = \sum_{i=1}^n p_i$ olduğu için,

$$\begin{aligned} E[e^{\alpha(X - \mu)}] &= \prod_{i=1}^n E[e^{\alpha(X_i - p_i)}] \\ &\leq \prod_{i=1}^n \exp(p_i e^{\alpha}) \\ &= \exp\left(\sum_{i=1}^n p_i e^{\alpha}\right) \\ &= \exp(\mu e^{\alpha}) \end{aligned} \quad (C.46)$$

dır. Bu yüzden, (C.43) eşitliği ve (C.44) ile (C.46) eşitsizliklerinden

$$\Pr\{X - \mu \geq r\} \leq \exp(\mu e^{\alpha} - \alpha r) \quad (C.47)$$

sonucuna varılır. $\alpha = \ln(r/\mu)$ seçerek (bakınız alıştırma C.5-7)

$$\begin{aligned} \Pr\{X - \mu \geq r\} &\leq \exp(\mu e^{\ln(r/\mu)} - r \ln(r/\mu)) \\ &= \exp(r - r \ln(r/\mu)) \\ &= \frac{e^r}{(r/\mu)^r} \\ &= \left(\frac{\mu e}{r}\right)^r \end{aligned}$$

elde ederiz. ■

Her bir örneğin gerçekleşme olasılığının aynı olduğu Bernoulli örneklerine uygulandığında Teorem C.8, sonraki sonuçtaki binom dağılımın sağ kuyruğunun sınırını elde eder.

Sonuç C.9

p başarı ve $q = 1 - p$ başarısızlık olasılığına sahip n Bernoulli örneklerin bir dizisini düşünelim. O zaman, $r > np$ için

$$\begin{aligned} \Pr\{X - np \geq r\} &= \sum_{k=\lceil np+r \rceil}^n b(k; n, p) \\ &\leq \left(\frac{npe}{r}\right)^r \end{aligned}$$

dir.

Kanıt (C.37) eşitliğinden, $\mu = E[X]$ dir. ■

Alıştırmalar**C.5-1 ***

Hangisi daha az olasıdır: Hilesiz bir madeni para n defa atıldığında tura gelmesi veya $4n$ defa atıldığında n defadan az tura gelmesi?

C.5-2 *

C.6 ve C.7 sonuçlarını kanıtlayınız.

C.5-3 *

Her $a > 0$ ve $0 < k < na/(a + 1)$ olacak şekilde tüm k lar için

$$\sum_{i=0}^{k-1} \binom{n}{i} a^i < (a + 1)^n \frac{k}{na - k(a + 1)} b(k; n, a/(a + 1))$$

olduğunu gösteriniz.

C.5-4 *

$0 < p < 1$ ve $q = 1 - p$ olduğunda, eğer $0 < k < np$ ise

$$\sum_{i=0}^{k-1} p^i q^{n-i} < \frac{kq}{np - k} \left(\frac{np}{k}\right)^k \left(\frac{nq}{n - k}\right)^{n-k}$$

olduğunu kanıtlayınız.

C.5-5 *

Teorem (C.8) in koşullarının

$$\Pr\{\mu - X \geq r\} \leq \left(\frac{(n - \mu)e}{r}\right)^r$$

yi gerektirdiğini gösteriniz. Benzer şekilde Sonuç (C.9) un koşullarının da

$$\Pr\{np - X \geq r\} \leq \left(\frac{nqe}{r}\right)^r$$

yi gerektirdiğini gösteriniz.

C.5-6 *

i sırasına denk gelen denemede $i = 1, 2, \dots, n$ için başarının p_i ve başarısızlığın $q_i = 1 - p_i$ olasılığıyla oluştuğu n Bernoulli örneklerinden bir diziyi ele alalım. X toplam gerçekleşenlerin sayısını gösteren rassal değişken ve $\mu = E[X]$ olsun. O zaman $r \geq 0$ için

$$\Pr \{X - \mu \geq r\} \leq e^{-r^2/2n}$$

olduğunu gösteriniz.

(İpucu: $p_i e^{\alpha q_i} + q_i e^{-\alpha p_i} \leq e^{\alpha^2/2}$ olduğunu kanıtlayınız. Bu eşitsizliği, (C.45) deki eşitsizlik yerine kullanan C.8 teoreminin kanıtının dizilimini takip ediniz.)

C.5-7 *

$\alpha = \ln(r/\mu)$ seçiminin (C.47) eşitsizliğinin sağ tarafını minimize ettiğini gösteriniz.

Problemler**C-1 Toplar ve kutular**

Bu problemde n tane topun b tane farklı kutuya yerleştirilmesini sağlayan yolların sayısı üzerine çeşitli varsayımların etkisini araştırıyoruz.

- a. n tane topun birbirinden farklı olduğunu ve bir kutu içindeki dizilimlerinin önemli olmadığını varsayalım. Topların kutuların içine b^n farklı şekilde yerleştirileceğini yorumlayınız.
- b. Topların birbirlerinden farklı ve her bir kutudaki topın bir sıra içinde olduğunu varsayalım. Topları kutulara yerleştirmenin kesin olarak $(b+n-1)!/(b-1)!$ farklı yolu olduğunu gösteriniz. (İpucu: n farklı top ve $b-1$ eş çubuğun bir sıra üzerindeki dizilim şeklinin sayısını düşününüz.)
- c. Topların birbirleriyle eş ve bir kutu içindeki dizilimlerinin önemsiz olduğunu varsayalım. Topları kutuların içine $\binom{b+n-1}{n}$ farklı yolla yerleştirilebileceğini gösteriniz. (İpucu: (b) kısmındaki düzenlemelerin kaç tanesi toplar birbirine benzer hale getirildiğinde tekrar eder?)
- d. Topların birbirleriyle eş olduğunu ve hiç bir kutunun birden fazla top içermediğini, dolayısıyla $n \leq b$ olduğunu varsayalım. Topların kutuların içine $\binom{b}{n}$ farklı şekilde yerleştirileceğini gösteriniz.
- e. Topların birbirleriyle eş olduğunu ve hiç bir kutunun boş olmadığını ve $n \geq b$ olduğunu varsayalım. Topların kutuların içine $\binom{n-1}{b-1}$ farklı şekilde yerleştirileceğini gösteriniz.

Ek notlar

Olasılık problemlerini çözmek için kullanılan ilk genel yöntemler B. Pascal ve P. de Fermat arasında 1654 de başlayan ve C. Huygens'in 1657 yılında yayınladığı kitapta tartışılmıştır. Kesin olasılık teorisi 1713 de J. Bernoulli ve 1730 da A. De Moivre'un çalışmalarıyla başladı. Teorideki daha sonraki gelişmelere P.-S. Laplace S.-D. Poisson ve C. F. Gauss tarafından katkı sağlanmıştır.

Rassal değişkenlerin toplamı ilk olarak P. L. Chebyshev ve A. A. Markov tarafından çalışılmıştır. A. N. Kolmogorov 1933 yılında olasılık teorisini aksiyomlaştırmıştır. Chernoff[66] ve Hoeffding[173] dağılımların kuyrukları üzerindeki sınırlara katkı sağlamıştır. Rassal kombinatorik yapılarıdaki ufuk açan çalışmalar P. Erdős tarafından gerçekleştirilmiştir.

Knuth[209] ve Liu[237] temel kombinatorik ve sayma için birer güzel referanstır. Billingsley[46], Chung[67], Drake[95], Feller[104] ve Rozanov[300] gibi kitaplar da olasılık üzerine kapsamlı girişler sunarlar.

D Matrisler

Matrisler, bilimsel programlamayı kapsayan fakat hiçbir şekilde sınırlı olmayan, sayısız uygulamada ortaya çıkar. Eğer matrisleri daha önceden gördüyseniz, bu bölümdeki materyalin çoğu tanıdık gelecektir, fakat bazıları yeni olabilir. Kısım D.1 temel matris tanımlarını ve işlemlerini içerir ve Kısım D.2 bazı temel matris özelliklerini ortaya koyar.

D.1 Matrisler ve matris işlemleri

Bu kısımda, matris teorisinin bazı temel kavramlarını ve matrislerin bazı temel özelliklerini inceleyeceğiz.

Matrisler ve vektörler

Bir **matris** sayıların dikdörtgensel dizilimidir. Örneğin,

$$\begin{aligned} A &= \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \end{pmatrix} \\ &= \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \end{aligned} \quad (\text{D.1})$$

$A = (a_{ij})$ olan bir 2×3 matristir. a_{ij} ile $i = 1, 2$ satırındaki ve $j = 1, 2, 3$ sütunundaki matris elemanını simgeleriz. Matrisleri göstermek için büyük harfleri ve elemanlarını göstermek için uygun altindisli küçük harfleri kullanırız. Reel değerli elemanlara sahip bütün $m \times n$ matrislerinin kümesini $\mathbb{R}^{m \times n}$ ile gösteririz ve genellikle, bir $S^{m \times n}$ kümesinden alınan elemanlara sahip $m \times n$ matrislerinin kümesini ile gösteririz.

Bir A matrisinin **transpozesi**, A nın satırlarını ve sütunlarını değiştirerek elde edilen bir A^T matrisidir. (D.1) denkleminin A matrisi için

$$A^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}.$$

Bir **vektör**, sayıların bir boyutlu dizisidir. Örneğin,

$$x = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}$$

boyu 3 olan bir vektördür. Bazen n -uzunluğundaki bir vektöre **n -vektör** deriz. $i = 1, 2, \dots, n$ için boyu n olan bir vektörünün i . elemanını ile gösteririz. Bir vektörün standart formunu, $n \times 1$ matrise denk olan bir **sütun vektörü** olarak ele alırız; uygun **satır vektörü**, transpoze alınarak elde edilir:

$$x^T = (2 \quad 3 \quad 5)$$

Bir e_i **birim vektörü**, i . elemanı 1 ve diğer bütün elemanları 0 olan vektördür. Genellikle, bir birim vektörün boyu içeriğinden açıkça görülmektedir.

Bir **sıfır matris**, bütün elemanları 0 olan matristir. 0 ların matrisi ile 0 sayısı arasındaki belirsizlik içeriğinden kolayca çözüldüğünden, böyle bir matris genellikle 0 ile gösterilir. Eğer 0 ların bir matrisi düşünülürse, o zaman bu matrisin boyunun da içeriğinden türetilmesi gerekir.

Kare matrisler

Kare $n \times n$ matrisler sıklıkla ortaya çıkarlar. Kare matrislerin birkaç özel durumuyla ayrıntılı olarak ilgilenilecektir.

1. Bir **diyagonal (köşegen) matriste** $i \neq j$ olduğunda $a_{ij} = 0$ dır. Köşegen dışındaki bütün elemanlar 0 olduğundan, köşegen boyunca elemanları listeleterek matrisi belirleyebiliriz:

$$\text{diag}(a_{11}, a_{22}, \dots, a_{nn}) = \begin{pmatrix} a_{11} & 0 & \dots & 0 \\ 0 & a_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_{nn} \end{pmatrix}.$$

2. Bir $n \times n$ **birim matrisi**, köşegen boyunca bütün elemanları 1 olan bir diyagonal matristir.

$$\begin{aligned} I_n &= \text{diag}(1, 1, \dots, 1) \\ &= \begin{pmatrix} 1 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 \end{pmatrix} \end{aligned}$$

bir altindis olmadan ortaya çıktığında, içeriğinden I nın boyunu türetebiliriz. Bir birim matrisin i . sütunu, e_i birim vektörüdür.

3. Bir **tridiagonal matrisi**, eğer $|i - j| > 1$ ise $t_{ij} = 0$ olan matristir. Sıfırdan farklı elemanlar sadece ana köşegen üzerinde, ana köşegenin hemen yukarısında ($i = 1, 2, \dots, n - 1$ için $t_{i,i+1}$) ya da ana köşegenin hemen aşağısında ($i = 1, 2, \dots, n - 1$ için $t_{i+1,i}$) bulunur:

$$T = \begin{pmatrix} t_{11} & t_{12} & 0 & 0 & \dots & 0 & 0 & 0 \\ t_{21} & t_{22} & t_{23} & 0 & \dots & 0 & 0 & 0 \\ 0 & t_{32} & t_{33} & t_{34} & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \dots & t_{n-2,n-2} & t_{n-2,n-1} & 0 \\ 0 & 0 & 0 & 0 & \dots & t_{n-1,n-2} & t_{n-1,n-1} & t_{n-1,n} \\ 0 & 0 & 0 & 0 & \dots & 0 & t_{n,n-1} & t_{nn} \end{pmatrix}.$$

4. Bir **üst-üçgensel matrisi**, eğer $i > j$ ise, $i > j$ $u_{ij} = 0$ olan bir matristir. Köşegenin aşağısındaki bütün elemanlar sıfırdır:

$$U = \begin{pmatrix} u_{11} & u_{12} & \dots & u_{1n} \\ 0 & u_{22} & \dots & u_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & u_{nn} \end{pmatrix}.$$

Eğer bir **üst-üçgensel matriste** köşegen boyunca bütün elemanlar 1 ise, bu matris **birim üst üçgenseldir**.

5. Bir **alt-üçgensel matrisi**, eğer $i < j$ ise, $l_{ij} = 0$ olan matristir. Köşegenin yukarısındaki bütün elemanlar sıfırdır:

$$L = \begin{pmatrix} l_{11} & 0 & \dots & 0 \\ l_{21} & l_{22} & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ l_{n1} & l_{n2} & \dots & l_{nn} \end{pmatrix}.$$

Eğer bir alt-üçgensel matriste köşegen boyunca bütün elemanlar 1 ise, bu matris **birim alt üçgenseldir**.

6. Bir **permütasyon matrisi** her bir satırda veya sütunda tam bir tane 1 e sahiptir ve diğer her yer 0 dır.

$$P = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

permütasyon matrisine bir örnektir.

Bir permütasyon matrisi ile bir vektörünün çarpımı, x in elemanlarını değiştirme (yeniden düzenleme) etkisine sahip olduğundan böyle bir matrise permütasyon matrisi denir. Alıştırma D.1-4 permütasyon matrisinin toplamsal özelliklerini araştırır.

7. Bir **simetrik matrisi**, $A = A^T$ koşulunu sağlar. Örneğin,

$$\begin{pmatrix} 1 & 2 & 3 \\ 2 & 6 & 4 \\ 3 & 4 & 5 \end{pmatrix}$$

bir simetrik matristir.

Temel matris işlemleri

Bir matris veya vektörün elemanları, reel sayılar, kompleks (karmaşık) sayılar veya bir p asal sayısı için mod p deki tam sayılar gibi bir sayı sisteminden sayılardır. Bir sayı sistemi, sayıların nasıl toplanacağını ve çarpılacağını tanımlar. Bu tanımları, matrislerin toplama ve çarpma işlemini kapsayacak şekilde genişletebiliriz.

Matris toplamını aşağıdaki gibi tanımlarız. Eğer $A = (a_{ij})$ ve $B = (b_{ij})$ iki $m \times n$ matris ise, o zaman bu matrislerin toplamı $C = (c_{ij}) = A + B$, $i = 1, 2, \dots, m$ ve $j = 1, 2, \dots, n$ için $c_{ij} = a_{ij} + b_{ij}$ ile tanımlı bir $m \times n$ matrisidir. Yani, matris toplamı bileşenlere göre yapılır. Sıfır matrisi, matris toplamı için birimdir:

$$A + 0 = A = 0 + A$$

Eğer λ bir sayı ve $A = (a_{ij})$ bir matris ise, o zaman $\lambda A = (\lambda a_{ij})$ A nın elemanlarının her biri ile λ ı çarparak elde edilen A nın bir **skalerle çarpımıdır**. $A = (a_{ij})$ nin **tersinin** $-1 \cdot A = -A$ olmasını özel bir durum olarak tanımlarız, böylece $-A$ nın ij . elemanı $-a_{ij}$ dir. Dolayısıyla

$$A + (-A) = 0 = (-A) + A \text{ dır.}$$

Matris çıkarmasını tanımlamak için bir matrisin toplamsal tersini kullanırız:
 $A - B = A + (-B)$

Matris çarpımını aşağıdaki gibi tanımlarız. Birbirine uygun olan yani A nın sütun sayısı B nin satır sayısına eşit olan A ve B matrisleri ile başlarız. (Genelde, AB matris çarpımını içeren bir ifade, A ve B matrislerinin uygun olması gerektiğini varsayar.) Eğer $A = (a_{ik})$ bir $m \times n$ matris ve $B = (b_{kj})$ bir $n \times p$ matris, ise o zaman $i = 1, 2, \dots, m$ ve $j = 1, 2, \dots, p$ için

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj} \quad (D.2)$$

olmak üzere A ve B nin matris çarpımı $C = AB$, $C = (c_{ij})$ şeklinde bir $m \times p$ matrisidir. Kısım 4.2 deki SQUARE-MATRIX-MULTIPLY prosedürü, $m = n = p$ olacak şekilde matrislerin kare olduğunu varsayarak (D.2) denklemine bağlı matris çarpımını basit bir şekilde yerine getirir. $n \times n$ matrisleri çarpmak için SQUARE-MATRIX-MULTIPLY, n^3 çarpma işlemi ve $n^2(n-1)$ toplama işlemi gerçekleştirir ve bu yüzden çalışma süresi $\Theta(n^3)$ dir.

Matrisler, sayıların tipik cebirsel özelliklerinin birçoğuna (fakat hepsine değil) sahiptir. Birim matrisler, matris çarpımı için birimdirler: her A $m \times n$ matrisi için $I_m A = A I_n = A$. Sıfır matrisi ile çarpma, sıfır matrisini verir:

$$A 0 = 0.$$

Matris çarpımı birleşmelidir: Uygun A , B ve C matrisleri için

$$A(BC) = (AB)C$$

Matris çarpımı, matris toplamı üzerine dağılır:

$$A(B + C) = AB + AC,$$

$$(B + C)D = BD + CD.$$

$n > 1$ için $n \times n$ matrislerin çarpımı değişmeli değildir. Örneğin, eğer

$$A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \text{ ve } B = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \text{ ise, o zaman}$$

$$AB = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

ve

$$BA = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$$

Matris-vektör çarpımlarının ya da vektör-vektör çarpımlarını, vektör $n \times 1$ matrise (ya da satır vektörü olması durumunda $1 \times n$ matrise) denkmiş gibi tanımlarız. Öyleyse, eğer A bir $m \times n$ matris ve x n -vektör ise, o zaman Ax bir m -vektördür. Eğer x ve y n -vektörlerse, o zaman

$$x^T y = \sum_{i=1}^n x_i y_i$$

x ve y nin **iç çarpımı** denilen bir sayıdır (gerçekte, bir 1×1 matristir). xy^T matrisi, $z_{ij} = x_i y_j$ ile, x ve y nin **dış çarpımı** denilen bir $\|x\|$ $n \times n$ matrisidir. Bir x n -vektörünün (Öklid) normu

$$\begin{aligned} \|x\| &= (x_1^2 + x_2^2 + \cdots + x_n^2)^{1/2} \\ &= (x^T x)^{1/2} \end{aligned}$$

ile tanımlanır. Bu yüzden, x in normu, n -boyutlu öklid uzayında x in uzunluğudur.

Alıştırma

D.1-1

Eğer A ve B $n \times n$ simetrik matrislerse, o zaman $A + B$ ve $A - B$ de $n \times n$ simetrik matrislerdir, gösteriniz.

D.1-2

$(AB)^T = B^T A^T$ dir ve $A^T A$ her zaman bir simetrik matristir, kanıtlayınız.

D.1-3

İki alt-üçgensel matrisin çarpımının alt-üçgensel olduğunu kanıtlayınız.

D.1-4

Eğer P bir $n \times n$ permütasyon matrisi ve A bir $n \times n$ matris ise, o zaman PA çarpım matrisinin satırları değiştirilmiş A matrisi ve AP çarpım matrisinin sütunları değiştirilmiş A matrisi olduğunu kanıtlayınız. İki permütasyon matrisinin çarpımının bir permütasyon matrisi olduğunu kanıtlayınız.

D.2 Temel matris özellikleri

Bu bölümde, matrislerle ilgili bazı temel özellikleri tanımlayacağız: tersler, lineer bağımlılık ve bağımsızlık, rank ve determinantlar. Ayrıca, pozitif-tanımlı matrislerin sınıfını tanımlayacağız.

Matrisin tersleri, rankları ve determinantları

Bir A $n \times n$ matrisinin **tersini**, $AA^{-1} = I_n = A^{-1}A$ olacak şekilde (eğer varsa) A^{-1} ile gösterilen bir $n \times n$ matris olarak tanımlarız. Örneğin,

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^{-1} = \begin{pmatrix} 0 & 1 \\ 1 & -1 \end{pmatrix}.$$

Sıfırdan farklı birçok $n \times n$ matrisin tersi yoktur. Ters olmayan bir matrise tersinin olmayan ya da tekil denir.

$$\begin{pmatrix} 1 & 0 \\ 1 & 0 \end{pmatrix}.$$

Sıfırdan farklı bir tekil matris örneğidir.

Eğer bir matrisin tersi varsa, o matrise **tersinir** ya da **tekil olmayan** denir. Matrisin tersleri var olduklarında, tektir (bakınız Alıştırma D.2-1). Eğer A ve B tekil olmayan $n \times n$ matrislerse, o zaman

$$(BA)^{-1} = A^{-1}B^{-1}.$$

Ters işlemi, transpoze işlemi ile değişmelidir:

$$(A^{-1})^T = (A^T)^{-1}.$$

Eğer $c_1x_1 + c_2x_2 + \cdots + c_nx_n = 0$ olacak şekilde hepsi sıfır olmayan $c_1, c_2, \dots, c_n$ katsayıları varsa, $x_1, x_2, \dots, x_n$ vektörleri **lineer bağımlıdır**. Örneğin, $2x_1 + 3x_2 - 2x_3 = 0$ olduğundan, satır vektörleri $x_1 = (1 \ 2 \ 3)$, $x_2 = (2 \ 6 \ 4)$ ve $x_3 = (4 \ 11 \ 9)$ lineer bağımlıdır. Eğer vektörler lineer bağımlı değillerse, o zaman bu vektörler **lineer bağımsızdır**. Örneğin, bir birim matrisin sütunları lineer bağımsızdır.

Sıfırdan farklı bir A $m \times n$ matrisinin **sütun rankı**, A nın lineer bağımsız sütunlarının en geniş kümesinin boyutudur. Benzer şekilde, A nın **satır rankı**, A nın lineer bağımsız satırlarının en geniş kümesinin boyutudur. Her A matrisinin temel özelliği, satır rankının her zaman sütun rankına eşit olmasıdır, böylece A nın **rankını** basitçe ifade edebiliriz. Bir $m \times n$ matrisinin rankı, 0 ve $\min(m, n)$ i içeren, 0 ve $\min(m, n)$ arasında bir tam sayıdır (Sıfır matrisinin rankı 0 dır ve bir $n \times n$ birim matrisinin rankı n dir). Bir diğer tanım da, sıfırdan farklı bir A $m \times n$ matrisinin rankının, $A = BC$ olmak üzere boyutları sırasıyla $m \times r$ ve $r \times n$ olan B ve C matrisleri var olacak şekilde en küçük r sayısı olmasıdır, fakat bu tanım ilk verdiğimiz tanıma denktir ve genellikle daha kullanışlıdır.

Eğer bir kare $n \times n$ matrisin rankı n ise, bu matris bir **tam ranka** sahiptir. Eğer bir $m \times n$ matrisin rankı n ise, bu matris bir **tam sütun rankına** sahiptir. Aşağıdaki teorem, rankların temel bir özelliğini verir.

Teorem D.1

Bir kare matris tam ranka sahiptir ancak ve ancak bu matris tekil olmayan matristir. ■

Bir A matrisi için **sıfır vektörü** $Ax = 0$ olacak şekilde sıfırdan farklı bir x vektörüdür. Aşağıdaki teorem (kanıtı, Alıştırma D.2-7 olarak bırakılmıştır) ve sonucu, sütun rankı ve tekillik kavramları ile sıfır vektörlerini ilişkilendirir.

Teorem D.2

Bir A matrisi tam sütun rankına sahiptir ancak ve ancak bu matrisin bir sıfır vektörü yoktur. ■

Sonuç D.3

Bir A kare matrisi tekildir ancak ve ancak bu matrisin bir sıfır vektörü vardır. ■

$n > 1$ için bir A $n \times n$ matrisinin ij -nci **minörü**, A nın i -nci satırını ve j -nci sütununu silerek elde edilen $A_{[ij]}$ $(n-1) \times (n-1)$ matrisidir. Bir A $n \times n$ matrisinin **determinantını**, rekürsif olarak minörleri cinsinden

$$\det(A) = \begin{cases} a_{11} & n = 1 \text{ ise,} \\ \sum_{j=1}^n (-1)^{1+j} a_{1j} \det(A_{[1j]}) & n > 1 \text{ ise,} \end{cases}$$

ile tanımlarız.

$(-1)^{i+j} \det(A_{[ij]})$ terimi, a_{ij} elemanının **kofaktörü** olarak bilinir.

Kanıtları burada yapılmayan aşağıdaki teoremler, determinantın temel özelliklerini ifade ederler.

Teorem D.4 (Determinant özellikleri)

Bir A kare matrisinin determinanı aşağıdaki özelliklere sahiptir:

- Eğer A nın herhangi bir satırı ya da herhangi bir sütunu sıfır ise, o zaman $\det(A) = 0$ dır.
- Eğer A nın herhangi bir satırının (ya da herhangi bir sütununun) elemanlarının hepsi λ ile çarpılırsa, A determinanı λ ile çarpılır.
- Eğer bir satırdaki (sırasıyla, sütundaki) elemanlar başka bir satırdakilere (sırasıyla, sütundakilere) eklenirse, A nın determinanı değişmez.
- A nın determinanı, A^T nin determinantına eşittir.
- Eğer herhangi iki satırın (ya da herhangi iki sütunun) yeri değiştirilirse, A nın determinanı -1 ile çarpılır.

Ayrıca, her A ve B kare matrisleri için $\det(AB) = \det(A) \det(B)$ dir ■

Teorem D.5

Bir A $n \times n$ matrisi tekildir ancak ve ancak $x^T A x > 0$ dır. ■

Pozitif tanımlı matrisler

Pozitif tanımlı matrisler birçok uygulamada önemli bir rol oynar. Eğer bütün $x \neq 0$ n -vektörleri için $x^T A x > 0$ ise A $n \times n$ matrisi **pozitif tanımlıdır**. Örneğin, sıfırdan farklı her

$$x = (x_1 \ x_2 \ \cdots \ x_n)^T,$$

$$\begin{aligned} x^T I_n x &= x^T x \\ &= \sum_{i=1}^n x_i^2 \\ &> 0 \end{aligned}$$

olduğundan birim matris pozitif-tanımlıdır.

Uygulamalarda ortaya çıkan matrisler, aşağıdaki teoremden dolayı, genellikle pozitif tanımlıdır.

Teorem D.6

Tam sütun rankına sahip her A matrisi için $A^T A$ matrisi pozitif-tanımlıdır.

Kanıt Sıfırdan farklı her x vektörü için $x^T (A^T A) x > 0$ olduğunu göstermemiz gerekir. Her x vektörü için

$$\begin{aligned} x^T (A^T A) x &= (Ax)^T (Ax) \quad (\text{Alıştırma D.1-2 den}) \\ &= \|Ax\|^2. \end{aligned}$$

$\|Ax\|^2$ nin Ax vektörlerinin elemanlarının kareleri toplamı olduğuna dikkat edelim. Bu yüzden $\|Ax\|^2 \geq 0$ dır. Eğer $\|Ax\|^2 = 0$ ise, Ax in her elemanı 0 dır, yani $Ax = 0$ dır. A tam sütun rankına sahip olduğundan Teorem D.2 gereği; $Ax = 0$, $x = 0$ olduğunu ifade eder. Dolayısıyla $A^T A$ pozitif-tanımlıdır. ■

Kısım 28.3, pozitif tanımlı matrislerin diğer özelliklerini inceler.

Alıştırmalar**D.2-1**

Matrisin terslerinin tek olduğunu, yani eğer B ve C , A nın tersleriyse o zaman $B = C$ olduğunu kanıtlayınız.

D.2-2

Bir alt-üçgensel ya da üst-üçgensel matrisin determinantının köşegen elemanlarının çarpımına eşit olduğunu kanıtlayınız. Eğer varsa, bir alt-üçgensel matrisin tersinin alt-üçgensel olduğunu kanıtlayınız.

D.2-3

Eğer P bir permütasyon matrisi ise, o zaman P tersinirdir, P nin tersi P^T dir ve P^T bir permütasyon matrisidir, kanıtlayınız.

D.2-4

A ve B , $AB = I$ olacak şekilde iki $n \times n$ matris olsun. Eğer A' i satırına j satırını ekleyerek A dan elde ediliyorsa, o zaman B nin j sütunundan i sütununu çıkararak A' ın tersi B' elde edilir, kanıtlayınız.

D.2-5

A kompleks elemanlara sahip tekil olmayan bir $n \times n$ matris olsun. A^{-1} in her elemanı reeldir ancak ve ancak A nın her elemanı reeldir, gösteriniz.

D.2-6

Eğer A tekil olmayan, simetrik bir $n \times n$ matris ise, o zaman A^{-1} in simetrik olduğunu gösteriniz.

Eğer B keyfi bir $n \times n$ matris ise, o zaman BAB^T çarpımı ile verilen $m \times m$ matrisinin simetrik olduğunu gösteriniz.

D.2-7

Teorem D.2 yi kanıtlayınız. Yani, bir A matrisi tam sütun rankına sahiptir ancak ve ancak $Ax = 0, x = 0$ olduğunu ifade eder, gösteriniz (*İpucu*: Bir sütunun diğerleri üzerindeki lineer bağımlılığını matris-vektör denklemi olarak ifade ediniz).

D.2-8

Uygun olan her A ve B matrisi için

$$\text{rank}(AB) \leq \min(\text{rank}(A), \text{rank}(B))$$

olduğunu kanıtlayınız. Burada, eğer A ya da B tekil olmayan bir kare matris ise, eşitlik gelir. (*İpucu*: Bir matrisin rankının diğer tanımını kullanınız).

Problemler**D-1 Vandermonde matrisi**

$x_0, x_1, \dots, x_{n-1}$ sayıları verilsin.

$$V(x_0, x_1, \dots, x_{n-1}) = \begin{pmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{pmatrix}$$

olan **Vandermonde matrisinin** determinantı

$$\det(V(x_0, x_1, \dots, x_{n-1})) = \prod_{0 \leq j < k \leq n-1} (x_k - x_j)$$

dir, kanıtlayınız. (*İpucu:* $i = n-1, n-2, \dots, 1$, için i sütununu $-x_0$ ile çarparak $i+1$ sütununa ekleyiniz ve daha sonra tümevarımdan yararlanınız).

D-2 $GF(2)$ üzerinde matris-vektör çarpımı tarafından tanımlanan permütasyonlar

$S_n = \{0, 1, 2, \dots, 2^n - 1\}$ kümesindeki tam sayıların permütasyonlarının bir sınıfı, $GF(2)$ üzerindeki matris çarpımı tarafından tanımlanır. S_n deki her bir x tam sayısı için $x = \sum_{i=0}^{n-1} x_i 2^i$ olacak şekilde x in ikili temsili bir

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{n-1} \end{pmatrix}$$

n -bit vektör olarak görüntüleriz. Eğer A her bir elemanı 0 ya da 1 olan bir $n \times n$ matris ise, o zaman S_n deki her bir x değerini ikili temsili Ax matris-vektör çarpımı olan bir sayıya eşleyen bir permütasyon tanımlayabiliriz. Burada, bütün değerleri 0 ya da 1 olan $GF(2)$ üzerinde bütün aritmetik işlemleri gerçekleştiririz ve sadece bir istisna dışında, toplama ve çarpma işlemlerinin klasik kuralları geçerlidir. Bu istisna $1 + 1 = 0$ dır. $GF(2)$ üzerindeki aritmetik işlemleri, sadece en önemsiz biti kullanmanız dışında tıpkı normal tam sayı aritmetiğinde olduğu gibi düşünebilirsiniz.

Örnek olarak, $S_2 = \{0, 1, 2, 3\}$ için

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$$

matrisi aşağıdaki π_A permütasyonu tanımlar: π_A : $\pi_A(0) = 0$, $\pi_A(1) = 3$, $\pi_A(2) = 2$, $\pi_A(3) = 1$. Neden $\pi_A(3) = 1$ olduğunu görmek için 1 in ikili temsili olan ve $GF(2)$ de gerçekleşen

$$\begin{aligned} \pi_A(3) &= \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \cdot 1 + 0 \cdot 1 \\ 1 \cdot 1 + 1 \cdot 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \end{aligned}$$

1 gözlemleriz.

Bu problemin geri kalanı için $GF(2)$ üzerinde çalışırız ve bütün matris ve vektör elemanları 0 ya da 1 dir. $GF(2)$ üzerindeki bir 0–1 matrisinin (her bir elemanı 0 ya da 1 olan bir matris) rankını, normal bir matrisle aynı şekilde fakat $GF(2)$ üzerinde gerçekleştirilen lineer bağımsızlığı belirleyen bütün aritmetik işlemlerle tanımlarız. Bir A $n \times n$ 0–1 matrisinin **değer kümesini**

$$R(A) = \{y : \text{bir } x \in S_n \text{ için } y = Ax\}$$

ile tanımlarız, böylece $R(A)$, S_n deki her bir x değerini A ile çarparak üretebileceğimiz S_n deki sayıların bir kümesidir.

- a.** Eğer A matrisinin rankı r ise, $|R(A)| = 2^r$ olduğunu kanıtlayınız. Eğer A tam ranka sahipse, A nın S_n üzerinde bir permütasyon tanımladığı sonucuna varınız.

Verilen bir $n \times n$ matrisi ve verilen bir $y \in R(A)$ değeri için y nin **öngörüntüsünü**

$$P(A, y) = \{x : Ax = y\}$$

ile tanımlarız, böylece $P(A, y)$ A ile çarpıldığında y e eşlenen S_n deki sayıların bir kümesidir.

- b.** Eğer bir A $n \times n$ matrisinin rankı r ve $y \in R(A)$ ise, $|P(A, y)| = 2^{n-r}$ olduğunu kanıtlayınız.

$0 \leq m \leq n$ olsun ve S_n i, i -nci bloğu $i2^m, i2^m + 1; i2^m + 2, \dots, (i+1)2^m - 1$ olacak şekilde 2^m tane sayı içeren ardışık sayı bloklarına böleriz. Her $S \subseteq S_n$ altkümesi için $B(S, m)$ S nin bazı elemanlarını içeren S_n nin 2^m -boyutlu bir altkümesi olarak tanımlarız. Örnek olarak, $n = 3, m = 1$ ve $S = \{1, 4, 5\}$ olduğunda, $B(S, m)$ 0 (1, 0-ncı blokta bulunduğu) ve 2 (4 ve 5 in her ikisi de blok-2 de olduğundan) bloklarını içerir.

- c.** A nın bir sol alt $(n - m) \times m$ altmatrisinin yani A nın en soldaki m tane sütunun ve alttaki $n - m$ tane satırın kesişimini alarak oluşturulan matrisin rankı r olsun. S, S_n nin $2m$ -boyutlu herhangi bir bloğu ve $S' = \{y : \text{bir } x \in S \text{ için } y = Ax\}$ olsun. $|B(S', m)| = 2^r$ olduğunu ve $B(S', m)$ deki her bir blok için S deki tam 2^{m-r} tane sayıyı o bloğa eşlediğini kanıtlayınız.

Herhangi bir matrisle sıfır vektörünü çarparak bir sıfır vektörü elde edileceğinden, $GF(2)$ üzerinde tam ranka sahip $n \times n$ 0–1 matrisleri ile çarparak tanımlanan S_n nin permütasyonlarının kümesi, S_n nin bütün permütasyonlarını içermez. Bir toplamsal terim eklemek için matris-vektör çarpımı tarafından tanımlanan permütasyonların sınıfını genişletelim, böylece $x \in S_n$, c nin bir n -bit vektör olduğu $Ax + c$ ile eşleşir ve toplama işlemi $GF(2)$ üzerinde gerçekleştirilir. Örneğin,

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$$

ve

$$c = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

olduğunda aşağıdaki $\pi_{A,c}$ permütasyonunu elde ederiz: $\pi_{A,c}(0) = 2$, $\pi_{A,c}(1) = 1$, $\pi_{A,c}(2) = 0$, $\pi_{A,c}(3) = 3$. Tam ranka sahip bir A $n \times n$ 0-1 matrisi ve bir c n -bit vektörü için $x \in S_n$ to $Ax + c$ ile eşleyen her permütasyona bir **doğrusal permütasyon** denir.

- d. S_n nin doğrusal permütasyonlarının sayısının, S_n nin permütasyonlarının sayısından daha az olduğunu göstermek için bir sayma argümanı kullanınız.
- e. Herhangi bir doğrusal permütasyon tarafından elde edilemeyen S_n nin bir permütasyonu ve bir n değeri için bir örnek veriniz (*İpucu*: Verilen bir permütasyon için bir birim vektörle bir matrisin çarpımının bu matrisin sütunları ile nasıl ilişkilendirildiği üzerine düşününüz).

Ek notları

Lineer-cebir ders kitapları, matrisler hakkında pek çok gerekli bilgiyi sağlar. Strang'ın [323,329] kitapları oldukça iyidir.

Bibliyografi

- [1] Milton Abramowitz and Irene A. Stegun, editors. *Handbook of Mathematical Functions*. Dover, 1965.
- [2] G. M. Adel'son-Vel'skiĭ and E. M. Landis. An algorithm for the organization of information. *Soviet Mathematics Doklady*, 3(5):1259–1263, 1962.
- [3] Alok Aggarwal and Jeffrey Scott Vitter. The input/output complexity of sorting and related problems. *Communications of the ACM*, 31(9):1116–1127, 1988.
- [4] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. PRIMES is in P. *Annals of Mathematics*, 160(2):781–793, 2004.
- [5] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *The Design and Analysis of Computer Algorithms*. Addison-Wesley, 1974.
- [6] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *Data Structures and Algorithms*. Addison-Wesley, 1983.
- [7] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.
- [8] Ravindra K. Ahuja, Kurt Mehlhorn, James B. Orlin, and Robert E. Tarjan. Faster algorithms for the shortest path problem. *Journal of the ACM*, 37(2):213–223, 1990.
- [9] Ravindra K. Ahuja and James B. Orlin. A fast and simple algorithm for the maximum flow problem. *Operations Research*, 37(5):748–759, 1989.
- [10] Ravindra K. Ahuja, James B. Orlin, and Robert E. Tarjan. Improved time bounds for the maximum flow problem. *SIAM Journal on Computing*, 18(5):939–954, 1989.
- [11] Miklós Ajtai, Nimrod Megiddo, and Orli Waarts. Improved algorithms and analysis for secretary problems and generalizations. In *Proceedings of the 36th Annual Symposium on Foundations of Computer Science*, pages 473–482, 1995.
- [12] Selim G. Akl. *The Design and Analysis of Parallel Algorithms*. Prentice Hall, 1989.
- [13] Mohamad Akra and Louay Bazzi. On the solution of linear recurrence equations. *Computational Optimization and Applications*, 10(2):195–210, 1998.
- [14] Noga Alon. Generating pseudo-random permutations and maximum flow algorithms. *Information Processing Letters*, 35:201–204, 1990.

- [15] Arne Andersson. Balanced search trees made simple. In *Proceedings of the Third Workshop on Algorithms and Data Structures*, volume 709 of *Lecture Notes in Computer Science*, pages 60–71. Springer, 1993.
- [16] Arne Andersson. Faster deterministic sorting and searching in linear space. In *Proceedings of the 37th Annual Symposium on Foundations of Computer Science*, pages 135–141, 1996.
- [17] Arne Andersson, Torben Hagerup, Stefan Nilsson, and Rajeev Raman. Sorting in linear time? *Journal of Computer and System Sciences*, 57:74–93, 1998.
- [18] Tom M. Apostol. *Calculus*, volume 1. Blaisdell Publishing Company, second edition, 1967.
- [19] Nimar S. Arora, Robert D. Blumofe, and C. Greg Plaxton. Thread scheduling for multiprogrammed multiprocessors. In *Proceedings of the 10th Annual ACM Symposium on Parallel Algorithms and Architectures*, pages 119–129, 1998.
- [20] Sanjeev Arora. *Probabilistic checking of proofs and the hardness of approximation problems*. PhD thesis, University of California, Berkeley, 1994.
- [21] Sanjeev Arora. The approximability of NP-hard problems. In *Proceedings of the 30th Annual ACM Symposium on Theory of Computing*, pages 337–348, 1998.
- [22] Sanjeev Arora. Polynomial time approximation schemes for euclidean traveling salesman and other geometric problems. *Journal of the ACM*, 45(5):753–782, 1998.
- [23] Sanjeev Arora and Carsten Lund. Hardness of approximations. In Dorit S. Hochbaum, editor, *Approximation Algorithms for NP-Hard Problems*, pages 399–446. PWS Publishing Company, 1997.
- [24] Javed A. Aslam. A simple bound on the expected height of a randomly built binary search tree. Technical Report TR2001-387, Dartmouth College Department of Computer Science, 2001.
- [25] Mikhail J. Atallah, editor. *Algorithms and Theory of Computation Handbook*. CRC Press, 1999.
- [26] G. Ausiello, P. Crescenzi, G. Gambosi, V. Kann, A. Marchetti-Spaccamela, and M. Protasi. *Complexity and Approximation: Combinatorial Optimization Problems and Their Approximability Properties*. Springer, 1999.
- [27] Shai Avidan and Ariel Shamir. Seam carving for content-aware image resizing. *ACM Transactions on Graphics*, 26(3), article 10, 2007.
- [28] Sara Baase and Alan Van Gelder. *Computer Algorithms: Introduction to Design and Analysis*. Addison-Wesley, third edition, 2000.
- [29] Eric Bach. Private communication, 1989.
- [30] Eric Bach. Number-theoretic algorithms. In *Annual Review of Computer Science*, volume 4, pages 119–172. Annual Reviews, Inc., 1990.
- [31] Eric Bach and Jeffrey Shallit. *Algorithmic Number Theory—Volume I: Efficient Algorithms*. The MIT Press, 1996.
- [32] David H. Bailey, King Lee, and Horst D. Simon. Using Strassen’s algorithm to accelerate the solution of linear systems. *The Journal of Supercomputing*, 4(4):357–371, 1990.

- [33] Surender Baswana, Ramesh Hariharan, and Sandeep Sen. Improved decremental algorithms for maintaining transitive closure and all-pairs shortest paths. *Journal of Algorithms*, 62(2):74–92, 2007.
- [34] R. Bayer. Symmetric binary B-trees: Data structure and maintenance algorithms. *Acta Informatica*, 1(4):290–306, 1972.
- [35] R. Bayer and E. M. McCreight. Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3):173–189, 1972.
- [36] Pierre Beauchemin, Gilles Brassard, Claude Crépeau, Claude Goutier, and Carl Pomerance. The generation of random numbers that are probably prime. *Journal of Cryptology*, 1(1):53–64, 1988.
- [37] Richard Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- [38] Richard Bellman. On a routing problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.
- [39] Michael Ben-Or. Lower bounds for algebraic computation trees. In *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing*, pages 80–86, 1983.
- [40] Michael A. Bender, Erik D. Demaine, and Martin Farach-Colton. Cache-oblivious B-trees. In *Proceedings of the 41st Annual Symposium on Foundations of Computer Science*, pages 399–409, 2000.
- [41] Samuel W. Bent and John W. John. Finding the median requires $2n$ comparisons. In *Proceedings of the Seventeenth Annual ACM Symposium on Theory of Computing*, pages 213–216, 1985.
- [42] Jon L. Bentley. *Writing Efficient Programs*. Prentice Hall, 1982.
- [43] Jon L. Bentley. *Programming Pearls*. Addison-Wesley, 1986.
- [44] Jon L. Bentley, Dorothea Haken, and James B. Saxe. A general method for solving divide-and-conquer recurrences. *SIGACT News*, 12(3):36–44, 1980.
- [45] Daniel Bienstock and Benjamin McClosky. Tightening simplex mixed-integer sets with guaranteed bounds. *Optimization Online*, July 2008.
- [46] Patrick Billingsley. *Probability and Measure*. John Wiley & Sons, second edition, 1986.
- [47] Guy E. Blelloch. *Scan Primitives and Parallel Vector Models*. PhD thesis, Department of Electrical Engineering and Computer Science, MIT, 1989. Available as MIT Laboratory for Computer Science Technical Report MIT/LCS/TR-463.
- [48] Guy E. Blelloch. Programming parallel algorithms. *Communications of the ACM*, 39(3):85–97, 1996.
- [49] Guy E. Blelloch, Phillip B. Gibbons, and Yossi Matias. Provably efficient scheduling for languages with fine-grained parallelism. In *Proceedings of the 7th Annual ACM Symposium on Parallel Algorithms and Architectures*, pages 1–12, 1995.
- [50] Manuel Blum, Robert W. Floyd, Vaughan Pratt, Ronald L. Rivest, and Robert E. Tarjan. Time bounds for selection. *Journal of Computer and System Sciences*, 7(4):448–461, 1973.
- [51] Robert D. Blumofe, Christopher F. Joerg, Bradley C. Kuszmaul, Charles E. Leiserson, Keith H. Randall, and Yuli Zhou. Cilk: An efficient multithreaded runtime system. *Journal of Parallel and Distributed Computing*, 37(1):55–69, 1996.

- [52] Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, 1999.
- [53] Béla Bollobás. *Random Graphs*. Academic Press, 1985.
- [54] Gilles Brassard and Paul Bratley. *Fundamentals of Algorithmics*. Prentice Hall, 1996.
- [55] Richard P. Brent. The parallel evaluation of general arithmetic expressions. *Journal of the ACM*, 21(2):201–206, 1974.
- [56] Richard P. Brent. An improved Monte Carlo factorization algorithm. *BIT*, 20(2):176–184, 1980.
- [57] J. P. Buhler, H. W. Lenstra, Jr., and Carl Pomerance. Factoring integers with the number field sieve. In A. K. Lenstra and H. W. Lenstra, Jr., editors, *The Development of the Number Field Sieve*, volume 1554 of *Lecture Notes in Mathematics*, pages 50–94. Springer, 1993.
- [58] J. Lawrence Carter and Mark N. Wegman. Universal classes of hash functions. *Journal of Computer and System Sciences*, 18(2):143–154, 1979.
- [59] Barbara Chapman, Gabriele Jost, and Ruud van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. The MIT Press, 2007.
- [60] Bernard Chazelle. A minimum spanning tree algorithm with inverse-Ackermann type complexity. *Journal of the ACM*, 47(6):1028–1047, 2000.
- [61] Joseph Cheriyan and Torben Hagerup. A randomized maximum-flow algorithm. *SIAM Journal on Computing*, 24(2):203–226, 1995.
- [62] Joseph Cheriyan and S. N. Maheshwari. Analysis of preflow push algorithms for maximum network flow. *SIAM Journal on Computing*, 18(6):1057–1086, 1989.
- [63] Boris V. Cherkassky and Andrew V. Goldberg. On implementing the push-relabel method for the maximum flow problem. *Algorithmica*, 19(4):390–410, 1997.
- [64] Boris V. Cherkassky, Andrew V. Goldberg, and Tomasz Radzik. Shortest paths algorithms: Theory and experimental evaluation. *Mathematical Programming*, 73(2):129–174, 1996.
- [65] Boris V. Cherkassky, Andrew V. Goldberg, and Craig Silverstein. Buckets, heaps, lists and monotone priority queues. *SIAM Journal on Computing*, 28(4):1326–1346, 1999.
- [66] H. Chernoff. A measure of asymptotic efficiency for tests of a hypothesis based on the sum of observations. *Annals of Mathematical Statistics*, 23(4):493–507, 1952.
- [67] Kai Lai Chung. *Elementary Probability Theory with Stochastic Processes*. Springer, 1974.
- [68] V. Chvátal. A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3):233–235, 1979.
- [69] V. Chvátal. *Linear Programming*. W. H. Freeman and Company, 1983.
- [70] V. Chvátal, D. A. Klarnier, and D. E. Knuth. Selected combinatorial research problems. Technical Report STAN-CS-72-292, Computer Science Department, Stanford University, 1972.
- [71] Cilk Arts, Inc., Burlington, Massachusetts. *Cilk++ Programmer’s Guide*, 2008. Available at <http://www.cilk.com/archive/docs/cilk1guide>.

- [72] Alan Cobham. The intrinsic computational difficulty of functions. In *Proceedings of the 1964 Congress for Logic, Methodology, and the Philosophy of Science*, pages 24–30. North-Holland, 1964.
- [73] H. Cohen and H. W. Lenstra, Jr. Primality testing and Jacobi sums. *Mathematics of Computation*, 42(165):297–330, 1984.
- [74] Douglas Comer. The ubiquitous B-tree. *ACM Computing Surveys*, 11(2):121–137, 1979.
- [75] Stephen Cook. The complexity of theorem proving procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, pages 151–158, 1971.
- [76] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
- [77] Don Coppersmith. Modifications to the number field sieve. *Journal of Cryptology*, 6(3):169–180, 1993.
- [78] Don Coppersmith and Shmuel Winograd. Matrix multiplication via arithmetic progression. *Journal of Symbolic Computation*, 9(3):251–280, 1990.
- [79] Thomas H. Cormen, Thomas Sundquist, and Leonard F. Wisniewski. Asymptotically tight bounds for performing BMMC permutations on parallel disk systems. *SIAM Journal on Computing*, 28(1):105–136, 1998.
- [80] Don Dailey and Charles E. Leiserson. Using Cilk to write multiprocessor chess programs. In H. J. van den Herik and B. Monien, editors, *Advances in Computer Games*, volume 9, pages 25–52. University of Maastricht, Netherlands, 2001.
- [81] Paolo D’Alberto and Alexandru Nicolau. Adaptive Strassen’s matrix multiplication. In *Proceedings of the 21st Annual International Conference on Supercomputing*, pages 284–292, June 2007.
- [82] Sanjoy Dasgupta, Christos Papadimitriou, and Umesh Vazirani. *Algorithms*. McGraw-Hill, 2008.
- [83] Roman Dementiev, Lutz Kettner, Jens Mehnert, and Peter Sanders. Engineering a sorted list data structure for 32 bit keys. In *Proceedings of the Sixth Workshop on Algorithm Engineering and Experiments and the First Workshop on Analytic Algorithmics and Combinatorics*, pages 142–151, January 2004.
- [84] Camil Demetrescu and Giuseppe F. Italiano. Fully dynamic all pairs shortest paths with real edge weights. *Journal of Computer and System Sciences*, 72(5):813–837, 2006.
- [85] Eric V. Denardo and Bennett L. Fox. Shortest-route methods: 1. Reaching, pruning, and buckets. *Operations Research*, 27(1):161–186, 1979.
- [86] Martin Dietzfelbinger, Anna Karlin, Kurt Mehlhorn, Friedhelm Meyer auf der Heide, Hans Rohnert, and Robert E. Tarjan. Dynamic perfect hashing: Upper and lower bounds. *SIAM Journal on Computing*, 23(4):738–761, 1994.
- [87] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, IT-22(6):644–654, 1976.
- [88] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.

- [89] E. A. Dinic. Algorithm for solution of a problem of maximum flow in a network with power estimation. *Soviet Mathematics Doklady*, 11(5):1277–1280, 1970.
- [90] Brandon Dixon, Monika Rauch, and Robert E. Tarjan. Verification and sensitivity analysis of minimum spanning trees in linear time. *SIAM Journal on Computing*, 21(6):1184–1192, 1992.
- [91] John D. Dixon. Factorization and primality tests. *The American Mathematical Monthly*, 91(6):333–352, 1984.
- [92] Dorit Dor, Johan Håstad, Staffan Ulfberg, and Uri Zwick. On lower bounds for selecting the median. *SIAM Journal on Discrete Mathematics*, 14(3):299–311, 2001.
- [93] Dorit Dor and Uri Zwick. Selecting the median. *SIAM Journal on Computing*, 28(5):1722–1758, 1999.
- [94] Dorit Dor and Uri Zwick. Median selection requires $(2 + \epsilon)n$ comparisons. *SIAM Journal on Discrete Mathematics*, 14(3):312–325, 2001.
- [95] Alvin W. Drake. *Fundamentals of Applied Probability Theory*. McGraw-Hill, 1967.
- [96] James R. Driscoll, Harold N. Gabow, Ruth Shrairman, and Robert E. Tarjan. Relaxed heaps: An alternative to Fibonacci heaps with applications to parallel computation. *Communications of the ACM*, 31(11):1343–1354, 1988.
- [97] James R. Driscoll, Neil Sarnak, Daniel D. Sleator, and Robert E. Tarjan. Making data structures persistent. *Journal of Computer and System Sciences*, 38(1):86–124, 1989.
- [98] Derek L. Eager, John Zahorjan, and Edward D. Lazowska. Speedup versus efficiency in parallel systems. *IEEE Transactions on Computers*, 38(3):408–423, 1989.
- [99] Herbert Edelsbrunner. *Algorithms in Combinatorial Geometry*, volume 10 of *EATCS Monographs on Theoretical Computer Science*. Springer, 1987.
- [100] Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965.
- [101] Jack Edmonds. Matroids and the greedy algorithm. *Mathematical Programming*, 1(1):127–136, 1971.
- [102] Jack Edmonds and Richard M. Karp. Theoretical improvements in the algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972.
- [103] Shimon Even. *Graph Algorithms*. Computer Science Press, 1979.
- [104] William Feller. *An Introduction to Probability Theory and Its Applications*. John Wiley & Sons, third edition, 1968.
- [105] Robert W. Floyd. Algorithm 97 (SHORTEST PATH). *Communications of the ACM*, 5(6):345, 1962.
- [106] Robert W. Floyd. Algorithm 245 (TREESORT). *Communications of the ACM*, 7(12):701, 1964.
- [107] Robert W. Floyd. Permuting information in idealized two-level storage. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 105–109. Plenum Press, 1972.

- [108] Robert W. Floyd and Ronald L. Rivest. Expected time bounds for selection. *Communications of the ACM*, 18(3):165–172, 1975.
- [109] Lestor R. Ford, Jr. and D. R. Fulkerson. *Flows in Networks*. Princeton University Press, 1962.
- [110] Lestor R. Ford, Jr. and Selmer M. Johnson. A tournament problem. *The American Mathematical Monthly*, 66(5):387–389, 1959.
- [111] Michael L. Fredman. New bounds on the complexity of the shortest path problem. *SIAM Journal on Computing*, 5(1):83–89, 1976.
- [112] Michael L. Fredman, János Komlós, and Endre Szemerédi. Storing a sparse table with $O(1)$ worst case access time. *Journal of the ACM*, 31(3):538–544, 1984.
- [113] Michael L. Fredman and Michael E. Saks. The cell probe complexity of dynamic data structures. In *Proceedings of the Twenty First Annual ACM Symposium on Theory of Computing*, pages 345–354, 1989.
- [114] Michael L. Fredman and Robert E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, 34(3):596–615, 1987.
- [115] Michael L. Fredman and Dan E. Willard. Surpassing the information theoretic bound with fusion trees. *Journal of Computer and System Sciences*, 47(3):424–436, 1993.
- [116] Michael L. Fredman and Dan E. Willard. Trans-dichotomous algorithms for minimum spanning trees and shortest paths. *Journal of Computer and System Sciences*, 48(3):533–551, 1994.
- [117] Matteo Frigo and Steven G. Johnson. The design and implementation of FFTW3. *Proceedings of the IEEE*, 93(2):216–231, 2005.
- [118] Matteo Frigo, Charles E. Leiserson, and Keith H. Randall. The implementation of the Cilk-5 multithreaded language. In *Proceedings of the 1998 ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 212–223, 1998.
- [119] Harold N. Gabow. Path-based depth-first search for strong and biconnected components. *Information Processing Letters*, 74(3–4):107–114, 2000.
- [120] Harold N. Gabow, Z. Galil, T. Spencer, and Robert E. Tarjan. Efficient algorithms for finding minimum spanning trees in undirected and directed graphs. *Combinatorica*, 6(2):109–122, 1986.
- [121] Harold N. Gabow and Robert E. Tarjan. A linear-time algorithm for a special case of disjoint set union. *Journal of Computer and System Sciences*, 30(2):209–221, 1985.
- [122] Harold N. Gabow and Robert E. Tarjan. Faster scaling algorithms for network problems. *SIAM Journal on Computing*, 18(5):1013–1036, 1989.
- [123] Zvi Galil and Oded Margalit. All pairs shortest distances for graphs with small integer length edges. *Information and Computation*, 134(2):103–139, 1997.
- [124] Zvi Galil and Oded Margalit. All pairs shortest paths for graphs with small integer length edges. *Journal of Computer and System Sciences*, 54(2):243–254, 1997.
- [125] Zvi Galil and Kunsoo Park. Dynamic programming with convexity, concavity and sparsity. *Theoretical Computer Science*, 92(1):49–76, 1992.

- [126] Zvi Galil and Joel Seiferas. Time-space-optimal string matching. *Journal of Computer and System Sciences*, 26(3):280–294, 1983.
- [127] Igal Galperin and Ronald L. Rivest. Scapegoat trees. In *Proceedings of the 4th ACM-SIAM Symposium on Discrete Algorithms*, pages 165–174, 1993.
- [128] Michael R. Garey, R. L. Graham, and J. D. Ullman. Worst-case analysis of memory allocation algorithms. In *Proceedings of the Fourth Annual ACM Symposium on Theory of Computing*, pages 143–150, 1972.
- [129] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [130] Saul Gass. *Linear Programming: Methods and Applications*. International Thomson Publishing, fourth edition, 1975.
- [131] Fănică Gavril. Algorithms for minimum coloring, maximum clique, minimum covering by cliques, and maximum independent set of a chordal graph. *SIAM Journal on Computing*, 1(2):180–187, 1972.
- [132] Alan George and Joseph W-H Liu. *Computer Solution of Large Sparse Positive Definite Systems*. Prentice Hall, 1981.
- [133] E. N. Gilbert and E. F. Moore. Variable-length binary encodings. *Bell System Technical Journal*, 38(4):933–967, 1959.
- [134] Michel X. Goemans and David P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, 1995.
- [135] Michel X. Goemans and David P. Williamson. The primal-dual method for approximation algorithms and its application to network design problems. In Dorit S. Hochbaum, editor, *Approximation Algorithms for NP-Hard Problems*, pages 144–191. PWS Publishing Company, 1997.
- [136] Andrew V. Goldberg. *Efficient Graph Algorithms for Sequential and Parallel Computers*. PhD thesis, Department of Electrical Engineering and Computer Science, MIT, 1987.
- [137] Andrew V. Goldberg. Scaling algorithms for the shortest paths problem. *SIAM Journal on Computing*, 24(3):494–504, 1995.
- [138] Andrew V. Goldberg and Satish Rao. Beyond the flow decomposition barrier. *Journal of the ACM*, 45(5):783–797, 1998.
- [139] Andrew V. Goldberg, Éva Tardos, and Robert E. Tarjan. Network flow algorithms. In Bernhard Korte, László Lovász, Hans Jürgen Prömel, and Alexander Schrijver, editors, *Paths, Flows, and VLSI-Layout*, pages 101–164. Springer, 1990.
- [140] Andrew V. Goldberg and Robert E. Tarjan. A new approach to the maximum flow problem. *Journal of the ACM*, 35(4):921–940, 1988.
- [141] D. Goldfarb and M. J. Todd. Linear programming. In G. L. Nemhauser, A. H. G. Rinnooy-Kan, and M. J. Todd, editors, *Handbook in Operations Research and Management Science, Vol. 1, Optimization*, pages 73–170. Elsevier Science Publishers, 1989.
- [142] Shafi Goldwasser and Silvio Micali. Probabilistic encryption. *Journal of Computer and System Sciences*, 28(2):270–299, 1984.

- [143] Shafi Goldwasser, Silvio Micali, and Ronald L. Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing*, 17(2):281–308, 1988.
- [144] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. The Johns Hopkins University Press, third edition, 1996.
- [145] G. H. Gonnet. *Handbook of Algorithms and Data Structures*. Addison-Wesley, 1984.
- [146] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Addison-Wesley, 1992.
- [147] Michael T. Goodrich and Roberto Tamassia. *Data Structures and Algorithms in Java*. John Wiley & Sons, 1998.
- [148] Michael T. Goodrich and Roberto Tamassia. *Algorithm Design: Foundations, Analysis, and Internet Examples*. John Wiley & Sons, 2001.
- [149] Ronald L. Graham. Bounds for certain multiprocessor anomalies. *Bell System Technical Journal*, 45(9):1563–1581, 1966.
- [150] Ronald L. Graham. An efficient algorithm for determining the convex hull of a finite planar set. *Information Processing Letters*, 1(4):132–133, 1972.
- [151] Ronald L. Graham and Pavol Hell. On the history of the minimum spanning tree problem. *Annals of the History of Computing*, 7(1):43–57, 1985.
- [152] Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. *Concrete Mathematics*. Addison-Wesley, second edition, 1994.
- [153] David Gries. *The Science of Programming*. Springer, 1981.
- [154] M. Grötschel, László Lovász, and Alexander Schrijver. *Geometric Algorithms and Combinatorial Optimization*. Springer, 1988.
- [155] Leo J. Guibas and Robert Sedgwick. A dichromatic framework for balanced trees. In *Proceedings of the 19th Annual Symposium on Foundations of Computer Science*, pages 8–21, 1978.
- [156] Dan Gusfield. *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge University Press, 1997.
- [157] H. Halberstam and R. E. Ingram, editors. *The Mathematical Papers of Sir William Rowan Hamilton*, volume III (Algebra). Cambridge University Press, 1967.
- [158] Yijie Han. Improved fast integer sorting in linear space. In *Proceedings of the 12th ACM-SIAM Symposium on Discrete Algorithms*, pages 793–796, 2001.
- [159] Yijie Han. An $O(n^3(\log \log n / \log n)^{5/4})$ time algorithm for all pairs shortest path. *Algorithmica*, 51(4):428–434, 2008.
- [160] Frank Harary. *Graph Theory*. Addison-Wesley, 1969.
- [161] Gregory C. Harfst and Edward M. Reingold. A potential-based amortized analysis of the union-find data structure. *SIGACT News*, 31(3):86–95, 2000.
- [162] J. Hartmanis and R. E. Stearns. On the computational complexity of algorithms. *Transactions of the American Mathematical Society*, 117:285–306, May 1965.

- [163] Michael T. Heideman, Don H. Johnson, and C. Sidney Burrus. Gauss and the history of the Fast Fourier Transform. *IEEE ASSP Magazine*, 1(4):14–21, 1984.
- [164] Monika R. Henzinger and Valerie King. Fully dynamic biconnectivity and transitive closure. In *Proceedings of the 36th Annual Symposium on Foundations of Computer Science*, pages 664–672, 1995.
- [165] Monika R. Henzinger and Valerie King. Randomized fully dynamic graph algorithms with polylogarithmic time per operation. *Journal of the ACM*, 46(4):502–516, 1999.
- [166] Monika R. Henzinger, Satish Rao, and Harold N. Gabow. Computing vertex connectivity: New bounds from old techniques. *Journal of Algorithms*, 34(2):222–250, 2000.
- [167] Nicholas J. Higham. Exploiting fast matrix multiplication within the level 3 BLAS. *ACM Transactions on Mathematical Software*, 16(4):352–368, 1990.
- [168] W. Daniel Hillis and Jr. Guy L. Steele. Data parallel algorithms. *Communications of the ACM*, 29(12):1170–1183, 1986.
- [169] C. A. R. Hoare. Algorithm 63 (PARTITION) and algorithm 65 (FIND). *Communications of the ACM*, 4(7):321–322, 1961.
- [170] C. A. R. Hoare. Quicksort. *Computer Journal*, 5(1):10–15, 1962.
- [171] Dorit S. Hochbaum. Efficient bounds for the stable set, vertex cover and set packing problems. *Discrete Applied Mathematics*, 6(3):243–254, 1983.
- [172] Dorit S. Hochbaum, editor. *Approximation Algorithms for NP-Hard Problems*. PWS Publishing Company, 1997.
- [173] W. Hoeffding. On the distribution of the number of successes in independent trials. *Annals of Mathematical Statistics*, 27(3):713–721, 1956.
- [174] Micha Hofri. *Probabilistic Analysis of Algorithms*. Springer, 1987.
- [175] Micha Hofri. *Analysis of Algorithms*. Oxford University Press, 1995.
- [176] John E. Hopcroft and Richard M. Karp. An $n^{5/2}$ algorithm for maximum matchings in bipartite graphs. *SIAM Journal on Computing*, 2(4):225–231, 1973.
- [177] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison Wesley, third edition, 2006.
- [178] John E. Hopcroft and Robert E. Tarjan. Efficient algorithms for graph manipulation. *Communications of the ACM*, 16(6):372–378, 1973.
- [179] John E. Hopcroft and Jeffrey D. Ullman. Set merging algorithms. *SIAM Journal on Computing*, 2(4):294–303, 1973.
- [180] John E. Hopcroft and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 1979.
- [181] Ellis Horowitz, Sartaj Sahni, and Sanguthevar Rajasekaran. *Computer Algorithms*. Computer Science Press, 1998.
- [182] T. C. Hu and M. T. Shing. Computation of matrix chain products. Part I. *SIAM Journal on Computing*, 11(2):362–373, 1982.
- [183] T. C. Hu and M. T. Shing. Computation of matrix chain products. Part II. *SIAM Journal on Computing*, 13(2):228–251, 1984.

- [184] T. C. Hu and A. C. Tucker. Optimal computer search trees and variable-length alphabetic codes. *SIAM Journal on Applied Mathematics*, 21(4):514–532, 1971.
- [185] David A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, 40(9):1098–1101, 1952.
- [186] Steven Huss-Lederman, Elaine M. Jacobson, Jeremy R. Johnson, Anna Tsao, and Thomas Turnbull. Implementation of Strassen’s algorithm for matrix multiplication. In *Proceedings of the 1996 ACM/IEEE Conference on Supercomputing*, article 32, 1996.
- [187] Oscar H. Ibarra and Chul E. Kim. Fast approximation algorithms for the knapsack and sum of subset problems. *Journal of the ACM*, 22(4):463–468, 1975.
- [188] E. J. Isaac and R. C. Singleton. Sorting by address calculation. *Journal of the ACM*, 3(3):169–174, 1956.
- [189] R. A. Jarvis. On the identification of the convex hull of a finite set of points in the plane. *Information Processing Letters*, 2(1):18–21, 1973.
- [190] David S. Johnson. Approximation algorithms for combinatorial problems. *Journal of Computer and System Sciences*, 9(3):256–278, 1974.
- [191] David S. Johnson. The NP-completeness column: An ongoing guide—The tale of the second prover. *Journal of Algorithms*, 13(3):502–524, 1992.
- [192] Donald B. Johnson. Efficient algorithms for shortest paths in sparse networks. *Journal of the ACM*, 24(1):1–13, 1977.
- [193] Richard Johnsonbaugh and Marcus Schaefer. *Algorithms*. Pearson Prentice Hall, 2004.
- [194] A. Karatsuba and Yu. Ofman. Multiplication of multidigit numbers on automata. *Soviet Physics—Doklady*, 7(7):595–596, 1963. Translation of an article in *Doklady Akademii Nauk SSSR*, 145(2), 1962.
- [195] David R. Karger, Philip N. Klein, and Robert E. Tarjan. A randomized linear-time algorithm to find minimum spanning trees. *Journal of the ACM*, 42(2):321–328, 1995.
- [196] David R. Karger, Daphne Koller, and Steven J. Phillips. Finding the hidden path: Time bounds for all-pairs shortest paths. *SIAM Journal on Computing*, 22(6):1199–1217, 1993.
- [197] Howard Karloff. *Linear Programming*. Birkhäuser, 1991.
- [198] N. Karmarkar. A new polynomial-time algorithm for linear programming. *Combinatorica*, 4(4):373–395, 1984.
- [199] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.
- [200] Richard M. Karp. An introduction to randomized algorithms. *Discrete Applied Mathematics*, 34(1–3):165–201, 1991.
- [201] Richard M. Karp and Michael O. Rabin. Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development*, 31(2):249–260, 1987.
- [202] A. V. Karzanov. Determining the maximal flow in a network by the method of preflows. *Soviet Mathematics Doklady*, 15(2):434–437, 1974.

- [203] Valerie King. A simpler minimum spanning tree verification algorithm. *Algorithmica*, 18(2):263–270, 1997.
- [204] Valerie King, Satish Rao, and Robert E. Tarjan. A faster deterministic maximum flow algorithm. *Journal of Algorithms*, 17(3):447–474, 1994.
- [205] Jeffrey H. Kingston. *Algorithms and Data Structures: Design, Correctness, Analysis*. Addison-Wesley, second edition, 1997.
- [206] D. G. Kirkpatrick and R. Seidel. The ultimate planar convex hull algorithm? *SIAM Journal on Computing*, 15(2):287–299, 1986.
- [207] Philip N. Klein and Neal E. Young. Approximation algorithms for NP-hard optimization problems. In *CRC Handbook on Algorithms*, pages 34-1–34-19. CRC Press, 1999.
- [208] Jon Kleinberg and Éva Tardos. *Algorithm Design*. Addison-Wesley, 2006.
- [209] Donald E. Knuth. *Fundamental Algorithms*, volume 1 of *The Art of Computer Programming*. Addison-Wesley, 1968. Third edition, 1997.
- [210] Donald E. Knuth. *Seminumerical Algorithms*, volume 2 of *The Art of Computer Programming*. Addison-Wesley, 1969. Third edition, 1997.
- [211] Donald E. Knuth. *Sorting and Searching*, volume 3 of *The Art of Computer Programming*. Addison-Wesley, 1973. Second edition, 1998.
- [212] Donald E. Knuth. Optimum binary search trees. *Acta Informatica*, 1(1):14–25, 1971.
- [213] Donald E. Knuth. Big omicron and big omega and big theta. *SIGACT News*, 8(2):18–23, 1976.
- [214] Donald E. Knuth, James H. Morris, Jr., and Vaughan R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2):323–350, 1977.
- [215] J. Komlós. Linear verification for spanning trees. *Combinatorica*, 5(1):57–65, 1985.
- [216] Bernhard Korte and László Lovász. Mathematical structures underlying greedy algorithms. In F. Gecseg, editor, *Fundamentals of Computation Theory*, volume 117 of *Lecture Notes in Computer Science*, pages 205–209. Springer, 1981.
- [217] Bernhard Korte and László Lovász. Structural properties of greedoids. *Combinatorica*, 3(3–4):359–374, 1983.
- [218] Bernhard Korte and László Lovász. Greedoids—A structural framework for the greedy algorithm. In W. Pulleybank, editor, *Progress in Combinatorial Optimization*, pages 221–243. Academic Press, 1984.
- [219] Bernhard Korte and László Lovász. Greedoids and linear objective functions. *SIAM Journal on Algebraic and Discrete Methods*, 5(2):229–238, 1984.
- [220] Dexter C. Kozen. *The Design and Analysis of Algorithms*. Springer, 1992.
- [221] David W. Krumme, George Cybenko, and K. N. Venkataraman. Gossiping in minimal time. *SIAM Journal on Computing*, 21(1):111–139, 1992.
- [222] Joseph B. Kruskal, Jr. On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society*, 7(1):48–50, 1956.
- [223] Leslie Lamport. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Transactions on Computers*, C-28(9):690–691, 1979.

- [224] Eugene L. Lawler. *Combinatorial Optimization: Networks and Matroids*. Holt, Rinehart, and Winston, 1976.
- [225] Eugene L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan, and D. B. Shmoys, editors. *The Traveling Salesman Problem*. John Wiley & Sons, 1985.
- [226] C. Y. Lee. An algorithm for path connection and its applications. *IRE Transactions on Electronic Computers*, EC-10(3):346–365, 1961.
- [227] Tom Leighton. Tight bounds on the complexity of parallel sorting. *IEEE Transactions on Computers*, C-34(4):344–354, 1985.
- [228] Tom Leighton. Notes on better master theorems for divide-and-conquer recurrences. Class notes. Available at <http://citeseer.ist.psu.edu/252350.html>, October 1996.
- [229] Tom Leighton and Satish Rao. Multicommodity max-flow min-cut theorems and their use in designing approximation algorithms. *Journal of the ACM*, 46(6):787–832, 1999.
- [230] Daan Leijen and Judd Hall. Optimize managed code for multi-core machines. *MSDN Magazine*, October 2007.
- [231] Debra A. Lelewer and Daniel S. Hirschberg. Data compression. *ACM Computing Surveys*, 19(3):261–296, 1987.
- [232] A. K. Lenstra, H. W. Lenstra, Jr., M. S. Manasse, and J. M. Pollard. The number field sieve. In A. K. Lenstra and H. W. Lenstra, Jr., editors, *The Development of the Number Field Sieve*, volume 1554 of *Lecture Notes in Mathematics*, pages 11–42. Springer, 1993.
- [233] H. W. Lenstra, Jr. Factoring integers with elliptic curves. *Annals of Mathematics*, 126(3):649–673, 1987.
- [234] L. A. Levin. Universal sorting problems. *Problemy Peredachi Informatsii*, 9(3):265–266, 1973. In Russian.
- [235] Anany Levitin. *Introduction to the Design & Analysis of Algorithms*. Addison-Wesley, 2007.
- [236] Harry R. Lewis and Christos H. Papadimitriou. *Elements of the Theory of Computation*. Prentice Hall, second edition, 1998.
- [237] C. L. Liu. *Introduction to Combinatorial Mathematics*. McGraw-Hill, 1968.
- [238] László Lovász. On the ratio of optimal integral and fractional covers. *Discrete Mathematics*, 13(4):383–390, 1975.
- [239] László Lovász and Michael D. Plummer. *Matching Theory*, volume 121 of *Annals of Discrete Mathematics*. North Holland, 1986.
- [240] Bruce M. Maggs and Serge A. Plotkin. Minimum-cost spanning tree as a path-finding problem. *Information Processing Letters*, 26(6):291–293, 1988.
- [241] Michael Main. *Data Structures and Other Objects Using Java*. Addison-Wesley, 1999.
- [242] Udi Manber. *Introduction to Algorithms: A Creative Approach*. Addison-Wesley, 1989.
- [243] Conrado Martínez and Salvador Roura. Randomized binary search trees. *Journal of the ACM*, 45(2):288–323, 1998.
- [244] William J. Masek and Michael S. Paterson. A faster algorithm computing string edit distances. *Journal of Computer and System Sciences*, 20(1):18–31, 1980.

- [245] H. A. Maurer, Th. Ottmann, and H.-W. Six. Implementing dictionaries using binary trees of very small height. *Information Processing Letters*, 5(1):11–14, 1976.
- [246] Ernst W. Mayr, Hans Jürgen Prömel, and Angelika Steger, editors. *Lectures on Proof Verification and Approximation Algorithms*, volume 1367 of *Lecture Notes in Computer Science*. Springer, 1998.
- [247] C. C. McGeoch. All pairs shortest paths and the essential subgraph. *Algorithmica*, 13(5):426–441, 1995.
- [248] M. D. McIlroy. A killer adversary for quicksort. *Software—Practice and Experience*, 29(4):341–344, 1999.
- [249] Kurt Mehlhorn. *Sorting and Searching*, volume 1 of *Data Structures and Algorithms*. Springer, 1984.
- [250] Kurt Mehlhorn. *Graph Algorithms and NP-Completeness*, volume 2 of *Data Structures and Algorithms*. Springer, 1984.
- [251] Kurt Mehlhorn. *Multidimensional Searching and Computational Geometry*, volume 3 of *Data Structures and Algorithms*. Springer, 1984.
- [252] Kurt Mehlhorn and Stefan Näher. Bounded ordered dictionaries in $O(\log \log N)$ time and $O(n)$ space. *Information Processing Letters*, 35(4):183–189, 1990.
- [253] Kurt Mehlhorn and Stefan Näher. *LEDA: A Platform for Combinatorial and Geometric Computing*. Cambridge University Press, 1999.
- [254] Alfred J. Menezes, Paul C. van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1997.
- [255] Gary L. Miller. Riemann’s hypothesis and tests for primality. *Journal of Computer and System Sciences*, 13(3):300–317, 1976.
- [256] John C. Mitchell. *Foundations for Programming Languages*. The MIT Press, 1996.
- [257] Joseph S. B. Mitchell. Guillotine subdivisions approximate polygonal subdivisions: A simple polynomial-time approximation scheme for geometric TSP, k -MST, and related problems. *SIAM Journal on Computing*, 28(4):1298–1309, 1999.
- [258] Louis Monier. *Algorithmes de Factorisation D’Entiers*. PhD thesis, L’Université Paris-Sud, 1980.
- [259] Louis Monier. Evaluation and comparison of two efficient probabilistic primality testing algorithms. *Theoretical Computer Science*, 12(1):97–108, 1980.
- [260] Edward F. Moore. The shortest path through a maze. In *Proceedings of the International Symposium on the Theory of Switching*, pages 285–292. Harvard University Press, 1959.
- [261] Rajeev Motwani, Joseph (Seffi) Naor, and Prabhakar Raghavan. Randomized approximation algorithms in combinatorial optimization. In Dorit Hochbaum, editor, *Approximation Algorithms for NP-Hard Problems*, chapter 11, pages 447–481. PWS Publishing Company, 1997.
- [262] Rajeev Motwani and Prabhakar Raghavan. *Randomized Algorithms*. Cambridge University Press, 1995.
- [263] J. I. Munro and V. Raman. Fast stable in-place sorting with $O(n)$ data moves. *Algorithmica*, 16(2):151–160, 1996.

- [264] J. Nievergelt and E. M. Reingold. Binary search trees of bounded balance. *SIAM Journal on Computing*, 2(1):33–43, 1973.
- [265] Ivan Niven and Herbert S. Zuckerman. *An Introduction to the Theory of Numbers*. John Wiley & Sons, fourth edition, 1980.
- [266] Alan V. Oppenheim and Ronald W. Schaffer, with John R. Buck. *Discrete-Time Signal Processing*. Prentice Hall, second edition, 1998.
- [267] Alan V. Oppenheim and Alan S. Willsky, with S. Hamid Nawab. *Signals and Systems*. Prentice Hall, second edition, 1997.
- [268] James B. Orlin. A polynomial time primal network simplex algorithm for minimum cost flows. *Mathematical Programming*, 78(1):109–129, 1997.
- [269] Joseph O’Rourke. *Computational Geometry in C*. Cambridge University Press, second edition, 1998.
- [270] Christos H. Papadimitriou. *Computational Complexity*. Addison-Wesley, 1994.
- [271] Christos H. Papadimitriou and Kenneth Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Prentice Hall, 1982.
- [272] Michael S. Paterson. Progress in selection. In *Proceedings of the Fifth Scandinavian Workshop on Algorithm Theory*, pages 368–379, 1996.
- [273] Mihai Pătraşcu and Mikkel Thorup. Time-space trade-offs for predecessor search. In *Proceedings of the 38th Annual ACM Symposium on Theory of Computing*, pages 232–240, 2006.
- [274] Mihai Pătraşcu and Mikkel Thorup. Randomization does not help searching predecessors. In *Proceedings of the 18th ACM-SIAM Symposium on Discrete Algorithms*, pages 555–564, 2007.
- [275] Pavel A. Pevzner. *Computational Molecular Biology: An Algorithmic Approach*. The MIT Press, 2000.
- [276] Steven Phillips and Jeffery Westbrook. Online load balancing and network flow. In *Proceedings of the 25th Annual ACM Symposium on Theory of Computing*, pages 402–411, 1993.
- [277] J. M. Pollard. A Monte Carlo method for factorization. *BIT*, 15(3):331–334, 1975.
- [278] J. M. Pollard. Factoring with cubic integers. In A. K. Lenstra and H. W. Lenstra, Jr., editors, *The Development of the Number Field Sieve*, volume 1554 of *Lecture Notes in Mathematics*, pages 4–10. Springer, 1993.
- [279] Carl Pomerance. On the distribution of pseudoprimes. *Mathematics of Computation*, 37(156):587–593, 1981.
- [280] Carl Pomerance, editor. *Proceedings of the AMS Symposia in Applied Mathematics: Computational Number Theory and Cryptography*. American Mathematical Society, 1990.
- [281] William K. Pratt. *Digital Image Processing*. John Wiley & Sons, fourth edition, 2007.
- [282] Franco P. Preparata and Michael Ian Shamos. *Computational Geometry: An Introduction*. Springer, 1985.

- [283] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes in C++: The Art of Scientific Computing*. Cambridge University Press, second edition, 2002.
- [284] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, third edition, 2007.
- [285] R. C. Prim. Shortest connection networks and some generalizations. *Bell System Technical Journal*, 36(6):1389–1401, 1957.
- [286] William Pugh. Skip lists: A probabilistic alternative to balanced trees. *Communications of the ACM*, 33(6):668–676, 1990.
- [287] Paul W. Purdom, Jr. and Cynthia A. Brown. *The Analysis of Algorithms*. Holt, Rinehart, and Winston, 1985.
- [288] Michael O. Rabin. Probabilistic algorithms. In J. F. Traub, editor, *Algorithms and Complexity: New Directions and Recent Results*, pages 21–39. Academic Press, 1976.
- [289] Michael O. Rabin. Probabilistic algorithm for testing primality. *Journal of Number Theory*, 12(1):128–138, 1980.
- [290] P. Raghavan and C. D. Thompson. Randomized rounding: A technique for provably good algorithms and algorithmic proofs. *Combinatorica*, 7(4):365–374, 1987.
- [291] Rajeev Raman. Recent results on the single-source shortest paths problem. *SIGACT News*, 28(2):81–87, 1997.
- [292] James Reinders. *Intel Threading Building Blocks: Outfitting C++ for Multi-core Processor Parallelism*. O'Reilly Media, Inc., 2007.
- [293] Edward M. Reingold, Jürg Nievergelt, and Narsingh Deo. *Combinatorial Algorithms: Theory and Practice*. Prentice Hall, 1977.
- [294] Edward M. Reingold, Kenneth J. Urban, and David Gries. K-M-P string matching revisited. *Information Processing Letters*, 64(5):217–223, 1997.
- [295] Hans Riesel. *Prime Numbers and Computer Methods for Factorization*, volume 126 of *Progress in Mathematics*. Birkhäuser, second edition, 1994.
- [296] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978. See also U.S. Patent 4,405,829.
- [297] Herbert Robbins. A remark on Stirling's formula. *American Mathematical Monthly*, 62(1):26–29, 1955.
- [298] D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis. An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3):563–581, 1977.
- [299] Salvador Roura. An improved master theorem for divide-and-conquer recurrences. In *Proceedings of Automata, Languages and Programming, 24th International Colloquium, ICALP'97*, volume 1256 of *Lecture Notes in Computer Science*, pages 449–459. Springer, 1997.
- [300] Y. A. Rozanov. *Probability Theory: A Concise Course*. Dover, 1969.

- [301] S. Sahni and T. Gonzalez. P-complete approximation problems. *Journal of the ACM*, 23(3):555–565, 1976.
- [302] A. Schönhage, M. Paterson, and N. Pippenger. Finding the median. *Journal of Computer and System Sciences*, 13(2):184–199, 1976.
- [303] Alexander Schrijver. *Theory of Linear and Integer Programming*. John Wiley & Sons, 1986.
- [304] Alexander Schrijver. Paths and flows—A historical survey. *CWI Quarterly*, 6(3):169–183, 1993.
- [305] Robert Sedgewick. Implementing quicksort programs. *Communications of the ACM*, 21(10):847–857, 1978.
- [306] Robert Sedgewick. *Algorithms*. Addison-Wesley, second edition, 1988.
- [307] Robert Sedgewick and Philippe Flajolet. *An Introduction to the Analysis of Algorithms*. Addison-Wesley, 1996.
- [308] Raimund Seidel. On the all-pairs-shortest-path problem in unweighted undirected graphs. *Journal of Computer and System Sciences*, 51(3):400–403, 1995.
- [309] Raimund Seidel and C. R. Aragon. Randomized search trees. *Algorithmica*, 16(4–5):464–497, 1996.
- [310] João Setubal and João Meidanis. *Introduction to Computational Molecular Biology*. PWS Publishing Company, 1997.
- [311] Clifford A. Shaffer. *A Practical Introduction to Data Structures and Algorithm Analysis*. Prentice Hall, second edition, 2001.
- [312] Jeffrey Shallit. Origins of the analysis of the Euclidean algorithm. *Historia Mathematica*, 21(4):401–419, 1994.
- [313] Michael I. Shamos and Dan Hoey. Geometric intersection problems. In *Proceedings of the 17th Annual Symposium on Foundations of Computer Science*, pages 208–215, 1976.
- [314] M. Sharir. A strong-connectivity algorithm and its applications in data flow analysis. *Computers and Mathematics with Applications*, 7(1):67–72, 1981.
- [315] David B. Shmoys. Computing near-optimal solutions to combinatorial optimization problems. In William Cook, László Lovász, and Paul Seymour, editors, *Combinatorial Optimization*, volume 20 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*. American Mathematical Society, 1995.
- [316] Avi Shoshan and Uri Zwick. All pairs shortest paths in undirected graphs with integer weights. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science*, pages 605–614, 1999.
- [317] Michael Sipser. *Introduction to the Theory of Computation*. Thomson Course Technology, second edition, 2006.
- [318] Steven S. Skiena. *The Algorithm Design Manual*. Springer, second edition, 1998.
- [319] Daniel D. Sleator and Robert E. Tarjan. A data structure for dynamic trees. *Journal of Computer and System Sciences*, 26(3):362–391, 1983.

- [320] Daniel D. Sleator and Robert E. Tarjan. Self-adjusting binary search trees. *Journal of the ACM*, 32(3):652–686, 1985.
- [321] Joel Spencer. *Ten Lectures on the Probabilistic Method*, volume 64 of *CBMS-NSF Regional Conference Series in Applied Mathematics*. Society for Industrial and Applied Mathematics, 1993.
- [322] Daniel A. Spielman and Shang-Hua Teng. Smoothed analysis of algorithms: Why the simplex algorithm usually takes polynomial time. *Journal of the ACM*, 51(3):385–463, 2004.
- [323] Gilbert Strang. *Introduction to Applied Mathematics*. Wellesley-Cambridge Press, 1986.
- [324] Gilbert Strang. *Linear Algebra and Its Applications*. Thomson Brooks/Cole, fourth edition, 2006.
- [325] Volker Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 14(3):354–356, 1969.
- [326] T. G. Szymanski. A special case of the maximal common subsequence problem. Technical Report TR-170, Computer Science Laboratory, Princeton University, 1975.
- [327] Robert E. Tarjan. Depth first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.
- [328] Robert E. Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.
- [329] Robert E. Tarjan. A class of algorithms which require nonlinear time to maintain disjoint sets. *Journal of Computer and System Sciences*, 18(2):110–127, 1979.
- [330] Robert E. Tarjan. *Data Structures and Network Algorithms*. Society for Industrial and Applied Mathematics, 1983.
- [331] Robert E. Tarjan. Amortized computational complexity. *SIAM Journal on Algebraic and Discrete Methods*, 6(2):306–318, 1985.
- [332] Robert E. Tarjan. Class notes: Disjoint set union. COS 423, Princeton University, 1999.
- [333] Robert E. Tarjan and Jan van Leeuwen. Worst-case analysis of set union algorithms. *Journal of the ACM*, 31(2):245–281, 1984.
- [334] George B. Thomas, Jr., Maurice D. Weir, Joel Hass, and Frank R. Giordano. *Thomas' Calculus*. Addison-Wesley, eleventh edition, 2005.
- [335] Mikkel Thorup. Faster deterministic sorting and priority queues in linear space. In *Proceedings of the 9th ACM-SIAM Symposium on Discrete Algorithms*, pages 550–555, 1998.
- [336] Mikkel Thorup. Undirected single-source shortest paths with positive integer weights in linear time. *Journal of the ACM*, 46(3):362–394, 1999.
- [337] Mikkel Thorup. On RAM priority queues. *SIAM Journal on Computing*, 30(1):86–109, 2000.
- [338] Richard Tolimieri, Myoung An, and Chao Lu. *Mathematics of Multidimensional Fourier Transform Algorithms*. Springer, second edition, 1997.
- [339] P. van Emde Boas. Preserving order in a forest in less than logarithmic time. In *Proceedings of the 16th Annual Symposium on Foundations of Computer Science*, pages 75–84, 1975.

- [340] P. van Emde Boas. Preserving order in a forest in less than logarithmic time and linear space. *Information Processing Letters*, 6(3):80–82, 1977.
- [341] P. van Emde Boas, R. Kaas, and E. Zijlstra. Design and implementation of an efficient priority queue. *Mathematical Systems Theory*, 10(1):99–127, 1976.
- [342] Jan van Leeuwen, editor. *Handbook of Theoretical Computer Science, Volume A: Algorithms and Complexity*. Elsevier Science Publishers and the MIT Press, 1990.
- [343] Charles Van Loan. *Computational Frameworks for the Fast Fourier Transform*. Society for Industrial and Applied Mathematics, 1992.
- [344] Robert J. Vanderbei. *Linear Programming: Foundations and Extensions*. Kluwer Academic Publishers, 1996.
- [345] Vijay V. Vazirani. *Approximation Algorithms*. Springer, 2001.
- [346] Rakesh M. Verma. General techniques for analyzing recursive algorithms with applications. *SIAM Journal on Computing*, 26(2):568–581, 1997.
- [347] Hao Wang and Bill Lin. Pipelined van Emde Boas tree: Algorithms, analysis, and applications. In *26th IEEE International Conference on Computer Communications*, pages 2471–2475, 2007.
- [348] Antony F. Ware. Fast approximate Fourier transforms for irregularly spaced data. *SIAM Review*, 40(4):838–856, 1998.
- [349] Stephen Warshall. A theorem on boolean matrices. *Journal of the ACM*, 9(1):11–12, 1962.
- [350] Michael S. Waterman. *Introduction to Computational Biology, Maps, Sequences and Genomes*. Chapman & Hall, 1995.
- [351] Mark Allen Weiss. *Data Structures and Problem Solving Using C++*. Addison-Wesley, second edition, 2000.
- [352] Mark Allen Weiss. *Data Structures and Problem Solving Using Java*. Addison-Wesley, third edition, 2006.
- [353] Mark Allen Weiss. *Data Structures and Algorithm Analysis in C++*. Addison-Wesley, third edition, 2007.
- [354] Mark Allen Weiss. *Data Structures and Algorithm Analysis in Java*. Addison-Wesley, second edition, 2007.
- [355] Hassler Whitney. On the abstract properties of linear dependence. *American Journal of Mathematics*, 57(3):509–533, 1935.
- [356] Herbert S. Wilf. *Algorithms and Complexity*. A K Peters, second edition, 2002.
- [357] J. W. J. Williams. Algorithm 232 (HEAPSORT). *Communications of the ACM*, 7(6):347–348, 1964.
- [358] Shmuel Winograd. On the algebraic complexity of functions. In *Actes du Congrès International des Mathématiciens*, volume 3, pages 283–288, 1970.
- [359] Andrew C.-C. Yao. A lower bound to finding convex hulls. *Journal of the ACM*, 28(4):780–787, 1981.
- [360] Chee Yap. A real elementary approach to the master recurrence and generalizations. Unpublished manuscript. Available at <http://cs.nyu.edu/yap/papers/>, July 2008.

- [361] Yinyu Ye. *Interior Point Algorithms: Theory and Analysis*. John Wiley & Sons, 1997.
- [362] Daniel Zwillinger, editor. *CRC Standard Mathematical Tables and Formulae*. Chapman & Hall/CRC Press, 31st edition, 2003.

İndeks

Bu indeks aşağıdaki düzenleri kullanır. Numaralar ayrıntılı şekilde alfabeleştirilmiştir; örneğin "2-3-4 ağaç", "iki-üç-dört ağaç" gibi indekslenmiştir. Etiketlenmiş bir sayfa numarası çoğunlukla, aslında kaynağın gösterildiği sayfa üzerinde şart olmadığı bir alıştırma veya problemin ilk sayfasını gösterir.

$\alpha(n)$, 574	$-$ (küme farkı), 1159
ϕ (altın oran), 59, 108 problem	$ $
$\hat{\phi}$ (altın oranın eşleniği), 59	(akış değeri), 710
$\phi(n)$ (Euler'in phi fonksiyonu), 943	(bir katarın uzunluğu), 986
$\rho(n)$ -yaklaşım algoritması, 1106, 1123	(kümenin kardinalitesi), 1161
o -notasyonu, 50-51, 64	$\times$
O -notasyonu, 45 şekil, 47-48, 64	(Kartezyen çarpım), 1162
O' -notasyonu, 62 problem	(vektörel çarpım), 1016
$\tilde{O}$ -notasyon, 62 problem	$\langle \rangle$
ω -notasyonu, 51	(dizi), 1166
Ω -notasyonu, 45 şekil, 48-49, 64	(standart kodlama), 1057
∞ -notasyonu, 62 problem	$\binom{n}{k}$ (kombinasyon), 1185
$\tilde{\Omega}$ -notasyonu, 62 problem	$\ \ $ (öklid normu), 1222
Θ -notasyonu, 44-47, 45 şekil, 64	$!$ (faktöriyel), 57
$\tilde{\Theta}$ -notasyonu, 62 problem	$\lceil \rceil$ (tavan), 54
$\{ \}$ (küme), 1158	$\lfloor \rfloor$ (zemin), 54
$\in$ (kümenin elemanı), 1158	$\sqrt[n]{}$ (alt karekök), 546
$\notin$ (kümenin elemanı değil), 1158	$\sqrt[n]{}$ (üst karekök), 546
$\emptyset$	$\sum$ (toplam), 1145
(boş dil), 1058	$\prod$ (çarpım), 1148
(boş küme), 1158	$\rightarrow$ (bitişiklik bağıntısı), 1169
$\subseteq$ (alt küme), 1159	$\leadsto$ (ulaşılabilirlik bağıntısı), 1170
$\subset$ (öz alt küme), 1159	$\wedge$ (AND), 697, 1071
$:$ (öyle ki), 1159	$\neg$ (NOT), 1071
$\cap$ (küme kesişimi), 1159	$\vee$ (OR), 697, 1071
$\cup$ (küme birleşimi), 1159	$\oplus$ (grup operatörü), 939

- ⊗ (evrişim operatörü), 901
- * (kapanış operatörü), 1058
- | (bölme bağıntısı), 927
- ⊥ (bölmez bağıntısı), 927
- ≡ (modülo n ye göre denk), 54, 1165 alıştırma
- ≠ (modülo n ye göre denk değil), 54
- $[a]_n$ (modülo n ye göre denklik sınıfı), 928
- $+_n$ (modülo n ye göre toplama), 940
- $\cdot_n$ (modülo n ye göre çarpma), 940
- $(\frac{a}{p})$ (Legendre sembolü), 982 problem
- ε (boş katar), 986, 1058
- $\square$ (önek bağıntısı), 986
- $\sqsubset$ (sonek bağıntısı), 986
- $\succ_x$ (üstte olma bağıntısı), 1022
- // (comment sembolü), 21
- $\gg$ (-den-daha-büyük bağıntısı), 547
- $\ll$ (-den-daha-küçük bağıntısı), 783
- $\leq_P$ (polinom-zamanlı indirgenebilirlik bağıntısı), 1067, 1077 alıştırma
- AA-ağacı, 338
- ABOVE, 1024
- Ackermann fonksiyonu, 585
- açgözlü algoritma, 414–450
 - da açgözlü seçme özelliği, 424–425
 - nın elemanları, 423–428
 - bir ağırlıklı kapsayan küme için, 1135 problem
 - bir ağırlıklı matroid üzerinde, 439–442
 - çevrimdışı önbelleğe almak için, 449 problem
 - çizelgeleme için, 443–446, 447–448 problem
 - Dijkstra algortması 658–664
 - dinamik programlama ile karşılaştırılan, 381, 390 alıştırma,
 - etkinlik seçimi için, 415–422
 - Huffman kodu için, 428–437
 - için çoklu-iş parçacıklı çizelgeleme, 781–783
 - içinde optimal altyapı, 425
 - kapsayan küme için, 1117–1122, 1139
 - kesirli sırt çantası problemi için, 426
 - Kruskal algoritması, 631–633
 - minumum kapsayan ağaç için, 631–638
 - para değiştirmek için, 446 problem
 - Prim algoritması, 634–636
 - ve matroidler, 437–443
- açgözlü çizelgeleme, 782
- açgözlü-seçme özelliği, 424–425
- ağırlıklı matroidin, 441
- etkinlik seçiminin, 417–418
- Huffman kodlarının, 433–434
- açık aralık, 348
- açık-adresli çırpı tablosu, 269–277
- çift çırpı fonksiyonu, 272–274, 277 alıştırma
- doğrusal deneme ile, 272
- ikinci dereceden denemeyle, 272, 283 problem
- ADD-SUBARRAY, 805 problem
- adresleme, açık *bakınız* açık-adresli kıyımlı arama tablosu
- ağ,
 - akışı, *bakınız* ağ akışı,
 - artık, 715–719
 - kabul edilebilir, 749–750
 - sıralama için, 811
- ağaç ayrıtı, 601, 603, 609
- ağaç yürüyüşü, 287, 293 alıştırma, 342, 1114
- ağaç, 1173–1180
 - ın çapı, 602 alıştırma
 - ın ikiye bölünmesi, 1181 problem
 - ın tam yürüyüşü, 1114
 - ın yüksekliği, 1177
 - ın yürüyüşü, 287, 293 alıştırma, 342, 1114
 - 2-3-4, 489, 503 problem
 - 2-3, 337, 504
 - AA-ağaçları, 338
 - ağırlık-dengeli ağaçlar, 338
 - aralık, 348–354
 - arama, *bakınız* arama ağacı,
 - AVL, 333 problem, 337
 - ayrıştırma, 1082
 - B-ağaçları, 484–504
 - bellek yığını, 151–169
 - binom, 527 problem
 - derinliğine, 603
 - dinamik, 482
 - en kısa yollar, 647–648, 673–676
 - enine, 594, 600
 - füzyon, 212, 483
 - ikili, *bakınız* ikili ağaç
 - k -komşu, 338
 - kapsayan ağaç,
 - kapsayan, *bakınız* minimum kapsayan ağaç,
 - karar, 192–193
 - kendi kendini dengeleyen, 338
 - kırmızı-siyah, kırmızı-siyah ağaç
 - köklü, 246–249, 1176
 - minimum kapsayan, *bakınız* minimum kapsayan ağaç

- optimal ikili arama, 397–404, 413
- özyineleme, 37, 88–93
- serbest, 1172–1176
- sıra-istatistiği, 339–345
- treap, 333 problem, 338
- van Emde Boas 531–560
- yayılmak, 338, 482
- yükseklik-dengeli, 333 problem
- ağırlık fonksiyonu
 - bir ağırlıklı matroidde, 439
 - bir çizge için, 591
- ağırlık-dengeli ağaç, 338, 473 problem
- ağırlık
 - ayrıtın, 591
 - bir yolun, 643
 - kesimin, 1127 alıştırma
 - ortalama, 680 problem
- ağırlıklı iki parçalı eşleme, 530
- ağırlıklı küme- örtüsü problemi, 1135 problem
- ağırlıklı matroid, 439–442
- ağırlıklı medyan, 225 problem
- ağırlıklı tepe örtüsü, 1124–1127, 1139
- ağırlıklı-birleşim sezgiseli, 566
- ağırlıksız en kısa yollar, 381
- ağırlıksız en uzun basit yollar, 382
- akış ağı, 709–714
 - nın kesimi, 720–724
 - çoklu kaynaklar ve çıkış düğümleri ile, 712
 - iki parçalı çizgeye karşılık gelen, 732
- akış korunumu, 709–710
- akış, 709–714
 - in artışı, 716
 - in değeri, 710
 - in iptali, 717
 - ağ, bir kesim boyunca, 720
 - aşırı, 736
 - durdurma, 765
 - tam sayı-değerli, 733
 - toparlama, 863
- akışı durdurma, 765
- akışın iptali, 717
- akışın korunması, 709–710
- aksiyom, olasılık için 1190
- alfabe, 995, 1057
- algoritma, 5
 - nın çalışma süresi, 25
 - nın doğruluğu, 6
 - bir teknoloji olarak, 13
 - sözcüğünün kökeni, 42
- Alice, 959
- ALLOCATE-NODE, 492
- ALLOCATE-OBJECT, 244
- alt karekök ($\sqrt[n]{}$), 546
- alt medyan, 213
- alt sınırlar,
 - ardışık tekrarlar için, 136–138, 142 alıştırma
 - asimptotik, 48
 - ayrık-küme veri yapıları için, 585
 - binom katsayıları üzerinde, 1186
 - bir optimal gezgin-tüccar turu uzunluğu, 1112–1115
 - birleştirme için, 208 problem
 - çoklu-iş parçacıklı hesaplama için, 780
 - dışbükey örtü için, 1038 alıştırma, 1047
 - eş zamanlı minimum ve maksimum için, 215 alıştırma
 - medyan bulmak için, 227
 - minimum-ağırlıklı tepe örtüsü için, 1124–1126
 - minimumu bulmak için, 214
 - optimal tepe örtüsünün ölçüsü için, 1110, 1135 problem
 - ortalama sıralama için, 207 problem
 - öncelikli-kuyruk işlemleri için, 531
 - öncesini bulmak için, 560
 - sıralama için, 191–194, 205 problem, 211, 531
 - testileri parçalara ayırmak için, 206 problem
 - toplamlar için, 1152, 1154
 - ve özyinelemeler, 67
 - ve potansiyel fonksiyonlar, 478
 - yaklaşımlar üzerinde, 1140
- alt üçgensel matris, 1219, 1222 alıştırma, 1225 alıştırma
- altağacı, 1176
 - koruma ölçülerinin, sıra-istatistiği ağaçlarında, 343–344
- altçizge-izomorfizma problemi, 1100 alıştırma
- altçizge, 1171
 - öncül, *bakınız* öncül alt çizge,
- altdizi, 391
- altgrup, 943–946
- altın oran (ϕ), 59, 108 problem
- altın oranın eşleniği ($\hat{\phi}$), 59
- altkatar, 1184
- altküme ($\subseteq$), 1159, 1161
 - nin bağımsız aileleri, 437
 - nin kalıtsal ailesi, 437
- altküme-toplam problemi,
 - için yaklaşım algoritması 1128–1134, 1139

- nin NP-tamlığı, 1097–1100
- birli hedef ile, 1101 alıştırma
- altkümelerin bağımsız ailesi, 437
- altkümelerin kalıtsal ailesi, 437
- altproblem çizgesi, 367–368
- altprogram
 - çağırma, 21, 23, 25 dipnot
 - yürütme, 25 dipnot
- altprogram yürütme, 25 dipnot
- altyol, 1170
- amaç değeri, 847, 851
- amaç fonksiyonu, 664, 847, 851
- amortize analiz, 451–478
 - nin hesaplama yöntemi, 456–459
 - nin potansiyel yöntemi, 459–463
- ağırlık-dengeli ağaçlar için, 473 problem
- ayrık-küme veri yapıları için, 566–567,
 - 568 alıştırma, 572 alıştırma, 575–581, 581–582 alıştırma
- bir yönlü çevrimsiz çizgede en kısa yollar için, 655
- derinliğine arama için, 606
- Dijkstra algoritması için 661
- dinamik tablolar için, 463–471
- enine arama için, 597
- Fibonacci bellek yığınları için, 509–512,
 - 517–518, 520–522, 522 alıştırma
- genel itme-yeniden etiketleme algoritması için, 746
- Graham taraması için, 1036
- ikili aramayı dinamik yapmak için, 473 problem
- İkincil depolama üzerindeki yığınlar için, 502 problem
- kırmızı-siyah ağaçları yeniden yapılandırmak için, 474 problem
- Knuth-Morris-Pratt algoritması için, 1006
- öne taşıma ile kendi kendini düzenleyen liste için, 476 problem
- tersine çevrilmiş bit permütasyonu için, 472 problem
- toparlama analizi, 367, 452–456
- amortize maliyet
 - hesaplama yönteminde, 456
 - potansiyel yönteminde, 459
 - toparlama analizinde, 452
- ana bellek, 484
- ana teorem, 94
 - in ispatı, 97–106
- anahtar azaltma
 - Fibonacci bellek yığınlarında, 519–522
 - 2-3-4 bellek yığınlarında, 529 problem
- anahtar değiştirme, Fibonacci bellek yığınlarında, 529 problem
- anahtar, 16, 147, 162, 229
 - açık, 959, 962
 - bir B-ağacı düğümünün medyanı, 493
 - bir doğal sayı olarak yorumlanmış, 263
 - durağan (statik), 277
 - gizli, 959, 962
 - yapay, 397
- anahtarların durağan kümesi, 277
- analiz ağacı, 1082
- AND fonksiyonu ($\wedge$), 697, 1071
- AND kapısı, 1070
- and**, kaba kodda, 22
- anti-paralel ayırtlar, 711–712
- ANY-SEGMENTS-INTERSECT, 1025
- APPROX-MIN-WEIGHT-VC, 1126
- APPROX-SUBSET-SUM, 1131
- APPROX-TSP-TOUR, 1112
- APPROX-VERTEX-COVER, 1109
- arakesit
 - belirleme, doğru parçaları kümesi için, 1021–1029, 1047
 - belirleme, iki doğru parçası için, 1017–1019
- dillerin, 1058
- kirişlerin, 345 alıştırma
- kümelerin ($\cap$), 1159
- aralarında asal ikili, 931
- aralarında asal, 931
- aralığı üçe ayırma, 348
- aralığın yüksek uç noktası, 348
- aralık ağacı, 348–354
- aralık çizgesi, 422 alıştırma
- aralık, 348
 - bulanık sıralamanın, 189 problem
- arama ağacı, *bakınız* dengeli arama ağacı, ikili arama ağacı, B-ağacı, üstel arama ağacı, aralık ağacı, optimal ikili arama ağacı, sıra-istatistiği ağacı, kırmızı-siyah ağaç, yayvan ağaç, 2-3 ağacı, 2-3-4 ağacı,
- arama, 22 alıştırma
 - açık-adresli çırpı tablolarında, 270–271
 - aralık ağaçlarında, 350–353
 - B-ağacında, 491–492
 - bağlı listelerde, 237
 - bir kesin aralık için, 354 alıştırma
 - bir sırasız dizide, 143 problem
 - doğrudan-adresli tablolarda, 254

- doğrusal arama, 22 alıştırma
- ikili arama ağaçlarında, 289–291
- ikili arama, 39 alıştırma, 799–800
- kırmızı-siyah ağaçlarda, 311
- kompakt listelerde, 250 problem
- ön van Emde Boas yapılarında, 540–541
- Van Emde Boas ağaçlarında, 550
- zincirli çırpı tablolarında, 258
- araya sokma sıralaması, 12, 16–20, 25–27
 - birleştirerek sıralama ile karşılaştırılan, 14 alıştırma
 - birleştirerek sıralamada, 39 problem
 - hızlı sıralama ile karşılaştırılan, 178 alıştırma
 - hızlı sıralamada, 185 alıştırma
 - için karar ağacı, 192 şekil
 - ikili arama kullanımı, 39 alıştırma
 - kova sıralamasında, 201–204
- araya sokma
 - 2-3-4 bellek yığınlarında, 529 problem
 - açık-adresli çırpı tablolarında, 270
 - aralık ağaçlarında, 349
 - B-ağacında, 493–497
 - bağlı listelerde, 237–238
 - bellek yığınlarında, 164
 - bir üstüste ikili ağaç ile bir bit vektöründe, 534
 - d-li bellek yığınlarında, 167 problem
 - dinamik tablolarda, 464–467
 - doğrudan-adress tablolarında, 254
 - Fibonacci bellek yığınlarında, 510–511
 - ikili arama ağaçlarında, 294–295
 - kırmızı-siyah ağaçlarda, 315–323
 - kuyruklarda, 234
 - ön van Emde Boas yapılarında, 544
 - sabit yüksekliğin bir üstüste ağacı ile bir bit vektöründe, 534
 - sıra-istatistiği ağaçlarında, 343
 - tarama-doğrusu durumlarında, 1024
 - temel, 465
 - treaps da, 333 problem
 - van Emde Boas ağaçlarında, 552–554
 - yığınlarda, 232
 - Young tablosunda, 167 problem
 - zincirlenmiş çırpı tablolarında, 258
- ard
 - arda eklenme
 - dillerin, 1058
 - katarların, 986
- ardıl
 - bağlı listelerde, 236
 - bir üstüste ikili ağacı ile bir bit vektöründe, 533
 - i'nciyi bulma, sıra-istatistiği ağacında bir düğümün, 344 alıştırma
 - ikili arama ağaçlarında, 291–292
 - kırmızı-siyah ağaçlarda, 311
 - ön van Emde Boas yapılarında, 543–544
 - sabit yüksekliğin bir üstüste ağacı ile bir bit vektöründe, 535
 - sıra-istatistiği ağaçlarında, 347 alıştırma
 - Van Emde Boas ağaçlarında, 550–551
- ardışık tekrarlar, 135–139
- aritmetik komutları, 23
- aritmetik seriler, 1146
- aritmetik, modüler, 54, 939–946
- arka ayırıt, 609, 613
- artan dizayn yöntemi, 29
 - bir dışbükey örtü bulmak için, 1030
- artık ağ, 715–719
- artık ayırıt, 716
- artık kapasite, 716, 719
- artık, 54, 928, 982 problem
- asal dağılım fonksiyonu, 965
- asal doğrusal program, 880
- asal sayı teoremi, 965
- asal sayı, 928
 - nın yoğunluğu, 965–966
- asallık testi, 965–975, 983
 - Miller-Rabin testi, 968–975, 983
 - sözde asallık testi, 966–968
- asimptotik alt sınır, 48
- asimptotik notasyon, 43–53, 62 problem
 - ve çizge algoritmaları, 588
 - ve toplamaların doğrusallığı, 1146
- asimptotik notasyonun yansımahlığı, 51
- asimptotik olarak daha büyük, 52
- asimptotik olarak daha küçük, 52
- asimptotik olarak negatif olmayan, 45
- asimptotik olarak pozitif, 45
- asimptotik olarak sıkı sınır, 45
- asimptotik üst sınır, 47
- asimptotik verimlilik, 43
- asimptotik gösterimin geçişliliği, 51
- asimptotik notasyonun transpoze simetriği, 52
- aşağı taşıması
 - bir kuyruğun, 234
 - bir yığının, 233
- aşağıdan yukarıya yöntemi, dinamik programlama için, 365
- aşırı akış, 736
- aşırı yüklü bir tepenin boşaltılması, 751
- aşıkâr bölün, 928

- aşık ar olmayan güç, 933 alıştırma
- ata, 1176
 - en küçük ortak, 584 problem
- atama,
 - çoklu, 21
 - doğruluk, 1072, 1079
 - gerçekleme, 1072, 1079
- atlama listesi, 338
- AVL ağacı, 333 problem, 337
- AVL-INSERT, 333 problem
- ayırılması,
 - 2-3-4 ağaçlarının, 503 problem
 - B-ağacı düğümlerinin, 493-495
- ayrık Fourier dönüşümü, 9, 909
- ayrık kümeler, 1161
- ayrık logaritma teoremi, 955
- ayrık logaritma, 955
- ayrık olasılık dağılımı, 1191
- ayrık rassal değişken 1196-1201
- ayrık-küme ormanı, 568-572
 - nın analizi, 575-581, 581 alıştırma
 - nin rank özellikleri, 575, 581 alıştırma
 - ayrıca bakınız ayrık-küme veri yapısı
- ayrık-küme veri yapıları, 561-585
 - nın ayrık-küme-ormanı gösterimi, 568-572
 - nin bağlı liste gösterimi, 564-568
 - nin analizi, 575-581, 581 alıştırma
 - nin doğrusal-zaman özel durumu, 585
 - bağımlı bileşenlerde, 562-564
 - çevrimdışı en küçük ortak atalarda, 584 problem
 - çevrimdışı minimumda, 582 problem
 - derinliğine kararda, 583 problem
 - görev çizelgelemede, 448 problem
 - Kruskal algoritmasında, 631
- ayrıt bağlanabilirliği, 731 alıştırma
- ayrıt kümesi, 1168
- ayrıt, 1168
 - in ağırlığı, 591
 - in kapasitesi, 709
 - in nitelikleri, 592
 - ağaç, 601, 603, 609
 - antiparalel, 711-712
 - artık, 716
 - çağrı, 778
 - çapraz, 609
 - derinliğine arama içinde sınıflandırma, 609-610
 - doğurmak, 778
 - doygun, 739
 - dönüş, 779
 - enine arama içinde sınıflandırma, 621 problem
 - geri, 609
 - güvenli, 626
 - hafif, 626
 - ileri, 609
 - kabul edilebilir 749
 - kabul edilemez, 749
 - köprü, 621 problem
 - kritik, 729
 - negatif-ağırlıklı, 645-646
 - süreklilik, 778
- ayrıtların sınıflandırılması,
 - bir çoklu-iş parçacıklı çevrimsiz yönlü çizge-
de, 778-779
 - derinliğine aramada, 609-610, 611 alıştırma
 - enine aramada, 621 problem
- 1-yaklaşım algoritması, 1107
- 1'in aşık ar olmayan karekökü, modülo n , 956
- B^* -ağacı, 489 dipnot
- B^+ -ağacı, 488
- B-TREE-EKLE 495
- B-TREE-DELETE, 499
- B-TREE-INSERT-NONFULL, 496
- B-TREE-INSERT, 492
- B-TREE-SEARCH, 492, 499 alıştırma
- B-TREE-SPLIT-CHILD, 494
- B-ağacı, 484-504
 - de araya sokma, 493-497
 - deki bir düğümü ayırma, 493-495
 - deki tam düğüm, 489
 - den silme, 499-502
 - nin minimum anahtarı, 497 alıştırma
 - nin minimum derecesi, 489
 - nin özellikleri, 488-491
 - nin yüksekliği, 489-490
 - 2-3-4 ağaçları, 489
 - arama, 491-492
 - kırmızı-siyah ağaçlarıyla karşılaştırılan, 484, 490
 - oluşturma, 492
- BAD-SET-COVER-INSTANCE, 1122 alıştırma
- bağ
 - binom ağaçlarının, 527 problem
 - bir ayrık-küme ormanı içinde ağaçların, 570-571
 - Fibonacci-bellek yığını köklerinin, 513

- bağımlı
 - ayrıca bakınız* bağımsız
 - doğrusal, 1223
 - ve rassal gösterge değişkenleri, 119
- bağımsız dizilimler, 789
- bağımsız küme, 1101 problem
 - görevlerin, 444
- bağımsızlık
 - dinamik programlamadaki altproblemlerin, 383–384
 - olayların, 1192–1193, 1195 alıştırma
 - rassal değişkenlerin, 1197
- bağıntı, 1163–1166
- bağlantılı bileşen, 1170
 - ayrık küme veri yapıları kullanarak tanımlanan, 562–564
 - derinliğine arama kullanarak tanımlanan, 612 alıştırma
- bağlantılı çizge, 1170
- bağlayıcı, 1079
- bağlı liste, 236–241
 - de araya sokma, 237–238
 - den silme, 238
 - arama, 237, 268 alıştırma
 - ayrık kümelerle uygulamak, 564–568
 - kendi kendini düzenleyen, 476 problem
 - kompakt, 245 alıştırma, 250 problem
 - komşu listesi, 750
- bakınız* yönlü çizge
- BALANCE, 333 problem
- basamaklı kesme, 520
- basit çevrim, 1170
- basit çizge, 1170
- basit çokgen, 1020 alıştırma
- basit düzgün çırpılama, 259
- basit şablon hesaplaması, 809 problem
- basit yol, 1170
 - en uzun, 382, 1048
- başarılı, bir Bernoulli deneyinde, 1201
- başarısızlık, Bernoulli denemesinde, 1201
- başlangıç dizilimi, 779
- başlangıç durumu, 995
- başlangıç zamanı, 415
- başlık
 - bağlı listesinin, 236
 - disk sürücüsünde, 485
 - kuyruğun, 234
- Bayes teoremi, 1194
- baz fonksiyon, 835
- bebek yüzölü, 602 alıştırma
- beklenen çalışma süresi 28, 117
- beklenen değer, 1197–1199
 - binom dağılımının, 1204
 - bir rassal gösterge değişkeninin, 118
 - geometrik dağılımın, 1202
- beklenti, *bakınız* beklenen değer
- beklentinin doğrusallığı, 1198
 - ve rassal gösterge değişkenleri, 119
- belirleme yarışı, 788
- belirleyici algoritma, 123
 - çoklu-iş parçacıklı, 787
- belirsiz çoklu-iş parçacıklı algoritma, 787
- belirsiz doğrusal denklemler sistem, 814
- belirsiz polinom zamanlı, 1064 dipnot
 - ayrıca bakınız* NP,
- bellek hiyerarşisi, 24
- bellek yığını özelliği, 152
 - ne karşı ikili arama ağacı özelliği, 289 alıştırma
 - nin bakımı, 154–156
- bellek yığını sıralaması, 151–169
- bellek yığını, 151–169
 - nda araya koyma, 164
 - ndaki anahtar artışı, 163–164
 - ndan maksimum anahtarın ayıklanması, 163
 - ndan silme, 166 alıştırma
 - nın maksimum anahtarı, 163
 - nın yüksekliği, 153
 - 2-3-4, 529 problem
 - binom, 527 problem
 - birleşebilir bir yığın uygulamak için, 506
 - birleşebilme, *bakınız* birleşebilen bellek yığını
 - d-li, 167 problem, 706 problem
 - Dijkstra algoritmasında, 662
 - Fibonacci bellek yığınları ile kıyaslanan, 506–507
 - Fibonacci, *bakınız* Fibonacci bellek yığını gevşemiş, 530
 - Huffman'ın algoritmasında, 433
 - inşaası, 156–159, 166 problem
 - Johnson'un algoritmasında, 704
 - maksimum yığın, 152
 - minimum bellek yığını, 153
 - öncelikli kuyruk olarak, 162–166
 - potansiyel yöntem ile analiz edilen, 462 alıştırma
 - Prim algoritmasında, 636
 - toplanmış gereksiz bilgi stoğu olarak, 151
 - üzerinde işlemlerin çalışma süreleri, 506
 - şekil

- ve treaps, 333 problem
- bellek, 484
- Bellman-Ford Algoritması, 651–655, 682
 - fark kısıtlılarının sistemlerini çözmek için, 668
 - Johnson algoritmasında, 702–704
 - tüm ikililerin en kısa yolları, 684
 - ve amaç fonksiyonları, 670 alıştıırma
 - Yen'in iyileştirdiği, 678 problem
- Bellman-Ford algoritmasına Yen'in iyileştirmesi, 678 problem
- BELLMAN-FORD, 651
- BELOW, 1024
- Bernoulli testi, 1201
 - ve ardışık tekrarlar, 135–139
 - ve toplar ve kutular, 133–134
- beyaz tepe (noktası), 594, 603
- beyaz yol teoremi, 608
- BFS, 595
- BIASED-RANDOM, 117 alıştıırma
- bileşen çizgesi, 617
 - ne tanık, 968
- bileşen,
 - bağlantılı 1170
 - güçlü bağlantılı, 1170
 - iki bağlantılı, 621 problem
- bileşik sayı, 928
- BINARY-SEARCH, 799
- binom açılımı 1186
- binom ağacı, 527 problem
- binom bellek yığını, 527 problem
- binom dağılımı, 1203–1206
 - nın kuyruğu, 1208–1215
 - nın maksimum değeri, 1207 alıştıırma
 - ve toplar ve kutular, 133
- binom katsayısı, 1186–1187
- bir ağacın çapı, 602 alıştıırma
- bir ağacın gezilebilirliği, 287, 293 alıştıırma, 342, 1114
- bir ağacın ikiye bölünmesi, 1181 problem
- bir ağacın tam yürüyüşü, 1114
- bir ağacın yürüyüşü, 287, 293 alıştıırma, 342, 1114
- bir akışın artışı, 716
- bir algoritma tarafından verilen karar 1058–1059
- bir algoritmanın doğruluğu, 6
- bir anahtarın artması, bir maksimum bellek yığınınında, 163–164
- bir aralığın düşük uç noktası, 348
- bir asal mod a göre bir polinomun sıfırı, 950 alıştıırma
- bir ayrıklar kümesine ilişkin, 626
- bir çalışma süresi yığımindan al, 188 problem
- bir çizgenin çevrimi, 1170
 - hamilton, 1049, 1061
 - minimum ortalama-ağırlıklı, 680 problem
 - negatif-ağırlıklı, *bakınız* negatif-ağırlıklı çevrim,
 - ve en kısa yollar, 646–647
- bir çokgenin çekirdeği, 1038 alıştıırma
- bir çokgenin içi, 1020 alıştıırma
- bir çokgenin sınırı, 1020 alıştıırma
- bir çokgenin yüzü, 1020 alıştıırma
- bir desen oluşumu, 985
- bir dışbükey örtü zinciri, 1038
- bir dilin tamlığı, 1077 alıştıırma
- bir dinamik tablonun açılımı, 464–467
- bir döngünün ortalama ağırlığı, 680 problem
- bir Fibonacci bellek yığımindaki çocuk listesi, 507
- bir Fibonacci bellek yığını budamak, 529 problem
- bir fonksiyonun argümanı, 1166–1167
- bir kesim boyunca ağ akışı, 720
- bir küme genişlemesi, 438
- bir kümenin elemanı olmayan ($\notin$), 1158
- bir kümenin kardinalitesi ($|$), 1161
- bir kümenin parçalanışı, 1161, 1164
- bir kümenin temsilcisi, 561
- bir listeyi kırpm, 1130
- bir matrisin negatifi, 1220
- bir matrisin öngörüntüsü, 1228 problem
- bir matroidde maksimal altküme, 438
- bir nesnenin niteliği, 21
- bir noktanın gölgesi 1038 alıştıırma
- bir optimal çözümü yeniden inşa etmek, dinamik programlamada, 387
- bir özyinelemenin çözmek için Akra-Bazzi yöntemi, 112–113
- bir özyinelemeyi çözmek için ana yöntem, 93–97
- bir polinom ile interpolasyon, 901, 906 alıştıırma
 - birimin karmaşık köklerinde, 912–913
- bir polinomun değerlendirilmesi, 41 problem, 900, 905 alıştıırma
 - nin türevleri, 922 problem
 - birden fazla noktada, 923 problem
- bir serinin integrallenmesi, 1147
- bir serinin türetimi, 1147
- bir serinin türevi, 1147
- bir sonlu otomatanın durumu, 995
- bir tepeyi ayırma, 1169
- bir toplamın sınırı, 1149–1156

- bir vektörün, 1222
- bir yığının altı, 233
- bir yığının en üstü, 232
- bir yönlü çizgenin karesi, 593 alıştırma
- birbirinden bağımsız olaylar, 1193
- birbirini dışlayan olaylar, 1190
- birebir eşleme, 1167
- birebir fonksiyon, 1167
- birebir fonksiyon, 1167
- birebir ve örten fonksiyon, 1167
- birim alt-üçgensel matris, 1219
- birim matris, 1218
- birim üst-üçgensel matris, 1219
- birim vektör, 1218
- birim-zaman görevi, 443
- birim, 939
- birim(1), 928
- birimin esas kökü, 907
- birimin karmaşık kökü, 906
 - nde interpolasyon, 912–913
- birincil bellek, 484
- birincil kümeleme, 272
- birleşme
 - 2-3-4 ağaçlarının, 503 problem
 - kırmızı-siyah ağaçların, 332 problem
- birleşim işlemi, 939
- birleşim, -in çoklu-iş parçacıklı hesaplamaları, 784 şekil
- birleşim
 - dillerin, 1058
 - dinamik kümelerin *bakınız* birleştirici
 - kümelerin ($\cup$), 1159
- birleşimsel devre, 1071
- birleşimsel öge, 1070
- birleştirerek sıralama, 12, 30–37
 - da araya sokma sıralamasının kullanımı, 39 problem
 - daki araya sokma sıralaması ile kıyaslama, 14 alıştırma
 - için çoklu-iş parçacıklı algoritma, 797–805, 812
- birleştirilebilir bellek yığını, 481, 505
 - nda işlemlerin çalışma süreleri, 506 şekil
 - nın bağlı liste gösterimi, 250 problem
 - 2-3-4 bellek yığınları, 529 problem
 - ayrıca bakınız* Fibonacci yığını,
 - binom bellek yığınları, 527 problem
 - gevşetilmiş bellek yığınları, 530
- birleştirilebilir maks-bellek yığını, 250 dipnot, 481 dipnot, 505 dipnot
- birleştirilebilir min-bellek yığını, 250 dipnot, 481 dipnot, 505
- birleştirme
 - 2-3-4 bellek yığınlarının, 529 problem
 - bağlı listelerin, 241 alıştırma
 - bellek yığınlarının, 506
 - Fibonacci bellek yığınlarının, 511–512
- birleştirme
 - için alt sınırlar, 208 problem
 - için çoklu-iş parçacıklı algoritma, 798–801
 - iki sıralı dizilerin, 30
 - k sıralı listelerin, 166 alıştırma
- birli, 1056
- bit düzeyinde ters çevrilmiş ikili sayaç, 472 problem
- bit işlemi, 927
 - Öklid algoritmasında, 981 problem
- bit vektörü, 255 alıştırma, 532–536
- bit-düzeyinde ters permütasyon, 472 problem, 918
- BIT-REVERSE-COPY, 918
- BIT-REVERSED-INCREMET, 472 problem
- bitiş zamanı, 444
- bitiş zamanı, etkinlik seçiminde, 415
- bitişik tepeler, 1169
- bitişiklik bağıntısı ($\rightarrow$), 1169
- bitişiklik-listesi gösterimi, 590
 - bir cırpı tablosu ile değiştirilen, 593 alıştırma
- bitişiklik-matrisi gösterimi, 591
- bitmiş köşe, 603
- Bob, 959
- boole bağlayıcı, 1079
- boole birleşimsel devresi, 1071
- boole birleşimsel eleman, 1070
- Boole eşitsizliği, 1195 alıştırma
- boole fonksiyonu, 1187 alıştırma
- boole formülü, 1049, 1066 alıştırma, 1079, 1086 alıştırma
- boole matris çarpımı, 832 alıştırma
- Bor'uvka algoritması, 641
- boş ağaç, 1178
- boş ağaç, 1178
- boş dil ($\emptyset$), 1058
- boş durum, 1190
- boş katar (ϵ), 986, 1058
- boş küme ($\emptyset$), 1158
- boş küme yasaları, 1159
- boş vektör, 1224
- boş yığın, 233

- boşluk karakteri, 989 alıştırma, 1002 alıştırma
 boşluk sezgiseli, 760 alıştırma, 766
 BOTTOM-UP-CUT-ROD, 366
 böl-ve-yönet yöntemi, 30–35, 65
 -nin analizi, 34–35
 birleştirerek sıralama için, 30–37, 797–805
 çarpma için, 920 problem
 çoklu-iş parçacıklı birleştirerek sıralama için, 797–805
 çoklu-iş parçacıklı matrix çarpımı için, 792–797
 dışbükey örtü bulmak için, 1030
 dinamik programlamaya dair, 359
 en yakın nokta çiftini bulmak için, 1040–1043
 hızlı Fourier dönüşümü için, 909–912
 hızlı sıralama için, 170–190
 için özyinelemeleri çözme, 83–106, 112–113
 ikili arama için, 39 alıştırma
 ikili sayıdan onluk tabana dönüştürme için, 933 alıştırma
 maksimum-altıdizi problemi için, 68–75
 matrisin tersi için, 829–831
 seçim için, 215–224
 Strassen algoritması için, 79–83
 bölün, 927–928
 ortak, 929
 ayrıca bakınız en büyük ortak bölen,
 böler bağıntısı ($\mid$), 927
 bölge, elverişli, 847
 bölme adımı, böl-ve-yönet içinde, 30, 65
 bölme komutu, 23
 bölme teoremi, 928
 bölme yöntemi, 263, 268–269 alıştırma
 bölmez bağıntısı ($\nmid$), 927
 bölüm, 928
 BUBBLESORT, 40problem
 BUCKET-SORT, 201
 buda ve ara yöntemi, 1030
 BUILD-MAX-HEAP, 157
 BUILD-MAX-HEAP', 167 problem
 BUILD-MIN-HEAP, 159
 bulanık sıralama, 189 problem
 bulma süresi, derinliğine aramada, 605
 bulunan tepe, 594, 603
 bütünlük teoremi, 734
 büyük-oh notasyonu, 45 şekil, 47–48, 64
 büyük-omega notasyonu, 45 şekil, 48–49, 64
 büyüme oranı, 28
 büyüme sırası, 28
 büzülme
 bir ayırıt tarafından bir yönsüz çizgenin, 1172
 bir dinamik tablonun, 467–471
 bir matroidin, 442
by, kaba kodda, 21
 Carmichael sayısı, 968, 975 alıştırma
 CASCADING-CUT, 519
 ceza, 444
 CHAINED-HASH-DELETE, 258
 CHAINED-HASH-INSERT, 258
 CHAINED-HASH-SEARCH, 258
 cıvıltı dönüşümü, 914 alıştırma
 Cilk, 774, 812
 Cilk++, 774, 812
 CIRCUIT-SAT, 1072
 CLIQUE, 1087
 CNF (ve- li normal biçim), 1049, 1082
 CNFgerçekleştirilebilirliği, 1127 alıştırma
 co-NP (karmaşıklık sınıfı), 1064
comment, kaba kodda ($//$), 21
 COMPACT-LIST-SEARCH, 250 problem
 COMPACT-LIST-SEARCH', 251 problem
 COMPACTIFY-LIST, 245 alıştırma
 COMPARE-EXCHANGE, 208 problem
 COMPUTE-PREFIX-FUNCTION, 1006
 COMPUTE-TRANSITION-FUNCTION, 1001
 CONNECTED-COMPONENTS, 563
 CONSOLIDATE, 516
 COUNTING-SORT, 195
 CREATE-NEW-RS-VEB-TREE, 557 problem
 CUT-ROD, 363
 CUT, 519
 cümlecik, 1081–1082
 çağrı ayrıtı, 778
 çağrı,
 bir altprogramın, 23, 25 dipnot
 bir çoklu-iş parçacıklı hesaplamada, 776
 değer tarafından, 21
 çakışmayan katar deseni, 1002 alıştırma
 çalışma süresi yığımına itme, 188 problem
 çapraz ayırıt, 609
 çapraz kesim, 626
 çarpan, 927
 en küçük ortak, 939 alıştırma
 modülo n ye göre bir elemanın, 946–950
 skaler, 1220
 çarpanlarına ayırma, 975–980, 984
 tek, 931

- çarpım ($\prod$), 1148
 - in kuralı, 1184
 - çapraz, 1016
 - dış, 1222
 - iç, 1222
 - Kartezyen, 1162
 - matrislerin, 1221, 1226 alıştırma
 - polinomların, 899
 - skaler akış, 714 alıştırma
- çarpım kuralı, 1184
- çarpım
 - için böl-ve-yönet yöntemi, 920 problem
 - kompleks sayıların, 83 alıştırma
 - matris zincirinin, 370–378
 - matris-vektör, çoklu-iş parçacıklı, 785–787, 789–790,, 792 alıştırma
 - matrislerin, *bakınız* matris çarpımı,
 - modülo n ($\cdot n$) ye göre bir elemanın, 940
 - polinomların, 899
- çarpımsal ters, modülo n ye göre, 949
- çarpışma-dayanıklı çarpı fonksiyonu, 964
- çarpışma, 257
 - açık adresleme tarafından yeniden çözülme, 269–277
 - zincirleme tarafından yeniden çözülme, 257–260
- çarpma yöntemi, 263–264
- çevrim-dışı problemi
 - en küçük ortak atalar, 584 problem
 - minimum, 582 problem
 - önbelleğe alma, 449 problem
- çevrim-içi kiralama problemi, 139–141
- çevrimiçi çoklu-iş parçacıklı zamanlayıcı, 781
- çevrimiçi dış büyükey örtü problemi, 1039 alıştırma
- çevrimsiz çizge, 1170
 - matroidlere ilişkin, 448 problem
- çıkarma komutu, 23
- çıkan değişken, 867
- çıkış düğümlü tepe noktası, 593 alıştırma, 709, 712
- çıkış yelpazesi, 1071
- çıktı
 - bir algoritmanın, 5
 - bir birleşimsel devrenin, 1071
 - bir mantık kapısının, 1070
- çırpı değeri, 256
- çırpı fonksiyonlarının evrensel koleksiyonu, 265
- çırpı fonksiyonu, 256, 262–269
 - (ϵ)-evrensel, 269 alıştırma
 - çarpışma-dayanıklı, 964
 - için bölme yöntemi, 263, 268–269 alıştırma
 - için çarpma yöntemi, 263–264
 - yardımcı, 272
 - evrensel, 265–268
- çırpı tablosu, 256–261
 - ayrıca bakınız* çırpı
 - dinamik, 471 alıştırma
 - ikincil, 278
- çırpılama, 253–285
 - açık adresleme ile, 269–277
 - bitişik listeleri yerleştirmek için 593 alıştırma
 - evrensel, 265–268
 - ikili, 272–274, 277 alıştırma
 - k -evrensel 284 problem
 - kusursuz, 277–282, 285
 - ön belleğe almada 365, 387
 - zincirleme ile, 257–260, 283 problem
- çift çırpılama, 272–274, 277 alıştırma
- çift taraflı işlem, 1017
- çift taraflı kuyruk, 236 alıştırma
- çift düze (bitonic) Öklid gezgin-satıcı problemi, 405 problem
- çift düze dizi, 682 problem
- çift düze tur, 405 problem
- Çin kalan teoremi, 950–954, 983
- çizelgeleme için doğal biçim, 444
- çizelgeleme, 443–446, 447 problem, 450, 1104
 - problem, 1136 problem
- çizelgeleme, 444, 1136 problem
 - olay-noktası, 1023
- çizelgeleyici, çoklu iş parçacıklı hesaplamalar
 - için, 777, 781–783, 812
 - açgözlü, 782
 - için iş-çalma algoritması, 812
 - merkezi 782
- çizge, 1168–1173
 - deki en kısa yol, 597
 - nin bitişik-listesi gösterimi, 590
 - nin bitişik-matris gösterimi, 591
 - nin derinliğine araması, 603–612, 623
 - nin enine araması, 594–602, 623
 - nin sıklık matrisi, 448 problem, 593 alıştırma
 - nin nitelikleri, 588, 592
 - nin renklendirilmesi, 1103 problem
 - nin tamamlayıcısı, 1090
 - nin turu, 1096
 - (ϵ)-yoğun, 706 problem
 - ağırlıklı, 591
 - altproblem, 367–368

- aralık, 422 alıştırma
- ayrıca bakınız* yönlü çevrimsiz çizge, yönlü çizge, akış ağı, yönsüz çizge, ağaç
- Bileşen, 617
- dinamik, 562 dipnot
- hamilton olmayan, 1061
- hamilton, 1061
- için algoritmalar, 587–766
- istatistik, 562 dipnot
- kısıt, 666–668
- seyrek, 589
- tek başına bağlı, 612 alıştırma
- ve asimptotik notasyon, 588
- yoğun, 589
- çizgede ulaşılabilirlik ($\leadsto$), 1170
- çizgesel matroid 437–438, 642
- çocuk
 - bir çoklu-iş parçacıklı hesaplamada, 776
 - bir ikili ağaçta, 1178
 - bir köklü ağaçta, 1176
- çok çekirdekli bilgisayar, 772
- çok işlemcili çip , 772
- çok işlemcili, 772
- çokboyutlu hızlı Fourier dönüşümü, 921 problem
- çokgen, 1020 alıştırma
 - in çekirdeği, 1038 alıştırma
 - yıldız-şekilli, 1038 alıştırma
- çokgenin dışı, 1020 alıştırma
- çoklu atama, 21
- çoklu çizge, 1172
 - denk yönsüz çizgeye dönüştüren, 593 alıştırma
- çoklu iş parçacıklı hesaplamanın iş yükü, 779
- çoklu kaynaklar ve çıkış düğümleri, 712
- çoklu komut, 23
- çoklu küme, 1158 dipnot
- çoklu-iş parçacıklı algoritma, 10, 772–812
 - basit bir hesaplama şablonu için, 809 problem
 - birleştirerek sıralama için, 797–805, 812
 - birleştirme için, 798–801
 - doğrusal denklemlerin çözüm sistemleri için, 806 problem
 - Fibonacci sayılarının hesaplanması için, 774–780
 - Floyd-Warshall algoritması, 797 alıştırma
 - hızlı Fourier dönüşümü için, 804 alıştırma
 - hızlı sıralama için, 811 problem
 - indirgeme için, 807 problem
 - LU ayrışımı için, 806 problem
 - LUP ayrışımı için, 806 problem
 - matris çarpımı için, 792–797, 806 problem
 - matris traspozesi için, 792 alıştırma, 797 alıştırma
 - matris-vektör çarpımı için, 785–787, 789–790, 792 alıştırma
 - matrisin tersi için, 806 problem
 - medyan için, 805 alıştırma
 - örnek hesaplama için, 807 problem
 - parçalama için, 804 alıştırma
 - sıra-istatistiği için, 805 alıştırma
 - Strassen algoritması, 795–796
- çoklu-iş parçacıklı birleştirme, 784 şekil
- çoklu-iş parçacıklı çizelgeleme, 781–783
- çoklu-iş parçacıklı hesaplama, 777
- çoklu-ürün akışı, 862–863
 - minimum-maliyetli, 864 alıştırma
- çöp toplama, 151, 243
- çözülebilirlik, 1048
- çözüm
 - bir hesaplama problemine, 6
 - bir somut probleme, 1055
 - bir soyut probleme, 1054
 - doğrusal denklemlerin bir sistemine, 814
 - elverişsiz, 851
 - elverişli, 665, 846, 851
 - optimal, 851
 - temel, 866
- çözümü tahmin etme, yerine koyma yönteminde, 84–85
- çubuk-kesme, 360–370, 390 alıştırma
- d*-düzgün çizge, 736 alıştırma
- d*-li bellek yığını, 167 problem
 - en-kısa yol algoritmalarında, 706 problem
- DAG-SHORTEST-PATHS, 655
- dag, *bakınız* yönlü çevrimsiz çizge,
- dağılım, 223 alıştırma
- dağılım
 - asal sayıların, 965
 - ayrık, 1191
 - binom, 1203–1206
 - düzgün, 1191
 - geometrik, 1202–1203
 - girdilerin, 116, 122
 - olasılık, 1190
 - seyrek-örtülü, 1046 problem
 - sürekli düzgün, 1192
- dağıtılmış bellek, 772

- dairesel bağlı liste, 236
 - ayrıca bakınız* bağlı liste
- dairesel, nöbetçili çift yönlü bağlı listeler, 239
- dallanma faktörü, B-ağaçlarında 487
- dallanma komutları, 23
- darboğazlı gezgin-satıcı problemi, 1117 alıştırma
- darboğazlı kapsayan ağaç, 640 problem
- DECREASE-KEY, 162, 505
- DECREASE-KEY, 456 alıştırma
- değeri
 - amaç, 847, 851
 - bir akışın, 710
 - bir fonksiyonun, 1166
- değişken değiştirme, yerine koyma yönteminde, 86–87
- değişken-uzunluklu kod, 429
- değişken
 - ayrıca bakınız* rastgele gösterge değişkeni,
 - çıkan, 867
 - gevşek, 855
 - giren, 867
 - kaba kodda, 21
 - rassal, 1196–1201
 - temel olmayan, 855
 - temel, 855
- değişmeli grup, 940
- değişmeli işlem, 940
- değiştirilen oyuncunun üzerindeki değer, 411
 - problem
- DELETE-LARGER-HALF, 463 alıştırma
- DELETE, 230, 505
- DeMorgan yasaları
 - kümeler için, 1160, 1162 alıştırma
 - önergeler mantığı için, 1083
- den-daha-büyük ($\ll$), 574
- den-daha-küçük ($\gg$), 783
- deneme dizisi, 270
- deneme, 270, 282 problem
 - ayrıca bakınız* doğrusal deneme, ikinci dereceden deneme
- deneme, Bernoulli, 1201
- deneyerek bölme, 966
- dengeli arama ağacı
 - 2-3 ağaçları, 337, 504
 - 2-3-4 ağaçları, 489, 503 problem
 - AA-ağaçları, 338
 - ağırlık-dengeli ağaçlar, 338, 473 problem
 - AVL ağaçları, 333 problem, 337
 - B-ağaçları, 484–504
 - k -komşu ağaçları, 338
 - kendini dengeleyen ağaçlar, 338
 - kırmızı-siyah ağaçlar, 308–338
 - treaps, 333 problem, 338
 - yayvan ağaçlar, 338, 482
- denk doğrusal programlar, 852
- denk olmayan ($\neq$), 54
- denklem
 - normal, 837
 - özyineleme, *bakınız* özyineleme,
 - ve asimptotik notasyon, 49–50
- denklik bağıntısı, 1164
 - ve modüler denklik, 1165 alıştırma
- denklik sınıfı, 1164
 - modülo n ($[a]_n$) ye göre, 928
- denklik, modüler ($\equiv$), 54, 1165 alıştırma
- depo komutu, 23
- depolama yönetimi, 151, 243–244, 245 alıştırma, 261 alıştırma
- DEQUEUE, 235
- derece-sınırı 898
- derece
 - bir binom ağacı kökünün, 527 problem
 - bir düğümün, 1177
 - bir polinomun, 55, 898
 - bir tepenin, 1169
 - maksimum, bir Fibonacci bellek yığınının, 509, 523–526
 - minimum, bir B-ağacının, 489
- derecede, 1169
- deriliğine arama, 603–612, 623
 - eklem noktalarını, köprüleri ve iki bağlantılı bileşenleri bulmada, 621 problem
 - güçlü bağlantılı bileşeni bulmada, 615–621, 623
 - topolojik sıralamada, 612–615
- derinliğine ağaç, 603
- derinliğine aramada bitiş zamanı, 605
 - ve güçlü bağlantılı bileşenlerde, 618
- derinliğine aramanın parantez yapısı, 606
- derinliğine ormanı, 603
- derinlik-belirleme problemi, 583 problem
- derinlik
 - bir devrenin, 919
 - bir yığının, 188 problem
 - hızlı sıralamalı özyineleme ağacının, 178 alıştırma
 - köklü ağacın bir düğümünün, 1177
 - ortalama, rassal oluşturulmuş ikili arama ağacının bir düğümünün, 304 problem
- desen eşleme, *bakınız* katar eşleme,

- desen, katar eşlemede, 985
 örtüşmeyen, 1002 alıştırma
 det, *bakınız* determinat
 determinant 1224–1225
 ve matris çarpımı, 832 alıştırma
 DETERMINISTIC-SEARCH, 143 problem
 devam eden ayırıt, 778
 devirli döndürme 1012 alıştırma
 devirli grup, 955
 devre
 -nin derinliği, 919
 boole birleşimsel, 1071
 hızlı Fourier dönüşümü için, 919–920
 devresel gerçekleştirilebilirlik, 1070–1077
 DFS-VISIT, 604
 DFS, 604
 DFT (ayrık Fourier dönüşümü), 9, 909
 dış düğüm, 1176
 dış ürün, 1222
 dış yol uzunluğu, 1180 alıştırma
 dış-derece, 1169
 dışbükey çokgen, 1020 alıştırma
 dışbükey fonksiyon, 1199
 dışbükey katmanlar, 1044 problem
 dışbükey küme, 714 alıştırma
 dışbükey örtü, 8, 1029–1039, 1046 problem
 Dijkstra algoritması, 658–664, 682
 enine aramaya benzer, 662, 663 alıştırma
 Fibonacci bellek yığınları ile uygulanan, 662
 Johnson algoritmasında, 702
 min-bellek yığınları ile uygulanan 662
 Prim algortimasına benzer, 634, 662
 tam sayı ağırlıklı ayırıt ile, 664 alıştırma
 tüm ikililerin en kısa yolları, 684, 704
 DIJKSTRA, 658
 dikdörtgen, 354 alıştırma
 dil, 1057
 -in doğrulanması, 1063
 -in NP-tamlığının ispatı, 1078–1079
 -in tamlığı, 1077 alıştırma
 dilim zaman
 bir çırpı tablosunun, 256
 bir doğrudan-erişimli tablonun, 254
 dinamik ağaç, 482
 dinamik çizge, 562 dipnot
 -nin geçişli kapanışı, 705 problem, 707
 için minimum-kapsayan-ağaç algortiması,
 637 alıştırma
 için tüm ikililerin en kısa yol algolaritmaları,
 707
 için veri yapıları, 483
 dinamik çoklu iş parçalama, 339–345
 dinamik çoklu iş parçalama, 773
 dinamik çoklu-iş parçacıklı algoritma, *bakınız*
 çoklu-iş parçacıklı algoritma
 dinamik küme, 229–231
 ayrıca bakınız veri yapısı
 dinamik programlama yöntemi, 359–413
 -nde optimal çözümün yeniden oluşturulması,
 387
 -nin elemanı, 378–390
 0-1 sırt çantası problemi için, 427 alıştırma
 ağgözlü algoritmalarıyla karşılaştırılan 381,
 390 alıştırma, 418, 423–427
 aşağıdan-yukarıya, 365
 bir ağırlıklı yönlü çevrimsiz çizgede en uzun
 yol için, 404 problem
 bir katarın parçalanması için, 410 problem
 böl-ve-yönet ile ilişkili, 359
 çift düze öklid gezgin-satıcı için,
 çubuk kırma için, 360–370
 düzgün baskı için, 405 problem
 en uzun ortak altdizi için, 390–397
 en uzun palindrom altdizi için, 405 problem
 envanter planlama için, 411 problem
 etkinlik seçimi için, 421 alıştırma
 Floyd-Warshall algoritması için, 693–697
 geçişli kapanış için, 697–699
 içinde çakışan altproblemler, 384–386
 içinde optimal altyapı, 379–384
 matris-zinciri çarpımı için, 370–378
 mesafe düzenleme için, 406 problem
 optimal ikili arama ağaçları için, 397–404
 ön belleğe alma ile yukarıdan-aşağıya 365
 problem, 405 problem
 serbest (signing free agents) için, 411 prob-
 lem
 tabaka oyma (seam carving) için, 409 prob-
 lem
 tüm-ikililerin en kısa yolları için 686–697
 ve ön belleğe alma, 387–389
 Viterbi algoritması için, 408 problem
 dinamik tablo, 463–471
 -nun yükleme faktörü, 463
 hesaplama yöntemi tarafından analiz edilen,
 465–466
 potansiyel yöntemi tarafından analiz edilen,
 466–471
 toparlama analizi tarafından analiz edilen,
 465

- DIRECT-ADDRESS-DELETE, 254
 DIRECT-ADDRESS-INSERT, 254
 DIRECT-ADDRESS-SEARCH, 254
 DIRECTION, 1018
 DISCHARGE, 751
 disk sürücüsü, 485–487
 ayrıca bakınız ikincil depolama
 disk üzerindeki sayfa, 486, 499 alıştırma, 502
 problem
 DISK-READ, 487
 DISK-WRITE, 487
 disk, 1028 alıştırma
 dizi ($()$),
 -de ters, 41 problem, 122 alıştırma, 345
 alıştırma
 çiftdüze, 682 problem
 denemek, 270
 sonlu, 1166
 sonsuz, 1166
 dizi, 21
 bir parametre olarak geçen, 21
 Monge, 110 problem
 dizilim, 777
 bağımsız, 789
 başlangıç, 779
 bitiş, 779
 paralelde mantıksallık, 778
 serilerde mantıksallık, 778
 dizin fonksiyonu, 537, 546
 dizisel tutarlılık 779, 812
 DNA, 6–7, 390–391, 406 problem
 DNF (veya-ı normal biçim), 1083
 doğal kübik spline, 840 problem
 doğal sayılar ($\mathbb{N}$) 1158
 olarak yorumlanmış anahtarlar, 263
 doğru parçası, 1015
 -nın dönüşünün belirlenmesi, 1017
 herhangi bir kesişim olup olmadığını belirle-
 yen, 1021–1029,
 iki kesişim olup olmadığını belirleyen,
 1017–1019
 karşılaştırılabilir, 1022
 doğrudan adresleme, 254–255, 532–536
 doğrudan-adres tablosu 254–255
 doğrulama algoritması, 1063
 doğrulaması, 1061–1066
 kapsayan ağaçların, 642
 doğruluk ataması, 1072, 1079
 doğruluk tablosu, 1070
 doğrusal arama, 22 alıştırma
 doğrusal bağımlılık, 1223
 doğrusal bağımsızlık, 1223
 doğrusal denklemler,
 -in tridiagonal sistemlerini çözme, 840 prob-
 lem
 modüler çözümü, 946–950
 sistemlerinin çözümü, 813–827
 doğrusal denklemlerin bir sistemi, 806 problem,
 813–827, 840 problem
 doğrusal denklemlerin aşırı belirsiz sistemi, 814
 doğrusal eşitlik, 845
 doğrusal eşitsizlik, 846
 doğrusal fonksiyon, 26, 845
 doğrusal hızlandırma, 780
 doğrusal inceleme, 272
 doğrusal kısıt, 846
 doğrusal permütasyon, 1229 problem
 doğrusal programlama rahatlığı, 1125
 doğrusal programlama, 7, 843–897
 -da bir başlangıç çözümü bulma, 886–891
 -daki dualite, 879–886
 -nın temel teoremi, 892
 -nın uygulamaları, 849
 ayrıca bakınız tam sayılı doğrusal programla-
 ma, 0-1 tam sayılı programlama,
 için algoritmalar, 850
 için elipsoid algoritması, 850, 897
 için gevşek biçim, 854–857
 için iç-nokta yöntemleri, 850, 897
 için Karmarkar algoritması, 897
 için simpleks algoritma, 864–879, 896
 için standart biçim, 850–854
 ve çokluürün akışı, 862–863
 ve maksimum akış, 860–861
 ve minimum-maliyet akışı, 861–862
 ve minimum-maliyet çokluürün akışı, 864
 alıştırma
 ve minimum-maliyet sürkülasyonu, 896
 problem
 ve tek-ikili en kısa yol, 859–860
 ve tek-kaynaklı en kısa yollar, 664–670, 863
 alıştırma
 doğrusal programlamanın temel teoremi, 892
 doğrusal sıralama, 1165
 doğrusal-eşitsizlik elverişliliği problemi, 894
 problem
 doğum günü paradoksu, 130–133, 142 alıştırma
 doğuran ayrıt, 778
down to, sözdekodda, 21
 doymuş ayrıt, 739

- doyuran itme, 739, 745
- doyurmayan itme, 739, 745
- döndürme ayrıtı, 779
- döndürme faktörü, 912
- döndürme komutu, 23
- döndürme
 - bir kırmızı-siyah ağaçta, 312–314
 - devirli, 1012 alıştırma
- dönen tarama, 1030–1038
- döngü değişmezi, 18–19
 - nin bakımı, 19
 - nin başlatılması, 19
 - nin kökeni, 42
- araya sokma sıralaması için, 18
- bellek yığını inşası için, 157
- bellek yığını sıralaması için, 160 alıştırma
- bir aralık ağacı araması için, 352
- bir bellek yığnında bir anahtar artırımı için, 166 alıştırma
- bir diziyi rassal değiştirmek için, 127, 128 alıştırma
- bir sıra istatitiği ağacında bir elemanın rankını belirlemek için, 342
- birleştirmesi için, 32
- Dijkstra algoritması için, 660
- enine arama için, 595
- Fibonacci bellek yığını kök listesini güçlendirmek için, 517
- genel itme-yeniden etiketleme algoritması için, 743
- genel minimum-kapsayan-ağaç yöntemi için, 625
- Graham taraması için, 1034
- Horner kuralı için, 41 problem
- katar-eşleme otomatası için, 998, 1000
- kırmızı-siyah ağacı araya sokma için, 318
- modüler üs alma için, 957
- öne-yeniden etiketleme algoritması için, 755
- parçalama için, 171
- Prim algoritması için, 636
- Rabin-Karp algoritması için, 993
- simpleks algoritma için, 872
- ve for döngüleri, 19 dipnot
- ve sonlandırma, 19
- döngü, simpleks algoritmasının, 875
- dual doğrusal program, 879
- dualite, 879–886, 895 problem
 - zayıf, 880–881, 886 alıştırma
- durağan çizge, 562 dipnot
- durağan iş parçacıklı, 773
- durma problemi, 1048
- durum, 1190
- düğüm, 1176
 - ayrıca bakınız* tepe noktası,
- düğüm noktası, spline nın, 840 problem
- düşman, 190
- düşük fonksiyon, 537, 546
- düzenli ağaç adımı, 287, 293 alıştırma, 342
- düzenlilik koşulu, 95
- düzey fonksiyonu, 576
- düzey
 - bir ağacın, 1177
 - bir fonksiyonun, 573
- düzgün çırpılama, 271
- düzgün olasılık dağılımı, 1191–1192
- düzgün rastgele pemütasyon, 116, 125
- ϵ -evrensel çırpı fonksiyonu, 269 alıştırma
- ϵ -yoğun çizge, 706 problem
- e, 55
- E[] (beklenen değer), 1197
- ebeveyn
 - bir çoklu-iş parçacıklı hesaplamada, 776
 - enine ağaçta, 594
 - köklü ağaçta, 1176
- ebob, *bakınız* en büyük ortak bölen,
- Edmonds-Karp algoritması 727–730
- eğri uydurma, 835–839
- eklem noktası, 621 problem
- ekok(en küçük ortak kat), 939 alıştırma
- eksen, 485
- ele almak 163, 507
- elipsoid algoritma 850, 897
- eliptik-eğri çarpanlara ayırma yöntemi 984
- else**, kaba kodda 20
- elseif**, kaba kodda, 20 dipnot
- elverişli bölge, 847
- elverişli çözüm, 665, 846, 851
- elverişli doğrusal program 851
- elverişlilik problemi 665, 894 problem
- elverişsiz çözüm, 851
- elverişsiz doğrusal program, 851
- en büyük ortak bölen (ebob), 929–930, 933 alıştırma
 - için ikili en büyük ortak bölen algoritması, 981 problem
 - için Öklid algoritması, 933–939, 981 problem, 983
 - için özyineleme teoremi, 934

- iki argümandan daha fazlasıyla, 939 alıştırma
- en kısa yollar, 7, 643–707
 - in ağacı, 647–648, 673–676
 - in en uygun altyapısı, 644–645, 687, 693–694
 - in öncül altçizge özelliği, 650, 676
 - in tahmini, 648
 - in üçgen eşitsizliği, 650, 671
 - in üst-sınır özelliği, 650, 671–672
 - in yakınsaklık özelliği, 650, 672–673
 - in yol olmama özelliği, 650, 672
 - in yol-gevşetme özelliği, 650, 673
 - ε-yoğun çizgelerde, 706 problem
 - bir ağırlıklı çizgede, 643
 - bir ağırlıksız çizgede, 381, 597
 - bir doğrusal program olarak 859–860
 - bir yönlü çevrimsiz çizgede, 655–658
 - çift düze yollar ile, 682 problem
 - için Bellman-Ford algoritması, 651–655
 - için Dijkstra algoritması, 658–664
 - için Floyd-Warshall algoritması 693–697,
 - için Gabow ölçekleme algoritması 679 problem
 - için Johnson algoritması 700–706
 - matris çarpımı ile, 686–693, 706–707
 - negatif-ağırlıklı ayrıtlar ile 645–646
 - problem çeşitleri, 644
 - tek-hedefli, 644
 - tek-ikili, 381, 644
 - tek-kaynaklı, 643–683
 - tekrarlı kareleme ile, 689–691
 - tüm-ikililer, 644, 684–707
 - ve en uzun yollar, 1048
 - ve enine arama, 597–600, 644
 - ve fark kısıtları, 664–670
 - ve gevşetme, 648–650
 - ve negatif-ağırlıklı çevrimler, 645, 653–654, 692 alıştırma, 700 alıştırma
- en kötü-durum çalışma süresi, 27, 49
- en küçük ortak ata, 584 problem
- en küçük ortak kat, 939 alıştırma
- en küçük-kareler yaklaşımı, 835–839
- en uzak-çift problemi 1030
- en uzak-gelecek stratejisi, 449 problem
- en uzun basit çevrim, 1101 alıştırma
- en uzun basit yol, 1048
 - bir ağırlıksız çizgede, 382
 - bir ağırlıklı yönlü çevrimsiz çizgede, 404 problem
- en uzun ortak altdizi, 7, 390–397, 413
- en uzun palindrom altdizi, 405 problem
- en yakın çifti, bulma, 1039–1044, 1047
- en yakın-nokta sezgiseli, 1117 alıştırma
- en-iyi çalışma süresi, 29 alıştırma, 49
- enine ağaç, 594, 600
- enine arama, 594–602, 623
 - Dijkstra algoritmasına benzerlik, 662, 663 alıştırma
 - maksimum akış içinde, 727–730, 766
 - ve en kısa yollar, 597–600, 644
- ENQUEUE, 235
- entropi fonksiyonu, 1187
- envanter planlama, 411 problem
- erken görev, 444
- erken-ilk biçim, 444
- error**, kaba kodda, 22
- eş dikey, 1024
- eş doğrusallık, 1016
- eşçarpan, 1224
- eşitlik kısıtı, 670 alıştırma, 852
 - nın ihlali, 865
 - sıkı 865
 - ve eşitsizlik kısıtları, 853
- eşitlik
 - doğrusal, 845
 - fonksiyonların, 1166
 - kümelerin, 1158
- eşitsizliğin kısıtları, 852
 - ve eşitliğin kısıtları, 853
- eşitsizlik, doğrusal, 846
- eşleme
 - ağırlıklı iki parçalı, 530
 - iki parçalı, 732, 763 problem
 - katarların, 985–1013
 - maksimal, 1110, 1135 problem
 - maksimum, 1135 problem
 - tam, 735 alıştırma
 - ve maksimum akış, 732–736, 747 alıştırma
- eşlenik transpozese, 832 alıştırma
- eşlenmemiş tepe, 732
- eşlenmiş tepe, 732
- eşzamanlı anahtar sözcükler, 774, 776, 785
- eşzamanlı platform, 773
- etki alanı sıklığı, 898
- etkinlik-seçim problemi, 415–422, 450
- EUCLID, 935
- Euler phi fonksiyonu, 943
- Euler sabiti, 943
- Euler teoremi, 954, 975 alıştırma
- Euler turu, 623 problem, 1048

- ve hamilton çevrimleri, 1048
- evreni, 1160
 - van Emde Boas ağaçlarındaki anahtarların, 532
- evrenin boyutu, 532
- evrensel çıkış düğümü, 593 alıştırma
- evrensel çırpılama, 265–268
- evrişim ($\otimes$), 901
- evrişim teoremi, 913
- EXACT-SUBSET-SUM, 1129
- EXTEND-SHORTEST-PATHS, 688
- EXTENDED-BOTTOM-UP-CUT-ROD, 369
- EXTENDED-EUCLID, 937
- EXTRACT-MAX, 162–163
- EXTRACT-MIN, 162, 505
- faktöriyel fonksiyonu (!), 57–58
- faktörü, 928
 - döndürme, 912
- fark denklemi, *bakınız* özyineleme
- fark kısıtları, 664–670
- fark kısıtlarının sistemi, 664–670
- Farkas lemması, 895 problem
- FASTER-ALL-PAIRS-SHORTEST-PATHS, 691, 692
 - alıştırma
- faz, yeniden öne-yeniden etiketleme algoritması, 758
- Fermat teoremi, 954
- FFT, *bakınız* Hızlı Fourier dönüşümü
- FFTW, 924
- FIB-HEAP-CHANGE-KEY, 529 problem
- FIB-HEAP-DECREASE-KEY, 519
- FIB-HEAP-DELETE, 522
- FIB-HEAP-EXTRACT-MIN, 513
- FIB-HEAP-INSERT, 510
- FIB-HEAP-LINK, 516
- FIB-HEAP-PRUNE, 529 problem
- FIB-HEAP-UNION, 512
- FIB, 775
- Fibonacci bellek yığını, 505–530
 - nda işlemlerin çalışma süreleri, 506 şekil
 - ndan minimum anahtar ayıklanması, 512–518
 - ndan silme, 522, 526 problem
 - nın en küçük anahtarı 511
 - nın maksimum derecesi, 509, 523–526
 - anahtar azaltma, 519–522
 - anahtar değiştirme, 529 problem
 - arayaya sokma, 510–511
 - birleştirici, 511–512
 - budama, 529 problem
 - Dijkstra algortimasında, 662
 - için potansiyel fonksiyon, 509
 - ikili bellek yığınlarıyla karşılaştırılan, 506–507
 - Johnson Algoritmasında, 704
 - Prim algoritmasında, 636
 - yaratma, 510
- Fibonacci sayıları, 59–60, 108 problem, 523
 - nın hesaplanması, 774–780, 981 problem
- Fibonacci-bellek yığını kök listesini sağlamlaştırma, 513–517
- FIFO (ilk giren, ilk çıkan), 232
 - ayrıca bakınız* kuyruk,
- FIND-DEPTH, 583 problem
- FIND-MAX-CROSSING-SUBARRAY, 71
- FIND-MAXIMUM-SUBARRAY, 72
- FIND-SET, 562
 - in ayırık-küme-orman uygulaması, 571, 585
 - in bağlı liste gösterimi, 564
- FINITE-AUTOMATON-MATCHER, 999
- Floyd-Warshall algoritması, 693–697, 699–700
 - alıştırma, 706
 - çoklu-iş parçacıklı, 797 alıştırma
- FLOYD-WARSHALL, 695
- FLOYD-WARSHALL', 699 alıştırma
- fonksiyon, 1166–1168
 - Ackermann, 585
 - amaç, 664, 847, 851
 - baz, 835
 - çırpı, *bakınız* çırpı fonksiyonu,
 - dışbükey, 1199
 - doğrusal, 26, 845
 - geçiş, 995, 1001–1002, 1012 alıştırma
 - ikinci dereceden, 27
 - indirgeme, 1067
 - örnek, 1003–1004
 - potansiyel, 459
 - son-durum, 996
 - sonек, 996
- fonksiyonel iterasyon, 58
- for**, kaba kodda, 20–21
 - ve döngü değişmezleri, 19 dipnot
- Ford-Fulkerson yöntemi, 714–731, 765
- FORD-FULKERSON-METHOD, 715
- FORD-FULKERSON, 724
- formül-gerçekleştirilebilirliği, 1079–1081, 1105
- Fourier dönüşümü, *bakınız* ayırık Fourier dönüşümü, hızlı Fourier dönüşümü

- FREE-OBJECT, 244
 füzyon ağacı, 212, 483
- Gauss yok etme, 819, 842
 geçerli kaydırma, 985
 geçersiz değişim, 985
 geçiş fonksiyonu, 995, 1001–1002, 1012 alıştırma
 ma
 geçişli bağıntı, 1163
 geçişli kapanış, 697–699
 ve boole matris çarpımı, 832 alıştırma
 dinamik çizgelerin, 705 problem, 707
 genel değişken, 21
 genel itme-yeniden etiketleme algoritması, 740–748
 genel sayı-alan eleği, 984
 GENERIC-MST, 626
 GENERIC-PUSH-RELABEL, 741
 genişletilmiş yol, 719–720, 763 problem
 geometri, hesaplanabilir 1014–1047
 geometrik dağılım, 1202–1203
 ve toplar ve kutular, 134
 geometrik seriler, 1147
 gerçekleşen atama, 1072, 1079
 gerçekleştirilebilir formül, 1049, 1079
 gerçekleştirilebilirlik, 1072, 1079–1081, 1105, 1123–1124, 1127 alıştırma, 1139
 gerçel sayıların üçe ayırma özelliği, 52
 geri yerine koyma, 817
 gevşek biçim, 846, 854–857
 -in tekliği, 876
 gevşek değişken, 855
 gevşek, 855
 gevşeklik,
 paralel, 781
 tümleyici, 894 problem
 gevşetme
 bir ayrıtın, 648–650
 doğrusal programlama, 1125
 gevşetilmiş bellek yığını, 530
 gevşetilmiş kırmızı-siyah ağaç, 311 alıştırma
 gezgin-satıcı problemi,
 -nin NP tamlığı, 1096–1097
 çift düze öklidyen, 405 problem
 darboğaz, 1117 alıştırma
 için yaklaşım algoritması, 1111–1117, 1139
 üçgen eşitsizliği ile, 1112–1115
 üçgen eşitsizliği olmadan, 1115–1116
 GF(2), 1227 problem
- girdi alfabesi, 995
 girdi,
 -nin dağılımı, 116, 122
 -nin uzunluğu, 25
 bir algoritmaya, 5
 bir birleşimsel devreye, 1071
 bir mantık kapısına, 1070
 giren değişken, 867
 gizli anahtar, 959, 962
 Goldberg algortması, *bakınız* itme-yeniden etiketleme algoritması
 görev çizelgeleme, 443
 görev çizelgeleme, 443–446, 448 problem, 450
 görüntü kümesi, 1166
 görüntü kümesi, 1167
 bir matrisin, 1228 problem
 görüntü sıkıştırma, 409 problem, 413
 görüntü, 1167
 GRAFT, 583 problem
 Graham taraması, 1030–1036, 1047
 GRAHAM-SCAN, 1031
 GRAPH-ISOMORPHISM, 1065 alıştırma
 greedoid, 450
 GREEDY-ACTIVITY-SELECTOR, 421
 GREEDY-SET-COVER, 1119
 GREEDY, 440
 gri köşe, 594, 603
 grup, 939–946
 devirli, 955
 operatörü ($\oplus$), 939
 güçlü bağlantılı bileşen, 1170
 -de ayrıştırma, 615–621, 623
 güçlü bağlantılı çizge, 1170
 güvenli ayrıt, 626
- hafif ayrıt, 626
 hakim bağıntı, 1045 problem
 Hall teoremi, 735 alıştırma
 HAM-CYCLE, 1062
 HAM-PATH, 1066 alıştırma
 hamilton çevrimi, 1049, 1061, 1091–1096, 1105
 hamilton çizgesi, 1061
 hamilton olmayan çizge, 1061
 hamilton yolu, 1066 alıştırma, 1101 alıştırma
 harmonik sayı, 1147, 1153–1154
 harmonik seriler, 1147, 1153–1154
 HASH-DELETE, 277 alıştırma
 HASH-INSERT, 270, 277 alıştırma
 HASH-SEARCH, 271, 277 alıştırma
 HEAP-DECREASE-KEY, 165 alıştırma

- HEAP-DELETE, 166 alıştırma
- HEAP-EXTRACT-MAX, 163
- HEAP-EXTRACT-MIN, 165 alıştırma
- HEAP-INCREASE-KEY, 164
- HEAP-MAXIMUM, 163
- HEAP-MINIMUM, 165 alıştırma
- HEAPSORT, 160
- hedef tepe, 644
- hedef, 1097
- hediye sarma, 1037, 1047
- Hermityen matris, 832 alıştırma
- hesaplama yöntemi, 456-459
 - dinamik tablolar için, 465-466
 - ikili sayaçlar için, 458
 - yığın işlemleri için, 457-458, 458 alıştırma
- hesaplama, çoklu-iş parçacıklı, 777
- hesaplanabilir derinlik, 812
- hesaplanabilir geometri, 1014-1047
- hesaplanabilir problem, 5-6
- hesaplanabilir yönlü çevrimsiz çizge, 777
- hızlanması, 780
 - bir rassal çoklu iş parçacıklı algoritmanın, 811 problem
- hızlı Fourier dönüşümü(FFT), 898-925
 - nün iteratif uygulaması, 915-918
 - nün özyinelemeli uygulaması, 909-912
- çok-boyutlu, 921 problem
- için çoklu-iş parçacıklı algoritma, 804 alıştırma
- ma
 - için devre, 919-920
 - modüler aritmetik kullanarak, 923 problem
- hızlı sıralama için ‘öldürücü düşman’, 190
- hızlı sıralama, 170-190
 - da araya sokma sıralamasının kullanım, 185 alıştırma
 - nın analizi, 174-185
 - nın en kötü-durum analizi, 180-181
 - nın iyi bir en-kötü-durum uygulaması, 223 alıştırma
 - nın kuyruk-özyinelemeli versiyonu, 188 problem
 - nın ortalama-durum analizi, 181-184
 - nın rassal versiyonu, 179-180, 187 problem
 - nın yığın derinliği, 188 problem
 - 3-medyan yöntemi ile, 188 problem
 - araya sokma sıralamasıyla karşılaştırılan, 178 alıştırma
 - eşit eleman değerleri ile, 186 problem
 - için “öldürücü düşman”, 190
 - için çoklu iş parçacıklı algoritma, 811 problem
 - lem
 - taban sıralaması ile kıyaslandığında, 199
 - hilesiz madeni para, 1191
 - hiper ayrıt, 1172
 - hiperçizge, 1172
 - ve iki parçalı çizge, 1173 alıştırma
 - HIRE-ASSISTANT, 115
 - HOARE-PARTITION, 185 problem
 - Hopcroft-Karp iki parçalı eşleştirme algoritması, 763 problem
 - HOPCROFT-KARP, 764 problem
 - Horner kuralı, 41 problem, 900
 - Rabin-Karp algoritmasında, 990
 - Huffman kodu, 428-437, 450
 - HUFFMAN, 431
 - ıraksak seriler, 1146
 - ışın, 1021 alıştırma
 - ızgara, 760 problem
 - 2-3 ağacı, 337, 504
 - 2-3-4 ağaçlar, 489
 - ayırma, 503 problem
 - birleşme, 503 problem
 - 2-3-4 bellek yığını, 529 problem
 - 2-CNF gerçekleştirilebilirliği, 1086 alıştırma
 - and 3-CNF gerçekleştirilebilirliği, 1049
 - 2-CNF-SAT, 1086 alıştırma
 - iç çarpım, 1222
 - iç düğüm, 1176
 - iç içe geçen seriler, 1148
 - iç içe geçen toplam, 1148
 - iç içe konan kutular, 678 problem
 - iç içe paralellik, 776, 805 problem
 - iç yol uzunluğu, 1180 alıştırma
 - iç-nokta yöntemi, 850, 897
 - içerme ve dışlama, 1163 alıştırma
 - içerme ve dışlama, 1163 alıştırma
 - içerme ve dışlamanın ilkesi, 1163 alıştırma
 - içermek, bir yolda, 1170
 - if, kaba kodda, 20
 - ihlal, bir eşitlik kısıtının, 865
 - iki bağlantılı bileşen, 621 problem
 - iki parçalı çizge, 1172
 - nin akış ağına ilişkin, 732
 - d-düzgün, 736 alıştırma
 - ve hiperçizgeler, 1173 alıştırma
 - iki parçalı eşleme için uygun akış ağı, 732
 - iki parçalı eşleme, 530, 732-736, 747 alıştırma, 766
 - için Hopcroft-Karp algoritması, 763 problem

- iki-geçişli yöntem, 571
- ikili ağaç, 1177
 - nın temsili, 246
 - ayrıca bakınız* ikili arama ağacı
 - bir bit vektörü üzerine üst üste, 533–534
 - farklı olanların sayısı, 306 problem
 - tam, 1178
- ikili arama ağacı, 286–307
 - daki ardıl, 291–292
 - nda araya sokma, 294–295
 - ndaki öncül, 291–292
 - ndan silme 295–298, 299 alıştırma
 - nın maksimum anahtarı, 291
 - nın minimum anahtarı, 291
 - nın sağ-dönüştürüleni, 314 alıştırma
 - AA-ağaçları, 338
 - ağırlık-dengeli ağaçlar, 338
 - arama, 289–291
 - AVL ağaçları, 333 problem, 337
 - ayrıca bakınız* kırmızı-siyah ağaç
 - eşit anahtarlar ile, 303 problem
 - k*-komşu ağaçları, 338
 - kendini dengeleyen ağaçlar, 338
 - optimal, 397–404, 413
 - rassal oluşturulan, 299–303, 304 problem
 - sıralama için, 299 alıştırma
 - sorgulama, 289–294
 - ve treaps, 333 problem
 - yayvan ağaçlar, 338
- ikili arama, 39 alıştırma
 - araya sokma sıralamasında, 39 alıştırma
 - B-ağaçları aramada, 499 alıştırma
 - çoklu-iş parçacıklı birleştirmede, 799–800
 - hızlı araya sokma ile, 473 problem
- ikili ayrık kümeler, 1161
- ikili bağımsızlık, 1193
- ikili bağıntı, 1163
- ikili bağlı liste, 236
 - ayrıca bakınız* bağlı liste,
- ikili bellek yığını, *bakınız* bellek yığını
- ikili en büyük ortak bölen algoritması, 981
 - problem
- ikili entropi fonksiyonu, 1187
- ikili karakter kodu, 428
- ikili sayaç
 - bit düzeyinde ters çevrilmiş, 472 problem
 - hesaplama yöntemi tarafından analiz edilen, 458
 - potansiyel yöntemi tarafından analizi, 461–462
 - toparlama analizi tarafından analiz edilen, 454–455
- ikili-arama-ağacı özelliği, 287
- ikili,sıralı, 1161
 - karşılık minimum-bellek yığını özelliği, 289
 - alıştırma
 - treaps da, 333 problem
- ikilik tabandan onluk tabana dönüştürmek, 933
 - alıştırma
- ikinci dereceden deneme, 272, 283 problem
- ikinci dereceden fonksiyon, 27
- ikinci dereceden kalan, 982 problem
- ikinci-en iyi minimum kapsayan ağaç, 638
 - problem
- ikincil çırpı tablosu, 278
- ikincil depolama,
 - için arama ağacı, 484–504
 - üzerinde yığınlar, 502 problem
- ikincil kümeleme, 272
- ileri ayır, 609
- ileri yerine koyma, 816–817
- ilk-giren, ilk-çıkan 232
 - ayrıca bakınız* kuyruk
- ilk-uygun sezgiseli, 1134 problem
- imza, 960
- inatçılık, 1048
- INCREASE-KEY, 162
- INCREMENT, 454
- indirgeme algoritması, 1052, 1067
- indirgeme fonksiyonu, 1067
- indirgenabilirlik, 1067–1068
- indirgenme, bir dizinin, 807 problem
- indirgenmiş altçizge, 1171
- indirgenmiş-uzaylı van Emde Boas ağacı, 557
 - problem
- INITIALIZE-PREFLOW, 740
- INITIALIZE-SIMPLEX, 871, 887
- INITIALIZE-SINGLE-SOURCE, 648
- INORDER-TREE-WALK, 288
- İnsan Genom Projesi, 6
- INSERT, 162, 230, 463 alıştırma, 505
- INSERTION-SORT, 18, 26, 208 problem
- integral, toplamlara yaklaşmak için, 1154–1156
- INTERVAL-DELETE, 349
- INTERVAL-INSERT, 349
- INTERVAL-SEARCH-EXACTLY, 354 alıştırma
- INTERVAL-SEARCH, 349, 351
- iptal lemması, 907
- İş Parçacığı İnşa Blokları, 774
- iş parçacığı, 773

- iş yükü çalan çizelgeleme algoritması, 812
- iş yükü yasası, 780
- işaretçi, 21
 - nin dizi uygulaması, 241–246
 - izleyen, 295
- işaretili düğüm, 508, 519–520
- işlemi düzenleme, 230
- iterasyon fonksiyonu, 577
- iteratif fonksiyon, 63 problem
- iteratif logaritma fonksiyonu, 58–59
- ITERATIVE-FFT, 917
- ITERATIVE-TREE-SEARCH, 291
- itme -yeniden etiketleme algoritması, 736–760, 765
 - ndaki temel işlemler, 738–740
 - nın taşan tepe noktasını boşaltarak, 760
 - alıştırma
 - bir maksimum iki-parçalı eşleme bulmak için, 747 alıştırma
 - için aralık segizeli, 760 alıştırma, 766
 - öne-yeniden etiketleme algoritması, 748–760
 - soysal algoritma, 740–748
 - taşan tepe noktalarının bir kuyruğu ile, 759
 - alıştırma
- itme işlemi(itme yeniden etiketleme algoritmalarında), 738–739
 - doyurmayan, 739, 745
 - doyuran, 739, 745
- iz, 486
- izleyen işaretçi, 295
- izole tepe noktası, 1169
- izomorfik çizgeler, 1171

- Jarvis marşı, 1037–1038, 1047
- Jensen eşitsizliği, 1199
- Johnson algoritması, 700–706
- Johnson, 704
- Josephus permütasyonu, 355 problem

- k -altkatar, 1184
- k -altküme, 1161
- k -birleşim 1185
- k -CNF, 1049
- k -evrensel çırpılama, 284 problem
- k -ıncı kuvvet, 933 alıştırma
- k -katar, 1184
- k -komşu ağaç, 338
- k -lı ağaç, 1179
- k -permutasyon, 126, 1184
- k -renklendirme, 1103 problem, 1180 problem
- k -sıralanmış, 207 problem
- k -ve-li normal biçim, 1049
- kaba kod, 16, 20–22
- kaba kodda blok yapısı, 20
- kaba kodda girinti, 20
- kablo, 1071
- kabul durumu, 995
- kabul edilebilir tam sayı çiftleri, 972
- kabul edilemez ayrıt, 749
- kabul
 - bir algoritma tarafından, 1058
 - sonlu bir otomata tarafından, 996
- kaçış problemi, 760 problem
- kalan komutu, 23
- kalan, 54, 928
- kalıcı veri yapısı, 331 problem, 482
- kapalı aralık, 348
- kapalı yarı halka, 707
- kapanış
 - bir dilin, 1058
 - geçişli, *bakınız* geçişli kapanış
 - grup özelliği, 939
 - operatör(*), 1058
- kapasite kısıtı, 709–710
- kapasite,
 - artık, 716, 719
 - bir ayrıtın, 709
 - bir kesimin 721
 - bir tepenin, 714 alıştırma
- kapı, 1070
- kapsayan ağaç, 439, 624
 - in doğrulanması, 642
 - ayrıca bakınız* minimum kapsayan ağaç
- darboğaz, 640 problem
- maksimum, 1137 problem
- karakter kodu, 428
- karar ağacı, 192–193
- karar problemi, 1051, 1054
 - ve optimizasyon problemleri, 1051
- kararlılık
 - sayısal, 813, 815, 842
 - sıralama algoritmaları için, 196, 200 alıştırma
- kardeş, 1176
- kare matris, 1218
- karekök, bir asal modülo ya göre, 982 problem
- karelema,tekrarlı,
 - bir sayının kuvvetini almak için, 956
 - tüm-ikililerin en kısa yolları için, 689–691

- Karmarkar algoritması, 897
 karmaşık sayılar
 -in çarpımı, 83 alıştıırma
 -in ters matrisleri, 832 alıştıırma
 karmaşıklık ölçüsü, 1059
 karmaşıklık sınıfı, 1059
 co-NP, 1064
 NP, 1049, 1064
 NPC, 1050, 1069
 P, 1049, 1055
 Karp'ın minimum ortalama-ağırlıklı çevrim
 algoritması, 680 problem
 karşılaştır-değiştir işlemi, 208 problem
 karşılaştırılabilir doğru parçaları, 1022
 karşılaştırma sıralaması, 191
 rassal, 205 problem
 ve ikili arama ağaçları, 289 alıştıırma
 ve seçim, 222
 Kartezyen çarpım ('), 1162
 Kartezyen toplam, 906 alıştıırma
 Katalan sayıları, 306 problem, 372
 katar eşleme otomatısı, 996–1002, 1002 alıştıırma
 katar eşleme, 985–1013
 boşluk karakterleri ile, 989 alıştıırma, 1002
 alıştıırma
 için Knuth-Morris-Pratt algoritması,
 1002–1013
 için Rabin-Karp algoritması, 990–995, 1013
 için sade algoritma, 988–990
 sonlu otomata ile, 995–1002
 tekrarlama etkenlerine dayanan, 1012 prob-
 lem
 katar, 985, 1184
 katmanlar,
 dış bükey, 1044 problem
 maksimal, 1045 problem
 katsayı temsili, 900
 ve hızlı çarpım, 903–905
 katsayı
 binom, 1186
 bir polinomun, 55, 898
 gevşek biçimde, 856
 kayan nokta veri tipi, 23
 kaydırma komutu, 24
 kaydırma, katar eşleşmede, 985
 kayıp çocuk, 1178
 kayıp, 449 problem
 kayıt, 147
 kayıtsız karşılaştırma-değişimi algoritması, 208
 problem
 kaynak tepesi, 594, 644, 709, 712
 kelebek işlemi, 915
 kendi kendine döngü, 1168
 kendi kendini düzenleyen liste, 476 problem, 478
 kendini dengeleyen ağaç, 338
 kesim
 -in ağırlığı, 1127 alıştıırma
 -in kapasitesi, 721
 basamaklı, 520
 bir akış ağının, 720–724
 bir yönsüz çizgenin, 626
 karşısında ağ akışı, 720
 minimum, 721, 731 alıştıırma
 kesin artan, 53
 kesin azalan, 53
 kesin olay, 1190
 kesirli sırt çantası problemi, 426, 428 alıştıırma
 kesme, Fibonacci bellek yığnında, 519
 keşfedilmiş tepe, 605
keywords, kaba kodda, 20–22
 çoklu-iş parçacıklı, 774, 776–777, 785–786
 kırmızı-siyah ağaç, 308–338
 -in birleştirilmesi, 332 problem
 -in maksimum anahtarı, 311
 -in minimum anahtarı, 311
 -in yüksekliği, 309
 -nın özellikleri, 308–312
 -ta arama, 311
 -ta araya sokma, 315–323
 -ta döndürme, 312–314
 -taki öncül, 311
 -taki öncül, 311
 -tan silme, 323–330
 ayrıca bakınız aralık ağacı,sıra-istatistiği
 ağacı,
 B-ağaçları ile kıyaslandığında, 484, 490
 bir görüntü kümesindeki anahtarları sayımla-
 mak için, 348 alıştıırma
 doğru parçalarının kesişip kesişmediğinin
 belirlenmesinde, 1024
 genişlemesi, 346–347
 gevşetilmiş, 311 alıştıırma
 yeniden inşa etme, 474 problem
 kısa-devre işlemcisi, 22
 kısıt çizgesi, 666–668
 kısıt, 851
 -in ihlali, 865
 doğrusal, 846
 eşitlik, 670 alıştıırma, 852–853
 eşitsizlik, 852–853

- fark, 665
- negatif olmama, 851, 853
- sıkı, 865
- kısmı sıralı küme, 1165
- kısmi sıralı bir kümenin maksimal elemanı, 1165
- kısmi sıralı, 1165
- kimlik doğrulama, 284 problem, 960–961, 964
- kiralama problemi, 114–115, 123–124, 145
 - nin olasılıksal analizi, 120–121
 - çevrim-içi, 139–141
- Kirchoff akım kuralı, 708
- kiriş, 345 alıştırma
- kirli alan, 208 problem
- Kleene yıldızı (*), 1058
- klik, 1086–1089, 1105
 - için yaklaşım algoritması, 1111 alıştırma, 1134 problem
- KMP algoritması, 1002–1013
- KMP-MATCHER, 1005
- Knuth-Morris-Pratt algoritması, 1002–1013
- kod sözcüğü, 429
- kod, 428–429
 - Huffman, 428–437, 450
- kol, 485
- kol
 - bir katar-eşleme otomatasının, 997 şekil
 - bir treap in, 333 problem
- kombinasyon $\binom{n}{k}$ 1185
- kombinasyon, 1185
- kompakt liste, 250 problem
- komşu listesi, 750
- komşu, 1172
- komşuluk, 735 alıştırma
- kontrol komutları, 23
- kontrol problemi, 122 alıştırma
- konumsal ağaç, 1178
- kopyalama komutu, 23
- koşullu bağımsızlık, 1195 alıştırma
- koşullu dallanma komutu, 23
- koşullu olasılık, 1192, 1194
- koşulsuz dal komutu, 23
- kova sıralaması, 200–204
- kova, 200
- kök listesi, bir Fibonacci bellek yığınının, 509
- kök
 - $\mathbb{Z}_n^*$ nin, 955
 - bir ağacın, 1176
 - birimin, 906–907
- köken, 1015
- köklü ağaç, 1176
 - in temsili, 246–249
- köprü, 621 problem
- köşegen matrisi, 1218
 - nin LUP ayrışımı, 827 alıştırma
- Kraft eşitsizliği, 1180 alıştırma
- kredi, 456
- kritik ayırıt, 729
- kritik yol
 - bir çoklu-iş parçacıklı hesaplamının, 779
 - bir yönlü çevrimsiz çizgenin, 657
- Kruskal algoritması, 631–633, 642
 - tam sayı kenar ağırlıkları ile, 637 alıştırma
- kupon toplayıcı problemi, 134
- kur farklılıklarından yararlanarak para alıp satma, 679 problem
- kusursuz çırpı fonksiyonu, 277–282, 285
- kusursuz eşleme, 735 alıştırma
- kutu paketleme, 1134 problem
- kutu, iç içe konan, 678 problem
- kutupsal açı, 1020 alıştırma
- kuvvet kümesi, 1161
- kuvvet serileri, 108 problem
- kuvvet, üs
 - aşıkarmayan, 933 alıştırma
 - bir elemanın ,modülo n ye göre, 954–958
 - k-ncı, 933 alıştırma
- kuyruk özyinelemesi, 188 problem, 419
- kuyruk, 232, 234–235
 - un bağlı liste uygulaması, 240 alıştırma
 - enine aramada, 595
 - itme-yeniden etiketleme algoritmalarında, 759 alıştırma
 - öncelik, *bakınız* öncelikli kuyruk, yığınlarla uygulanan, 236 alıştırma
- kuyruk
 - bağlı listenin, 236
 - binom dağılımının, 1208–1215
 - bir kuyruğun, 234
- kübik spline ile interpolasyon, 840 problem
- kübik spline, 840 problem
- küçük-oh notasyonu, 50–51, 64
- küçük-Omega notasyonu, 51
- küme ($\{ \}$), 1158–1163
 - bağımsız, 1101 problem
 - birleşimi ($\cup$), 1159
 - dışbükey, 714 alıştırma
 - eleman olmayan ($\notin$), 1158
 - elemanı ($\in$), 1158
 - farkı ($-$), 1159
 - kardinalitesi ($||$), 1161
 - kesişimi ($\cap$), 1159

- küme örtüsü problemi, 1117–1122, 1139
 - ağırlıklı, 1135 problem
- küme parçalama problemi, 1101 alıştırma
- küme
 - ön van Emde Boas yapılarında, 538
 - paralel hesaplama için, 772
 - sabit yüksekliğin bir üst üste ağacı ile bir bit vektöründe, 534
 - van Emde Boas ağaçlarında, 546
- kümeleme, 272
- kümeler için birleşim yasası, 1160
- kümeler için değişme yasaları, 1159
- kümeler için eşgüçlülük yasaları, 1159
- kümeler için yutma yasası, 1160
- kümelerde dağılma yasası, 1160
- kümelerin farkı ($-$), 1159
 - simetrik, 763 problem
- kümenin elemanı ($\in$), 1158
- kümenin elemanı (ϵ), 1158

- Lagrange formülü, 902
- Lagrange teoremi, 944
- Lam'e teoremi, 936
- LCA, 584 problem
- LCS-LENGTH, 394
- LCS, 7, 390–397, 413
- LEFT-ROTATE, 313, 353 alıştırma
- LEFT, 152
- Legendre sembolü ($\frac{a}{p}$), 982 problem
- lg (ikili logaritma), 56
- $\lg^k$ (logaritmaların üssü), 56
- lg (logaritmaların birleşimi), 56
- $\lg^*$ (iteratif logaritma fonksiyonu), 58–59
- LIFO (son-içeri, ilk-dışarı), 232
 - ayrıca bakınız ön bellek,
- LINK, 571
- LIST-DELETE, 238
- LIST-DELETE', 238
- LIST-INSERT, 238
- LIST-INSERT', 240
- LIST-SEARCH, 237
- LIST-SEARCH', 239
- liste, *bakınız* bağlı liste
- literal, 1082
- L_m -uzaklık 1044 alıştırma
- ln (doğal logaritma), 56
- logaritma fonksiyonu (log), 56–57
 - ayrık, 955
 - iteratif ($\lg^*$), 58–59
- LONGEST-PATH-LENGTH, 1060 alıştırma
- LONGEST-PATH, 1060 alıştırma
- LOOKUP-CHAIN 388
- loop**, kaba kodda, 20
 - paralel, 785–787
- LU ayrışımı, 806 problem, 819–822
- LU-DECOMPOSITION, 821
- LUP ayrışımı, 806 problem, 815
 - nın hesaplaması, 822–825
 - bir köşegen matrisin, 827 alıştırma
 - bir permütasyon matrisinin, 827 alıştırma
 - kullanımı, 815–819
 - matris tersinde, 828
 - ve matris çarpımı, 832 alıştırma
- LUP-DECOMPOSITION, 824
- LUP-SOLVE, 817

- madeni para değiştirme, 446 problem
- MAKE-HEAP, 505
- MAKE-SET, 561
 - in ayrık-küme-orman uygulaması, 571
 - in bağlı-liste uygulaması, 564
- MAKE-TREE, 583 problem
- maks-akış min-kesim teoremi, 723
- maks-bellek yığını özelliği, 152
 - nin bakımı, 154–156
- maks-bellek yığını, 152
 - nda araya sokma, 164
 - nda bir anahtarı artırım, 163–164
 - ndan maksimum anahtarı dışa aktarım, 163
 - ndan silme, 166 alıştırma
 - nın maksimum anahtarı, 163
 - bellek yığını sıralamasında, 159–162
 - bir maks-öncelikli kuyruk olarak, 162–166
 - birleştirilebilir, 250 dipnot, 481 dipnot, 505
 - dipnot
 - d -li, 167 problem
 - inşası, 156–159
- maks-öncelikli kuyruk, 162
- maksimal eşleme, 1110, 1135 problem
- maksimal katmanlar, 1045 problem
- maksimal nokta, 1045 problem
- maksimizasyon doğrusal program, 846
 - ve minimizasyon doğrusal programlar, 852
- maksimum akış, 708–766
 - bir doğrusal program olarak, 860–861
 - güncelleme, 762 problem
 - için artma algoritması, 762 problem, 765
 - için Edmonds-Karp algoritması, 727–730

- MAX-CUT problemi, 1127 alıştırma
 MAX-FLOW-BY-SCALING, 763 problem
 MAX-HEAP-INSERT, 164
 ile bir bellek yığını inşası, 166 problem
 MAX-HEAPIFY, 154
 MAYBE-MST-A, 641 problem
 MAYBE-MST-B, 641 problem
 MAYBE-MST-C, 641 problem
 medyan anahtarı, bir B-ağacı düğümünün 493
 medyan, 213–227
 ağırlıklı, 225 problem
 için çoklu-iş parçacıklı algoritma, 805 alıştırma
 iki sıralı listelerin, 804 alıştırma
 sıralı listelerin, 223 alıştırma
 MEMOIZED-CUT-ROD-AUX, 366
 MEMOIZED-CUT-ROD, 365
 MEMOIZED-MATRIX-CHAIN, 388
 MERGE-LISTS, 1129
 MERGE-SORT, 34
 MERGE-SORT', 797
 MERGE, 31
 mertebe, bir grubun, 945
 mesafe düzeni, 406 problem
 metin, katar eşlemede, 985
 Miller-Rabin asalılık testi, 968–975, 983
 MILLER-RABIN, 970
 min-bellek yığını özelliği, 153, 507
 -ne karşı ikili-arama-ağacı özelliği, 289
 alıştırma
 -niün bakımı, 156 alıştırma
 treaps da, 333 problem
 min-bellek yığını sıralaması, 507
 min-bellek yığını, 153
 bir min-öncelikli kuyruk olarak, 165 alıştırma
 birleştirilebilir, 250 dipnot, 481 dipnot, 505
 d-li, 706 problem
 Dijkstra algoritmasında, 662
 Huffman algoritmasında, 433
 inşası, 156–159
 Johnson algoritmasında, 704
 potansiyel yöntem ile analiz edilen, 462
 alıştırma
 Prim algoritmasında, 636
 minimizasyon doğrusal programlar, 846
 ve maksimizasyon doğrusal programlar, 852
 MIN-GAP, 354 alıştırma
 MIN-HEAP-INSERT, 165 alıştırma
 MIN-HEAPIFY, 156 alıştırma
 min-öncelikli kuyruk, 162
 Dijkstra algoritmasında, 661
 Huffman kodlarının oluşturulmasında, 431
 Prim algoritmasında, 634, 636
 minimum ağırlıklı kapsayan ağaç, *bakınız* minimum kapsayan ağaç
 minimum ağırlıklı tepe noktası kaplama, 1124–1127, 1139
 minimum anahtar ayıklanması, 2-3-4 bellek yığınlarından, 529 problem
 Fibonacci bellek yığınlarından, 512–518
 Young tablolarından, 167 problem
 minimum derece, B-ağacının, 489
 minimum düğüm, Fibonacci bellek yığınının, 508
 minimum kapsayan ağaç, 624–642
 dinamik çizgelerde, 637 alıştırma
 gezgin-satıcı problemi, 1112
 için Bor'uvka algoritması, 641
 için genel yöntem, 625–630
 için Kruskal algoritması, 631–633
 için Prim algoritması, 634–636
 için yaklaşım algoritmasında, ikinci-en iyi, 638 problem
 matroidlere ilişkin, 437, 439–440
 minimum kesim, 721, 731 alıştırma
 minimum orta-ağırlıklı çevrim, 680 problem
 minimum yol örtüsü, 761 problem
 minimum-maliyetli akış, 861–862
 minimum-maliyetli çoklu-ürün akışı, 864 alıştırma
 minimum-maliyet sirkülasyonu, 896 problem
 minimum-maliyetli kapsayan ağaç *bakınız* minimum kapsayan ağaç
 MINIMUM, 162, 214, 230, 505
 minimum, 213
 2-3-4 bellek yığınlarında, 529 problem
 B-ağaçlarında, 497 alıştırma
 bir sabit yüksekliğin bir üstüste ağacı ile bir bit vektöründe, 535
 bir üstüste ikili ağacı ile bir bit vektöründe, 533
 bulma, 214–215
 çevrim-dışı, 582 problem
 Fibonacci bellek yığınlarında, 511
 ikili arama ağaçlarında, 291
 kırmızı-siyah ağaçlarda, 311
 ön van Emde Boas yapılarında, 541–542
 sıra istatistiği ağaçlarında, 347 alıştırma
 van Emde Boas ağaçlarında, 550
 mod, 54, 928
 MODULAR-EXPONENTIATION, 957

MODULAR-LINEAR-EQUATION-SOLVER, 949
 modüler aritmetik, 54, 923 problem, 939–946
 modüler denklik, 54, 1165 alıştırmaya
 modüler doğrusal denklemler 946–950
 modüler üs alma, 956
 modülo n ye göre çarpımsal grup, 941
 modülo n ye göre toplamsal grup, 940
 modülo, 54, 928
 Monge dizisi, 110 problem
 monoton artan, 53
 monoton azalan, 53
 monoton dizi, 168
 Monty Hall problemi, 1195 alıştırmaya
 MST-KRUSKAL, 631
 MST-PRIM, 634
 MST-REDUCE, 639 problem
 MULTIPOP, 453
 MULTIPUSH, 456 alıştırmaya
 mutlak yakınsak seriler, 1146
 mükemmel doğrusal hızlanma, 780

$\mathbb{N}$ (Doğal Sayılar kümesi) 1158
 n -küme, 1161
 n -tuple, 1162
 n -vektör, 1218
 NAIVE-STRING-MATCHER, 988
 negatif olmama kısıtı, 851, 853
 negatif-ağırlıklı ayrıtlar, 645–646
 negatif-ağırlıklı çevrim
 ve en kısa yollar, 645, 653–654, 692 alıştırmaya, 700 alıştırmaya
 ve fark kısıtları, 667
 ve gevşeme, 677 alıştırmaya
 nesne, 21
 -nin dizi uygulaması, 241–246
 -nin tahsisi ve serbestliği, 243–244
 bir parametre olarak geçen, 21
 nesnelerin serbest bırakılması, 243–244
 nesnelerin serbest bırakılması, 243–244
 nesnelerin tahsisi, 243–244
 NEXT-TO-TOP, 1031
 NIL, 21
 nokta-değer temsili, 901
 noktaların dışbükey birleşimi, 1015
 normal denklem, 837
 NOT fonksiyonu ($\neg$), 1071
 NOT kapısı, 1070
 nöbetçi, 31, 238–240, 309
 NP (karmaşıklık sınıfı), 1049, 1064, 1066 alıştırmaya,

ma, 1105
 NP-tam, 1050, 1069
 NP-tamlığı, 9–10, 1048–1105
 -nı ispatlama, bir dilin, 1078–1079
 0-1 tam sayılı programlamanın, 1100 alıştırmaya
 3-CNF-gerçekleştirilebilirlik probleminin, 1082–1085, 1105
 altçizge-izomorfizma probleminin, 1100 alıştırmaya
 altküme-toplam probleminin, 1097–1100
 bağımsız-yol probleminin, 1101 problem
 bir boole formülünün totoloji olup olmadığının belirlenmesi, 1086 alıştırmaya
 çizge-renklendirme probleminin, 1103 problem
 devresel gerçekleştirilebilirlik probleminin, 1070–1077
 en uzun-basit-çevrim probleminin, 1101 alıştırmaya
 formül-gerçekleştirilebilirlik probleminin, 1079–1081, 1105
 gezgin-satıcı probleminin, 1096–1097
 hamilton-çevrim probleminin, 1091–1096, 1105
 hamilton-yol probleminin, 1101 alıştırmaya
 kazançlar ve bitiş zamanları ile çizelgelemenin, 1104 problem
 klik probleminin, 1086–1089, 1105
 küme örtüsü probleminin, 1122 alıştırmaya
 küme parçalama probleminin, 1101 alıştırmaya
 tam sayılı doğrusal programlamanın, 1101 alıştırmaya
 tepe örtüsü probleminin, 1089–1091, 1105
 yarı 3-CNF gerçekleştirilebilirlik problemi, 1101 alıştırmaya
 NP-zor, 1069
 NPC (karmaşıklık sınıfı), 1050, 1069

$\widetilde{O}$ -notasyonu, 62 problem
 O' -notasyonu, 62 problem
 O -notasyonu, 45 şekil, 47–48, 64
 o -notasyonu, 50–51, 64
 Θ -notasyonunun simetriği, 52
 OFF-LINE-MINIMUM, 583 problem
 olasılık dağılımı fonksiyonu, 204 alıştırmaya
 olasılık dağılımı, 1190
 olasılık yoğunluğu fonksiyonu, 1196
 olasılık, 1189–1196

- olasılıksal analiz, 115–116, 130–142
 - açık-adresli çarpılamanın, 274–276, 277
 - alıştırma
 - algoritmalar, 123–124
 - aralıkların bulanık sıralanmalarının, 189
 - problem
 - ardışık tekrarların, 135–139
 - bir kompakt listenin aranmasının, 250 problem
 - çakışmaların, 261 alıştırma, 282 alıştırma
 - çevrimiçi kiralama probleminin 139–141
 - çarpılama için en uzun-deneme sınırının, 282
 - problem
 - doğum günü paradoksunun, 130–133
 - dosya karşılaştırmanın, 995 alıştırma
 - eş anahtarlara sahip ikili arama ağacına eklemenin, 303 problem
 - evrensel çarpılamanın, 265–268
 - hızlı sıralamanın, 181–184, 187–188 problem
 - 303 alıştırma
 - için yaklaşım algoritmasının,
 - kiralama probleminin, 120–121, 139–141
 - kova sıralamasının, 201–204, 204 alıştırma
 - MAX-3-CNF gerçekleştirilebilirliği, 1124
 - Miller-Rabin asallık testinin, 971–975
 - mükemmel çarpılamanın, 279–282
 - noktaların orijinden uzaklıklarına göre sıralanmasının, 204 alıştırma
 - olasılıksal saymanın, 143 problem
 - parçalanmanın, 179 alıştırma, 185 alıştırma, 187–188 problem
 - Pollard rho sezgiseli, 977–980
 - Rabin-Karp algoritmasının, 994
 - rassal oluşturulmuş ikili arama ağacında ortalama düğüm derinliğinin, 304 problem
 - rassal oluşturulmuş ikili arama ağacının yüksekliğinin, 299–303
 - rassal seçimin, 217–219, 226 problem
 - seyrek-örtülü dağılım üzerindeki bir dışbükey örtünün, 1046 problem
 - sıralama için alt sınırın, 205 problem
 - topların ve kutuların, 133–134
 - ve çoklu iş parçacıklı algoritmalar, 811 problem
 - ve ortalama girdiler, 28
 - zincirleme için dilim zaman-boyutlu sınırını 283 problem
 - zincirleme ile çarpılamanın, 258–260
- olasılıksal olarak sınanabilir ispat, 1105, 1140
- olasılıksal sayma, 143 problem
- olay noktası, 1023
- olay-noktası çizelgeleme, 1023
- olmayan çocuk, 1178
- Omega-notasyonu, 45 şekil, 48–49, 64
- ON-LINE-MAXIMUM, 140
- ON-SEGMENT, 1018
- onanır ağ, 749–750
- onanır ayırıt, 749
- OpenMP, 774
- optimal alt küme, bir matroidin, 439
- optimal altyapı
 - 0-1 sırt çantası probleminin, 426
 - açgözlü algoritmalarda, 425
 - ağırlıklı matroidlerin, 442
 - ağırlıksız en kısa yolların, 382
 - çubuk-kesmenin, 362
 - dinamik programlamada, 379–384
 - en kısa yolların, 644–645, 687, 693–694
 - en uzun ortak altdizilerin, 392–393
 - etkinlik seçiminin, 416
 - Huffman kodlarında, 435
 - ikili arama ağaçlarının, 399–400
 - kesirli sırt çantası probleminin, 426
 - matris-zinciri çarpımının, 373
- optimal çözüm, 851
- optimal ikili arama ağacı, 397–404, 413
- optimal amaç değeri, 851
- optimal tepe örtüsü, 1108
- OPTIMAL-BST, 402
- optimizasyon problemi, 359, 1050, 1054
 - için yaklaşım algoritmaları, 10, 1106–1140
 - ve karar problemleri, 1051
- OR fonksiyonu ($\vee$), 697, 1071
- OR kapısı, 1070
- or, kaba kodda, 22
- orman, 1172–1173
 - ayrık-küme, 568–572
 - derinliğine, 603
- orta tepe, 693
- ortak altdizi, 7, 391
 - en uzun, 7, 390–397, 413
- ortak altifade, 915
- ortak anahtar şifreleme sistemi, 958–965, 983
- ortak anahtar, 959, 962
- ortak bölen, 929
 - en büyük, *bakınız* en büyük ortak bölen
- ortak çarpan, 939 alıştırma
- ortak olasılık yoğunluk fonksiyonu, 1197
- ortalama-durum çalışma süresi, 28, 116
- ortalama, *bakınız* beklenen değer

- ortanormal, 842
- OS-KEY-RANK, 344 alıştırma
- OS-RANK, 342
- OS-SELECT, 341
- otomata,
 - sonlu, 995
 - katar-eşleme, 996–1002
- Öklid algoritması, 933–939, 981 problem, 983
- öklid mesafesi, 1039
- öklid normu ($\| \cdot \|$), 1222
- ölçekleme,
 - maksimum akışta, 762 problem, 765
 - tek-kaynaklı en kısa yollarda, 679 problem
- ön altmatris, 833, 839 alıştırma
- ön belleğe alma, 365, 387–389
- ön sıra ağaç yürüyüşü, 287
- ön sıra,tam, total, 1165
- ön sıralamada, 1043
- ön van Emde Boas yapısı, 538–545
 - nda araya sokma, 544
 - ndaki ardıl, 543–544
 - ndaki küme, 538
 - ndaki maksimum, 544 alıştırma
 - ndaki minimum, 541–542
 - ndaki öncül, 544 alıştırma
 - ndaki özet, 540
 - ndaki üyelik, 540–541
 - ndan silme, 544
 - van Emde Boas ağaçları ile kıyaslanan, 547
- ön-vEB yapısı, *bakınız* ön van Emde Boas yapısı,
- önakiş, 736, 765
- önalım, 447 problem
- önbelleğe alma, çevrimdışı 449 problem
- önbelleğe rastlama yüzdesi, 449 problem
- önbelleğe rastlamama yüzdesi, 449 problem
- önbellek unutkanlığı, 504
- önbellek, 24, 449 problem
- öncelikli kuyruk, 162–166
 - un bellek yığını uygulaması, 162–166
 - un ön van Emde Boas yapı uygulamasında, 538–545
 - un van Emde Boas ağacı uygulaması, 531–560
- ayrıca bakınız* ikili arama ağacı, binom bellek yığını ,Fibonacci bellek yığını,
- Dijkstra algoritmasında, 661
- Huffman kodlarının üretilmesinde, 431
- için alt sınırlar, 531
- max-öncelikli kuyruk, 162
- min-öncelikli kuyruk, 162, 165 alıştırma
- monoton çıkarımlar ile, 168
- Prim algoritmasında, 634, 636
- öncül altçizge,
 - derinliğine aramada, 603
 - enine aramada, 600
 - tek-kaynaklı en kısa yollarda, 647
 - tüm-ikililerin en kısa yollarında, 685
- öncül matris, 685
- öncül-altçizge özelliği, 650, 676
- öncül
 - B-ağaçlarında, 497 alıştırma
 - bağılantılı listelerde, 236
 - bir üstüste ikili ağacı ile bir bit vektöründe, 534
 - en kısa yol ağaçlarında, 647
 - enine ağaçlarda, 594
 - ikili arama ağaçlarında, 291–292
 - kırmızı-siyah ağaçlarda, 311
 - ön van Emde Boas yapılarında, 544 alıştırma
 - sabit yüksekliğin bir üstüste ağacı ile bir bit vektöründe, 535
 - sıra-istatistiği ağaçlarında, 347 alıştırma
 - van Emde Boas ağaçlarında, 551–552
- öne-taşıyan sezgisel, 476 problem, 478
- öne-yeniden etiketleme algoritması, 748–760
 - nın fazı, 758
- örnek fonksiyonu iterasyon lemması, 1007
- örnek fonksiyonu, 1003–1004
- örnek hesaplaması, 807 problem
- örnek kodu, 429
- örnek
 - bir dizinin, 392
 - bir katarın ($\sqsubset$), 986
- örnek olmayan, 1056 dipnot
- örnek problemlerin kodlanması 1055–1057
- örnek uzayı, 1189
- örnek
 - bir problemin, 5
 - bir soyut problemin, 1051, 1054
- örnekleme, 129 alıştırma, 179
- örten fonksiyon, 1167
- örten, 1167
- örtü, dışbükey, 8, 1029–1039, 1046 problem
- örtü
 - bir altküme tarafından, 1118
 - tepe, 1089, 1108, 1124–1127, 1139
 - yol, 761 problem
- örtüşen altproblemler, 384–386

- örtüşen aralıklar, 348
 - in hepsini bulma, 354 alıştırma
 - maksimum örtüşme noktası, 354 problem
- örtüşen dikdörtgenler, 354 alıştırma
- örtüşen sonek lemması, 987
- öyle ki (:), 1159
- öz altgrup, 944
- öz altküme ($\subset$), 1159
- özellik değıştirme, 437
- özet,
 - ön Van Emde Boas yapılarında, 540
 - sabit yüksekliğin bir üstüste ağacı ile bir bit vektöründe, 534
 - Van Emde Boas ağaçlarında, 546
- özyineleme ağacı, 37, 88–93
 - ana yöntemin ispatında, 98–100
 - ve yerine koyma yöntemi, 91–92
- özyineleme denklemi, *bakınız* özyineleme
- özyineleme, 30
- özyineleme, 34, 65–67, 83–113
 - Akra-Bazzi yöntemiyle çözüm, 112–113
 - ana yöntemle çözüm, 93–97
 - özyinelemeli-ağaç yöntemiyle çözüm, 88–93
 - yerine koyma yöntemiyle çözüm, 83–88
- özyinelemeli durum, 65
- özyinelemenin yaprakların irileşmesi
 - birleştirerek sıralamada, 39 problem
 - özyinelemeli doğurma olduğunda, 787
- $\rho(n)$ -yaklaşım algoritması, 1106, 1123

- P (karmaşıklık sınıfı) 1049, 1055, 1059, 1061
 - alıştırma, 1105
- P-FIB, 776
- P-MATRIX-MULTIPLY-RECURSIVE, 794
- P-MERGE-SORT, 803
- P-MERGE, 800
- P-SCAN-1, 808 problem
- P-SCAN-2, 808 problem
- P-SCAN-3, 809 problem
- P-SCAN-DOWN, 809 problem
- P-SCAN-UP, 809 problem
- P-SQUARE-MATRIX-MULTIPLY, 793
- P-TRANSPose, 792 alıştırma
- paket sarma, 1037, 1047
- palindrom, 405 problem
- para değış tokuşu 390 alıştırma, 679 problem
- paralel algoritma, 10, 772
 - ayrıca bakınız* çoklu-iş parçacıklı algoritma,
- paralel bilgisayar, 772
 - ideal, 779
 - paralel döngü, 785–787, 805 problem
 - paralel gevşeklik, 781
 - temel kararı, 783
 - Paralel Görev Kütüphanesi, 774
 - paralel önek, 807 problem
 - paralel rasgele-erişimli makine 811
 - paralel-makine-çizelgeleme problemi, 1136
 - problem
 - paralel, -da mantıksal olan dizilimler, 778
 - paralellik
 - bir çoklu-iş parçacıklı hesaplamının, 780
 - bir rassal çoklu-iş parçacıklı algoritmanın, 811 problem
 - içiçe, 776
 - mantıksal, 777
 - parallel for**, kaba kodda, 785–786
 - parametre, 21
 - geçen maliyetleri, 107 problem
 - parantez teoremi, 606
 - parça, *bakınız* yönlü doğru parçası,
 - parçacık, bileşen, 1092
 - parçalama fonksiyonu, 361 dipnot
 - parçalanma, 171–173
 - 3 elemanın medyan çevresinde, 185 alıştırma
 - için çoklu-iş parçacıklı algoritma, 804 alıştırma
 - ma
 - için Hoare's yöntemi, 185 problem
 - rassal, 179
 - PARENT, 152
 - PARTITION, 171
 - PARTITION', 186 problem
 - Pascal üçgeni, 1188 alıştırma
 - PATH, 1051, 1058
 - paylaşılmış bellek, 772
 - PERMUTE-BY-CYCLIC, 129 alıştırma
 - PERMUTE-BY-SORTING, 125
 - PERMUTE-WITH-ALL, 129 alıştırma
 - PERMUTE-WITHOUT-IDENTITY, 128 alıştırma
 - permütasyon matrisi, 1220, 1222 alıştırma, 1226
 - alıştırma
 - nin LUP ayrışımı, 827 alıştırma
 - Permütasyon, 1167
 - bir kümenin, 1184
 - doğrusal, 1229 problem
 - düzgün rassal, 116, 125
 - Josephus, 355 problem
 - k -permütasyon, 126, 1184
 - rassal, 124–128
 - ters-bit, 472 problem, 918
 - yerde, 126

- PERSISTENT-TREE-INSERT, 331 problem
 PERT tablosu, 657, 657 alıştırma
 phi fonksiyonu ($\phi(n)$), 943
 PISANO-DELETE, 526 problem
 PIVOT, 869
 pivot
 doğrusal programlamada, 867, 869–870, 878
 alıştırma
 hızlı sıralamada, 171
 LU ayrışımında, 821
 plak, 485
 polilogaritmik sınırlı, 57
 polinom-zamanlı algoritma, 927, 1048
 polinom-zamanlı çözülebilirlik, 1055
 polinom-zamanlı doğrulama, 1061–1066
 polinom-zamanlı hesap edilebilirlik, 1056
 polinom-zamanlı indirgenebilirlik ($\leq_p$), 1067,
 1077 alıştırma
 polinom-zamanlı kabul, 1058
 polinom-zamanlı karar, 1059
 polinom-zamanlı yaklaşım şeması, 1107
 maksimum klik için, 1134 problem
 polinom, 55, 898
 -un asimptotik davranışı, 61 problem
 -un çarpımı, 899, 903–905, 920 problem
 -un değerlendirmesi, 41 problem, 900, 905
 alıştırma, 923 problem
 -un katsayı temsili, 900
 -un nokta-değer temsili, 901
 -un toplamı 898
 -un türevleri, 922 problem
 ile interpolasyon, 901, 906 alıştırma
 polinomiyal ilişkili, 1056
 polinomiyal sınırlı, 55
 polinomiyal-büyüme koşulu, 113
 Pollard rho sezgiseli, 976–980, 980 alıştırma, 984
 POLLARD-RHO, 976
 POP, 233, 452
 postane konumlandırma problemi, 225 problem
 potansiyel fonksiyon, 459
 alt sınırlar için, 478
 potansiyel yöntem, 459–463
 ayrık-küme veri yapıları için, 575–581, 582
 alıştırma
 dinamik tablolar için, 466–471
 Fibonacci bellek yığınları için, 509–512,
 517–518, 520–522
 genel itme-yeniden etiketleme algoritması
 için, 746
 ikili sayılar için, 461–462
 kırmızı-siyah ağaçların yeniden inşası için,
 474 problem
 min-bellek yığınları için, 462 alıştırma
 öne-taşıma ile kendi kendini düzenleyen
 listeler için, 476 problem
 yığın işlemleri için, 460–461
 potansiyeli, bir veri yapısının, 459
 pozitif tanımlı matris, 1225
 Pr{ } (olasılık dağılımı), 1190
 PRAM, 811
 PREDECESSOR, 230
 Prim algoritması, 634–636, 642
 bir bitişik matris ile, 637 alıştırma
 Dijkstra'nın algoritmasına benzerlik, 634,
 662
 Fibonnaci bellek yığını ile uygulanan, 636
 gezgin-satıcı problemi, 1112
 için yaklaşım algoritmasında,
 min-bellek yığını ile uygulanan, 636
 seyrek çizgeler için, 638 problem
 tam sayılı ayrıt ağırlıkları ile, 637 alıştırma
 PRINT-ALL-PAIRS-SHORTEST-PATH, 685
 PRINT-CUT-ROD-SOLUTION, 369
 PRINT-INTERSECTING-SEGMENTS, 1028 alıştırma
 PRINT-LCS, 395
 PRINT-OPTIMAL-PARENS, 377
 PRINT-PATH, 601
 PRINT-SET, 572 alıştırma
 problem
 çözülebilir, 1048
 çözülemez, 1048
 çözümü, 6, 1054–1055
 hesaplanabilir, 5–6
 karar, 1051, 1054
 optimizasyon, 359, 1050, 1054
 somut, 1055
 soyut, 1054
 profesyonel güreşçi, 602 alıştırma
 program sayacı, 1073
 programlama, *bakınız* dinamik programlama,
 doğrusal programlama,
 prosedür, 6, 16–17
 PROTO-VEB-INSERT, 544
 PROTO-VEB-MEMBER, 541
 PROTO-VEB-MINIMUM, 542
 PROTO-VEB-SUCCESSOR, 543
 PSEUDOPRIME, 967
 PUSH
 itme-yeniden etiketleme işlemi, 739
 yığın işlemi, 233, 452

- QUICKSORT, 171
 QUICKSORT', 186 problem
- $\mathbb{R}$ (reel sayılar kümesi), 1158
 Rabin-Karp algoritması, 990–995, 1013
 RABIN-KARP-MATCHER, 993
 RACE-EXAMPLE, 788
 RADIX-SORT, 198
 RAM modelin komutları, 23
 RAM, 23–24
 RANDOM-SAMPLE, 130 alıştırma
 RANDOM-SEARCH, 143 problem
 RANDOM, 117
 RANDOMIZE-IN-PLACE, 126
 RANDOMIZED-HIRE-ASSISTANT, 124
 RANDOMIZED-PARTITION, 179
 RANDOMIZED-QUICKSORT, 179, 303 alıştırma
 rassal oluşturulmuş ikili arama ağaçlarına
 ilişkin, 304 problem
 RANDOMIZED-SELECT, 216
 rank ile birleşim, 569
 rank
 bir ayırık-küme ormanındaki bir düğümün,
 569, 575, 581 alıştırma
 bir matrisin, 1223, 1226 alıştırma
 bir sıralı kümedeki bir sayının, 300, 339
 satır, 1223
 sıra-istatistiği ağaçlarında, 341–343, 344–345
 alıştırma
 sütun, 1223
 tam, 1223
 rassal algoritma, 116–117, 122–130
 -nın en-kötü-durum performansı, 180 alıştırma
 aralıkların bulanık sıralaması için, 189 problem
 bir diziyi karmak için, 124–128
 bir kompakt listeyi aramak için, 250 problem
 çoklu iş parçacıklı, 811 problem
 eşit anahtarlara sahip bir ikili arama ağacını
 araya sokma için, 303 problem
 evrensel çırpılama, 265–268
 hızlı sıralama, 179–180, 185 alıştırma,
 187–188 problem
 kıyaslama sıralaması, 205 problem
 kiralama problemi için, 123–124
 MAX-3-CNF gerçekleştirilebilirliği için,
 1123–1124, 1139
 Miller-Rabin asalılık testi, 968–975, 983
 parçalama için, 179, 185 alıştırma, 187–188
 problem
 Pollard rho sezgiseli, 976–980, 980 alıştırma,
 984
 rassal yuvarlama, 1139
 seçim için, 215–220
 ve olasılıksal analiz için, 123–124
 ve ortalama girdiler, 28
 rassal değişken, 1196–1201
 gösterge, *bakınız* rassal gösterge değişkeni,
 rassal gösterge değişkeni, 118–121
 -nin beklenen değeri, 118
 ardışık tekrarların analizinde, 138–139
 binom dağılımının sağ kuyruğunun sınırla-
 masında, 1212–1213
 bir treap (tree-heap) araya sokma analizinde,
 333 problem
 çırpılama analizinde, 259–260
 doğum günü paradoksunun çözümlemesinde,
 132–133
 evrensel-çırpılama analizinde, 265–266
 hızlı sıralama analizinde, 182–184, 187 prob-
 lem
 kiralama problemi analizinde, 120–121
 kova sıralaması analizinde, 202–204
 MAX-3-CNF gerçekleştirilebilirliği için bir
 yaklaşım algoritmasında, 1124
 rassal oluşturulmuş ikili arama ağacının bek-
 lenen yüksekliğinin analizinde, 300–303
 rassal-seçim analizinde, 217–219, 226 prob-
 lem
 ve beklentinin doğrusallığı 119
 rassal örnekleme, 129 alıştırma, 179
 rassal permütasyon, 124–128
 düzgün, 116, 125
 rassal üretilmiş ikili arama ağacı, 299–303, 304
 problem
 rassal yuvarlama, 1139
 rassal-erişimli makine, 23–24
 paralel, 811
 rassal-sayı üretici, 117
 RB-DELETE-FIXUP, 326
 RB-DELETE, 324
 RB-ENUMERATE, 348 alıştırma
 RB-INSERT-FIXUP, 316
 RB-INSERT, 315
 RB-JOIN, 332 problem
 RB-TRANSPLANT, 323
 RECURSIVE-ACTIVITY-SELECTOR, 419
 RECURSIVE-FFT, 911

- RECURSIVE-MATRIX-CHAIN, 385
- reddetme,
 - bir algoritma tarafından, 1058
 - bir sonlu otomata tarafından, 996
- REDUCE, 807 problem
- reel sayılar ($\mathbb{R}$), 1158
- rekabetçi analiz, 476 problem
- RELABEL-TO-FRONT, 755
- RELABEL, 740
- RELAX, 649
- renk, bir kırmızı-siyah ağaç düğümünün, 308
- renklendirme, 1103 problem, 1180 problem
- repeat**, kaba kodda, 20
- REPETITION-MATCHER, 1013 problem
- RESET, 459 alıştırma
- resmi kuvvet serileri, 108 problem
- return**, kaba kodda, 22
- rho sezgiseli, 976–980, 980 alıştırma, 984
- RIGHT-ROTATE, 313
- RIGHT, 152
- RS-vEB ağacı, 557 problem
- RSA ortak-anahtarlı şifreleme sistemi, 958–965, 983
- 0-1 sıralama lemması, 208 problem
- 0-1 sırtçantası problemi, 425, 427 alıştırma, 1137 problem, 1139
- 0-1 tam sayılı programlama, 1100 alıştırma, 1125
- sabermetrik, 412 dipnot
- sabit-uzunluklu kod, 429
- sade algoritma, katar eşleme için, 988–990
- sağ altağaç, 1178
- sağ çocuk, 1178
- sağ kol, 333 problem
- sağ yatay ışıını, 1021 alıştırma
- sağa çevirim, 314 alıştırma
- sağa döndürme, 312
- sahte vuruş, 991
- SAME-COMPONENT, 563
- sanal bellek, 24
- SAT, 1079
- satır rankı, 1223
- satır vektörü, 1218
- satır-öncelikli sıra, 394
- satranç-oylayan program, 790–791
- sayaç, *bakınız* ikili sayaç
- sayı-alan eleği, 984
- sayılabilir sonsuz küme, 1161
- sayılamaz küme, 1161
- sayısal imza, 960
- sayısal sabitlik, 813, 815, 842
- sayma sıralaması, 194–197
 - taban sıralamasında, 198
- sayma, 1183–1189
 - olasılıksal, 143 problem
- SCAN, 807 problem
- Schur tümleyeni lemması, 834
- Schur tümleyeni, 820, 834
- SCRAMBLE-SEARCH, 143 problem
- SEARCH, 230
- seçici tepe, 1093
- seçim, 213
 - beklenen doğrusal zamanda, 215–220
 - çoklu iş parçacıklı, 805 alıştırma
 - en kötü-durum doğrusal zamanında, 220–224
 - etkinliklerin, 415–422, 450
 - sıra-istatistiği ağaçlarında, 340–341
 - ve karşılaştırma sıralamaları, 222
- seçme sıralaması, 29 alıştırma
- SEGMENTS-INTERSECT, 1018
- SELECT, 220
- sembol tablosu, 253, 262, 265
- serbest ağaç, 1172–1176
- serbest bırakma zamanı, 447 problem
- serbest liste, 243
- serbest, 411 problem
- seri algoritmaya karşı paralel algoritma 772
- seriler, 108 problem, 1146–1148
 - de mantıksal olan dizilimler, 778
- serileştirme, bir çoklu iş parçacıklı algoritmanın 774, 776
- sertifika
 - bir şifreleme sisteminde, 964
 - doğrulama algoritmaları için, 1063
- seyrek çizge, 589
 - tüm-ikililerin en kısa yolları için, 700–705
 - ve Prim algoritması, 638 problem
- seyrek-örtülü dağılım 1046 problem
- Shell sıralaması, 42
- SHORTEST-PATH, 1050
- Sıfır matrisi, 1218
- sıkı kısıt, 865
- sıklık 1169
- sıklık matrisi
 - bir yönlü çizgenin, 448 problem, 593 alıştırma
 - bir yönsüz çizgenin, 448 problem
 - ve fark kısıtları, 666
- sınıf

- karmaşıklık, 1059
- denklik, 1164
- sınır koşulu, bir özyinelemede, 67, 84
- sınır
 - asimptotik alt, 48
 - asimptotik olarak sıkı, 45
 - asimptotik üst, 47
 - ayrıca bakınız* alt sınırlar
 - binom dağılımları üzerinde, 1206
 - binom katsayıları üzerinde, 1186–1187
 - bir binom dağılımının kuyrukları üzerinde, 1208–1215
 - polilogaritmik, 57
- sınırlı olmayan doğrusal programlama, 851
- sıra istatistiği ağacı, 339–345
 - sorgulama, 347
- sıra-istatistiği, 213–227
 - dinamik, 339–345
 - için çoklu-iş parçacıklı algoritma, 805
- sıralama ağı, 811
- sıralama, 5, 16–20, 30–37, 147–212, 797–805
 - 0-1 sıralama lemması, 208
 - arayaya sokma sıralaması, 12, 16–20
 - bellek yığını sıralaması, 151–169
 - bir ikili arama ağacını kullanarak 299
 - birleştirerek sıralama, 12, 30–37, 797–805
 - bulanık, 189
 - çalışma sürelerinin tablosu, 149
 - değişken-uzunluk öğeleri ile 206
 - doğrusal zamanda, 194–204, 206
 - hızlı sıralama, 170–190
 - için alt sınırlar, 191–194, 211, 531
 - için olasılıksal alt sınır, 205
 - ihmalî karşılaştırma-değiştirme algoritmaları ile, 208
 - k -sıralaması, 207
 - kabarcık sıralaması, 40
 - kıyaslı, 196
 - karşılaştırma sıralaması, 191
 - kova sıralaması, 200–204
 - kutupsal açı ile noktaların, 1020
 - sayma sıralaması, 194–197
 - seçme sıralaması, 29
 - Shell sıralaması, 42
 - sözlüksel, 304
 - sütun sıralama, 208
 - taban sıralaması, 197–200
 - topolojik, 8, 612–615, 623
 - yerde, 17, 148, 206
- sıralama
 - doğrusal, 1165
 - kısmi, 1165
 - tam, 1165
- sıralı ağaç, 1177
- sıralı bağlı liste, 236
 - ayrıca bakınız* bağlı liste,
- sıralı ikili, 1161
- sıralı olmayan bağlı liste, 236
 - ayrıca bakınız* bağlı liste,
- sırtçantası problemi,
 - 0-1, 425, 427
 - alıştırma, 1137
 - problem, 1139
 - kesirli, 426, 428
- silme
 - 2-3-4 bellek yığınlarından, 529
 - problem
 - açık-adresli çarpı tablolarından, 271
 - aralık ağaçlarından, 349
 - B-ağaçlardan, 499–502
 - bağlı listelerden, 238
 - bellek yığınlarından, 166
 - alıştırma
 - bir üst üste ikili ağaç ile bir bit vektöründen, 534
 - dinamik tablolarından, 467–471
 - doğrudan-adresli tablolarından, 254
 - Fibonacci bellek yığınlarından, 522, 526
 - problem
 - ikili arama ağaçlarından, 295–298, 299
 - alıştırma
 - kırmızı-siyah ağaçlarından, 323–330
 - kuyruklardan, 234
 - ön van Emde Boas yapılarından, 544
 - sabit yüksekliğin bir üst üste ağacı ile bir bit vektöründen, 535
 - sıra-istatistiği ağaçlarından, 343–344
 - tarama-doğrusu durumlarından, 1024
 - van Emde Boas ağaçlarından, 554–556
 - yığınlarından, 232
 - zincirli çarpı tablolarından, 258
- simetrik bağıntı, 1163
- simetrik fark, 763
- simetrik matris, 1220, 1222
- alıştırma, 1226
- simetrik pozitif-tanımlı matris, 832–835, 842
- simpleks algoritması, 848, 864–879, 896–897
- simpleks, 848
- SIMPLEX, 871
- siyah tepe, 594, 603
- siyah-yükseklik, 309
- skaler akış çarpımı, 714
- alıştırma

- skaler çarpım, 1220
- SLOW-ALL-PAIRS-SHORTEST-PATHS, 689
- sol altağaç, 1178
- sol çocuk, 1178
- sol dönme, 312
- sol kol, 333 problem
- sol-çocuk, sağ-kardeş gösterimi, 246, 249 alış-
tırma
- somut problem, 1055
- son dizilim, 779
- son sıra ağaç yürüyüşü, 287
- son görev, 444
- son-durum fonksiyonu, 996
- son-içeri, ilk-dışarı, 232
 - ayrıca bakınız* önbellek,
- sonek ($\square$), 986
- sonek fonksiyonu eşitsizliği, 999
- sonek fonksiyonu özyineleme lemması, 1000
- sonek fonksiyonu, 996
- sonlu dizi, 1166
- sonlu grup, 940
- sonlu küme, 1161
- sonlu otomata, 995
 - katar eşlemesi için, 996–1002
- sonsuz dizi, 1166
- sonsuz küme, 1161
- sonsuz toplam, 1145
- sonsuz, ile aritmetik, 650
- sonsuzluklar içeren aritmetik, 650
- sorgu, 230
- soyut problem, 1054
- sözde asal, 966–968
- sözde rassal-sayı üretici 117
- sözde ters, 837
- sözlük sıralaması, 304 problem
- sözlük, 229
- sözlüksel olarak daha az, 304 problem
- spawn**, kaba kodda, 776–777
- spline, 840 problem
- SQUARE-MATRIX-MULTIPLY-RECURSIVE, 77
- SQUARE-MATRIX-MULTIPLY, 75, 689
- STACK-EMPTY, 233
- standart biçim, 846, 850–854
- standart sapma, 1200
- standart şifreleme ($\langle \rangle$), 1057
- Stirling yaklaşımı, 57
- Strassen algoritması, 79–83, 111–112
 - çoklu iş parçacıklı, 795–796
- STRONGLY-CONNECTED-COMPONENTS, 617
- SUBSET-SUM, 1097
- SUCCESSOR, 230
- SUM-ARRAYS, 805 problem
- SUM-ARRAYS', 805 problem
- süre yasası, 780
- süre, çoklu iş parçacıklı hesaplamanın, 779
- süper bilgisayar, 772
- süper çıkış düğümü, 712
- süper kaynak, 712
- süper polinom, 1048
- süre, *bakınız* çalışma süresi,
- sürekli düzgün olasılık dağılımı, 1192
- sütun rankı, 1223
- sütun sıralama, 208 problem
- sütun vektörü, 1218
- sütun-öncelikli sıra, 208 problem
- SVD, 842
- sync*, kaba kodda, 776–777
- şablon hesaplama, 809 problem
- şablon, 809 problem
- şifreleme sistemi, 958–965, 983
- şifreli metin, 960
- tabaka oyma, 409 problem, 413
- taban ağacı, 304 problem
- taban sıralaması, 197–200
 - hızlı sırlama ile kıyaslanan, 199
- taban-a sözde asal, 967
- taban, DNA da, 391
- TABLE-DELETE, 468
- TABLE-INSERT, 464
- tablo şeklinde 674
- TAIL-RECURSIVE-QUICKSORT, 188 problem
- tam bağıntı, 1165
- tam çizge, 1172
- tam düğüm, 489
- tam ikili ağaç, 1178, 1180 alıştırma
 - en iyi kodla ilgili, 430
- tam k -lı ağaç, 1179
 - ayrıca bakınız* bellek yığını
- tam ön sıralama, 1165
- tam parantezli matris-zinciri çarpımı, 370
- tam polinom-zamanlı yaklaşım şeması, 1107
 - altküme toplamı için, 1128–1134, 1139
- tam rank, 1223
- tam sıralama, 1165
- tamamlama adımı 782
- tamamlama süresi, 447 problem, 1136 problem

- tamamlanmamış adım, 782
- tam sayı-değerli akış, 733
- tam sayılar ($\mathbb{Z}$), 1158
- tam sayıların tek çarpanlarına ayrılması, 931
- tam sayılı doğrusal programlama, 850, 895 problem, 1101 alıştıırma
- tam sayılı veri türü, 23
- tanık, bir sayının bileşikliğine, 968
- tanım kümesi, 1166
- tarama-doğrusu durumları, 1023–1024
- tarama-doğrusu, 1022
- tarama, 1021–1029, 1045 problem dönel, 1030–1038
- tarama, 807 problem
- Tarja'nın çevrim dışı en küçük ortak atalar algoritması, 584 problem
- taşan tepe noktası, 736
 - nın boşalması, 751
- taşma
 - bir kuyruğun, 235
 - bir yığının, 233
- tavan fonksiyonu ($\lceil \cdot \rceil$), 54
 - ana teoremdede, 103–106
- tavan komutu, 23
- Taylor serileri, 306 problem
- tek başına bağlantılı çizgeler, 612 alıştıırma
- tek başına bağlı listeler, 236
 - ayrıca bakınız bağlı liste,
- tek çocuk, 1161
- tek-geçiş yöntemi, 585
- tek-hedefli en kısa yollar, 644
- tek-ikili en kısa yol, 381, 644
 - bir doğrusal program olarak, 859–860
- tek-kaynaklı en kısa yollar için Gabow'un ölçeklendirme algoritması, 679 problem
- tek-kaynaklı en kısa yollar, 643–683
 - bir (ϵ)-yoğun çizgelerde, 706 problem
 - bir doğrusal program olarak, 863 alıştıırma
 - bir yönlü çevrimsiz çizgede, 655–658
 - çiftdüze yollar ile, 682 problem
 - için Bellman-Ford algoritması, 651–655
 - için Dijkstra algoritması, 658–664
 - için Gabow ölçekleme algoritması 679 problem
 - ve en uzun yollar, 1048
 - ve fark kısıtları, 664–670
- tekil değer ayrıştırması, 842
- tekil matris, 1223
- tekil olmayan matris, 1223
- tekrarlama çarpanı, bir katarın, 1012 problem
- tekrarlı kareleme,
 - tüm-ikililerin en kısa yolları için, 689–691
 - bir sayının kuvvetini almak için, 956
- temel araya sokma, 465
- temel çözüm, 866
- temel değişken, 855
- temel durum, 1189
- temel durum, 65, 84
- temel olmayan değişken, 855
- temel elverişli çözüm, 866
- temiz alan, 208 problem
- tepe kümesi, 1168
- tepe noktası girilmesi, 1169
- tepe noktası,
 - nın kapasitesi, 714 alıştıırma
 - nın özellikleri, 592
 - bir çizgede, 1168
 - bir çokgenin, 1020 alıştıırma
 - eklem noktası, 621 problem
 - izole edilmiş, 1169
 - orta, 693
 - seçici, 1093
 - taşan, 736
 - yeniden etiketlenmiş, 740
- tepe örtüsü problemi
 - nin yaklaşım algoritması, 1108–1111, 1139
 - nin NP-tamlılığı, 1089–1091, 1105
- tepe örtüsü, 1089, 1108, 1124–1127, 1139
- ters simetrik bağıntı, 1164
- ters
 - bir bijektif fonksiyonun, 1167
 - bir grupta, 940
 - bir matrisin, 827–831, 842, 1223, 1225 alıştıırma
 - çarpımsal, modülo n ye göre, 949
- ters
 - bir dizide, 41 problem, 122 alıştıırma, 345 alıştıırma
 - bir kendi kendini düzenleyen listede, 476 problem
- tersinir matris, 1223
- tersinir olmayan matris, 1223
- tersleyici, 1070
- test edilmesi
 - asallığın, 965–975, 983
 - sözde asallığın, 966–968
- then cümlecığı, 20 dipnot
- Theta-notasyonu, 44–47, 64
- to, sözde kodda, 20
- tokalaşma lemması, 1172 alıştıırma
- TOP, 1031

- toparlama akışı, 863
- toparlama analizi, 452-456
 - ayrık-küme veri yapıları için, 566-567, 568
 - alıştırma
 - bir yönlü çevrimsiz çizgedeki en kısa yollar için, 655
 - çubuk-kesme için, 367
 - derinliğine arama için, 606
 - Dijkstra algoritması için, 661
 - dinamik tablolar için, 465
 - enine arama için, 597
 - Fibonacci bellek yığınları için, 518, 522
 - alıştırma
 - Graham taraması için, 1036
 - ikili sayaçlar için, 454-455
 - Knuth-Morris-Pratt algoritması için, 1006
 - Prim algoritması için, 636
 - yığın işlemleri için, 452-454
- toplam ($\sum$), -1, 1145
 - in kuralı, 1183
 - iç içe geçen, 1148
 - Kartezyen, 906 alıştırma
 - matrislerin, 1220
 - polinomların, 898
 - sonsuz, 1145
- toplam lemması, 908
- toplam yol uzunluğu, 304 problem
- toplam, 1145-1157
 - in doğrusallığı, 1146
 - in formülleri ve özellikleri, 1145-1149
 - asimtotik notasyonda, 49-50, 1146
 - sınırlama, 1149-1156
- toplam,
 - ikili tam sayıların, 22 alıştırma
 - matrislerin, 1220
 - modülo n ($+n$) ye göre, 940
 - polinomların, 898
- toplama komutu, 23
- toplama kuralı, 1183
 - beklenen, 28, 117
 - bir çizge algoritmasının, 588
 - büyüme derecesi, 28
 - büyüme oranı, 28
 - çalışma süresi, 25
 - en iyi-durum, 29 alıştırma, 49
 - en kötü-durum, 27, 49
 - ortalama-durum, 28, 116
 - ve çoklu iş parçacıklı hesaplama, 779-780
- toplamları ayırma, 1152-1154
- toplamların doğrusallığı, 1146
- toplar ve kutular, 133-134, 1215 problem
- Toplitz matrisi, 921 problem
- TOPOLOGICAL-SORT, 613
- topolojik sıralama, 8, 612-615, 623
 - bir yönlü çevrimsiz çizgede tek-kaynaklı en kısa yolları hesaplamada, 655
- topuk, 602 alıştırma
- torun, 1176
- totoloji, 1066 alıştırma, 1086 alıştırma
- TRANSITIVE-CLOSURE, 698
- TRANSPLANT, 296, 323
- transpoze
 - bir matrisin, 1217
 - bir matrisin, çoklu iş parçacıklı, 792 alıştırma
 - bir yönlü çizgenin, 592 alıştırma
 - eşlenik, 832 alıştırma
- TREAP-INSERT, 333 problem
- treap, 333 problem, 338
- TREE-DELETE, 298, 299 alıştırma, 323-324
- TREE-INSERT, 294, 315
- TREE-MAXIMUM, 291
- TREE-MINIMUM, 291
- TREE-PREDECESSOR, 292
- TREE-SEARCH, 290
- TREE-SUCCESSOR, 292
- tridiagonal matris, 1219, 1222 alıştırma, 1225
 - alıştırma
- tridiagonal doğrusal sistemler, 840 problem
- tridiagonal matris, 1219
- trie (taban ağacı), 304 problem
 - y -hızlı, 558 problem
- TRIM, 1130
- TSP (Gezgin-Satıcı Problemi), 1096
- tuple, 1162
- tur
 - bir çizgenin, 1096
 - çift düze, 405 problem
 - Euler, 623 problem, 1048
- tutarlılık
 - in literalleri, 1088
 - sıralı, 779, 812
- tüm-ikililerin en kısa yolları, 644, 684-707
 - ϵ -yoğun çizgelerde, 706 problem
 - dinamik çizgelerde, 707
 - için Floyd-Warshall algoritması, 693-697, 706
 - için Johnson algoritması, 700-706
 - matris çarpımından, 686-693, 706-707
 - tekrarlı kareleme tarafından, 689-691
- tümleyen gevşeklik, 894 problem

- tümleyen
 - bir çizgenin, 1090
 - bir dilin, 1058
 - bir kümenin, 1160
 - bir olayın, 1190
 - Schur, 820, 834
- uç noktası,
 - aralığın, 348
 - bir doğru parçasının, 1015
- UNION, 505, 562
 - nun ayrık- küme- orman uygulaması, 571
 - nun bağlı-liste uygulaması, 565–567, 568
 - ex/
- until**, kaba kodda, 20
- uygun ata, 1176
- uygun etkinlikler, 415
- uygun matrisler, 371, 1221
- uygun paralel bilgisayar, 779
- uygun torun, 1176
- uzak veri, uydu veri, 147, 229
- uzaklık
 - n en kısa yolu, 597
 - L_m , 1044 alıştırma
 - düzenleme, 406 problem
 - Manhattan, 225 problem, 1044 alıştırma
 - Öklid, 1039
- uzunluk
 - bir algoritmanın girdisinin, 25, 926–927, 1055–1057
 - bir binom ağacının, 527 problem
 - bir boole birleşimsel devrenin, 1072
 - bir Fibonacci bellek yığındaki bir altağacın, 524
 - bir kliğin, 1086
 - bir kümenin, 1161
 - bir tepe örtüsünün, 1089, 1108
- uzunluk
 - bir dizinin, 1166
 - bir katarın, 986, 1184
 - bir kolun, 333 problem
 - bir yolun, 1170
- 3-CNF gerçekleştirilebilirliği, 1082–1085, 1105
 - için yaklaşım algoritması, 1123–1124, 1139
 - ve 2-CNF gerçekleştirilebilirliği, 1049
- 3-CNF-SAT, 1082
- 3-CNF, 1082
- 3-COLOR, 1103 problem
- 3-medyan yöntemi, 188 problem
- 3-ve-li normal biçim, 1082
- üçe ayırma aralığı, 348
- üçgen eşitsizliği, 1112
 - en kısa yollar için, 650, 671
- üreteç fonksiyonu, 108 problem
- üreteç
 - bir altgrubun, 944
 - $\mathbb{Z}_n^*$ nin, 955
- üretim süresi, 1136 problem
- ürün, 862
- üst karekök ($\sqrt[n]{}$), 546
- üst medyan, 213
- üst sınır, 47
- üst-sınır özelliği, 650, 671–672
- üst-üçgensel matris, 1219, 1225 alıştırma
- üstel arama ağacı, 212, 483
- üstel fonksiyon 55–56
- üstel komut, 24
- üstel, modüler 956
- üstel seriler, 1147
- üstel yükseklik, 300
- üstte olma bağıntısı ($\succ_x$), 1022
- üyelik,
 - ön van Emde Boas yapılarında, 540–541
 - van Emde Boas ağaçlarında, 550
- van Emde Boas ağacı, 531–560
 - daki öncül, 551–552
 - nda araya sokma, 552–554
 - nda küme, 546
 - ndaki ardıl, 550–551
 - ndaki maksimum, 550
 - ndaki minimum, 550
 - ndaki özet, 546
 - ndaki üyelik, 550
 - ndan silme, 554–556
 - indirgenmiş uzayla, 557 problem
 - ön van Emde Boas yapıları ile karşılaştırılan, 547
- Vandermonde matrisi, 902, 1226 problem
- Var [] (varyans), 1199
- varyansı, 1199
 - binom dağılımının, 1205
 - geometrik dağılımın, 1203
- ve-li normal form, 1049, 1082
- vEB ağacı, *bakınız* van Emde Boas ağacı,
- vEB-AĞAÇ-SİLME 554

- vEB-EMPTY-TREE-INSERT, 553
- vEB-TREE-INSERT, 553
- vEB-TREE-MAXIMUM, 550
- vEB-TREE-MEMBER, 550
- vEB-TREE-MINIMUM, 550
- vEB-TREE-PREDECESSOR, 552
- vEB-TREE-SUCCESSOR, 551
- vektör, 1218, 1222–1224
 - ün evrişimi, 901
 - ün vektörel çarpımı, 1016
 - düzlemde, 1015
 - ortonormal, 842
- vektörel çarpım ($\times$), 1016
- Venn diyagramı, 1160
- veri türü, 23
- veri yapılarını genişletme, 339–355
- veri yapısı, 9, 229–355, 481–585
 - nın genişletilmesi, 339–355
 - nın potansiyeli, 459
 - 2-3 ağaçları, 337, 504
 - 2-3-4 ağaçları, 489, 503 problem
 - 2-3-4 bellek yığınları, 529 problem
 - AA-ağaçları, 338
 - ağırlık-dengeli ağaçlar, 338
 - aralık ağaçları, 348–354
 - atlama listeleri, 338
 - AVL ağaçları, 333 problem, 337
 - ayrık kümeler için, 561–585
 - B-ağaçlar, 484–504
 - bağlı listeler, 236–241
 - bellek yığınları, 151–169
 - binom bellek yığınları, 527 problem
 - birleştirilebilir bellek yığınları, 505
 - bit vektörleri, 255 alıştırma, 532–536
 - çırpı tabloları, 256–261
 - çift taraflı kuyruklar, 236 alıştırma
 - dinamik ağaçlar, 482
 - dinamik çizgeler için, 483
 - dinamik kümeler, 229–231
 - doğrudan-adresli tablolar, 254–255
 - Fibonacci bellek yığınları, 505–530
 - füzyon ağaçları, 212, 483
 - gevşetilmiş bellek yığınları, 530
 - ikili arama ağaçları, 286–307
 - ikincil depolama üzerinde, 484–487
 - k -komşu ağaçları, 338
 - kalıcı, 331 problem, 482
 - kendini dengeleyen ağaçlar, 338
 - kırmızı-siyah ağaçlar, 308–338
 - köklü ağaçlar, 246–249
 - kuyruklar, 232, 234–235
 - ön van Emde Boas yapıları, 538–545
 - öncelikli kuyruklar, 162–166
 - sıra-istatistiği ağaçları, 339–345
 - sözlükler, 229
 - taban ağaçları, 304 problem
 - treaps, 333 problem, 338
 - üstel arama ağaçları, 212, 483
 - van Emde Boas ağaçları, 531–560
 - yayvan ağaçlar, 338, 482
 - yığınlar, 232–233
- veri-paralel modeli, 811
- veri-taşıma komutları, 23
- VERTEX-COVER, 1090
- veya-lı normal biçim 1083
- Viterbi algoritması, 408 problem
- VORP, 411 problem
- vuruş
 - önbellek, 449 problem
 - sahte, 991
- while**, kaba kodda, 20
- WITNESS, 969
-
- y -hızlı trie, 558 problem
- yakınsak seriler, 1146
- yakınsaklık özelliği, 650, 672–673
- yaklaşım hatası, 836
- yaklaşım algoritması, 10, 1105–1140
 - 0-1 sırt çantası problemi için, 1137 problem, 1139
 - ağırlıklı kapsayan küme için, 1135 problem
 - alt küme toplamı için, 1128–1134, 1139
 - gezgin-satıcı problemi için, 1111–1117, 1139
 - kutu paketleme için, 1134 problem
 - küme örtüsü için, 1117–1122, 1139
 - maksimum eşleme için, 1135 problem
 - maksimum kapsayan ağaç, 1137 problem
 - maksimum klik için, 1111 alıştırma, 1134 problem
 - maksimum-ağırlıklı kesim için, 1127 alıştırma
 - MAX-3-CNF gerçekleştirilebilirliği, 1123–1124, 1139
 - MAX-CNF gerçekleştirilebilirliği, 1127 alıştırma
 - minimum-ağırlıklı tepe örtüsü için, 1124–1127, 1139
 - paralel makina çizelgeleme, 1136 problem
 - rassal, 1123

- tepe örtüsü için, 1108–1111, 1139
- yaklaşım oranı, 1106, 1123
- yaklaşım şeması, 1107
- yaklaşım
 - en küçük kareler tarafından, 835–839
 - integrallerle toplamının, 1154–1156
- yansımali bağıntı, 1163
- yapay anahtar, 397
- yapılanma, 1074
- yaprak, 1176
- yardımcı çırpı fonksiyonu, 272
- yardımcı doğrusal program, 886
- yarı 3-CNF gerçekleştirilebilirliği, 1101 alıştırma
- yarı bağlantılı çizge, 621 alıştırma
- yarı-açık aralık, 348
- yarıya indirme lemması, 908
- yarış, 787–790
- yatay ışın, 1021 alıştırma
- yay, *bakınız* ayrıt
- yayılma, 478
- yayvan ağaç, 338, 482
- yeniden ağırlıklandırma
 - tek-kaynaklı en kısa yollarda, 679 problem
 - tüm-ikililerin en kısa yollarında, 700–702
- yeniden etiketleme işlemi, itme-yeniden etiketleme algoritmaları, 740, 745
- yeniden etiketlenmiş tepe noktası, 740
- yerel değişken, 21
- yerinde sıralama, 17, 148, 206 problem
- yerine koyma yöntemi, 83–88
 - ve özyineleme ağaçları, 91–92
- yığın, 232–233
 - in bağlı liste uygulaması, 240 alıştırma
 - Graham taramasında, 1030
 - hesaplama yöntemi ile analiz edilmiş işlemler, 457–458
 - ikincil depolama için, 502 problem
 - kuyruklarla uygulanan, 236 alıştırma
 - potansiyel yöntem ile analiz edilmiş işlemler, 460–461
 - prosedür yürütmesi için, 188 problem
 - toparlama analizi ile analiz edilmiş işlemler, 452–454
- *Sokrates, 790
- yıldız-şekilli çokgen, 1038 alıştırma
- yoğun çizge, 589
 - ϵ -yoğun 706 problem
- yoğunluk
 - asal sayıların, 965–966
 - bir çubuğun, 370 alıştırma
- yol bulma, 569
- yol olmama özelliği, 650, 672
- yol örtüsü, 761 problem
- yol sıkıştırma, 569
- yol uzunluğu, bir ağacın, 304 problem, 1180 alıştırma
- yol-gevşetme özelliği, 650, 673
- yol, 1170
 - un ağırlığı, 643
 - basit, 1170
 - bulmak, 569
 - en kısa, *bakınız* en kısa yollar
 - en uzun, 382, 1048
 - genişletilmiş, 719–720, 763 problem
 - hamilton, 1066 alıştırma
 - kritik, 657
- Young tablosu, 167 problem
- yozlaşma, 874
- yönet adımı, böl-ve-yönet algoritmalarında, 30, 65
- yönlü çevrimsiz çizge (dag), 1172
 - nin topolojik sıralanışı, 612–615, 623
 - bir çoklu-iş parçacıklı hesaplamayı göstermek için, 777
 - için tek-kaynaklı en kısa-yollar algoritması, 655–658
 - içinde en uzun basit yol, 404 problem
 - ve arka ayrıtlar, 613
 - ve bileşen çizgeler, 617
 - ve hamilton yolları, 1066 alıştırma
- yönlü çizge, 1168
 - deki en kısa yol, 643
 - deki evrensel çıkış düğümü, 593 alıştırma
 - deki tek-kaynaklı en kısa yollar, 643–683
 - deki tüm ikililerin en kısa yolları, 684–707
 - nin Euler turu, 623 problem, 1048
 - nin geçişli kapanışı, 697
 - nin hamilton çevrimi, 1049
 - nin karesi, 593 alıştırma
 - nin transpozese, 592 alıştırma
 - nin yol örtüsü, 761 problem
 - ayrıca bakınız* yönlü çevrimsiz çizge, çizge, ağ
 - kısıt çizgesi, 666
 - PERT diyagramı, 657, 657 alıştırma
 - tek bağlantılı, 612 alıştırma
 - ve en uzun yollar, 1048
 - yarı bağlantılı, 621 alıştırma
- yönlü çizgenin yönsüz versiyonu, 1172
- yönlü doğru parçası, 1015–1017

yönsüz çizge, 1168
 -de bir minimum kapsayan ağacın hesapla-
 ması, 624-642
 -deki klik, 1086
 -nin bağımsız kümesi, 1101 problem
 -nin eklem noktası, 621 problem
 -nin eşlemesi, 732
 -nin iki bağlantılı bileşeni 621 problem
 -nin köprüsü, 621 problem
 -nin renklendirilmesi, 1103 problem, 1180
 problem
 -nin tepe örtüsü, 1089, 1108
 -ye dönüştürme, bir çoklu çizgeden, 593
 alıştırma
ayrıca bakınız çizge,
d-düzenli, 736 alıştırma
 hamilton olmayan, 1061
 hamilton, 1061
 ızgara, 760 problem
 yönsüz çizgenin yönlü versiyonu, 1172
 yukarıdan aşağıya yöntemi, dinamik programla-
 ma için, 365
 yumuşatılmış analizi, 897
 yuvarlama, 1126
 rassal, 1139
 yük faktörü,
 bir çırpı tablosunun, 258
 bir dinamik tablonun, 463
 yükle komutu, 23
 yüksek fonksiyon, 537, 546
 yükseklik dengeli ağaç, 333 problem
 yükseklik fonksiyonu, itme-yeniden etiketleme
 algoritmalarında, 738

yükseklik
 bir ağacın, 1177
 bir ağaçtaki bir düğümün, 1177
 bir B-ağacın, 489-490
 bir bellek yığınınındaki bir düğümün, 153, 159
 alıştırma
 bir bellek yığınının, 153
 bir binom ağacının, 527 problem
 bir *d*-li bellek yığınının, 167 problem
 bir kırmızı-siyah ağacın, 309
 karar ağacının, 193
 siyah-, 309
 üstel, 300
 $\mathbb{Z}_n^*$ nin bir elemanının indisi, 955
 $\mathbb{Z}_n^*$ nin ilkel kökü, 955
 $\mathbb{Z}_n^*$, (modülo *n* ye göre çarpımsal grubun eleman-
 ları), 967
 $\mathbb{Z}_n$, (modülo *n* ye göre denklik sınıfları), 928
 $\mathbb{Z}$, (tam sayılar kümesi), 1158
 $\mathbb{Z}_n^+$, kümesinin 0 dan farklı elemanları), 967
 zaman alanı, 898
 zaman pulu, 603, 611 alıştırma
 zaman-bellek maliyet dengesi, 365
 zayıf dualite, 880-881, 886 alıştırma, 895
 problem
 zemin fonksiyonu ($\lfloor \rfloor$), 54
 ana teoremden, 103-106
 zemin komutu, 23
 zincirleme, 257-260, 283 problem
 zonk, 1195 alıştırma